Архив статей wasm.ru. Том 9: Введение в крэкинг с нуля, используя OllyDbg

Собрание русскоязычных статей сайта wasm.ru о программировании, ассемблере, реверс-инжиниринге и компьютерной безопасности. Том 9 из 11. Разделы: Введение в крэкинг с нуля, используя OllyDbg. Все приложенные файлы находятся в wasm_files.zip.

Больше крутых материалов в моём телеграм канале @grogrigs

Введение в крэкинг с нуля, используя OllyDbg

Введение в крэкинг с нуля, используя OllyDbg - Глава 1

Автор: Рикардо Нарваха, пер. Aquila

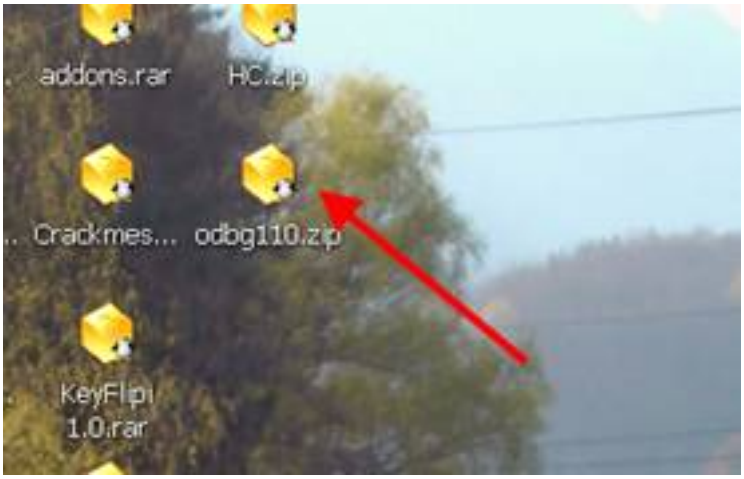
Дата: 2006-09-01

Целью данного «Введения в крэкинг с нуля, используя OllyDbg», является дать тем, кто только начал осваивать искусство крэкнинга, базовые знания и, вместе с тем, сделать это так, чтобы эти знания позволили в дальнейшем читать и понимать более продвинутые туториалы – такие, которые можно встретить в «Новом курсе от CracksLatinos», который, разумеется, остаётся открытым для новых добавлений и пополнений.

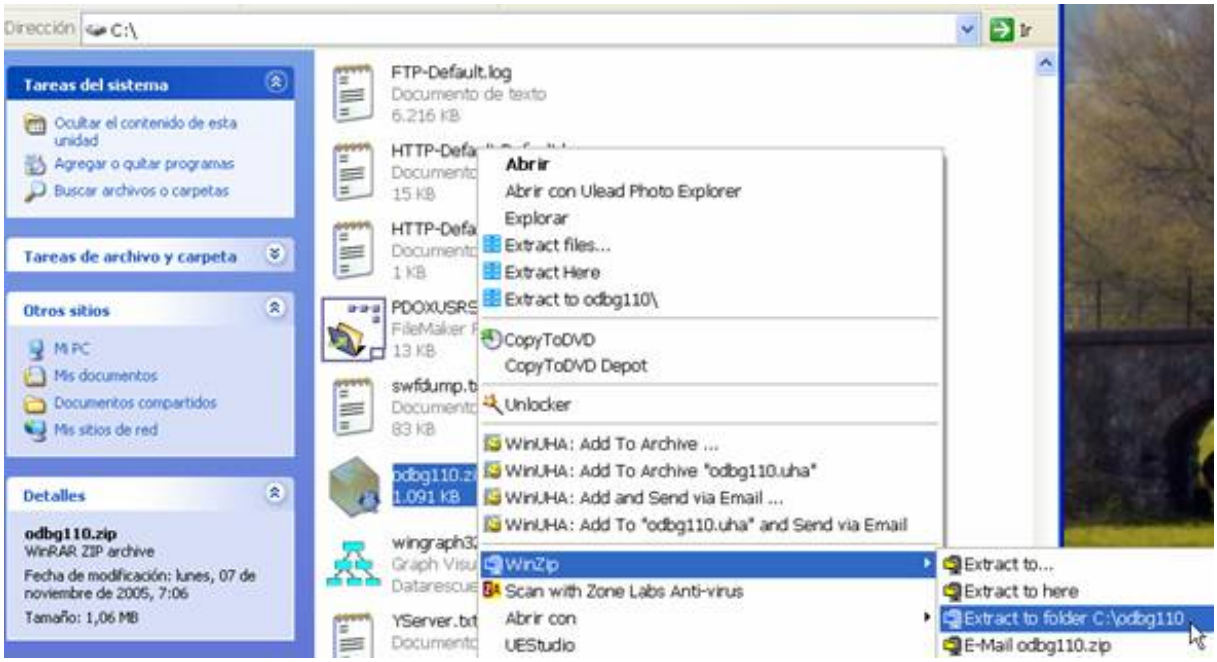
Идея создания курса родилась из-за того, что многие туториалы в «Новом курсе от CracksLations» оказывались слишком сложными для новичков, и те, не сумев достигнуть нужного уровня, оказывались разочарованными и во многих случаях отказывались продолжать. Поэтому целью данного «Введения...» является не повторение прекрасных туториалов из «Нового курса...», число которых уже перевалило за 500, но заложить основу, чтобы те, кто закончит данный курс, смогли читать более сложные пособия. Это, как и всё в нашем ремесле, требует значительных усилий, и главной задачей является сократить их количество, дав базовые знания и позволив в дальнейшем понимать более сложный материал.

Почему OLLYDBG? Мы не будем здесь рассуждать о вечном противостоянии Soft-Ice против OllyDbg, думаю, что даже фанатики от Soft-Ice признают, что проще начать с OllyDbg, так как о нём много информации и его проще изучать. Нам нужно войти в мир крэкинга через дверь под названием «OllyDbg», а уже потом тот, кому нужно, сможет перейти на любой другой отладчик, который потребуется, так как меняются лишь их способы использования, но суть остаётся неизменной. **Начнём с начала**

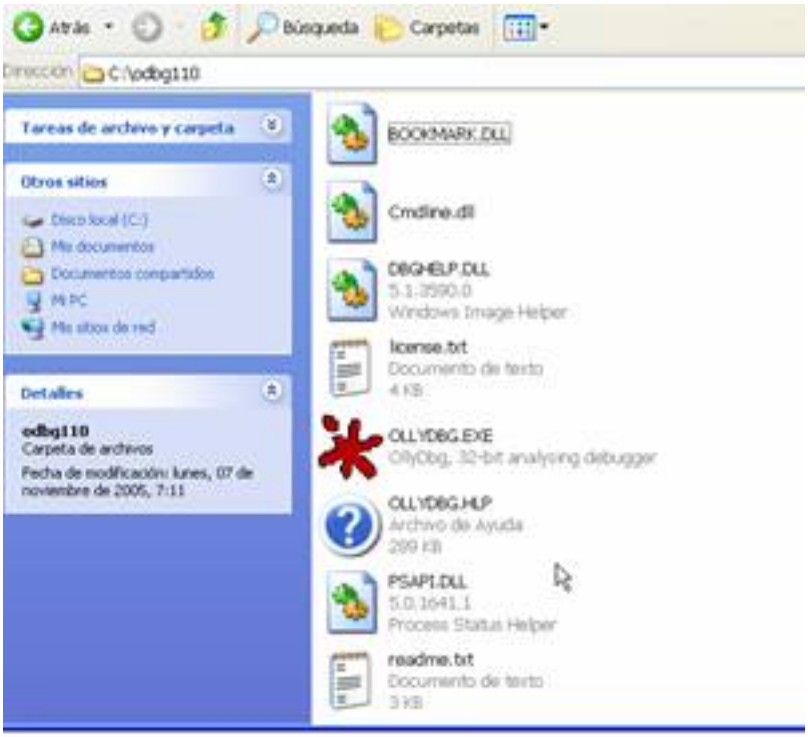
Сначала нужно вооружиться инструментом, который мы собираемся в основном использовать, для чего кликните тут и скачайте его.



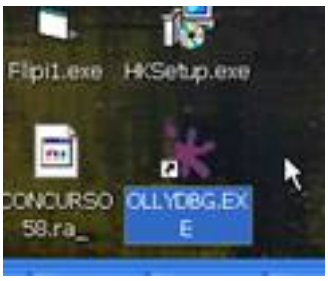
Поскольку мы начинаем с нуля, то для начала нам нужно распаковать скачанный архив в такую папку на жёстком диске, до которой можно легко добраться. Хорошей идеей будет создать папку на диске C:/. Хотя будет работать и в любом другом месте, я буду исходить из того, что выбран диск C:/.



После того, как файл был распакован, заходим в созданную папку и видим:

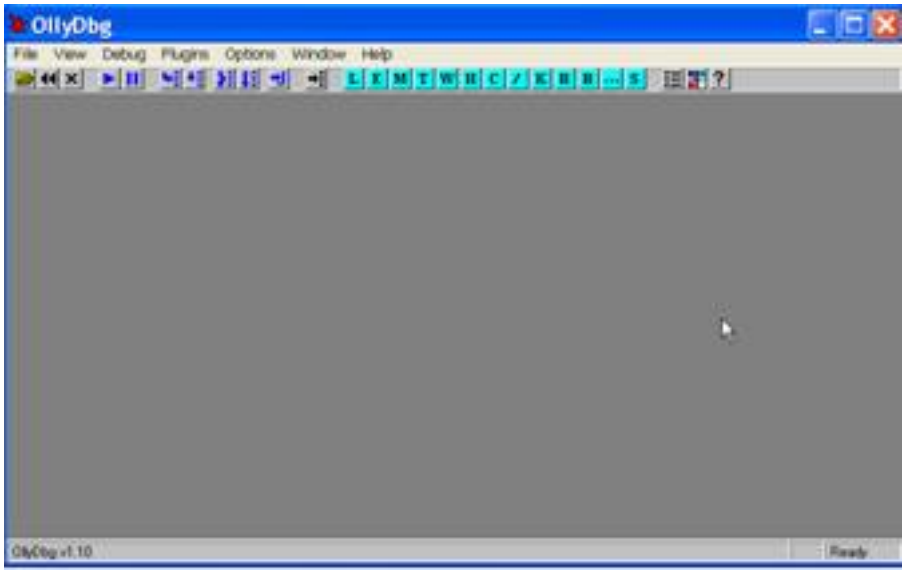


В ней находится исполняемый файл OLLYDBG.exe, который нам и нужно запустить и ярлык которого я для удобства сделал на своём рабочем столе.



Ок, всё готово к запуску. Кликаем на OllyDbg:

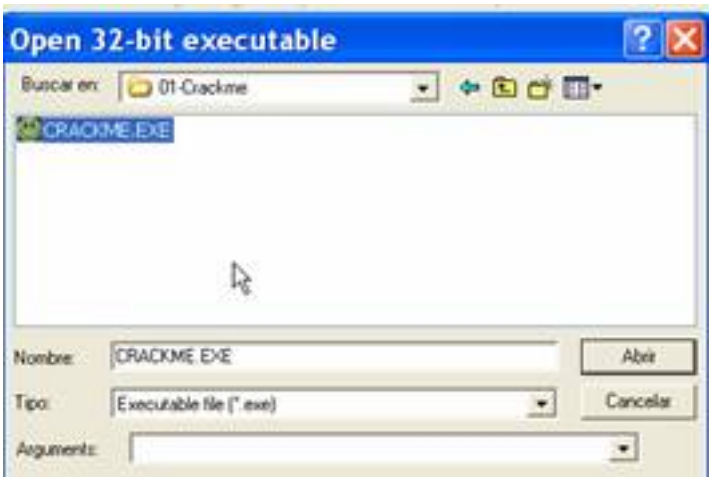
Перед нами повляется сообщение о том, что DLL, находящаяся в библиотеке, старше, чем такая же системная DLL, и если выберем «Да», то тогда старая DLL будет стёрта из папки, а использоваться будет системная. Хотя я не вижу особых различий между этими двумя, тем не менее, предпочитаю ту, которая идёт с дистрибутивом, и поэтому всегда нажимаю «Нет».

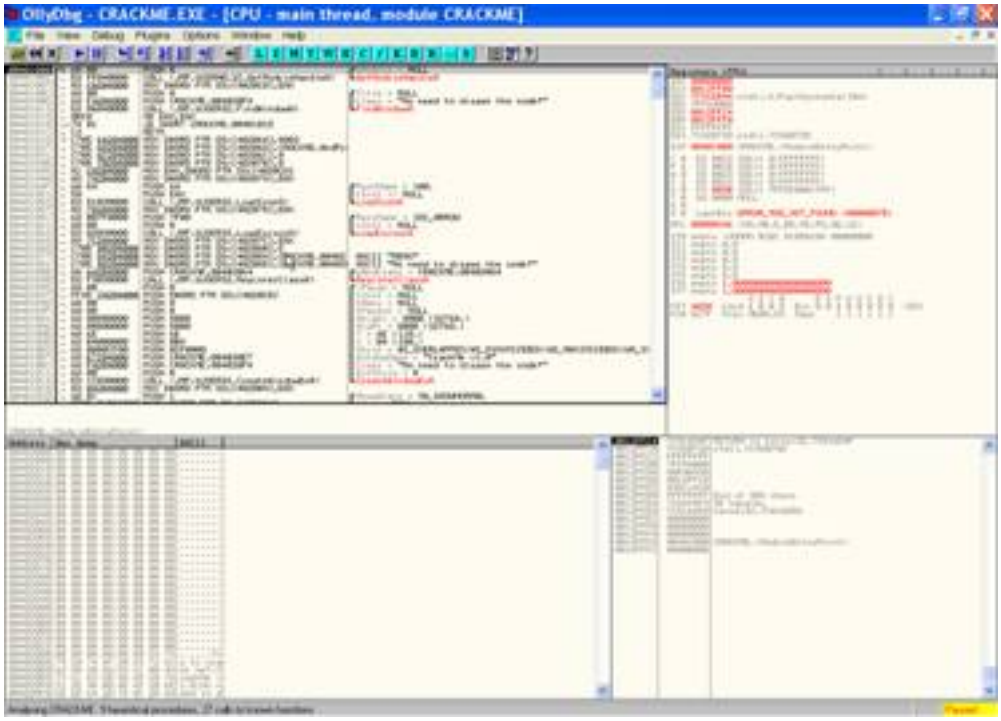


Это чистый OllyDbg, и первой программой, которую мы откроем только для того, чтобы ознакомиться с OllyDbg, станет знаменитый CrackMe (находится в архиве `wasm_files.zip`: `files/0419/ollydbg01-Crackme.zip`) CrueHead'a, который приложен к данному туториалу.

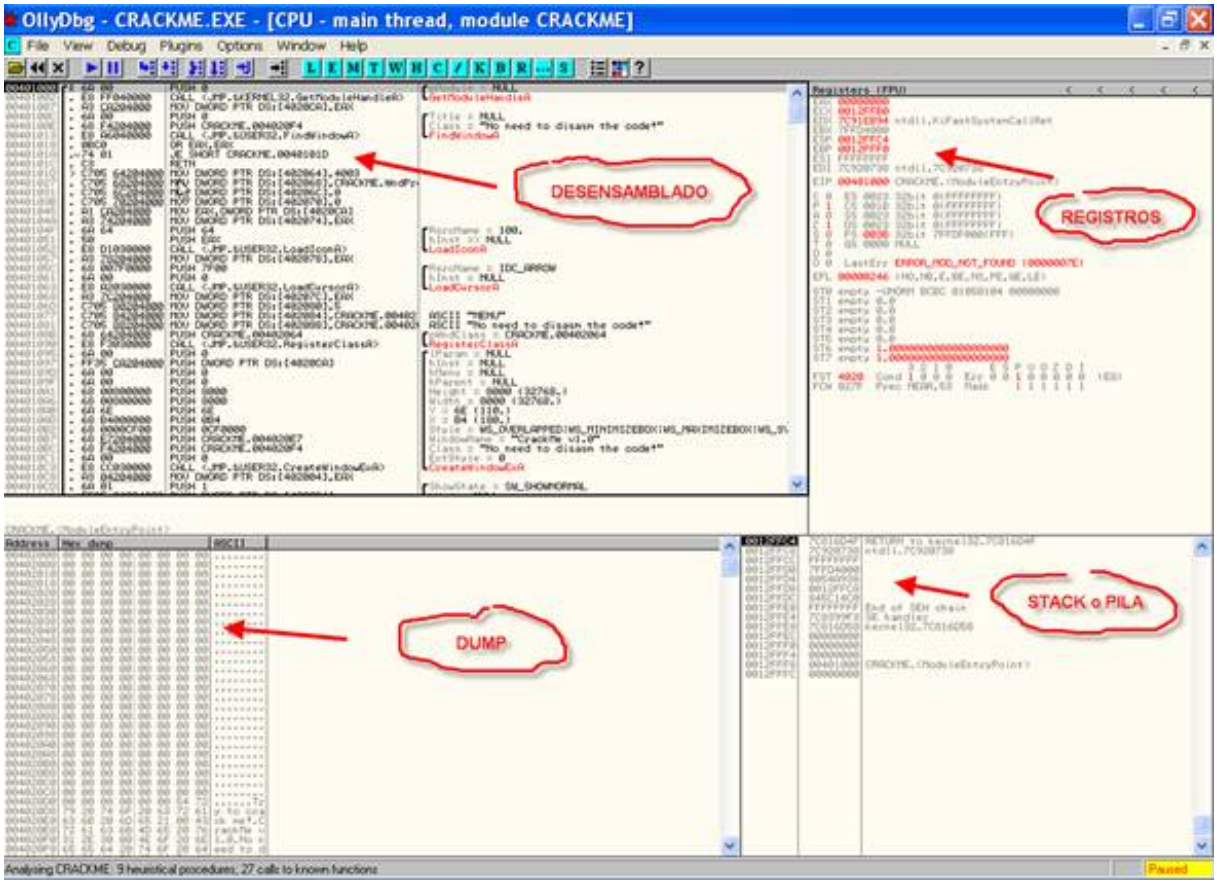
Чтобы открыть файл в OllyDbg, идём в File -> Open или нажимаем на иконку:

Откроется окно, с помощью которого можно найти нужный файл, в данном случае это crackme CrueHead'a.





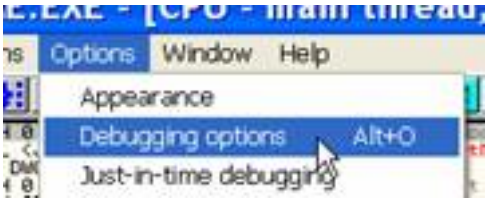
Откроется вышеупомянутый crackme, и на данный момент неважно, что совершенно непонятно, что означает открывшийся нам вид – пока мы только пройдёмся по различным частям и функциям OllyDbg и некоторым настройкам, чтобы когда в последующих туториалах будет написано, скажем, «зайдите в DUMP», вы, по крайней мере, знали, где находится эта опция.

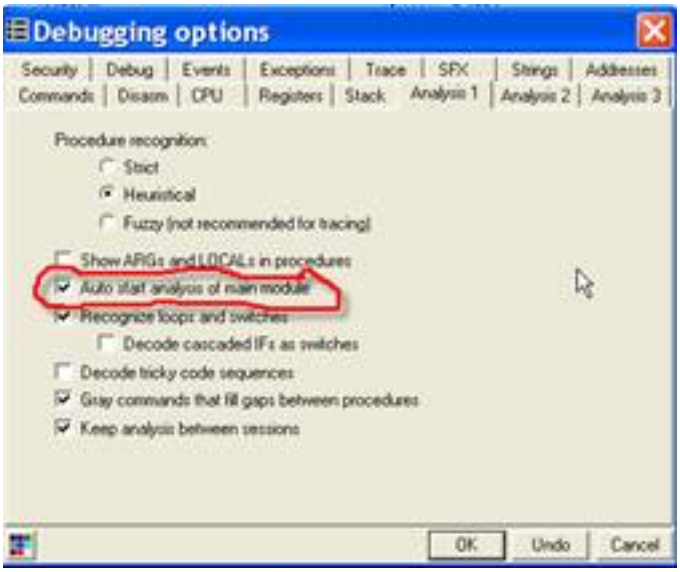


Здесь мы рассмотрим четыре части главного окна OllyDbg:

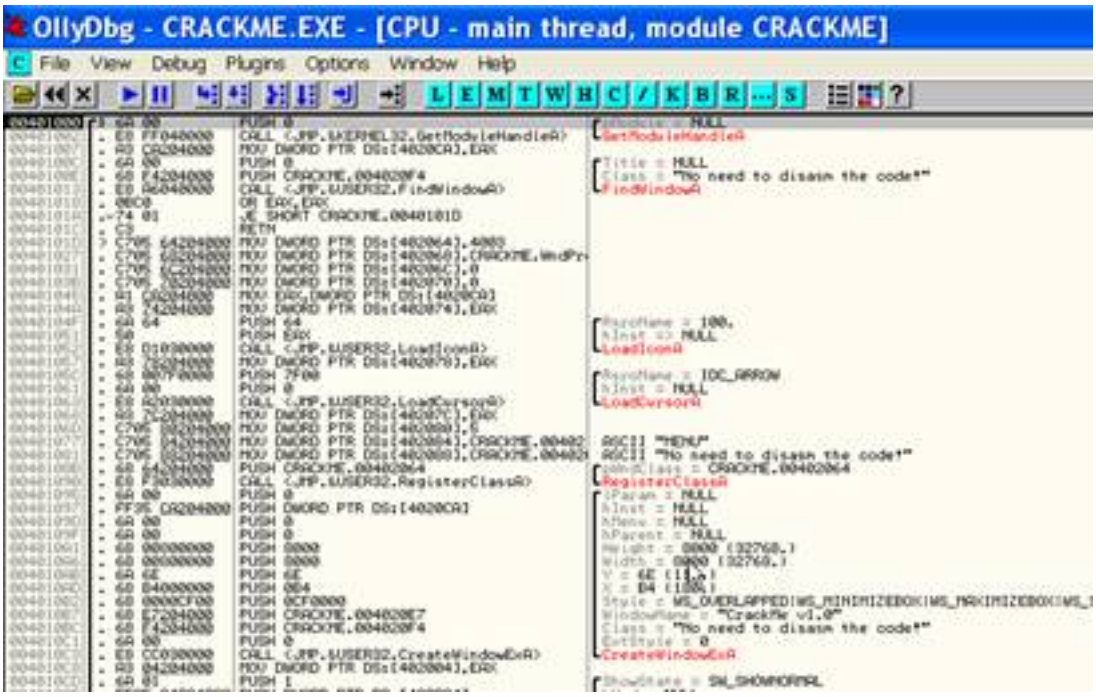
1) Дизассемблированный код

Также называется листингом. Тут Олли показывает нам дизассемблированный код программы, которую мы собираемся отлаживать; по умолчанию Олли сконфигурирована так, чтобы анализировать программу при её открытии. Это можно изменить в Options -> DEBUGGING OPTIONS.





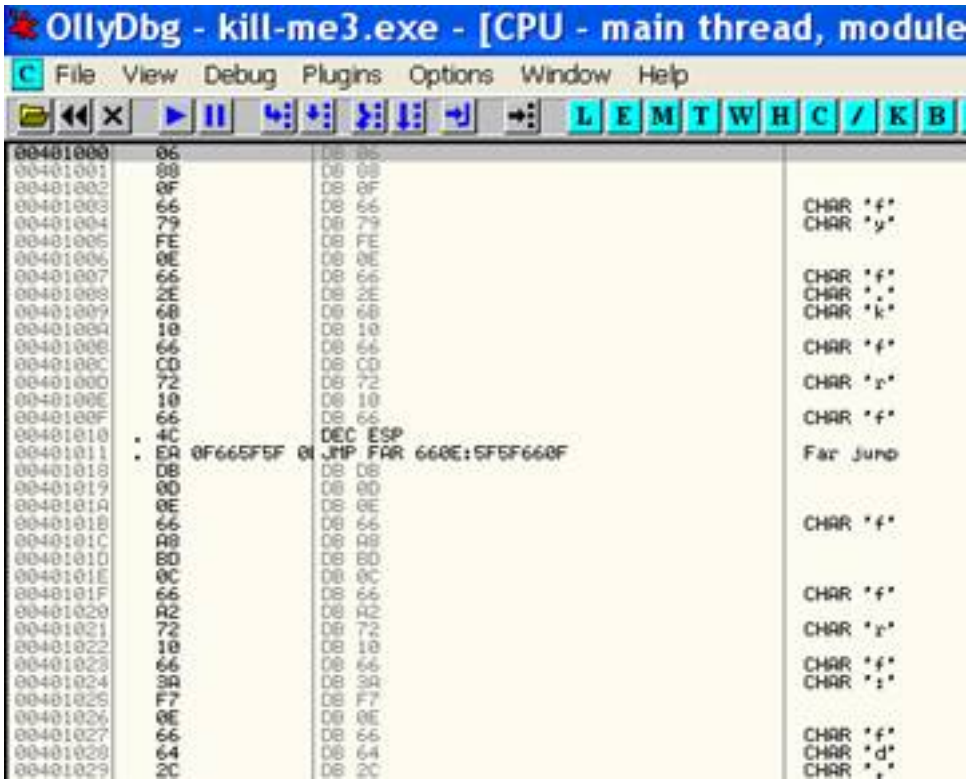
То есть, если отмечена галочка «AUTO START ANALISIS OF MAIN MODULE», OllyDbg проанализирует программу и покажет о ней дополнительную информацию.



Это начало проанализированного листинга крэкми CrueHead'a, и если мы откроем его без анализа, то сможем увидеть разницу.

В окне анализа содержится множество информации, которая, несмотря на то, что пока нам не очень понятна, выглядит весьма интересной. В то же время приятно знать, что её можно в любой момент убрать, если анализ оказался не слишком точным или в него закралась какая-то ошибка.

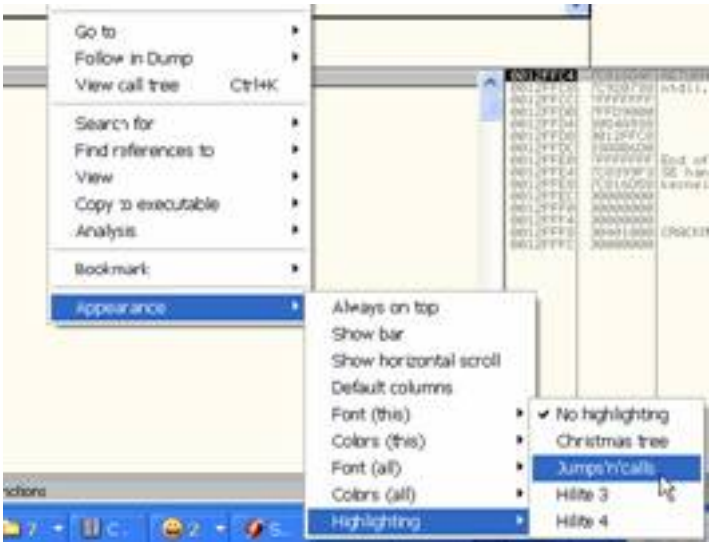
Зачастую OllyDbg отображает некоторые части программы некорректно, так как ошибочно интерпретирует исполняемый код как данные, и тогда она отображает примерно следующее:



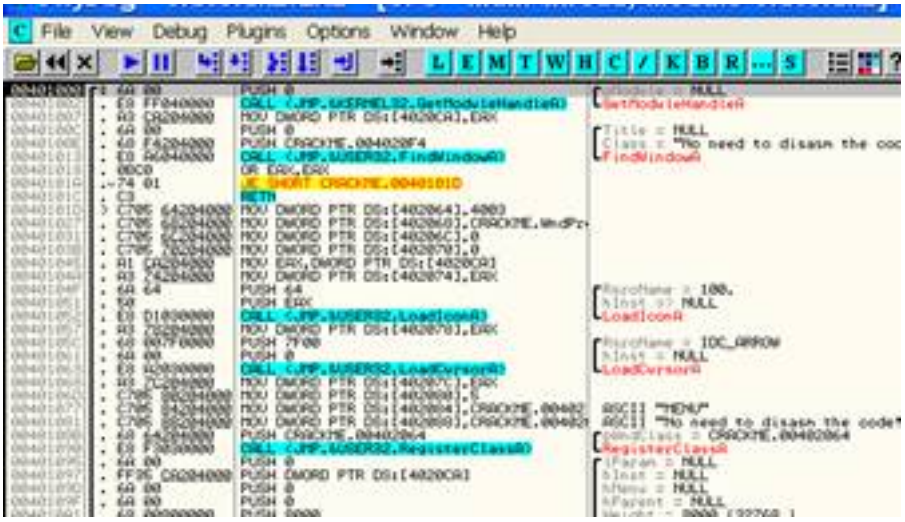
В этом случае можно вручную убрать анализ, кликнув на листинге правой кнопкой мыши и выбрав «ANALISIS -> REMOVE ANALYSIS FROM MODULE».

И тогда листинг отобразится корректно.

Другой опцией, которой вы можете воспользоваться для облегчения раобты и которая мне лично не очень нравится (но вкусы бывают разные), является подсветка jump (переходов) и call (вызовов) – кликните на листинге правой кнопкой мыши и выберите «APPEARENCE -> HIGHLIGHTING -> JUMPS AND CALLS».



Получится следующее:

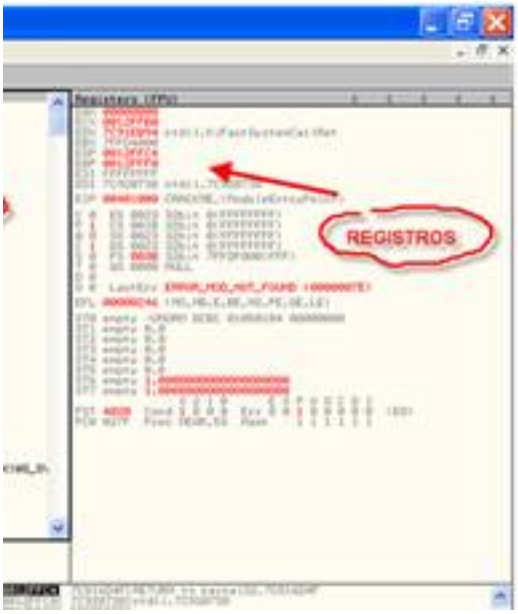


Здесь мы видим, что call'ы подсвечены лазурным цветом, а переходы – жёлтым.

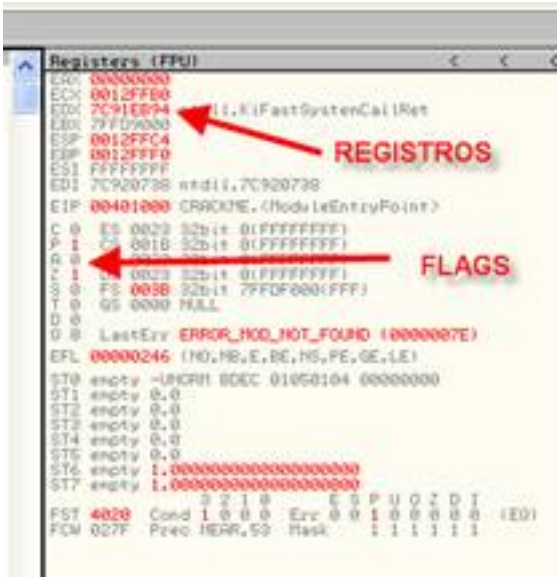
Теперь листинг стал более читаемым, но пока у нас нет ни малейшей идеи, что это означает, но хорошо иметь инструмент подготовленным к дальнейшему использованию.

2) Регистры

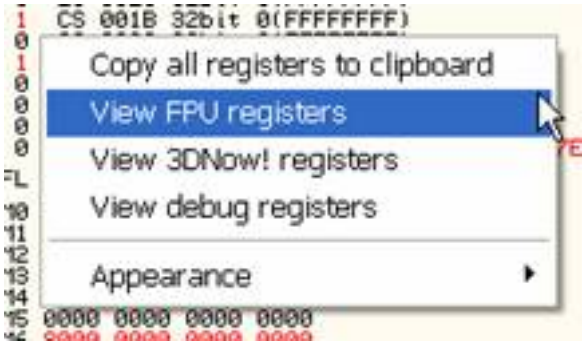
Второе важное окно – это окно регистров.



Вспомним, что окно регистров находится в самой правой части OllyDbg, и там отображается значительное количество информации.

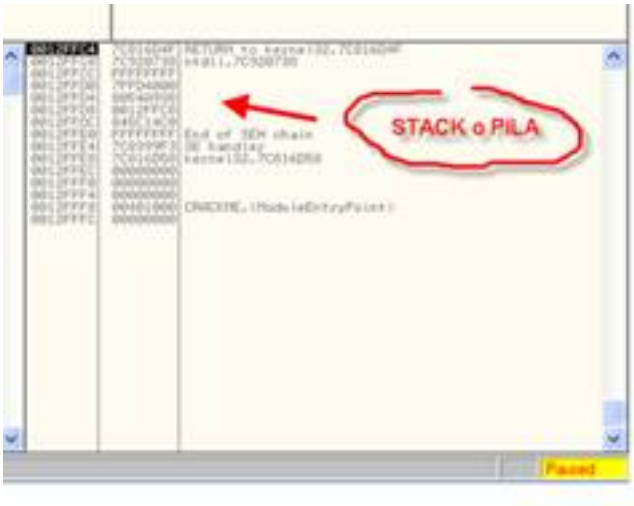


Есть значительно больше информации, которую мы не видим, но можно устанавливать режим отображения в три состояния («VIEW FPU REGISTERS» – отображать FPU-регистры, «VIEW 3D NOW REGISTERS» – отображать “3D NOW”-регистры и «VIEW DEBUG REGISTERS» – отображать отладочные регистры). По умолчанию отображаются первые.



3) Стек или «куча»

Теперь переходим к «стеку или куче». Здесь не слишком много конфигурационных опций – разве что возможность отобразить информацию, касающуюся регистров ESP и EBP.



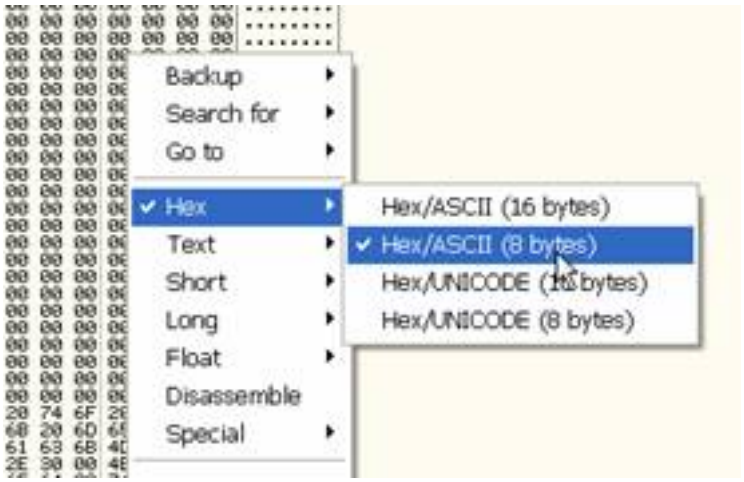
По умолчанию стоить режим отображения информации, связанной с ESP (и он же является самым полезным), но его можно сменить на режим отображения информации, связанной с EBP, для чего необходим кликнуть в этом окне правой кнопкой мыши и выбрать пункт «GO TO EBP», а дальнейшее использование пункта «GO TO ESP» вернёт нас в прежний режим.

В дальнейших главах я объясню подробнее функциональность стека, но пока мы рассматриваем только то, что можно изменить с помощью конфигурации.

4) Дамп

У окно дампа есть много режимов отображения, которые можно менять, кликнув правую кнопку мыши в окне дампа и выбрав тот, который нужен. По умолчанию используется используется режим 8-ми байтовый Hex/ASCII.

Режим, используемый по умолчанию, также является и чаще всего используемым, но вместе с тем, у нас есть возможность изменять его на показ дизассемблированного кода (DISASSEMBLE), текста (TEXT) и других форматов (SHORT, LONG, FLOAT).

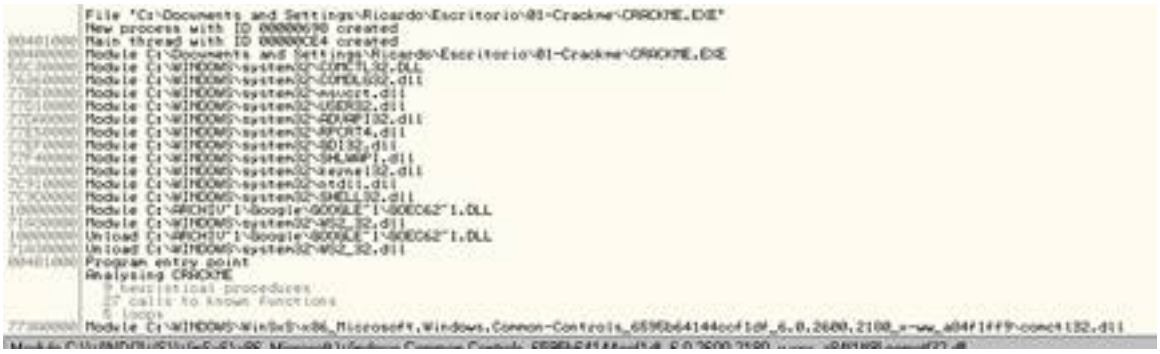


И, накоец, опция SPECIAL -> PE HEADER, которая, как мы это увидим в ближайших главах, может быть очень полезной.

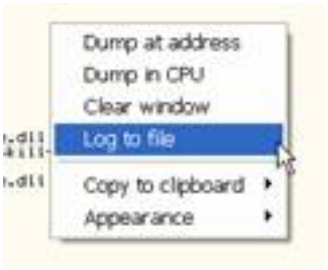
Теперь мы знаем основные части главного окна OllyDbg, но есть ещё окна, которые недоступны напрямую, но могут быть вызваны как через меню, так и через кнопки на панели управления.

Рассмотрим каждую из них.

Кнопка L или VIEW->LOG показывает нам то, что OllyDbg пишет в окне лога. Его можно сконфигурировать на отображение различного рода информации, а по умолчанию в окне лога сохраняется вся информация о запуске, а также информация, связанная с «BREAKPOINTS CONDICIONAL LOGS» (условными логами точек останова). С последней мы встретимся гораздо позднее, а пока давайте посмотрим информацию о запущенном процессе (в нашем случае это крэкми CrueHead'a) и библиотеках, которые он загружает.



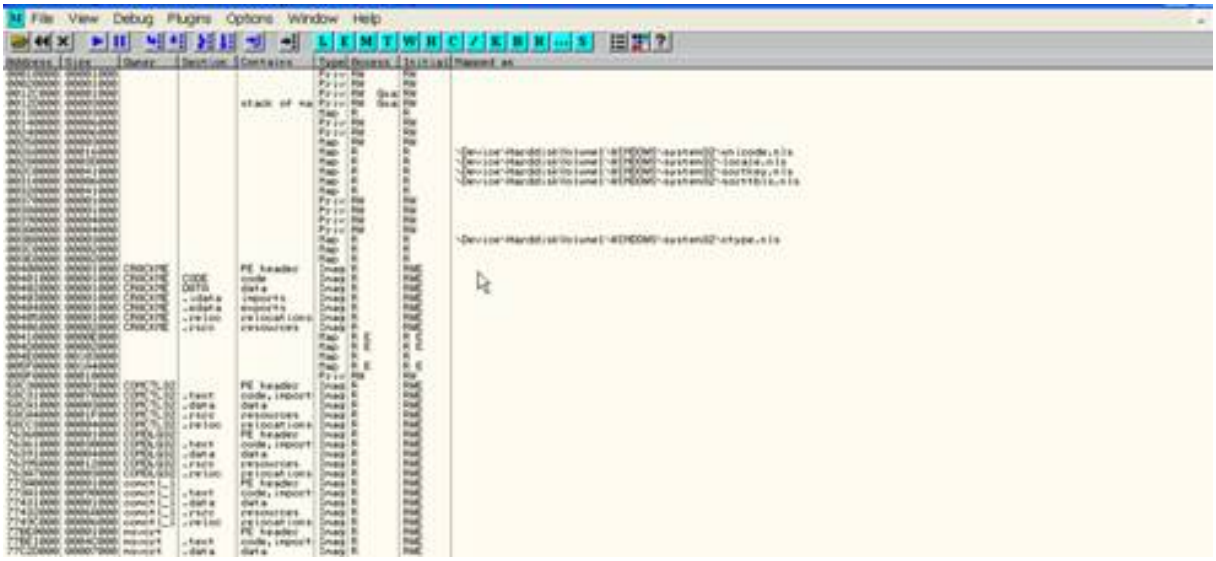
Одна из самых главных опций данного окна – это ведение лога в файле на тот случай, если мы хотим сохранять информацию в текстовом файле. Чтобы активировать эту опцию, нажмите на правую кнопку мыши и выберите «LOG TO FILE».



Кнопка E или VIEW->EXECUTABLES показывает нам список модулей, которые использует программа: exe, dll, ocx и прочие.

Здесь также правая кнопка мыши вызывает множество опций, которые пока мы смотреть не будем, но которые мы уже видели при исследовании главного окна OllyDbg.

Кнопка M или VIEW->MEMORY отображает память, занятую нашей программой. Здесь мы видим секции приложения, библиотеки, использующиеся процессом, стек и различные секции, занятые системой, и зачастую программы в процессе выполнения занимают новые секции памяти.

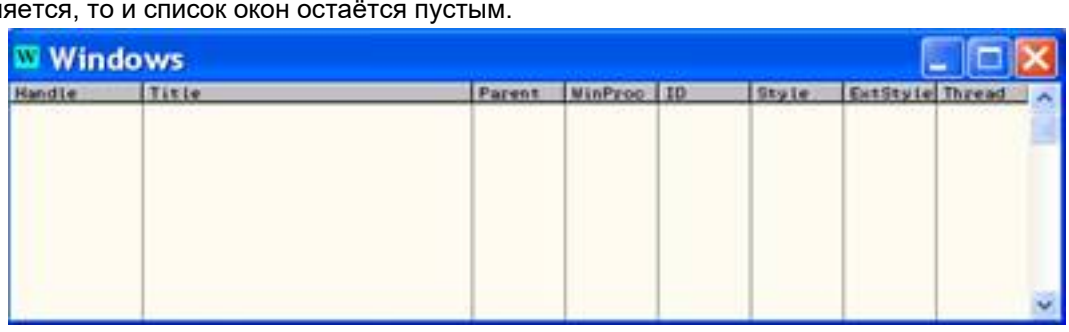


Кликнув правой кнопкой мыши можем сделать SEARCH в памяти, чтобы найти в ней строки различных родов (текстовые, шестнадцатиричные, юникодовые), также есть возможность подстветить точки останова в секциях, равно как и возможность изменить права доступа к последним (выберите SET ACCESS).

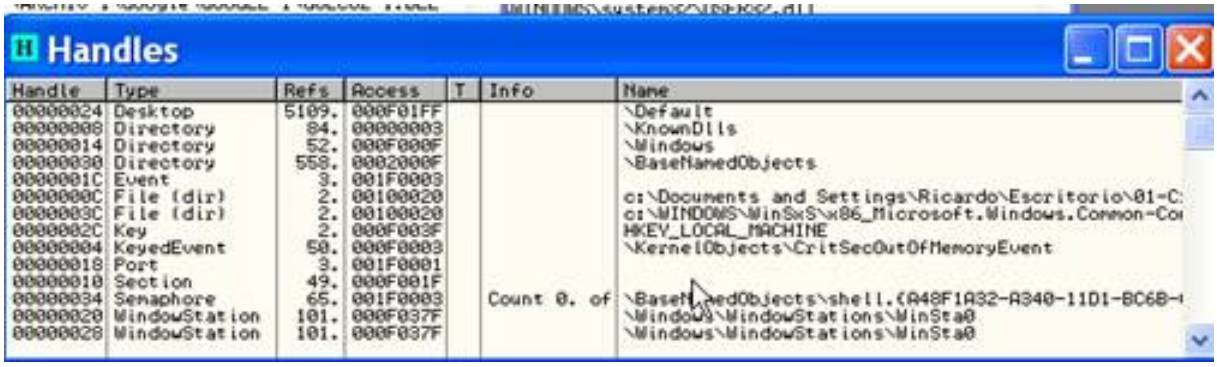
Кнопка T или VIEW->THREADS показывает нам список нитей (потоков) программы.

Хотя сейчас мы не знаем что это такое, а узнаем это лишь в последующих главах, ознакомиться с каждым из окон лишним не будет. Пользоваться ими мы научимся позднее.

Кнопка W или VIEW->WINDOWS отображает нам окна программы, но поскольку она ещё не выполняется, то и список окон остаётся пустым.



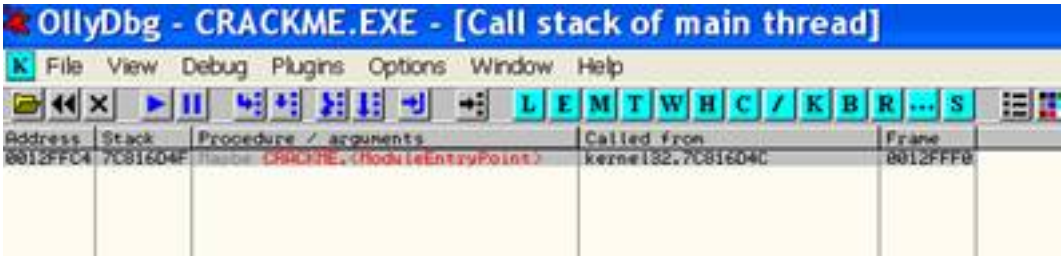
Кнопка H или VIEW->HANDLES отображает хэндлы, позже я объясню, для чего они служат.



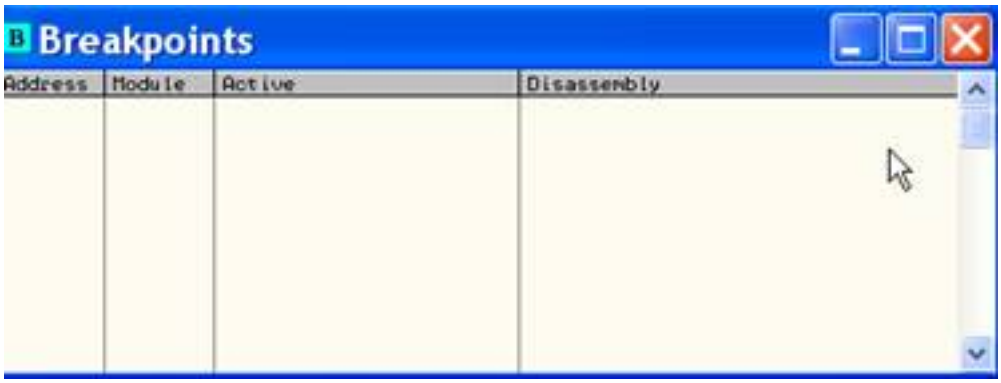
Кнопка C или VIEW->CPU возвращает нас к главному окно OllyDbg.

Кнопка / или VIEW->PATCHES показывает список наложенных патчей, если программа была изменена. Поскольку пока никаких изменений не вносилось, сейчас окно остаётся пустым.

Кнопка K или VIEW->CALL STACK отображает «call stack», списко вызовов, которые нам встретились до того места, где программа остановилась.

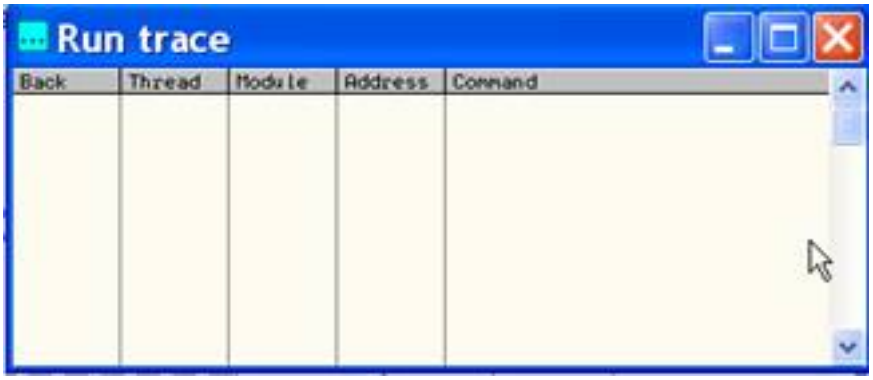


Кнопка B или VIEW->BREAKPOINTS вызывает список обычных точек останова, расположенных в программе. Здесь нет ни точек останова аппаратного обеспечения, ни точек останова памяти, только обычные.



Кнопка R или VIEW->REFERENCES показывает окно ссылок, полученных нами в результате поиска ссылок в Олли.

Кнопка «…» или VIEW->RUN TRACE отображает результат выполнения команды RUN TRACE. Здесь мы также можем выбрать опцию LOG TO FILE), чтобы сохраненить результаты трассировки в текстовом файле.

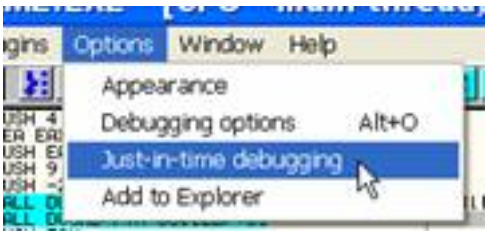


Мы рассмотрели панель с самыми важными кнопками, чтобы вы были знакомы с возможностями, которые они предоставляют, когда мы начнём их более глубокое изучение в последующих главах.

Как сконфигуровать OllyDbg стала JIT (JUST IN TIME DEBUGGER)

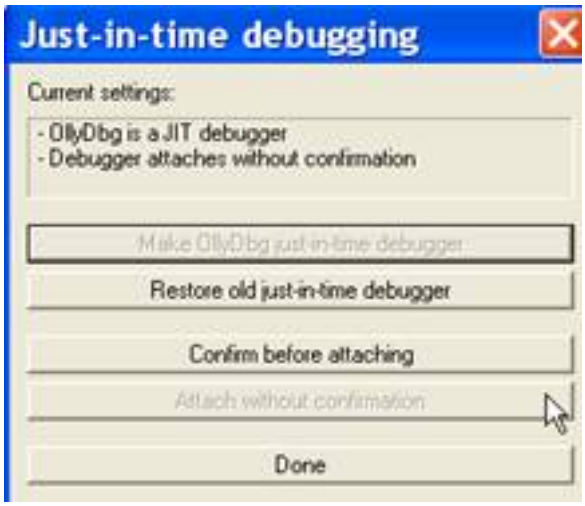
Конечно, мы не будем всё время использовать JIT, а только в специальных случаях, так как если случится ошибка с какой-либо запущенной программой на нашей машине, то нам совсем не нужно, чтобы использовалась Олли (по умолчанию в качестве JIT используется dr.watson).

Чтобы сделать OllyDbg JIT-отладчиком, нужно перейти в OPTIONS->JUST IN TIME DEBUGGING



и нажать последовательно кнопки MAKE OLLYDBG JUST IN TIME DEBUGGER y DONE

Чтобы убрать эту функцию, нужно в том же месте нажать на RESTORE JUST IN TIME DEBUGGER и DONE.

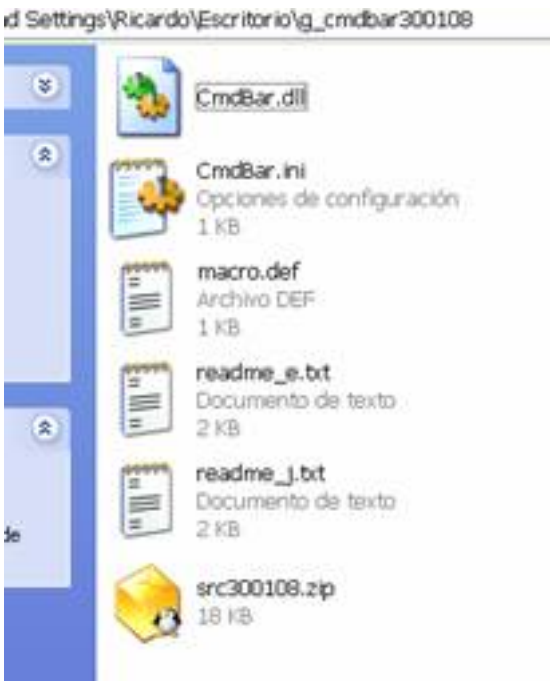


Подключение плагинов в OllyDbg

OllyDbg позволяет подключать плагины, которые могут оказаться полезными для решения той или иной задачи. Пока мы ограничимся подключением плагина COMMAND BAR, чтобы научиться как это делать.

Скачиваем данный плагин отсюда (находится в архиве `wasm_files.zip`: `files/0419/cmdbar310109c.zip`) (ссылка, указанная в оригинальной статье, умерла, поэтому я скачал плагин из инета и разместил её на wasm.ru - *прим.пер.*).

После этого распаковываем плагин и смотрим содержимое папки, где это было сделано:

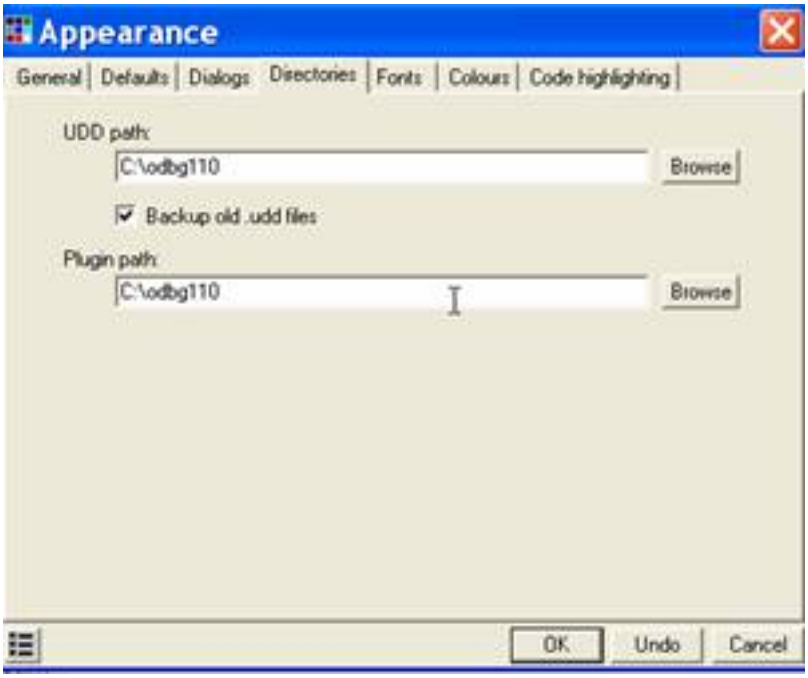


Прежде всего нужно создать папку для плагинов. Я создам её на C:/ и назову PLUGINS.

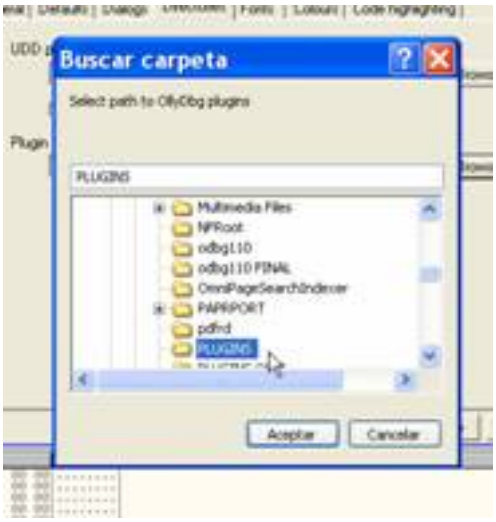
Разумеется, плагины могут находиться где угодно, но мне нравится размещать всё на C. Как бы то ни было, теперь мы должны сконфигурировать OllyDbg, чтобы она распознавала эту папку как местонахождение всех плагинов.

Для этого идём в OPTIONS->APPEARANCE.

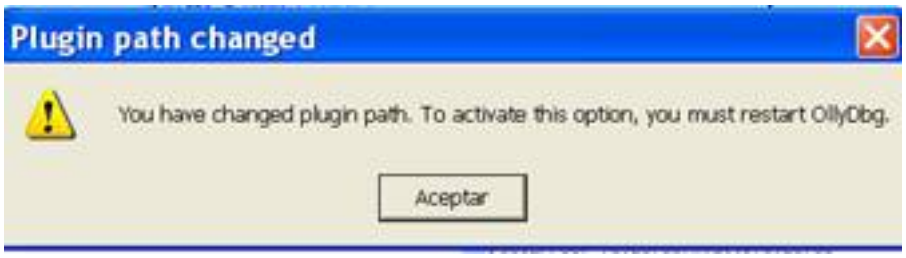
И в открывшемся окне открываем вкладку DIRECTORIES.



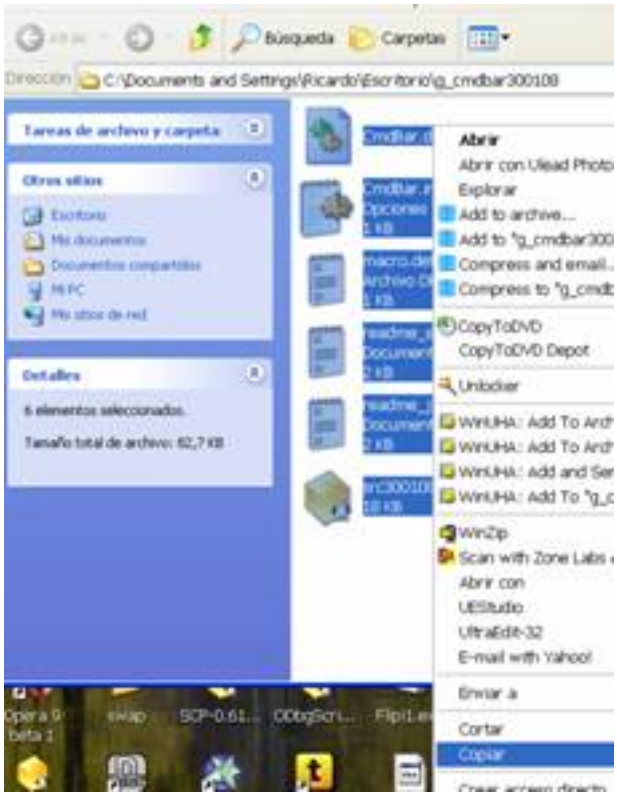
Видим, что в качестве пути к плагинам указана директория, где находится сам OllyDbg.exe, и можно было бы помещать плагины туда, но мне нравится держать их отдельно, и тогда нажимем на PLAGIN PATH->BROWSE, чтобы найти созданную нами папку.



Выбираем папку PLUGINS и сохраняем изменения.



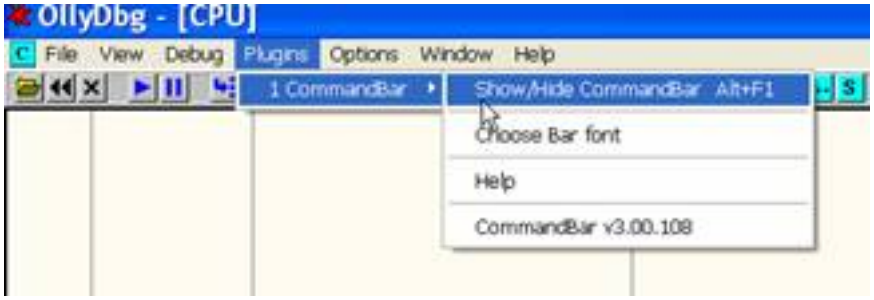
То есть нужно перезапустить Олли, чтобы она признала новую папку с плагинам, но сначала следует скопировать в последнюю скачанный плагин.



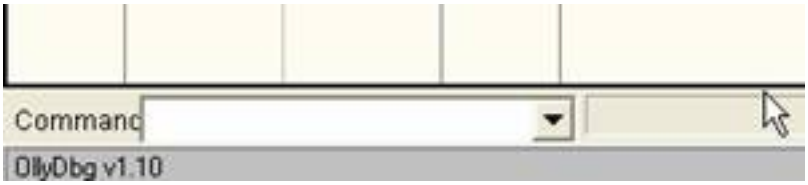
Копируем всё содержимое архива в папку PLUGINS.

Теперь все файлы плагина «Command Bar» находятся в папке PLUGINS, в неё же следует помещать и остальные (зачастую можно копировать не все файлы в архиве, а только dll).

Теперь закрываем OllyDbg, если она всё ещё была закрыта, и запускаем по новой. Видим, что в меню PLUGINS появился COMMAND BAR и его опции.



В нижней части OllyDbg видим установленный COMMAND BAR.



Это текстовое поле для ввода команд, которые нам могут пригодиться во многих случаях, и позднее мы увидим их применение, а пока важно научиться подключать плагины.

Чтобы деинсталлировать PLUGIN достаточно просто стереть соответствующую dll из папки PLUGINS и перезапустить OllyDbg, и плагин исчезнет. Вместе с тем, благоразумно всегда держать COMMAND BAR включённым.

Снова открываем крэкми CrueHead'в в OllyDbg.

Наиболее полезные клавиши в OllyDbg следующие:

F7: Выполняет одну строку кода (если находимся на CALL, то переходим внутрь вызванного участка кода)

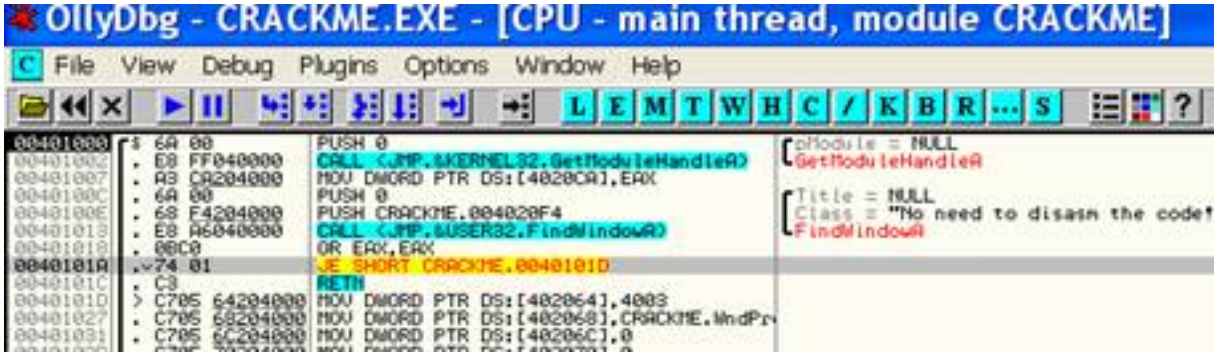
F8: Выполняет одну строку кода (если находимся на CALL, то просто выполняет вызов без перехода внутрь и переходит на следующую за CALL строку).

Это два вида ручной трассировки очень разные и в каких случаях использовать каждую из них мы рассмотрим позже.

F2: устанавливает обычную точку останова на отмеченной линии. Чтобы убрать эту точку останова, надо ещё раз нажать F2.

Например:

Хотим установить точку установа в позиции 40101A, поэтому отмечаем эту линию с помощью мыши.

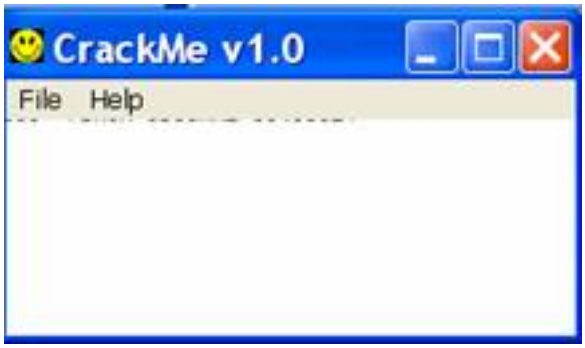


С помощью одного клика мыши она помечается и становится серого цвета как на картинке. Затем нажимаем F2.

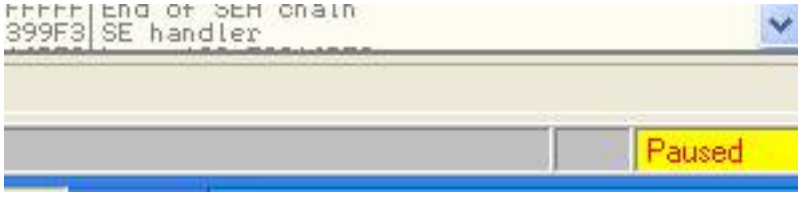
Видим, что соответствующая позиция в первой колонке становится красной, что говорит о том, что здесь есть точка останова. Нажав F2 ещё раз можно её убрать.

F9: Запускает программу, которая будет выполняться до тех пор, пока не встретить точку останова, не совершиться какое-либо исключение или же просто прекратит работу по каким-либо причинам. Когда программа запущена, в нижнем правом угле OllyDbg отображается слово RUNNING.

Запустив крэкми CrueHead'a, мы увидим следующее:



Чтобы временно прекратить выполнение программы, нужно нажать F12 или DEBUG->PAUSE.



Видим, что OllyDbg отображает слово PAUSED (пауза). Продолжить выполнение программы можно нажав F9 или DEBUG->RUN.

Чтобы закрыть отлаживаемую программу выберите DEBUG->CLOSE.

Это был краткий обзор OllyDbg, более глубокое изучение множества опций и возможностей которой мы продолжим изучать в последующих главах. Главное, чтобы вы скачали программу, сконфигурировали, ещё раз посмотрели всё, о чём было рассказано в этом туториале, так же подключили плагин, позапускали и поостанавливали крэкми CrueHead'a, попробовали поставить точки останова, чтобы в следующей главе все эти вещи не вызывали у вас колебаний и сомнений.

Введение в крэкинг с нуля, используя OllyDbg - Глава 2

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2006-09-25

После того, как мы рассмотрели строение, основные элементы и принципы OllyDbg, необходимо изучить системы счисления и что такое стек, чтобы иметь о них хоть какое-то представление.

Системы счисления

Наиболее часто используемыми системами счисления являются двоичная, десятеричная и шестнадцатеричная.

Основное, что о них нужно знать:

• ДВОИЧНАЯ: представляется символами 0 и 1, и поэтому называется двоичной.
• ДЕСЯТЕРИЧНАЯ: представляется десятью символами (от 0 до 9), и поэтому называется десятеричной.
• ШЕСТНАДЦАТЕРИЧНАЯ: представляется символами от 0 до F (0-9, а также A, B, C, D, E и F, то есть 16 символов в итоге).

Обычно, если будет упоминаться какое-то число, то будет считаться, что оно в шестнадцатеричной системе, если иное не указано явно.

Существуют не слишком приятные математические формулы, которые мы не будем использовать, для конвертации чисел из одной системы в другу. Настоящий крэкер использует для этого «Windows-калькулятор», что гораздо быстрее и проще и избавляет от работы со степенями, суммами и прочим при конвертации чисел.

Открывает калькулятор и подготавливаем его для работы.

Теперь идём в меню «View», чтобы включить научный режим.

Здесь мы видим, что по умолчанию стоит десятеричная система счисления. Сбоку можно изменить её на одну из трёх: шестнадцатеричную (HEX), восьмиричную (OCT) и двоичную (BIN).

Восьмеричная система, использующую 8 символов, не слишком используется в крэкинге, но если потребуется, то в калькуляторе такая опция есть.

Поэтому, чтобы перевести число из одной системы счисления в другую наиболее простым образом, переводим калькулятор в исходную систему счисления, например, если хотим перевести 55 из десятечной в шестнадцатеричную, то ставить в калькуляторе десятеричную систему счисления и набираем 55.

Теперь переводим калькулятор в шестнадцатеричную систему счисления, и введённое число автоматически конвертируется в него.

Таким образом, видимо, что 55 в десятеричной системе – это 37 в шестнадцатеричной.

При этом появились неиспользованные в десятеричной системе счисления клавиши с символами A, B, C, D, E, F, чтобы мы могли их вводить.

Думаю, что этот способ более практичен в использовании, и позволяет нам переводить числа из одной системы счисления в другую без особых сложностей.

Отрицательные шестнадцатеричные числа

Это понять гораздо сложнее, поэтому начнём издалека. Как в шестнадцатеричной системе можно представить отрицательное число, если мы не можем использовать для это цели минус как в трационной десятеричной системе? Как представить в шестнадцатеричном формате, например, -1?

Рассмотрим эту проблему, и надеюсь, что всё станет понятно.

Если у нас есть возможность писать шестнадцатеричные числа от 00000000 до FFFFFFFF, то как нам представлять отрицательные числа?

Мы можем использовать половину этого диапазона для положительных чисел, а половину – для отрицательных.

Положительные числа будут идти от 00000000 до 7FFFFFFF, а отрицательные – до 80000000.

Положительные

000000000 всё равно, что 0 в десятеричной

000000001 всё равно, что 1 в десятеричной

...

...

7FFFFFFF всё равно, что 2147483647 в десятеричной (и это максимальное положительно е число)

Отрицательные

FFFFFFFF всё равно, что -1 в десятеричной

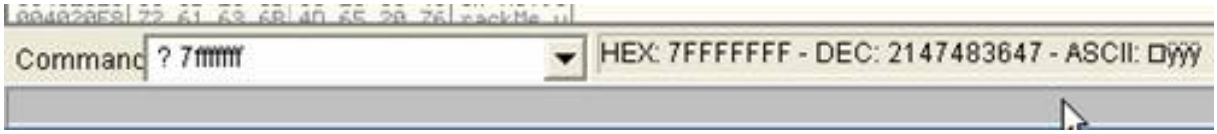
FFFFFFFE всё равно, что -2 в десятеричной

...

...

80000000 всё равно, что -2147483648 в десятеричной (и это максимальное отрицательное число)

Можно попробовать узнать в Command Bar значение 7FFFFFFF в десятеричной системе, для чего используется вопросительный знак, после которого идёт собственно само значение.

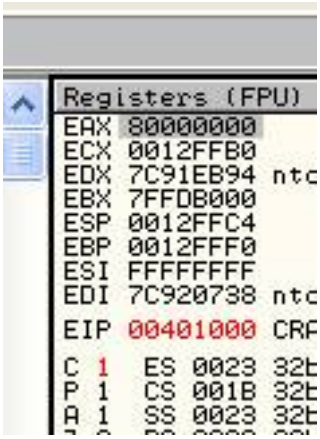


Справа видим, что получили соответствующее десятеричное значение, которое равно 2147483647, то есть всё в порядке.

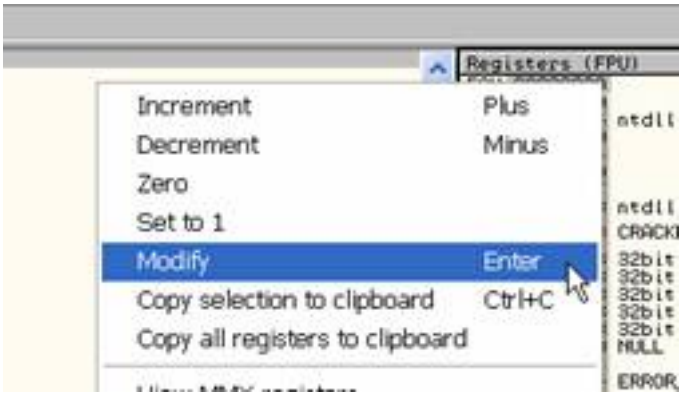
Теперь когда хотим выяснить, является ли значение 80000000 отрицательным, видим, что не показываются результаты для значений после 7FFFFFFF (это баг в Command Bar), поэтому как же нам выяснить его значение в OllyDbg?

Для этого есть маленький трюк.

Идём в регистры и выделяем EAX.



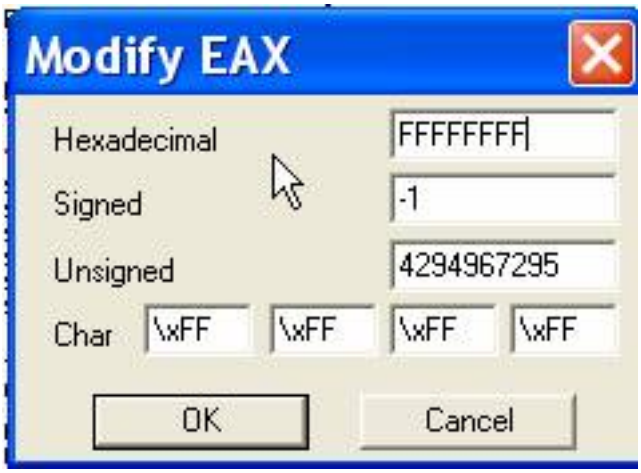
Теперь нажимаем на правую кнопку мыши и выбираем “MODIFY”.



Нам показывается окно, с помощью которого мы можем поместить в EAX нужное нам значение, хотя также это окно полезно для совершения различного рода конверсий. В первом поле задаётся шестнадцатеричное значение, которое мы хотим сконвертировать, а во втором появляется соответствующее значение в десятеричной.

В данном случае видим, что 80000000 соответствует десятеричному -214783648.

Если хотим убедиться, что FFFFFFFF это десятеричное -1.



Таким образом, в окне модификации регистра мы можем легко проверять значение отрицательных числе, после чего можем выйти, нажав “Cancel”, не изменив на самом деле регистр никаким образом.

ASCII-символы

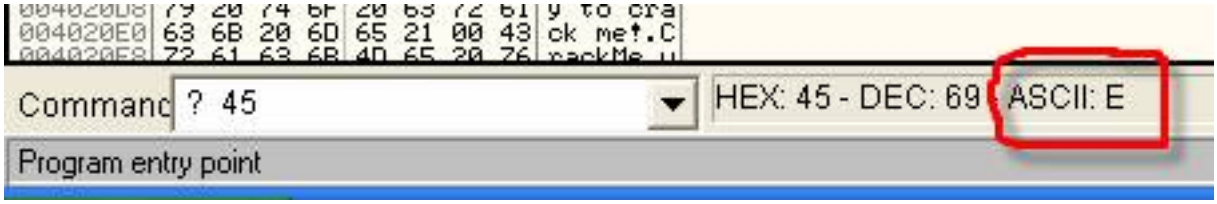
Одна из тем, которую мы должны освоить, это вид представления данных на экране, при котором каждому символу присваивается шестнадцатеричное значение, что позволяет нам интерпретировать их как буквы, символьные значения и т.д.

В таблице, скопированной (xe-xe) из «Теории ассемблера» (Caos Reptante), можно видеть десятеричное значение и соответствующие ему шестнадцатеричное и символьные значения. Например, если нужно использовать символ пробела в Олли, то можем применить 20 (hex) или 32 (dec).

Dec.	Hex.	Carac	Dec.	Hex.	Carac	Dec.	Hex.	Caract
32	20	esp	64	40	@	96	60	`
33	21	!	65	41	A	97	61	a
34	22	"	66	42	B	98	62	b
35	23	#	67	43	C	99	63	c
36	24	$	68	44	D	100	64	d
37	25	%	69	45	E	101	65	e
38	26	&	70	46	F	102	66	f
39	27	'	71	47	G	103	67	g
40	28	(	72	48	H	104	68	h
41	29	)	73	49	I	105	69	i
42	2A	*	74	4A	J	106	6A	j
43	2B	+	75	4B	K	107	6B	k
44	2C	,	76	4C	L	108	6C	l
45	2D	-	77	4D	M	109	6D	m
46	2E	.	78	4E	N	110	6E	n
47	2F	/	79	4F	O	111	6F	o
48	30	0	80	50	P	112	70	p
49	31	1	81	51	Q	113	71	q
50	32	2	82	52	R	114	72	r
51	33	3	83	53	S	115	73	s
52	34	4	84	54	T	116	74	t
53	35	5	85	55	U	117	75	u
54	36	6	86	56	V	118	76	v
55	37	7	87	57	W	119	77	w
56	38	8	88	58	X	120	78	x
57	39	9	89	59	Y	121	79	y
58	3A	:	90	5A	Z	122	7A	z
59	3B	;	91	5B	[	123	7B	{
60	3C	<	92	5C	\	124	7C	|
61	3D	=	93	5D	]	125	7D	}
62	3E	>	94	5E	^	126	7E	~
63	3F	?	95	5F	_	127	7F	□

В остальных случаях мы можем узнать символьное значение шестнадцатеричного числа с помощью Command Bar:

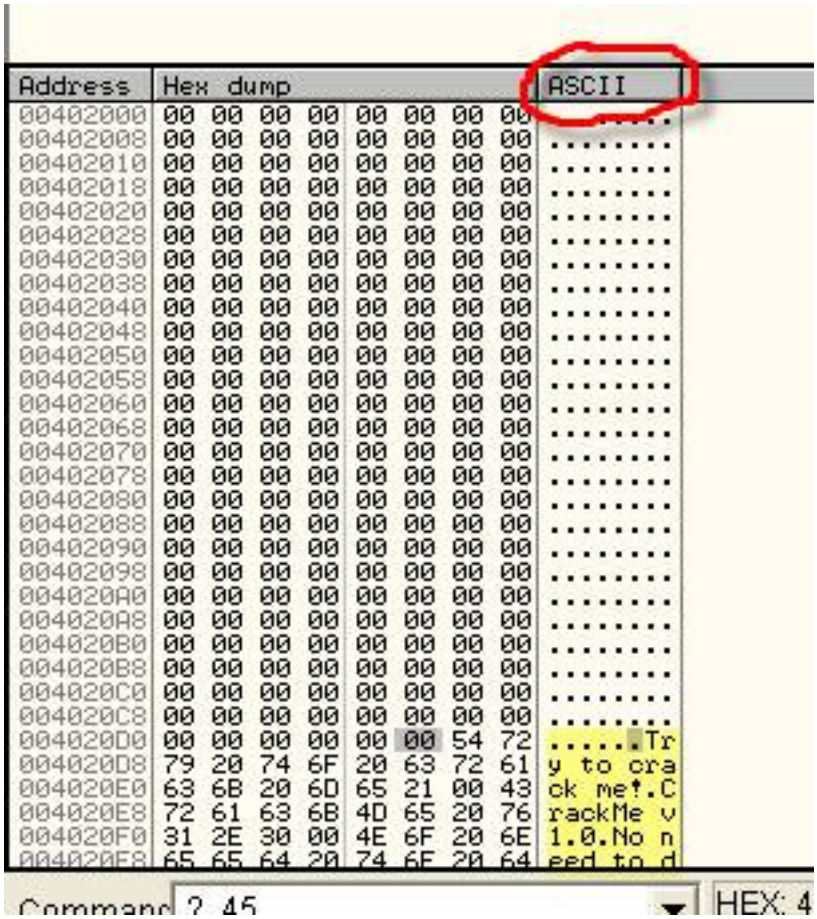
? 45



Видим, что 45 соответствует заглавное букве “E”, и если поищем число 45 в средней колонке вышеприведённой таблицы, то убедимся, что это и есть шестнадцатеричное значение буквы “E”.

69	45	E

Также в OllyDbg есть окно DUMP, где находится колонка с ASCII-символами. Давайте посмотрим, увидим ли мы какие-нибудь из них в крэкми CrueHead’a.



Видим, что сбоку от колонки, отображающей шестнадцатеричные значения, есть колонка ASCII, где можно видеть соответствующие им текстовые строки, состоящие из ASCII-символов.

Что такое стек или «куча»

Стек или «куча» - это зона памяти, где сохраняются данные, которые должны быть восстановлены в ближайшее время.

Слово «куча» напоминает о пачках или кучах писем или игральных карт, встречающихся на столах. В этих пачках самое новое письмо помещается сверху, а если письмо берётся из такой пачки, то также сверху.

Это и есть основная черта стека. Письмо помещается на верх кучи, и оно же будет тем, которое уйдёт самым первым.

В дальнейшем мы рассмотрим, как работать в OllyDbg со стеком.

Хорошо, думаю, что на этот раз достаточно. В третьей главе мы рассмотрим, что такое регистры и флаги, и для чего они предназначаются.

Введение в крэкинг с нуля, используя OllyDbg - Глава 3

Автор: Рикардо Нарваха, пер. Aquila

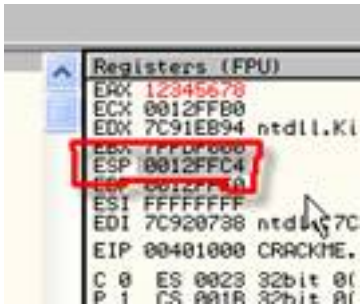
Дата: 2006-10-06

Что такое регистры и для чего они предназначаются

Для чего же нужны регистры и что это такое?

Процессору нужны помощники для выполнения программ. Таковыми и являются регистры: когда необходимо выполнить какую-либо инструкцию, например, которая складывает содержимое двух ячеек памяти, процессору нужно разместить содержимое одной из них в регистре, а затем сложить то, что в нём находится с содержимым другой ячейки. Это один из примеров использования регистров.

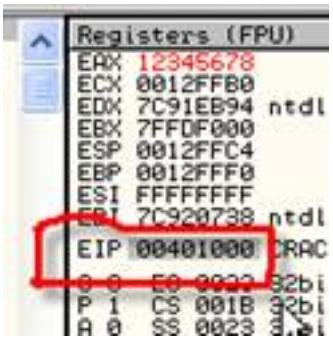
ESP указывает на самое верхнее значение стека, посмотрим это на примере крэкми от Cruehead'a.



ESP равняется 12FFC4, и если посмотрим в OllyDbg на стек,

то увидим, что регистр указывает на верхнее значение нашего стека или, говоря образно, на верхнее письмо в пачке.

EIP - это другой очень важный регистр, который указывает на инструкцию, выполняющуюся в данный момент.



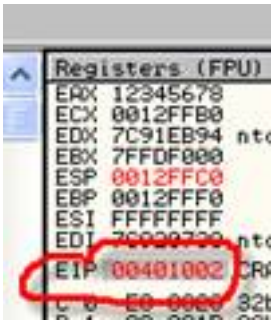
Видим в листинге, что при загрузке крэкми адресом первой выполняемой инструкции является `401000`, и очевидно, что именно это значение и содержится в `EIP`.

Если нажать на F7, то выполнится первая инструкцию и наступит очередь второй.

Теперь `EIP` равен `401002`, и в листинге видим, что первая инструкция выполнилась и находимся сейчас на второй.

Другие регистры могут содержать различные значения и служить для помощи процессору в выполнении инструкций.

Вспомним, где OllyDbg показывает значения регистров:



Взглянув туда, узнаём, что существуют `EAX`, `ECX`, `EDX`, `EBX`, `ESP`, `EBP`, `ESI`, `EDI` у `EIP`.

Они называются 32-х битными регистрами.

OllyDbg отображает их содержимое в шестнадцатеричной форме. Например, минимальное значение `EAX` может быть равно `00000000`, а максимальное - `FFFFFFFF`, что в двоичной системе счисления будет `11111111111111111111111111111111`.

Видим, что есть 32 бита, каждый из которых может быть равен 0 или 1, поэтому регистры и называются 32-х битными.

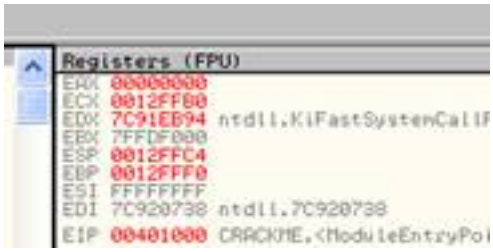
В ассемблере можно обращаться к частям этих 32-х битных регистров.

Рассмотрим пример работы с регистрами, чтобы немного попрактиковаться в OllyDbg.

Изменим значение `EAX` на то, которое нужно нам, в данном случае это `12345678`. Открываем OllyDbg, где загружен крэкми Cruehead'a (хотя там может быть и что-то

другое).

В открывшемся окне пишем в поле “Hexdecimal” 12345678.



Вот так:

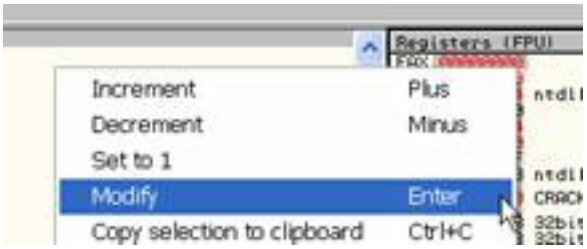
Потом нажимаем OK.

Видимо, что теперь в регистре нужное значение. OllyDbg выделяет изменённые значения красным цветом.

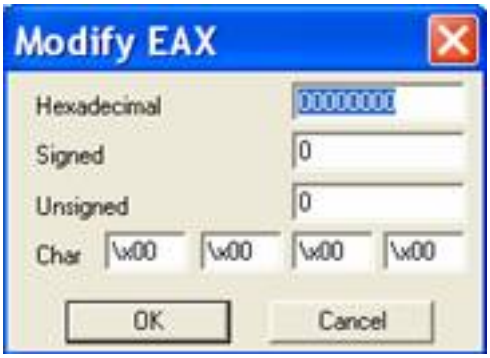
При желании можно использовать часть EAX, в данном случае AX будет 16-ти битным регистром, например, в примере выше в нём будет находится 5678, в чём можно убедиться, напечатав в Command Bar:

```
? AX
```

(знак вопроса также служит для нахождения значения выражения или регистра).



После нажатия клавиши ENTER.



видим пресловутые 5678, т.е. AX содержит последние четыре цифры EAX. Также существуют AL и AH, значения которых также можно посмотреть в OllyDbg.

```
? AL
? AH
```

Или если EAX = 12345678, то AX - это последние 4 цифры.

AH - это 5 и 6 цифры, а AL - это две последние.

Таким же образом EBX делится на BX, BH и BL, равно как подобное деление существует и почти для всех остальных регистров.

Как изменить значение регистров

Как мы увидели, в OllyDbg можно изменять значения регистров. Всё, что мы сделали с регистром EAX применимо и к другим регистрам: отметьте регистр, значение которого вы хотите изменить, затем кликните по правой кнопке и нажмите “Modify”. Единственное исключение - это EIP, указывающий на инструкцию, которая должна выполнится.

Чтобы изменить его, сделайте следующее: поскольку EIP всегда указывает на инструкцию, которая должна выполниться, то просто выбираем новую инструкцию в листниге.

После того, как отметили, например, `40101A`, кликаем правую кнопку мыши -> “New origin here”, и `EIP` изменится на `40101A`, и таким образом, программа продолжит выполнение именно с этго места.

Как видим, `EIP` содержит `40101A`.

Что такое флаги?

Как мы увидели в первой главе, в OllyDbg под регистрами находятся флаги.

Видим, что флаги бывают `C`, `P`, `A`, `Z`, `S`, `T`, `D` у `O`.

Видим, что они могут иметь только два значения: ноль или один. Определённые инструкции при выполнении могут изменять их значение.

Мы собираем рассмотреть следующие флаги:

i) Флаг “O” или флаг переполнения

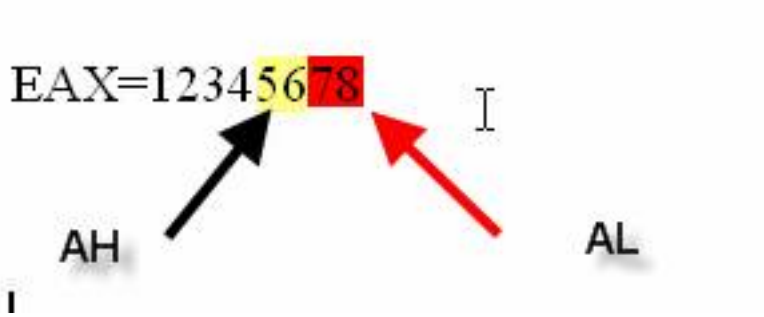
Этот флаг активируется, когда в результате операции изменяется знак и возвращается неправильное значение.

Рассмотрим в OllyDbg следующий пример, как обычно используя крэкми Cruehead'a.

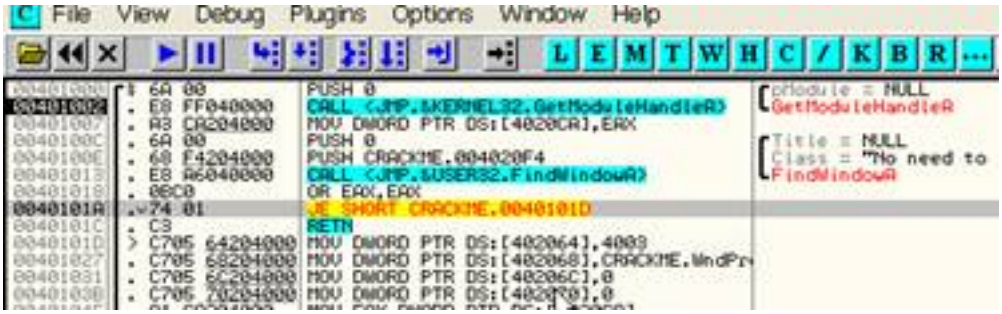
Меняем таким же, как и раньше, образом значение `EAX` на `7FFFFFFF`, которое является максимальным положительным.

Теперь прибавим 1, из-за чего сумма превысит максимальное позитивное число, потому что, как мы знаем, `80000000` соответствует числу отрицательному.

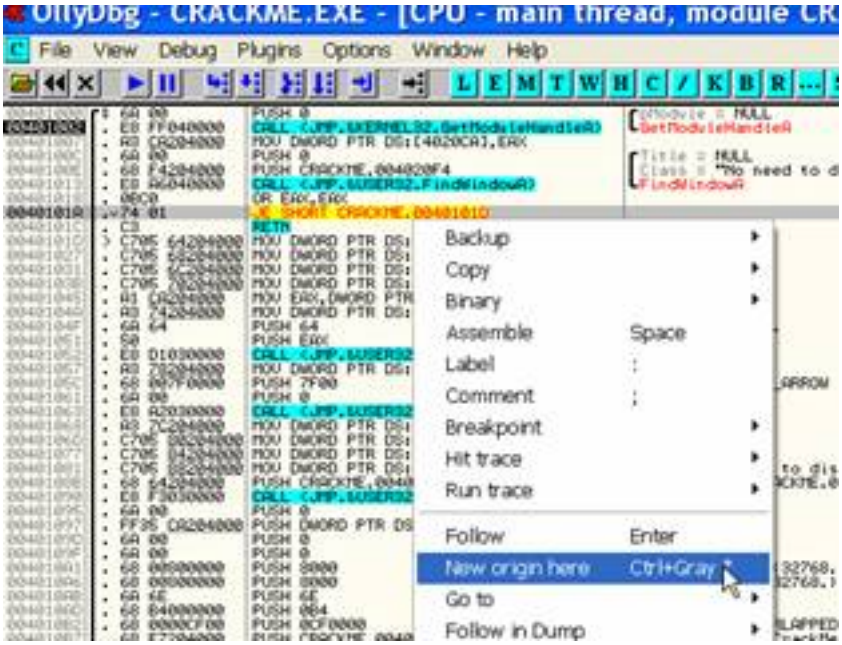
Для этого вызовем окно, где можно писать инструкции.



В нём пишем `ADD EAX, 1`.



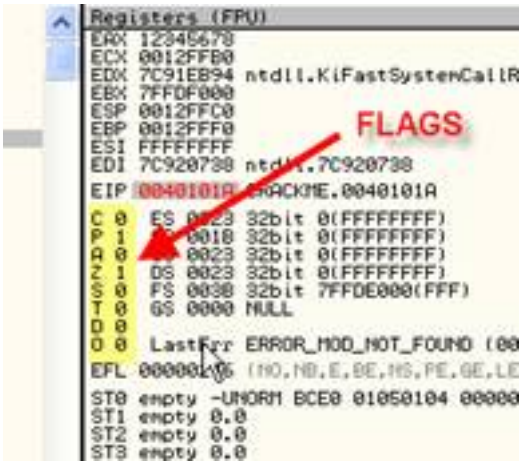
После нажатия на кнопку “ASSEMBLE” увидим, что инструкция по адресу `401000` изменится на ту, которая была введена.



`ADD EAX, 1`, как мы узнаем, когда будут перечисляться и объясняться инструкции, прибавляет к `EAX` число 1 и сохраняет в этом же регистре полученное значение.

До выполнения этой строки с помощью F7 флаг “O” равен нулю.

Если выполнить данную строку с помощью F7, чтобы увидеть, что произойдёт, то в `EAX` окажется `80000000` и знак числа будет изменён.



Флаг “O” станет активным и будет показывать 1, то есть, что результат выполненной инструкции превзошёл максимально возможный результат. Назначения этого флага - показывать, когда это происходит.

ii) Флаг “A” или дополнительный флаг переноса

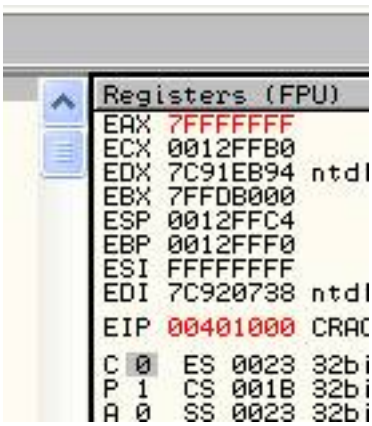
Имеет схожее назначение при выполнении операция с другими форматом, поэтому сейчас он нам не интересен.

iii) Флаг “P” или флаг чётности

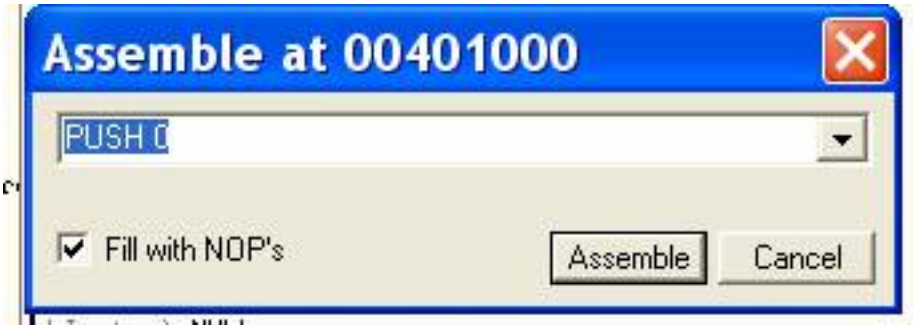
Данный флаг активируется, когда результат выполнения инструкция в двоичной форме содержит чётное количество единиц, например, `1010`, `1100` или `1111000`.

Чтобы опробовать его, у нас уже есть заданная в OllyDbg инструкция `ADD EAX, 1`, и что выполнить это строку снова, выделим строку `401000`, кликнем правую кнопку мыши и выберем “New orgin here”, чтобы вернуться назад, и теперь, если нажмём F7, выполнится инструкция, которую мы задали - `ADD EAX, 1`.

Сейчас у нас в `EAX` содержится значения `00000000`, а флаг “P” равен 1 (как результат выполнения предыдущей инструкции), посмотрим, что случится, когда прибавим к `EAX` 1.

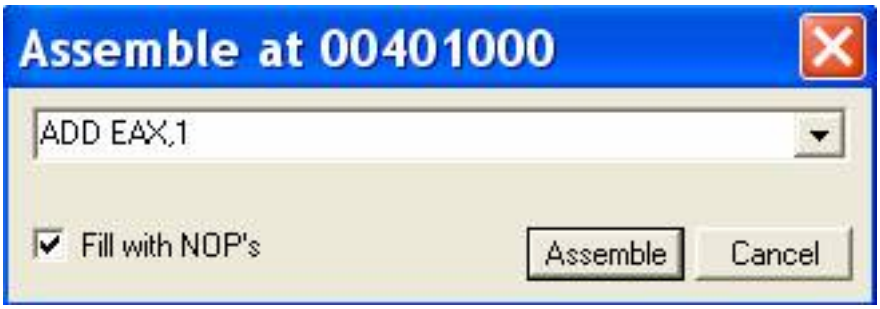


Нажимаем F7.

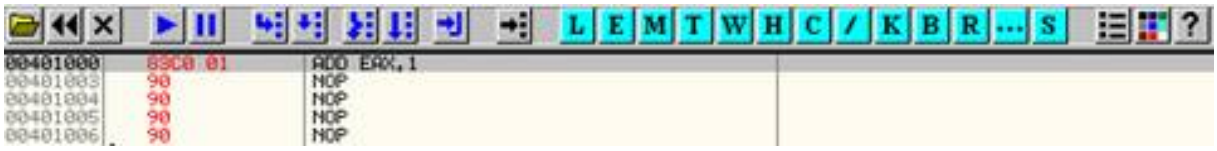


Видим, что “P” стал равен 0, так как результат, содержащийся в EAX, равен в двоичной форме 1, что в двоичной форме содержит всего лишь одну единицу, то есть нечётное количество.

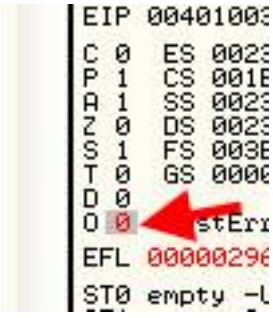
И снова кликаем на ADD EAX, 1, а потом правой кнопкой мыши и выбираем “New origin here”, чтобы снова прибавить 1. Нажимаем F7.



Видим, что в EAX, содержавшем прежде значение 1, теперь находится 2, что в двоичной форме равно 10. Число единиц опять нечётно и флаг “P” неактивен. Повторим процесс с прибавлением единицы ещё раз.



Теперь в EAX содержится значение 3, что в двоичной форме равно 11, и сейчас результат содержит чётное количество единиц, а флаг чётности стал активен.

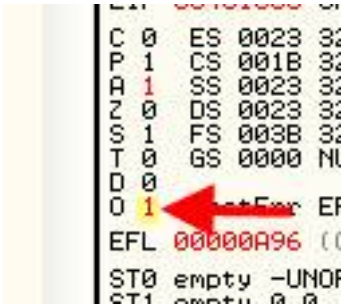


Вот мы и познакомились с тем, как работает вышеуказанный флаг: если результат выполнения инструкции содержит в двоичной форме чётное количество единиц, то флаг активен.

iv) Флаг “Z” или флаг нуля

Это один из самых известных и используемых в крэкинге флагов, который становится активным, когда результатом выполненения инструкции является ноль.

Вернёмся к нашей инструкции `ADD EAX, 1` по адресу `401000` (клик правой кнопкой мыши - “New origin here”), но сделаем значение `EAX` равным `FFFFFFFF`, что равно десятиричному -1, поэтому когда нажмём F7 и запустим `ADD EAX, 1`, результатом -1 + 1 будет ноль и флаг “Z” должен стать активным.



Видим, что после нажатия F7 `EAX` содержит ноль, а раз результат - ноль, то флаг `Z` активен и равен 1.

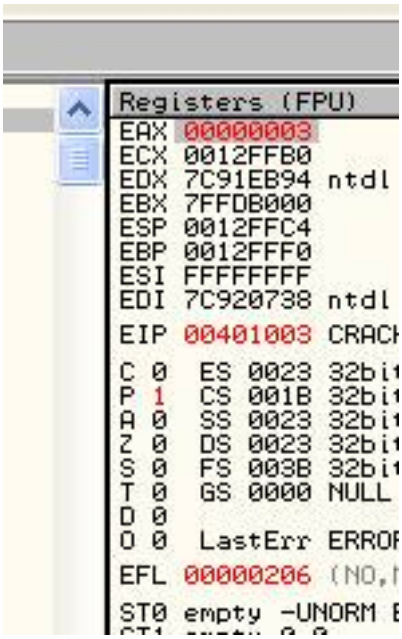
Думаю, теперь стало ясным, что данный флаг активен, когда результат инструкции равен нулю. Различные способы достичь этого мы узнаем позже.

v) Флаг “S” или флаг знака

Этот флаг равен 1, когда результат операции отрицателен. Что посмотреть его в действии, изменим занчение `EAX` на `FFFFFFF8`, что равно -8 в десятиричной форме.

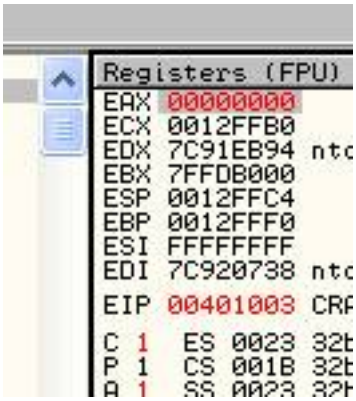
С помощью “New origin here” и кнопки A7 можно снова выполнить `ADD EAX, 1`. Результатом будет `FFFFFFF9`, то есть -7 в десятиричной форме, которое является отрицательным числом, поэтому флаг знака должен стать активным.

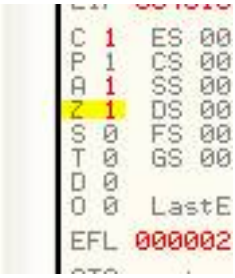
Нажимаем F7 и после выполнения инструкции активируется флаг “S”, который теперь равен 1, то есть ясно, что отрицательный результат его активирует.



vi) Флаг “C” или флаг переноса

Активен, когда превышается максимальное значение, которое может содержать регистр, например, если поместим в EAX значение FFFFFFFF и прибавим 1, то увидим, что флаг переноса стал равен 1.





vii) Флаги T, D и I

Я не буду объяснять их назначение сейчас, так как это достаточно сложная тема, и пока нам они не слишком инетересны. Пока мы будем рассматривать более простые инструкции, а эту тему оставим на потом.

Таким образом, мы получили представление о каждом регистра и в каких случаях активируются флаги. С помощью этой информации мы можем начать изучение инструкций одну за другой, так как пока мы только видели инструкцию `ADD`.

Если вам ещё что-то осталось непонятным, попрактикуйтесь в изменении значений флагов с помощью использованной нами инструкции `ADD EAX, 1`, а затем переходите к следующей части данного «Введения...»

Очень важно, чтобы вы поняли основую суть, так что не колебайтесь попрактиковаться и пееречитать этот материал ещё раз.

C 1 ES 0023 32
P 1 CS 001B 32
A 1 SS 0023 32
Z 0 DS 0023 32
S 1 FS 003B 32
T 0 GS 0000 NU
D 0
O 0 LastErr ER
EFL 00000297 (N

Введение в крэкинг с нуля, используя OllyDbg - Глава 4

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2007-01-27

ИНСТРУКЦИИ

Как уже говорилось в прошлых частях, главной целью данного "Введения..." является объяснение теории, практика в OllyDbg, чтобы её усвоить, а в дальнейшем, после приобретения уверенности в своих навыках, перейти к возможностям, которые они дают.

Рассмотрев в OllyDbg почти все флаги, мы перейдём к рассмотрению наиболее важных инструкций. Если вы встретите в листинге инструкцию, которой не окажется здесь, вы можете проконсультироваться с каким-нибудь более полным руководством по ассемблеру.

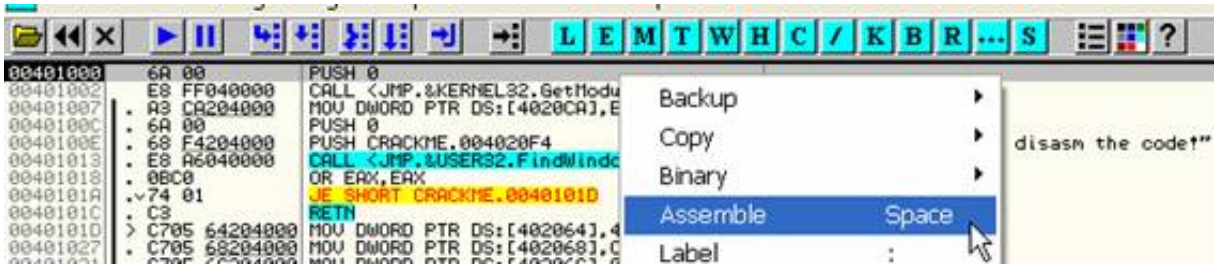
NOP (NO OPERATION)

Это инструкция, которая при запуске не производит никаких изменений в регистрах, стеке или памяти, поэтому по-английски её название расшифровывается "NO OPERATION", то есть у неё нет никакого специального анзначения, и поэтому её можно использовать, например, если нужно заменить одну инструкцию на другую, более короткую. Чтобы процессор не столкнулся с ошибками, лишнее место заполняется NOP'ами.

Также она служит для полного уничтожения другой инструкции, для этого нужно заменить её соответствующим количество NOP'ов, или попросту заNOPать.

Снова откроем крэкми CrueHead'a.

В самом начале видим оригинальный код. Чтобы занопать первую инструкцию PUSH 0, которая имеет размер 2 байта, отмечаем соответствую строку листинга с помощью мыши, а затем нажимаем клавишу "пробел" или же правая кнопка мышь - ASSEMBLE.



Здесь видим, что действительно можно просто нажать "пробел", после чего откроется окошко для ввода желаемой инструкции.

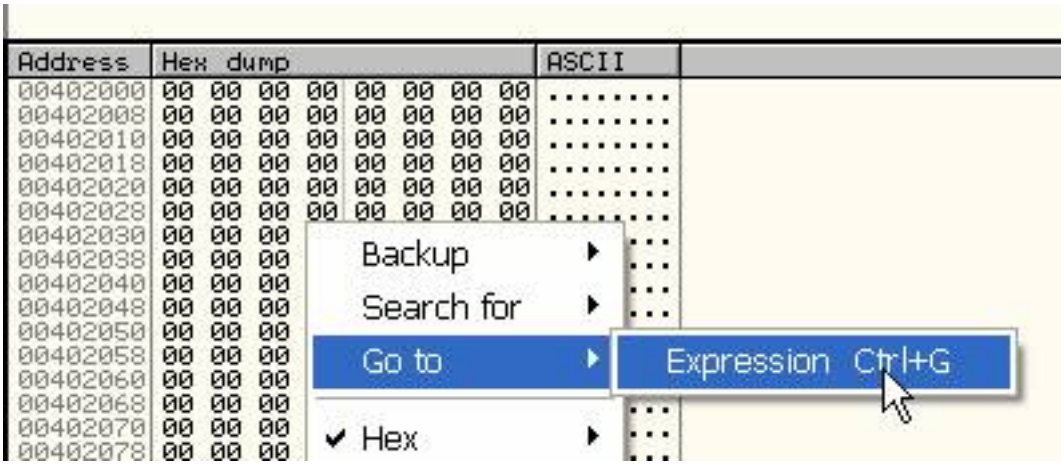
Пишем NOP и нажимаем ASSEMBLE.

Видим, что Олли, кроме того, что ввела NOP, будучи весьма умной программой, учла, что PUSH занимает два байта, и чтобы не оставлять неприкаянным следующий байт, ввела дополнительный NOP.

Теперь на месте, где был "PUSH 0", находятся два NOP'a, которые при исполнении ничего не делают, в чём мы можем убедиться, нажав два раза F7, чтобы стала отмеченной инструкция CALL. Увидим, что изменился только регистр EIP, содержащий адрес инструкции, которая должна выполнится следующей, но ничего больше не изменило своего значения: ни другие регистры, ни стек, ни флаги, ни память.

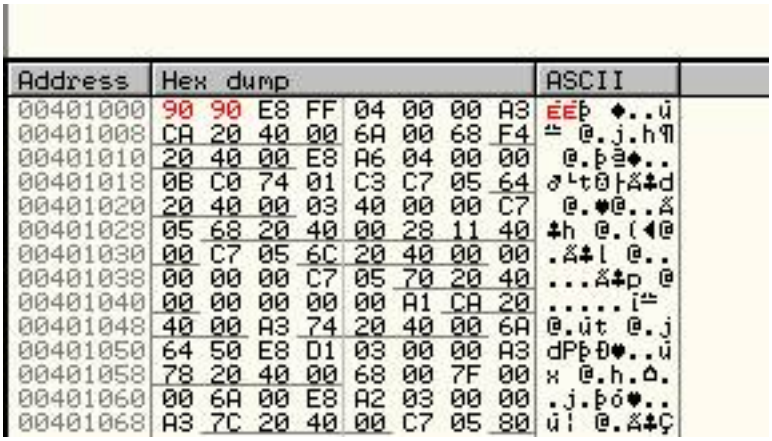
Теперь нам нужно посмотреть эти два новых байта в DUMP, и для того, чтобы найти их там, нам нужен их адрес в памяти, и это 401000 и 401001.

Идём в окно DUMP, правая кнопка мыши - "Go to" - "Expression", после чего нужно будет ввести адрес, где располагаются требуемые нам байты.



Вводим 401000.

Видим, что:



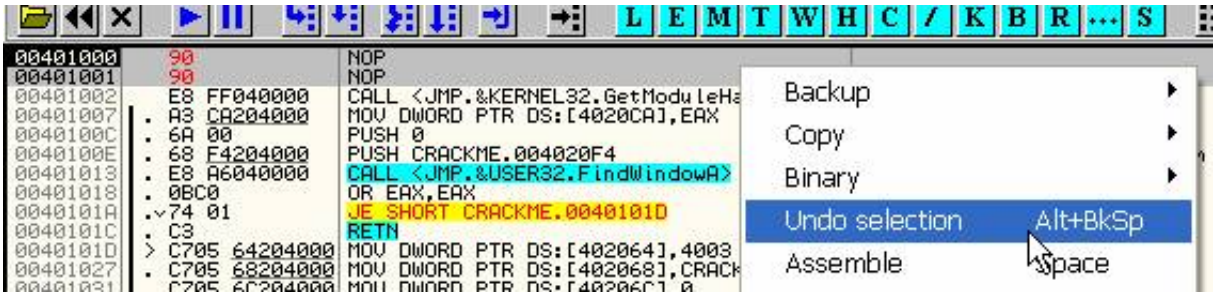
Красным выделены изменённые Олли байты. Первым идут два 90, а затем E8, FF и все остальные байты, относящиеся к следующей за NOP'ами инструкцией, которой является CALL.

Может Олли вернуть обратно изменённые нами байты?

Хе-хе, да, может.

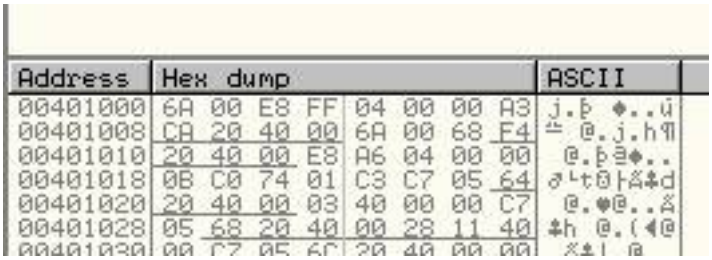
В любом из двух окон - DUMP или листинге, отмечаем оба байта.

Потом кликаем на правую кнопку мыши и выбираем "UNDO SELECTION"



И затем снова появляется изначальный PUSH.

Также, если посмотрим в DUMP, увидим, что там находятся изначальные байты.



Это всё, что касается инструкции NOP.

ИНСТРУКЦИИ СТЕКА

Мы уже говорили, что стек - это пачка писем, поверх которой последние помещаются или сверху которой они берутся.

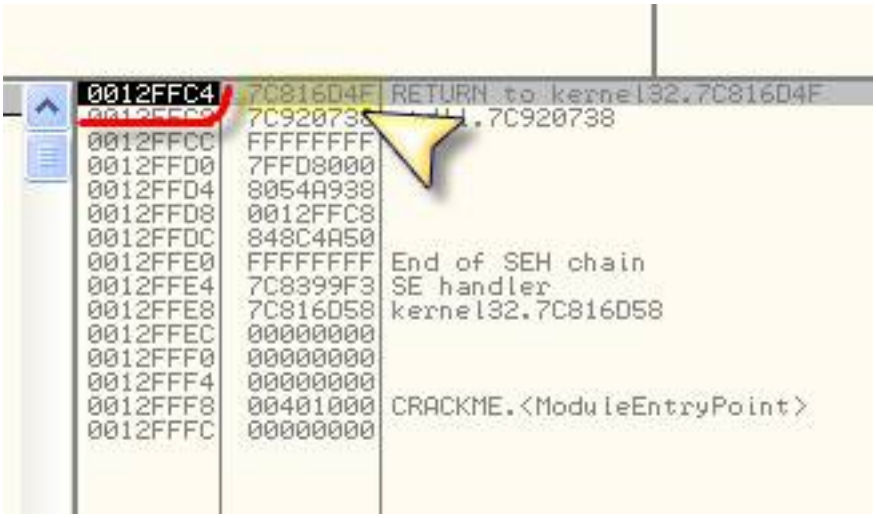
Далее следуют инструкции для помещения или удаления "писем".

PUSH

Инструкция PUSH - типичная инструкция для помещения "письма" или значения на стек. Мы можем видеть, что первая инструкция в крэкми CrueHead'a - это PUSH.

В данном случае это 'PUSH 0', и, будучи выполненной, данная инструкция поместит 0 на самый верх стека, а то, что было до этого наверху, окажется под этим значением.

Мы можем увидеть, как выглядит стек после выполнения PUSH. На вашей машине адреса могут отличаться, но эффект будет тот же самый.

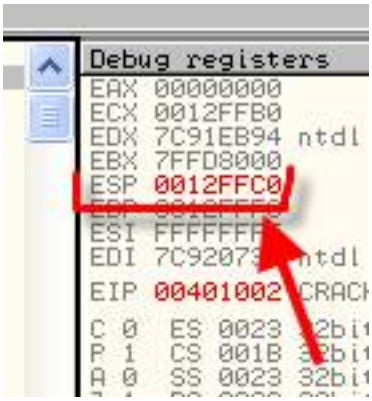


Это стек на моей машине. Адрес 12FFC4 может отличеаться у вас, так как стек каждый раз может располагаться по разному адресу, и его начальное содержимое также может отличаться, то есть у вас может быть значение, отличное от 7c816d4f. По нажатию F7 на самый верх попадёт ноль, а всё остальное окажется внизу него. Нажмём F7.

Видим, что по нажатию F7, ноль словно присоединился сверху того, что мы уже видели. Внизу по адресу 12ffc4 по-прежнему находится значение 7c816d4f, и ни одного из других значений в стеке также не изменилось.

Главным отличием является то, что самое верхнее значение стека находится теперь по адресу 12ffc0 (именно там и неаохдится ноль, который мы поместили с помощью инструкции PUSH), это действительно как положить поверх пачки писем новое, которое окажется на самом верху, а остальные - под ним, но ничего в них не изменится.

Также видим, что ESP, в котором находится адрес самого верхнего значения стека, теперь содержит 12FFc0.



Конечно, инструкция PUSH имеет несколько вариантов, позволяющих помещать в стек не только числа:

PUSH EAX помещает значение EAX на верх стека. Подобным образом мы можем поместить в стек значение любого регистра, числа и т.п.

Также можно поместить значение, находящееся в памяти по определённому адресу:

```
PUSH [401008]
```

Обратите внимание, что это будет интерпретироватиться отличным образом от следующего:

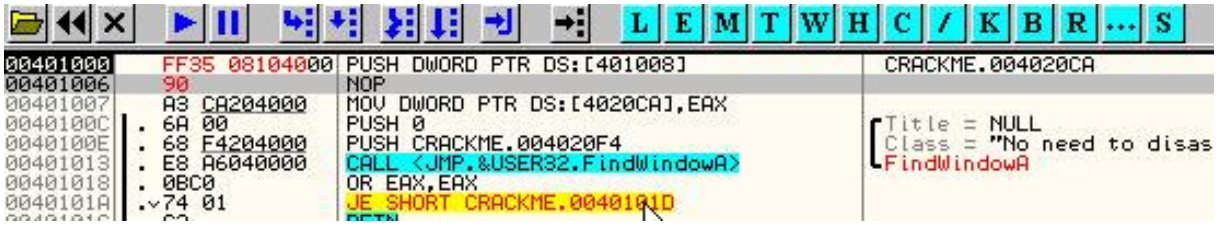
```
PUSH 401008
```

(Без квадратных скобок)

Если выполним 'PUSH 401008', то в стек будет помещено число 401008.

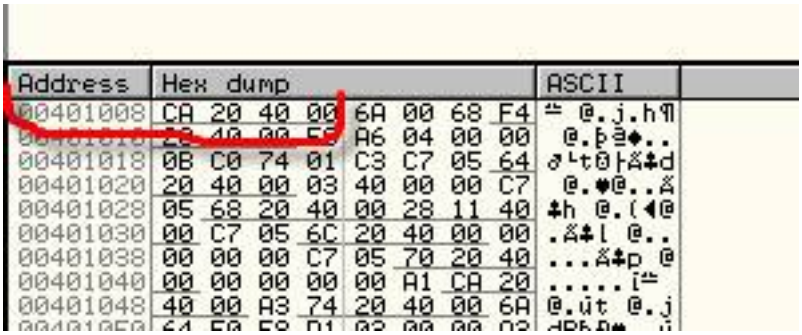
После выполнения имеем следующее:

Если заменить на 'PUSH [401008]'



Квадратные скобки указывают на содержимое ячейки памяти по адресу 401008, то есть мы должны пойти в DUMP и посмотреть, что там содержится.

С помощью "GOTO EXPRESSION 401008" видим:



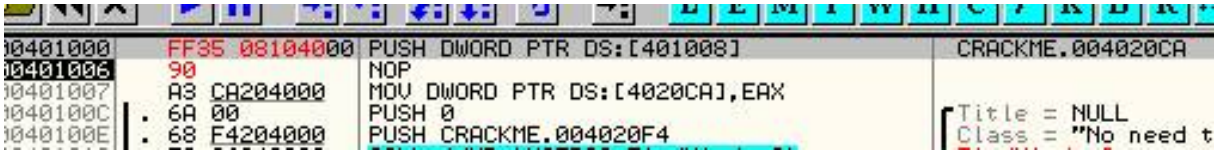
В этих четырёх байтах находятся CA 20 40 00. Запускаем PUSH с помощью F7.

Видим, что в стеке находится значение, которое мы смотрели на предыдущем шаге, но байты перевёрнуты, то есть они были помещены в обратном порядке.

Это одно из свойств процессора: при чтении или записи содержимого из/в память байты всегда переворачиваются. Жалуйтесь по этому поводу производителю процессора, хе-хе.

В общем, главная идея состоит в том, что без квадратных скобок задаваемый параметр является просто числом, а с ними оно является адресом ячейки памяти, где содержится помещаемое в стек значение.

Теперь видим, что Олли, когда пишем 'PUSH [401000]' интерпретирует и отображает



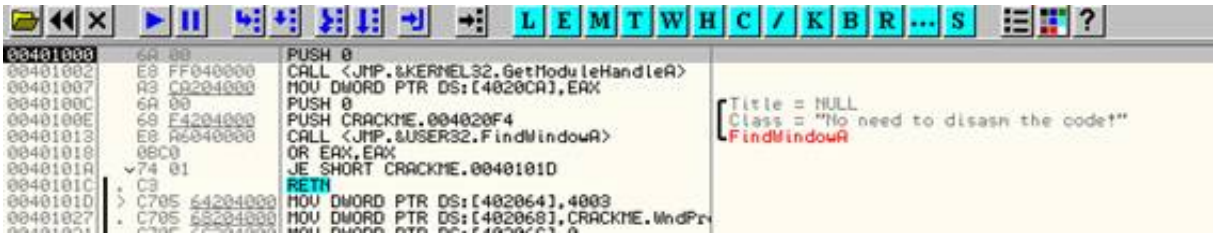
```
PUSH DWORD PTR DS:[401008]
```

Это потому, что если иное не задано специально, Олли считает, что нужно читать 4 байта из памяти, то есть DWORD. Остальные варианты мы рассмотрим в других инструкциях.

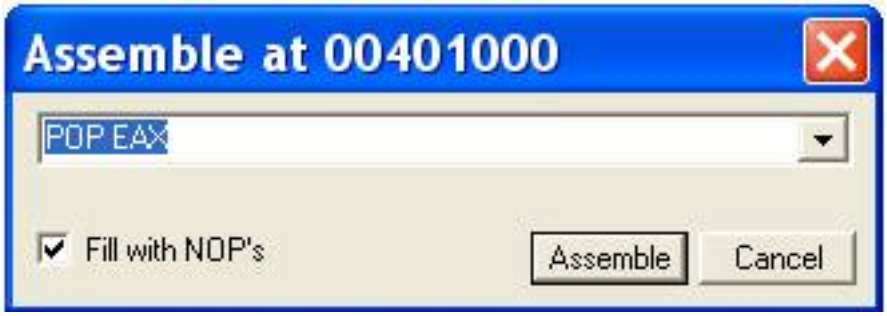
POP

Инструкция POP является обратной по отношению к PUSH: она достаёт первое письмо или первое значение из стека и помещает его в указанное параметром инструкции место назначения. Например, POP EAX берёт первое значение из стека и помещает его в EAX, после чего то значение, которое шло следующим, становится верхним.

Видимо, что в начале крэкми CrueHead'a есть следующая инструкция:

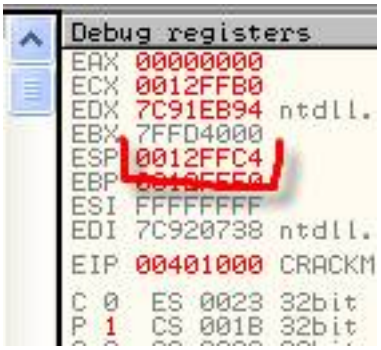


Заменим данную инструкцию на 'POP EAX', отметив первую строку и нажав "пробел":



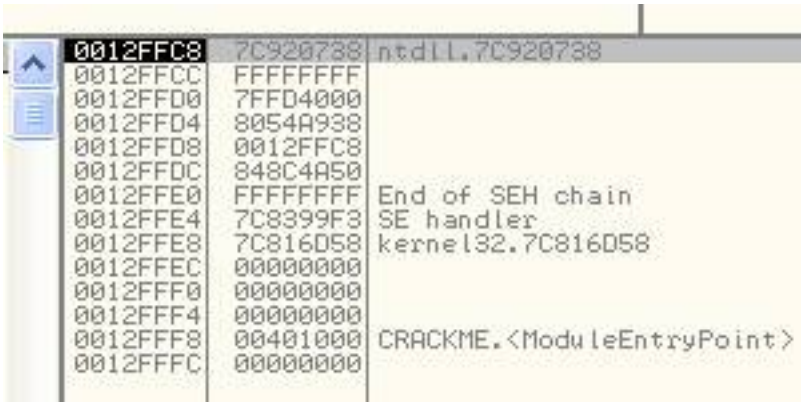
Вот что будет содержать стек до выполнения этой инструкции:

A ESP указывает на 12FFc4, что является адресом верхнего значения, содержащегося в стеке.

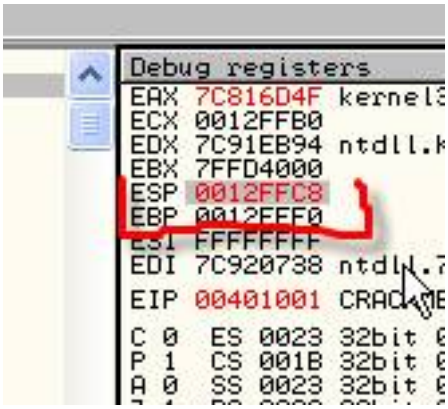


И видим что в EAX содержится ноль (в моём случае).

Нажимаем F7.



Видим, что в стеке исчезло наше первое "письмо", а ESP указывает на 12ffc8.



Но куда делось наше "письмо"? Так как инструкция был 'POP EAX', то видим, что сейчас EAX содержит значение 7c816d4f - это в моём случае, а в вашем там будет то занчение, которое лежало сверху стека.

Таким же образом, если бы была инструкция 'POP ECX' то верхнее значение было бы помещено в ECX или иной регистр, который был бы указан.

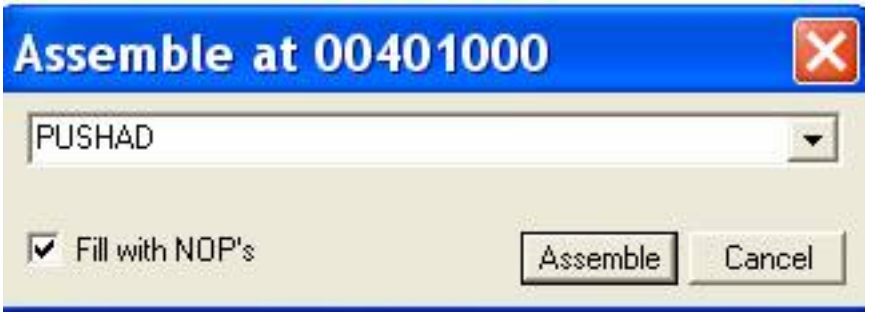
Итак, мы рассмотрели инструкции, которые помещают или берут из стека значения по одному за раз.

PUSHAD

PUSHAD помещает содержимое регистров в стек в определённом порядке, то есть PUSHAD эквивалентно 'push EAX, ECX, EDX, EBX, ESP, EBP, ESI, EDI'.

Посмотрим, правда ли то, что нам говорит камрад CAOS REPTANTE в своём туториале про ассемблер, хе-хе.

Загрузим снова крэкми CrueHead'a, вызовем меню для замены инструкции и напишем PUSHAD.



Это мой начальный стек и состояние регистров до выполнения регистров.

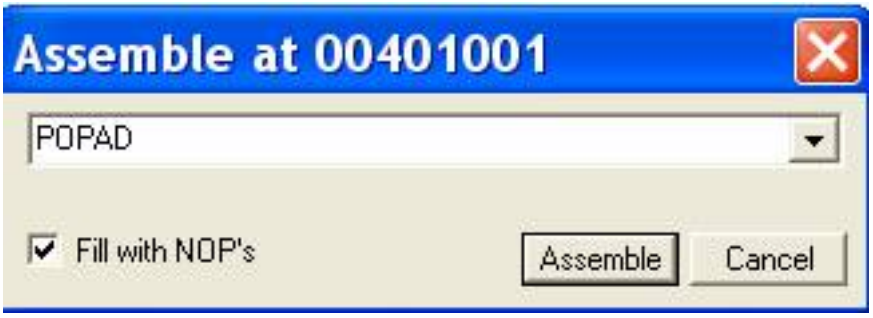
Нажимаем F7 и видим, что теперь находится в стеке:

Видим, что все регистры были подвергнуты PUSH'у. По адресу 12ffc4 находится значение, которое лежало сверху стека до выполнения инструкции, а сейчас после него лежит ноль (PUSH EAX), затем содержимое ECX, а в 12ffb0 - то, что находилось в ECX. Дальше находится содержимое регистров от ESP до EDI.

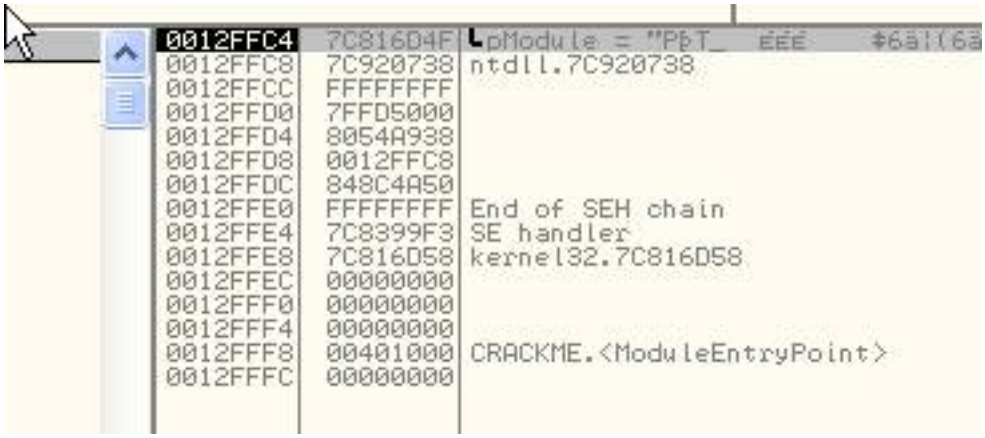
POPAD

Обратной по отношению к PUSHAD инструкцией является POPAD, которая берёт значения из стека и помещает их в соответствующие регистры. POPAD эквивалентна 'pop EDI, ESI, EBP, ESP, EBX, EDX, ECX, EAX'.

Как и в предыдущем примере введём инструкцию POPAD:



Поскольку стек уже содержит значения регистров, то выполнение POPAD возвратит его в изначальное состояние.



Теперь он точно такой же, как и был до выполнения PUSHAD, а регистры восстановили свои значения.

Комбинация PUSHAD-POPAD часто используется, когда нужно за раз сохранить содержимое всех регистров, потом совершить какие-то другие операции, меняющие значение регистра и стека, а затем восстановить их изначальное состояние с помощью POPAD.

Существуют также следующие варианты:

• PUSHA эквивалентно 'push AX, CX, DX, BX, SP, BP, SI, DI'.
• POPA эквивалентно 'pop DI, SI, BP, SP, BX, DX, CX, AX (восстанавливаемые значения, соответствующие ESP и SP не помещаются в эти регистры, а исключаются).

PUSHA и POPA похожи на своих сестёр PUSHAD и POPAD, не считая того, что используются в 16-ти битных программах, поэтому нас они не интересуют, так как OLLYDBG - это отладчик для 32-х битных программ.

ИНСТРУКЦИИ ДЛЯ ПЕРЕМЕЩЕНИЯ ДАННЫХ

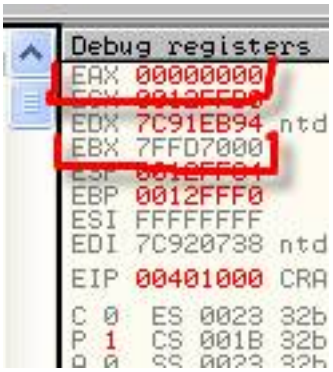
MOV

Эта инструкция перемещает второй операнд в первый, например:

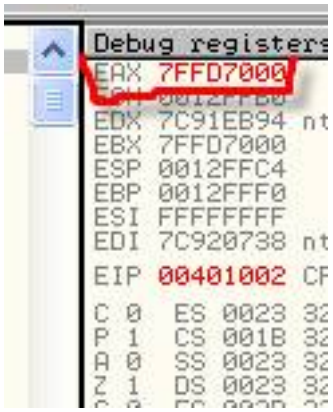
```
MOV EAX, EBX
```

Здесь идёт перемещение значения EBX в EAX. Смотрим в Олли и наше старое доброе крэкми от CrueHead'a.

Уже не буду повторять, как ввести нужную нам инструкцию, просто смотрим регистры:



На моей машине EAX содержит 0, а ECX 7c91eb94. Эти начальные значения могут у вас отличаться, но важно то, что по нажатию F7 значение EBX переместиться в EAX, так что нажмём на эту клавишу.



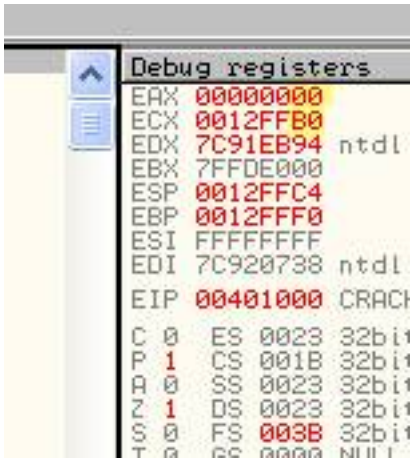
Понятно?

У MOV есть разные варианты, например:

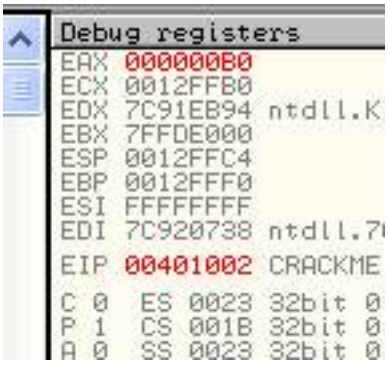
```
MOV AL, CL
```

Это перемещает содержимое CL в AL. Напишем эту инструкцию в Олли.

Регистры:



Помните, что AL - это две последних цифры EAX и CL - две последних цифры ECX. Нажмём F7.

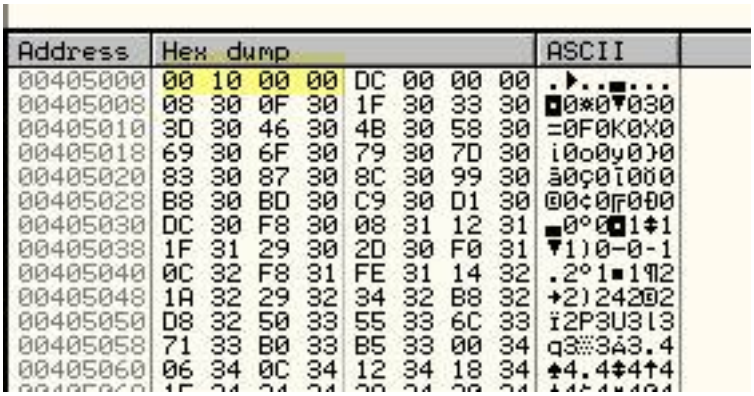


Видим, что было только скопировано B0 в AL без изменения остального содержимого EAX и ECX, то есть две последние цифры EAX.

Также можем переместить содержимое какого-либо регистра в нужную ячейку памяти или наоборот.

В данном случае перемещаем содержимое по адресу 405000 в EAX, и как было сказано ранее, DWORD значит, что нужно переместить четыре байта. Эта инструкция может вызвать ошибку, если задаваемая ячейка памяти не существует. Мы можем легко проверить это с помощью Олли.

Идём в DUMP и делаем GOTO EXPESSION 405000.



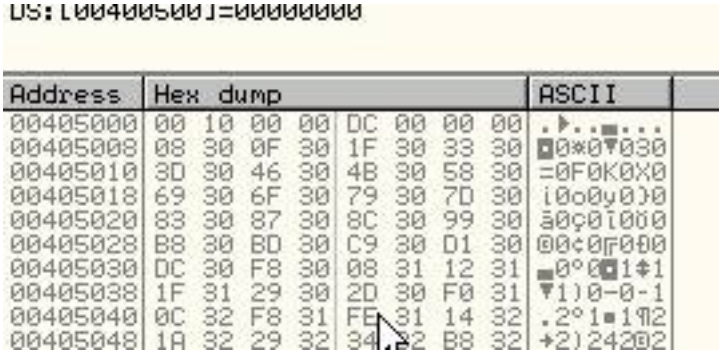
Видим, что содержимое ячейки памяти - 00 10 00 00. Так как в памяти содержимое хранится в перевёрнутом виде, то в EAX попадёт 00 00 10 00. Нажимаем F7 и смотрим, что получилось.

Вот значение 1000, которое было прочитано из памяти. Теперь если хотим записать значение по данному адресу:

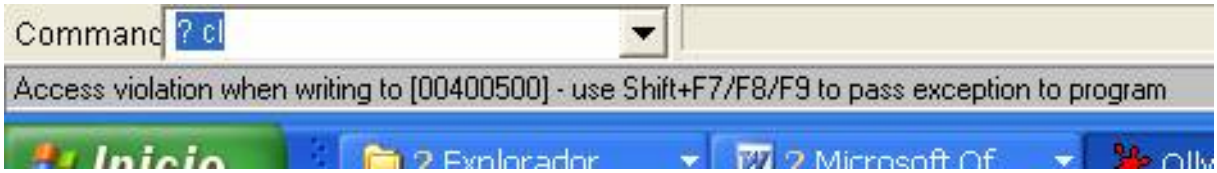
```
MOV DWORD PTR DS:[400500],EAX
```

Введём эту инструкцию.

С помощью DUMP видим по адресу 405000:



Однако по нажатию на F7 происходит облом:



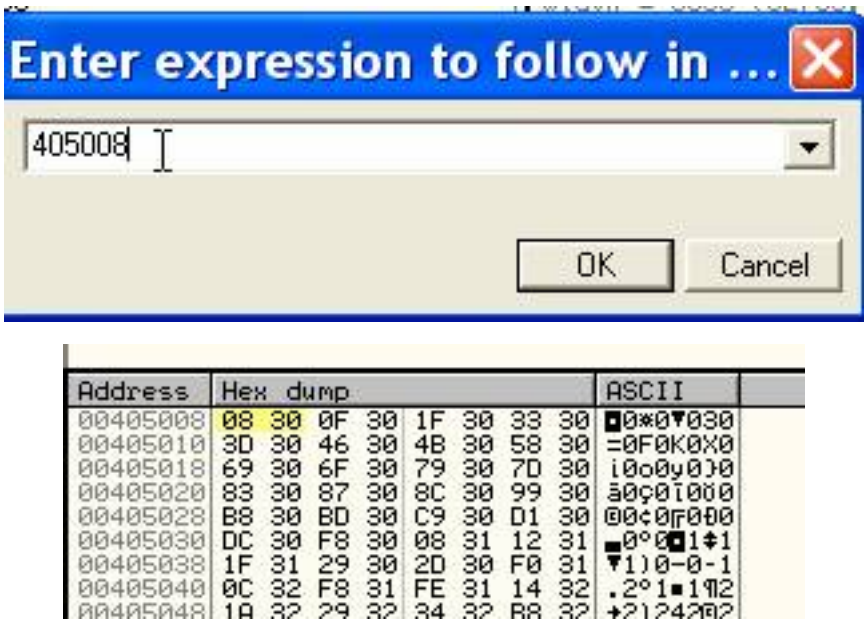
Вылетает исключение, и это происходит из-за того, что хотим записать в секцию, в которую писать, то есть изменять в ней байты, запрещено.Ок, как менять права на работу с секциями мы узнаем потом, пока важно изучить инструкции. Очевидно, что если для перемещания 4 байтов испольется слово DWORD, то для перемещения двух байтов используем WORD, а слово BYTE - для перемещения одного.

Смотрим.

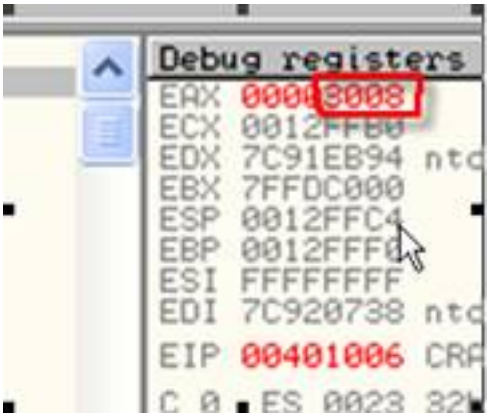
```
MOV AX,WORD PTR DS:[405008]
```

Здесь перемещаются два байта из памяти по адресу 405000 в AX. В данном случае мы не можем написать EAX, так как перемещаем только два байта, поэтому необходимо использовать 16-ти битный регистр.

С помощью DUMP смотрим, что находится по адресу 405000.



По нажатию F7 должны переместить только эти два байта в AX. Проверяем:

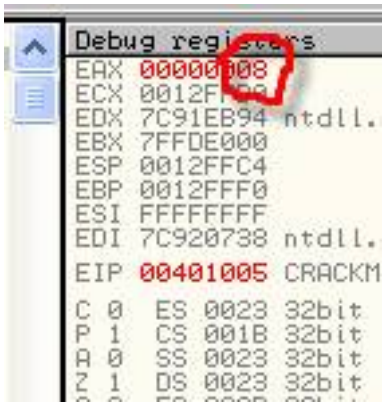


И вот в AX в обратном порядке было считано содержимое памяти, а остальная часть EAX не изменилась.

Таким же образом испольуется BYTE.

```
MOV AL, BYTE PTR DS:[405008]
```

В данном случае в AL перемещается только последний байт, то есть 08.

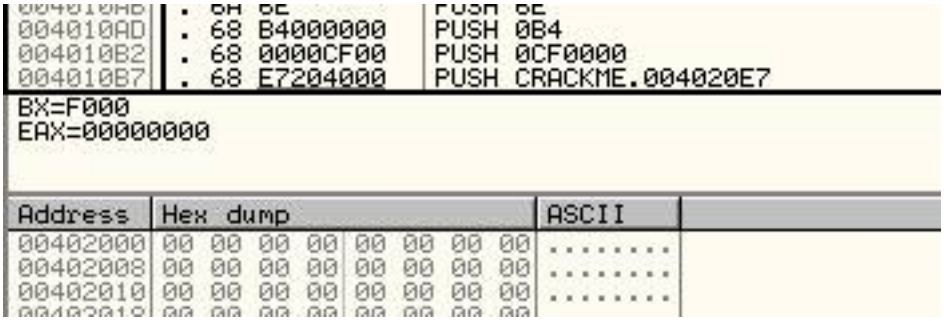


MOVSX (Move with Sign-Extension)

Копирует содержимое второго операнда, который может быть регистром или адресом памяти, в первый (который должен быть в два раза больше, чем второй), заполняя остальные биты слева значением самого значимого бита второго операнда. Далее идёт пример.

Определение взято из туториала CAOS'a. Сейчас мы попробуем тестовый пример в OllyDbg, для чего используем нашего друга CrueHead'a.

Не буду здесь много говорить, так как хочу, чтобы вы всё сделали сами и поискали значения операнда самостоятельно, хе-хе. В OllyDbg есть окошко с поясненениям прямо снизу листинга и сверху окна DUMP.



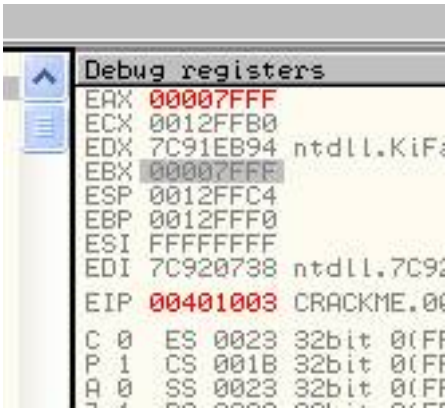
Тут видим, что окошко с пояснениями показывает нам значение операнда нашей инструкции. В моём случае BX содержит F000.

И также видим, что EAX содержит ноль, так что OllyDbg нам всегда помогает интерпретировать инструкцию, которая будет выполнена (надеюсь, что вы освоили концепцию того, где и что нужно искать, хе-хе).

Нажимаем F7.

Видим, что в AX скопировался BX, который содержал F000, и что остальное место заполнилось FFFF, так как F000 - отрицательное 16-ти битное число. Если бы BX содержал 1234, то EAX был бы равен 00001234, то есть левые байты были бы забиты нулями, так как 1234 - положительное 16-ти битное число.

Концепция положительных и отрицательных чисел в 16-ти битах работает так же как и для 32-х битных, только в интервале от 0000 до FFFF. Числа от 0000 до 7FFF являются положительными, а все, что выше до FFFF - отрицательными. Давайте посмотрим, что будет, если мы изменим BX на 7FFF, а EAX на ноль и снова выполним инструкцию.



В AX было скопировано 7FFF, а остальное - забито нулями, так как 7FFF является положительным числом. Проделаем то же самое с BX=8000 (отрицательное).

Выполним инструкцию по-новой с помощью F7.

В BX было скопировано 8000, а в остальное FFFF, так как 8000 является отрицательным числом.

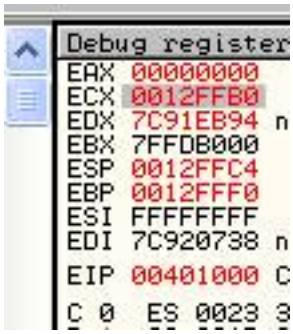
MOVZX (Move with Zero-Extend) MOVZX похож на предыдущую инструкцию, но в данном случае свободное место просто заполняется нулями и не зависит от того, является ли второй оператор положительным или нет. Не будем приводить примеры для этой инструкции, так как они похожи на те, что выше, за исключением того, что в верхней части EAX всегда будет 0000. В AX же будет копироваться BX. **LEA (Load Effective Address)**

Похожа на инструкцию MOV, но первый операнд - это регистр общего назначения, а второй - адрес ячейки памяти. Эта инструкцикция полезна в первую очередь тогда, когда адрес зависит от предыдущих вычислений.

Введём в OllyDbg следующую инструкцию:

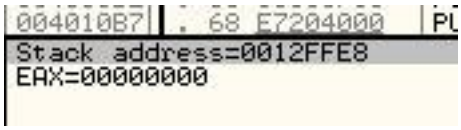
В данном случае есть квадратные скобки, однако в них задан не адрес на прямую, а вычисления, по которым он получается.

В моём случае ECX равен 12FFb0.



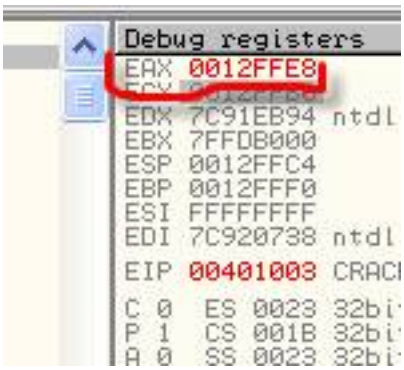
И в это примере LEA прибавит к значению ECX 38 и полученное значение поместит в EAX.

В окне пояснений будут отображены оба операнда.



Оно показывает, что операнд равен 12Ffe8, что является суммой ECX+38, а EAX до выполнения инструкции содержит ноль.

Нажимаем F7.



Указанный адрес был помещён в EAX, хотя квадратные скобки могли заставить нас подумать, что туда должно было быть помещено содержимое, находящееся по этому адресу, как бы это сделала инструкция MOV, но LEA просто поместила адрес в первый операнд, а не его содержимое.

XCHG (Exchange Register/Memory with Register)

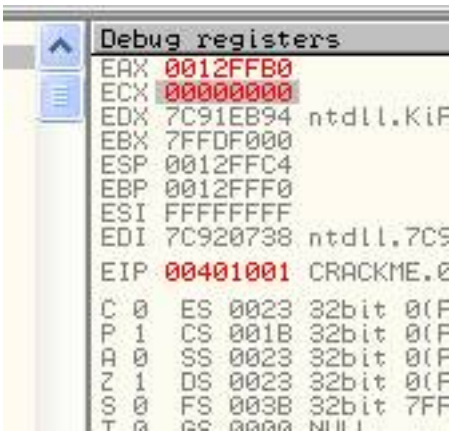
Эта инструкция обменивает содержимое двух операндов, например:

```
XCHG EAX,ECX
```

Значение EAX будет помещено в ECX и наоборот. Проверим это в Олли.

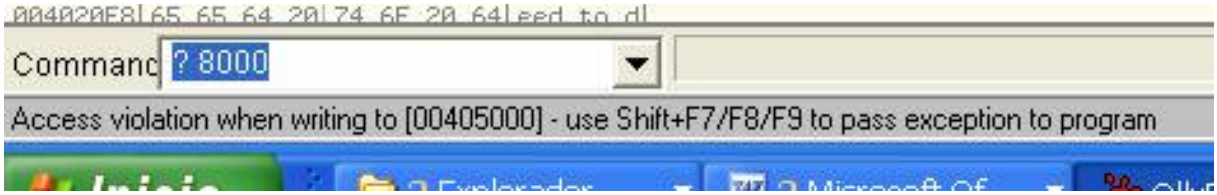
На моей машине до выполнения EAX содержал ноль, а ECX - 12FFb0.

Нажав F7, увидим, что они обменялись значениями.



Также можно использовать эту инструкцию для обмена значения регистра и ячейки памяти, если у нас есть разрешение на запись в данную секцию.

По нажатию на F7:



Также как и в примере с MOV было сгенерировано исключение, так как у нас нет прав на запись.

Ок, думаю, что перввая из частей, посвящённых инструкциям, была интересной и полезной, надеюсь, что примеры также достаточно хорошо поясняли предмет изложения. В следующей части мы продолжим изучение инструкций.

Введение в крэкинг с нуля, используя OllyDbg - Глава 5

Автор: Рикардо Нарваха, пер. Aquila

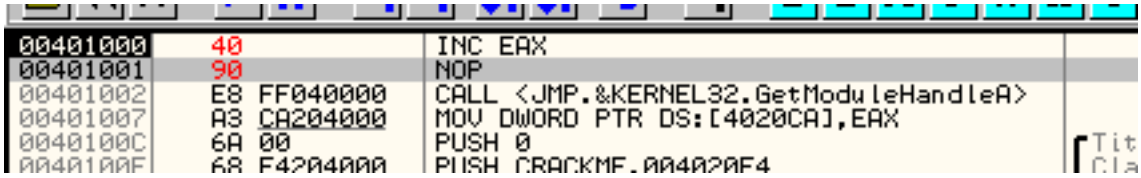
Дата: 2007-01-31

МАТЕМАТИЧЕСКИЕ ИНСТРУКЦИИ

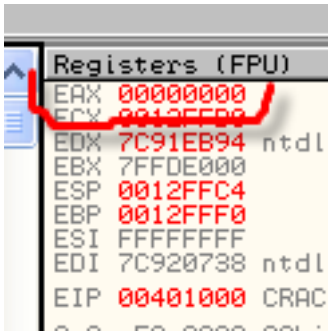
INC И DEC

Эти инструкции увеличивают или уменьшают значение операнда, прибавляя к нему один в случае с INC или, соответственно, отнимая один в случае с DEC.

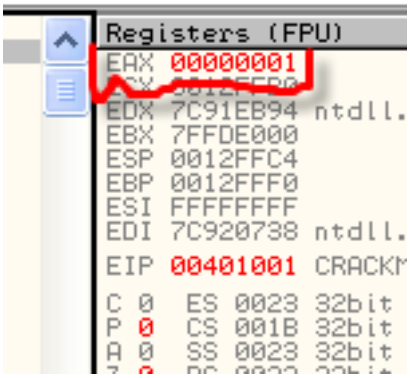
Идём в Олли и, как обычно, открываем крэкми CrueHead'и и в первой строке пишем:



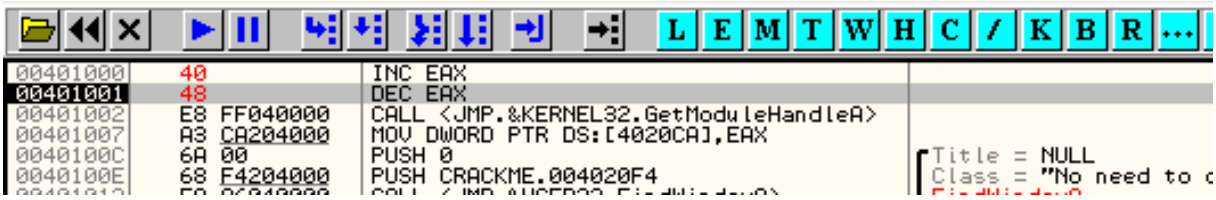
На моей машине EAX изначально равен нулю. Если у вас это не так, вы можете изменить это вручную.



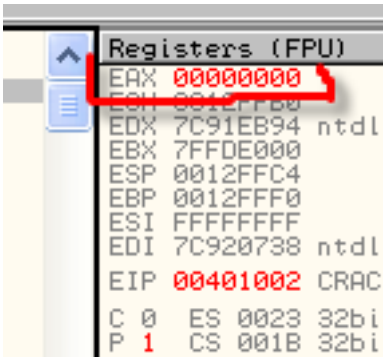
Затем нажимаем F7 и выполняем инструкцию INC EAX, увеличивая его на один.



Таким же образом можем ввести DEC сразу под предыдущей инструкцией.

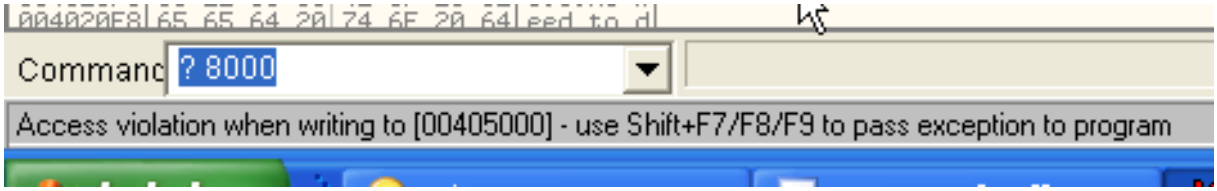


Нажимаем F7 и выполняем инструкцию INC EAX, которая уменьшает значение регистра на 1, после чего он принимает предыдущее значение - 0.

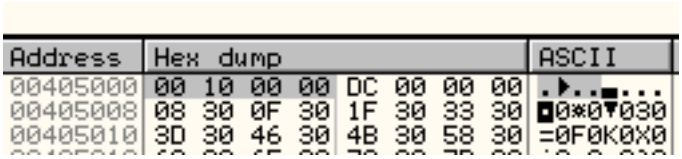


Также можно увеличивать или уменьшать значение ячейки памяти.

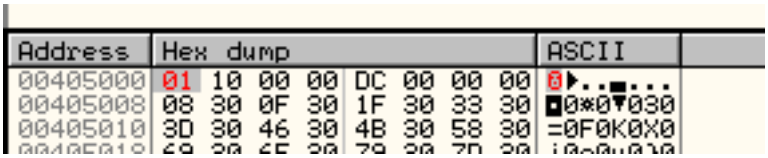
Хотя сейчас ничего не выйдет, так как у секции нет разрешения на запись, поэтому совершить данную операцию нельзя, в связи с чем генерируется исключение.



Что было бы, если бы у секции было выставлено разрешение на запись? Идём в DUMP по адресу 405000.

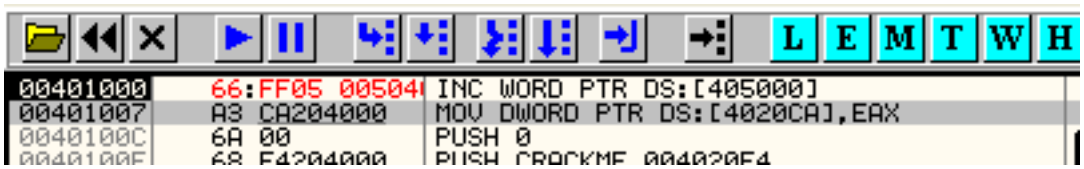


Содержимое, если его прочитать наоборот, будет равно 00001000, значит, если бы оно было увеличено, то было бы 00001001



В данном случае для DWORD прибавление 1 производилось к 4 байтам содержимого.

Для WORD суммирование будет применено только к двум последним байтам.



И при BYTE суммирование будет применено только к последнему байту.

ADD

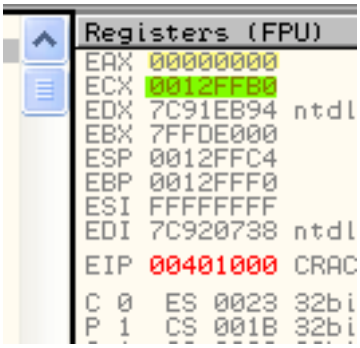
ADD, как можно видеть, суммирует оба операнда, сохраняя результат в первом.

ADD EAX, 1 похоже на INC EAX.

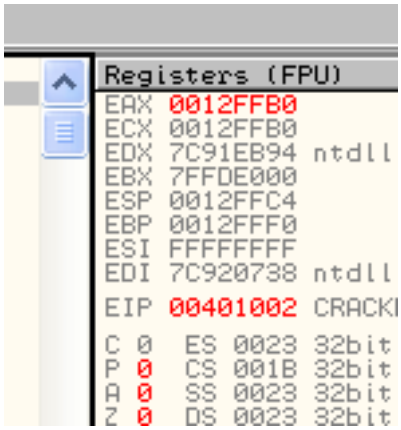
Также можем складывать регистры.

Идём в Олли.

До выполнения инструкции:

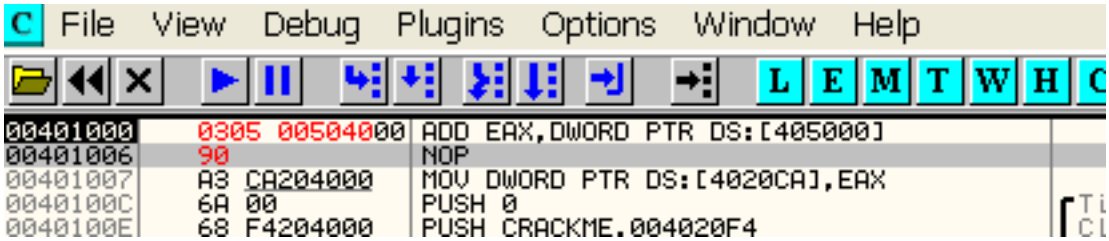


На моейм машине EAX содержит 00000000, а EAX - 12FFB0. У вас данные регистры могут содержать другие значения, также вы можете, если хотите, их изменить, но когда вы нажмёте F7, оба значения будут сложены, а результат помещён в EAX. Смотрим:



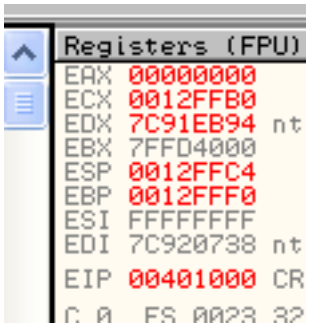
Здесь EAX стал красного цвета, потому что он был модифицирован и содержит результат суммирования.

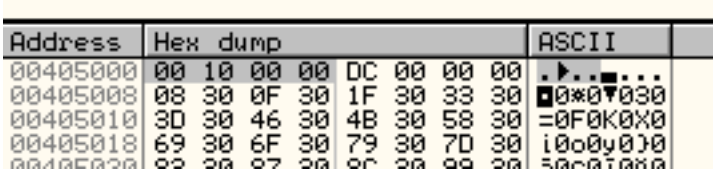
Также можем сложить регистр с содержимым ячейки памяти.



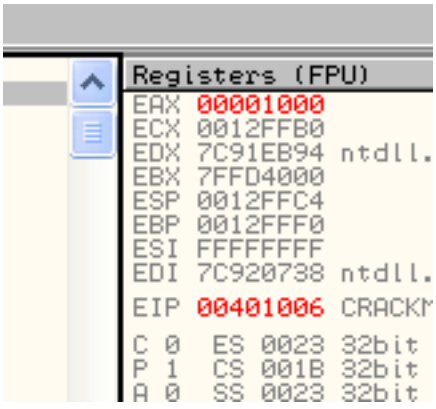
В данном случае нет проблем с правами на запись, так как меняется и сохраняет результат регистр EAX, а содержимое [405000] остаётся неизменным. Соответственно, не генерится и никакого исключения.

До нажания F7 видим, что EAX содержит 0, а по адресу 405000 находится значение 00001000.



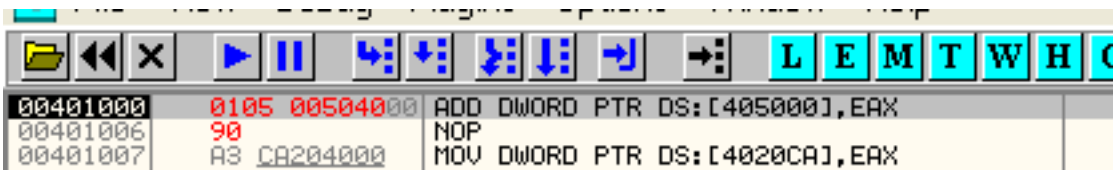


Нажимае F7, после чего вычисляется сумма.

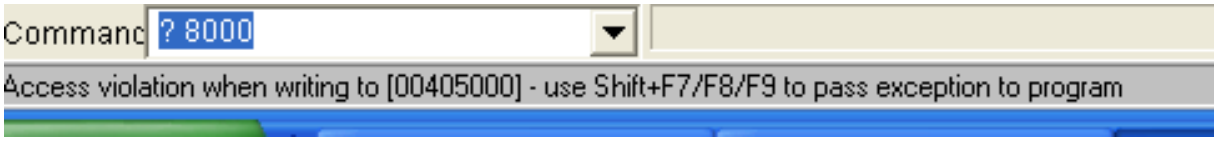


Так как EAX = 0, то в EAX сохраняется результат, равный 1000.

Если хотим, чтобы результат сохранился в памяти, то пишем:

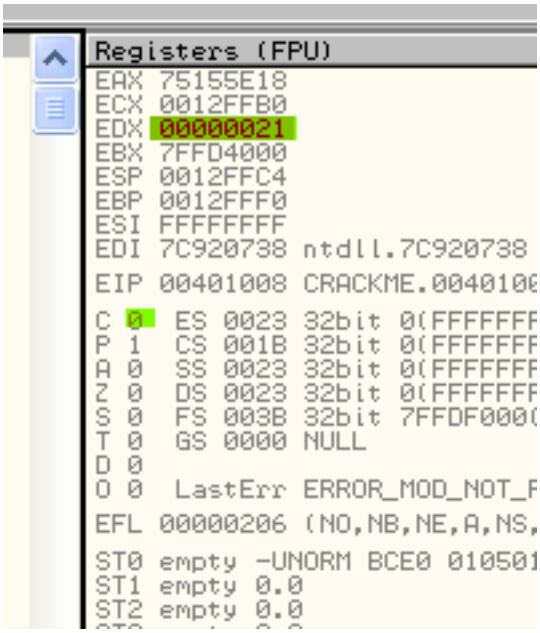


В этом случае результат сохранится в 405000, и так как это модифицирует память, то в нашем случае при нажатии на F7 будет сгенерировано исключение об отсутствии прав на запись.

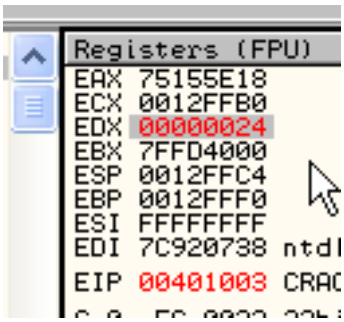


ADC (ADD С ПЕРЕНОСОМ)

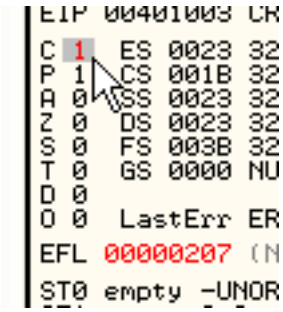
В данном случае вычисляется сумма обоих операндов и флага переноса, а конечный результат сохраняется в первом из операндов.



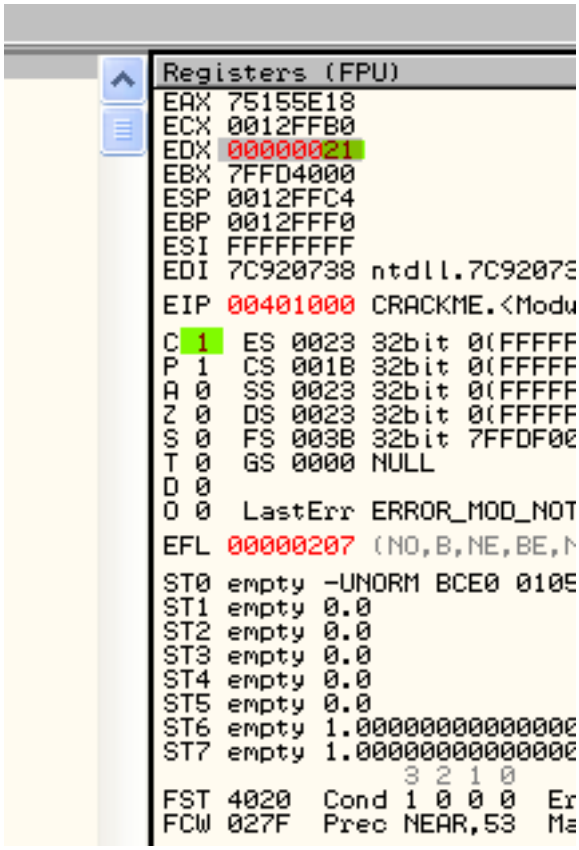
Здесь видим, что здесь будет суммирован EAX, содержащий 21, со числом 3, а также вычислен флаг C, который сейчас равен нулю. Нажимаем F7.



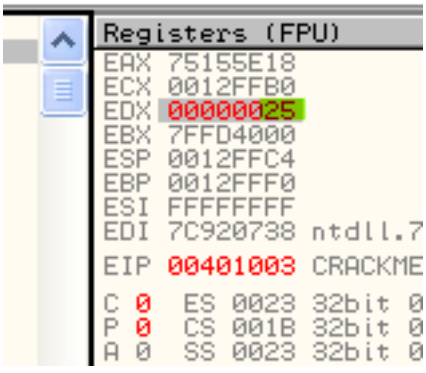
Видим, что результат равен 24. Теперь повторим эту операцию, но с флагом C равным 1, значение которого мы можем изменить, если дважды кликнем по нему мышью.



Сделав это, повторяем выполнение инструкции, изменив значение флага C.



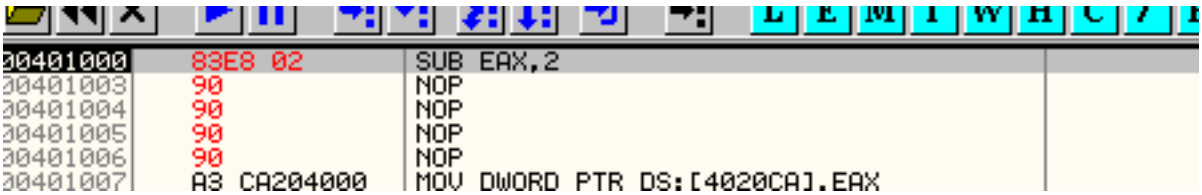
Нажимем F7 и вуаля - результат равен 25.



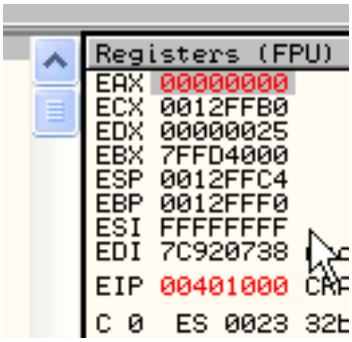
Так как сумма EDX (21) с 3 и флагом C, равном 1, равна 25.

SUB

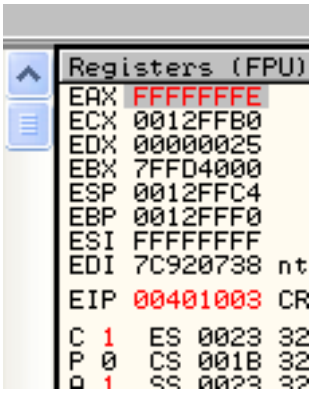
Это операция является обратной по отношению к ADD - она вычисляет остаток от вычитания второго операнда от первого и сохраняет в последнем полученный результат.



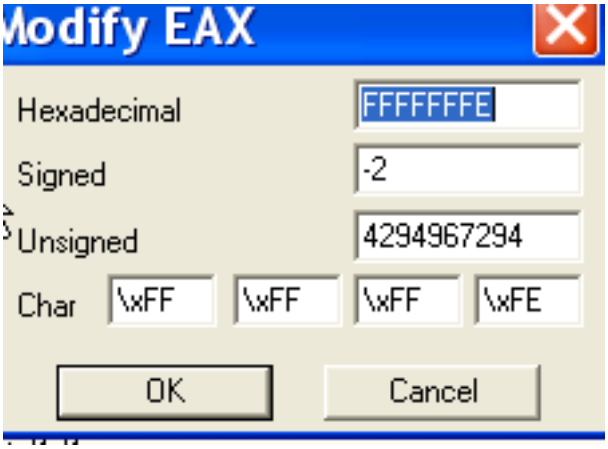
На моей машине регистры до выполнения инструкции содержали следующее:



По нажатию F7 инструкция вычтет от EAX, который содержит ноль, значение 2.



Результатом является -2, что в шестнадцатеричной системе выглядит как FFFFFFFE, в чём мы можем убедиться, если кликнем два раза мышью на данном значении:



Видим, что в десятеричной системе это равно -2.

Также можно отнимать значение регистров и ячеек памяти примерно таким же образом, как и при использовании инструкции ADD.

```
SUB EAX,ECX<
```

То есть EAX-ECX с сохранением результата в EAX.

И

```
SUB EAX,DWORD PTR DS:[405000]<
```

Вычитает от EAX содержимое ячейки памяти по адресу 405000, сохраняя результат в EAX.

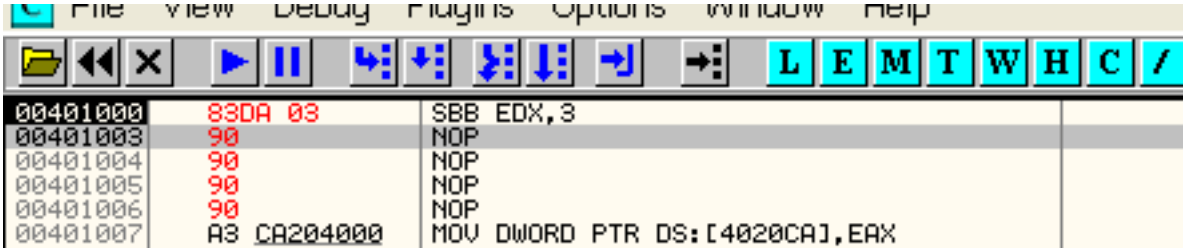
Обратный случай:

```
SUB DWORD PTR DS:[405000],EAX<
```

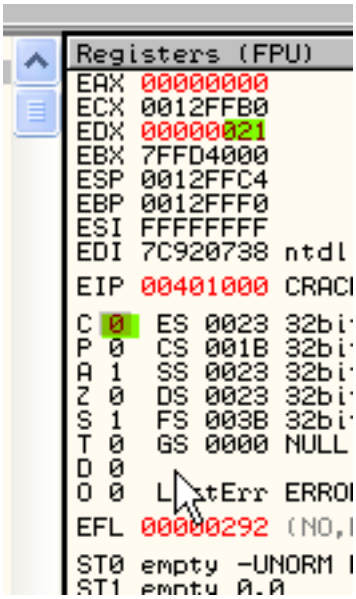
Так как результата сохраняется .в первый операнд, то, поскольку у нас нет прав на запись в секцию, то получим исключение.

SBB

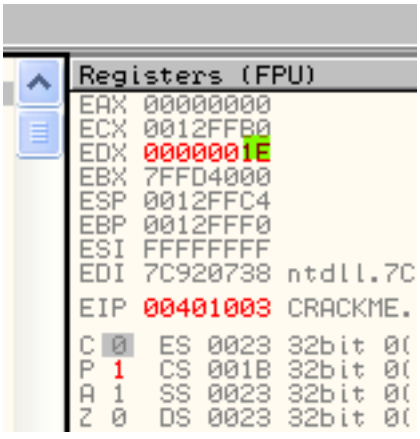
Это операция является обратной к ADC: она вычисляет разность между операндами, отнимает от того, что получилось, значение флага переноса (флаг C) и сохраняется окончательный результат в первый операнд.



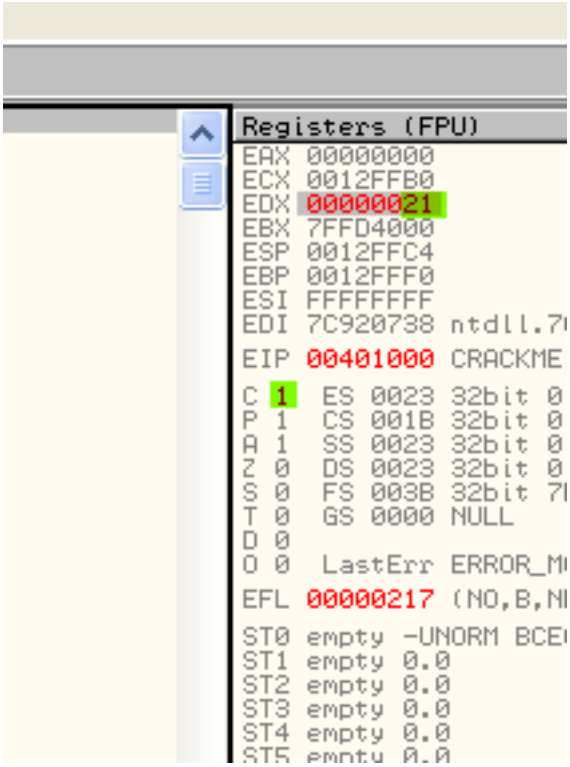
До выполнения в EDX находилось 21, а флаг переноса был равен 0.



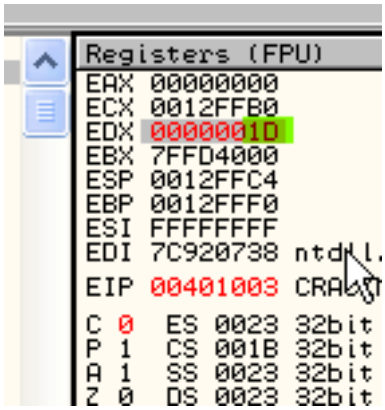
Нажатие на F7 вычтет из EDX число 3, а затем ноль, которому равен флаг C.



Теперь повторим операцию, присвоив флагу C значение 1.



Нажимает на F7 и теперь из EDX вычитается 3, а затем единица, которой равен флаг C.



В данном случае результат равен 1D.

MUL

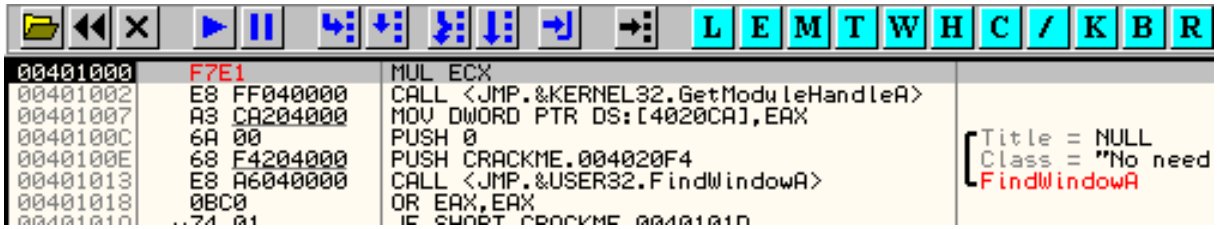
Ок, есть две инструкции для выполнения умножения. Первой из них является MUL, которая не учитывает знаки чисел. Она использует только один операнд, а другим всегда является EAX Результат сохраняется в EDX:EAX.

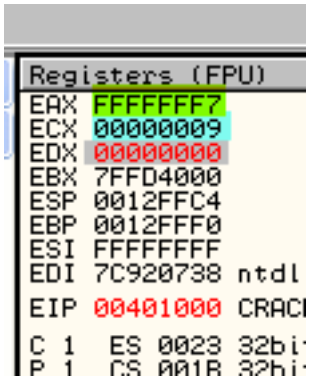
Например:

```
MUL ECX<
```

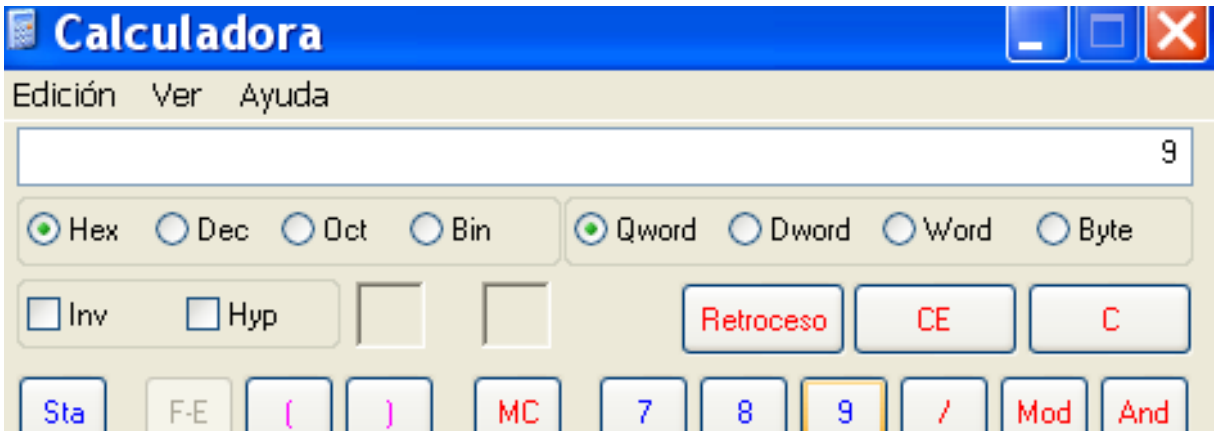
Здесь ECX умножается на EAX без учёта знаков их значений, а результат сохраняется в EDX:EAX.

Рассмотрим следующий пример с помощью OllyDbg:

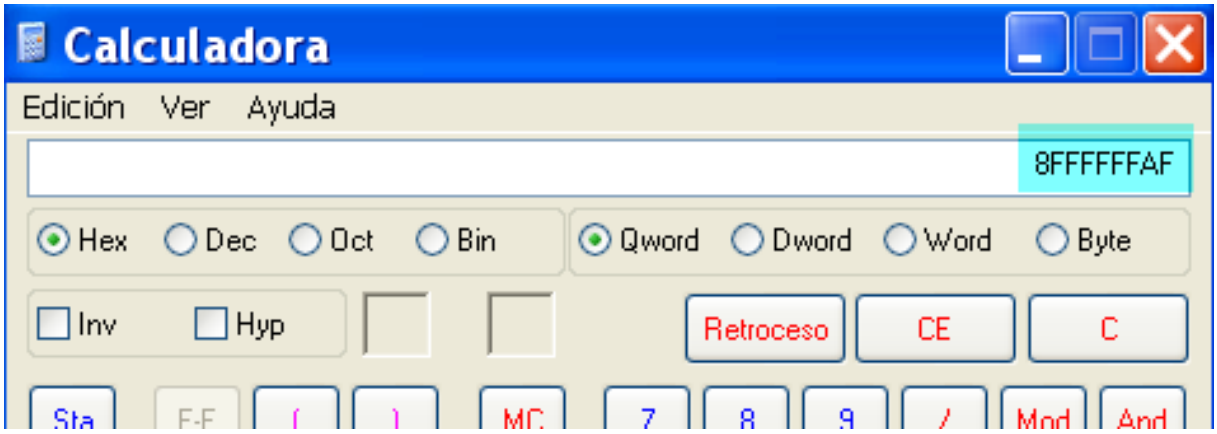




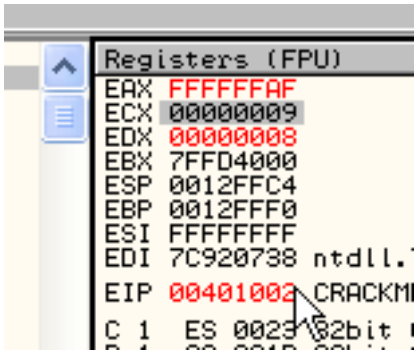
Поместим в EAX число FFFFFFF7, в ECX - 9, а затем посчитаем результат с помощью стандартного калькулятора Windows:



Получится:



Это не поместится в EAX. Посмотрим, что будет в Олли и нажмём на F7.



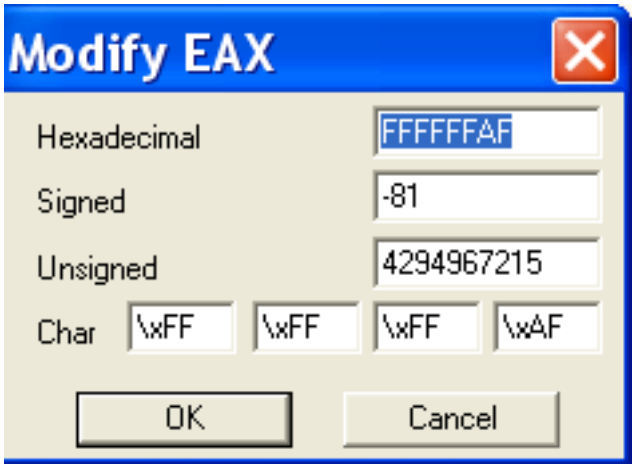
EDX и EAX выделены красным, так как их значение было изменено, и, как видим, в EAX было сохранено то, что поместилось, а остальное в EDX. В данном случае 8-рка, которую не смог вместить EAX, попала в EDX, поэтому в таких случая говорят, что результат содержится в EDX:EAX, то есть два регистра используются как один двойного размера.

В случае с

```
MUL DWORD PTR DS:[405000]<
```

Происходит умножение ячейки памяти по адресу 405000 на EAX, а результат, как обычно, помещается в EDX:EAX без учёта знаков операндов.

Чтобы посмотреть в Олли как выглядит какое-либо шестнадцатеричное значение без учёта знака, кликните два раза мышью на любом регистре.



Как можно видеть, во втором ряду находится значение со знаком (в данном случае FFFFFFAF это -81, если учитывать знак), но в таких операциях как MUL все числа считаются БЕЗЗНАКОВЫМИ. Третья строка как раз и содержит десятеричное значение без знака. Сейчас видим, что число FFFFFFAF - это 4294967215, если рассматривать его как положительное, то есть беззнаковое.

IMUL (умножение без знака)

Инструция IMUL - это умножение со знаком и употребляется аналогично MUL.

IMUL ECX

Данная операция производит умножение ECX на EAX, а результат сохраняется в EDX:EAX с учётом знака операндов.

Несмотря на сходство с предыдущей инструкцией, IMUL позволяет использовать больше одного операнда, что невозможно в случае с MUL.

Из туториала CAOS'a:

> *Несмотря на использовальние регистров EAX и EDX по умолчанию, можно указать другие источники и приёмники данных вплоть до трёх операндов. Первый - место, куда необходимо поместить результат и которое всегда должно быть регистром, второй и третий - два значения, которые нужно перемножить. В этих примерах увидим применение этой инструкции с двумя или тремя операндами.*

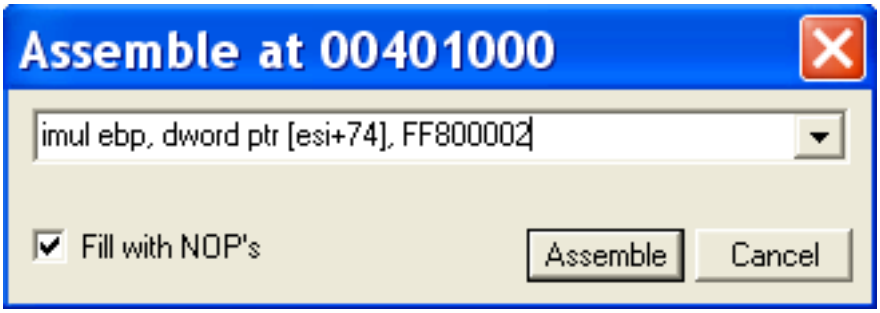
```
F7EB imul ebx     EAX x EBX -> EDX:EAX<
```

Это первый пример, который похож на MUL, не считая того, что здесь учитываются знаки.

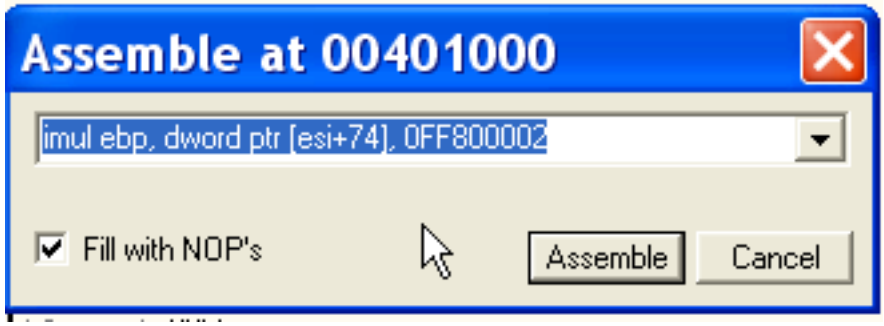
696E74020080FF imul ebp, dword ptr [esi+74], FF800002 [ESI+74] x FF800002 -> EBP

Это второй пример, в котором три операнда: содержимое ESI+74 умножается на FF800002, а результат сохраняется в EBP. Мы можем посмотреть его в действии с помощью Олли.

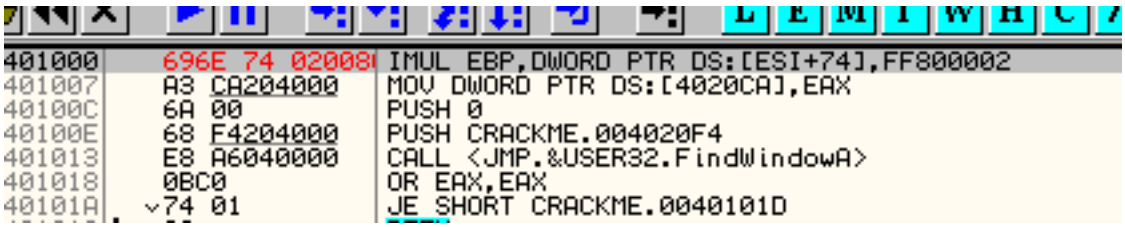
Скопируем строку "imul ebp, dword ptr [esi+74], FF800002" в Олли.



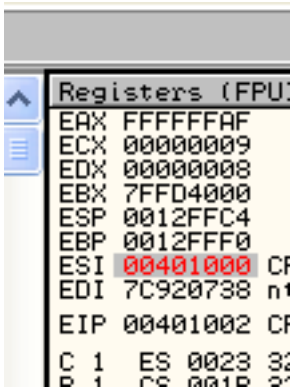
Видим, что нажатие кнопки "ASSEMBLE" выдаёт ошибку. Это происходит из-за того, что в Олли к числам, которые начинаются с букв, нужно слева приписывать ноль. Исправляем:



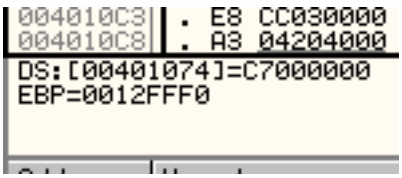
Теперь нажатие на ASSEMBLE проходит без ошибок.



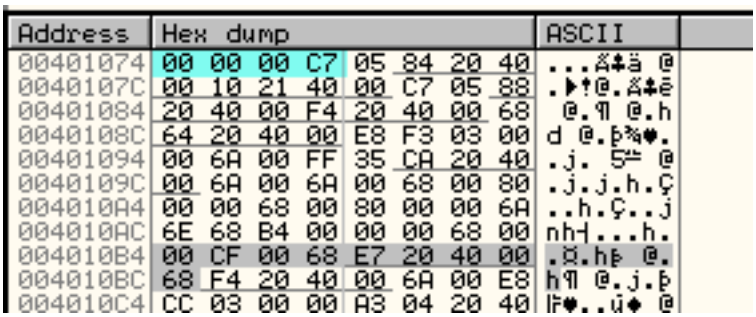
Изменяем значение ESI на 401000, чтобы быть уверенными, что адрес ESI + 74, из котого будет происходить чтение, существует.



Видим в пояснениях:



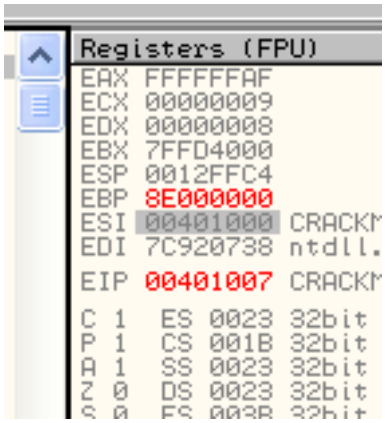
Здесь говорится, что ESI + 74 - это адрес 401074, содержимым которого является C7000000. Идём в DUMP с помощью "GOTO EXPRESSION 401074".



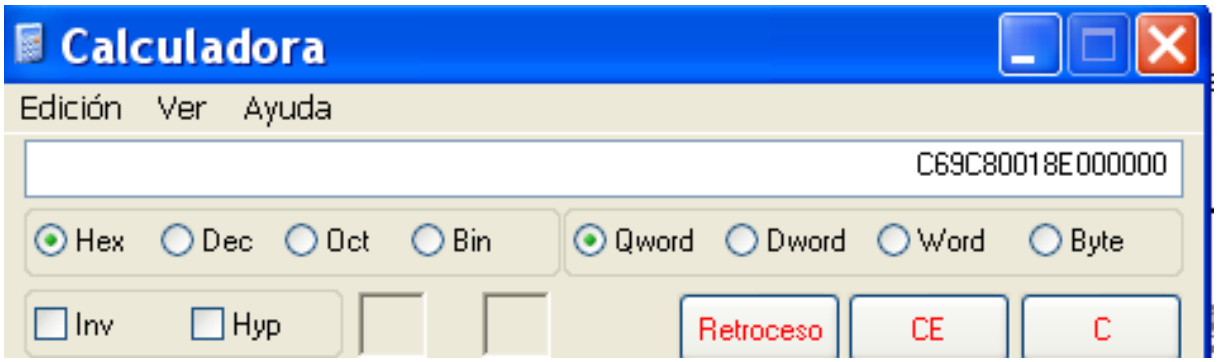
Это значение, которое, если прочитать его наоборот, равно C7000000, будет умножено на FF800002, а результат данной операции будет сохранён в EBP.

```
imul ebp, dword ptr [esi+74], FF800002<
```

Нажав F7, видим, что EBP стал красным, так как сейчас он содержит результат.



Калькулятор даёт нам следующий результат операции C7000000 * FF800002:



Но поскольку EBP не может вместить его полностью, то в нём остаётся только то, что поместилось, а остальное отбрасывается.

В третьем примере только два операнда, оба из которых перемножаются, а результат сохраняется в первый из них.

```
0FAF55E8            imul edx, dword ptr [ebp-18]  EDX x [EBP-18] -> EDX<
```

Как видим, основной способ для умножения больших чисел - использовать IMUL только с одним операндом, так как в этом случае результат сохраняется в EDX:EAX, то есть он может быть двойного размера, чего нельзя добиться при использовании двух или трёх операндах, соответственно, данную возможность следует применять для маленьких чисел.

DIV (Unsigned Divide) / IDIV (Signed Divide)

Это инструкции, обратные по отношении к MUL и IMUL соответственно.

DIV имеет только один операнд, не учитывает знаки и сохраняет результат в EDX:EAX.

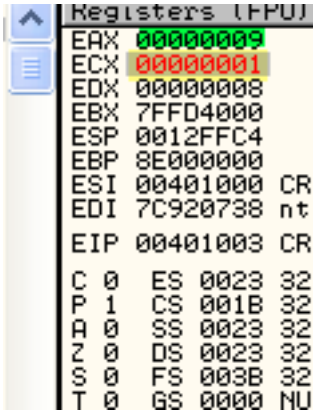
IDIV всегда учитывает знаки. Если используется один операнд, то, как и DIV, сохраняет результат в EDX:EAX, в случае двух - делит один на другой и сохраняет результат в первый, а при трёх операндах делит второй на третий и сохраняет то, что получилось, в первый.

Не думаю, что нужно приводить здесь примеры, похожие на те, что были в разделе, посвящённом MUL и IMUL.

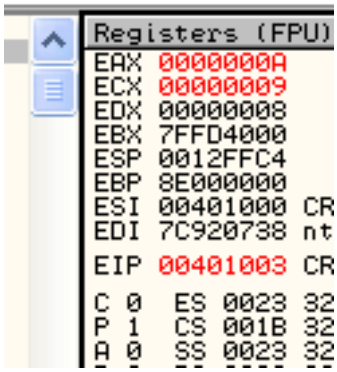
XADD (Обменять и сложить)

Как можно догадаться, это смесь XCHG и ADD, то есть если :

```
XADD EAX,ECX<
```



Видим, что ECX равен 1, а EAX - 9. По нажатию на F7 они обменяются значениям, т.е. в EAX попадёт 1, а в ECX - 9, после чего произойдёт сложение.



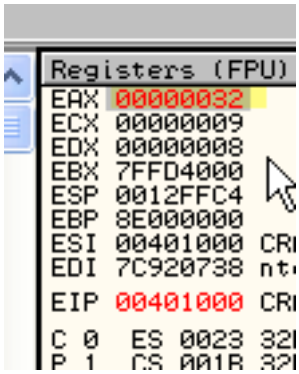
Видно, что результат затем был сохранён в первый операнд. Регистр ECX содержит число 9, которое находилось до выполнения операции в EAX.

NEG

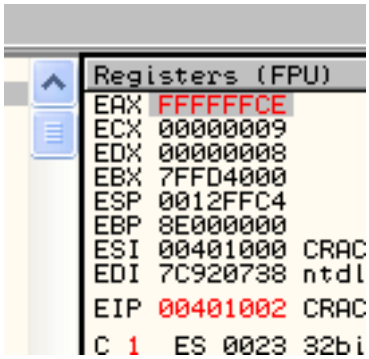
Назначением данной инструкции является менять знак заданного числа на противоположный, т.е. если у нас есть число 32 в шестнадцатеричной системе, и мы применяем к ней NEG, то результатом будет его отрицательный аналог.

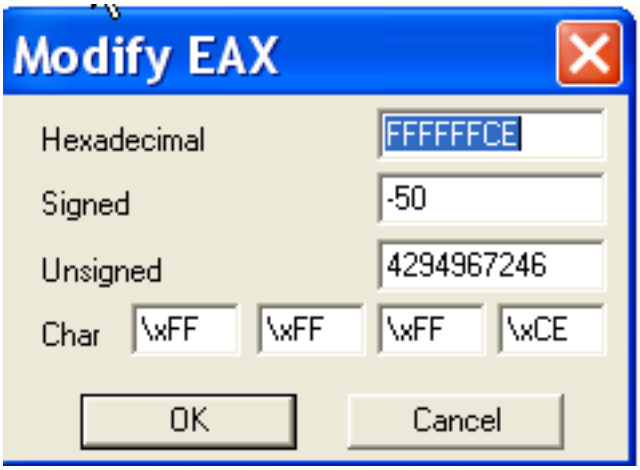
Пример:

Пишем в Олли NEG EAX и заносим в EAX число 32.

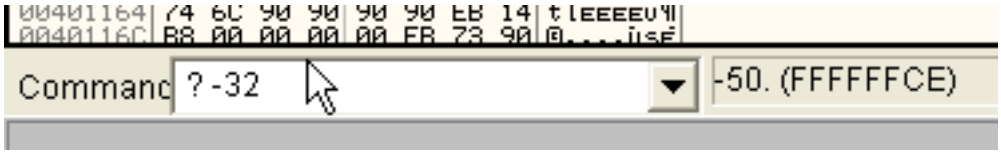


После нажатия на F7 в EAX окажется отрицательный вариант числа 32. Смотрим:





Во втором ряду видим, что результатом является -50 в десятеричной системе (это соответствует -32 в шестнадцатеричной).



Если напечатаем в CommandBar'e число -32 в шестнадцатеричной системе, чтобы выяснить, чему оно равно в десятеричной, то получим ответ, что это значение эквивалентно -50, а также пояснение, что это FFFFFFCE.

Так что видим, что NEG меняет знак операнда на обратный.

ЛОГИЧЕСКИЕ ИНСТРУКЦИИ

Далее следуют логические инструкции, принимающие по два операнда, производящие над ними побитовые операции и сохраняющие результат в первый операнд.

AND

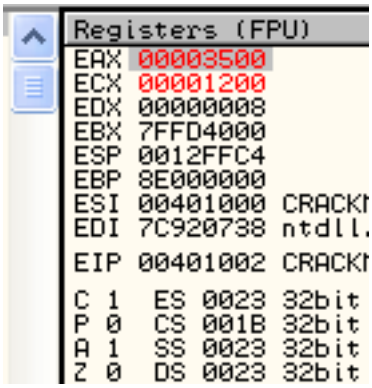
Результатом этой операции над двумя битам является 1, если они оба равны 1, и 0 во всех остальных случаях.

• 1 and 1 = 1
• 1 and 0 = 0
• 0 and 1 = 0
• 0 and 0 = 0

Посмотрим пример в OllyDbg:

```
AND EAX,ECX<
```

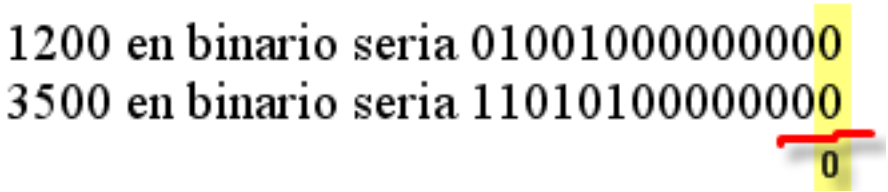
Делаем ECX=00001200, а EAX=3500.



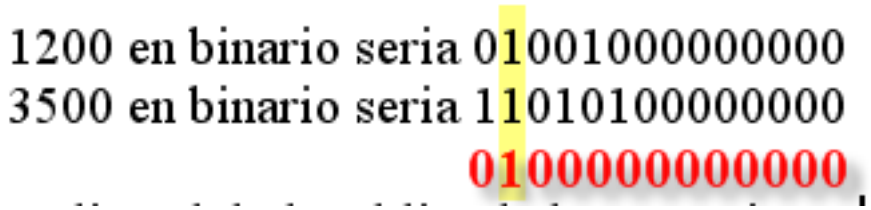
Чтобы сделать эту операцию вручную, нужно перевести оба числа в двоичный вид:

• 1200 в двоичном виде будет равно 01001000000000
• 3500 в двоичном виде будет равно 11010100000000

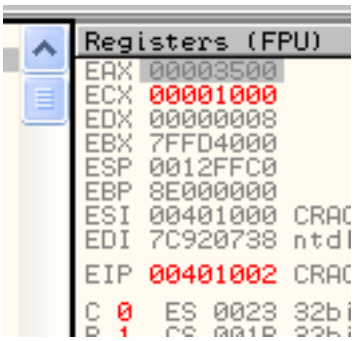
Применим вышеприведённую таблицу операции AND побитно, например, к последней цифре:



Из двух нулей получится ноль. Повторим эту процедуру над всеми битами и получим 01000000000000, поскольку ситуация, когда оба биты равны 1 у нас встречается только один раз, а посему и в результате единица встретится тоже только один раз.



Если нажмём F7 в Олли, то увидим, что в ECX находится результат 1000, что эквивалентно 01000000000000 в двоичном виде.



OR

В этой инструкции используется примерно тот же метод, что и в предыдущем, за исключением того, что используется другая таблица.

Получается 1, если один или оба бита равны 1, и 0 во всех остальных случаях.

• 1 or 1 = 1
• 1 or 0 = 1
• 0 or 1 = 1
• 0 or 0 = 0

XOR

Здесь то же самое, только используем таблицу функции XOR.

Получается 1, если один и только один из двух битов равен 1, и 0 во всех остальных случаях.

• 1 xor 1 = 0
• 1 xor 0 = 1
• 0 xor 1 = 1
• 0 xor 0 = 0

NOT

Просто инвертирует значение:

• not 1 = 0
• not 0 = 1

Пример: not 0110 = 1001

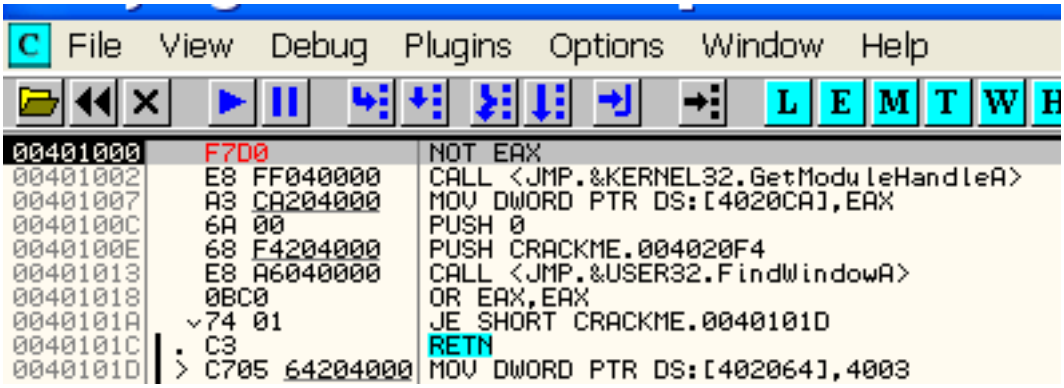
Если у нас, например, EAX=1200, что в двоичном виде равно 1001000000000, то все нули превратятся в 1, а все 1 в 0.

```
00000000000000000001001000000000<
```

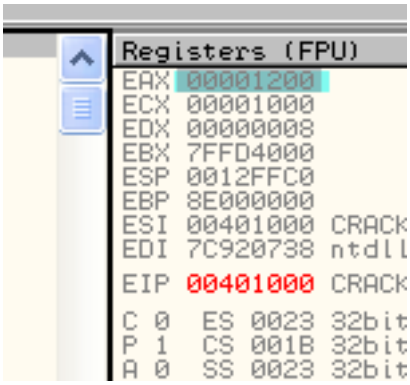
После применения NOT получим

```
11111111111111111110110111111111<
```

что в калькуляторе Windows в шестнадцатеричном виде отобразится как FFFFEDFF.



В Олли до нажатия на F7 сделаем EAX равным 1200.



Нажимаем F7.

Видим, что результат тот же самый.

Ок, здесь мы закончим пятую часть "Введения...". Пока подумайте над тем, что узнали в этой главе, а нам ещё предстоит рассмотреть инструкции сравнения, выходов, call и ret.

Шаг за шагом дорога приведёт нас к цели.

Введение в крэкинг с нуля, используя OllyDbg - Глава 6

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2007-03-18

СРАВНЕНИЯ И УСЛОВНЫЕ ПЕРЕХОДЫ

Как правило, у инструкции сравнения есть два операнда, а результатом этого сравнения является решение, должна или нет программа совершать переход, указанный далее в коде.

Мы узнаем, что когда программа просит у нас серийный номер для регистрации, то в этот момент она должна решить, правилен он или нет, а для этого ей нужно провести одно или более сравнений и совершить соответствующие результаты перехода.

Далее мы рассмотрим подробно, как работают сравнение и переход.

Также мы узнаем, что, согласно изученному нами в предыдущих версиях данного «Введения...», когда в зависимости от результата выполнения инструкции меняются те или иные флаги, наиболее частым случаем является ноль в флаге Z. Также мы рассмотрим другие изменения во флагах, связанных с выполнениями тех или иных инструкций.

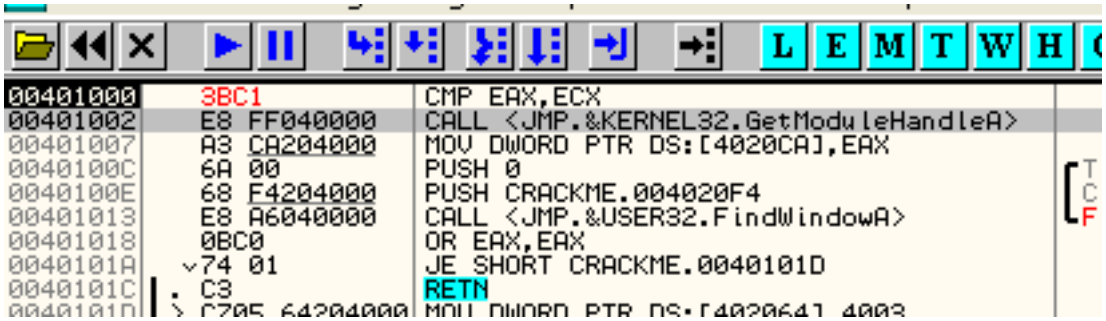
CMP

Это наиболее известная инструкция сравнения. Она принимает два операнда и, на самом деле, работает также как и SUB, только не сохраняет результат в первый операнд. В обоих из них остаётся то же значение, что и прежде, а изменяются только флаги согласно результату.

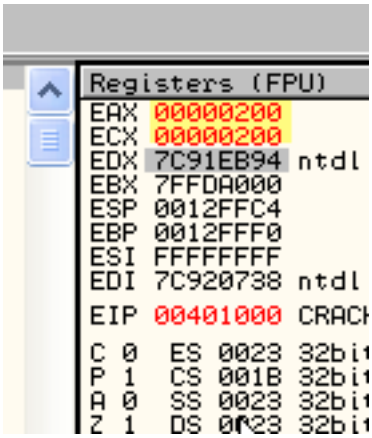
Рассмотрим пример:

```
CMP EAX, ECX
```

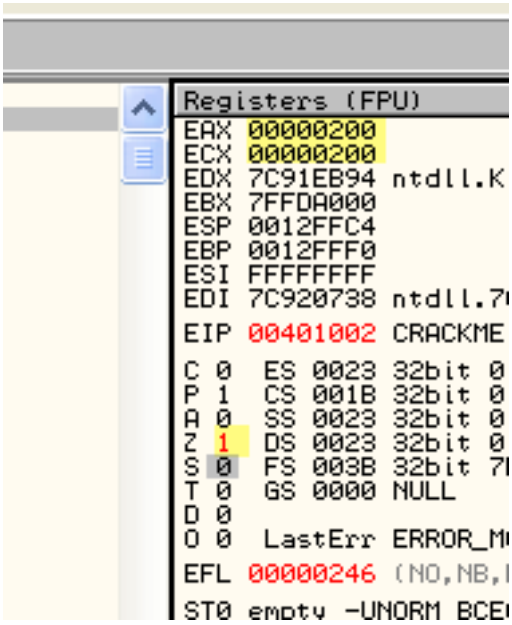
Пусть EAX и ECX равны, они будут вычтены друг из друга, значение их не изменится, но результатом операции станет то, что активируется флаг Z. Рассмотрим пример в OllyDbg.



Вводим инструкцию и модифицируем EAX и ECX так, чтобы они были равны.



Нажимаем F7 и видим, что значения EAX и ECX не изменились, но зато стал активным флаг Z, так как результатом вычитания одного операнда от другого был ноль, хотя мы и не можем увидеть его напрямую или сохранить куда-либо.



В реальности нам не важен точный результат вычитания, а только равен ли EAX ECX'у или нет (что чаще).

Хотя мы ещё и не дошли до самих условных переходов, есть две возможности: согласно значению флагов решить, делать переход или не делать. Самый понятный пример, который работает в сочетании с предыдущим – это инструкция JZ, которая совершает переход, если флаг Z активен, т.е. равен ЕДИНИЦЕ, и не совершает, если он не активен, т.е. равен НУЛЮ.

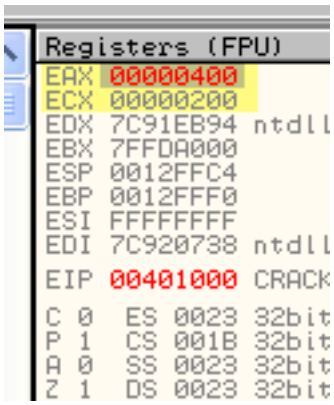
Таким же образом программа может вынести решение относительно двух серийных номеров, один из которых интересует вас на предмет регистрации программы и содержится, скажем, в EAX, а второй находится в EAX и является правильным серийным номером. Программа может определить с помощью CMP, являются ли они равными, и если да (флаг Z будет активным), совершить переход с помощью JZ в ту область, где происходит регистрация. А если введённый вами серийный номер, находящийся в EAX, и правильный в ECX окажутся не равны, то флаг Z будет содержать НОЛЬ, и, соответственно, никакого перехода не произойдёт.

Давайте рассмотрим более конкретные примеры условных переходов.

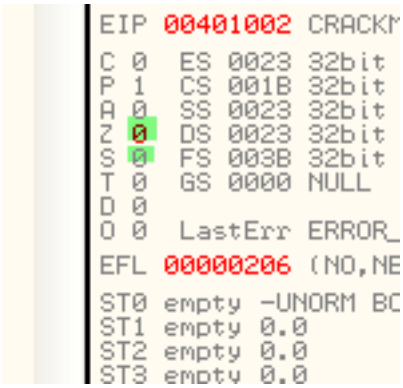
Точно так же флаг S или флаг знака предоставляет возможность сравнения, является ли первый операнд больше второго или наоборот.

Рассмотрим пример.

Повторим «CMP EAX, ECX», но в этот раз сделаем EAX больше ECX.

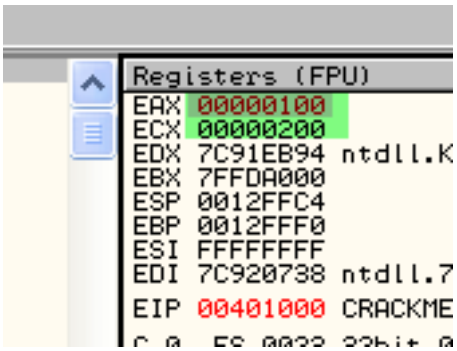


Если нажмём F7:

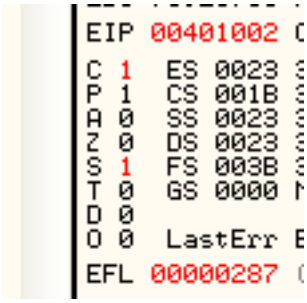


Видим, что флаг Z равен нулю, потому что, как мы уже знаем, значения не равны, а исходя из того, что флаг S тоже равен нулю, можно опеределить, что остаток от EAX-ECX является положительным, то есть EAX больше ECX.

Таким же образом повторим предыдущую операцию, но EAX меньшим чем ECX.

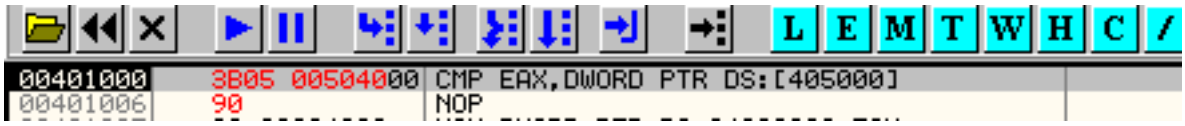


И нажимаем F7.

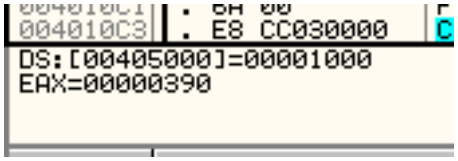


Здесь видим, что остаток от вычитания ECX от EAX отрицателен, то есть значение в ECX больше, что активизирует флаг S.

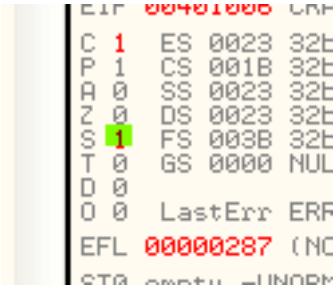
Различные результаты сравнения активизируют соответствующие флаги , и, соответственно им, программа может переходить или не переходить при активации одного или больше из этих флагов.



Здесь сравниваем EAX с содержимым 405000, и, как обычно, окошко с пояснениями нам даёт значение каждого из операндов. В моём случае они следующие:



EAX в данном примере меньше чем содержимое ячейки памяти по адресу 405000, которое равно 1000, что приведёт к отрицательному остатку и активирует флаг S.



Существуют похожие формы

```
CMP AX,WORD PTR DS:[405000]
```

и

```
CMP AL,BYTE PTR DS:[405000]
```

В этих случая сравниваются 2 или 1 байт содержимого памяти соответственно.

TEST (Логическое сравнение)

Эта инструкция работает похожим, до некоторой степени, образом, что и CMP – производится определённая операция над двумя значениями, и результат её не сохраняется, однако могут измениться состояния определённых флагов (в данном случае SF, ZF и PF), в следствии чего программа может определить, следует или нет совершать переход. Отличие от предыдущей инструкции заключается в том, что пресловутой операцией является AND.

Это нам рассказал наш друг CAOS в своё туториале по ассемблеру. Рассмотрим несколько примеров, чтобы уяснить данное определение:

```
TEST EAX,EAX
```

Вы спрашиваете зачем сравнивать с самим собой EAX или любой другой регистр? Для того, чтобы определить равен ли EAX нулю. Как это работает?

Пишем в Олли:

```
TEST EAX,EAX
```

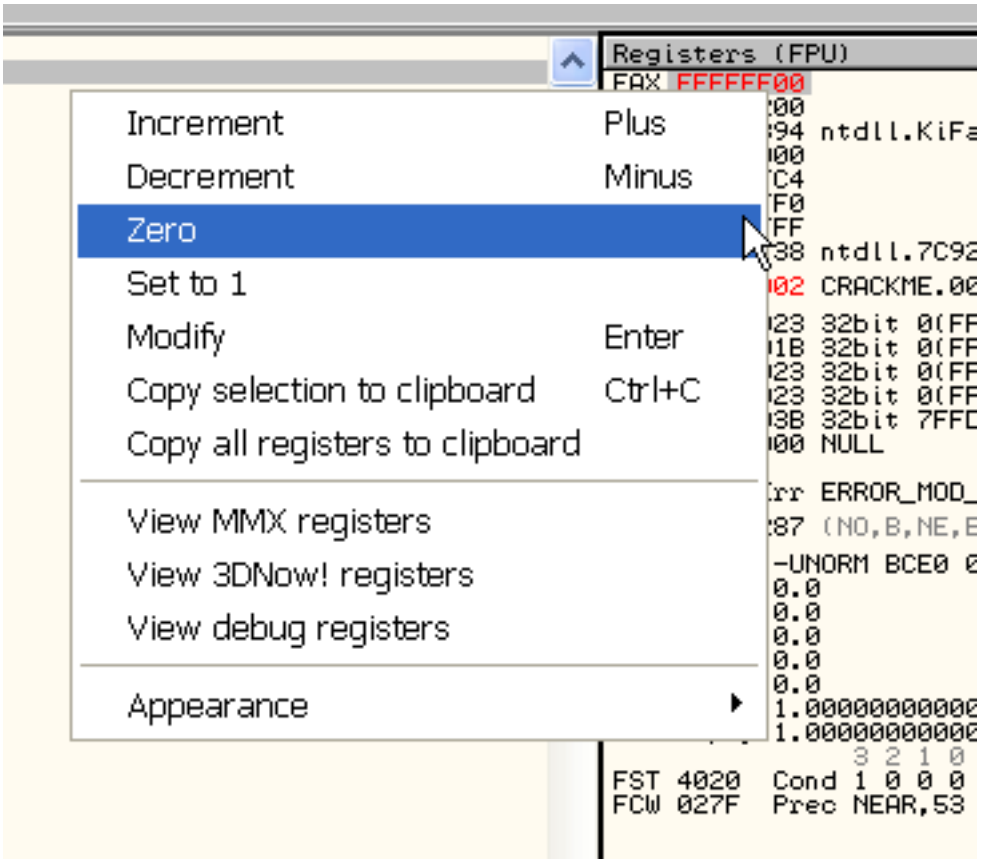
Таблица результатов для операции AND следующая:

Результать равен 1, если оба операнда равны 1, и 0 во всех остальных случаях.

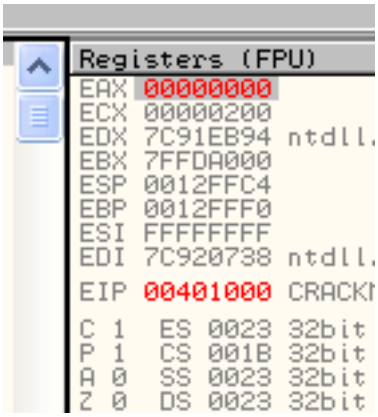
```
1 and 1 = 1
1 and 0 = 0
0 and 1 = 0
0 and 0 = 0
```

Видим, что единственный случай, когда результат равен нулю, это если оба операнда равны нулю (нас не интересуют случаи, когда у операндов разные значения, так как мы сравниваем EAX с самим собой, поэтому они всегда будут равны), соответственно, если у EAX в двоичной форме какой-нибудь из битов равен 1, то результатом операции никак не может быть ноль.

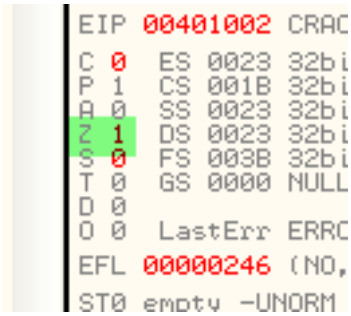
Сделаем EAX равным НУЛЮ:



В OllyDbg это делается очень легко правым кликом мыши на нужном регистре и вводом НУЛЯ.

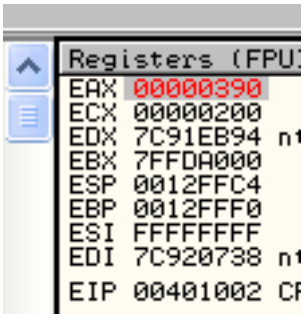


Теперь нажимаем F7.

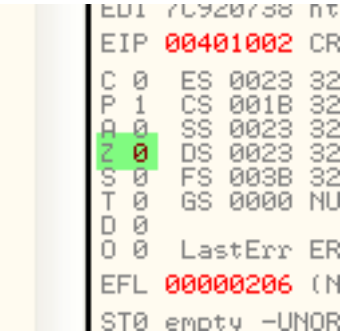


Видим, что активен флаг Z, то есть знаем, что результатом операции AND был НОЛЬ, в виду чего он и активировался.

Если повторим эту операцию на EAX со значением, отличающимся от нуля:



Нажимем F7.



И флаг Z не является активным, так как результат отличается от нуля.

Если используем калькулятор, то можем вычислить, что результатом 390 AND 390 будет 390.

В двоичной форме 390 – это 1110010000.

```
1110010000
1110010000
----------
1110010000
```

Так как операция AND, если оба операнда равны НУЛЮ в качестве результата также выдаст НОЛЬ, а если они оба равны ЕДИНИЦЕ, то результатом будет НОЛЬ, то видим, что в случае с 390 результат будет 390 и флаг Z не станет активным.

Это была большая часть инструкций сравнения, а теперь настал момент перейти к переходам.

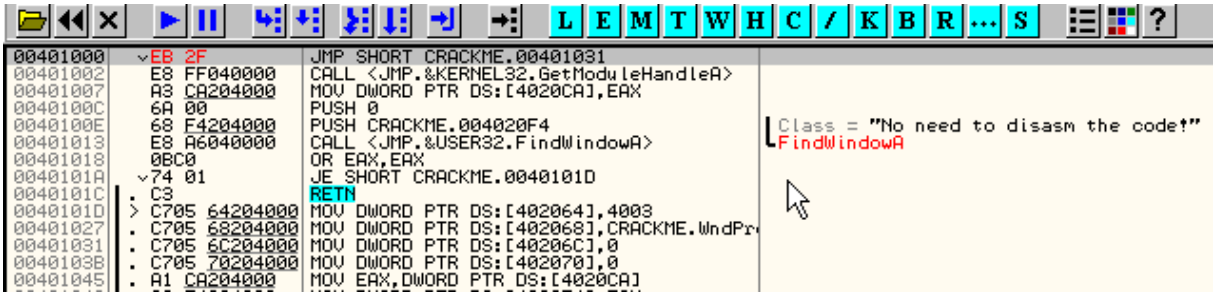
ПЕРЕХОДЫ

Все инструкции переходов принимают один операнд, задающий адрес, на который должна перейти программа. Различие между ними мы увидим в следующем списке.

• JMP – переход
• JE, JZ – переход, если равно нулю
• JNE, JNZ – переход, если не равно нулю
• JS – переход, если знак отрицателен
• JNS – переход, если знак не отрицателен
• JP, JPE – переход, если чётно
• JNP, JOP – переход, если нечётно
• JO – переход, если произошло переполнение
• JNO – переход, если переполнения не произошло
• JB, JNAE – переход, если ниже
• JNB, JAE – переход, если выше или равно
• JBE, JNA – переход, если ниже или равно
• JNBE, JA – переход, если выше
• JL, JNGE – переход, если меньше
• JNL, JGE – переход, если больше или равно
• JLE, JNG – переход, если меньше или равно
• JNLE, JG – переход, если больше

JMP

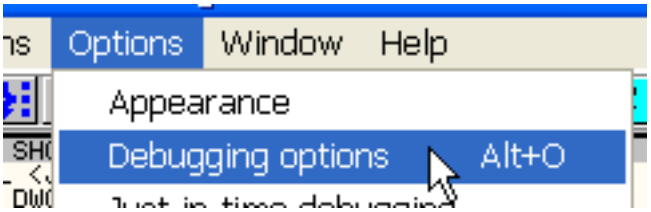
Это безусловный переход, то есть никакого решения не делается, и всегда происходит переход по заданном адресу. Например, напишем в Олли:



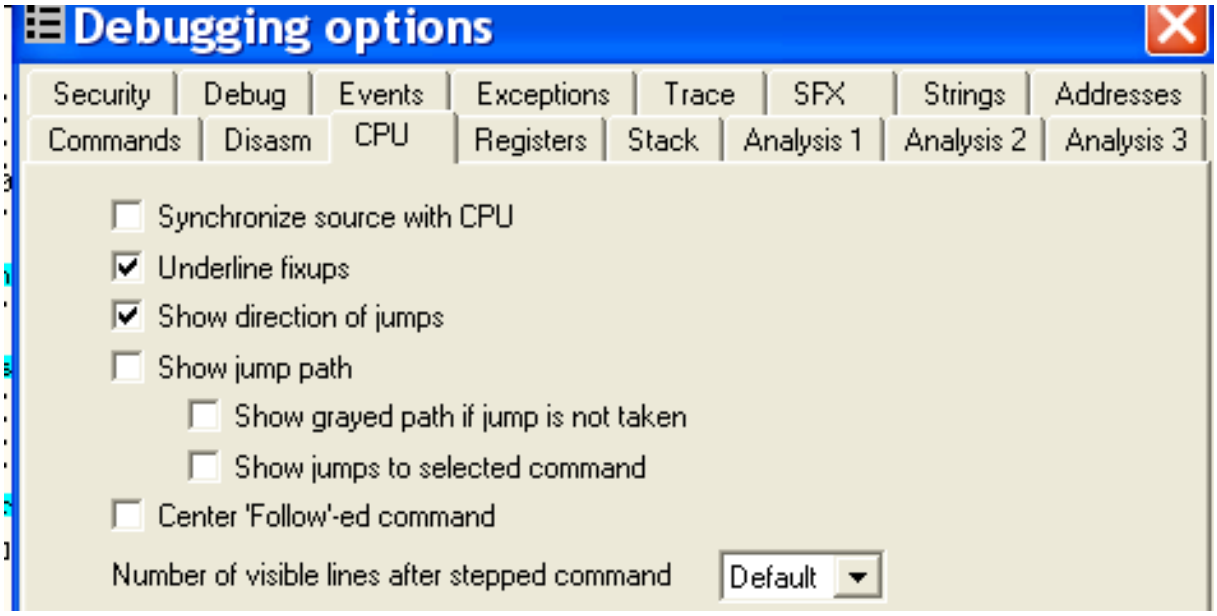
При выполнении этой инструкции программа перейдёт в 401031 и продолжит выполнение уже оттуда.

В Олли есть пара опций, которые позволяют сделать переходы более наглядными.

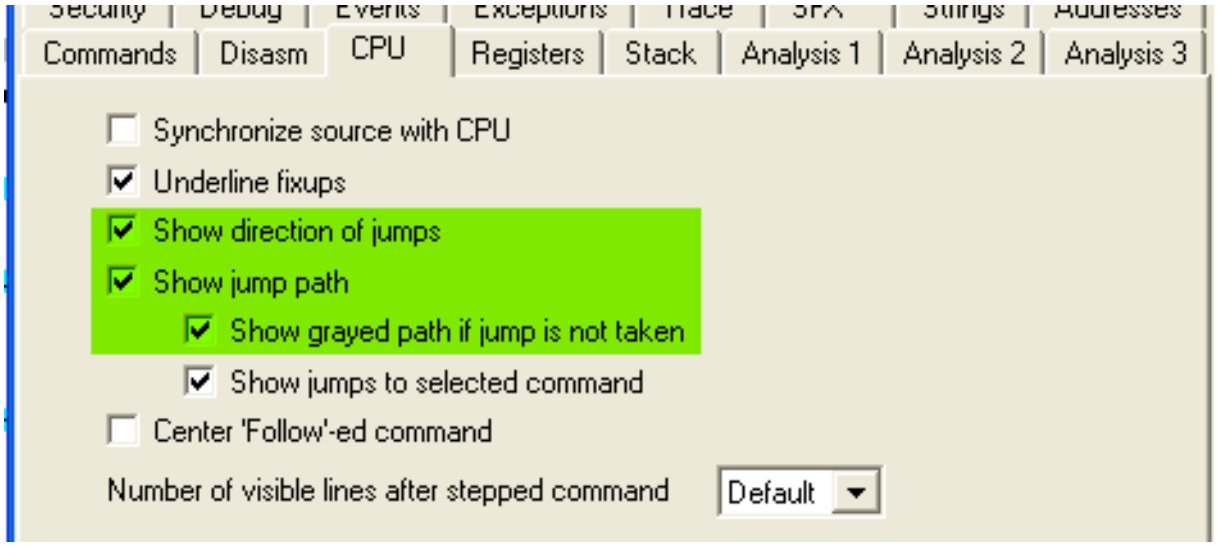
Если зайдём в OPTIONS-DEBUGGING OPTIONS:



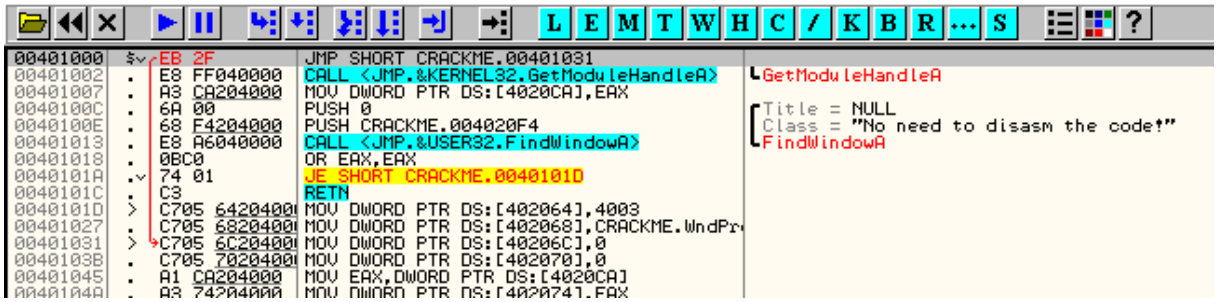
А там во вкладку CPU:



Ставим следующие три галочки:

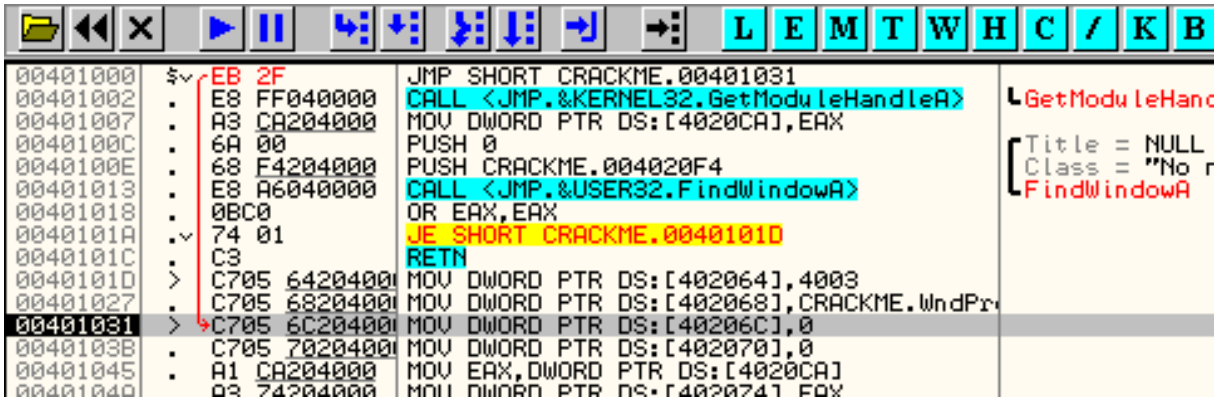


Видим, что предоставляемая информация стала гораздо нагляднее.

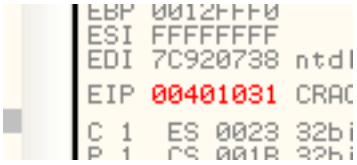


Видим, что теперь OllyDbg показывает нам красной линией откуда и куда совершается переход, в данном случае в 401031.

Если запустим с помощью F7:



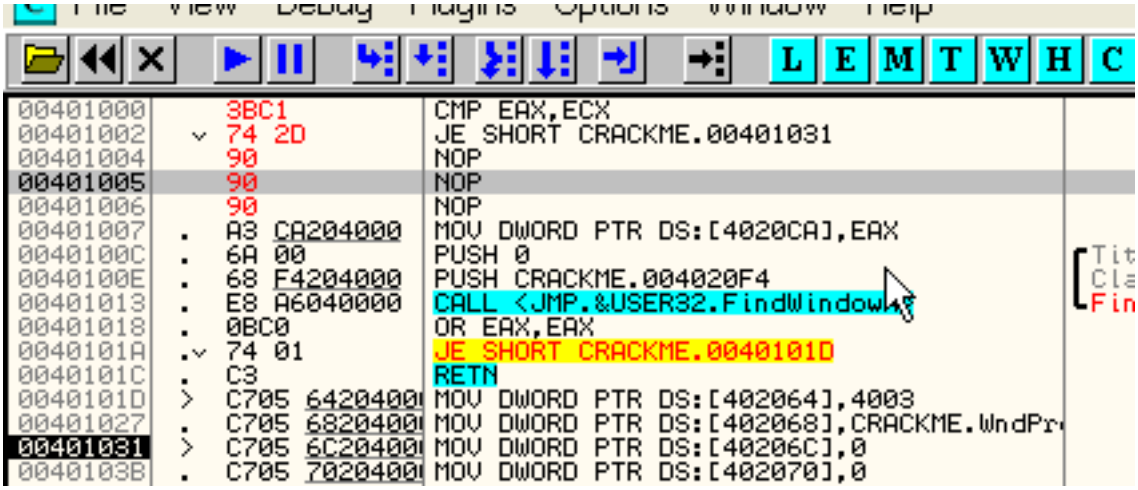
Совершается переход и EIP становится равным 401031.



Он равен этому значению, так как EIP содержит адрес инструкции, которая должна быть выполнена.

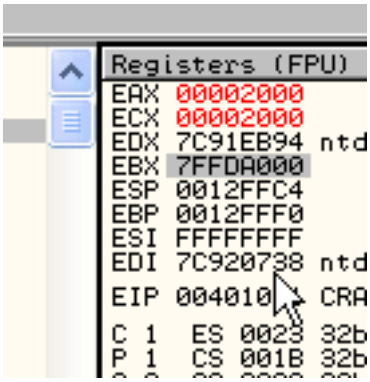
JE или JZ

И то и другое являются одним и тем же условным переходом, который можно записать двумя разными образами. JZ «прыгает» тогда, когда активен флаг Z.

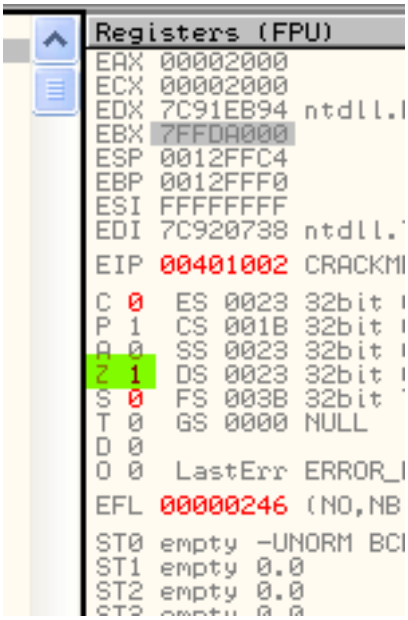


Напишем в OllyDbg две инструкции, которые делают сравнение и совершают переход, если сравнение оказалось удачным.

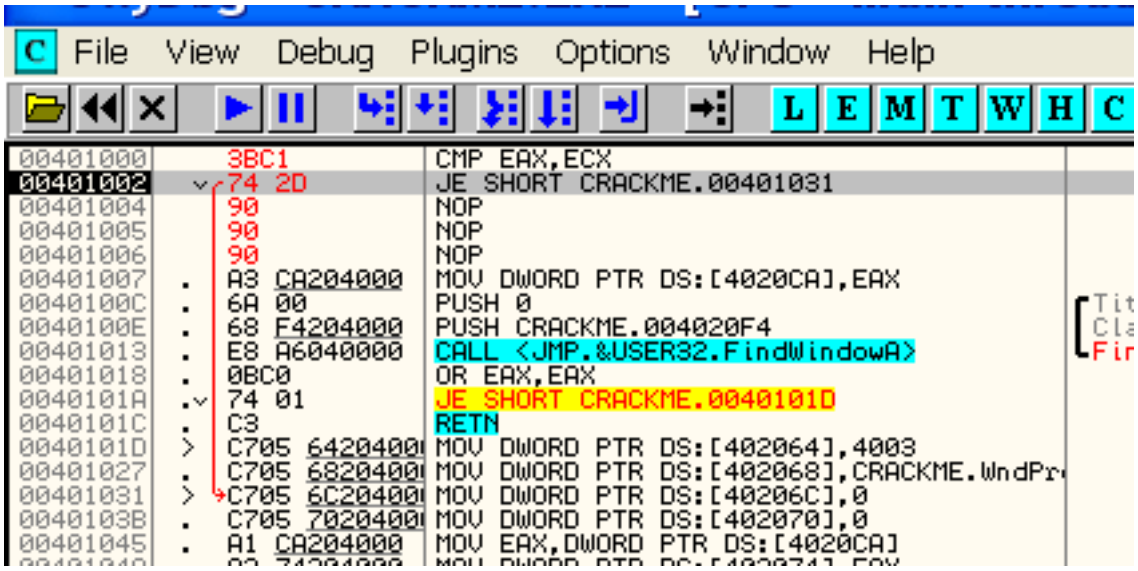
Делаем EAX и ECX одинаковыми.



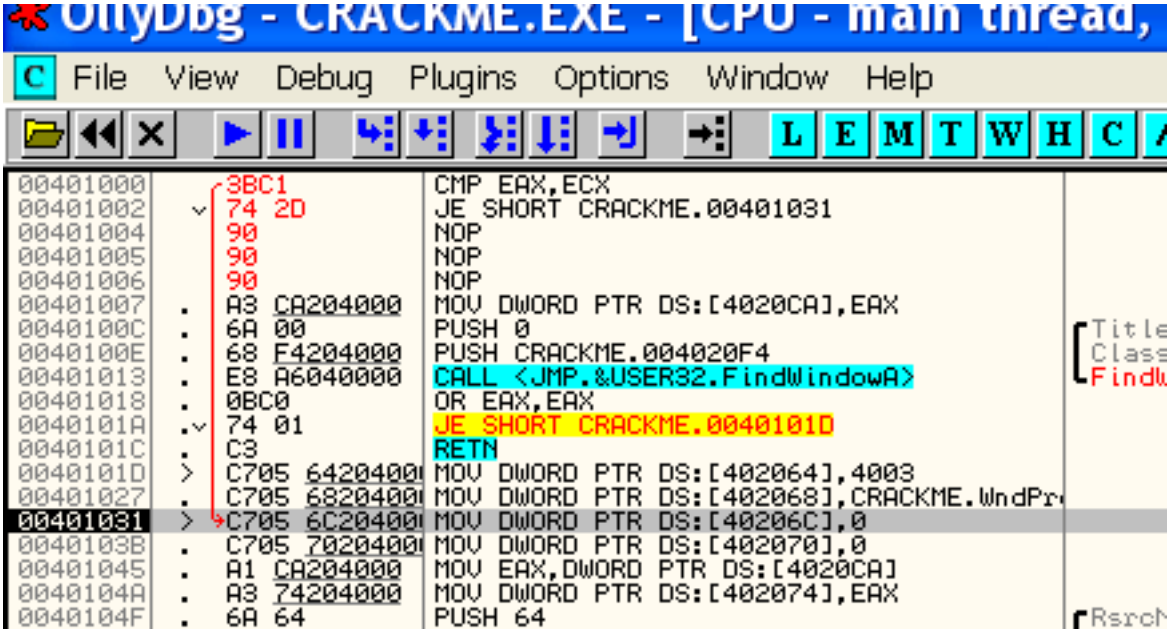
При выполнении сравнения будет произведено вычитание одного параметра из другого, и так как они оба равны, то результат будет равен нулю, а флаг Z станет активным.



Следующая инструкция – это условный переход:

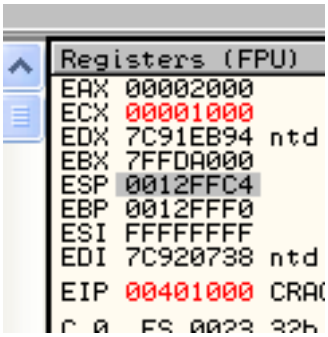


OllyDbg сообщает нам принятое программой решение и, поскольку флаг Z активен, показывает красной линией, куда будет совершён переход. Если бы она была серого цвета, то это означало бы, что принято решение не совершать переход. Нажимаем F7.

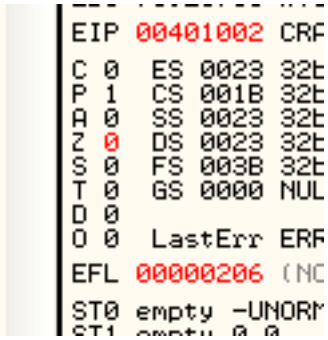


Видим, что был совершён переход и EIP сейчас равен 401031.

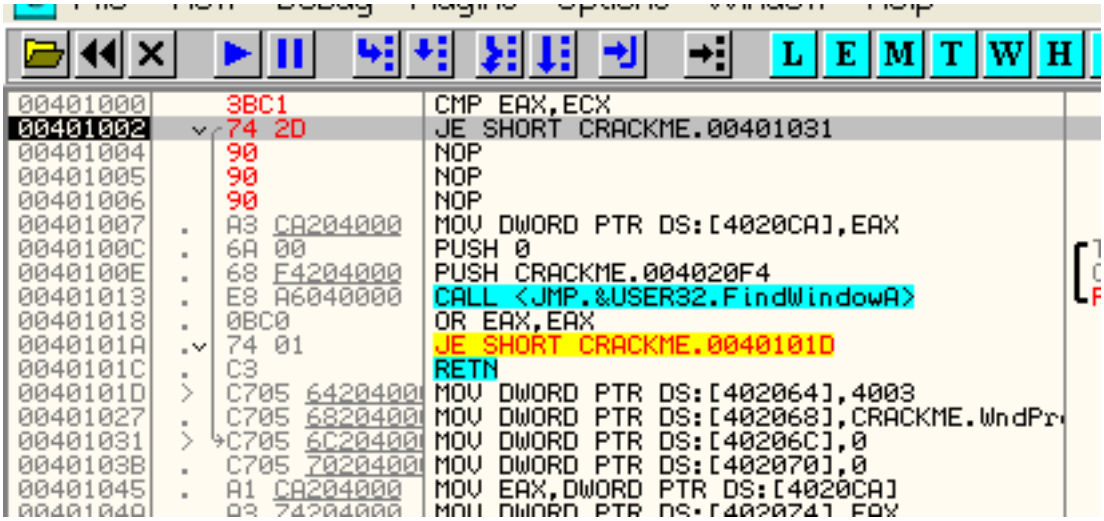
Повторим пример с EAX, отличным от ECX.



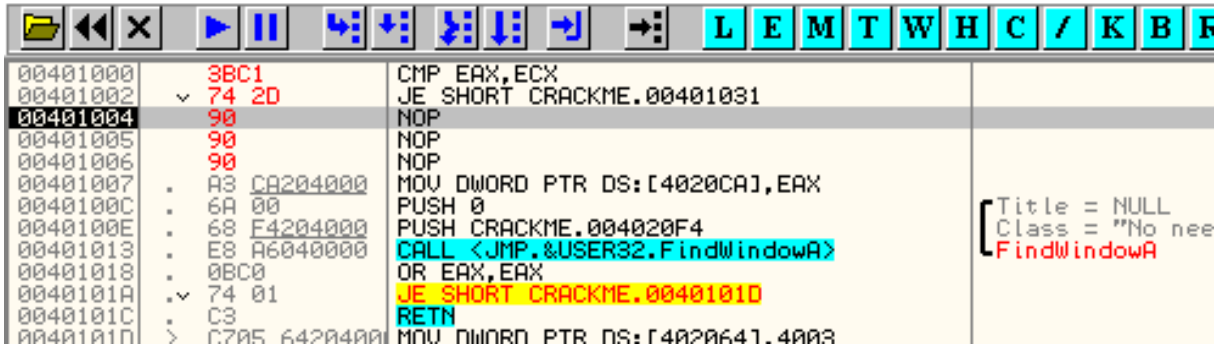
При нажатии на F7, поскольку результат первой инструкции отличен от нуля, флаг Z остаётся неактивным.



И идём в Олли:

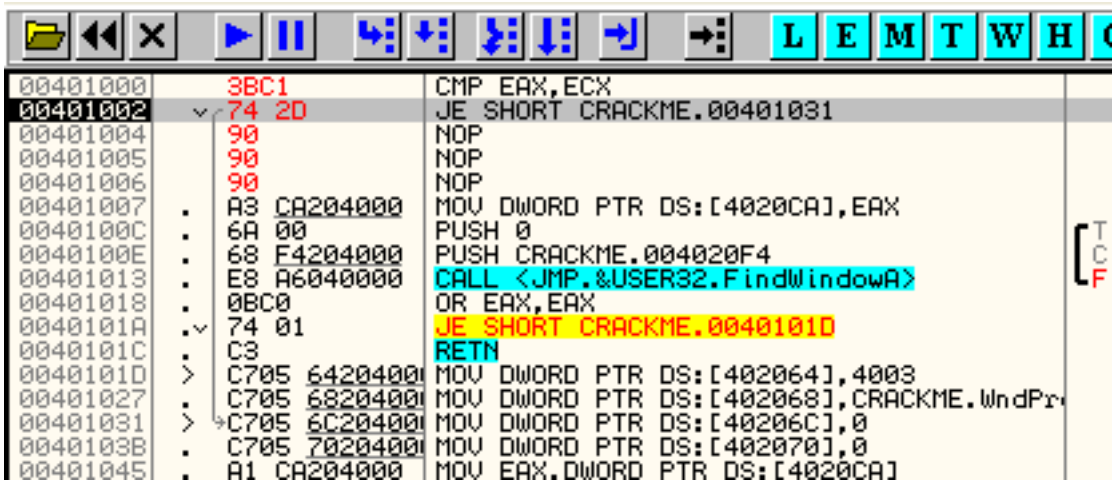


Поскольку перехода не произойдёт, то линия перехода отображается серым цветом. Ещё раз нажимаем F7.

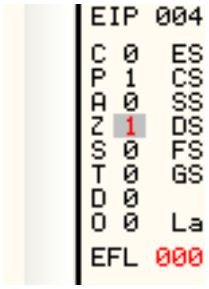


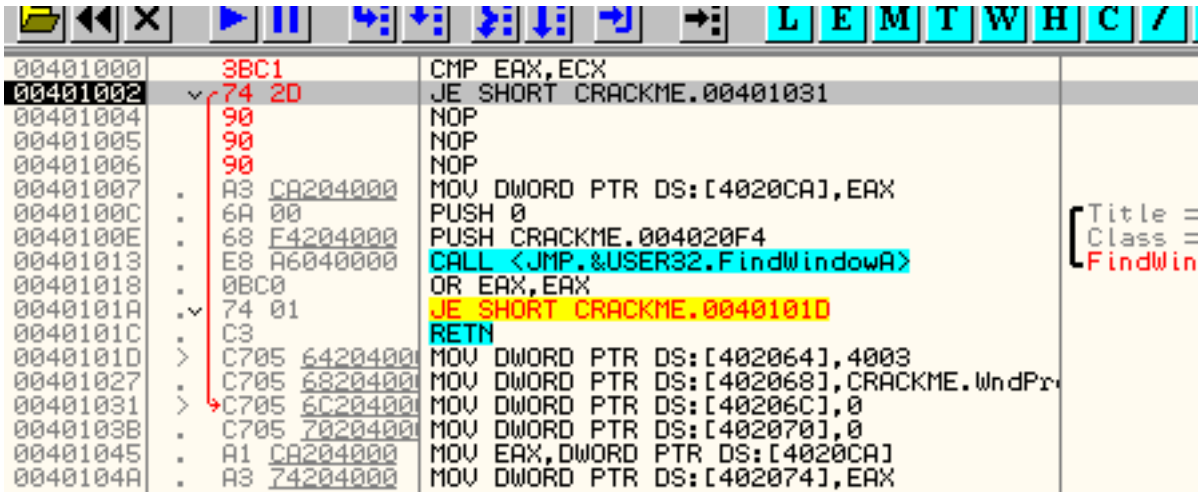
Видим, что действительно перехода не произошло, и выполнение программы продолжается в 401004. Примерно таким образом совершаются сравнения во всех программах.

Если повторим предыдущий пример до шага с переходом, но не будем его совершать:



Мы знаем, что флаг Z равен нулю и перехода не будет. Теперь что случится, если кликнем два раза на флаге Z и изменим его значение на 1?



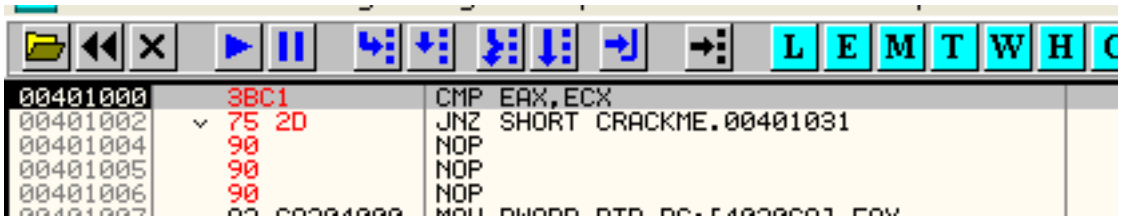


Видим, что линия стала красного цвета, а переход будет совершён, независимо от результата сравнения. Хотя программа уже приняла решение о переходе, его можно изменить, напрямую манипулируя значением флага.

Рассмотрим коротко другие флаги на наглядных примерах.

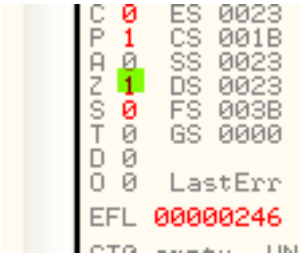
JNE или JNZ

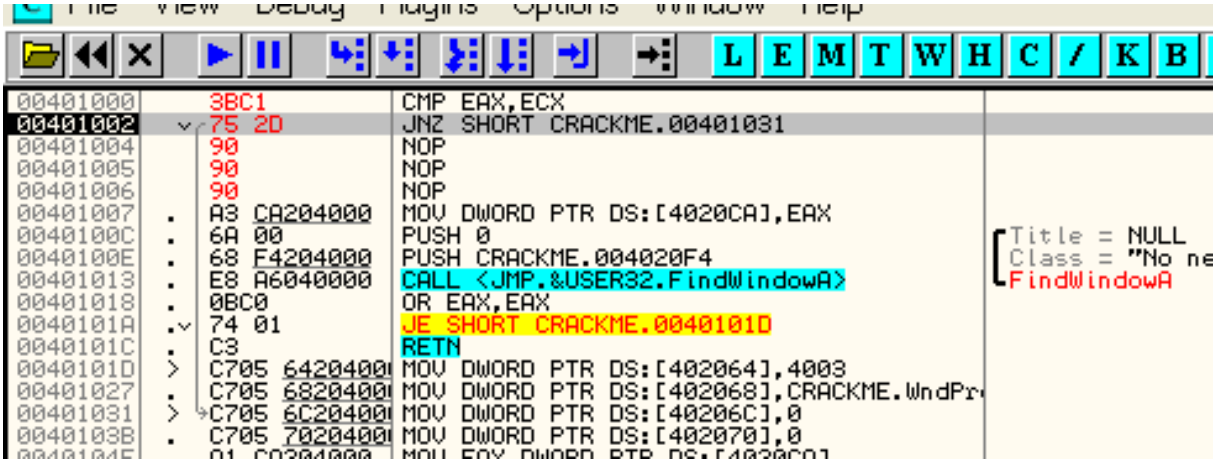
Эта инструкция противоположна предыдущей: переход совершается, если флаг Z неактивен, то есть если результат операции отличен от нуля.



Здесь пишем сравнение и JNZ.

Если EAX и ECX равны, то флаг Z будет активным.



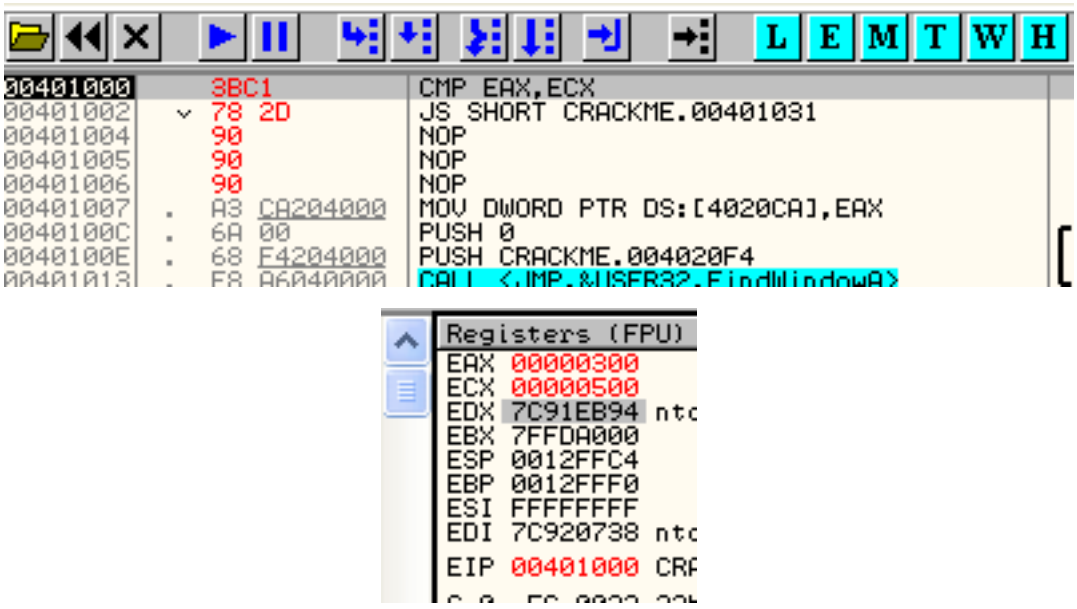


И в отличии от инструкции JZ, совершавшей переход, когда флаг Z был активен, эта, напротив, переходит тогда, когда флаг Z равен нулю, то есть неактивен.

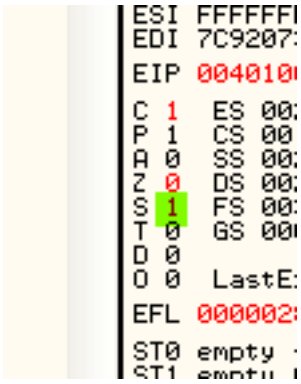
Как можно понять, если EAX отличен от ECX, то результат операции также будет отличен от нуля, что сделает флаг Z неактивным и, в свою очередь, приведёт к переходу.

JS

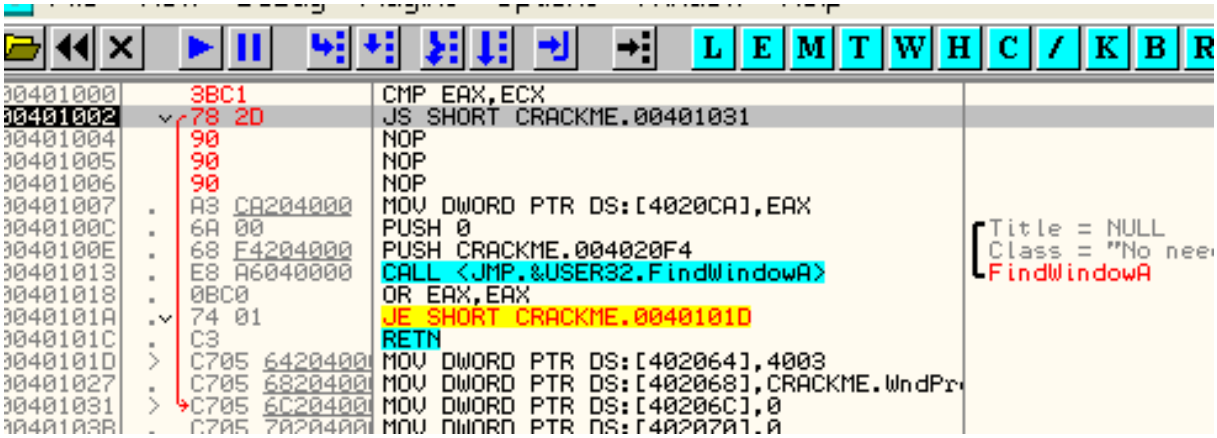
Как видно из таблицы выше, этот переход выполняется, если результат сравнения отрицателен, то есть если в предыдущем примере EAX меньше чем ECX.



Нажимаем F7:



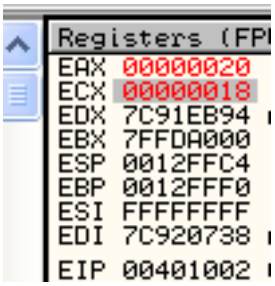
Флаг S равен 1, поэтому происходит переход.



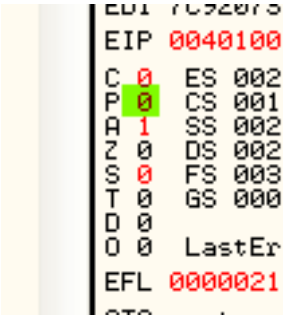
Видим, что красная линия указывает нам, куда произойдёт переход. В случае, если EAX больше ECX, флаг S будет неактивным и переход JS не сработает.

JNS Этот переход противоположен предыдущему. Переход совершается, когда флаг S равен нулю, то есть, если рассматривать предыдущий пример, когда EAX больше ECX. **JP или JPE**

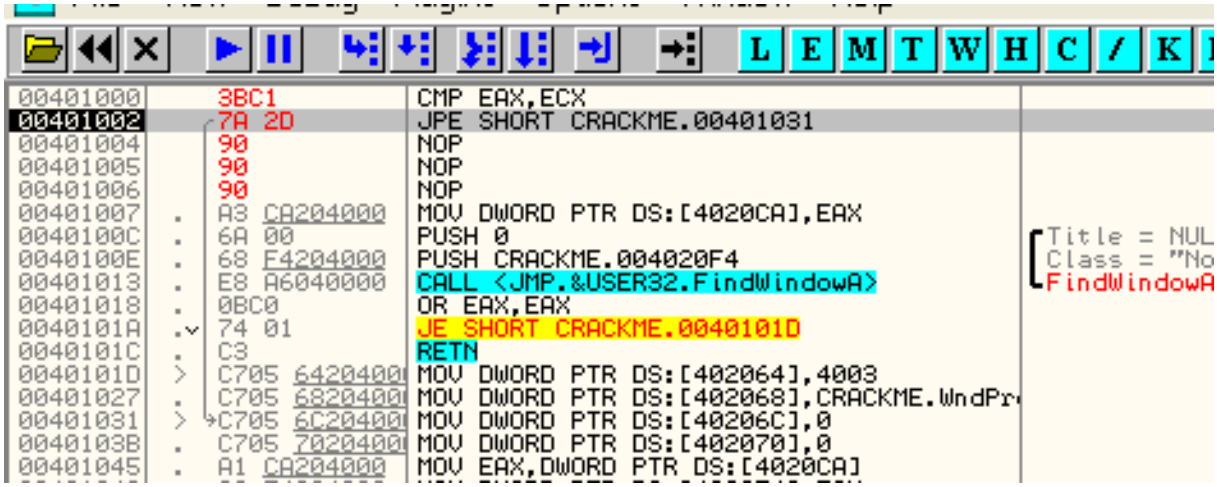
Условный переход JP срабатывает, когда активен флаг P, а это происходит, когда результат сравнения является чётным (понимая её так, как это мы рассматривали раньше).



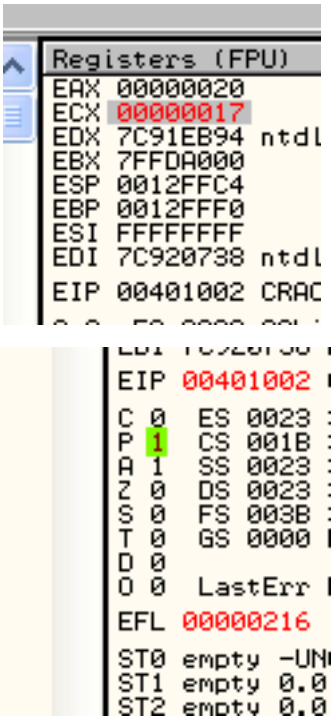
Имеем 20 в EAX и 18 в ECX. Нажимаем F7:



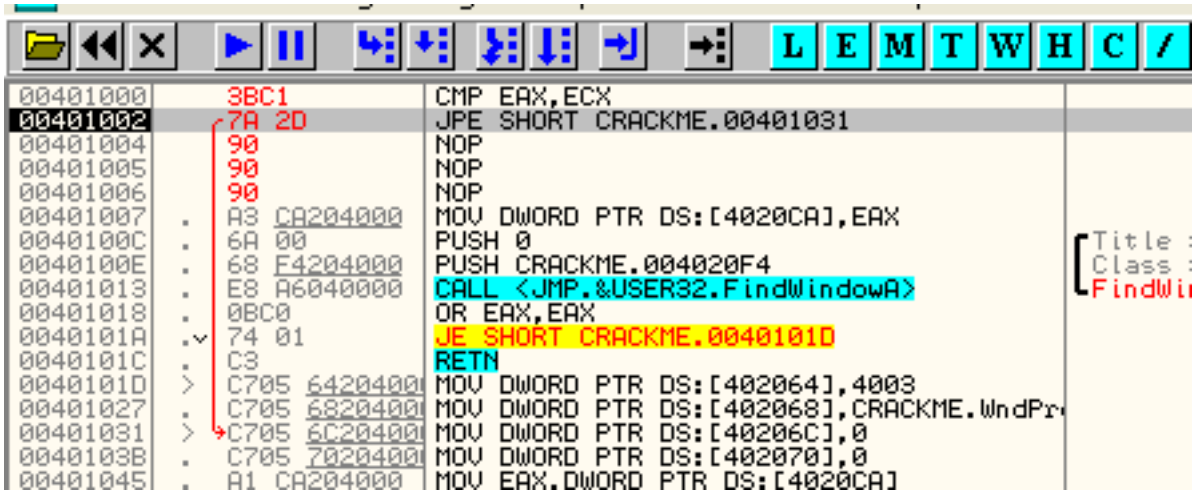
Разность между EAX и ECX равна 2, что в двоичной системе отображается как 10. В этом числе всего лишь одна единица, то есть нечётное их количество, поэтому флаг P неактивен и перехода JPE не происходит.



Теперь заменим значение в ECX на 17 и снова нажмём F7:



Видим, что результат равен 3, что в двоичной системе выглядит как 11 – в этом числе чётное количество единиц, флаг P активен, поэтому переход JPE срабатывает.

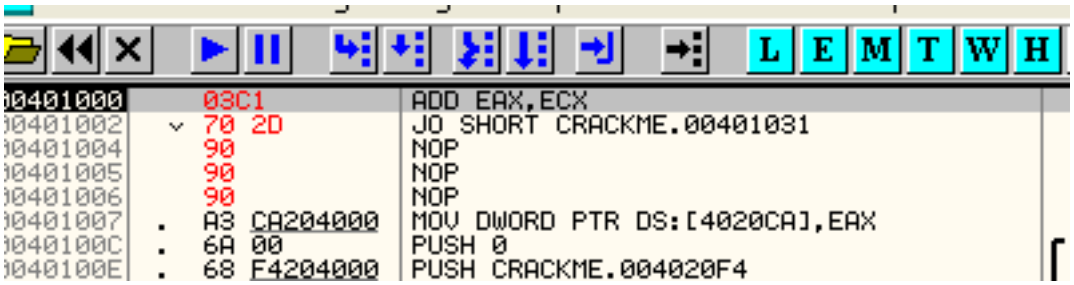


JNP или JNPE

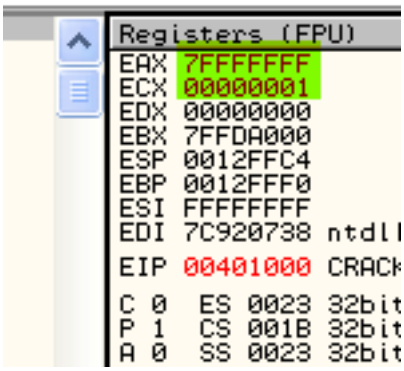
Этот переход противоположен предыдущему и срабатывает, когда флаг P равен нулю, то есть количество единиц в числе нечётно. В рассмотренном только что примере переход произошёл бы, когда результат был равен 2, и не сработал бы, когда он равнялся 3, то есть обратно действию JP.

JO

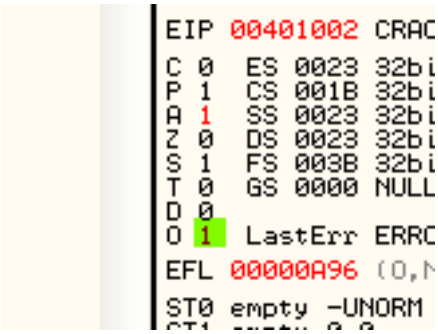
Срабатывает, когда происходит переполнение, в виду чего активируется флаг O.

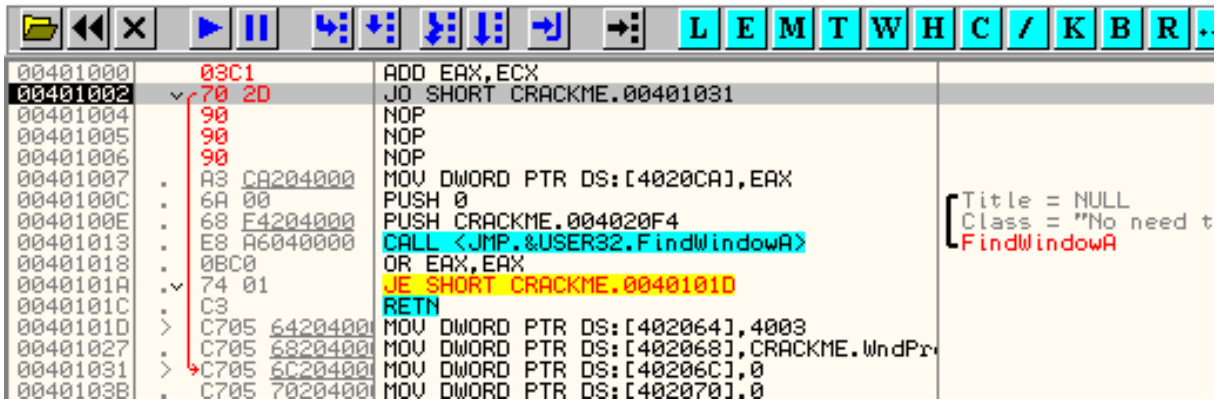


Здесь меняем инструкцию сравнения, так как нам необходимо активировать флаг O, для чего нужно, чтобы произошло переполнение, а этого можно добиться с помощью операции суммирования.



Нажимем F7:

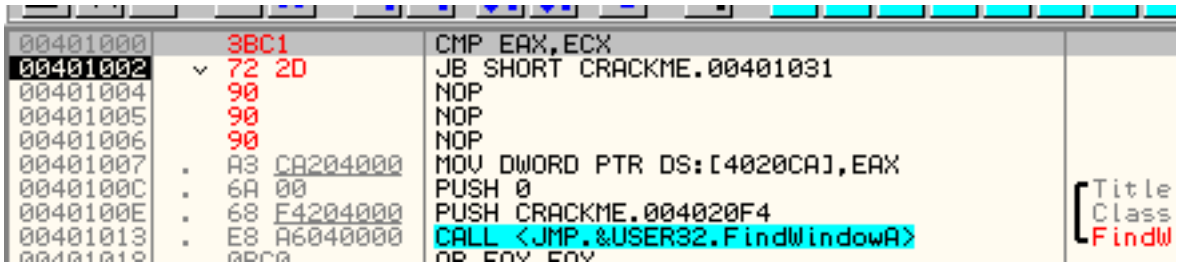


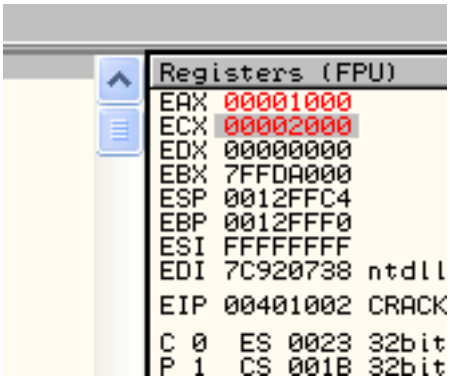


И срабатывает переход JO, так как из-за произошедшего переполнения активировался флаг O.

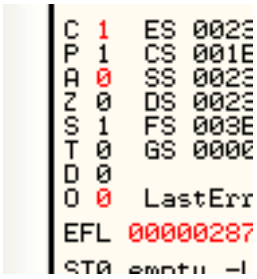
JNO Противоположен пердыдущему переходу и срабатывает, когда флаг или равен нулю, то есть переполнения не произошло. **JB**

Переход происходит, если первый операнд ниже чем второй. Смотрим пример.





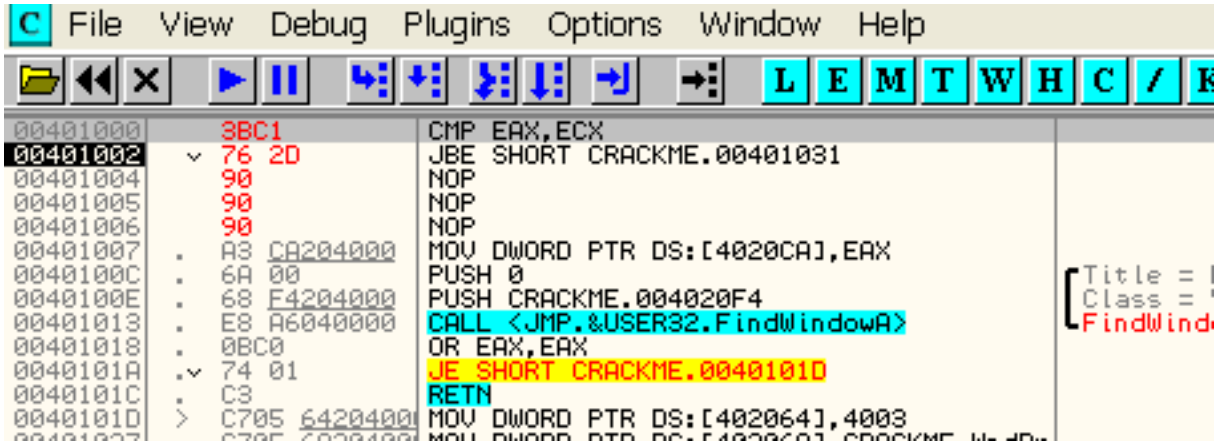
Видим, что EAX ниже чем ECX, то есть должен сработать переход. Нажимем F7:



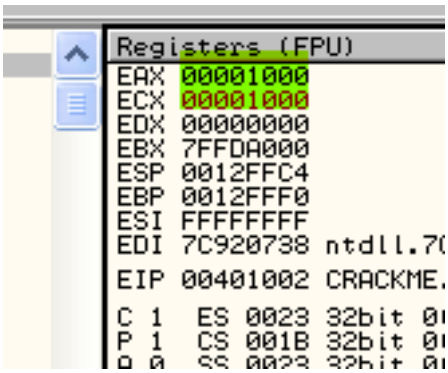
Флаг C активен, так как разность между этими двумя операндами является отрицательным числом, то есть установлен самый значимый бит, на основании чего и делается вывод, что EAX меньше ECX.

JNB Противоположен предыдущему – срабатывает, если флаг C равено нулю, то есть результат сравнения был положителен. В предыдущем примере это произошло бы, если EAX был бы больше ECX. **JBE**

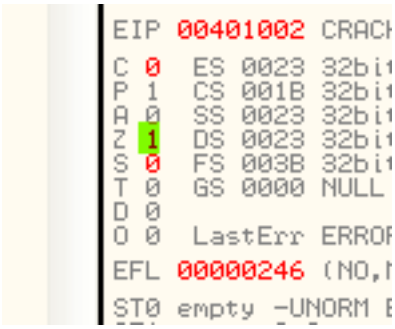
Это переход если ниже или равно, то есть проверяются два флага – активен ли флаг C, и тогда переход срабатывает, а также активен ли флаг Z, и тогда он тоже срабатывает, то есть если EAX равен ECX или меньше.



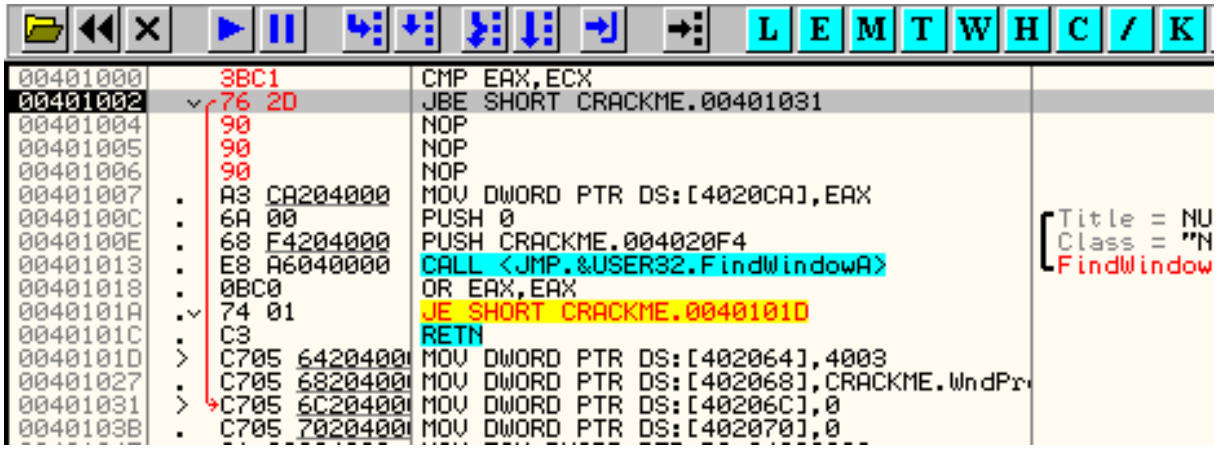
Сделаем EAX равными друг другу.



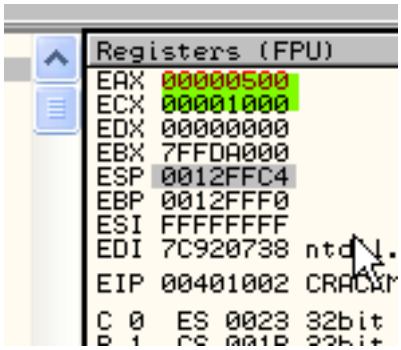
Нажимаем F7.



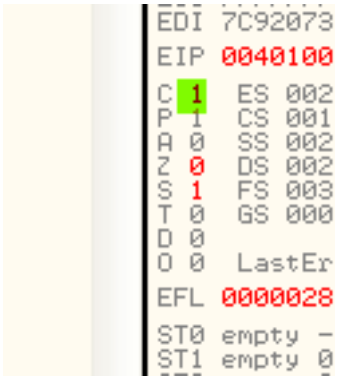
Видим, что флаг Z активен.



Если EAX меньше ECX:

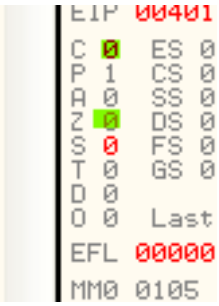


Нажимаем F7.

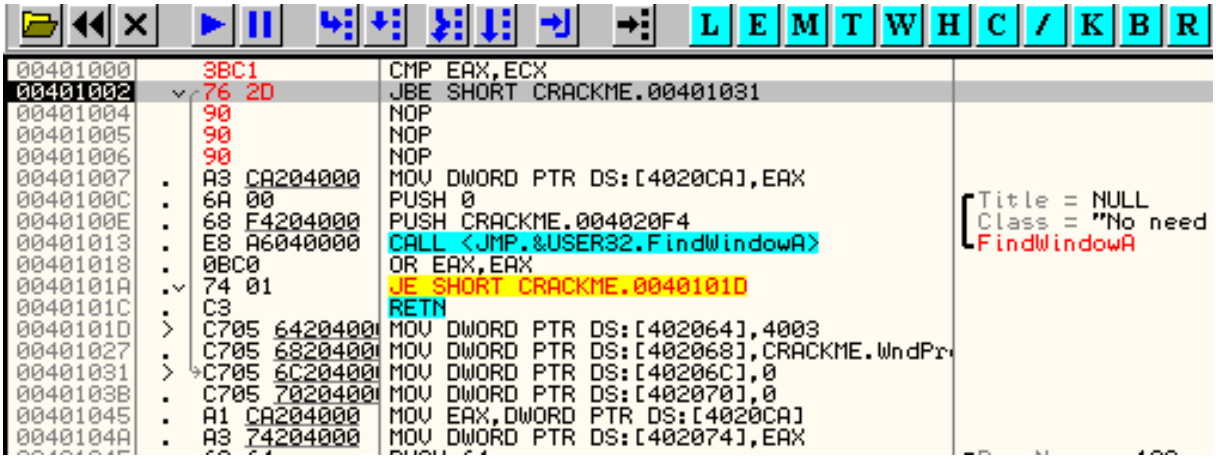


В данном случае активируется флаг C, так как результат отрицателен (установлен самый значимый бит), то есть EAX меньше ECX.

В последнем примере EAX больше ECX. Нажимаем F7.



Оба флага Z и C равны нулю, поэтому перехода не происходит.

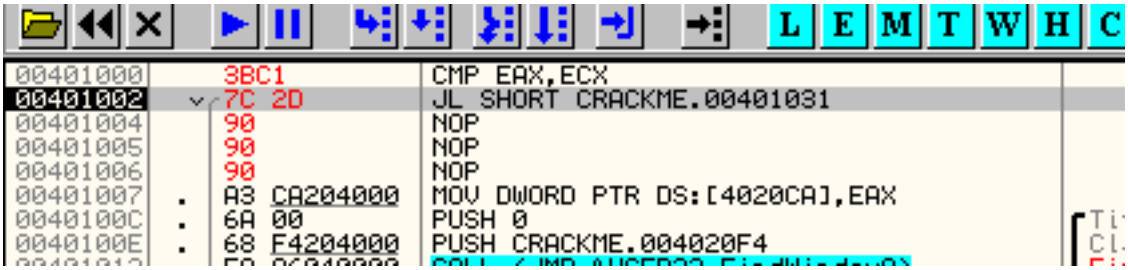


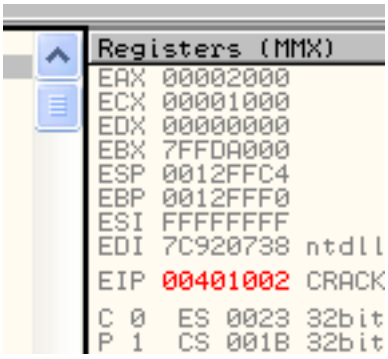
То есть, JBE срабатывает, если в нашем примере EAX ниже или равен ECX.

JNBE Этот переход противоположен предыдущему и срабатывает, если оба флага Z и C равны нулю. В предыдущем примере переход произошёл бы только в последнем случае. **JL**

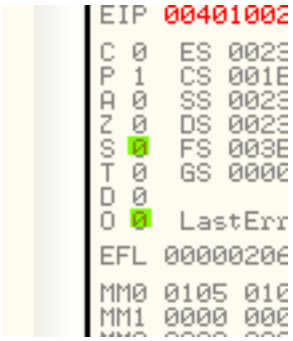
JL – это переход «если меньше», но в несколько другой форме. Здесь проверяется, отличается ли флаг S от флага O, и в этом случае и совершается переход.

Рассмотрим пример, когда EAX и ECX положительны и EAX больше ECX.



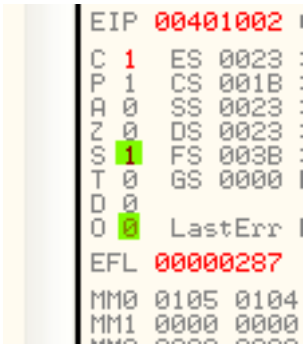


При нажатии на F7 перехода не произойдёт, так как EAX больше ECX и результат положителен, поэтому ни флаг O, ни флаг S не будут активны.

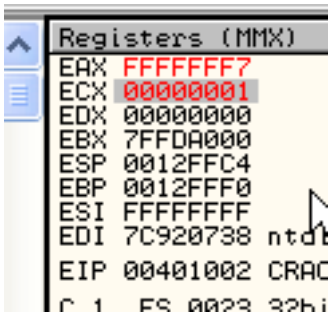


Повторим пример при EAX меньше ECX, но оба по-прежнему положительны:

Нажмём F7:

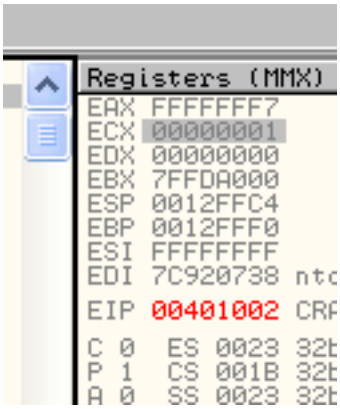


И так как флаги S и или отличаются друг от друга, то срабатывает переход, то есть он происходит, если первый сравниваемый операнд меньше второго и оба положительны. Рассмотрим другой пример.

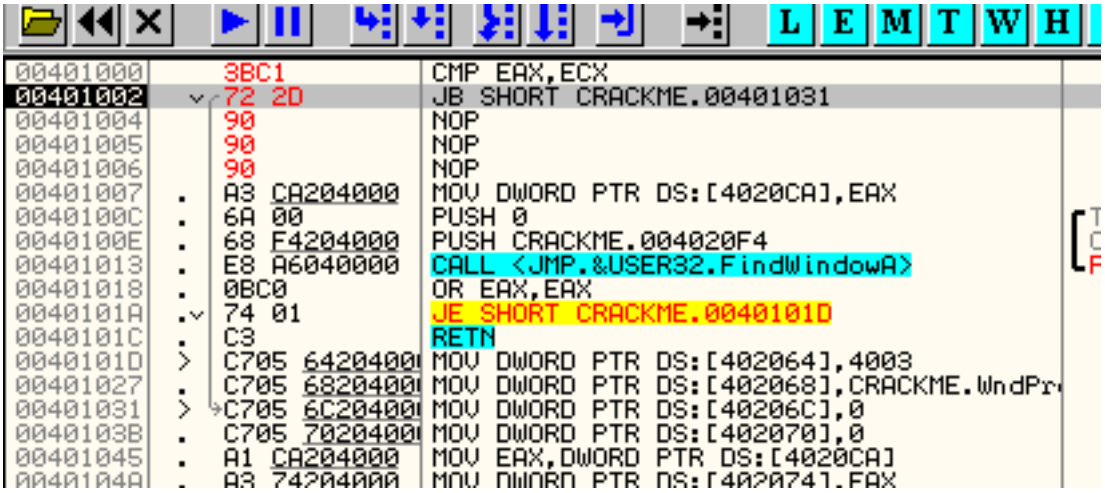


В данном случае EAX меньше ECX, так как первый является отрицательным. Смотрим, что произойдёт.

Переход совершается прекрасно, а теперь попробуем эти же два значения с переходом JB.



Нажимаем F7.



Видим, что JB не срабатывает, так как JB сравнивает оба значения как если бы они были положительными, то есть считает их беззнаковыми, поэтому если переход должен учитывать знак, то нужно использовать JL – это и есть основное различие между ними.

	La comparación se realiza con	
Para saltar si	**Números sin signo**	**Números con signo**
El destino es más grande que la fuente	JA	JG
Destino es igual a la fuente	JE	JE
El destino no es igual a la fuente	JNE	JNE
El destino es menor que la fuente	JB	JL
El destino es menor o igual a la fuente	JBE	JLE
El destino es más grande o igual a la fuente	JAE	JGE

Как видно, эти условные переходы делятся на две категории: те, в которых знаки операндов учитываются, и те, в которых нет.

Видим, что JA, JB, JBE И JAE считают оба операнда положительными, в то время как JG, JL, JLE и JGE рассматривают их как числа со знаком. JE и JNE работают в обоих случаях.

Думаю, что тема условных переходов и сравнений достаточно хорошо раскрыта, в дальнейшем увидим, как они используются в программах.

Следующая часть будет также посвящена ассемблеру, в ней будут рассматриваться вызовы (call) и возвраты (ret), а также режимы адресации. Потерпите ещё чуть-чуть, хе-хе.

Введение в крэкинг с нуля, используя OllyDbg - Глава 7

Автор: Рикардо Нарваха, пер. Quantum

Дата: 2007-03-18

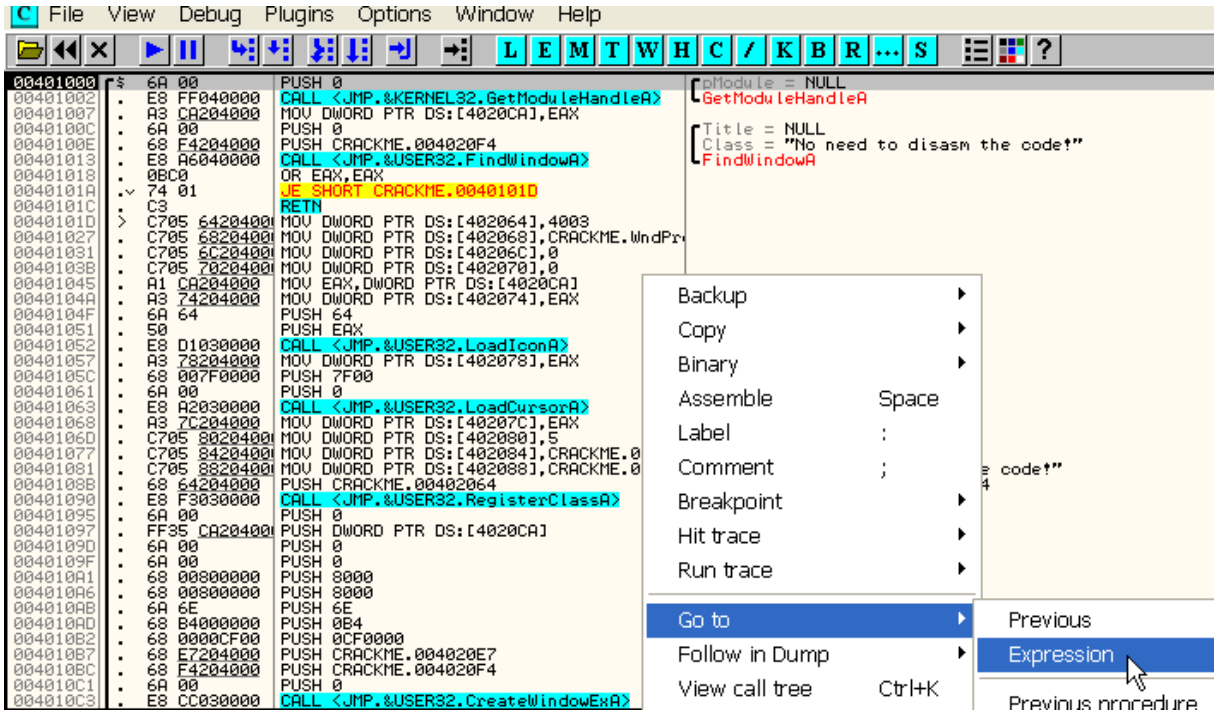
CALL и RET

Я намеренно оставил эти 2 инструкции напоследок. На данном этапе у Вас уже достаточно базовых знаний, чтобы вплотную подойти к изучению CALL и RET.

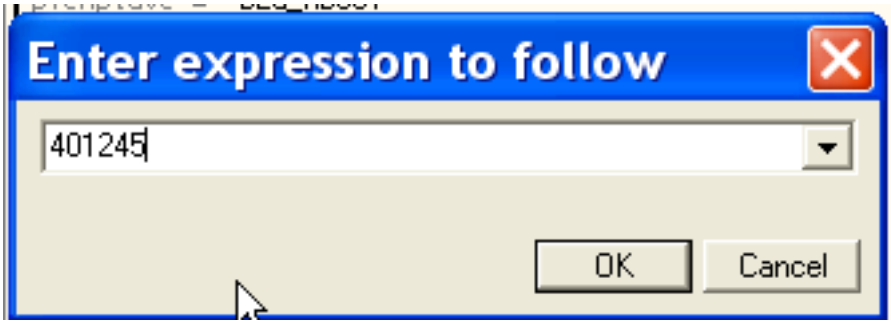
Несмотря на кажущуюся простоту этих 2х инструкций, многие новички не до конца понимают их суть. Поэтому я их выделил в отдельную главу и хочу ещё раз подчеркнуть, что к данной теме желательно отнестись внимательно - вообще эти инструкции в крэкинге играют очень важную роль. Также желательно при возможности повторить уже пройденный материал.

Итак, давайте снова загрузим крэкми CrueHead'a в OllyDbg.

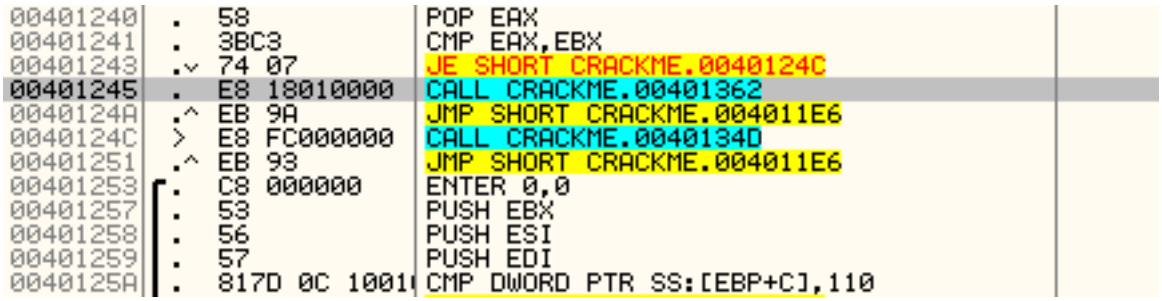
Правая кнопка мыши на любой строке листинга - "Go to" - Expression".



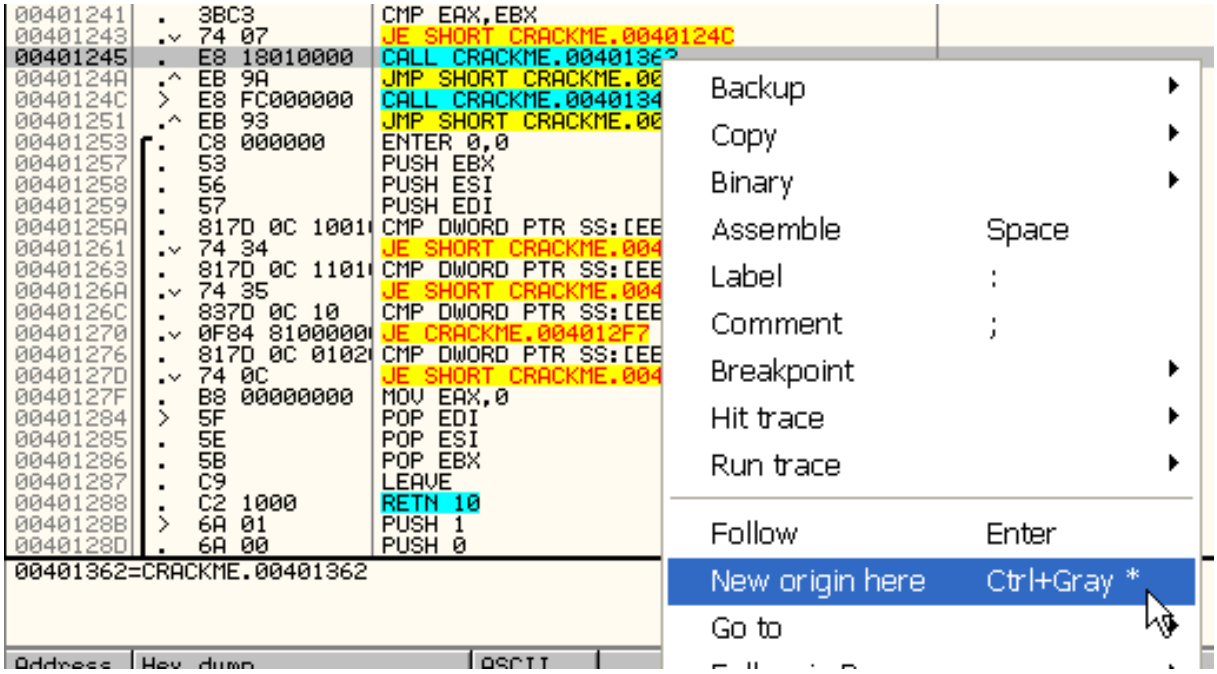
Во всплывшем диалоге вводим 401245



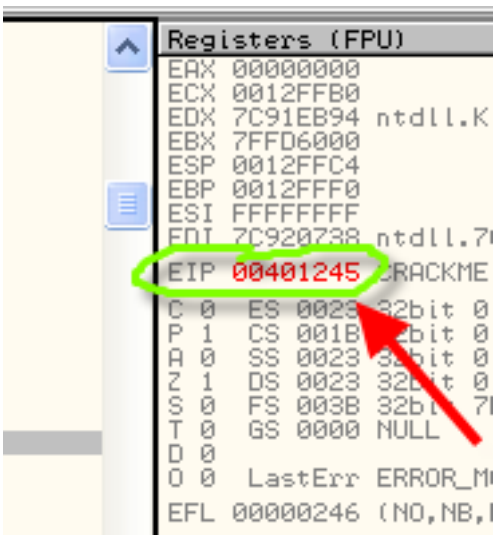
В окне дизассемблера разворачивается код по только что заданному адресу, где видим CALL, который нам и нужен для практики.



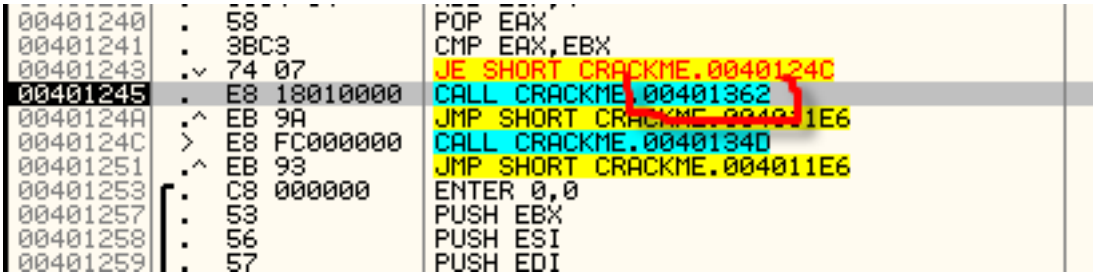
Чтобы выполнить этот CALL, подсвечиваю строчку - правая кнопка мыши - "New origin here". Теперь значение EIP указывает на 401245, а это значит, что следующей инструкцией выполнится наш CALL.



Тут можем проследить как изменяется значение EIP.



Вернёмся же к нашему CALL.



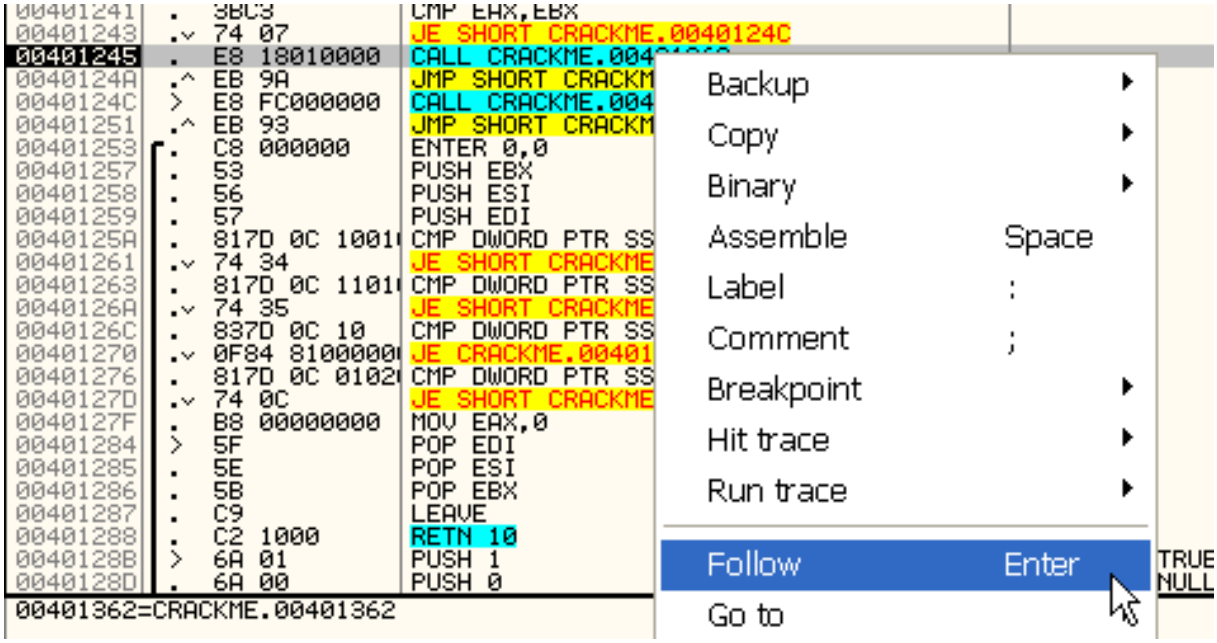
Инструкция CALL передаёт управление заданной процедуре (или просто подпрограмме), адрес которой указан в операнде. Например:

CALL 401362 означает, что управление будет передано на адрес 401362, т.е. процессор пойдёт и выполнит подпрограмму, которая находится по этому адресу, и как только подпрограмма завершится управление вернётся к следующей за CALL инструкции.

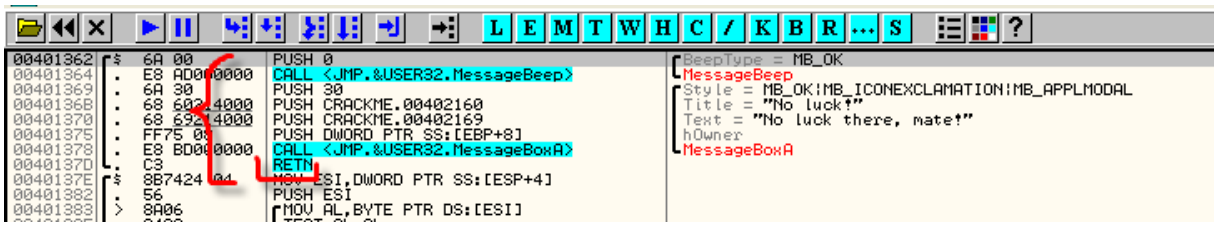
В данном случае, после завершения процедуры 401362, управление вернётся на адрес 40124A.

Когда мы имеем дело с инструкцией CALL, OllyDbg предоставляет несколько полезных механизмов трассирования. Если нам интересно продолжить трассировку по внутренностям данной процедуры, можно "нырнуть" в неё клавишей F7. Если же нам хочется просто глянуть на содержимое процедуры, чтобы далее принять решение трассировать её или нет, можно воспользоваться следующей опцией: правая кнопка мыши - "Follow". Наконец, если мы не хотим трассировать данную процедуру, т.к. её содержимое не сулит ничего интересного, можем выполнить её на одном дыхании клавишей F8 и продолжить трассировку по адресу следующей за CALL инструкции.

Итак, чтобы посмотреть содержимое процедуры, не переводя на неё управление, подсвечиваем CALL - правая кнопка мыши - "Follow":



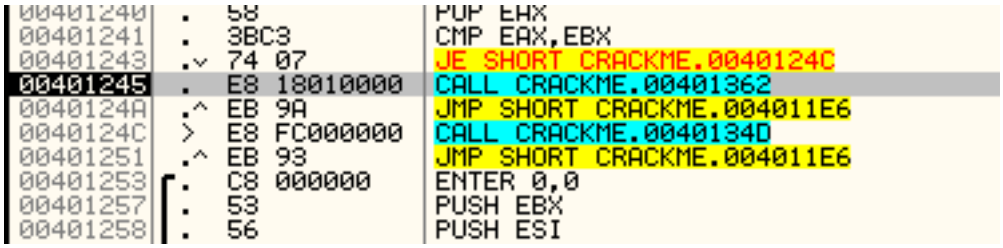
Как видите, значение EIP по-прежнему 401245, т.к. FOLLOW просто подгружает в дизассемблер код по заданному адресу и ждёт дальнейших распоряжений, но ничего при этом не выполняет.



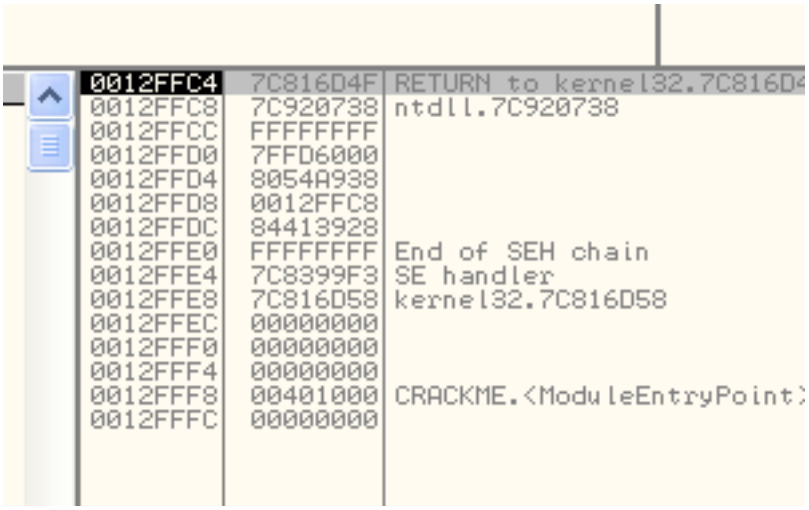
Итак, перед нами код процедуры, которая, естественно, начинается по адресу 401362, в соответствии со значением операнда CALL. Где же конец процедуры? - В данном случае конец приходится на следующую инструкцию RET, которая находится чуть ниже. OllyDbg использует мнемонику RETN вместо RET, что одно и тоже. Эта инструкция завершает процедуру, возвращая управление на адрес 40124A, где находится следующая за CALL инструкция.

Мы теперь знаем как подсмотреть код процедуры не переводя на неё управление. Вернёмся же назад - для этого достаточно нажать на клавишу (-), т.е. "минус". Эта клавиша всегда возвращает нас на один шаг назад в цепочке FOLLOW, ничего при этом не выполняя.

Вот мы снова видим изначальный CALL:

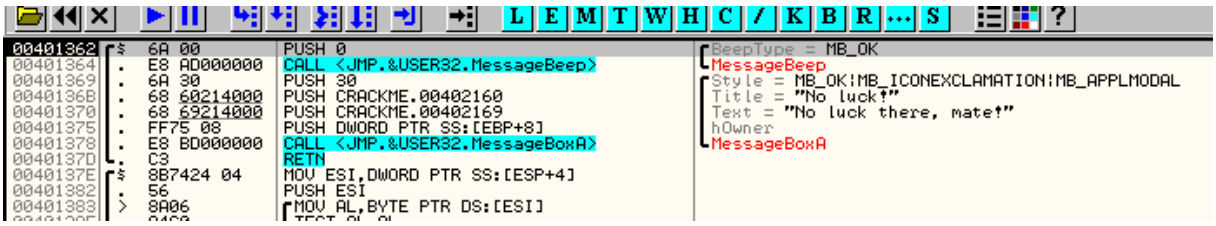


Далее мы погрузимся в процедуру, т.е. переведём на неё управление клавишей F7, но сначала обратите внимание на стек. Это очень важно, т.к. в стеке хранится адрес возврата, который использует инструкция RET, чтобы передать управление обратно.



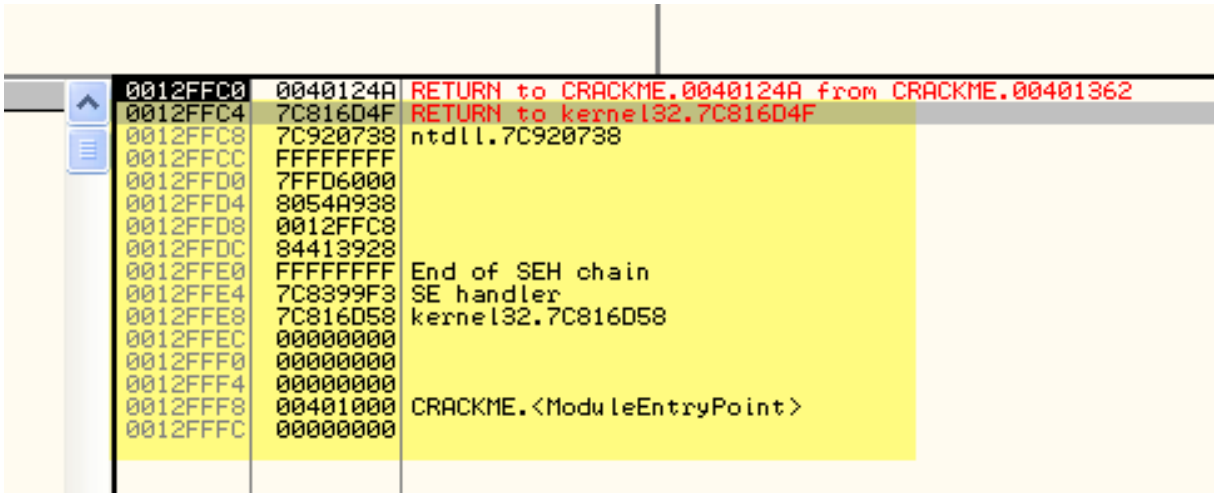
Предыдущая картинка отображает содержимое стека на моей машине. Возможно, у Вас будут другие значения, но это несущественно.

Нажимаю на F7:



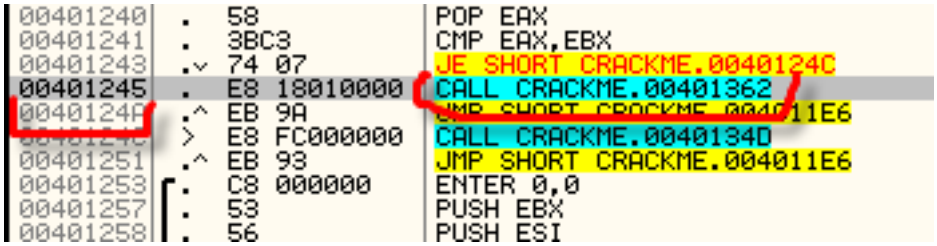
"Погружаемся" в процедуру и видим, что на этот раз, в отличие от предыдущих манипуляций с FOLLOW, значение EIP изменилось - теперь оно равно 401362 и это значит, что мы действительно начали выполнять код процедуры.

Давайте обратим внимание на стек

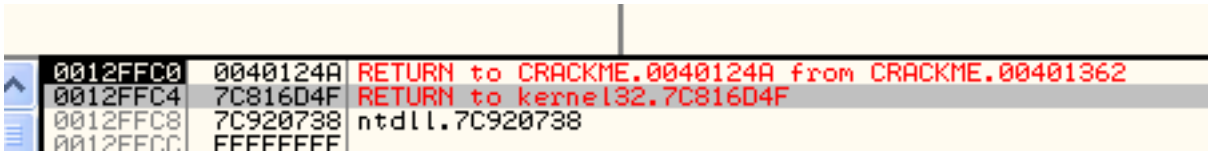


На предыдущей картинке я выделил жёлтым состояние стека до входа в процедуру. Как видите, в стеке появилась новая ячейка, т.е. при выполнении инструкции CALL в стек добавляется адрес возврата, чтобы RET знала, куда вернуть управление при завершении процедуры.

Значение этой новой ячейки - 40124A - соответствует адресу следующей после CALL инструкции. Если Вы вдруг успели забыть об этом, следующая картинка освежит вашу память:



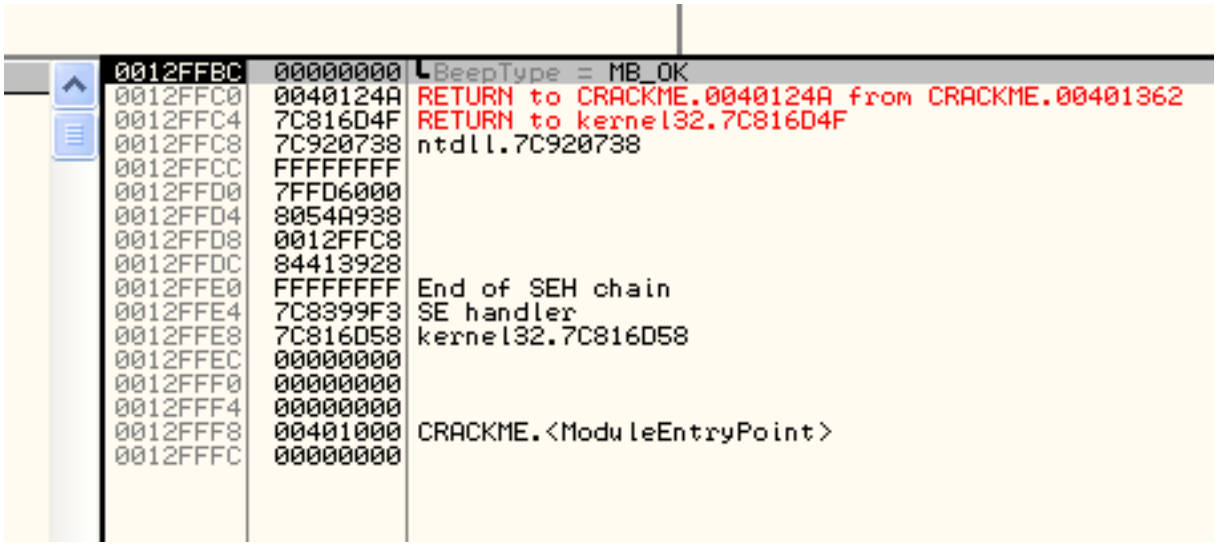
Причём Olly заботливо добавляет к этой ячейке некоторую дополнительную информацию.



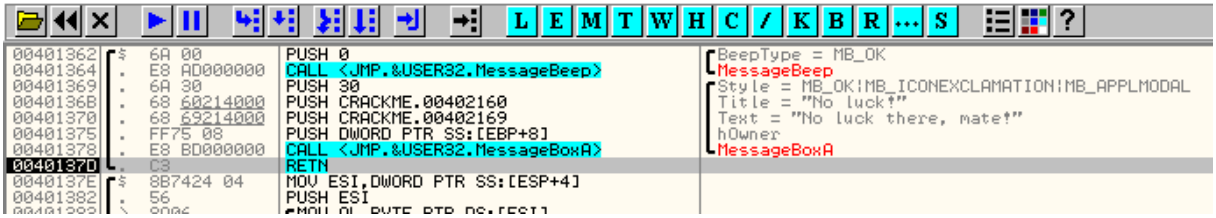
Сообщает нам, что это адрес возврата из 401262 в 40124A.

Т.е. Olly ещё не в курсе, где находится RET, но точно знает, что процедура начинается по адресу 401362 и заканчивается по какому-нибудь адресу, где вероятнее всего находится инструкция RET, которая вернёт управление на адрес 40124A.

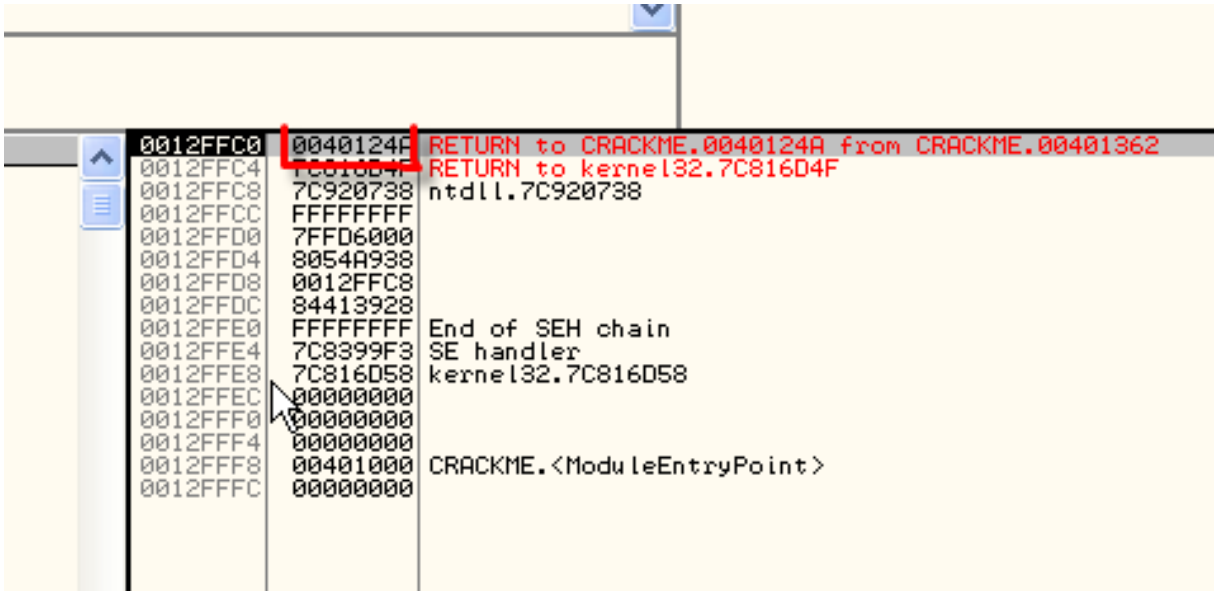
Давайте ещё раз нажмём на F7, чтобы выполнить PUSH 0, и посмотрим, как этот ноль добавится в стек, сместив вниз наш адрес возврата.



Процедура может содержать тысячи стековых операций (PUSH, POP и др.), добавляя и удаляя из стека различные значения, но при завершении процедуры в верхушке стека должен снова оказаться адрес возврата. Давайте продолжим выполнять процедуру в пошаговом режиме, нажимая F8, чтобы не трассировать код внутри процедур, пока не доберёмся до инструкции RET.



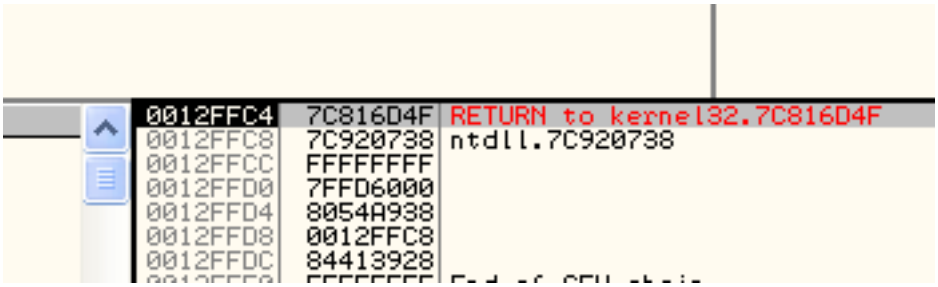
Вот мы и добрались до завершающей инструкции RET, и в верхушке стека снова находится адрес возврата.



Таким образом, можно заключить, что RET обычно является прямой противоположностью CALL, т.е., если CALL вызывает процедуру, RET возвращает управление обратно. RET также удаляет за ненадобностью из стека адрес возврата.

Жмём F7.

и возвращаемся к адресу 40124A. При этом состояние стека становится таким, каким оно было до вызова процедуры.

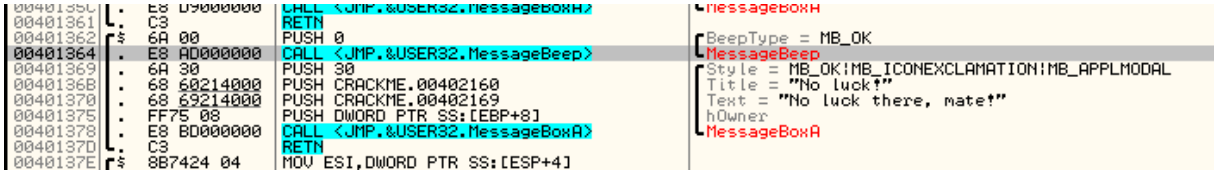


Стоит добавить, что RET можно использовать не только для завершения текущего CALL, например:

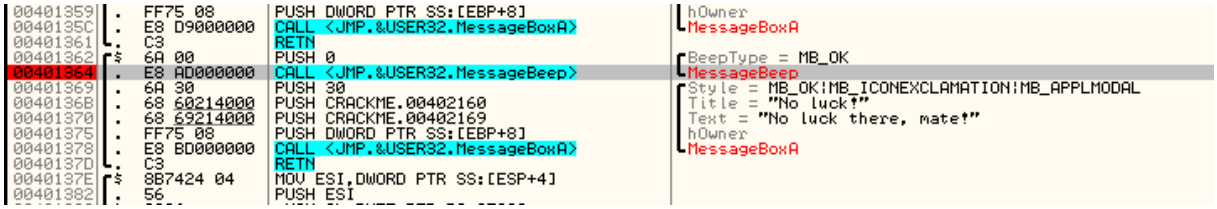
```
PUSH 401256
RET
```

Сначала PUSH затолкнёт в стек значение 401256. Далее, следующий за ним RET воспримет это значение как адрес возврата из текущего CALL, хотя это вовсе не так, и передаст управление на адрес 401256. Таким образом, этот код отработает точно также как JMP 401256.

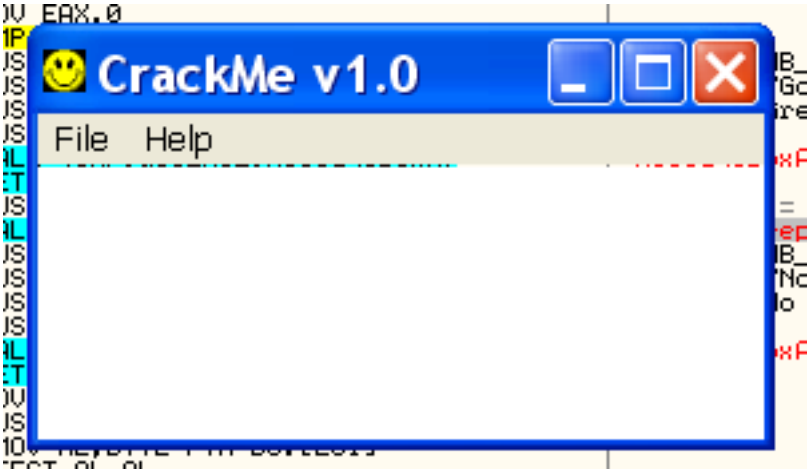
Далее мы рассмотрим ещё один пример использования CALL/RET. Перезагружаем крэкми CrueHead'a. "Go to" - "Expression" - вводим 401364.



По этому адресу нажимаем F2, устанавливая таким образом точку останова (об этом мы подробно поговорим позже). Olly прервёт исполнение как только начнёт выполняться инструкция по адресу, на который установлена точка останова.

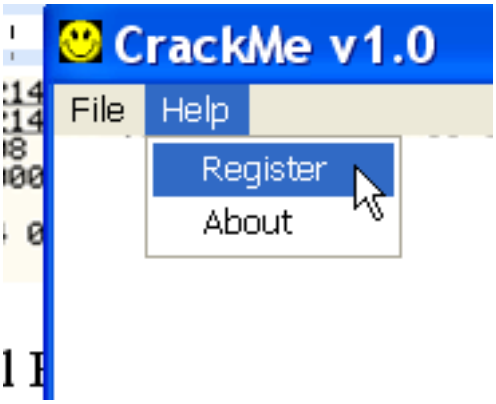


Адрес выделяется красным цветом и это значит, что точка останова (BREAKPOINT) установлена. Нажимаем на F9 (RUN), чтобы приложение запустилось...

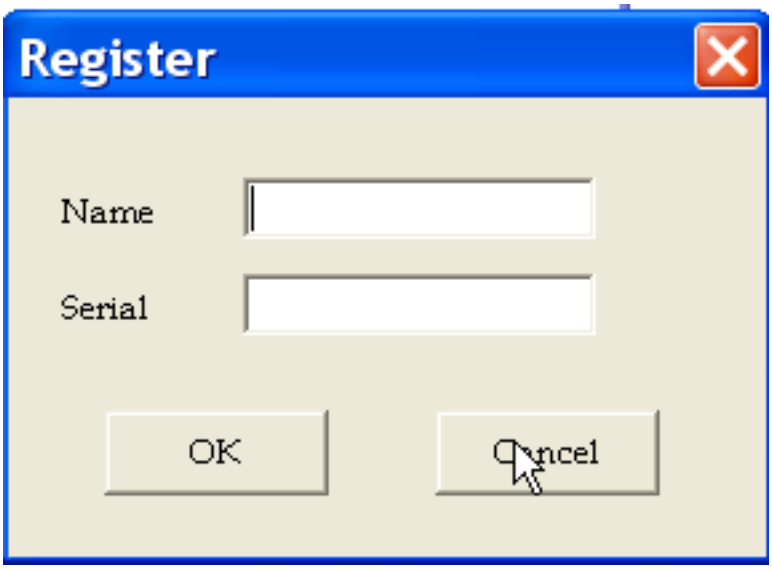


Появилось окно крэкми. Если Вы его не видите, поищите хорошенько через Alt+Tab :)

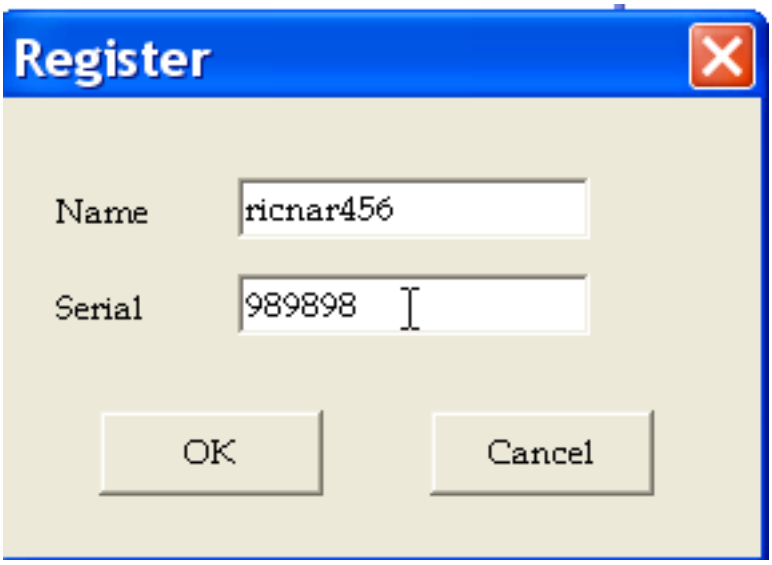
Приложение ещё не выполнило код, на который мы поставили точку останова. В окне крэкми вызовите меню и выберите опцию "Help" - "Register".



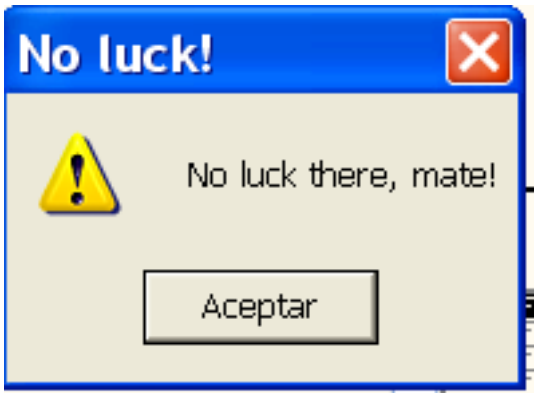
Появилось окно, через которое нужно ввести имя и серийник.



Пишем что угодно.



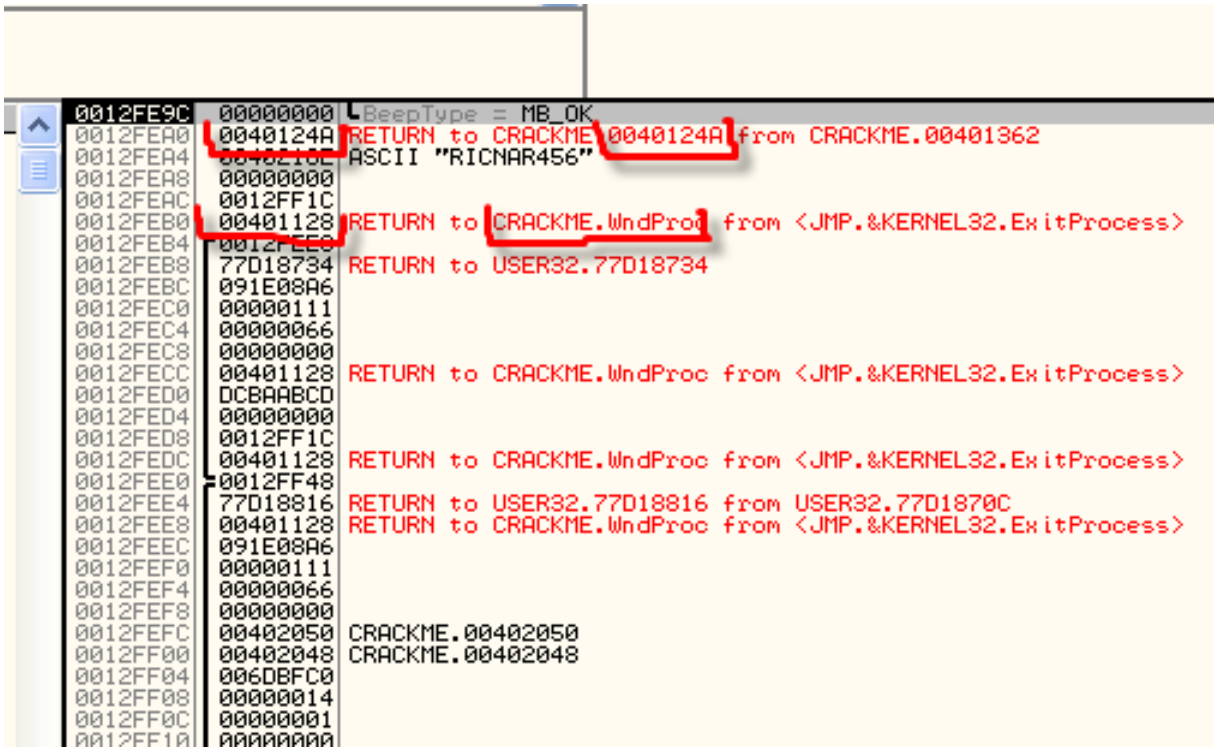
Нажимаем OK.



Выскакивает сообщение о том, что нам не повезло, т.е. крэкми не понравились наши имя и серийник (было бы просто удивительно, если бы мы сразу угадали правильные регистрационные данные :-) Если закрыть это сообщение, активизируется наконец наша точка останова.

```
0040135C   .  E8 D9000000   CALL <JMP.&USER32.MessageBoxA>          MessageBoxA
00401361   .  C3            RETN
00401362   $  6A 00         PUSH 0                                  BeepType = MB_OK
00401364   .  E8 AD000000   CALL <JMP.&USER32.MessageBeep>          MessageBeep
00401369   .  6A 30         PUSH 30                                 Style = MB_OK|MB_ICONEXCLAMATION|MB_APPLMODAL
0040136B   .  68 60214000   PUSH CRACKME.00402160                   Title = "No luck!"
00401370   .  68 69214000   PUSH CRACKME.00402169                   Text = "No luck there, mate!"
00401375   .  FF75 08        PUSH DWORD PTR SS:[EBP+8]               hOwner
00401378   .  E8 BD000000   CALL <JMP.&USER32.MessageBoxA>          MessageBoxA
0040137D   .  C3            RETN
0040137E   $  8B7424 04     MOV ESI,DWORD PTR SS:[ESP+4]
00401382   .  56            PUSH ESI
```

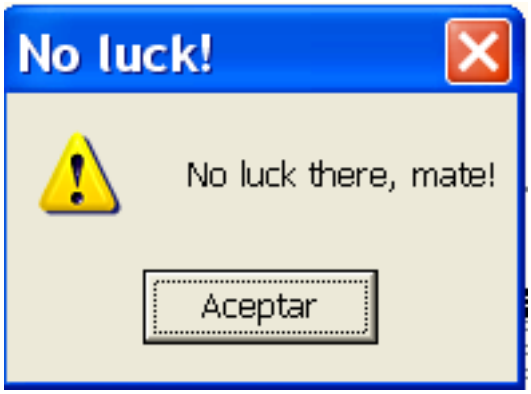
Мы находимся прямо посреди выполнения кода программы, но кое-какую базовую информацию можем почерпнуть например из стека:



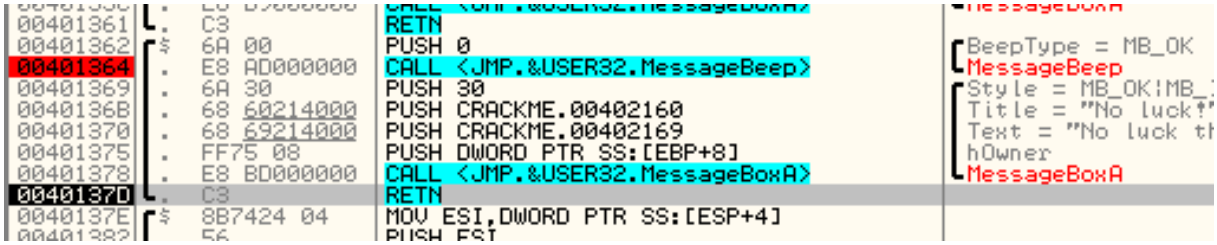
Видим сразу несколько RETURN TO... Видимо, мы находимся внутри процедуры и самый верхний RETURN TO содержит адрес возврата, на который передаст управление ближайший RET. Этот адрес, вероятно, окажется на верхушке стека, перед выполнением RET и управление будет передано на адрес 40124A.

Если присмотреться, можно заметить также, что перед нами уже знакомый код. Но на этот раз он вызван не нами, а в ходе нормального выполнения программы.

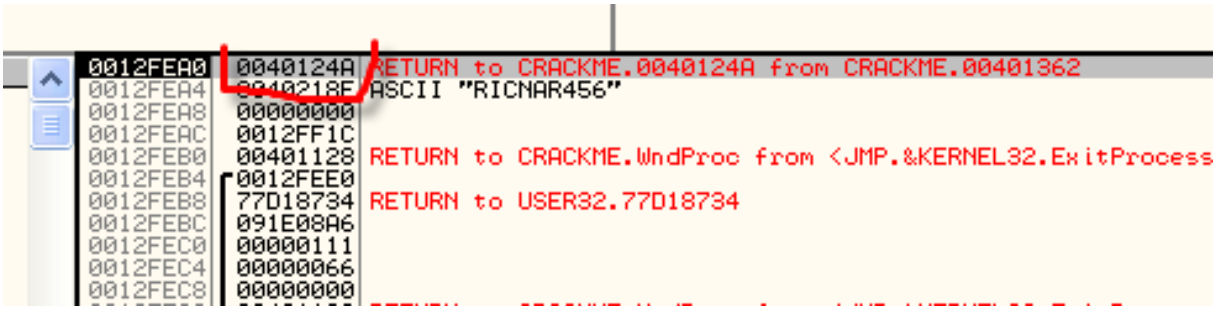
Жмём на F8, чтобы добраться до RET, как и раньше. Только в этот раз при выполнении последнего CALL сразу перед RET должно появиться сообщение, которое нужно закрыть, чтобы продолжить трассирование.



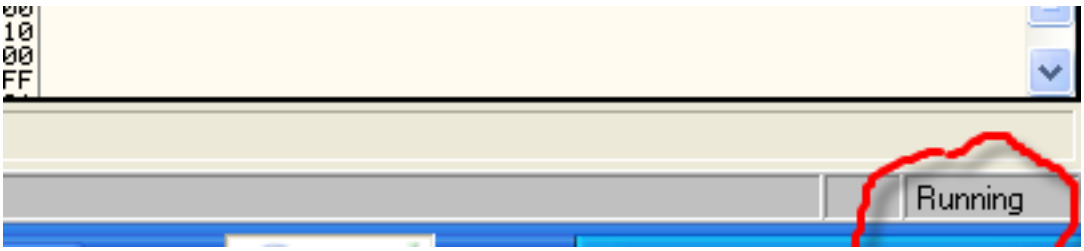
Закрываем его.

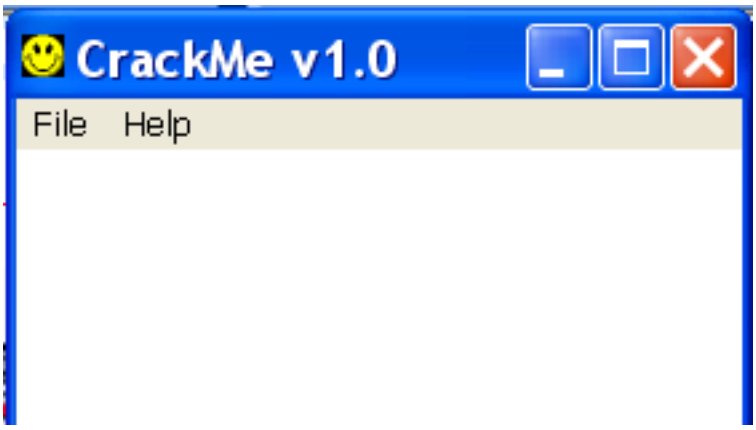


Вот мы и добрались до инструкции RET и верхушка стека теперь содержит адрес возврата, как мы и предполагали.



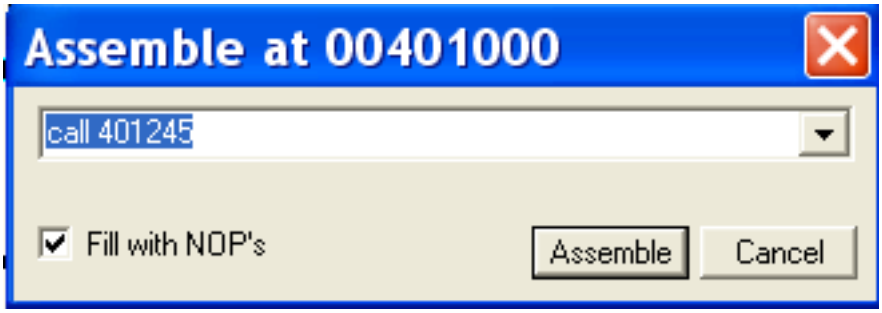
В прошлый раз мы выполнили процедуру самостоятельно, принудительно изменив значение EIP. В этот раз мы попали в код процедуры в следствии нормального выполнения программы. Достаточно снова нажать на F9, чтобы программа продолжила выполняться как ни в чём не бывало.

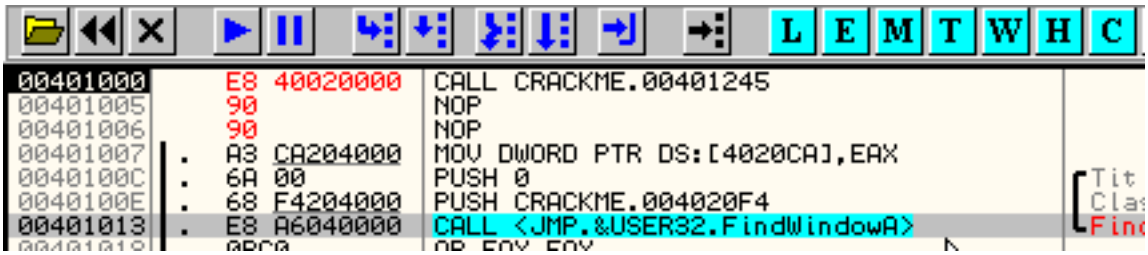




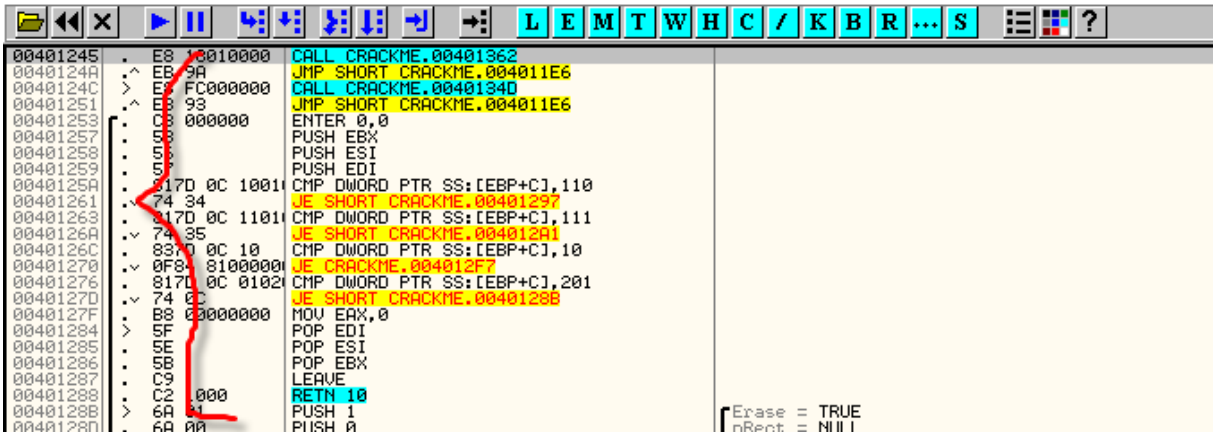
В общем, я хотел показать, что при срабатывании точек останова в процессе свободного выполнения программы можно почерпнуть из стека немало полезной информации. В частности, беглого взгляда по содержимому стека обычно хватает, чтобы узнать откуда была вызвана текущая процедура и куда она вернёт управление после завершения. Если же в стеке видно несколько ячеек с комментарием RETURN TO, то, очевидно, текущая процедура была вызвана из другой процедуры, которая в свою очередь тоже была откуда-то вызвана - это называется вложенностью процедур.

Рассмотрим ещё один закрепляющий пример. Для этого нужно перезагрузить крэкми в отладчике, сразу нажать на клавишу пробела и ввести следующую инструкцию: CALL 401245.



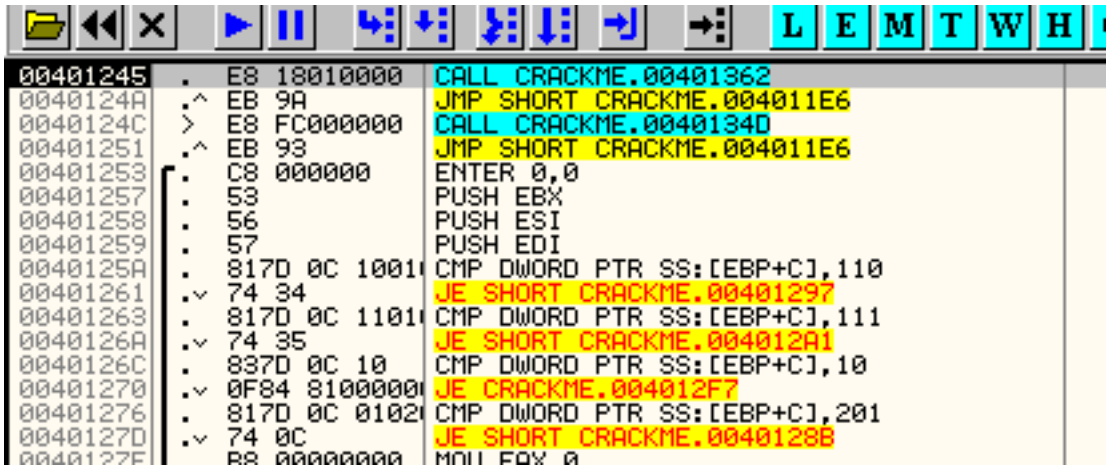


Готово. Далее воспользуемся опцией FOLLOW, чтобы попасть во внутрь функции.

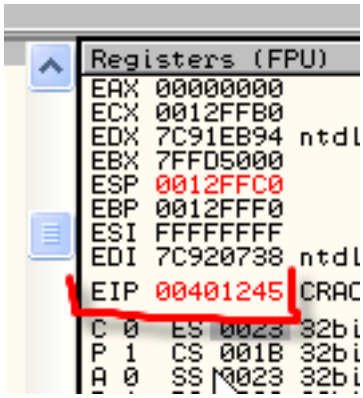


Процедура начинается по адресу 401245 и заканчивается по адресу 401288 (где находится инструкция RETN 10, которая немного отличается от знакомого нам уже RET, но об этом позже). Обратите внимание на вложенный CALL (первая инструкция процедуры является вызовом другой процедуры).

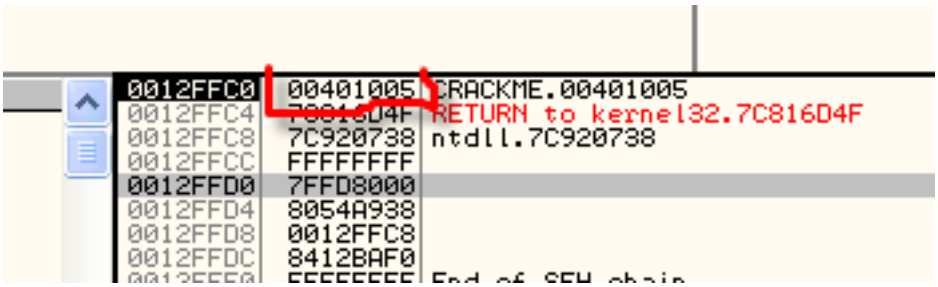
Нажмите МИНУС, чтобы выйти из FOLLOW. Теперь нажмите F7, чтобы войти в процедуру, фактически передав на неё управление.



Вот мы и внутри, о чём свидетельствует значение EIP: оно указывает на адрес 401245.



В верхушке стека хранится адрес возврата к следующей за нашим свежевписанным CALL инструкции.

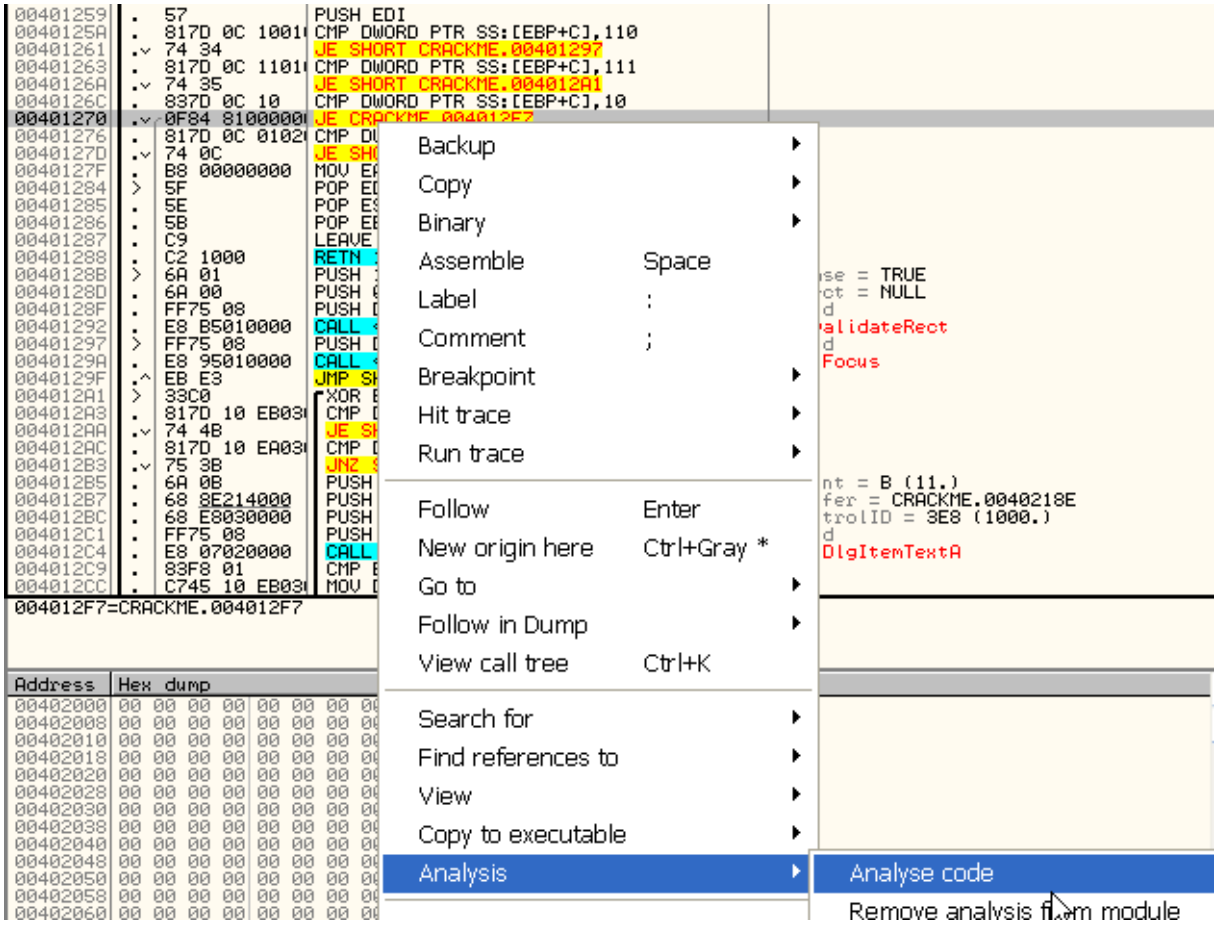


Адрес возврата имеется, но OllyDbg не выделил его как RETURN TO 401005. Чтобы понять почему он в этот раз не проявил сообразительность, нужно принять во внимание, что мы вписали CALL после того, как OllyDbg выполнил предварительный анализ кода.

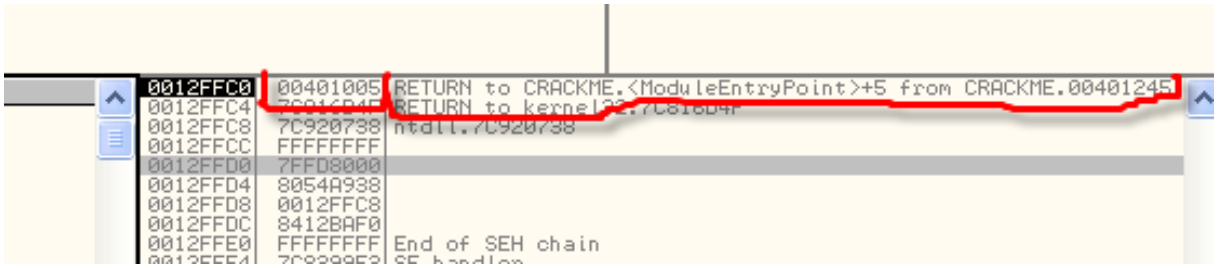
Представьте себе, что OllyDbg - это Чапаев, пересекающий реку верхом :-) Прямо посреди реки мы модифицируем его скакуна, чем, мягко говоря, дезориентируем Василия Ивановича. Точно также в растерянность впадает и OllyDbg.

(В общем, OllyDbg в воде не тонет, но пословица тут не при чём - прим. переводчика ;-))

Чтоб исправить ситуацию, достаточно нажать на правую кнопку мыши в любом месте окна дизассемблера - "Analysis" - "Analyse code". Таким образом, отладчик снова пробежится по коду анализатором и скорректирует подсказки в стековом кадре.



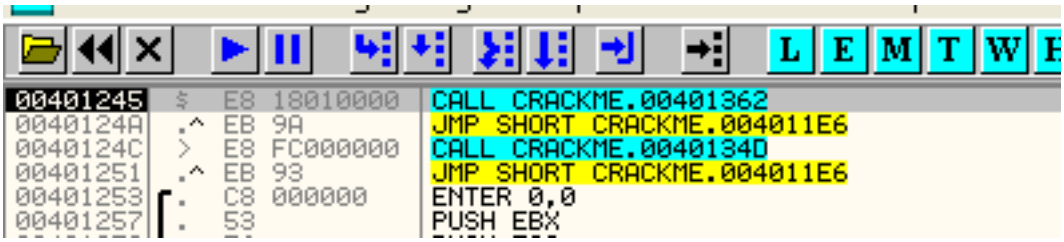
Теперь адрес возврата в верхушке стека корректно распознан анализатором:



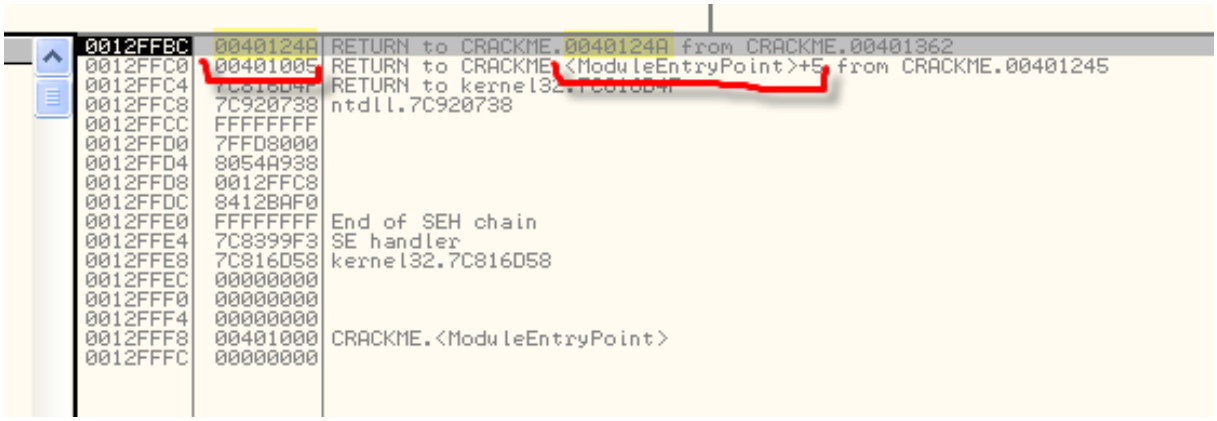
Адрес возврата в данном случае обозначен относительно стартовой точки программы:
MODULE ENTRY POINT + 5 = 401000 + 5 = 4010005.

В общем, не забывайте запускать анализатор после внесения изменений в код программы (это касается также изменений кода самой программой). Иногда анализ кода может оказаться ошибочным и его следует сбросить опцией "Analysis" - "Remove analysis from module".

Давайте вернёмся к нашему примеру.



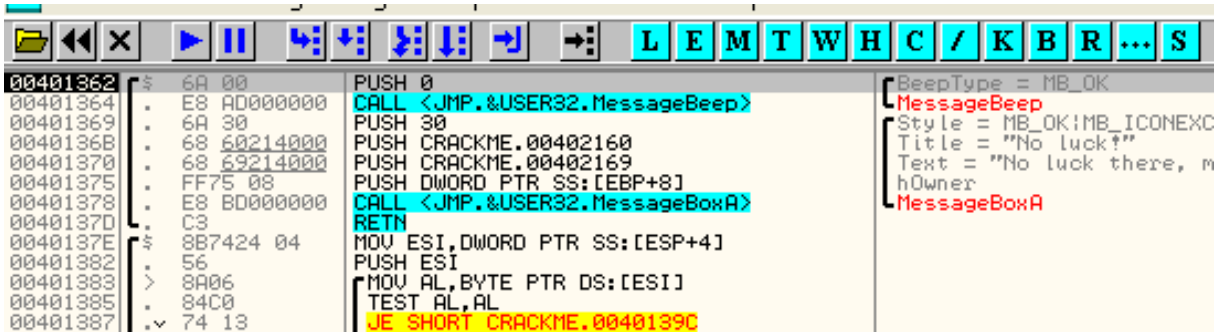
Жмём F7 и "погружаемся" в следующий CALL.



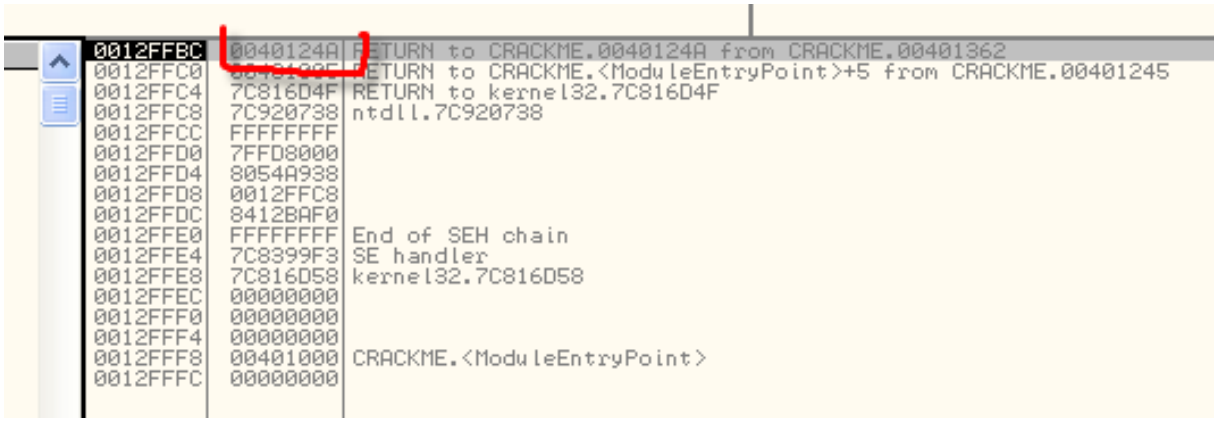
Над предыдущим адресом возврата теперь появился ещё один. В общем случае, между адресами возврата вложенных процедур могут находиться произвольные значения, попавшие в стек в результате промежуточных инструкций PUSH и других операций.

Если Вы попадёте в подобную ситуацию, например, в результате срабатывания точки останова, не имея представления о ходе выполнения программы до останова, взглянув на стек можете сориентироваться:

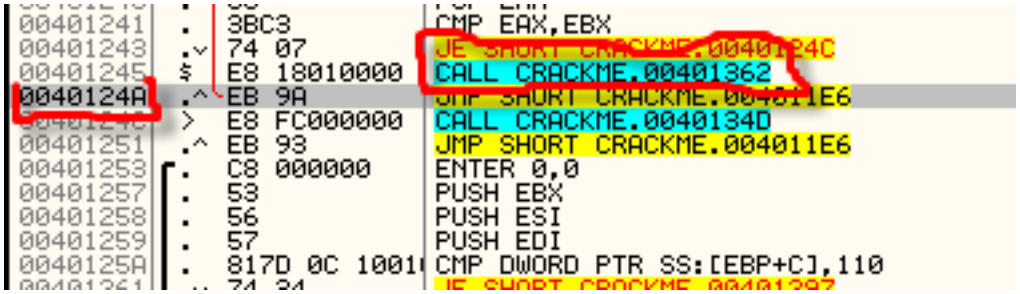
Ниже в окне дизассемблера виден RET, что наталкивает на мысль о том, что мы находимся внутри процедуры.



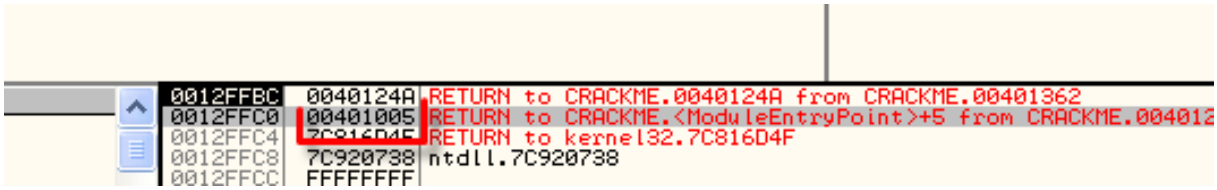
Смотрим в стек, чтобы удостовериться:



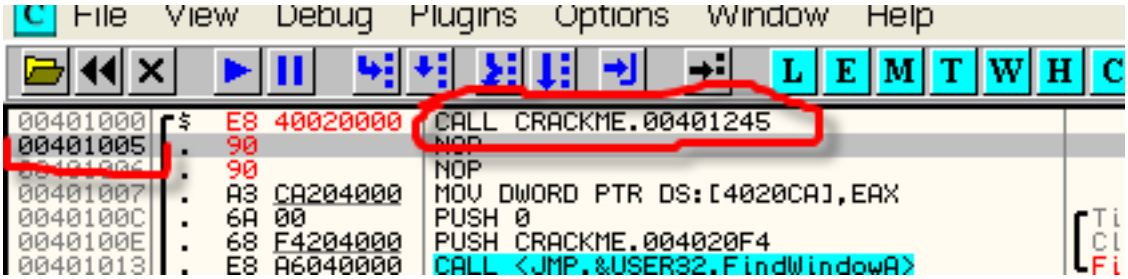
Сверху вниз первый RETURN TO указывает на возможный адрес возврата из текущей процедуры. В данном конкретном случае - это адрес 40124A.



К тому же, ниже есть ещё один RETURN TO, а это может значить, что текущая процедура была вызвана из другой процедуры.



Второй RETURN TO говорит нам, что в конечном итоге программа должна вернуться к адресу 401005 и непосредственно над этим адресом находится CALL, который инициировал всю эту цепочку вызовов. Кстати, вот он:



Привыкайте реконструировать таким образом цепочки вызовов, т.к. в подобной ситуации вместо 2х вложенных процедур может оказаться, например, 30 и трассировать их индивидуально ни у кого не хватит терпения.

Если что-то непонятно - спрашивайте. До следующей главы!

Введение в крэкинг с нуля, используя OllyDbg - Глава 8

Автор: Рикардо Нарваха, пер. Quantum

Дата: 2007-04-25

В этой главе мы рассмотрим ещё несколько базовых инструкций, которые остались без внимания в предыдущих главах. Как только мы их изучим, сможем, наконец, перейти непосредственно к крэкингу!

Циклы

Для реализации циков можно использовать инструкции, рассмотренные ранее. Например, можно назначить счётчиком какой-либо регистр общего назначения (обычно для этой цели используется ECX), проинициализировать его числом, равным количеству повторов, при каждом прохождении тела цикла уменьшать счётчик на 1, в конце цикла проверять счётчик на равенство нулю и выполнять условный переход в случае неравенства. Вот как это может выглядеть в коде:

```
        MOV ECX,15h
```

Инициализируем счётчик повторов значением 15h. Далее следует сам цикл:

```
метка_начала_цикла:
        DEC  ECX
```

Счётчик уменьшается на 1 при каждом повторе.

Далее следуют произвольные инструкции, которые мы пытаемся зациклить - это тело цикла.

В конце нужно добавить проверку на равенство нулю или любому другому условию завершения цикла:

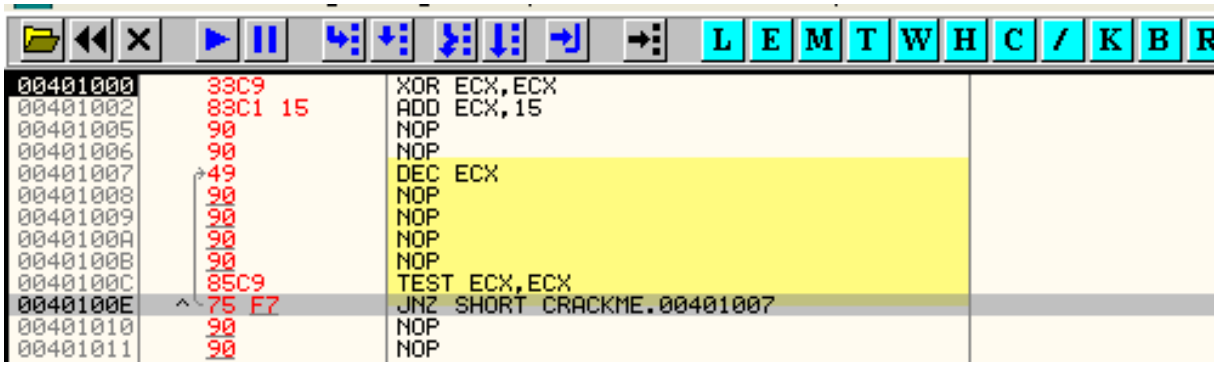
```
        CMP ECX,0
        JNE метка_начала_цикла
```

При первой проверке на равенство нулю, в счётчике будет значение 14h, ведь мы уже успели 1 раз выполнить DEC ECX. Т.к. 14h не равно 0, цикл повторится снова и снова, пока значение счётчика не достигнет значения 0.

Давайте перепишем наш цикл в более компактной форме с целью оптимизации по размеру:

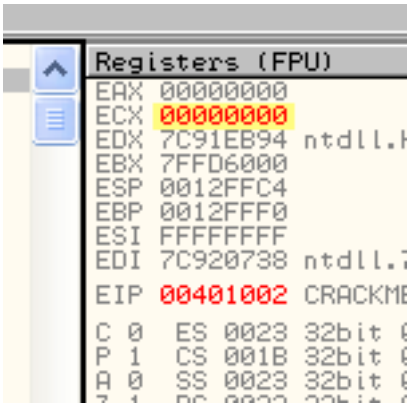
```
        XOR ECX,ECX
        ADD ECX,15h
метка_начала_цикла:
 DEC ECX
 ; тело цикла
 TEST ECX,ECX
 JNE метка_начала_цикла
```

Можно придумать ещё более компактные версии циклов, но пока ограничимся последней. Давайте впишем этот код в OllyDbg:

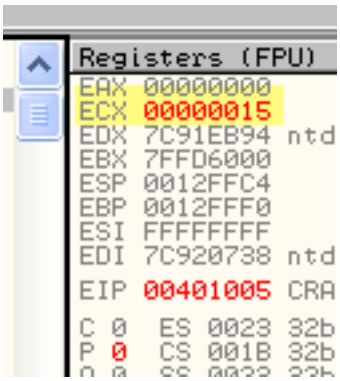


Жёлтым выделен код цикла, который будет повторяться пока значение ECX не достигнет нуля. Тело цикла содержит несколько инструкций NOP.

Давайте потрассируем код цикла, чтобы воочию убедиться в его функциональности. Нажимаем на F7 - видим как обнуляется счётчик ECX.

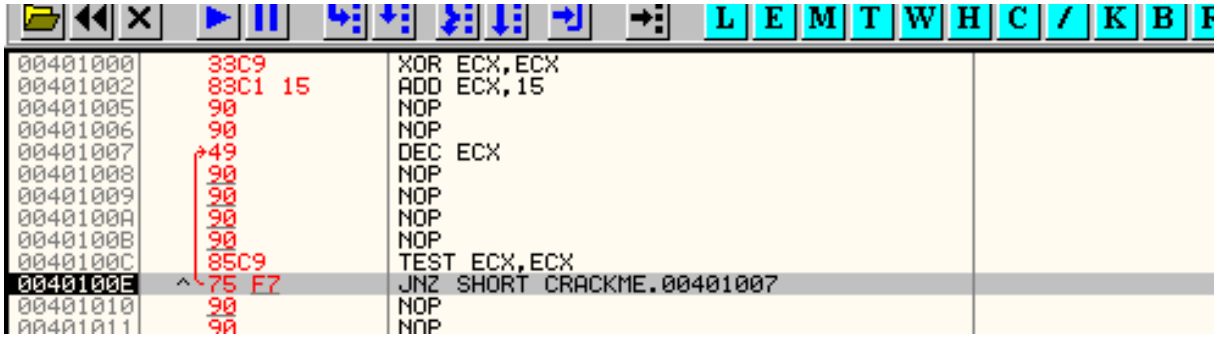


Ещё раз F7 - прибавляем к счётчику 15h, чем и задаём желаемое количество повторов.



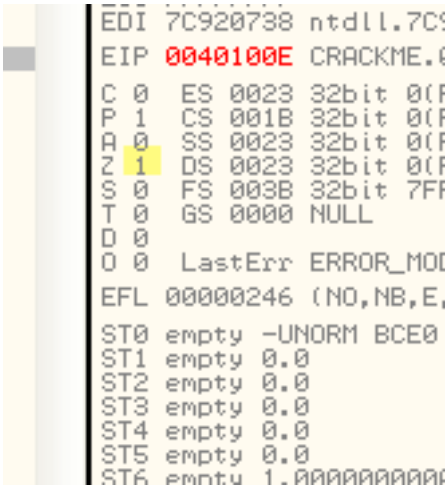
Далее нажимаем на F7 пока не выполнится инструкция DEC ECX, которая сократит значение нашего счётчика до 14h.

Продолжаем трассировать до инструкции TEST ECX, ECX. Что делает эта инструкция в данном случае? - Правильно, проверяет значение счётчика на предмет равенства нулю.

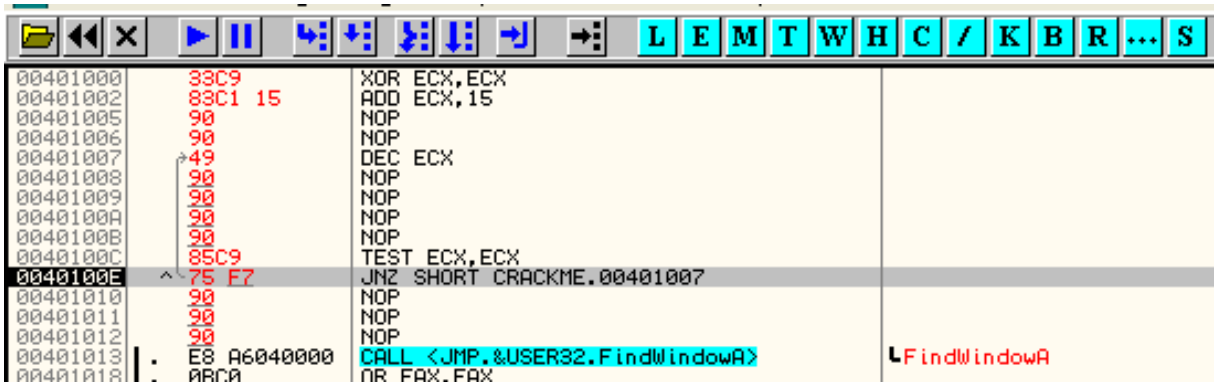


T.к. счётчик ещё не равен нулю, флаг Z не устанавливается и, следовательно, выполняется условный переход JNZ - он передаёт управление на адрес 401007. Далее счётчик снова уменьшается - на этот раз до 13h. Продолжаем трассировать пока он не достигнет значения 0.

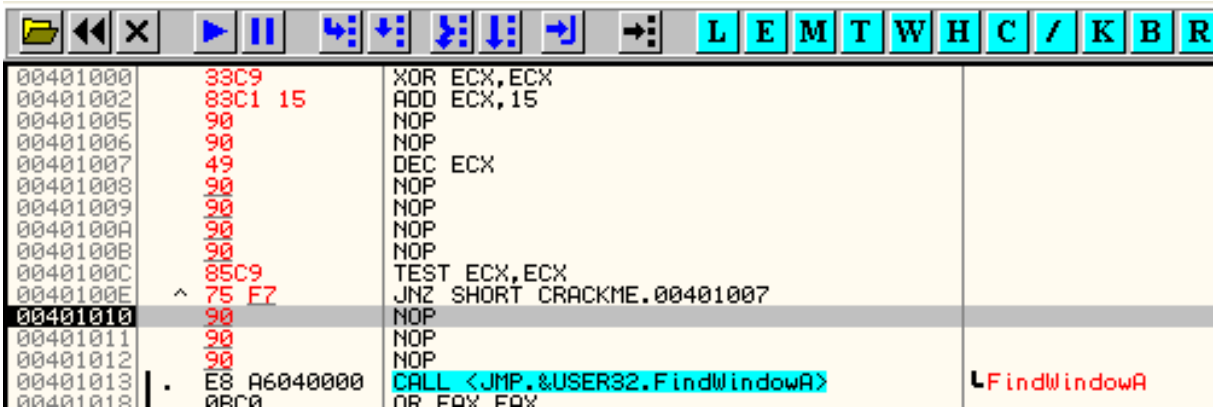
При нулевом значении счётчика, сравнение на ноль устанавливает флаг Z. Поэтому JNZ более не выполняется.



Помните, раньше мы использовали инструкцию JZ, которая осуществляла переход при установленном флаге Z? JNZ работает с точностью до наоборот - переход осуществляется при сброшенном флаге Z.



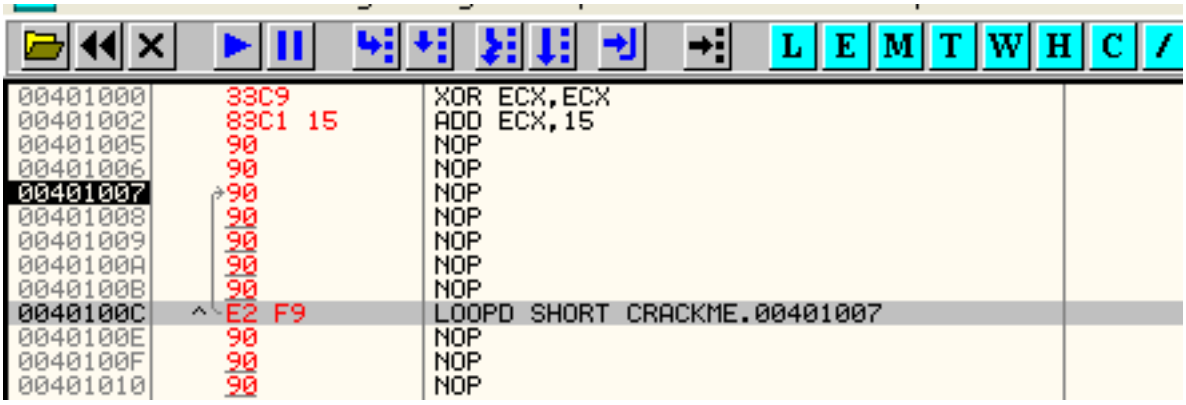
Видите серую стрелочку? - это значит, что переход не будет осуществлён. Повторное нажатие на F7 выводит нас из цикла.



Мы рассмотрели простейший пример цикла с использованием уже знакомых нам инструкций. Существуют ещё и специальные инструкции для реализации циклов. Давайте рассмотрим и их.

LOOP

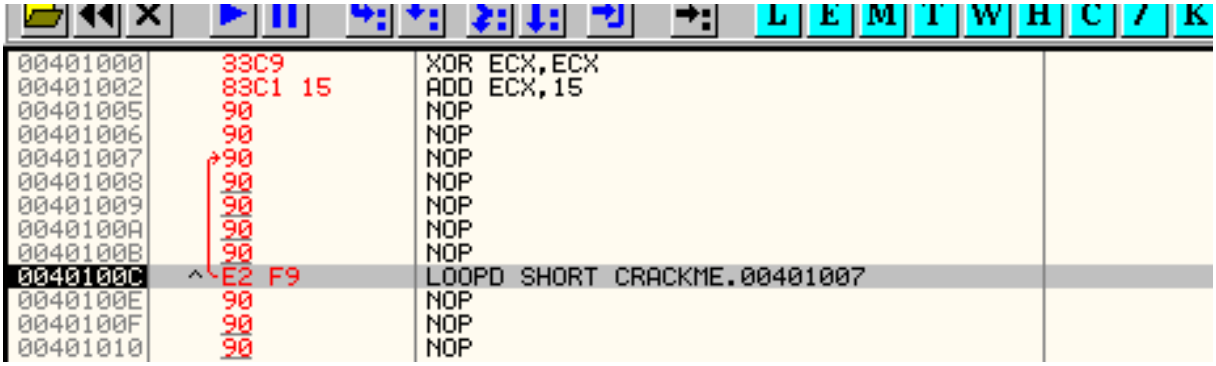
Инструкция LOOP берёт на себя уменьшение значения счётчика ECX, проверку на равенство нулю и переход по заданному адресу - всё в одном флаконе. (К сожалению, на большинстве современных процессоров данная инструкция отличается более низкой скоростью, чем аналог с DEC и JNZ - прим. пер.)



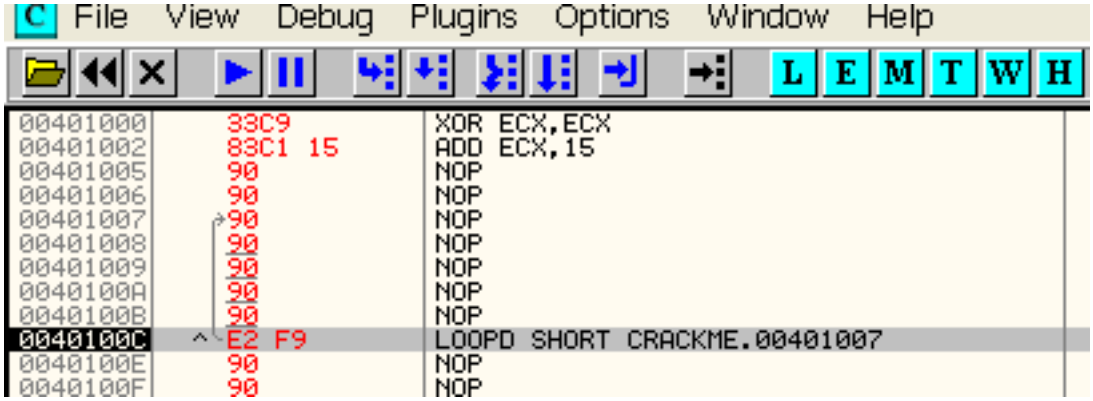
Подсвечиваем инструкцию DEC ECX - осуществляем правый клик мышью - Binary - Fill with NOPs. Аналогично поступаем с инструкциями TEST ECX,ECX и JNZ 401007. Все 3 инструкции можно заменить одной - LOOP 401007.

Подсвечиваем первую строчку (401000) - осуществляем правый клик мышью - New origin here. Теперь мы готовы опробовать наш новый цикл, использующий инструкцию LOOP.

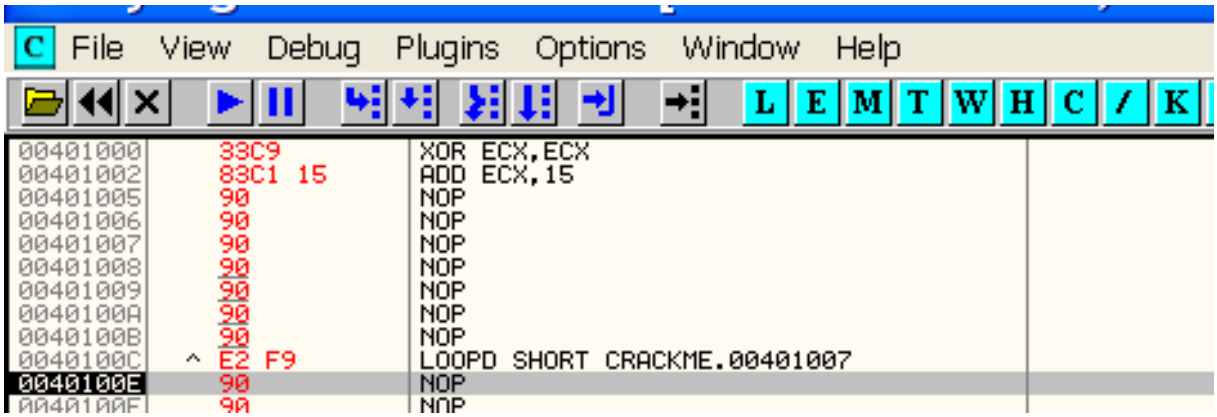
Нажимаем F7 - снова видим как счётчик ECX сначала инициализируется значением 0, потом ему присваивается значение 15h. Продолжаем трассировать пока не достигнем инструкцию LOOP.



Также как и раньше, осуществляется переход на адрес 401007, т.к. счётчик ещё не равен нулю. Замечаем, что счётчик уменьшился на 1 - теперь в ECX хранится значение 14h.



Продолжаем трассировать. Когда счётчик, наконец, достигает нуля, цикл более не будет повторяться.



Существуют некоторые вариации инструкции LOOP:

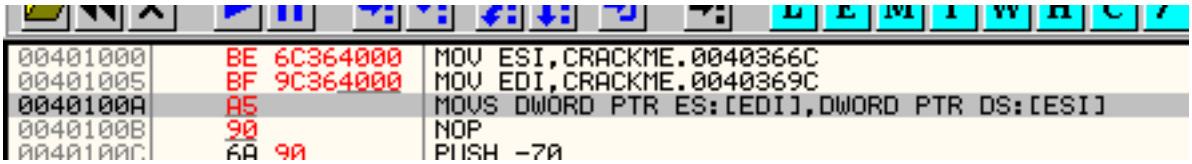
• LOOPZ, LOOPE Цикл повторяется пока флаг Z установлен
• LOOPNZ, LOOPNE Цикл повторяется пока флаг Z сброшен

Обе вариации работают как обычный LOOP, т.е. повторяют тело цикла пока счётчик не достигнет нуля, уменьшая на 1 значение счётчика при каждом повторе. В добавок, LOOPZ и LOOPNZ ещё проверяют значение флага Z и прерывают цикл если Z сброшен или установлен соответственно. Таким образом, цикл можно прервать досрочно (до того как ECX достигнет нуля), используя флаг Z.

Обратите внимание на поведение цикла в том случае, если счётчик изначально равен нулю. - прим. пер.

Цепочечные инструкции Далее рассмотрим основные инструкции, относящиеся к категории цепочечных, т.е. позволяющие обрабатывать сразу цепочки байт. **MOVS**

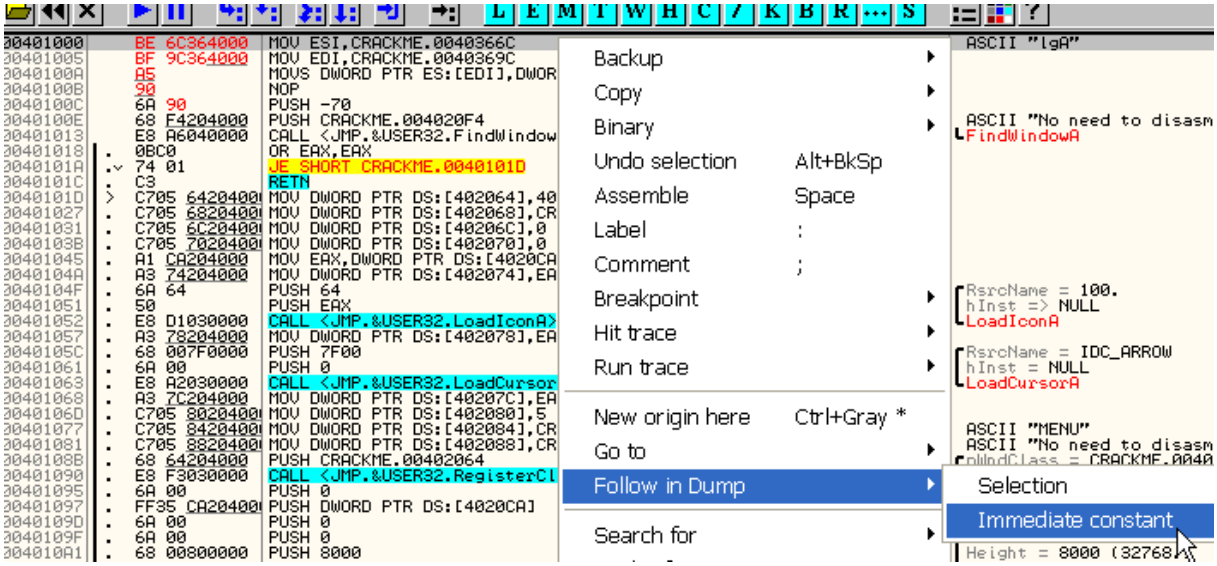
Данная инструкция копирует данные из одного адреса в другой. Адрес источника хранится в регистре ESI, адрес приёмника - в EDI. Инструкция не нуждается в явном указании параметров, но OllyDbg дизассемблирует её как MOVS DWORD PTR ES:[EDI],DWORD PTR DS:[ESI] для наглядности. Вот как это выглядит:

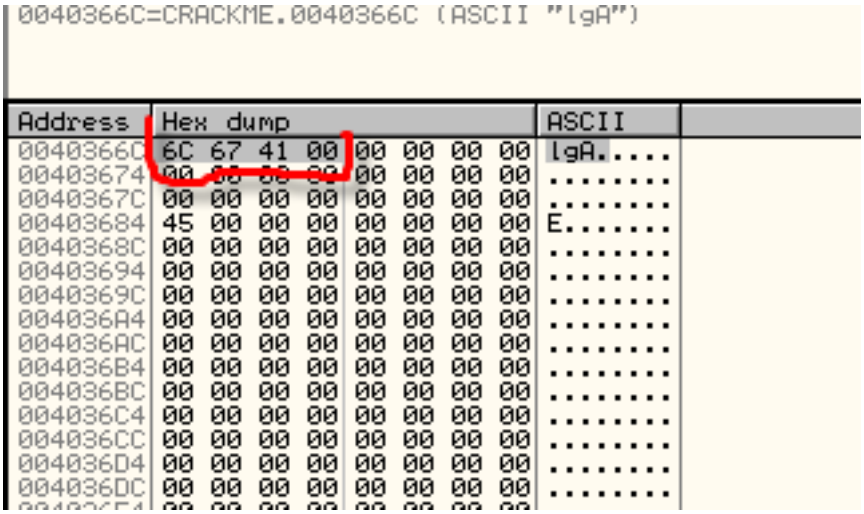


Предварительно инициализируем ESI адресом ячейки, содержимое которой нужно скопировать (источник), и EDI - адресом ячейки, в которую нужно осуществить копирование (проёмник).

Давайте посмотрим содержимое обоих ячеек в окне дампа непосредственно перед копированием.

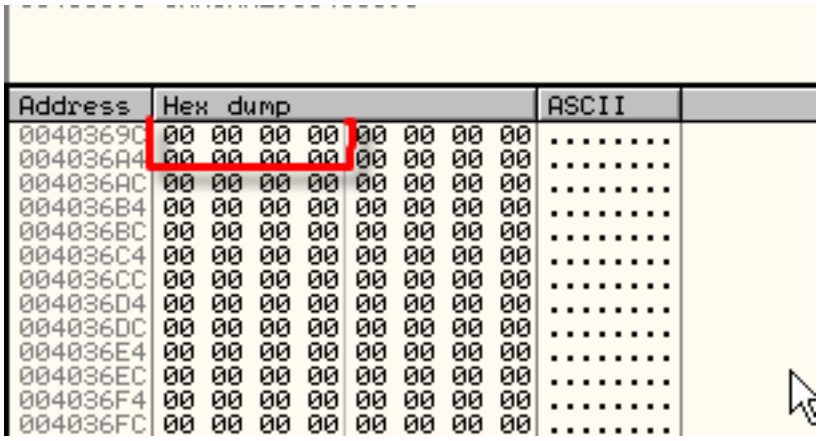
В окне дампа можно задать Go to - Expression = 40366C. Есть более простой способ: Follow in dump - Immediate constant, как показано ниже:





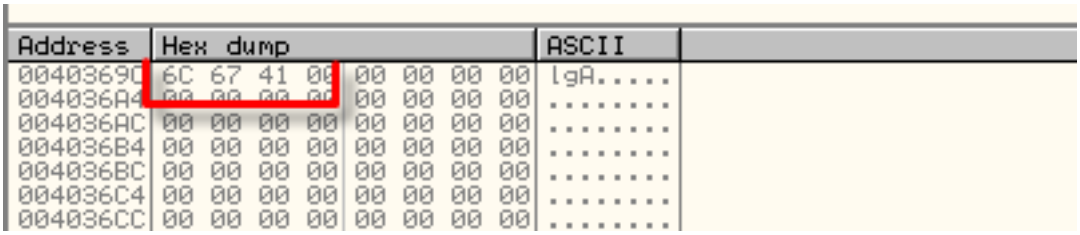
В дампе выделено содержимое ячейки, адрес которой указан в источнике (ESI).

А теперь посмотрим куда указывает EDI:



Это приёмник - сюда будет скопировано содержимое ячейки-источника.

Нажимаем F7 пока не выполнится MOVS - в приёмнике теперь находятся те самые 4 байта.



MOVS копирует 4х-байтные значения, т.е. DWORD, из-за чего этой инструкции соответствует ещё одна мнемоника - MOVSD. Существуют ещё инструкции MOVSW и MOVSB, копирующие 2 байта (WORD) и 1 байт (BYTE) соответственно.

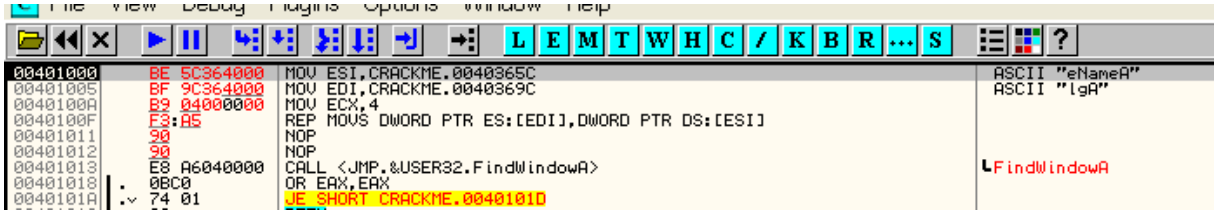
Обратите внимание как изменяются значения ESI и EDI при копировании, в зависимости от флага D - прим. пер.

REP

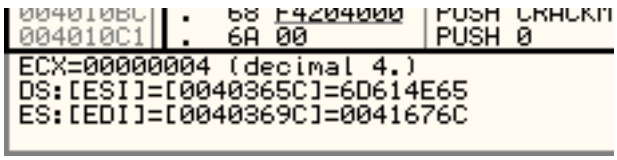
Это префикс, который можно использовать перед некоторыми инструкциями, в частности MOVS. Данный префикс указывает, что текущую инструкцию нужно выполнить ECX количество раз. Фактически счётчик ECX уменьшается при каждом повторе, как и в обычном цикле. Таким образом, REP MOVS копирует уже не строго 4 байта, а массив размером 4 * ECX байт, ведь каждый раз указатели ESI и EDI сами увеличиваются на 4 (или уменьшаются, в зависимости от флага D - прим. пер). Полезная фишка, не так ли?

Нет более простого и компактного способа скопировать произвольное количество байт из одного адреса в другой, хотя производительность данного способа на современных процессорах нельзя назвать оптимальной.

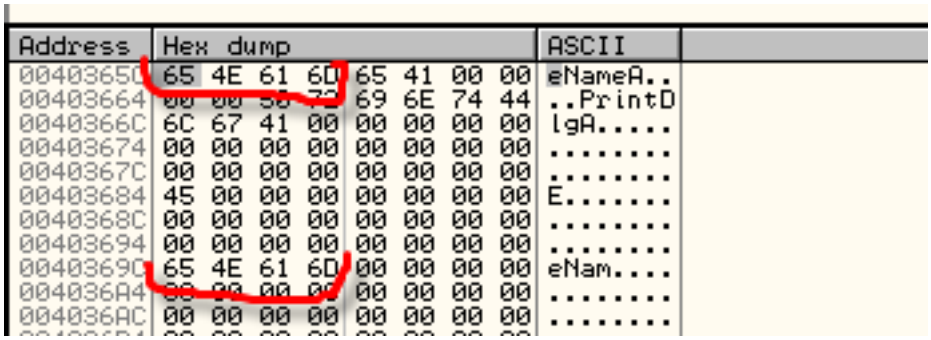
Давайте немного подправим наш предыдущий пример: добавим префикс REP и проинициализируем ECX.



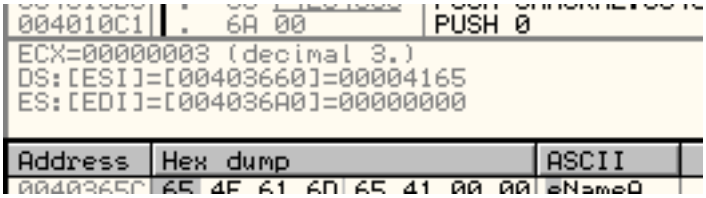
Адрес источника теперь будет 40365C (всего лишь для наглядности), а приёмника - тот же, что и раньше, т.е. 40369C. Возвращаемся к первой строчке кода дежурным New origin here и трассируем до REP MOVS.



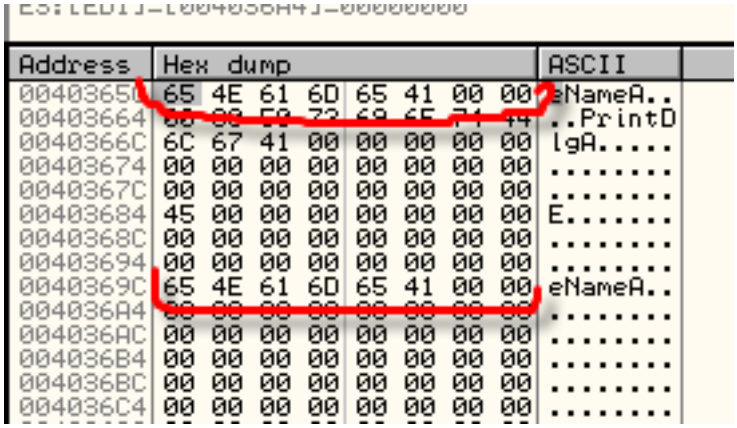
Обратите внимание на подсказку в OllyDbg - там отображаются адреса источника и проёмника, содержимое начальных ячеек и другая полезная информация. Нажимаем F7.



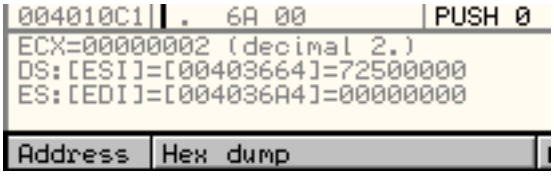
Как видите, первые 4 байта были успешно скопированы, но мы всё ещё находимся на REP, ведь цикл должен выполняться пока ECX не достигнет нуля. На данном этапе в ECX уже хранится значение 3, а указатели ESI и EDI уже увеличились на 4, указывая таким образом на следующие 4 байта, которые будут скопированы как только мы снова нажмём F7.



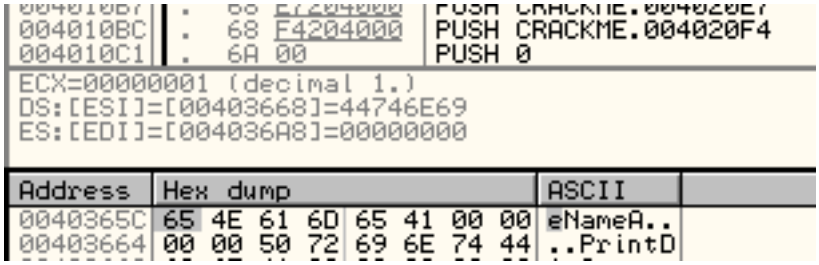
Нажимаем F7.



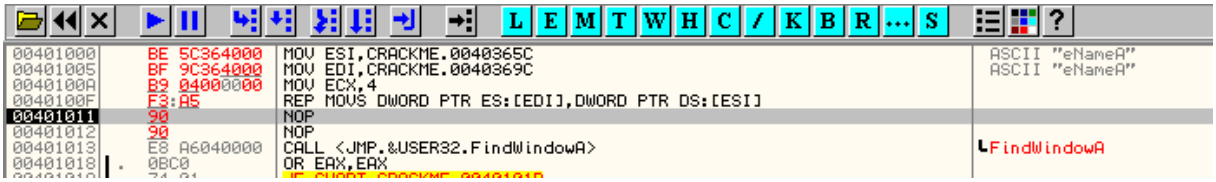
Вот и следующие 4 байта были удачно перемещены и счётчик ECX уже равен 2.



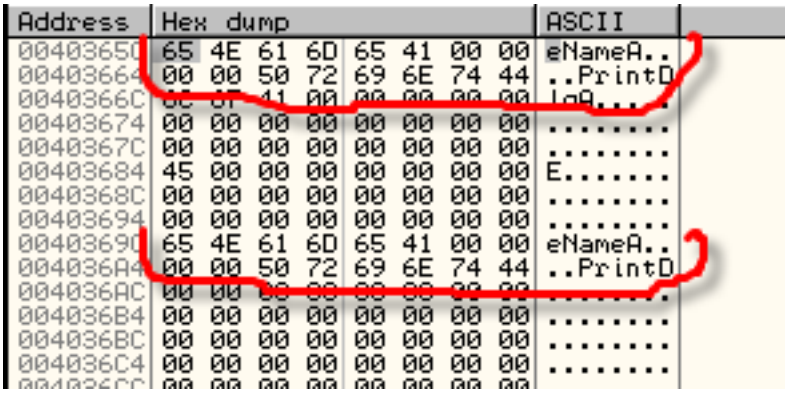
Ещё раз F7 - ECX равен 1.



И ещё раз.



Всё, цикл завершился, т.к. счётчик достиг нуля.



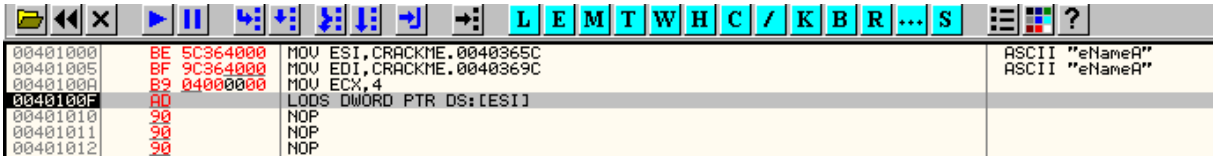
Подведём итог: инструкция MOVS выполнилась 4 раза, благодаря префиксу REP, тем самым переместив цепочку из 16 байт из одного массива в другой.

Имейте в виду, что MOVS, также как и обычная MOV, не может записать данные по адресу, принадлежащему странице не имеющей прав на запись. В таком случае будет вызвано исключение.

Кроме REP есть ещё 2 производных префикса REPE/REPZ и REPNE/REPNZ, которые позволяют прервать цикл досрочно по флагу Z. Т.к. MOVS никак не влияет на флаг Z, использовать REPE/REPZ или REPNE/REPNZ совместно с MOVS не имеет особого смысла, но есть и другие инструкции, которые также поддерживают префиксы повторения и, в добавок, влияют на флаги. Например, CMPS и SCAS являются таковыми. Одну из них мы рассмотрим чуть позже.

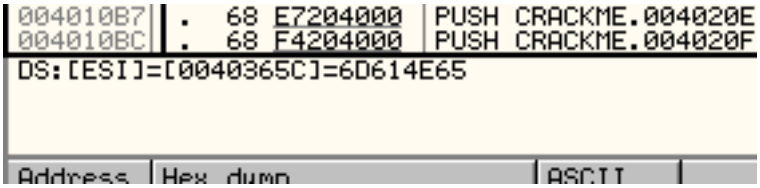
LODS

Данная инструкция загружает (от англ. LOAD) данные из источника (куда, как обычно, указывает ESI) в аккумулятор.

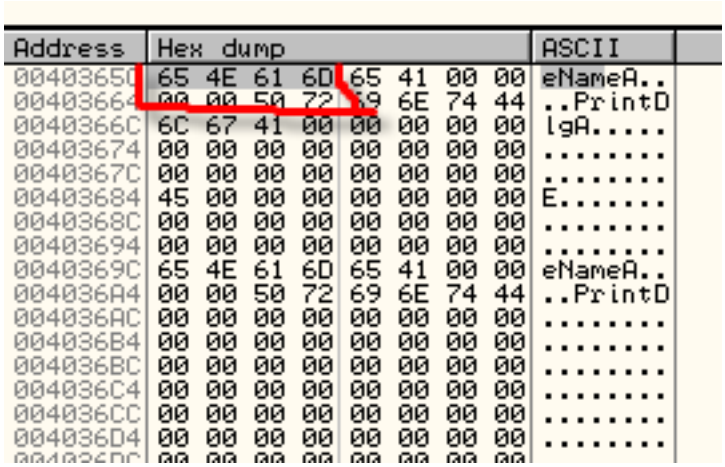


OllyDbg в своём репертуаре: дизассемблирует LODS как LODS DWORD PTR DS:[ESI].

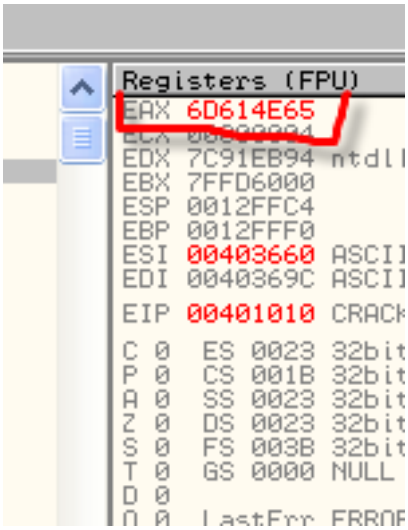
Если кому-то лень посмотреть на значение ESI на панели регистров, OllyDbg заботливо дублирует его в окне подсказок.



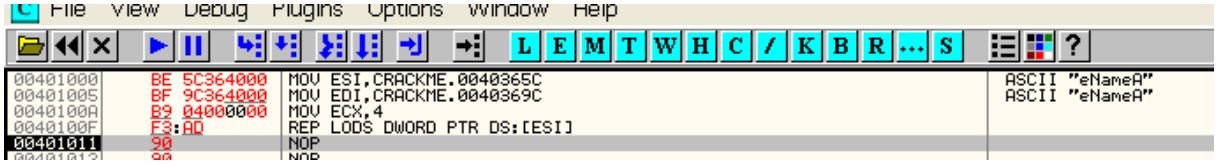
Что у нас в дампе?



Эти 4 байта будут загружены в аккумулятор (EAX). Нажимаем F7.

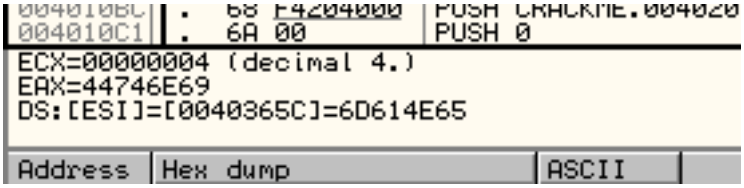


Свершилось! EAX действительно содержит те самые 4 байта.

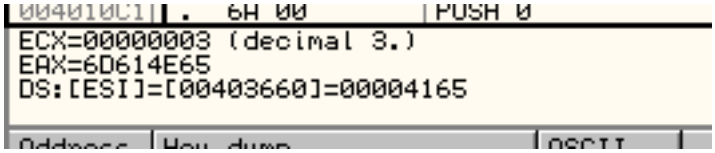


Префикс REP можно применить и перед инструкцией LODS для автоповтора, пока счётчик ECX не достигнет нуля.

При выполнении цикла REP LODS, в окне подсказки оказываются текущие значения ECX, ESI и содержимое ячеек памяти, которые будут загружены в аккумулятор при следующем повторе.



Нажимаем F7.



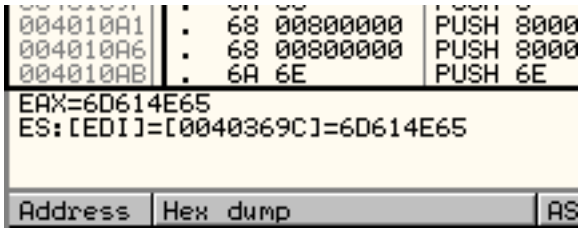
ECX теперь имеет значение 3 и адрес в ESI увеличился на 4 - теперь он указывает на следующие 4 байта, которые будут загружены в EAX.

Есть ещё инструкции LODSW и LODSB для загрузки 2-байтовых (WORD) и 1-байтных (BYTE) значений соответственно.

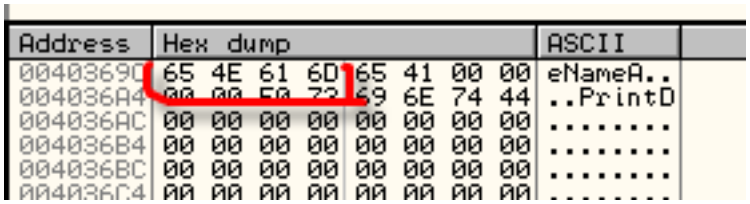
STOS

Сохраняет (от англ. STORE) значение аккумулятора по адресу приёмника EDI.

При трассировании данного примера, когда очередь доходит до инструкции STOS, в окне подсказки появляется текущее значение аккумулятора и адрес, по которому будет сохранено это значение.



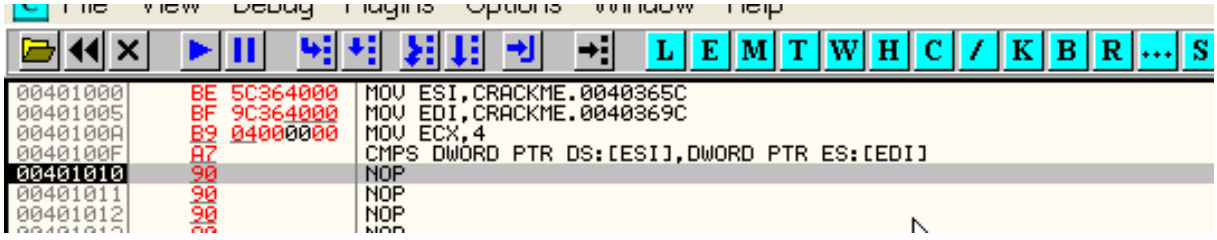
Вот результат в дампе:



Также как и в предыдущем случае, можно использовать префикс REP для автоповторения и существуют 2-байтовая и 1-байтная версии: STOSW и STOSB.

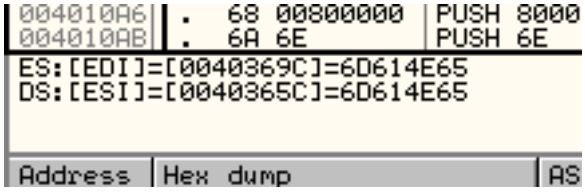
CMPS

Осуществляет сравнение ячеек, указанных в источнике (ESI) и приёмнике (EDI).

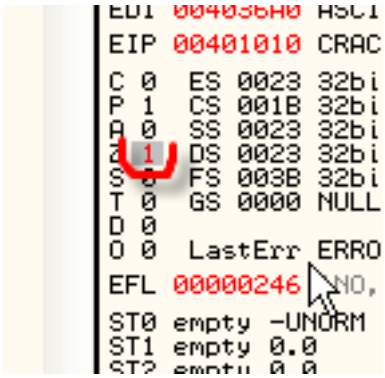


OllyDbg дизассемблирует данную инструкцию как CMPS DWORD PTR DS:[ESI],DWORD PTR ES:[EDI].

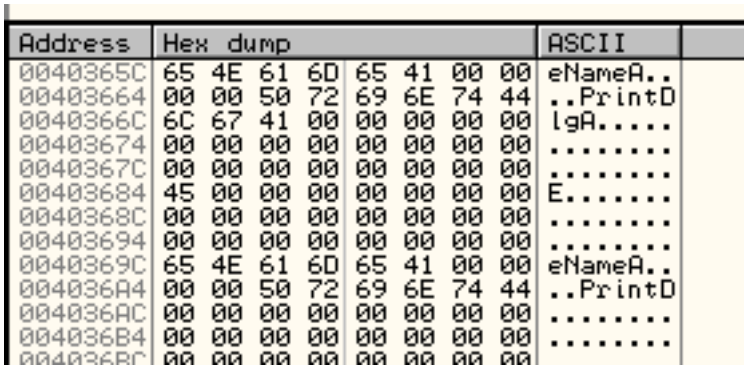
В окне подсказки отображаются значения, подлежащие сравнению:



Как Вы уже знаете, сравнение кроет за собой арифметическую операцию вычитания. В данном случае значения равны и в результате мы получаем ноль, а это значит, что флаг Z будет установлен.

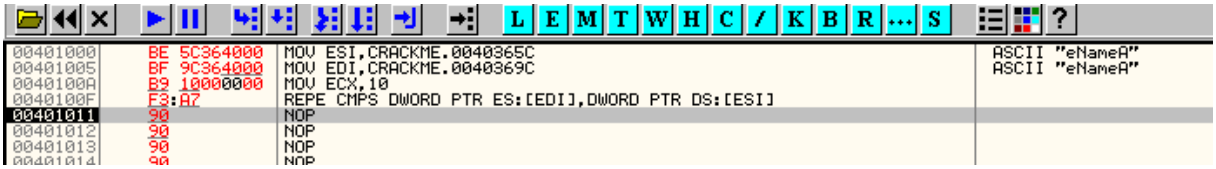


Т.к. операция сравнения влияет на состояние флага Z, можно использовать префикс REPE или REPZ для цепочечного сравнения до тех пор пока не истощится счётчик ECX или флаг Z не окажется сброшен.

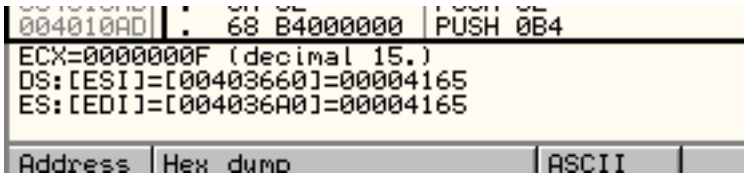


От предыдущих примеров по адресам 40365C и 40369C осталось то, что видим в дампе.

Инициализируем ECX = 10h. При использовании префикса REPE, цикл прервётся когда ECX станет равен нулю или же по состоянию флага Z - любое из этих условий приведёт к завершению цикла.

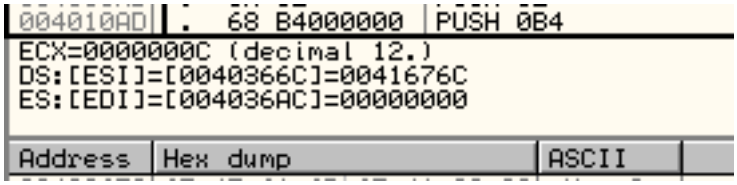


Трассируем до строчки с REPE. Значения в обоих ячейках оказываются равны, что приводит к установке флага Z.

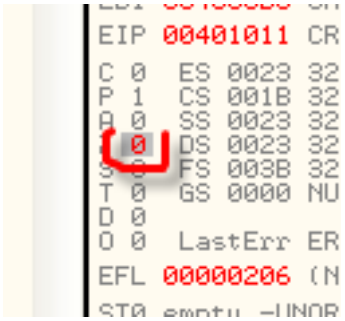


Установленный флаг Z сигнализирует REPE, что пока не нужно прерывать цикл. Нажимаем F7.

Продолжаем нажимать F7 до тех пор, пока операнды не окажутся разными. В данном случае это должно произойти до того как истощится счётчик ECX. Ага, вот:



Последующее нажатие на F7 сбрасывает флаг Z и цикл прерывается.



Если бы все 16 двойных слов оказались равными в обоих цепочках, цикл завершился бы по условию ECX = 0, а не раньше.

Ранее упоминался префикс REPNZ, который тоже применим к CMPS и позволяет прервать цикл досрочно, но условие в этом случае противоположно только что рассмотренному REPE/REPZ.

Мы уже изучили основную массу наиболее полезных инструкций, за исключением, пожалуй, только инструкций FPU (для операций над дробными числами), которые мы рассмотрим позже. Не лишним будет повторить пройденный материал, если у Вас остались какие-то пробелы.

СПОСОБЫ АДРЕСАЦИИ

Прямая адресация

Это самый простой способ - адрес явно указывается в одном из операндов инструкции. Например:

```
MOV dword ptr [00513450], ecx
MOV ax, word ptr [00510A25]
MOV al, byte ptr [00402811]
CALL 452200
JMP 421000
```

Не нужно производить какие-либо вычисления для разрешения адреса, т.к. его значение указано открытым текстом в листинге.

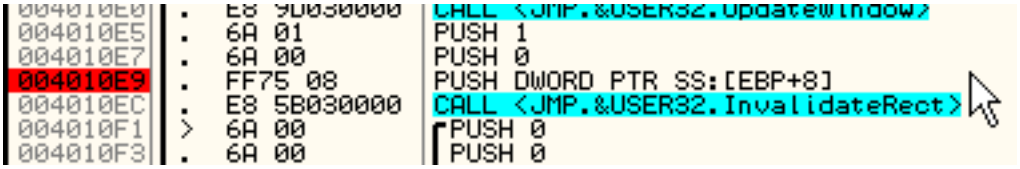
Косвенная адресация

```
   MOV dword ptr [eax], ecx
CALL EAX
JMP [ebx + 4]
```

Чтобы узнать настоящий адрес в любой из предыдущих инструкций, необходимо остановить отладчик на нужной инструкции и посмотреть значения регистров или, что гораздо удобнее, посмотреть в окно подсказки.

Во многих программах используется косвенная адресация, чтобы усложнить задачу реверсера, ведь начальный анализ, проводимый отладчиком, не покажет настоящие адреса и, в общем случае, придётся дожидаться пока не настанет черёд выполнения этих инструкций, чтобы посмотреть значения регистров непосредственно перед этим и, таким образом, получить интересующие нас адреса.

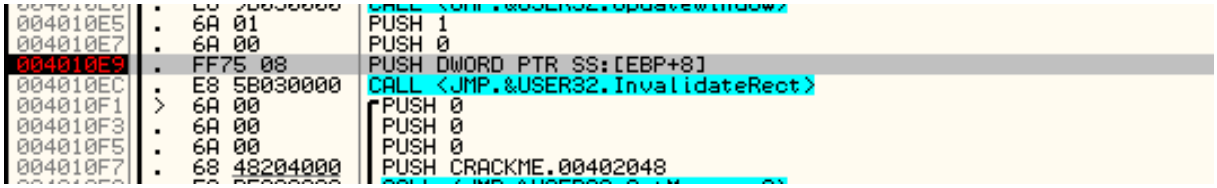
Чтобы было нагляднее, давайте снова воспользуемся крэкми Cruehead'a. Загружаем его в отладчик.



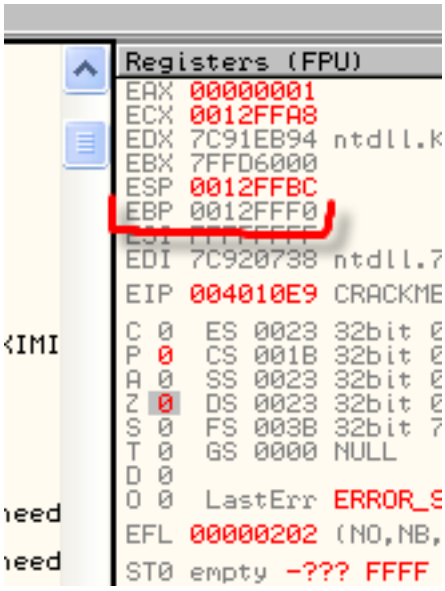
Подмечаем пример косвенной адресации: PUSH [ebp+8]

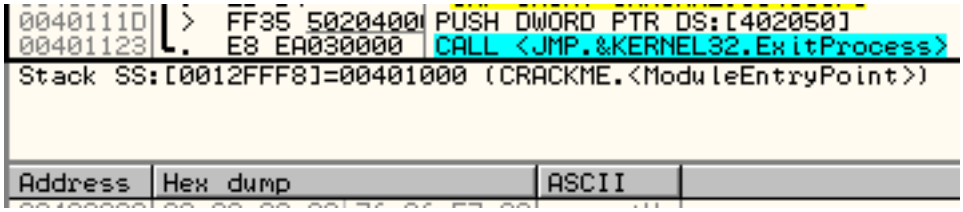
Т.к. мы всё ещё находимся на точке входа (Entry Point), мы не можем заранее узнать значение регистра EBP к моменту выполнения того самого PUSH'a. Поэтому, нам не известно пока ещё что именно попадёт в стек. Давайте выделим данный PUSH (он находится по адресу 401009) и нажмём F2, чтобы повесить на него точку останова (бряк).

Потом нажимаем F9 (RUN) - крэкми запускается и вскоре останавливается там, где мы установили бряк.



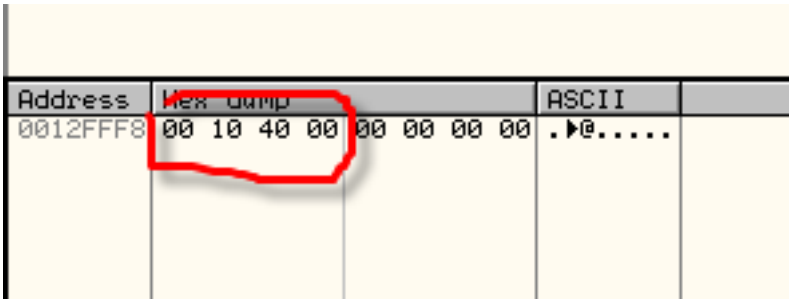
В окне подсказки высвечивается настоящий адрес EBP+8 = 12FFF8. Кстати, у Вас адрес может отличаться - это свойственно стековым адресам! Значение EBP в данный момент у меня равно 12FFF0, на основании чего и получилось, что EBP+8 = 12FFF8.



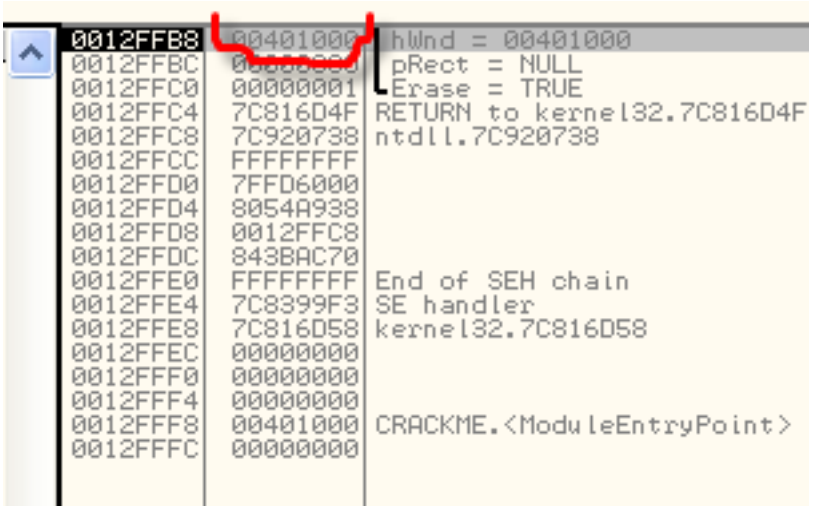


Теперь давайте посмотрим, что хранится в дампе по адресу EBP+8 : Go to - Expression = EBP+8

И что мы там видим?



Это значение будет помещено в стек. Нажимаем F7 - так и есть.



В случае любой инструкции с косвенной адресацией, настоящее значение адреса можно получить только в момент выполнения данной инструкции.

Существуют другие способы адресации, но мы их частично уже рассмотрели в примерах. На данном этапе мне нечего добавить.

Кто смог осилить весь материал (включая, наверное, и переводчика :) - прим. пер.) имеет неплохие шансы познать захватывающий мир крэкинга, но для этого понадобится ещё очень много практики и чтения серьёзной литературы. В 9й главе мы, наконец, окунёмся в азы прикладного крэкинга.

Перевод последнего абзаца получился немного вольным, но смысл сохранился. Особенно хочется подчеркнуть, что автор не спроста упоминает о серьёзных книгах - учебниках по ассемблеру, архитектуре ПК, операционных систем и т.д. Многие теоретические основы, касающиеся ассемблера вообще и крэкинга в частности, не попадают в туториалы и только лишь книги помогают до конца разобраться в тех или иных темах.

Введение в крэкинг с нуля, используя OllyDbg - Глава 9

Автор: Рикардо Нарваха, пер. Quantum

Дата: 2007-06-20

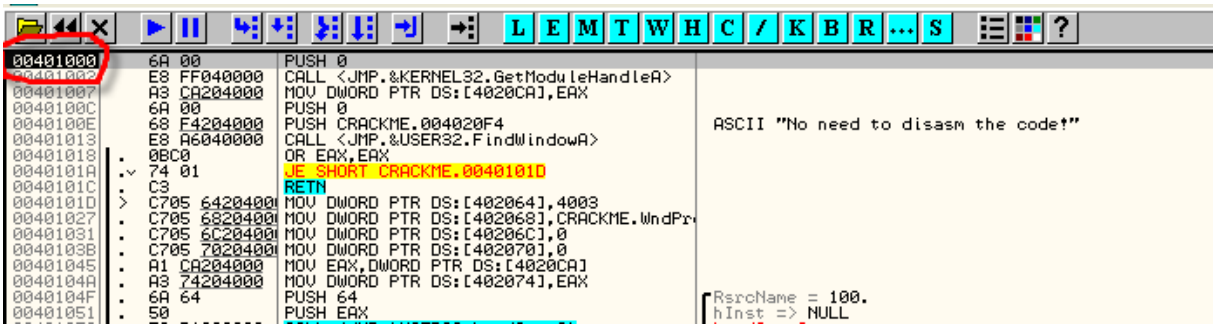
Как и было обещано, теперь мы займёмся непосредственно крэкингом. Начнём с базовых понятий, не спеша, шаг за шагом пополняя наш крэкерский арсенал знаний. Теоретические пробелы мы постараемся заполнить по ходу дела.

Экспериментировать мы будем, естественно, с уже знакомым нам крэкми Cruehead'a, но не ограничимся рассмотрением различных способов взлома этого несложного крэкми - по ходу мы постараемся изучить стандартные приёмы, применимые в дальнейшем к более "продвинутым" жертвам.

Итак, давайте загрузим крэкми в отладчик.

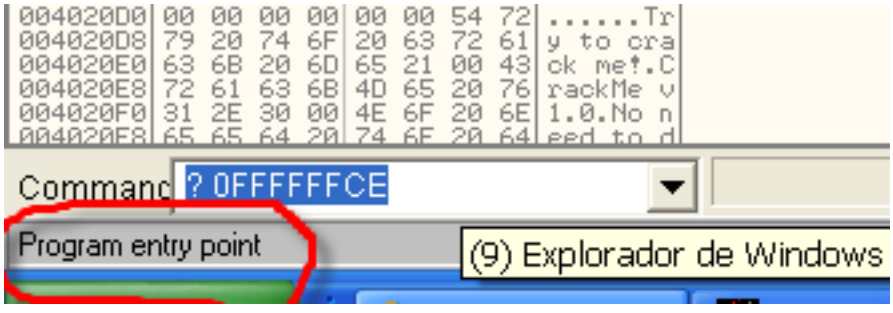
На примере данного крекми можно рассмотреть несколько общих понятий.

Точка Входа (Entry Point). Адрес инструкции, с которой стартует программа. Не путать с понятием OEP (Original Entry Point), которое мы рассмотрим чуть позже. При загрузке приложения в OllyDbg, отладчик останавливается на точке входа, анализирует код и ждёт дальнейших инструкций от пользователя.



-

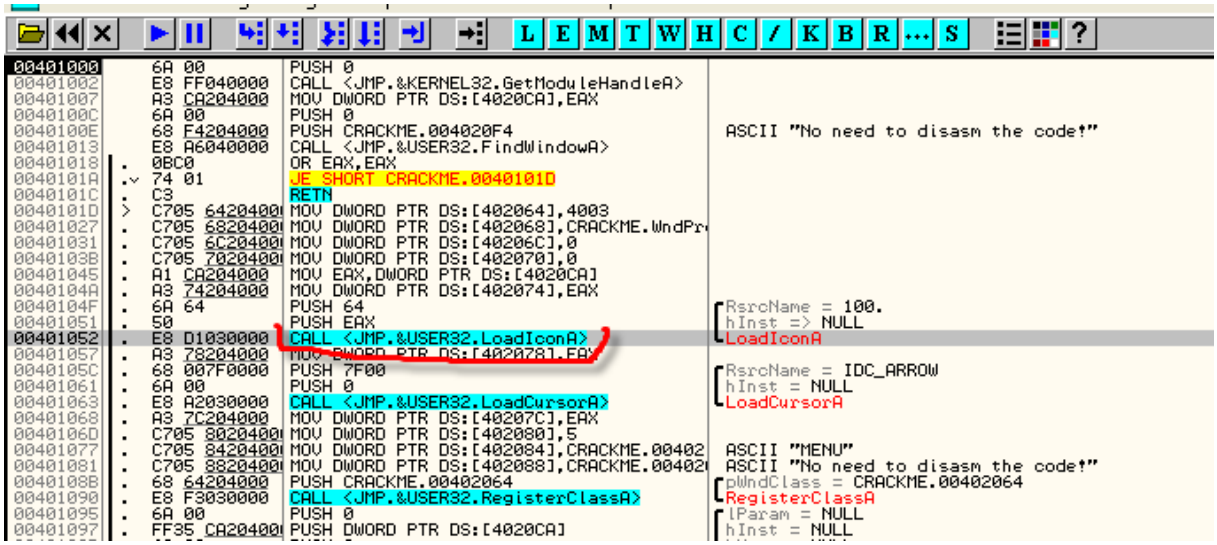
В данном случае точка входа соответствует адресу 401000. Обычно в строке состояния выводится причина, по которой остановлен отладчик. Сейчас там находится сообщение о том, что мы находимся на точке входа:



-

Почти все программы на старте останавливаются на точке входа. Есть такие, которые не останавливаются, т.к. используют способы обхода отладчика. Об этих способах мы поговорим позже. Пока что примите к сведению существование подобных приёмов антиотладки.

Давайте, заодно, рассмотрим понятие DLL (динамические библиотеки) и экспортируемые ими функции.



-

Обратите внимание на выделенную инструкцию. Вместо адреса, вроде CALL 401020, в данном случае указано имя внешней функции:

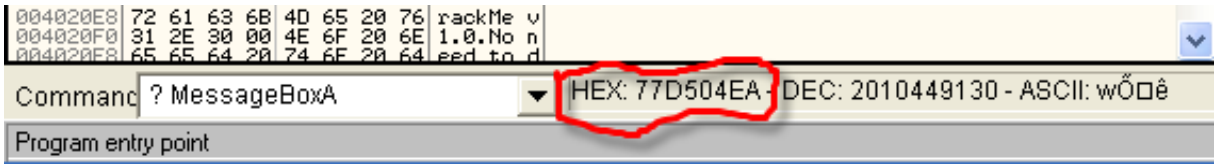
```
CALL LoadIconA
```

В крайнем правом столбике выводится какая-то дополнительная информация, но что представляет собой LoadIconA?

Операционная система Windows поддерживает так называемые динамические библиотеки (файлы с расширением DLL), которые имеют тотже формат, что и обычные исполнимые файлы EXE. В динамических библиотеках содержатся функции, которые могут быть использованы другими исполнимыми файлами (EXE и DLL). Вместо статического копирования одной и тойже функции в несколько экзешников, её можно поместить в DLL, а в экзешниках указать ссылки на эту DLL. Если обьём подобных функций велик, получается неплохая экономия на размере экзешников и, что более существенно, расходуемой памяти. Базовые функции для работы с файлами, динамической памятью, процессами и потоками, графикой, звуком, сетью и т.д. и т.п. реализованы в стандартных динамических библиотеках. LoadIconA как раз и является одной из стандартных функций, реализованной в одной из стандартных библиотек - в USER32.DLL. Стандартные функции ещё называют API.

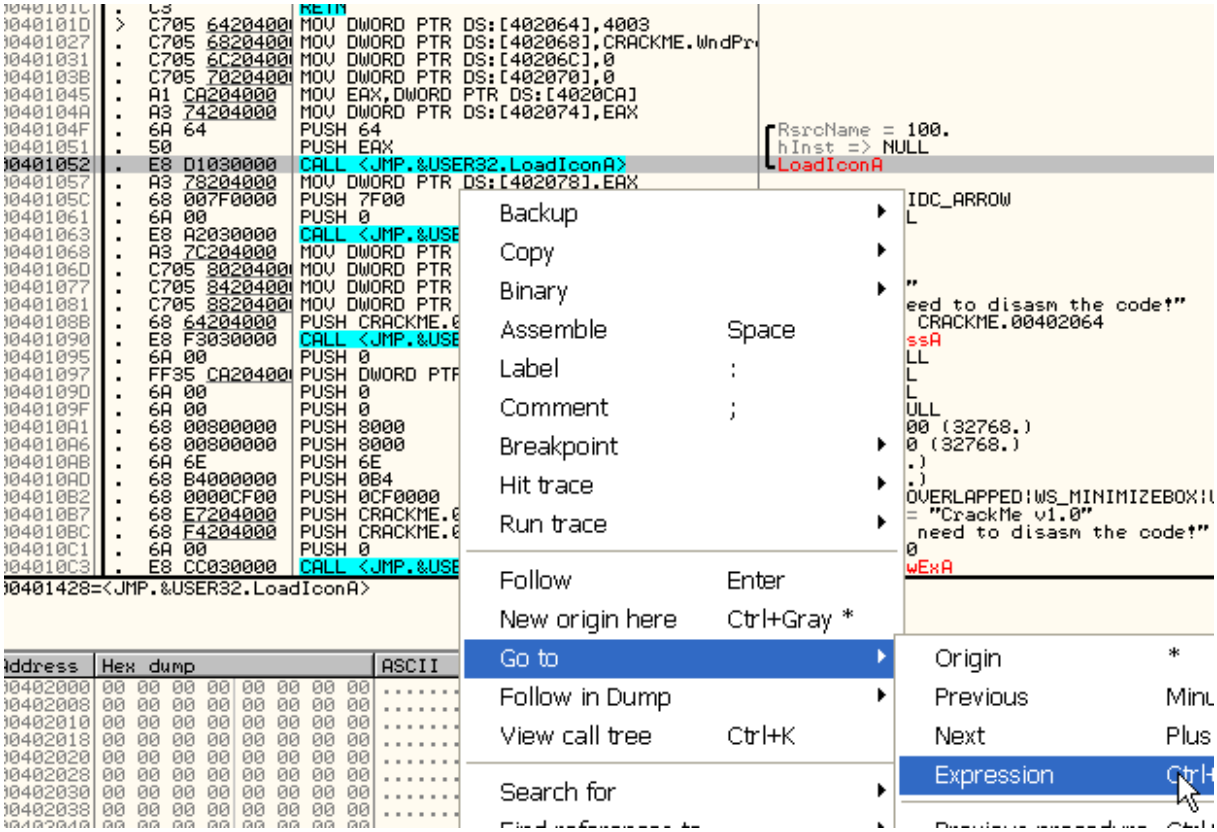
Давайте рассмотрим пример другой API'шной функции - MessageBoxA.

В окне Command Bar вводим: ? MessageBoxA



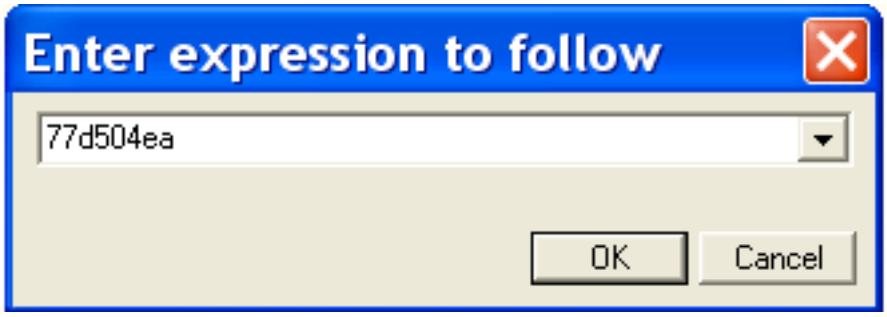
-

Получаем короткий отчёт, в котором первым делом указан настоящий адрес данной функции. Давайте пройдём по этому адресу, т.е. воспользуемся командой Go to - Expression и введём тот адрес, что выдал нам Command Bar.



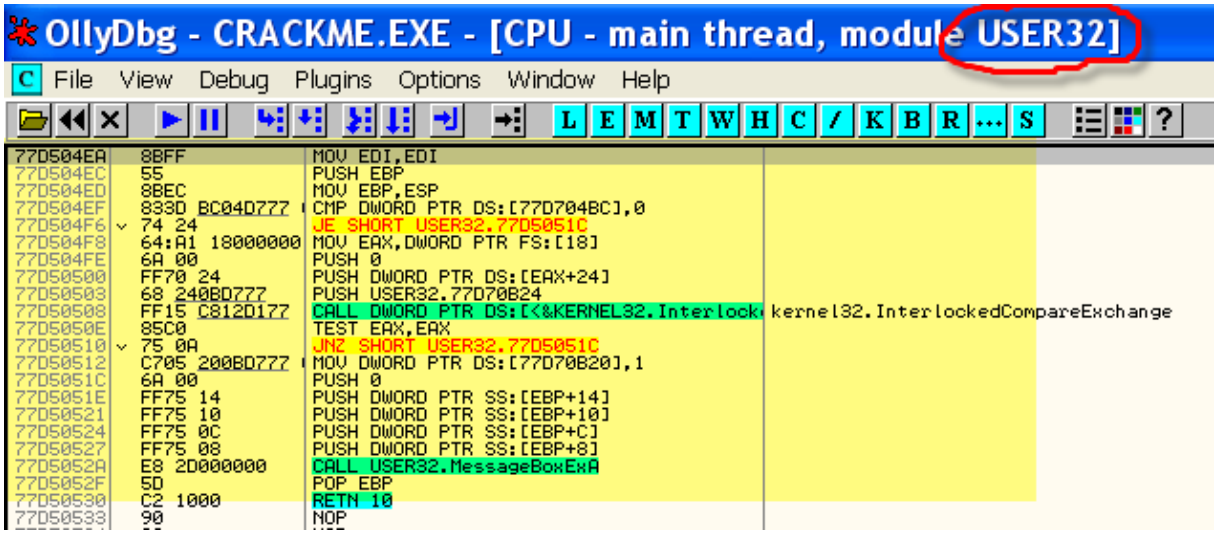
-

Вводим тот адрес, что выдал Command Bar (у Вас этот адрес может быть другим!)



-

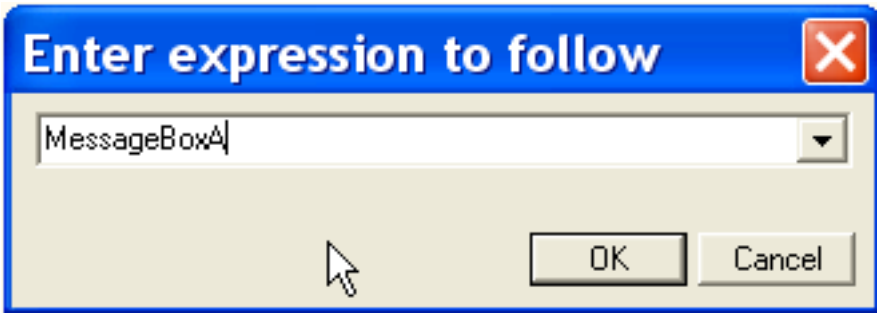
Если у Вас установлена Windows 9x, этот фокус не сработает, о чём мы подробнее поговорим ниже.



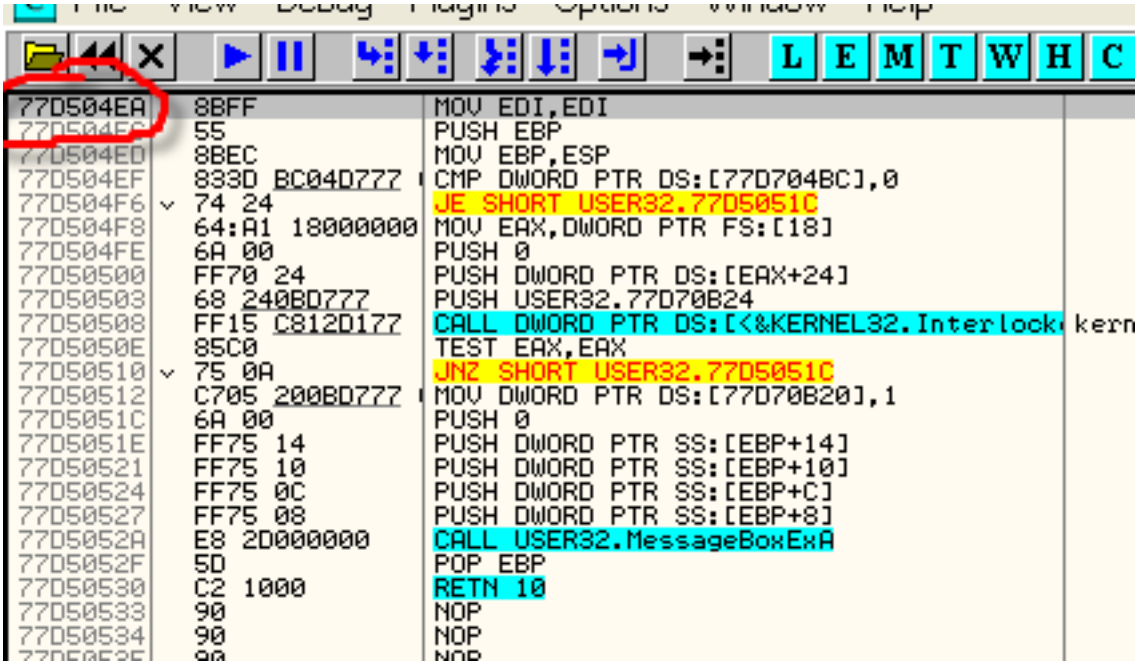
-

Сразу видим, что функция принадлежит USER32.DLL, но мы это уже знали, ведь имя DLL обычно указывается вместе с именем функции в дизассемблере (вроде CALL USER32.MessageBoxA). Ещё можно заметить, что MessageBoxA - это обычная функция, которая начинается по текущему адресу и завершается инструкцией RET. Код данной функции находится в библиотеке, а не в экзешнике, что позволяет сэкономить на размере и упрощает работу крэкера :)

Чтоб вернуться к текущей инструкции, достаточно нажать МИНУС. В окне Go to - Expression можно было прямо ввести MessageBoxA.



-

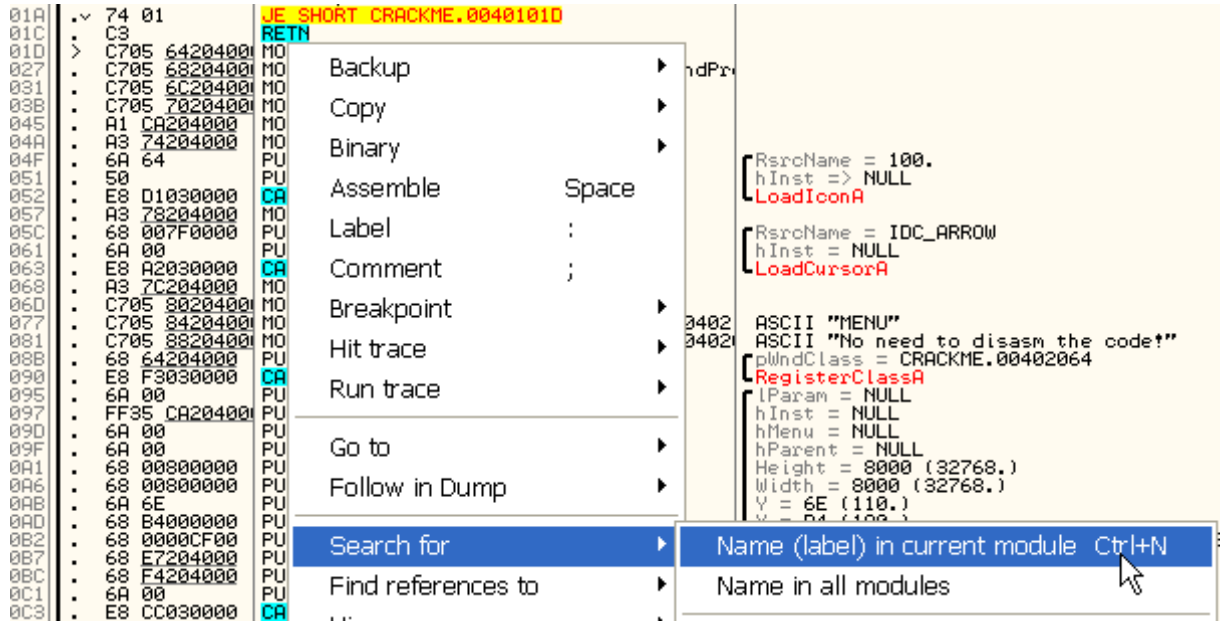


-

Вот, мы снова оказались внутри функции MessageBoxA из библиотеки USER32.DLL.

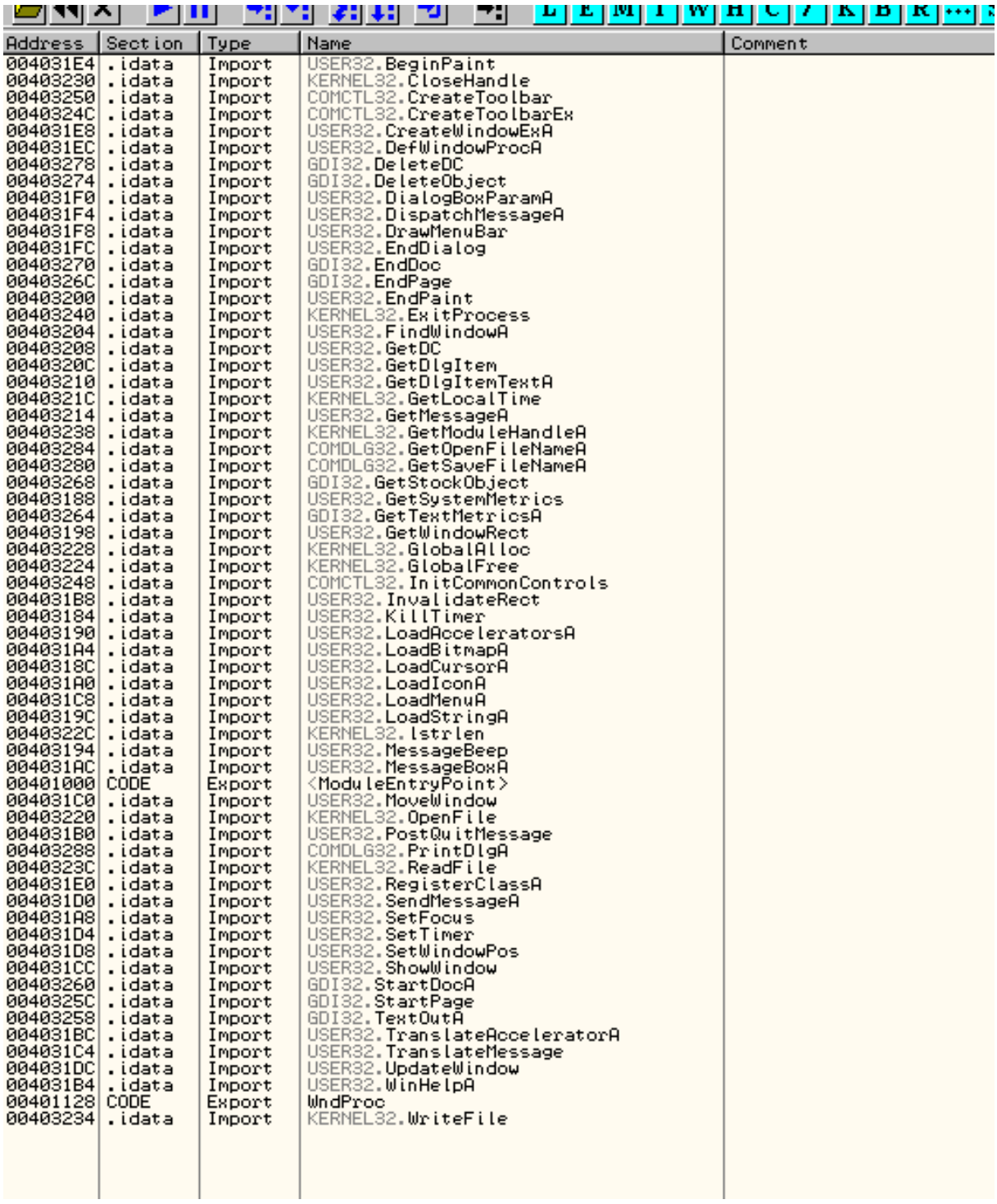
В имени функции важно правильно указывать строчные и прописные буквы (MessageBoxA и messageboxa - разные имена). Как же узнать правильный регистр той или иной буквы?

Нажимаем МИНУС - возвращаемся к точке входа.



-

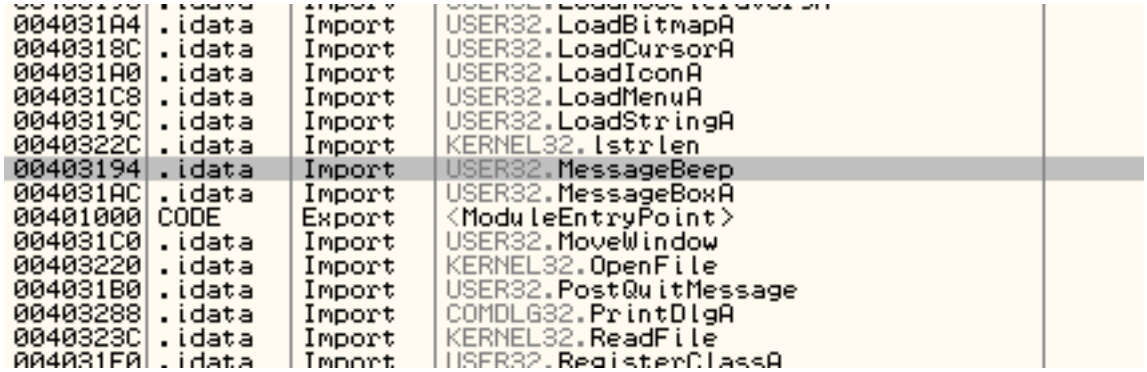
Правая кнопка мыши в листинге: Search for - Name (label) in current module. Получаем полный список имён API'шных функций, задействованных в данном экзешнике.



Address	Section	Type	Name	Comment
004031E4	.idata	Import	USER32.BeginPaint	
00403230	.idata	Import	KERNEL32.CloseHandle	
00403250	.idata	Import	COMCTL32.CreateToolbar	
0040324C	.idata	Import	COMCTL32.CreateToolbarEx	
004031E8	.idata	Import	USER32.CreateWindowExA	
004031EC	.idata	Import	USER32.DefWindowProcA	
00403278	.idata	Import	GDI32.DeleteDC	
00403274	.idata	Import	GDI32.DeleteObject	
004031F0	.idata	Import	USER32.DialogBoxParamA	
004031F4	.idata	Import	USER32.DispatchMessageA	
004031F8	.idata	Import	USER32.DrawMenuBar	
004031FC	.idata	Import	USER32.EndDialog	
00403270	.idata	Import	GDI32.EndDoc	
0040326C	.idata	Import	GDI32.EndPage	
00403200	.idata	Import	USER32.EndPaint	
00403240	.idata	Import	KERNEL32.ExitProcess	
00403204	.idata	Import	USER32.FindWindowA	
00403208	.idata	Import	USER32.GetDC	
0040320C	.idata	Import	USER32.GetDlgItem	
00403210	.idata	Import	USER32.GetDlgItemTextA	
0040321C	.idata	Import	KERNEL32.GetLocalTime	
00403214	.idata	Import	USER32.GetMessageA	
00403238	.idata	Import	KERNEL32.GetModuleHandleA	
00403284	.idata	Import	COMDLG32.GetOpenFileNameA	
00403280	.idata	Import	COMDLG32.GetSaveFileNameA	
00403268	.idata	Import	GDI32.GetStockObject	
00403188	.idata	Import	USER32.GetSystemMetrics	
00403264	.idata	Import	GDI32.GetTextMetricsA	
00403198	.idata	Import	USER32.GetWindowRect	
00403228	.idata	Import	KERNEL32.GlobalAlloc	
00403224	.idata	Import	KERNEL32.GlobalFree	
00403248	.idata	Import	COMCTL32.InitCommonControls	
004031B8	.idata	Import	USER32.InvalidateRect	
00403184	.idata	Import	USER32.KillTimer	
00403190	.idata	Import	USER32.LoadAcceleratorsA	
004031A4	.idata	Import	USER32.LoadBitmapA	
0040318C	.idata	Import	USER32.LoadCursorA	
004031A0	.idata	Import	USER32.LoadIconA	
004031C8	.idata	Import	USER32.LoadMenuA	
0040319C	.idata	Import	USER32.LoadStringA	
0040322C	.idata	Import	KERNEL32.lstrlen	
00403194	.idata	Import	USER32.MessageBeep	
004031AC	.idata	Import	USER32.MessageBoxA	
00401000	CODE	Export	<ModuleEntryPoint>	
004031C0	.idata	Import	USER32.MoveWindow	
00403220	.idata	Import	KERNEL32.OpenFile	
004031B0	.idata	Import	USER32.PostQuitMessage	
00403288	.idata	Import	COMDLG32.PrintDlgA	
0040323C	.idata	Import	KERNEL32.ReadFile	
004031E0	.idata	Import	USER32.RegisterClassA	
004031D0	.idata	Import	USER32.SendMessageA	
004031A8	.idata	Import	USER32.SetFocus	
004031D4	.idata	Import	USER32.SetTimer	
004031D8	.idata	Import	USER32.SetWindowPos	
004031CC	.idata	Import	USER32.ShowWindow	
00403260	.idata	Import	GDI32.StartDocA	
0040325C	.idata	Import	GDI32.StartPage	
00403258	.idata	Import	GDI32.TextOutA	
004031BC	.idata	Import	USER32.TranslateAcceleratorA	
004031C4	.idata	Import	USER32.TranslateMessage	
004031DC	.idata	Import	USER32.UpdateWindow	
004031B4	.idata	Import	USER32.WinHelpA	
00401128	CODE	Export	WndProc	
00403234	.idata	Import	KERNEL32.WriteFile	

-

Не нужно перечитывать весь список - чтобы найти нужную функцию, достаточно начать набирать её имя. Нажимаем M:

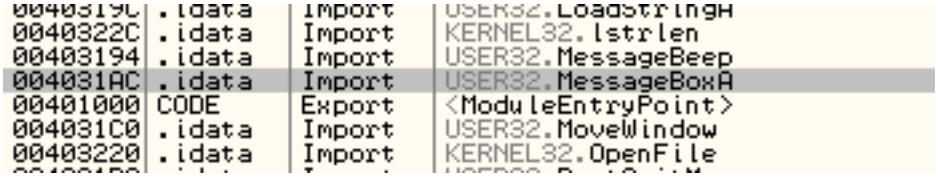


-

Курсор позиционируется на первой функции, имя которой начинается на M.

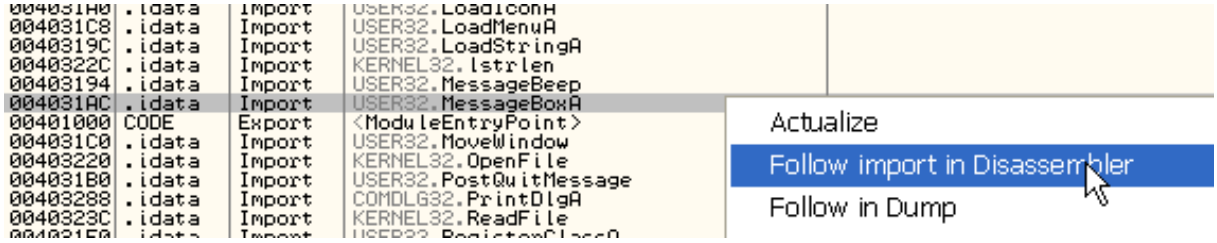
-

В заголовке окна выводятся буквы, по которым осуществляется поиск.



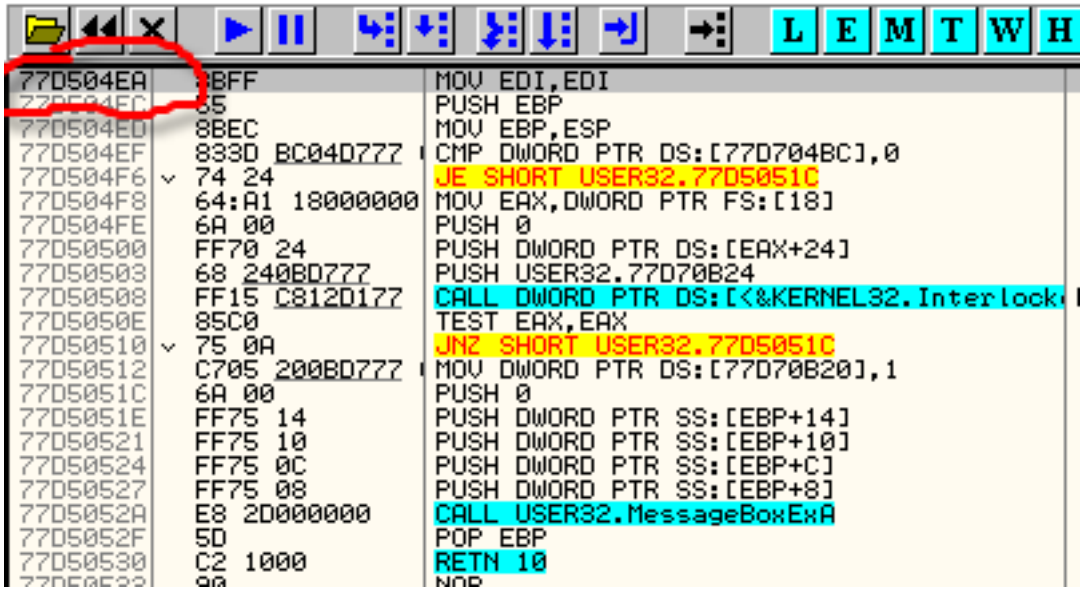
-

Правая кнопка мыши по имени функции вызывает меню со следующими опциями:



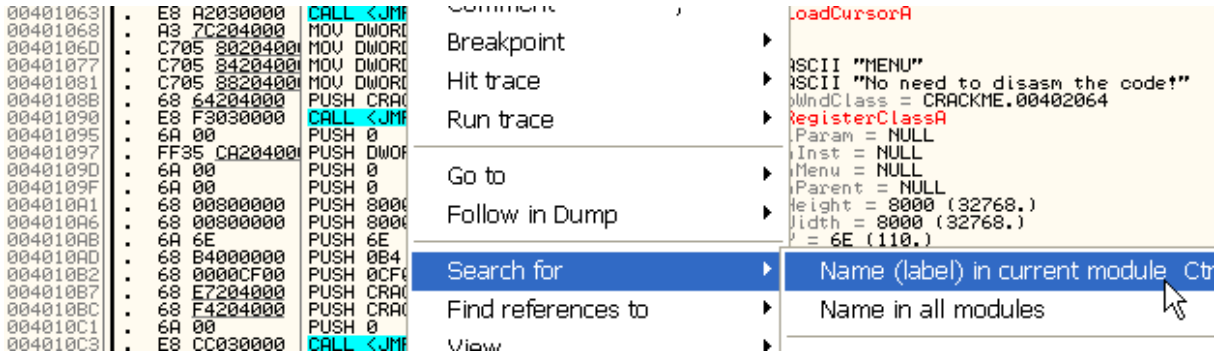
-

Опция Follow import in Disassembler подгружает эту функцию в дизассемблер - это ещё один способ попасть внутрь API'шной функции.



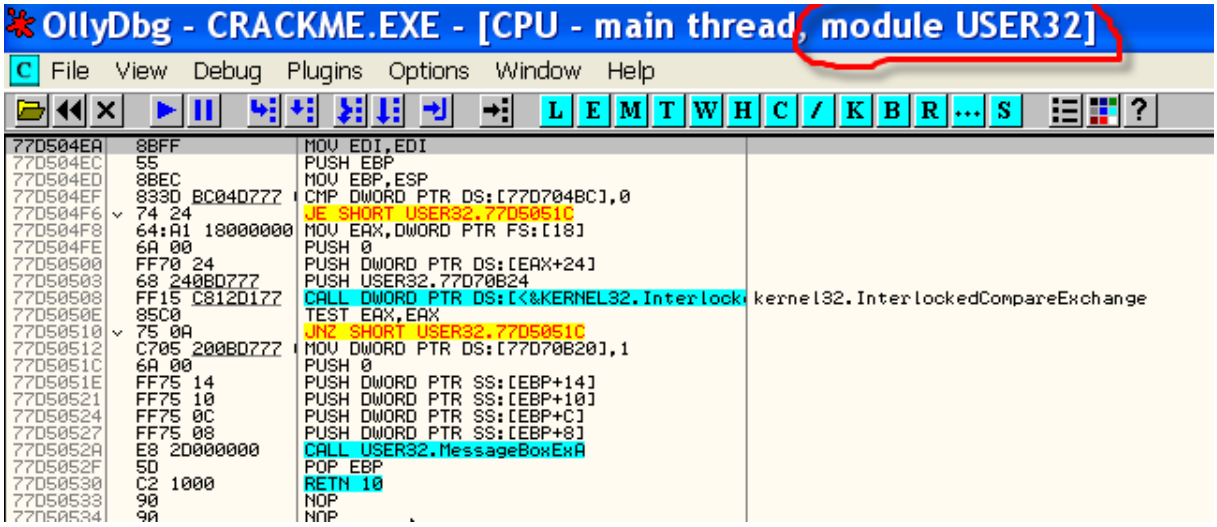
-

Часто новички ошибочно задают поиск символов (Search for - Name (lable) in current module), находясь при этом внутри какой-либо внешней функции. К примеру, если мы воспользуемся данной опцией внутри функции MessageBoxA, мы получим список имён функций, которые импортирует библиотека USER32.DLL, а вовсе не подопытный эзешник. В меню чётко сказано, что поиск осуществляется в текущем модуле (current module), а текущем в данном случае окажется USER32, ведь функция MessageBoxA находится именно в нём.



-

Дабы избежать подобных ошибок, не забывайте поглядывать на заголовок окна отладчика - там отображается, в частности, имя текущего модуля.

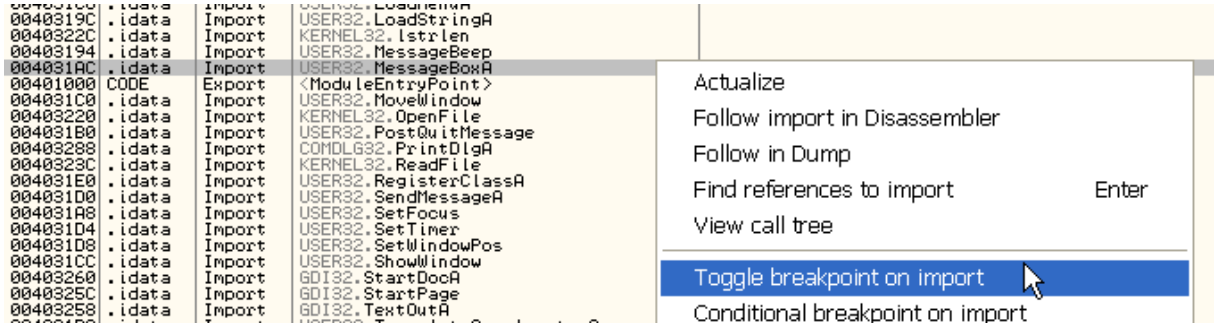


-

Даже если мы просто заглянули во внешнюю функцию в дизассемблере, а не передали управление в отладчике, т.е. функция даже не выполняется ещё, текущим модулем всё равно окажется библиотека, в которой находится внешняя функция. В таком случае нужно понажимать на МИНУС пока Вы не окажитесь в "основном" модуле, чтобы воспользоваться такими опциями как Search for - Names и т.п.

Windows NT: 2000, XP и 2003

Дальнейшее обсуждение касается операционных систем NT: 2000, XP и 2003. Если у Вас мастдай (пардон, тут автор использовал термин Windows 95/98 - прим. пер.), смею порекомендовать перейти на любую из вышеперечисленных осей, т.к. на NT OllyDbg обладает бульшими возможностями (и глючит меньше - опять прим. пер.) Если такой возможности нет, можете пропустить дальнейшее обсуждение и перейти сразу к "Приложение. Windows 9x"



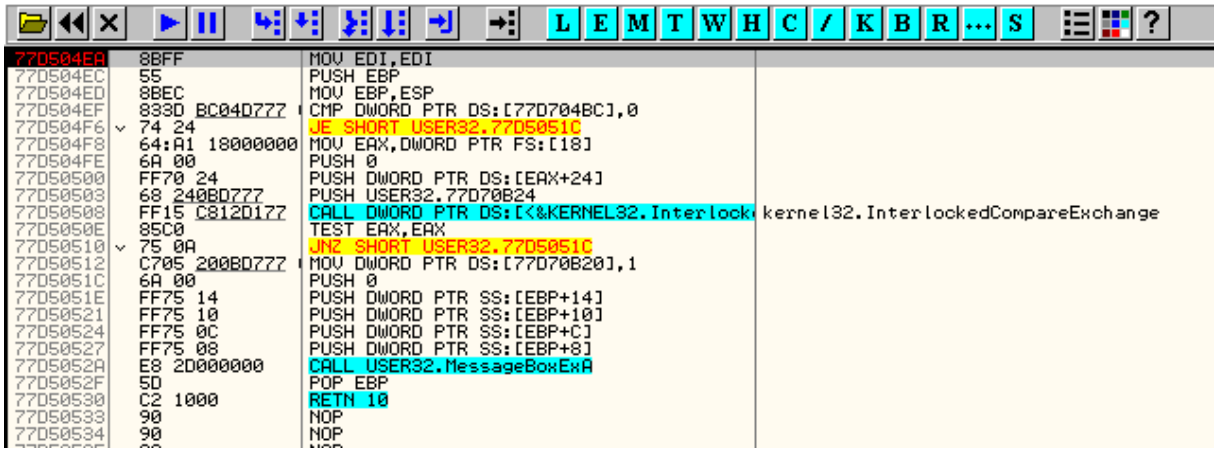
-

В перечне API'шных функций, которые использует крэкми, есть ещё одна полезная опция в контекстном меню: Toggle breakpoint on import. Данная опция позволяет установить точку останова на вызове функции API.

Точку останова на вызовах конкретной API'шной функции можно установить и через Command Bar:

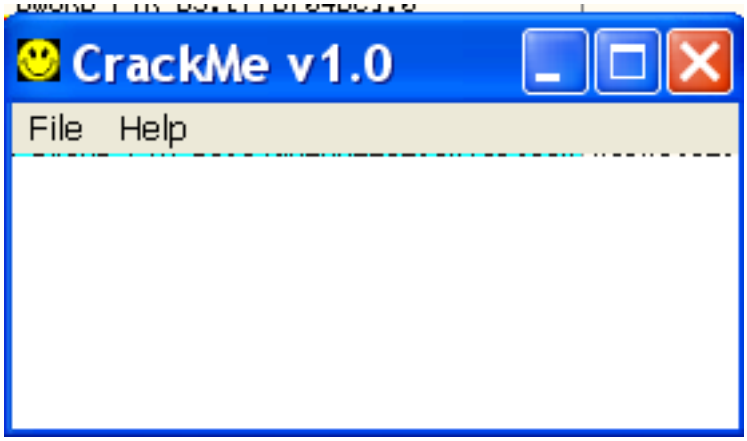
```
bp MessageBoxA
```

В отличие от bpx, bp ставит точку останова на первой инструкции внутри заданной функции, а не на самих инструкциях вызова (call или jmp) в текущем модуле. Давайте снова зайдём в функцию MessageBoxA (любым из вышеописанных способов) и удостоверимся, что на первой инструкции данной функции появилась точка останова. Так и есть:



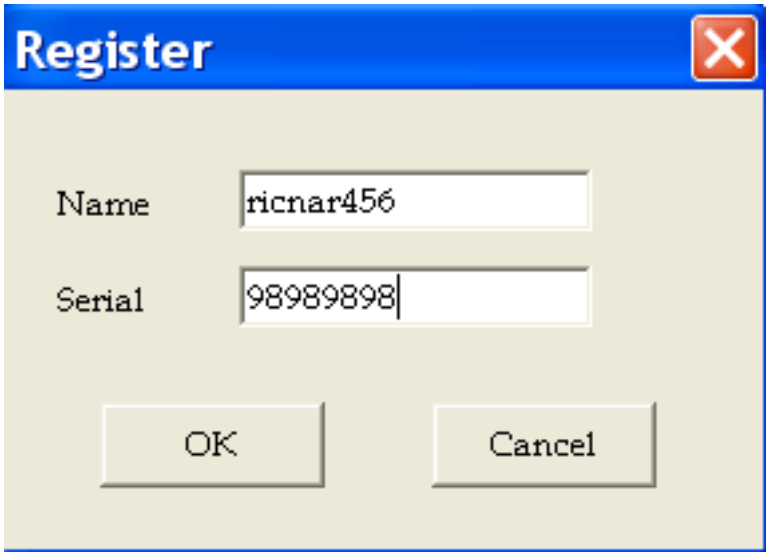
-

Таким образом, при вызове функции MessageBoxA, должна сработать точка останова. Давайте нажмём F9, чтобы запустить крэкми.



-

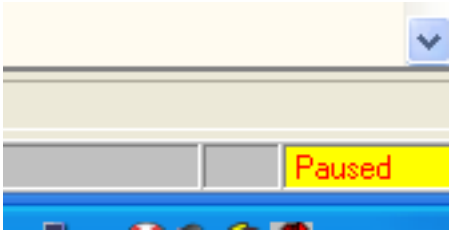
Появилось главное окно крэкми. В меню выбираем Help - Register:



-

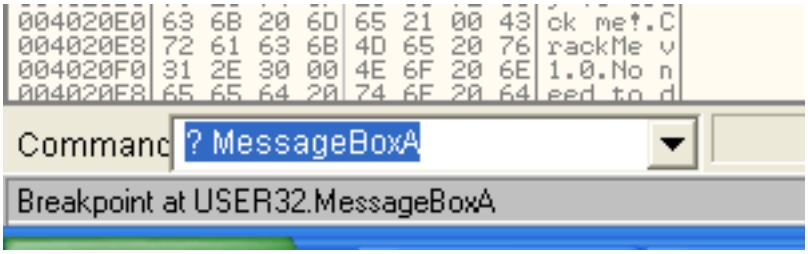
Вводим произвольные данные в поля Name и Serial; нажимаем OK. В этот момент должен произойти вызов функции MessageBoxA, который должна зафиксировать наша точка останова... Ага, отладчик прервал выполнение! В правом нижнем углу светится надпись Paused (будто нам и

так непонятно, что процесс остановлен - прим. пер.).



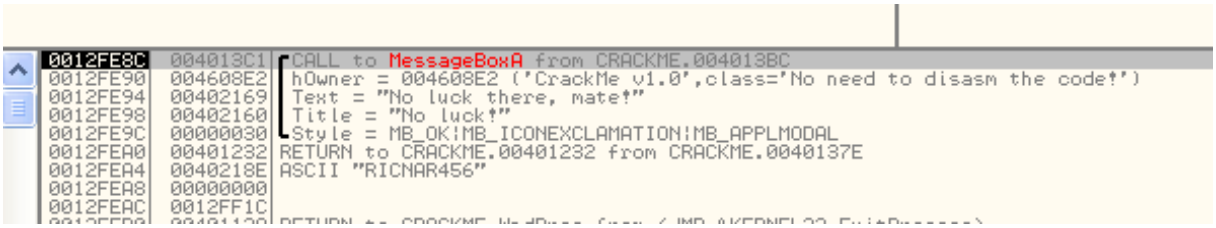
-

Левее указана причина, по которой произошла остановка.



-

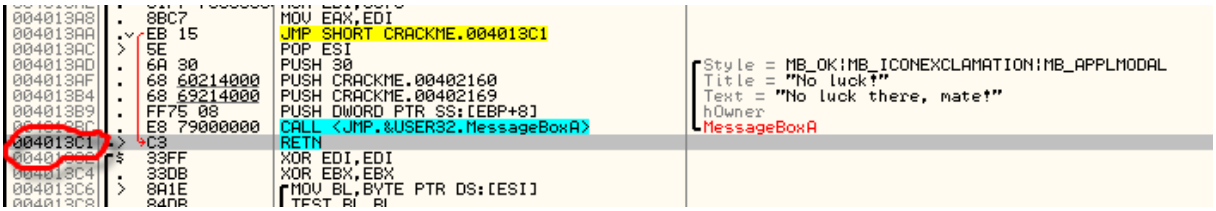
Там написано: "Breakpoint at USER32.MessageBoxA", что значит "Точка останова на USER32.MessageBoxA". Наша точка останова сработала.



-

В момент вызова функции можно посмотреть на значения аргументов данной функции. Аргументы API'шных функций обычно передаются через стек справа налево, согласно соглашению stdcall. Давайте рассмотрим их внимательно.

В самом верху стека хранится адрес возврата. В данном случае - 4013C1.



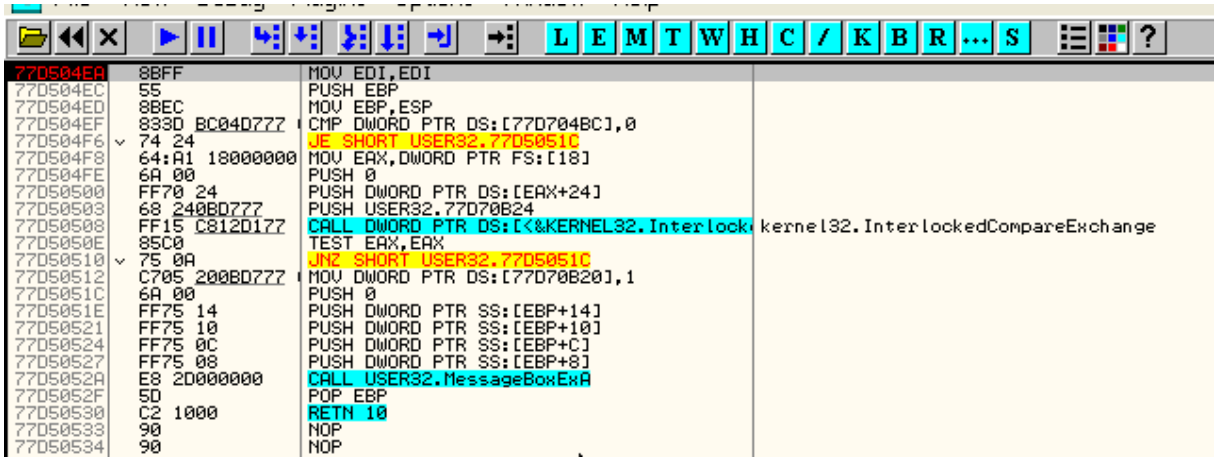
-

Когда мы проходили инструкции CALL и RET в подготовительной части этого цикла (Глава 7), мы имели возможность убедиться, что при вызове любой подпрограммы (т.е. функции) верхушка стека содержит адрес возврата - адрес, на который перейдёт управление, после завершения функции. В данном случае, после завершения MessageBoxA, управление перейдёт на адрес 4013C1.

Далее (т.е. ниже) идут аргументы функции. MessageBoxA принимает 4 аргумента (можете посмотреть описание функции API MessageBox в MSDN, чтобы убедиться): дескриптор родительского окна, текст сообщения, текст заголовка и стиль.

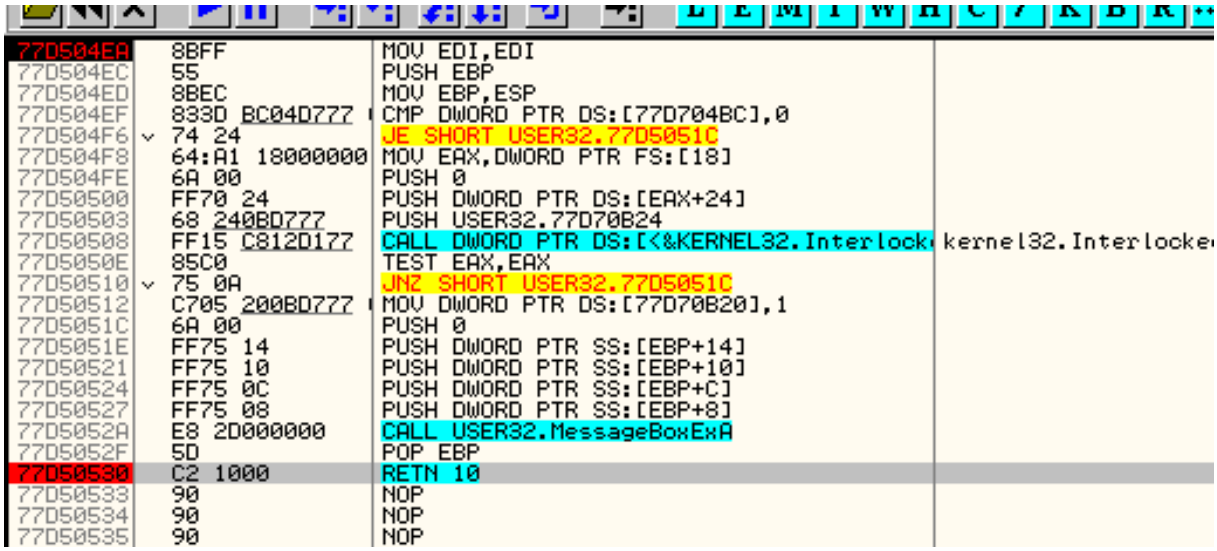
Текст сообщения - "No luck there, mate!" - констатирует, что Вы не угадали правильные параметры регистрации :)

Вот-вот вылезет роковое сообщение...



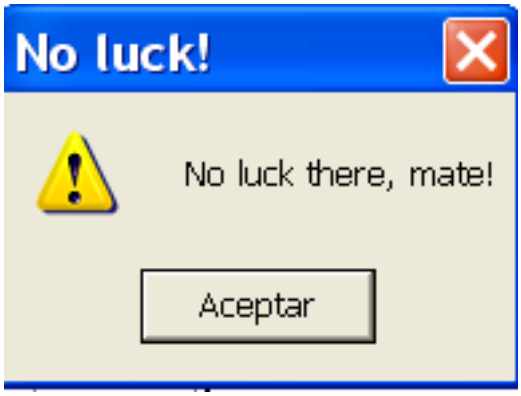
-

Чтобы убедиться, что текущий вызов MessageBoxA - это тот самый MessageBox, что уведомляет нас о неправильно набранном серийнике, если у Вас есть сомнения... Давайте изолируем текущий вызов, поставив точку останова на инструкции RETN 10h, которую можно найти чуть ниже. Адрес инструкции возврата у Вас, возможно, будет отличаться, но, в любом случае, это должен быть первый RET внутри функции.



-

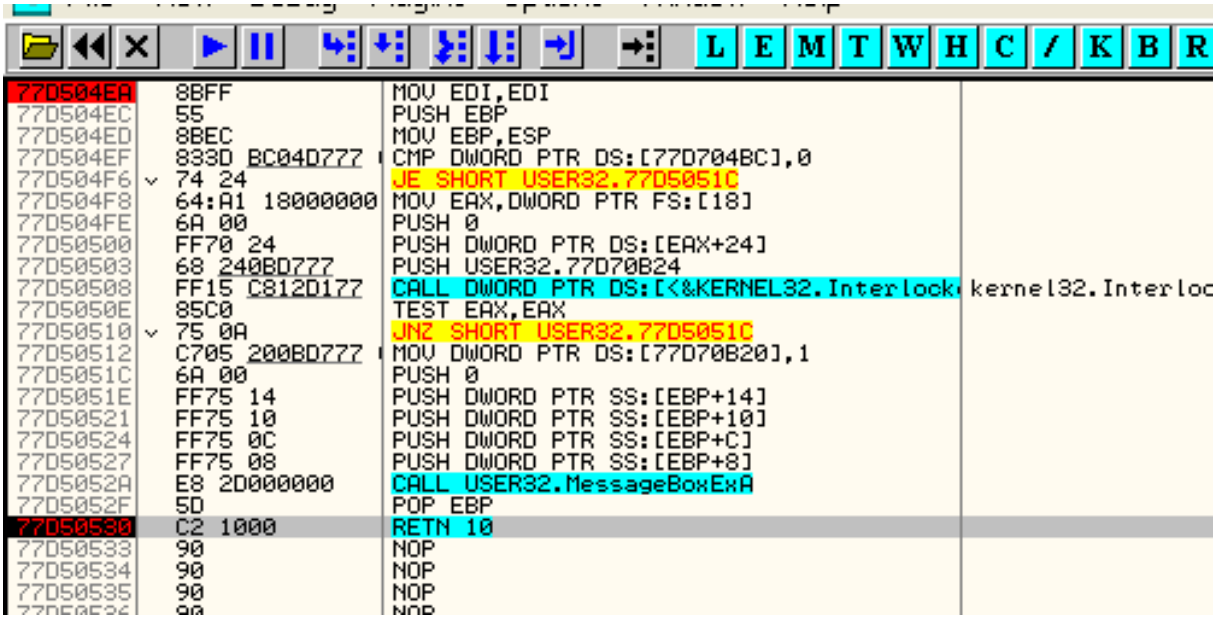
Нажимаем F9.



-

Окно MessageBox появилось на экране. Текст заголовка и сообщения соответствует параметрам функции, которые мы только что перехватили: "Lo luck!" и "No luck there, mate!". Серийник не подошёл :)

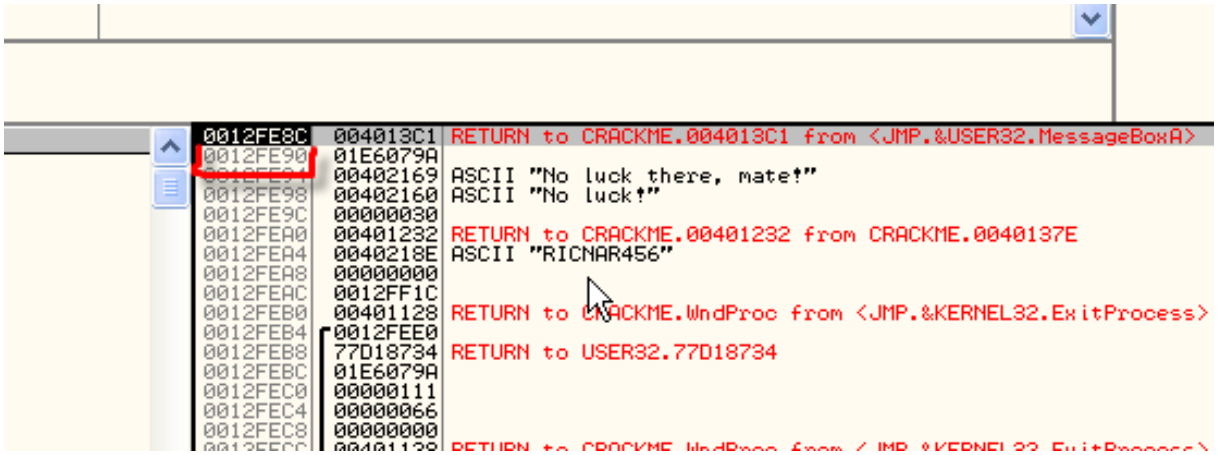
Нажимаем "Да" - срабатывает точка останова на RETN 10h.



-

Таким образом, можно заключить, что сообщение выскочило в процессе выполнения функции MessageBoxA.

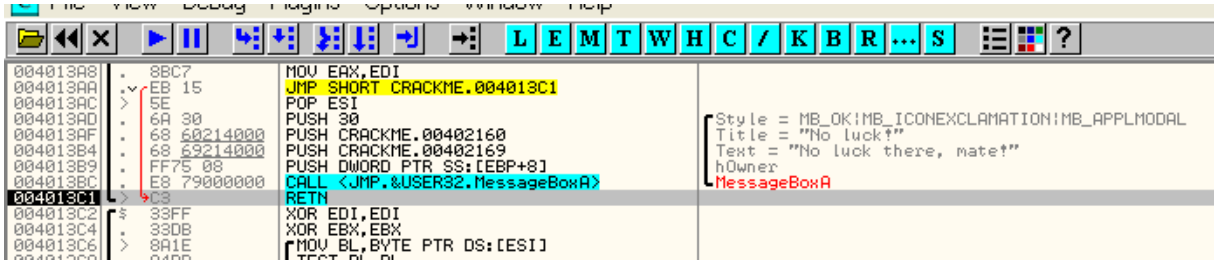
Чем отличается RETN 10h от обычного RET? Обычный RET просто вернул бы управление на адрес возврата - 4013C1.



-

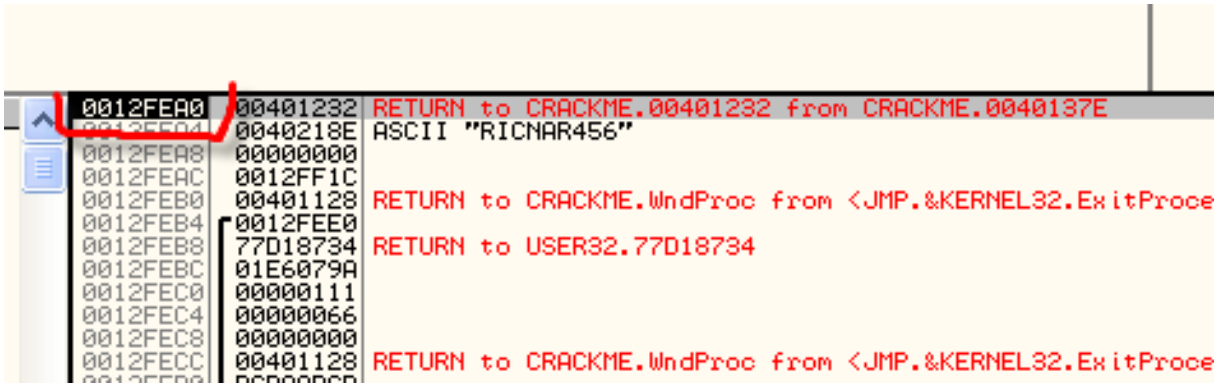
Из стека извлекается адрес возврата. Указатель на верхушку стека перемещается на 4 байта вниз (точнее, в сторону старших адресов). Таким образом, ESP увеличивается на 4 в результате выполнения инструкции RET. В случае с RET 10h, в добавок к тому, что делает обычный RET, ESP увеличивается ещё на 10h. В общей сложности, к значению ESP прибавляется 10h + 4 = 14h = 20. Давайте проверим: нажимаем F7.

Дальнейшее обсуждение касается обоих семейств Windows: 9x и NT.



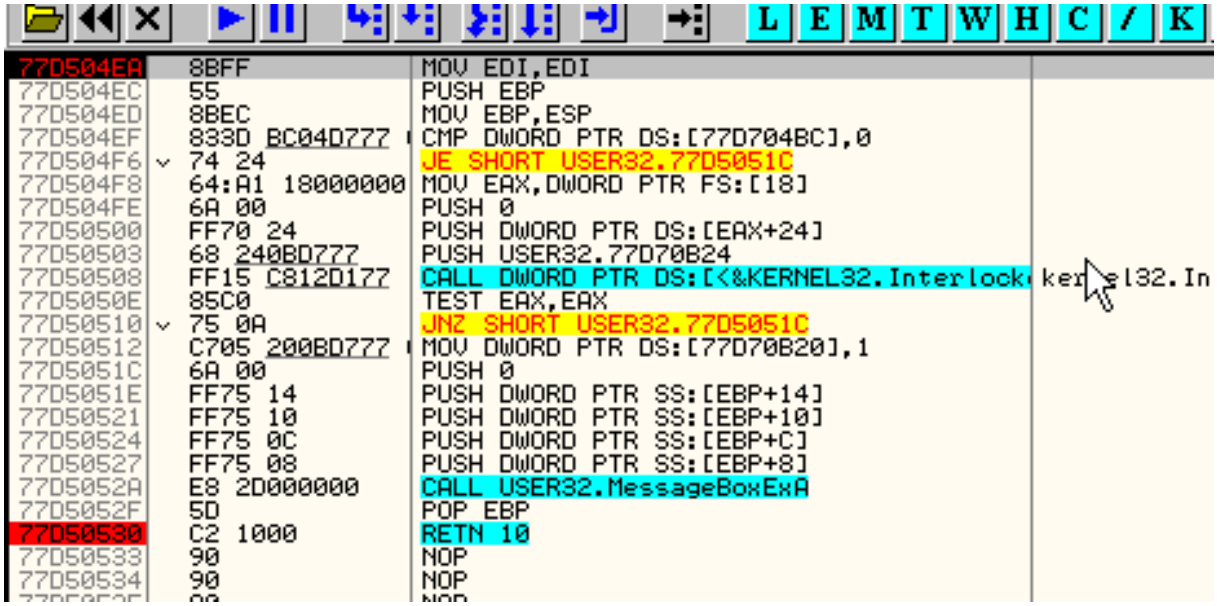
-

Вот мы и вернулись из API-функции обратно в код крэкми. Обратите внимание, что указатель верхушки стека сдвинулся на 20 байт вниз, как и следовало ожидать, т.е. RET N не просто извлекает из стека адрес возврата, но и очищает аргументы, переданные в функцию, как того требует соглашение stdcall.

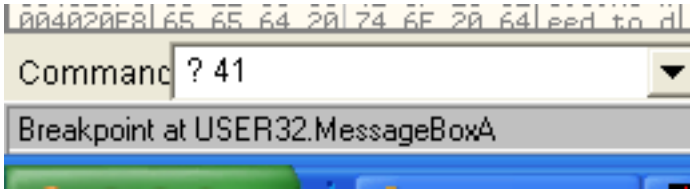


-

В данный момент крэкми уже точно знает, что серийник не верен и даже успел нам об этом сообщить. На данном этапе уже поздно что-либо предпринимать. Нажимаем F9.

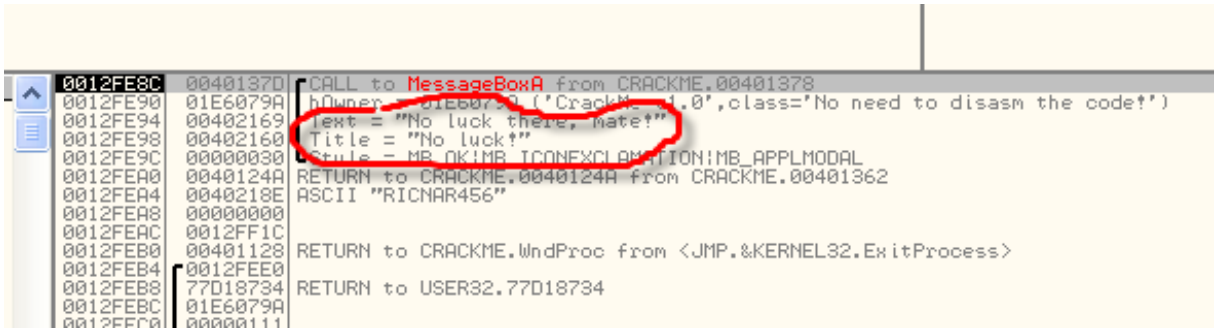


-



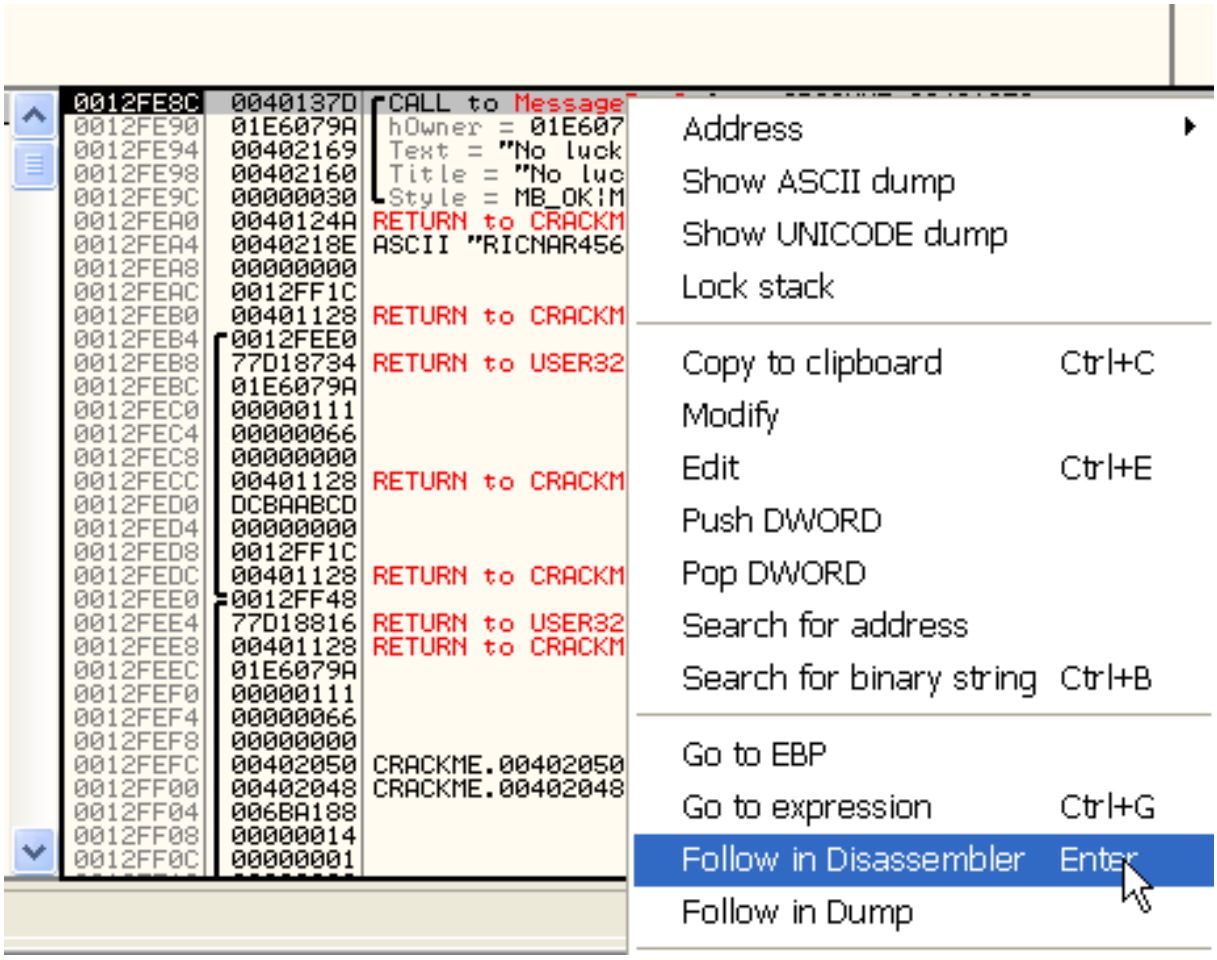
-

Снова срабатывает точка останова. Оказывается, крэкми решил снова вывести сообщение о неправильном серийнике, будто мы успели забыть, что серийник не подошёл :)



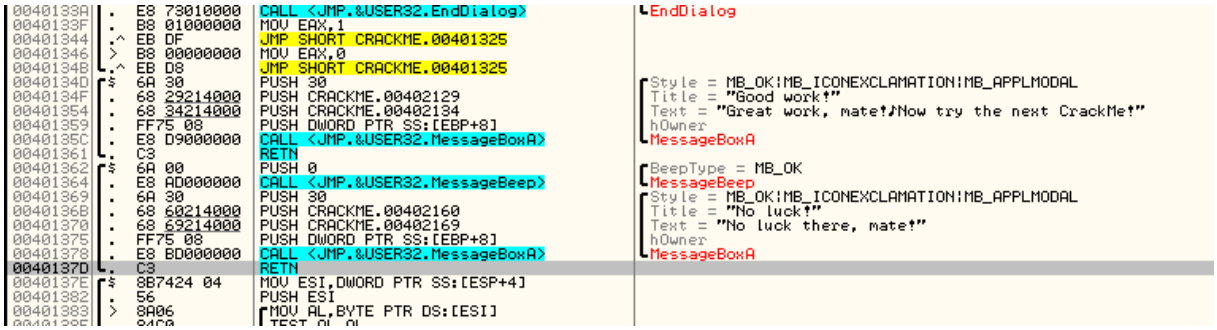
-

Адрес возврата на этот раз указывает на 40137D. Давайте посмотрим, что за код находится по этому адресу. Для этого можно воспользоваться опцией Go to - Expression: 40137D. Или прямо по правому щелчку в верхней ячейке стека - Follow in Disassembler.



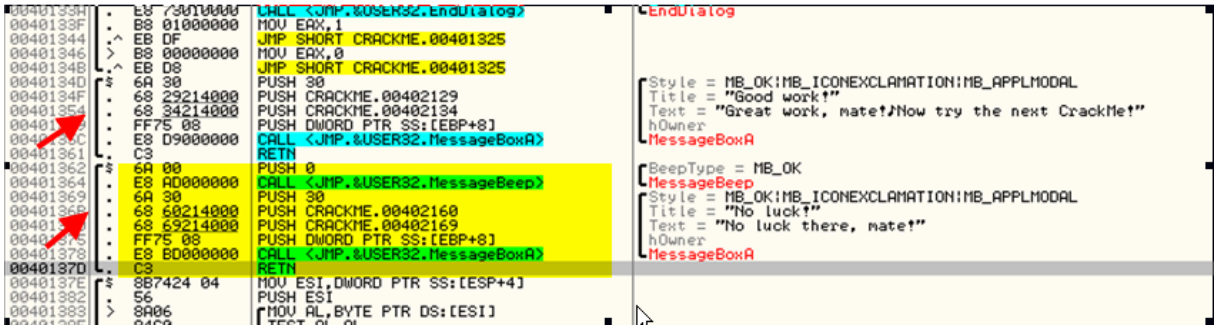
-

Сразу над строчкой по адресу возврата (40137D) находится инструкция CALL, которая вызвала функцию MessageBox (401378).



-

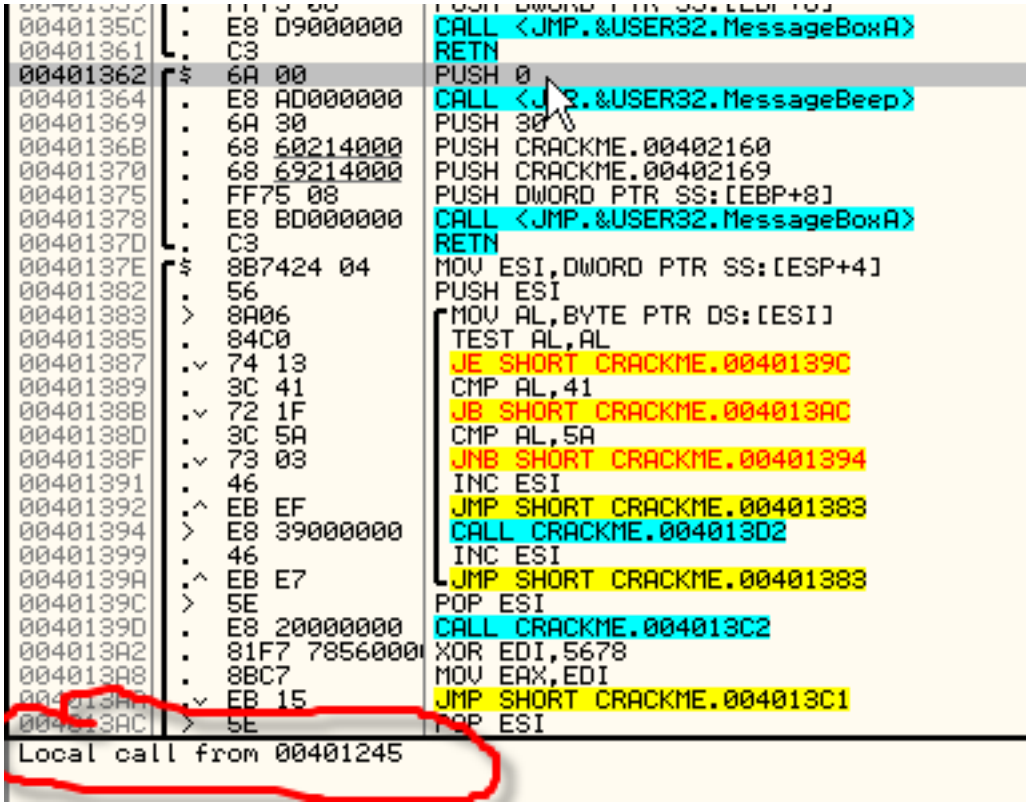
Чуть выше виден другой MessageBox, но с совершенно другим текстом: Great work, mate! Похоже, что это сообщение выскакивает, если серийник оказывается правильным :) Для начала было бы неплохо попасть на тот другой MessageBox, вместо того, что вот-вот покажется на экране.



-

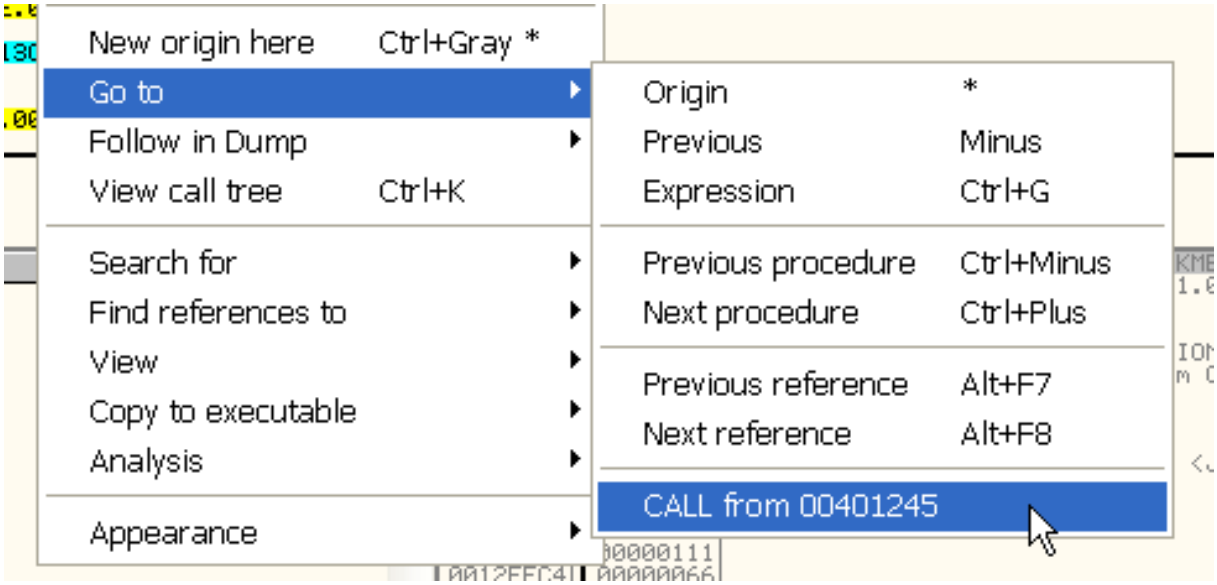
Предварительный анализ проделанный отладчиком показывает, что данный код входит в состав функции, о чём свидетельствует жирная скобка в колонке с машинным кодом. Таких функций несколько: одна начинается по адресу 401362 и содержит сообщение "No luck...", другая - "Great work..." и начинается по адресу 40134D.

Если выделить первую строчку функции "No luck..." (по адресу 401362), в окне подсказки покажется следующая информация:

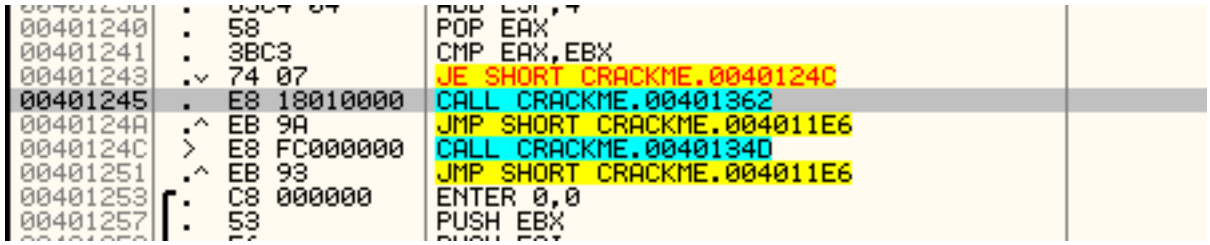


-

Отладчик в курсе откуда вызывается данная функция, но так бывает только с прямыми вызовами! Щелчёк правой кнопкой мыши - Go to - CALL from 401245.

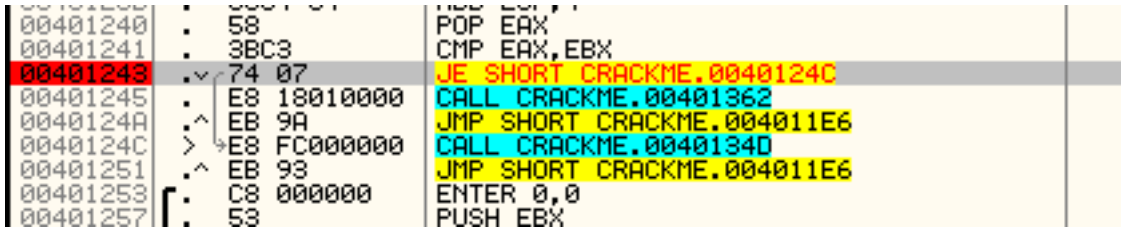


-



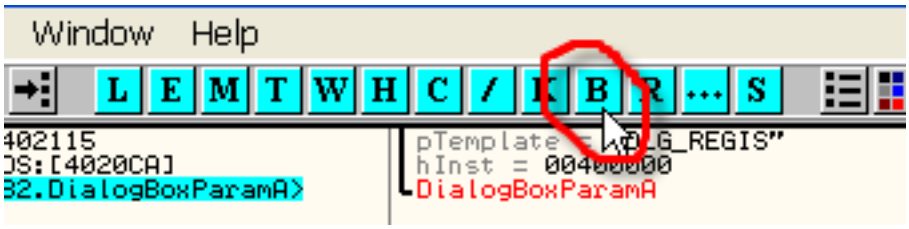
-

Занятненько... Перед нами типичный код сравнения некоторого значения, в результате которого вызывается одна из тех двух функций с сообщениями: "No luck..." и "Great work...". Такое нельзя пропускать! Ставим точку останова на условный переход.

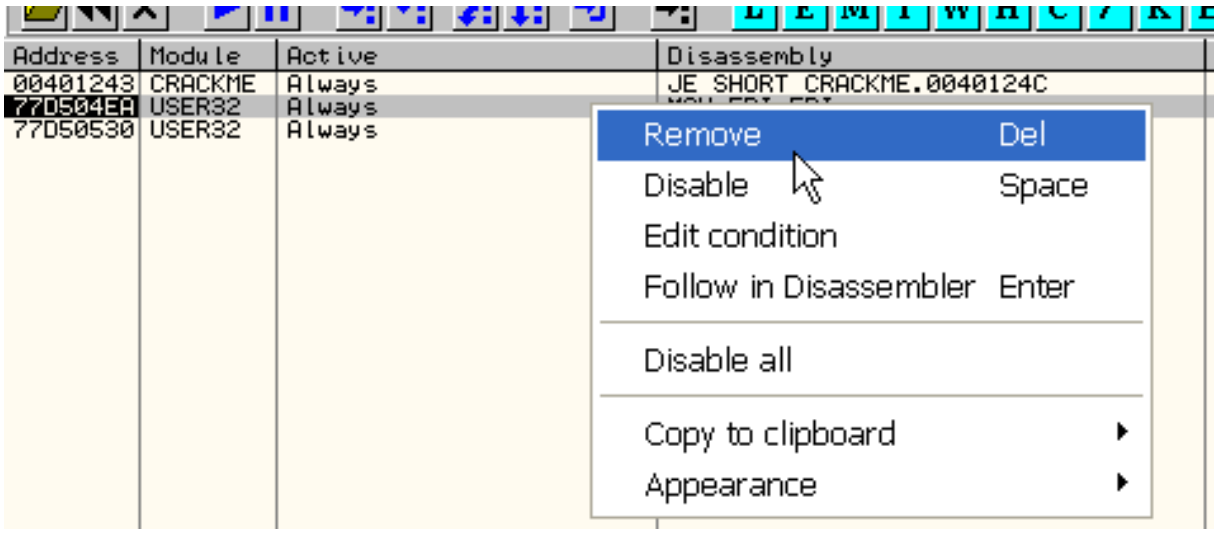


-

За одно, убираем точки останова на MessageBox. Это можно сделать через окно [B], т.е. Breakpoints:

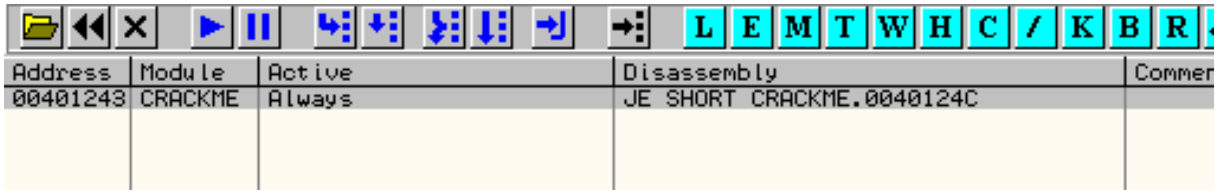


-



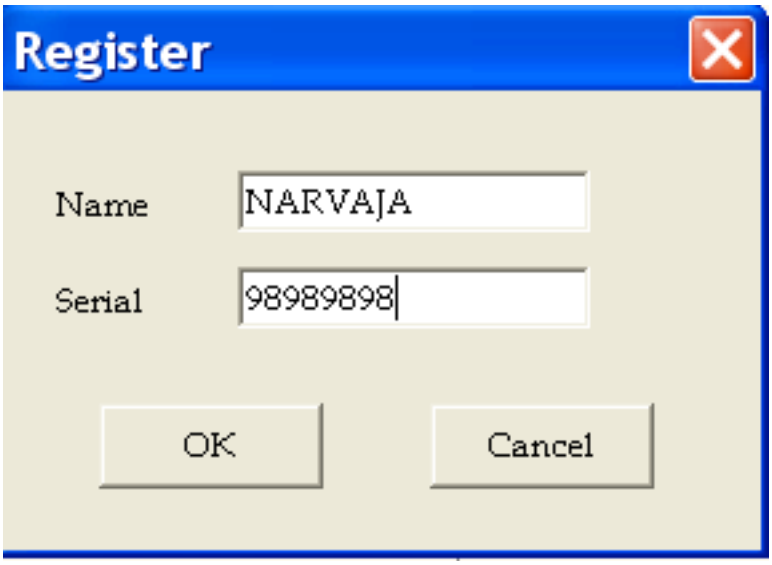
-

Правая кнопка мыши - Remove. Таким образом убираем обе точки останова на MessageBox и оставляем только свежепоставленную точку на том подозрительном условном переходе.



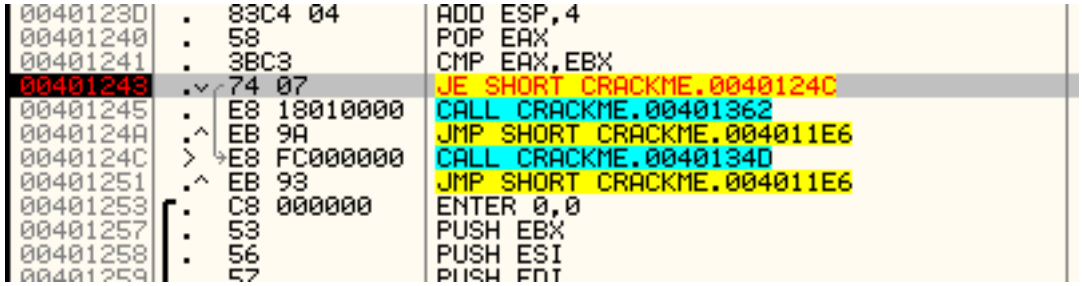
-

Командуем Run (F9), принимаем сообщение "No luck...", которое мы только что трассировали. Снова вводим имя и серийник (используйте те же, что и на снимке).



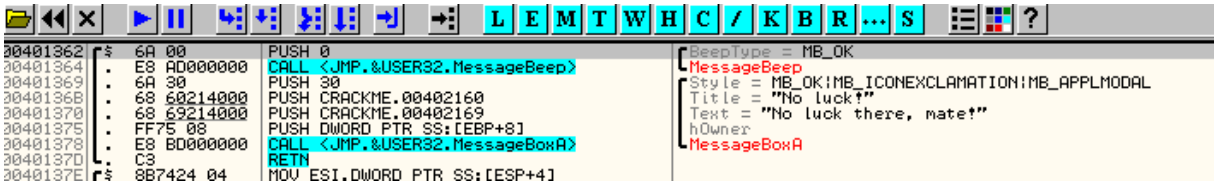
-

Нажимаем OK.



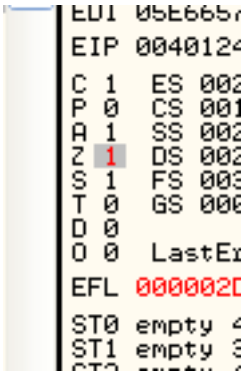
-

Переход не осуществляется, т.к. предварительное сравнение EAX и EBX обернулось неравенством. Таким образом, выполняется вызов CALL 401362 - там нехорошее сообщение, если кто-то уже забыл. Можете нажать правую кнопку мыши - Follow и освежить память.



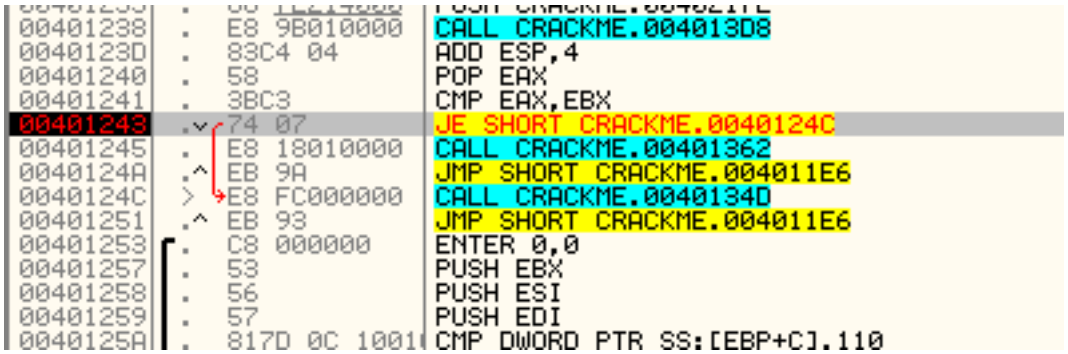
-

А что будет, если повлиять на условие перехода, изменив значение флага Z? Двойной щелчёк по флагу в окне регистров меняет его текущее состояние на протовоположное.



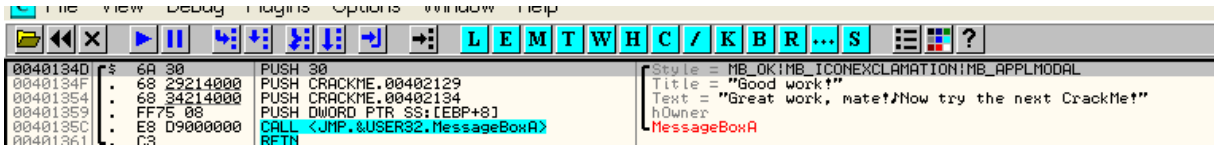
-

Установленный флаг Z означает, что EAX = EBX, т.е. разность EAX - EBX равна нулю. Переход осуществляется.



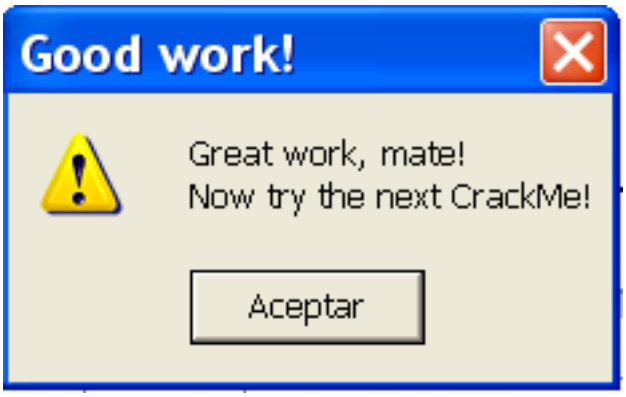
-

Теперь вызовется другая функция. Смотрим через Follow:



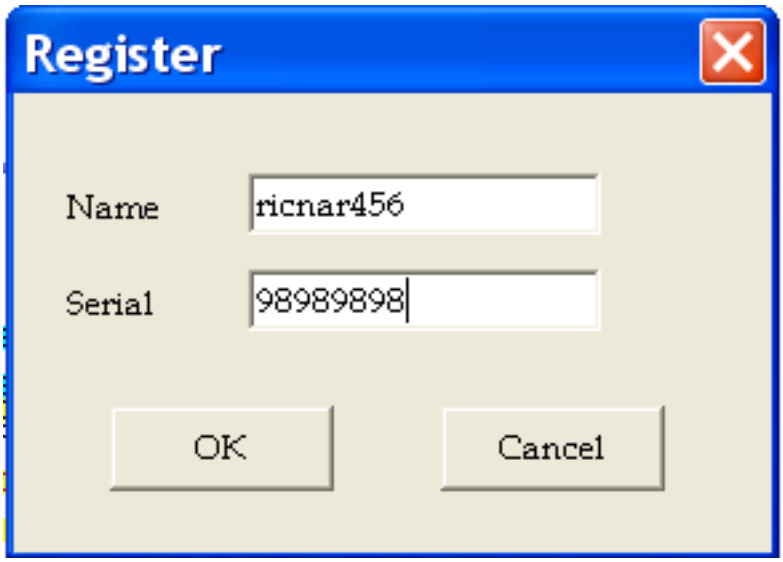
-

Нажимаем Run (F9) :)))



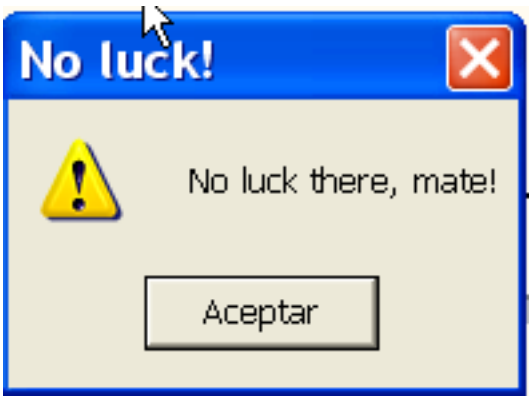
-

Значит, то сравнение является ключевым моментом в проверке правильности серийника. Если значения EAX и EBX равны, выскакивает "хорошее" сообщение, иначе выводится "Lo luck...". Раньше мы заметили, что "нехорошие" сообщения в коде имеются в двух экземплярах, но в последний раз выскочило только одно из них. Когда крэкми замечает, что в имени есть цифры (в ricnar456 как раз они есть), сначала выводится одно сообщение, а потом, после ключевого сравнения, выводится ещё одно. Давайте проверим ещё раз.



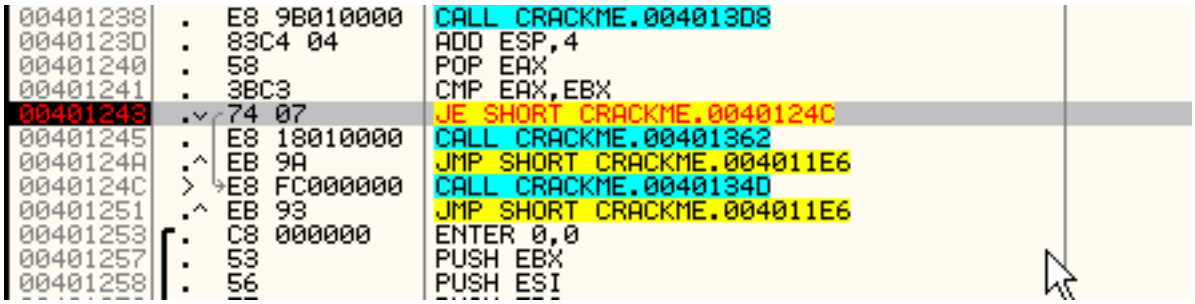
-

Вводим имя, содержащее цифры. Нажимаем OK.



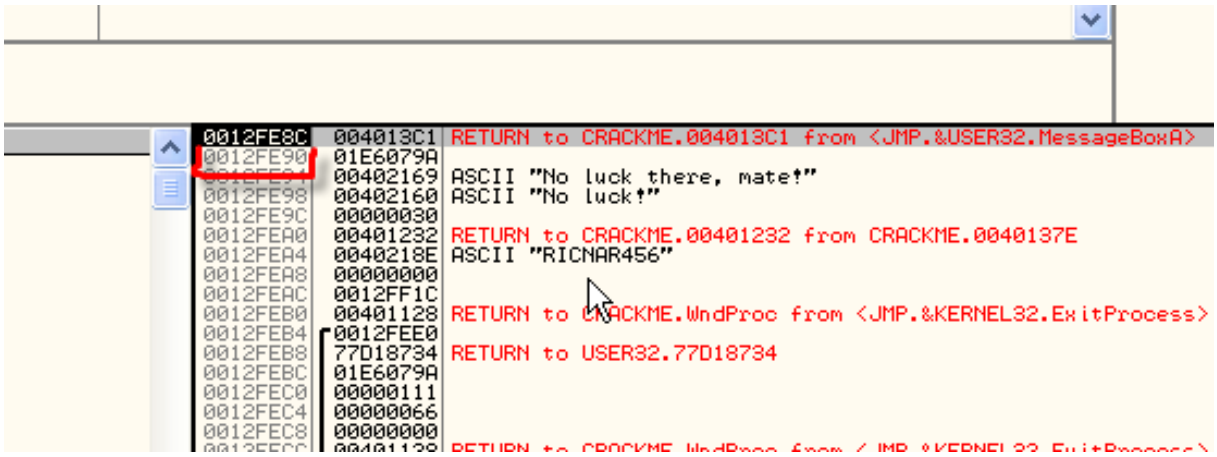
-

Появляется сообщение о неправильном серийнике и уже ПОСЛЕ этого срабатывает точка останова на условном переходе!



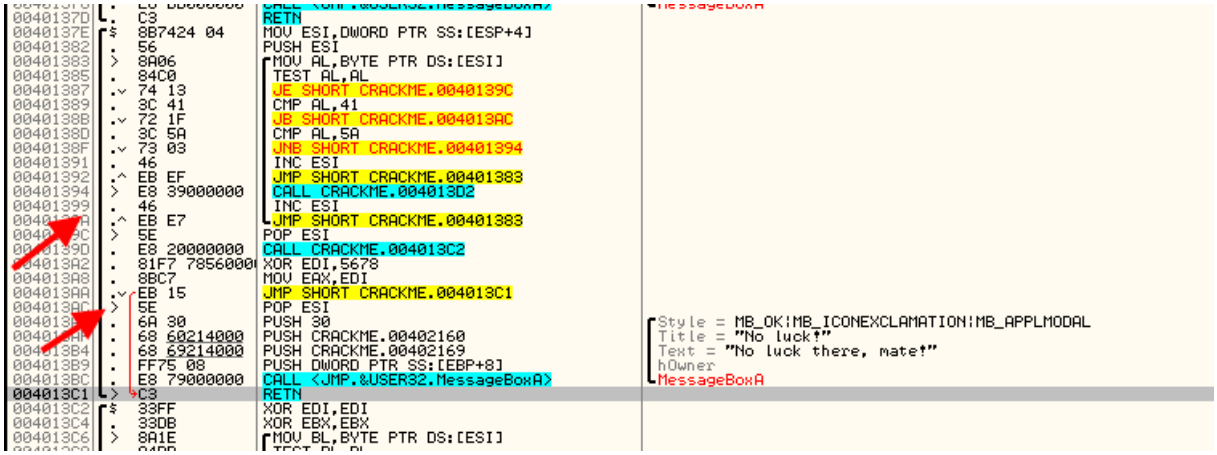
-

Потом появляется окончательное сообщение. Когда мы первый раз остановились на входе в MessageBox (на начальной стадии данного пособия), это был тот первый MessageBox, т.е. не окончательный.



-

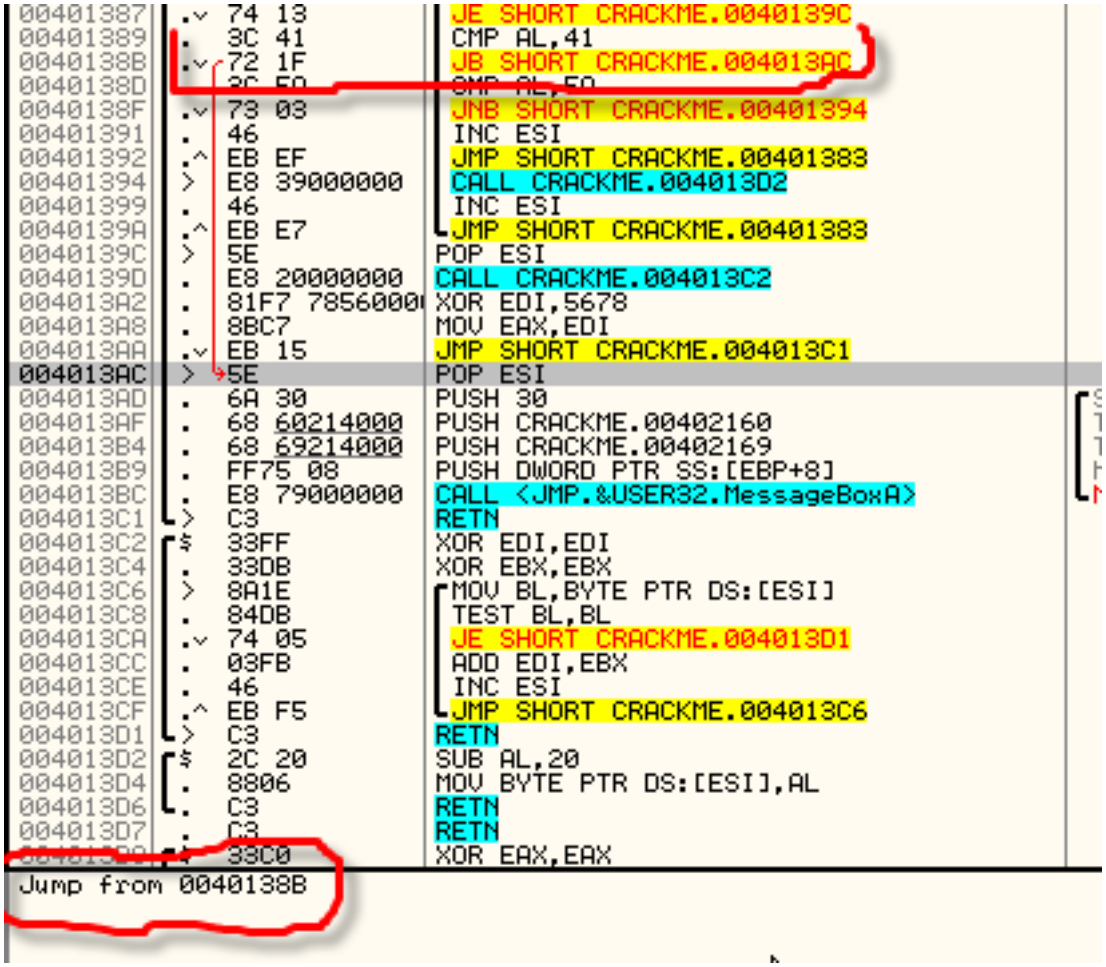
Адрес возврата соответствовал 4013C1.



-

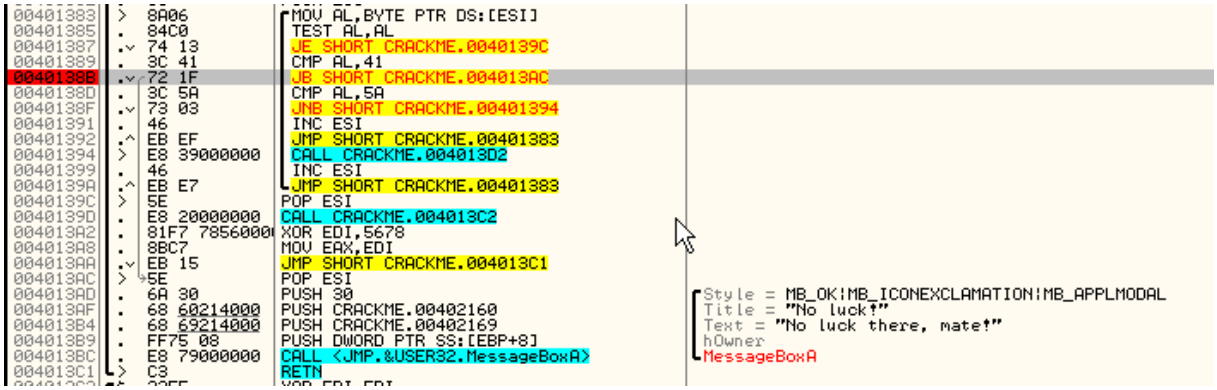
В окне дизассемблера выделена функция (см. жирную скобку), начинающаяся по адресу 40137E и заканчивающаяся по адресу 4013C1: там находится завершающий RET сразу после вызова MessageBox.

Обратите внимание, что по адресу 4013AC есть стрелочка (>), а это значит, что данный адрес используется в каком-то прямом переходе. Давайте выделим эту строчку, чтобы узнать где находится этот переход.



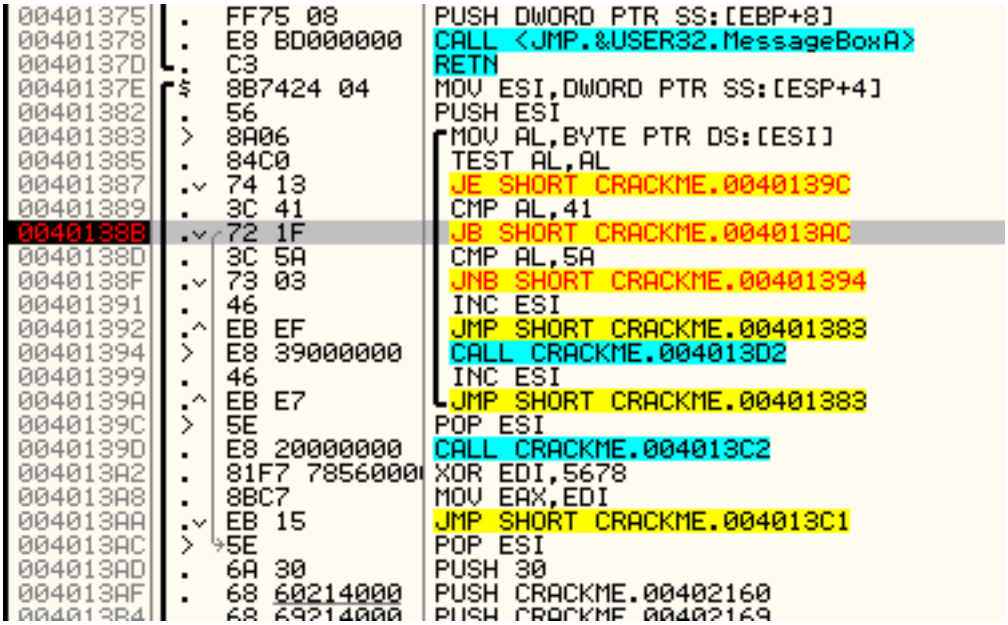
-

Перед нами ещё одно сравнение и условный переход, который приводит к отображению нехорошего сообщения. Давайте поставим точку останова в этом месте.



-

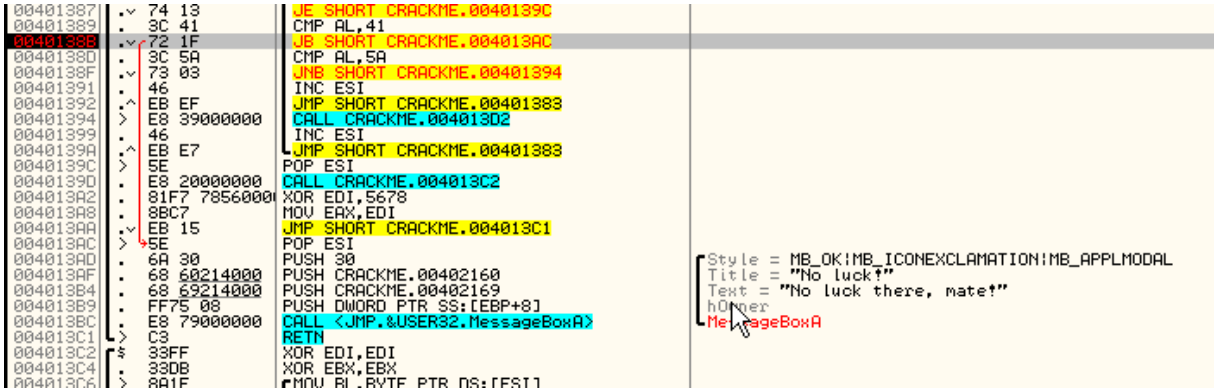
Снова пускаем приложение (RUN); принимаем сообщение; опять вводим наши дежурные данные регистрации (имя содержащее цифры!) и опять нажимаем OK.



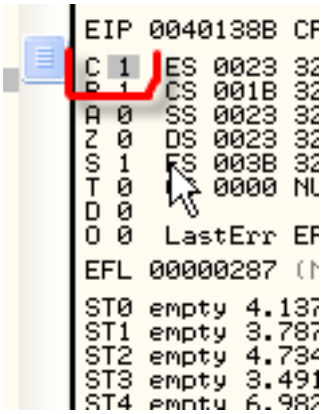
-

Данный код проверяет все буквы в имени пользователя. Злосчастный переход осуществляется, как только встретится цифра. При каждом прохождении цикла срабатывает точка останова. Нажмите F9 - остановка произойдёт уже на 2й букве.

На седьмом прохождении цикла (первая цифра в имени ricnar456 - это ‘4’ и занимает 7ую позицию) переход будет осуществлён, но мы этому воспрепятствуем :)

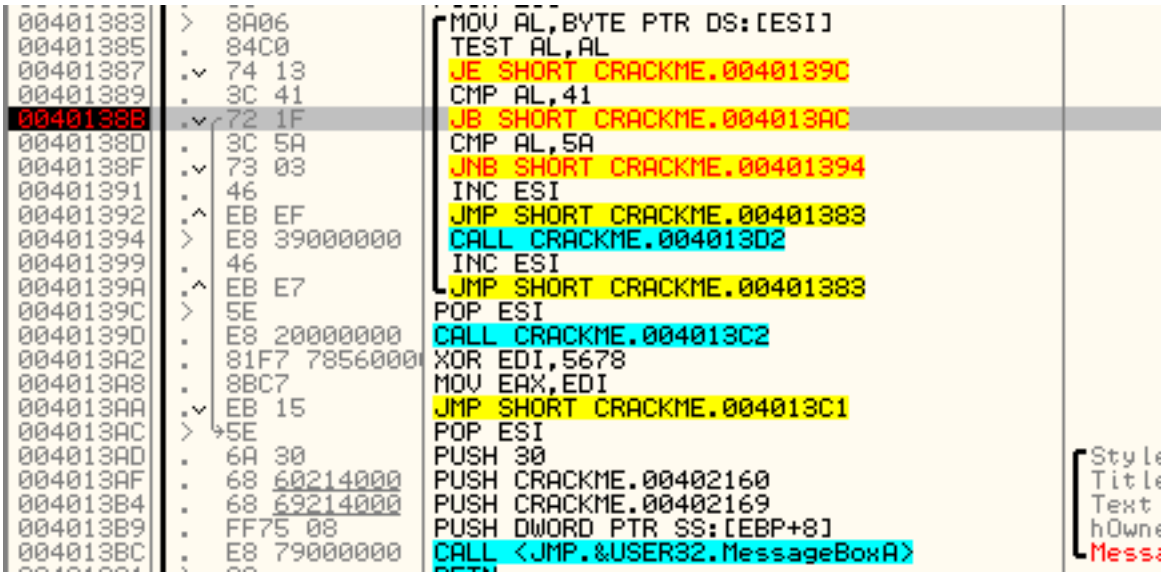


-



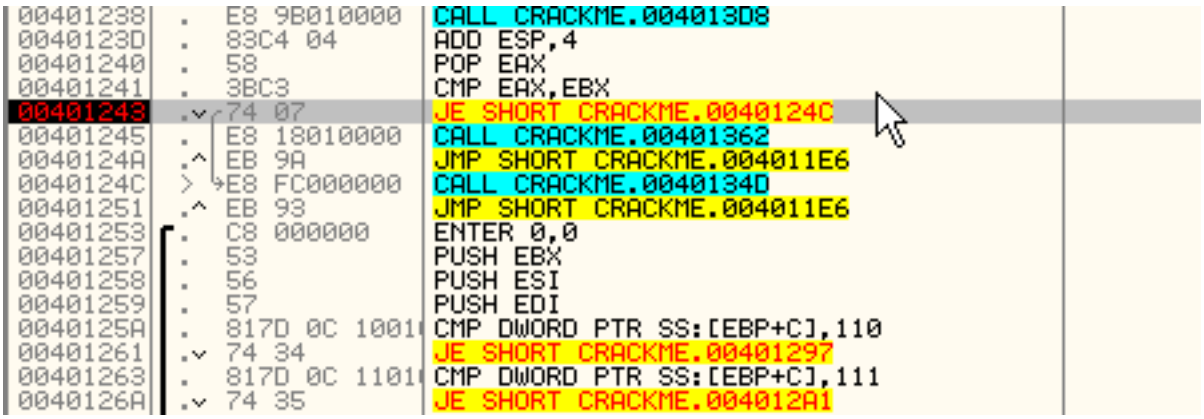
-

Условный переход JB осуществляется в зависимости от состояния флага C. Делаем двойной щелчёк по этому флагу, чтобы изменить его состояние.



-

Таким образом, переход не будет выполнен. Повторяем фокус с флагом для 2х оставшихся цифер, чтобы не сработал этот переход.



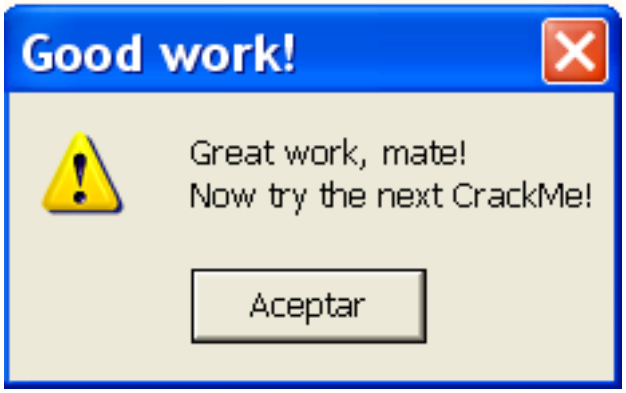
-

Далее мы попадаем на сравнение EAX и EBX, где нужно повлиять на флаг Z, как в прошлый раз, чтобы переход осуществился.

```
0040123D   .  83C4 04      ADD ESP,4
00401240   .  58           POP EAX
00401241   .  3BC3         CMP EAX,EBX
00401243   .v 74 07        JE SHORT CRACKME.0040124C
00401245   .  E8 18010000  CALL CRACKME.00401362
0040124A   .^ EB 9A        JMP SHORT CRACKME.004011E6
0040124C   >  E8 FC000000  CALL CRACKME.0040134D
00401251   .^ EB 93        JMP SHORT CRACKME.004011E6
00401253   .  C8 000000    ENTER 0,0
00401257   .  53           PUSH EBX
00401258   .  56           PUSH ESI
00401259   .  57           PUSH EDI
```

-

Нажимаем F9.



-

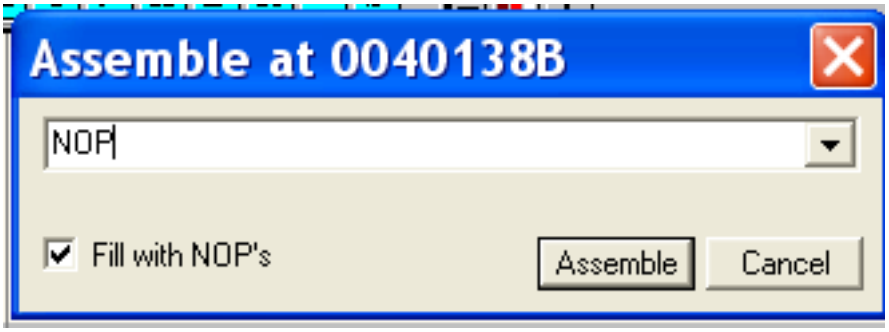
Не будем же мы всё время ручками менять флаги C и Z?! Настало время рассмотреть каким образом это дело можно зафиксировать, чтобы крэкми безропотно принимал любой серийник, причём вне отладчика.

Выделяем первый условный переход (где мы поставили точку останова).

```
00401382   .  56           PUSH ESI
00401383   >  8A06         MOV AL,BYTE PTR DS:[ESI]
00401385   .  84C0         TEST AL,AL
00401387   .v 74 13         JE SHORT CRACKME.0040139C
00401389   .  3C 41         CMP AL,41
0040138B   .v 72 1F         JB SHORT CRACKME.004013AC
0040138D   .  3C 5A         CMP AL,5A
0040138F   .v 73 03         JNB SHORT CRACKME.00401394
00401391   .  46            INC ESI
00401392   .^ EB EF         JMP SHORT CRACKME.00401383
00401394   >  E8 39000000   CALL CRACKME.004013D2
00401399   .  46            INC ESI
0040139A   .^ EB E7         JMP SHORT CRACKME.00401383
0040139C   >  5E            POP ESI
0040139D   .  E8 20000000   CALL CRACKME.004013C2
004013A2   .  81F7 7856000( XOR EDI,5678
004013A8   .  8BC7          MOV EAX,EDI
004013AA   .v EB 15         JMP SHORT CRACKME.004013C1
004013AC   >  5E            POP ESI
004013AD   .  6A 30         PUSH 30
004013AF   .  68 60214000   PUSH CRACKME.00402160
004013B4   .  68 69214000   PUSH CRACKME.00402169
```

-

Чтоб этот переход не выполнялся никогда, независимо от состояния флага C, его можно просто и грубо забить инструкциями NOP. Нажимаем на данном переходе пробел и вводим NOP.



-

```
0040138B    90          NOP
0040138C    90          NOP
0040138D    3C 5A       CMP AL,5A
0040138F    73 03       JNB SHORT CRACKME.00401394
00401391    46          INC ESI
00401392    EB EF       JMP SHORT CRACKME.00401383
00401394    E8 39000000 CALL CRACKME.004013D2
```

-

Точку останова уже можно убрать. Для этого достаточно нажать F2.

```
00401383    8A06        MOV AL,BYTE PTR DS:[ESI]
00401385    84C0        TEST AL,AL
00401387    74 13       JE SHORT CRACKME.0040139C
00401389    3C 41       CMP AL,41
0040138B    90          NOP
0040138C    90          NOP
0040138D    3C 5A       CMP AL,5A
0040138F    73 03       JNB SHORT CRACKME.00401394
00401391    46          INC ESI
00401392    EB EF       JMP SHORT CRACKME.00401383
00401394    E8 39000000 CALL CRACKME.004013D2
```

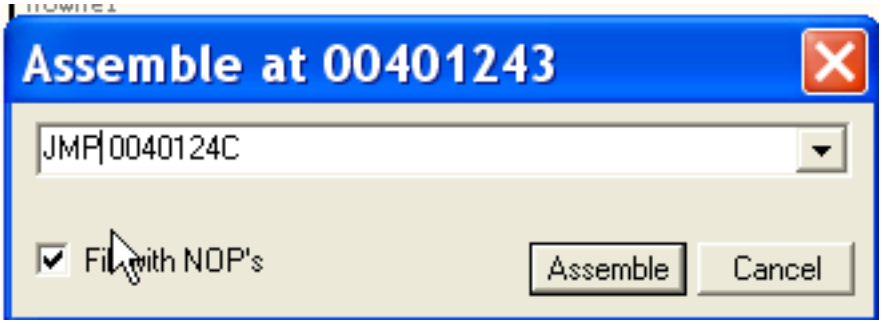
-

Идём ко второму ключевому условному переходу.

```
00401240    58          POP EAX
00401241    3BC3        CMP EAX,EBX
00401243    74 07       JE SHORT CRACKME.0040124C
00401245    E8 18010000 CALL CRACKME.00401362
0040124A    EB 9A       JMP SHORT CRACKME.004011E6
0040124C    E8 FC000000 CALL CRACKME.0040134D
00401251    EB 93       JMP SHORT CRACKME.004011E6
00401253    C8 000000   ENTER 0,0
00401257    53          PUSH EBX
00401258    56          PUSH ESI
```

-

В данном случае нам нужно, чтобы переход всегда выполнялся. Для этого его можно сделать безусловным, т.е. поменять JE на JMP.

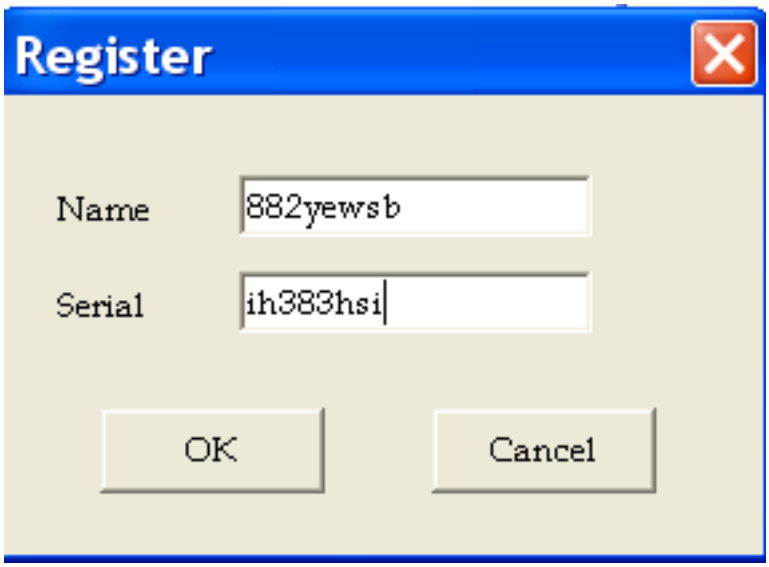


-

```
00401238  .   E8 9B010000   CALL CRACKME.004013D8
0040123D  .   83C4 04       ADD ESP,4
00401240  .   58            POP EAX
00401241  .   3BC3          CMP EAX,EBX
00401243  v   EB 07         JMP SHORT CRACKME.0040124C
00401245  .   E8 18010000   CALL CRACKME.00401362
0040124A  .^  EB 9A         JMP SHORT CRACKME.004011E6
0040124C  >   E8 FC000000   CALL CRACKME.0040134D
00401251  .^  EB 93         JMP SHORT CRACKME.004011E6
00401253  .   C8 000000     ENTER 0,0
00401257  .   53            PUSH EBX
00401258  .   56            PUSH ESI
00401259  .   57            PUSH EDI
```

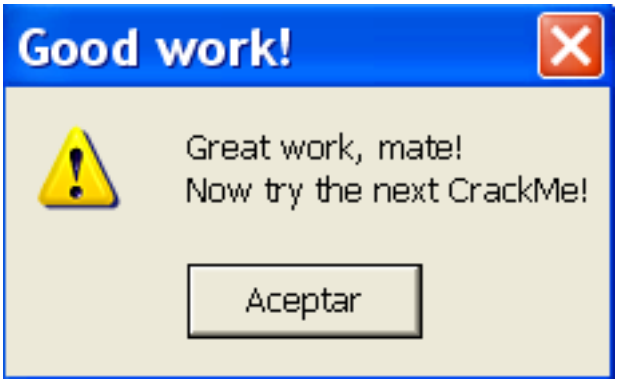
-

Давайте снимем и эту точку останова (F2). Пока ещё не выходим из OllyDbg! Чтобы опробовать в свободном полёте наши достижения, нажимаем F9.



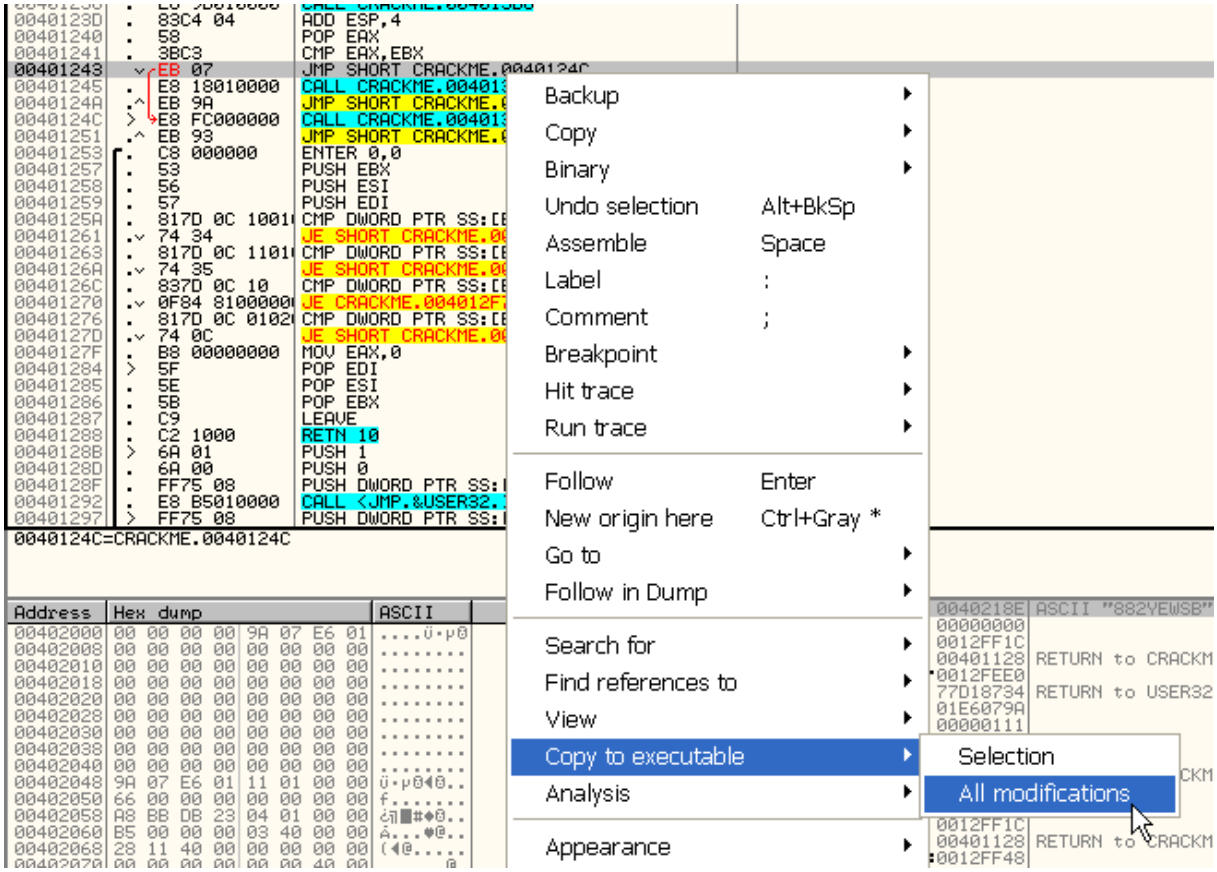
-

Нажимаем OK.



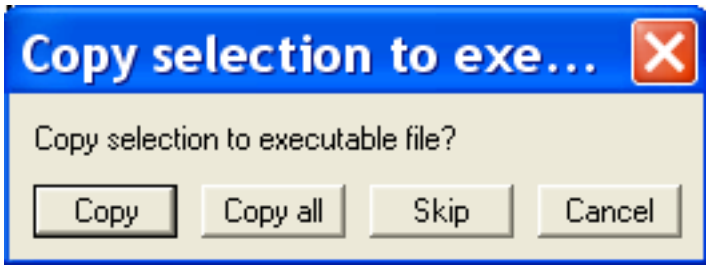
-

Все изменения проделанные через опцию Assemble (пробел) улетучиваются при закрытии программы в отладчике. Эти изменения осуществляются в памяти и никак не влияют на оригинальный образ в экзешнике. Чтобы применить изменения к образу в экзешнике, делаем следующие манипуляции:



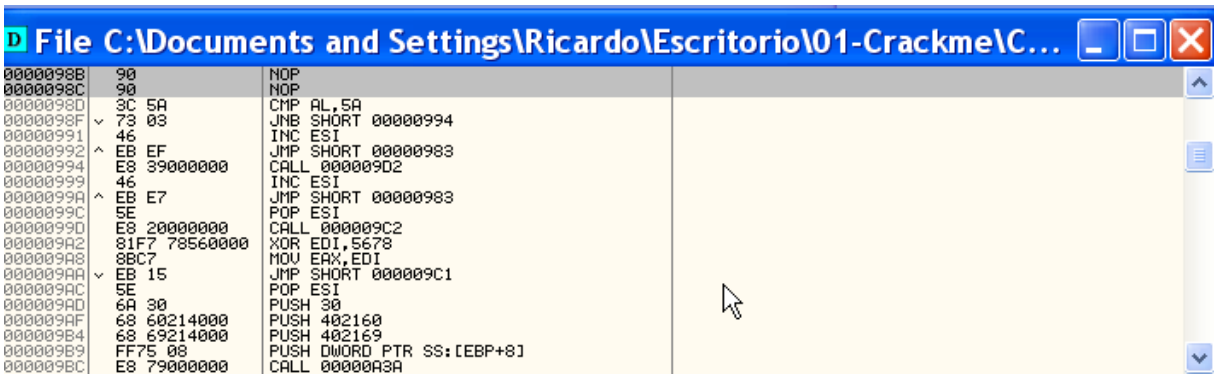
-

Правый щелчёк в любом месте листинга - Copy to executable - All modifications. Далее выводится следующий запрос:



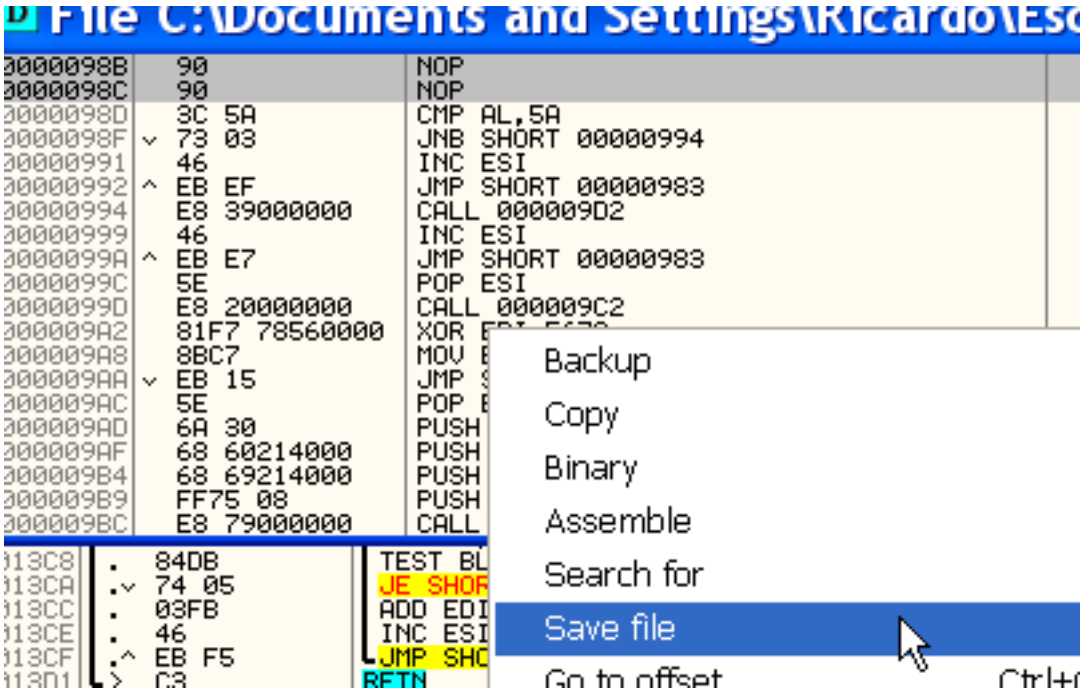
-

Выбираем опцию Copy all чтобы перенести все изменения (в данном случае ими являются двойной NOP и один JMP).

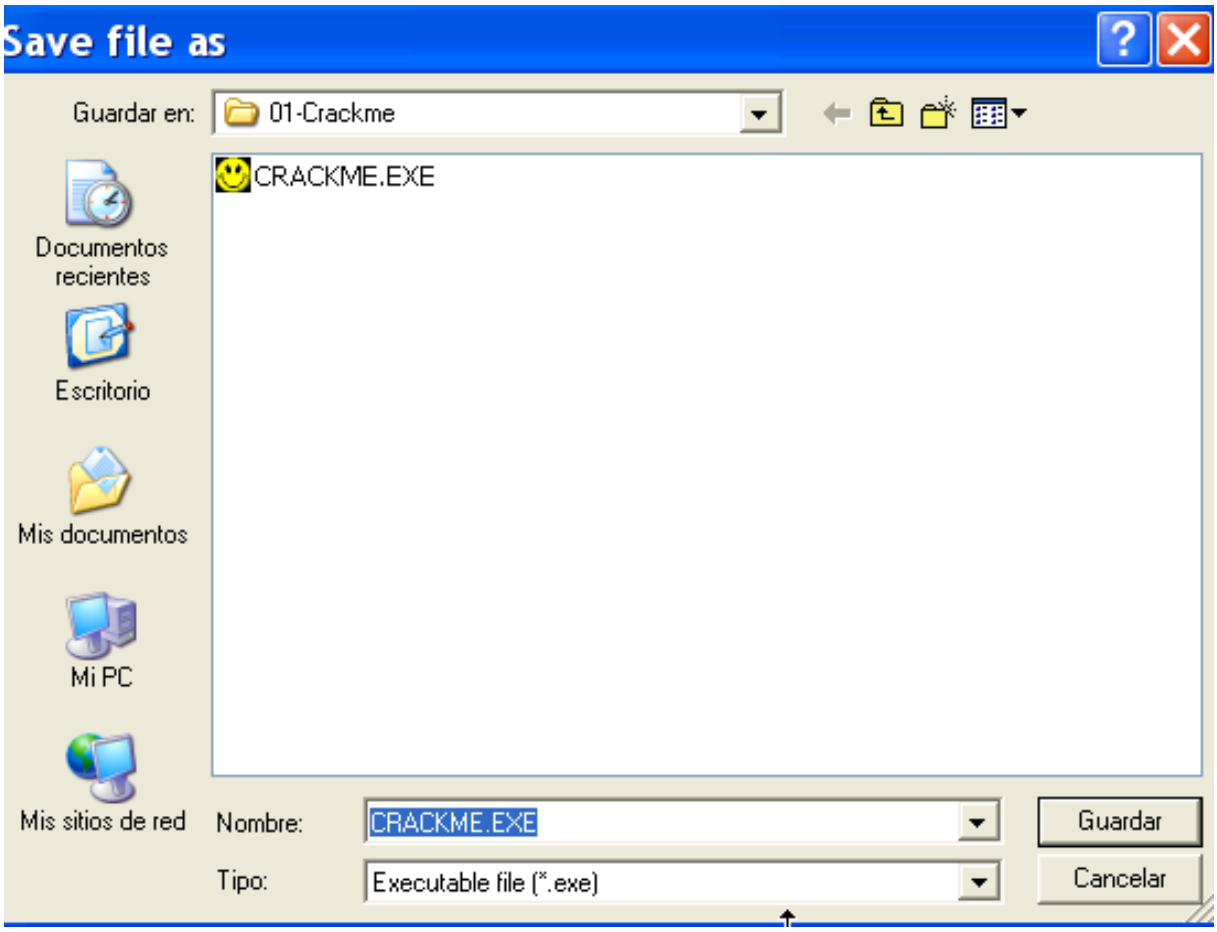


-

Открывается новое окно, в котором нужно опять сделать правый щелчёк - Save file.



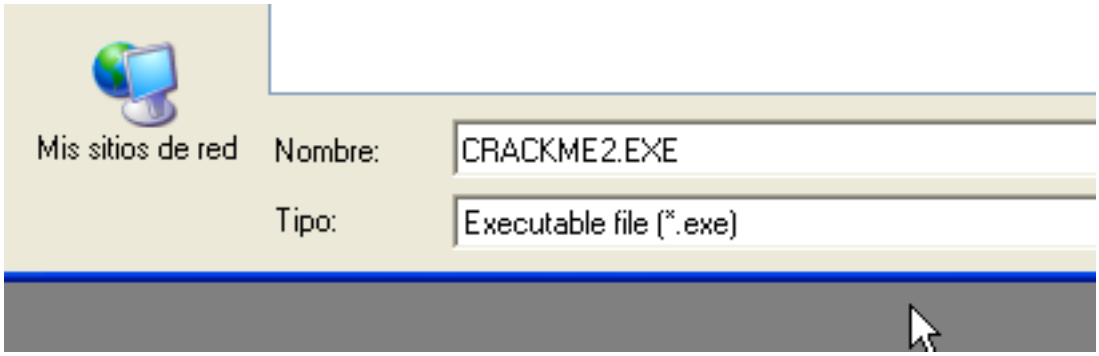
-



-

Сохраняем файл под НОВЫМ именем (старый экзешник нам ещё пригодится, да и перезаписать его всё равно не получится, т.к. он всё ещё загружен операционной системой). Назовём новый

экзешник CRACKME2.EXE.

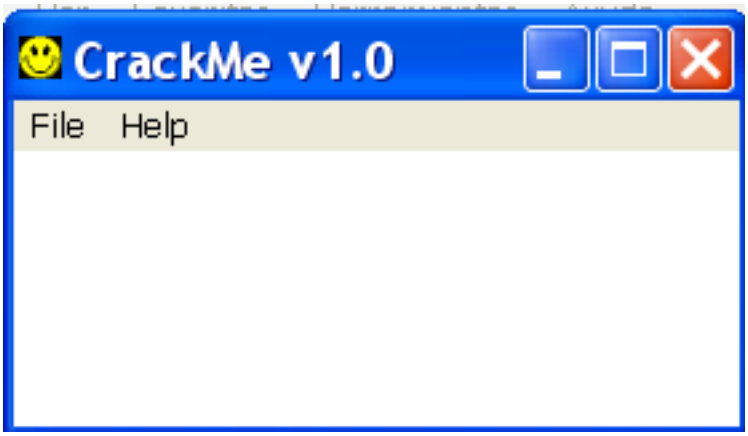


-

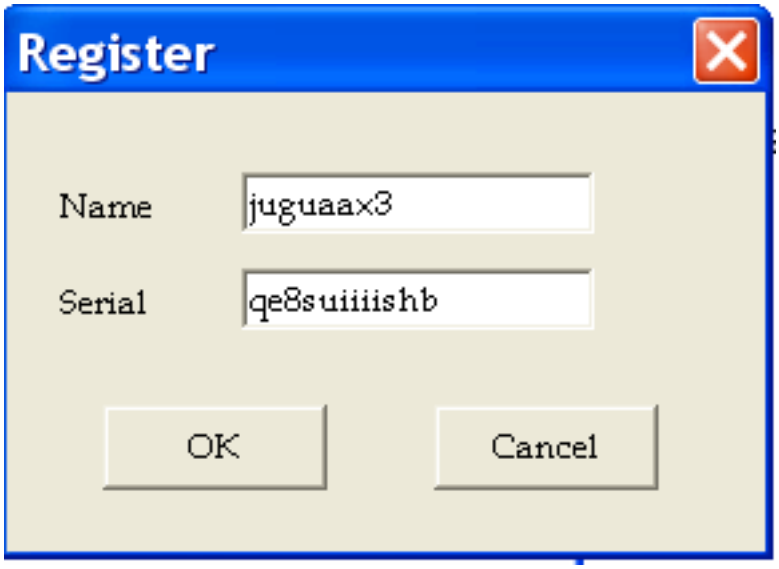
Вот, теперь можно, наконец, закрыть OllyDbg.

-

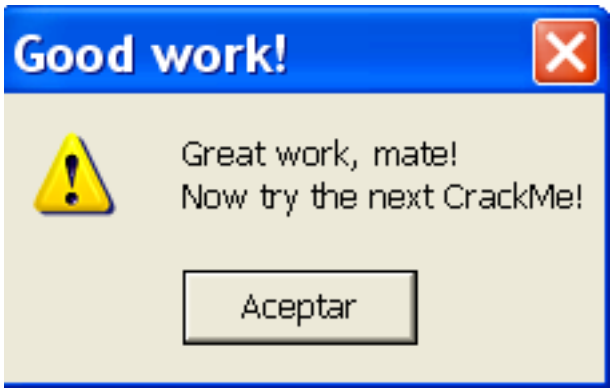
Запускаем новую версию крэкми (уже вне отладчика) и проверяем, что регистрация проходит успешно при указании любого серийника.



- Задаём Help - Register:



- Нажимаем OK.



-

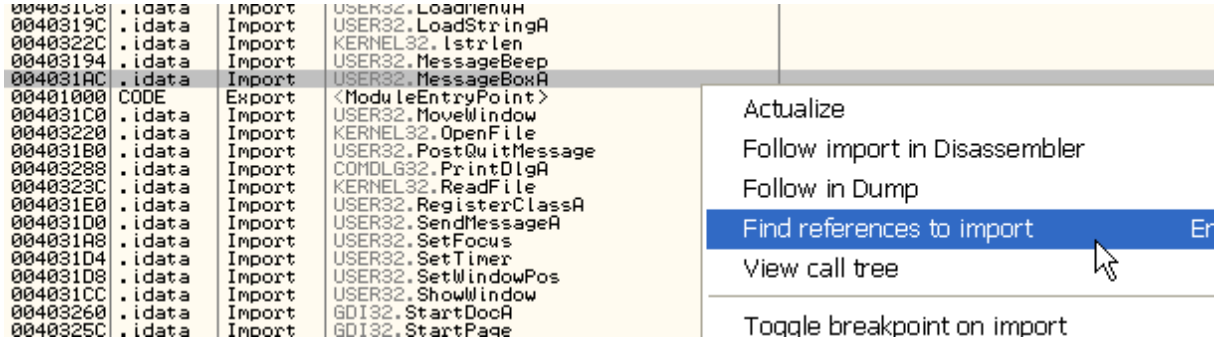
Вот мы и пропатчили крэкми! Но этого мало - в дальнейшем попробуем реверсировать полностью алгоритм проверки серийника, чтобы получить настоящие регистрационные данные, без использования патча. Но для этого надо ещё многому научиться.

Приложение. Windows 9x

В мастдае возможности отладчика ограничены. К примеру, нельзя ставить точки останова прямо на вызовы API-функций, как было показано на примере MessageBox в NT/XP. При выполнении поиска API-функций по импорту:

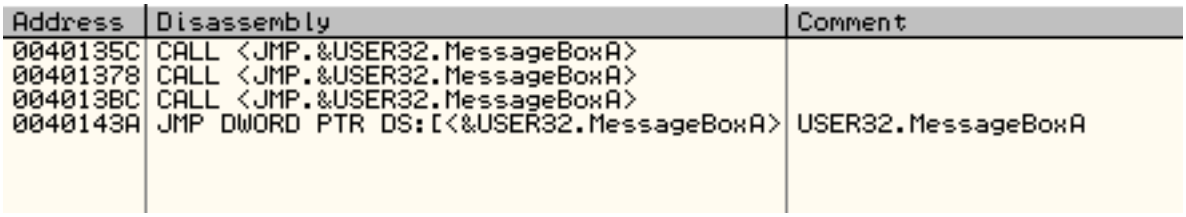
-

Получаем аналогичный список импортируемых символов, но опция Toggle breakpoint on import не будет доступна, т.к. 9x не позволяет ставить точки останова прямо на внешние функции.



-

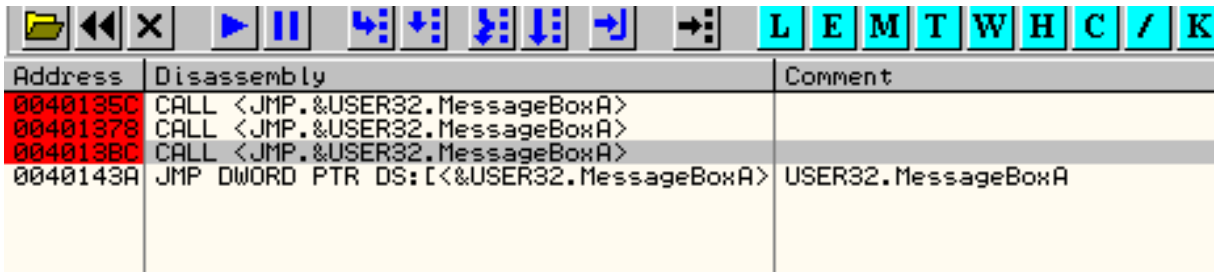
В остальном, контекстное меню будет сходным с полнофункциональным вариантом в NT.



-

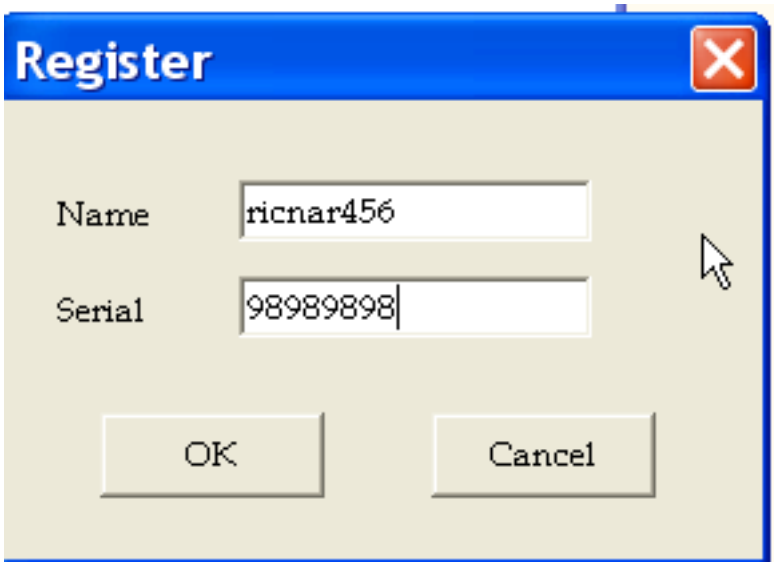
Таким образом, вызовы API-функций могут идти в обход точек останова. Достаточно вызвать функцию косвенным образом, чтобы анализатор не смог его автоматически засечь, и точка останова уже не сработает. Досадное ограничение, в общем.

Вместо одной точки останова на начало функции MessageBox в данном случае Вам придётся поставить 3 точки останова (F2): по одной на каждый вызов MessageBox, которые выводятся в списке внешних символов.



-

Нажимаем Run (F9); выбираем опцию Help - Register; вводим следующие регистрационные данные:



-

Нажимаем OK - срабатывает одна из точек останова.

```
0040139D   .  E8 20000000   CALL CRACKME.004013C2
004013A2   .  81F7 7856000( XOR EDI,5678
004013A8   .  8BC7          MOV EAX,EDI
004013AA   .v EB 15         JMP SHORT CRACKME.004013C1
004013AC   >  5E            POP ESI
004013AD   .  6A 30         PUSH 30                          Style = MB_OK|MB_ICONEXCLAMATION|MB_APPLMODAL
004013AF   .  68 60214000   PUSH CRACKME.00402160           Title = "No luck!"
004013B4   .  68 69214000   PUSH CRACKME.00402169            Text = "No luck there, mate!"
004013B9   .  FF75 08       PUSH DWORD PTR SS:[EBP+8]        hOwner
004013BC   .  E8 79000000   CALL <JMP.&USER32.MessageBoxA>   MessageBoxA
004013C1   L> C3            RETN
004013C2   r$ 33FF          XOR EDI,EDI
004013C4   .  33DB          XOR EBX,EBX
004013C6   >  8A1E          MOV BL,BYTE PTR DS:[ESI]
004013C8   .  84DB          TEST BL,BL
004013CA   .v 74 05         JE SHORT CRACKME.004013D1
```

-

В NT/XP в этом месте мы ставили ещё одну точку останова на инструкции RET, внутри функции MessageBox. В 9x мы это не можем сделать, т.к. внутрь функции нас не пускают. Поэтому ограничимся точкой останова на следующей после CALL инструкции, т.е. уже после возвращения из функции.

```
004013AF   .  68 60214000   PUSH CRACKME.00402160           Title = "No luck!"
004013B4   .  68 69214000   PUSH CRACKME.00402169            Text = "No luck there, mate!"
004013B9   .  FF75 08       PUSH DWORD PTR SS:[EBP+8]        hOwner
004013BC   .  E8 79000000   CALL <JMP.&USER32.MessageBoxA>   MessageBoxA
004013C1   L> C3            RETN
004013C2   r$ 33FF          XOR EDI,EDI
004013C4   .  33DB          XOR EBX,EBX
004013C6   >  8A1E          MOV BL,BYTE PTR DS:[ESI]
```

-

Аналогично поступаем в любой подобной ситуации по ходу развития учебного цикла. В 9x вместо точек останова на обращениях к внешим функциям, их приходится ставить на ссылки в конкретных вызовах. Теперь можете вернуться к основному материалу статьи , где написано:

Дальнейшее обсуждение касается обоих семейств Windows: 9x и NT.

Введение в крэкинг с нуля, используя OllyDbg - Глава 10

Автор: Рикардо Нарваха, пер. Quantum

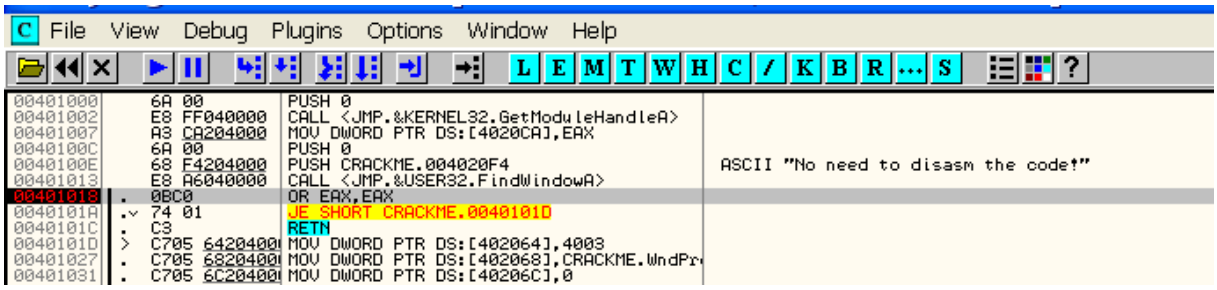
Дата: 2007-07-22

Точки останова Эту главу мы посвятим изучению различных видов точек останова. Они позволяют приостановить выполнение кода программы в нужном месте. И снова роль подопытной жертвы достаётся бессменному крэкми CrueHead'a. **Обычная точка останова, она же BPX**

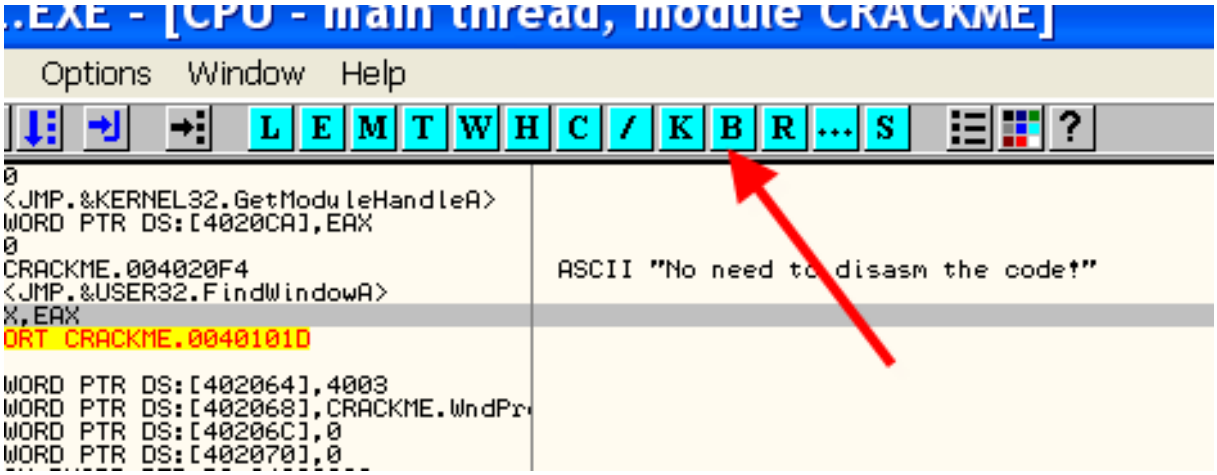
Это самая обыкновенная точка останова, которую мы уже не раз использовали в предыдущих главах. Иногда она обозначается сокращённо BPX как одноимённая команда, которая используется в отладчике SoftIce для установки обычных точек останова. Справедливости ради, стоит заметить, что в OllyDbg используется сокращение BP для обозначения данного вида точек останова.

Для установки BPX достаточно выделить нужную строчку кода и нажать F2. Повторное нажатие F2 снимает BPX.

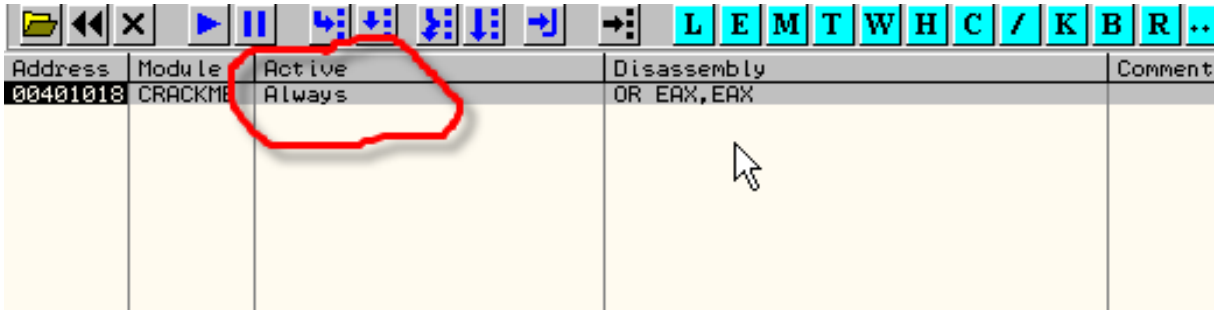
Смотрим крэкми CrueHead'a на точке входа:



Выделяем, например, строчку 401018; нажимаем F2 - адрес выделился красным цветом, чтобы показать, что тут находится обыкновенная точка останова. Данный адрес также добавляется в список точек останова в окне [B]reakpoints.

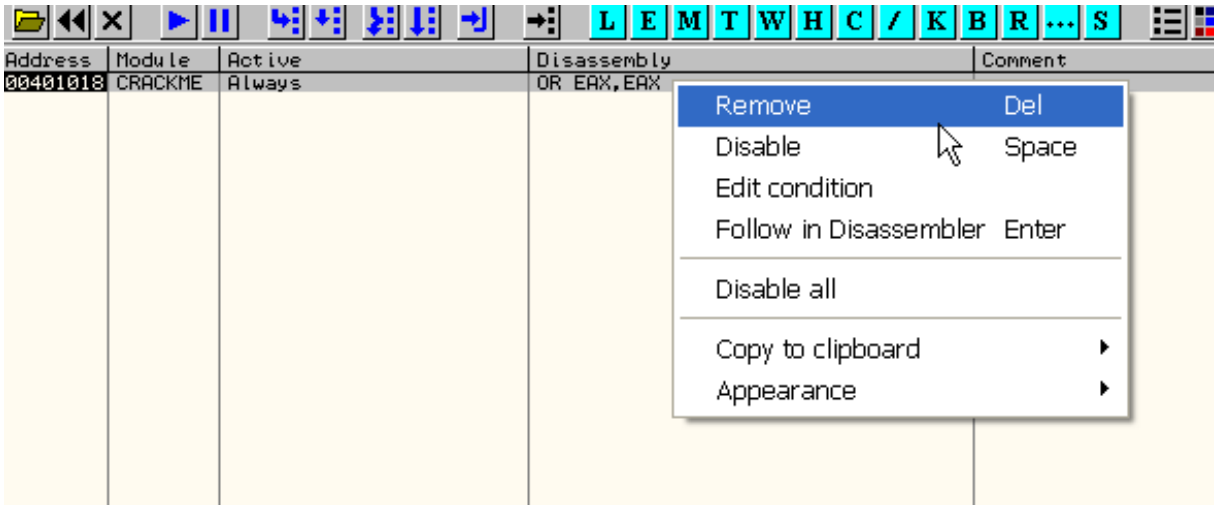


В списке появился наш BPX и видно, что он активен в данный момент.

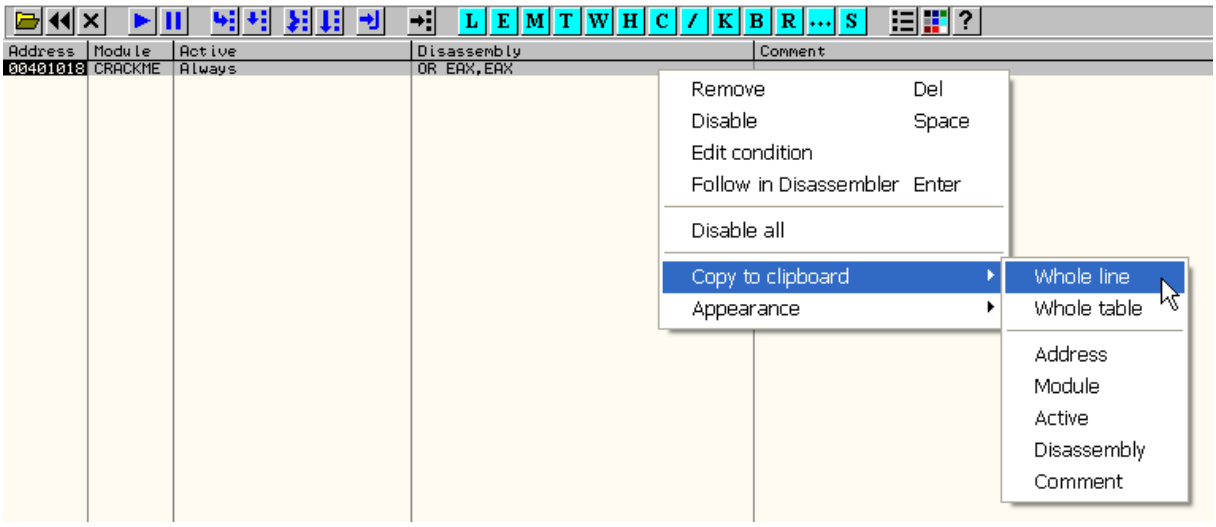


В колонке Active отображается статус Always, а это значит, что данный BPX (должен - прим. пер.) будет выполнен всегда, т.е. он всё время активен.

Правый щелчёк мышью в списке BPX'ов - всплывает контекстное меню, через которое можно индивидуально контролировать точки останова.



• REMOVE: удалить данную точку останова из списка равносильно её снятию.
• DISABLE: отключить точку останова, не удаляя её из списка. В отключенном состоянии точка останова не будет срабатывать.
• EDIT CONDITION: задаёт условие, таким образом, превращая обычную точку останова в условную, но об этом мы поговорим позже.
• FOLLOW IN DISASSEMBLER: показать эту точку останова в листинге дизассемблера.
• DISABLE ALL / ENABLE ALL: отключить / включить все точки останова в списке. В данном случае нет опции Enable all, так как единственная точка останова в списке не отключена.
• COPY TO CLIPBOARD: помещает в буфер обмена информацию о выбранной точке останова. Давайте попробуем это сделать:

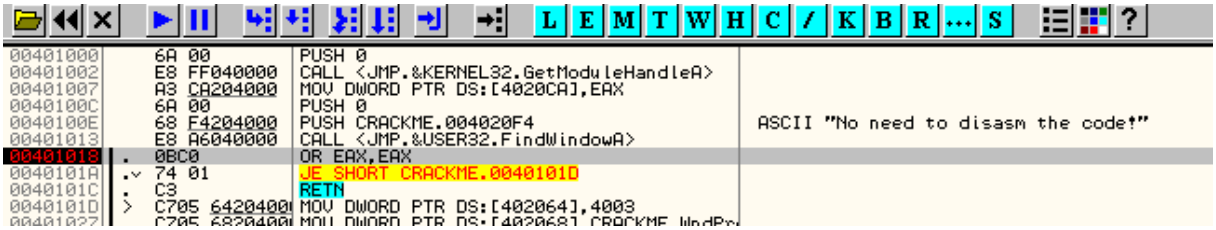


Опция Whole line помещает в буфер обмена информацию об одной (выбранной) точке останова, тогда как Whole table распространяется сразу на все точки останова, находящиеся в списке.

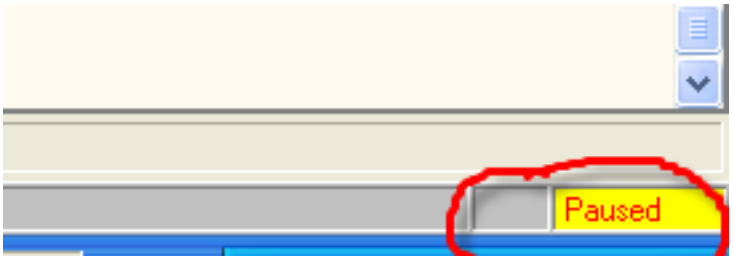
```
Breakpoints, item 0
 Address=00401018
 Module=CRACKME
 Active=Always
 Disassembly=OR EAX,EAX
```

В буфер помещаются: инструкция, на которой висит точка останова, статус этой точки, адрес и т.д.

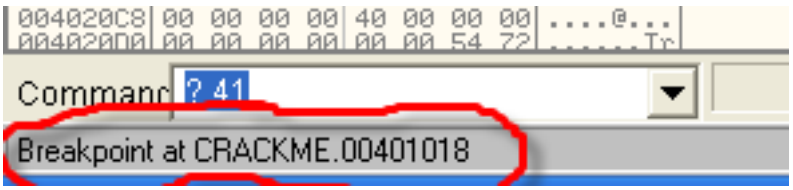
Нажимаем F9 (Run) - отладчик начинает выполнять крэкми, но, вскоре, достигает точки останова и, как можно догадаться, останавливается.



В строке состояния указано, что отладка приостановлена.

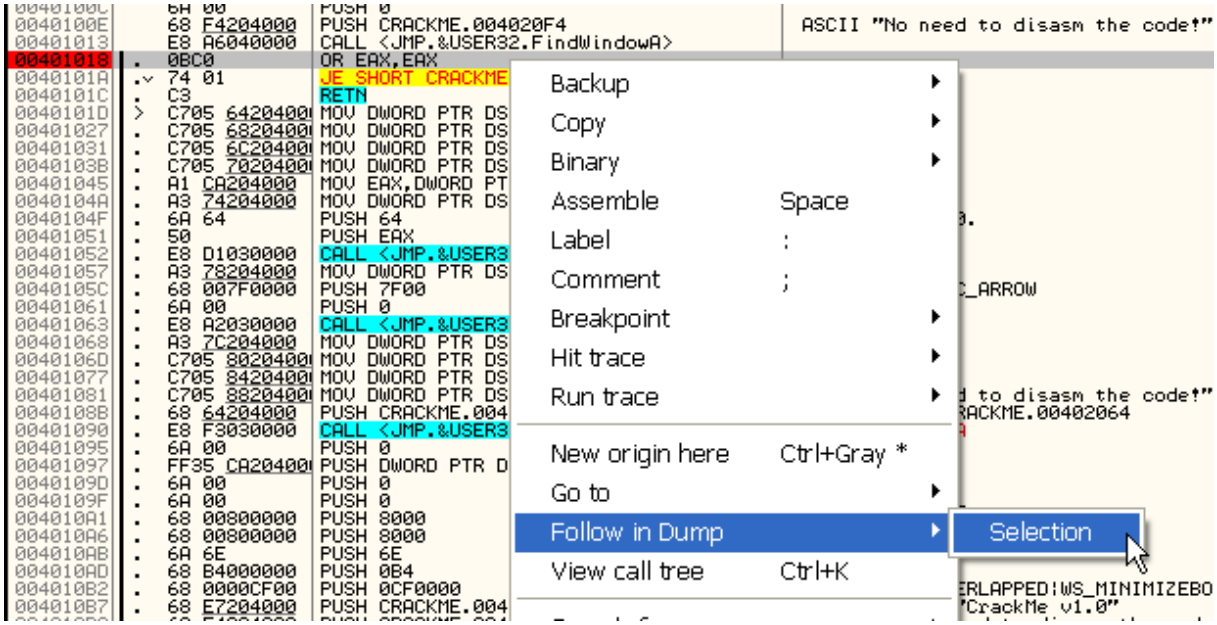


И причина, по которой произошла остановка:

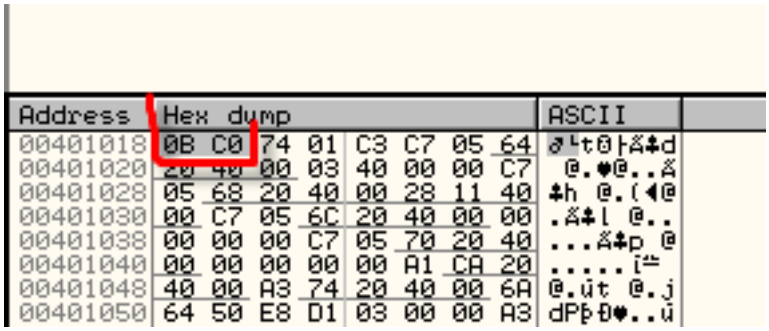


Давайте попробуем разобраться, что же на самом деле представляет собой точка останова. Происходит ли какое-либо изменение в двоичном коде инструкции при установке точки останова?

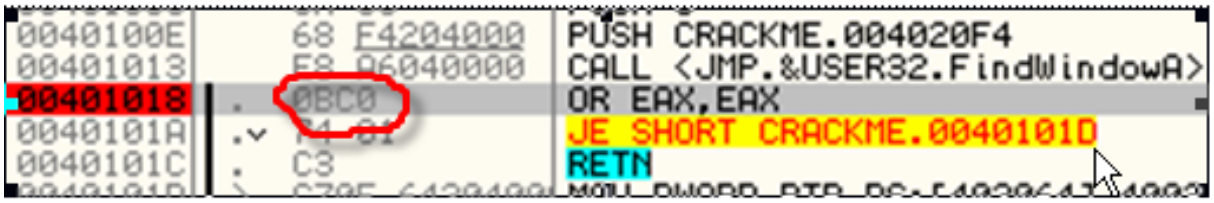
Правый щелчёк: Follow in Dump – Selection



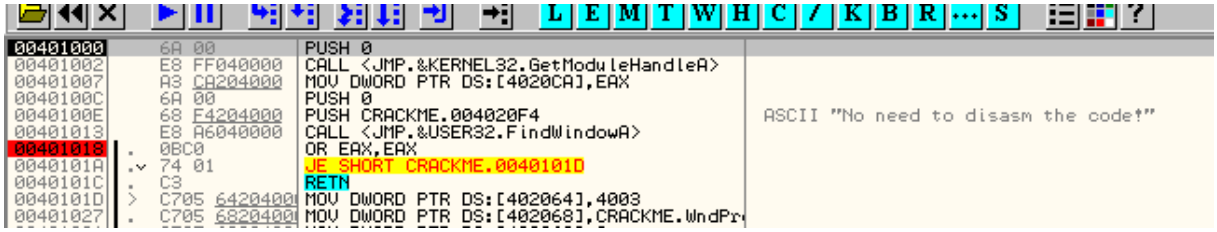
Смотрим в дампе содержимое по адресу 401018:



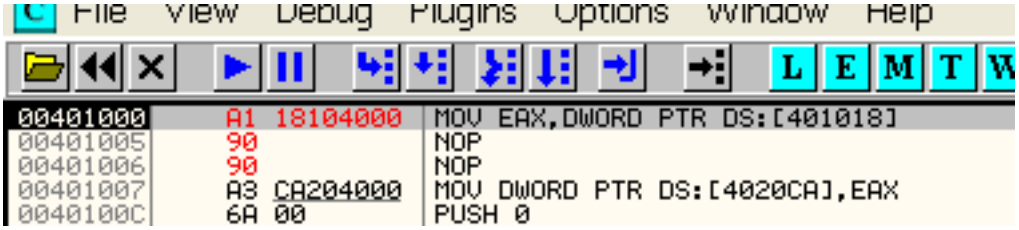
На первый взгляд, содержимое дампа совпадает с первоначальным кодом инструкции. И в листинге дизассемблера в колонке машинного кода мы видим то же самое:



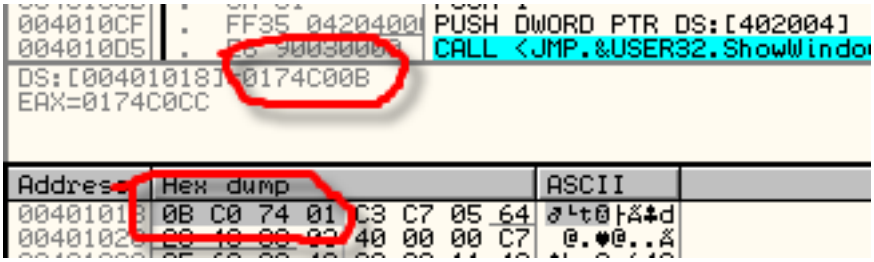
Везде - и в дампе, и в дизассемблере - мы видим значение 0B C0, которое соответствует инструкции OR EAX,EAX. Похоже, что никаких изменений нет, но так ли это на самом деле? Перезагрузите крэкми и убедитесь, что точка останова всё ещё висит на месте (по адресу 401018).



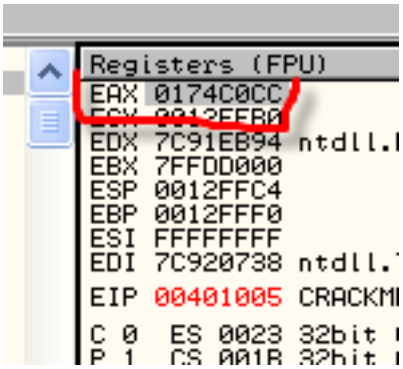
Давайте введём следующую инструкцию, чтобы фактически прочитать из памяти машинный код инструкции OR EAX,EAX, на которой висит точка останова:



Данная инструкция помещает в EAX значение ячейки по адресу 401018. Что нам по этому поводу подсказывает отладчик?



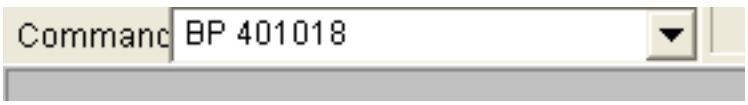
В дампе, в листинге и теперь ещё и в окне подсказки - везде показано одно и то же значение: 0B C0. Тем не менее, давайте всё-таки нажмём F7 и проверим что окажется в EAX.



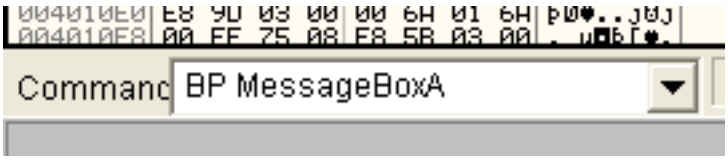
По адресу 401018 оказалось немного не то, что нам изначально показывал OllyDbg... Значение 0B C0 74 01 задом наперёд (т.е. в формате little endian) выглядит как 0174C00B, а вовсе не 0174C0CC. Итак, вместо 0B по адресу 401018 хранится CC. Каким образом это значение связано с точками останова мы детально рассмотрим позже, когда займёмся изучением исключений. Чтобы не "загрязнять" пользовательский интерфейс, отладчик скрывает от нас, что при установке обычной точки останова, первый байт инструкции заменяется на CC. Тем не менее, сама программа запросто может прочитать из памяти настоящее значение, что может использоваться для обнаружения точек останова в антиотладочных целях. Поэтому не стоит удивляться, когда обычные точки останова иногда мистическим образом не срабатывают. Быть может, сама программа их "палит" :)

Кроме F2, есть ещё способ установки обычной точки останова через CommandBar:

```
BP 401018
```



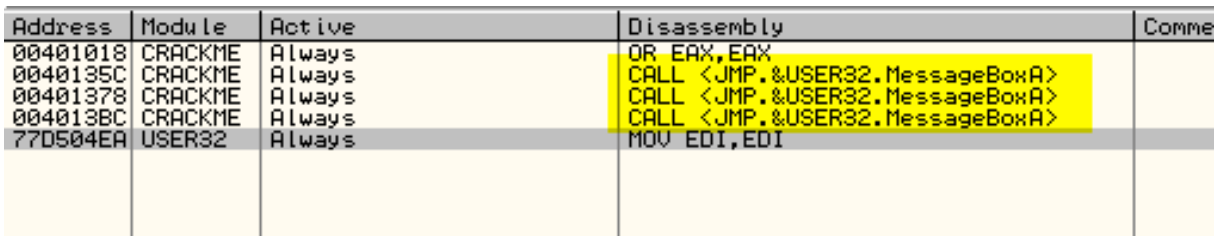
На NT (2000, XP и 2003) можно запросто ставить точки останова на адреса API-функций - мы это уже рассмотрели в предыдущей главе. Чтобы установить точку останова на обращения к MessageBoxA, вводим:



В данном случае нужно указывать точное имя, и это касается регистра букв. В Windows 98, напротив, точки останова прямо по адресам API не поддерживаются - вместо этого приходится довольствоваться точками останова на самих вызовах:

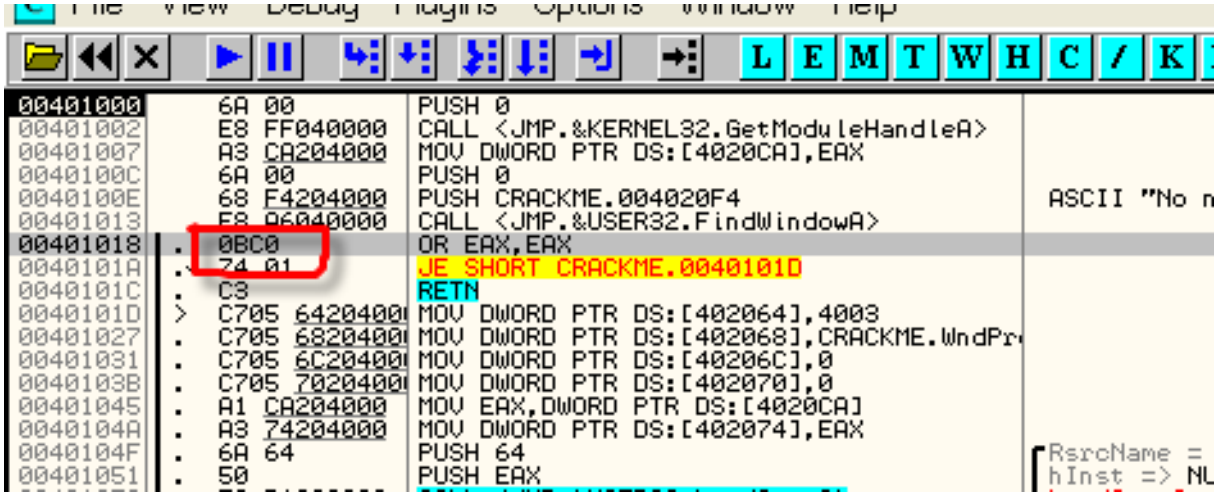
BPX ставит точку останова не на указанный адрес, как это делает BP, а на инструкции, которые ссылаются на данный адрес. Важно отметить, что отладчик может определить далеко не все обращения к API-функциям (вызовы могут быть "замаскированы" различными способами, чтобы сбить с толку крэкера - прим. пер.) BPX можно использовать и в NT, но обычно предпочтение отдаётся более надёжной команде BP.

Вот как выглядит список точек останова после использования BPX MessageBoxA. Как видите, отладчик обнаружил 3 вызова данной функции и поставил 3 точки останова: по одной на каждый вызов.



На протяжении всего цикла статей мы будет использовать преимущественно BP (пользователи Windows 98 будут использовать BPX - у них нет выбора). Так что, когда в тексте встречается термин BREAKPOINT или BPX, под этим следует понимать команду BP.

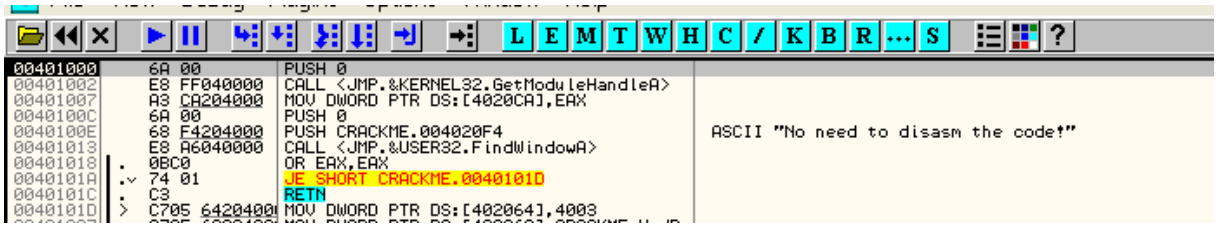
Ещё один способ установить точку останова: двойной щелчёк в области машинного кода (колонка с байтами) по строчке с инструкцией в окне дизассемблера. Повторный двойной щелчёк снимает точку останова.



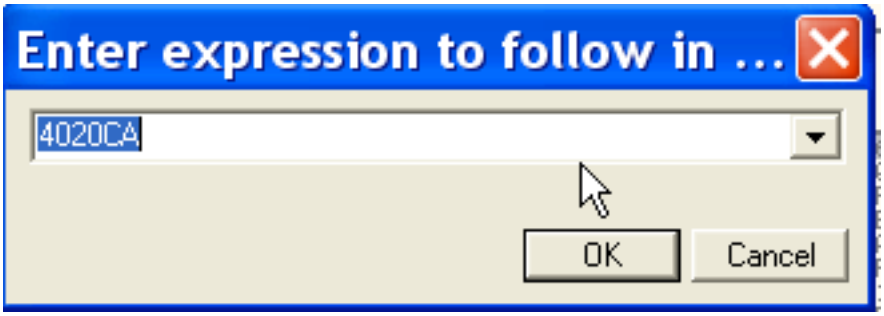
Точки останова на обращения к памяти

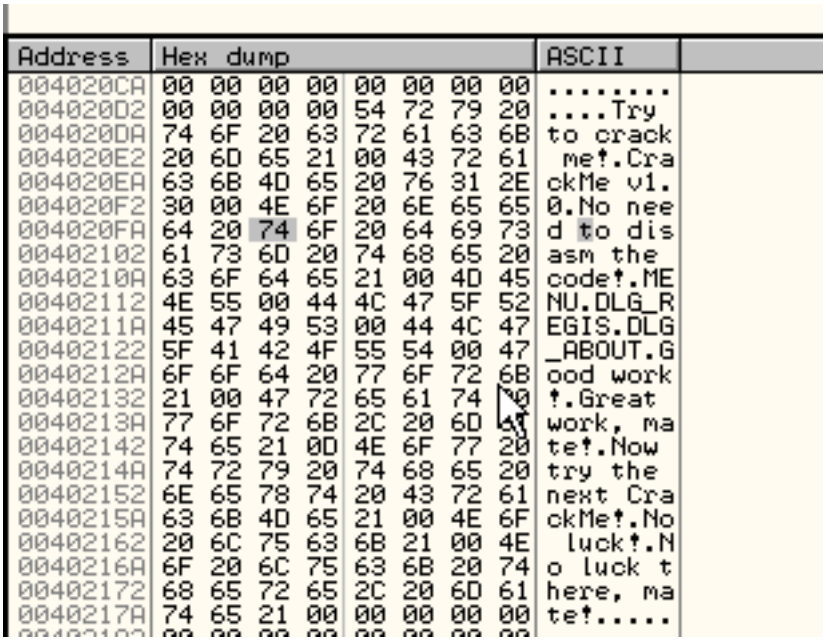
Точки останова на доступ к памяти иногда также называются BPM (от англ. Breakpoint on Memory), но не нужно путать это сокращение с командой BPM в старом добром SoftIce - это совершенно разные вещи.

Данный вид точек останова модифицирует свойства страниц памяти, которым принадлежит область, на которую вешается BPM. Например: BPM on access срабатывает на любых обращениях к данной области байт (чтение, запись или выполнение кода). Ещё существует BPM on write, который срабатывает только на обращениях на запись (чтение и выполнение в этом случае не отлавливаются). Давайте рассмотрим несколько примеров:

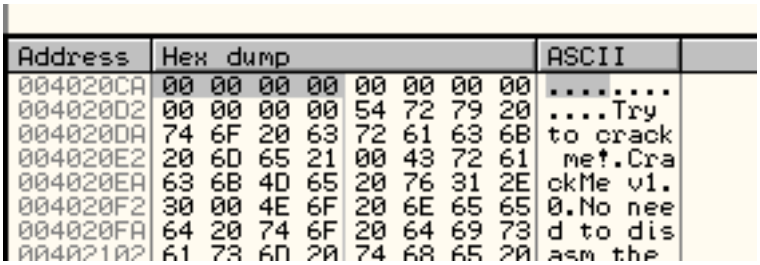


Находясь на точке входа в крэкми CrueHead'a, давайте попробуем установить несколько BPM. В окне дампа задаём Go to - Expression: 4020CA, чтобы переместиться на данный адрес.

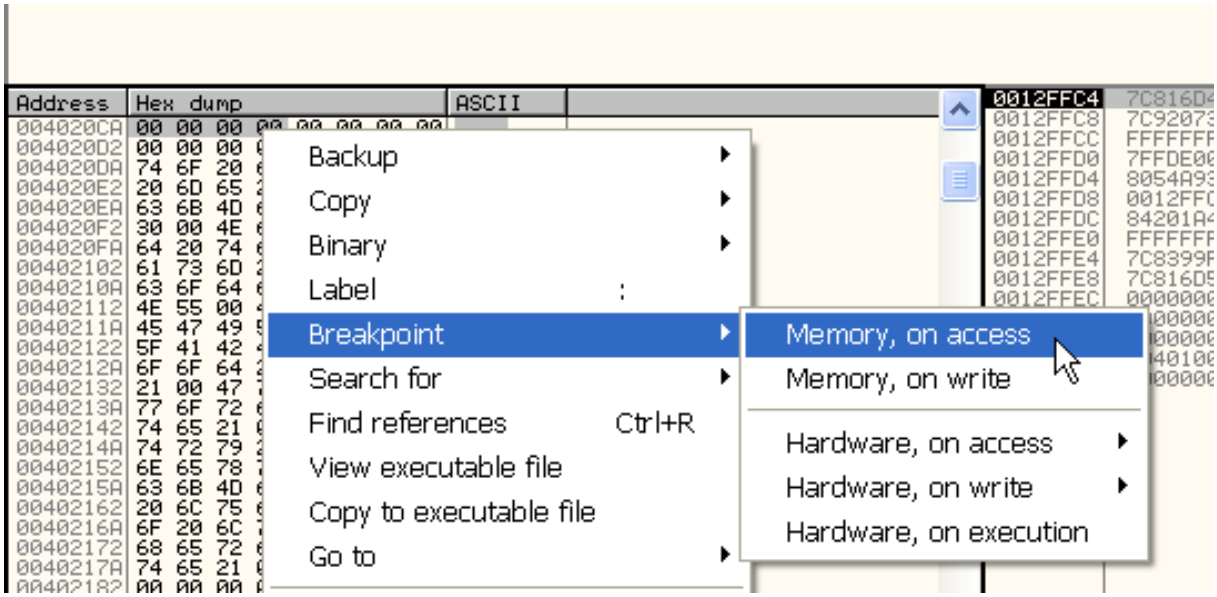




Тут мы и установим первый BPM ON ACCESS: на 4 байта по адресу 4020CA. В дальнейшем, после команды Run, данная точка останова должна будет сработать при попытке чтения содержимого этих байтов, записи или исполнения (если бы там вдруг оказался исполнимый код). Давайте выделим все 4 байта:

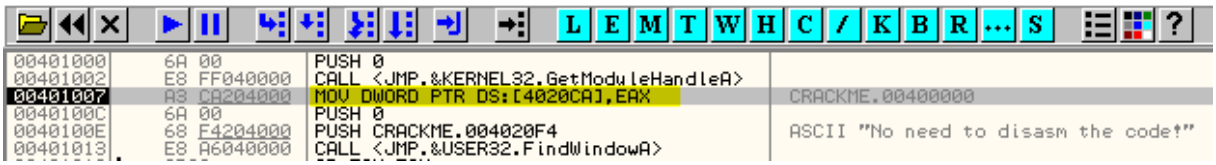


Область байт выделена, правый щелчёк мышью: Breakpoint - Memory, on access. Вовсе не обязательно использовать именно 4-байтовые BPM. Аналогично можно было выделить область меньшего или большего диапазона.

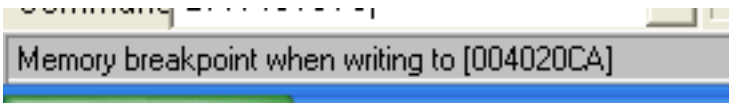


Есть у BPM'ов парочка недостатков: они не отображаются в списке [B], да и ни в каком другом месте. Поэтому приходится запоминать, где его установили. Причём нельзя одновременно использовать более одного. Если установить ещё один, старый BPM автоматически сотрётся.

Итак, даём команду Run... Остановка происходит по адресу 401007, где текущая инструкция пытается что-то записать по адресу, попадающему в диапазон нашего BPM, т.е. в 4020CA.

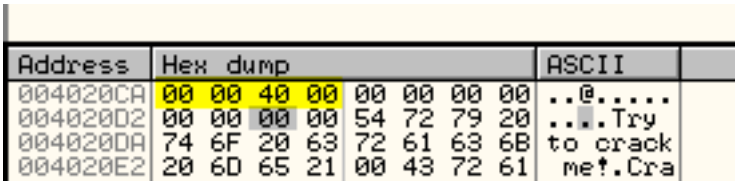


Внизу - в строке состояния - внятно описана причина, по которой мы остановились:

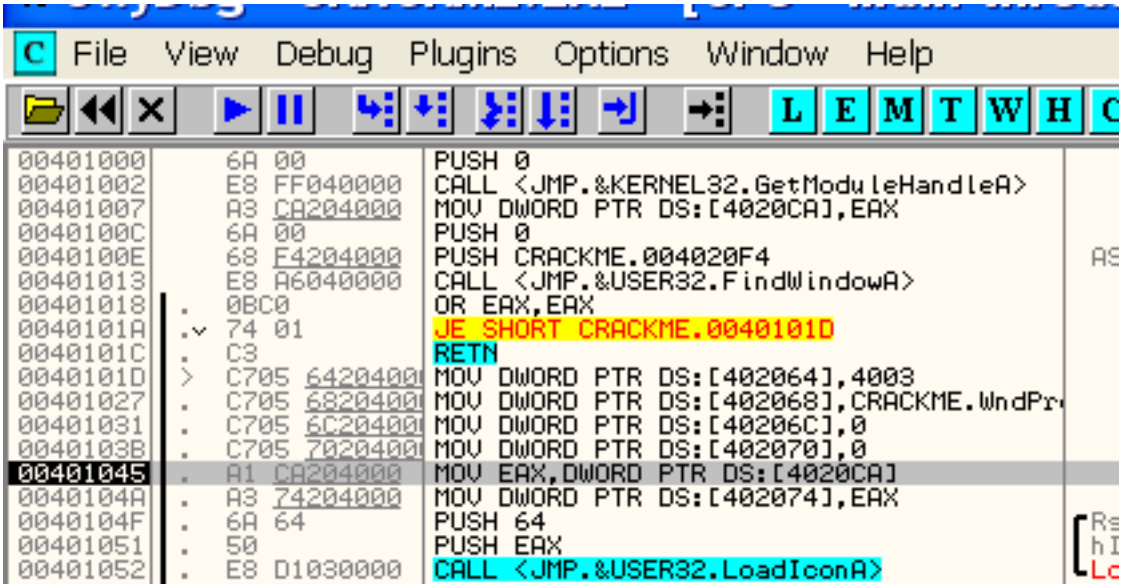


Текущая инструкция намеревается записать значение регистра EAX по адресу 4020CA, что и спровоцировало остановку. Помните, когда мы пытались что-то записать в область памяти, без свойства (W)riteable, происходило исключение? Собственно, аналогичным образом действует BPM on access: отладчик сбрасывает все свойства доступа страницы, которой принадлежит заданная область памяти. Любое обращение к данной странице провоцирует исключение, которое перехватывает отладчик.

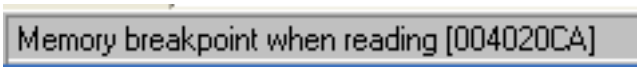
Если нажать F7, значение EAX всё-таки запишется, так как отладчик принимает меры, чтобы исключение в данном случае лишний раз не повторилось.



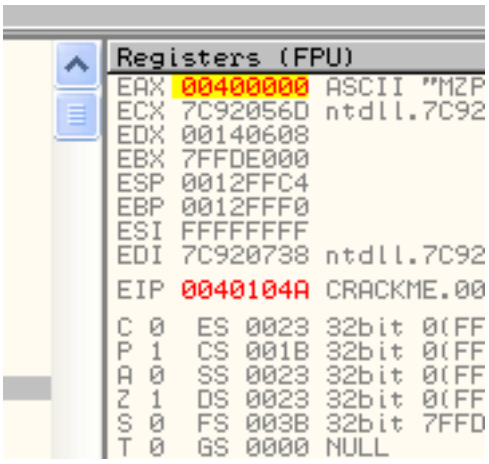
Значение 400000 успешно записалось. Давайте снова дадим команду Run. Так как BPM всё ещё активен, он сработает снова, если приложение попытается что-то записать/прочитать/исполнить по заданному адресу.



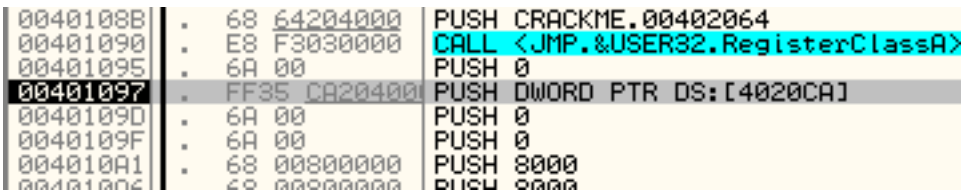
Как видите, теперь остановка произошла при попытке считать значение из той самой ячейки, о чём свидетельствует сообщение Memory breakpoint when reading [004020CA].



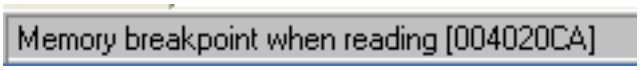
Опять же, по F7 операция всё-таки совершается, т.е. значение удачно считывается в EAX.



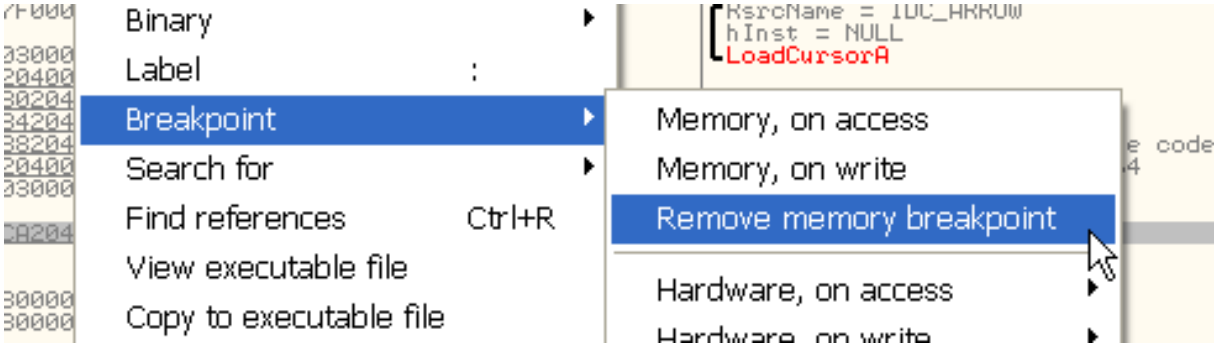
Снова F9... Сработает ли снова наш BPM? - Да, так и есть:



Отладчик снова засёк попытку чтения содержимого 4020CA, но на этот раз инструкцией PUSH, которая пытается скопировать это содержимое в стек.

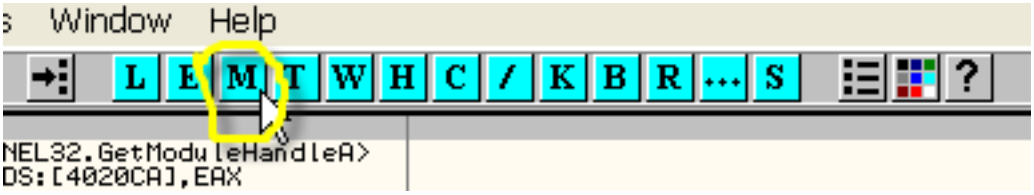


Чтобы удалить BPM, достаточно щёлкнуть правой кнопкой мыши в дампе: Breakpoint - Remove memory breakpoint. Также можно установить новый BPM - при этом старый, как мы уже говорили, ликвидируется.

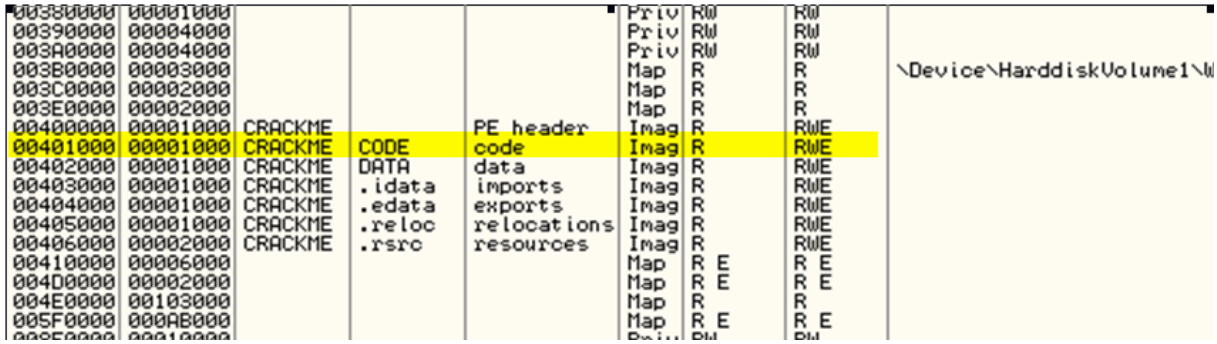


Если вместо "Memory, on access" указать "Memory, on write", BPM будет ловить только обращения на запись (чтение содержимого области PBM не будет вызывать остановку).

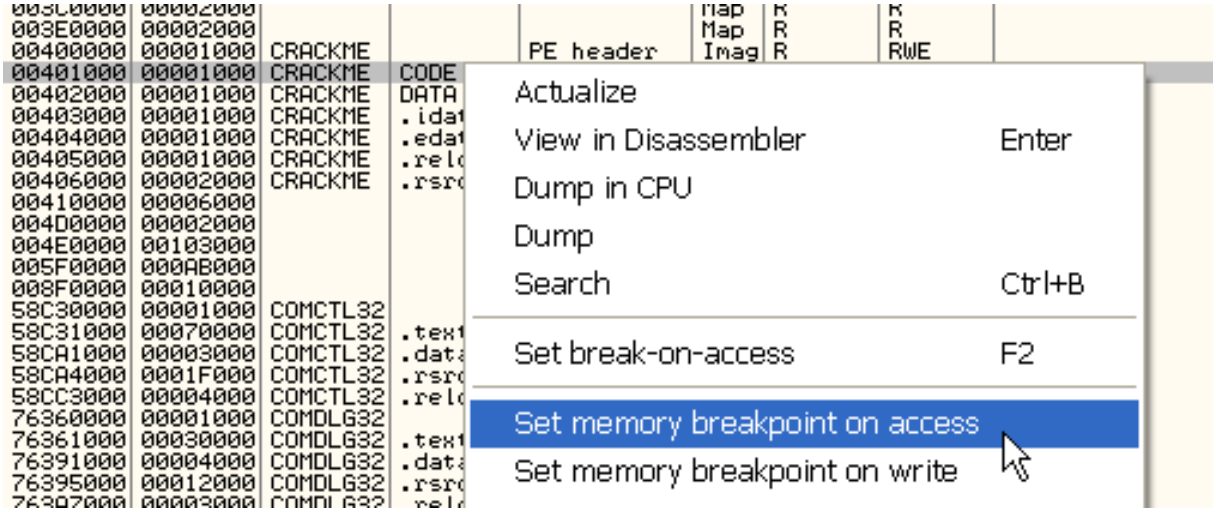
OllyDbg позволяет устанавливать BPM сразу на целую секцию (что приходится очень кстати при трассировании различного рода упаковщиков - прим. переводчика). Согласитесь, что выделять мышкой все байты нужной секции в дампе - мягко говоря, неудобно. Поэтому идём в закладку View [M]emory:



Данный список содержит в начале секции загруженного процесса (т.е. нашего подопытного крэкми), следом за которыми расположены секции загруженных библиотек DLL. Можете выбрать любую секцию, например ту, что начинается по адресу 401000:

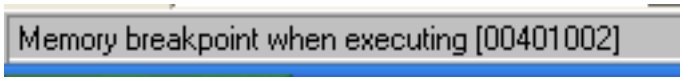


Правый щелчёк по выделенной секции: Set memory breakpoint on access.

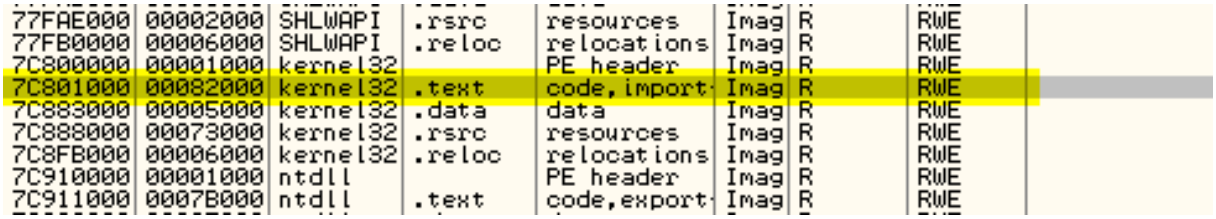


Можно было выбрать опцию on write, т.е. обращения на запись, но в данном случае речь идёт о секции кода, поэтому логичнее задействовать BPM on access. Командуем Run.

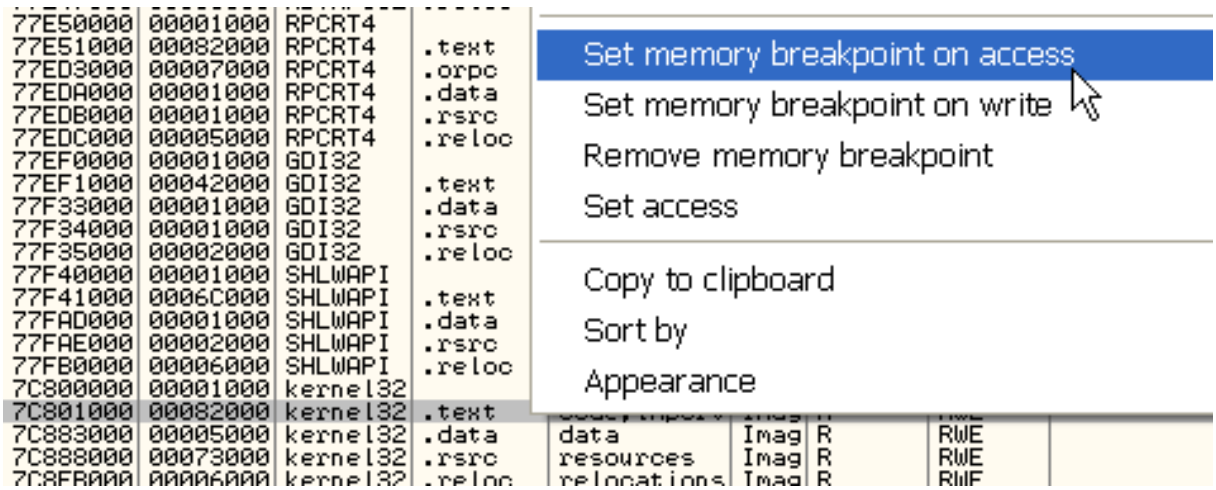
Точка останова срабатывает буквально на следующей же инструкции, так как:



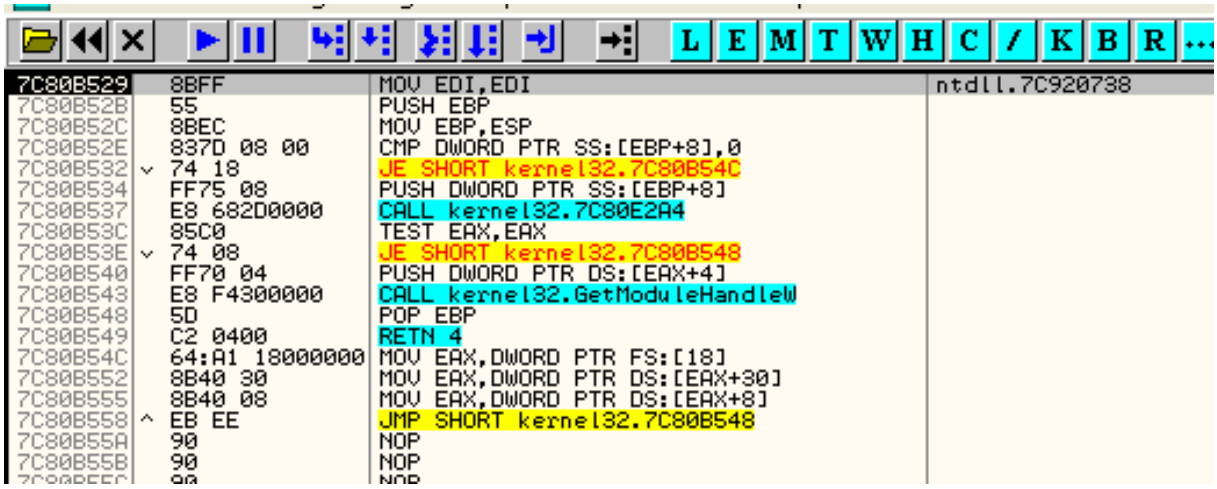
Правильно, остановка произошла when executing, т.е. при выполнении кода, ведь BPM установлен как раз на текущей секции кода и попытка выполнить любую инструкцию из этой секции приводит к срабатыванию BPM.



Давайте установим BPM на секцию кода kernel32, которую можно найти чуть ниже в списке. Вот она:

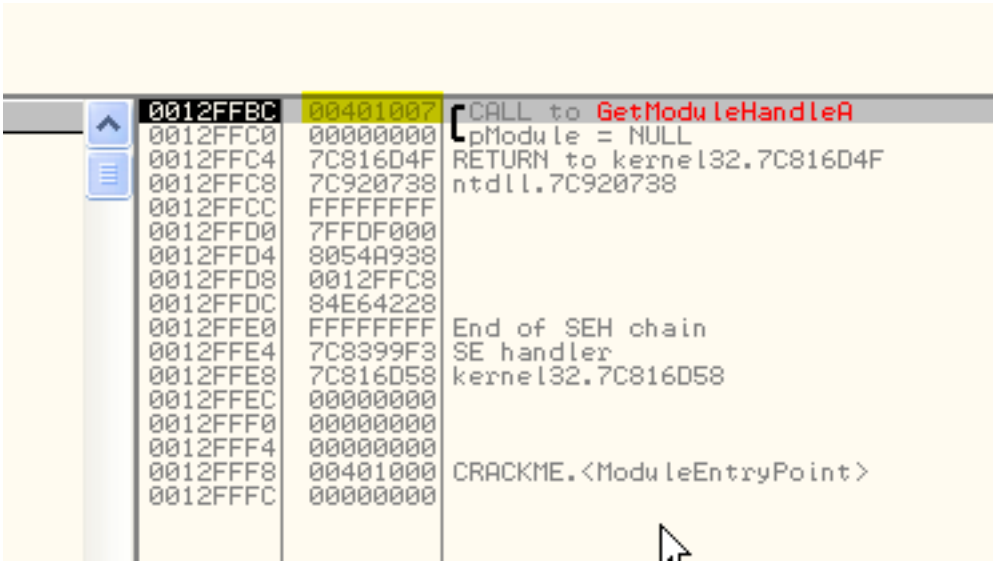


Условием по прежнему ставим on access, чтобы отлавливались все попытки чтения/записи/выполнения, проецирующиеся на данную секцию кода kernel32. Снова Run.

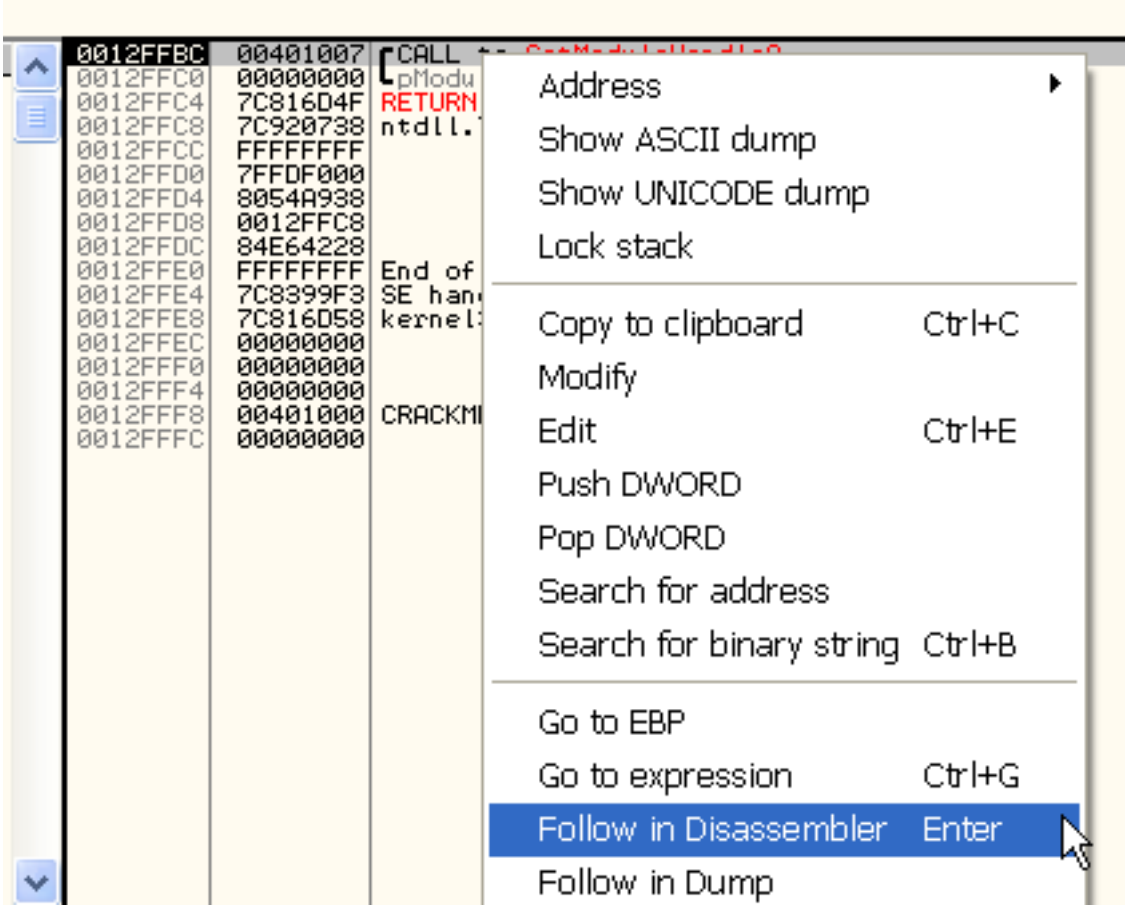


Судя по подсказке в стеке, остановка произошла на вызове API-функции:

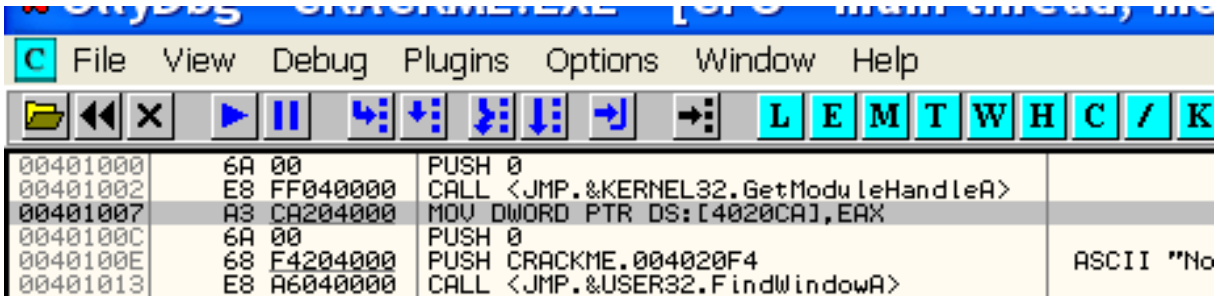
Таким образом можно отловить первое обращение к заданной DLL. Адрес возврата можно посмотреть в стеке:



Правый щелчёк по адресу возврата: Follow in disassembler.

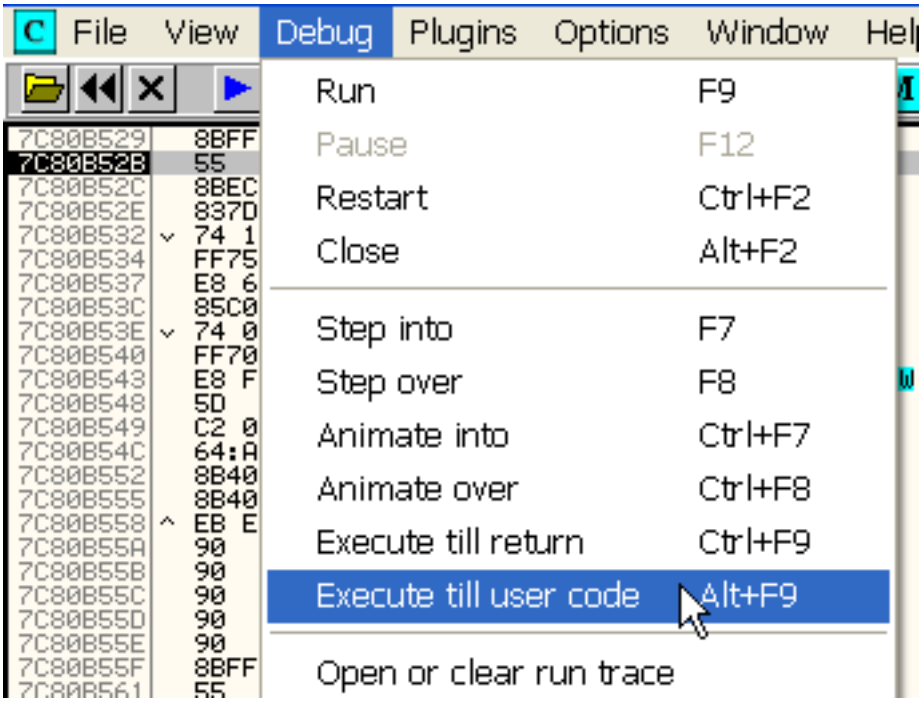


Адрес возврата указывает на 401007, сразу после вызова функции GetModuleHandleA в начале программы.



Чуть ниже находится вызов функции FindWindowA, но этот вызов не спровоцирует остановку по BPM, так как данная функция принадлежит другой DLL - user32.dll. Если снова задать Run, остановка произойдёт тут же на следующей инструкции, так как следующая инструкция тоже принадлежит секции, находящейся под влиянием BPM, ровно как и все остальные инструкции текущей функции. Иными словами, BPM будет срабатывать на каждом шагу. Поэтому мы его лучше снимем опцией Remove memory breakpoint.

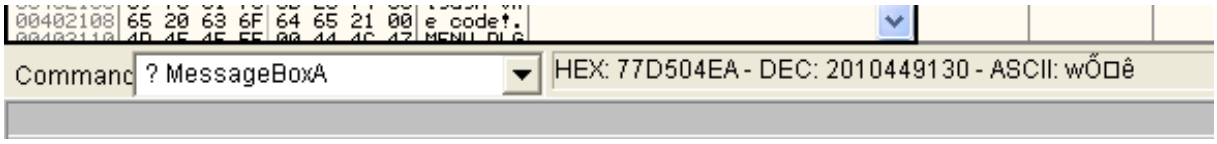
Чтоб вернуться снова в код основного модуля, правый щелчёк: Execute till user code. Иногда, если этот способ не срабатывает, можно задать опцию Execute till return, чтоб перескочить на выход из текущей функции, и потом трассировать далее привычными средствами: F7 и F8.



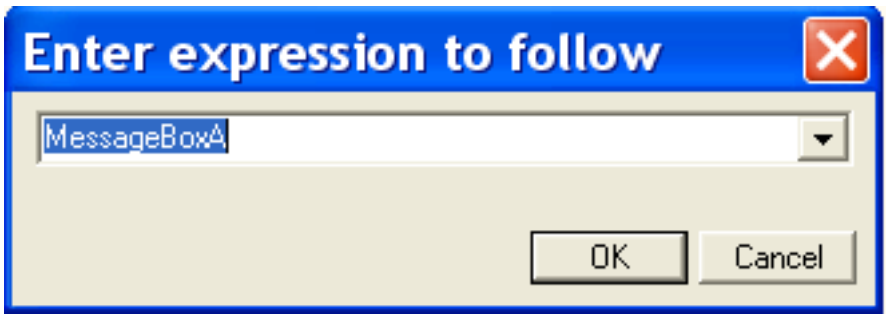
В данном случае Execute till user code вполне справляется с задачей вывести нас обратно в код основного модуля. Позже мы рассмотрим ситуации, в которых данный способ не применим.

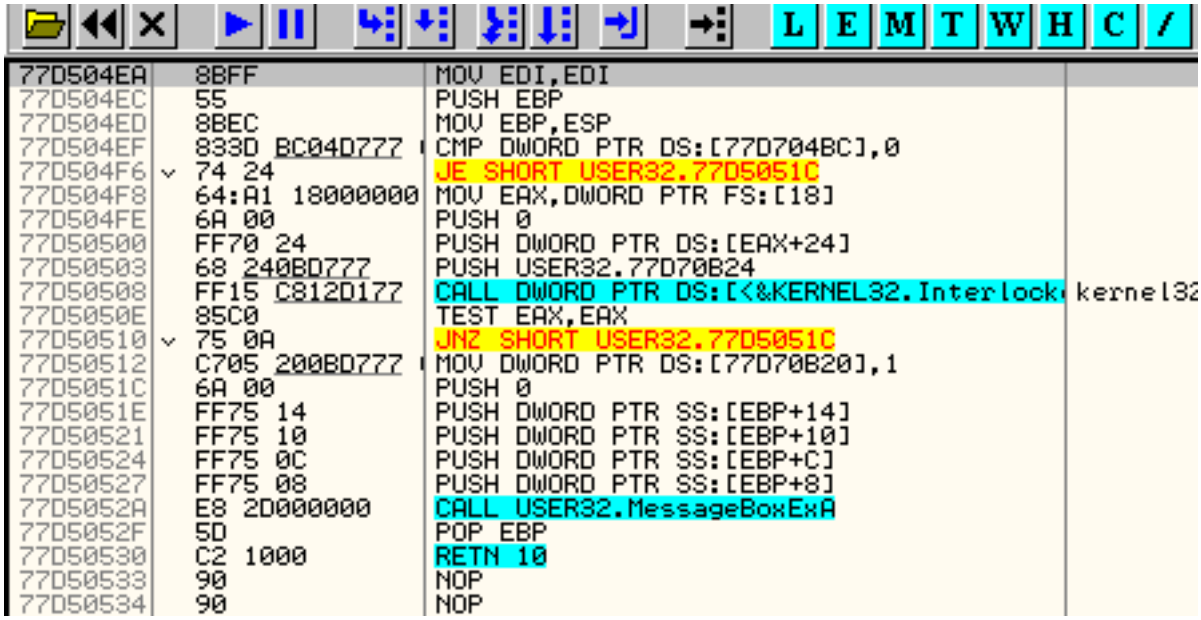
Теперь можно снова поставить BPM на kernel32.dll, чтобы поймать следующее обращение к этой библиотеке и т.д.

Например, если нужно поймать вызов функции MessageBoxA, но BP MessageBoxA не срабатывает, так как программа считывает первый байт этой функции и проверяет его на равенство CC, можно попробовать BPM. Давайте рассмотрим этот пример более подробно.

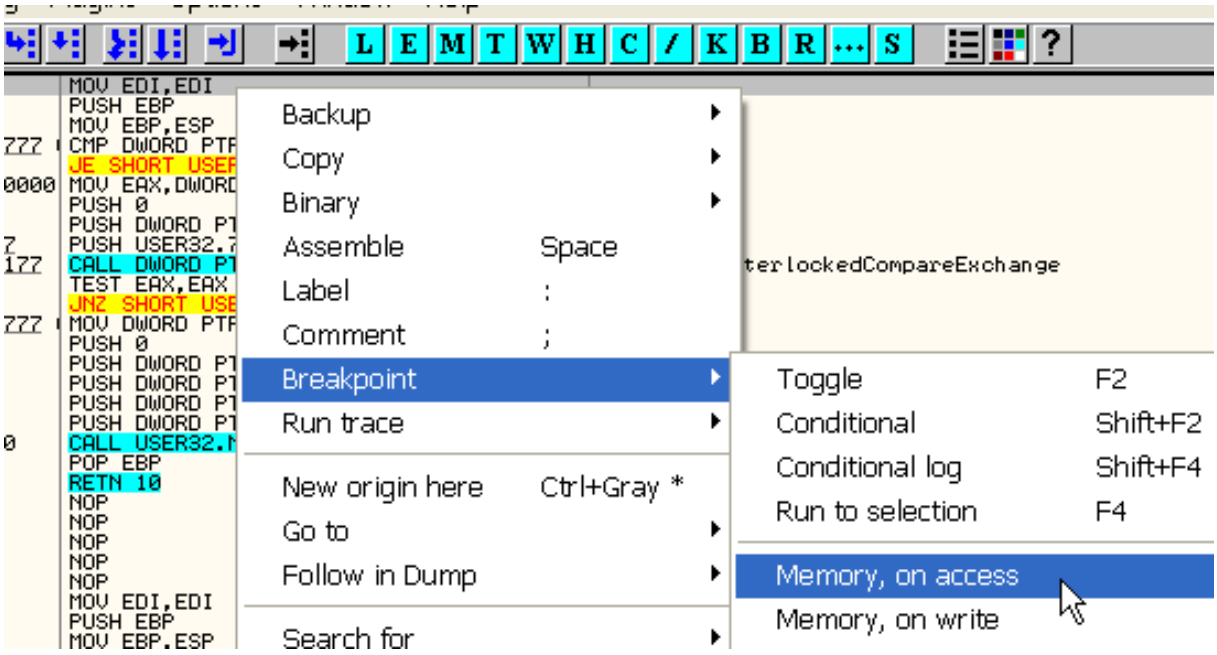


Адрес функции MessageBoxA на моей машине соответствует 77D504EA. Перемещаемся на соответствующий адрес через Go to - Exression. Можно указать сам адрес или точное имя функции:

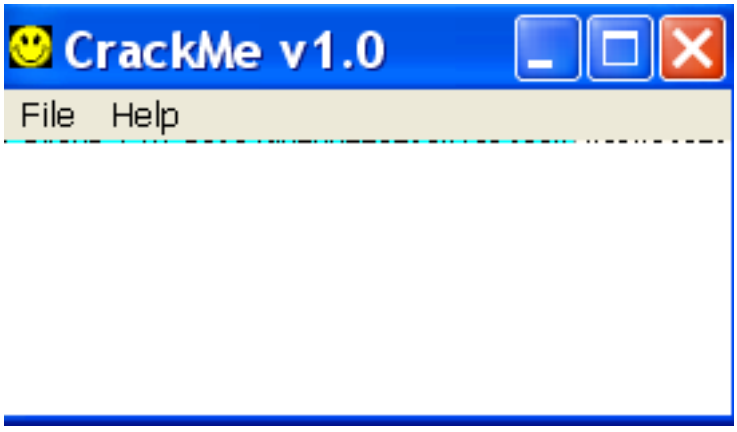




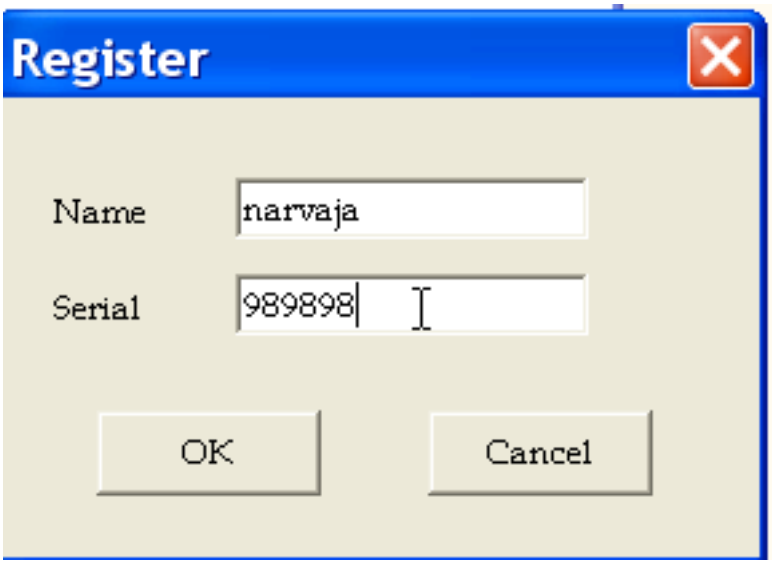
Подсвечиваем первую инструкцию:



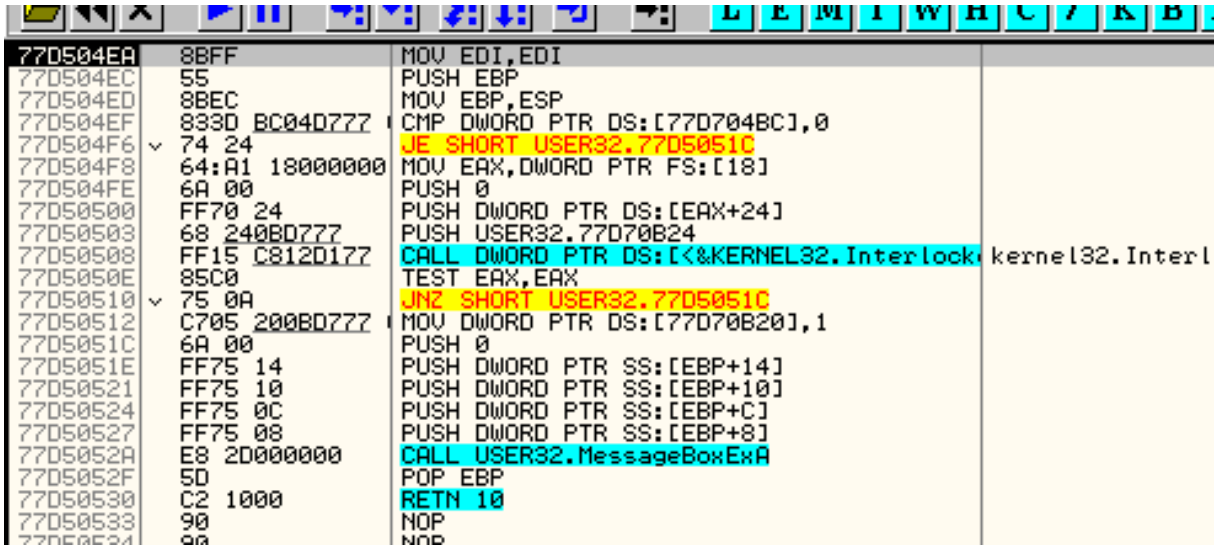
В данном случае тоже предоставляется на выбор: on access или on write. Выбираем первое и отдаём команду Run.



Выбираем Help - Register, вводим произвольные данные:



Нажимаем OK.



Происходит остановка на обращении к данной API-функции точно так же, как при использовании обычного BP. Если обычные точки останова не срабатывают, возможно, данный способ поможет обойти защиту. Этому способу тоже можно противодействовать проверкой свойств доступа к страницам памяти, но подобные антиотладочные приёмы встречаются реже.

В следующий раз мы рассмотрим аппаратные точки останова (Hardware breakpoints) и Message breakpoints. Ещё есть условные точки останова, но о них мы поговорим позже.

Введение в крэкинг с нуля, используя OllyDbg - Глава 11

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2007-10-05

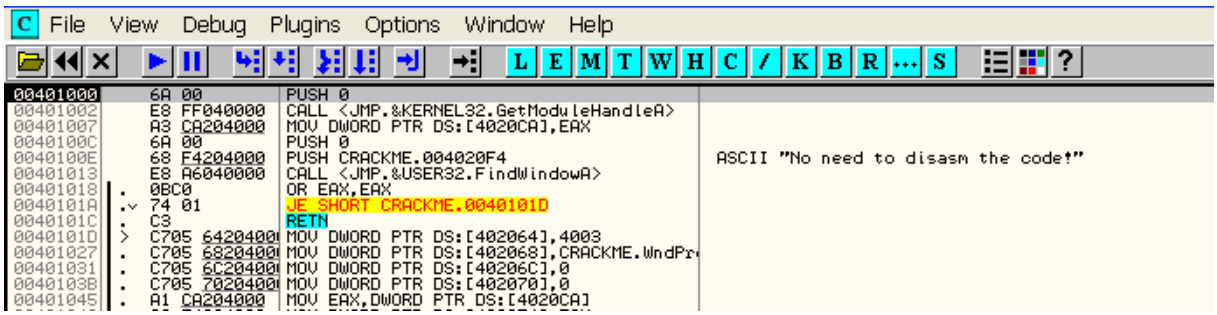
Хорошо, нам осталось рассмотреть процессорные точки останова (hardware breakpoints), условные точки останова и message breakpoints, и после этого мы закончим с этой темой.

ПРОЦЕССОРНЫЕ ТОЧКИ ОСТАНОВА

Процессорные точки останова (hardware breakpoints, HBP) - это возможность, предоставляемая процессором. Как это устроено на самом деле, я не знаю, но тем не менее мы можем устанавливать их, чтобы программа, когда их встречает, останавливалась и передавала управление.

Мы можем установить до 4 четырёх HBP, на пятый Ollydbg попросит нас указать, какой из четырёх уже установленных убрать.

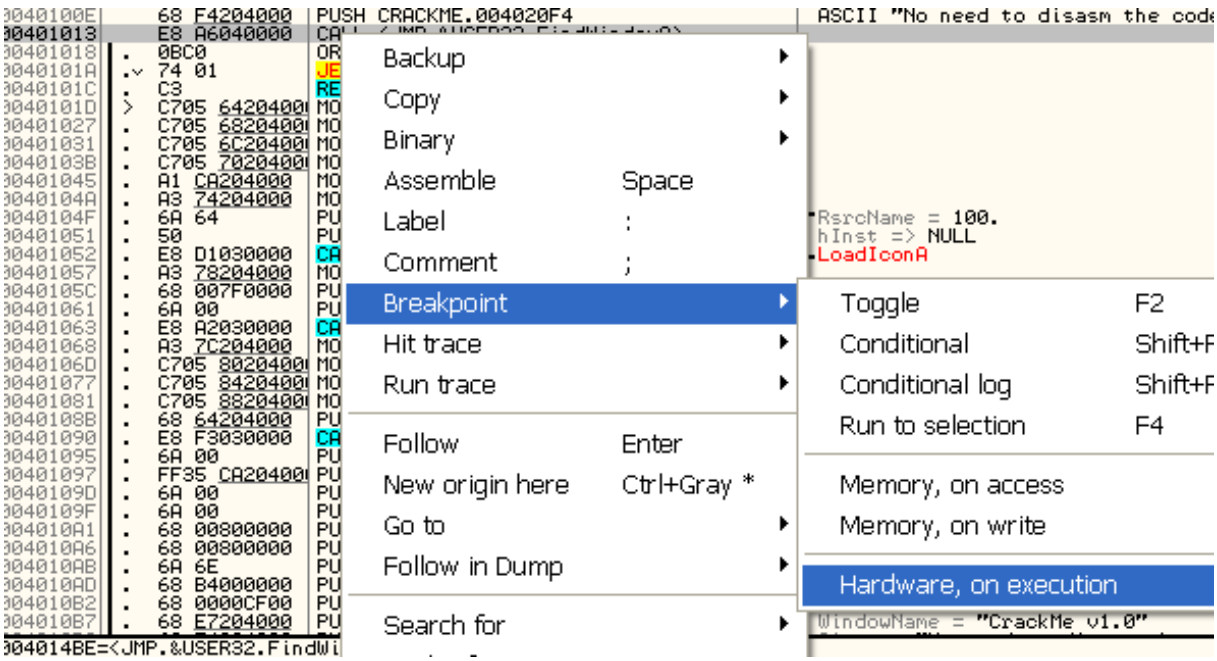
Как обычно, для практики будем использовать крэкми CrueHead'a.



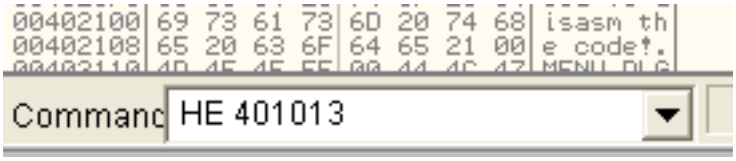
Есть три возможности: HBP ON EXECUTION, ON WRITE и ON ACCESS.

Установка HBP ON EXECUTION выполняет ту же роль, что и BPX на адрес, но HBP не изменяет код и его сложнее обнаружить, хотя есть программы, которые используют кое-какие приёмы, чтобы сбросить HBP. Эти приёмы и способы противостоять им, мы рассмотрим в следующих главах.

Если хотим установить HBP ON EXECUTION на 401013, отмечаем строку, кликаем на правую кнопку мыши и выбираем BREAKPOINT-HARDWARE ON EXECUTION.

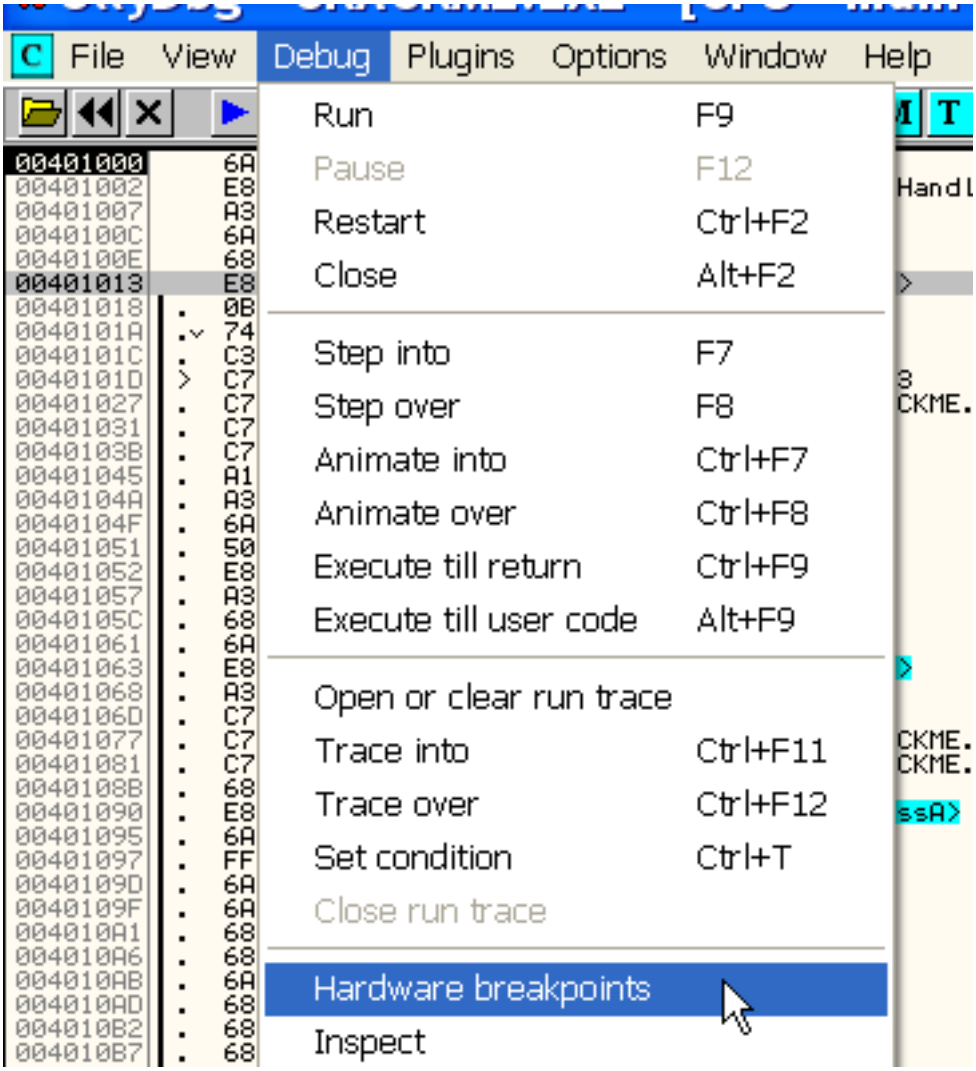


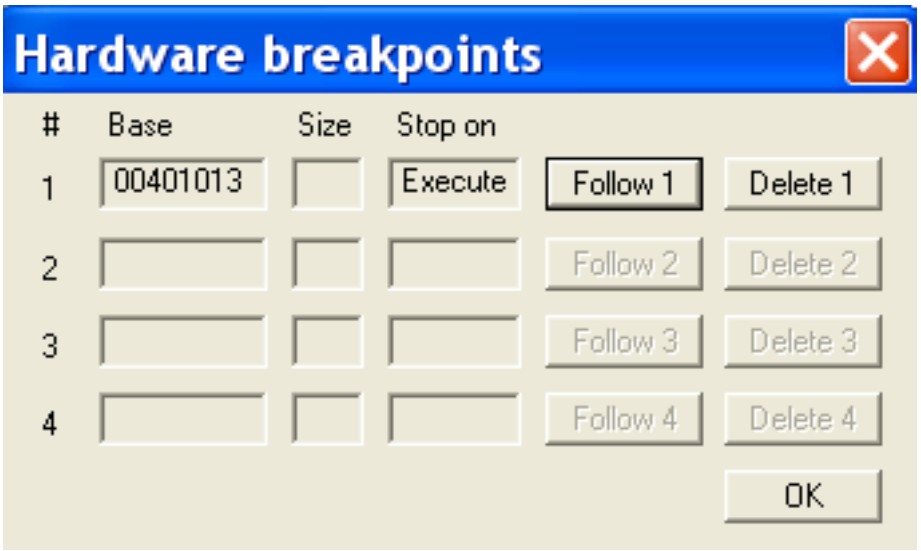
Также можем написать в коммандной строке:



После этого HBP установлен.

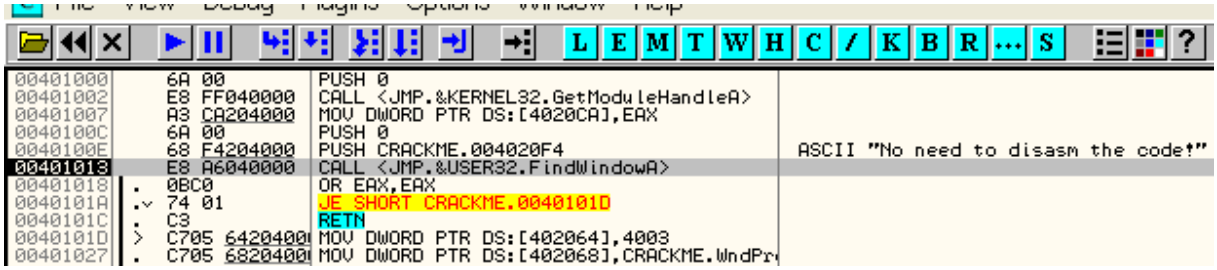
У Ollydbg есть специальное окно, с помощью которого мы можем просмотреть и управлять HBP'ами. Чтобы открыть его, идём в DEBUG-HARDWARE BREAKPOINTS.





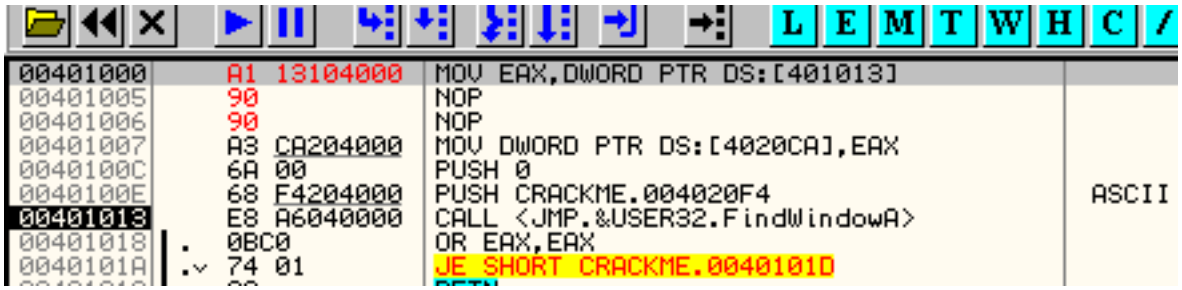
Здесь мы видим окошко с HBP, и если нажмём FOLLOW, мы окажем в листинге на той строке, где он был установлен. С помощью DELETE мы можем его удалить.

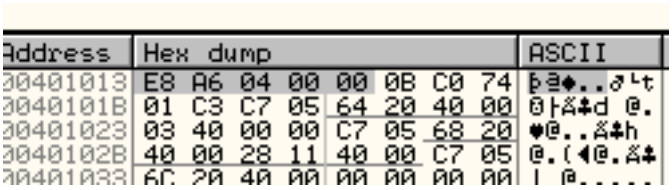
Теперь нажмём F9 для того, чтобы начать выполнение программы.



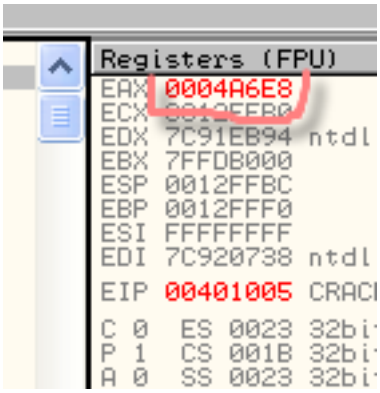
И остановимся на 401013.

Как видим, он ведёт себя как обычный BPX, если же сделаем то же, что делали и с BPX - напишем MOV EAX, DWORD PTR DS:[401013] и запустим, то увидим, что код не изменился.



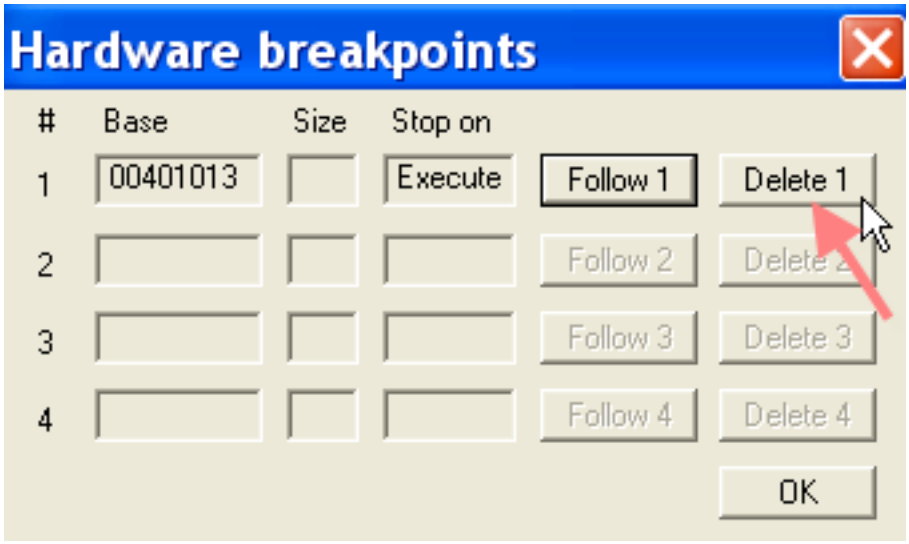


Изменим EIP на 401000 с помощью NEW ORIGIN HERE и нажмём F7.

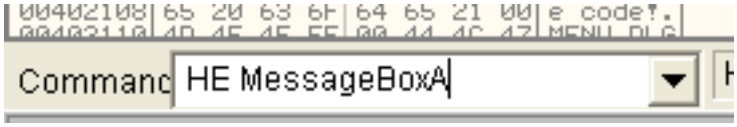


Видим, что в EAX содержатся E8 A6 04 00 (в обратной форме, поэтому мы видим 00004A6E8), то есть никаких изменений в коде не было.

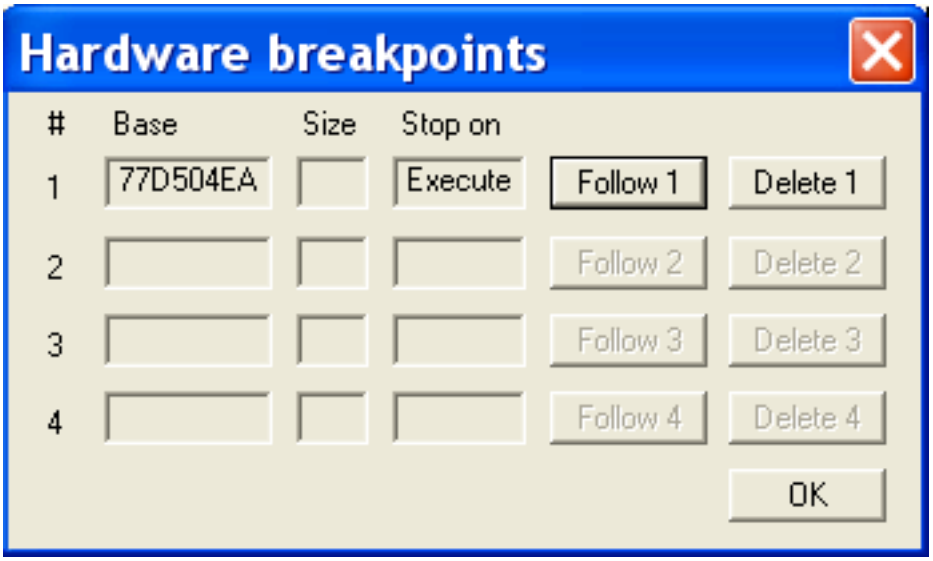
Если мы перезапустим Ollydbg, то увидим, что HBP остался установленным.



Убираем его и устанавливаем HBP на MessageBoxA таким же образом, как и BPX.



Теперь смотрим список HBP.

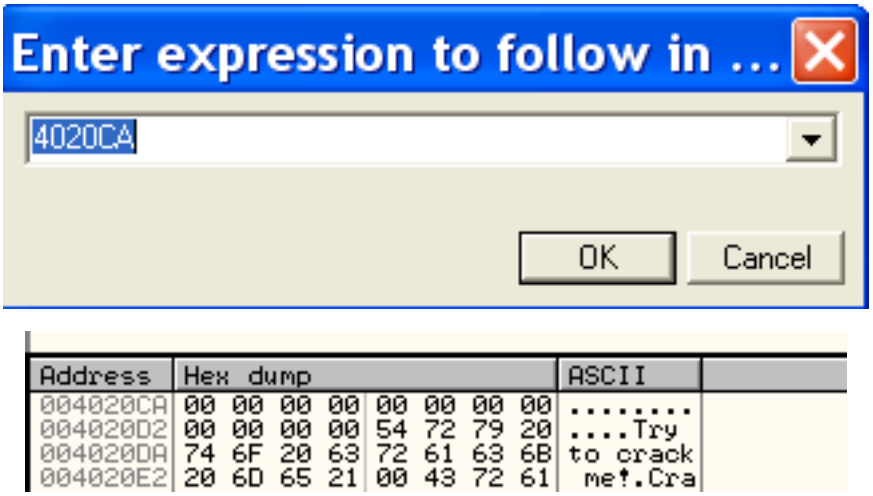


Не будем повторять снова, но знаем, что если запустим программу и введём юзера и пароль, и мы согласимся, то выполнение остановится на API MessageBoxA, т.е. примерно так же, как если бы мы использовали обычный BPX.

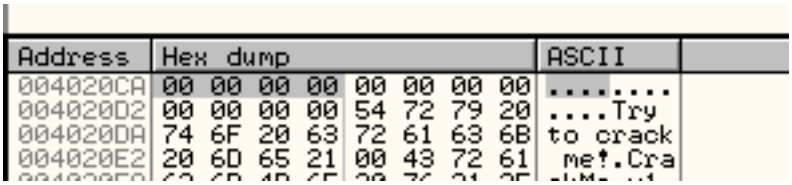
HBP ON ACCESS и ON WRITE могут охватывать только 1, 2 или 4 байта, если же мы выберем в DUMP зону большего размера, то будут иметь значение только первые четыре байта.

Сбросим все HBP, установленные нами ранее, и попытаемся установить HBP ON ACCESS на адрес 4020CA.

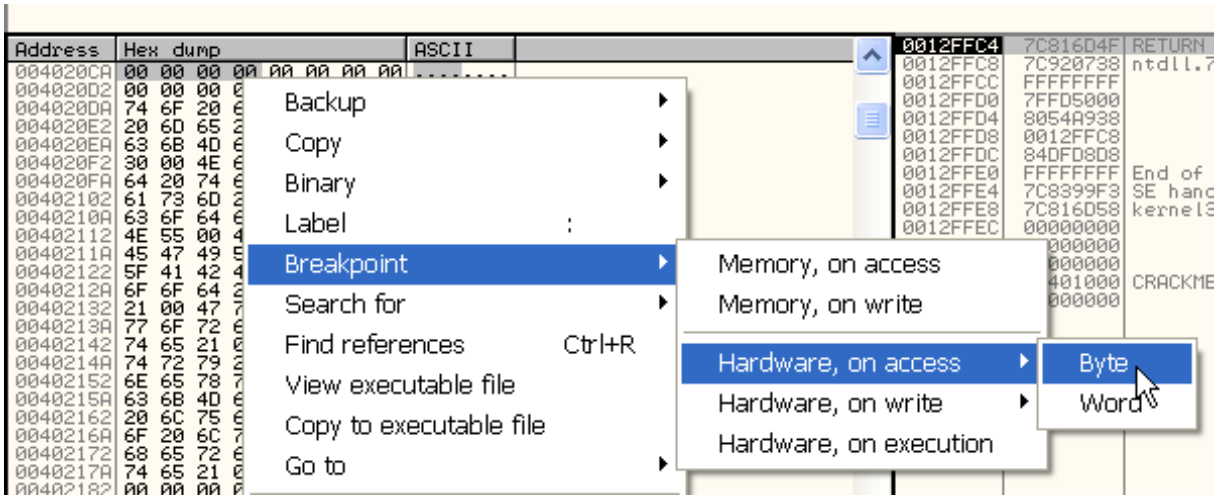
Посмотрим в DUMP'e, что находится по этому адресу.



А теперь отметим первые 4 байта.

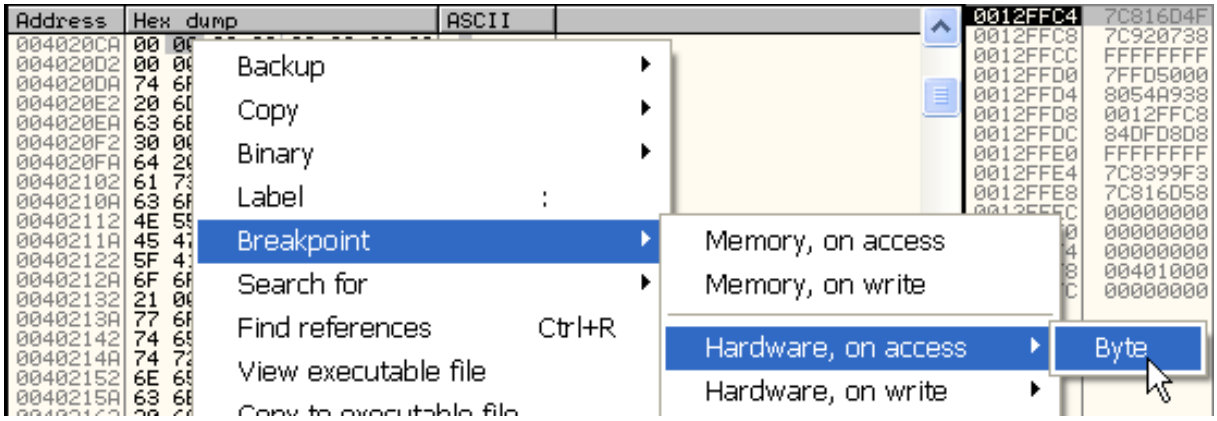


Нажимаем правую кнопку мыши.

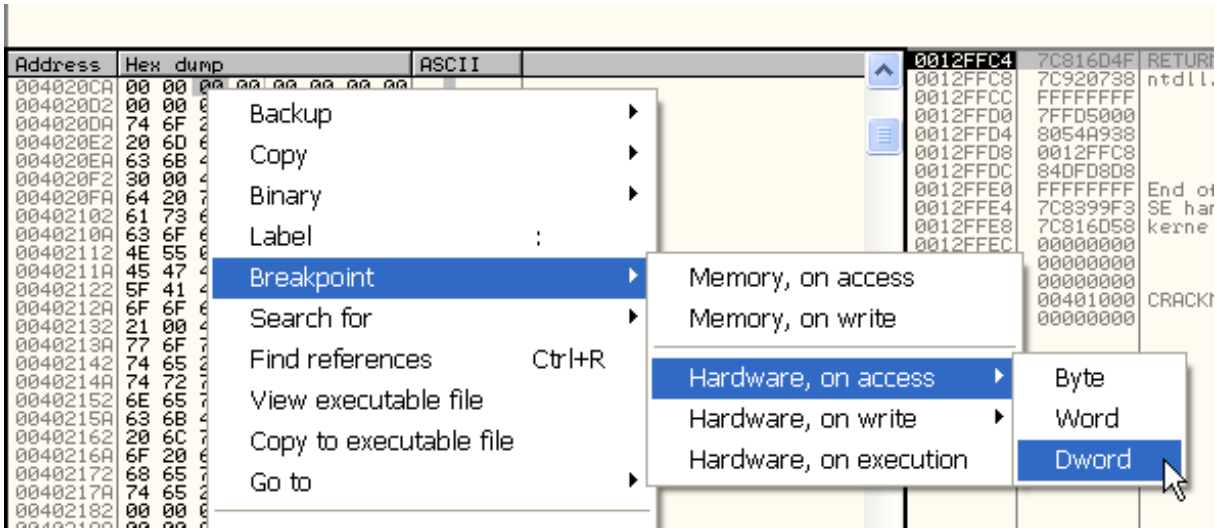


Видим, что для отмеченной нами зоны было предложен HBP на байт или два байта (WORD), выбор четырёх байтов предлагается только для адресов, которые кратны 4. В данном случае это условие не соблюдается.

Проделаем ту же операцию для следующего байта и выберем HARDWARE ON ACCESS, и теперь есть только возможность выбора BYTE.

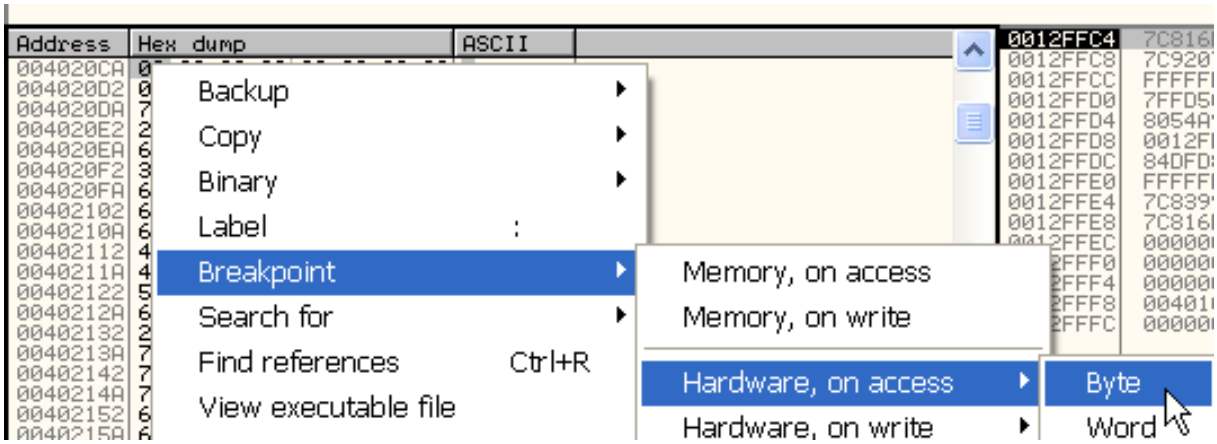


А на следующем появилась, наконец-то, опция DWORD, то есть HBP на четыре байта.

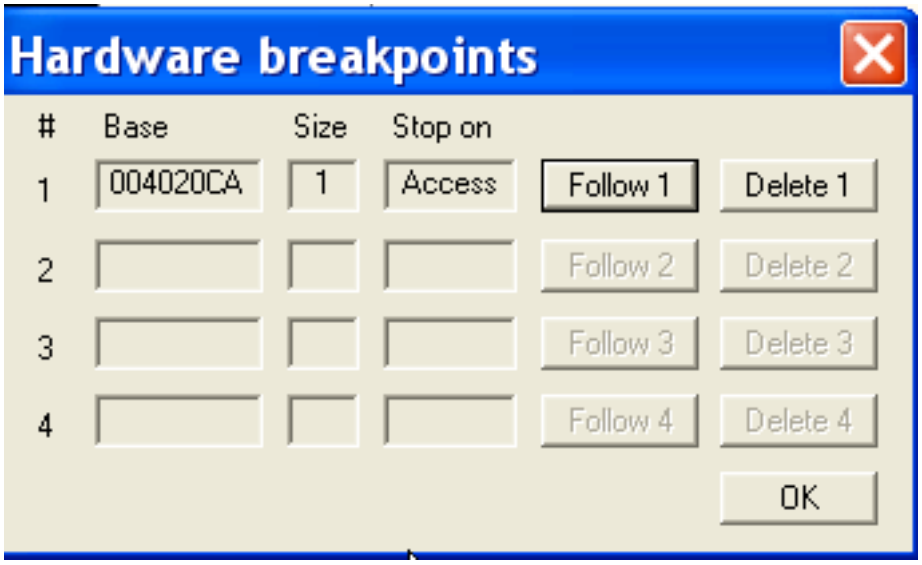


Идея в том, что если нам нужно контролировать сохранение или чтение по адресу 4020CA, то для этой цели достаточно поставить HBP на один байт.

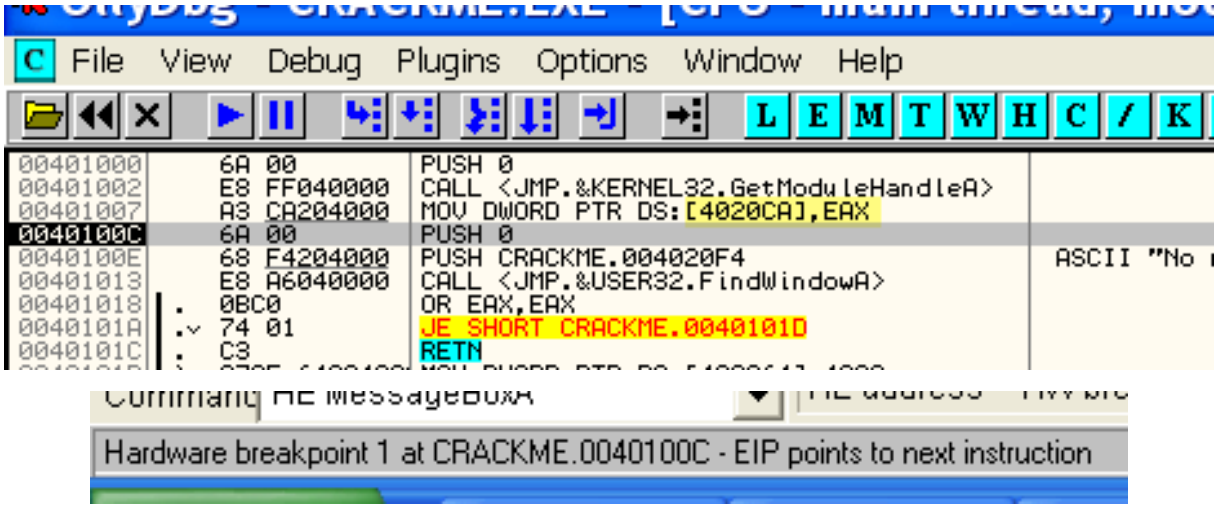
Возвратимся к 4020CA и поставим HBP ON ACCESS - BYTE.



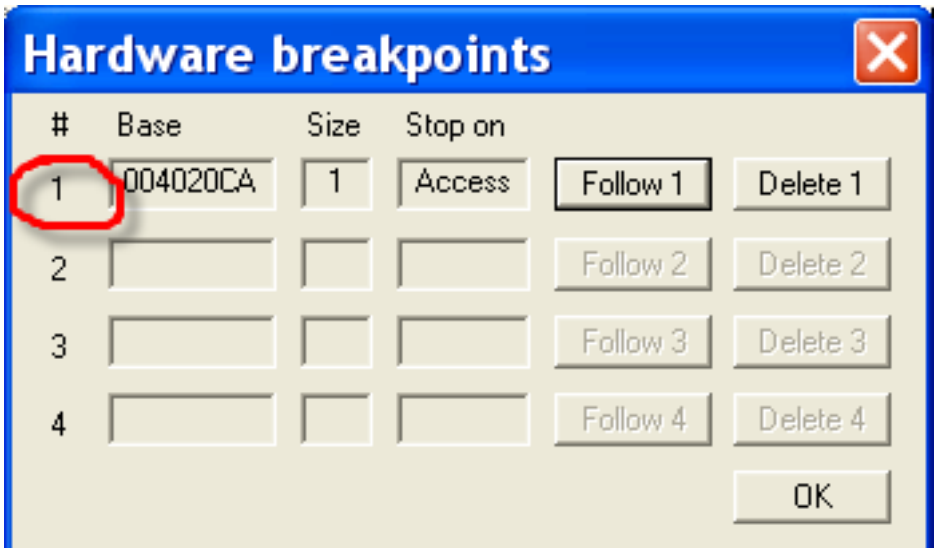
Видим, что поставленный нами HBP в списке имеет разме 1, т.е. байт (BYTE).



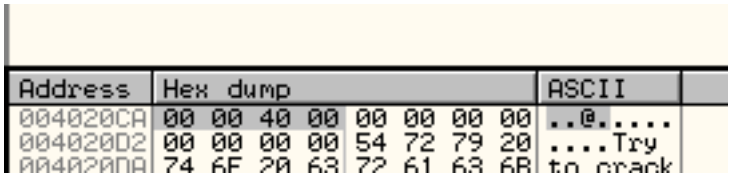
Делаем RUN.



Здесь OLLY говорит нам, что сработал HBP 1, т.е. тот, который идёт первым в списке HBP.



Видим, что выполнение программы прекращается на строке, следующей за той, где производилось сохранение или чтение. Это верно как для HBP ON ACCESS, так и для ON WRITE - инструкция, на которой срабатывает HBP, выполняется, а программа останавливается на следующей.



Как вы можете видеть, отличие от MEMORY BREAKPOINT заключается в том, что программа остановилась бы на 401007 - непосредственно на той инструкции, которая вызвала срабатывание точки останова.

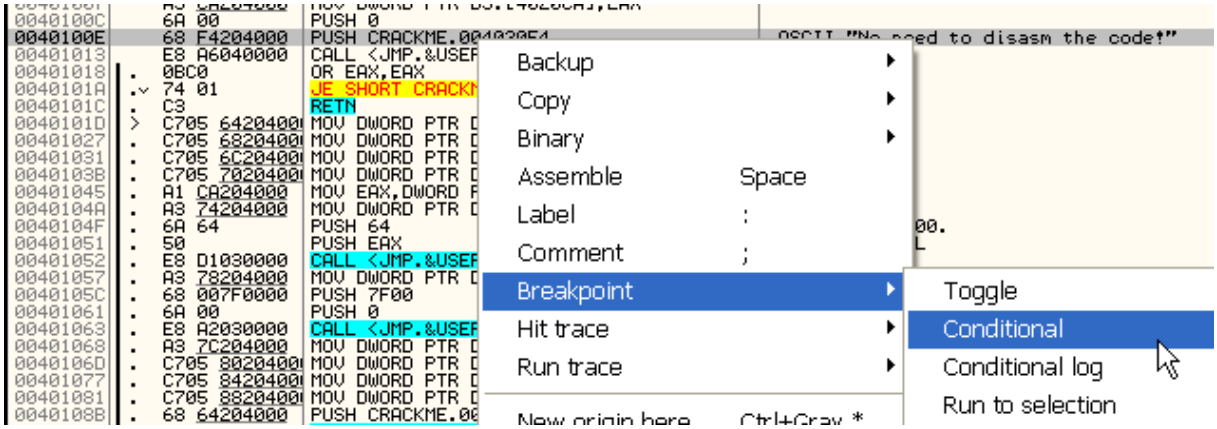
Как вы можете догадаться, HBP ON WRITE срабатывает, когда происходить сохранение, а не чтение, и остановка программы происходит на инструкции, следующей за той, которая вызвала срабатывание HBP.

УСЛОВНЫЕ ТОЧКИ ОСТАНОВА

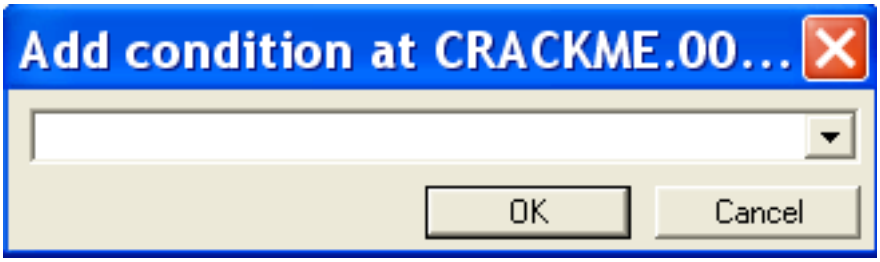
В действительности это вариант обычных BPX, отличающийся тем, что при активации такого BPX OllyDbg проверяет, верно ли некое условие, и если да, то останавливает выполнение программы, а если нет, то продолжает выполнение программы, как будто BP не существует.

Рассмотрим пример.

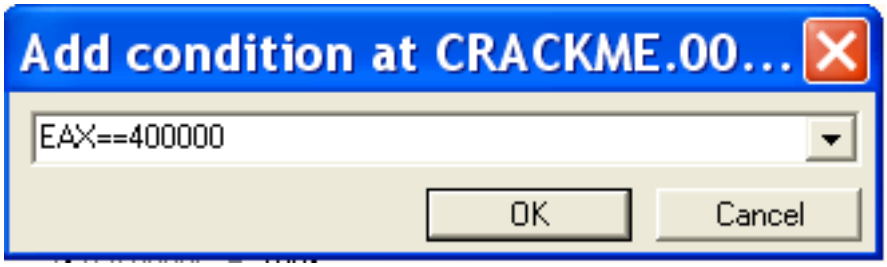
Сбросим всё, что мы делали ранее, и установим BREAKPOINT CONDICIONAL на 40100E, для чего выделим строку, нажмём правую кнопку мыши и выберем BREAKPOINT CONDICIONAL.



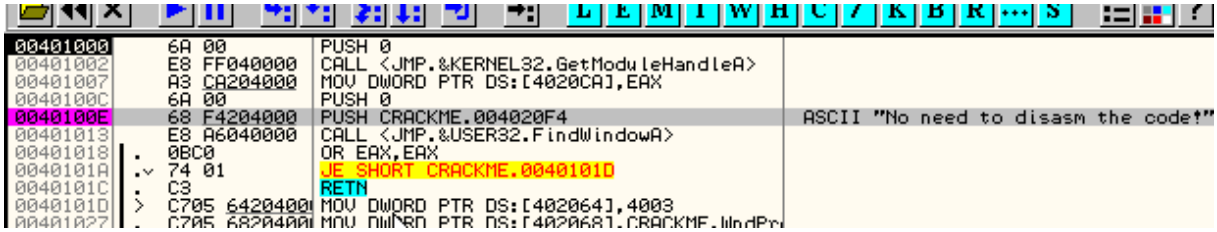
Открывается окошко, в котором нужно задать условие.



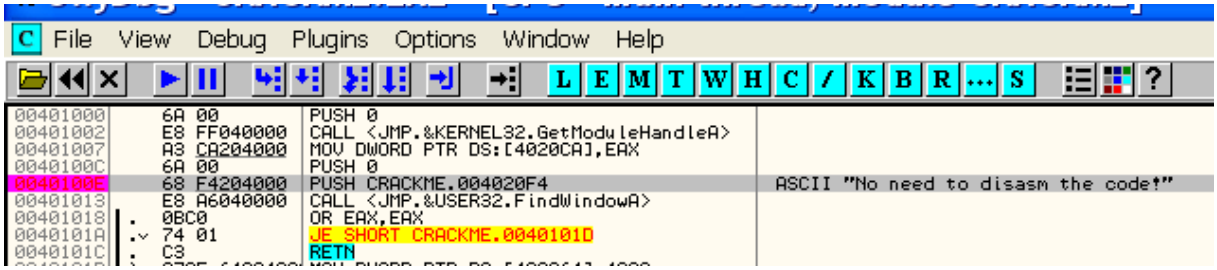
Например, если хотим задать, чтобы останов происходил, когда EAX равен 400000, то условие должно быть таким: "EAX==400000".



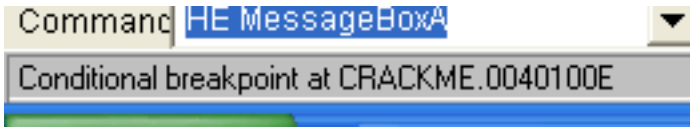
В хелпах, прилагаемых к OllyDbg, написано, какие символы мы можем использовать и каким условиям они соответствуют.



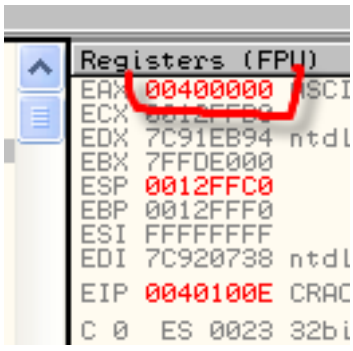
Видим, что в случае с BPX CONDICIONAL используется розовый цвет. Нажимаем F9:



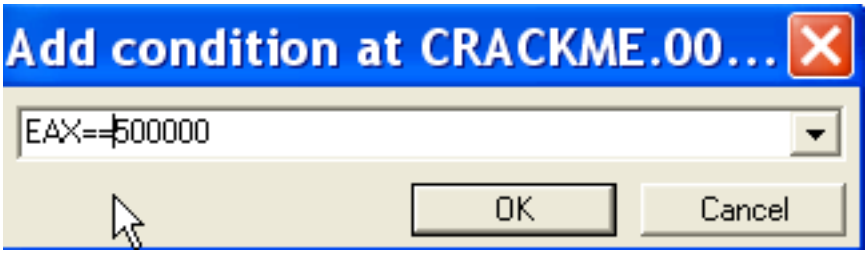
Видим, что произошёл останов, и OllyDbg нам сообщает:



Раз EAX равен 400000, значит сработало заданное условие.



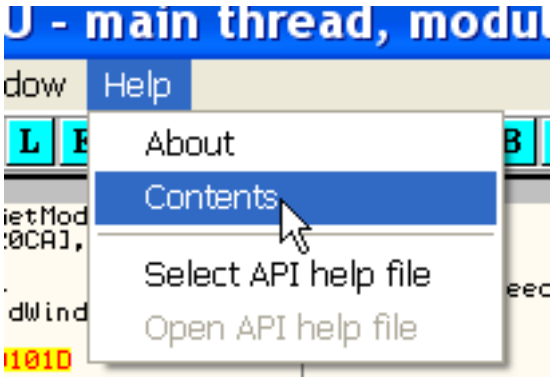
Начнём выполнение заново, сотрём BPX CONDICIONAL и поставим другой, где, например, EAX==500000.

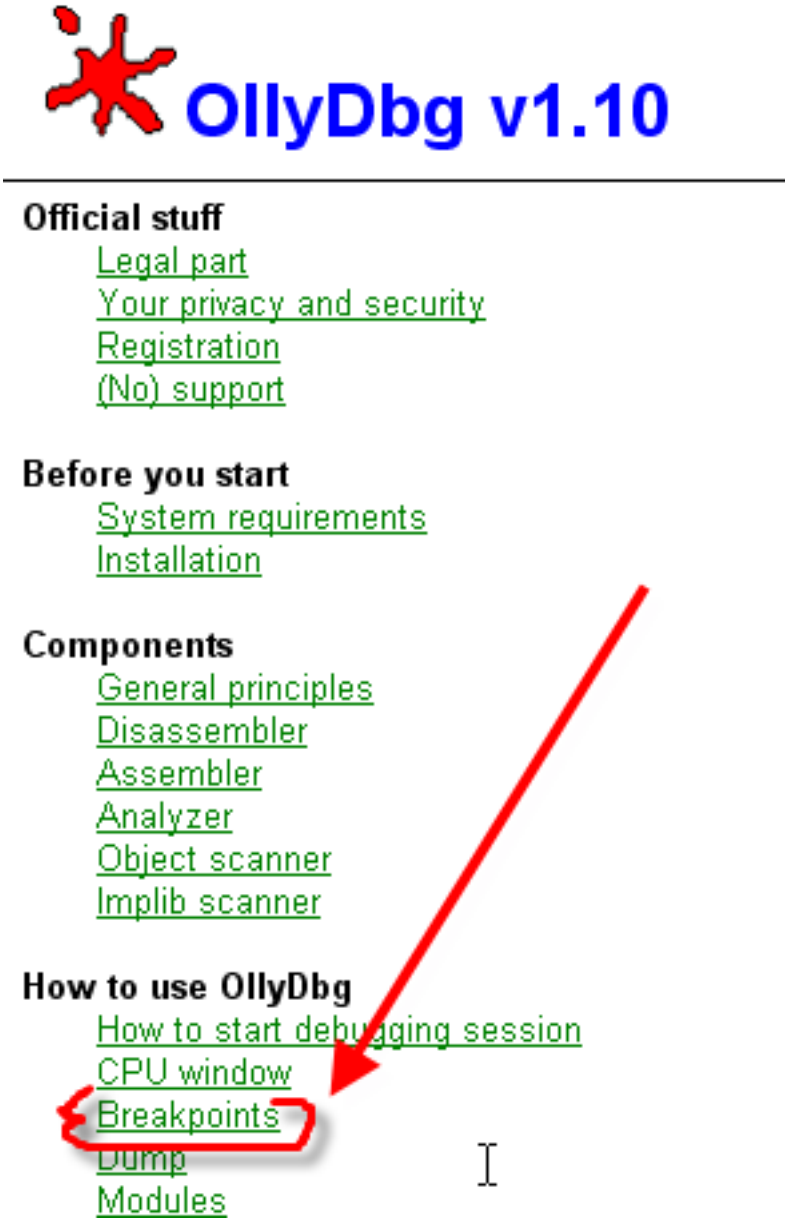


И сделаем RUN.

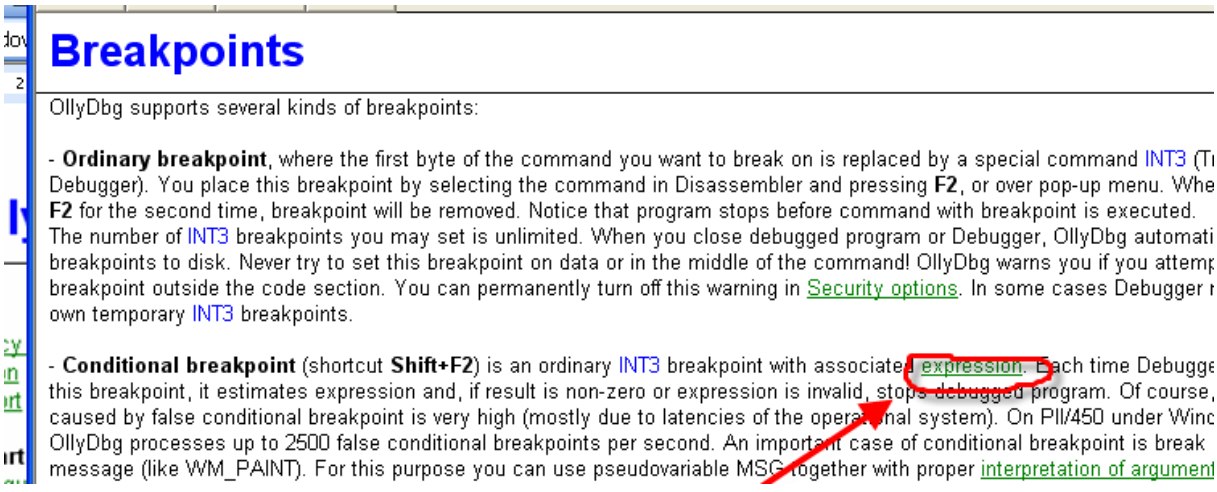
Видим, что крэкми выполнился и останова не произошло, т.к. EAX всегда был равен 400000 и условие не выполнялось.

Если откроем HELP-CONTENTS.





Вот BREAKPOINTS и место, где объясняются выражения:



Также в хелпе есть примеры выражений, которые можно использовать для условий.

Evaluation of expressions

OllyDbg supports very complex expressions. Formal grammar of expressions is described at the end of this topic, but hone not interested in it, are you? So I'll begin with examples:

10 - constant 0x10 (unsigned). All integer constants are assumed hexadecimal unless followed by a decimal point;

10. - decimal constant 10 (signed);

'A' - character constant 0x41;

EAX - contents of register EAX, interpreted as unsigned number;

EAX. - contents of register EAX, interpreted as signed number;

[123456] - contents of unsigned doubleword at address 123456. By default, OllyDbg assumes doubleword operands;

DWORD PTR [123456] - same as above. Keyword PTR is optional;

[SIGNED BYTE 123456] - contents of signed byte at address 123456. OllyDbg allows both MASM- and IDEAL-like mei expressions;

STRING [123456] - ASCII zero-terminated string that begins at address 123456. Square brackets are necessary becau the contents of memory;

[[123456]] - doubleword at address that is stored in doubleword at address 123456;

2+3*4 - evaluates to 14. OllyDbg assigns standard C priorities to arithmetical operations;

(2+3)*4 - evaluates to 20. Use parentheses to change the order of operations;

EAX.<0. - 0 if EAX is in range 0..0x7FFFFFFF and 1 otherwise. Notice that constant 0 is also signed. When comparin unsigned, OllyDbg always converts signed operand to unsigned.

EAX<0 - always 0 (false), because unsigned numbers are always positive.

MSG==111 - true if message is WM_COMMAND. 0x0111 is the code for WM_COMMAND. Use of MSG makes sense conditional or conditional logging breakpoint set on call to or entry of known function that processes messages.

[STRING 123456]=="Brown fox" - true if memory starting from address 0x00123456 contains ASCII string "Brown fox FOX JUMPS", "brown fox???" or similar. The comparison is case-insensitive and limited in length to the length of text c

EAX=="Brown fox" - same as above, EAX is treated as a pointer.

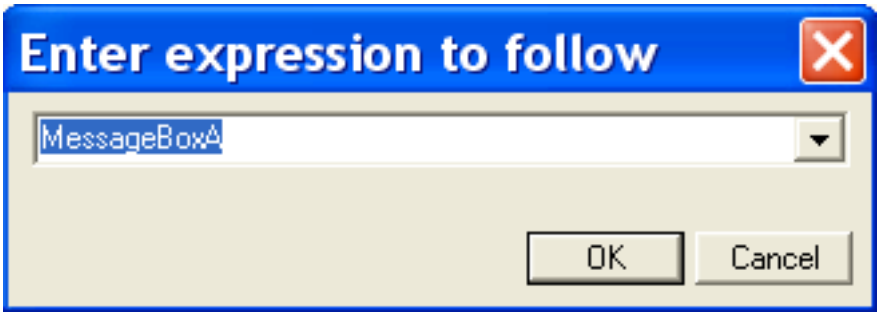
UNICODE [EAX]=="Brown fox" - OllyDbg treats EAX as a pointer to UNICODE string, converts it to ASCII and compa

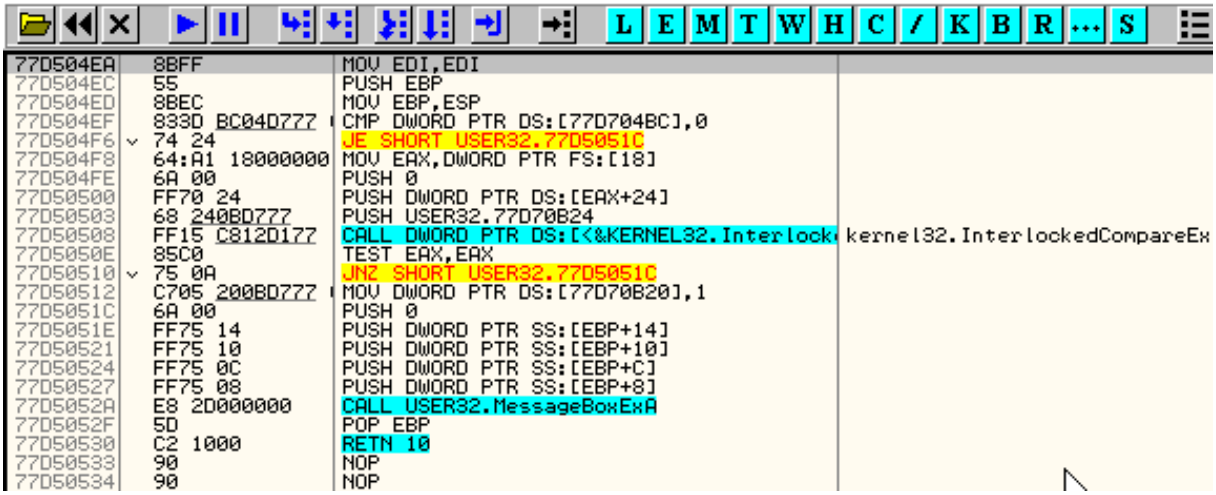
УСЛОВНЫЕ ТОЧКИ ОСТАНОВА С ЛОГИРОВАНИЕМ

Это условный BPX, такой же как и предыдущий, только в этом случае мы можем указать, чтобы все точные значения сохранялись при прохождении точки останова. Там есть много опций, и мы продемонстрируем применение этого BPX на примере одного API, который встречается в разных местах, и нам хочется, чтобы сохранялись данные, передающиеся ему.

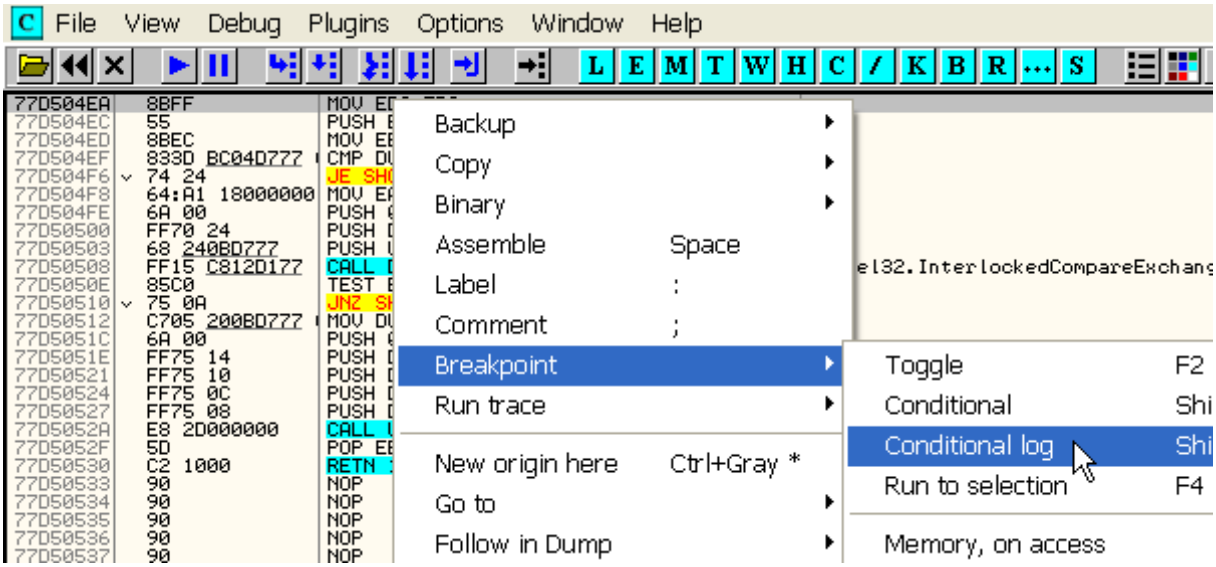
Установим BPX CONDICIONAL LOG на адрес, где находится API, перезапустив предварительно OllyDbg.

Идем к нужному адресу, где находится api, и уже знаем, как это сделать: CLICK DERECHO-GOTO EXPRESSION.

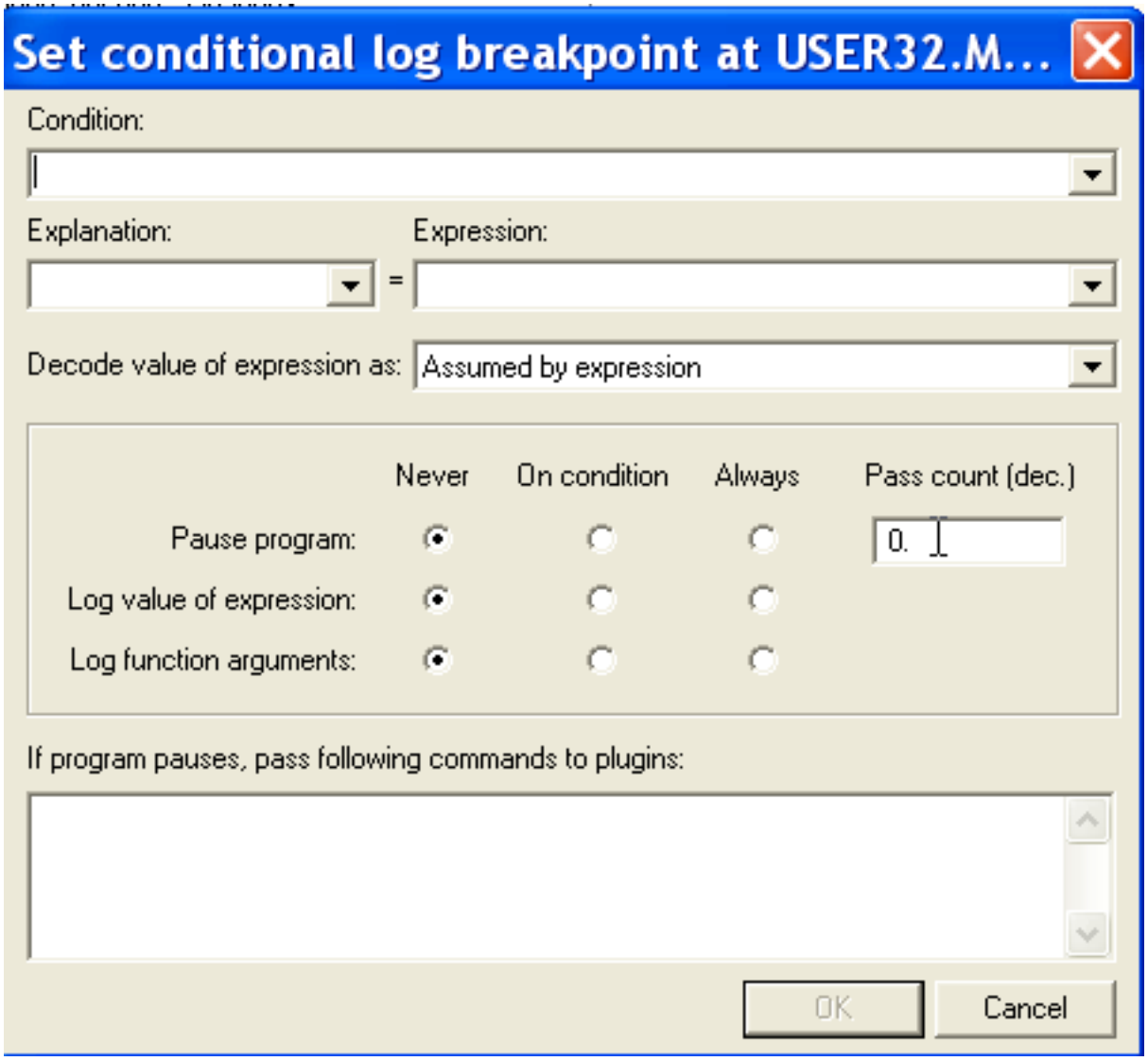




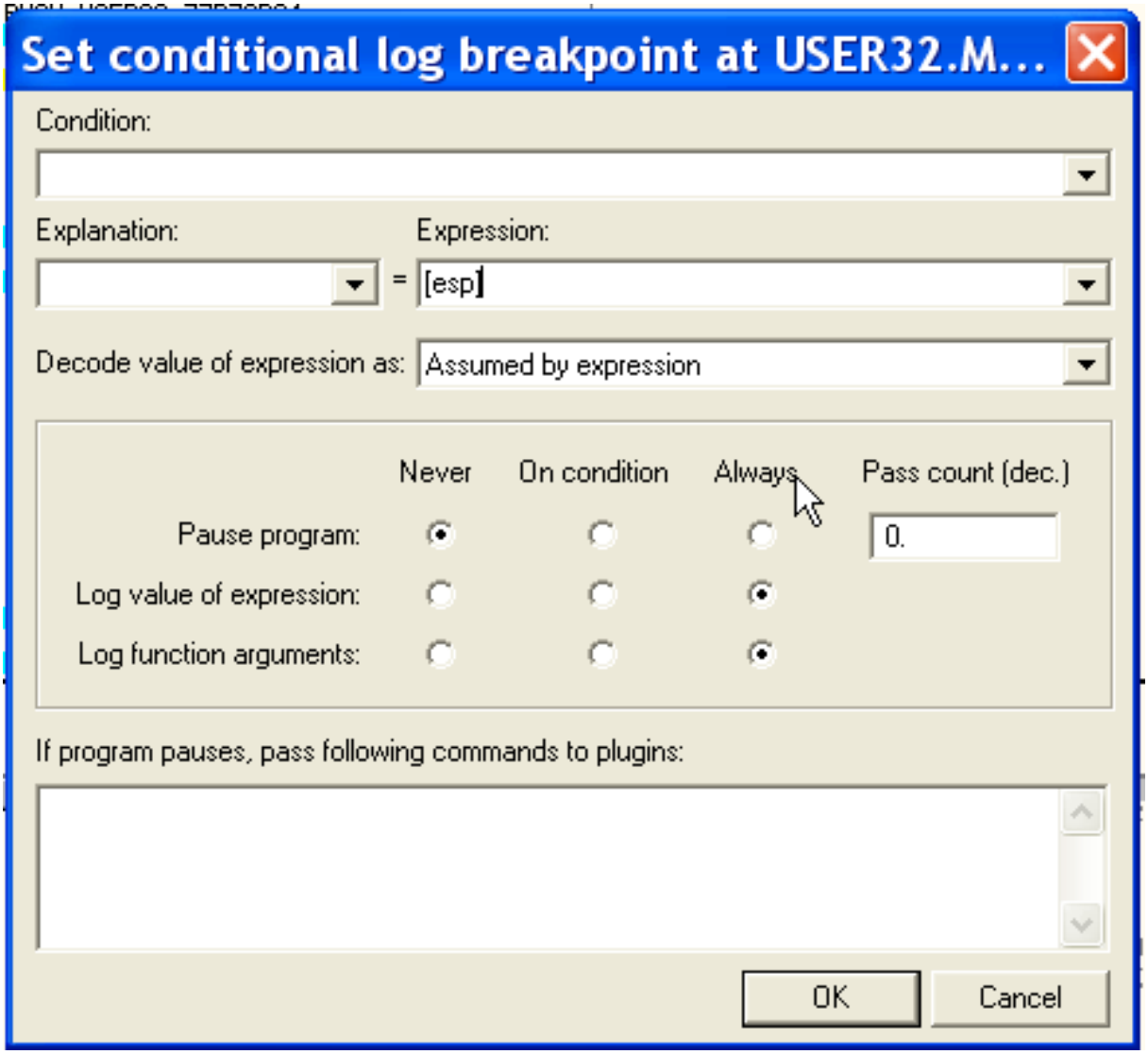
Находясь здесь, нажимаем CLICK DERECHO- BREAKPOINT- CONDICIONAL LOG.



Видим окошко с множеством опций.

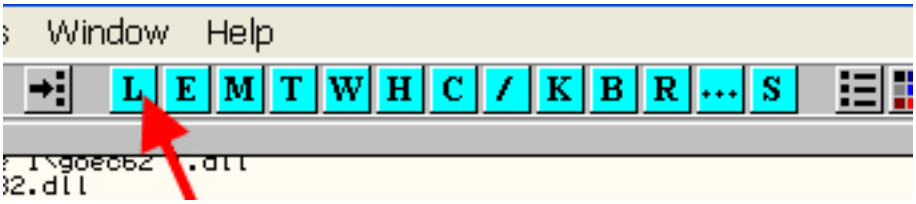


В данном случае нам нужно только, чтобы сохранялись требуемые нам данные.

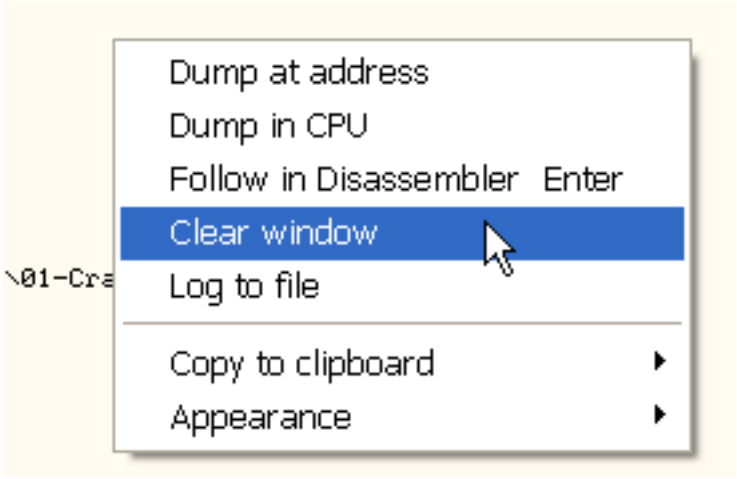


Так как мы не хотим, чтобы происходил останов, то не задаём никакого условия, а в опции PAUSE PROGRAM выбираем, чтобы этого не происходило, то есть отмечаем NEVER. Потом в LOG VALUE OF EXPRESSION можем сделать так, чтобы сохранялось значение [esp], которое мы написали в поле EXPRESSION, в котором, как мы знаем, будет адрес возврата в основное тело программы после выполнения api, то есть самое верхнее значение стека, поэтому во второй строке LOG VALUE OF EXPRESSION выбираем ALWAYS, то есть логировать всегда. В третьей строке находится опция, задающая хотим ли мы логировать аргументы/параметры функции/api, и поскольку их немного, можем поставить ALWAYS.

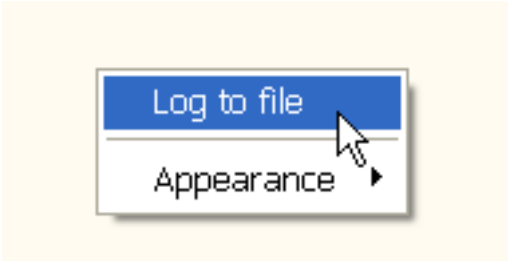
Идём в окно LOG или L.



И очищаем его, чтобы ясно видеть, что сохраняет программа. Кликаем CLICK DERECHO-CLEAR WINDOW.



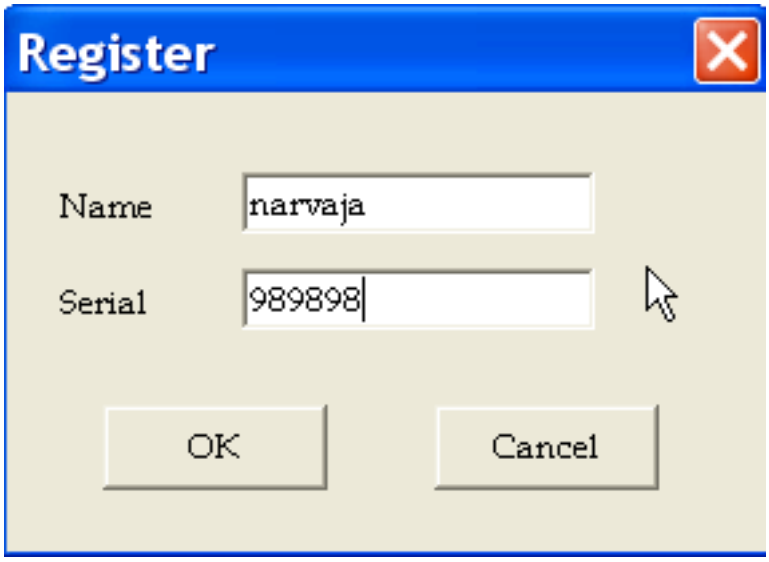
Видим, что есть опция, позволяющая сохранять лог в текстовый файл, если это необходимо.



Теперь запускаем выполнение программы с помощью F9.

Идём в окошко крэкми и пока в LOG'е ничего нет, так как заданное нами api не вызывалось.

Идём в REGISTER, где можем задать имя и пароль пользователя.



Когда нажимаем OK

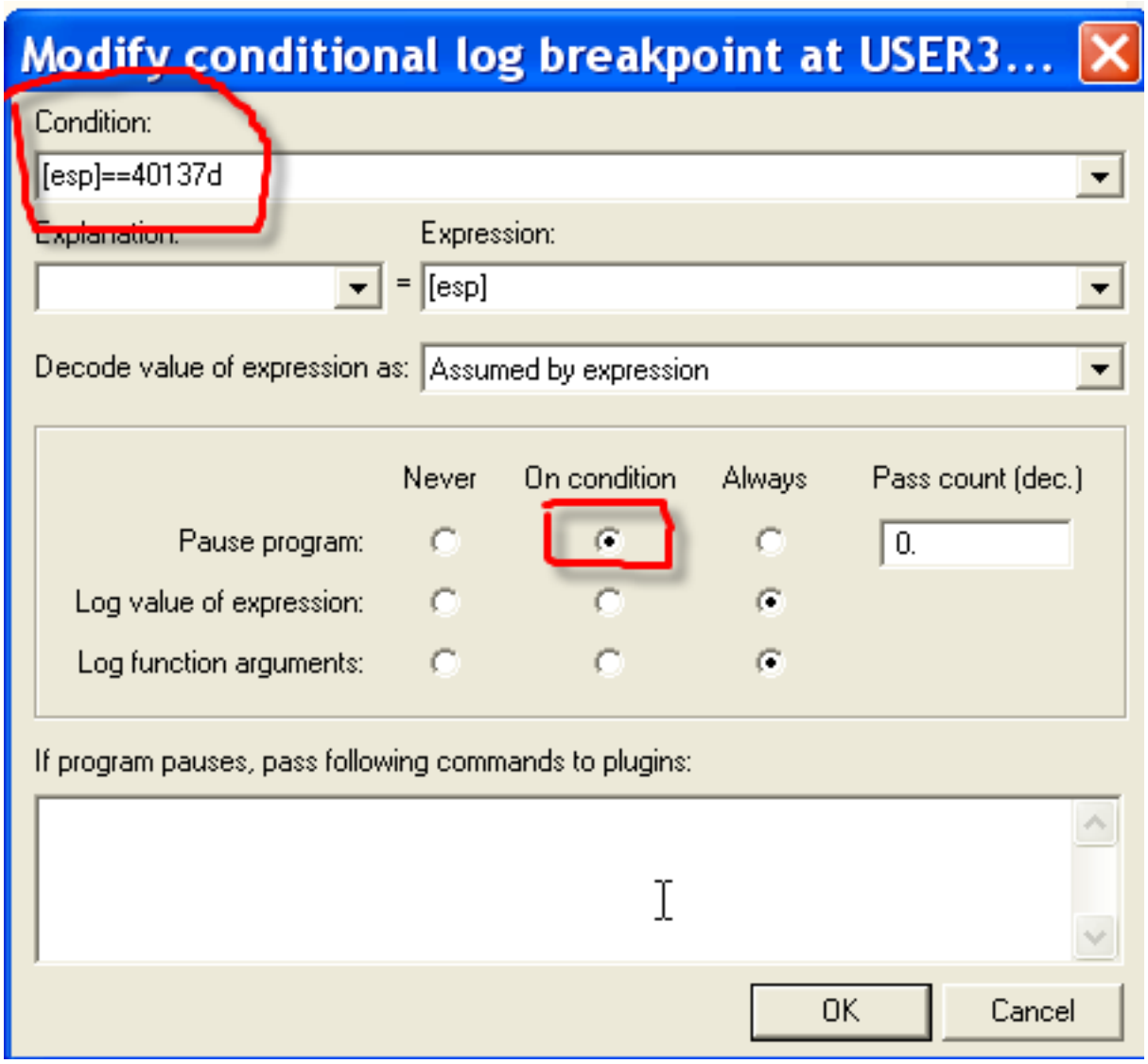
```
773A0000 Module C:\WINDOWS\WinSxS\x86_Microsoft.Windows.Common-Controls_6595b64144ccf1c
5B150000 Module C:\WINDOWS\system32\uxtheme.dll
062D0000 Module C:\WINDOWS\system32\SSSensor.dll
60300000 Module C:\Archivos de programa\Yahoo!\Messenger\idle.dll
77D504EA COND: 0040137D
77D504EA CALL to MessageBoxA from CRACKME.00401378
           hOwner = 00090884 ('CrackMe v1.0',class='No need to disasm the code!')
           Text = "No luck there, mate!"
           Title = "No luck!"
           Style = MB_OK|MB_ICONEXCLAMATION|MB_APPLMODAL
Command
```

Видим, что в окошке лога отобразилась информация о MessageBoxA, самое верхнее значение стека (которое равно 40137D и является адресом возврата из api), а затем параметры api, в данном случае это адрес, из которого оно было вызвано, текст, заголовок и т.п.

В данном случае нам отобразилась информация только об одном вызове, но если api вызывается, скажем, 100 раз, то можем сохранять лог в текстовый файл. Если же нам нужно, чтобы отображались только некоторые из этих вызовов, то можем задать определённое условие.

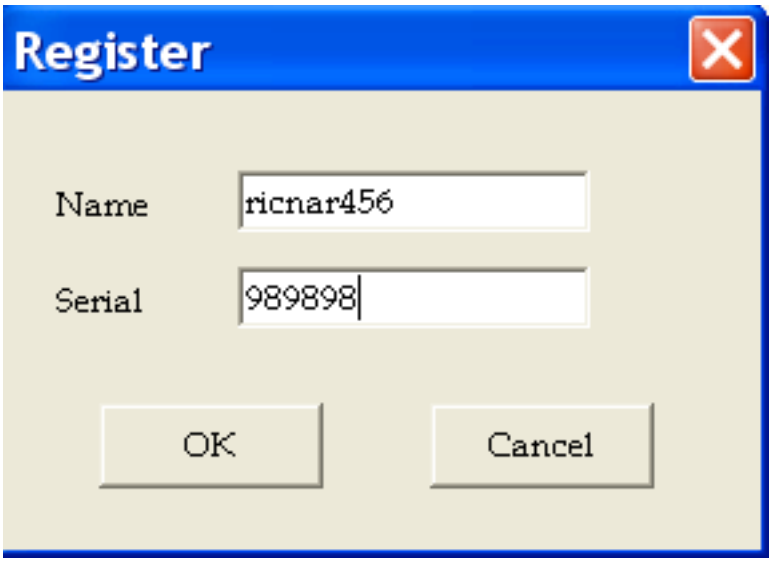
Если у нас есть список с информацией о 100 различных вызовах данного api, выполненных программой, то можно задать какие-нибудь данные в поле условия, например, адрес возврата. В этом случае из всех 100 случаев вызова отобразятся только те, которые возвращаются на 40137D, а остальные нет.

Для этого перегрузим крэкми, снова найдём api и установим на неё BPX CONDICIONAL LOG.

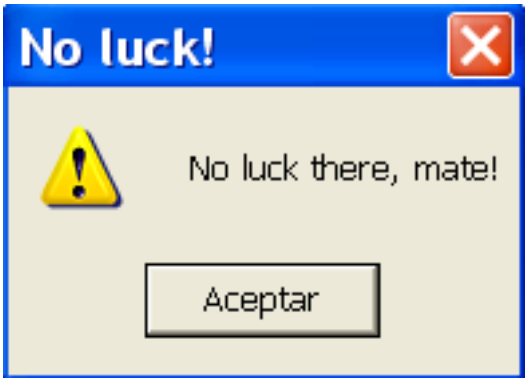


Изменим опции, так чтобы точка останова срабатывала только тогда, когда выполняется условие и зададим его, чтобы проверялось, равен ли [esp] 40137D.

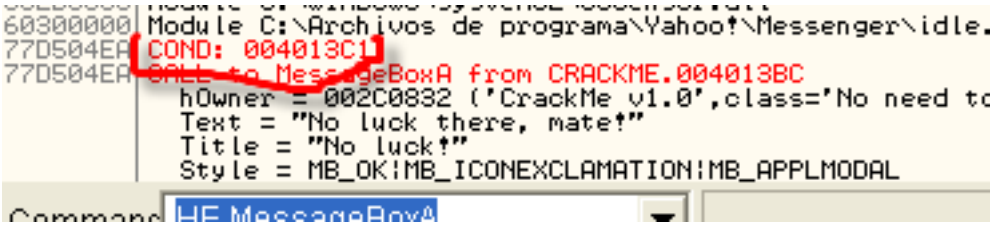
Запускаем программу, и когда появится окошко регистрации, введём имя пользователя ‘ricnar456’ и пароль ‘989898’.



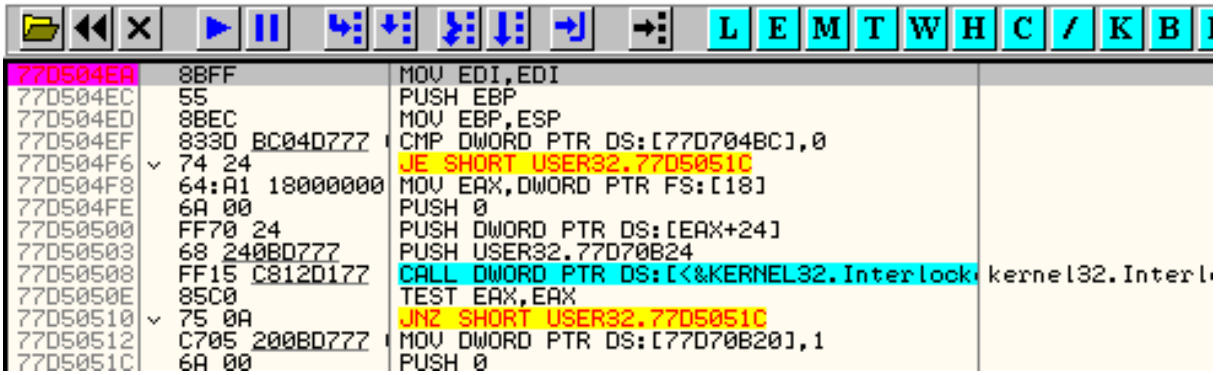
Помните, что когда вводится имя пользователя, в котором есть цифры, messageboxa вызывается 2 раза? Нажимаем OK.



Видим, что на первый MessageBoxA Олли никак не отреагировала. В логах показано, что адрес возврата был равен 4013C1, поэтому условие не выполнялось.

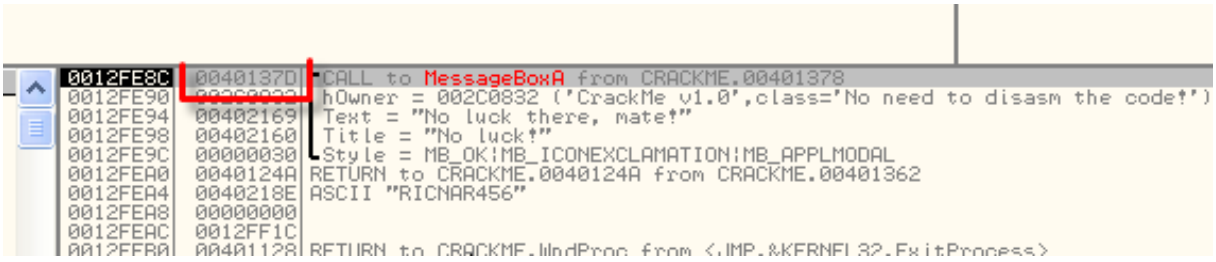


Когда нажимаем кнопку OK



Это второй раз, когда вызывается api. Здесь условие выполняется и срабатывает точка останова.

Видим, что верхнее элемент стека содержит значение 40137d, которое активирует BPX CONDICIONAL LOG.



В обоих случая OllyDbg сохранил информацию об обоих вызовах.

```
Module C:\WINDOWS\system32\SSSensor.dll
Module C:\Archivos de programa\Yahoo!\Messenger\idle.dll
COND: 004013C1
CALL to MessageBoxA from CRACKME.004013BC
  hOwner = 002C0832 ('CrackMe v1.0',class='No need to disasm the code!')
  Text = "No luck there, mate!"
  Title = "No luck!"
  Style = MB_OK|MB_ICONEXCLAMATION|MB_APPLMODAL
COND: 0040137D
CALL to MessageBoxA from CRACKME.00401378
  hOwner = 002C0832 ('CrackMe v1.0',class='No need to disasm the code!')
  Text = "No luck there, mate!"
  Title = "No luck!"
  Style = MB_OK|MB_ICONEXCLAMATION|MB_APPLMODAL
Conditional breakpoint at USER32.MessageBoxA
```

Во втором случае адресом возврата было 40137D, и мы видим, что в этот раз условная точка останова сработала.

```
Module C:\Archivos de programa\Yahoo!\Messenger\idle.dll
COND: 004013C1
CALL to MessageBoxA from CRACKME.004013BC
  hOwner = 002C0832 ('CrackMe v1.0',class='No need to dis
  Text = "No luck there, mate!"
  Title = "No luck!"
  Style = MB_OK|MB_ICONEXCLAMATION|MB_APPLMODAL
COND: 0040137D
CALL to MessageBoxA from CRACKME.00401378
  hOwner = 002C0832 ('CrackMe v1.0',class='No need to dis
  Text = "No luck there, mate!"
  Title = "No luck!"
  Style = MB_OK|MB_ICONEXCLAMATION|MB_APPLMODAL
Conditional breakpoint at USER32.MessageBoxA
Command HE MessageBoxA
```

Ок, думаю, что этого достаточно, чтобы вам в чём попрактиковаться некоторое время. Ещё остались MESSAGE BREAKPOINTS, но я не хочу вас уморить, так что ждите 12 часть, а после того, как рассмотрим последний вид точек останова, обещаю, что продолжим крэкать по-немногу, так, чтобы не заскучать.

Введение в крэкинг с нуля, используя OllyDbg - Глава 12

Автор: Рикардо Нарваха, пер. Quantum

Дата: 2007-10-05

Данная глава посвящена практическому использованию сообщений Windows.

Следующая цитата вкратце описывает назначение сообщений Windows:

Сообщения в Windows широко используются для уведомления о различных событиях. Если Вы хотите заставить какое-либо окно или контрол (элемент пользовательского интерфейса, на самом деле тоже являющийся окном, как то: кнопка, edit, Toolbar, Tree view и т.д.) совершить определённое действие, отправьте ему соответствующее сообщение. Сообщения также могут исходить от других приложений. Система уведомляет о таких событиях как передвижение мыши, нажатие клавиши на клавиатуре и т.д. также посредством сообщений.

Как мы уже рассмотрели ранее, в OllyDbg можно расставлять точки останова BPX на вызовах различных API-функций. Зачастую более эффективным способом "отловить" интересующие нас участки кода являются точки останова на сообщениях. Их ещё называют BMSG в честь одноимённой команды в ядерном отладчике SoftIce.

Оконные приложения имеют по крайней мере один так называемый цикл обработки сообений (англ. - message loop). Этот цикл являет собой вызов определеённых API-функций (чаще всего используются функции GetMessage и DispatchMessage, но встречаются и другие варианты воплощения message loop). Для более глубокого ознакомления с циклом обработки сообщений автор рекомендует статью "Understanding the Message Loop" (C) Brook Miles:

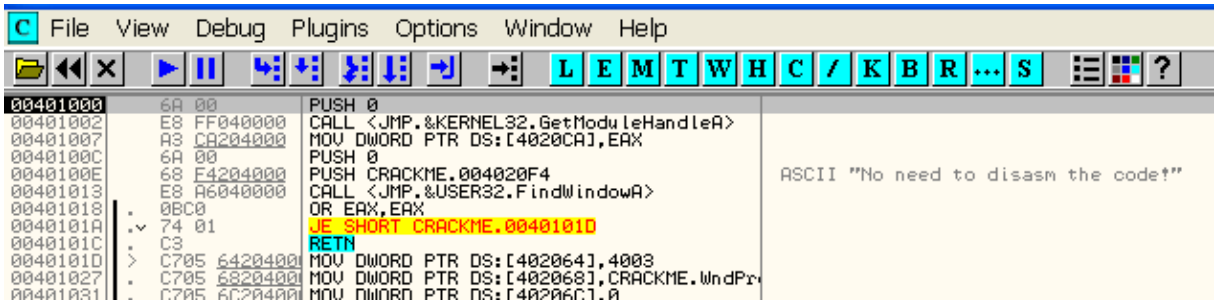
http://winprog.org/tutorial/messageloop.html

На русском и гораздо более подробно про цикл сообщений можно почитать у братьев Фроловых:

http://www.frolov-lib.ru/books/bsp/v11/ch1.htm#ch14

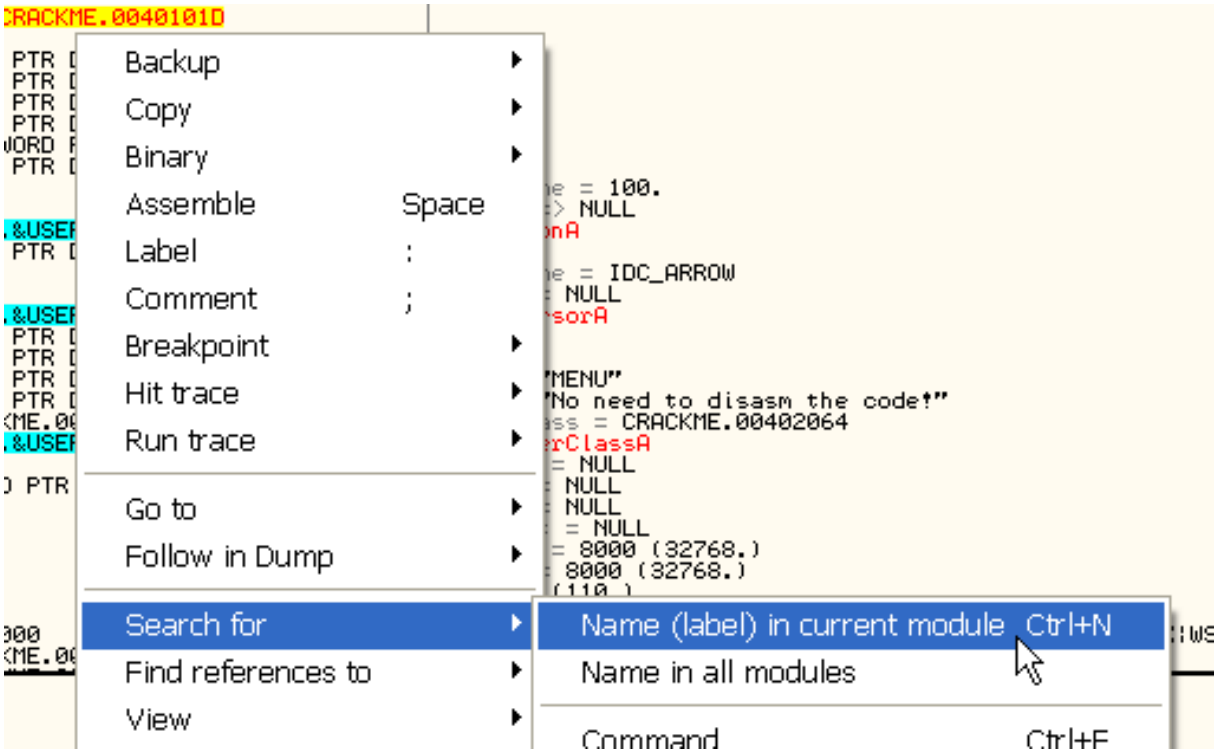
Правда, там речь идёт о Win16, но именно по данной теме особых разногласий с Win32 в тексте не наблюдается. Спасибо IceStudent'у за ссылку!

Давайте рассмотрим простой пример: для того, чтобы выудить серийник из крэкми, мы установим BMSG на сообщение о нажатии кнопки, ведь именно это сообщение должно инициировать проверку введенных данных. Итак, загружаем крэкми CrueHead'a в OllyDbg.



Но сначала попробуем выудить серийник классическим способом, а потом через BMSG и сравним. Для начала проанализируем API-функции, которые импортирует программа, чтобы найти ту, которой скорее всего считывается введенный текст для последующей проверки.

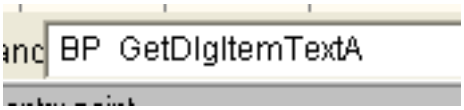
Правый щелчёк в ассемблерном листинге - Search for - Name (label) in current module.



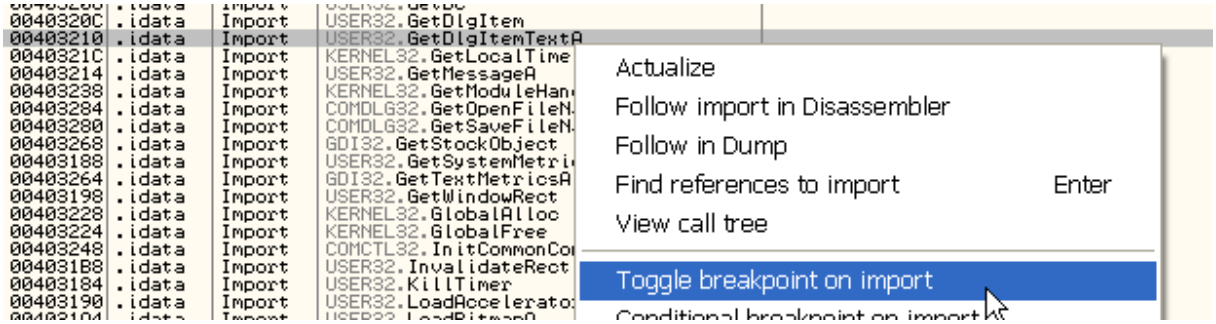
Чаще всего для считывания текста из edit'a используются функции GetDlgItemTextA и GetWindowTextA. Конечно, могут использоваться и юникодовые версии данных функций (с суффиксом W), либо сообщение может посылаться напрямую (тут тоже есть несколько способов различного уровня сложности), либо более экзотический вариант с прямым считыванием буфера edit'a. Короче, и не надейтесь, что в продвинутых крэкми текст из edit'a будет извлекаться банальным вызовом GetDlgItemTextA или GetWindowTextA, но для начала мы рассмотрим именно этот вариант.

В списке видна функция GetDlgItemTextA. Это вовсе не значит, что данная функция используется именно для считывания имени пользователя и серийника (в программе могут быть и другие поля ввода), если вообще используется. Автор крэкми мог намеренно добавить данную функцию в импорт, чтобы ввести нас в заблуждение. Между прочим, функции API можно подгружать динамически (и даже разными способами), т.е. не через импорт. Таким образом, крэкми может на самом деле использовать GetDlgItemTextA, но в списке импорта функции не будет. Не будем пока усложнять и без того нелёгкий материал и предположим, что крэкми действительно использует GetDlgItemTextA для считывания текста из edit'a ;)

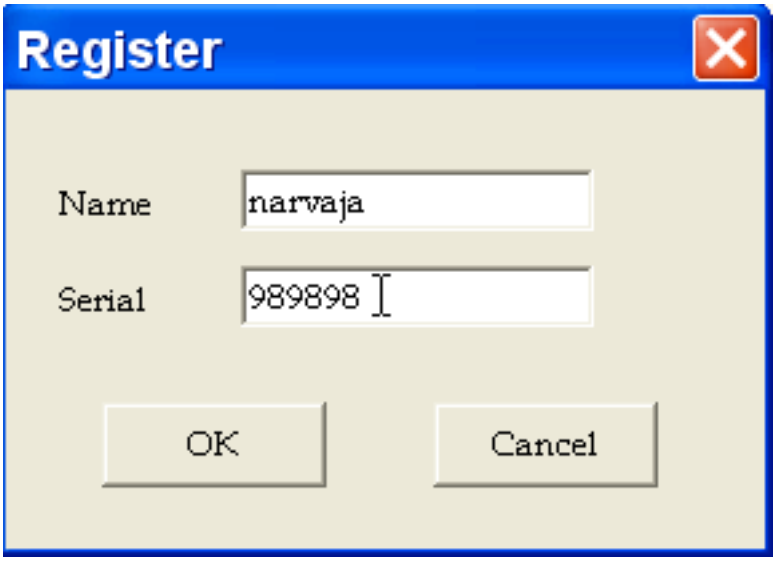
Ставим BP на вызов данной функции:



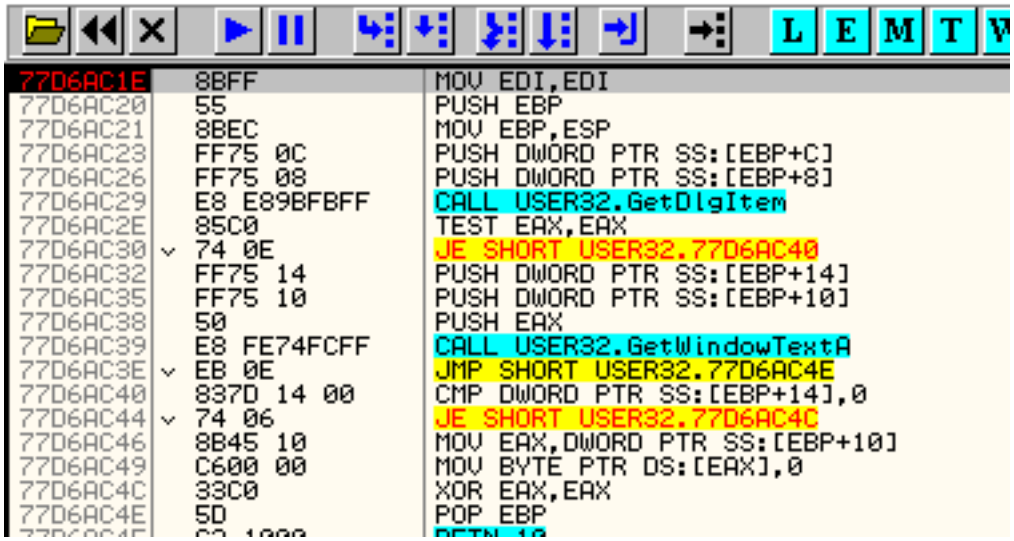
Или так:



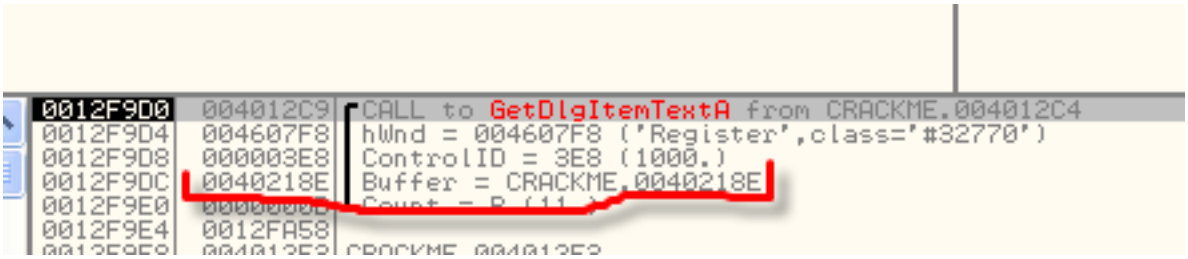
После установки BPX, запускаем программу на выполнение, вводим имя пользователя и серийник:



Нажимаем OK - "брякаемся" в коде:

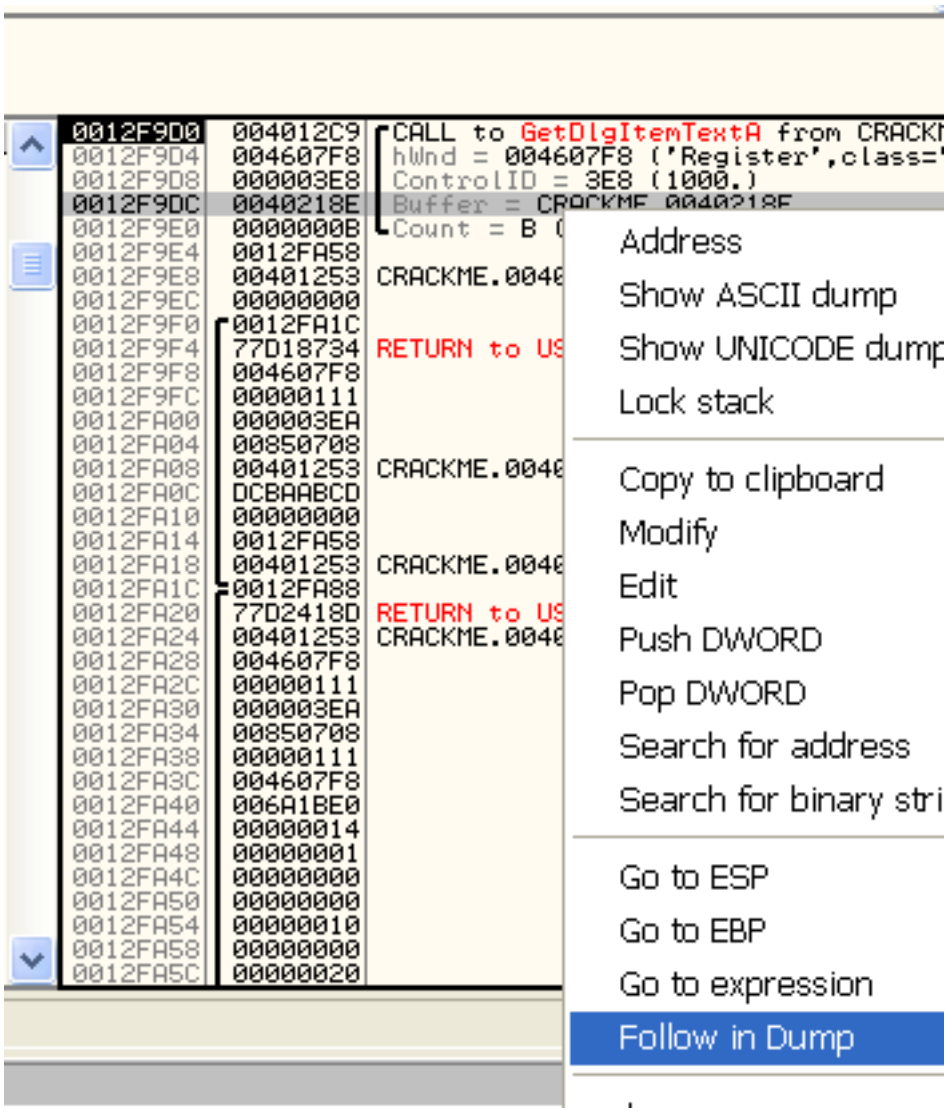


Обратите внимание на стек.

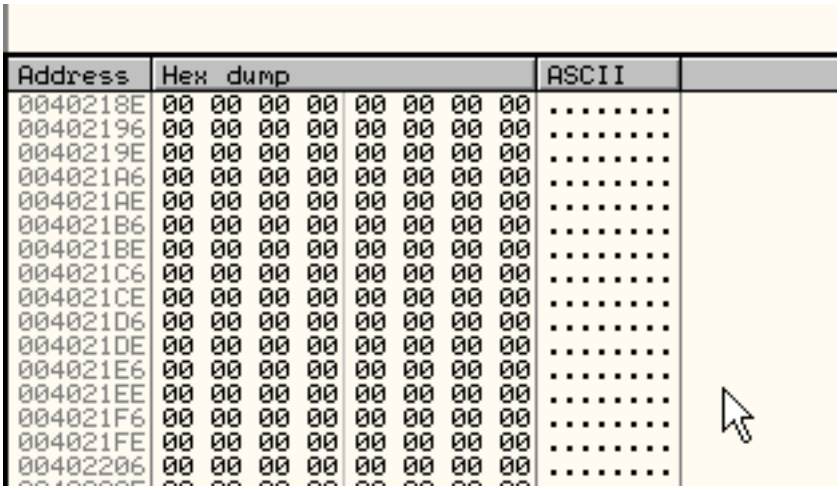


Видим, что один из параметров функции является указателем на буфер, в который будет скопировано содержимое edit'a.

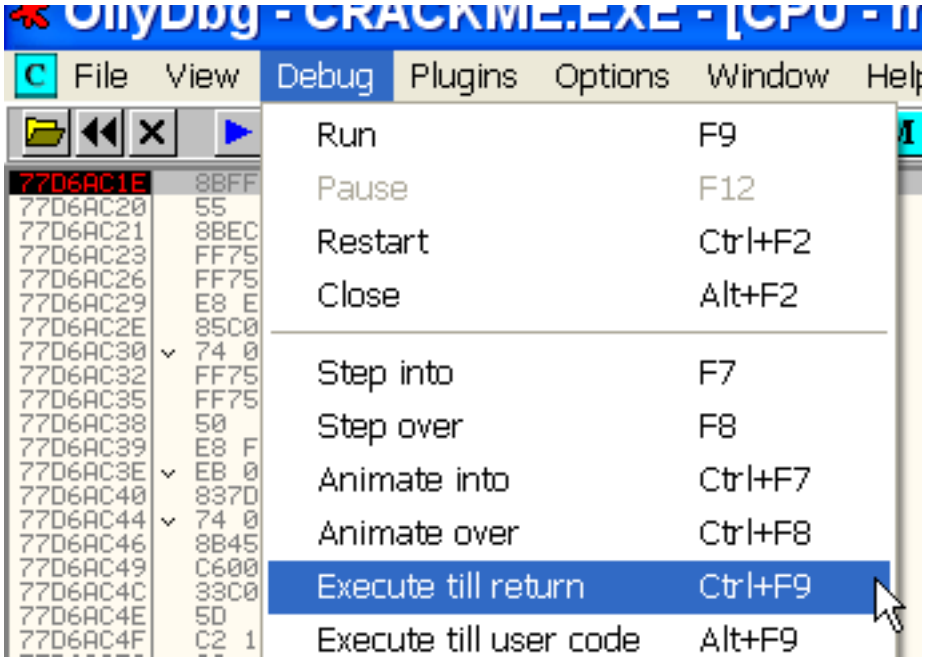
Давайте загрузим данный буфер в дамп посредством команды Follow in dump или через Go to - Expression: 40218E.



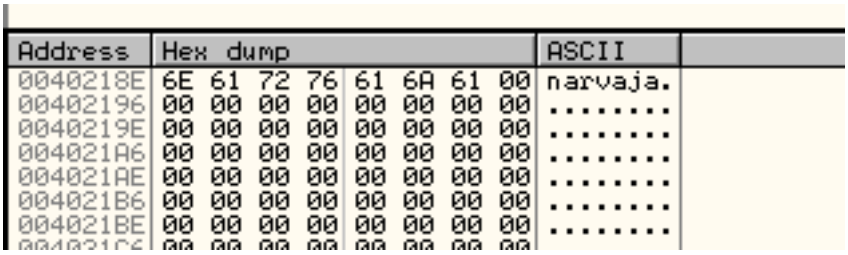
Сейчас в буфере пусто, ведь функция ещё не выполнилась.



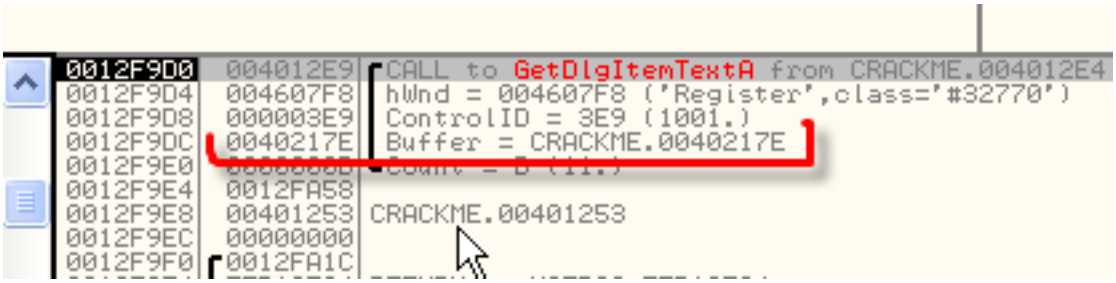
Давайте дадим этой функции возможность выполниться: Debug - Execute till return.



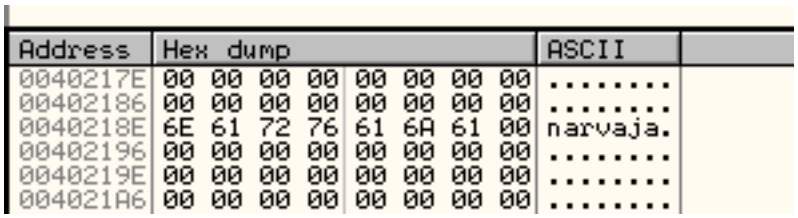
Теперь в дампе буфера находится имя, которое Вы ввели для регистрации.



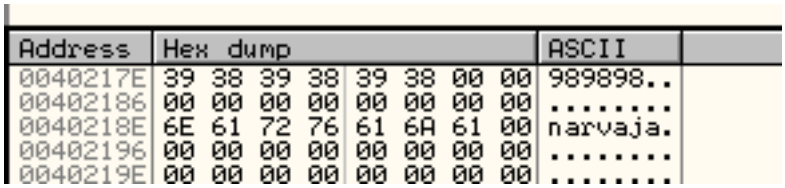
Снова задаём команду Run (F9) - опять срабатывает наша точка останова.



Теперь параметр буфера указывает на адрес 40217E. Давайте посмотрим в дампе:



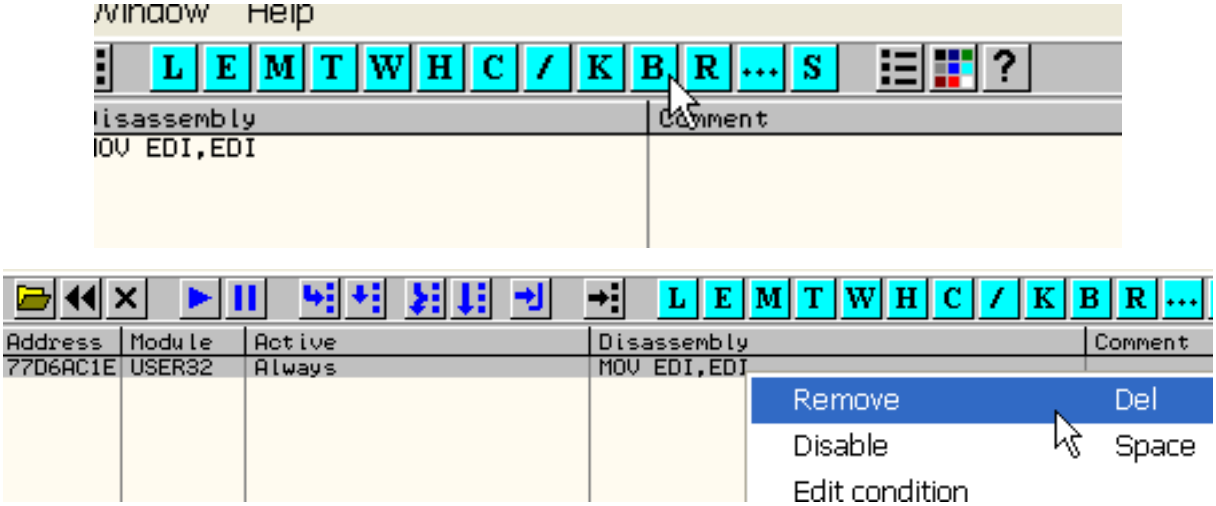
Задаём Execute till return.



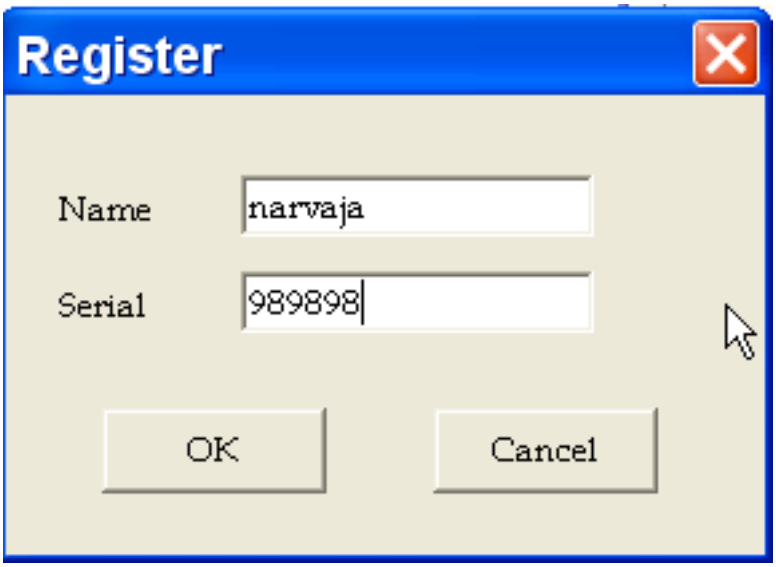
А вот и наш серийник!

Как Вам уже должно быть понятно, для извлечения правильного серийника, первым делом нужно поймать момент доступа приложения к введенным данным (в данном случае - имя и пробный серийник). Дальнейший план действий мы рассмотрим чуть позже, а пока попробуем повторить первый шаг, но с использованием BMSG. Многие алгоритмы проверки специально не используют общепринятые API для считывания полей ввода текста, а посылают сообщения напрямую, чтобы насолить реверсеру.

Снимаем все действующие BPX в окне [B] командой Remove.

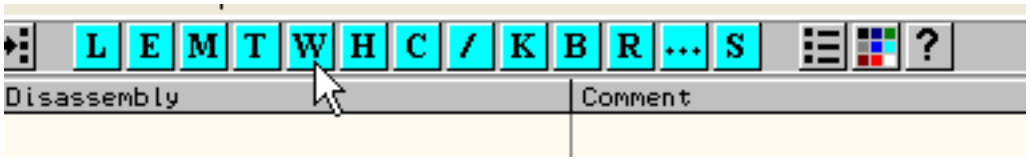


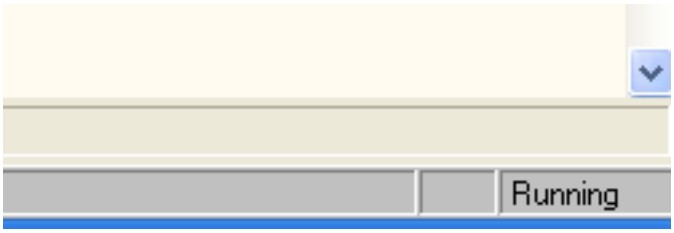
Снова запускаем приложение, вводим имя и серийник, но пока ещё НЕ нажимаем OK.



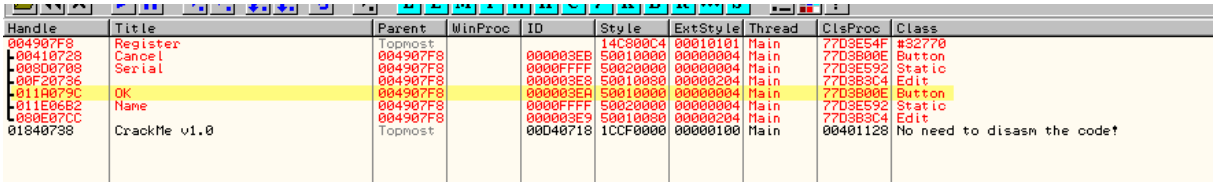
Одна из особенностей использования BMSG по сравнению с BPX на API-функции заключается в том, что BPX можно расставлять заранее, ещё до запуска процесса, тогда как BMSG имеет смысл устанавливать только после создания окна, чьи сообщения нужно перехватить.

Открываем окно [W]indows в процессе отладки (не нужно предварительно останавливать процесс, т.е. статус процесса вполне может быть Running).





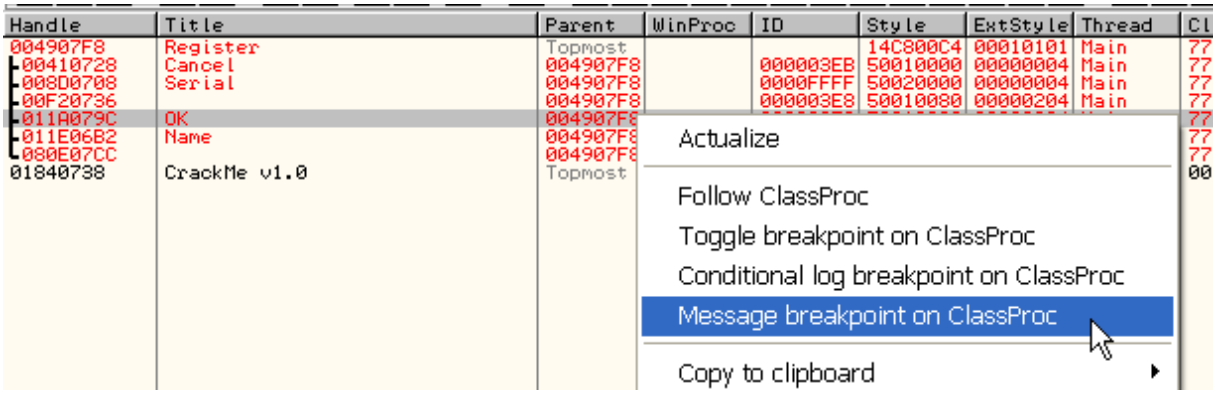
Если в списке [W]indows пусто, нужно сделать правый щелчёк мышью и выбрать пункт Actualize.

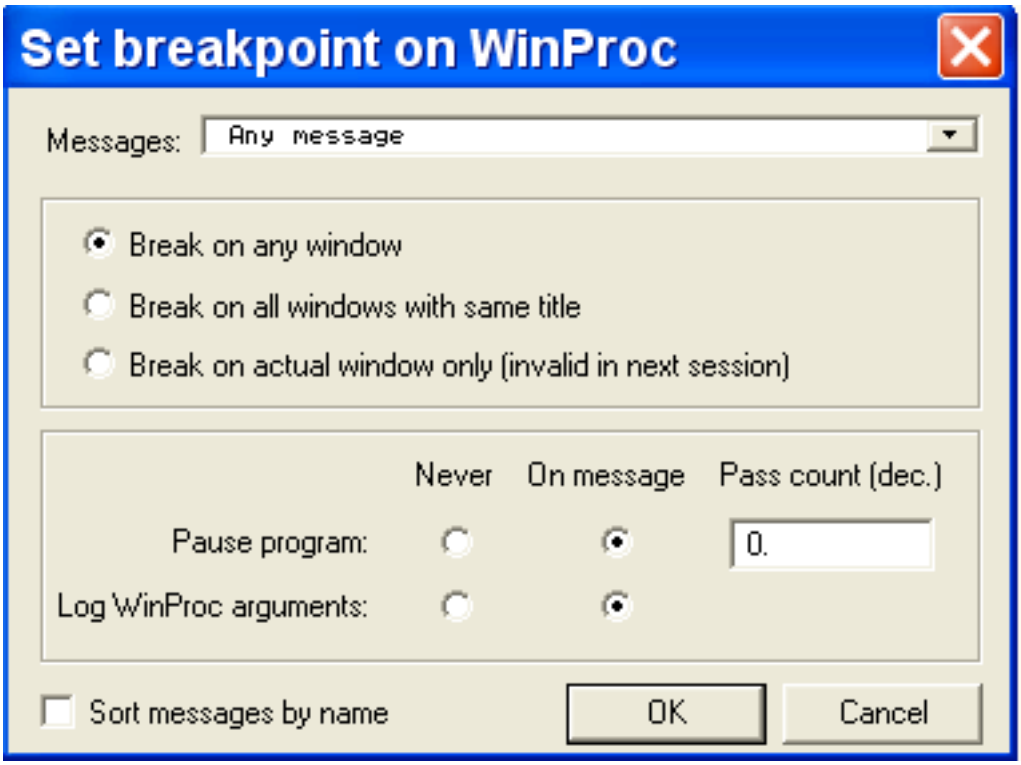


Handle	Title	Parent	WinProc	ID	Style	ExtStyle	Thread	ClsProc	Class
004907F8	Register	Topmost			14C800C4	00010101	Main	77D3E54F	#32770
00410728	Cancel	004907F8		000003EB	50010000	00000004	Main	77D3B00E	Button
008D0708	Serial	004907F8		0000FFFF	50020000	00000004	Main	77D3E592	Static
00F20736		004907F8		000003E8	50010080	00000204	Main	77D3B3C4	Edit
011A079C	OK	004907F8		000003EA	50010000	00000004	Main	77D3B00E	Button
011E06B2	Name	004907F8		0000FFFF	50020000	00000004	Main	77D3E592	Static
080E07CC		004907F8		000003E9	50010080	00000204	Main	77D3B3C4	Edit
01840738	CrackMe v1.0	Topmost		00D40718	1CCF0000	00000100	Main	00401128	No need to disasm the code!

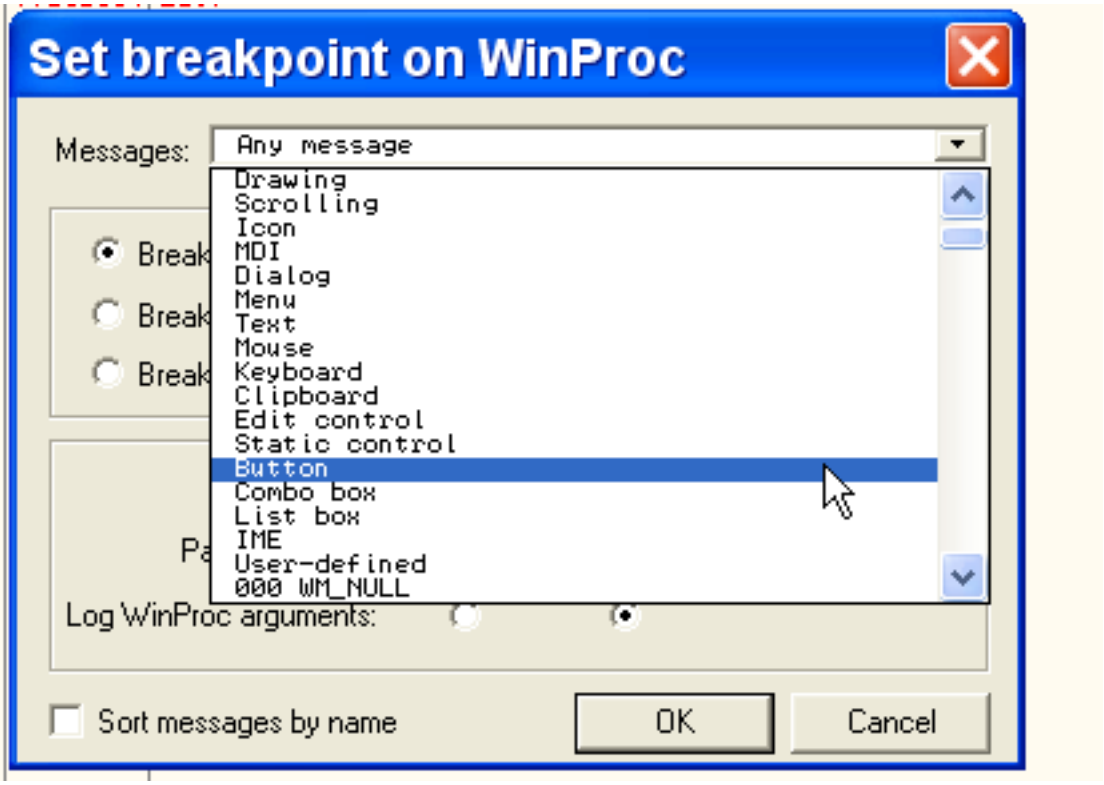
Ищем в списке контрол "кнопка"... В колонке Class должно быть значение Button, а в Title - OK. Есть такой контрол!

Правый щелчёк в строчке данного контрола - выбираем Message breakpoint on ClassProc.

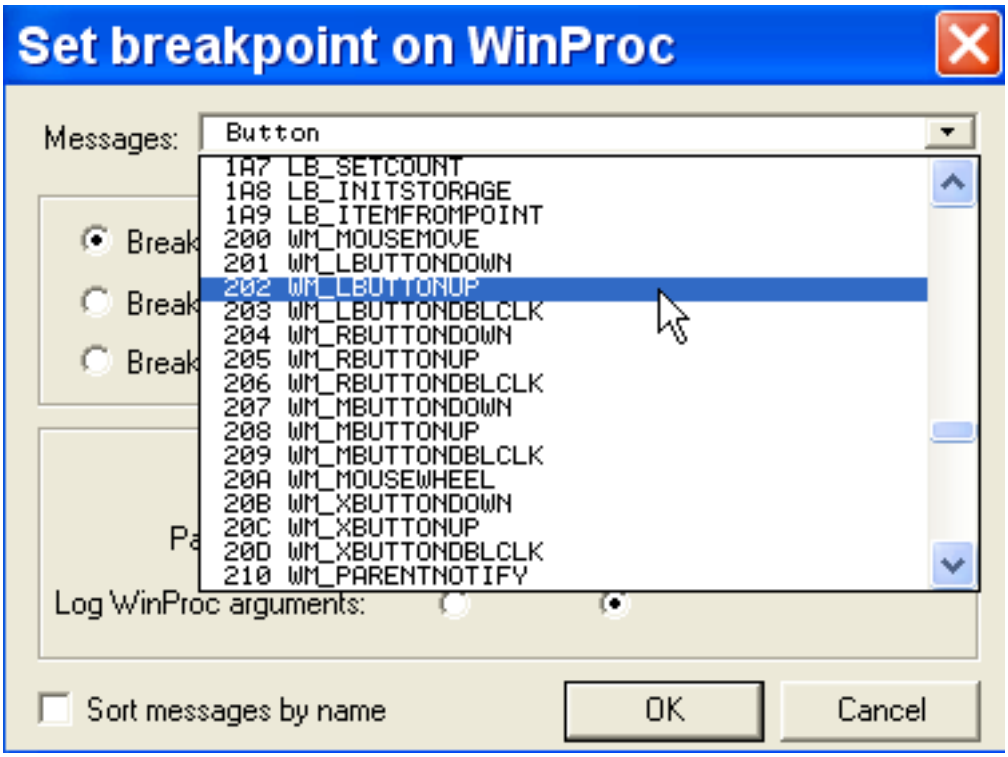




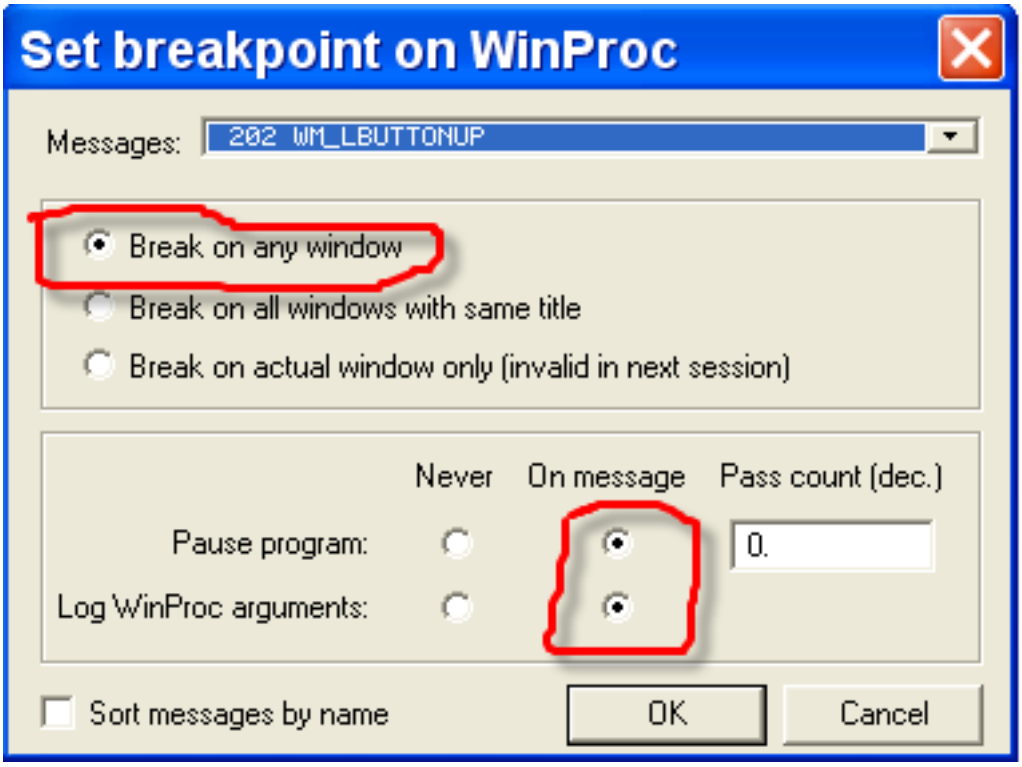
В открывшемся окне нужно указать сообщение, которое мы собираемся поймать. Во всплывающем списке:



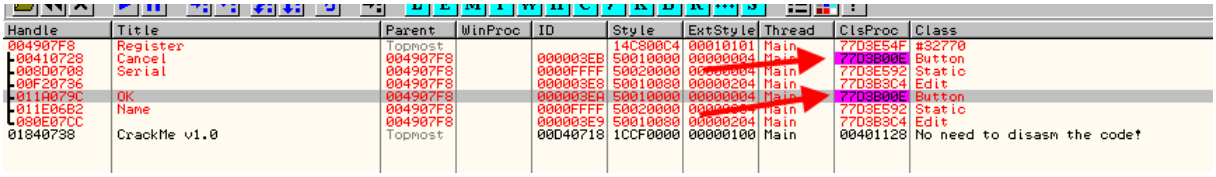
Есть тут и стандартные сообщения для статических текстовых контролов и кнопок, и мышиные сообщения, и для буфера обмена и т.д. Если Вы не знаете точно какое сообщение нужно ловить, можете попробовать стандартные группы сообщений (в начале списка). При нажатии левой кнопки мыши система посылает сообщение WM_LBUTTONDOWN (L означает LEFT, т.е. левая). При отжатии посылается сообщение WM_LBUTTONUP. На это последнее сообщение мы и повесим BMSG - оно имеет номер 202.



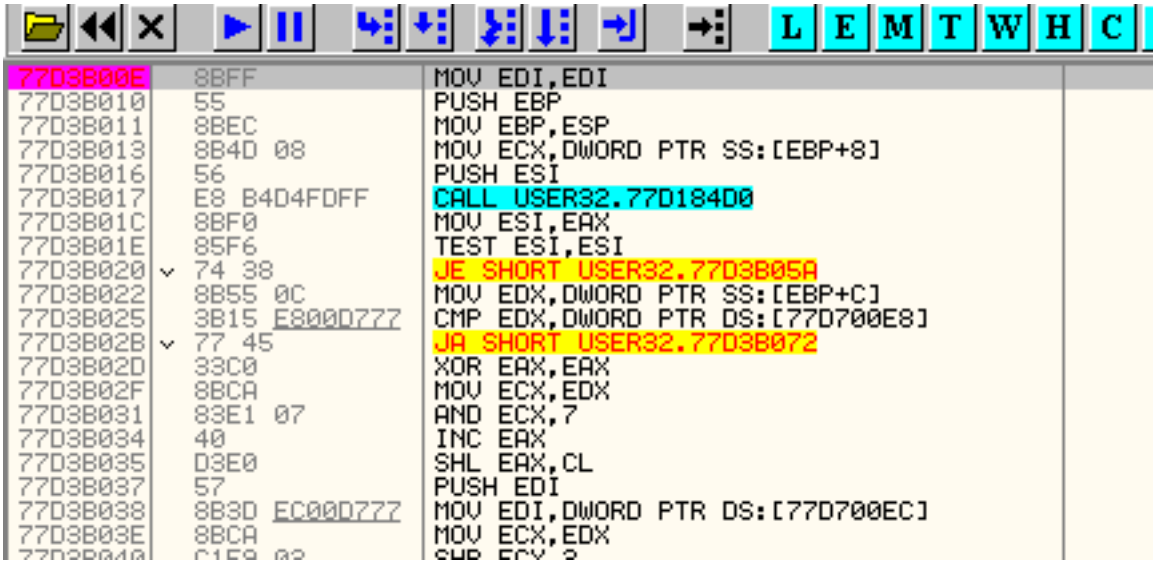
Вот как это должно выглядеть:



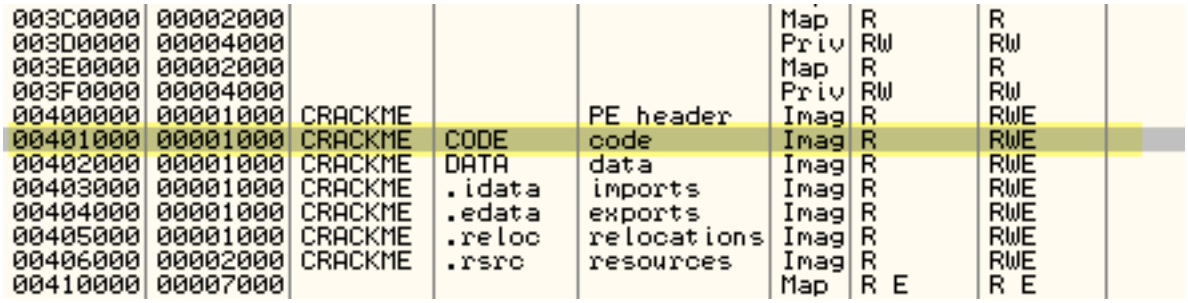
Мы выбрали сообщение 202 WM_LBUTTONUP. Щёлкаем по "Break on any window" и по "Pause program: On message" чтобы отлаживаемый процесс останавливался при поимке указанного сообщения. Ниже выбираем "Log WinProc arguments: On message" ибо регистрация каких-либо аргументов в логе не бывает лишней.



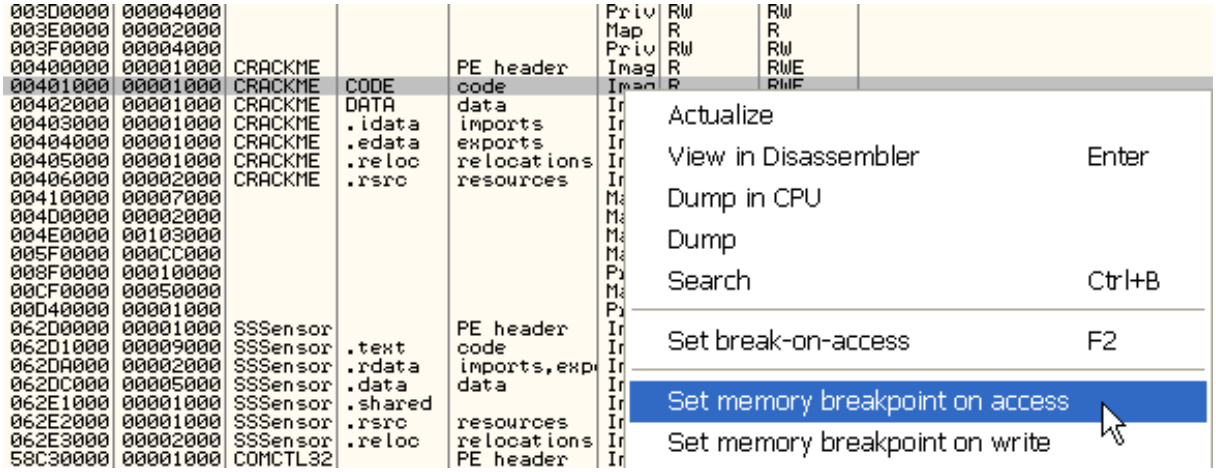
Данный BMSG будет срабатывать как при нажатии OK, так и Cancel, ведь мы указали, что BMSG вешается на все окна. Щёлкаем левой кнопкой мыши по кнопке OK.



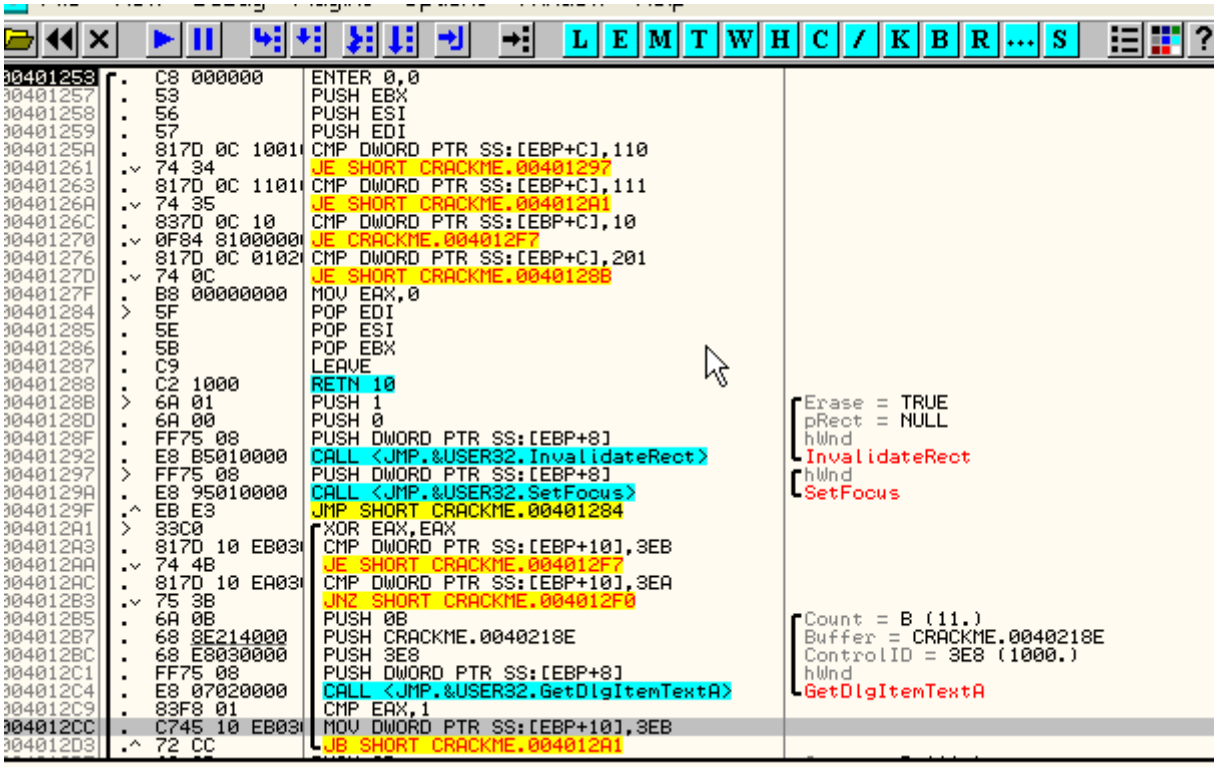
В этом месте теряются многие новички, т.к. при срабатывании BMSG попадают в незнакомый участок кода, не принадлежащий основному коду процесса. На самом же деле вернуться к основному коду вовсе не сложно.



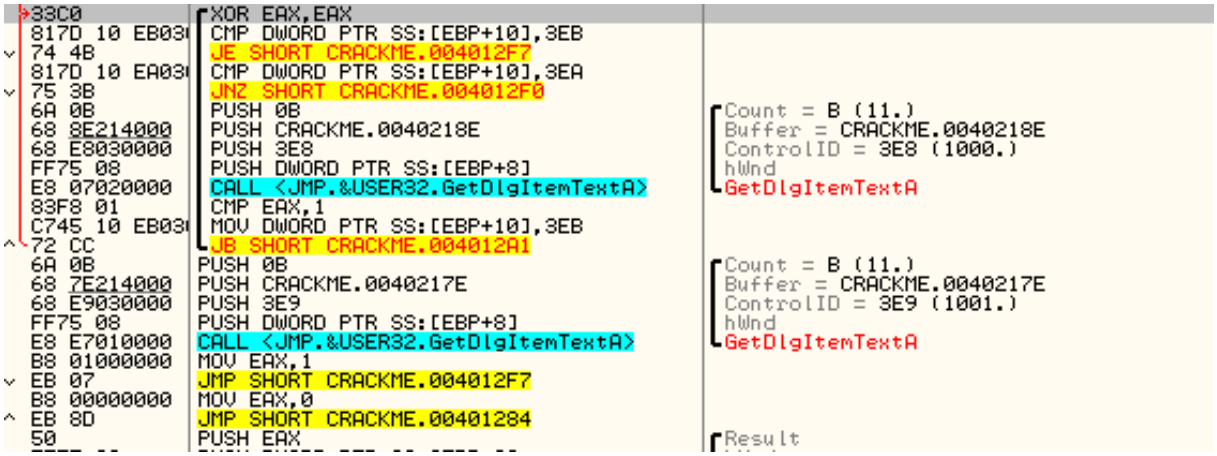
Как мы уже знаем, основной код находится в секции, которая начинается по адресу 401000. Ставим BPM on access на данную секцию, чтобы остановиться как только управление попадёт в диапазон адресов этой секции.



Жмём на Run и вскоре останавливаемся уже в коде программы.



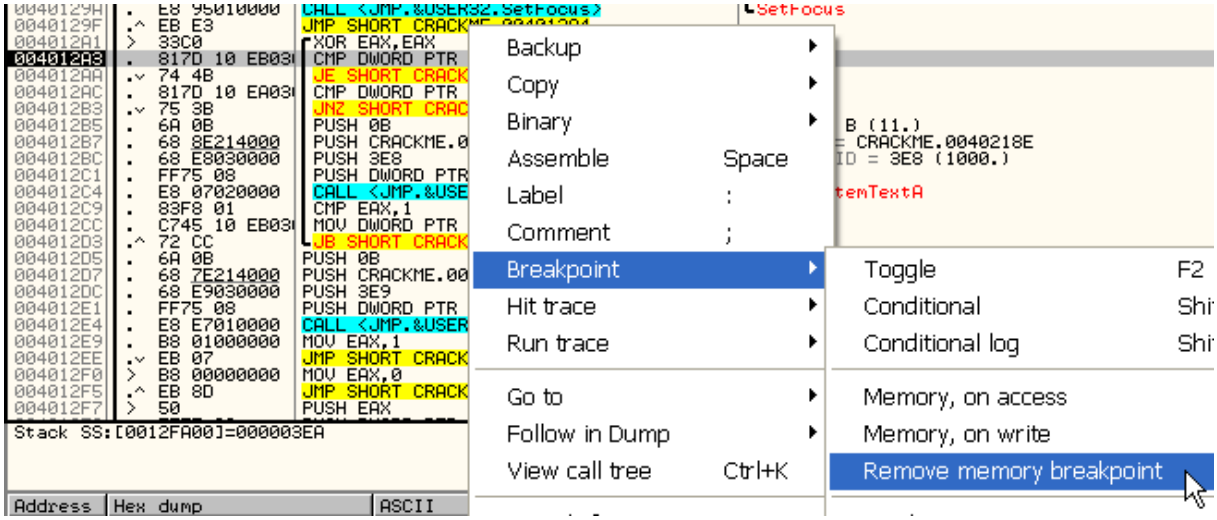
Не снимая BPM on access продолжаем нажимать F9 - получается пошаговое выполнение инструкций, но только в пределах текущей секции.



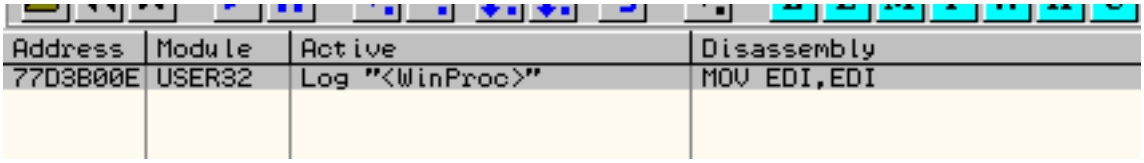
Сразу после выхода по 2м RET, попадаем на участок кода с вызовами GetDlgItemTextA, которые считывают имя пользователя и серийник. Таким образом, мы снова очутились на интересующем нас участке кода, но в этот раз мы не использовали BPX на обращения к API.

Если бы приложение не использовало вызовы API для чтения серийника, методом BMSG мы бы всё равно отыскали данный участок. В этом и заключается преимущество последнего.

Если мы хотим отловить нажатие кнопки OK но не мышью, а через клавиатуру, первым делом снимаем BPM on access: правый щелчёк - Remove memory breakpoint.

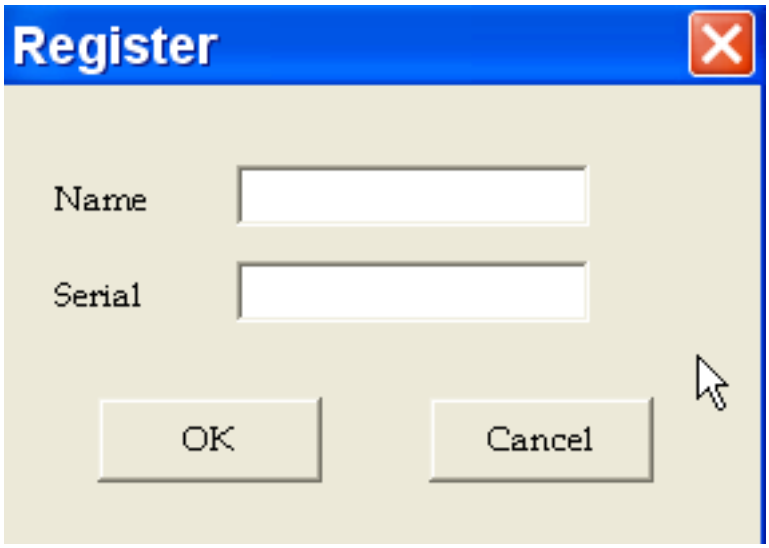


Открываем список точек останова в окне [B] и удаляем все BMSG опцией Remove.

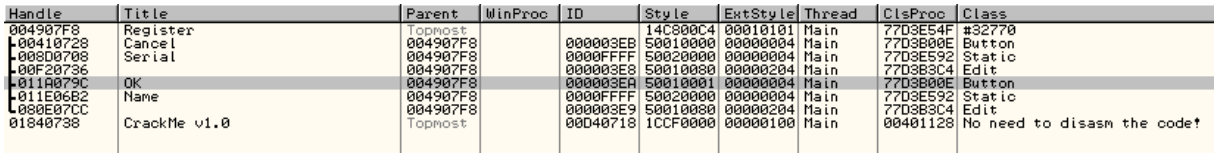


Address	Module	Active	Disassembly
77D3B00E	USER32	Log "<WinProc>"	MOV EDI,EDI

Снова запускаем программу и открываем окно ввода серийника, но пока ничего не вводим.

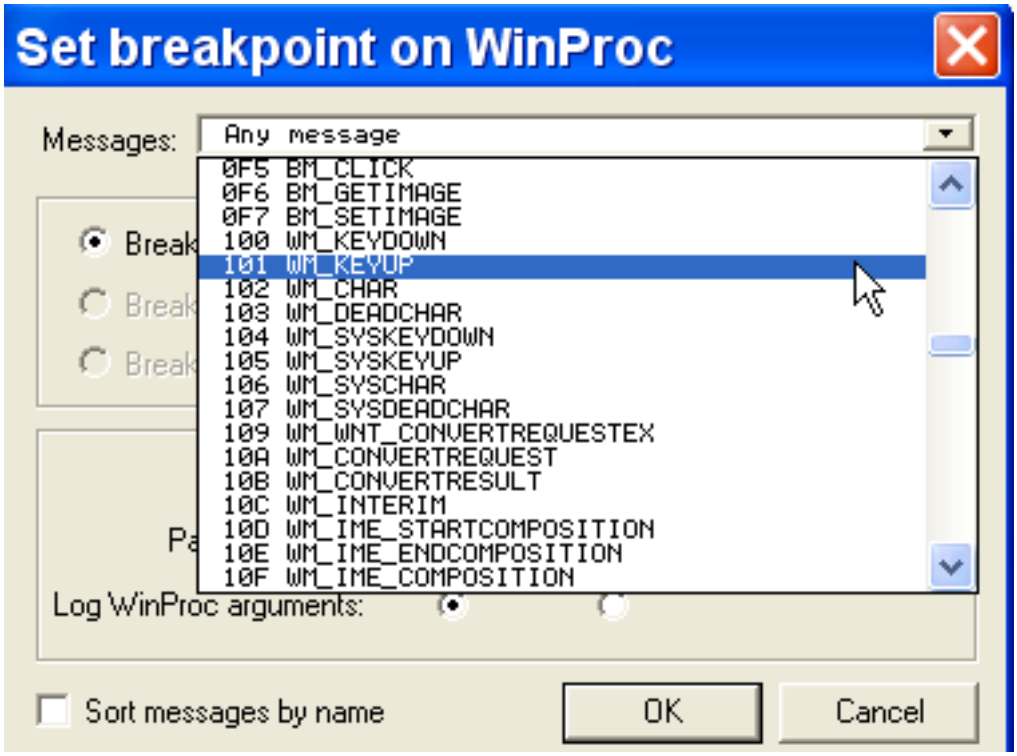


Открываем закладку [W].

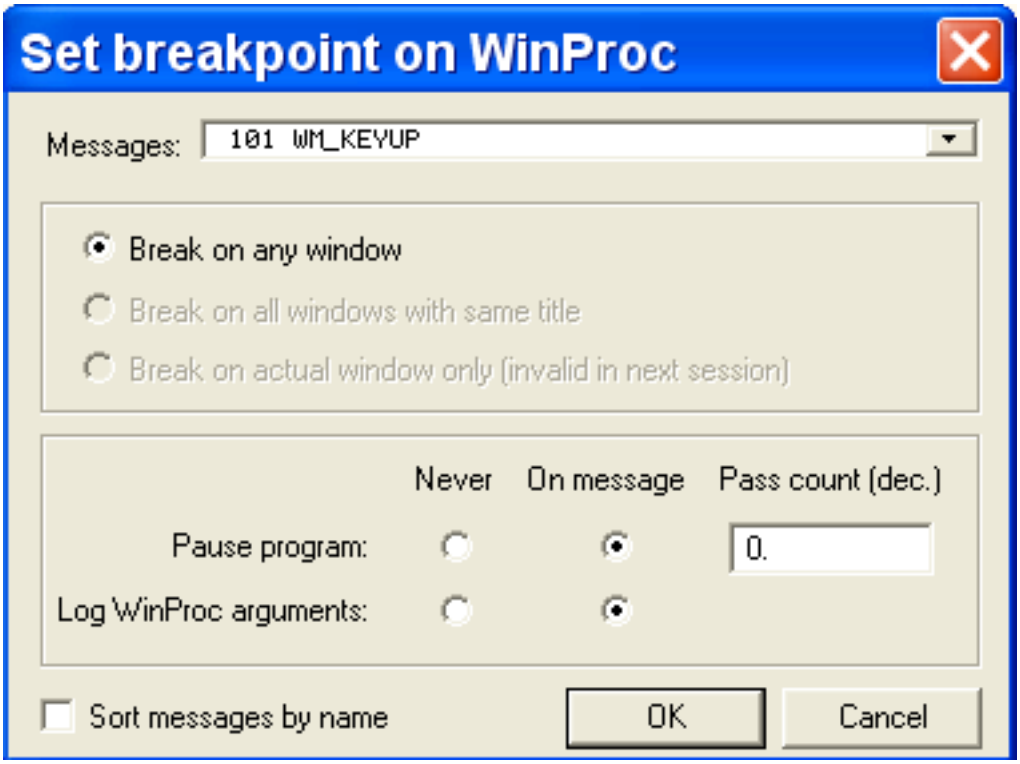


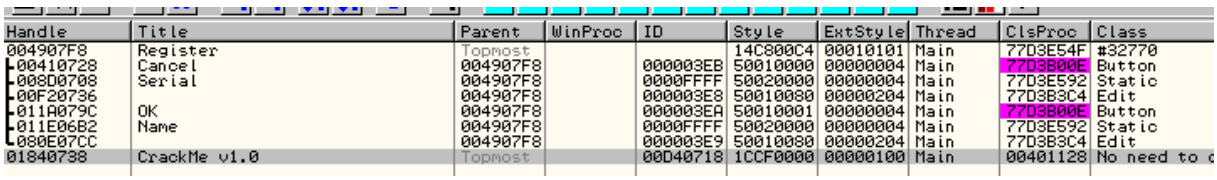
Handle	Title	Parent	WinProc	ID	Style	ExtStyle	Thread	ClsProc	Class
004907F8	Register	Topmost			14C800C4	00010101	Main	77D3E54F	#32770
00410728	Cancel	004907F8		000003EB	50010000	00000004	Main	77D3B00E	Button
008D0708	Serial	004907F8		0000FFFF	50020000	00000004	Main	77D3E592	Static
00F20736		004907F8		000003E8	50010080	00000204	Main	77D3B3C4	Edit
011A079C	OK	004907F8		000003EA	50010001	00000004	Main	77D3B00E	Button
011E06B2	Name	004907F8		0000FFFF	50020000	00000004	Main	77D3E592	Static
080E07CC		004907F8		000003E9	50010080	00000204	Main	77D3B3C4	Edit
01840738	CrackMe v1.0	Topmost		00D40718	1CCF0000	00000100	Main	00401128	No need to disasm the code!

Повторяем уже знакомые действия, но в этот раз выбираем:



Сообщение WM_KEYUP под номером 101 - это сообщение генерируется при отжатии клавиши.





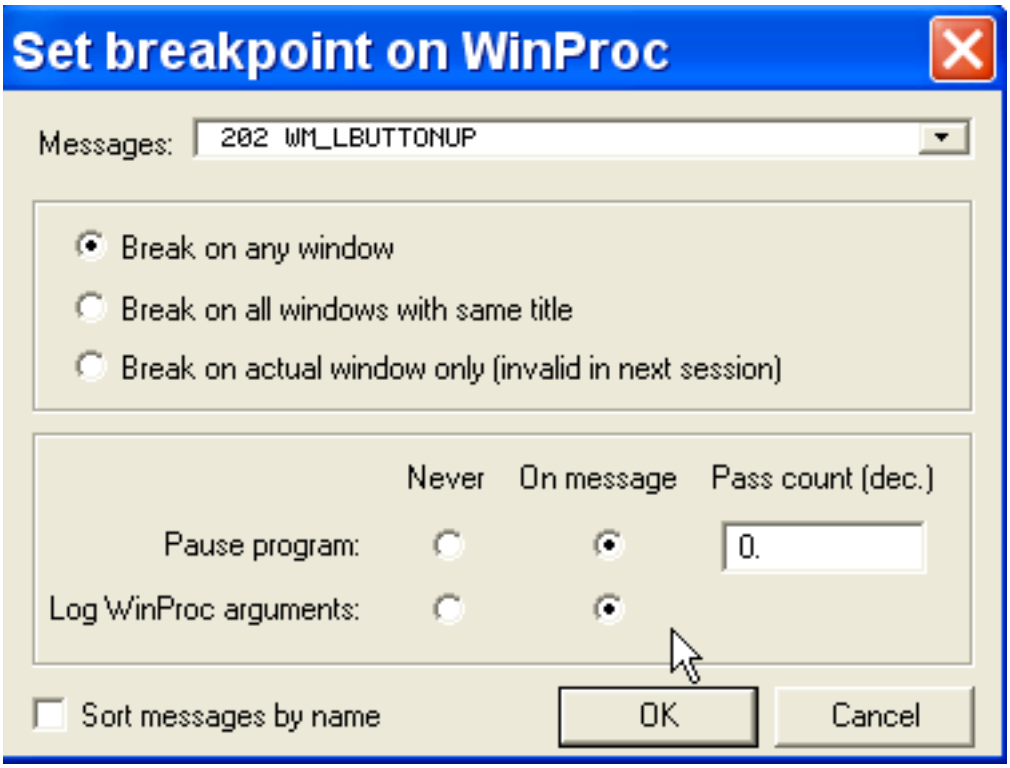
Handle	Title	Parent	WinProc	ID	Style	ExtStyle	Thread	ClsProc	Class
004907F8	Register	Topmost			14C800C4	00010101	Main	77D3E54F	#32770
00410728	Cancel	004907F8		000003EB	50010000	00000004	Main	77D3B00E	Button
008D0708	Serial	004907F8		0000FFFF	50020000	00000004	Main	77D3E592	Static
00F20736		004907F8		000003E8	50010080	00000204	Main	77D3B3C4	Edit
011A079C	OK	004907F8		000003EA	50010001	00000004	Main	77D3B00E	Button
011E06B2	Name	004907F8		0000FFFF	50020000	00000004	Main	77D3E592	Static
080E07CC		004907F8		000003E9	50010080	00000204	Main	77D3B3C4	Edit
01840738	CrackMe v1.0	Topmost		00D40718	1CCF0000	00000100	Main	00401128	No need to c

Нажимаем и отпускаем какую-либо клавишу для ввода первого символа имени пользователя - BMSG не срабатывает, т.к. чтение данных пользователя осуществляется не посимвольно, а целиком. Некоторые крэкми предпочитают проверять серийник по мере ввода, реагируя именно на

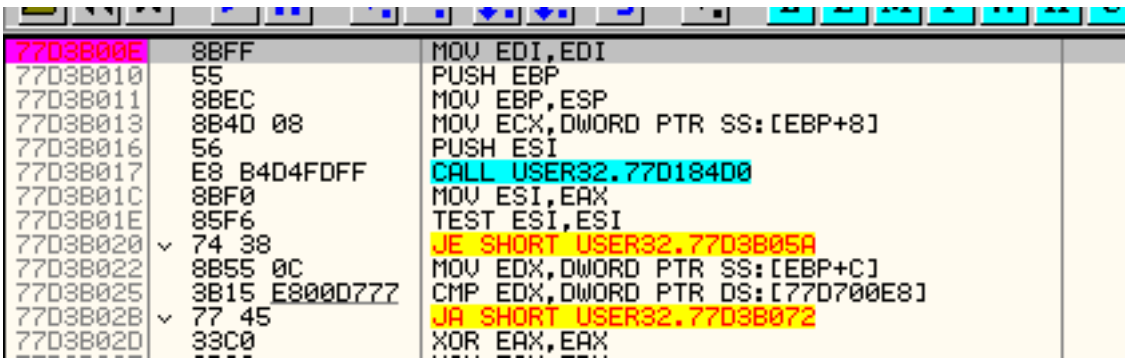
клавиатурные сообщения - такую возможность важно учитывать!

Можно ли заставить отладчик показывать сообщения "в живую", при этом не останавливаясь, чтобы потом поставить BMSG на те из них, которые представляют интерес?

Для этого нужно сначала установить BMSG на какое-либо сообщение. В данном случае можно снова использовать сообщение под номером 202, предварительно удалив лишние точки останова в списке [B], потом спровоцировать его срабатывание нажав и отпустив левую кнопку мыши.

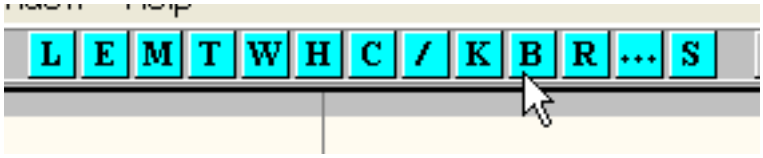


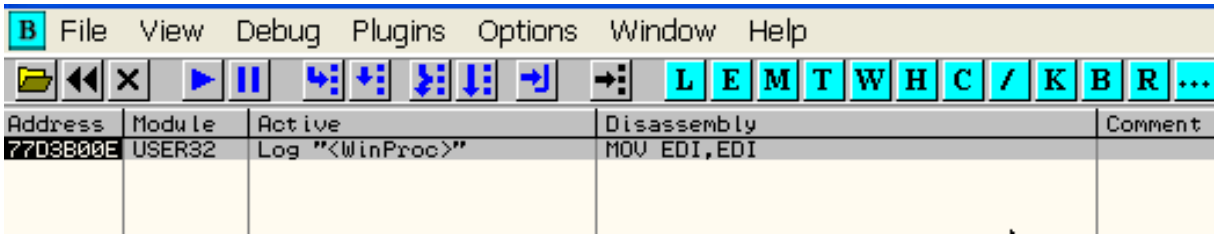
Устанавливаем BMSG, нажимаем мышью кнопку OK (в окне регистрации крэкми).



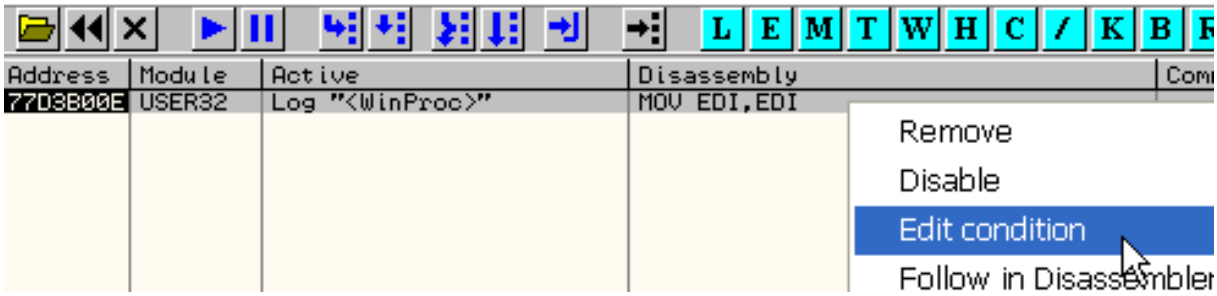
Срабатывает наш BMSG. Теперь мы его немного переделаем, чтобы ловить все сообщения и сбрасывать в лог.

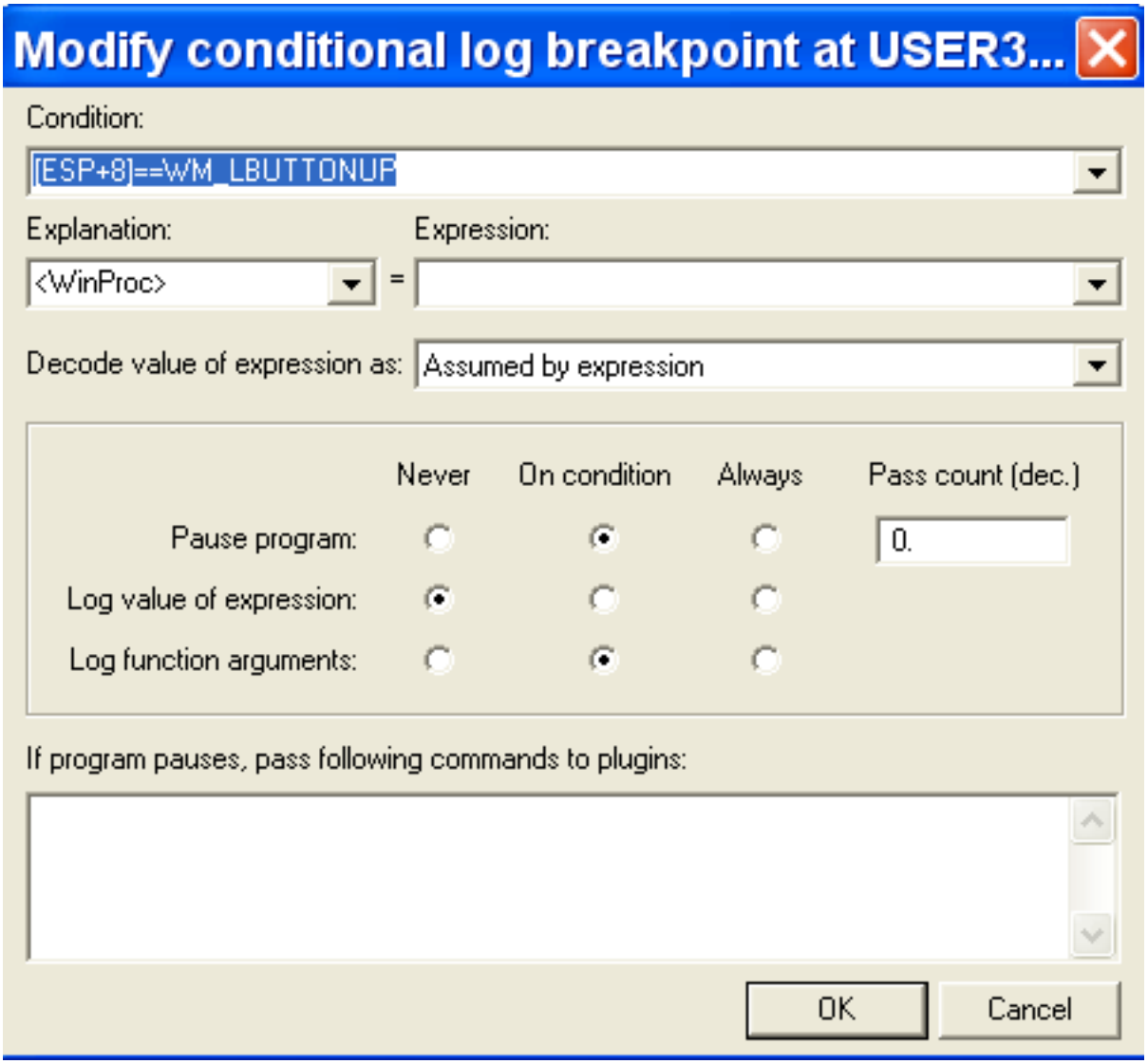
Открываем список точек останова по закладке [B].



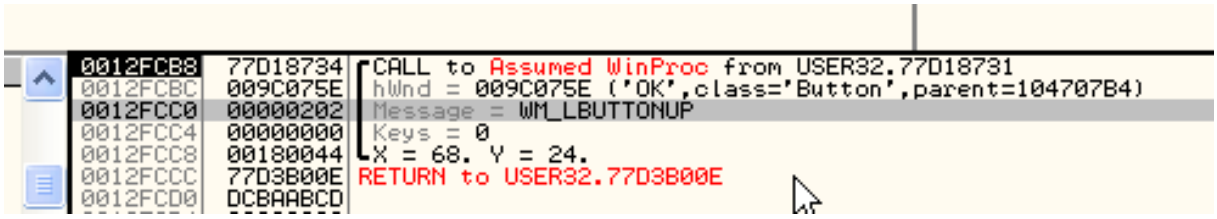


В списке находится наш BMSG: правый щелчёк - Edit condition.



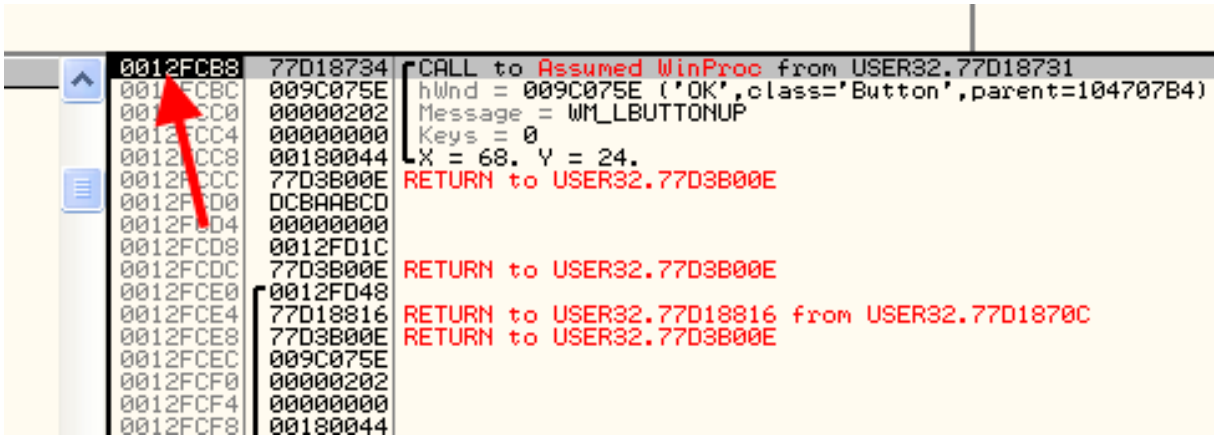


Кстати, обратите внимание на то, что BMSG - это всего лишь условная точка останова, условием которой является значение параметра [ESP+8] - оно должно равняться 202, т.е. WM_LBUTTONUP. Давайте посмотрим на стек:

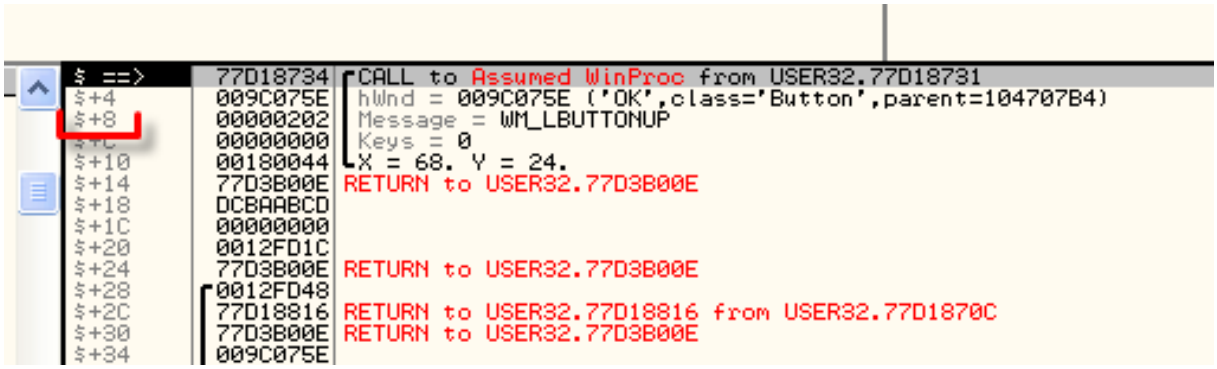


Там хранится значение 202 по адресу ESP+8, чем и вызвано срабатывание BMSG.

Если Вы уже забыли что собой представляет [ESP+8], сделайте двойной щелчёк:

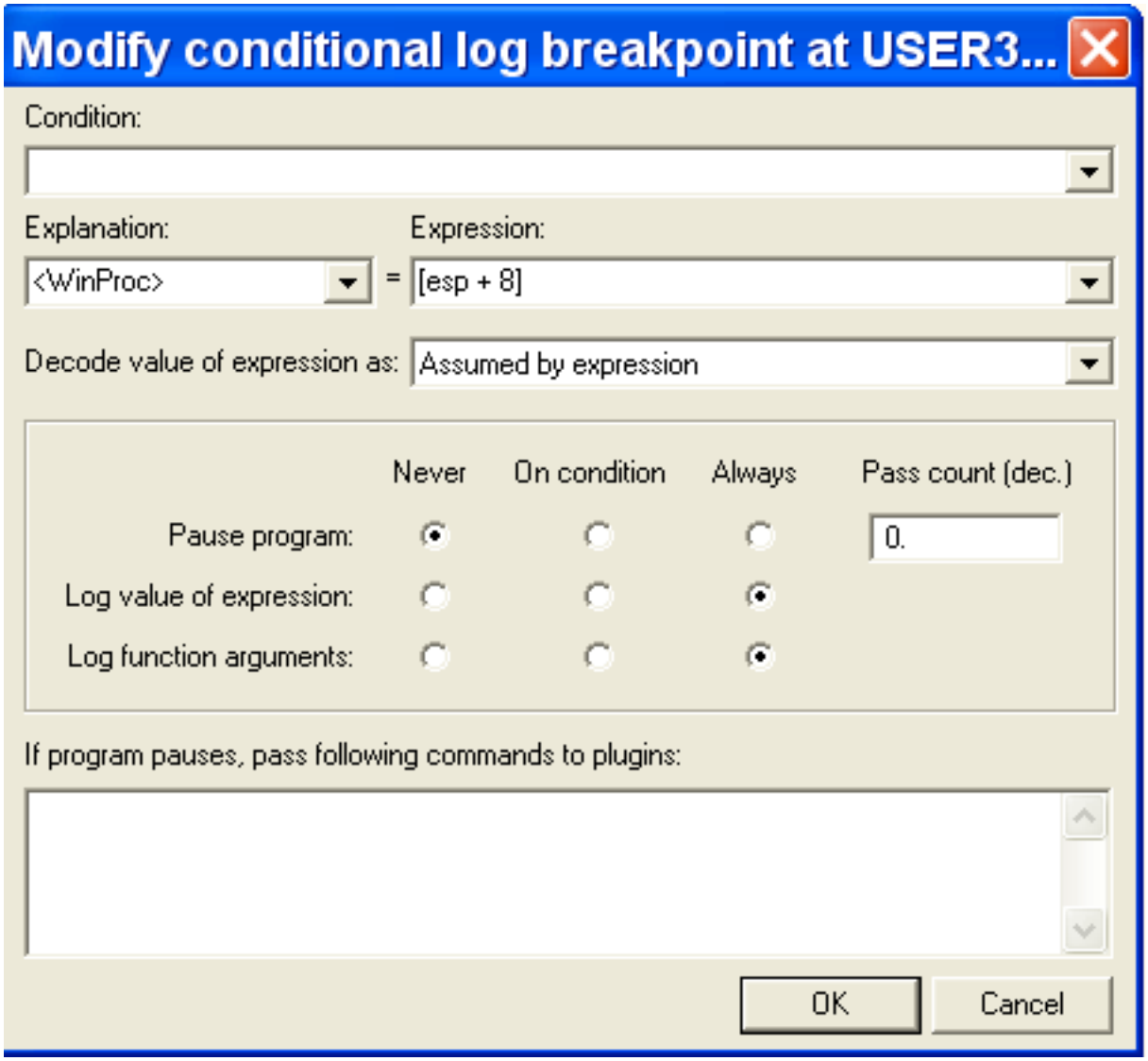


По самой верхней ячейке стека.



Адресация теперь показана относительно ESP: $+8 в данном случае означает ESP+8.

В общем, в ячейке [ESP+8] находится номер текущего сообщения, которое нам нужно сбрасывать в лог. Изменяем параметры условной точки останова:



В настройках указываем, что значение в выражении, т.е. [ESP+8], нужно записывать в лог вместе с другими параметрами функции; чтоб выполнение не останавливалось при срабатывании условного BPX ставим Never в Pause program.

Подтверждаем изменения кнопкой OK, дописываем имя пользователя и серийник, нажимаем OK, смотрим LOG сообщений:

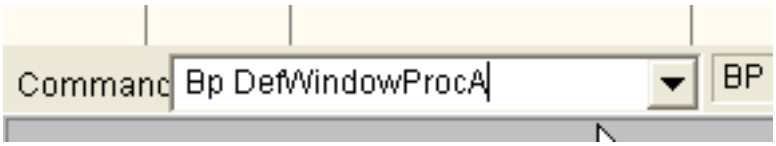
```
77D3B00E COND: <WinProc> = 00000087
77D3B00E CALL to Assumed WinProc from USER32.77D18731
           hWnd = 00F60728 ('OK',class='Button',parent=01270778)
           Message = WM_GETDLGCODE
           wParam = 0
           lParam = 0
77D3B00E COND: <WinProc> = 00000087
77D3B00E CALL to Assumed WinProc from USER32.77D18731
           hWnd = 041507DA ('Cancel',class='Button',parent=01270778)
           Message = WM_GETDLGCODE
           wParam = 0
           lParam = 0
77D3B00E COND: <WinProc> = 000000F4
77D3B00E CALL to Assumed WinProc from USER32.77D18731
           hWnd = 00F60728 ('OK',class='Button',parent=01270778)
           Message = BM_SETSTYLE
           Style = BS_DEFPUSHBUTTON
           Redraw = TRUE
77D3B00E COND: <WinProc> = 00000201
77D3B00E CALL to Assumed WinProc from USER32.77D18731
           hWnd = 00F60728 ('OK',class='Button',parent=01270778)
           Message = WM_LBUTTONDOWN
           Keys = MK_LBUTTON
           X = 46. Y = 7.
77D3B00E COND: <WinProc> = 00000007
77D3B00E CALL to Assumed WinProc from USER32.77D18731
           hWnd = 00F60728 ('OK',class='Button',parent=01270778)
           Message = WM_SETFOCUS
           hWndLose = 00A3075E (class='Edit',parent=01270778)
           lParam = 0
77D3B00E COND: <WinProc> = 000000F3
77D3B00E CALL to Assumed WinProc from USER32.77D18731
           hWnd = 00F60728 ('OK',class='Button',parent=01270778)
           Message = BM_SETSTATE
           Highlight = TRUE
           lParam = 0
77D3B00E COND: <WinProc> = 00000200
77D3B00E CALL to Assumed WinProc from USER32.77D18731
           hWnd = 00F60728 ('OK',class='Button',parent=01270778)
           Message = WM_MOUSEMOVE
           Keys = MK_LBUTTON
           X = 47. Y = 10.
77D3B00E COND: <WinProc> = 000000F3
77D3B00E CALL to Assumed WinProc from USER32.77D18731
           hWnd = 00F60728 ('OK',class='Button',parent=01270778)
           Message = BM_SETSTATE
           Highlight = TRUE
           lParam = 0
77D3B00E COND: <WinProc> = 00000202
77D3B00E CALL to Assumed WinProc from USER32.77D18731
           hWnd = 00F60728 ('OK',class='Button',parent=01270778)
           Message = WM_LBUTTONUP
           Keys = 0
           X = 47. Y = 10.
77D3B00E COND: <WinProc> = 000000F3
77D3B00E CALL to Assumed WinProc from USER32.77D18731
           hWnd = 00F60728 ('OK',class='Button',parent=01270778)
           Message = BM_SETSTATE
           Highlight = FALSE
           lParam = 0
77D3B00E COND: <WinProc> = 00000008
77D3B00E CALL to Assumed WinProc from USER32.77D18731
           hWnd = 00F60728 ('OK',class='Button',parent=01270778)
           Message = WM_KILLFOCUS
           hWndGet = 01270778 ('Register',class='#32770')
           lParam = 0
77D3B00E COND: <WinProc> = 00000002
77D3B00E CALL to Assumed WinProc from USER32.77D18731
           hWnd = 00F60728 ('OK',class='Button',parent=01270778)
```

Как видно в логе, сначала приходит сообщение 201 WM_LBUTTONDOWN, а потом - 202 WM_LBUTTONUP. Клавиатурные сообщения WM_KEYDOWN и WM_KEYUP не обрабатываются, как мы уже заметили.

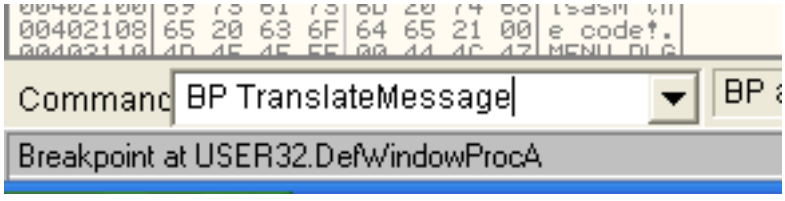
Чтобы отловить и записать в лог все сообщения, которые получает программа (кнопки, поля ввода текста и т.д.) можно установить условные точки останова на функции обработки сообщений: TranslateMessage и/или DefWindowProcA.

Для более полной картины можете развесить условные BPX на обе функции. Вот как это делается:

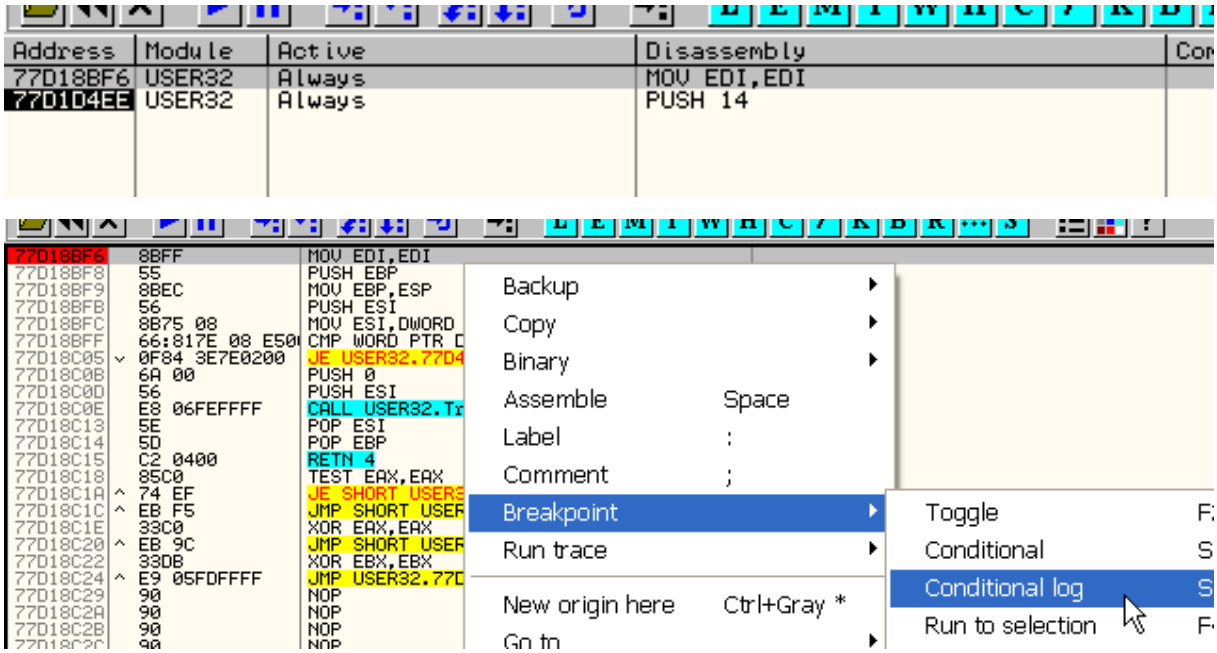
Чтобы побыстрее установить оба BPX, давайте воспользуемся плагином CommandBar: BP TranslateMessage, BP DefWindowProcA.



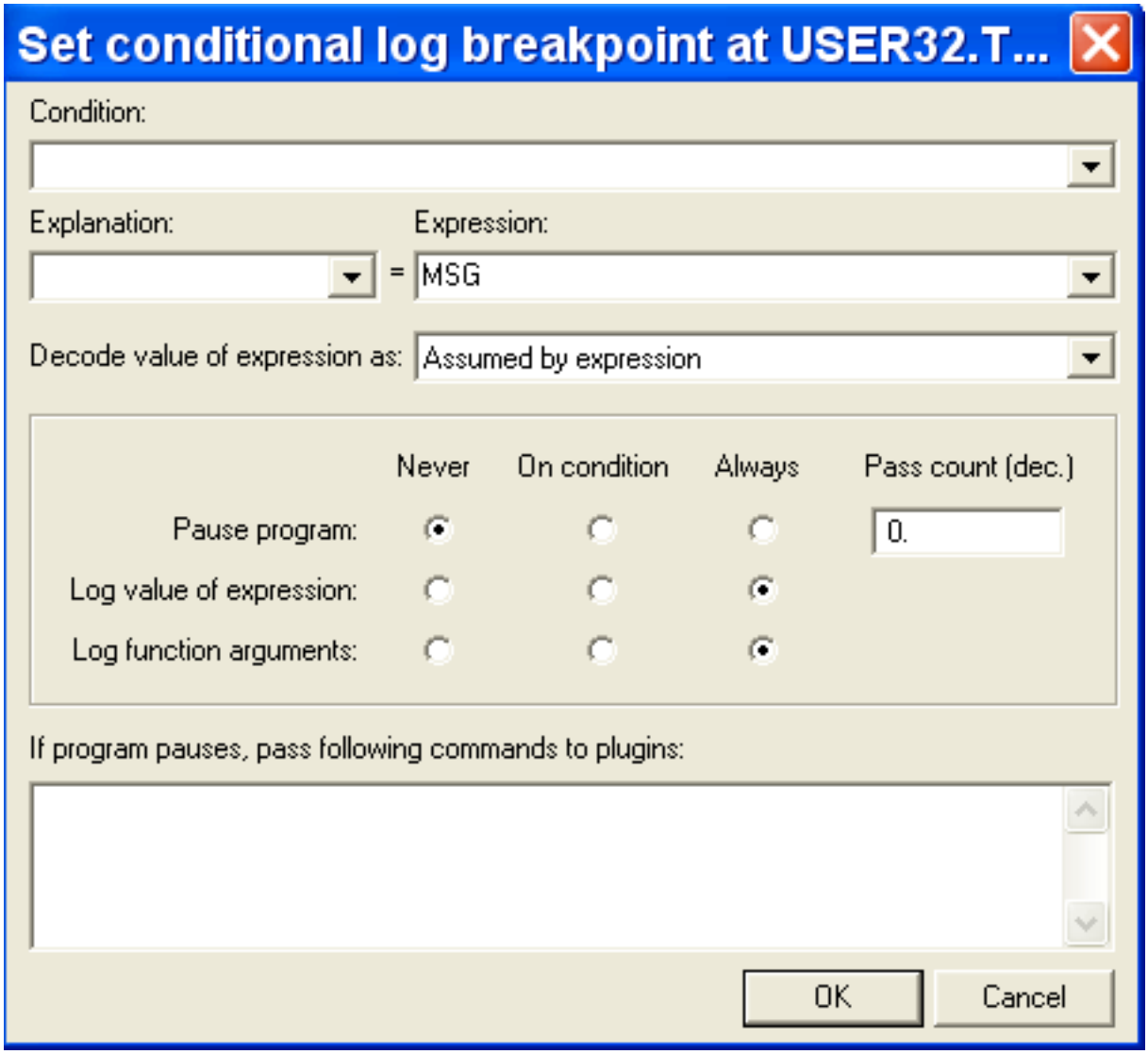
и



Таким образом мы установили 2 обыкновенных BPX на функции API. Давайте переделаем их в условные. Для этого открываем закладку [B], правый щелчёк по первому BPX - Follow in disassembler.

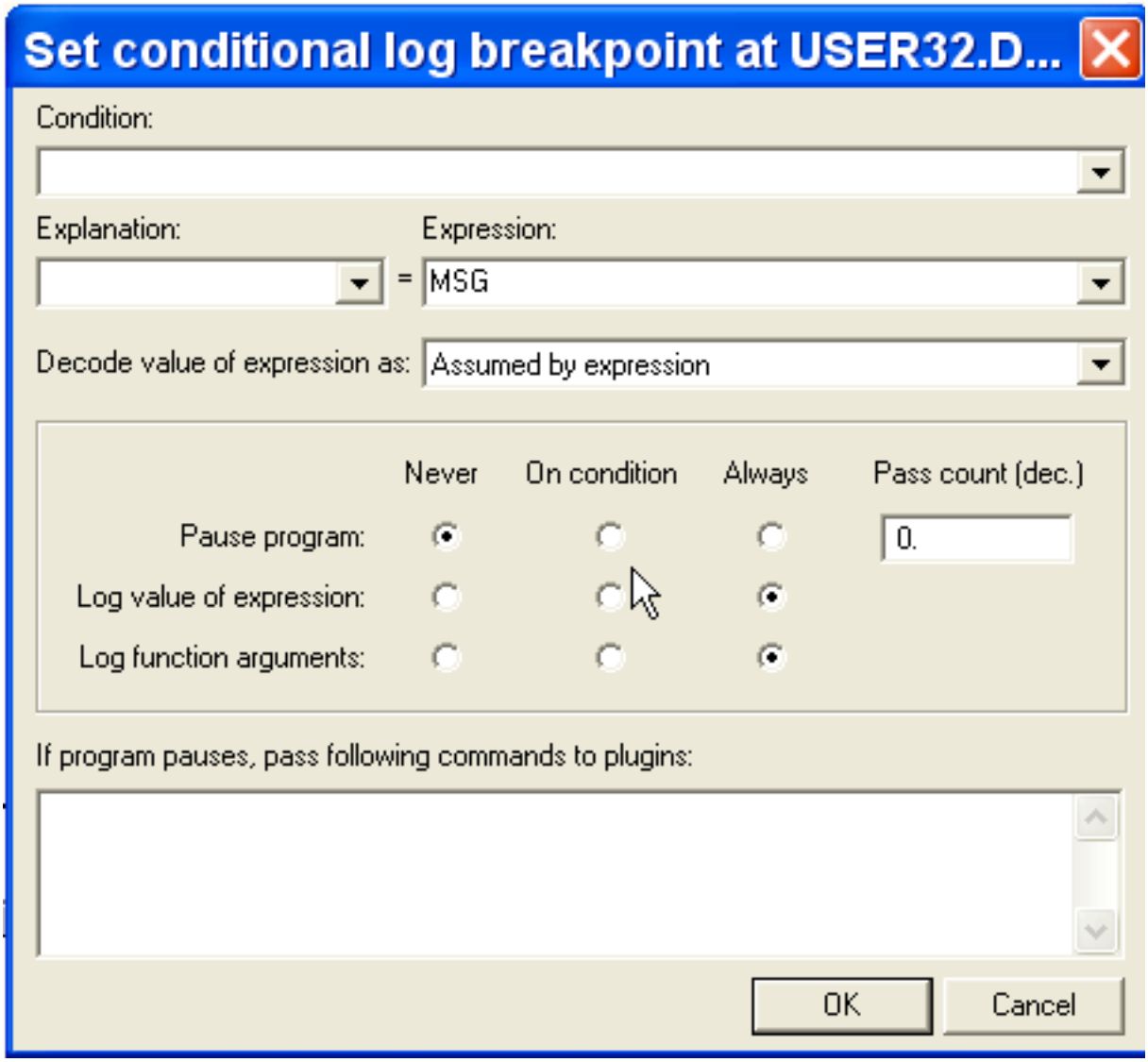


Уже "на месте" точки останова задаём Breakpoint - Conditional log.

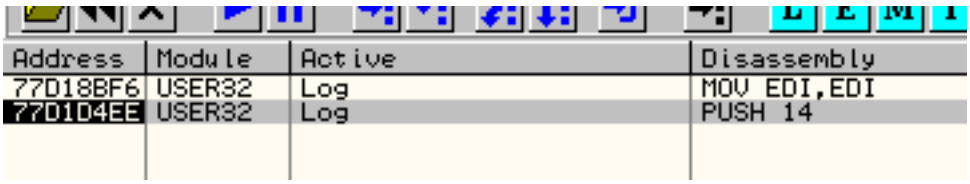


В качестве выражения, которое нужно записывать в лог, указываем MSG - это номер сообщения (например, для WM_LBUTTONUP, MSG = 202).

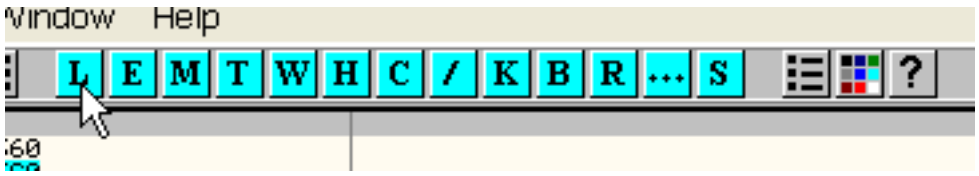
Потом в настройках указываем, что приложение не должно останавливаться и чтобы в лог всё время записывались как текущее выражение так и аргументы функции. Аналогичным образом перенастраиваем второй BPX.



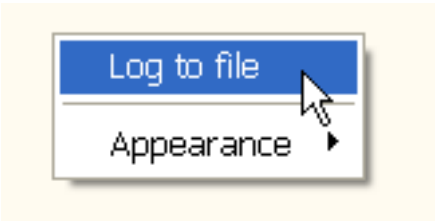
Итак, оба BPX уже превратились в условные.



Так-как результатов будет довольно много, лучше всего сохранять их сразу в файл.



Закладка [L] - правый щелчёк - Log to file.



Задаём имя текстового файла. Жмём Run.

```
          Message = WM_ENTERIDLE
          Source = MSGF_MENU
          hWnd = 031507F8 (class='#32768',parent=00140742)
77D1D4EE  COND: 00000121
77D1D4EE  CALL to DefWindowProcA from CRACKME.0040118C
          hWnd = 00140742 ('CrackMe v1.0',class='No need to disasm the code!')
          Message = WM_ENTERIDLE
          Source = MSGF_MENU
          hWnd = 031507F8 (class='#32768',parent=00140742)
77D1D4EE  COND: 00000121
77D1D4EE  CALL to DefWindowProcA from CRACKME.0040118C
          hWnd = 00140742 ('CrackMe v1.0',class='No need to disasm the code!')
          Message = WM_ENTERIDLE
          Source = MSGF_MENU
          hWnd = 031507F8 (class='#32768',parent=00140742)
77D1D4EE  COND: 00000085
77D1D4EE  CALL to DefWindowProcA from CRACKME.0040118C
          hWnd = 00140742 ('CrackMe v1.0',class='No need to disasm the code!')
          Message = WM_NCPAINT
          Region = E0041935 (region)
          lParam = 0
77D1D4EE  COND: 00000014
77D1D4EE  CALL to DefWindowProcA from CRACKME.0040118C
          hWnd = 00140742 ('CrackMe v1.0',class='No need to disasm the code!')
          Message = WM_ERASEBKGND
          hDC = 9401167D
          lParam = 0
77D1D4EE  COND: 00000125
77D1D4EE  CALL to DefWindowProcA from CRACKME.0040118C
          hWnd = 00140742 ('CrackMe v1.0',class='No need to disasm the code!')
          Message = WM_UNINITMENUPOPUP
          hMenu = 005406F4
          ID = 0
77D1D4EE  COND: 0000011F
77D1D4EE  CALL to DefWindowProcA from CRACKME.0040118C
          hWnd = 00140742 ('CrackMe v1.0',class='No need to disasm the code!')
          Message = WM_MENUSELECT
          Item = 0, Flags = MF_BYPOSITION|MF_SEPARATOR|3|MF_BITMAP|MF_OWNERDRAW|MF_POPUP|MF_ME
          hMenu = NULL
77D1D4EE  COND: 00000212
77D1D4EE  CALL to DefWindowProcA from CRACKME.0040118C
          hWnd = 00140742 ('CrackMe v1.0',class='No need to disasm the code!')
          Message = WM_EXITMENULOOP
          IsPopUp = FALSE
          lParam = 0
77D1D4EE  COND: 00000086
77D1D4EE  CALL to DefWindowProcA from CRACKME.0040118C
          hWnd = 00140742 ('CrackMe v1.0',class='No need to disasm the code!')
          Message = WM_NCACTIVATE
          Active = FALSE
          lParam = 0
77D1D4EE  COND: 00000006
77D1D4EE  CALL to DefWindowProcA from CRACKME.0040118C
          hWnd = 00140742 ('CrackMe v1.0',class='No need to disasm the code!')
          Message = WM_ACTIVATE
          WA_INACTIVE Minimized = 0
          hWnd = NULL
77D1D4EE  COND: 0000001C
77D1D4EE  CALL to DefWindowProcA from CRACKME.0040118C
          hWnd = 00140742 ('CrackMe v1.0',class='No need to disasm the code!')
          Message = WM_ACTIVATEAPP
          Activate = FALSE
          ThreadId = AC0
77D1D4EE  COND: 00000008
77D1D4EE  CALL to DefWindowProcA from CRACKME.0040118C
          hWnd = 00140742 ('CrackMe v1.0',class='No need to disasm the code!')
          Message = WM_KILLFOCUS
          hWndGet = NULL
          lParam = 0
77D18BF6  COND: 0000000F
77D18BF6  CALL to TranslateMessage from CRACKME.0040110C
          pMsg = WM_PAINT hw = 140742 ("CrackMe v1.0")
77D1D4EE  COND: 0000000F
77D1D4EE  CALL to DefWindowProcA from CRACKME.0040118C
          hWnd = 00140742 ('CrackMe v1.0',class='No need to disasm the code!')
          Message = WM_PAINT
```

Теперь в логе находятся все сообщения, полученные и обработанные оконной прецедурой приложения. С помощью этого лога можно составить представление о сообщениях, которые обрабатываются приложением и точнее подобрать кандидатов для последующих BMSG.

Владение различными техниками BMSG бывает очень кстати при снятии NAG'ов и других типичных для реверсинга задач - вот увидите!

В следующей главе мы наконец получим правильный серийник для крэкми CrueHead'a.

Введение в крэкинг с нуля, используя OllyDbg - Глава 13

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2007-11-09

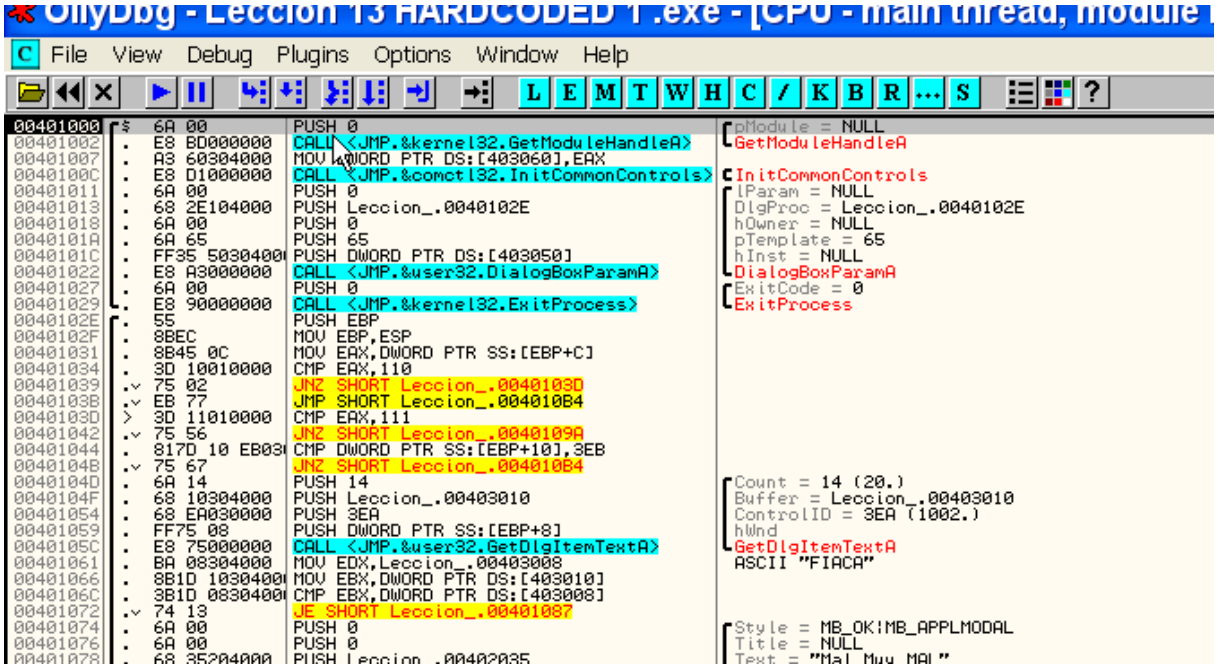
Файлы к статье (находится в архиве `wasm_files.zip`: `files/0464/ollydbg13.rar`)

Думаю, что одной из самых актуальных тем сегодняшнего дня является поиск серийных номеров, в первую очередь из-за существования сильных защит, призванных его усложнить, которые мы и рассмотрим шаг за шагом, начиная с самых простых и заканчивая более сложными, практикуясь в такой степени, чтобы закрепить навыки работы с серийными номерами.

Ок, эта часть будет посвящена работе с так называемыми HARDCODED-серийниками (жёстко заданными), которые не высчитываются на основе имени пользователя, то есть являющиеся текстом или числом, которые всегда одни и те же, и зачастую так просты, как никакие другие, и считаются наиболее лёгкими для поиска, поэтому мы и начнём с этого типа. Мы рассмотрим как совсем несложные, так и потруднее.

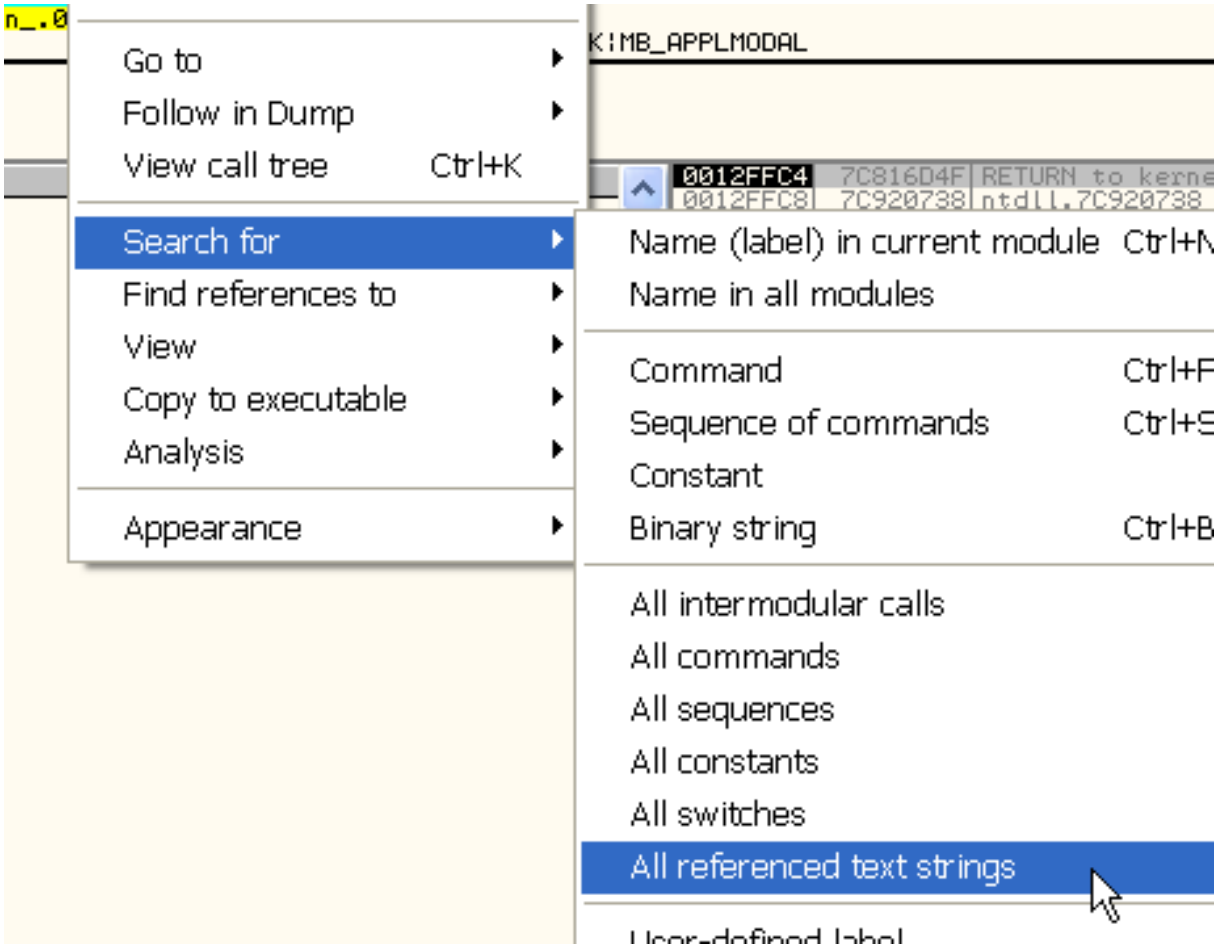
Для практики у нас есть четыре экземпляра, двое из которых приходятся на эту главу и два посложнее на 14-ую.

Первый из этих крэкми называется “LECCION 13 HARDCODED 1”, и он - самый простой. Откроем его в OllyDbg.

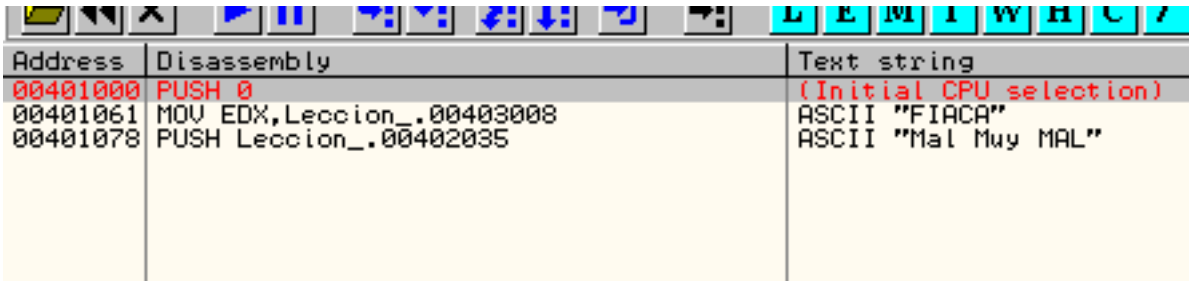


Здесь, как в самом простом случае с жёстко заданными серийными номерами правильный ответ находится в строках, используемых программой, которые мы можем посмотреть.

Чтобы найти строки, используемые программой, кликнем на правой кнопке мыши и выберем SEARCH FOR - ALL REFERENCED TEXT STRINGS.



Результаты будут следующие:

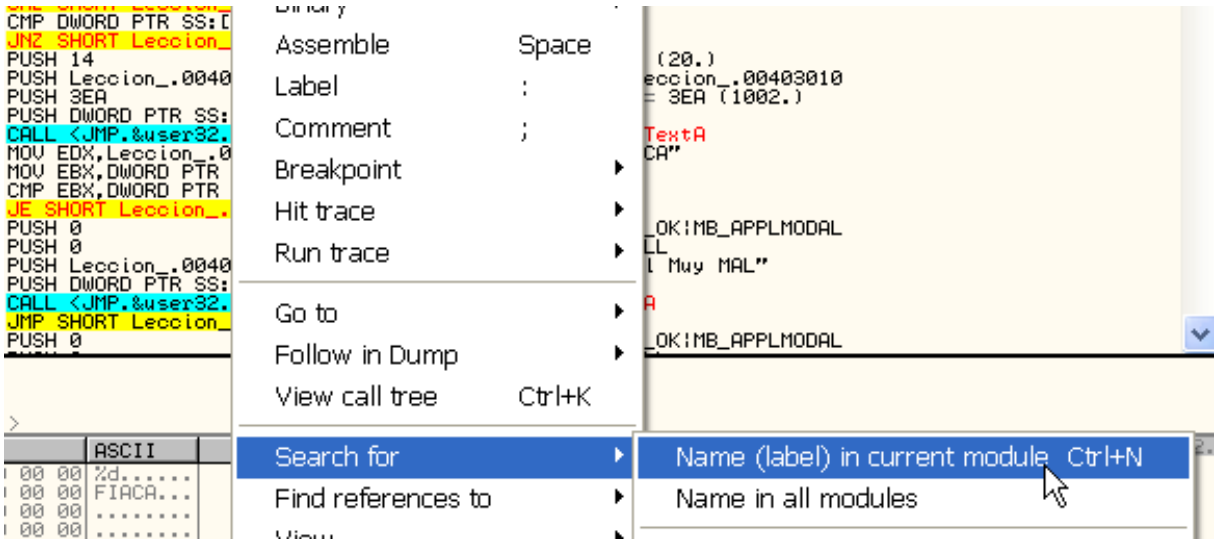


Здесь мы видим слово «FIACA», которое может быть ключом, и если мы не хотим работать, то можем просто перебирать встречающиеся в этом списке строки, чтобы найти верный, но этот метод не рекомендуется по следующим причинам:

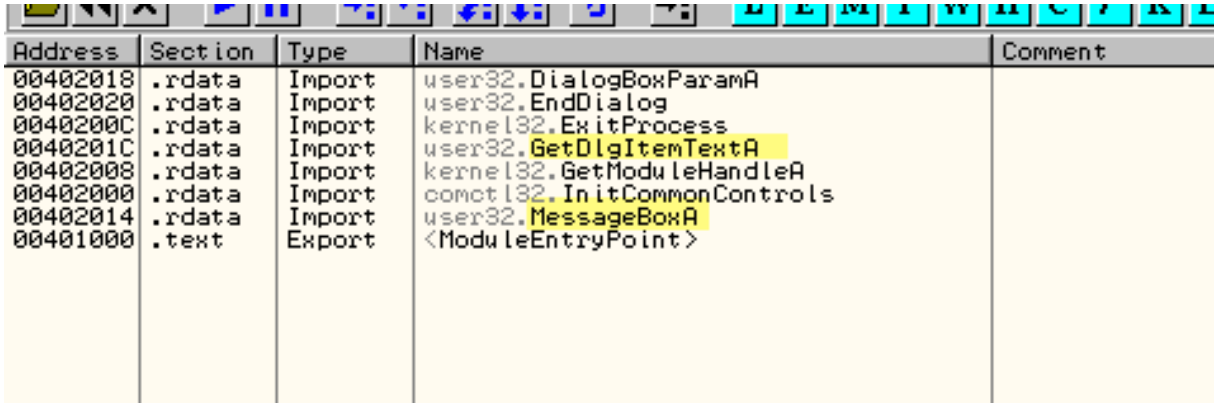
В данном случае строк всего две, но есть программы, в которых тысячи строк и пробовать их одна за одной просто безумие, и хотя в данном случае мы сразу видим возможный правильный серийный номер, если у нас их будет тысяча, то это будет маловерятно.

Есть программы, у которых в этом списке есть строки, не являющиеся серийным номером, но выглядящие специально на него похожими и являющееся ловушками для крэкеров, поэтому гораздо лучше убедиться и проверить, что серийный номер действительно является верным.

Сначала посмотрим, какие API-функции используются, с помощью клика правой кнопкой мыши и выбора SEARCH FOR- NAME (LABEL) IN CURRENT MODULE.

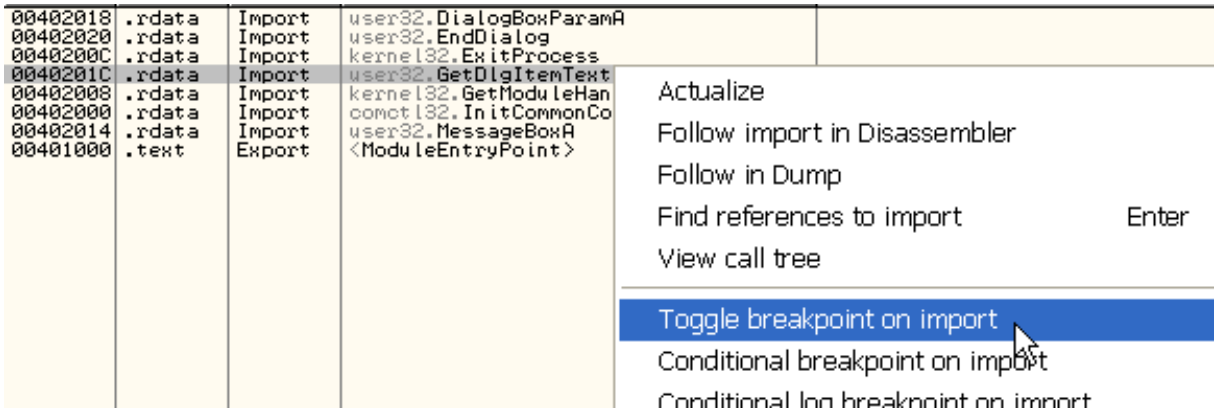


Выводятся API-функции, используемые программой, и видим, что среди них есть нам известные.

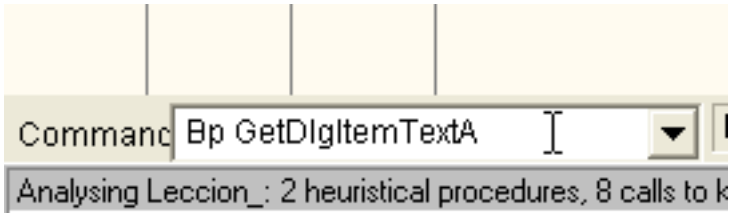


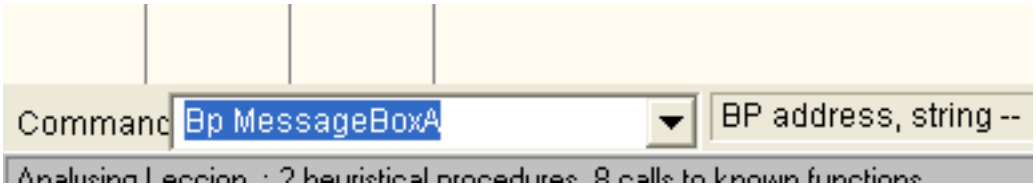
Address	Section	Type	Name	Comment
00402018	.rdata	Import	user32.DialogBoxParamA	
00402020	.rdata	Import	user32.EndDialog	
0040200C	.rdata	Import	kernel32.ExitProcess	
0040201C	.rdata	Import	user32.GetDlgItemTextA	
00402008	.rdata	Import	kernel32.GetModuleHandleA	
00402000	.rdata	Import	comctl32.InitCommonControls	
00402014	.rdata	Import	user32.MessageBoxA	
00401000	.text	Export	<ModuleEntryPoint>	

Видим, что используется GetDlgItemTextA для получения серийного номера, введённого пользователем, и MessageBoxA для вывода сообщений, является ли серийным номер верным или нет. Установим точку останова на каждую из этих API-функций.

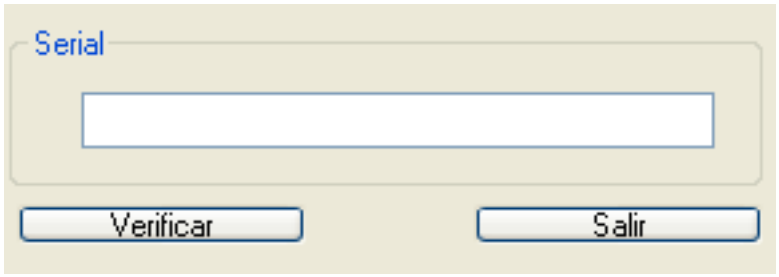


Кликаем на обоих правой кнопкой мыши - TOGGLE BREAKPOINT ON IMPORT или то же самое с помощью commandbar'a.

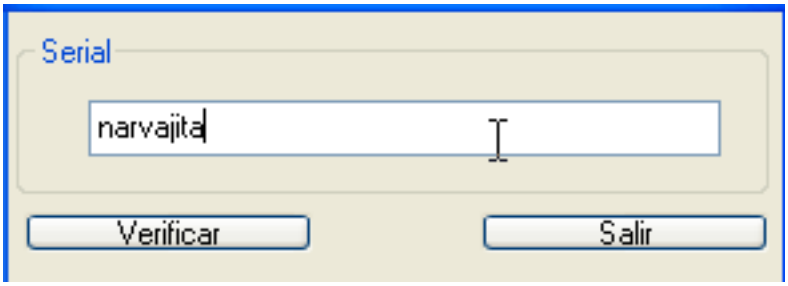




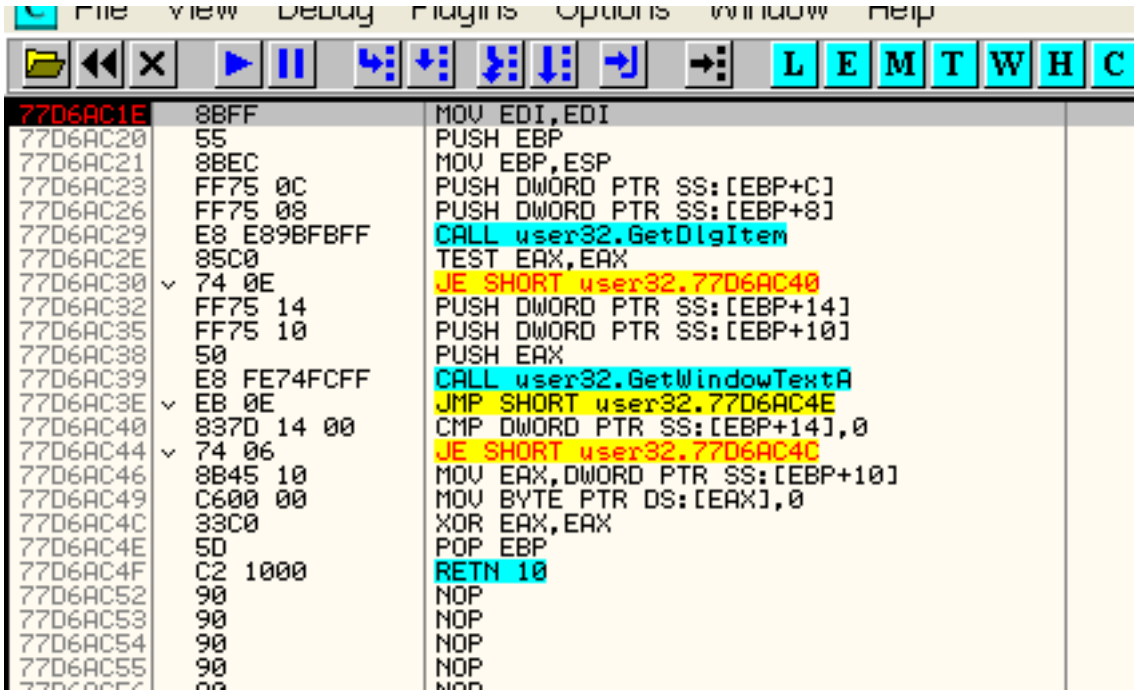
Хорошо, нажимаем F9, чтобы запустить крэкми.



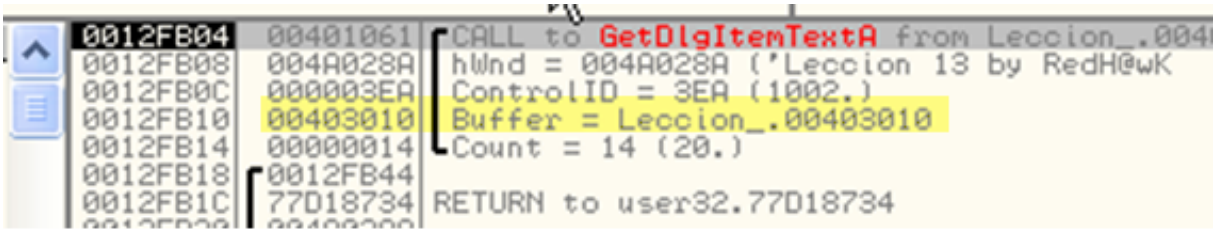
Появляется окошко для ввода серийного номера, давайте введём что-нибудь, например «narvajita», хе-хе (фамилия автора «Narvaja» - прим.пер.).



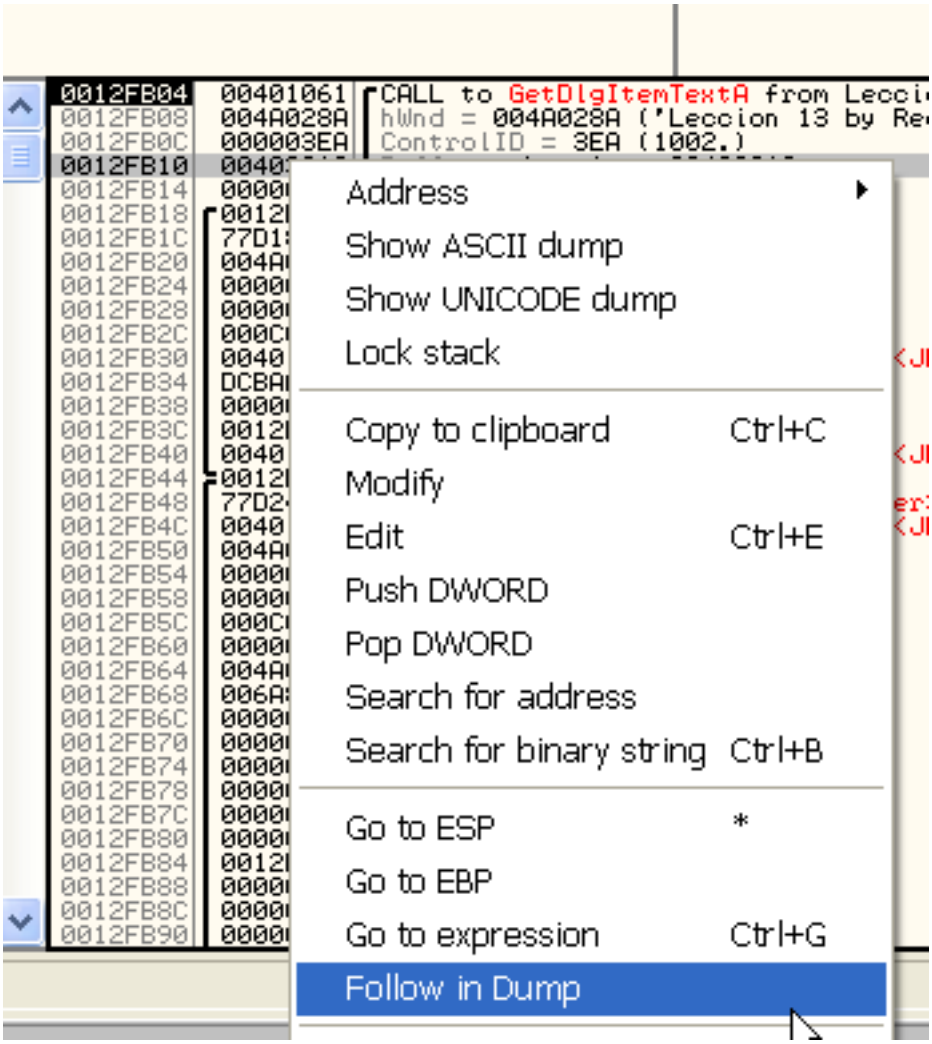
Нажимаем на «Verificar» (проверить) и видим, что сработал BPX:



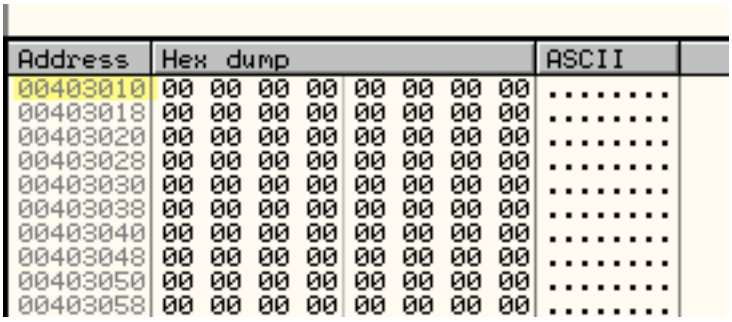
Если посмотрим стек, то увидим, что остановка произошла на GetDlgItemTextA, получающая введённый пользователем серийный номер, а в параметрах функции находится адрес буфера, куда он сохраняется, в данном случае - 403010.



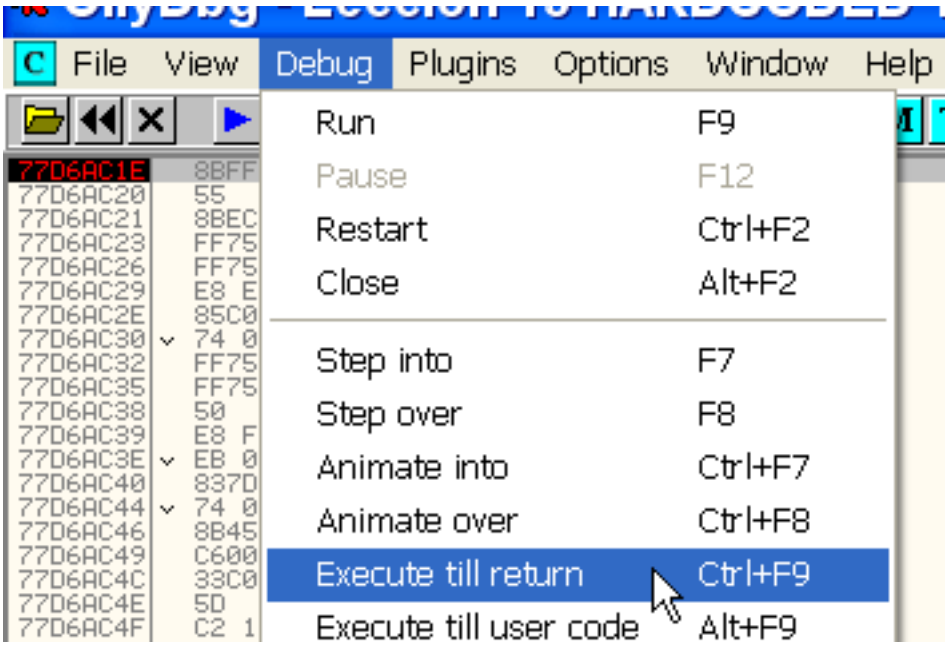
Посмотрим, что находится по этому адресу через DUMP. Правая кнопка мыши - FOLLOW IN DUMP.



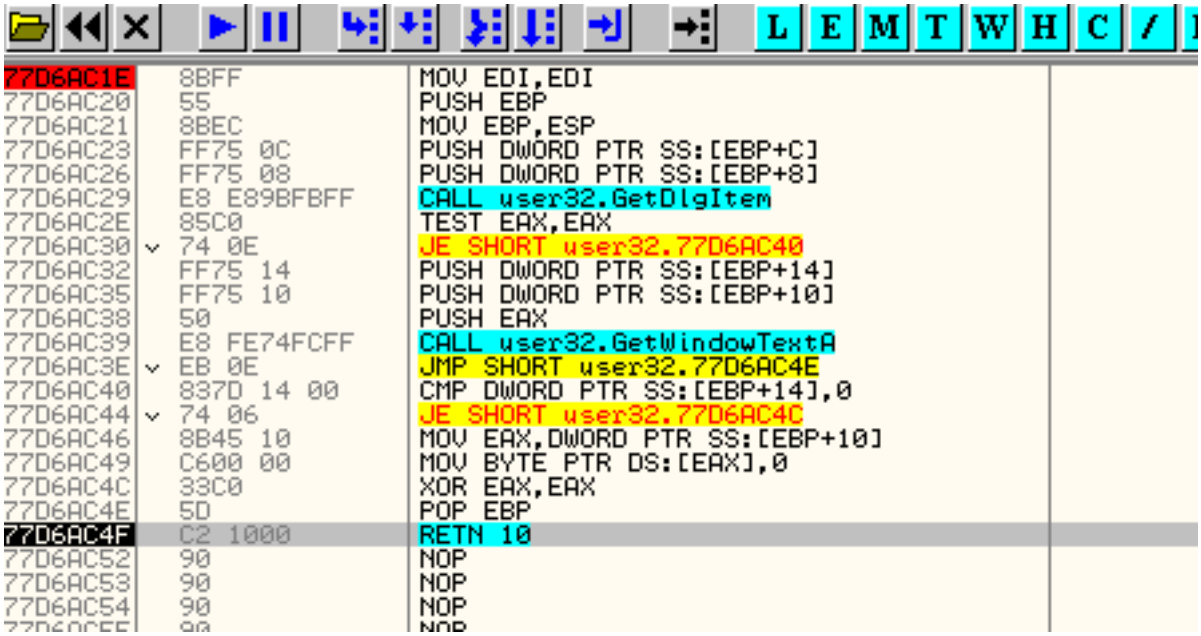
Видим буфер, но в нём ничего нет, так как API-функция ещё не выполнялась.



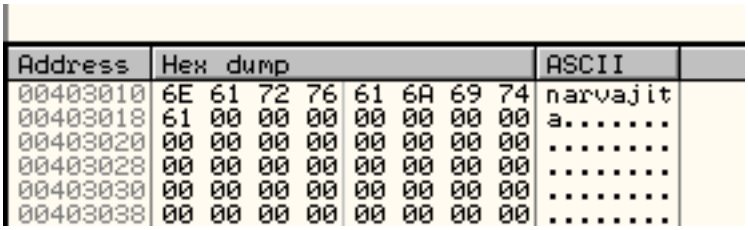
Выберем EXECUTE TILL RETURN, чтобы она выполнилась до инструкции RET.



Стоим на RET'е.

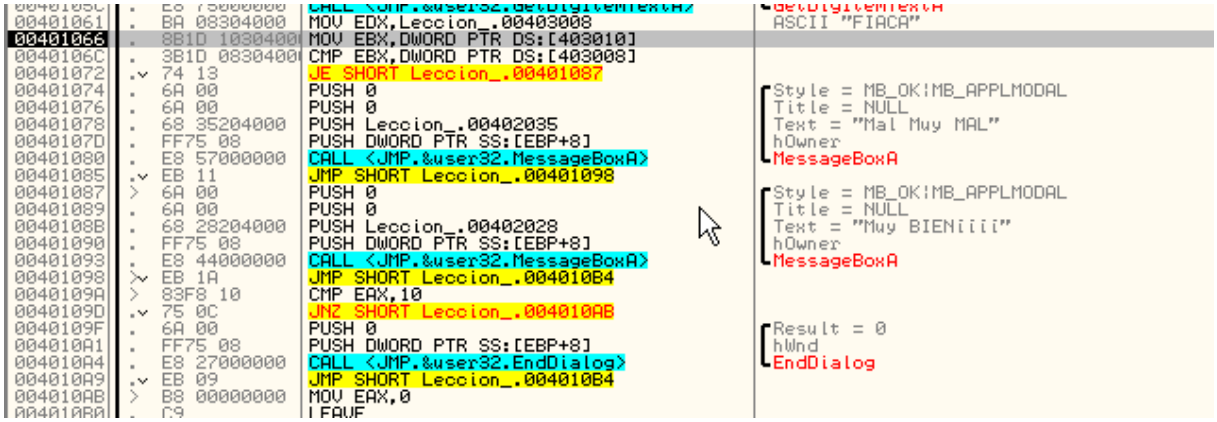


А в буфере - то, что мы ввели.



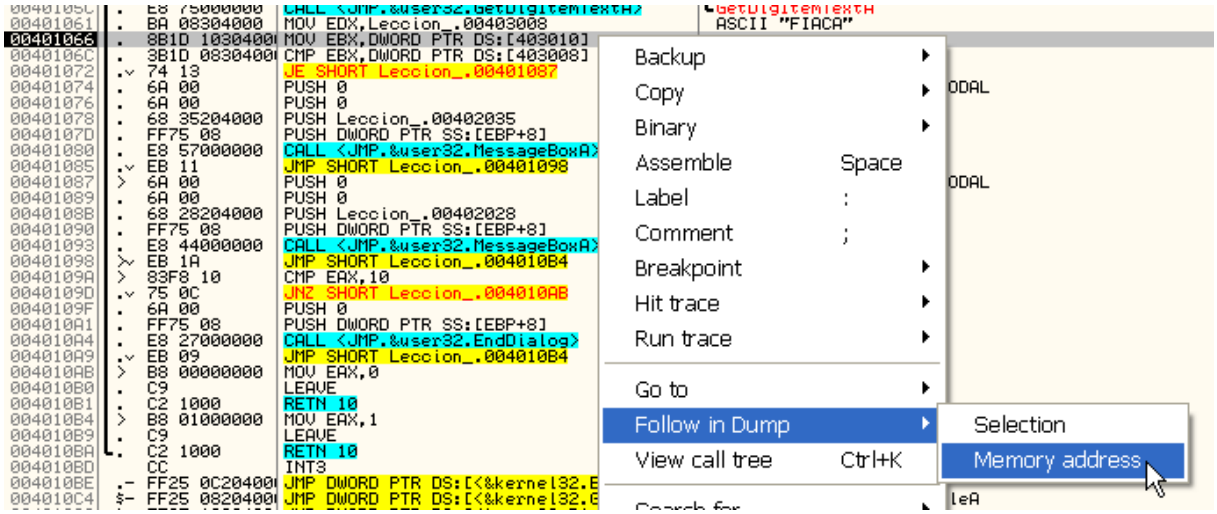
С этим понятно.

Если нажмём F7 и вернёмся в программу.

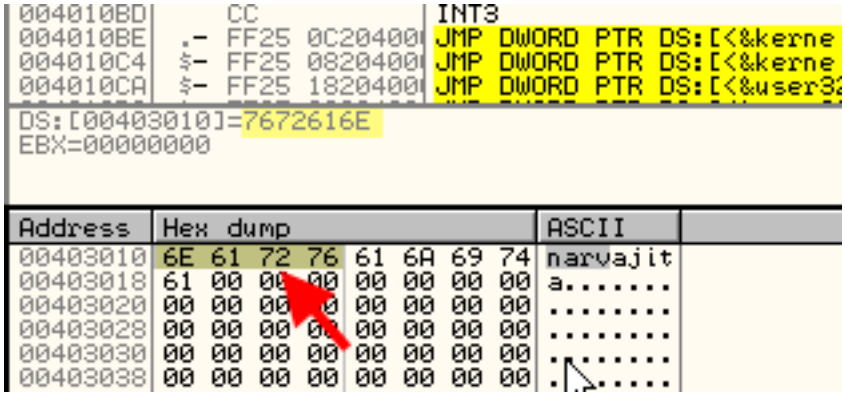


Увидим, что начинается сравнение и условный переход, который переходит либо на MessageBoxA, показывающий надпись “Mal Muy MAL” («плохо, очень плохо» - прим. пер.), либо на MessageBoxA, показывающий надпись “Muy BIEN” («очень хорошо» - прим.пер.) соответственно. Очевидно, что если мы исправим этот переход на JMP 401087, то будет всегда выполняться переход на вывод сообщения “MUY BIEN”, но в данном случае я хочу найти серийный номер, поэтому будем смотреть, что сравнивается.

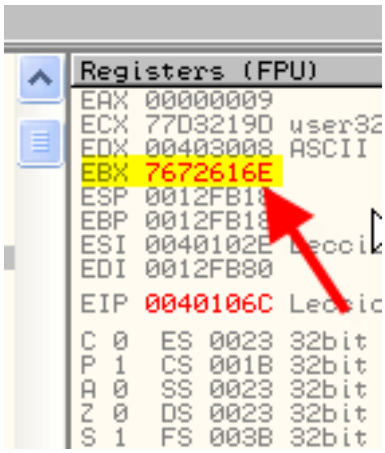
В 401066 в EBX перемещается содержимое памяти по адресу 403010, которое является DWORD’ом. Посмотрим, что находится по адресу 403010 через DUMP.



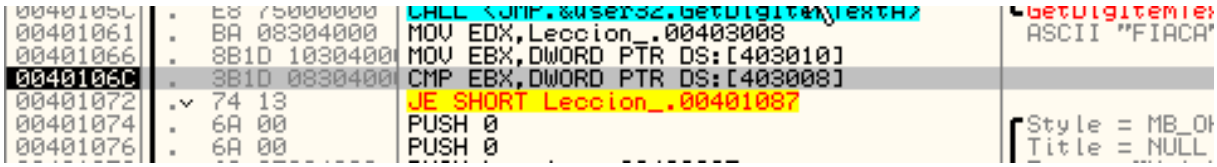
Отметим нужную строку, через правую кнопку мыши выберем FOLLOW IN DUMP-MEMORY ADDRESS и увидим, что там находится.



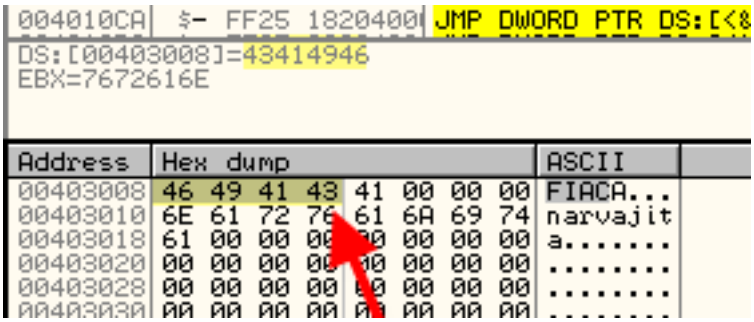
В пояснении видим, что там находится 7672616E, являющееся перевёрнутыми 4-мя байтами содержимого 403010, и это первые 4 байта неправильного серийного номера, который мы ввели. Они-то и были перемещены в EBX.



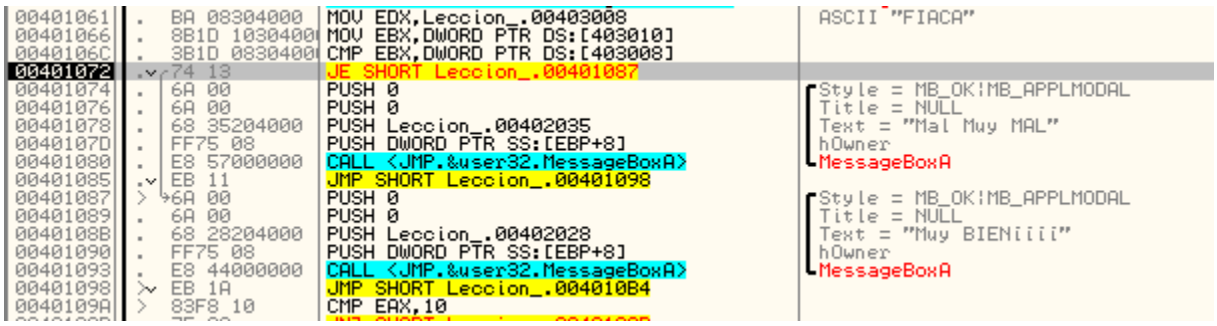
Нажимаем F7 и переходим к следующей строке.



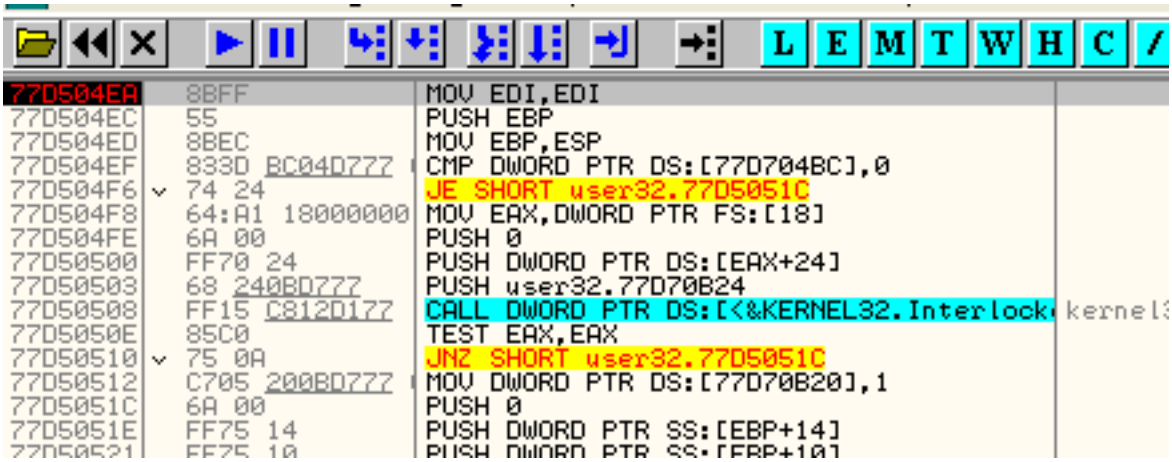
Здесь видим, что EBX, содержащий первые 4 байта неправильного серийного номера, сравнивается с содержимым ячейк памяти по адресу 403008. Посмотрим через DUMP, что там находится, так же как мы делали это в прошлый раз.



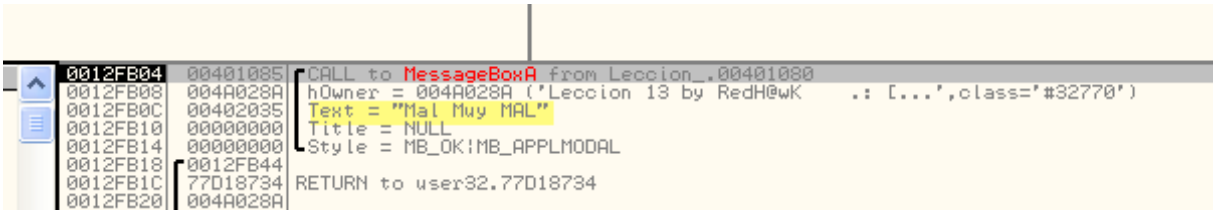
Видим, что там находятся первые четыре байта слова “FIACA”, которые и сравниваются с первыми четыремя байтами введённого серийного номера, из чего мы можем заключить, что если бы они были одинаковы, то активировался бы флаг Z, так как между ними не было было разницы, выполнился бы переход JE и вывелось бы сообщение “MUY BIEN”, но так как они не одинаковы, то нас ждёт суровое наказание, хе-хе.



Раз они не одинаков, то перехода не происходит и выполнение идёт к окошку “MAL MUY MAL”. Нажимаем F9.

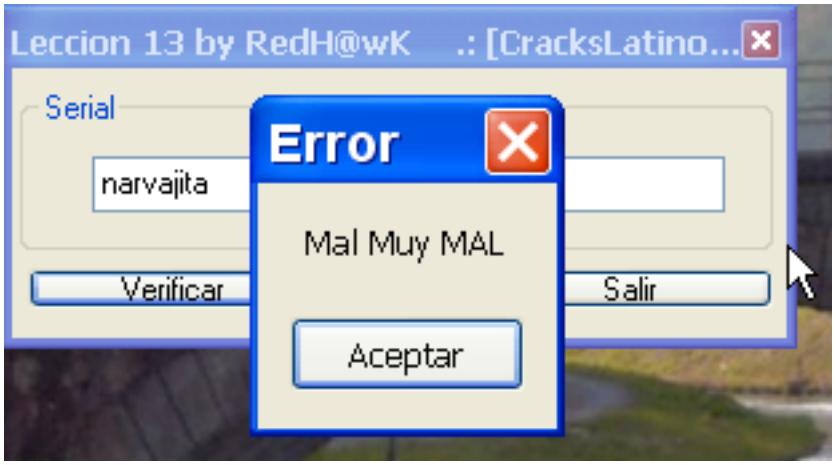


Мы стоим на другом вызове MessageBoxA.

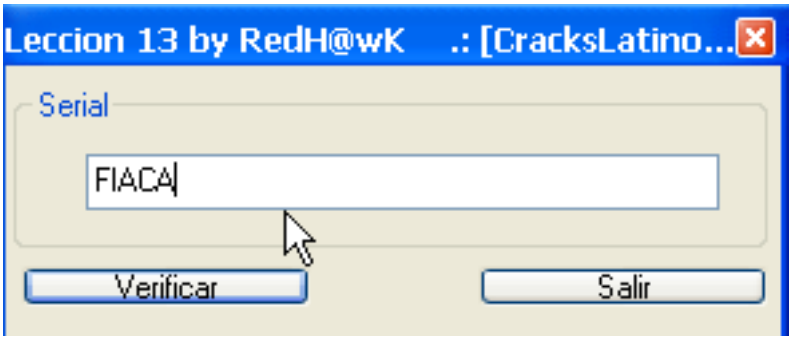


И уже в параметрах функции видим, что она нам покажет.

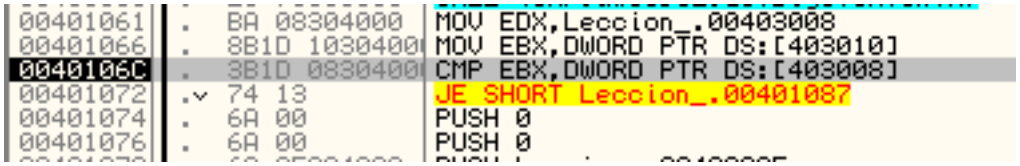
Продолжаем выполнение программы.



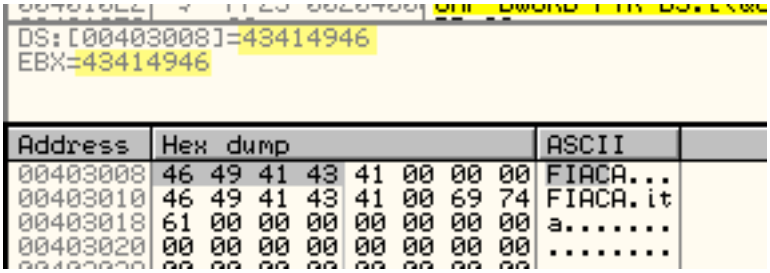
Как видим, было показано окошко с сообщением, что мы ошиблись, жмём на “OK” и вводим правильный код “FIACA”.



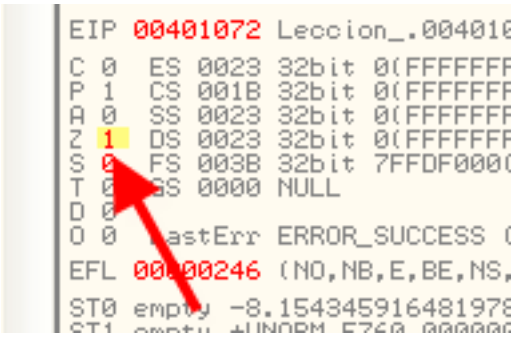
Нажимаем на “Verificar” и повторяем процесс, пока не доходим до сравнения.



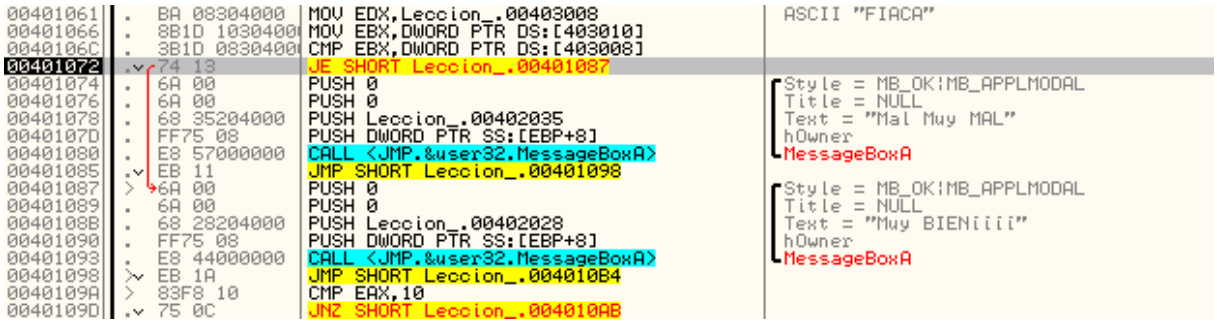
Как и в прошлый раз, содержимое EBX сравненивается с содержимым 403008.



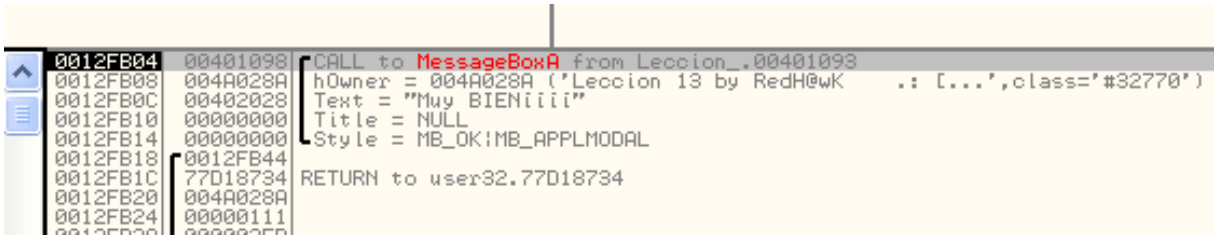
В пояснении видим, что они оба одинаковы, поэтому в результате их вычитания друг от друга получится ноль и активируется флаг Z. Выполним сравнение с помощью F7.



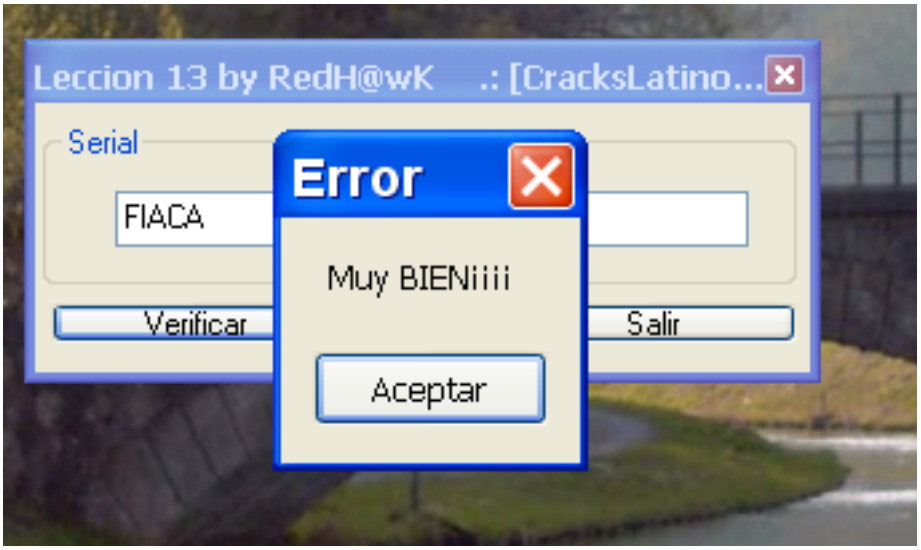
Видим, что всё так и есть, поэтому выполнится JE.



Видим, что переход произойдёт на MessageBoxA с MUY BIEN. Посмотрим с помощью F9.



Здесь видим стек API-функции, которая отобразит нам правильное окошко.

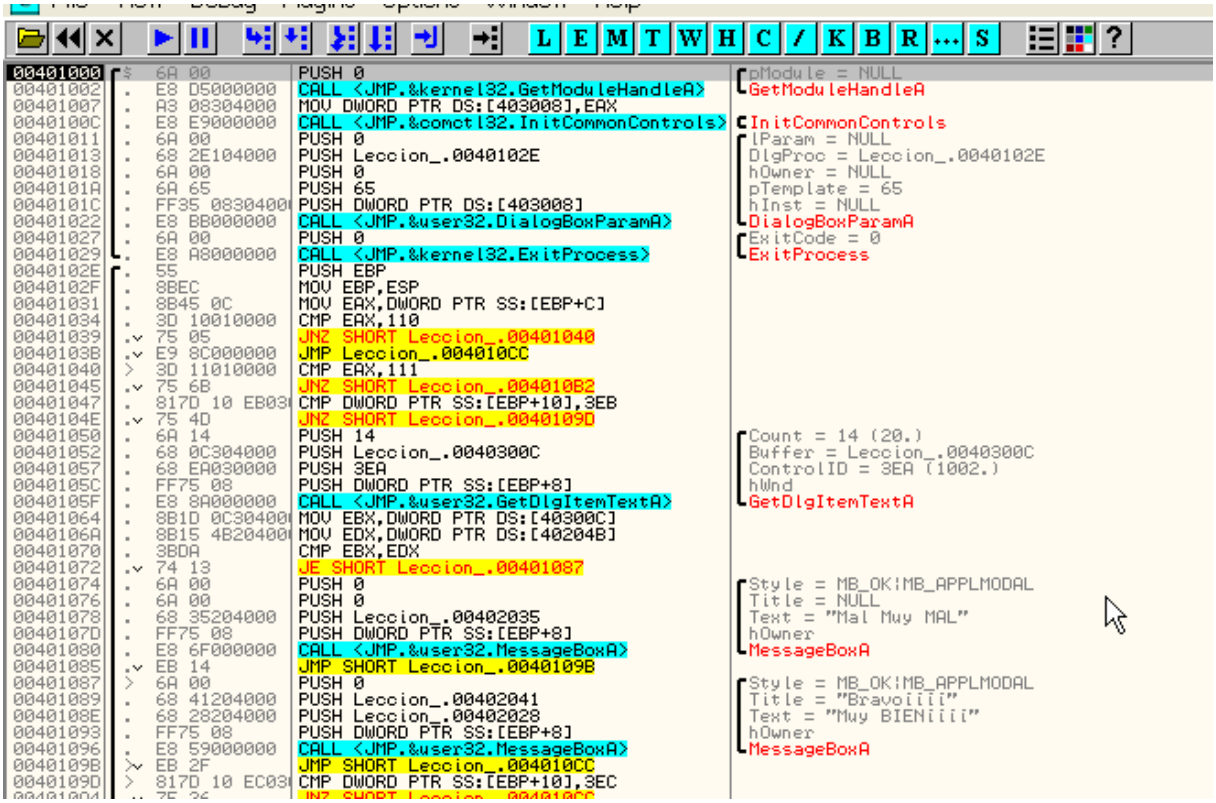


Хотя создатель крэкми очевидно забыл поменять название окошка, это не важно, так как мы нашли правильный серийный номер, который был жёстко и очень просто задан.

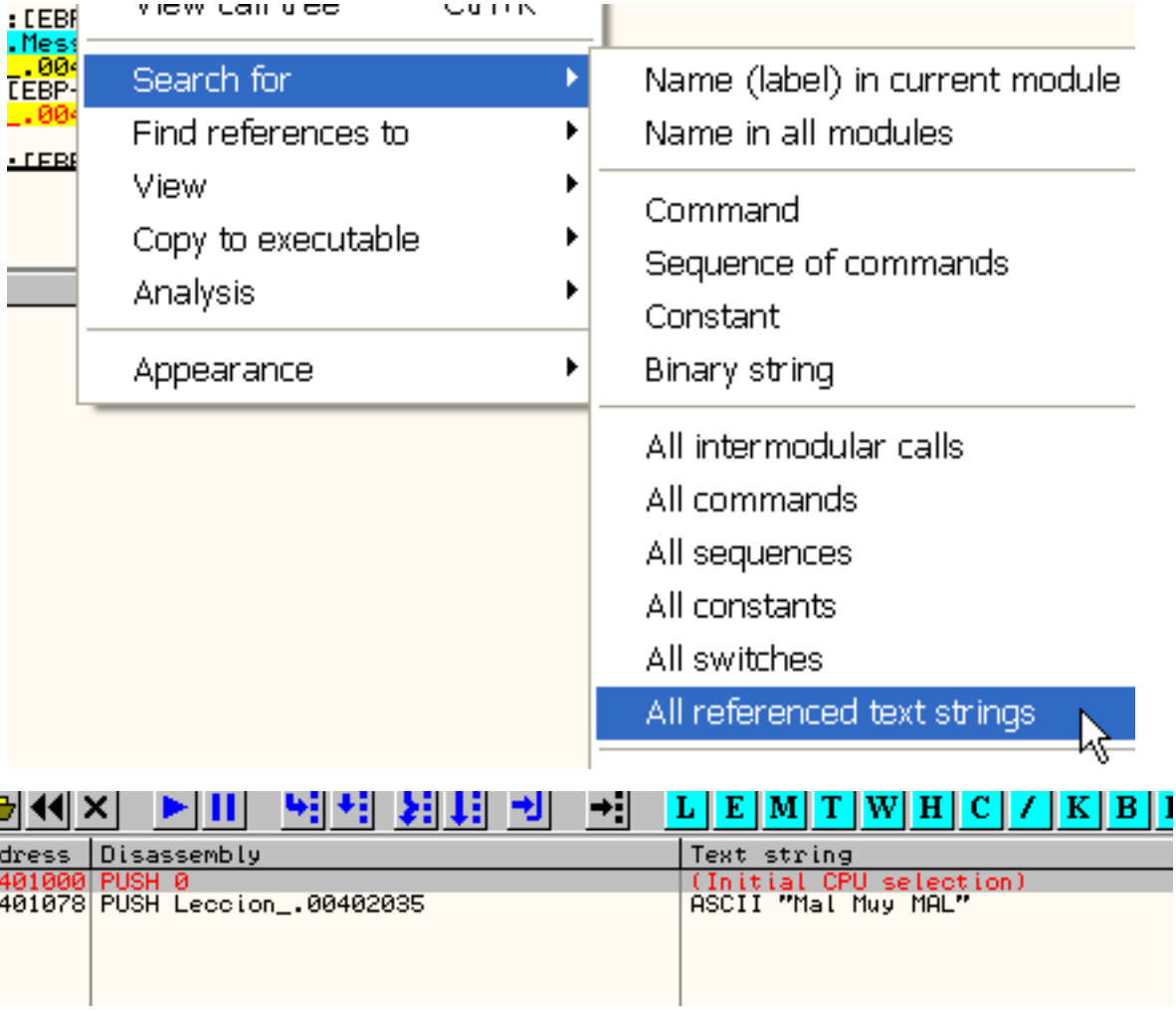
Следующий крэкми, который мне предоставил мой друг REDHAWK, похож на предыдущий, но с небольшими изменениями (это он был так ленив, что не изменил заголовок окна на правильный, ха ха ха).

Abramos el LECCION 13 HARDCODED 2 en el OLLYDBG

Открываем «LECCION 13 HARDCODED 2» в OllyDbg.



Как видим, всё очень похоже, но в используемых строках нет правильного кода.



Нет ни “FIACA”, ни чего-либо другого похожего, хехе.

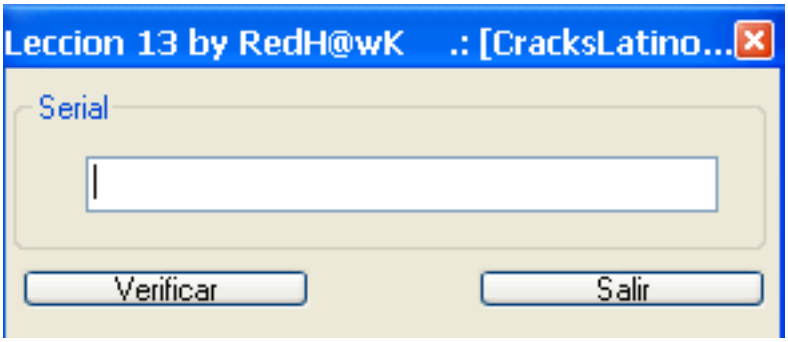
Да и верный код вовсе не “FIACA”, хехе.

Раз уже знаем, что здесь примерно происходит, идём сразу к сравнению.

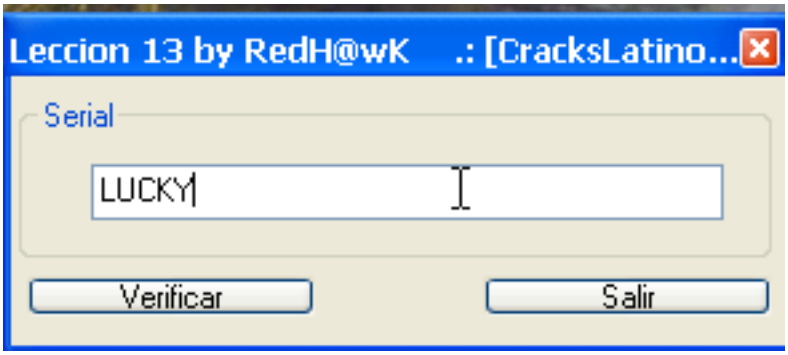
```
0040105F   .  E8 8A000000   CALL <JMP.&user32.GetDlgItemTextA>          GetDlgItemTextA
00401064   .  8B1D 0C30400( MOV EBX,DWORD PTR DS:[40300C]
0040106A   .  8B15 4B20400( MOV EDX,DWORD PTR DS:[40204B]
00401070   .  3BDA          CMP EBX,EDX
00401072   .v 74 13         JE SHORT Leccion_.00401087
00401074   .  6A 00         PUSH 0                                      Style = MB_OK|MB_APPLMODAL
00401076   .  6A 00         PUSH 0                                      Title = NULL
00401078   .  68 35204000   PUSH Leccion_.00402035                      Text = "Mal Muy MAL"
0040107D   .  FF75 08       PUSH DWORD PTR SS:[EBP+8]                   hOwner
00401080   .  E8 6F000000   CALL <JMP.&user32.MessageBoxA>              MessageBoxA
00401085   .v EB 14         JMP SHORT Leccion_.0040109B
00401087   >  6A 00         PUSH 0                                      Style = MB_OK|MB_APPLMODAL
00401089   .  68 41204000   PUSH Leccion_.00402041                      Title = "Bravoiiii"
0040108E   .  68 28204000   PUSH Leccion_.00402028                      Text = "Muy BIENiiii"
00401093   .  FF75 08       PUSH DWORD PTR SS:[EBP+8]                   hOwner
00401096   .  E8 59000000   CALL <JMP.&user32.MessageBoxA>              MessageBoxA
0040109B   >v EB 2F         JMP SHORT Leccion_.004010CC
0040109D   >  817D 10 EC03( CMP DWORD PTR SS:[EBP+10],3EC
```

Устанавливаем на 401064 BPX и делаем RUN.

```
00401057  .   68 EA030000    PUSH 3EA                          ControlID = 3EA (1002.)
0040105C  .   FF75 08        PUSH DWORD PTR SS:[EBP+8]         hWnd
0040105F  .   E8 8A000000    CALL <JMP.&user32.GetDlgItemTextA> GetDlgItemTextA
00401064  .   8B1D 0C30400(  MOV EBX,DWORD PTR DS:[40300C]
0040106A  .   8B15 4B20400(  MOV EDX,DWORD PTR DS:[40204B]
00401070  .   3BDA           CMP EBX,EDX
00401072  .v  74 13          JE SHORT Leccion_.00401087
00401074  .   6A 00          PUSH 0                            Style = MB_OK|MB_APPLMODAL
00401076  .   6A 00          PUSH 0                            Title = NULL
00401078  .   68 35204000    PUSH Leccion_.00402035            Text = "Mal Muy MAL"
0040107D  .   FF75 08        PUSH DWORD PTR SS:[EBP+8]         hOwner
00401080  .   E8 6F000000    CALL <JMP.&user32.MessageBoxA>    MessageBoxA
00401085  .v  EB 14          JMP SHORT Leccion_.0040109B
00401087  >   6A 00          PUSH 0                            Style = MB_OK|MB_APPLMODAL
00401089  .   68 41204000    PUSH Leccion_.00402041            Title = "Bravoiii"
0040108E  .   68 28204000    PUSH Leccion_.00402028            Text = "Muy BIENiiii"
00401093  .   FF75 08        PUSH DWORD PTR SS:[EBP+8]         hOwner
00401096  .   E8 59000000    CALL <JMP.&user32.MessageBoxA>    MessageBoxA
0040109B  >v  EB 2F          JMP SHORT Leccion_.004010CC
0040109D  >   817D 10 EC03(  CMP DWORD PTR SS:[EBP+10],3EC
```



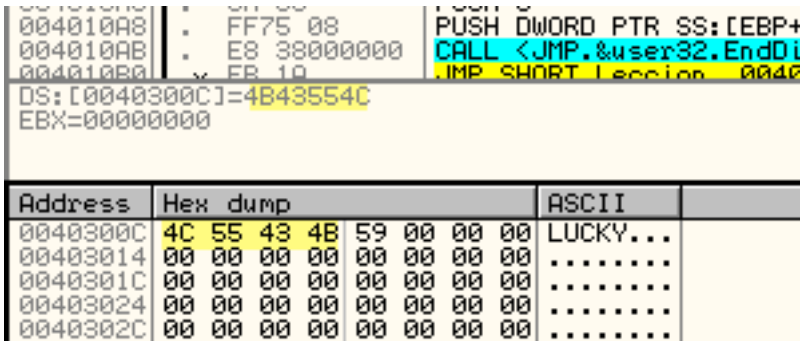
Введём неправильный серийный номер (в данном случае LUCKY).



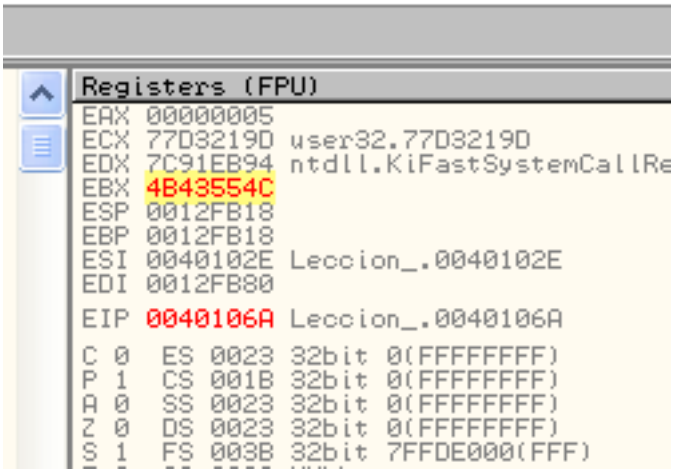
Нажимаем на “Verificar”.

```
0040105F  .   E8 8A000000    CALL <JMP.&user32.GetDlgItemTextA> GetDlgItemTextA
00401064  .   8B1D 0C30400(  MOV EBX,DWORD PTR DS:[40300C]
0040106A  .   8B15 4B20400(  MOV EDX,DWORD PTR DS:[40204B]
00401070  .   3BDA           CMP EBX,EDX
00401072  .v  74 13          JE SHORT Leccion_.00401087
00401074  .   6A 00          PUSH 0                            Style = MB_OK|MB_APPLMODAL
00401076  .   6A 00          PUSH 0                            Title = NULL
00401078  .   68 35204000    PUSH Leccion_.00402035            Text = "Mal Muy MAL"
```

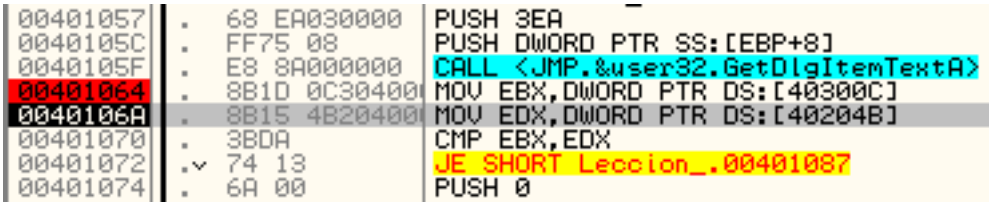
И при срабатывании BPX видим, что в EBX помещается содержимое памяти по адресу 40300C. Смотрим через DUMP.



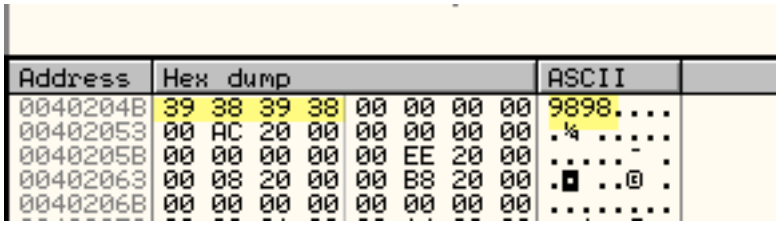
Попсле выполнения этой строки в EBX перемещаются 4 байта, которые меняют порядок при перемещении в регистр.



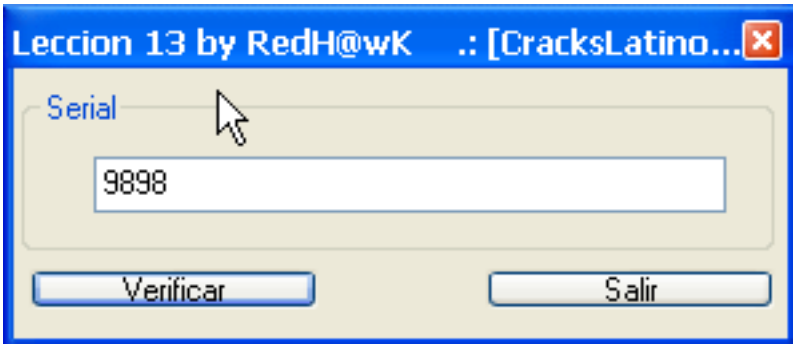
Нажимаем на F7 и оказываемся на EBX. MOV EBX, DWORD PTR DS: [40300C] перемещает только четыре байта.



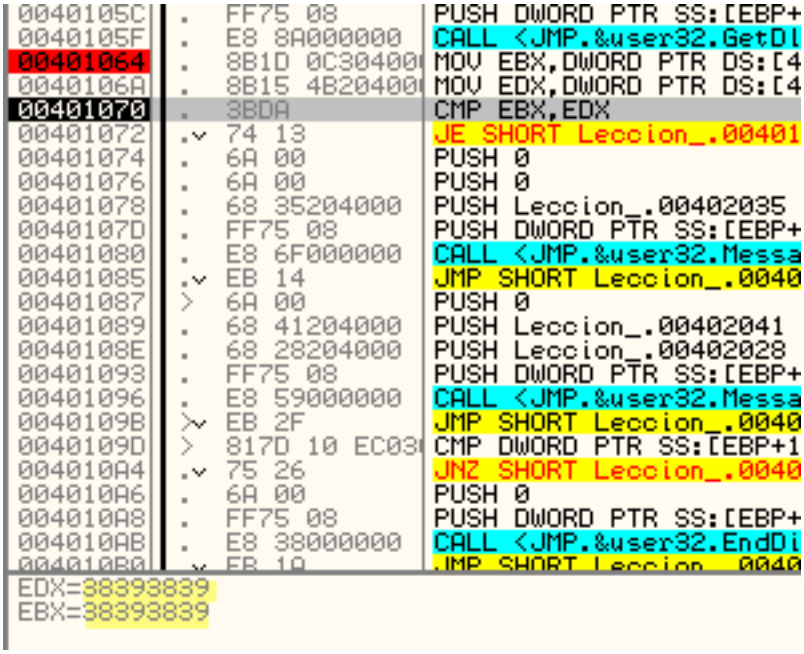
В этом примере, который почти ничем не отличается, в EDX помещается содержимое 40204B, смотрим, что там.



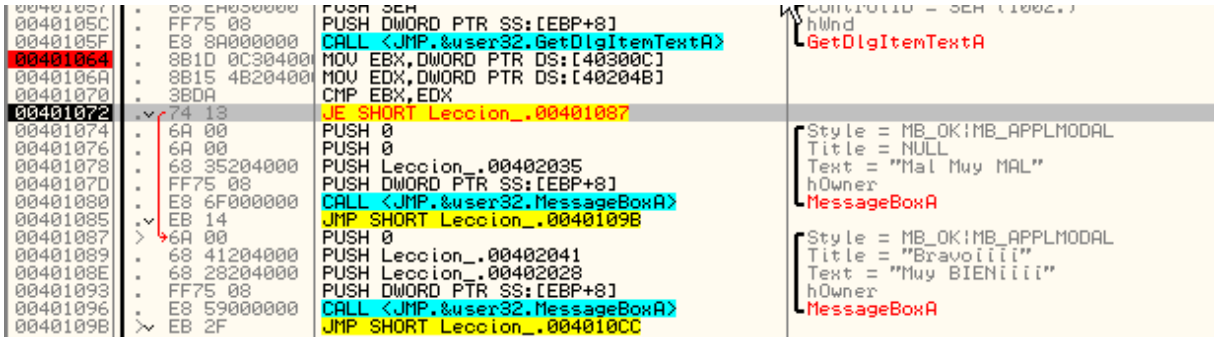
Там находятся четыре байта, соответствующие строке 9898 и являющиеся правильным серийным номер. Они сравниваются с четырьмя байтами введённого нами серийного номера, и если они одинаковы, то совершается переход в область MUY BIEN, но в данном случае этого не происходит, поэтому нам нужно вернуться и ввести 9898.



Нажимаем “Verificar”.



При сравнении EBX с EDX оба равны 38393839, что соответствует строке 9898, поэтому совершается переход на вывод окна MUY BIEN.



Видим, что в этом крэкми мой друг сделал заголовок окна “BRAVO”, ха-ха! Но главное, что мы выяснили правильный серийный номер - 9898.

Далее мы увеличим сложность - у нас осталось два более трудных жёстко заданных серийных номера, на которых мы рассмотрим техники, применямые в более запутаннх случаях.

Если вы думаете, что я один буду работать, а вы нет, то вот вам третий крэкми (mielecrackme1.zip).

В качестве подсказки скажу, что API-функция lstrcmpA, встречающаяся в этом крэкми, используется напрямую для сравнивания строк. Когда встретите её, посмотрите на параметры в стеке.

В главе 14 рассмотрим два более сложных примера жёстко заданных серийных номеров, но перед этим нужно потренироваться на более простых примерах, так что попрактикуйтесь на третьем крэкми, который я вам дал.

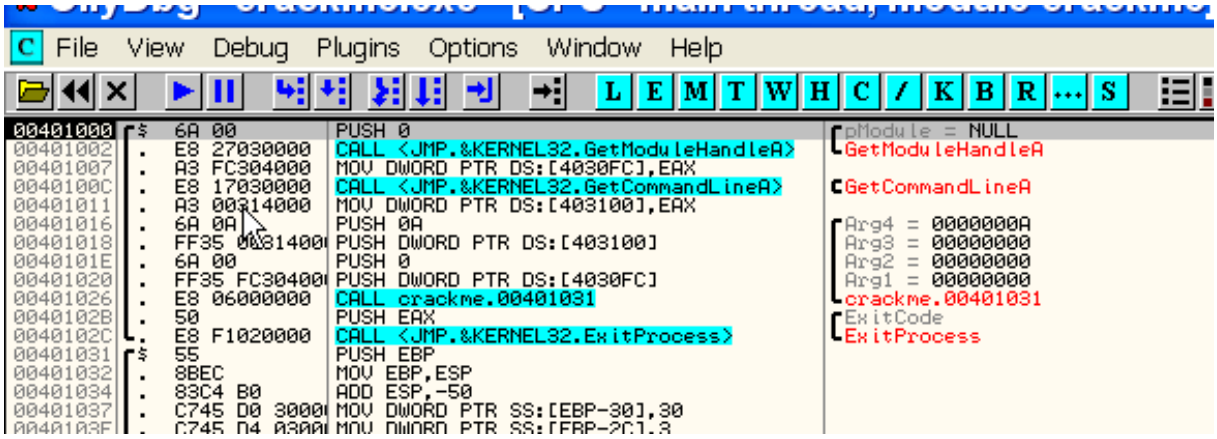
Введение в крэкинг с нуля, используя OllyDbg - Глава 14

Автор: Рикардо Нарваха, пер. Aquila

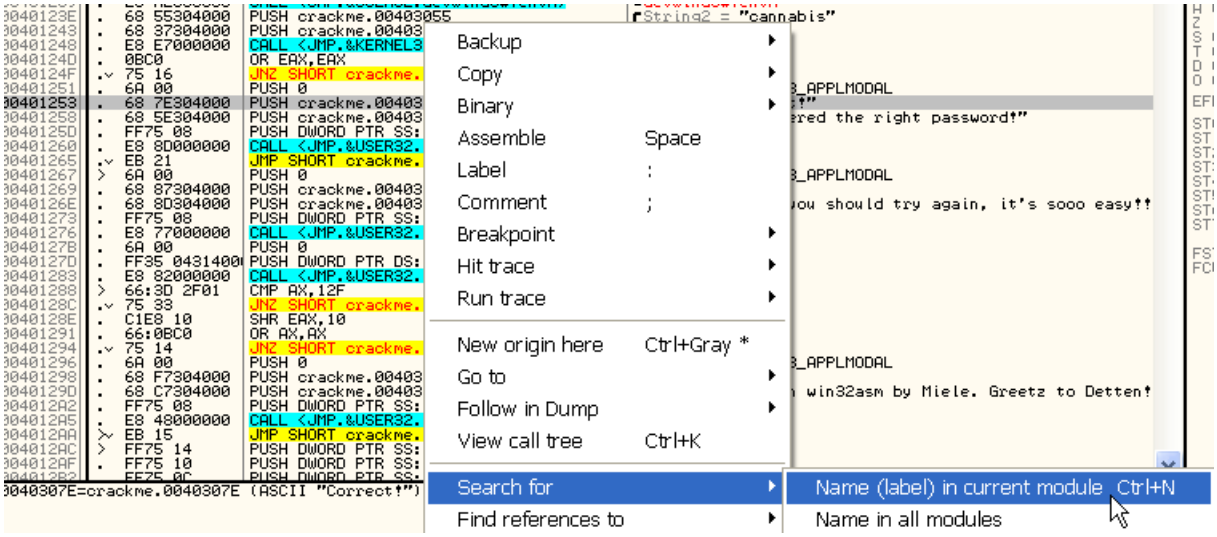
Дата: 2007-11-13

Файлы к статье (находится в архиве `wasm_files.zip`: `files/0466/ollydbg14.zip`)

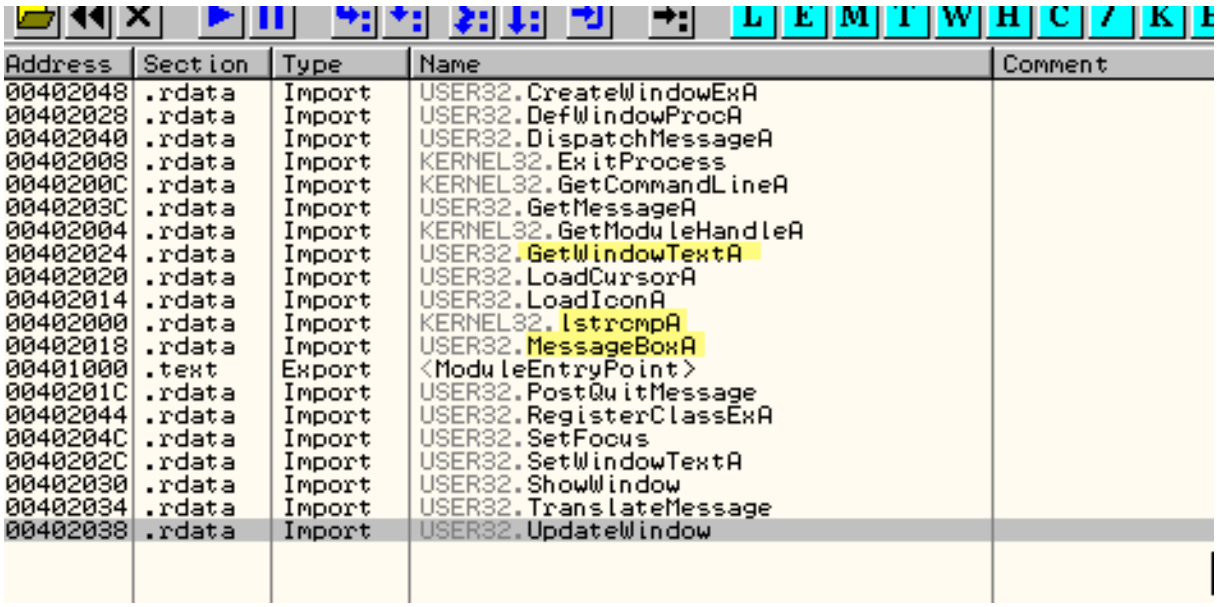
Прежде всего я собираюсь объяснить, как решается крэкми, над которым мы начали работать в главе 13.



Это mielecrackme, загруженный в OllyDbg, и его точка входа. Посмотрим, какие функции API используются с помощью SEARCH FOR-NAME (LABEL) IN CURRENT MODULE.



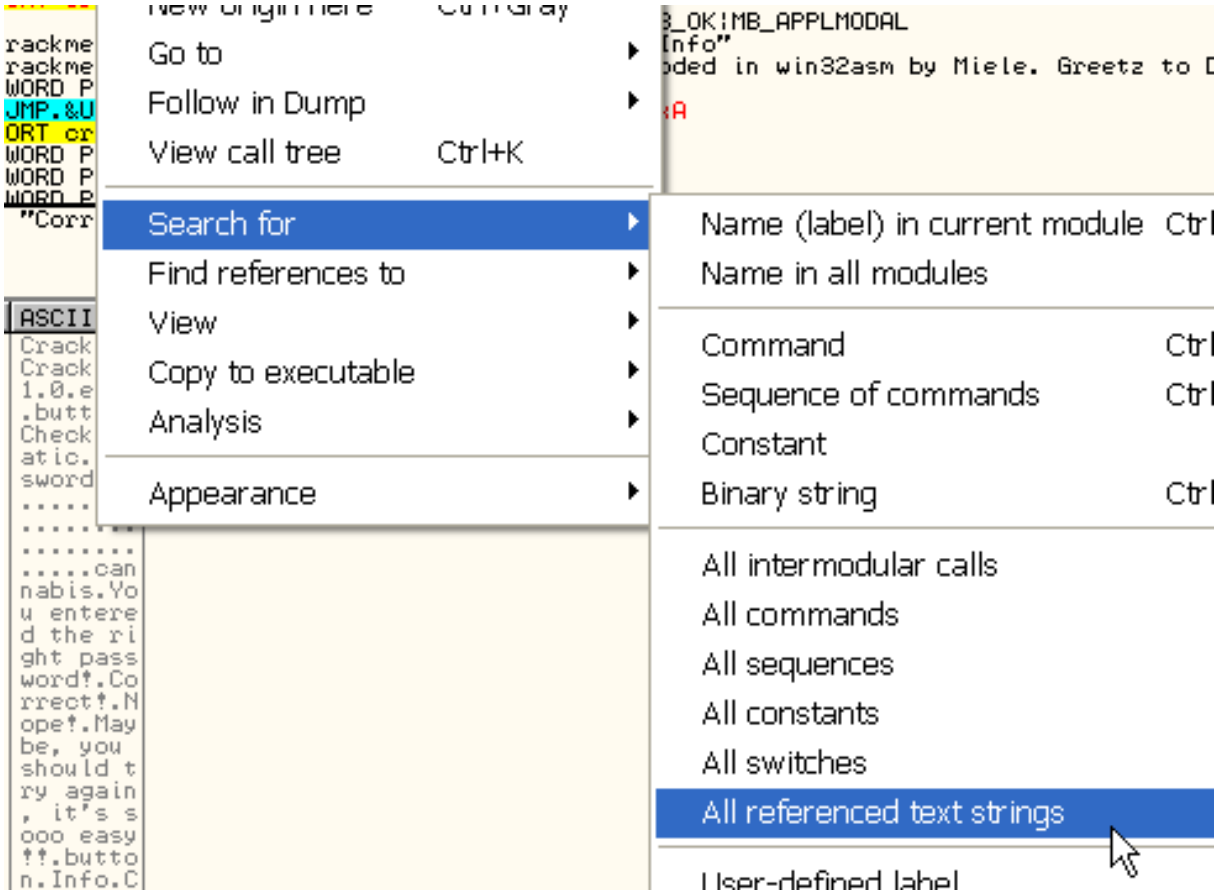
Вот список найденных api-функций.



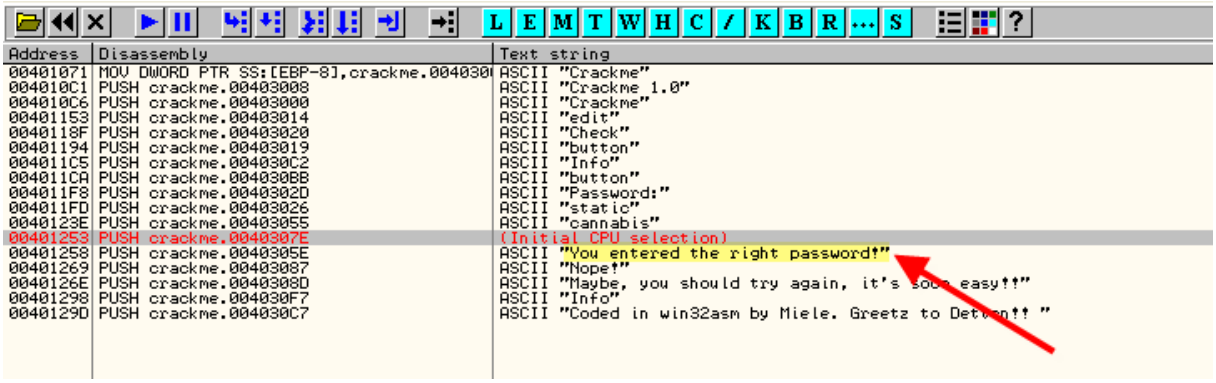
Address	Section	Type	Name	Comment
00402048	.rdata	Import	USER32.CreateWindowExA	
00402028	.rdata	Import	USER32.DefWindowProcA	
00402040	.rdata	Import	USER32.DispatchMessageA	
00402008	.rdata	Import	KERNEL32.ExitProcess	
0040200C	.rdata	Import	KERNEL32.GetCommandLineA	
0040203C	.rdata	Import	USER32.GetMessageA	
00402004	.rdata	Import	KERNEL32.GetModuleHandleA	
00402024	.rdata	Import	USER32.GetWindowTextA	
00402020	.rdata	Import	USER32.LoadCursorA	
00402014	.rdata	Import	USER32.LoadIconA	
00402000	.rdata	Import	KERNEL32.lstrcmpA	
00402018	.rdata	Import	USER32.MessageBoxA	
00401000	.text	Export	<ModuleEntryPoint>	
0040201C	.rdata	Import	USER32.PostQuitMessage	
00402044	.rdata	Import	USER32.RegisterClassExA	
0040204C	.rdata	Import	USER32.SetFocus	
0040202C	.rdata	Import	USER32.SetWindowTextA	
00402030	.rdata	Import	USER32.ShowWindow	
00402034	.rdata	Import	USER32.TranslateMessage	
00402038	.rdata	Import	USER32.UpdateWindow	

Среди них указаны самые важные: GetWindowTextA, использующаяся при вводе серийного номера, lstrcmpA, которая встречалась в предыдущей главе и используется для сравнения строк, и MessageBoxA, отображающая сообщение о том, верен ли серийный номер или нет.

Мы можем установить BPX на эти функции, чтобы произошла остановка, когда будет введён неверный серийный номер, но в данном случае, который весьма прост, можно сделать всё гораздо быстрее, если посмотреть СТРОКИ, используемые программой.



Выполнение SEARCH FOR – ALL REFERENCED TEXT STRINGS даёт нам список строк, которые использует крэкми.

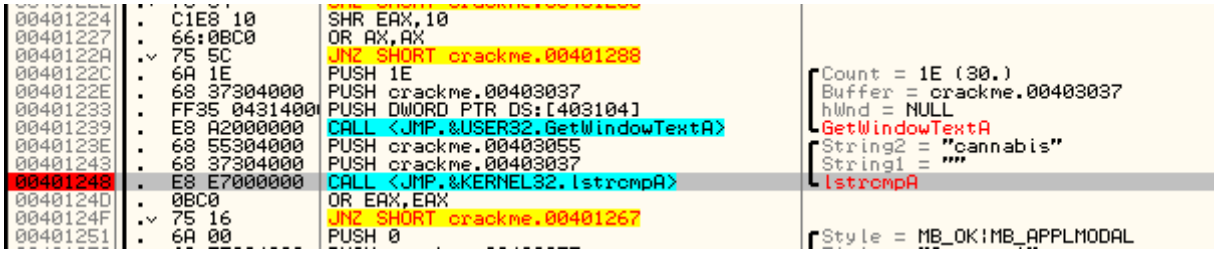


Здесь видим как строку, отображающуюся как в успешном случае, так и строку, показывающуюся в случае неудачи. Если сделаем на какой-нибудь из них двойной клик мышью, то окажемся в окрестностях MessageBoxA. Попробуем сделать двойной клик на “YOU ENTERED THE RIGHT PASSWORD” («Вы ввели правильный пароль»).

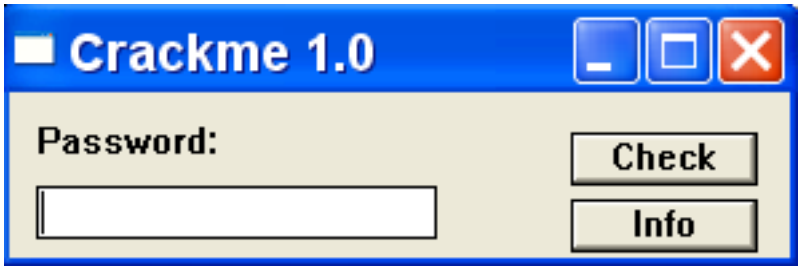
Оказались в соответствующем месте.

Сначала идёт GetWindowTextA, считывающая серийный номер, который мы напечатали, затем lstrcmpA, сравнивающая считанный номер с правильным, а затем, если они верны, MessageBoxA показывает «YOU ENTERED THE RIGHT PASSWORD», а если нет, то происходит переход к другому MessageBoxA, отображающему «YOU SHOULD TRY AGAIN, IT'S SO EASY» («Попробуй ещё раз, это же так просто»).

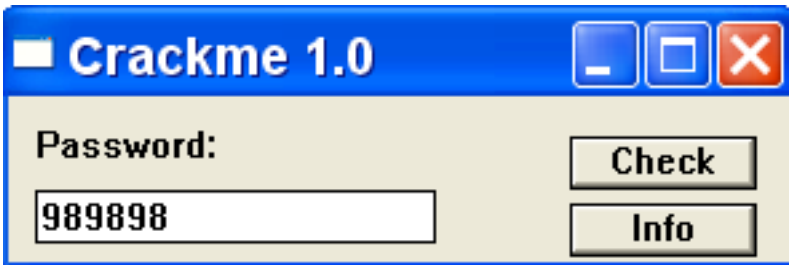
Установи BPX на вызове lstrcmpA, чтобы посмотреть, что она сравнивает.



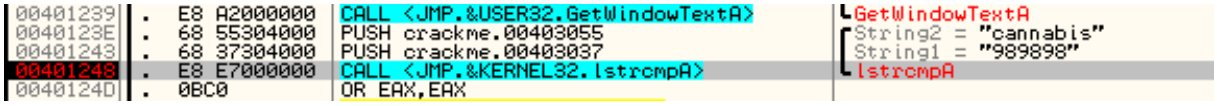
Запустим программу с помощью F9.



Выскочило окно ввода серийного номера. Введём в него что-нибудь вроде 989898.

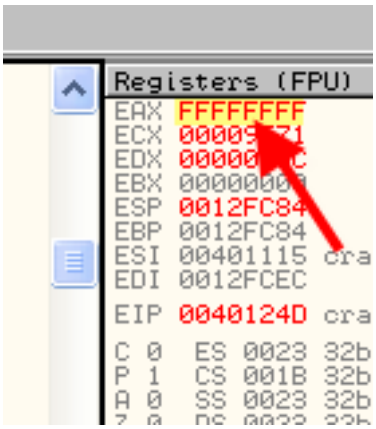


Нажмём кнопку CHECK, чтобы сработал установленный нами BPX.

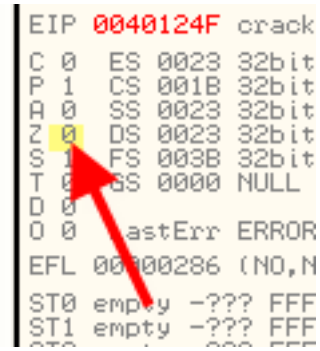


Видим, что OLLY показывает и параметры функции, которые являются двумя сравниваемыми строками, в данном случае сравниваются строки «989898» и «cannabis».

Чтобы выполнить вызов этой API-функции, нажмём F8.



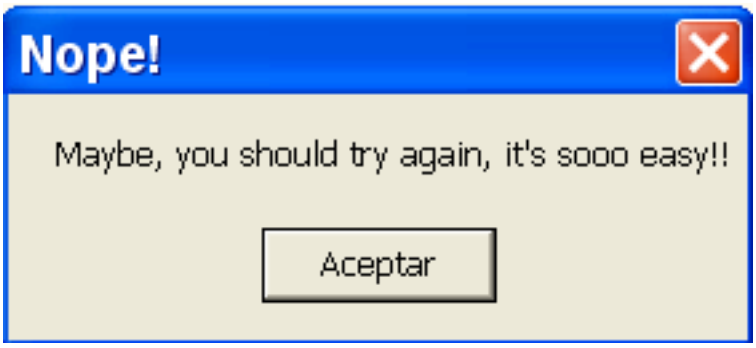
В EAX помещается результат, равный FFFFFFFF или -1, а это означает, что строки не одинаковые.



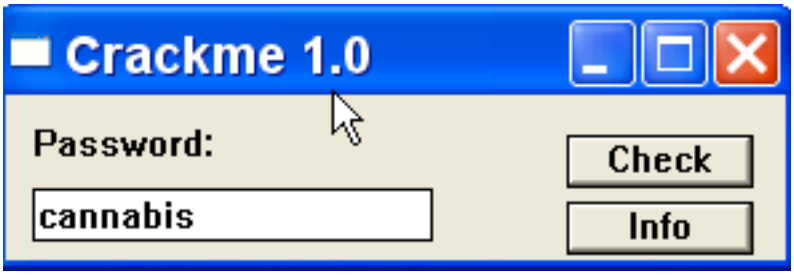
В результате этого сравнения флаг Z неактивен и инструкция JNZ выполняет переход, т.к. флаг Z равен нулю (вспомним, что JNZ совершает переход, если флаг Z равен нулю, а JZ, наоборот, если флаг Z активен, т.к. равен 1).

```
00401243  .  68 37304000    PUSH crackme.00403037                  String1 = "989898"
00401248  .  E8 E7000000    CALL <JMP.&KERNEL32.lstrcmpA>          lstrcmpA
0040124D  .  0BC0           OR EAX,EAX
0040124F  .v 75 16          JNZ SHORT crackme.00401267
00401251  .  6A 00          PUSH 0                                 Style = MB_OK|MB_APPLMODAL
00401253  .  68 7E304000    PUSH crackme.0040307E                  Title = "Correct!"
00401258  .  68 5E304000    PUSH crackme.0040305E                  Text = "You entered the right passwo
0040125D  .  FF75 08        PUSH DWORD PTR SS:[EBP+8]              hOwner
00401260  .  E8 8D000000    CALL <JMP.&USER32.MessageBoxA>         MessageBoxA
00401265  .v EB 21          JMP SHORT crackme.00401288
00401267  > >6A 00          PUSH 0                                 Style = MB_OK|MB_APPLMODAL
00401269  .  68 87304000    PUSH crackme.00403087                  Title = "Nope!"
0040126E  .  68 8D304000    PUSH crackme.0040308D                  Text = "Maybe, you should try again,
00401273  .  FF75 08        PUSH DWORD PTR SS:[EBP+8]              hOwner
00401276  .  E8 77000000    CALL <JMP.&USER32.MessageBoxA>         MessageBoxA
0040127B  .  6A 00          PUSH 0                                 Text = NULL
0040127D  .  FF35 0431400(  PUSH DWORD PTR DS:[403104]             hWnd = 02240882 (class='Edit',parent
00401283  .  E8 82000000    CALL <JMP.&USER32.SetWindowTextA>      SetWindowTextA
00401288  > 66:3D 2F01      CMP AX,12F
```

Раз мы делаем переход, то получаем сообщение с ошибкой, но зато теперь мы знаем, что сравнение происходит со словом «cannabis», то есть это и есть верный серийный номер. Снова продолжим выполнение программы.



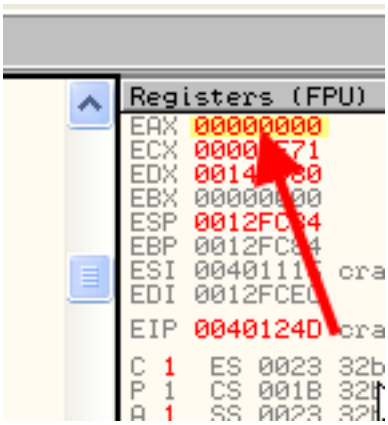
Нажав на «OK», снова вернёмся в окно ввода серийного номера и напечатаем правильный – «cannabis».



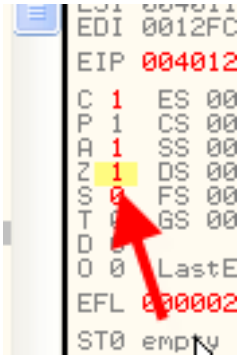
Нажмём кнопку CHECK, и срабатывает BPX.

```
00401239  .  E8 A2000000    CALL <JMP.&USER32.GetWindowTextA>      GetWindowTextA
0040123E  .  68 55304000    PUSH crackme.00403055                  String2 = "cannabis"
00401243  .  68 37304000    PUSH crackme.00403037                  String1 = "cannabis"
00401248  .  E8 E7000000    CALL <JMP.&KERNEL32.lstrcmpA>          lstrcmpA
0040124D  .  0BC0           OR EAX,EAX
0040124F  .v 75 16          JNZ SHORT crackme.00401267
```

Видим, что будут сравниваться две одинаковые строки. Жмём F8, чтобы выполнить вызов API-функции..



Обе строки были равны, и в EAX был сохранён результат функции – ноль, что активирует флаг Z.



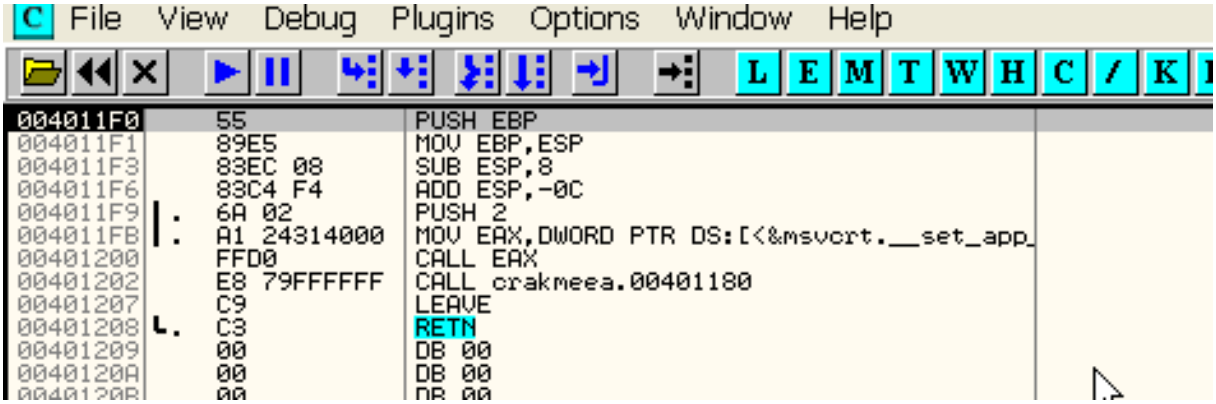
И поэтому JNZ не будет совершать переход.

А раз так, то как только мы продолжим выполнение программы, отобразится окошко, сообщающее о том, что мы ввели верный серийный номер.

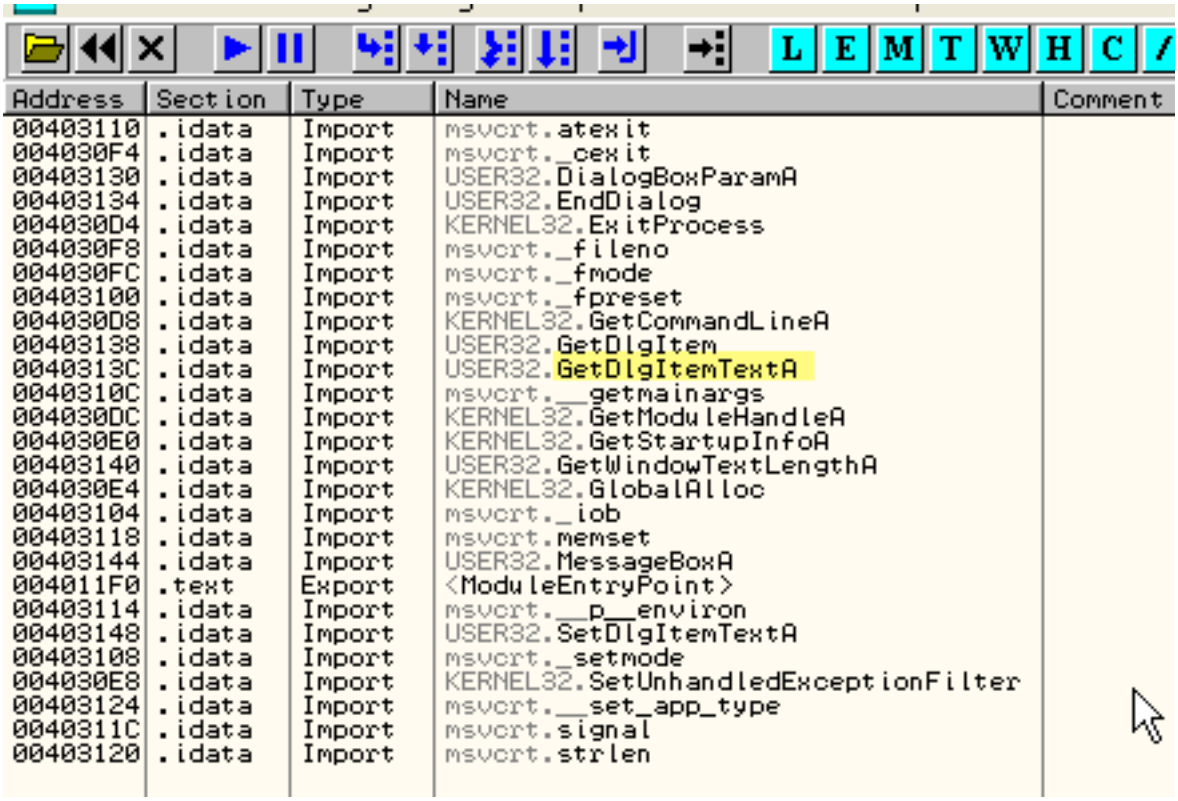
Это и есть решение крэкми из главы 13. Хе-хе, серийным номером является слово «каннабис».

Продолжим дальше и рассмотрим более сложный случаи жёстко заданных серийных номером, чем те, что мы рассматривали ранее.

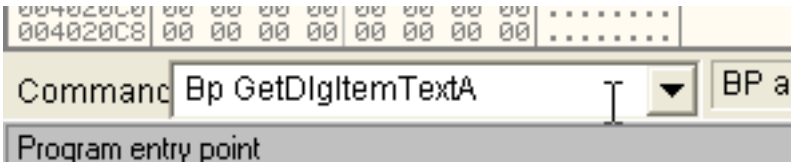
В следующем случае уже не производится прямого сравнения серийного номера с введённой строкой. Откроем крэкми crackmeeasy в OllyDbg.



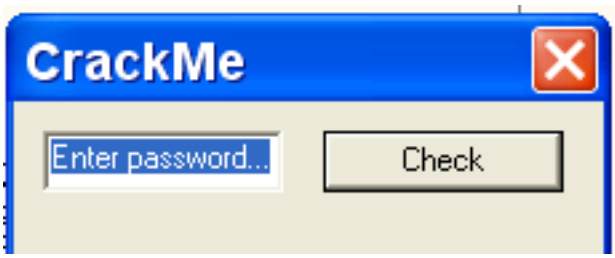
Мы уже знаем, как посмотреть используемые API-функции. В списке также есть GetDlgItemTextA, на который мы и установим BPX.



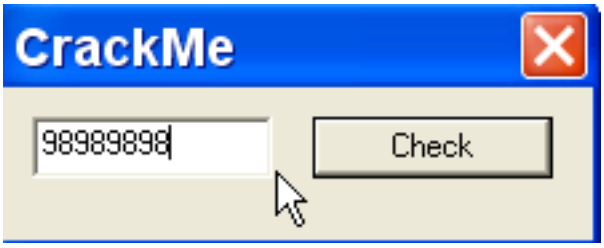
В commandbar’е напечатаем:



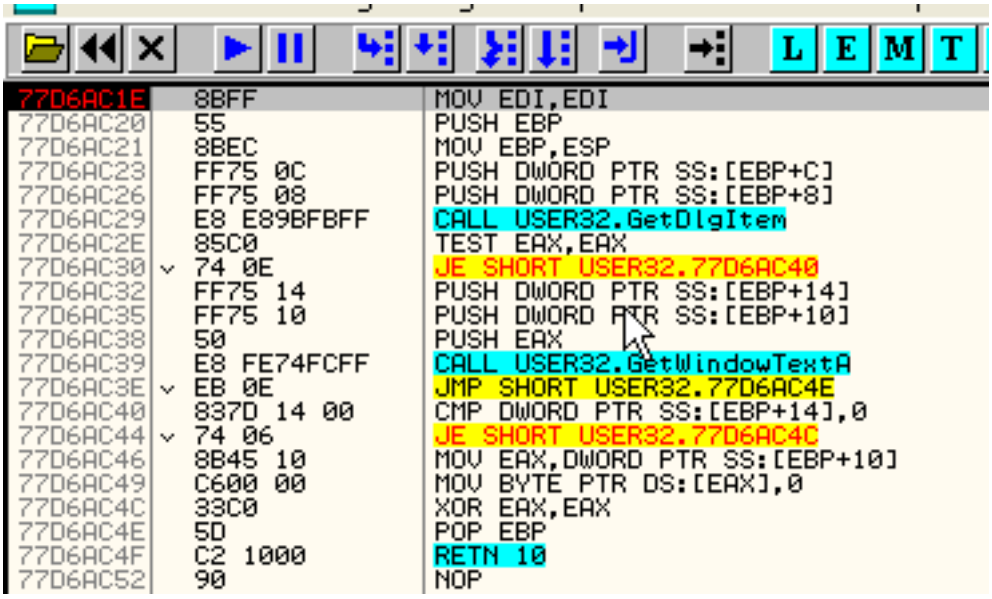
Теперь запустим программу с помощью F9, и откроется окно ввода серийного номера.



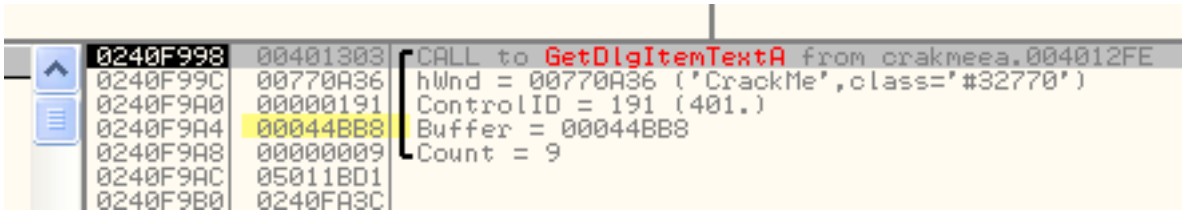
Напечатаем неверный серийный номер.



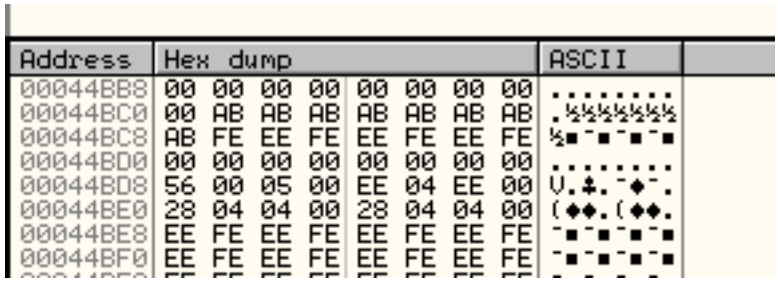
Кликнем по кнопке “Check” и попадём на вызов нужной нам функции.



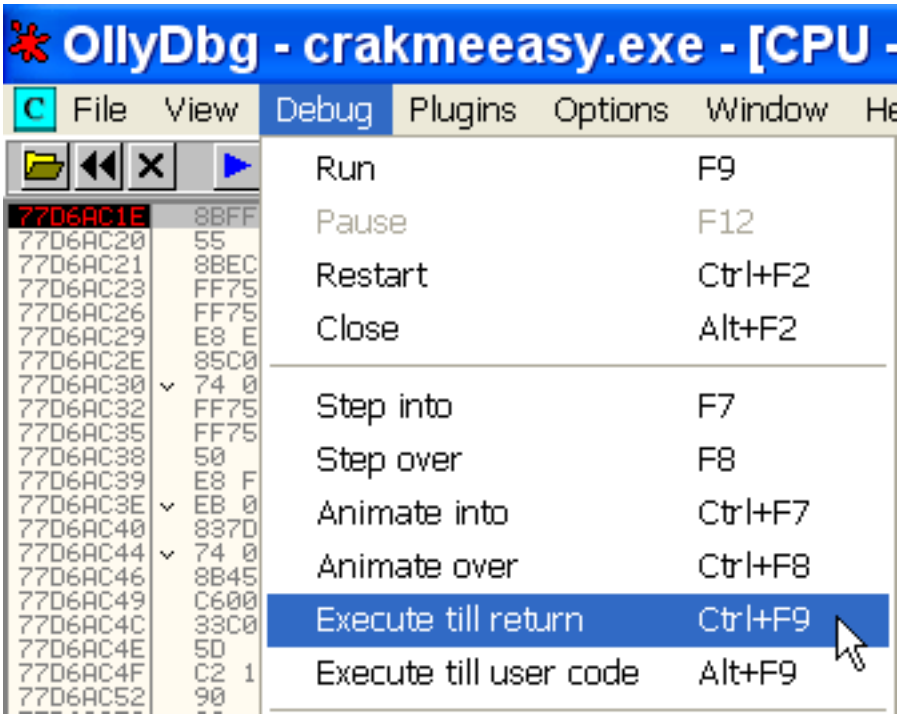
Посмотрим, какие параметры в стеке.



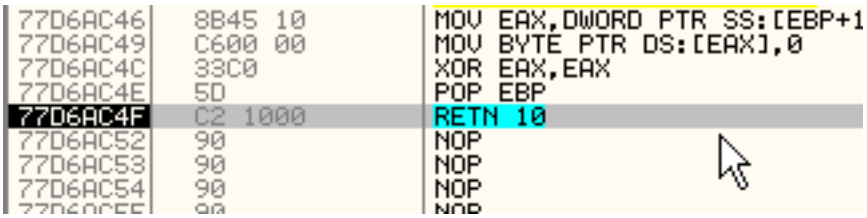
Здесь видим буфер, в котором сохраняется неправильный серийный номер, отметим эту строку, затем правый клик мышью и FOLLOW IN DUMP.



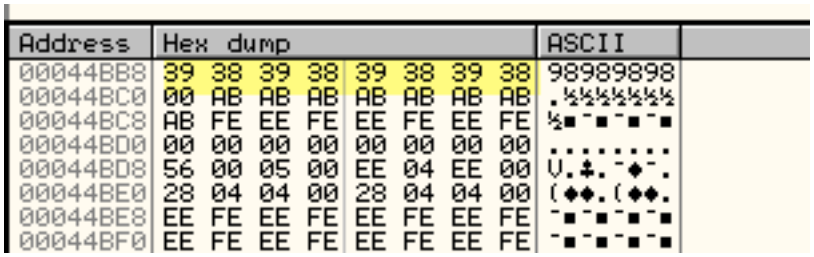
Буфер пуст, так как функция ещё не запустилась, поэтому выберем DEBUG-EXECUTE TILL RETURN.



Это приведёт к выполнению функции, которое остановится на инструкции RET.



Нажмём F7, чтобы вернуться в программу.



И видим, что в буфере окажется введенный нами неверный серийный номер.

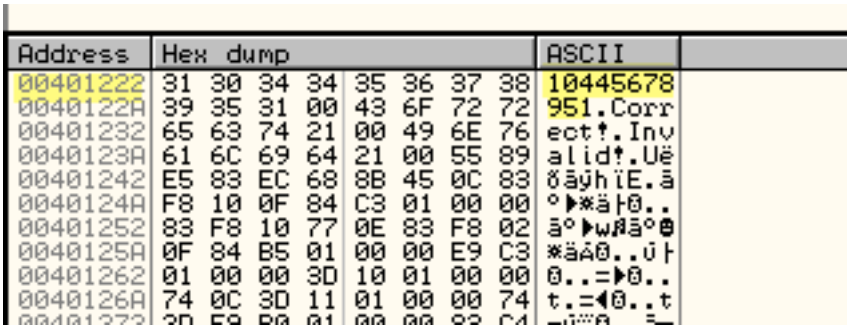
```
004012FA  .  8B45 08       MOV EAX,DWORD PTR SS:[EBP+8]
004012FD  .  50            PUSH EAX                          hWnd
004012FE  .  E8 4D030000   CALL <JMP.&USER32.GetDlgItemTextA> GetDlgItemTextA
00401303  .  C745 F0 0000  MOV DWORD PTR SS:[EBP-10],0
0040130A  .  B8 22124000   MOV EAX,crakmeea.00401222         ASCII "10445678951"
0040130F  .  8B10          MOV EDX,DWORD PTR DS:[EAX]
00401311  .  8955 D0       MOV DWORD PTR SS:[EBP-30],EDX
00401314  .  8B50 04       MOV EDX,DWORD PTR DS:[EAX+4]
00401317  .  8955 D4       MOV DWORD PTR SS:[EBP-2C],EDX
0040131A  .  8B40 08       MOV EAX,DWORD PTR DS:[EAX+8]
0040131D  .  8945 D8       MOV DWORD PTR SS:[EBP-28],EAX
00401320  .  8D45 DC       LEA EAX,DWORD PTR SS:[EBP-24]
00401323  .  83C4 FC       ADD ESP,-4
00401326  .  6A 08         PUSH 8                            n = 8
00401328  .  6A 00         PUSH 0                            c = 00
0040132A  .  50            PUSH EAX                          s
0040132B  .  E8 F0020000   CALL <JMP.&msvcrt.memset>         memset
00401330  .  83C4 10       ADD ESP,10
00401333  .  C745 CC 0000  MOV DWORD PTR SS:[EBP-34],0
0040133A  .  8DB6 0000000  LEA ESI,DWORD PTR DS:[ESI]
00401340  >  83C4 F4       ADD ESP,-0C
00401343  .  8D45 D0       LEA EAX,DWORD PTR SS:[EBP-30]
00401346  .  50            PUSH EAX                          s
00401347  .  E8 DC020000   CALL <JMP.&msvcrt.strlen>         strlen
0040134C  .  83C4 10       ADD ESP,10
0040134F  .  89C0          MOV EAX,EAX
00401351  .  8D50 FF       LEA EDX,DWORD PTR DS:[EAX-1]
00401354  .  3955 F0       CMP DWORD PTR SS:[EBP-10],EDX
00401357  .v 72 07         JB SHORT crakmeea.00401360
00401359  .v EB 35         JMP SHORT crakmeea.00401390
0040135B     90            NOP
0040135C     8D7426 00     LEA ESI,DWORD PTR DS:[ESI]
00401360  >  8B45 F4       MOV EAX,DWORD PTR SS:[EBP-C]
00401363  .  8B55 F0       MOV EDX,DWORD PTR SS:[EBP-10]
00401366  .  01D0          ADD EAX,EDX
00401368  .  0FBE10        MOVSX EDX,BYTE PTR DS:[EAX]
0040136B  .  8D42 EC       LEA EAX,DWORD PTR DS:[EDX-14]
0040136E  .  8D55 D0       LEA EDX,DWORD PTR SS:[EBP-30]
00401371  .  8B4D F0       MOV ECX,DWORD PTR SS:[EBP-10]
00401374  .  0FBE1411      MOVSX EDX,BYTE PTR DS:[ECX+EDX]
00401378  .  39D0          CMP EAX,EDX
0040137A  .v 75 0D         JNZ SHORT crakmeea.00401389
0040137C  .  8D45 D0       LEA EAX,DWORD PTR SS:[EBP-30]
0040137F  .  8B55 F0       MOV EDX,DWORD PTR SS:[EBP-10]
Stack SS:[0240FA04]=0240000F
```

Здесь мы также видим большую процедуру, делающую что-то с какой-то строкой в виде числа. Кому-нибудь пришла мысль, что это правильный серийный номер? Я знаю, что нет, хе-хе.

```
004012FE  .  E8 4D030000   CALL <JMP.&USER32.GetDlgItemTextA> GetDlgItemTextA
00401303  .  C745 F0 0000  MOV DWORD PTR SS:[EBP-10],0
0040130A  .  B8 22124000   MOV EAX,crakmeea.00401222         ASCII "10445678951"
0040130F  .  8B10          MOV EDX,DWORD PTR DS:[EAX]
00401311  .  8955 D0       MOV DWORD PTR SS:[EBP-30],EDX
00401314  .  8B50 04       MOV EDX,DWORD PTR DS:[EAX+4]
```

Здесь в EAX помещается число 401222, которое является адресом, указывающем на строку, содержащую число-константу.

```
Registers (FPU)
EAX 00401222 ASCII "10445678951"
ECX 77D3219D USER32.77D3219D
EDX 00040608
EBX 00000000
ESP 0240F9AC
EBP 0240FA14
ESI 00401240 crakmeea.00401240
EDI 0240FA7C

EIP 0040130F crakmeea.0040130F

C 0  ES 0023 32bit 0(FFFFFFFF)
P 1  CS 001B 32bit 0(FFFFFFFF)
A 0  SS 0023 32bit 0(FFFFFFFF)
```

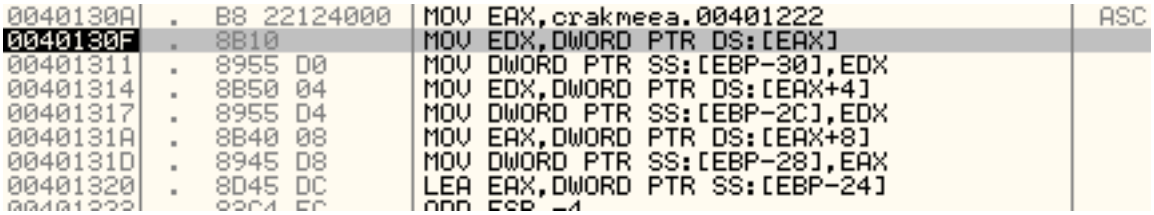


На следующей строке, где EAX равен 401222:

MOV EDX,DWORD PTR DS:[EAX]

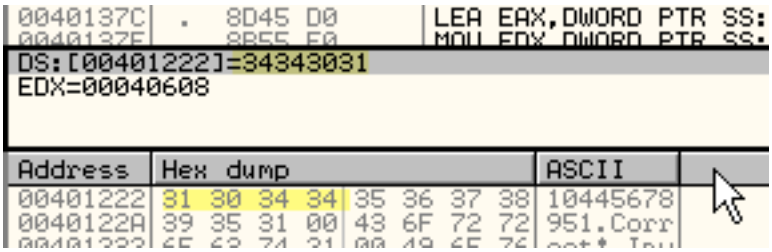
Это, на самом деле, всё равно, что:

MOV EDX,DWORD PTR DS:[401222]

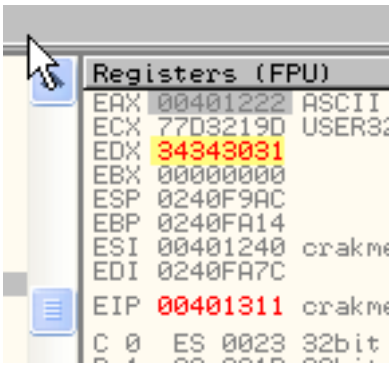


То есть содержимое памяти по адресу 401222 перемещается в 401222.

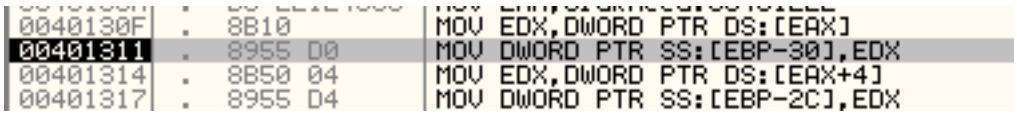
Олли показывает, что это первые 4 байта числа 10445678951.



Выполнение строки с помощью F7 перемещает их в EDX (они всегда перемещаются в регистр в порядке, обратном тому, как они располагались в памяти).



На следующей строке байты, находящиеся в EDX, перемещаются в [EBP-30].



Из пояснений Олли видно, что [EBP-30] на моей машине – это 240f9e4. Поищем это значение в DUMP'е.

```
0040137C  .  8D45 D0      LEA EAX
0040137F     8B55 F0      MOV EDX
EDX=34343031
Stack SS:[0240F9E4]=0004443C
```

Address	Hex dump	ASCII
0240F9E4	3C 44 04 00 00 02 00 00	<D♦..☻..
0240F9EC	03 00 00 00 03 00 00 00	♥...♥...
0240F9F4	57 00 00 00 14 00 00 00	W...¶...
0240F9FC	11 00 00 00 3C 44 04 00	◄...<D♦.
0240FA04	00 00 00 00 B8 4B 04 00	....@K♦.

F7 копирует байты, находящиеся в EDX.

Address	Hex dump	ASCII
0240F9E4	31 30 34 34 00 02 00 00	1044.☻..
0240F9EC	03 00 00 00 03 00 00 00	♥...♥...
0240F9F4	57 00 00 00 14 00 00 00	W...¶...

После чего

```
00401311  .  8955 D0      MOV DWORD PTR SS:[EBP-30],EDX
00401314  .  8B50 04      MOV EDX,DWORD PTR DS:[EAX+4]
00401317  .  8955 D4      MOV DWORD PTR SS:[EBP-2C],EDX
0040131A  .  8B40 08      MOV EAX,DWORD PTR DS:[EAX+8]
0040131D  .  8945 D8      MOV DWORD PTR SS:[EBP-28],EAX
00401320  .  8D45 DC      LEA EAX,DWORD PTR SS:[EBP-24]
00401323  .  83C4 FC      ADD ESP,-4
00401326  .  6A 08        PUSH 8
```

в EDX перемещаются следующие 4 байта числа-константы.

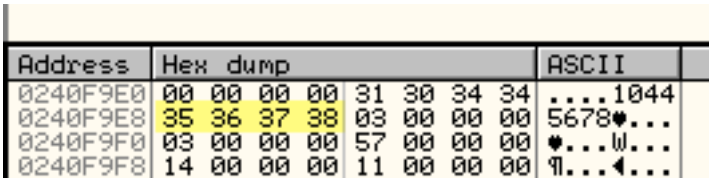
```
0040137F     8B55 F0      MOV EDX,DWORD PTR SS:
DS:[00401226]=38373635
EDX=34343031
```

Address	Hex dump	ASCII
00401222	31 30 34 34 35 36 37 38	10445678
0040122A	39 35 31 00 43 6F 72 72	951.Corr
00401232	65 63 74 21 00 49 6E 76	ect!.Inv
0040123A	61 6C 69 64 21 00 55 89	alid!.Uë

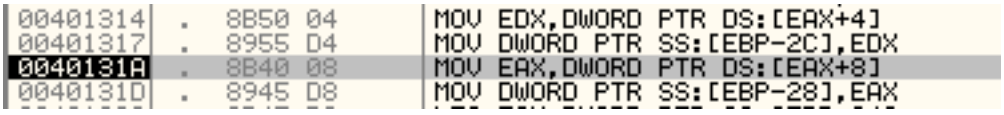
Пояснения Олли показывают, что [eax+4] содержат 401226, и выполнение инструкции с помощью F7 перемещает следующие 4 байта в EDX.

```
Registers (FPU)
EAX 00401222 ASC
ECX 77D3219D USE
EDX 38373635
EBX 00000000
ESP 0240F9AC
EBP 0240FA14
ESI 00401240 cra
EDI 0240FA7C
EIP 00401317 cra
C 0  ES 0023 32b
```

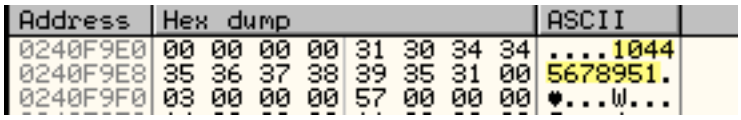
И копируется продолжение того, что копировалось ранее.



В действительности происходит вот что: этот номер копируется по 4 байта в другую часть памяти.

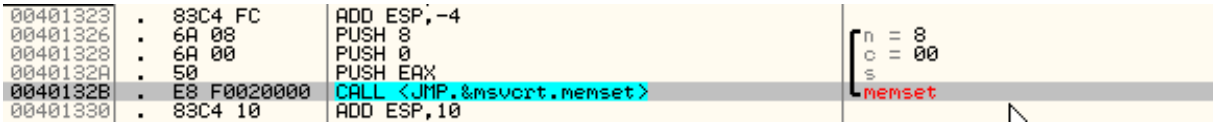


И наконец копируются последние 4 байта.



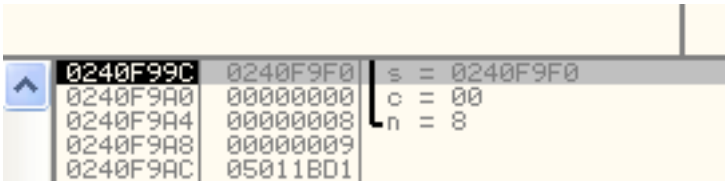
Наконец-то номер скопирован полностью.

Видим, что чуть ниже находится вызов функции memset. Посмотрим её параметры с помощью OllyDbg.

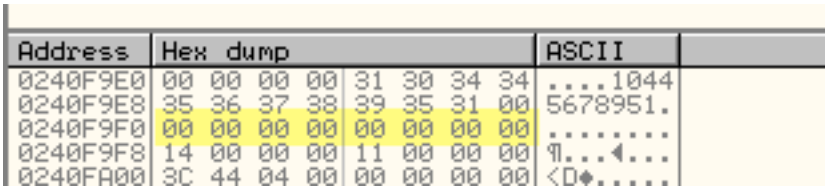


Здесь три значение (n, c и s).

s – начальный адрес
n – количество байтов, которых нужно заполнить требуемым значением
c – значение, которым будет заполняться указанная область памяти

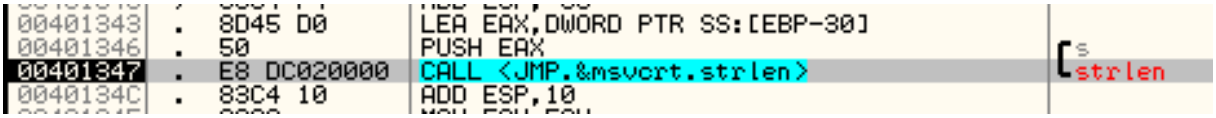


В стеке находятся вышеуказанные параметры: область памяти по адресу 240f9f0 продолжительностью в 8 байт должна быть заполнена нулями.

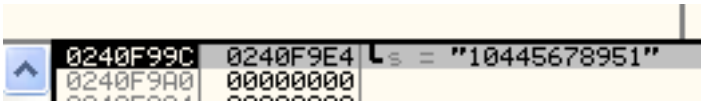


Выполнив эту функцию с помощью F8, мы видим, что указанная нами область памяти действительно была заполнена нулями.

Ещё ниже находится вызов lstrlen, которая считает длину строки, адрес которой задаётся непосредственно до вызова самой функции.

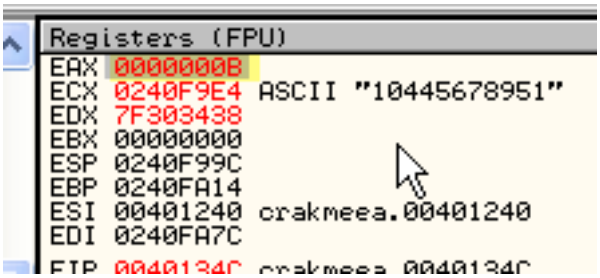


В стеке находятся следующие параметры:



То есть будет вычислена длина строки, которая начинается по адресу 240f9E4, что является адресом уже известного нам числа, хех.

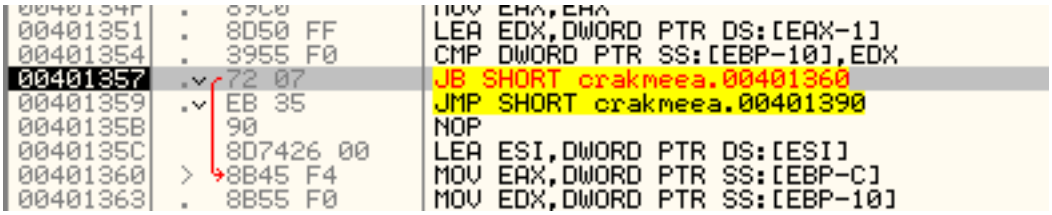
F8, и происходит выполнение strlen, а в EAX возвращается длина строки.



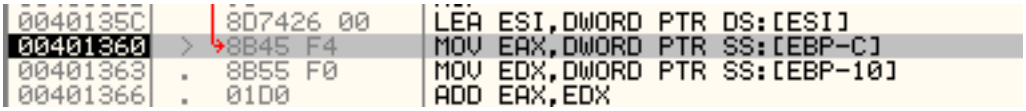
Видим, что размер равен 0B, то есть 11 в десятиричной системе, что является длиной нашего номера.

Здесь LEA перемещает в EDX значение EAX минус 1, то есть 0A.

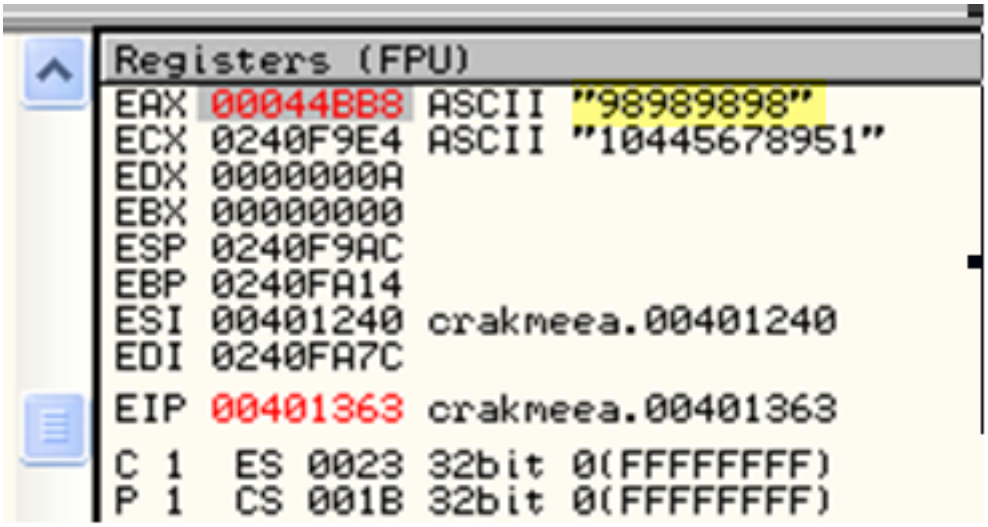
На следующей строке происходит сравнение 0A с содержимым [EBX-10], то есть с нулём.



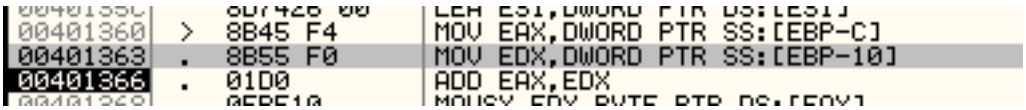
И раз ноль меньше чем 0A, то переходим на 40135C.

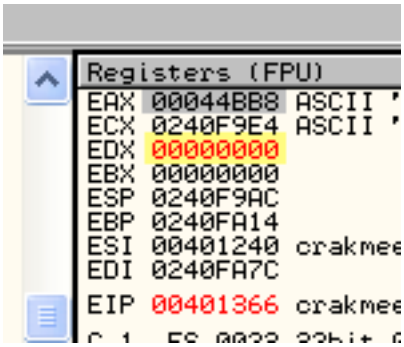


На следующей строке в EAX помещается указатель на наш неверный серийный номер, и поэтому когда нажмём на F7, то EAX будет указывать на “98989898”.

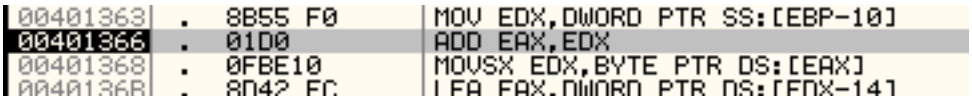


На следующей строке перемещаем в EDX ноль.





На следующей строке

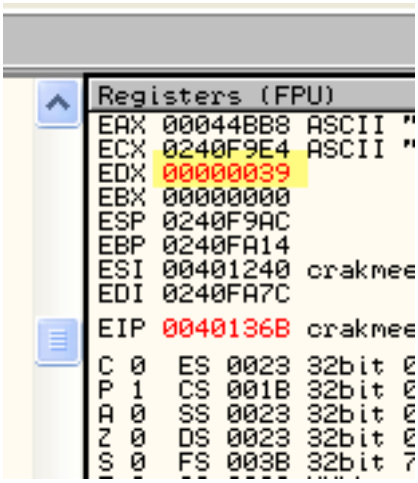


Так как EAX указывает на начало неверного серийного номер, прибавляем к нему EDX (который изначально равен нулю) для создания цикла, каждый увеличивая значение EDX на 1 (1, 2, 3 и т.п.), чтобы получить все байты неверного серийного номера по одном в указанном цикле.

```
00401363  .   8B55 F0        MOV EDX,DWORD PTR SS:[EBP-10]
00401366  .   01D0            ADD EAX,EDX
00401368  .   0FBE10          MOVSX EDX,BYTE PTR DS:[EAX]
0040136B  .   8D42 EC         LEA EAX,DWORD PTR DS:[EDX-14]
0040136E  .   8D55 D0         LEA EDX,DWORD PTR SS:[EBP-30]
00401371  .   8B4D F0         MOV ECX,DWORD PTR SS:[EBP-10]
00401374  .   0FBE1411        MOVSX EDX,BYTE PTR DS:[ECX+EDX]
00401378  .   39D0            CMP EAX,EDX
0040137A  .v  75 0D           JNZ SHORT crakmeea.00401389
0040137C  .   8D45 D0         LEA EAX,DWORD PTR SS:[EBP-30]
0040137F  .   8B55 F0         MOV EDX,DWORD PTR SS:[EBP-10]
00401382  .   C60402 73       MOV BYTE PTR DS:[EDX+EAX],73
00401386  .   FF45 CC         INC DWORD PTR SS:[EBP-34]
00401389  >   FF45 F0         INC DWORD PTR SS:[EBP-10]
0040138C  .^  EB B2           JMP SHORT crakmeea.00401340
0040138E      89F6            MOV ESI,ESI
00401390  >   B8 2E124000     MOV EAX,crakmeea.0040122E
00401395  .   8B10            MOV EDX,DWORD PTR DS:[EAX]
00401397  .   8955 B0         MOV DWORD PTR SS:[EBP-50],EDX
0040139A  .   8B50 04         MOV EDX,DWORD PTR DS:[EAX+4]
0040139D      8955 B4         MOV DWORD PTR SS:[EBP-4C],EDX
DS:[00044BB8]=39 ('9')
EDX=00000000
```

Как мы уже знаем, MOVSX переместит байт в EDX и если он положителен, то заполняет остаток регистра нулями, а если отрицателен – ‘F’.

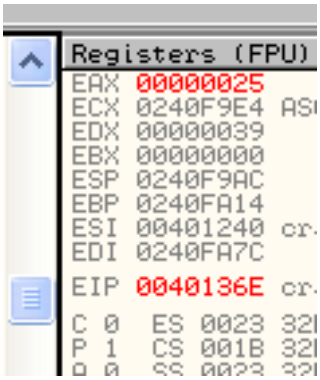
В данном случае первый байт неверного серийного номера переместится в EDX и выполнение строки покажет, что EDX стал равным 39.



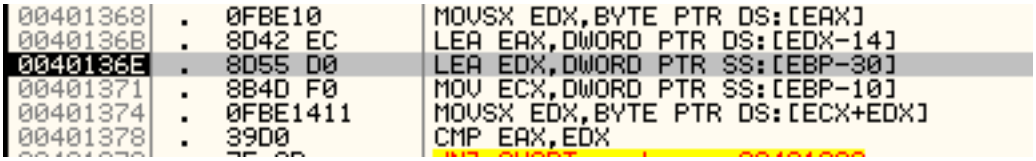
Следующая строка с LEA.

```
00401366  .   01D0            ADD EAX,EDX
00401368  .   0FBE10          MOVSX EDX,BYTE PTR DS:[EAX]
0040136B  .   8D42 EC         LEA EAX,DWORD PTR DS:[EDX-14]
0040136E  .   8D55 D0         LEA EDX,DWORD PTR SS:[EBP-30]
00401371  .   8B4D F0         MOV ECX,DWORD PTR SS:[EBP-10]
00401374  .   0FBE1411        MOVSX EDX,BYTE PTR DS:[ECX+EDX]
00401378  .   39D0            CMP EAX,EDX
```

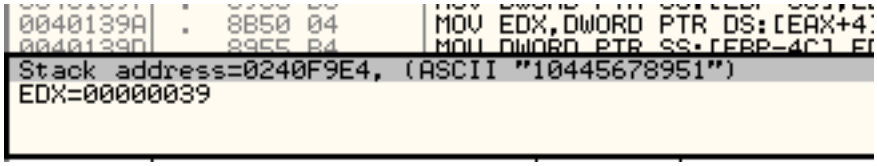
EDX равен 39, вычитаем из него 14 и с помощью LEA помещаем результат непосредственно в EAX.



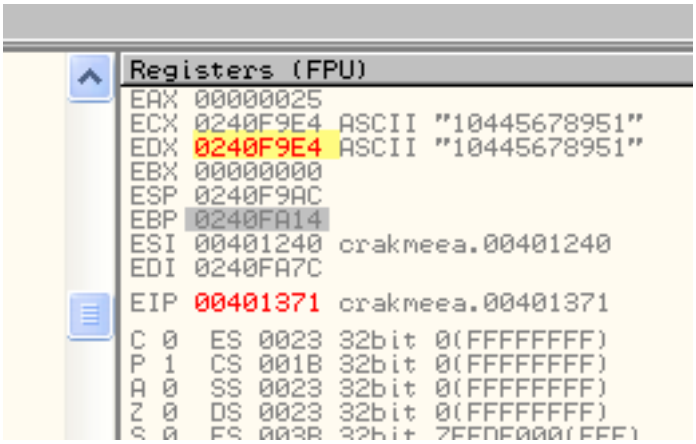
То есть в результате произведённых операций получили 39 (шестнадцатиричное значение первого символа моего серийного номера), отняли от него 14 и результат, равный 25, поместили в EAX.



Следующая строка перемещает значение, находящееся по адресу EBP-30 (на моей машине равное 240f9E4, и оно указывает на начало номера-константы), в EDX.



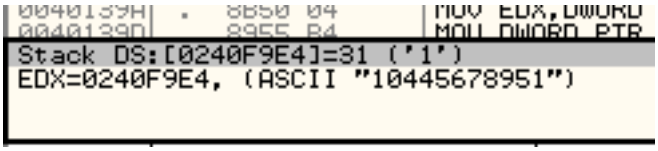
Нажмём F7.



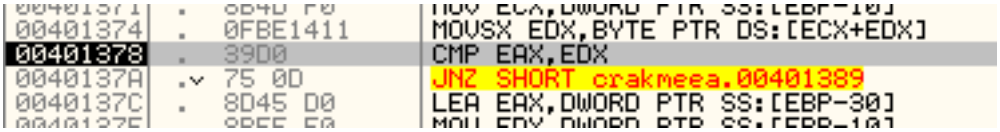
EDX указывает на начало номера-константы.

Далее видим, что в ECX перемещается сначала значение равное нулю, а затем при каждом новом проходе оно увеличивается, чтобы ECX+EDX указывали на разные байты номера-константы.

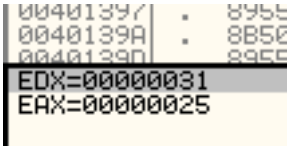
Здесь видим, что ECX теперь содержит ноль, а EDX указывает на начало номер, поэтому в данном случае в EDX будет помещён первый байт номера, дополнительная информация о котором выводится в пояснении OllyDbg.



Пояснение говорит, что 31 соответствует ‘1’ в кодировке ASCII, что является первой цифрой номера-константы.



Здесь мы дошли до сравнение, где участвуют:



В EAX находится значение 25 (значение первого байта неверного серийного номера (39), от которого было отнято значение 14), а в EDX первый байт номера-константы или 31.

А потому видим, что

CMP EAX,EDX

на самом деле является

CMP (ПЕРВЫЙ БАЙТ НЕПРАВИЛЬНОГО СЕРИЙНИКА– 14), ПЕРВЫЙ БАЙТ НОМЕРА-КОНСТАНТЫ

CMP 25,31

И поскольку разность между этими двумя операндами равна нулю, то флаг Z не активируется и выхолняется переход JNZ.

Мы вводили неверный серийный номер, а сравнение может дать верный результат, если будет введён правильный серийный номер, чтобы сравниваемые значения были верны.

CMP (ПЕРВЫЙ БАЙТ ПРАВИЛЬНОГО СЕРИЙНИКА– 14), 31

Это и есть условие, при котором сравниваемые значения будут равны.

ПЕРВЫЙ БАЙТ ПРАВИЛЬНОГО СЕРИЙНИКА-14 = ПЕРВЫЙ БАЙТ НОМЕРА-КОНСТАНТЫ

Поэтому

ПЕРВЫЙ БАЙТ ПРАВИЛЬНОГО СЕРИЙНИКА = ПЕРВЫЙ БАЙТ НОМЕРА-КОНСТАНТЫ + 14

ПЕРВЫЙ БАЙТ ПРАВИЛЬНОГО СЕРИЙНИКА = 31 + 14

ПЕРВЫЙ БАЙТ ПРАВИЛЬНОГО СЕРИЙНИКА = 45, что соответствует “E” в ASCII.

То есть первая буква серийного номера равна E.

Эта побайтовая проверка повторяется в цикле до конца.

ПЕРВЫЙ БАЙТ ПРАВИЛЬНОГО СЕРИЙНИКА = ПЕРВЫЙ БАЙТ НОМЕРА-КОНСТАНТЫ + 14

ВТОРОЙ БАЙТ ПРАВИЛЬНОГО СЕРИЙНИКА = ВТОРОЙ БАЙТ НОМЕРА-КОНСТАНТЫ +14

ТРЕТИЙ БАЙТ ПРАВИЛЬНОГО СЕРИЙНИКА = ТРЕТИЙ БАЙТ НОМЕРА-КОНСТАНТЫ +14

Вот таким образом.

Применим этот алгоритм для каждого байта, прибавляя 14 и получая значение байта, соответствующее верному серийному номеру.

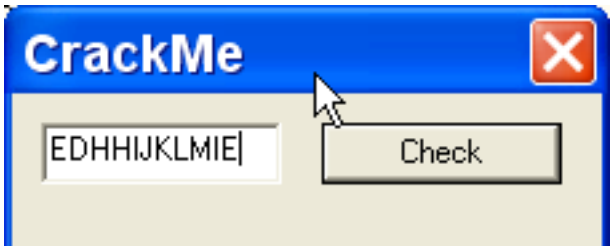
```
31 30 34 34 35 36 37 38  10445678
39 35 31 00 00 00 00 00  951.....
```

31 + 14 = 45 это буква **E** в ASCII
30 + 14 = 44 это буква **D** в ASCII
34 + 14 = 48 это буква **H** в ASCII
34 + 14 = 48 это буква **H** в ASCII
35 + 14 = 49 это буква **I** в ASCII
36 + 14 = 4A это буква **J** в ASCII
37 + 14 = 4B это буква **K** в ASCII
38 + 14 = 4C это буква **L** в ASCII
39 + 14 = 4D это буква **M** в ASCII
35 + 14 = 48 это буква **I** в ASCII
31 + 14 = 45 это буква **E** в ASCII

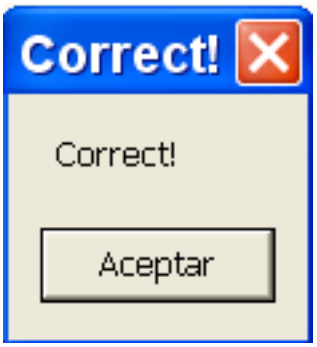
Поэтому правильный серийный номер следующий:

EDHHIJKLMIE

Введём его в окне серийного номера, убрав предварительно все точки останова.



Нажмём “Check”.



Мы рассмотрели как выполняется цикл сравнения, как формировались значения, как последовательно перебирались байты номера-константы и неверного серийного номера и исходя из чего выдавалось сообщение о корректности или некорректности введённого серийного номера.

Этот крэкми не такой простой как предыдущий, но думаю, что вам полезно попрактиковаться, чтобы играть в эти игры лучше.

Ок, прилагаю к этой части крэкми «Splish», попробуйте найти серийный номер, захардкоженный в нём.

Введение в крэкинг с нуля, используя OllyDbg - Глава 15

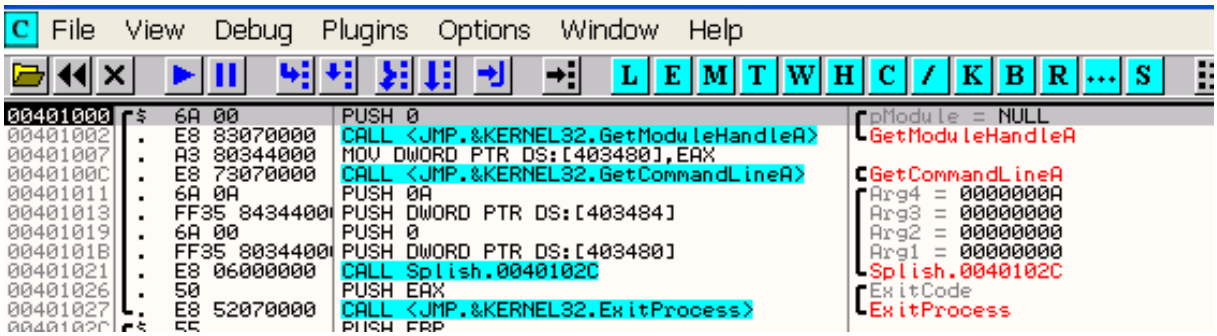
Автор: Рикардо Нарваха, пер. Aquila

Дата: 2007-12-05

Файлы к статье (находится в архиве `wasm_files.zip`: `files/0467/ollydbg15.zip`)

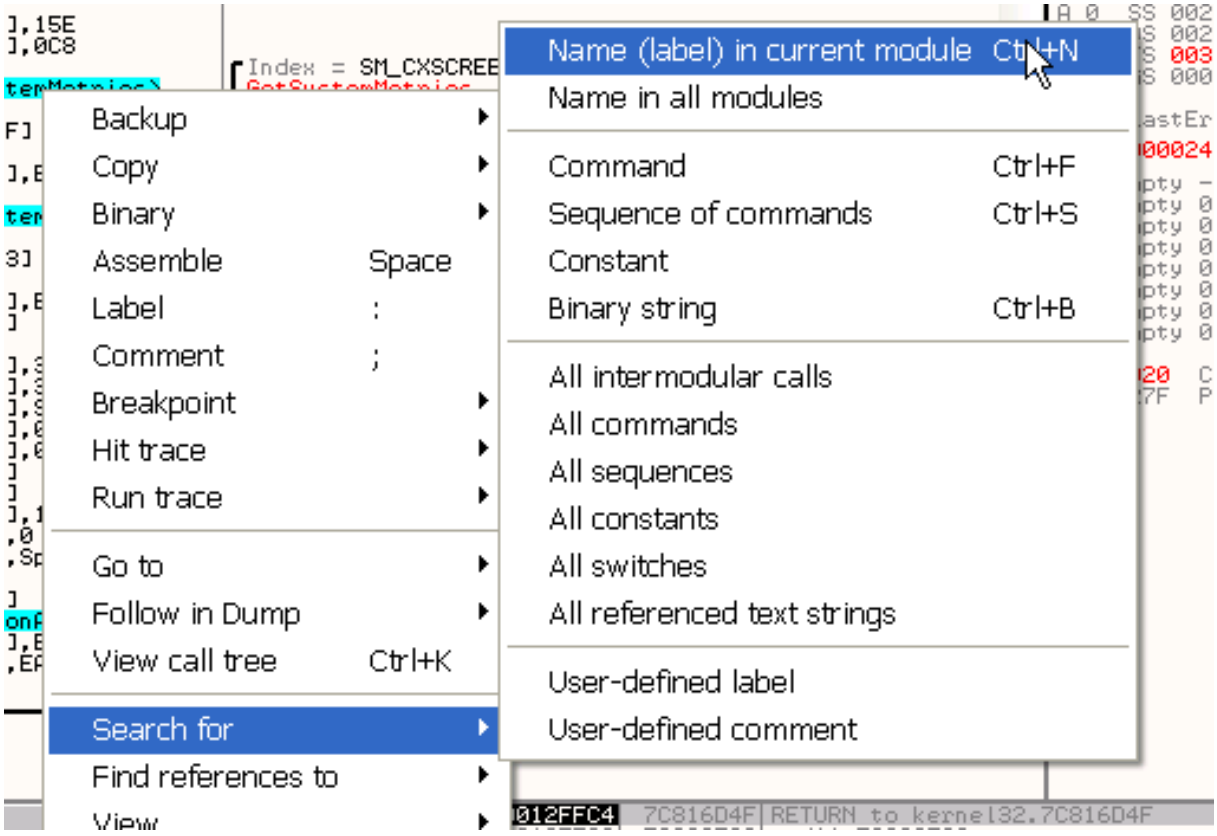
Прежде, чем перейти к последнему из крэкми с жёстко-заданными серийными номерами, рассмотрим тот, который я оставил вам в прошлый раз для самостоятельной работы. Он назывался SPLISH.

Откроем его в OllyDbg.

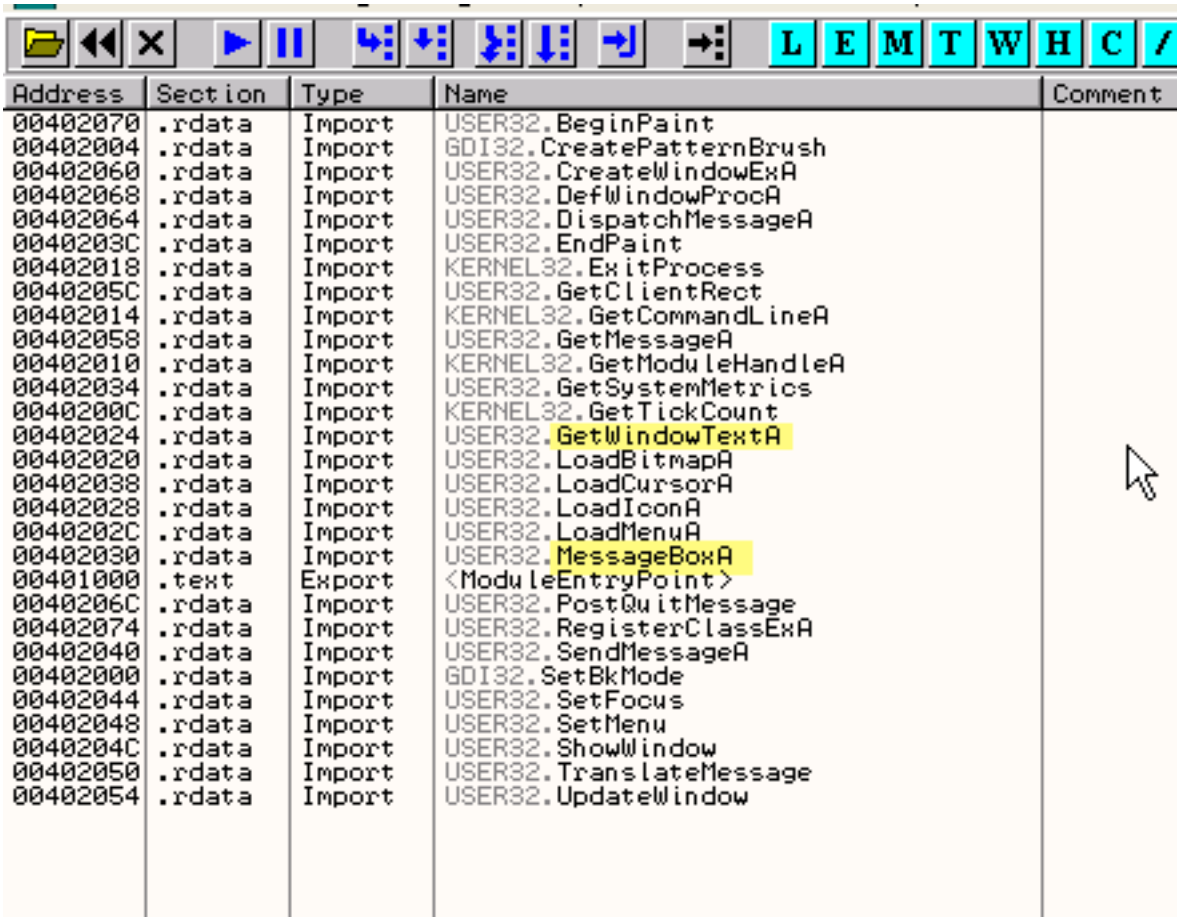


Он открывается на точке входа (ENTRY POINT).

Смотрим используемые API-функции (правая кнопка мыши - SEARCH FOR-NAME (LABEL) IN CURRENT MODULE).

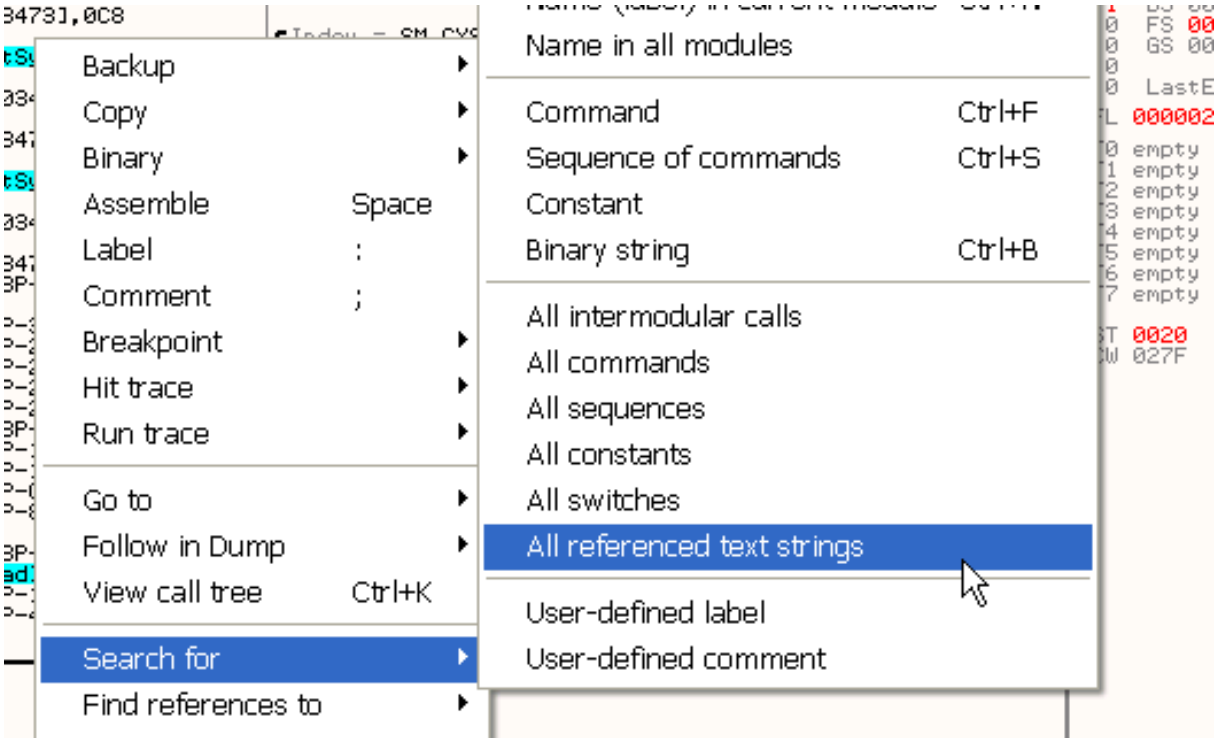


Видим список используемых API-функций.

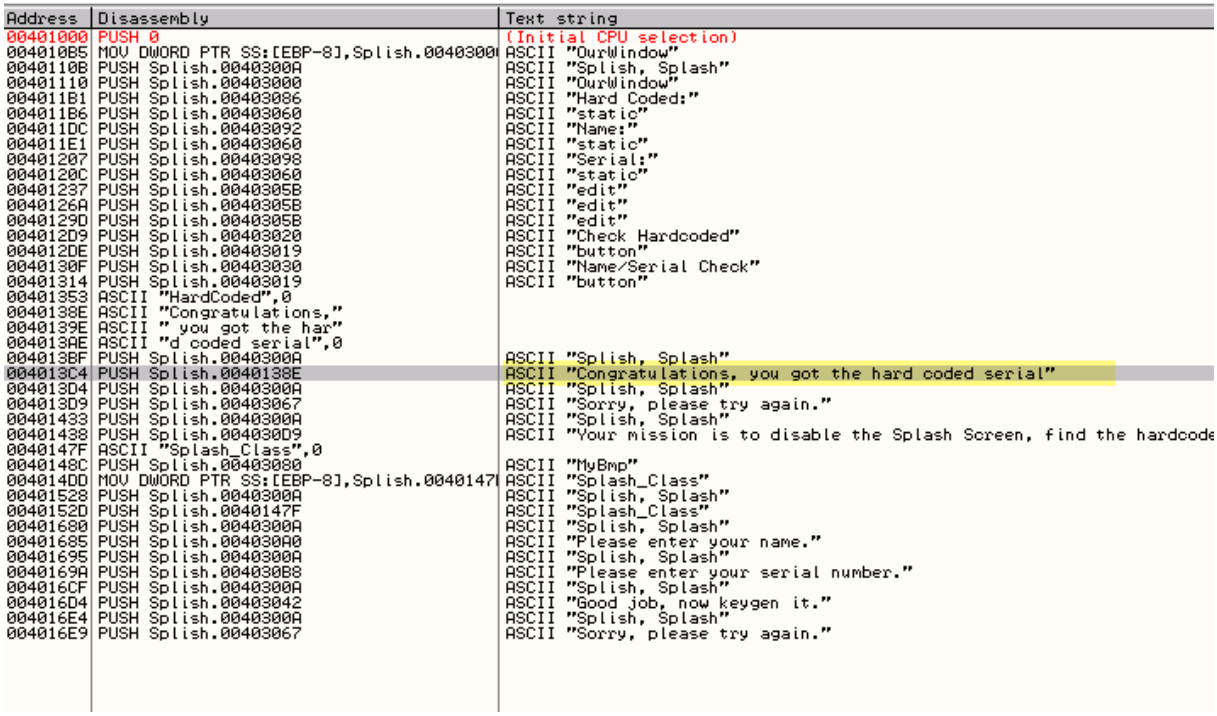


Address	Section	Type	Name	Comment
00402070	.rdata	Import	USER32.BeginPaint	
00402004	.rdata	Import	GDI32.CreatePatternBrush	
00402060	.rdata	Import	USER32.CreateWindowExA	
00402068	.rdata	Import	USER32.DefWindowProcA	
00402064	.rdata	Import	USER32.DispatchMessageA	
0040203C	.rdata	Import	USER32.EndPaint	
00402018	.rdata	Import	KERNEL32.ExitProcess	
0040205C	.rdata	Import	USER32.GetClientRect	
00402014	.rdata	Import	KERNEL32.GetCommandLineA	
00402058	.rdata	Import	USER32.GetMessageA	
00402010	.rdata	Import	KERNEL32.GetModuleHandleA	
00402034	.rdata	Import	USER32.GetSystemMetrics	
0040200C	.rdata	Import	KERNEL32.GetTickCount	
00402024	.rdata	Import	USER32.GetWindowTextA	
00402020	.rdata	Import	USER32.LoadBitmapA	
00402038	.rdata	Import	USER32.LoadCursorA	
00402028	.rdata	Import	USER32.LoadIconA	
0040202C	.rdata	Import	USER32.LoadMenuA	
00402030	.rdata	Import	USER32.MessageBoxA	
00401000	.text	Export	<ModuleEntryPoint>	
0040206C	.rdata	Import	USER32.PostQuitMessage	
00402074	.rdata	Import	USER32.RegisterClassExA	
00402040	.rdata	Import	USER32.SendMessageA	
00402000	.rdata	Import	GDI32.SetBkMode	
00402044	.rdata	Import	USER32.SetFocus	
00402048	.rdata	Import	USER32.SetMenu	
0040204C	.rdata	Import	USER32.ShowWindow	
00402050	.rdata	Import	USER32.TranslateMessage	
00402054	.rdata	Import	USER32.UpdateWindow	

Видим, что используется GetWindowTextA при вводе серийного номера и MessageBoxA для отображения сообщения, правилен ли он или нет. Можем установить BPX на обе функции, но сначала ещё глянем, что покажет список строк, используемых программой.



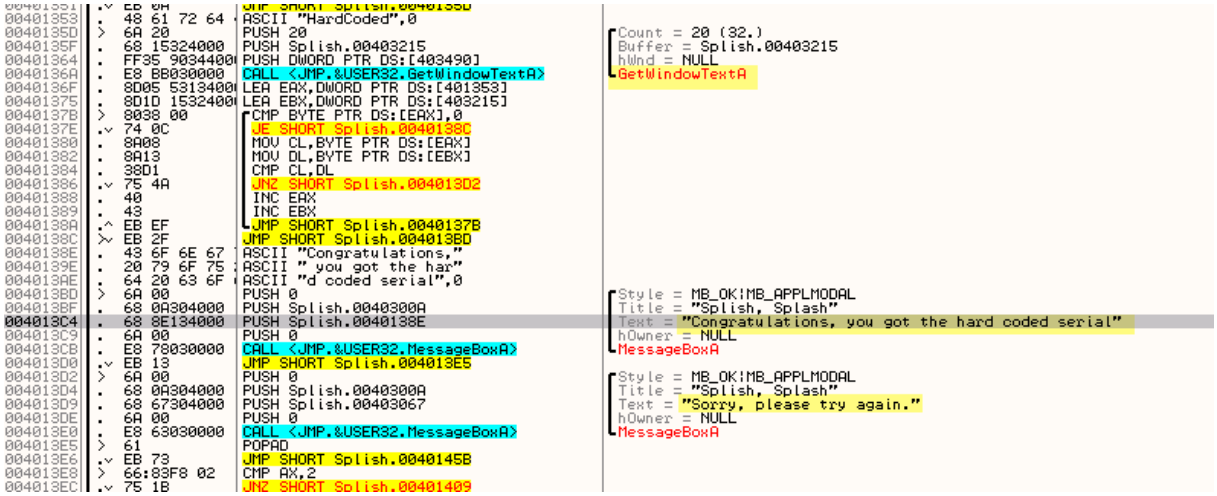
Кликаем правой кнопкой мыши на листинге и выбираем SEARCH FOR-ALL REFERENCED TEXT STRING.



```
Address   Disassembly                                Text string
00401000  PUSH 0                                     (Initial CPU selection)
004010B5  MOV DWORD PTR SS:[EBP-8],Splish.0040300(   ASCII "OurWindow"
0040110B  PUSH Splish.0040300A                       ASCII "Splish, Splash"
00401110  PUSH Splish.00403000                       ASCII "OurWindow"
004011B1  PUSH Splish.00403086                       ASCII "Hard Coded:"
004011B6  PUSH Splish.00403060                       ASCII "static"
004011DC  PUSH Splish.00403092                       ASCII "Name:"
004011E1  PUSH Splish.00403060                       ASCII "static"
00401207  PUSH Splish.00403098                       ASCII "Serial:"
0040120C  PUSH Splish.00403060                       ASCII "static"
00401237  PUSH Splish.0040305B                       ASCII "edit"
0040126A  PUSH Splish.0040305B                       ASCII "edit"
0040129D  PUSH Splish.0040305B                       ASCII "edit"
004012D9  PUSH Splish.00403020                       ASCII "Check Hardcoded"
004012DE  PUSH Splish.00403019                       ASCII "button"
0040130F  PUSH Splish.00403030                       ASCII "Name/Serial Check"
00401314  PUSH Splish.00403019                       ASCII "button"
00401353  ASCII "HardCoded",0
0040138E  ASCII "Congratulations,"
0040139E  ASCII " you got the har"
004013AE  ASCII "d coded serial",0
004013BF  PUSH Splish.0040300A                       ASCII "Splish, Splash"
004013C4  PUSH Splish.0040138E                       ASCII "Congratulations, you got the hard coded serial"
004013D4  PUSH Splish.0040300A                       ASCII "Splish, Splash"
004013D9  PUSH Splish.00403067                       ASCII "Sorry, please try again."
00401433  PUSH Splish.0040300A                       ASCII "Splish, Splash"
00401438  PUSH Splish.004030D9                       ASCII "Your mission is to disable the Splash Screen, find the hardcode
0040147F  ASCII "Splash_Class",0
0040148C  PUSH Splish.00403080                       ASCII "MyBmp"
004014DD  MOV DWORD PTR SS:[EBP-8],Splish.0040147(   ASCII "Splash_Class"
00401528  PUSH Splish.0040300A                       ASCII "Splish, Splash"
0040152D  PUSH Splish.0040147F                       ASCII "Splash_Class"
00401680  PUSH Splish.0040300A                       ASCII "Splish, Splash"
00401685  PUSH Splish.004030A0                       ASCII "Please enter your name."
00401695  PUSH Splish.0040300A                       ASCII "Splish, Splash"
0040169A  PUSH Splish.004030B8                       ASCII "Please enter your serial number."
004016CF  PUSH Splish.0040300A                       ASCII "Splish, Splash"
004016D4  PUSH Splish.00403042                       ASCII "Good job, now keygen it."
004016E4  PUSH Splish.0040300A                       ASCII "Splish, Splash"
004016E9  PUSH Splish.00403067                       ASCII "Sorry, please try again."
```

Здесь видим текст окошка с поздравлением, которое отображается, если был введён правильный серийный номер, поэтому кликнем два раза на этом тексте, чтобы увидеть, где он используется. Мы окажемся в окрестностях того места, где к нему есть обращение.

Попадаем в область, где ведётся работа с серийным номером.



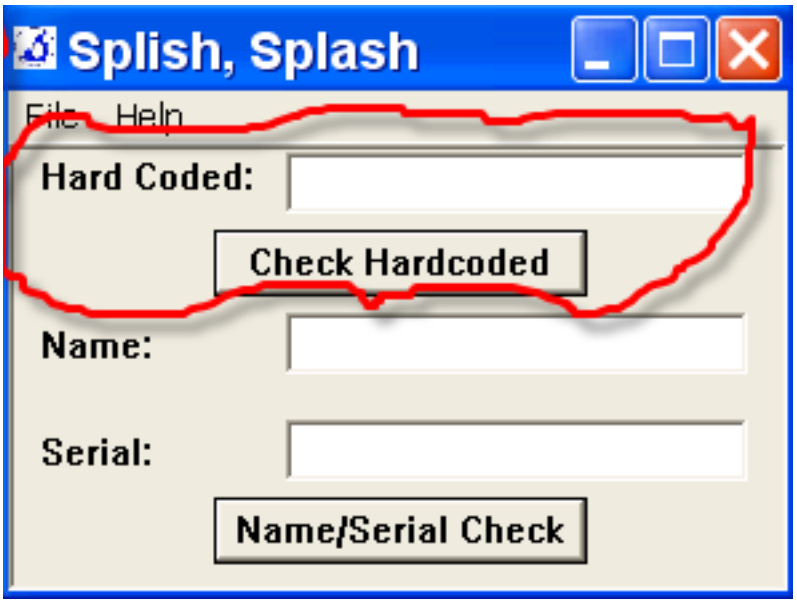
```
00401353  .  48 61 72 64  ASCII "HardCoded",0
0040135D  >  6A 20        PUSH 20                    Count = 20 (32.)
0040135F  .  68 15324000  PUSH Splish.00403215       Buffer = Splish.00403215
00401364  .  FF35 9034400 PUSH DWORD PTR DS:[403490] hWnd = NULL
0040136A  .  E8 BB030000  CALL <JMP.&USER32.GetWindowTextA>  GetWindowTextA
0040136F  .  8D05 5313400 LEA EAX,DWORD PTR DS:[401353]
00401375  .  8D1D 1532400 LEA EBX,DWORD PTR DS:[403215]
0040137B  >  8038 00      CMP BYTE PTR DS:[EAX],0
0040137E  .v 74 0C        JE SHORT Splish.0040138C
00401380  .  8A08         MOV CL,BYTE PTR DS:[EAX]
00401382  .  8A13         MOV DL,BYTE PTR DS:[EBX]
00401384  .  38D1         CMP CL,DL
00401386  .v 75 4A        JNZ SHORT Splish.004013D2
00401388  .  40           INC EAX
00401389  .  43           INC EBX
0040138A  .^ EB EF        JMP SHORT Splish.0040137B
0040138C  >v EB 2F        JMP SHORT Splish.004013BD
0040138E  .  43 6F 6E 67  ASCII "Congratulations,"
0040139E  .  20 79 6F 75  ASCII " you got the har"
004013AE  .  64 20 63 6F  ASCII "d coded serial",0
004013BD  >  6A 00        PUSH 0                     Style = MB_OK|MB_APPLMODAL
004013BF  .  68 0A304000  PUSH Splish.0040300A       Title = "Splish, Splash"
004013C4  .  68 8E134000  PUSH Splish.0040138E       Text = "Congratulations, you got the hard coded serial"
004013C9  .  6A 00        PUSH 0                     hOwner = NULL
004013CB  .  E8 78030000  CALL <JMP.&USER32.MessageBoxA>  MessageBoxA
004013D0  .v EB 13        JMP SHORT Splish.004013E5
004013D2  >  6A 00        PUSH 0                     Style = MB_OK|MB_APPLMODAL
004013D4  .  68 0A304000  PUSH Splish.0040300A       Title = "Splish, Splash"
004013D9  .  68 67304000  PUSH Splish.00403067       Text = "Sorry, please try again."
004013DE  .  6A 00        PUSH 0                     hOwner = NULL
004013E0  .  E8 63030000  CALL <JMP.&USER32.MessageBoxA>  MessageBoxA
004013E5  >  61           POPAD
004013E6  .v EB 73        JMP SHORT Splish.0040145B
004013E8  >  66:83F8 02   CMP AX,2
004013EC  .v 75 1B        JNZ SHORT Splish.00401409
```

Видим ввод текста с помощью функции GetWindowTextA и два MessageBoxA с разным текстом: правилен ли серийный номер или нет.

Ставим здесь на вызов API-функции GetWindowTextA BPX, чтобы начать с того места, где вводится серийный номер.

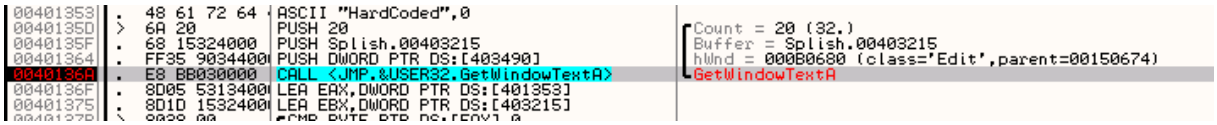
```
00401351  .v EB 0A        JMP SHORT Splish.0040135D
00401353  .  48 61 72 64  ASCII "HardCoded",0
0040135D  >  6A 20        PUSH 20                          ; Count = 20 (32.)
0040135F  .  68 15324000  PUSH Splish.00403215             ; Buffer = Splish.00403215
00401364  .  FF35 9034400 PUSH DWORD PTR DS:[403490]       ; hWnd = NULL
0040136A  .  E8 BB030000  CALL <JMP.&USER32.GetWindowTextA> ; GetWindowTextA
0040136F  .  8D05 5313400 LEA EAX,DWORD PTR DS:[401353]
00401375  .  8D1D 1532400 LEA EBX,DWORD PTR DS:[403215]
0040137B  >  8038 00      CMP BYTE PTR DS:[EAX],0
0040137E  .v 74 0C        JE SHORT Splish.0040138C
00401380  .  8A08         MOV CL,BYTE PTR DS:[EAX]
00401382  .  8A13         MOV DL,BYTE PTR DS:[EBX]
00401384  .  38D1         CMP CL,DL
00401386  .v 75 4A        JNZ SHORT Splish.004013D2
00401388  .  40           INC EAX
00401389  .  43           INC EBX
0040138A  .^ EB EF        JMP SHORT Splish.0040137B
0040138C  >v EB 2F        JMP SHORT Splish.004013BD
0040138E  .  43 6F 6E 67  ASCII "Congratulations,"
0040139E  .  20 79 6F 75  ASCII " you got the har"
004013AE  .  64 20 63 6F  ASCII "d coded serial",0
004013BD  >  6A 00        PUSH 0                           ; Style = MB_OK|MB_APPLMODAL
004013BF  .  68 0A304000  PUSH Splish.0040300A             ; Title = "Splish, Splash"
004013C4  .  68 8E134000  PUSH Splish.0040138E             ; Text = "Congratulations, you got the hard coded serial"
004013C9  .  6A 00        PUSH 0                           ; hOwner = NULL
004013CB  .  E8 78030000  CALL <JMP.&USER32.MessageBoxA>   ; MessageBoxA
004013D0  .v EB 13        JMP SHORT Splish.004013E5
004013D2  >  6A 00        PUSH 0                           ; Style = MB_OK|MB_APPLMODAL
004013D4  .  68 0A304000  PUSH Splish.0040300A             ; Title = "Splish, Splash"
004013D9  .  68 67304000  PUSH Splish.00403067             ; Text = "Sorry, please try again."
004013DE  .  6A 00        PUSH 0                           ; hOwner = NULL
004013E0  .  E8 63030000  CALL <JMP.&USER32.MessageBoxA>   ; MessageBoxA
004013E5  >  61           POPAD
004013E6  .v EB 73        JMP SHORT Splish.0040145B
```

Запускаем крэкми с помощью F9 или RUN.

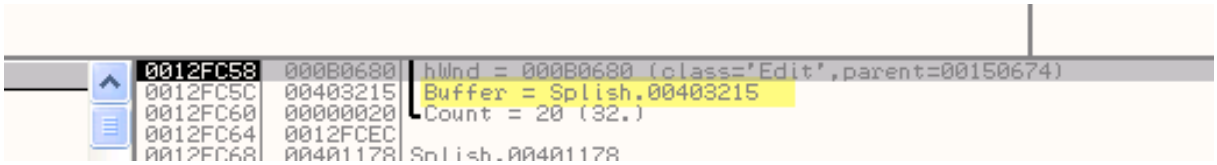


Так как пока что мы можем решить только ту часть, которая относится к жёстко-заданным серийным номерам, пишем в поле HARDCODED, которое находится наверху, неверный серийник или нажимаем на кнопку CHECK HARDCODED.

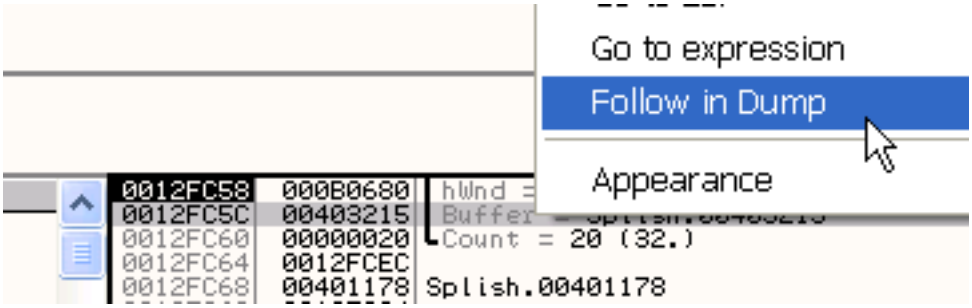
Останавливаемся на BPX, установленный ранее.

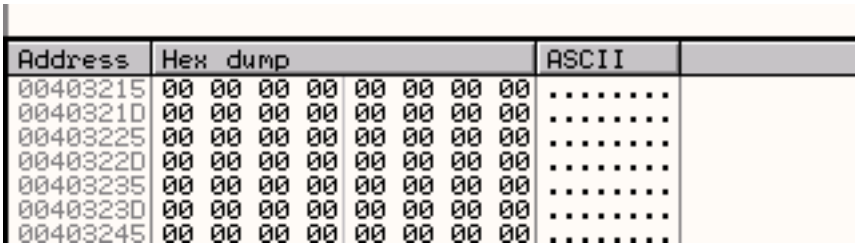


Пояснения показывают нам, что находится в стеке, и это даёт нам сразу понять, что буфер, в котором будет сохранён введенный серийный номер находится по адресу 403215.

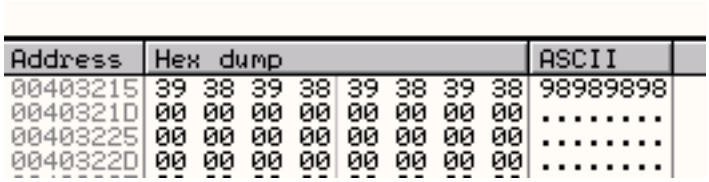


Посмотрим через DUMP, что находится в буфере.





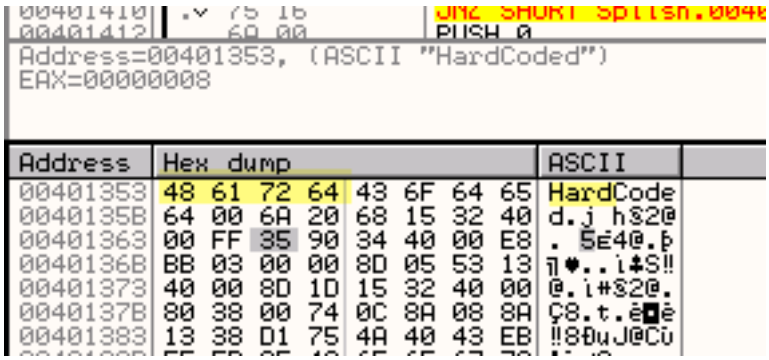
Вот этот буфер, куда будет сохранён введённый серийный номер. Нажимаем F8, чтобы выполнить вызов API-функции.



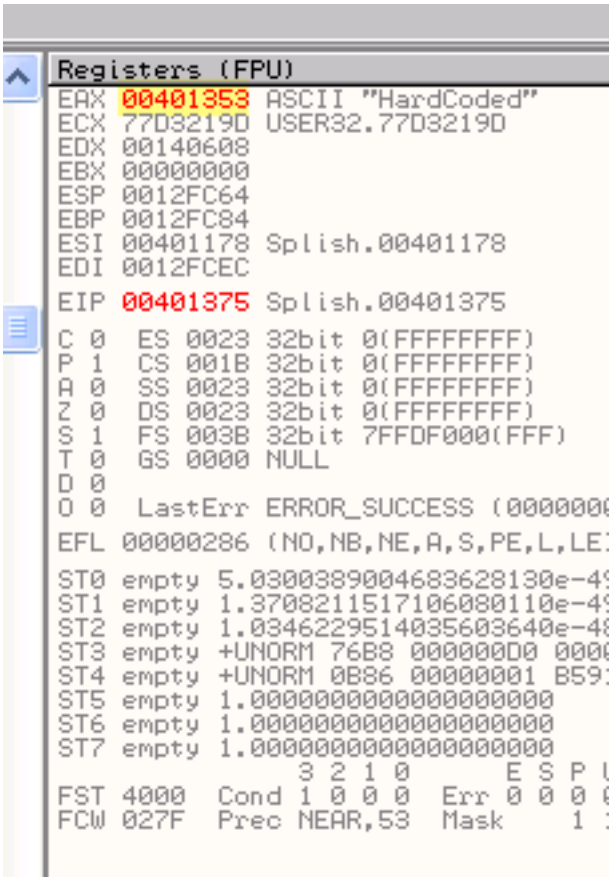
Так как использовали F8, выполняется вся функция, так что неправильный серийник уже находится в буфере.

Ок, следующая строка переместит 401353 в EAX (помните, что LEA не перемещает содержимое ячейки памяти по заданному адресу, а только результат того, что находится в квадратных скобках; в данном случае это будет 401353). Видим в DUMP'е, что по вышеуказанному адресу (кликаем на строке правой кнопкой мыши и выбираем FOLLOW IN DUMP-MEMORY ADDRESS, и это нам покажет, что находится по адресу 401353).

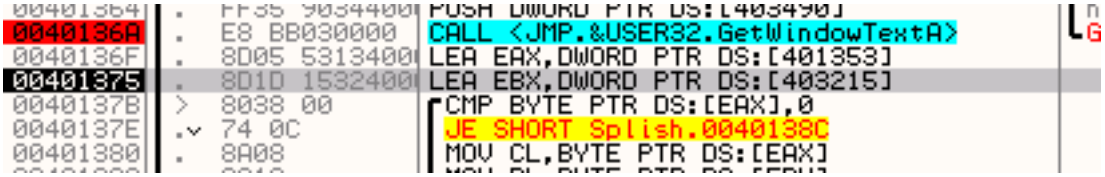
По адресу 401353 находится строка "HARDCODED"; нажимаем F7, чтобы выполнилась LEA.



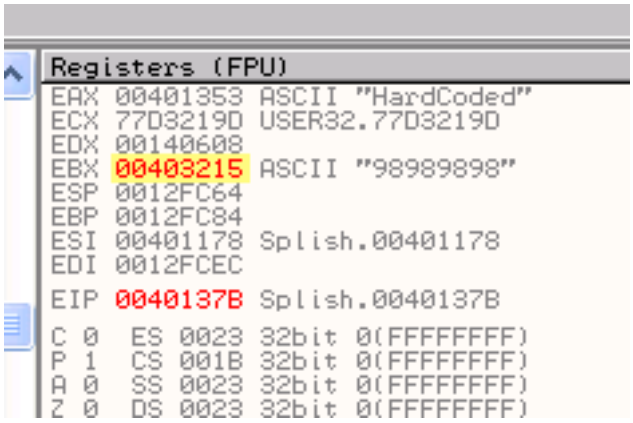
После выполнения в EAX остаётся 401353, а с правой стороны появляется пояснение, что данный адресу указывает на строку "HARDCODED".



Следующая строка загружает в EBX другой адрес, в данному случае это 403215.

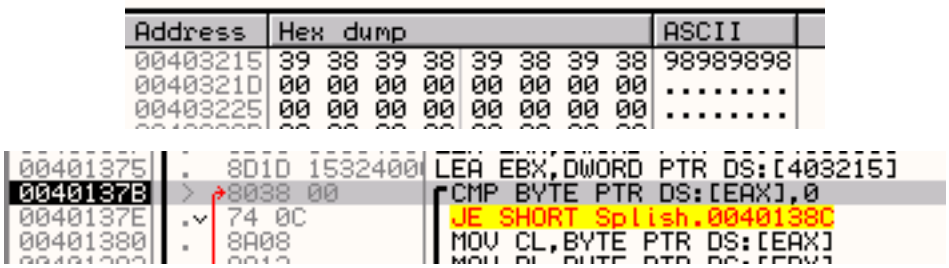


После нажатия F7 EBX содержит значение 403215.

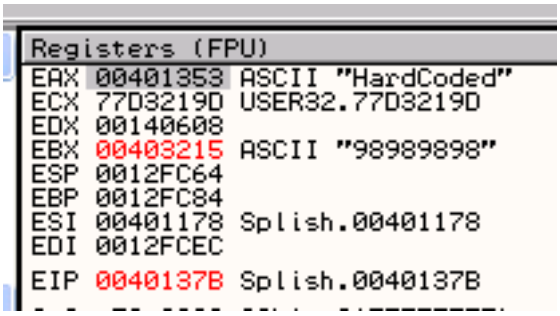


И справа Олли показывает нам, что этот адрес указывает на строку “98989898”, являющуюся введёнными нами неправильным серийным номером, который мы посмотрим в DUMP’е так же, как сделали это для предыдущей инструкции.

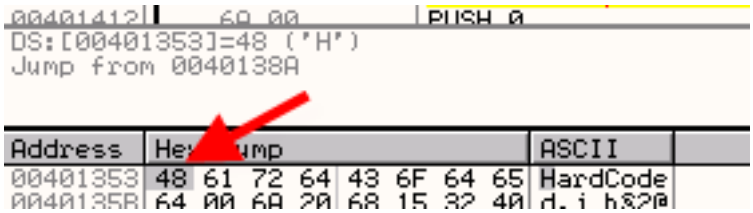
Здесь 403215 указывает на наш неправильный серийный номер.



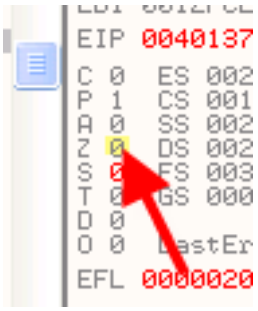
Следующая строка проверяет, равен ли нулю первый байт содержимого EAX, который равен 401353.



Так как мы уже смотрели содержимое по этому адресу через DUMP, то уже знаем, что там находятся байты строки “HARDCODED”.

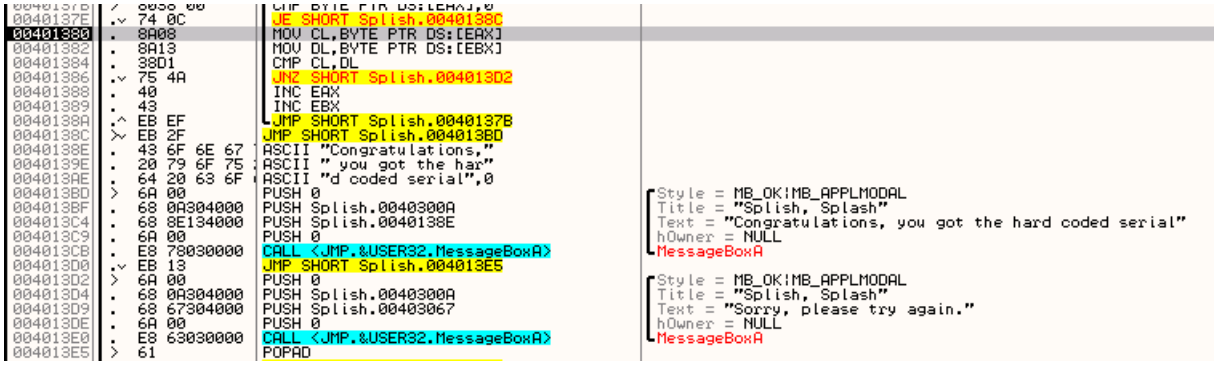


Первый байт в данном случае равен 48, пояснения OllyDbg нам также говорят, что это соответствует символу “H” в кодировке ASCII, а это первая буква слова “HARDCODED”. Так как это не ноль, то продолжаем.



Так как сравнение не активирует флаг Z (операнды не равны друг другу), JE не совершает перехода (помните, что JE срабатывает только тогда, когда активируется флаг Z),

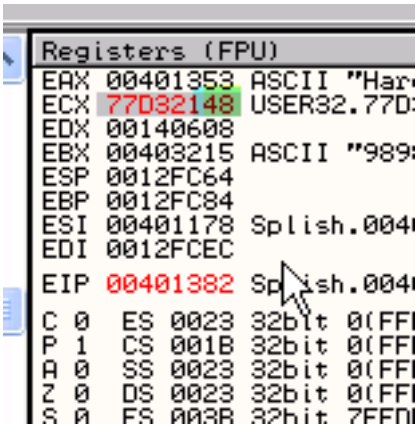
Нажимаем на F7 и продолжаем.



Здесь очень хорошо видно, что сначала будет перемещён первый байт [EAX], равный 48, что соотвтествует слову “HARDCODED”, а на следующей строке перемещается [EBX], являющийся первым байтом неправильного серийного номера, после чего оба сравниваются и если они не равны, то происходит переход на 4013D2, где выводится сообщение “SORRY, PLEASE TRY AGAIN”.

Проверим это следующим образом.

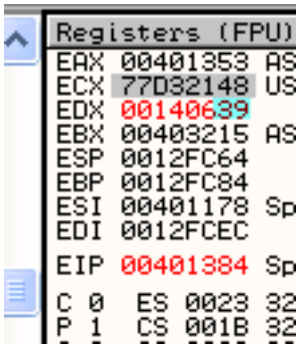
Нажмём F7 и переместим в CL значение 48.



CL, как мы помним, это регистр, соответствующий двум последним цифрам ECX, здесь перемещаем 48, и вот, что получится в результате.

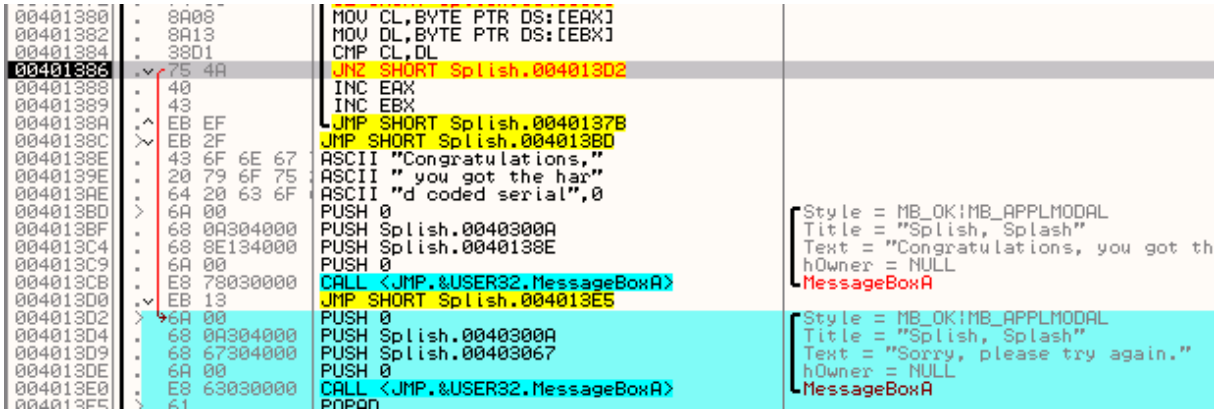
Затем в DL перемещается первый байт неправильного серийного номера.

Выполним строку с помощью F7, видим, что в DL находится значение 39.

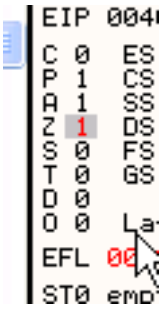


Теперь CL сравнивается с DL.

Пояснение OllyDbg не оставляет сомнение: сравнивается 39 (что есть 9, т.е. первый символ введённого мной серийного номера с H, являющейся первым символом строки “HARDCODED”).

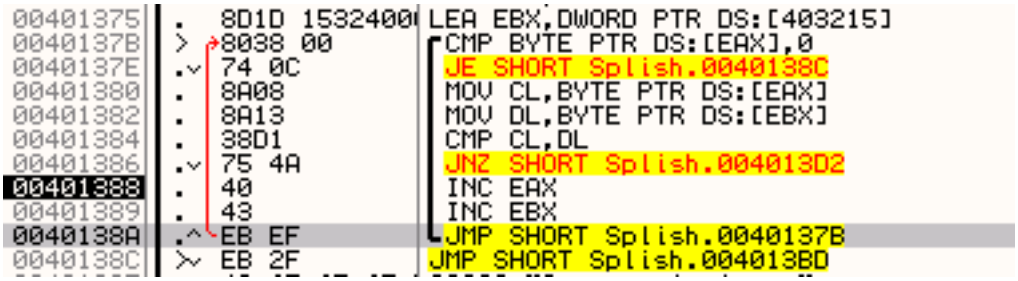


Видим, что они не равны, поэтому совершается переход на сообщение с ошибкой, а поведение если бы они были равны можем сэмулировать, изменив значение флага Z, сделав по нему двойной щелчок.



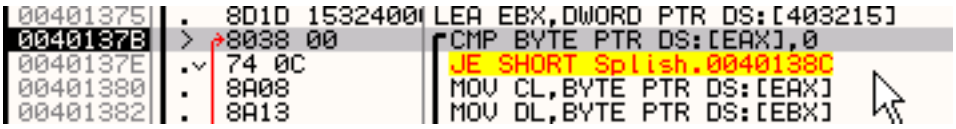
Теперь Z равен 1, как если бы в сравнении двух байтов они были равны и их вычитание друг от друга дало бы ноль, активировав данный флаг.

Видим, что в этом случае происходит увеличение EAX и EBX и переход с помощью JMP на начало цикла.

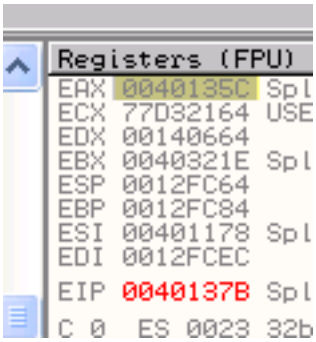


Теперь EAX указывает на второй байт строки “HARDCODED” и видим, что цикл повторяет побайтовое сравнение, пока [EAX] не будет указывать на ноль, то есть пока не завершится строка “HARDCODED” (в конце строки за последним символом находится ноль).

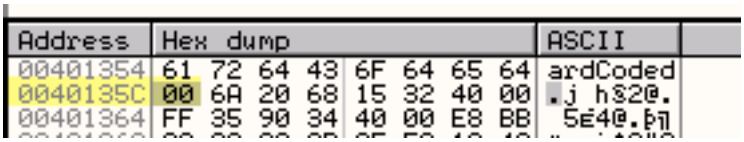
Тогда после увеличения значения EAX и EBX на 1 будут считаны и сравнены друг с другом вторые байты, если равны, то цикл повторится, но уже будут рассматриваться третьи байты, а когда слово “HARDCODED” закончится, то если все сравнения на равенство CL с DL прошли успешно, перехода на окошко с ошибкой не произойдёт, и попадём вот куда:



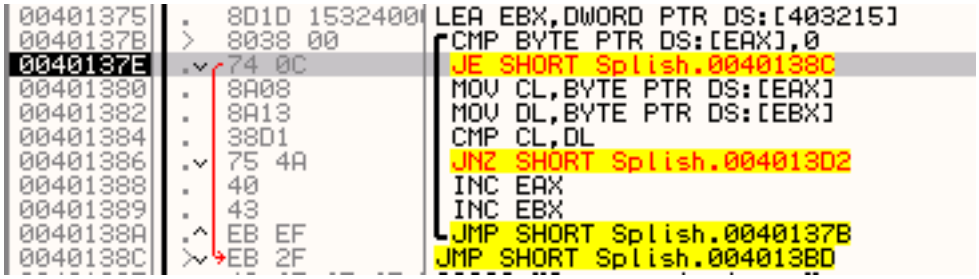
Здесь EAX указывает не на какой-либо из байтов слова “HARDCODED”, а на ноль, так как строка уже закончилась.



Вот и тот самый ноль, а это значит, что сработает соответствующая проверка.



Срабатывает JE, и мы выходим из цикла.

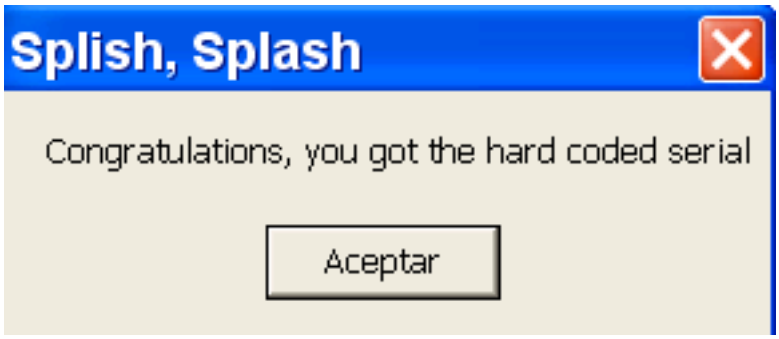


И попадаем в то место, где отображается сообщение о правильно введённом номере (так как поменяли значение флага Z при побайтовом сравнении, хе-хе).

```
00401389  .  43          INC EBX
0040138A  .^ EB EF       JMP SHORT Splish.0040137B
0040138C  >v EB 2F       JMP SHORT Splish.004013BD
0040138E  .  43 6F 6E 67 ASCII "Congratulations,"
0040139E  .  20 79 6F 75 ASCII " you got the har"
004013AE  .  64 20 63 6F ASCII "d coded serial",0
004013BD  > 6A 00        PUSH 0                          Style = MB_OK|MB_APPLMODAL
004013BF  .  68 0A304000 PUSH Splish.0040300A            Title = "Splish, Splash"
004013C4  .  68 8E134000 PUSH Splish.0040138E            Text = "Congratulations, you got the hard coded serial"
004013C9  .  6A 00       PUSH 0                          hOwner = NULL
004013CB  .  E8 78030000 CALL <JMP.&USER32.MessageBoxA>  MessageBoxA
004013D0  .v EB 13       JMP SHORT Splish.004013E5
```

Очевидно, что менять значение флага Z пришлось при каждом сравнении CL с DL, чтобы программа поверила в одинаковость обоих серийников и не перешла на сообщение об ошибке и не вышла, но в любом случае мы теперь знаем, что правильный серийный номер – это слово “HardCoded” (с соблюдением прописных и строчных букв, так как у них разный ASCII-код).

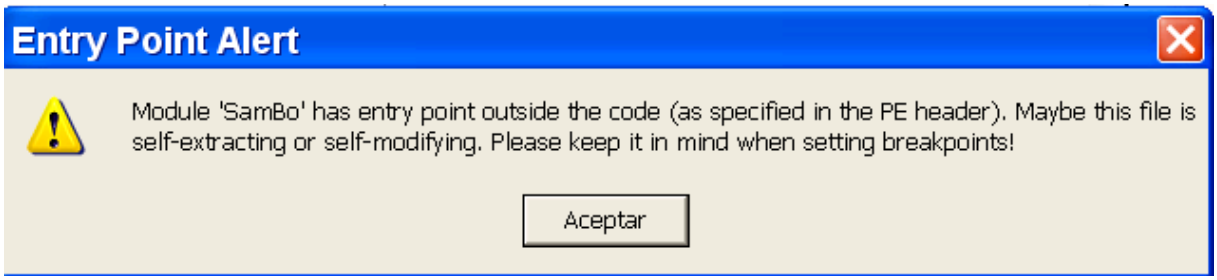
Уберём все точки останова, введём найденный серийный номер и нажмём кнопку “CHECK HARDCODED”.



Ещё одна победа, поздравляю.

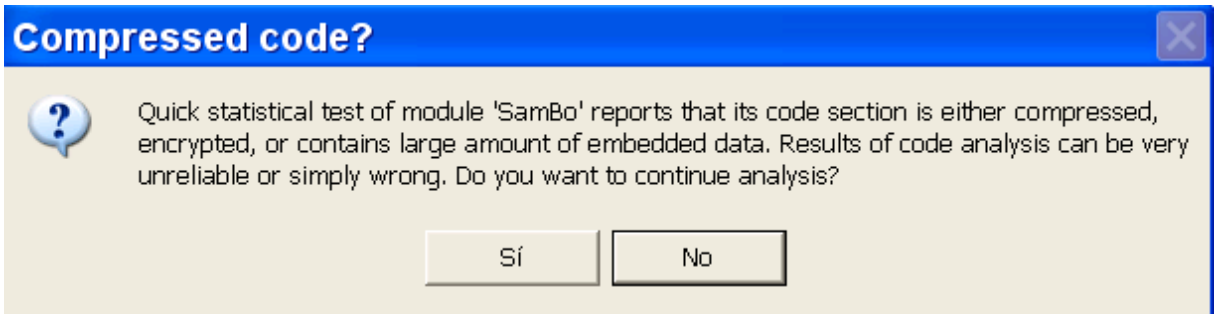
Хорошо, тогда переходим к последнему крэкми с жёстко заданным серийным номером и заканчиваем на этом с ними, а в главе 16 перейдём к следующей теме.

Алгоритм следующего крэкми, который назвается SAMBO, отличается от предыдущих. Откроем его в OllyDbg.

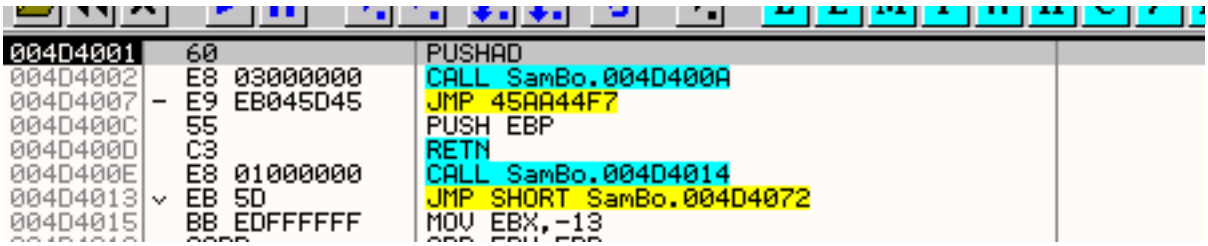


Выходим из этого окошка, которое говорит нам, что файл, возможно, самораспаковываемый или автомодифицируемый. Такие файлы на крэкерском жаргоне мы называем пакованными, запакованными или сжатыми. В дальнейшем мы изучим их подробно, но пока запустим его в OllyDbg и, несмотря на то, что он запакован, попробуем найти серийный номер.

Соглашаемся с предупреждением OllyDbg и прибываем в EP.

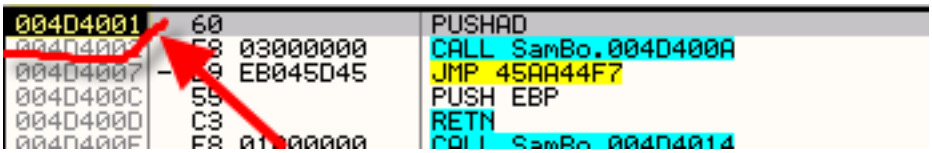


С помощью ещё одного окошка OllyDbg сообщает нам, что программа, возможно, запакована или зашифрована, поэтому анализ её будет настолько же неэффективным, как пепельница в мотоцикле, так как программа будет распаковывать себя во время выполнения, поэтому пока что выберем “Нет”.

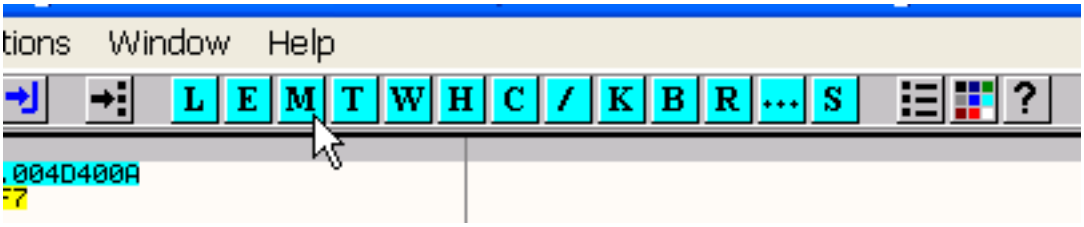


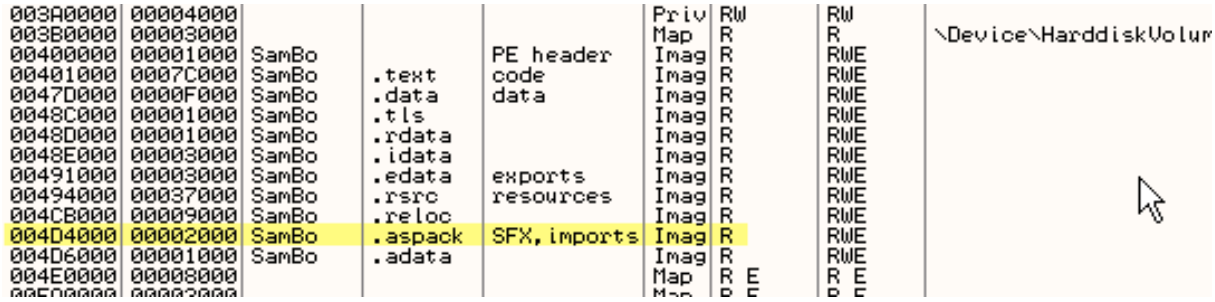
Уже на первых строках листинга мы можем заметить, что этот крэкми выполняется не так, как его незапакованные сородичи, чья первая секция начинается с 401000.

Здесь точка входа равна 4d4001.



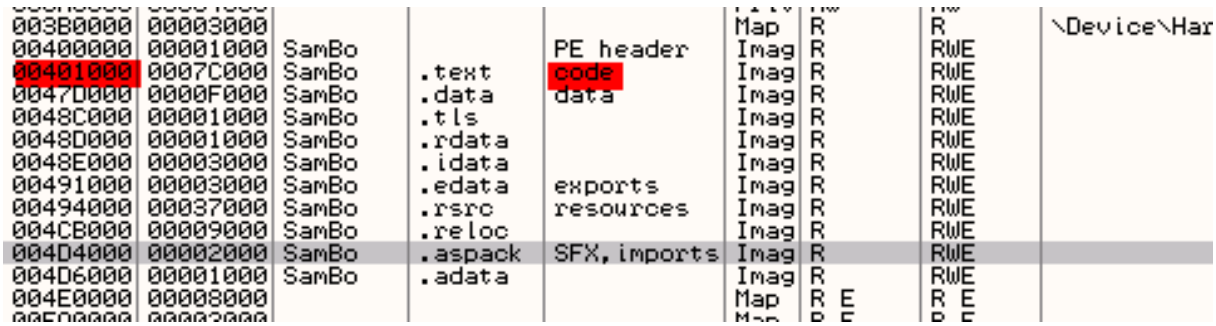
Посмотрим, является ли секция исполняемой, для этого идём VIEW-MEMORY или нажмём клавишу M.





Видим, что программа стартует в секции, начинающейся с 4D4000, размер которой равен 2000 (в шестнадцатеричной системе), и поэтому 4d4001 – это адрес, который относится к данной секции.

Это и есть то, о чём говорила OllyDbg – точка входа находится за пределами секции code.

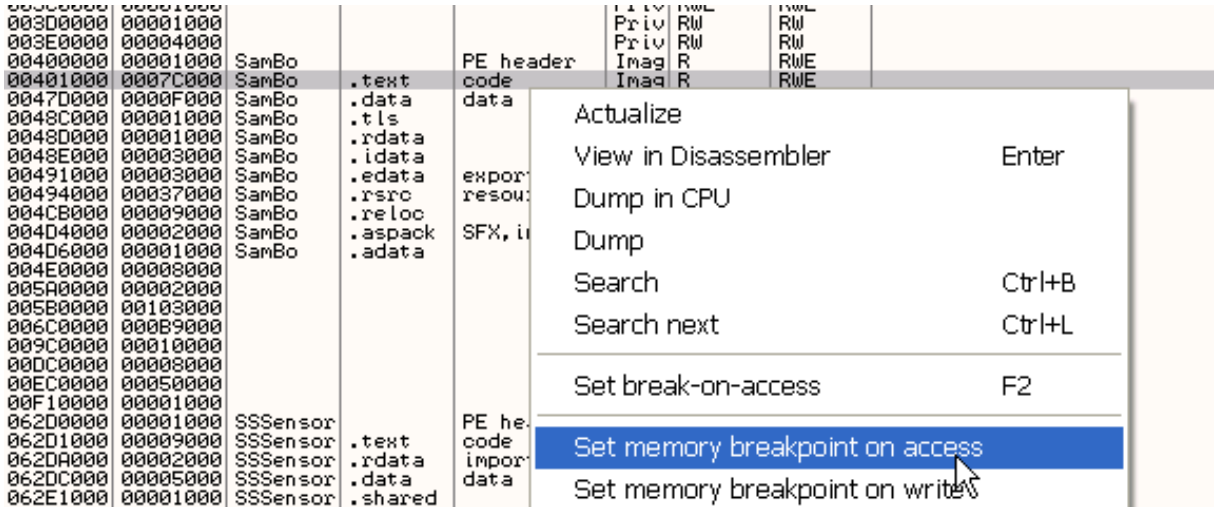


Секция CODE начинается с 401000, а выполнение начинается с адреса 4D4001, который относится к другой секции. OllyDbg нам сообщила, что это характеристика многих запакованных программ.

Секция, в которой находится точка входа, соответствует распаковщику, и как только он отработает и сделает то, что должен сделать, он сделает переход на на секцию code, где и будет выполнена сама программа.

А раз так, нажимаем F9.

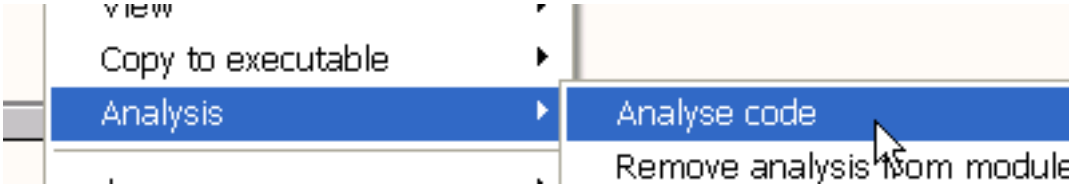
Мы уже знаем, что когда появится окошко крэкми для ввода серийника, программа уже распакована в памяти и выполняется в секции code, поэтому можем установить на эту секцию BPM ON ACCESS , чтобы остановить выполнение на её первой строке.



При попытке вернуться в окошко крэкми OllyDbg остановится в секции code (т.е. первой секции после PE-заголовка).

```
0045C854   807D FB 00      CMP BYTE PTR SS:[EBP-5],0
0045C858   74 05           JE SHORT SamBo.0045C85F
0045C85A   E8 F5010200     CALL SamBo.0047CA54                  JMP to USER32.WaitMessage
0045C85F   33C0            XOR EAX,EAX
0045C861   5A              POP EDX
0045C862   59              POP ECX
0045C863   59              POP ECX
0045C864   64:8910         MOV DWORD PTR FS:[EAX],EDX
0045C867   68 81C84500     PUSH SamBo.0045C881
0045C86C   8D45 F0         LEA EAX,DWORD PTR SS:[EBP-10]
0045C86F   BA 02000000     MOV EDX,2
```

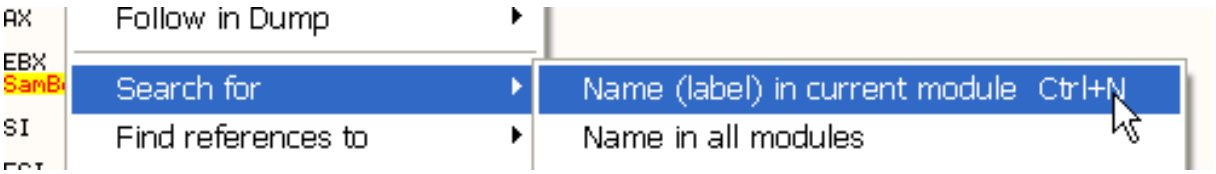
Видим, что теперь программа распакована в памяти, можем её анализировать. Кликнем правую кнопку мыши в листинге.

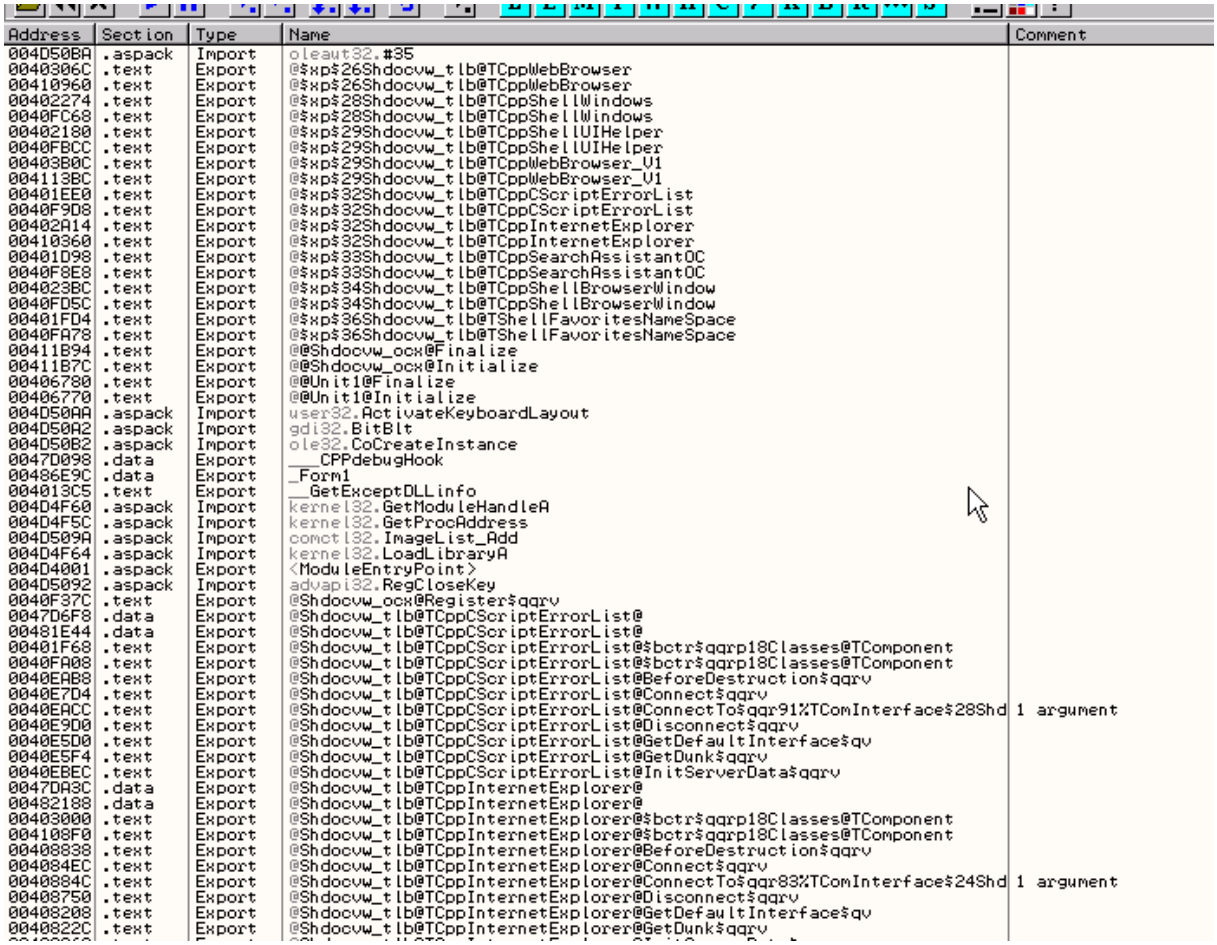



```
0045C82B   .  8B45 FC      MOV EAX,DWORD PTR SS:[EBP-4]
0045C82E   .  E8 29F9FFFF  CALL SamBo.0045C15C
0045C833   .  E8 E46FFDFF  CALL SamBo.00433810
0045C838   >  E8 A1FA0100  CALL SamBo.0047C2DE                  GetCurrentThreadId
0045C83D   .  8B15 906C480 MOV EDX,DWORD PTR DS:[486C90]        SamBo.00487458
0045C843   .  3B02         CMP EAX,DWORD PTR DS:[EDX]
0045C845   .v 75 0D        JNZ SHORT SamBo.0045C854
0045C847   .  E8 9827FCFF  CALL SamBo.0041EFE4
0045C84C   .  84C0         TEST AL,AL
0045C84E   .v 74 04        JE SHORT SamBo.0045C854
0045C850   .  C645 FB 00   MOV BYTE PTR SS:[EBP-5],0
0045C854   >  807D FB 00   CMP BYTE PTR SS:[EBP-5],0
0045C858   .v 74 05        JE SHORT SamBo.0045C85F
0045C85A   .  E8 F5010200  CALL SamBo.0047CA54                  WaitMessage
0045C85F   >  33C0         XOR EAX,EAX
0045C861   .  5A           POP EDX
0045C862   .  59           POP ECX
0045C863   .  59           POP ECX
0045C864   .  64:8910      MOV DWORD PTR FS:[EAX],EDX
0045C867   .  68 81C84500  PUSH SamBo.0045C881
0045C86C   >  8D45 F0      LEA EAX,DWORD PTR SS:[EBP-10]
0045C86F   .  BA 02000000  MOV EDX,2
0045C874   .  E8 EB72FDFF  CALL SamBo.00433B64
0045C879   .  C3           RETN
0045C87A   .^ E9 E96EFDFF  JMP SamBo.00433768
0045C87F   .^ EB EB        JMP SHORT SamBo.0045C86C
0045C881   .  5F           POP EDI
0045C882   .  5E           POP ESI
0045C883   .  5B           POP EBX
0045C884   .  8BE5         MOV ESP,EBP
0045C886   .  5D           POP EBP
0045C887   .  C3           RETN
0045C888   $  53           PUSH EBX
0045C889   .  56           PUSH ESI
0045C88A   .  57           PUSH EDI
0045C88B   .  8BFA         MOV EDI,EDX
0045C88D   .  A1 047C4800  MOV EAX,DWORD PTR DS:[487C04]
0045C892   .  E8 C9D3FFFF  CALL SamBo.00459C60
```

Видим, что OllyDbg прекрасно показывает нам проанализированный код.

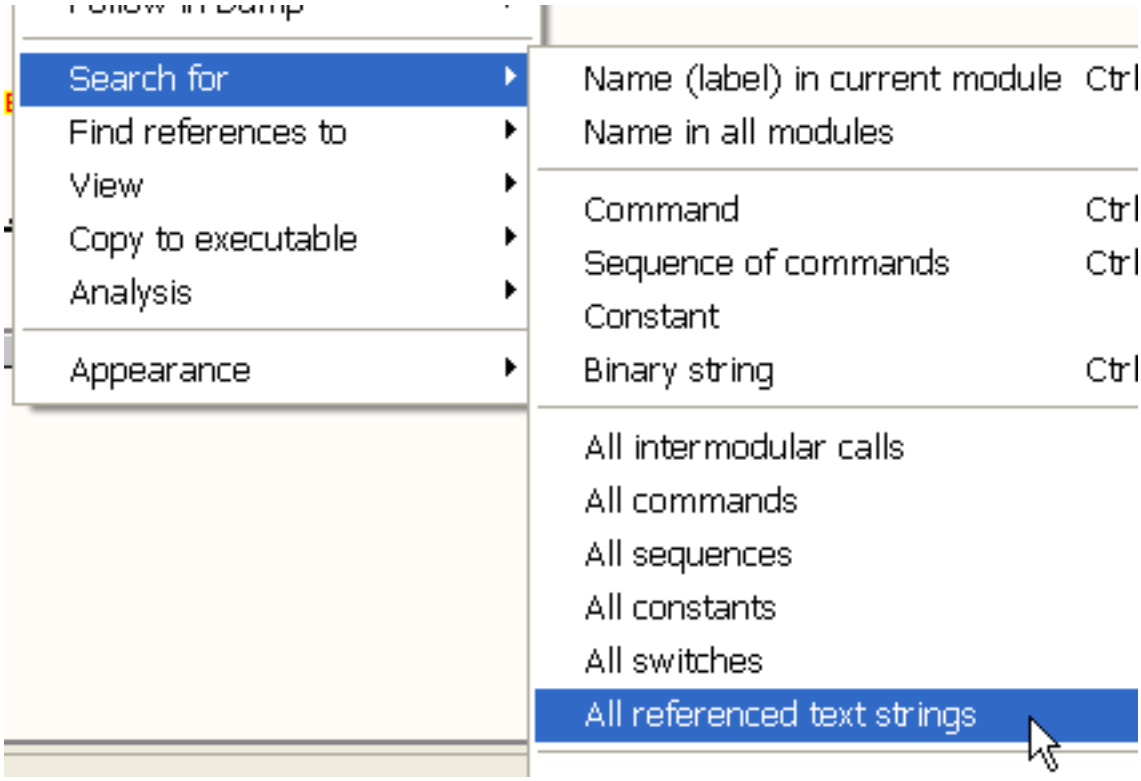
Теперь, когда мы находимся в секции code, можем посмотреть, какие api используются программой – то, что мы не могли посмотреть в начале, так как в тот момент нам показывались только API-функции, используемые распаковщиком, а не те, которые используются программой; сейчас же никаких проблем нет.



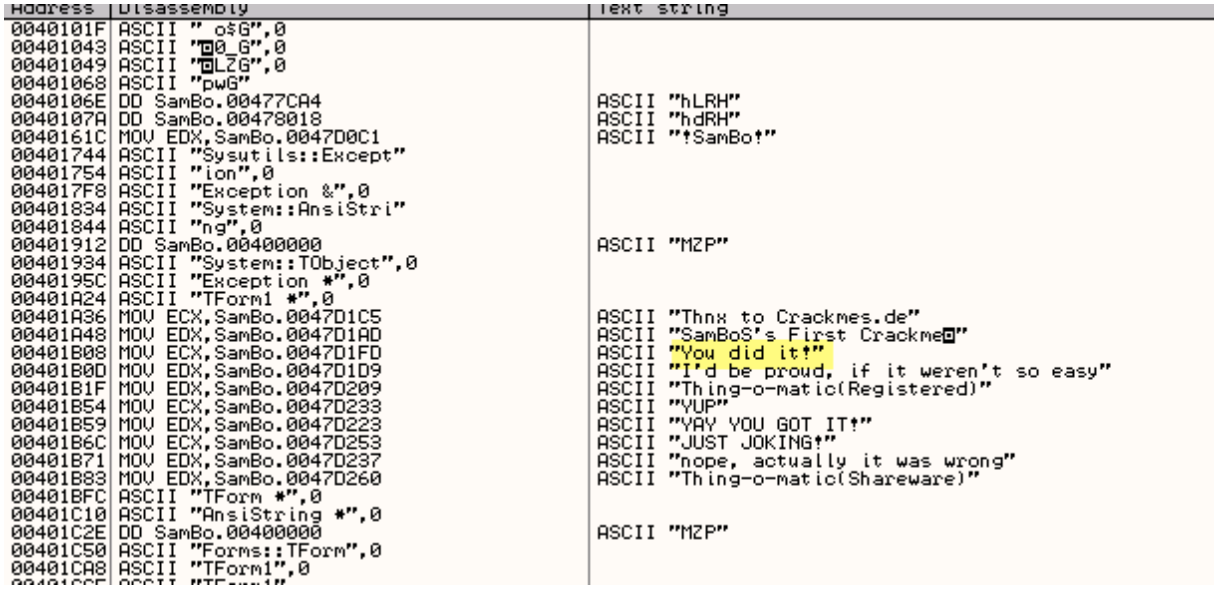


Address	Section	Type	Name	Comment
004D50BA	.aspack	Import	oleaut32.#35	
0040306C	.text	Export	@xp26Shdocvw_tlb@TCppWebBrowser	
00410960	.text	Export	@xp26Shdocvw_tlb@TCppWebBrowser	
00402274	.text	Export	@xp28Shdocvw_tlb@TCppShellWindows	
0040FC68	.text	Export	@xp28Shdocvw_tlb@TCppShellWindows	
00402180	.text	Export	@xp29Shdocvw_tlb@TCppShellUIHelper	
0040FBCC	.text	Export	@xp29Shdocvw_tlb@TCppShellUIHelper	
00403B0C	.text	Export	@xp29Shdocvw_tlb@TCppWebBrowser_V1	
004113BC	.text	Export	@xp29Shdocvw_tlb@TCppWebBrowser_V1	
00401EE0	.text	Export	@xp32Shdocvw_tlb@TCppCScriptErrorList	
0040F9D8	.text	Export	@xp32Shdocvw_tlb@TCppCScriptErrorList	
00402A14	.text	Export	@xp32Shdocvw_tlb@TCppInternetExplorer	
00410360	.text	Export	@xp32Shdocvw_tlb@TCppInternetExplorer	
00401D98	.text	Export	@xp33Shdocvw_tlb@TCppSearchAssistantOC	
0040F8E8	.text	Export	@xp33Shdocvw_tlb@TCppSearchAssistantOC	
004023BC	.text	Export	@xp34Shdocvw_tlb@TCppShellBrowserWindow	
0040FD5C	.text	Export	@xp34Shdocvw_tlb@TCppShellBrowserWindow	
00401FD4	.text	Export	@xp36Shdocvw_tlb@TShellFavoritesNameSpace	
0040FA78	.text	Export	@xp36Shdocvw_tlb@TShellFavoritesNameSpace	
00411B94	.text	Export	@@Shdocvw_ocx@Finalize	
00411B7C	.text	Export	@@Shdocvw_ocx@Initialize	
00406780	.text	Export	@@Unit1@Finalize	
00406770	.text	Export	@@Unit1@Initialize	
004D50AA	.aspack	Import	user32.ActivateKeyboardLayout	
004D50A2	.aspack	Import	gdi32.BitBlt	
004D50B2	.aspack	Import	ole32.CoCreateInstance	
0047D098	.data	Export	___CPPdebugHook	
00486E9C	.data	Export	_Form1	
004013C5	.text	Export	__GetExceptDLLinfo	
004D4F60	.aspack	Import	kernel32.GetModuleHandleA	
004D4F5C	.aspack	Import	kernel32.GetProcAddress	
004D509A	.aspack	Import	comctl32.ImageList_Add	
004D4F64	.aspack	Import	kernel32.LoadLibraryA	
004D4001	.aspack	Export	<ModuleEntryPoint>	
004D5092	.aspack	Import	advapi32.RegCloseKey	
0040F37C	.text	Export	@Shdocvw_ocx@Register$qqrv	
0047D6F8	.data	Export	@Shdocvw_tlb@TCppCScriptErrorList@	
00481E44	.data	Export	@Shdocvw_tlb@TCppCScriptErrorList@	
00401F68	.text	Export	@Shdocvw_tlb@TCppCScriptErrorList@$bctr$qqrp18Classes@TComponent	
0040FA08	.text	Export	@Shdocvw_tlb@TCppCScriptErrorList@$bctr$qqrp18Classes@TComponent	
0040EAB8	.text	Export	@Shdocvw_tlb@TCppCScriptErrorList@BeforeDestruction$qqrv	
0040E7D4	.text	Export	@Shdocvw_tlb@TCppCScriptErrorList@Connect$qqrv	
0040EACC	.text	Export	@Shdocvw_tlb@TCppCScriptErrorList@ConnectTo$qqr91%TComInterface$28Shd	1 argument
0040E9D0	.text	Export	@Shdocvw_tlb@TCppCScriptErrorList@Disconnect$qqrv	
0040E5D0	.text	Export	@Shdocvw_tlb@TCppCScriptErrorList@GetDefaultInterface$qv	
0040E5F4	.text	Export	@Shdocvw_tlb@TCppCScriptErrorList@GetDunk$qqrv	
0040EBEC	.text	Export	@Shdocvw_tlb@TCppCScriptErrorList@InitServerData$qqrv	
0047DA3C	.data	Export	@Shdocvw_tlb@TCppInternetExplorer@	
00482188	.data	Export	@Shdocvw_tlb@TCppInternetExplorer@	
00403000	.text	Export	@Shdocvw_tlb@TCppInternetExplorer@$bctr$qqrp18Classes@TComponent	
004103F0	.text	Export	@Shdocvw_tlb@TCppInternetExplorer@$bctr$qqrp18Classes@TComponent	
00408838	.text	Export	@Shdocvw_tlb@TCppInternetExplorer@BeforeDestruction$qqrv	
004084EC	.text	Export	@Shdocvw_tlb@TCppInternetExplorer@Connect$qqrv	
0040884C	.text	Export	@Shdocvw_tlb@TCppInternetExplorer@ConnectTo$qqr83%TComInterface$24Shd	1 argument
00408750	.text	Export	@Shdocvw_tlb@TCppInternetExplorer@Disconnect$qqrv	
00408208	.text	Export	@Shdocvw_tlb@TCppInternetExplorer@GetDefaultInterface$qv	
0040822C	.text	Export	@Shdocvw_tlb@TCppInternetExplorer@GetDunk$qqrv	

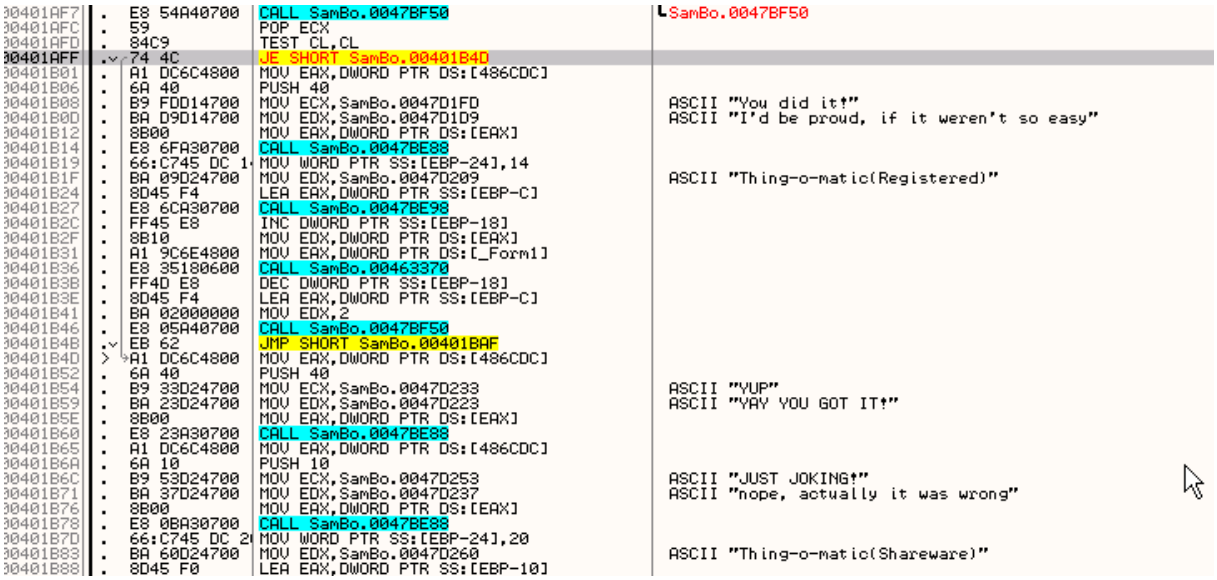
Уфф, какой ужасный список и ничего знакомого, посмотрим список строк; ясно, что с этим у нас тоже были бы проблемы в начале, так как находились в другой секции, и кроме того, строки были запакованы вместе с программой, так что ничего бы мы не увидели.



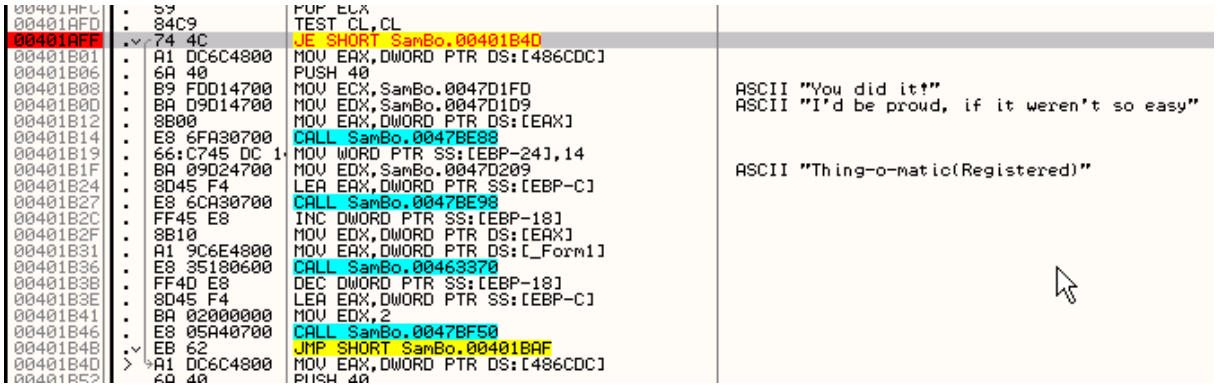
Ещё один ужасный список.



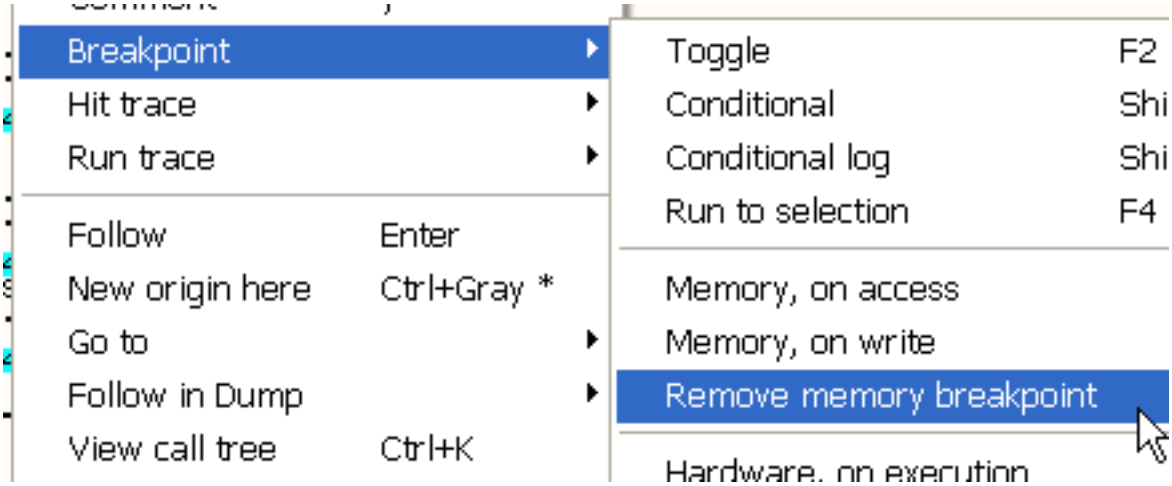
“YOU DID IT” – одна из строк с поздравлением.



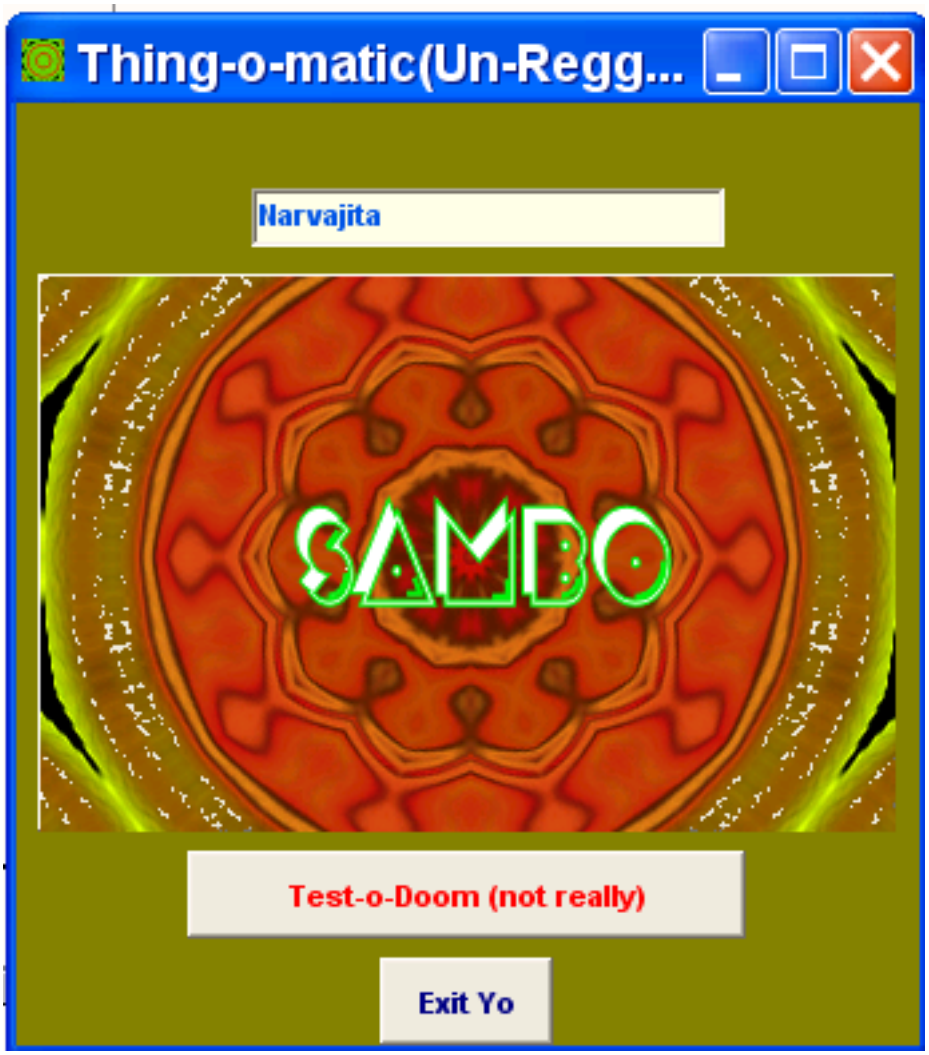
Хорошо, есть сравнение и переход, где мы видим строки с поздравлением и ошибкой, хотя здесь и нет MessageBoxA.



Устанавливаем BPX на условный переход, чтобы проверить, решающий ли это переход, и убираем BPM ON ACCESS щелчком правой кнопки мыши.



И делаем RUN.

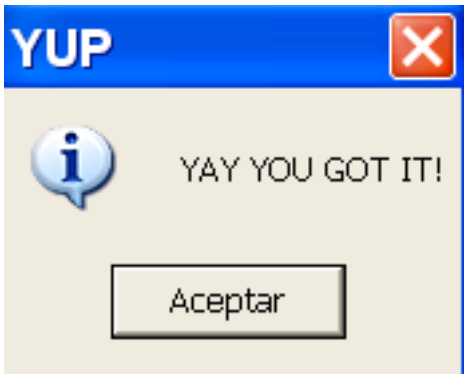


В окне крэкми вводим “Narvajita” или любой другой неправильный серийный номер.

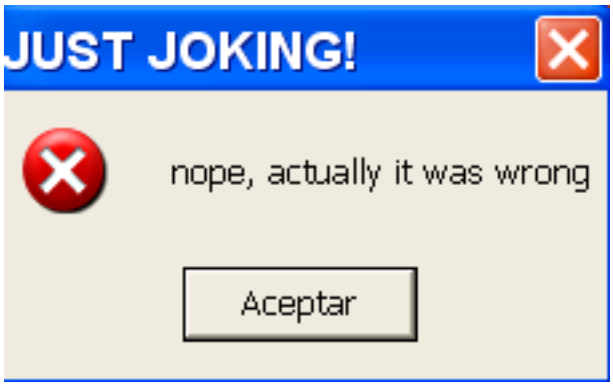
Нажимаем кнопку “Test-o-Doom”.

```
00401AFD  .  84C9          TEST CL,CL
00401AFF  .v 74 4C         JE SHORT SamBo.00401B4D
00401B01  .  A1 DC6C4800   MOV EAX,DWORD PTR DS:[486CDC]
00401B06  .  6A 40         PUSH 40
00401B08  .  B9 FDD14700   MOV ECX,SamBo.0047D1FD          ASCII "You did it!"
00401B0D  .  BA D9D14700   MOV EDX,SamBo.0047D1D9          ASCII "I'd be proud, if it weren't so easy"
00401B12  .  8B00          MOV EAX,DWORD PTR DS:[EAX]
00401B14  .  E8 6FA30700   CALL SamBo.0047BE88
00401B19  .  66:C745 DC 1( MOV WORD PTR SS:[EBP-24],14
00401B1F  .  BA 09D24700   MOV EDX,SamBo.0047D209          ASCII "Thing-o-matic(Registered)"
00401B24  .  8D45 F4       LEA EAX,DWORD PTR SS:[EBP-C]
00401B27  .  E8 6CA30700   CALL SamBo.0047BE98
00401B2C  .  FF45 E8       INC DWORD PTR SS:[EBP-18]
00401B2F  .  8B10          MOV EDX,DWORD PTR DS:[EAX]
00401B31  .  A1 9C6E4800   MOV EAX,DWORD PTR DS:[_Form1]
00401B36  .  E8 35180600   CALL SamBo.00463370
00401B3B  .  FF4D E8       DEC DWORD PTR SS:[EBP-18]
00401B3E  .  8D45 F4       LEA EAX,DWORD PTR SS:[EBP-C]
00401B41  .  BA 02000000   MOV EDX,2
00401B46  .  E8 05A40700   CALL SamBo.0047BF50
00401B4B  .v EB 62         JMP SHORT SamBo.00401BAF
00401B4D  > >A1 DC6C4800   MOV EAX,DWORD PTR DS:[486CDC]
00401B52  .  6A 40         PUSH 40
00401B54  .  B9 33D24700   MOV ECX,SamBo.0047D233          ASCII "YUP"
```

Видим, что произошёл переход, нажимаем F9.



Тогда соглашаемся с окошком с поздравлением, что у нас получилось.



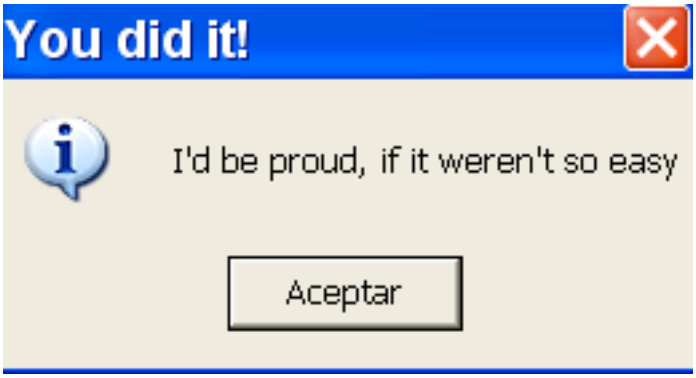
Оно говорит, что это была шутка, и ничего не вышло на самом деле, ха ха! Делаем RUN по-новой и снова прибываем к переходу, связанному с кнопкой “Test-o-Doom”.

```
00401AF7  .  E8 54A40700   CALL SamBo.0047BF50               SamBo.0047BF50
00401AFC  .  59            POP ECX
00401AFD  .  84C9          TEST CL,CL
00401AFF  .v 74 4C         JE SHORT SamBo.00401B4D
00401B01  .  A1 DC6C4800   MOV EAX,DWORD PTR DS:[486CDC]
00401B06  .  6A 40         PUSH 40
00401B08  .  B9 FDD14700   MOV ECX,SamBo.0047D1FD          ASCII "You did it!"
00401B0D  .  BA D9D14700   MOV EDX,SamBo.0047D1D9          ASCII "I'd be proud, if it weren't so easy"
00401B12  .  8B00          MOV EAX,DWORD PTR DS:[EAX]
00401B14  .  E8 6FA30700   CALL SamBo.0047BE88
00401B19  .  66:C745 DC 1( MOV WORD PTR SS:[EBP-24],14
00401B1F  .  BA 09D24700   MOV EDX,SamBo.0047D209          ASCII "Thing-o-matic(Registered)"
00401B24  .  8D45 F4       LEA EAX,DWORD PTR SS:[EBP-C]
00401B27  .  E8 6CA30700   CALL SamBo.0047BE98
00401B2C  .  FF45 E8       INC DWORD PTR SS:[EBP-18]
00401B2F  .  8B10          MOV EDX,DWORD PTR DS:[EAX]
00401B31  .  A1 9C6E4800   MOV EAX,DWORD PTR DS:[_Form1]
00401B36  .  E8 35180600   CALL SamBo.00463370
00401B3B  .  FF4D E8       DEC DWORD PTR SS:[EBP-18]
00401B3E  .  8D45 F4       LEA EAX,DWORD PTR SS:[EBP-C]
00401B41  .  BA 02000000   MOV EDX,2
00401B46  .  E8 05A40700   CALL SamBo.0047BF50
00401B4B  .v EB 62         JMP SHORT SamBo.00401BAF
00401B4D  > >A1 DC6C4800   MOV EAX,DWORD PTR DS:[486CDC]
00401B52  .  6A 40         PUSH 40
00401B54  .  B9 33D24700   MOV ECX,SamBo.0047D233          ASCII "YUP"
```

Инвертируем переход, чтобы посмотреть, отобразится ли окошко с поздравлением и понять, отвечает ли за это вообще данный переход.

```
00401AFD  .  84C9          TEST CL,CL
00401AFF  .v 74 4C         JE SHORT SamBo.00401B4D
00401B01  .  A1 DC6C4800   MOV EAX,DWORD PTR DS:[486CDC]
00401B06  .  6A 40         PUSH 40
00401B08  .  B9 FDD14700   MOV ECX,SamBo.0047D1FD          ASCII "You did it!"
00401B0D  .  BA D9D14700   MOV EDX,SamBo.0047D1D9          ASCII "I'd be proud, if it weren't so easy"
00401B12  .  8B00          MOV EAX,DWORD PTR DS:[EAX]
00401B14  .  E8 6FA30700   CALL SamBo.0047BE88
00401B19  .  66:C745 DC 1  MOV WORD PTR SS:[EBP-24],14
00401B1F  .  BA 09D24700   MOV EDX,SamBo.0047D209          ASCII "Thing-o-matic(Registered)"
00401B24  .  8D45 F4       LEA EAX,DWORD PTR SS:[EBP-C]
00401B27  .  E8 6CA30700   CALL SamBo.0047BE98
00401B2C  .  FF45 E8       INC DWORD PTR SS:[EBP-18]
00401B2F  .  8B10          MOV EDX,DWORD PTR DS:[EAX]
00401B31  .  A1 9C6E4800   MOV EAX,DWORD PTR DS:[_Form1]
00401B36  .  E8 35180600   CALL SamBo.00463370
00401B3B  .  FF4D E8       DEC DWORD PTR SS:[EBP-18]
00401B3E  .  8D45 F4       LEA EAX,DWORD PTR SS:[EBP-C]
00401B41  .  BA 02000000   MOV EDX,2
00401B46  .  E8 05A40700   CALL SamBo.0047BF50
00401B4B  .v EB 62         JMP SHORT SamBo.00401BAF
00401B4D  > >A1 DC6C4800   MOV EAX,DWORD PTR DS:[486CDC]
```

После двойного щелчка на флаге Z и изменения его значения, делаем RUN.



И отображается окошко с поздравлением, значит, это и есть то место, где решается правильность или неправильность серийного номера. Посмотрим сравнение до перехода.

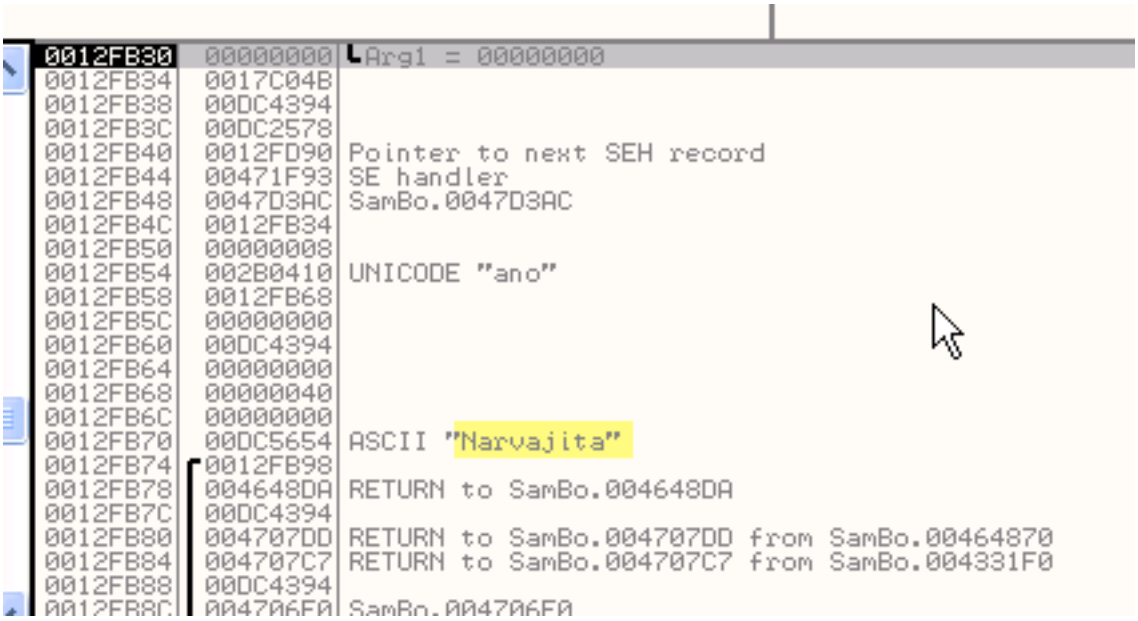
```
00401AFC  .  59            POP ECX
00401AFD  .  84C9          TEST CL,CL
00401AFF  .v 74 4C         JE SHORT SamBo.00401B4D
00401B01  .  A1 DC6C4800   MOV EAX,DWORD PTR DS:[486CDC]
```

Это TEST CL, CL, где проверяется, равен ли CL нулю или единице, что задаётся где-то до этого, возможно в CALL, вызываемый ранее, на который мы можем поставить BPX.

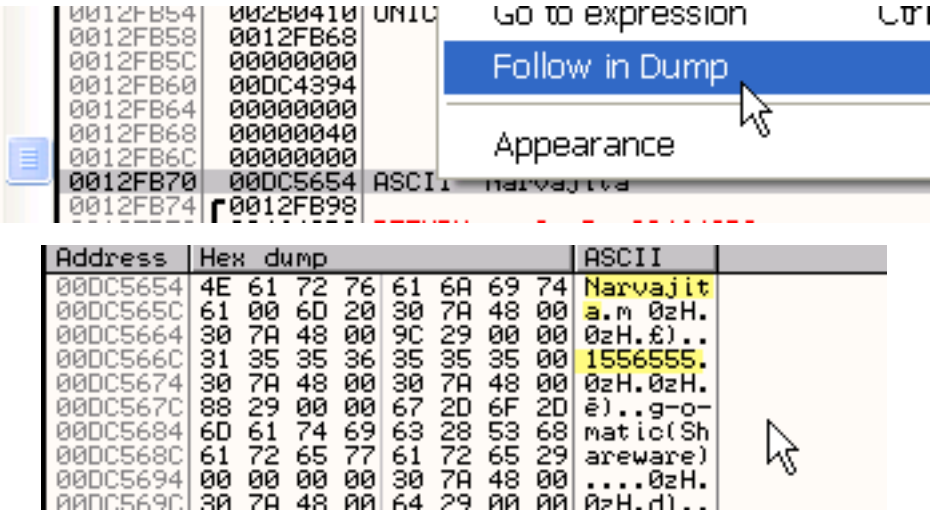
```
00401AF2  .  BA 02000000   MOV EDX,2
00401AF7  .  E8 54A40700   CALL SamBo.0047BF50
00401AFC  .  59            POP ECX
00401AFD  .  84C9          TEST CL,CL
00401AFF  .v 74 4C         JE SHORT SamBo.00401B4D
00401B01  .  A1 DC6C4800   MOV EAX,DWORD PTR DS:[486CDC]
```

Делаем RUN, возвращаемся в окно и снова нажимаем кнопку Test, чтобы остановиться на CALL.

Остановившись, осмотримся любопытным взором вокруг, хе-хе.

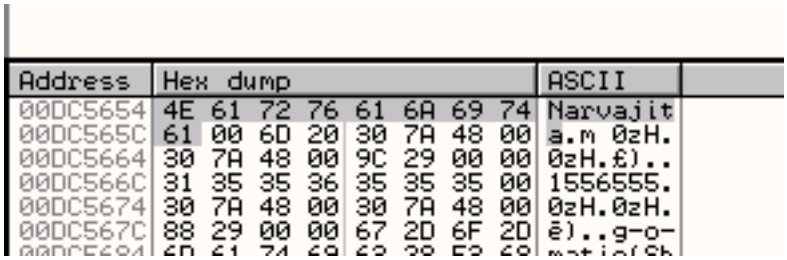


Если посмотрим в стек, то увидим введённый неправильный серийный номер, заглянем в DUMP (через FOLLOW IN DUMP).

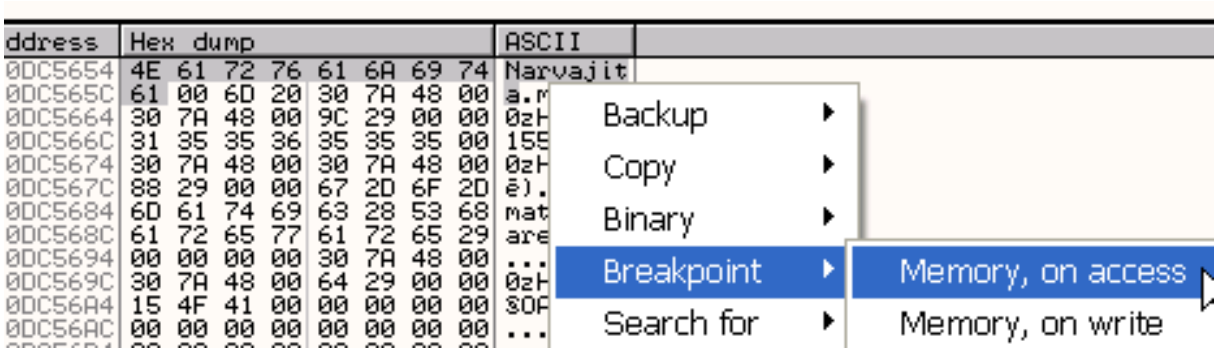


Мой глаз крэкера видит здесь строку, которая, возможно, является верным серийником; могли бы посмотреть, так ли это, но пока делать этого не будем и продолжим поиск серьёзным образом.

Если внутри данного CALL производятся какие-либо сравнения или операции с неправильным серийником, то можно установить BPX ON ACCESS на него, чтобы программа остановилась, когда с ним будет что-нибудь делаться, то есть на какой-либо операции или сравнении.



Выделим неправильный серийный номер, нажмём правую кнопку мыши и выберем BREAKPOINT-MEMORY ON ACCESS.

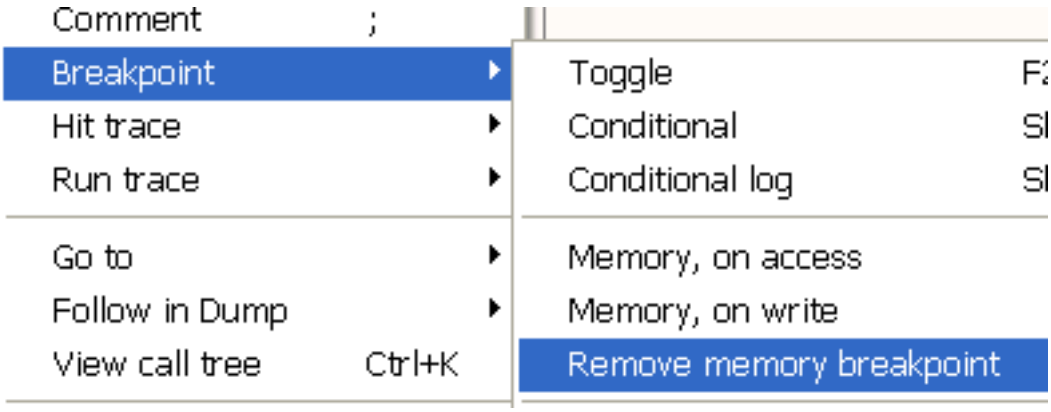


После этого, когда продолжим выполнение, то если внутри call происходит какая-либо работа с этим серийником, OllyDbg остановится.

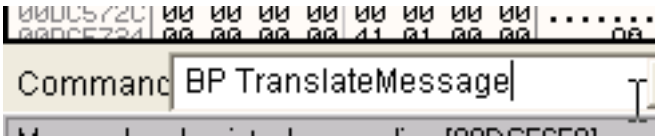
Нажимаем F9.

Видим, что остановки не произошло, возможно, сравнение происходит раньше, так что мы можем поставить точки останова на CALL'ы, находящиеся ещё выше, и повторить то, что делали прежде, но лучше пустим в ход другой способ остановиться на том месте, где вводится серийный номер: здесь не используется API-функция GetWindowTextA, но у нас есть сообщения Windows, которые послужат нам не хуже.

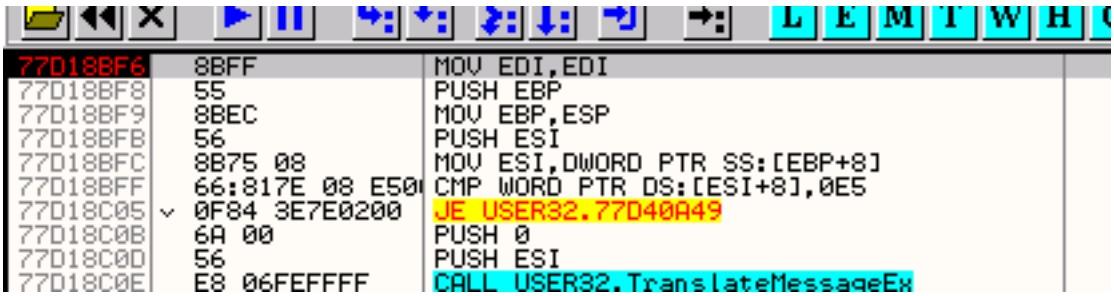
Убираем BPM ON ACCESS.



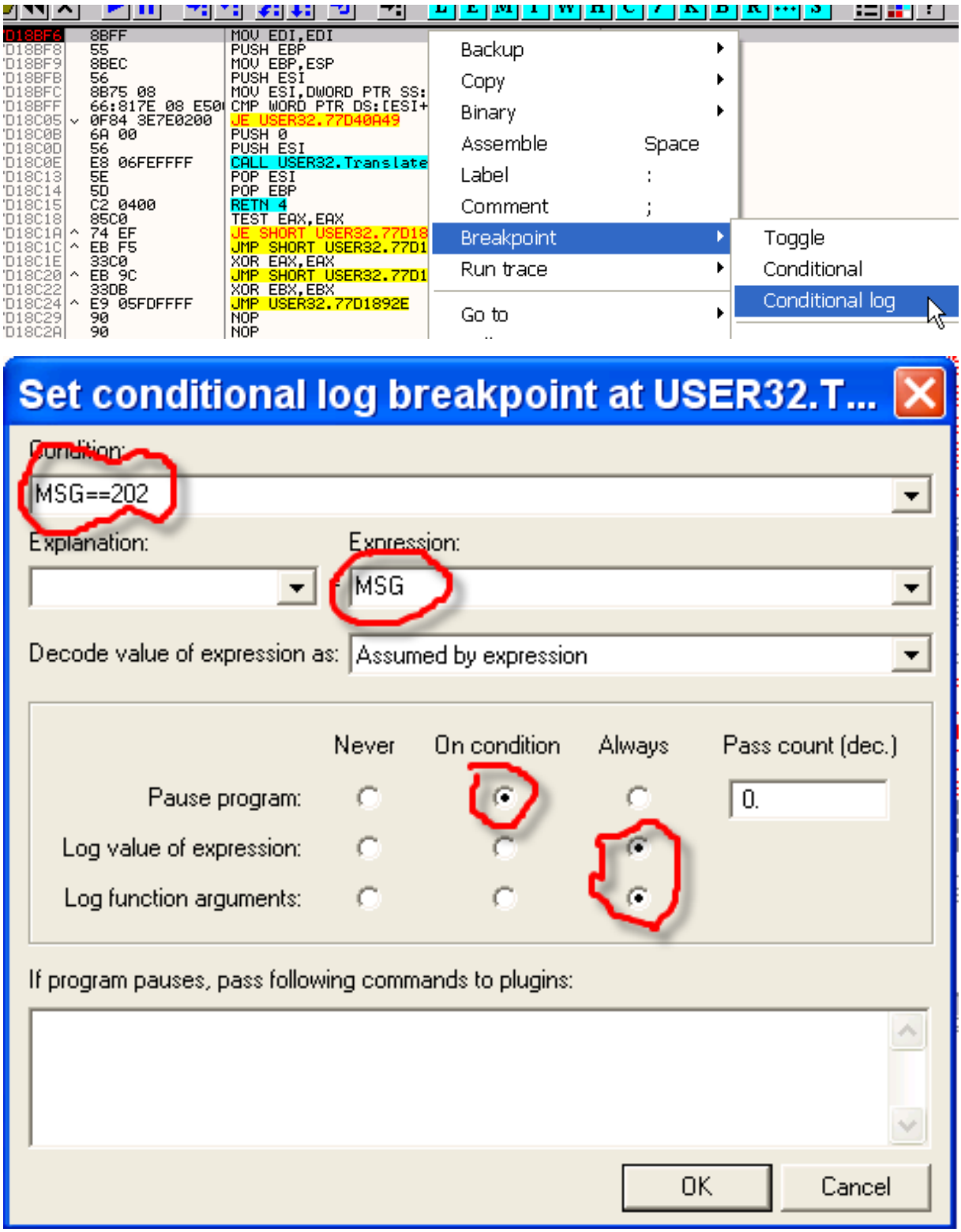
Ставим быстрым способом BP TranslateMessage.



А теперь RUN.

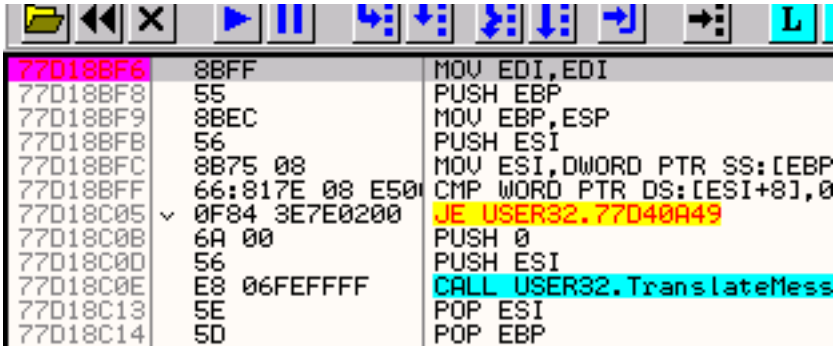


Остановшись на API-функции, нажимаем на правую кнопку мыши и выбираем:



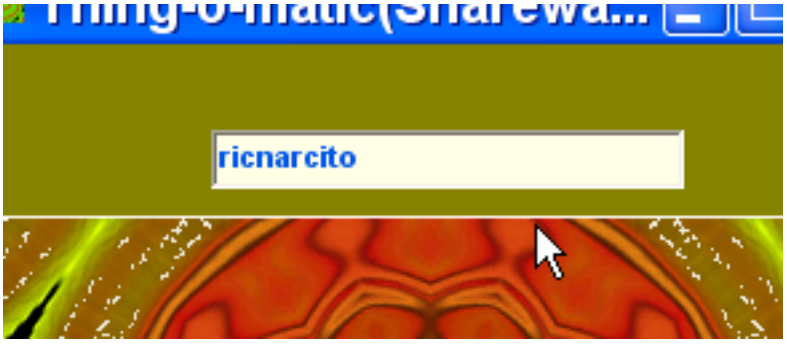
И в окошке печатаем условие MSG==202, которое является значением WM_LBUTTONUP.

И устанавливаем переключатели так, чтобы остановка происходила по ON CONDITION, и чтобы логгировалось всё.



Здесь розовым цветом обозначен BPX CONDICIONAL. Делаем RUN.

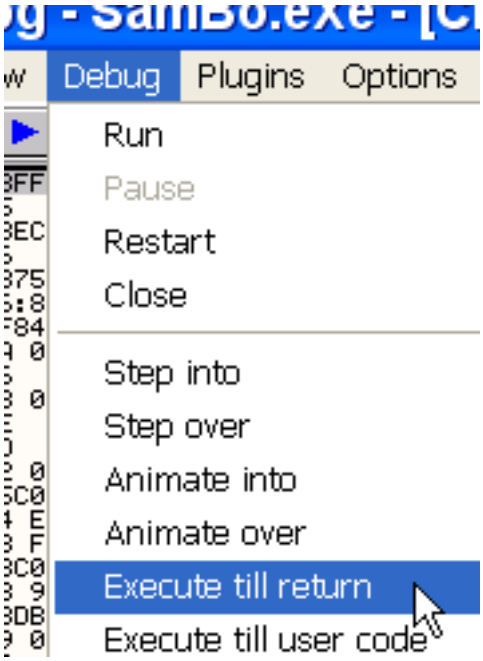
Открывается окошко крэкми и, чтобы избежать путаницы, меняю серийный номер на тот случай, если предыдущий остался в памяти.

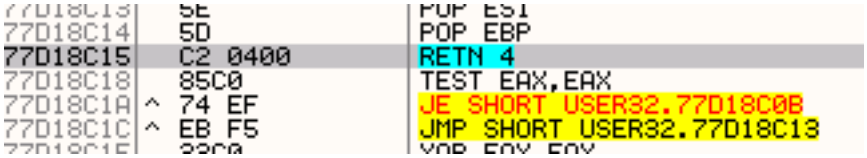


И по нажатию на кнопку срабатывает BPX CONDICIONAL.

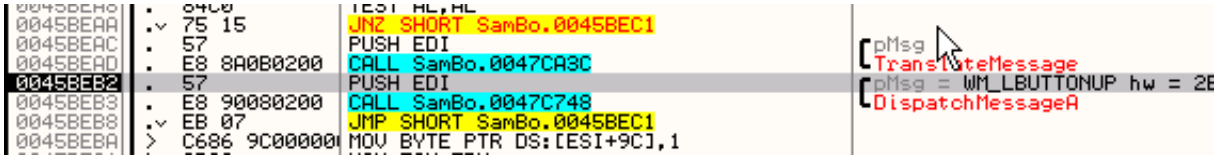
Остановка произошла на WM_LBUTTONUP.

Возвращаемся в программу с помощью EXECUTE TILL RETURN.



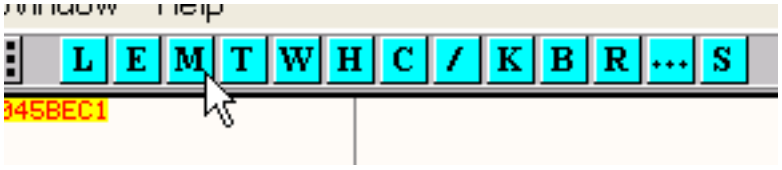


Нажимаем F7 и возвращаемся в программу.

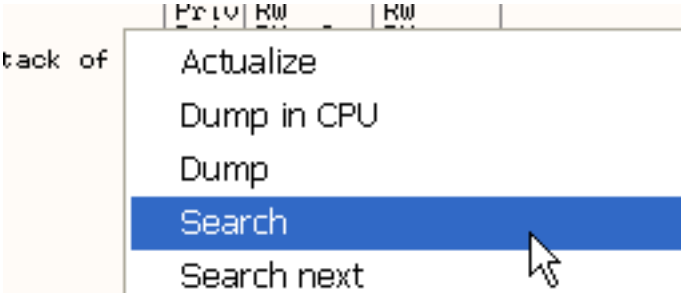


Здесь у меня появляется две возможности. Установить BPM ON ACCESS в первой секции, чтобы сделать переход с помощью F9 и посмотреть, куда ведёт процедура сравнения серийного номера, что мне кажется сложноватым, так как я уже видел, что эта процедура весьма большая.

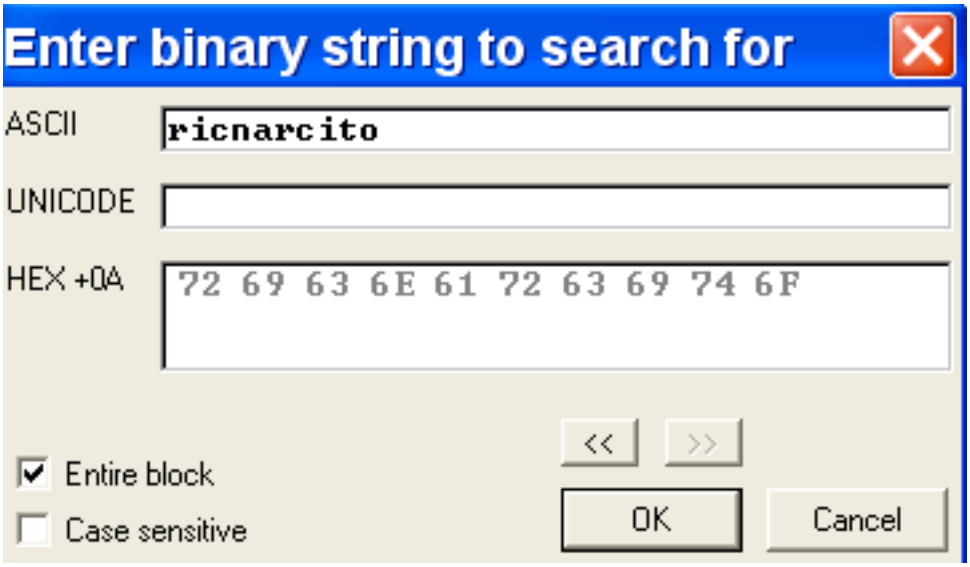
Другая возможность – это посмотреть, находится ли уже в памяти введённый нами серийный номер, так что идём в окно M.



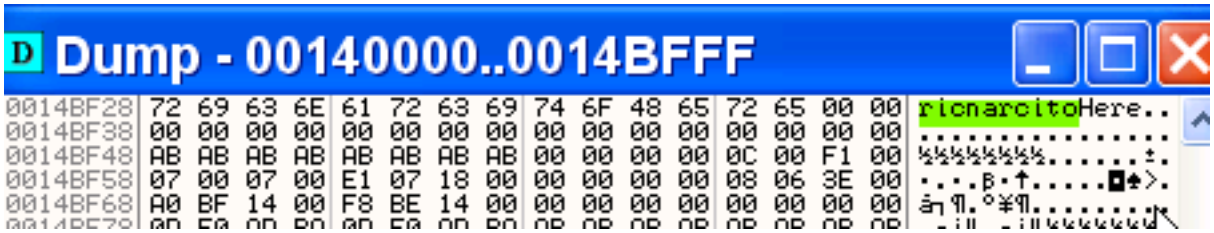
Кроме того, что оно позволяет просматривать секции, здесь можно искать строки, находящиеся в памяти, кликаем правую кнопку мыши и выбираем SEARCH.



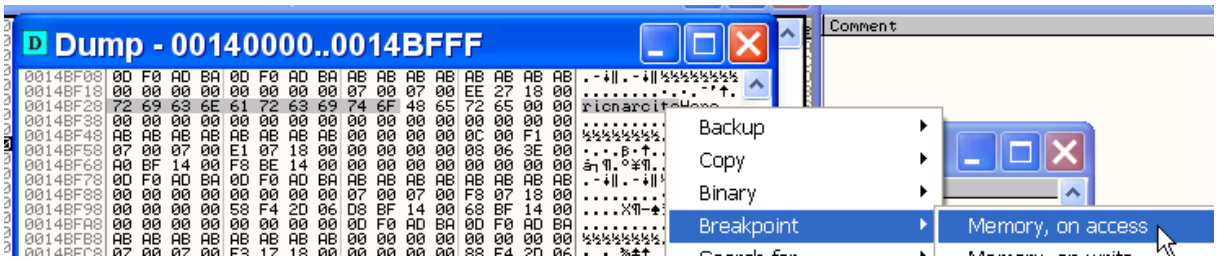
В появившемся окне в поле ASCII введём наш неправильный серийник.



Нажимаем OK, чтобы начать поиск в памяти.

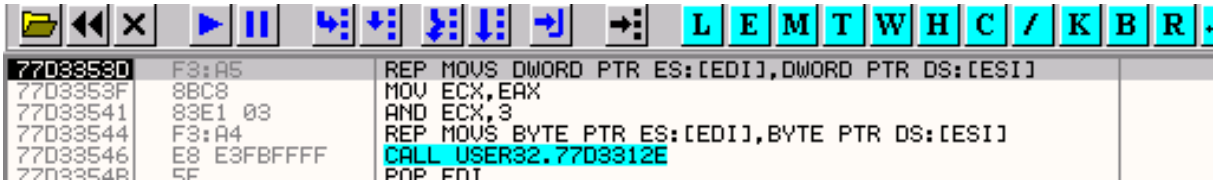


С помощью CTRL+L можно искать остальные вхождения этого слова, но больше ничего не находим ни в секции, которую мы открыли, ни в остальной памяти.

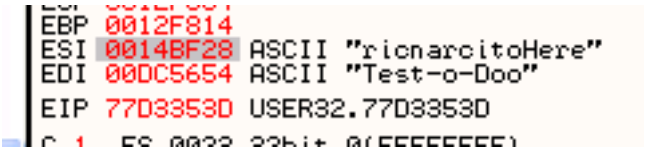


Устанавливаем BPM ON ACCESS на этот неправильный серийник, , чтобы найти момент, когда с ним производятся какие-либо операции, чтение или сравнение.

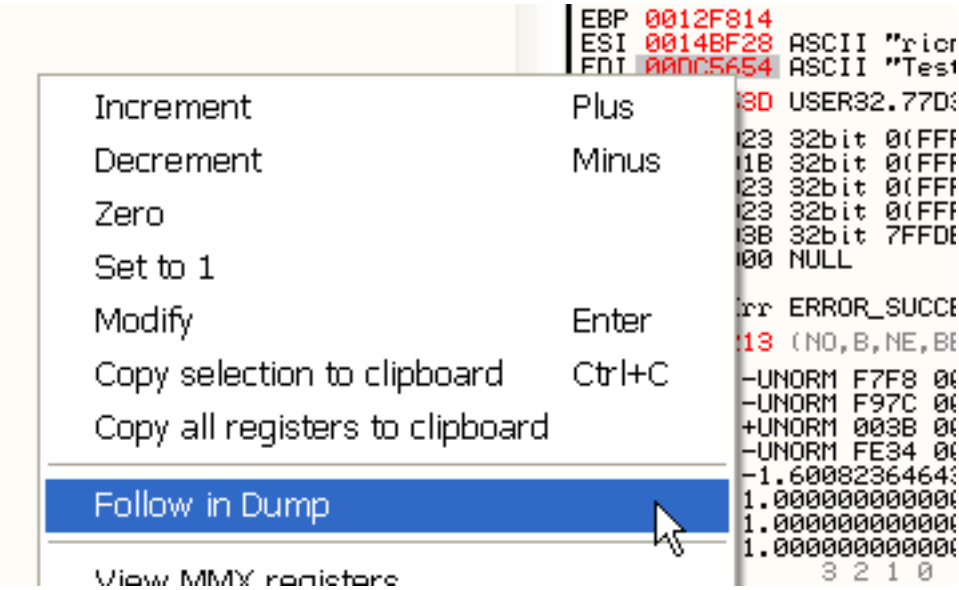
И нажимаем F9.

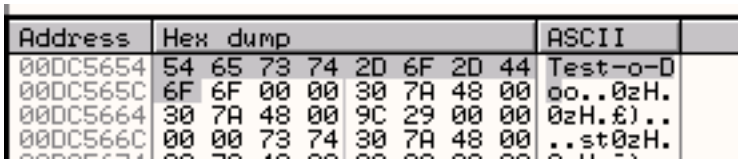


Останавливаемся там, где он копируется в то место, которое мы видели в последнем CALL'е.

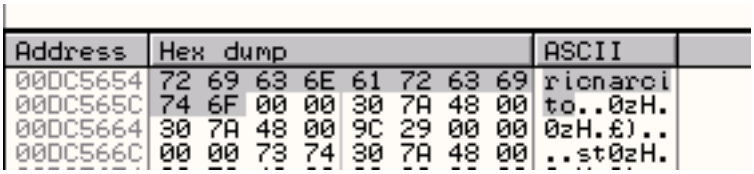


Мы знаем, что REP MOVS копирует содержимое из ESI в EDI, так что посмотрим EDI в DUMP (правая кнопка мыши на EDI-FOLLOW IN DUMP).

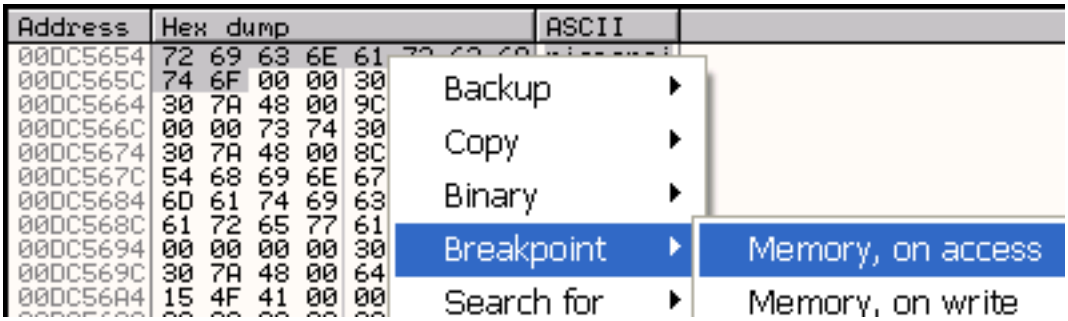




Сюда REP MOVS скопирует серийник, нажмём F8, чтобы он это сделал.

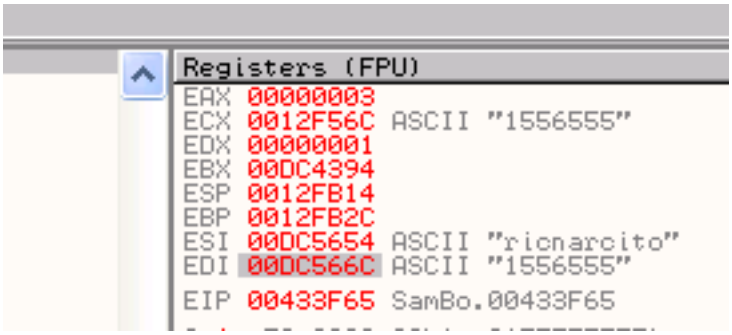


Оказались после этого на RET, серийник полностью скопировался, и устанавливаем на него BPM ON ACCESS.



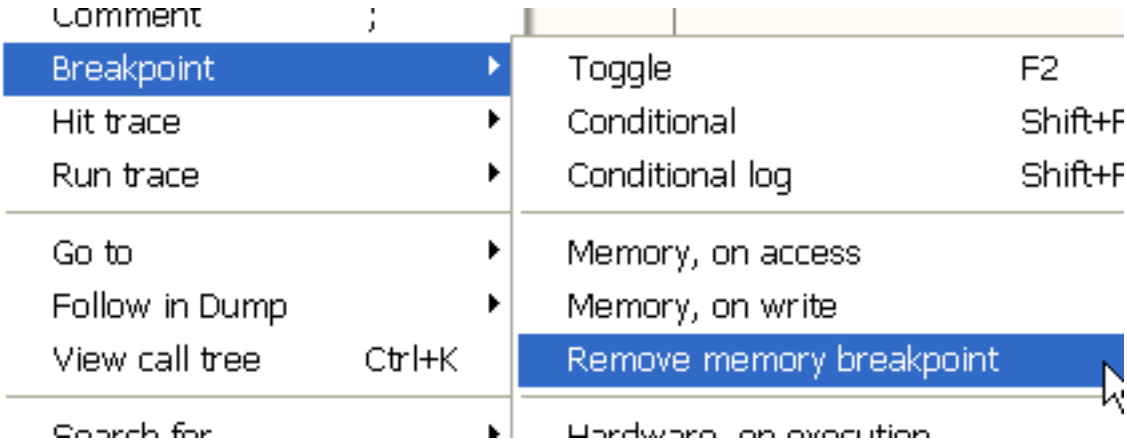
Выполнив RUN, останавливаемся здесь.

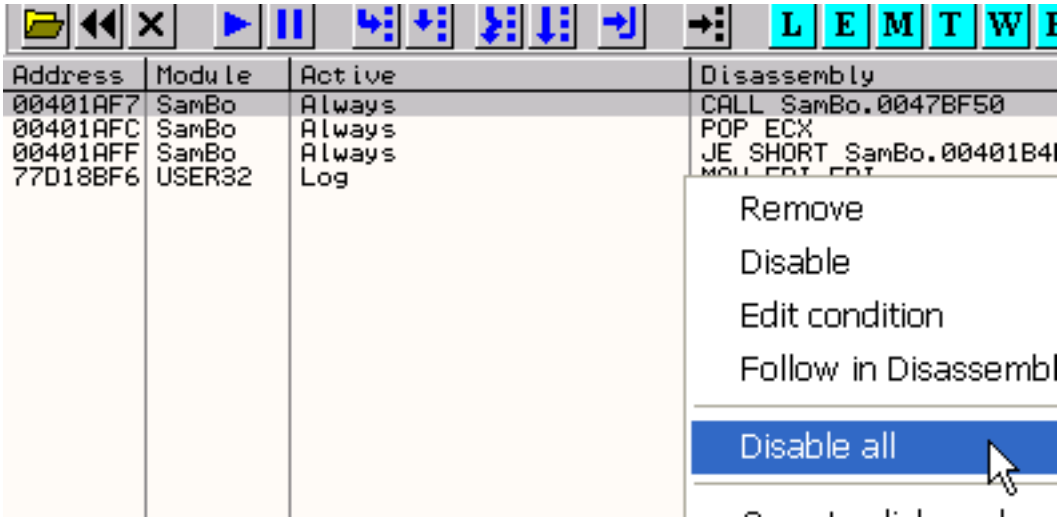
Точнёхонько на сравнение, хех.

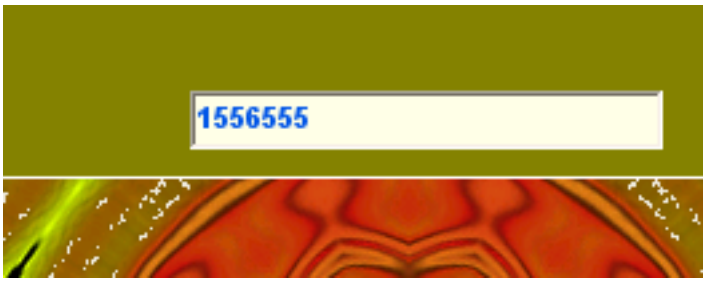


ESI указывает на мой неправильный серийник, а EDI на правильный, который равен «1556555».

Убираем все BPX.







Появляется окошко с поздравлением.

В следующей главе мы начнём разбор крэкми, в который используются не только серийный номер, но и имя пользователя.

Введение в крэкинг с нуля, используя OllyDbg - Глава 16

Автор: Рикардо Нарваха, пер. Aquila

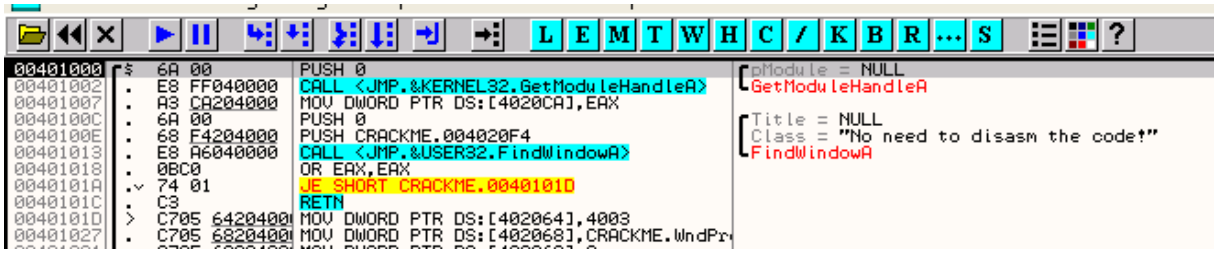
Дата: 2008-01-21

В этой части мы начнём разбор крэкми, которые отличаются от предыдущих тем, что серийный номер является переменным и высчитывается на основе имени, которое мы вводим.

Способ его решения очень похож, но рассмотрим несколько примеров, чтобы всё стало понятным.

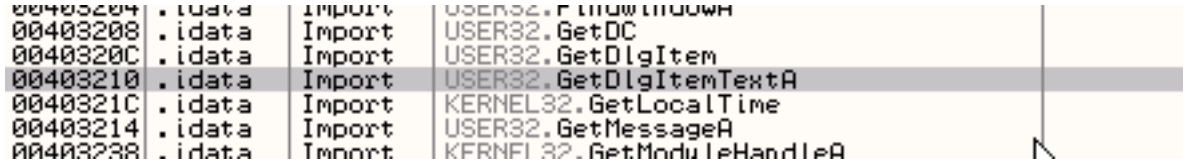
Начнём с поиска правильного серийного номера в крэкми CrueHead'a.

Откроем его в OllyDbg.

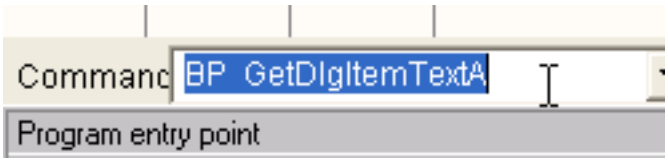


Мы находимся на точке входа.

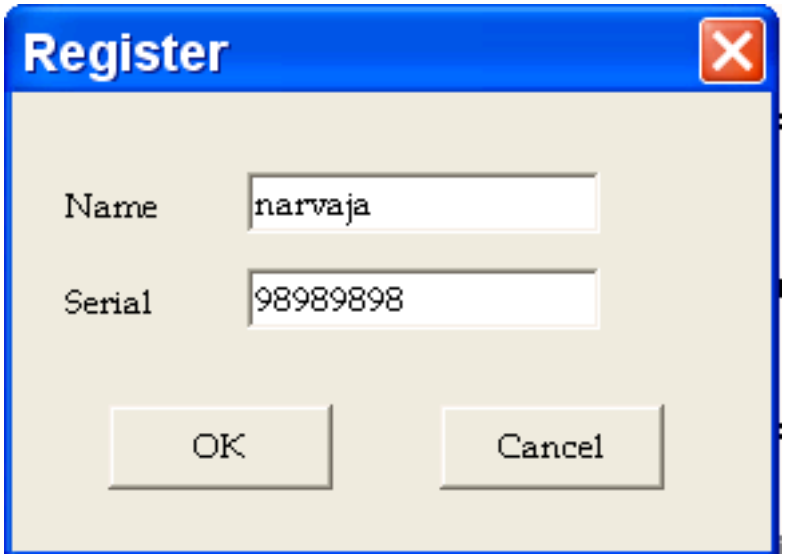
Смотрим, какие API-функции используются для получения введённого серийного номера.



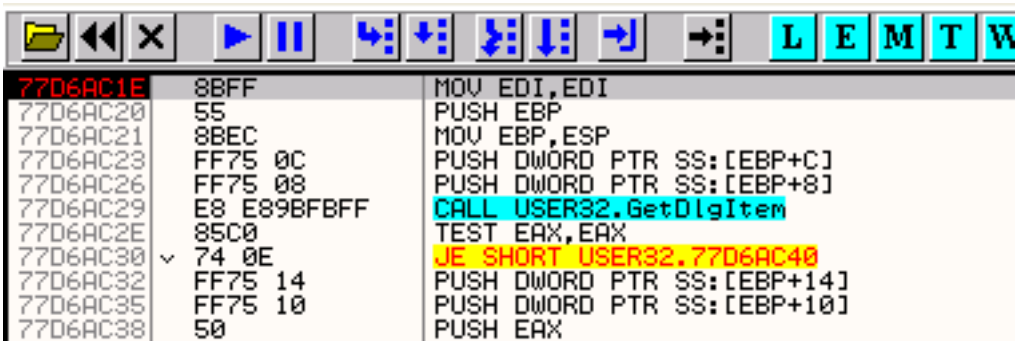
Попробуем установить BP на указанной функции, чтобы остановиться на том месте, где будет считываться имя и серийный номер.

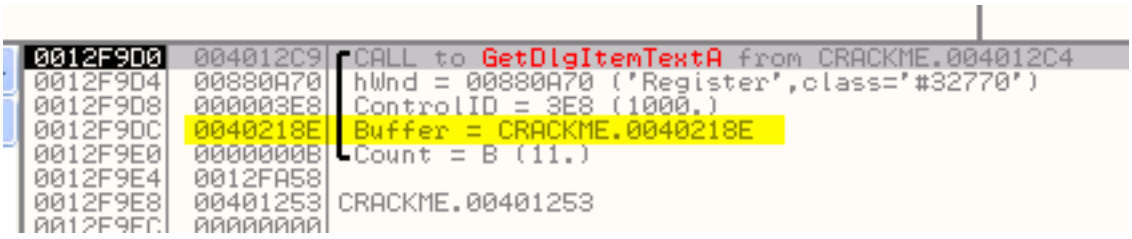


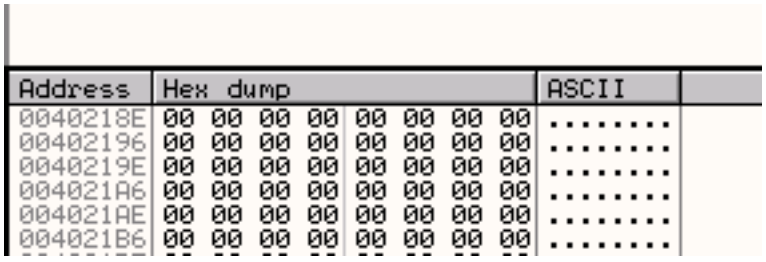
Делаем RUN.



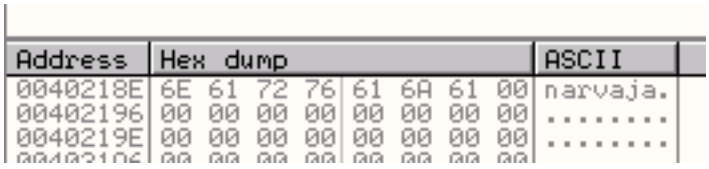
После нажатия на OK оказываемся на api-функции и в стеке видим параметры.





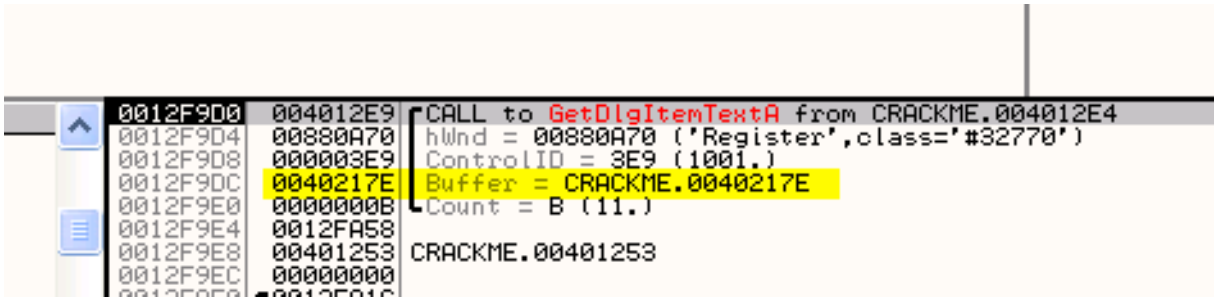


Буфер, в котором сохраняется введённый текст, начинается с 40218E, посмотрим через DUMP, что там находится – правый клик мыши и FOLLOW IN DUMP на строке, где приводится информация о буфере.

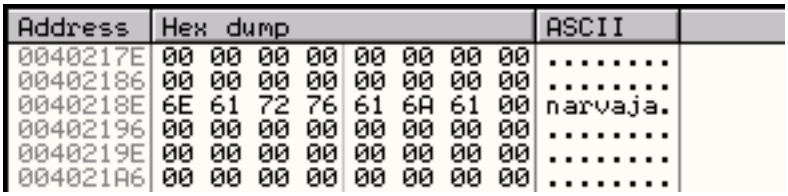


Здесь ничего нет, потому что API-функция ещё не выполнилась, так что выполняем DEBUG-EXECUTE TILL RETURN, и когда оказываемся на RET, нажимаем F7, чтобы вернуться в

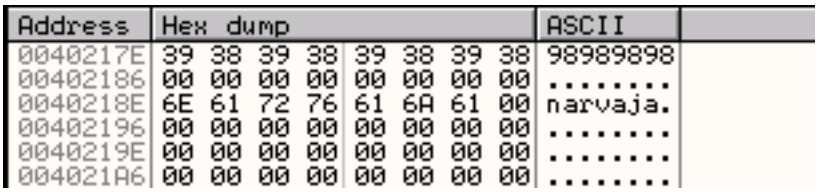
программу.



Видим, что в буфере сохранилось моё имя, чтобы в дальнейшем произвести соответствующие операции и создать правильный ключ для него. Если мы хотим сделать кейген, то должны изучить алгоритм, который выполняется программой для вычисления ключа из имени, но пока что просто дадим программе его сгенерировать и попытаемся посмотреть, где с ним сравнивается введённый нами серийный номер.

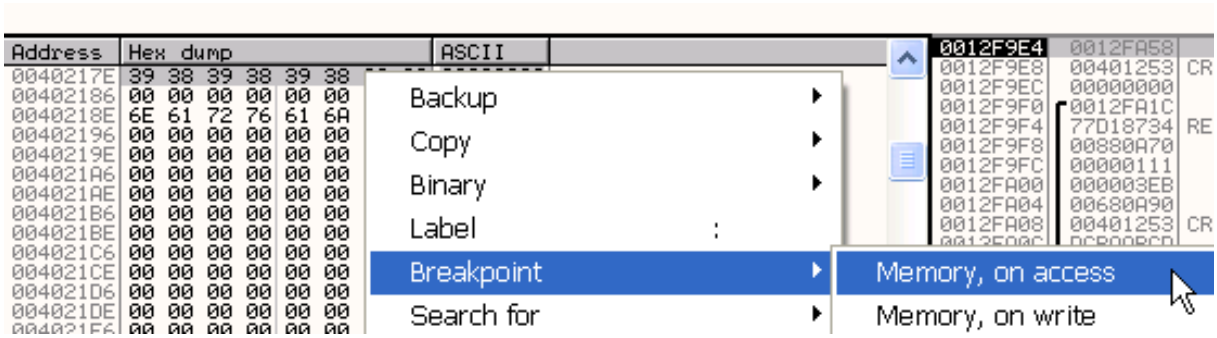


Встречаем ту же API-функцию во второй раз. Здесь используется новый буфер, куда помещается введённый серийник. Посмотрим его через дамп.

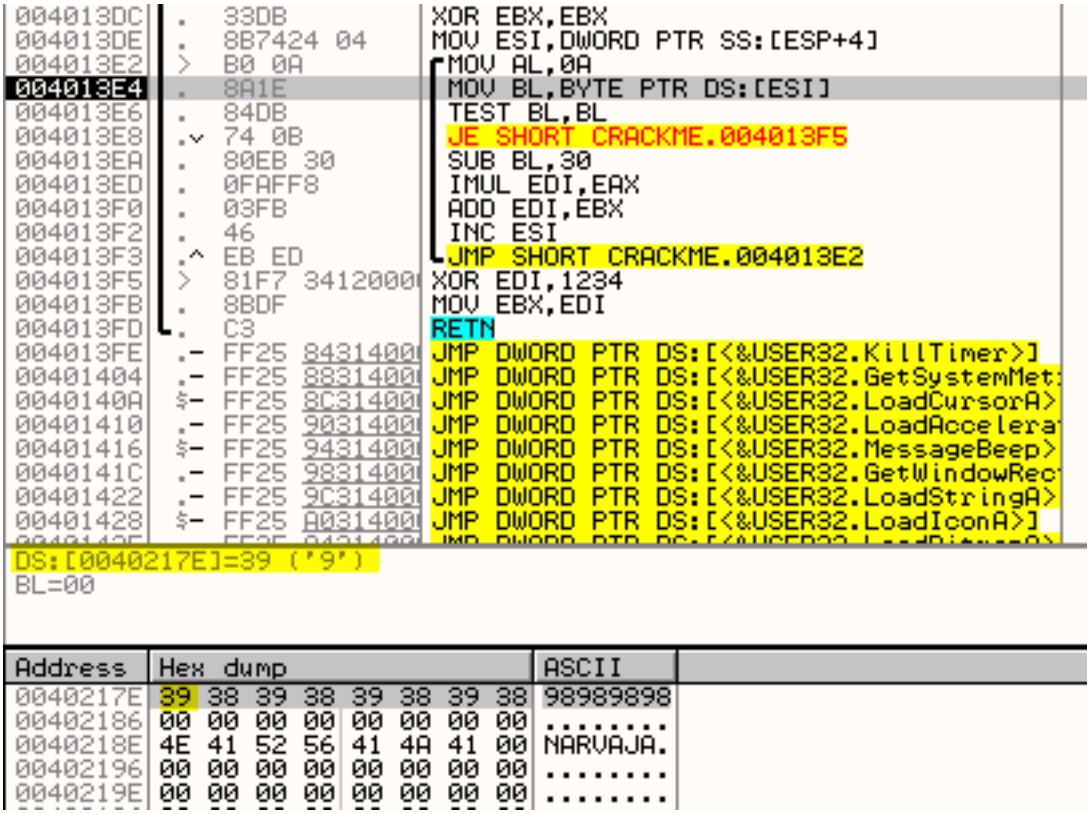


То же самое, что и в прошлый раз; выполняем API-функцию с помощью EXECUTE TILL RETURN, а чтобы вернуться в программу, нажмём F7.

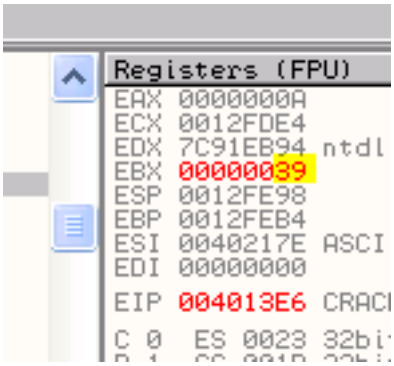
Теперь здесь наш неправильный серийный номер, начиная с этого места программа будет сравнивать его с правильным, который высчитывается на основании имени, поэтому можем установить BPM ON ACCESS на неверном серийнике, чтобы узнать, что с ним делает программа.



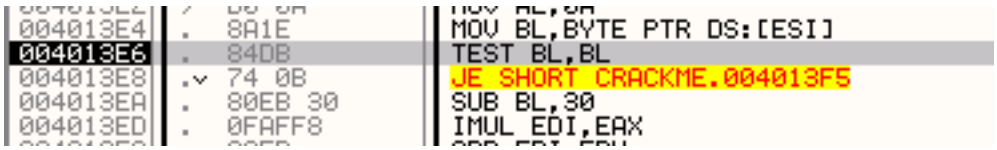
Отметим нужные байты, правый клик – BREAKPOINT – MEMORY ON ACCESS, и RUN.



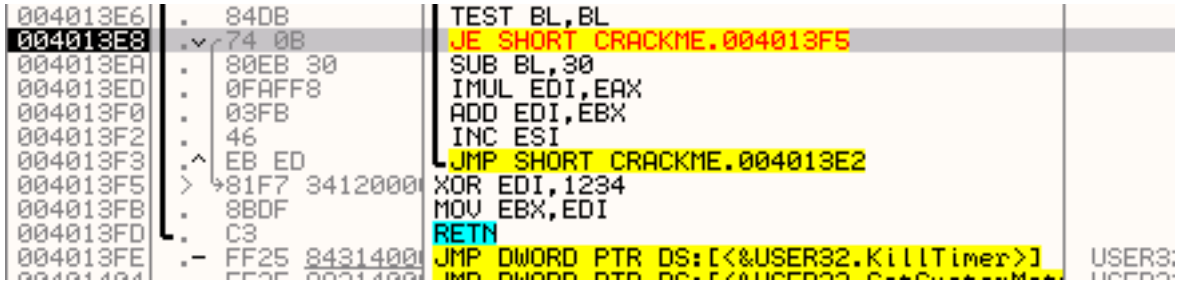
Видим, что остановка произошла там, где программа считывает первый байт неправильного серийного номера и перемещает его в BL. Нажимаем F7.



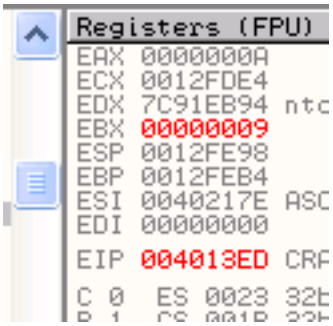
В BL находится 39 – значение первого байта. Нам нужно отловить, что программа делает с ним и, по возможности, записать, какие математические действия программа над ним совершает.



Здесь делается проверка на то, равен ли он нулю, если да – это конец строки и произойдёт выход из цикла.

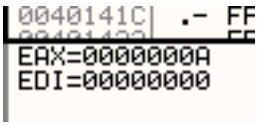


Так как нулю первый байт не равен, то перехода не происходит и попадаем на SUB BL, 30.



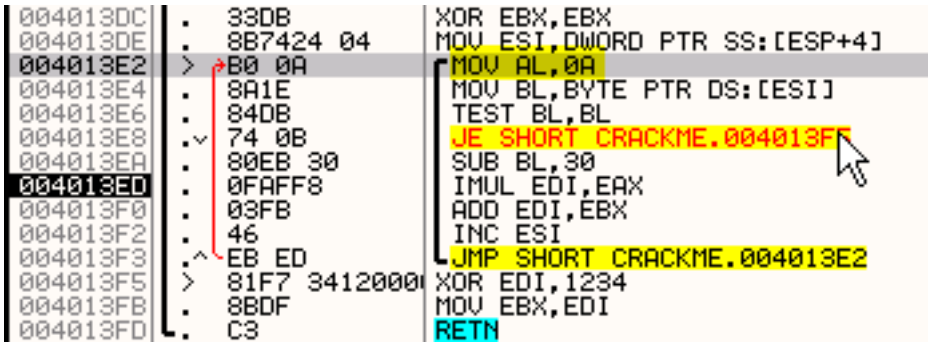
После того, как было вычтено 30, в BL осталось 9.

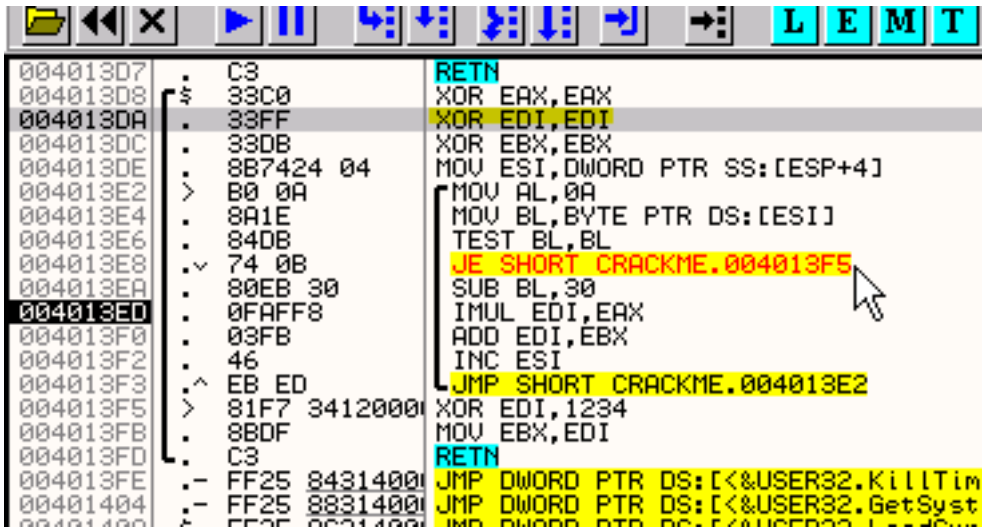
На следующей строке EDI умножается на EAX.



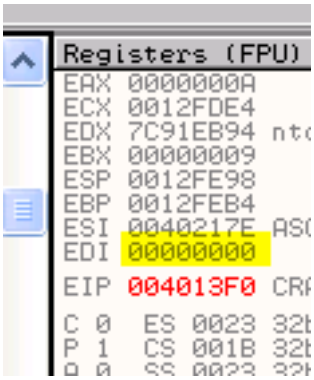
s

Эти регистры были инициализированы следующими значениями: в начале процедуры в EAX было помещено 0A, а EDI до входа в цикл приравнен к нулю с помощью инструкции XOR EDI, EDI.

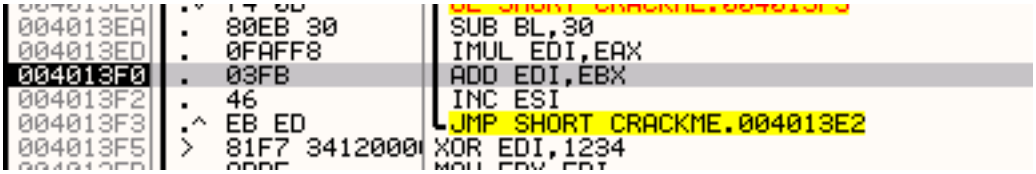




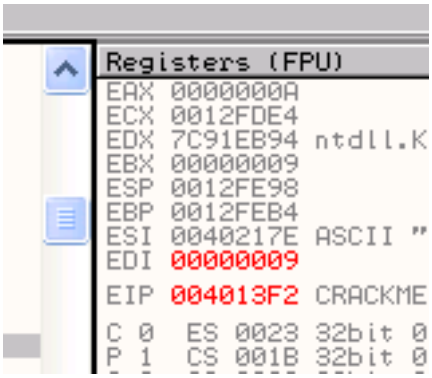
По нажатию F7, как мы помним, исходя из определения инструкции IMUL с двумя операндами, в данном случае, они будут умножены с учётом знака, а результат сохранён в первый операнд, т.е. в EDI.



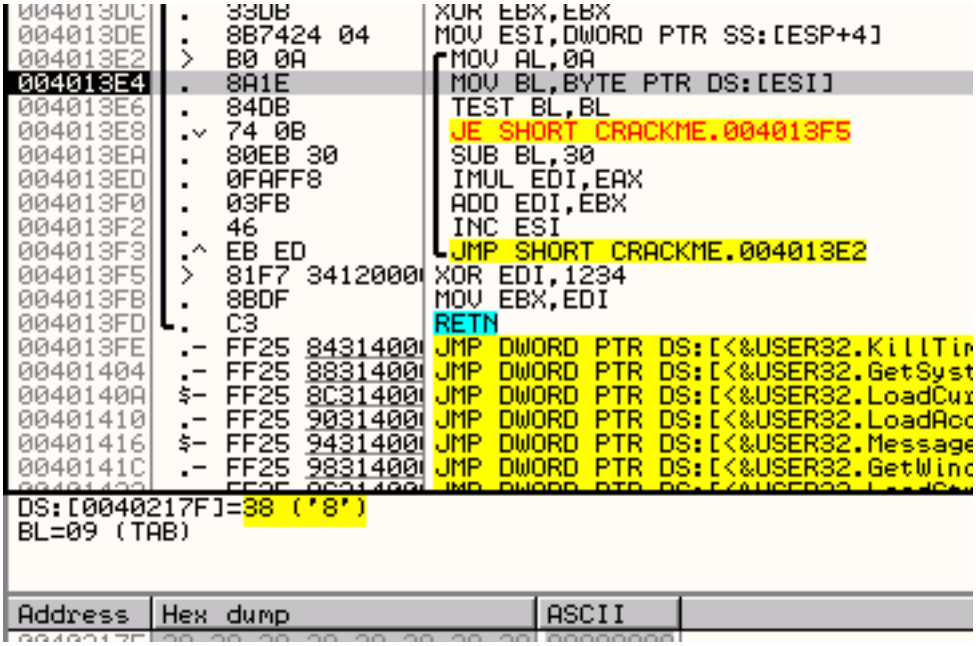
Таким образом, в EDI остаётся ноль, ставший результатом выполнения предыдущей операции.



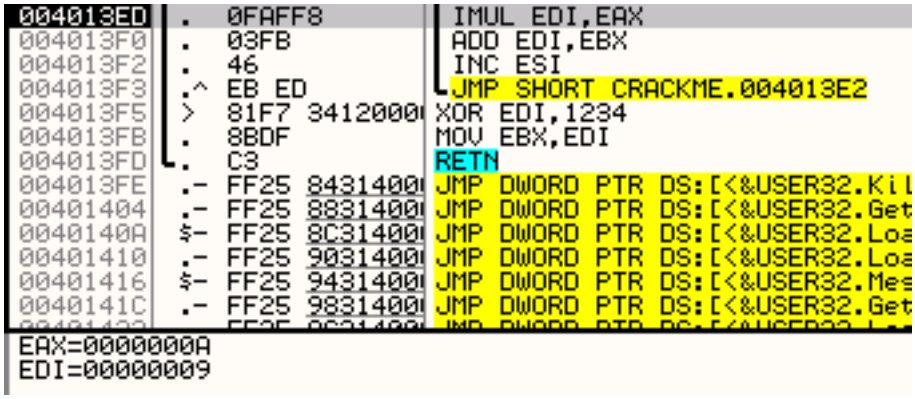
Дальше идёт сложение EDI со значением в EBX.



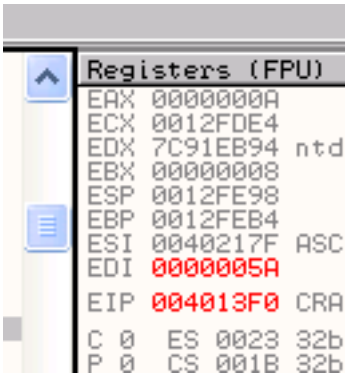
В результате в EDI остаётся 9, а на следующей строке выполняется инструкция INC ESI, что увеличивает ESI, чтобы затем вернуться на начало цикла и прочитать следующий байт неправильного серийника.



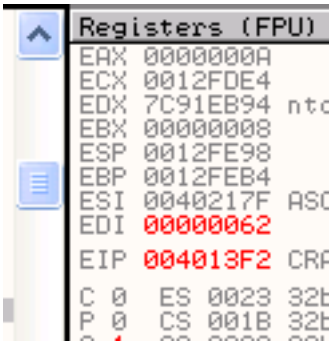
После это в AL снова помещается значение 0A, прочитывается следующий байт неправильного серийного номера (это будет 38), проверяется, равно ли оно нулю, из него вычитается 30 и прибываем в IMUL.



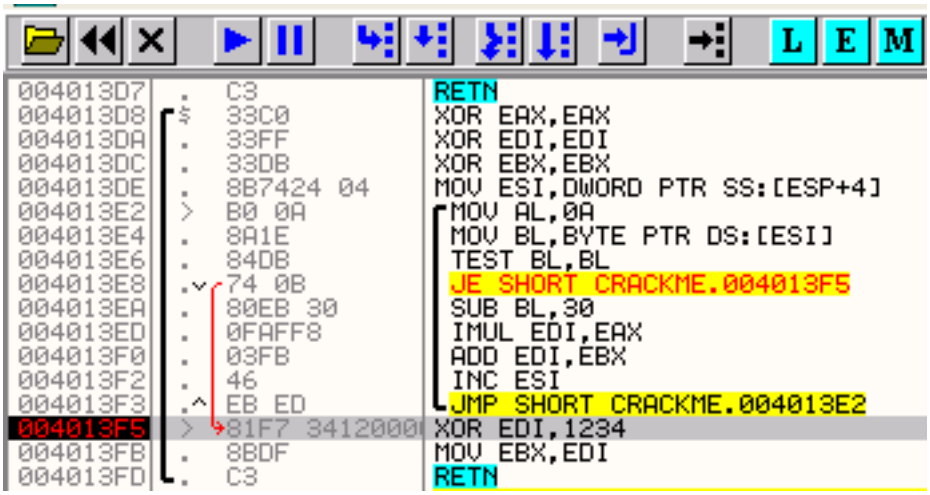
Видим, что умножается EDI, в котором осталось значение ещё с прошлого пробега цикла, на 0A, а результат помещается в EDI же.



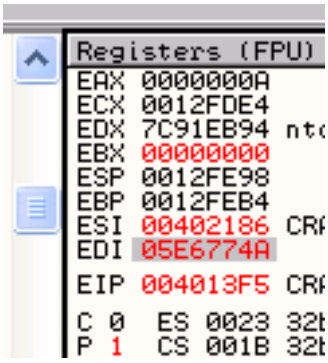
Теперь EDI содержит 5A, а на следующей строке он складывается с EBX.



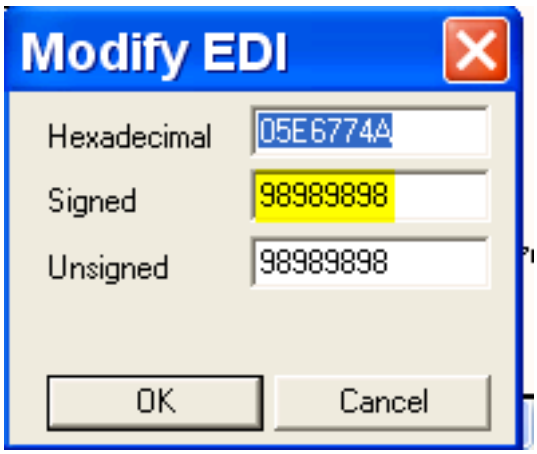
То есть результат всех производящихся операций накапливается в EDI; тем, кто усерден и прилежен на ниве крэкинга, уже ясно, что происходит. Объясним.



Достаточно установить точку останова на выход из цикла: нажимаем F9 и, когда происходит остановка, видим, чему равен EDI.

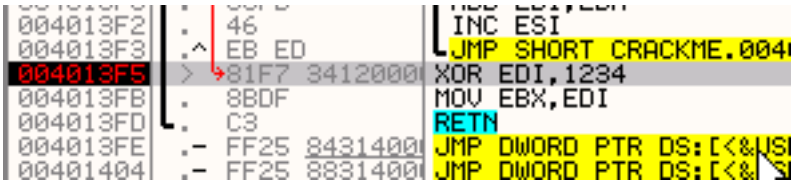


Если кликнем два раза на EDI.

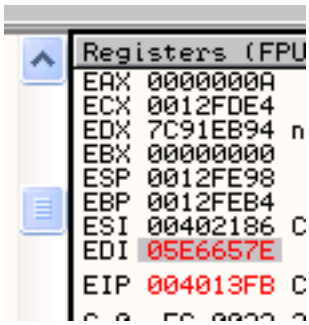


Значение EDI – значение шестнадцатеричное, во втором поле видим его же в десятеричной системе счисления, и это значение соответствует серийному номеру, введённому мной, т.е. после выхода из цикла в EDI содержится серийник в шестнадцатеричном виде.

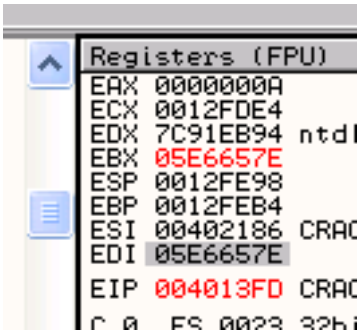
Т.е. резюмируя кратко: если напечатаем слово 98989898, оно будет трансформировано в десятеричное число 98989898 или шестнадцатеричное 5E6774A.



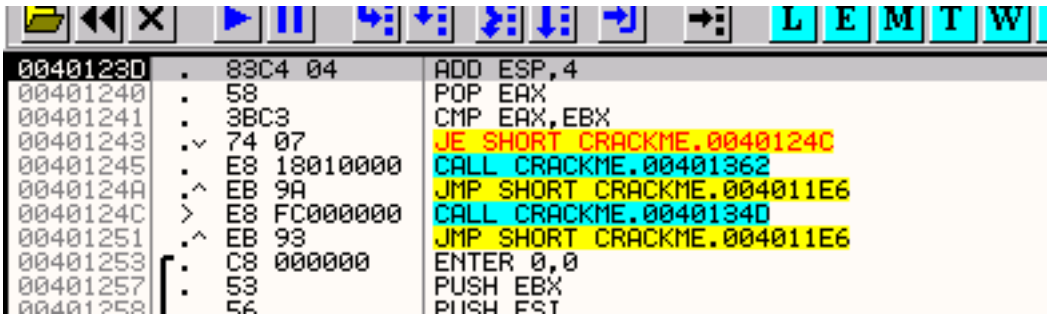
На следующей строке EDI хог’ится с 1234.

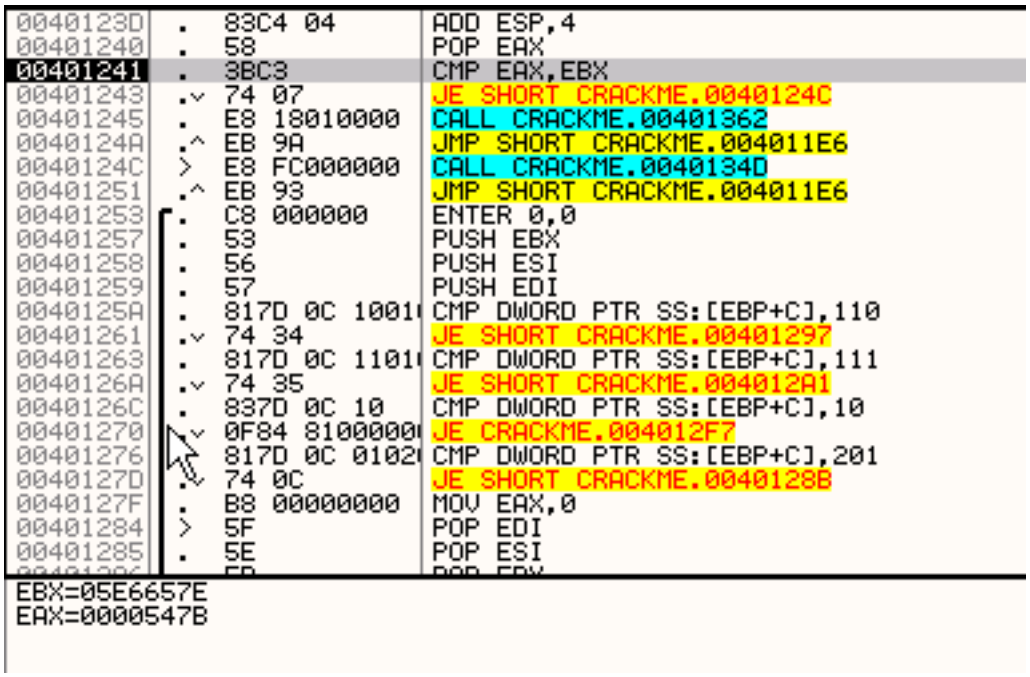


В результате получается 5E6657E, которое перемещается в EBX, а затем мы прибываем в RET.



Видим, что выйдя через RET, мы оказались у сравнения EAX с EBX, то есть туда, где находится переход, решающий, правилен или нет серийный номер.





И видим, что сравнивается вычисленное значение, находящееся в EBX с EAX, который содержит 547B.

Так как EAX определяется программой, а EBX – высчитывается, то очевидно, что поскольку мой серийный номер неверен, то и значения этих регистров не будет равно.

Если EBX был бы равен EAX, то программа сделала бы нужный переход, поэтому нам нужно проанализировать, отчего произошло неравенство регистров.

Смотрим сделанные нами заметки.

EBX= (Шестнадцатиричное значение неверного серийного номера) XOR 1234

Мне нужно, чтобы EBX был равен EAX, тогда проверка на равенство выдаст нужный результат.

Если EAX=EBX

Reemplazo EBX por EAX ya que son iguales

Заменим EBX на EAX, то есть чтобы они были равны.

EAX=(Шестнадцатиричное значение правильного серийного номера) XOR 1234

При выполнении этого условия введенный серийный номер будет верным.

Переформулируем.

EAX XOR 1234= (Шестнадцатиричное значение правильного серийного номера)

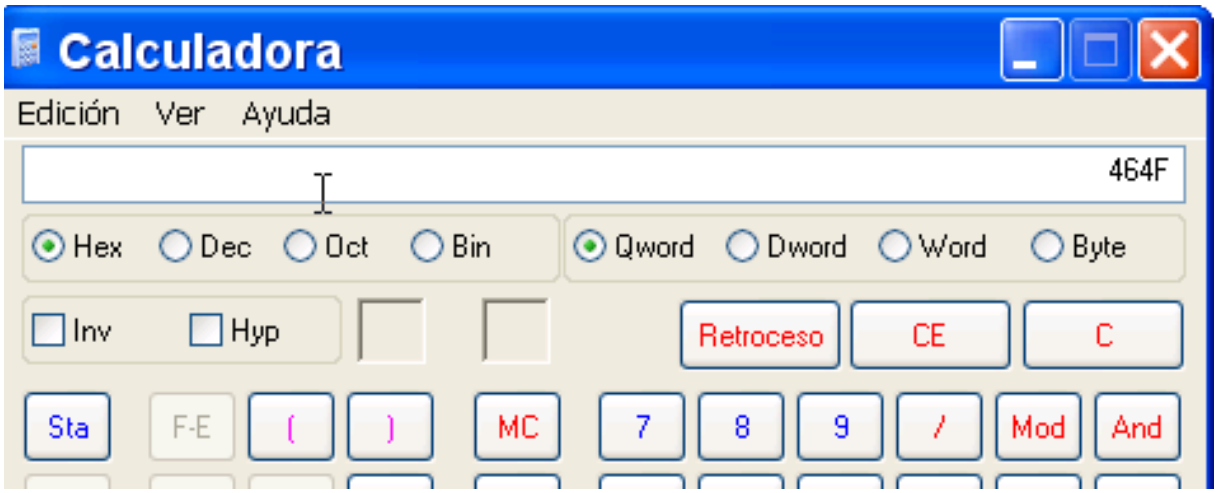
И поскольку в EAX есть значение (547B), то можем заменить на него EAX.

547B XOR 1234 = (Шестнадцатеричное значение правильного серийника)

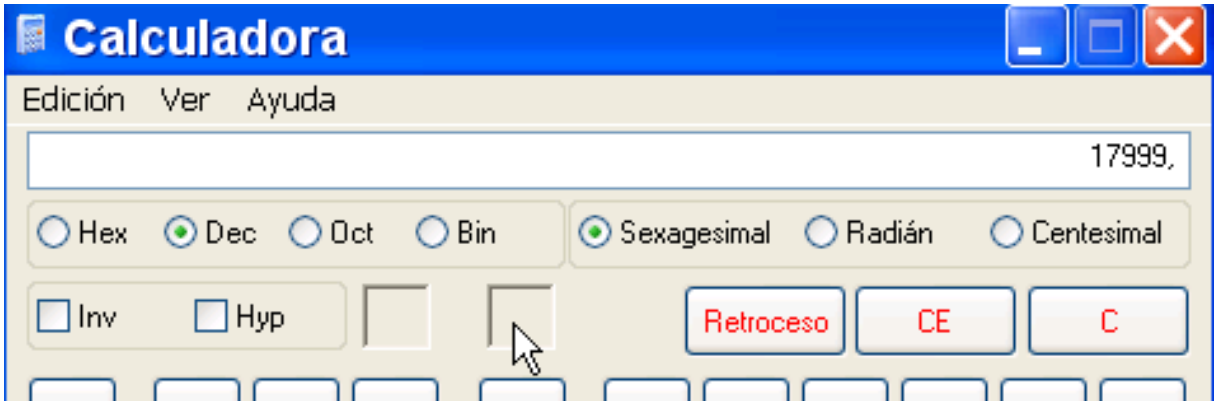
Если посчитаем результат операции XOR:

464F = (Шестнадцатеричное значение правильного серийника)

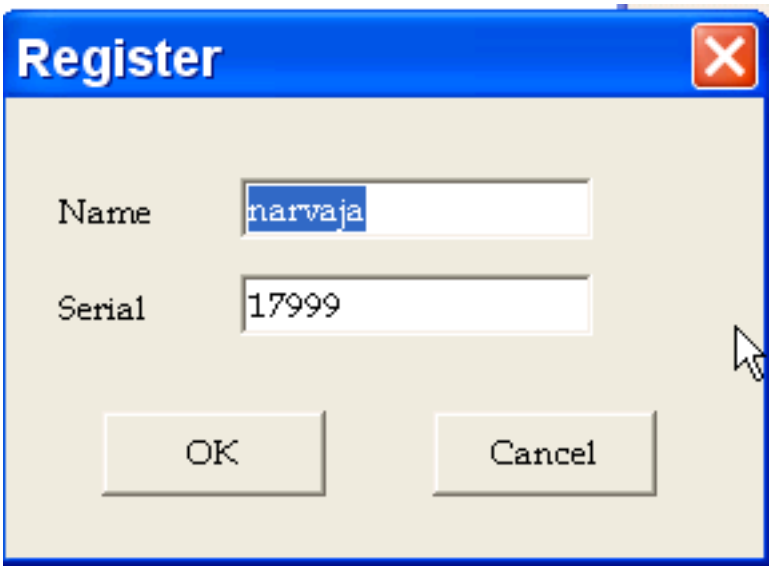
Если 464F – это шестнадцатеричное значение, то соответствующее десятеричное, которое нужно ввести, будет:



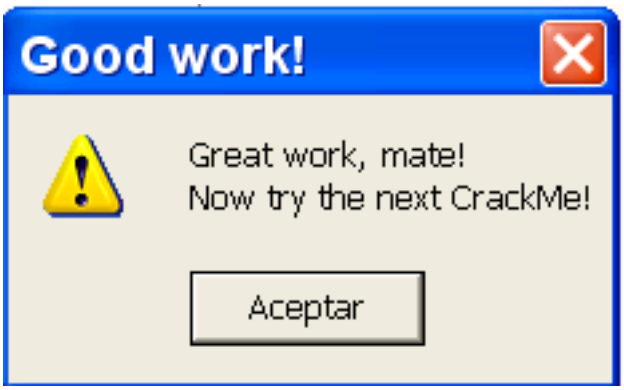
Переводим в десятеричную систему.



Это и есть правильный серийный номер для имени “narvaja”. Убираем все BPM и BP.



Нажимаем OK.



Так мы получили серийник для моего имени.

Это один способ решения. Можно прибегнуть и к другому ходу мысли, чтобы придти к тому же результату.

Если над моим неправильным серийным номером выполняются некие операции, трансформирующие его, то эти операции назовём функцией F.

F (неправильный серийный номер) =EBX

То есть над неправильным серийником производятся операции, называемые функцией F, результат чего сохраняется в EBX, с которым и происходит сравнение.

Другим членом сравнения является EAX, что приводит нас к следующему равенству:

F (правильный серийный номер)=EAX

Применение вышеуказанных операций над правильным серийным номером в данном случае даст то же значение, что и в EAX, поэтому если у нас есть EAX, то мы также имеем на руках данные для сравнения и производимые операции.

Чтобы получить правильный серийный номер, нужно применить над EAX операции, обратные используемым при проверке введённого серийника.

Правильный серийный номер = (операции, обратные F)EAX

То есть, если неправильный серийный номер перед сравнением складывается с каким-то значением, то мы должны это значение отнять от EAX, таким образом применяя обратную операцию.

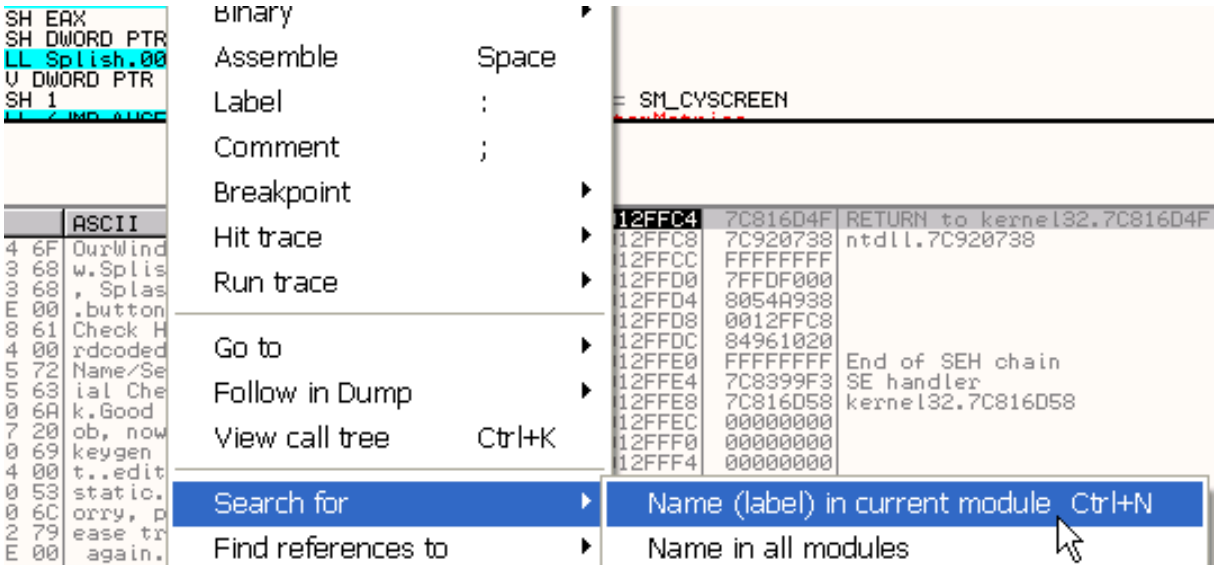
В данном случае операцией, обратной XOR, является тот же самый XOR, так как эта операция инвертируемая.

Поэтому XOR EAX будет функцией обратной F и даст мне шестнадцатеричное значение правильного серийного номера, из которого, если переведём его в десятеричную систему, получится собственно серийник.

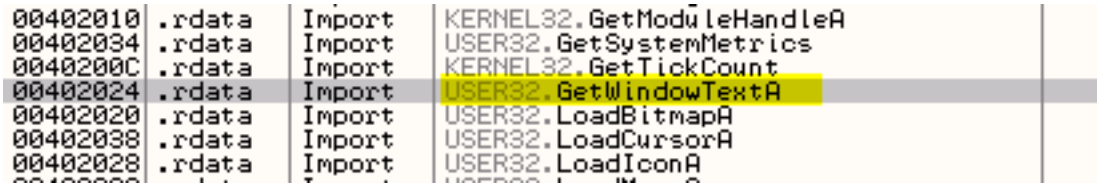
Таким образом, можем применять любой из этих способов, чтобы получить правильный серийный номер, за исключением тех случаев, когда у операции нет обратного эквивалента. Мы рассмотрим и такие варианты.

Теперь снова вернёмся к SPLISH. Мы решили ту часть, которая касалась жёстко заданного серийника, а теперь решим вторую с вводом имени пользователя.

Загрузим SPLISH в OllyDbg и окажемся на точке входа.

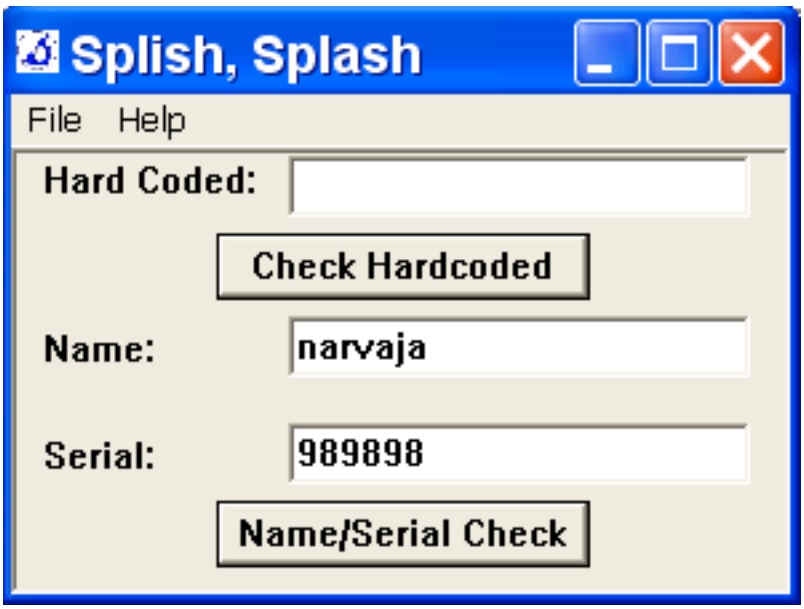


Смотрим используемые API-функции.

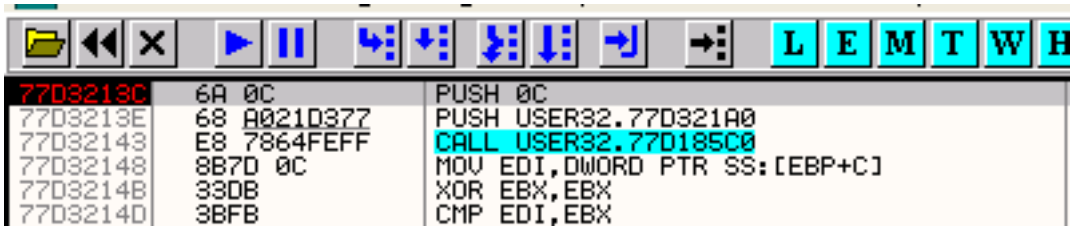


Ставим BPX на уже знакомую нам GetWindowTextA.

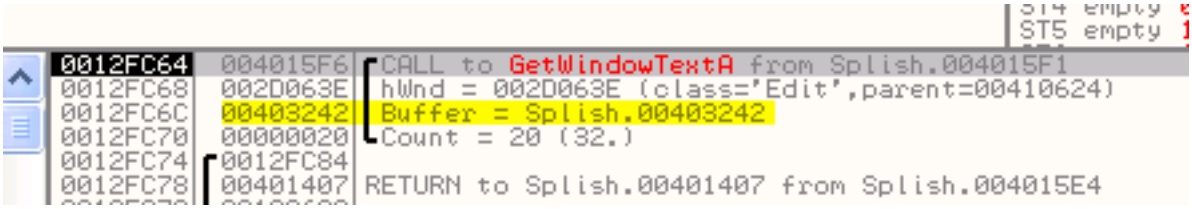
И запускаем программу с помощью F9.



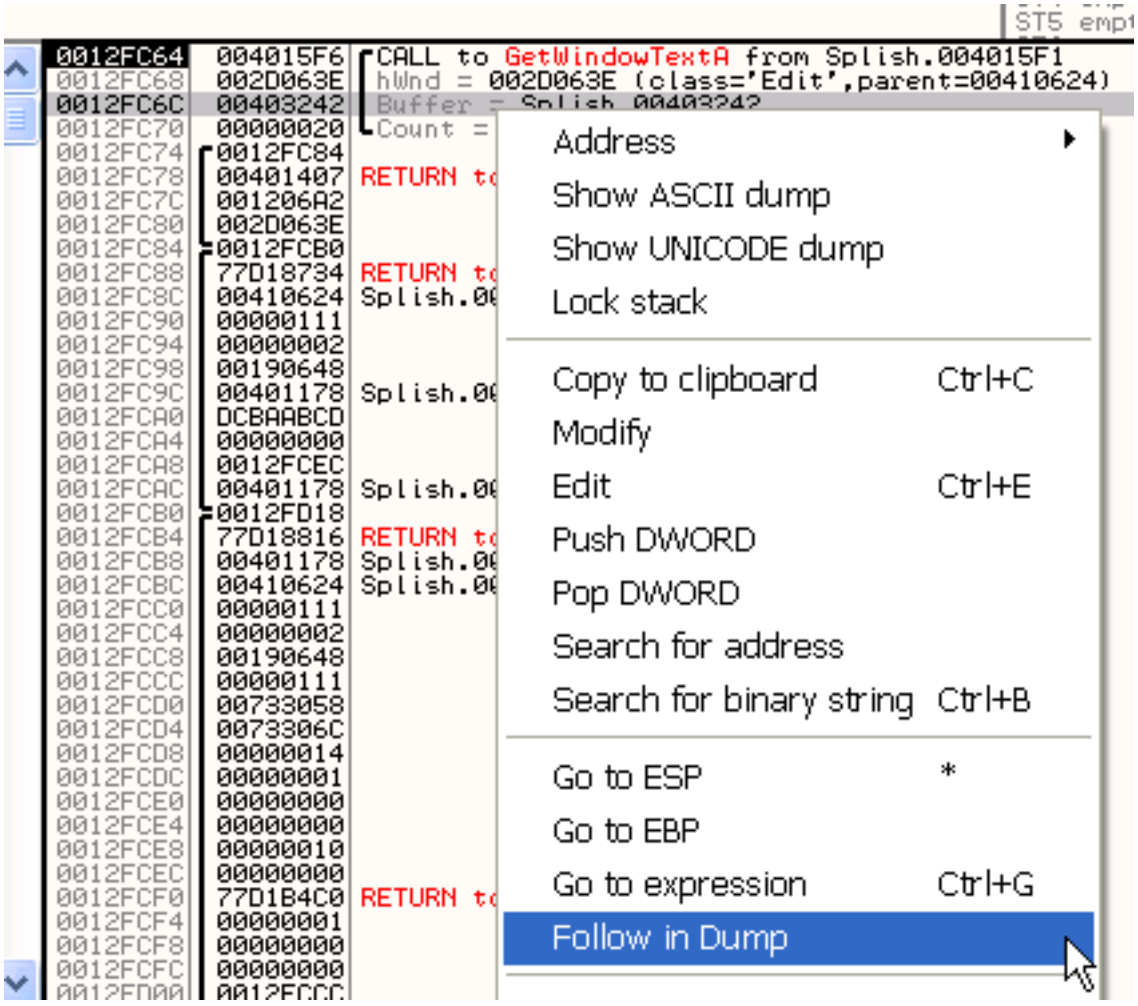
Вводим имя и неверный серийник, после чего нажимаем кнопку NAME/SERIAL CHECK.

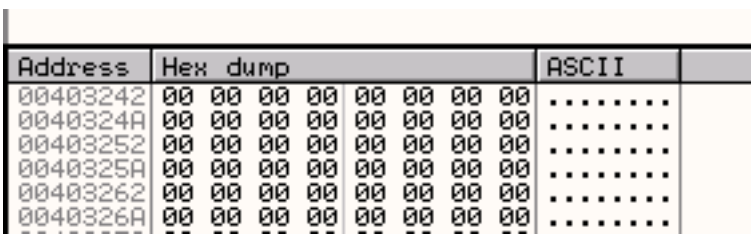


Остановившись на вызове API-функции, смотрим буфер.

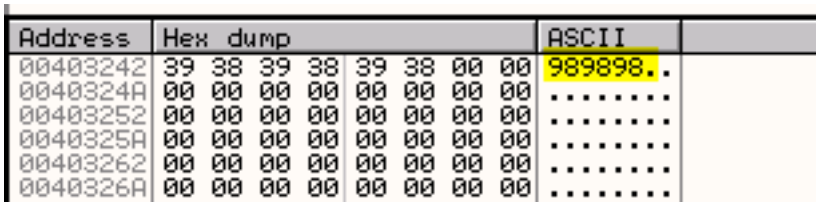


Ищем буфер в DUMP’е.

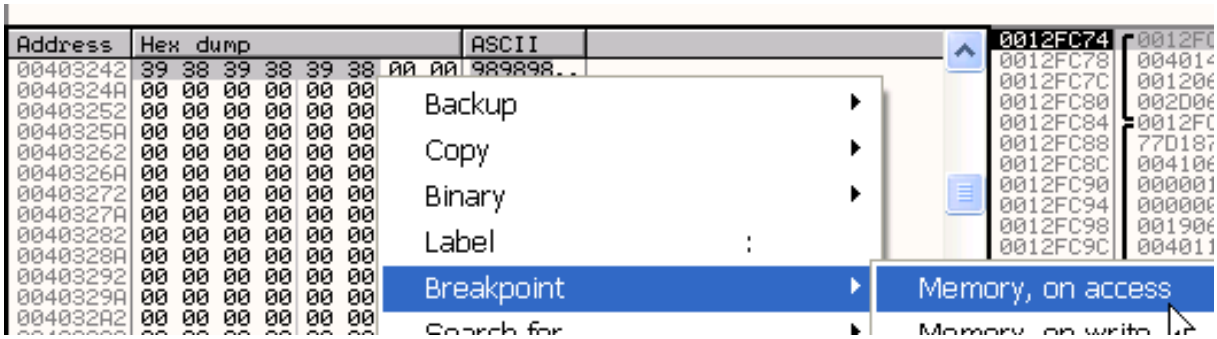




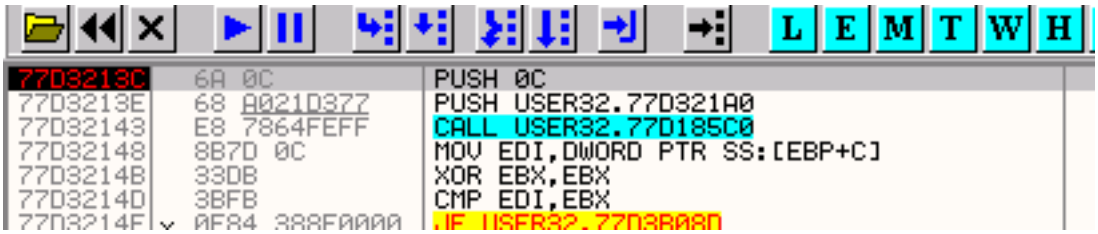
Так как здесь ничего нет, выполним EXECUTE TILL RETURN, чтобы остановиться на RET, а заме нажмём F7 для возврата в программу.



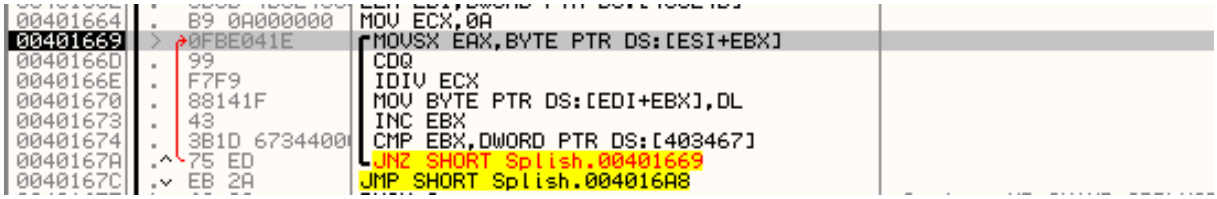
Видим, что сначала получили неправильный серийный номер, поэтому отмечаем его и устанавливаем BPM ON ACCESS.



Делаем RUN.

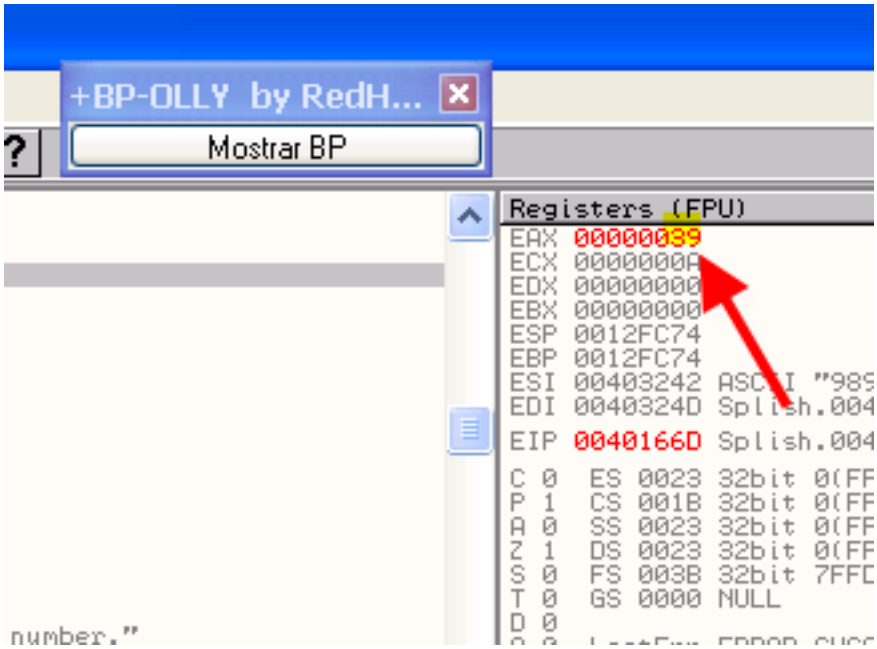


Снова останавливаемся на вызове той же функции, которая в этот раз получает из формы имя, но поскольку нам это сейчас не интересно, снова нажимаем RUN.



Останавливаемся на этой процедуре.

Первая строка перемещает первый байт неправильного серийник. Если выполним её:



Дальше следует инструкция CDQ. Посмотрим, что о ней говорит наш друг Google.

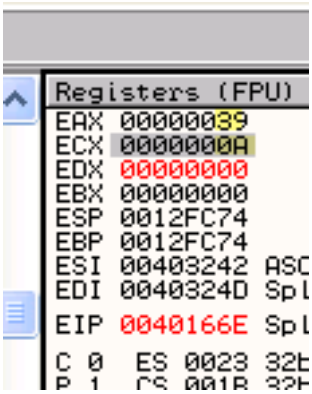
CDQ

IDIV ESI

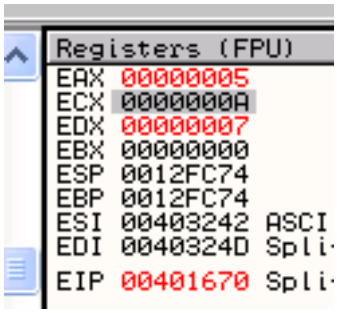
Инструкция IDIV ESI делит содержимое EDX:EAX на ESI, сохраняя результат деления в EAX, а остаток в EDX. Чтобы быть это деление было эффективным, содержимое EDX должно быть равно нулю, что можно сделать с помощью CDQ или XOR EDX, EDX. В этом случае не нужно помещать ноль в регистр EDX, потому что результат предыдущей инструкции уже делает это.

В этом случае EDX подготавливается перед делением для получения остатка, а поскольку у нас здесь цикл, то каждый раз для этой цели перед выполнением команды IDIV используется CDQ.

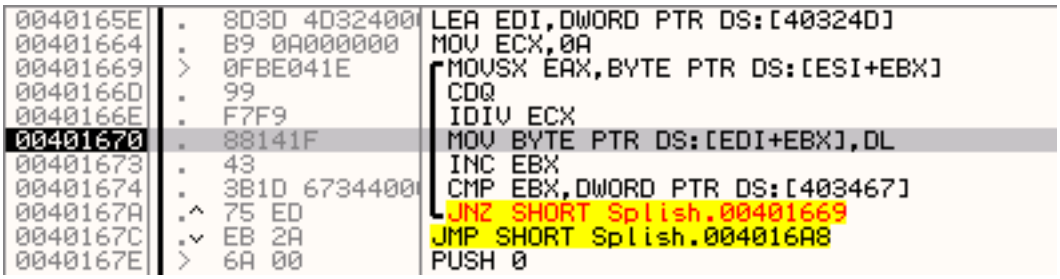
EDX:EAX делится на ECX, результат помещается в EAX, а остаток в EDX. Ок, это посмотрим, как это работает. Первый байт равен 39, он делится на ECX, который содержит 0A.



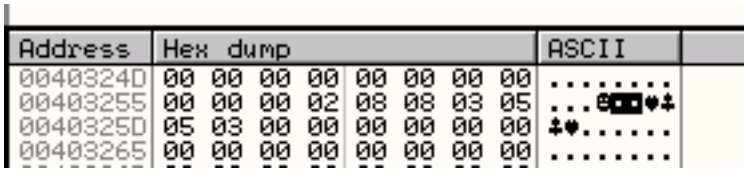
Видим, что получилось.



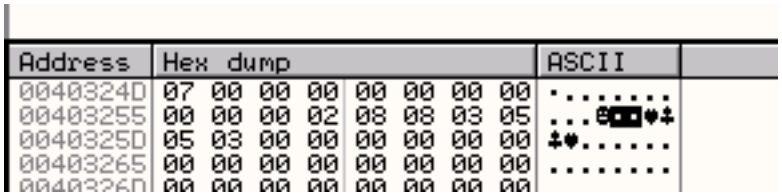
Здесь результат деления (5) оказывается в EAX, а остаток (7) – в EDX.



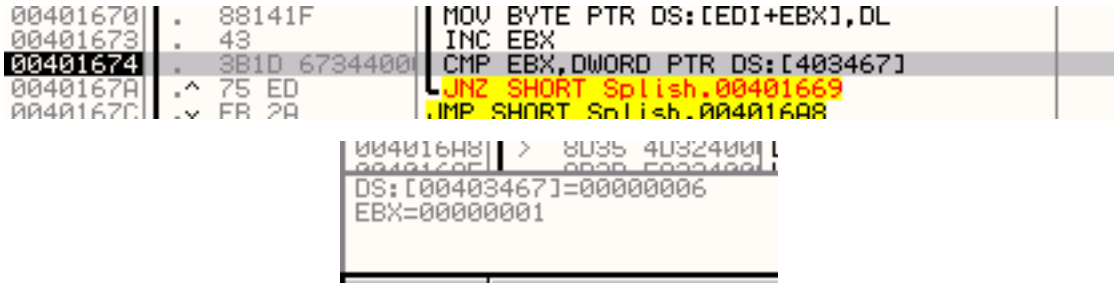
Видим, что следующая строка сохраняет результат деления в 40324D. Посмотрим через DUMP, что там.



Продолжаем выполнение с помощью F7.

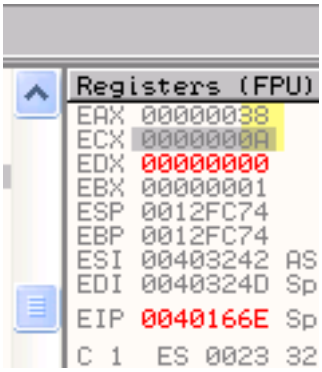


Здесь сохраняется 7.

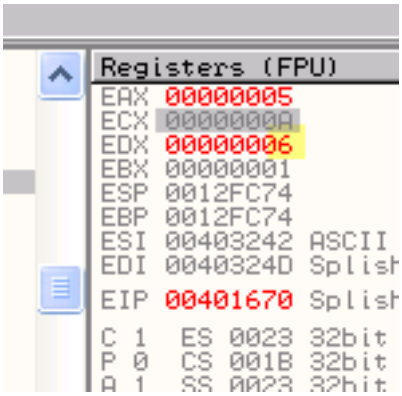


Видим, что EBX, равный нулю, увеличивается на 1 и сравнивается с 6, и так как эти значения не равны, то цикл повторяется.

Теперь посмотрим, что происходит со вторым байтом.



Снова происходит деление на 0A и сохранение остатка.



```
00401670  .  88141F          MOV BYTE PTR DS:[EDI+EBX],DL
00401673  .  43              INC EBX
00401674  .  3B1D 6734400(   CMP EBX,DWORD PTR DS:[403467]
0040167A  .^ 75 ED           JNZ SHORT Splish.00401669
0040167C  .v EB 2A           JMP SHORT Splish.004016A8
0040167E  >  6A 00           PUSH 0
00401680  .  68 0A304000     PUSH Splish.0040300A
00401685  .  68 A0304000     PUSH Splish.004030A0
0040168A  .  6A 00           PUSH 0
0040168C  .  E8 B7000000     CALL <JMP.&USER32.MessageBoxA>
00401691  .v EB 62           JMP SHORT Splish.004016F5
00401693  >  6A 00           PUSH 0
00401695  .  68 0A304000     PUSH Splish.0040300A
0040169A  .  68 B8304000     PUSH Splish.004030B8
0040169F  .  6A 00           PUSH 0
004016A1  .  E8 A2000000     CALL <JMP.&USER32.MessageBoxA>
004016A6  .v EB 4D           JMP SHORT Splish.004016F5
004016A8  >  8D35 4D32400(   LEA ESI,DWORD PTR DS:[40324D]

DL=06
DS:[0040324E]=00

Address   Hex dump                        ASCII
0040324D  07 00 00 00 00 00 00 00  ........
00403255  00 00 00 02 08 08 03 05  ...◘◘♥♦
0040325D  05 03 00 00 00 00 00 00  ♣♥......
00403265  00 00 00 00 00 00 00 00  ........
0040326D  00 00 00 00 00 00 00 00  ........
00403275  00 00 00 00 00 00 00 00  ........
```

Сохраняем…

```
Address   Hex dump                        ASCII
0040324D  07 06 00 00 00 00 00 00  •♠......
00403255  00 00 00 02 08 08 03 05  ...◘◘♥♦
0040325D  05 03 00 00 00 00 00 00  ♣♥......
00403265  00 00 00 00 00 00 00 00  ........
0040326D  00 00 00 00 00 00 00 00  ........
00403275  00 00 00 00 00 00 00 00  ........
```

Ок, одна и та же операция производится над всеми байтами.

```
Address   Hex dump                        ASCII
0040324D  07 06 07 06 07 06 00 00  •♠•♠•♠..
00403255  00 00 00 02 08 08 03 05  ...◘◘♥♦
0040325D  05 03 00 00 00 00 00 00  ♣♥......
00403265  00 00 00 00 00 00 00 00  ........
```

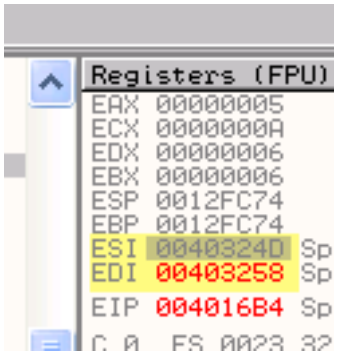
И выходим из цикла, так как не происходит перехода с помощью JNZ, возвращавшего нас раньше.

```
00401674  .  3B1D 6734400(   CMP EBX,DWORD PTR DS:[403467]
0040167A  .^ 75 ED           JNZ SHORT Splish.00401669
0040167C  .v EB 2A           JMP SHORT Splish.004016A8
0040167E  >  6A 00           PUSH 0                          Style = MB_OK|MB_APPLMODAL
00401680  .  68 0A304000     PUSH Splish.0040300A            Title = "Splish, Splash"
00401685  .  68 A0304000     PUSH Splish.004030A0            Text = "Please enter your name."
0040168A  .  6A 00           PUSH 0                          hOwner = NULL
0040168C  .  E8 B7000000     CALL <JMP.&USER32.MessageBoxA>  MessageBoxA
00401691  .v EB 62           JMP SHORT Splish.004016F5
00401693  >  6A 00           PUSH 0                          Style = MB_OK|MB_APPLMODAL
00401695  .  68 0A304000     PUSH Splish.0040300A            Title = "Splish, Splash"
0040169A  .  68 B8304000     PUSH Splish.004030B8            Text = "Please enter your serial number."
0040169F  .  6A 00           PUSH 0                          hOwner = NULL
004016A1  .  E8 A2000000     CALL <JMP.&USER32.MessageBoxA>  MessageBoxA
004016A6  .v EB 4D           JMP SHORT Splish.004016F5
004016A8  >  8D35 4D32400(   LEA ESI,DWORD PTR DS:[40324D]
004016AE  .  8D3D 5832400(   LEA EDI,DWORD PTR DS:[403258]
```

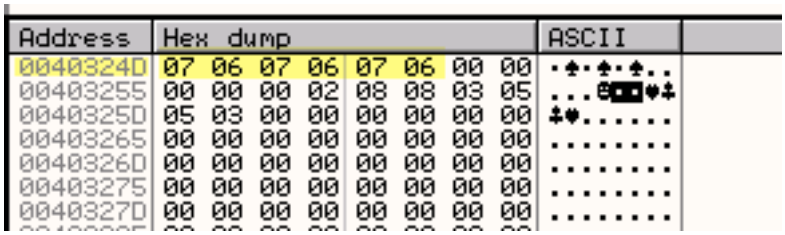
Попадаем сюда и видим сообщения о правильном и неправильном вводе серийного номера, так что похоже, что находимся рядом с тем, что нам нужно.

```
004016A1  .  E8 A2000000   CALL <JMP.&USER32.MessageBoxA>                  MessageBoxA
004016A6  .v EB 4D         JMP SHORT Splish.004016F5
004016A8  > 8D35 4D324000  LEA ESI,DWORD PTR DS:[40324D]
004016AE  .  8D3D 5832400( LEA EDI,DWORD PTR DS:[403258]
004016B4  .  33DB          XOR EBX,EBX
004016B6  >  3B1D 6334400( CMP EBX,DWORD PTR DS:[403463]
004016BC  .v 74 0F         JE SHORT Splish.004016CD
004016BE  .  0FBE041F      MOVSX EAX,BYTE PTR DS:[EDI+EBX]
004016C2  .  0FBE0C1E      MOVSX ECX,BYTE PTR DS:[ESI+EBX]
004016C6  .  3BC1          CMP EAX,ECX
004016C8  .v 75 18         JNZ SHORT Splish.004016E2
004016CA  .  43            INC EBX
004016CB  .^ EB E9         JMP SHORT Splish.004016B6
004016CD  >  6A 00         PUSH 0                                          Style = MB_OK|MB_APPLMODAL
004016CF  .  68 0A304000   PUSH Splish.0040300A                            Title = "Splish, Splash"
004016D4  .  68 42304000   PUSH Splish.00403042                           Text = "Good job, now keygen it."
004016D9  .  6A 00         PUSH 0                                          hOwner = NULL
004016DB  .  E8 68000000   CALL <JMP.&USER32.MessageBoxA>                  MessageBoxA
004016E0  .v EB 13         JMP SHORT Splish.004016F5
004016E2  >  6A 00         PUSH 0                                          Style = MB_OK|MB_APPLMODAL
004016E4  .  68 0A304000   PUSH Splish.0040300A                            Title = "Splish, Splash"
004016E9  .  68 67304000   PUSH Splish.00403067                           Text = "Sorry, please try again."
004016EE  .  6A 00         PUSH 0                                          hOwner = NULL
004016F0  .  E8 53000000   CALL <JMP.&USER32.MessageBoxA>                  MessageBoxA
004016F5  >  C9            LEAVE
004016F6  L. C2 0800       RETN 8
```

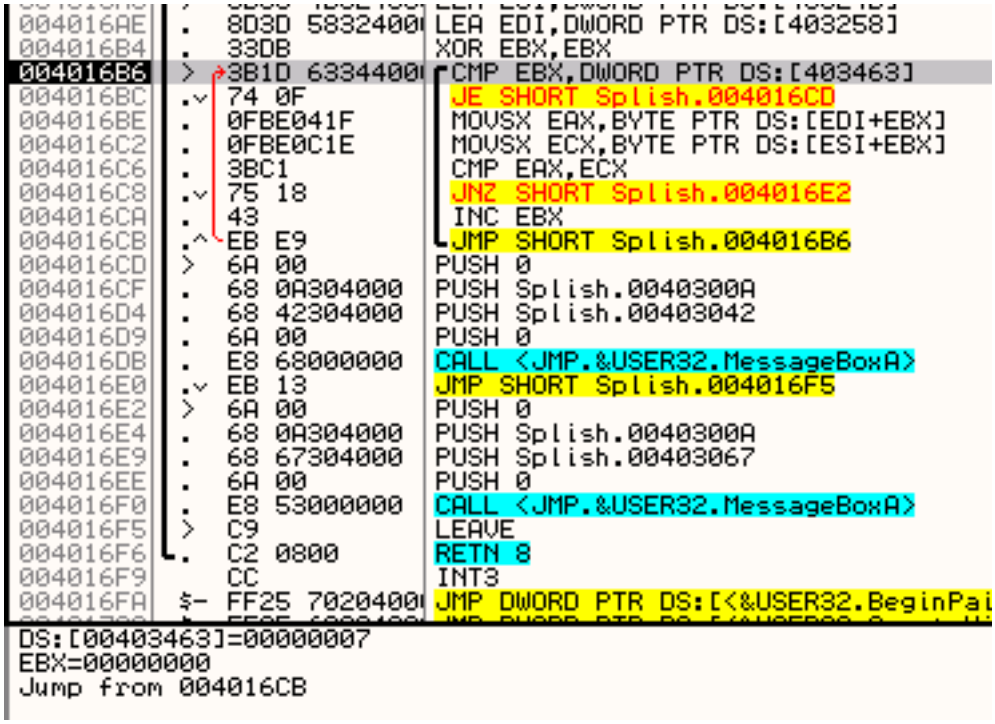
Здесь два LEA, которые помещают адреса в ESI и EDI.



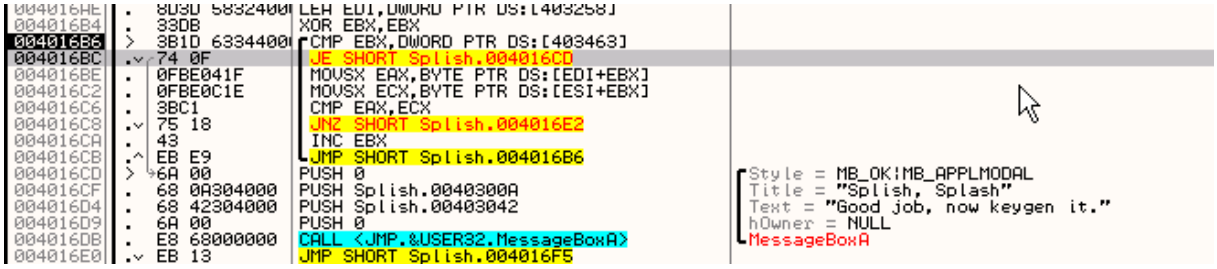
ESI указывает туда, где сохраняются остатки, а EDI, возможно, на что-то интересное?



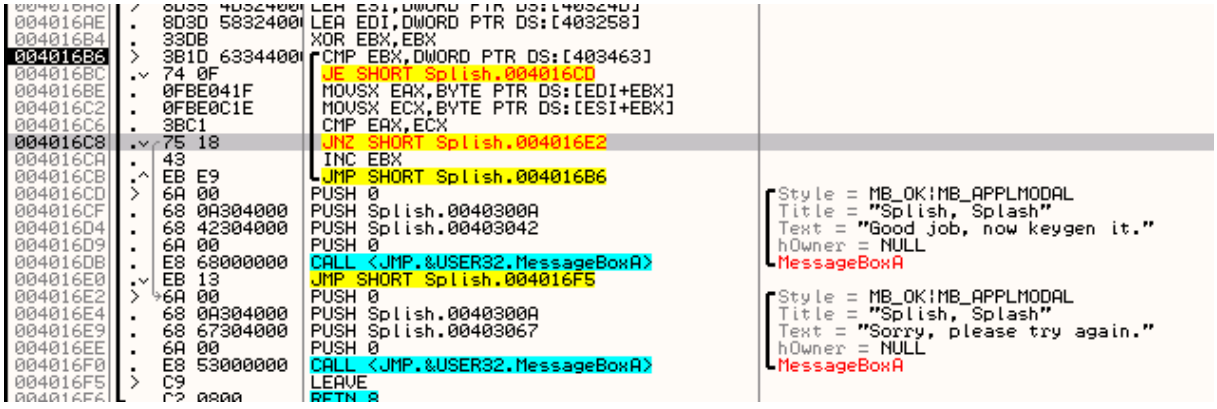
Продолжаем.



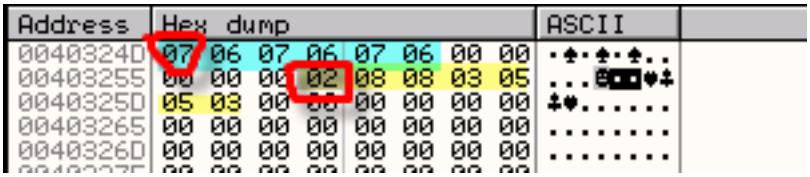
Видим, что EBX содержит ноль, который сравнивается с 7, и если бы они были равны…



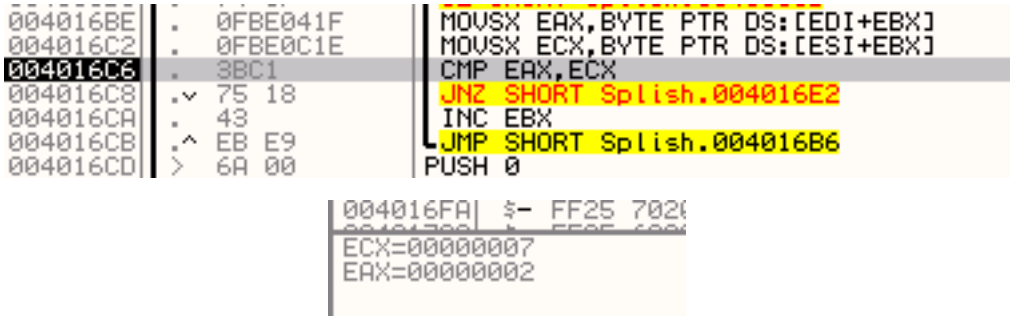
Перешли бы на окошко с сообщением о правильном вводе. В середине находится другой JNZ, который перемещает на другое окошко с сообщением о вводе неправильном.



Ок, видим, что сравнивается.



В EAX переместится первый байт, на который указывает EDI, то есть 02, а в ECX будет перемещён первый байт наших остатков, то есть 7.



И если остаток первого байта был бы равен 02, то был бы совершён переход.

В моём случае найти остаток можно было бы следующим образом.

39= 5 .0A + 7

То есть результатом деления 39 на 0A является 5 с остатком 7. Переформулируя, можно сказать, что умножив 0A на 5, получим 32 и прибавление 7 даст 39.

39= 5 x 0A + 7

Правильный остаток должен быть равен двум в случае правильного серийного номера.

BYTE CORRECTO= 5 x 0A + 2

BYTE CORRECTO= 32 +2 = 34

34 это цифра ‘4’ в ASCII.

Мы можем также увидеть, что

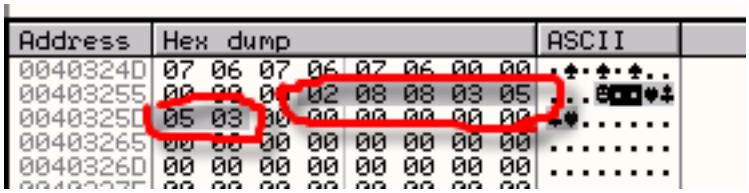
BYTE CORRECTO= 5 x 0A + 2

Даёт нам одно из цифровых значений (от 30 до 39). В случае, если результат выходит за эту границу, то нужно сделать понижение и считать так:

BYTE CORRECTO= 4 x 0A + RESTO

Ок, посчитали, что первый байт равен 34 или ‘4’ в ASCII.

Высчитаем другие правильные байты с помощью наших остатков.



С 02 уже посчитали.

Продолжаем с 08.

ПРАВИЛЬНЫЙ БАЙТ = 5 x 0A + 8

ПРАВИЛЬНЫЙ БАЙТ = 32 + 8

Получилось 40, что перевышаем максимальное установленное нами значение 39 (‘9’ в ASCII), поэтому понижаем и пересчитываем заново:

ПРАВИЛЬНЫЙ БАЙТ = 4 x 0A + 8

ПРАВИЛЬНЫЙ БАЙТ = 28 + 8 = 30 que es 0 en ASCII

Если хотите доказательство, то смотрим, что 30, делённое на 0A, даёт 4 с остатком 8.

Поэтому у нас есть второй правильный байт, равный 30, то есть ‘0’ в ASCII.

Посчитаем всё остальное байт за байтом.

Следующий опять 08, то есть результатом будет 30 или ‘0’ в ASCII.

Потом 03.

ПРАВИЛЬНЫЙ БАЙТ = 5 x 0A + 3

ПРАВИЛЬНЫЙ БАЙТ = 32+ 3= 35, то есть ‘5’ в ASCII.

Затем 05.

ПРАВИЛЬНЫЙ БАЙТ = 5 x 0A + 5

ПРАВИЛЬНЫЙ БАЙТ = 32+ 5= 37, то есть ‘7‘ в ASCII

Потом снова идёт 5, то есть повторяется 37 или ‘7’ в ASCII.

Последним идёт 3, который, как мы уже знаем, даёт 35 или ‘5’ в ASCII.

Таким образом, правильным серийным номером для имени ‘narvaja’ будет 4005775. Убираем все BPM и BPX и запускаем RUN.

Нажимаем кнопку NAME/SERIAL CHECK.

Ещё одна победа.

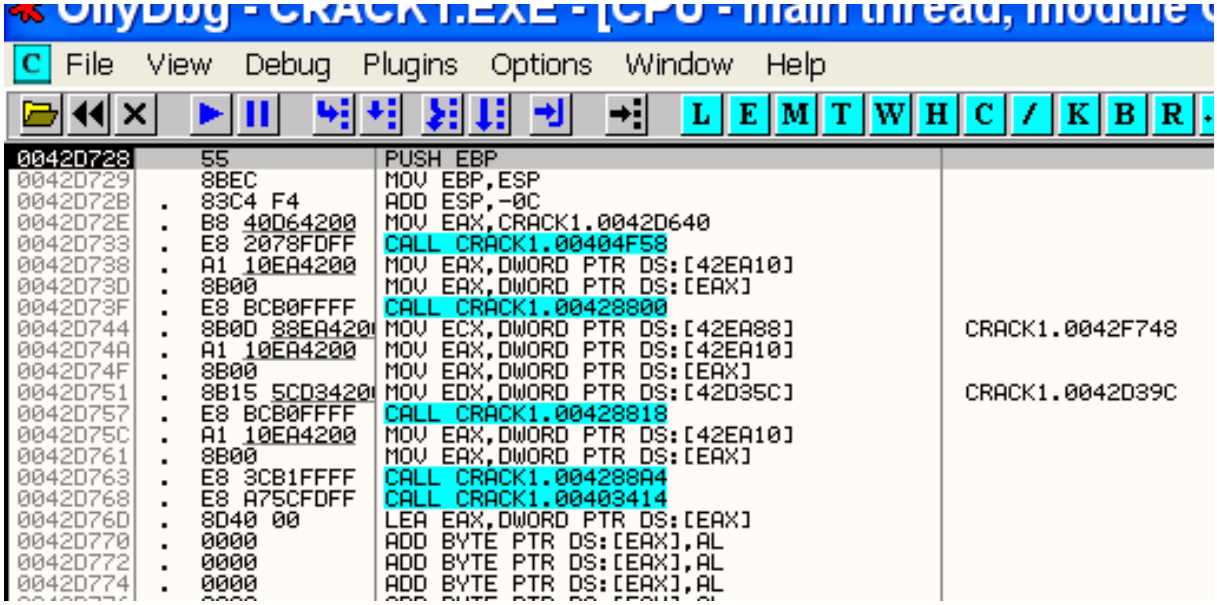
К этой главе прилагаетcz крэкми для практики под названием MEXCRK1.ZIP (находится в архиве `wasm_files.zip`: `files/0470/mexcrk1.zip`).

Введение в крэкинг с нуля, используя OllyDbg - Глава 17

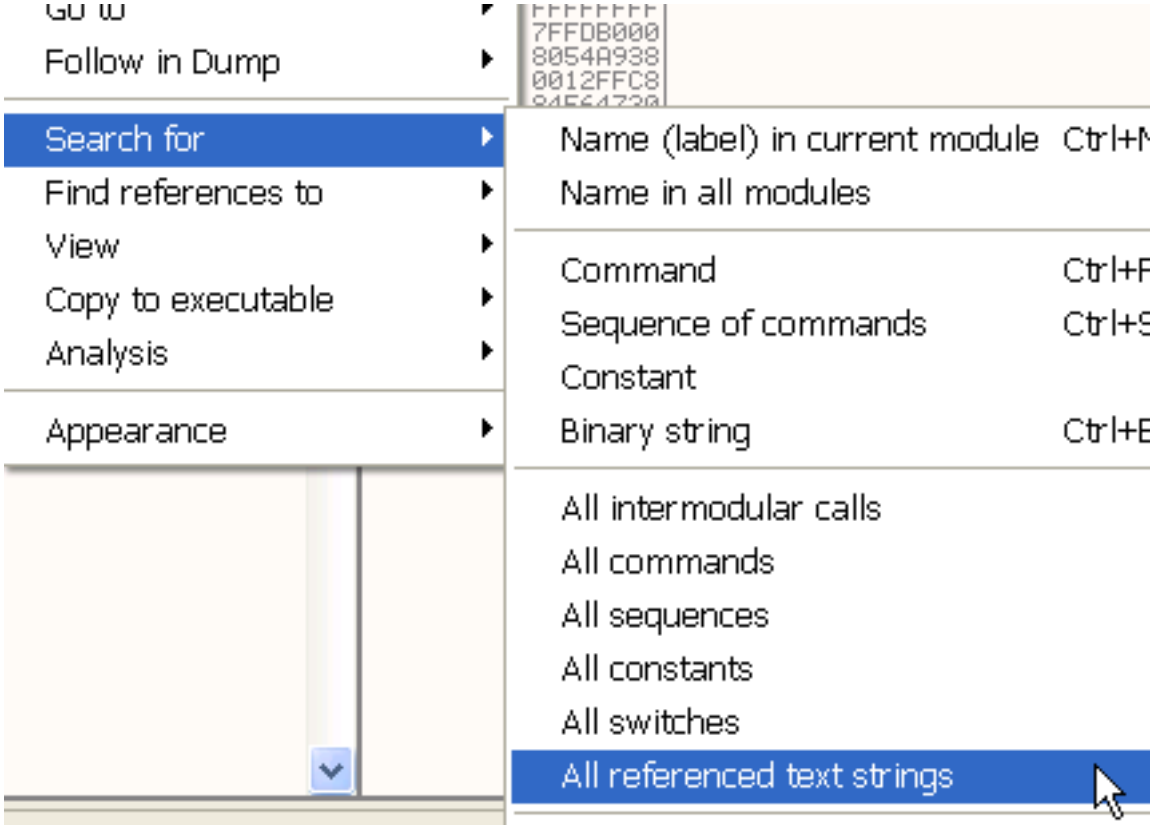
Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-01-25

Хорошо, прежде всего мы решим крэкми MEXCRK1.ZIP (находится в архиве `wasm_files.zip`: `files/0471/mexcrk1.zip`). Он очень лёгкий. Откроем его в OllyDbg и окажемся в точке входа.



Смотрим строки, которые используются программой. Нажимаем правую кнопку мыши.



Видим среди строк следующие:

```
0042D4F6 ASCII "TForm1"
0042D507 ASCII "crackme"
0042D537 MOV EDX,CRACK1.0042D590          ASCII "Benadryl"
0042D543 MOV EDX,CRACK1.0042D5A4          ASCII "Wrong Code DUDE"
0042D555 MOV EDX,CRACK1.0042D5BC          ASCII "Thanks you made it"
0042D590 ASCII "Benadryl",0
```

Можем кликнуть на той, которая сообщает об успехе, или сообщающей о неудаче, чтобы попасть в интересующую нас область программы.

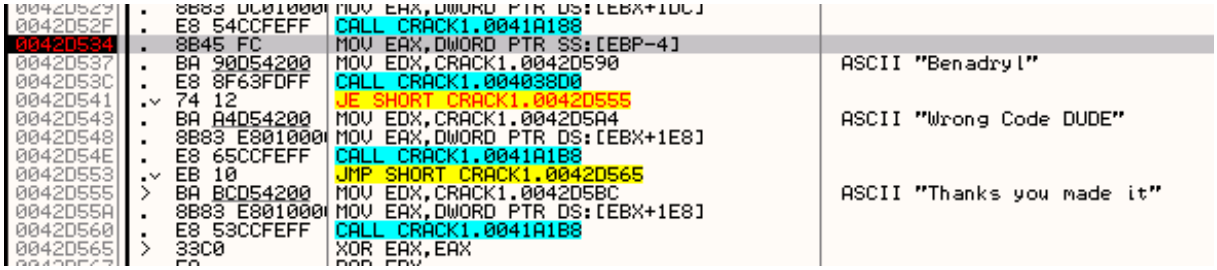
```
0042D526  .  8D55 FC       LEA EDX,DWORD PTR SS:[EBP-4]
0042D529  .  8B83 DC01000  MOV EAX,DWORD PTR DS:[EBX+1DC]
0042D52F  .  E8 54CCFEFF   CALL CRACK1.0041A188
0042D534  .  8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
0042D537  .  BA 90D54200   MOV EDX,CRACK1.0042D590          ASCII "Benadryl"
0042D53C  .  E8 8F63FDFF   CALL CRACK1.004038D0
0042D541  .v 74 12         JE SHORT CRACK1.0042D555
0042D543  .  BA A4D54200   MOV EDX,CRACK1.0042D5A4          ASCII "Wrong Code DUDE"
0042D548  .  8B83 E801000  MOV EAX,DWORD PTR DS:[EBX+1E8]
0042D54E  .  E8 65CCFEFF   CALL CRACK1.0041A1B8
0042D553  .v EB 10         JMP SHORT CRACK1.0042D565
0042D555  >  BA BCD54200   MOV EDX,CRACK1.0042D5BC          ASCII "Thanks you made it"
0042D55A  .  8B83 E801000  MOV EAX,DWORD PTR DS:[EBX+1E8]
0042D560  .  E8 53CCFEFF   CALL CRACK1.0041A1B8
0042D565  >  33C0          XOR EAX,EAX
0042D567  .  5A            POP EDX
```

Видим, что здесь есть call, после которого либо происходит управление передаётся на код либо с «WRONG CODE DUDE», либо с «THANKS YOU MADE IT».

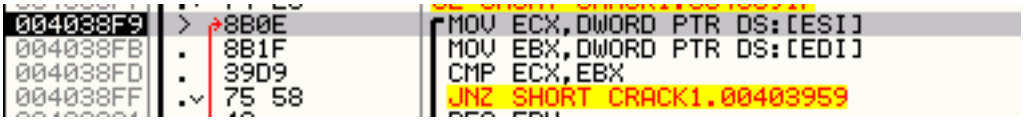
Устанавливаем BPX на CALL в 42d534 и нажимаем RUN.

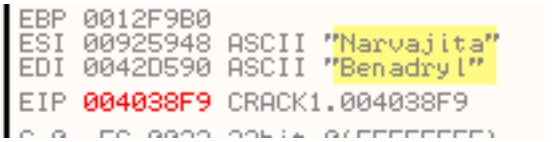
Введём неправильный серийный номер в поле, которое называется «Enter Serial #».

В моём случае печатаем Narvajita.



Если войдём в режиме трассировки в CALL, то увидим, что сравнивается введённый серийник «Narvajita» со строкой «Benadryl».





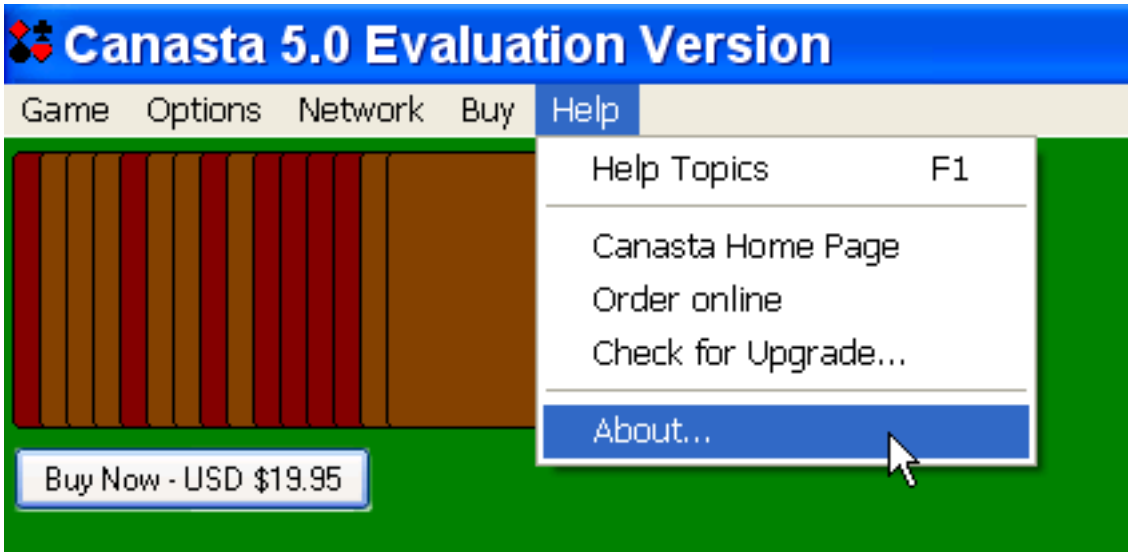
Ок, теперь мы оказываемся рядом со сравнением, так что правильным серийным номером является слово «Benadryl». Сделаем RUN по-новой, убрав предварительно все BPX.

Здесь видим, что нас поздравляют с нахождением правильного серийного номера .

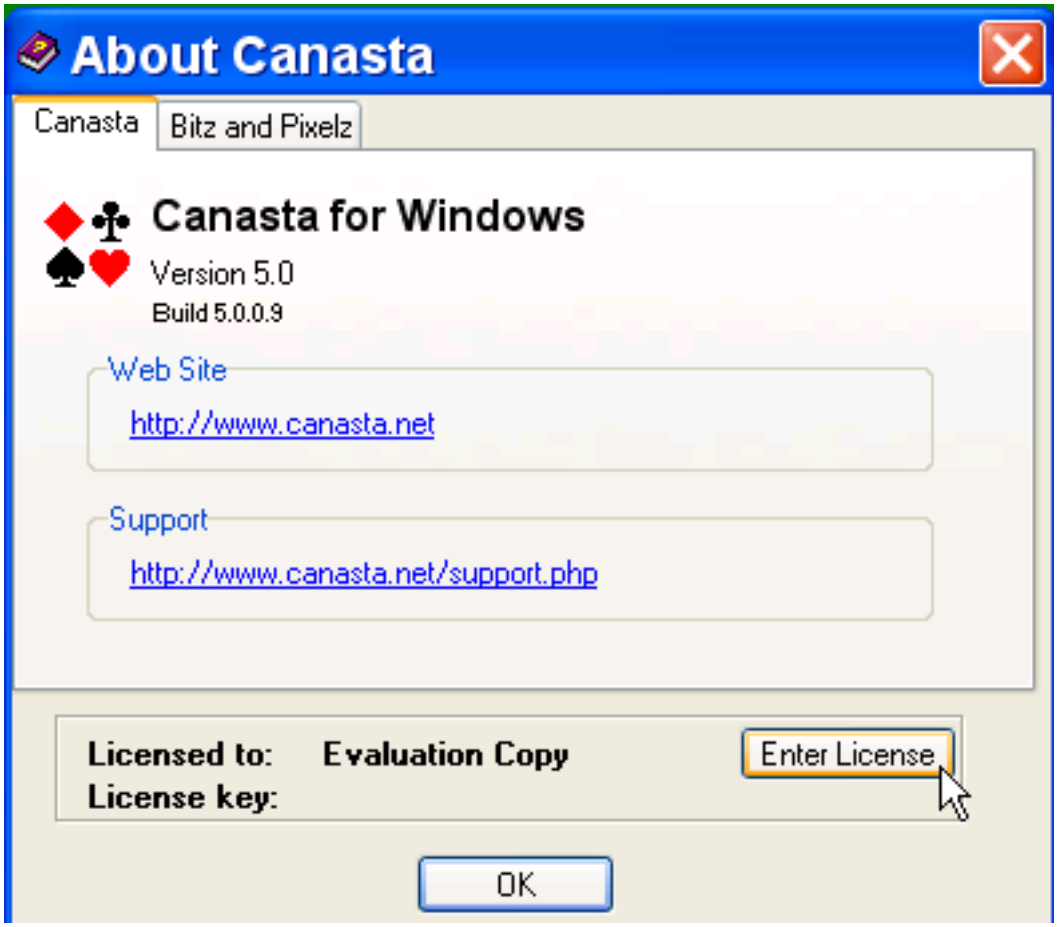
Ок, теперь разберём последний случай подобного типа, а со следующей главы начнём новую тему. Это небольшая программка, являющееся игрой под названием canasta 5.0. Она приложена (находится в архиве `wasm_files.zip`: `files/0471/ollydbg17.zip`) к статье.

Эта программа относится к той категории, где при кнопка OK в начале недоступна для нажатия, но когда пользователь заканчивает вводить свои данные, то если они верны, на неё становится возможным нажать.

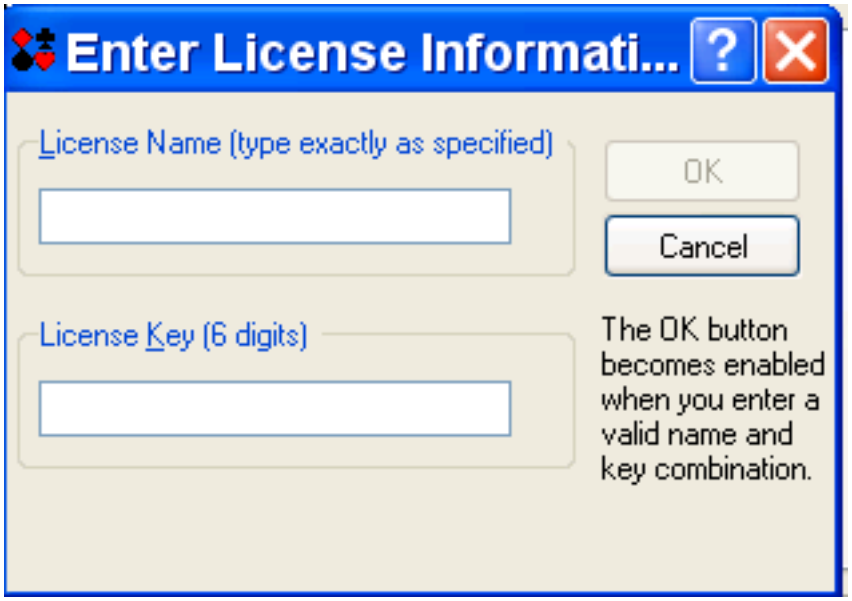
Установим программу и зайдём в About.



Она стоит, друзья, 20 долларов, вернее 19.95.

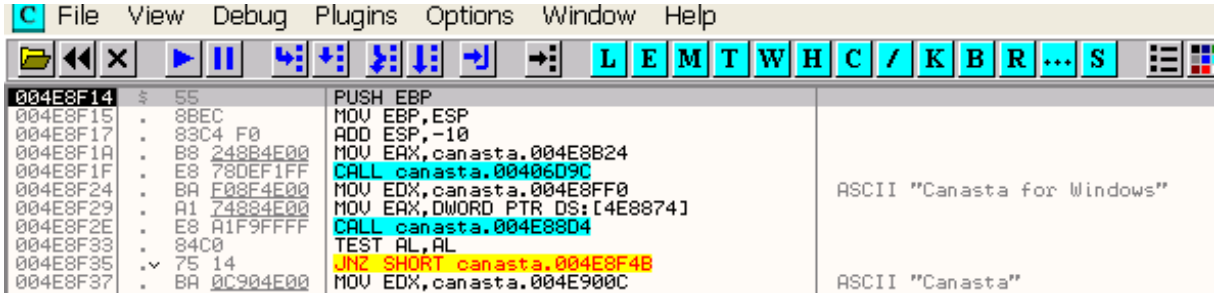


Видим кнопку «ENTER LICENSE».

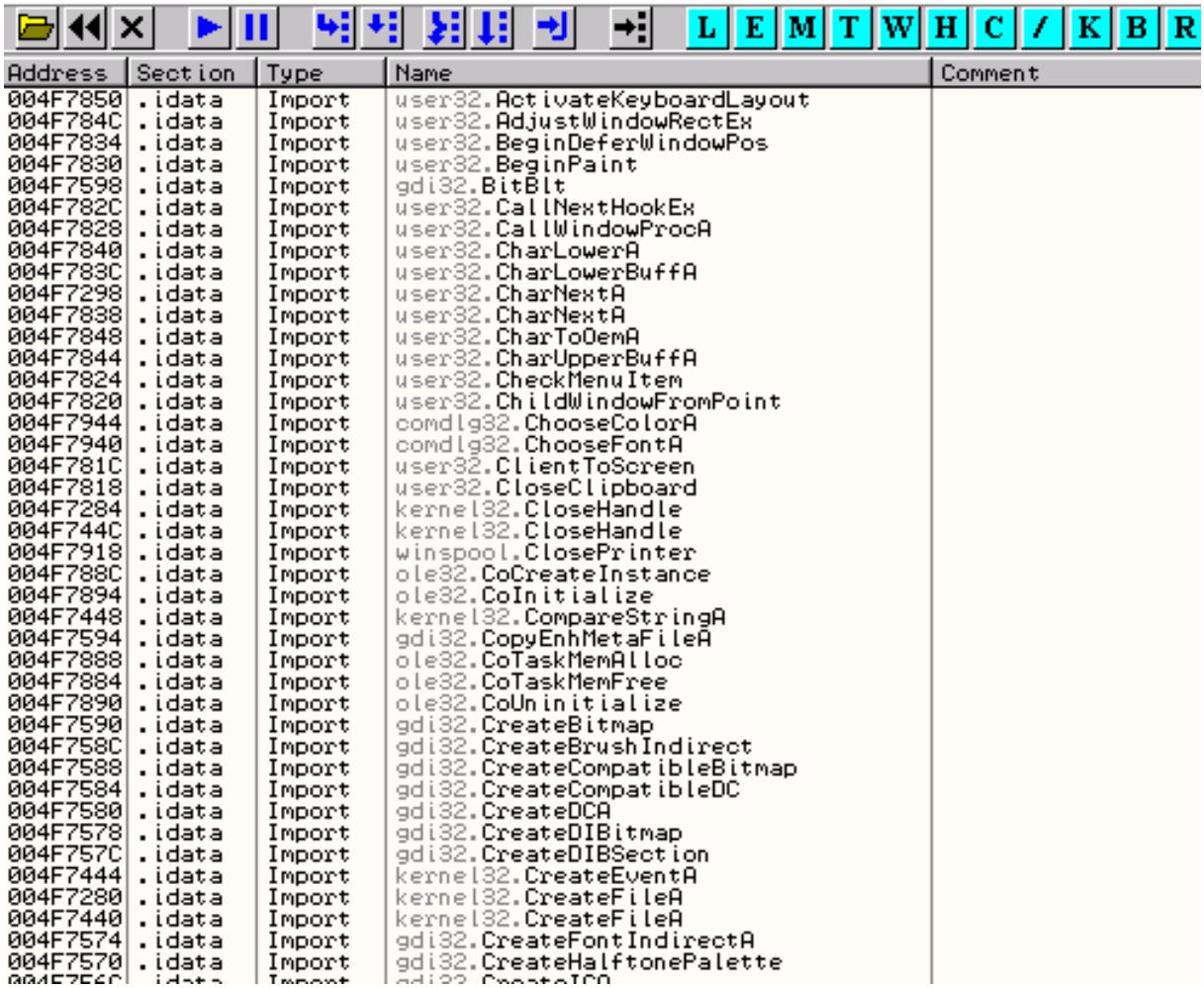


Здесь говорится, что кнопка OK станет доступна, когда будет введена правильная комбинация.

Запустим программу в OllyDbg.



Мы оказываемся в точке входа.

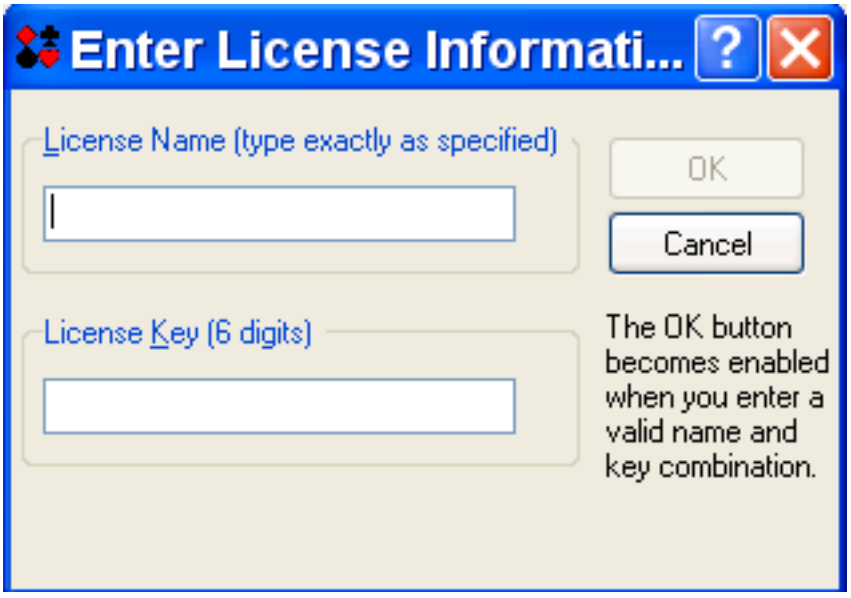


Address	Section	Type	Name	Comment
004F7850	.idata	Import	user32.ActivateKeyboardLayout	
004F784C	.idata	Import	user32.AdjustWindowRectEx	
004F7834	.idata	Import	user32.BeginDeferWindowPos	
004F7830	.idata	Import	user32.BeginPaint	
004F7598	.idata	Import	gdi32.BitBlt	
004F782C	.idata	Import	user32.CallNextHookEx	
004F7828	.idata	Import	user32.CallWindowProcA	
004F7840	.idata	Import	user32.CharLowerA	
004F783C	.idata	Import	user32.CharLowerBuffA	
004F7298	.idata	Import	user32.CharNextA	
004F7838	.idata	Import	user32.CharNextA	
004F7848	.idata	Import	user32.CharToOemA	
004F7844	.idata	Import	user32.CharUpperBuffA	
004F7824	.idata	Import	user32.CheckMenuItem	
004F7820	.idata	Import	user32.ChildWindowFromPoint	
004F7944	.idata	Import	comdlg32.ChooseColorA	
004F7940	.idata	Import	comdlg32.ChooseFontA	
004F781C	.idata	Import	user32.ClientToScreen	
004F7818	.idata	Import	user32.CloseClipboard	
004F7284	.idata	Import	kernel32.CloseHandle	
004F744C	.idata	Import	kernel32.CloseHandle	
004F7918	.idata	Import	winspool.ClosePrinter	
004F788C	.idata	Import	ole32.CoCreateInstance	
004F7894	.idata	Import	ole32.CoInitialize	
004F7448	.idata	Import	kernel32.CompareStringA	
004F7594	.idata	Import	gdi32.CopyEnhMetaFileA	
004F7888	.idata	Import	ole32.CoTaskMemAlloc	
004F7884	.idata	Import	ole32.CoTaskMemFree	
004F7890	.idata	Import	ole32.CoUninitialize	
004F7590	.idata	Import	gdi32.CreateBitmap	
004F758C	.idata	Import	gdi32.CreateBrushIndirect	
004F7588	.idata	Import	gdi32.CreateCompatibleBitmap	
004F7584	.idata	Import	gdi32.CreateCompatibleDC	
004F7580	.idata	Import	gdi32.CreateDCA	
004F7578	.idata	Import	gdi32.CreateDIBitmap	
004F757C	.idata	Import	gdi32.CreateDIBSection	
004F7444	.idata	Import	kernel32.CreateEventA	
004F7280	.idata	Import	kernel32.CreateFileA	
004F7440	.idata	Import	kernel32.CreateFileA	
004F7574	.idata	Import	gdi32.CreateFontIndirectA	
004F7570	.idata	Import	gdi32.CreateHalftonePalette	

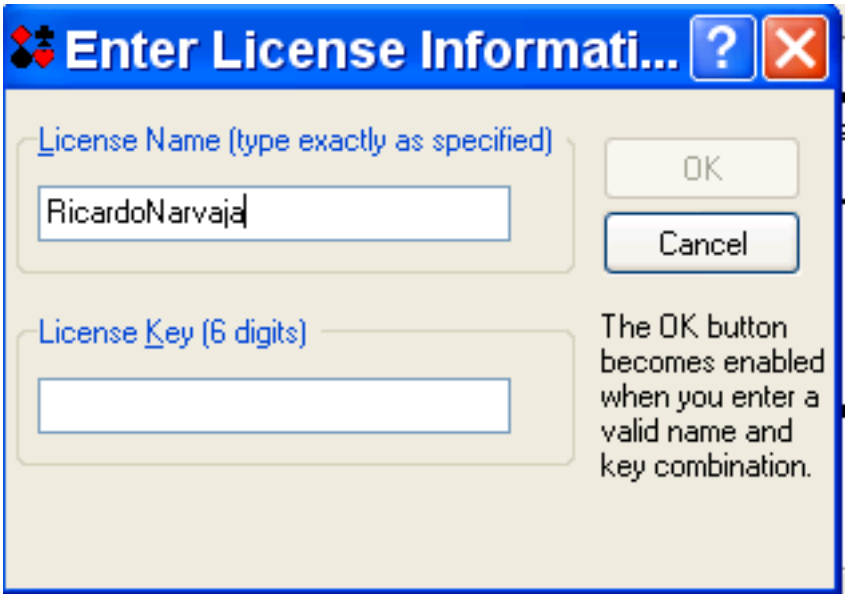
Уфф, список используемых API-функций велик, также как и список строк.

```
Address   Disassembly                                Text string
004B40E5  ASCII "als/RTDJ"
004B40F1  ASCII "Error"
004B4130  DD canasta.004B417C                        ASCII 08,"TLicense"
004B4150  DD canasta.004B417C                        ASCII 08,"TLicense"
004B417D  ASCII "TLicense"
004B4188  DD canasta.004B41D4                        ASCII 0F,"TLicenseManager"
004B41A8  DD canasta.004B41D4                        ASCII 0F,"TLicenseManager"
004B41D5  ASCII "TLicenseManager"
004B4204  MOV ESI,canasta.004B423C                   ASCII 11,"Unregistered copy"
004B423D  ASCII "Unregistered cop"
004B424D  ASCII "y"
004B42A3  MOV EDX,canasta.004B4370                   ASCII "Settings\"
004B42DF  MOV EDX,canasta.004B4370                   ASCII "Settings\"
004B4370  ASCII "Settings\",0
004B43CF  MOV EDX,canasta.004B446C                   ASCII "Settings\"
004B4400  MOV EDX,canasta.004B446C                   ASCII "Settings\"
004B446C  ASCII "Settings\",0
004B4484  MOV ESI,canasta.004B44A4                   ASCII 11,"Unregistered copy"
004B44A5  ASCII "Unregistered cop"
004B44B5  ASCII "y"
004B462D  MOV EDX,canasta.004B4690                   ASCII "Unregistered copy"
004B4690  ASCII "Unregistered cop"
004B46A0  ASCII "y",0
004B46CF  MOV EDX,canasta.004B46E8                   ASCII "Unregistered copy"
004B46E8  ASCII "Unregistered cop"
004B46F8  ASCII "y",0
004B47EB  MOV EDX,canasta.004B4880                   ASCII "Settings\DateFormat"
004B4804  PUSH canasta.004B489C                      ASCII "Registration Key"
004B4813  MOV ECX,canasta.004B48B8                   ASCII "User Name"
004B4823  PUSH canasta.004B48CC                      ASCII "CodePageEx"
004B4832  MOV ECX,canasta.004B48E0                   ASCII "CodePage"
004B4880  ASCII "Settings\DateFor"
004B4890  ASCII "mat",0
004B489C  ASCII "Registration Key"
004B48AC  ASCII 0
004B48B8  ASCII "User Name",0
004B48CC  ASCII "CodePageEx",0
004B48E0  ASCII "CodePage",0
004B493E  MOV EDX,canasta.004B49C4                   ASCII "Settings\DateFormat"
004B495C  PUSH canasta.004B49E0                      (Initial CPU selection)
004B496B  MOV ECX,canasta.004B49FC                   ASCII "User Name"
004B497B  PUSH canasta.004B4A10                      ASCII "CodePageEx"
004B498A  MOV ECX,canasta.004B4A24                   ASCII "CodePage"
004B49C4  ASCII "Settings\DateFor"
004B49D4  ASCII "mat",0
004B49E0  ASCII "Registration Key"
004B49F0  ASCII 0
004B49FC  ASCII "User Name",0
004B4A10  ASCII "CodePageEx",0
004B4A24  ASCII "CodePage",0
004B4AC6  MOV EAX,canasta.004B4B24                   ASCII "LICENSE INVALID"
004B4B24  ASCII "LICENSE INVALID",0
004B4BB8  MOV EDX,canasta.004B4D70                   ASCII "http://www.canasta.net/keychecker/"
004B4BE0  MOV EAX,canasta.004B4D9C                   ASCII "000"
004B4C0A  MOV EAX,canasta.004B4DA8                   ASCII "yyyy/mm/dd"
004B4C2D  MOV EAX,canasta.004B4DA8                   ASCII "yyyy/mm/dd"
004B4C49  MOV EAX,canasta.004B4DBC                   ASCII "app=%s&name=%s&postfix=%s&key=%s&instal
004B4D70  ASCII "http://www.canas"
004B4D80  ASCII "ta.net/keychecke"
```

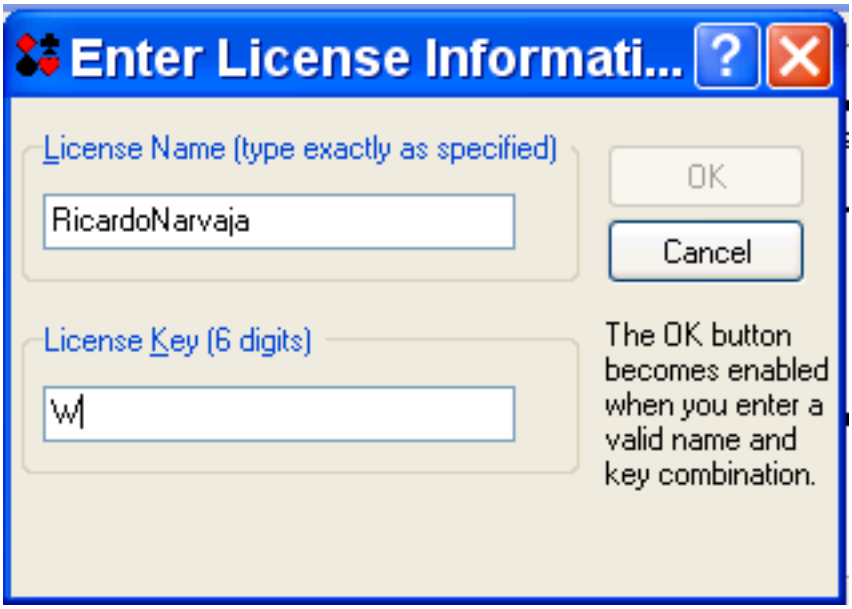
Среди этих строк нет никаких, которые подходили бы нам по смыслу, хех.



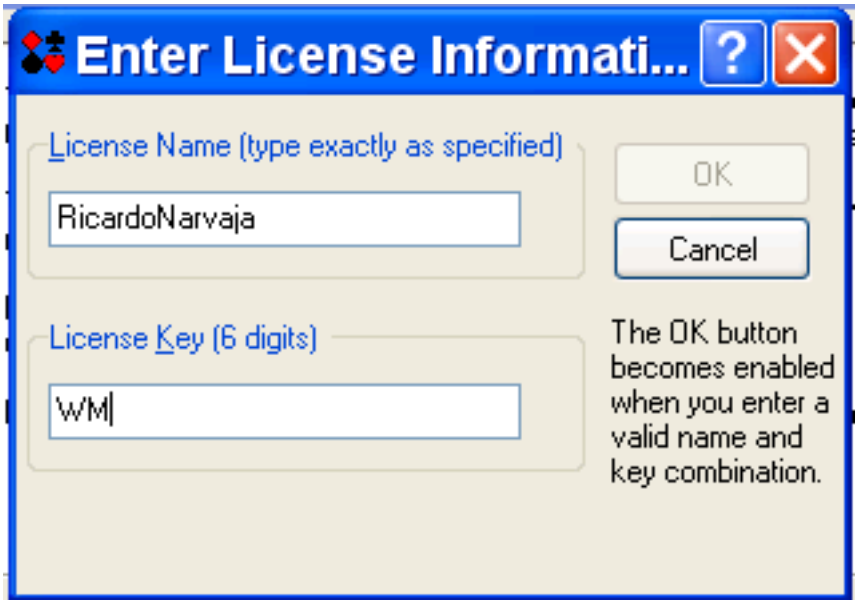
Запускаем программу в OllyDbg и оказываемся в окне регистрации. Попробуем применить какой-нибудь из методов взлома данного типа защит.



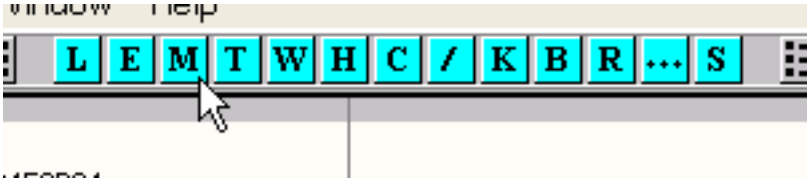
Вводим имя, предполагая, что оно может быть любым, поскольку узнать мы это можем только потом, затем вводим какой-нибудь редкий License Key, состоящий из 6 символов, например, WMYXSZ.



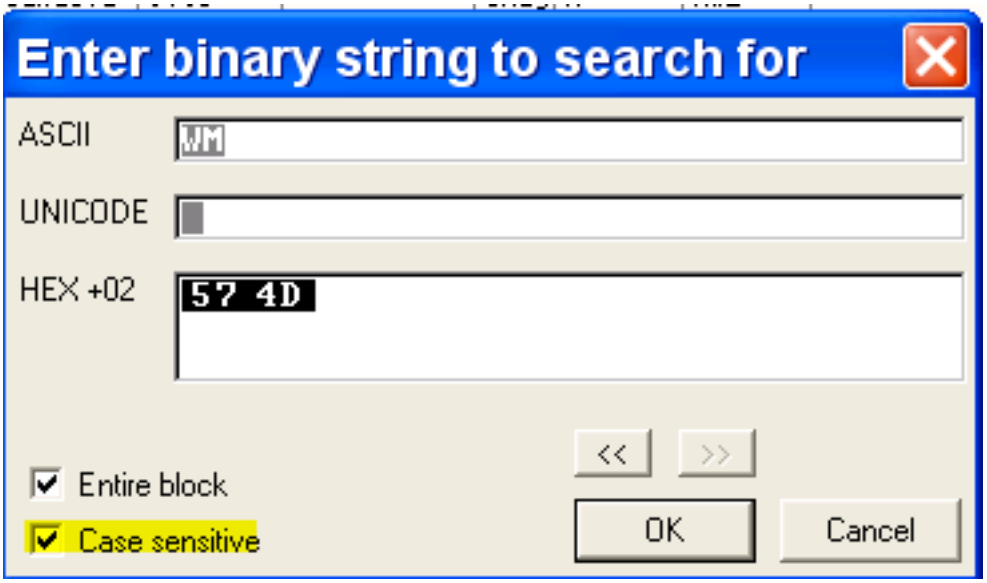
Теперь нажимаем M.



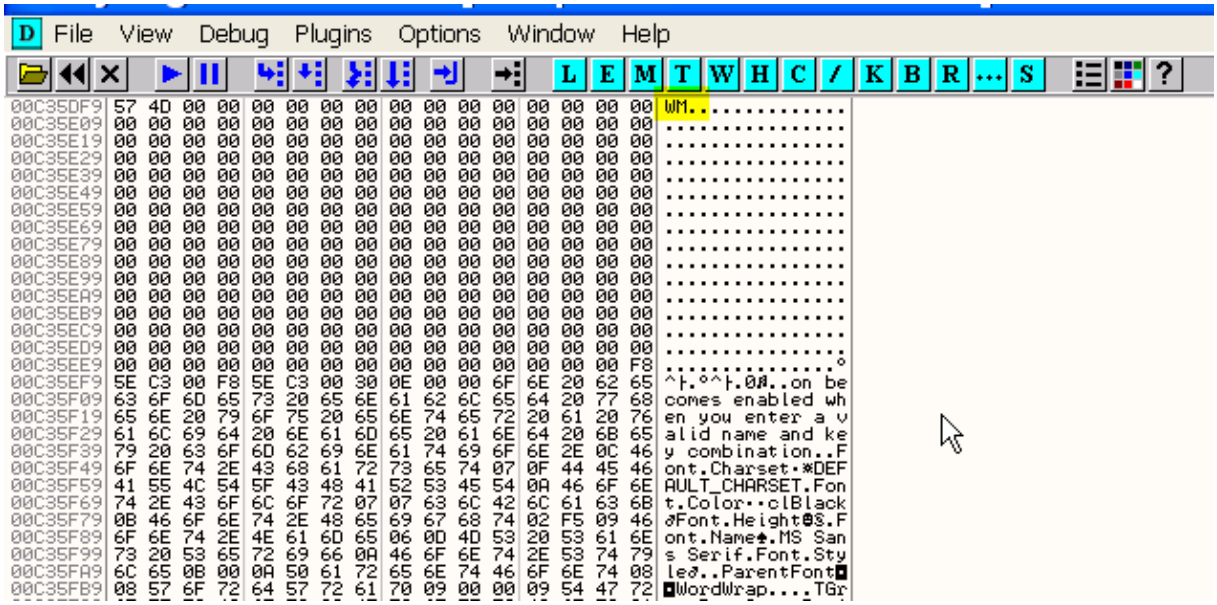
Поищем, нет ли в памяти строки «WM», соответствующей двум уже введённым буквам.



Нажимаем клавишу M, чтобы перейти в VIEW-MEMORY, и начать поиск вышеуказанной строки.

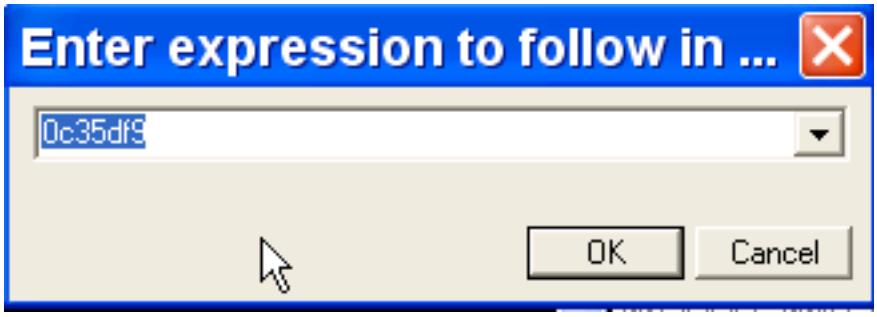


В данном случае отмечаем опцию CASE SENSITIVE, чтобы не выдавались аналоги WM в других регистрах.

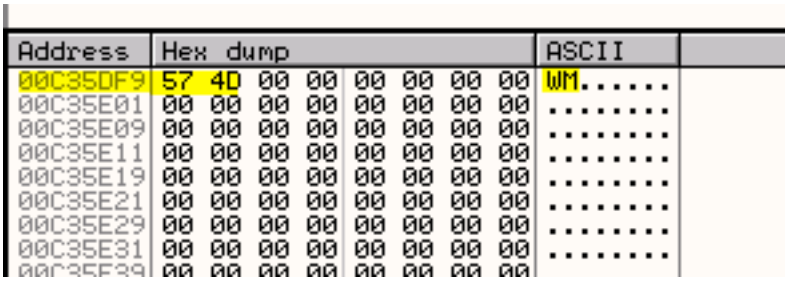


После двух или трёх раз, когда встретятся слова, начинающиеся с WM, и которые можно пропустить, нажав CTRL+L, и после того, как пройдём одну секцию и будем искать ниже в окне M, нам встретится WM, не входящая в состав других слов, и ещё ниже находится текст, говорящий о том, что на кнопку OK можно будет нажать после введения правильного серийного номера.

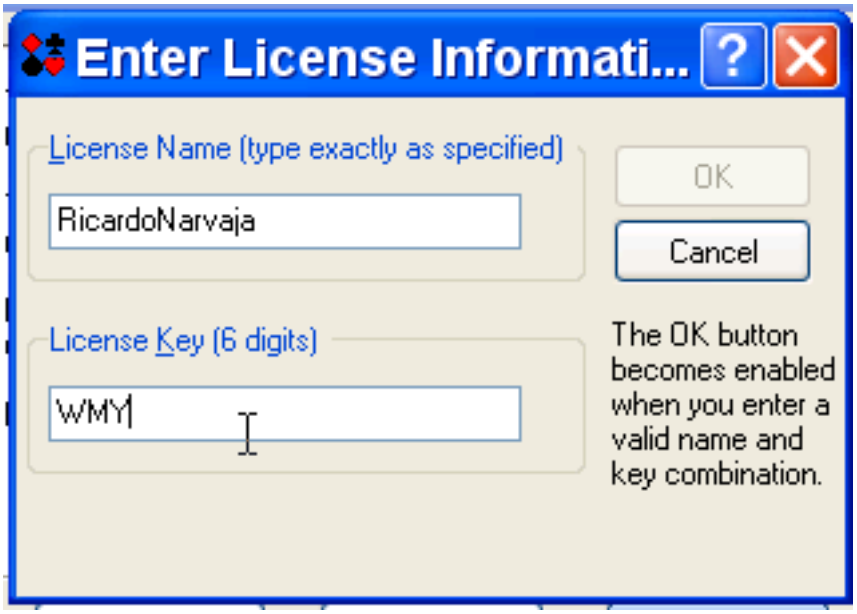
Поищем это место через DUMP.

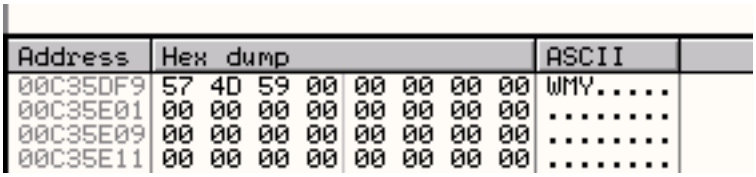


Вспомним, что адрес, который начинается с буквы (как в данном случае), должен начинаться с нуля, иначе OllyDbg его не признает.

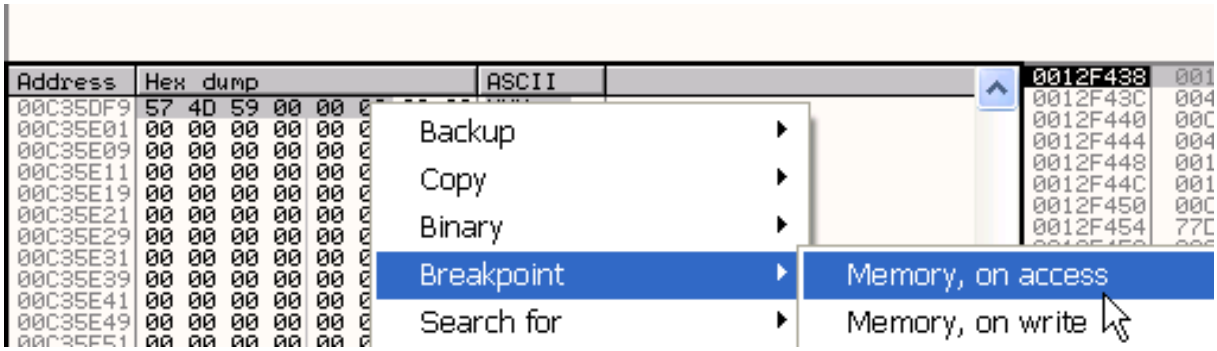


Чтобы быть уверенными, что это именно та область, где сохраняется серийный номер, введём следующую букву в поле серийного номера.

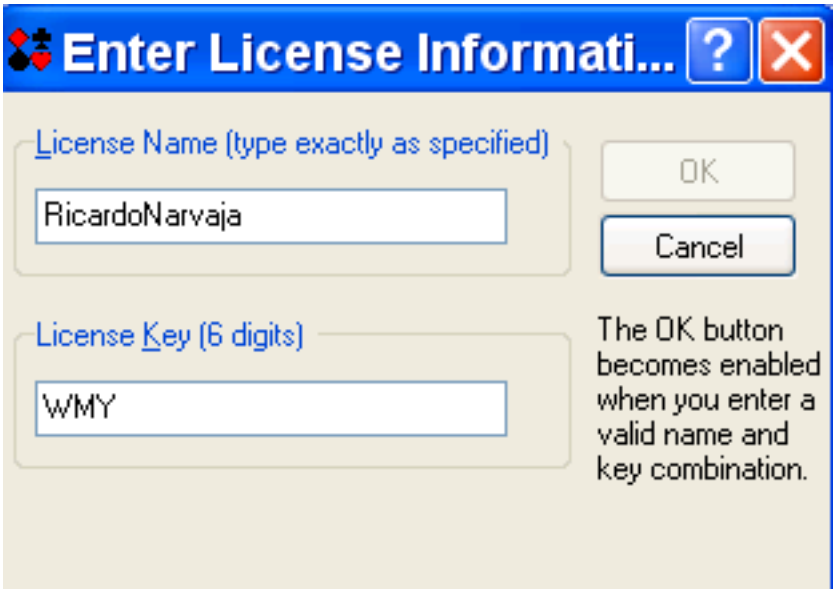




После этого смотрим в DUMP, символ был добавлен, а значит это и есть область, где сохраняется серийный номер, поэтому ставим сюда BREAKPOINT MEMORY ON ACCESS, охватывающий 6 цифр (так как мы знаем, что их будет ровно шесть).



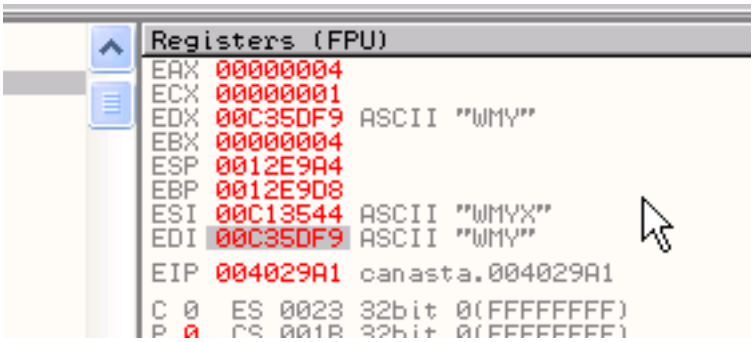
Теперь делаем RUN.



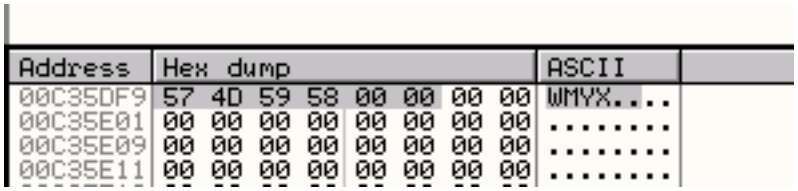
Появляется окно, в котором вводим следующий символ – “X”.

```
004029A0   .  FD            STD
004029A1   .  F3:A5         REP MOVS DWORD PTR ES:[EDI],DWORD PTR DS:[ESI]
004029A3   .  89C1          MOV ECX,EAX
004029A5   .  83E1 03       AND ECX,3
004029A8   .  83C6 03       ADD ESI,3
004029AB   .  83C7 03       ADD EDI,3
004029AE   .  F3:A4         REP MOVS BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
004029B0   .  FC            CLD
004029B1   >  5F            POP EDI
004029B2   .  5E            POP ESI
004029B3   .  C3            RETN
004029B4   $  53            PUSH EBX
004029B5   .  56            PUSH ESI
004029B6   .  57            PUSH EDI
004029B7   .  55            PUSH EBP
```

Останавливаемся на том месте, где происходит сохранение.

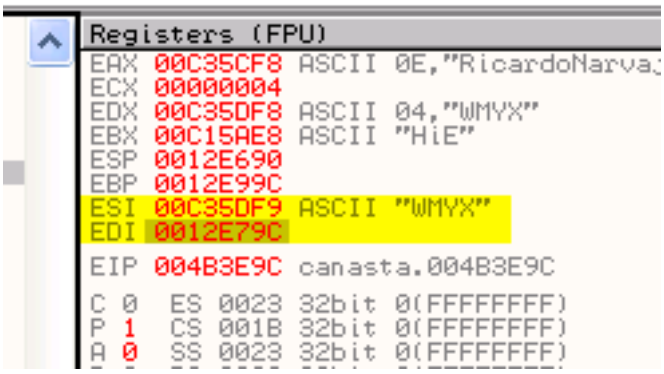


Здесь по нажатию на F8 происходит копирование ESI в EDI.

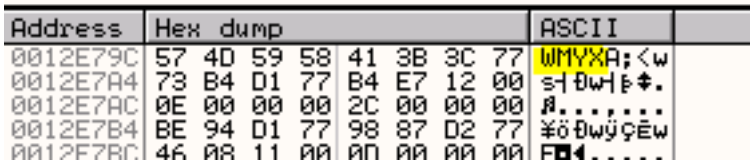



```
004B3E97   .  33C9          XOR ECX,ECX
004B3E99   .  8A0E          MOV CL,BYTE PTR DS:[ESI]
004B3E9B   .  41            INC ECX
004B3E9C   .  F3:A4         REP MOVS BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
004B3E9E   .  8BF0          MOV ESI,EAX
004B3EA0   .  8DBD FFFEFFFF LEA EDI,DWORD PTR SS:[EBP-101]
004B3EA6   .  33C9          XOR ECX,ECX
004B3EA8   .  8A0E          MOV CL,BYTE PTR DS:[ESI]
004B3EAA   .  41            INC ECX
```

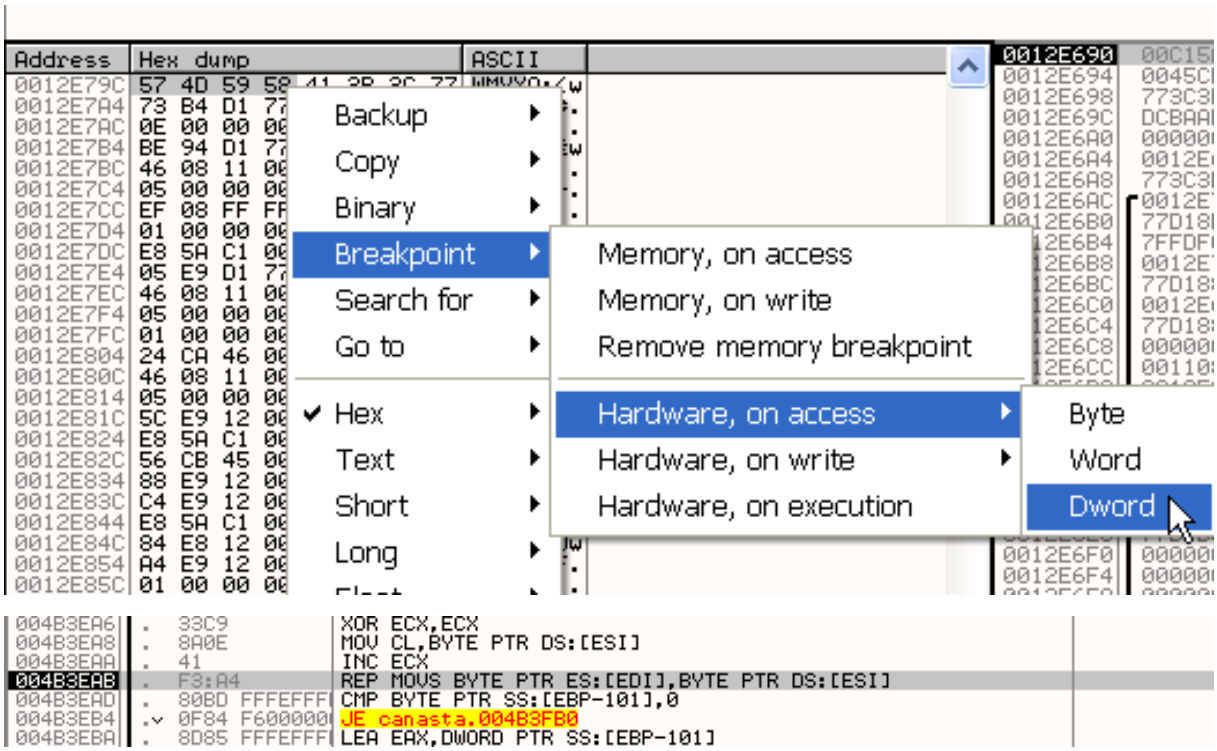
Здесь происходит копирование четырёх байтов, введённых нами ранее.



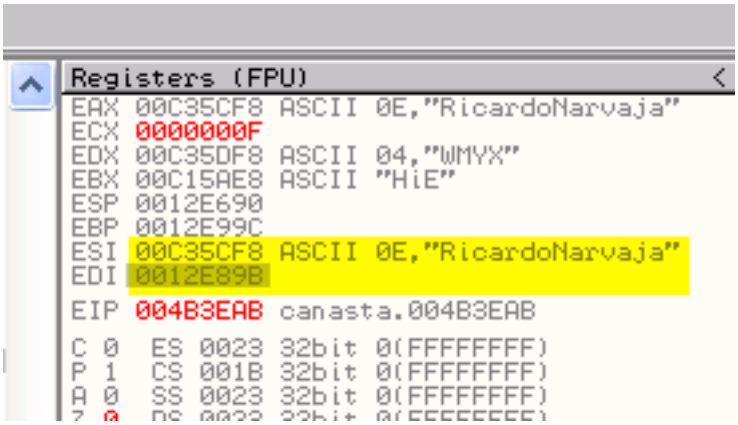
По нажатию на F8 они копируются в 12E79C.

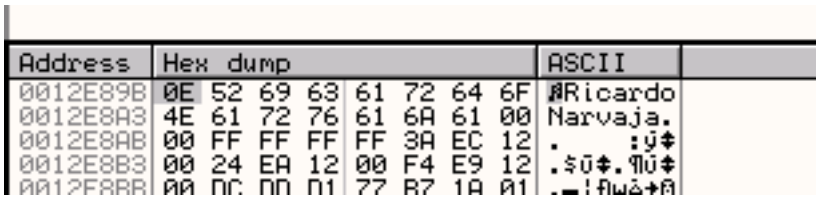


Так как мы уже используем BREAKPOINT MEMORY ON ACCESS, то можем установить HARDWARE BPX ON ACCESS.

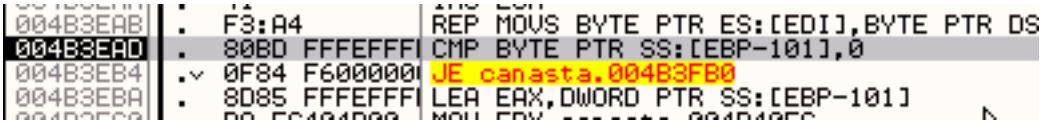


Если оттрассируем немного, увидим моё имя также перемещается в стек.

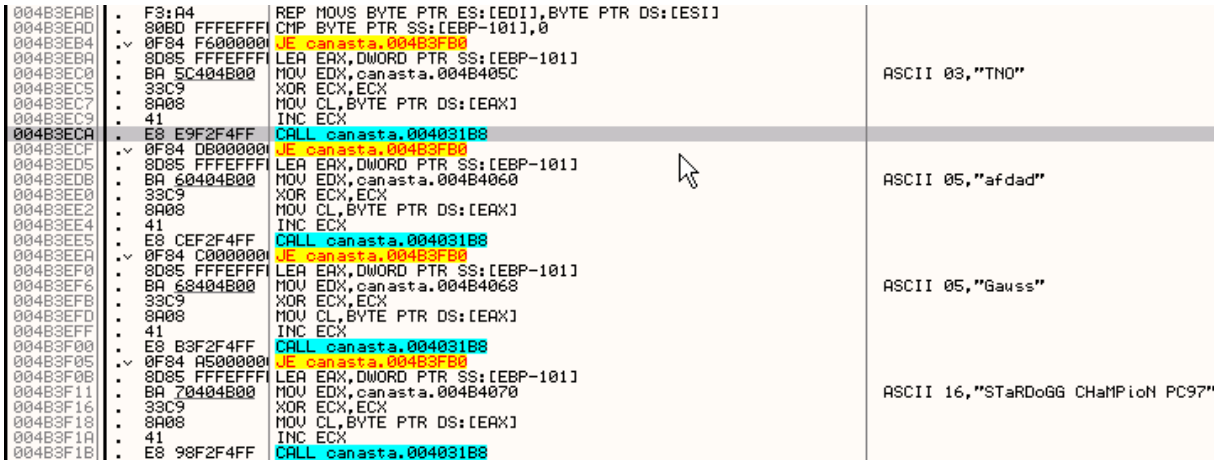




Точно перед именем находится 0E, которое является его длиной, и она сравнивается с нулём.



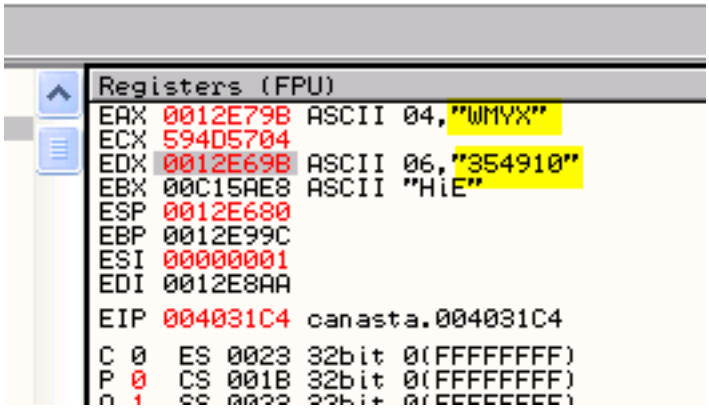
Так как не равно нулю, выполнение программы продолжается.



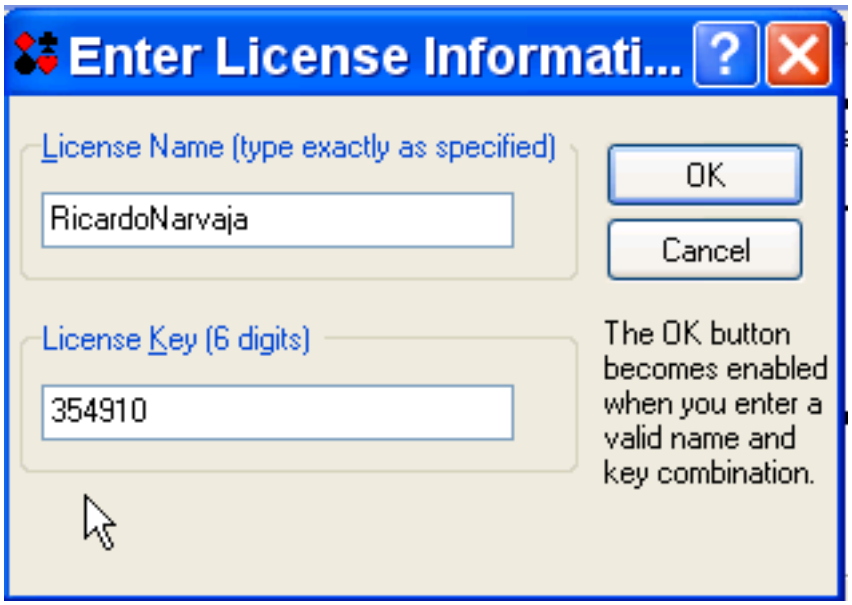
Затем происходит сравнение имени с теми, кто находится в чёрном списке, то есть оно не должно быть TNO, afdad и т.п.

Снова делаем RUN.

Останавливаемся на сравнении с серийным номером.



И можем посмотреть, действительно ли это правильный серийный номер. Убираем все точки останова.



После его введения становится доступной кнопка “OK”, после её нажатия мы регистрируемся, хе-хе. Другим способом, на объяснении которого я не буду задерживаться, но который не слишком сложен, является использование WM_KEYUP каждый раз, когда нажимаем клавишу, чтобы останавливать OllyDbg и отслеживать, что происходит с введенным нами символом, но это довольно скучно, и способ, который мы применили, гораздо более простой.

Ок, давайте отдохнём немного. В следующей части мы изучим немного теории и рассмотрим несколько примеров по новой теме.

Введение в крэкинг с нуля, используя OllyDbg - Глава 18

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-01-27

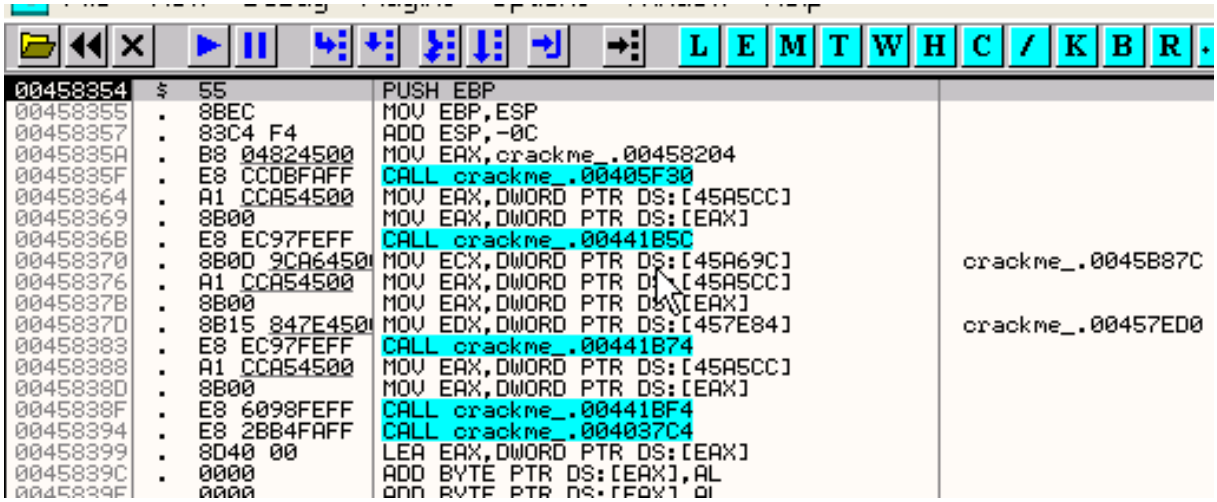
Приложение к статье (находится в архиве `wasm_files.zip`: `files/0472/ollydbg18.rar`)

Ок, в прошлой главе я упомянул, что собираюсь начать другую тему, связанную с обнаружением OllyDbg программами, запускающимися в ней (что, как правило, называется антиотладкой), как этого избежать и в каких случаях какие плагины можно использовать.

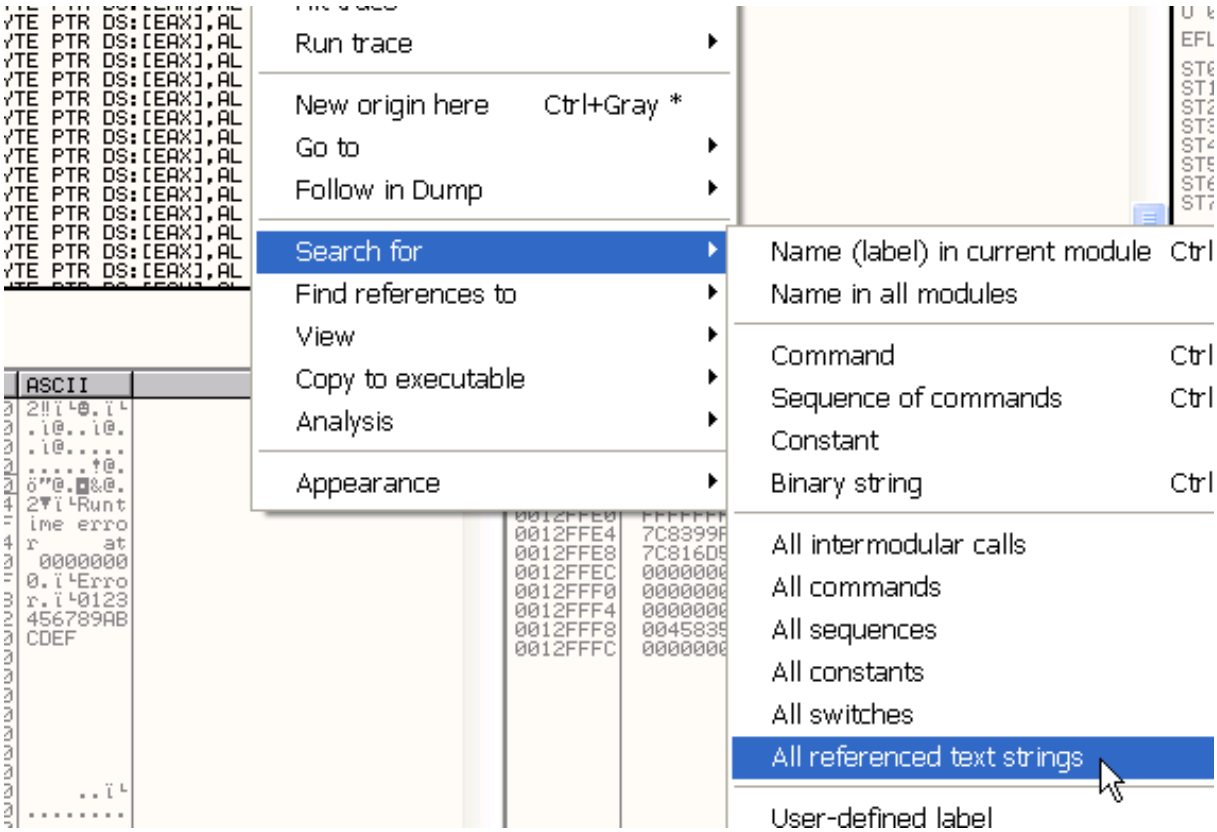
Но эту тему мы начнём в 19-ой главе, так как среди тех, кто изучает этот курс, нашлись люди, написавшие мне на почту и попросившие разобрать пример крэкми без кнопок как в прошлой части, и рассмотреть применение другой техники, использующей WM_KEYUP.

Хорошо, используем для этого крэкми, предоставленный мне Stzwei'ем (спасибо ему), который называется _4stz.exe. Его мы и рассмотрим в данном туториале.

Откроем крэкми в OllyDbg.

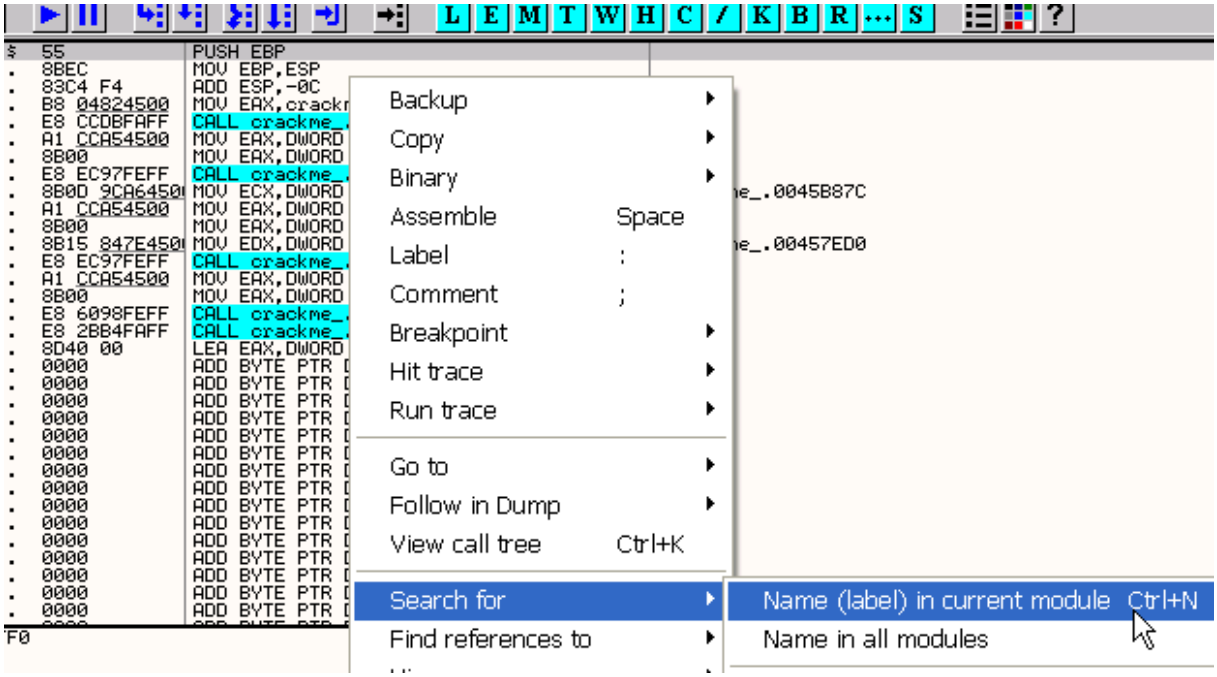


Находимся на точке входа. Посмотрим, какие здесь есть незапакованные строки.



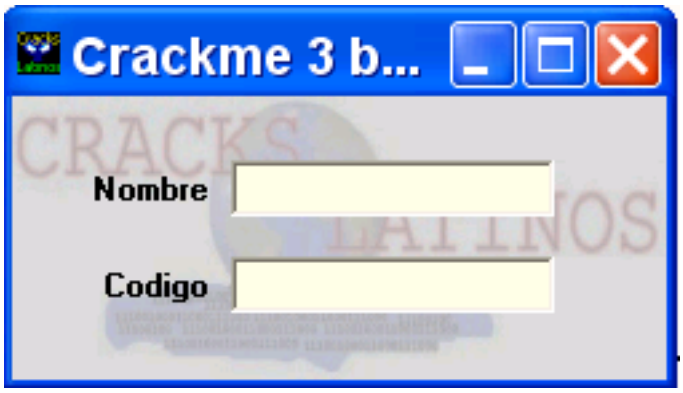
Отображается список строк, используемых программой.

Здесь нет ничего интересного, посмотрим API-функции.

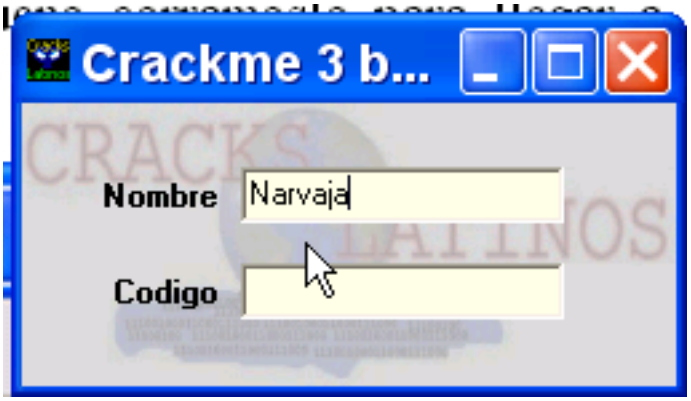

L E M T W H C / K B R ... S ?
Backup
Copy
Binary
Assemble Space
Label :
Comment ;
Breakpoint
Hit trace
Run trace
Go to
Follow in Dump
View call tree Ctrl+K
Search for
Find references to
Name (label) in current module Ctrl+N
Name in all modules

Address	Section	Type	Name	Comment
0045C600	.idata	Import	user32.DestroyCursor	
0045C5FC	.idata	Import	user32.DestroyIcon	
0045C5F8	.idata	Import	user32.DestroyMenu	
0045C5F4	.idata	Import	user32.DestroyWindow	
0045C5F0	.idata	Import	user32.DispatchMessageA	
0045C5EC	.idata	Import	user32.DrawEdge	
0045C5E8	.idata	Import	user32.DrawFrameControl	
0045C5E4	.idata	Import	user32.DrawIcon	
0045C5E0	.idata	Import	user32.DrawIconEx	
0045C5DC	.idata	Import	user32.DrawMenuBar	
0045C5D8	.idata	Import	user32.DrawTextA	
0045C5D4	.idata	Import	user32.EnableMenuItem	
0045C5D0	.idata	Import	user32.EnableScrollBar	
0045C5CC	.idata	Import	user32.EnableWindow	
0045C5C8	.idata	Import	user32.EndPaint	
0045C0FC	.idata	Import	kernel32.EnterCriticalSection	
0045C2C0	.idata	Import	kernel32.EnterCriticalSection	
0045C2BC	.idata	Import	kernel32.EnumCalendarInfoA	
0045C5C4	.idata	Import	user32.EnumThreadWindows	
0045C5C0	.idata	Import	user32.EnumWindows	
0045C5BC	.idata	Import	user32.EqualRect	
0045C3A4	.idata	Import	gdi32.ExcludeClipRect	
0045C164	.idata	Import	kernel32.ExitProcess	
0045C5B8	.idata	Import	user32.FillRect	
0045C160	.idata	Import	kernel32.FindClose	
0045C15C	.idata	Import	kernel32.FindFirstFileA	
0045C2B8	.idata	Import	kernel32.FindResourceA	
0045C5B4	.idata	Import	user32.FindWindowA	
0045C2B4	.idata	Import	kernel32.FormatMessageA	
0045C5B0	.idata	Import	user32.FrameRect	
0045C158	.idata	Import	kernel32.FreeLibrary	
0045C2B0	.idata	Import	kernel32.FreeLibrary	
0045C2AC	.idata	Import	kernel32.FreeResource	
0045C3A0	.idata	Import	gdi32.GdiFlush	
0045C5AC	.idata	Import	user32.GetActiveWindow	
0045C39C	.idata	Import	gdi32.GetBitmapBits	
0045C398	.idata	Import	gdi32.GetBrushOrgEx	
0045C5A8	.idata	Import	user32.GetCapture	
0045C5A4	.idata	Import	user32.GetClassInfoA	
0045C5A0	.idata	Import	user32.GetClientRect	
0045C59C	.idata	Import	user32.GetClipboardData	
0045C394	.idata	Import	gdi32.GetClipBox	
0045C154	.idata	Import	kernel32.GetCommandLineA	
0045C2A8	.idata	Import	kernel32.GetCPInfo	
0045C390	.idata	Import	gdi32.GetCurrentPositionEx	
0045C2A4	.idata	Import	kernel32.GetCurrentProcessId	
0045C0F0	.idata	Import	kernel32.GetCurrentThreadId	
0045C2A0	.idata	Import	kernel32.GetCurrentThreadId	
0045C598	.idata	Import	user32.GetCursor	
0045C594	.idata	Import	user32.GetCursorPos	
0045C590	.idata	Import	user32.GetDC	
0045C58C	.idata	Import	user32.GetDCEx	
0045C38C	.idata	Import	gdi32.GetDCOrgEx	
0045C588	.idata	Import	user32.GetDesktopWindow	
0045C380	.idata	Import	gdi32.GetDeviceCaps	
0045C388	.idata	Import	gdi32.GetDIBColorTable	
0045C384	.idata	Import	gdi32.GetDIBits	
0045C29C	.idata	Import	kernel32.GetDiskFreeSpaceA	
0045C37C	.idata	Import	gdi32.GetEnhMetaFileBits	
0045C378	.idata	Import	gdi32.GetEnhMetaFileHeader	
0045C374	.idata	Import	gdi32.GetEnhMetaFilePaletteEntries	
0045C188	.idata	Import	kernel32.GetFileSize	

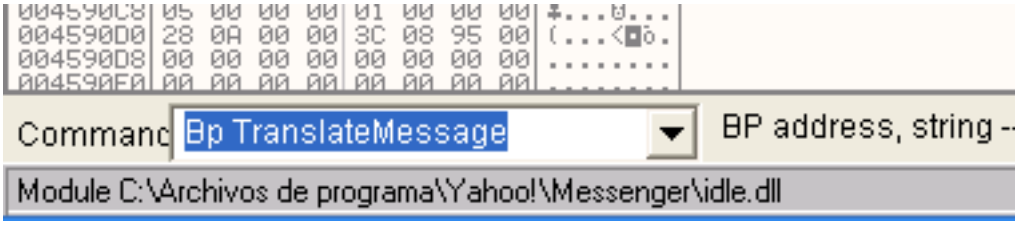
Уфф, так много.. Ок, запустим программу и окажемся в окошке, где вводится серийный номер.



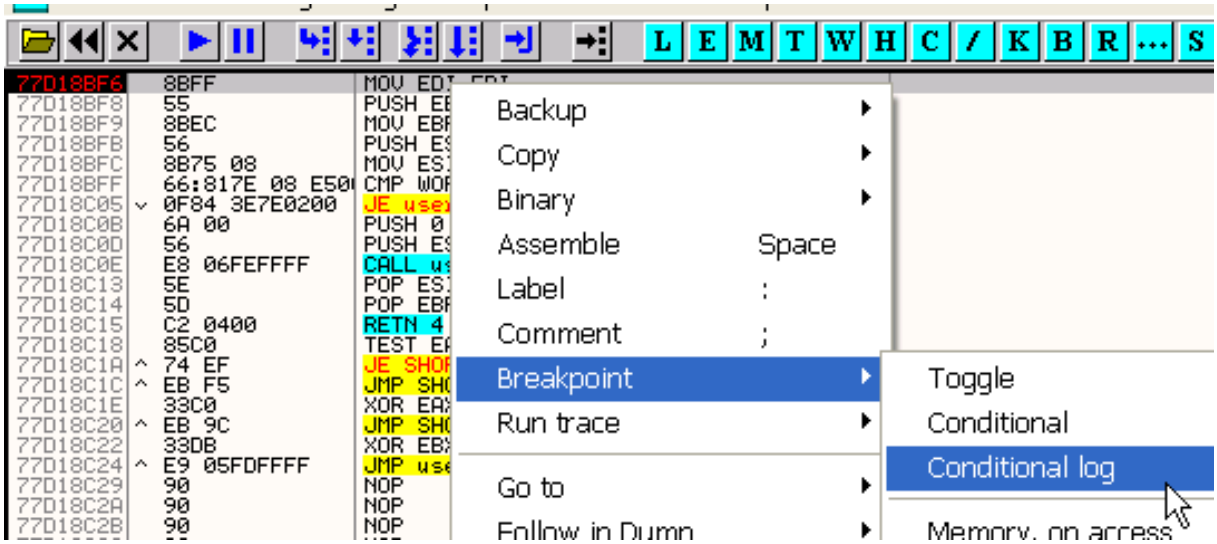
Видим, что кнопки регистрации нет. Введём полное имя.



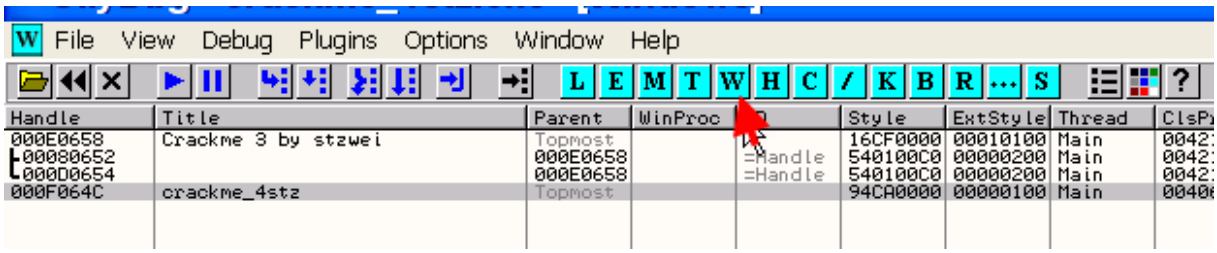
И установим BPX CONDITIONAL LOG на TranslateMessage – сначала обычный BPX.



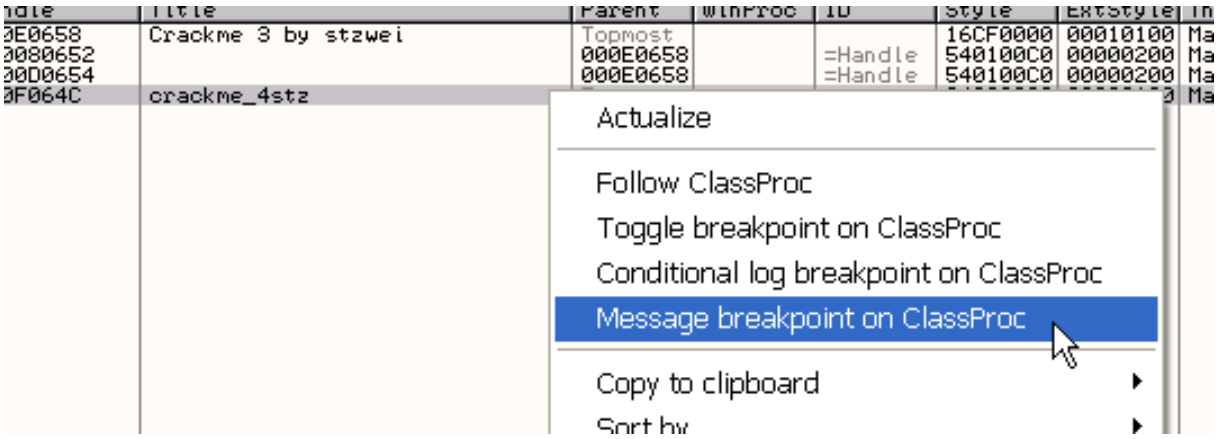
А когда вернёмся в крэкми, отредактируем установленный BP.



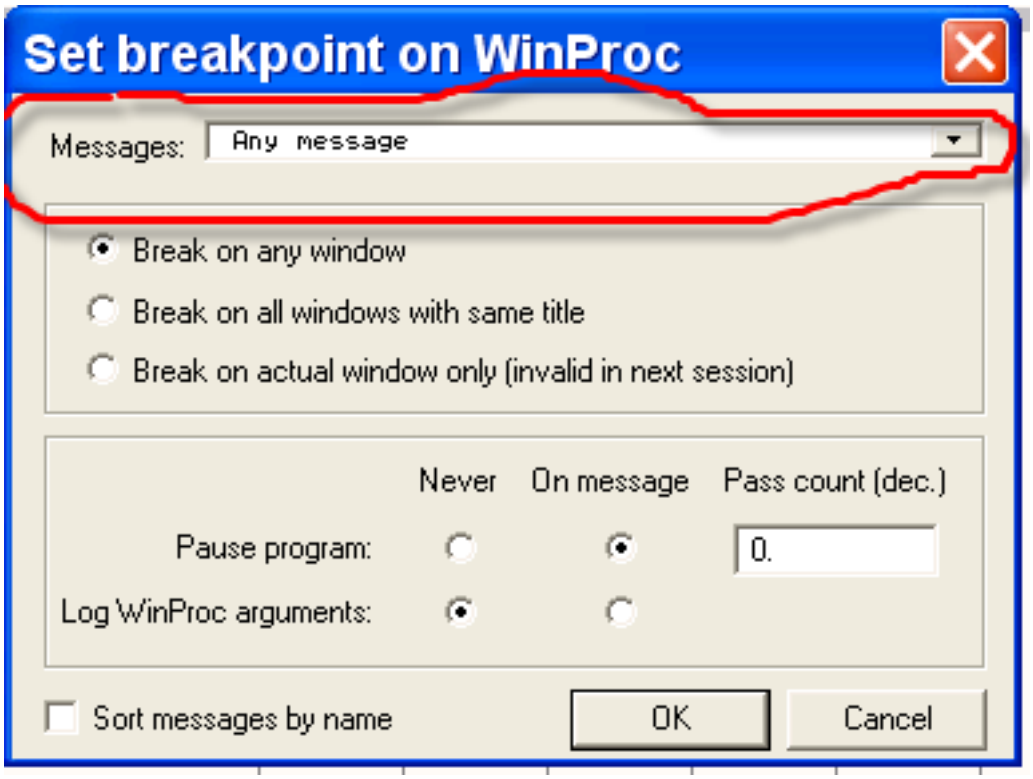
Если кто-то не помнит значение WM-сообщения, которое собираемся использовать, то нужно перейти в окно W, которое показывает, какие окна используются программой.



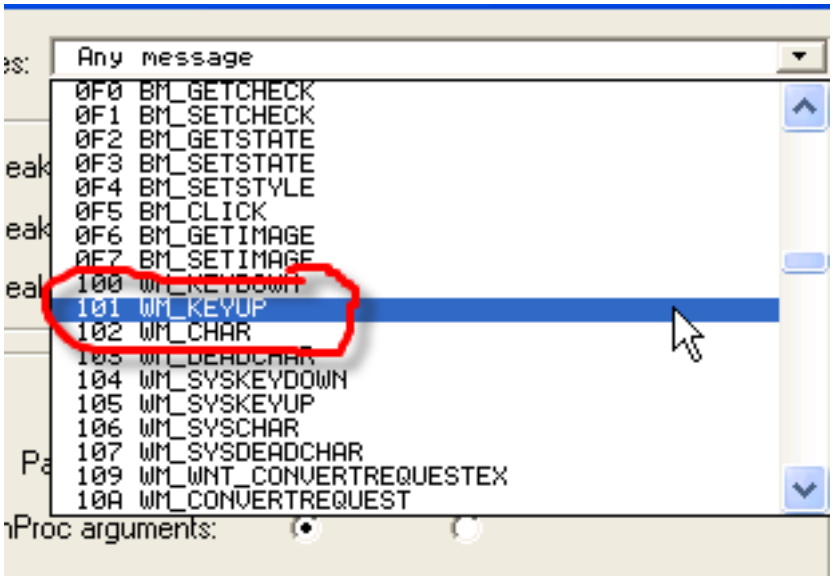
И отображается список окон, осталось только узнать числовое значение, поэтому кликаем правой кнопкой мыши.



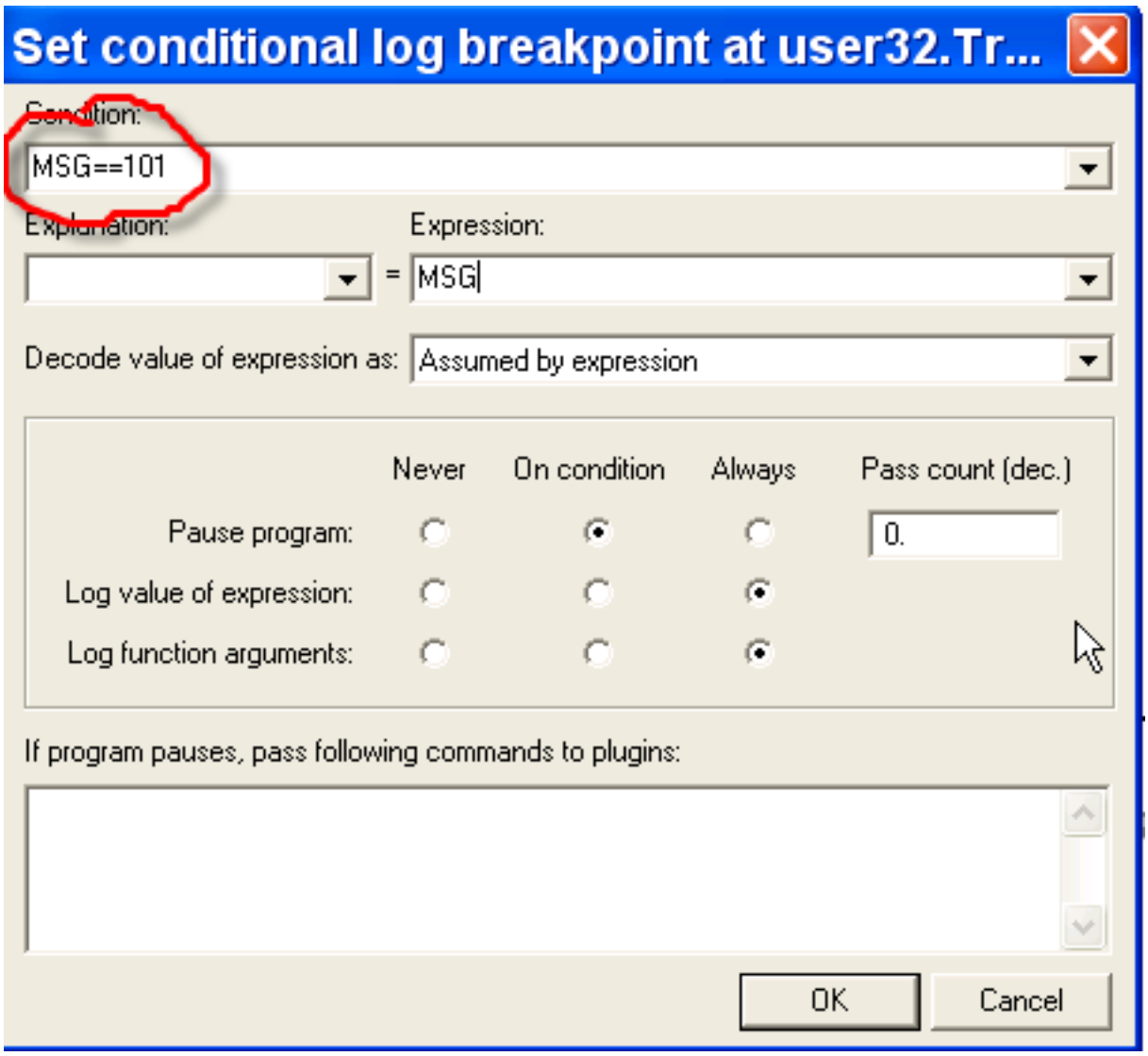
И мы попадаем в окно точек останова на сообщения с раскрываемым меню с WM-значениями.



Поищем в списке WM_KEYUP.



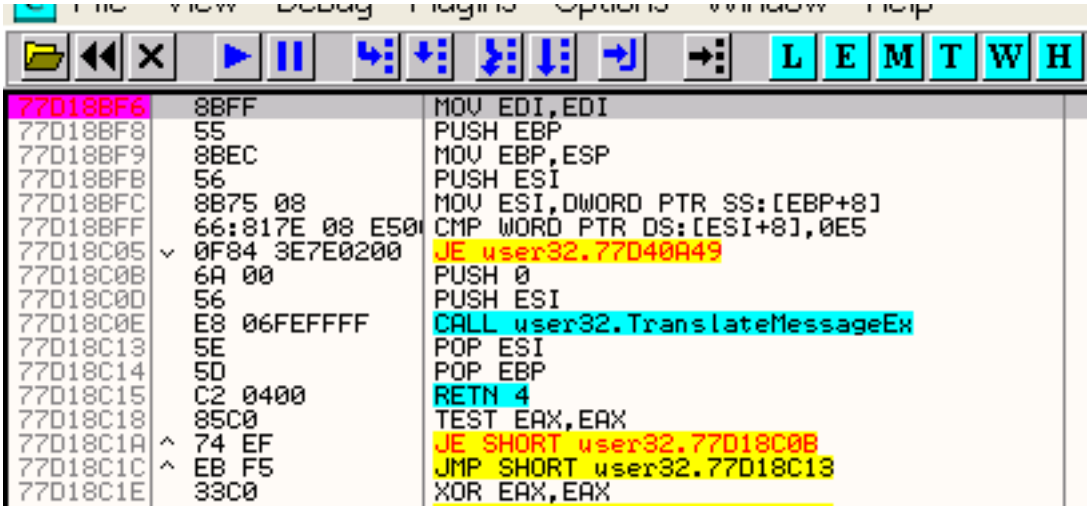
Видим, что это 101, поэтому уберём всё, что мы делали только для того, чтобы узнать это значение, и вернёмся к BREAKPOINT CONDITIONAL LOG в TranslateMessage.



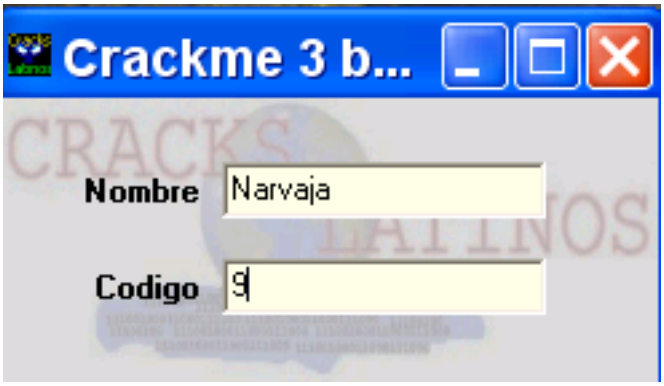
И устанавливаем здесь MSG==1 (обратите внимание на двойной знак равенства), также, если бы не нашли значение, могли бы написать MSG==WM_KEYUP и это бы сработало, но мне больше

нравятся числовые значения. Здесь каждый волен поступать так, как ему это нравится.

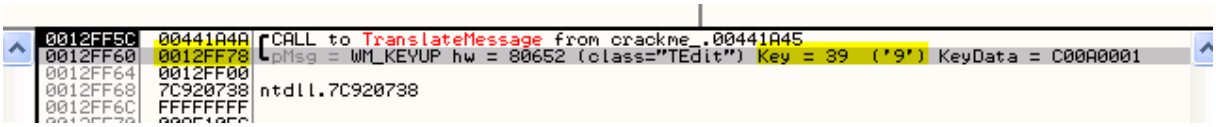
Хорошо, точка останова трансформировалась в условную (розовый цвет).



Делаем RUN и вводим первый символ неправильного серийного номера.

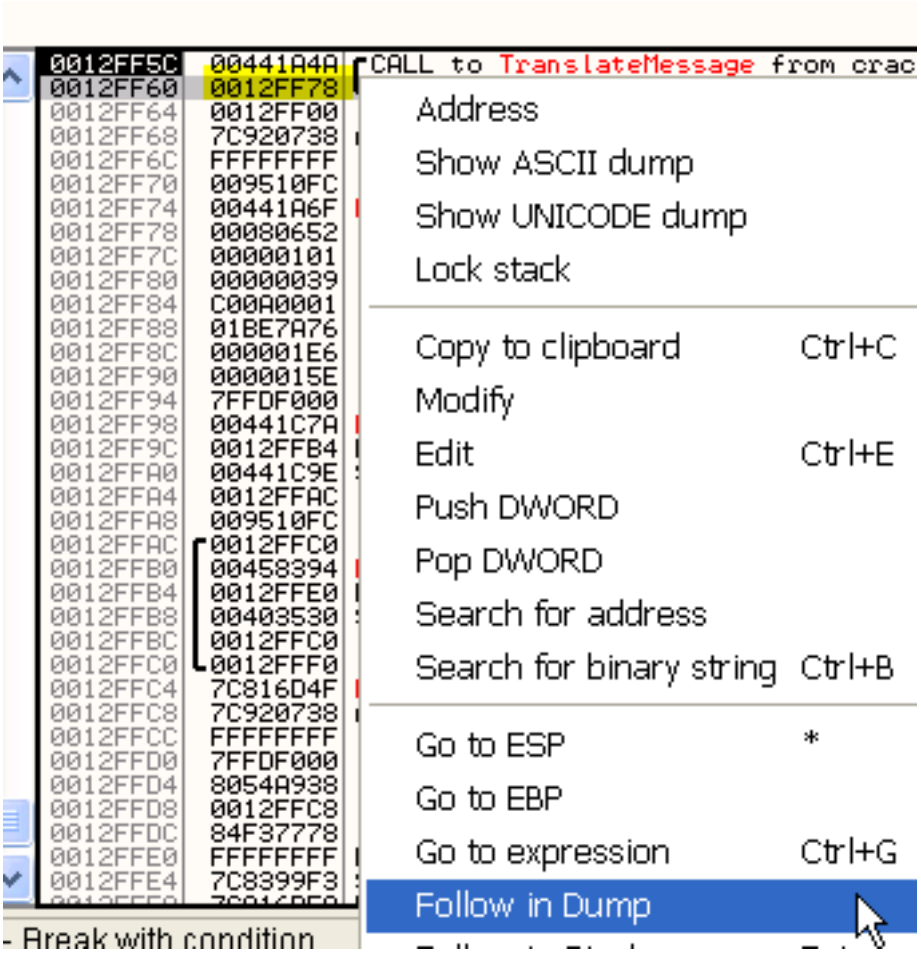


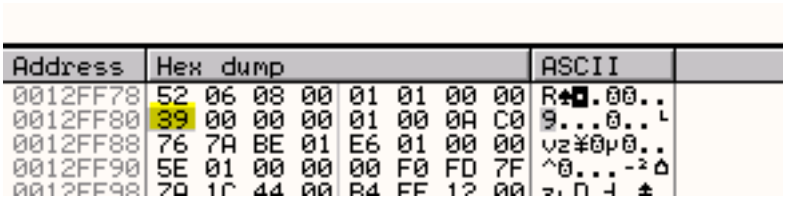
И тут срабатывает условная точка останова.



Смотрим параметры API-функции, по адресу 12ff78 находится структура, в которой сохранены значения нажатой клавиши, в данном случае это 39, что соответствует цифре 9.

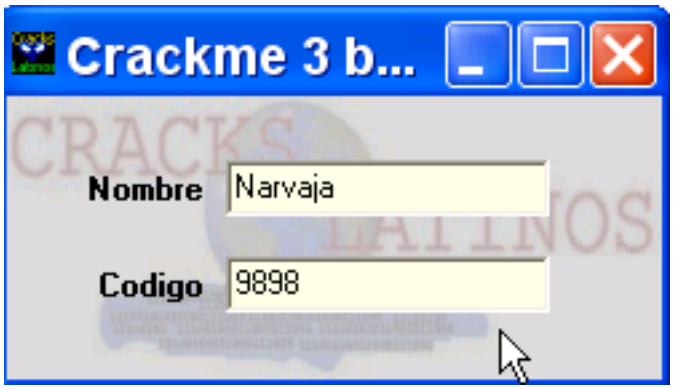
Смотрим через DUMP указанную область памяти – нажимаем правую кнопку мыши на указанном адресе и выбираем FOLLOW IN DUMP.





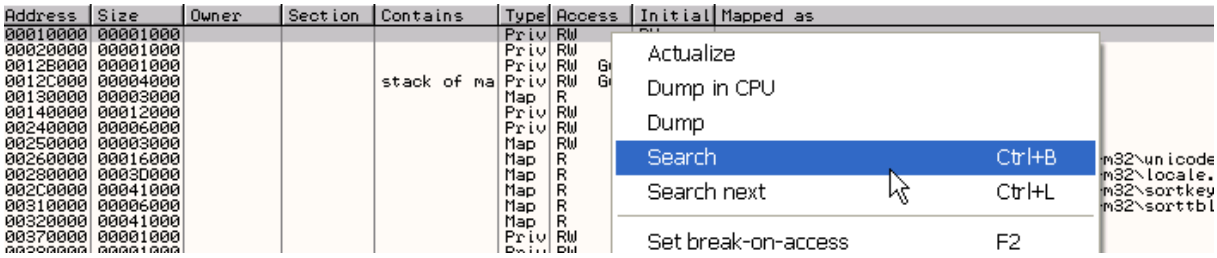
Видим, что мы удалось узнать, куда помещается байт. Проблема в том, что это программа на Delphi (что мы рассмотрим в другой раз), и если мы установим на него BPM ON ACCESS, программа будет ходить сто раз туда-сюда, прежде чем дойдёт до сравнения, поэтому метод, который мы рассматривали в прошлый раз, работает здесь гораздо лучше, так что его мы и используем. В любом случае, вы уже знаете, как остановить программу по нажатию клавиши и как установить местонахождение байта. Разумеется, в крэкми, не написанном на Delphi, гораздо проще использовать BPM ON ACCESS.

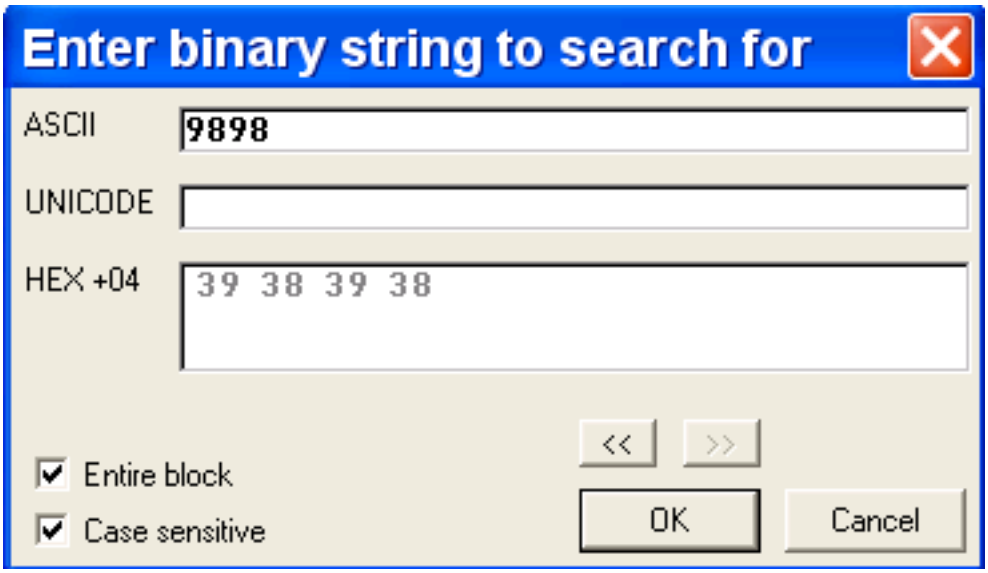
И запускаем.



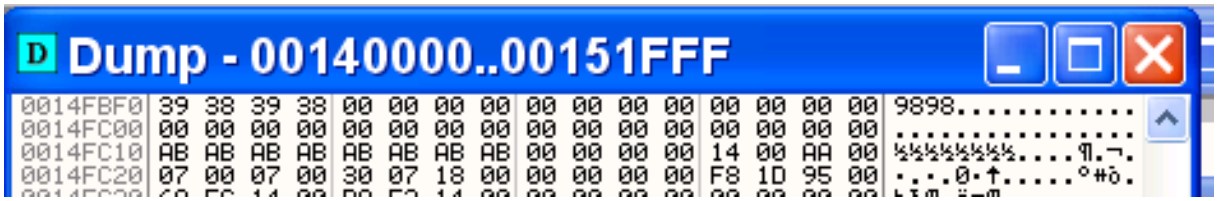
Вводим 9898, идём в окно M и ищем по всей памяти.

Нажимаем правую кнопку мыши и выбираем SEARCH.



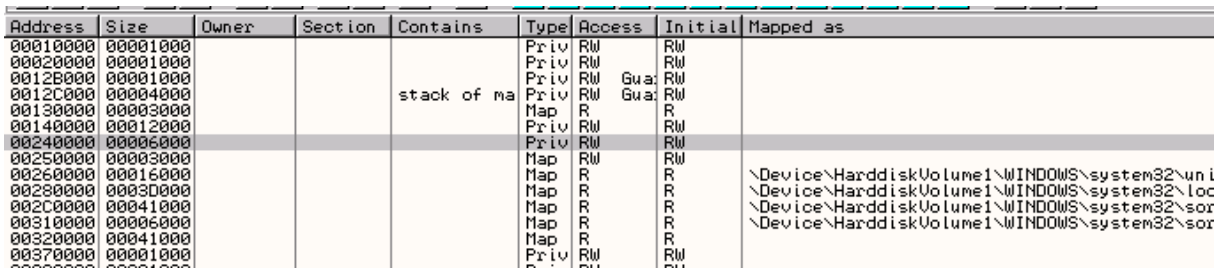


И ищем строку 9898.

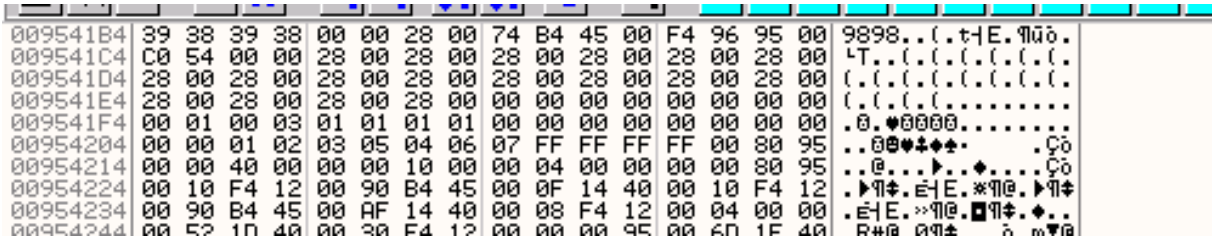


Первое вхождение обнаруживается в стеке. Нажимаем CTRL+L несколько раз, чтобы увидеть, сколько раз оно в нём встречается.

Как только нам внизу отобразится жёлтым цветом слово ITEM NOT FOUND, выходим и возвращаемся в окно M.

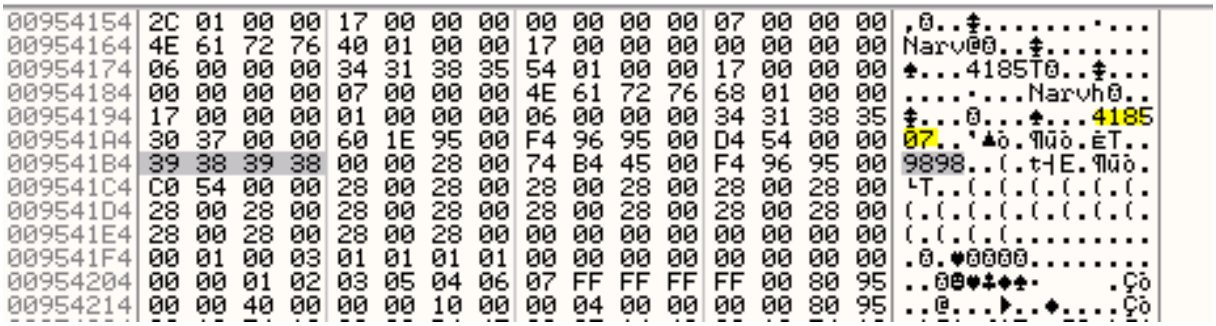


Оказываемся там, откуда искали, так что идём дальше вниз с помощью CTRL + L.

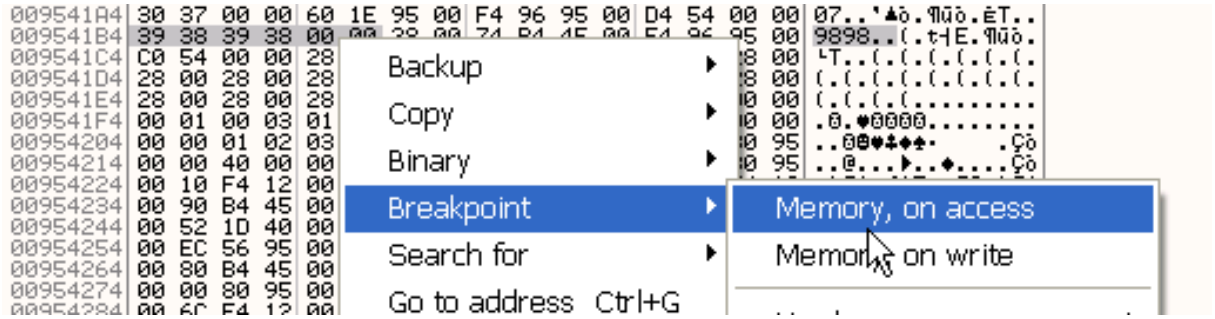


Первый раз в этой секции, нажимаем CTRL + L, чтобы посмотреть, найдётся ли здесь что-либо.

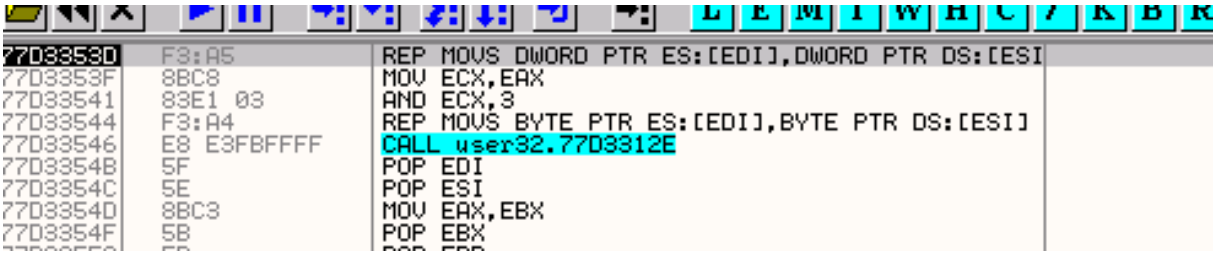
Нет, ничего, нет, если посмотрим область, находится вхождение чего-то подозрительного.



Видим возможный серийный номер 418507, установим на это место BMP on ACCESS, чтобы посмотреть, не с ним ли происходит сравнение.

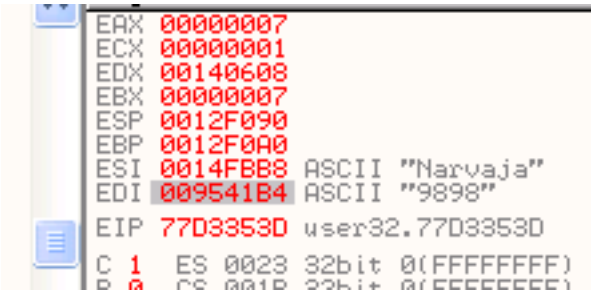


Делаем RUN.



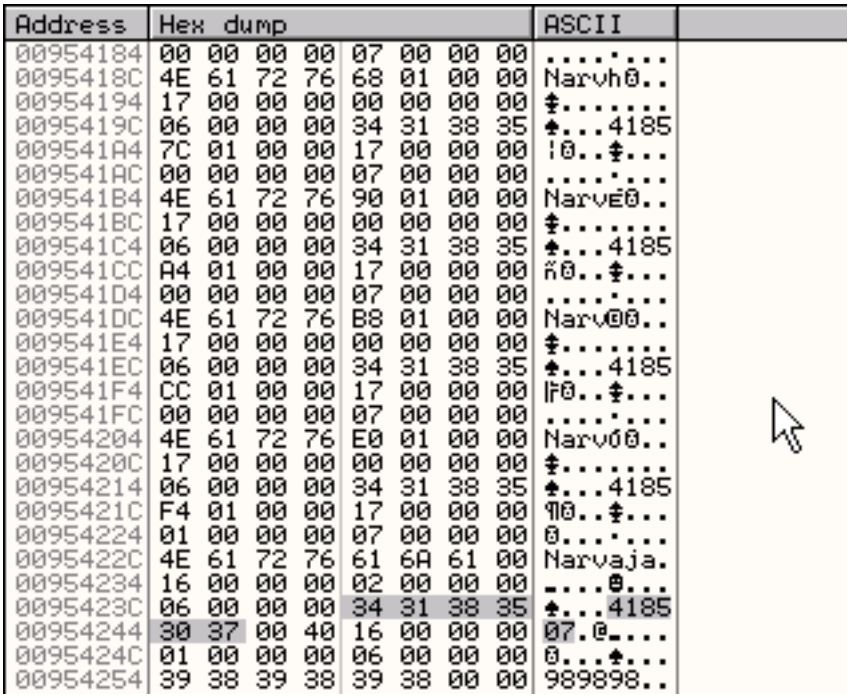
```
77D3353D   F3:A5          REP MOVS DWORD PTR ES:[EDI],DWORD PTR DS:[ESI
77D3353F   8BC8           MOV ECX,EAX
77D33541   83E1 03        AND ECX,3
77D33544   F3:A4          REP MOVS BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
77D33546   E8 E3FBFFFF    CALL user32.77D3312E
77D3354B   5F             POP EDI
77D3354C   5E             POP ESI
77D3354D   8BC3           MOV EAX,EBX
77D3354F   5B             POP EBX
```

Останавливаемся здесь и видим, что копируется моё имя.



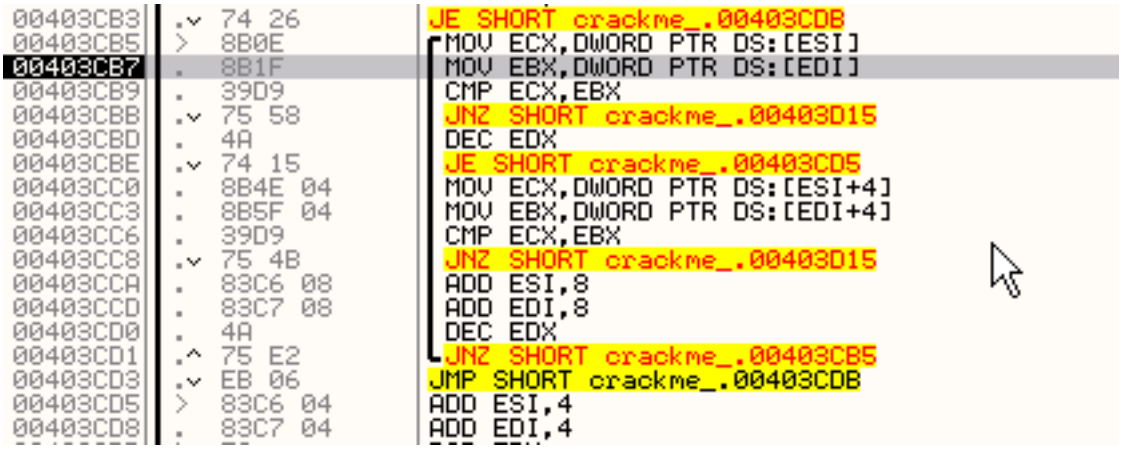
```
EAX 00000007
ECX 00000001
EDX 00140608
EBX 00000007
ESP 0012F090
EBP 0012F0A0
ESI 0014FBB8 ASCII "Narvaja"
EDI 009541B4 ASCII "9898"

EIP 77D3353D user32.77D3353D

C 1  ES 0023 32bit 0(FFFFFFFF)
```

Видим, что каждый раз, когда печатаем, по новой копируется мой неправильный серийный номер, и с каждым следующим символом по новой копируется правильный серийный номер. Как только введём 6 символов, то установим на них BPM (помните, в предполагаемом серийном номере как раз 6 символов? – прим. пер.).



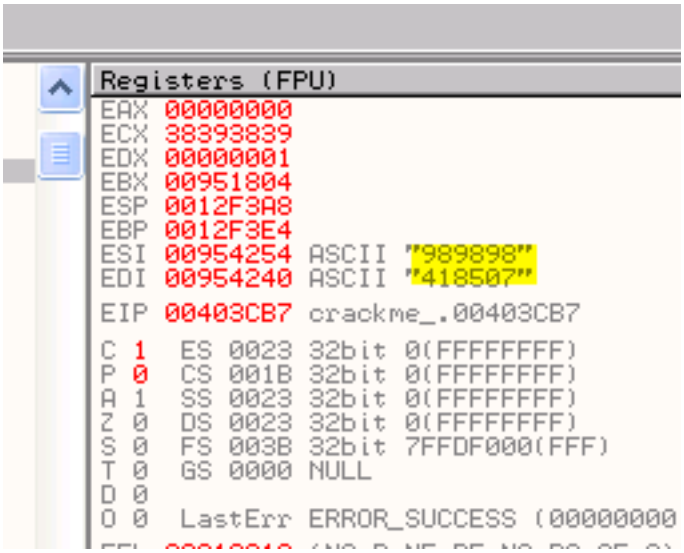
Address	Hex dump	ASCII
00954184	00 00 00 00 07 00 00 00	....•...
0095418C	4E 61 72 76 68 01 00 00	Narvh☺..
00954194	17 00 00 00 00 00 00 00	↨.......
0095419C	06 00 00 00 34 31 38 35	♠...4185
009541A4	7C 01 00 00 17 00 00 00	|☺..↨...
009541AC	00 00 00 00 07 00 00 00	....•...
009541B4	4E 61 72 76 90 01 00 00	NarvÉ☺..
009541BC	17 00 00 00 00 00 00 00	↨.......
009541C4	06 00 00 00 34 31 38 35	♠...4185
009541CC	A4 01 00 00 17 00 00 00	ñ☺..↨...
009541D4	00 00 00 00 07 00 00 00	....•...
009541DC	4E 61 72 76 B8 01 00 00	Narv☺☺..
009541E4	17 00 00 00 00 00 00 00	↨.......
009541EC	06 00 00 00 34 31 38 35	♠...4185
009541F4	CC 01 00 00 17 00 00 00	|F☺..↨...
009541FC	00 00 00 00 07 00 00 00	....•...
00954204	4E 61 72 76 E0 01 00 00	Narvó☺..
0095420C	17 00 00 00 00 00 00 00	↨.......
00954214	06 00 00 00 34 31 38 35	♠...4185
0095421C	F4 01 00 00 17 00 00 00	¶☺..↨...
00954224	01 00 00 00 07 00 00 00	☺...•...
0095422C	4E 61 72 76 61 6A 61 00	Narvaja.
00954234	16 00 00 00 02 00 00 00	▬...☻...
0095423C	06 00 00 00 34 31 38 35	♠...4185
00954244	30 37 00 40 16 00 00 00	07.@▬...
0095424C	01 00 00 00 06 00 00 00	☺...♠...
00954254	39 38 39 38 39 38 00 00	989898..

Останавливаемся прямо на сравнении.

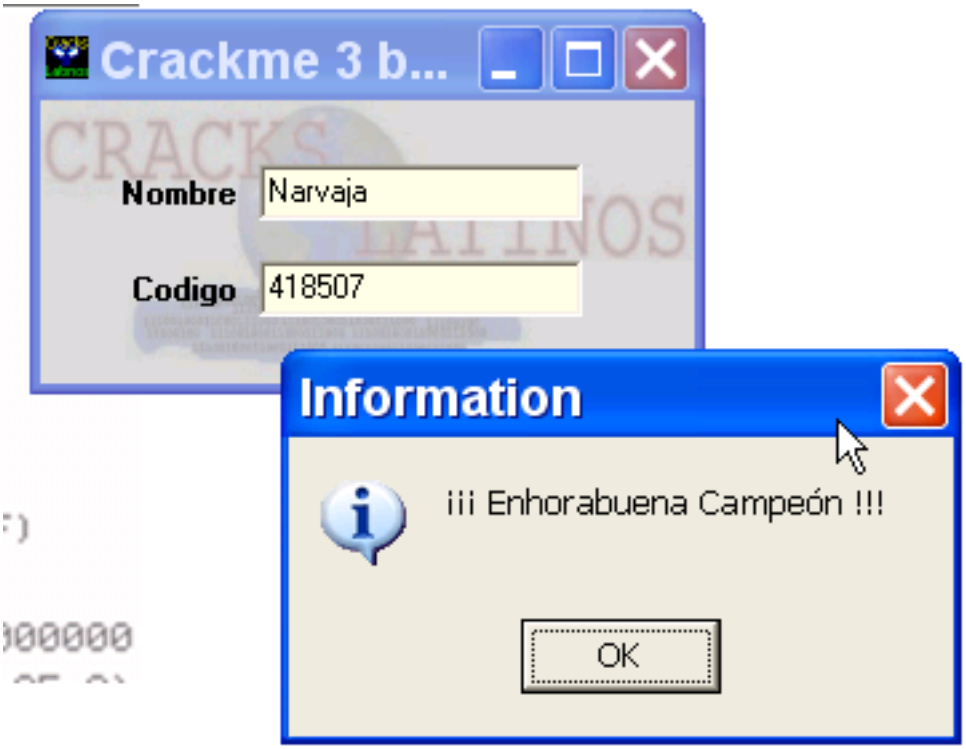


```
00403CB3  .v 74 26        JE SHORT crackme_.00403CDB
00403CB5  > 8B0E          MOV ECX,DWORD PTR DS:[ESI]
00403CB7  . 8B1F          MOV EBX,DWORD PTR DS:[EDI]
00403CB9  . 39D9          CMP ECX,EBX
00403CBB  .v 75 58        JNZ SHORT crackme_.00403D15
00403CBD  . 4A            DEC EDX
00403CBE  .v 74 15        JE SHORT crackme_.00403CD5
00403CC0  . 8B4E 04       MOV ECX,DWORD PTR DS:[ESI+4]
00403CC3  . 8B5F 04       MOV EBX,DWORD PTR DS:[EDI+4]
00403CC6  . 39D9          CMP ECX,EBX
00403CC8  .v 75 4B        JNZ SHORT crackme_.00403D15
00403CCA  . 83C6 08       ADD ESI,8
00403CCD  . 83C7 08       ADD EDI,8
00403CD0  . 4A            DEC EDX
00403CD1  .^ 75 E2        JNZ SHORT crackme_.00403CB5
00403CD3  .v EB 06        JMP SHORT crackme_.00403CDB
00403CD5  > 83C6 04       ADD ESI,4
00403CD8  . 83C7 04       ADD EDI,4
```

Тут видим, что ESI и EDI указывают на неправильный и правильный серийные номера.



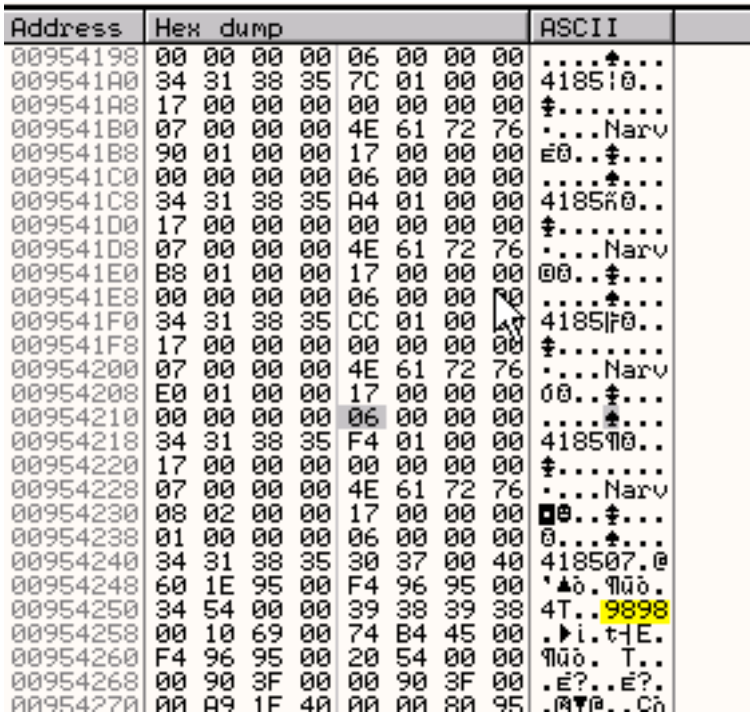
Они сравниваются по четыре байта за оборот цикла. Проверим, является ли найденный серийник действительно верным.



(Сообщение гласит: «Поздравляю, победитель!» - прим. пер.)

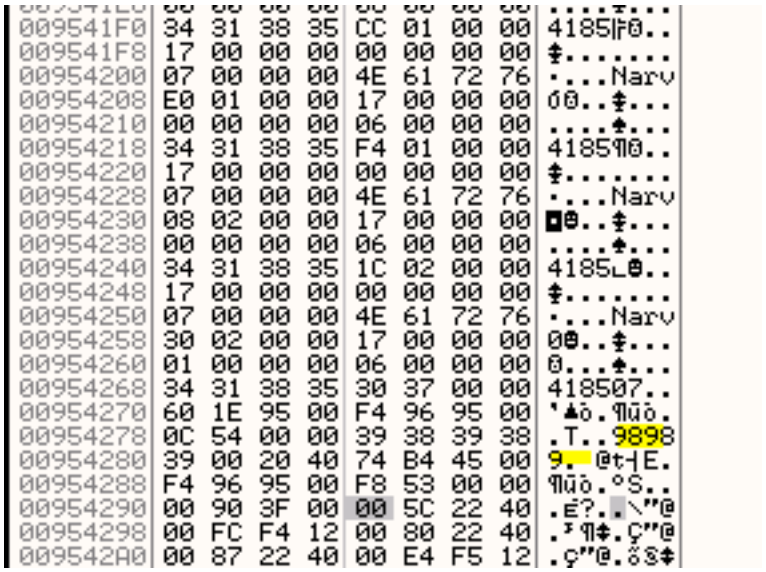
Видим, что не смотря на то, что это сложный крэкми, можно поймать точный момент, когда происходит сравнение, для чего нужно отслеживать местоположение неправильного серийного номера и ставить на него BPM.

Есть и другая возможность. Видим, что в памяти появляются 4 байта неправильного серийного номера.



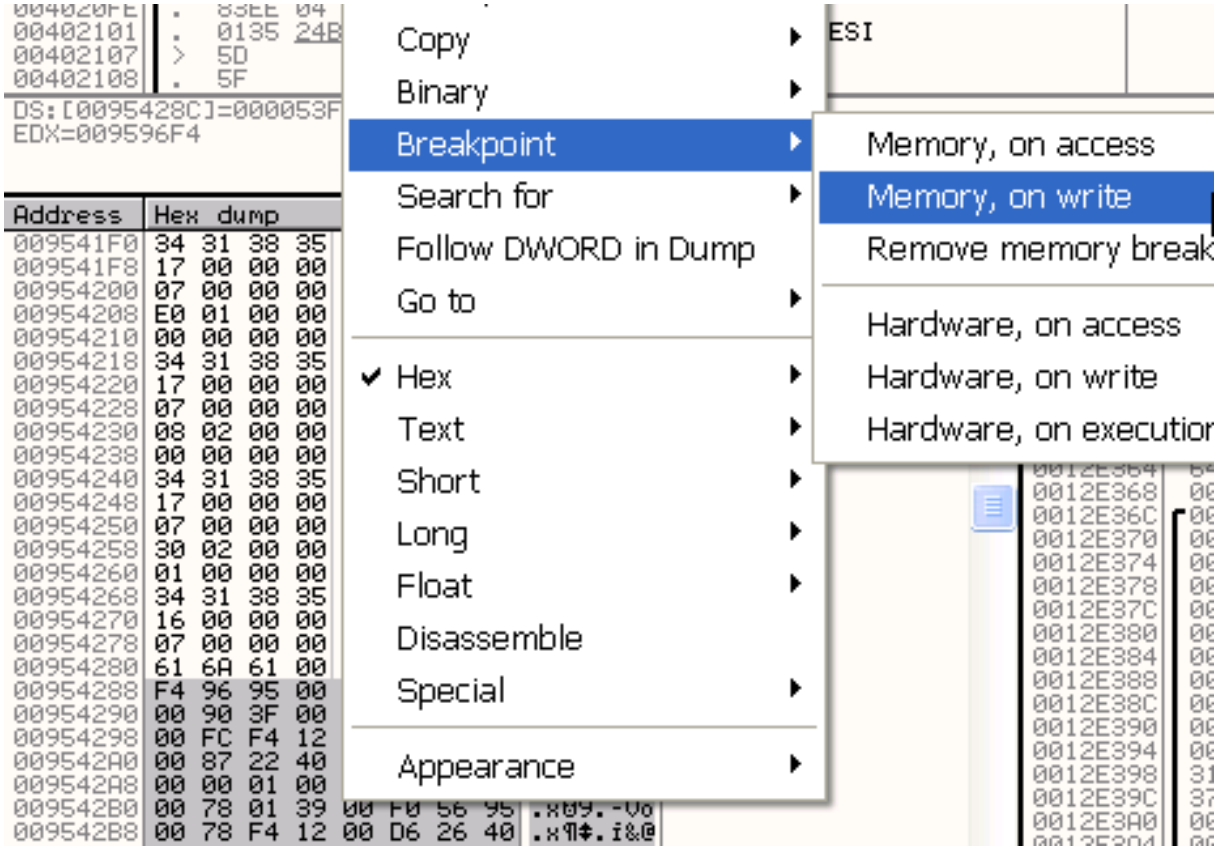
Address	Hex dump	ASCII
00954198	00 00 00 00 06 00 00 00	....♠...
009541A0	34 31 38 35 7C 01 00 00	4185|☺..
009541A8	17 00 00 00 00 00 00 00	↨.......
009541B0	07 00 00 00 4E 61 72 76	•...Narv
009541B8	90 01 00 00 17 00 00 00	É☺..↨...
009541C0	00 00 00 00 06 00 00 00	....♠...
009541C8	34 31 38 35 A4 01 00 00	4185ñ☺..
009541D0	17 00 00 00 00 00 00 00	↨.......
009541D8	07 00 00 00 4E 61 72 76	•...Narv
009541E0	B8 01 00 00 17 00 00 00	╕☺..↨...
009541E8	00 00 00 00 06 00 00 00	....♠...
009541F0	34 31 38 35 CC 01 00 00	4185╠☺..
009541F8	17 00 00 00 00 00 00 00	↨.......
00954200	07 00 00 00 4E 61 72 76	•...Narv
00954208	E0 01 00 00 17 00 00 00	ó☺..↨...
00954210	00 00 00 00 06 00 00 00	....♠...
00954218	34 31 38 35 F4 01 00 00	4185¶☺..
00954220	17 00 00 00 00 00 00 00	↨.......
00954228	07 00 00 00 4E 61 72 76	•...Narv
00954230	08 02 00 00 17 00 00 00	◘☻..↨...
00954238	01 00 00 00 06 00 00 00	☺...♠...
00954240	34 31 38 35 30 37 00 40	418507.@
00954248	60 1E 95 00 F4 96 95 00	`▲ò.¶ûò.
00954250	34 54 00 00 39 38 39 38	4T..9898
00954258	00 10 69 00 74 B4 45 00	.►i.t┤E.
00954260	F4 96 95 00 20 54 00 00	¶ûò. T..
00954268	00 90 3F 00 00 90 3F 00	.É?..É?.
00954270	00 A9 1E 40 00 00 80 95	.⌐▼@..Cò

Он же, но уже из 5-ти первых байтов, чуть ниже.

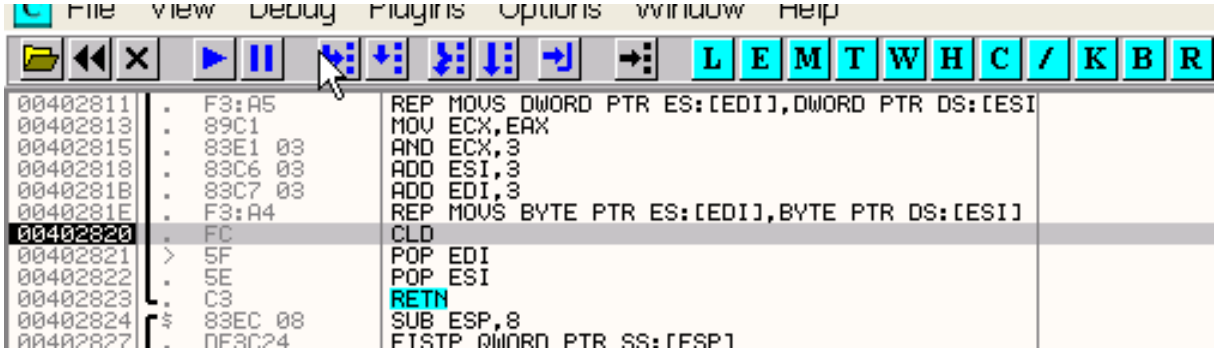


009541F0	34 31 38 35 CC 01 00 00	4185╠☺..
009541F8	17 00 00 00 00 00 00 00	↨.......
00954200	07 00 00 00 4E 61 72 76	•...Narv
00954208	E0 01 00 00 17 00 00 00	ó☺..↨...
00954210	00 00 00 00 06 00 00 00	....♠...
00954218	34 31 38 35 F4 01 00 00	4185¶☺..
00954220	17 00 00 00 00 00 00 00	↨.......
00954228	07 00 00 00 4E 61 72 76	•...Narv
00954230	08 02 00 00 17 00 00 00	◘☻..↨...
00954238	00 00 00 00 06 00 00 00	....♠...
00954240	34 31 38 35 1C 02 00 00	4185∟☻..
00954248	17 00 00 00 00 00 00 00	↨.......
00954250	07 00 00 00 4E 61 72 76	•...Narv
00954258	30 02 00 00 17 00 00 00	0☻..↨...
00954260	01 00 00 00 06 00 00 00	☺...♠...
00954268	34 31 38 35 30 37 00 00	418507..
00954270	60 1E 95 00 F4 96 95 00	`▲ò.¶ûò.
00954278	0C 54 00 00 39 38 39 38	.T..9898
00954280	39 00 20 40 74 B4 45 00	9. @t┤E.
00954288	F4 96 95 00 F8 53 00 00	¶ûò.°S..
00954290	00 90 3F 00 00 5C 22 40	.É?..\"@
00954298	00 FC F4 12 00 80 22 40	.ⁿ¶↕.Ç"@
009542A0	00 87 22 40 00 E4 F5 12	.ç"@.õS↕

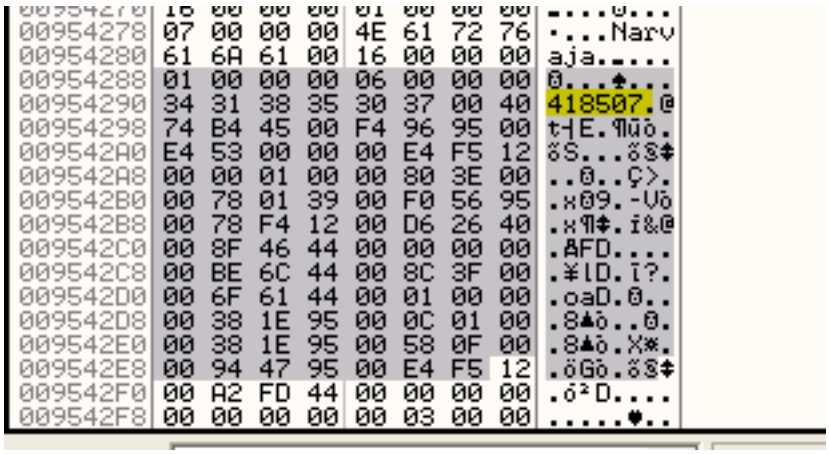
Соответственно, можем установить BPM ON WRITE на область, находящуюся ниже, где будет записан следующий символ.



Отметим место, куда будет помещён следующий символ, а затем делаем RUN.



Здесь записываю наиболее возможный серийный номер.



Делаем RUN.

```
77D33538   8BC1          MOV EAX,ECX
77D3353A   C1E9 02       SHR ECX,2
77D3353D   F3:A5         REP MOVS DWORD PTR ES:[EDI],DWORD PTR DS:[ESI
77D3353F   8BC8          MOV ECX,EAX
77D33541   83E1 03       AND ECX,3
77D33544   F3:A4         REP MOVS BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
77D33546   E8 E3FBFFFF   CALL user32.77D3312E
77D3354B   5F            POP EDI
77D3354C   5E            POP ESI
```

И далее программа останавливается на том месте, где копируется неправильный серийный номер, и прежде, чем это произойдёт, можем установить на него BPM ON ACCESS, а затем выполнить RUN и снова остановиться точно на сравнении.

```
00403CB5   8B0E          MOV ECX,DWORD PTR DS:[ESI]
00403CB7   8B1F          MOV EBX,DWORD PTR DS:[EDI]
00403CB9   39D9          CMP ECX,EBX
00403CBB   75 58         JNZ SHORT crackme_.00403D15
00403CBD   4A            DEC EDX
00403CBE   74 15         JE SHORT crackme_.00403CD5
00403CC0   8B4E 04       MOV ECX,DWORD PTR DS:[ESI+4]
00403CC3   8B5F 04       MOV EBX,DWORD PTR DS:[EDI+4]
00403CC6   39D9          CMP ECX,EBX
00403CC8   75 4B         JNZ SHORT crackme_.00403D15
00403CCA   83C6 08       ADD ESI,8
00403CCD   83C7 08       ADD EDI,8
00403CD0   4A            DEC EDX
00403CD1   75 E2         JNZ SHORT crackme_.00403CB5
00403CD3   EB 06         JMP SHORT crackme_.00403CDB
00403CD5   83C6 04       ADD ESI,4
```

Хорошо, думаю, что это был сложный случай, какой бы способ вы не избрали, так как место, в котором сохранялся серийник, постоянно менялось.

Ладно, оставляю домашнее упражнение – крэкми CrueHead’a 2, а со следующей главы мы переходим к новой теме.

Введение в крэкинг с нуля, используя OllyDbg - Глава 19

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-01-27

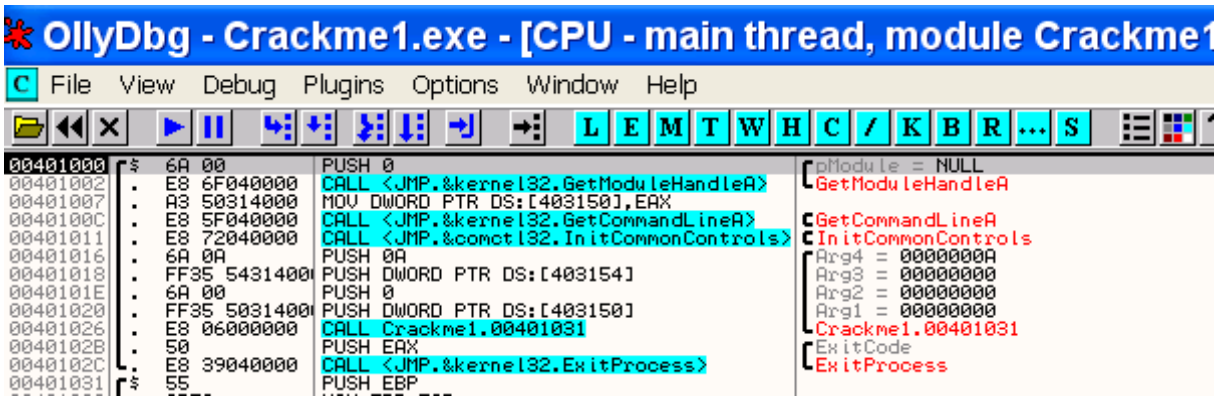
Приложение к статье (находится в архиве `wasm_files.zip`: `files/0473/ollydbg19.zip`)

Обнаружение OllyDbg - IsDebuggerPresent

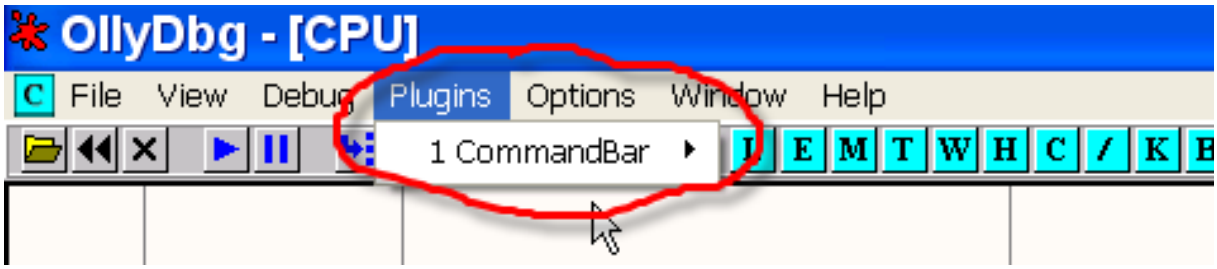
Как уже говорилось в предыдущей части, мы сконцентрируемся на том, какими способами избежать обнаружения OllyDbg отлаживаемой программой, и какие плагины для этого использоваться. Многие программы, когда обнаруживают, что они работают под отладчиком, закрываются или начинают работать по-другому, поэтому нужно любыми способами добиться того, чтобы жертва этого не узнала.

Эта часть посвящена способу обнаружения отладчика с помощью API-функции IsDebuggerPresent, которая наиболее часто используется.

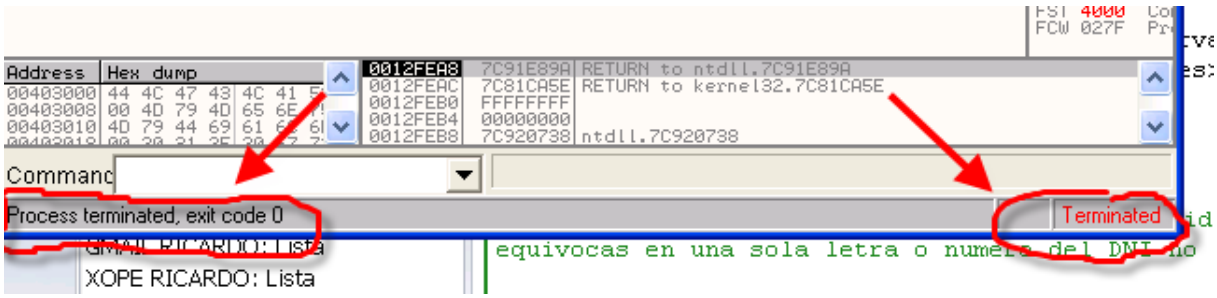
Для этого используем Crackme1.exe, который идёт вместе с данной лекцией. Загрузим его в OllyDbg.



Помните, что у моей OllyDbg пока что установлен только COMMAND BAR, а значит и нет никакого плагина, который мог бы избежать обнаружения с помощью IsDebuggerPresent, но как вообще работает эта функция?

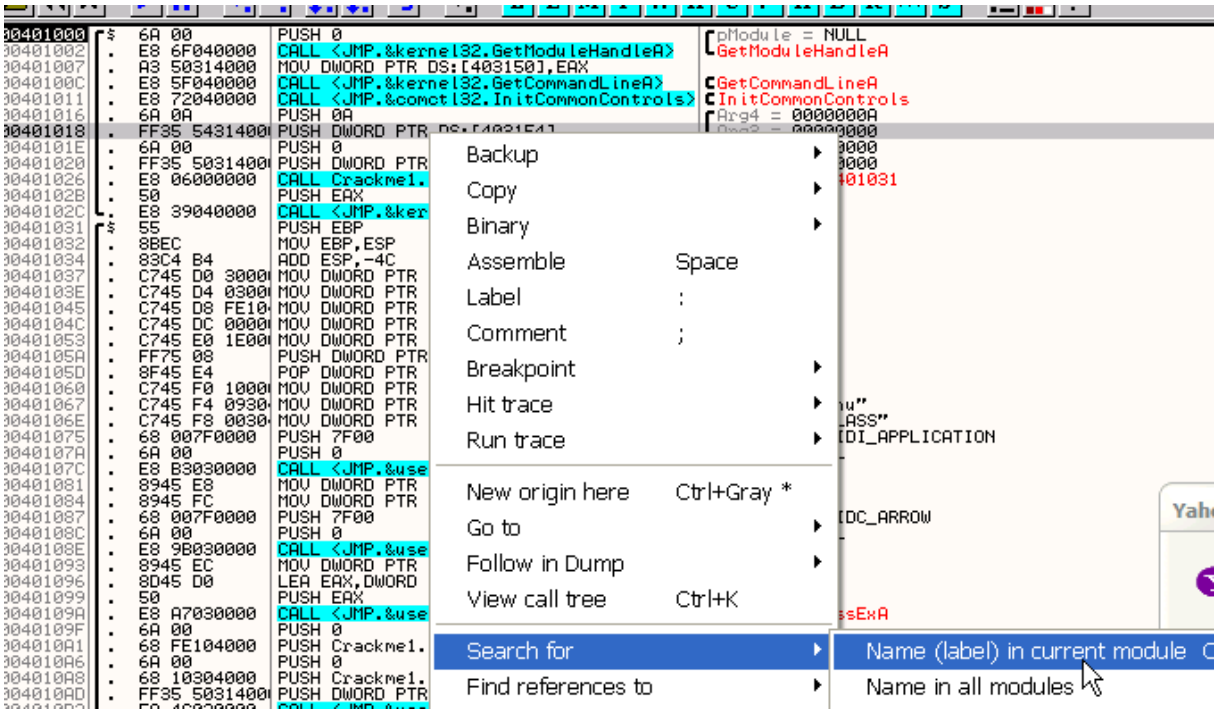


Если запустим крэкми с помощью F9, то увидим, что не только не открылось окошко крэкми, но и программа не запустилась.

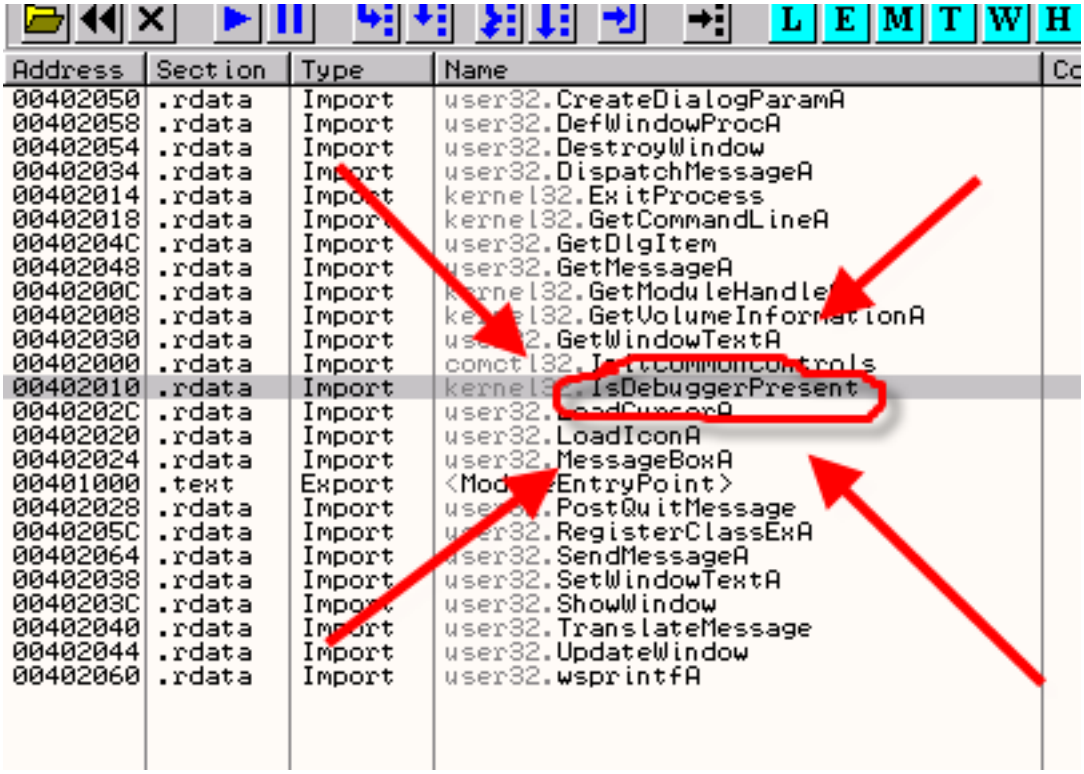


В нижней части окна OllyDbg сообщает нам, что программа прекратила выполнение, и, соответственно, никакого окна мы не увидим, хе-хе. Скорее всего, крэкми использует наиболее известный метод для обнаружения, использующий API-функцию IsDebuggerPresent.

Перезагрузим крэкми и посмотрим, находится ли она в списке API, используемых жертвой. Жмём на правую кнопку мыши на листинге и выберем..

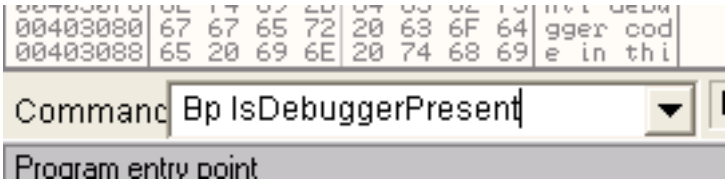


Видим список используемых API-функций.

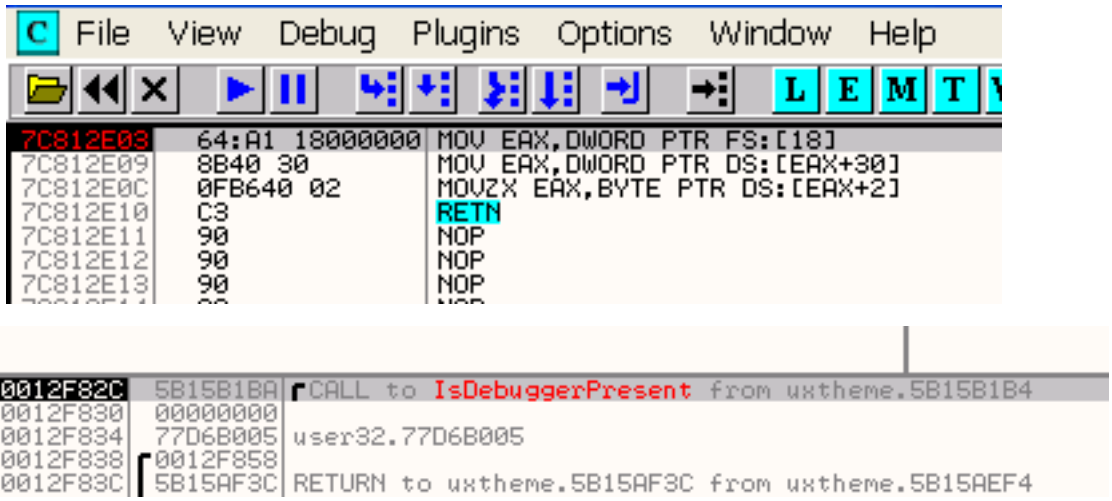


И смотрите, что мы нашли, хе-хе.

Устанавливаем BP на эту функцию, чтобы выяснить, где она применяется программой.



Нажимаем RUN и останавливаемся на вызове этой функции.



Из стека видно, что это API-функция без параметров, единственное, что она делает, это проверяет, отлаживается ли сейчас программа или нет, а если у вас есть какие-либо сомнения относительно данной функции или того, как она работает, то проконсультируйтесь со справочником по API-функциям.

Вот страница руководства, которым пользуюсь я.

IsDebuggerPresent Quick Info

[New - Windows NT]

The **IsDebuggerPresent** function indicates whether the calling process is running under the context of a debugger.

This function is exported from KERNEL32.DLL.

BOOL IsDebuggerPresent(VOID)

Parameters

This function has no parameters.

Return Value

If the current process is running in the context of a debugger, the return value is nonzero.

If the current process is not running in the context of a debugger, the return value is zero.

Remarks

This function allows an application to determine whether or not it is being debugged, so that it can modify its behavior. For example, an application could provide additional information using the **OutputDebugString** function if it is being debugged.

See Also

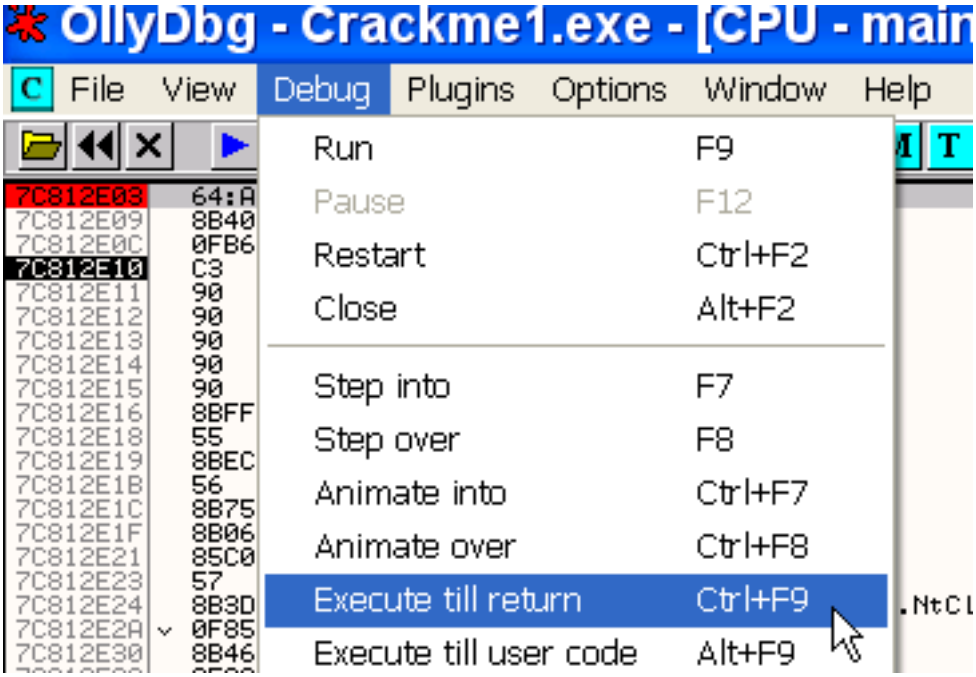
OutputDebugString

Это разъяснение функции. Переведём его.

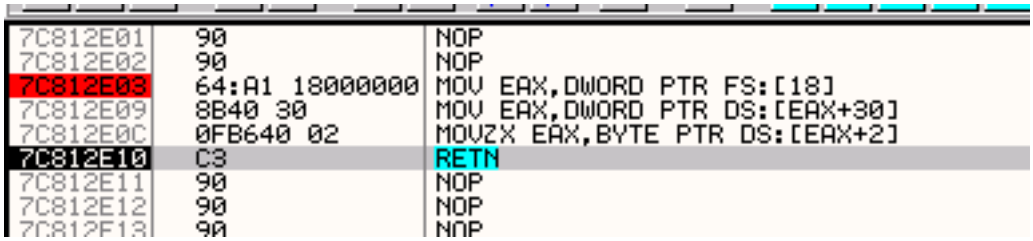
API-функция IsDebuggerPresent показывает, находится ли процесс, который её вызывает, в контексте отладчика, то есть отлаживается ли он.

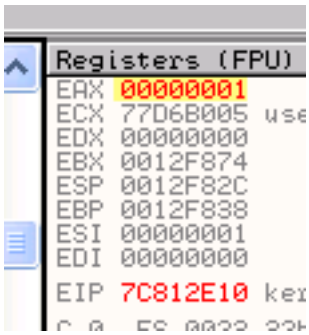
Далее говорится, что функция экспортируется из KERNEL32.dll, у которой, как видим, нет параметров, и если программа находится под отладкой, то возвращаемое значение равно единице, а если нет, то нулю.

Это очень важная информация, выполняем API-функцию до RET, чтобы посмотреть, куда возвращается интересующая нас информация.



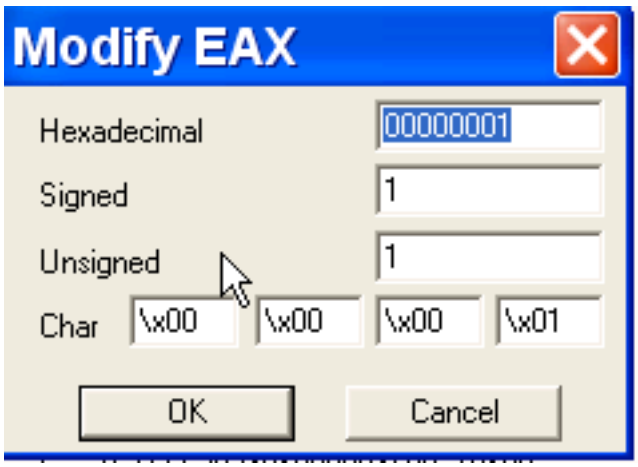
Доходим до RET, где смотрим регистры.

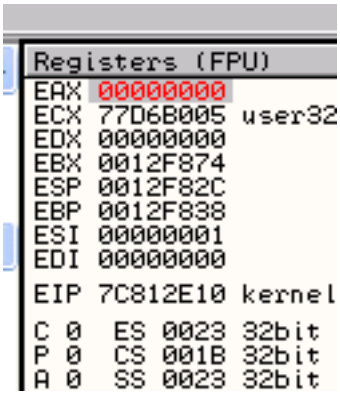




Выделенное розовым цветом – это место, где изменяется EAX. Как практически все API-функции, IsDebbuggerPresent помещает возвращаемое значение в EAX, тогда если в нём 1, то это говорит о том, что программа находится под отладкой.

Попробуем, что случится, если вручную сделать EAX равным 0, как если бы программа не отлаживалась.

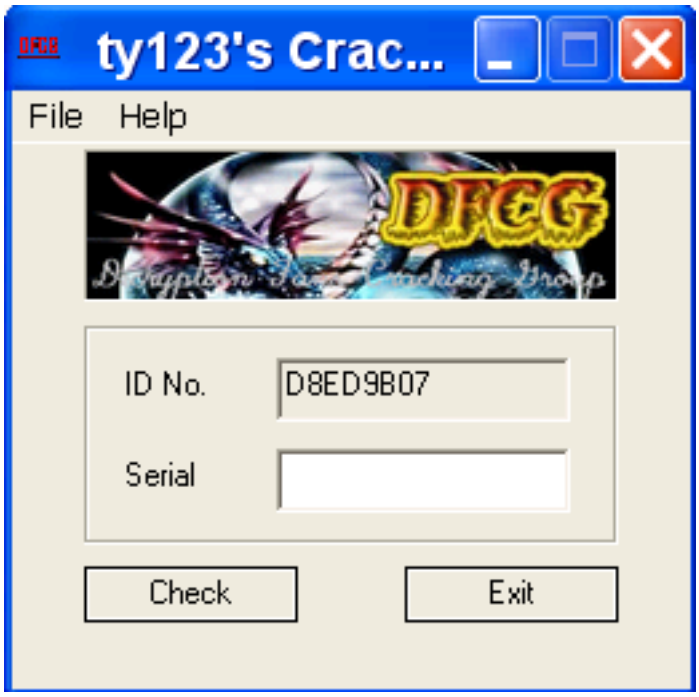




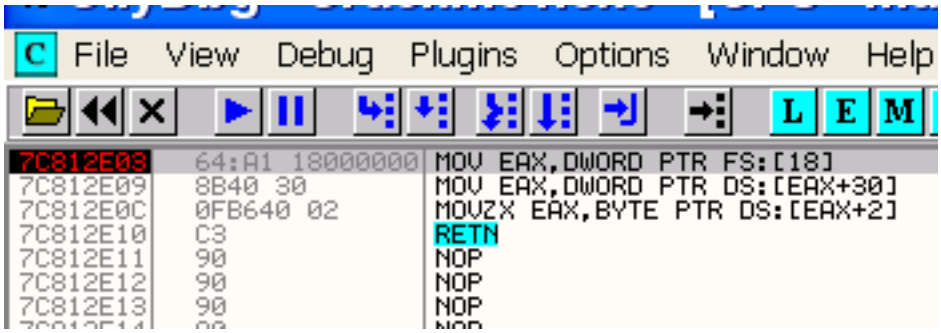
Делаем RUN.

```
7C812E03   64:A1 18000000   MOV EAX,DWORD PTR FS:[18]
7C812E09   8B40 30          MOV EAX,DWORD PTR DS:[EAX+30]
7C812E0C   0FB640 02        MOVZX EAX,BYTE PTR DS:[EAX+2]
7C812E10   C3               RETN
7C812E11   90               NOP
7C812E12   90               NOP
```

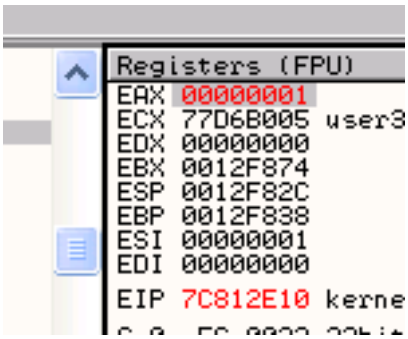
Снова останавливаемся на API-функции, поэтому снова доходим до RET и меняем EAX на ноль.



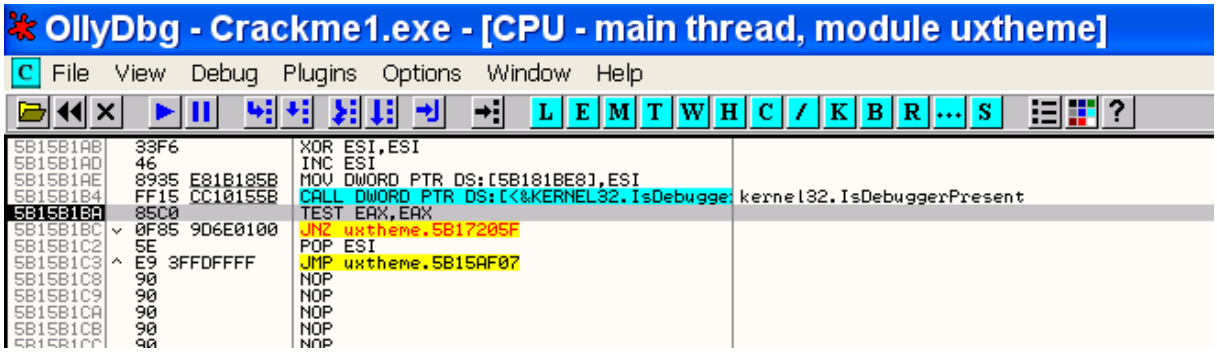
Видим, что программа запустилась, то есть вся её система обнаружения отладки основывалась на рассматриваемой API-функции. Перезапустим крэкми, и когда он остановится на IsDebuggerPresent, посмотрим, что происходит с возвращаемым значением.



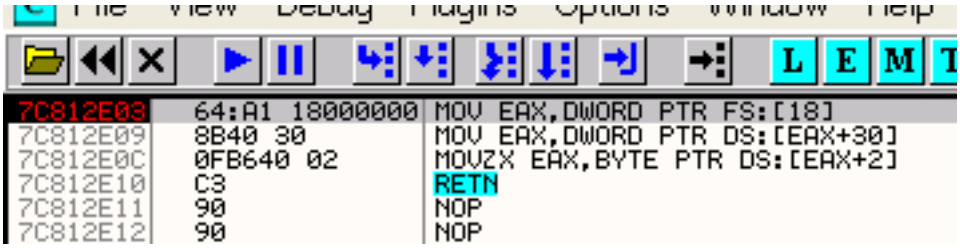
Снова доходим до RET, теперь можем перейти к пошаговой отладке с помощью F8.



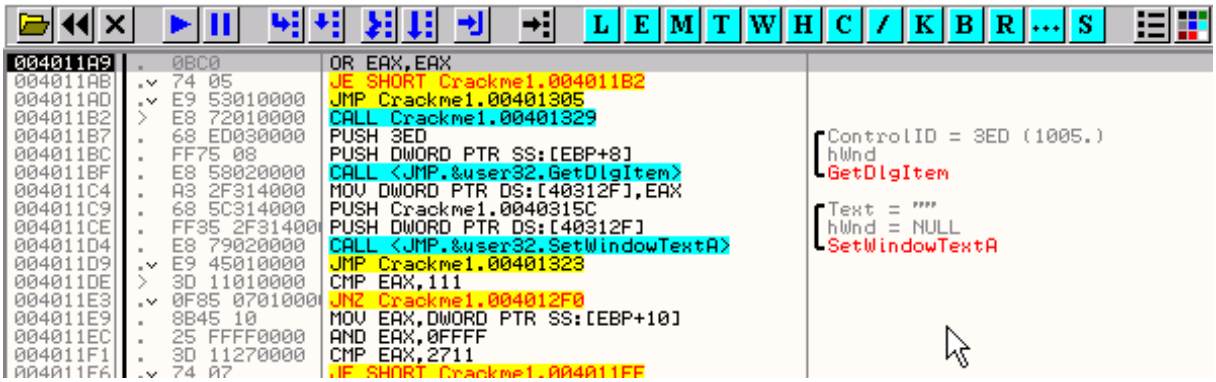
Когда доходим до RET, EAX равен 1, вернёмся в программу, нажав на F8.



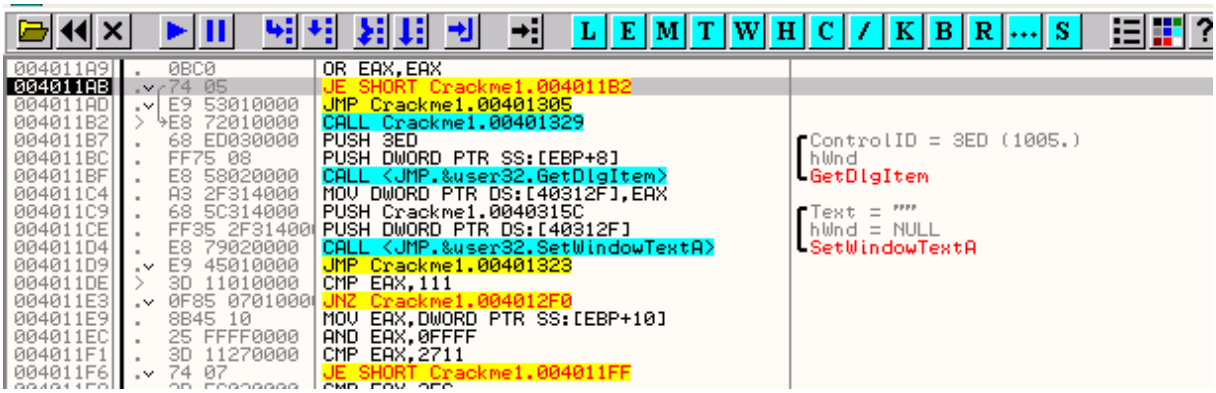
Здесь мы, прежде всего, видим, что если нет DLL uxtheme, то вызова не произойдёт. Делаем RUN и теперь происходит остановка на API-функции.



Там останавливаемся во второй раз, доходим до RET и возвращаемся в программу с помощью F8.

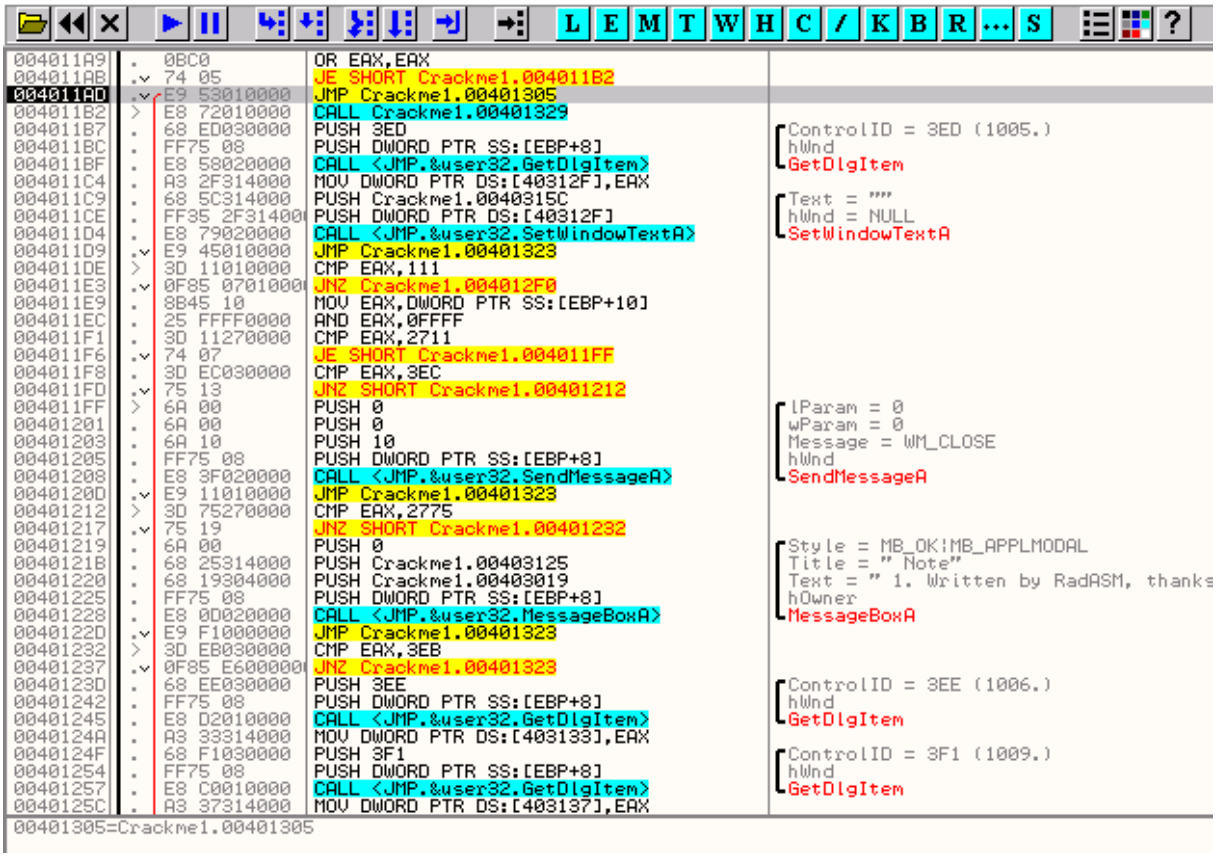


Оказываемся у JE, где происходит тестирование, равен ли EAX нулю или нет. Идём дальше.

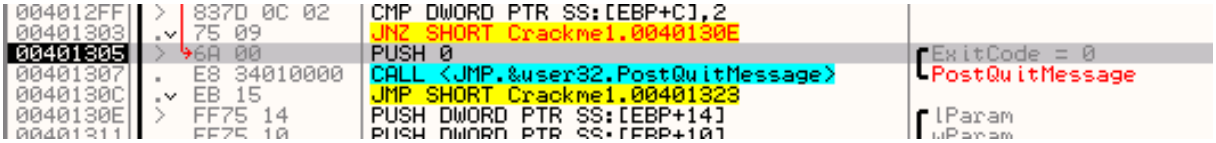


Видим, что если EAX отличается от нуля, то перехода не происходит, также замечаем, что если переход происходит, то выполнение программы продолжается функцией GetDlgItem, которая считывает информацию из окна крэкми.

После невыполнившегося перехода находится JMP, миновать который нельзя.



Продолжим трассировку, чтобы узнать, куда попадём.



Новая API-функция – PostQuitMessage, посмотрим её описание.

PostQuitMessage Quick Info

The **PostQuitMessage** function indicates to Windows that a thread has made a request to terminate (quit). It is typically used in response to a WM_DESTROY message.

VOID PostQuitMessage(
int *nExitCode* // exit code
);

Parameters

nExitCode
Specifies an application exit code. This value is used as the *wParam* parameter of the WM_QUIT message.

Return Values

This function does not return a value.

Remarks

The **PostQuitMessage** function posts a WM_QUIT message to the thread's message queue and returns immediately; the function simply indicates to the system that the thread is requesting to quit at some time in the future.

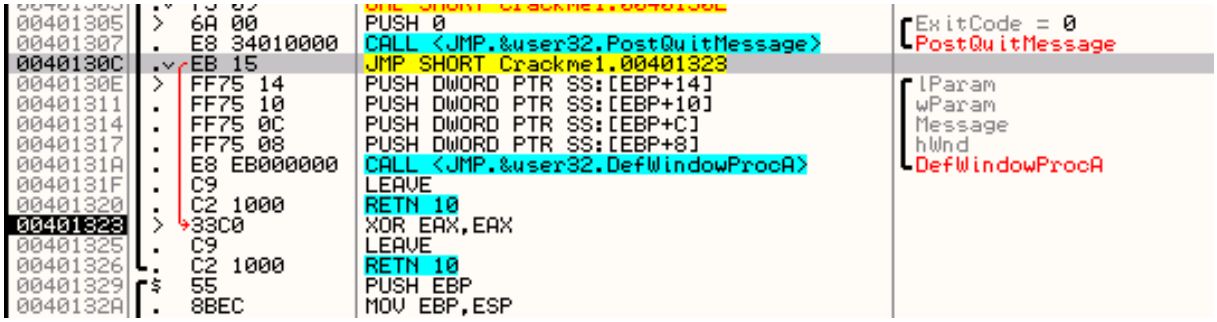
When the thread retrieves the WM_QUIT message from its message queue, it should exit its message loop and return control to Windows. The exit value returned to Windows must be the *wParam* parameter of the WM_QUIT message.

See Also

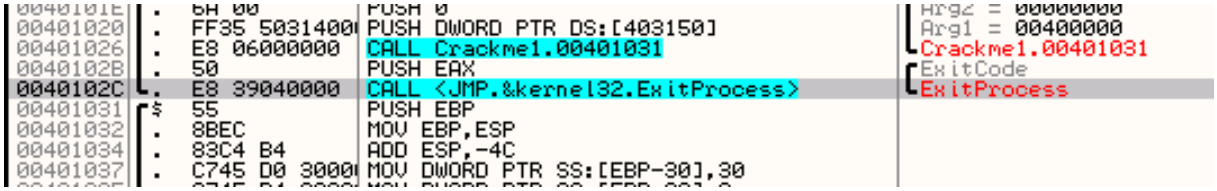
GetMessage, **PeekMessage**, **PostMessage**, WM_DESTROY, WM_QUIT

В переводе это означает, что данная API-функция посылает сообщение для закрытия окна WM_QUIT, то есть указание на то, что выполнение скоро будет завершено.

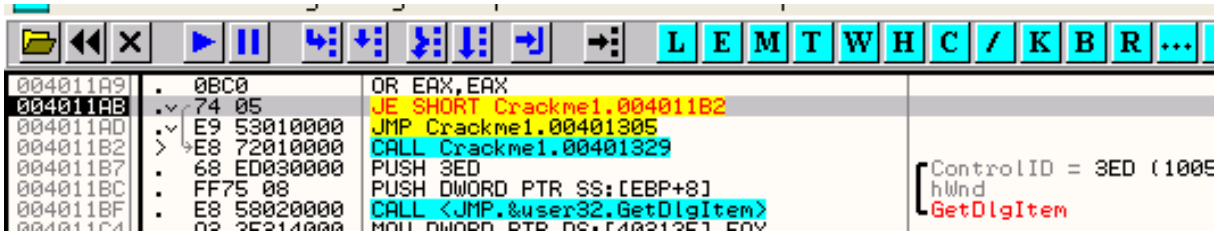
По возвращению оттуда:



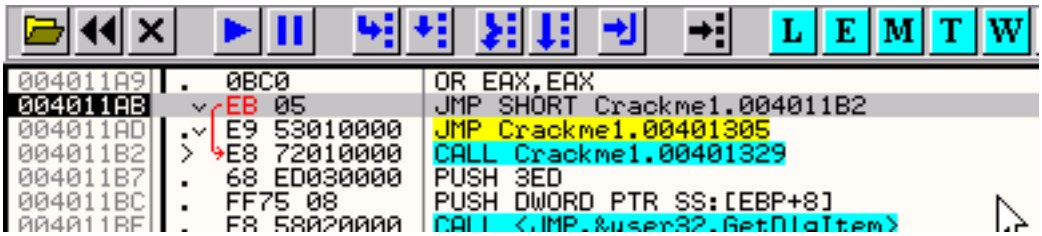
И если у нас будет терпение оттрассировать достаточно до того момента, когда всё закрывается, то обнаружим, что вызывается API-функция ExitProcess, которая и завершает процесс (если терпения нет, то установите на неё ExitProcess и остановитесь прямо на ней).



Так что теперь видим, что переход, который находится после возвращения из API-функции IsDebuggerPresent, отвечает за принятие решения о том, продолжать выполнение программы или нет. Одним из решений может быть патч, перезапустим программу и выполним её до пресловутого перехода.

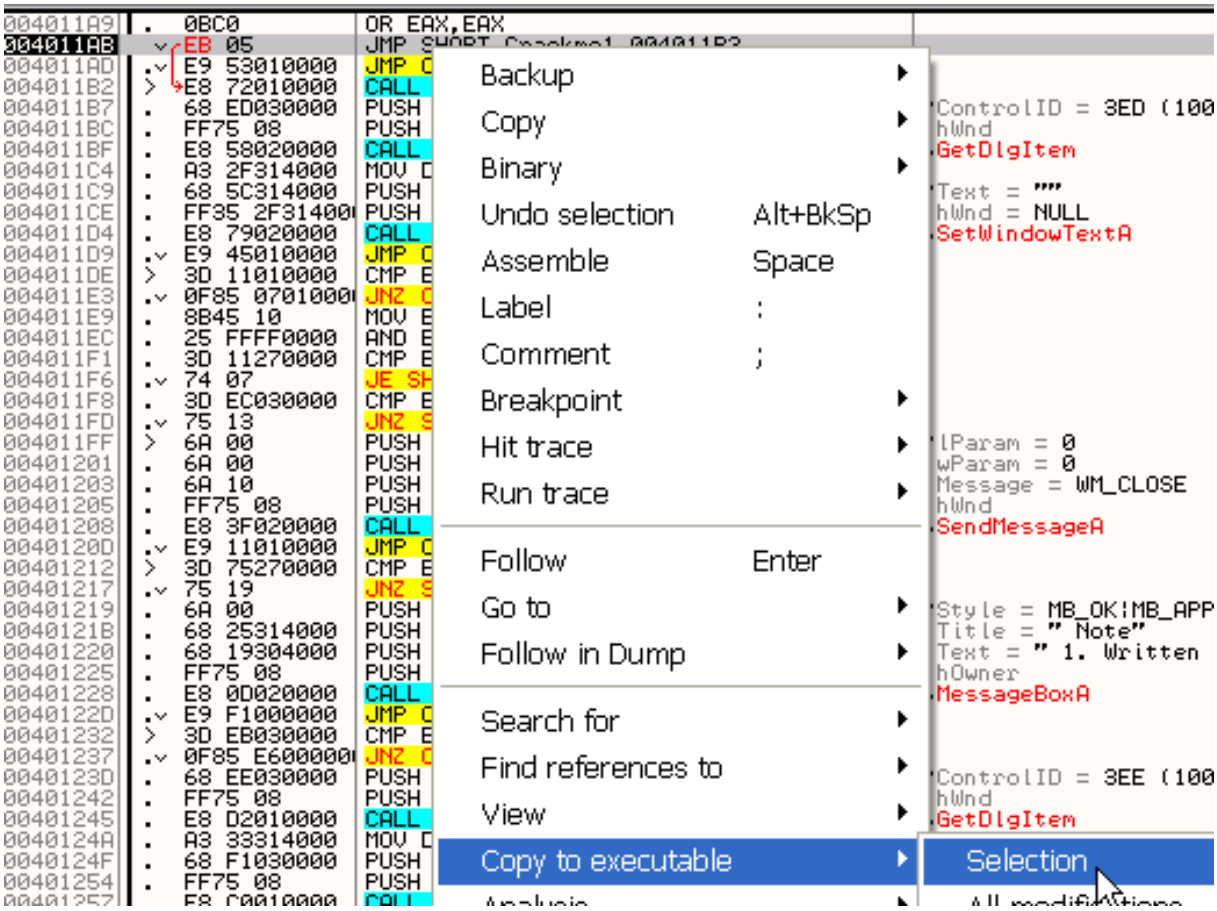


Останавливаемся там и изменим JE на JMP 4011b2 (для этого нажмите пробел).

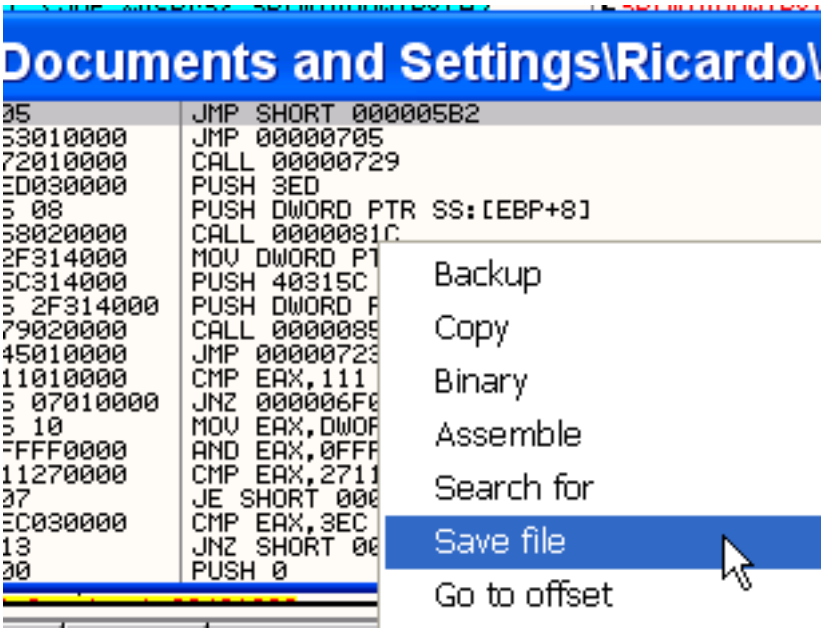


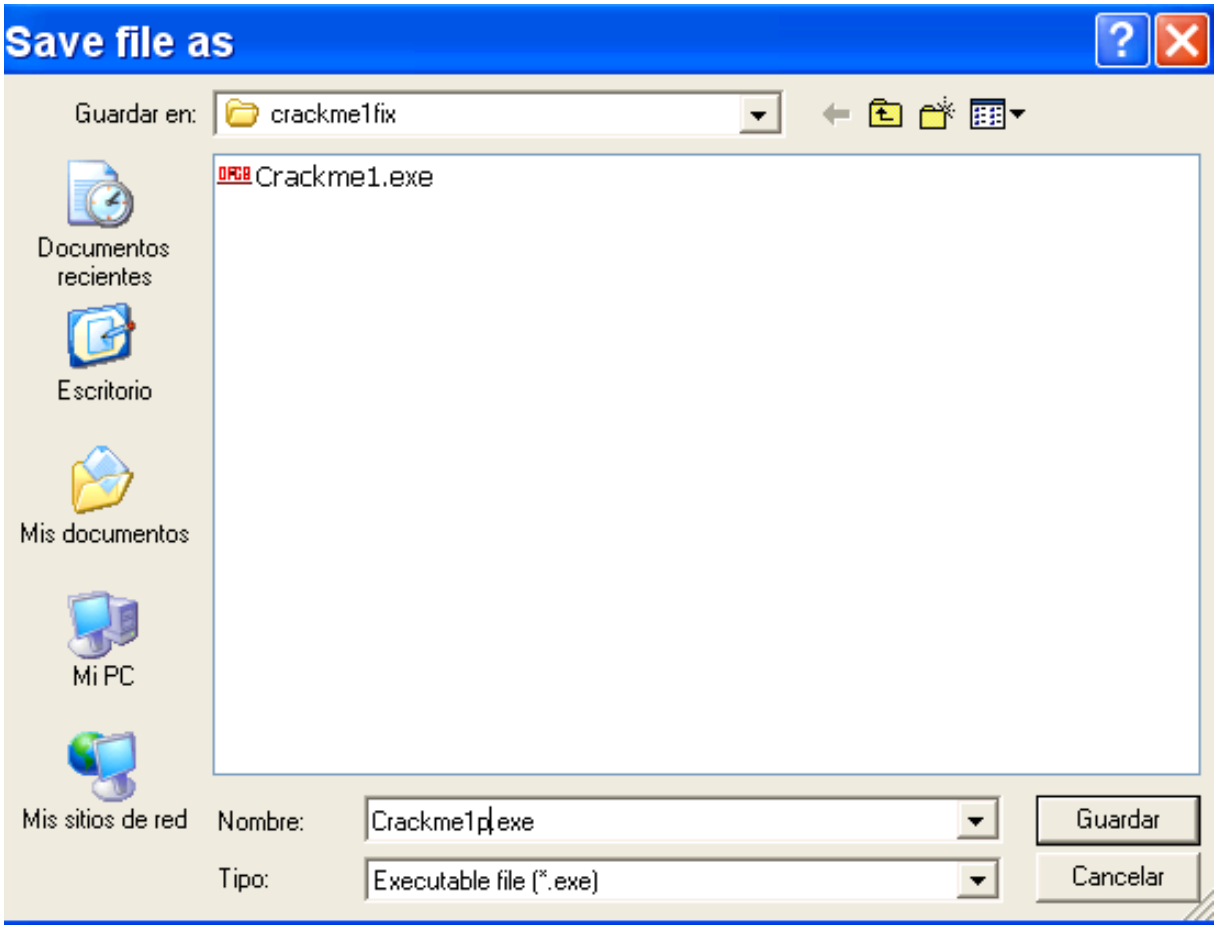
А здесь видим, что избежали перехода, который ведёт к завершению выполнения программы, поэтому она продолжила выполнение.

Сохраним программу с внесёнными нами изменениями под каким-нибудь другим именем. Нажимаем правую кнопку мыши.



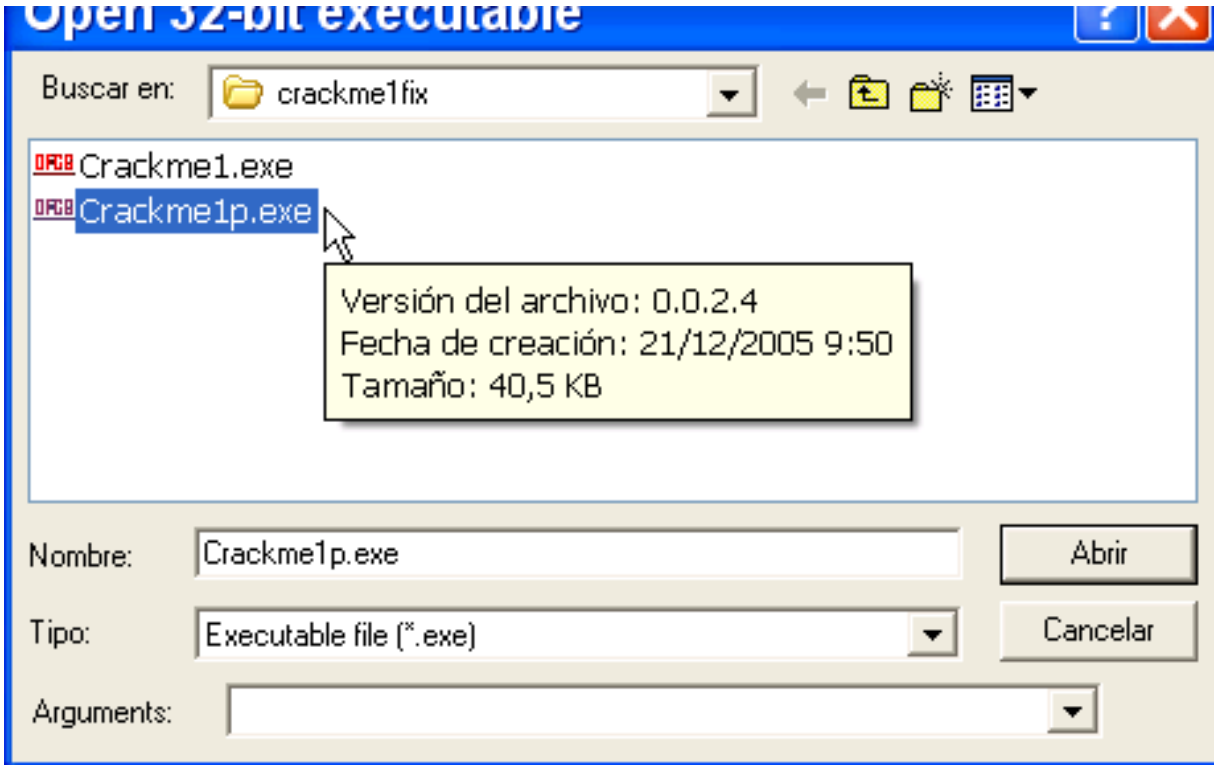
И в открывшемся окне снова нажимаем правую кнопку.





Сохраняем под именем crackme1p, так что теперь у нас исходная программа и пропатченная.

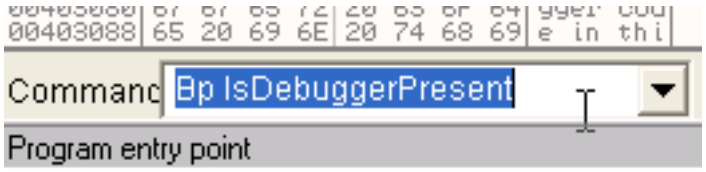
Теперь откроем пропатченную программу в OllyDbg, чтобы проверить, выполнится ли она в отладчике. Никаких BP устанавливать не будем.

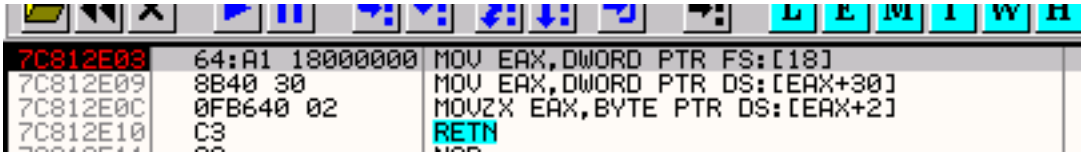


Выполняется превосходно, так что теперь мы знаем, что эту программу можно заставить выполняться, изменив значение, возвращаемое функцией IsDebuggerPresent в EAX, также знаем, как пропатчить программу, чтобы не приходилось делать это каждый раз вручную.

С этим крэкми у нас нет проблем – пропатчили и работает, но, конечно, существуют более простые способы, использующие плагины, которые позволяют нам избежать всех этих хлопот. Тем не менее, полезно знать, как это делать самим.

А сейчас загрузим исходный файл и рассмотрим API-функцию поподробнее.

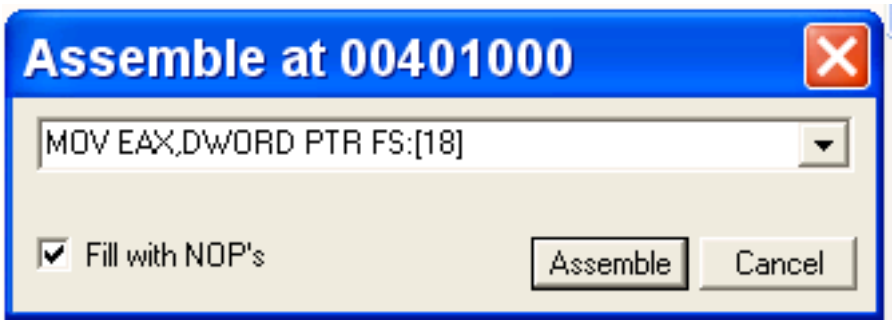


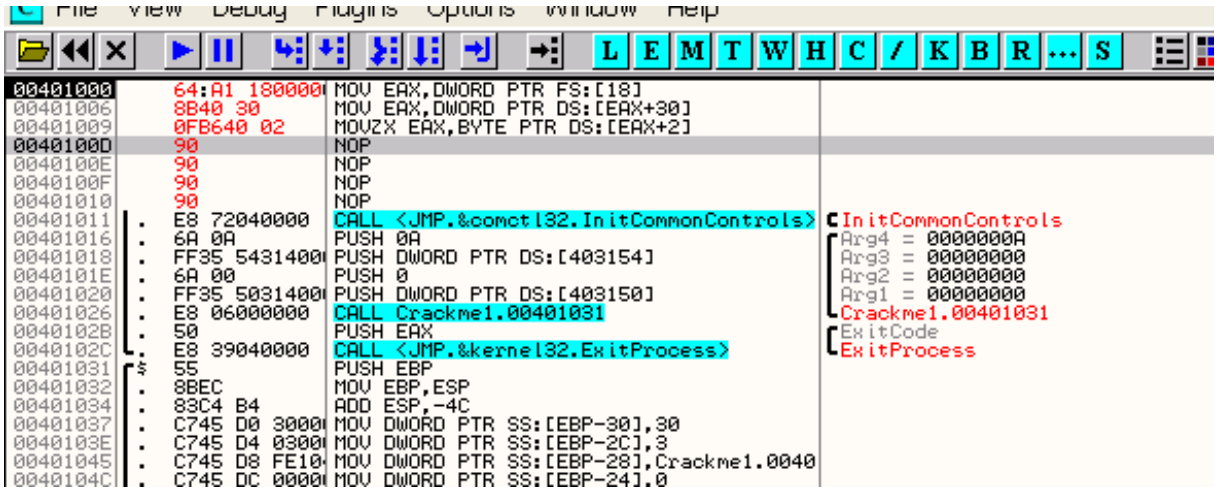


Здесь три невзрачных инструкции MOV, с помощью которых данная API-функция каким-то образом определяет, отлаживается ли программа или нет. Первое, что мне пришло в голову – это то, что если программа отлаживается, то они не может выполнить эти три строки, и в EAX попадает значение 1. Попробуем перезагрузить исходный файл крэкми и добавить в точку входа следующие строки:

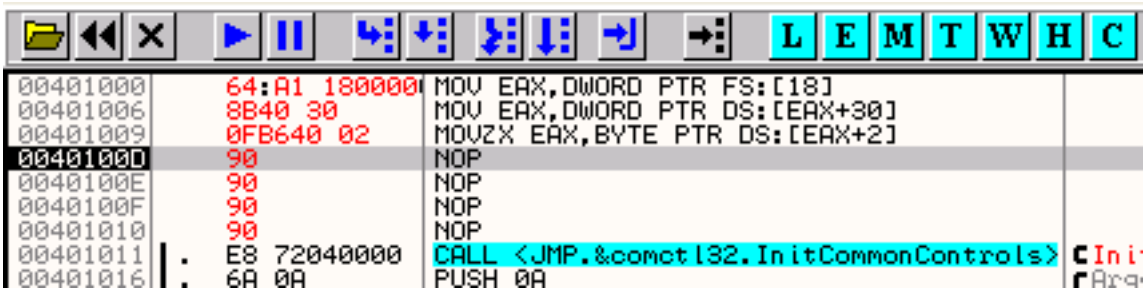
```
MOV EAX,DWORD PTR FS:[18]
MOV EAX,DWORD PTR DS:[EAX+30]
MOVZX EAX,BYTE PTR DS:[EAX+2]
```

Можем их просто скопировать их по очереди.

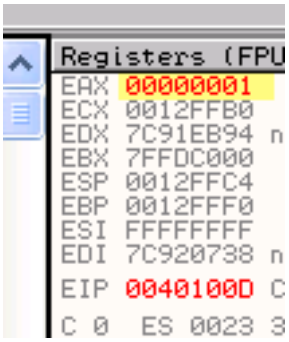




Итак, добавили три строки.



Видим, что после выполнения последней из них, в EAX содержится значение 1, также, как после выполнения API-функции, из которой мы и взяли эти строки.



Естественно, что программа не остановилась на IsDebuggerPresent, потому что мы её не вызывали, но тем не менее, мы определили, что программа отлаживается точно с таким же успехом.

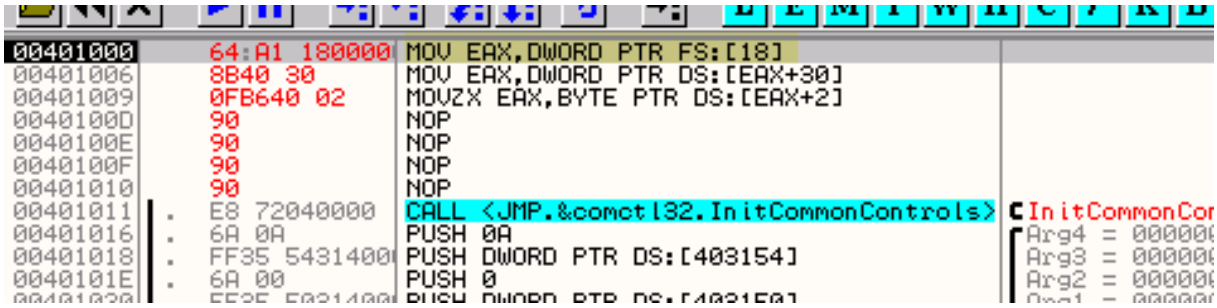
Поэтому очень важно определить, что же действительно считывается и что именно говорят эти три строки.

Когда программа запускается, система сохраняет в определённом месте в памяти, находится ли она (программа) под отладкой, и если там находится ноль, то значит, что нет. Этот байт и

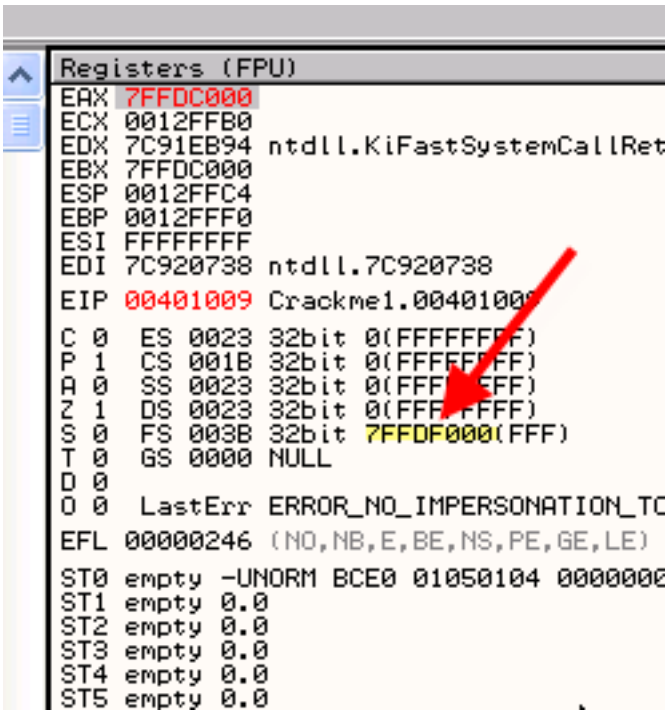
считывают эти три строки и соответствующая API-функция.

Как мы можем найти этот байт? Давайте взглянем на первую строку из указанных трёх, и шаг за шагом их проанализируем.

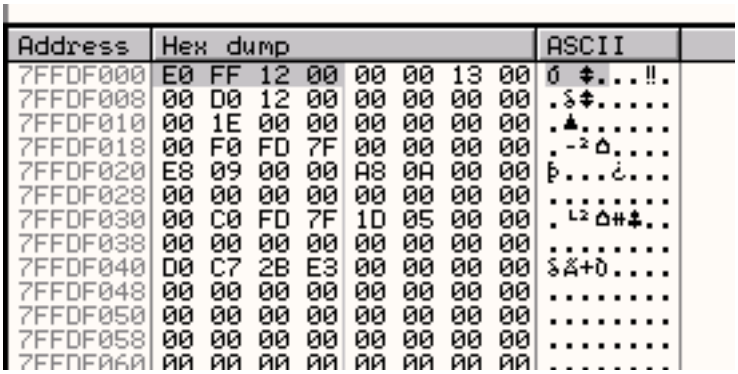
Первая строка:



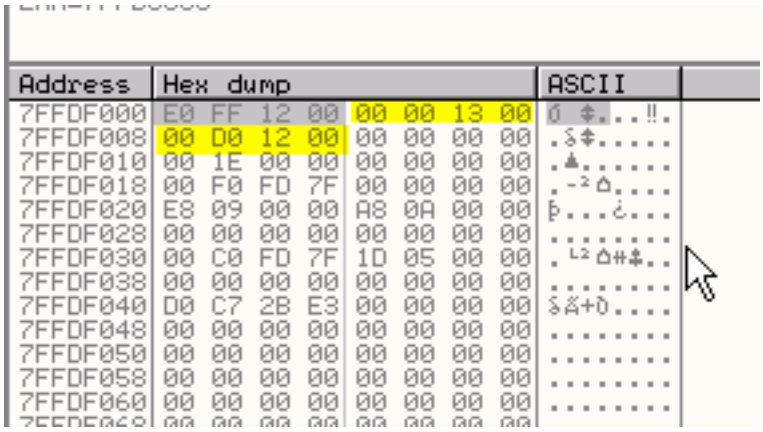
В OllyDbg идём сюда:



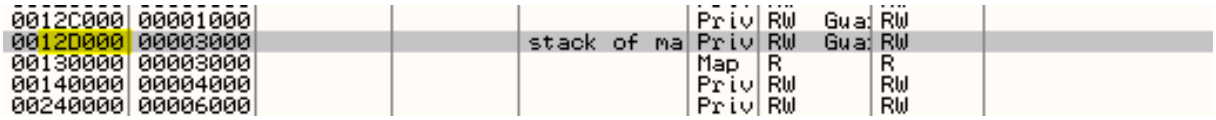
В окне регистров отображается очень важное значение, не хочу нагружать сложными названиями, но это значение указывает на структуру, где содержится также очень важная информация о выполняемой программе. Посмотрим через DUMP, что находится по данному адресу (на вашем компьютере этот адрес может быть другим, так что смотрите в вашей OllyDbg правильное значение).



Эта структура называется TEB или TIB и хранить полезную информацию, которая нам может быть нам очень полезной. Например, в TIB хранится, откуда начинается и где заканчивается стек программы.

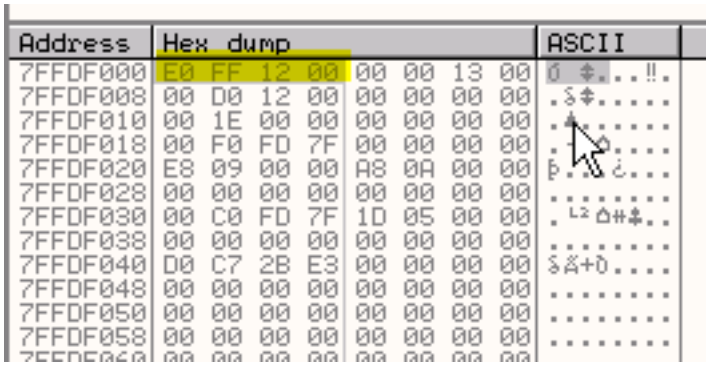


Посмотрим в окне M секцию стека.

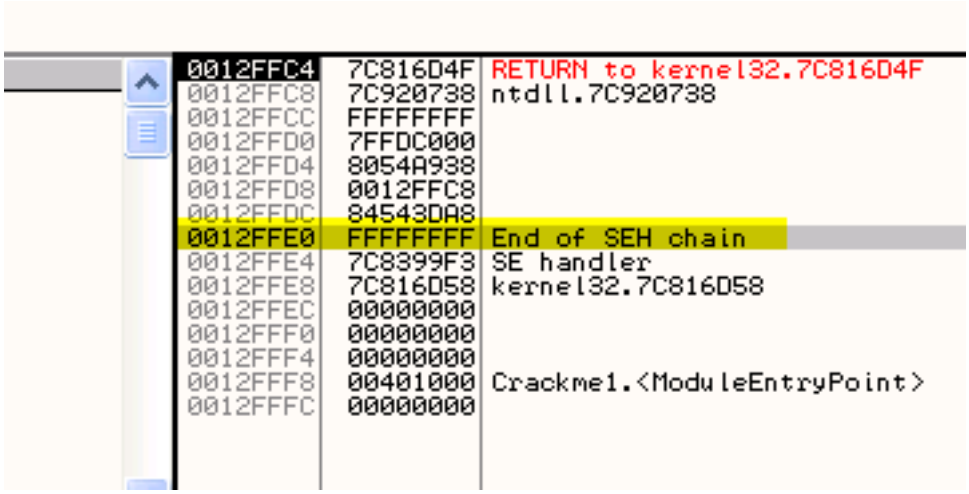


Видим, что начинается в 12d000 и заканчивается там, где начинается следующая секция (130000).

Есть много других интересных значений, которые мы рассмотрим, когда будем изучать исключения, но пока, если взглянем на стек, то увидим по вышеуказанному адресу значение 12ffE0 (у меня).

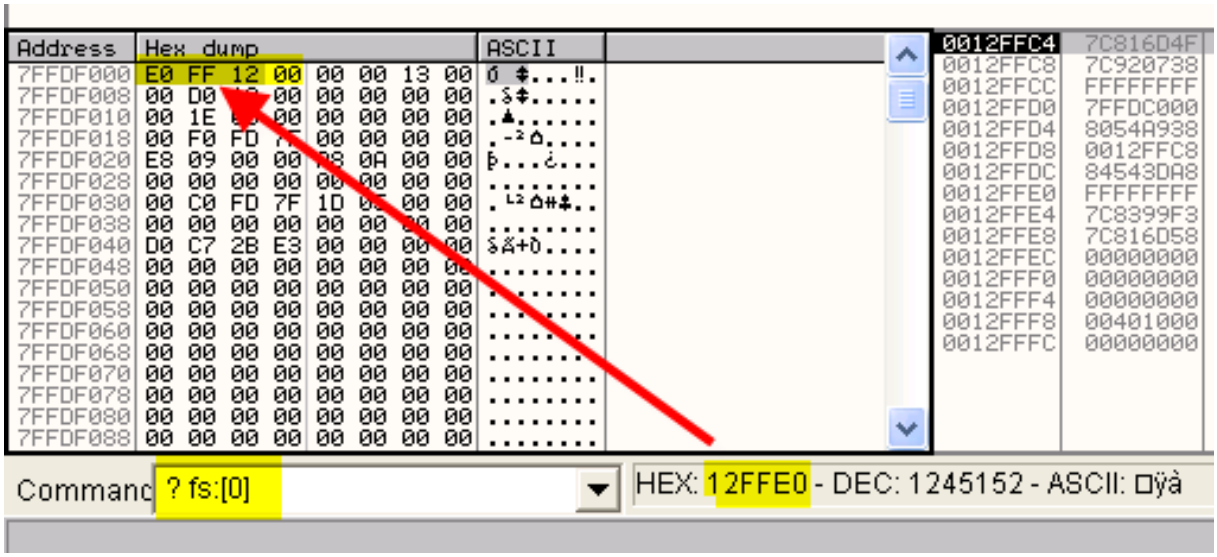


Поищем его в стеке.

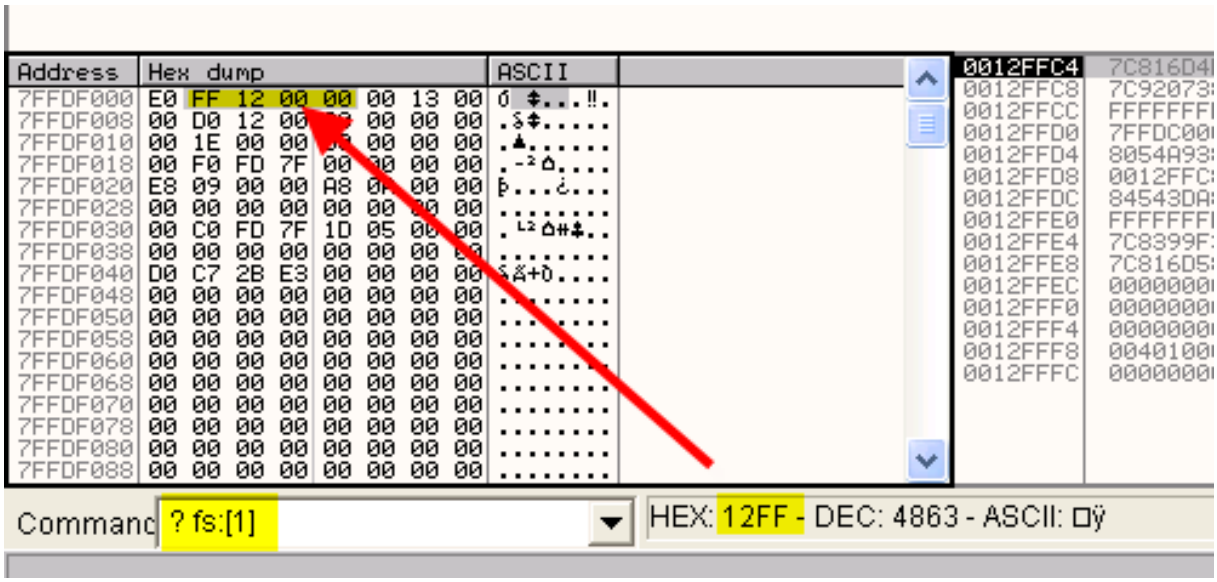


Видим, что оно помечено как END OF SEH CHAIN (конец SEH-цепочки). Пока что это нам мало что говорит, но это связано с исключениями.

Интересно то, что следуя определённому алгоритму можно достать из него любое значение, например, которое находится после Fs:[0].



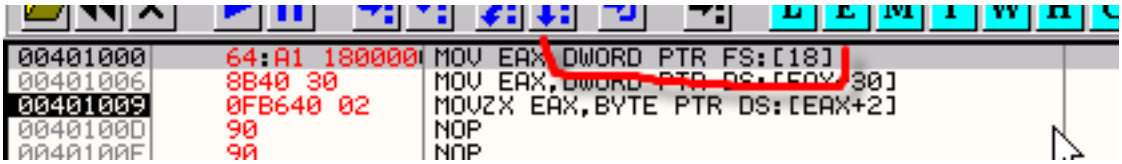
Это Fs:[0], если поищем в command bar'е значение fs:[1], то получим:



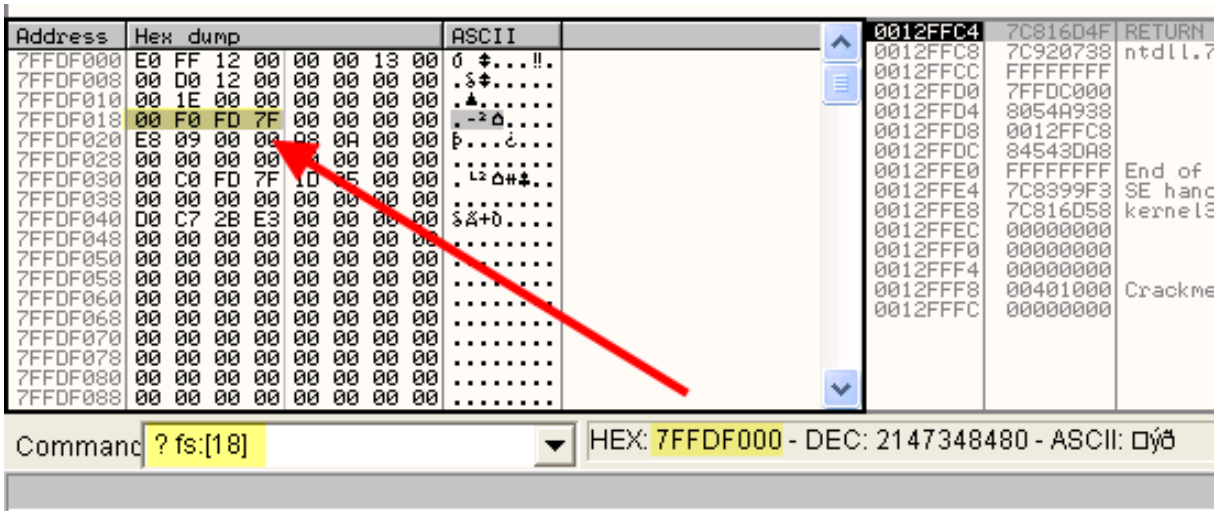
Поэтому:

```
Fs: [0] на моей машине здесь содержится 7ffdf000
Fs: [1] на моей машине здесь содержится 7ffdf001
……..
……..
Fs: [18] на моей машине здесь содержится 7ffdf018
```

То есть, это значение, которое, как мы помним, было на первой строке API-функции.

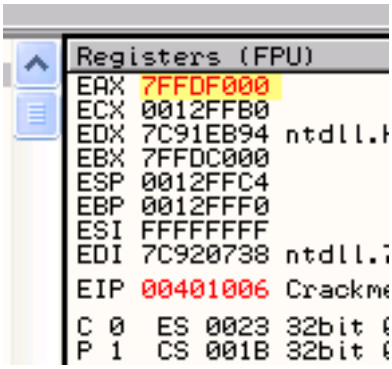


То есть видим содержимое 77ffdf018 (оно же FS:[18]).

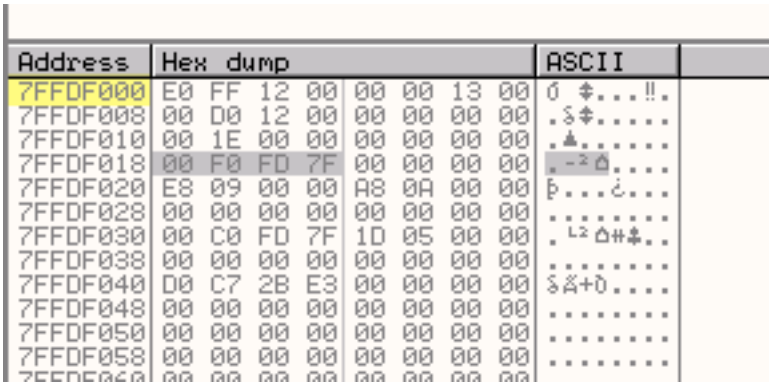


Вот это значение, которое сохранено в TIB в FS:[18] – значение, которое отображается в регистрах, указывающее на начало TIB.

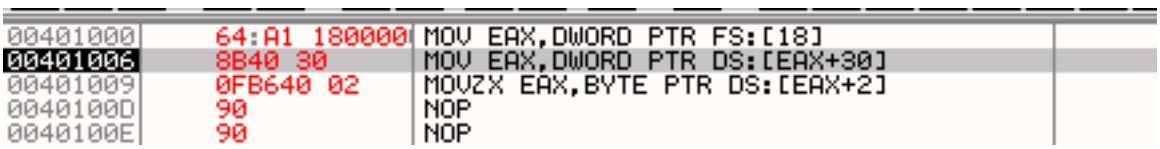
Таким образом, первая строка загружает в EAX указатель на начало TIB, конкретное значение может отличаться на разных машинах. Нажав F8, мы переместим это значение в EAX.



Хорошо, теперь в EAX находится указатель на TIB.

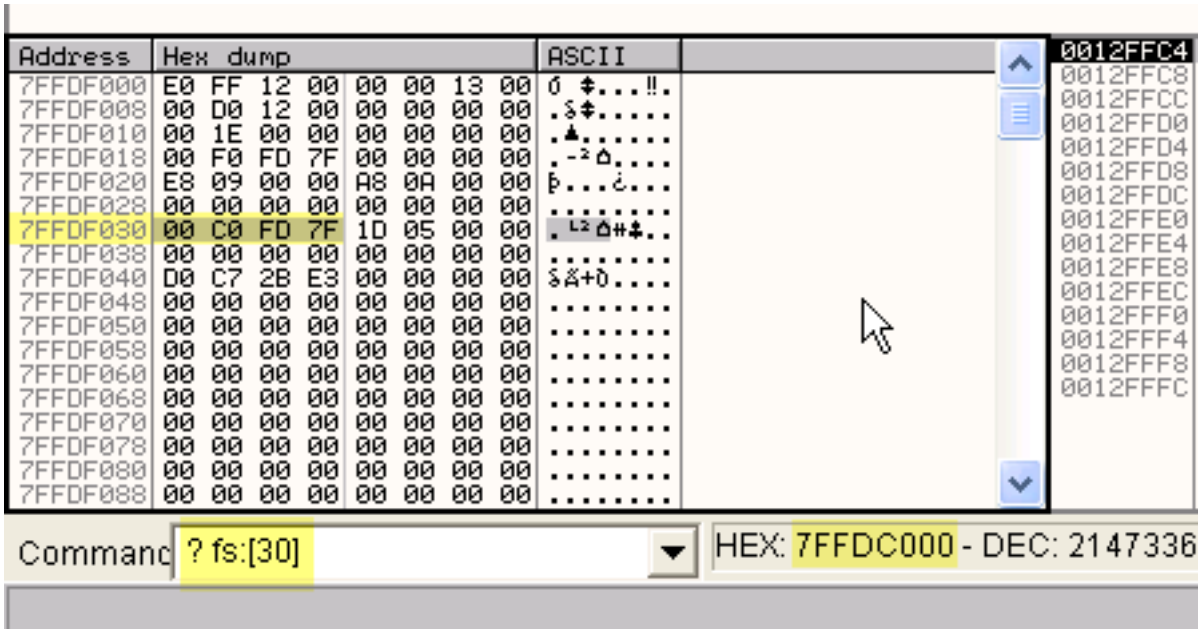


Теперь следующая строка.

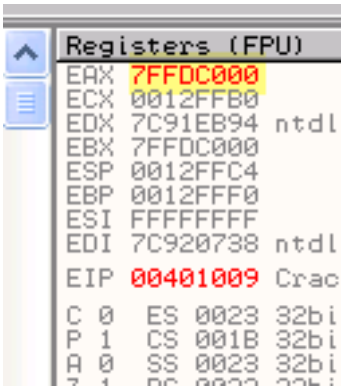


К EAX прибавляется 30, на моей машине в результате получается 7ffdf000 + 30 =7ffdf030.

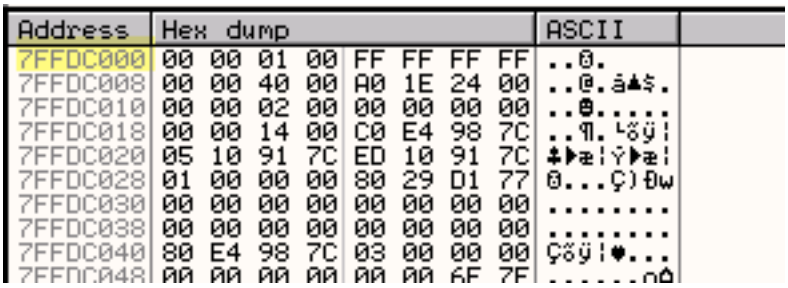
Это fs:[30].



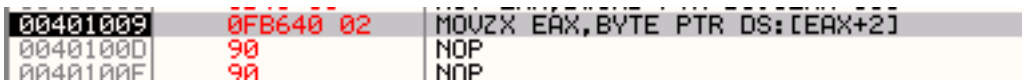
То есть в EAX помещается содержимое 7ffdf030 или fs:[30], на моей машине это содержимое равно 7ffdc000, не спутайте это с началом TIB, эти значения различаются.



Это указатель на что-то другое, посмотрим на что именно через DUMP:



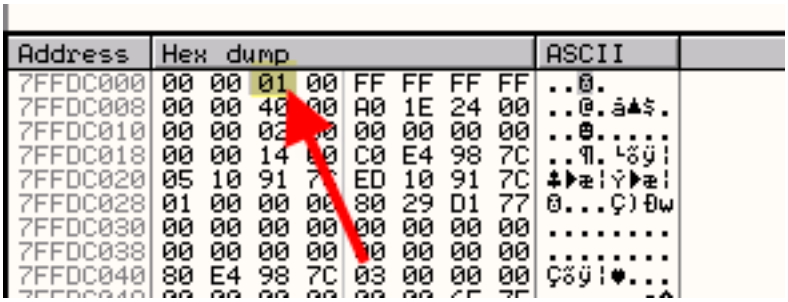
Последняя строка:



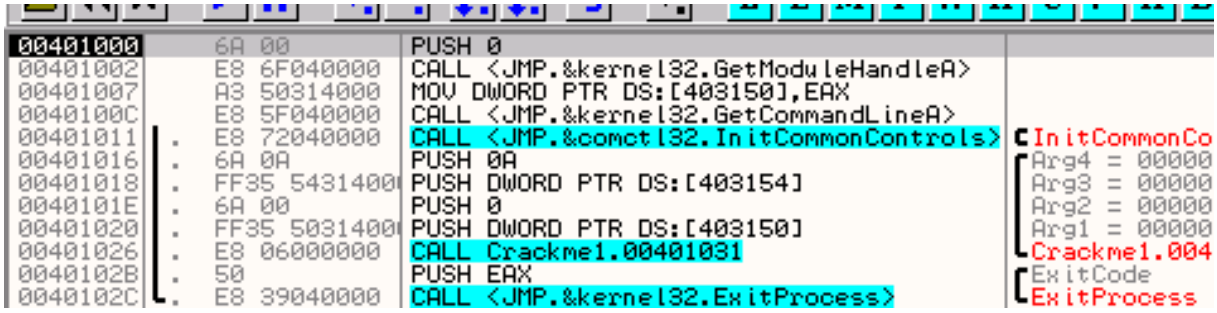
Сумма 2 и EAX равна:

```
7FFDC000 + 2= 7FFDC002
```

И в EAX перемещается байт, адрес которого получился в результате, а он и есть искомый байт. На моей машине он находится по адресу 7FFDC002.

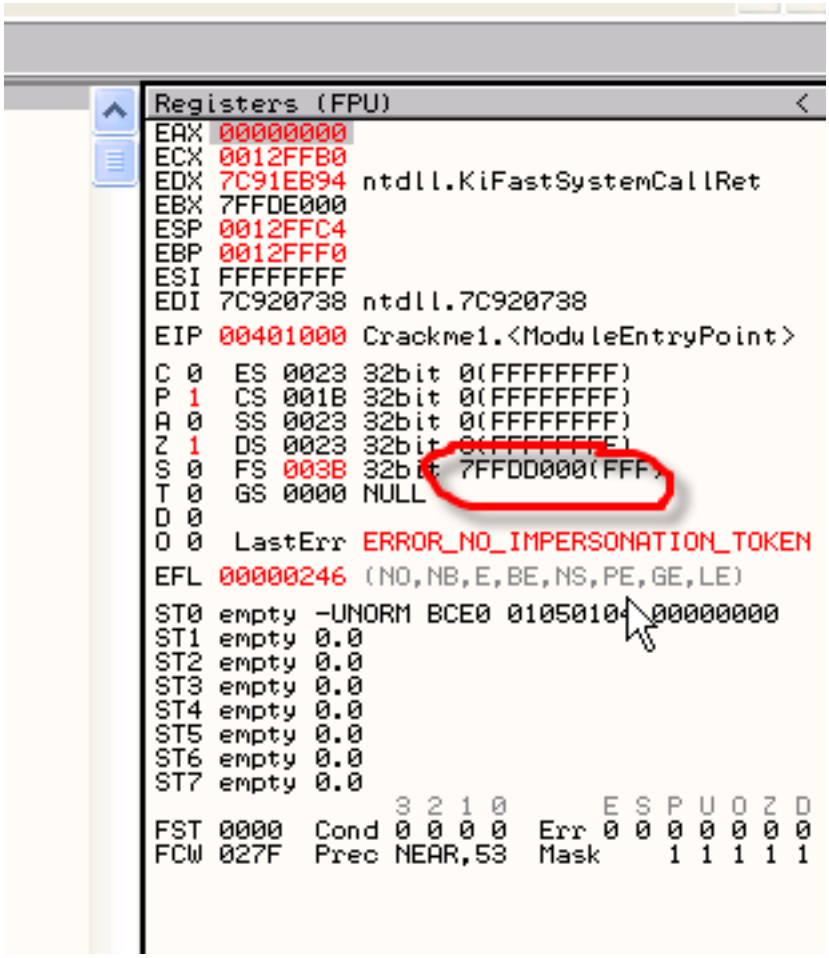


Система сохраняет 1 в этом байт, который считывается API-функцией IsDebuggerPresent. У вас адрес этого байта может отличаться, но вы сможете легко его найти, используя вышеуказанный метод. Загрузим заново крэкми в OllyDbg.

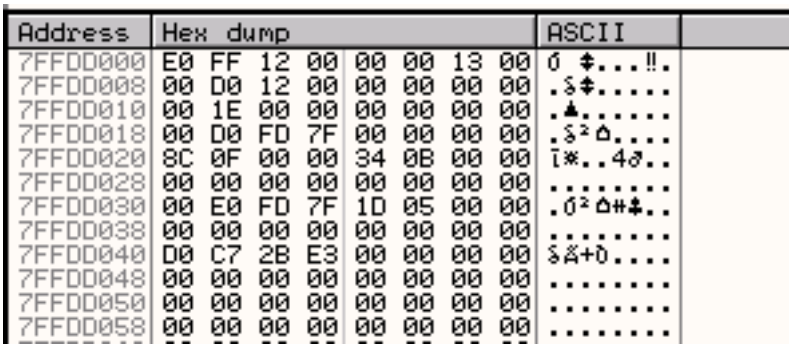


Вот краткое изложение того, как найти искомый байт, используемый в IsDebuggerPresent.

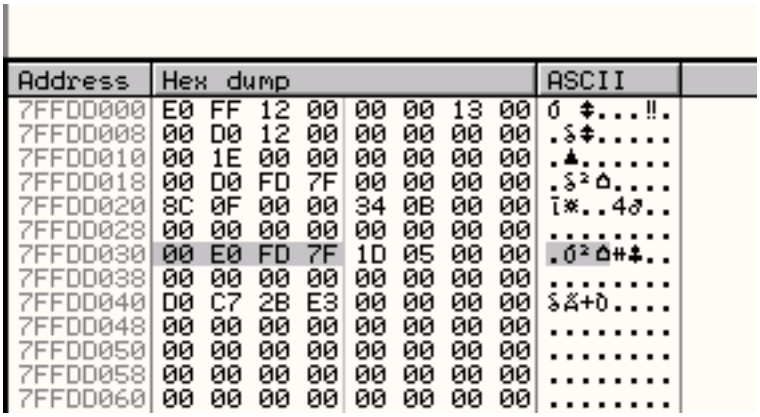
Ищем начало TIB в окне регистров.



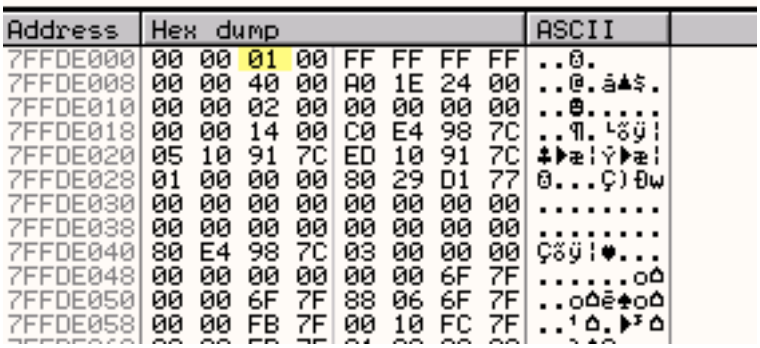
Смотрим TIB через DUMP (он уже не будет располагаться по тому адресу, что в прошлый раз).



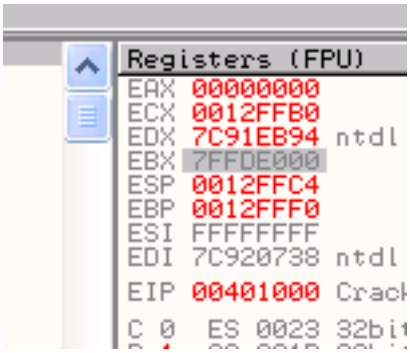
Ищем fs:[30], то есть к адресу, указывающему на TIB, прибавляем 30 и смотрим содержимое, находящееся по получившемуся адресу.



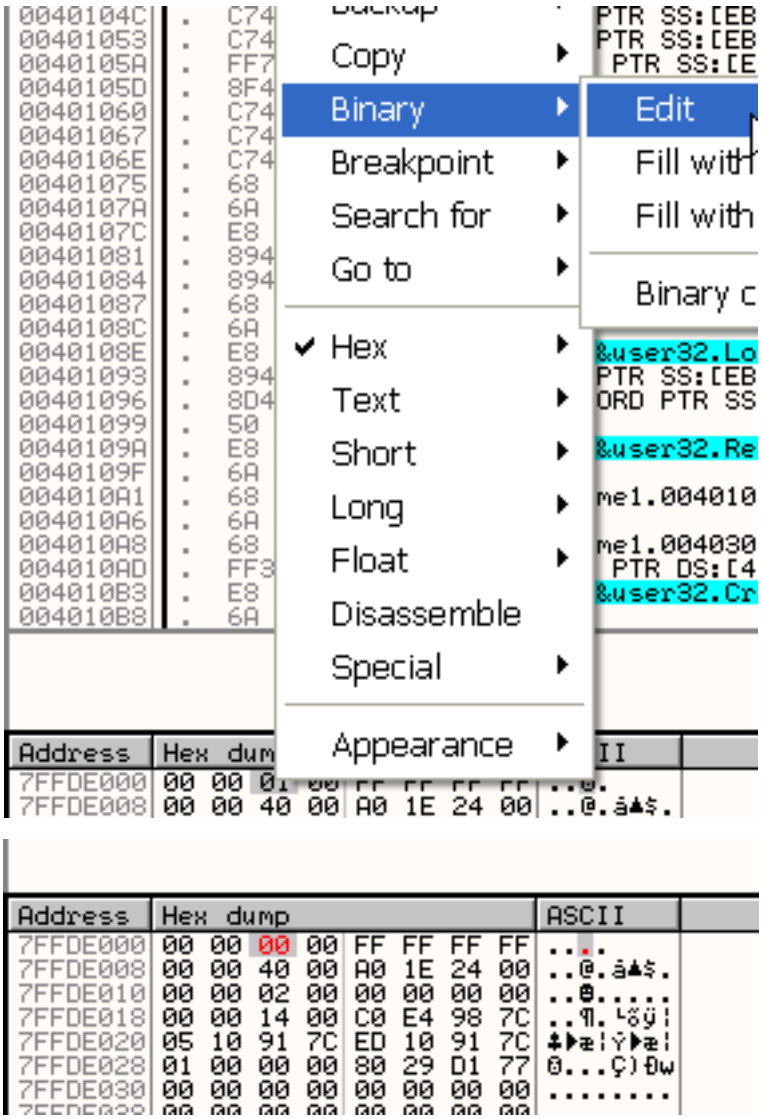
Его содержимое (7ffde000) также является адресом, смотрим через DUMP.



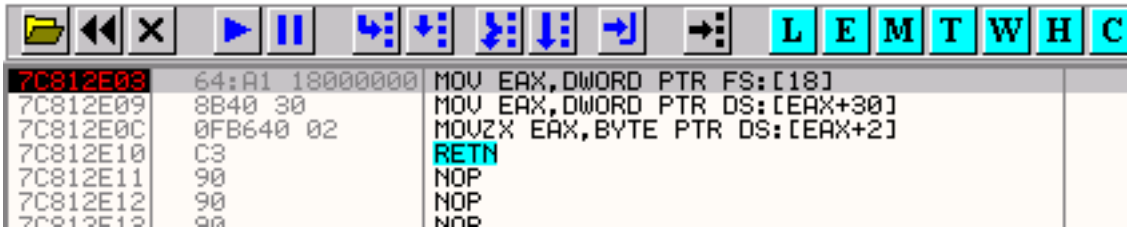
И прибавляем 2 – вот и искомый байт. (Многие обращают внимание, что когда программа запускается, в EBX уже находится указатель на эту область, так вот, EBX=fs:[30]).



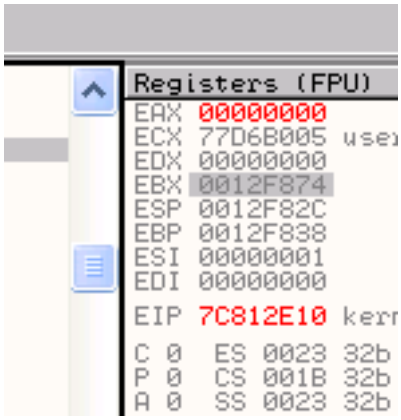
Ок, это и есть весь метод целиком, который можно использовать, если не находимся в начале программы и не знаем значения, которое имел EBX при старте. Теперь поместим ноль в найденный байт.



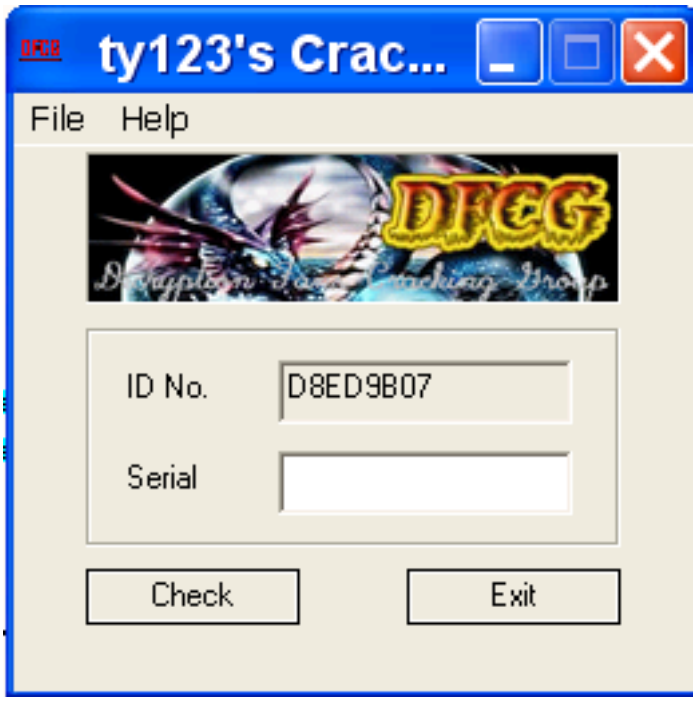
Теперь установим BP на функцию IsDebuggerPresent, чтобы посмотреть, работает ли данный подход.



Трассируем до RET.



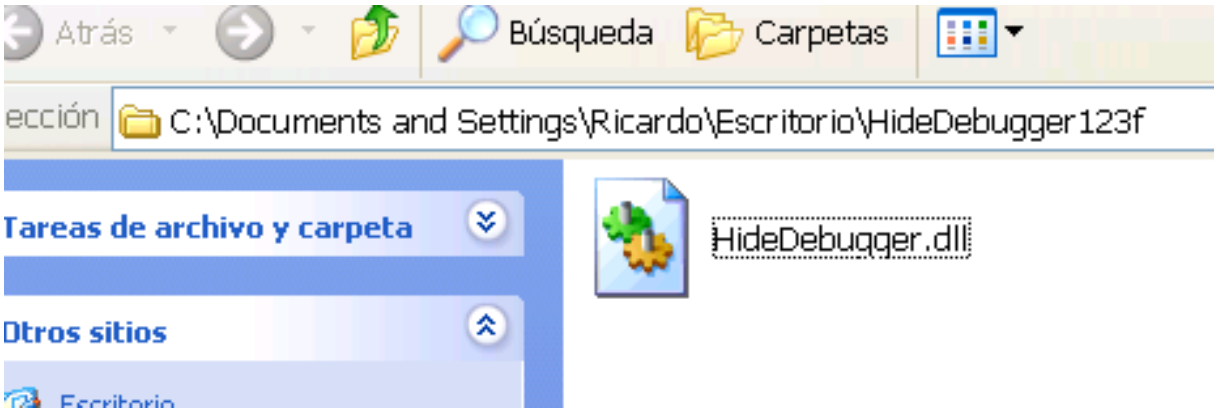
Видим, что возвращённое в EAX значение равно нулю, так как поменяли значение байта, из которого берётся значение, и программа верит, что она не находится под отладкой.



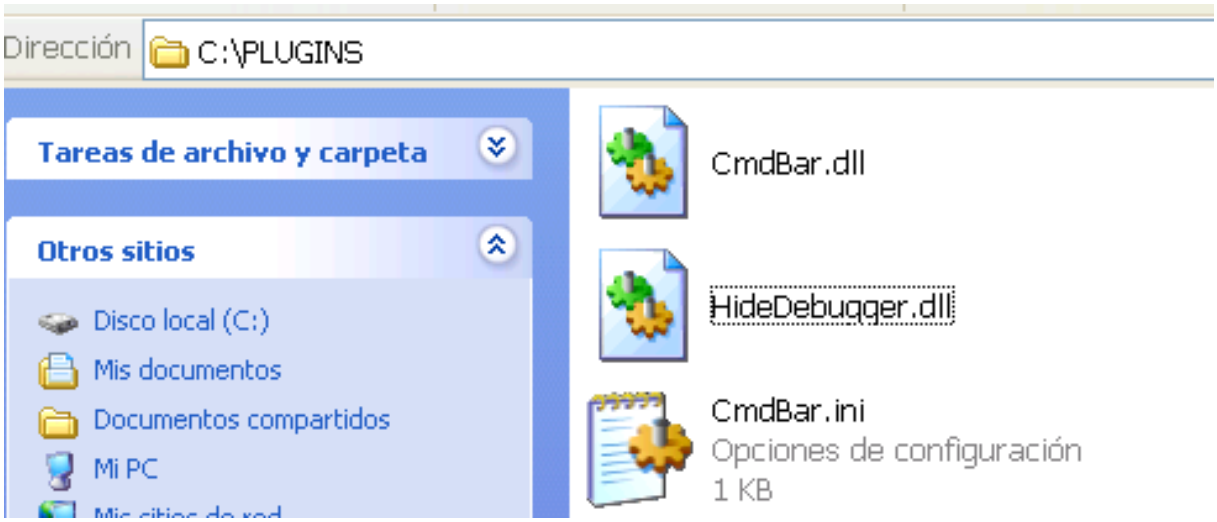
И поэтому выполняется без проблем.

Разумеется, есть множество плагинов, которые делают то же самое, один из них – это HideDebugger 1.24.

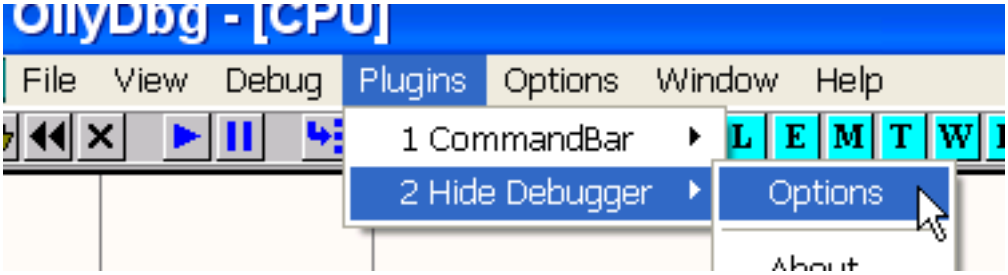
Сначала скачиваем его, затем копируем dll в директорию с плагинами.



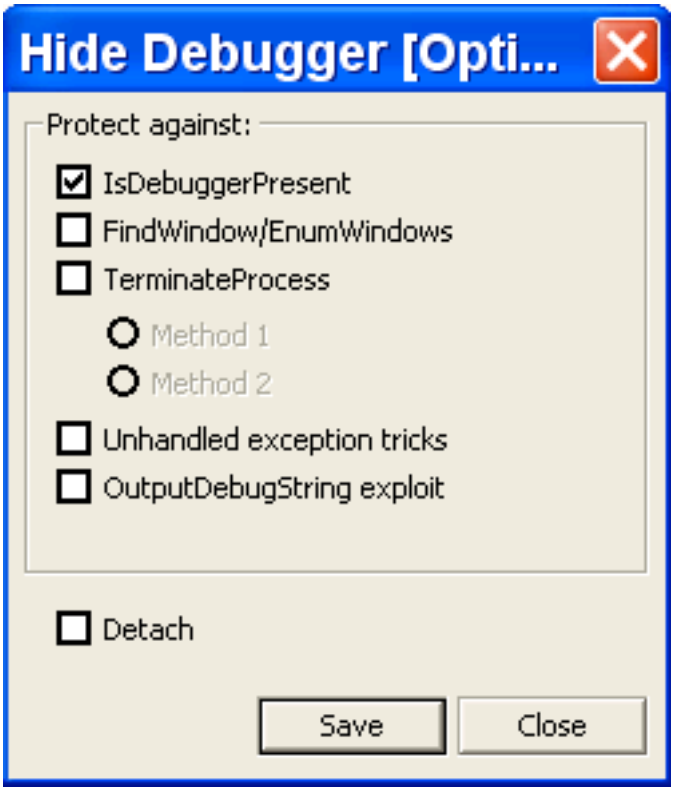
Копируем, хехе.



Теперь перезапускаем OllyDbg.

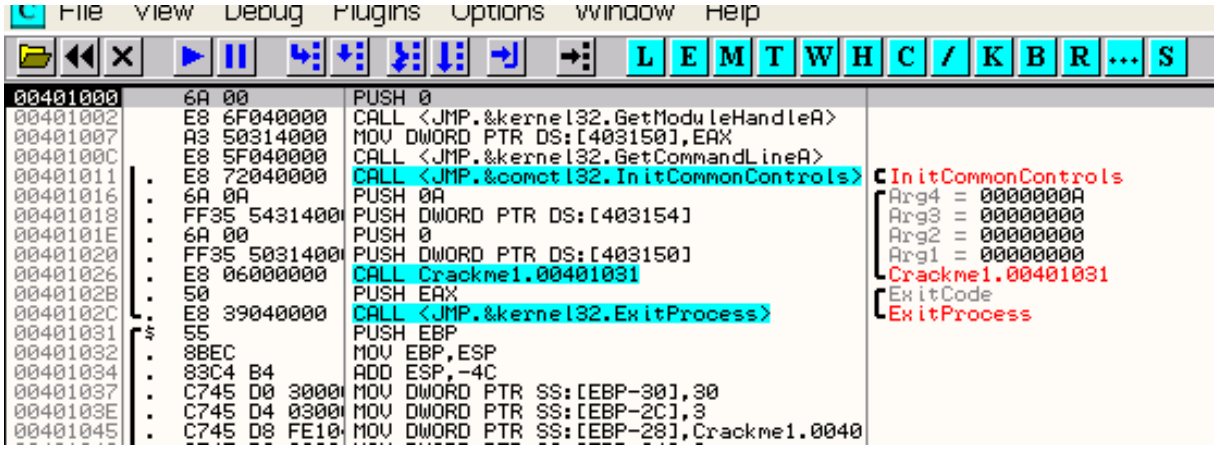


Смотрим настройки плагина.

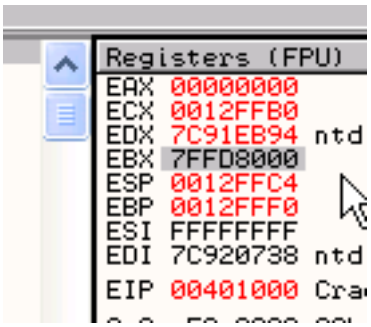


Видим, что уже отмечена опция против IsDebuggerPresent, то есть плагин делает за нас работу, которую мы делали руками. Остальные опции пока оставляем неотмеченными, когда до них очередь, мы их подробно изучим.

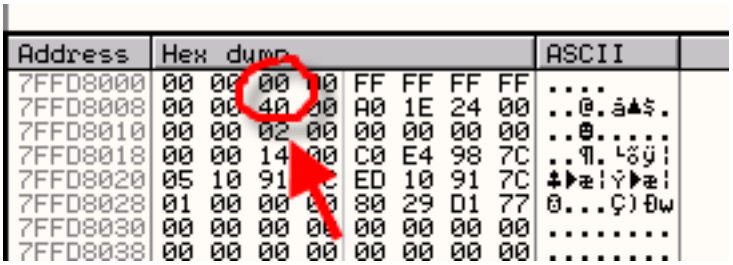
Нажимаем SAVE.



Запускаем по новой крэкми . Обратите внимание, что EBX указывает на область, где находится тот самый байт.



Теперь EBX-FOLLOW IN DUMP.



И видим, что всё, что делает плагин – это помещает ноль в этот байт, преодолевая таким образом данный тип защиты. Можно было бы сразу это сделать, но я считаю, что очень важно понимать, как это всё работает на самом деле.

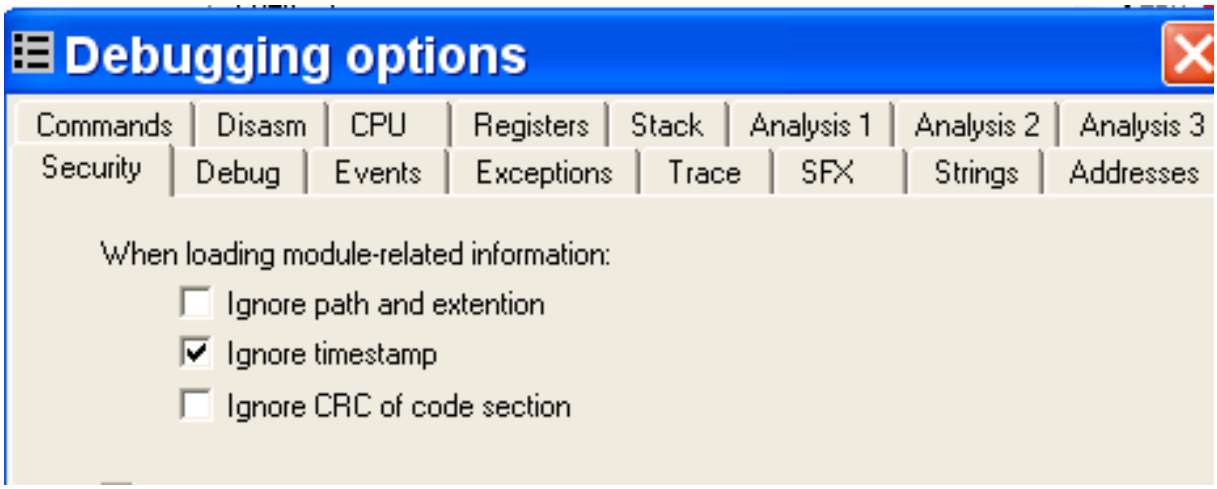
Введение в крэкинг с нуля, используя OllyDbg - Глава 20

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-01-27

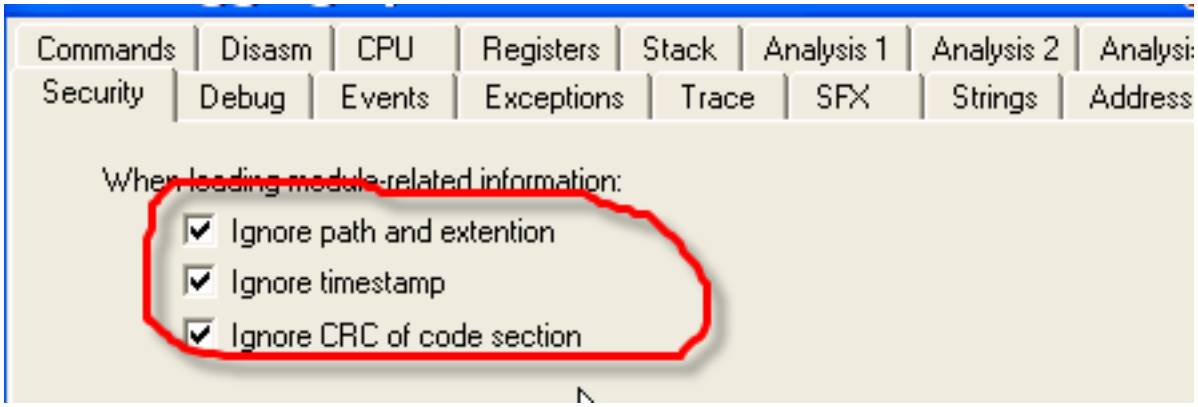
Следующий приём против отладки в OllyDbg, который мы рассмотрим, это обнаружение её по имени процесса, но прежде всего настроим OllyDbg так, как это необходимо в данном случае.

Если перейдём в DEBUGGING OPTIONS-SECURITY:



Здесь находится три галочки. Если мы их все отметим, то увидим, что установив BPX на API-функцию и если переинициализируем OllyDbg (Restart), то BPX останутся установленными, что позволяет нам избежать повторение всего процесса каждый раз, когда мы делаем рестарт.

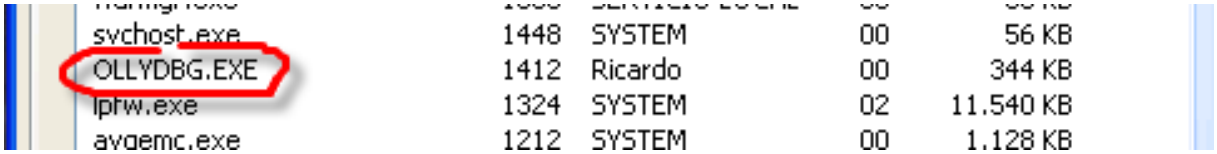
Строго говоря, я не знаю, как именно работает OllyDbg «внутри», но на практике достаточно просто отметить эти три галочки, чтобы установленные на API-функции BPX оставались установленными после перезапуска.



Именно так, как мне кажется, без особых проблем и хлопот, мы и начинаем изучение темы антиотладки по имени процесса.

Обнаружение OllyDbg по имени процесса

Запустив OllyDbg, мы можем посмотреть список процессов с помощью CTRL + ALT + SUPR.

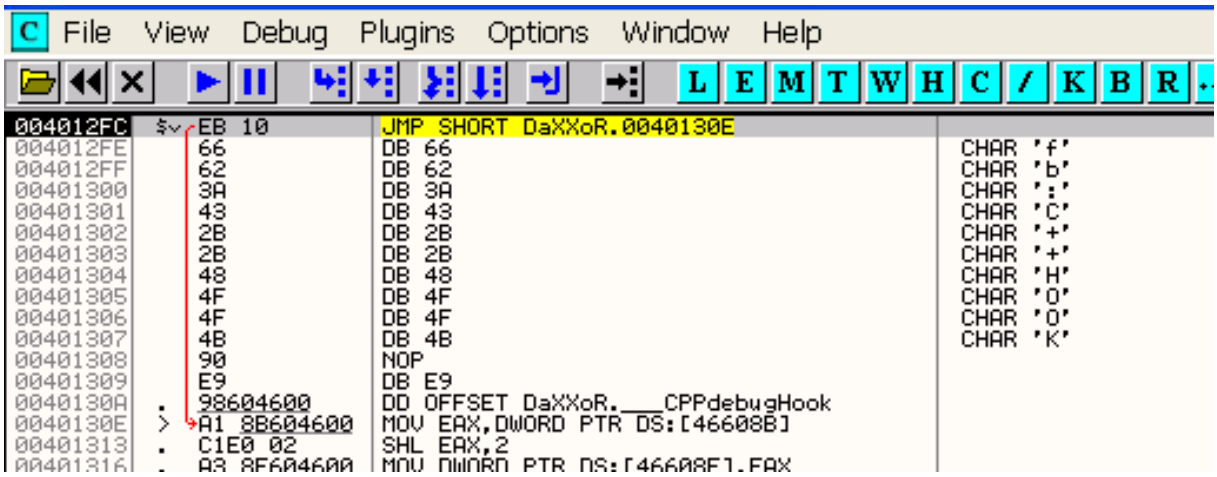


Видим, что имя процесса находится среди остальных, что позволяет программе пробежаться по их списку, и если она встретит называющийся OllyDbg, она закрывается, оставив противника ни с чем, ха-ха!

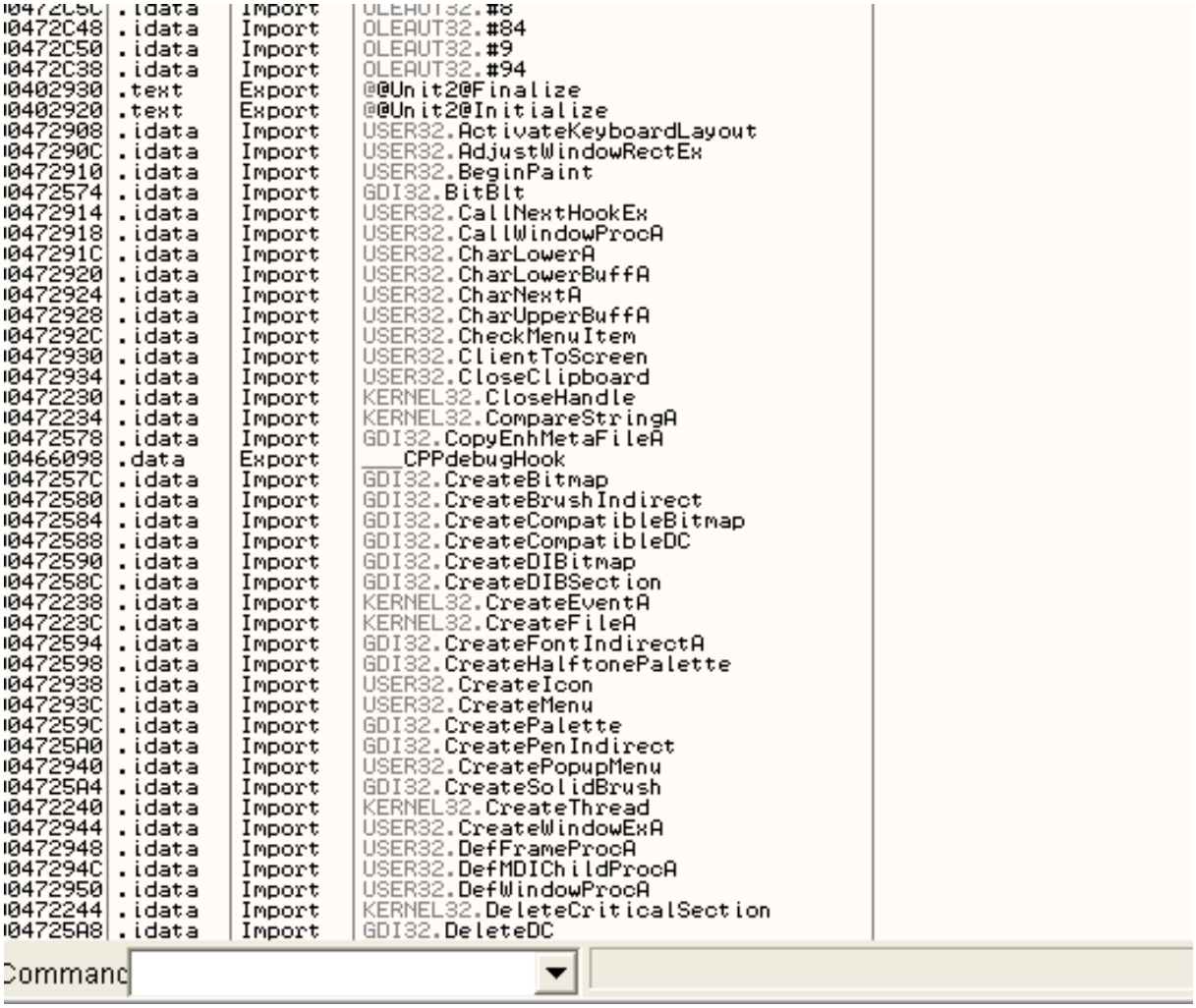
Крэкми, который мы используем, решать пока не будем, так как он находится на более продвинутом уровне, чем материал, изученный ранее – мы сделаем это в другой раз, но на его примере мы изучим то, как программы находят OllyDbg по имени процесса, как этого избежать вручную и автоматически, хе-хе.

Приложенный к статье крэкми (находится в архиве `wasm_files.zip`: `files/0474/daxxor.zip`) называется DAXXOR. Если запустим его и OllyDbg по отдельности, то увидим, что крэкми запускается, но закрывает отладчик, а если загрузим программу в OllyDbg, то закроются оба.

Изучим, как это происходит. Откроем крэкми в OllyDbg.

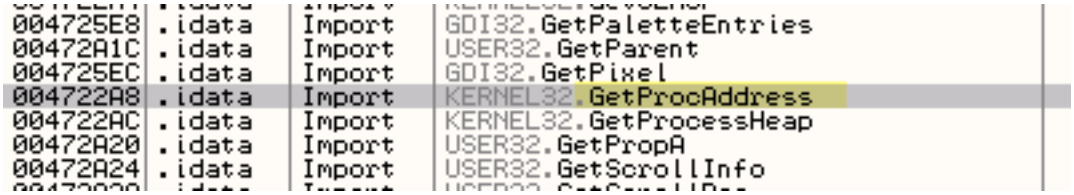


Оказавшись в точке входа, посмотрим, какие API-функции используются.

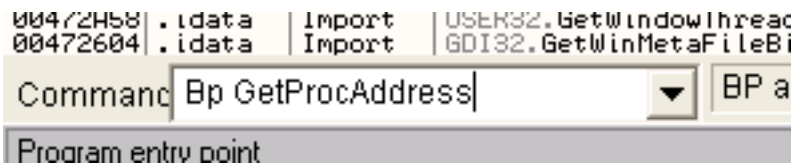


Ок, их тут немало, но дело в том, что они используются защитой, а не самим крэкми, который сам в начале не грузится, поэтому API-функции, используемые для обнаружения OllyDbg, мы сможем посмотреть только в середине выполнения. Заодно я объясню этот способ защиты, который не позволяет нам увидеть список API-функций. С другой стороны, теперь мы точно знаем, что эти среди них есть важные API-функции, раз программа их скрывает и они не появляются в списке.

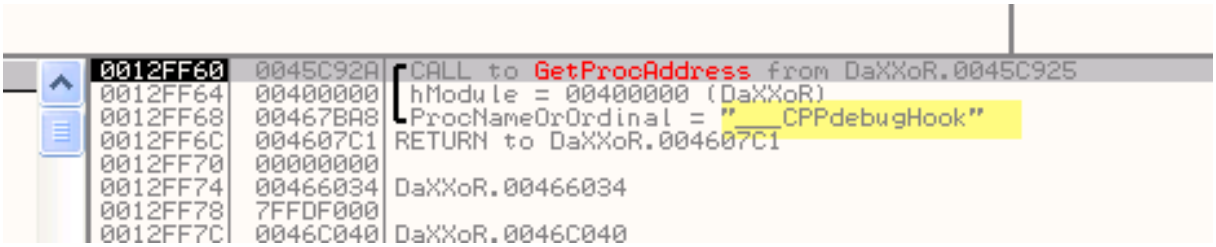
Почти в каждом таком случае, когда программа не запакована, как здесь, используется API-функция GetProcAddress.



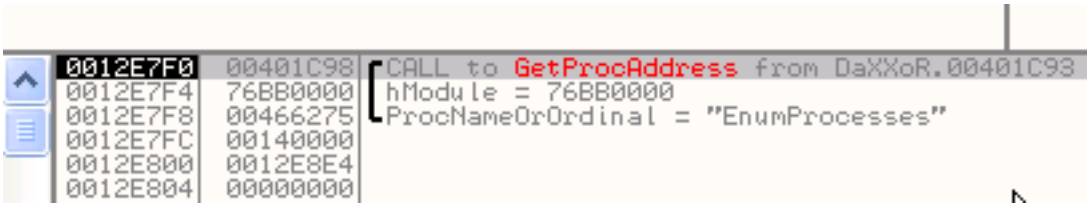
Она используется программой, чтобы загружать другие API-функции, которые не находятся в списке используемых. Мы более подробно рассмотрим GetProcAddress в главе, посвящённой распаковке, но пока просто установим на неё BP.



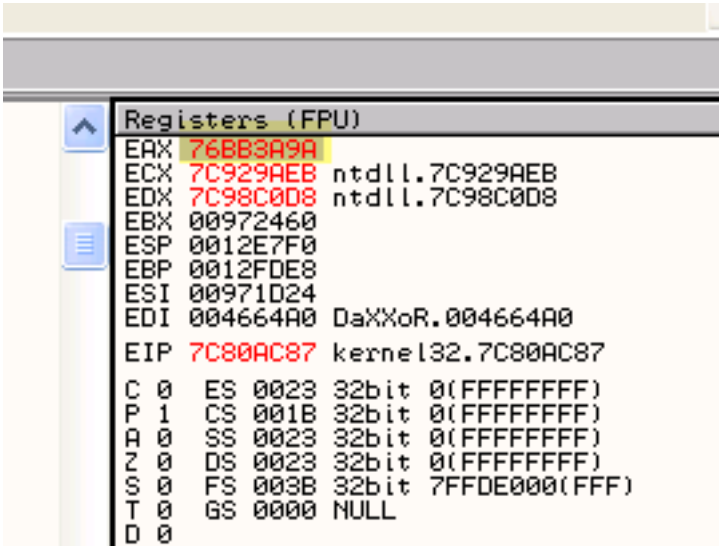
И делаем RUN.



Каждый раз, останавливаясь на ней, видим, что именно запрашивается, в данном случае первой запрашивается функция __CPPdebugHook, не связанная с защитой, которую мы изучаем, поэтому снова жмём RUN.

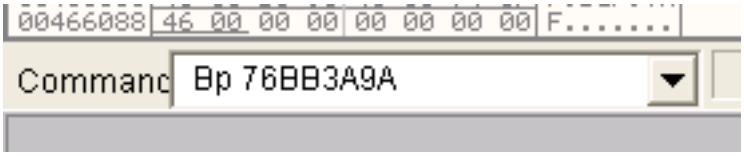


Таким образом, с помощью F9 доходим до API-функций, относящихся к защите. Здесь видим, что используется функция EnumProcesses, так что после того, как дойдём до RET в GetProcAddress, в EAX будет возвращен адрес функции, действительный для конкретной машины, и туда мы установим BPX, после чего выполним RET с помощью EXECUTE TILL RETURN.

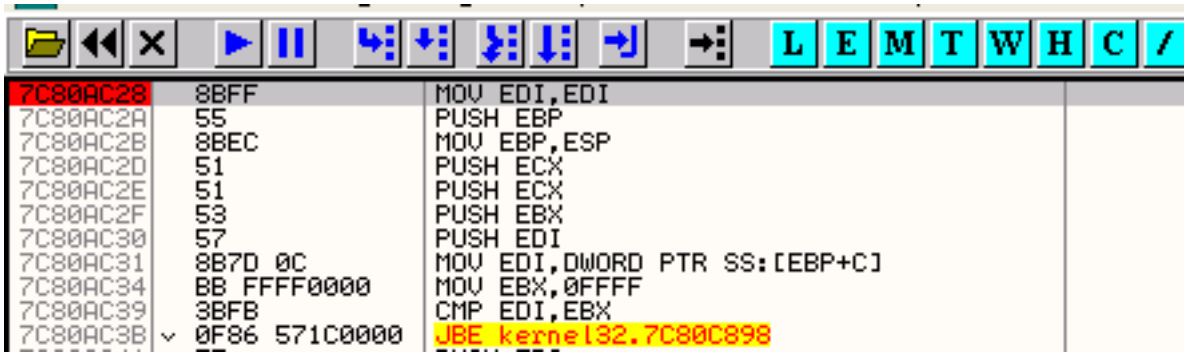


Здесь в EAX возвращается адрес указанной функции для конкретной машины, в моём случае это 76BB3A9A (он может отличаться у вас).

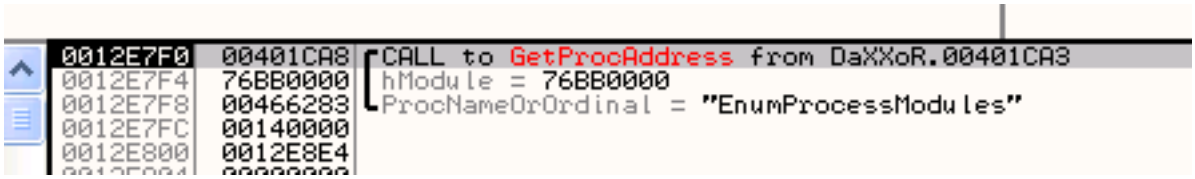
С другой стороны, видим, что OllyDbg не обнаружила эту API-функцию, и её нет в списке NAMES, поэтому мы не можем установить BP напрямую по имени функции, и поэтому устанавливаем точку останова на адрес.



Вот она установилась.

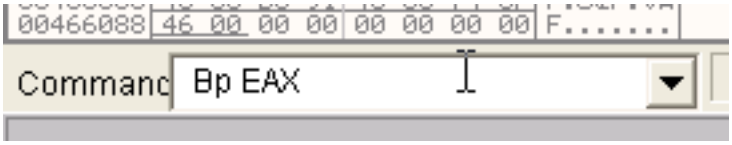


Итак, мы установили BP на подозрительную API-функцию, продолжаем выполнять программу в OllyDbg, чтобы увидеть, загружается ли ещё что-нибудь.

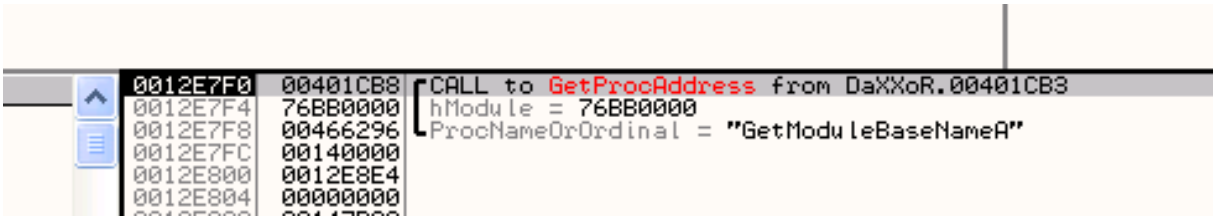


Хм, перебираются модули процесса, дойдём до RET и установим BPX на адрес, который будет возвращён в EAX.

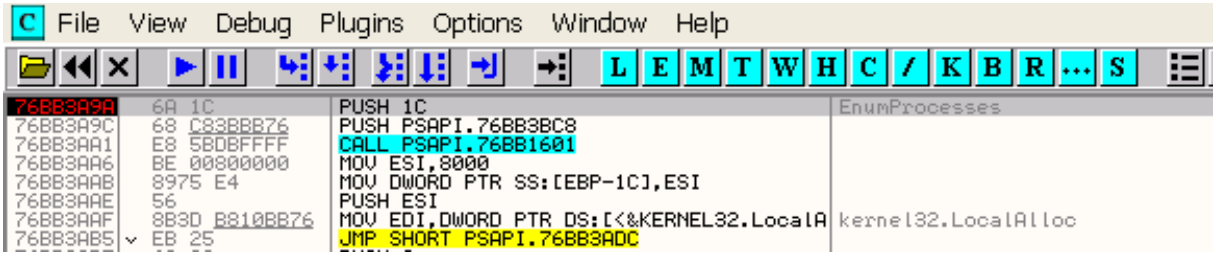
Чтобы много не печатать, то находясь на RET, можем сделать так:



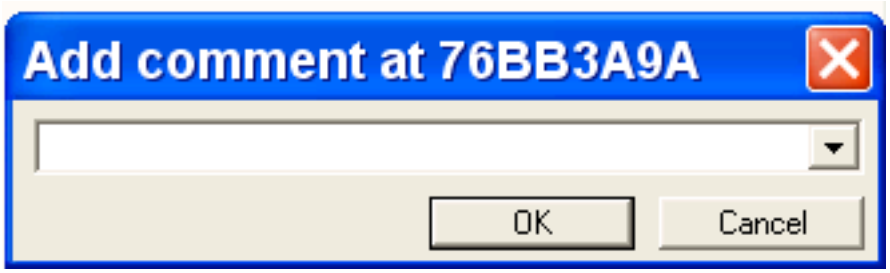
Так я устанавливаю BPX на все API-функции, когда нахожусь на RET.

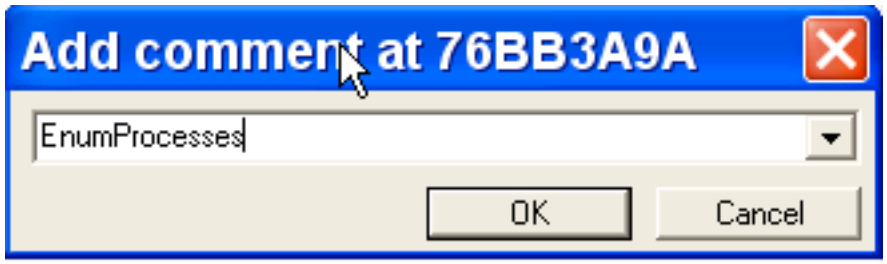


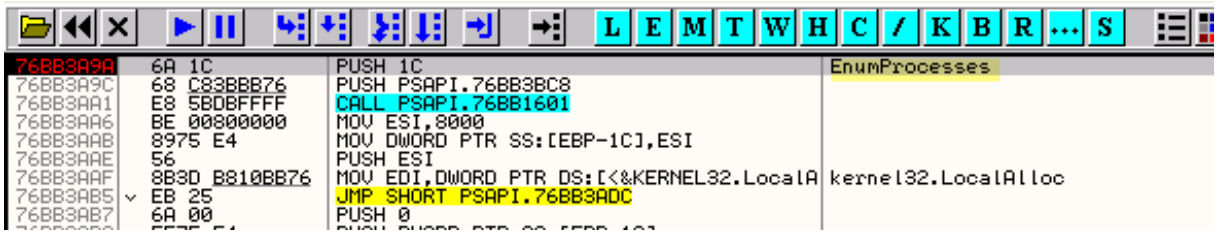
Ещё одна подозрительная API-фукнция, устанавливаем на неё BP таким же образом, что и в прошлые разы, делаем и останавливаемся на EnumProcesses.



Комментарий справа, содержащий имя API, был добавлен мной. Комментарии можно добавлять с помощью двойного щелчка на нужной области.





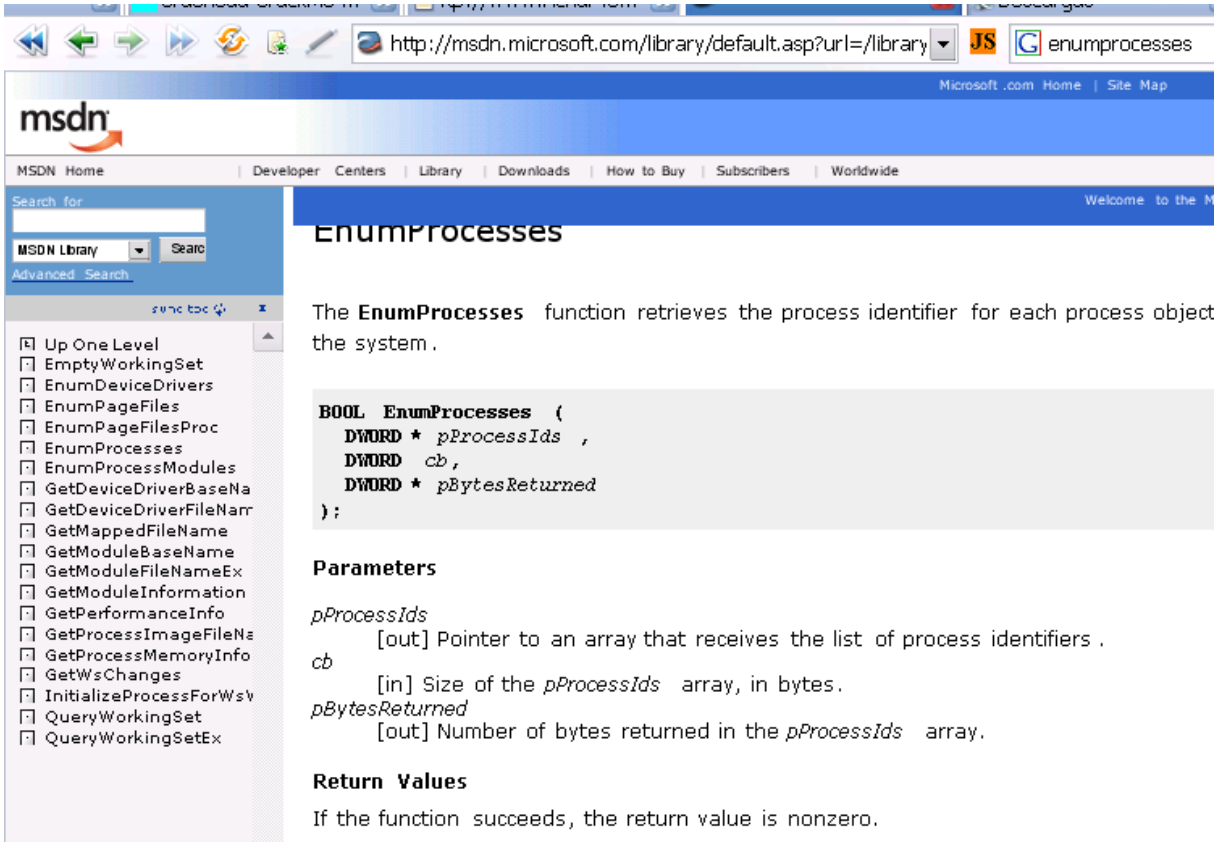


Все эти функции, загруженные программой, не были увидены OllyDbg и не были отражены в списке, также она не даёт по ним какую-либо информацию.

Поищем в гугле, что это за функция “EnumProcess”.

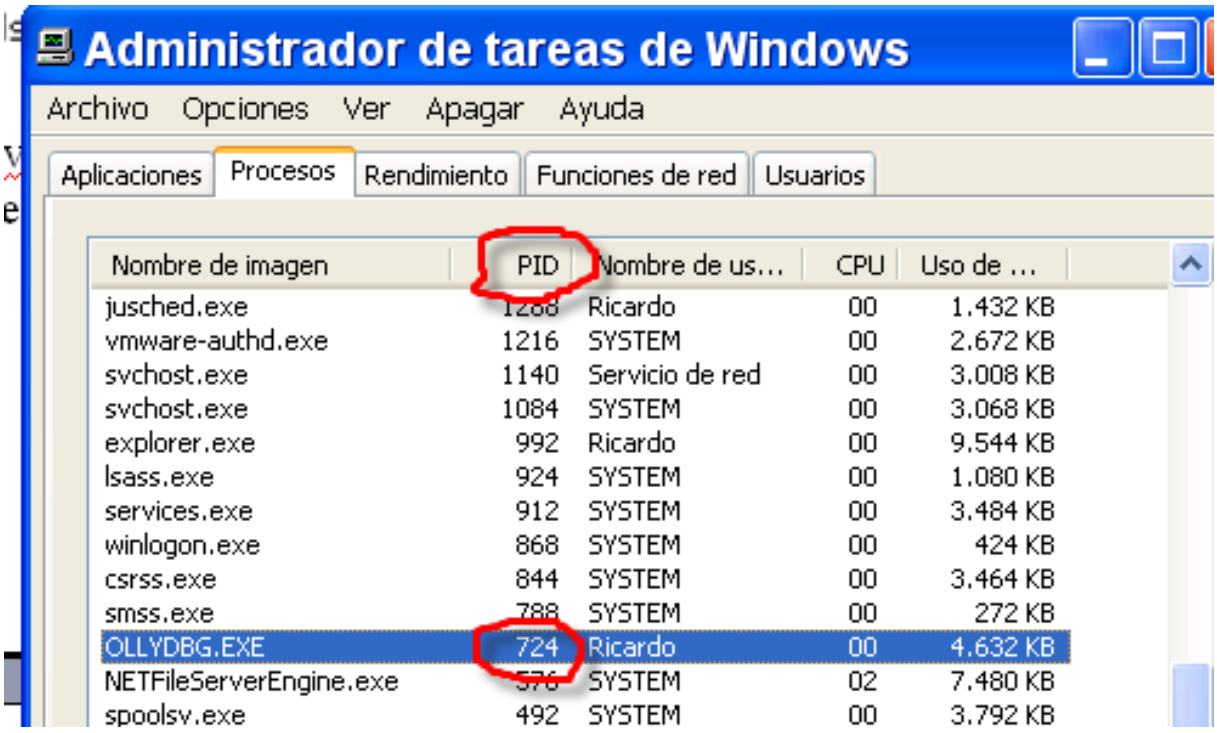
Находим её на сайте Микрософта, адрес страницы:

http://msdn2.microsoft.com/en-us/library/ms682629.aspx

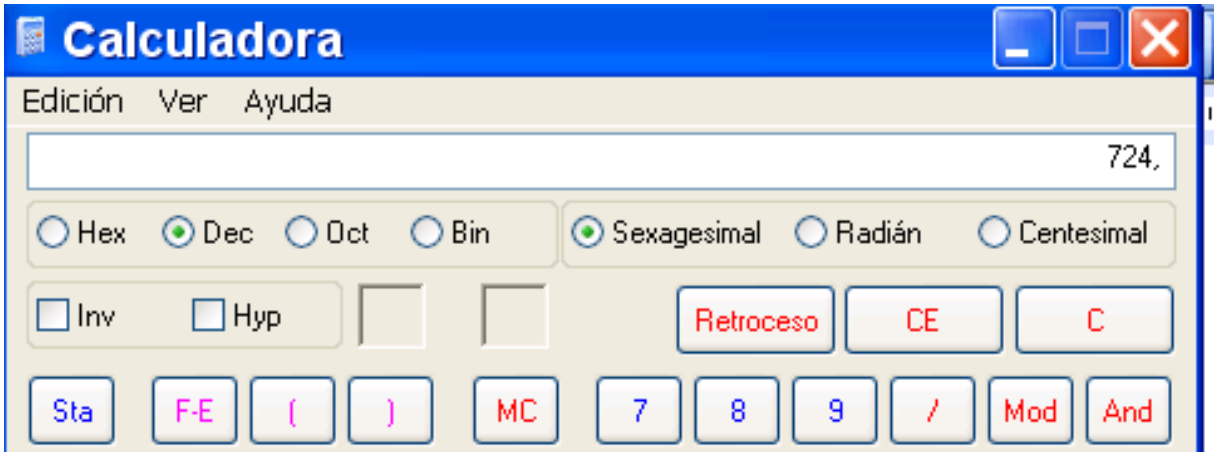


Хорошо, здесь говорится, что вышеупомянутая функция возвращает нам идентификатор или PID каждого запущенного процесса, ок, рассмотрим что такое этот PID, хе-хе.

Каждому запущенному процессу присвоен идентификатор – число, меняющееся каждый раз при запуске процесса. Посмотрим на список процессов.



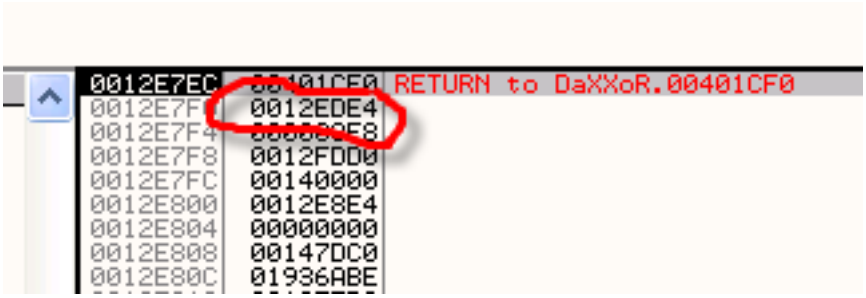
Видим, что в данном случае OllyDbg присвоен PID 724 – в десятеричной системе счисления, так как программа просмотра процессов работает именно с ней. Переведём вышеуказанное число в шестнадцатиричную систему с помощью калькулятора Windows.



Нажимаем на кнопку HEX, чтобы перейди в шестнадцатеричную систему.

PID OllyDbg в шестнадцатеричной системе равен 2D4. Можем закрыть и запустить по новой OllyDbg, и увидим, что PID изменился. Каждый раз, когда процесс запускается заново – PID меняется.

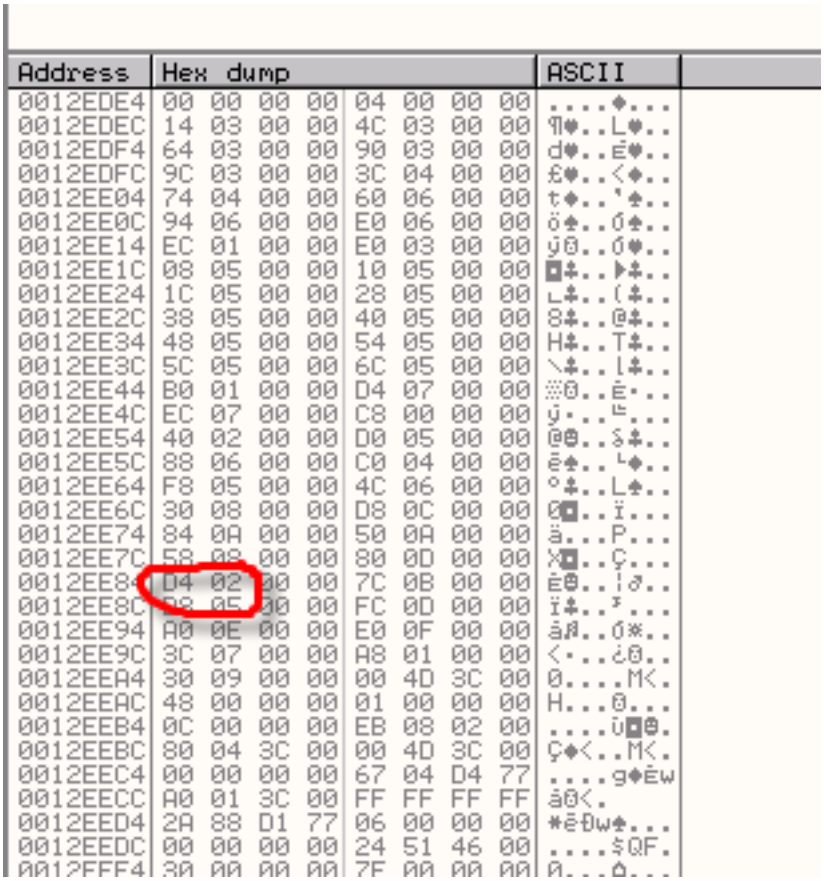
Также нам не повезло с отображением параметров функции, так как для OllyDbg она не существует.



Из страницы на сайте Микрософта, знаем, что у неё три параметра.

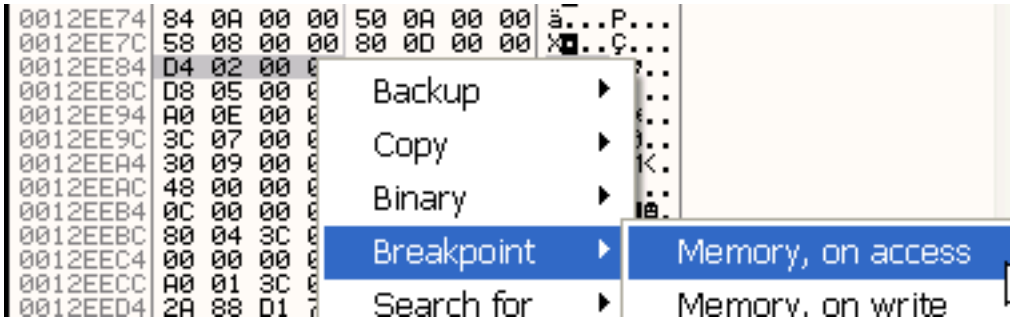
pProcessIds
[out] Pointer to an array that receives the list of process identifiers.
cb
[in] Size of the *pProcessIds* array, in bytes.
pBytesReturned
[out] Number of bytes returned in the *pProcessIds* array.

То есть в 12EDE4 сохраняется список PID’ов всех процессов, запущенных на моей машине, делаем EXECUTE TILL RETURN, чтобы вернуться из API-функции и смотрим через DUMP, что там лежит.

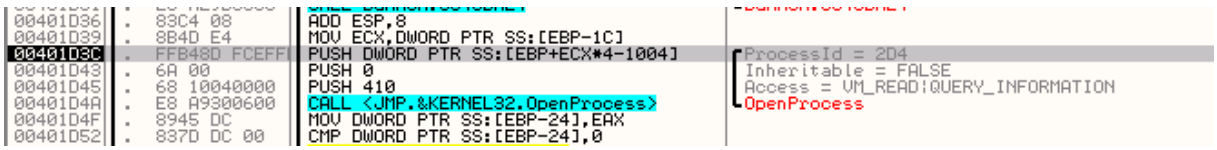


Там находится список PID’ов, а среди них и PID OllyDbg, хнык, хнык.

Ставим здесь BPM ON ACCESS, чтобы посмотреть, где он используется.



Теперь делаем RUN.



Видим, что остановились там, где будет применена функция OpenProcess, которая проверяет, работает ли процесс, и если да, то возвращает его хэндл (логический номер).

Какая разница между PID и хэндлом? Это очень просто – PID – это общий идендификатор – на всей машине, в какой угодно программе PID OllyDbg остаётся одним и тем же, пока процесс не будет перезапущена, в моём случае это 2d4, а хэндл – это номер, который позволяет позволяет вашей программе управлять процессом и который будет отличаться в другой программе. Это своего рода

заявка на контролирование процесса, без номера не сможете им управлять, а чтобы получить номер, нужно попросить систему, которая вернёт вам хэндл. Тогда сможете и управлять и подстроить какую-нибудь гадость, хе-хе.

Вот более подробное определение OpenProcess в WinApis32, у неё тоже много параметров, но нам они не интересны.

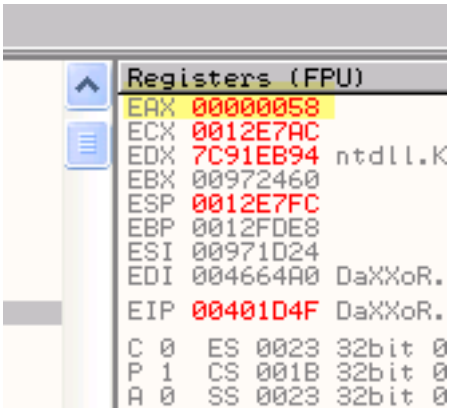
Return Values

If the function succeeds, the return value is an open handle of the specified process.

If the function fails, the return value is NULL. To get extended error information, call **GetLastError**.

То есть возвращается логический номер исполняющегося процесса, коротко говоря, это то, что программа в данном случае и хочет узнать.

Трассируем с помощью F8, пока не остановимся на API-функции.

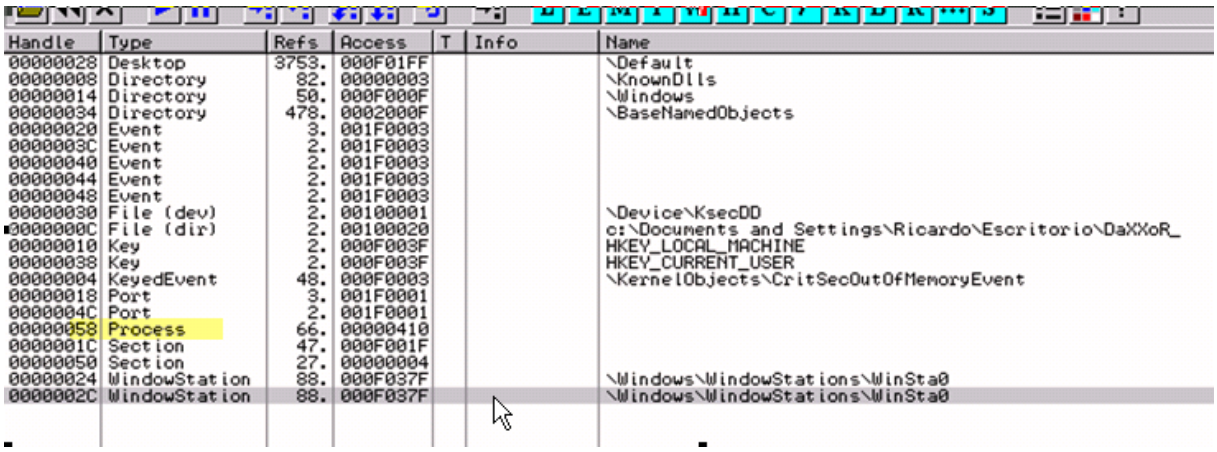


Y en EAX devuelve el handle o manejador del OLLYDBG que en mi caso es 58.

В EAX возвращается логический номер OllyDbg, который в моём случае равен 58.

El mismo OLLYDBG nos muestra los HANDLES con los cuales esta trabajando el programa en la ventana H.

Тот же OllyDbg умеет показывать логические номера, с которыми программа работает в настоящее время, с помощью окна H.



Handle	Type	Refs	Access	T	Info	Name
00000028	Desktop	3753.	000F01FF			\Default
00000008	Directory	82.	00000003			\KnownDlls
00000014	Directory	50.	000F000F			\Windows
00000034	Directory	478.	0002000F			\BaseNamedObjects
00000020	Event	3.	001F0003			
0000003C	Event	2.	001F0003			
00000040	Event	2.	001F0003			
00000044	Event	2.	001F0003			
00000048	Event	2.	001F0003			
00000030	File (dev)	2.	00100001			\Device\KsecDD
0000000C	File (dir)	2.	00100020			c:\Documents and Settings\Ricardo\Escritorio\DaXXoR_
00000010	Key	2.	000F003F			HKEY_LOCAL_MACHINE
00000038	Key	2.	000F003F			HKEY_CURRENT_USER
00000004	KeyedEvent	48.	000F0003			\KernelObjects\CritSecOutOfMemoryEvent
00000018	Port	3.	001F0001			
0000004C	Port	2.	001F0001			
00000058	Process	66.	00000410			
0000001C	Section	47.	000F001F			
00000050	Section	27.	00000004			
00000024	WindowStation	88.	000F037F			\Windows\WindowStations\WinSta0
0000002C	WindowStation	88.	000F037F			\Windows\WindowStations\WinSta0

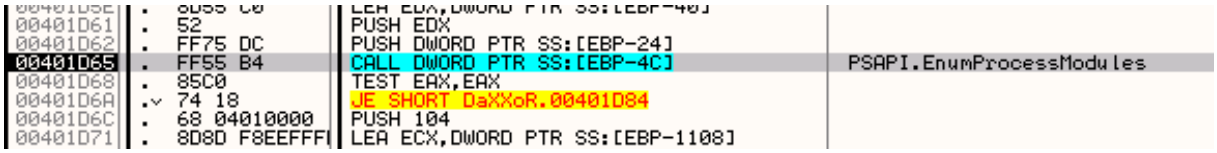
Видим, что здесь есть 58 и TYPE или тип является PROCESS, т.е. процесс, таким образом, у программы есть логический номер 58, действительный в процессе OllyDbg.

Если другой процесс использует EnumProcess, чтобы получить PID, то для OllyDbg, если она не будет перезапущена, он будет равен 2d4, а если потом попросить логический номер данного

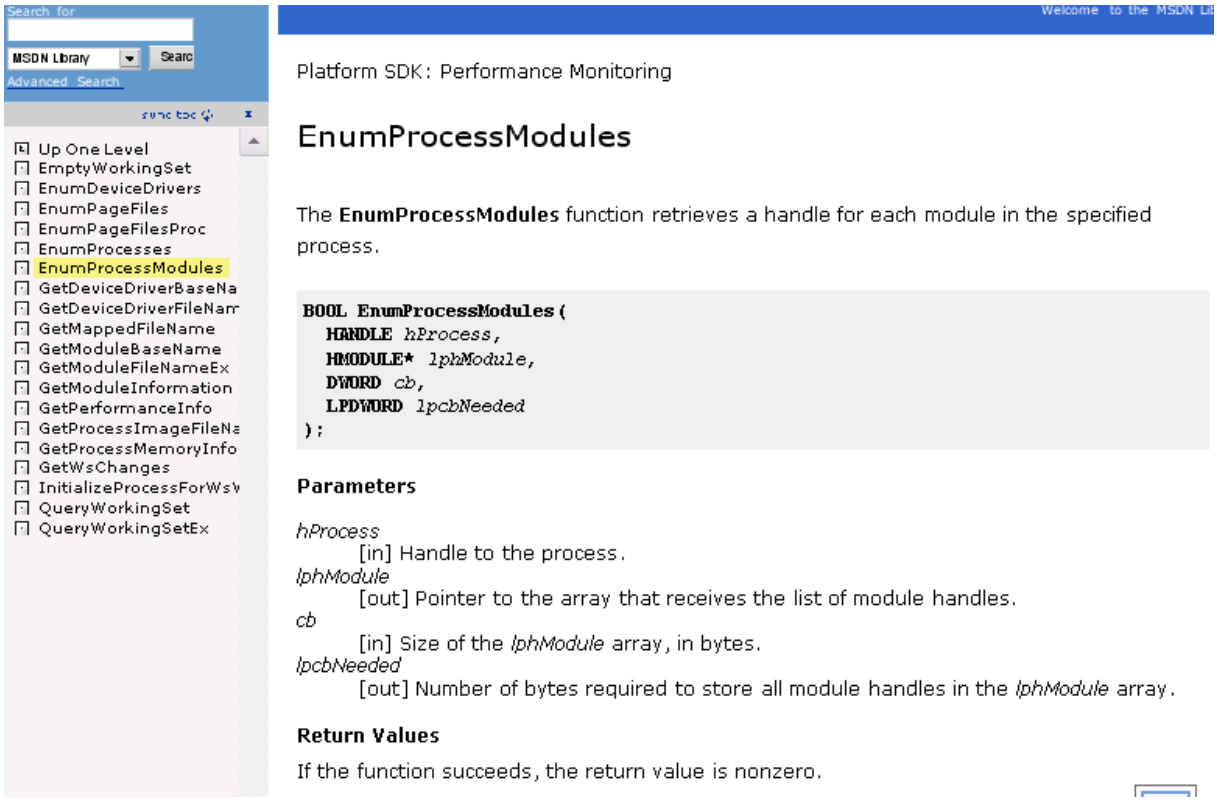
процесса, то он будет каким-нибудь другим, так как хэндлы для каждого процесса, их запрашивающих, уникальны.

В данном случае, опасность для OllyDbg заключается в том, что у программы уже есть логический номер, с которым она может делать всё, что хочет. Хотя она и не знает, что выполняется из под OllyDbg, она может проверить имя процесса, чтобы определить, не называется ли он OllyDbg, и очевидно, что для этого будут перебираться все PID'ы, полученные в списке процессов, поэтому бросаем всё и отправляемся туда, где это будет происходить, для чего установим BPM ON ACCESS на соответствующий PID.

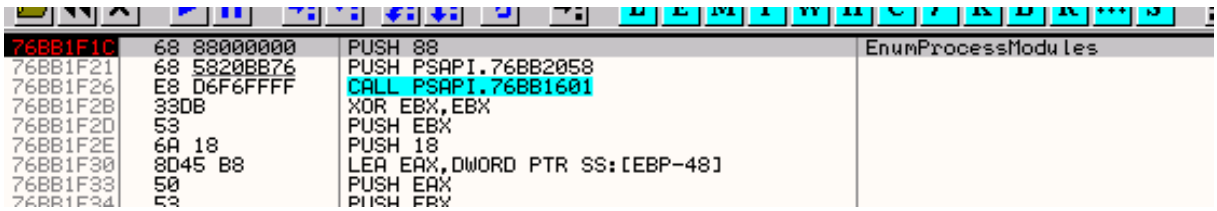
Продолжаем трассировать с помощью F8.



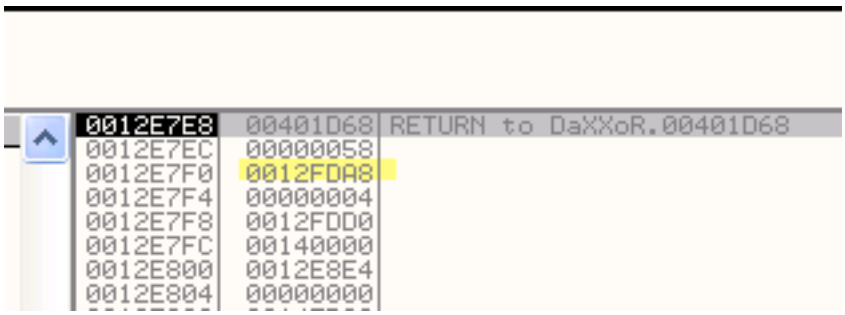
Дошли до ещё одной API-функции, которую от нас пытались скрыть, – EnumProcessModules. Посмотрим её описание на том же сайте, что и ранее.



То есть будет проводиться поиск в исследуемом процессе, получаем список модулей, используемых им. Продолжаем трассировать с помощью F7.

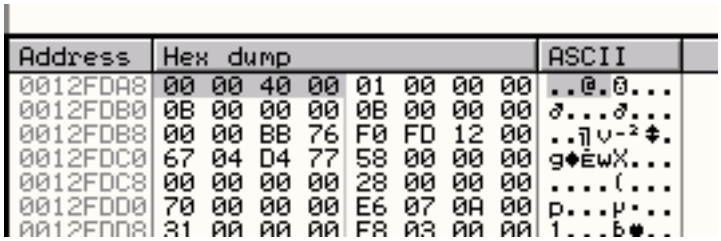


В стеке видим параметры:

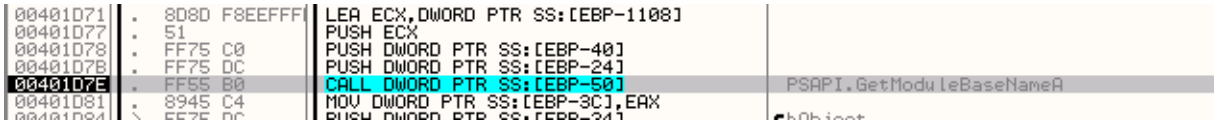


Список хэндлов всех модулей сохранён по выделенному адресу, а первым параметром, как мы видим, является логический номер процесса OllyDbg.

Здесь нужно прояснить одну вещь: когда запрашиваем логические номера модулей, система возвращает нам адресную базу, или откуда начинается указанный процесс в памяти. В данном случае нам было возвращёно 400000, так как именного отсюда начинается процесс OllyDbg.



Продолжаем трассировать и находим ещё одну API-функцию.



GetModuleBaseNameA

GetModuleBaseName

The **GetModuleBaseName** function retrieves the base name of the specified module.

```
DWORD GetModuleBaseName(
  HANDLE hProcess,
  HMODULE hModule,
  LPTSTR lpBaseName,
  DWORD nSize
);
```

Parameters

hProcess
[in] Handle to the process that contains the module. If this parameter is NULL, **GetModuleBaseName** uses the current process.
The handle must have the PROCESS_QUERY_INFORMATION and PROCESS_VM_READ access rights. For more information, see Process Security and Access Rights.

hModule
[in] Handle to the module. If this parameter is NULL, this function returns the name of the file used to create the calling process.

lpBaseName
[out] Pointer to the buffer that receives the base name of the module. If the base name is longer than maximum number of characters specified by the *nSize* parameter, the base name is truncated.

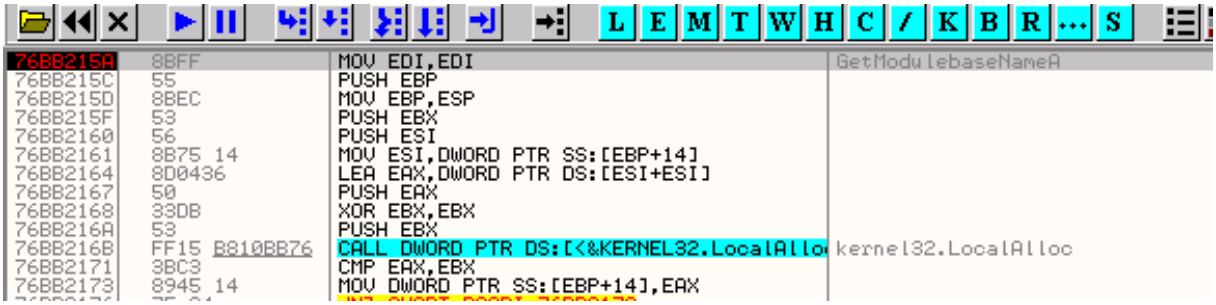
nSize
[in] Size of the *lpBaseName* buffer, in characters.

Return Values

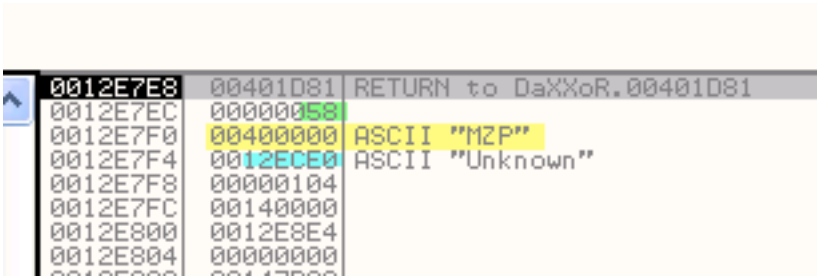
If the function succeeds, the return value specifies the length of the string copied to the buffer, in characters.

If the function fails, the return value is zero. To get extended error information, call **GetLastError**.

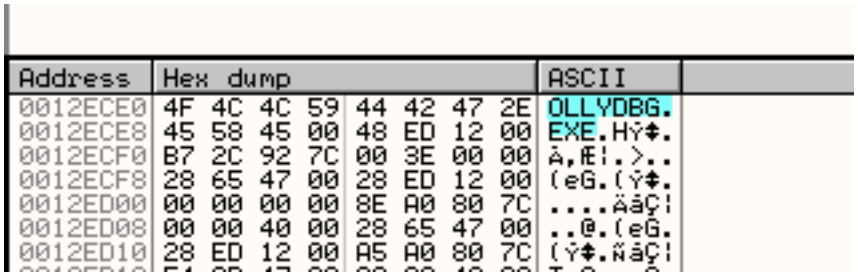
Т.е., что с помощью этой функции ходим найти имя модуля, так как эта API-функция помещает в буфер по адресу lpBaseName имя модуля, соответствующего адресу, выясненному ранее.



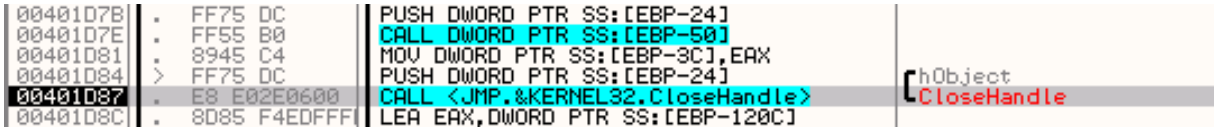
В стеке следующие параметры:



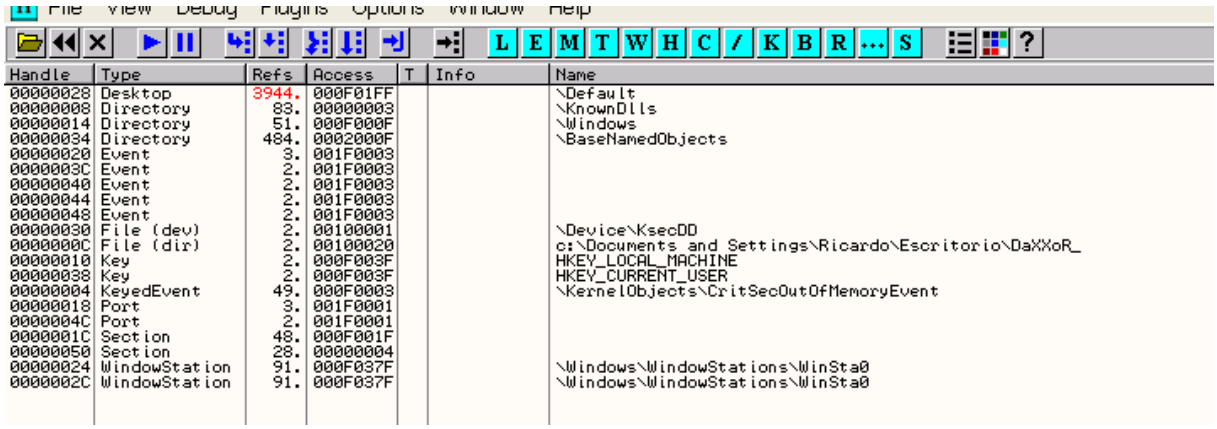
58 – это логический номер OllyDbg, 400000 – это база главного модуля, а буфер располагается по адресу 12ECE0, так что смотрим эту область памяти через дамп.



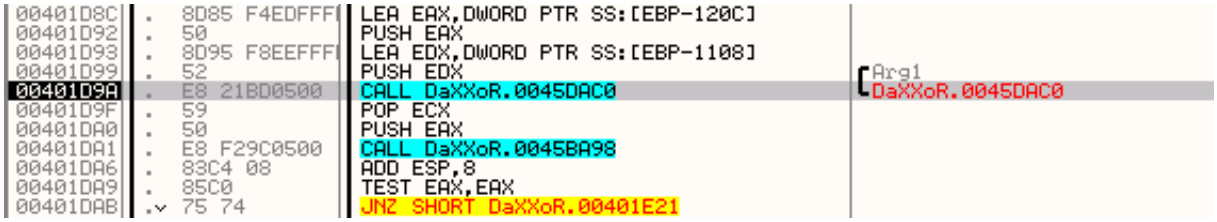
Хе-хе, уже понятно, что эта операция будет проведена для каждого процесса, который запущен в машине, чтобы узнать, не является ли его именем ‘OLLYDBG.exe’.



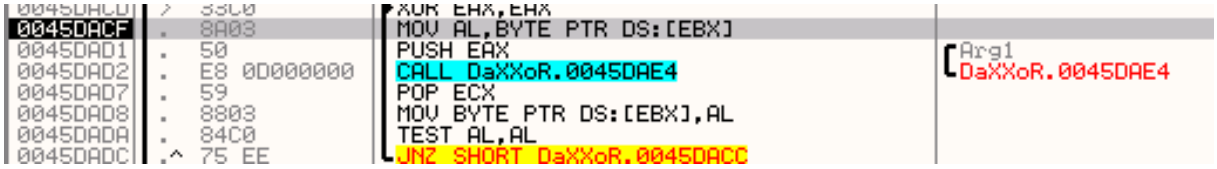
Доходим до CloseHandle, где закрывается логический номер, то есть 58 исчезает из их списка.

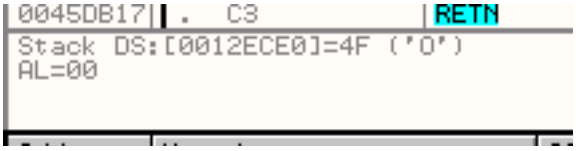


Ну да, теперь мы не можем делать никаких шалостей, так как логический номер закрыт, и мы больше не можем полагаться на него.



Дошли до CALL, входим в него, нажав F7.





Здесь считывается первая буква “OLLYDBG” (4F) и помещается в стек с помощью PUSH, затем идёт новый CALL, в который мы также заходим.

Видим, что в этом вызове не происходит ничего особенно важного, поэтому выходим и оказываемся рядом со следующим CALL'ом.

Заходим в него.

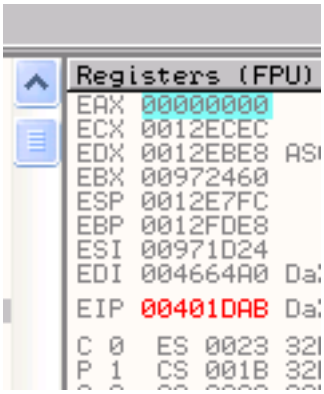
```
0045BA98  $  8B4C24 04    MOV ECX,DWORD PTR SS:[ESP+4]
0045BA9C  .  8B5424 08    MOV EDX,DWORD PTR SS:[ESP+8]
0045BAA0  .  53           PUSH EBX
0045BAA1  .  33C0         XOR EAX,EAX
0045BAA3  .  33DB         XOR EBX,EBX
0045BAA5  >  8A01         MOV AL,BYTE PTR DS:[ECX]
0045BAA7  .  8A1A         MOV BL,BYTE PTR DS:[EDX]
0045BAA9  .  2BC3         SUB EAX,EBX
0045BAAB  .v 75 34        JNZ SHORT DaXXoR.0045BAE1
0045BAAD  .  84DB         TEST BL,BL
0045BAAF  .v 74 30        JE SHORT DaXXoR.0045BAE1
0045BAB1  .  8A41 01      MOV AL,BYTE PTR DS:[ECX+1]
0045BAB4  .  8A5A 01      MOV BL,BYTE PTR DS:[EDX+1]
0045BAB7  .  2BC3         SUB EAX,EBX
0045BAB9  .v 75 26        JNZ SHORT DaXXoR.0045BAE1
0045BABB  .  84DB         TEST BL,BL
0045BABD  .v 74 22        JE SHORT DaXXoR.0045BAE1
0045BABF  .  8A41 02      MOV AL,BYTE PTR DS:[ECX+2]
0045BAC2  .  8A5A 02      MOV BL,BYTE PTR DS:[EDX+2]
0045BAC5  .  2BC3         SUB EAX,EBX
0045BAC7  .v 75 18        JNZ SHORT DaXXoR.0045BAE1
0045BAC9  .  84DB         TEST BL,BL
0045BACB  .v 74 14        JE SHORT DaXXoR.0045BAE1
0045BACD  .  8A41 03      MOV AL,BYTE PTR DS:[ECX+3]
0045BAD0  .  8A5A 03      MOV BL,BYTE PTR DS:[EDX+3]
0045BAD3  .  2BC3         SUB EAX,EBX
0045BAD5  .v 75 0A        JNZ SHORT DaXXoR.0045BAE1
0045BAD7  .  83C1 04      ADD ECX,4
0045BADA  .  83C2 04      ADD EDX,4
0045BADD  .  84DB         TEST BL,BL
0045BADF  .^ 75 C4        JNZ SHORT DaXXoR.0045BAA5
0045BAE1  >  5B           POP EBX
0045BAE2  .  C3           RETN
```

```
Registers (FPU)
EAX 00000000
ECX 0012ECE0 ASCII "OLLYDBG.EXE"
EDX 0012EBDC ASCII "OLLYDBG.EXE"
EBX 00000000
ESP 0012E7EC
EBP 0012FDE8
ESI 00971D24
EDI 004664A0 DaXXoR.004664A0
EIP 0045BAA5 DaXXoR.0045BAA5
C 0  ES 0023 32bit 0(FFFFFFFF)
```

Ах, где-то здесь сравнивается полученное имя процесса с ‘OLLYDBG.EXE’, и если они равны, то всё, капут.. хе-хе, сейчас они равны, посмотрим, что случится. Возвращаемся через RET.

```
.   50              PUSH EAX
.   E8 F29C0500     CALL DaXXoR.0045BA98
.   83C4 08         ADD ESP,8
.   85C0            TEST EAX,EAX
.v  75 74           JNZ SHORT DaXXoR.00401E21
.   C745 E0 0100(   MOV DWORD PTR SS:[EBP-20],1
.   8B4D E4         MOV ECX,DWORD PTR SS:[EBP-1C]
.   FFB48D FCEFF(   PUSH DWORD PTR SS:[EBP+ECX*4-1004]      ProcessId
.   6A 00           PUSH 0                                  Inheritable = FALSE
.   6A 01           PUSH 1                                  Access = TERMINATE
.   E8 31300600     CALL <JMP.&KERNEL32.OpenProcess>        OpenProcess
.   8945 DC         MOV DWORD PTR SS:[EBP-24],EAX
.   837D DC 00      CMP DWORD PTR SS:[EBP-24],0
.v  74 3F           JE SHORT DaXXoR.00401E0F
.   6A 00           PUSH 0                                  ExitCode = 0
.   FF75 DC         PUSH DWORD PTR SS:[EBP-24]              hProcess
.   E8 78300600     CALL <JMP.&KERNEL32.TerminateProcess>   TerminateProcess
.   85C0            TEST EAX,EAX
.v  74 17           JE SHORT DaXXoR.00401DF5
.   FF75 DC         PUSH DWORD PTR SS:[EBP-24]              hObject
.   E8 862E0600     CALL <JMP.&KERNEL32.CloseHandle>        CloseHandle
.   FF75 D0         PUSH DWORD PTR SS:[EBP-30]              hLibModule
.   E8 CC2E0600     CALL <JMP.&KERNEL32.FreeLibrary>        FreeLibrary
.   33C0            XOR EAX,EAX
.v  E9 AF020000     JMP DaXXoR.004020A4
>   FF75 DC         PUSH DWORD PTR SS:[EBP-24]              hObject
.   E8 6F2E0600     CALL <JMP.&KERNEL32.CloseHandle>        CloseHandle
.   FF75 D0         PUSH DWORD PTR SS:[EBP-30]              hLibModule
.   E8 B52E0600     CALL <JMP.&KERNEL32.FreeLibrary>        FreeLibrary
.   B8 5A020000     MOV EAX,25A
.v  E9 95020000     JMP DaXXoR.004020A4
>   FF75 D0         PUSH DWORD PTR SS:[EBP-30]              hLibModule
.   E8 A32E0600     CALL <JMP.&KERNEL32.FreeLibrary>        FreeLibrary
.   B8 5C020000     MOV EAX,25C
.v  E9 83020000     JMP DaXXoR.004020A4
>  →FF45 E4         INC DWORD PTR SS:[EBP-1C]
T taken
aXXoR.00401E21
```

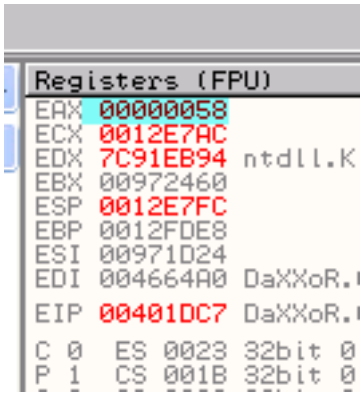
Если бы они не были равны, то EAX должен был бы отличаться от нуля, и был бы совершён переход, чего не происходит, если EAX равен нулю, как в данном случае.



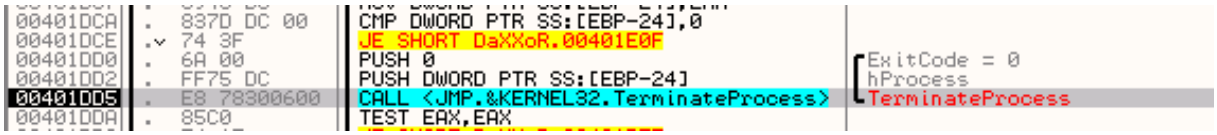
Перехода не происходит, и снова вызывается OpenProcess, чтобы получить логический номер процесса и убить последний.

```
00401DB4  .  8B4D E4         MOV ECX,DWORD PTR SS:[EBP-1C]
00401DB7  .  FFB48D FCEFF(   PUSH DWORD PTR SS:[EBP+ECX*4-1004]      ProcessId
00401DBE  .  6A 00           PUSH 0                                  Inheritable = FALSE
00401DC0  .  6A 01           PUSH 1                                  Access = TERMINATE
00401DC2  .  E8 31300600     CALL <JMP.&KERNEL32.OpenProcess>        OpenProcess
00401DC7  .  8945 DC         MOV DWORD PTR SS:[EBP-24],EAX
00401DCA  .  837D DC 00      CMP DWORD PTR SS:[EBP-24],0
```

Если выполним с помощью F8.



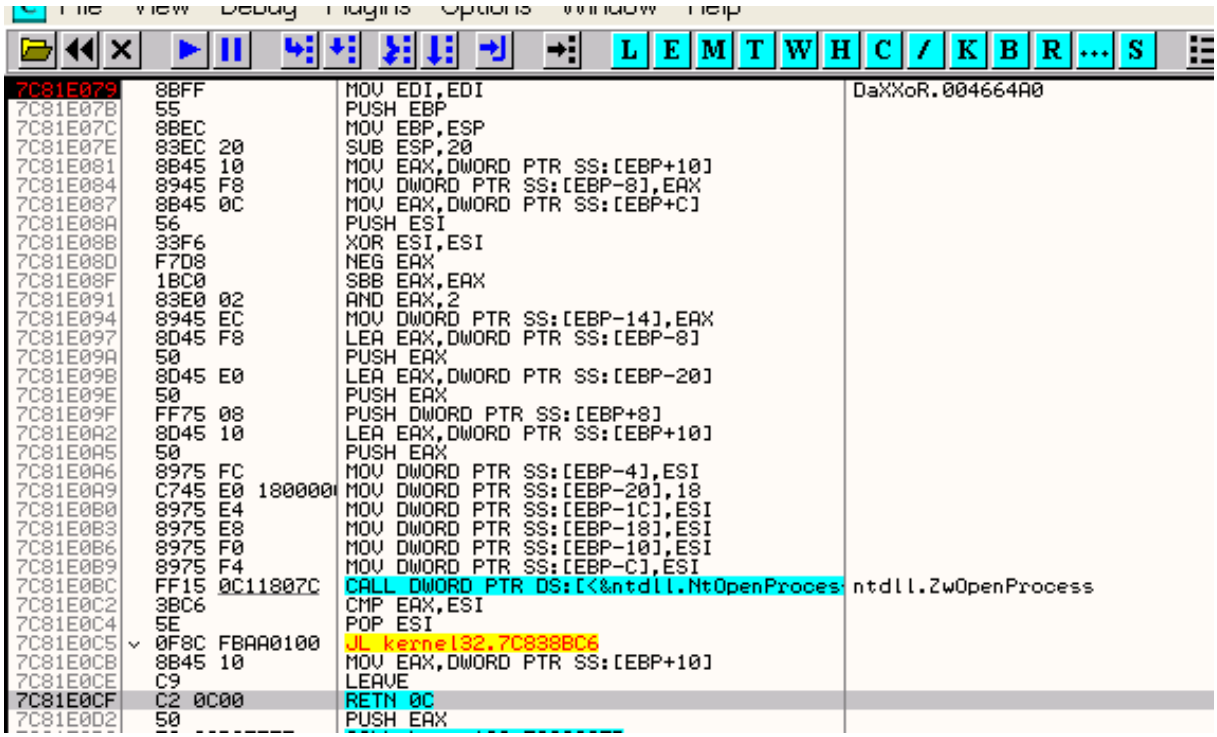
Система возвращает номер 58, так что продолжаем.



Как видим, API-функция TerminateProcess, которой передаётся логический номер 58, служит для того, чтобы убить OllyDbg.

И по нажатию F8 – до свиданья, Олли. Всё закрывается. Вот так мы и исследовали, как работает обнаружение по имени.

Ладно, я слегка устал от роли убийцы, хе-хе, перезапускаем OllyDbg и устанавливаем BP на OpenProcess.

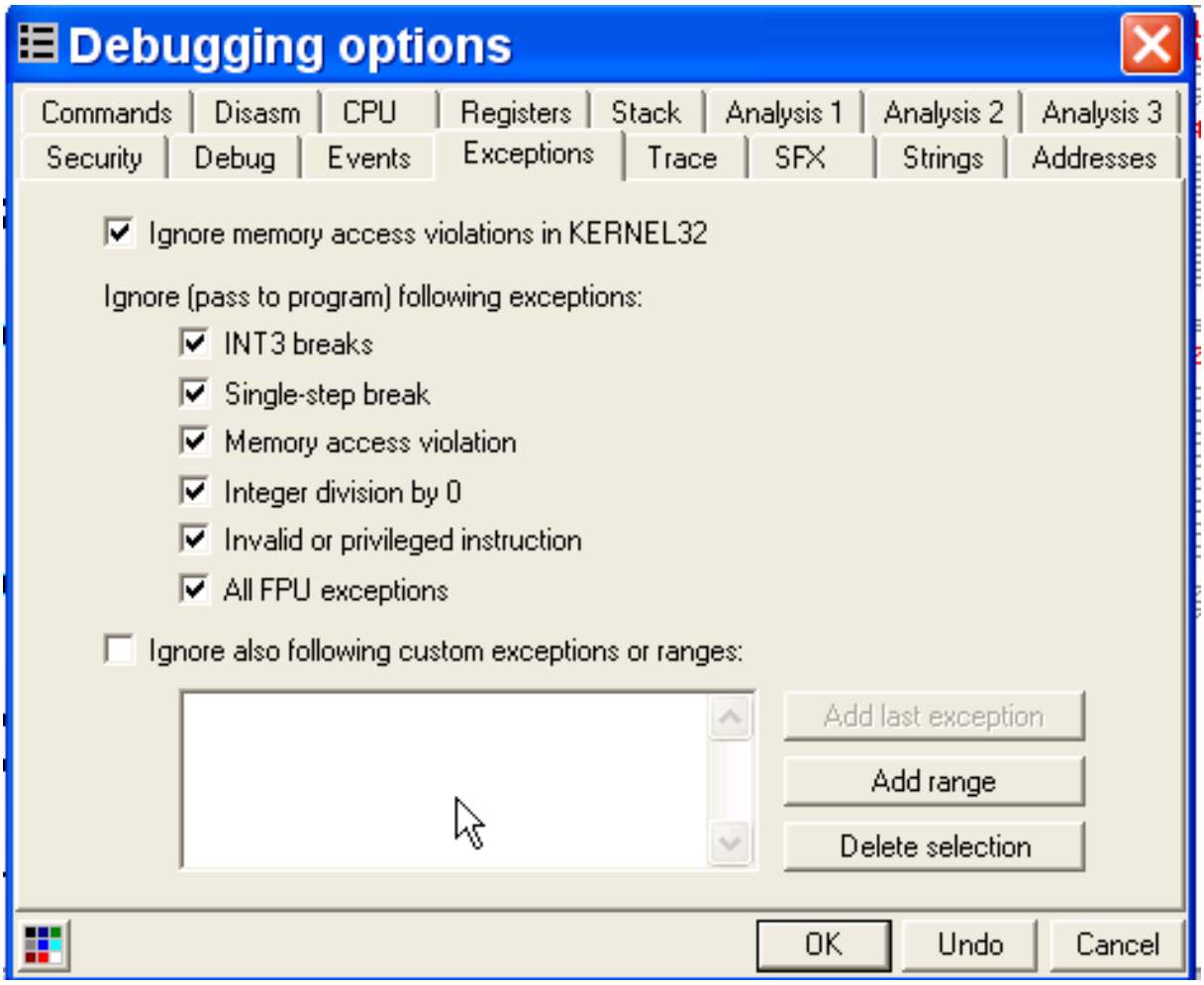


Останавливаемся тут, и если подредактируем API-функцию так, чтобы она всегда возвращала ноль, то программа будет думать, что никакие другие процессы не выполняются, и у неё не будет

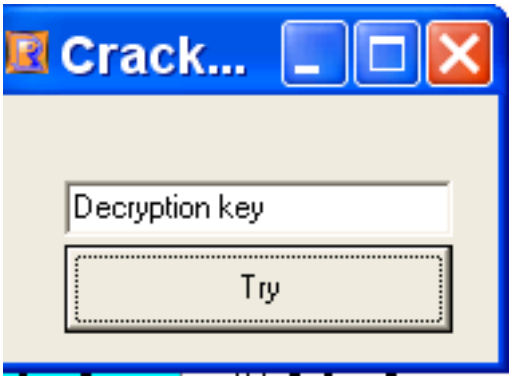
никаких логических номеров. Для этого можно изменить последние строки.

```
7C81E0BC   FF15 0C11807C   CALL DWORD PTR DS:[<&ntdll.NtOpenProces ntdll.ZwOpenProcess
7C81E0C2   3BC6            CMP EAX,ESI
7C81E0C4   5E              POP ESI
7C81E0C5   90              NOP
7C81E0C6   90              NOP
7C81E0C7   90              NOP
7C81E0C8   90              NOP
7C81E0C9   90              NOP
7C81E0CA   90              NOP
7C81E0CB   33C0            XOR EAX,EAX
7C81E0CD   90              NOP
7C81E0CE   C9              LEAVE
7C81E0CF   C2 0C00         RETN 0C
```

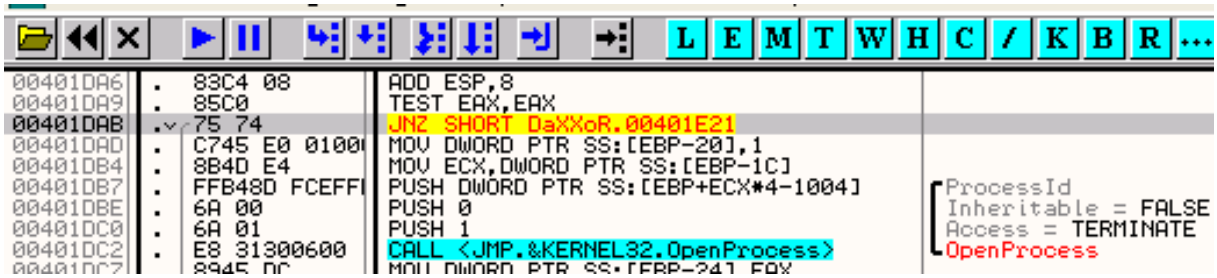
Вот так, теперь эта API-функция всегда возвращает ноль. Теперь убираем все точки останова, и так как программа генерирует кучу исключений, то, хотя мы ещё и не рассматривали данную тему, чтобы она нормально выполнялась, отметим все опции в DEBUGGING OPTIONS-EXCEPTION следующим образом:



Пока что просто отметьте все опции, чтобы программы перепрыгивала через все исключения, так как их очень много (для прохождения через них надо нажимать SHIFT+F9, а вышеуказанная конфигурация позволяет миновать их автоматически). Делаем RUN.

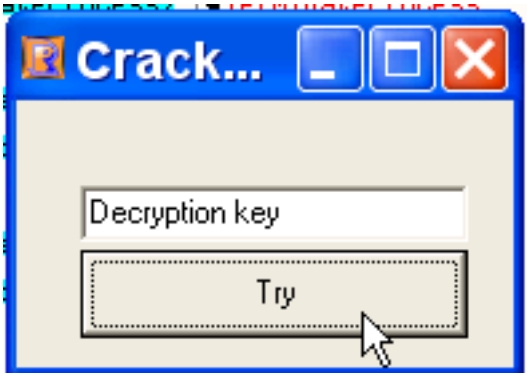


И вот, она выполняется без проблем, но вот ещё, что можно сделать:



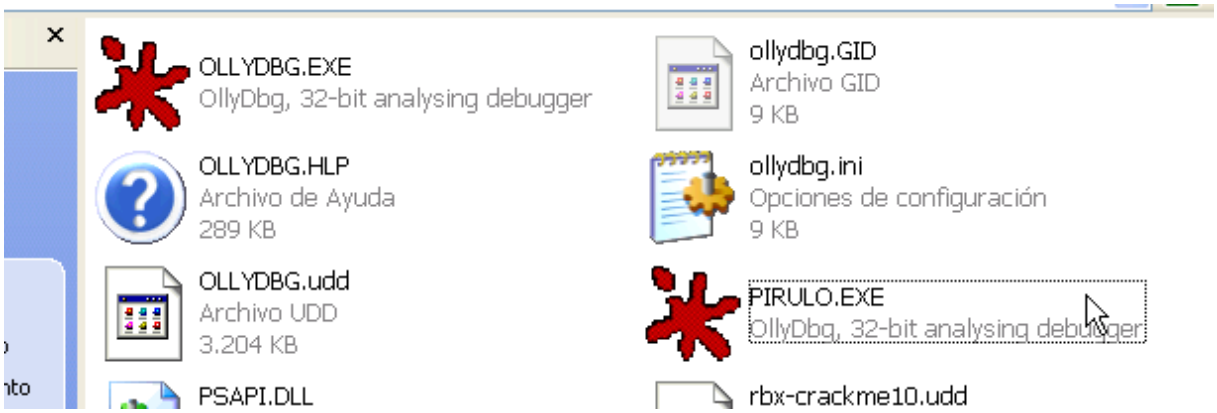
Меняем JNZ на JMP, чтобы избежать выполнения кода защиты.

Программа запускается и показывает нам окно, которое исчезает после нажатия на TRY.

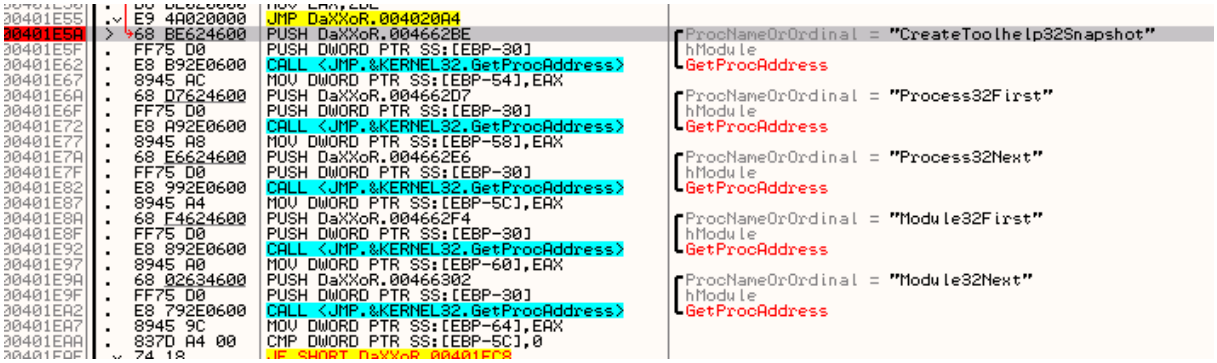


Выскакивает сообщение об ошибке. Прекрасно, потому что это говорит о том, что защита против отладки была побеждена.

Всё это не обязательно делать, чтобы обойти подобную защиту, достаточно скопировать OLLYDBG.exe под другим название, скажем, PIRULO.exe, работать отладчик после этого будет также, а вот отлаживаемая в нём программа не найдёт ни одного процесса под имени OLLYDBG, так как теперь он называется PIRULO.



Важно помнить, что переименованный файл необходимо держать в той же папке, что и исходный, во избежание проблем с плагинами.



Может быть защита и такого рода, когда программа идёт на уступки, если вводим правильный серийный номер, а также использовать другие API для нахождения имён процессов. Всё это мы рассмотрим в следующих главах.

Введение в крэкинг с нуля, используя OllyDbg - Глава 21

Автор: Рикардо Нарваха, пер. Aquila

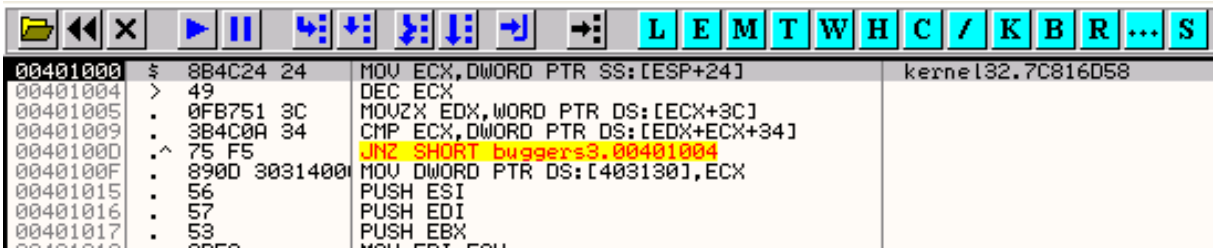
Дата: 2008-02-04

Продолжаем углубляться в различные методы антиотладки. Сейчас будем использовать крэкми, которое было специально модифицировано для целей данной статьи.

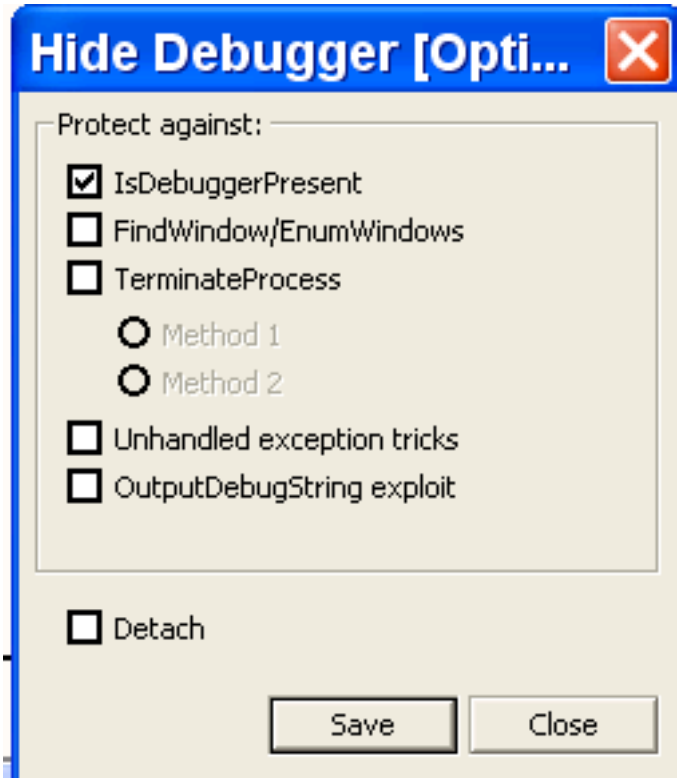
Это крэкми buggers3 (находится в архиве `wasm_files.zip`: `files/0475/buggers3.zip`), в котором были сделаны некоторые изменения, чтобы можно было объяснить нахождение имени процесса с помощью других API-функций, использующихся в этом крэкми, и обнаружение OllyDbg по имени окна или его класса, которое также применяется здесь.

Откроем крэкми в оригинальном OllyDbg, не переименованном, потому что также будем изучать разновидность метода, применённого в 20-й главе, и поэтому необходимо, чтобы файл OllyDbg назывался ollydbg.exe. Тогда защита сработает, и мы сможем её изучить.

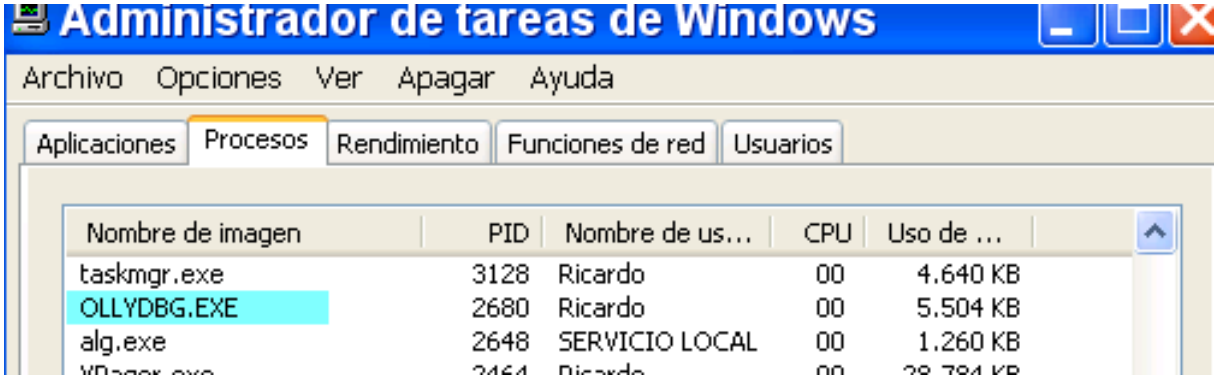
После того, как загрузили его в OllyDbg, настроим HideDebugger 1.23f против использования программой API-функции IsDebuggerPresent.



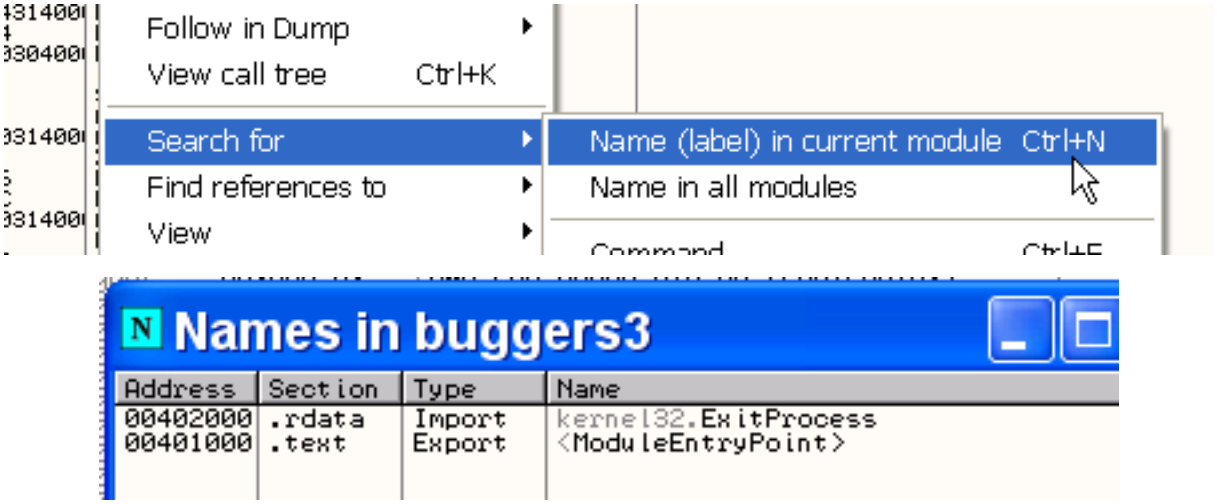
Посмотрим текущую конфигурацию плагина HideDebugger:



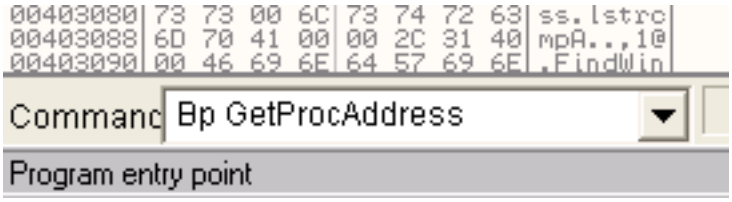
Пусть он защищает только от IsDebuggerPresent. Также откроем список процессов и убедимся, что используем непереименованный файл, то есть имя процесса ‘OLLYDBG.exe’.



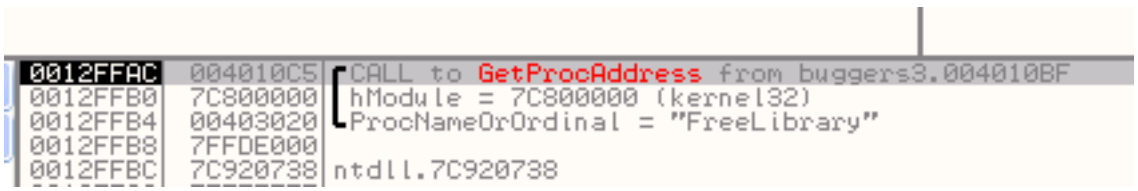
Ок, возвращаемся к buggers3. Посмотрим, какие API-функции он использует.



Опа! Единственная функция в списке – это ExitProcess, всё остальное грузится через GetProcAddress, но и сама она в списке не присутствует.

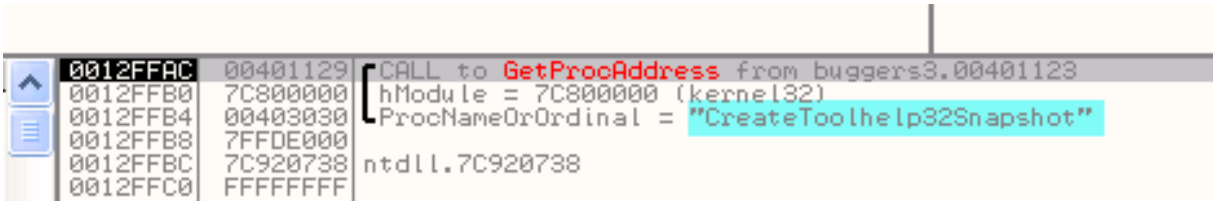


Поставим BP, и что будет, если сделаем RUN?



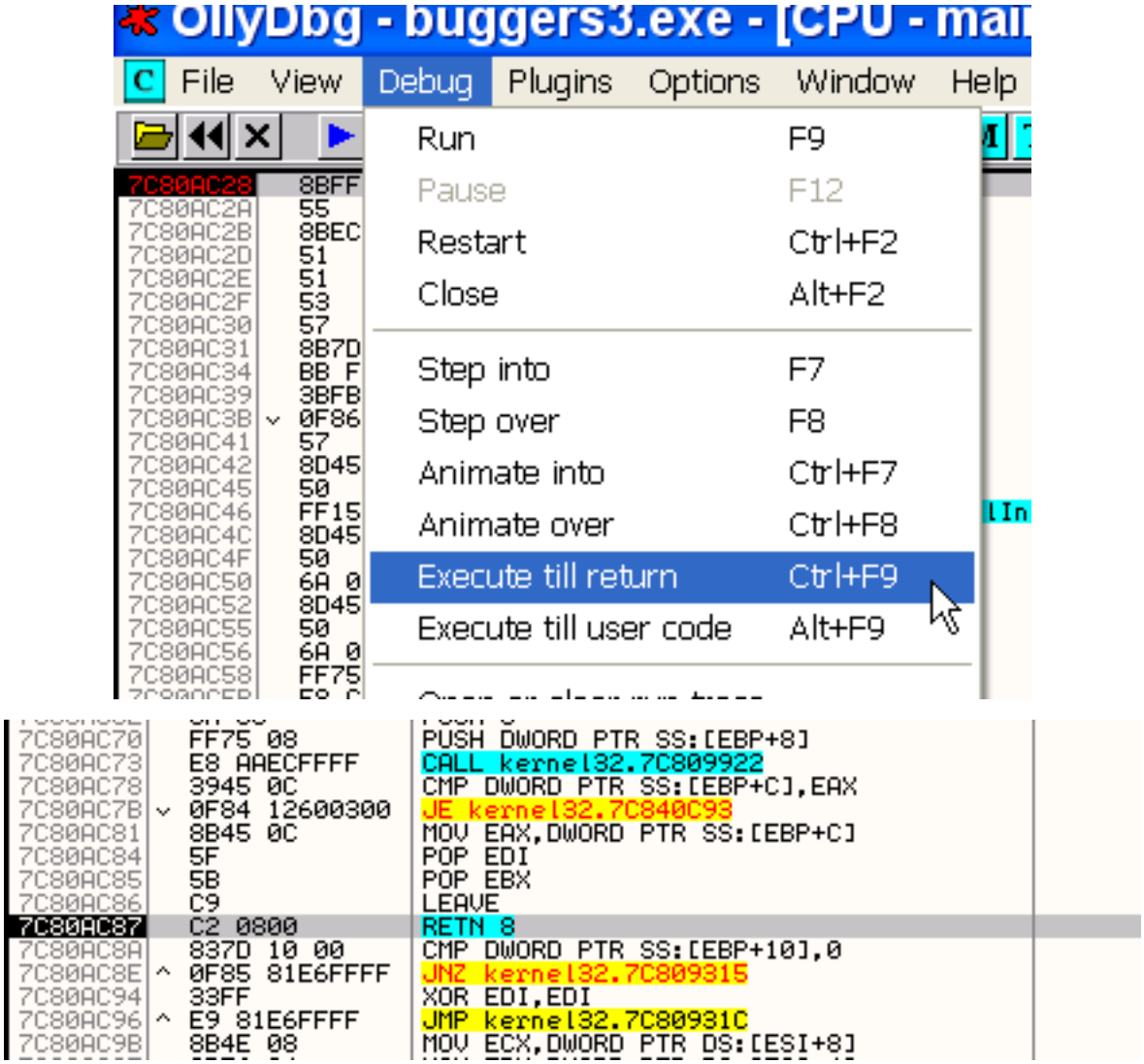
Видим загружающиеся API-функции, конечно, если находим такую, что нам интересна, идём до RET, чтобы узнать её адрес, и также можем сделать BP EAX, так как именно в EAX будет находится возвращаемый GetProcAddress адрес.

В данном случае ничего интересного нет, поэтому продолжаем, нажав F9.

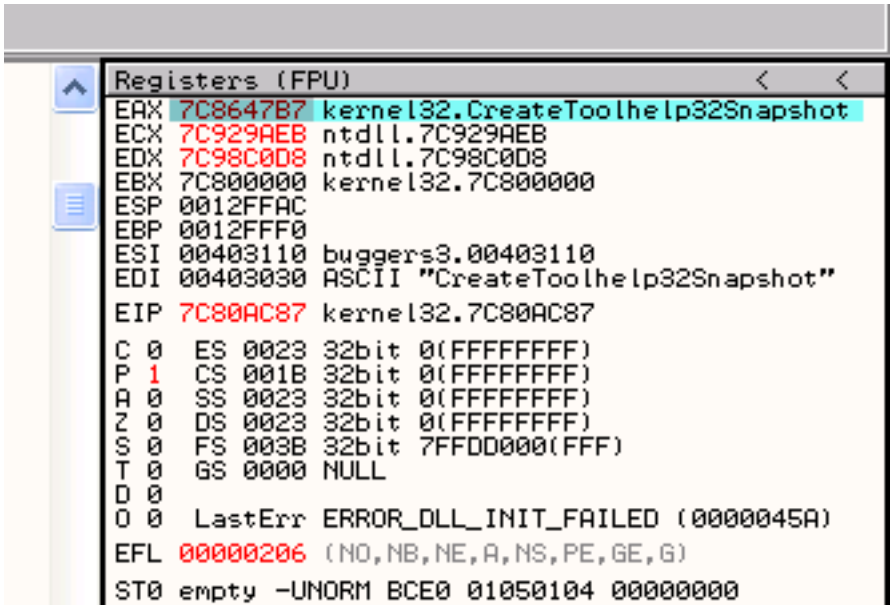


В некоторых случаях вы можете спросить, как я определил защиту ещё до того, как она сработала. Как и в прошлый раз, это потому что я уже знал о ней, чтобы продемонстрировать, как она работает и какие API-функции в ней используются.

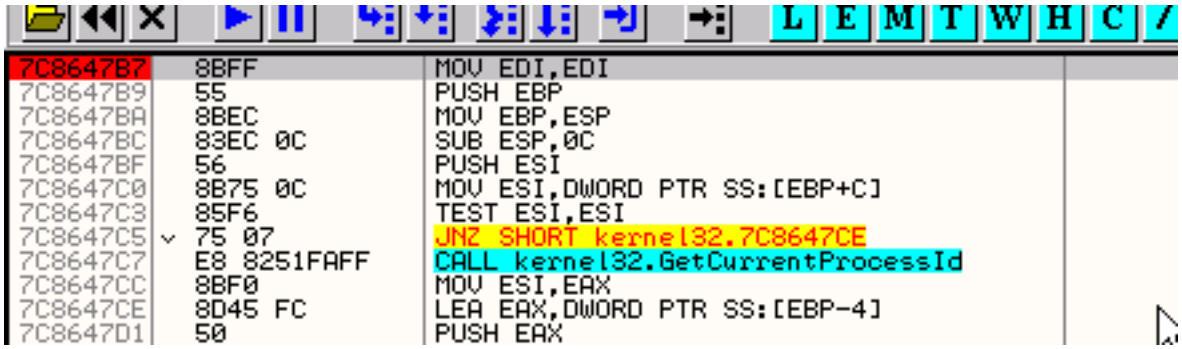
Ок, делаем EXECUTE TILL RETURN, чтобы дойти до RET.



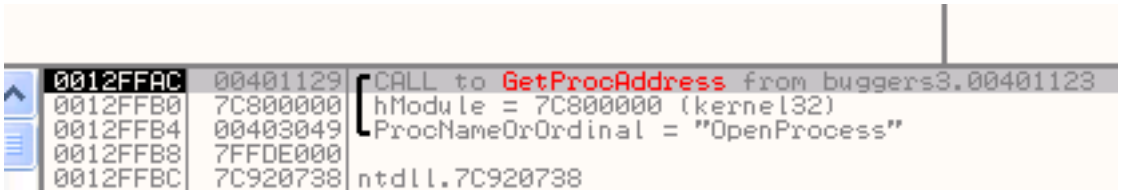
Доходим до RET, и конечно, в EAX находится адрес загружаемой API-функции на моей машине, в данном случае это CreateToolhelp32SnapShot. Чтобы узнать, когда эта функция используется, установим точку останова с помощью BP EAX.



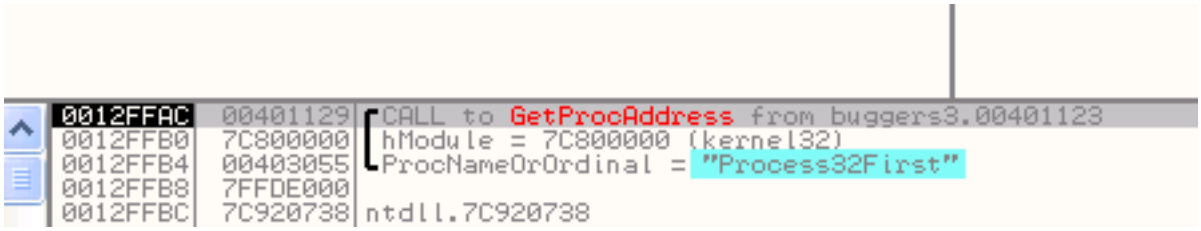
Здесь осталось установить BP на API-функцию.



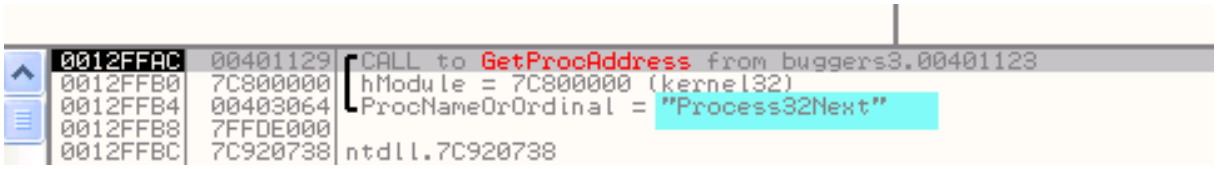
Снова делаем RUN, чтобы посмотреть, будут ли загружаться ещё какие-нибудь подозрительные функции.



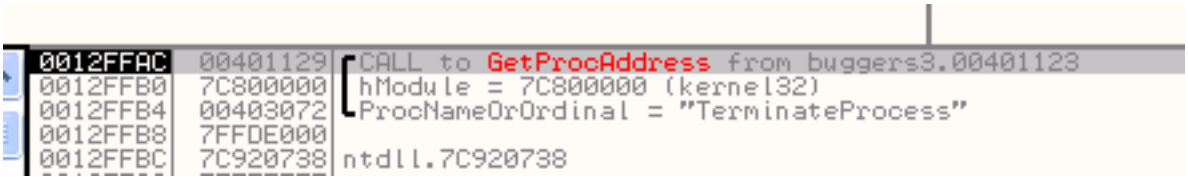
Хорошо, уже знаем, чем это рискованно – для того, чтобы принудительного завершить процесс нужно получить логический номер (о чём мы говорили в главе 20), а это делается с помощью OpenProcess, так что дойдём до RET и установим BP EAX также и на эту функцию.



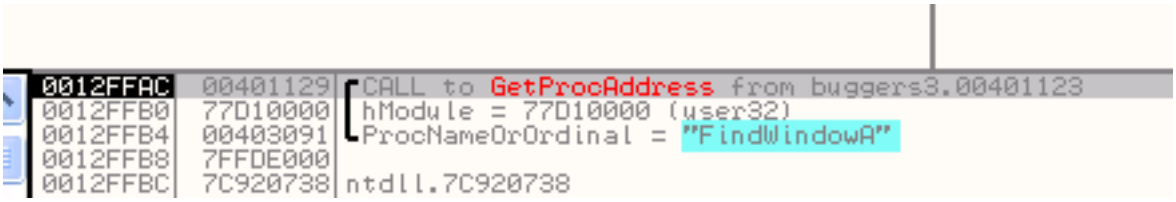
Другая подозреваемая в убийстве процессов, хе-хе, пока что таким же образом установим на неё и её сестрёнку Process32Next.



Дальше идёт TerminateProcess.

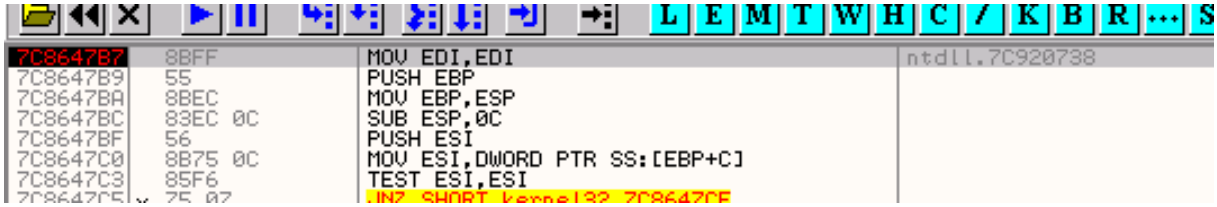


Мы уже знаем, что эта функция закрывает OllyDbg. Пока что не будем устанавливать BP, но эта функция всегда является кандидатом на точку останова, хе-хе.

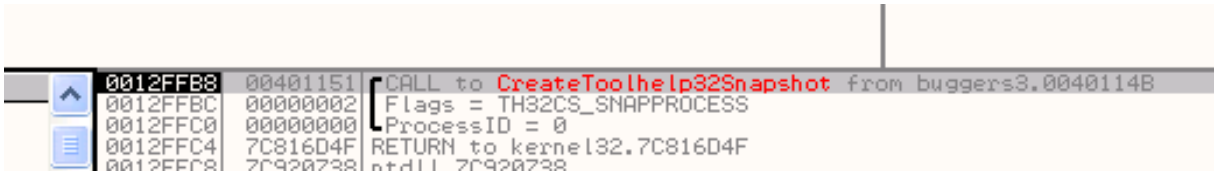


Другая преступница, хе-хе, устанавливаем на неё BP вышеописанном способом.

В следующий раз мы останавливаемся на API-функции CreateToolhelp32SnapShot.



Стек:



Посмотрим описание этой функции.

CreateToolhelp32Snapshot

Takes a snapshot of the processes and the heaps, modules, and threads used by the processes.

```
HANDLE WINAPI CreateToolhelp32Snapshot(DWORD dwFlags,
    DWORD th32ProcessID);
```

Parameters

dwFlags

Flags specifying portions of the system to include in the snapshot. These values are defined:

TH32CS_INHERIT	Indicates that the snapshot handle is to be inheritable.
TH32CS_SNAPALL	Equivalent to specifying the TH32CS_SNAPHEAPLIST, TH32CS_SNAPMODULE, TH32CS_SNAPPROCESS, and TH32CS_SNAPTHREAD values.
TH32CS_SNAPHEAPLIST	Includes the heap list of the specified process in the snapshot.
TH32CS_SNAPMODULE	Includes the module list of the specified process in the snapshot.
TH32CS_SNAPPROCESS	Includes the Win32 process list in the snapshot.
TH32CS_SNAPTHREAD	Includes the Win32 thread list in the snapshot.

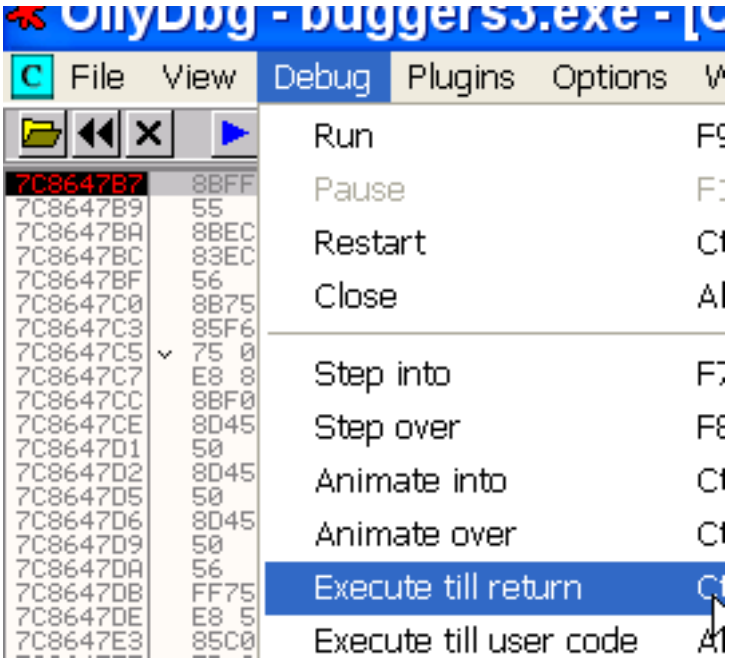
th32ProcessID

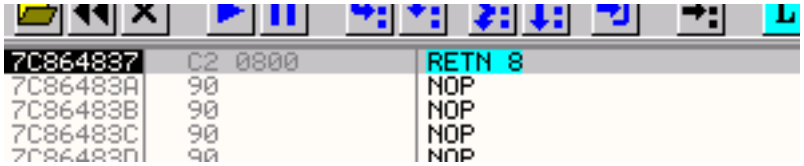
Process identifier. This parameter can be zero to indicate the current process. This parameter is used when the TH32CS_SNAPHEAPLIST or TH32CS_SNAPMODULE value is specified. Otherwise, it is ignored.

Return Value

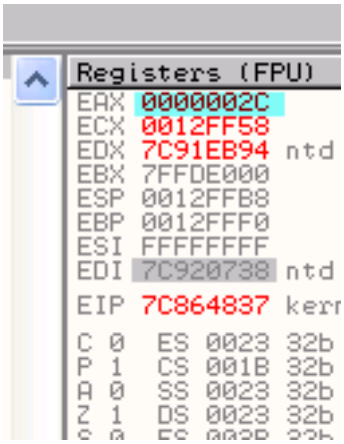
Returns an open handle to the specified snapshot if successful or – 1 otherwise.

То есть эта функция делает «фотографию» (моментальный снимок) процессов, которые запущены на машине, но возвращается нам только логический номер, и в параметрах нет никакого буфера, где должен быть сохранён список процессов. Выполняем до RET.

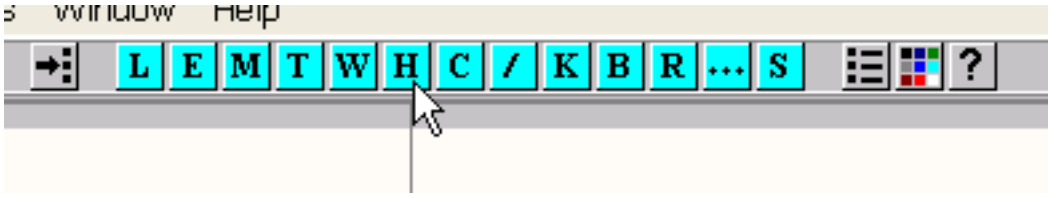




И в EAX получаем логический номер.

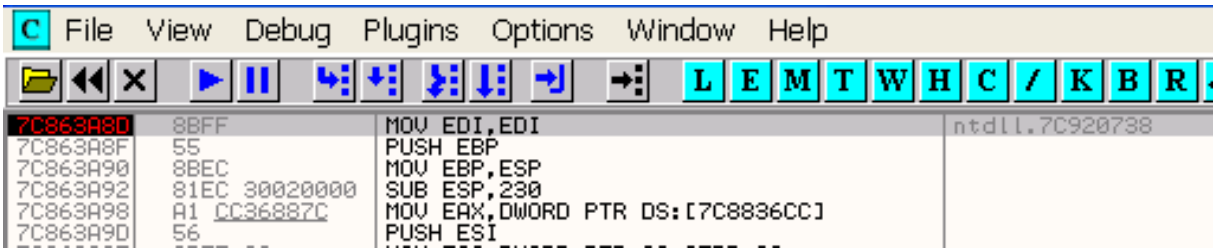


Который в моём случае равен 2C. Мы можем посмотреть информацию о нём с помощью Олли.

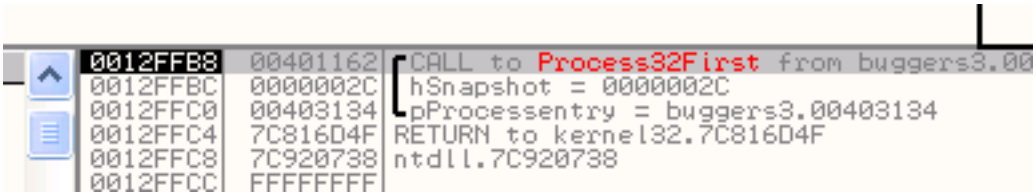


Нажмём кнопку H, что откроет окно логических номеров.

Не слишком понятно, что такое 2C, но ладно, программа получала этот номер, сделаем RUN, чтобы посмотреть, как его получить с его помощью доступ к указанному списку процессов.



Останавливаемся на API-функции Process32First, которая вместе с Process32Next служит для чтения полученного моментального снимка, извлекая информацию о выполняющихся процессах.



Ок, вот описание этой функции.

Process32First

Retrieves information about the first process encountered in a system snapshot.

```
BOOL WINAPI Process32First(HANDLE hSnapshot, LPPROCESSENTRY32 lppe);
```

Parameters

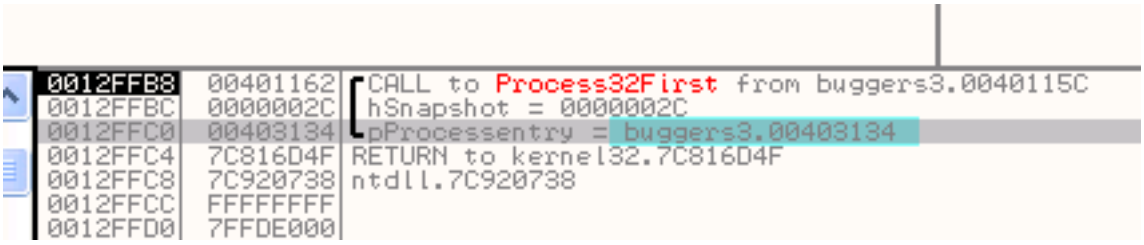
hSnapshot

Handle of the snapshot returned from a previous call to the **CreateToolhelp32Snapshot** function.

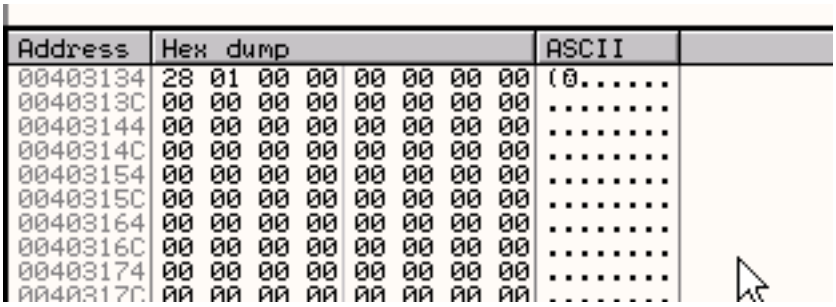
lppe

Address of a **PROCESSENTRY32** structure.

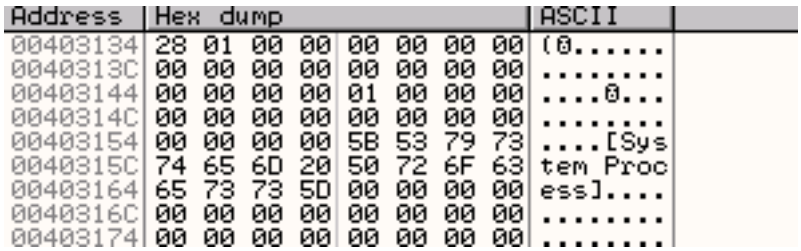
Здесь говорится, что функция получает информацию о первом процессе, записанном в моментальном снимке, чей логический номер передаётся в первом параметре (в моём случае это 2C). Вторым параметром является адрес буфера, куда собственно помещается информация.



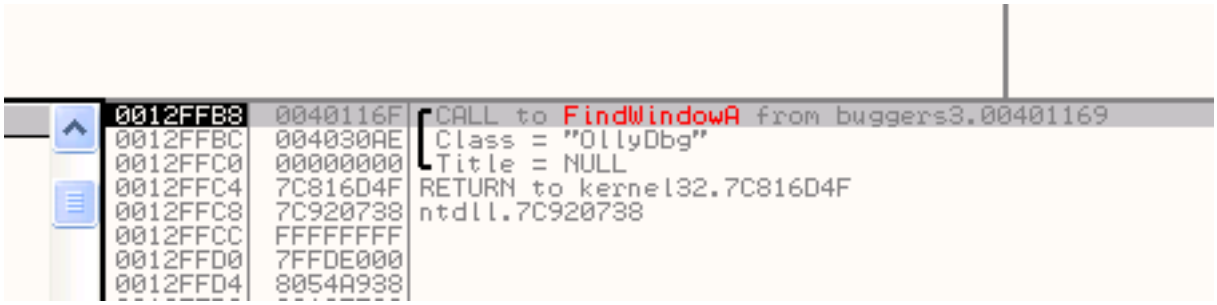
Эта API-функция возвращает информацию только о первом процессе в списке, а для остальных надо использовать Process32Next.



Буфер, куда сохраняется информация о первом процессе, виден через DUMP, так что делаем «Execute till RET», чтобы она там появилась.



Видим имя первого процесса, которое всегда будет «SYSTEM PROCESS». Делаем RUN и продолжаем.

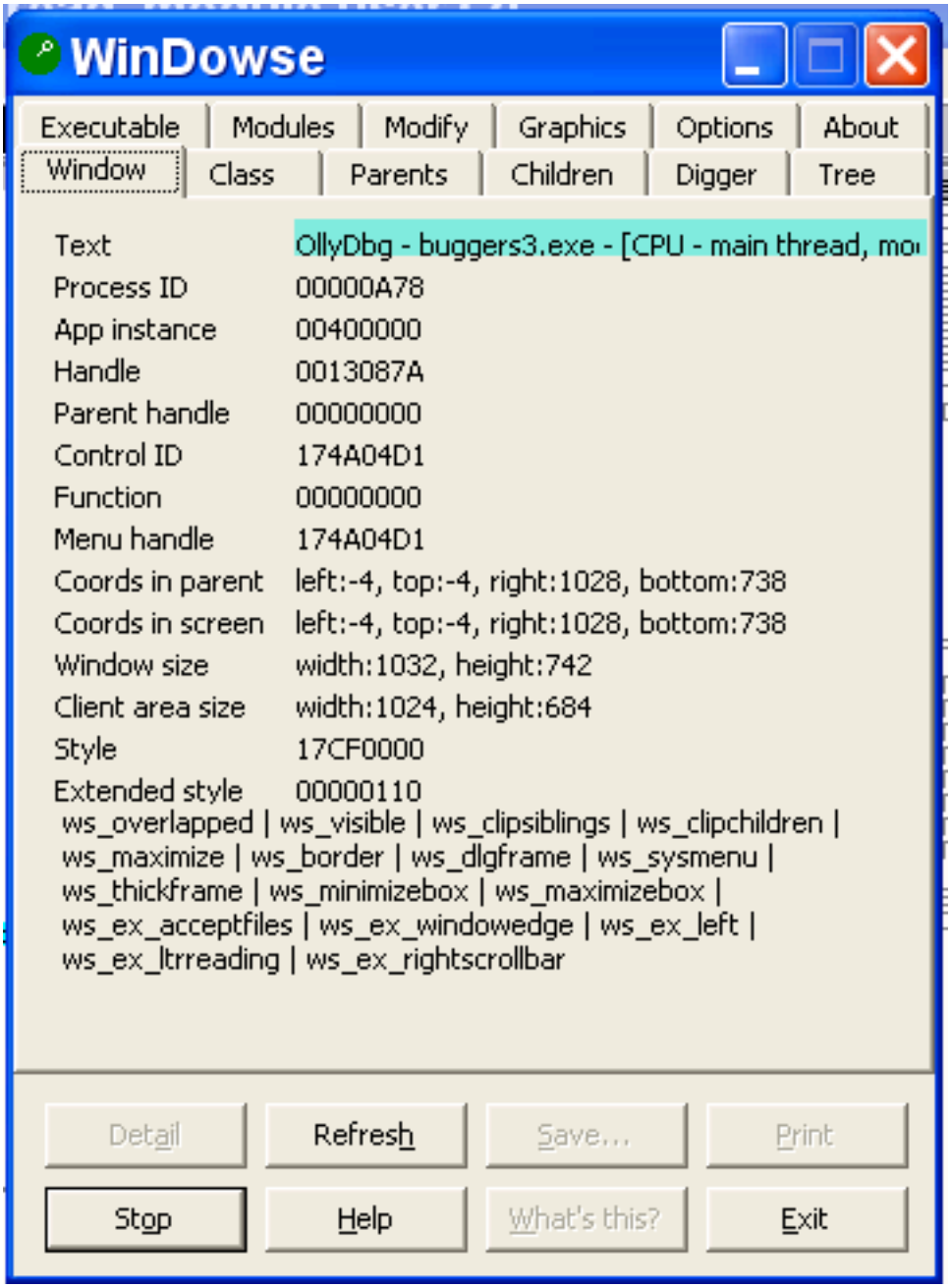


Ха-ха, а здесь другой трюк, использующий FindWindowA. Окно OllyDbg относится к одноимённому классу, так что можно искать отладчик по имени окна. В данном случае запрашивается класс окна, являющимся главным по отношению к текущему, которым, очевидно, является OllyDbg.

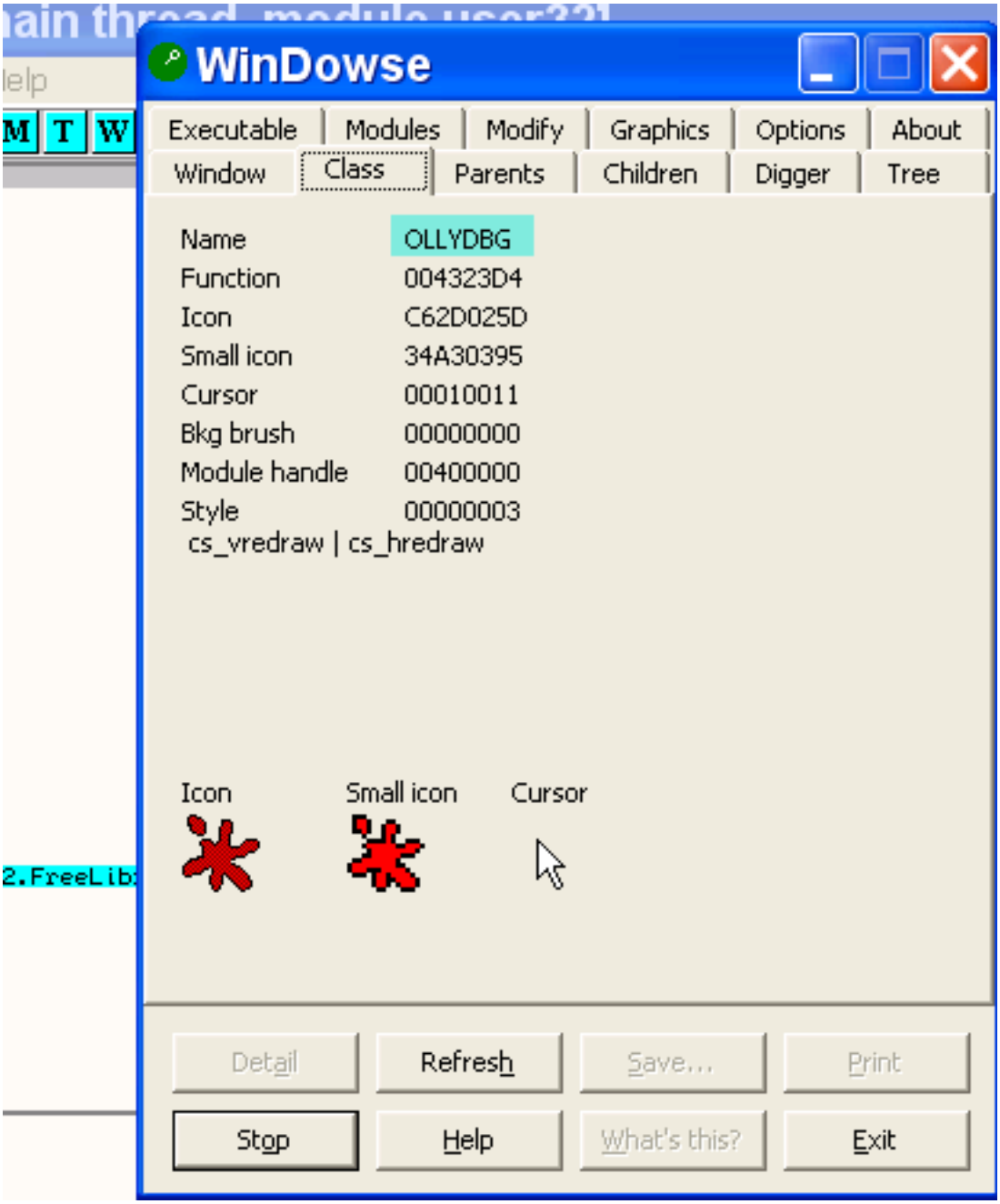
Класс окна можно узнать с помощью следующей утилиты.

http://www.ricnar456.dyndns.org/HERRAMIENTAS/V-W-X-Y-Z/WindowseGREATIS5setup.exe

Я знаю, что существуют плагины для OllyDbg, которые позволяют выяснить класс и другую информацию об окне, но, по правде говоря, эта программа предоставляет куда более подробные сведения, так что установим и запустим её.



Видим, что во вкладке «Window» выводится имя окна OllyDbg, а во вкладе «Class» - его класс.



Как видим это OllyDbg.

Видим, что API-функция возвращает логический номер окна, с помощью которого мы можем сделать с этим окном всё, что захотим.

FindWindow Quick Info

The **FindWindow** function retrieves the handle to the top-level window whose class name and window name match the specified strings. This function does not search child windows.

```
HWND FindWindow(
  LPCTSTR lpClassName,        // pointer to class name
  LPCTSTR lpWindowName        // pointer to window name
);
```

Parameters

lpClassName

Points to a null-terminated string that specifies the class name or is an atom that identifies the class-name string. If this parameter is an atom, it must be a global atom created by a previous call to the **GlobalAddAtom** function. The atom, a 16-bit value, must be placed in the low-order word of *lpClassName*; the high-order word must be zero.

lpWindowName

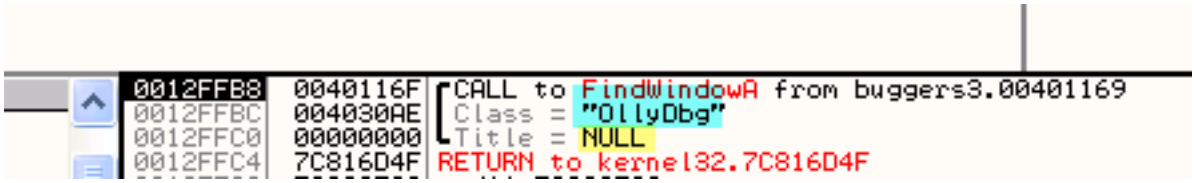
Points to a null-terminated string that specifies the window name (the window's title). If this parameter is NULL, all window names match.

Return Values

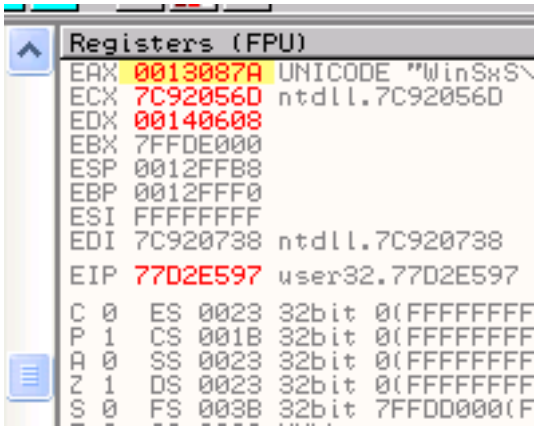
If the function succeeds, the return value is the handle to the window that has the specified class name and window name.

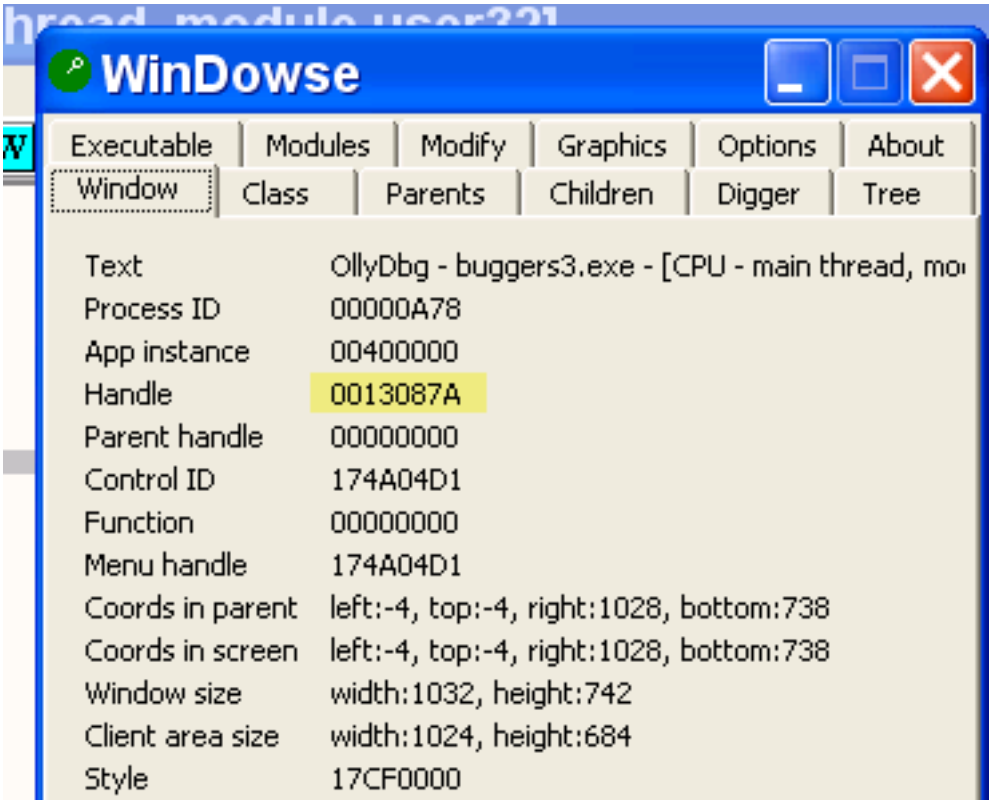
If the function fails, the return value is NULL. To get extended error information, call **GetLastError**.

То есть нет необходимости передавать оба параметра - и имя и класс, можно искать только по одному из них, а вместо другого передать ноль.



Хорошо, доходим до RET в API-функции, чтобы посмотреть, вернёт ли она логический номер окна.





Конечно, он совпадает с тем, который отображается в Windowse.

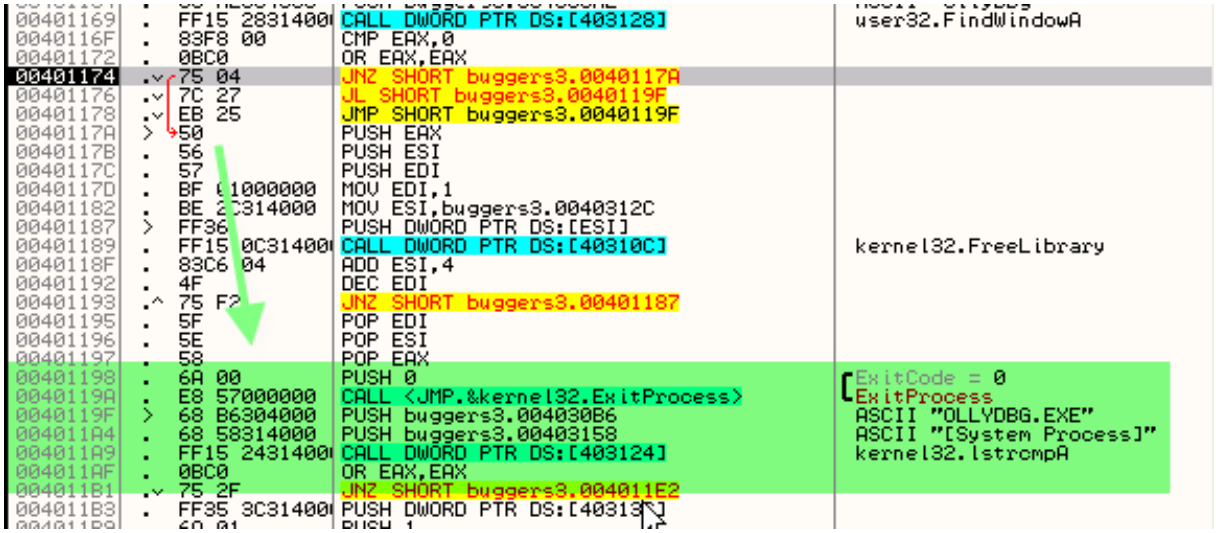
Ладно, трассируем дальше, чтобы посмотреть, что программа делает с окном.

```
00401164  .  68 AE304000    PUSH buggers3.004030AE                  ASCII "OllyDbg"
00401169  .  FF15 2831400(  CALL DWORD PTR DS:[403128]              user32.FindWindowA
0040116F  .  83F8 00        CMP EAX,0
00401172  .  0BC0           OR EAX,EAX
00401174  .v 75 04          JNZ SHORT buggers3.0040117A
00401176  .v 7C 27          JL SHORT buggers3.0040119F
00401178  .v EB 25          JMP SHORT buggers3.0040119F
0040117A  > 50              PUSH EAX
0040117B  .  56             PUSH ESI
0040117C  .  57             PUSH EDI
0040117D  .  BF 01000000    MOV EDI,1
00401182  .  BE 2C314000    MOV ESI,buggers3.0040312C
00401187  > FF36            PUSH DWORD PTR DS:[ESI]
00401189  .  FF15 0C31400(  CALL DWORD PTR DS:[40310C]              kernel32.FreeLibrary
0040118F  .  83C6 04        ADD ESI,4
00401192  .  4F             DEC EDI
00401193  .^ 75 F2          JNZ SHORT buggers3.00401187
00401195  .  5F             POP EDI
00401196  .  5E             POP ESI
00401197  .  58             POP EAX
00401198  .  6A 00          PUSH 0                                  [ExitCode = 0
0040119A  .  E8 57000000    CALL <JMP.&kernel32.ExitProcess>        ExitProcess
0040119F  > 68 B6304000     PUSH buggers3.004030B6                  ASCII "OLLYDBG.EXE"
004011A4  .  68 58314000    PUSH buggers3.00403158                  ASCII "[System Process]"
004011A9  .  FF15 2431400(  CALL DWORD PTR DS:[403124]              kernel32.lstrcmpA
004011AF  .  0BC0           OR EAX,EAX
004011B1  .v 75 2F          JNZ SHORT buggers3.004011E2
```

То есть, сравнивает полученный номер с нулём, и если он ему равен, то значит, нет окна с классом OllyDbg, а значит и такого процесса нет. Если же было возвращено значение отличное от нуля, то значит, существует окно с таким классом, и программа вызывает ExitProcess.

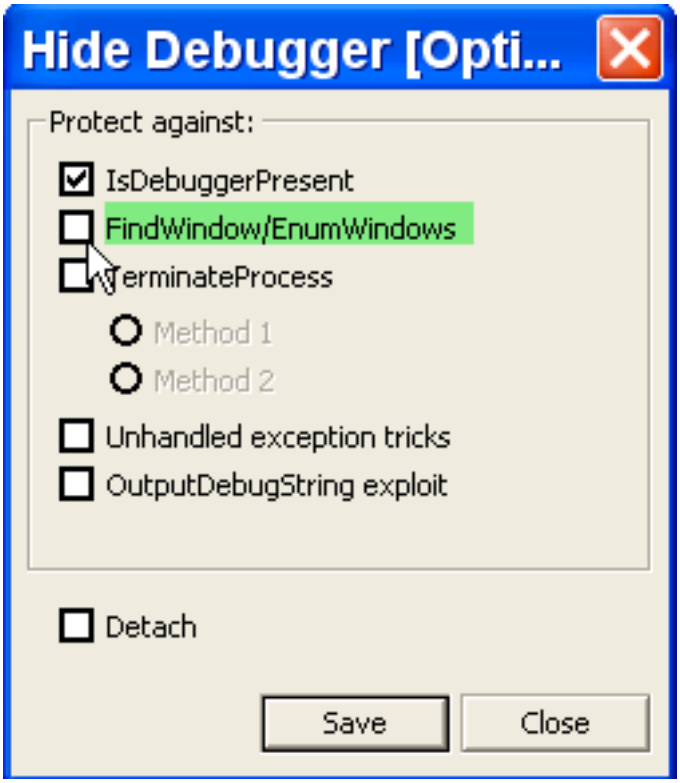
```
00401164  .  68 AE304000    PUSH buggers3.004030AE                  ASCII "OllyDbg"
00401169  .  FF15 2831400(  CALL DWORD PTR DS:[403128]              user32.FindWindowA
0040116F  .  83F8 00        CMP EAX,0
00401172  .  0BC0           OR EAX,EAX
00401174  .v 75 04          JNZ SHORT buggers3.0040117A
00401176  .v 7C 27          JL SHORT buggers3.0040119F
00401178  .v EB 25          JMP SHORT buggers3.0040119F
0040117A  > 50              PUSH EAX
0040117B  .  56             PUSH ESI
0040117C  .  57             PUSH EDI
```

Переход сразу ведёт к выходу программ без отображения каких-либо сообщений.

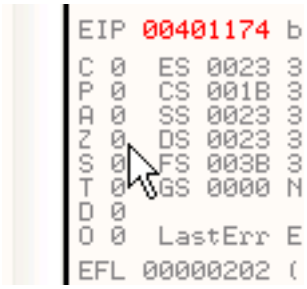


То есть суть в том, что нам нужно, чтобы FindWindowA вернула в EAX ноль.

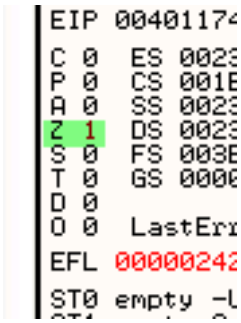
Ок, плагин HideDebugger 1.23f позволяет не делать это вручную. Откроем его настройки.



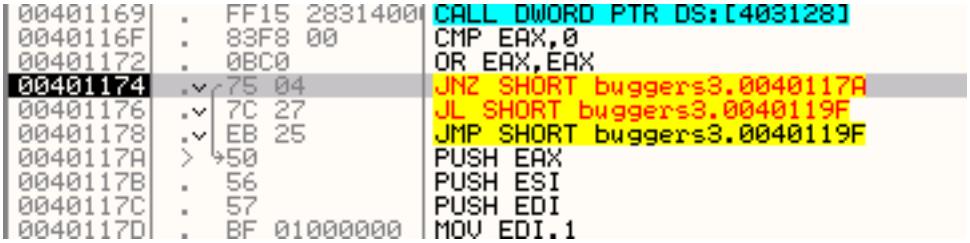
Если отметим вторую опцию, то плагин станет защищать нас от обнаружения с помощью FindWindow и EnumWindows (это другая функция, с помощью которой можно узнать имя окна), но нам нужно научиться избегать этих функции вручную и понимать, как это работает, поэтому пока что галочку ставить не будем, так как надо будет перезапустить OllyDbg, чтобы это оказало необходимый эффект. Сделаем так, чтобы переход, который отправляет нас к функции ExitProcess, не был совершён, и выполнение программы продолжилось.



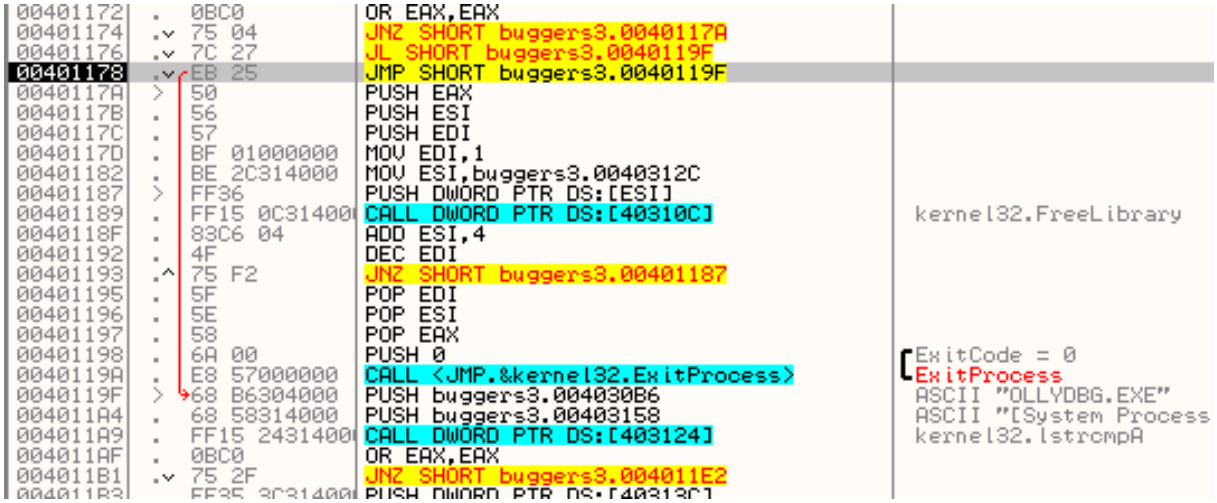
Кликаем два раза на флаге Z, меняем его значение на 1, после чего JNZ не должен совершить переход.



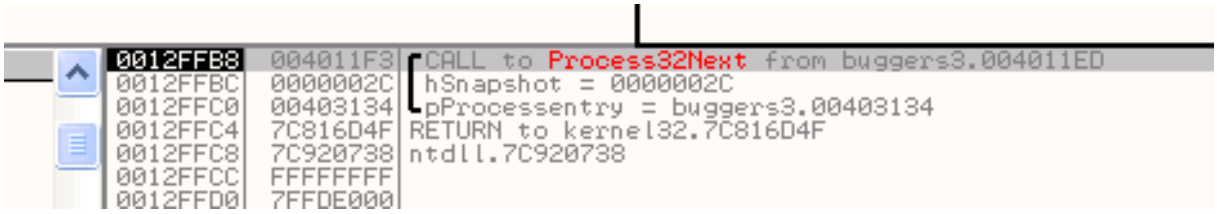
И правда, JNZ не срабатывает.



И доходим до JMP, перепрыгивающего через вызов ExitProcess.

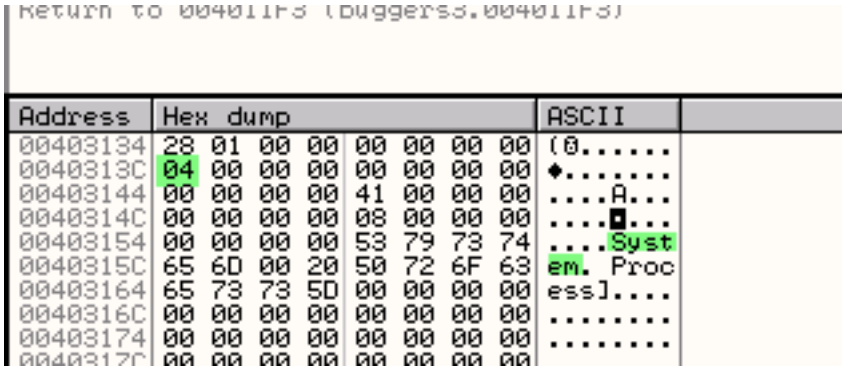


Ок, продолжаем, мы изучили, как предотвратить приём с FindWindowA, а теперь продолжим изучать метод, использующий имена процессов. Делаем RUN.

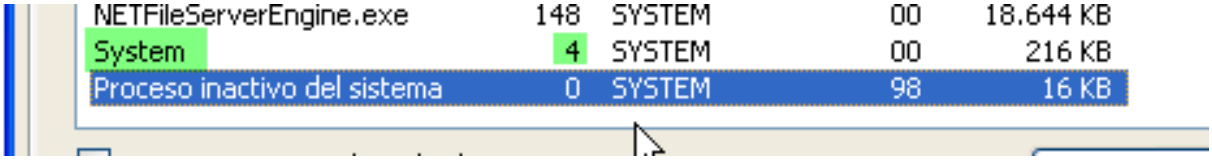


Видим, что вызывается Process32Next, чтобы получить информацию о втором процессе из моментального снимка, которая сохраняется в 403134.

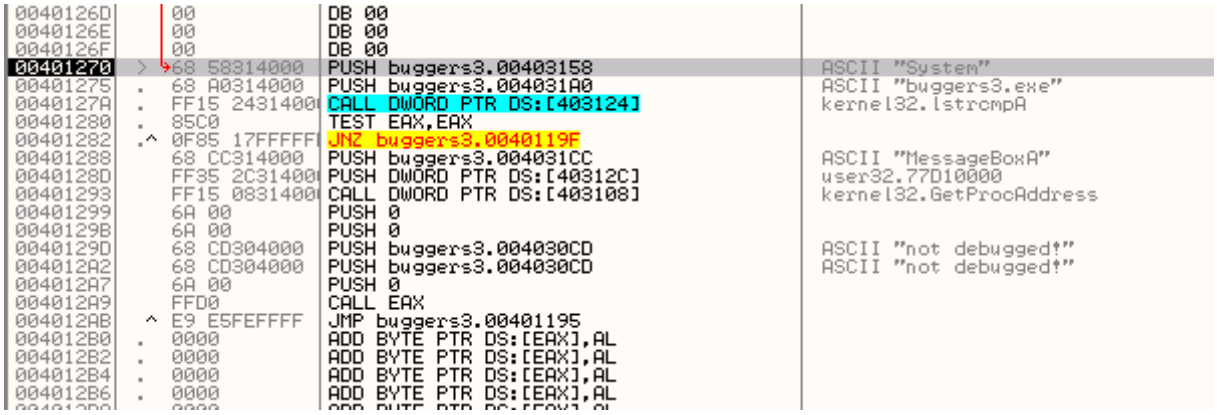
Делаем EXECUTE TILL RETURN и смотрим, что было сохранено.



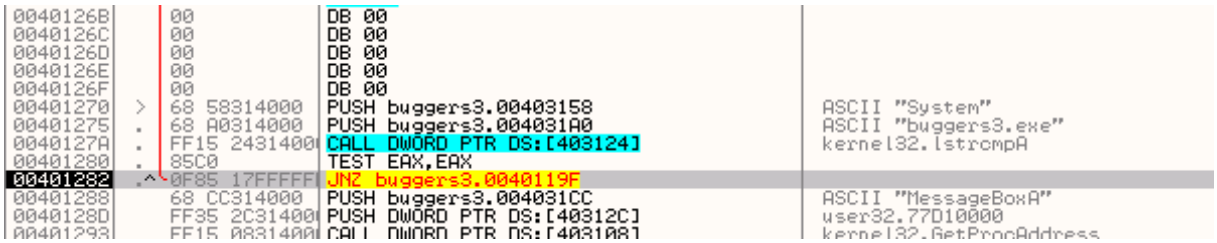
Здесь находится имя «System» и его PID, равный 4. Посмотрим список процессов.



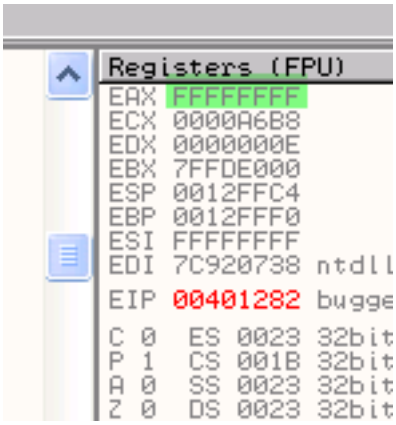
Здесь видим, что программа делает с каждым процессом. Трассируем.



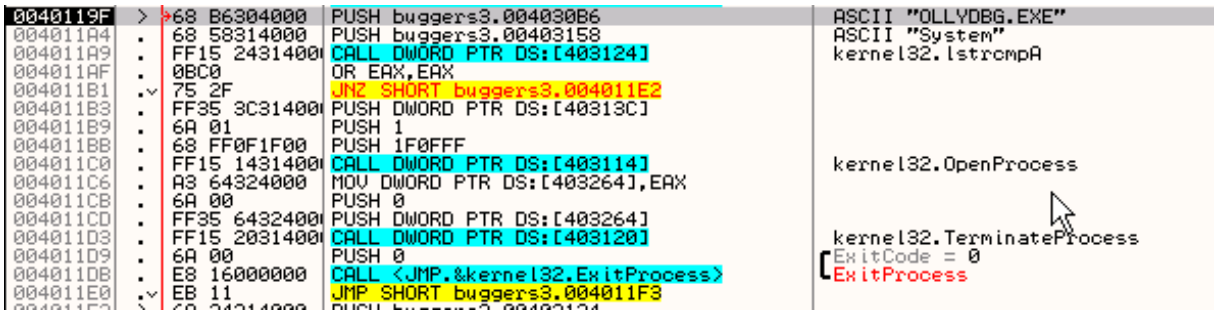
Видим, что здесь вызывается API-функция lstrcmpA, с помощью которой сравнивается строка «System», являющейся именем процесса, с «buggers3.exe», т.е. именем процесса самого крэкми. Если они равны, то вызывается MessageBoxA, сообщающий «NOT DEBUGGED». Но в данном случае мы сюда не попадём, так что продолжаем трассировать.



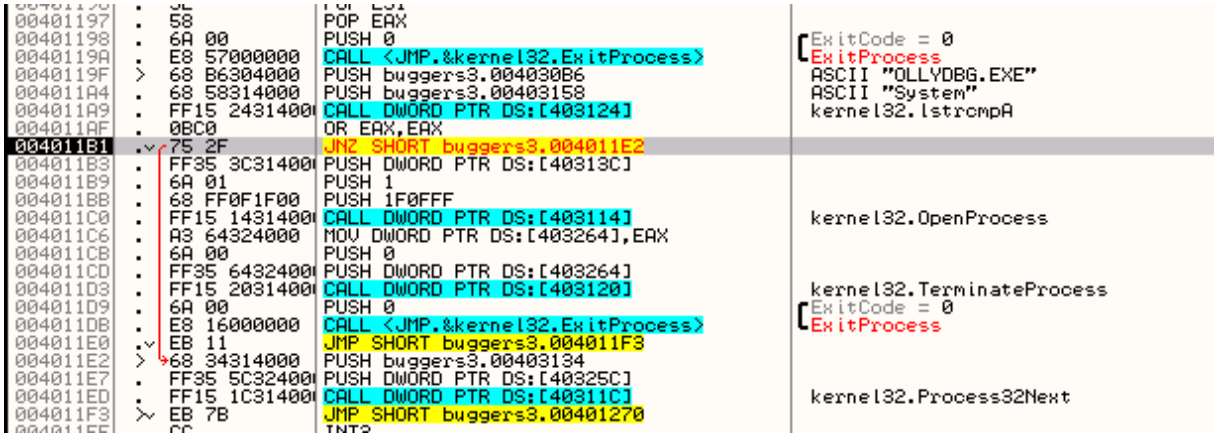
Если строки не равны, то результатом сравнения является FFFFFFFF.



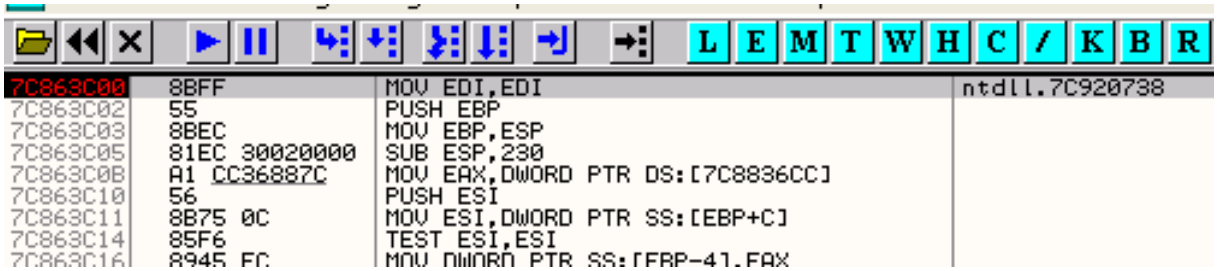
И так как это не ноль, то переходим в 40119f.

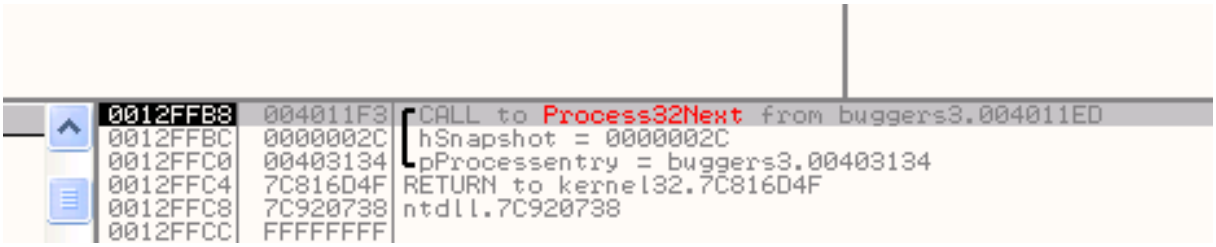


Здесь видим насыщенную действием часть программы: сравнивается имя первого процесса с «OLLYDBG.EXE», и если они равны, тогда результатом является ноль и перехода не происходит, в следствии чего вызывается OpenProcess, чтобы узнать логический номер процесса, а потом – TerminateProcess, закрывающий его, как это мы уже видели в главе 20-ой.

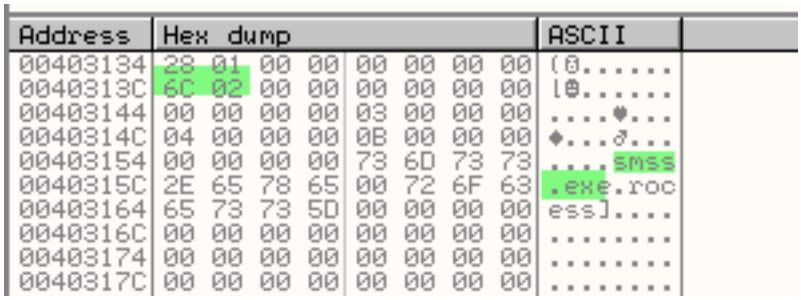


Видим, что первый процесс не является OllyDbg.exe, поэтому переходим на Process32Next, чтобы найти второй процесс.

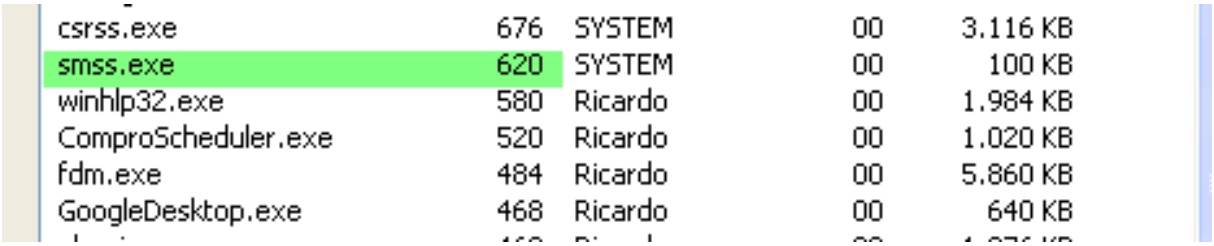




В то же место сохраняется имя второго процесса. Делаем EXECUTE TILL RETURN.

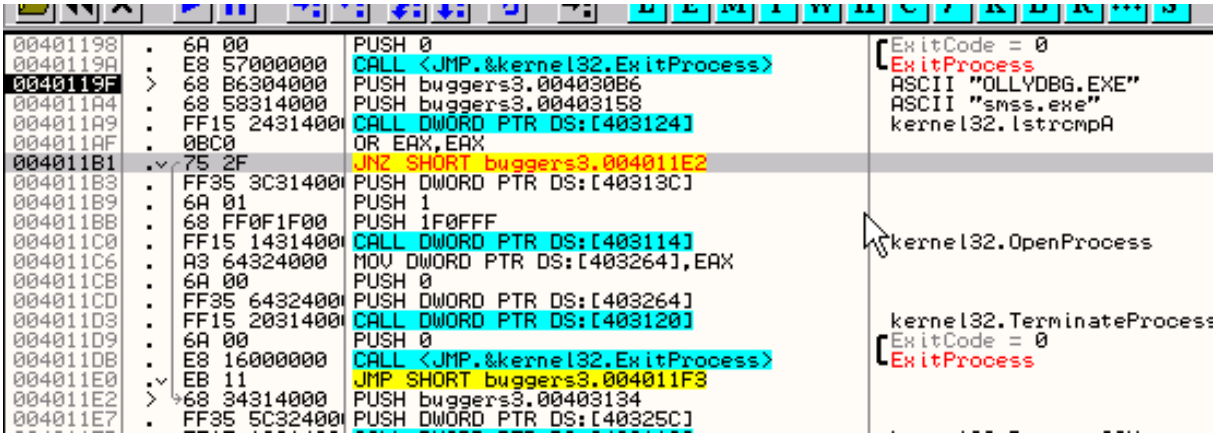


Второй процесс – это smss.exe, чей PID равен 026C. Заглянем в список процессов.

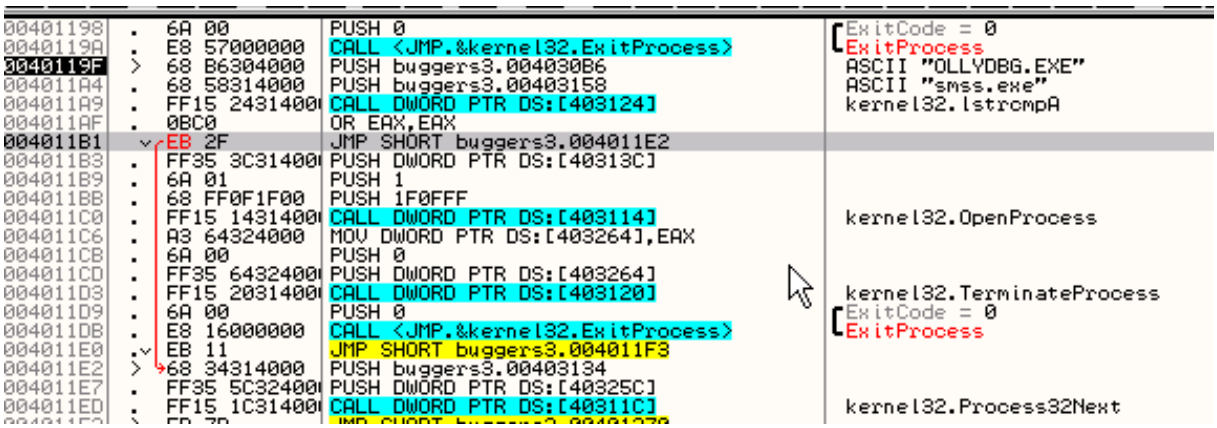


Вот PID, равный 620 в десятеричной системе, то есть 026C в шестнадцатеричной.

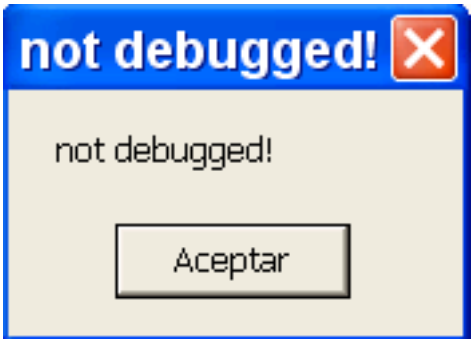
Хорошо, похоже, что все выполняющиеся процессы сравниваются один за другим с «OLLYDBG.EXE».



И каждый раз условный переход будет переносить нас на 4011B1, до тех пор, пока не встретиться процесс OLLYDBG.exe. Тогда перехода не произойдёт и Олли будет закрыта, так что если мы заменим JNZ на JMP, то избежим этого.

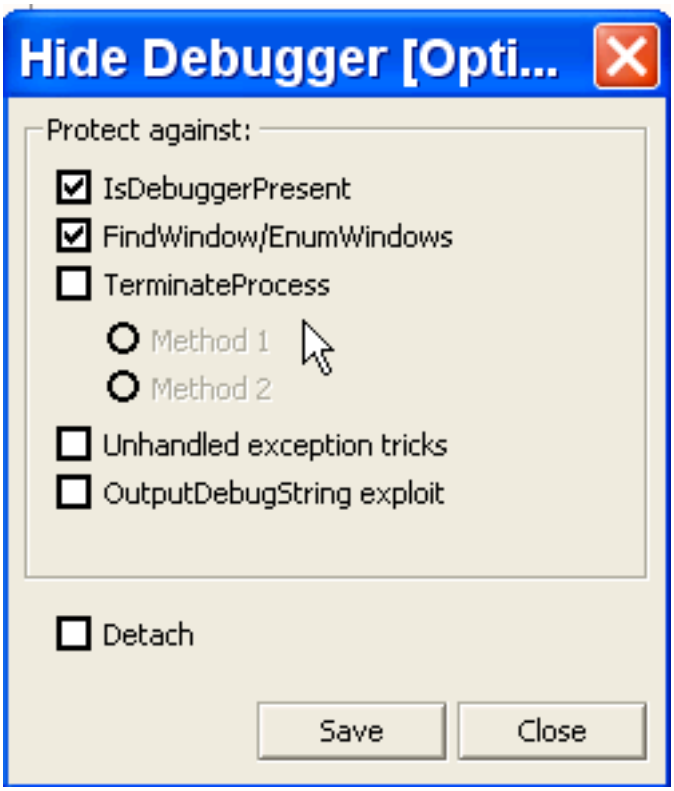


Теперь убираем все точки останова и делаем RUN.



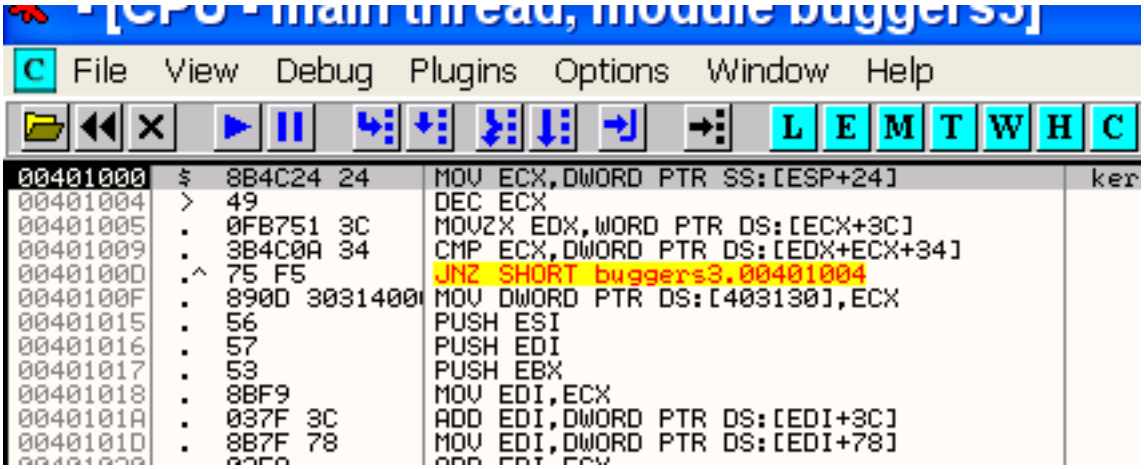
Таким образом, защита была побеждена. Сейчас мы уже знаем, что плагин HideDebugger позволяет спрятать окно OllyDbg от обнаружения с помощью FindWindowA. Также можно попробовать избежать обнаружения процесса и выполнить программу, используя упомянутый ранее способ переименования OllyDbg в PIRULO.exe.

Откроем PIRULO.exe.

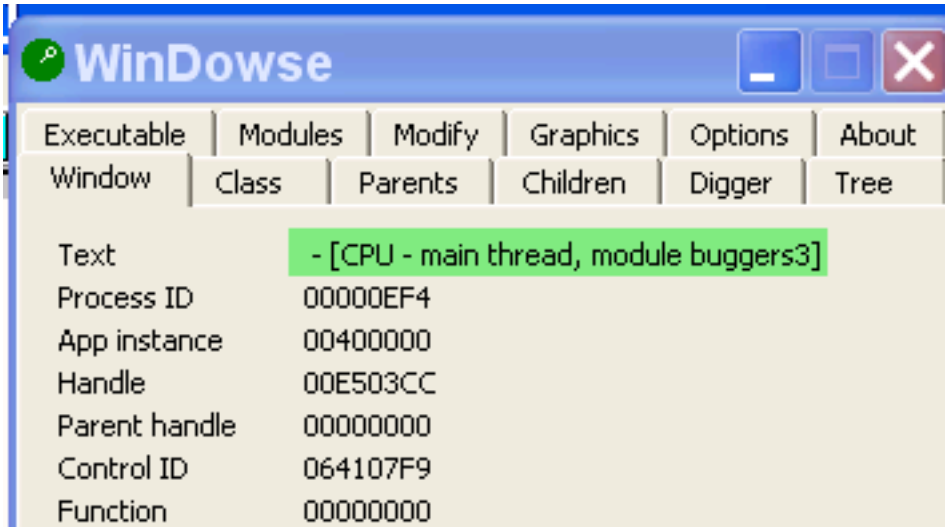


Отметим галочкой опцию, включающей защиту от FindWindowA, и нажмём SAVE.

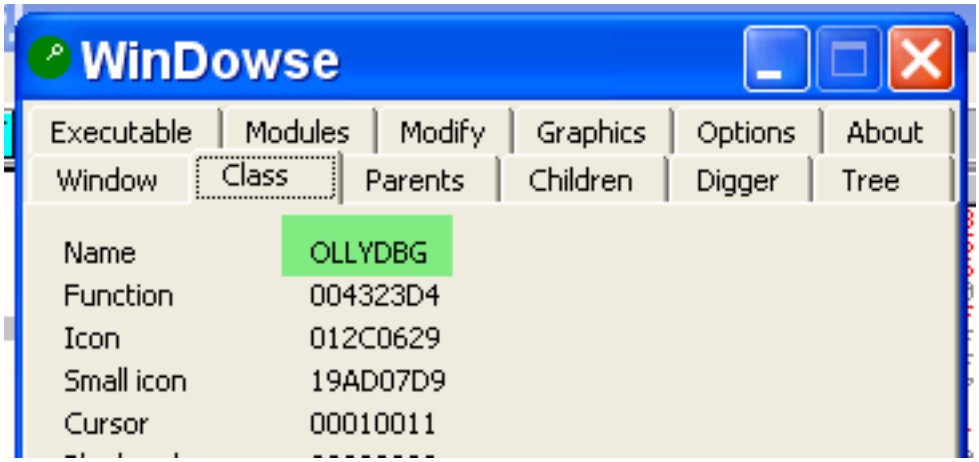
Потом перезапустим OllyDbg.



И откроем buggers3, но прежде я хочу избавиться от сомнений, поэтому посмотрю, что сделал плагин с окном Олли, чтобы оно не было обнаружено.



Видим, что имя OLLYDBG не отображается в заголовке окна, а класс?

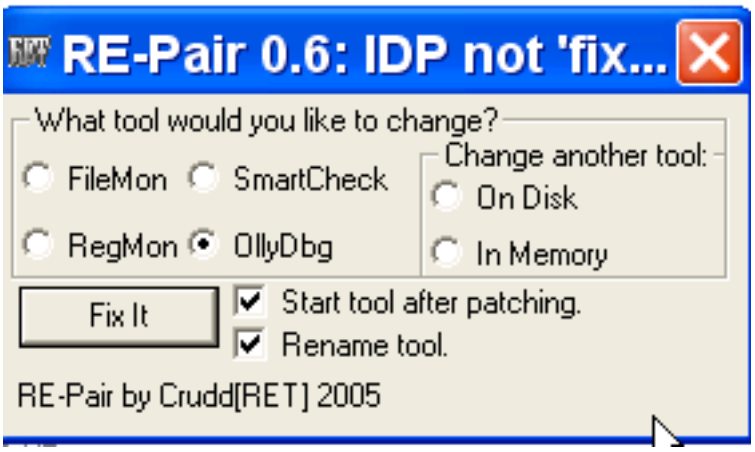


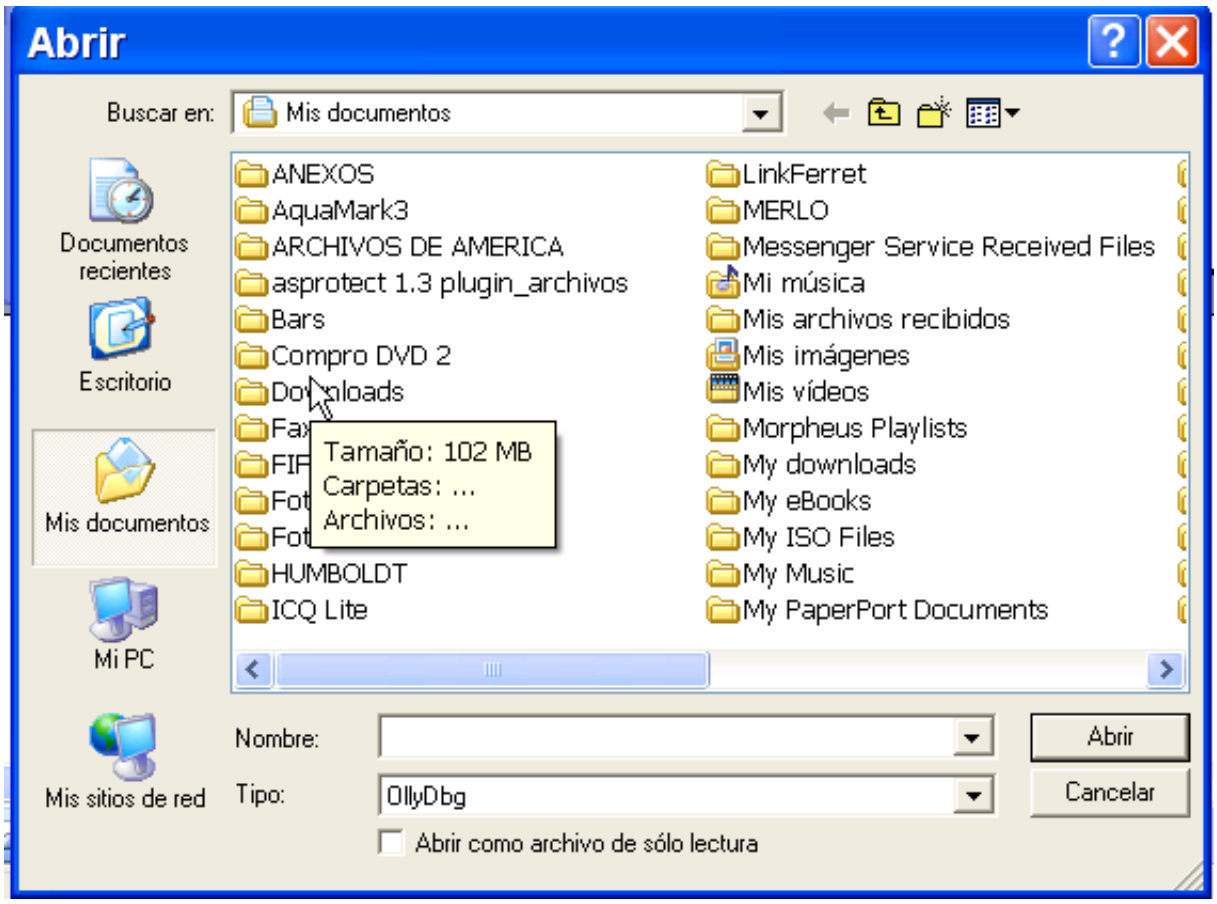
Видим, что от нахождения по классу плагин нас не защитил, поэтому надо искать другой способ.

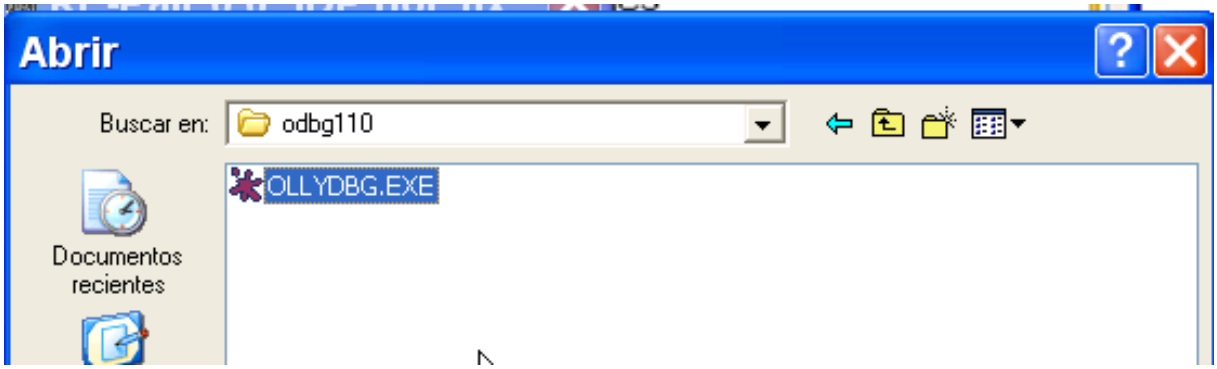
Утилита, которая должна помочь нам с тем, что не смог сделать плагин, называется Repair 0.6. Это патчер OllyDbg, и его можно скачать отсюда.

http://www.ricnar456.dyndns.org/HERRAMIENTAS/Q-R-S-T-U/repair0.6.zip

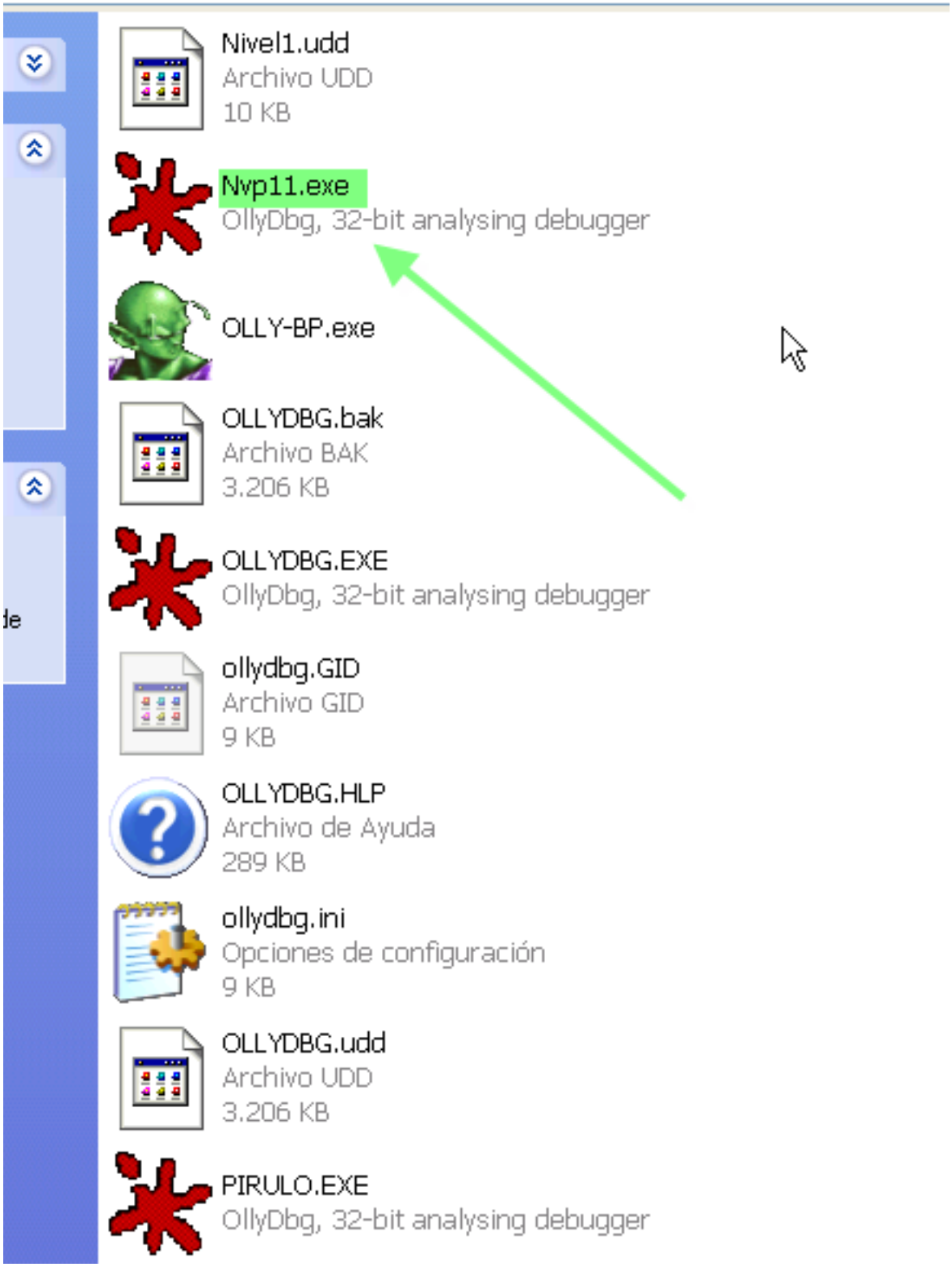
Скачаем, а потом закроем OllyDbg и запустим патчер.



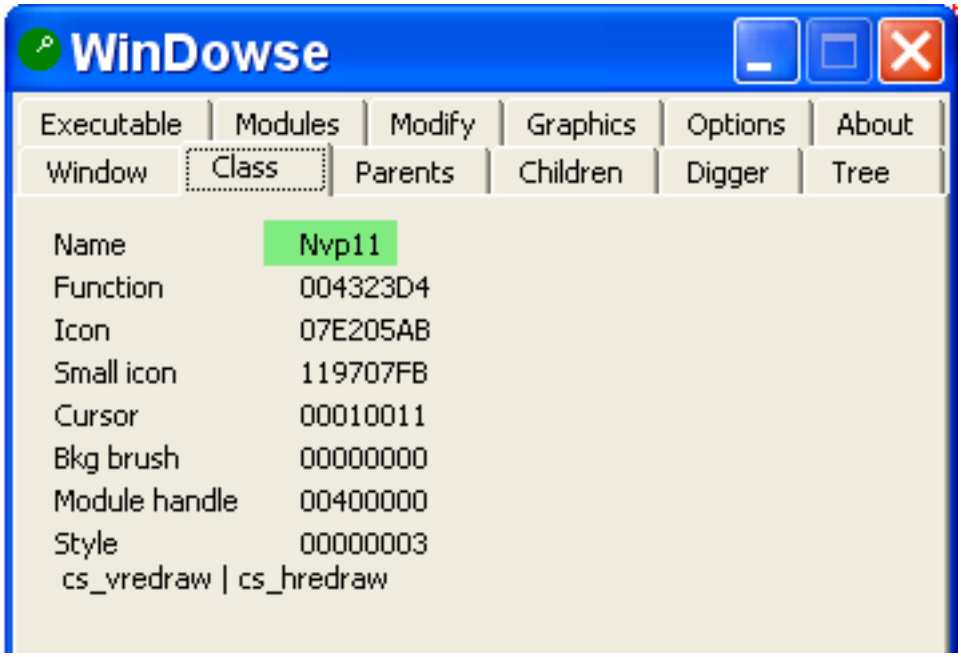




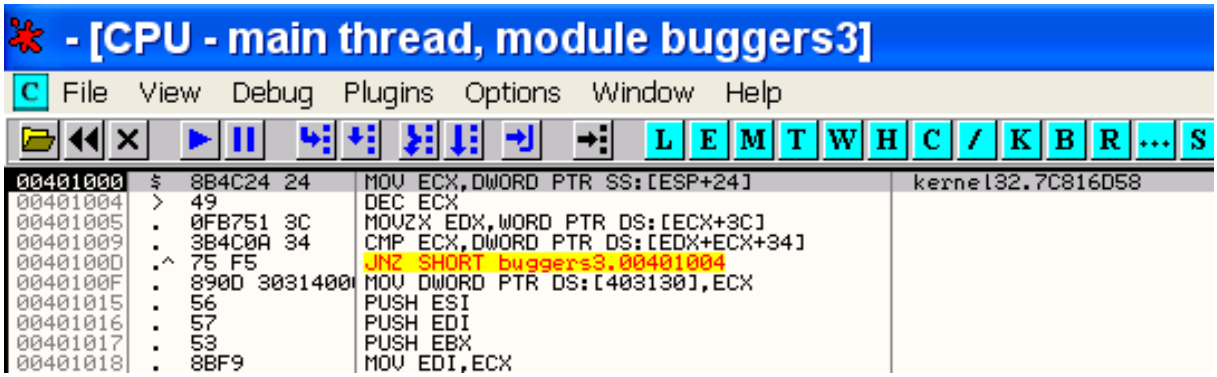
Так что теперь у нас есть третий файл OllyDbg, который называется NVP11.exe. Заглянем в папку, где он находится.



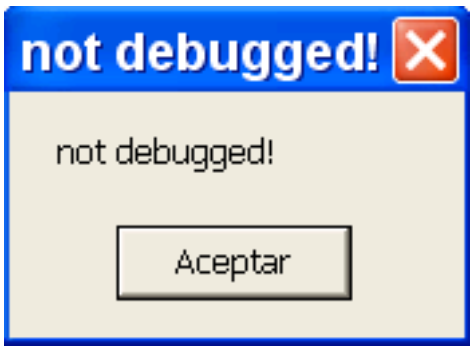
Запустим его и посмотрим, какой у него класс окна.



Видим, что класс окна – это «Nvp11», так же как и имя процесса, а значит, buggers3 теперь должен выполняться превосходно, не требуя каких-либо изменений в нём. Пробуем.



Делаем RUN и…



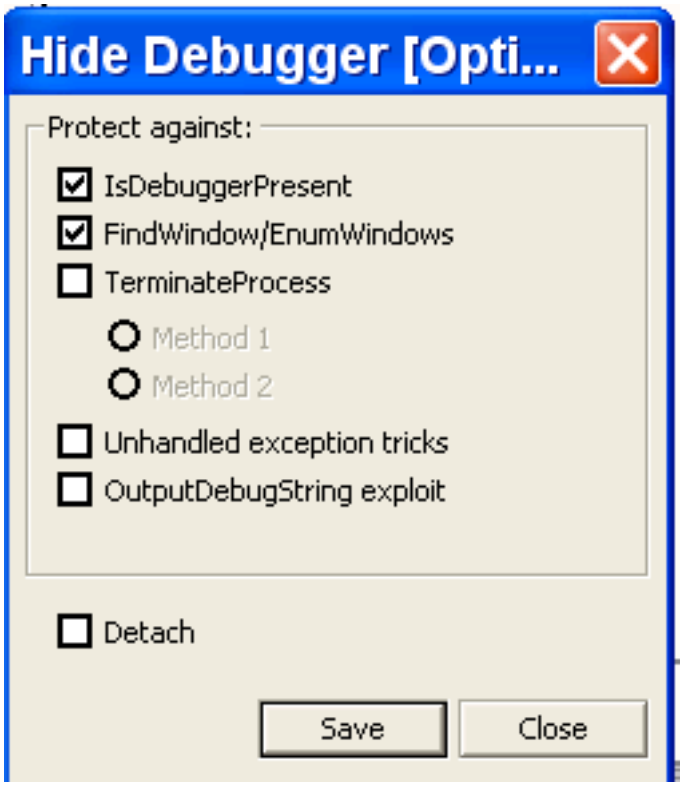
Хочу сказать, что наш пропатченный OllyDbg с каждым днём становится всё менее подверженным обнаружению, теперь его нельзя найти ни по имени процесса, ни по имени или классу окна, хе-хе, в главе 22 продолжим укреплять отладчик и изучать, как работают другие методы и способы антиотладки, как их понять, как победить вручную и, наконец, какой плагин применить, чтобы не слишком перетрудиться, хе-хе.

Введение в крэкинг с нуля, используя OllyDbg - Глава 22

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-02-06

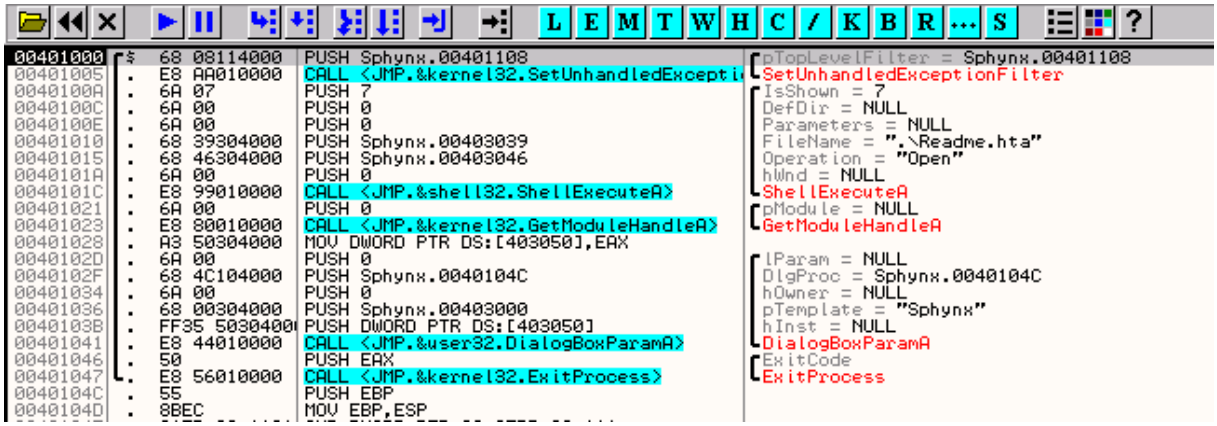
В этой главе мы продолжим изучение противотладочных приёмов, а конкретнее – рассмотрим сразу два одновременно, так как они работают в паре. В этом туториале мы будем использовать пропатченный в 21-ой главе OllyDbg, который называется NVP11, с плагином HideDebugger, сконфигурированным следующим образом.



Здесь видим, что среди настроек есть опция под названием UNHANDLED EXCEPTION TRICKS. Этот противоотладочный приём мы и будем изучать. Он работает совместно с другим приёмом, который можно использовать и отдельно – это API-функция ZwQueryInformationProcess, применяющаяся для обнаружения отладчика.

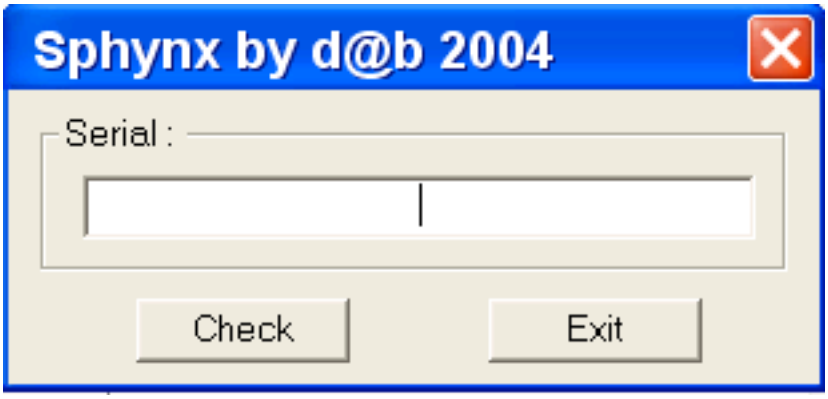
Используем крэкми, который идёт в файле sphynx.zip (находится в архиве `wasm_files.zip`: `files/0476/sphynx.rar`). Как вы уже могли догадаться, если включить вышеозначенную опцию, то он выполнится без каких-либо проблем, но нам нужно исследовать, как работают оба метода.

Сразу скажу, что сейчас мы не будем решать этот крэкми, а займёмся гораздо позже. Чтобы найти для него правильный серийный номер, нужно использовать метод «грубой силы», который будет рассматривать в свой черёд, а пока что просто запустим этот крэкми в OllyDbg и изучим вышеупомянутый метод защиты.



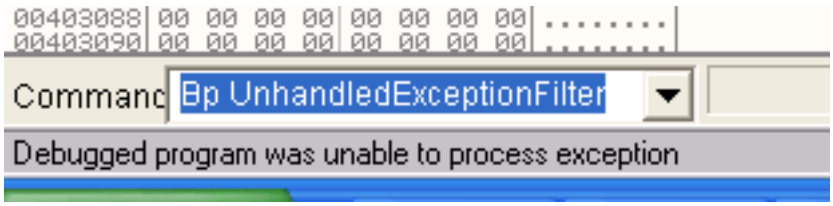
Загружаем крэкми в Nvp11 (пропатченный OllyDbg) и проверяем настройки плагина HideDebugger, которые должны соответствовать нижеприведённой картинке (отмечены все исключения).

Делаем RUN.



И появляется главное окно крэкми, а защита сработает после него.

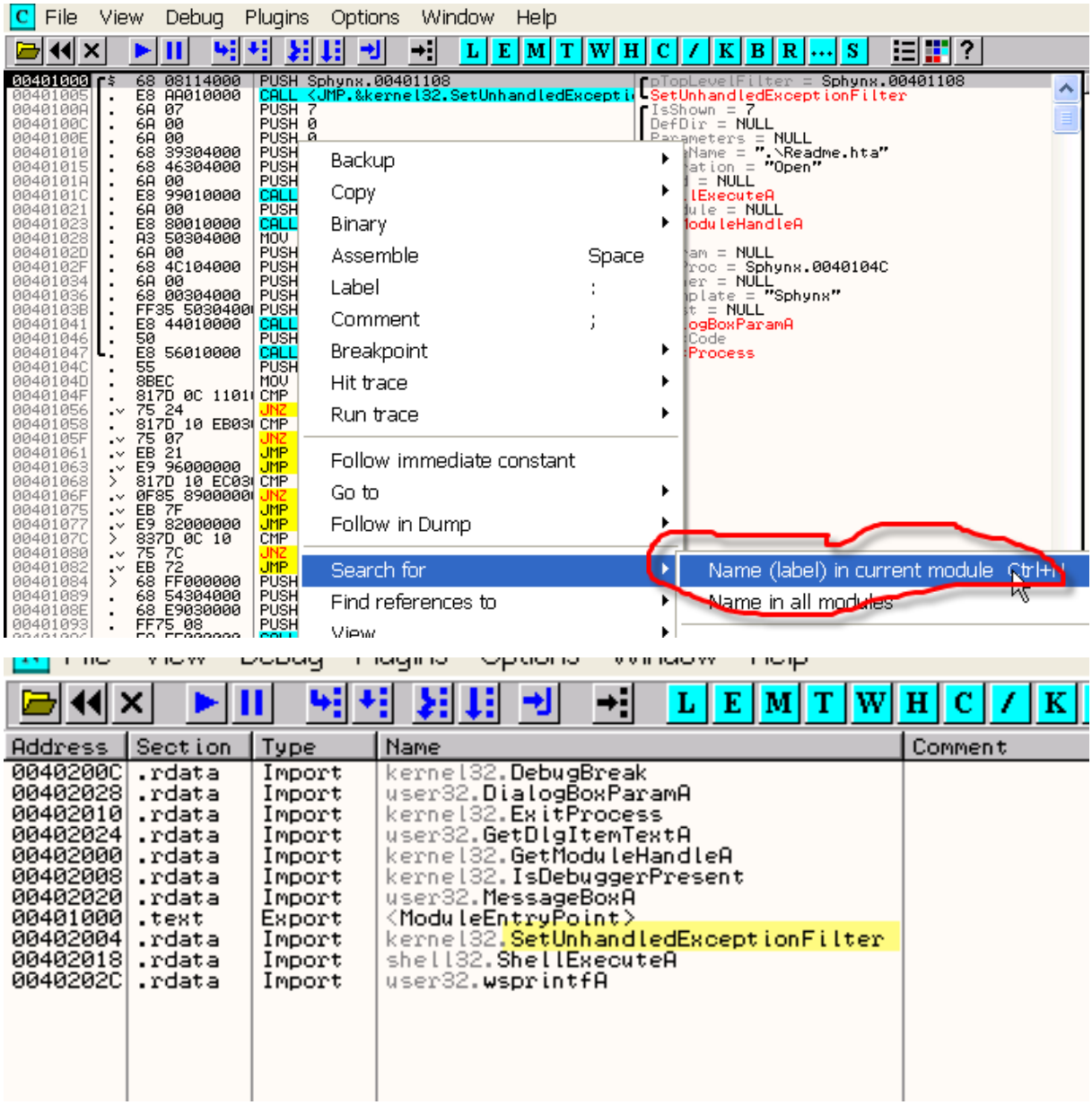
Вводим неправильный серийный номер и нажимаем "check".



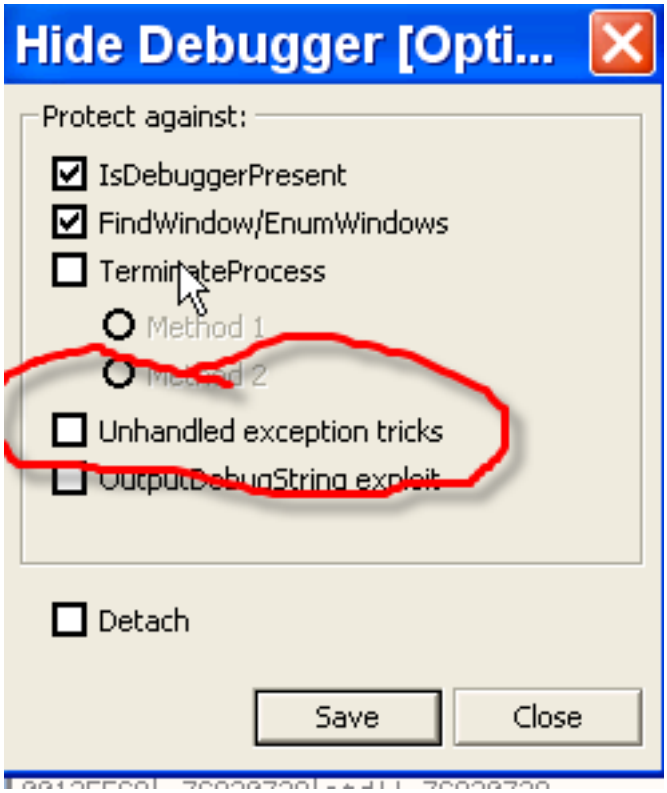
Нам говорят, что невозможно обработать исключение, и программа закроется, если сделаем RUN.

Ок, если бы мы запустили крэкми не из-под OllyDbg, то крэкми бы не закрылся, а продолжил бы работать и дал возможность попробовать другие серийные номера.

Ладно, перезагрузим крэкми и посмотрим, какие API-функции используются.



Помните, что у плагина HideDebugger есть настройка для защиты от этого приёма.



Но прежде, чем её включать, мы должны посмотреть своими глазами, как работает этот метод.

Смотрим описание SetUnhandledExceptionFilter.

SetUnhandledExceptionFilter Quick Info

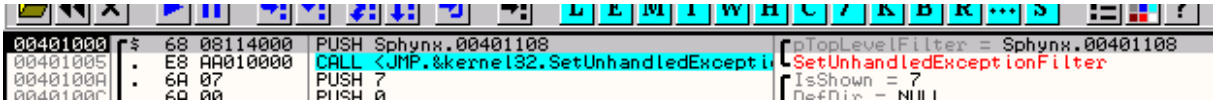
The **SetUnhandledExceptionFilter** function lets an application supersede the top-level exception handler that Win32 places at the top of each thread and process.

After calling this function, if an exception occurs in a process that is not being debugged, and the exception makes it to the Win32 unhandled exception filter, that filter will call the exception filter function specified by the *lpTopLevelExceptionFilter* parameter.

```
LPTOP_LEVEL_EXCEPTION_FILTER SetUnhandledExceptionFilter(
  LPTOP_LEVEL_EXCEPTION_FILTER lpTopLevelExceptionFilter        // exception filter function
);
```

Видим, что единственным параметром функции является адрес, откуда начинать выполнение, когда программа вызывает исключение, всегда когда программа НЕ НАХОДИТСЯ ПОД ОТЛАДКОЙ, хе-хе, что можно использовать для определения, отлаживается ли программа или нет, и не использовать ZwQueryInformationProcess.

Вернёмся к программе, как раз к тому месту, где вызывается эта API-функция.

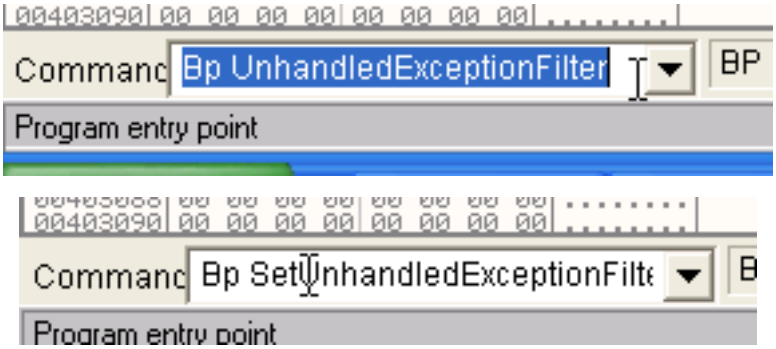


Как видим, один-единственный параметр - адрес, откуда продолжается выполнение, когда программы вызывает исключение, если она не находится под отладкой, хе-хе. В этом случае управление перейдёт по адресу 401108, в противном случае, если программа отлаживается, по адресу 401108.

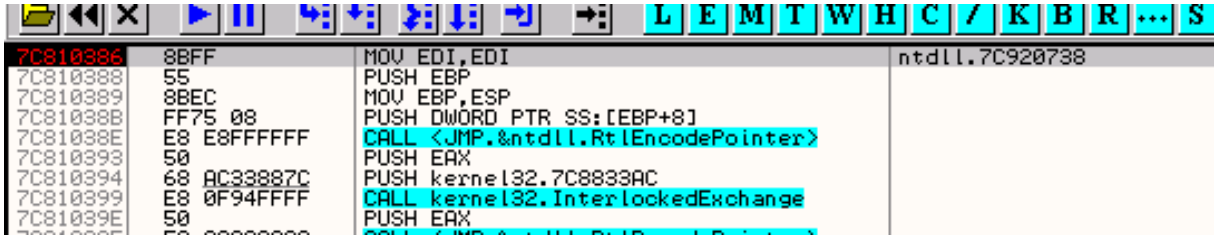
То есть, мы увидели, что программа может устанавливать обработчики исключений, и когда она использует один из них, то выполнение передаётся по заданному адресу, каждый раз, когда случается данное исключение, но работает этот только тогда, когда программа не находится под

отладкой.

Ладно, установим BP на обе функции: SetUnhandledExceptionFilter и UnhandledExceptionFilter, так как они - сёстры и работают вместе.

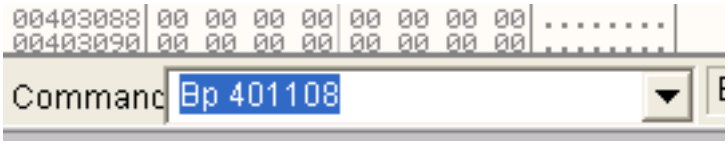


А теперь делаем RUN.

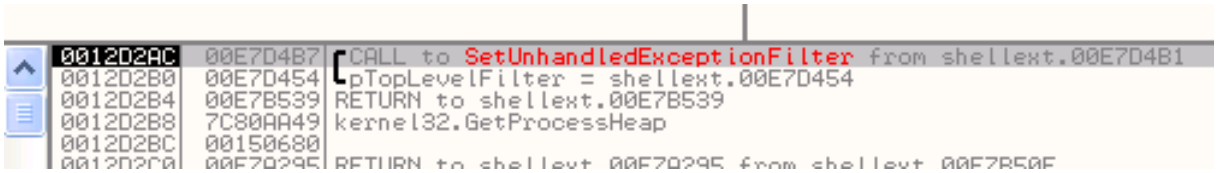


Программа должна продолжить выполнение по заданному адресу, когда возникнет исключение.

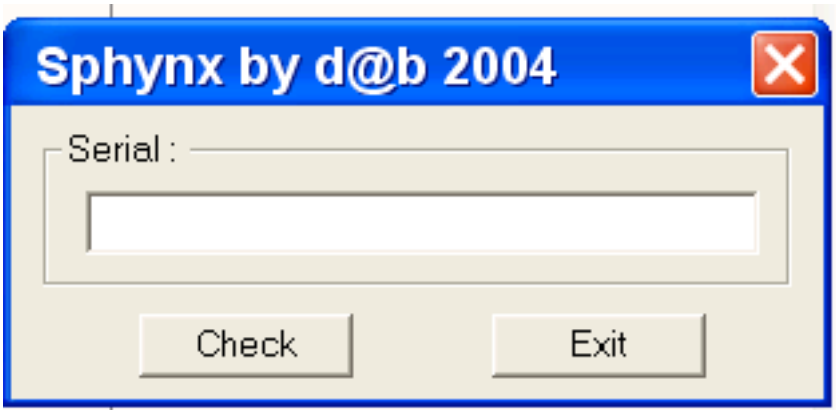
Как видим, адрес - 401108, установим на него BP.



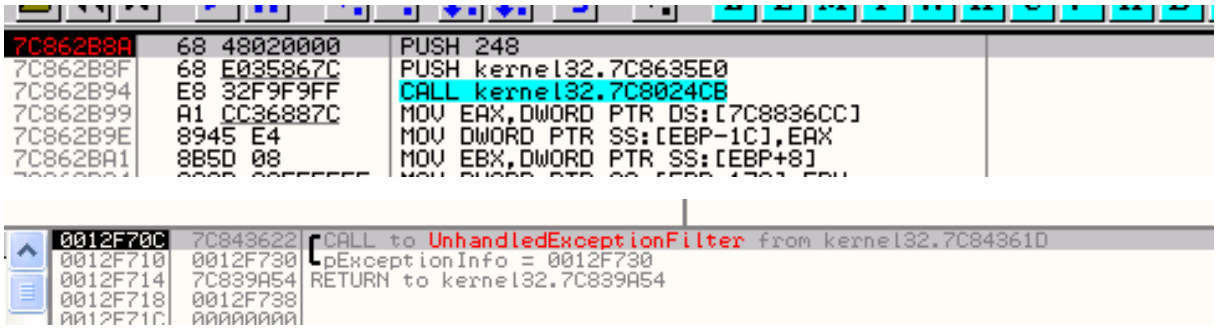
Теперь делаем RUN и наблюдаем за вызывающейся из DLL-библиотек функцией SetUnhandledExceptionFilter.



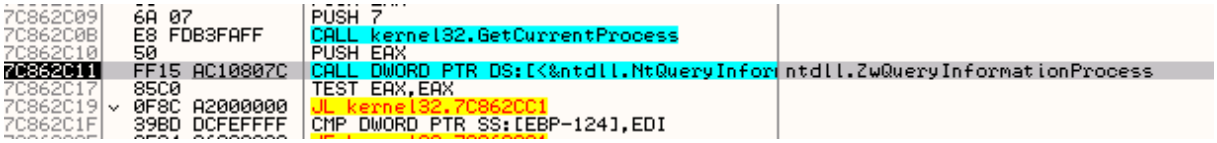
Нам это не интересно, так как мы уже узнали, где находится обработчик, установленный программой, поэтому уберём с этой функции точку останова.



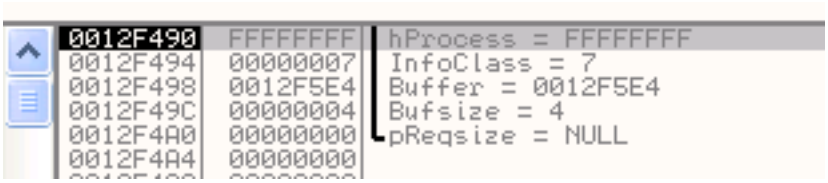
Теперь введём неправильный серийный номер, нажмём CHECK и остановимся на другой API-функции в UnhandledExceptionFilter, так как возникло исключение, которое, разумеется, было сгенерировано программой нарочно, чтобы сюда попасть, и эта функция определит, отлаживается программа или нет, и таким образом управление перейдёт или по адресу 401108, или в мусорную корзину, хе-хе.



Эта API-функция проверяет, не отлаживается ли процесс, и осуществляет переход в 401108. Оттрасируем функцию с помощью F8, чтобы узнать, как она определяет отладчик.

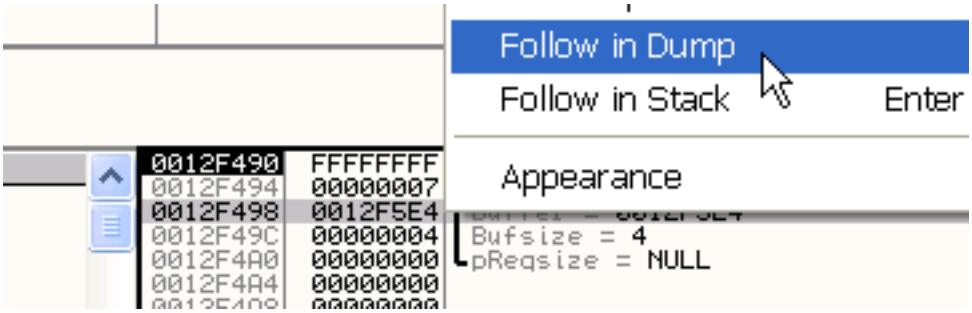


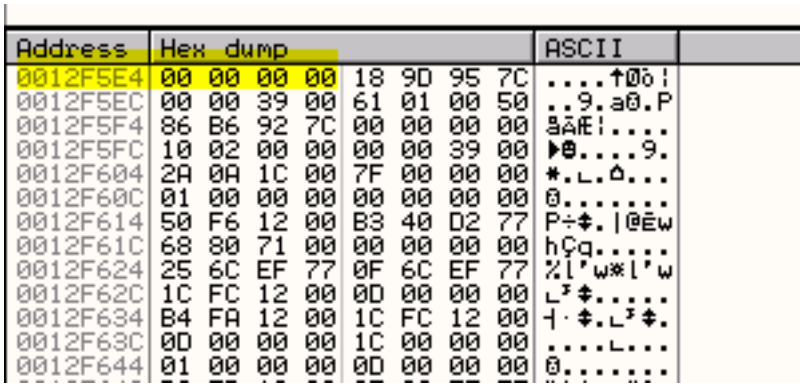
Прибыли, здесь используется второй антиотдочный приём, который мы сегодня рассмотрим. Его можно применять отдельно, и заключается он в вызыве ZwQueryInformationProcess напрямую с параметром INFO CLASS = 7.



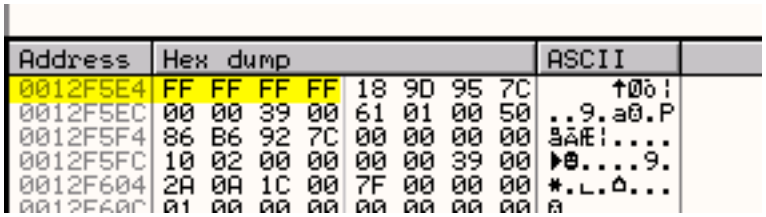
Эта API-функция возвращает информацию о выполняющемся процессе и с параметром INFOCLASS = 7 возвращает через буфер информацию о том, отлаживается ли процесс или нет.

Смотрим, что находится в буфере через DUMP.

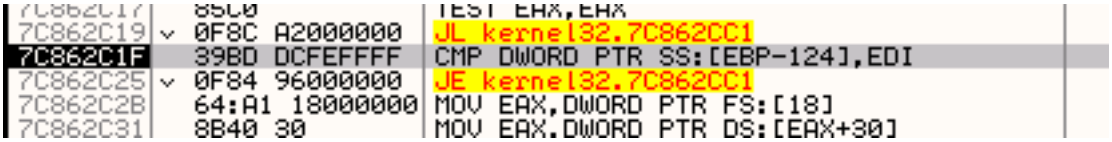




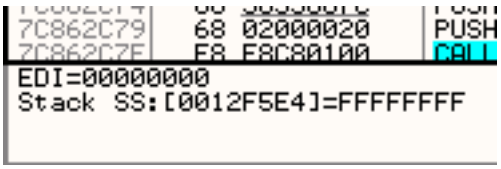
Это буфер, видим, что его размер равен 4 байтам, и эта API-функция возвращает FFFFFFFF, если процесс находится под отладкой, и ноль, если нет. С помощью F8 оттрасируем дальше, чтобы посмотреть, что будет возвращено в этот раз.



Видим, что было возвращено значение FFFFFFFF, то есть процесс отлаживается, мой друг, хе-хе.



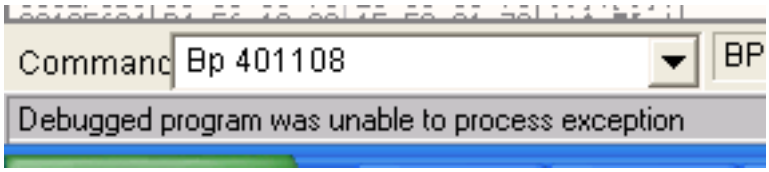
Видим, что затем проводится сравнение, равно ли это значение нулю или нет, так как EDI равно нулю.



В данном случае перехода не происходит.

```
7C862C11   FF15 AC10807C   CALL DWORD PTR DS:[<&ntdll.NtQueryInfor ntdll.ZwQueryInfo
7C862C17   85C0             TEST EAX,EAX
7C862C19   0F8C A2000000    JL kernel32.7C862CC1
7C862C1F   39BD DCFEFFFF    CMP DWORD PTR SS:[EBP-124],EDI
7C862C25   0F84 96000000    JE kernel32.7C862CC1
7C862C2B   64:A1 18000000   MOV EAX,DWORD PTR FS:[18]
7C862C31   8B40 30          MOV EAX,DWORD PTR DS:[EAX+30]
7C862C34   F640 69 01       TEST BYTE PTR DS:[EAX+69],1
7C862C38   0F84 F1070000    JE kernel32.7C86342F
7C862C3E   8B03             MOV EAX,DWORD PTR DS:[EBX]
7C862C40   3930             CMP DWORD PTR DS:[EAX],ESI
7C862C42   75 3F            JNZ SHORT kernel32.7C862C83
7C862C44   6A 01            PUSH 1
7C862C46   68 0854887C      PUSH kernel32.7C885408
7C862C4B   E8 5D6BFAFF      CALL kernel32.InterlockedExchange
7C862C50   85C0             TEST EAX,EAX
7C862C52   75 2F            JNZ SHORT kernel32.7C862C83
7C862C54   8B03             MOV EAX,DWORD PTR DS:[EBX]
7C862C56   68 C435867C      PUSH kernel32.7C8635C4          ASCII ".cxr (conte
7C862C5B   FF73 04          PUSH DWORD PTR DS:[EBX+4]
7C862C5E   68 AC35867C      PUSH kernel32.7C8635AC          ASCII ".exr (excep
7C862C63   50               PUSH EAX
7C862C64   68 8C35867C      PUSH kernel32.7C86358C          ASCII "Code perfo
7C862C69   FF70 0C          PUSH DWORD PTR DS:[EAX+C]
7C862C6C   68 6C35867C      PUSH kernel32.7C86356C          ASCII "Invalid ad
7C862C71   FF70 18          PUSH DWORD PTR DS:[EAX+18]
7C862C74   68 3835867C      PUSH kernel32.7C863538          ASCII "access vio
7C862C79   68 02000020      PUSH 20000002
7C862C7E   E8 E8C80100      CALL <JMP.&ntdll.RtlApplicationVerifier
Jump is NOT taken
7C862CC1=kernel32.7C862CC1
```

Если буфер не равен нулю, перехода не происходит и программа завершается.



Видим, что выдаётся ошибка, а затем приложение завершается.

Теперь перезапустим его и повторим весь процесс, но на этот раз изменим результат, возвращаемый ZwQueryInformationProcess.

Когда прибудем в

```
7C862C08   50               PUSH EAX
7C862C09   6A 07            PUSH 7
7C862C0B   E8 FDB3FAFF      CALL kernel32.GetCurrentProcess
7C862C10   50               PUSH EAX
7C862C11   FF15 AC10807C    CALL DWORD PTR DS:[<&ntdll.NtQueryInfor ntdll.ZwQueryInformationProcess
7C862C17   85C0             TEST EAX,EAX
7C862C19   0F8C A2000000    JL kernel32.7C862CC1
7C862C1F   39BD DCFEFFFF    CMP DWORD PTR SS:[EBP-124],EDI
```

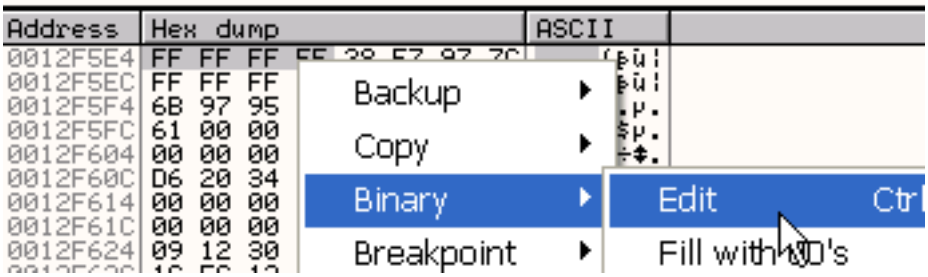
Посмотрим буфер через DUMP.

Address	Hex dump	ASCII
0012F5E4	00 00 00 00 28 E7 97 7C	....(ηù¦
0012F5EC	FF FF FF FF 07 E7 97 7C	····ηù¦
0012F5F4	6B 97 95 7C 00 00 E6 00	kùò¦..µ.
0012F5FC	61 00 00 50 78 24 E6 00	a..Px$µ.
0012F604	00 00 00 00 54 F6 12 00	....T÷↕.
0012F60C	D6 20 34 7C 00 00 E6 00	╓ 4¦..µ.
0012F614	00 00 00 00 DE 20 34 7C	....▐ 4¦

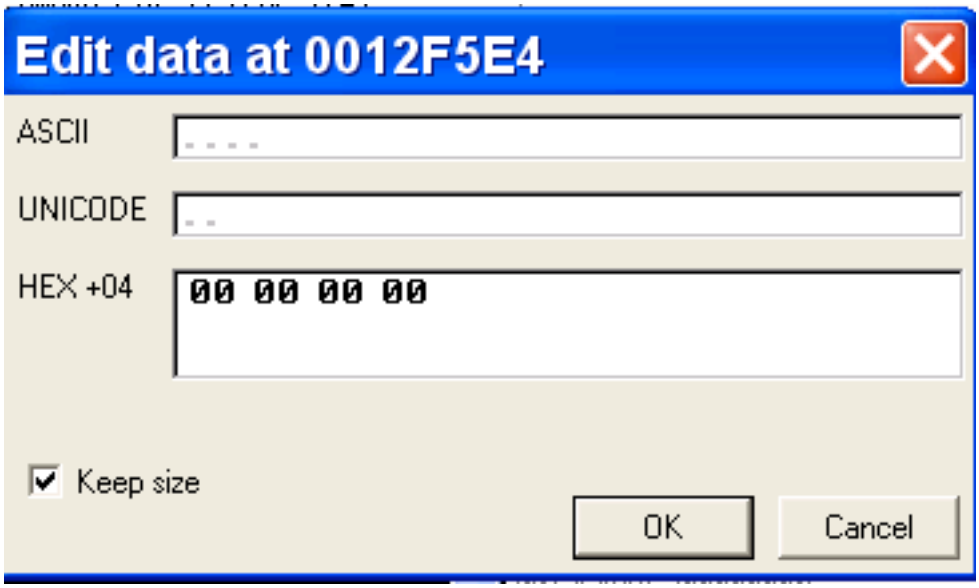
Пройдём по API-функции с помощью F8.

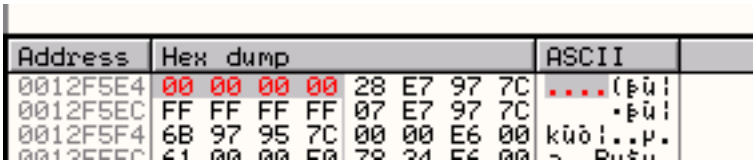
Address	Hex dump	ASCII
0012F5E4	FF FF FF FF 28 E7 97 7C	····(ηù¦
0012F5EC	FF FF FF FF 07 E7 97 7C	····ηù¦
0012F5F4	6B 97 95 7C 00 00 E6 00	kùò¦..µ.
0012F5FC	61 00 00 50 78 24 E6 00	a..Px$µ.
0012F604	00 00 00 00 54 F6 12 00	....T÷↕.

В буфере лежит то же самое, что и в прошлый раз - FFFFFFF, так как программа под отладкой. Изменим на ноль.

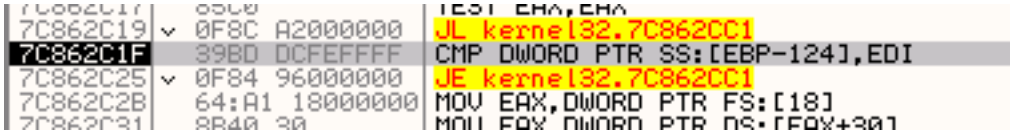


Меняем всё на нули.



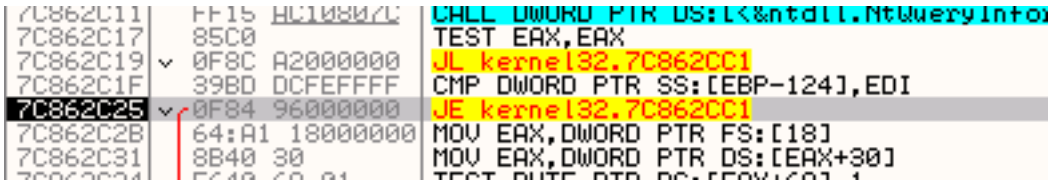


Доходим до сравнения.



Так как оба операнда равны нулю

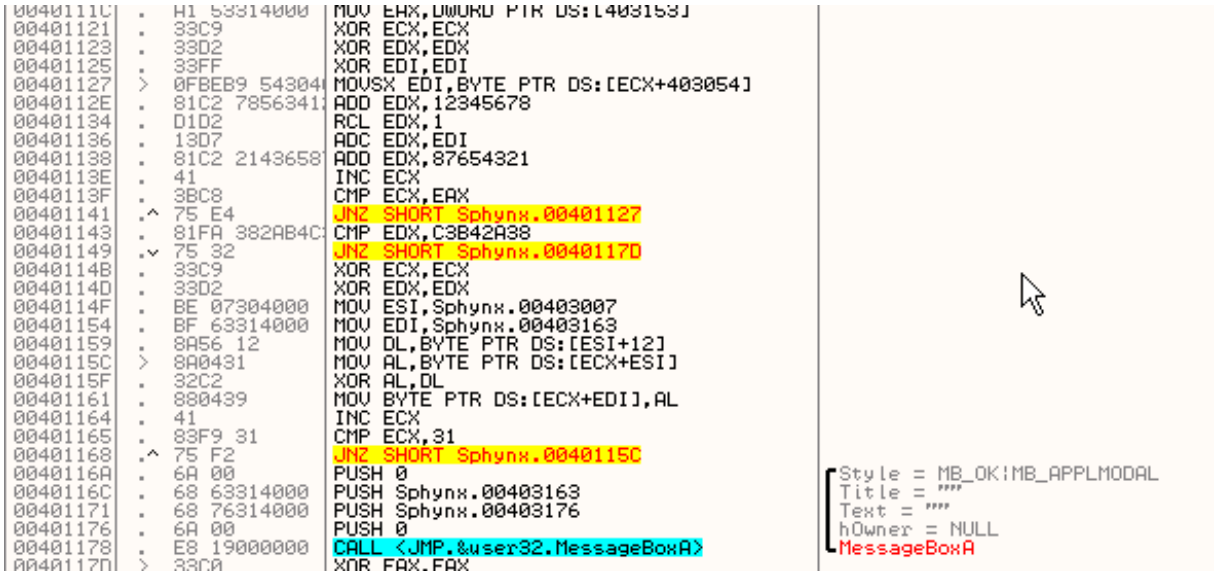
Теперь переход сработает.



И если сделаем RUN.

```
00401108  .  8B75 08        MOV ESI,DWORD PTR SS:[EBP+8]
0040110B  .  8B46 04        MOV EAX,DWORD PTR DS:[ESI+4]
0040110E  .  05 B8000000    ADD EAX,0B8
00401113  .  8BF0           MOV ESI,EAX
00401115  .  8B00           MOV EAX,DWORD PTR DS:[EAX]
00401117  .  83C0 0E        ADD EAX,0E
0040111A  .  8906           MOV DWORD PTR DS:[ESI],EAX
0040111C  .  A1 53314000    MOV EAX,DWORD PTR DS:[403153]
00401121  .  33C9           XOR ECX,ECX
00401123  .  33D2           XOR EDX,EDX
```

В этот раз прибываем в 401108 и ниже видим интересующее нас место.

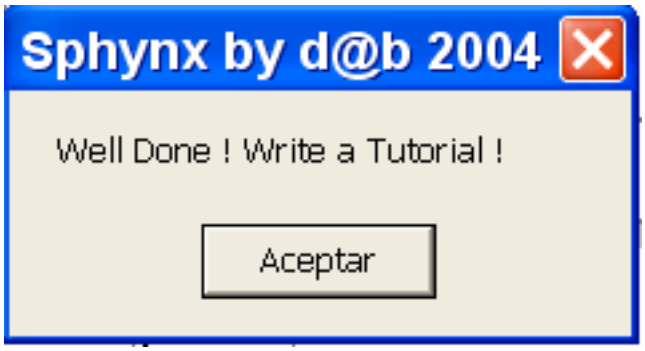


И окошко с сообщением о решении крэкми, можем проверить.

Если изменить переход так, чтобы он не срабатывал, то будет показано сообщение о том, что крэкми был решён.

```
00401141  .^ 75 E4         JNZ SHORT Sphynx.00401127
00401143  .  81FA 382AB4C3 CMP EDX,C3B42A38
00401149  .v 75 32         JNZ SHORT Sphynx.0040117D
0040114B  .  33C9          XOR ECX,ECX
0040114D  .  33D2          XOR EDX,EDX
0040114F  .  BE 07304000   MOV ESI,Sphynx.00403007
00401154  .  BF 63314000   MOV EDI,Sphynx.00403163
00401159  .  8A56 12       MOV DL,BYTE PTR DS:[ESI+12]
0040115C  >  8A0431        MOV AL,BYTE PTR DS:[ECX+ESI]
0040115F  .  32C2          XOR AL,DL
00401161  .  880439        MOV BYTE PTR DS:[ECX+EDI],AL
00401164  .  41            INC ECX
00401165  .  83F9 31       CMP ECX,31
00401168  .^ 75 F2         JNZ SHORT Sphynx.0040115C
0040116A  .  6A 00         PUSH 0                          Style = MB_OK|MB_APPLMODAL
0040116C  .  68 63314000   PUSH Sphynx.00403163            Title = ""
00401171  .  68 76314000   PUSH Sphynx.00403176            Text = ""
00401176  .  6A 00         PUSH 0                          hOwner = NULL
00401178  .  E8 19000000   CALL <JMP.&user32.MessageBoxA>  MessageBoxA
0040117D  >  33C0          XOR EAX,EAX
0040117F  .  48            DEC EAX
```

Теперь делаем RUN.



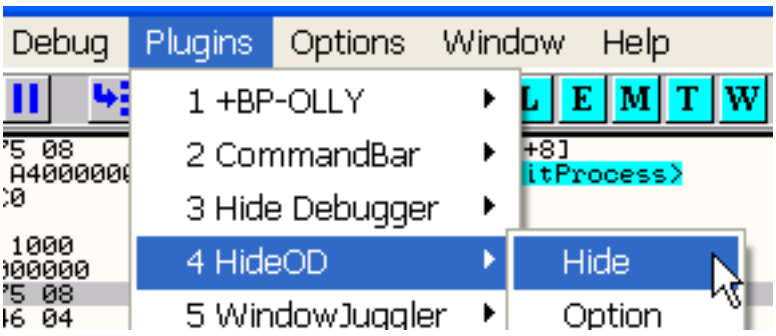
Ок, как уже говорилось, всего этого можно избежать, если включим в плагине HideDebugger настройку UnhandledExceptionTricks, тогда этот крэкми будет работать без каких-либо проблем.

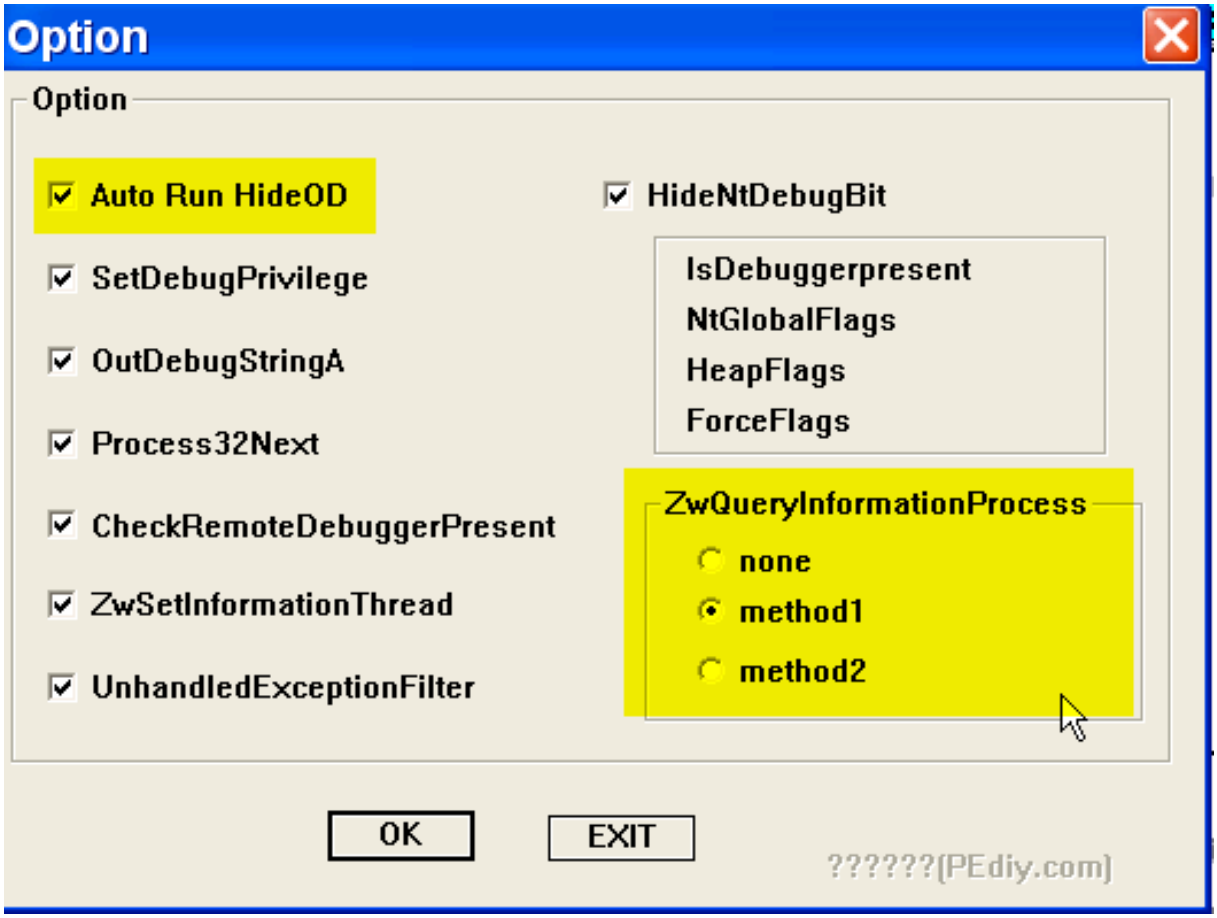
Осталось рассмотреть ещё один вопрос - каки избежать обнаружения с помощью API-функции ZwQueryInformationProcess, когда она применяется напряму, а не через UnhandledExceptionFilter.

Конечно, обойти её вручную несложно - достаточно изменить на ноль значение, возвращаемое ею, но существует ли плагин, который позволяет сделать это автоматически?

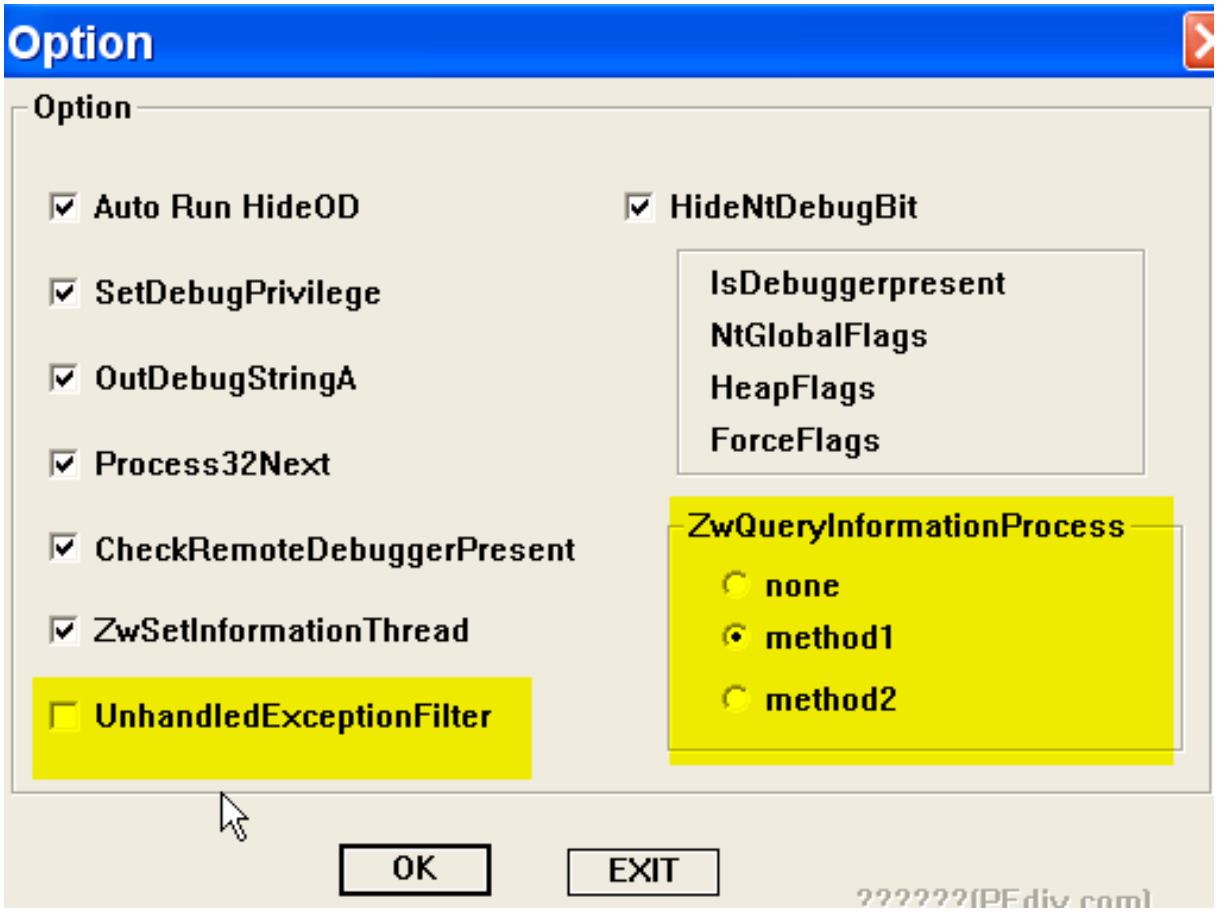
HideOD (находится в архиве `wasm_files.zip`: `files/0476/hideod.rar`)

Этот плагин обеспечивает защиту от вышеуказанной функции без необходимости менять что-либо вручную.





Видим, что здесь есть настройки, которых нет в HideDebugger, так что они могут дополнять друг друга, но главное, не забудьте отметить "Auto Run HideOD", чтобы вам не приходилось нажимать "HIDE" каждый раз, когда вы запускаете OllyDbg.



Видим, что несмотря на отключённую защиту от UnhandledExceptionFilter, так как работает противодействие ZwQueryInformationProcess, то это нас обезопасило в равной мере, так как именно эта функция используется для нахождения отладчика.

Мы научились обходить защиту с помощью UnhandledExceptionFilter, обнаруживающую защиту посредством ZwQueryInformationProcess. В этом нам помог плагин HideOD, который в сочетании с HideDebugger делает нашу пропатченную Олли чуть ли не крепостью, хе-хе.

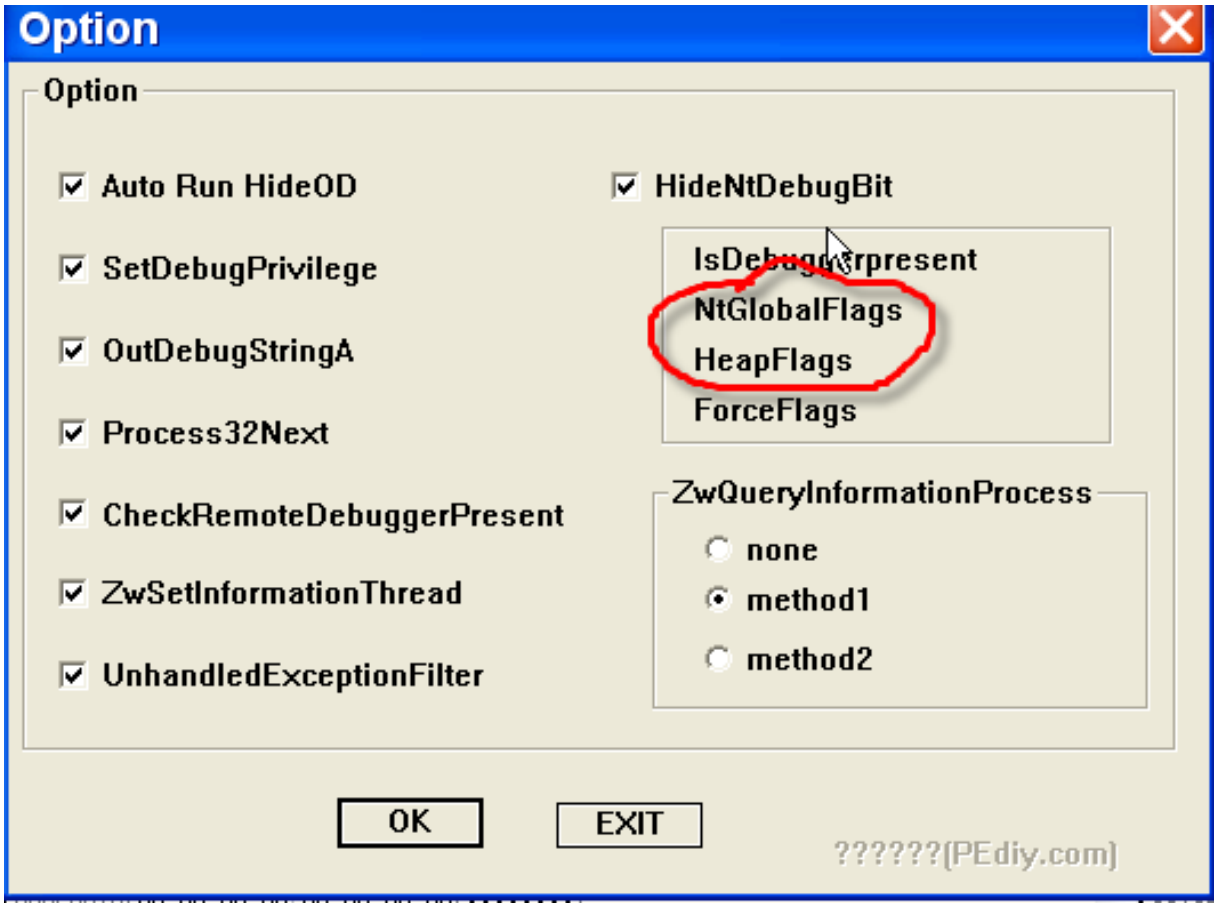
Введение в крэкинг с нуля, используя OllyDbg - Глава 23

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-02-27

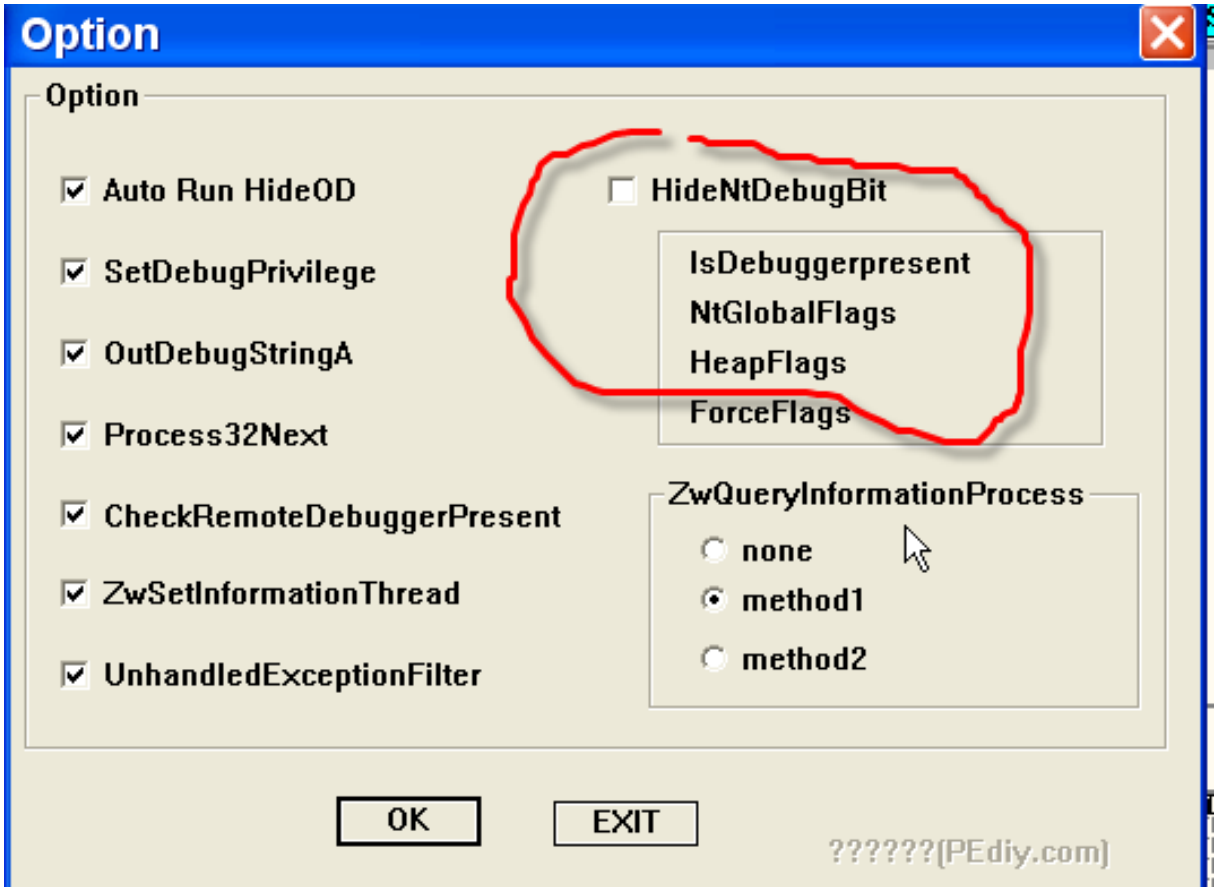
Это последняя глава, посвящённая антиотладке, в которой рассказывается о флагах ProcessHeap и NTGlobalFlag, и как они используются. Таким образом, мы получим представление о самых известных антиотладочных приёмах. конечно, это не все, но думаю, что это все основных, которые нужно знать и которые обычно применяются. Конечно, есть протекторы вроде execryptor, который является чемпионом по обнаружению OllyDbg и использует кроме приёмов, описанных здесь, ещё 4 или 5, но они весьма специфичны, и прочитать о них можно в туториалах Хуана Хосе об этом протекторе, а также нужно знать, что в каждой следующей версии execryptor'a добавляются новые приёмы, поэтому для этого нужно изучать уже не антиотладку вообще, а конкретно данный протектор.

Мы уже знаем, что плагины позволяют нам избежать антиотладки, поэтому сначала нужно выставить правильные настройки.



Это плагин HideOD, красным обведены опции, которые прячут OllyDbg от обнаружения с помощью этих флагов, но мы будем исследовать вручную.

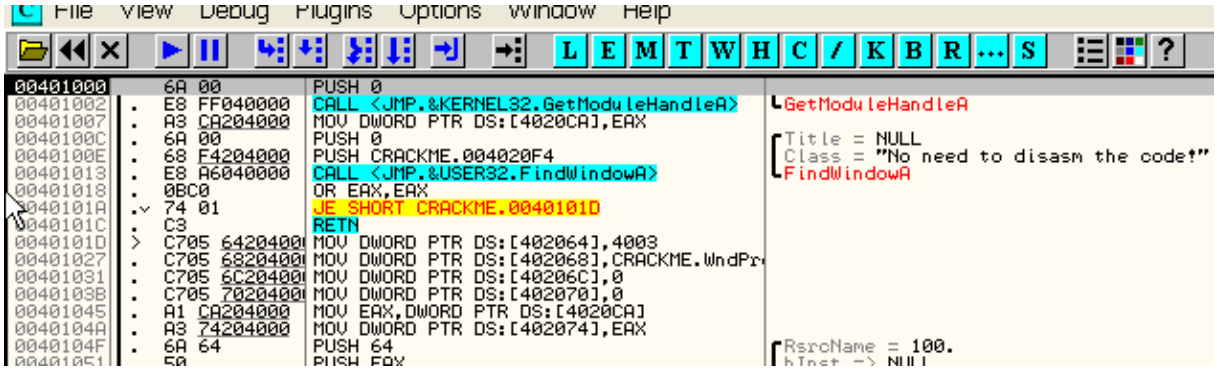
Эти два флага показывают, что процесс находится под отладкой, их легко найти. Если вы не можете вспомнить, как найти область, где находится байт IsDebuggerPresent, то прочитайте главу 19, в которой объясняется как найти её врунчную, так как эти флаги располагаются неподалёку.



Пока отключим эту настройку, чтобы посмотреть, какие значения у вышеуказанных флагов, а иначе плагин их скроет.

Чтобы попрактиковаться с этими флагами, будем использовать крэкми Cruehead'a I, в котором и отыщем оба из них.

Откроем его в OllyDbg, предварительно убедившись, что настройка, указанная на картинке выше, отключена.

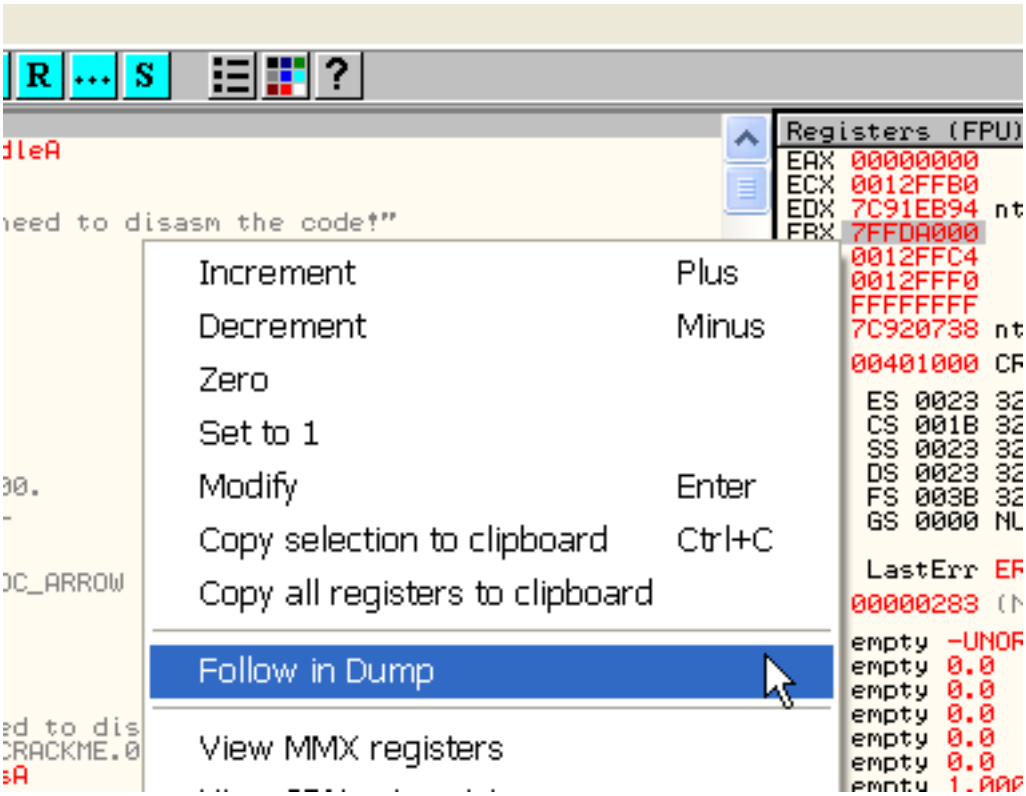


Хорошо, теперь посмотрим, как найти и изменить оба эти флага вручную.

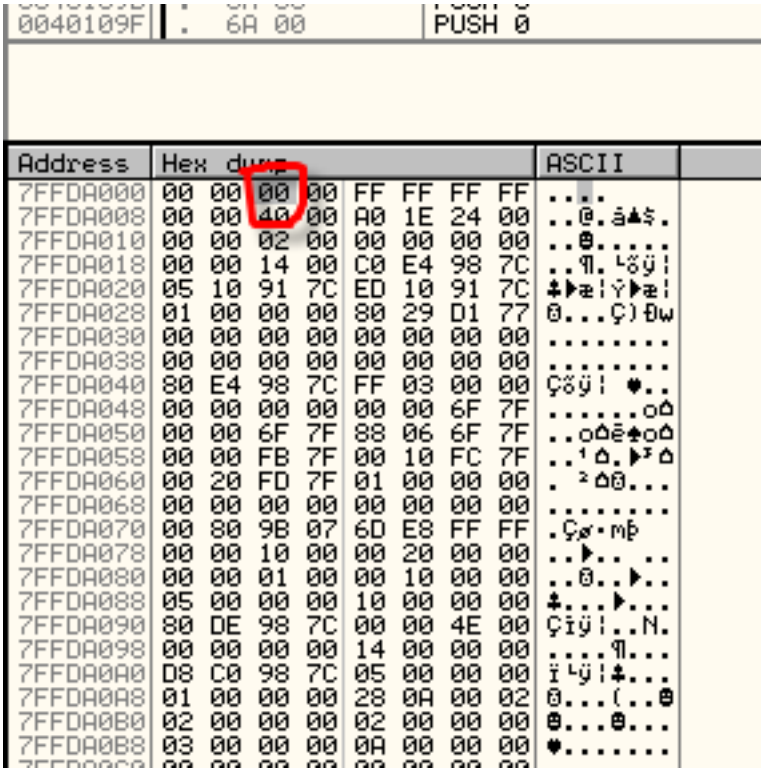
Их можно найти в той же области, где находится и байт IsDebuggerPresent, о котором рассказывалось в 19-ой главе. Простой способ заключается в том, чтобы прямо на точке входа отметить EBX и выбрать FOLLOW IN DUMP, а о полном варианте, если вы его забыли, прочитайте в вышеуказанной главе.

Находим на точке входа.

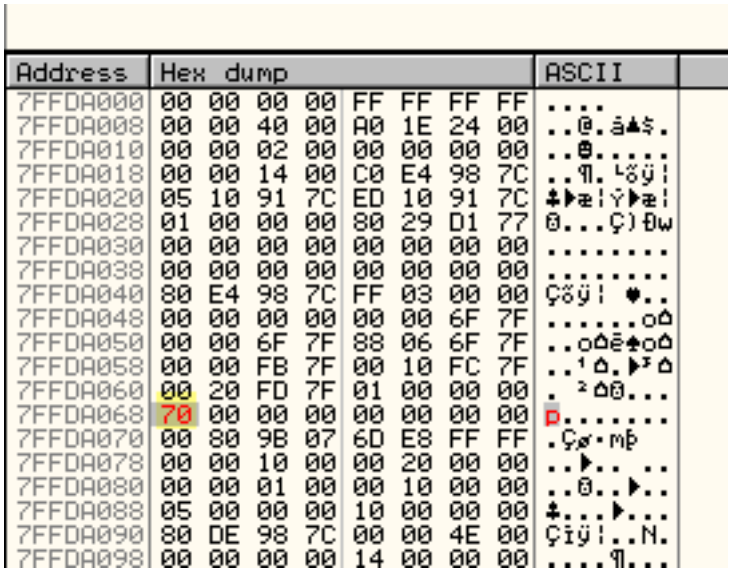
Отмечаем EBX-FOLLOW IN DUMP.



В дампе видим нужную область, помните, что её местонахождение у вас может отличаться, более того, оно меняется каждый раз, когда программа перезапускается.

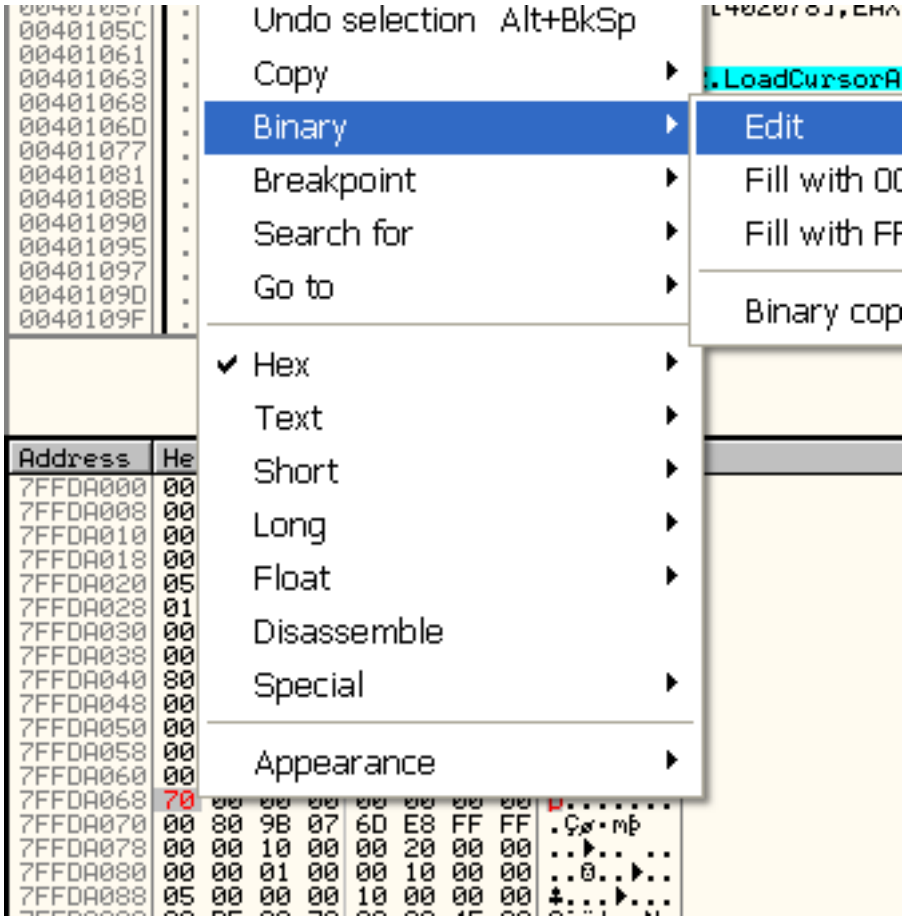


Разумеется, мы помним, что этот байт заполняется API-функцией IsDebuggerPresent, а NTGlobalFlag находится по соседству - надо прибавить 68 к адресу, который находился в EBX, в моём случае EBX содержал 7ffda000, прибавляем 68 и получается 7ffda068.

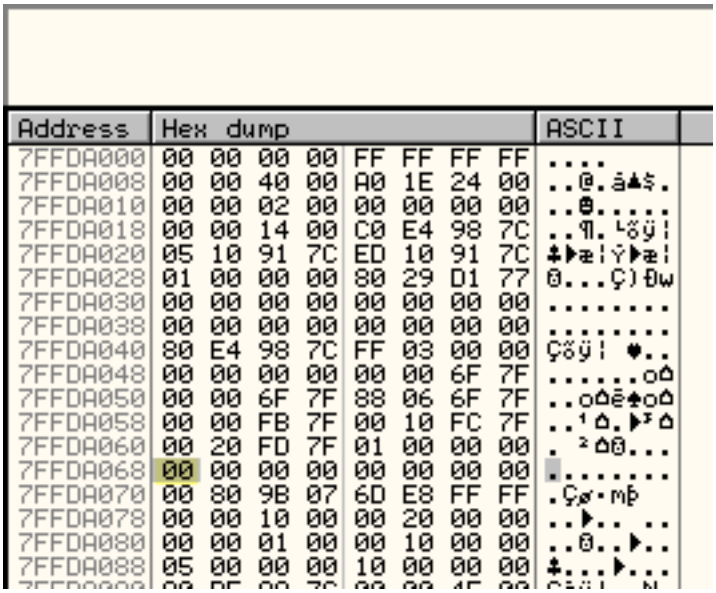


Address	Hex dump	ASCII
7FFDA000	00 00 00 00 FF FF FF FF	....
7FFDA008	00 00 40 00 A0 1E 24 00	..@.á▲$.
7FFDA010	00 00 02 00 00 00 00 00	..☻.....
7FFDA018	00 00 14 00 C0 E4 98 7C	..¶.└öÿ¦
7FFDA020	05 10 91 7C ED 10 91 7C	♣►æ¦ý►æ¦
7FFDA028	01 00 00 00 80 29 D1 77	☺...Ç)Ðw
7FFDA030	00 00 00 00 00 00 00 00	....
7FFDA038	00 00 00 00 00 00 00 00	....
7FFDA040	80 E4 98 7C FF 03 00 00	Çöÿ¦ ♥..
7FFDA048	00 00 00 00 00 00 6F 7F	......o⌂
7FFDA050	00 00 6F 7F 88 06 6F 7F	..o⌂ê♠o⌂
7FFDA058	00 00 FB 7F 00 10 FC 7F	..¹⌂.►²⌂
7FFDA060	00 20 FD 7F 01 00 00 00	. ²⌂☺...
7FFDA068	70 00 00 00 00 00 00 00	p.......
7FFDA070	00 80 9B 07 6D E8 FF FF	.Çø·mÞ
7FFDA078	00 00 10 00 00 20 00 00	..►.. ..
7FFDA080	00 00 01 00 00 10 00 00	..☺..►..
7FFDA088	05 00 00 00 10 00 00 00	♣...►...
7FFDA090	80 DE 98 7C 00 00 4E 00	Çiÿ¦..N.
7FFDA098	00 00 00 00 14 00 00 00	....¶...

Это и есть знаменитый NTGlobalFlag, значение которого отличается от нуля, если есть отладчик и ноль, если нет, так что можем вручную выставить его значение в ноль.



Помещаем сюда ноль.

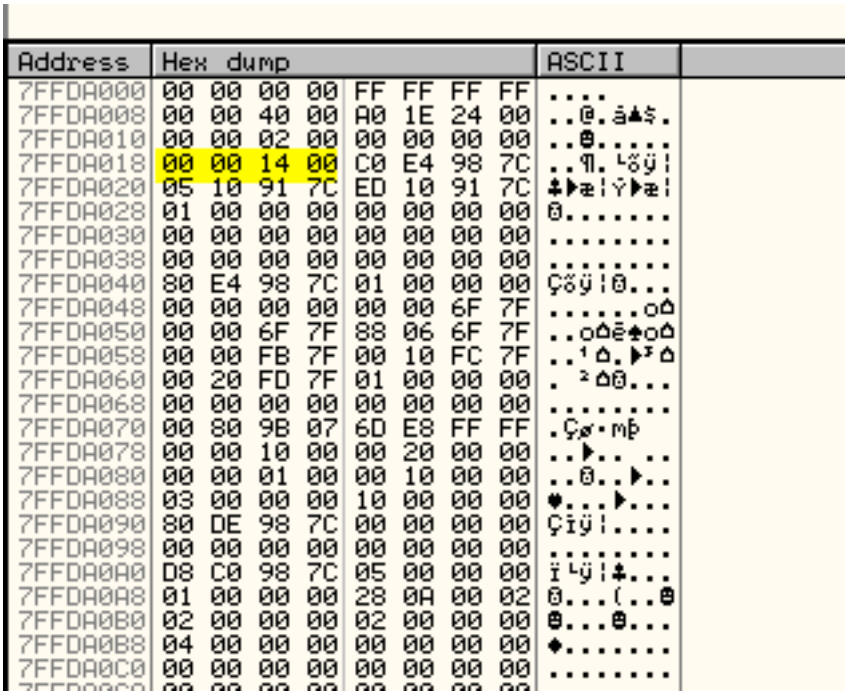


Address	Hex dump	ASCII
7FFDA000	00 00 00 00 FF FF FF FF	....
7FFDA008	00 00 40 00 A0 1E 24 00	..@.á▲$.
7FFDA010	00 00 02 00 00 00 00 00	..☻.....
7FFDA018	00 00 14 00 C0 E4 98 7C	..¶.└öÿ¦
7FFDA020	05 10 91 7C ED 10 91 7C	♣►æ¦ý►æ¦
7FFDA028	01 00 00 00 80 29 D1 77	☺...Ç)Ðw
7FFDA030	00 00 00 00 00 00 00 00	....
7FFDA038	00 00 00 00 00 00 00 00	....
7FFDA040	80 E4 98 7C FF 03 00 00	Çöÿ¦ ♥..
7FFDA048	00 00 00 00 00 00 6F 7F	......o⌂
7FFDA050	00 00 6F 7F 88 06 6F 7F	..o⌂ë♠o⌂
7FFDA058	00 00 FB 7F 00 10 FC 7F	..¹⌂.►³⌂
7FFDA060	00 20 FD 7F 01 00 00 00	. ²⌂☺...
7FFDA068	00 00 00 00 00 00 00 00	....
7FFDA070	00 80 9B 07 6D E8 FF FF	.Çø·mþ
7FFDA078	00 00 10 00 00 20 00 00	..►.. ..
7FFDA080	00 00 01 00 00 10 00 00	..☺..►..
7FFDA088	05 00 00 00 10 00 00 00	♣...►...

Так мы нашли флаг NtGlobalFlag и поместили в него ноль.

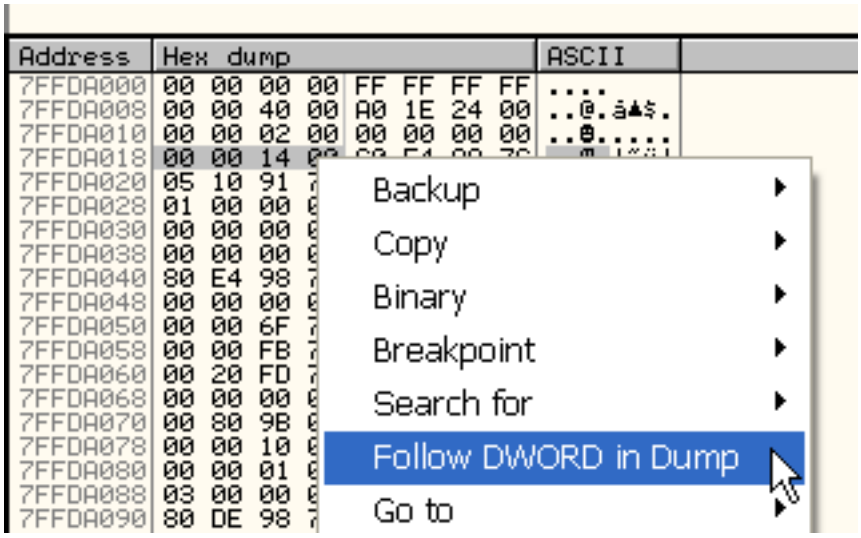
Теперь нужно найти другой флаг ProcessHeap, что также нетрудно сделать.

К значение, которое содержится в EBX на точке входа, надо прибавить 18, это адрес, по которому в моём случае лежит 014000, это тоже адрес - адрес "кучи", то есть область памяти, созданная при запуске программы, куда та может помещать какие-то свои данные, не будем заострять на этом много внимания.

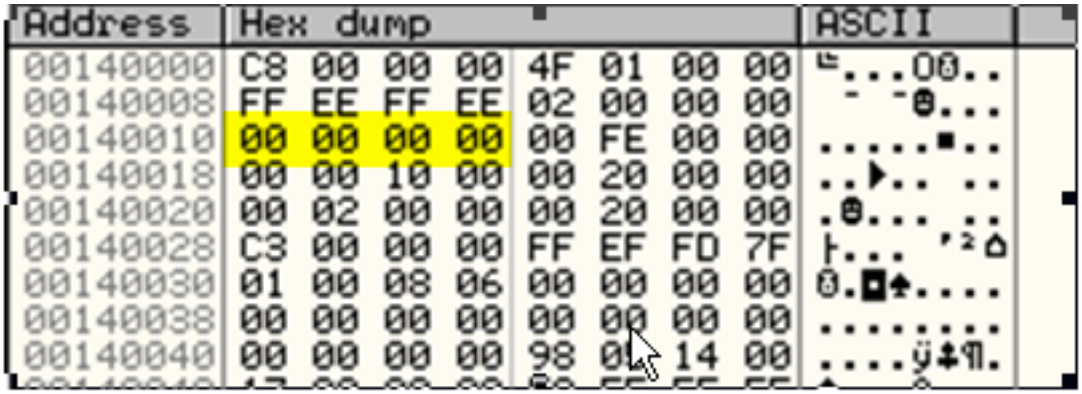


Address	Hex dump	ASCII
7FFDA000	00 00 00 00 FF FF FF FF	....
7FFDA008	00 00 40 00 A0 1E 24 00	..@.á▲$.
7FFDA010	00 00 02 00 00 00 00 00	..☻.....
7FFDA018	00 00 14 00 C0 E4 98 7C	..¶.└öÿ¦
7FFDA020	05 10 91 7C ED 10 91 7C	♣►æ¦ý►æ¦
7FFDA028	01 00 00 00 00 00 00 00	☺.....
7FFDA030	00 00 00 00 00 00 00 00	....
7FFDA038	00 00 00 00 00 00 00 00	....
7FFDA040	80 E4 98 7C 01 00 00 00	Çöÿ¦☺...
7FFDA048	00 00 00 00 00 00 6F 7F	......o⌂
7FFDA050	00 00 6F 7F 88 06 6F 7F	..o⌂ë♠o⌂
7FFDA058	00 00 FB 7F 00 10 FC 7F	..¹⌂.►³⌂
7FFDA060	00 20 FD 7F 01 00 00 00	. ²⌂☺...
7FFDA068	00 00 00 00 00 00 00 00	....
7FFDA070	00 80 9B 07 6D E8 FF FF	.Çø·mþ
7FFDA078	00 00 10 00 00 20 00 00	..►.. ..
7FFDA080	00 00 01 00 00 10 00 00	..☺..►..
7FFDA088	03 00 00 00 10 00 00 00	♥...►...
7FFDA090	80 DE 98 7C 00 00 00 00	Çiÿ¦....
7FFDA098	00 00 00 00 00 00 00 00	....
7FFDA0A0	D8 C0 98 7C 05 00 00 00	ï└ÿ¦♣...
7FFDA0A8	01 00 00 00 28 0A 00 02	☺...(..☻
7FFDA0B0	02 00 00 00 02 00 00 00	☻...☻...
7FFDA0B8	04 00 00 00 00 00 00 00	♦.....
7FFDA0C0	00 00 00 00 00 00 00 00	....

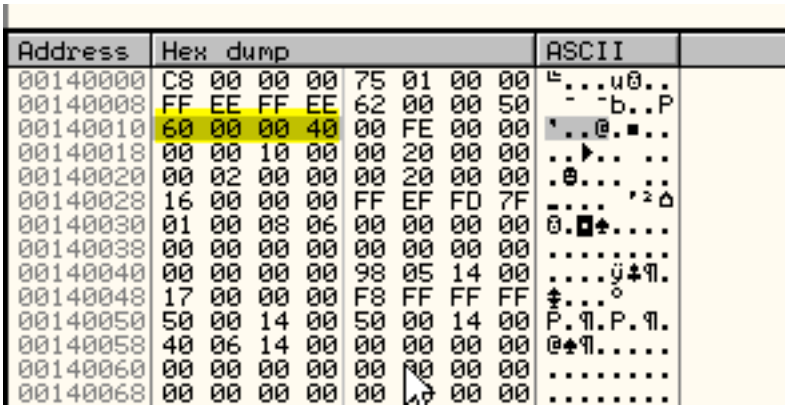
Идём посмотреть, что находится в "куче".



Отмечаем байти и выбираем FOLLOW DWORD IN DUMP, что перенесёт нас в область памяти, зарезервированной под "кучу".

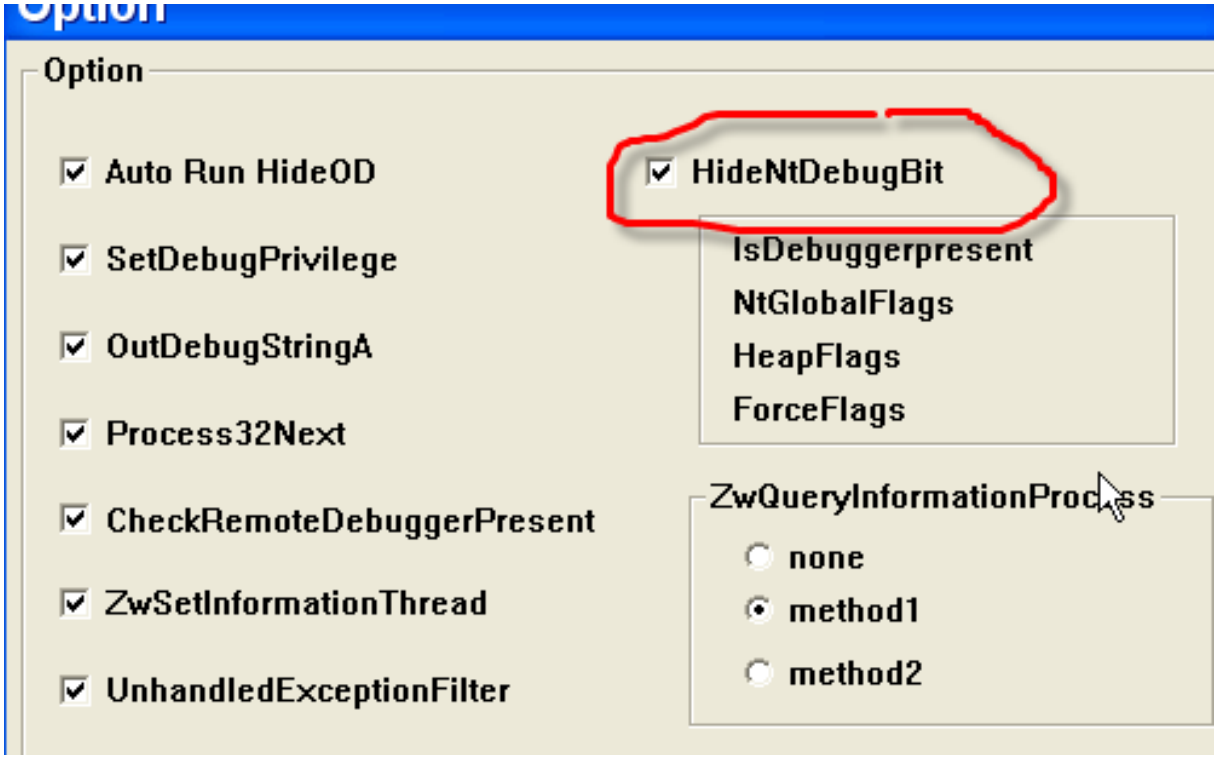


И прибавляем 10 к исходному адресу кучи, это отмеченный на картинке dword, в котором содержится ноль, если отладчика нет, значит сейчас там ноль из-за установленных плагинов, поэтому откроем крэкми в OllyDbg без плагинов.

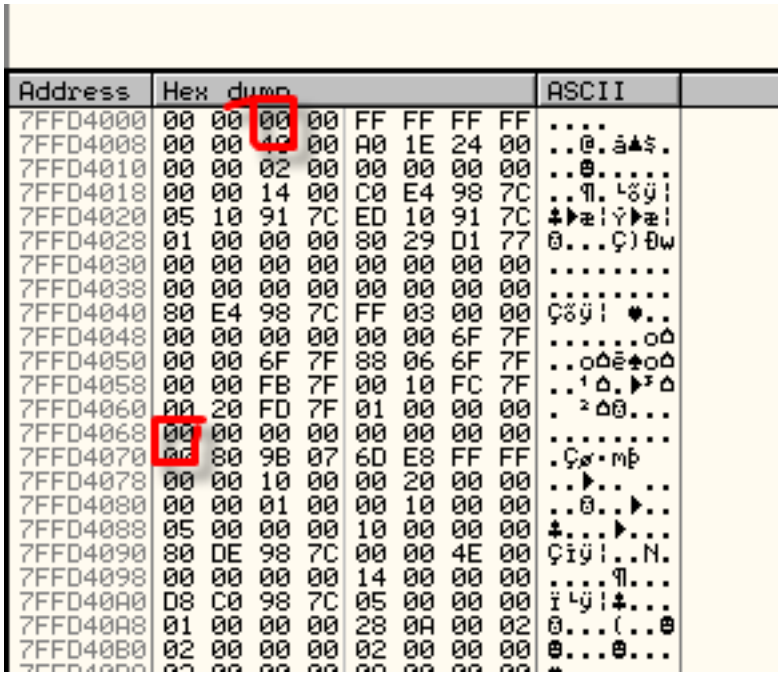


Сейчас здесь содержится не ноль, что говорит о наличии отладчиков, а значит какой-то плагин менял значение этого байта на ноль, хотя мы и не включали какую-либо специальную настройку.

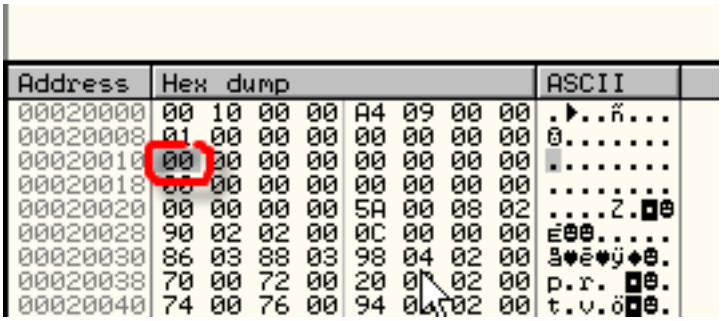
Теперь вернёмся, чтобы включить настройки в HideOdb.



И перезапустим крэкми cruehead'a.

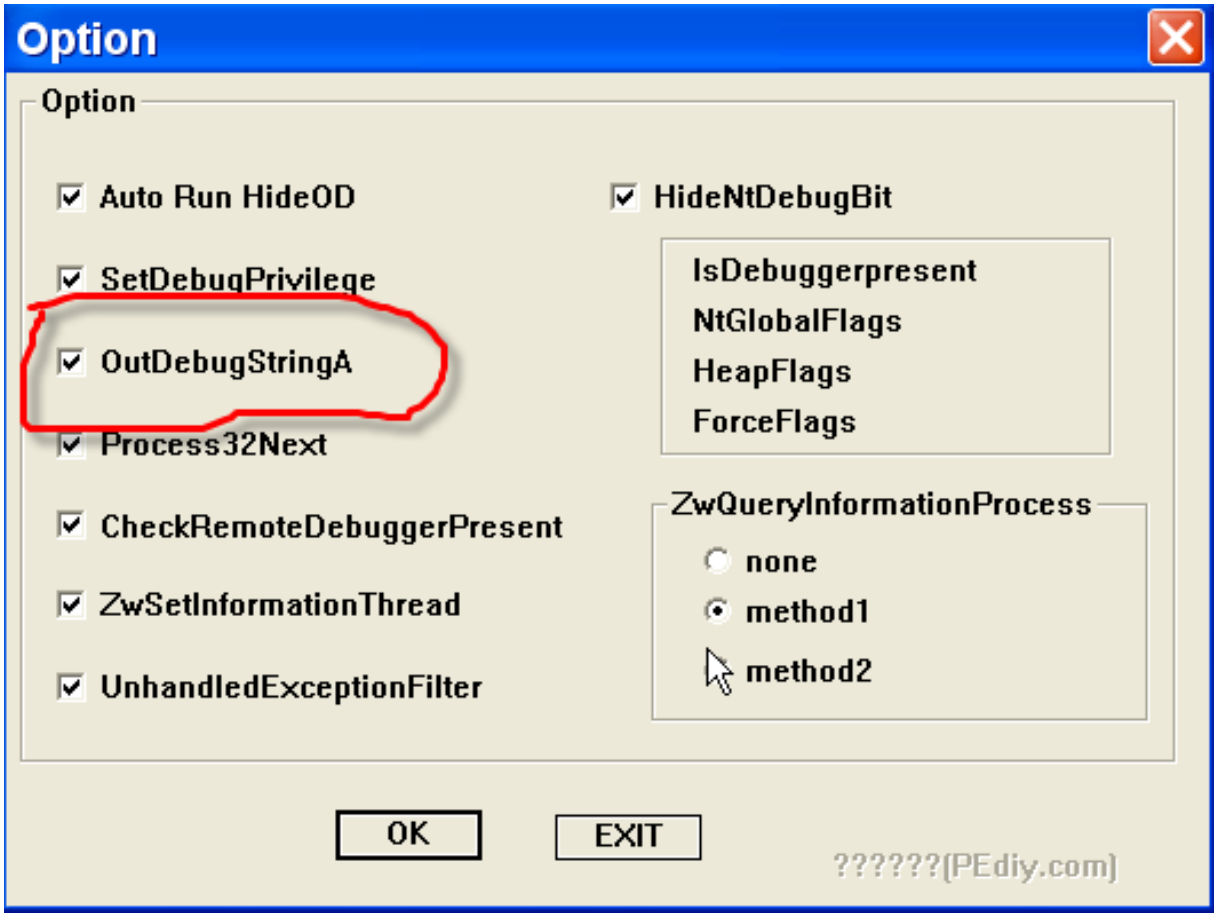


Видим, что остановились на точке входа, флаги IsDebbuggerPresent, как и NtGlobalFlag равны нулю, и поищем ProcessHeapFlag.

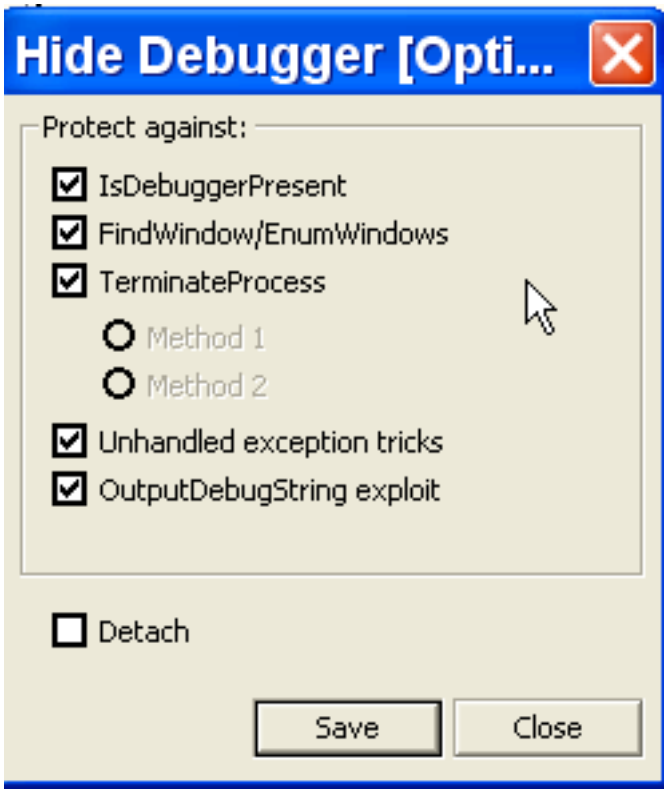


Он также равен нулю, то есть плагин работает правильно. Кроме того, мы научились искать и менять эти флаг вручную.

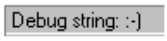
Последний противооллиный приём также предотвращается установленными плагинами.



И



Bueno, me dejo de rollos, de esta manera me di cuenta que el problema venia con el truco antiOlly de **OutputDebugString**, que justamente utiliza este programa y no los demás execryptos que había probado. Este truco consiste en mandarle un string preparado al Olly, que no lo sabe aceptar, y te da un error que cierra el Olly (creo ¿?) porque en realidad nunca me he encontrado con él, y si algún programa lo ha tenido el **plugin HideDebugger** lo neutraliza, y te manda una linda sonrisa:



La forma de realizarlo es muy simple tienes que llamar a **OutputDebugStringA** con una cadena de caracteres **%s** bien larga, yo he contado 100d:

Текст на картинке гласит:

"Хорошо, таким же образом я понял, что проблема связана с противооллиным приёмом, основанный на OutputDebugString и использованный данной программой. Этот приём заключается в передаче Олли специально подготовленной строки, которую та не может обработать, и происходит ошибка, закрывающая OllyDbg. В реальности я никогда не встречался с этим, так как плагин HideDebugger нейтрализует данный антиотладочный метод и выдаёт следующее сообщение:

Способ реализации очень прост: вызывается OutputDebugStringA, которой передаётся строка, состоящая из множества символов %s, я насчитал 100d:"

Это описание из прекрасного туториала. Плагины позаботятся об этом баге OllyDbg в лучшем виде.

Чтобы попрактиковаться, вот крэкми (находится в архиве `wasm_files.zip`: `files/0479/antisocial1.zip`), где применяются все приёмы, о которых рассказывалось в главах, посвящённых антиотладке, плюс пара других, хе-хе. Идея в том, чтобы запустить его в OllyDbg с отключенными плагинами, и вручную обезвредить антиотладочные приёмы. Для этого вам потребуется немного воображения, например, там есть приём, который можно победить, забив

строку нопами. Чтобы обнаружить такие места, нужно зайти в DEBUGGIN OPTION-EXCEPTIONS и убрать все галочки, так как при использовании INT68 генерируется ошибка, которая, если эти настройки включены, приведёт к тому, что Olly не выдаст нам предупреждение об исключении и просто закроется.

То есть, знайте, что если остановились на каком-то исключении (которое не было отмечено в настройках), то нужно нажать SHIFT+F9, чтобы миновать его. Мы рассмотрим это подробнее в следующей главе. Но если исключение генерируется через INT68, то надо забить NOPами это место и сделать RUN, чтобы пройти его без каких-либо проблем.

И в качестве развлечения, можете попробовать запустить крэкми со всеми плагинами, помотреть, как действует INT68 и выполнится ли программа, а во второй раз уже откройте в OllyDbg без плагинов и попытаться избежать все приёмы вручную, хе-хе.

Введение в крэкинг с нуля, используя OllyDbg - Глава 24

Автор: Рикардо Нарваха, пер. Aquila

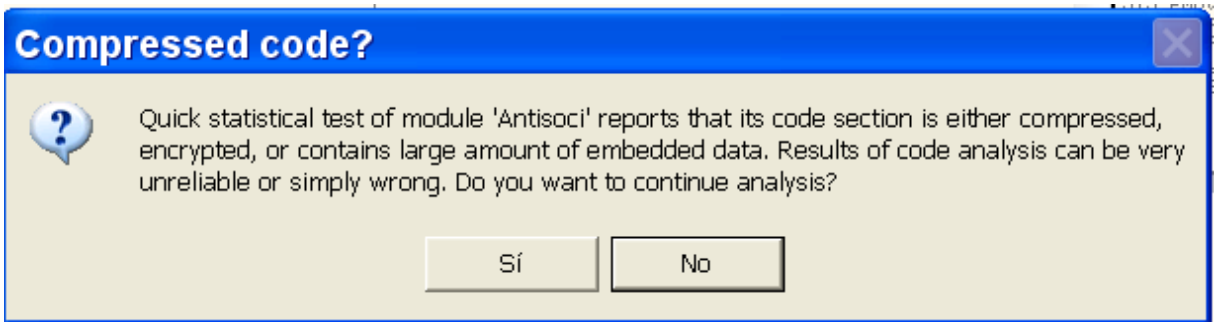
Дата: 2008-02-29

Мне попросили, чтобы прежде, чем начинать рассказ об исключениях, я рассказал о том, как запустить в OllyDbg крэкми «antisocial», которое было приложено к предыдущей статье.

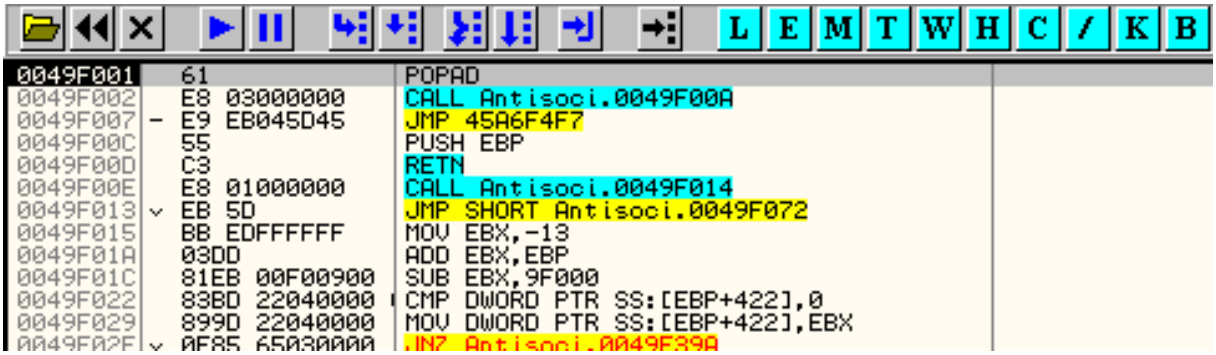
Прежде всего, необходимо знать, что это запакованная программа, поэтому сначала необходимо сделать определённые изменения в памяти без сохранения и только, когда она будет распакована, сохранить насовсем.

Сначала загрузим её в переименованную OllyDbg со всеми плагинами, чтобы посмотреть, почему не крэкми не выполняется и попытаться это исправить.

Итак, запускаем antisocial в переименованном и пропатченном OllyDbg со всеми доступными плагинами.

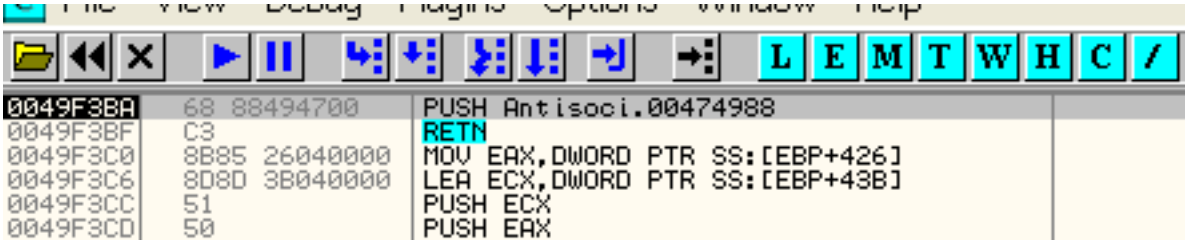


Это уже даёт нам идею о том, что программа может быть запакована. Оказываемся на точке входа.

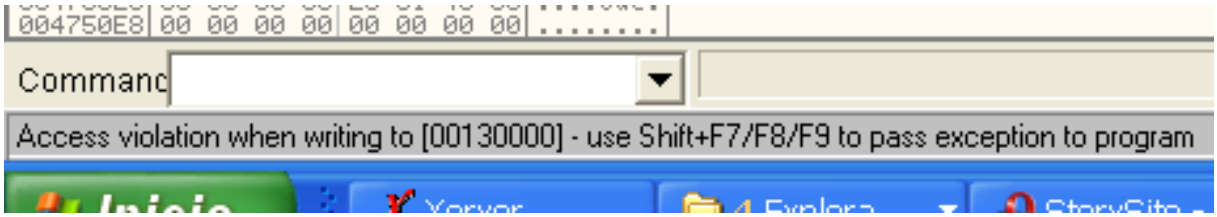


Здесь видим что-то странное, а именно инструкцию POPAD, которая служит для получения значений из стека, помещённых туда ранее с помощью PUSHAD, а никаких PUSHAD до этого нет, что весьма подозрительно.

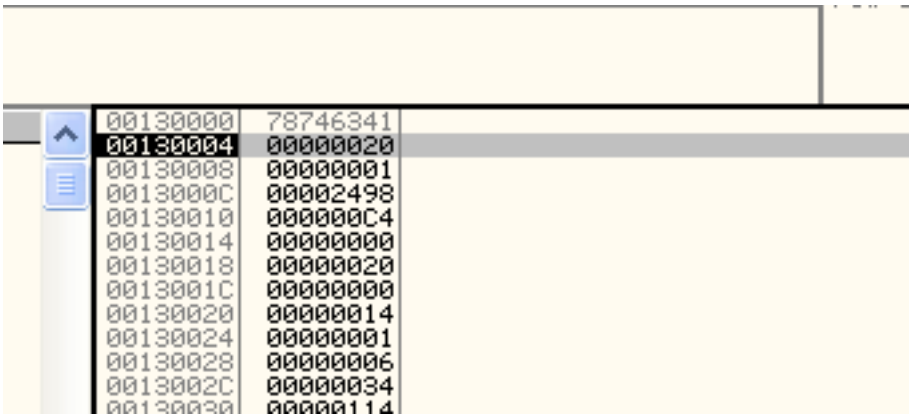
Запускаем, чтобы посмотреть, до куда дойдёт.



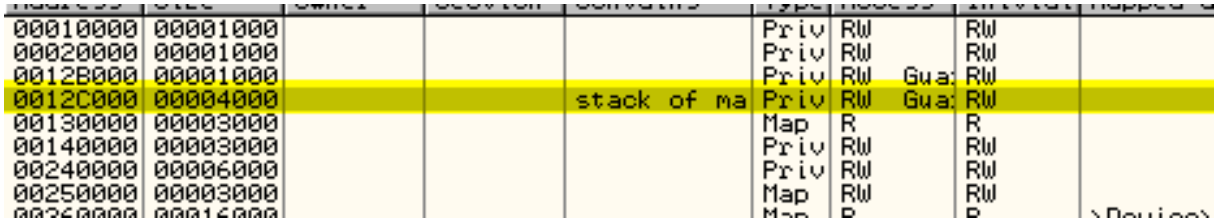
Досюда.



То есть, когда крэкми пытается сделать PUSH, выдётся ошибка, что нет прав писать в стек, но права писать в стек должны быть всегда. Взглянем на стек.

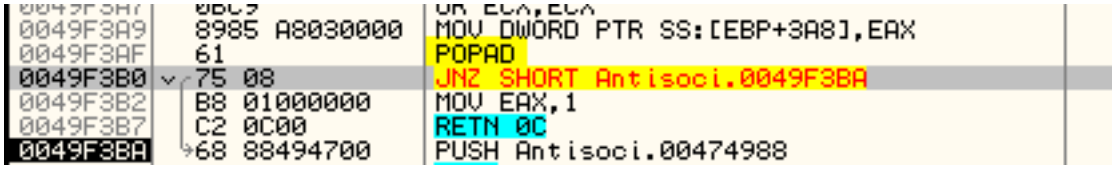


То есть, значение наверху стека равно 130000, и если рестартуем программу:

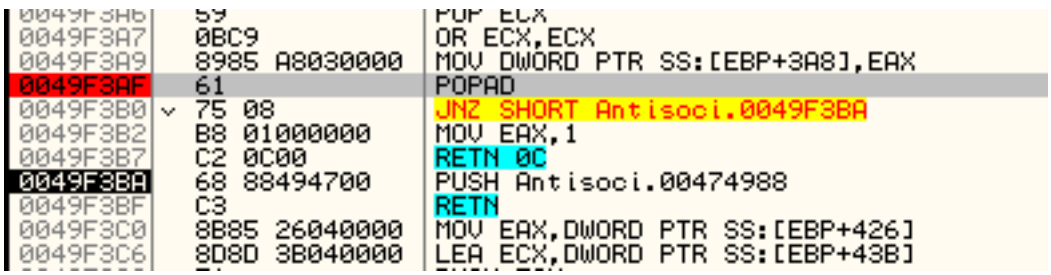


Посмотрим секции и увидим, что стек на моей машине идёт с 12c000 до 12ffff, ошибка возникает, потому что стек выше за пределы своей секции на другую, которая начинается с 130000, а туда-то писать как раз прав и нет.

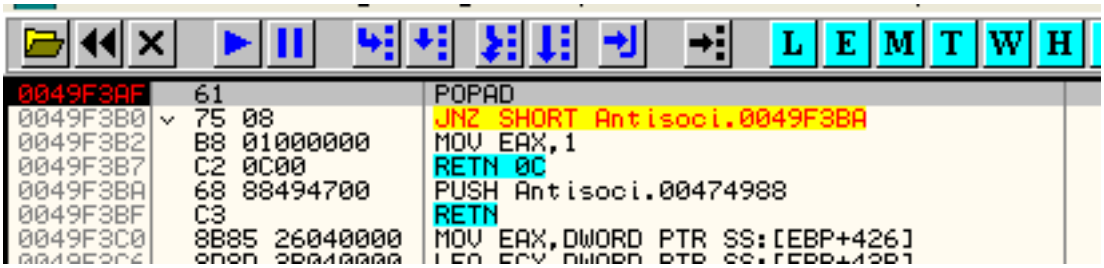
Вернёмся обратно в место, где произошла ошибка.



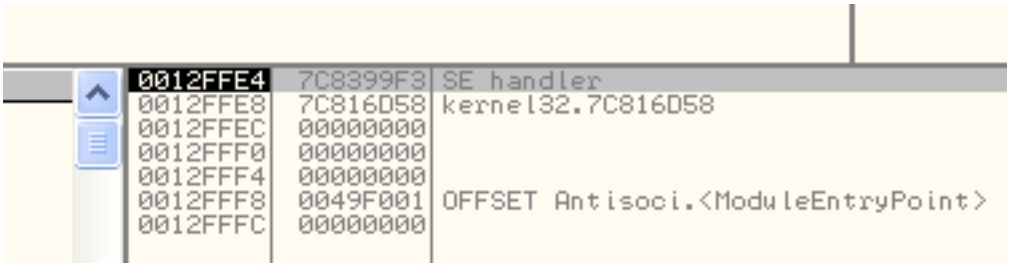
Видим, что программа выполняет ещё один popad и совершает JNZ-переход на PUSH, который также вызывает ошибку. Установим BPX на этот popad до того, как произойдёт ошибка.



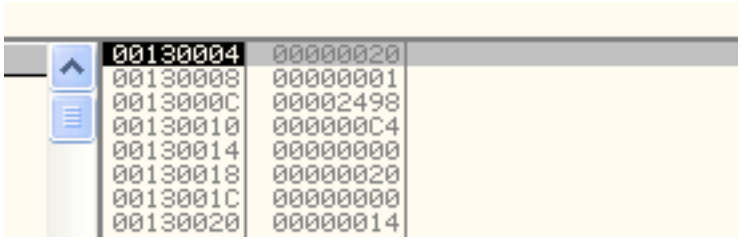
Теперь перезагрузим программу, сделаем RUN и остановимся здесь.



Смотрим стек.

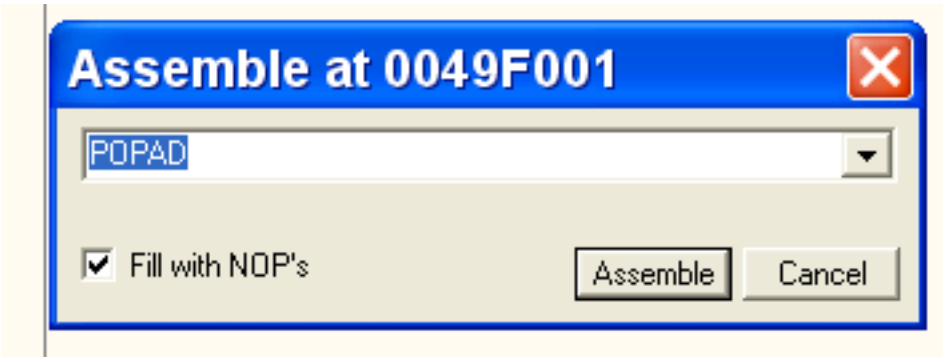


Находимся в правильной секции. Запускается POPAD.

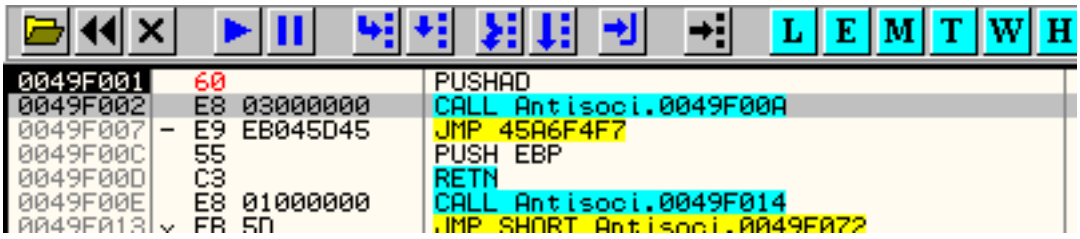


Здесь заканчивается секция, так что проблему можно решить, забив POPAD NOP'ами, но вообще, POPAD должен предшествоваться PUSHAD, которая сохраняет начальные значения регистров в стек, а этот POPAD тогда будет получать их обратно, поэтому давайте посмотрим, что будет, если заменить POPAD на PUSHAD. Рестартуем программу.

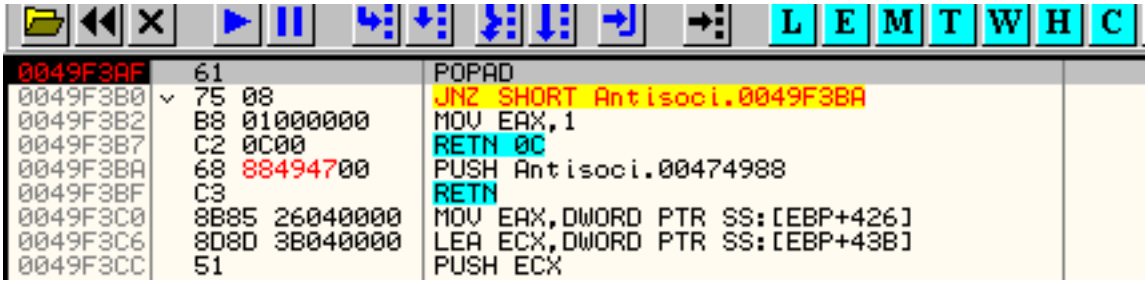
Нажимаем пробел.



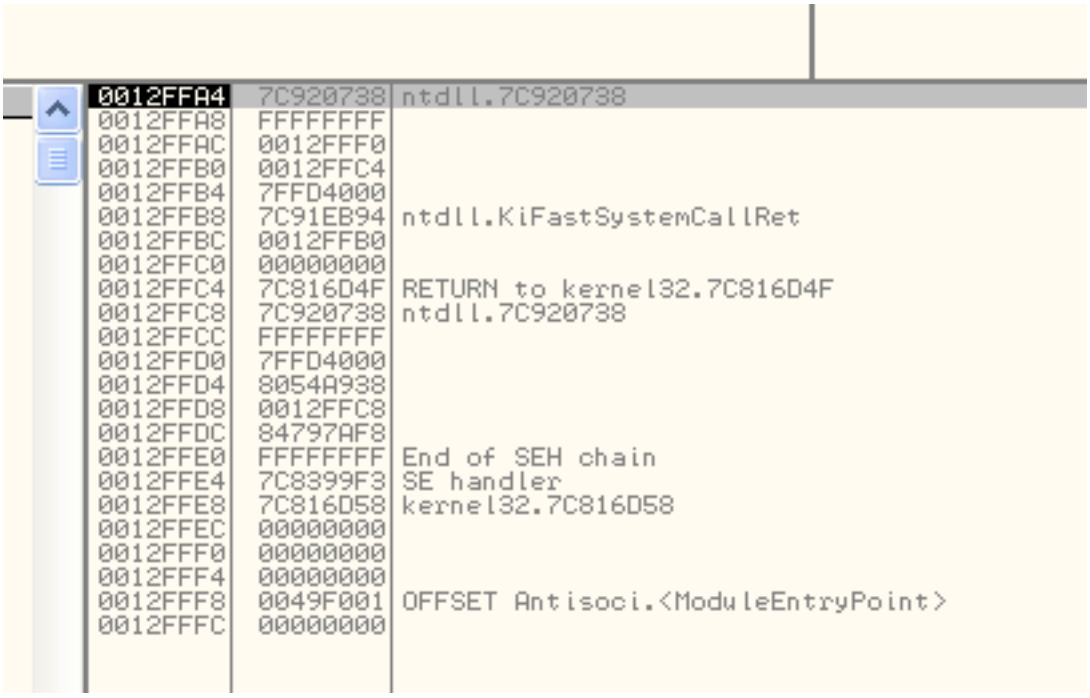
Пишем PUSHAD.



Теперь нажимаем RUN и останавливаемся на второй POPAD.

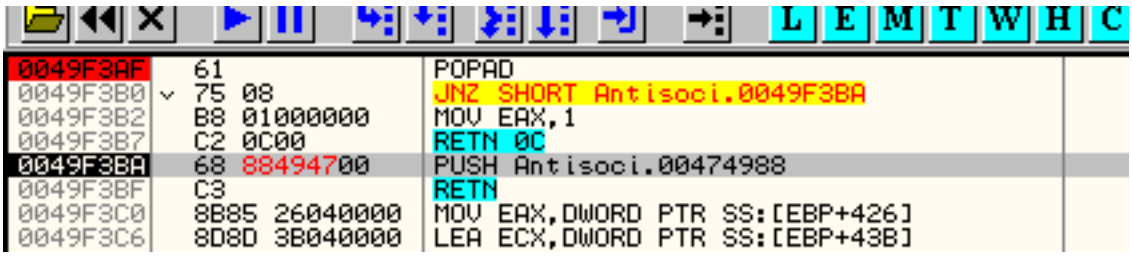


Но в этот раз стек в порядке.

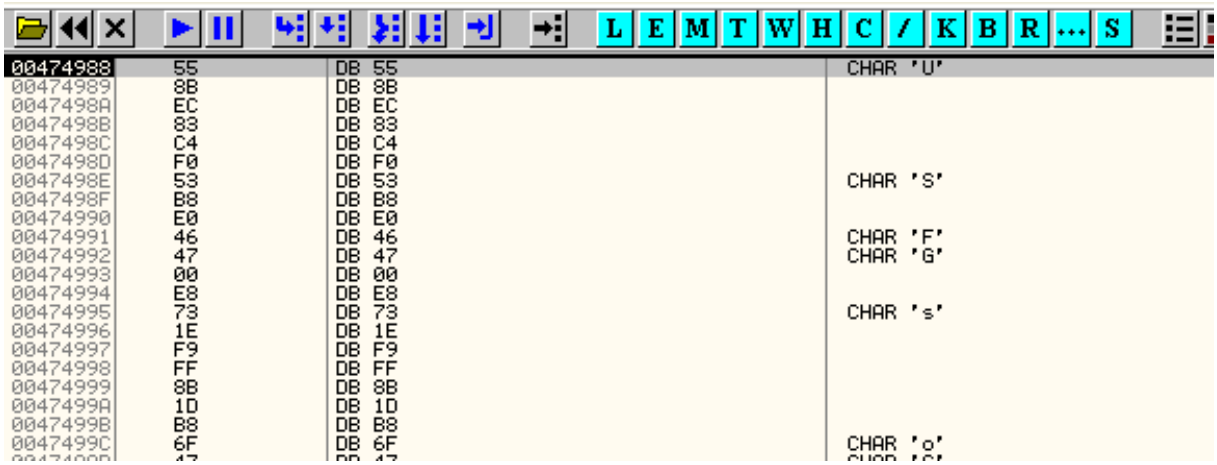


Поэтому POPAD не выходит за границу секции стека, и мы можем миновать это место с помощью F8.

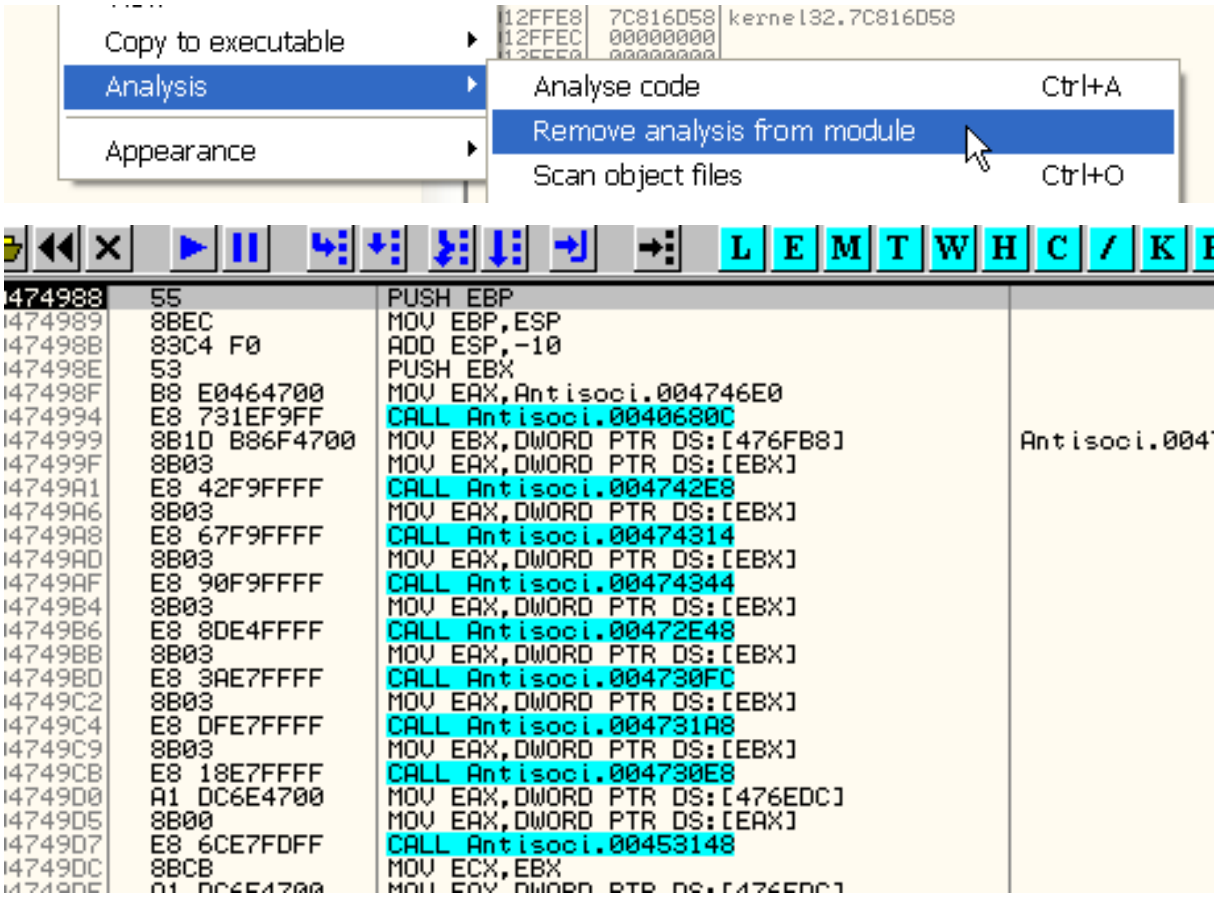
Видим, что всё прошло без проблем.



Доходим до PUSH и RET, трассируем и, нажимая на F8, доходим до RET.

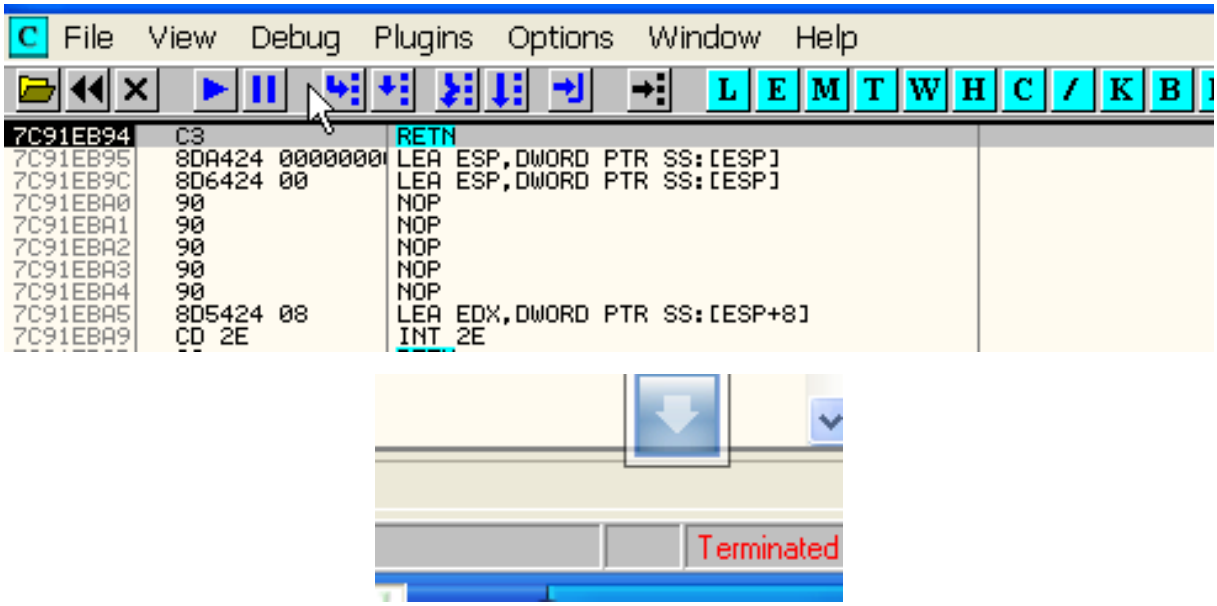


Хорошо, здесь есть проблема с анализированием.



Здесь это особенно видно.

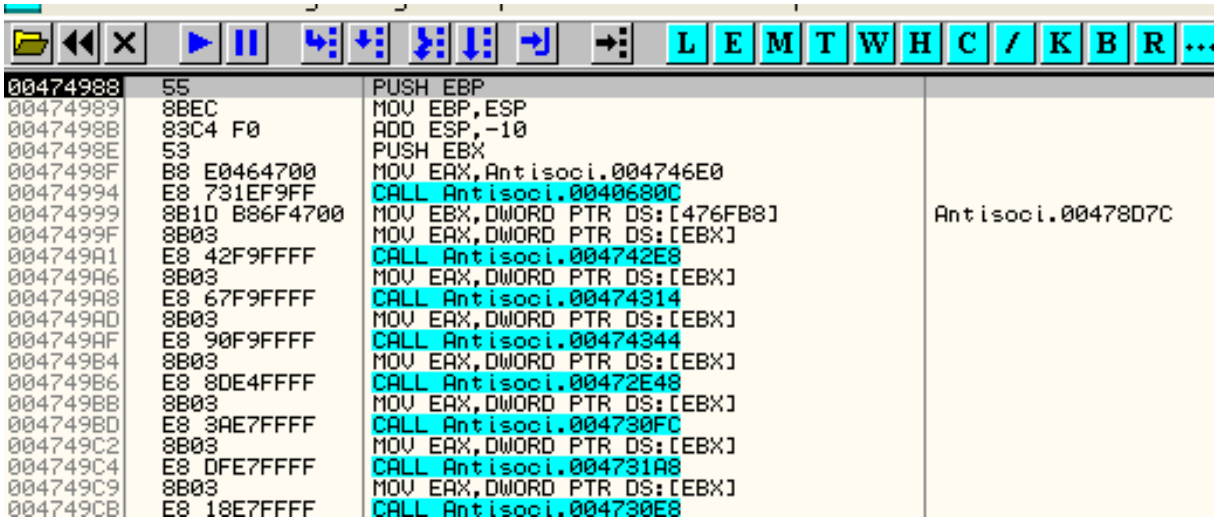
Посмотрим, что случится, если сделаем RUN.



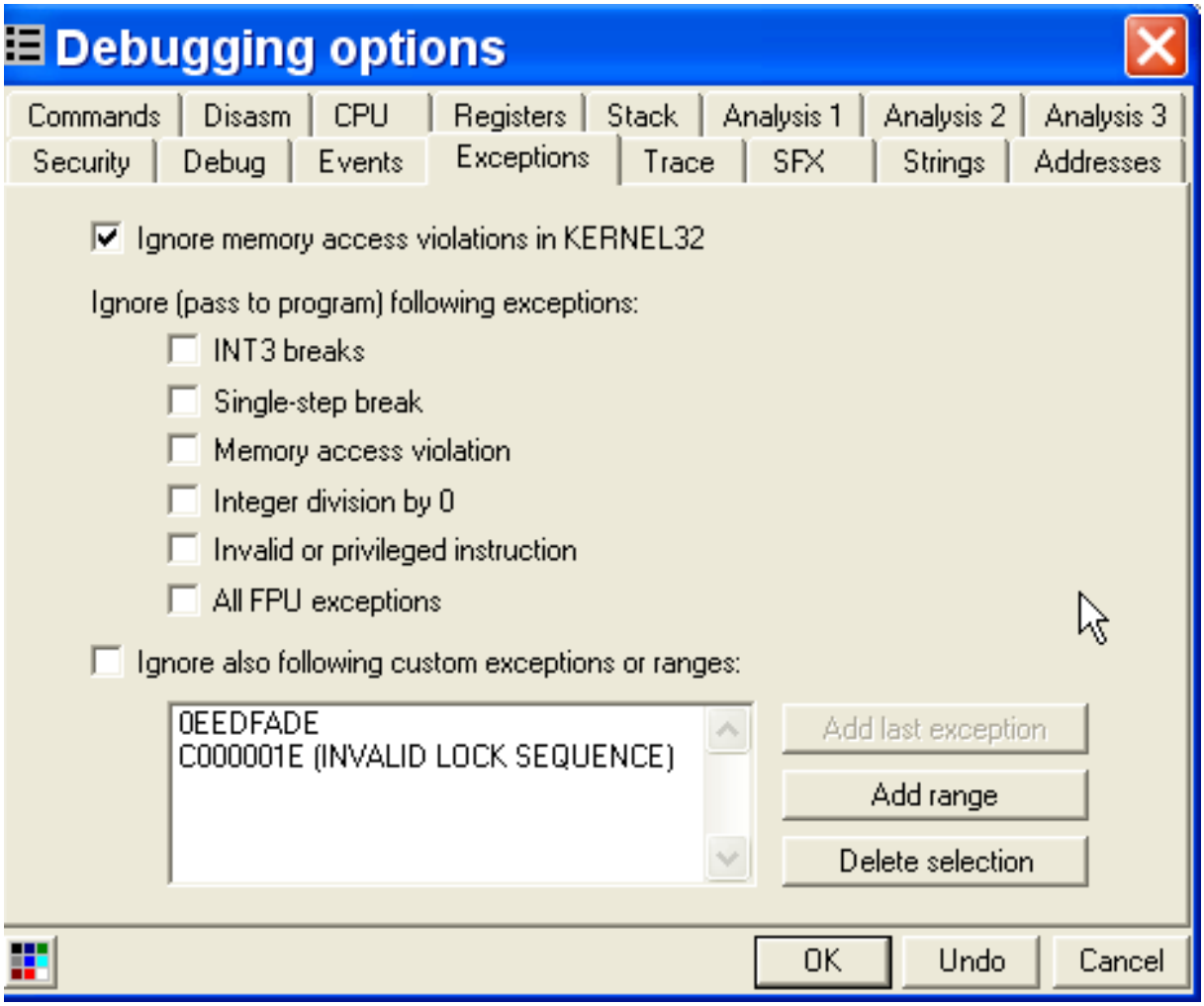
Так как программа завершилась, посмотрим в LOG'e OllyDbg, есть ли там что-нибудь интересное.

```
0049F001 Program entry point
         Analysing Antisoci
           0 heuristical procedures
0049F3AF Breakpoint at Antisoci.0049F3AF
004730EA Access violation when reading [FFFFFFFF]
5B150000 Module C:\WINDOWS\system32\uxtheme.dll
69930000 Module C:\WINDOWS\system32\faultrep.dll
77BD0000 Module C:\WINDOWS\system32\VERSION.dll
76630000 Module C:\WINDOWS\system32\USERENV.dll
76310000 Module C:\WINDOWS\system32\WINSTA.dll
597F0000 Module C:\WINDOWS\system32\NETAPI32.dll
76F10000 Module C:\WINDOWS\system32\WTSAPI32.dll
778F0000 Module C:\WINDOWS\system32\SETUPAPI.dll
77F40000 Module C:\WINDOWS\system32\SHLWAPI.dll
77B10000 Module C:\WINDOWS\system32\Apphelp.dll
597F0000 Unload C:\WINDOWS\system32\NETAPI32.dll
69930000 Unload C:\WINDOWS\system32\faultrep.dll
76310000 Unload C:\WINDOWS\system32\WINSTA.dll
76630000 Unload C:\WINDOWS\system32\USERENV.dll
76F10000 Unload C:\WINDOWS\system32\WTSAPI32.dll
778F0000 Unload C:\WINDOWS\system32\SETUPAPI.dll
         Process terminated, exit code C0000005 (-1073741819.)
```

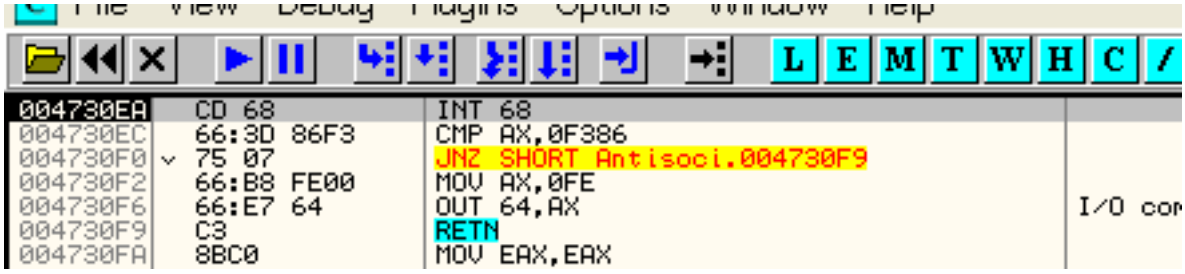
Видим, что возникло исключение после точки останова на POPAD. Рестартуем и повторим все шаги, чтобы оказаться там, где были.



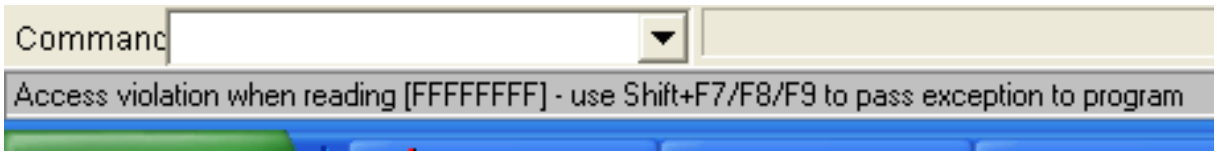
Уберём галочки со всех опций, которые относятся к исключениям.



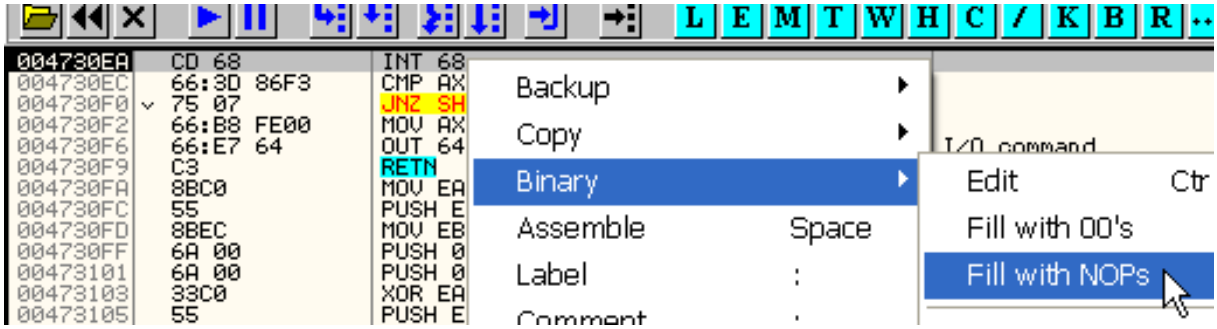
Делаем RUN.

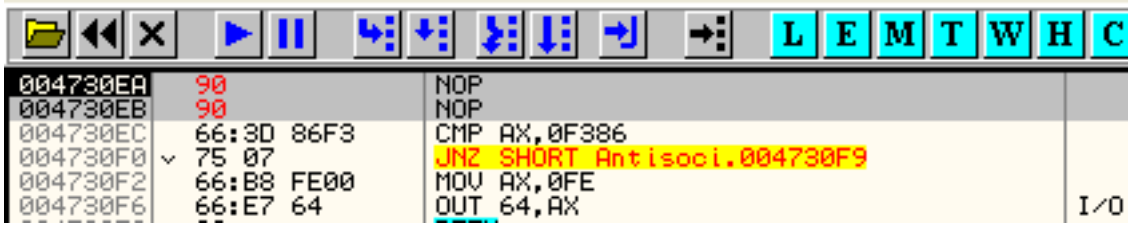


Останавливаемся на исключении, что отображается в LOG'е.

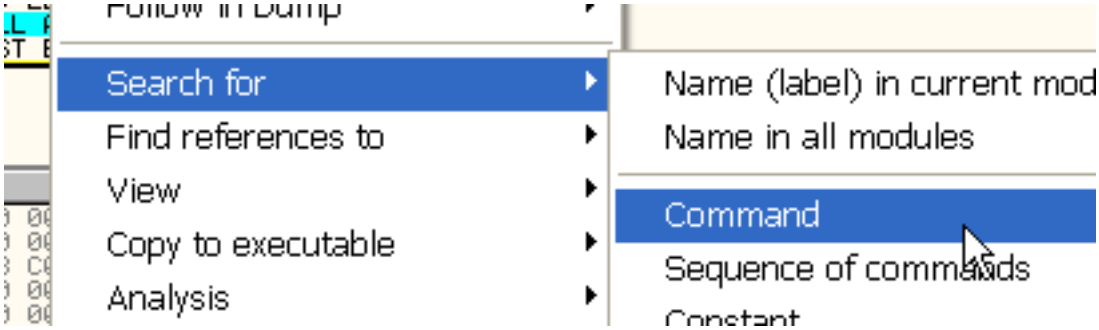


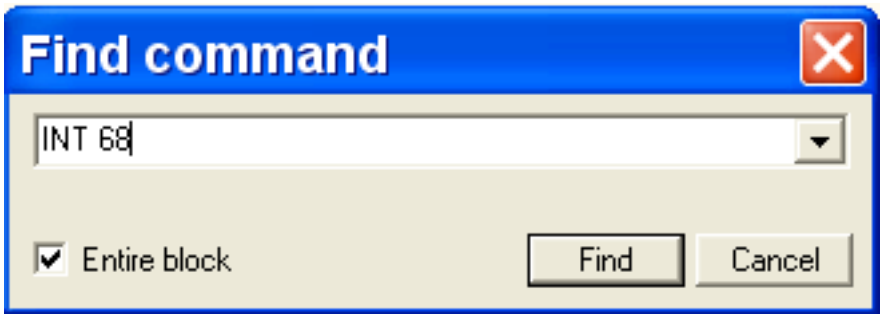
INT68 – одно из немногих исключений, которое OllyDbg не может избежать, но можем забить его NOP'ами.

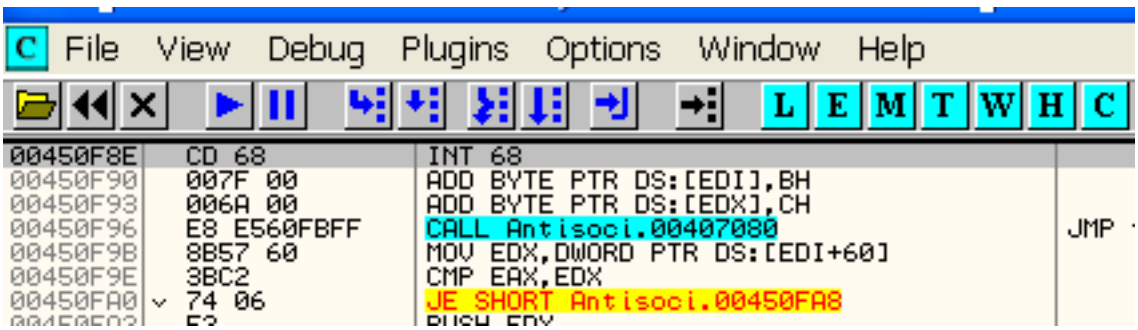




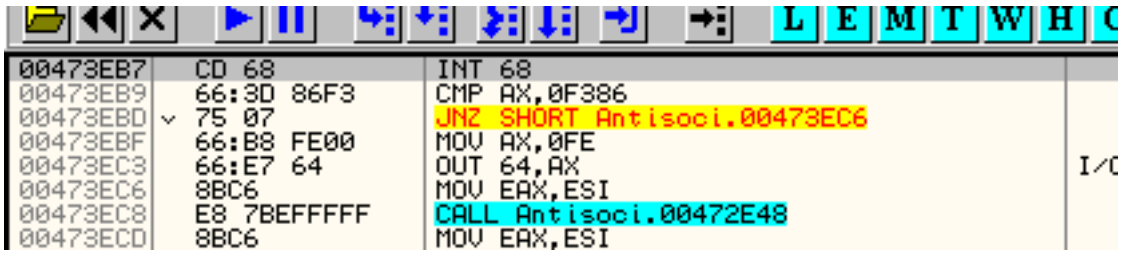
С другой стороны, знаем, что могут быть ещё INT68, которые будут мешать, поэтому поищем их и также забьём NOP’ами.







Видим, что нашлось ещё одно исключение, забиваем его NOP’ами.



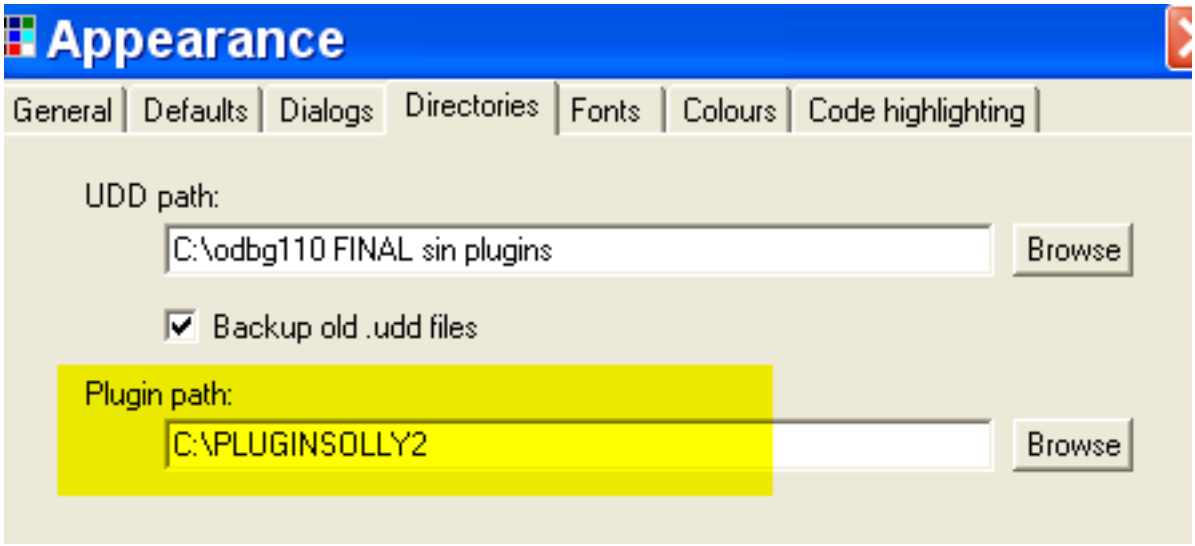
Ищем заново, находим, забиваем NOP'ами.

Если нажмём CTRL + L, то продолжим искать то, что искали в последний раз.

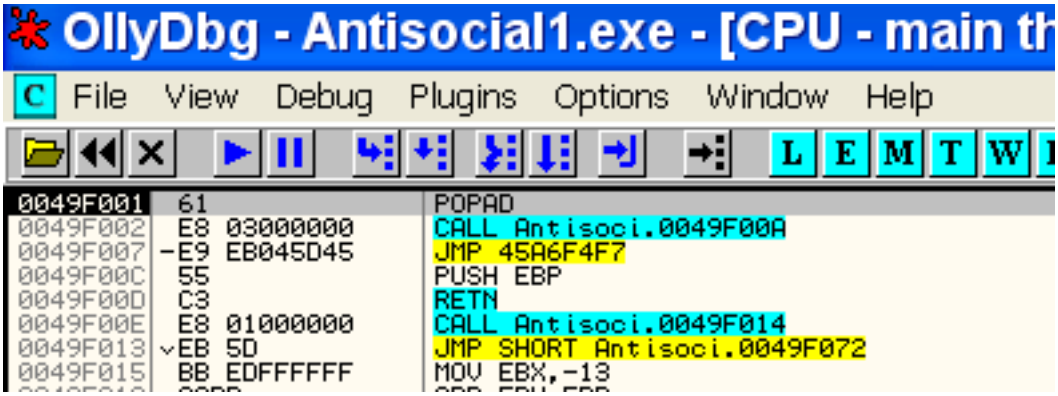
Когда ничего больше не находим, делаем RUN.

Как видим, это всё, что нужно сделать, чтобы запустить крэкми из-под OllyDbg с плагинами, а теперь попытаемся сделать тоже самое, но без плагинов, переименования и патчевания.

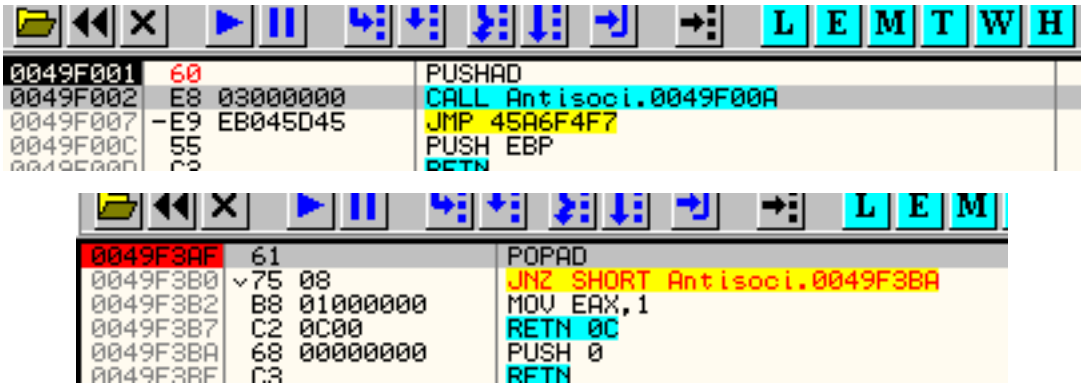
Я распаковал OllyDbg без плагинов, всё, что есть – это command bar, так что никакой защиты. Для этого используем другую директорию и другой путь, указывающий на плагины.



Как видим, здесь только один плагин. Запускаем OllyDbg.

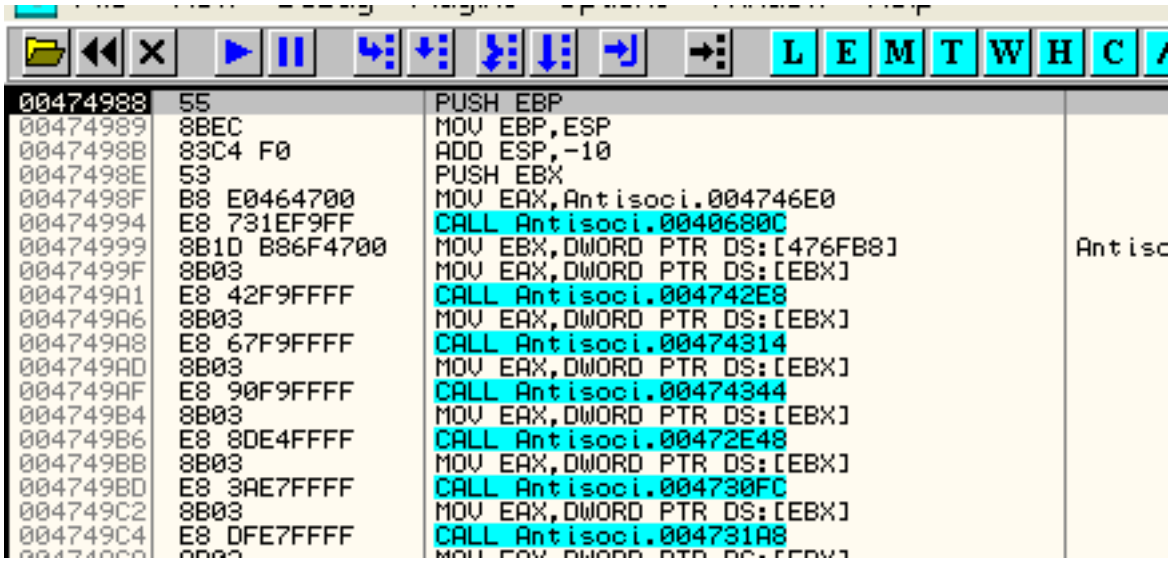


Повторяем операции по замене popad.

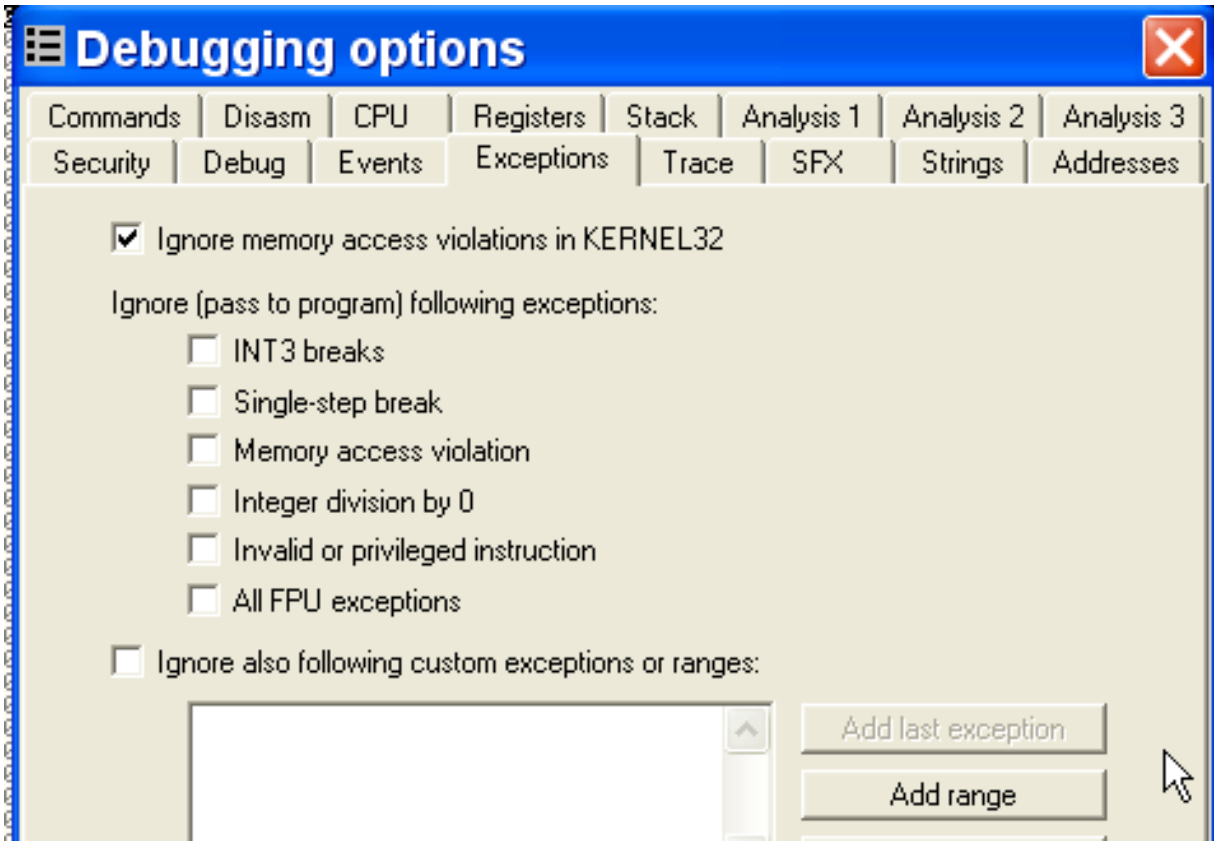


Устанавливаем BPX на POPAd, нажимаем RUN и смотрим, куда попадём.

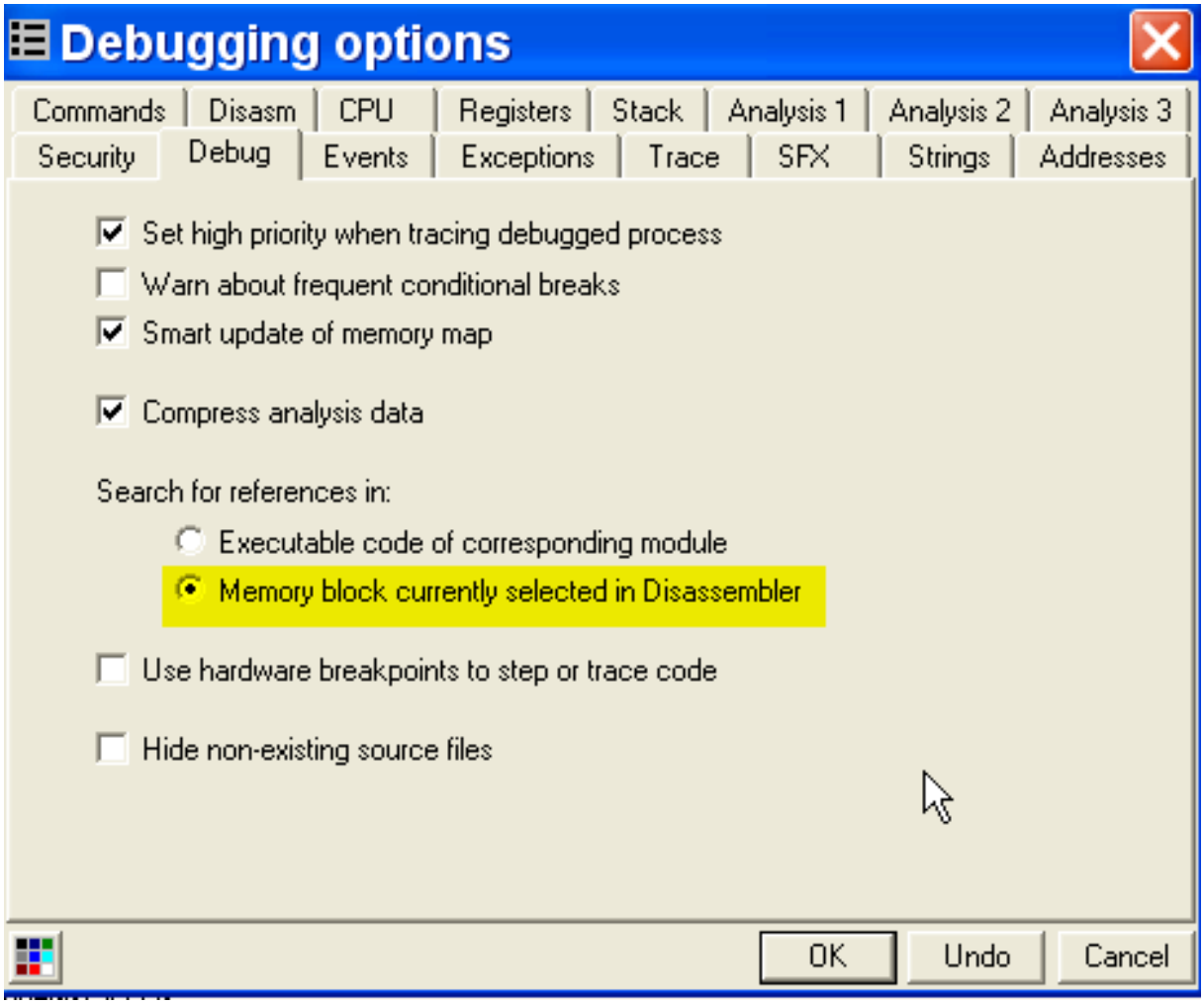
Прибываем туда, где распаковывается код программы.



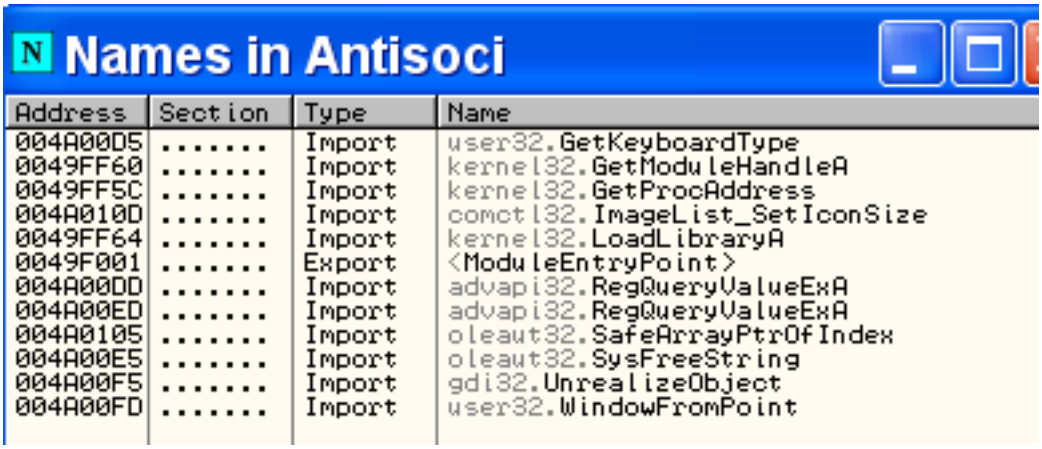
Можем забить INT68 или снять галочки с исключений, и каждый раз, останавливаясь на INT68, забивать его NOP'ами, а не идти дальше с помощью SHIFT+F9, как это мы обычно делаем с другими исключениями.



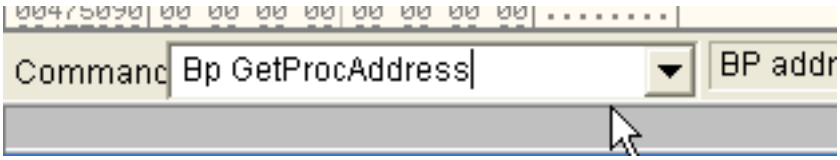
Посмотрим, какие API-функции используются в программе, для чего нужно вспомнить следующую настройку:



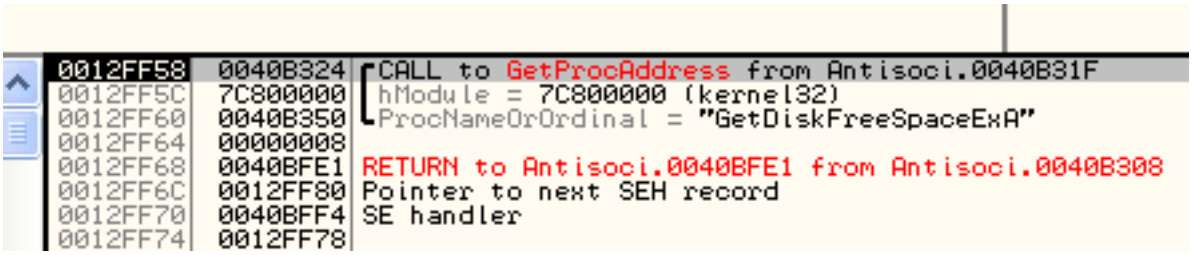
Необходимо отметить её, чтобы отображалась информация о секции, в которой мы сейчас находимся.



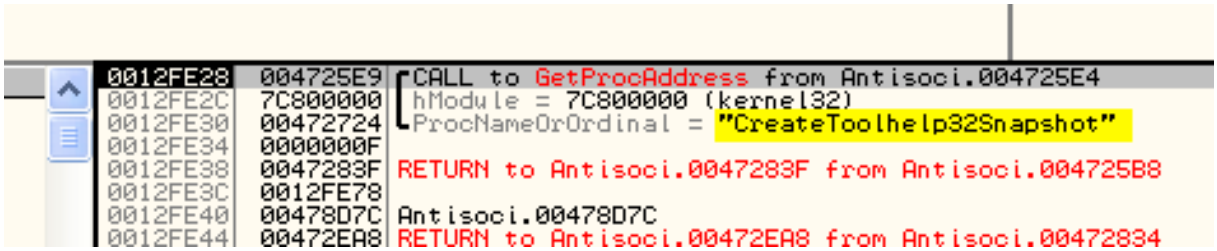
Отлично, видим, что подозрительных API-функций нет, но их используется очень мало, а также есть GetProcAddress для загрузки новых, поэтому ставим на неё BPX.



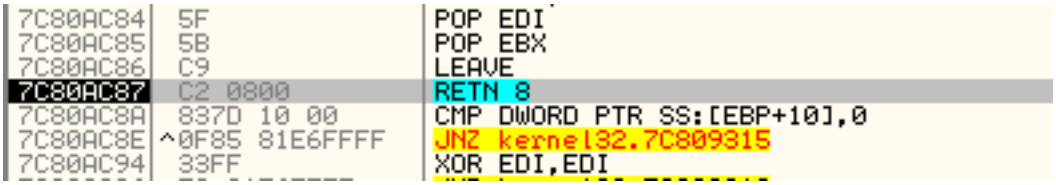
Жмём RUN.

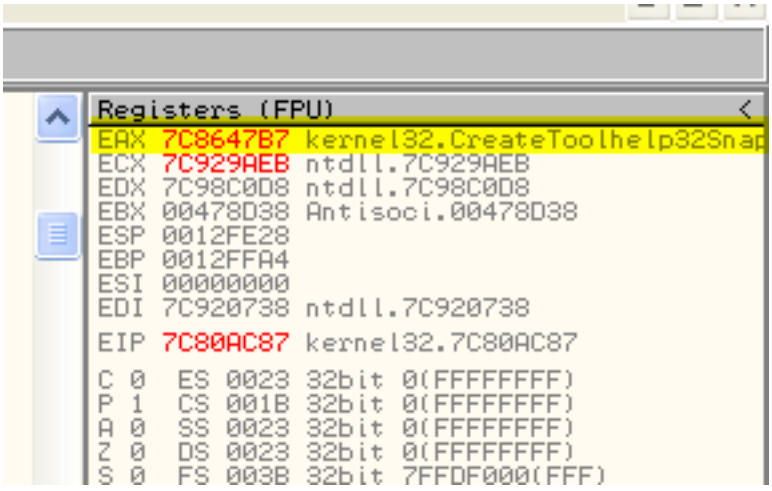


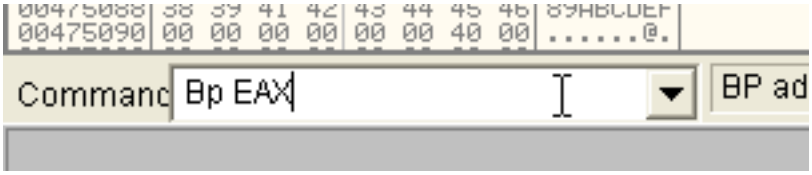
Ничего подозрительного нет, продолжаем нажимать RUN, пока не встретится что-нибудь странное.

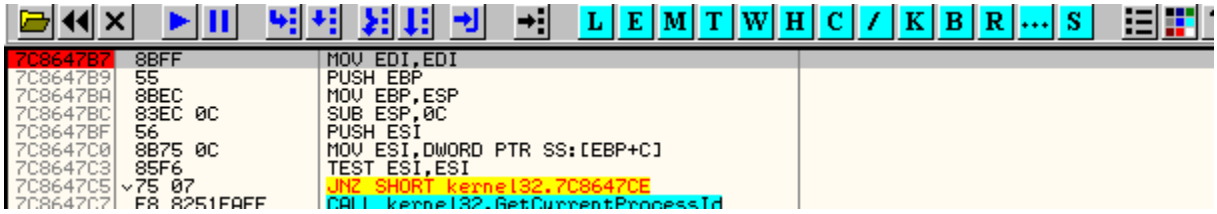


Здесь нам повстречалась первая странность: API-функция, которая делает снимок состояния всех выполняющихся процессов. Доходим до RET и ставим BP EAX, так как EAX будет содержать адрес функции.





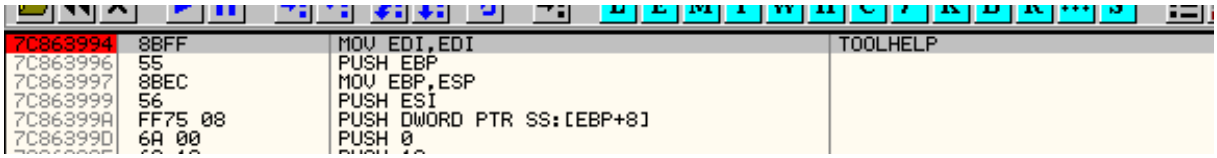




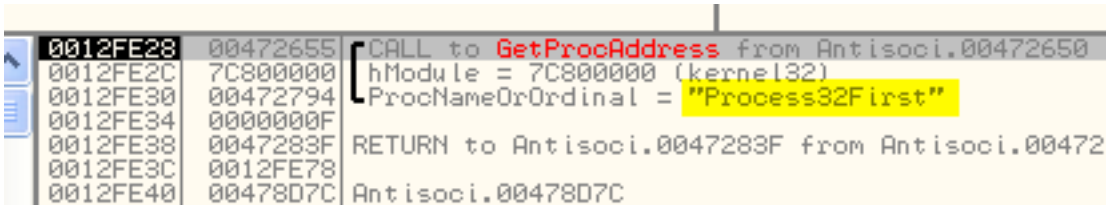
Там, где находится BP, оставим комментарий, чтобы не забыть, к какой API-функции это относится.

Список продолжается.

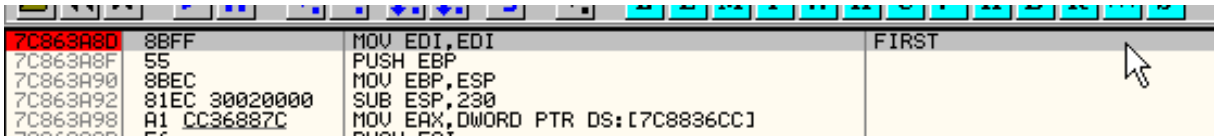
Ещё что-то похожее на прошлый случай, на всякий случай поставим BP и сюда.

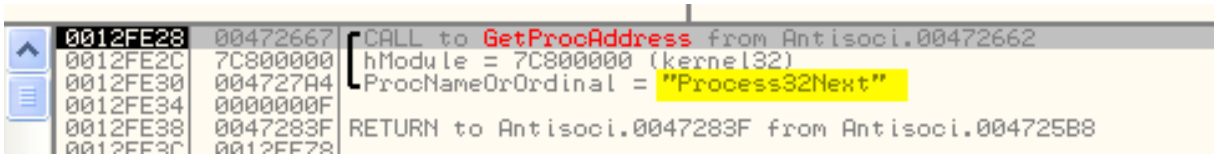


Жмём RUN.

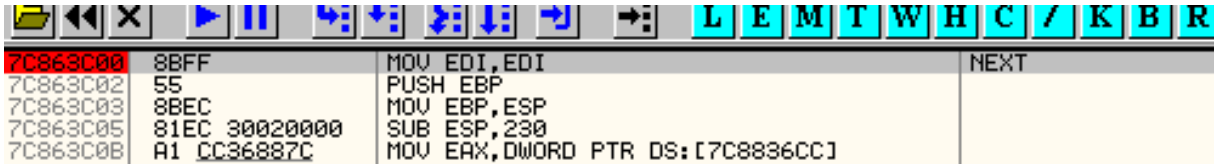


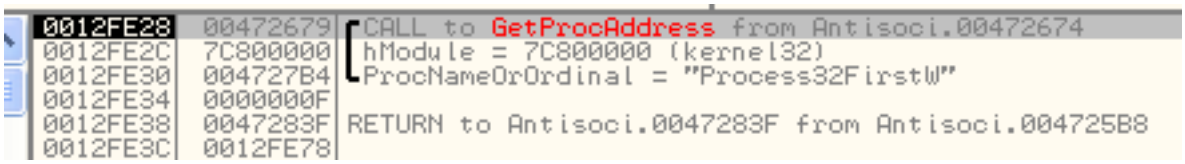
Ставим BP на неё.

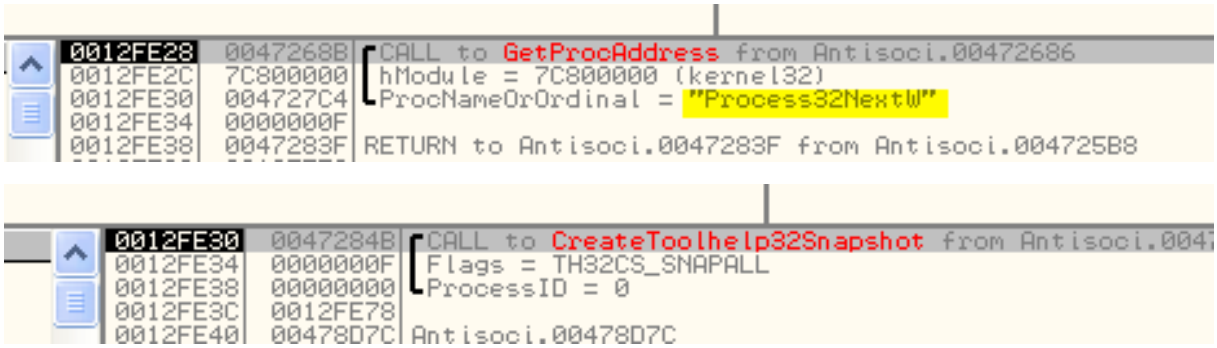




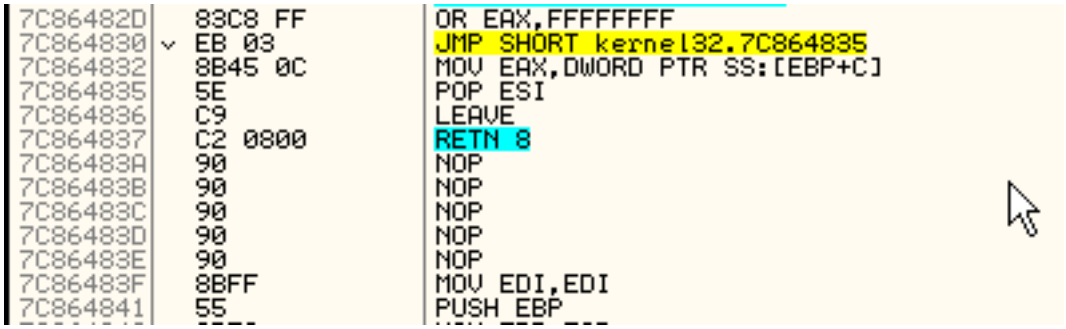
То же самое.



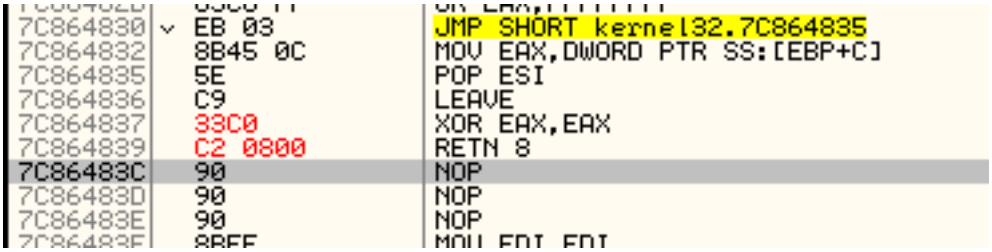




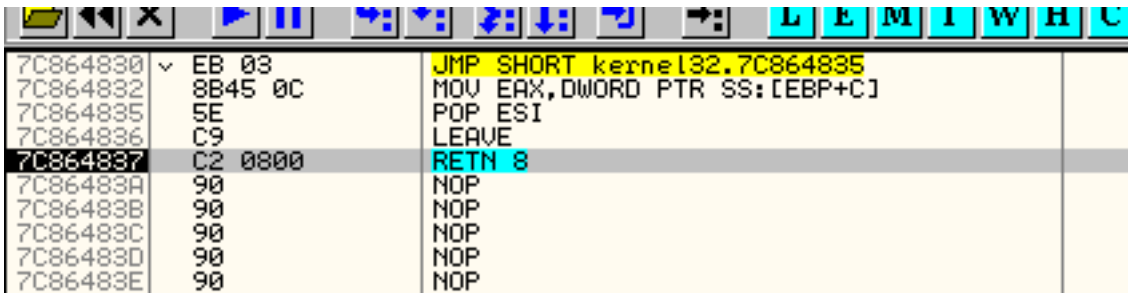
Останавливаемся на API-функции, которая делает снимок, от которого и зависит вся защита. С его помощью производится обработка списка процессов, потоков и множество других вещей. Мы можем попытаться пропатчит эту API-функцию так, чтобы она не возвращала логический номер снимка, для чего проследуем к RET функции.



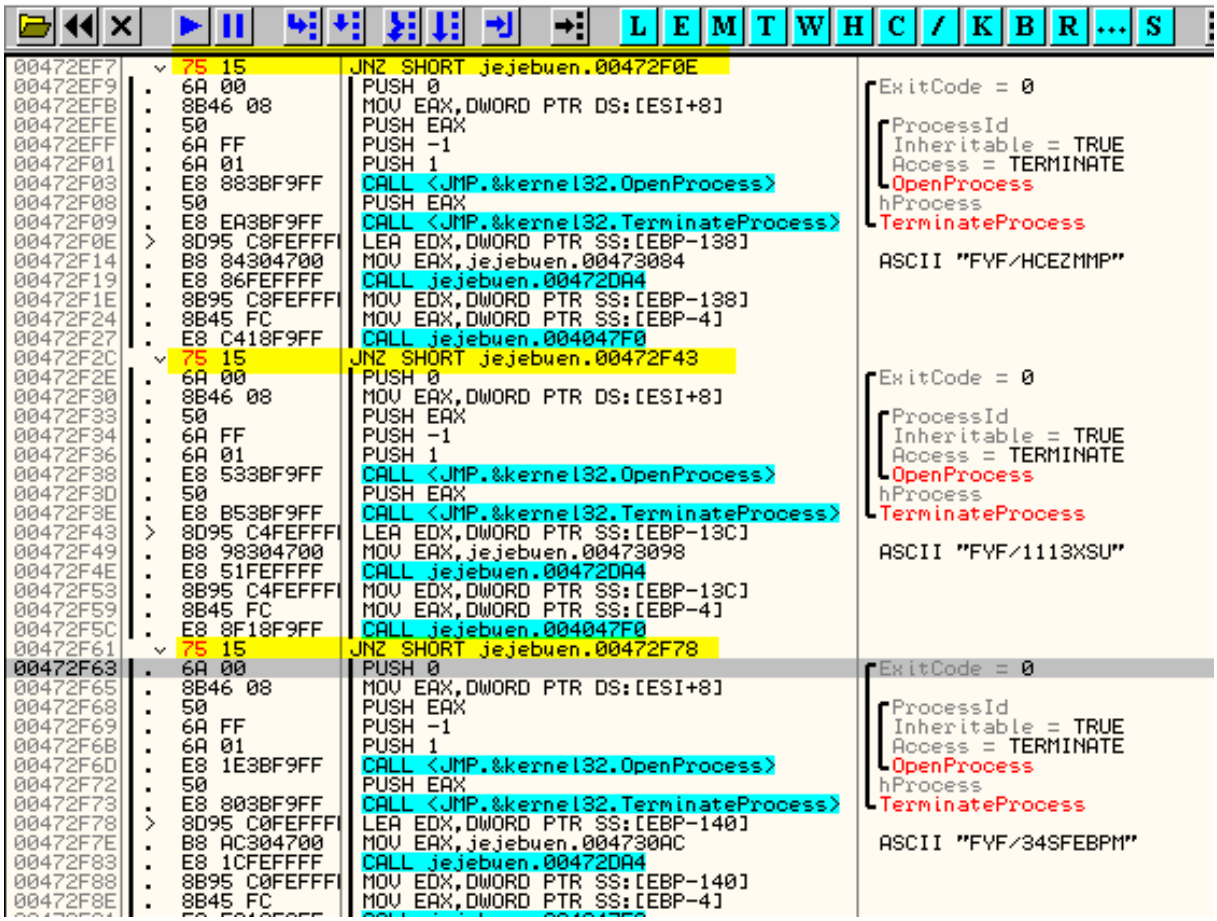
Видим пустые пробелы (NOP'ы), так что можем сделать EAX равным нулю, до того как его значение будет возвращено.



Так мы возвращаем ноль, и у программы нет логического номера, с помощью которого она могла бы обработать снимок или что-то узнать о процессах. Это один способ, а другой состоит в изменении кода самой программы. Выполним неизменённый RET функции.

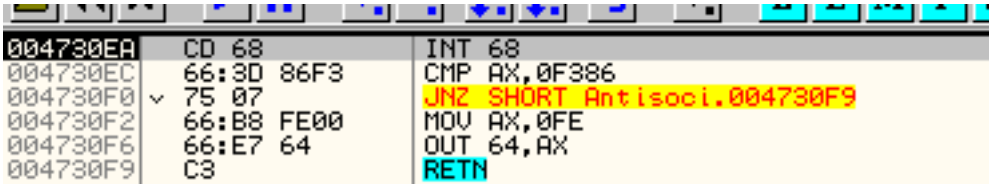


Чуть пониже есть JNZ-переходы, а ещё чуть ниже их – вызов TerminateProcess, который закрывает OllyDbg, и перед которым есть OpenProcess, получающая логический номер программы.



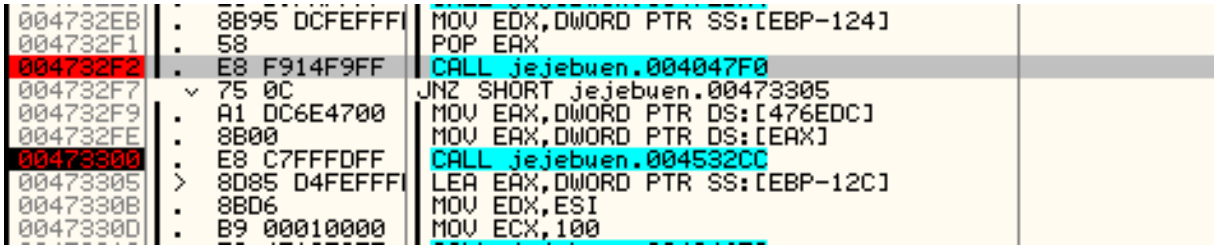
Если заменить все JNZ на JMP, можно избежать перехода на TerminateProcess и закрытия OllyDbg.

Первая часть защиты преодолена, теперь переходим ко второй части. Жмём RUN.

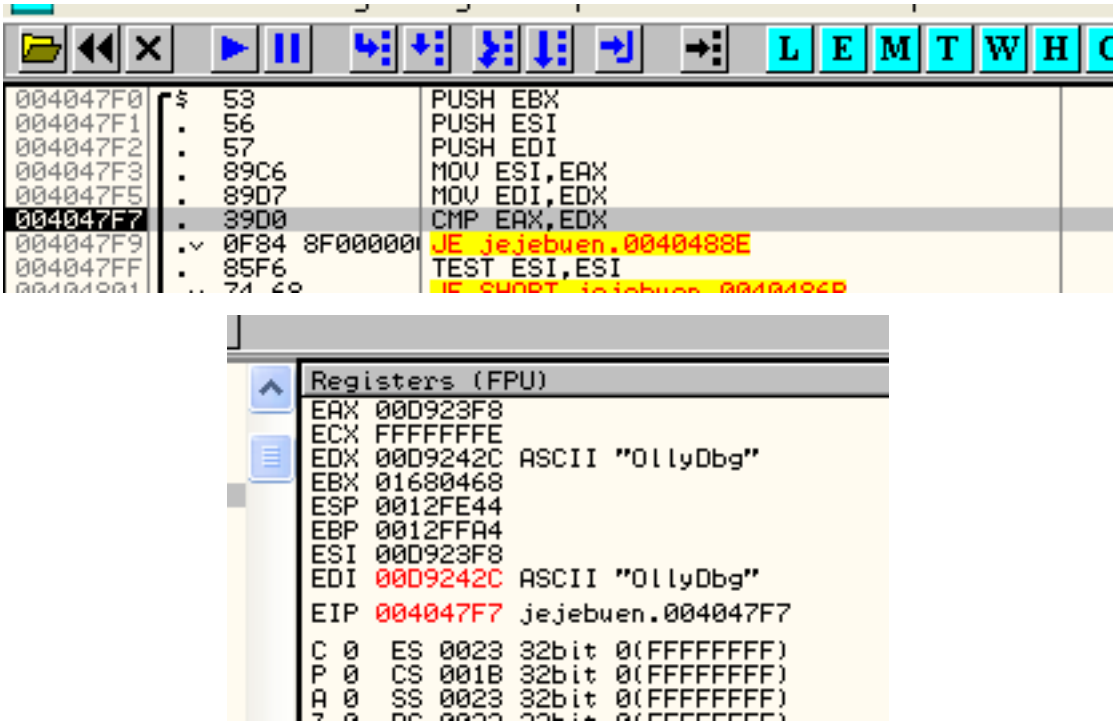


Забиваем NOP’ами, запускаем выполнение программы, но она всё равно закрывается. Если поставить плагин HideDebugger, который защищает от FindeWindows/EnumWindows, то выполнится нормально, если повторить сделанные до этого шаги.

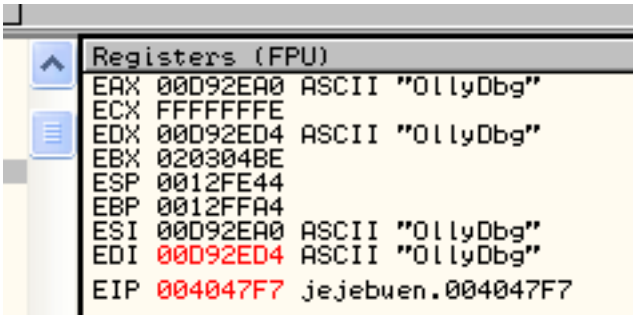
Место, где происходит закрытие, следующее:

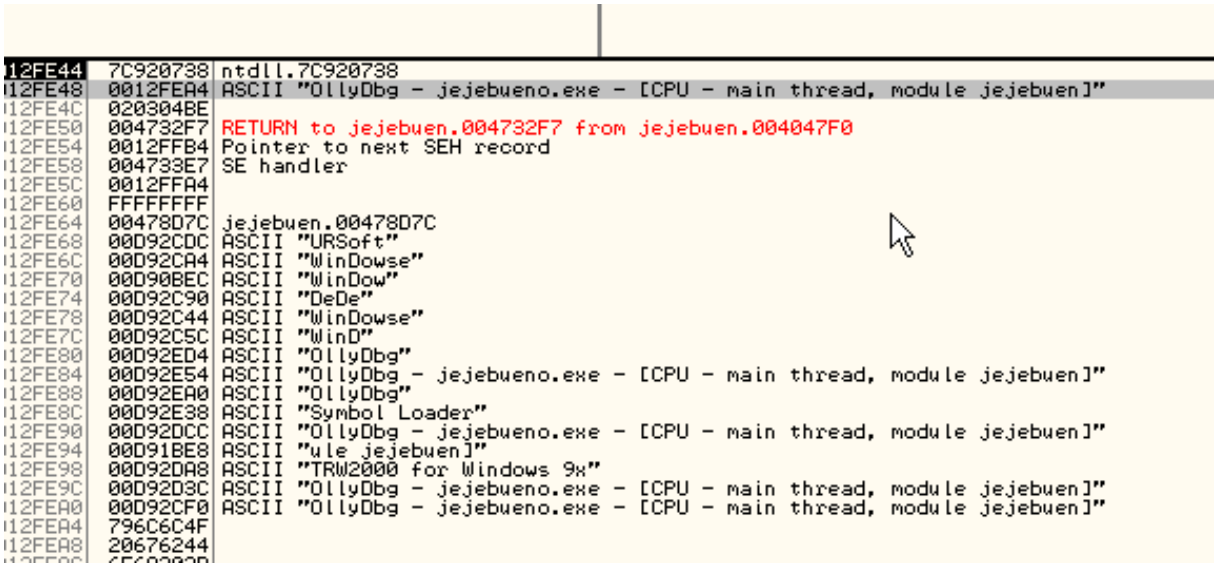


Выполняем программу и когда останавливаемся на верхнем вызове,



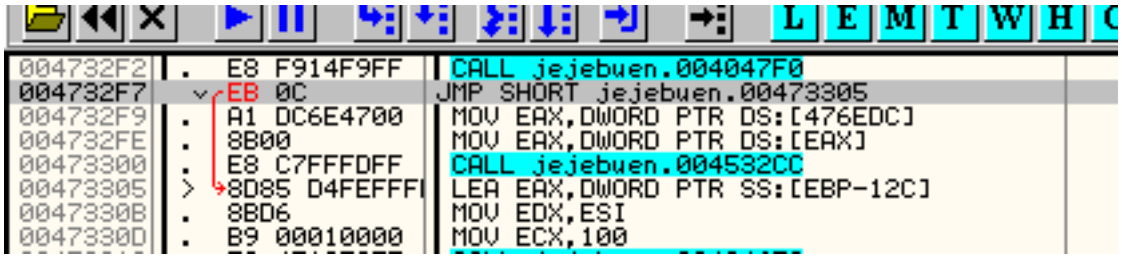
Видим, что здесь сравнивается.



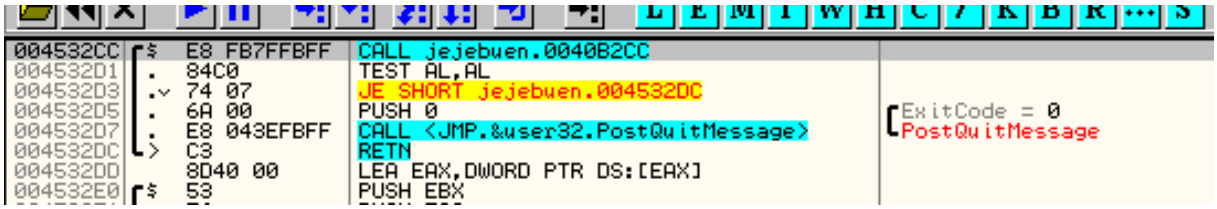


Вопрос в том, как избежать сравнения этих нехороших вещей, хе-хе, можно сделать это, заменив JNZ, который стоит после первого CALL, на JMP.

Этот загадочный переход заменяется на JMP.



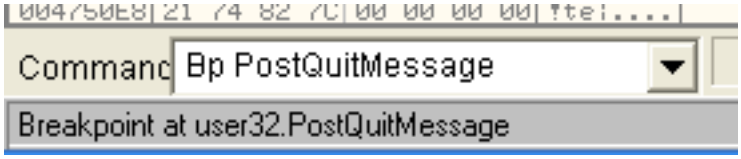
Избегаем второго CALL, внутри которого находится следующее.

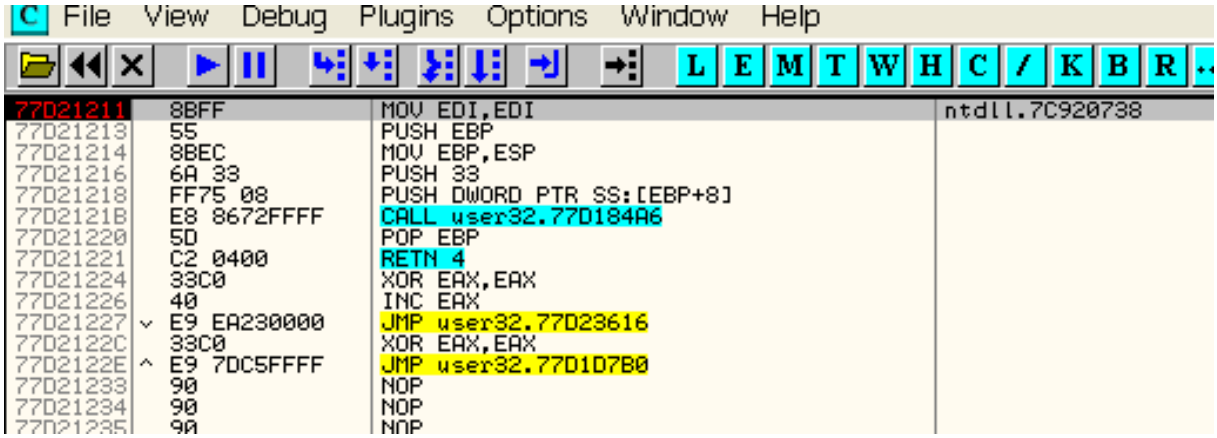


Здесь находится PostQuitMessage, который даст приложению сигнал о необходимости выхода из цикла обработки сообщений и завершения приложения.

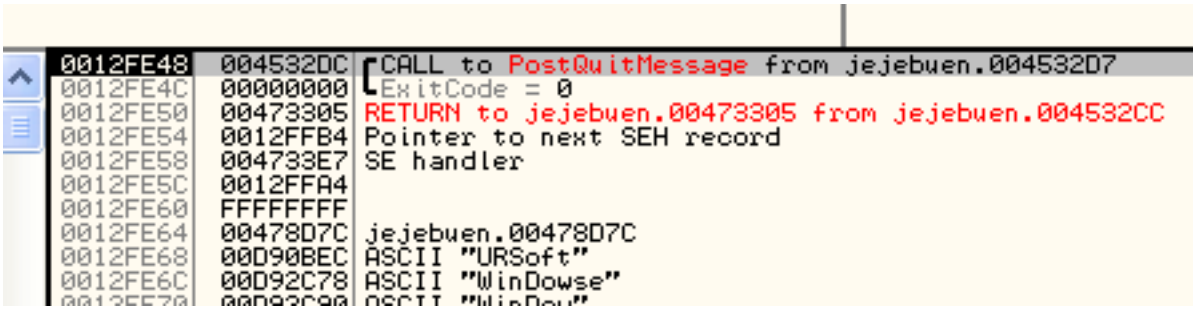
Конечно, если спросите меня, как найти этот JNZ, то это легко.

Как только прошли первую часть защиты, устанавливаем BPX на PostQuitMessage.

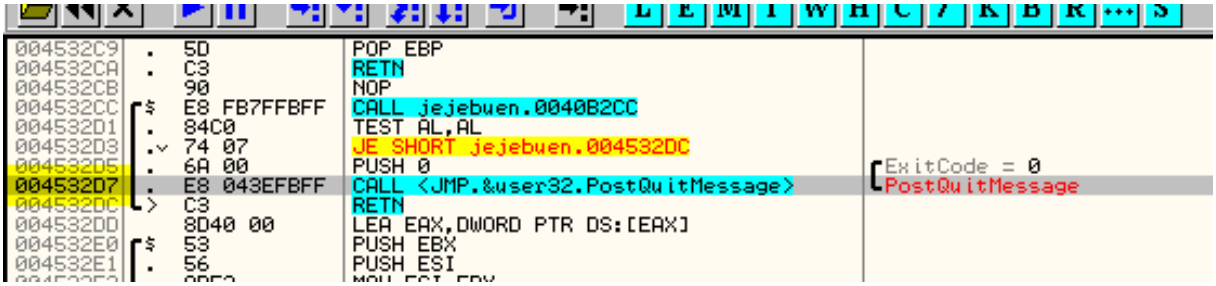




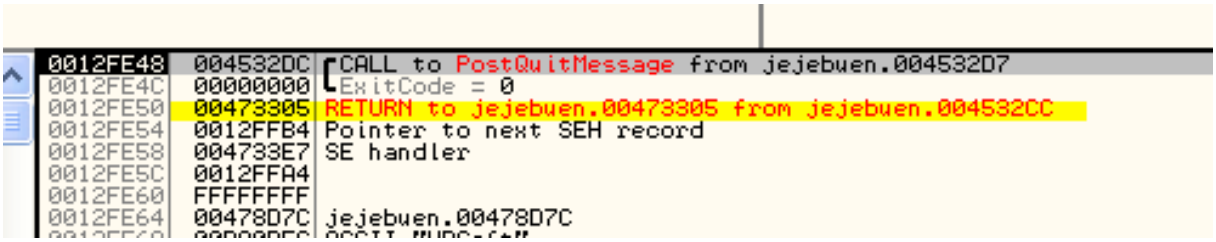
Останавливаемся на API, смотрим на стек, чтобы узнать, откуда вызывается.



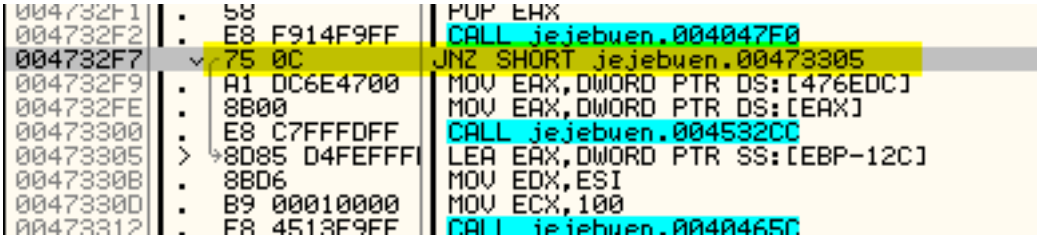
Идём в 4532d7, так что стек расскажет, откуда происходит вызов.



Конечно, я попробовал изменить JE, стоящий перед PostQuitMessage, на обратный и не смог избежать закрытия окна, так что идём ко втором RETURN TO, который есть в стеке.



Он ведёт в 4532CC.



Как видим, этот call можно избежать, обратив JNZ, и тогда не будет вызван и PostQuitMessage, и изменив JNZ на JMP видим, что крэкми выполняется без помощи каких-либо плагинов.

К статье приложен (находится в архиве `wasm_files.zip`: `files/recovered/480/muysocial.rar`) распакованный крэкми, выполняющийся под OllyDbg без каких-либо ухищрений. В этом курсе вы не будете учиться распаковке, но если вам интересен этот крэкми, то вы можете использовать приложенный файл, чтобы посмотреть, как он работает. С него уже снята распаковка и внесены необходимые изменения, чтобы он запускался под Олли без плагинов.

До встречи в 25-ой главе, где мы будем изучать исключения.

Введение в крэкинг с нуля, используя OllyDbg - Глава 25

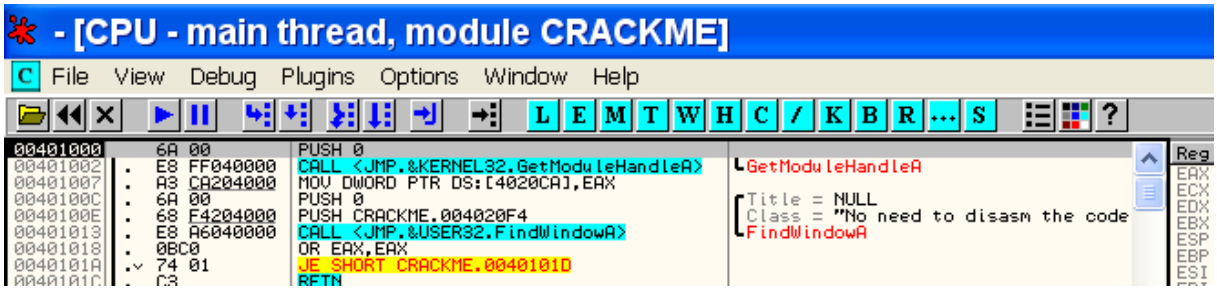
Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-03-03

ИСКЛЮЧЕНИЯ

В этой главе мы рассмотрим обработчики исключений, которые обычно являются камнем преткновения для новичков, но на самом деле это не слишком сложная тема, если немного в неё углубиться.

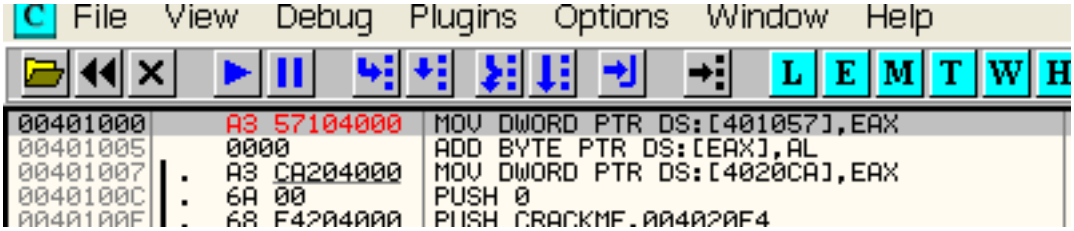
Исключение создаётся программой, когда процессор выполняет неправильную операцию. Мы увидим различные примеры того, как это случается в OllyDbg, для чего откроем крэкми CrueHead'a.



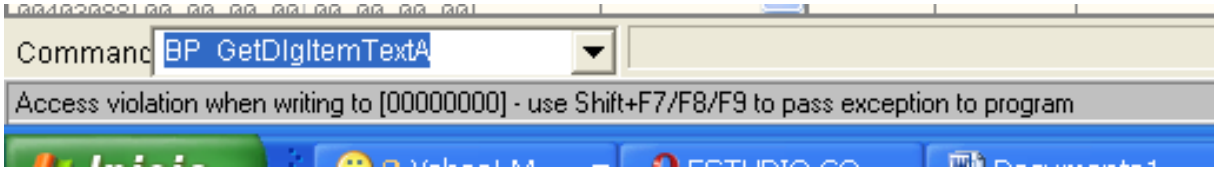
Мы находимся в точке входа и в первой строке будем писать разные инструкции ассемблера, которые приводят к возникновению исключений. Будем использовать туториал Mr Silver'a об исключениях и иллюстрировать последние конкретными примерами.

Нет доступа к памяти: возникает, когда тред пытается получить доступ к памяти не в том режиме, на который у него есть права. Данный тип исключения, например, может случиться, если поток попытается записать что-то в ячейку памяти, имея только права на её чтения.

Напишем в OllyDbg следующую строку.



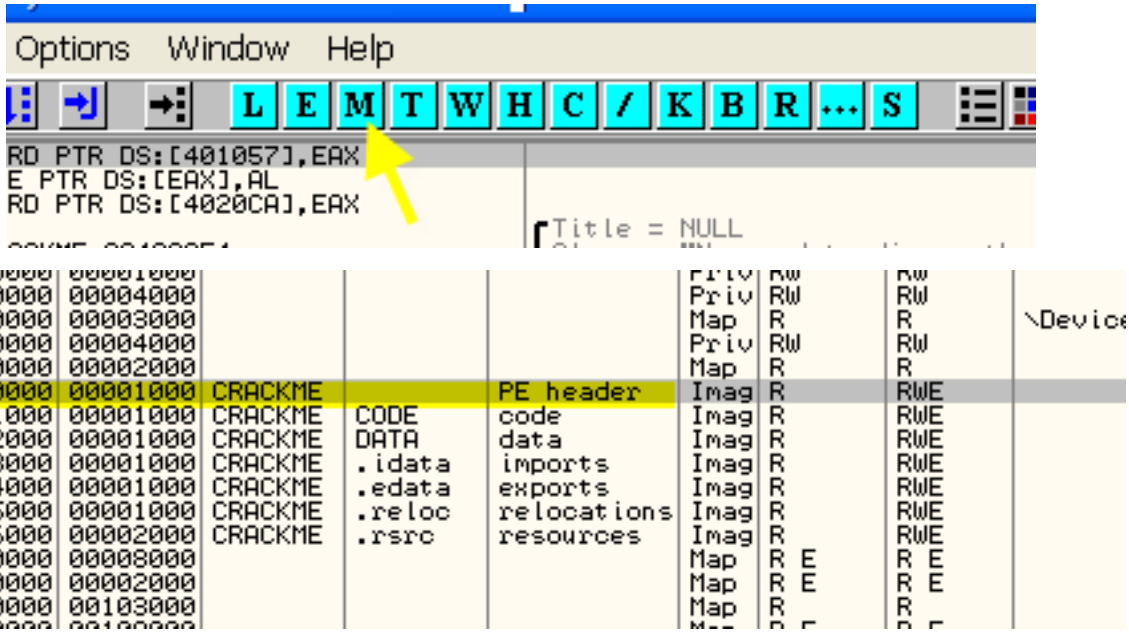
В данном случае есть только к ячейке памяти по адресу 401057 есть только право на чтение и исполнение, но не на запись, поэтому при попытке записать что-либо в данную позицию возникает исключение. Нажмём F8.



У каждой секции есть начальные настройки доступа, хранящиеся в заголовоке, до того как программа не выполнит какую-нибудь API-функцию, например VirtualProtect, позволяющую изменить настройки доступа, заданные изначально, во время выполнения.

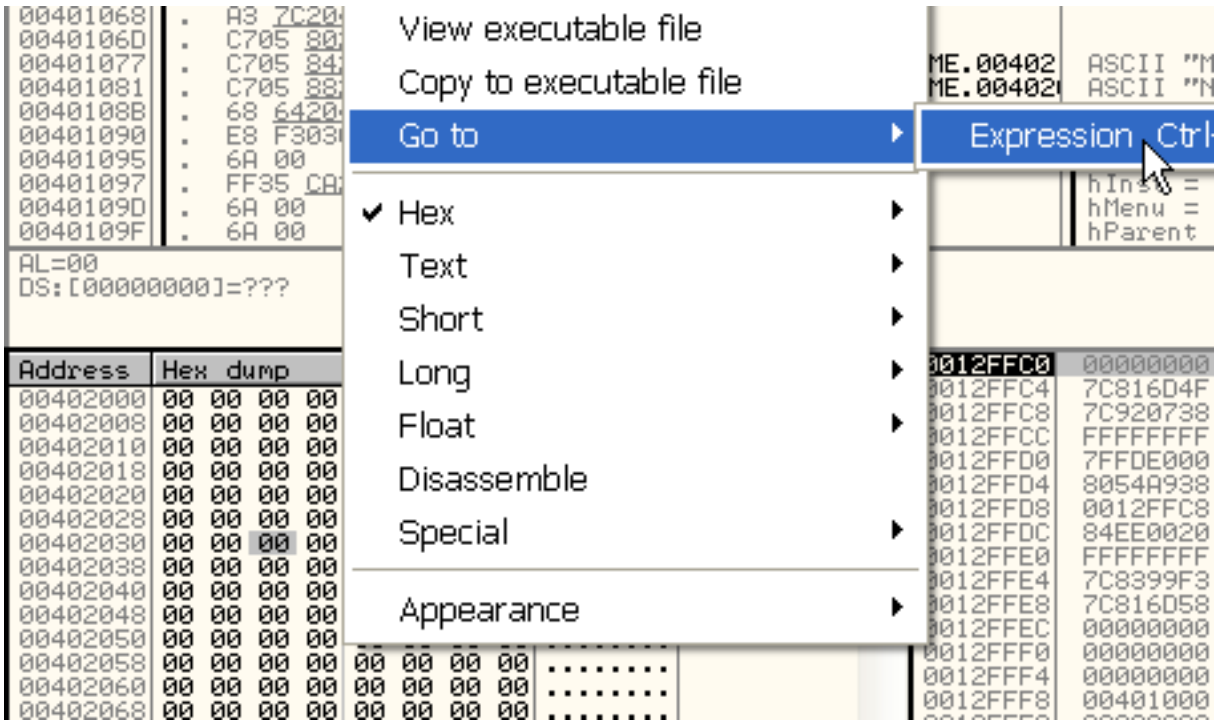
Где в OllyDbg мы можем посмотреть начальные настройки доступа для каждой секции и изменить, если нужно?

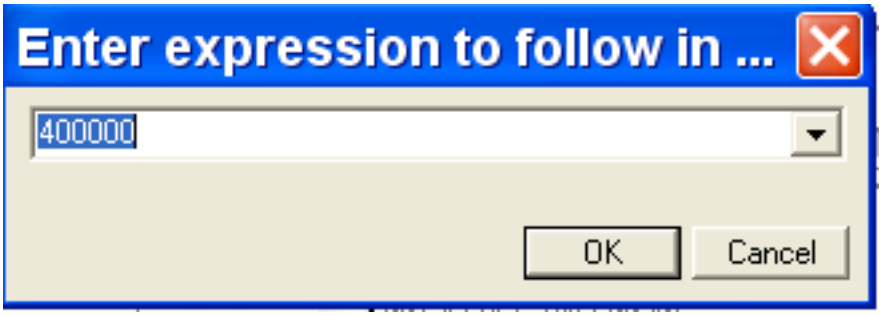
Для этого нажмём кнопку M.



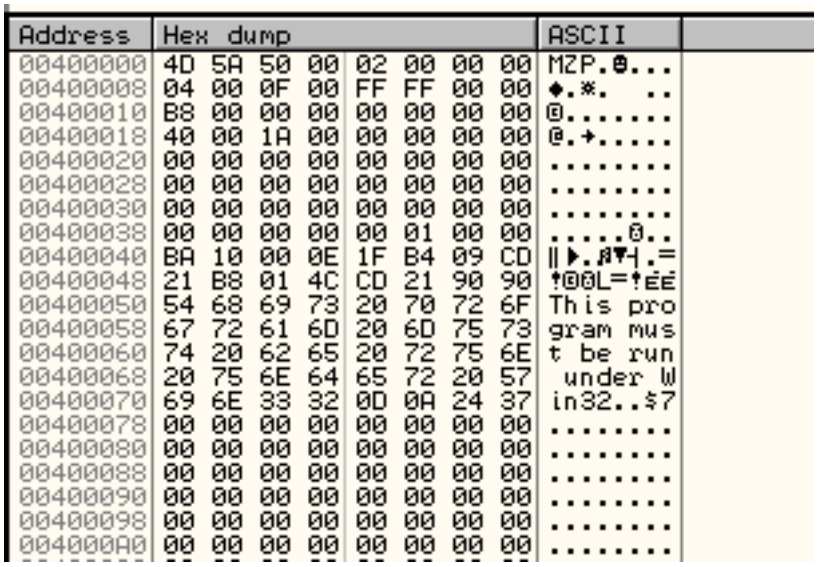
Видим, что секция, начинающаяся с 400000, т.е. первая, являющаяся заголовком PE (исполняемого файла), имеет небольшой размер в 1000 байт, в котором хранятся данные о секциях, именах, размере, в общем, всю необходимую информацию о запуске файла.

Смотрим заголовок через DUMP.

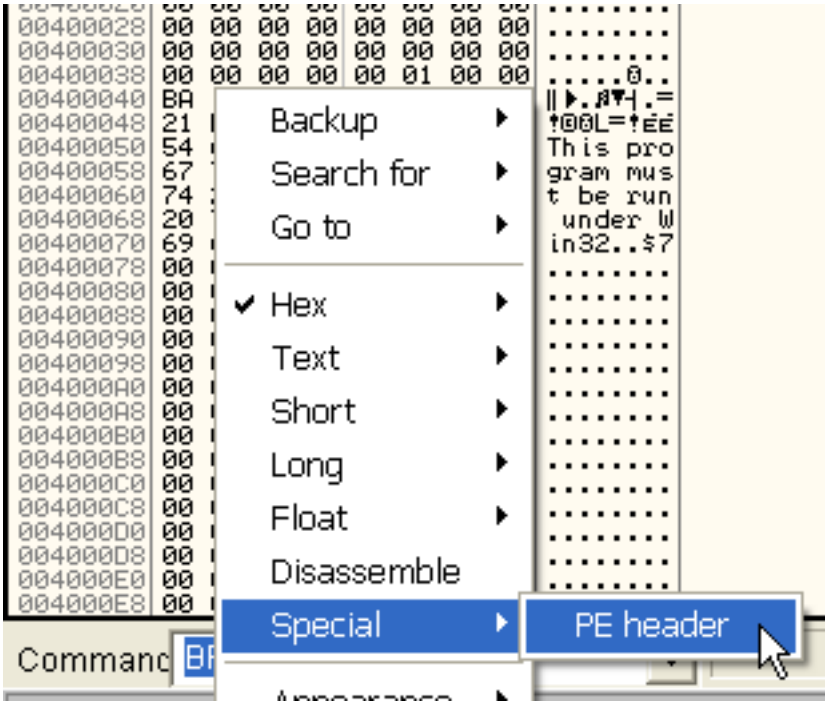




Вводим 400000, то есть адрес, откуда начинается заголовка.



У OllyDbg есть опция для интерпретация большинства параметров заголовка. Находясь в DUMP, нажимаем на правую кнопку мыши.



```
DS:[00000000]=???

Address   Hex dump    Data            Comment
00400000    4D 5A     ASCII "MZ"      DOS EXE Signature
00400002    5000      DW 0050          DOS_PartPag = 50 (80.)
00400004    0200      DW 0002          DOS_PageCnt = 2
00400006    0000      DW 0000          DOS_ReloCnt = 0
00400008    0400      DW 0004          DOS_HdrSize = 4
0040000A    0F00      DW 000F          DOS_MinMem = F (15.)
0040000C    FFFF      DW FFFF          DOS_MaxMem = FFFF (65535.)
0040000E    0000      DW 0000          DOS_ReloSS = 0
00400010    B800      DW 00B8          DOS_ExeSP = B8
00400012    0000      DW 0000          DOS_ChkSum = 0
00400014    0000      DW 0000          DOS_ExeIP = 0
00400016    0000      DW 0000          DOS_ReloCS = 0
00400018    4000      DW 0040          DOS_TablOff = 40
0040001A    1A00      DW 001A          DOS_Overlay = 1A
0040001C    00        DB 00
0040001D    00        DB 00
0040001E    00        DB 00
0040001F    00        DB 00
```

Видим, что OllyDbg отобразила нам различную информацию, содержащуюся в заголовке.

```
00400039    00          DB 00
0040003A    00          DB 00
0040003B    00          DB 00
0040003C    00010000    DD 00000100    Offset to PE signature
00400040    BA          DB BA
00400041    10          DB 10
00400042    00          DB 00
00400043    0E          DB 0E
```

Если начнём спускаться вниз, то первое, что мы встретим – это PE SIGNATURE, являющееся значением, которое говорит нам, где действительно расположена информация о заголовке. Видим, что оно содержит 100, так что нужно прибавить это число к 400000 и получим 400100. Спускаемся туда.

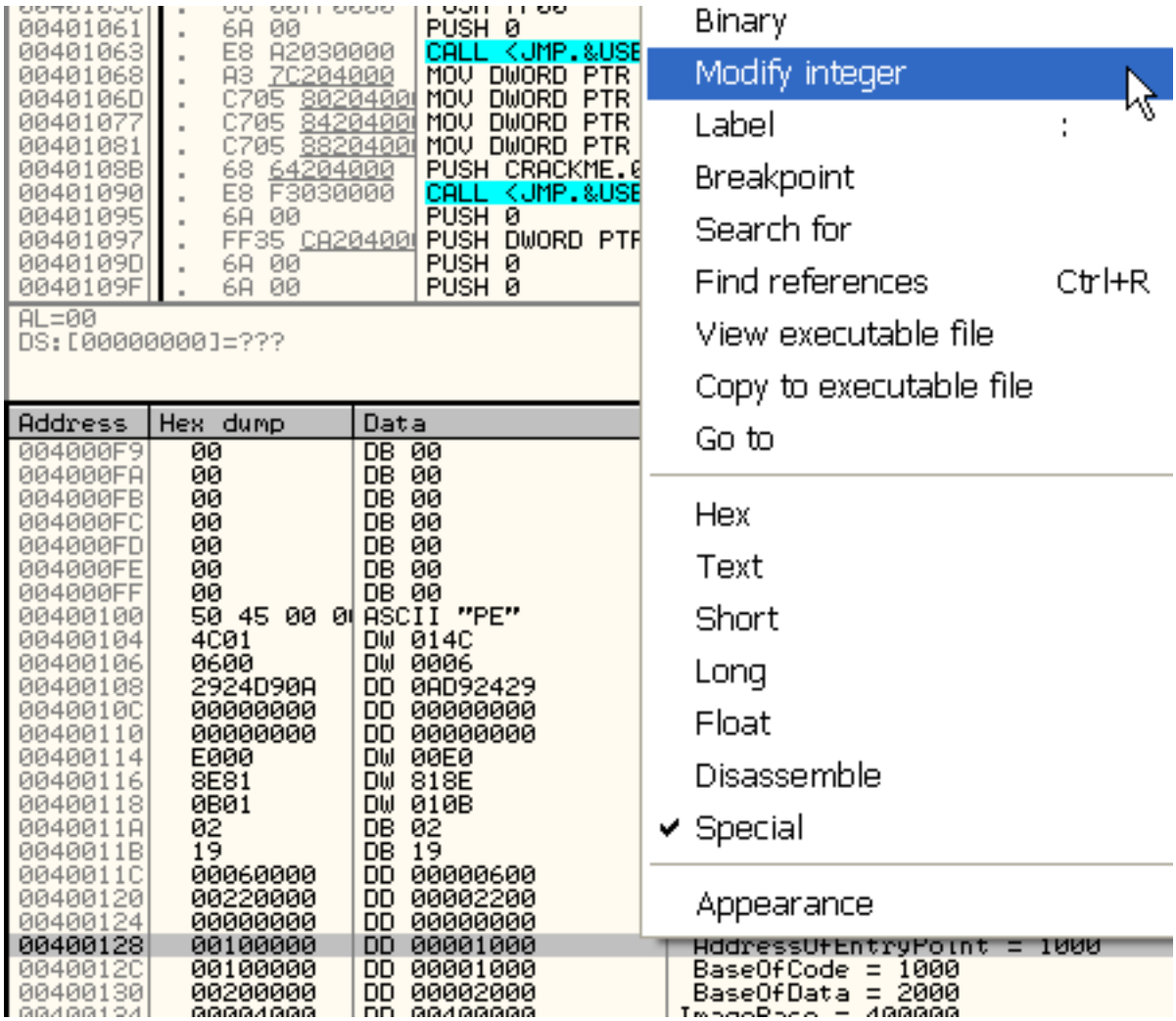
```
004000FE    00          DB 00
004000FF    00          DB 00
00400100    50 45 00 0( ASCII "PE"      PE signature (PE)
00400104    4C01        DW 014C         Machine = IMAGE_FILE_MACHINE_I386
00400106    0600        DW 0006          NumberOfSections = 6
00400108    2924D90A    DD 0AD92429      TimeDateStamp = AD92429
0040010C    00000000    DD 00000000      PointerToSymbolTable = 0
00400110    00000000    DD 00000000      NumberOfSymbols = 0
00400114    E000        DW 00E0          SizeOfOptionalHeader = E0 (224.)
00400116    8E81        DW 818E          Characteristics = EXECUTABLE_IMAGE|32BIT_MF
00400118    0B01        DW 010B         MagicNumber = PE32
0040011A    02          DB 02            MajorLinkerVersion = 2
0040011B    19          DB 19            MinorLinkerVersion = 19 (25.)
0040011C    00060000    DD 00000600      SizeOfCode = 600 (1536.)
00400120    00220000    DD 00002200      SizeOfInitializedData = 2200 (8704.)
00400124    00000000    DD 00000000      SizeOfUninitializedData = 0
00400128    00100000    DD 00001000      AddressOfEntryPoint = 1000
0040012C    00100000    DD 00001000      BaseOfCode = 1000
00400130    00200000    DD 00002000      BaseOfData = 2000
00400134    00004000    DD 00400000     ImageBase = 400000
00400138    00100000    DD 00001000      SectionAlignment = 1000
0040013C    00020000    DD 00000200      FileAlignment = 200
00400140    0100        DW 0001          MajorOSVersion = 1
Command  BP_GetDlgItemTextA
```

Как видим, никакой ошибки – именно отсюда начинается важная информация о программе.

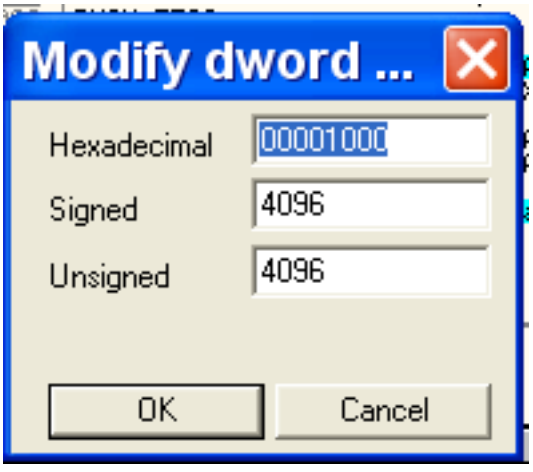
Поясним более подробно, что означают некоторые из этих параметры.

```
0040011C    00060000    DD 00000600      SizeOfCode = 600 (1536.)
00400120    00220000    DD 00002200      SizeOfInitializedData = 2200 (8704.)
00400124    00000000    DD 00000000      SizeOfUninitializedData = 0
00400128    00100000    DD 00001000      AddressOfEntryPoint = 1000
0040012C    00100000    DD 00001000      BaseOfCode = 1000
00400130    00200000    DD 00002000      BaseOfData = 2000
00400134    00004000    DD 00400000     ImageBase = 400000
```

То есть, значение этого указателя (1000) надо прибавить к 400000, и получим адрес точки входа программы. Если хотим изменить его, например на 2000, нажимаем на правую кнопку мыши.



Мы можем написать любое число, которое ходим, например, если нужно, чтобы программа начиналась с 402000, то пишем тут 2000, так как отсчёт всегда начинается с 400000, то есть с этого адреса начинается заголовок, который является первой секцией программы.



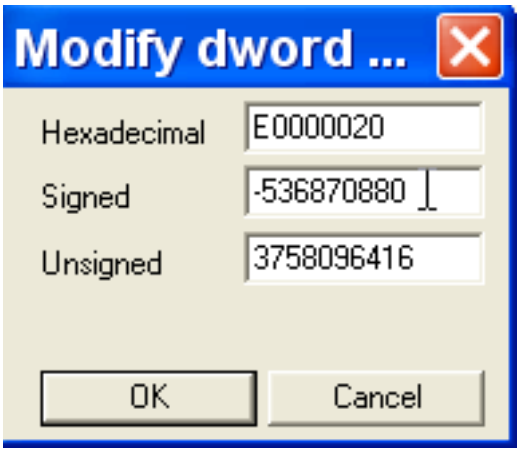
Затем, чтобы сохранить изменения в файле, нажимаем правую кнопку мыши – COPY TO EXECUTABLE, а в возникшем окне ещё раз правая кнопка мыши и SAVE FILE. Сейчас этого делать не будем, я просто хотел, чтобы вы знали об этой возможности.

Ок, продолжаем идти вниз по заголовку.

```
004001F4   00000000   DD 00000000          Reserved
004001F8   43 4F 44 4 ASCII "CODE"         SECTION
00400200   00100000   DD 00001000          VirtualSize = 1000 (4096.)
00400204   00100000   DD 00001000          VirtualAddress = 1000
00400208   00060000   DD 00000600          SizeOfRawData = 600 (1536.)
0040020C   00060000   DD 00000600          PointerToRawData = 600
00400210   00000000   DD 00000000          PointerToRelocations = 0
00400214   00000000   DD 00000000          PointerToLineNumbers = 0
00400218   0000       DW 0000              NumberOfRelocations = 0
0040021A   0000       DW 0000              NumberOfLineNumbers = 0
0040021C   20000060   DD 60000020          Characteristics = CODE|EXECUTE|READ
```

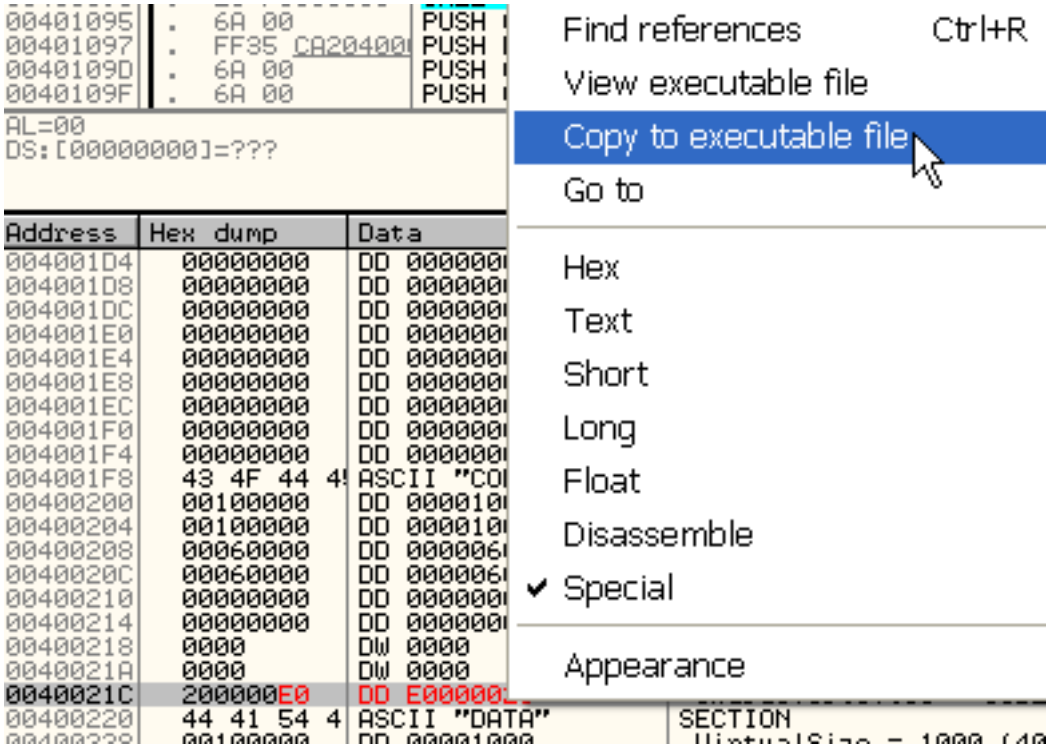
Здесь начинаются данные секций, видим, что секция начинается с позиции 1000, т.е. 401000, а её характеристиками являются CODE, EXECUTE и READ.

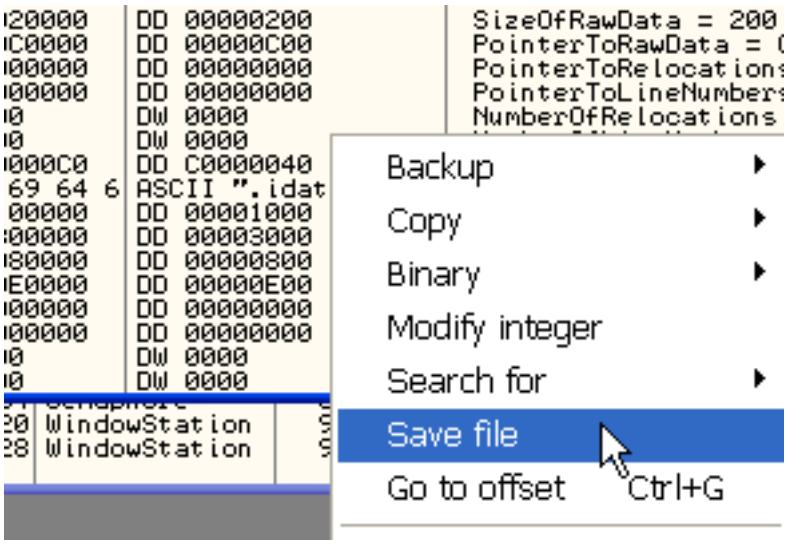
Если мы хотим, чтобы у этой секции было разрешение на запись, то нужно изменить значение этого поля с 60000020 на E0000020. Это число задаёт сразу права на всё, хе-хе, пробуем.

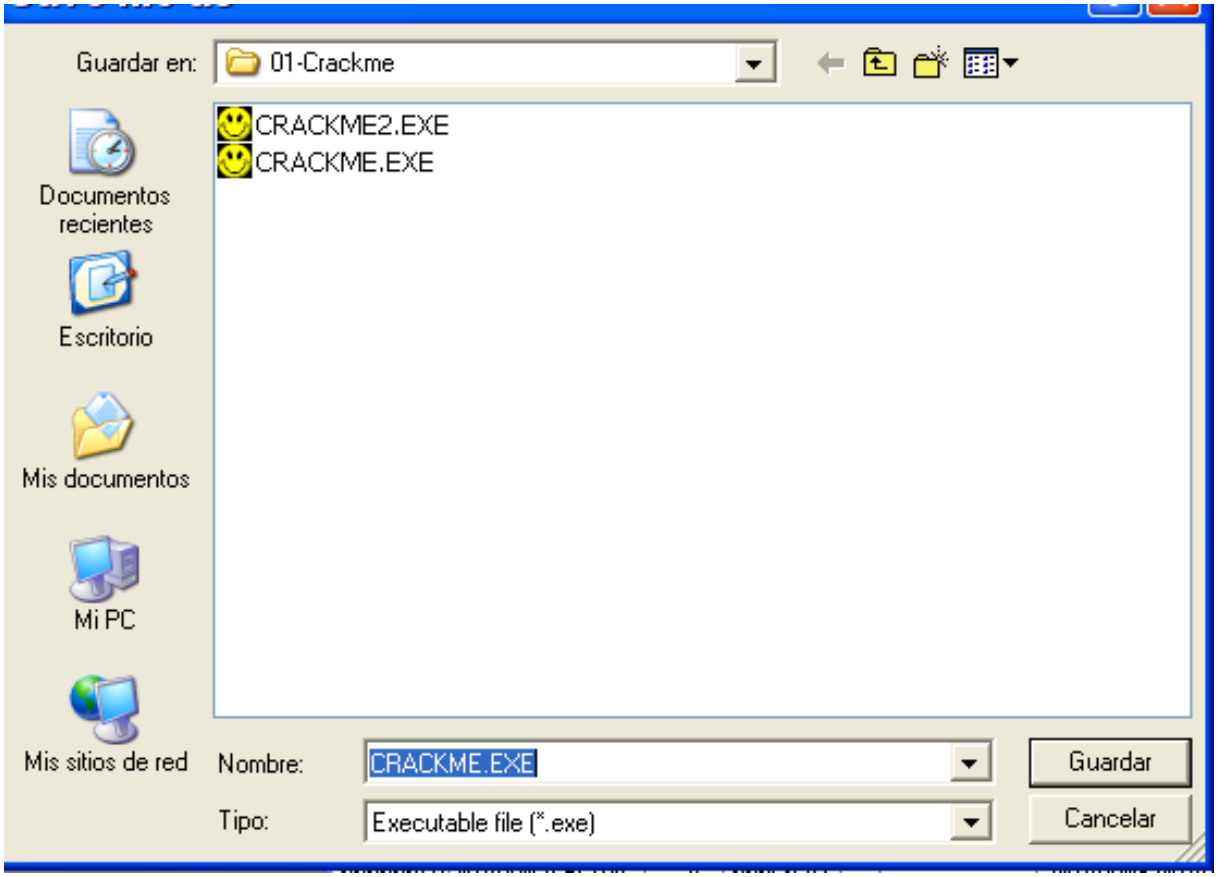



```
004001F4   00000000   DD 00000000          Reserved
004001F8   43 4F 44 4 ASCII "CODE"         SECTION
00400200   00100000   DD 00001000          VirtualSize = 1000 (4096.)
00400204   00100000   DD 00001000          VirtualAddress = 1000
00400208   00060000   DD 00000600          SizeOfRawData = 600 (1536.)
0040020C   00060000   DD 00000600          PointerToRawData = 600
00400210   00000000   DD 00000000          PointerToRelocations = 0
00400214   00000000   DD 00000000          PointerToLineNumbers = 0
00400218   0000       DW 0000              NumberOfRelocations = 0
0040021A   0000       DW 0000              NumberOfLineNumbers = 0
0040021C   200000E0   DD E0000020          Characteristics = CODE|EXECUTE|READ|WRITE
00400220   44 41 54 4 ASCII "DATA"         SECTION
00400228   00100000   DD 00001000          VirtualSize = 1000 (4096.)
0040022C   00200000   DD 00002000          VirtualAddress = 2000
00400230   00020000   DD 00000200          SizeOfRawData = 200 (512.)
```

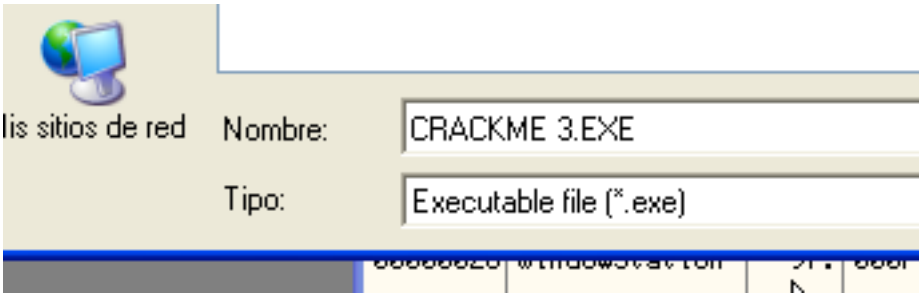
Хорошо, теперь сохраняем значения с помощью обычной процедуры.


Find references Ctrl+R
View executable file
Copy to executable file
Go to
Hex
Text
Short
Long
Float
Disassemble
Special
Appearance
Address
Hex dump
Data
AL=00
DS:[00000000]=???

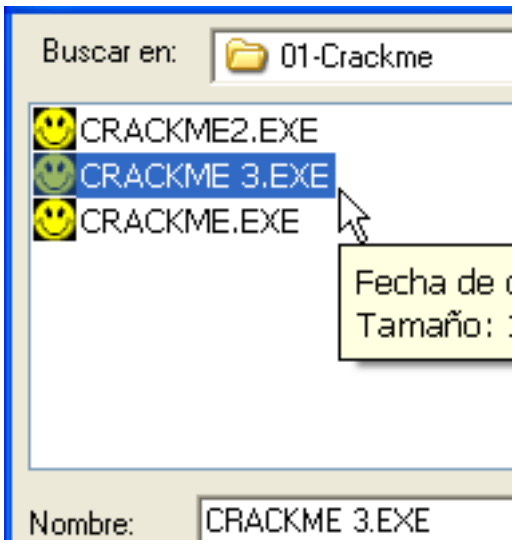

Backup
Copy
Binary
Modify integer
Search for
Save file
Go to offset Ctrl+G
SizeOfRawData = 200
PointerToRawData =
PointerToRelocations
PointerToLineNumbers
NumberOfRelocations

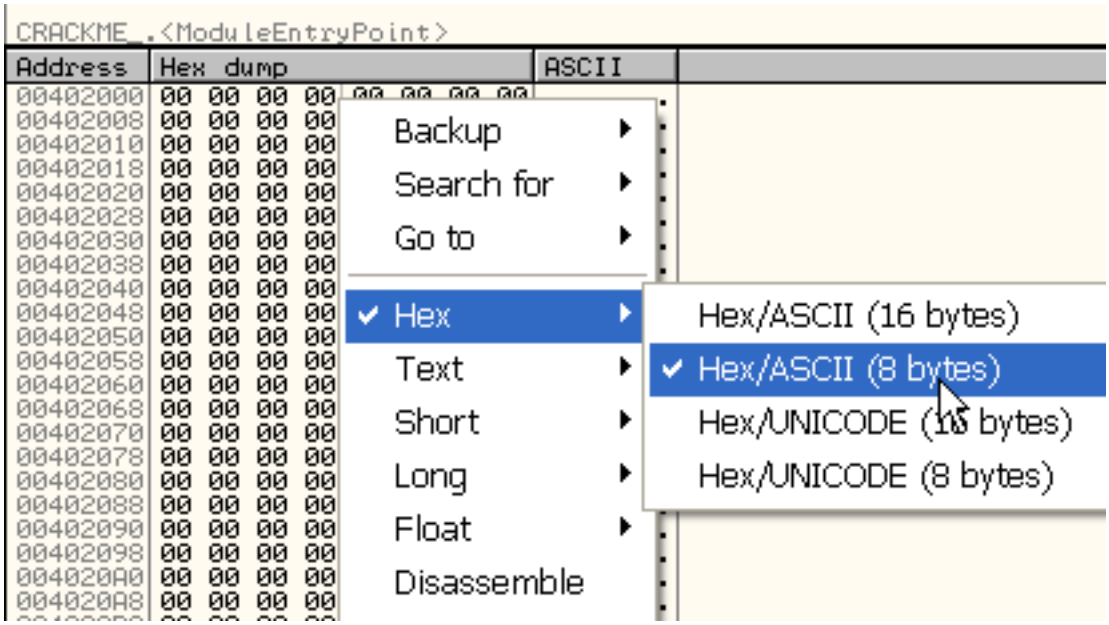


Меняем имя на CRACKME 3, чтобы знать, что это модифицированный нами вариант.



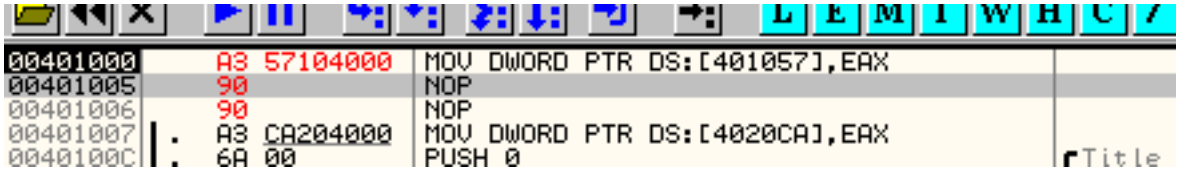
Теперь откроем этот crackme 3 в OllyDbg.





Вернём обычный режим отображения DUMP'а.

Теперь попробуем ту инструкцию, которая вызывала ошибку при записи.



Нажимаем F8.

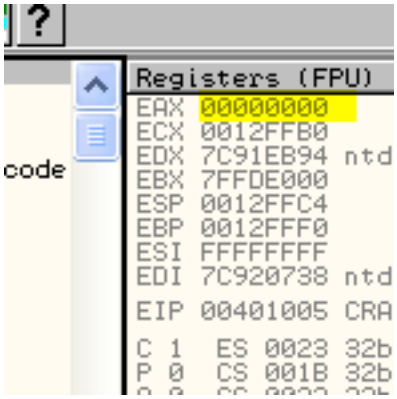
Видим, что исключения не возникло и значение EAX прекрасно сохранилось по адресу 401057.

Другим типом исключения является:

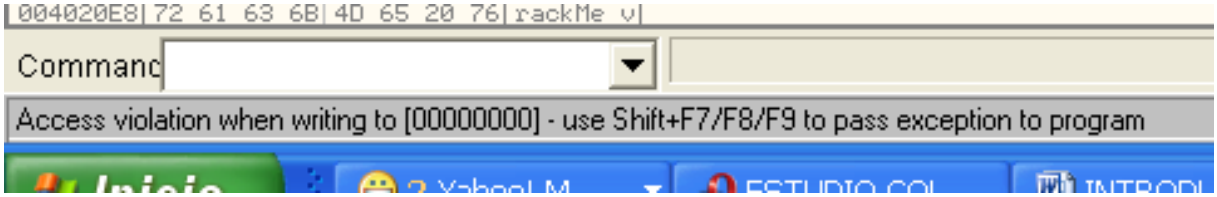
Деление на 0: вызывается, когда пытаемся поделить число на 0.

Например, напишем в OllyDbg:

Здесь происходит деление ECX на EAX, и если последний равен нулю, то будет вызвано исключение.



Нажимаем F8.



Не всегда OllyDbg предоставляет ясную информацию, в данном случае, она не сказала, что происходит деление на ноль. Перепрыгиваем через исключение.

Неверная инструкция, попытка запуска привилегированной инструкции: возникает, когда процессоры пытается выполнить инструкцию, не относящуюся к набору, поддерживаемого им, например, при встрече неизвестной операции.

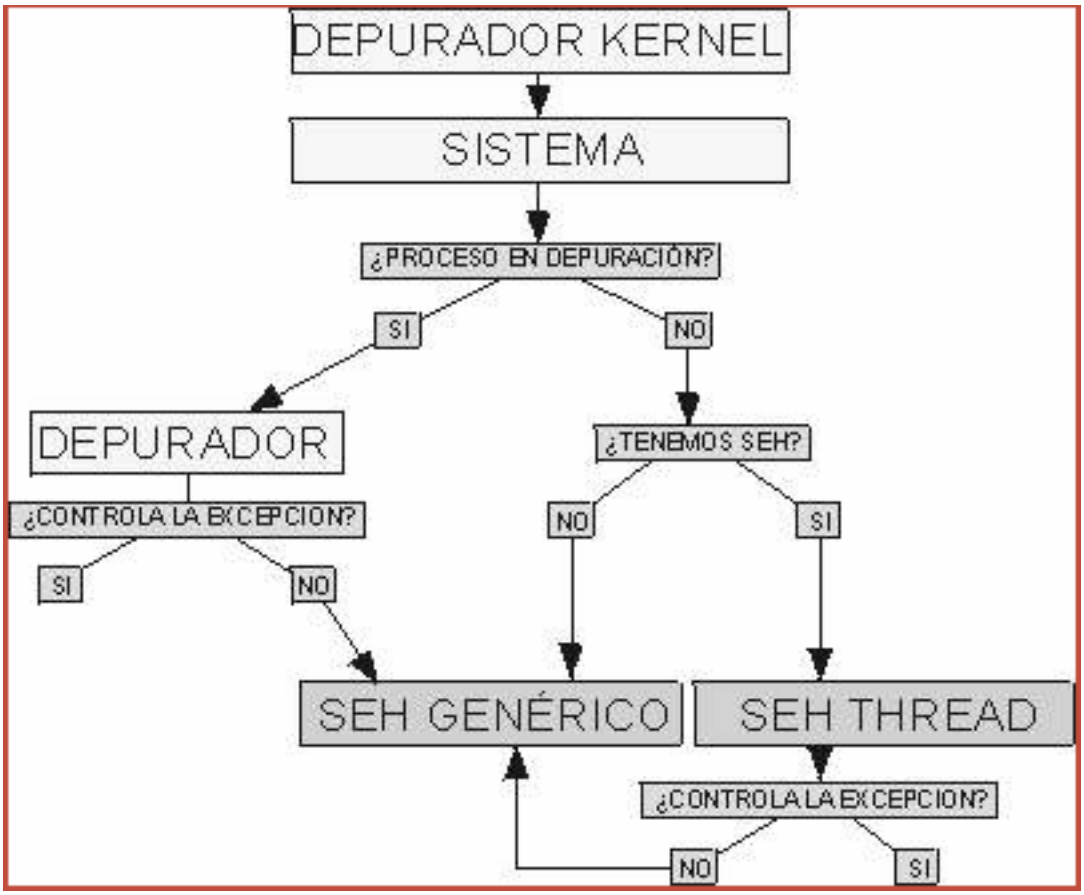
Этот случай мы не сможем опробовать, так как OllyDbg не даёт нам задавать инструкции, которые не существуют, но программист может задать такую, которую процессор не будет поддерживать, и тогда выдастся соответствующая ошибка.

Также очень известным является INT 3, которое генерирует исключение, и используется отладчиком, когда устанавливаем обычный BPX, чтобы остановить программу и передать управление ею туда, где INT 3 был вызван.

Также некоторые программы вызывают INT 3 напрямую, это исключение является одним из наиболее возможных.

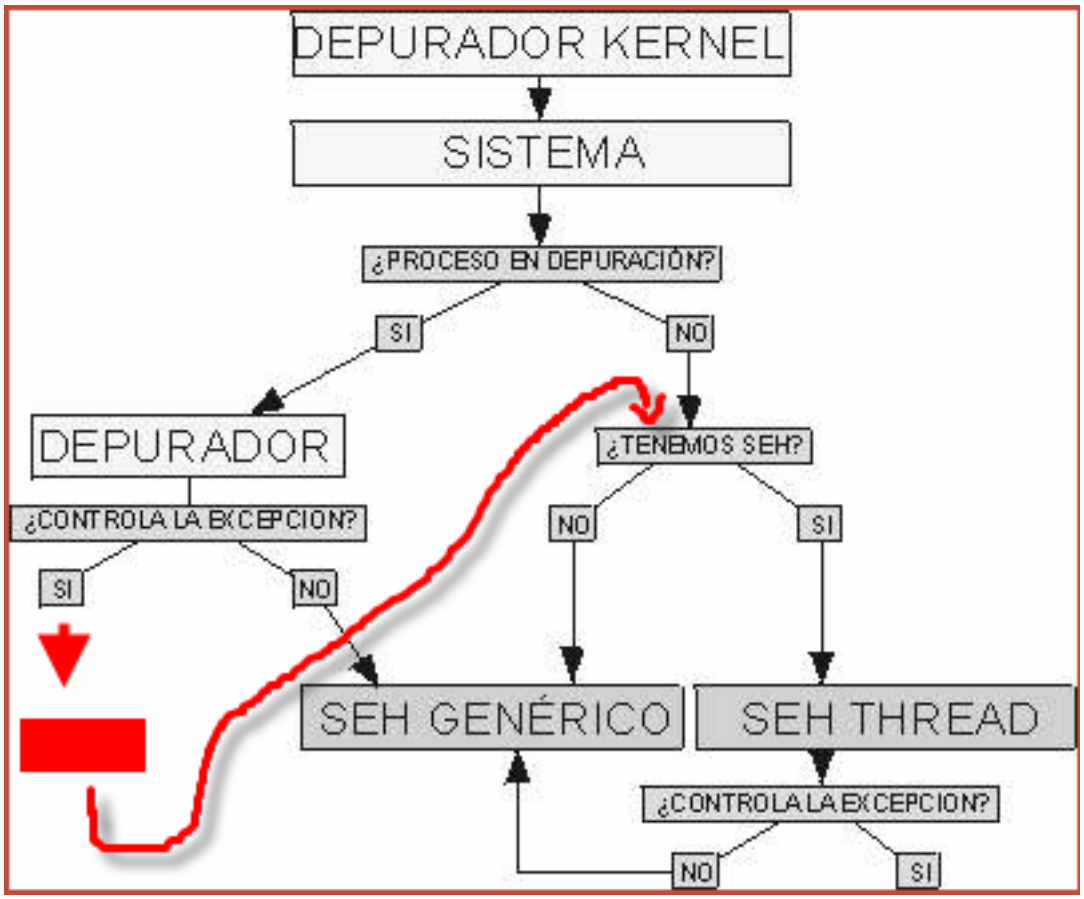
Есть много других исключение, и все мы рассматривать не будем. Вот простой пример.

Мы теперь знаем, что программа может генерировать исключения, но что именно при этом происходит? Рассмотрим следующую схему:



На этом графике, который я стащил из туториала SILVER'a, изображено, как возникает исключение при условии, что процесс находится под отладкой.

В случае, если программа находится под отладкой, контроль получает отладчик, который (как это изображено на картинке, смотрит, может ли он контролировать исключение или нет, почти всегда ответ утвердительный и тогда, если включена опция в DEBUGGING OPTIONS-EXCEPTIONS для перепрыгивания через этот тип исключений, контроль возвращается обратно программе, если не включена, то для этого нужно нажать SHIFT+F9, то есть на графике, где сказано CONTROLA LA EXCEPCION-SI отсутствует изображение, отображающее процесс принятия решения, останавливать или продолжать выполнение программы в зависимости от вышеуказанной опции. Смотрим следующую схему.



Здесь видим весь процесс полностью – добавлена красная стрелка. После CONTROLAR LA EXCEPCION происходит определение, нужно ли останавливать или продолжать выполнение программы в зависимости от того, включена ли соответствующая опция, если происходит остановка, то ждём нажатия SHIFT+F9.

Далее видим, что когда возвращаемся из OllyDbg, то происходит проверка, установлен ли SHE, если да, то переходим к нему, если нет, то используем SEH по умолчанию. Это может показаться сложным, но на самом деле, это не так. Объясню поподробнее.

ЧТО ТАКОЕ SEH

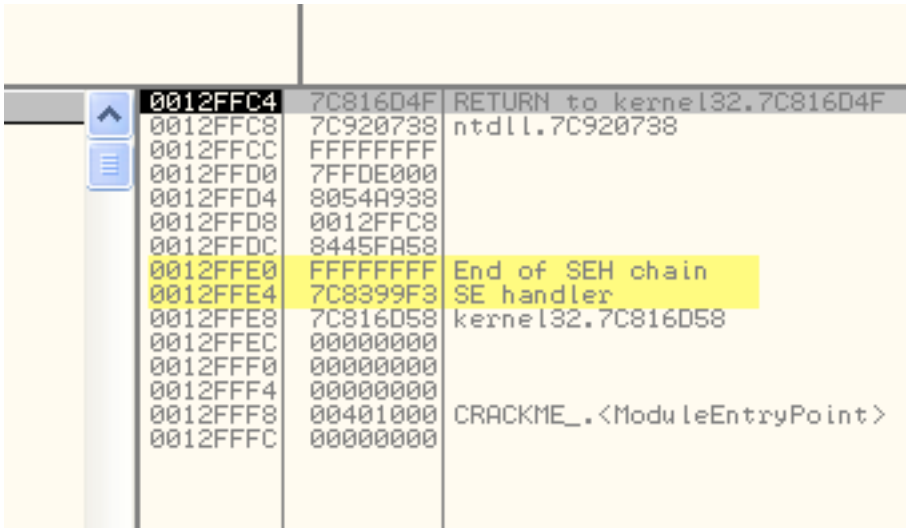
SEH или Structured Exception Handling – это обработчик, который служит для того, чтобы программа имела возможность восстановиться после ошибки, то есть, если не установили собственный SHE-обработчик, то при возникновении ошибки выполняется системный обработчик, который отображает нам сообщение об ошибке в приложении, программа закрывается и адью. Установив же собственный SEH-обработчик, можем перехватить ошибку, обработать её и возвратить управление программе, которая, вместо закрытия и назойливых сообщений от системы, продолжит

выполняться.

Также, хотя мы и не рассматривали серьёзно треды, вот минимальное определение этого термина: это «нити» программы или части, которые могут выполняться одновременно, соответственно, у каждого треда есть собственный обработчик исключений, то есть, если тред установил его, то последний будет выполняться только в нём, а не в других.

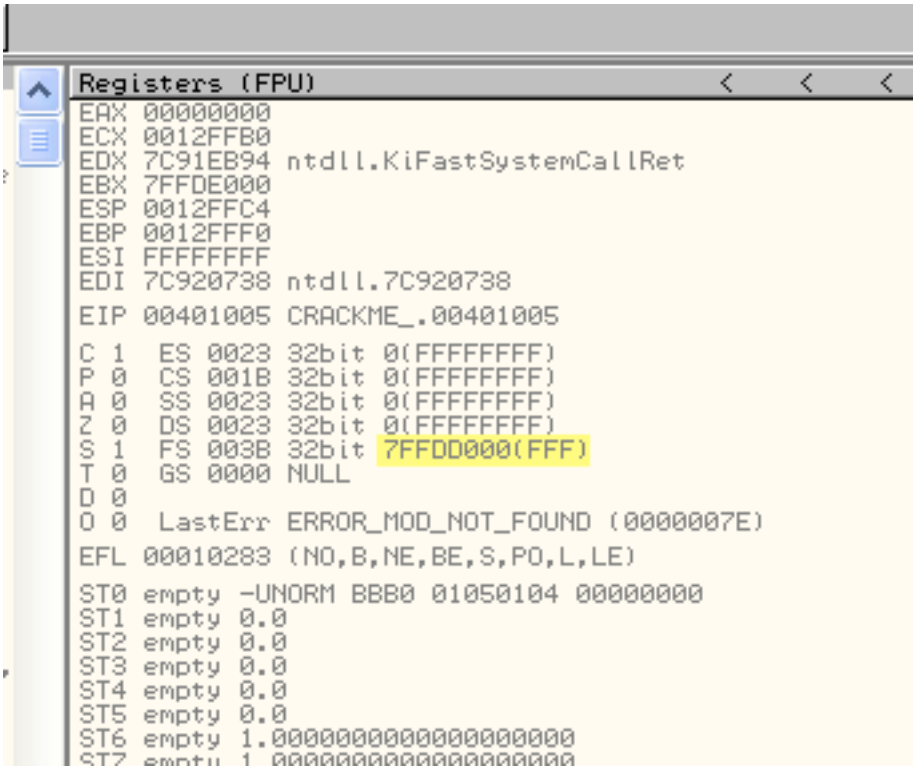
Как устанавливается обработчик исключений

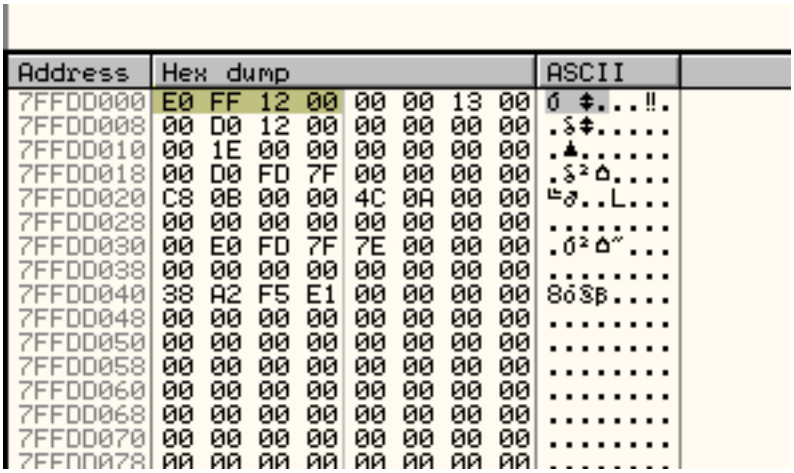
Ок, снова откроем крэкми CRUEHEAD'а и посмотрим в стек.



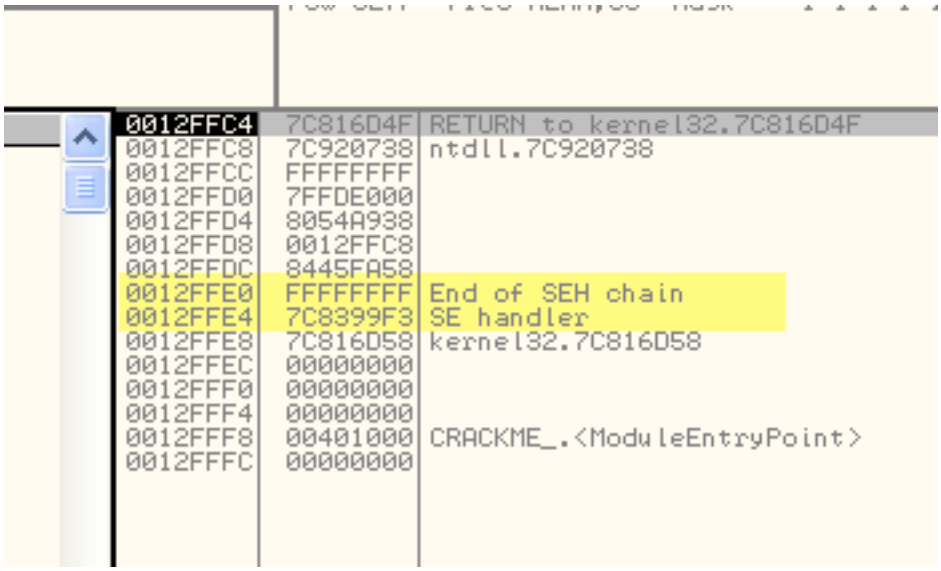
Это SEH-обработчик по умолчанию, устанавливаемый системой, и он будет срабатывать при любой ошибке до тех пор, пока не будет заменён на наш собственный, выдавая зловещее сообщение об ошибке. Посмотрим, где находятся указатели на системный SEH-обработчик.

Как можем видеть, в FS:[0] находится текущий указатель на обработчик исключений, можем здесь перейти в DUMP.

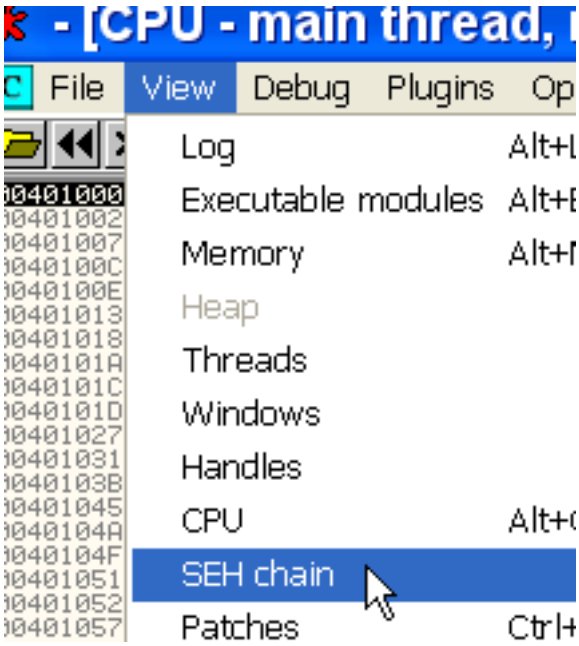




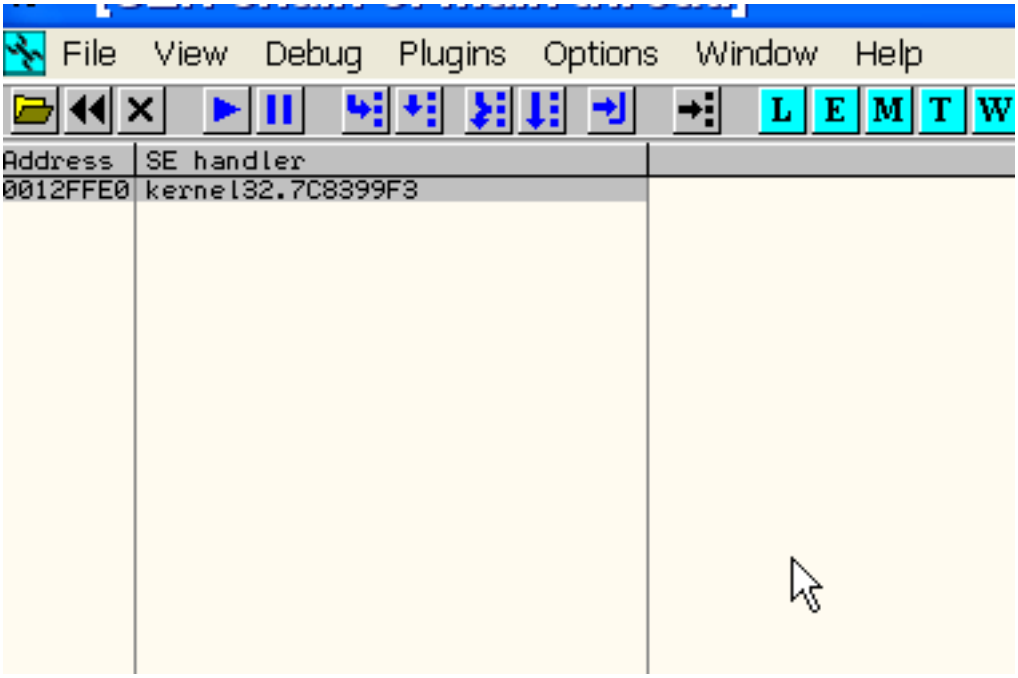
Как можно видеть, FS:[0] – это содержимое памяти по адресу, отмеченное OllyDbg, оно может быть разным на разных машинах, но в моём случае FS:[0] всегда содержит 12ffe0.



Как видим в стеке, 12ffe0 указывает на последний обработчик в цепочке, то есть на тот, который работает именно в данном треде, и именно по этому адресу происходит передача управления, когда возникает исключение. Системный обработчик при этом отображает нам знаменитое сообщение об ошибке в приложении.



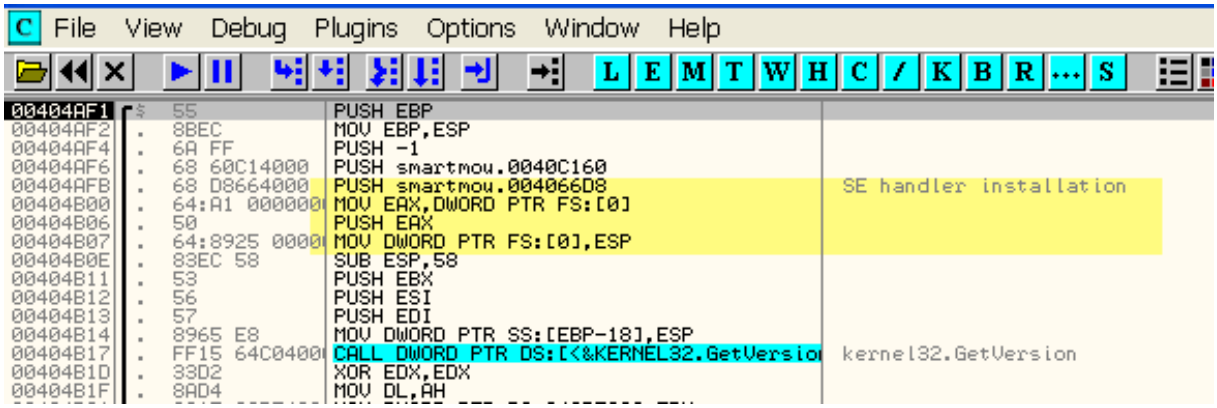
И если перейдём по VIEW-SEH CHAIN:



Видим адрес системного обработчика, который является и единственным установленным, а если бы их было больше, то обработкой исключений занимался бы находящийся в самом верху списка.

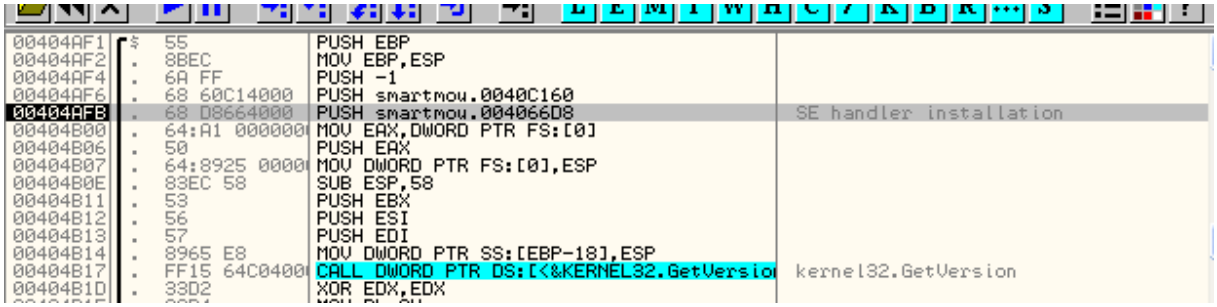
Так как крэкми CRUEHEAD'а не устанавливает собтсвенных обработчиков исключений, используем программку под названием SMARTMOUSE, которая прилагается к этой главе.

Загрузим её в OllyDbg.

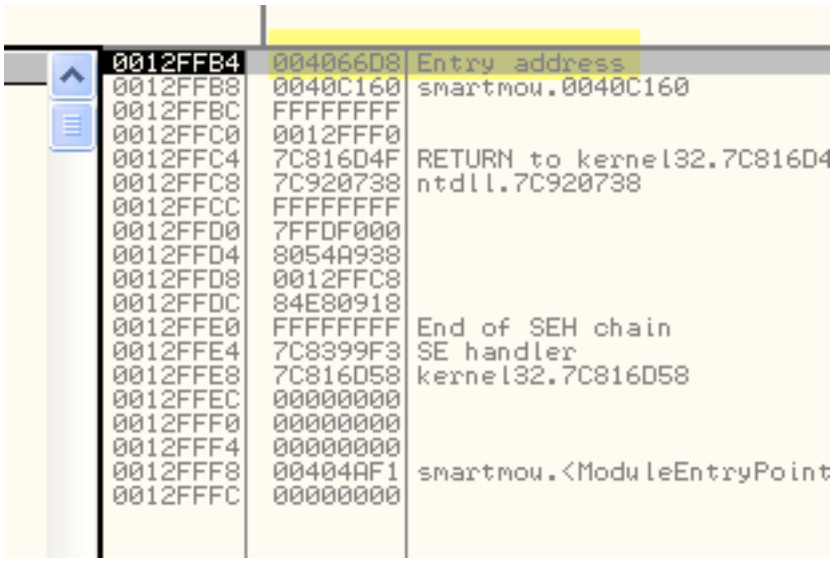


Здесь видно, что в начале программа собирается установить собственный обработчик исключений, о чём прямо говорится в комментарии OllyDbg.

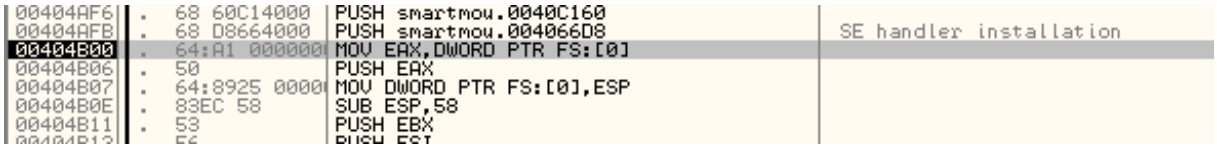
Оттрассируем по шагам и объясним каждый из них.



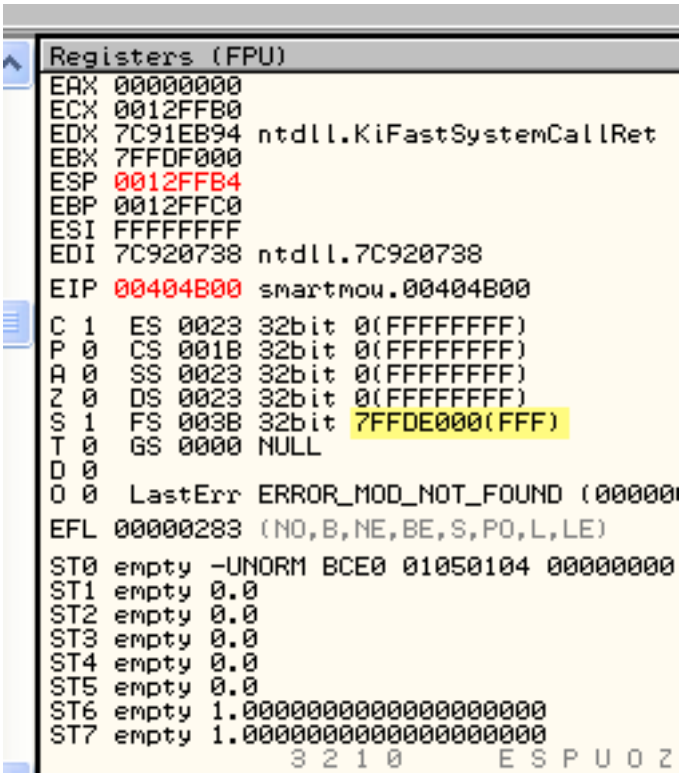
Инструкция, с которой, как указывает OllyDbg, начинается установка обработчика исключений, это PUSH адреса, на который должна переходить программа при возникновении исключения. В данном случае этот адрес будет. Выполняем данную инструкцию.



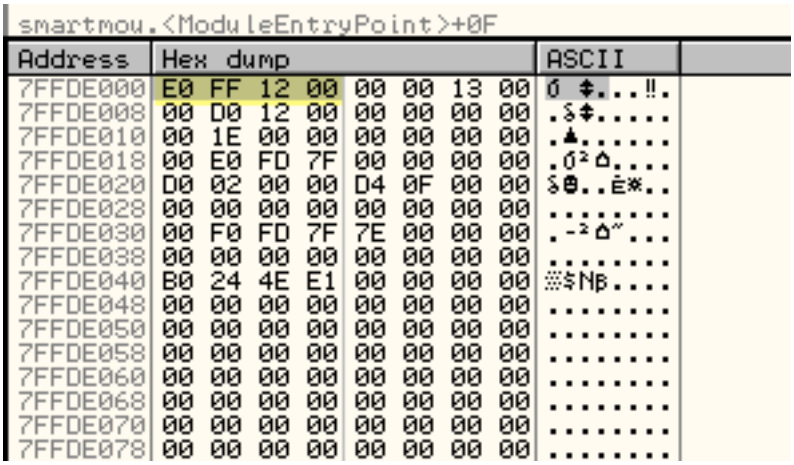
Адрес сохраняется в стек.



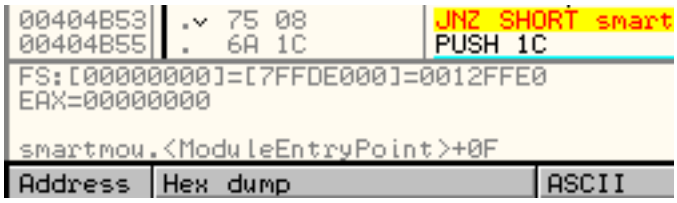
Следующая строка перемещает действительное значение fs : [0] в EAX. Посмотрим, чему равен FS:[0] через DUMP.



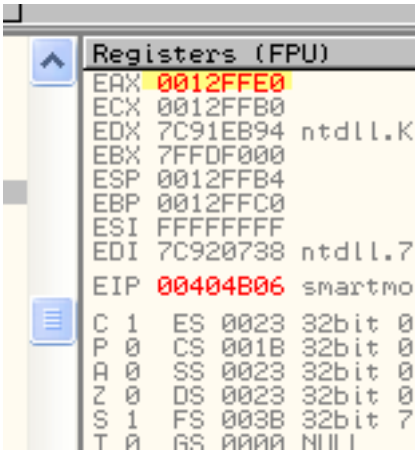
Идём туда.



Видим, что fs:[0] равно 12ffe0. Можем посмотреть, какую информацию предоставляет по этому поводу OllyDbg.



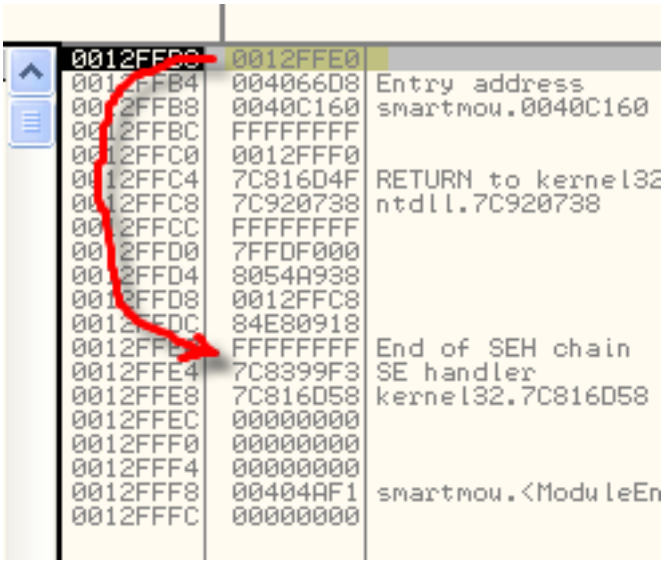
Теперь выполняем эту строку.



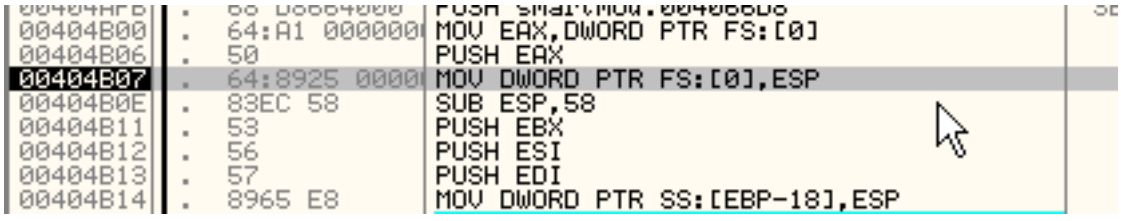
Перемещаем это значение в EAX.

Ahora hace un PUSH con ese valor

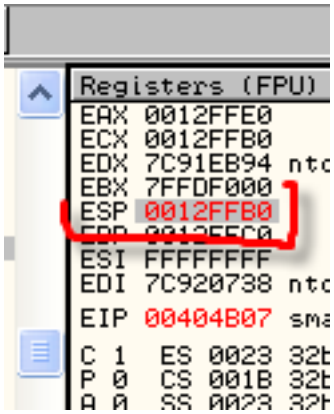
Теперь делаем PUSH с этим значением.

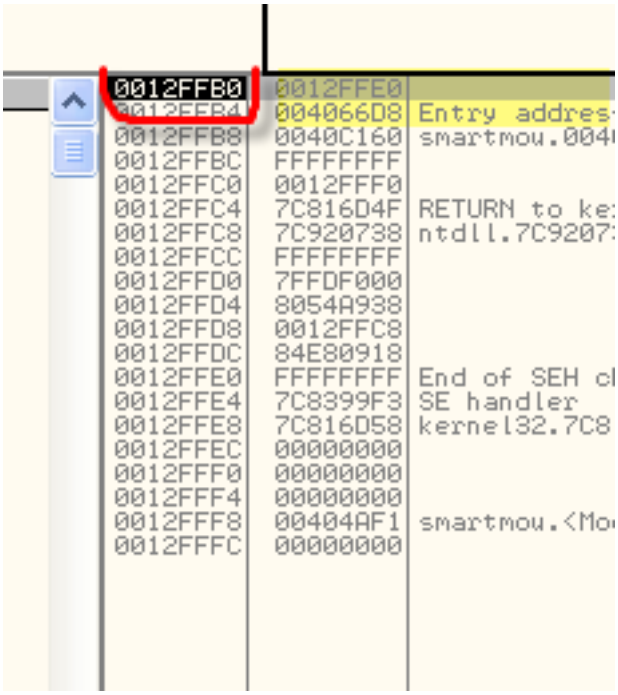


SEH CHAIN означает, что это цепочка. Сохраняемое значение указывает на предыдущий обработчик, являющийся на данный момент активным.

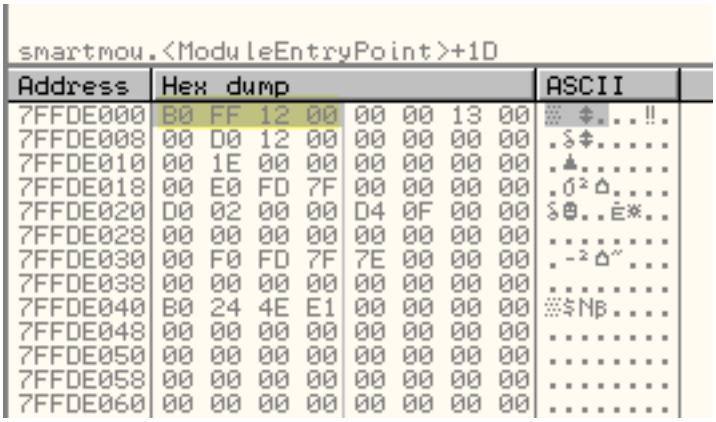


Последняя строка меняет содержимое fs:[0], куда помещается содержимое ESP, указывающая туда, где находится новая структура.

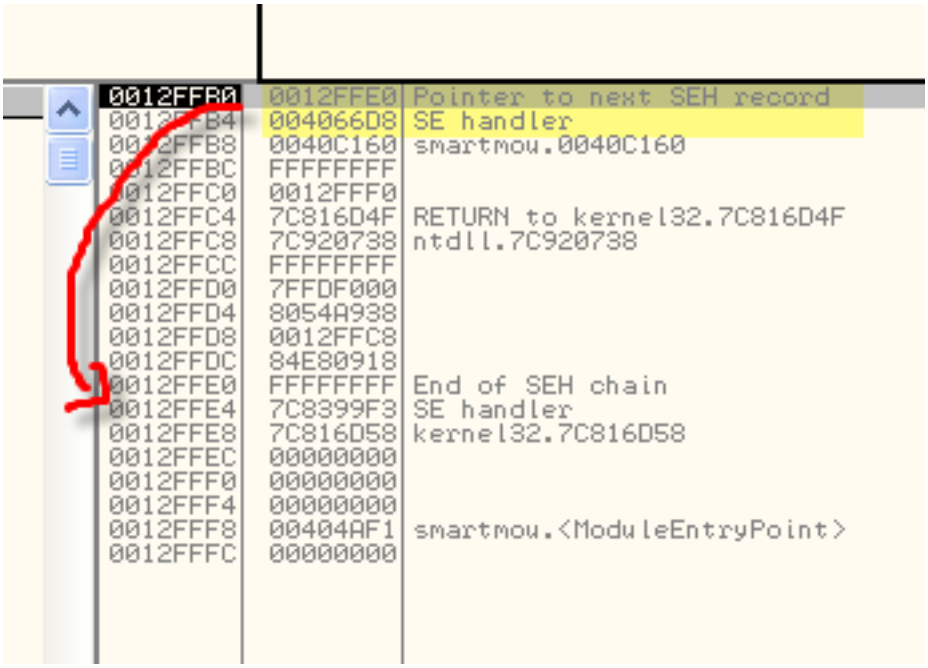




После выполнения строки:

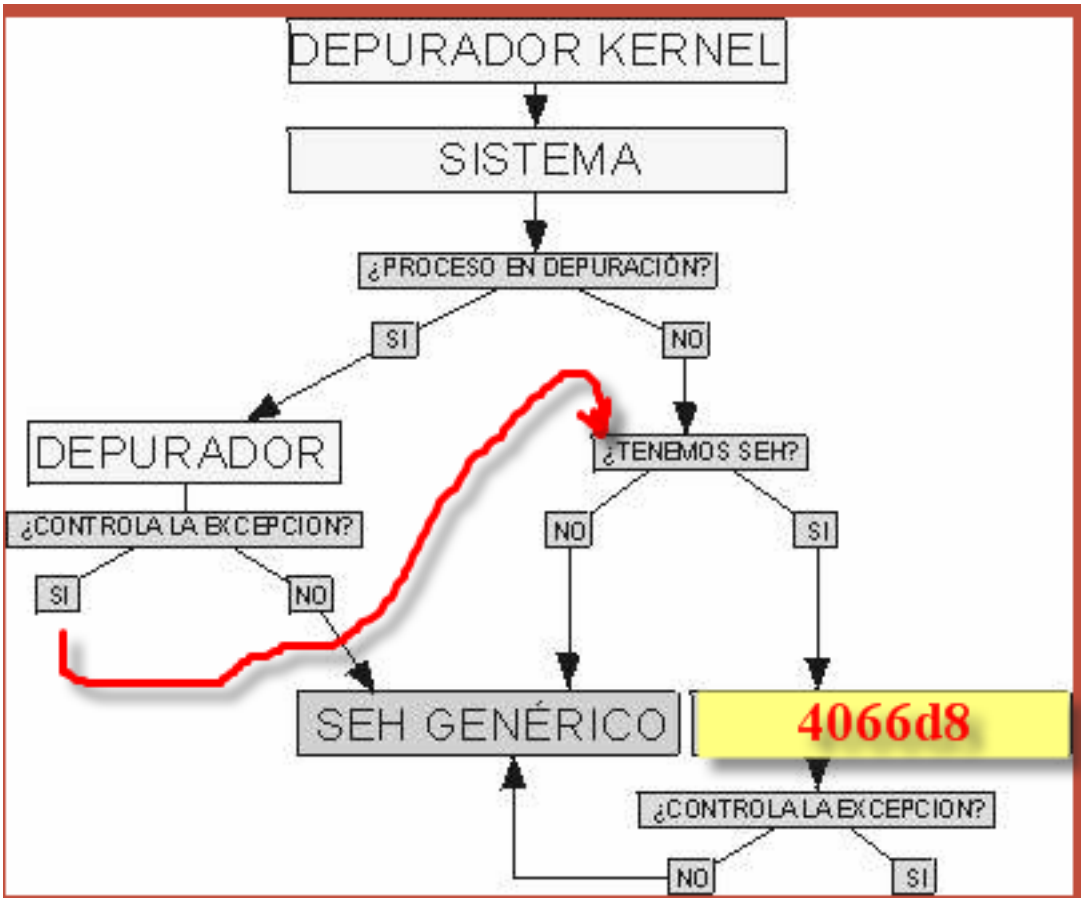


Наш обработчик установлен по адресу 12ffb0, как всегда видим, что fs:[0] указывает на действующий в настоящее время обработчик.



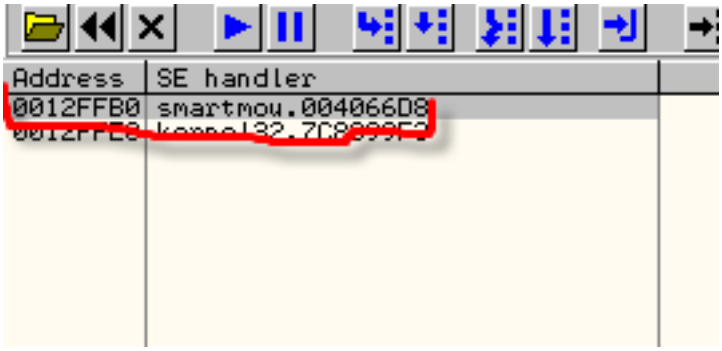
OllyDbg говорит нам, что это SEH-обработчик, первое значение указывает на старый, а второй – это адрес, по которому будет передано управление, когда возникнет исключение.

То есть, вот что происходит, когда возникает исключение:



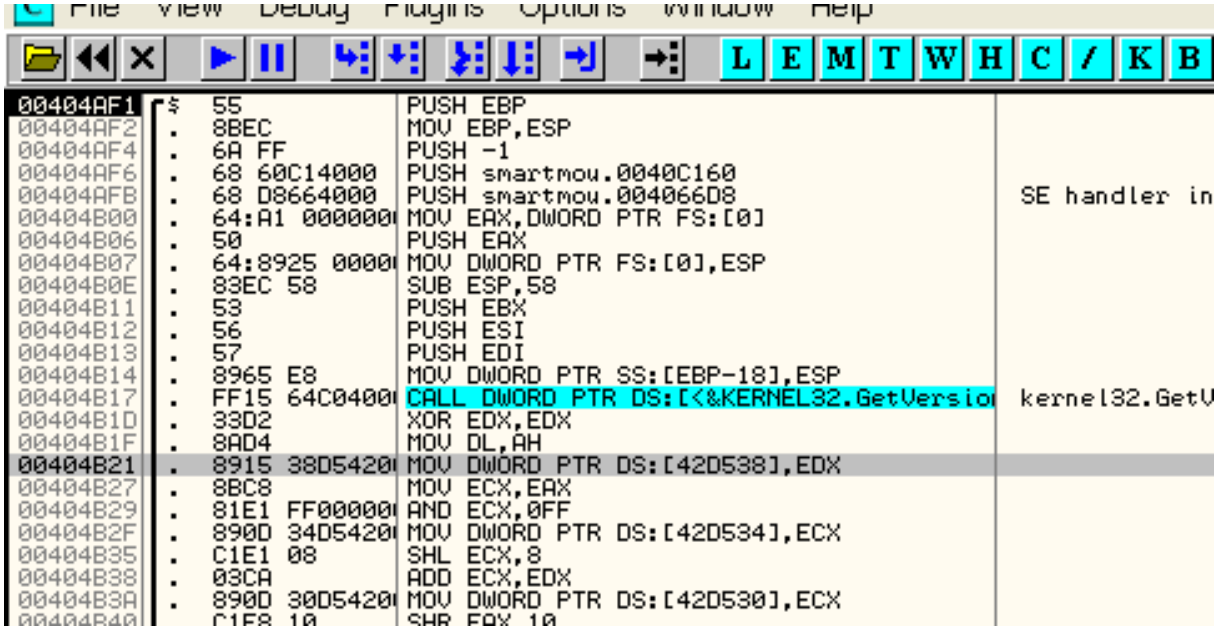
Система передаёт управление отладчика, который останавливает выполнение программы или нет (в зависимости от того, включена ли соответствующая данному исключению опция), а затем возвращает управление программе – сразу или ожидая нажатия SHIFT+F9, и как показывает рисунок, теперь, когда установлен SEH-обработчик, выполнение продолжается по адресу 4066d8.

Если посмотрим SEH CHAIN, то теперь увидим:

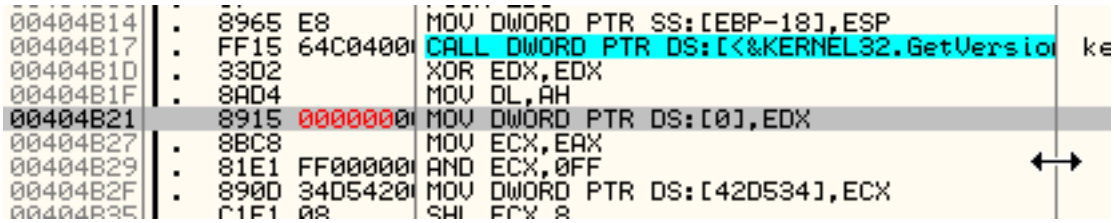


Видим, что здесь присутствуют как активный обработчик, установленный нами, так и системный, который в данный момент не функционирует.

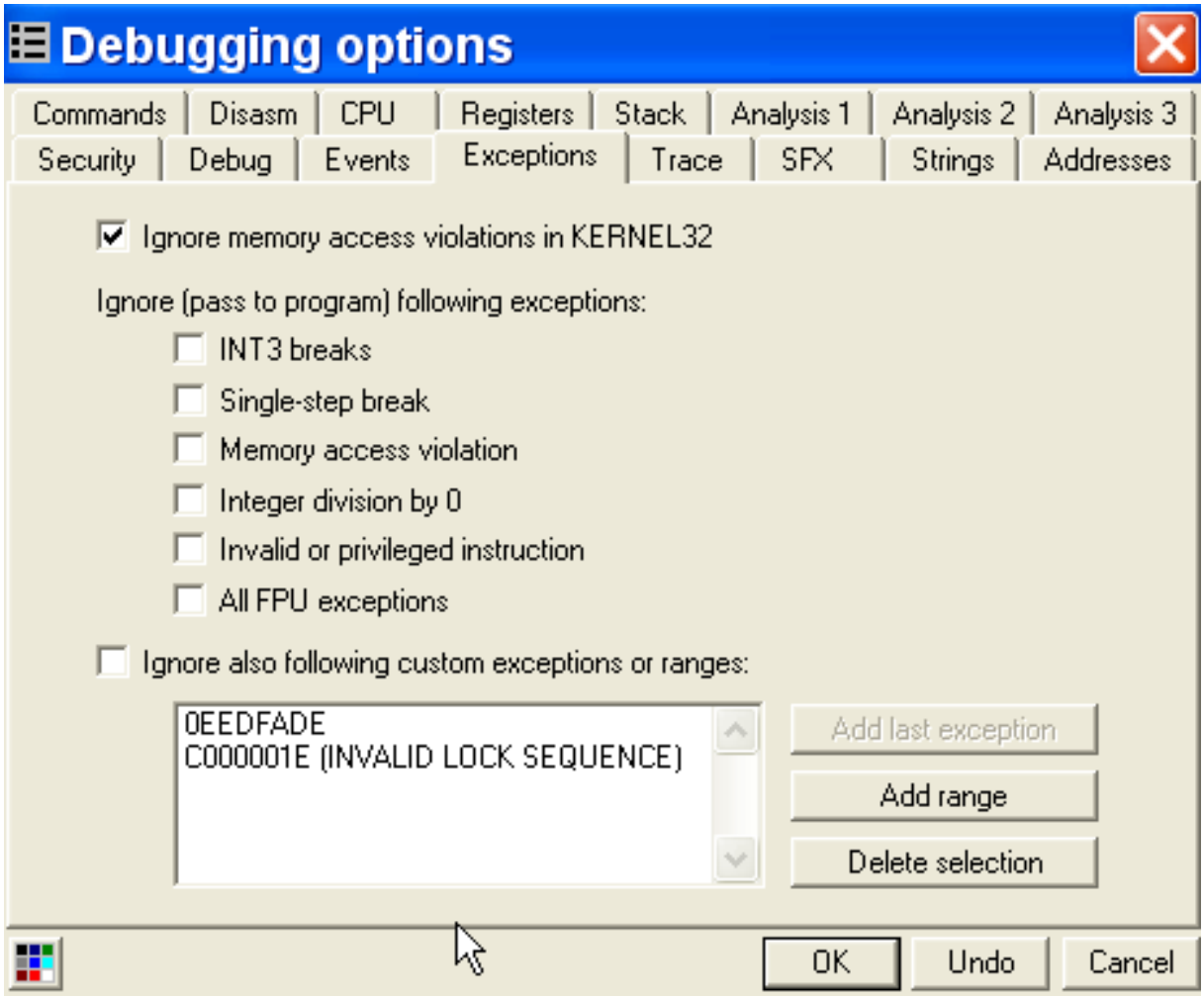
Вызовем исключение в программе.



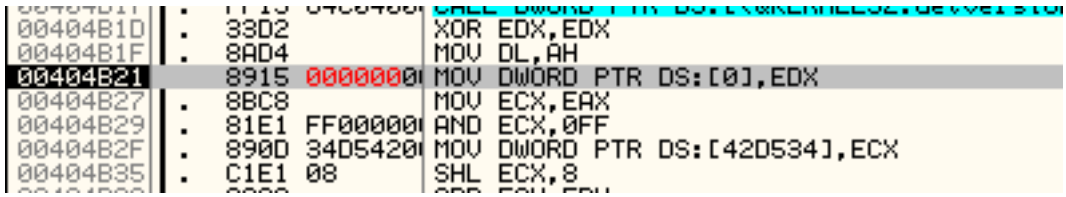
Изменим данную строку на:



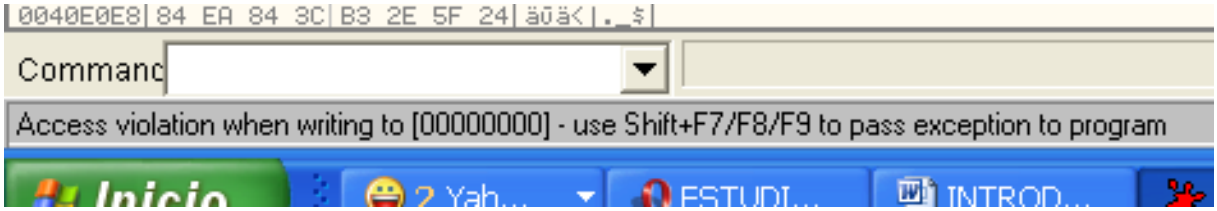
Это вызовет ошибку, так как не можем писать по адресу 0. Выключим все опции в DEBUGGING OPTION-EXCEPTIONS, кроме первой.



Делаем RUN.

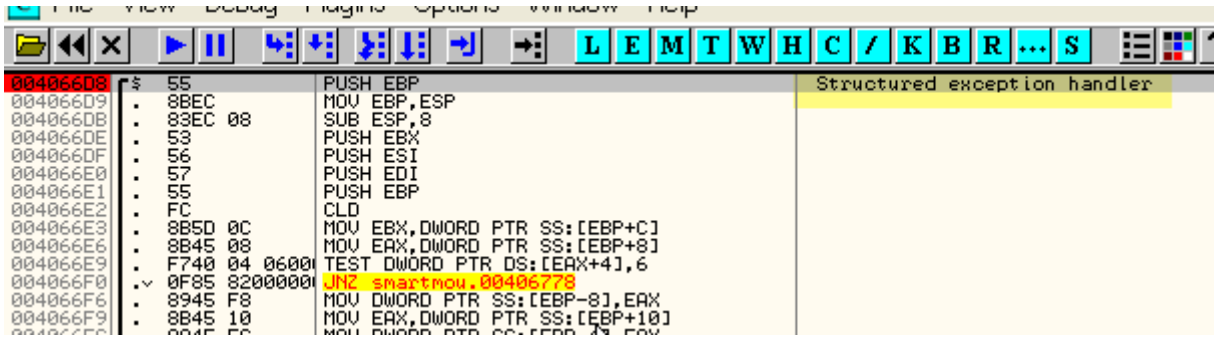


Показывается ошибка и выполнение программы прерывается.

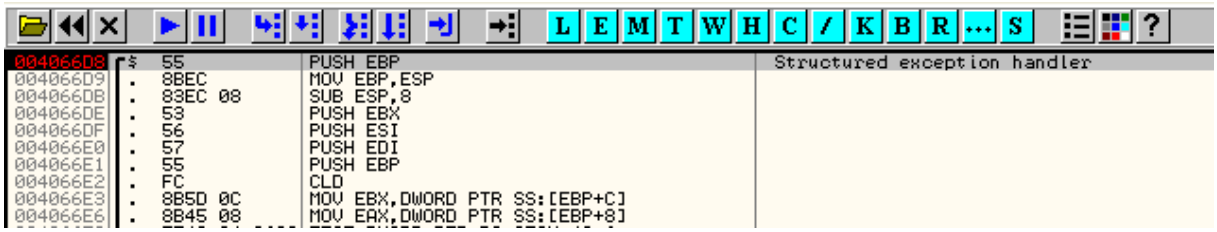


Теперь программа должна продожить выполнение с SHE-обработчика, пытаясь восстановиться после ошибки.

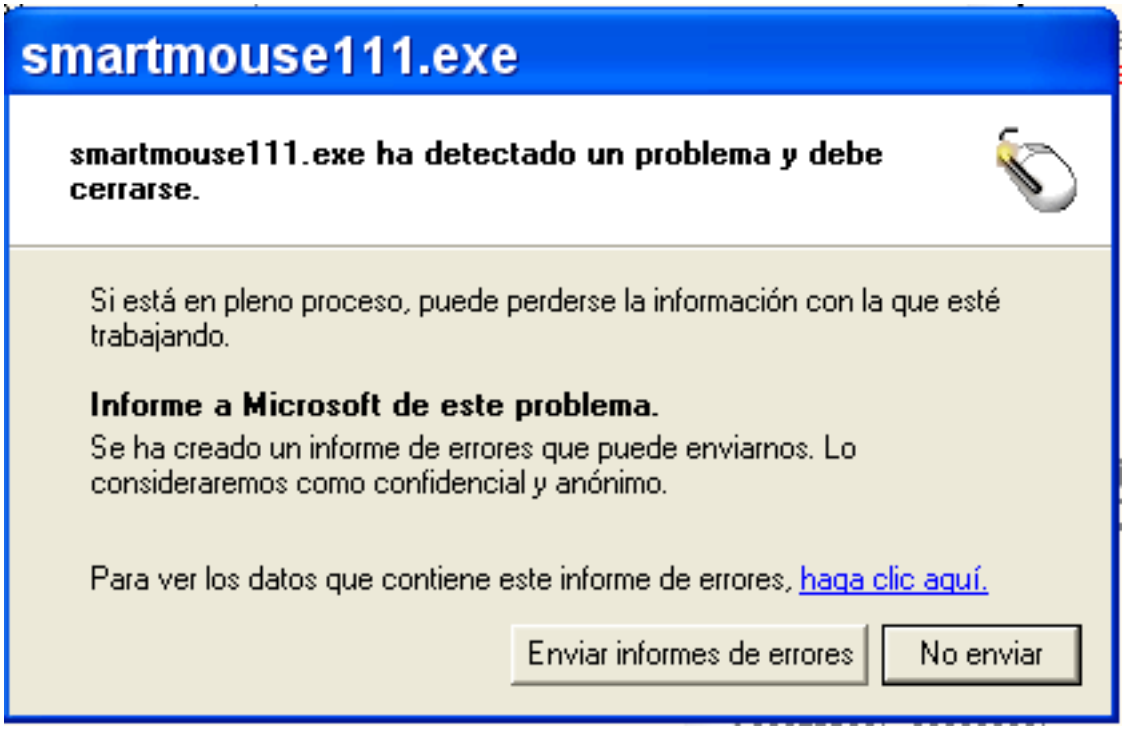
Обработчик установлен по адресу 4066d8. Идём туда и ставим BP.



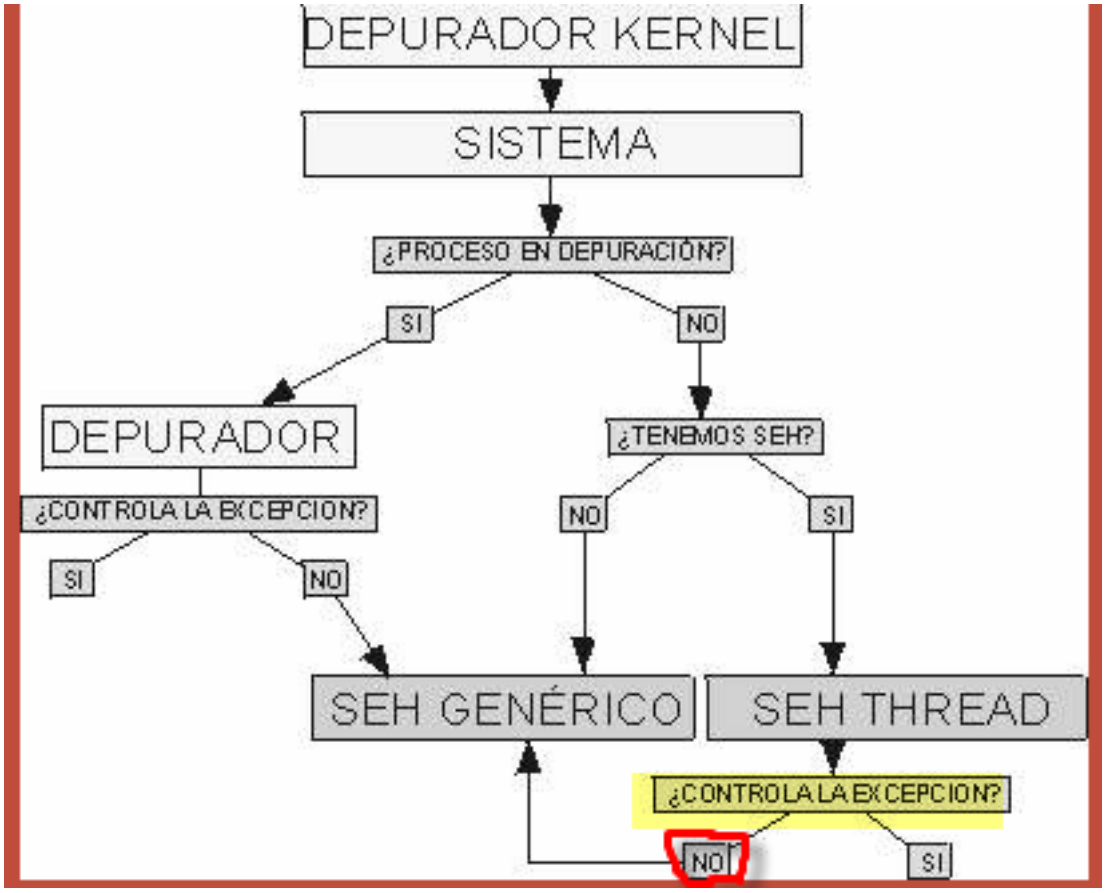
Как видим, OllyDbg всегда нам всё рассказывает, хе-хе, нажимаем SHIFT+F9, чтобы продолжить выполнение программы.



Как видим, останавливаемся на обработчике. Узнаем, что он делает, и может ли он оправиться от такой серьёзной ошибки как изменение кода программы.



Видим, что не можем продолжить выполнение и переходим на системный обработчик.

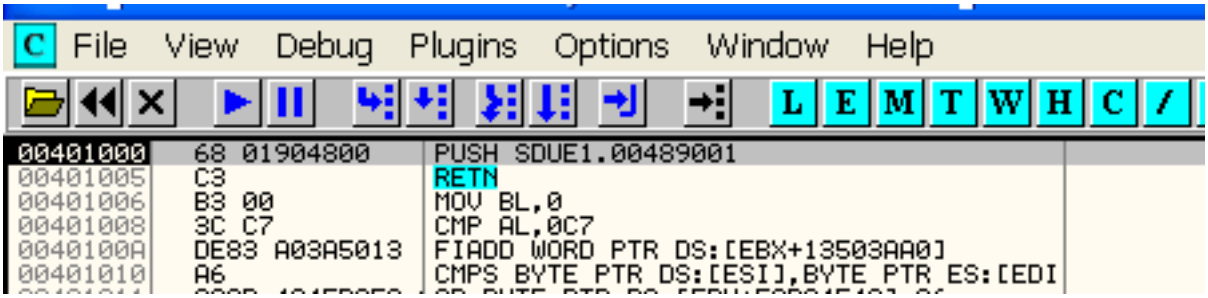


Это произошло, потому что ошибка очень серьёзная, которую программа преодолеть не смогла и передала управление системному обработчику, показавшему зловещее окошко с сообщением и закрывшему приложение.

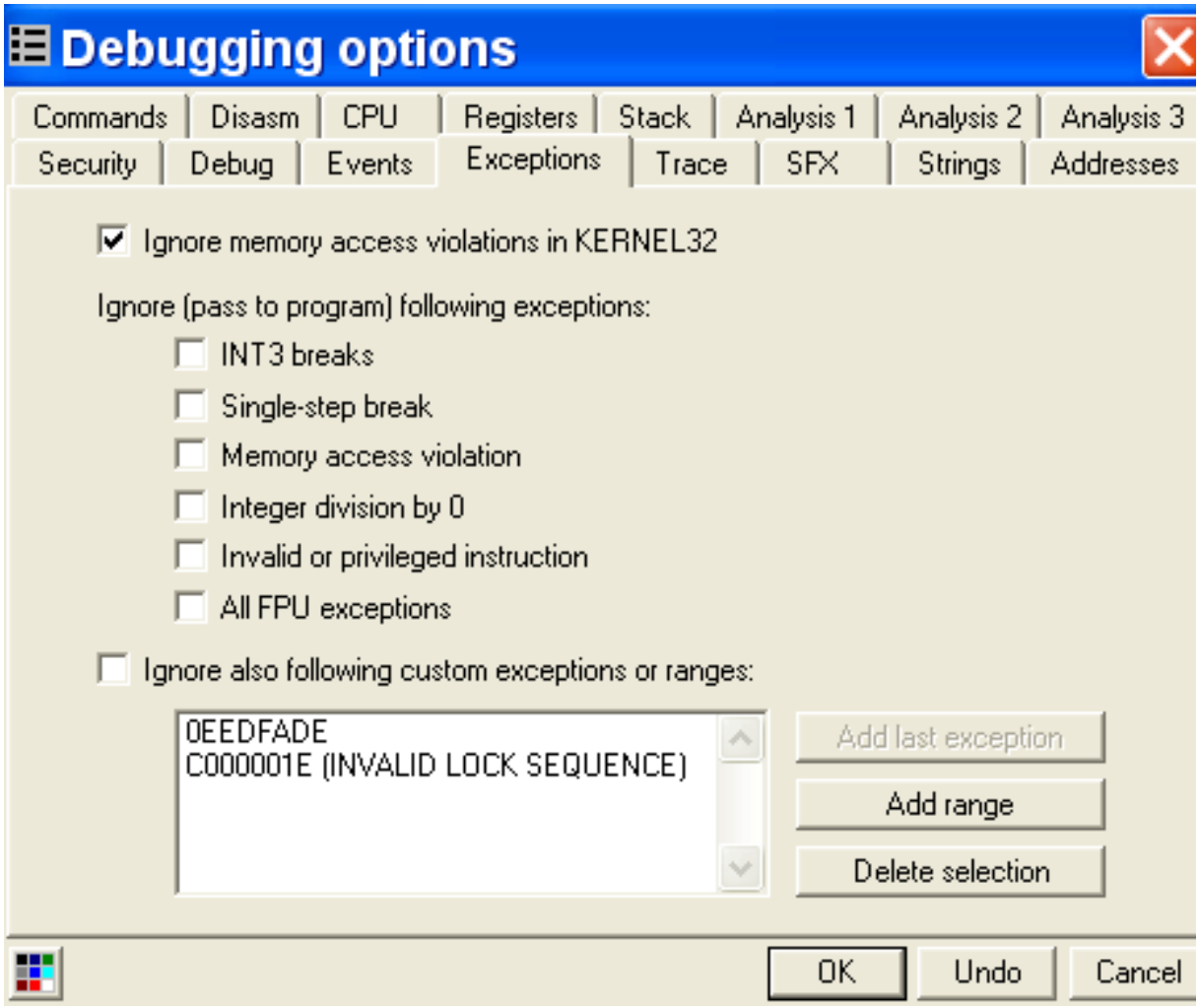
Очевидно, что обработчик исключений, установленный программой, ожидает другой тип исключений, а не тот, что вызывается из-за изменения кода, но зато увидели, что происходит, если происходит неконтролируемое исключение.

Чтобы рассмотреть другой случай, когда исключение успешно обрабатывается, посмотрим крэкми SDUE, которое прилагается к данной главе.

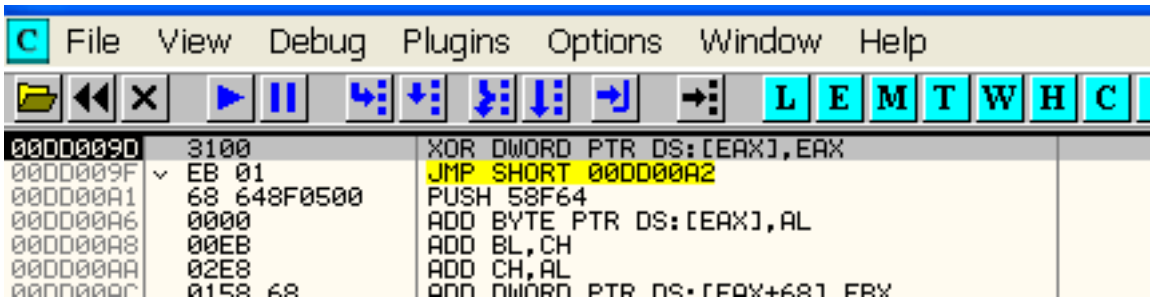
Откроем его в OllyDbg и сообщение отладчика о том, что приложение может самомодифицироваться и т.д. и т.п., говорит нам о том, что крэкми запакован.



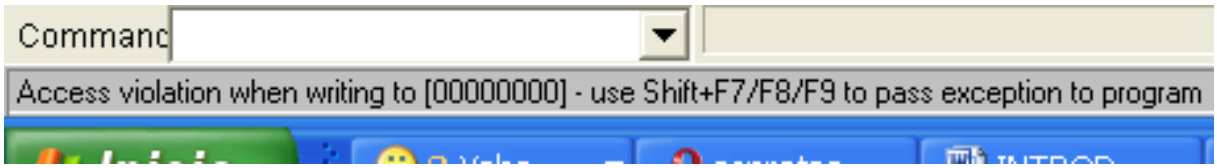
Снимем все галочки кроме первой в DEBUGGING OPTIONS-EXCEPTIONS.



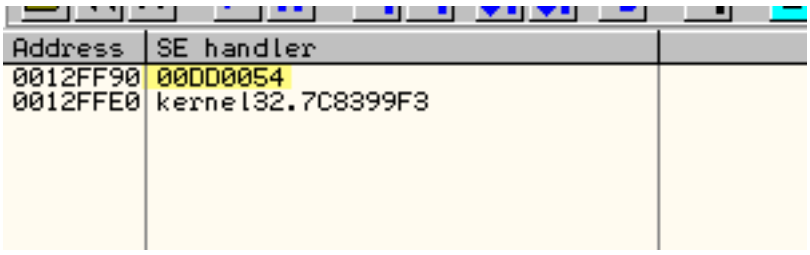
И делаем RUN.



Останавливаемся на исключении.

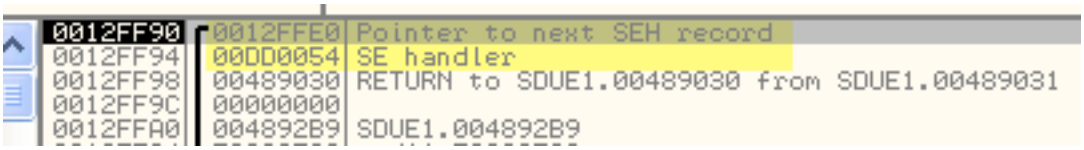


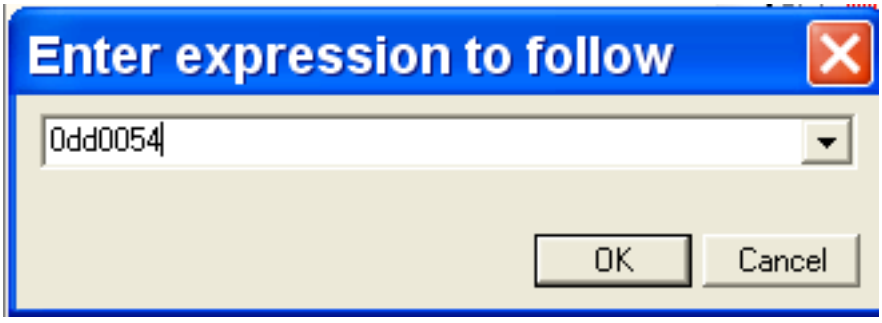
Где находится обработчик:



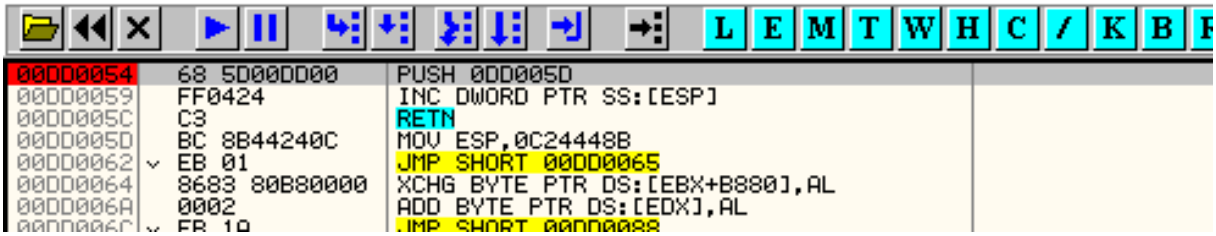
На вашей машине этот адрес может отличаться, так как секция создаётся во время выполнения программы.

Устанавливаем сюда BP.

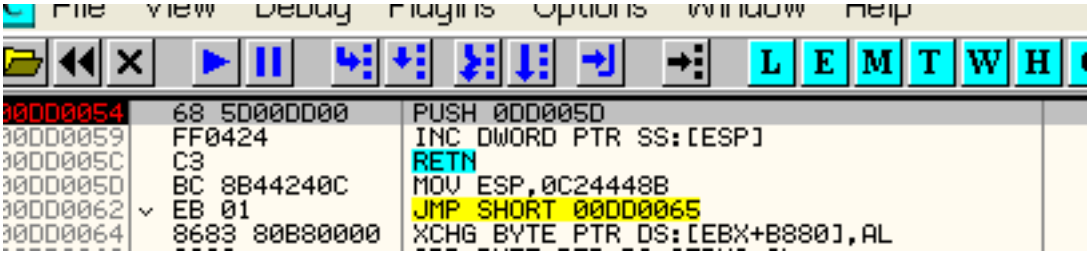




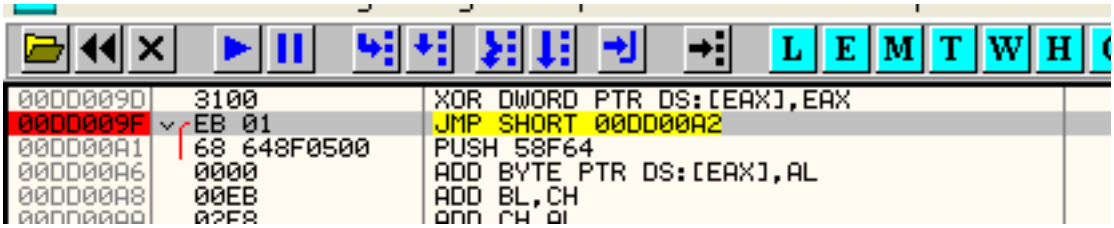
Устанавливаем сюда UN BPX.



Нажимаем SHIFT+F9.



Происходит остановка на обработчике, который, если нет никаких странных изменений, должен вернуться на строку следующую за той, что вызвала исключение, после чего выполнение программы продолжится.



Установим UN BP на следующую строку после той, где генерируется исключение, и нажмём RUN.

Видим, что ошибка была обработана и программа продолжает выполняться без каких-либо сообщений об ошибках.

Как видим, это простая процедура, и в обычном случае возвращение происходит на строку следующей за той, которая вызвала исключение, хотя адрес возврата можно изменить в самом обработчике. В этом случае можно установить BPM ON ACCESS на секцию, где возникает исключение, и когда оттрассируем обработчик строка за строкой с помощью F9 и вернёмся в программу, то сработает BPM ON ACCESS и мы точно узнаем, где находимся.

Очевидно, что если хотим остановиться в том момент, когда программа устанавливает обработчик исключений, то лучше создать HARDWARE BPX ON WRITE в FS:[0]. Во многих случаев DLL также устанавливают обработчики, затем они убираются и при возвращении в программу остаются те, которые были раньше, и вышеуказанным образом можно контролировать их установку и снятие во время выполнения программы.

Установить обработчик исключений можно и другим способом – с помощью API-функции SetUnhandledExceptionFilter, которой передаётся адрес обработчика.

Ладно, думаю, что мы рассмотрели полезную информацию, которая позволит нам продвинуться дальше в крэкинг. В дальнейшем мы рассмотрим подробные примеры по этой теме.

Файлы к статье (находится в архиве `wasm_files.zip`: `files/0481/ollydbg25.zip`)

Введение в крэкинг с нуля, используя OllyDbg - Глава 26

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-04-09

ВЗЛОМ ПРИЛОЖЕНИЙ, НАПИСАННЫХ НА VISUAL BASIC, С ПОМОЩЬЮ OLLYDBG

В этой и последующих главах мы будем учиться крэкать программы, написанные на Visual Basic'е, с помощью OllyDbg, на что вы можете мне сказать, что для этого существует специальный инструменты, но так как этот курс посвящён именно отладчику OllyDbg, то будем стараться использовать как можно меньше других инструментов, и только, если мы никак не можем добиться того, что нужно, с помощью OllyDbg, тогда и только тогда мы будем прибегать к ним.

Прежде всего, я приготовил для вас небольшой подарок (находится в архиве `wasm_files.zip`: `files/recovered/482/olly_parcheado_para_vb.rar`).

Он называется "OLLYDBG PARCHEADO PARA BUSCAR OEPs", который, как правило, требуется только для распаковки, но на самом деле, он также очень полезен при анализе программ, написанных на VISUAL BASIC и скомпилированных в «родной» код (но не для тех, что используют P-CODE).

Между обоими случаями есть значительное различие, и будет несколько глав, посвящённая P-CODE, но кратко можно сказать, что при «родном» коде, код, как и полагается, выполняется инструкциями в секции «code» программы, а при P-CODE запускается специальная DLL-библиотека, которая читает из секции «code» байты, содержащие информацию о том, какие команды надо исполнять.

То есть, если программа, написанная на Visual Basic'е, никогда не выполняет строки из секции кода, то в этом случае она использует P-CODE, и как с ним работать, мы изучим гораздо позже.

Чем отличается пропатченный OllyDbg? Тем, что при установке BPM ON ACCESS остановка происходит не по событиям ON READ и ON EXECUTE, а только по ON EXECUTE, и это очень полезно для нахождения OEP'ов и для Visual Basic, так как программа не будет останавливаться тысячу раз в библиотеке VB, а будет каждый раз возвращаться обратно в код программы, и мы не потонем в DLL-библиотеке Visual Basic'а.

Также, чтобы работать с Visual Basic, необходимо знать API-функции VB, которые отличаются от обычных, и их нельзя встретить в стандартных мануалах (как, например, WINAPIS32).

В остальном, есть множество туториалов, специально посвящённых VB и содержащие множество приёмов и способов, которые мы не будем использовать, но если вы захотите ознакомиться с ними, то обратитесь к «Новому курсу…» от Crackslatinos и найдите туториалы COCO и других участников, что может весьма вам помочь в данной области. Здесь мы не будем использовать данные методики и будем крэкать только с помощью OllyDbg.

Пропатченный OllyDbg помещаем в ту же папку, где лежит и непропатченный. Естественно, использовать первый имеет смысл тогда, когда взламываем VB-программы, или когда ищем OEP'ы, или в любом другом случае, когда нужно чтобы BPM ON ACCESS работал как BPM ON EXECUTION.

Конечно, если используем этот OllyDbg, то мы не можем использовать BPM ON ACCESS так, чтобы он останавливался по событиям ON READ и ON EXECUTE, в этом случае иногда можно применить HARDWARE BPX ON.

Основная проблема с API-функциями Visual Basic'a – это то, что Microsoft не распространяет информацию о них. Если нужно что-то узнать о какой-то функции, то надо делать поиск в Google, и если повезёт, то найдётся какая-нибудь страница, где будет объяснено, что она делает и для чего служат. Поэтому перед тем, как перейти к собственно крэкингу, в этом главе приводятся собранные мной материалы о некоторых наиболее важных API-функциях и содержащих некоторую общую информацию о программах на Visual Basic'e.

ЗНАЧИМЫЕ СОСТАВЛЯЮЩИЕ ИМЕНИ API-ФУНКЦИИ VB

В именах функций, которые мы повстречаем, могут встречаться следующие аббревиатуры:

```
bool    Boolean
str     String
i2      Integer (2 bytes)
ui2     Unsigned integer (2 bytes unsigned integer)
i4      Long (4 bytes integer)
r4      Single (4 bytes real)
r8      Double (8 bytes real)
cy      Currency
var     Variant or variable
fp      Floating point
cmp     compare
comp    compare
```

Ниже идёт таблица с определениями типов переменных. Имея представление о них, мы уже многое можем понять о самой функции.

Например:

__vbaI2Str говорит о том, что строка конвертируется в число.

Вот ещё некоторые определения:

1) Примеры API-функций, конвертирующих данные:

1. __vbaI2Str конвертирует String в Integer
2. __vbaI4Str конвертирует String в Long
3. __vbar4Str конвертирует String в Single
4. __vbar8Str конвертирует String в Double
5. VarCyFromStr конвертирует String в Currency
6. VarBstrFromI2 конвертирует Integer в String

Видим, что аббревиатуры дают нам представление о типах. Вот ещё несколько API-функций:

2) Перемещение данных

1. __vbaStrCopy – копирует String в память
2. __vbaVarCopy - копирует Variable в память
3. __vbaVarMove – копирует Variable в память

3) Математические

1. __vbavaradd – сумма переменных типа Variable
2. __vbavarsub – сумма переменных типа Variable
3. __vbavarmul – сумма переменных типа Variable
4. __vbavaridiv – поделить две переменные, результат возвращается в виде Integer
5. __vbavarxor – функция XOR

4) Вспомогательные:

1. __vbavarfornext - цикл
2. __vbafreestr – освободить неиспользующуюся переменную типа String.
3. __vbafreeobj – освободить неиспользущийся объект.

4. __vbastrvarval – взять значение из места, заданного строкой.
5. multibytetowidechar – сконвертировать String в widechar-формат строки
.
6. rtcMsgBox – отображает окошко с сообщение – похоже на API-функцию Windows messagebox/a/exa.
7. __vbavarcat – соединить две переменные.
8. __vbafreevar – освободить неиспользующуюся переменную типа Variable.
9. __vbaobjset Активировать объект.
10. __vbaLenBstr – получить размер строки.
11. rtcInputBox – отобразить окошко ввода, похоже на API-функции Windows API getwindowtext/a, GetDlgItemtext/a.
12. __vbaNew – показать диалоговое окно – похоже на API-функцию Windows Dialogbox.
13. __vbaNew2 - показать диалоговое окно – похоже на API-функцию Windows Dialogboxparam/a.
14. rtcTrimBstr – подрезать String.
15. __vbaEnd – завершение программы.
16. __vbaLenVar – получить размер переменной.
17. rtcMidCharVar –получить определённый символ строки.
18. _rtcDir – проверка, существует ли файл.
19. __vbaFileOpen – открыть файл.

5) Сравнения:

1. __vbastrcomp – сравнение двух строк - как lstrcmp.
2. __vbastrcmp – сравнение двух строк – как lstrcmp.
3. __vbavartsteq – проверка на равенство двух переменных.
4. __vbaFpCmpCy – сравнение переменной с плавающей точкой с переменной типа Currency.
5. __vbavartstNe – сравнение двух переменных на неравенство.

Эти описания помогают нам получить представление о том, что программа делает в данный момент. Имя функции не даёт нам много информации, возможно, она неполная, но тем не менее она помогает нам понимать, что происходит.

И хотя пока что мы не рассматриваем крэкинг в SMART CHECK’e, вот адрес со списком внешних функций VB, с которым может консультироваться крэкер.

http://www.w3schools.com/vbscript/vbscript_ref_functions.asp

Например, в Smartcheck’e нам показывается функции ASC или MID, которые не соответствуют тому, что мы видим в OllyDbg или функциям в языке программирования Visual Basic, но на этой странице хорошо объяснена каждая из возможных команд и их параметры, поэтому крэкинг в SMART CHECK’e существенно легче.

Например, в SMART CHECK’e видим команду LEN, которая соответствует API-функции __vbaLenVar.

На странице смотрим определение LEN.

The Len Function

Back

The Len function returns the number of characters in a string.

Syntax

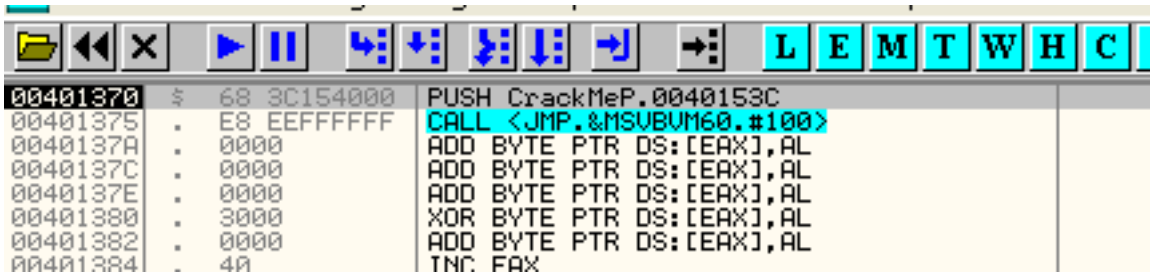
```
Len(string|varname)
```

Parameter	Description
string	A string expression
varname	A variable name

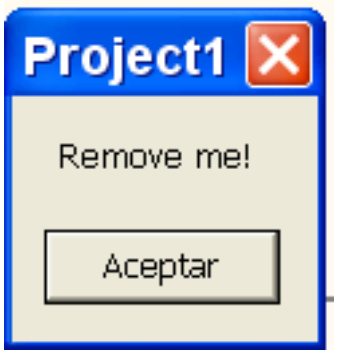
Это поможет тем, у кого есть проблемы с использованием SMART CHECK'a. Хотя в этом туториале мы не будем его рассматривать, думаю, что ни одна из статей, посвящённых этому инструменту, не даёт точный список используемых им выражений, поэтому я и упомянул его.

Ладно, теперь, когда мы всё это знаем, перейдём к крэканию CrackMePls.

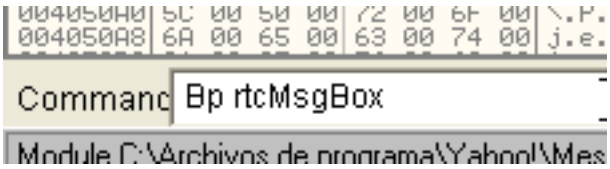
Открываем его в пропатченном вышеуказанным образом OllyDbg.



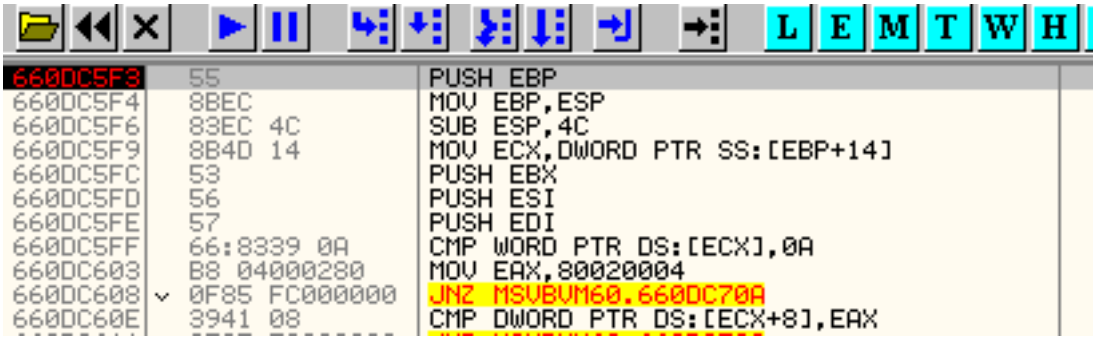
Этот крэкми очень простой: есть наг (nag), который нужно убрать, и серийный номер, который нужно найти. Смотрим и делаем RUN.



Закрываем наг-окошко, которое похоже на MessageBoxA, хотя в Visual Basic'е этому соответствует API-функция rtcMsgBox. Установим на неё BP.



И рестартуем, чтобы посмотреть, произойдёт ли остановка до того, как выйдем из наг-окошка.

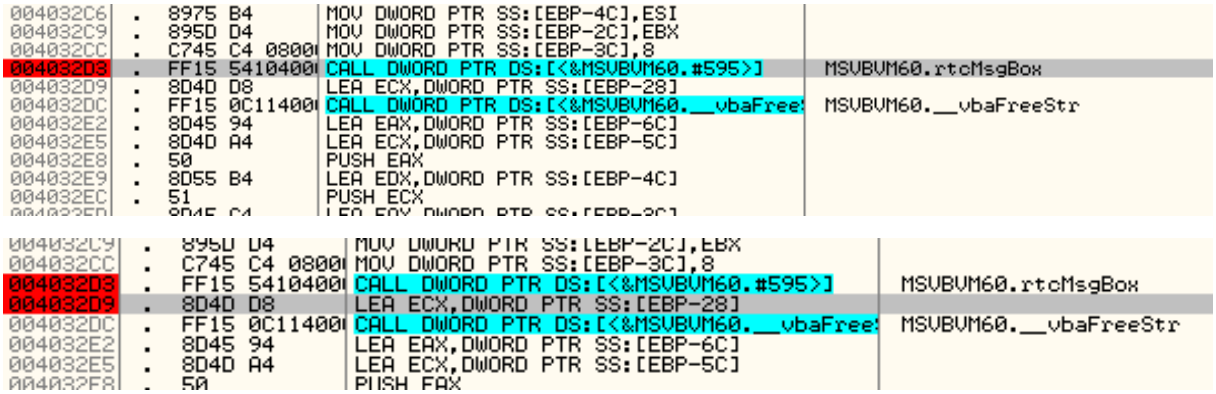


Останавливаемся здесь, нет никакой подсказки относительно параметров или чего бы то ни было ещё, но это неважно.

Видим, откуда вызывается эта API-функция, первое значение стека – это адрес, по которому будет осуществлён возврат управления.

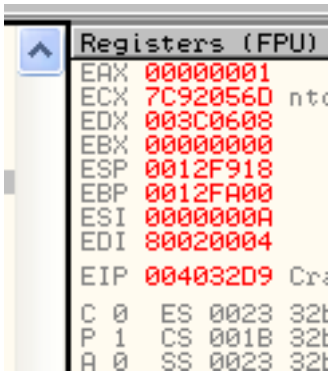
Смотрим место, откуда произошёл вызов.

Вот вызов API-функции, а ниже, соответственно, находится точка возврата, адрес которой 4032d9.



Устанавливаем BP на точку возврата из API-функции и нажимаем RUN.

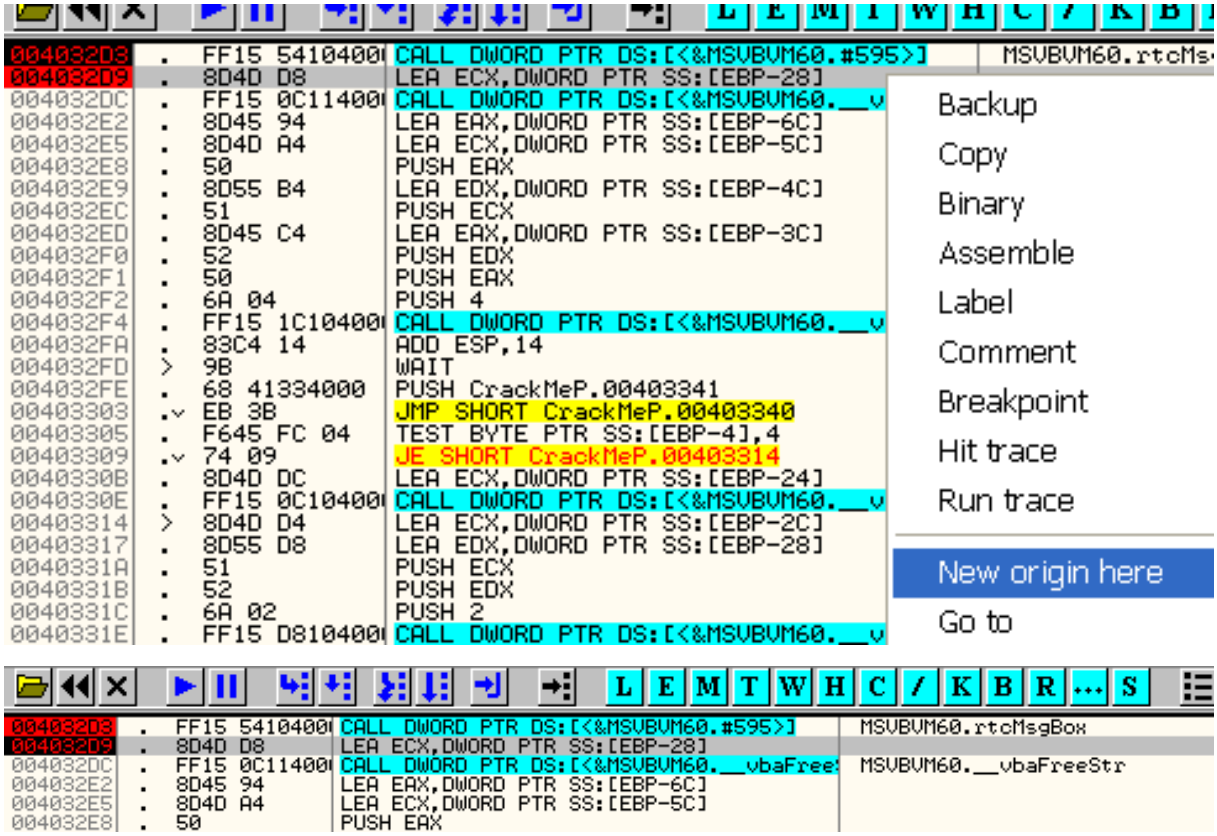
Нажимаем и останавливаемся на возврате.



EAX=1 означает, что API-функция выполнилась успешно. Теперь надо рестартовать крэкми, чтобы затем каким-нибудь образом забить наг-окно NOP’ами.

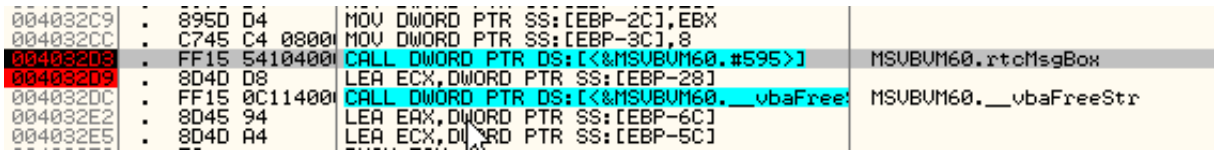
Рестартуем крэкми и останавливаемся на вызове.

Если кликнем правой кнопкой мыши на строке, куда происходит возврат, и выберем NEW ORIGIN HERE, то выполнение продолжится отсюда без отображения окошка. Это симуляция забивки вызова NOP’ами.

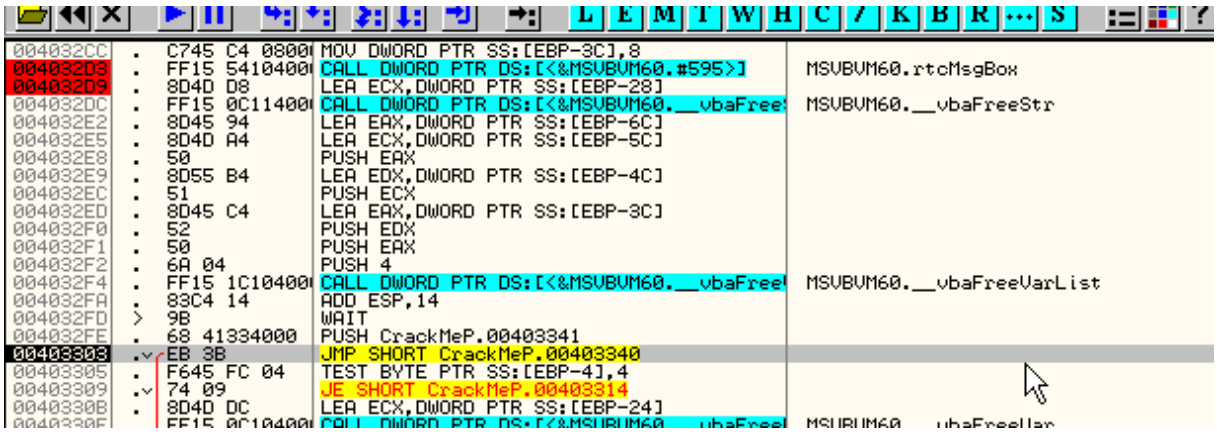


Здесь делаем RUN и смотрим, что происходит.

Очевидно, что решение забить NOP’ами вызов API-функции не работает. Попробуем другой способ. Рестартуем крэкми.



Доходим до 4032d9 и продолжаем трассировать с помощью F8.



Доходим до JUMP и продолжаем трассировать.

JMP, перед которым находится PUSH 403341, переходит на RET, то есть тот используется для перехода на 403341, о чём нам также говорит и OllyDbg. Этот приём используется для того, чтобы не дать нам заметить адрес возврата в стеке. Продолжаем трассировать.

Доходим до RETN 8.

Здесь, по адресу 402f95 заканчивается выход из CALL'a, но адрес возврата не постаётся в стеке благодаря трюку, используемого для скрытия CALL'a и адреса возврата.

Устанавливаем BP на этот CALL и рестартуем крэкми.

```
00402F8B  .  8B0E          MOV ECX,DWORD PTR DS:[ESI]
00402F8D  .  8D95 74FFFFFF LEA EDX,DWORD PTR SS:[EBP-8C]
00402F93  .  52            PUSH EDX
00402F94  .  56            PUSH ESI
00402F95  .  FF91 0C070000 CALL DWORD PTR DS:[ECX+70C]      CrackMeP.00401E74
00402F9B  .  8D8D 74FFFFFF LEA ECX,DWORD PTR SS:[EBP-8C]
00402FA1  .  FF15 0C10400( CALL DWORD PTR DS:[<&MSVBVM60.__vbaFree MSVBVM60.__vbaFreeVar
00402FA7  >  E8 D0F2FFFF   CALL CrackMeP.0040227C
00402FAC  .  8985 2CFFFFFF MOV DWORD PTR SS:[EBP-D4],EAX
```

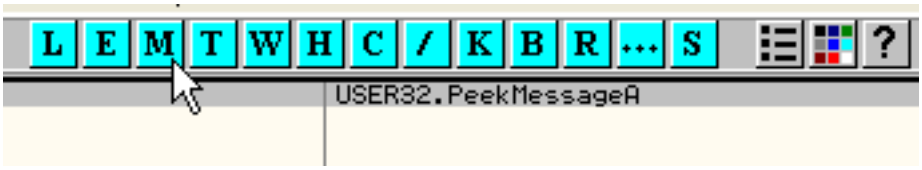
Забиваем CALL нопами, чтобы посмотреть, запустится ли он без наг-окошка.

Делаем RUN.

Пока-пока, наг, хе-хе, позже мы рассмотрим другие примеры того, как убирать наги, но этот пример был достаточно прост, не считая маленького рассмотренного выше трюка. Вопрос состоял в том, чтобы идти от появления нага до возвратов из CALL’ов, чтобы увидеть, можно ли забить NOP'ами какой-то из них для устранения назойливого окошка.

Теперь крэкми работает и предлагает нам ввести правильный серийный номер.

Тут мы увидим возможность, даваемую пропатченным OllyDbg. Хотим поставить BPM ON ACCESS (выполнение) на секцию кода, идём в M.



Вспоминаем, что в этом OllyDbg это BPM ON EXECUTION, при котором не происходит остановки при чтении и записи в секции, где выполняется DLL-библиотека Visual Basic'a, а быстро происходит возврат в программу, благодаря чему мы избегаем тысячи лишний остановов.

Нажимаем OK.

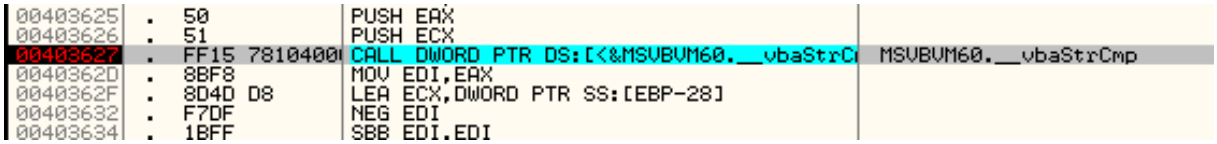
```
00401E45  .  816C24 04 3F( SUB DWORD PTR SS:[ESP+4],3F
00401E4D  .v E9 BE0D0000   JMP CrackMeP.00402C10
00401E52  .  816C24 04 43( SUB DWORD PTR SS:[ESP+4],43
00401E5A  .v E9 410E0000   JMP CrackMeP.00402CA0
00401E5F  .  816C24 04 3B( SUB DWORD PTR SS:[ESP+4],3B
00401E67  .v E9 A40E0000   JMP CrackMeP.00402D10
00401E6C  .  816C24 04 FF( SUB DWORD PTR SS:[ESP+4],0FFFF
00401E74  .v E9 F7110000   JMP CrackMeP.00403070
00401E79  .  816C24 04 FF( SUB DWORD PTR SS:[ESP+4],0FFFF
00401E81  .v E9 FA140000   JMP CrackMeP.00403380
00401E86  .  816C24 04 FF( SUB DWORD PTR SS:[ESP+4],0FFFF
00401E8E  .v E9 0D170000   JMP CrackMeP.004035A0
00401E93  .  816C24 04 37( SUB DWORD PTR SS:[ESP+4],37
00401E9B  .v E9 801B0000   JMP CrackMeP.00403A20
00401EA0     00            DB 00
00401EA1     00            DB 00
```

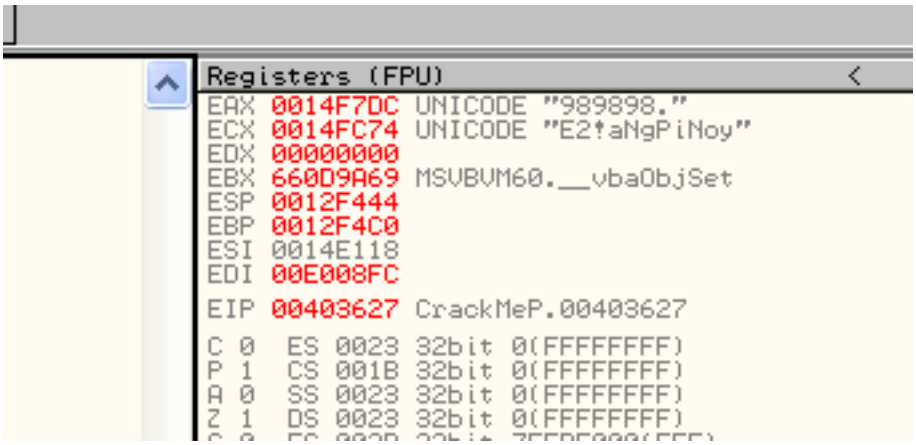
Останавливаемся здесь и немного потрассируем.

Доходим до этого вызова. Быстро проходим через вызовы API-функций и заходим во внутренние функции программы.

Тогда заходим с помощью F7.

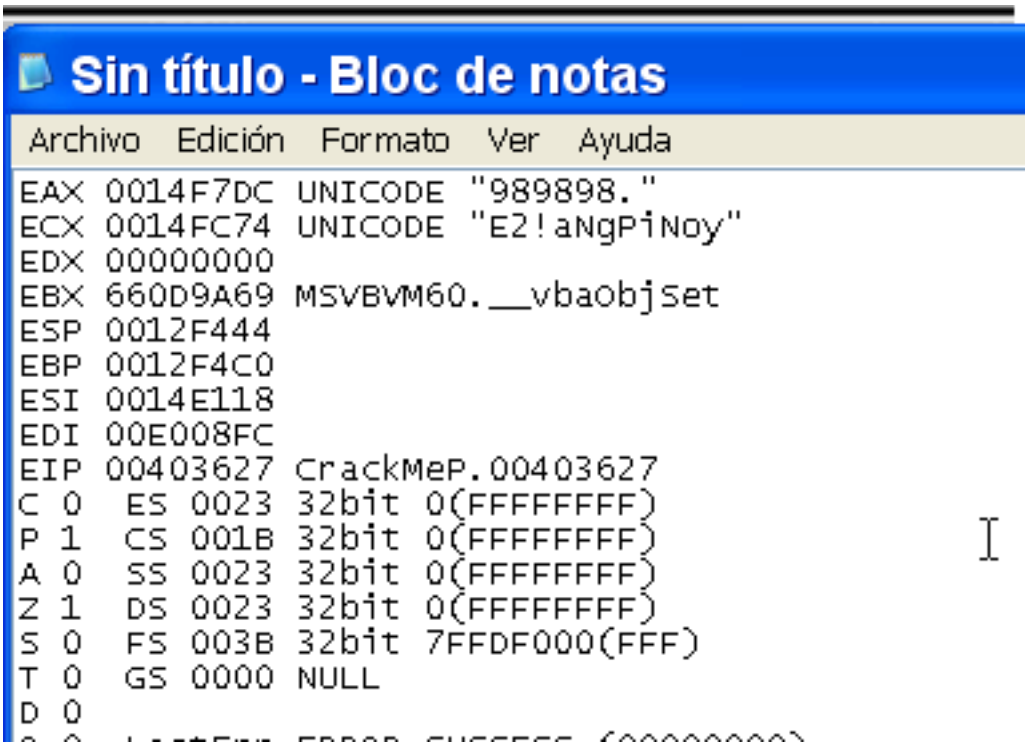
Доходим до места, возможно, представляющего интерес, внизу видим vbaStrCmp, что может являться возможным сравнением строк. Устанавливаем сюда BP и снимаем BPM до того, как сделаем RUN.





Видим, что происходит сравнение введённой строки «989898» с правильным серийным номером. Копируем его.

И помещаем содержимое буфера в блокнот..



Отсюда легко можем скопировать правильный серийный номер.

Пробуем поместить его в окошко крэкми.

Хорошо, получилось. Конечно, нужно сохранить изменения (мы же забили наг-окошко NOP'ами), чтобы теперь это крэкми выполнялось нормально.

В главе 27 продолжим углубление в крэкинг приложений на Visual Basic и будем использовать более сложные примеры.

Введение в крэкинг с нуля, используя OllyDbg - Глава 27

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-04-16

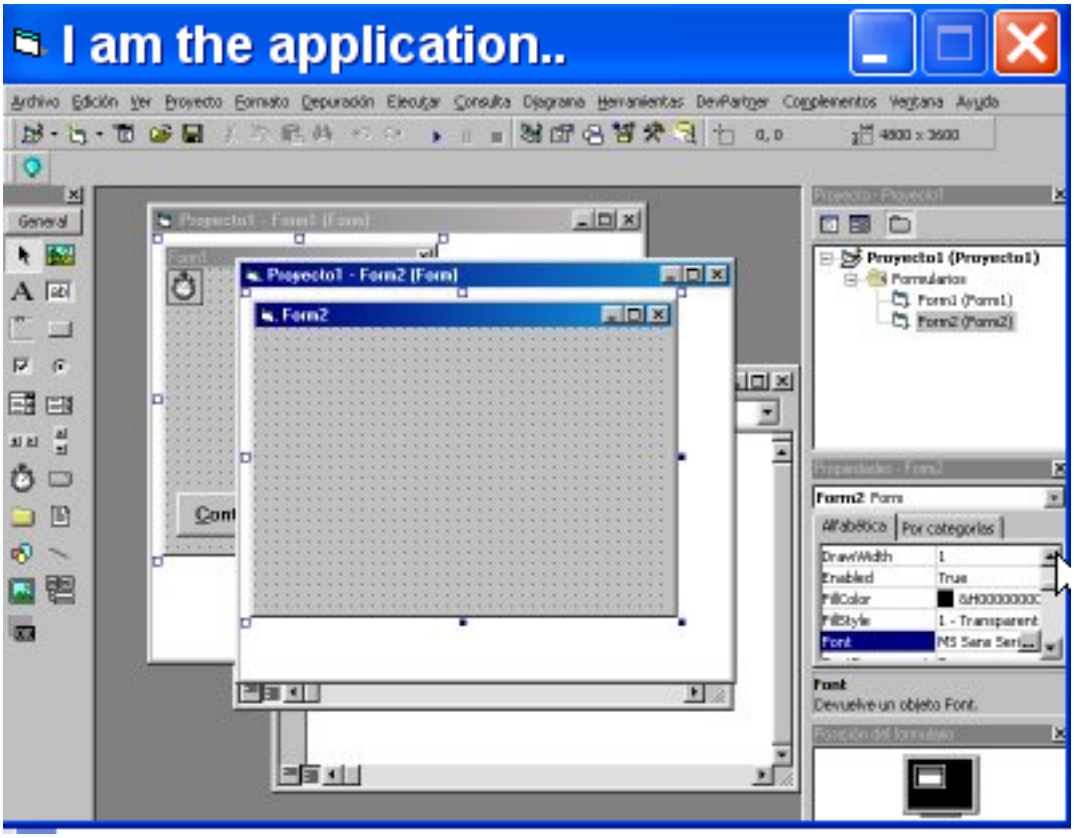
Продолжаем с Visual Basic'ом

Ок, продолжаем с Visual Basic’ом. Здесь у нас крэкми, в котором нужно избавиться от наг-окошка. Я это сделаю это тем способом, который подойдёт для данного конкретного случая (мне нравится размышлять и пробовать новое), но конечно, используя OllyDbg, специально заточенный для VB, тот же, что и в прошлой главе. Потом мы рассмотрим механический способ, позволяющий сэкономить время. Можно было бы сразу дать его, но я считаю, что очень важно самому попытаться это сделать, чтобы понять как его работает, поэтому сначала рассмотрим мой способ снятия нага, а потом – механический способ, потому что вы должны знать и уметь применять оба.

Killme – это крэкми, которое мы будем изучать. В нём нет крэкми, а есть только назойливое наг-окошко, появляющееся при запуске программы и которое нужно убрать.

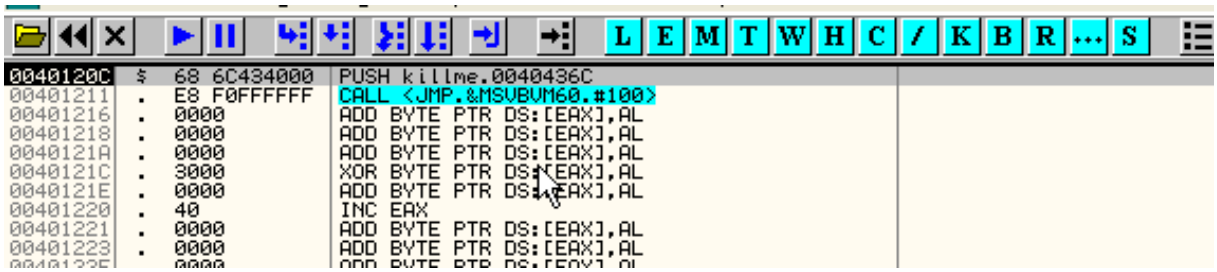
Если запустим крэкми в OllyDbg, то увидим наг-окошко.

Видим, что есть недоступное для нажатия окошко и таймер от пяти до нуля, и когда он становится равным нулю, то кнопку CONTINUE становится возможным нажать.



Появляется указанное приложение, которое должно запускать сразу без каких-либо нагов. Изучим немного о том, как работает крэкми, разумеется, с помощью OllyDbg.

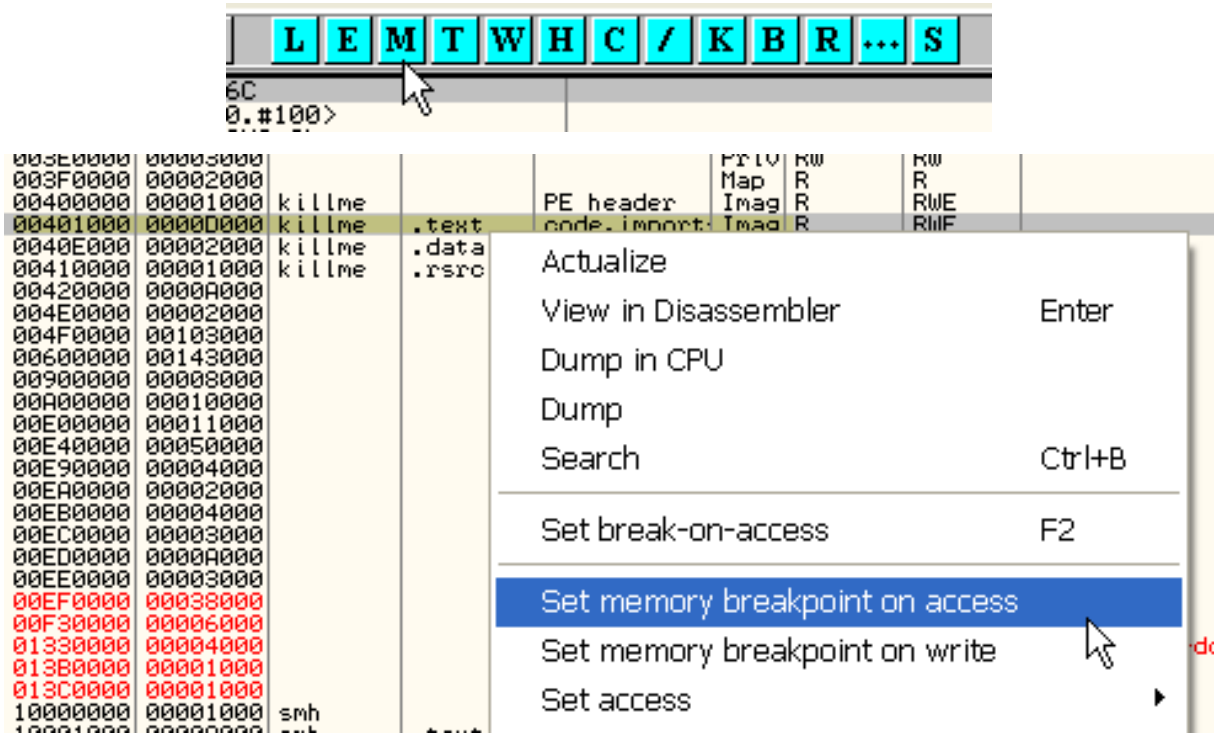
Первое побеждение, возникающее при исследовании программы с наг-окном — это посмотреть JMP-переходы, ведущие в разные части программы.



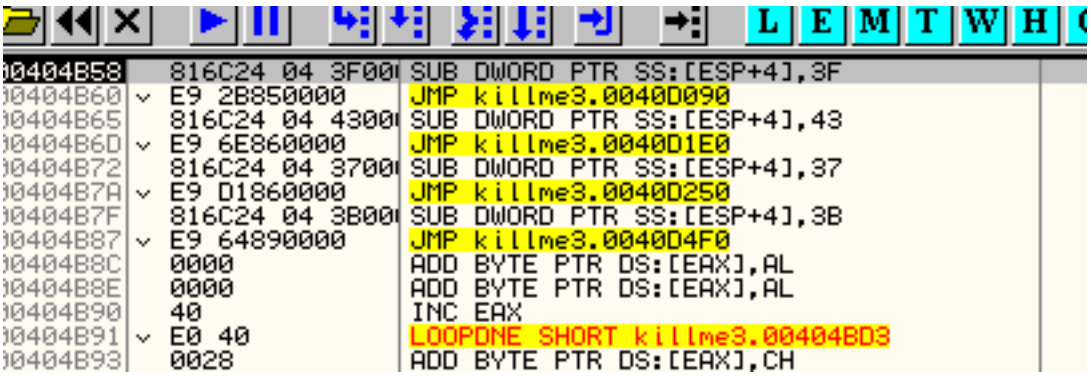
Запускаем программу и нажимаем F9.

Указанное время прошло и кнопка CONTINUE становится доступной для нажатия.

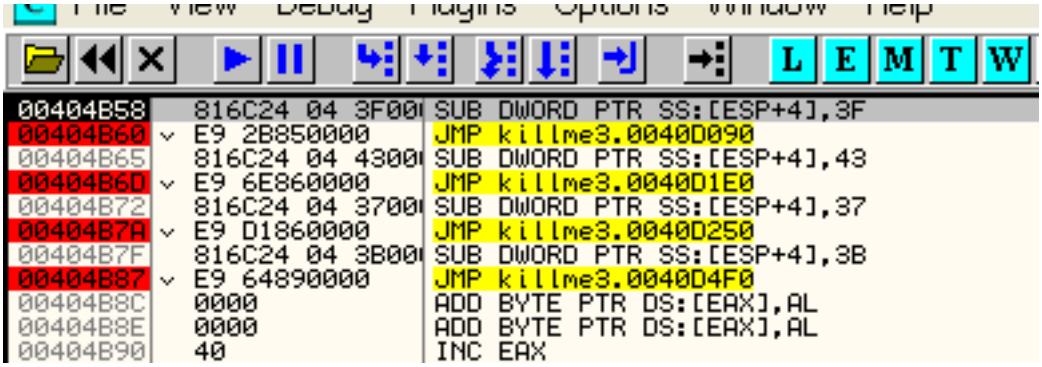
Знаем, что при закрытии нага осуществляется проход через JMP в секции кода, поэтому устанавливаем BPM ON ACCESS в указанной секции (что в реальности будет BPM ON EXECUTION) и нажимаем кнопку CONTINUE.



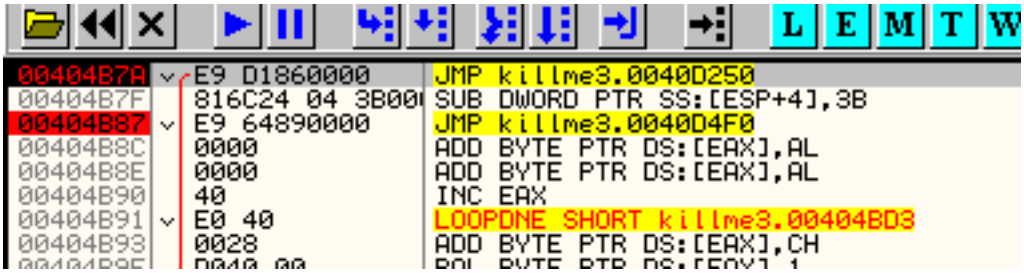
Останавливаемся здесь.



То есть видим, что собственно программа начинает выполнение в 40D090, то есть мы видим JMP-переход на 40D090, где начинается указанная часть и видим остальные JMP на разные части программы. Устанавливаем BP на все эти JMP, чтобы куда и в каком случае они ведут.

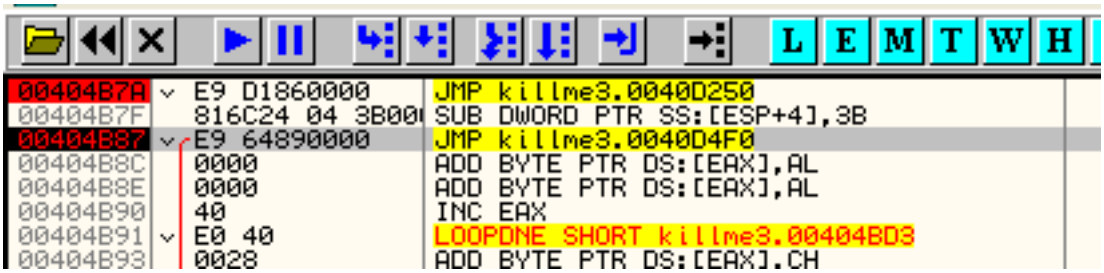


Перезапускаем программу.



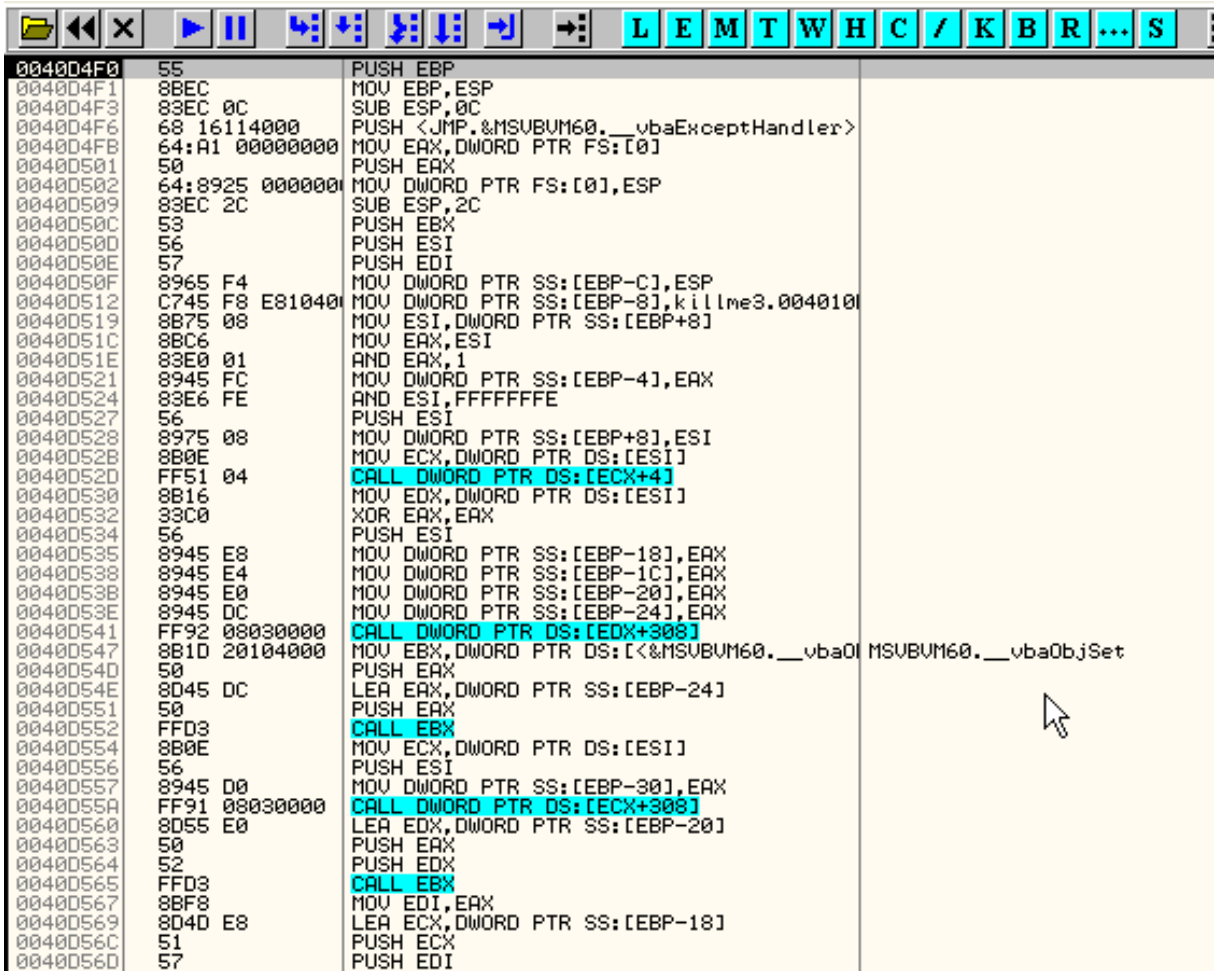
Видим, что после выхода из нага останавливаемся здесь. Есть большой соблазн поменять JMP на JMP 409090, чтобы происходил прямой переход на программу, но это вызывет ошибку и запуска не происходит, так что опять делаем RUN.

Видим, что останавливаемся на следующем JMP и если нажмём RUN снова:



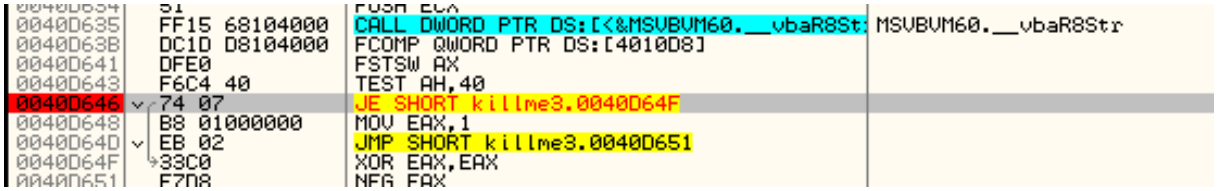
Несколько секунд и останавливаемся снова, и так – 5 раз, что говорит мне о том, что в этой процедуре содержится таймер.

Оттрасируем эту процедуру и посмотрим, что в ней находится.



Ничего хорошего, продолжаем трассировать с помощью F8.

Доходим до каких-то условных переходов. Можно посмотреть, что произойдёт, если заменим их на обратные: перестанет ли работать таймер и станет ли доступной для нажатия кнопка. Попытаемся выяснить, для чего они служат.



Это сравнение, использующее плавающую точку, пока не знаем, для чего она, так что менять переходы на обратные даст нам не очень много.

```
0040D641    DFE0            FSTSW AX
0040D643    F6C4 40         TEST AH,40
0040D646    74 07           JE SHORT killme3.0040D64F
0040D648    B8 01000000     MOV EAX,1
0040D64D    EB 02           JMP SHORT killme3.0040D651
0040D64F    33C0            XOR EAX,EAX
```

Видим, что в первые проходы здесь происходил переход, а в последний раз – нет, делаем вывод, что это и есть сравнение таймера, которое заканчивает работу, когда достигает нуля. Забиваем условный переход NOP'ами и смотрим, что произойдёт.

```
0040D63B    DC1D D8104000   FCOMP QWORD PTR DS:[4010D8]
0040D641    DFE0            FSTSW AX
0040D643    F6C4 40         TEST AH,40
0040D646    90              NOP
0040D647    90              NOP
0040D648    B8 01000000     MOV EAX,1
0040D64D    EB 02           JMP SHORT killme3.0040D651
0040D64F    33C0            XOR EAX,EAX
0040D651    F7D8            NEG EAX
0040D653    8D4D E8         LEA ECX,DWORD PTR SS:[EBP-18]
```

И нажимаем RUN.

Видим, что попали в цель, изменили условный переход, прервав тем самым счёт таймера и сделав доступной кнопку CONTINUE, так что пока что нам подойдёт этот вариант. Теперь надо добиться, чтобы кнопка была доступна после возврата из этой процедуры, нужно, чтобы происходил переход в начало программы.

Перезапустим программу.

Идём к строке, которую забиваем NOP'ами до запуска чего бы то ни было.

```
0040D641    DFE0            FSTSW AX
0040D643    F6C4 40         TEST AH,40
0040D646    90              NOP
0040D647    90              NOP
0040D648    B8 01000000     MOV EAX,1
0040D64D    EB 02           JMP SHORT killme3.0040D651
```

```
0040D63B   DC1D D8104000   FCOMP QWORD PTR DS:[4010D8]
0040D641   DFE0           FSTSW AX
0040D643   F6C4 40        TEST AH,40
0040D646   90             NOP
0040D647   90             NOP
0040D648   B8 01000000     MOV EAX,1
0040D64D   EB 02           JMP SHORT killme3.0040D651
```

Убираем все BP и ставим один сюда. Теперь делаем RUN.

```
0040D63B   DC1D D8104000   FCOMP QWORD PTR DS:[4010D8]
0040D641   DFE0           FSTSW AX
0040D643   F6C4 40        TEST AH,40
0040D646   90             NOP
0040D647   90             NOP
0040D648   B8 01000000     MOV EAX,1
```

Останавливаемся здесь и трассируем, чтобы посмотреть, где завершается процедура и происходит возврат в библиотеку Visual Basic'a.

```
0040D707   6A 02           PUSH 2
0040D709   FF15 10104000   CALL DWORD PTR DS:[<&MSVBVM60.__vbaFreeE MSVBVM6
0040D70F   83C4 18         ADD ESP,18
0040D712   C3              RETN
0040D713   C3              RETN
0040D714   8B45 08         MOV EAX,DWORD PTR SS:[EBP+8]
0040D717   50              PUSH EAX
0040D718   8B08            MOV ECX,DWORD PTR DS:[EAX]
0040D71A   FF51 08         CALL DWORD PTR DS:[ECX+8]
0040D71D   8B45 FC         MOV EAX,DWORD PTR SS:[EBP-4]
0040D720   8B4D EC         MOV ECX,DWORD PTR SS:[EBP-14]
0040D723   5F              POP EDI
0040D724   5E              POP ESI
0040D725   64:890D 000000(  MOV DWORD PTR FS:[0],ECX
0040D72C   5B              POP EBX
Return to 0040D714 (killme3.0040D714)
```

Видим, что дошли до RETN, но переходит не в DLL VB, а продолжаем с 40d714.

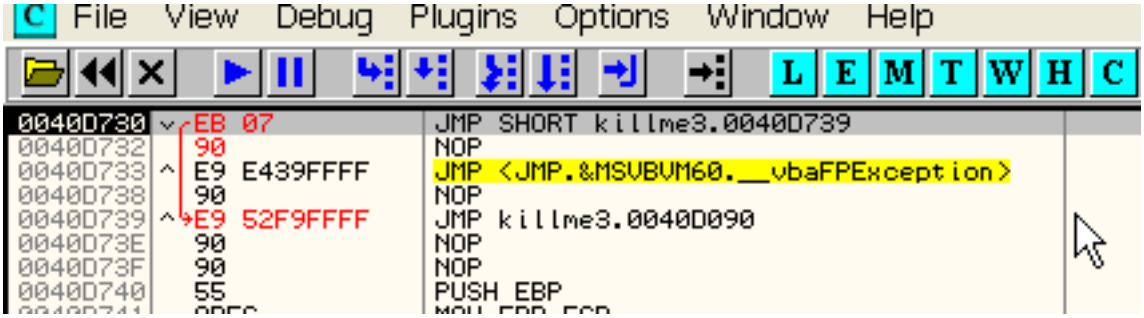
Продолжаем трассировку.

```
0040D72C   5B              POP EBX
0040D72D   8BE5            MOV ESP,EBP
0040D72F   5D              POP EBP
0040D730   C2 0400         RETN 4
0040D733 ^ E9 E439FFFF     JMP <JMP.&MSVBVM60.__vbaFPException>
0040D738   90              NOP
0040D739   90              NOP
0040D73A   90              NOP
0040D73B   90              NOP
0040D73C   90              NOP
0040D73D   90              NOP
0040D73E   90              NOP
0040D73F   90              NOP
0040D740   55              PUSH EBP
0040D741   8BEC            MOV EBP,ESP
0040D743   83EC 0C         SUB ESP,0C
0040D746   68 16114000     PUSH <JMP.&MSVBVM60.__vbaExceptHandler>
0040D74B   64:A1 00000000  MOV EAX,DWORD PTR FS:[0]
0040D751   50              PUSH EAX
Return to 66051FB3 (MSVBVM60.66051FB3)
Address   Hex dump                ASCII
0040E000  00 00 00 00 00 00 00 00 ........
```

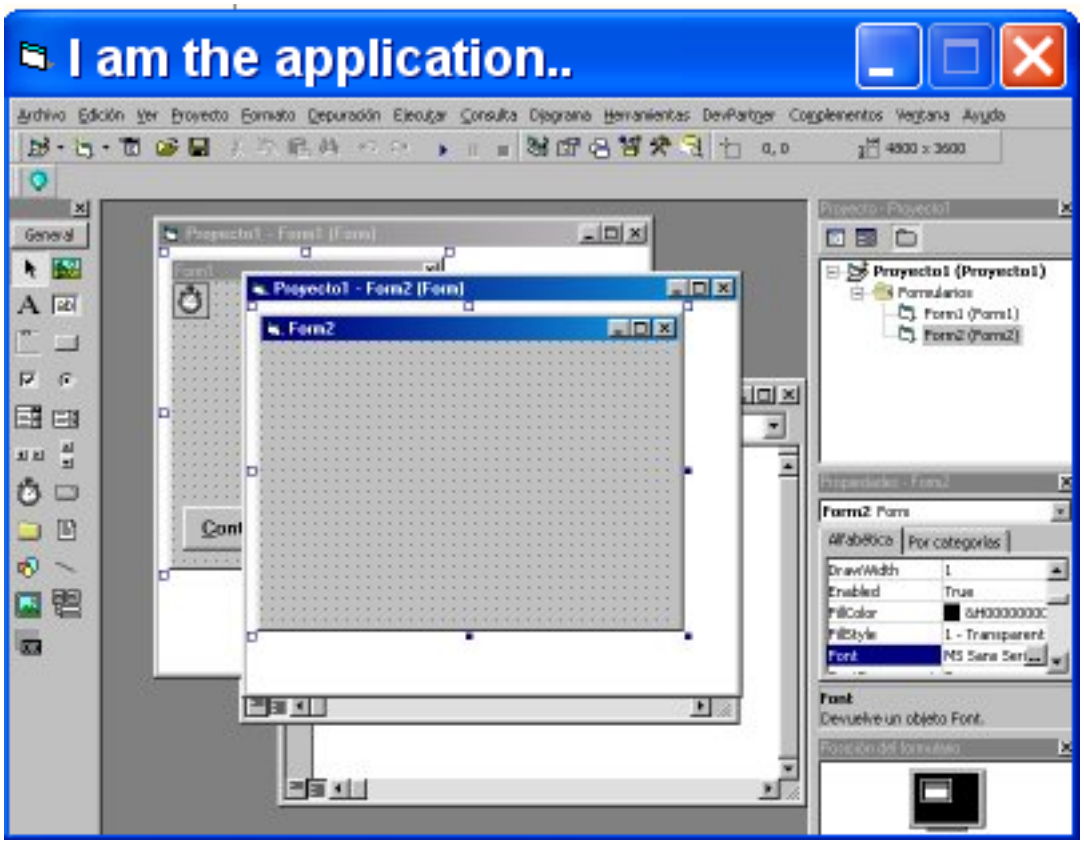
Здесь видим RETN, который, согласно пояснению, производит возврат в DLL Visual Basic'a, поскольку тут уже всё подготовлено для запуска программы, так что можем попытаться изменить этот RETN 4 на переход в начало программы. Смотрим.

```
0040D72D   8BE5          MOV ESP,EBP
0040D72F   5D            POP EBP
0040D730   EB 07         JMP SHORT killme3.0040D739
0040D732   90            NOP
0040D733   E9 E439FFFF   JMP <JMP.&MSVBVM60.__vbaFPException>
0040D738   90            NOP
0040D739   90            NOP
0040D73A   90            NOP
0040D73B   90            NOP
0040D73C   90            NOP
0040D73D   90            NOP
0040D73E   90            NOP
0040D73F   90            NOP
```

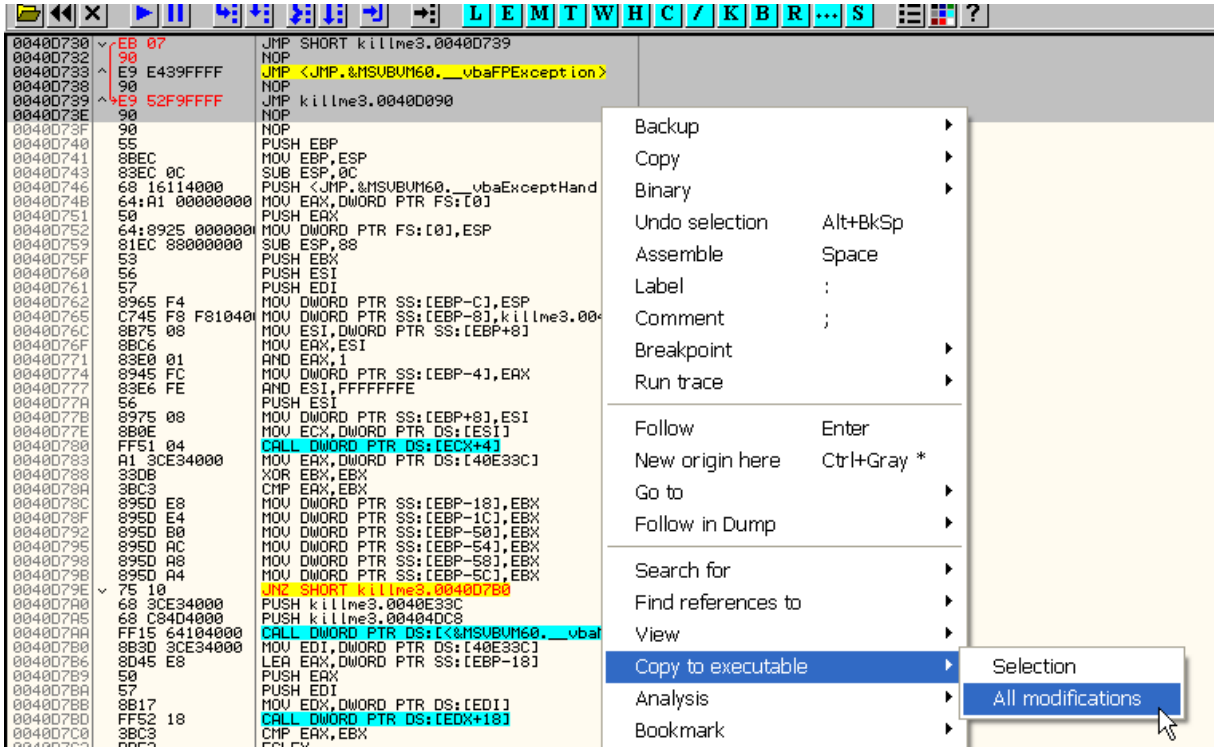
Прыгаем сюда, поскольку здесь достаточно места, чтобы написать всё, что нам нужно, а затем переходим на 40d090, откуда начинается программа.



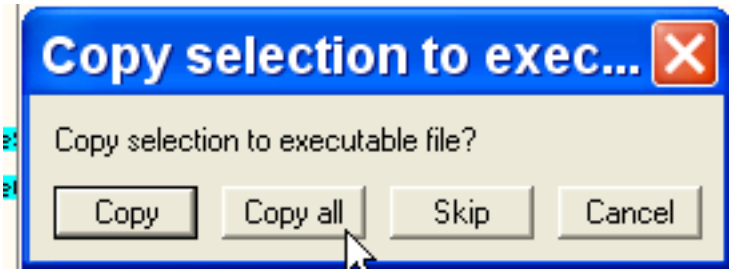
Пробуем, запустится ли. Делаем RUN.



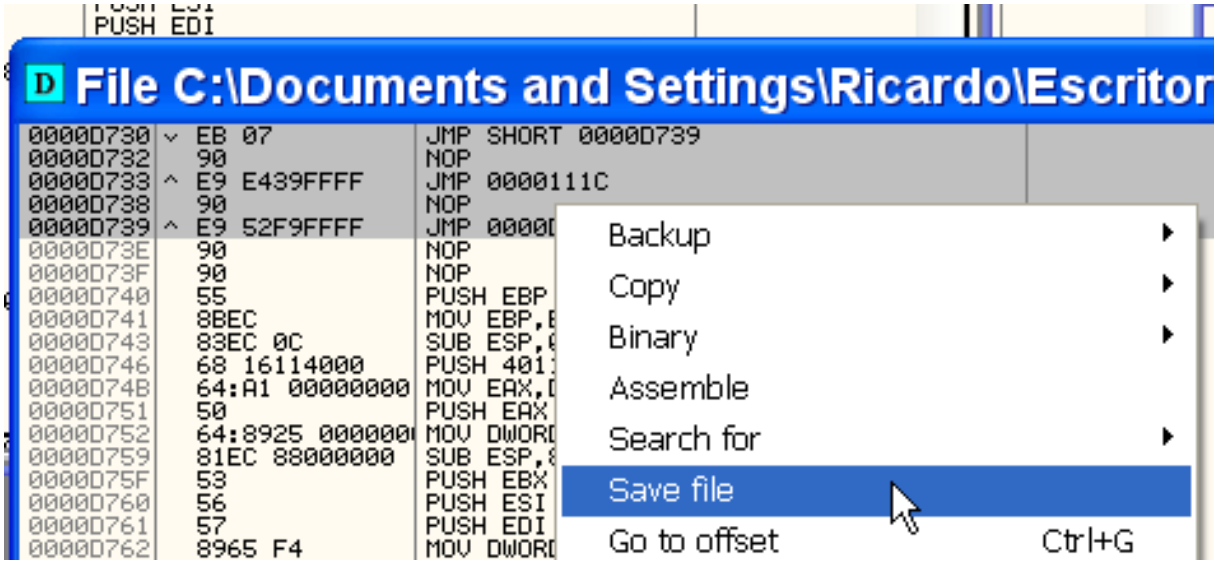
Можем сохранить эти изменения, начальные NOP'ы и эти два перехода.



Нажимаем правую кнопку мыши – COPY TO EXECUTABLE – ALL MODIFICATION и сохраняем всё, что изменили.

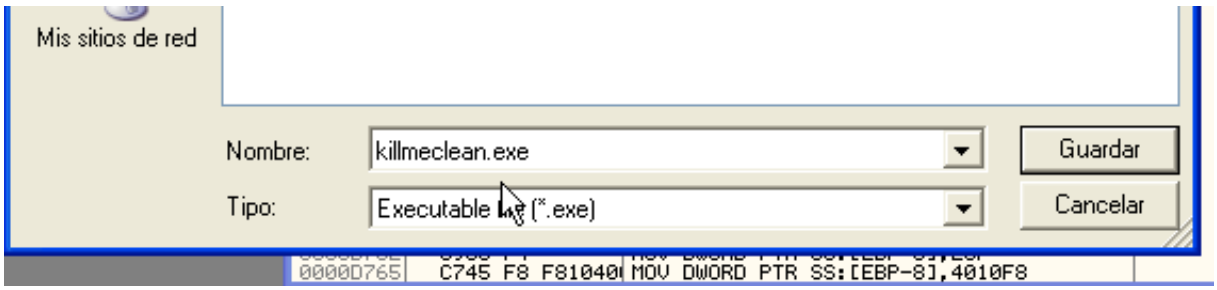


Нажимаем COPY ALL.

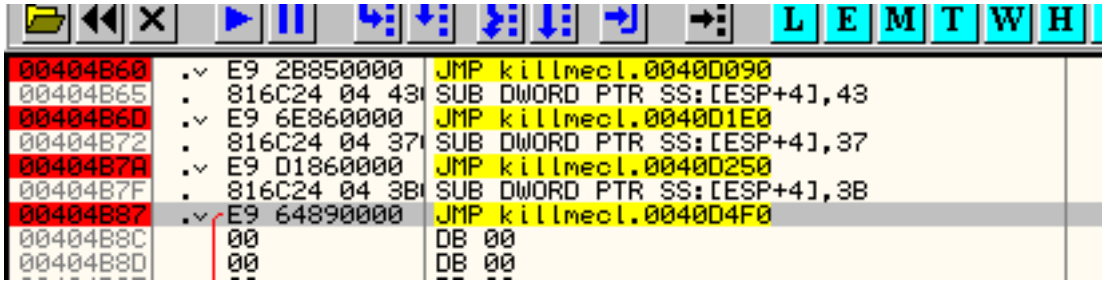


Ahora click derecho SAVE FILE

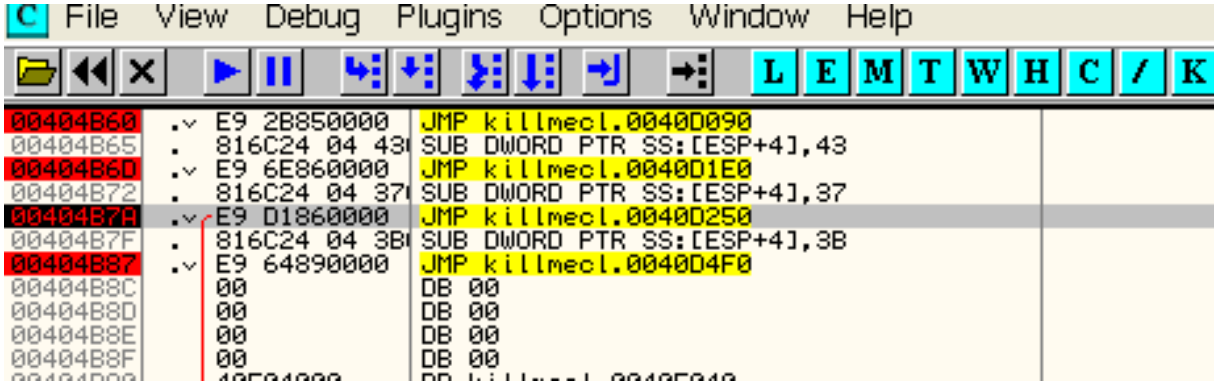
Теперь правая кнопка мыши – SAVE FILE.



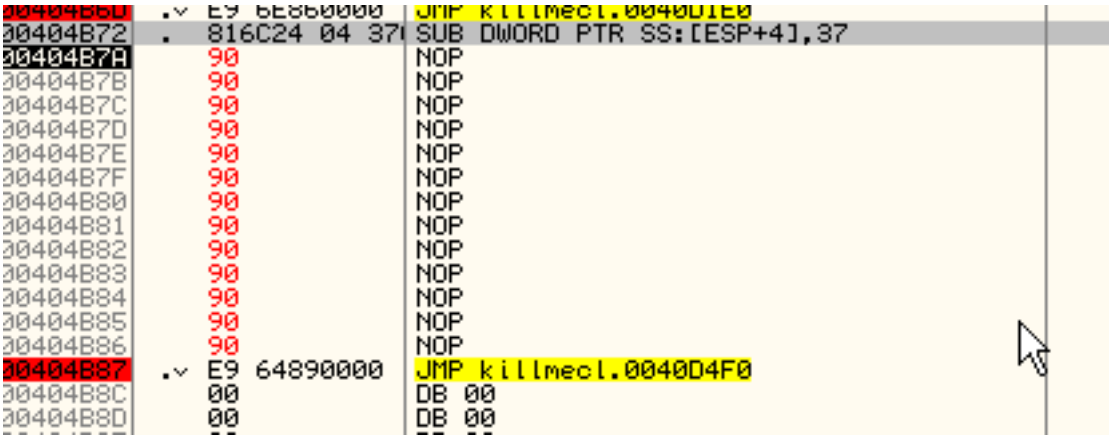
Ок, теперь мы гораздо ближе к решению. При запуске наг-окошко появляется и после нескольких секунд исчезает, так что снова устанавливаем BPX на эти JMP-переходы.



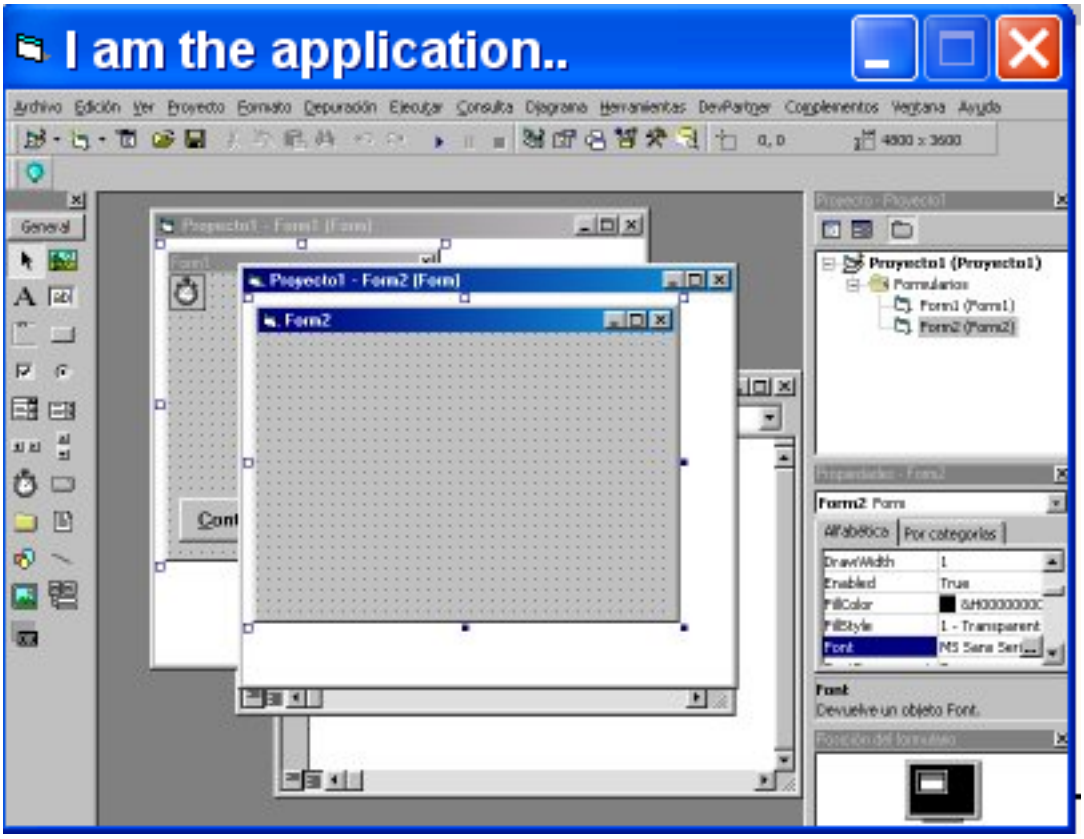
Делаем RUN.



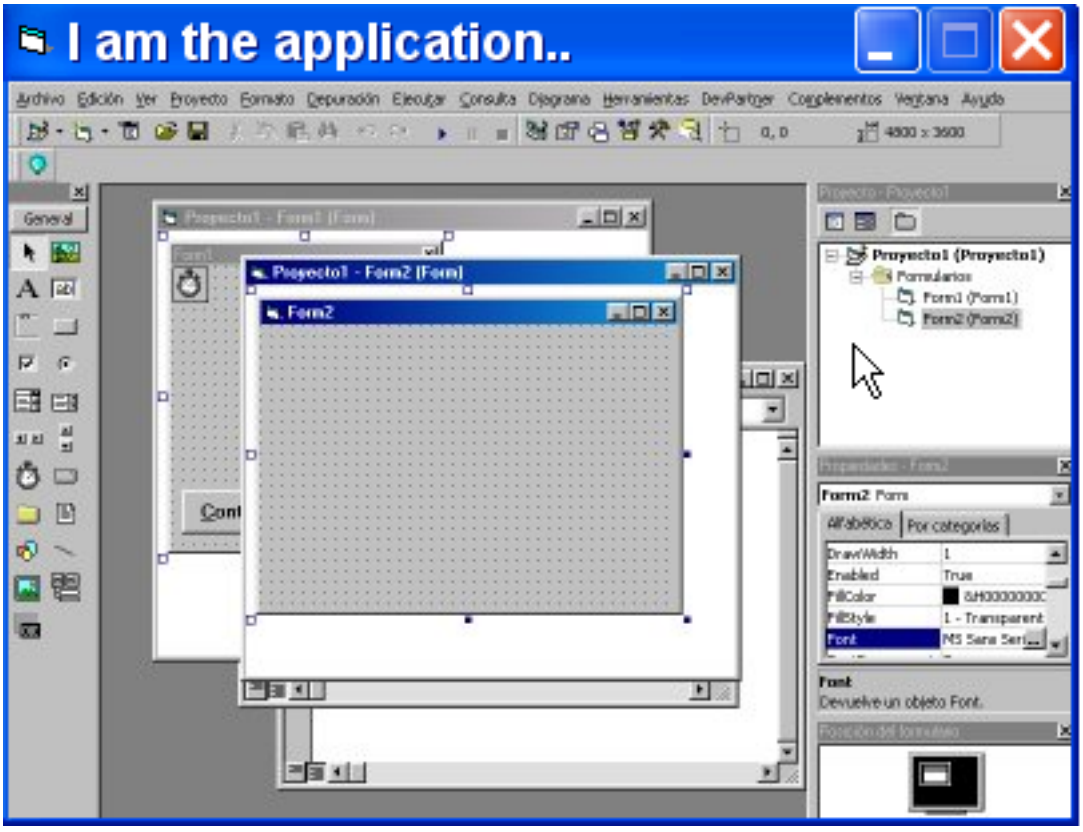
Видим, что сначала происходит остановка на том переходе, который создает наг-окошко (404b7a), а спустя нескольких секунд происходит закрытие окошка и остановка на 404b87, после чего сразу запускается программа благодаря нашему патчу, так что нам нужно изменить переход, который создаёт наг-окно, чтобы сначала выполнялась пропатченная нами процедура, а затем сразу запускалась сама программа.



Забиваем переход, который создаёт наг-окно, и идём сразу туда, где инициализируется таймер и начинается программа. Смотрим, что получилось, нажав F9.



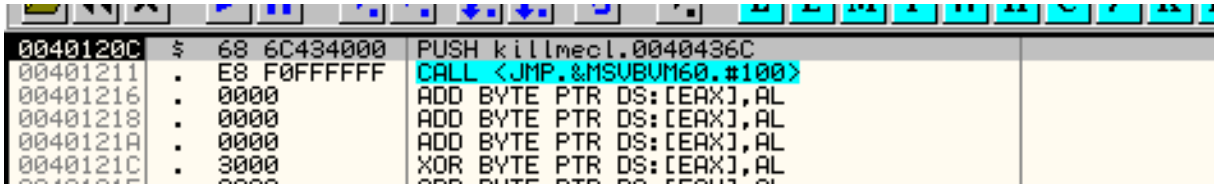
Сохраняем изменения и пробуем ещё раз.



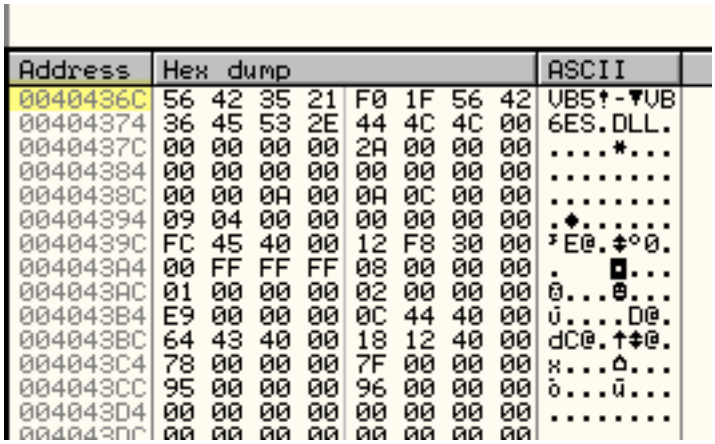
Происходит «чистый» запуск без каких-либо всплывающих окон – чистый как слеза младенца.

Метод 4c – это механический метод, который очень полезен, чтобы сделать это быстро и просто, и основывается на знании того, как устроен VB-файл.

Посмотрим на точку входа:



Видим, что программы, написанные на VB начинаются с PUSH и CALL (если вам они не встретились, значит, что exe-файл был изменён, тогда необходимо найти PUSH и записать адрес, который кладётся в стек, в данном случае 40436C).

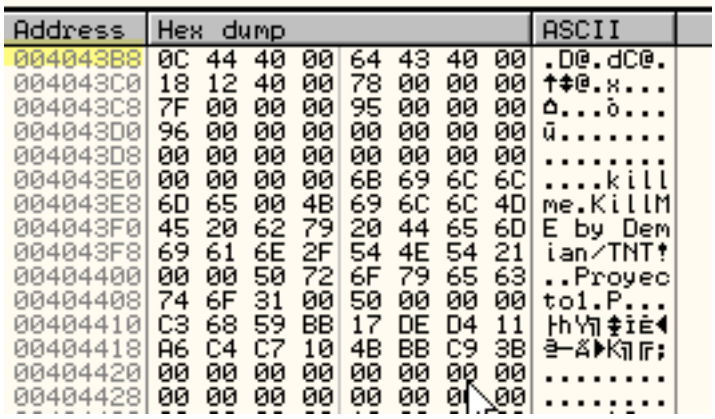


Как видим, это адрес VB-заголовка.

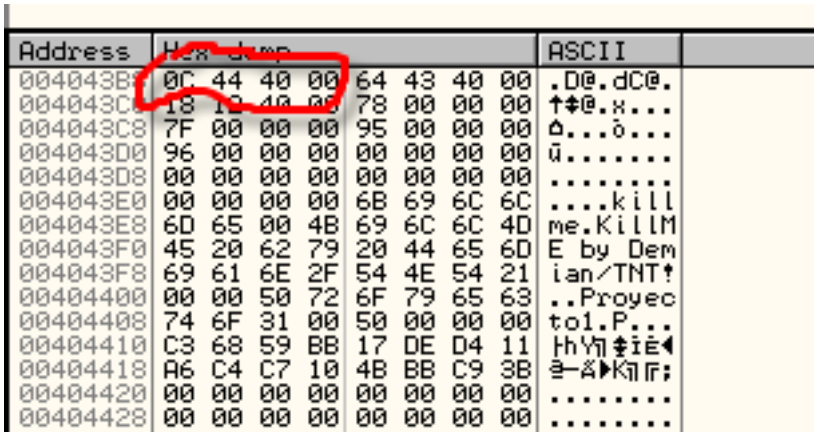
К этому адресу необходимо прибавить 4C.

40436c + 4c

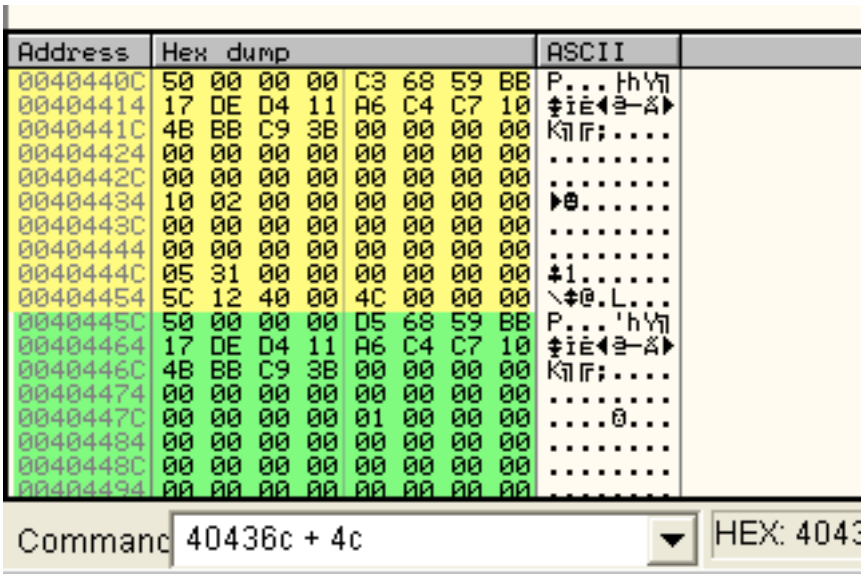
Это 4043b8.



Вот он, теперь ищем адрес, который здесь лежит – это 4044c0.

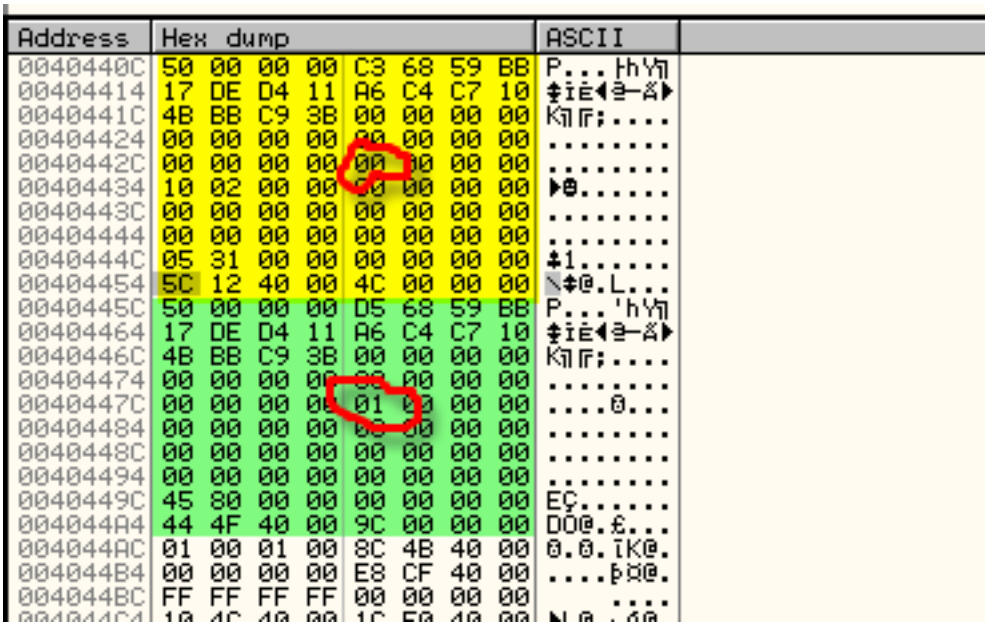


Находим данный адрес в DUMP’е.

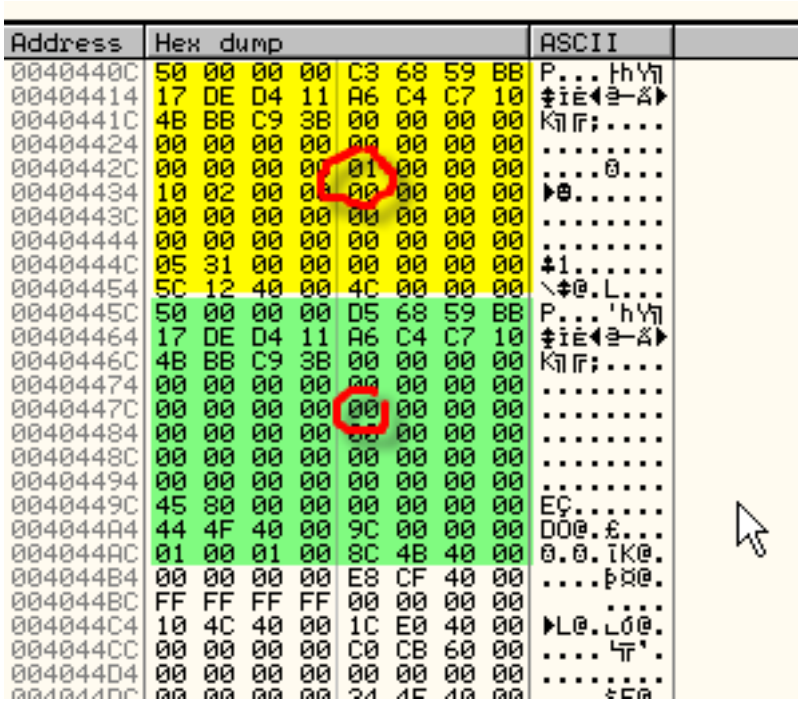


Видим похожие элементы, каждый из которых соответствует какой-то форме (FORM). Двадцать четвёртый байт каждого из этих элементов – это порядок, в котором формы должны появляться. Смотрим:

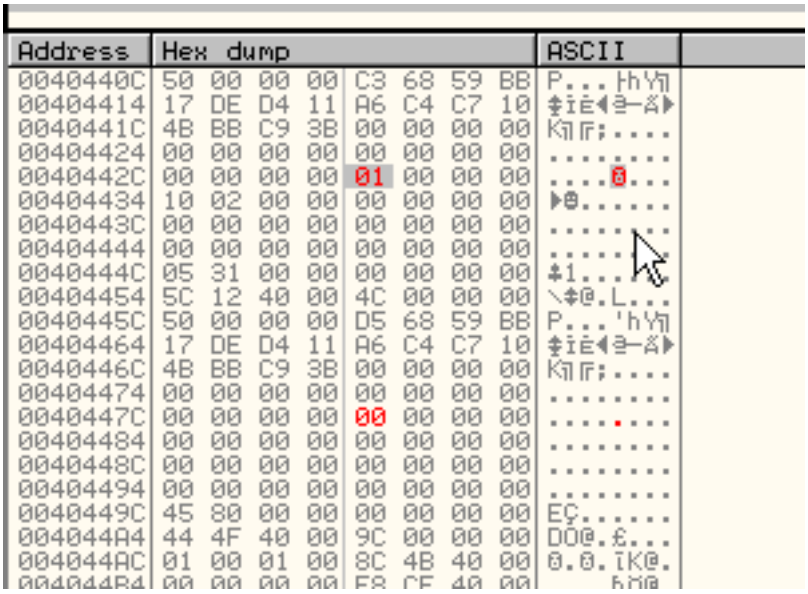
40440c +24=404430



Здесь находится 00, то есть эта форма будет появляться первой, а 01 означает, что это вторая, так что можем изменить порядок.



Так что первой появится программа, хе-хе, а наг-окно вторым или вообще не появится, так как при закрытии программы приложение завершится, и у второй формы не будет шансов.



Вот мы и решили крэкми с помощью метода 4c, конечно он гораздо легче и быстрее, но всегда полезно понимать как программа работает и сражаться с ней, поэтому я показал вам оба метода, так как автоматический метод подойдёт не во всех случаях, и придётся исследовать программу глубже.

К этой главе прилагаются два крэкми – один простой и один посложнее, посмотрите, сможете ли вы найти серийные номера, избавиться от наг-окон и так далее. В следующей главе мы их решим.

Файлы к статье (находится в архиве `wasm_files.zip`: `files/0486/ollydbg27.rar`)

Введение в крэкинг с нуля, используя OllyDbg - Глава 28

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-05-21

Ещё техники для Visual Basic (абсолютная война)

Лично я слышал немало крэкеров, которые говорили, что тот или другой метод – это не «чистый» крэкинг, не «элегантный» крэкинг или что-нибудь ещё вроде этого. Для меня «элегантный» крэкинг – тот, который работает, и это всё, что имеет значение, потому что на войне как на войне: программисты, создающие упаковщики и протекторы не задумываются о том, насколько «элегантны» их действия, они используют любые методы, которые могут найти, не думая о том, как те повлияют на наши машины. Мы рассмотрим примеры грязных трюков, которые используются упаковщиков в отчаянной попытке сделать так, чтобы никто не мог распаковать защищаемую ими программу.

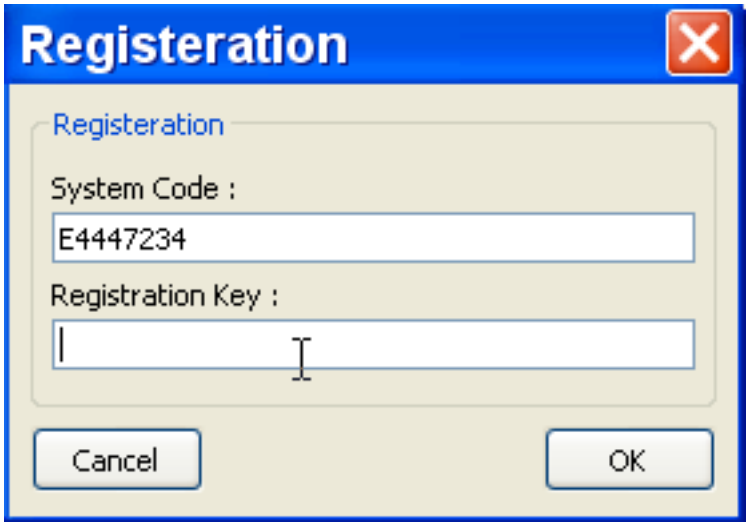
Поэтому я думаю, что если враг использует ракеты, ограничивать себя в бою револьвером – это значит заведомо проиграть, так что я использую ЛЮБОЙ метод, который нормально работает, не портит другие программы, корректен и юзабелен, и то, что он не одобрен АЭК (Ассоциацией элегантных крэкеров) меня не остановит.

Прояснив этот момент, переходим к рассмотрению следующего крэкми, которое будет объектом сегодняшнего изучения.

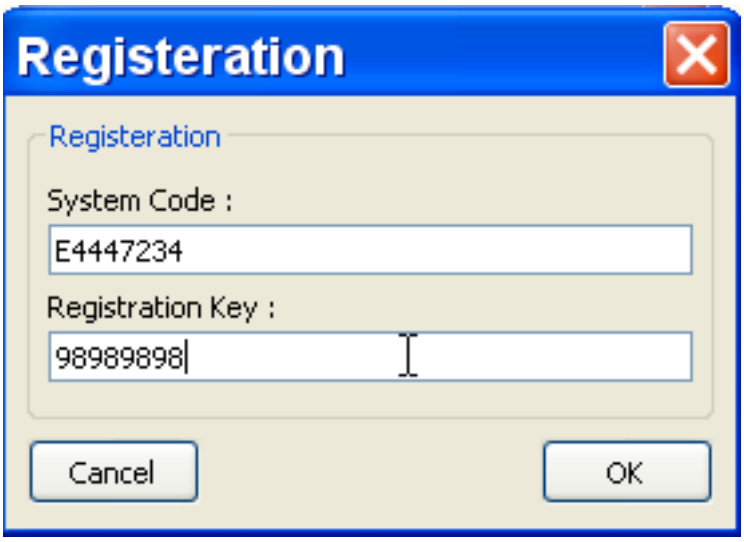
С помощью метода 4c мы может легко избавиться от наг-окна. Часть, связанная с серийным номером, сделана с помощью PCODE, и его мы оставим до того момента, как начнём рассматривать эту тему.

Найти в этом крэкми серийный номер очень просто. Главный вопрос с его окном в том, является ли оно нагом или нет. Если наложить метод 4c, то можно увидеть, что там две формы и, используя этот метод, избежать их появления друг за другом нельзя.

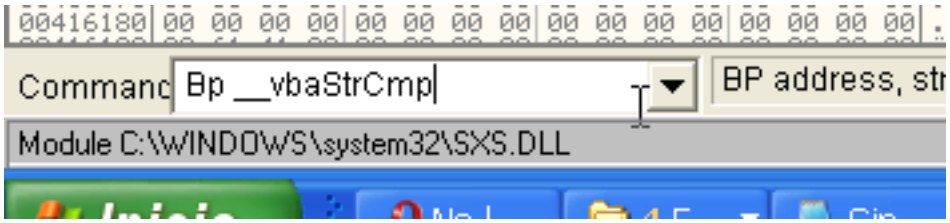
Прежде всего посмотрим, что там с вводом серийного номера.



Оказываемся в окне крэкми и вводим неправильный серийный номер.



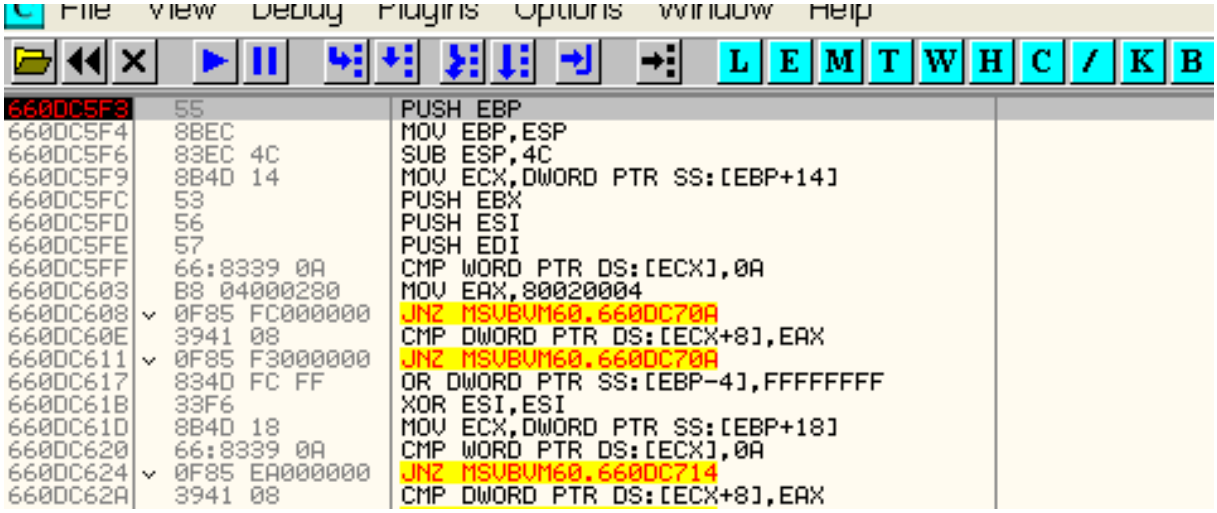
Одна из самых полезных функций для сравнения – это vbaStrCmp.



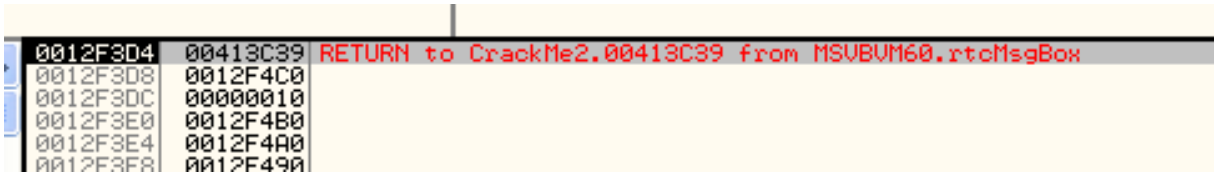
Устанавливаем на неё BP, чтобы посмотреть, будет ли она использоваться для определения, является ли введённый серийный номер правильными.

Так как останавливаемся множество раз и постоянно нажимать на F9 лень, то посмотрим, можно ли перехватить сообщение об ошибке и оттуда попасть в сравнение, которое должно быть где-то поблизости. Снимаем BP на сравнение и устанавливаем его на вывод окошка сообщения.

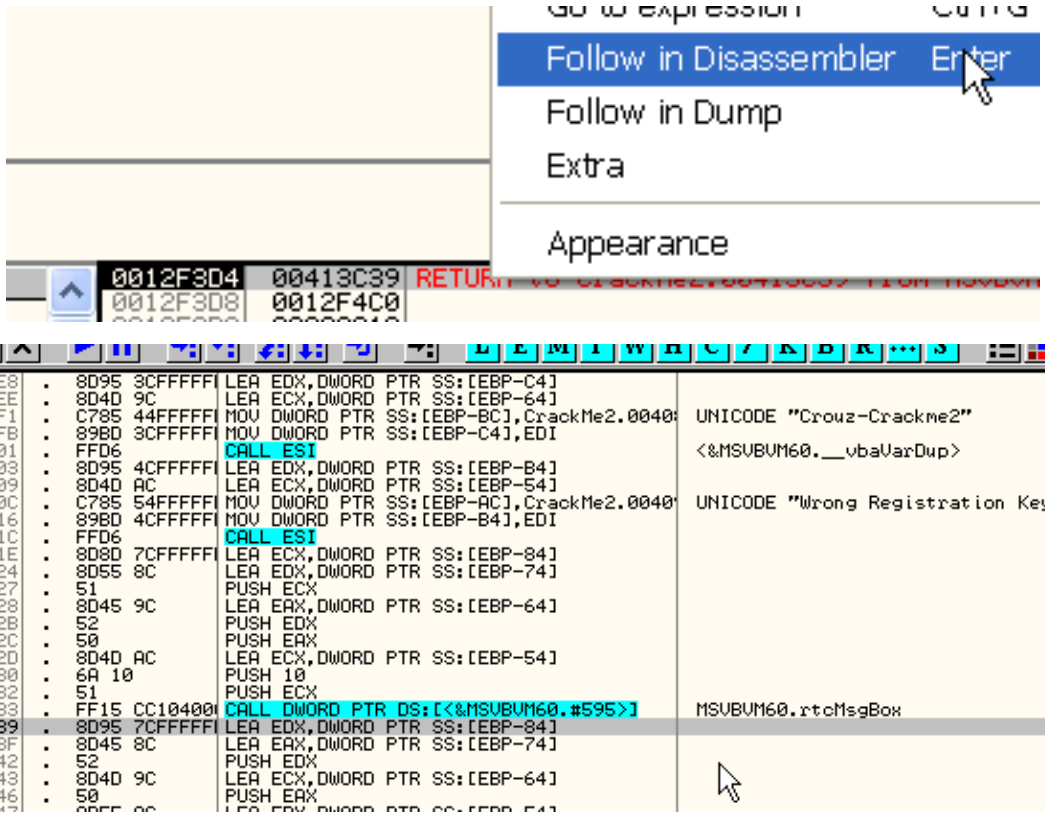
Bp rtcMsgBox



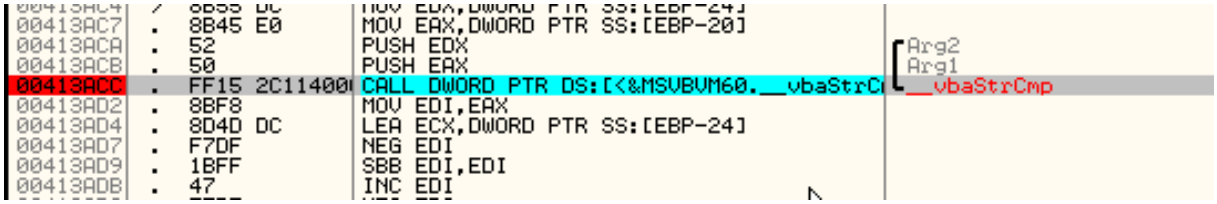
Останавливаемся здесь, когда должно быть отображено сообщение о том, что серийный номер неверен, смотрим, откуда произошёл вызов в первой строке стека, которая содержит строку возврата из данной API-функции.



Идём сюда.

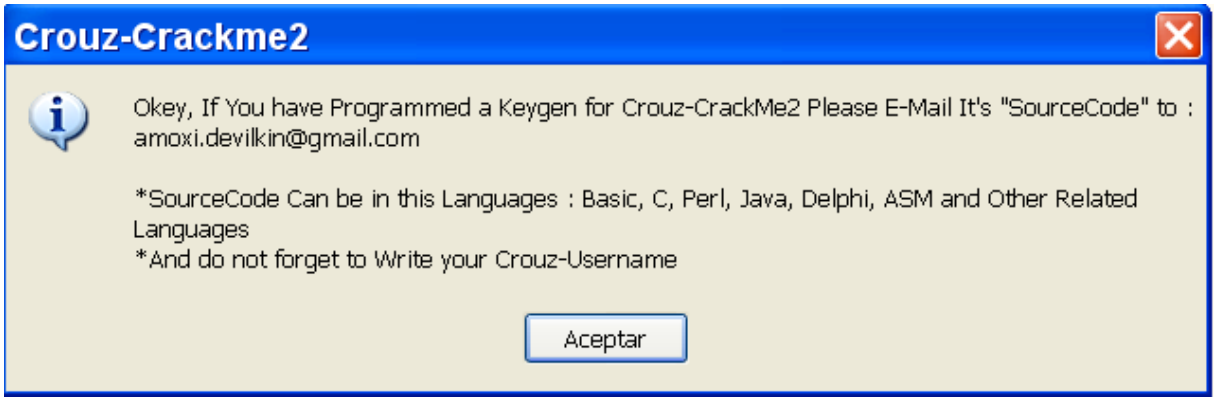


Вот, куда произойдёт возврат. Это вызов API-функции и смотрим выше, есть ли там какое-нибудь сравнение.



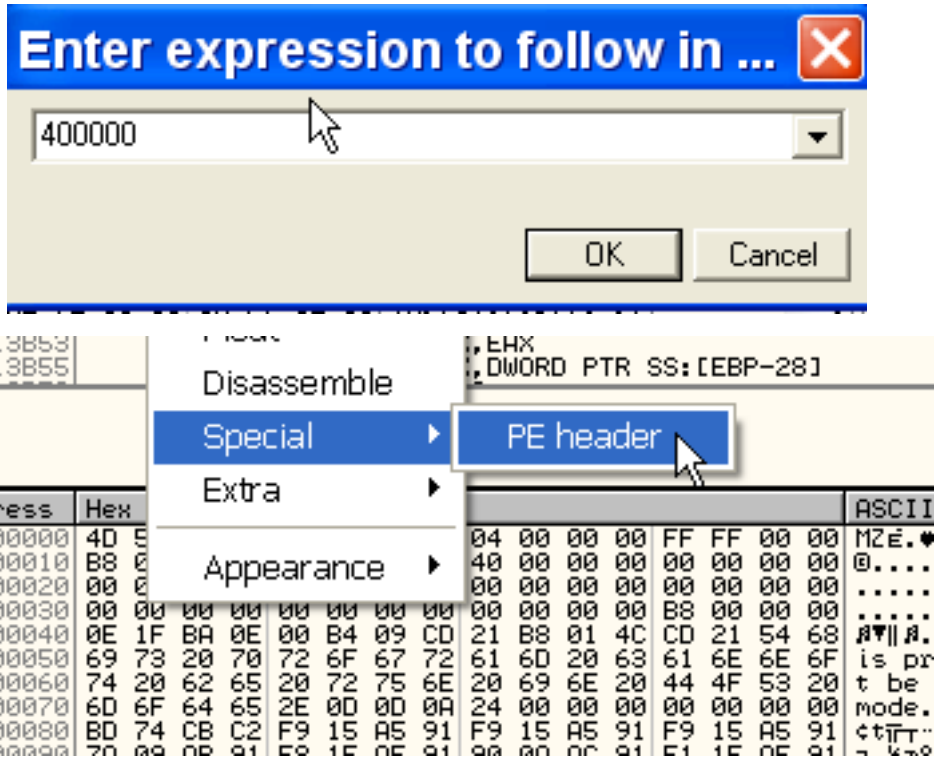
Немного ранее видим его, устанавливаем BP, чтобы проверить, верно ли наше предположение, нажимаем F9.

Здесь видим, что мой серийный номер сравнивается с другим числом. В моём случае это 4887649. Смотрим, является ли этот серийный номер правильным. Убираем все BP и снова попадаем в окно ввода серийного номера.

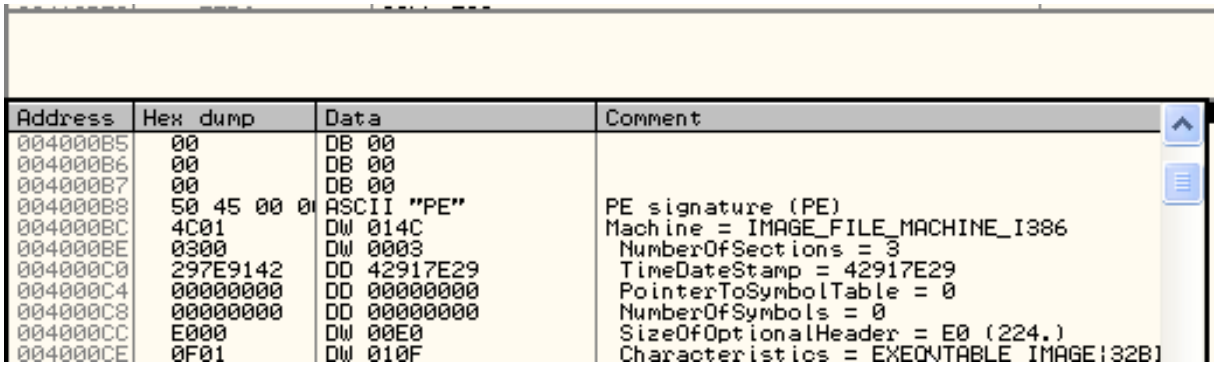


Да, это правильный серийный номер, кейген оставим когда-нибудь на потом.

Сначала, для того чтобы убрать наг-окно, нужно сделать возможным модификацию крэкми, то есть установить разрешение на запись в секцию кода, для чего идём в заголовок программы, который начинается с 400000.

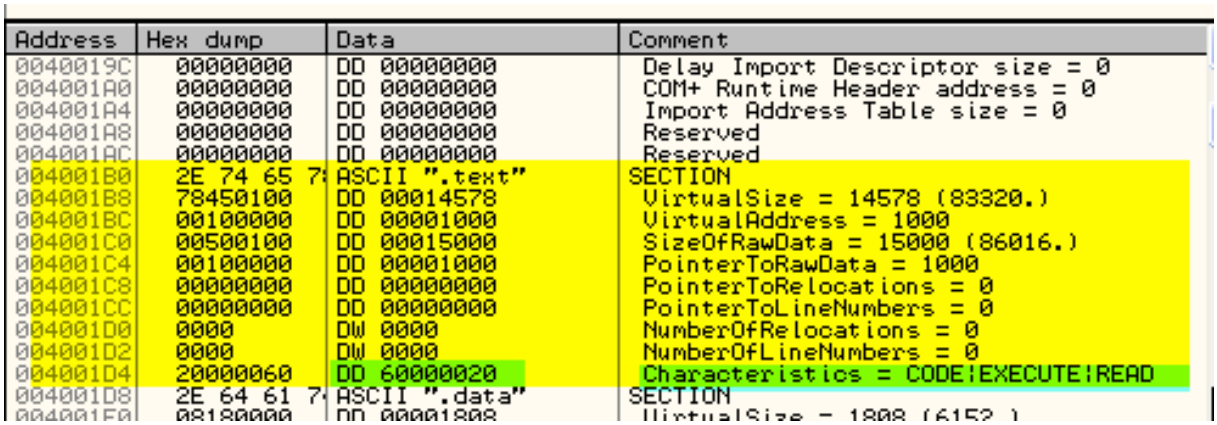


И переходим в режим PE HEADER.



Address	Hex dump	Data	Comment
004000B5	00	DB 00	
004000B6	00	DB 00	
004000B7	00	DB 00	
004000B8	50 45 00 0	ASCII "PE"	PE signature (PE)
004000BC	4C01	DW 014C	Machine = IMAGE_FILE_MACHINE_I386
004000BE	0300	DW 0003	NumberOfSections = 3
004000C0	297E9142	DD 42917E29	TimeDateStamp = 42917E29
004000C4	00000000	DD 00000000	PointerToSymbolTable = 0
004000C8	00000000	DD 00000000	NumberOfSymbols = 0
004000CC	E000	DW 00E0	SizeOfOptionalHeader = E0 (224.)
004000CE	0F01	DW 010F	Characteristics = EXECUTABLE IMAGE!32B

Спускаемся оттуда, где начинается PE SIGNATURE заголовка, до места, где находится первая секция.



Address	Hex dump	Data	Comment
0040019C	00000000	DD 00000000	Delay Import Descriptor size = 0
004001A0	00000000	DD 00000000	COM+ Runtime Header address = 0
004001A4	00000000	DD 00000000	Import Address Table size = 0
004001A8	00000000	DD 00000000	Reserved
004001AC	00000000	DD 00000000	Reserved
004001B0	2E 74 65 7	ASCII ".text"	SECTION
004001B8	78450100	DD 00014578	VirtualSize = 14578 (83320.)
004001BC	00100000	DD 00001000	VirtualAddress = 1000
004001C0	00500100	DD 00015000	SizeOfRawData = 15000 (86016.)
004001C4	00100000	DD 00001000	PointerToRawData = 1000
004001C8	00000000	DD 00000000	PointerToRelocations = 0
004001CC	00000000	DD 00000000	PointerToLineNumbers = 0
004001D0	0000	DW 0000	NumberOfRelocations = 0
004001D2	0000	DW 0000	NumberOfLineNumbers = 0
004001D4	20000060	DD 60000020	Characteristics = CODE!EXECUTE!READ
004001D8	2E 64 61 7	ASCII ".data"	SECTION
004001E0	00180000	DD 00001800	VirtualSize = 1800 (6152.)

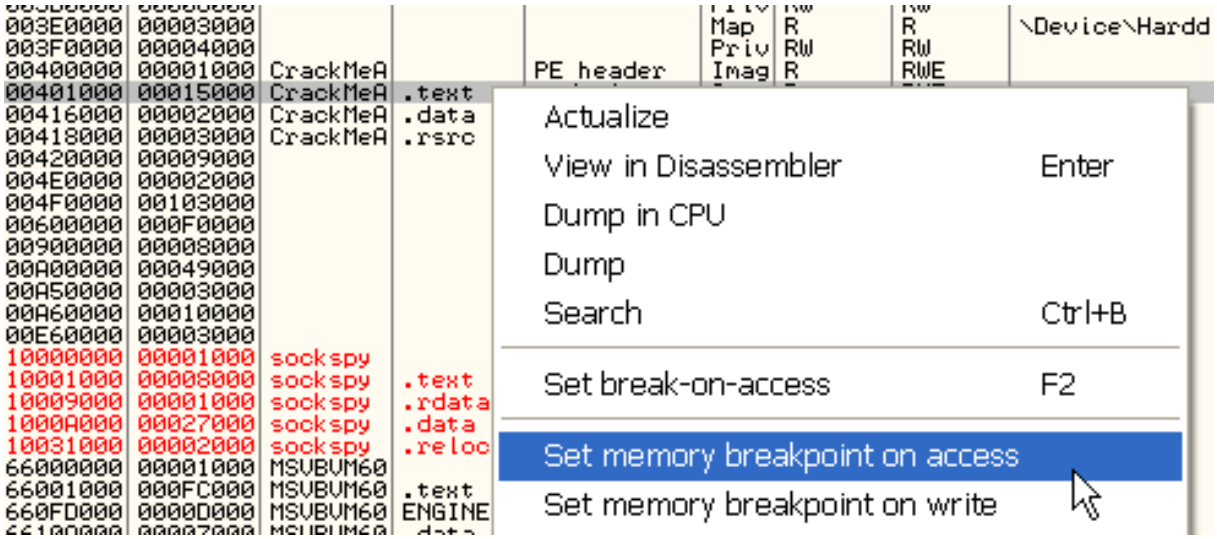
Здесь видим характеристики. Знаем, что если изменим на E0000020, то сможем производить запись в данную секцию.

Хорошо, теперь сохраняем изменения.

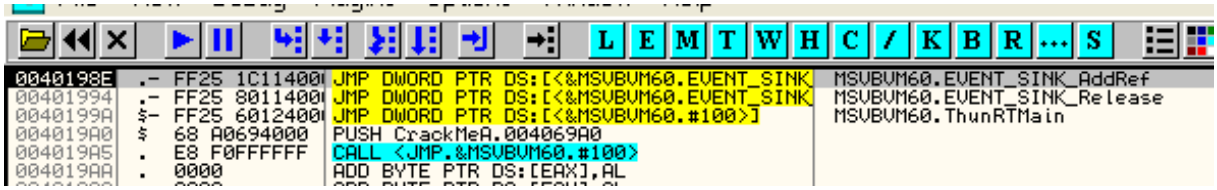
Сохраняем как CrackmeA.exe.

Теперь открываем CrackmeA в версии OllyDbg, пропатченной на предмет OEP'ов и Visual Basic'а.

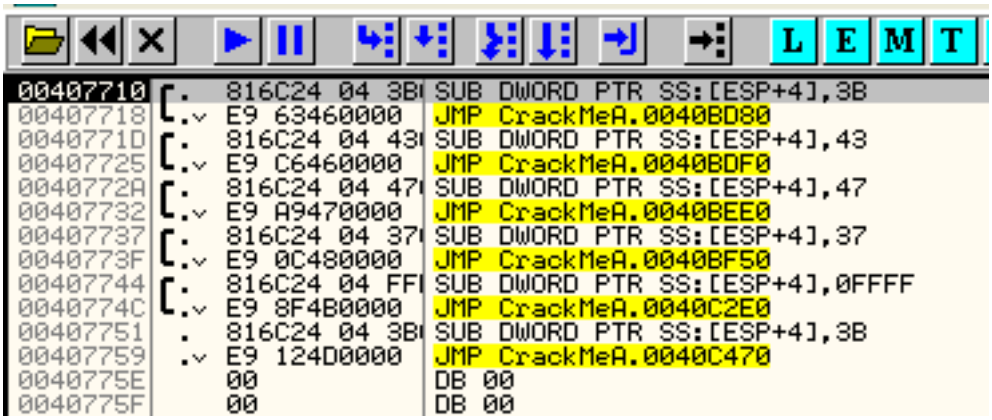
Устанавливаем BPM ON ACCESS (которые исчезли после выхода из отладчика) на секцию CODE.



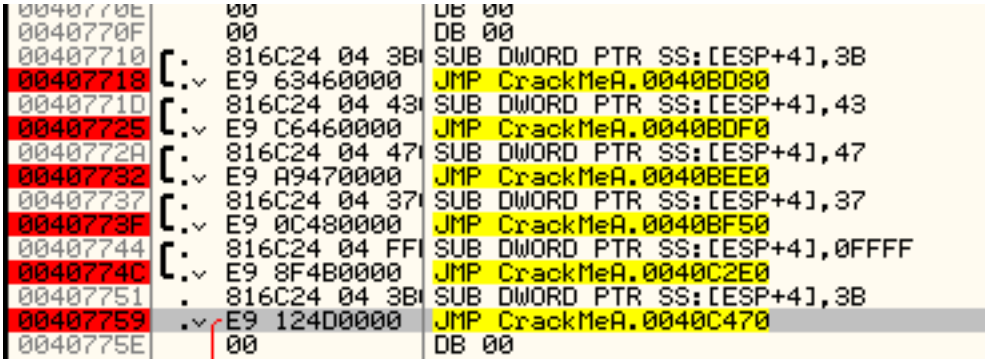
Делаем RUN несколько раз досюда.



Продолжаем, пока не остановимся на знакомых нам JMP'ах, уводящих в разные части программы.



Видим, что в первый раз останавливаемся на первом JMP и переходим на 40bd80, то есть здесь выполняется первая часть программы. Смотрим, появляется ли наг-окно до того как происходит возврат к этим переходам. Убираем BPM ON ACCESS, так как сейчас будет выполняться значительная часть программы в секции кода, и останавливаться на каждой строке нам совсем не нужно, лучше установим BP на эти конкретные переходы в разные части программы.

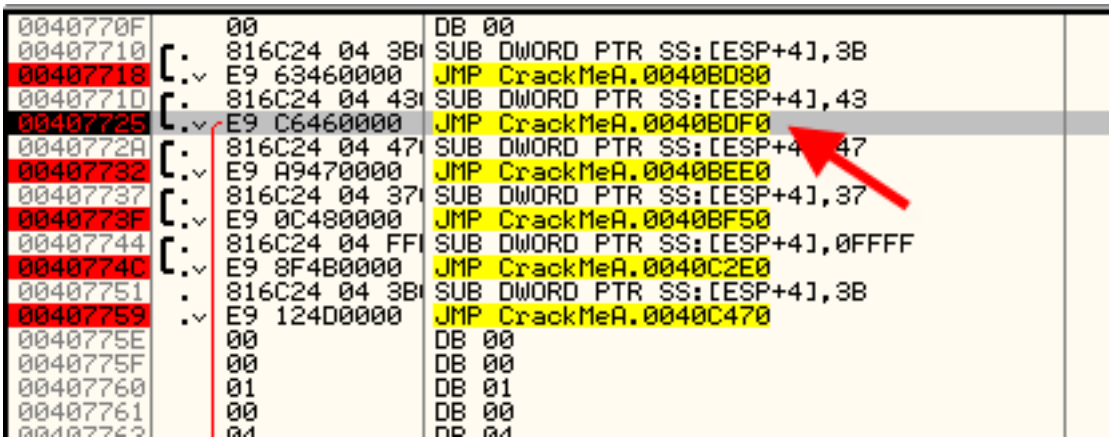


Посмотрим, будет ли наг-окно или перейдём к другому JMP. Делаем RUN.

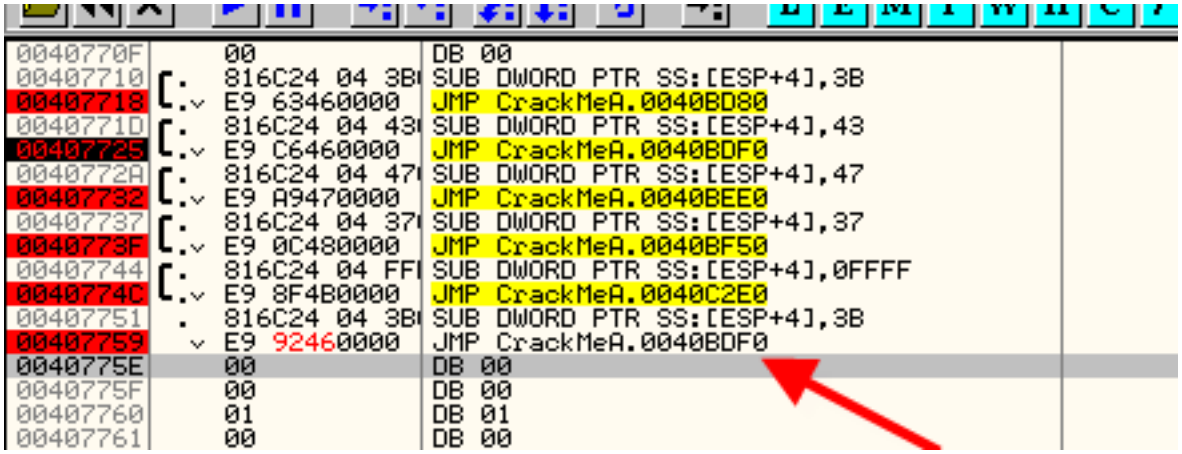
Сейчас останавливаемся на последнем переход, который ведёт в 40c470, но нага ещё нет.

Снова делаем RUN.

Здесь появляется наг-окошко, то есть часть программы, которая начинается с 40c470 ответственна за наг, нажимаем кнопку «Register».



Возвратившись из окна, останавливаемся на втором JMP, который, предположительно, ведёт в начало программы, так что можно попробовать, что произойдёт, если мы заменим последний JMP, ведущий в наг-окно на этот, переход на который осуществляется при выходе из этого окна.



Меняем второй переход, которые ведёт в часть, ответственную за появление наг-окошка на третий, который должен вести в начало программы. Сохраним изменения и посмотрим, что произойдёт.

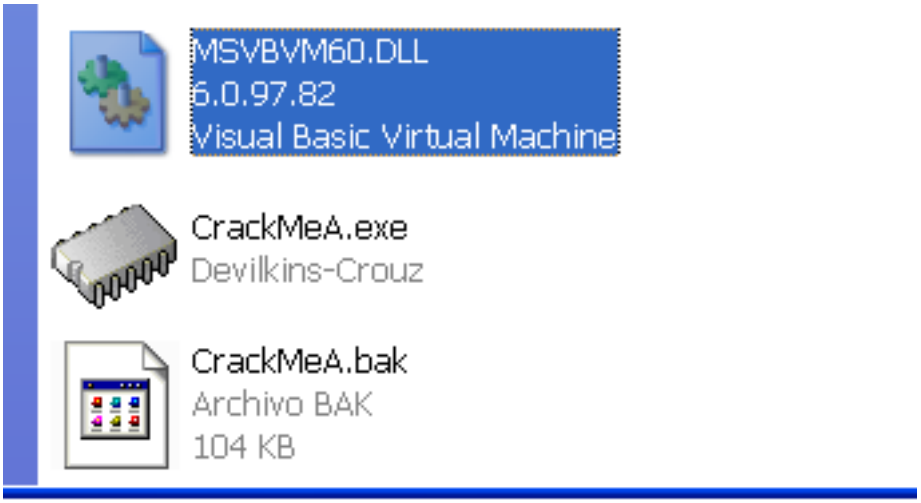
Запускаем изменённый нами CrackmeA не из-под OllyDbg.

Видим, что наше изменение слегка улучшило состояние дел с проблемой, хотя и не решило её, т.к. изначальное наг-окно не исчезло и требует, чтобы была нажата кнопка «Register», для чего отображает окошко для ввода серийного номера, которое находится сзади под первым. Это большой шаг вперёд, так как раньше если мы делали наг невидимым, то мы не могли нажать вышеупомянутую кнопку. Теперь же мы можем сделать так, что окно ввода отобразится, а наг сделать невидимым, так что окно для ввода серийного номера останется одно на экране.

Поскольку наши попытки с JMP не решили проблему полностью, не остаётся ничего другого, как вносить изменения в VB-библиотеку. Это может стать для многих крэкеров поводом, чтобы сказать, что не подобает изменять VB-библиотеку, но это верно только тогда, когда она находится в system32 и используется всеми программами. Мы скопируем её в ту же папку, где и располагается программа, так что она будет использовать только последней, а остальные программы будут продолжать использовать библиотеку в system32.

Вопрос заключается в том, как именно называется эта DLL.

Копируем в папку с программой.

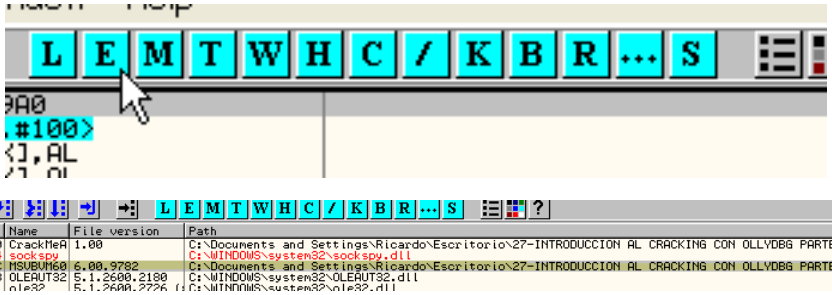


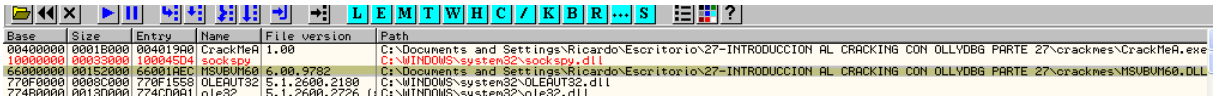
Теперь она находится вместе с CrackmeA, то есть чтобы перестала использоваться системная VB DLL, нужно скопировать её в папку с программой, и та будет использовать именно эту копию библиотеки.

Соответственно, когда некая программа использует какую-то нестандартную DLL, то первым дело надо поискать её в папке этой программы, а если её там нет, то в system32. Подобное положение вещей нас очень даже устраивает, хе-хе, потому что позволяет нам вносить изменения только для конкретной программы, не меняя ничего для других.

Снова открываем CrackmeA в OllyDbg, в этот раз используя обычный, неизменённый вариант отладчика, чтобы внести те строки кода или изменения, которые нельзя сделать в пропатченной версии.

Проверяем с помощью кнопки E, используется ли DLL из директории программы или из system32.

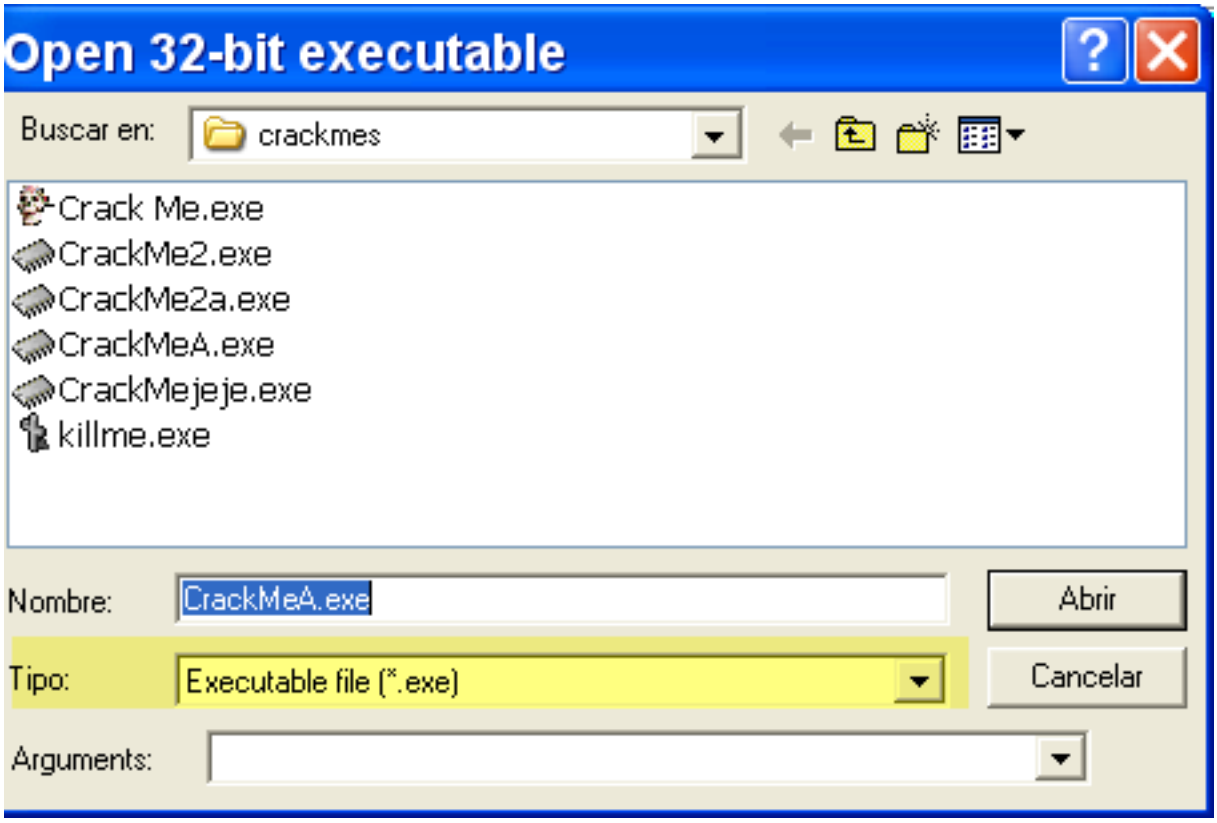




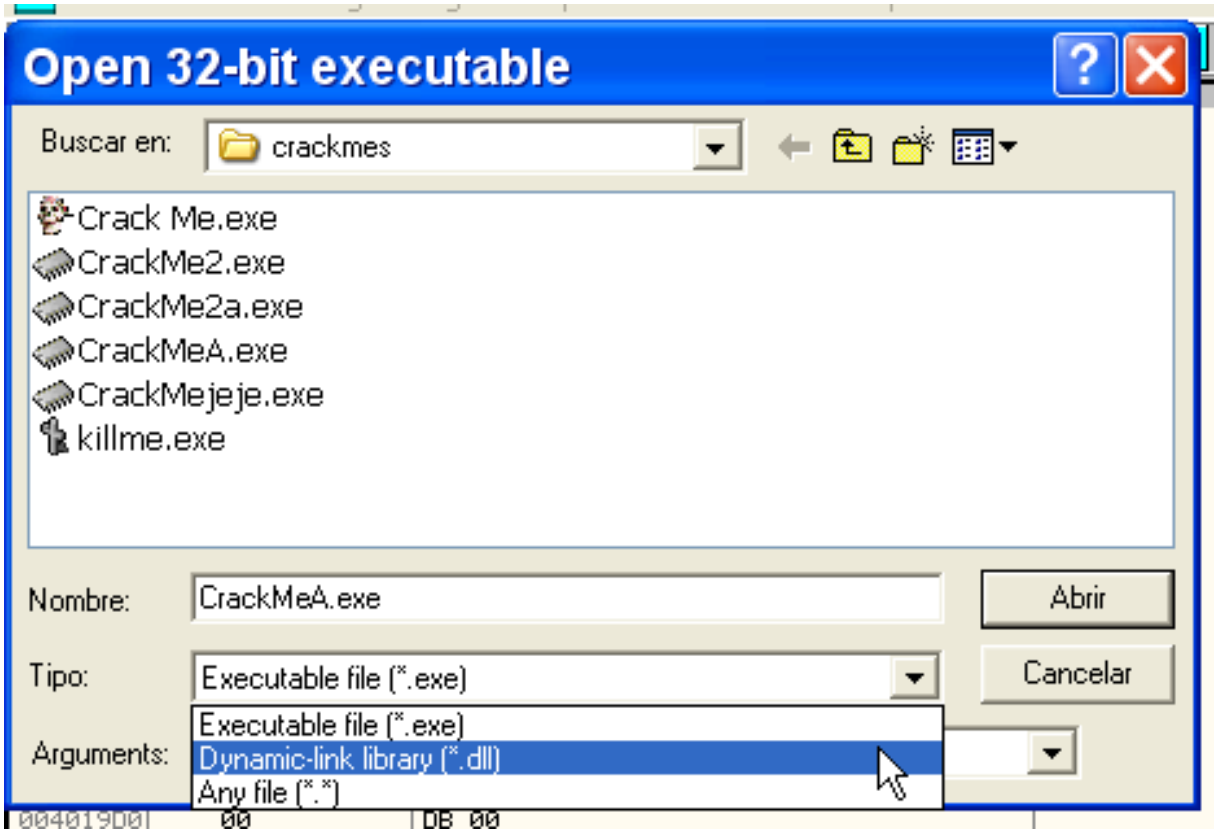
Видим, что загружена DLL из папки крэкми, поэтому все изменения, которые мы внесём в эту библиотеку, окажут воздействие только на данный крэкми и больше ни на какую программу.

Хорошо, у нас есть VB DLL. Для того, чтобы можно было вносить в неё изменения, нам нужно выставить в секции кода права на запись. Это можно сделать следующим образом.

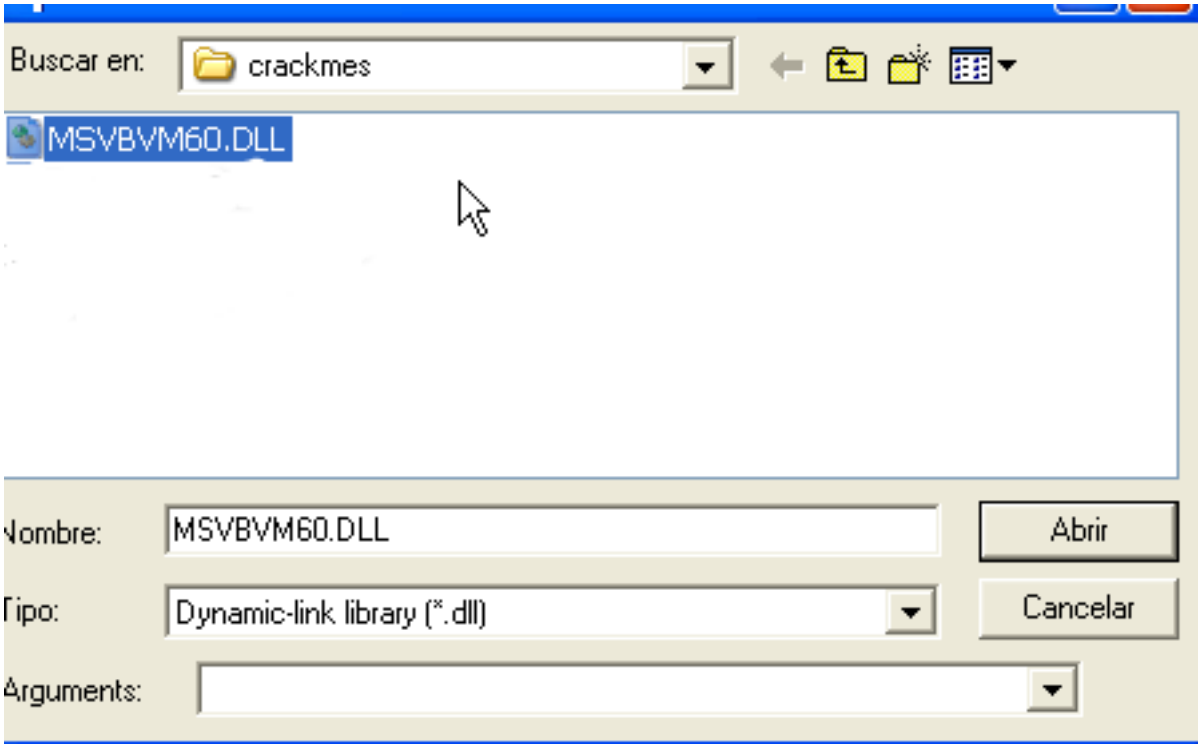
Открываем OllyDbg и идём в меню OPEN.

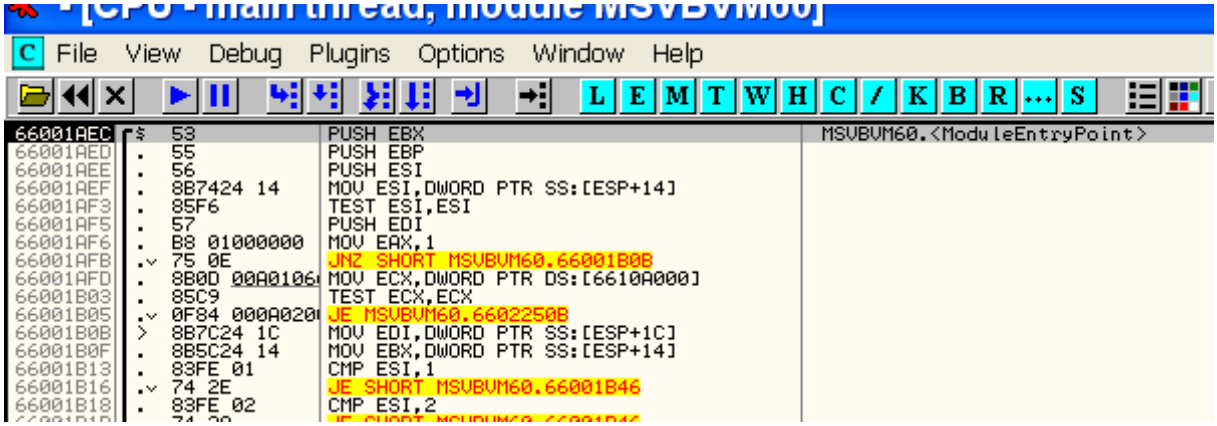


Видим, что по умолчанию стоит отображение EXE-файлов, но сейчас нам нужно открывать DLL, так что меняем соответствующую настройку внизу.



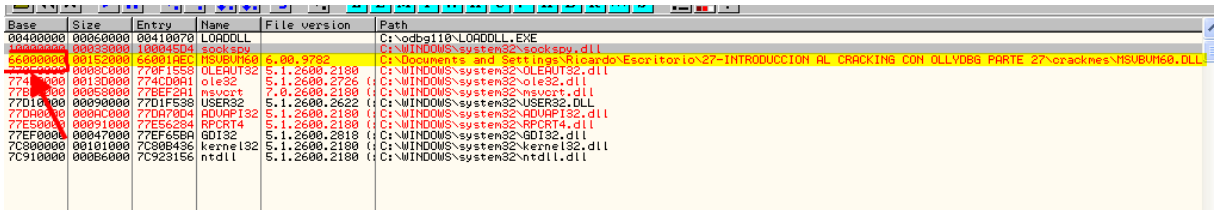
Ставим «Dynamic-link library (*.dll)», что позволит нам открыть DLL Visual Basic’a.





Останавливаемся на точке входа библиотеки.

Ок, сейчас нам нужно найти заголовок, который в данном случае не будет располагаться там, где обычно. Посмотрим, чему равна база образа файла, то есть адрес, откуда он начинается. Нажимаем кнопку E.

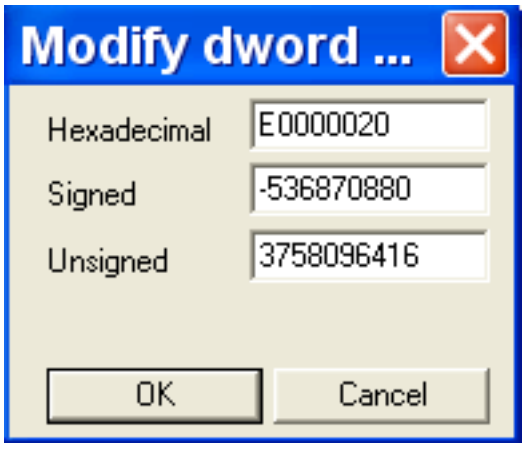


В колонке «Base» мы находим базу образа DLL, которая в моём случае равна 66000000 и может отличаться на вашей машине.

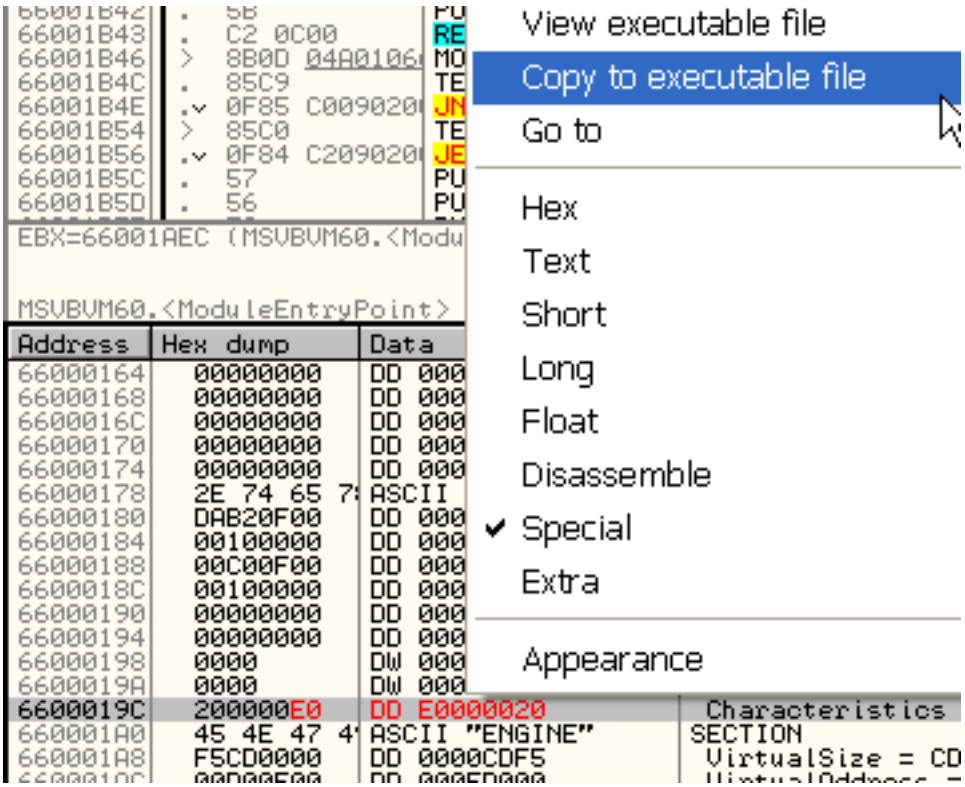
Идём в DUMP и смотрим заголовок.

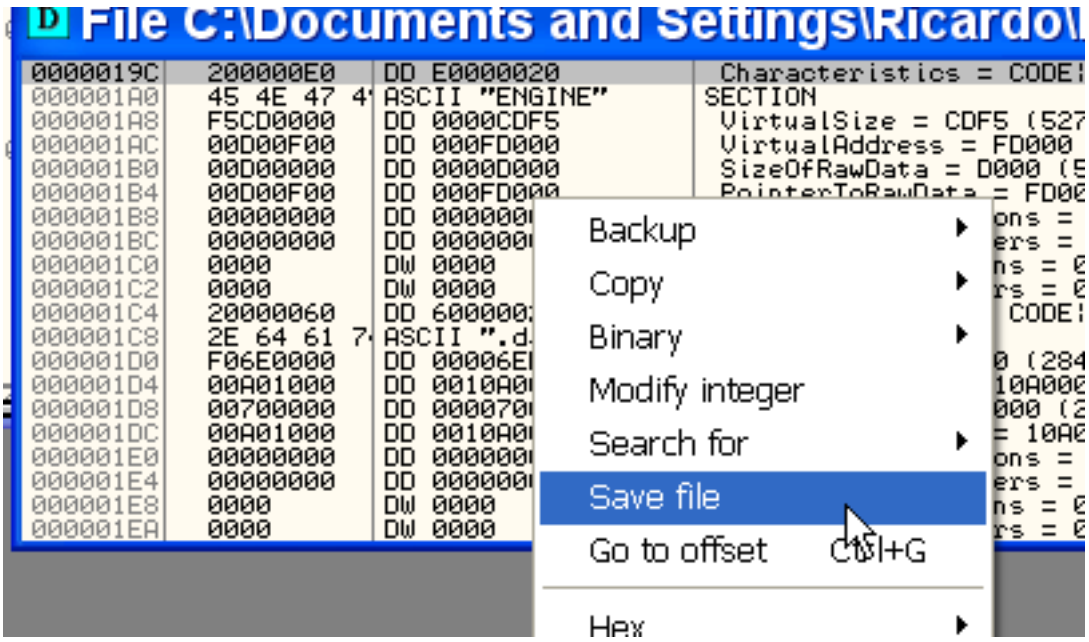
Теперь меняем режим «Special - PE HEADER» согласно процедуре, которую делали в крэкми. Спускаемся и ищем секцию кода.

Меняем на E0000020, чтобы установить разрешение на запись.



Сохраняем изменения.

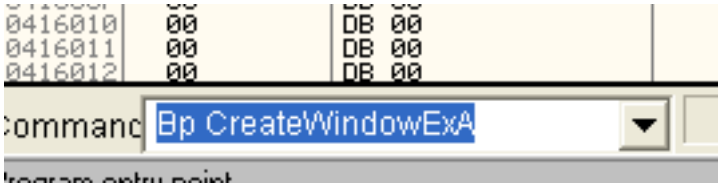




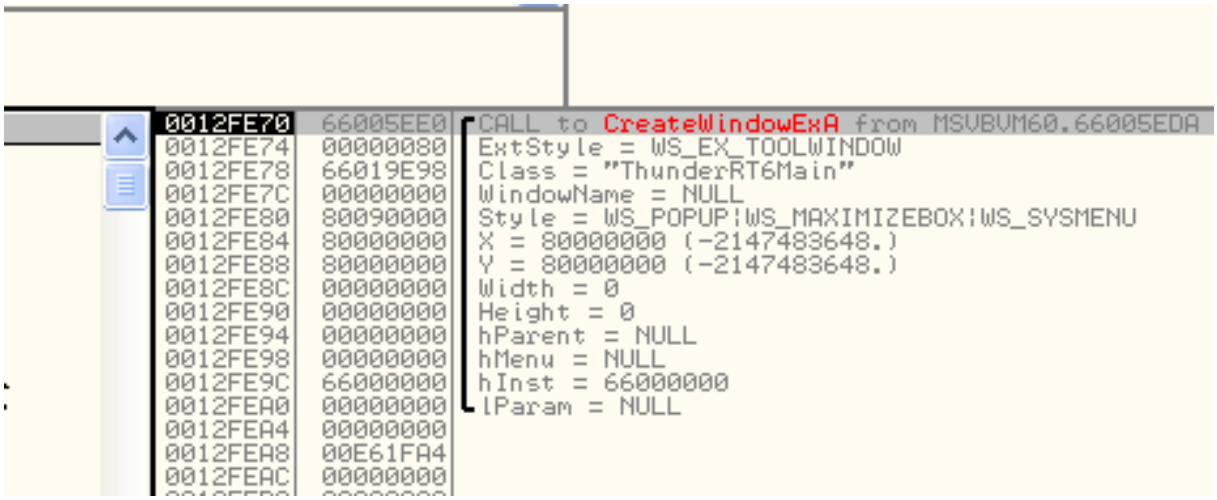
Здесь не меняем имя, а сохраняем под тем же.

Как только у нас есть крэкми и DLL с возможностью писать в неё, до того как делать вставку, покажу в чём заключается идея того, как сделать так, чтобы не появлялось наг-окно.

Открываем в OllyDbg крэкми, которое нам нужно пропатчить, останавливаемся, когда поверх окна ввода серийного номера появляется наг-окно, устанавливаем BP на API-функцию создания окон с помощью команды “Bp CreateWindowExA”.

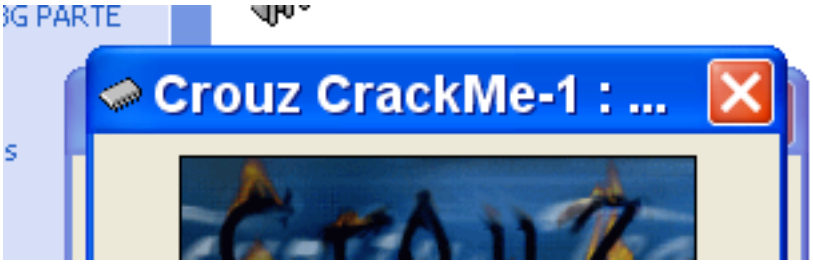


Теперь открываем крэкми.

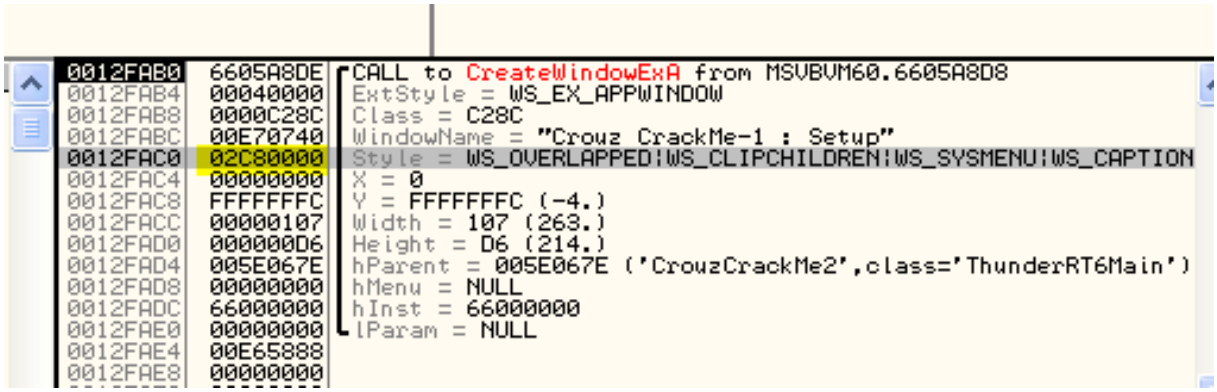


Несколько раз останавливаемся, когда создаются различные окошки и кнопки, используемые крэкми. Прибываем в то место, где создаётся наг-окно.

Тут мы легко обнаружили её по имени окна, т.к. оно совпадает с названием в заголовке окна.



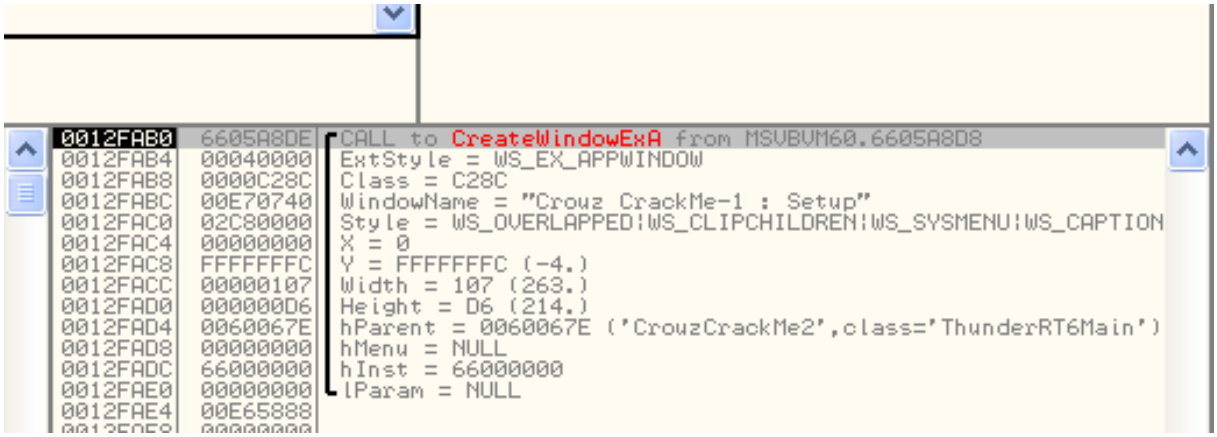
Хорошо, тут создаётся наг, можем здесь поменять его стиль, давайте посмотрим, что произойдёт если поставим 40000000.



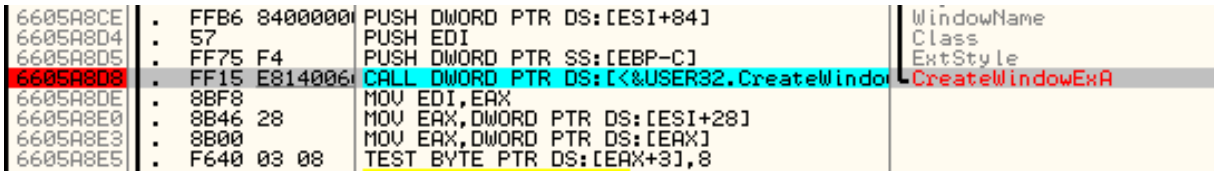
Меняем стиль на WS_CHILD. Вы можете поэкспериментировать с разными значениями. Теперь убираем все BP и делаем RUN.

Видим, что появляется только окно регистрации, а наг – нет. То есть можно добиться цели и не меняя DLL и других программ, как если бы мы боксировали со связанными руками.

Ок, повторяем процедуру до дого момента, как создаётся окно, чтобы вмешаться в API-функцию CreateWindowsExA.

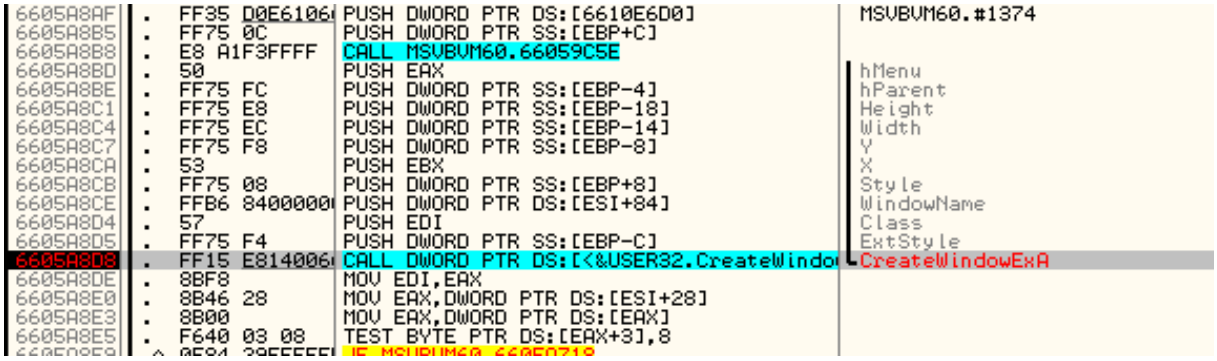


Видим, что API-функция вызывается из DLL Visual Basic'a, смотрим, откуда происходит вызов, информация об этом содержится в первой строке стека. Идём туда.

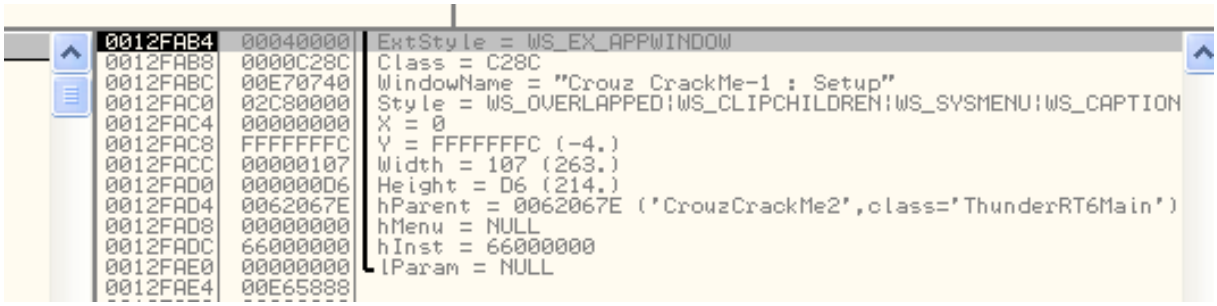


Устанавливаем BP сюда и выходим из API-фукнции.

Теперь посмотрим, когда произойдёт останов. Рестартуем и делаем RUN.



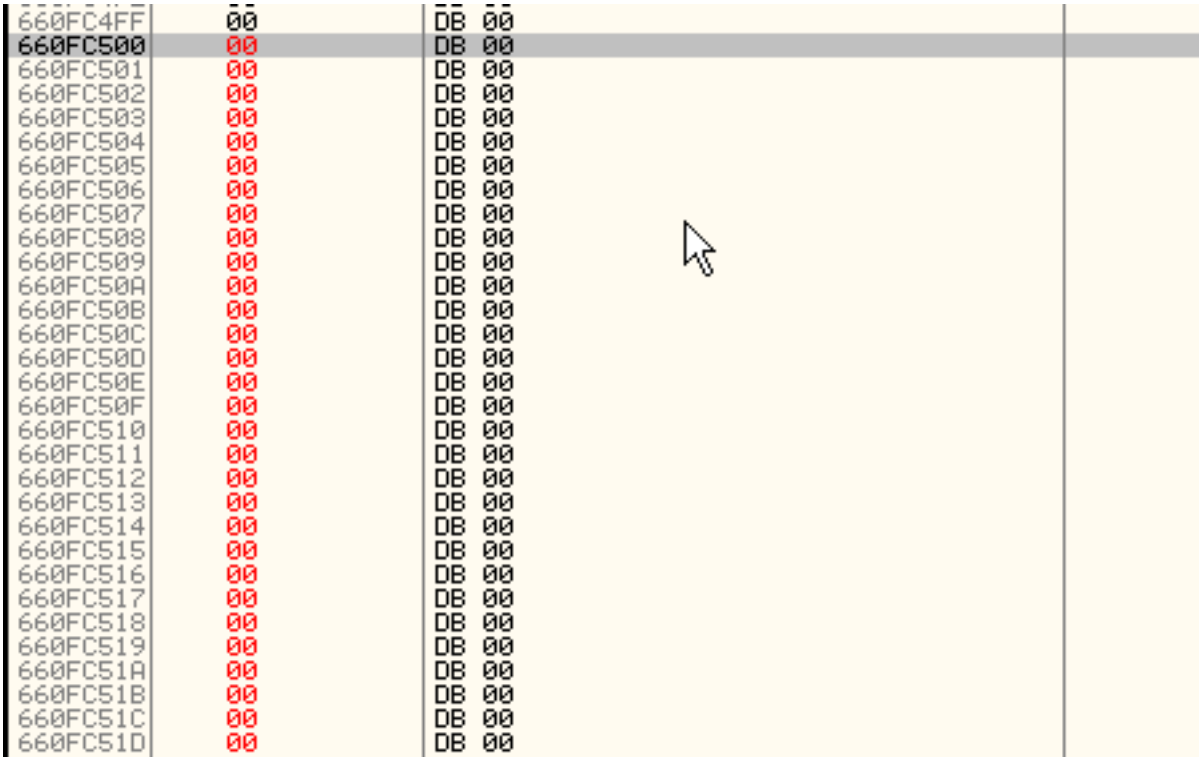
Останавливаемся здесь и смотрим стек.



Видим, что в первый раз останов происходит при создании наг-окна, а так как в последующие разы останова на этой API-функции, вызов происходит не отсюда.

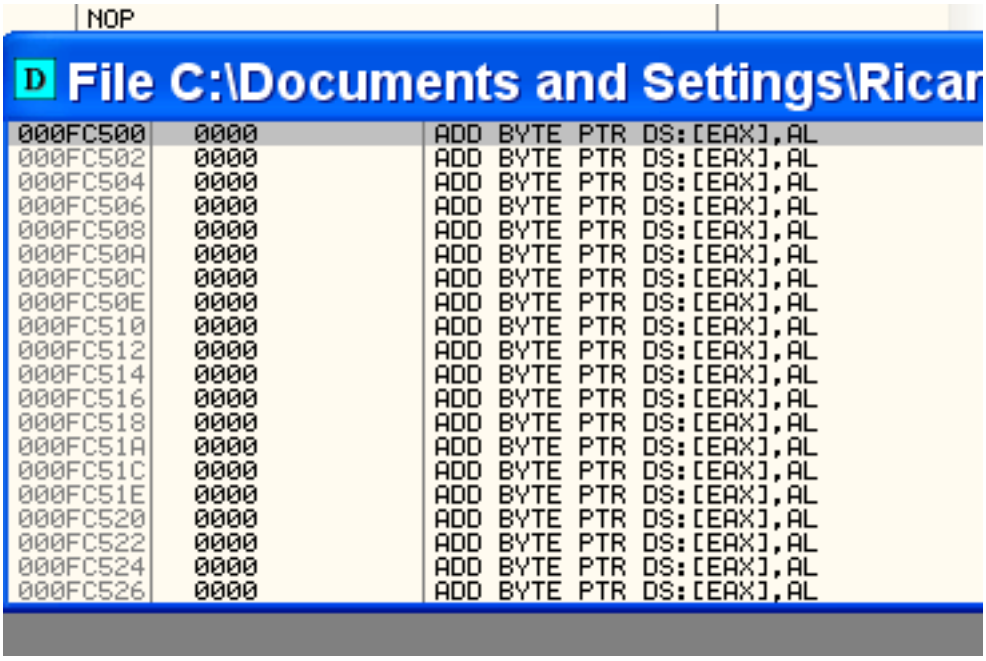
К сожалению, это не единственный раз, когда мы мы здесь проходим, так как когда создаётся окно регистрации, оно тоже делается здесь, так что мы должны быть внимательными, чтобы сделать выборочную вставку.

Вставка начинается с JMP в пустую область, куда можно писать. Если поищем конец секции, то найдём там такую зону.



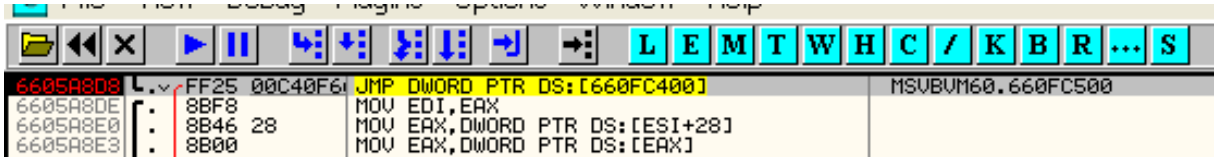
На моей машине такая область находится в конце секции кода, идите туда, где много нулей.

Нажмите правую кнопку мыши – VIEW EXECUTABLE, чтобы посмотреть эту область и подготовиться к сохранению изменений в exe-файле.



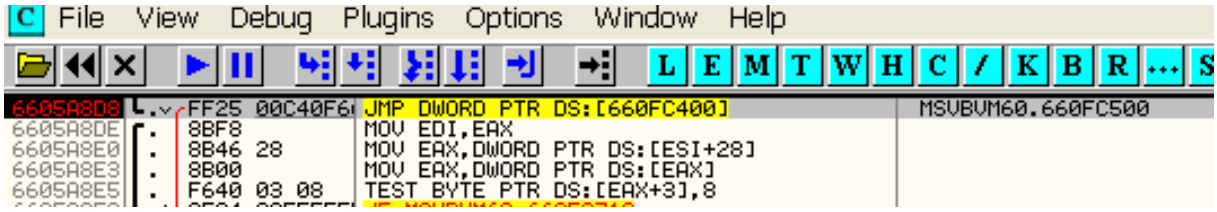
Видим, что область находится в исполняемом файле.

Ок, нам нужно сделать вставку, так что проверяем, можем ли мы сохранить изменения в выбранной нами зоне, так как очень часто бывает, что часть секции со множеством нулей существует только в памяти, а не в исполняемом файле, и в таком случае будет невозможно сохранить изменения в exe-файле (это будет дополнительно объяснено, когда речь пойдёт о распаковке).



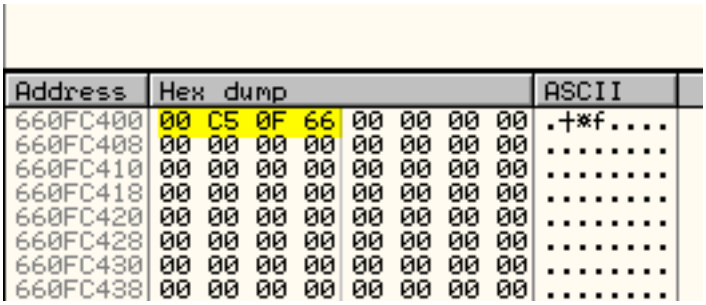
.

Сначала в том месте, где находится вызов API-функции, делаем переход на вставку.



Делаем косвенный переход, так как это делает возможным не модифицировать следующую строку после вызова – “MOV EDI, EAX”. Всегда надо следить, чтобы добавление кода не разрушало инструкций, идущих после. Это обеспечивает нормальное выполнение программы. Если перезапишем какой-либо байт “MOV EDI, EAX”, то после возврата из вставки, получим ошибку.

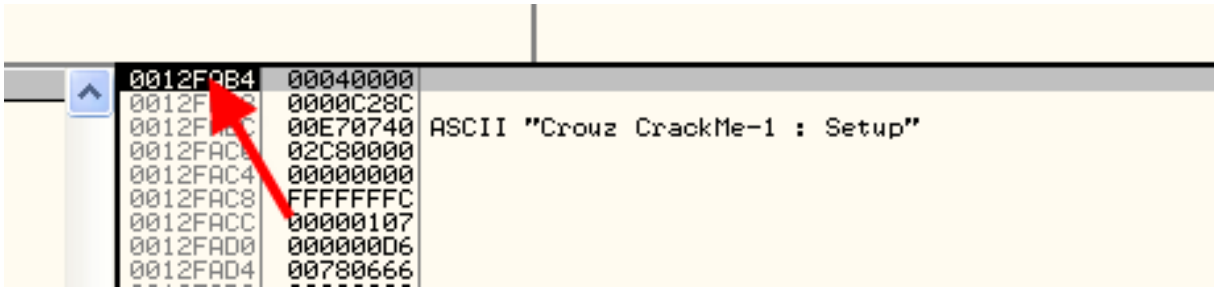
Делаем косвенный переход, который содержит адрес 660fc400, указывающий на место, находящееся чуть ранее моей вставки, где хранится собственно её адрес. Можно было бы сделать прямой переход, но ладно, обе возможности равноценны.



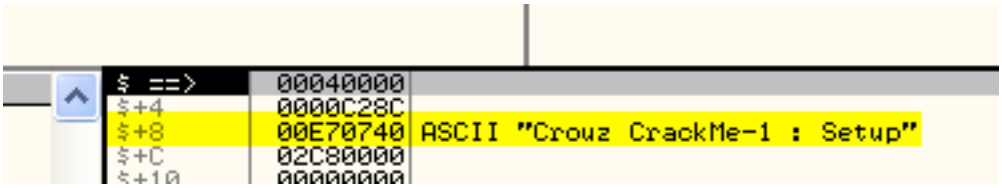
Как видим, здесь находится адрес начала вствки, когда мы дойдёт до JMP, то будет совершён переход по нему.

Конечно, в стеке находятся параметры API-функции, видим имя окна.

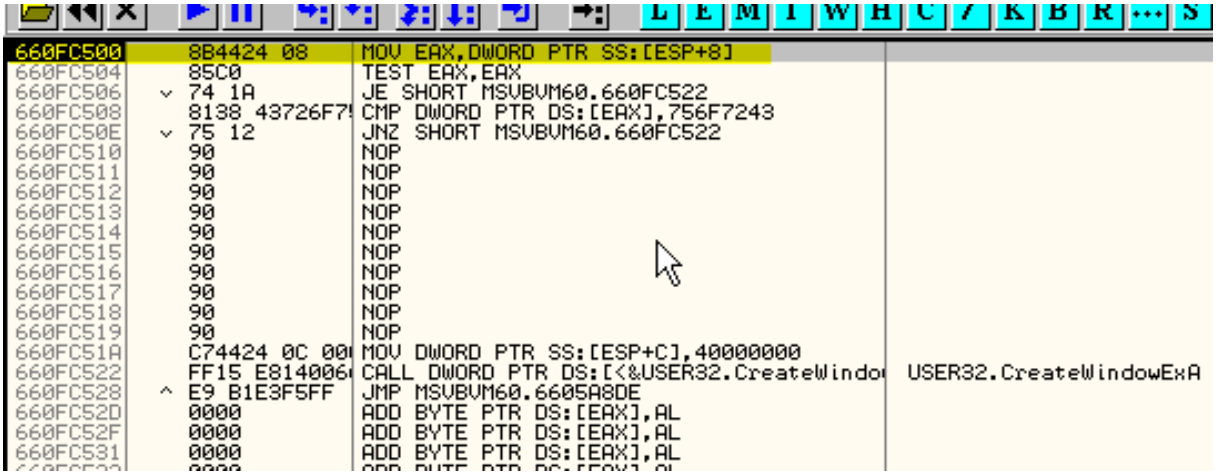
Нам нужно сначала проверить, существует ли такое имя, так как иногда имя может быть равно нулю, и если не проверить, то возникнет ошибка.



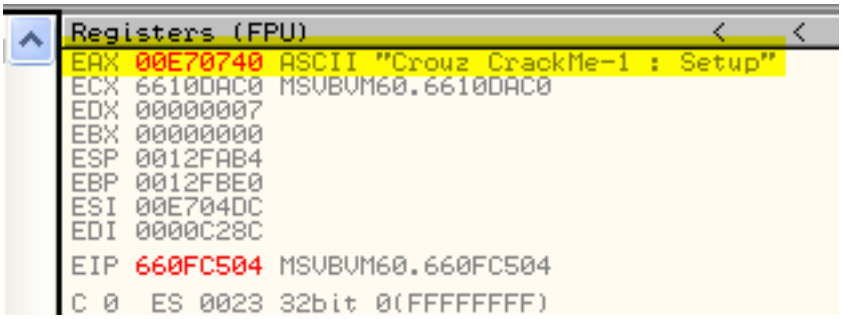
Если сделаем двойной щелчок здесь в стеке, то увидим, что изменилось.



Это значит, что первая строка является ESP, вторая ESP+4 и так далее. Видим то, что нас интересует, находится в ESP+8. Перемещаем значение, находящееся там, в EAX.



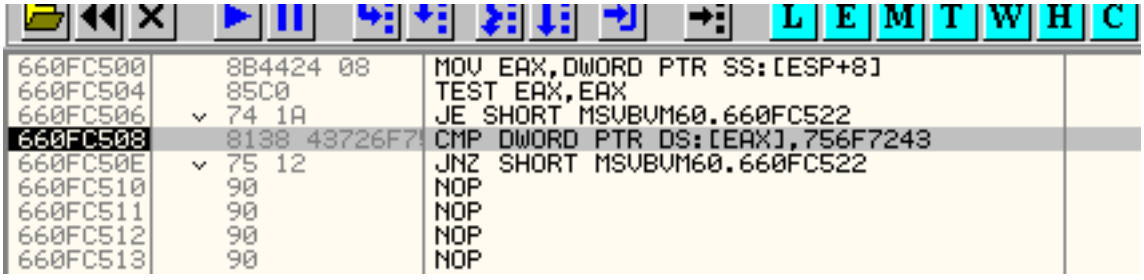
Будем трассировать вставку, чтобы понять, как она работает. После выполнения этой строки в EAX остаётся.



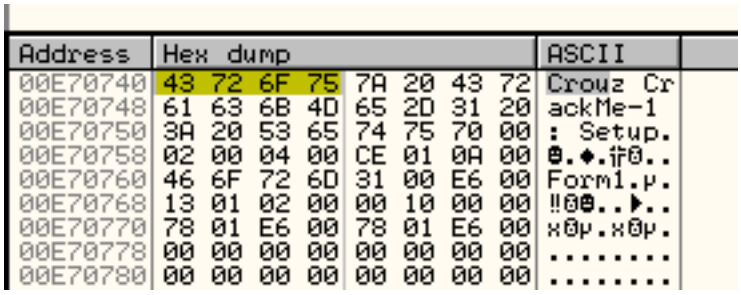
Указатель на строку с названием, затем следующая строка проверяет, равен ли EAX нулю, что может случиться, если окно без названия.

Здесь видим, что если EAX равен нулю, то не надо ничего модифицировать, так что идём прямо к вызову API-функции.

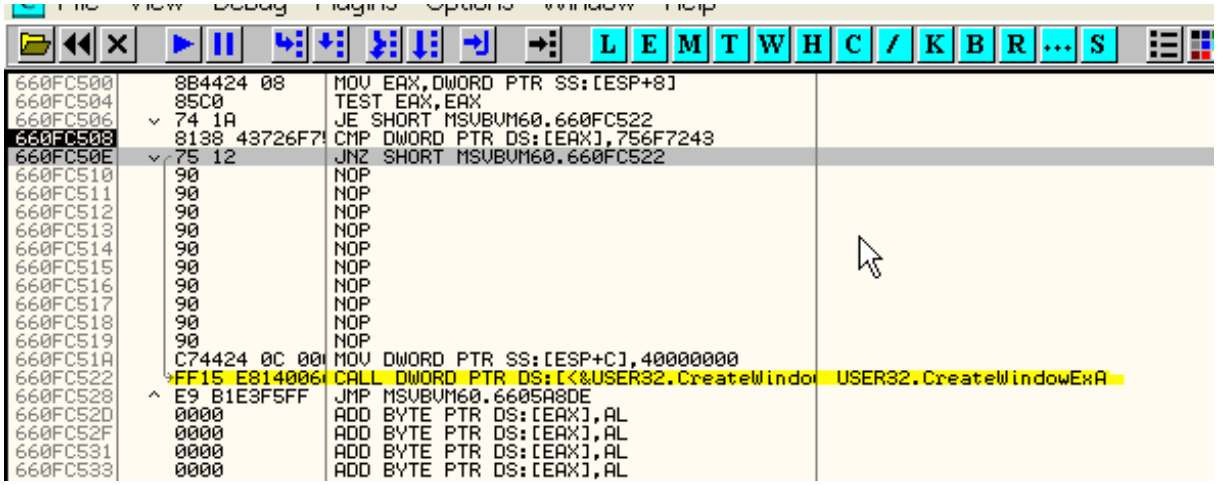
Если EAX не равен нулю, то в этом случае переходим к следующей строка.



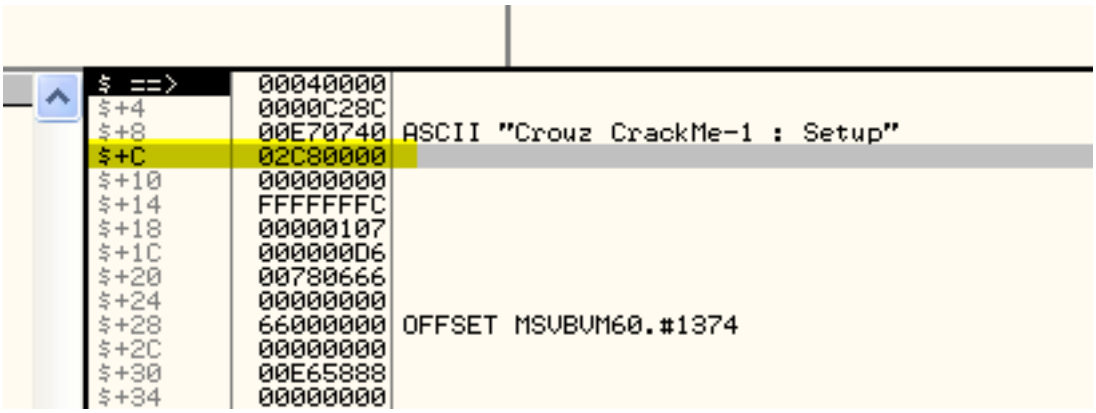
Здесь проверяется, так ли это текстовая строка, что мы ищем, то есть равны ли первые четыре байта названия 756f7243.



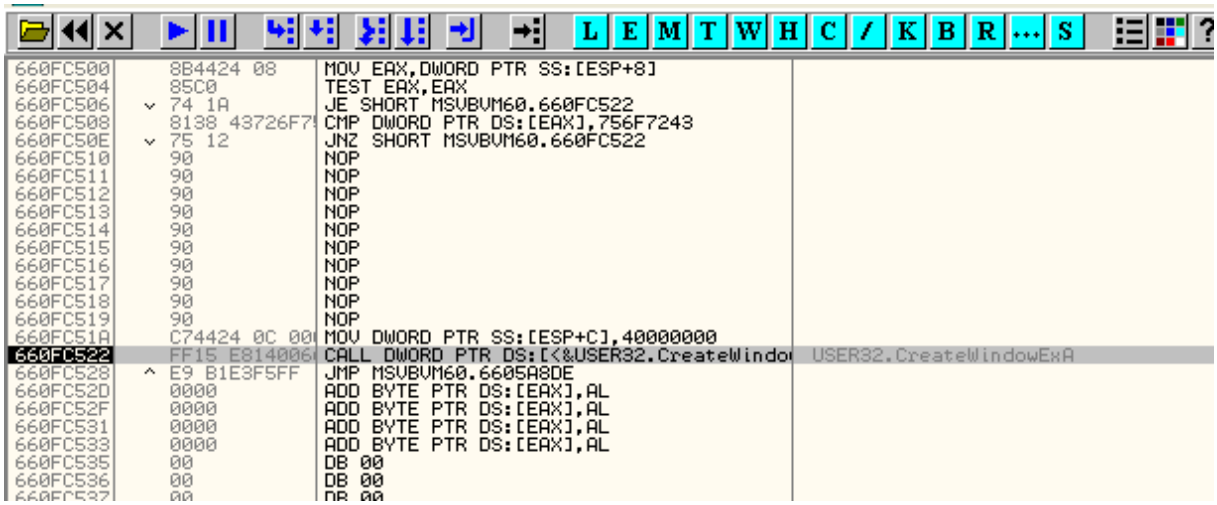
В данном случае равенство верно, то есть должно быть создано наг-окно. Не совершаем переход, а если бы это было бы какое-то другое окно, то перешли бы к API-функций без каких-либо изменений.



Здесь видим, что если не равны, то переходим к API-функции без каких-либо изменений, сейчас равны, то есть это нужное нам окно, поэтому продолжаем и меняем параметр в ESP+C, являющийся стилем окна, на 40000000.



Здесь, после выполнения строки:

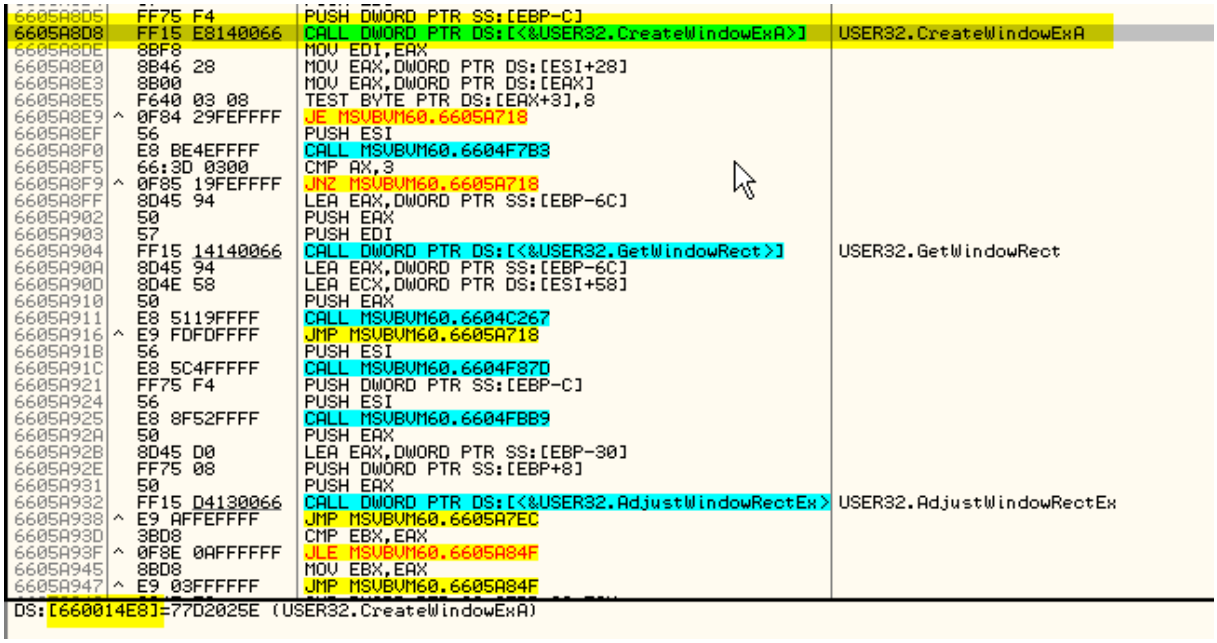


Доходим до API-функции и видим, что параметры следующие:

Vemos que como cambiamos ESP+C por 40000000 ahora el estilo es WS_CHILD como queríamos.

Видим, что изменили ESP+C на 40000000, то есть на стиль WS_CHILD, как и хотели.

Следующая строка – это вызов API-функции, которая должна совершаться в прямом виде, а так как это было в оригинальном вызове.

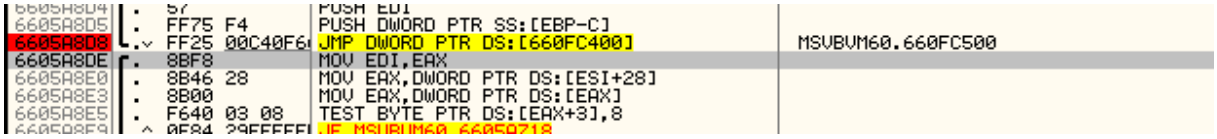


Видим, что это косвенный CALL, считывающий правильный адрес API-функции из 66014e8, для чего оно должно быть туда записано.

CALL [660014e8]

И OllyDbg изменит и поместит имя API-функции, куда совершается переход, однако всегда важно учитывать, что это косвенный вызов, идентичный исходному, работающий на любой машине. Подобные вещи мы рассмотрим глубже в части, посвящённой распаковки и IAT'ам.

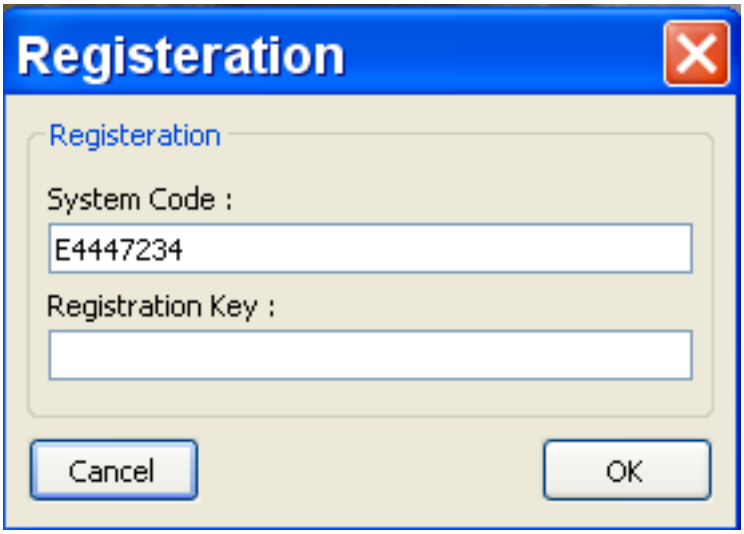
После вызова API-функции возвращаемся на следующую за оригинальным вызовом строку. На моей машине это "MOV EDI, EAX".



Продолжается выполнение программы.

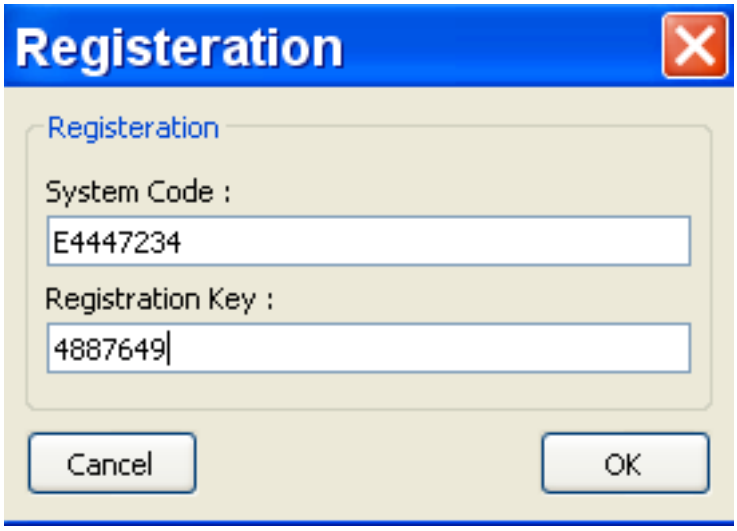
Как видим, то, что нужно сделать – это заменить вызов API-функции на собственную процедуру, которая проверяет окна, и если это наг, то меняет стиль.

Сохраняем все изменения и пробуем.



Видим, что пропатчив крэкми и DLL, избавились от наг-окна. Пробуем правильный серийный номер, выясненный нами.

Вводим его в соответствующем поле.



Нажимаем OK.

Видим, что всё работает превосходно. Не нужно быть ханжами, надо делать то, что проще всего.

Мне хотелось бы, чтобы для практики вы посмотрели, сможете ли вы убрать наг из другого крэкми, которое было приложено к прошлой части, но в этот раз не используя метод 4c.

Файлы к статье (находится в архиве `wasm_files.zip`: `files/recovered/487/ollydbg28.zip`)

Введение в крэкинг с нуля, используя OllyDbg - Глава 29

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-06-14

Файлы к статье (находится в архиве `wasm_files.zip`: `files/0488/ollydbg29.zip`)

Думаю, что в предыдущих главах мы рассмотрели все основные вещи относительно Visual Basic'a, а те, кто хочет изучить эту тему поглубже, могут прочитать посвящённые VB туториалы с cracklatinos (на испанском – прим. пер.), которые укрепят и дополнят ваши знания.

Среди них встречаются очень хорошие туториалы о Visual Basic'е, написанные COCO, которые достаточно сложны и могут стать хорошей практикой, туториалы ARAMPUMK о волшебных точках Visual Basic'a, а также другие прекрасные туториалы, которые открывают дорогу для дальнейшего углубления в данную область. А мы меж тем перейдём к следующей теме, а именно к P-CODE.

Программы на Visual Basic'е могут быть двух типов: NATIVE, который мы рассматривали ранее, и P-CODE (псевдокод). Это то, что мы будем рассматривать в данной главе.

Основная разница заключается в том, что программы в NATIVE-коде выполняют строки кода в секции кода программы, в то время как если мы откроем программу, где используется P-CODE, в OllyDbg, модифицированном на предмет OEP'ов и VB, и установим BPM ON ACCESS в секции CODE, то увидим, что при выполнении не происходит остановок в секции кода, кроме тех случаев, когда встречается вызов какой-либо API-функции. Это очевидным образом указывает на то, что никакого исполняемого кода в данной секции нет.

Дизассемблирование программы, использующей P-CODE, ничем не поможет, так как там нет исполняемого кода. В ней всегда запускается DLL Visual Basic'a, которая читает значения из секции кода, указывающие ей, что нужно сделать. Например:

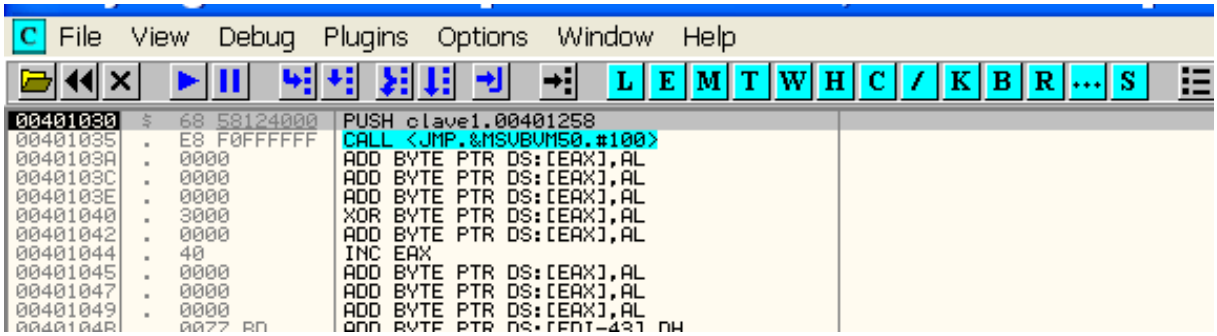
```
1e делает  Безусловный переход
```

1e означает условный переход (похоже, Рикардо не смог определиться, какой же переход он имел в виду – условный или безусловный? – прим. пер.). Он выполняется в DLL Visual Basic'a, то есть, то есть она считывает эти значения из секции кода, и они указывают DLL, что нужно сделать. Таким образом, никакого выполнения кода в соответствующей секции не происходит, только считываются из неё значения.

А сейчас, раз мы такие нахальные, то возьмём нож в руки и решительно атакуем то, за что ещё никто не брался: оттрасируем и интерпретируем действия, выполняемые крэкми с помощью псевдокода, и всё это в OllyDbg, опкод за опкодом, хе-хе.

Рассмотрим первый крэкми под названием clave1, в котором необходимо найти серийный номер. Этот крэкми мы позаимствовали у нашего друга JB DUC'a, написавшего очень хорошие статьи по данной тематике.

Очевидно, что большая часть исследования P-CODE проводится с помощью прекрасного отладчика WKT. Если хотите почитать туториалы с его применением, то поищите те, что были написаны JB DUC'ом, а также в «Новом курсе» от CracksLatinos также есть превосходные статьи о P-CODE. Здесь мы будем использовать OllyDbg и EXDEC, который нам поможет видеть имена опкодов, так как Билл Гейтс не предоставил нам их список, хе.



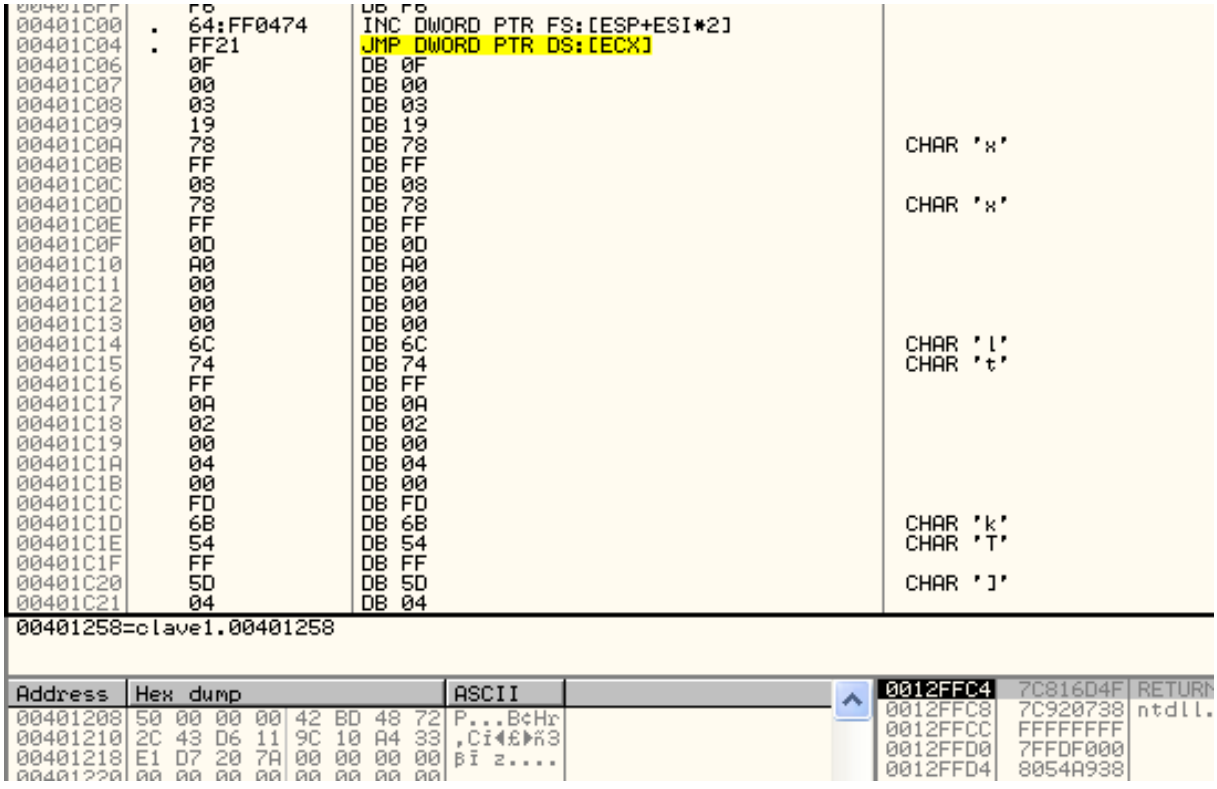
-

Здесь открываем крэкми в обычном непропатченном OllyDbg с установленными плагинами для его сокрытия, которые мы рассматривали в прошлых главах.

Поверхностный осмотр показывает, что как и в NATIVE-крэкми метод 4c может быть применён и здесь, и мы можем найти место, где находятся формы таким же образом, что и в NATIVE-приложениях (4c для снятия наг-окон отлчино работает и в P-CODE, так что можно использовать именно этот метод, когда его возможно применить).

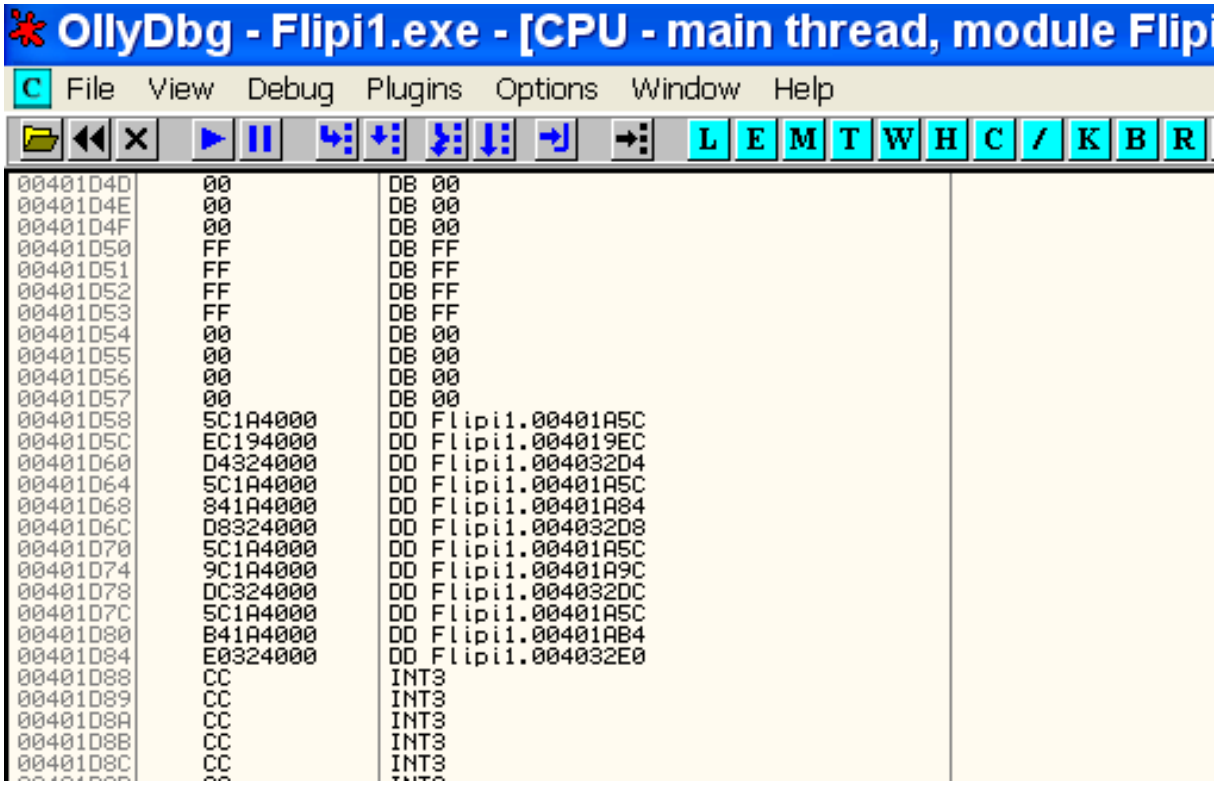
Что мы ещё видим?

Если пойдём вниз от точки входа, то не видим строк с кодом.



-

Только мусор в подобном роде, поэтому не надо пытаться бессмысленно анализировать его, вспомним, что в NATIVE-приложениях на Visual Basic’е мы видели примерно то же самое, если спускали вниз от точки входа.



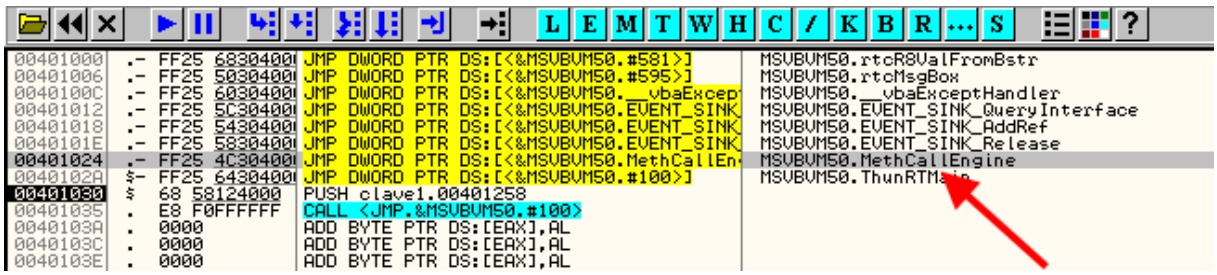
-

Появляется мусор, но если продолжим дальше, то:

-

Встречаем кусок кода довольно большого размера, который идёт до самого конца секции и представляющий собой исполняемый код вместо P-CODE, за исключением некоторых строк, которые, учитывая близлежащие байты, OllyDbg интерпретирует как инструкции, хотя на самом деле это тоже мусор.

Возвращаемся к крэкми с псевдокодом.

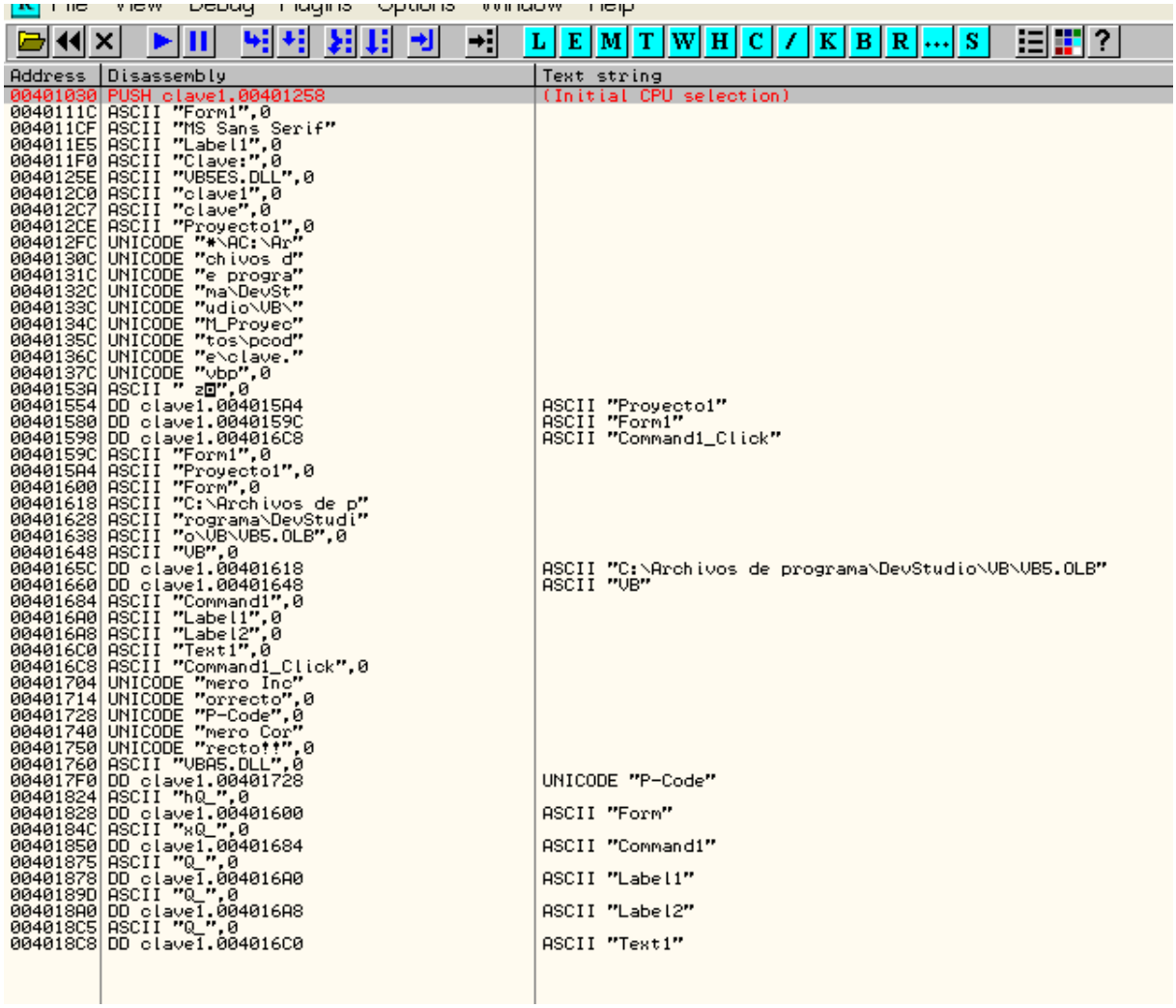


-

Другая особенность – это API-функция под названием MethCallEngine, которую мы встречаем в крэкми, использующих псевдокод, так что теперь, когда нам нужно сделать первый шаг – определить, использует ли программа псевдокод или нет, то уже знаем, что нужно делать: надо посмотреть, есть ли исполняемый код в секции CODE или встречается ли там упомянутая нами функция.

Первое, что нам стоит сделать – это посмотреть, найдём ли мы какие-нибудь строки.

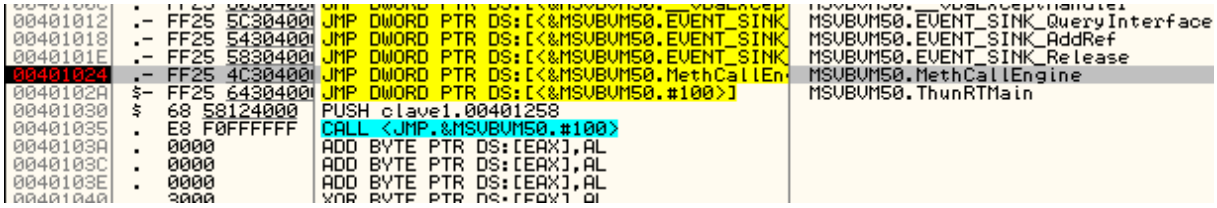
-



-

Это не слишком нам поможет, хотя, возможно, поможет что-нибудь другое.

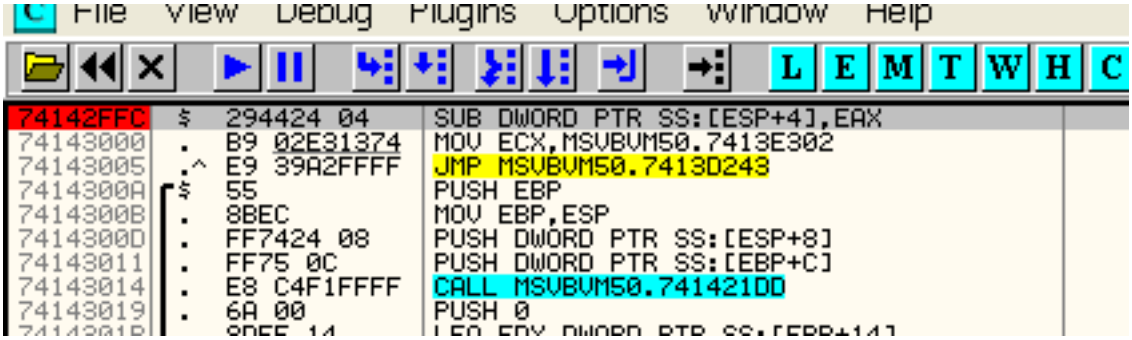
Хорошо, установим BP прямо на JMP, который является прямым переходом к функции MethCallEngine,



-

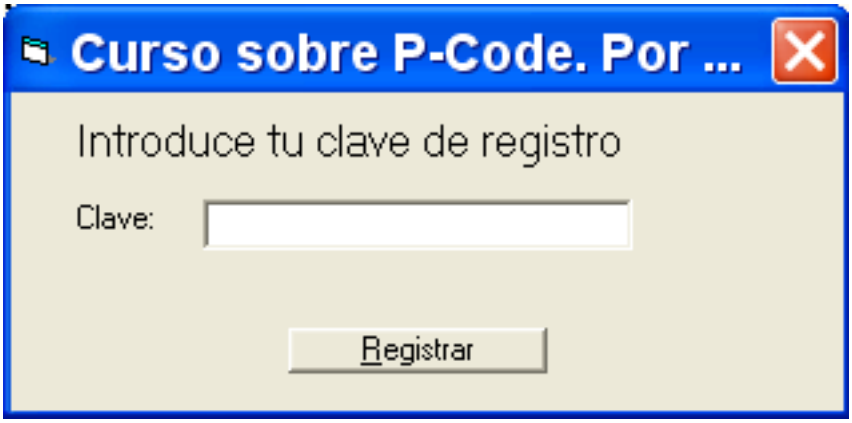
Ищем сверху от точки входа и быстро встречаем JMP на функцию MethCallEngine и, находясь прямо на этой строке, делаем правый клик мышью и выбираем FOLLOW, что приведёт нас прямо к данной функции, где и устанавливаем BP.

-



-

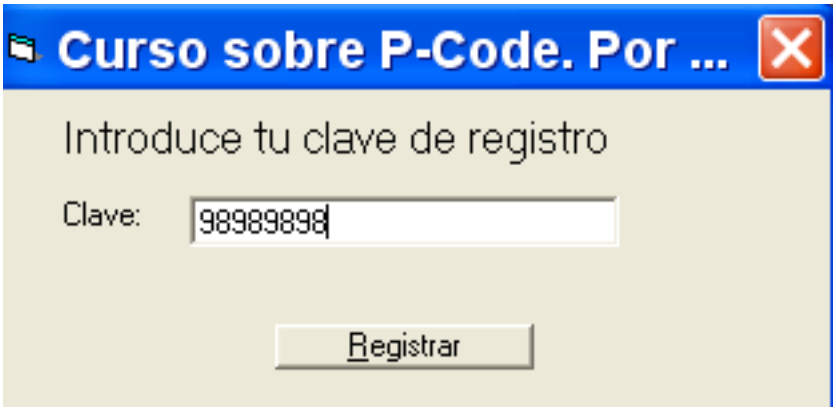
Теперь делаем RUN.



-

Видим, что окно для ввода серийного номера появилось до остановки на вышеуказанной API-функции, что логично, так как создание окна и всё, что с этим связано, происходит таким же образом, как и в VB-приложениях, использующих «настоящий» код.

Теперь вводим ложный серийный номер.

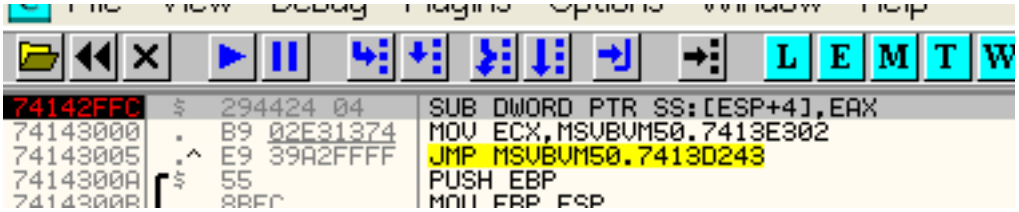


-

И нажимаем «REGISTRAR».

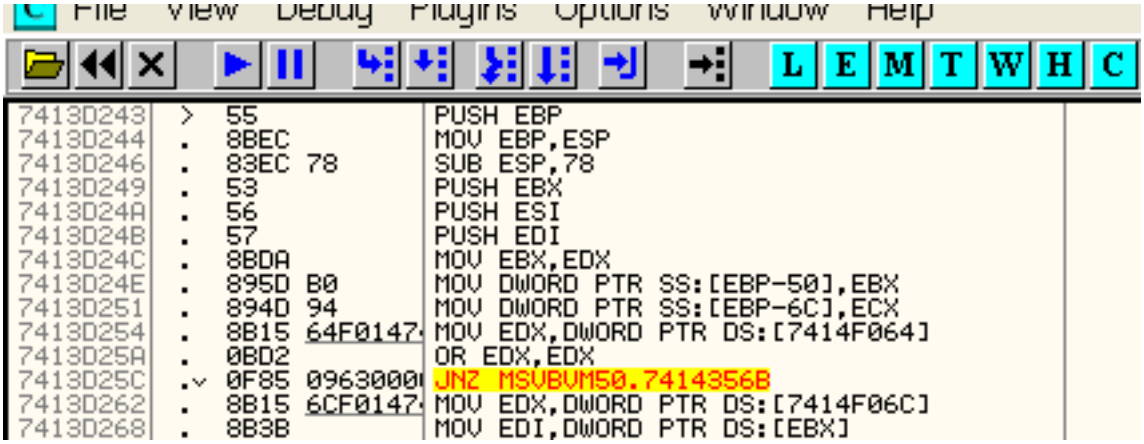
-

Останавливаемся на JMP, который начинает истинную часть P-CODE.



-

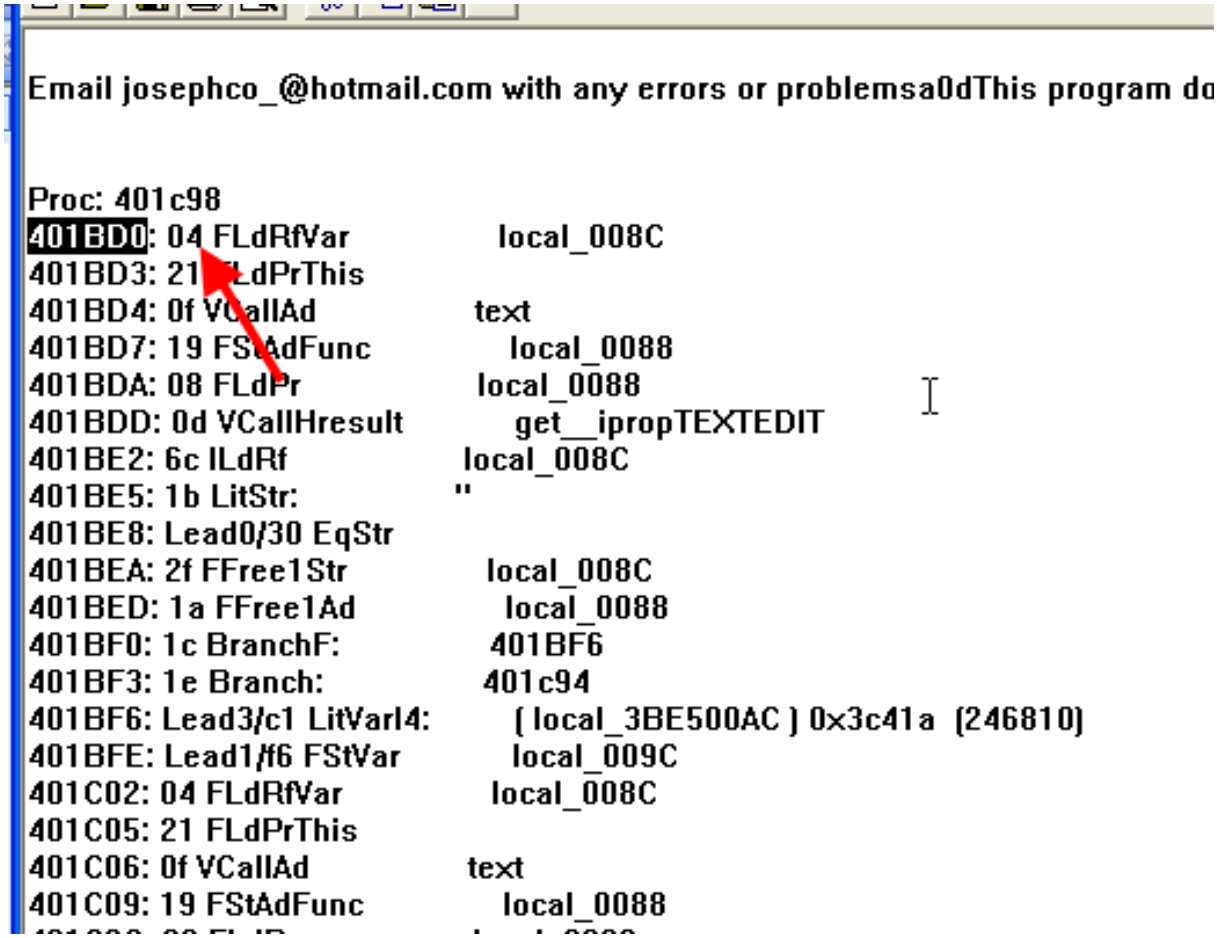
Здесь входим в API-функцию, посмотрим, что она будет делать.



-

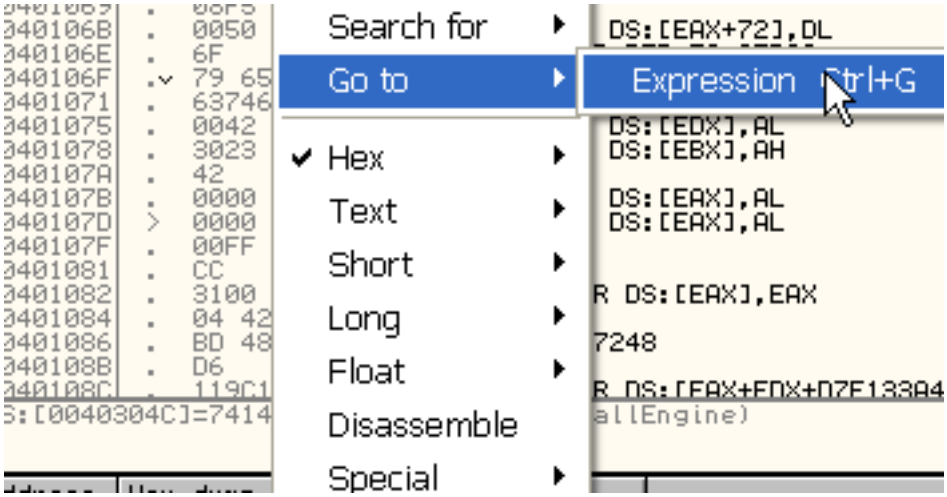
Здесь начинается.

Теперь, если откроем то же крэкми с помощью exdec, являющегося дизассемблером P-CODE, чтобы немного помочь себе, то увидим следующее:



-

То есть первый байт, который будет прочтён – это 04, находящийся по адресу 401BD0. Он находится не очень далеко отсюда, так что установим на него BPM ON ACCESS.



-

-

-

Этот первый байт, который будет прочтён. Когда остановимся, то окажемся в начале, и таким образом, мы можем оказаться там без помощи EXDEC. Как только остановимся на BP, установленном на API-функции MethCallEngine, мы можем поместить BPM ON ACCESS на секцию

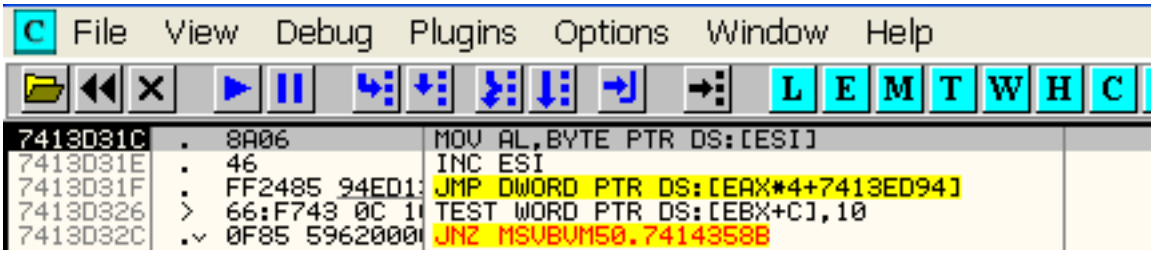
кода.

-

И делаем RUN. Видим, что останов происходит несколько раз.

-

Но только в одном из них происходит чтение содержимого ESI, в котором находится адрес, указывающий на вышеуказанный байт. Это место мы быстро находим после нескольких остановок (на моей машине их было точно 10).

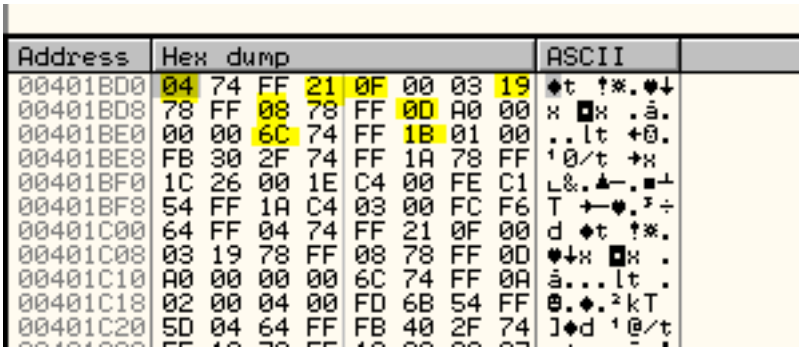


-

В первый раз, когда происходит останов и байт из [ESI] считывается и перемещается в AL – это то место, где начинается чтение первого опкода P-CODE. Данным образом можно найти первый байт, не используя EXDEC.

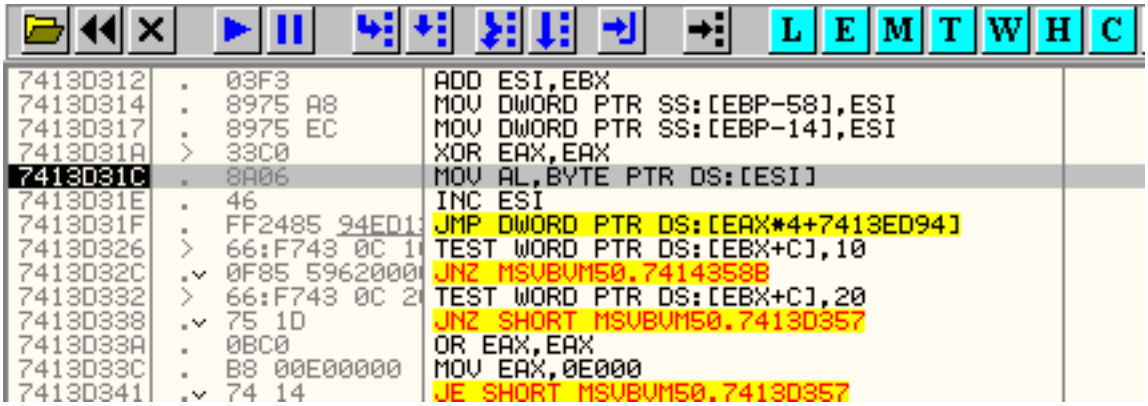
Как видим, последующие опкоды, которые отображаются EXDEC'ом идут вслед за предыдущим.

-



-

Как видим, порядок опкодов на обоих картинках соответствует друг другу. Между ними располагаются параметры, которые необходимы опкоду для выполнения.

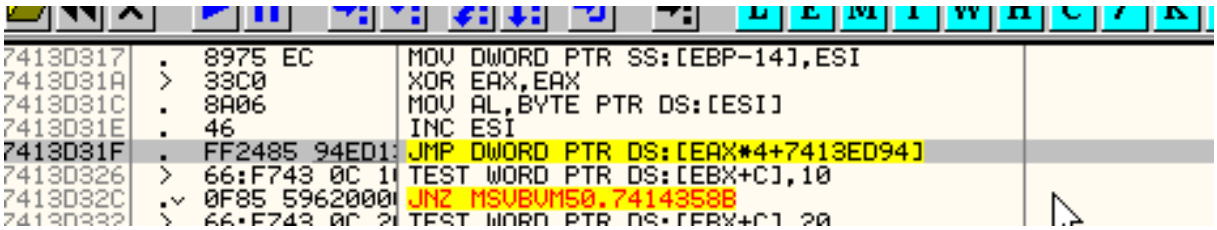


-

Как видим, здесь читается первый байт.

-

На который, как видим, указывает ESI. На следующей строке никаких действий по выполнению опкода не предпринимается. Значение ESI увеличивается на 1, чтобы затем прочитать параметры опкода.



-

Затем наконец доходим до косвенного JMP, который отправляет нас к строкам, выполняющим опкода, в данном случае это 04, как видим в EXDEC'e.

```
401BD0: 04 FLdRfVar              local_008C
```

Видим, что такого загадочного делает этого опкод.

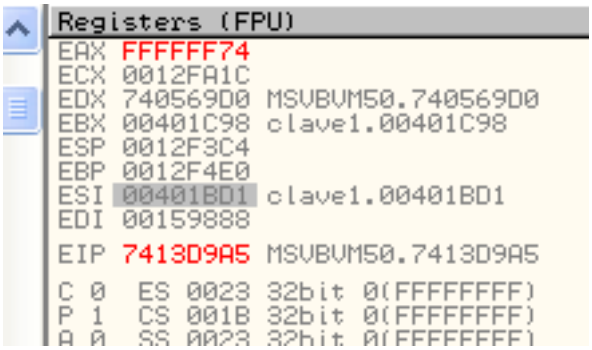
-

Здесь видим выполнение опкода 04 FLdRfVar, это несколько маленьких строк кода, ничего такого, чтобы испугаться, хе-хе, и в конце видим, что заканчивается XOR EAX, EAX – это подготовка к чтению следующего опкода.

Первое, что делается – это берутся параметры опкода, которыми являются два байта, следующих за ним.

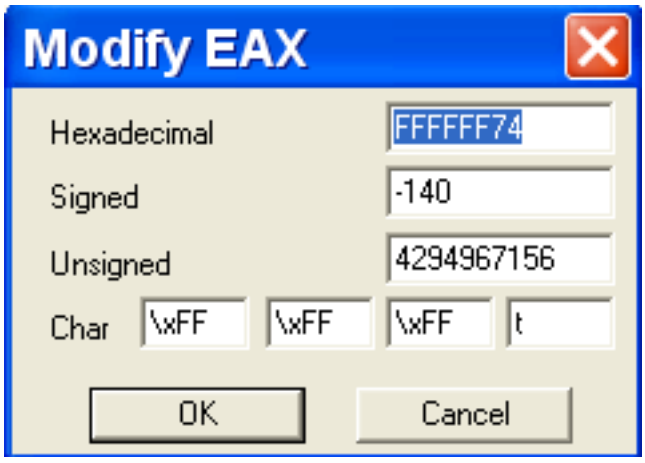
-

Хорошо, они помещаются в EAX с помощью инструкции MOVSX, и значение FF74 является отрицательным (мы рассматривали это в главах, посвящённым ассемблеру). Продолжаем трассировать.



-

Это значение в EAX равно -8c, и если кликнем по нему два раза:



-

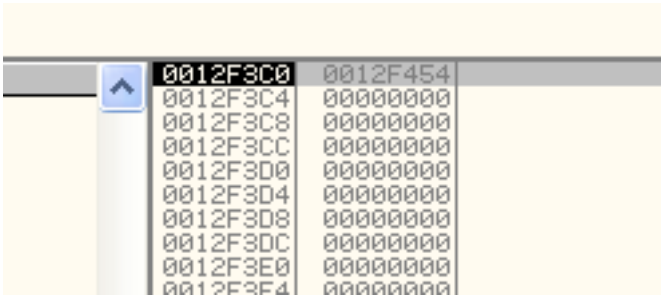
В окне показывается, что это значение равно -140 в десятеричной системе счисления или -8c в шестнадцатеричной. В EXDEX нам показывается 8c.

```
401BD0: 04 FLdRfVar             local_008C
```

На следующей строке значение, считанное из параметров опкода, суммируется с EBP.

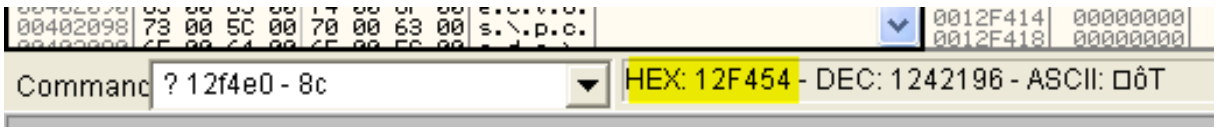
-

И к этому значению применяется PUSH.



-

То есть это эквивалент PUSH EBP-8c – выделение в стеке места для локальной переменной. На моей машине EBP равен 12f4e0, если отнимем 8c, то получится 12f454, то есть значение, которое останется в EAX и будет передано инструкции PUSH.



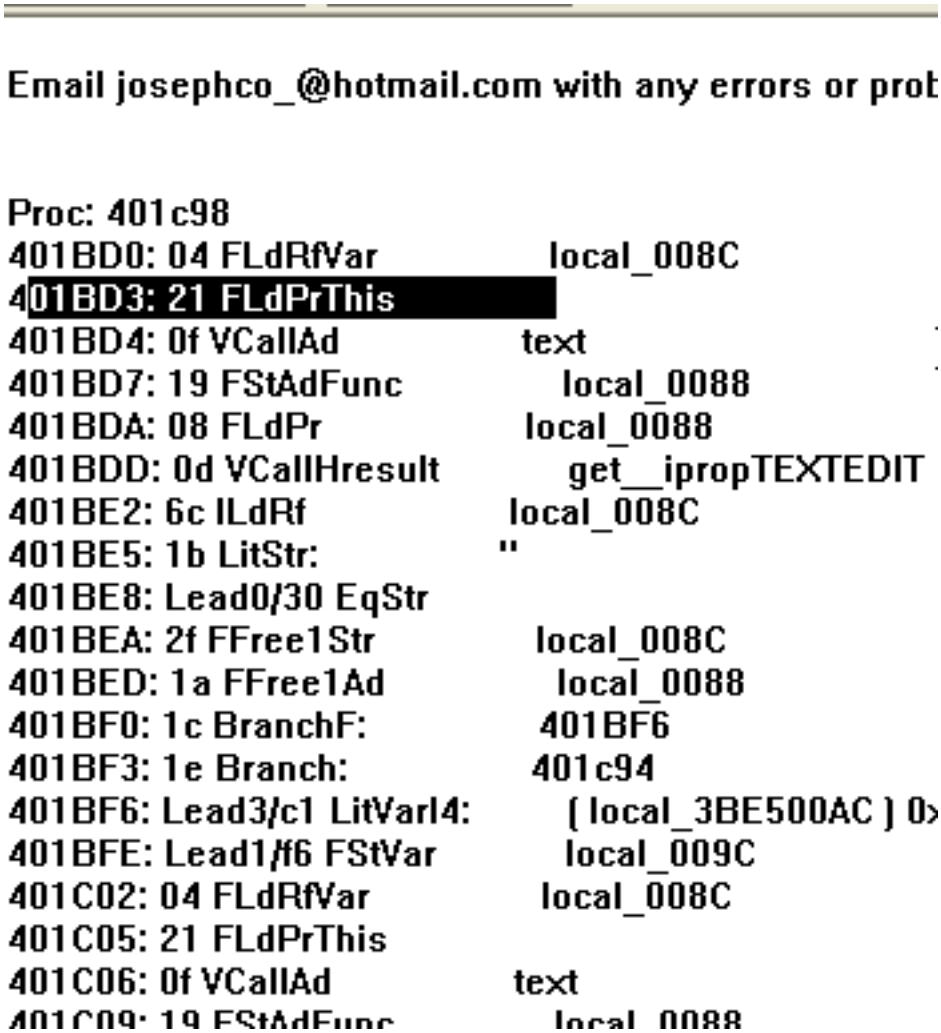
-

-

Ничего хорошего. Продолжаем.

-

Видим, что далее в EAX помещается ноль, что означает завершение проводимых операций с данным опкодом и инициализацию регистров для чтения следующего. На следующей строке, действительно, читается следующий опкод.



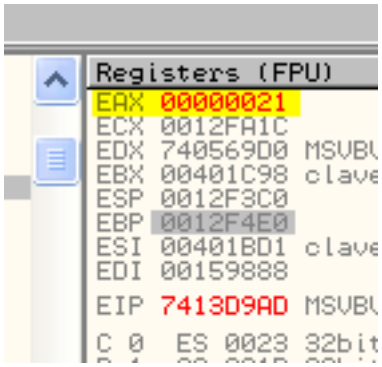
-

Второй опкод – это 21.

Видим его тут.

```
7413D9A7  .  50            PUSH EAX
7413D9A8  .  33C0          XOR EAX,EAX
7413D9AA  .  8A46 02       MOV AL,BYTE PTR DS:[ESI+2]
7413D9AD  .  83C6 03       ADD ESI,3
7413D9B0  .  FF2485 94ED1  JMP DWORD PTR DS:[EAX*4+7413ED94]
7413D9B7  >  8B45 08       MOV EAX,DWORD PTR SS:[EBP+8]
7413D9BA  .  8945 B4       MOV DWORD PTR SS:[EBP-4C],EAX
7413D9BD  .  33C0          XOR EAX,EAX
7413D9BF  .  8A06          MOV AL,BYTE PTR DS:[ESI]
7413D9C1  .  46            INC ESI
7413D9C2  .  FF2485 94ED1  JMP DWORD PTR DS:[EAX*4+7413ED94]
7413D9C9  >  0FBF06        MOVSX EAX,WORD PTR DS:[ESI]
7413D9CC  .  5B            POP EBX
7413D9CD  .  66:891C28     MOV WORD PTR DS:[EAX+EBP],BX
7413D9D1  .  33C0          XOR EAX,EAX
7413D9D3  .  8A46 02       MOV AL,BYTE PTR DS:[ESI+2]
7413D9D6  .  83C6 03       ADD ESI,3
7413D9D9  .  FF2485 94ED1  JMP DWORD PTR DS:[EAX*4+7413ED94]
7413D9E0  >  0FBF06        MOVSX EAX,WORD PTR DS:[ESI]
7413D9E3  .  8F0428        POP DWORD PTR DS:[EAX+EBP]
7413D9E6  .  33C0          XOR EAX,EAX
7413D9E8  .  8A46 02       MOV AL,BYTE PTR DS:[ESI+2]
7413D9EB  .  83C6 03       ADD ESI,3
7413D9EE  .  FF2485 94ED1  JMP DWORD PTR DS:[EAX*4+7413ED94]
7413D9F5  >  0FBF06        MOVSX EAX,WORD PTR DS:[ESI]
7413D9F8  .  D91C28        FSTP DWORD PTR DS:[EAX+EBP]
7413D9FB  .  DFE0          FSTSW AX
7413D9FD  .  A8 0D         TEST AL,0D
7413D9FF  .v 0F85 A1C7000  JNZ MSVBVM50.7414A1A6
7413DA05  .  33C0          XOR EAX,EAX
7413DA07  .  8A46 02       MOV AL,BYTE PTR DS:[ESI+2]
7413DA0A  .  83C6 03       ADD ESI,3
7413DA0D  .  FF2485 94ED1  JMP DWORD PTR DS:[EAX*4+7413ED94]
7413DA14  >  8B3C24        MOV EDI,DWORD PTR SS:[ESP]
7413DA17  .  66:8B07       MOV AX,WORD PTR DS:[EDI]
7413DA1A  >  80E4 BF       AND AH,0BF
7413DA1D  .  66:83F8 09    CMP AX,9
7413DA21  .v 0F85 C06A000  JNZ MSVBVM50.741444E7
7413DA27  >  33C0          XOR EAX,EAX
7413DA29     8006          MOV AL,BYTE PTR DS:[ESI]
DS:[00401BD3]=21 ('!')
AL=00

Address | Hex dump | ASCII | 0012F3C0
```

-



-

Как обычно, он помещается в AL.

-

А теперь к значению в ESI прибавляется 3, чтобы регистр указывал на параметры опкода. Затем управление переходит к косвенному JMP, который ведёт на опкод 21.

Посмотрим, что нам скажет гугл.

```
'21, FLdPrThis
(Загрузить ссылочный указатель в указатель на элемент данных.)
```

Хорошо, у нас здесь несколько указателей, про которые мы не знаем, для чего они служат, но примерно дело состоит в том, что берётся ссылочный указатель и загружается в указатель на элемент данных. В ассемблере это было бы чтение указателя из стека и загрузка его в другое место стека же.

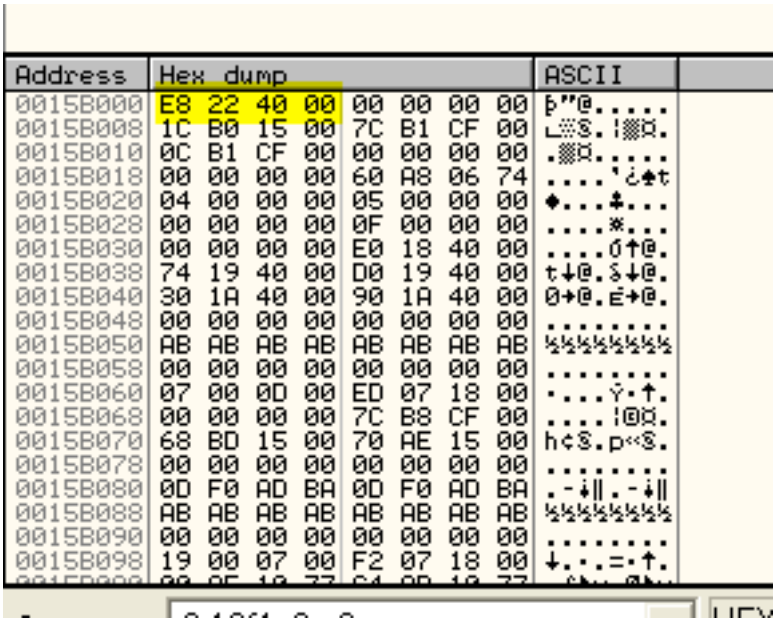
Если продолжим трассировать:

-

Видим, что читается содержимое EBP+8 (ссылочный указатель) и сохраняется в в EBP-4c (указатель на элемент данных).

Хорошо, прочитанное значение на моей машине равно 15b000. Если посмотрим в DUMP'e:

-



-

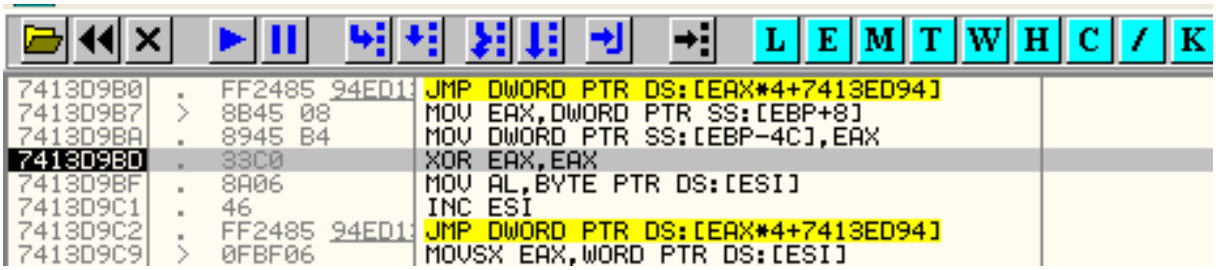
Видим, что здесь находится указатель на 4022e8, а если посмотрим в DUMP, то увидим:

-

Видим, что здесь начинается таблица, так что 15b000 – это ссылочный указатель, указывающий на что-то, что находится в таблице, хотя это не очень поможет нам для крэкинга, всегда хорошо постараться расшифровать, что происходит в программе.

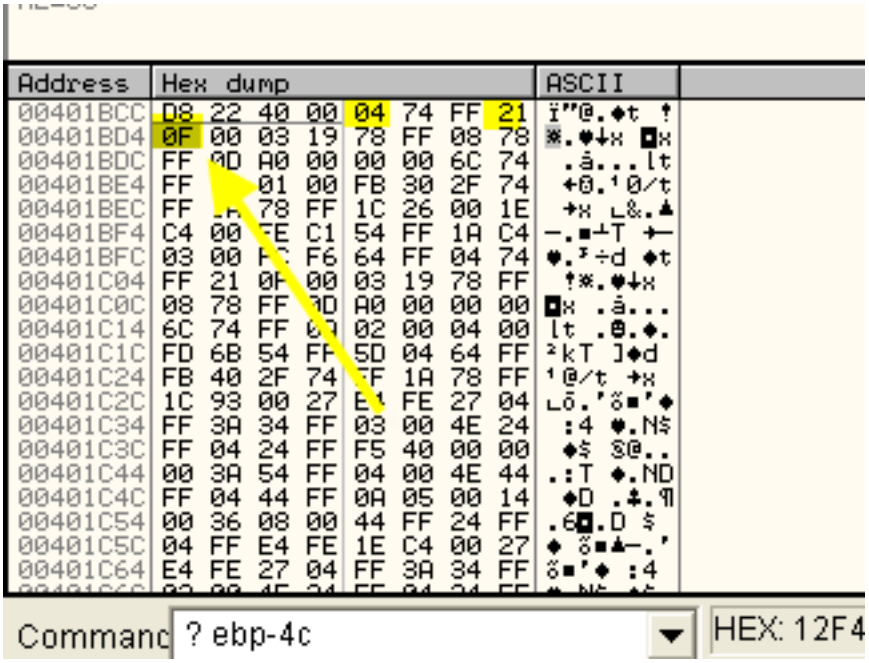
В остальном, это команда без параметров, поэтому никаких вариаций у неё нет. При исполнении она всегда читает ссылочный указатель и сохраняет его в указатель на элемент данных.

Хорошо, продолжаем.



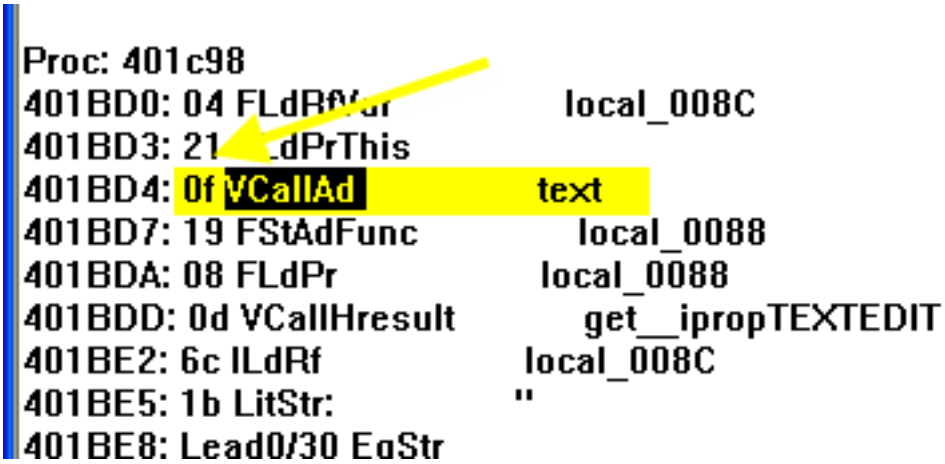
-

Далее обнуляется EAX, а на следующей строке читается третий опкод.



-

Смотрим в EXDEC, что такое 0F.

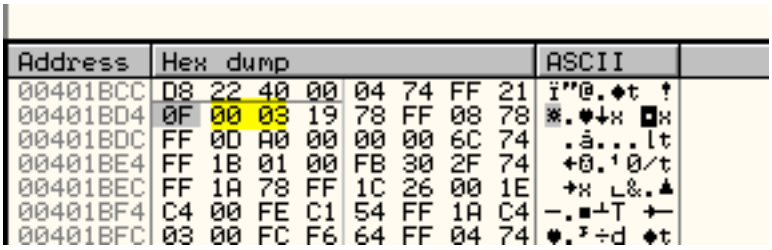


-

VcallAd

-

Ок, здесь видим, в чём заключается смысл опкода 0F, видим, что у него один параметр из двух байтов.

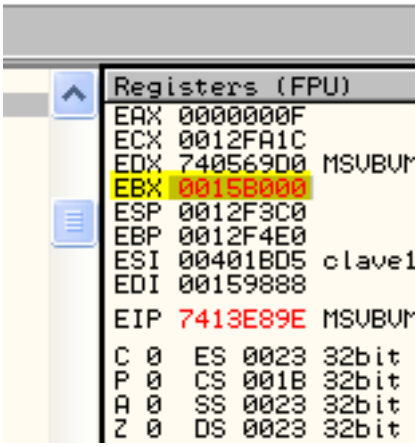


-

Параметр в моём случае – это 0300, и он указывает смещение в таблице дескрипторов элементов данных. Хм, видим, что доходим до косвенного JMP, посмотрим, совпадёт ли это с тем, что говорится.

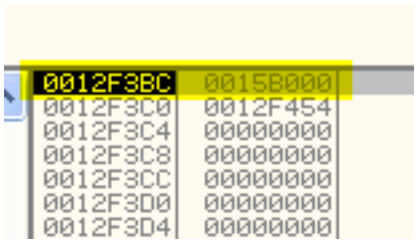
-

Здесь читается содержимое EBP-4c и помещается в EBX.



-

Здесь располагается указанное значение 15b000. Оно помещается в стек.



-

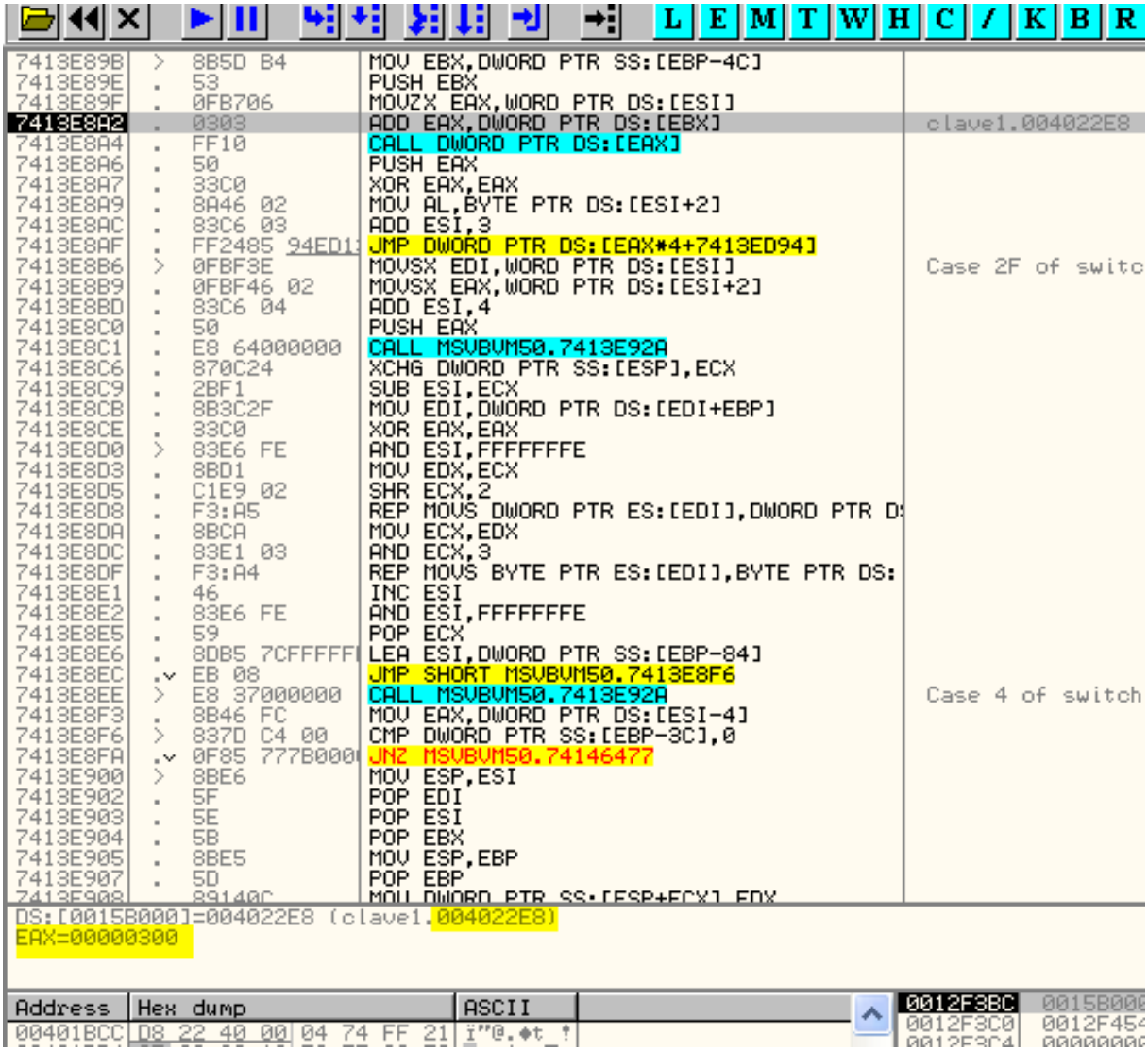
-

Затем читаем параметры. В моём случае, это 300.

-

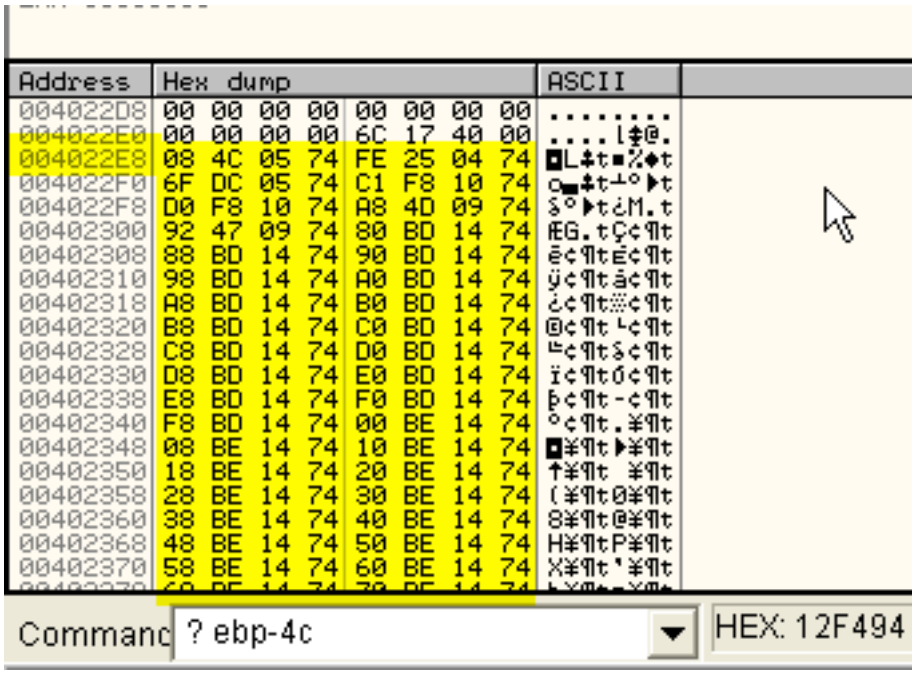
-

Суть дела состоит в том, что содержимое EBX, равное 15b000, это начало таблички, которую мы ищем, то есть таблица дескрипторов элементов данных. Адрес – 4022e8.



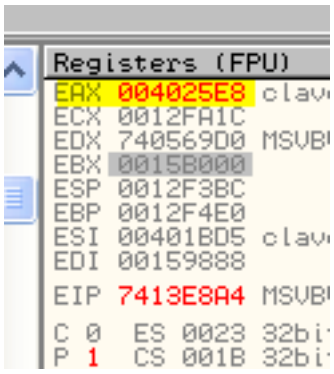
-

И к этом прибавляем 300, то есть смещение до адреса, где начинается искомое значение.



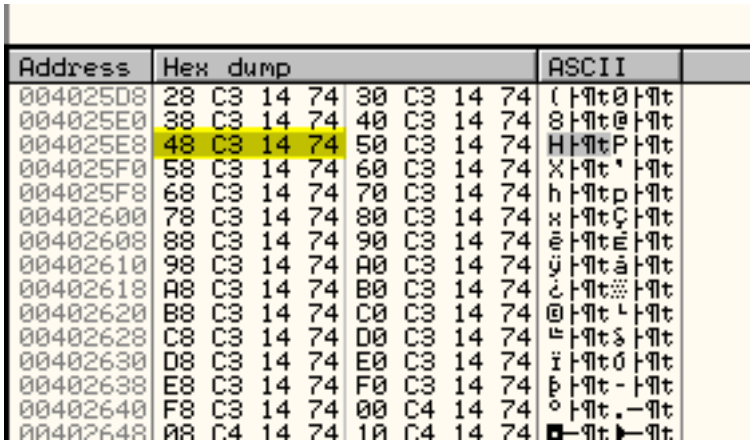
-

Таким образом, прибавив к началу таблицы 300, получим в EAX значение 4025e8.



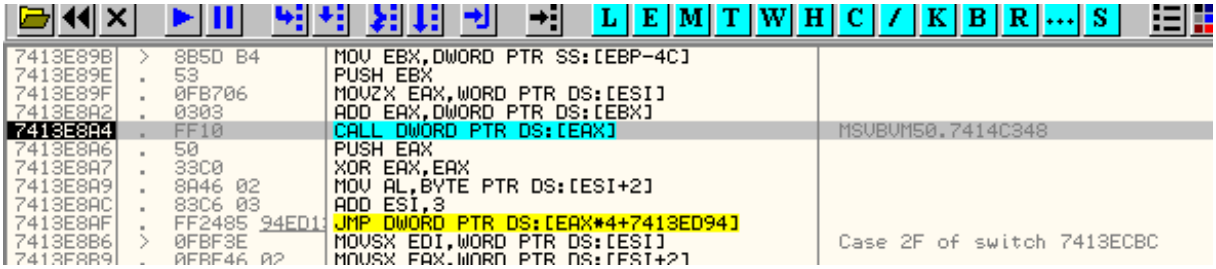
-

Это значение указывает сюда:



-

То есть опкод 0f, используя предоставленный параметр, получает сохранённое в таблице значение.



-

Дальше у нас вызов по адресу, где читается данная таблица, и как видим, содержимое EAX равно:

-

Заходить в CALL не будем, сразу посмотрим, что останется у нас при выходе из него. Как уже говорилось, значения сохраняются в стек.

Продолжаем трассировать, проходим CALL с F8.

```
7413E89E   53              PUSH EBX
7413E89F   0FB706          MOVZX EAX,WORD PTR DS:[ESI]
7413E8A2   0303            ADD EAX,DWORD PTR DS:[EBX]
7413E8A4   FF10            CALL DWORD PTR DS:[EAX]
7413E8A6   50              PUSH EAX
7413E8A7   33C0            XOR EAX,EAX
7413E8A9   8A46 02         MOV AL,BYTE PTR DS:[ESI+2]
```

-

-

То есть после всего этого в стеке остаётся значение, и нам необходимо выяснить, для чего оно служит. Пока что мы знаем, что это значение читается из таблицы элементов данных, параметр равен 0300, и результат кладётся в стек.

```
Proc: 401c98
401BD0: 04 FLdRfVar          local_008C
401BD3: 21 FLdPrThis
401BD4: 0f VCallAd           text
401BD7: 19 FStAdFunc           local_0088
401BDA: 08 FLdPr             local_0088
401BDD: 0d VCallHresult        get__ipropTEXTEDIT
```

-

Следующий опкод равен 19, он работает с локальной переменной 88, которая становится источником его параметра.

-

Здесь выходим из его выполнения.

```
7413E50A   BB FFFFFFFF     MOV EBX,-1
7413E50F   0FBF06          MOVSX EAX,WORD PTR DS:[ESI]
7413E512   83C6 02         ADD ESI,2
7413E515   03C5            ADD EAX,EBP
7413E517   59              POP ECX
7413E518   53              PUSH EBX
7413E519   50              PUSH EAX
7413E51A   51              PUSH ECX
7413E51B   E8 301E0000     CALL MSVBVM50.74140350
7413E520   33C0            XOR EAX,EAX
7413E522   8A06            MOV AL,BYTE PTR DS:[ESI]
7413E524   46              INC ESI
7413E525   FF2485 94ED137  JMP DWORD PTR DS:[EAX*4+7413ED94]
```

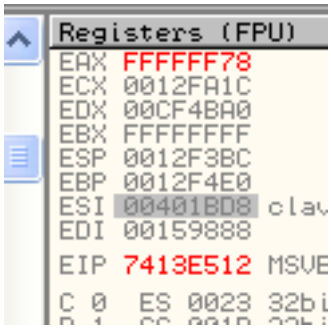
-

Что у нас тут?

```
Address    Hex dump                  ASCII
00401BD0   04 74 FF 21 0F 00 03 19   ♦t !☼.♥↓
00401BD8   78 FF 08 78 FF 0D A0 00   x ◘x .à.
00401BE0   00 00 6C 74 FF 1B 01 00   ..lt ←☺.
00401BE8   FB 30 2F 74 FF 1A 78 FF   ¹0/t →x
```

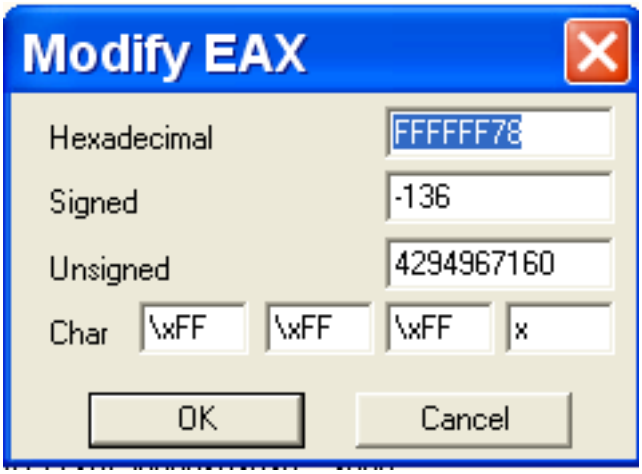
-

С помощью MOVSX считываются параметры, отрицательные значения дополняются FF.



-

Как уже предсказывалось ранее, это значение -88 в шестнадцатеричной системе счисления, о чём нам говорит EXDEC.

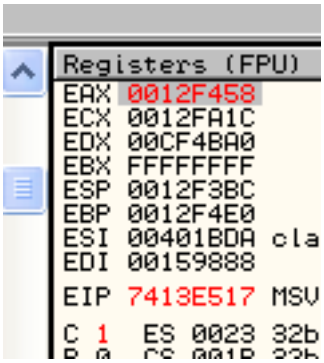


-

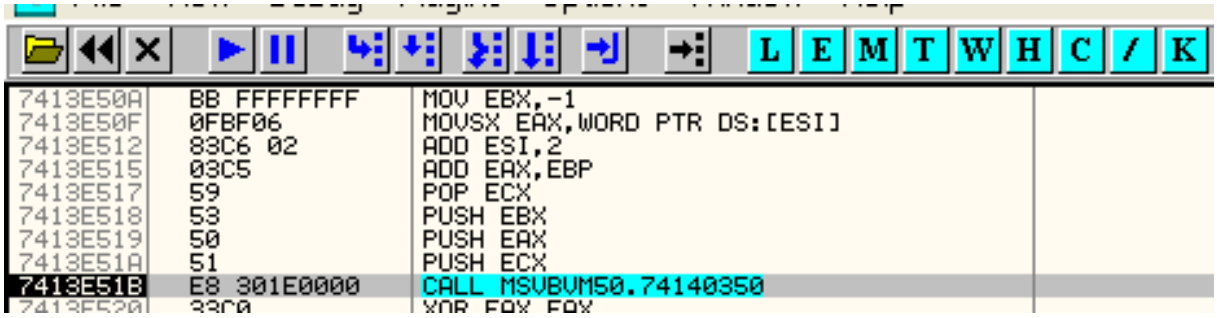
-136 в десятеричной равной -88 в шестнадцатеричной.

-

Здесь увеличиваем указатель ESI на два и складываем EBP с -88, и результат этой операции помещается в EAX.

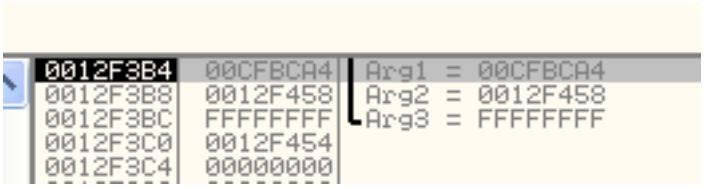


-



-

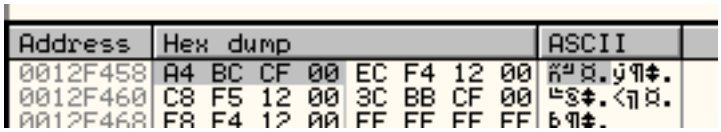
Видим, что к тому моменту, когда доходим до вызова, в стеке находятся три параметра.



-

Первым является значение, которое было сохранено предыдущим опкодом, а два других значения в моём случае равны 12f458 (локальная переменная ebp-88) и -1 – третий параметр. Нажимаем F8, чтобы не заходить внутрь вызова.

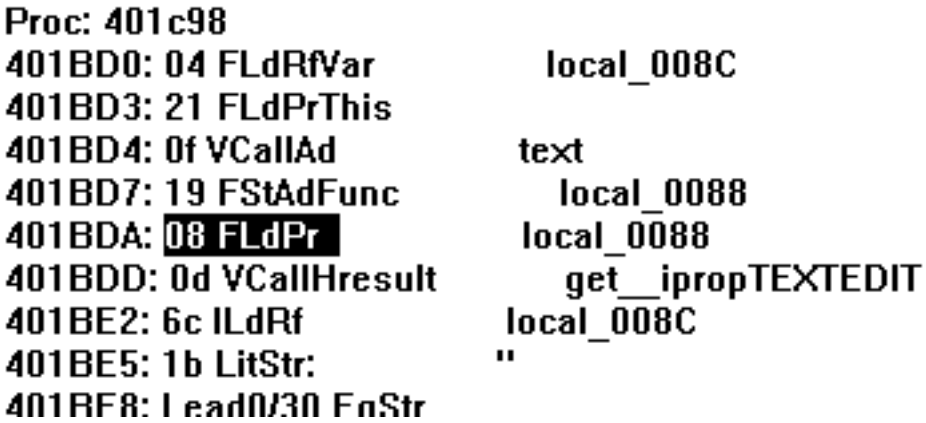
После его выполнения видим, что в ebp-88 сохранилось значение, найденное предыдущим опкодом.



-

Всё остальное осталось таким же, не считая изменений в стеке и того, что ECX возвратил 0, вероятно для того, чтобы отметить, что процесс был завершён.

То есть то, что было помещено в EBP-88 – это значение, полученное предыдущим опкодом.



-

Доходим до другого опкода, равного 08, которая также пытается работать с той же локальной переменной 88, то есть ebp-88.

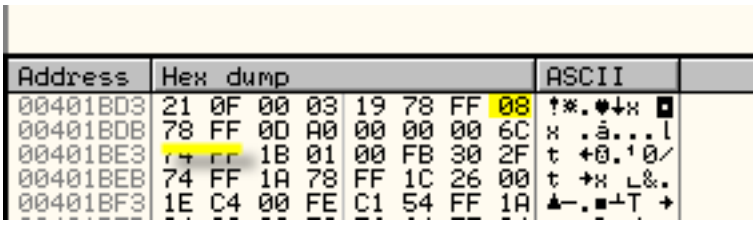
-

Входим в опкод.

-

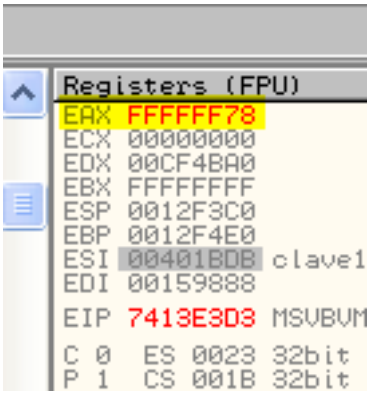
В куске кода, видим “XOR EAX, EAX”, завершающий выполнение опкода, а чуть выше – условный переход. Посмотрим, что он делает.

Сначала EAX’у передаются параметры опкода.

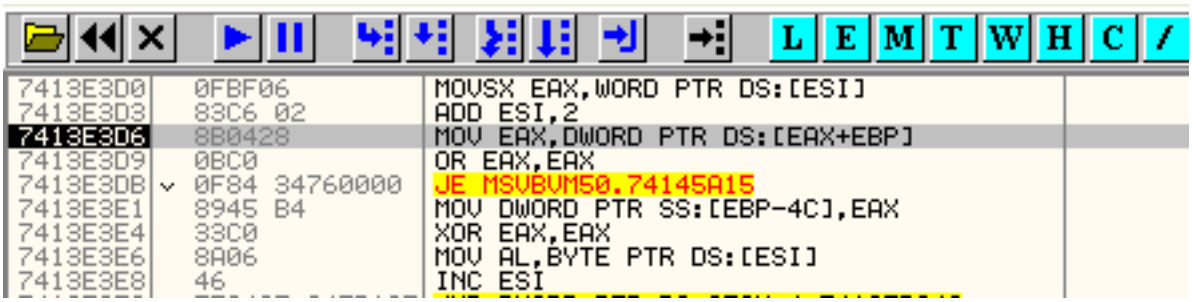


-

Как и в прошлый раз, значение FF78, перемещённое с помощью MOVSX, имеет FFы в своём составе, то есть это отрицательное число, равное -88 в шестнадцатеричной системе.



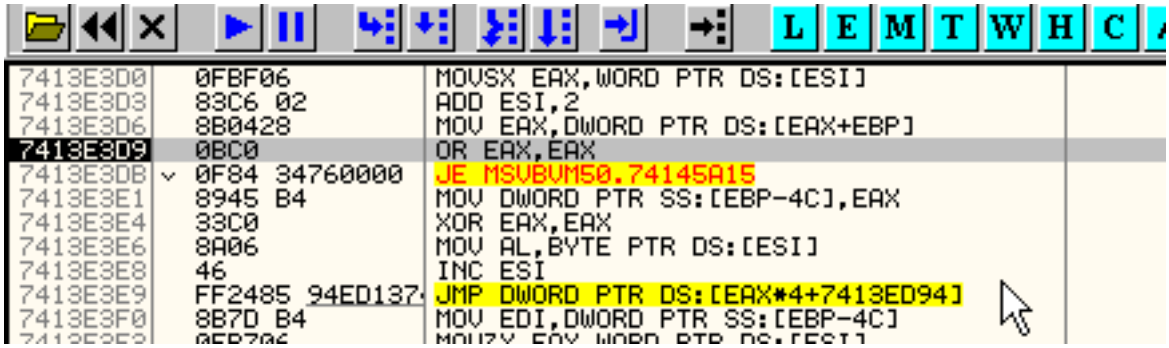
-



-

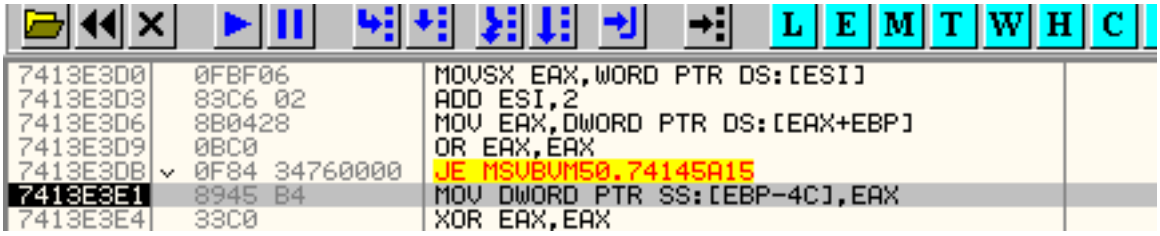
На это строке напрямую задаётся сумма EAX+EBP, то есть EBP-88, и перемещается значение, содержащееся по данному адресу. Очевидно, что мы берём его из таблицы элементов данных.

-



-

Тестируем, равно ли нулю, если бы так и было, то переход был бы совершён, но так как сейчас нуля нет, то продолжаем.



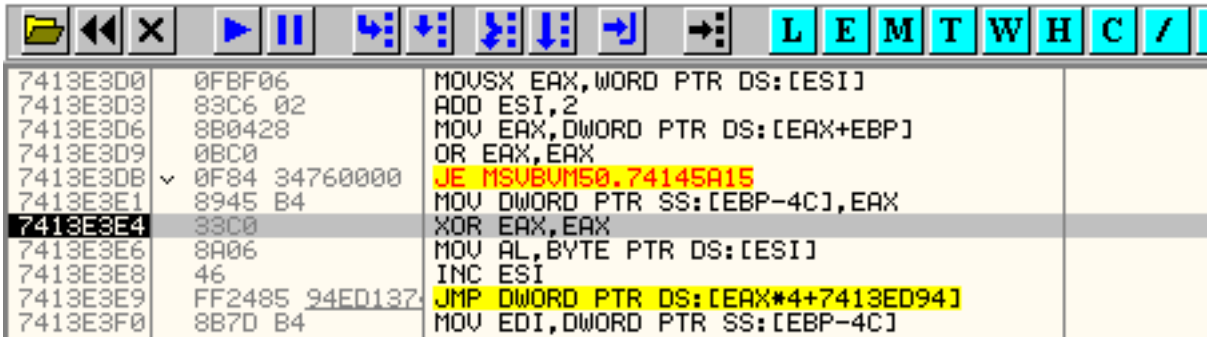
-

Сохраняем это значение в EBP-4C.

Здесь нам нужно вспомнить то, о чём говорилось в самом начале.

То, что мы видим – это чтение содержимого EBP + 8 (ссылочный указатель) и сохранение оного в EBP.

Так как ebp-4c – это указатель на элемент данных, и мы уже убедились, что он содержит корректное значение и в нём нет нуля, то поэтому эта переменная ebp-4c называется «указатель на элемент данных», так как это значение связано с таблицей элементов данных.

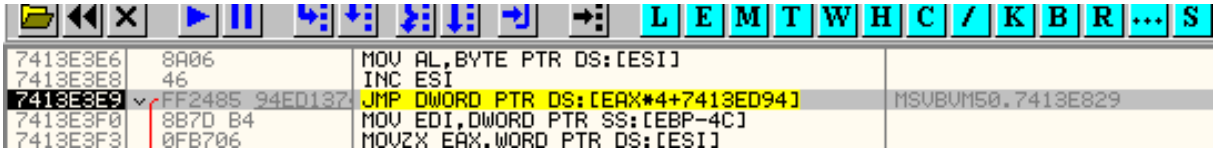


-

Доходим до следующего опкода.

```
Proc: 401c98
401BD0: 04 FLdRfVar          local_008C
401BD3: 21 FLdPrThis
401BD4: 0f VCallAd           text
401BD7: 19 FStAdFunc           local_0088
401BDA: 08 FLdPr             local_0088
401BDD: 0d VCallHresult        get__ipropTEXTEDIT
401BE2: 6c ILdRf             local_008C
401BE5: 1b LitStr:           ''
```

-

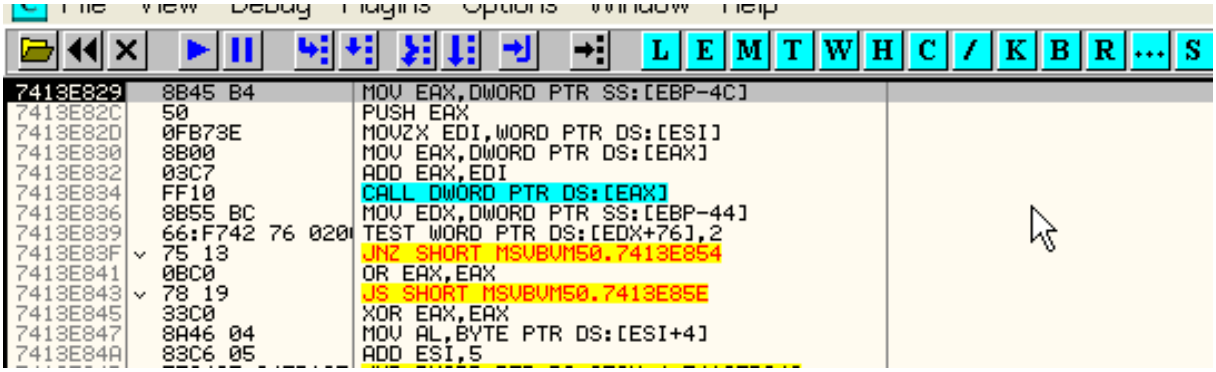


-

В пресвятом Гугле смотрим, для чего служит этот опкод.

0d VCallHresult #получает текст из поля ввода текста (textbox)

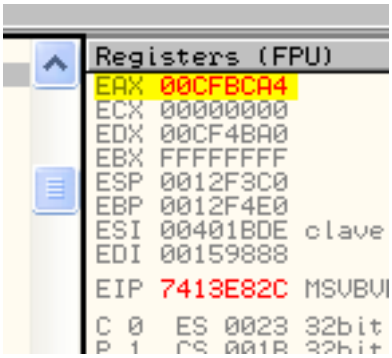
То есть будет прочитан неправильный серийный номер, который мы ввели в текстовое поле. Трассируем и смотрим, так ли это.



-

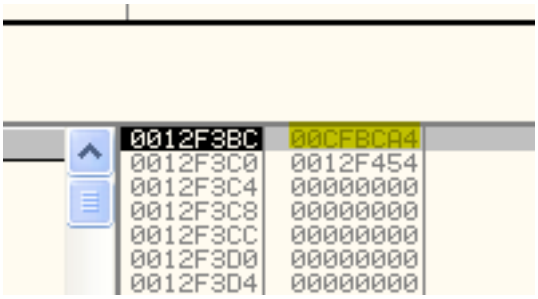
Видим опкод, завершающийся “XOR EAX, EAX” как обычно.

Первое, что здесь происходит – это считывается содержимое из EBP-4c (а это, как мы говорили, указатель на элемент данных) и помещается в EAX. Это архизнакомое нам значение, считывающееся из таблицы элементов данных.



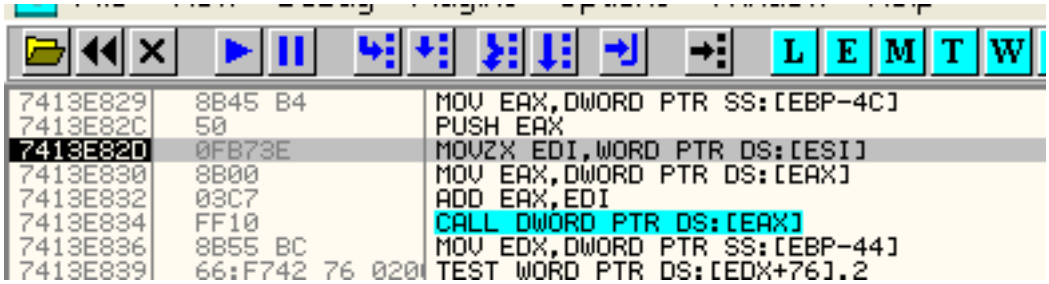
-

Дальше это значение помещается в стек.

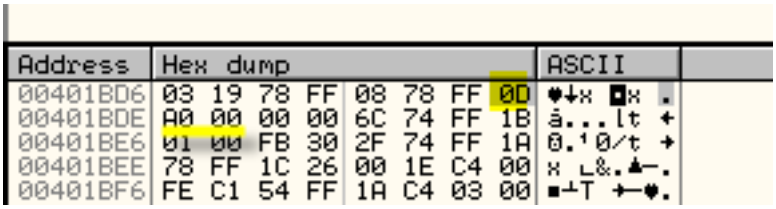


-

Потом считывается параметр опкода.



-

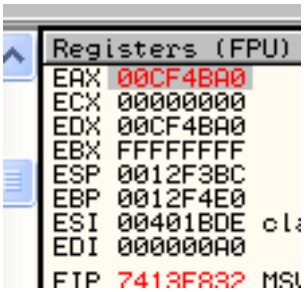


-

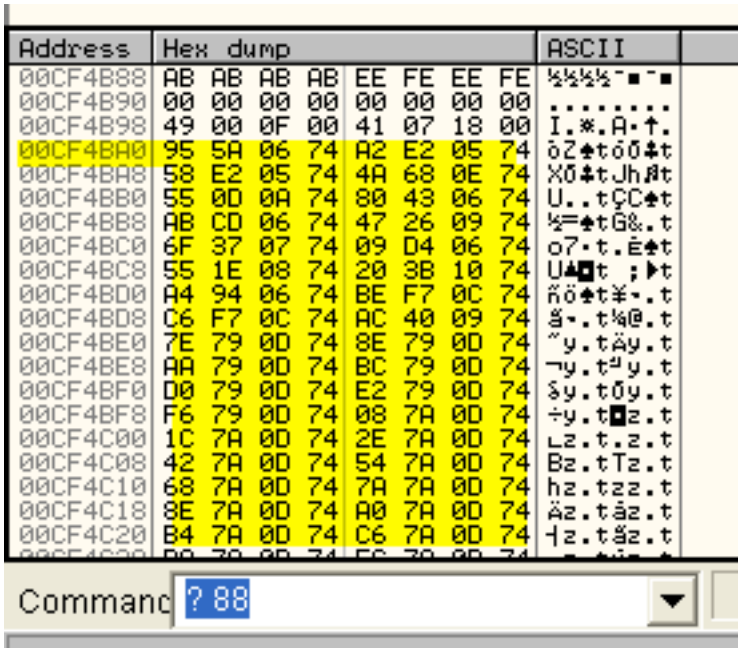
-

И помещается в EDI.

Затем читается содержимое EAX, являющееся началом другой таблицы.

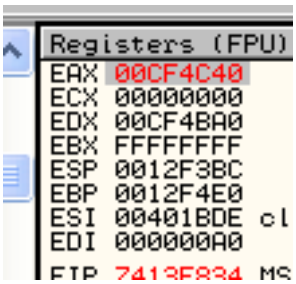


-

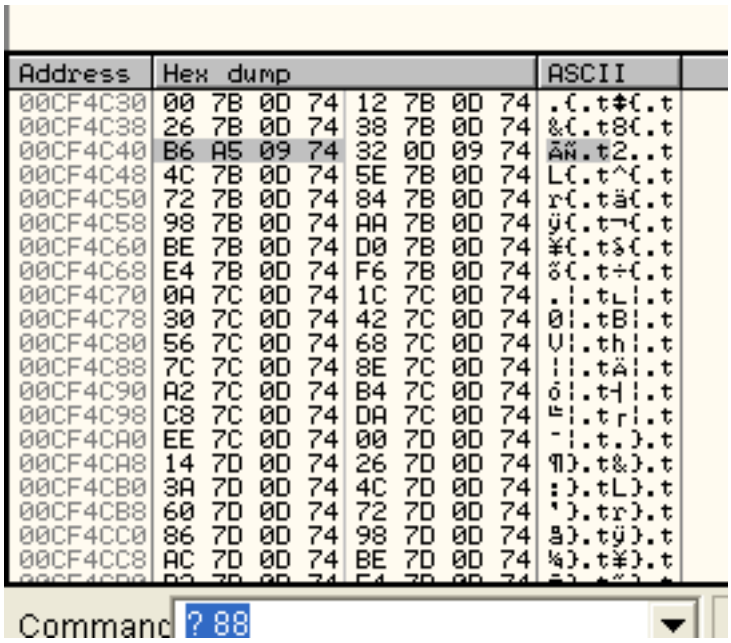


-

И к этому значению прибавляется параметр 00a0, чтобы получить окончательный адрес в её пределах.

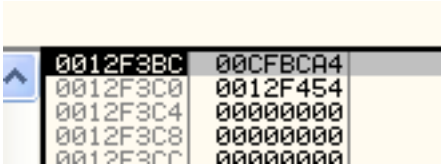


-



-

И затем совершается CALL по адресу в этой таблице. Конечно, не будет трассировать этот вызов, посмотрим на то, какие значения останутся в стеке от выполнившихся опкодов.

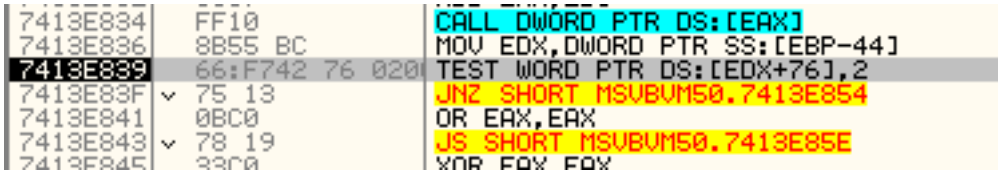


-

Выполняем CALL с помощью F8.

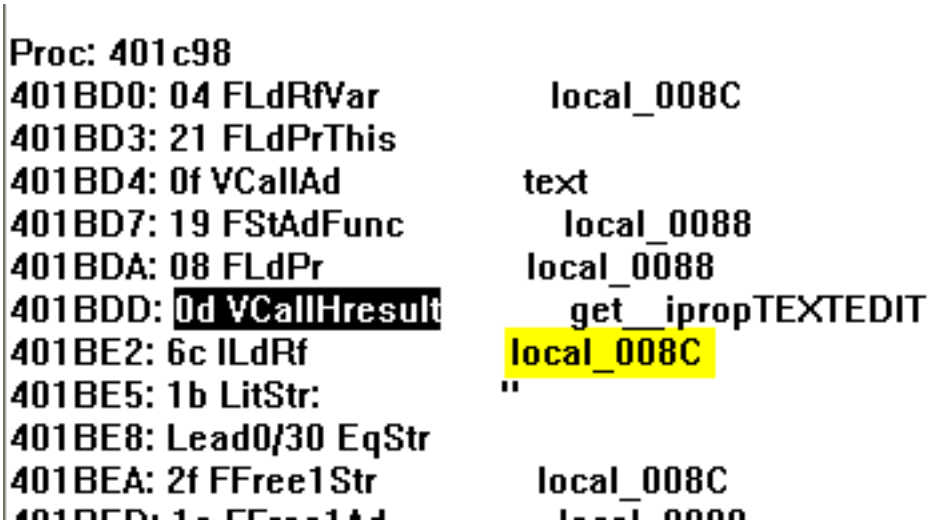
Затем помещаем в EDX значение из EBP-44.

-



-

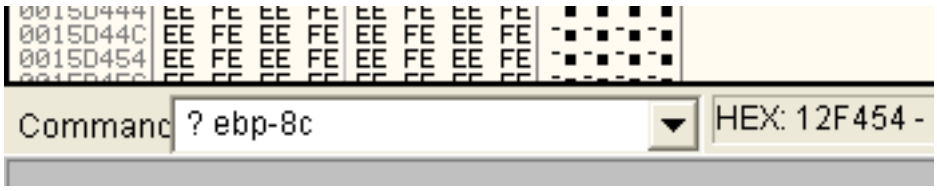
И после сравнения пары значений доходим до следующего опкода, но вы можете спросит, читает ли он то, что мы ввели, то есть наш неправильный опкод? Смотрим ответ на этот вопрос в EXDEC.



-

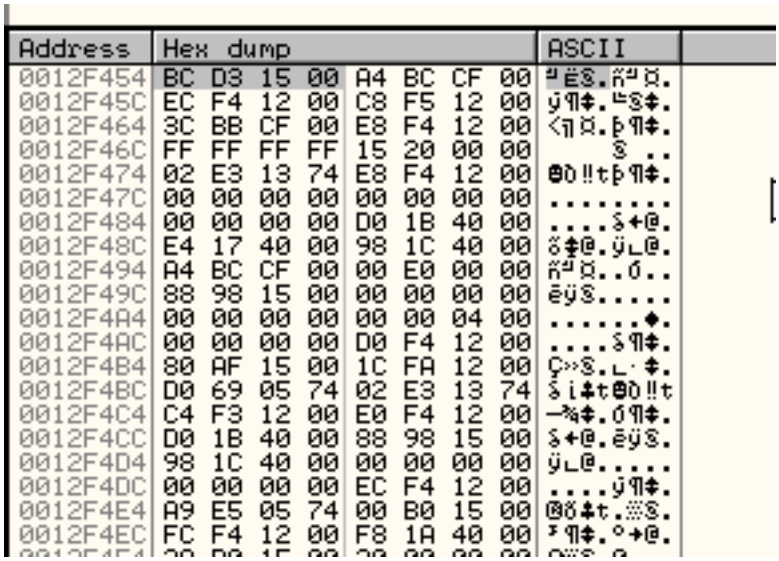
Видим, что продолжается работа с локальной переменной 8c, которую мы также знаем как EBP-8C.

Если поищем значение EBP-8c:



-

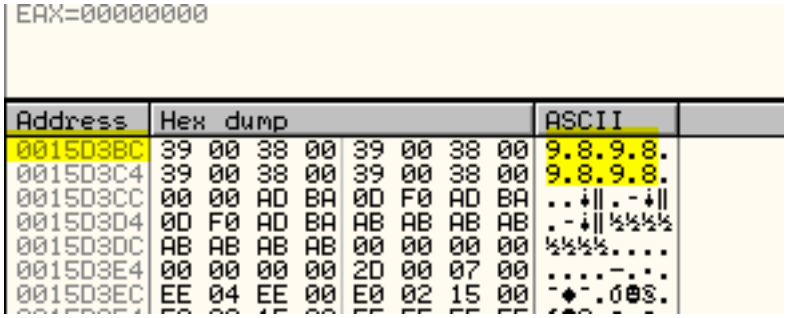
Оно равно 12f454.



Address	Hex dump	ASCII
0012F454	BC D3 15 00 A4 BC CF 00	
0012F45C	EC F4 12 00 C8 F5 12 00	
0012F464	3C BB CF 00 E8 F4 12 00	
0012F46C	FF FF FF FF 15 20 00 00	
0012F474	02 E3 13 74 E8 F4 12 00	
0012F47C	00 00 00 00 00 00 00 00	
0012F484	00 00 00 00 D0 1B 40 00	
0012F48C	E4 17 40 00 98 1C 40 00	
0012F494	A4 BC CF 00 00 E0 00 00	
0012F49C	88 98 15 00 00 00 00 00	
0012F4A4	00 00 00 00 00 00 04 00	
0012F4AC	00 00 00 00 D0 F4 12 00	
0012F4B4	80 AF 15 00 1C FA 12 00	
0012F4BC	D0 69 05 74 02 E3 13 74	
0012F4C4	C4 F3 12 00 E0 F4 12 00	
0012F4CC	D0 1B 40 00 88 98 15 00	
0012F4D4	98 1C 40 00 00 00 00 00	
0012F4DC	00 00 00 00 EC F4 12 00	
0012F4E4	A9 E5 05 74 00 B0 15 00	
0012F4EC	FC F4 12 00 F8 1A 40 00	

-

И это указатель на введенный нами неправильный серийный номер, который находится в 15d3bc.



EAX=00000000

Address	Hex dump	ASCII
0015D3BC	39 00 38 00 39 00 38 00	9.8.9.8.
0015D3C4	39 00 38 00 39 00 38 00	9.8.9.8.
0015D3CC	00 00 AD BA 0D F0 AD BA	
0015D3D4	0D F0 AD BA AB AB AB AB	
0015D3DC	AB AB AB AB 00 00 00 00	
0015D3E4	00 00 00 00 2D 00 07 00	
0015D3EC	EE 04 EE 00 E0 02 15 00	

-

Уф, пришлось попотеть, но мы, наконец, добрались до того места, где считывается неправильный серийник.

Следующий опкод – это 6c IldRf

-

-

```
'6C, ILdRf, 0C, 00
'Load reference value.
'Parameter 1 = 2 bytes.
'Parameter 1 is offset into local Frame.
'Offset = &hC.
'Address pointer is retrieved from local Frame at offset.
'Address pointer is pushed onto stack.
'Stack operations: Push x1.
```

-

Здесь говорится, что этот опкод занимается загрузкой ссылочного значения. Заходим в него.

-

Что у нас здесь?

-

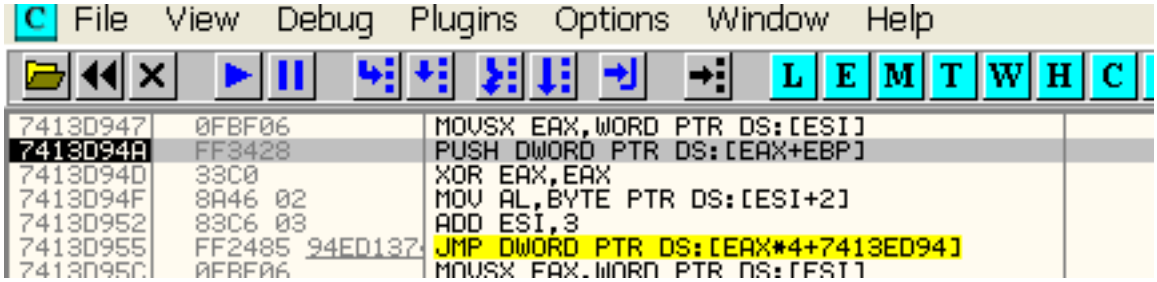
Сначала осуществляется перемещение параметра с помощью MOVSX, который является отрицательным (FF).

-

Смотрим, что у нас оказывается в EAX.

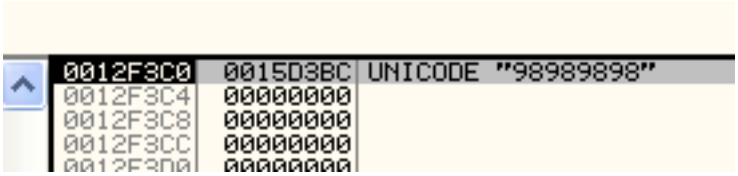
-

Это -8c в шестнадцатеричной системе.



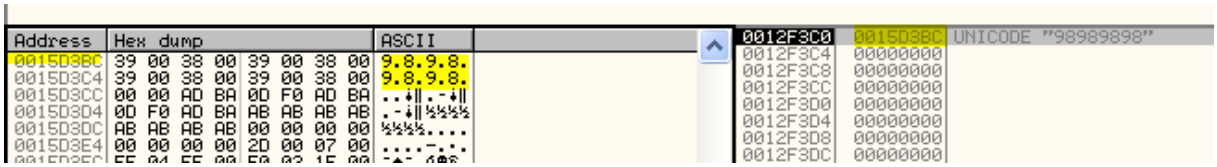
-

Следующая инструкция суммирует его с EBP, что в результате даёт EBP-8c. Содержимое по этому адресу кладётся в стек.



-

Он содержимое ebp-8c – это указатель на неправильный серийник, то есть сейчас этот указатель находится у нас в стеке.



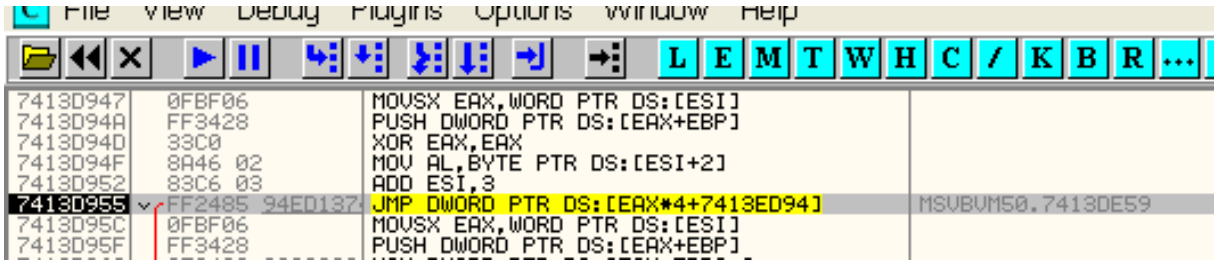
-

Если посмотрим через DUMP, то ясно увидим, что он указывает на неправильный серийный номер. Таким образом, это и является действием опкода загрузки ссылочного значения, который кладёт в стек значение из локальной переменной.

Следующий опкод – это:

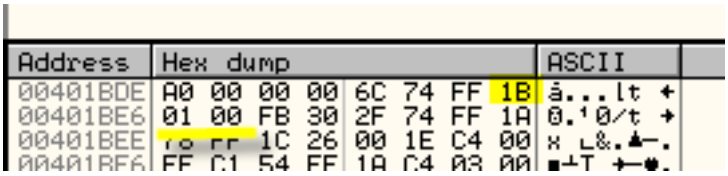
1b LitStr , что расшифровывается как “Literal String” (символьная строка)

Смотрим, что он делает.



-

Входим в опкод.



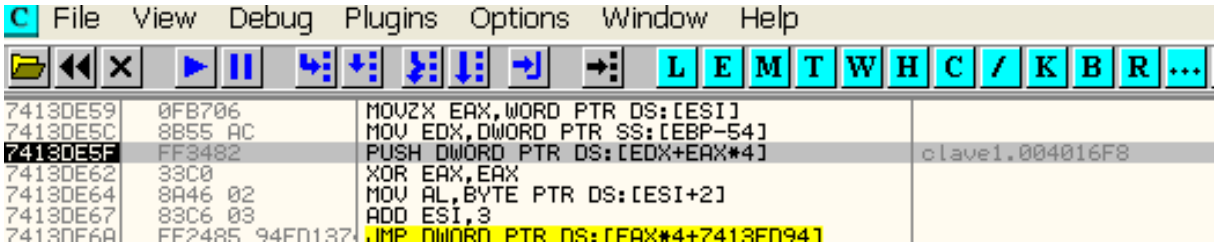
-

Сначала читается параметр, который здесь равен 0001 (положительный, т.к. нет FF). Он помещается в EAX.

-

-

Видим, что на следующей строке в EDX помещается значение 4017E4. Для чего оно, мы не знаем.

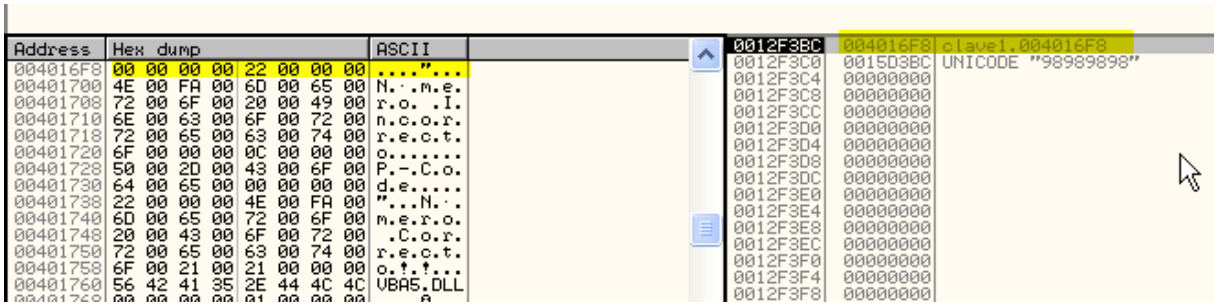


-

И для чего может служить помещение в стек 4016f8?

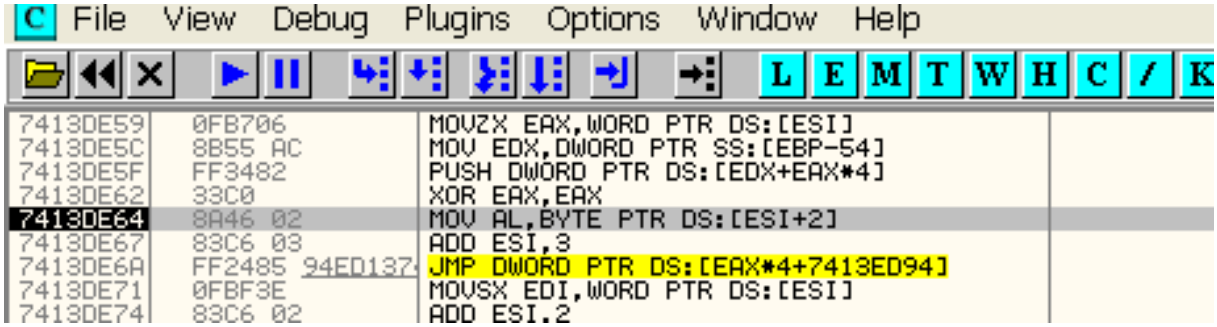
-

Видим, что в пояснении EXDEC'a показываются только две одинарные кавычки, то есть значение, помещаемое в стек, является пустой строкой.



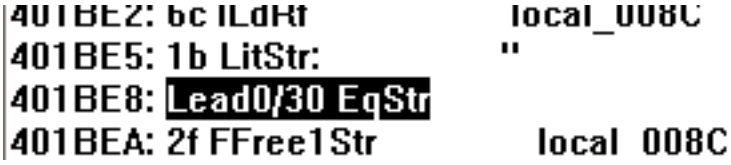
-

Конечно же, смотрим через DUMP, что указывает на пустую строку, заключённую в кавычки, то есть то, с чем будет работать следующий опкод, проверяющий, напечатали мы что-нибудь или оставили поле ввода пустым и нажали “Register”.



-

Что есть:



-

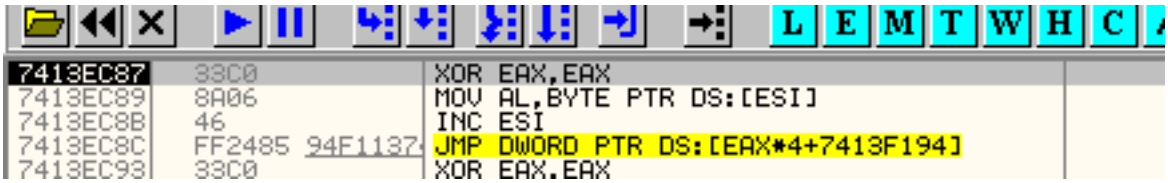
Ок, “LEAD 0” – это операция, а “30 EqStr” – это вторая часть опкода. Посмотрим, что это такое в пресвятом Гугле.

Lead0/30 EqStr – сравнение двух строк.

То есть здесь будут сравниваться эти две строки. Это двойной опкод, то есть сначала считывается то, что в дальнейшем будет использоваться для косвенного перехода.

-

И здесь первый опкод завершается “XOR EAX, EAX”, где ничего не делается, и читается второй опкод.



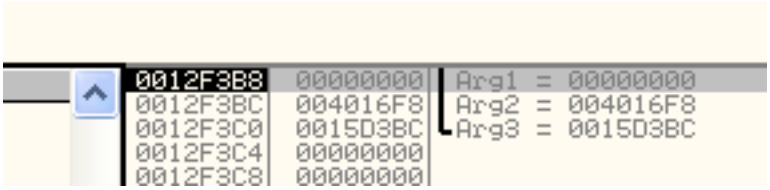
-

-

Здесь читается второй опкод 30, но разница в случае с двойным опкодом заключается в считывании параметров, так как в таком опкоде первый завершается с помощью «XOR EAX, EAX» и считывается второй, но параметры первого опкода остаются доступными для чтения.

-

Здесь находится второй опкод, который делает PUSH 0.



-

Когда доходим до CALL'а, у нас есть три аргумента. Минуем CALL с помощью F8 и смотрим, что изменилось.

Видим, что стек сместился, но значения остались без изменений, только находятся теперь чуть выше.

Следующая строка «CMP AL, 0» сообщает, что здесь в AL сохраняется результат. В моём случае это:

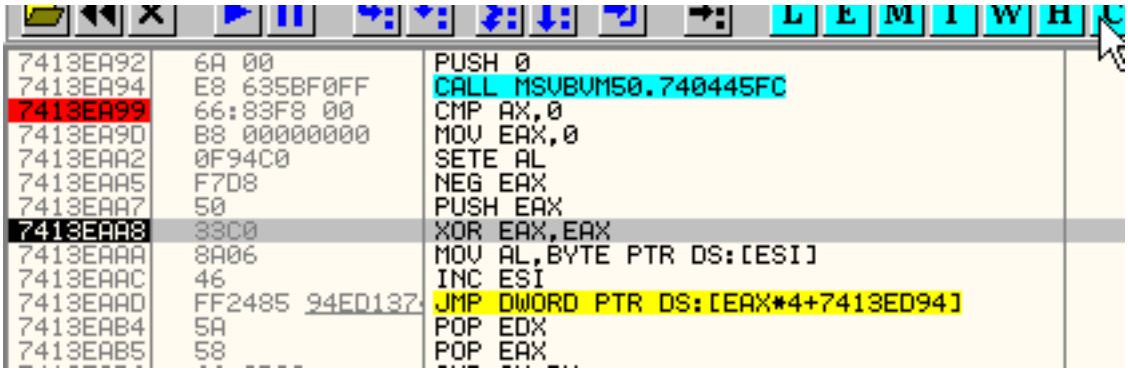
-

AL=01

Потому что строки не равны.

-

После сравнения в EAX помещается значение ноль, а в конечном итоге это значение помещается в стек, так как это результат опкода. Ноль, если не равны, а если бы были равно, то вместо ноля было бы FFFFFF. Можете проверить, хе-хе.

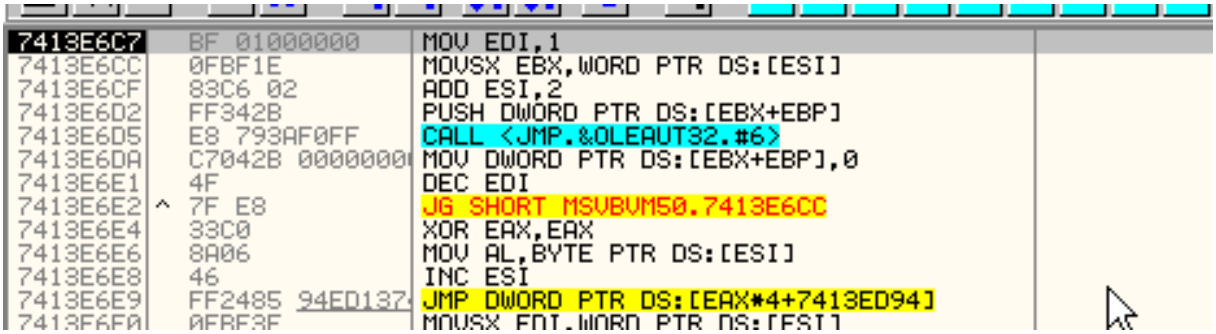


-

Доходим до следующего опкода.

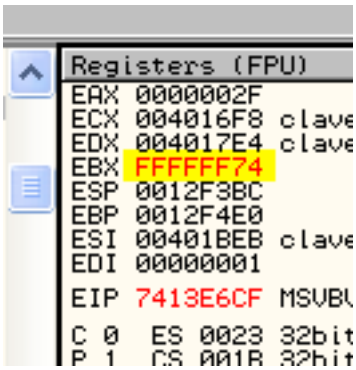
-

Пресвятой Гугл сообщает нам, что это примерно то же, что и SysFreeString, которая освобождает память, занятую неиспользуемой строкой. Видим, что в данном случае освободится ebp-8c.

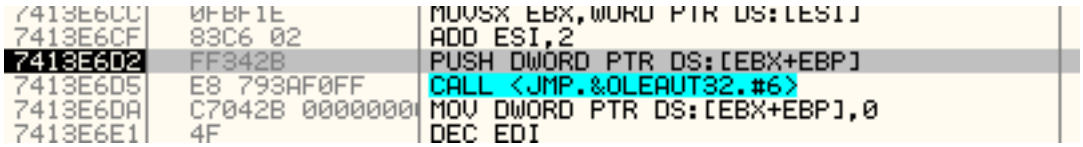


-

Видим, что в EDI перемещается значение 1, затем в EBX – FF74 (в шестнадцатеричной системе это -8c). Как обычно, используется MOVSX, чтобы перемещать вместе с FF'ами, если значение отрицательно.



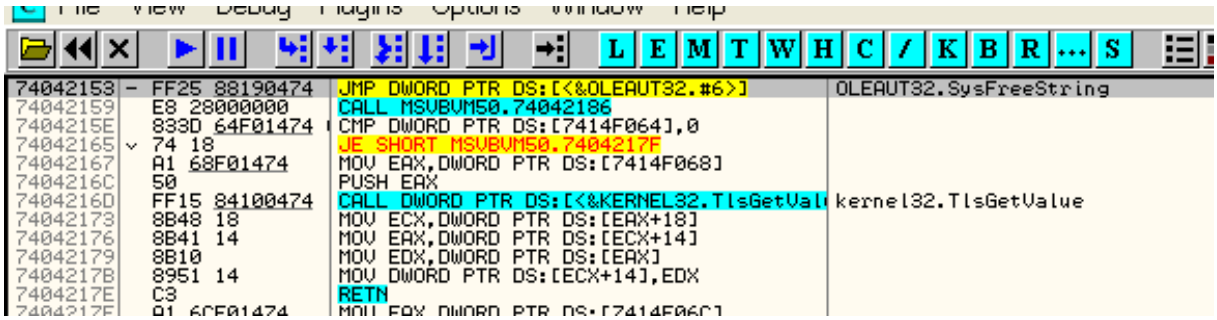
-



-

EBX+EBP равно EBP-8c, так что в стек помещается указатель на неправильный серийный номер.

-



-

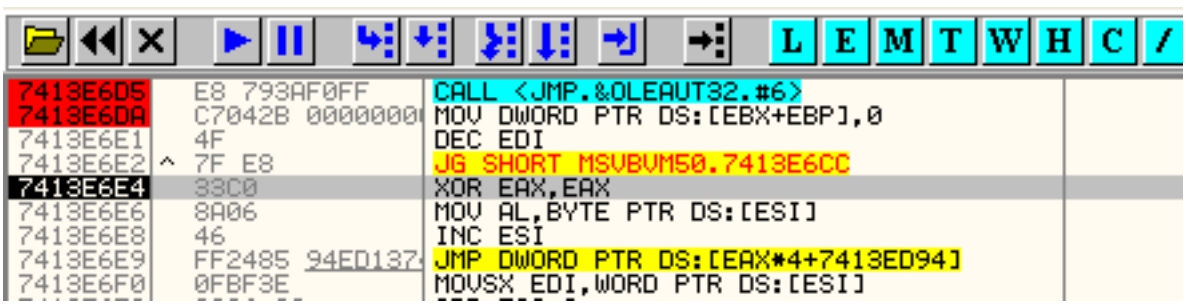
Видим, что идёт вызов API-функции SysFreeString, а по возвращению из неё:

-

Указатель на наш неправильный серийный номер был заменён на нули.

-

Сам неправильный серийник не был стёр, он как был, так и остаётся по адресу 15d3bc. Был стёрт указатель на него, находившийся в EBP-8c.



-

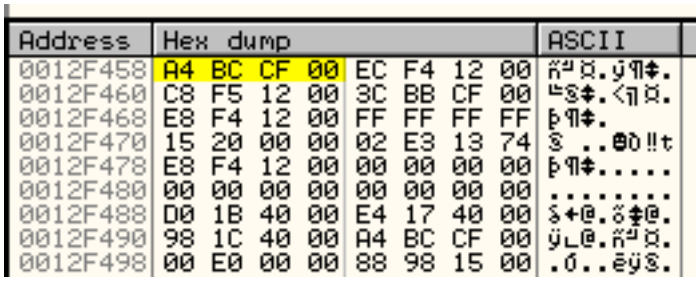
Доходим до следующего опкода.

-

Это сотрёт содёржимое локальной переменной ebp-88.

А что там находится?

-

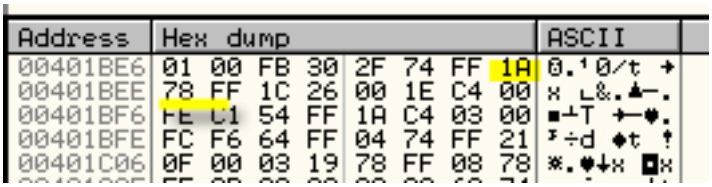


-

Ах, это значение, которое мы искали в таблице элементов данных.

-

Здесь смотрим параметры опкода:



-

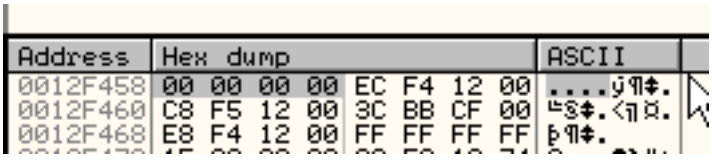
FF78, который является шестнадцатеричным значением -88.

-

Далее в EAX помещается содержимое EBP-88, проверяется, равно ли оно нулю, так как оно не равно, то продолжаем дальше.

-

И доходим до вызова, который, как и раньше, освобождает это значение, и стирает содержимое ebp-88.



-

Здесь помещаем в EAX ноль.

```
7413E746  ^ 7F E3          JG SHORT MSVBVM50.7413E72B
7413E748    33C0           XOR EAX,EAX
7413E74A    8A06           MOV AL,BYTE PTR DS:[ESI]
7413E74C    46             INC ESI
7413E74D    FF2485 94ED137 JMP DWORD PTR DS:[EAX*4+7413ED94]
7413E754    0FBF3E         MOVSX EDI,WORD PTR DS:[ESI]
7413E757    83C6 02        ADD ESI,2
7413E75A    D1EF           SHR EDI,1
```

-

Следующий опкод – это:

```
401BEA: 2f FFree1Str          local_008C
401BED: 1a FFree1Ad            local_0088
401BF0: 1c BranchF:           401BF6
401BF3: 1e Branch:            401c94
401BF6: Lead3/c1 LitVarI4:      [ local_3BE500AC
```

-

Это условный переход, так как все BRANCH’и – это переходы.

-

То есть это переход, срабатывающий, если результат равен «лжи», а затем идёт JMP, то есть этот переход, если мы что-то вводим в поле ввода, переходит на проверку серийного номера, а если нет, то процесс будет повторён, используя JMP.

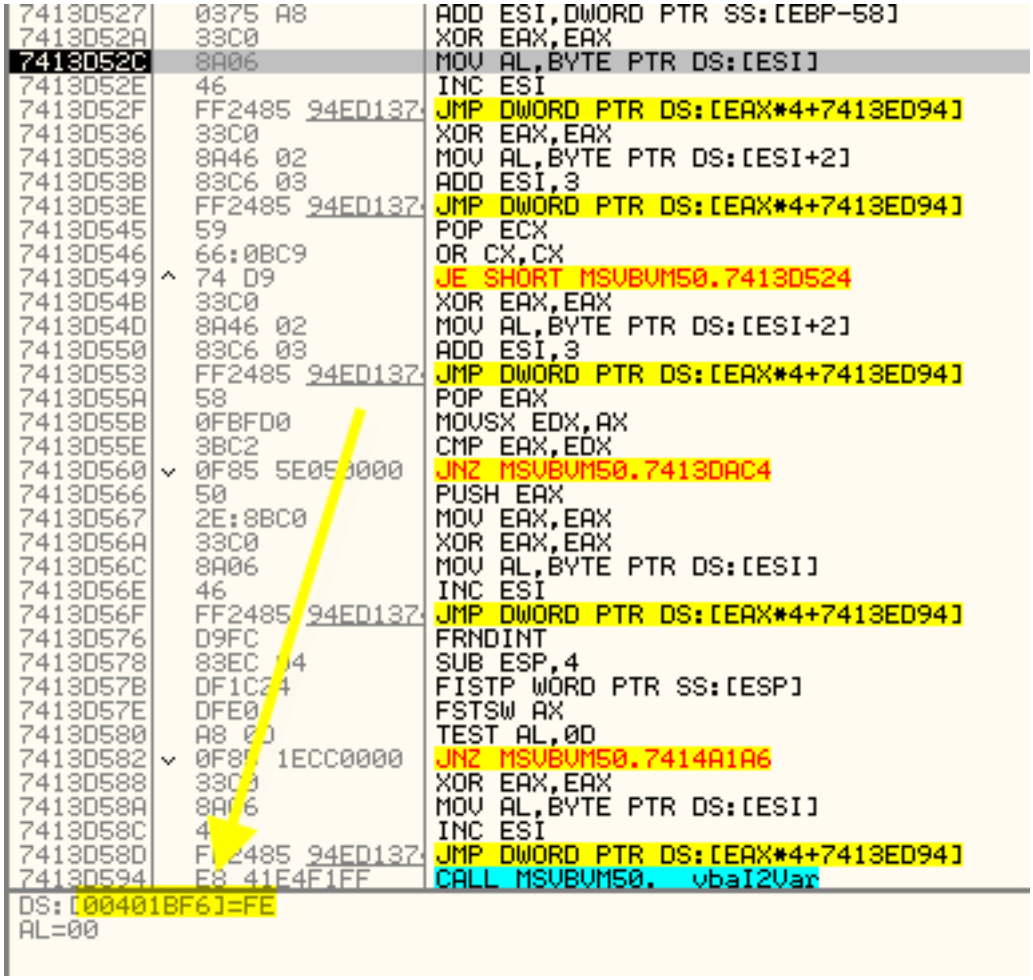
Видим, что это так, если будет переход, то следующим опкодом станет:

```
401BED: 1a FFree1Ad            local_0088
401BF0: 1c BranchF:           401BF6
401BF3: 1e Branch:            401c94
401BF6: Lead3/c1 LitVarI4:      [ local_3BE500AC ] 0x3c41a  (246810)
401BFF: Lead1/f6 FStVar         local_009C
```

- Так как это условный переход, избегаем JMP в 401bf3.

-

Здесь опкод завершается и читается следующий.



-

Как можно предположить из FE в 401bfg6, условный переход срабатывает и минут JMP, хе-хе.

Lead3/c1 LitVarI4

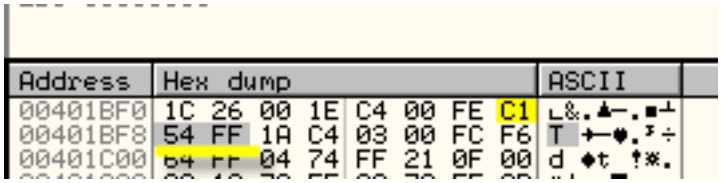
Это двойной опкод. Посмотрим, что он делает.

-

Здесь завершается первый опкод и читается второй.

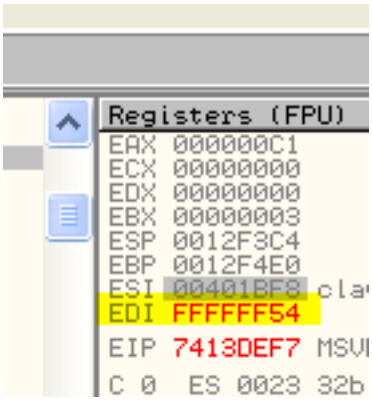
-

Который начинается здесь. Терпеливо трассируем его.



-

Считываются параметры опкода.

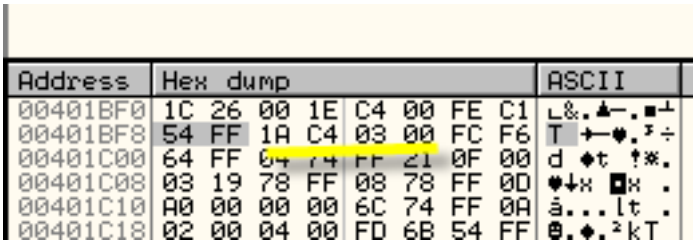


-

Уже знаем, что они дополняются FF, если содержат отрицательное значение, как в данном случае.

-

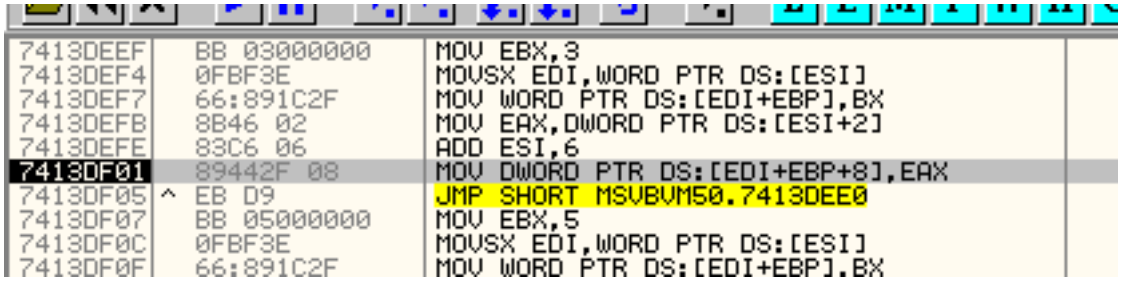
Параметры считываются дальше, в данном случае это целый DWORD, который помещается в EAX.



-

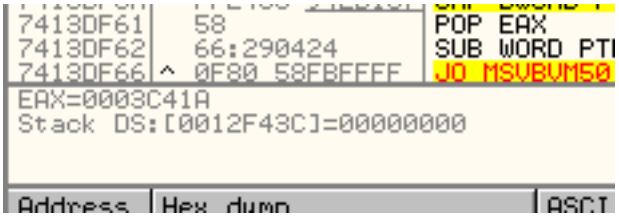
-

И это значение сохраняется в локальную переменную.

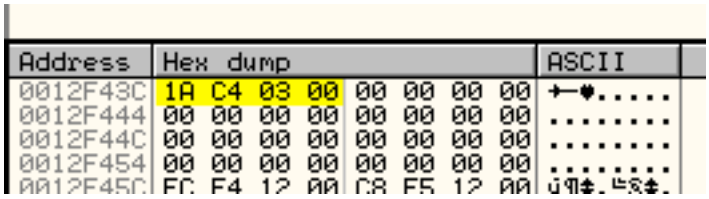


-

EBP+ FFFFFF54 + 8, то есть сумма первого параметра и 8 даёт 12f43c, а здесь это значение сохранено.



-

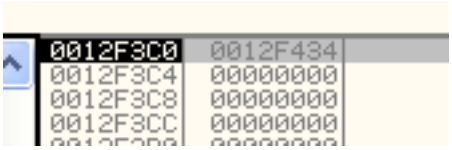


-

JMP помещает нас в:

-

Где 12f434 помещается в стек.



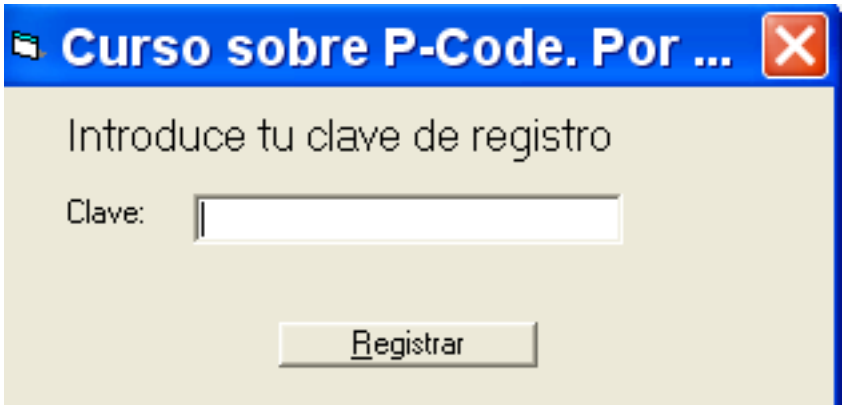
-

Это указатель на структуру, начинающуюся с 3, которое уже было сохранено, а чуть пониже – сохранённое значение.

-

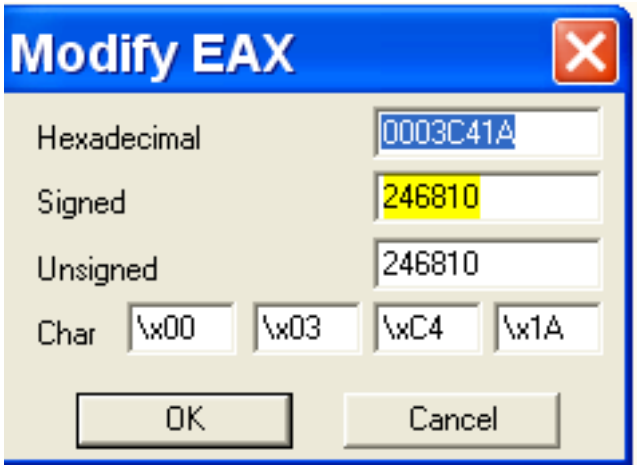
Продолжаем.

В случае, если серийный номер здесь жёстко задано, можем попробовать передать это значение в десятеричном виде и посмотреть, не является ли он правильным серийным номером. Откроем другой экземпляр крэкми не из-под OllyDbg.



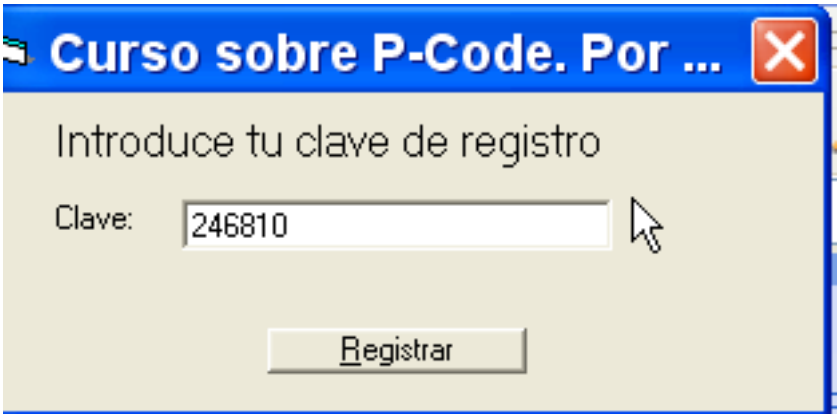
-

Посмотрим в OllyDbg, чему равно это число в десятеричной системе.



-

Оно равно 246810. Введём его в крэкми.



-

-

Хе-хе, я подозревал это, но так легко нам не отделаться. Доходим до сравнения.

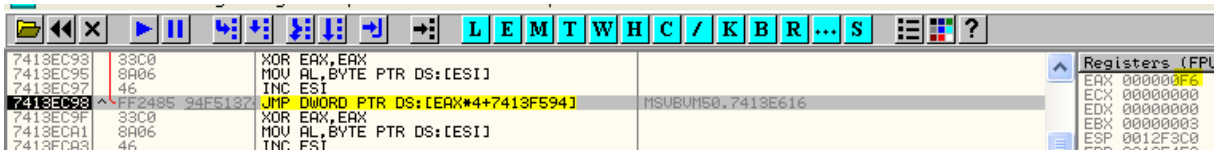
```
401BF3: 1e Branch:          401c94
401BF6: Lead3/c1 LitVarI4:    ( local_3BE500AC ) 0x3c41a  (246810)
401BFE: Lead1/f6 FStVar       local_009C
401C02: 04 FLdRfVar         local_008C
401C05: 21 FLdPrThis
401C06: 0f VCallAd          text
401C09: 19 FStAdFunc          local_0088
```

-

Следующий опкод также двойной.

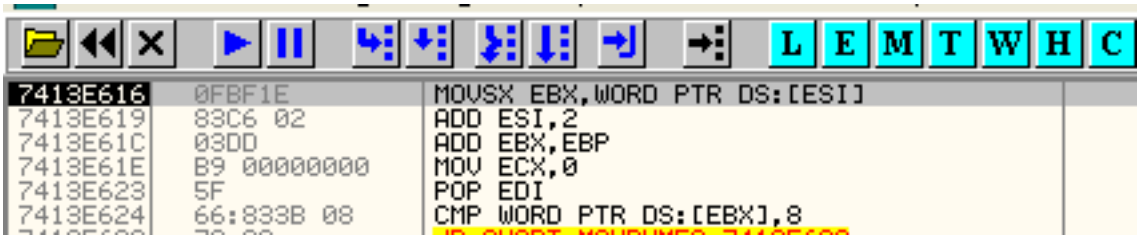
Это FC и немедленно он завершается и начинается второй.

-



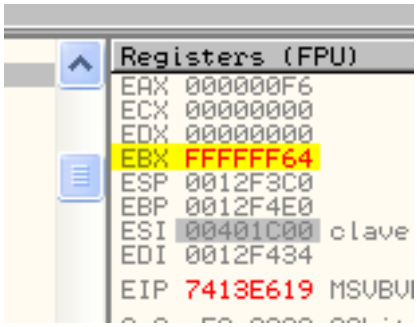
-

Это F6, бесстрашно заходим в него, EXDEC нам говорит, что будем работать с локальной переменной 9c (то есть ebp-9c).



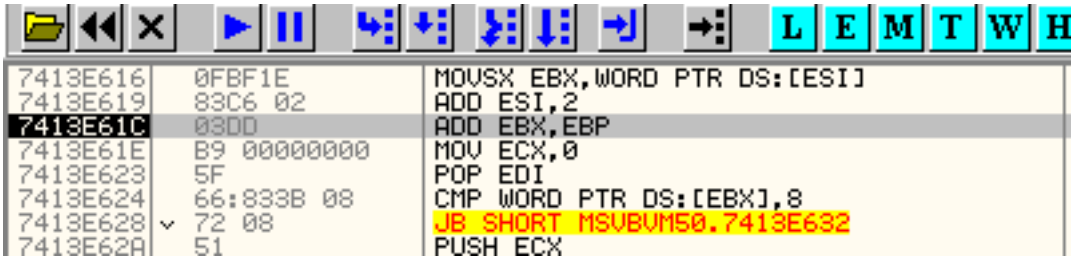
-

Здесь читаются параметры, которые дополнены FF.



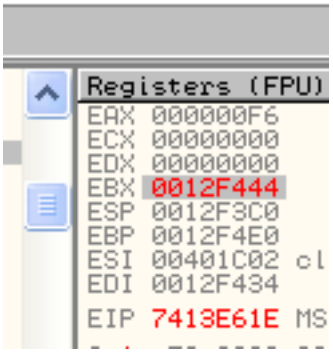
-

Конечно, это значение -9c.

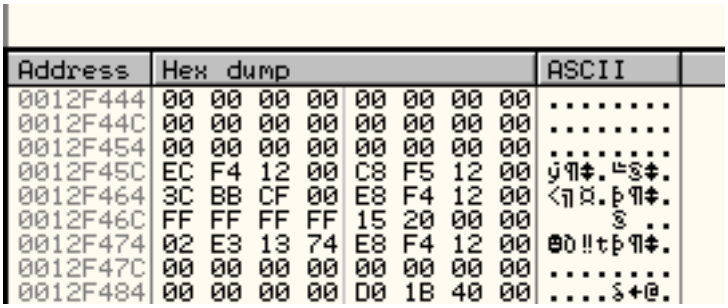


-

Прибавляем к EBP, чтобы получилось EBP-9c или 12f444, которое пока что остаётся пустым.

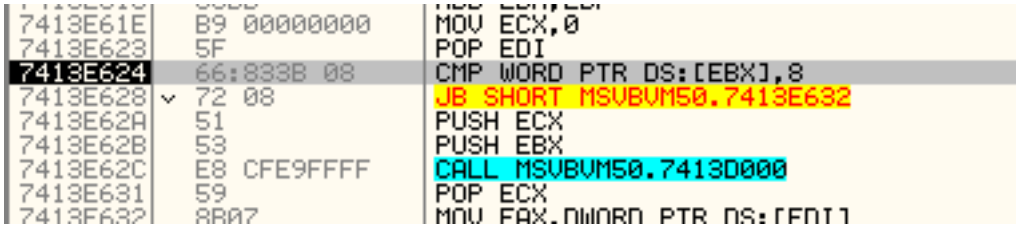


-



Address	Hex dump	ASCII
0012F444	00 00 00 00 00 00 00 00	....
0012F44C	00 00 00 00 00 00 00 00	....
0012F454	00 00 00 00 00 00 00 00	....
0012F45C	EC F4 12 00 C8 F5 12 00	ì¶↕.╚§↕.
0012F464	3C BB CF 00 E8 F4 12 00	<╗¤.Φ¶↕.
0012F46C	FF FF FF FF 15 20 00 00	▪ §..
0012F474	02 E3 13 74 E8 F4 12 00	☻π‼tΦ¶↕.
0012F47C	00 00 00 00 00 00 00 00	....
0012F484	00 00 00 00 D0 1B 40 00	....╨←@.

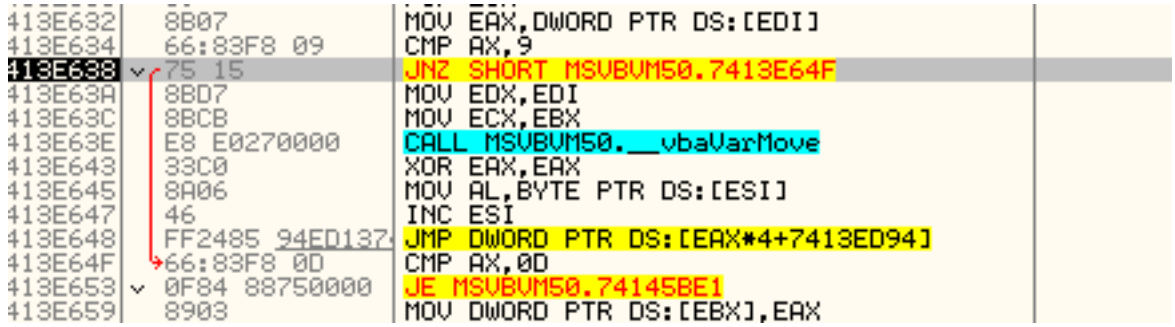
-



```
7413E61E   B9 00000000     MOV ECX,0
7413E623   5F              POP EDI
7413E624   66:833B 08      CMP WORD PTR DS:[EBX],8
7413E628   72 08           JB SHORT MSVBVM50.7413E632
7413E62A   51              PUSH ECX
7413E62B   53              PUSH EBX
7413E62C   E8 CFE9FFFF     CALL MSVBVM50.7413D000
7413E631   59              POP ECX
7413E632   8B07            MOV EAX,DWORD PTR DS:[EDI]
```

-

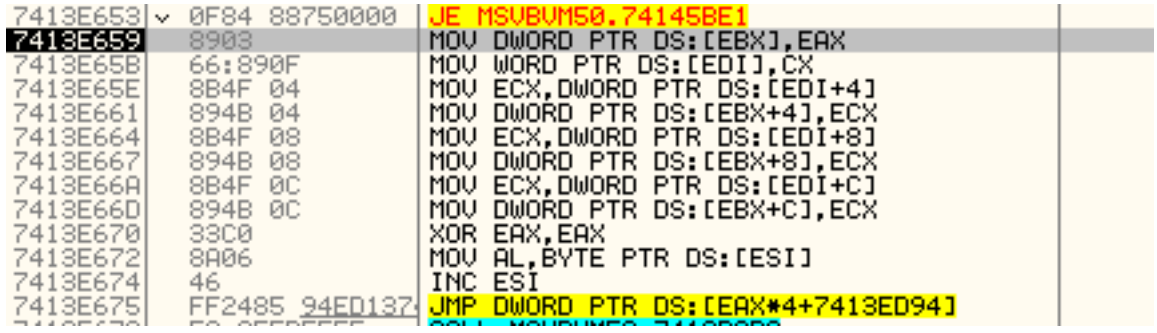
Затем проводится проверка, равно ли это значение 8, если да, то делаем переход.



```
413E632   8B07            MOV EAX,DWORD PTR DS:[EDI]
413E634   66:83F8 09      CMP AX,9
413E638   75 15           JNZ SHORT MSVBVM50.7413E64F
413E63A   8BD7            MOV EDX,EDI
413E63C   8BCB            MOV ECX,EBX
413E63E   E8 E0270000     CALL MSVBVM50.__vbaVarMove
413E643   33C0            XOR EAX,EAX
413E645   8A06            MOV AL,BYTE PTR DS:[ESI]
413E647   46              INC ESI
413E648   FF2485 94ED137  JMP DWORD PTR DS:[EAX*4+7413ED94]
413E64F   66:83F8 0D      CMP AX,0D
413E653   0F84 88750000   JE MSVBVM50.74145BE1
413E659   8903            MOV DWORD PTR DS:[EBX],EAX
```

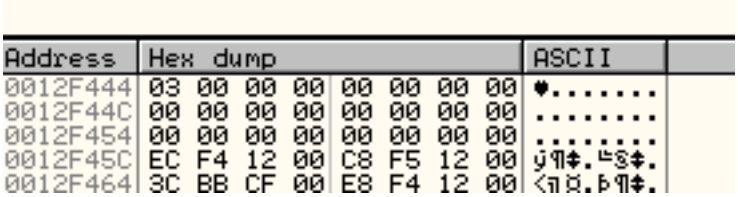
-

Далее снова переход, но не будет вдаваться в детаил. Доходим до последних строк опкода.



```
7413E653   0F84 88750000   JE MSVBVM50.74145BE1
7413E659   8903            MOV DWORD PTR DS:[EBX],EAX
7413E65B   66:890F         MOV WORD PTR DS:[EDI],CX
7413E65E   8B4F 04         MOV ECX,DWORD PTR DS:[EDI+4]
7413E661   894B 04         MOV DWORD PTR DS:[EBX+4],ECX
7413E664   8B4F 08         MOV ECX,DWORD PTR DS:[EDI+8]
7413E667   894B 08         MOV DWORD PTR DS:[EBX+8],ECX
7413E66A   8B4F 0C         MOV ECX,DWORD PTR DS:[EDI+C]
7413E66D   894B 0C         MOV DWORD PTR DS:[EBX+C],ECX
7413E670   33C0            XOR EAX,EAX
7413E672   8A06            MOV AL,BYTE PTR DS:[ESI]
7413E674   46              INC ESI
7413E675   FF2485 94ED137  JMP DWORD PTR DS:[EAX*4+7413ED94]
```

-

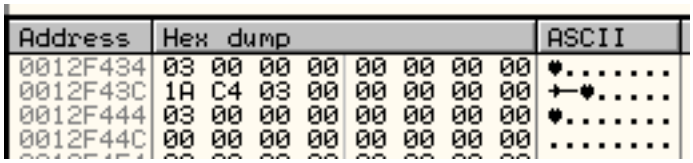


Address	Hex dump	ASCII
0012F444	03 00 00 00 00 00 00 00	♥.....
0012F44C	00 00 00 00 00 00 00 00	....
0012F454	00 00 00 00 00 00 00 00	....
0012F45C	EC F4 12 00 C8 F5 12 00	ì¶↕.╚§↕.
0012F464	3C BB CF 00 E8 F4 12 00	<╗¤.Φ¶↕.

-

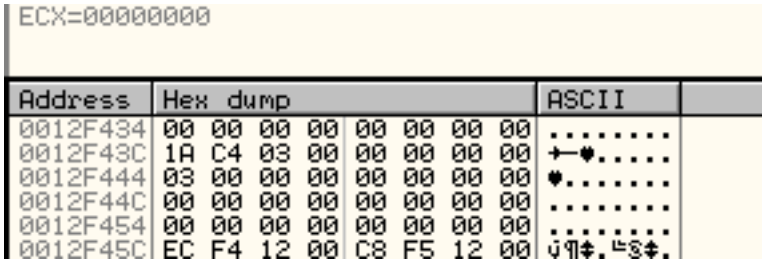
Помещаем значение 3, находящееся в EAX в переменную EBP-9c

И стираем 3 из структуры, встреченной нами ранее.



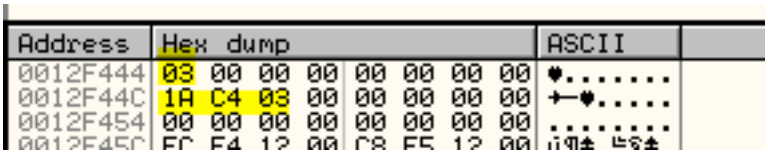
-

При выполнении:



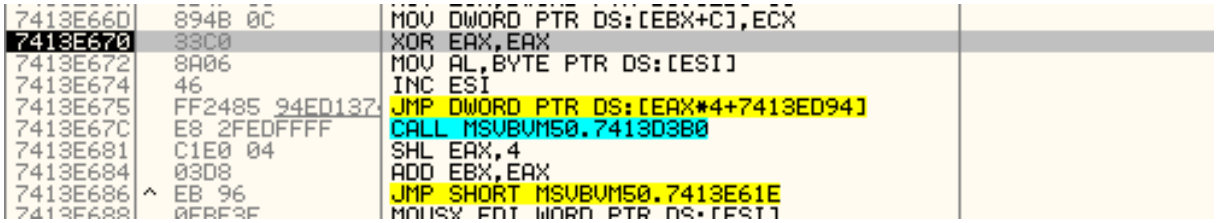
-

Затем последующие строки копируют всё, что у нас здесь есть в новое местоположение, то есть в EBP-9c.



-

Здесь видим число, которое является правильным серийным номером и находящееся в структуре, начинающейся в EBP-9c.



-

Доходим до следующего опкода.

```
401C02: 04 FLdRfVar               local_008C
```

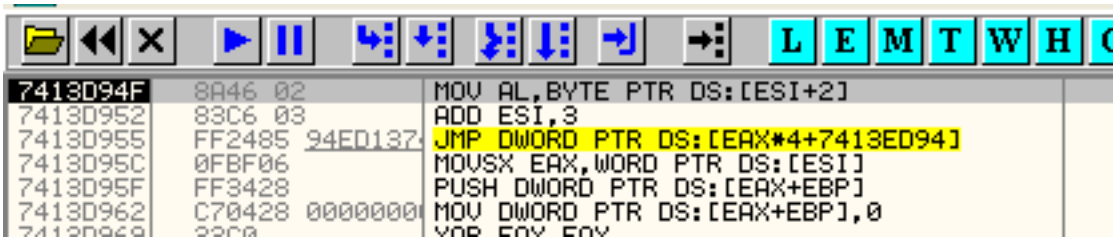
Видим, что повторяется всё, что мы уже видели в начале.

```
401C02: 04 FLdRfVar               local_008C
401C05: 21 FLdPrThis
401C06: 0f VCallAd                text
401C09: 19 FStAdFunc              local_0088
401C0C: 08 FLdPr                  local_0088
401C0F: 0d VCallHresult           get__ipropTEXTEDIT
401C14: 6c ILdRf                  local_008C
```

Всё похоже на то, как было в начале. Следующий опкод:

-

Поэтому, чтобы перепрыгнуть через всё, что уже было, устанавливаем BPM ON ACCESS на 401c17, чтобы остановиться на считывании опкода.



-

Останавливаемся здесь и читаем опкод 0A. Смотрим, что это такое в EXDEC'e.

Это ImpAdCallFPR4 – вызов API-функции. EXDEC показывает какой именно.

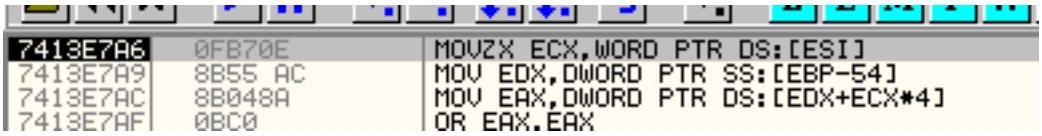
Например:

```
4017F5: 0a ImpAdCallFPR4:       _rtcMsgBox   <---Llamada a la función
```

-

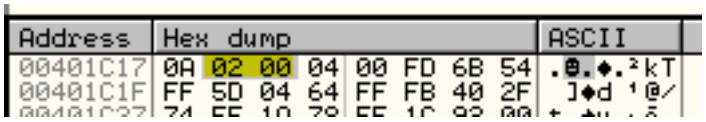
В данном примере произойдёт вызов API-функции rtcMsgBox. В нашем же случае это вызов функции:

401C17: 0a ImpAdCallFPR4: _rtcR8ValFromBstr

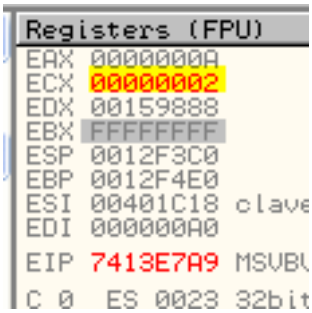


-

Читаются параметры опкода.

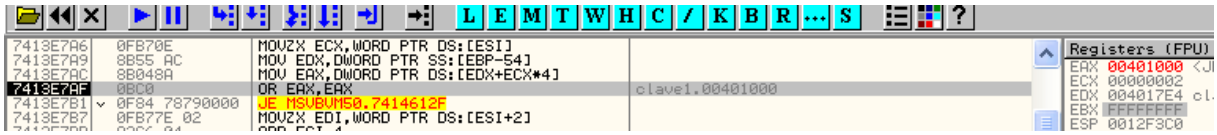


-



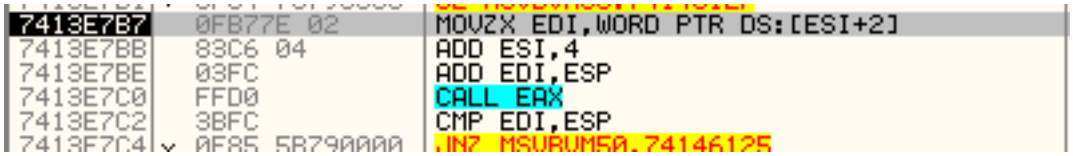
-

Они помещаются в ECX.



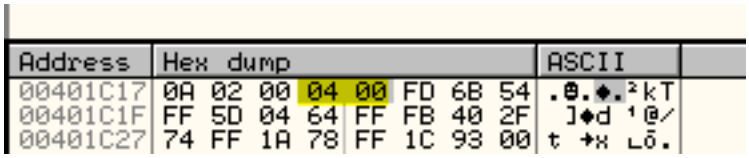
-

Затем в EAX помещается значение 401000 и тестируется, не равно ли оно нулю.

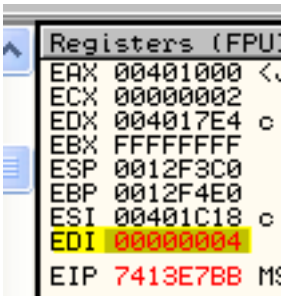


-

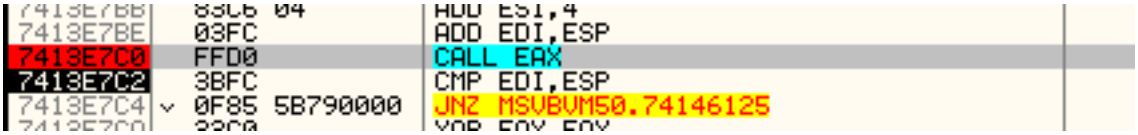
Далее читается второй параметр.



-

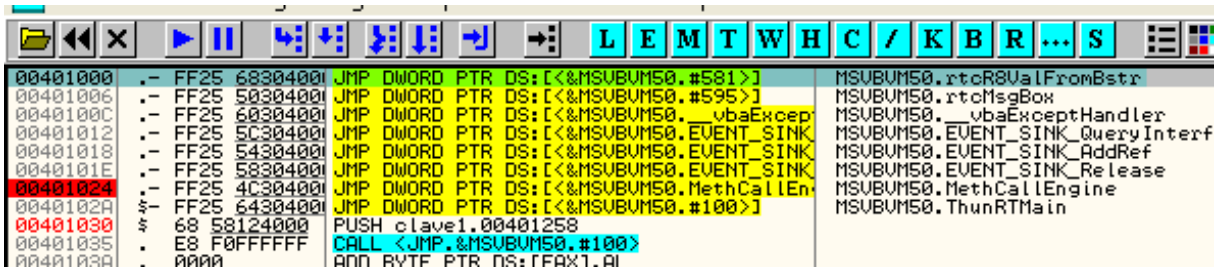


-



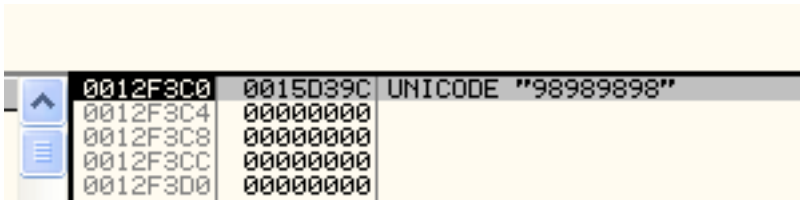
-

И доходим до “CALL EAX”, где EAX равен 401000. Смотрим, куда это ведёт – а ведёт это к упомянутой выше API-функции.



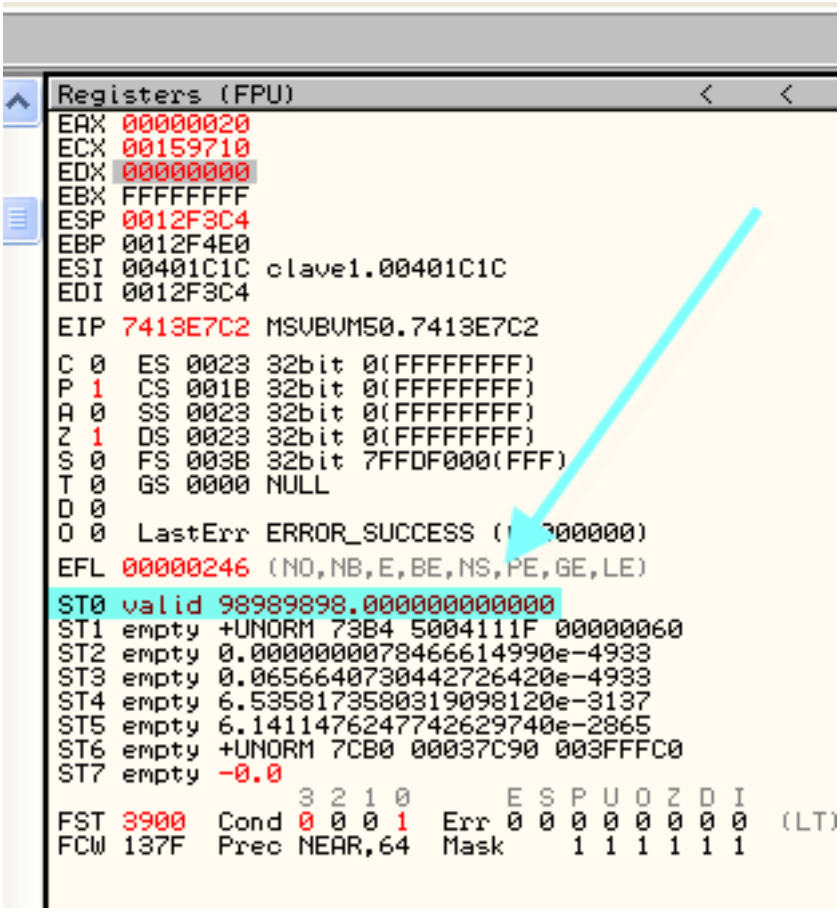
-

Параметр, передающийся через стек API-фукнкции:



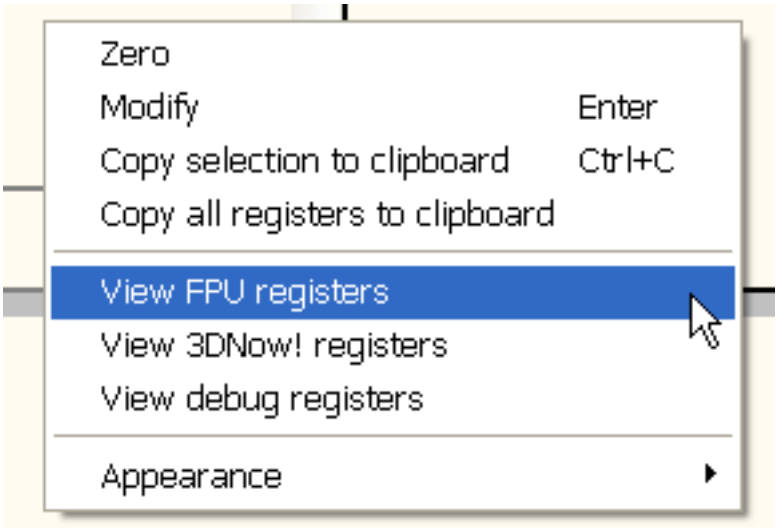
-

Мой неправильный серийный номер:



-

Который был загружен в St0, являющаяся верхним элементом стека плавающей запятой. Мы не говорили об этом раньше, но ничего. Находятся эти элементы под регистрами, если у вас они не отображаются, то нужно использовать специальную опцию. Нажимаем правую кнопку мыши.



-

Здесь загружается мой неправильный серийный номер.

```
7413E7C2   3BFC              CMP EDI,ESP
7413E7C4   0F85 5B790000     JNZ MSVBVM50.74146125
7413E7CA   33C0              XOR EAX,EAX
7413E7CC   8A06              MOV AL,BYTE PTR DS:[ESI]
7413E7CE   46                INC ESI
7413E7CF   FF2485 94ED137    JMP DWORD PTR DS:[EAX*4+7413ED94]
7413E7D6   0FB70E            MOVZX ECX,WORD PTR DS:[ESI]
7413E7D9   8B55 AC           MOV EDX,DWORD PTR SS:[EBP-54]
```

-

Доходим до следующего опкода.

401C1C: Lead2/6b CVarR8

Итак, находимся здесь, заходим в опкод.

```
7413EC9F   33C0              XOR EAX,EAX
7413ECA1   8A06              MOV AL,BYTE PTR DS:[ESI]
7413ECA3   46                INC ESI
7413ECA4   FF2485 94F9137    JMP DWORD PTR DS:[EAX*4+7413F994]
```

-

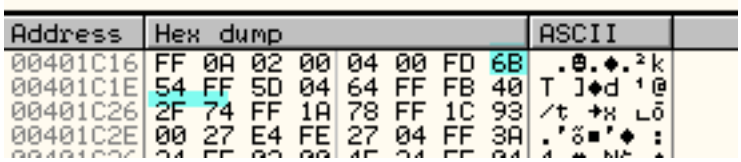
Так как опкод двойной, то завершается первый и грузится второй.

```
7413D804   BA 05000000       MOV EDX,5
7413D809   0FBF06            MOVSX EAX,WORD PTR DS:[ESI]
7413D80C   03C5              ADD EAX,EBP
7413D80E   DD58 08           FSTP QWORD PTR DS:[EAX+8]
7413D811   66:8910           MOV WORD PTR DS:[EAX],DX
7413D814   50                PUSH EAX
7413D815   DFE0              FSTSW AX
7413D817   A8 0D             TEST AL,0D
7413D819   0F85 87C90000     JNZ MSVBVM50.7414A1A6
7413D81F   33C0              XOR EAX,EAX
```

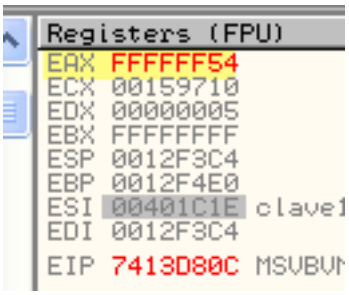
-

Ок, здесь несколько инструкций плавающей запятой, которые мы не рассматривали.

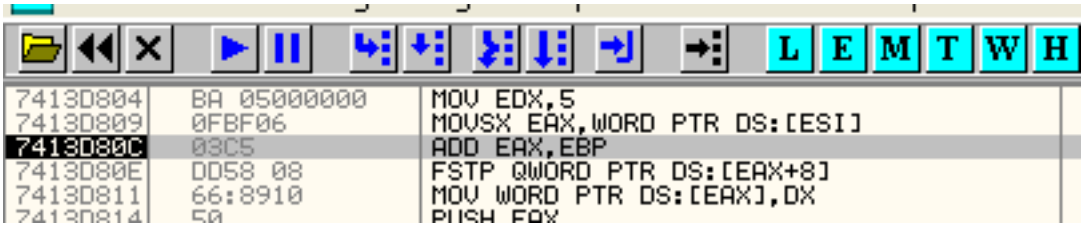
Но видим, что вначале загружаются параметры.



- В данном случае:

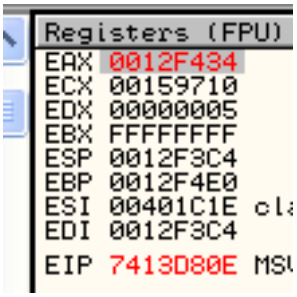


-

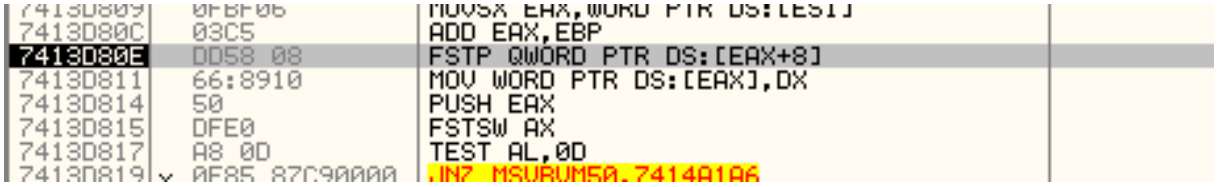


-

Суммируются с EBP, и в EAX остаётся:

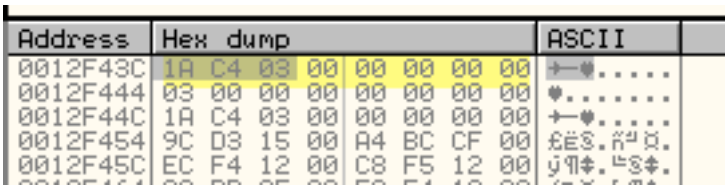


-



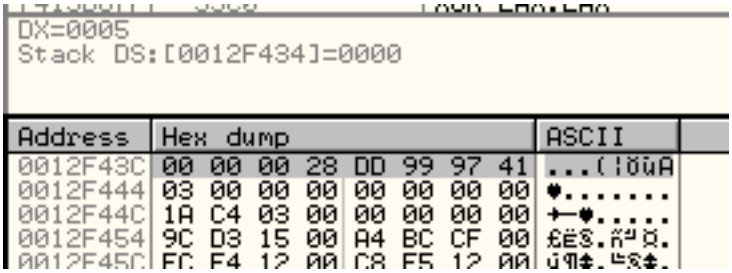
-

FSTP сохраняет первое первое значение из стека плавающей запятой, то есть из ST(0), в указанную ячейку памяти (в данном случае [EAX+8], то есть 12f43c) и делает следующий элемент стека верхним. Ок, мы рассмотрим это позже.



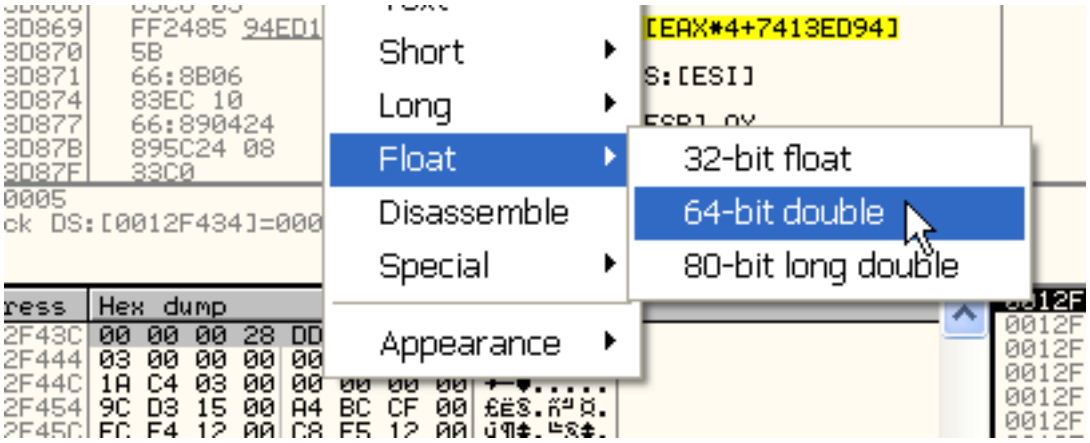
-

После выполнения:

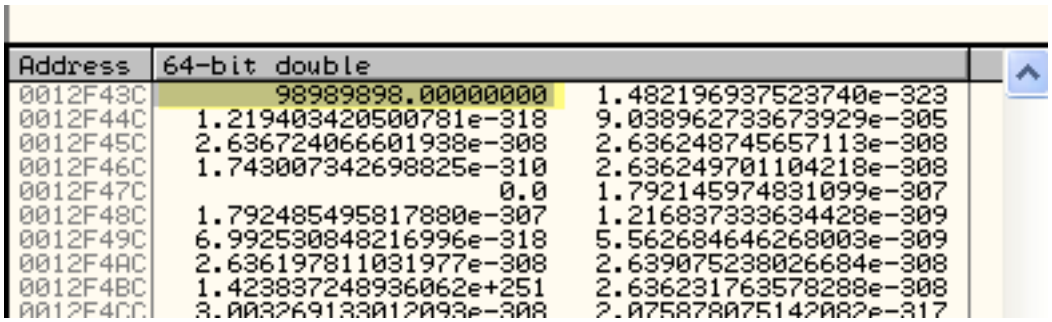


-

Я уже знаю, что у вас могут зародиться кое-какие подозрения о том, что это наш серийный номер, сконвертированный в 64-х битное число. Посмотрим, так ли это, кликнув на правую кнопку мыши.

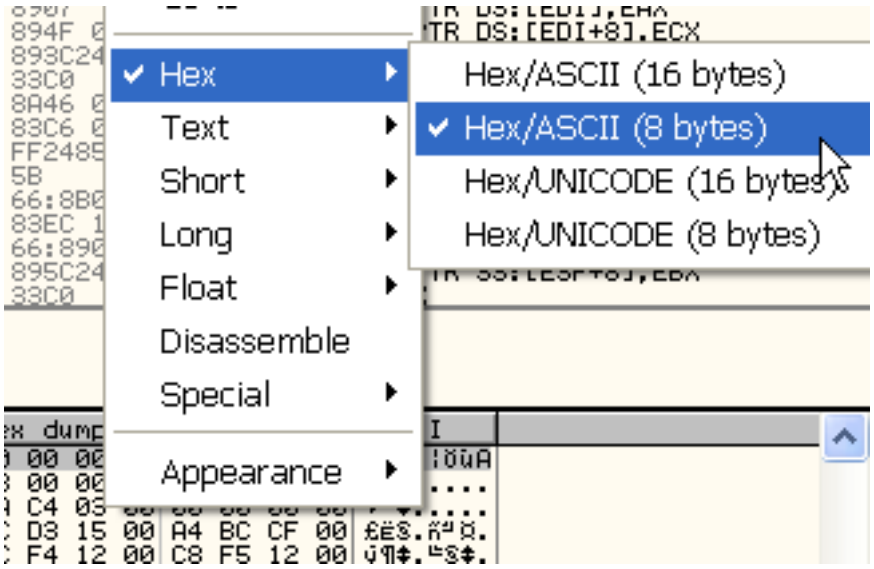


-



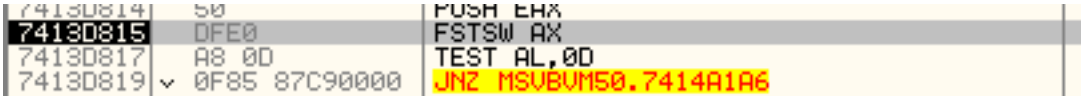
-

Видим, что действительно это неправильный серийный номер, но только в другом представлении – 8 байтов, то есть 64-битном. Логично, раз 4 байта – это 32 бита, то двойное слово состоит из 64-х бит и выглядит по другому.



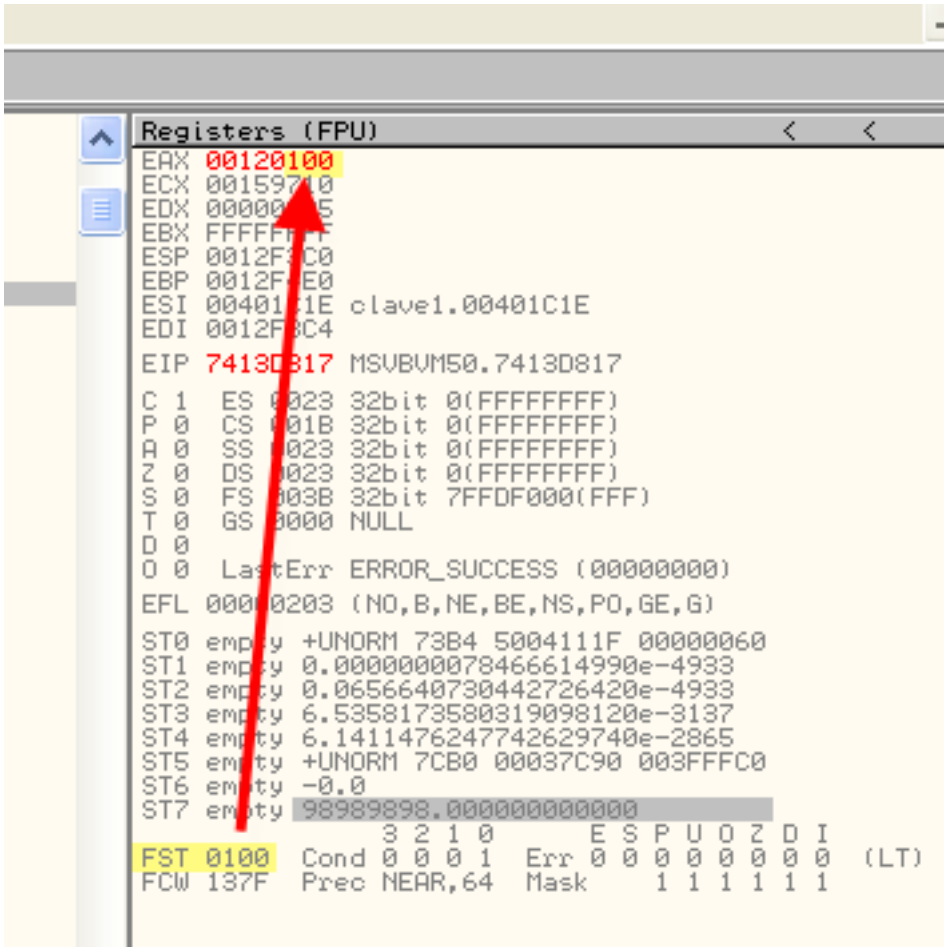
-

Включаем обратно обычный режим представления.



-

Эта инструкция сохраняет значение регистра SR (регистр состояния FPU) в AX. Выполняем:



-

```
7413D819  0F85 87C90000   JNZ MSVBVM50.7414A1A6
7413D81F  33C0            XOR EAX,EAX
7413D821  8A46 02         MOV AL,BYTE PTR DS:[ESI+2]
7413D824  83C6 03         ADD ESI,3
7413D827  FF2485 94ED137  JMP DWORD PTR DS:[EAX*4+7413ED94]
7413D82E  8B0424          MOV EAX,DWORD PTR SS:[ESP]
7413D831  66:8108 0080    OR WORD PTR DS:[EAX],8000
7413D836  33C0            XOR EAX,EAX
7413D838  8A06            MOV AL,BYTE PTR DS:[ESI]
7413D83A  46              INC ESI
```

-

Здесь опкод завершается.

```
401C20: 5d HardType
```

Ок, у меня нет ни малейшей идеи, что это такое, но попытаемся это выяснить.

```
7413D827  FF2485 94ED137  JMP DWORD PTR DS:[EAX*4+7413ED94]
7413D82E  8B0424          MOV EAX,DWORD PTR SS:[ESP]
7413D831  66:8108 0080    OR WORD PTR DS:[EAX],8000
7413D836  33C0            XOR EAX,EAX
7413D838  8A06            MOV AL,BYTE PTR DS:[ESI]
7413D83A  46              INC ESI
7413D83B  FF2485 94ED137  JMP DWORD PTR DS:[EAX*4+7413ED94]
```

-

Ок, всего лишь две строки, которые грузят содержимое ESP в EAX.

```
Registers (FPU)
EAX 0012F434
ECX 00159710
EDX 00000005
EBX FFFFFFFF
ESP 0012F3C0
EBP 0012F4E0
ESI 00401C21 cla
EDI 0012F3C4

EIP 7413D831 MSV

C 0  ES 0023 32b
P 1  CS 001B 32b
A 1  SS 0023 32b
Z 0  DS 0023 32b
S 0  FS 003B 32b
```

-

```
Address   Hex dump                 ASCII
0012F434  05 00 00 00 00 00 00 00  ♣.......
0012F43C  00 00 00 28 DD 99 97 41  ...(¦ÖùA
0012F444  03 00 00 00 00 00 00 00  ♥.......
```

-

Видим, что там находится число, находящееся чуть выше моего трансформированного фальшивого серийного номера. Возможно, что это число указывает, в каком формате он хранится.

```
Address   Hex dump                 ASCII
0012F434  05 80 00 00 00 00 00 00  ♣Ç......
0012F43C  00 00 00 28 DD 99 97 41  ...(¦ÖùA
0012F444  03 00 00 00 00 00 00 00  ♥.......
```

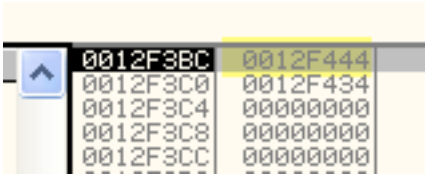
-

Следующий опкод.

```
401C21: 04 FLdRfVar             local_009C
```

То есть PUSH ebp-9c, в данном случае:

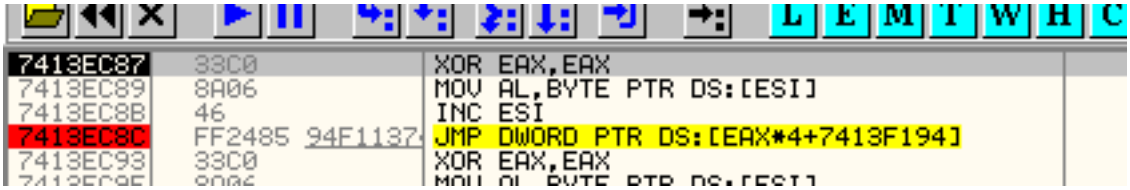
-



-

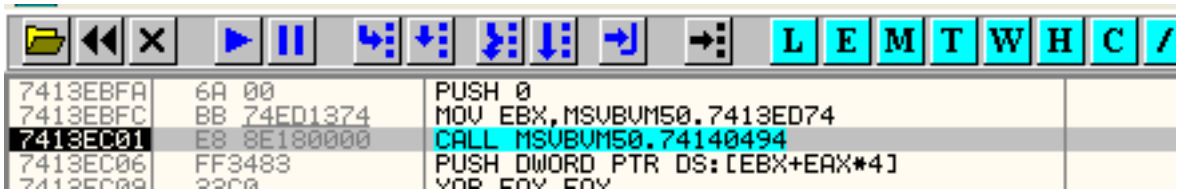
Далее идёт двойной опкод.

```
401C24: Lead0/40 NeVarBool
```



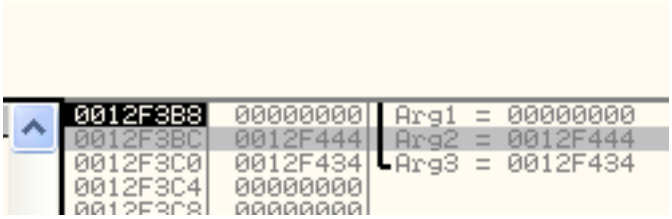
-

Здесь читается второй опкод.



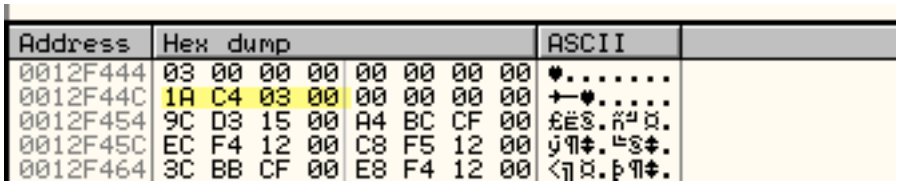
-

Доходим до вызова, а параметры в стеке следующие:

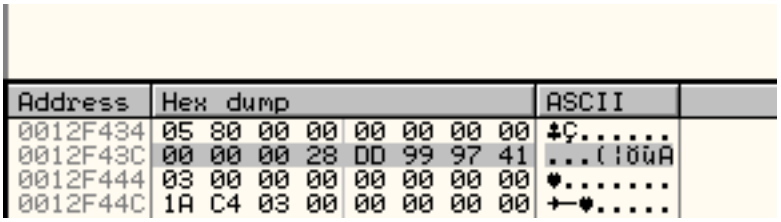


-

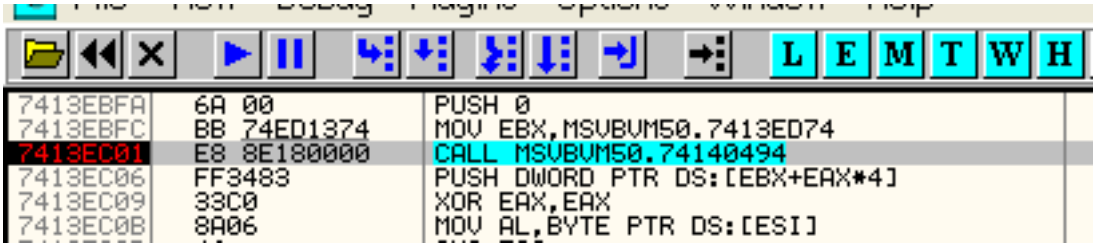
Один – это указатель на правильный серийный номер, другой – на трансформированный неправильный. Будет их сравнение?



-

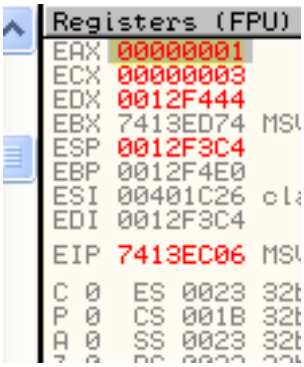


-



-

Устанавливаем BP сюда.

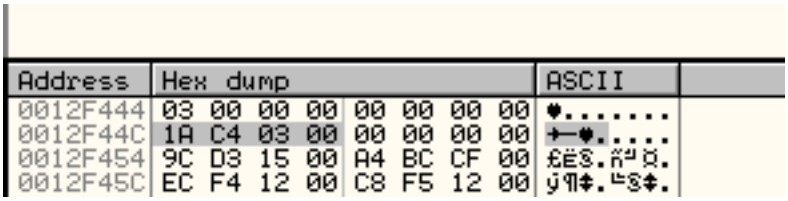


-

Видим, что после выхода из call’a EAX содержит 1.

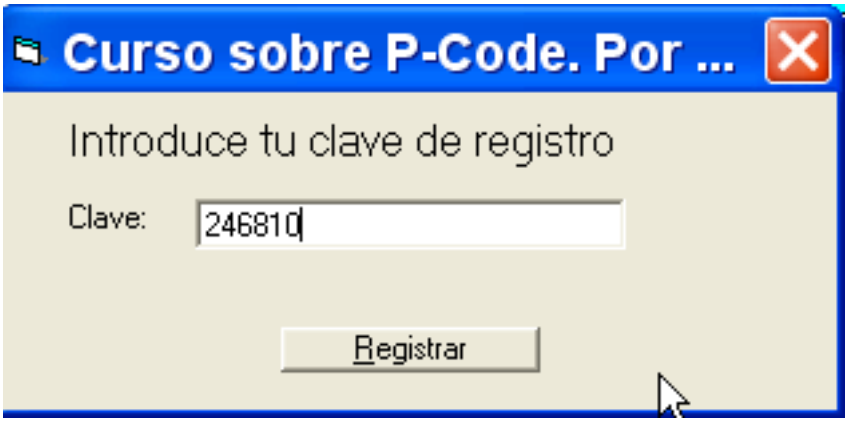
-

И поскольку делается PUSH значения FFFFFFFF в стек, то можно практически безошибочно предположить, что именно здесь проводится сравнение, так что рестартуем крэкми, установив BP, и вводим правильный серийный номер, который был получен из этого сравнения.



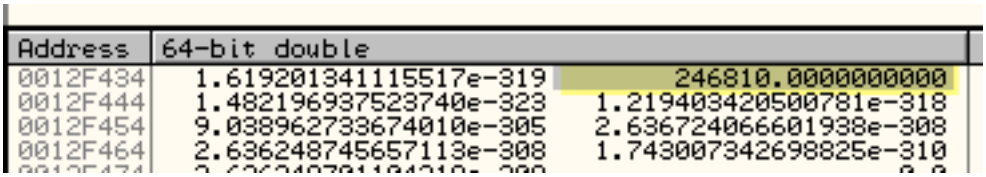
-

03c41a в десятеричной системе – это 246810. Вводим его и снова идём к сравнению.



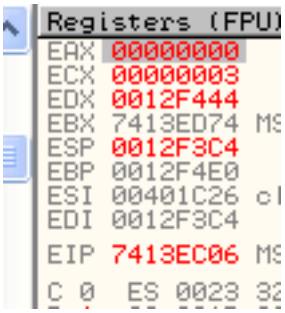
-

Остановившись, видим, что происходит сравнение:



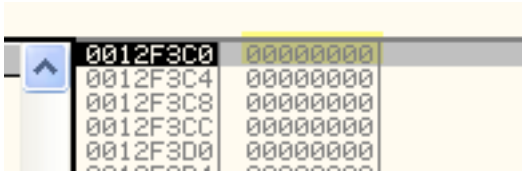
-

246810 со прежним шестнадцатеричным значением. Дело в том, что они в разных форматах, но сверху каждого из них есть число, указывающее, в каком именно хранится значение, поэтому возможно, что внутри каждого call'а они будут трансформированы и сравнены. Смотрим, что у нас получится в результате.



-

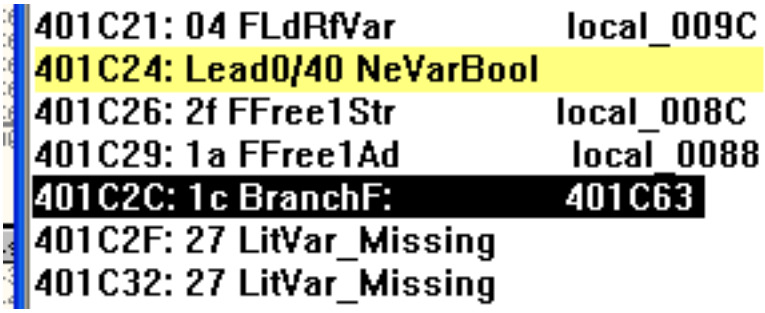
Видим, что теперь у нас EAX = 0 (раньше было 1).



-

И в стек также уходит ноль.

Это означает, что это и есть сравнение, к тому же прямо внизу видим несколько Branch'ей, которые совершают переход в зависимости от того, окошко с каким сообщением надо показать, а потом идёт освобождение строк и использованных значений.



-

Вы можете сказать, что было слишком много работы, но это был первый раз, поэтому мы подробно рассматривали каждый опкод, и это необязательно повторять в будущем. Отладчик WKT может показать вам имена опкодов. Вы не знаете, что делает каждый из них, но с помощью OllyDbg можете определить, для чего они служат, и получить представление о том, как работает программа.

В следующей части мы рассмотрим крэкми “clave 2”, используя тот же метод. Мне бы хотелось, чтобы вы попробовали самостоятельно его сделать, но если не получится, то в следующей части вы сможете прочитать решение этого крэкми.

Введение в крэкинг с нуля, используя OllyDbg - Глава 30

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-07-08

Файлы к статье (находится в архиве `wasm_files.zip`: `files/0492/ollydbg30.zip`)

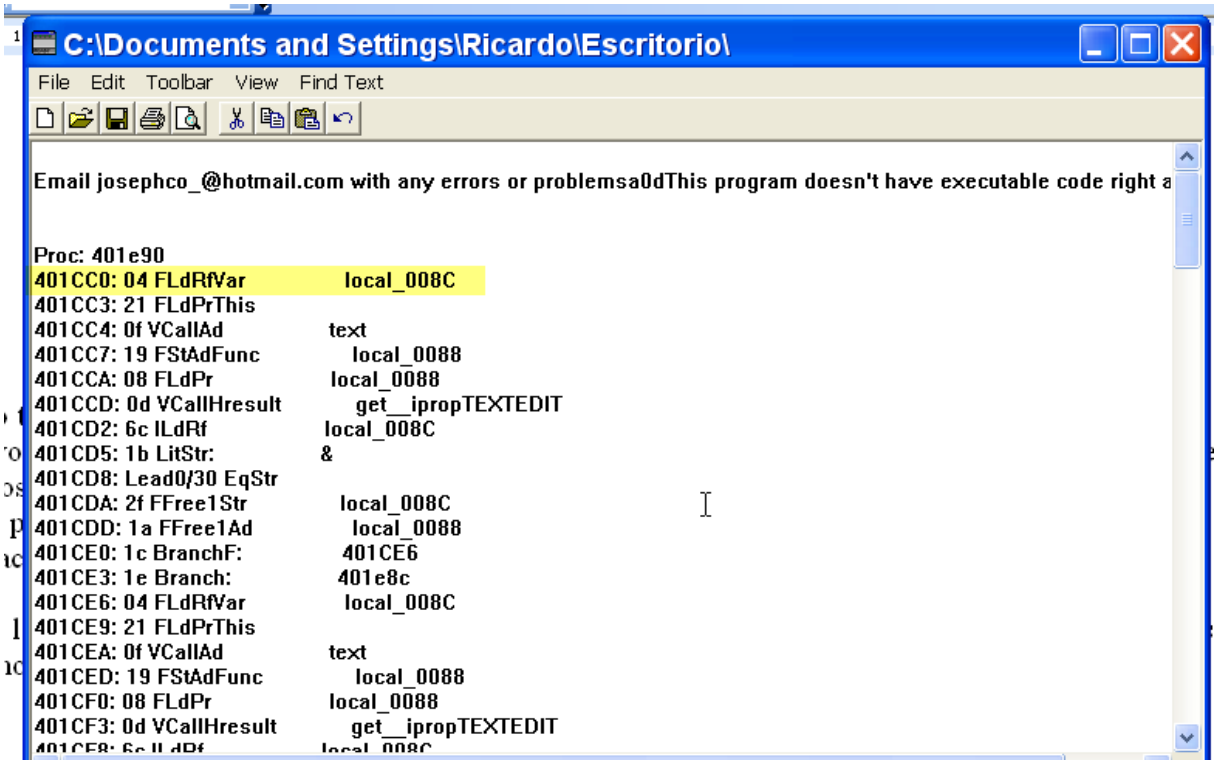
Ок, после того, как попрактиковались, и если вы ещё живы, продолжаем с PCODE.

У нас есть ещё кое-какие опкоды, собранные из туториалов JBDUC'a.

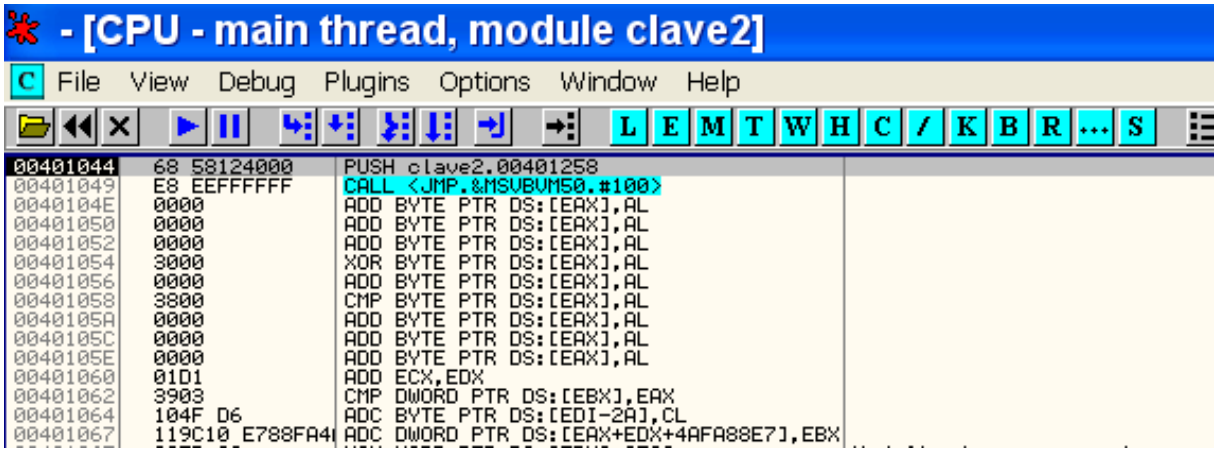
• 6c -> ILdRf Поместить адрес в стек
• 1b -> LitStr5 Поместить строку в стек
• fb -> Lead0 Сравнить две строки (хе-хе, для чего это могло служить □)
• 30 -> EqStr Сравнить две строки(хе-хе, для чего это могло служить □)
• 2f -> FFree1Str Освободить используемую память
• 1a -> FFree1Ad Освободить используемую память
• 0f -> VCallAd Выполнить опкод в виртуальной машине
• 1c -> BranchF Сделать переход, если предшествующее сравнение дало ложь (аналог jne/jnz в ассемблере)
• 1d -> BranchT Сделать переход, если предшествующее сравнение дало истину (аналог je/jz в ассемблере)
• 1e -> Безусловный переход (хе-хе, угадайте, для чего её можно использовать)
• fc -> Lead1 Прерывает выполнение программы (хе-хе, прекрасно...)
• c8 -> End Прерывает выполнение программы (хе-хе, прекрасно...)
• f3 -> LitI2 Сохраняет заданное число типа Integer в стек
• f4 -> LitI2_Byte Конвертирует Byte-значение в Integer и помещает его в стек
• 70 -> FStI2 Сохраняет последний Integer, помещённый в стек, в заданную глобальную переменную
• 6b -> FLdI2 Загружает в стек Integer из заданной локальной переменной
• a9 -> AddI2 Складывает два последних Integer'a, помещённых в стек, на вершину которого помещает результат
• ad -> SubI2 Использует для вычитания два последних Integer'a, помещённых в стек, и кладёт в последний результат
• b1 -> MulI2 Умножает два последних Integer'a, помещённых в стек, и кладёт в последний результат. Думаю, что если переполнение игнорируется.

Ок, есть ещё опкоды, которые мы сейчас рассматривать не будем. К статье прилагает файл под названием "P-CODE OPCODES", который, предположительно, распространялся Микрософтом. Он может немного помочь, так как в нём перечислены опкоды и что они делают (но не все, хе-хе), так что если встретим незнакомый опкод, то можем посмотреть, есть ли там описание.

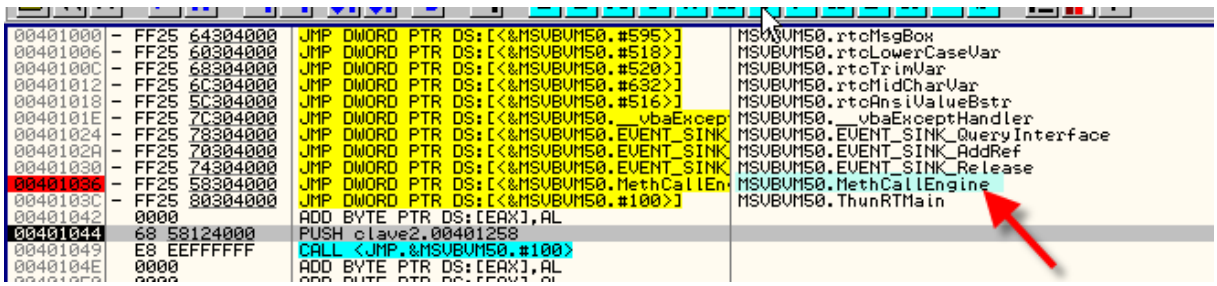
Как я и обещал вам, сначала рассмотрим крэкми «clave 2», которое осталось у нас в качестве домашнего упражнения с прошлого раза. Посмотрим, можем ли мы получить листинг с помощью EXDEC'a.



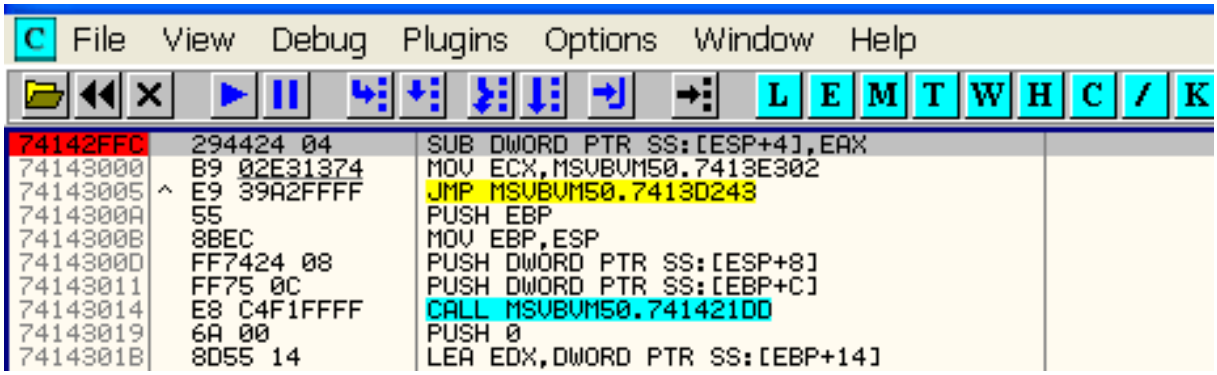
Вот он, и видим, что начинается с 401cc0. Не будем слепо верить EXDEC'у, так как бывали случаи, когда он обманывал. Посмотрим, сможем ли мы найти первый опкод вручную, как мы это делали в предыдущей части.



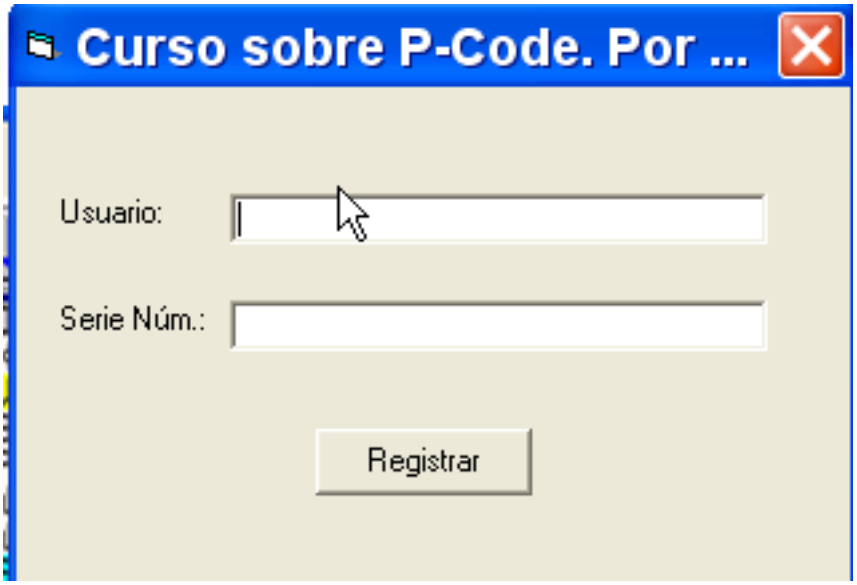
Смотрим, находится ли переход на API-функцию MethCallEngine за EP.



Вот он. Становимся на него, делаем FOLLOW, чтобы перейти к API-функции и также устанавливаем там BP.

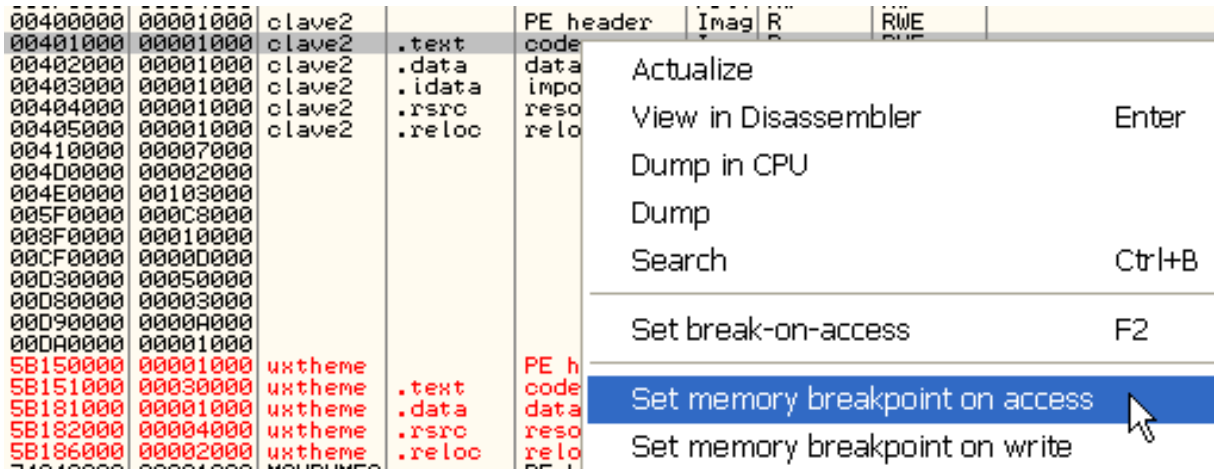


Готово, теперь делаем RUN, чтобы остановиться на одном из установленных BP.

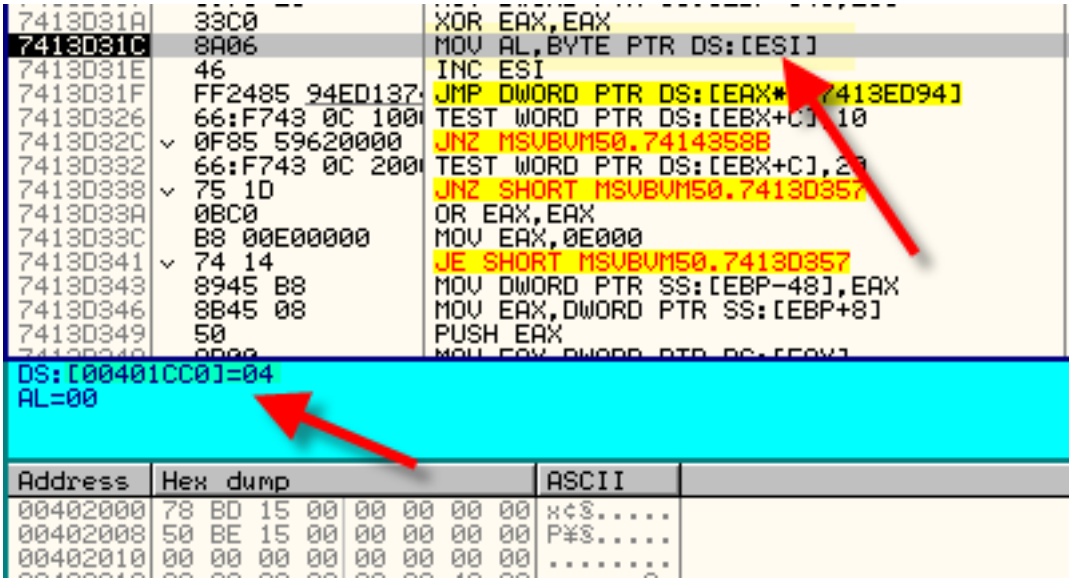


Окно появляется до того, как сработает останов, дело в том, что окно появляется до того, как начинается P-CODE, хотя нужно пояснить, что такое бывает не всегда, может сначала сработать останов, а потом появиться окно, но в данном случае оно сделало это первым. Вводим имя и неправильный серийный номер.

После нажатия на «REGISTRAR» срабатывает останов на JMP. Идём в первую секцию (помните, что не надо использовать пропатченный OllyDbg для OEP'ов) и устанавливаем "BPM ON ACCESS" на секцию кода.



Теперь жмём RUN, пока не встретится знакомая инструкция, где читается опкод.



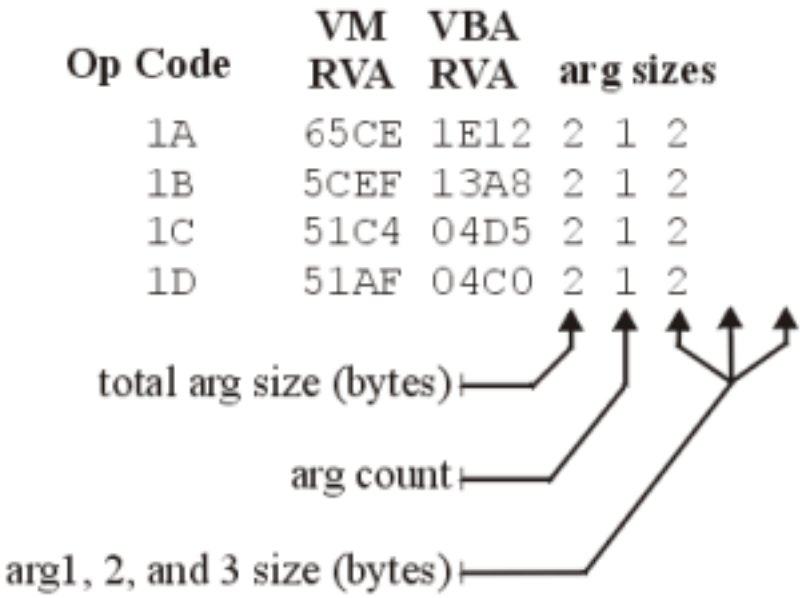
Вот тут, помним, что ESI всегда должен указывать на опкод (который сам перемещается в AL).

Как видим, EXDEC не ошибся, и первый опкод – это 04 и начинается с 401cc0.

```
Proc: 401e90
401CC0: 04 FLdRfVar               local_008C
```

Помним, что 04 – это просто PUSH, в данном случае его аргументом является EBP-8C, как это видно из пояснения рядом с опкодом

```
04    567B 0B8E 2 1 2      Push arg
```



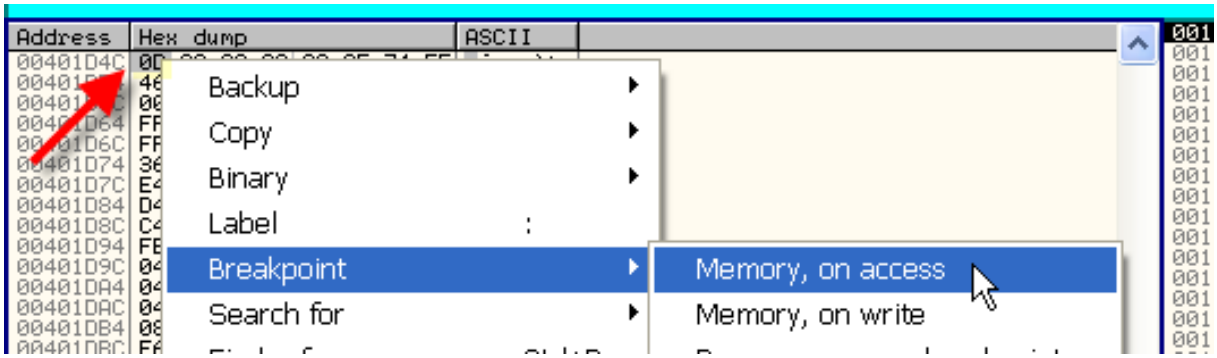
Из этого нам становится понятно, что каждая цифра, в нашем случае 0B8E, является RVA опкода, 2 – это количество байтов, которые в общей сложности занимают параметры, 1 нам говорит, что аргумент только один, и последнее значение (2) указывает нам размер, занимаемый каждым аргументом в отдельности.

Хорошо, вот PUSH EBP-8C, продолжаем изучать исходник, пока без трассировки шаг за шагом. Смотрим.

```
401D4C: 0d VCallHresult           get__ipropTEXTEDIT
```

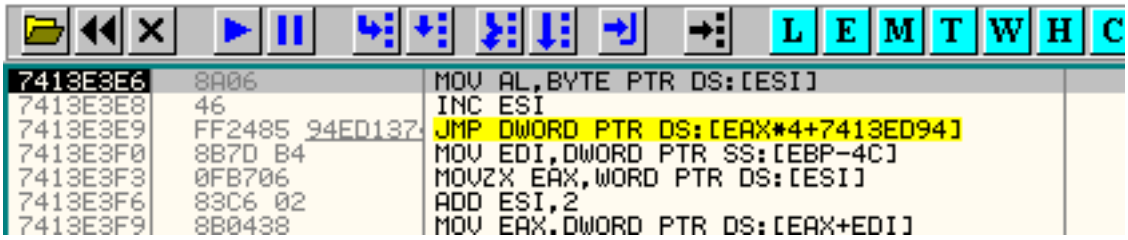
```
401DFB: 0d VCallHresult           get__ipropTEXTEDIT
```

Видим, что используется два вызова для считывания введённого в окне регистрации, вероятно, что в первый раз читает имя, а во второй – неправильный серийник, использованный нами. Устанавливаем BPM ON ACCESS на первом вызове, то есть на 401d4C.



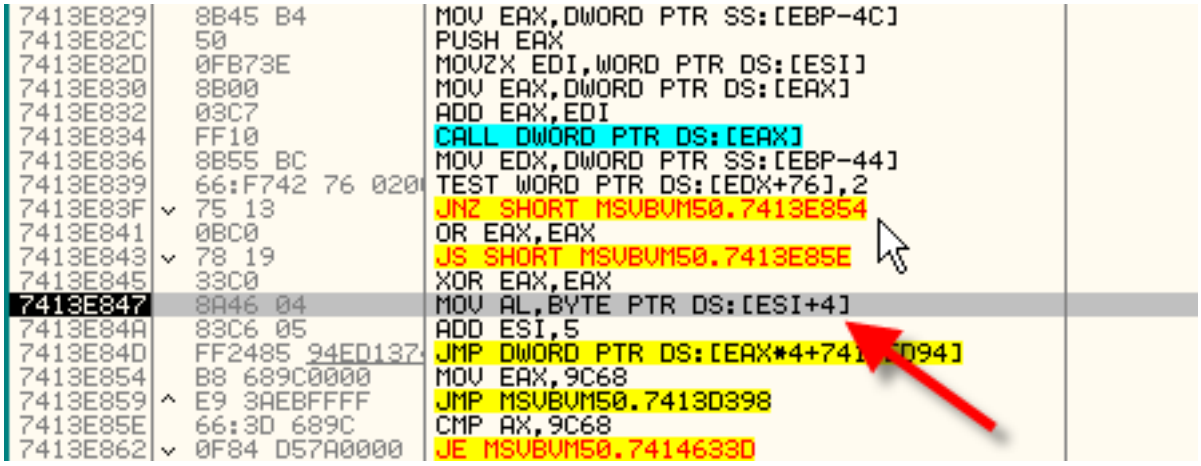
Allí esta el opcode y le coloco el BPM y doy RUN.

Вот опкод, устанавливаем на него BPM и делаем RUN.



Bueno ahora traceemos hasta el siguiente opcode, a ver si ingresa el nombre.

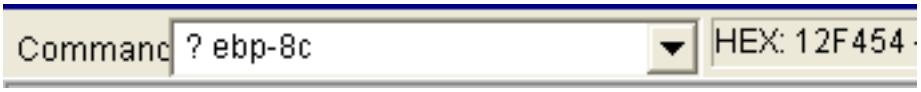
Ок, теперь трассируем до следующего опкода, чтобы посмотреть, что будет происходить с именем.



Дошли до следующего опкода, смотрим стек, чтобы узнать, что там находится

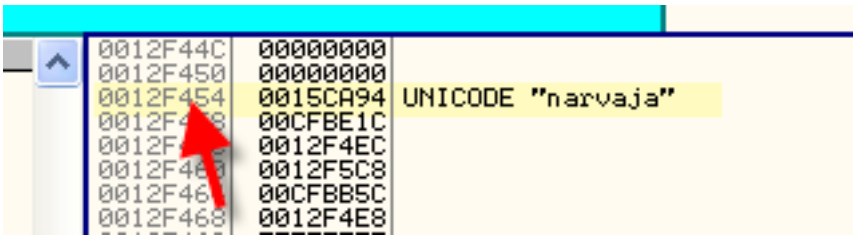
```
401D4C: 0d VCallHresult           get__ipropTEXTEDIT
401D51: 3e FLdZeroAd              local_008C
```

Как видим, продолжается работа с локальной переменной 8c (или EBP-8C). Проверим, что там сохранено. На моей машине в ebp-8c содержит 12f454.



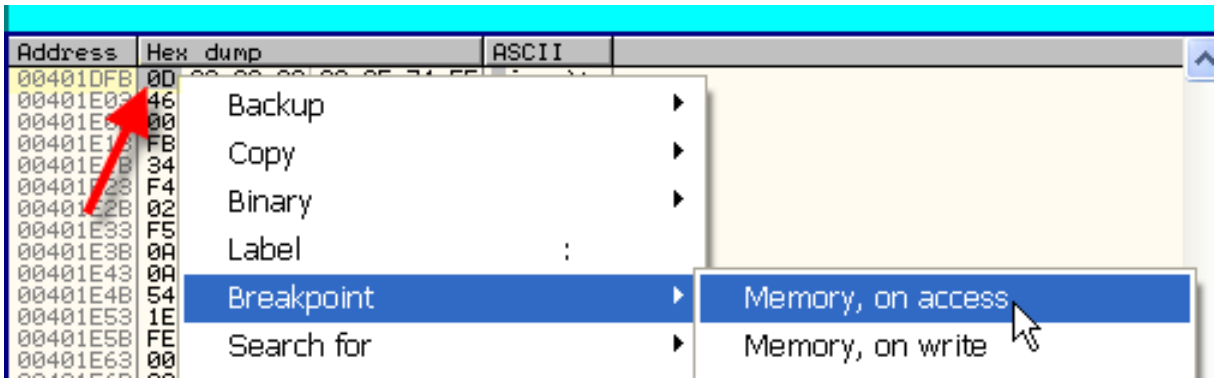
Busco en el stack esa dirección a ver si esta allí mi nombre

Ищем этот адрес в стеке и видим, что там находится моё имя.

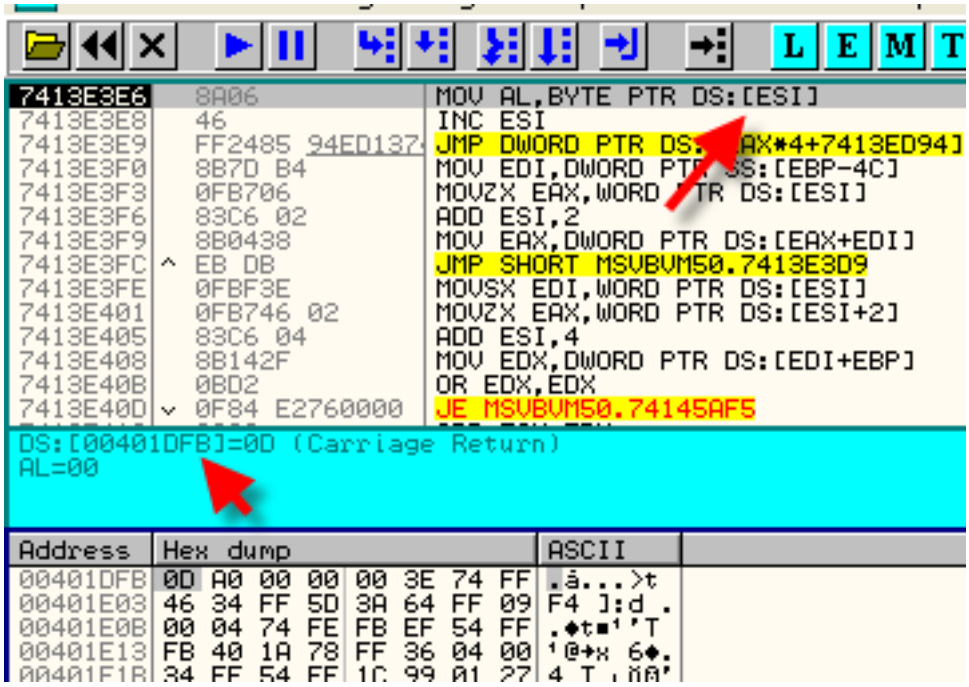


Конечно же, оно находится здесь, так что всё идёт хорошо. Раз здесь находится моё имя, то логично предположить, что в следующем будет идти работа с серийником, поэтому устанавливаем на второй опкод BPM ON ACCESS.

```
401DFB: 0d VCallHresult           get__ipropTEXTEDIT
```



Готово, теперь жмём RUN, чтобы оказаться рядом с этим опкодом.



Как и в предыдущем случае, трассируем до следующего опкода.

```
7413E83F  v 75 13          JNZ SHORT MSVBVM50.7413E854
7413E841    0BC0           OR EAX,EAX
7413E843  v 78 19          JS SHORT MSVBVM50.7413E85E
7413E845    33C0           XOR EAX,EAX
7413E847    8A46 04        MOV AL,BYTE PTR DS:[ESI+4]
7413E84A    83C6 05        ADD ESI,5
7413E84D    FF2485 94ED137 JMP DWORD PTR DS:[EAX*4+7413ED94]
7413E854    B8 689C0000    MOV EAX,9C68
7413E859  ^ E9 3AEBFFFF    JMP MSVBVM50.7413D398
7413E85E    66:3D 689C     CMP AX,9C68
7413E862  v 0F84 D57A0000  JE MSVBVM50.7414633D
7413E868    57             PUSH EDI
7413E869    0FB75E 02      MOVZX EBX,WORD PTR DS:[ESI+2]
7413E86D    8B55 AC        MOV EDX,DWORD PTR SS:[EBP-54]
```

И смотрим, сохранён ли в используемой переменной неправильный серийный номер.

```
401DFB: 0d VCallHresult           get__ipropTEXTEDIT
401E00: 3e FLdZeroAd              local_008C
```

Как и в прошлый раз используется EBP-8C.

```
0012F444  00000000
0012F448  00000000
0012F44C  00000000
0012F450  00000000
0012F454  0015CA94  UNICODE "989898"
0012F458  00CFBCC4
0012F45C  0012F4EC
0012F460  0012F5C8
0012F464  00CFBB5C
0012F468  0012F4E8
```

Хорошо, вот мы и дошли до места, куда помещён наш неправильный серийник. Видим, что зная опкоды, не обязательно трассировать всю программу, можем просто установить BPM и оказаться в нужной части программы. В прошлой главе мы трассировали всё подряд, чтобы понять механизмы, согласно которым работает PCODE, но как правильно, нам не нужно делать это. Как мы увидим в следующей главе, когда исследуем большую программу, будем локализовывать интересующую нас часть программы и работать именно с ней.

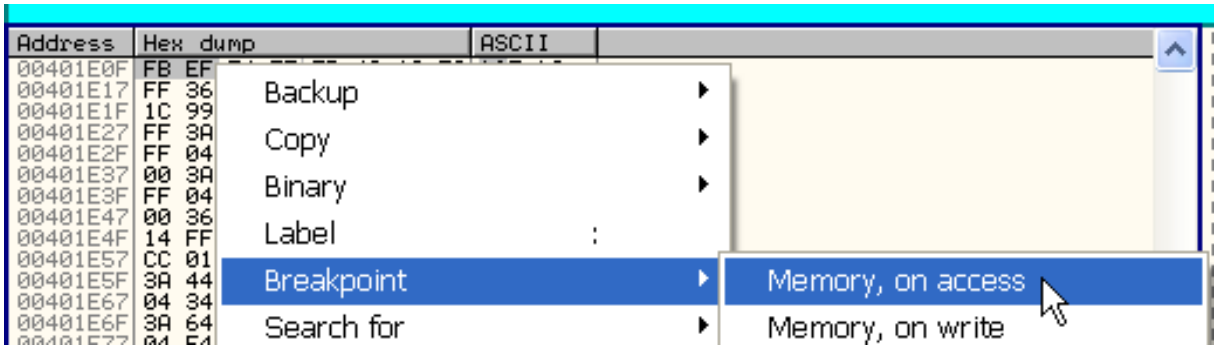
```
401E0F: Lead0/ef ConcatVar
401E13: Lead0/40 NeVarBool
401E15: 1a FFree1Ad               local_0088
401E18: 36 FFreeVar         local_00CC local_00AC
401E1F: 1c BranchF:               401E59
```

Далее видим, что здесь есть все признаки сравнения, потом освобождение локальных переменных с помощью FREE, а затем условный переход, который ведёт в 401e59, где вызывается rtcMsgBox с сообщением о правильном серийном номере, а если перехода не происходит, то отображается сообщение о том, что серийник неверен.

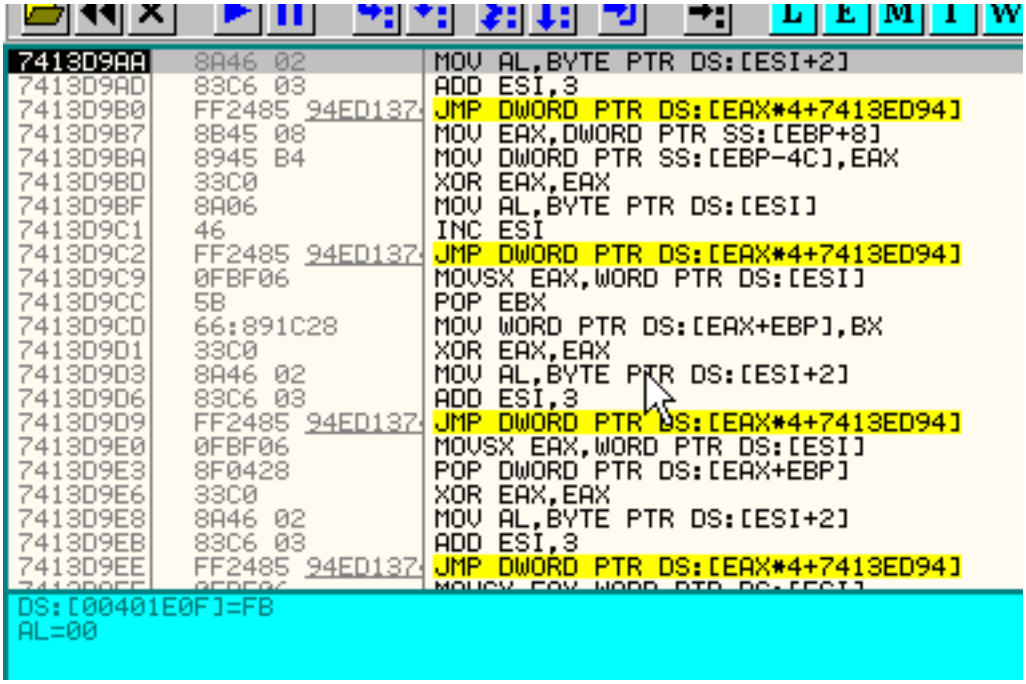
```
401E0F: Lead0/ef ConcatVar
401E13: Lead0/40 NeVarBool
401E15: 1a FFree1Ad             local_0088
401E18: 36 FFreeVar         local_00CC local_00AC
401E1F: 1c BranchF:              401E59
401E22: 27 LitVar_Missing
401E25: 27 LitVar_Missing
401E28: 3a LitVarStr:           ( local_00BC ) P-Code
401E2D: 4e FStVarCopyObj            local_00CC
401E30: 04 FLdRfVar             local_00CC
401E33: f5 LitI4:              0x10  16  (....)
401E38: 3a LitVarStr:           ( local_009C ) Clave No Válida!
401E3D: 4e FStVarCopyObj            local_00AC
401E40: 04 FLdRfVar             local_00AC
401E43: 0a ImpAdCallFPR4:            _rtcMsgBox
401E48: 36 FFreeVar         local_00AC local_00CC local_00EC local_010C
401E53: 1e Branch:               401e8c
401E56: 1e Branch:               401e8c
401E59: 27 LitVar_Missing
401E5C: 27 LitVar_Missing
401E5F: 3a LitVarStr:           ( local_00BC ) P-Code
401E64: 4e FStVarCopyObj            local_00CC
401E67: 04 FLdRfVar             local_00CC
401E6A: f5 LitI4:              0x30  48  (...0)
401E6F: 3a LitVarStr:           ( local_009C ) Clave Correcta!!
401E74: 4e FStVarCopyObj            local_00AC
401E77: 04 FLdRfVar             local_00AC
401E7A: 0a ImpAdCallFPR4:            _rtcMsgBox
401E7F: 36 FFreeVar         local_00AC local_00CC local_00EC local_010C
```

Тут всё совершенно понятно. Вероятное сравнение и условный переход, в зависимости от которого отображается окошко с соответствующим сообщением, так что устанавливаем сюда BPM ON ACCESS.

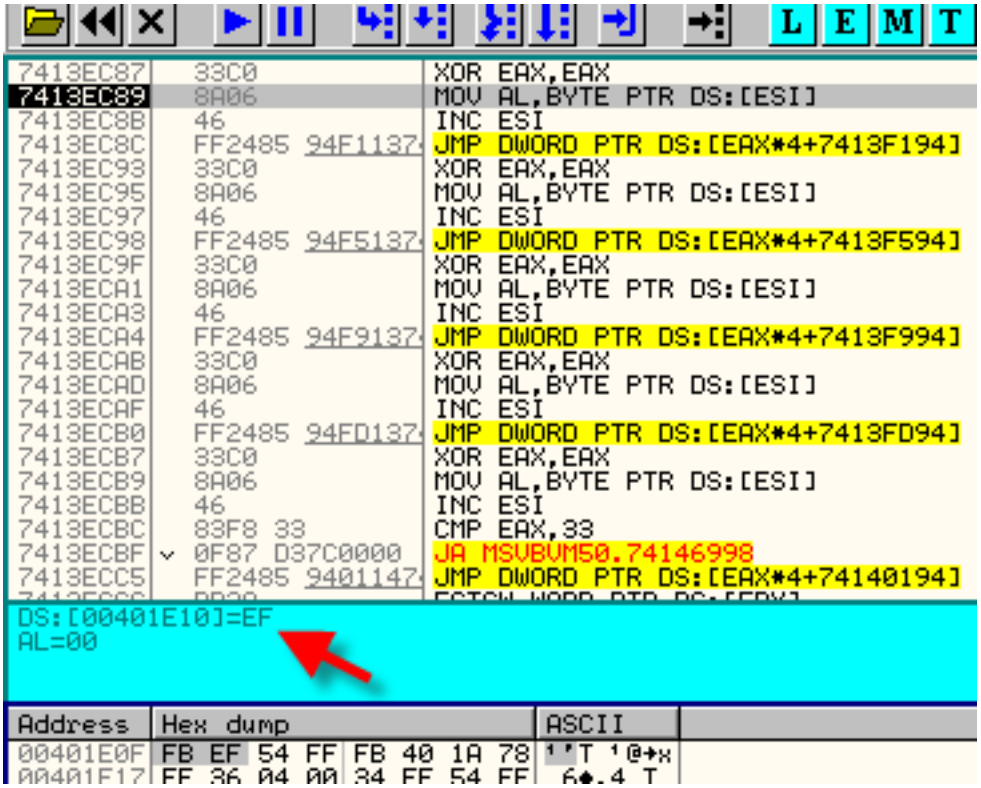
```
401E0F: Lead0/ef ConcatVar
401E13: Lead0/40 NeVarBool
```



Ок, делаем RUN.



Останавливаемся тут, в первом опкоде и трассируем до второго, который производит некую операцию.



Второй опкод равен EF.

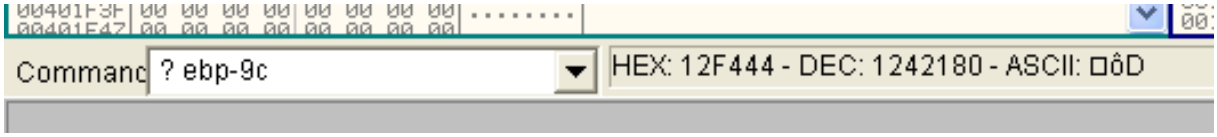
Смотрим в списке опкодов.

```
FB EF 6BAB 25AD 2
FB F0 6B99 259B 0          vbaStrCat
FB F1 C423 B981 0          Push [FC0D134]
```

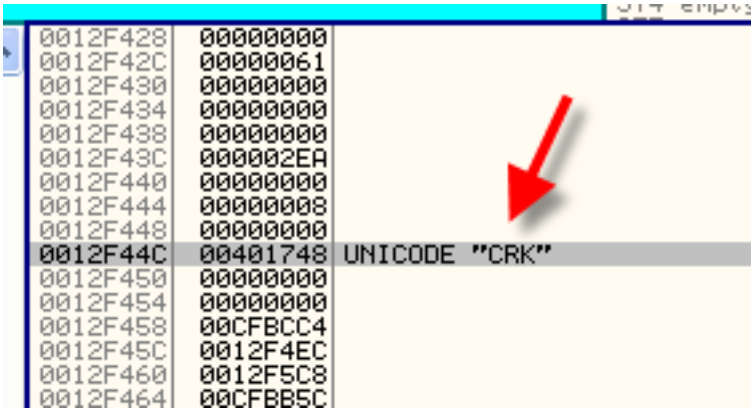
Что такое FB EF нам ещё не совсем понятно, EXDEC говорит нам что-то о конкатенации переменных, смотрим.

```
401E07: 3a LitVarStr:              ( local_009C ) CRK
401E0C: 04 FLdRfVar                local_018C
401E0F: Lead0/ef ConcatVar
401E13: Lead0/40 NeVarBool
```

Видим, что прямо до этого опкода идёт работа с двумя локальными переменными, в одной из которых находится строка CRK, а другая является EBP-018C. Выясним, что находится в каждой из вышеупомянутых переменных.

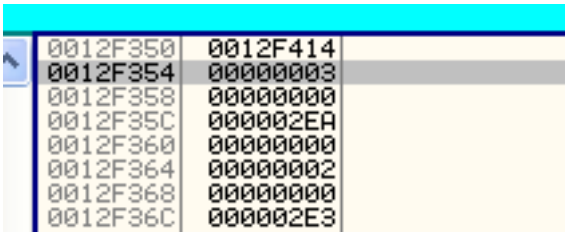


Первая переменная – это EBP-9C, которая на моей машине равна 12f444 и находится там следующее:

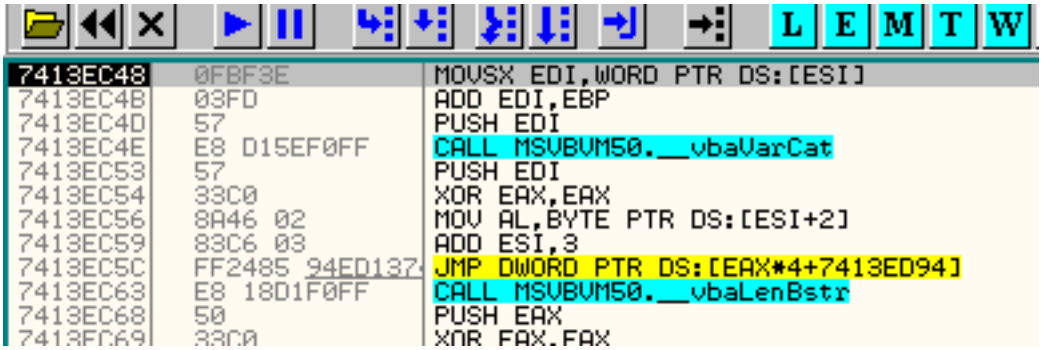


Как мы знаем, в случае с переменными сначала идёт байт, указывающий тип, в данном случае это 3, а ещё выше находится сама переменная, которая равна 2EA.

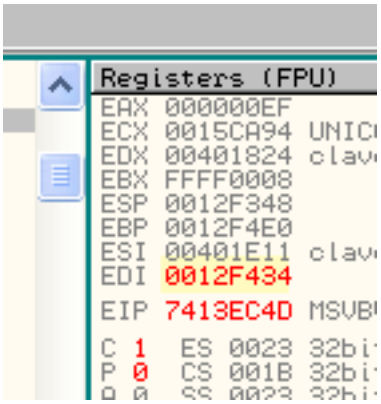
Входим в опкод.



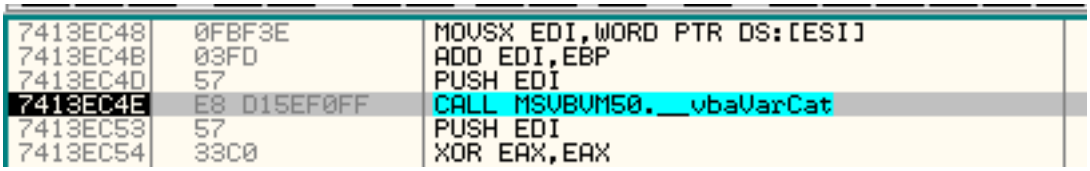
Здесь читаются параметры.



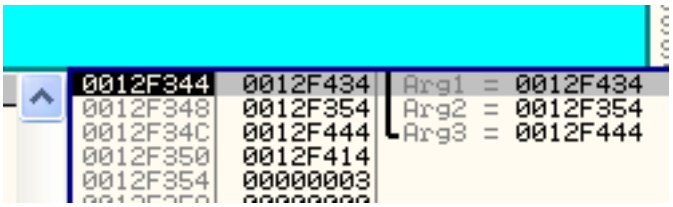
После сложения с EBP получается 12f434.



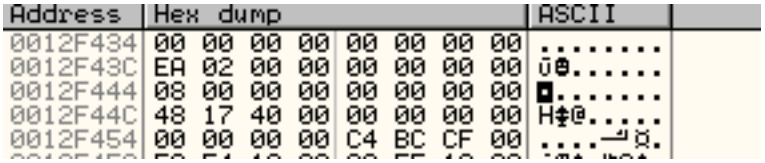
Доходим до API-функции vbaVarCat, у которой три аргумента в стеке.



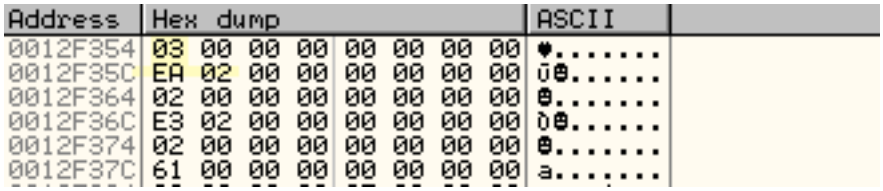
Смотрим, что находится в каждом из них. Отметим, что когда идёт работа с переменной, то отображается маленькая структура, в которой первый байт задаёт её тип.



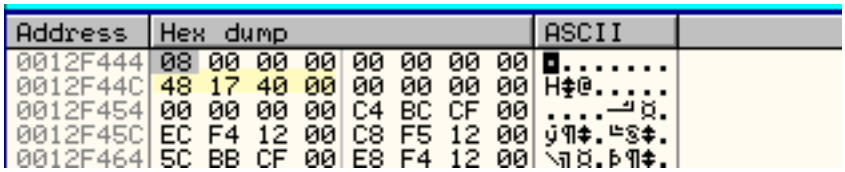
Это первый аргумент.



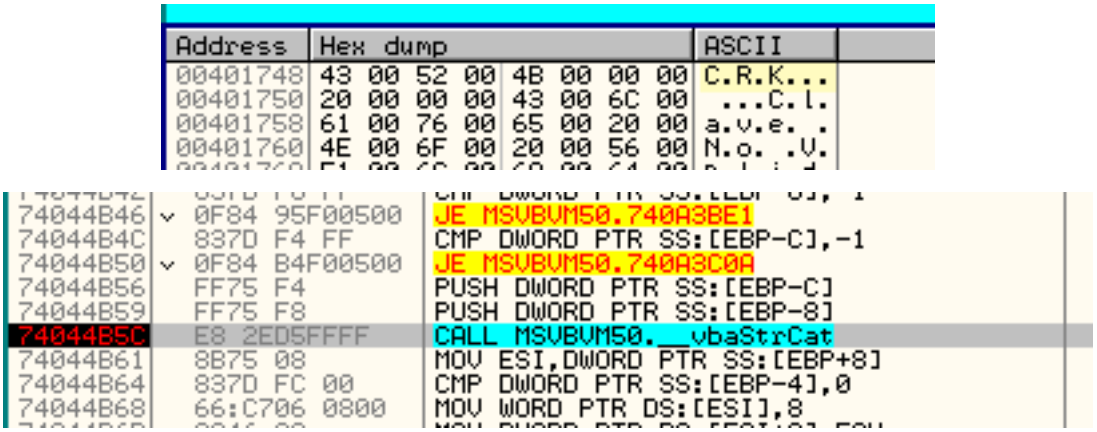
Во втором сначала идёт 3, а потом 02EA, задающее значение.



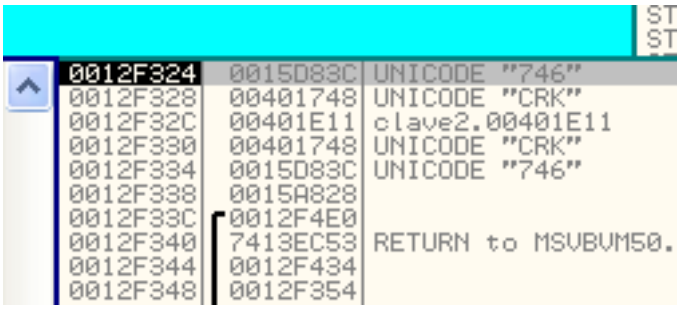
А в третьем в начале 8, означающее, что 4017648 – это указатель на строку CRK.



Ок, у нас тут мешанина из переменных, посмотрим, что останется. Если войдём в функцию vbaVarCat, до окажемся у внутренней API-функции, которая ясно покажет нам, что будет объединяться.



То есть предстоит объединение CRK с 746. А теперь выясним, что значит параметр 02EA.



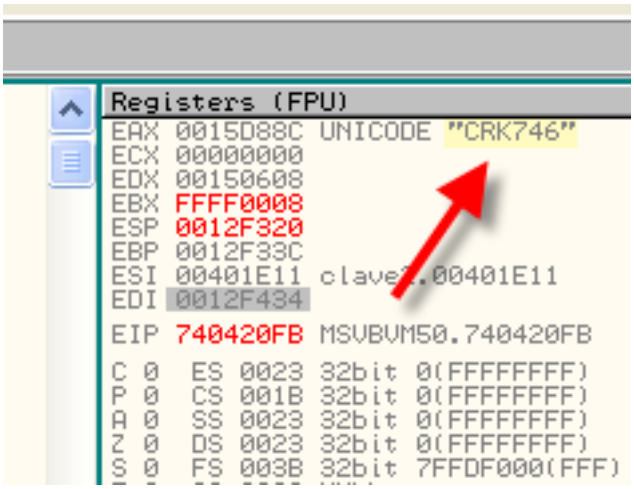
Lo transforma a string ya que 02EA es

Он конвертируется в строку. 02EA – это:

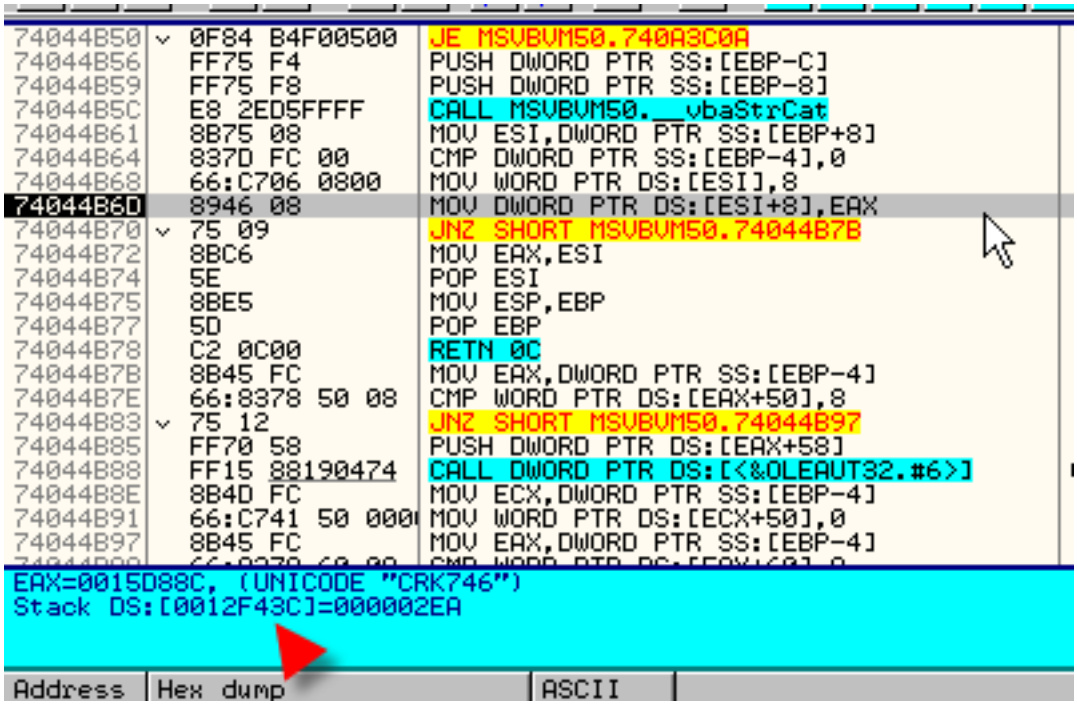
То есть vbaVarCat в данном случае получила строку и численную переменную, которая конвертируется в строку, после чего обе объединяются.

Продолжаем трассировать с помощью F8.

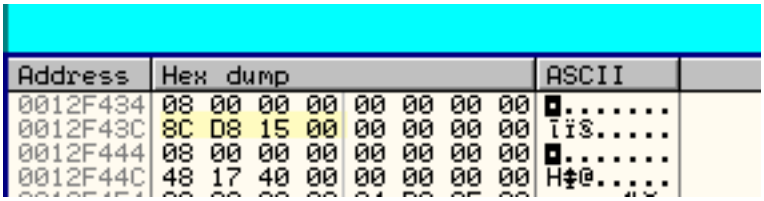
Дойдя до RET'а из функции, видим:



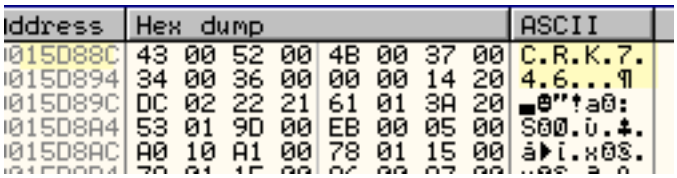
Обе переменных объединились в одну строку.



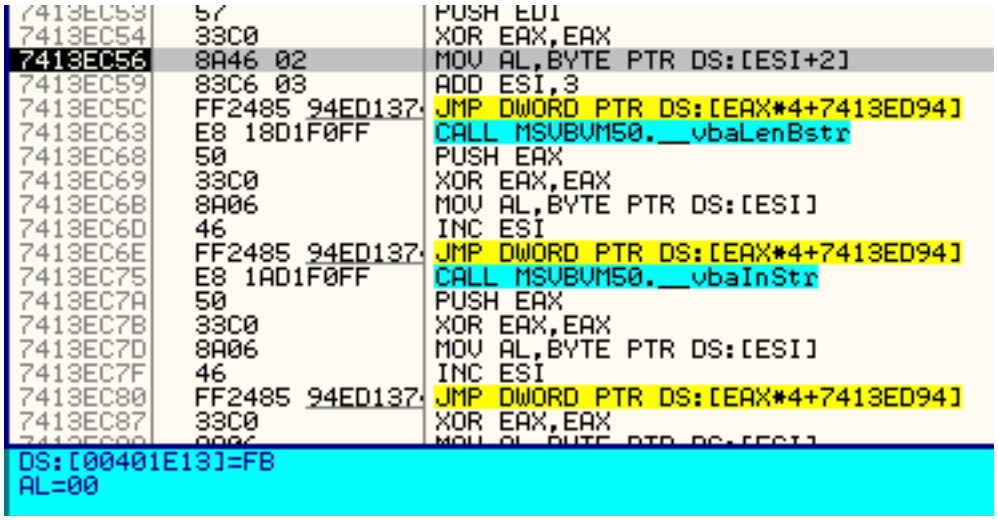
И, как и раньше, до начала следующего опкода это значение сохраняется в качестве первого аргумента.



Видим, что первый аргумент сейчас имеет тип 8, то есть строку, и указывает на 15d88c, то есть на сконкатенированную строку.

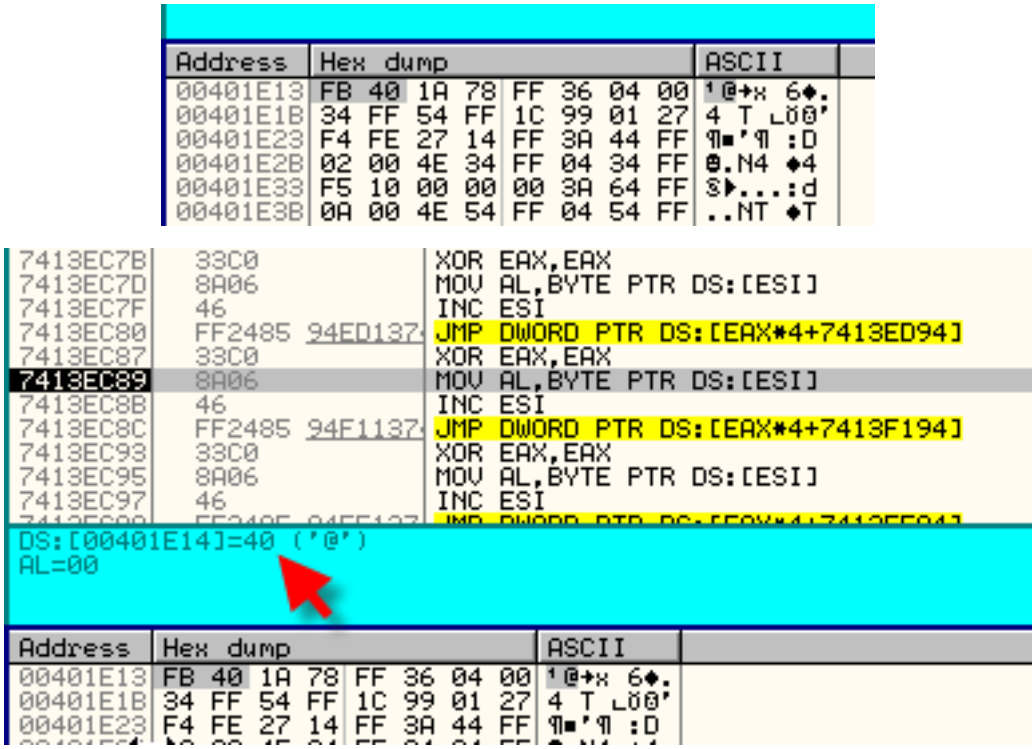


Ок, теперь снова должно произойти сравнение, быстро доходим до следующего опкода.

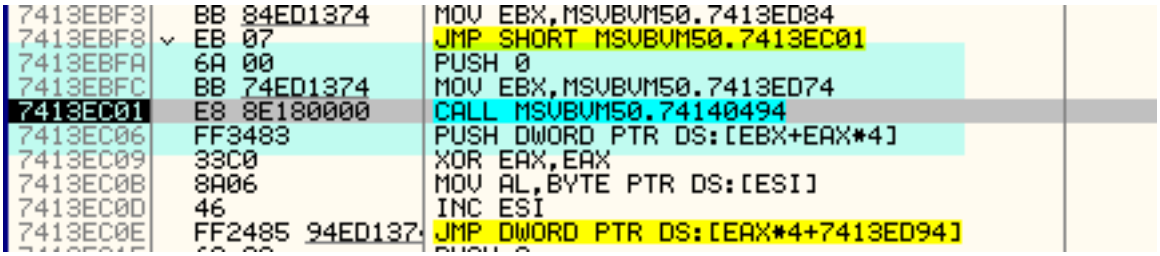


```
401E13: Lead0/40 NeVarBool
```

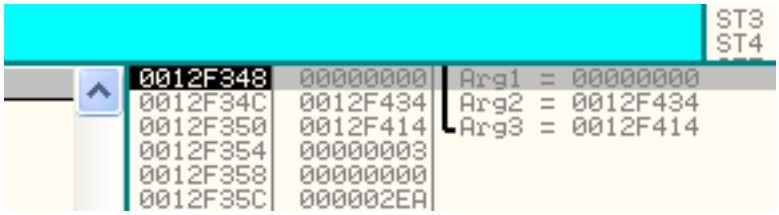
Это FB40, и он двойной. Трассируем, пока не дойдём до считывания второго опкода.



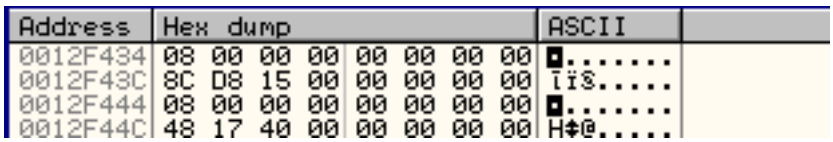
Конечно, список опкодов от Микрософта ничего нам о нём не говорит, так что трассируем опкод, чтобы узнать, что он делает.



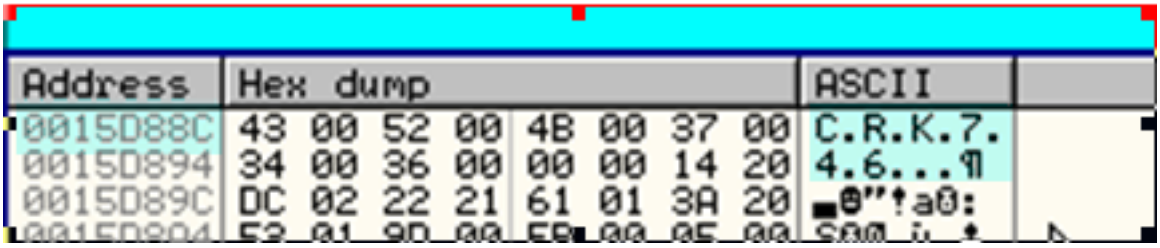
Видим, что в опкоде всего один вызов, после которого сразу следует завершение. Смотрим параметры вызова.



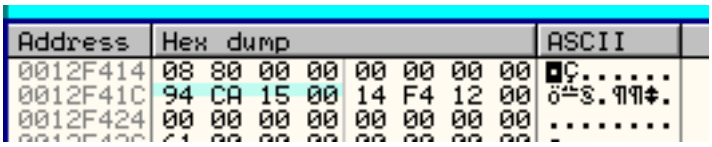
Первый равен нулю, а второй 12f434. Смотрим, что там находится.



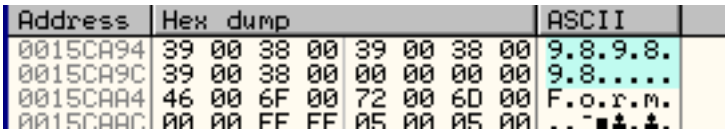
Ок, 08 говорит нам, что это строка, теперь осталось посмотрим, на что указывает 15d88c.



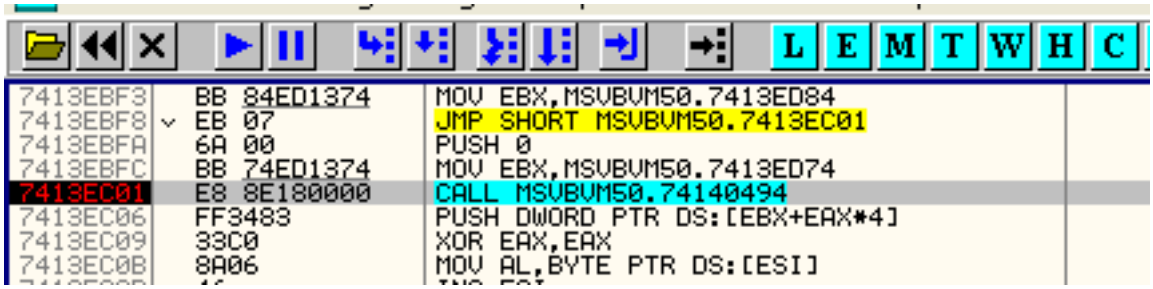
Вот строка, смотрим следующий аргумент.



В данном случае 15ca94 указывает на строку с нашим неправильным серийным номером.

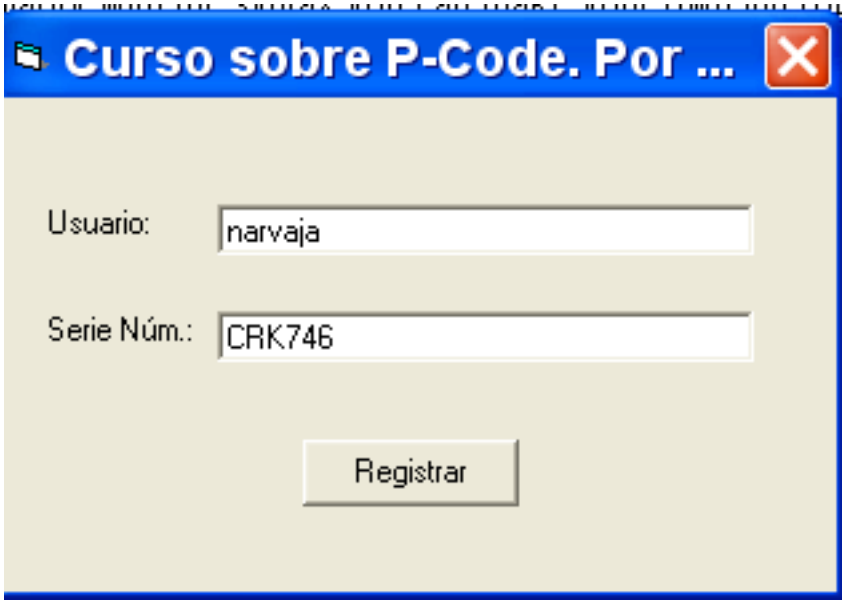


Похоже, что это сравнение двух строк.

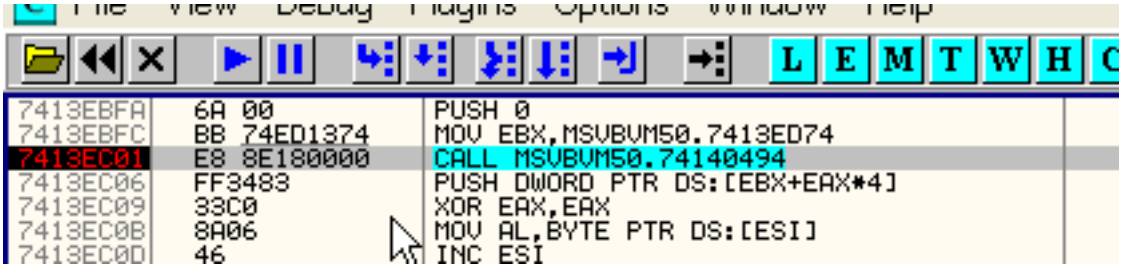


Чтобы рассеять сомнения, устанавливаем BP, минуем вызов с помощью f8 и идём к следующему опкоду.

Видим, что в стеке осталось значение FFFFFFFF, вероятно, означающее, что строки не равны. Запишем возможный серийный номер, и попробуем его использовать.

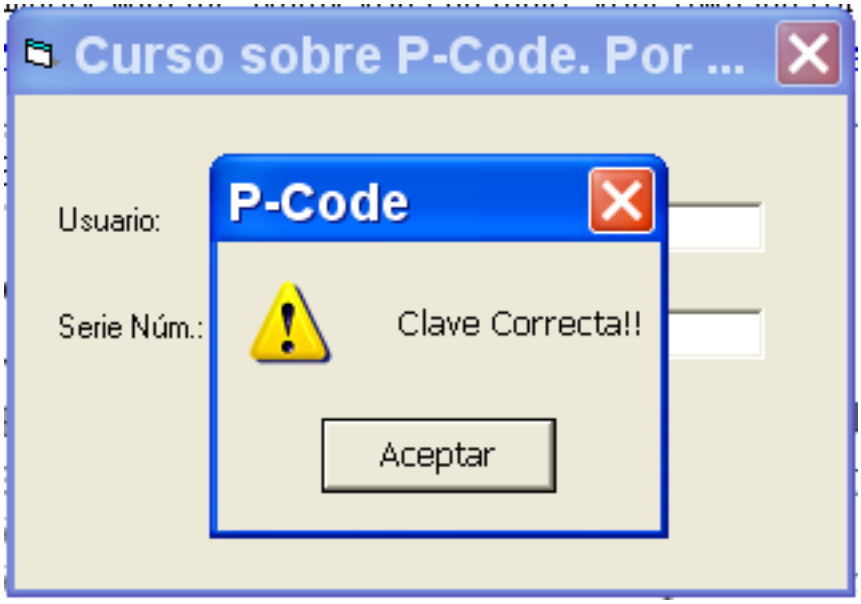


Нажимаем на кнопку “Registrar”.



Снова оказались у этого вызова, проходим его с помощью F8 и идём к следующему опкоду, как в прошлый раз.

Видим, что в данном случае результатом является ноль, так как обе строки одинаковы.



Как мы увидели, не обязательно трассировать всё подряд. Нужно найти подозрительную область и трассировать только неизвестные опкоды. Этого оказалось достаточным, чтобы найти серийный номер исследуемой программы.

Для заинтересованных – более детализированный список, где изложены сведения, которые нам удалось узнать.

```
401CC0: 04 FLdRfVar               local_008C
401CC3: 21 FLdPrThis                  ; Load reference pointer into item pointer. Загрузка
    ссылочного указателя в указатель на элемент
401CC4: 0f VCallAd                text   ; Доступ к методу ITEM DESCRIPTOR TABLE
401CC7: 19 FStAdFunc              local_0088
401CCA: 08 FLdPr                  local_0088
401CCD: 0d VCallHresult           get__ipropTEXTEDIT  ; Читаем содержимое из текстового поля
401CD2: 6c ILdRf                  local_008C   ; Имя
401CD5: 1b LitStr:                &     ; Поместить строку в стек
401CD8: Lead0/30 EqStr                ; Сравнить две строки
401CDA: 2f FFree1Str              local_008C
401CDD: 1a FFree1Ad               local_0088
401CE0: 1c BranchF:               401CE6    ; Переход, если сравнение не удалось
401CE3: 1e Branch:                401e8c    ; Безусловный переход
401CE6: 04 FLdRfVar               local_008C
```

```
401CE9: 21 FLdPrThis
401CEA: 0f VCallAd                text   ; Доступ к методу ITEM DESCRIPTOR TABLE
401CED: 19 FStAdFunc              local_0088
401CF0: 08 FLdPr                  local_0088
401CF3: 0d VCallHresult           get__ipropTEXTEDIT  ; Читаем содержимое текстового поля
401CF8: 6c ILdRf                  local_008C   ; Имя
401CFB: 4a FnLenStr
401CFC: f5 LitI4:                 0x6  6  (....)  ; Передаём элемент из 4-ёх байт
401D01: d1 LtI4                       ; Сравнение меньше чем (?)
401D02: 2f FFree1Str              local_008C
401D05: 1a FFree1Ad               local_0088
401D08: 1c BranchF:               401D3F    ; Переход если ложноSalta si falso (>= 6)
401D0B: 27 LitVar_Missing
401D0E: 27 LitVar_Missing
401D11: 3a LitVarStr:             ( local_00BC ) P-Code
401D16: 4e FStVarCopyObj          local_00CC
401D19: 04 FLdRfVar               local_00CC
401D1C: f5 LitI4:                 0x40  64  (...@)
401D21: 3a LitVarStr:             ( local_009C ) Минимум 6 символов
401D26: 4e FStVarCopyObj          local_00AC
401D29: 04 FLdRfVar               local_00AC
401D2C: 0a ImpAdCallFPR4:         _rtcMsgBox
401D31: 36 FFreeVar         local_00AC local_00CC local_00EC local_010C
401D3C: 1e Branch:                401e8c    ;Если < 6 символов в строке
401D3F: 04 FLdRfVar               local_008C
401D42: 21 FLdPrThis
401D43: 0f VCallAd                text
401D46: 19 FStAdFunc              local_0088
401D49: 08 FLdPr                  local_0088
401D4C: 0d VCallHresult           get__ipropTEXTEDIT  ; Читаем содержимое текстового поля
401D51: 3e FLdZeroAd              local_008C   ; Имя
401D54: 46 CVarStr                local_00AC
401D57: 04 FLdRfVar               local_00CC
401D5A: 0a ImpAdCallFPR4:         _rtcLowerCaseVar  ; Конвертируем в строчные символы
401D5F: 04 FLdRfVar               local_00CC
401D62: 04 FLdRfVar               local_00EC
401D65: 0a ImpAdCallFPR4:         _rtcTrimVar
401D6A: 04 FLdRfVar               local_00EC
401D6D: Lead1/f6 FStVar           local_011C
401D71: 1a FFree1Ad               local_0088
401D74: 36 FFreeVar         local_00AC local_00CC
401D7B: 04 FLdRfVar               local_011C
401D7E: Lead0/eb FnLenVar
401D82: Lead1/f6 FStVar           local_012C
401D86: 28 LitVarI2:              ( local_00BC ) 0x1  (1)
401D8B: 04 FLdRfVar               local_013C
401D8E: 04 FLdRfVar               local_012C
401D91: Lead3/68 ForVar:          (when done) 401DE0  , Начало цикла for next
401D97: 28 LitVarI2:              ( local_00AC ) 0x1  (1)
401D9C: 04 FLdRfVar               local_013C
401D9F: Lead1/22 CI4Var
401DA1: 04 FLdRfVar               local_011C
401DA4: 04 FLdRfVar               local_00CC
401DA7: 0a ImpAdCallFPR4:         _rtcMidCharVar  ; Загрузка символов из...
401DAC: 04 FLdRfVar               local_00CC   ; ... имени
401DAF: Lead2/fe CStrVarVal       local_008C
401DB3: 0b ImpAdCallI2            _rtcAnsiValueBstr  ; конвертируем значения символов в  hexadec
401DB8: 44 CVarI2                 local_00BC
401DBB: Lead1/f6 FstVar    local_016C
401DBF: 2f FFree1Str           local_008C
401DC2: 36 FFreeVar               local_00AC local_00CC
401DC9: 04 FLdRfVar               local_017C
401DCC: 04 FLdRfVar               local_016C
401DCF: Lead0/94 AddVar           local_00AC
401DD3: Lead1/f6 FStVar           local_017C
401DD7: 04 FLdRfVar               local_013C
401DDA: Lead3/7e NextStepVar:     (continue) 401D97  ; Конец цикла for-next?
401DE0: 04 FLdRfVar               local_017C
401DE3: 04 FLdRfVar               local_012C
401DE6: Lead0/94 AddVar           local_00AC
401DEA: Lead1/f6 FStVar           local_018C
```

```
401DEE: 04 FLdRfVar               local_008C
401DF1: 21 FLdPrThis
401DF2: 0f VCallAd                text
401DF5: 19 FStAdFunc              local_0088
401DF8: 08 FLdPr                  local_0088
401DFB: 0d VCallHresult           get__ipropTEXTEDIT  ; Читаем содержимое текстового поля
401E00: 3e FLdZeroAd              local_008C   ; Серийный номер
401E03: 46 CVarStr                local_00CC
401E06: 5d HardType
401E07: 3a LitVarStr:             ( local_009C ) CRK
401E0C: 04 FLdRfVar               local_018C
401E0F: Lead0/ef ConcatVar
401E13: Lead0/40 NeVarBool
401E15: 1a FFree1Ad               local_0088
401E18: 36 FFreeVar         local_00CC local_00AC
401E1F: 1c BranchF:               401E59
401E22: 27 LitVar_Missing
401E25: 27 LitVar_Missing
401E28: 3a LitVarStr:             ( local_00BC ) P-Code
401E2D: 4e FStVarCopyObj          local_00CC
401E30: 04 FLdRfVar               local_00CC
401E33: f5 LitI4:                 0x10  16  (....)
401E38: 3a LitVarStr:             ( local_009C ) Серийный номер неверен!
401E3D: 4e FStVarCopyObj          local_00AC
401E40: 04 FLdRfVar               local_00AC
401E43: 0a ImpAdCallFPR4:         _rtcMsgBox
401E48: 36 FFreeVar         local_00AC local_00CC local_00EC local_010C
401E53: 1e Branch:                401e8c
401E56: 1e Branch:                401e8c
401E59: 27 LitVar_Missing
401E5C: 27 LitVar_Missing
401E5F: 3a LitVarStr:             ( local_00BC ) P-Code
401E64: 4e FStVarCopyObj          local_00CC
401E67: 04 FLdRfVar               local_00CC
401E6A: f5 LitI4:                 0x30  48  (...0)
401E6F: 3a LitVarStr:             ( local_009C ) Серийный номер верен!!
401E74: 4e FStVarCopyObj          local_00AC
401E77: 04 FLdRfVar               local_00AC
401E7A: 0a ImpAdCallFPR4:         _rtcMsgBox
401E7F: 36 FFreeVar         local_00AC local_00CC local_00EC local_010C
401E8A: Lead1/c8 End
401E8C: 13 ExitProcHresult
```

Отсюда ясно, как работает крэкми.

Полезно знать, как работать с P-CODE в OllyDbg, так как есть программы, которые защищены от WKT и EXDEC, но в случае с OllyDbg мы можем использовать плагины, которые спрячут её практически ото всех, кроме очень редких исключений.

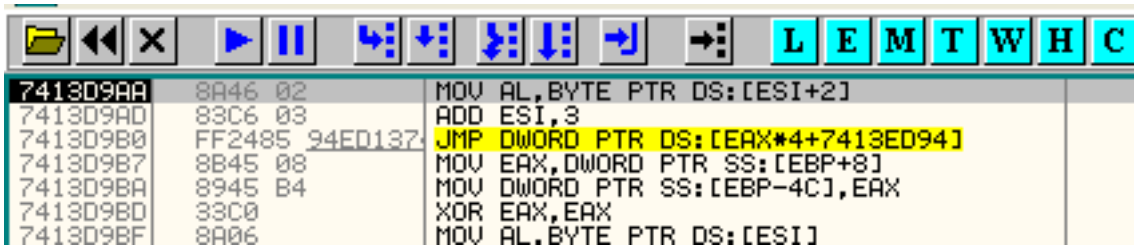
Загружаем прилагающийся крэкми nags1, который просит нас убрать первоначальное наг-окно. Смотрим его листинг в EXDEC.

```
Proc: 401a40
401A14: 27 LitVar_Missing
401A17: 27 LitVar_Missing
401A1A: 27 LitVar_Missing
401A1D: f5 LitI4:                 0x0  0  (....)
401A22: 3a LitVarStr:             ( local_0094 ) NAG
401A27: 4e FStVarCopyObj          local_00A4
401A2A: 04 FLdRfVar               local_00A4
401A2D: 0a ImpAdCallFPR4:         _rtcMsgBox
401A32: 36 FFreeVar         local_00A4 local_00C4 local_00E4 local_0104
401A3D: 13 ExitProcHresult

Proc: 401958

401954: Lead1/c8 End
401956: 13 ExitProcHresult
```

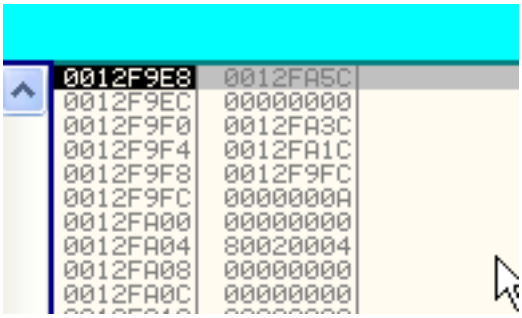
Видим, что знаменитый NAG – это просто rtcMsgBox. В P-Code нет NOP'ов, хе-хе, поэтому нам нужно забить эту функцию опкодами, которые не изменят хода выполнения программы.

Устанавливаем BPM на опкоде, вызывающем rtcMsgBox, и запускаем программу.

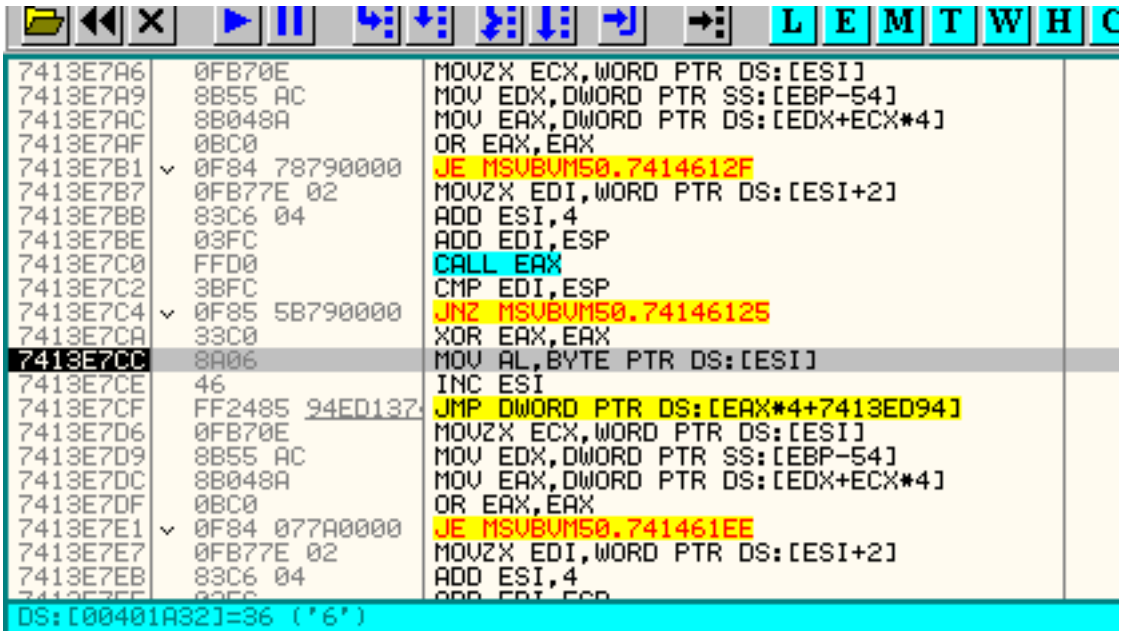
```
401A2D: 0a ImpAdCallFPR4:          _rtcMsgBox
```



Останавливаемся, когда считывается опкод.

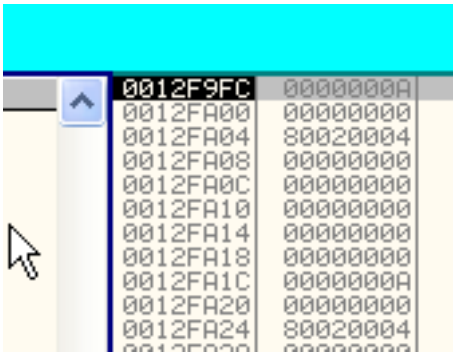


Видим, что в стеке находится 12f9e8. Идём к следующему опкоду. Сначала, конечно, выскочит наг-окошко, поэтому надо будет нажать «Aceptar».



Здесь мы доходим до следующего опкода после CALL EAX, который вызывал API-функцию rtcMsgBox.

В стеке находится 12f9fc.



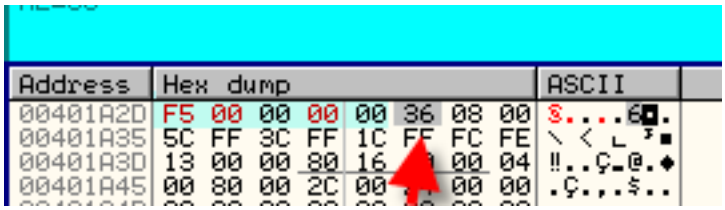
То есть, для того, чтобы оставить равные, нужно сделать различные POP'ы. Сюда мы не вмешиваемся, можем попробовать использовать PUSH с помощью F5.

```
F5    5CBE 1377 4 1 4      Push imm#4
```

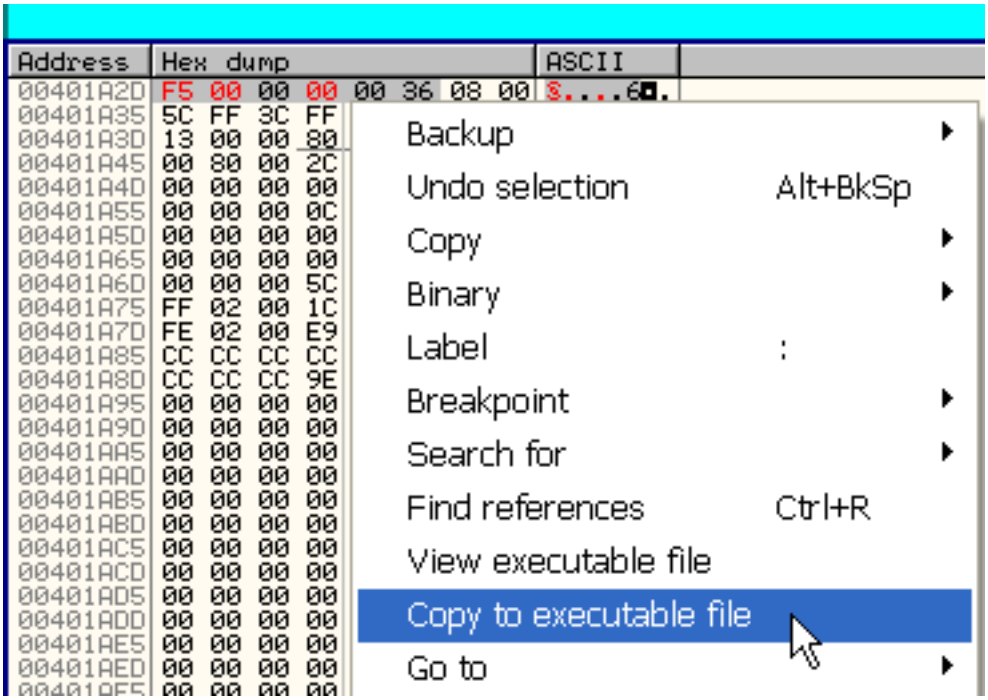
У нас 4 параметра размером равные 0A. Столько нам нужно заменить.

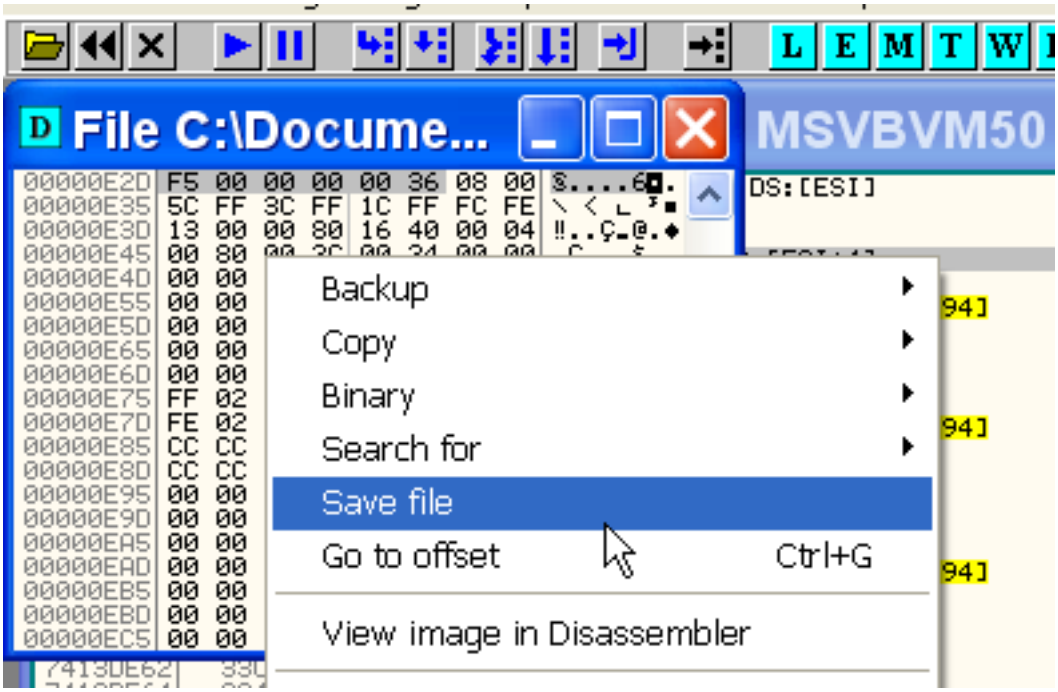
```
0A    664E 1F30 4 2 2 2
```

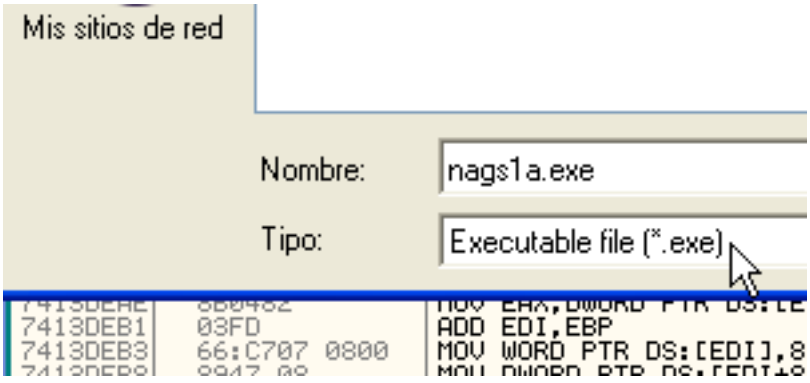
Пробуем изменить 0A на F5 и делаем все параметры равными нулю.

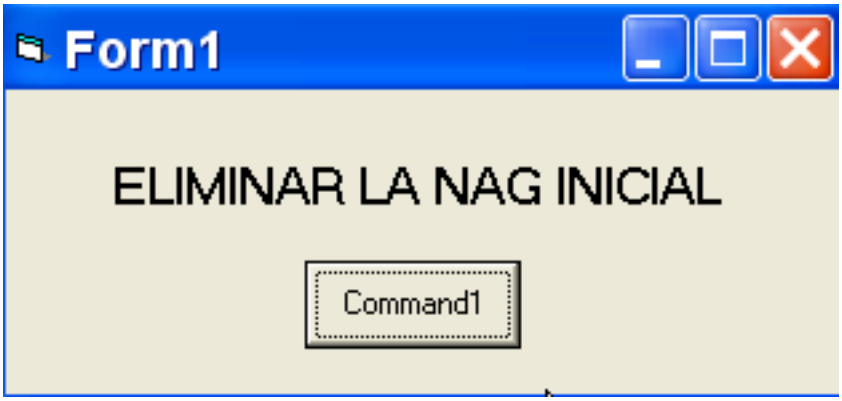


Следующий опкод – это 36, как показывает нам EXDEC. Мы всегда должны быть уверенными, что замещающий опкод имеет такое же количество параметров, как и у замещаемого, чтобы не было проблем. Сохраняем изменения.


Address
Hex dump
ASCII
Backup
Undo selection Alt+BkSp
Copy
Binary
Label :
Breakpoint
Search for
Find references Ctrl+R
View executable file
Copy to executable file
Go to


File C:\Docume...
MSVBVM50
DS:[ESI]
Backup
Copy
Binary
Search for
Save file
Go to offset Ctrl+G
View image in Disassembler


Mis sitios de red
Nombre: nags1a.exe
Tipo: Executable file (*.exe)
7413DEB1 03FD ADD EDI,EBP
7413DEB3 66:C707 0800 MOV WORD PTR DS:[EDI],8



Доходим досюда без появления наг-окна. В другой раз, вероятно, можно использовать иной опкод для патчинга. В качестве домашнего упражнения вы можете попробовать решить крэкми nags2, прилагающийся к данной статье.

31 часть будет последней посвящённой P-CODE, и в ней будет рассмотрена коммерческая программа.

Введение в крэкинг с нуля, используя OllyDbg - Глава 31

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-07-26

Вводные сведения о распаковке

Вы скажете, что объявлять третью часть, посвящённую P-CODE, а потом начинать с распаковки, это безумие. Это всё потому, что многие крэкеры, с которыми я советовался, жаждут улучшить свои навыки в распаковке, а о P-CODE есть множество туториалов, включая те, где используется WKT, да и программ, написанных с его использованием, не так много, поэтому думаю, что лучше начать рассказывать о распаковке. Поскольку распаковщиков тысячи, то всех их рассматривать не будет, но постараемся освоить общие принципы и примеры, которые помогут нам понять работу пакеров, которых мы ещё не видели, чтобы не искать потом отчаянно туториалы о них (хотя иногда это помогает, хе-хе).

В этой первой части рассмотрим некоторые основные концепции и идеи, которые послужат нам в будущих частях для распаковки специальных примеров.

Ок, вот в чём состоит идея упаковки программ.

Мы уже видели, что в распакованной программе с самого начала можно легко изменять нужны байты, так как они легко доступны, не меняются, листинг программы всё время постоянен и в любой момент можем изменить что угодно без особых проблем, а потом сохранить изменения.

Следовательно, в упакованной программе изменить что-либо легко и просто не получится, так как изначальный код зашифрован и недоступен.

Когда приложение упаковывается выбранным упаковщиком, тот шифрует и хорошо прячет изначальный (исполняемый, а не исходный) код программы, затем, обычно, добавляет одну (или больше) секцию, в которую помещает загрузчик, и перенаправляет точку входа на этот загрузчик-распаковщик.

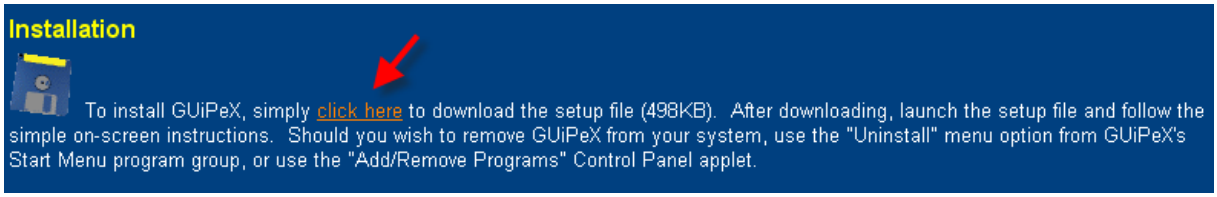
В результате всего этого изначальный код программы становится не виден, если откроем программу в OllyDbg и остановим её, то окажемся в точке входа распаковщика, откуда и будет начинаться выполнение.

Этот распаковщик ищет сохранённую информацию о зашифрованном изначальном коде, расшифровывает её и сохраняет в изначальное места. Как только раскриптовка завершена, он переходит на OEP или изначальную точку входа (Original Entry Point), т.е. та, которая была до упаковки программы.

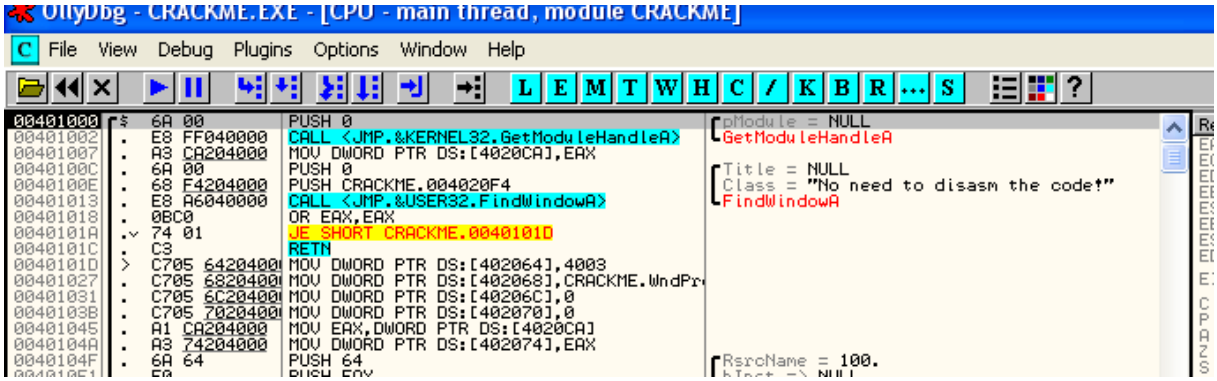
Поищем наиболее простой из существующих упаковщиков. Это UPX. Качаем версию, что с GUI, поскольку она самая простая. Называется она GUIPEX и доступна по следующему адресу.

`http://www.blueorbsoft.com/guipex/`

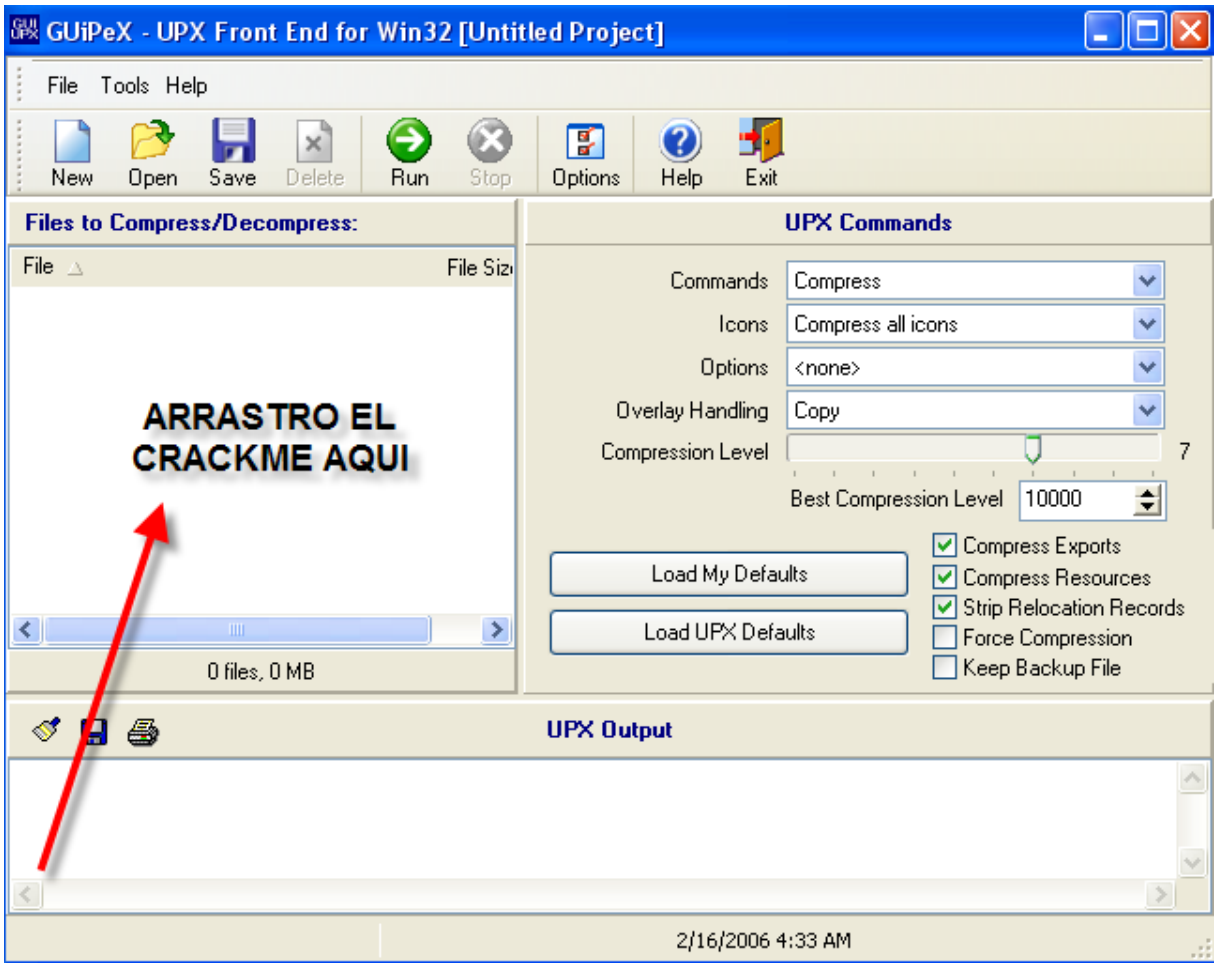
Ок, устанавливаем её и запускаем.



Хорошо, у нас тут самый простой упаковщик, но чтобы подстраховаться и вы не опухли, используем знаменитый крэкми CrueHead'a и сначала откроем его в OllyDbg без упаковки.



Он открыт в OllyDbg, точка входа - `401000`, то есть если мы его запустим, это будет первая выполнившаяся строка.



Таким образом, если мы его запакуем, GUIPEX изменит и добавит секции, закриптует изначальный код, куда-то сохранит его, а затем изменит точку входа так, чтобы

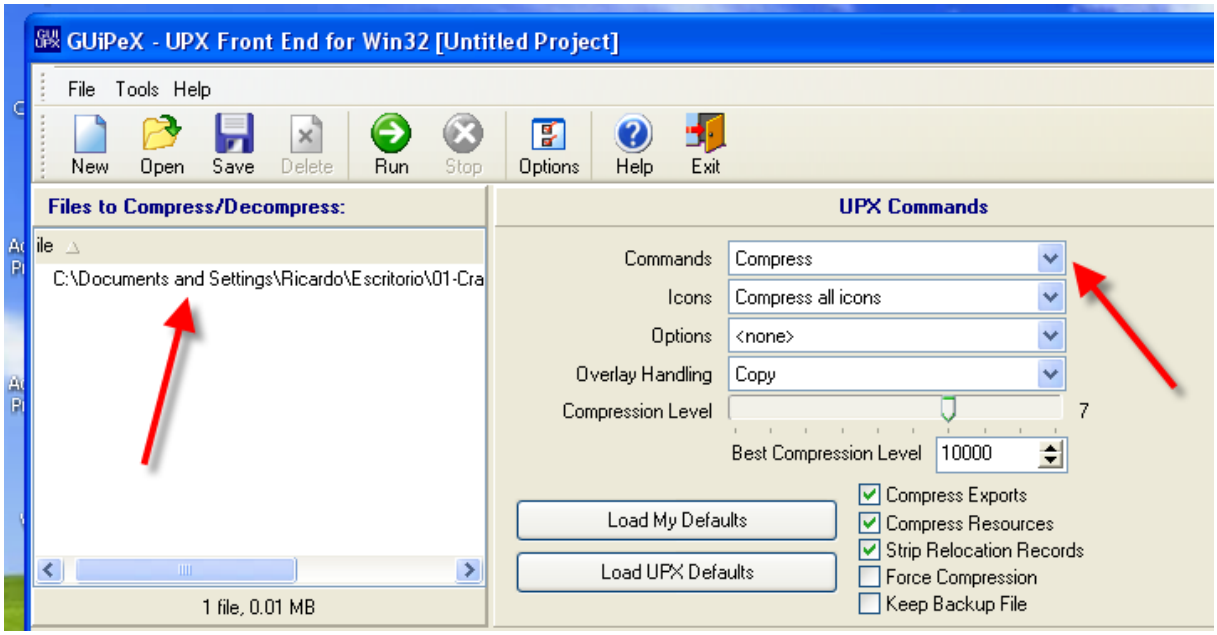
она указывала на загрузчик-распаковщик - и вуаля, готово.

Тогда, когда мы запустим упакованную программу, распаковщик достанет из ларца зашифрованный изначальный код, расшифрует его и поместит в первую секцию, а прежде чем завершить свою работу, перейдёт к первой строке изначального кода, которая должна быть выполнена, то есть OEP или изначальная точка входа. В нашем случае, как мы знаем, это `401000`, то есть программа до упаковки начиналась оттуда, и это первая строка, которая должна выполниться.

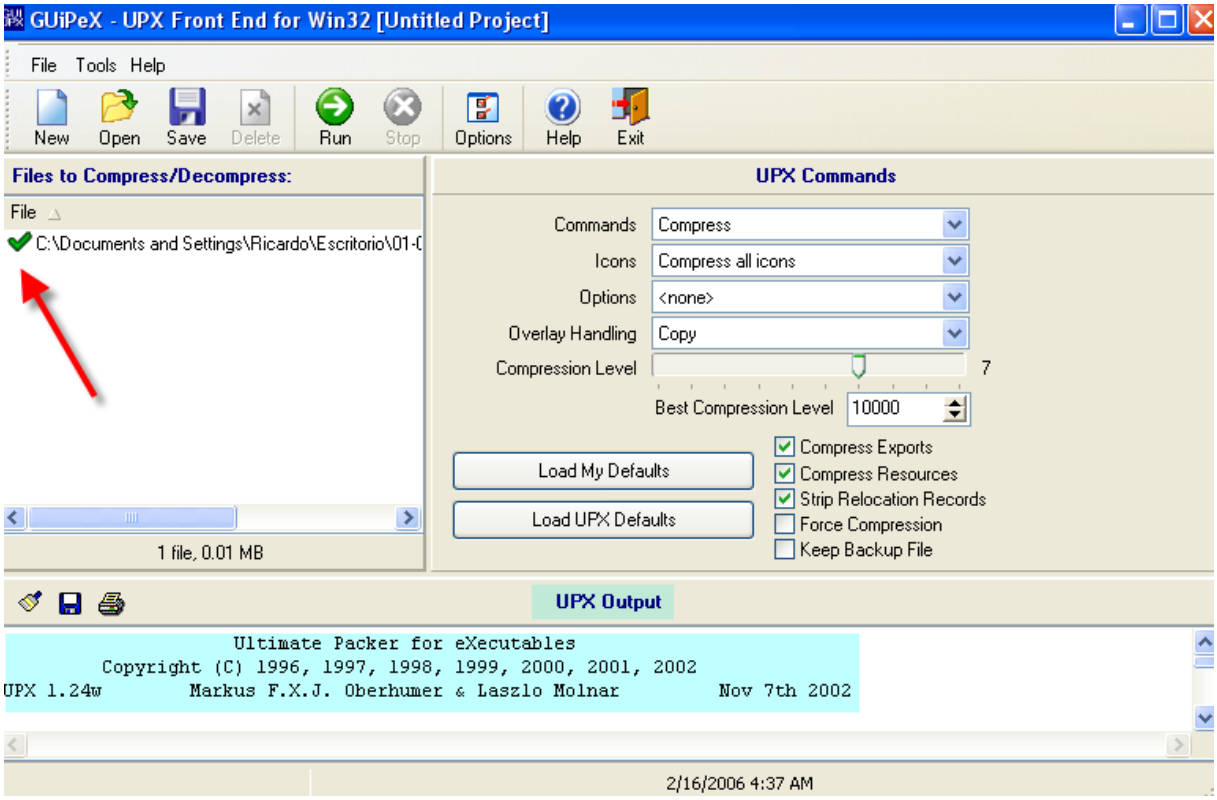
Логично предположить, что когда мы начинаем исследовать упакованную программу, у нас нет оригинала, с помощью которого мы могли бы узнать OEP, то есть первую строку изначальной программы, поэтому нам нужно изучить различные техники и способы для её нахождения. Для этого сначала будем практиковаться на CC (крэкми CrueHead'a, чтобы короче).

Сохраним копию CC в надёжное место, например, на рабочий стол, так как UPX его изменит, и нам потребуется оригинал для сравнения.

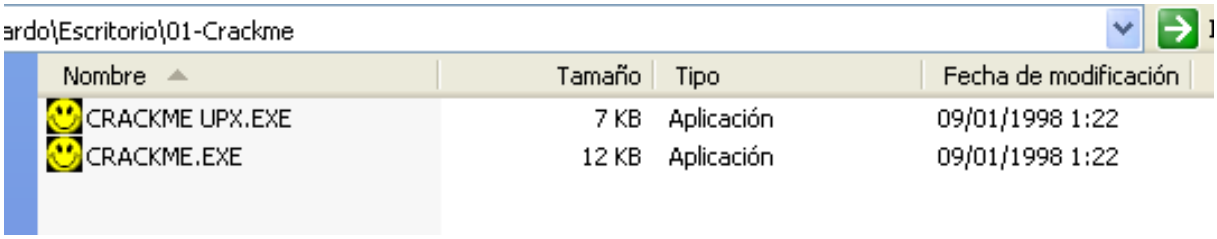
Открываем GUIPEX.



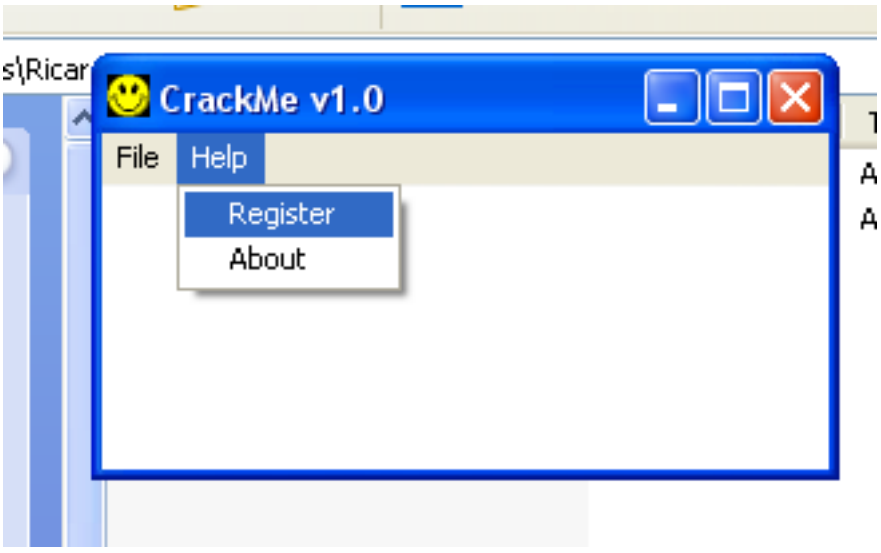
Тащим крэкми и бросаем сюда.



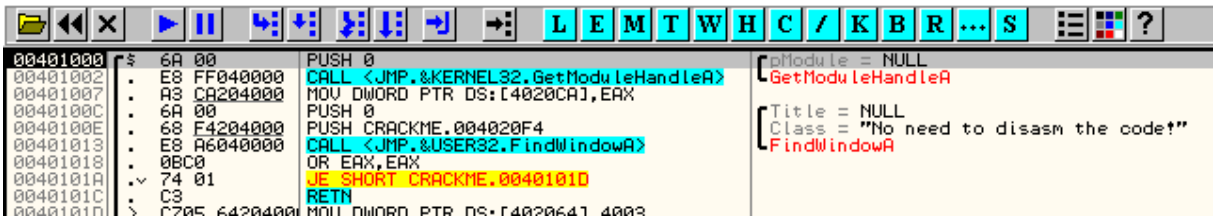
Видим, что программа отображает крэкми с полным путём до него и доступные команды. Выбираем COMPRESS, чтобы упаковать крэкми. С помощью данной утилиты мы также можем и распаковывать.



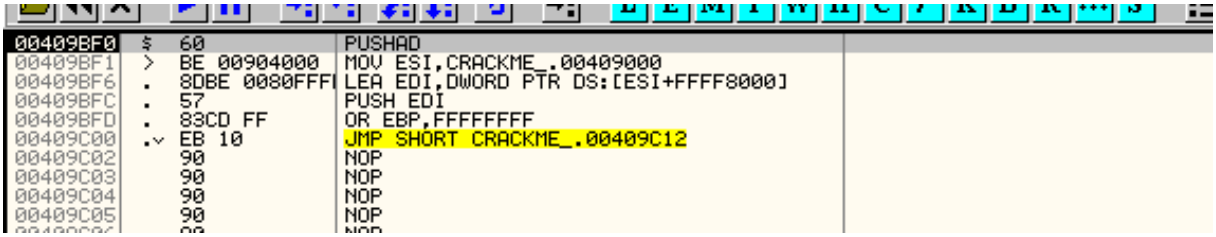
Как только это сделали, нажимаем кнопку RUN, чтобы выполнить заданную GUIPEX'у операцию.



Видим, что всё прошло успешно, и крэкми было упакованно.



Изменить имя упакованного крэкми, чтобы отличать его от исходного `CRACKME.EXE`.

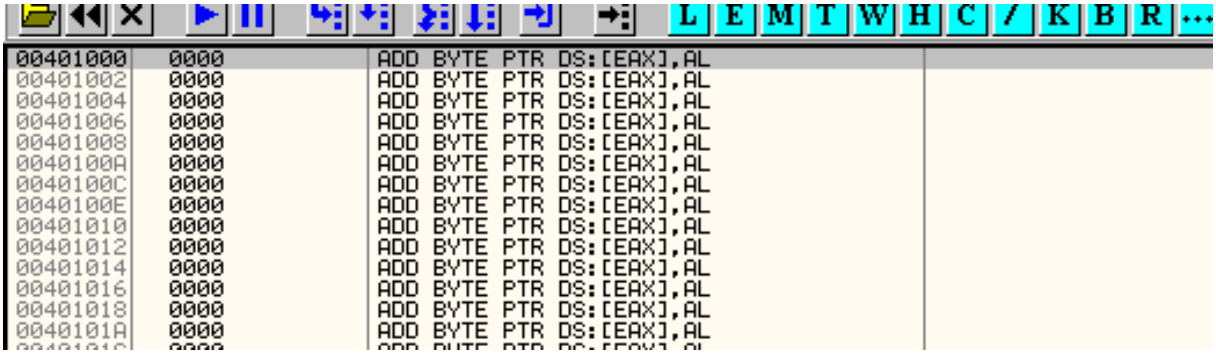


Como vemos el crackme empacado es mas pequeño que el original, en una epoca esto era asi, ahora hay packer que agregan tanto codigo para proteger que son mucho mas grandes los empacados que el original, jeje.

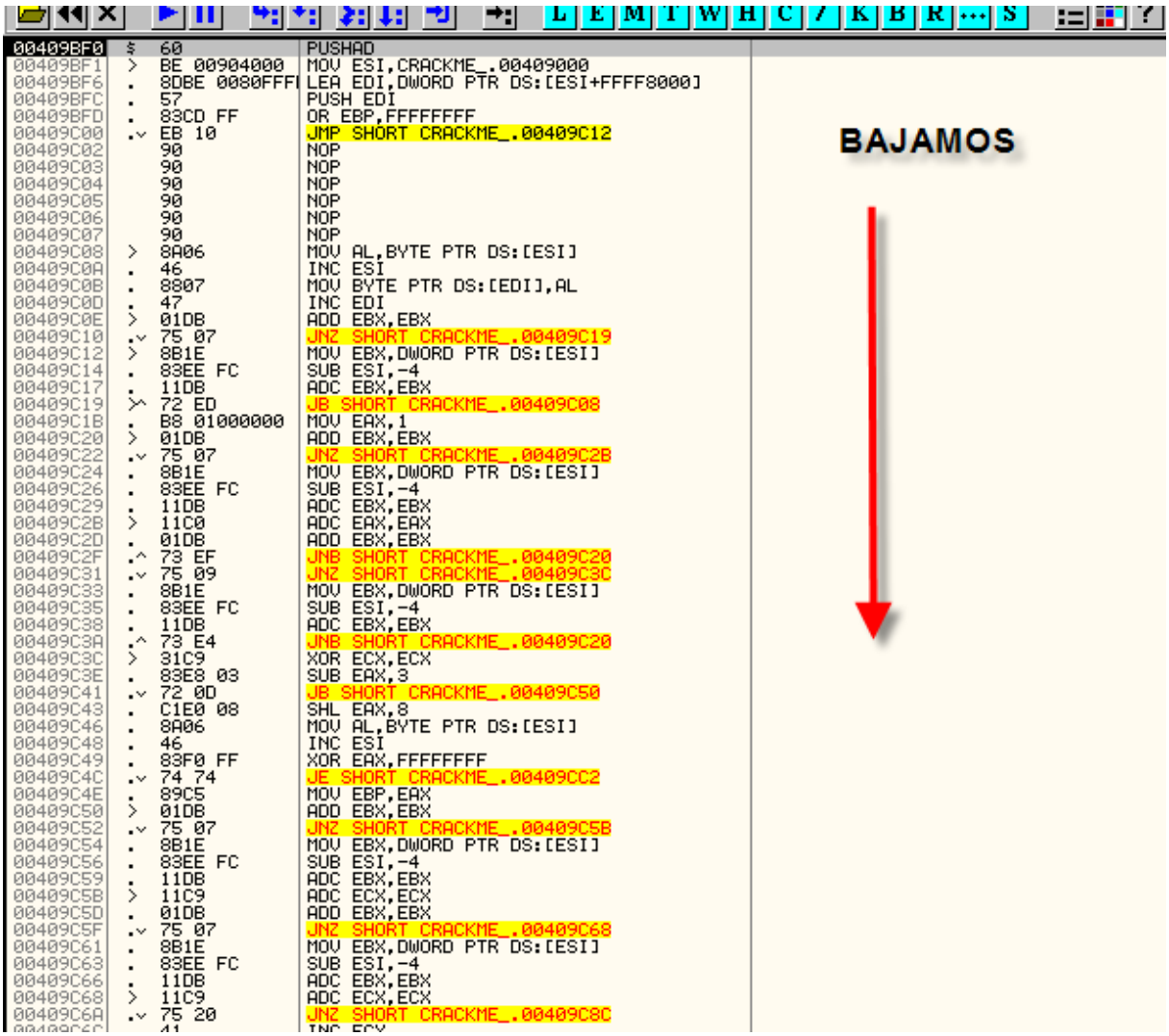
Как видим, запакованный крэкми гораздо меньше, чем исходный. В наше время есть такие упаковщики, которые навешивают столько кода для защиты, что после упаковки файлы становятся куда больше, чем были до этого, хе-хе.

Если запустим упакованный крэкми, то увидим, что он работает точно так же как и исходный. Теперь посмотрим на них с другой стороны. Откроем два OllyDbg, в одном из которых “CRACKME UPX.exe”, а в другом “CRACKME.exe”, и сравним их.

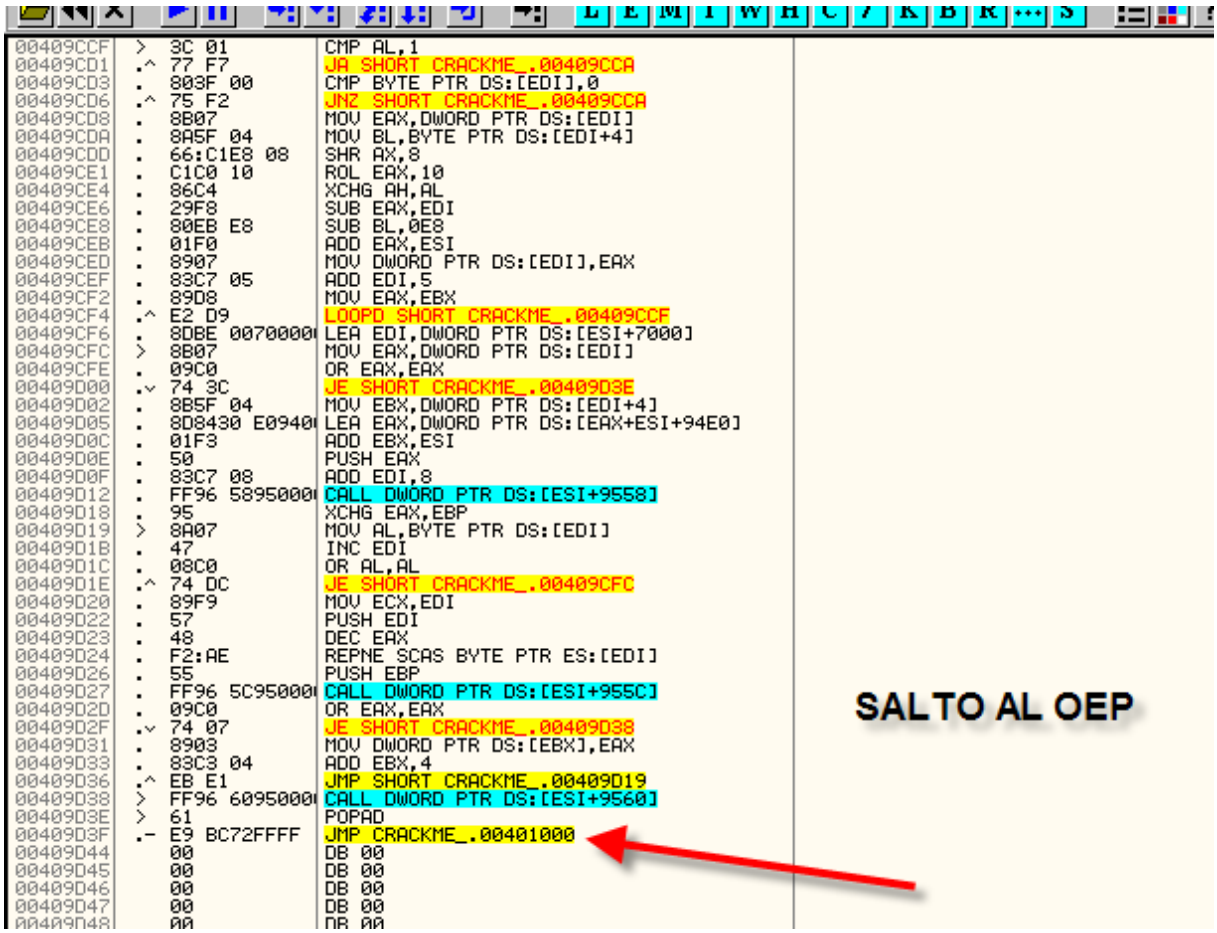
ENTRY POINT DEL `CRACKME.exe`



ENTRY POINT DEL `CRACKME UPX.exe`



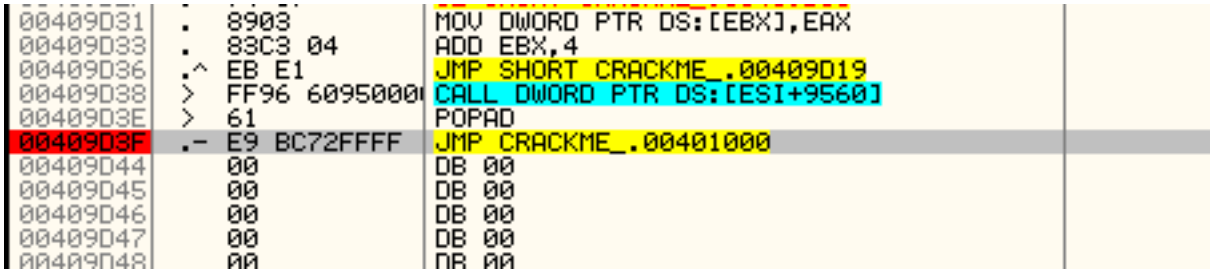
Как видим, они разные. В “CRACKME UPX” точка входа заменена на `409bf0`, где будет начинаться выполнение распаковщика, а если пойдём посмотрим, что находится в `401000`, то не найдём ни следа изначального кода.



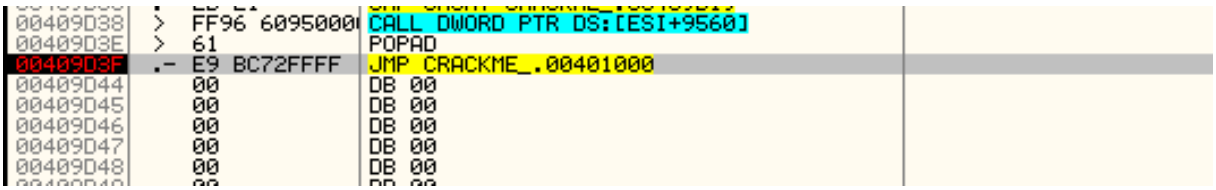
Как видим, первая секция полностью пуста, то есть упаковщик обнулил все зашифрованные байты, сохранив их содержимое куда-то в другое место, оставив в данном случае первую секцию совершенно пустой.

В общем случае, большая часть упаковщиков создают секцию, из которой начинается выполнение распаковщика, который достаёт зашифрованные байты изначального кода и помещает расшифрованный код в ту самую первую секцию.

Посмотрим снова процедуру распаковщика, не запуская её. Идём вниз.



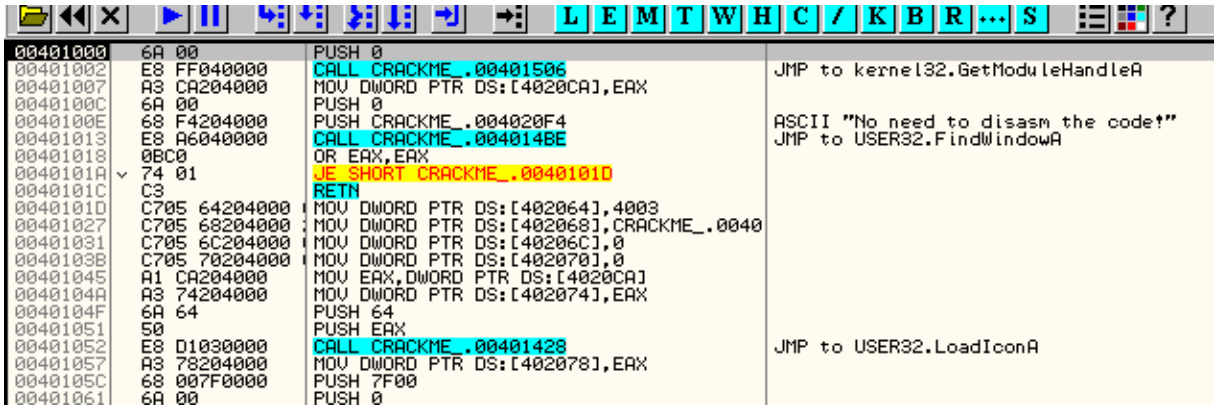
Продолжаем, пока не увидим:



Это бесхитростный упаковщик, видим, что начинается выполнение раскриптовки, а когда она заканчивается, происходит переход на OEP. Ничего не прячется. Поглядев на настоящие упаковщики, гляда на такое можно рассмеяться, но ладно,

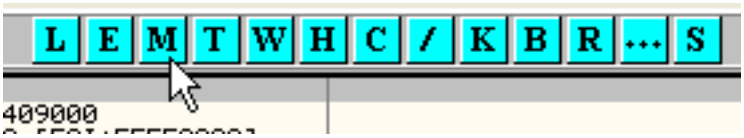
продолжаем работу.

Устанавливаем BP на этот `JMP`.

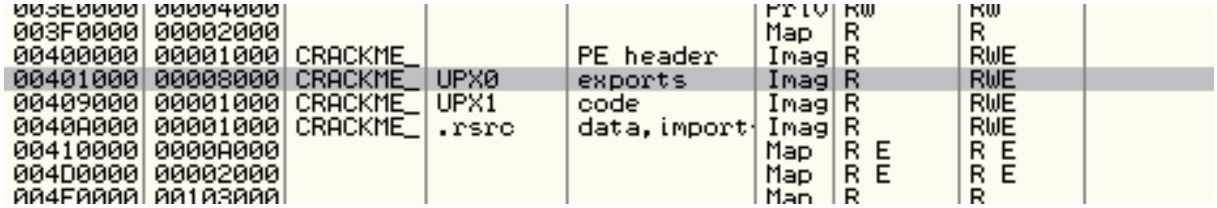


И делаем RUN.

Останавливаемся на нём, и поскольку наполнение первой секции завершено, переходим на OEP (то есть первую строку изначального кода), нажав на F7.



Вот OEP с тем же кодом, что и в оригинальном крэкми Cruehead'a. Распаковщик завершил свою работу, расшифровав весь изначальный код, после чего сделал переход на OEP, чтобы начать выполнение программы. Сперва, как мы видели, здесь были только нули.



Вот схема работы запакованной программы:

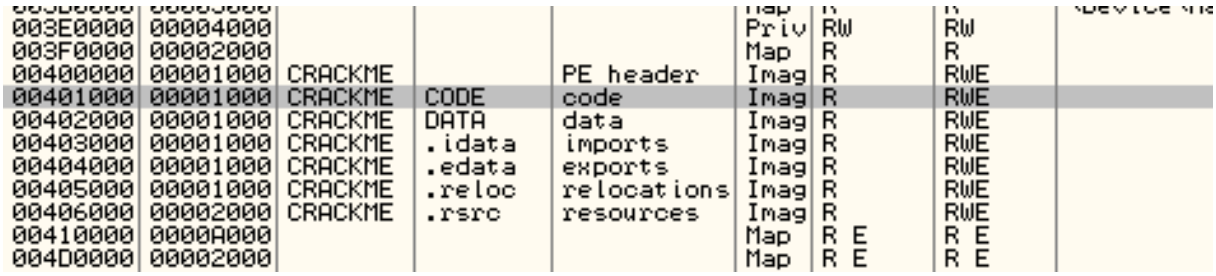
1. Запуск распаковщика
2. Восстановление и расшифровка секции, где должен выполняться изначальный код
3. Переход на OEP
4. Запуск программы

По этой схеме работало множество упаковщиков в течении долгих лет. Со временем создатели упаковщиков стали слегка её изменять, используя разные хитрости, чтобы спрятать OEP, и другие трюки, которые мы рассмотрим позже. Но в целом, всё функционирует так, как сказано выше.

Рестартуем “CRACKME UPX”.

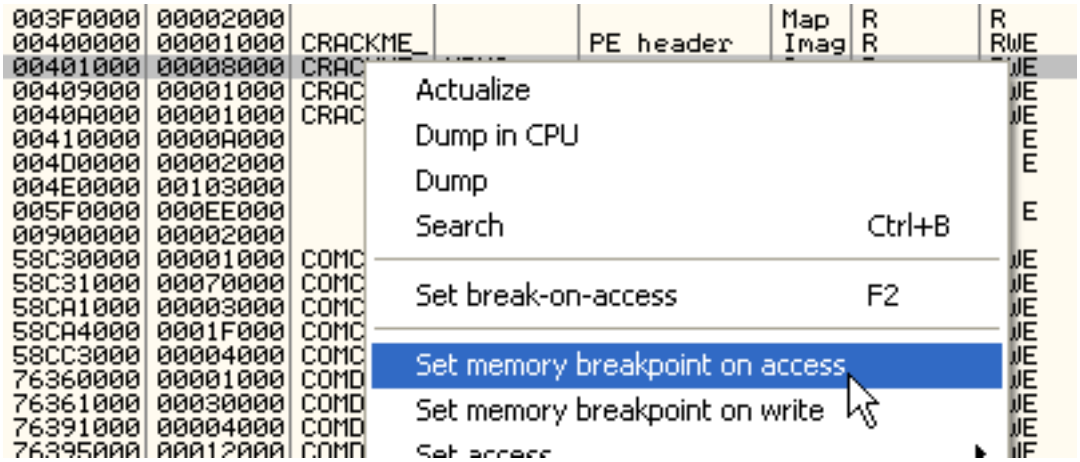
Очевидно, что если первая секция является пустой, как в данном случае, и в ней будет восстановлен изначальный код, то необходимо осуществлять запись в данную секцию, поэтому если используем BMP ON ACCESS на ней, то остановимся на процедуре, занимающейся расшифровкой кода.

Идём в M.



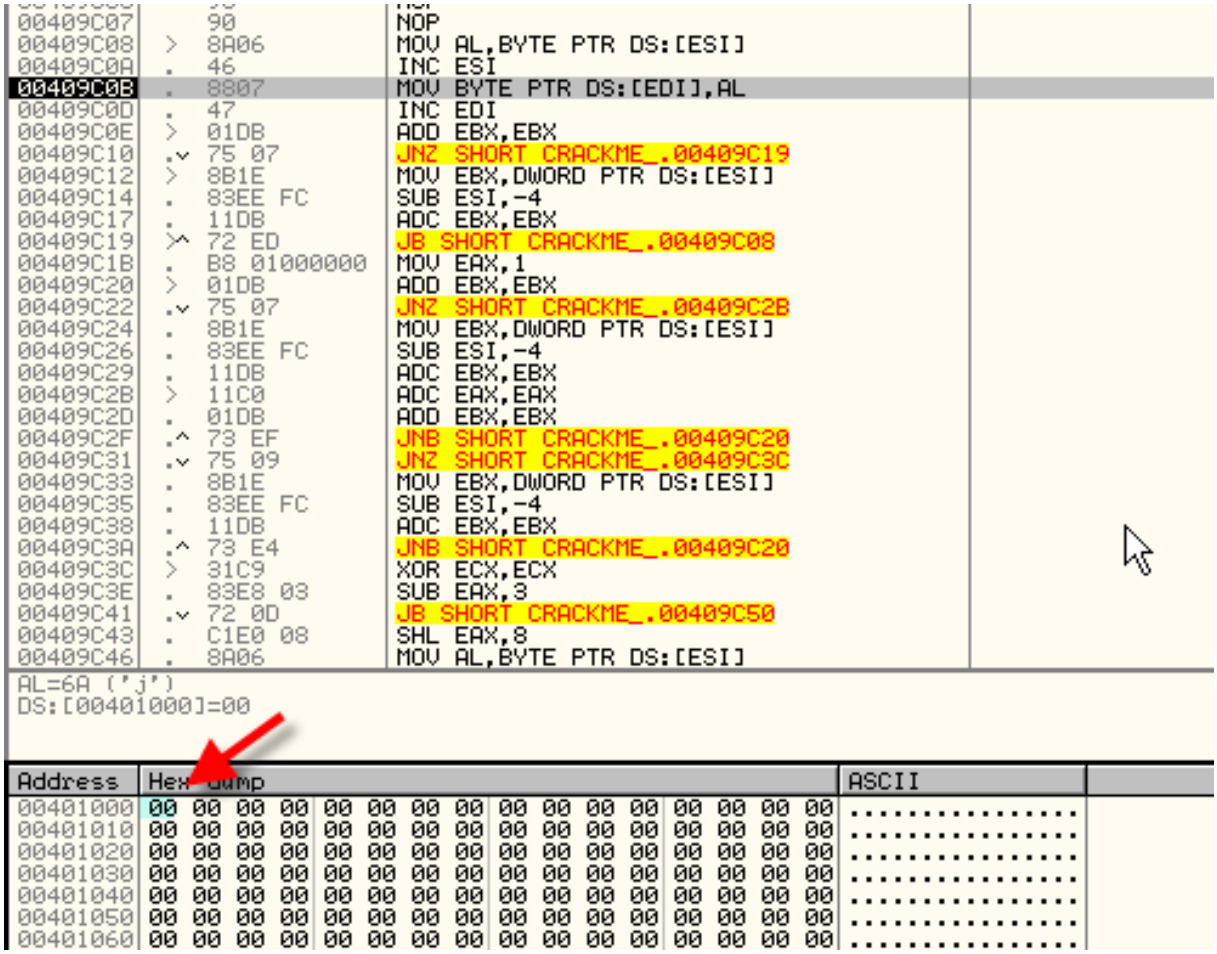
003E0000	00004000				Priv	RW	RW
003F0000	00002000				Map	R	R
00400000	00001000	CRACKME		PE header	Imag	R	RWE
00401000	00001000	CRACKME	CODE	code	Imag	R	RWE
00402000	00001000	CRACKME	DATA	data	Imag	R	RWE
00403000	00001000	CRACKME	.idata	imports	Imag	R	RWE
00404000	00001000	CRACKME	.edata	exports	Imag	R	RWE
00405000	00001000	CRACKME	.reloc	relocations	Imag	R	RWE
00406000	00002000	CRACKME	.rsrc	resources	Imag	R	RWE
00410000	0000A000				Map	R E	R E
004D0000	00002000				Map	R E	R E

Видим, что размеры секций были изменены в соответствии с начальными.

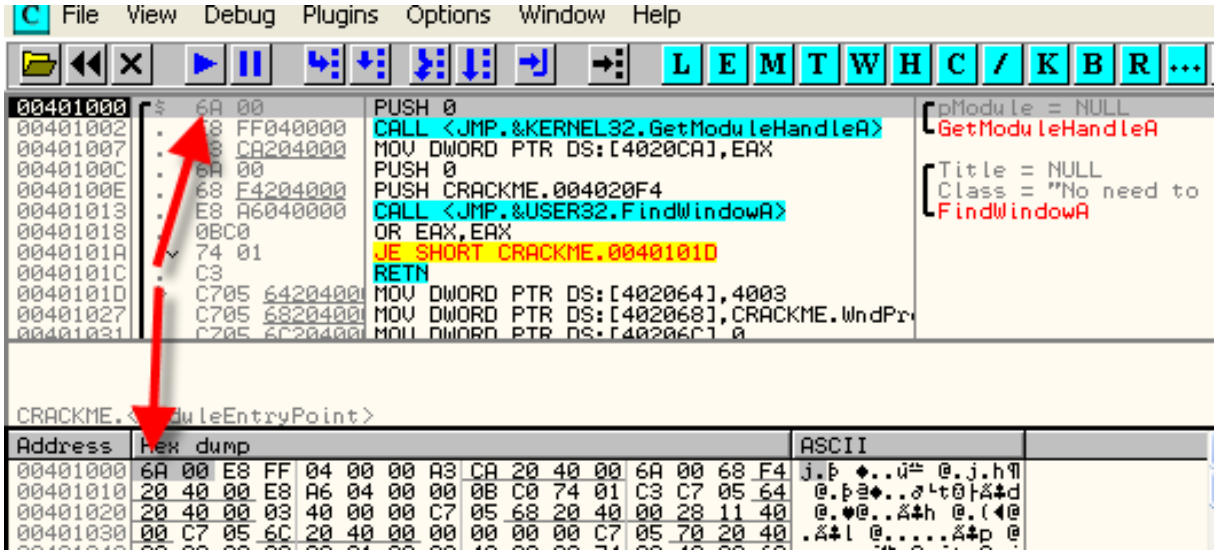


Видим, что секция CODE в исходном файле начинается в `401000` и занимает `1000` байтов, в то время как в `CRACKME UPX` она начинается в `409000`.

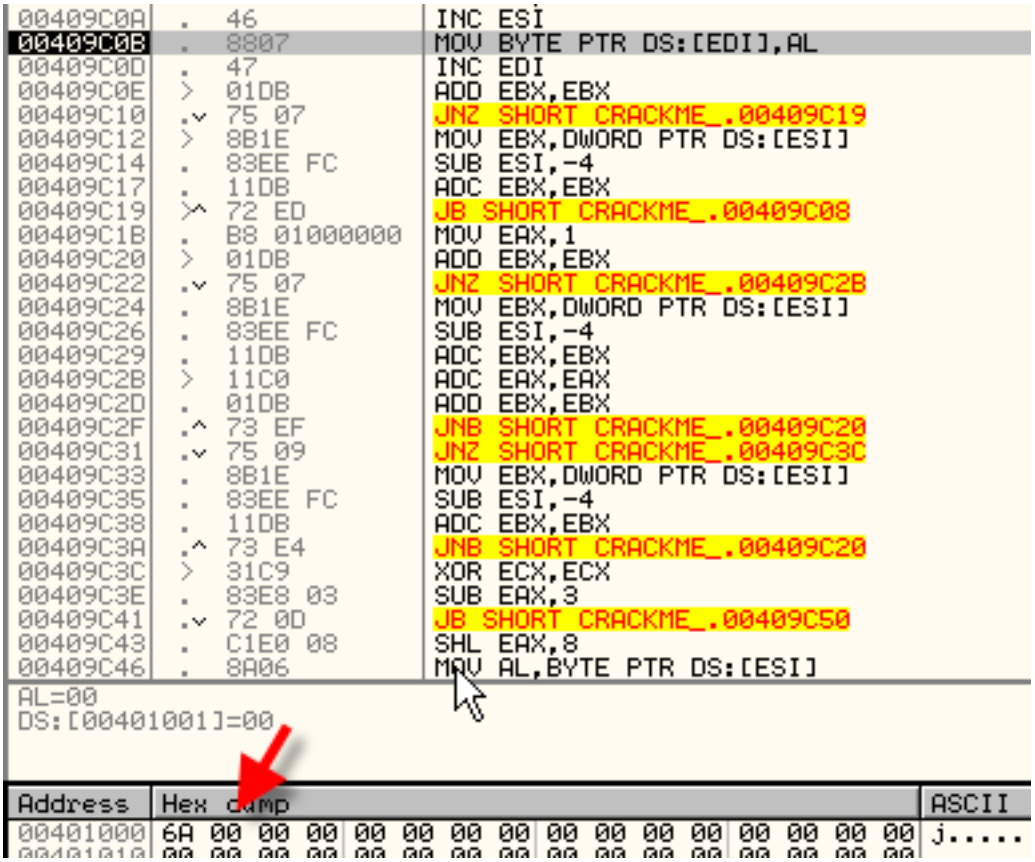
Ладно, устанавливаем BPM ON ACCESS на первой секции запакованного крэкми, чтобы посмотреть, произойдёт ли останов, когда начнётся распаковка. Делаем RUN.



Точно, видим, что останавливается на моменте сохранения AL, равного 6A, в 401000. Смотрим, что находится в оригинальном файле по этому адресу.



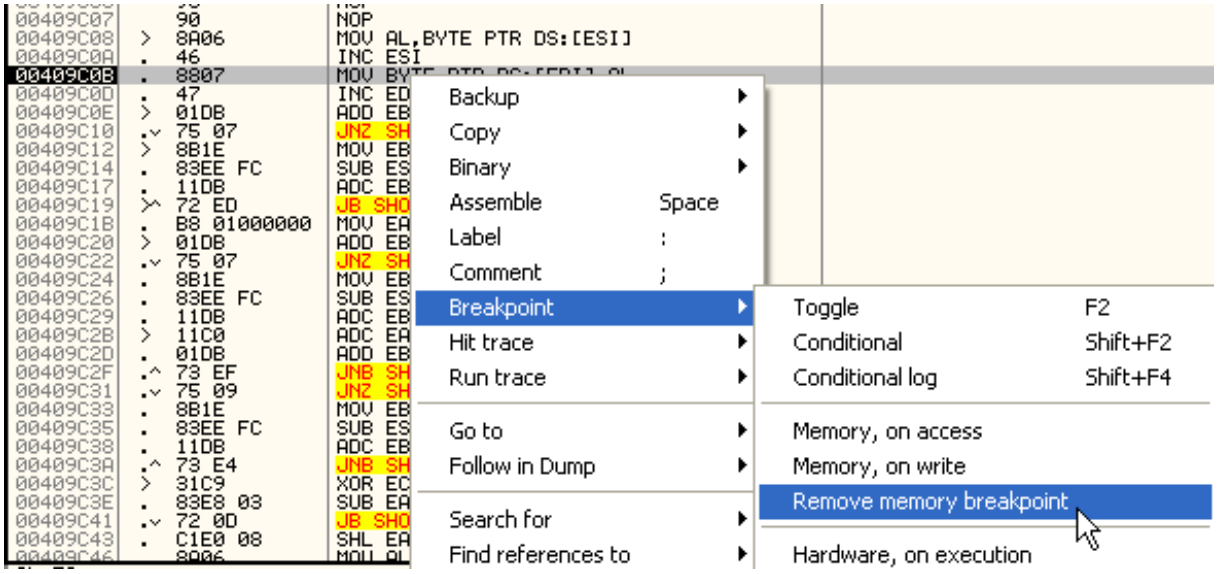
Видим, что сохраняется первый байт, находящийся в оригинальном файле по адресу 401000. Если нажмём F9, то должен будет сохраниться следующий байт, то есть 00. Смотрим.



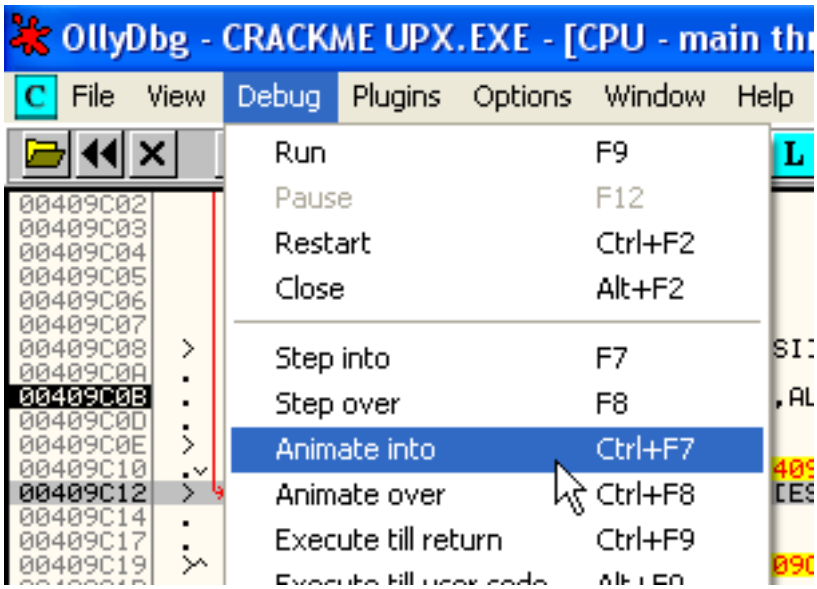
И так можем продолжать байт за байтом, сохраняя расшифрованные файлы и реконструируя изначальный код. Конечно, не все упаковщики распаковывают по порядку или вообще начинают с начала, как в данном случае, но для начала мы будем рассматривать самый нормальный вариант.

Если мы не поленимся трассировать дальше, то увидим, что это LOOP, где считывается зашифрованный байти, над ним проводятся математические операции, чтобы его расшифровать (суммирование, умножение и так далее, как пример), а после того, как получается оригинальное значение, оно сохраняется после предыдущего.

Если снимем BPM ON ACCESS, то если нажмём ANIMATE INTO, то увидим забавное зрелище как будто в кино, как первая секция заполняется изначальными значениями, пока не дойдём до BPX, который установили на JMP, ведущему на OEP.

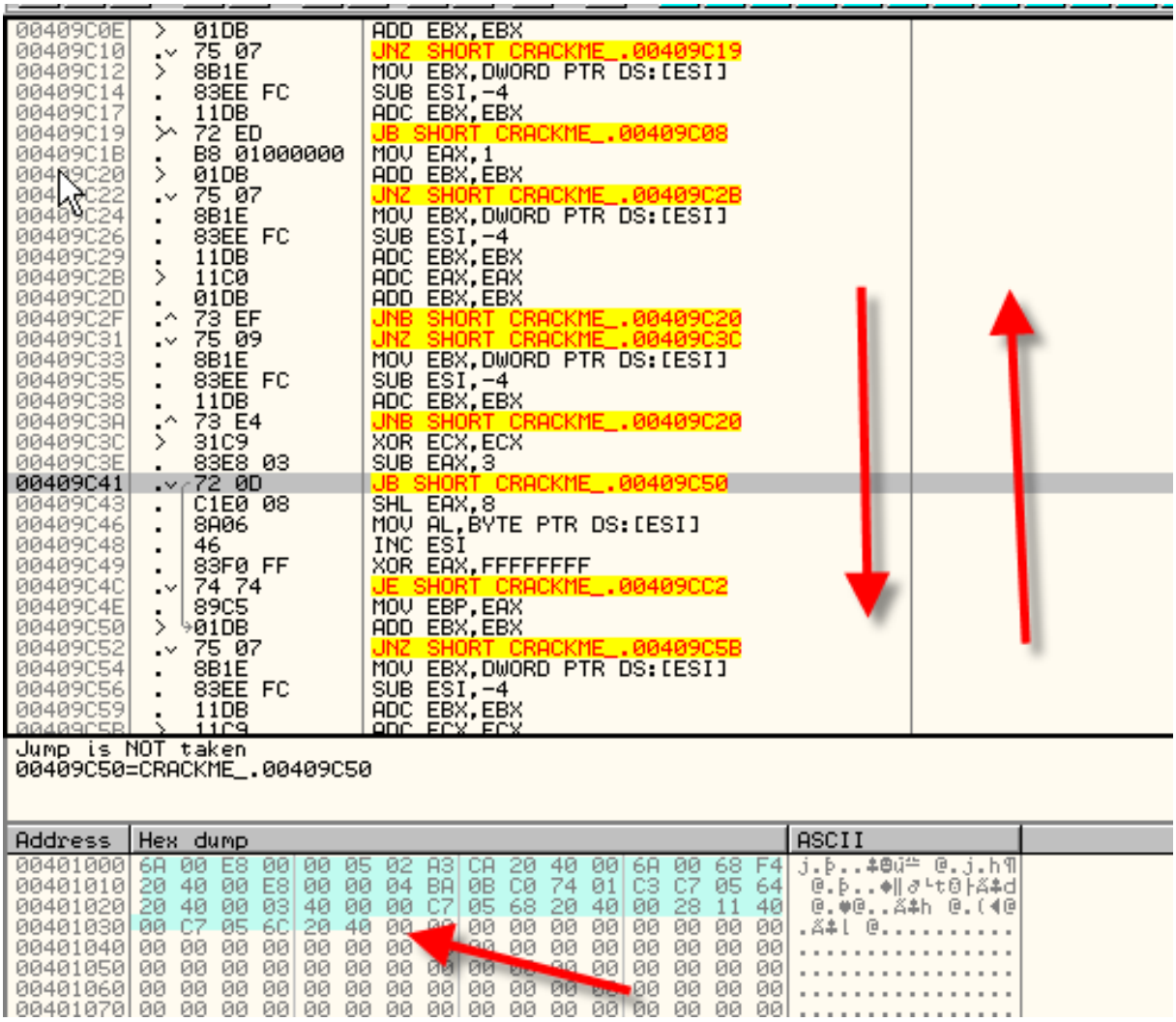


Видим, как работает программа, раскриптовывающая секцию CODE.

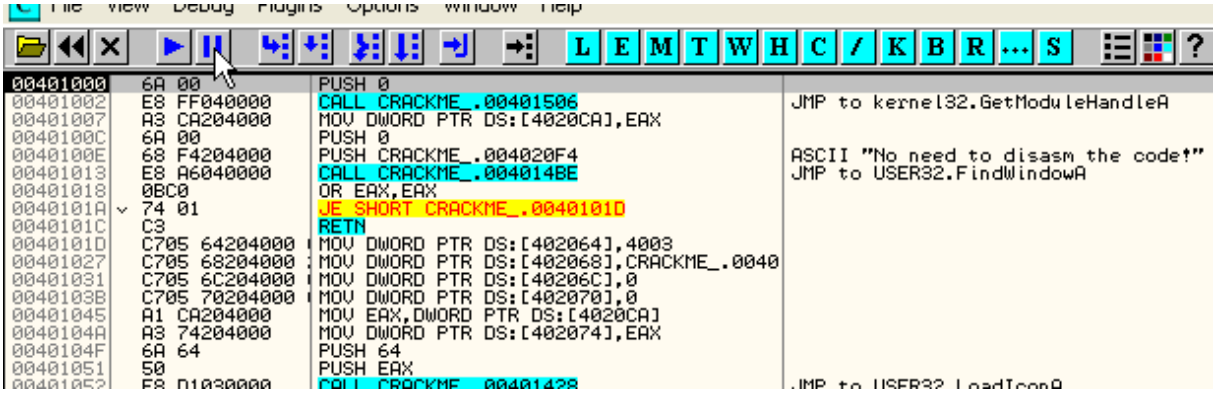


Дойдя до сюда, видим как распаковщий в поте лица перемещает байты в цикле, заполняя между тем первую секцию изначальными значениями. После завершения этой процедуры, JMP совершает переход на OEP.

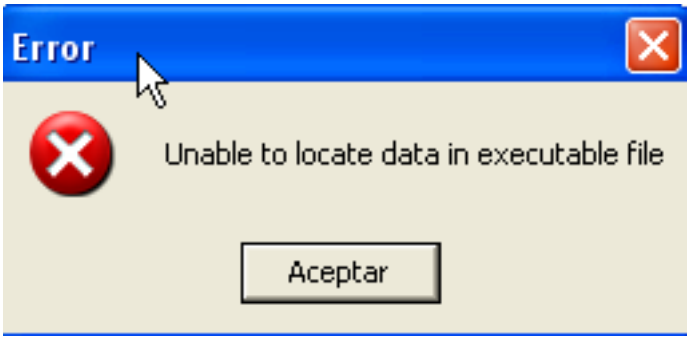
Остановившись на JMP, нажимаем F7, чтобы попасть на OEP.



Находимся на OEP. С помощью OllyDbg мы не сможем менять и сохранять код, как привыкли в данном случае.



Если, например, возьмём два первых байта `6A 00` и решим поменять иа, скажем, `90 90`, а затем попытаемся сохранить эти изменения, OllyDbg сообщит:



То есть нельзя этого сделать, так как OllyDbg не встретила этот код в исполняемой файле.

Но если настоять и с помощью шестнадцатеричного редактора поместить `90 90` в `401000`, открыть крэкми в OllyDbg, а затем посмотреть, что находится в `401000`, то сначала будут идти `90 90`, нули, а кода запустится распаковочная процедура и сохранит изначальные значения в первую секцию, наши изменения будут перезаписаны.

То есть, если хотите изменить эти байты на самом деле, то нужно отыскать, откуда распаковщик читает эти байты на самом деле, посмотреть, какие операции применяются к ним, чтобы сконвертировать их в изначальные значения, после чего можно высчитать, какие новые значения должны быть, уфф.. Слишком много работы, гораздо проще распаковать файл, чтобы можно было изменять, всё, что душе заблагорассудится, хе-хе.

Конечно, существуют другие техники для патчения в памяти, например, лоадеры, но это не является темой данной части. Лоадеры загружают крэкми, ждут, когда произойдёт распаковка в память, находят нужные байты, после чего делают необходимые измениня в памяти, чтобы тот заработал, но это совсем другая история, будем пока идти медленно-медленно, хе-хе.

Ладно, на этом заканчиваю этот краткий и простой вводный обхор, в следующих двух частях будем рассматривать распаковку знаменитого крэкми CrueHead'a.

Введение в крэкинг с нуля, используя OllyDbg - Глава 32

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-08-17

Файлы к статье (находится в архиве `wasm_files.zip`: `files/0494/ollydbg32.zip`)

В предыдущей части мы попытались прояснить концецию OEP, то есть первой строки, выполняемой изначальной программой. Это строка в 99% случаев находится в первой секции (хотя в этой главе будет рассматриваться случай, когда OEP не находится в первой секции, такой я вредный, хе-хе).

Видим, что когда доходим досюда, и смотрим память, её содержимое напоминает то, что было изначально, что даёт нам возможность попытаться сдампить и создать файл, который будет максимально близок к исходному, в целях его последующего реконструирования.

Классический метод следующий:

1. Находим OEP
2. Делаем дамп
3. Чиним IAT
4. Проверяем, если ли в файле антидамповые и иные проверки, которые могут препятствовать запуску программу, и исправляем эти проблемы

Это классический метод (с небольшими изменениями, зависящими от упаковщика), который обычно используется. В этой главе мы сфокусируемся на методах, которые можно использовать, чтобы добраться до OEP.

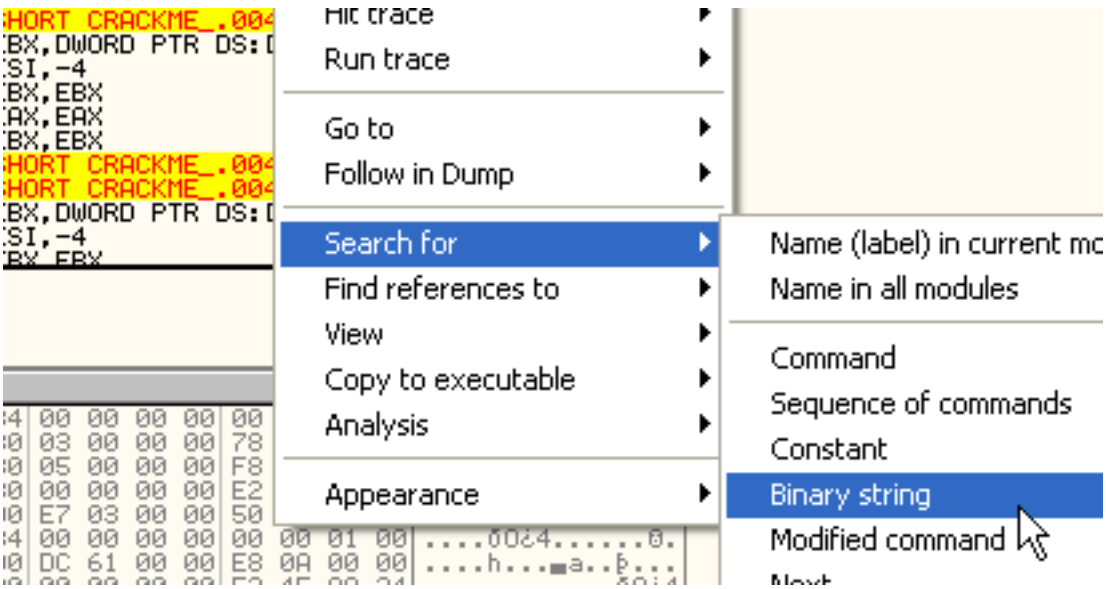
Нужно сказать, что во многих случаях необходимо пробовать и ещё раз пробовать. Во многих случаях эти методы работают, в других упаковщики могут препятствовать работоспособности этих методов, поэтому, конечно, нужно приложить немного изобретательности. Идея заключается в том, чтобы найти OEP, так что мы рассмотрим, как можно использовать для этого инструменты, имеющиеся в нашем распоряжении.

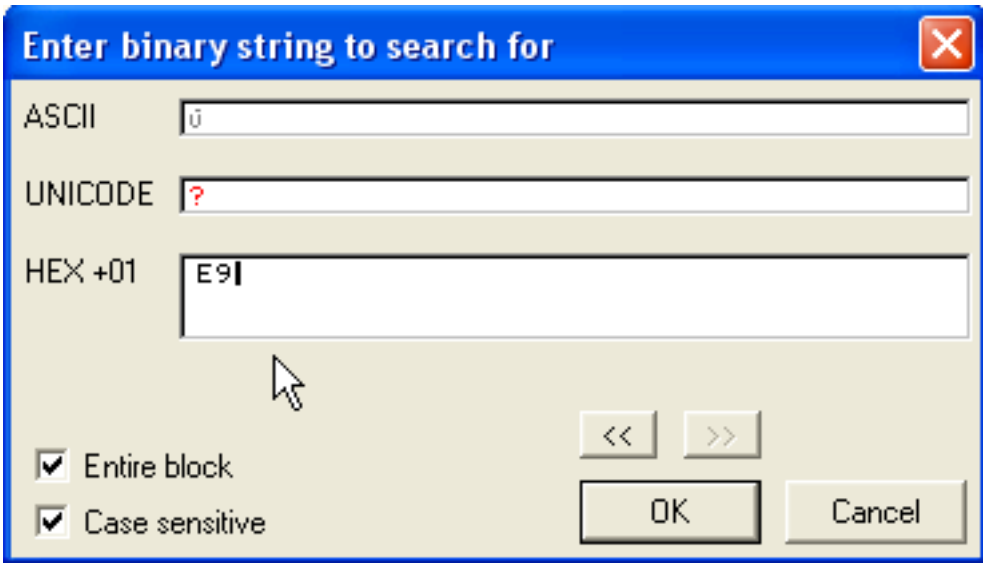
Пока для объяснений будем использовать упакованный крэкми Cruehead'a. Потом рассмотрим другие упаковщики, но для общих разъяснений сойдёт и так.

1) Просмотр или поиск опкодов в в листинге до выполнения процедуры распаковки

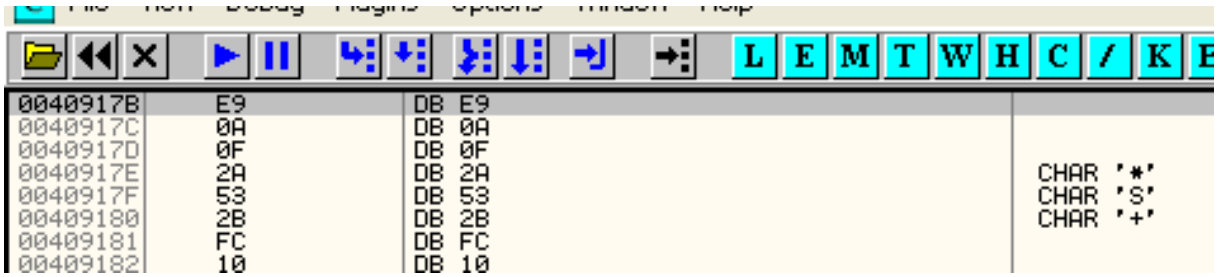
Это сработает только для невинных пакеров. Мне бы не пришло в голову испробовать этот метод на ASProtect, например, хе-хе, но в некоторых случаях можно поискать опкоды JMP LARGO (0E9) или CALL LARGO (0E8), так как пакеру необходиму совершить переход или длинный вызов на первую секцию, чтобы перейти к OEP (если верны молитвы о том, что тот находится в начале).

В данном случае это было бы так:

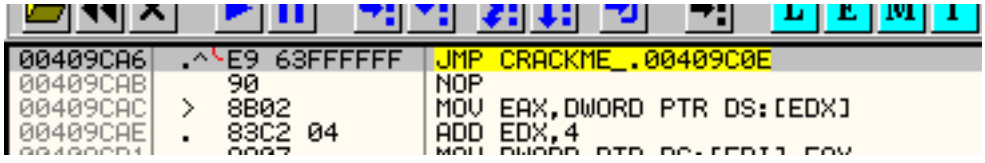




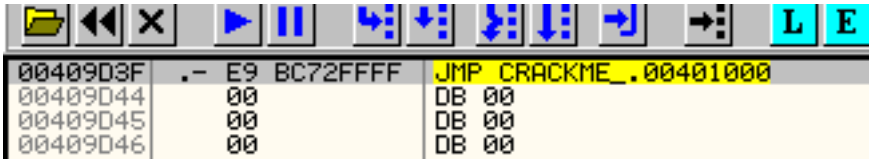
Когда останавливаемся, смотрим, является ли это переход на первую секцию, и если нет, то нажимаем CTRL+L, чтобы искать следующий E9.



Нажимаем CTRL+L.

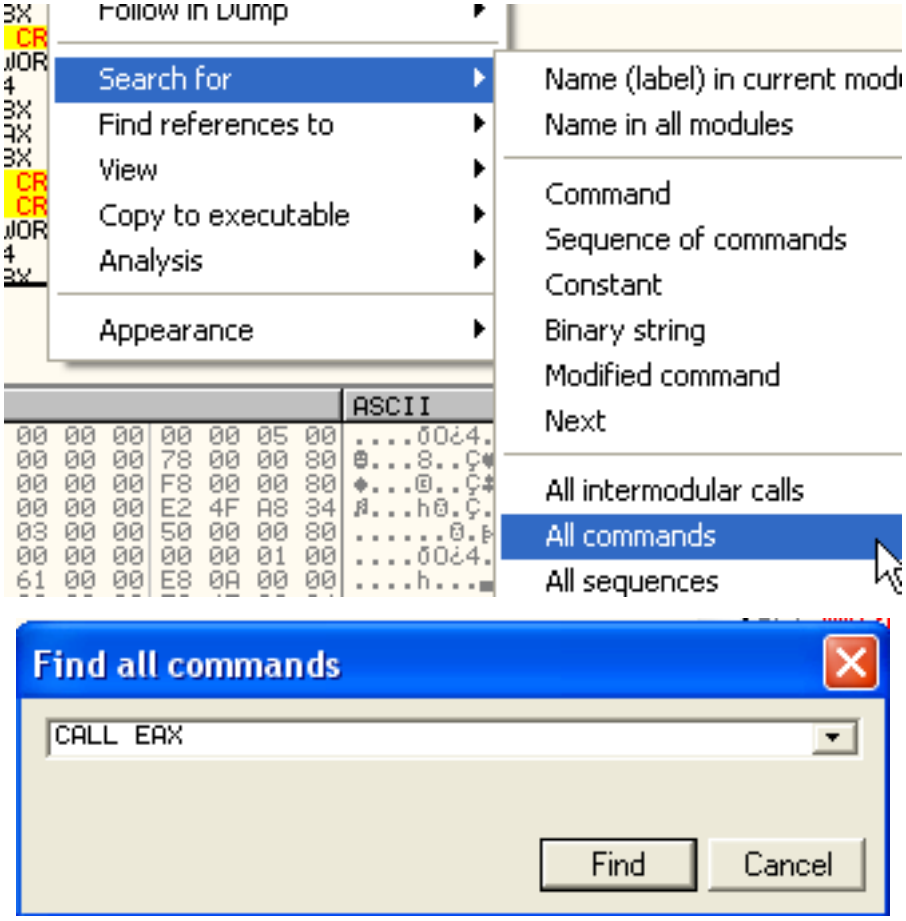


Находим несколько длинных переходов, которые не ведут к первой секции, пока, наконец, не оказываемся здесь:



Это переход на первую секцию. Можем поставить BPX, и когда остановимся, нажимаем F7 и оказываемся на OEP.

Также, если есть желание порыться в коде пакера, можно попробовать “CALL EAX”, “CALL EBX”, “JMP EAX”, так как многие упаковщики используют регистры для скрытия перехода на OEP. В данном случае, хорошо то, это полноценные команды, и их можно искать через “SEARCH FOR – ALL COMMANDS” и установить на все BPX. Рассмотрим пример.

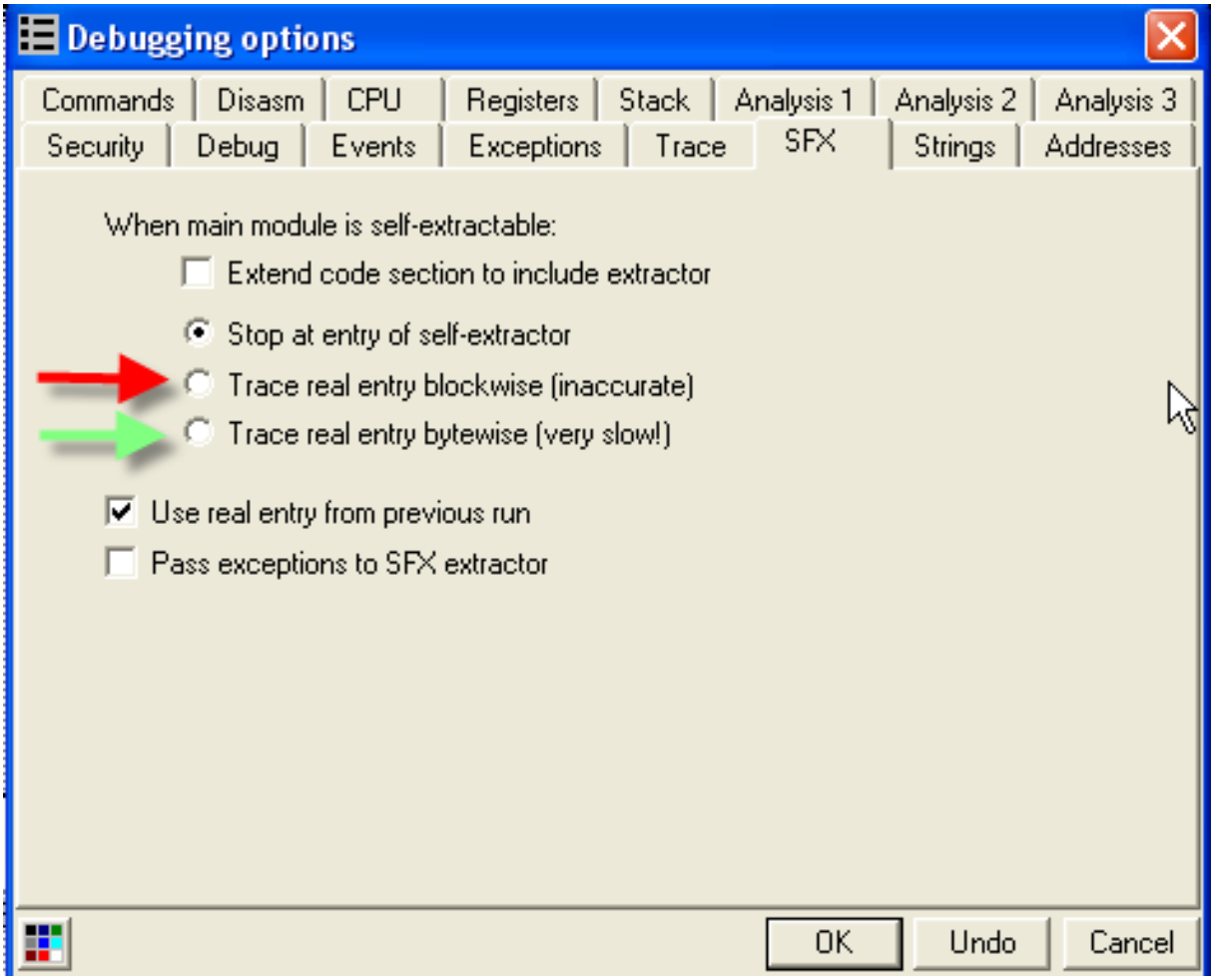


В данном случае не находим никакого результата, но если бы он был, то появился бы список с найденными вхождениями, и можно было бы нажать правой кнопкой мыши на них, чтобы установить BPX. Останавливаясь на каждом из них, можно было бы посмотреть, какое значение находится в EAX, чтобы посмотреть, ведёт ли данный CALL или JMP на первую секцию.

Это способ поиска по листингу не очень часто используется, так как большинство современных распаковочных процедур самомодифицируемые, особенно в районе перехода на OEP, чтобы избежать подобных проверок, но в случае, если упаковщик старый или плохой, то этот метод может сработать.

2) Использование искателя OEP’ов, встроенного в OLLYDBG

Открываем крэкми UPX и идём в DEBUGGING OPTIONS-SFX:



На вкладке SFX видим две опции, чтобы OllyDBG искал OEP'ы. Одна, отмеченная красным, более быстрая, а другая, на которую указывает зелёная стрела, более медленная, но работает временами несколько лучше. Опробуем. Кликаем на опцию, отмеченную красной стрелкой.

Перегрузив крэкми, видим, что в данном случае метод не сработал. Находим объяснение этому.

Self-extracting (SFX) files

Self-extracting file consists of extracting routine and packed original program. When troubleshooting SFX, you usually want to skip extractor and stop on the entry point of original program ("real entry"). OllyDbg contains several functions that facilitate this task.

Usually extractor loads to address that is outside the executable section of the original program. In this case OllyDbg recognizes file as SFX.

When SFX options request tracing of real entry, OllyDbg sets memory breakpoint on the whole code section. Initialy this is empty or contains compressed data. When program attempts to execute some command within protected area which is neither RET nor JMP, OllyDbg reports real entry. This is how bytewise extraction works.

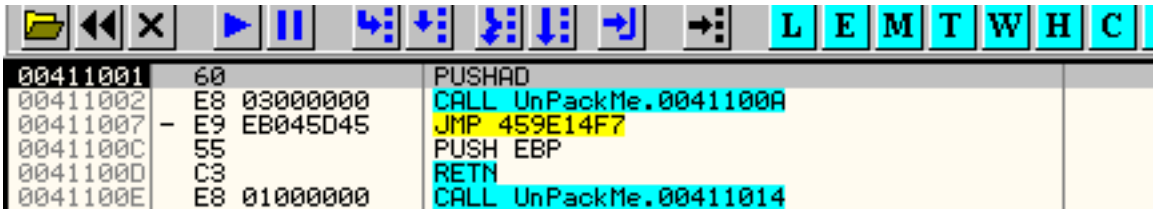
This method is very slow. There is another, much faster method. Each time exception on data read occurs, OllyDbg enables reading from this 4-K memory block and disables previous read window. On each data write exception it enables writing to this block and disables previous write window. When program executes command within remaining protected area, OllyDbg reports real entry. However, when real entry is inside read or write window, its location will be reported incorrectly.

В помощи к OllyDbg видим, что эта опция работает только тогда, когда OllyDbg обнаруживает, что точка входа находится в секции кода, как в большинстве упакованных программ, однако в данном случае этого не происходит, так как UPX меняет секцию кода на ту, в которой запускается распаковщик, поэтому EP находит в той же секции кода, и OllyDbg ничего не находит, так как всё запаковано, и в данном случае метод не работает, хотя упаковщики, делающее подобное изменение, довольно редки.

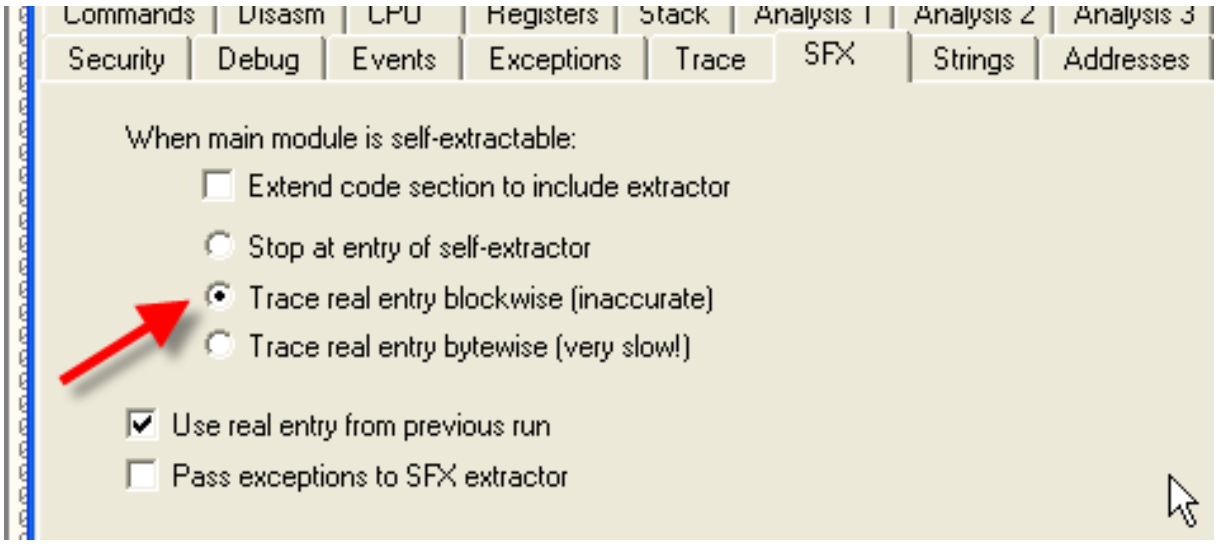
Чтобы продемонстрировать, как работает этот метод, используем другой крэкми, прилагающийся к статье. Он запакован с помощью aspack'a – ещё один простой упаковщик.

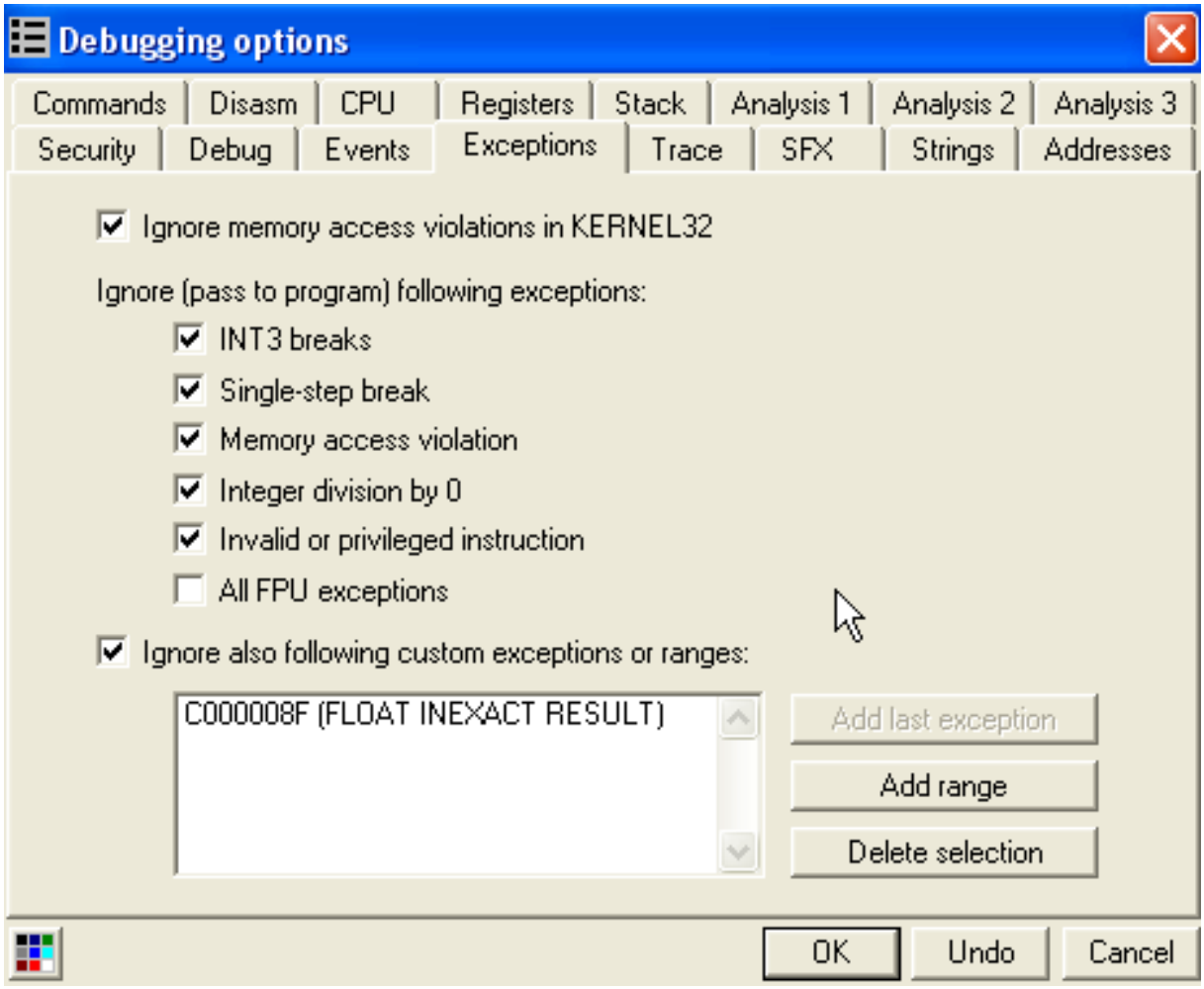
Отметим сначала ту опцию, которая была изначально, и увидим, что OllyDbg определяет наличие пакера, посмотрев, находит ли точка входа внутри секции CODE.

Отображает окошко с соответствующим сообщением, и если нажать на кнопку, то оказываемся в точке входа.



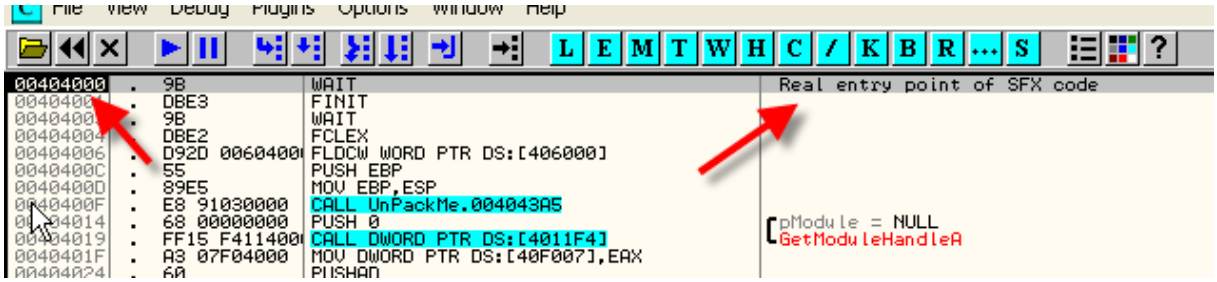
Теперь делаем активной опцию, отмеченную красной стрелкой. Убеждаемся также, что во вкладке EXCEPTIONS также поставлены галочки, чтобы мы не останавливались на исключениях, после чего перегружаем программу в OllDbg.





Делаем RUN.

Видим, что останавливаемся в 404000, который помечается как «REAL ENTRY POINT OF SFX CODE», т.е. «настоящая точка входа SFX-кода» (хотя она не находится в первой секции, как уже было сказано, этот крэкми – особый случай, распаковка происходит в третью секцию, что необычно, но возможно).



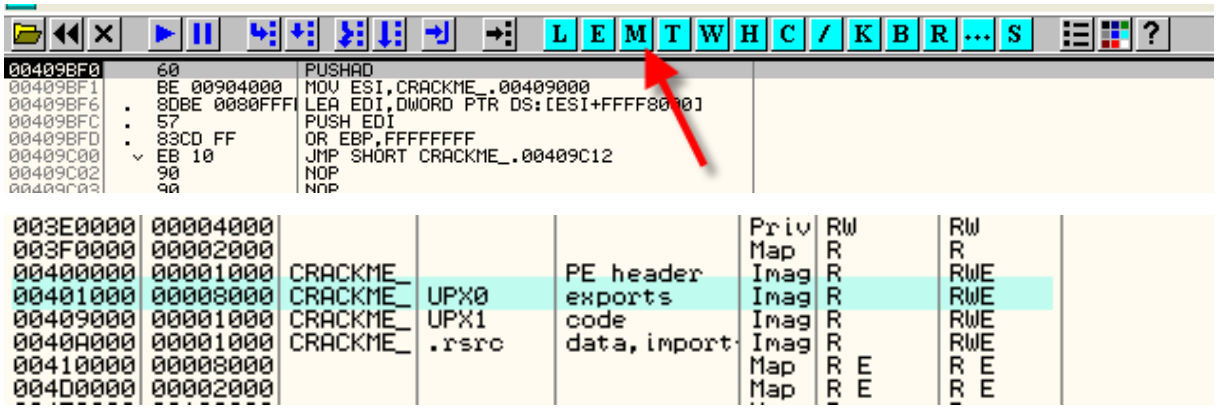
В данном случае метод сработал, это и есть OEP. Теперь посмотрим, насколько будет медленнее, если отметим опцию, на которую указывает зелёная стрелка.

Перезапускаем, работает не намного медленнее, так как код распаковщика невелик. В обоих случаях метод работает, и OEP был правильно определён.

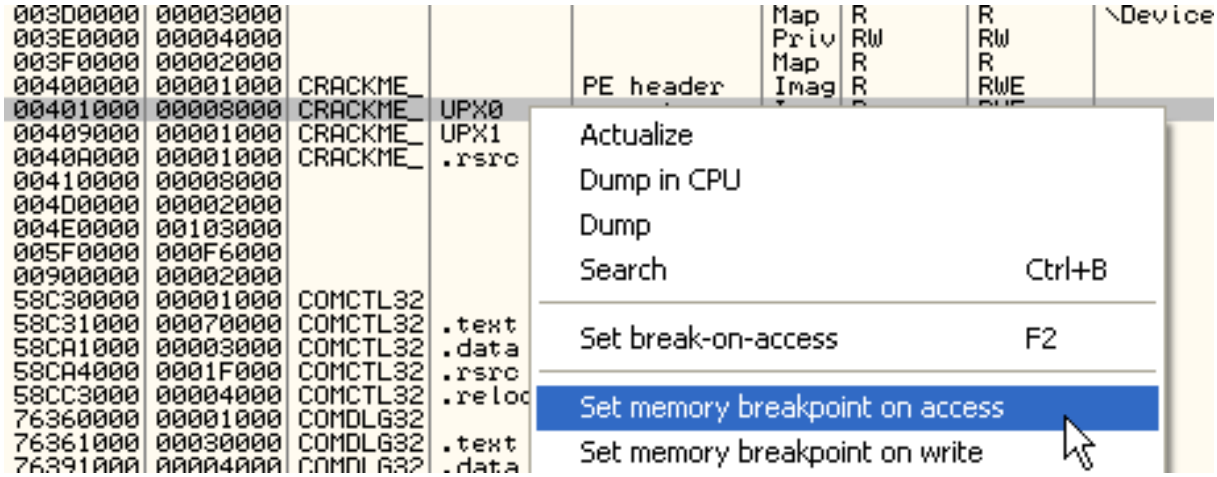
Всегда нужно помнить, что после использования данного метода, нужно установить изначальную опцию, чтобы OllyDbg не искала всё время OEP потом.

3) Использование пропатченного OllyDbg для поиска OEP'ов s

Это тот же OllyDbg, который мы использовали в главах, посвящённых Visual Basic'y. В нём, когда устанавливается BPM ON ACCESS, он срабатывает только на запуск, но не на чтение и запись (ON READ или ON WRITE), что идеально подходит для поиска OEP'ов. Рассмотрим это на примере UPM, идём в M и смотрим секции.



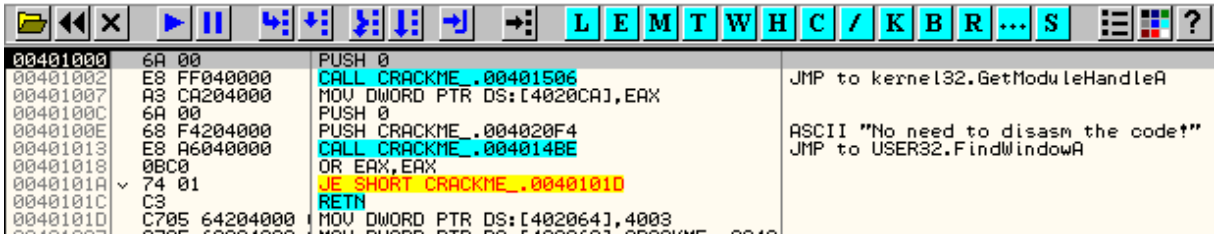
Вот первая секция, в ней распаковщик раскриптовывает и записывает изначальные байты, и нам не нужно, чтобы останов происходил во время этой работы, так как это бы происходило бы тысячи раз до того, как собственно добрались до OEP. Благодаря модифицированному OllyDbg, во время чтения и записи в вышеуказанную секцию остановок происходить не будет, а только когда в ней произойдёт выполнение, а это нам и нужно – узнать, на какой строке произойдёт первое выполнение в первой секции, так как это почти всегда и будет OEP.



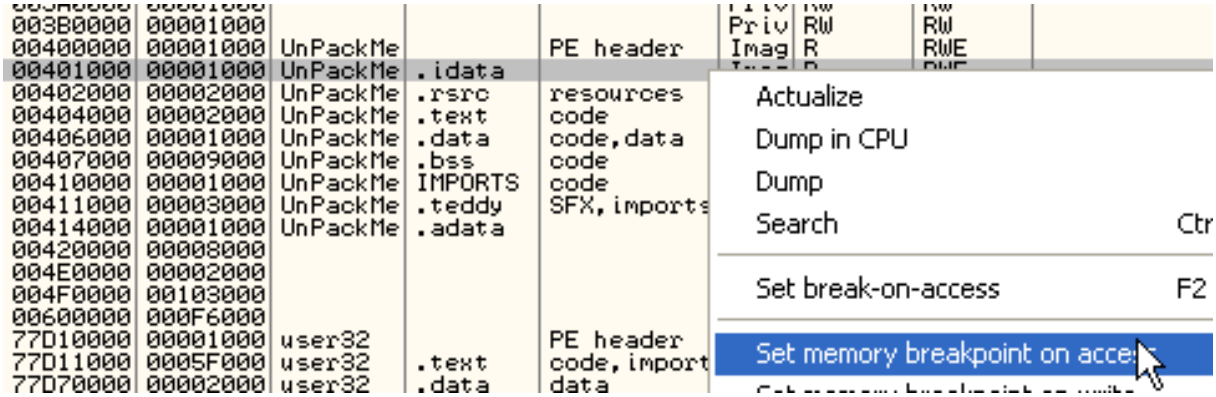
Убеждаемся, что во вкладке EXCEPTIONS отмеченны опции, чтобы не происходило останова во время исключений, делаем RUN, и лично я отправляюсь за чашечкой матэ (те, кто не из Аргентины, могут предпочесть чай или кофе, хе-хе).

Конечно, этот метод довольно медленный, поэтому я и сказал, что можно пойти за чашечкой матэ, в зависимости от упаковщика это может занять от пары секунд до нескольких минут.

Когда возвращаемся с матэ, то видим, что OEP найден:

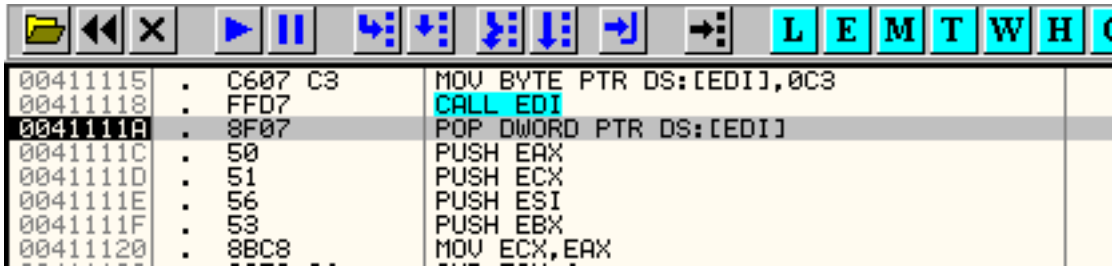


Если попробуем на aspack'e:



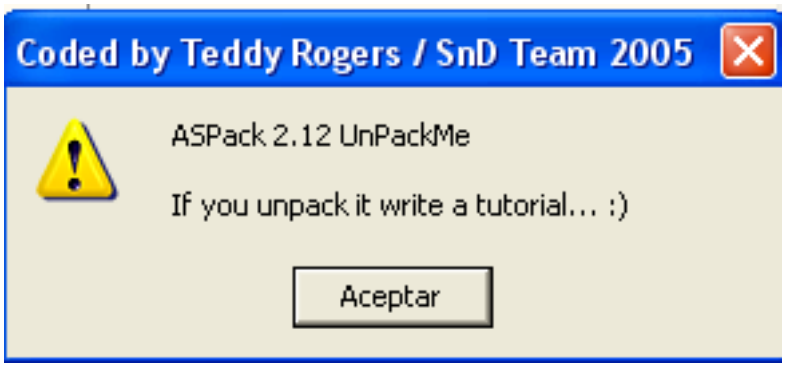
Делаем RUN.

Видим первую строку, где произошло выполнение, смотрим, что случится, если нажмём F7.



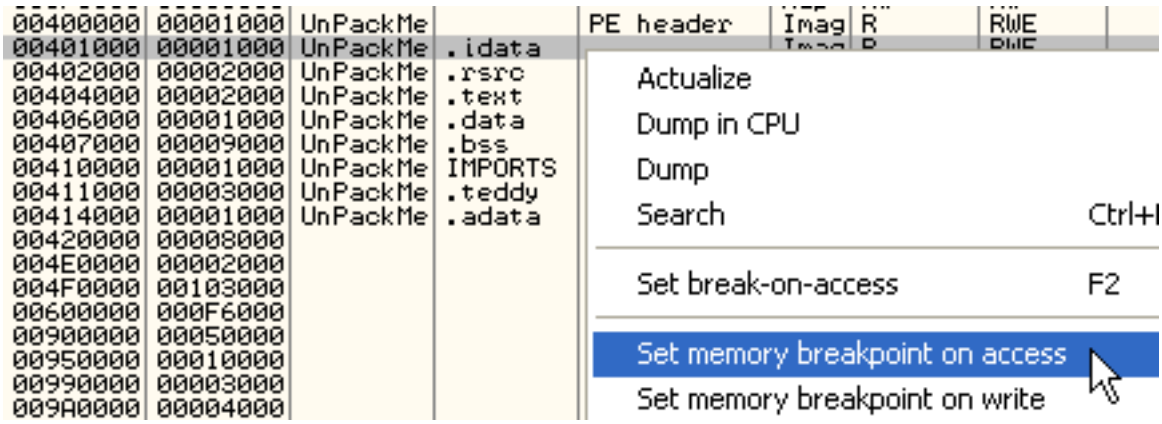
Возвращаемся из процедуры распаковщика, для чего снова жмём RUN, и видим, что программа запускается без остановки. Почему это происходит?

Если посмотрим внимательно, то увидим, что когда ищем OEP с помощью OllyDbg, то в данном случае распаковщик не распаковывает в первую секцию, поэтому нужно установить BPM ON ACCESS на другую секцию, а не на первую. Для того, чтобы определить на какую именно, нужно запустить крэкми без установки BPM'ов.

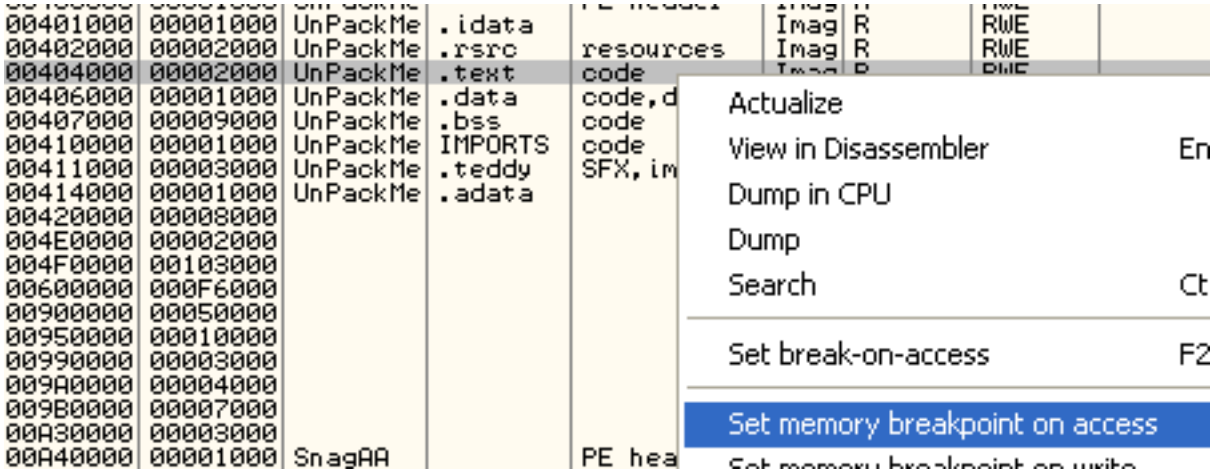


.

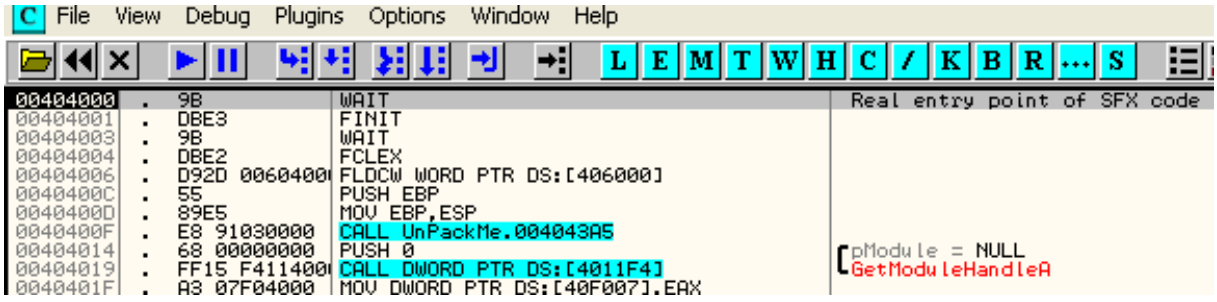
Как того появится вышеукаазанное окно, то мы знаем, что произошла распаковка в память. Теперь, чтобы узнать, в какой секции происходит выполнение, можем установить BPM ON ACCESS в каждой секции по очереди, и если программа продолжает выполняться и не останавливается в OllyDbg, то это значит, что в данной секции выполнения не происходит, и нужно пробовать следующую.



Устанавливаем BPM ON ACCESS на первую, и ничего не происходит, на вторую – тоже ничего, затем смотрим третью, которая начинается в 404000.



Видим, что происходит останов в OllyDdbg при попытке посмотреть окно крэкми, это значит, что это и есть секция, где приосходит выполнение, так что повторяем процесс сначала, чтобы найти OEP, но BPM ON ACCESS уже ставим сразу на третью секцию.

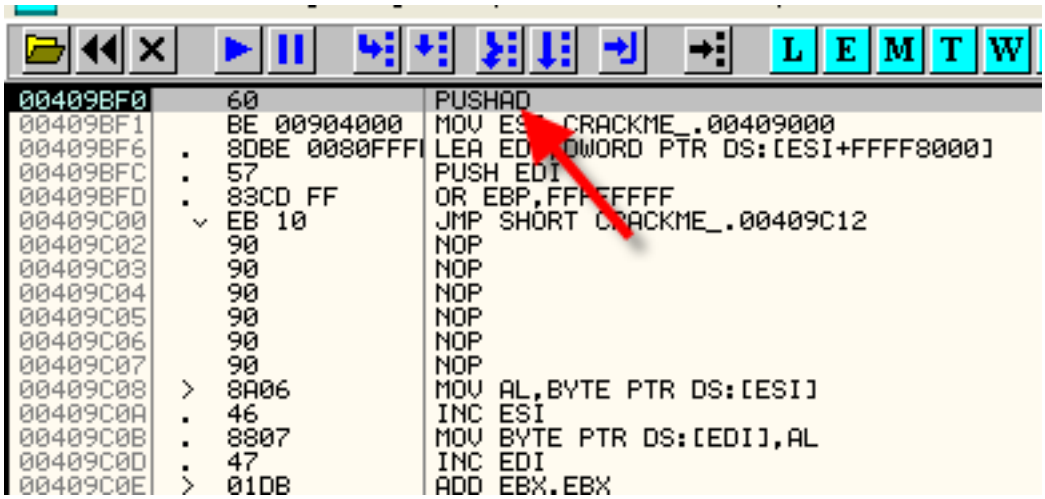


Идём за матэ, кофе, в общем за чем хотите, и когда возвращаемся, находим, что произошёл останов на OEP, хе-хе. Это ещё один метод, который работает на множестве упаковщиков.

4) Метод PUSHAD

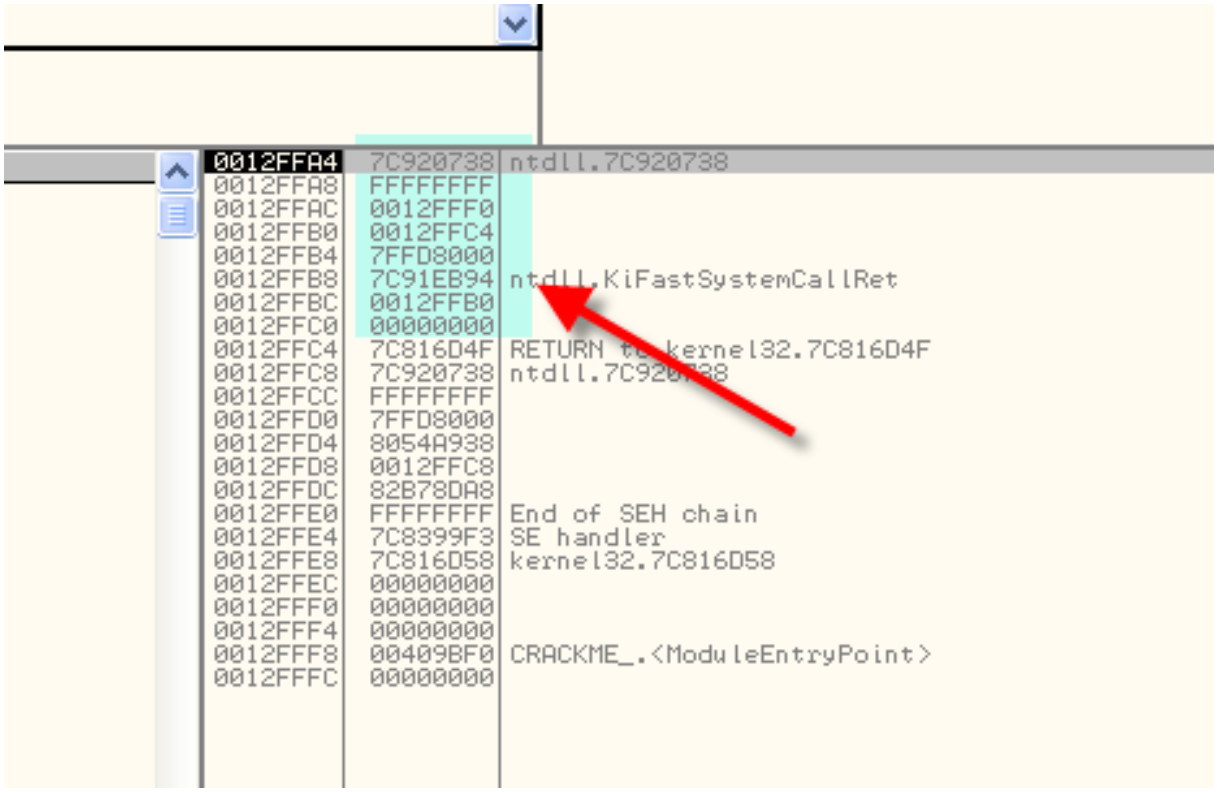
Этот метод работает на многих упаковщиках и основывается на предыдущем. Многие упаковщики сначала выполняют инструкцию PUSHAD, чтобы сохранить исходные значения регистров, а после распаковки, но до перехода на OEP, получают значения обратно с помощью инструкции POPAD.

Смотрим в крэкми, запакованном UPX.



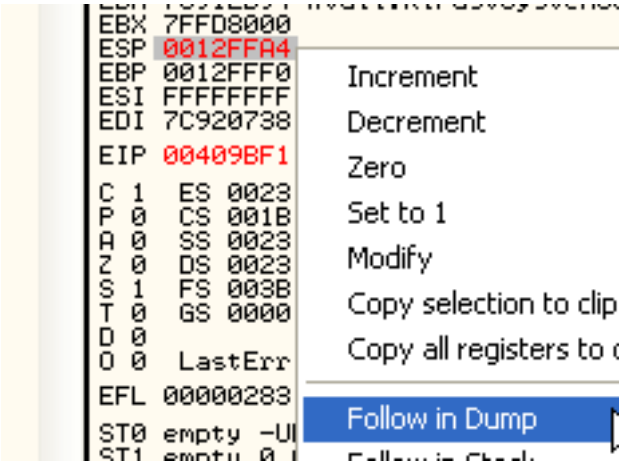
Видим, что PUSHAD находится в начале, иногда он может находится немного дальше, в иногда упаковщики применяют отдельный PUSH для каждого регистра (PUSH EAX, PUSH EBX и так далее), но в случае, когда сохраняются изначальные значения регистров, они должны быть извлечены обратно до перехода на OEP.

Если выполним PUSHAD с помощью F7:

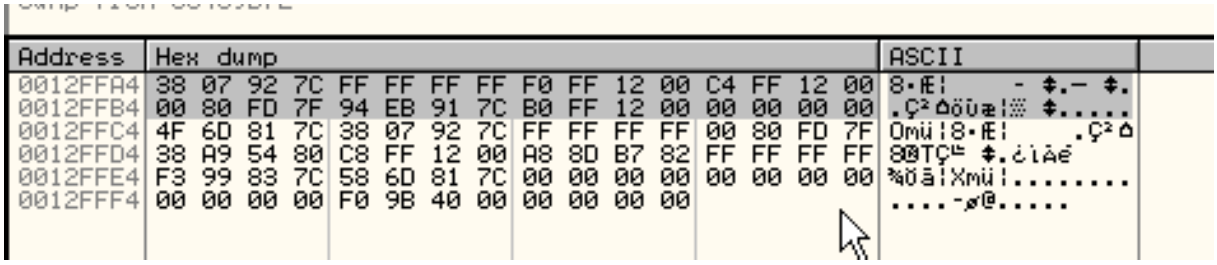


Видим, что сюда были сохранены изначальные значения регистров, и если они считываются до перехода на OEP, можем установить HARDWARE BPX ON ACCESS на одно из этих значений, чтобы остановиться, когда оно будет читаться, и окажемся непосредственно перед переходом на OEP.

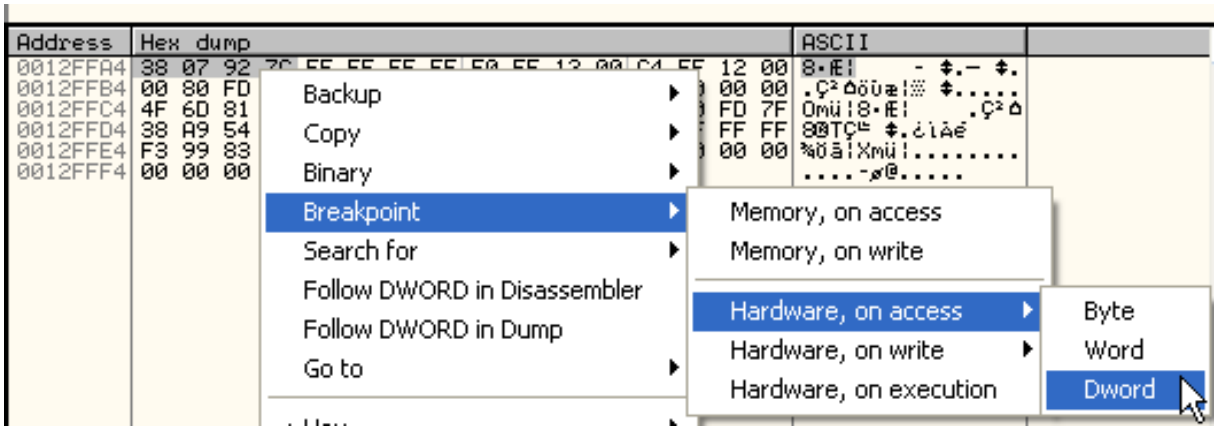
Ищем эти значения в DUMP, отмечаем регист ESP и делаем FOLLOW IN DUMP.



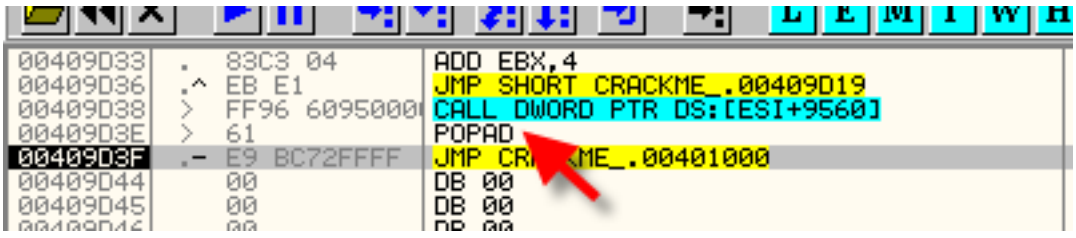
Через DUMP нам показывается содержимое стека.



Здесь видим сохранённые значения, можно отметить первый байт или первые четыре и установить HARDWARE BPX ON ACCESS.

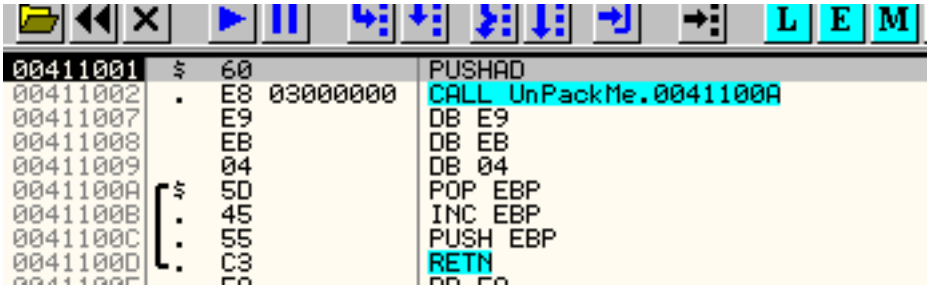


Всё равно, что выбрать, BYTE, WORD или DWORD, главное, что остановка произойдёт во время чтения этого значения. Нажимаем RUN.

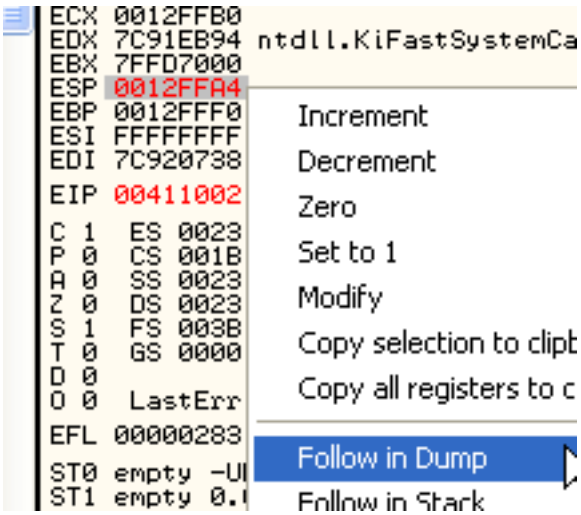


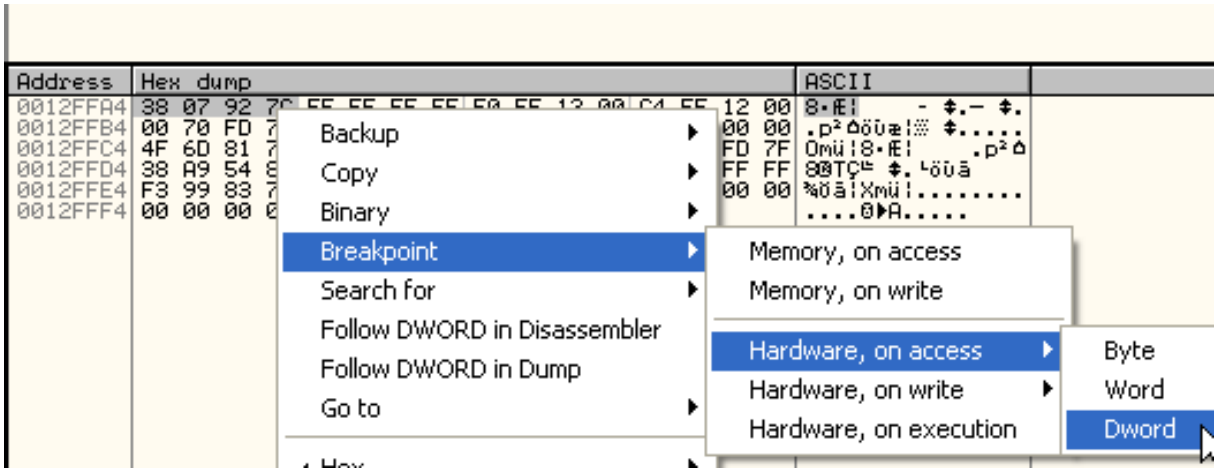
Видим, что для восстановления сохранённых в стек значений выполняется POPAD, они считываются и происходит останов, и чуть ниже находится переход на OEP, так что можно поздравить себя, что этот метод прекрасно сработал.

Пробуем на aspack'e.



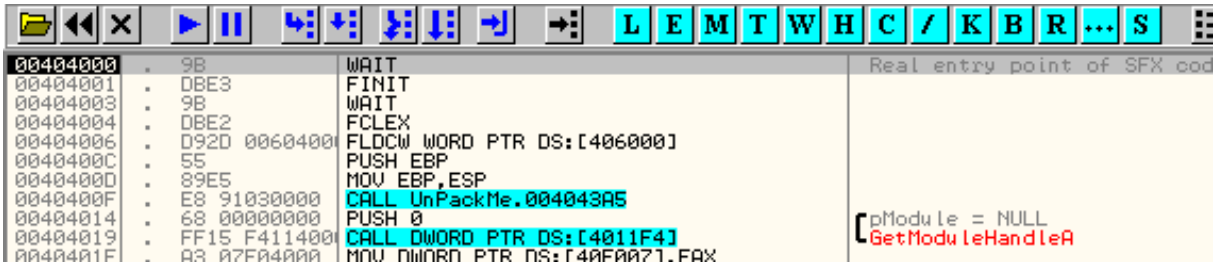
Видим PUSHAD, проходим его с помощью F7, а затем делаем ESP->FOLLOW IN DUMP.





И делаем RUN.

Видим, что останавливаемся тут прямо перед переходом на OEP, который в данном случае выглядит как PUSH 404000, а затем RET. Трассируем с помощью F7.



Без проблем доходим до OEP.

Нужно сказать, что многие новые упаковщики имеют защиту от данного метода, но ладно, нужно знать разные методы и пробовать, какой из них подойдёт.

Продолжаем рассматривать методы для поиска OEP.

5) Для программ, написанных на Visual Basic'e (NATIVE или P-CODE)

Ок, найти OEP запакованной программы на VB очень легко, так как в ней всегда есть PUSH и CALL API-функции VB в начале, так что используем OllyDbg для VB, и когда запустим программ и остановимся на точке входа, идём в M, ищем DLL VB, и на секцию кода данной DLL ставим BPM ON ACCESS.

Таким образом программы распакуется и остановится прямо на первой строке, выполняемой библиотекой, а на первой строке стека находится адрес возврата из вызова, который привёл сюда. Этот вызов находится на второй строке после OEP, а на самой OEP – PUSH. Так и находится OEP данным методом.

Понятно, что метод, использующий модифицированный для VB OllyDbg, тоже будет работать, если установить BPM ON ACCESS прямо на первую секцию программы, но есть упаковщики с защитой от этого, поэтому может потребоваться попробовать оба метода, так что вы должны знать, как они работают.

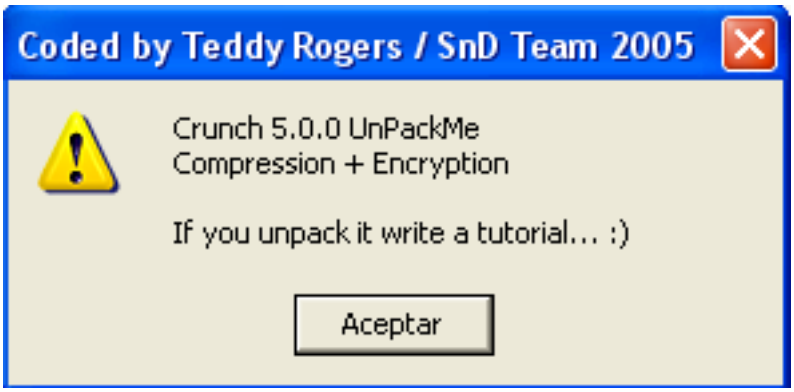
Не будем рассматривать примеры, так как это всё очень легко понять. В будущем, если попадётся подходящий запакованный VB-крэкми, опробуем этот метод в деле.

6) Метод исключений

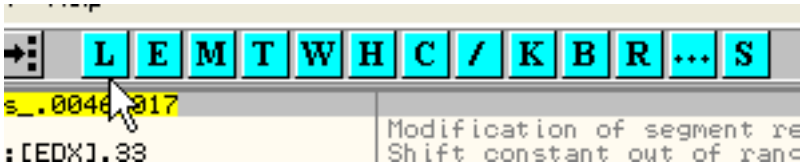
Если у нас есть упаковщик, который генерирует множество исключений при распаковке, то существует следующий метод, который мы рассмотрим на примере приложенного к главе крэкми bitarts.

Открываем его в OllyDbg для VB с необходимыми плагинами для защиты от обнаружения.

Включаем все опции во вкладке EXCEPTIONS и запускаем крэкми в OllyDbg, после чего прибываем сюда.

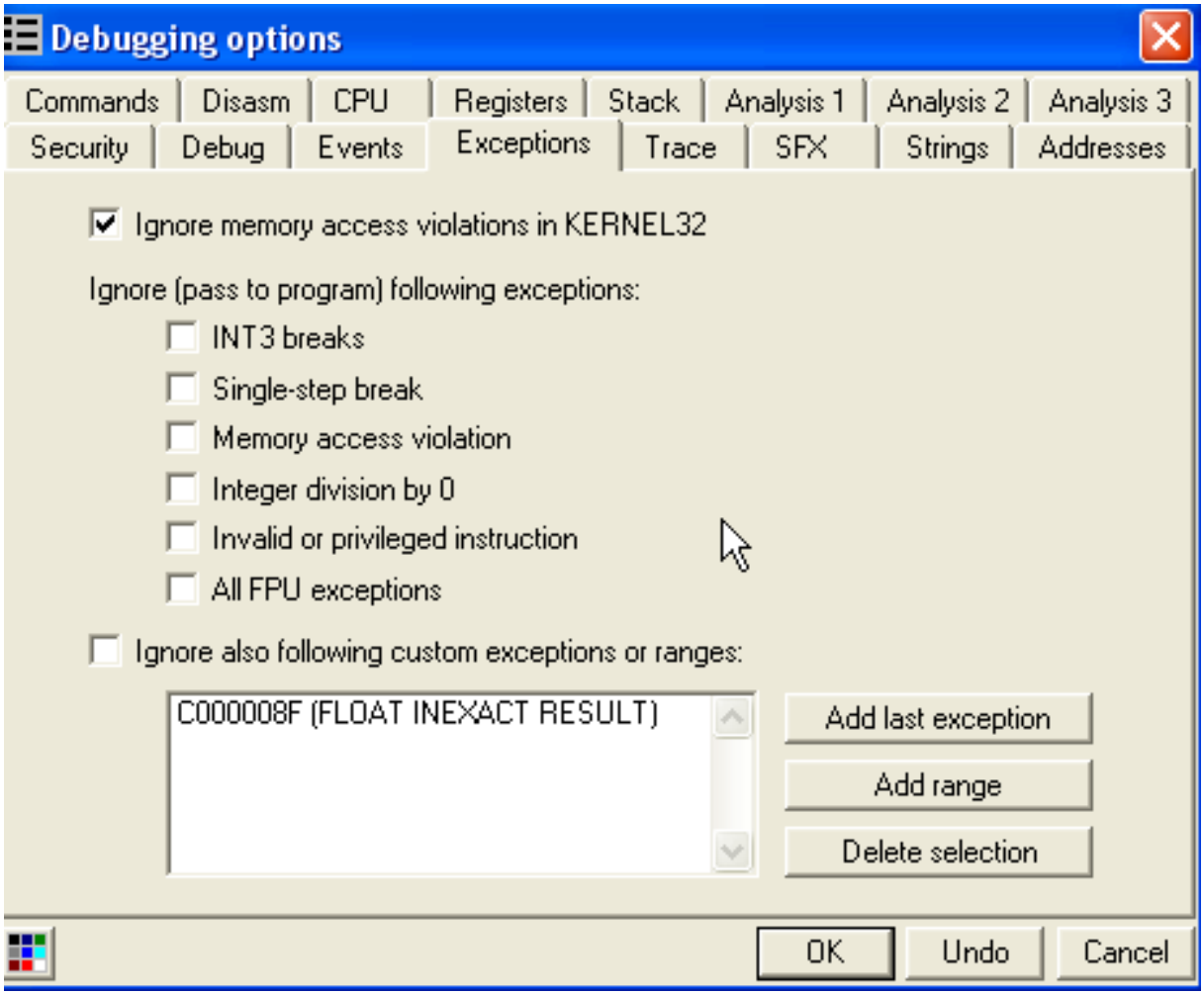


Как только оказались здесь, открываем логи OllyDbg, нажав L.

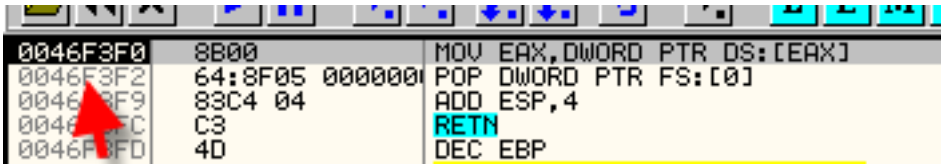


Определяем последнее произошедшее исключение, которое произошло не в первой секции, то есть произошдшее не во время выполнения программы, самое последнее во время распаковки, в данном случае – это 46e88f.

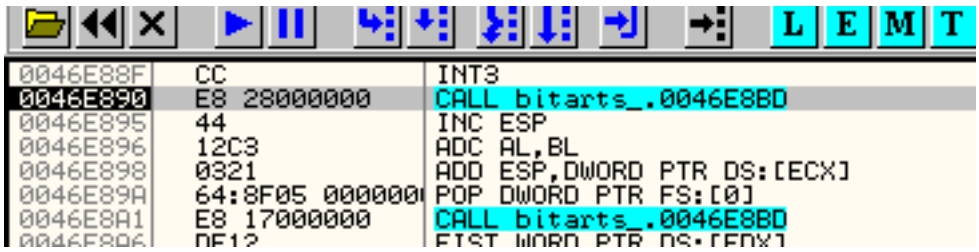
Теперь перегружаем и выключаем все опции в исключениям, оставляя только первую.



Жмём RUN и каждый раз при остановке проходим её с помощью SHIFT+F9, пока не доходим до последнего, которое запомнили, в данном случае это 46e88f.



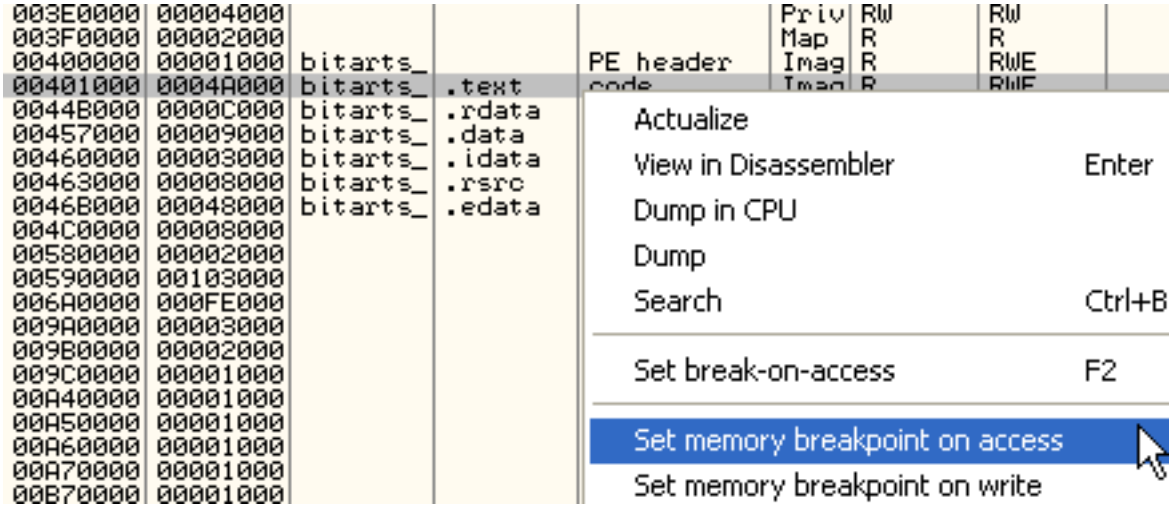
Останавливаемся здесь, это не то, что нам нужно, нажимаем shift+f9, чтобы выйти из исключения. Обратите внимание, что OllyDbg останавливается на следующей после INT3 строки, то есть на 46e890.



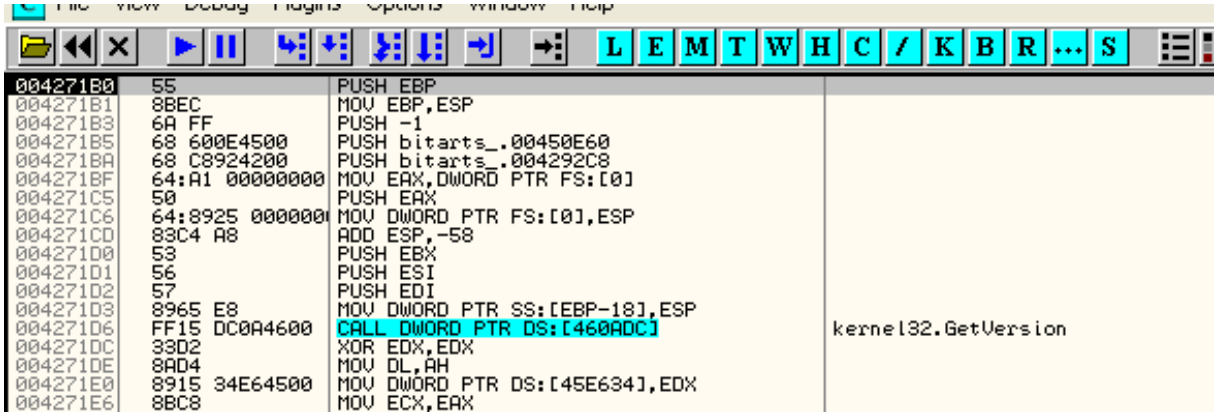
Находимся прямо после последнего исключения, сгенерирвоанного распаковщиком, после чего запускается сама программа.

Здесь есть несколько возможностей. Можно установить BPM ON ACCESS на секцию кода. Многие спростя, почему не установить его в самом начале, и ответ состоит в том, что многие упаковщики могут определять BPM, если он установлен в самом начала, а так мы уже, скорее всего, прошли

этап обнаружения.

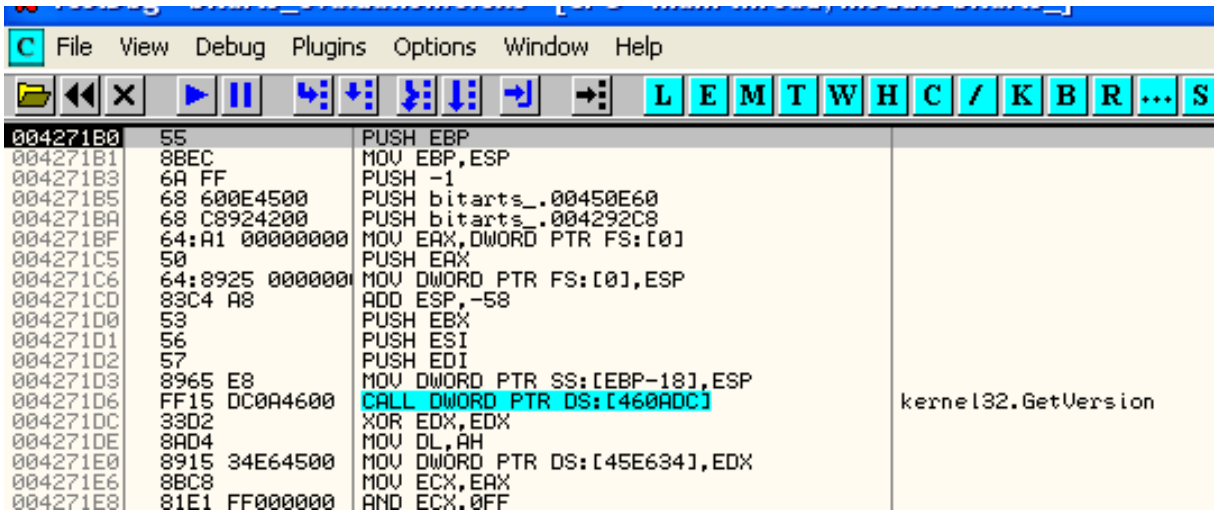


На забываем, что нужно нажимать SHIFT+F9, так как находимся в исключении.



Останавливаемся на OEP, можем попробовать узнать, определяет ли упаковщи BPM, установленный в самом начале.

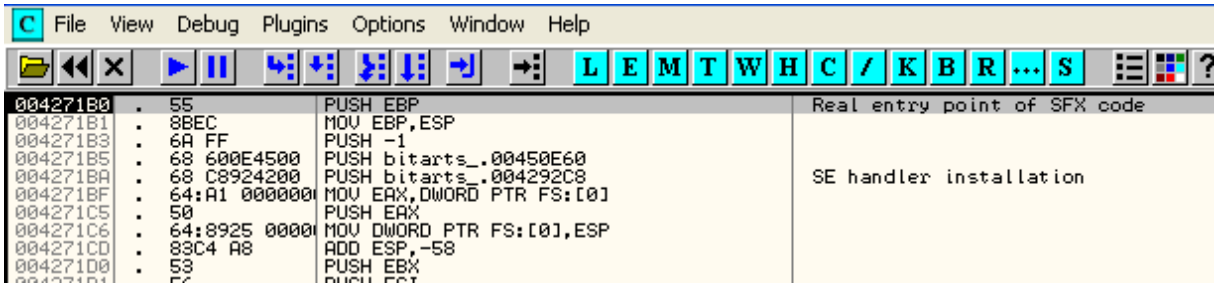
Перегрузим программу и снова включим все опции во вкладке настроек исключений, затем идём в M, где устанавливаемя BPM ON ACCESS на первую секцию, и так как будем использовать OllyDbg для нахождения OEP, то есть время сходить за парой матэ, хе-хе, так как в данном случае это может занять несколько минут.



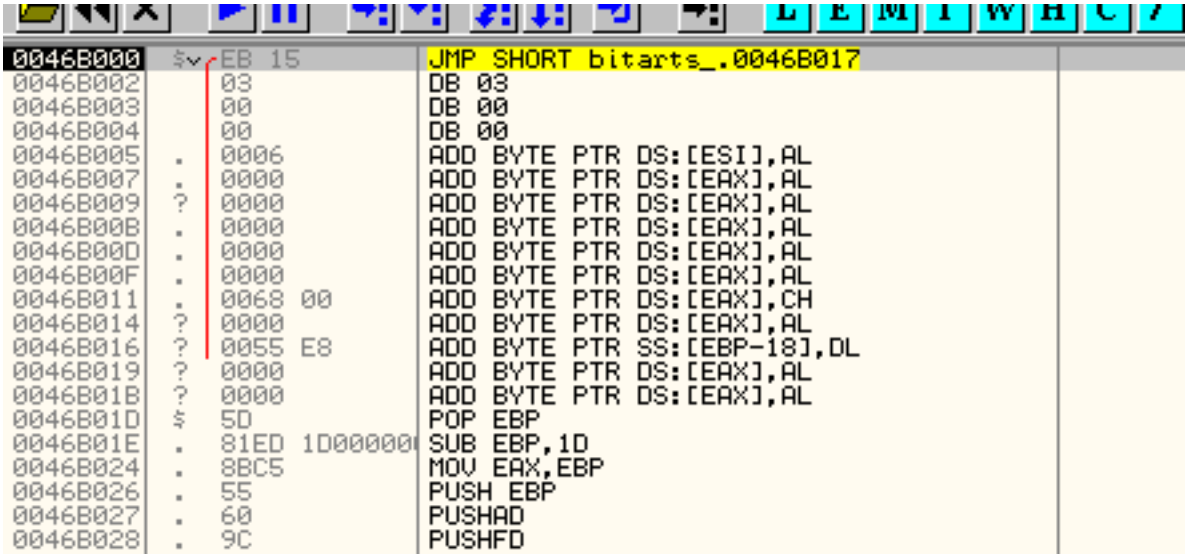
Видим, что всё сработало замечательно, но знать метод исключений полезно, если BPM ON ACCESS определяется упаковщиком, и в этом случае необходимо сначала подобраться к OEP так

близко, как это возможно, чтобы избежать обнаружения.

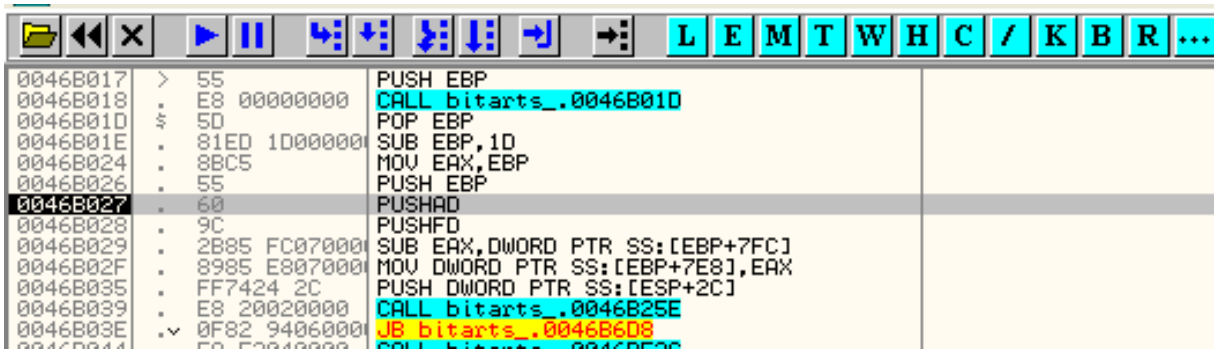
Видим, что для этого этого упаковщика метод поиска OEP с помощью OllyDbg также прекрасно работает.



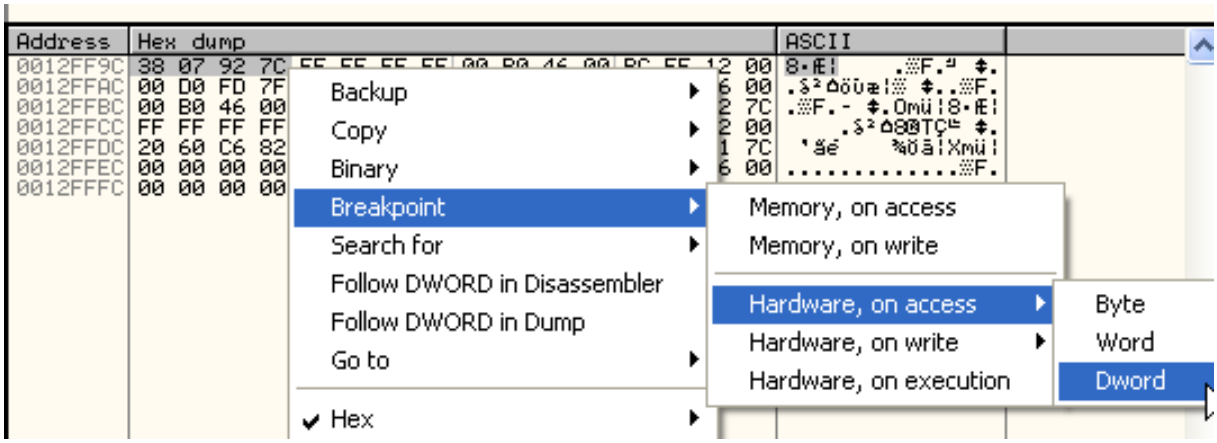
Метод PUSHAD тоже работает, смотрим вначале:



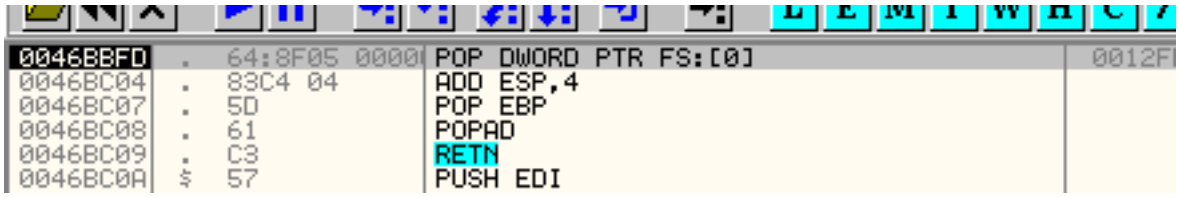
Если оттрассируем немного с помощью F7, то через несколько строк встретим PUSHAD.



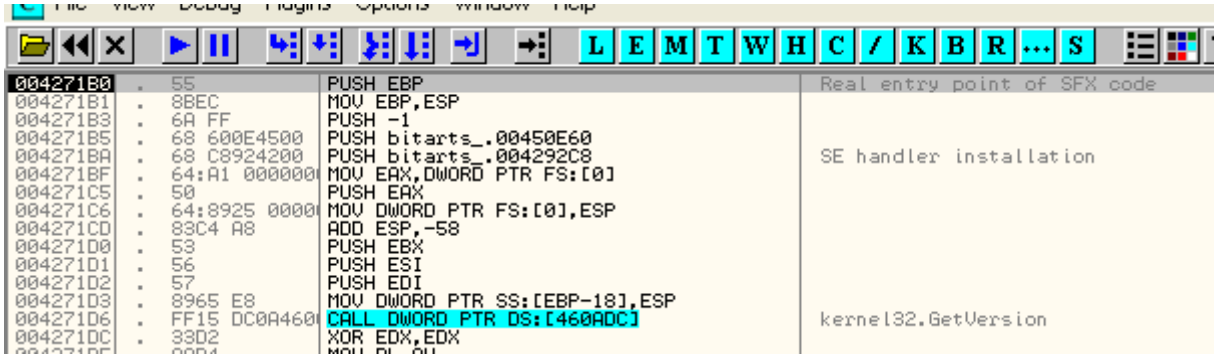
Проходим его с помощью F7 и помечаем (ESP-FOLLOW IN DUMP).



Делаем RUN.

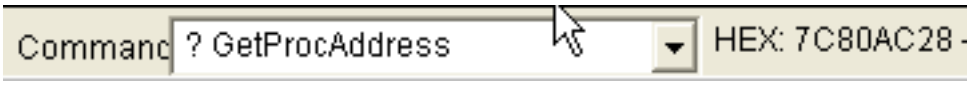


И останавливаемся на считывании сохранённых значений, трассируем до RET, пройдя который окажемся на OEP.

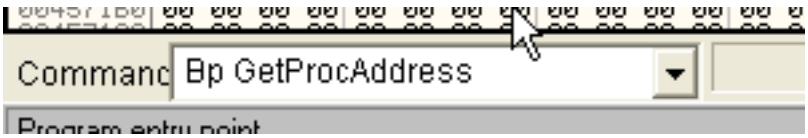


7) Метод, использующий самую используемую распаковщиком API-функцию

Перегружаем этот bitarts в OllyDbg для OEP'ов, отметив все опции во вкладке исключений, и ищем часто используемую API-функцию, обычно это GetProcAddress. LoadLibrary также может довольно часть использоваться, ExitThread, в общем, зависит от упаковщика. Попробуем этот метод с GetProcAddress.

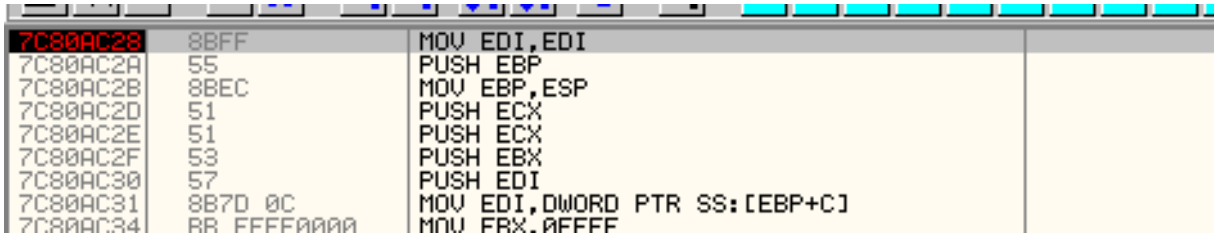


Смотрим в CommandBar'e адрес API-функции и устанавливаем на неё BP.

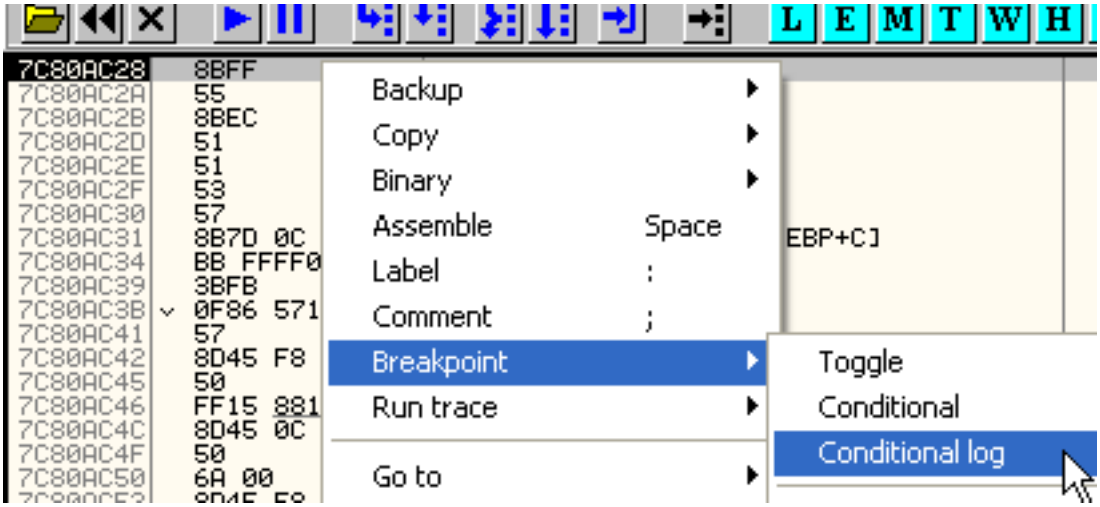


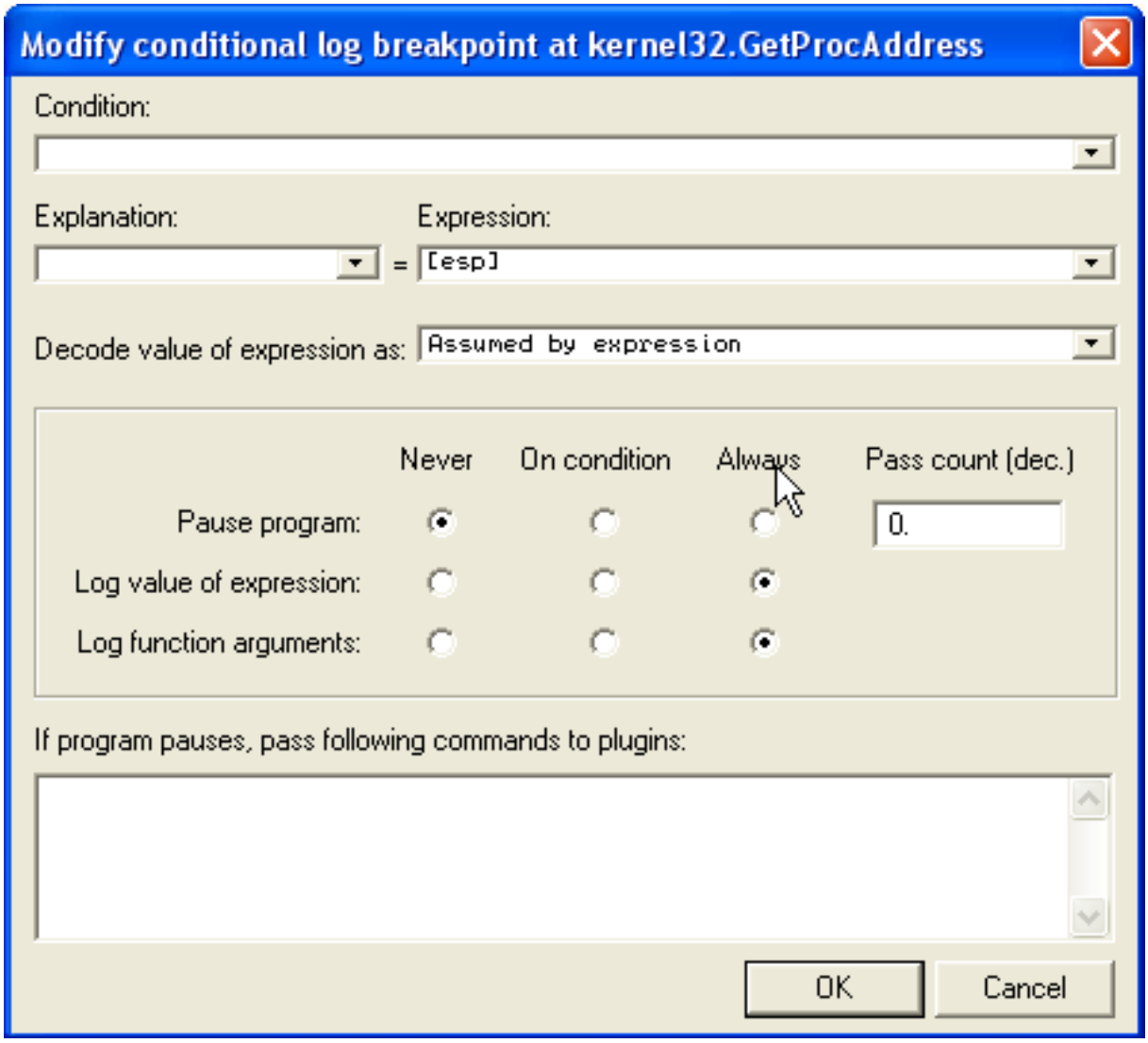
Мы не можем поставить такой BP, какой нам нужен, поэтому нам придётся подправить это вручную.

Делаем RUN.

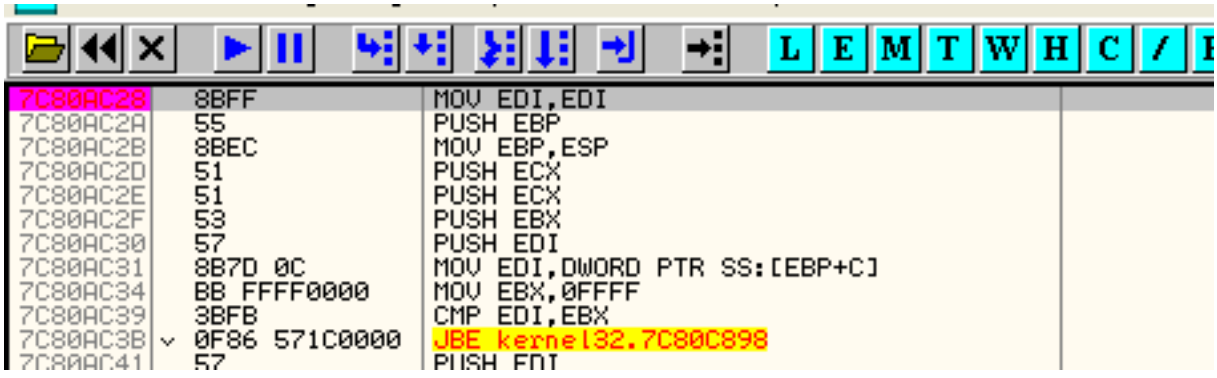


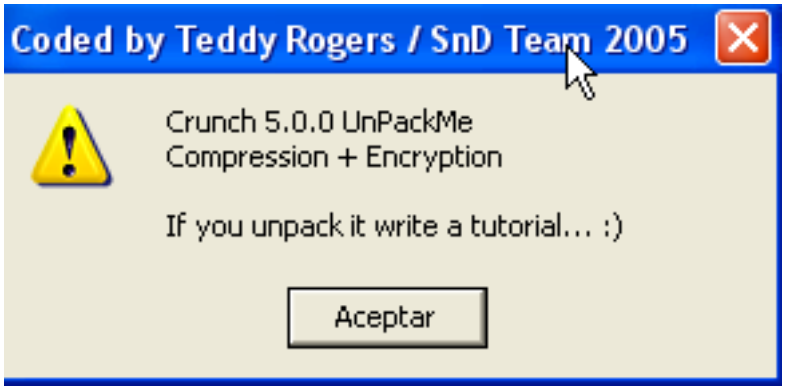
Меняем BP на BP CONDITIONAL LOG, который не производит остановку, а только пишет в лог.



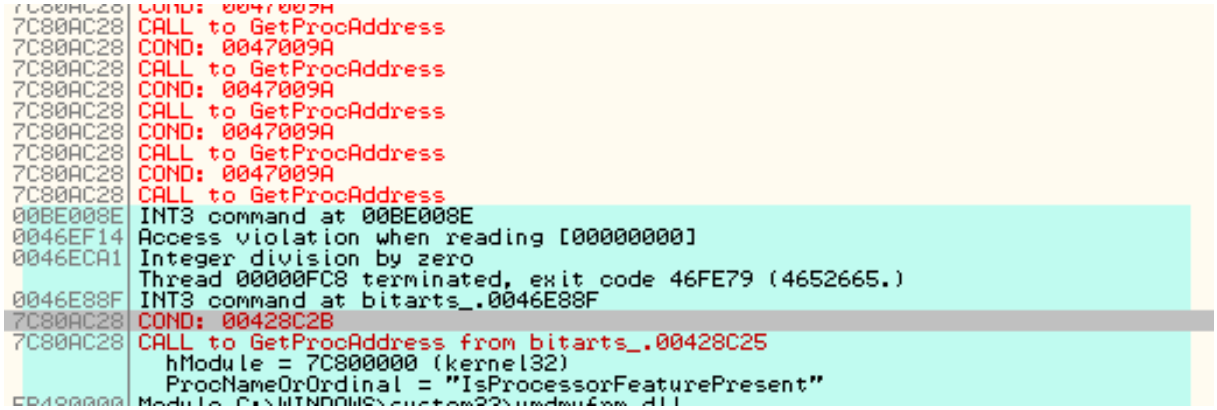


Устанавливаем, чтобы не происходило остановки, но всегда записывалось в лог, и кроме того, чтобы записывалось значение [esp], то есть значение возврата. Это позволит нам узнать, откуда был произведён вызов API-функции. Очищаем лог (правая кнопка мыши-CLEAR LOG).



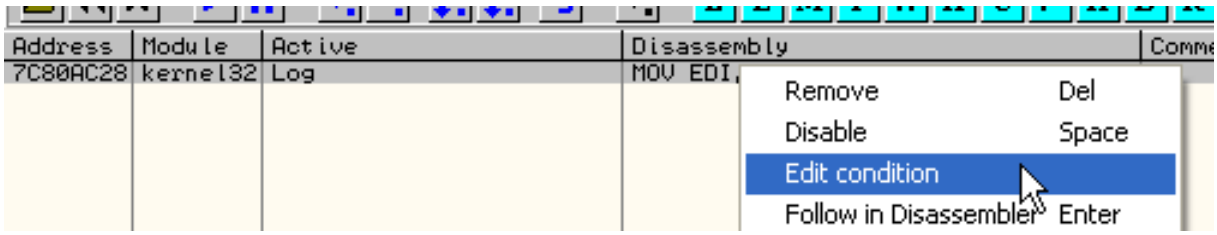


Делаем RUN и оказываемся в собственно программе, смотрим последний вызов нашей API-функции, производимой не из первой секции.

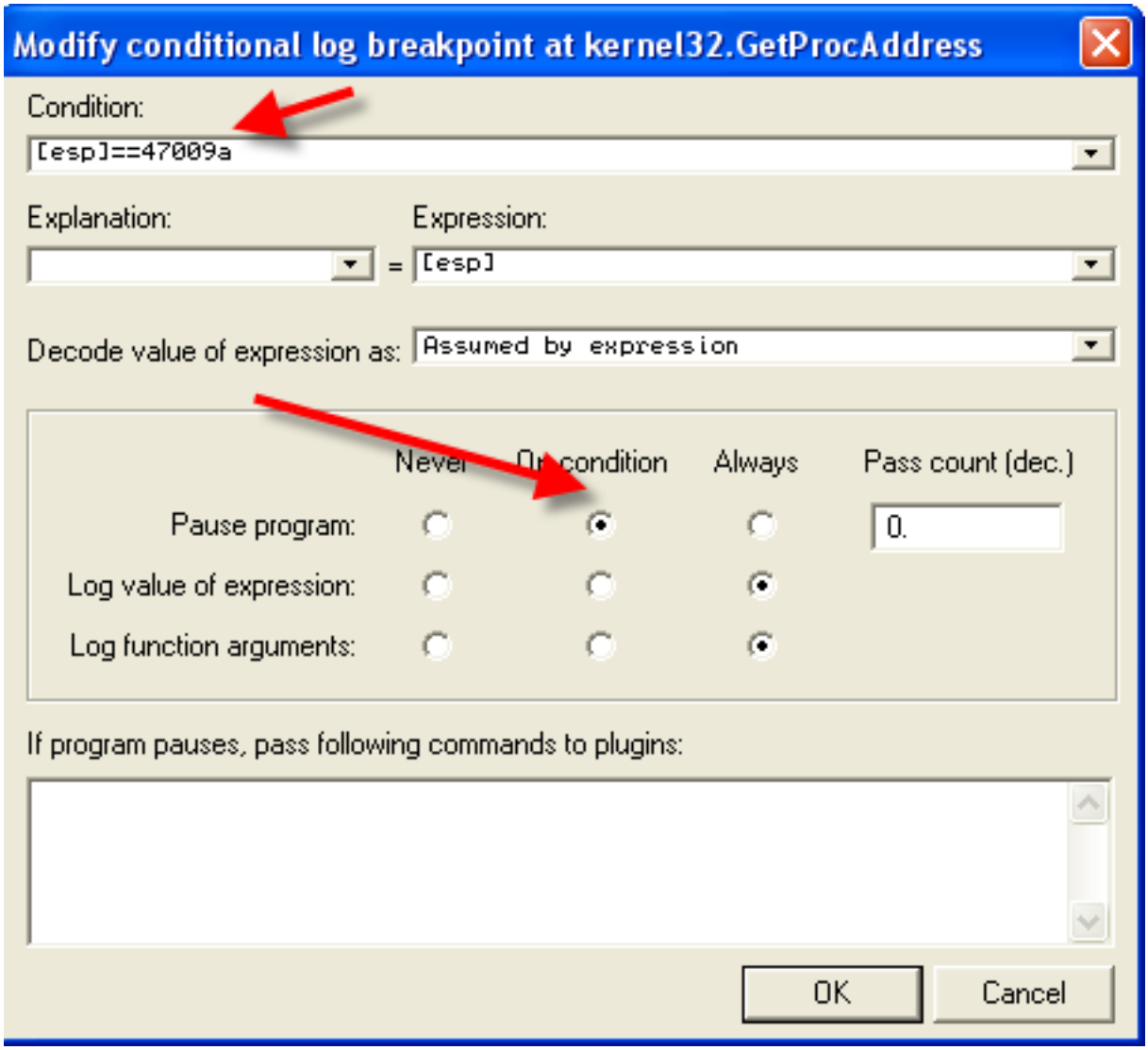


Видим, что все вызовы были произведены из распаковщика, кроме последнего из 428c2B, то есть уже из первой секции программе, а значит, относящегося уже к самой программе. Таким образом, последний раз эта функция была вызвана распаковщиком из 47009A, так что можем установить условие, чтобы там происходил останов.

Перегружаем.

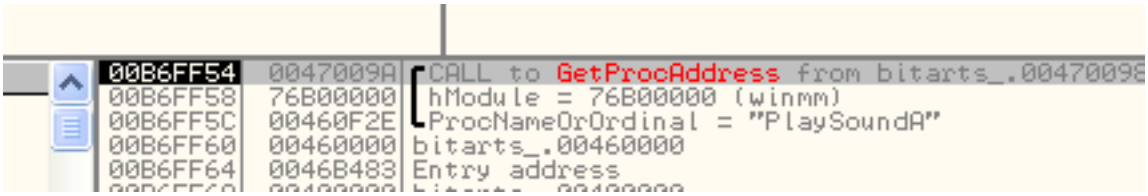


Редактируем BPX CONDITIONAL.



Устанавливаем условие, отмечая опцию останова на ON CONDITION.

И делаем RUN.

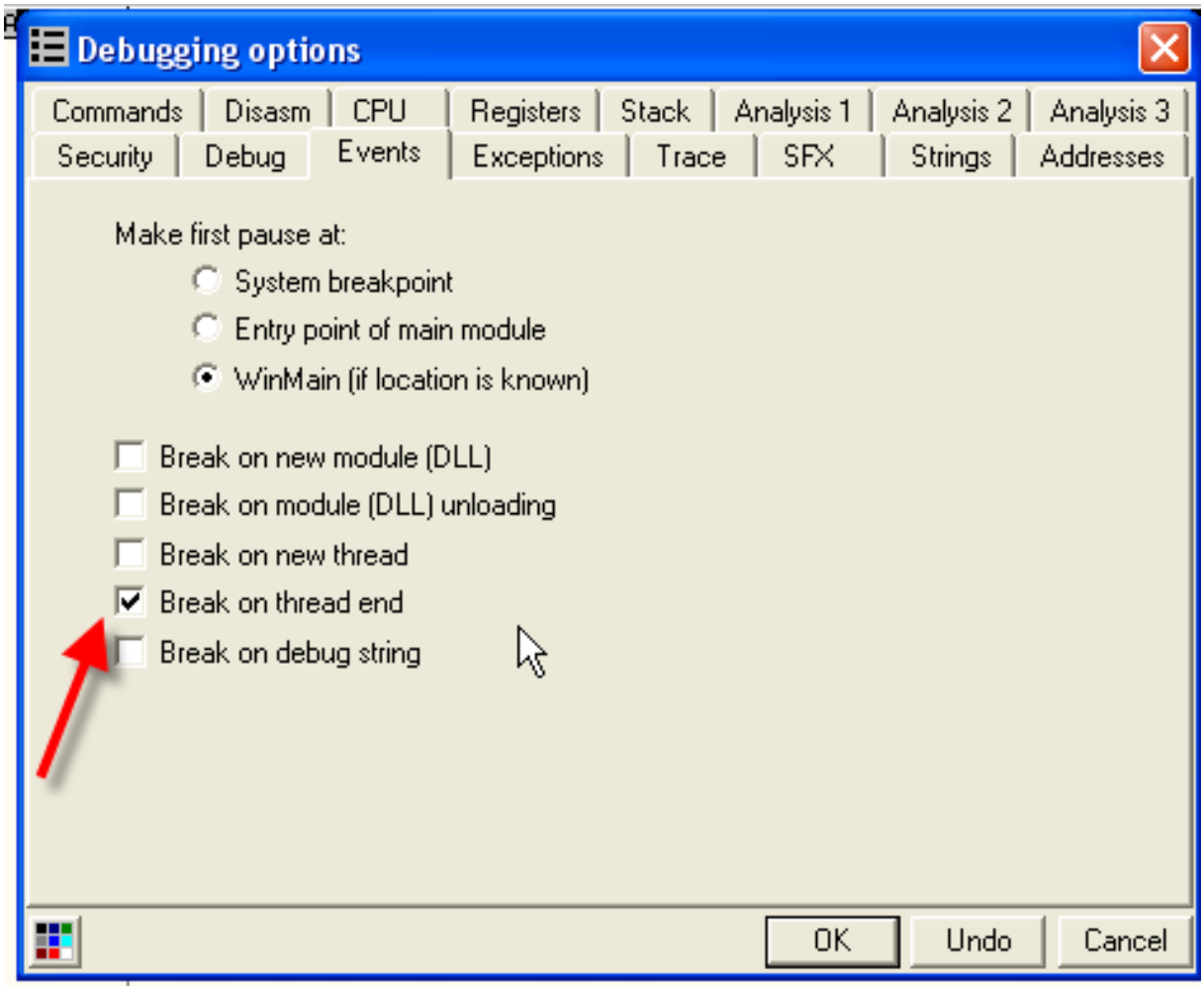


Здесь произошёл останов. Это метод можно сочетать с установкой BPM ON ACCESS на секцию кода, чтобы избежать определение BPM, так как тут у нас такая проблема, что многие новые упаковщики определяют BPX, установленные на API-функции, поэтому во многих случаях лучше подобраться поближе к OEP с помощью BPM ON ACCESS, и оттуда трассировать с помощью автоматического трассировщика, встроенного в OllyDbg (TRACE INTO).

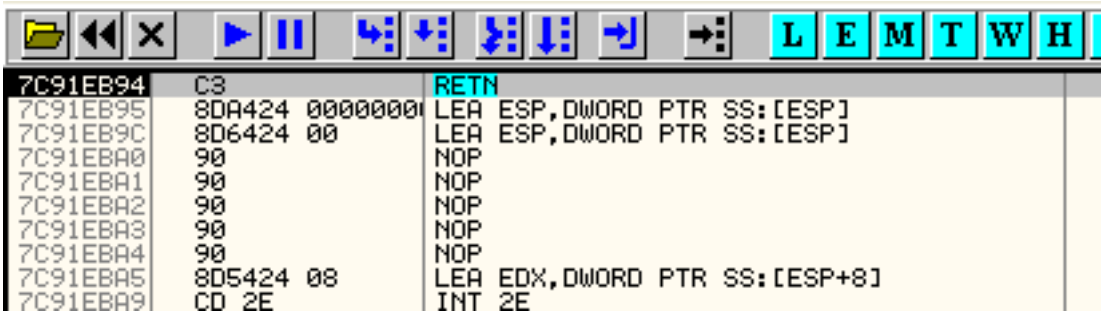
Если подобрались достаточно близко, то можно использовать метод поиска с помощью API-функцию, если повезёт, если нет, можем поменять её. Если заглянем в лог, то увидим, что одна из функций – это завершение треда.

```
0046F3F0 Access violation when reading [00000000]
7C810856 New thread with ID 00000DB0 created
0046EF14 Access violation when reading [00000000]
0046ECA1 Integer division by zero
00B8008E INT3 command at 00B8008E
0046EF14 Access violation when reading [00000000]
0046ECA1 Integer division by zero
00BA008E INT3 command at 00BA008E
0046EF14 Access violation when reading [00000000]
0046ECA1 Integer division by zero
00BC008E INT3 command at 00BC008E
0046EF14 Access violation when reading [00000000]
0046ECA1 Integer division by zero
00BE008E INT3 command at 00BE008E
0046EF14 Access violation when reading [00000000]
0046ECA1 Integer division by zero
         Thread 00000DB0 terminated, exit code 46FE79 (465266
0046E88F INT3 command at bitarts_.0046E88F
5B480000 Module C:\WINDOWS\system32\umdmxfrm.dll
5B150000 Module C:\WINDOWS\system32\uxtheme.dll
746B0000 Module C:\WINDOWS\system32\MSCTF.dll
73260000 Module C:\WINDOWS\system32\RICHED32.DLL
```

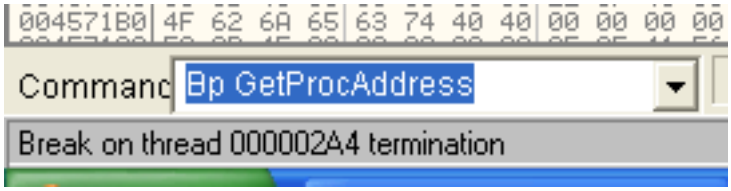
Поэтому можем установить BP ExitThread, подредактировав его конфигурацию в OllyDbg таким образом, чтобы останов происходил каждый раз, когда тред завершается.

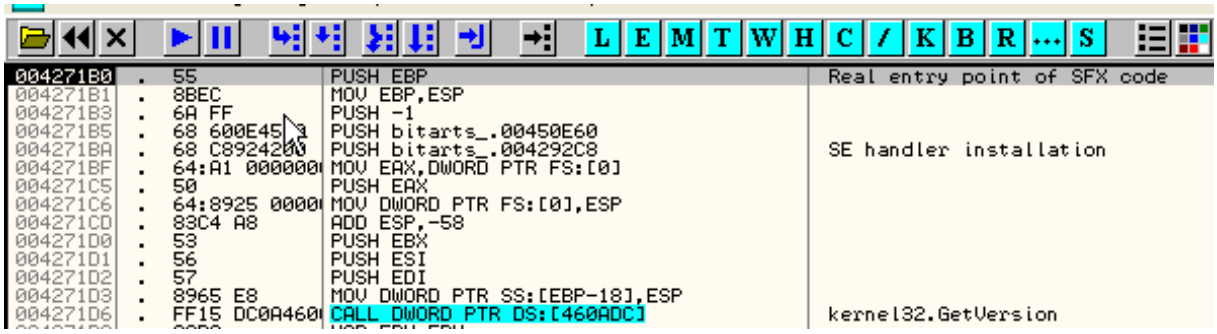


Делаем RUN.



Останавливаемся на завершении треда.

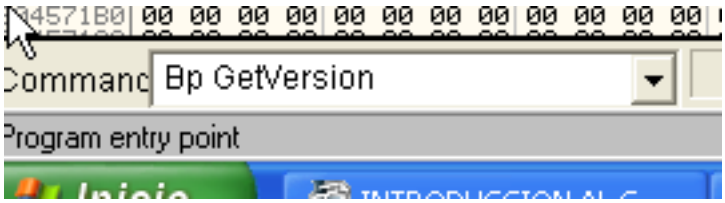




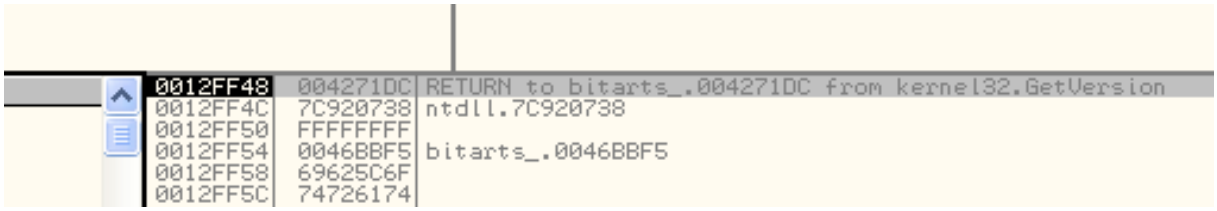
Если установлен BPM ON ACCESS на секцию CODE, то останов также прекрасно произойдёт.

8) Метод первой API-функции, выполняемой программой

Этот метод заключается в установке BP напрямую на API-функцию, относительно которой есть подозрение, что это первая, выполняемая программа. Обычно вначале программы выполняют GetVersion, GetModuleHandleA, ряд распакованных программ получают список наиболее часто используемых API-функций. В случае с bitarts посмотрим GetVersion, а в CrueHead'e – GetModuleHandleA. Устанавливаем в bitarts BP GetVersion.

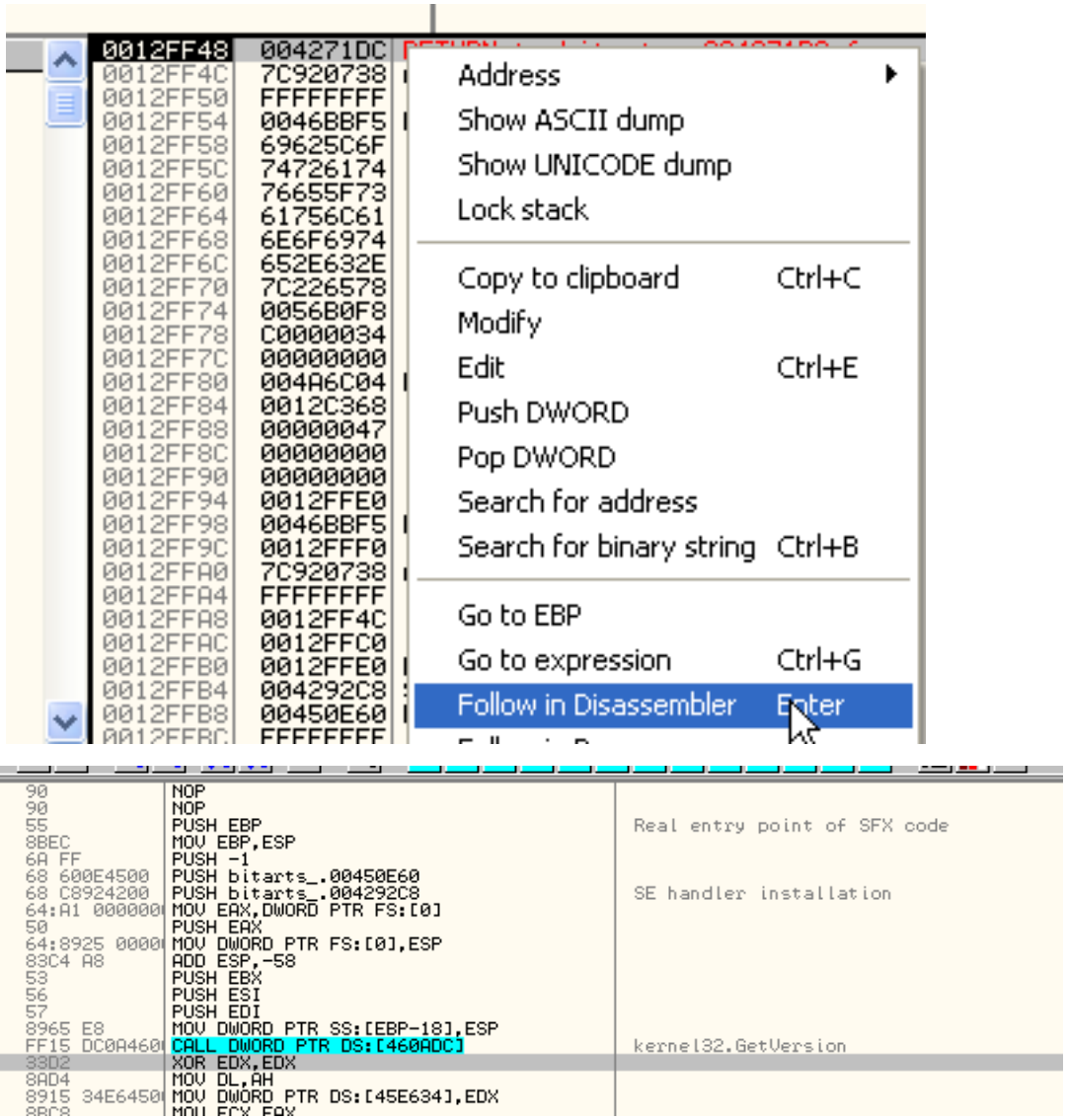


Делаем RUN.



Мы должны быть уверены, что вызов производится из программы, то есть из первой секции.

Когда останавливаемся, смотрим первое значение в стеке (адрес возврата) и идём туда.



Тут видим OEP, полученный нами с помощью метода первой используемой программой API-функции. Если программа обнаружит, что на GetVersion установлен BP, можно попробовать установить его на RET из неё.

Думаю, что методов множество, здесь были приведены примеры наиболее часто используемых. Эти методы можно расширять и адаптировать к конкретному случаю, но главно, чтобы вы поняли общую идею, стоящую за ними, чтобы у вас была прочная база.

К главе приложен крэкми UnPackMe_tElock0.98.exe, на котором можно попрактиковаться в поиске OEP. В нём используются несколько трюков, и не все из вышеперечисленных методов будут работать. Будет прекрасно, если вы сумеете сами найти OEP.

Помните, что если распаковщик обнаруживает BP или HBP и не запускается, то нужно следить, чтобы не было установлено никаких BP или HBP, а когда он запустится, можно попробовать метод PUSHAD. Если он не сработает, то нужно попробовать другой.

Следующая глава будет посвящена дампу и починке IAT'ов.

Введение в крэкинг с нуля, используя OllyDbg - Глава 33

Автор: Рикардо Нарваха, пер. Aquila

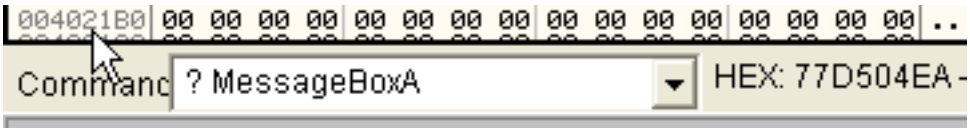
Дата: 2008-09-22

Что такое IAT, и как её починить

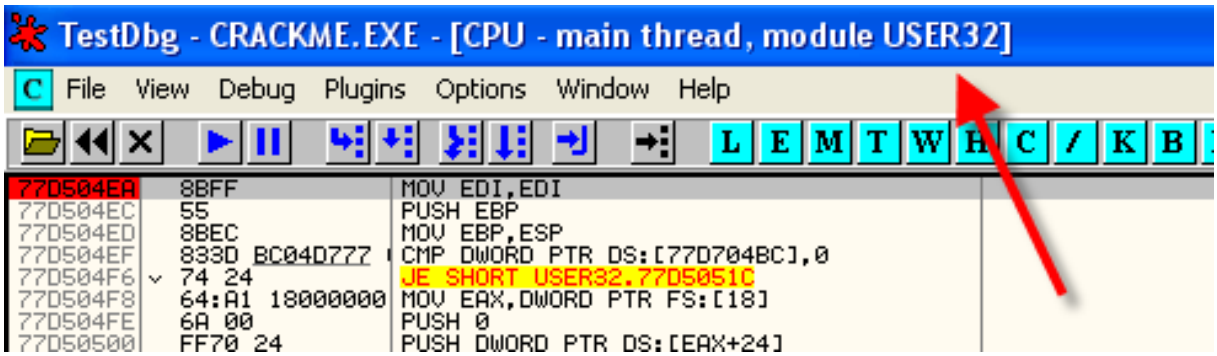
Перед тем, как начинать, как безумные, чинить IAT'ы, необходимо введение в то, что такое вообще IAT. Мы рассмотрим на примере оригинального крэкми CrueHead'a и упакованного варианта, где располагается IAT и что делает с ним упаковщик.

Ок, сначала общая идея, для чего нужен IAT.

Дело в том, что у каждой API-функции есть свой адрес, например, откроем оригинальный крэкми CrueHead'a в OllyDbg и напечатаем:



Видим, что на моей машине адрес этой функции 77D50EA, если вы пойдёте по соответствующему адресу на вашей машине, то у одних это может быть тот же адрес, у других нет, зависит от вашей версии Windows, от установленных обновлений, которых, как мы знаем множество, и каждая новая версия DLL, содержащая API-функцию (в данном случае это User32.dll) обычно меняет адрес.

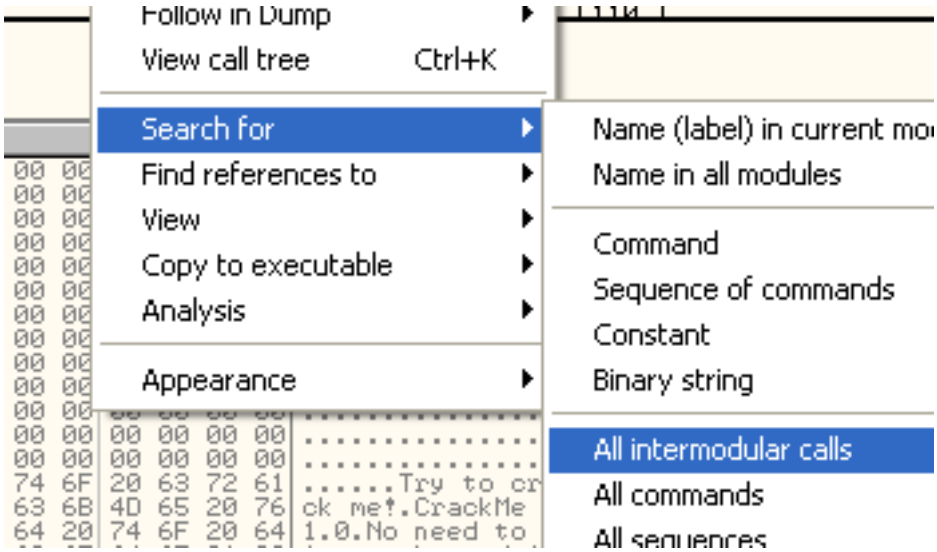


Почти каждый раз, когда Microsoft выпускает новую версию этой DLL, меняются адреса API-функций, входящих в неё, поэтому если я запрограммирую в крэкми CrueHead'a, чтобы при совершении перехода на API-функцию MessageBoxA, это был переход на 77D504EA, то на моей машине это будет прекрасно работать, а также у тех, у кого та же версия User32.dll, но у остальных, у кого не Windows XP или стоит другая версия User32.dll, нужной функции по этому адресу не будет, и произойдёт ошибка.

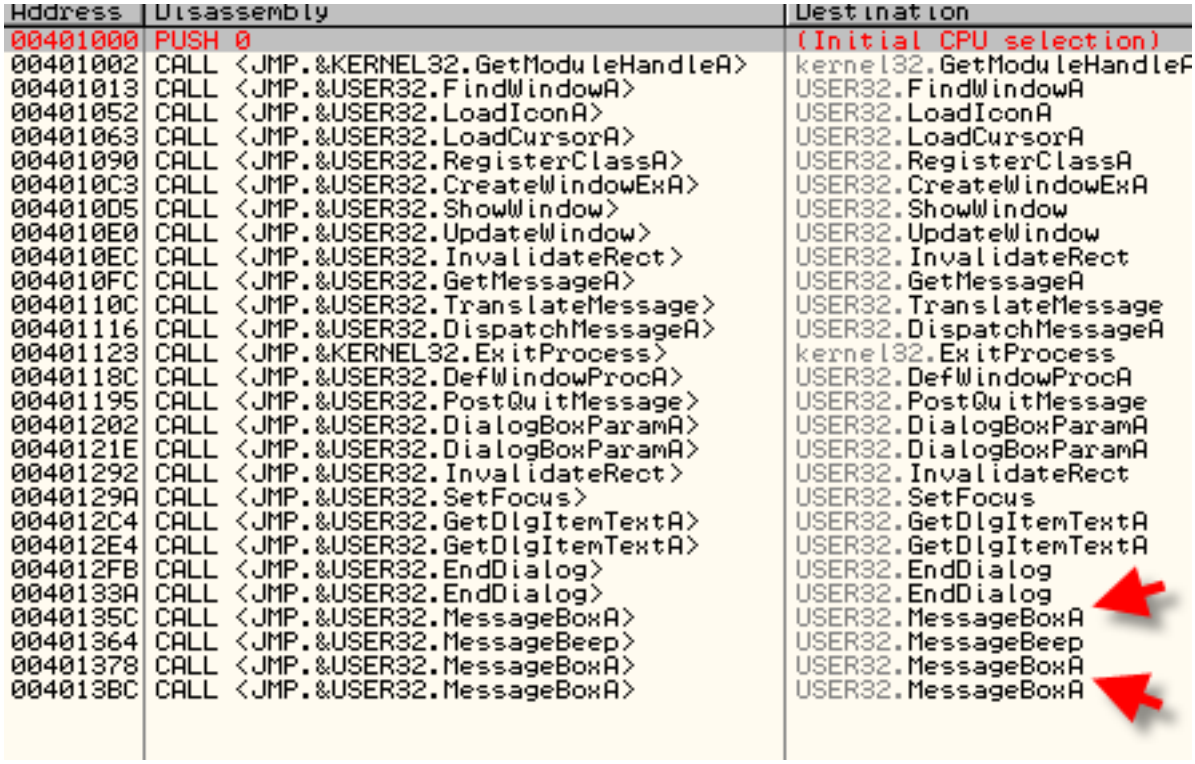
Тогда вопрос заключается в том, что операционная система должна предоставлять какой-то способ, чтобы мой крэкми работал на других Windows и с другими версиями DLL.

Для этого были созданы знаменитые таблицы под названием IT (import table – таблица импортов) и IAT (Import Address Table – таблица импортированных адресов).

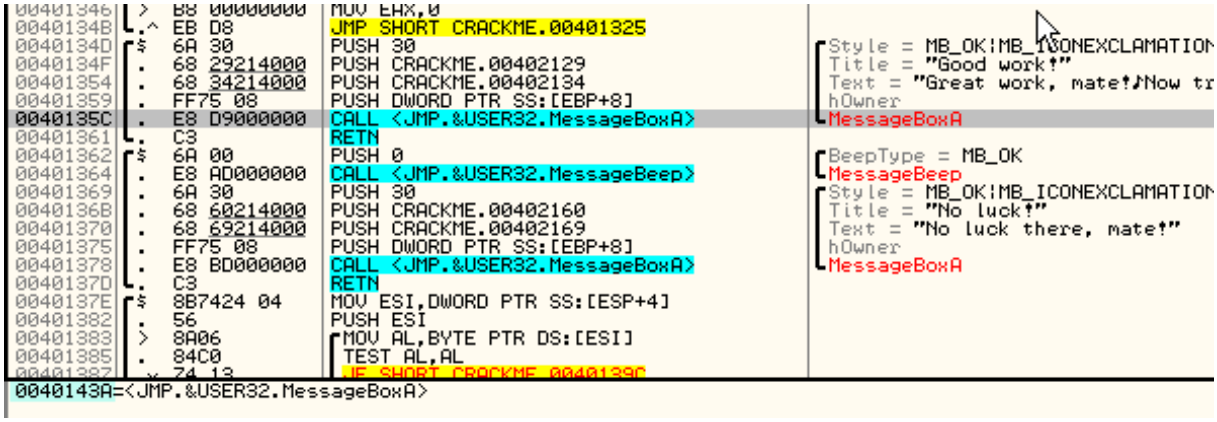
Не надо бояться этих страшных названий – в этой главе мы подробно рассмотрим, где они находятся в распакованной программе и для чего служат.



Теперь делаем SEARCH FOR-ALL INTERMODULAR CALLS и видим вызовы, которые идут в другие модули или DLL, то есть могут быть вызовами API-функций.

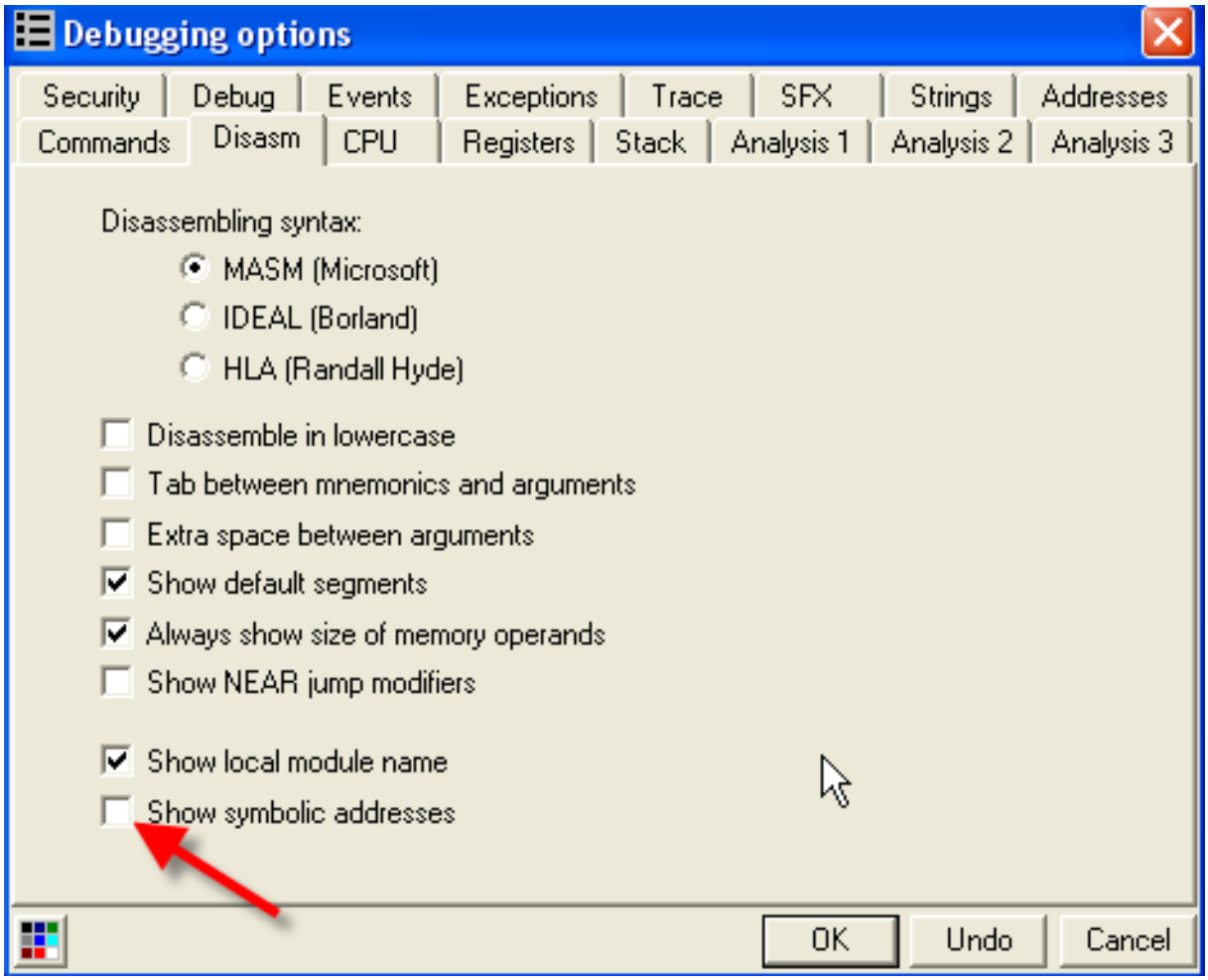


Здесь видим различные вызовы, например функции MessageBoxA, если сделаем двойной щелчок по первой из них:

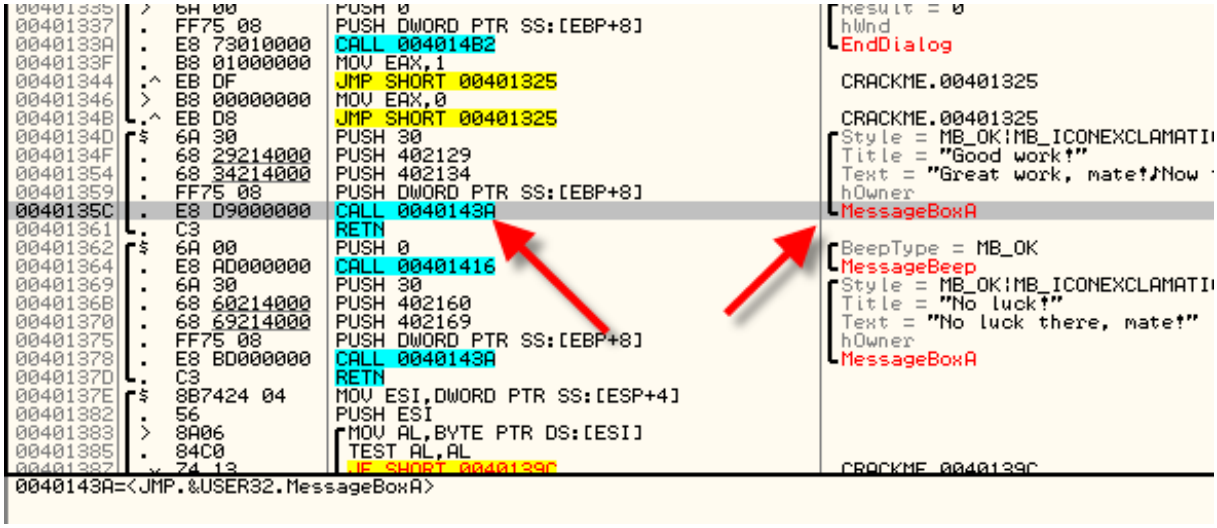


Видим, что на самом деле это вызов CALL 40143A, и OllyDbg нам поясняет, что произойдёт переход на API-функцию MessageBoxA. Она заключает MessageBoxA в угловые скобки, чтобы показать, что вызов прямой, а не косвенный.

CALL 40143A



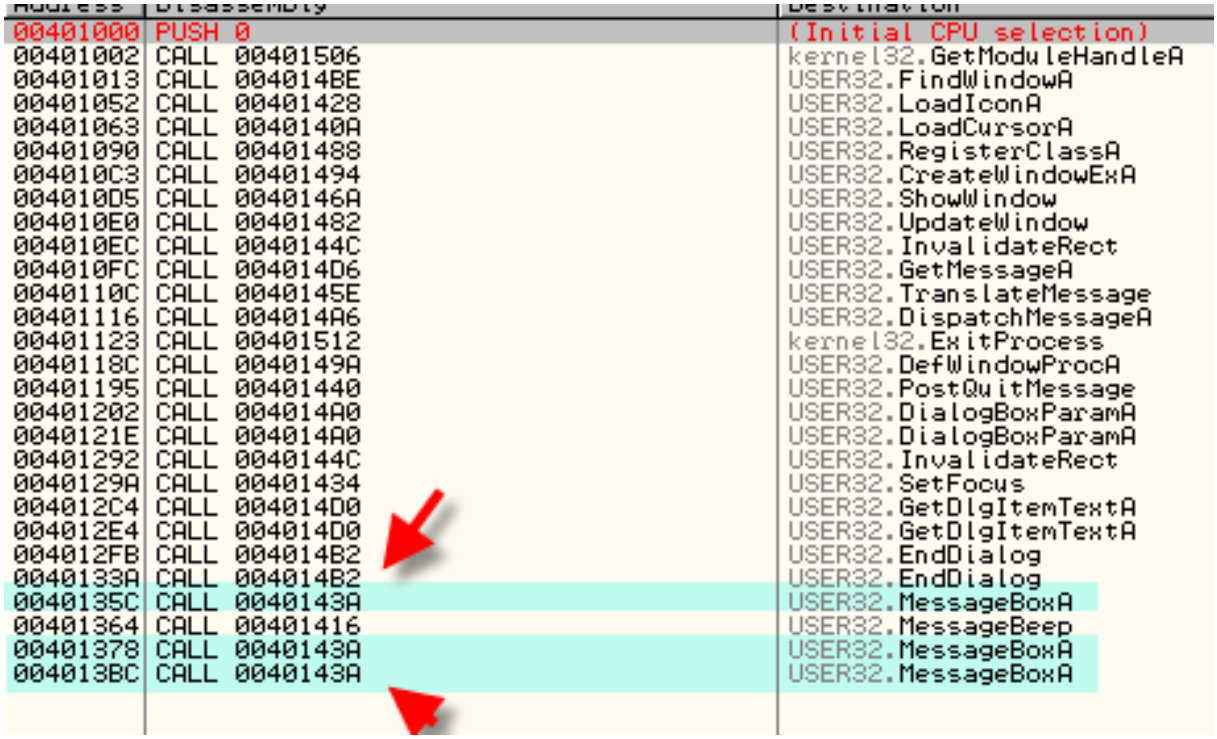
Видим, что среди опций OllyDbg во вкладке DISASM есть SHOW SIMBOLIC ADDRESSES. Если уберём её, то всё становится более ясным.



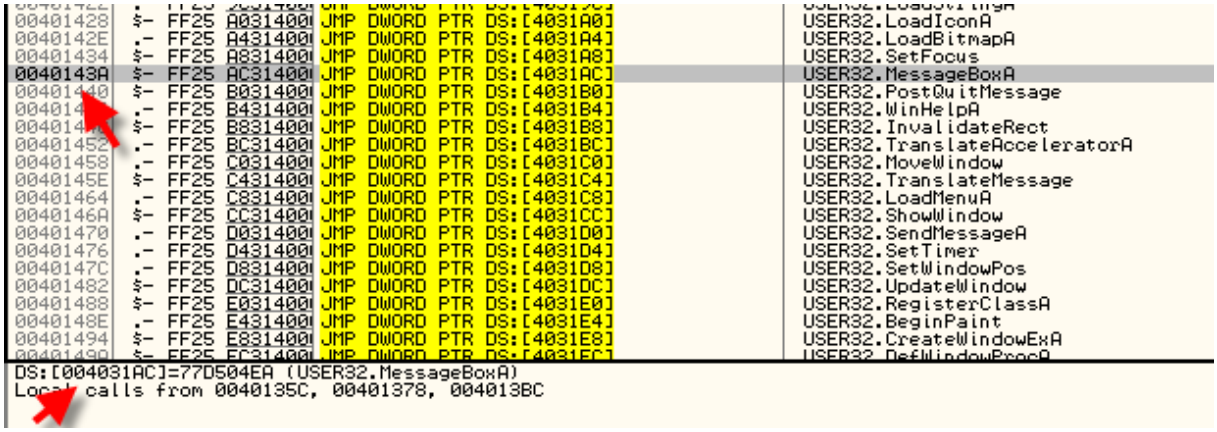
Видим, что справа также остаются пояснения с параметрами и названием API-функции, куда будет совершён переход, но теперь CALL действительно похож на прямой переход.

CALL 40143A

B SEARCH FOR INTERMODULARS CALLS

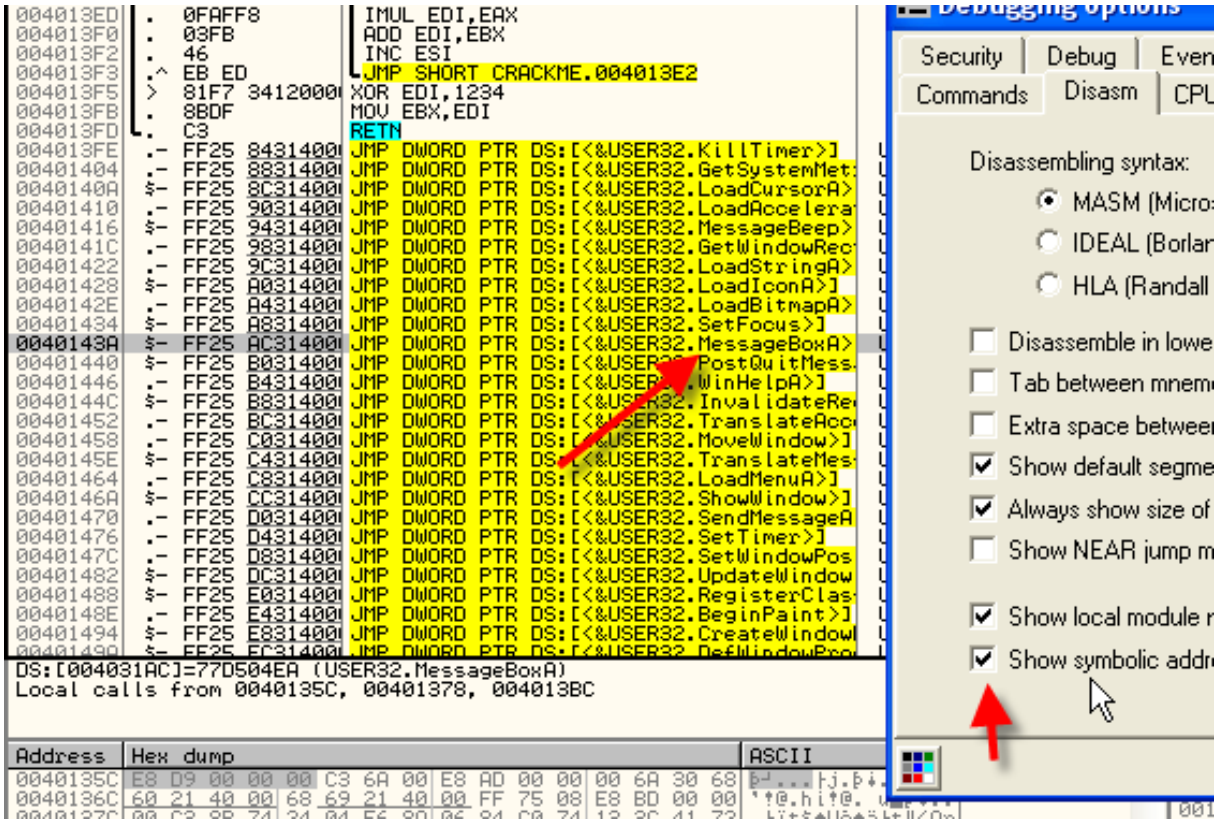


Видим, что в реальности три вызова MessageBoxA – это вызовы 40143A, посмотрим, что там находится.



Здесь видим, что вся загвоздка тут, чтобы добраться до API-функции, делает косвенный JMP, который таким образом получает значение, куда надо на самом деле перейти, из 4031AC.

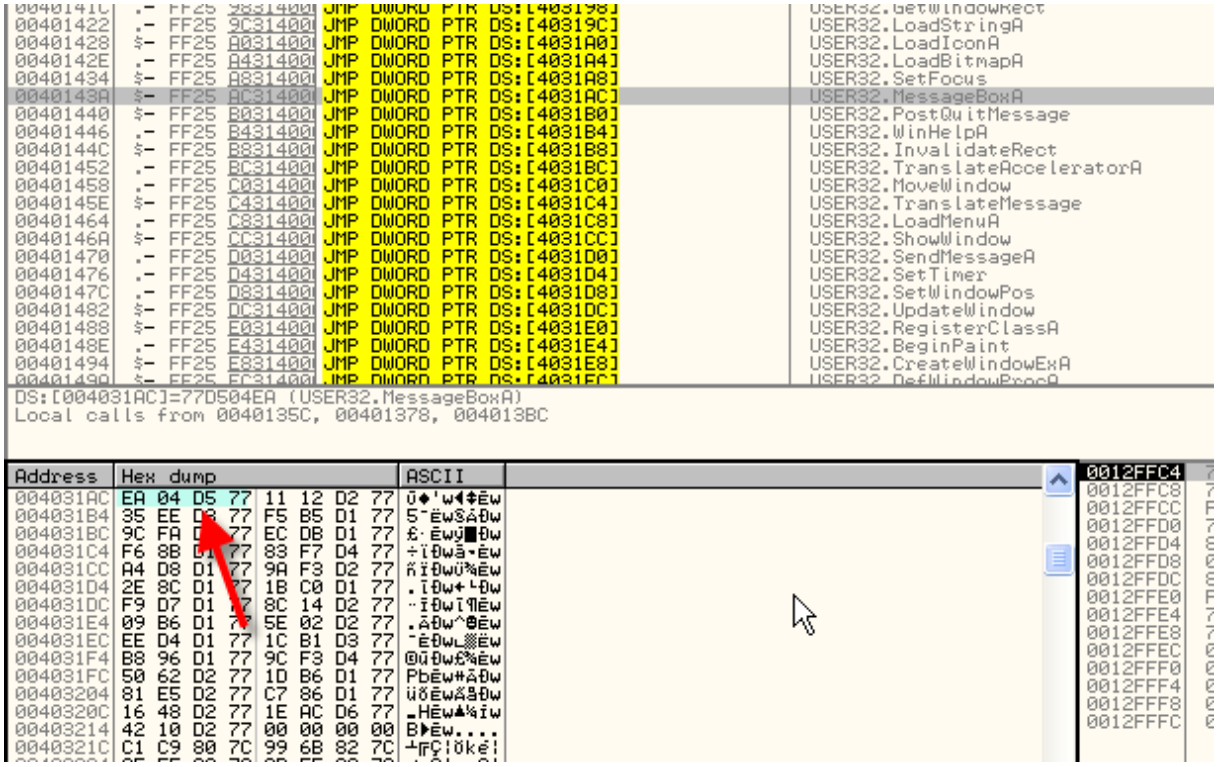
JMP [4031AC]



Также видим, что если включить эту опцию, то понять, что это косвенный переход сложнее, поэтому её лучше выключать, когда работаем с IAT.

И вот этот трюк: программа переходит на API-функцию с помощью косвенного JMP, который получает адрес API-функции из 4031AC, и видим, что для остальных функций есть похожие косвенные переходы.

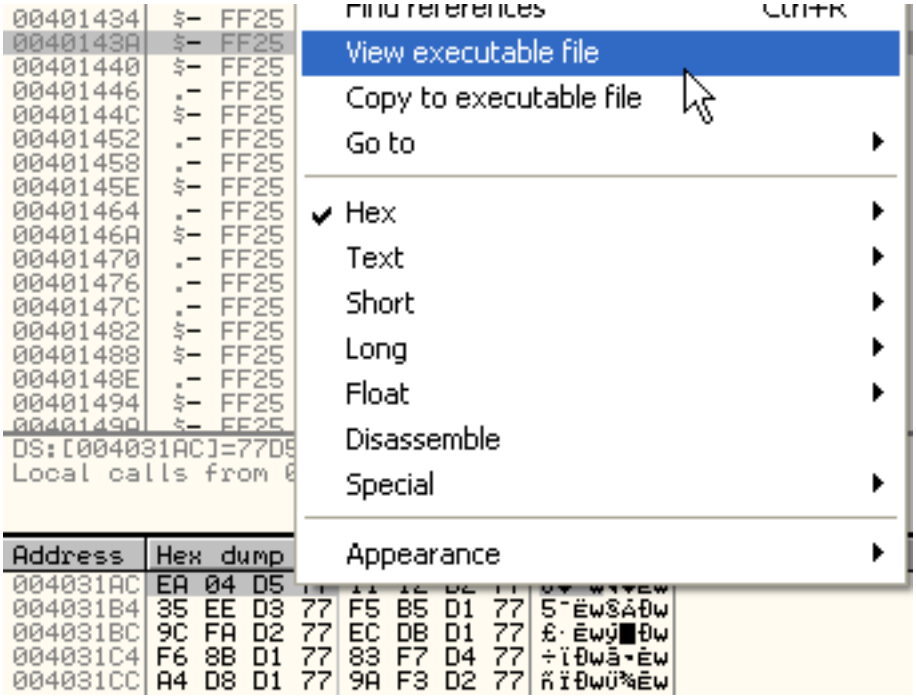
Кажется, вопрос о переносимости между разными компьютерами начинает проясняться. Когда программе нужно перейти на какую-либо API-функцию, делается косвенный переход, считывается адрес, по которому та располагается, из некоего рода хранилища. Посмотрим через DUMP, что это за хранилище.



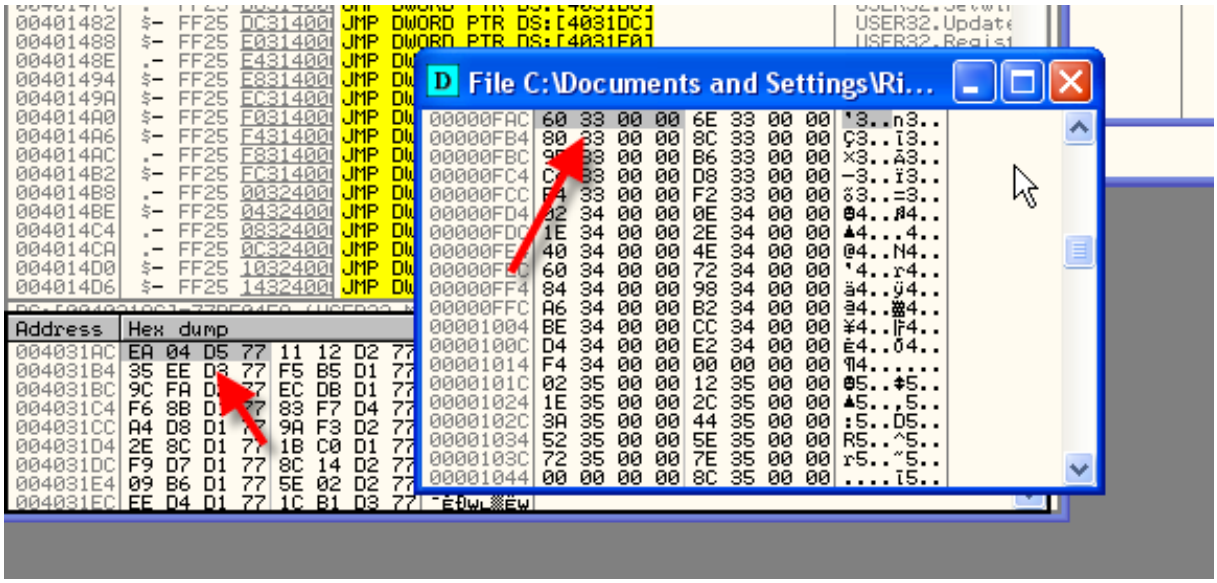
Здесь, и правда, видим, что 4031AC – это часть хранилища, которое содержит все адреса API-функций, по которым располагаются последние на моей машине. Это хранилище и есть знаменитая IAT или IMPORT ADRESS TABLE, которая является ключом к переносимости, и это означает, что все программы имеют одни и те же косвенные переходы на API-функции, а меняются только значения, находящиеся в хранилище. На разных машинах эти значения могут отличаться.

Но постой, скажет мне кто-то, 4031AC – это адрес в программе, поэтому если на каждой машине такие адреса могут отличаться, то и он может отличаться в разных случаях?

Хе-хе, хороший вопрос (маэстро сам себе плохих вопросов не задаст □ - прим. пер), сейчас мы рассмотрим, как вся эта система работает. Главная её цель заключается в том, чтобы наполнить IAT правильными адресами.



Мы знаем, что если сделать VIEW-EJECUTABLE FILE, то можно увидеть, что находится по адресу 4031AC в самом исполняемом файле, находящемся на нашем жёстком диске.



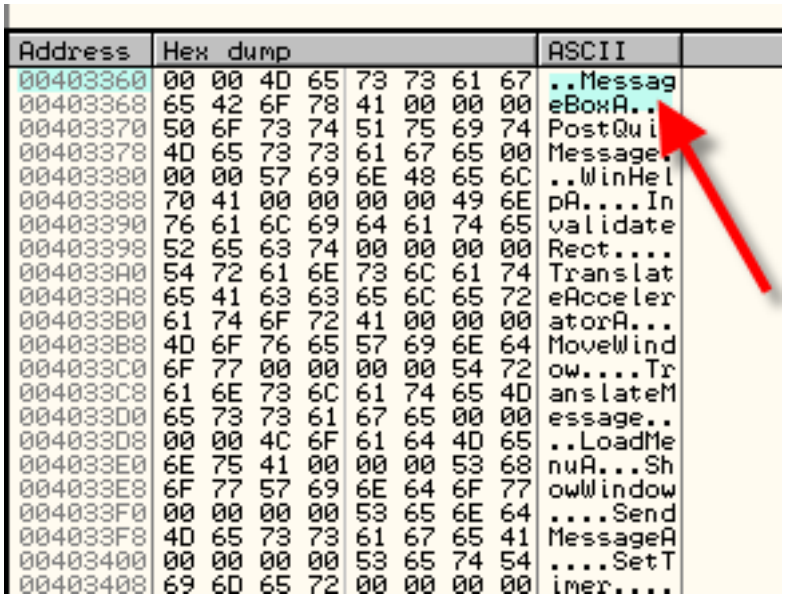
Видим, что в exe-файле по этому адресу находится значение 60 33, а когда мы находимся на точке входа, то в памяти в этом месте находится адрес API-функции EA 04 D5 77, и это означает, что операционная система, загружая программу, берёт значение 33600, находящееся по СМЕЩЕНИЮ 0FAC, которое соответствует адресу 4031AC, и замещает его реальным адресом API-функции на моей машине.

Магия?

Нет-нет, так работает Windows, для каждого запускающегося исполняемого файла операционная система заполняет IAT правильными для моей машины адресами API-функций, в данном случае в 4031AC помещается адрес API-функции MessageBoxA, а соседние элементы IAT заполняются адресами других функций, пока вся IAT не будет заполнена.

Конечно, система не настолько магическая, как можно подумать, видим, что в 0FAC находится значение 3360, которое указывает на что-то, может быть на то, какую API-функцию надо вызывать?

Если прибавим 3360 к базе образа, то получим 403360, и что мы найдём по этому адресу?



Вот в чём дело, это указатель на строку MessageBoxA, то есть система смотрит на этот указатель, ищет нужную функцию по имени и, с помощью GetProcAddress, получает её адрес на нашей машине, а затем заносит его в IAT, перезаписывая им 3360. Таким образом, гарантируется, что программа будет работать с любой версией DLL, так как адрес будет найден ещё до того, как выполнение перейдёт к точке входа в программу, а как только оно там окажется, IAT уже будет заполнен адресами API-функций. Если посмотрим IAT на другой машине, то увидим, что содержимое 4031AC отличается, так как адрес функции другой.

JMP [4031AC]

Переход на MessageBoxA, и программисту не нужно думать о тонкостях, так как значение в IAT всегда будет правильным, будучи заполненным правильными адресами при старте программы.

То есть, чтобы система корректно заполнила IAT адресами API-функций, в файле должны находится следующие указатели:

1)В элементе IAT в самом exe-файле должен находится указатель на строку с текстом, который говорит системе, адрес какой API-функции нужно поместить в этот элемент.

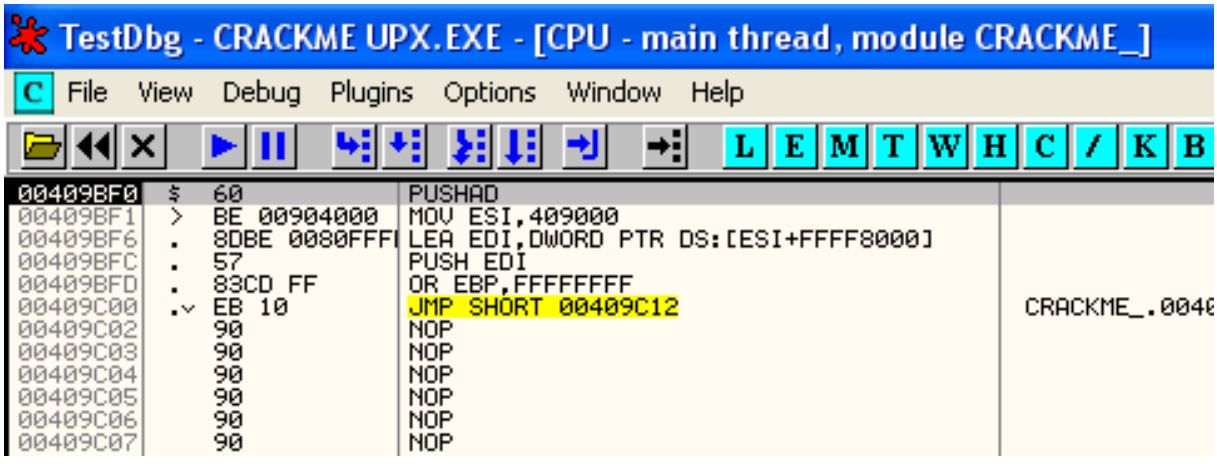
2)Конечно, строка с текстом должна быть именем API-функции.

Если эти два пункта выполнены, то можем заключить, что программа запустится, и система заполнит IAT правильными значениями (позже рассмотрим полностью, как всё это работает).

Ок, мы оказываемся на OEP распакованной программы, вопрос заключается в том, когда мы туда добрались и делаем дамп программы, зачем нам нужно реконструировать IAT, упаковщики её, что, ломают? Да, одни больше, другие меньше, смысл в том, что упаковщику не нужен IAT программы, потому что загрузка начинается с процедуры распаковки, которая может прочитать какие API-функции нужны в IAT, узнать их адреса и заполнить ими IAT, строки же, которые задают имена функций, стираются, так как они программе не нужны – когда она будет загружена, IAT уже содержит правильные адреса.

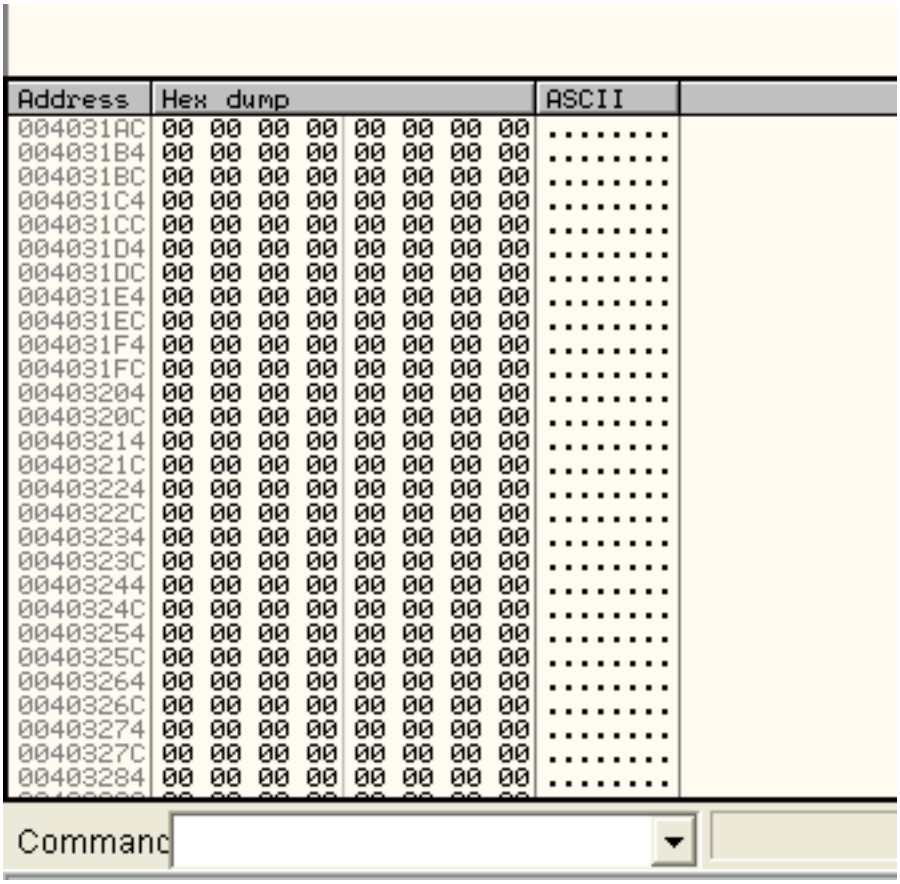
Большинство упаковщиков хранят эти строки в зашифрованном виде или в таком месте, где найти их нам, крэкерам, не так уж просто.

Посмотрим на примере крэкми, запакованного UPX'ом, отличие от нормального исполняемого файла.

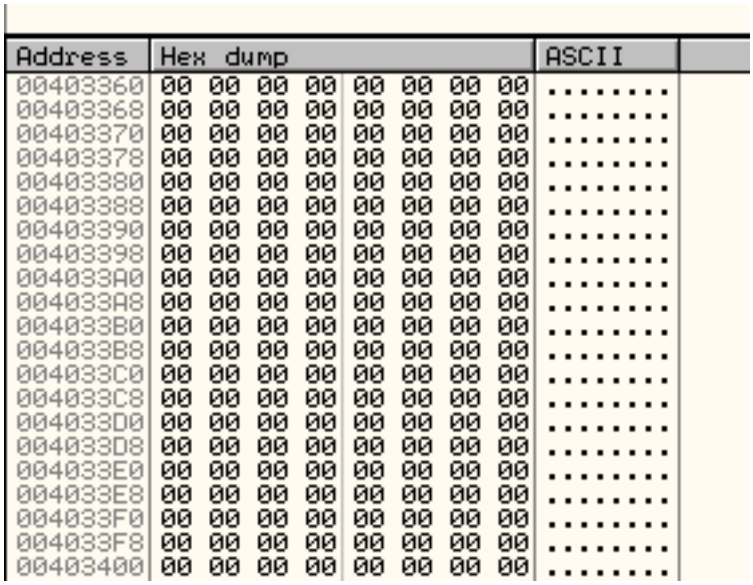


Вот у нас крэкми CrueHead'a, упакованный UPX'ом, который мы уже использовали в прошлых главах.

Смотрим, что находится по адресу 4031AC (в оригинальном крэкми там находится хранилище).



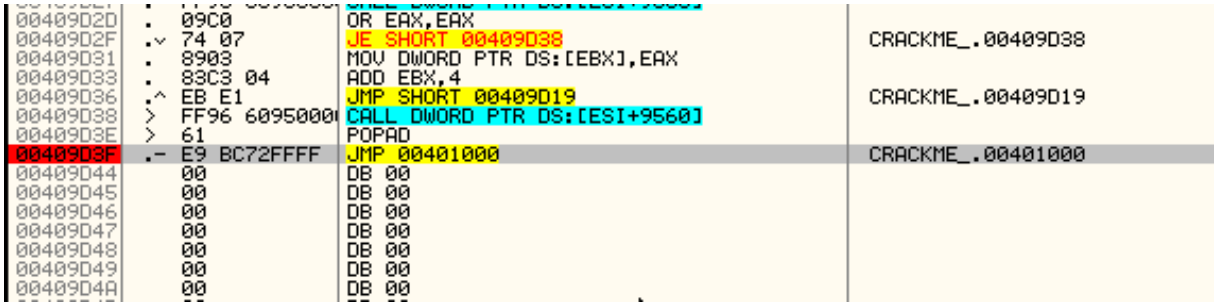
Ничего, всё стёрто, а строки с именами API-функций, располагавшиеся в 403360, где они?



Ничего, совсем ничего, доходим до OEP и смотрим, что находится по тем же адресам, так как чтобы программа работала, в IAT должно быть что-то, чтобы был совершён переход на API-функцию.

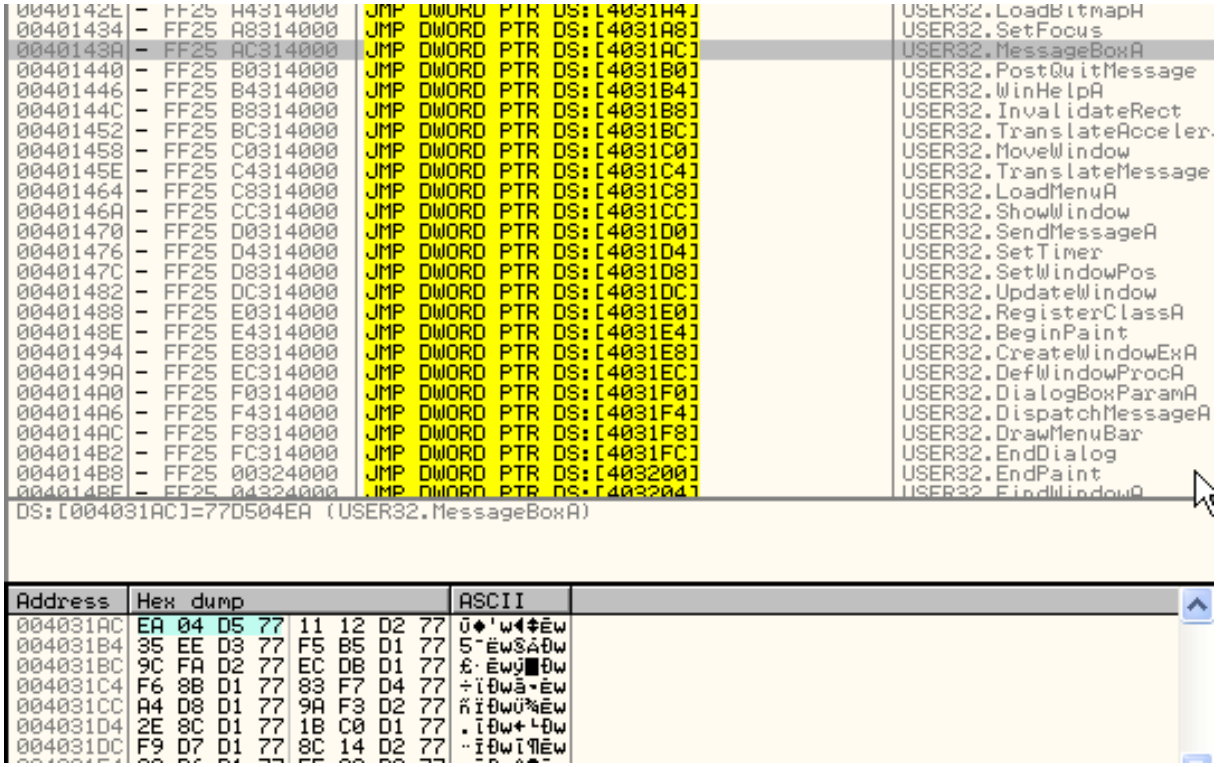
JMP [4031ac]

Если IAT пуста, то это вызовет ошибку, то есть упаковщик должен сделать работу, которую обычно делает операционная система. Идём в OEP.



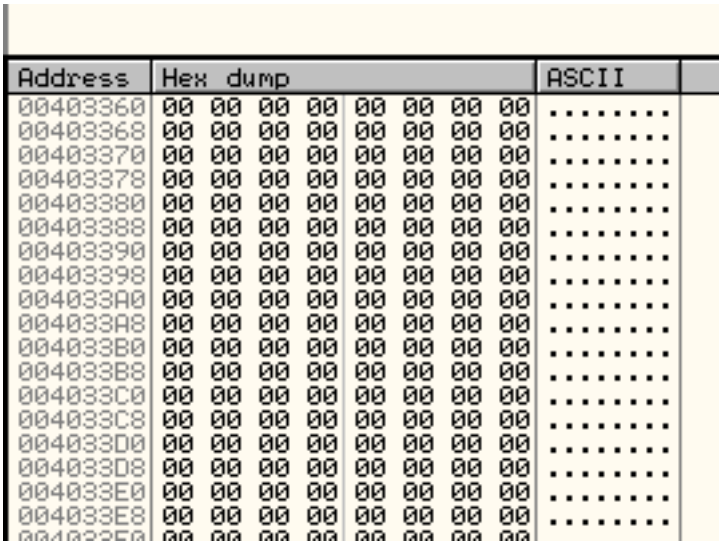
Устанавливаем BP на переход на OEP, делаем RUN и доходим досюда.

Смотрим, что находится в том месте, где располагается IAT.



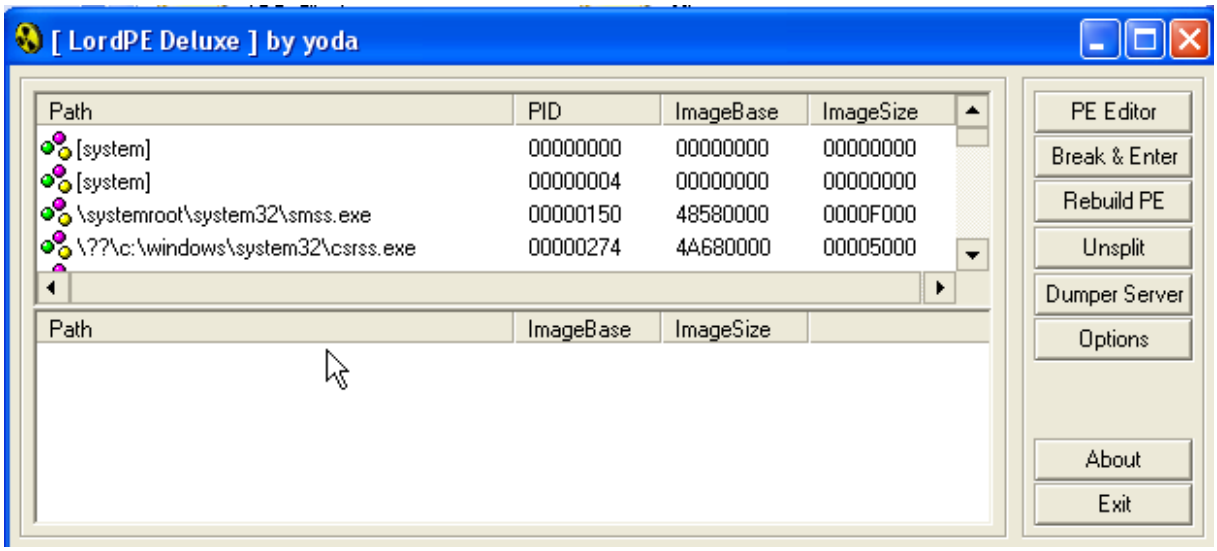
Видим, что для обеспечения нормальной работы программы распаковщик заполняет IAT правильными адресами API-функций на моей машине. Если мы сдампим программу в этот момент, то возникнет проблема, так как у получившейся программы не будет достаточно данных, чтобы запуститься.

Строки, которые идентифицировали каждую из API-функций и находились в 403360, сейчас там отсутствуют.

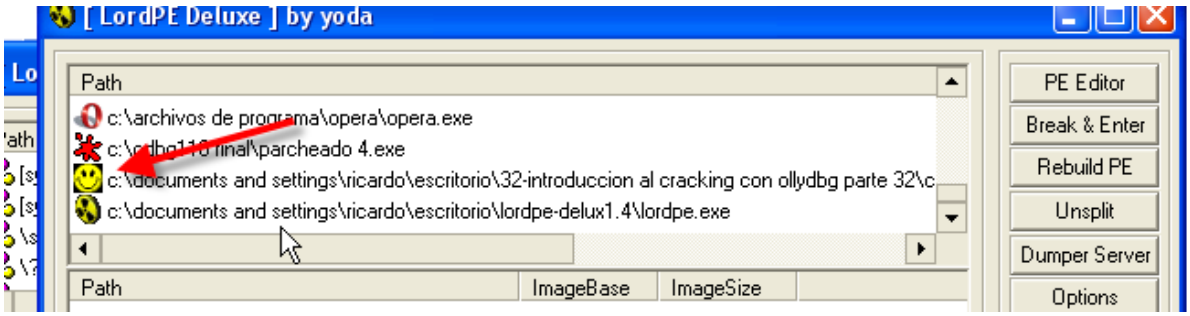


Сделаем дамп, чтобы посмотреть, что останется, но как мы уже отметили, хотя код программы будет правильный, но запуститься сдампленная программа не запустится, так как система не сможет заполнить IAT из-за отсутствия данных.

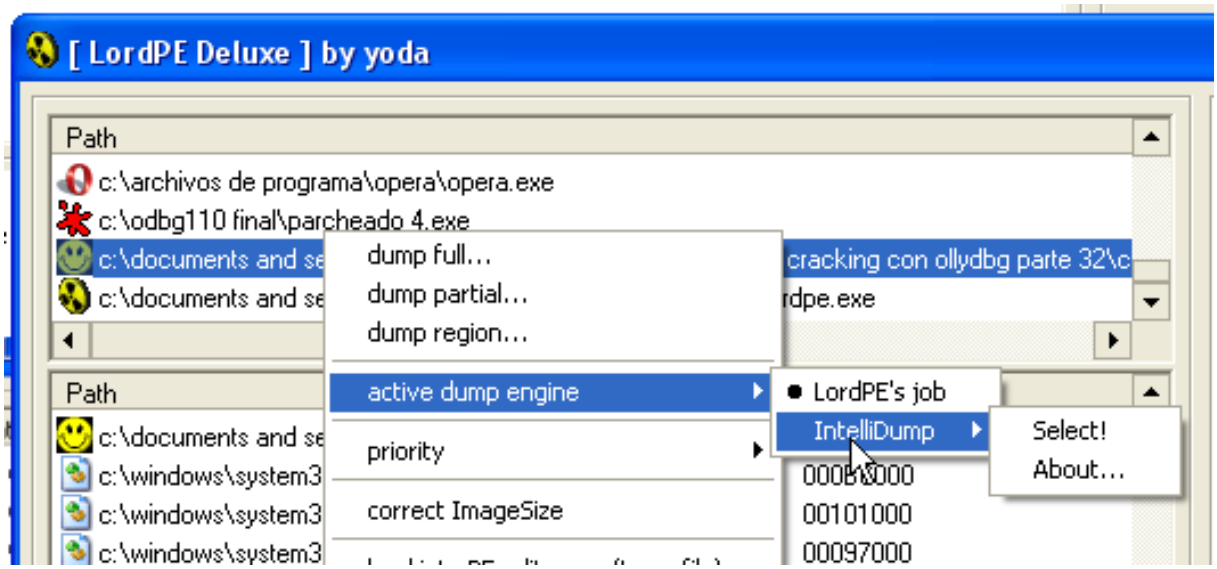
Для того, чтобы сделать дамп, будем использовать внешнюю программу, которая называется LORDPE DELUXE. Её можно скачать отсюда.



Запускаем LORD PE DELUXE, ищем процесс, который нужно сдампить, в нашем случае это crackmeUPX.

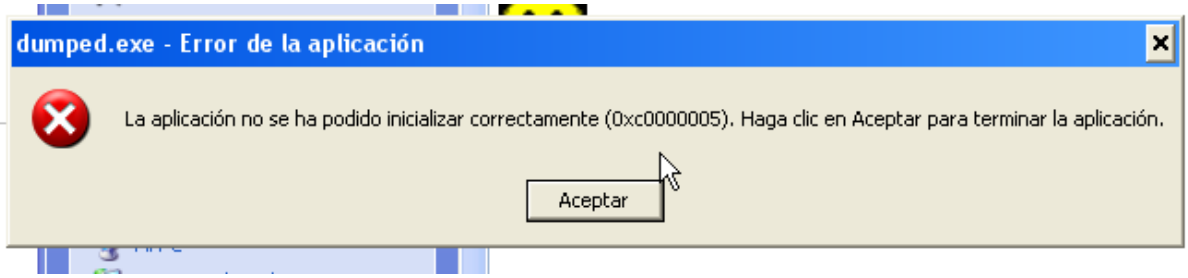


Находим, отмечаем.



Нажимаем правую кнопку мыши, выбираем INTELLIDUMP, а затем – DUMP FULL.

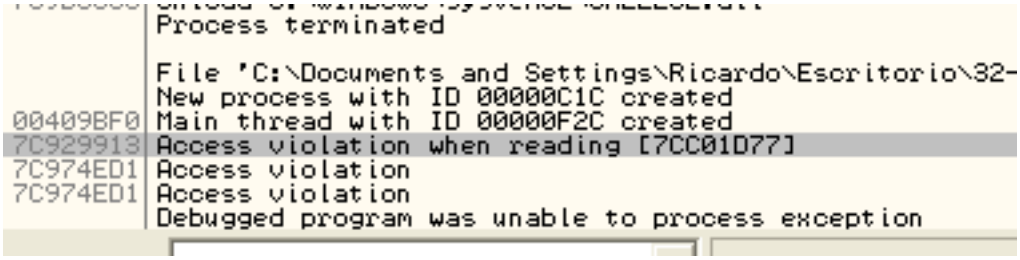
Дамп сохраняется под именем DUMPED.exe.



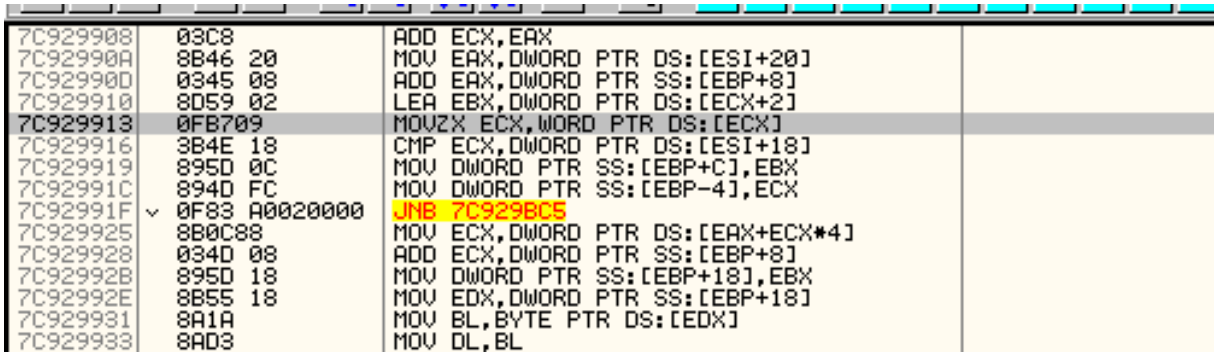
Si corremos el dumpeado vemos que no arranca tratemos de abrirlo en OLLYDBG

Если попробуем запустить его, то увидим, что происходит ошибка. Открываем дамп в OllyDbg.

Посмотрим, какие ошибки показываются в LOG'e OllyDbg.

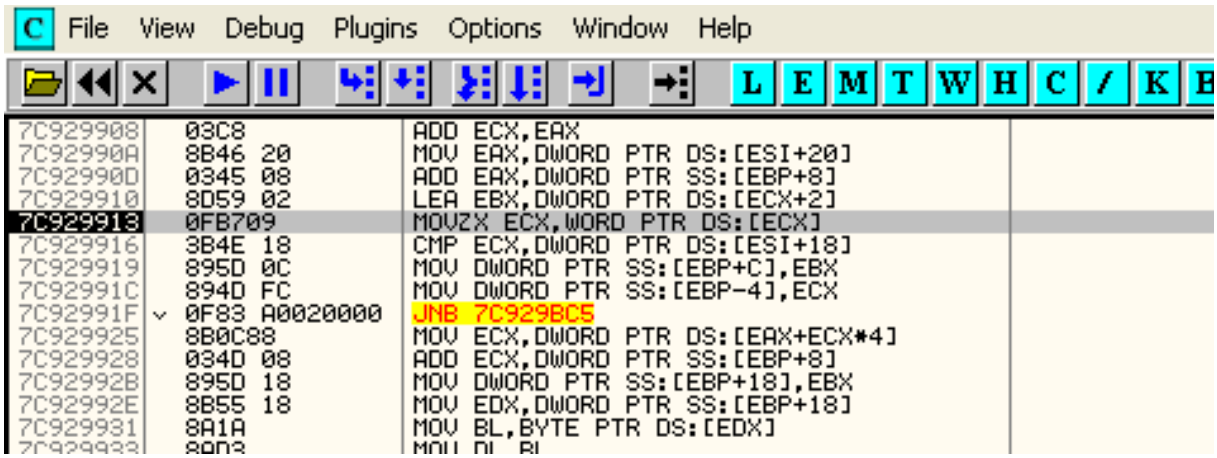


Ошибка происходит в 7c929913 на моей машине. Идём по этому адресу (у вас он может быть другим, поэтому надо смотреть в LOG'e).



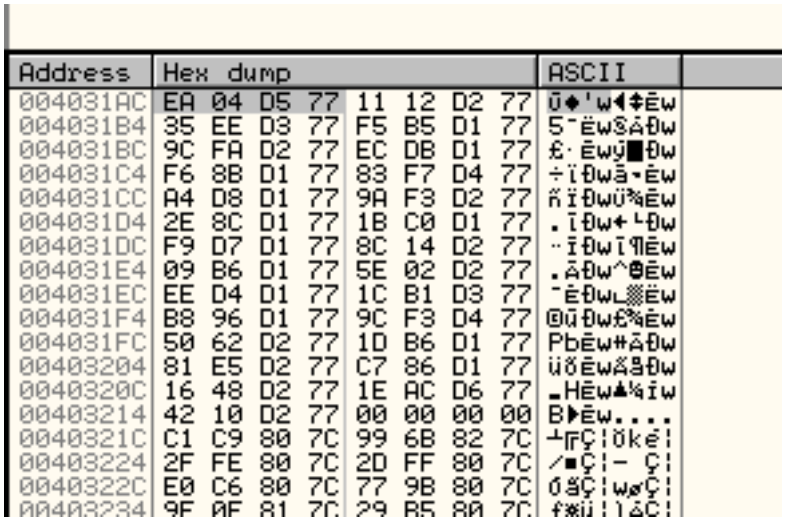
Устанавливаем здесь HARDWARE BREAKPINT ON EXECUTION и BP, что увидеть, что происходит до возникновения ошибки.

Ок, видим, что останова не произошло, но зато случается другая ошибка из-за исключения. Рестартуем:



Видим, что останавливаемся до прибытия в точку входа из за ошибки.

Раз уж мы оказались здесь, смотрим, что находится в IAT.



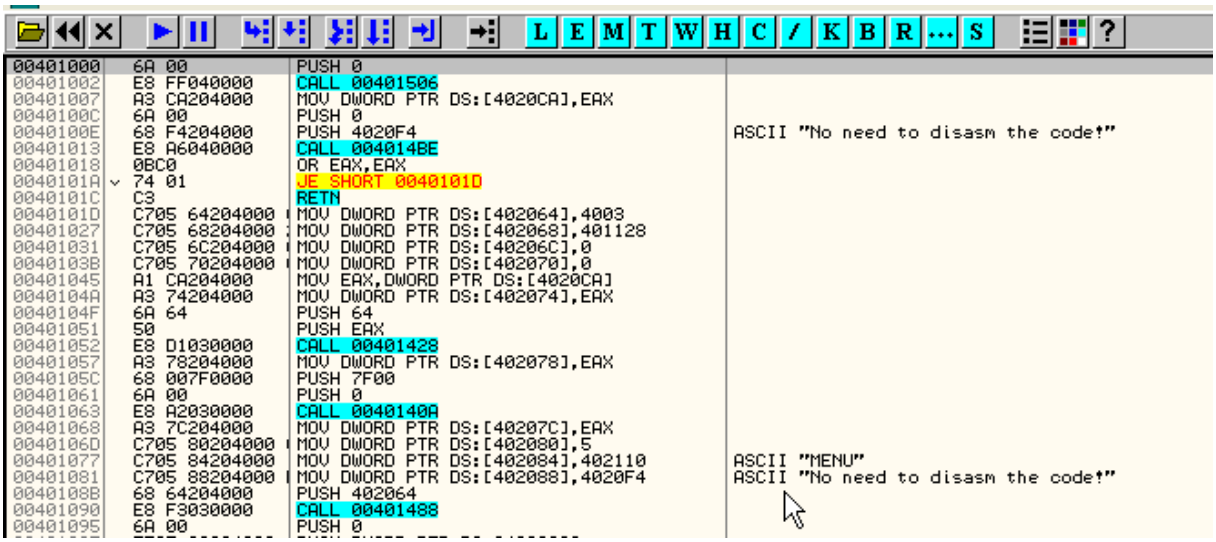
Видим, что дампер сохранил значения, которые были в IAT на момент прибытия в OEP, и они являются правильными на моей мошине, но для того, чтобы exe-файл запускался на любой машине, там должен быть сохранён указатель на строки с именами API-функций, а так как система не может их найти, то и происходит ошибка.

Не нужно быть гением, чтобы понять, что для того, чтобы программа запустилась, нужно восстановить имена API-функций, и указатели на них. Если делать это вручную, то это адская

прорва работы: нужно было бы заменить содержимое 4031AC на указатель на строку MessageBoxA, и так со всеми.

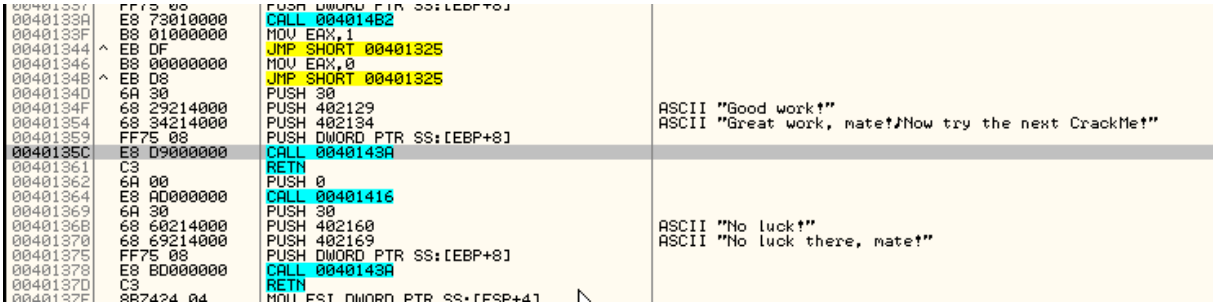
Итак, нам брошен вызов: кроме того, что нам нужно было получить правильный дамп, нам ещё нужно починить его, чтобы он запускался.

Так как у нас правильный код, то без всяких сомнений идём в 401000, и смотрим, что там находится.

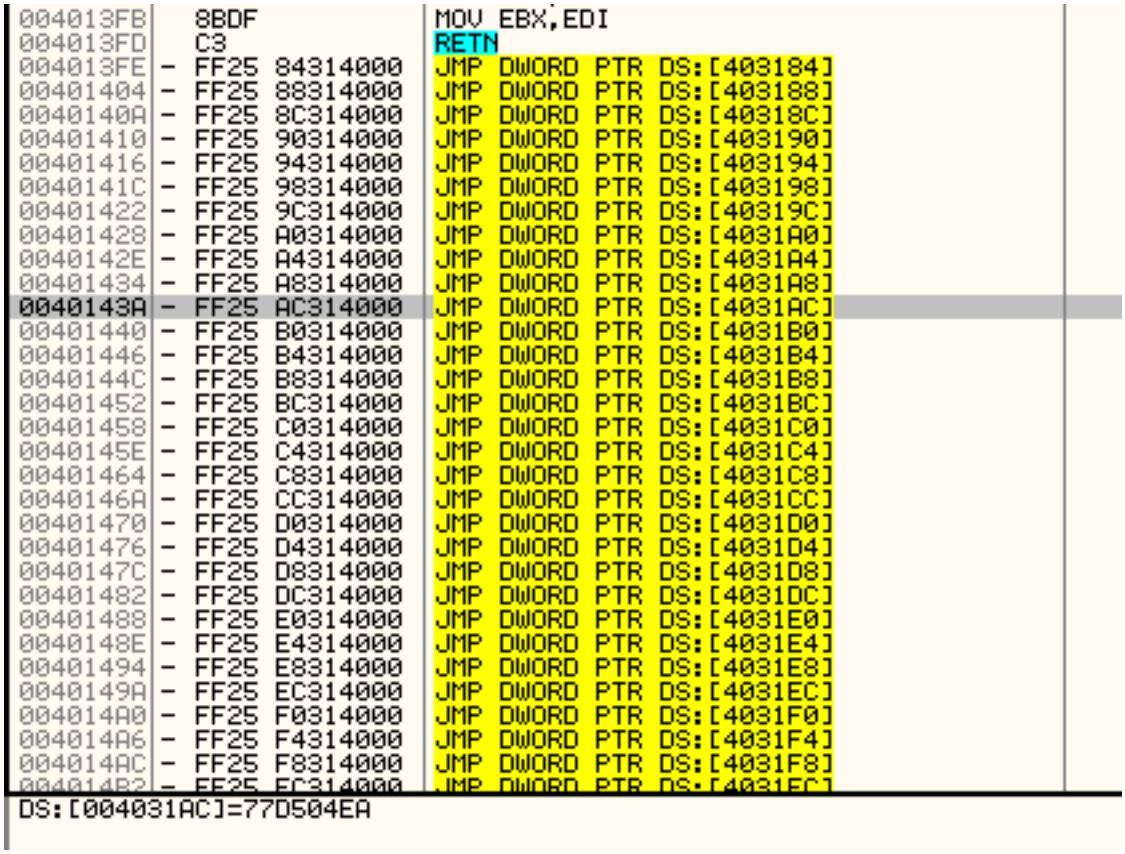


Если переместимся в зону переходов на API-функции:

Здесь находится вызов MessageBoxA, уже виденный нами ранее.



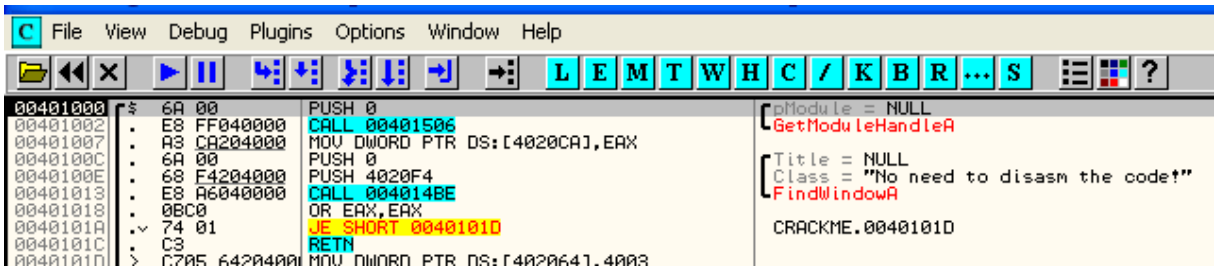
В вызове стоит правильный адрес, переходим на тот же JUMP, что и раньше.



JUMP'ы являются тем, что может не работать, так как в нормальных условиях система должна заполнить IAT правильными значениями, основываясь на указателях на строки с названиями функций, которые должны быть здесь до старта программы.

Прежде, чем заняться данной областью, посмотрим правильные определения и их расположение в исходной программе с неупаковонными IAT и IT.

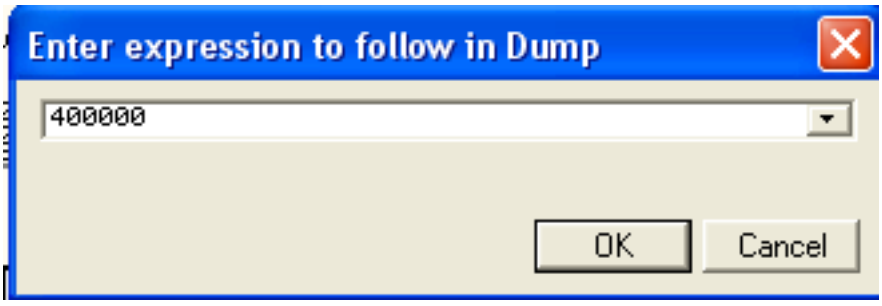
Откроем оригинальный CC в OllyDbg.



Ок, находим нужные нам части, которые должны нормально работать в исходном файле.

В заголовке находим несколько важных указателей.

Идём туда же.



CRACKME.<ModuleEntryPoint>

Address	Hex dump	ASCII
00400000	4D 5A 50 00 02 00 00 00	MZP.☻...
00400008	04 00 0F 00 FF FF 00 00	♦.☼. ..
00400010	B8 00 00 00 00 00 00 00	◘.......
00400018	40 00 1A 00 00 00 00 00	@.→.....
00400020	00 00 00 00 00 00 00 00	....
00400028	00 00 00 00 00 00 00 00	....
00400030	00 00 00 00 00 00 00 00	....
00400038	00 00 00 00 00 01 00 00	.....☺..
00400040	BA 10 00 0E 1F B4 09 CD	‖►.♬▼┤.=
00400048	21 B8 01 4C CD 21 90 90	!◘☺L=!ÉÉ
00400050	54 68 69 73 20 70 72 6F	This pro
00400058	67 72 61 6D 20 6D 75 73	gram mus
00400060	74 20 62 65 20 72 75 6E	t be run
00400068	20 75 6E 64 65 72 20 57	under W
00400070	69 6E 33 32 0D 0A 24 37	in32..$7
00400078	00 00 00 00 00 00 00 00	....
00400080	00 00 00 00 00 00 00 00	....
00400088	00 00 00 00 00 00 00 00	....
00400090	00 00 00 00 00 00 00 00	

Меняем режим SPECIAL-PE HEADER.

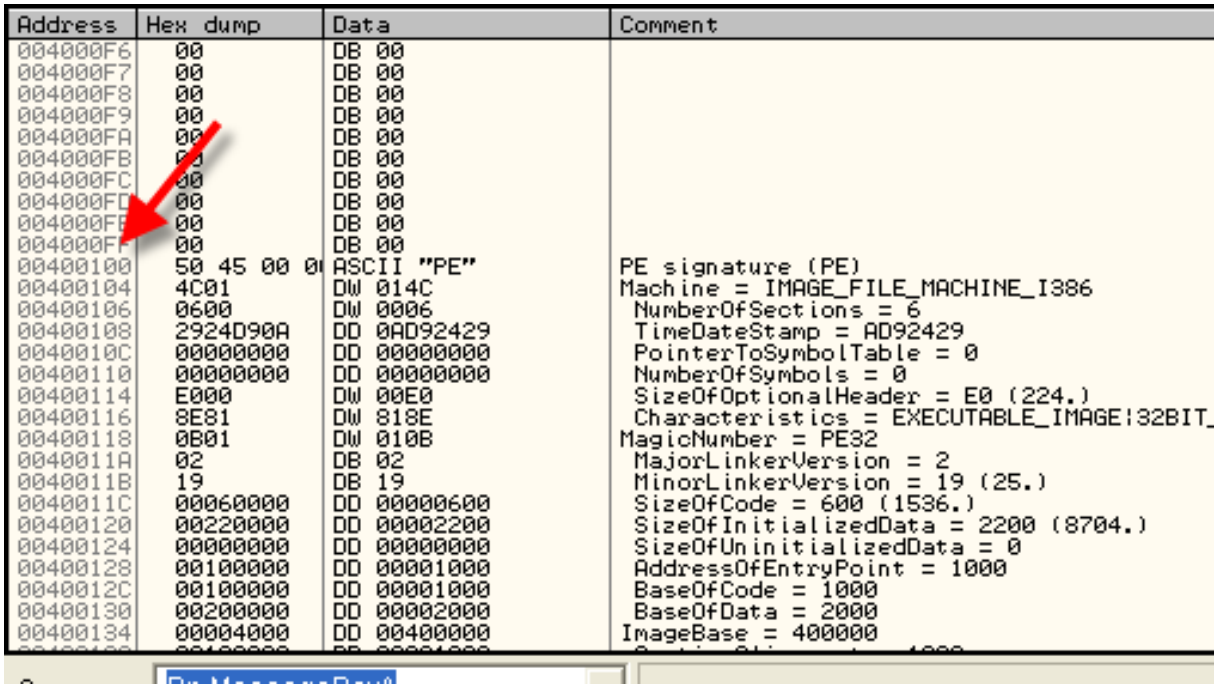
CRACKME.<ModuleEntryPoint>

Address	Hex dump	Data	Comment
00400000	4D 5A	ASCII "MZ"	DOS EXE Signature
00400002	5000	DW 0050	DOS_PartPag = 50 (80.)
00400004	0200	DW 0002	DOS_PageCnt = 2
00400006	0000	DW 0000	DOS_ReloCnt = 0
00400008	0400	DW 0004	DOS_HdrSize = 4
0040000A	0F00	DW 000F	DOS_MinMem = F (15.)
0040000C	FFFF	DW FFFF	DOS_MaxMem = FFFF (65535.)
0040000E	0000	DW 0000	DOS_ReloSS = 0
00400010	B800	DW 00B8	DOS_ExeSP = B8
00400012	0000	DW 0000	DOS_ChkSum = 0
00400014	0000	DW 0000	DOS_ExeIP = 0
00400016	0000	DW 0000	DOS_ReloCS = 0
00400018	4000	DW 0040	DOS_TablOff = 40
0040001A	1A00	DW 001A	DOS_Overlay = 1A
0040001C	00	DB 00	
0040001D	00	DB 00	
0040001E	00	DB 00	
0040001F	00	DB 00	

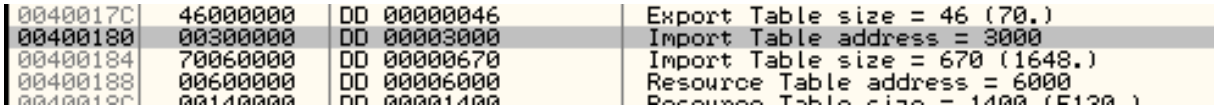
Спускаемся.

Address	Hex dump	Data	Comment
00400035	00	DB 00	
00400036	00	DB 00	
00400037	00	DB 00	
00400038	00	DB 00	
00400039	00	DB 00	
0040003A	00	DB 00	
0040003B	00	DB 00	
0040003C	00010000	DD 00000100	Offset to PE signature
00400040	BA	DB BA	
00400041	10	DB 10	
00400042	00	DB 00	
00400043	0E	DB 0E	
00400044	1F	DB 1F	
00400045	B4	DB B4	
00400046	09	DB 09	
00400047	CD	DB CD	

Реальные данные идут с 100, где начинается PE SIGNATURE.



«Начинается со 100» - это значит, что начинается в 400100.

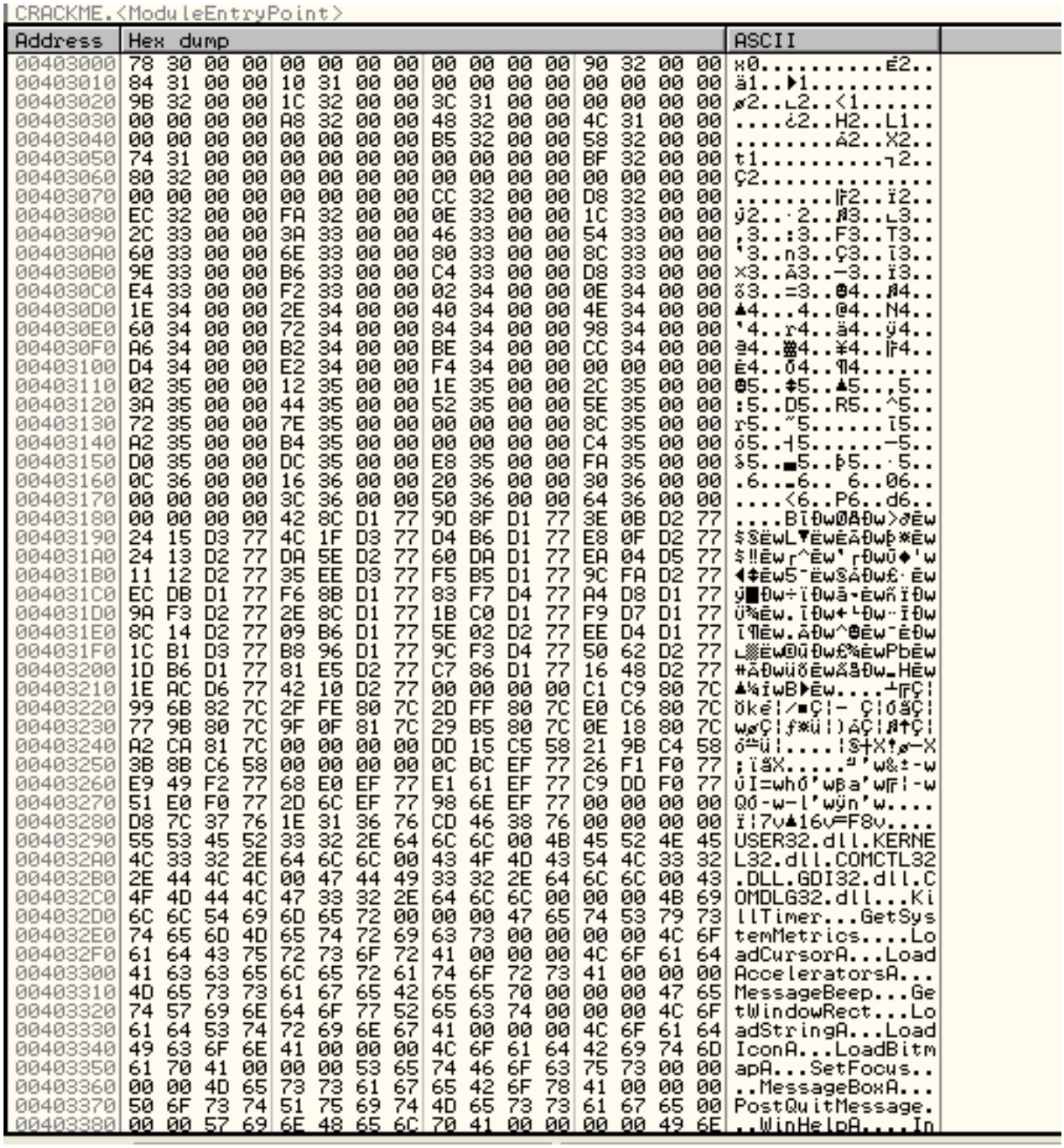


Здесь находится указатель на IT, то есть таблицу импорта, которую не надо путать с IAT

• IT = IMPORT TABLE
• IAT = IMPORT ADDRESS TABLE

Как видим, IAT – это хранилище, где система сохраняет после запуска правильные для моей машины адреса. А что такое IT? Идём туда. Как видим, она начинается с 3000 (403000), и её размер 670, то есть заканчивается в 403670. Смотрим, что там.

Поскольку мы ушли из заголовка, выходим из MODE SPECIAL-PE HEADER.



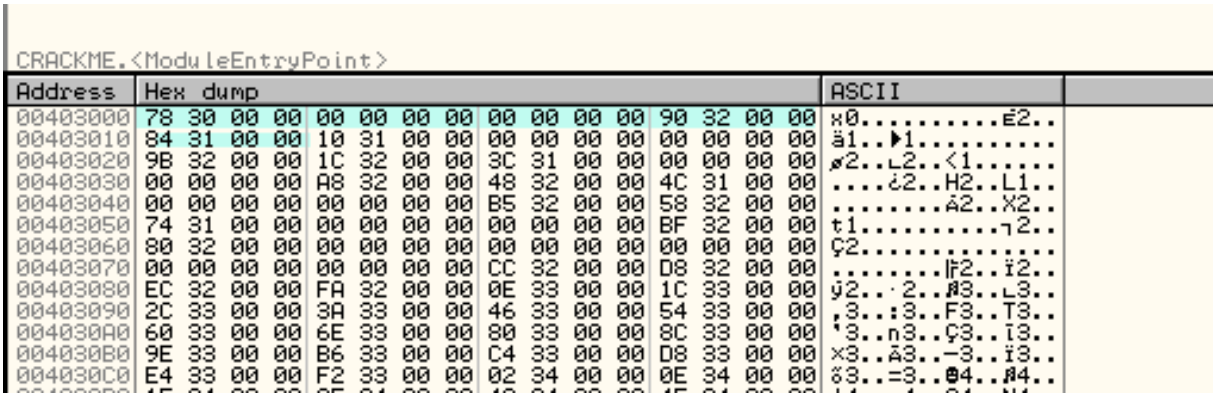
Это знаменитая IT, она у нас в руках, если мы поймём её, и хотя мы не знаем номер каждого из указателей, мы знаем, чем является каждый из элементов и куда указывает. Тогда мы спасены.

Плохо то, что приходится объяснять это в OllyDbg, так как гораздо удобнее и нагляднее делать это, представив IT в виде 5 колонок.

Знаменитая IT состоит из так называемых IMAGE IMPORT DESCRIPTOR'ов, являющиеся строками из 5 dword-значений, по одному дескриптору на каждую из загружающихся с программой DLL-библиотек.

Смотрим IT, помним, что каждый элемент (IID) занимает 5 колонок.

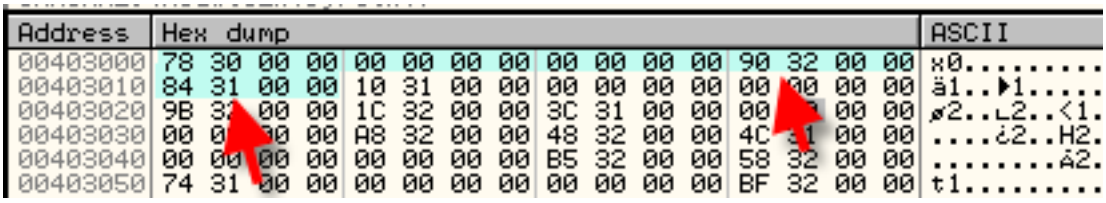
Рассмотрим пример.



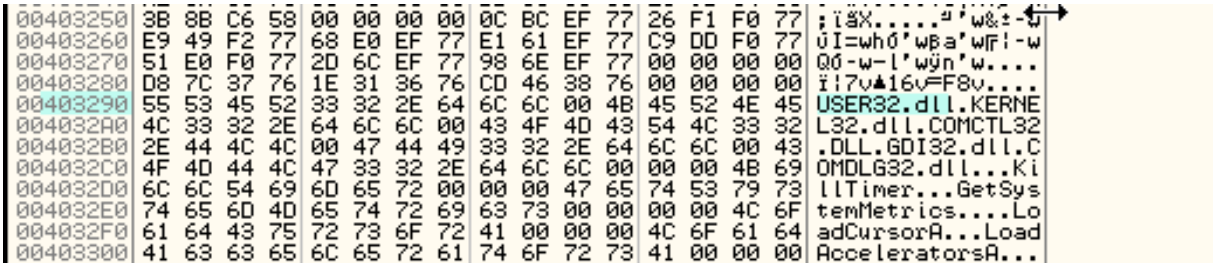
Это первый IID нашей IT, в котором 5 DWORD'ов, означающих следующее:

- OriginalFirtsThunks
- TimeDateStamp
- ForwarderChain
- Указатель на имя DLL
- FirtsThunk (Указывает туда, где должен производиться в IAT поиск первого вхождения данной DLL)

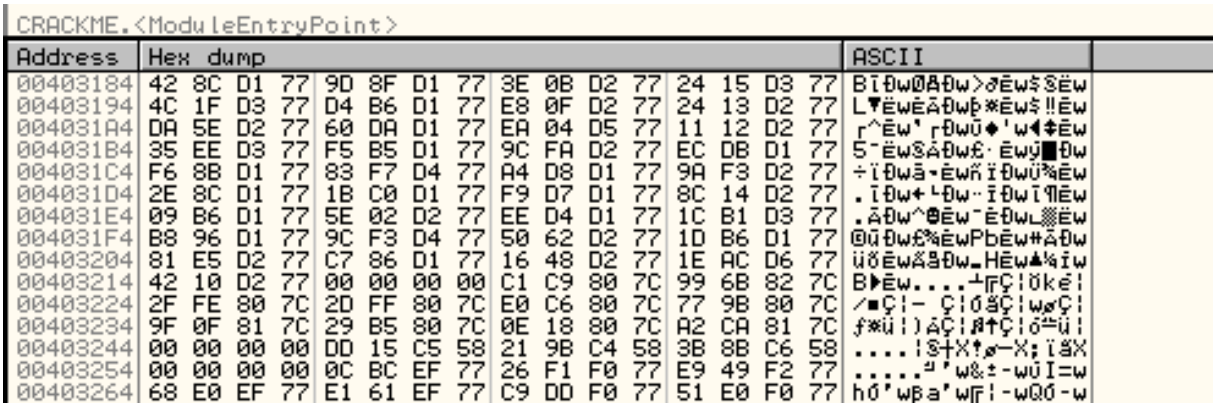
Три первых обычно не слишком важны для крэкинга, нас интересуют указатели 4 и 5.



Как видим, 4-ый указывает на имя DLL, которой соответствует данный IID, по адресу 403290 смотрим, что это за DLL.



Здесь у нас первая DLL, которую будет искать система – это USER32.DLL. 5 dword – это указатель, на вхождения этой DLL в IAT, в данном случае это 403184.



Вот IAT и её первый элемент, всё внутри IT, которая заканчивается в 403670. Таким образом, каждой DLL в IT соответствует один элемент, называющийся IID или IMAGE IMPORT DESCRIPTOR,

а внутри IT находится IAT, то есть хранилище адресов API-функций, всё в компактном виде, готовое для использования системой.

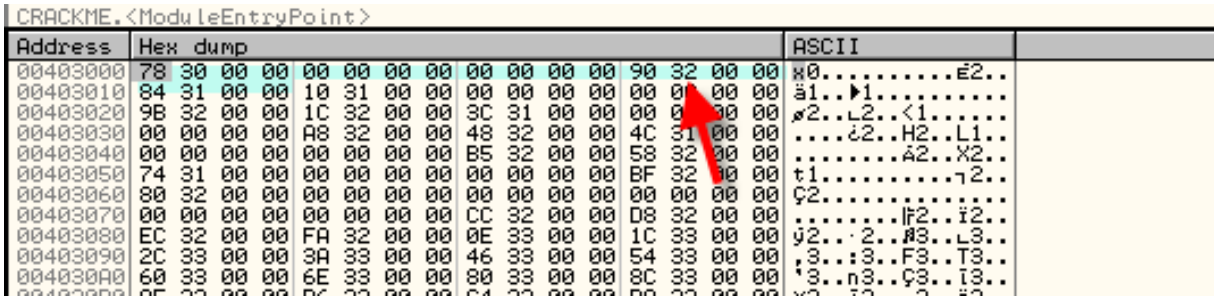
Многие крэкеры, экссперементировавшие с IAT, говорят, что IAT не всегда находится внутри IT, так как указатель на IID может указывать куда угодно, поэтому тот может располагаться в любом месте программа, куда есть разрешение на запись, чтобы при запуске exe-файла, программа могла сохранить правильные адреса API-функций, но тем не менее, общая последовательность того, что система делает для заполнения IAT, является следующий:

1. Ищет IT
2. Ищет первый IID и устанавливает, к какой DLL относится 4-ый указатель.
3. Затем смотрит первый указатель, чтобы выяснить, где располагается первый элемент IAT
4. Там находится указатель на строку с именем API-функции.
5. С помощью GetProcAddress ищется адрес и сохраняется в том же элементе.
6. Когда в IAT встречается элемент с нулям, это означает, что первая DLL закончилась, и теперь нужно смотреть второй IID, чтобы найти, где располагается информация о второй, и повторить процесс.

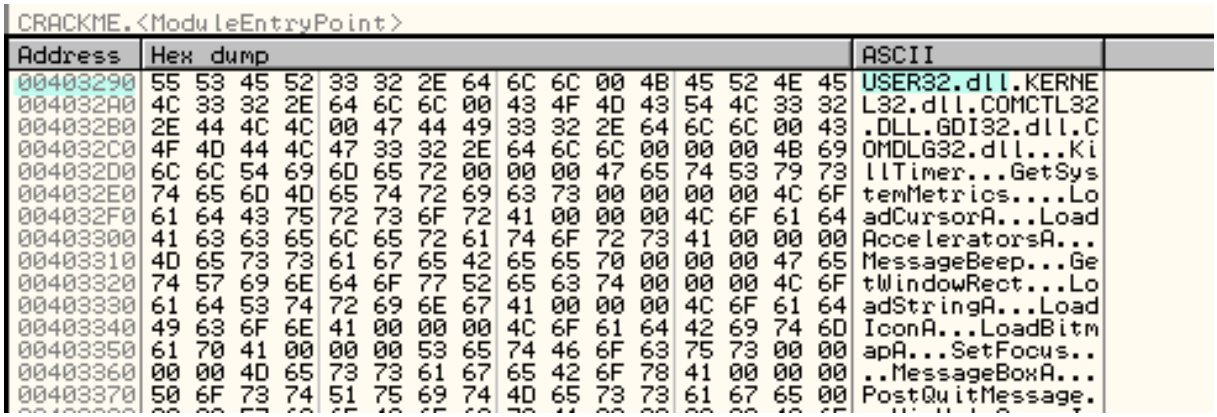
Таким образом, важно хорошо понимать, что делает система, и что произойдёт, если она встретит неправильные указатели.

1) Ищем адрес в IT

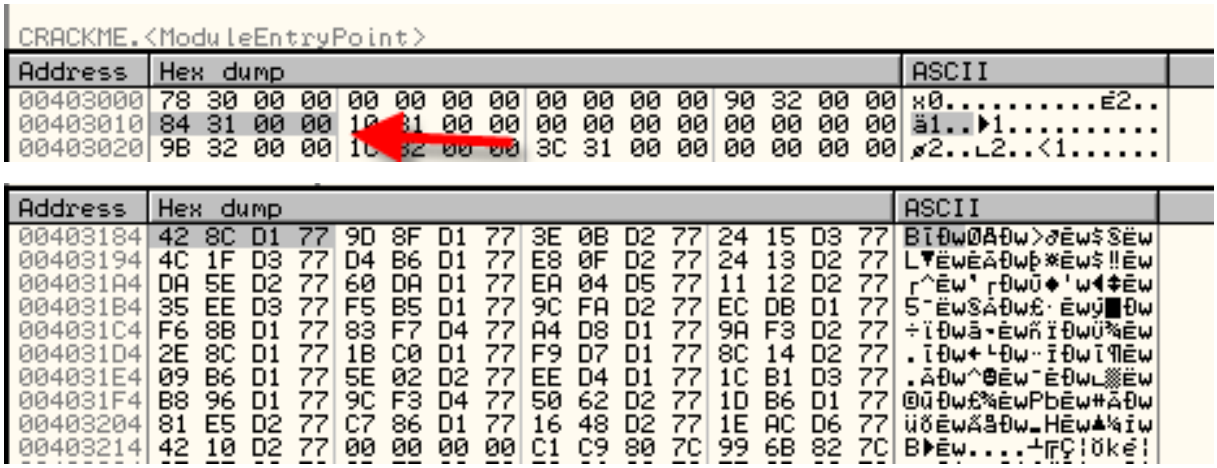
2) Идём по этом адресу



3) В первой IID ищем 4-ый DWORD, который указывает на имя DLL, в данном случае это USER32.DLL.

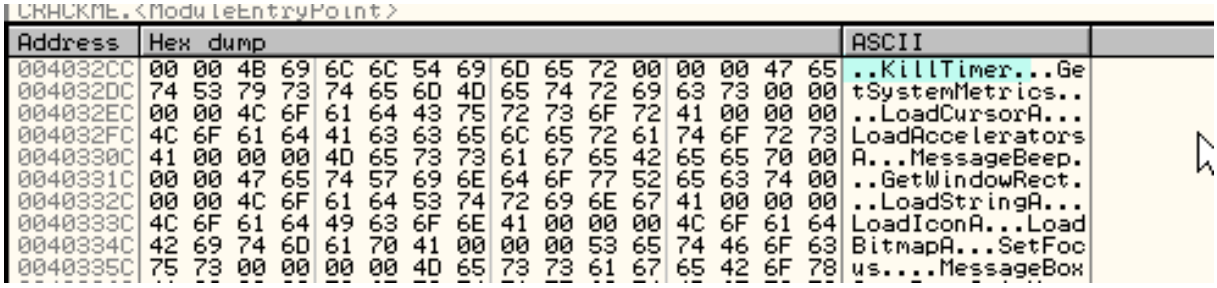


Затем смотрим содержимое 5-ого указателя, чтобы найти первый элемент IAT. Он равен 403184. Идём туда.

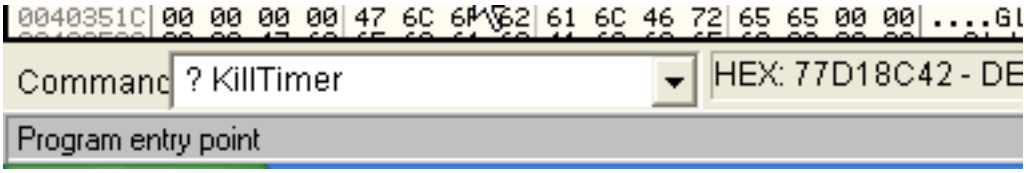


Здесь читаем значение и смотрим, на что оно указывает.

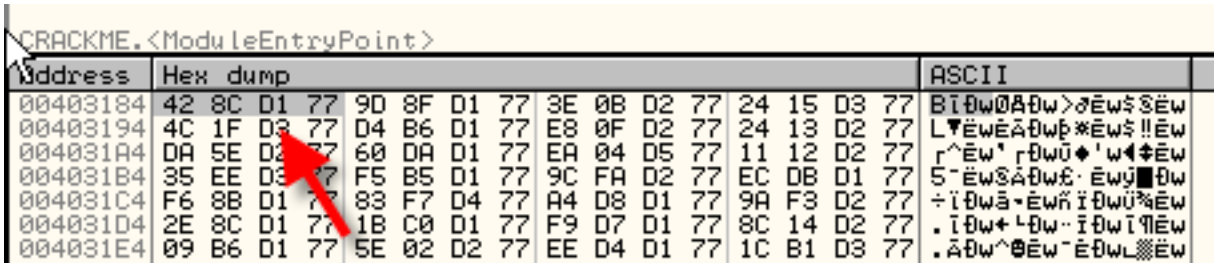
Это имя первой API-функции, оно располагается в 4032CC, идём туда.



Видим, что первая API-функция – это KillTimer, её адрес находится с помощью GetProcAddress, мы можем сделать это с помощью OllyDbg.

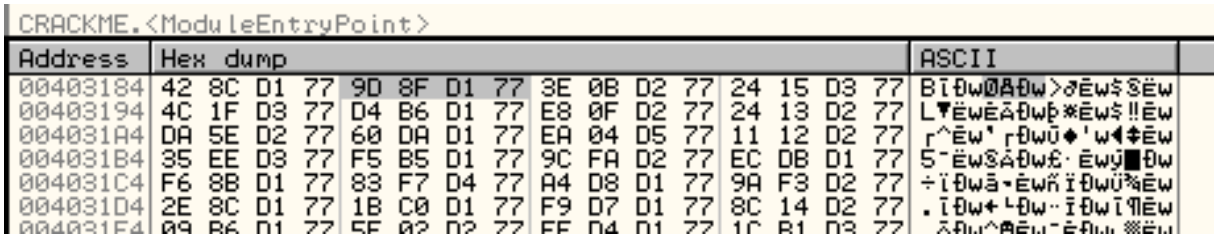


И это значение, которое на моей машине равно 77d18c42, сохраняется в тот же элемент IAT, перезаписывая его предыдущее значение.



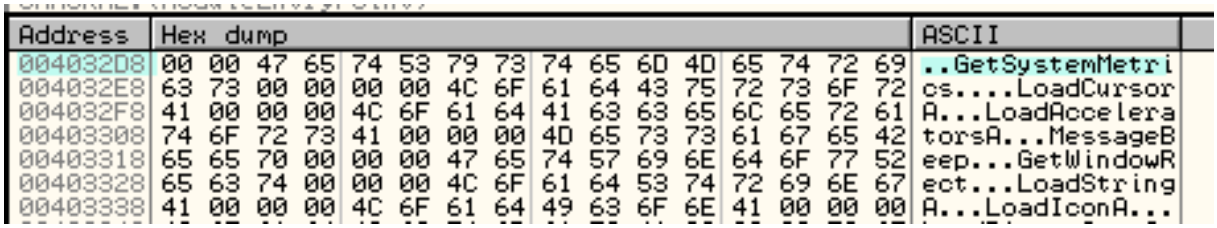
То, что мы здесь видим – это первый элемент IAT, и значение, которое оно содержит на моей машине.

Затем переходим к следующему элементу IAT, который легко найти, прибавив 4 к адресу предыдущего, и находим указатель на имя следующей API-функции.

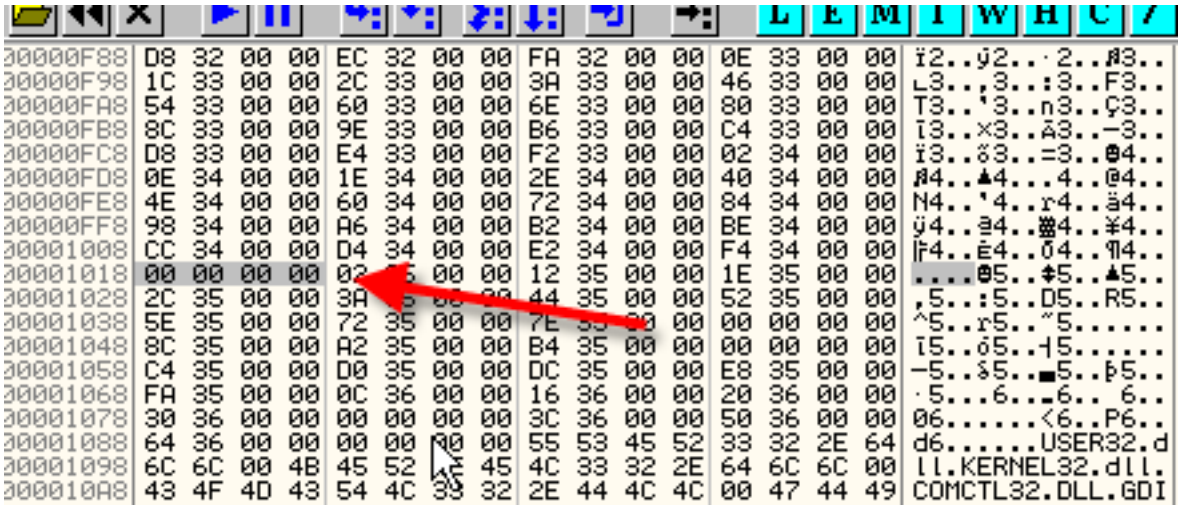


Конечно, смотрим исполняемый файл и вот оно:

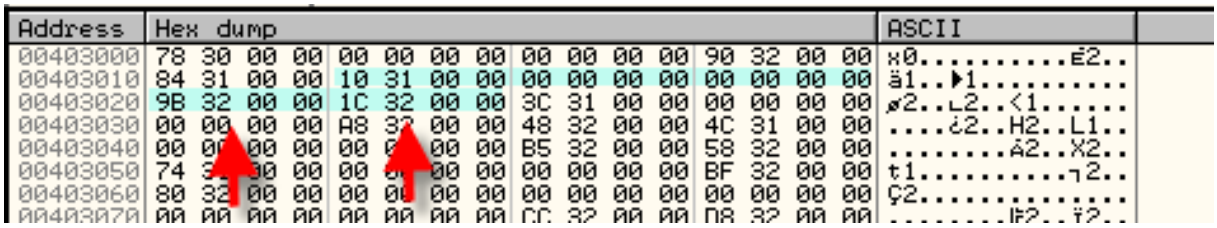
32d8, то есть 4032d8, смотрим имя следующей API-функции по этому адресу, и так как в этом элементе нет ноля, это означает, что продолжается список функций user32.dll.



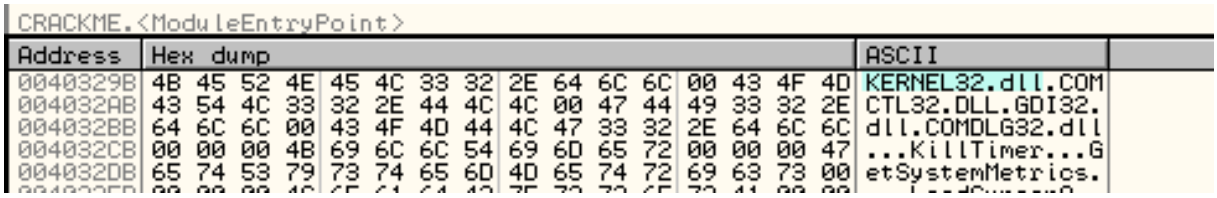
Вторая API-функция – это GetSystemMetrics, её адрес будет найдён и сохранён во второй элемент IAT, и так будут находиться все API-функции библиотеки user32.dll, пока не будет обнаружен элемент, в котором содержутся все нули. Посмотрим, где он находится в исполняемом файле.



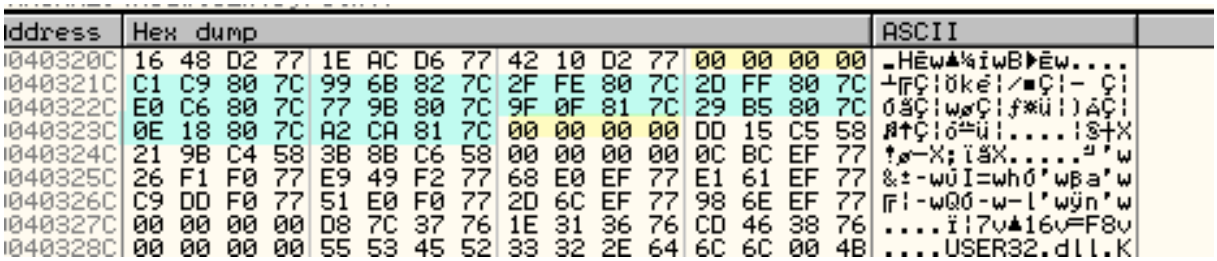
Видим, что поиск в IT user32.dll будет продолжаться, пока не найдётся вот этот элемент с нулями, после чего произойдёт переход к следующему IID.



Вот 4-ый и 5-ый указатели, 4-ый говорит нам, что в 40329b находится имя 2-ой dll, где будет происходить поиск функций.



Это kernel32.dll, и первый элемент её IAT находится после элемента с нулями, т.е. начиная с адреса 40321c.



Здесь видим ноль, который задаёт конец API-функций из user32.dll, и в 40321c начинается IAT Kernel32.dll, где будут искаться указатели на имена функций и которая будет заполнена их реальными адресами.

Думаю, что очень важно понимать этот процесс, я попытался объяснить его и повторить это несколько раза, чтобы вы хорошо поняли его, так как это основа всей темы реконструкции IAT, и хотя мы рассмотрим в следующей части соответствующие инструменты, которые могут нам помочь сделать рутинную работу за нас, очень важно знать, как они работаю, чтобы в случае каких-либо проблем мы не сошли с дистанции и всегда могли понять, почему это произошло.

Ок, на этот раз достаточно, в следующей главе мы приведём IAT в порядок.

Введение в крэкинг с нуля, используя OllyDbg - Глава 34

Автор: Рикардо Нарваха, пер. Aquila

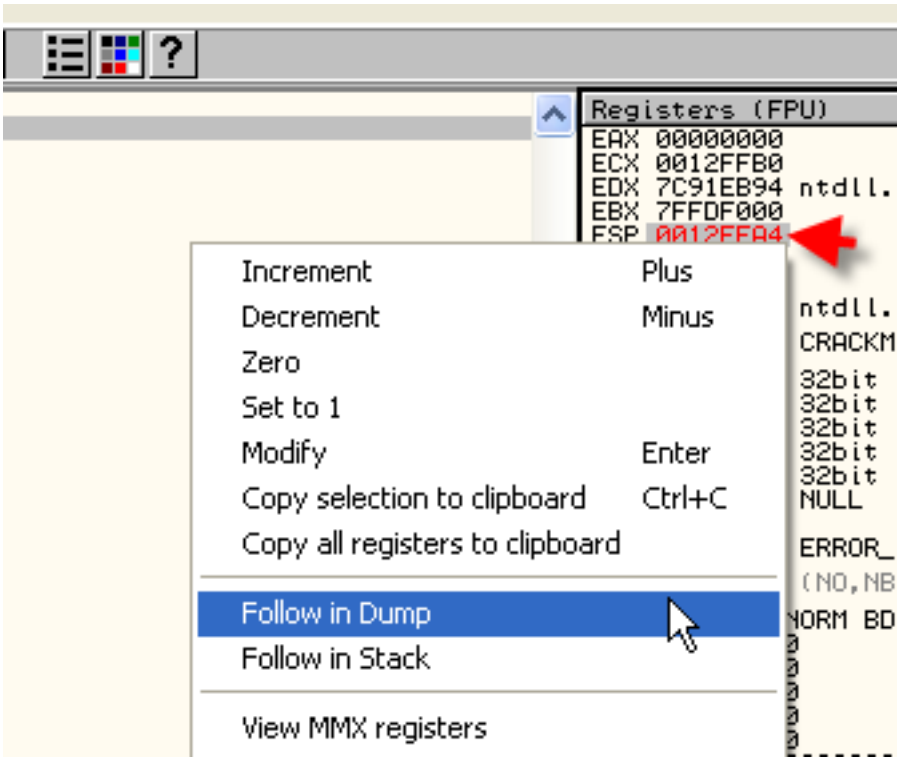
Дата: 2008-10-07

В 33-ей части мы рассмотрели, как работают IT и IAT. Кто-то мог бы подумать, что для починки IAT вовсе не необходимо знать, как она работает, так как есть инструменты, позволяющие это сделать автоматическим образом, но я вас уверяю – знание того, как и что работает – это очень хорошо, так как многие упаковщики вводят подобные инструменты в заблуждение, и те отказываются работать, и в этих случаях необходимо поразмыслить и подумать, как осуществить подобную правку собственноручно, и двигаться дальше.

Начнём с чего-нибудь лёгкого, например, с крэкми CrueHead'a, запакованного UPX. Сделаем полную распаковку, чтобы подготовить нас к главному, а потом починим IAT, чтобы она работала.

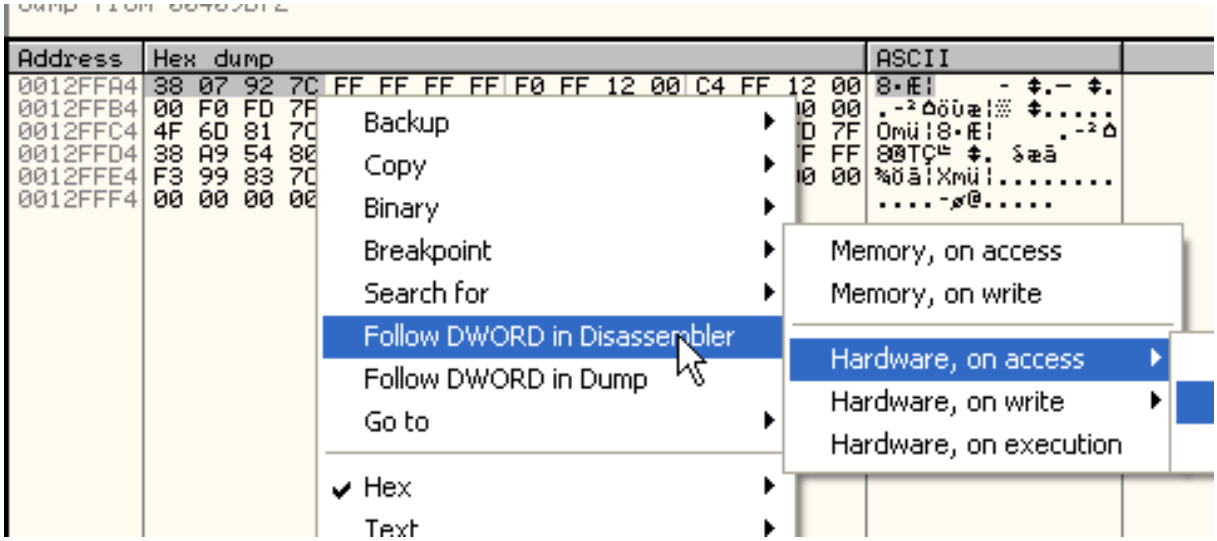
Как видим, первый шаг – это прибытие в OEP, для чего открываем крэкми в OllyDbg.

Применяем метод PUSHAD, чтобы дойти до OEP, жмём F7, чтобы пройти PUSHAD.



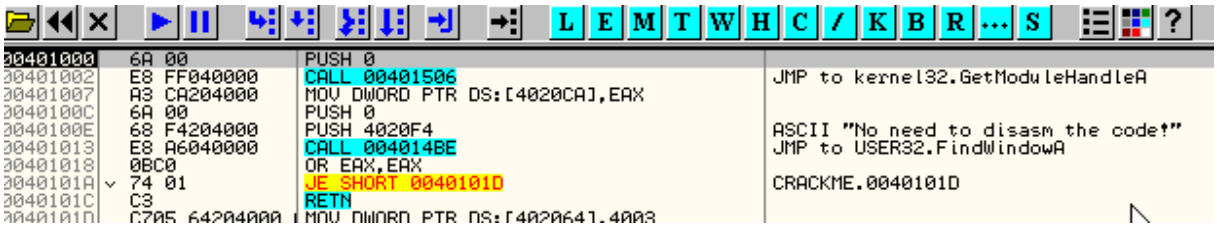
Теперь отмечаем ESP-FOLLOW IN DUMP.

И в дампе смотрим сохранённые регистры, отмечаем первые 4 байта и устанавливаем HARDWARE BPX ON ACCESS.



И делаем RUN, чтобы остановиться затем на переходе на OEP.

Хорошо, я нахожусь на переходе на OEP, нажимаю F7 и иду дальше.



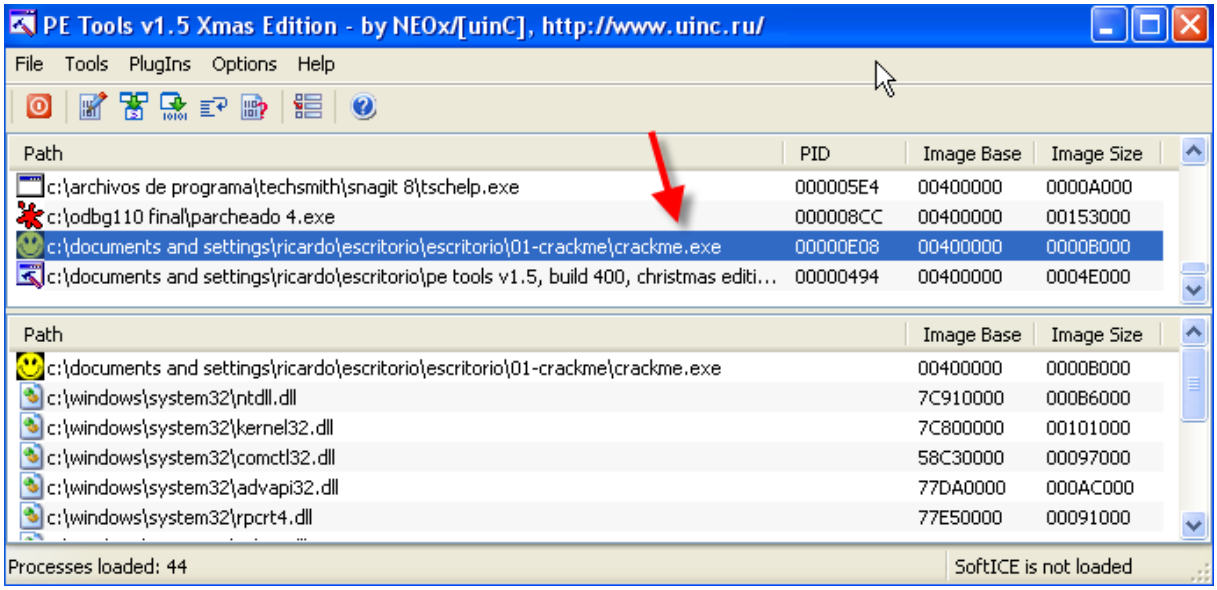
Ок, прибыли в то место, где программа распакована в памяти. Теперь сдампим её.

В прошлой части я уже говорил, что существует много разных дамперов, в том числе и для OllyDbg существует плагин под названием OllyDmp, который очень хорошо дампит, но поскольку мы рассматривали ранее LORD PE, то будем использовать PE TOOLS, которые можно скачать по ссылке ниже:

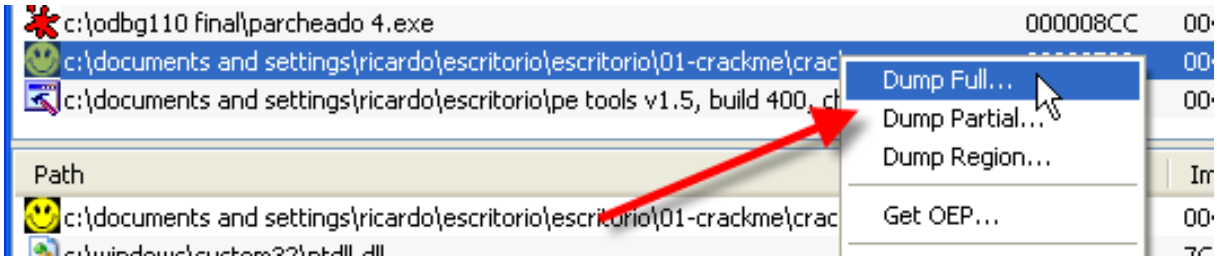
http://www.uinc.ru/files/neox/PE_Tools.shtml

В следующие разы мы, возмжно, также будем использовать OllyDbg, чтобы научиться использовать оба инструмента.

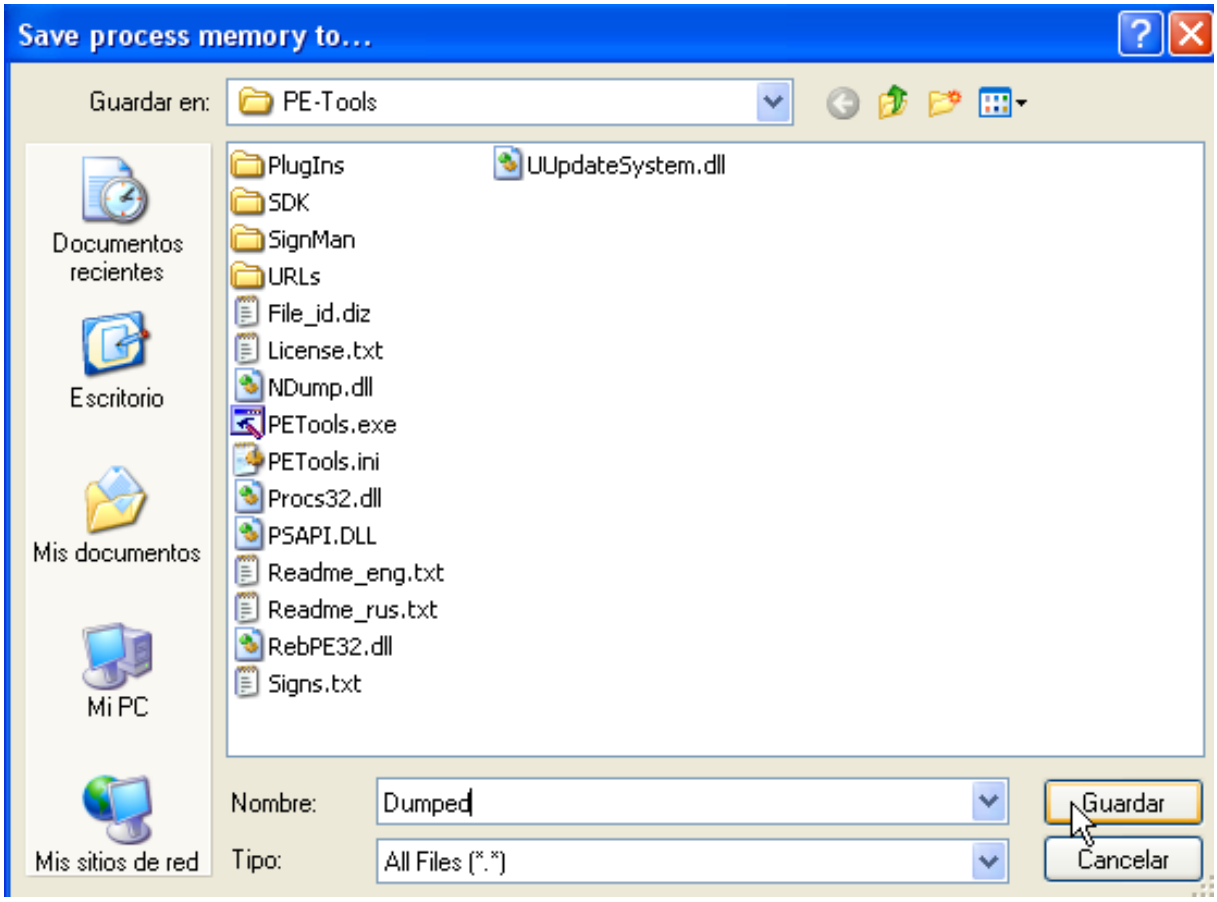
Открываем PE-TOOLS без закрытия OllyDbg, где мы по-прежнему стоим на OEP.



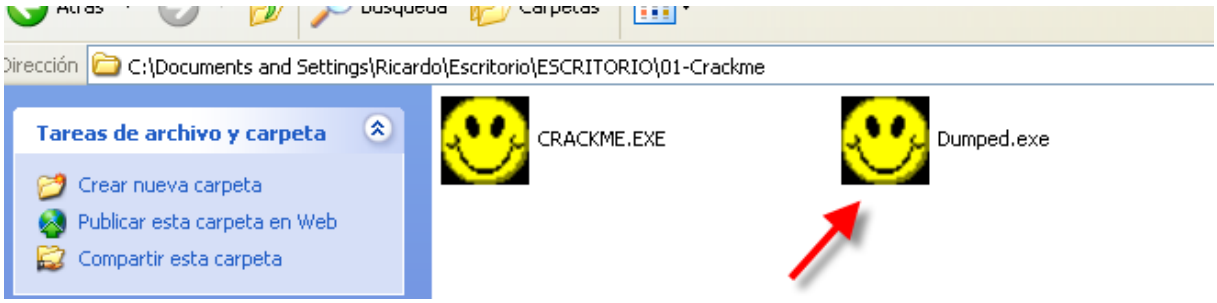
Ок, вот нужный нам процесс, который в данном случае называется crackme.exe, поскольку я потерял тот, что назывался crackmeUPX, пришлось сделать новый, но я забыл поменять имя, но это точно такой же крэкми, запакованный UPX'ом, что и в прошлый раз.



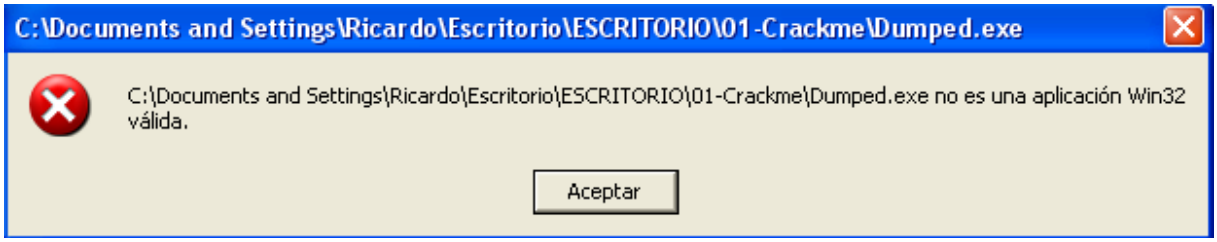
Делаем щелчок правой кнопкой мыши – DUMP FULL.



Ок, сдампили, теперь переходим к починке IAT. Закрываем PE TOOLS, видим, что сдампленный файл сохранился в директорию PE TOOLS, так что ищем его и копируем в директорию, где находится упакованный UPX'ом крэкми CrueHead'a.



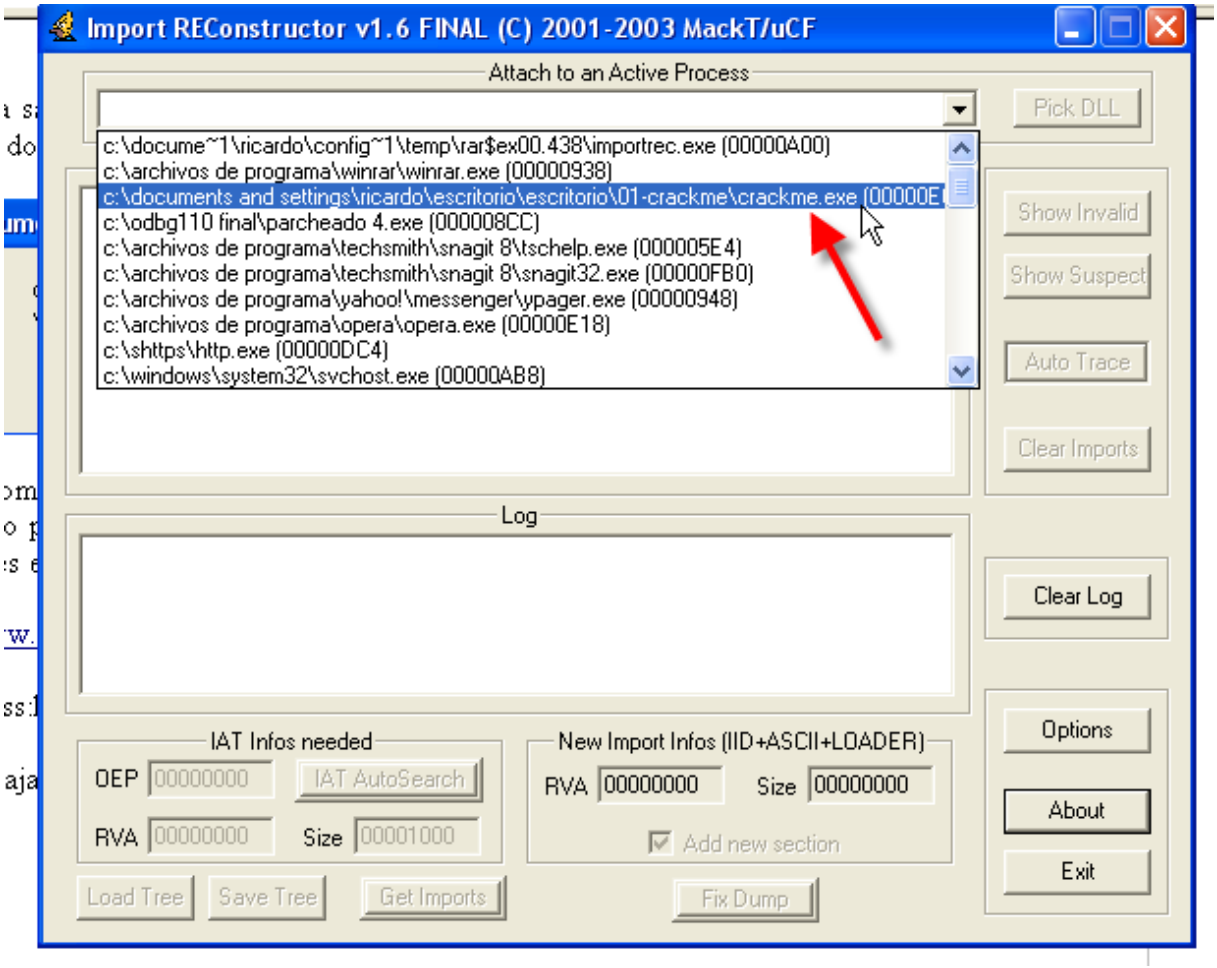
Попробуем запусть этот файл, хотя и знаем, что ничего хорошего из этого не выйдет. Делаем двойной щелчок на dumped.exe.



Ок, как видим, нужно починить IAT, чтобы файл мог работать на любой машине, а не только на нашей, для чего используем IMPORT RECONSTRUCTOR, конечно, не закрывая OllyDbg с загруженным крэкми CC, остановленным на OEP, так как IMP REC будет работать, используя его.

http://ricardonarvaja.info/WEB/OTROS/HERRAMIENTAS/F-G-H-I-J-K/ImportReconstructor16f.zip

Программу можно скачать по ссылке выше.



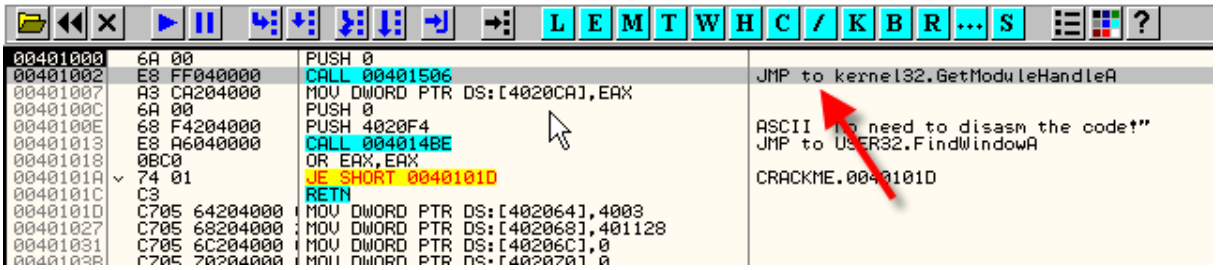
Запускаем её и ищем процесс с крэкми CrueHead'а, упакованного UPX'ом, которое остановлено на точке входа.

Ок, сейчас встаёт проблема, которая озадачивает множество начинающих, а именно – как найти начало и конец IAT. Конечно, в крэкми, которое у нас остановлено на OEP, мы уже видели, что упаковщик разрушает IT, поэтому 4-ый и 5-ый указатели первого IID (Imaga Import Descriptor), т.е. имя DLL и адрес первого элемента её IAT, у нас нет, поэтому для её нахождения нужно использовать другие способы.

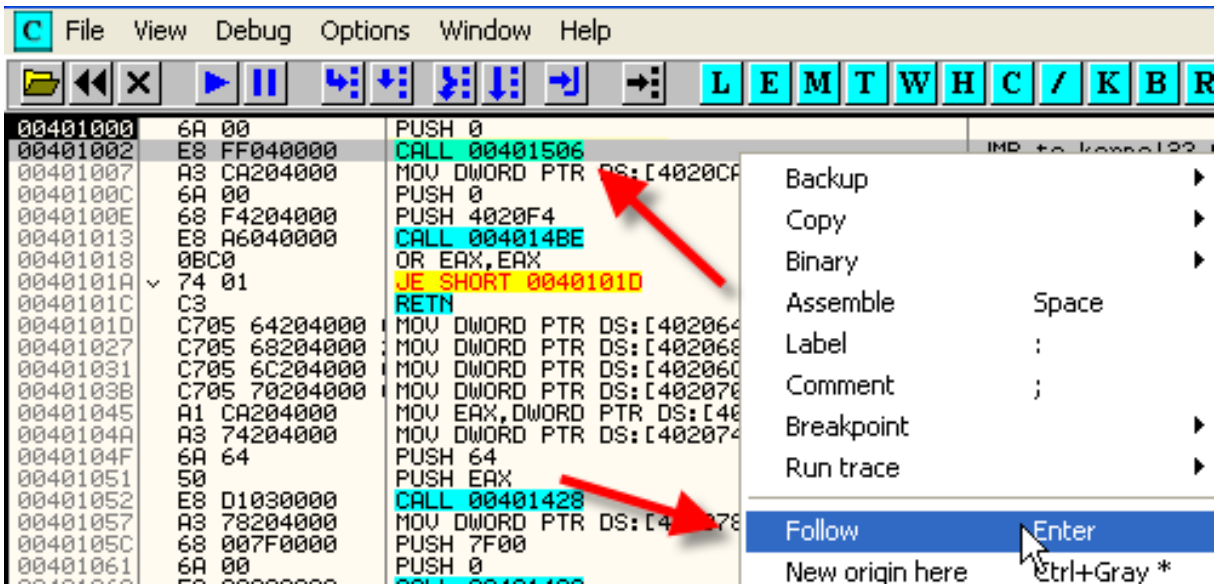
Конечно, знаем, что в общем случае вызовы API-функций реализуются с помощью косвенных JMP или CALL'ов.

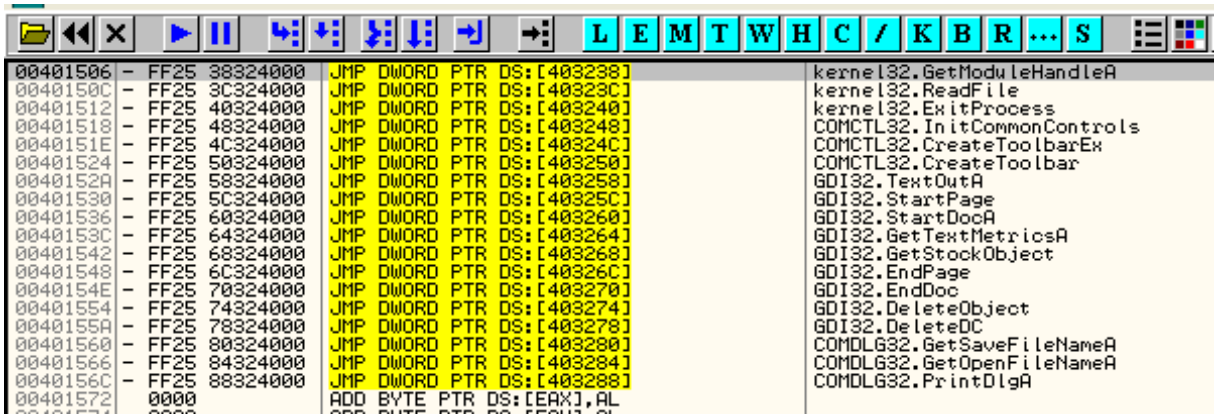
```
JMP [xxxxxxx] o CALL [xxxxxx]
```

Как мы рассматривали в предыдущей части, программа берёт адрес API-функции на нашей машине из IAT, которая является хранилищем адресов API-функций на нашей машине. Ищем переход на API-функцию в упакованном файле, для чего снова идём в OllyDbg с загруженным упакованным CC, остановленным на OEP.



Здесь на второй строке видим CALL, о котором OllyDbg нам говорит, что это API-функция, хотя на самом деле это подготовительный шаг косвенного перехода, так что отмечаем эту строку и делаем FOLLOW.

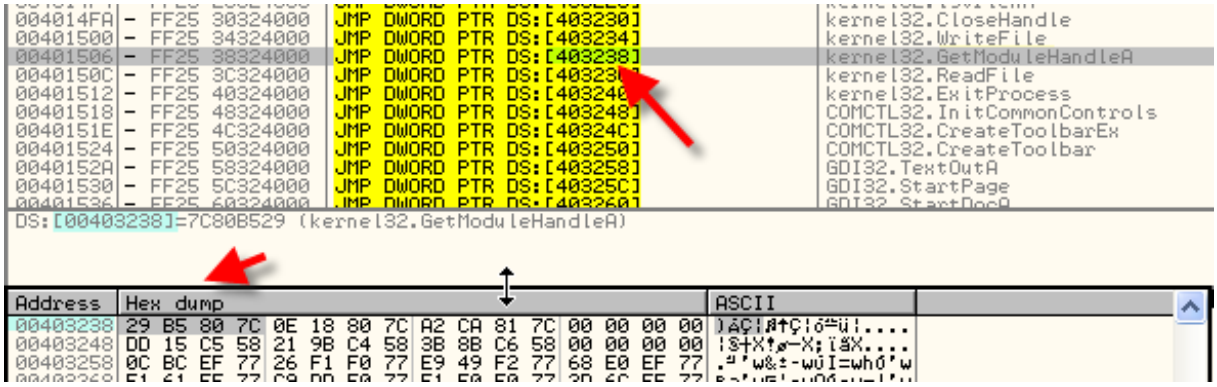




Здесь находим таблицу переходов, содержащую значения IAT, сюда мы попадаем при вызове каждой API-функции. Как видим, эти переходы начинаются с опкодов FF 25, поэтому во многих туториалах сразу делается поиск двоичной строки ‘FF 25’, чтобы попасть сюда гораздо быстрее.

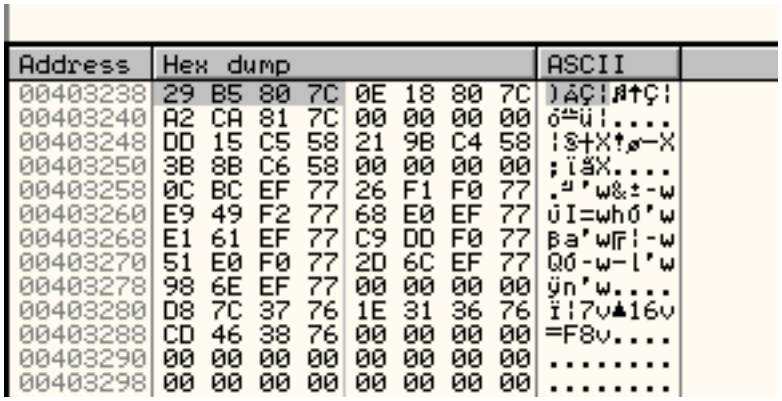
Дело заключается в том, что не все программы используют косвенные переходы, чтобы добраться до этих API-функций, поэтому иногда этот метод не срабатывает, и лучший способ, который никогда не подводит – это поискать вызов API-функции, и посмотреть, откуда берётся сохранённое значение, которое приводит нас к API-функции, и это значение должно быть сохранено в IAT.

Вот в примере JMP [403238]:

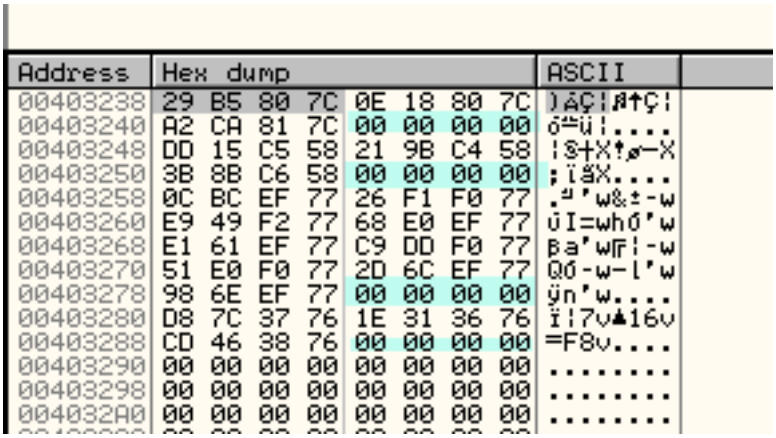


Довольно очевидно, что 403238 – это элемент IAT, где сохранён адрес API-функции GetModuleHandleA, и таким же образом можно посмотреть в DUMP'е часть IAT, что необходимо для определения где она начинается и где заканчивается.

Конечно, можно посмотреть на все эти косвенные JMP и определить, у какого из них минимальный, а какого максимальный адрес, но это очень медленно, а лучше пойти в DUMP и подняться немного. Как мы знаем, в каждом элементе содержится адрес одной API-функции, в данном случае это 7C80B529, то есть если перевернуть, получится 29 B5 80 7C. Переключитесь в режим одновременно просмотра двух колонок, чтобы был обзор получше.



Здесь видим структуру IAT. В прошлой части мы рассматривали, что все элементы, относящиеся к одной DLL, идут последовательно, и перед началом следующей идёт элемент-разделитель, состоящий из одних нулей.



Некоторые навороченные упаковщики перезаписывают нули мусором, чтобы сделать реконструкцию более сложной. Все эти элементы не используются, и так как программа уже не должна сама запускаться, не нужно и оставлять эти нули, но здесь они остались, а также IAT для

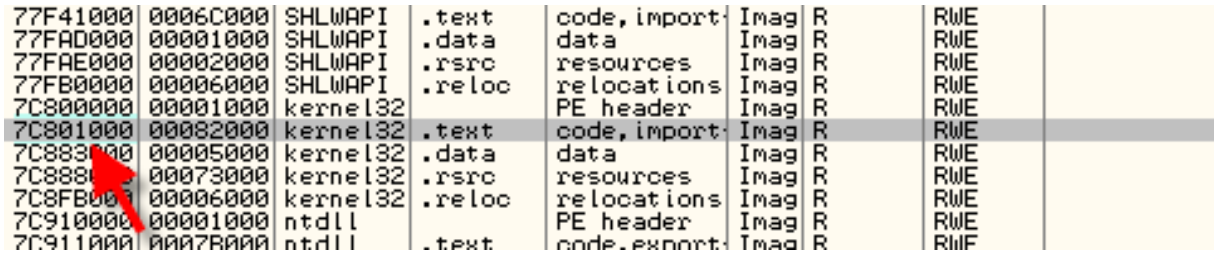
разных DLL находятся друг за другом. Ниже находятся

```
00403238  29 B5 80 7C   0E 18 80 7C  )µ€|##€|
00403240  A2 CA 81 7C   00 00 00 00
```

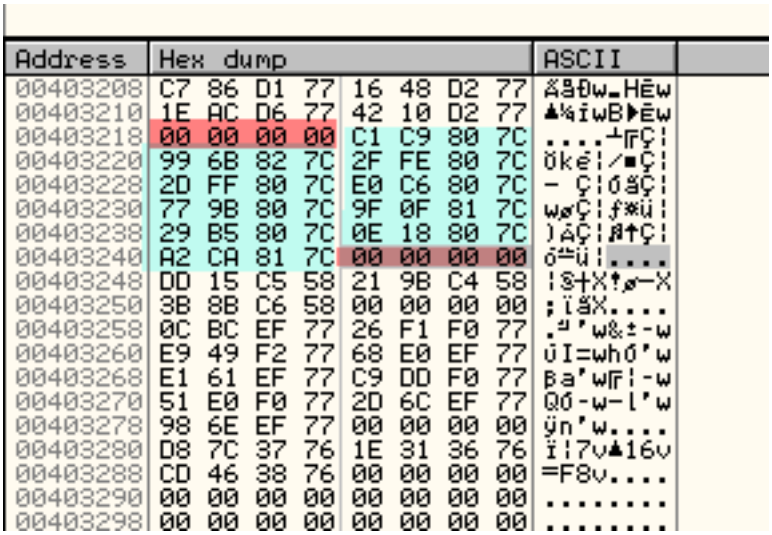
три API-функции, которые указывают на адреса 7Cxxxxxx, а затем идёт нулевой элемент, который отделяет следующую DLL.

Если используем VIEW-M, то увидим, что секция CODE данной DLL-библиотеки соответствует этим адресами вида 7Cxxxxxx.

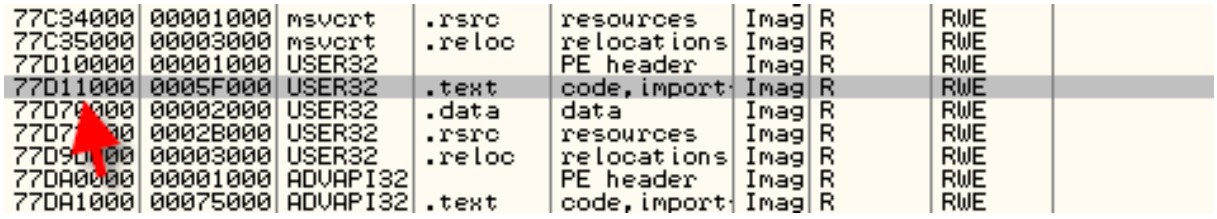
Итак, все эти адреса указывают на секцию кода kernel32.dll.



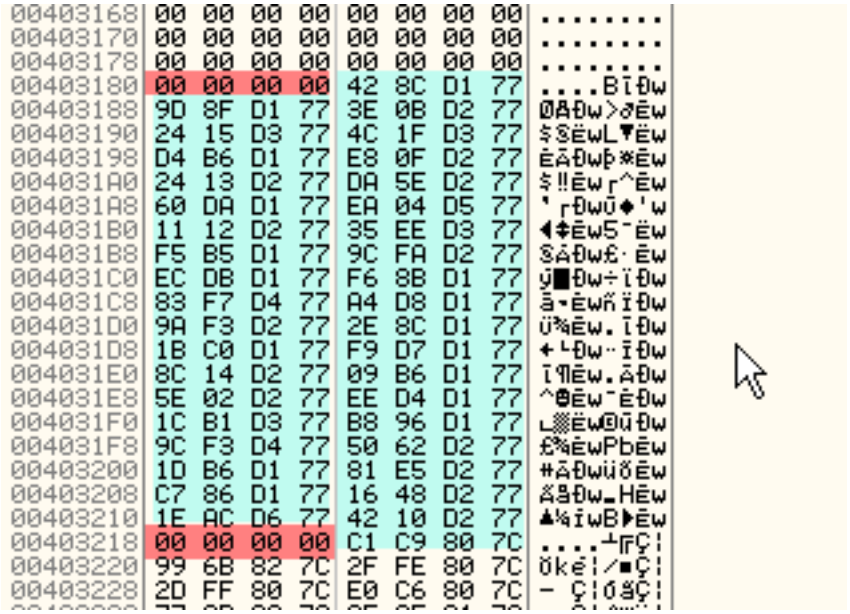
Конечно, на ваших машинах DLL-библиотеки могут находится по другим адресам, но здесь мы увидели, что на моей машине эти элементы соответствуют секции CODE kernel32.dll.



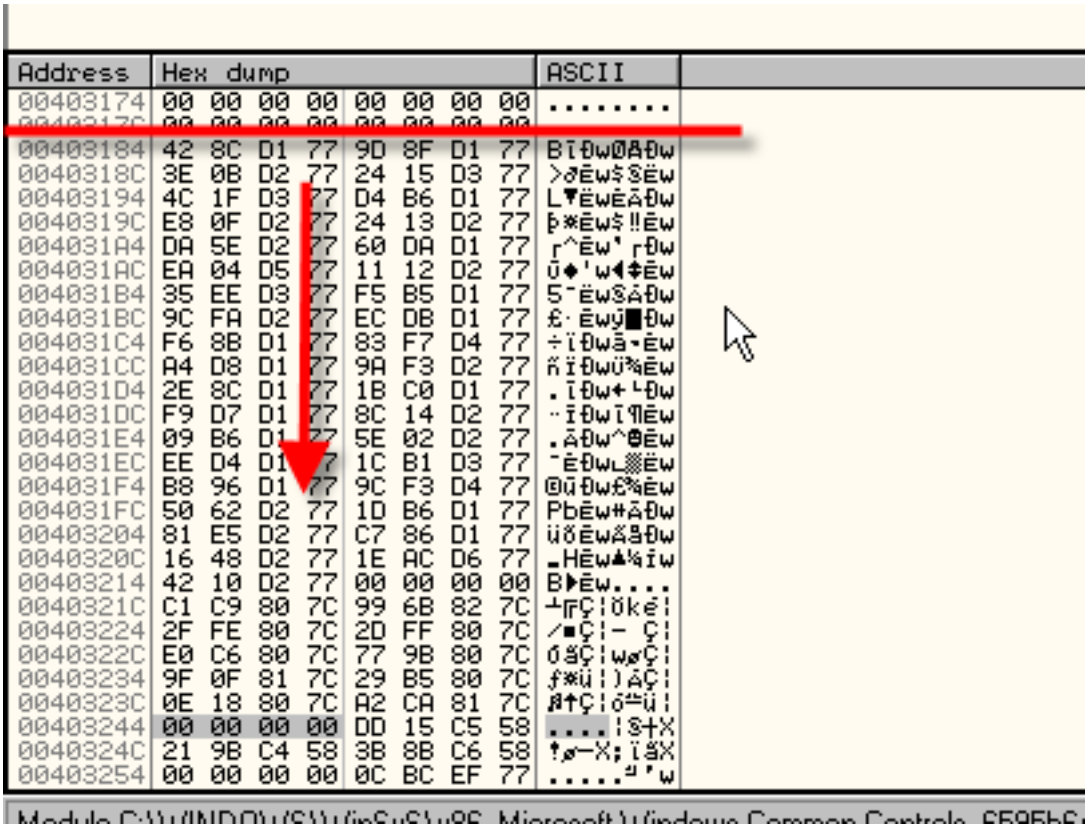
Ок, рассмотрели все элементы, указывающие на kernel32.dll, видим нулевой разделитель, помеченный розовым цветам, а выше находятся элементы, которые ведут в другую DLL, в данном случае в её секцию CODE, среди них встречаются элементы, начинающиеся на 77Dxxxxx, так что смотрим в M, куда они ведут.



Адреса, находящиеся выше разделителя, соответствуют секции CODE библиотеки user32.dll. Продолжаем спускаться до следующего разделителя.



Здесь видим все API-функции, которые находятся внутри секции CODE библиотеки user32.dll и разделитель, но выше ничего нет, это значит, что начало IAT – в 403184, и это первый действительный элемент.

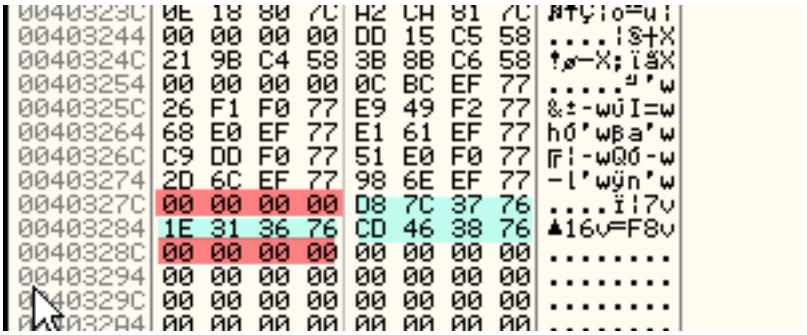


Ясно видим, что выше нет элементы, которые вели бы в какую-либо DLL, и в этом случае, вдобавок, ещё выше находятся все нули, которые нам облегчают нахождения начала IAT. Некоторые сложные упаковщики обычно заполняют мусором место перед IAT и после неё, что затрудняет обнаружение её начала, но как только мы знаем, что элементы IAT всегда должны вести в секцию кода библиотеки, то мы легко можем определить, что это мусор, так как он не ведёт в код никакой библиотеки.

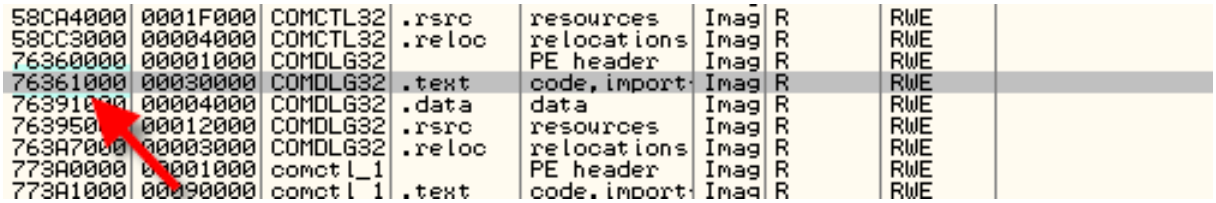
Итак, в образе началом IAT является 403184, теперь идёт вниз, пока не найдём конец IAT, используя тот же метод, используя тот же метод: продолжаем, пока элементы указывают на секцию

кода какой-либо DLL.

Позже мы также рассмотрим упаковщики, которые меняют элементы IAT и перенаправляют на свои процедуры, в свою очередь совершающие переход на API-функцию. В этом случае метод выше не сработает, и мы будем изучать дальше, что делать, а пока будем исходить из того, что элементы IAT – это адреса на API-функции, которые должны указывать на секцию кода DLL-библиотеки.



Здесь видим последние элементы IAT, которые ведут по адресам 76xxxxxx. Проверим, какой DLL они соответствуют.



В данном случае они соответствуют секции CODE в библиотеке COMDLG32.DLL, и после них больше нет элементов, так что это конец IAT, таким образом, адресом, отмечающим конец таблицы, будет 40328C, мы также могли бы принять, что концом является последний элемент 403288, и это бы было тоже самое, но для большей ясности рассмотрим, что конец в 40328C, и установим это как конец IAT.

Поэтому у нас теперь есть начало и конец IAT.

```
НАЧАЛО: 403184
КОНЕЦ: 40328C
```

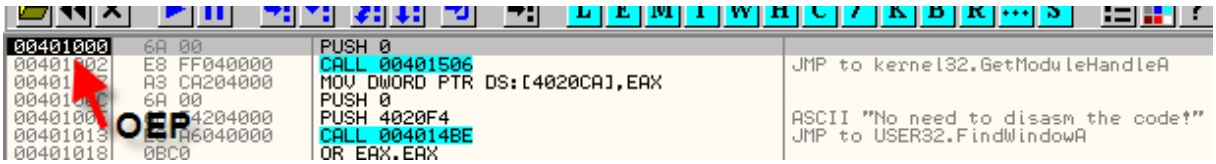
IMPORT RECONSTRUCTOR запрашивает у нас сведения о следующих трёх вещах:

1)Начало IAT, но нужно сначала отнять от 403184 базу образа, которая в данном случае равна 400000, так что получится 3184.
2)Второе значение, которое у нас запрашивается — это размер IAT. Чтобы его узнать, нужно отнять КОНЕЦ от НАЧАЛА.

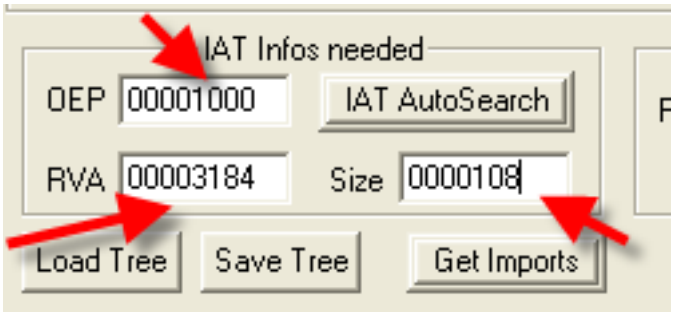
Размер=40328c-403184=108

Поэтому второе значение равно 108.

3) Третье – это OEP, также с вычтенной базой образа, то есть 401000-400000=1000

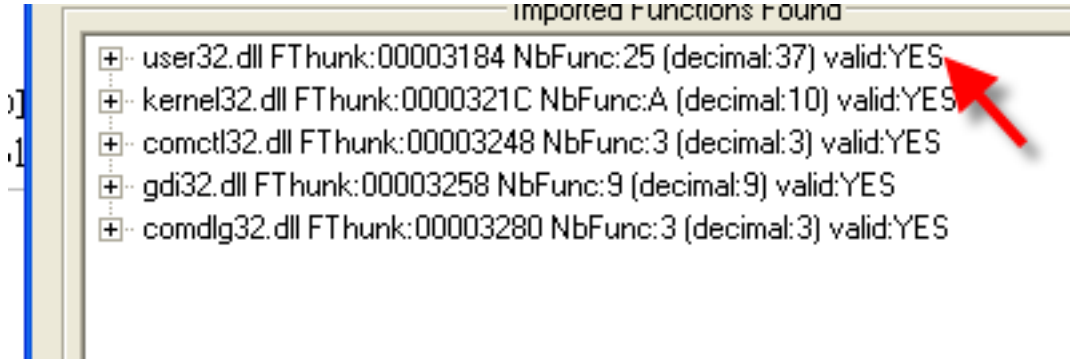


Эти данные вводятся в IMP REC.



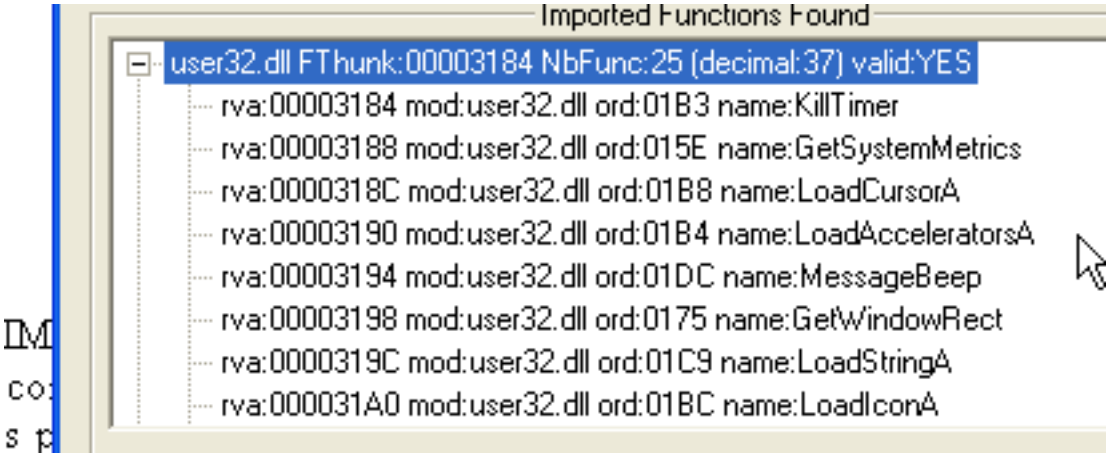
Здесь видим данные, которые ввели в IMP REC, в OEP ввели 1000, то есть 401000 с вычтенной базой образа, в RVA начало таблицы, также с вычтенной базой, и в Size – размер IAT.

Здесь нажимаем GET IMPORTS.

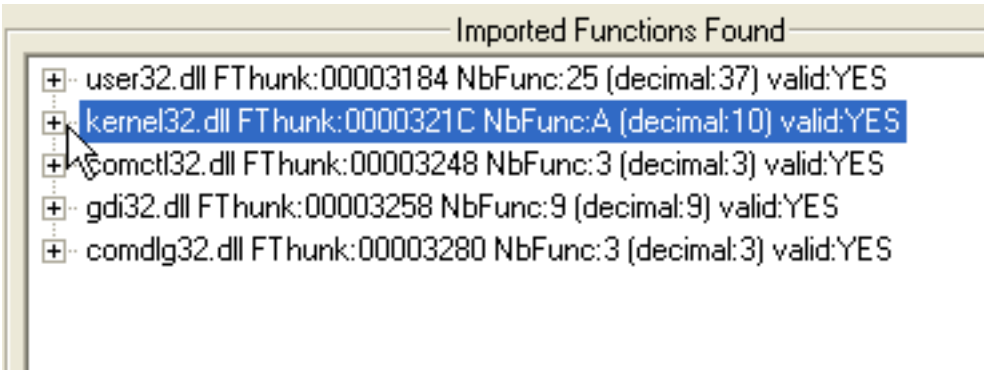


Видим, что IMP REC, чья задача установить, какие API-функции относятся к каждому элементу IAT, и кроме их нахождения, он сообщает, правильны ли они, отмечая их словом YES. В случае, если элементы не ведут напрямую на API-функцию, как это бывает с некоторыми упаковщиками, он показывает NO, и в этом случае необходимо установить, куда этот элемент ведёт на самом деле. После того, как он приведён в порядок, IMP REC распознает его как корректный элемент и скажет «YES», и как только всё станет «YES», мы можем починить дамп.

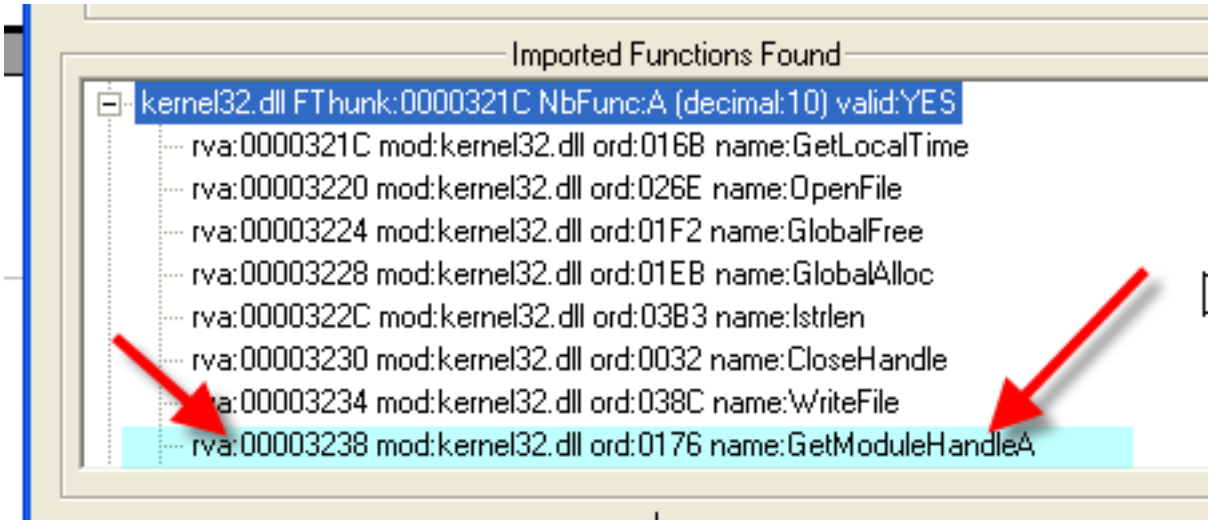
Прежде, чем это делать, посмотрим на каждую DLL, развернув содержимое путём нажатия на +.



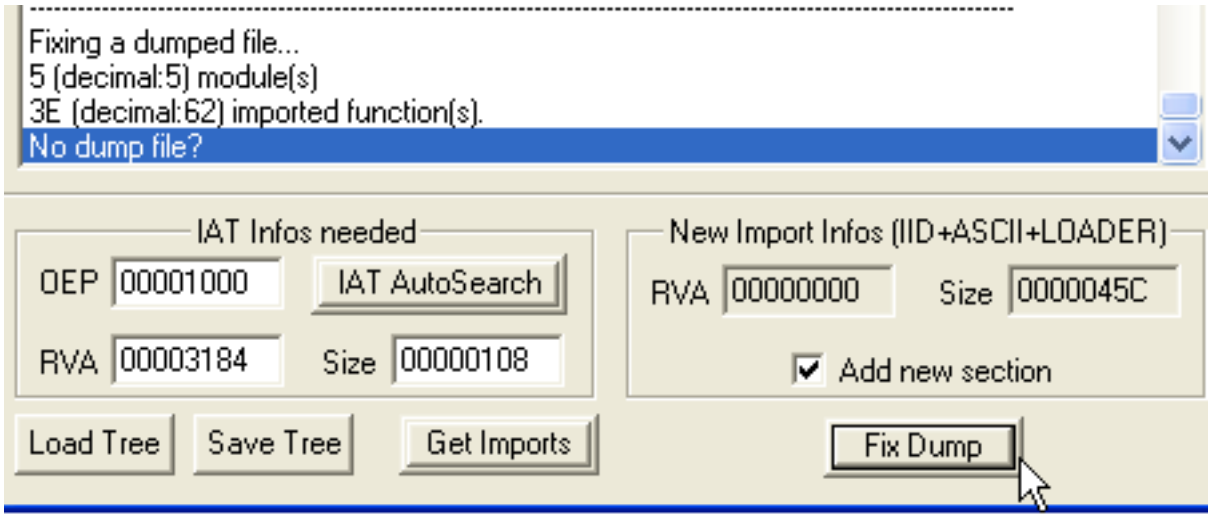
Тут у нас все элементы, к которым относятся API-функции, и если есть желание, то можно поместить первую из них (а это GetModuleHandleA) в IAT.



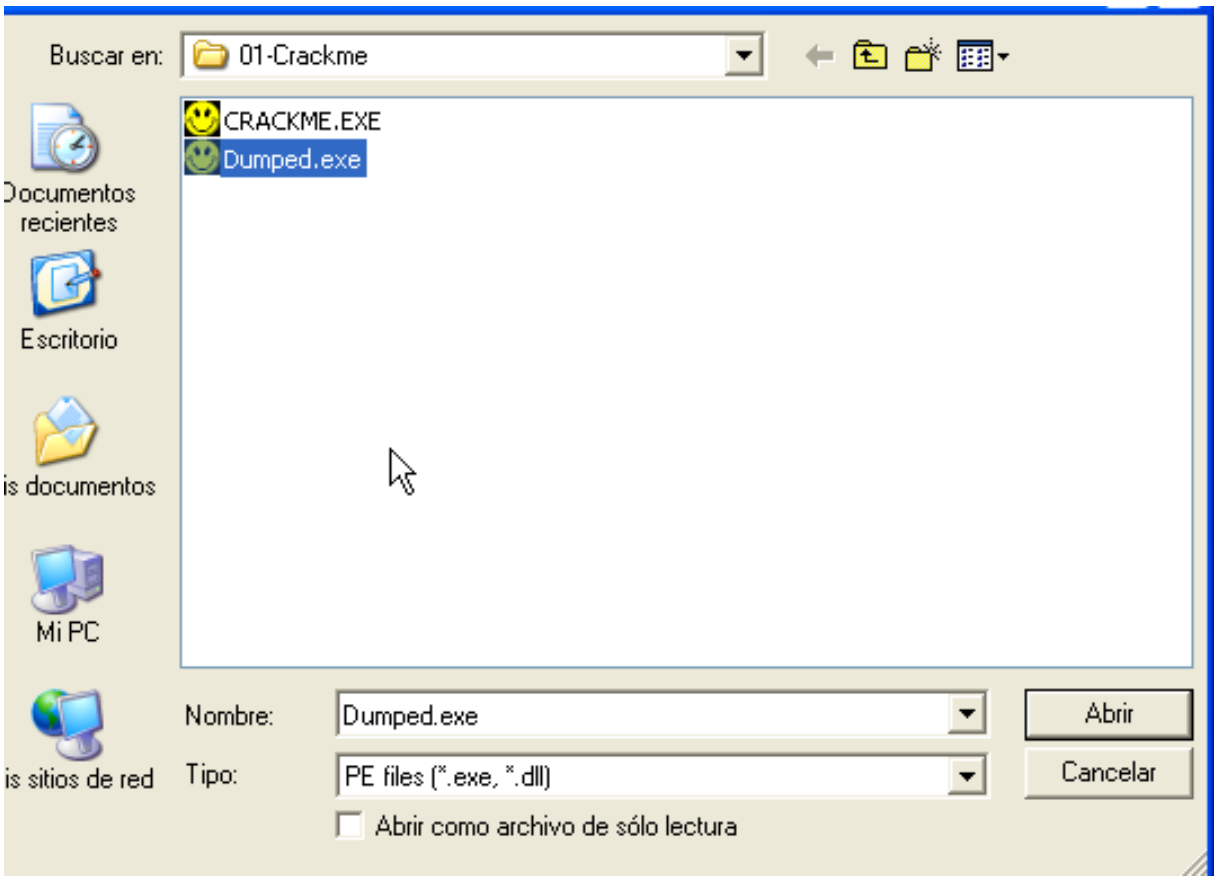
Если помните, это был первый элемент, который мы смотрели в IAT. Он соответствует 403238, но в IMP REC, конечно, все значения содержатся с вычтенной базой образа, поэтому ищем 3238. Кроме того, мы знаем, что он относится к Kernel32.dll.



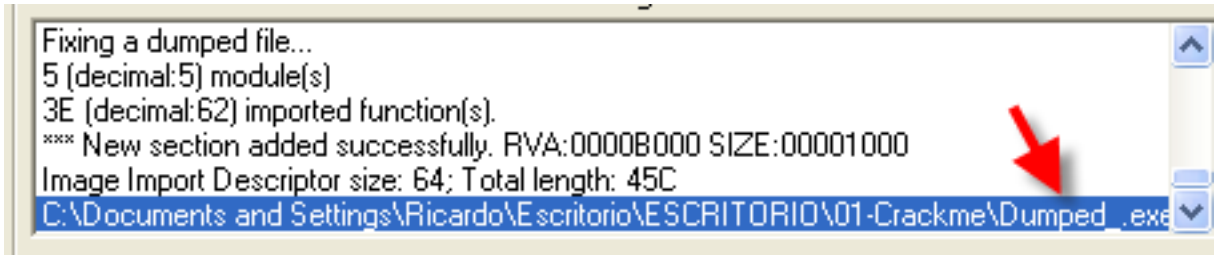
Поэтому мы должны открыть kernel32.dll, для чего сделаем щелчок мышью на «+» слева от названия библиотеки.



Видим, что элемент 3238 соответствует GetModuleHandleA. Так что сейчас починим дамп, идём к кнопку FIX DUMP.

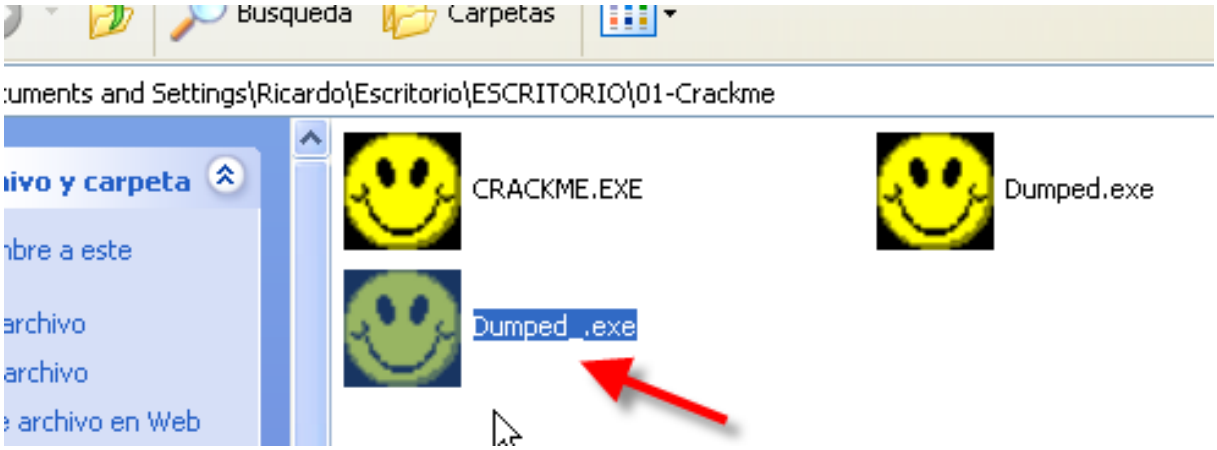


Ищем дамп и открываем его.

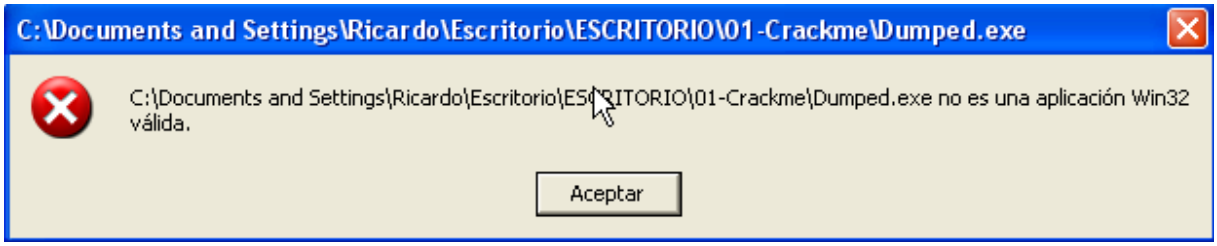


Ок, видим, что IMP REC починил его, хотя сохранил изменённый файл не под именем DUMPED, а под именем DUMPED_.exe.

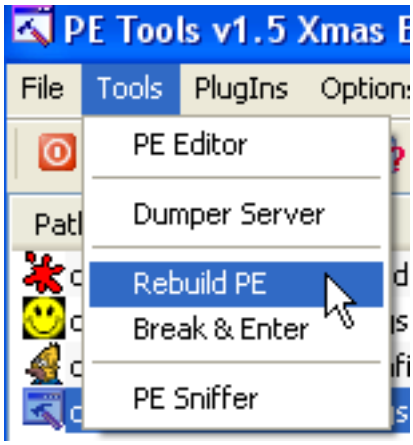
Смотрим, что находится в папке.



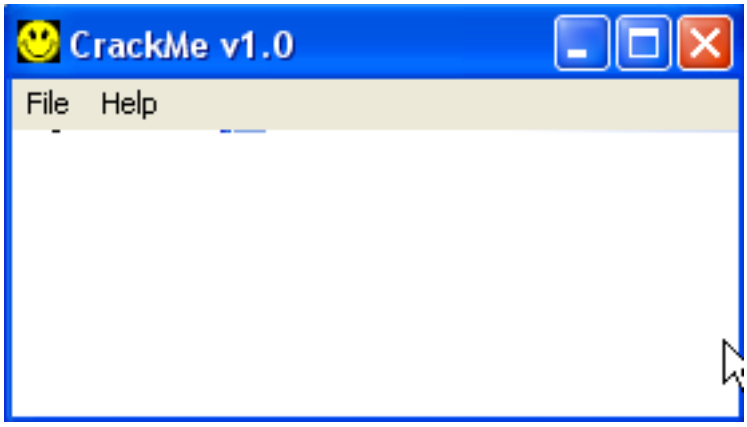
Можем попытаться запустить его, чтобы узнать, нормально ли он работает.



Хе-хехе, похоже, что не работает. Не пугайтесь, такое часто происходит после починки IAT. В PE TOOLS есть решение этой проблемы.

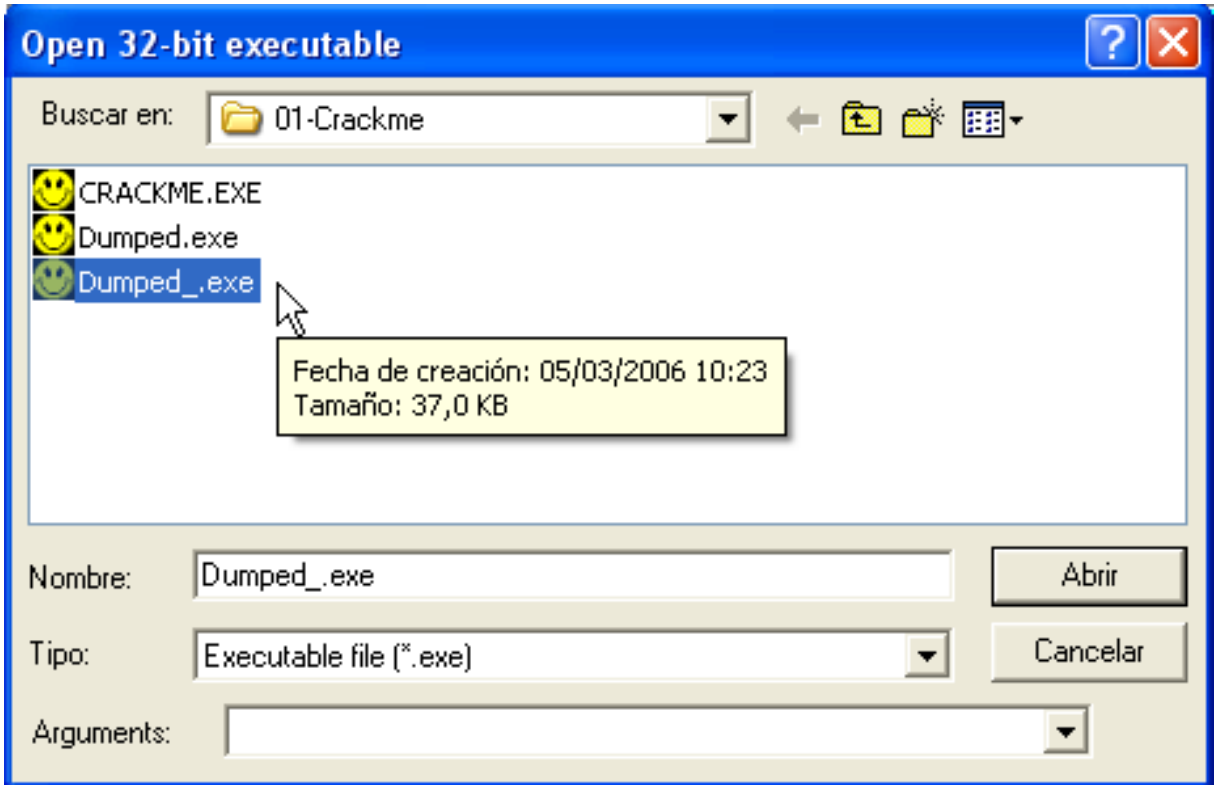


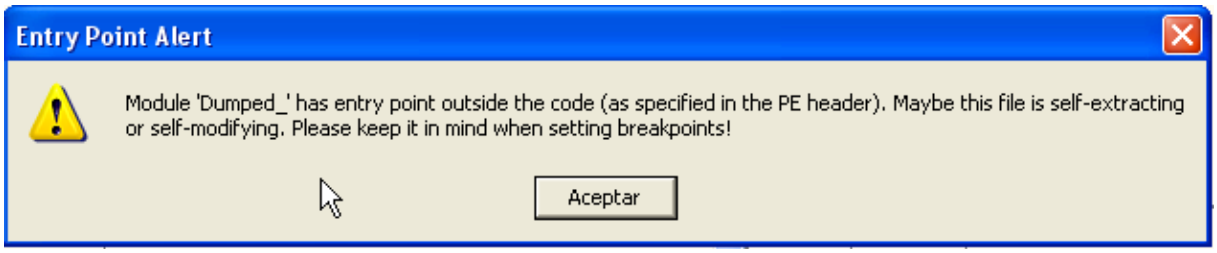
И идём в REBUILD PE, ищем DUMPED_.EXE, оно прекрасно срабатывает, а теперь запускаем его ииииии…



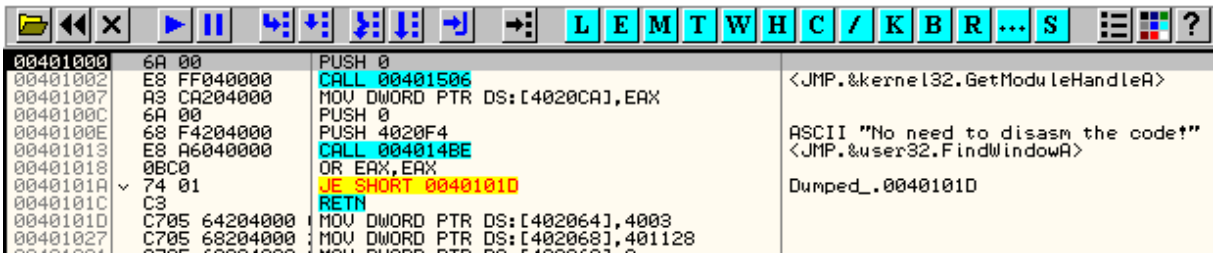
Рабооооотает! И теперь будет работать на любой машине, потому что IAT починен с помощью IMP REC, которые поместил правильные вызовы API-функций во все элементы IAT, заново записав туда имена API-фукций и приведя в порядк IT, чтобы она соответствовала тому, как должна быть, и что программа запускалась без ошибки.

Если откроем DUMPED_exe в OllyDbg.

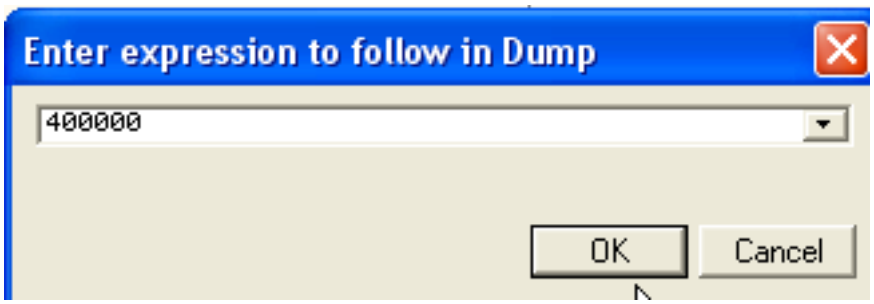


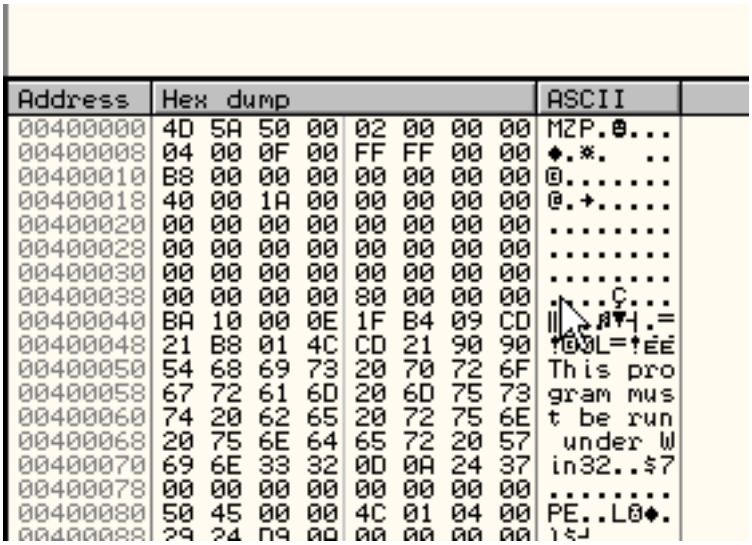


OllyDbg говорит нам, что точка входа была обнаружена за пределами секции кода, и это из-за того, что UPX поменял секцию кода на 3-ю. Данное положение вещей также можно исправить.

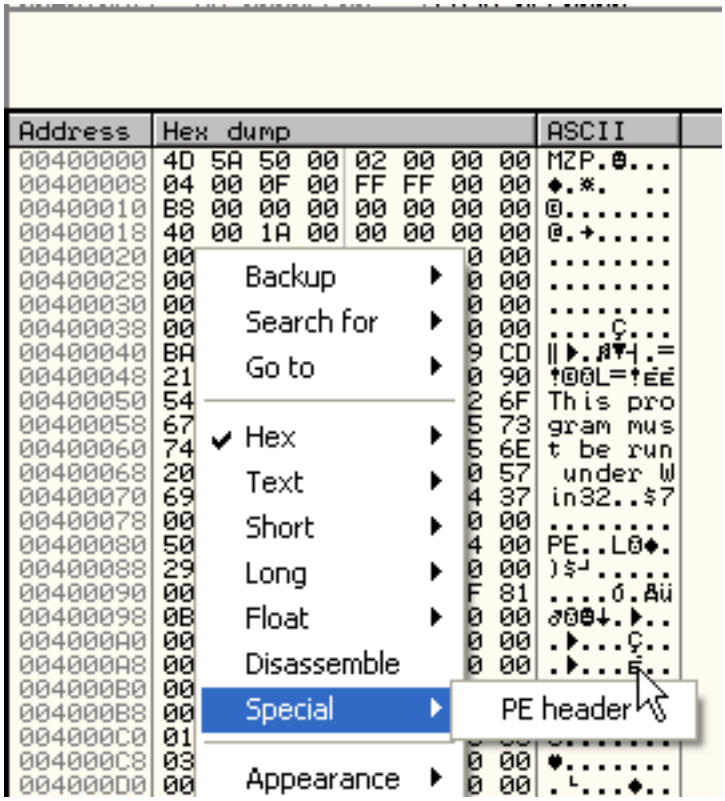


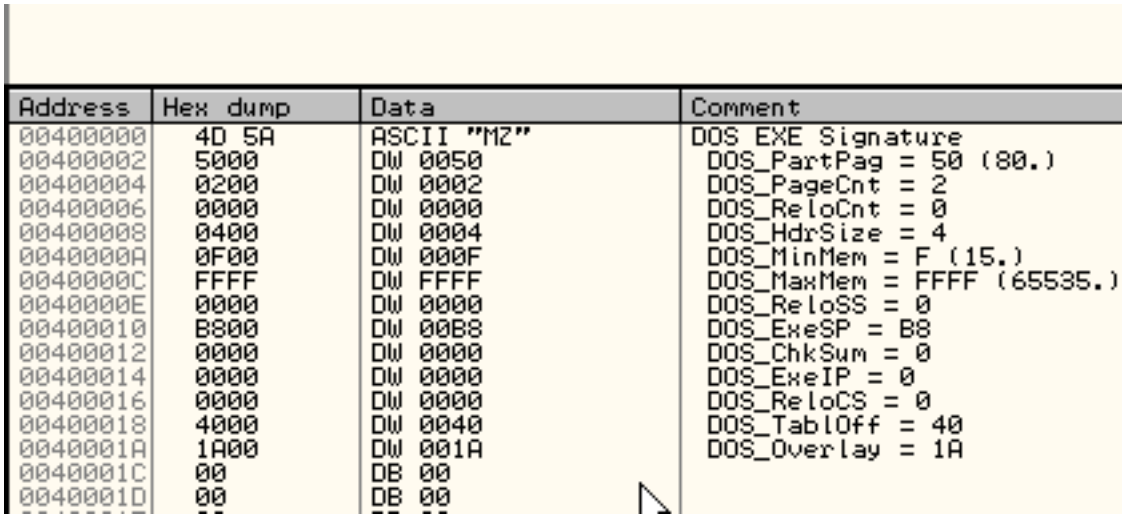
Прибываем в точку входа и идём смотреть заголовок с помощью GOTO EXPRESSION=400000.





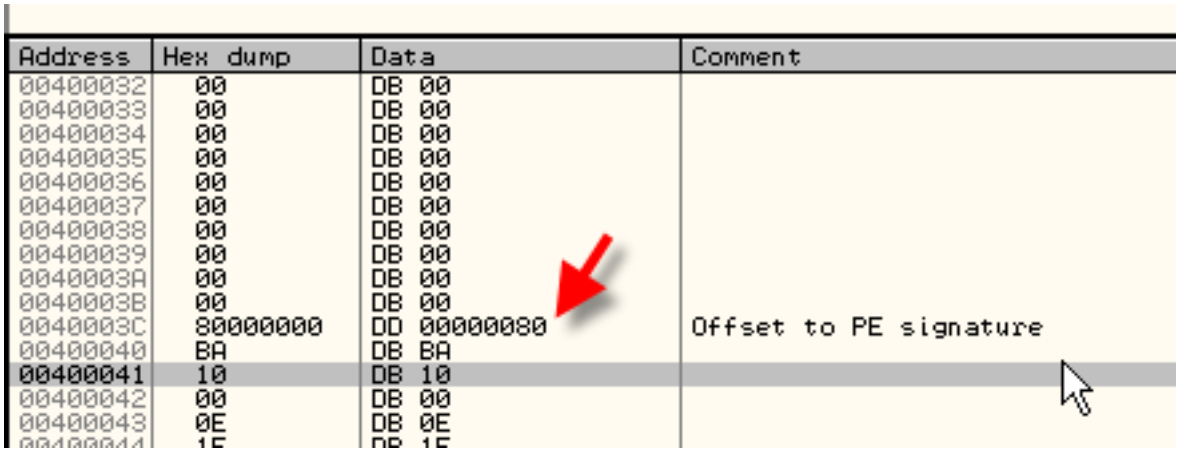
Меняем режим на SPECIAL-PE HEADER.





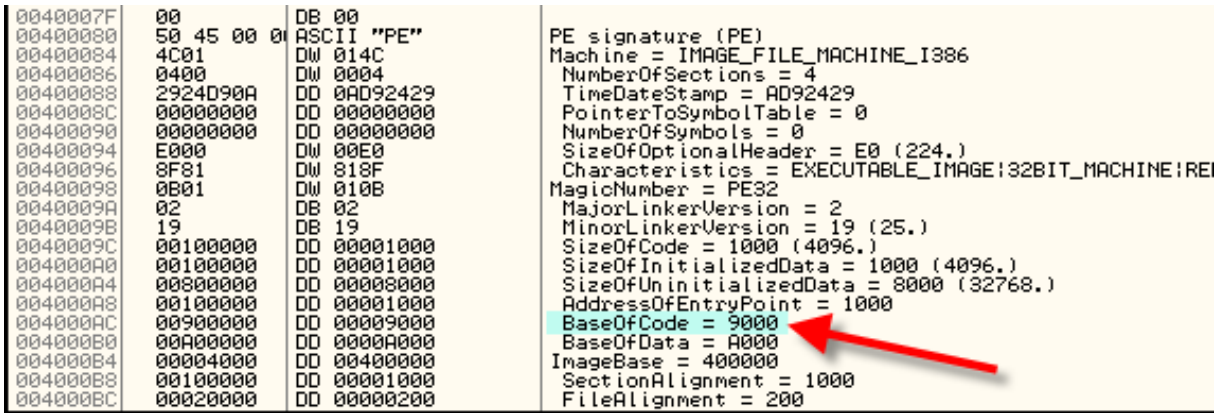
Address	Hex dump	Data	Comment
00400000	4D 5A	ASCII "MZ"	DOS EXE Signature
00400002	5000	DW 0050	DOS_PartPag = 50 (80.)
00400004	0200	DW 0002	DOS_PageCnt = 2
00400006	0000	DW 0000	DOS_ReloCnt = 0
00400008	0400	DW 0004	DOS_HdrSize = 4
0040000A	0F00	DW 000F	DOS_MinMem = F (15.)
0040000C	FFFF	DW FFFF	DOS_MaxMem = FFFF (65535.)
0040000E	0000	DW 0000	DOS_ReloSS = 0
00400010	B800	DW 00B8	DOS_ExeSP = B8
00400012	0000	DW 0000	DOS_ChkSum = 0
00400014	0000	DW 0000	DOS_ExeIP = 0
00400016	0000	DW 0000	DOS_ReloCS = 0
00400018	4000	DW 0040	DOS_TablOff = 40
0040001A	1A00	DW 001A	DOS_Overlay = 1A
0040001C	00	DB 00	
0040001D	00	DB 00	

Чуть ниже:



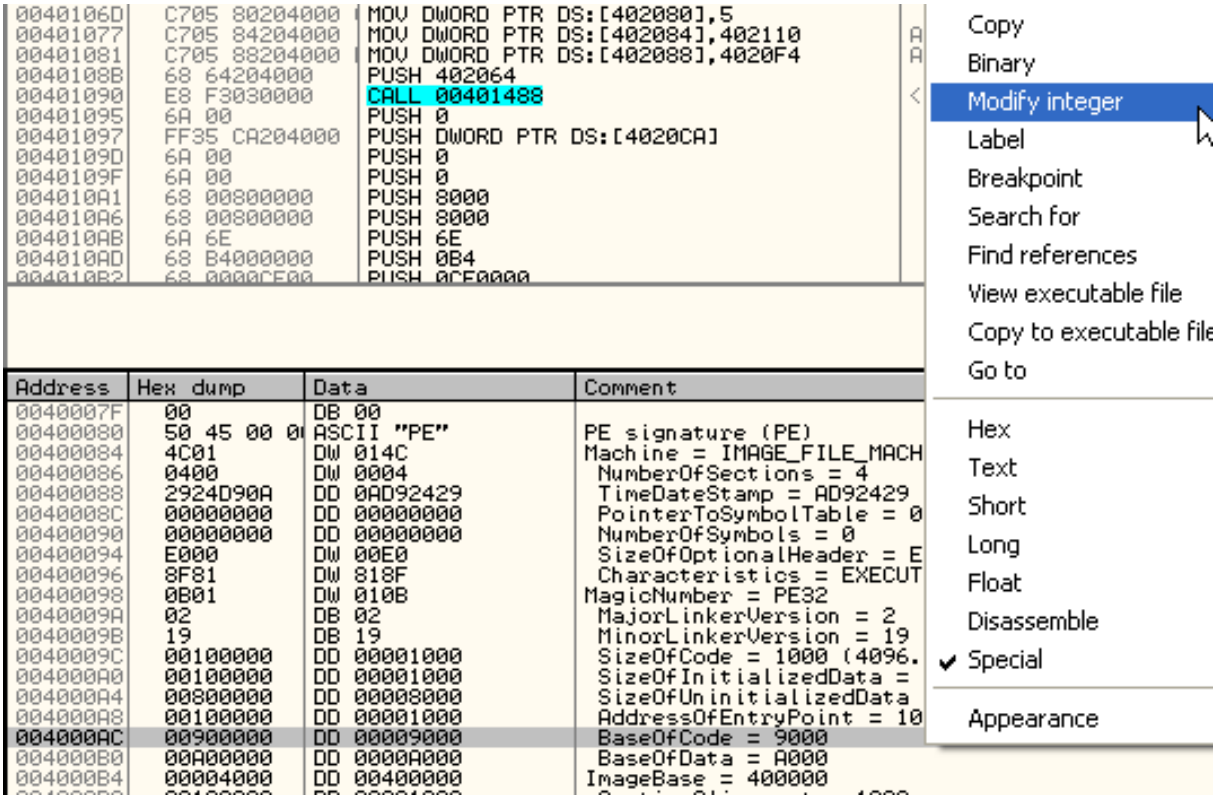
Address	Hex dump	Data	Comment
00400032	00	DB 00	
00400033	00	DB 00	
00400034	00	DB 00	
00400035	00	DB 00	
00400036	00	DB 00	
00400037	00	DB 00	
00400038	00	DB 00	
00400039	00	DB 00	
0040003A	00	DB 00	
0040003B	00	DB 00	
0040003C	80000000	DD 00000080	Offset to PE signature
00400040	BA	DB BA	
00400041	10	DB 10	
00400042	00	DB 00	
00400043	0E	DB 0E	

Видим, что сигнатура PE начинается в 80, идём туда (400080).

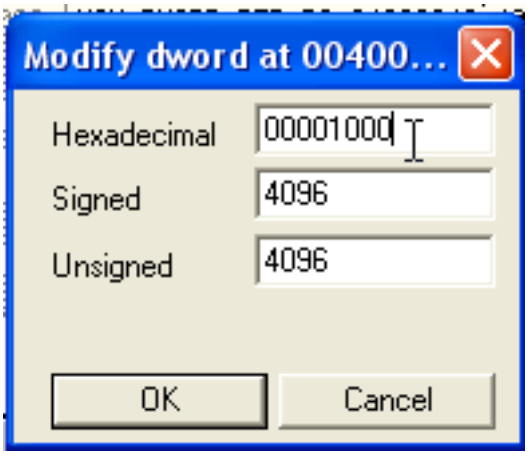


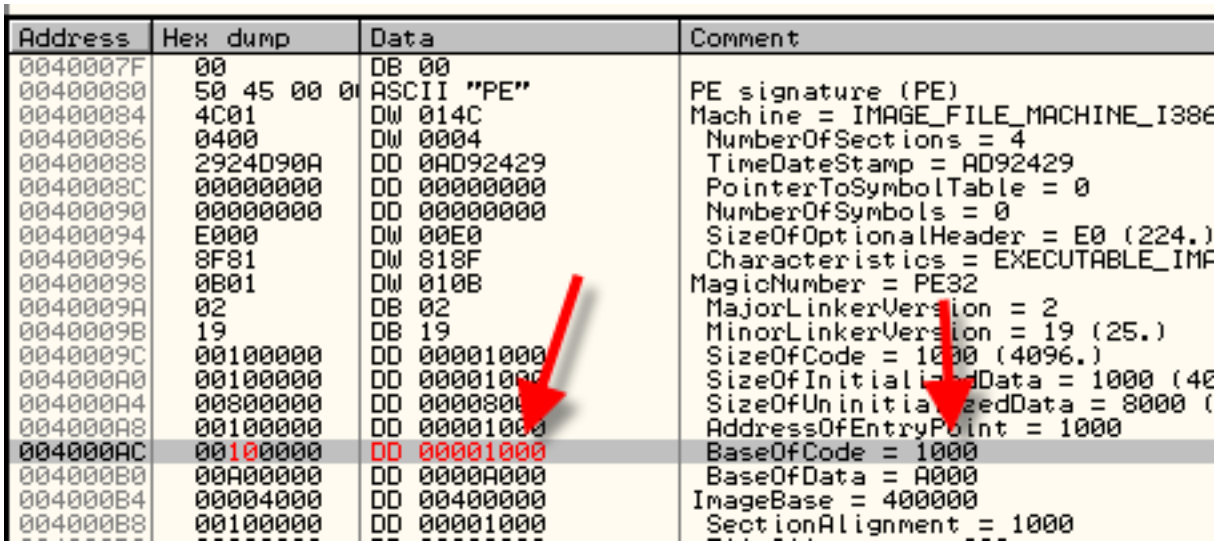
0040007F	00	DB 00	
00400080	50 45 00 0(	ASCII "PE"	PE signature (PE)
00400084	4C01	DW 014C	Machine = IMAGE_FILE_MACHINE_I386
00400086	0400	DW 0004	NumberOfSections = 4
00400088	2924D90A	DD 0AD92429	TimeDateStamp = AD92429
0040008C	00000000	DD 00000000	PointerToSymbolTable = 0
00400090	00000000	DD 00000000	NumberOfSymbols = 0
00400094	E000	DW 00E0	SizeOfOptionalHeader = E0 (224.)
00400096	8F81	DW 818F	Characteristics = EXECUTABLE_IMAGE!32BIT_MACHINE!REI
00400098	0B01	DW 010B	MagicNumber = PE32
0040009A	02	DB 02	MajorLinkerVersion = 2
0040009B	19	DB 19	MinorLinkerVersion = 19 (25.)
0040009C	00100000	DD 00001000	SizeOfCode = 1000 (4096.)
004000A0	00100000	DD 00001000	SizeOfInitializedData = 1000 (4096.)
004000A4	00800000	DD 00008000	SizeOfUninitializedData = 8000 (32768.)
004000A8	00100000	DD 00001000	AddressOfEntryPoint = 1000
004000AC	00900000	DD 00009000	BaseOfCode = 9000
004000B0	00A00000	DD 0000A000	BaseOfData = A000
004000B4	00004000	DD 00400000	ImageBase = 400000
004000B8	00100000	DD 00001000	SectionAlignment = 1000
004000BC	00020000	DD 00000200	FileAlignment = 200

Здесь находится этот негодный указатель, означающий BASE OF CODE=9000, а мы хотим, чтобы первая секция была секцией кода, которая начинается в 401000, так что меняем 9000 на 1000.

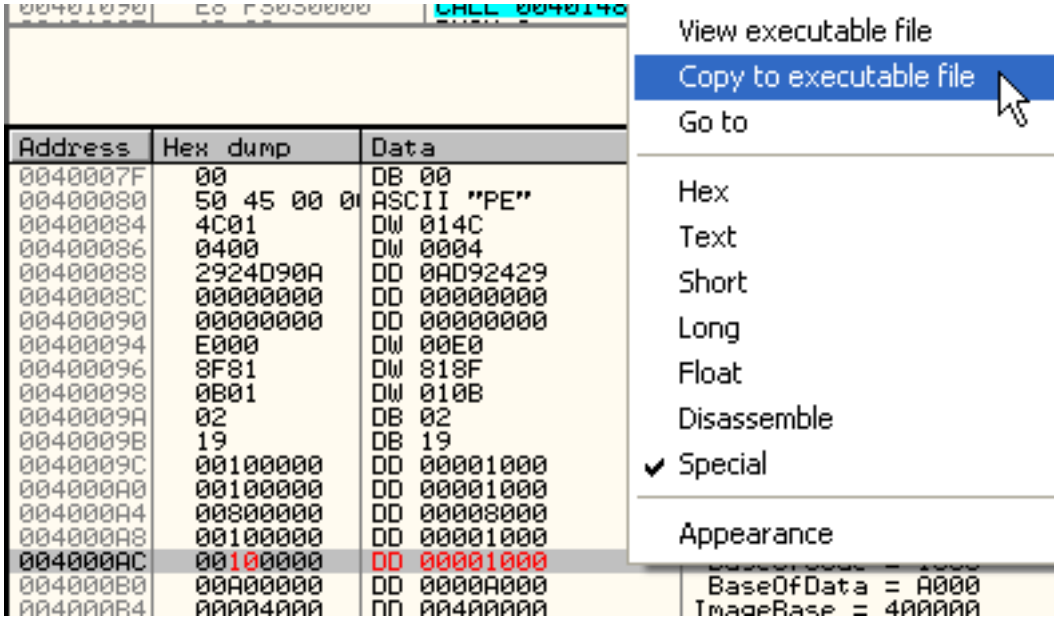


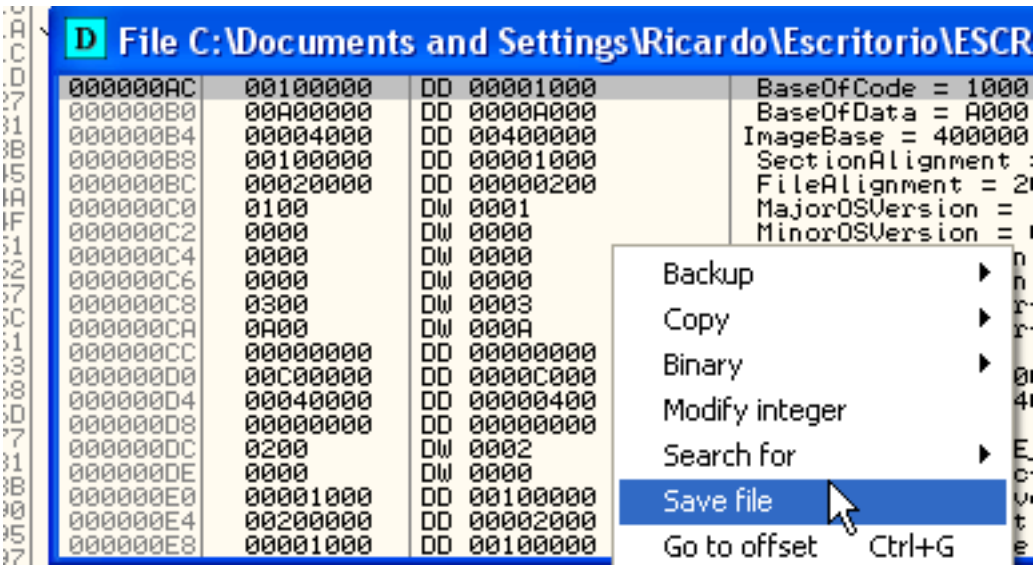
Отмечаем и идём в MODIFY INTEGER.

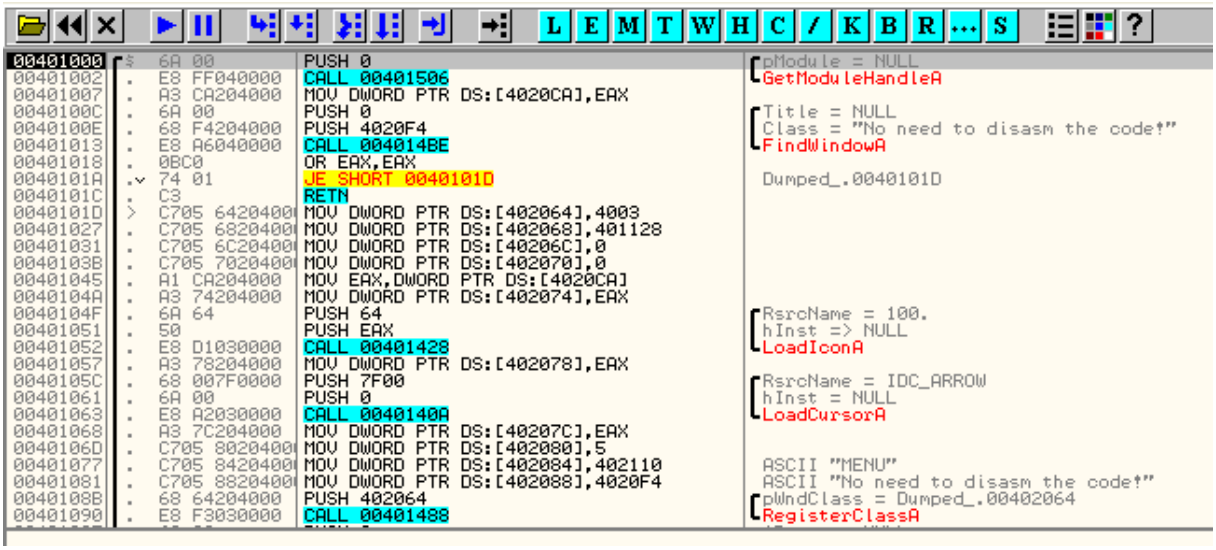




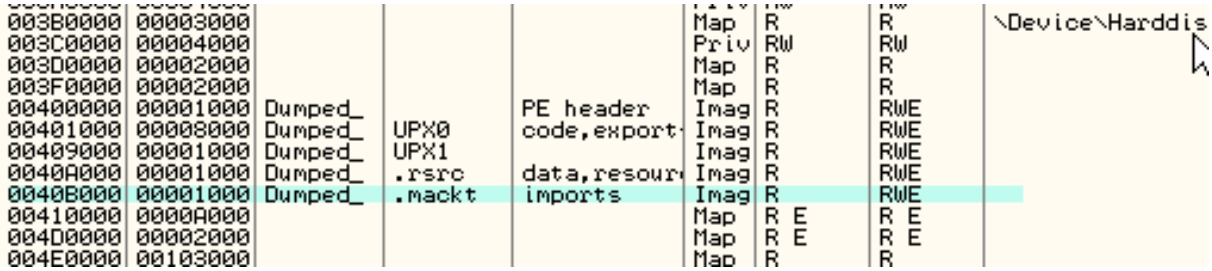
Теперь сохраняем изменения как обычно с помощью щелчка правой кнопки мыши – COPY TO EXECUTABLE, в открывшемся окне опять правый щелчок и SAVE FILE.



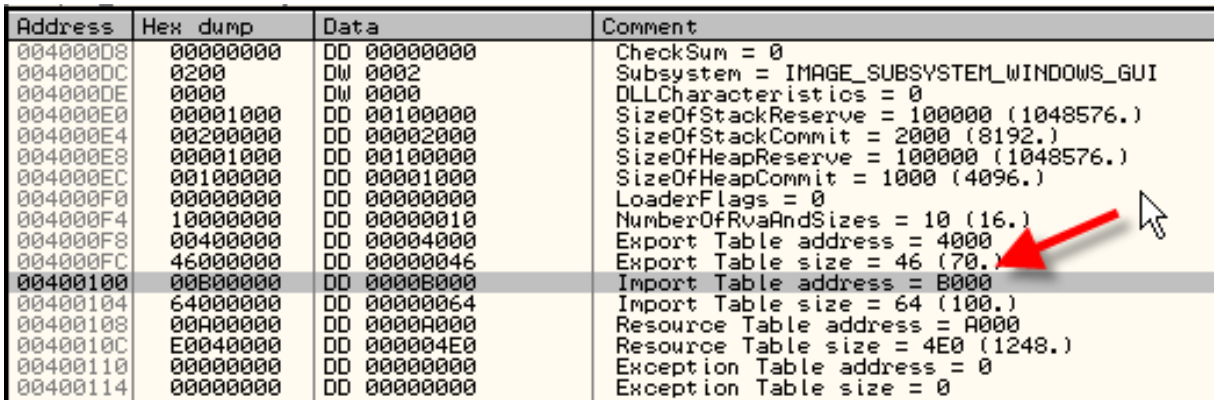




Теперь рестартуем крэкми в OllyDbg и увидим, что назойливое окошко исчезло, так как OllyDbg больше не считает ту секцию секцией кода.



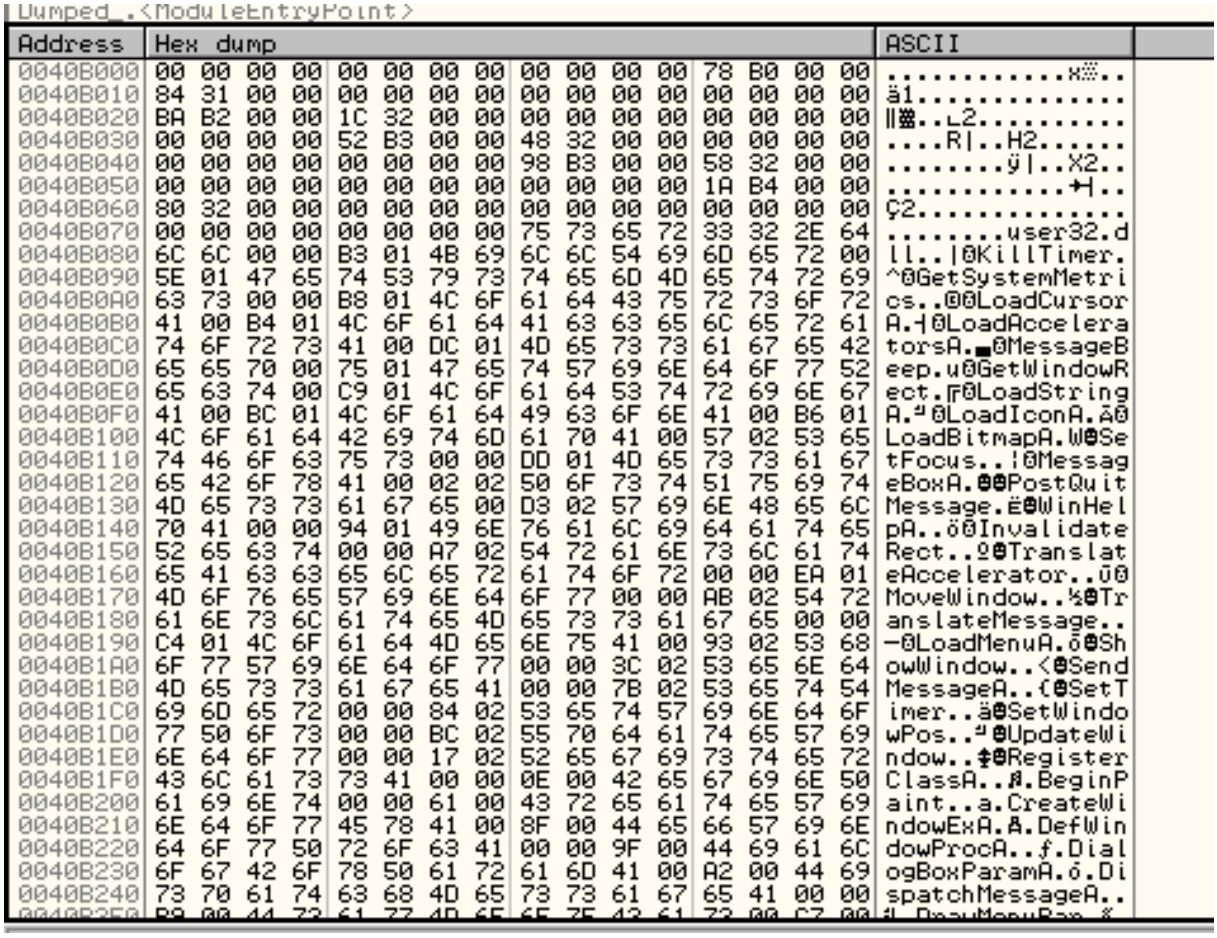
Если посмотрим секции сдампленного файла, то увидим, что для его починки IMP REC добавил новую секцию, названную mackt, где находится новая IT. Проверим её. В том же заголовки в режиме SPECIAL посмотрим указатель на IT.



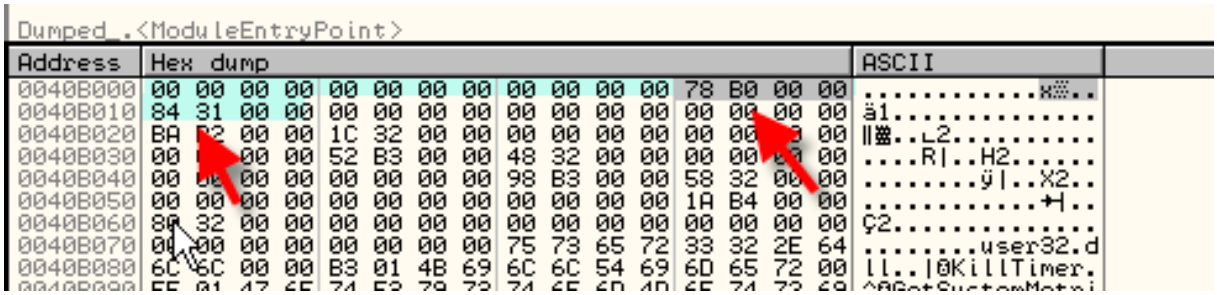
Vemos que la IT esta ahora en B000 o sea 40B000 que es la direccion de la seccion que agrego el IMP REC vayamos a ver alli quitando el modo SPECIAL.

Видим, что IT сейчас находится в B000, то есть в 40B000, являющимся адресом секции, которую добавил IMP REC. Идём туда, выходя из режима SPECIAL.

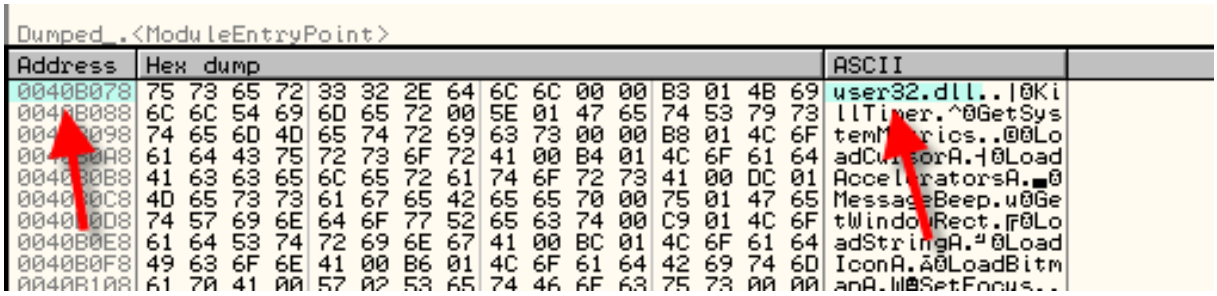
Видим, что IT находится в B000, то есть 40B000, что является адресом секции, которая добавляет IMP REC. Идём посмотреть, что там, выходя из режима SPECIAL.



Здесь видим такую же IT, как и ту, что мы видели в предыдущей главе, указывающую на другой адрес, но полностью рабочую, и если хотим, то можем найти указатели таким же образом, как в прошлой главе.

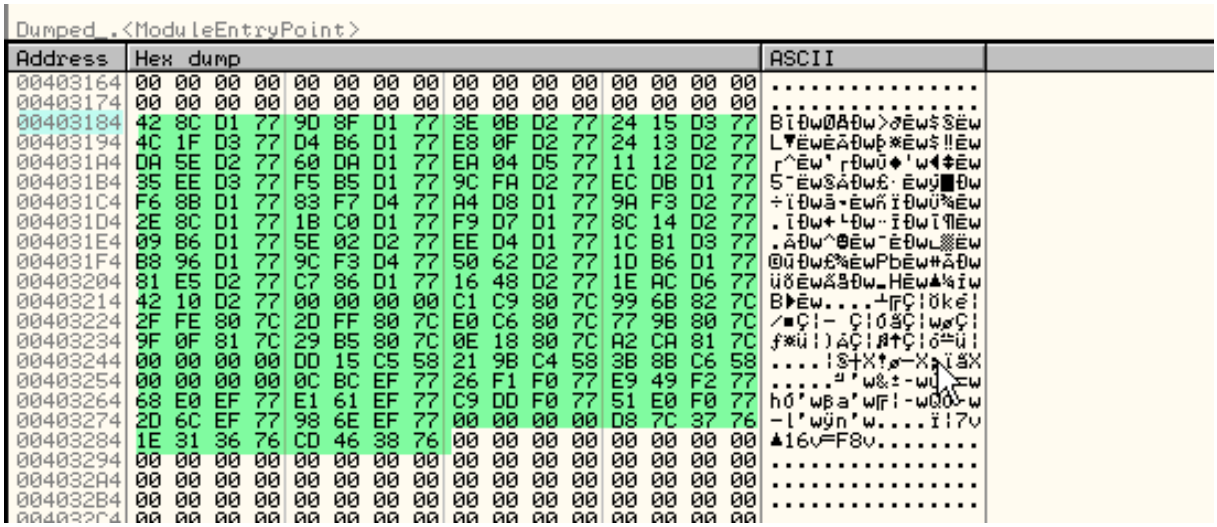


Здесь у нас первый IID, относящийся к первой DLL, и его 4-ый DWORD указывает на имя DLL, и в данном случае имя этой DLL находится по адресу B078, то есть 40B078.

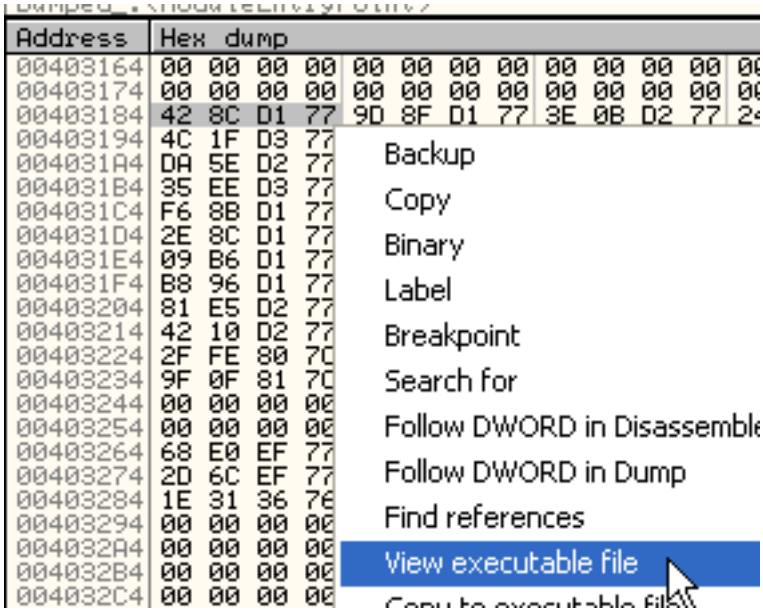


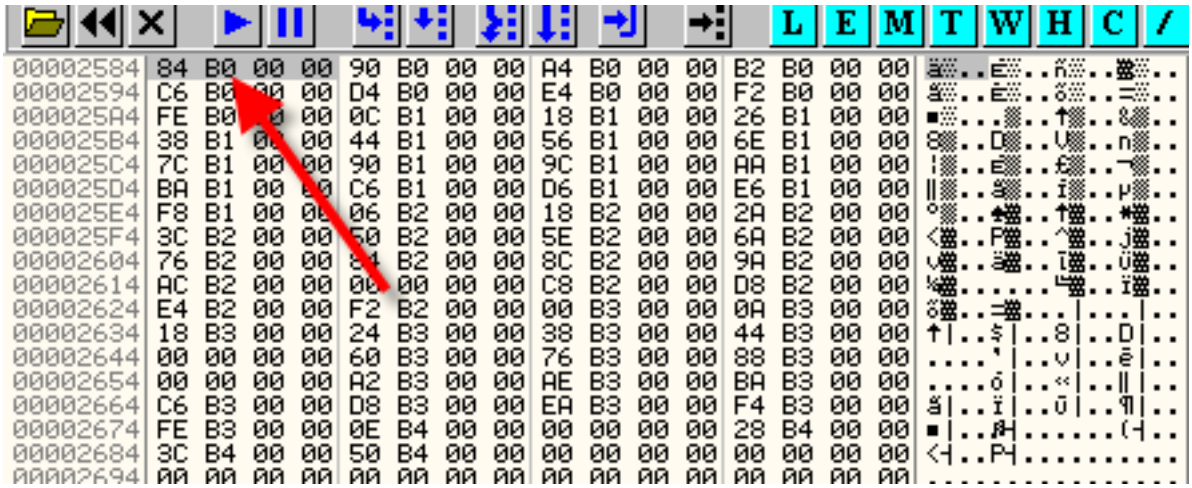
Это user32.dll.

И 5-ый указатель, как мы знаем из предыдущей главы, - это 3184, поэтому первый элемент реконструированной IAT находится в 403184, т.е. её начало.

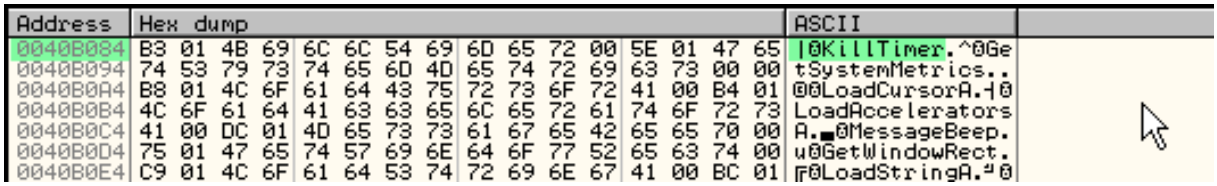


И, конечно, первый элемент IAT у нас сейчас содержит значение API-функции, но если посмотрим в исполняемый файл, то до того, как быть перезаписанным адресом API-функции на моей машине, он должен был содержать указатель на имя API-функции.





В этом элементе IAT в исполняемом файле есть значение B084, которое соответствует 40B084, где должно находится имя API-функции, чей адрес будет помещён в этот элемент.



И это KillTimer.

Видим, что IMP REC провёл изумительную работу, обнаружил все API-функции, сконструировал новую IT, в которую добавил все указатели, и реконструировал список имён всех API-функций, чтобы система знала при запуске программы, какому элементу IAT какая функция соответствует. Настоящее чудо.

Хорошо, это было наше первое восстановление IAT, самое лёгкое, какое можно было придумать, но это база, на которой основывается всё остальное, и важно, что изложенное в этой и предыдущих главах было понятно, потому что в дальнейшем мы встретим IAT'ы, которые полностью растерзаны, переадресованные API-функции и другие сложные случаи, поэтому необходимо хорошо понимать, как это всё работает.

Кроме того, этот упаковщик не применяет противодамповых приёмов, поэтому мы их не рассматривали, но в дальнейшем, когда мы погрузимся в сложность упаковщиков, мы встретим случаи, где такие приёмы применяются.

Таким образом, мы мягко повышаем сложность, чтобы новые концепции не испугали вас, идём шаг за шагом, хе-хе-хе.

Введение в крэкинг с нуля, используя OllyDbg - Глава 35

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-11-10

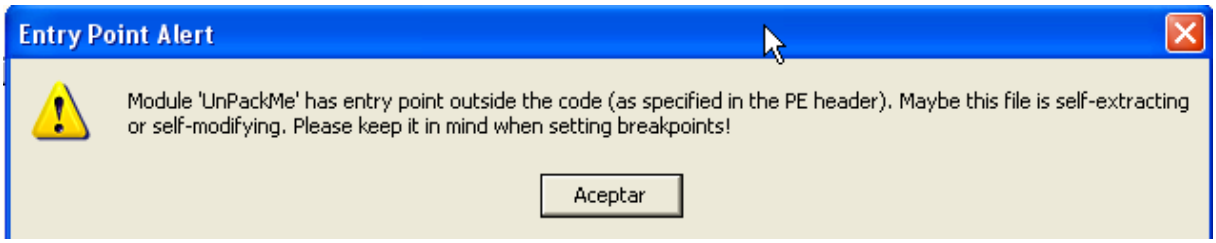
Продолжаем практиковаться в распаковке, каждый раз используя более трудный упаковщик, постепенно повышая тем самым уровень сложности.

Следующим упаковщиком на шкале сложности является aspack, очень похожий на UPX, и для которого у нас есть крэкми UnPackMe_ASPack2.12.exe, использовавшееся в главе 32.

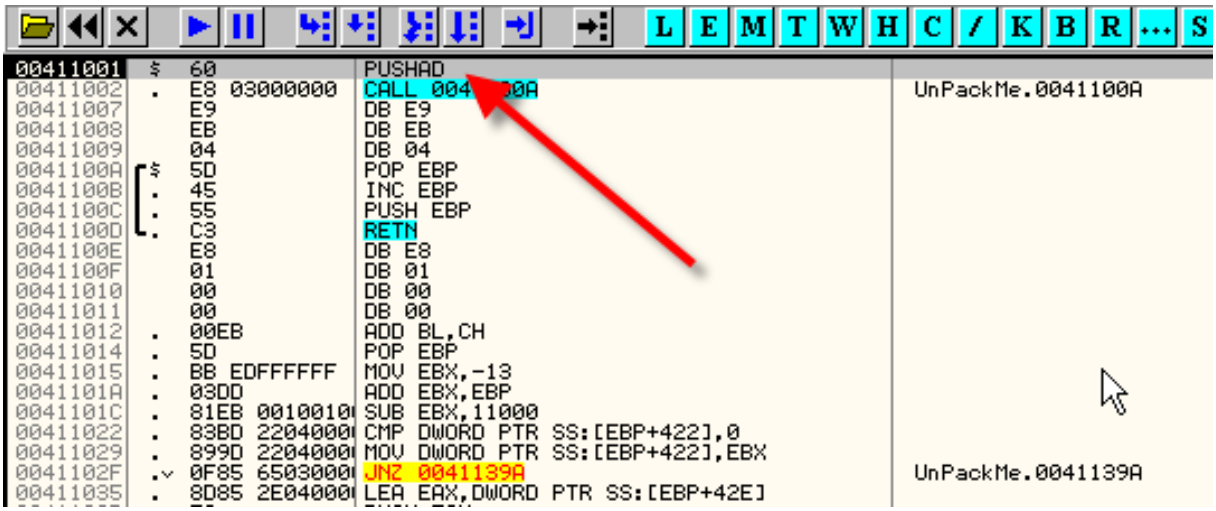
Для практики установим OllyDump в директорию с плагинами, чтобы сделать дамп с его помощью.

Здесь была ссылка, но она устарела. Возможно, поможет вот эта.

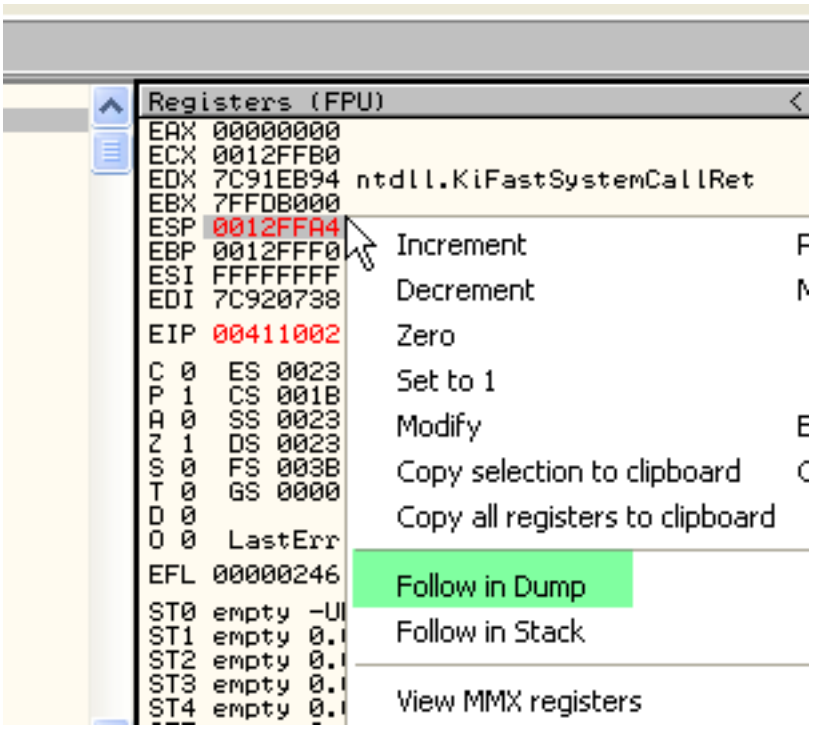
Открываем OllyDbg, защищённый плагинами от обнаружения, и находим OEP с помощью метода PUSHAD.



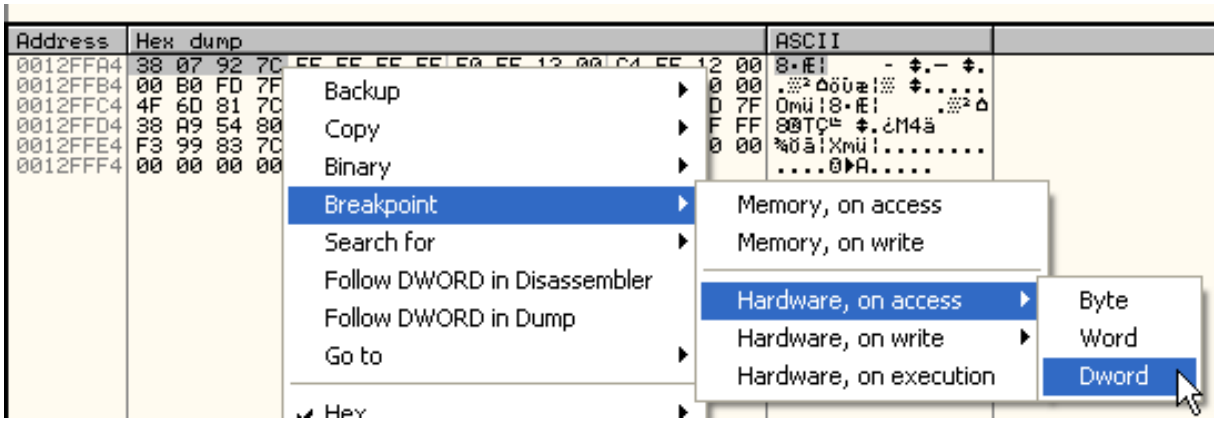
Видим, что здесь нам сообщают о том, что точка входа находится за пределами секции кода, как это обычно бывает с большинством упаковщиков.



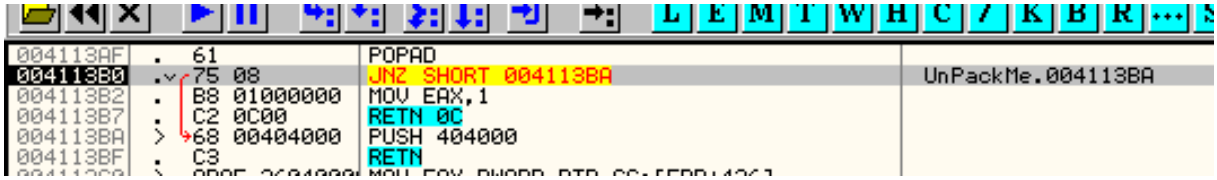
Видим начальный PUSHAD, который проходим с помощью F7, а затем делаем



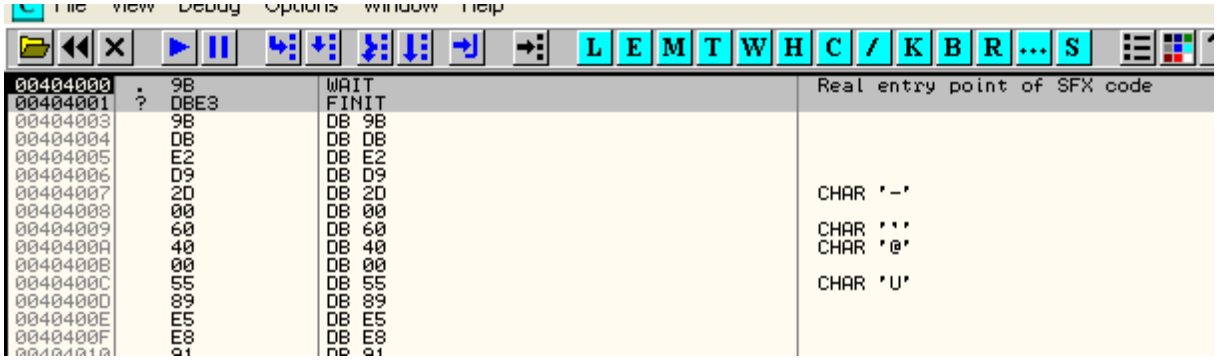
ESP-FOLLOW IN DUMP, чтобы установить в DUMP'е железную (hardware) точку останова на доступ к значениям двух регистров, которые сохраняются инструкцией PUSHAD.



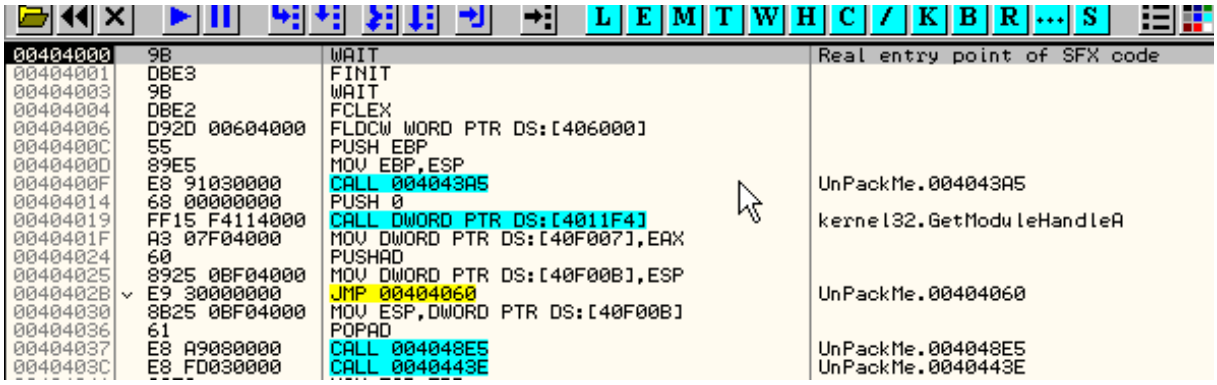
Затем нажимаем F9.



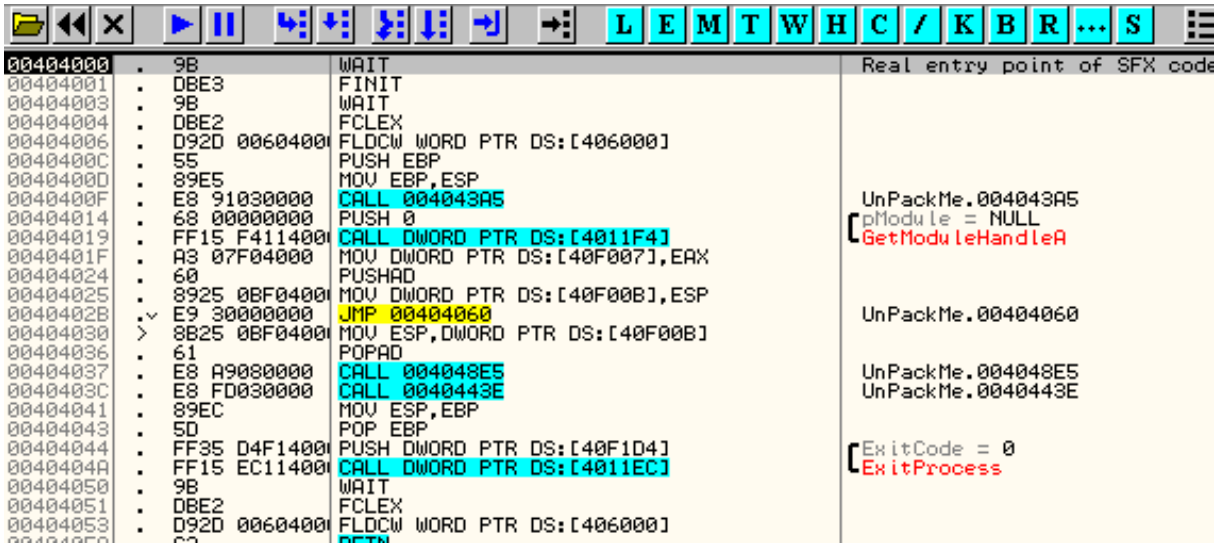
И останавливаемся прямо после POPAD, который восстанавливает сохранённые в эти регистры значения. Трассируем с помощью F7, пока не прибудем в OEP.



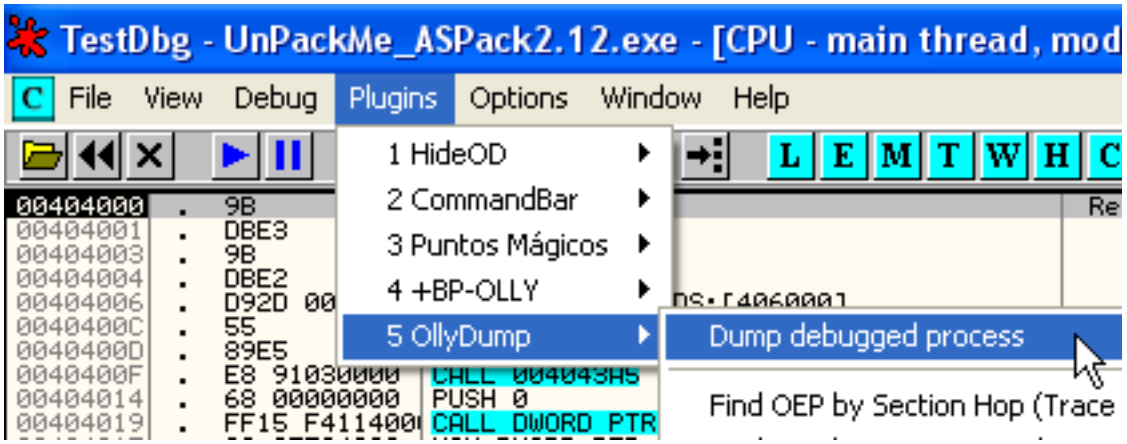
Как видим, код был не понят, выходим из анализа.

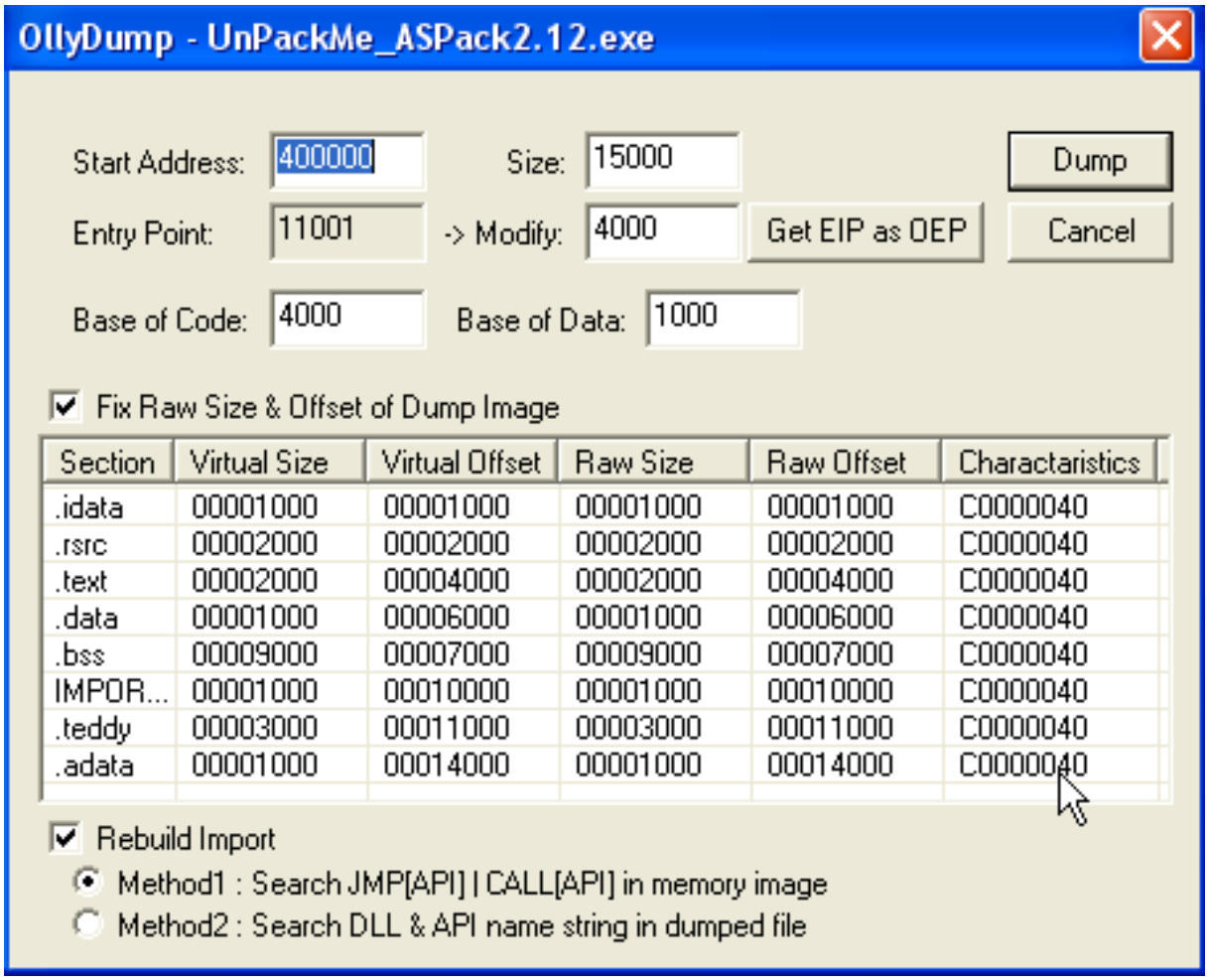


И видим, что если проанализировать его ещё раз, всё становится гораздо лучше.



Затем приступаем к дампу файла. Идём в меню PLUGINS и ищем там OllyDump.

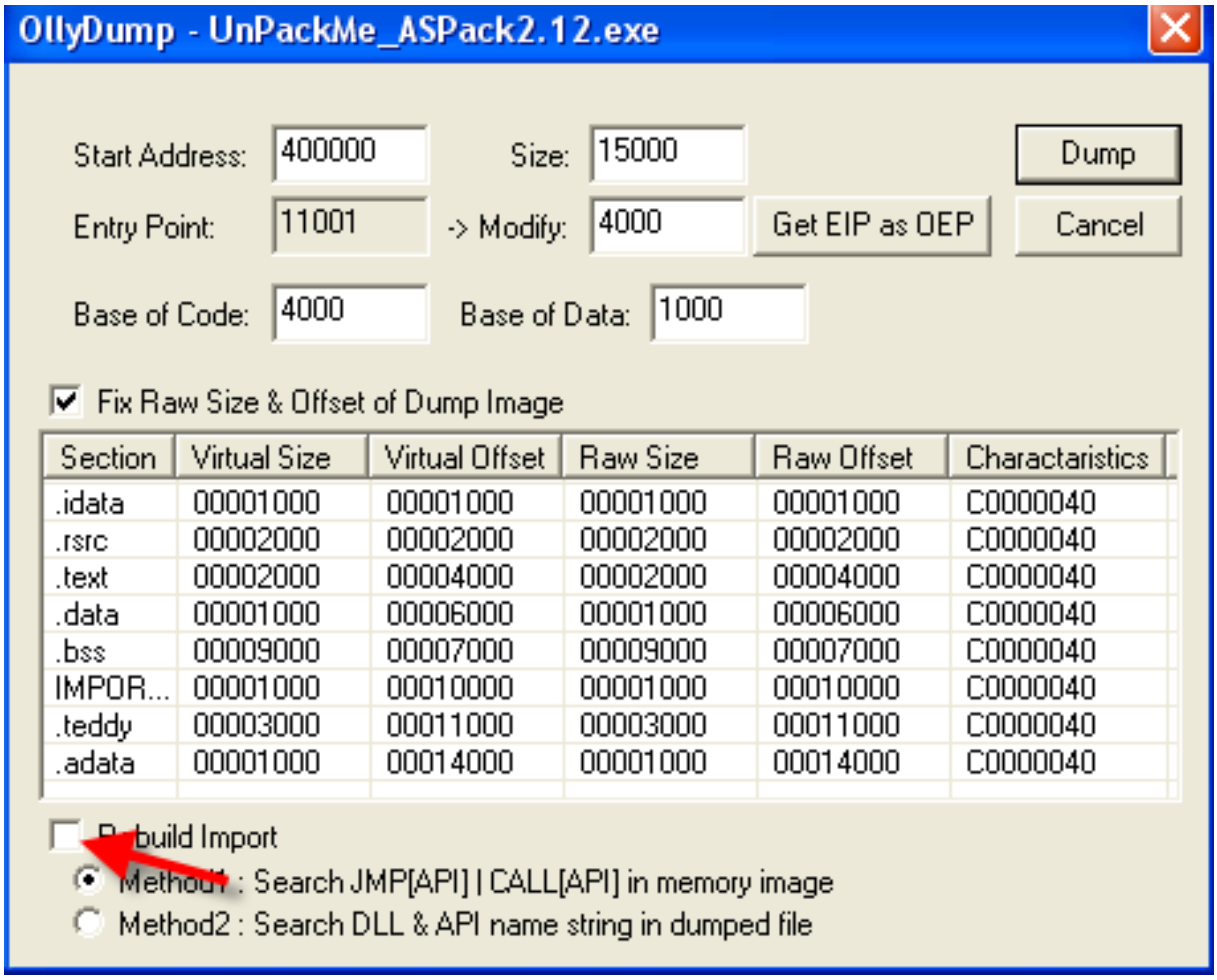




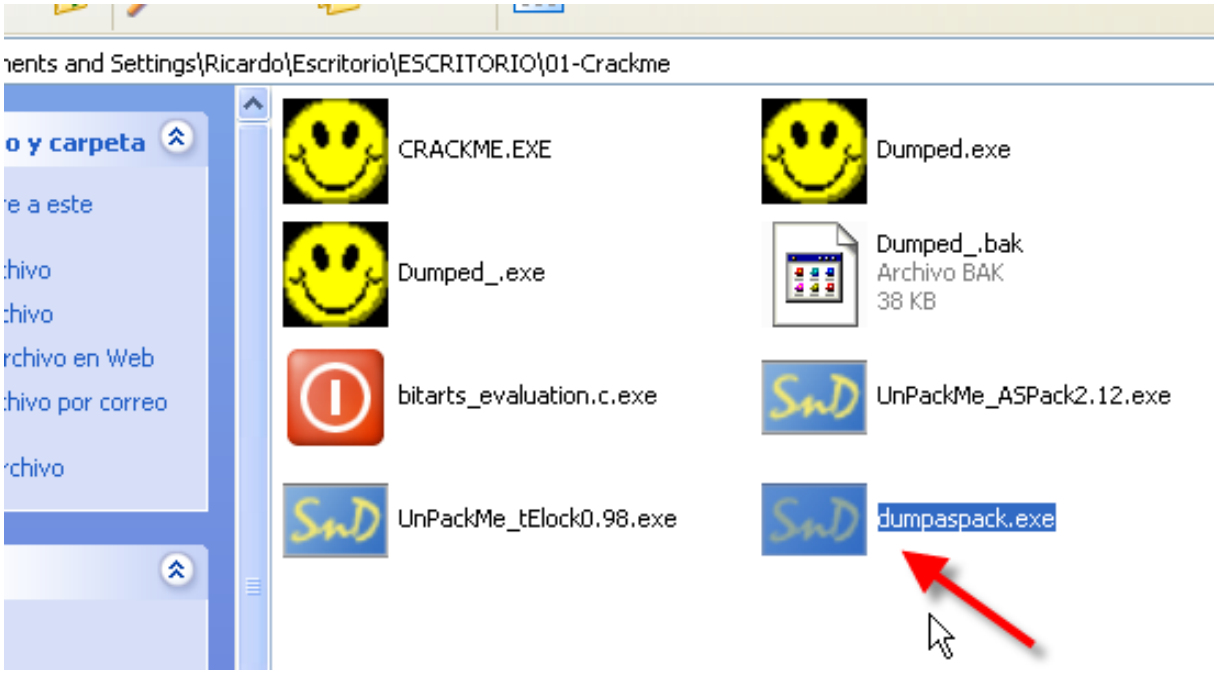
Нам открывается окно плагина, в котором уже отображено то, что можно модифицировать. Можно исправить базу кода без необходимости потом править её в заголовке. В окне видим, что база кода равна 4000, а если помним, то aspack выполняется не в первой секции, поэтому не меняем это значение, которое соответствует 404000, где находится OEP и выполняется программа.

Ещё одна важная фишка – это опция «REBUILD IMPORT», находящаяся внизу окна. OllyDbg пытается сделать работу IMP REC'a, для чего ему служат METHOD1 и METHOD2, которые могут работать для простых упаковщиков. Вы можете сделать два дампа, используя каждый из этих методов, и посмотреть, работает ли хотя бы один из них. Это не всегда срабатывает, но иногда может.

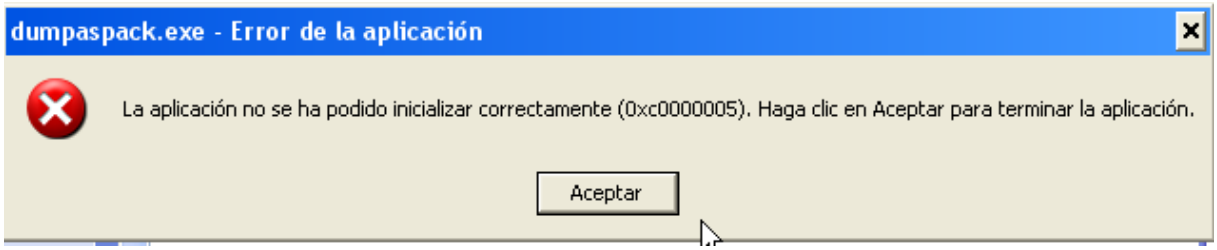
Мы снимаем галочку с «REBUILD IMPORT», так как мы будем использовать IMP REC для большей надёжности.



Ок, сейчас сделаем дамп и посмотрим, что из этого выйдет.



Вот дамп, если запустим его без починки IAT, то может запуститься без проблем, но только на моей машине, а может выдать ошибку. Посмотрим.

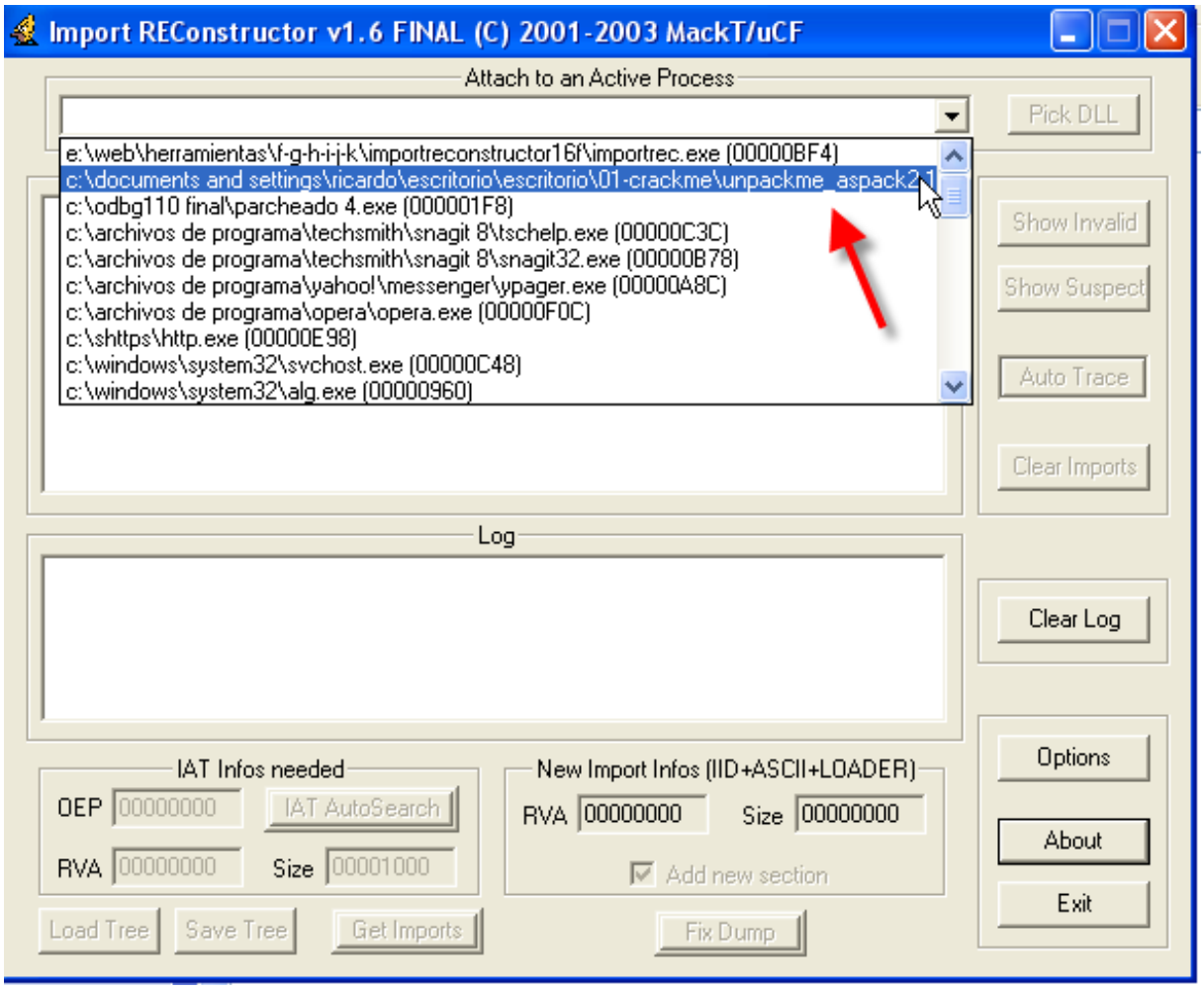


Ок, не закрывая упакованный архив, который остановлен на точке входа, открываем IMP REC и выбираем вышеуказанный процесс в выпадающем меню.

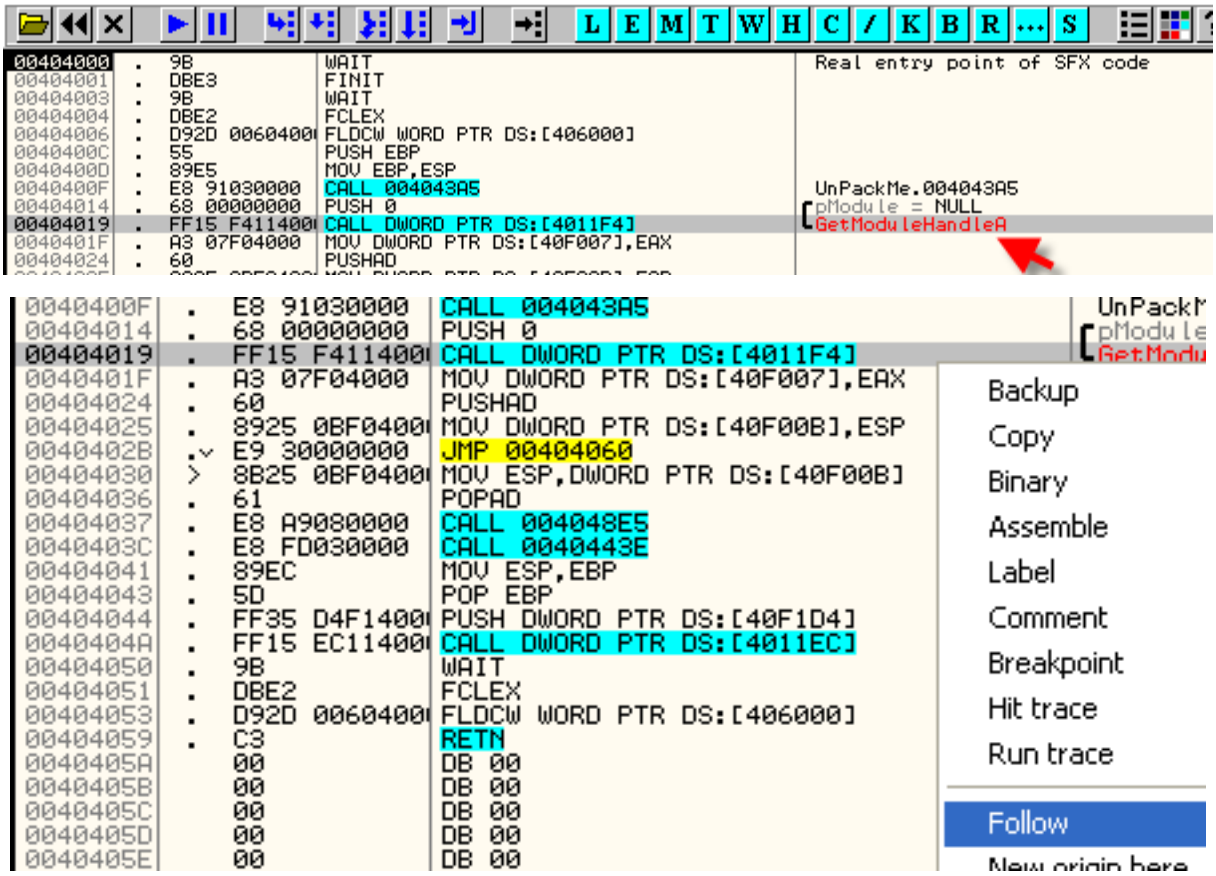
Возвращаемся в OllyDbg, чтобы найти значения начала IAT, её размера и OEP.

OEP равна 404000, то есть в IMP REC это будет 4000, так как нужно отнять базу образа, равную 400000.

Ищем начало и конец IAT, для этого нам нужно найти вызов какой-нибудь API-функции. Прямо под OEP есть вызов GetModuleHandleA.



Отмечаем данную строку, затем правая кнопка мыши и FOLLOW.

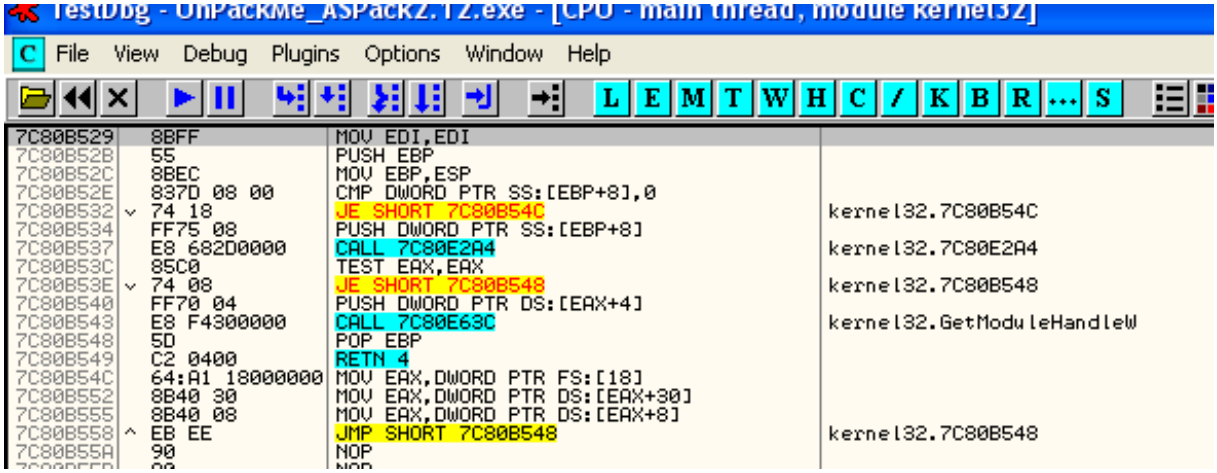


Видим, что идём напрямую на API-функцию без посредничества косвенного JMP, по крайней мере в этом вызове (хотя если поищем, то увидим, что косвенные переходы есть, значит, используются они не всегда, как в данном случае).

Это значит, что здесь используется косвенный вызов для перехода на функцию, поэтому выяснить, откуда считывается адрес API (а он считывается из IAT), легко.

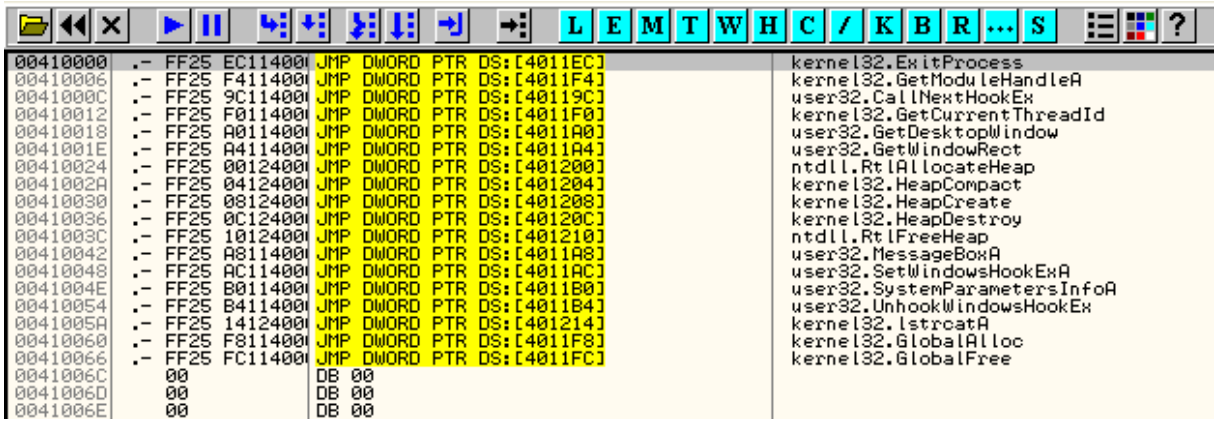
Легко видеть, что 4011F4 – это элемент IAT, где сохраняется адрес API-функции GetModuleHandleA.

Те, кто хочет увидеть косвенные переходы, могут поискать FF 25, чтобы их найти.



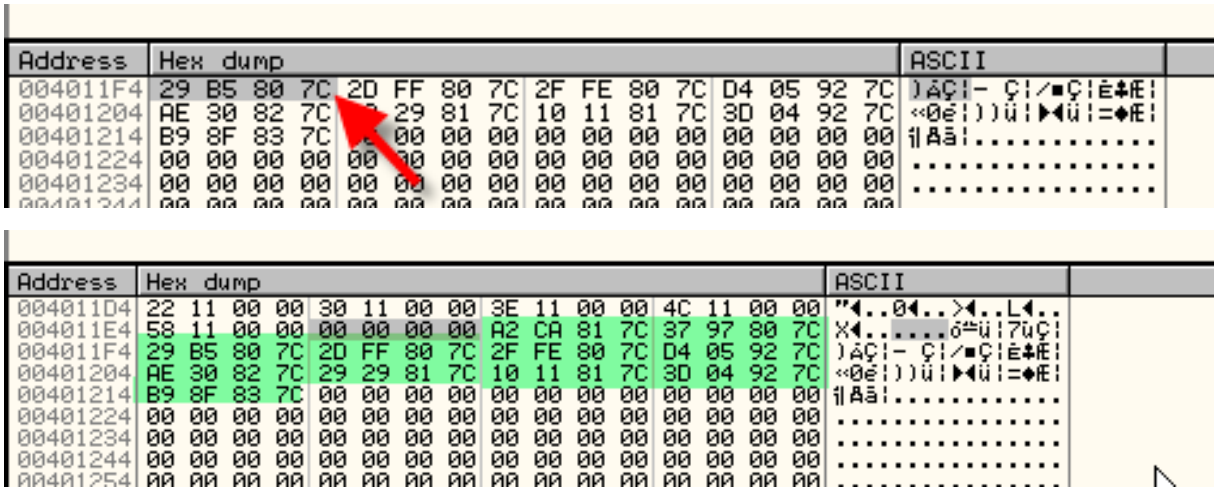
И получаем тот же результат, так как JMP к GetModuleHandleA читает значения из того же элемента IAT.

Идём в DUMP, чтобы посмотреть данный элемент в частности и IAT в общем.



```
00410000  .- FF25 EC114000  JMP DWORD PTR DS:[4011EC]   kernel32.ExitProcess
00410006  .- FF25 F4114000  JMP DWORD PTR DS:[4011F4]   kernel32.GetModuleHandleA
0041000C  .- FF25 9C114000  JMP DWORD PTR DS:[40119C]   user32.CallNextHookEx
00410012  .- FF25 F0114000  JMP DWORD PTR DS:[4011F0]   kernel32.GetCurrentThreadId
00410018  .- FF25 A0114000  JMP DWORD PTR DS:[4011A0]   user32.GetDesktopWindow
0041001E  .- FF25 A4114000  JMP DWORD PTR DS:[4011A4]   user32.GetWindowRect
00410024  .- FF25 00124000  JMP DWORD PTR DS:[401200]   ntdll.RtlAllocateHeap
0041002A  .- FF25 04124000  JMP DWORD PTR DS:[401204]   kernel32.HeapCompact
00410030  .- FF25 08124000  JMP DWORD PTR DS:[401208]   kernel32.HeapCreate
00410036  .- FF25 0C124000  JMP DWORD PTR DS:[40120C]   kernel32.HeapDestroy
0041003C  .- FF25 10124000  JMP DWORD PTR DS:[401210]   ntdll.RtlFreeHeap
00410042  .- FF25 A8114000  JMP DWORD PTR DS:[4011A8]   user32.MessageBoxA
00410048  .- FF25 AC114000  JMP DWORD PTR DS:[4011AC]   user32.SetWindowsHookExA
0041004E  .- FF25 B0114000  JMP DWORD PTR DS:[4011B0]   user32.SystemParametersInfoA
00410054  .- FF25 B4114000  JMP DWORD PTR DS:[4011B4]   user32.UnhookWindowsHookEx
0041005A  .- FF25 14124000  JMP DWORD PTR DS:[401214]   kernel32.lstrcatA
00410060  .- FF25 F8114000  JMP DWORD PTR DS:[4011F8]   kernel32.GlobalAlloc
00410066  .- FF25 FC114000  JMP DWORD PTR DS:[4011FC]   kernel32.GlobalFree
0041006C     00             DB 00
0041006D     00             DB 00
0041006E     00             DB 00
```

Здесь видимо, что все элементы соответствующие тем, что мы видели в начале, все идут в секцию кода той же DLL. Посмотрим с помощью VIEW-M, какой DLL они соответствуют.



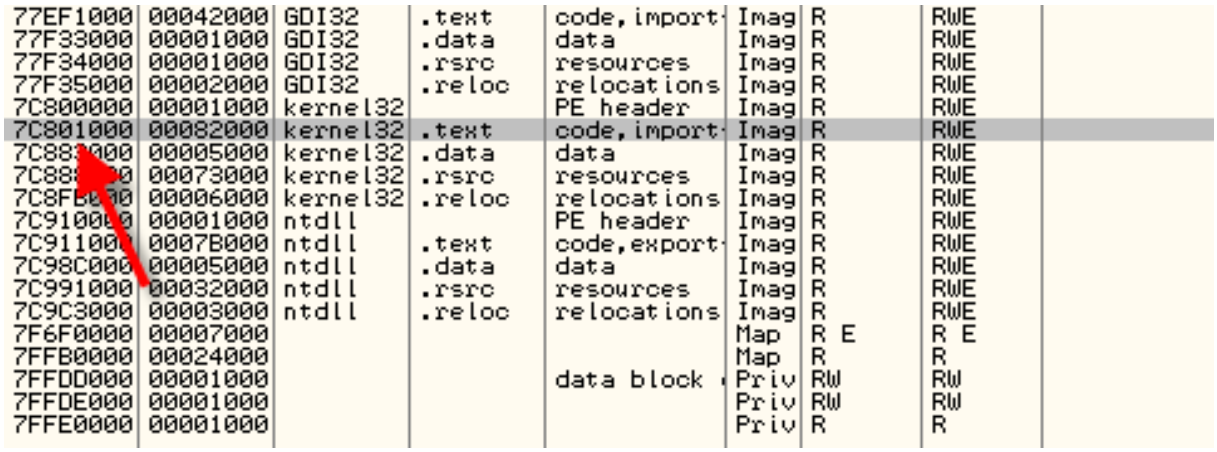
```
Address   Hex dump                                          ASCII
004011F4  29 B5 80 7C 2D FF 80 7C 2F FE 80 7C D4 05 92 7C  )µÇ|- ÿ|/■Ç|╒♪Æ|
00401204  AE 30 82 7C 29 29 81 7C 10 11 81 7C 3D 04 92 7C  «0é|))ü|►◄ü|=♦Æ|
00401214  B9 8F 83 7C 00 00 00 00 00 00 00 00 00 00 00 00  ╣Åâ|............
00401224  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00401234  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```



```
Address   Hex dump                                          ASCII
004011D4  22 11 00 00 30 11 00 00 3E 11 00 00 4C 11 00 00  "◄..0◄..>◄..L◄..
004011E4  58 11 00 00 00 00 00 00 A2 CA 81 7C 37 97 80 7C  X◄.....óⁿü|7ùÇ|
004011F4  29 B5 80 7C 2D FF 80 7C 2F FE 80 7C D4 05 92 7C  )µÇ|- ÿ|/■Ç|╒♪Æ|
00401204  AE 30 82 7C 29 29 81 7C 10 11 81 7C 3D 04 92 7C  «0é|))ü|►◄ü|=♦Æ|
00401214  B9 8F 83 7C 00 00 00 00 00 00 00 00 00 00 00 00  ╣Åâ|............
00401224  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00401234  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00401244  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

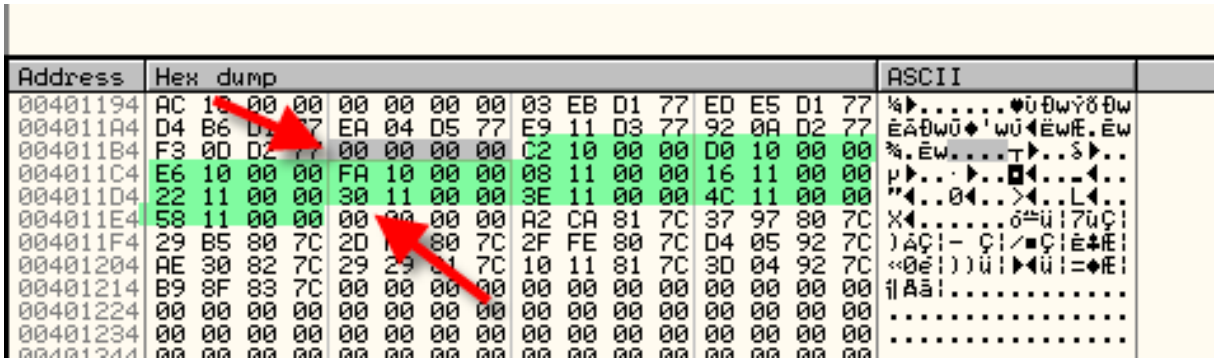
Все находятся внутри указанной секции, поэтому видим, что эти элементы, относящиеся к Kernel32.dll, указывают на секцию CODE.

Здесь же можем посмотреть конец IAT. Снизу от 401218 особенно ничего нет, так что конец IAT – это 401218, и теперь нам осталось найти начало.



```
77EF1000  00042000  GDI32     .text    code,import  Imag  R    RWE
77F33000  00001000  GDI32     .data    data         Imag  R    RWE
77F34000  00001000  GDI32     .rsrc    resources    Imag  R    RWE
77F35000  00002000  GDI32     .reloc   relocations  Imag  R    RWE
7C800000  00001000  kernel32            PE header    Imag  R    RWE
7C801000  00082000  kernel32  .text    code,import  Imag  R    RWE
7C883000  00005000  kernel32  .data    data         Imag  R    RWE
7C888000  00073000  kernel32  .rsrc    resources    Imag  R    RWE
7C8FB000  00006000  kernel32  .reloc   relocations  Imag  R    RWE
7C910000  00001000  ntdll               PE header    Imag  R    RWE
7C911000  0007B000  ntdll     .text    code,export  Imag  R    RWE
7C98C000  00005000  ntdll     .data    data         Imag  R    RWE
7C991000  00032000  ntdll     .rsrc    resources    Imag  R    RWE
7C9C3000  00003000  ntdll     .reloc   relocations  Imag  R    RWE
7F6F0000  00007000                                   Map   R E  R E
7FFB0000  00024000                                   Map   R    R
7FFDD000  00001000                     data block    Priv  RW   RW
7FFDE000  00001000                                   Priv  RW   RW
7FFE0000  00001000                                   Priv  R    R
```

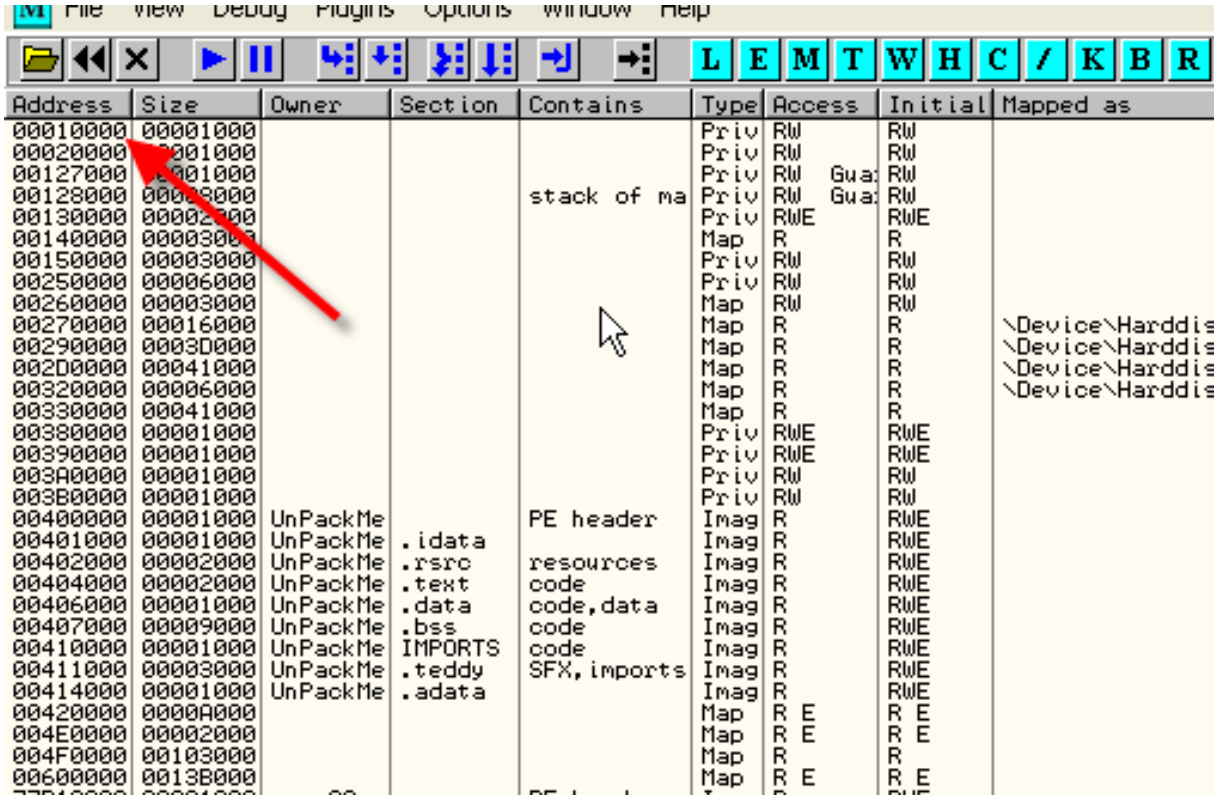
Видим разделяющие нули, а после них другую группу элементов.



Address | Hex dump | ASCII

```
00401194  AC 10 00 00 00 00 00 00 03 EB D1 77 ED E5 D1 77  ¼►......♥ûÐwýõÐw
004011A4  D4 B6 D1 77 EA 04 D5 77 E9 11 D3 77 92 0A D2 77  ÉÂÐwû♦'wú◄ëwÆ.Ëw
004011B4  F3 0D D2 77 00 00 00 00 C2 10 00 00 D0 10 00 00  ¾.Ëw....┬►..š►..
004011C4  E6 10 00 00 FA 10 00 00 08 11 00 00 16 11 00 00  µ►..·►..◘◄...◄..
004011D4  22 11 00 00 30 11 00 00 3E 11 00 00 4C 11 00 00  "◄..0◄..>◄..L◄..
004011E4  58 11 00 00 00 00 00 00 A2 CA 81 7C 37 97 80 7C  X◄......ó╩ü¦7ùÇ¦
004011F4  29 B5 80 7C 2D FF 80 7C 2F FE 80 7C D4 05 92 7C  )ÁÇ¦- Ç¦/■Ç¦È♣Æ¦
00401204  AE 30 82 7C 29 29 81 7C 10 11 81 7C 3D 04 92 7C  «0é¦))ü¦►◄ü¦=♦Æ¦
00401214  B9 8F 83 7C 00 00 00 00 00 00 00 00 00 00 00 00  ╣Å¦............
00401224  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00401234  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Которые точно являются элементами, смотрим адреса, куда на которые они указывают (10xx или 11xx), Это не DLL, ни что-нибудь ещё, так как самый нижний адрес в карте памяти – это 10000.

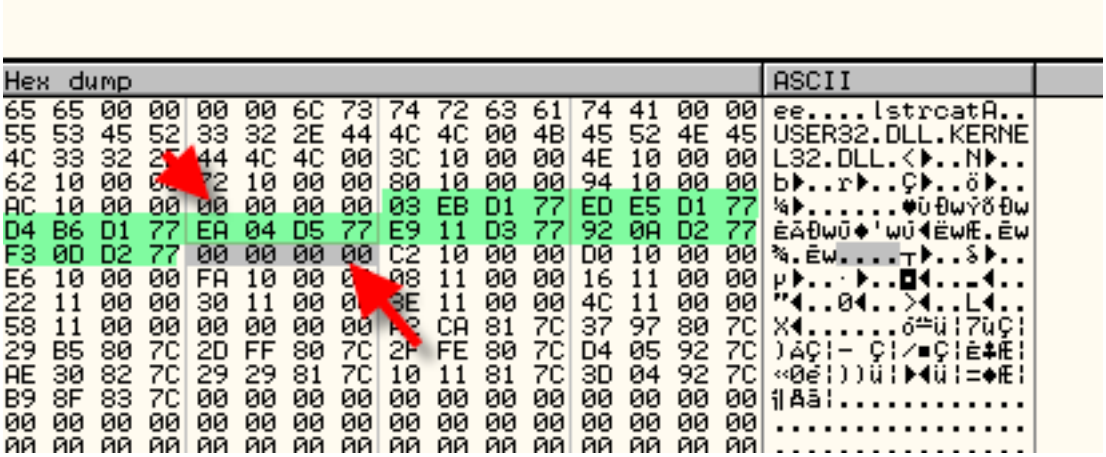


File View Debug Plugins Options Window Help

L E M T W H C / K B R

Address	Size	Owner	Section	Contains	Type	Access	Initial	Mapped as
00010000	00001000				Priv	RW	RW	
00020000	00001000				Priv	RW	RW	
00127000	00001000				Priv	RW Gua	RW	
00128000	00008000			stack of ma	Priv	RW Gua	RW	
00130000	00002000				Priv	RWE	RWE	
00140000	00003000				Map	R	R	
00150000	00003000				Priv	RW	RW	
00250000	00006000				Priv	RW	RW	
00260000	00003000				Map	RW	RW	
00270000	00016000				Map	R	R	\Device\Harddis
00290000	0003D000				Map	R	R	\Device\Harddis
002D0000	00041000				Map	R	R	\Device\Harddis
00320000	00006000				Map	R	R	\Device\Harddis
00330000	00041000				Map	R	R	
00380000	00001000				Priv	RWE	RWE	
00390000	00001000				Priv	RWE	RWE	
003A0000	00001000				Priv	RW	RW	
003B0000	00001000				Priv	RW	RW	
00400000	00001000	UnPackMe		PE header	Imag	R	RWE	
00401000	00001000	UnPackMe	.idata		Imag	R	RWE	
00402000	00002000	UnPackMe	.rsrc	resources	Imag	R	RWE	
00404000	00002000	UnPackMe	.text	code	Imag	R	RWE	
00406000	00001000	UnPackMe	.data	code,data	Imag	R	RWE	
00407000	00009000	UnPackMe	.bss	code	Imag	R	RWE	
00410000	00001000	UnPackMe	IMPORTS	code	Imag	R	RWE	
00411000	00003000	UnPackMe	.teddy	SFX,imports	Imag	R	RWE	
00414000	00001000	UnPackMe	.adata		Imag	R	RWE	
00420000	0000A000				Map	R E	R E	
004E0000	00002000				Map	R E	R E	
004F0000	00103000				Map	R	R	
00600000	0013B000				Map	R E	R E	

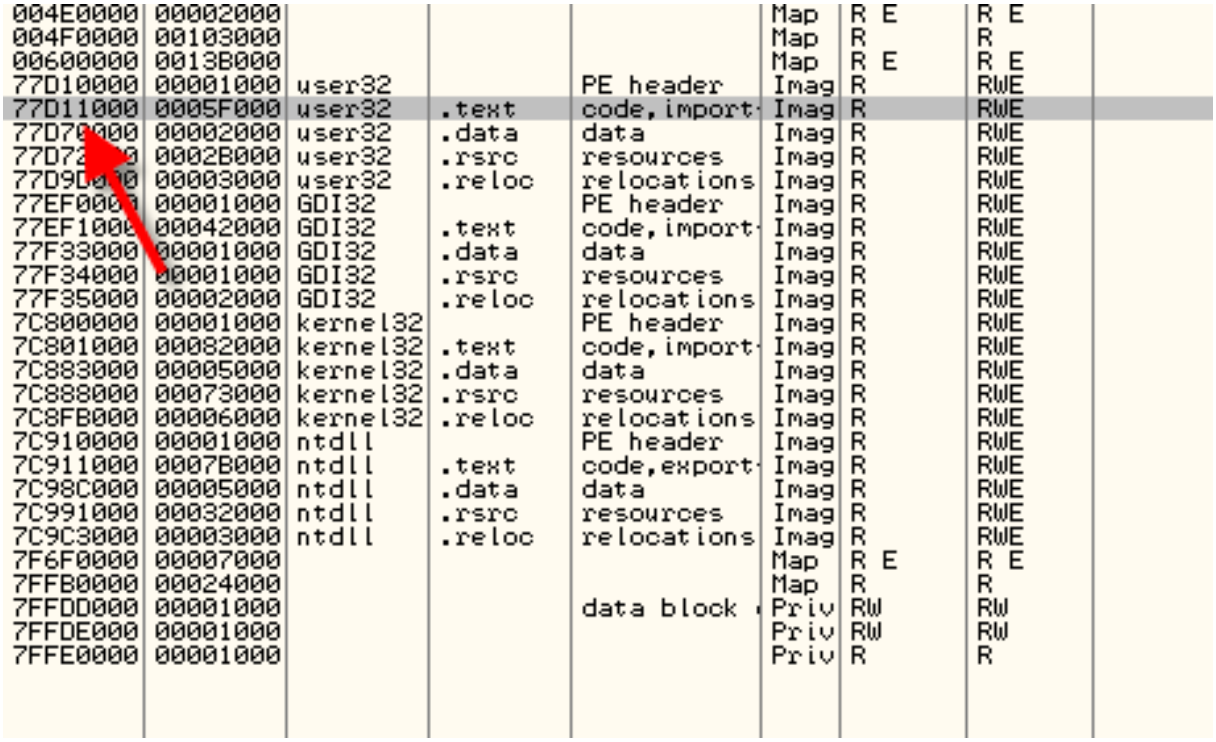
Так что здесь находятся элементы, которые не ведут ни в какую-либо DLL, ни в какую-то реальную секцию. Так же они могли бы указывать на секцию, созданную упаковщиков, но не в данном случае. Это мусор, разбросанный для того, чтобы заставить нас думать, для чего он нужен. Спускаемся дальше.



Hex dump | ASCII

```
65 65 00 00 00 00 6C 73 74 72 63 61 74 41 00 00  ee....lstrcatA..
55 53 45 52 33 32 2E 44 4C 4C 00 4B 45 52 4E 45  USER32.DLL.KERNE
4C 33 32 2E 44 4C 4C 00 3C 10 00 00 4E 10 00 00  L32.DLL.<►..N►..
62 10 00 00 72 10 00 00 80 10 00 00 94 10 00 00  b►..r►..Ç►..ö►..
AC 10 00 00 00 00 00 00 03 EB D1 77 ED E5 D1 77  ¼►......♥ûÐwýõÐw
D4 B6 D1 77 EA 04 D5 77 E9 11 D3 77 92 0A D2 77  ÉÂÐwû♦'wú◄ëwÆ.Ëw
F3 0D D2 77 00 00 00 00 C2 10 00 00 D0 10 00 00  ¾.Ëw....┬►..š►..
E6 10 00 00 FA 10 00 00 08 11 00 00 16 11 00 00  µ►..·►..◘◄...◄..
22 11 00 00 30 11 00 00 3E 11 00 00 4C 11 00 00  "◄..0◄..>◄..L◄..
58 11 00 00 00 00 00 00 A2 CA 81 7C 37 97 80 7C  X◄......ó╩ü¦7ùÇ¦
29 B5 80 7C 2D FF 80 7C 2F FE 80 7C D4 05 92 7C  )ÁÇ¦- Ç¦/■Ç¦È♣Æ¦
AE 30 82 7C 29 29 81 7C 10 11 81 7C 3D 04 92 7C  «0é¦))ü¦►◄ü¦=♦Æ¦
B9 8F 83 7C 00 00 00 00 00 00 00 00 00 00 00 00  ╣Å¦............
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

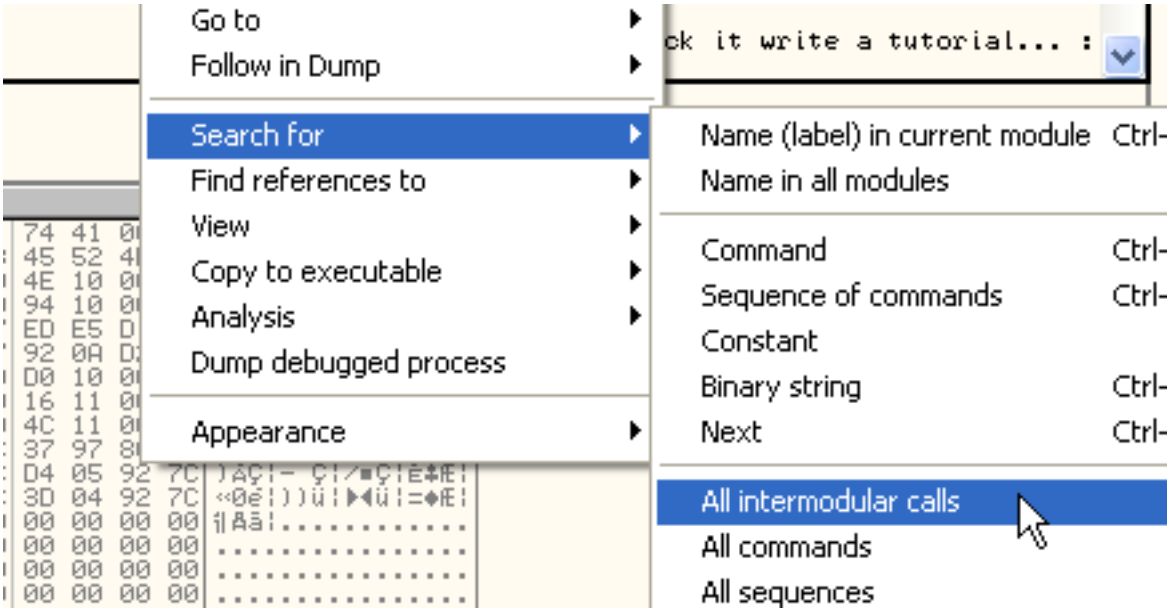
Видим, что среди нулей есть другая группа элементов, указывающих на адреса 77Dxxxxx. Смотрим в карте памяти, какой DLL они соответствуют.

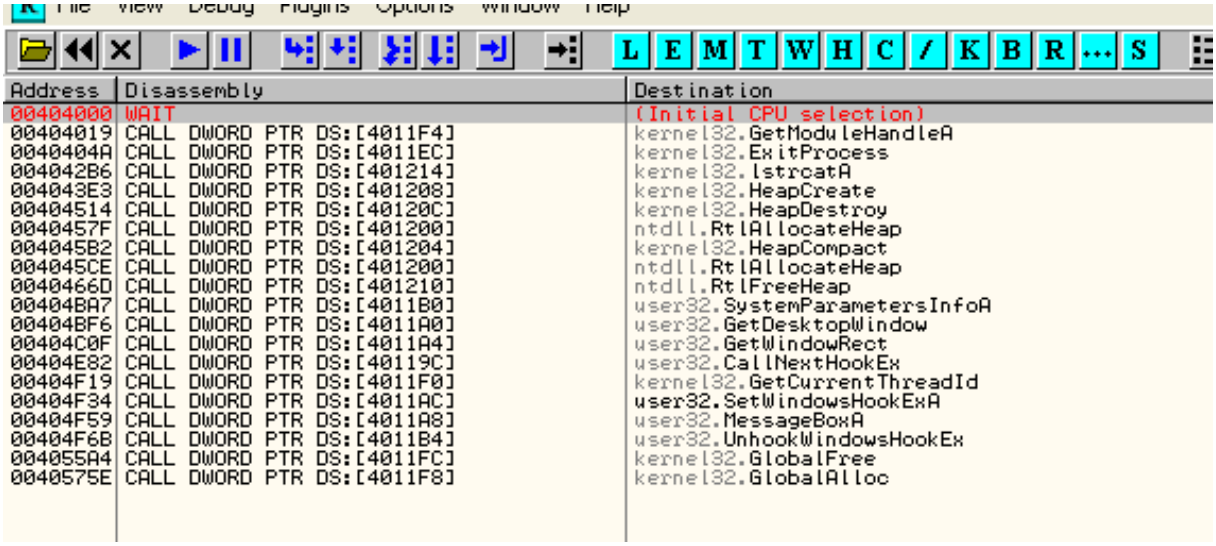


004E0000	00002000				Map	R E	R E
004F0000	00103000				Map	R	R
00600000	0013B000				Map	R E	R E
77D10000	00001000	user32		PE header	Imag	R	RWE
77D11000	0005F000	user32	.text	code,import	Imag	R	RWE
77D70000	00002000	user32	.data	data	Imag	R	RWE
77D72000	0002B000	user32	.rsrc	resources	Imag	R	RWE
77D9D000	00003000	user32	.reloc	relocations	Imag	R	RWE
77EF0000	00001000	GDI32		PE header	Imag	R	RWE
77EF1000	00042000	GDI32	.text	code,import	Imag	R	RWE
77F33000	00001000	GDI32	.data	data	Imag	R	RWE
77F34000	00001000	GDI32	.rsrc	resources	Imag	R	RWE
77F35000	00002000	GDI32	.reloc	relocations	Imag	R	RWE
7C800000	00001000	kernel32		PE header	Imag	R	RWE
7C801000	00082000	kernel32	.text	code,import	Imag	R	RWE
7C883000	00005000	kernel32	.data	data	Imag	R	RWE
7C888000	00073000	kernel32	.rsrc	resources	Imag	R	RWE
7C8FB000	00006000	kernel32	.reloc	relocations	Imag	R	RWE
7C910000	00001000	ntdll		PE header	Imag	R	RWE
7C911000	0007B000	ntdll	.text	code,export	Imag	R	RWE
7C98C000	00005000	ntdll	.data	data	Imag	R	RWE
7C991000	00032000	ntdll	.rsrc	resources	Imag	R	RWE
7C9C3000	00003000	ntdll	.reloc	relocations	Imag	R	RWE
7F6F0000	00007000				Map	R E	R E
7FFB0000	00024000				Map	R	R
7FFDD000	00001000			data block	Priv	RW	RW
7FFDE000	00001000				Priv	RW	RW
7FFE0000	00001000				Priv	R	R

Видим, что они принадлежат User32.dll.

Также видим, что в карте памяти есть много других DLL, таких как GDI32 и Ntdll, которые могут загружаться упаковщиком для собственного использования, а не для использования программой. Чтобы проверить это, сделаем в этом же списке SEARCH FOR – ALL INTERMODULAR CALLS.

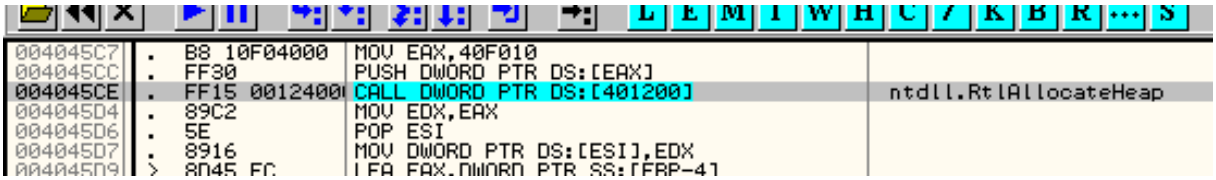




Address	Disassembly	Destination
00404000	WAIT	(Initial CPU selection)
00404019	CALL DWORD PTR DS:[4011F4]	kernel32.GetModuleHandleA
0040404A	CALL DWORD PTR DS:[4011EC]	kernel32.ExitProcess
004042B6	CALL DWORD PTR DS:[401214]	kernel32.lstrcatA
004043E3	CALL DWORD PTR DS:[401208]	kernel32.HeapCreate
00404514	CALL DWORD PTR DS:[40120C]	kernel32.HeapDestroy
0040457F	CALL DWORD PTR DS:[401200]	ntdll.RtlAllocateHeap
004045B2	CALL DWORD PTR DS:[401204]	kernel32.HeapCompact
004045CE	CALL DWORD PTR DS:[401200]	ntdll.RtlAllocateHeap
0040466D	CALL DWORD PTR DS:[401210]	ntdll.RtlFreeHeap
00404BA7	CALL DWORD PTR DS:[4011B0]	user32.SystemParametersInfoA
00404BF6	CALL DWORD PTR DS:[4011A0]	user32.GetDesktopWindow
00404C0F	CALL DWORD PTR DS:[4011A4]	user32.GetWindowRect
00404E82	CALL DWORD PTR DS:[40119C]	user32.CallNextHookEx
00404F19	CALL DWORD PTR DS:[4011F0]	kernel32.GetCurrentThreadId
00404F34	CALL DWORD PTR DS:[4011AC]	user32.SetWindowsHookExA
00404F59	CALL DWORD PTR DS:[4011A8]	user32.MessageBoxA
00404F6B	CALL DWORD PTR DS:[4011B4]	user32.UnhookWindowsHookEx
004055A4	CALL DWORD PTR DS:[4011FC]	kernel32.GlobalFree
0040575E	CALL DWORD PTR DS:[4011F8]	kernel32.GlobalAlloc

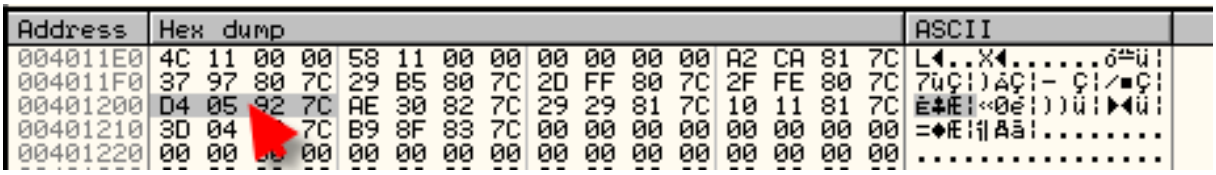
Видим, что есть вызовы 3-х DLL, две из них – те, которые мы нашли, и другая – это Ntdll. Смотрим.

Если, например, перейдём к вызовам этой DLL.



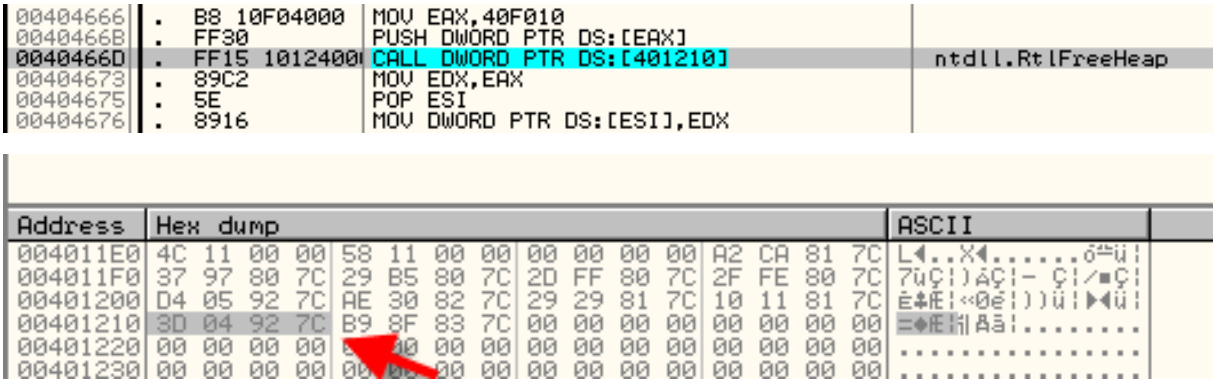
004045C7	B8 10F04000	MOV EAX,40F010	
004045CC	FF30	PUSH DWORD PTR DS:[EAX]	
004045CE	FF15 0012400(	CALL DWORD PTR DS:[401200]	ntdll.RtlAllocateHeap
004045D4	89C2	MOV EDX,EAX	
004045D6	5E	POP ESI	
004045D7	8916	MOV DWORD PTR DS:[ESI],EDX	

Видим, что соответствующий элемент находится в 401200, то есть это смешение (mezclada – чего-то я не совсем понял, о чём он толкует, кто понял – отпишитесь – прим. пер.) с kernel32.dll.



Address	Hex dump	ASCII
004011E0	4C 11 00 00 58 11 00 00 00 00 00 00 A2 CA 81 7C	L◄..X◄.....ó╩ü¦
004011F0	37 97 80 7C 29 B5 80 7C 2D FF 80 7C 2F FE 80 7C	7ùÇ¦)ÁÇ¦- Ç¦/■Ç¦
00401200	D4 05 92 7C AE 30 82 7C 29 29 81 7C 10 11 81 7C	╘♣Æ¦«0é¦))ü¦►◄ü¦
00401210	3D 04 92 7C B9 8F 83 7C 00 00 00 00 00 00 00 00	=♦Æ¦╣Åâ¦........
00401220	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....

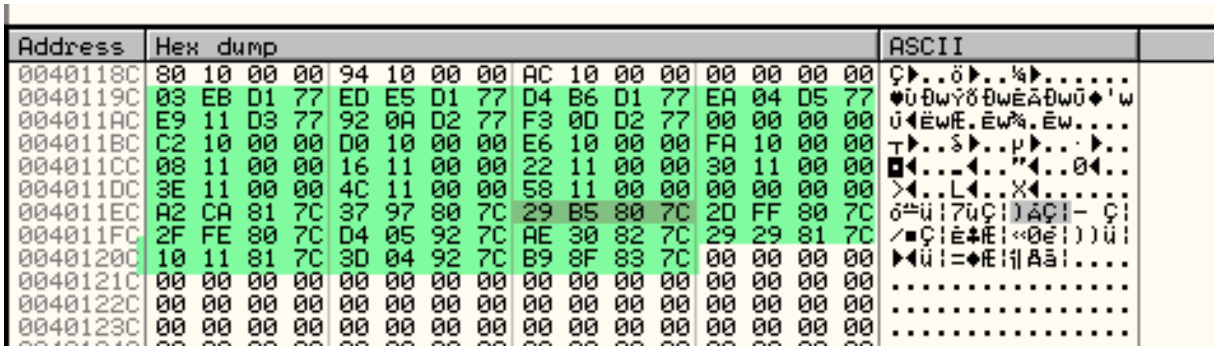
То же самое относится и к другой из указанных DLL, которая смешана с kernel32.dll. Мы не заметили, что эти две секции кода находятся рядом друг с другом, но это так.



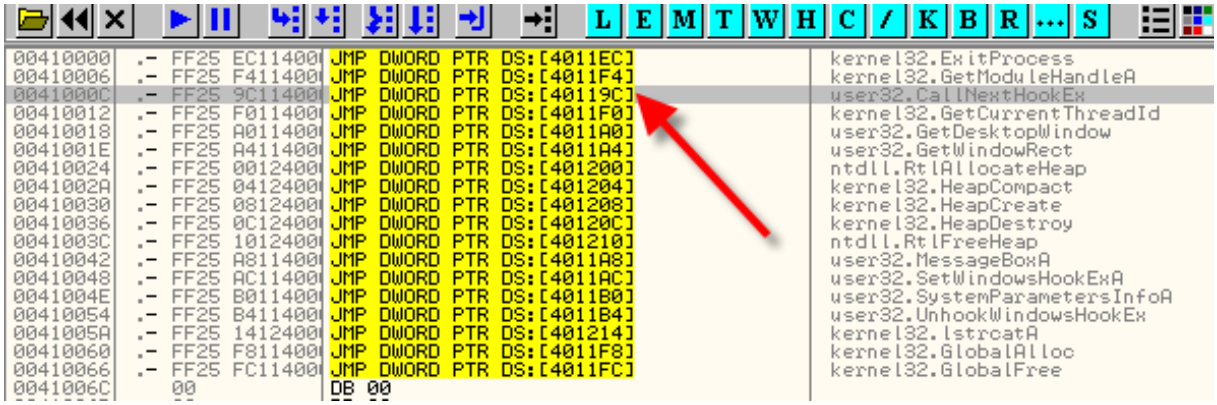
00404666	B8 10F04000	MOV EAX,40F010	
0040466B	FF30	PUSH DWORD PTR DS:[EAX]	
0040466D	FF15 1012400(	CALL DWORD PTR DS:[401210]	ntdll.RtlFreeHeap
00404673	89C2	MOV EDX,EAX	
00404675	5E	POP ESI	
00404676	8916	MOV DWORD PTR DS:[ESI],EDX	

Address	Hex dump	ASCII
004011E0	4C 11 00 00 58 11 00 00 00 00 00 00 A2 CA 81 7C	L◄..X◄.....ó╩ü¦
004011F0	37 97 80 7C 29 B5 80 7C 2D FF 80 7C 2F FE 80 7C	7ùÇ¦)ÁÇ¦- Ç¦/■Ç¦
00401200	D4 05 92 7C AE 30 82 7C 29 29 81 7C 10 11 81 7C	╘♣Æ¦«0é¦))ü¦►◄ü¦
00401210	3D 04 92 7C B9 8F 83 7C 00 00 00 00 00 00 00 00	=♦Æ¦╣Åâ¦........
00401220	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
00401230	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....

Ок, посмотрим какие проблемы могут быть из-за подобного начала IAT. Вот все эти элементы:



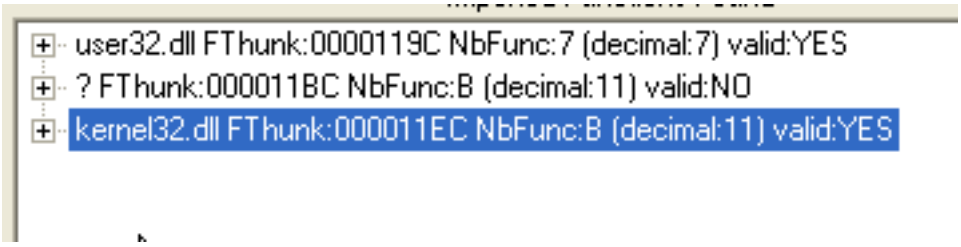
Начала IAT находится в 40119C, которое совпадает с меньшим значением, найденным в таблице переходов.



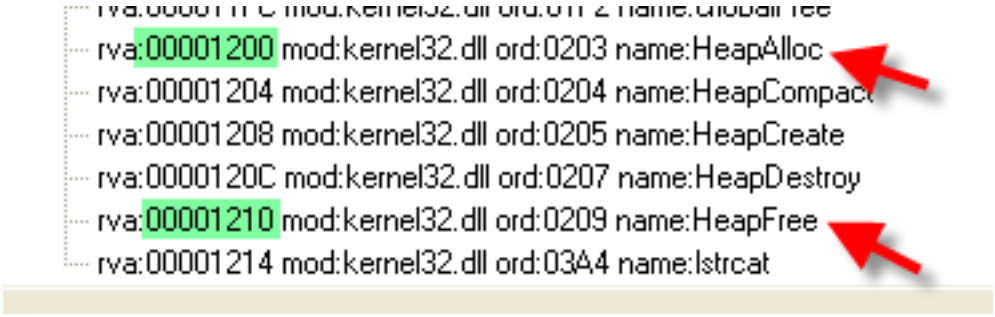
Видим, что оно самое маленькое из всех этих значений, так что у нас получается:

```
OEP=4000
RVA или НАЧАЛО IAT=119C
РАЗМЕР = КОНЕЦ МИНУС НАЧАЛО = 401218-40119c = 7C
```

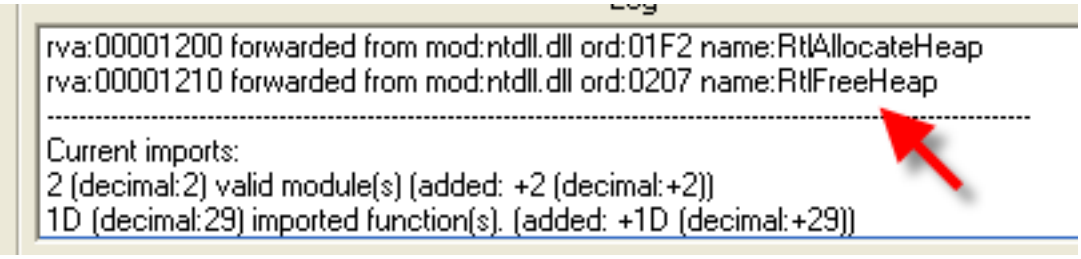
Задаём эти значения в IMP REC, чтобы посмотреть, что получится, видим, что оба смешанных элемента относятся к ntdll и kernel32.



Видим, что указанная часть не та, которая соответствует мусорным элементам, и внизу видим, что у нас есть только элементы для kernel32, и если посмотрим редко встречающиеся элементы, соответствующие 401200 b 401210, то увидим, что:

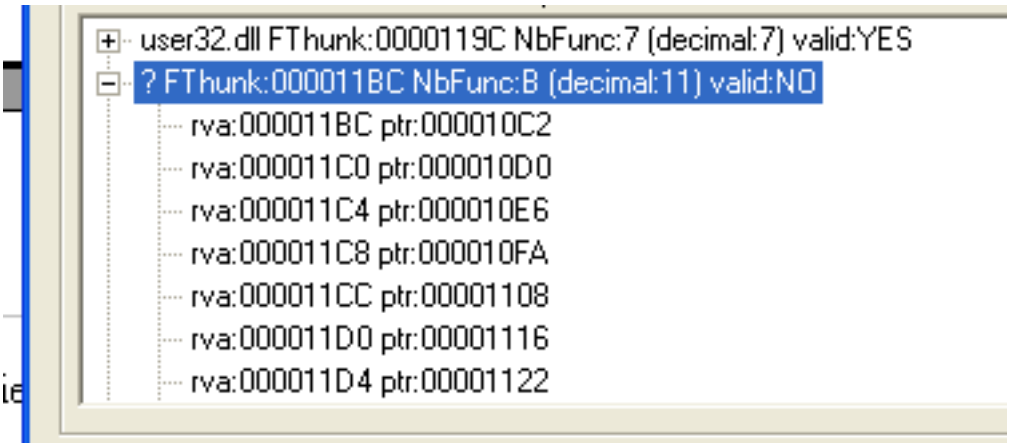


Заменяем его на похожие элементы, соответствующие kernel32.dll, и что это будет означать?

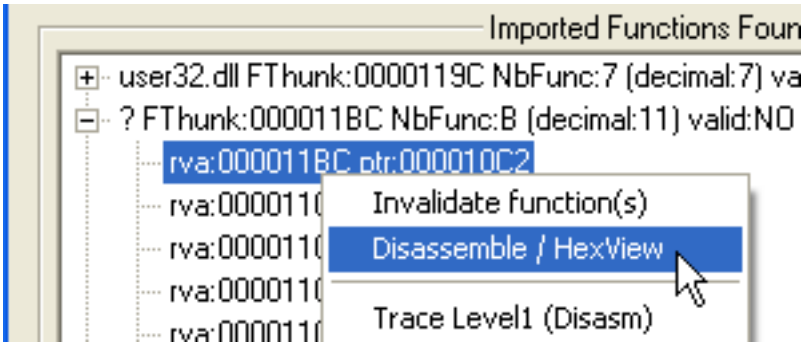


Видим, что теперь в логе нам говорят, что эти элемент похожи на элементы из kernel32.dll, и пакер может свободно менять их, что всё запутать и усложнить.

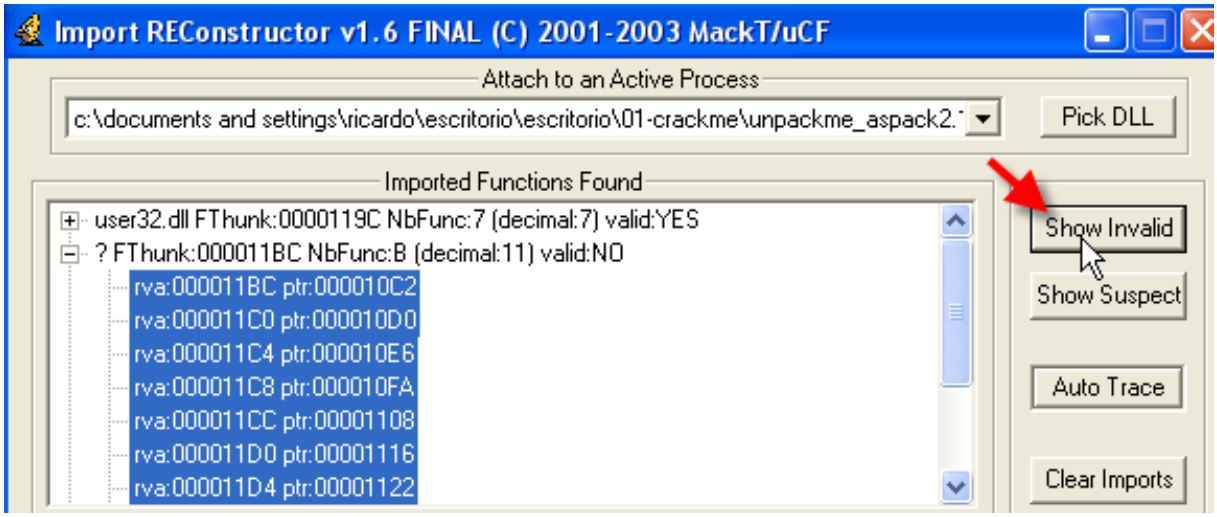
Хорошо, теперь нам осталось убрать мусор из середины. Проверяем эти элементы и смотрим, если написано NO, то это мусор. Чтобы убедиться в этом, идём к одному из них.



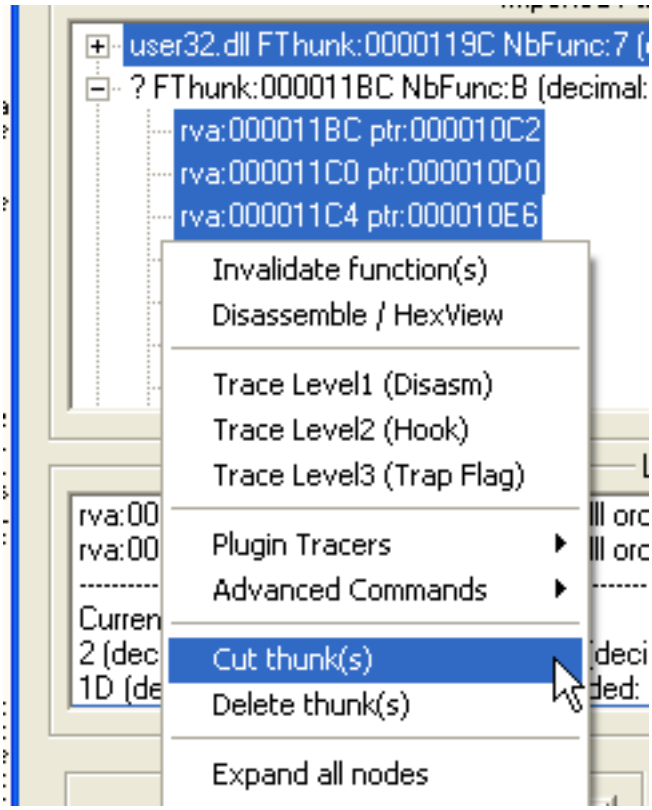
Отмечаем первый, нажимаем правую кнопку мыши и выбираем DISASSEMBLE-HEX VIEW.

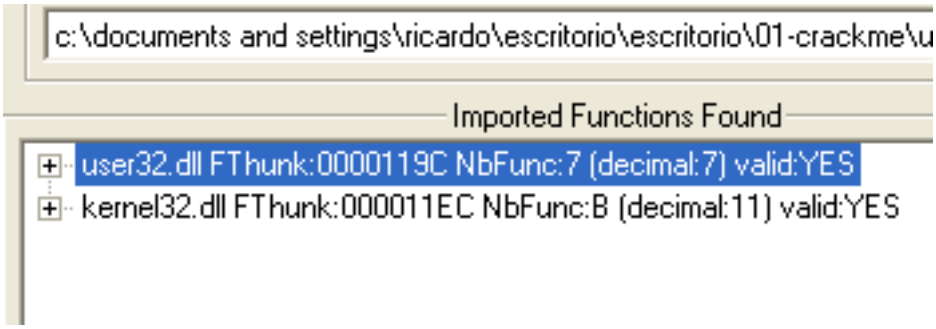


Здесь видим, что это не приводит нас к реально существующему месту, так что отмечаем все элементы как мусор.



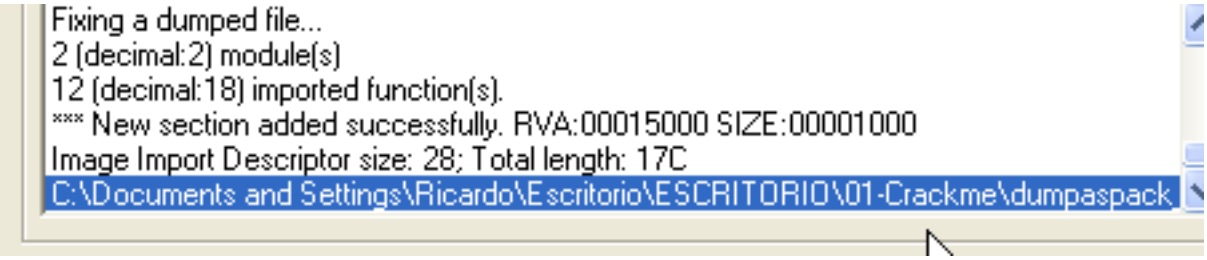
Нажимаем SHOW INVALID, а затем, держа все эти элементы отмеченными, нажимаем на правую кнопку мыши и выбираем CUT THUNKS.





Таким образом мы аннулировали все эти неверные элементы, чтобы система при загрузке не выдавала ошибку, пытаясь загрузить несуществующие API-функции.

Теперь можно нажать FIX DUMP, так как все элементы отмечены как YES, то есть как правильные.



И создаётся dumpaspack_.exe, файл, который предположительно должен быть починен, запускаем и смотрим, так ли это.

Работает превосходно, поэтому на этот раз здесь и остановимся. Сложность нужно повышать постепенно.

Введение в крэкинг с нуля, используя OllyDbg - Глава 36

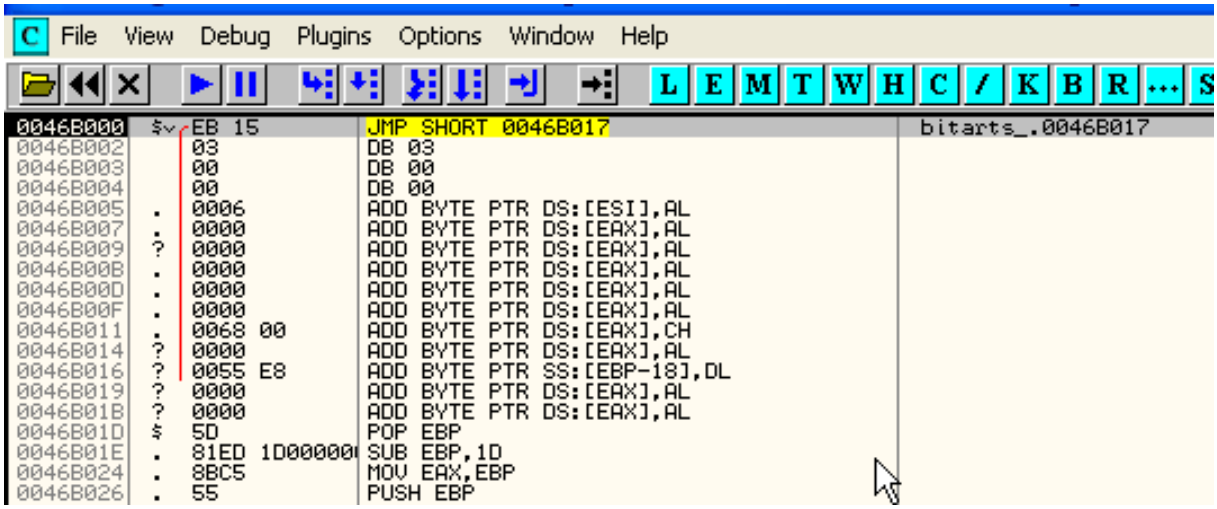
Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-11-12

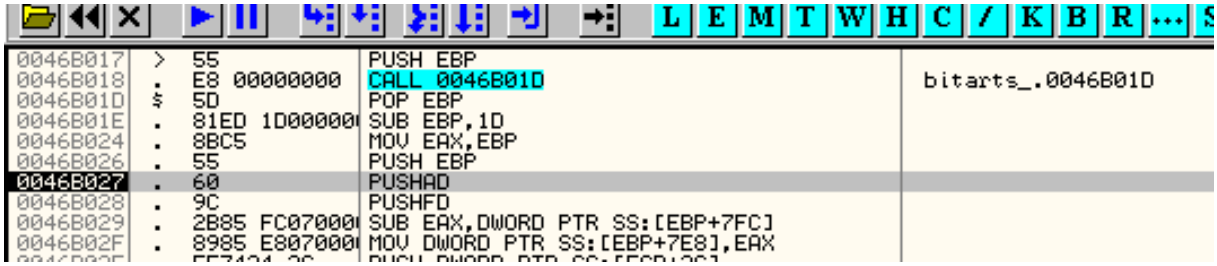
Файлы к статье (находится в архиве `wasm_files.zip`: `files/0500/ollydbg36.zip`)

Продолжаем, очень плавно повышая сложность изучаемых упаковщиков. В этой главе мы рассмотрим два распакуй-меня – crunch (или bitatrs 5.0) и telock 0.98, которые послужат нам для освоения темы переадресовочных элемент IAT. Оба распакуй-меня приложены к данному туториалу, поэтому у вас не должно быть проблем, где их взять.

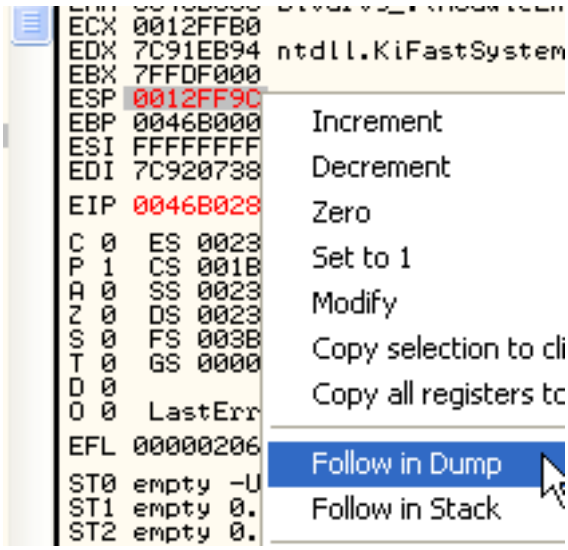
Начнём с самого простого – с BITARTS. Загрузим его в OllyDbg.



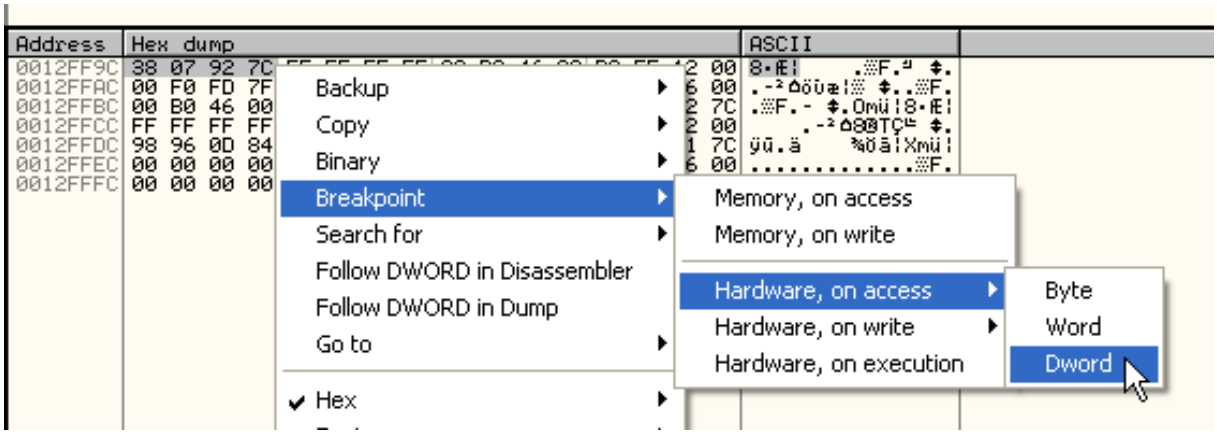
Видим, что начального PUSHAD нет, так что потрассируем немного с помощью F7.



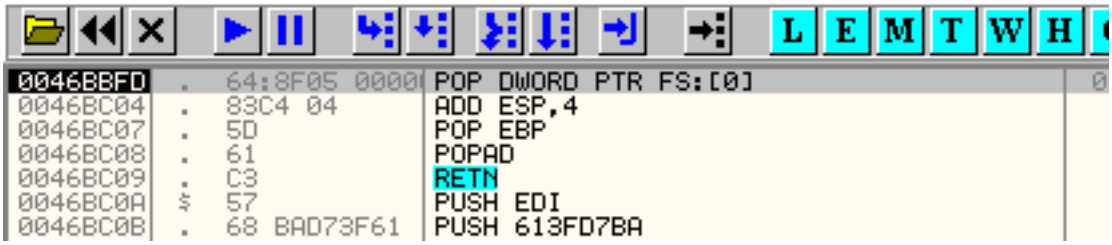
Доходим до PUSHAD, проходим его с помощью F7 и делаем ESP – FOLLOW IN DUMP.



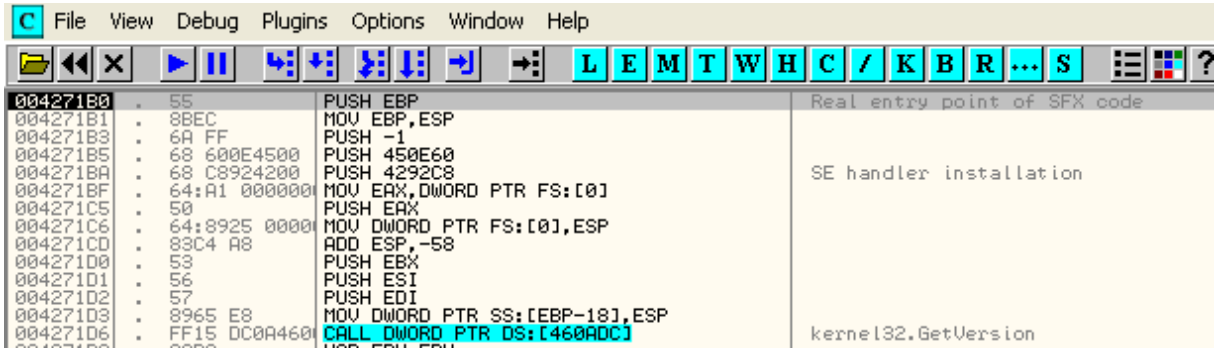
И устанавливаем точку аппаратного останова (hardware breakpoint) на доступ к первым байтам.



Теперь нажимаем F9 и после прохода через другое исключение останавливаемся на точке аппаратного останова.

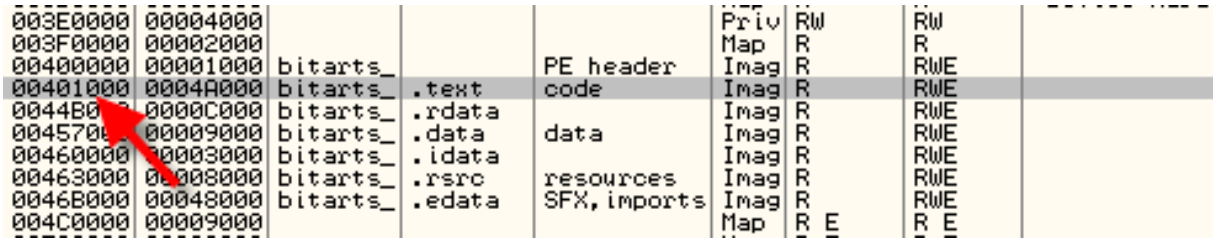


Трассируем строки дальше и оказываемся в OEP.



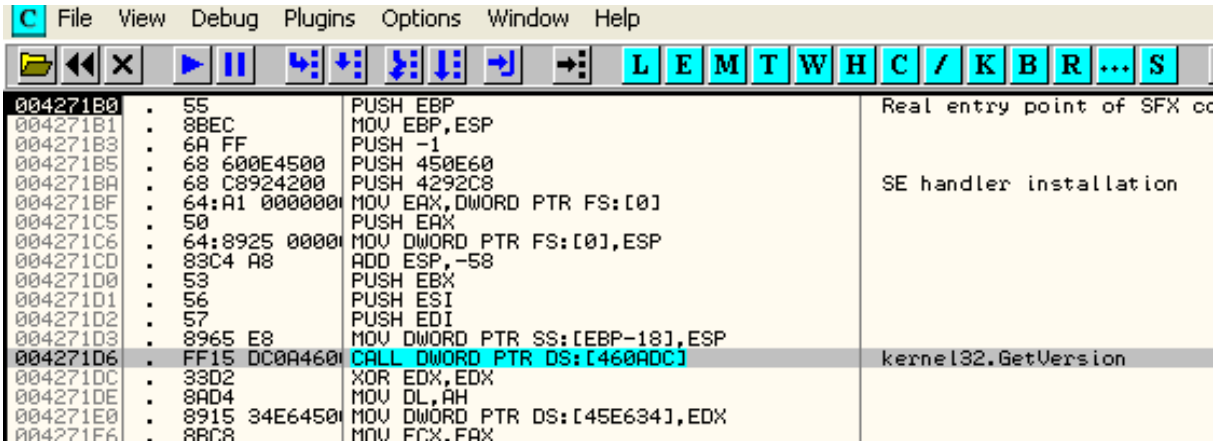
Также оказаться в OEP можно с помощью поисковика OEP OllyDbg, если последняя пропатчена для этого, или почти любым другим методом, рассмотренным в предыдущих главах «Введения…».

Как видим, указанный адрес находится в первой секции после заголовка.



003E0000	00004000				Priv	RW	RW
003F0000	00002000				Map	R	R
00400000	00001000	bitarts_		PE header	Imag	R	RWE
00401000	0004A000	bitarts_	.text	code	Imag	R	RWE
0044B0[illegible]	0000C000	bitarts_	.rdata		Imag	R	RWE
004570[illegible]	00009000	bitarts_	.data	data	Imag	R	RWE
00460000	00003000	bitarts_	.idata		Imag	R	RWE
00463000	00008000	bitarts_	.rsrc	resources	Imag	R	RWE
0046B000	00048000	bitarts_	.edata	SFX,imports	Imag	R	RWE
004C0000	00009000				Map	R E	R E

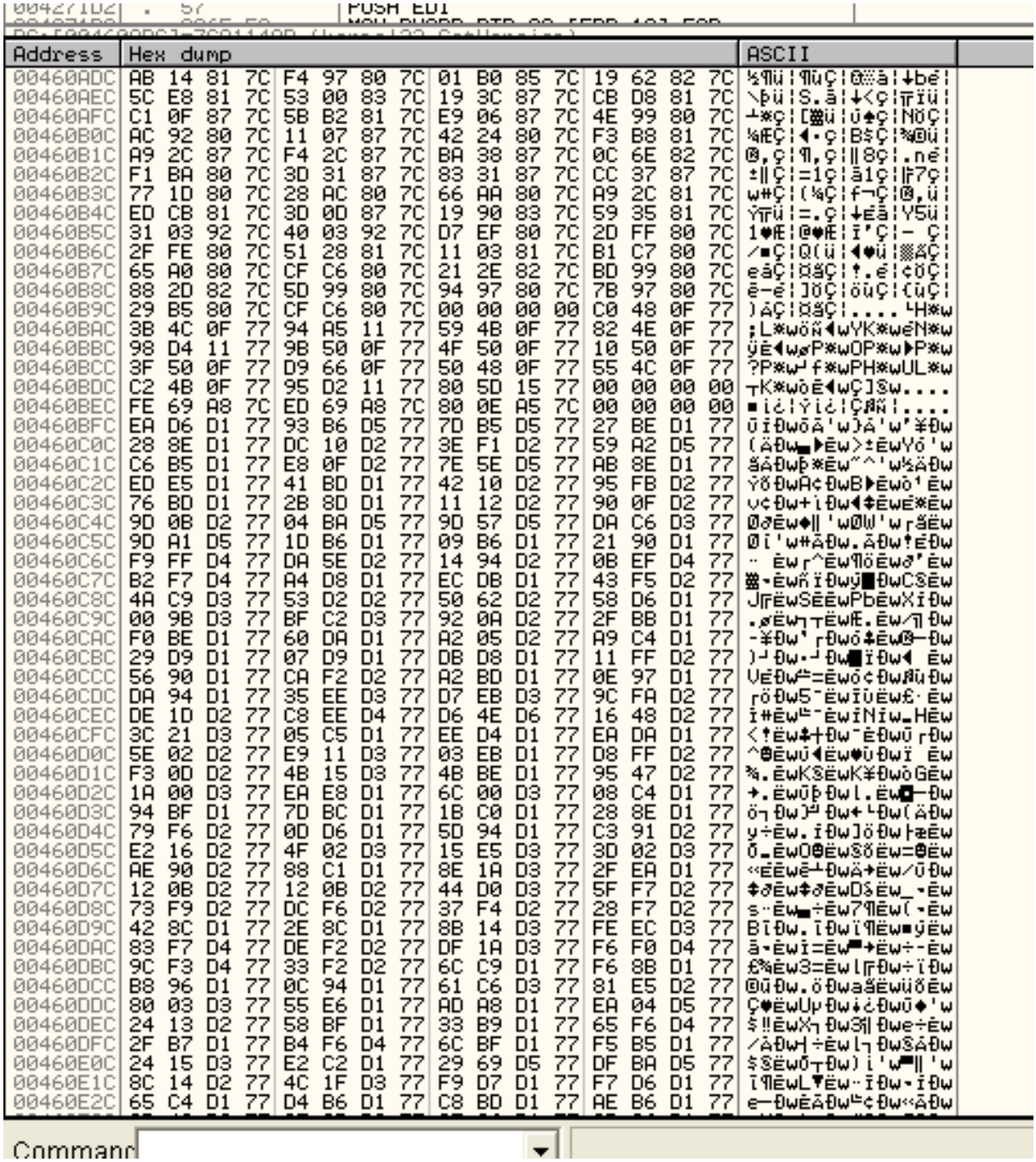
Поэтому кажется, что всё в порядке, включая отмеченную секцию CODE, так что проблем не должно быть.



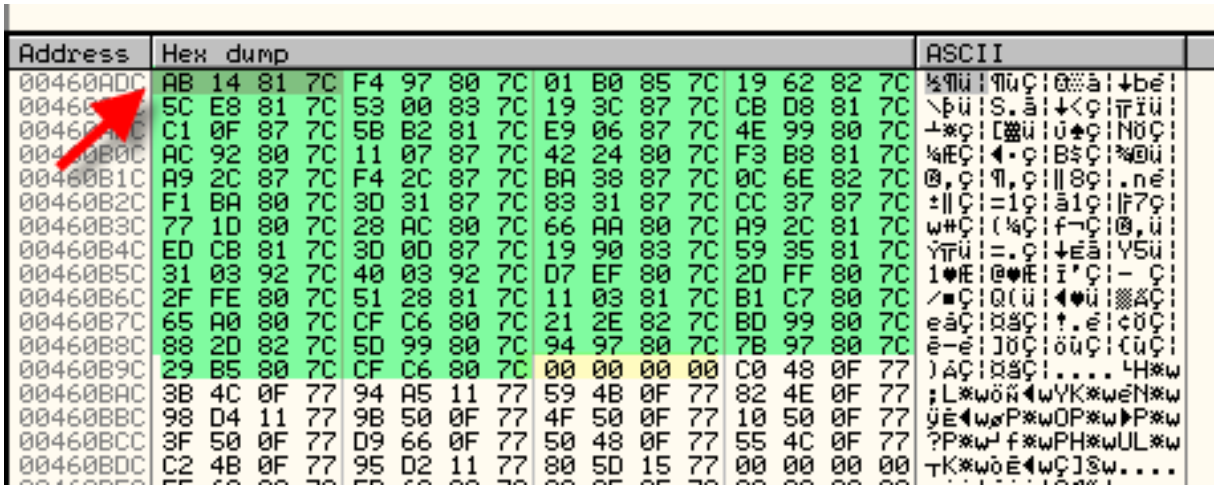
Видим, что в данном случае первая вызываемая API-функция – это GetVersion, поэтому также устанавливаем на данную API-функцию BP, и когда останавливаемся при вызове её из первой секции, то по возвращению мы оказываемся в области OEP.

Ок, OEP не представляет секрета, и теперь заглянем в IAT. Конечно, рассматриваемый нами вызов выглядит как прямой, но легко убедиться, что, на самом деле, он косвенный: загружается правильный для моей машины адрес API-функции GetVersion для того, чтобы совершить на него переход.

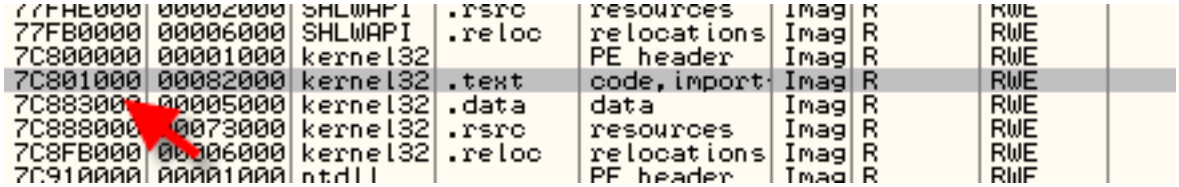
Значит, 460ADC – это элемент IAT, соответствующий GetVersion. Идём в DUMP и сверяемся с IAT.



Address	Hex dump																ASCII
00460ADC	AB	14	81	7C	F4	97	80	7C	01	B0	85	7C	19	62	82	7C	½¶ü¦¶üÇ¦@▒à¦↓bé¦
00460AEC	5C	E8	81	7C	53	00	83	7C	19	3C	87	7C	CB	D8	81	7C	\Þü¦S.à¦↓<ç¦╦Ïü¦
00460AFC	C1	0F	87	7C	5B	B2	81	7C	E9	06	87	7C	4E	99	80	7C	┴☼ç¦[▒ü¦ú♠ç¦NöÇ¦
00460B0C	AC	92	80	7C	11	07	87	7C	42	24	80	7C	F3	B8	81	7C	¼ÆÇ¦◀•ç¦B$Ç¦³Øü¦
00460B1C	A9	2C	87	7C	F4	2C	87	7C	BA	38	87	7C	0C	6E	82	7C	⌐,ç¦¶,ç¦‖8ç¦.né¦
00460B2C	F1	BA	80	7C	3D	31	87	7C	83	31	87	7C	CC	37	87	7C	±‖Ç¦=1ç¦â1ç¦╠7ç¦
00460B3C	77	1D	80	7C	28	AC	80	7C	66	AA	80	7C	A9	2C	81	7C	w#Ç¦(¼Ç¦f¬Ç¦⌐,ü¦
00460B4C	ED	CB	81	7C	3D	0D	87	7C	19	90	83	7C	59	35	81	7C	ÿ╦ü¦=.ç¦↓Éâ¦Y5ü¦
00460B5C	31	03	92	7C	40	03	92	7C	D7	EF	80	7C	2D	FF	80	7C	1♥Æ¦@♥Æ¦ï'Ç¦- Ç¦
00460B6C	2F	FE	80	7C	51	28	81	7C	11	03	81	7C	B1	C7	80	7C	/■Ç¦Q(ü¦◀♥ü¦▒ÁÇ¦
00460B7C	65	A0	80	7C	CF	C6	80	7C	21	2E	82	7C	BD	99	80	7C	eáÇ¦¤ãÇ¦!.é¦¢öÇ¦
00460B8C	88	2D	82	7C	5D	99	80	7C	94	97	80	7C	7B	97	80	7C	ê-é¦]öÇ¦öùÇ¦{ùÇ¦
00460B9C	29	B5	80	7C	CF	C6	80	7C	00	00	00	00	C0	48	0F	77	)ÁÇ¦¤ãÇ¦....└H☼w
00460BAC	3B	4C	0F	77	94	A5	11	77	59	4B	0F	77	82	4E	0F	77	;L☼wöñ◀wYK☼wéN☼w
00460BBC	98	D4	11	77	9B	50	0F	77	4F	50	0F	77	10	50	0F	77	ÿÈ◀w∟P☼wOP☼w▶P☼w
00460BCC	3F	50	0F	77	D9	66	0F	77	50	48	0F	77	55	4C	0F	77	?P☼w┘f☼wPH☼wUL☼w
00460BDC	C2	4B	0F	77	95	D2	11	77	80	5D	15	77	00	00	00	00	┬K☼wòÈ◀wÇ]§w....
00460BEC	FE	69	A8	7C	ED	69	A8	7C	80	0E	A5	7C	00	00	00	00	■i¿¦Ýi¿¦Çñ¦....
00460BFC	EA	D6	D1	77	93	B6	D5	77	7D	B5	D5	77	27	BE	D1	77	ÜïðwõÁ'w}Á'w'¥ðw
00460C0C	28	8E	D1	77	DC	10	D2	77	3E	F1	D2	77	59	A2	D5	77	(ÄðwÃ▶ËwÒ>±ËwYó'w
00460C1C	C6	B5	D1	77	E8	0F	D2	77	7E	5E	D5	77	AB	8E	D1	77	ãÁðwþ☼Ëwˆˆˆ



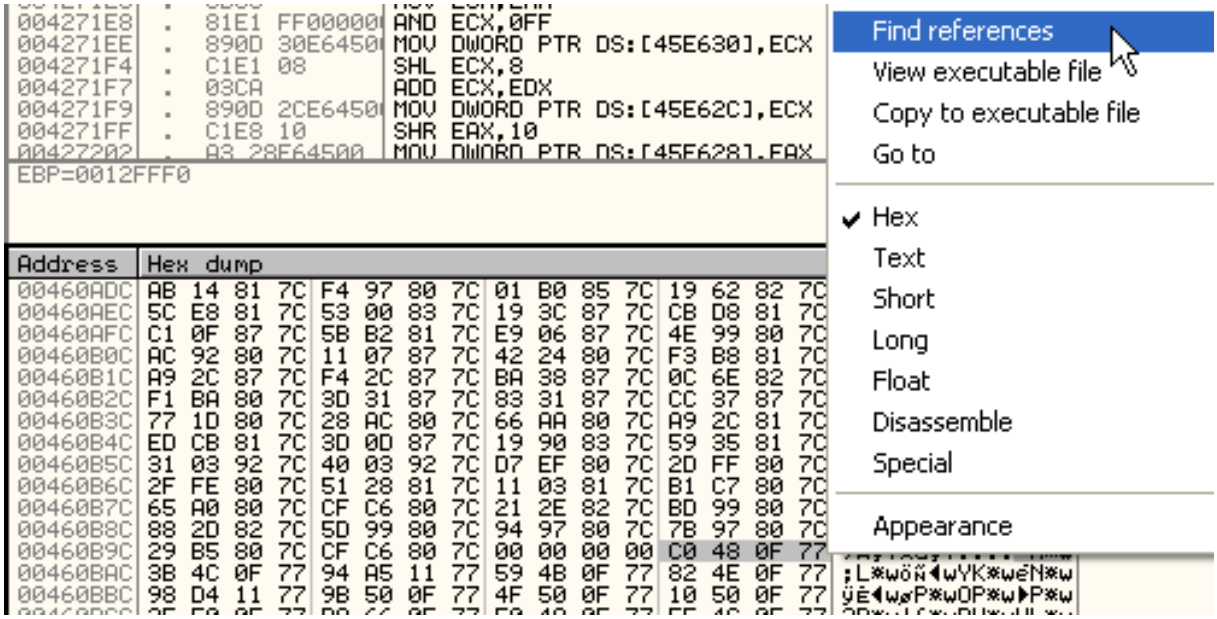
Первый элемент в IAT – это GetVersion, который соответствует Kernel32.dll, можно посмотреть в карте памяти, куда ведёт указанный адрес 7Cxxxxxx.



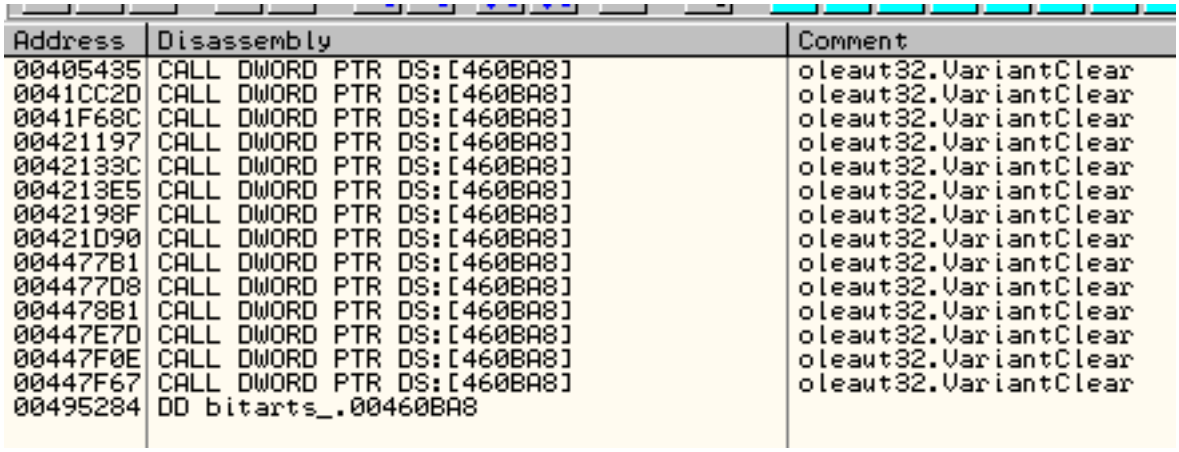
Видим, что всё соответствует Kernel32.dll, рядом с серединой, как и в прошлой части, есть пара смешений, соответствующих ntdll, но мы уже проверили, что система выдаёт исключения для этих API-функций, которые изначально относились к kernel32.dll и были заменены на похожие из ntdll.dll, но для совместимости они считаются системой вызываемыми из kernel32.dll без каких-либо проблем. В любом другом случае система не станет делать исключений, и API-функции должны относиться к соответствующей DLL, которые, в свою очередь, должны быть отделены нулями, как того требует правильно составленная IAT.

Хорошо, после этого спускаем ниже и находим группу API-функций с адресами вида 77xxxxxx.

Если есть какие-то сомнения относительно того, к какой DLL принадлежит элемент, то кроме анализа карты памяти, можно отметить его в IAT, нажать правую кнопку мыши и выбрать FIND REFERENCES.

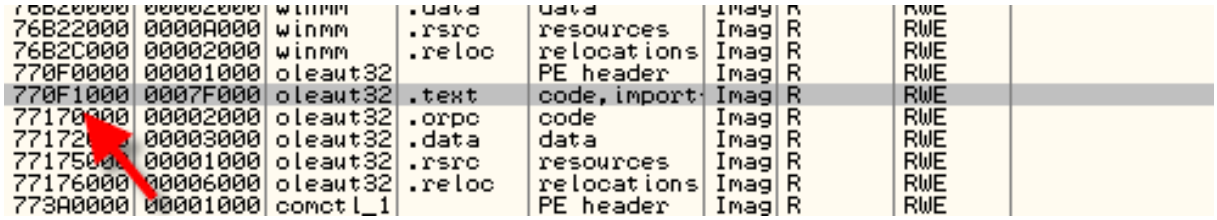


Так что ищем в списке все инструкции, использующие данный элемент (это будет работать всегда: и когда список показывает секцию программы, где собственно выполняется работа, в данном случае первая секцию, если будем искать в ней ссылки, и в списке, отображающем другую секцию, поищем в ней, вероятно, что ничего не найдём, так что нужно проверять перед использованием данного метода, находимся ли мы в правильной секции, где исполняется распакованная программа, или рядом с областью OEP, что одно и то же),



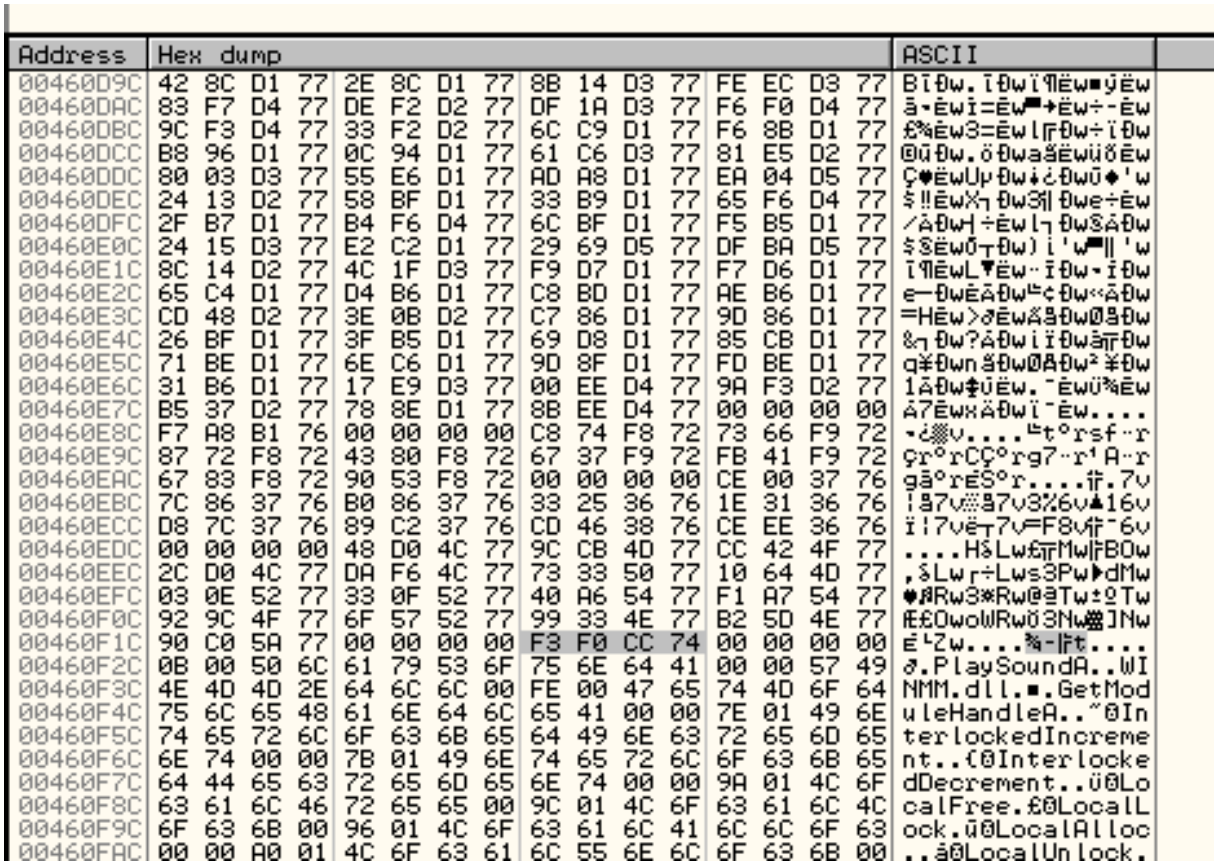
Address	Disassembly	Comment
00405435	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
0041CC2D	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
0041F68C	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
00421197	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
0042133C	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
004213E5	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
0042198F	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
00421D90	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
004477B1	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
004477D8	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
004478B1	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
00447E7D	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
00447F0E	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
00447F67	CALL DWORD PTR DS:[460BA8]	oleaut32.VariantClear
00495284	DD bitarts_.00460BA8	

Здесь видимы CALL’ы, которые есть в первой секции и соответствующие указанному элементу. Та, которая относится к API-функции VariatClear из OleAut32.dll:



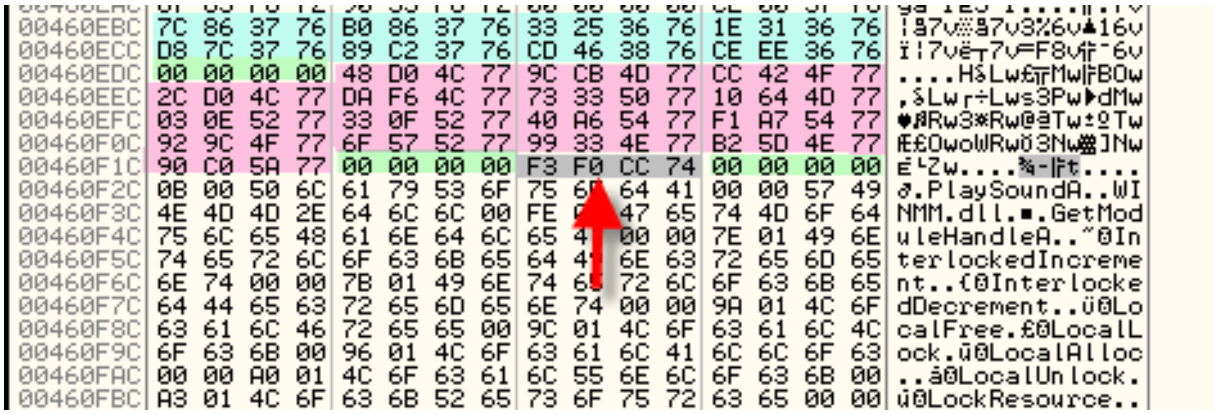
76B22000	0000A000	winmm	.rsrc	resources	Imag	R	RWE
76B2C000	00002000	winmm	.reloc	relocations	Imag	R	RWE
770F0000	00001000	oleaut32		PE header	Imag	R	RWE
770F1000	0007F000	oleaut32	.text	code,import	Imag	R	RWE
77170000	00002000	oleaut32	.orpc	code	Imag	R	RWE
77172000	00003000	oleaut32	.data	data	Imag	R	RWE
77175000	00001000	oleaut32	.rsrc	resources	Imag	R	RWE
77176000	00006000	oleaut32	.reloc	relocations	Imag	R	RWE
773A0000	00001000	comctl_1		PE header	Imag	R	RWE

Смотрим в карте памяти, видим, что, конечно, следующая группа попадает в секцию кода OleAut32.dll.



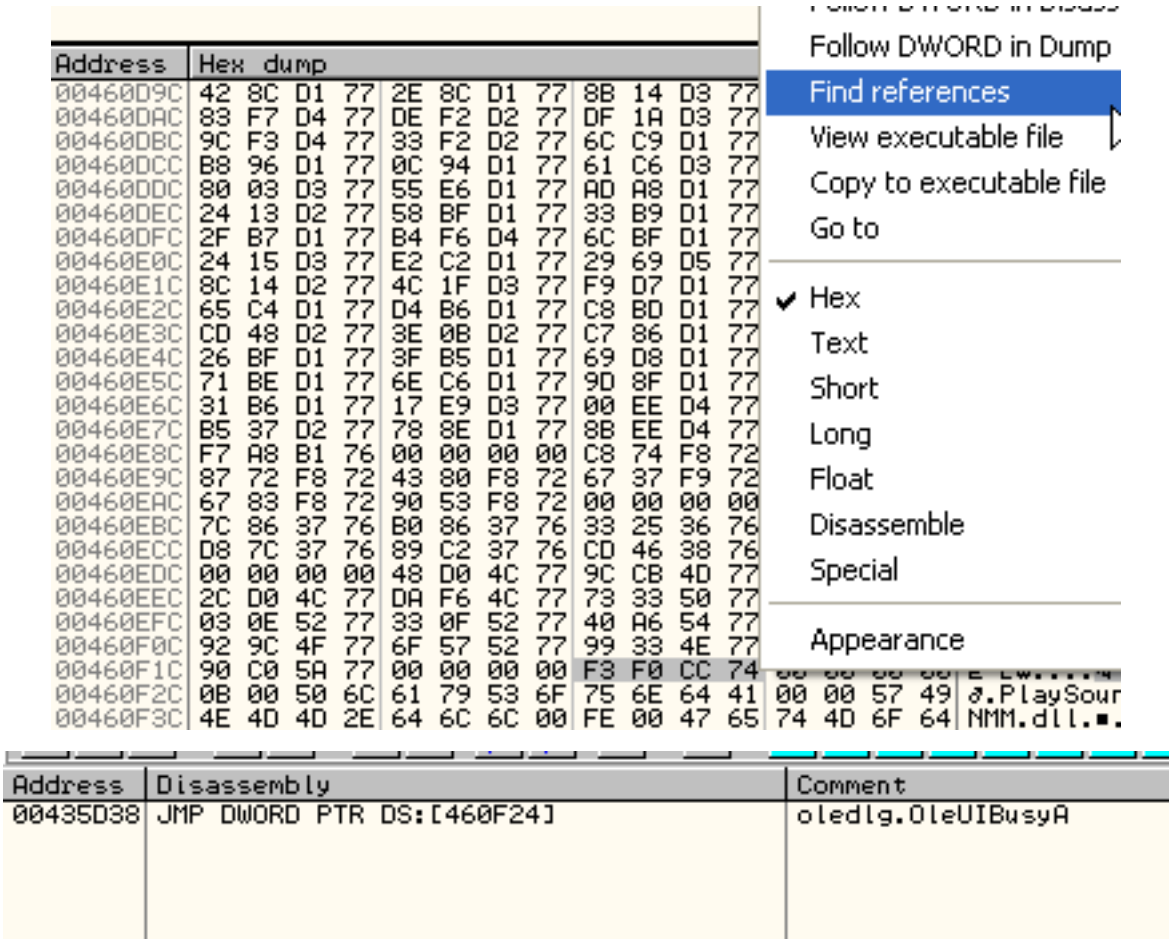
Address	Hex dump	ASCII
00460D9C	42 8C D1 77 2E 8C D1 77 8B 14 D3 77 FE EC D3 77	
00460DAC	83 F7 D4 77 DE F2 D2 77 DF 1A D3 77 F6 F0 D4 77	
00460DBC	9C F3 D4 77 33 F2 D2 77 6C C9 D1 77 F6 8B D1 77	
00460DCC	B8 96 D1 77 0C 94 D1 77 61 C6 D3 77 81 E5 D2 77	
00460DDC	80 03 D3 77 55 E6 D1 77 AD A8 D1 77 EA 04 D5 77	
00460DEC	24 13 D2 77 58 BF D1 77 33 B9 D1 77 65 F6 D4 77	
00460DFC	2F B7 D1 77 B4 F6 D4 77 6C BF D1 77 F5 B5 D1 77	
00460E0C	24 15 D3 77 E2 C2 D1 77 29 69 D5 77 DF BA D5 77	
00460E1C	8C 14 D2 77 4C 1F D3 77 F9 D7 D1 77 F7 D6 D1 77	
00460E2C	65 C4 D1 77 D4 B6 D1 77 C8 BD D1 77 AE B6 D1 77	
00460E3C	CD 48 D2 77 3E 0B D2 77 C7 86 D1 77 9D 86 D1 77	
00460E4C	26 BF D1 77 3F B5 D1 77 69 D8 D1 77 85 CB D1 77	
00460E5C	71 BE D1 77 6E C6 D1 77 9D 8F D1 77 FD BE D1 77	
00460E6C	31 B6 D1 77 17 E9 D3 77 00 EE D4 77 9A F3 D2 77	
00460E7C	B5 37 D2 77 78 8E D1 77 8B EE D4 77 00 00 00 00	
00460E8C	F7 A8 B1 76 00 00 00 00 C8 74 F8 72 73 66 F9 72	
00460E9C	87 72 F8 72 43 80 F8 72 67 37 F9 72 FB 41 F9 72	
00460EAC	67 83 F8 72 90 53 F8 72 00 00 00 00 CE 00 37 76	
00460EBC	7C 86 37 76 B0 86 37 76 33 25 36 76 1E 31 36 76	
00460ECC	D8 7C 37 76 89 C2 37 76 CD 46 38 76 CE EE 36 76	
00460EDC	00 00 00 00 48 D0 4C 77 9C CB 4D 77 CC 42 4F 77	
00460EEC	2C D0 4C 77 DA F6 4C 77 73 33 50 77 10 64 4D 77	
00460EFC	03 0E 52 77 33 0F 52 77 40 A6 54 77 F1 A7 54 77	
00460F0C	92 9C 4F 77 6F 57 52 77 99 33 4E 77 B2 5D 4E 77	
00460F1C	90 C0 5A 77 00 00 00 00 F3 F0 CC 74 00 00 00 00	
00460F2C	0B 00 50 6C 61 79 53 6F 75 6E 64 41 00 00 57 49	
00460F3C	4E 4D 4D 2E 64 6C 6C 00 FE 00 47 65 74 4D 6F 64	
00460F4C	75 6C 65 48 61 6E 64 6C 65 41 00 00 7E 01 49 6E	
00460F5C	74 65 72 6C 6F 63 6B 65 64 49 6E 63 72 65 6D 65	
00460F6C	6E 74 00 00 7B 01 49 6E 74 65 72 6C 6F 63 6B 65	
00460F7C	64 44 65 63 72 65 6D 65 6E 74 00 00 9A 01 4C 6F	
00460F8C	63 61 6C 46 72 65 65 00 9C 01 4C 6F 63 61 6C 4C	
00460F9C	6F 63 6B 00 96 01 4C 6F 63 61 6C 41 6C 6C 6F 63	
00460FAC	00 00 A0 01 4C 6F 63 61 6C 55 6E 6C 6F 63 6B 00	

Если продолжим спускаться без указания каждой DLL, то увидим группы элементов, ведущие в другие элементы, разделение нулями, другую группу, следующее разделение, и так пока не дойдём до самого конца этой IAT-схемы.

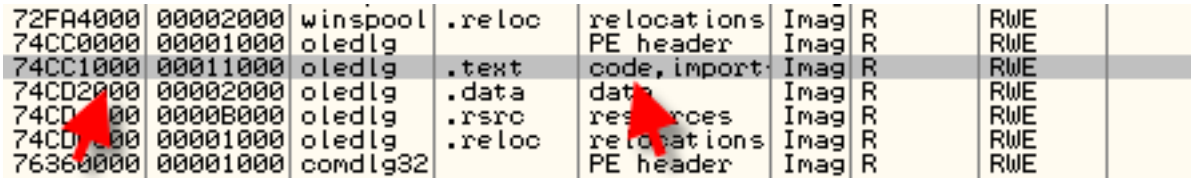


00460EBC	7C 86 37 76 B0 86 37 76 33 25 36 76 1E 31 36 76	
00460ECC	D8 7C 37 76 89 C2 37 76 CD 46 38 76 CE EE 36 76	
00460EDC	00 00 00 00 48 D0 4C 77 9C CB 4D 77 CC 42 4F 77	
00460EEC	2C D0 4C 77 DA F6 4C 77 73 33 50 77 10 64 4D 77	
00460EFC	03 0E 52 77 33 0F 52 77 40 A6 54 77 F1 A7 54 77	
00460F0C	92 9C 4F 77 6F 57 52 77 99 33 4E 77 B2 5D 4E 77	
00460F1C	90 C0 5A 77 00 00 00 00 F3 F0 CC 74 00 00 00 00	
00460F2C	0B 00 50 6C 61 79 53 6F 75 6E 64 41 00 00 57 49	
00460F3C	4E 4D 4D 2E 64 6C 6C 00 FE 00 47 65 74 4D 6F 64	
00460F4C	75 6C 65 48 61 6E 64 6C 65 41 00 00 7E 01 49 6E	
00460F5C	74 65 72 6C 6F 63 6B 65 64 49 6E 63 72 65 6D 65	
00460F6C	6E 74 00 00 7B 01 49 6E 74 65 72 6C 6F 63 6B 65	
00460F7C	64 44 65 63 72 65 6D 65 6E 74 00 00 9A 01 4C 6F	
00460F8C	63 61 6C 46 72 65 65 00 9C 01 4C 6F 63 61 6C 4C	
00460F9C	6F 63 6B 00 96 01 4C 6F 63 61 6C 41 6C 6C 6F 63	
00460FAC	00 00 A0 01 4C 6F 63 61 6C 55 6E 6C 6F 63 6B 00	
00460FBC	A3 01 4C 6F 63 6B 52 65 73 6F 75 72 63 65 00 00	

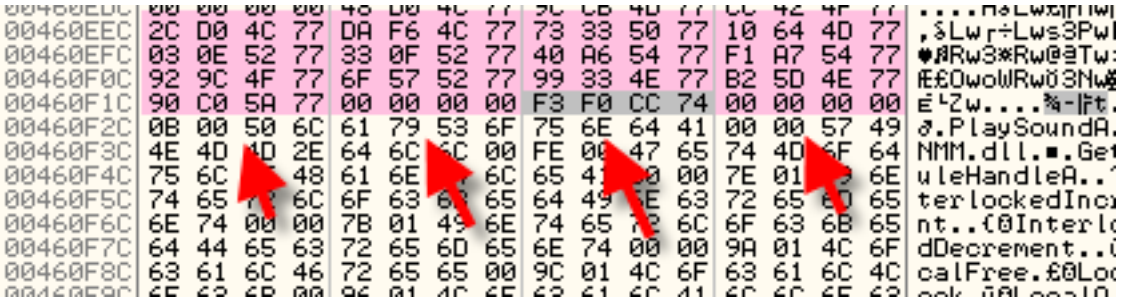
Здесь видим завершаю часть, лазурным цветом выделена группа API-функций, соответствующих некоей DLL, затем – разделение, затем розовым цветом обозначена другая группа, разделение, а после – элемент, отмеченный стрелкой и ещё одно разделение, после которого привычная схема IAT уже не соблюдается. Мы можем проверить, является ли отмеченный стрелкой элемент ссылкой на API-функцию. Это можно сделать двумя способами.



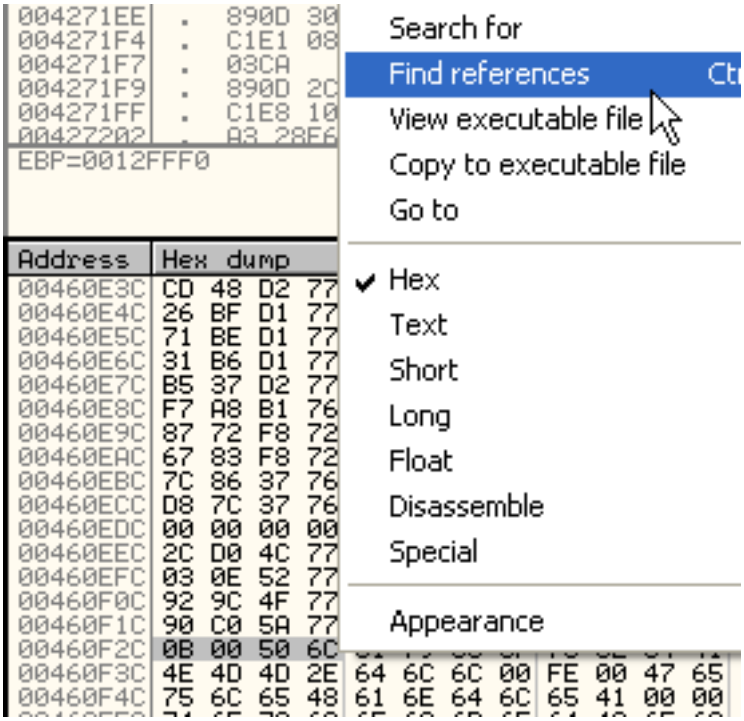
Видим, что есть элемент IAT, который приводит нас в данном случае к OleUiBusyA из oledlg.dll. Проверим через карту памяти.

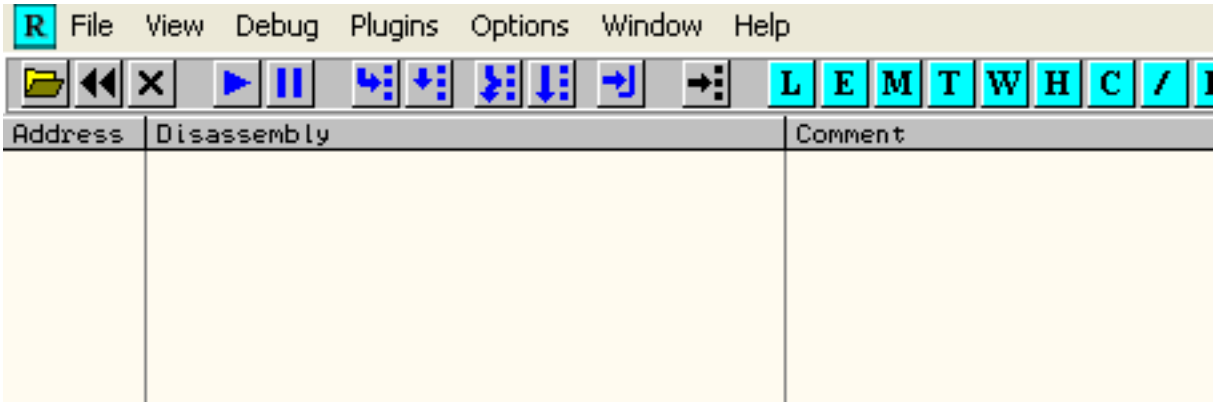


Логично, что элемент ведёт в секцию кода указанной DLL, поэтому это элемент IAT, единственный для данной DLL. Далее видим разделение, а после него группу множества переходов, которые ни сгруппированы так, чтобы указывать на последовательные адреса, ни сами эти адреса не указывают на какую-либо DLL.



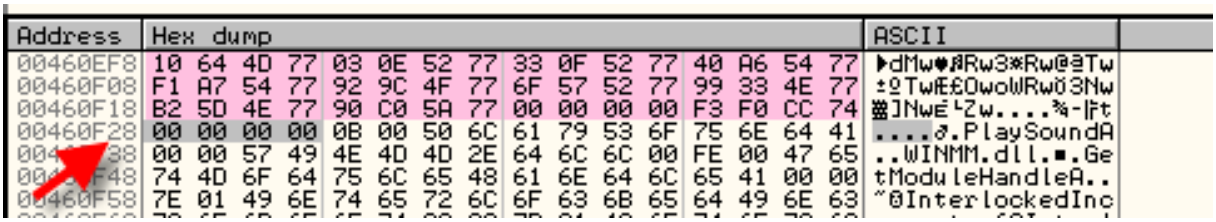
Если отметим какой-нибудь из них и сделаем FIND REFERENCES:



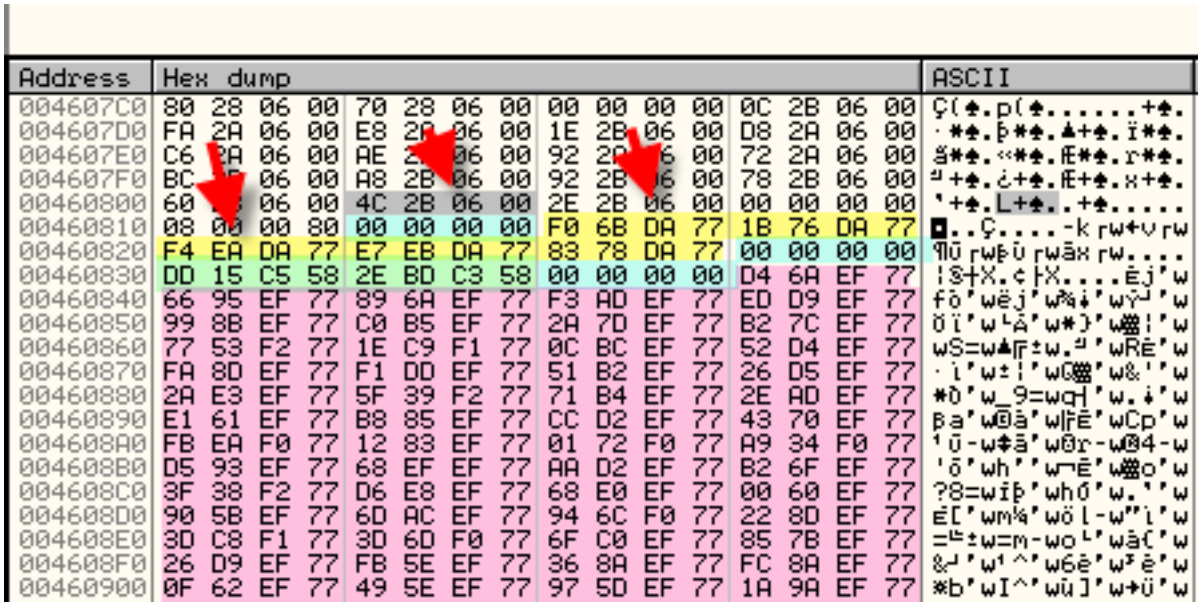


Никаких результатов, приведших бы нас к API-функциям, поэтому можно прийти к выводу, что ниже мы также не встретим элементов, ссылающихся на API-функции, то есть концом IAT является:

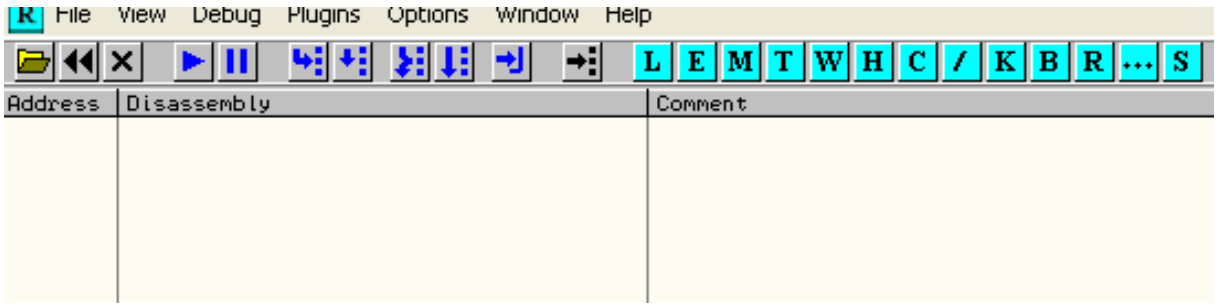
КОНЕЦ: 460F28



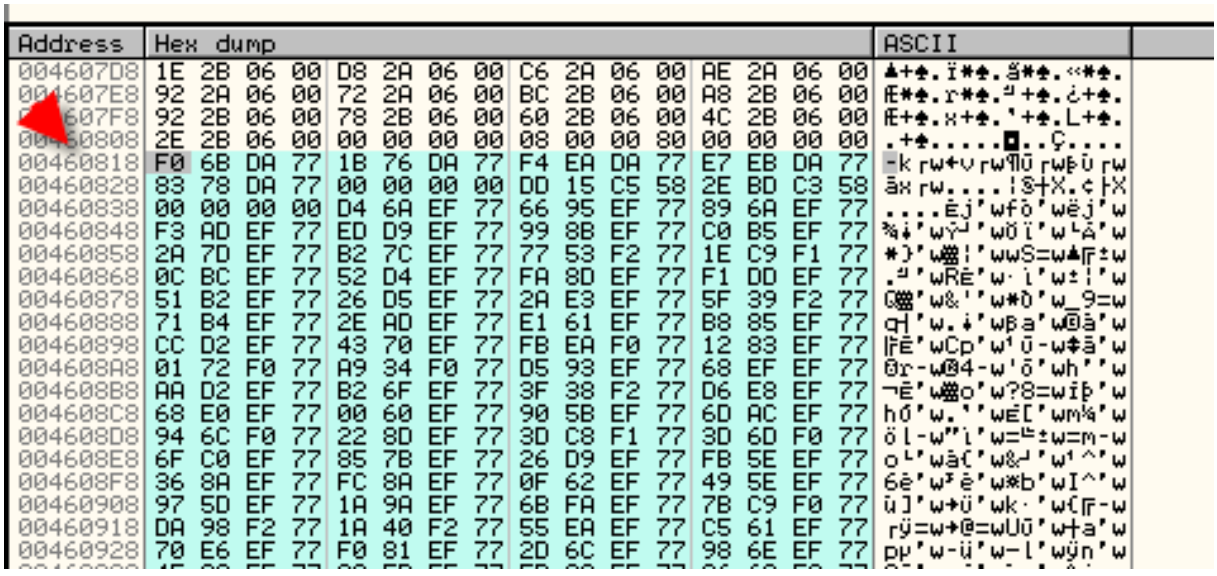
Теперь проверим, откуда начинается IAT-схема.



Идём вверх и видим, что схема повторяется, пока не доходим до сюда. Лазурным отмечены разделения, группа, отмеченная жёлтым, опознаётся как ведущая в одну и ту же DLL, далее есть одно разделение, а затем элемент, ведущий в 80000008. Так как не знаем, соответствует ли он какой-либо DLL, то проверяем это с помощью FIND REFERENCES.



Никаких результатов ни для этого элемента, ни для идущих перед ним, так что можем быть уверены, что первым элементом является 460818.



Теперь у нас есть начало и конец IAT. Вычисляем размер:

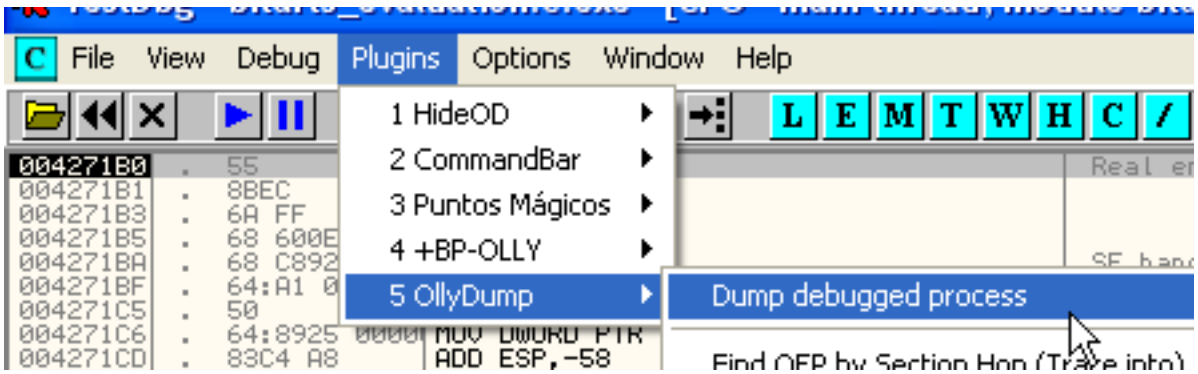
НАЧАЛО – КОНЕЦ = 460F28 – 460818

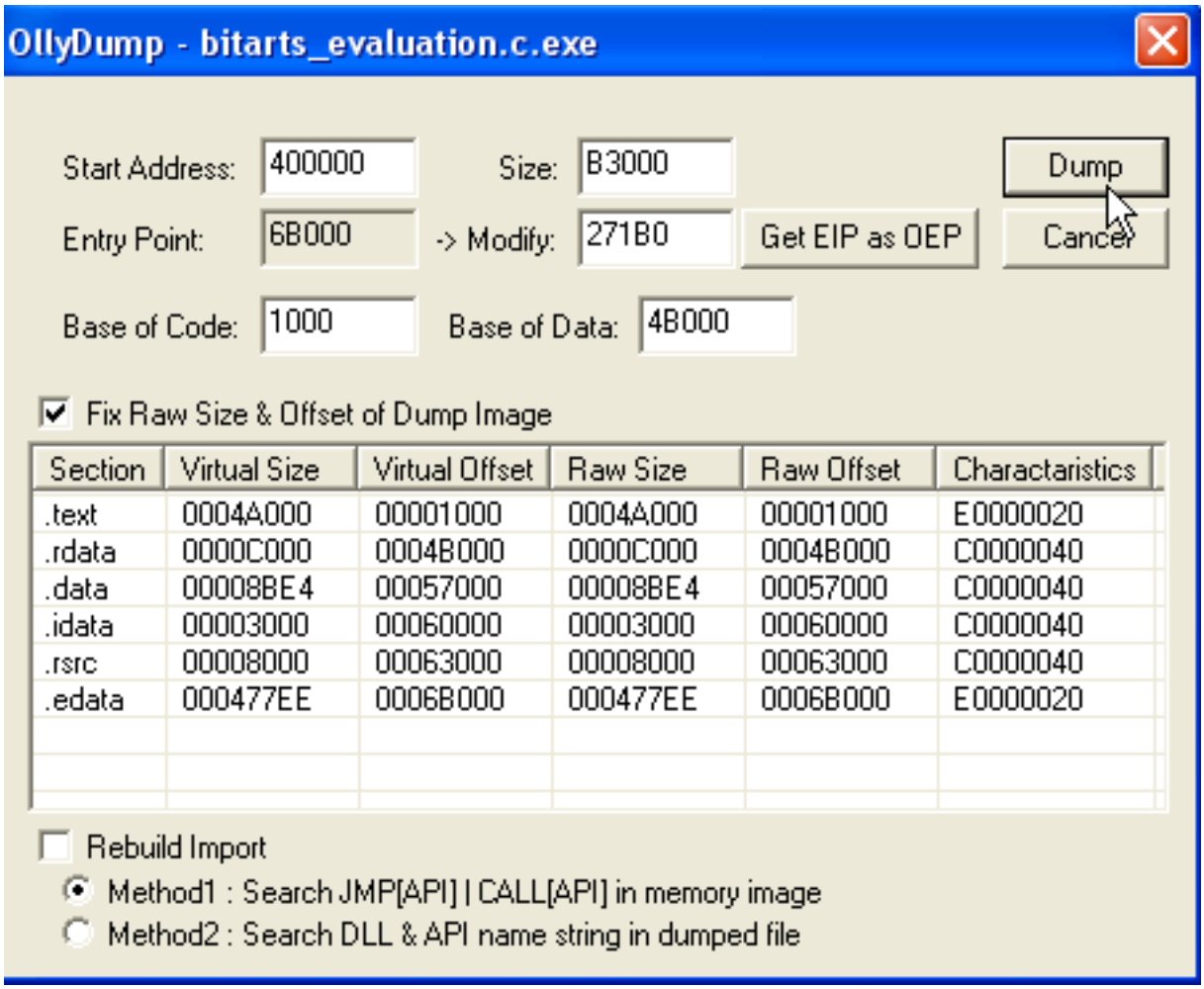
То есть размер равен 710.

Данные для IMP REC (от OEP и НАЧАЛА надо отнять базу образа):

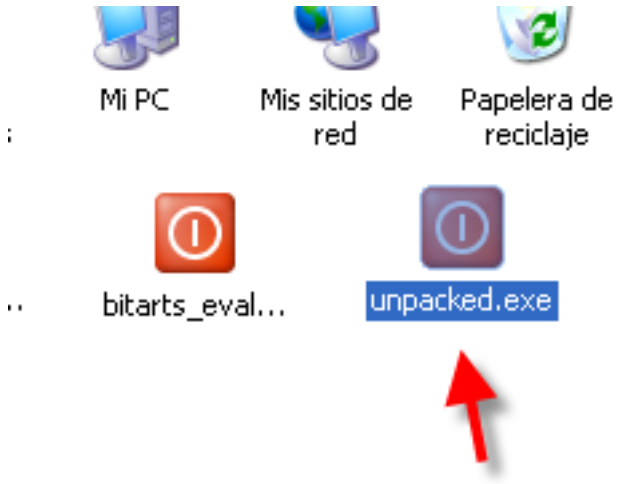
```
OEP = 271B0
НАЧАЛО или RVA = 60818
РАЗМЕР = 710
```

Хорошо, теперь сделаем дамп с помощью OllyDump.

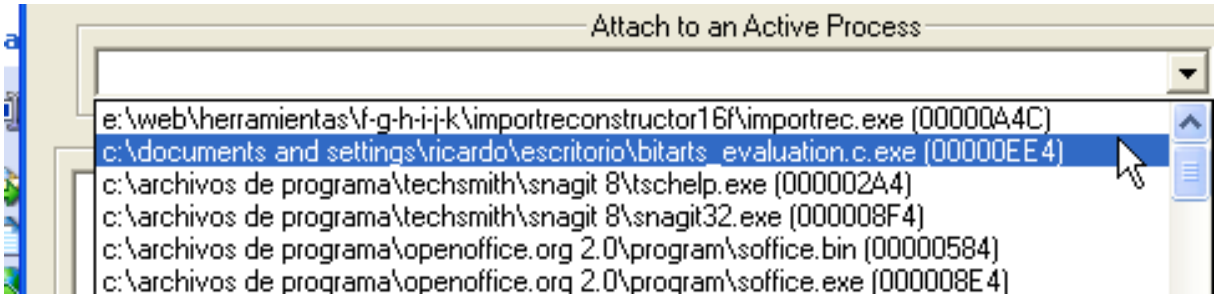




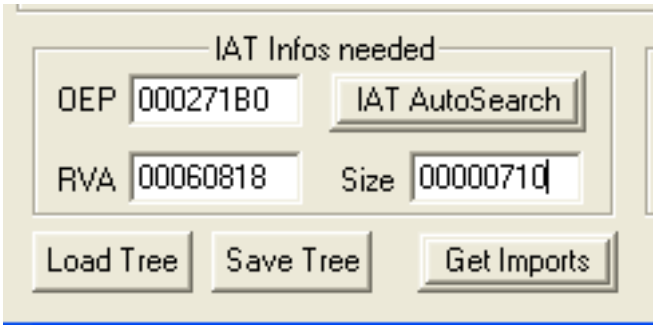
Убираем галочку с Rebuild Import и дампим.



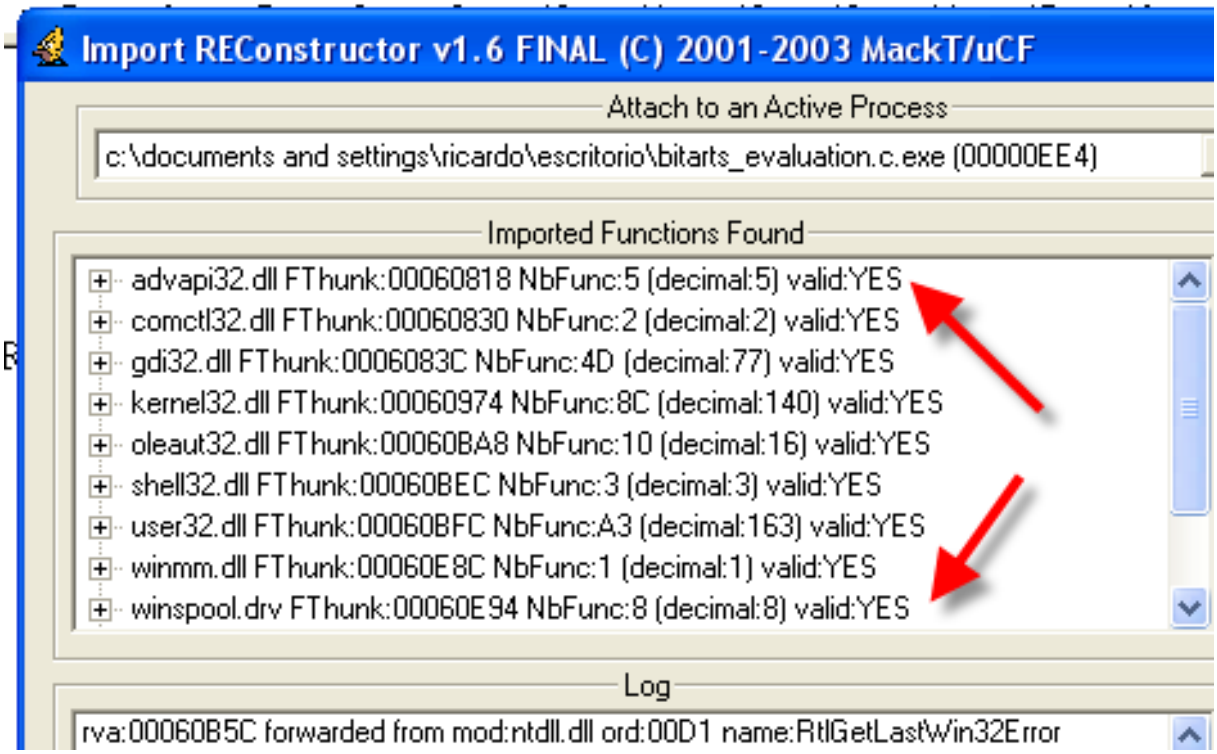
Далее открываем IMP REC без закрытия OllyDbg, в котором загружен исходный файл, остановленный на OEP.



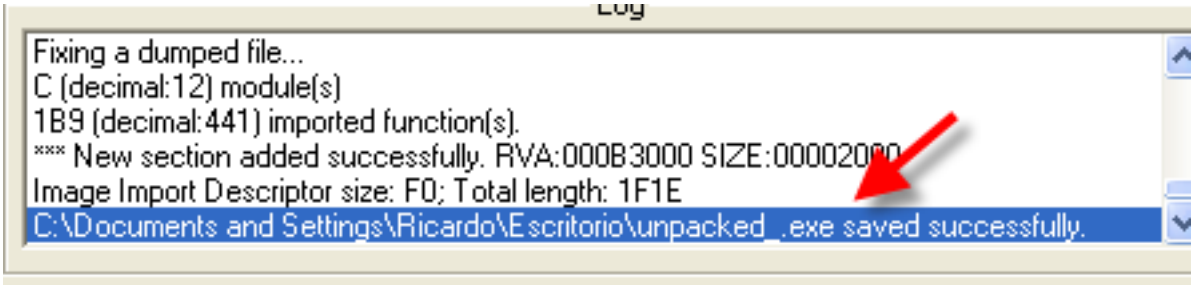
Ищем процесс в выпадающем меню и устанавливаем найденные значения.



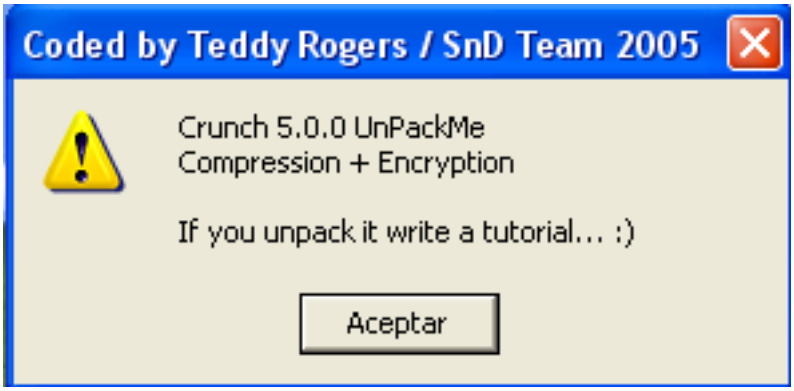
И нажимаем GET IMPORTS.



Видим, что упаковщик не делает ничего особенного, чтобы затруднить нам работу здесь, все DLL правильны, так что нажимаем FIX DUMP и ищем дамп для починки.

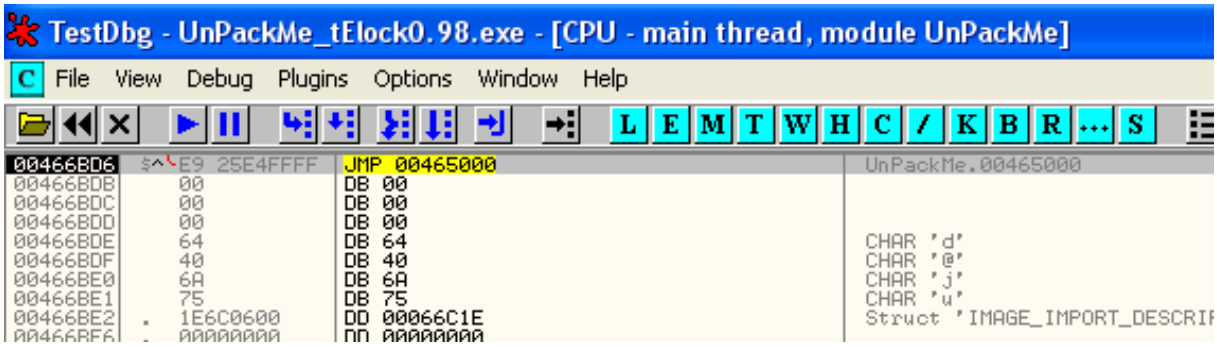


Починенный файл сохраняется под именем unpacked_.exe, смотрим, работает ли он.

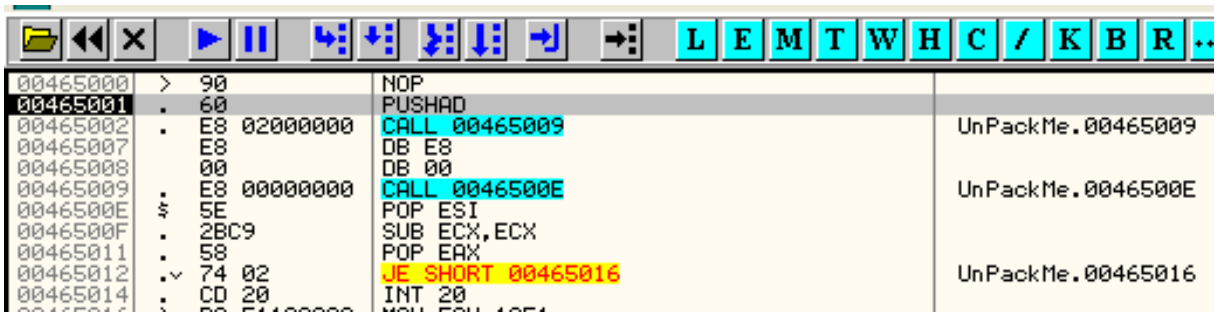


Работает прекрасно, то есть не используются антидамповые приёмы, которые, как правило, можно встретить в более продвинутых упаковщиках.

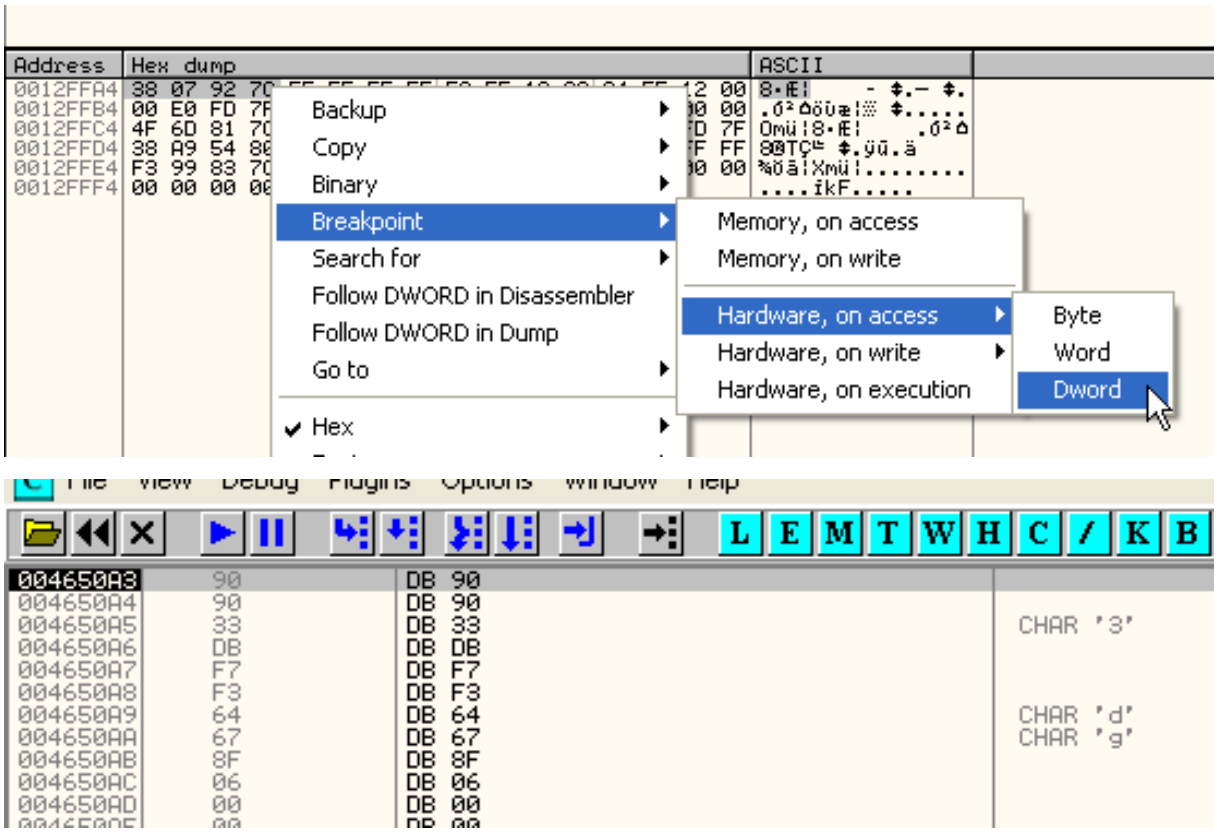
Следующая жертва – это telock 0.98, которая послужит нам для того, чтобы открыть тему о переадресовочных элементах IAT.



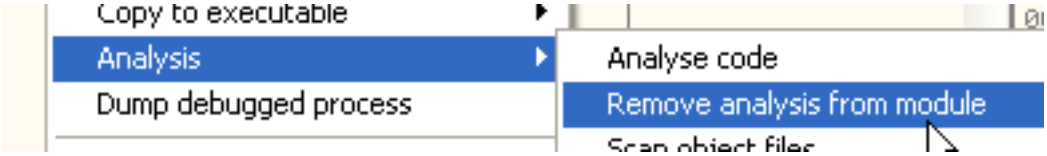
Пробуем метод PUSHAD, трассируем немного, чтобы понять, есть ли здесь какой-нибудь PUSHAD.



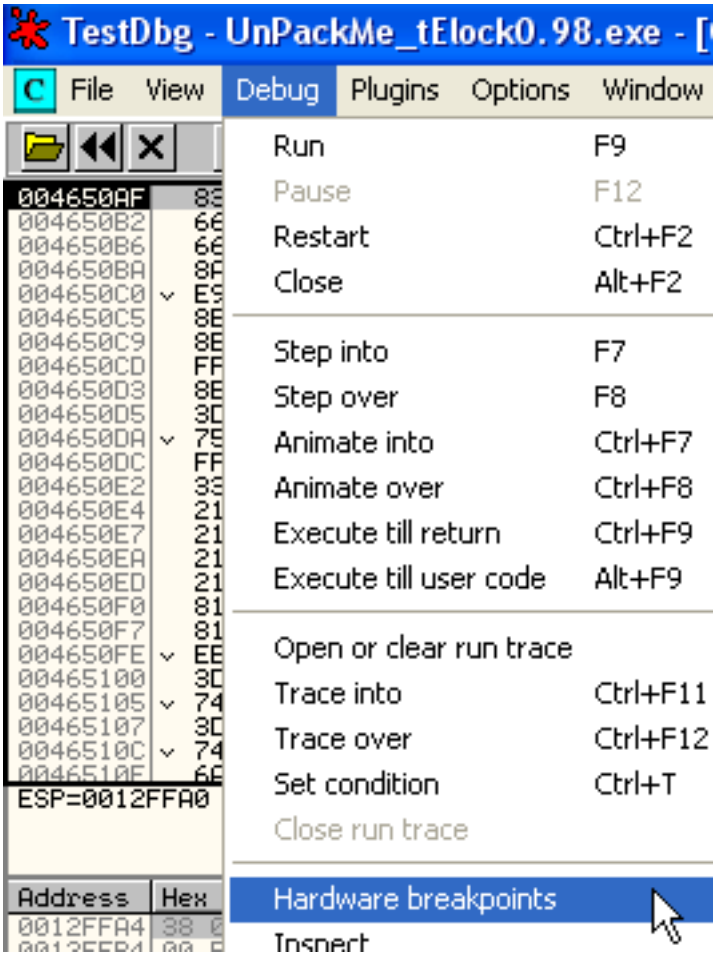
Проходим его с помощью F7, затем ESP-FOLLOW IN DUMP.

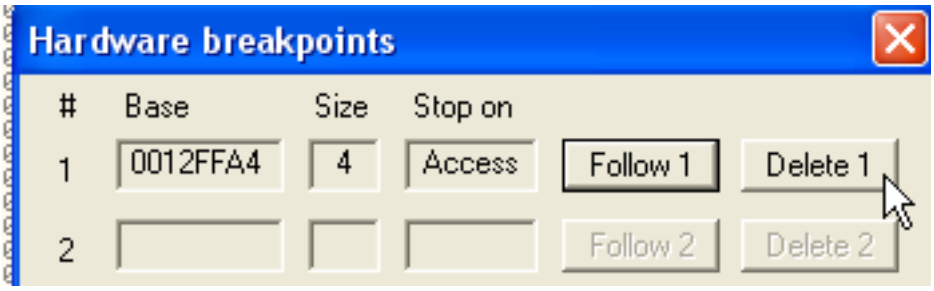


Убираем анализ, чтобы посмотреть код.



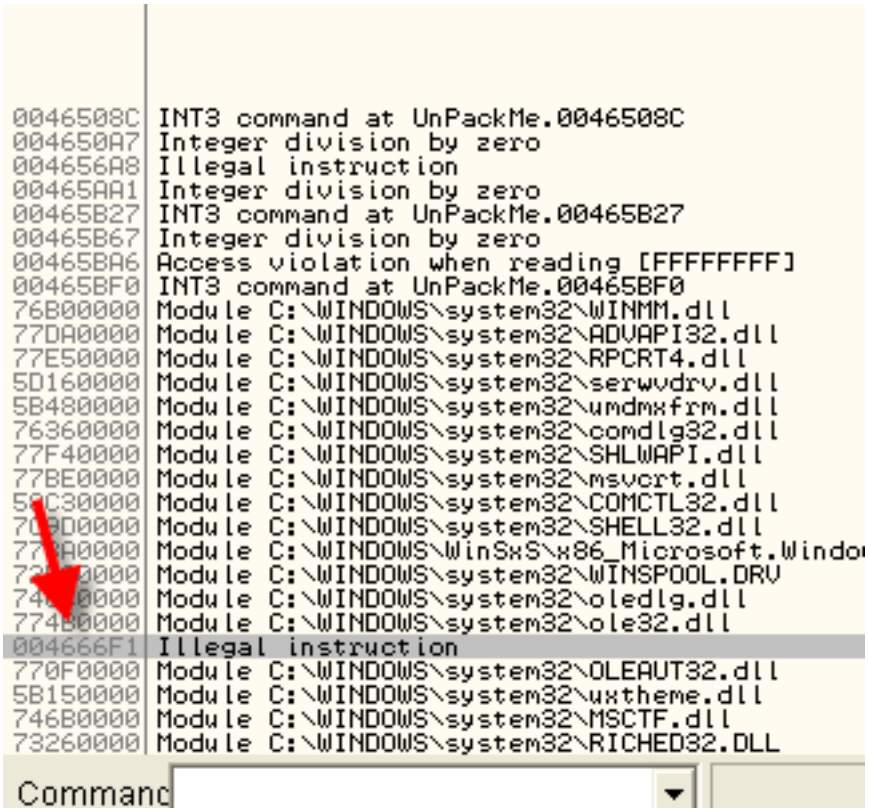
Видим, что метод PUSHAD не работает, а также есть защита от точек аппаратного останова, потому что при их использовании возникнет ошибка, так что убираем их.



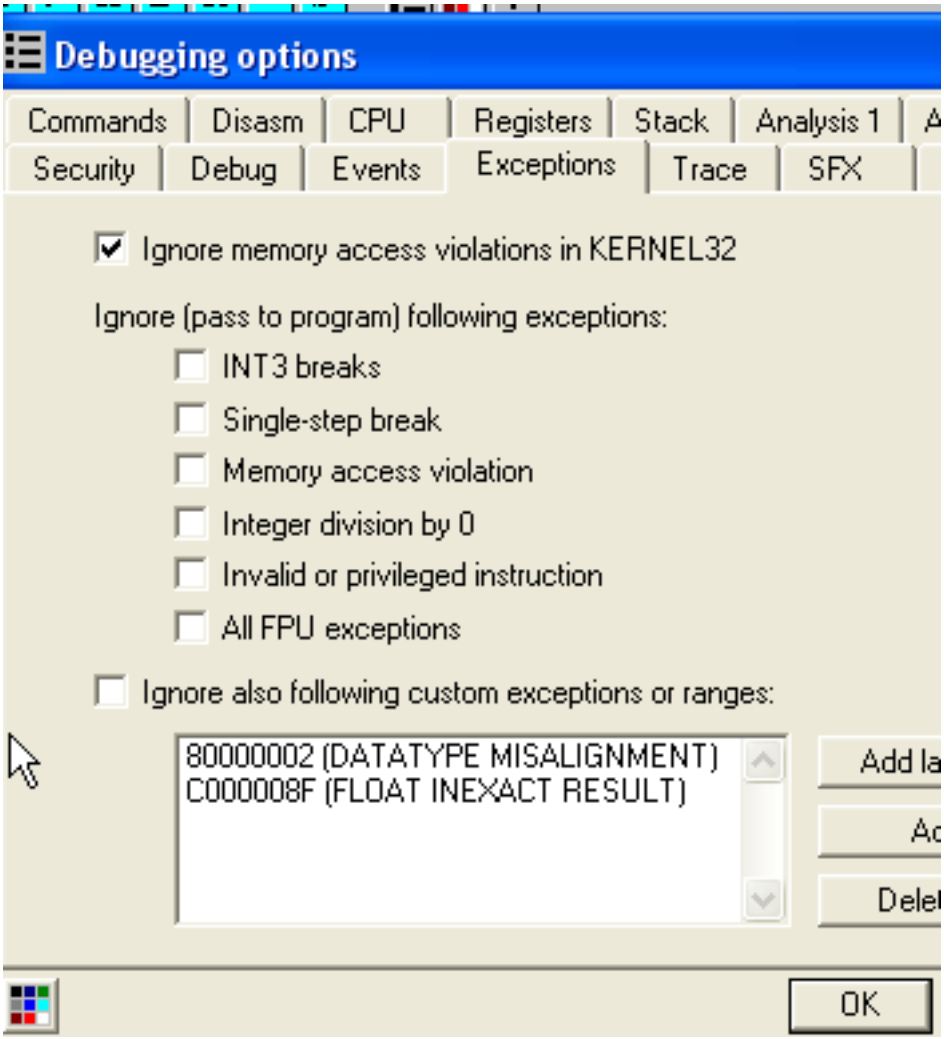


И перезапускаем OllyDbg.

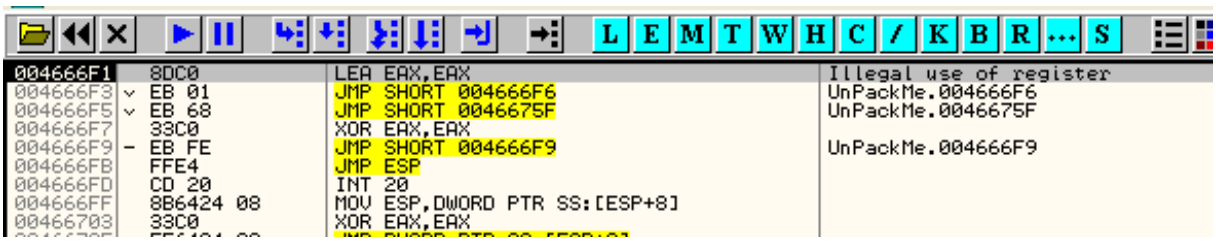
Так, этот метод не сработал, пробуем основанный на исключениях. Очищаем LOG и запускаем программу, чтобы отобразились случившиеся из них.



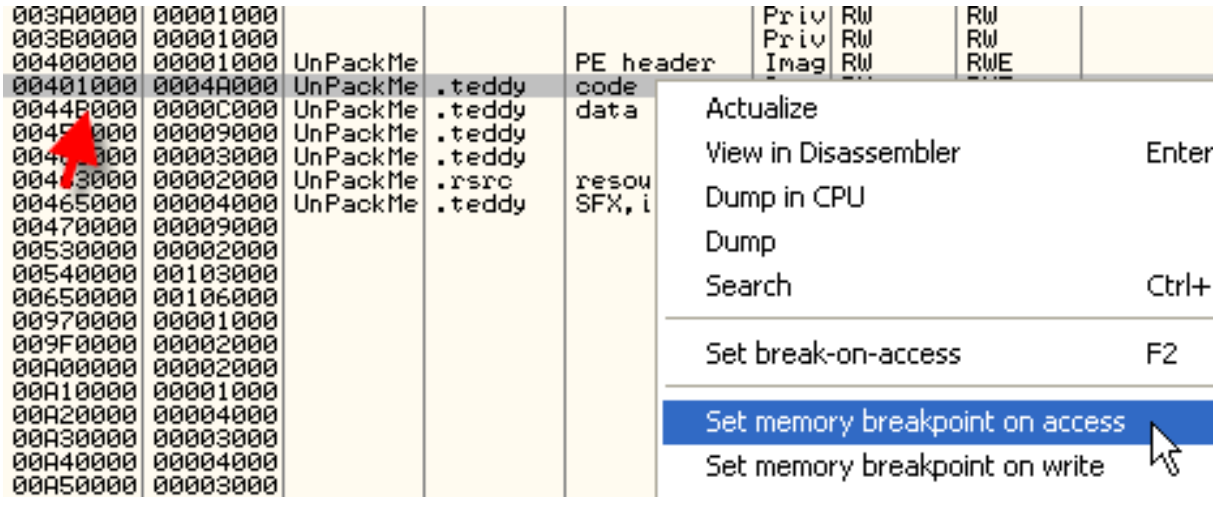
Убираем галочки с исключений и перезапускаем, пытаясь отловить 4666a1, которая является последней в распаковщике.



После прохода через несколько исключений с помощью SHIFT+F9, доходим до 4666f1.

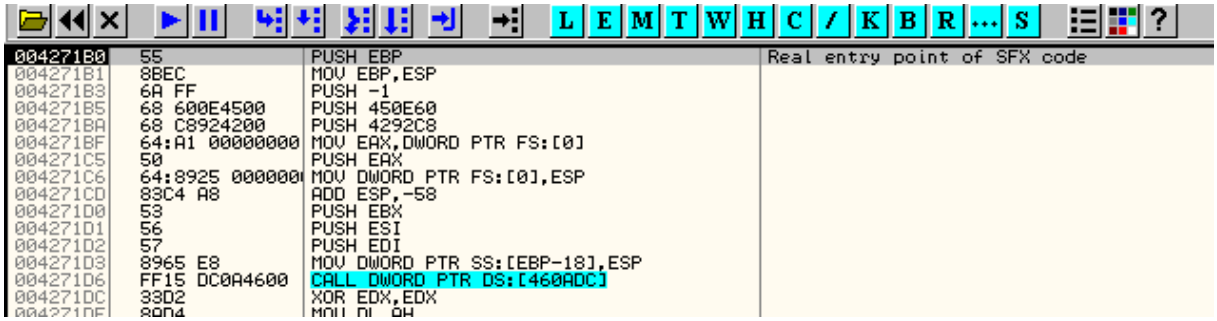


Теперь устанавливаем MEMORY BREAKPOINT ON ACCESS на первую секцию.



Помним, что нужно нажать SHIFT+F9, чтобы миновать исключение, на котором остановились, без ошибок.

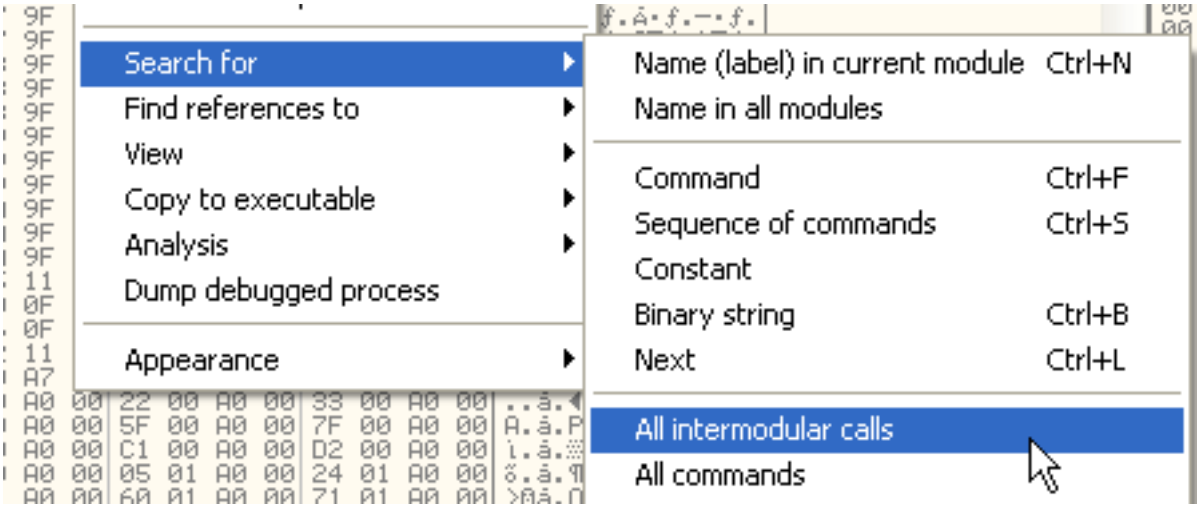
Ок, останавливаемся на паре исключений типа SINGLE STEP и прибываем в OEP в первой секции, где оказываемся на MEMORY BREAKPOINT ON EXECUTION.



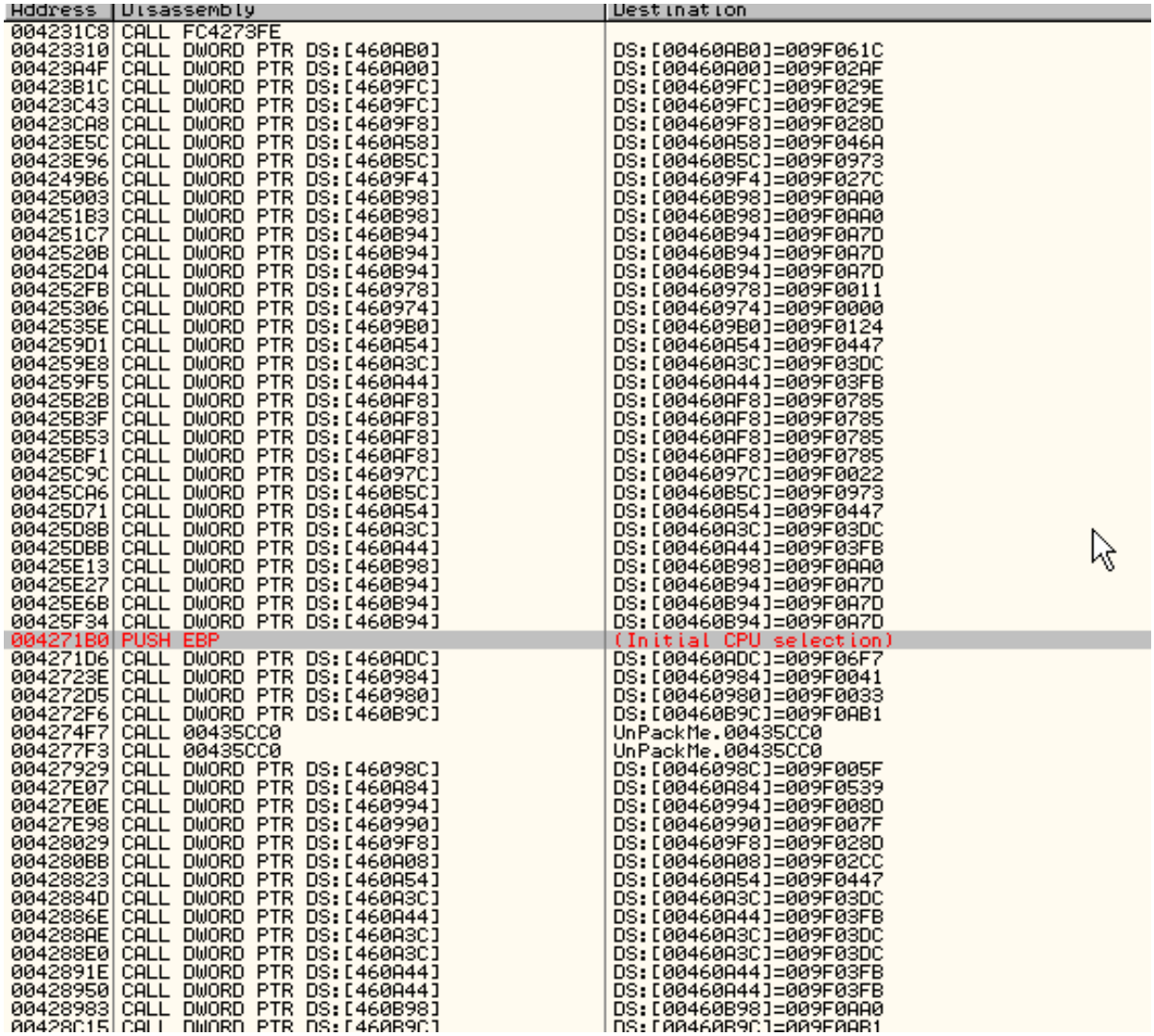
Значит, OEP равен 4271B0, такой же, как и у предыдущего примера CRUNCH, поэтому похоже, что это та же программа, упакованная другим пакером, но всё равно будем работать с ней, как если бы мы ничего не знали о ней.

Ок, в предыдущем примере мы знали, что этот видимый вызов ведёт на API-функцию GetVersion, но в данном случае это не так, здесь мы начинаем тему переадресовочных элементов, хе-хе.

Если сделаем



для того, чтобы увидеть вызовы на другие секции и выяснить, есть ли вызовы каких-либо API-функций.



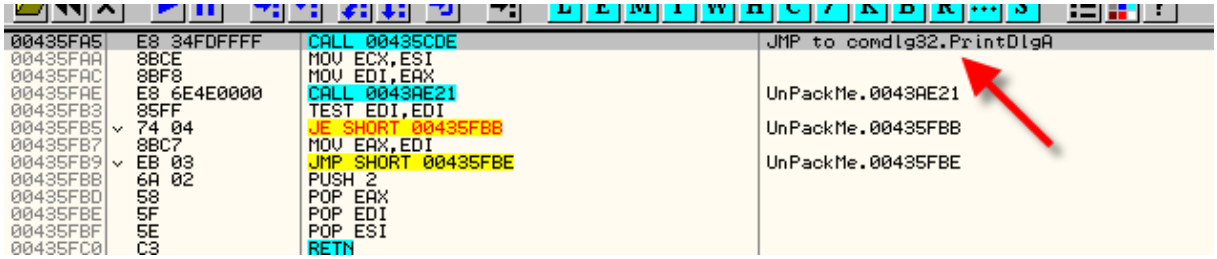
Address	Disassembly	Destination
004231C8	CALL FC4273FE	
00423310	CALL DWORD PTR DS:[460AB0]	DS:[00460AB0]=009F061C
00423A4F	CALL DWORD PTR DS:[460A00]	DS:[00460A00]=009F02AF
00423B1C	CALL DWORD PTR DS:[4609FC]	DS:[004609FC]=009F029E
00423C43	CALL DWORD PTR DS:[4609FC]	DS:[004609FC]=009F029E
00423CA8	CALL DWORD PTR DS:[4609F8]	DS:[004609F8]=009F028D
00423E5C	CALL DWORD PTR DS:[460A58]	DS:[00460A58]=009F046A
00423E96	CALL DWORD PTR DS:[460B5C]	DS:[00460B5C]=009F0973
004249B6	CALL DWORD PTR DS:[4609F4]	DS:[004609F4]=009F027C
00425003	CALL DWORD PTR DS:[460B98]	DS:[00460B98]=009F0AA0
004251B3	CALL DWORD PTR DS:[460B98]	DS:[00460B98]=009F0AA0
004251C7	CALL DWORD PTR DS:[460B94]	DS:[00460B94]=009F0A7D
0042520B	CALL DWORD PTR DS:[460B94]	DS:[00460B94]=009F0A7D
004252D4	CALL DWORD PTR DS:[460B94]	DS:[00460B94]=009F0A7D
004252FB	CALL DWORD PTR DS:[460978]	DS:[00460978]=009F0011
00425306	CALL DWORD PTR DS:[460974]	DS:[00460974]=009F0000
0042535E	CALL DWORD PTR DS:[4609B0]	DS:[004609B0]=009F0124
004259D1	CALL DWORD PTR DS:[460A54]	DS:[00460A54]=009F0447
004259E8	CALL DWORD PTR DS:[460A3C]	DS:[00460A3C]=009F03DC
004259F5	CALL DWORD PTR DS:[460A44]	DS:[00460A44]=009F03FB
00425B2B	CALL DWORD PTR DS:[460AF8]	DS:[00460AF8]=009F0785
00425B3F	CALL DWORD PTR DS:[460AF8]	DS:[00460AF8]=009F0785
00425B53	CALL DWORD PTR DS:[460AF8]	DS:[00460AF8]=009F0785
00425BF1	CALL DWORD PTR DS:[460AF8]	DS:[00460AF8]=009F0785
00425C9C	CALL DWORD PTR DS:[46097C]	DS:[0046097C]=009F0022
00425CA6	CALL DWORD PTR DS:[460B5C]	DS:[00460B5C]=009F0973
00425D71	CALL DWORD PTR DS:[460A54]	DS:[00460A54]=009F0447
00425D8B	CALL DWORD PTR DS:[460A3C]	DS:[00460A3C]=009F03DC
00425DBB	CALL DWORD PTR DS:[460A44]	DS:[00460A44]=009F03FB
00425E13	CALL DWORD PTR DS:[460B98]	DS:[00460B98]=009F0AA0
00425E27	CALL DWORD PTR DS:[460B94]	DS:[00460B94]=009F0A7D
00425E6B	CALL DWORD PTR DS:[460B94]	DS:[00460B94]=009F0A7D
00425F34	CALL DWORD PTR DS:[460B94]	DS:[00460B94]=009F0A7D
004271B0	PUSH EBP	(Initial CPU selection)
004271D6	CALL DWORD PTR DS:[460ADC]	DS:[00460ADC]=009F06F7
0042723E	CALL DWORD PTR DS:[460984]	DS:[00460984]=009F0041
004272D5	CALL DWORD PTR DS:[460980]	DS:[00460980]=009F0033
004272F6	CALL DWORD PTR DS:[460B9C]	DS:[00460B9C]=009F0AB1
004274F7	CALL 00435CC0	UnPackMe.00435CC0
004277F3	CALL 00435CC0	UnPackMe.00435CC0
00427929	CALL DWORD PTR DS:[46098C]	DS:[0046098C]=009F005F
00427E07	CALL DWORD PTR DS:[460A84]	DS:[00460A84]=009F0539
00427E0E	CALL DWORD PTR DS:[460994]	DS:[00460994]=009F008D
00427E98	CALL DWORD PTR DS:[460990]	DS:[00460990]=009F007F
00428029	CALL DWORD PTR DS:[4609F8]	DS:[004609F8]=009F028D
004280BB	CALL DWORD PTR DS:[460A08]	DS:[00460A08]=009F02CC
00428823	CALL DWORD PTR DS:[460A54]	DS:[00460A54]=009F0447
0042884D	CALL DWORD PTR DS:[460A3C]	DS:[00460A3C]=009F03DC
0042886E	CALL DWORD PTR DS:[460A44]	DS:[00460A44]=009F03FB
004288AE	CALL DWORD PTR DS:[460A3C]	DS:[00460A3C]=009F03DC
004288E0	CALL DWORD PTR DS:[460A3C]	DS:[00460A3C]=009F03DC
0042891E	CALL DWORD PTR DS:[460A44]	DS:[00460A44]=009F03FB
00428950	CALL DWORD PTR DS:[460A44]	DS:[00460A44]=009F03FB
00428983	CALL DWORD PTR DS:[460B98]	DS:[00460B98]=009F0AA0
00428C15	CALL DWORD PTR DS:[460B9C]	DS:[00460B9C]=009F0AB1

Видим, что множество косвенных вызовов, которые вместо того, чтобы указывать на API-функции, ведут на моей машине в секцию 9fxxxx. Очевидно, что на других машинах эти адреса могут меняться и иметь другие значения.

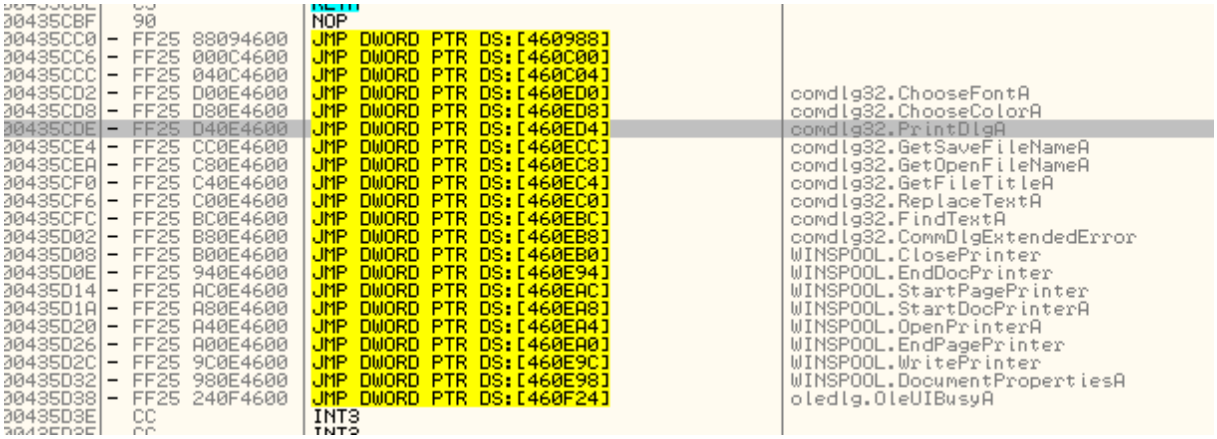
Если посмотрим, что в этом списке ниже:

```
00435D96 CALL DWORD PTR DS:[460B74]      DS:[00460B74]=009F09F0
00435DF1 CALL DWORD PTR DS:[460AB0]      DS:[00460AB0]=009F061C
00435E06 CALL DWORD PTR DS:[460AB4]      DS:[00460AB4]=009F0636
00435FA5 CALL 00435CDE                   comdlg32.PrintDlgA
00436004 CALL DWORD PTR DS:[460AD0]      DS:[00460AD0]=009F06B5
0043602E CALL DWORD PTR DS:[460AD0]      DS:[00460AD0]=009F06B5
0043606C CALL DWORD PTR DS:[460AD0]      DS:[00460AD0]=009F06B5
004360AB CALL DWORD PTR DS:[460AD0]      DS:[00460AD0]=009F06B5
004360D1 CALL 00435CDE                   comdlg32.PrintDlgA
0043611B CALL DWORD PTR DS:[460878]      DS:[00460878]=00A10124
0043623D CALL DWORD PTR DS:[460B74]      DS:[00460B74]=009F09F0
004362EC CALL DWORD PTR DS:[460CAC]      DS:[00460CAC]=00A00360
00436315 CALL DWORD PTR DS:[460CD4]      DS:[00460CD4]=00A00419
00436351 CALL 00435CEA                   comdlg32.GetOpenFileNameA
00436358 CALL 00435CE4                   comdlg32.GetSaveFileNameA
00436374 CALL DWORD PTR DS:[460DF4]      DS:[00460DF4]=00A009B7
00436382 CALL DWORD PTR DS:[460CB0]      DS:[00460CB0]=00A00370
00436407 CALL DWORD PTR DS:[460E78]      DS:[00460E78]=00A00C52
0043644E CALL DWORD PTR DS:[460E78]      DS:[00460E78]=00A00C52
004366CA CALL DWORD PTR DS:[460DFC]      DS:[00460DFC]=00A009F0
004368C5 CALL DWORD PTR DS:[460D48]      DS:[00460D48]=00A00669
004368DB CALL DWORD PTR DS:[460D48]      DS:[00460D48]=00A00669
004368F1 CALL DWORD PTR DS:[460D48]      DS:[00460D48]=00A00669
00436907 CALL DWORD PTR DS:[460D48]      DS:[00460D48]=00A00669
0043691D CALL DWORD PTR DS:[460D48]      DS:[00460D48]=00A00669
00436933 CALL DWORD PTR DS:[460D48]      DS:[00460D48]=00A00669
00436A6A CALL DWORD PTR DS:[460E78]      DS:[00460E78]=00A00C52
00436BD0 CALL 00435CFC                   comdlg32.FindTextA
00436BD7 CALL 00435CF6                   comdlg32.ReplaceTextA
00436CB1 CALL DWORD PTR DS:[460AA8]      DS:[00460AA8]=009F05EC
00436CC3 CALL DWORD PTR DS:[460AAC]      DS:[00460AAC]=009F05FD
0043799F CALL DWORD PTR DS:[460D48]      DS:[00460D48]=00A00669
```

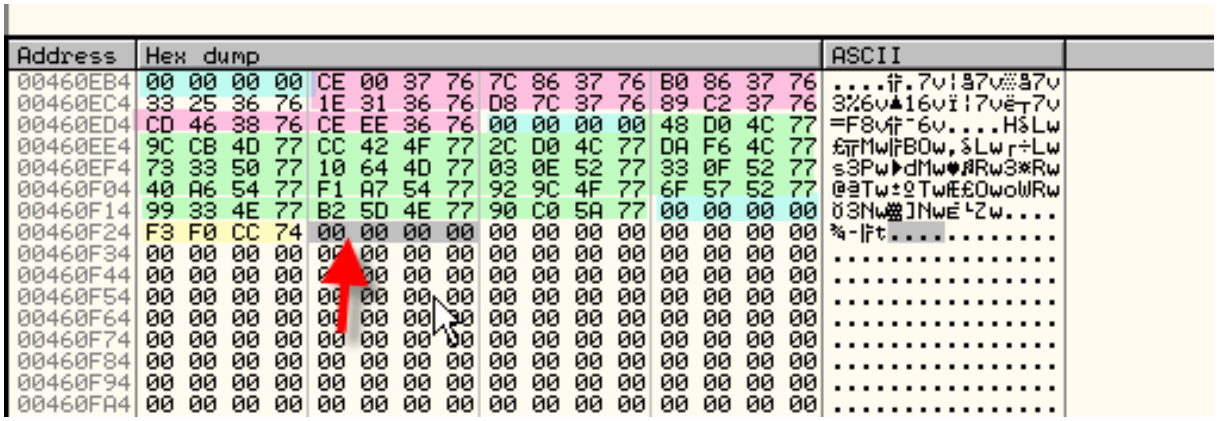
Видим, что есть кое-какие прямые вызовы API-функций, отмеченные лазурным цветом, очевидно с помощью косвенного JMP (что-то он меня запутал – прим.пер.). Идёмте посмотрим некоторые из этих вызовов.



Тут у нас один – это CALL 435CDE, который точно выведет нас на косвенные переходы к API-функциям, отмечаем его, нажимаем правую кнопку мыши и выбираем FOLLOW.

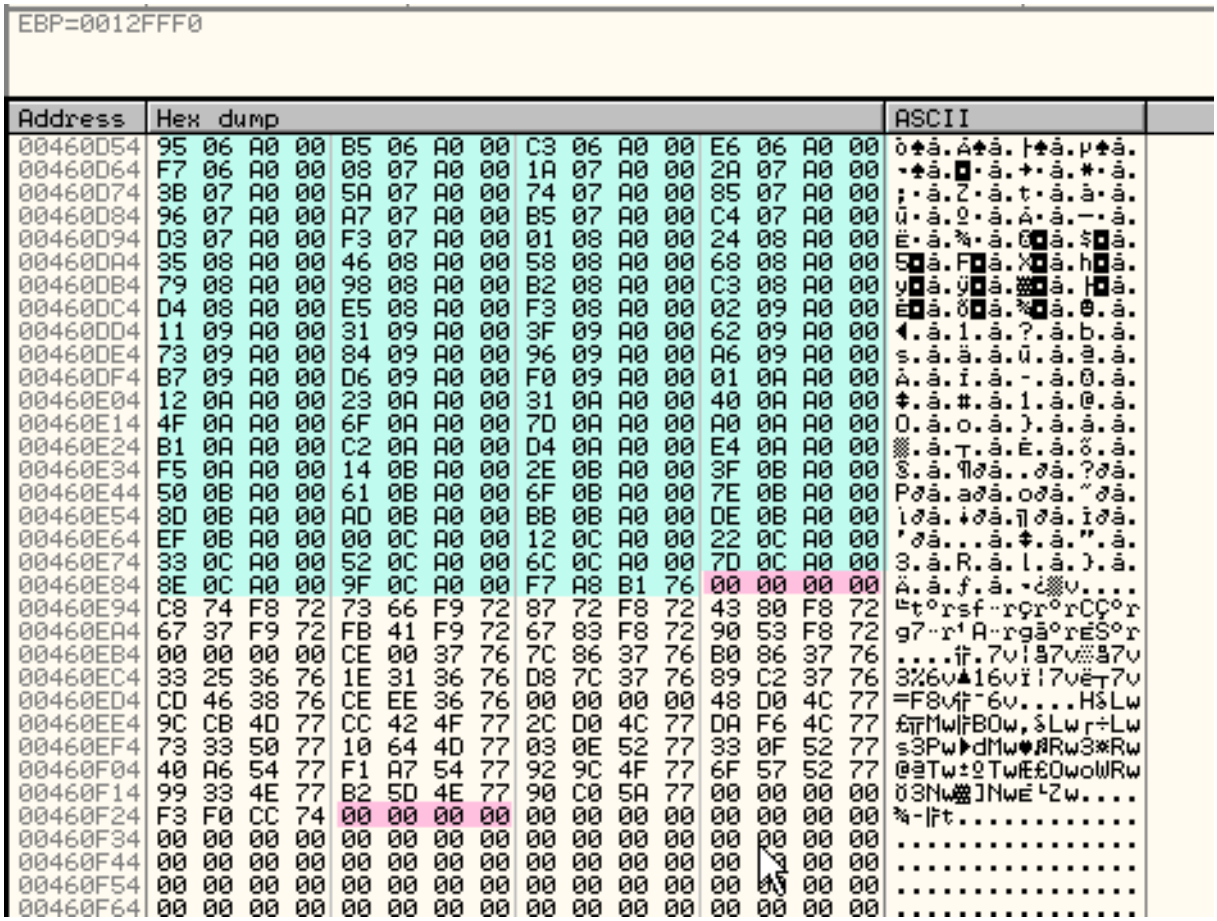


Видим косвенные переходы на API-функции, поэтому знаем, что берутся значения из IAT, т.е. 460ED4 – это элемент IAT. Идём в DUMP, чтобы посмотреть на него.



Address	Hex dump	ASCII
00460EB4	00 00 00 00 CE 00 37 76 7C 86 37 76 B0 86 37 76	....Îv.7v|†7v°†7v
00460EC4	33 25 36 76 1E 31 36 76 D8 7C 37 76 89 C2 37 76	3%6v▲16vÏ|7vë┬7v
00460ED4	CD 46 38 76 CE EE 36 76 00 00 00 00 48 D0 4C 77	=F8vÎî⁻6v....HšLw
00460EE4	9C CB 4D 77 CC 42 4F 77 2C D0 4C 77 DA F6 4C 77	£╦Mw╠BOw,šLw┌÷Lw
00460EF4	73 33 50 77 10 64 4D 77 03 0E 52 77 33 0F 52 77	s3Pw►dMw♥♫Rw3*Rw
00460F04	40 A6 54 77 F1 A7 54 77 92 9C 4F 77 6F 57 52 77	@ªTw±§Tw'£OwoWRw
00460F14	99 33 4E 77 B2 5D 4E 77 90 C0 5A 77 00 00 00 00	Ö3Nw▒]NwÉ└Zw....
00460F24	F3 F0 CC 74 00 00 00 00 00 00 00 00 00 00 00 00	¾-╠t............
00460F34	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
00460F44	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
00460F54	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
00460F64	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
00460F74	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
00460F84	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
00460F94	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
00460FA4	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....

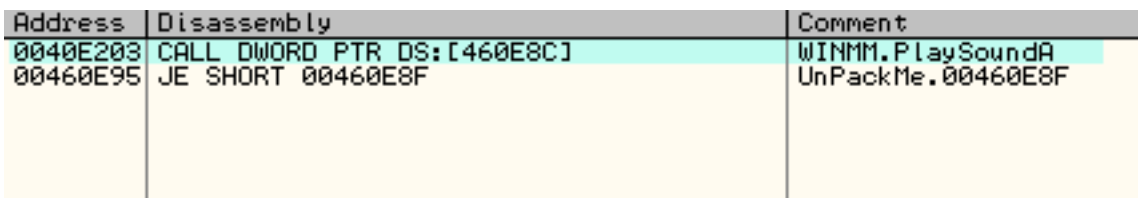
Видим, что заключительная часть IAT корректна и совпадает с той, что в примере CRUNCH. Конец IAT здесь также 460F28. Здесь конец легко определить, так как потом идут одни нули. Теперь идём вниз.



EBP=0012FFF0

Address	Hex dump	ASCII
00460D54	95 06 A0 00 B5 06 A0 00 C3 06 A0 00 E6 06 A0 00	ò♠á.Á♠á.├♠á.µ♠á.
00460D64	F7 06 A0 00 08 07 A0 00 1A 07 A0 00 2A 07 A0 00	~♠á.◘·á.→·á.*·á.
00460D74	3B 07 A0 00 5A 07 A0 00 74 07 A0 00 85 07 A0 00	;·á.Z·á.t·á.à·á.
00460D84	96 07 A0 00 A7 07 A0 00 B5 07 A0 00 C4 07 A0 00	û·á.º·á.Á·á.─·á.
00460D94	D3 07 A0 00 F3 07 A0 00 01 08 A0 00 24 08 A0 00	Ë·á.¾·á.☺◘á.$◘á.
00460DA4	35 08 A0 00 46 08 A0 00 58 08 A0 00 68 08 A0 00	5◘á.F◘á.X◘á.h◘á.
00460DB4	79 08 A0 00 98 08 A0 00 B2 08 A0 00 C3 08 A0 00	y◘á.ÿ◘á.▒◘á.├◘á.
00460DC4	D4 08 A0 00 E5 08 A0 00 F3 08 A0 00 02 09 A0 00	É◘á.ő◘á.¾◘á.☻.á.
00460DD4	11 09 A0 00 31 09 A0 00 3F 09 A0 00 62 09 A0 00	◄.á.1.á.?.á.b.á.
00460DE4	73 09 A0 00 84 09 A0 00 96 09 A0 00 A6 09 A0 00	s.á.ä.á.ü.á.ª.á.
00460DF4	B7 09 A0 00 D6 09 A0 00 F0 09 A0 00 01 0A A0 00	Á.á.Í.á.-.á.◙.á.
00460E04	12 0A A0 00 23 0A A0 00 31 0A A0 00 40 0A A0 00	↕.á.#.á.1.á.@.á.
00460E14	4F 0A A0 00 6F 0A A0 00 7D 0A A0 00 A0 0A A0 00	O.á.o.á.}.á.á.á.
00460E24	B1 0A A0 00 C2 0A A0 00 D4 0A A0 00 E4 0A A0 00	▒.á.┬.á.╔.á.ő.á.
00460E34	F5 0A A0 00 14 0B A0 00 2E 0B A0 00 3F 0B A0 00	⌠.á.¶♂á..♂á.?♂á.
00460E44	50 0B A0 00 61 0B A0 00 6F 0B A0 00 7E 0B A0 00	P♂á.a♂á.o♂á.~♂á.
00460E54	8D 0B A0 00 AD 0B A0 00 BB 0B A0 00 DE 0B A0 00	ì♂á.¡♂á.╗♂á.ì♂á.
00460E64	EF 0B A0 00 00 0C A0 00 12 0C A0 00 22 0C A0 00	'♂á...á.↕.á."·á.
00460E74	33 0C A0 00 52 0C A0 00 6C 0C A0 00 7D 0C A0 00	3.á.R.á.l.á.}.á.
00460E84	8E 0C A0 00 9F 0C A0 00 F7 A8 B1 76 00 00 00 00	Ä.á.ƒ.á.~¿▒v....
00460E94	C8 74 F8 72 73 66 F9 72 87 72 F8 72 43 80 F8 72	╚t°rsf¨rçr°rCÇ°r
00460EA4	67 37 F9 72 FB 41 F9 72 67 83 F8 72 90 53 F8 72	g7¨r¹A¨rgâ°rÉS°r
00460EB4	00 00 00 00 CE 00 37 76 7C 86 37 76 B0 86 37 76	....Îv.7v|†7v°†7v
00460EC4	33 25 36 76 1E 31 36 76 D8 7C 37 76 89 C2 37 76	3%6v▲16vÏ|7vë┬7v
00460ED4	CD 46 38 76 CE EE 36 76 00 00 00 00 48 D0 4C 77	=F8vÎî⁻6v....HšLw
00460EE4	9C CB 4D 77 CC 42 4F 77 2C D0 4C 77 DA F6 4C 77	£╦Mw╠BOw,šLw┌÷Lw
00460EF4	73 33 50 77 10 64 4D 77 03 0E 52 77 33 0F 52 77	s3Pw►dMw♥♫Rw3*Rw
00460F04	40 A6 54 77 F1 A7 54 77 92 9C 4F 77 6F 57 52 77	@ªTw±§Tw'£OwoWRw
00460F14	99 33 4E 77 B2 5D 4E 77 90 C0 5A 77 00 00 00 00	Ö3Nw▒]NwÉ└Zw....
00460F24	F3 F0 CC 74 00 00 00 00 00 00 00 00 00 00 00 00	¾-╠t............
00460F34	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
00460F44	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
00460F54	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
00460F64	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....

Видим, что следующая группа конфликтует, элемент, непосредственно стоящий до разделения соответствует 76B1A8F7. Если отметим его, нажмём правую кнопку мыши и выберем FIND REFERENCES:



Address	Disassembly	Comment
0040E203	CALL DWORD PTR DS:[460E8C]	WINMM.PlaySoundA
00460E95	JE SHORT 00460E8F	UnPackMe.00460E8F

Видим, что он относится к API-функции PlaySoundA из WINMM.dll, больше других значений, ведущих в DLL не встречается, однако если поищем ссылки:

```
Address   Disassembly                        Comment
004038A6  CALL DWORD PTR DS:[460E48]         DS:[00460E48]=00A00B61
004047D0  CALL DWORD PTR DS:[460E48]         DS:[00460E48]=00A00B61
00404923  CALL DWORD PTR DS:[460E48]         DS:[00460E48]=00A00B61
00404AD9  CALL DWORD PTR DS:[460E48]         DS:[00460E48]=00A00B61
0041331B  CALL DWORD PTR DS:[460E48]         DS:[00460E48]=00A00B61
004171DE  CALL DWORD PTR DS:[460E48]         DS:[00460E48]=00A00B61
0041A61E  CALL DWORD PTR DS:[460E48]         DS:[00460E48]=00A00B61
00421045  CALL DWORD PTR DS:[460E48]         DS:[00460E48]=00A00B61
0043D912  CALL DWORD PTR DS:[460E48]         DS:[00460E48]=00A00B61
0043DCC0  CALL DWORD PTR DS:[460E48]         DS:[00460E48]=00A00B61
0043E9D6  CALL DWORD PTR DS:[460E48]         DS:[00460E48]=00A00B61
0043EA8A  CALL DWORD PTR DS:[460E48]         DS:[00460E48]=00A00B61
00441B0A  CALL DWORD PTR DS:[460E48]         DS:[00460E48]=00A00B61
```

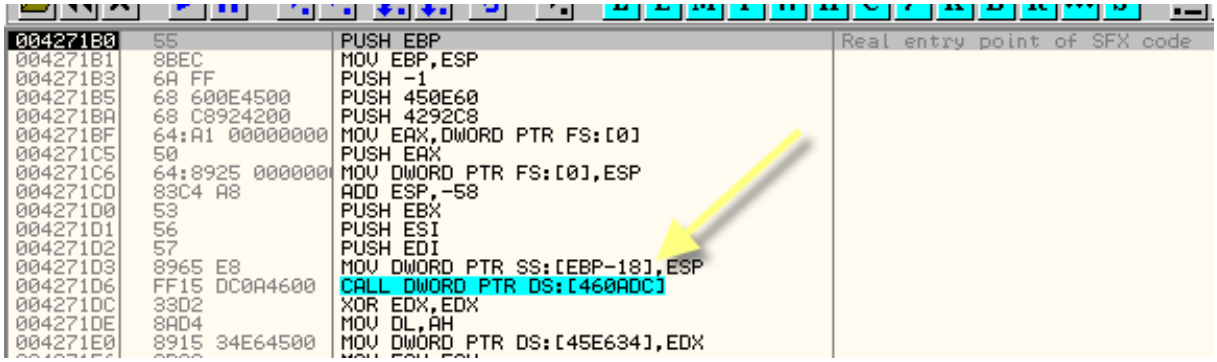
Видим, что раз существуют ссылки, то вот в чём дело, когда мы идём вверх и вниз от начала IAT, то если, например, искали до начала, то не нашли никаких ссылок, программа при этом переходит на API-функции из DLL с помощью IAT, за пределами её мы также не нашли никаких ссылок, теперь мы видим, что это возможные элементы IAT, так как нашли ссылки, которые берут значения из них в коде, но вместо перехода в DLL, происходит переход на секцию, которая в моём случае является Axxxxx (а на другой машине может быть другой), что это такое?

Это как раз то, что называется переадресовочными элементами. При запуске упаковщик перезаписывает некоторые элементы IAT значениями, указывающими на его собственные процедуры. В случае с предыдущим изображением:

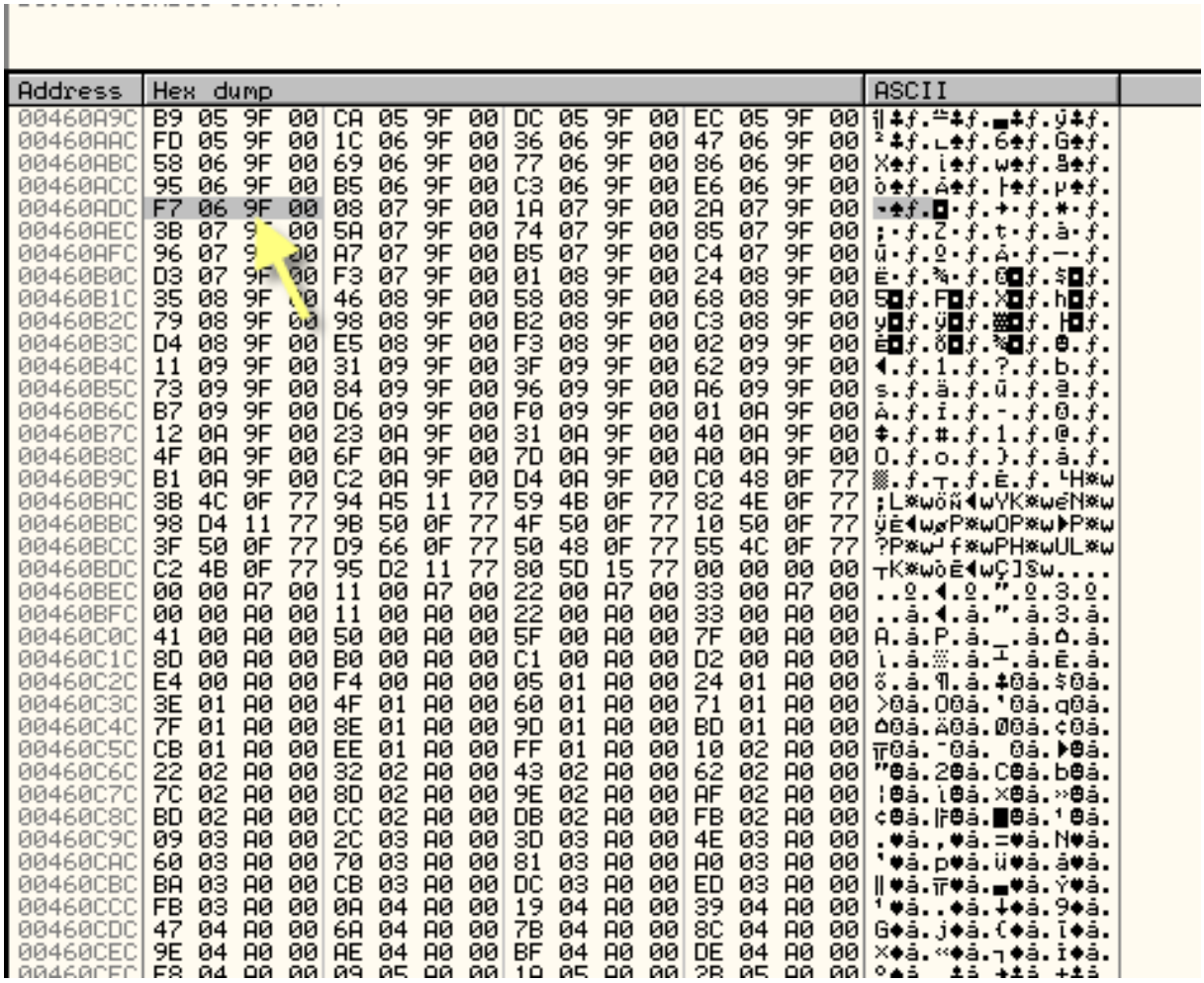
```
004038A6 CALL DWORD PTR DS:[460E48]

Comment=DS:[00460E48]=00A00B61
```

Вместо сохранения правильного на моей машине адреса API-функции распаковщик заменяет его адресом собственного секции, им же созданной, во время выполнения, и там находится процедура, которая в результате выполняет правильную API-функцию.

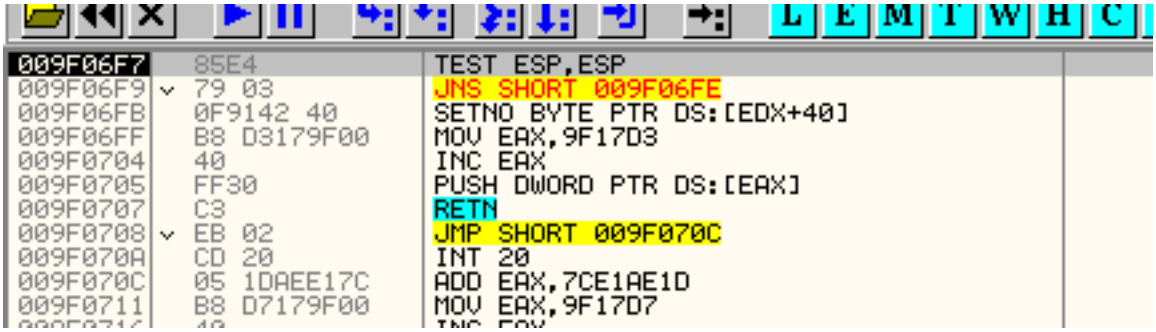
Чтобы лучше понять это, давайте посмотрим на вхождение API-функции GetVersion, которое встречается под OEP.



На самом деле мы не знаем, что оно ведёт на GetVersion, мы знаем только, что в предыдущем примере, где была запакована такая же программа с помощью CRUNCH. Здесь находится косвенный CALL, доходим до него с помощью F7 и входим внутрь, чтобы посмотреть, что там находится.

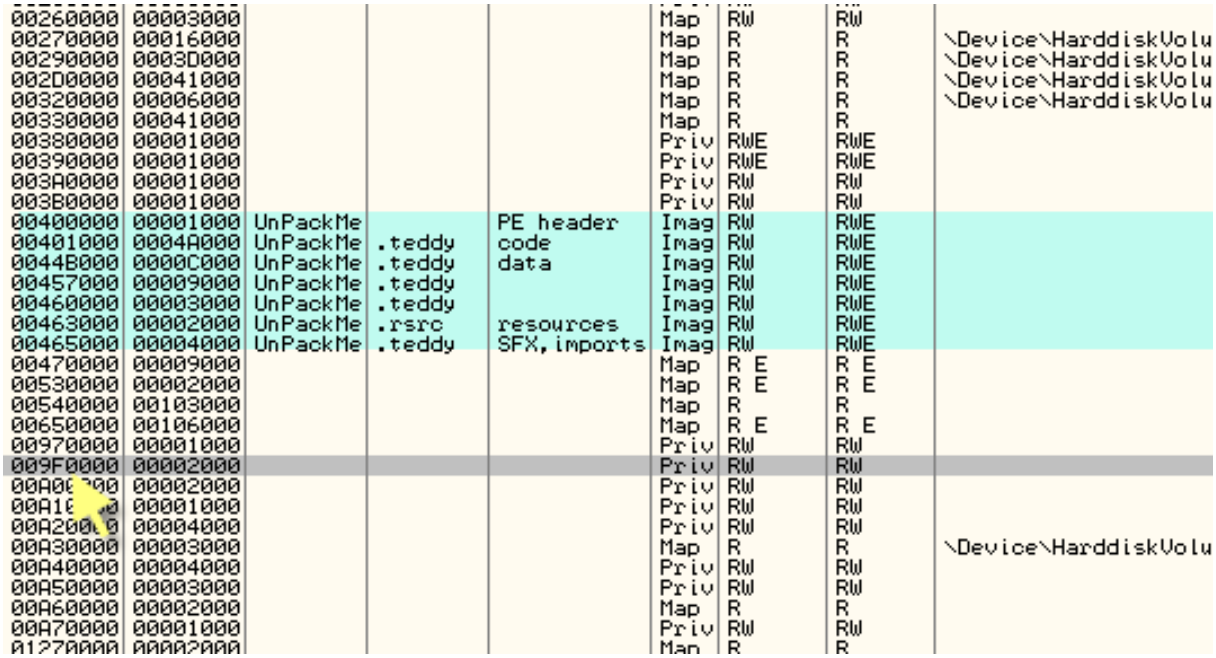


Видим, что откуда-то берутся значения, откуда мы точно не знаем, но возможно, что из элементов IAT, так как ниже видим правильные вхождения вызовов API-функций.



Поэтому программа при запуске CALL идёт сюда, на моей машине это адрес 9F06F7, на ваших оно может быть другим.

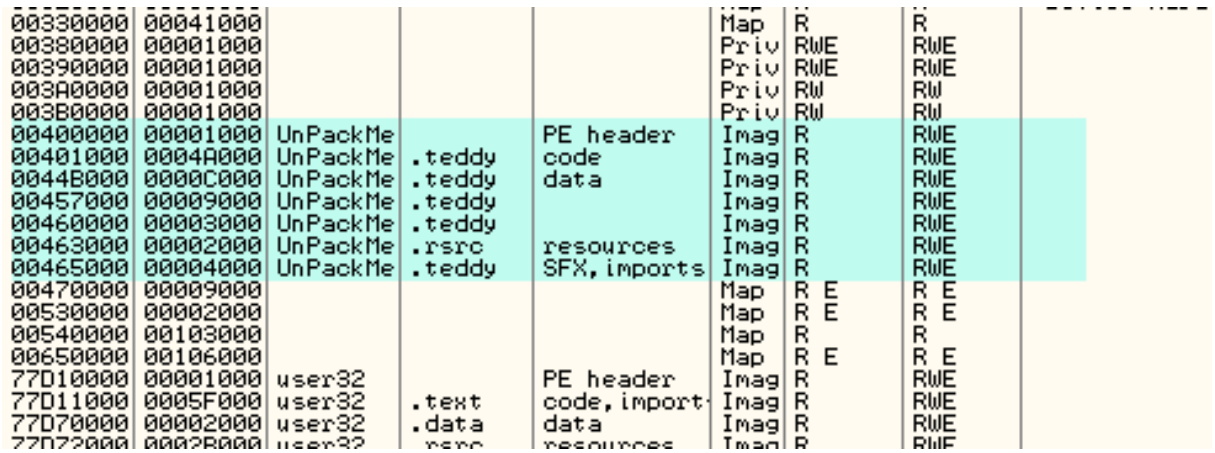
Это адрес не принадлежит какой-либо секции программы.



00260000	00003000				Map	RW	RW	
00270000	00016000				Map	R	R	\Device\HarddiskVolu
00290000	0003D000				Map	R	R	\Device\HarddiskVolu
002D0000	00041000				Map	R	R	\Device\HarddiskVolu
00320000	00006000				Map	R	R	\Device\HarddiskVolu
00330000	00041000				Map	R	R	
00380000	00001000				Priv	RWE	RWE	
00390000	00001000				Priv	RWE	RWE	
003A0000	00001000				Priv	RW	RW	
003B0000	00001000				Priv	RW	RW	
00400000	00001000	UnPackMe		PE header	Imag	RW	RWE	
00401000	0004A000	UnPackMe	.teddy	code	Imag	RW	RWE	
0044B000	0000C000	UnPackMe	.teddy	data	Imag	RW	RWE	
00457000	00009000	UnPackMe	.teddy		Imag	RW	RWE	
00460000	00003000	UnPackMe	.teddy		Imag	RW	RWE	
00463000	00002000	UnPackMe	.rsrc	resources	Imag	RW	RWE	
00465000	00004000	UnPackMe	.teddy	SFX,imports	Imag	RW	RWE	
00470000	00009000				Map	R E	R E	
00530000	00002000				Map	R E	R E	
00540000	00103000				Map	R	R	
00650000	00106000				Map	R E	R E	
00970000	00001000				Priv	RW	RW	
009F0000	00002000				Priv	RW	RW	
00A0[illegible]	00002000				Priv	RW	RW	
00A1[illegible]	00001000				Priv	RW	RW	
00A200[illegible]	00004000				Priv	RW	RW	
00A30000	00003000				Map	R	R	\Device\HarddiskVolu
00A40000	00004000				Priv	RW	RW	
00A50000	00003000				Priv	RW	RW	
00A60000	00002000				Map	R	R	
00A70000	00001000				Priv	RW	RW	
01270000	00002000				Map	R	R	

Здесь видим секции программы (выделены лазурным цветом), а ниже секцию без имени, где находится процедура, куда переходит программа.

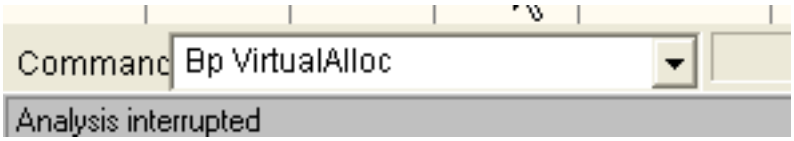
Если перезапустим программу, то увидим, что эта секция в начале не существует.



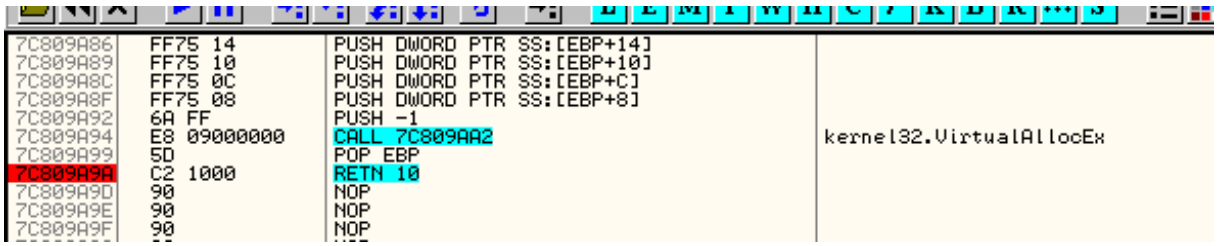
00330000	00041000				Map	R	R	
00380000	00001000				Priv	RWE	RWE	
00390000	00001000				Priv	RWE	RWE	
003A0000	00001000				Priv	RW	RW	
003B0000	00001000				Priv	RW	RW	
00400000	00001000	UnPackMe		PE header	Imag	R	RWE	
00401000	0004A000	UnPackMe	.teddy	code	Imag	R	RWE	
0044B000	0000C000	UnPackMe	.teddy	data	Imag	R	RWE	
00457000	00009000	UnPackMe	.teddy		Imag	R	RWE	
00460000	00003000	UnPackMe	.teddy		Imag	R	RWE	
00463000	00002000	UnPackMe	.rsrc	resources	Imag	R	RWE	
00465000	00004000	UnPackMe	.teddy	SFX,imports	Imag	R	RWE	
00470000	00009000				Map	R E	R E	
00530000	00002000				Map	R E	R E	
00540000	00103000				Map	R	R	
00650000	00106000				Map	R E	R E	
77D10000	00001000	user32		PE header	Imag	R	RWE	
77D11000	0005F000	user32	.text	code,import	Imag	R	RWE	
77D70000	00002000	user32	.data	data	Imag	R	RWE	
77D72000	0002B000	user32	.rsrc	resources	Imag	R	RWE	

Поэтому понятно, что она создаётся программой во время распаковки. Теперь, можем ли мы найти момент, когда создаётся данная секция?

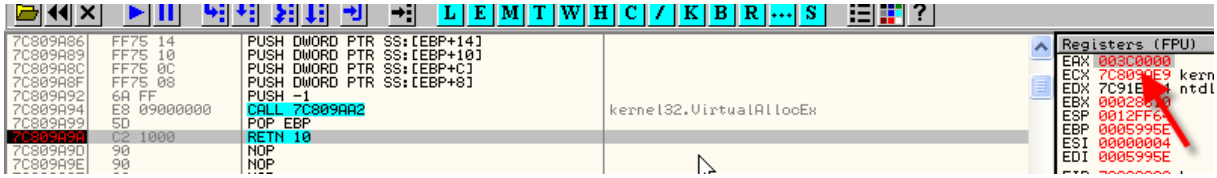
Можем поставить BP на API-функцию VirtualAlloc, которая используется для создания виртуальных секций.



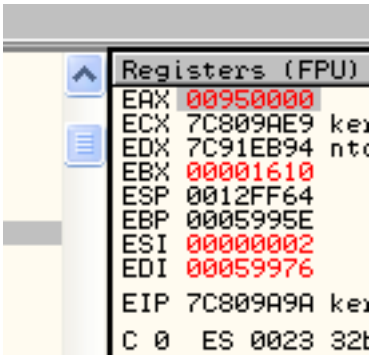
Теперь делаем RUN и, если все галочки в исключениях были отмечены, то увидим, что программа завершилась и более не выполняется. Очевидно, что она обнаружила установленный BP, попробуем поставить его на RET из вышеуказанной функции.



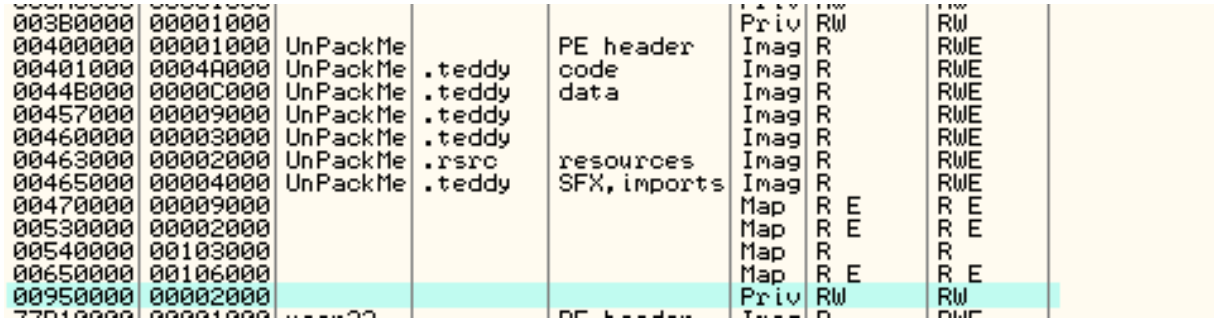
Теперь делаем RUN.



Когда выполнение останавливается на возвращении из API-функции, в EAX содержится адрес созданной секции.

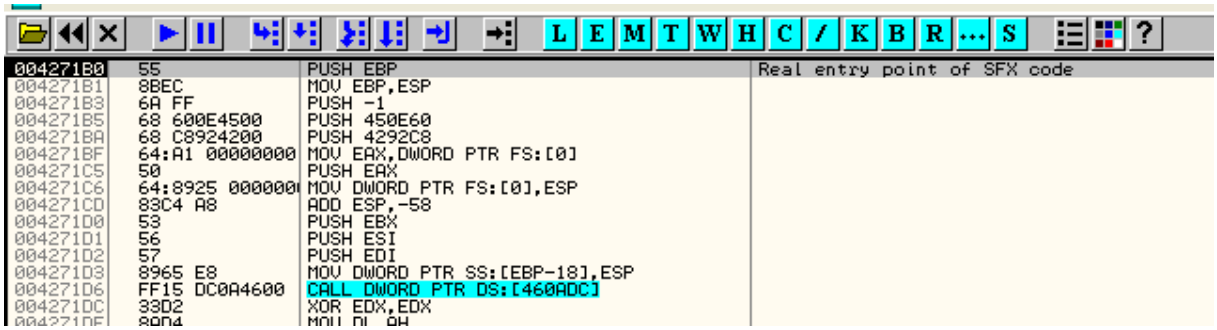


Созданные секции можно посмотреть в карте памяти.

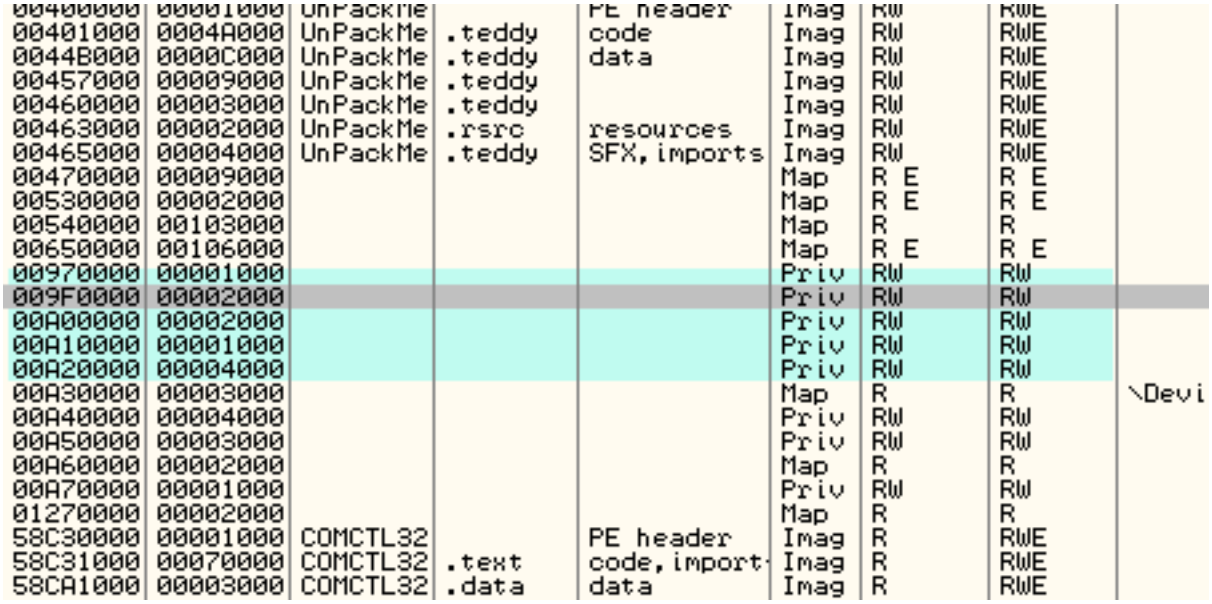


Нам может помочь то, что когда программа создаёт секции для собственного использования, они помечаются как PRIV (т.е. private или собственная), это означает, что этих секций не было в начале до распаковки, так что точно знаем, что они были созданы распаковщиком.

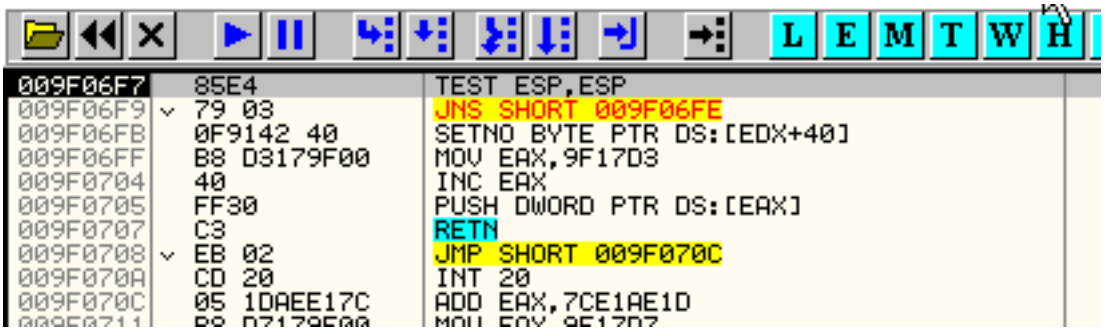
Поэтому убираем BP и прибываем в OEP, как и раньше убрав галочки с исключений, и дойдя до последнего, устанавливая BPM ON ACCESS на первую секцию. Последнее исключение проходим с помощью SHIFT+F9.



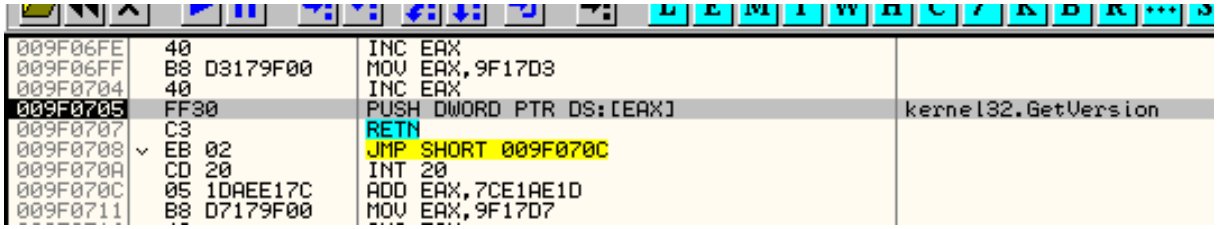
Теперь снова прибываем в OEP и посмотрим в карту памяти:



Видим созданные секции, которые будут использоваться программой, отмеченные как PRIV, и сюда будут перенаправляться косвенные вызовы. К ним мы и возвратимся, используя F7.



Трассируем, чтобы узнать, куда приведёт данная процедура.



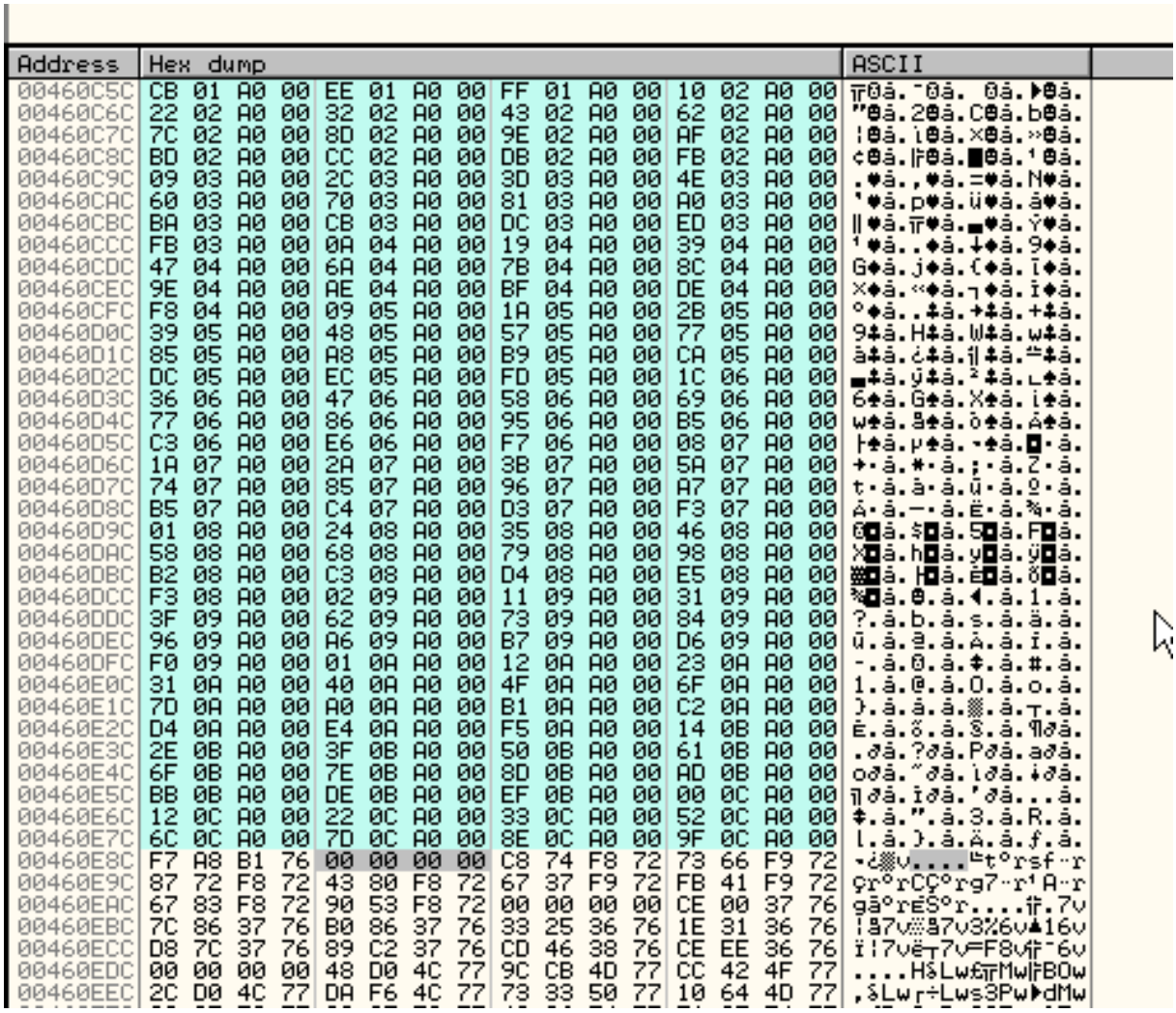
Видим, что дошли до PUSH, который помещает адрес GetVersion в стек, а затем переходит на API-функцию, дойдя до RET, поэтому данная процедура служит посредником для вызова API-функции GetVersion.

То есть распаковщик заменяет вхождения API-функции GetVersion адресом, указывающим на собственную секцию, то есть созданную им самим, и если оттрасируем до конца, то дойдём до места, где вызывается правильный API-адрес.

Это и есть переадресовочный элемент.

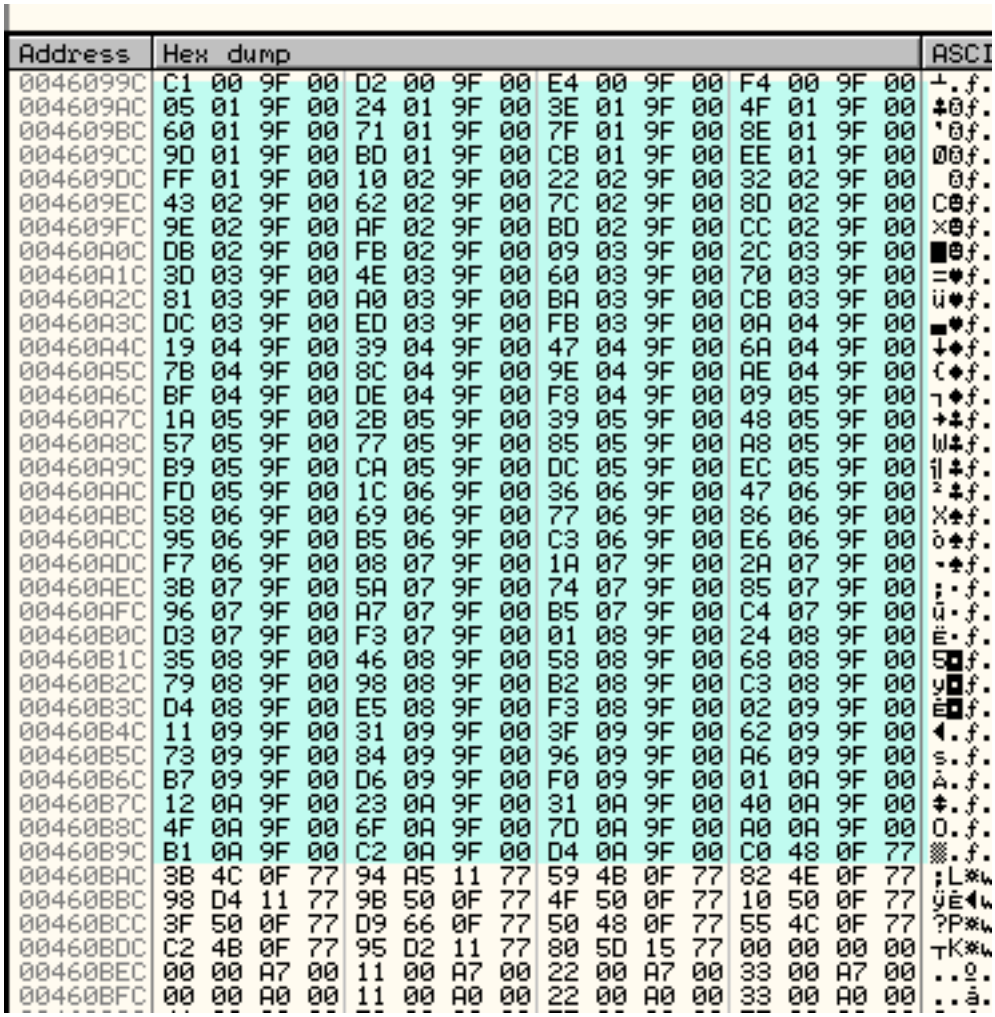
Поэтому когда проверяем начало и конец IAT, не только нужно проверять, что элемент ведёт в DLL, но так же нужно учитывать существование элементов IAT, которые содержат ссылки и ведут нас на собственный код упаковщика, служащий посредником, вызывающим собственно API-функцию.

Поэтому возвращаемся в IAT, который мы уже смотрели, чтобы найти НАЧАЛО и КОНЕЦ.



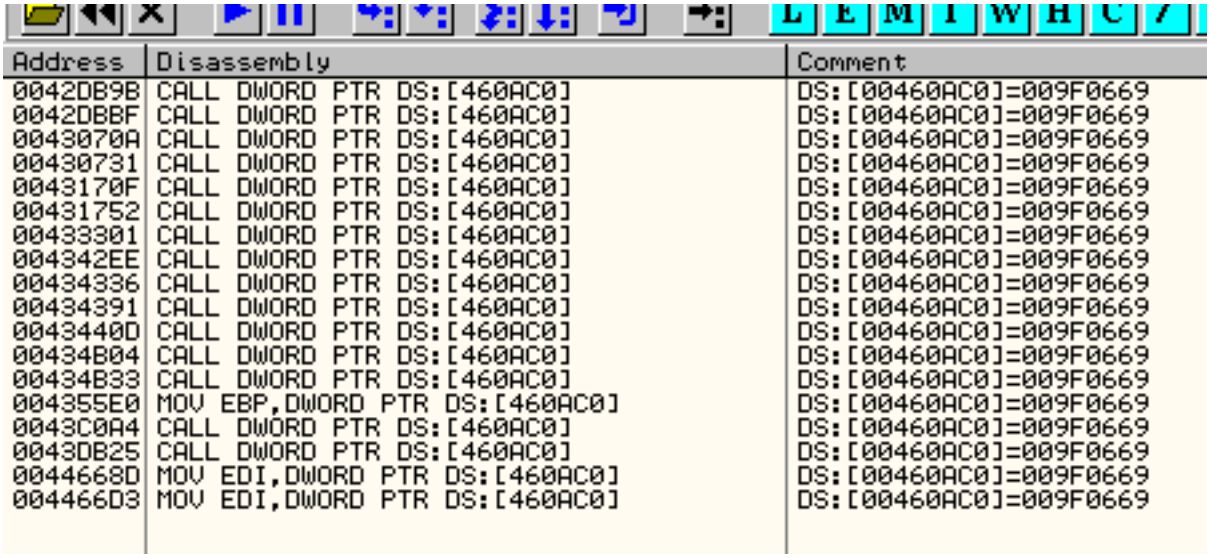
Address	Hex dump	ASCII
00460C5C	CB 01 A0 00 EE 01 A0 00 FF 01 A0 00 10 02 A0 00	
00460C6C	22 02 A0 00 32 02 A0 00 43 02 A0 00 62 02 A0 00	
00460C7C	7C 02 A0 00 8D 02 A0 00 9E 02 A0 00 AF 02 A0 00	
00460C8C	BD 02 A0 00 CC 02 A0 00 DB 02 A0 00 FB 02 A0 00	
00460C9C	09 03 A0 00 2C 03 A0 00 3D 03 A0 00 4E 03 A0 00	
00460CAC	60 03 A0 00 70 03 A0 00 81 03 A0 00 A0 03 A0 00	
00460CBC	BA 03 A0 00 CB 03 A0 00 DC 03 A0 00 ED 03 A0 00	
00460CCC	FB 03 A0 00 0A 04 A0 00 19 04 A0 00 39 04 A0 00	
00460CDC	47 04 A0 00 6A 04 A0 00 7B 04 A0 00 8C 04 A0 00	
00460CEC	9E 04 A0 00 AE 04 A0 00 BF 04 A0 00 DE 04 A0 00	
00460CFC	F8 04 A0 00 09 05 A0 00 1A 05 A0 00 2B 05 A0 00	
00460D0C	39 05 A0 00 48 05 A0 00 57 05 A0 00 77 05 A0 00	
00460D1C	85 05 A0 00 A8 05 A0 00 B9 05 A0 00 CA 05 A0 00	
00460D2C	DC 05 A0 00 EC 05 A0 00 FD 05 A0 00 1C 06 A0 00	
00460D3C	36 06 A0 00 47 06 A0 00 58 06 A0 00 69 06 A0 00	
00460D4C	77 06 A0 00 86 06 A0 00 95 06 A0 00 B5 06 A0 00	
00460D5C	C3 06 A0 00 E6 06 A0 00 F7 06 A0 00 08 07 A0 00	
00460D6C	1A 07 A0 00 2A 07 A0 00 3B 07 A0 00 5A 07 A0 00	
00460D7C	74 07 A0 00 85 07 A0 00 96 07 A0 00 A7 07 A0 00	
00460D8C	B5 07 A0 00 C4 07 A0 00 D3 07 A0 00 F3 07 A0 00	
00460D9C	01 08 A0 00 24 08 A0 00 35 08 A0 00 46 08 A0 00	
00460DAC	58 08 A0 00 68 08 A0 00 79 08 A0 00 98 08 A0 00	
00460DBC	B2 08 A0 00 C3 08 A0 00 D4 08 A0 00 E5 08 A0 00	
00460DCC	F3 08 A0 00 02 09 A0 00 11 09 A0 00 31 09 A0 00	
00460DDC	3F 09 A0 00 62 09 A0 00 73 09 A0 00 84 09 A0 00	
00460DEC	96 09 A0 00 A6 09 A0 00 B7 09 A0 00 D6 09 A0 00	
00460DFC	F0 09 A0 00 01 0A A0 00 12 0A A0 00 23 0A A0 00	
00460E0C	31 0A A0 00 40 0A A0 00 4F 0A A0 00 6F 0A A0 00	
00460E1C	7D 0A A0 00 A0 0A A0 00 B1 0A A0 00 C2 0A A0 00	
00460E2C	D4 0A A0 00 E4 0A A0 00 F5 0A A0 00 14 0B A0 00	
00460E3C	2E 0B A0 00 3F 0B A0 00 50 0B A0 00 61 0B A0 00	
00460E4C	6F 0B A0 00 7E 0B A0 00 8D 0B A0 00 AD 0B A0 00	
00460E5C	BB 0B A0 00 DE 0B A0 00 EF 0B A0 00 00 0C A0 00	
00460E6C	12 0C A0 00 22 0C A0 00 33 0C A0 00 52 0C A0 00	
00460E7C	6C 0C A0 00 7D 0C A0 00 8E 0C A0 00 9F 0C A0 00	
00460E8C	F7 A8 B1 76 00 00 00 00 C8 74 F8 72 73 66 F9 72	
00460E9C	87 72 F8 72 43 80 F8 72 67 37 F9 72 FB 41 F9 72	
00460EAC	67 83 F8 72 90 53 F8 72 00 00 00 00 CE 00 37 76	
00460EBC	7C 86 37 76 B0 86 37 76 33 25 36 76 1E 31 36 76	
00460ECC	D8 7C 37 76 89 C2 37 76 CD 46 38 76 CE EE 36 76	
00460EDC	00 00 00 00 48 D0 4C 77 9C CB 4D 77 CC 42 4F 77	
00460EEC	2C D0 4C 77 DA F6 4C 77 73 33 50 77 10 64 4D 77	

Все элементы, которые ведут в секцию 0Axxxxxx (на моей машине) являются переадресовочными элементами. Идём в код, созданный распаковщиком, который не существует при запуске программы с OEP, поэтому идём ниже, чтобы увидеть, найдётся ли НАЧАЛО IAT.



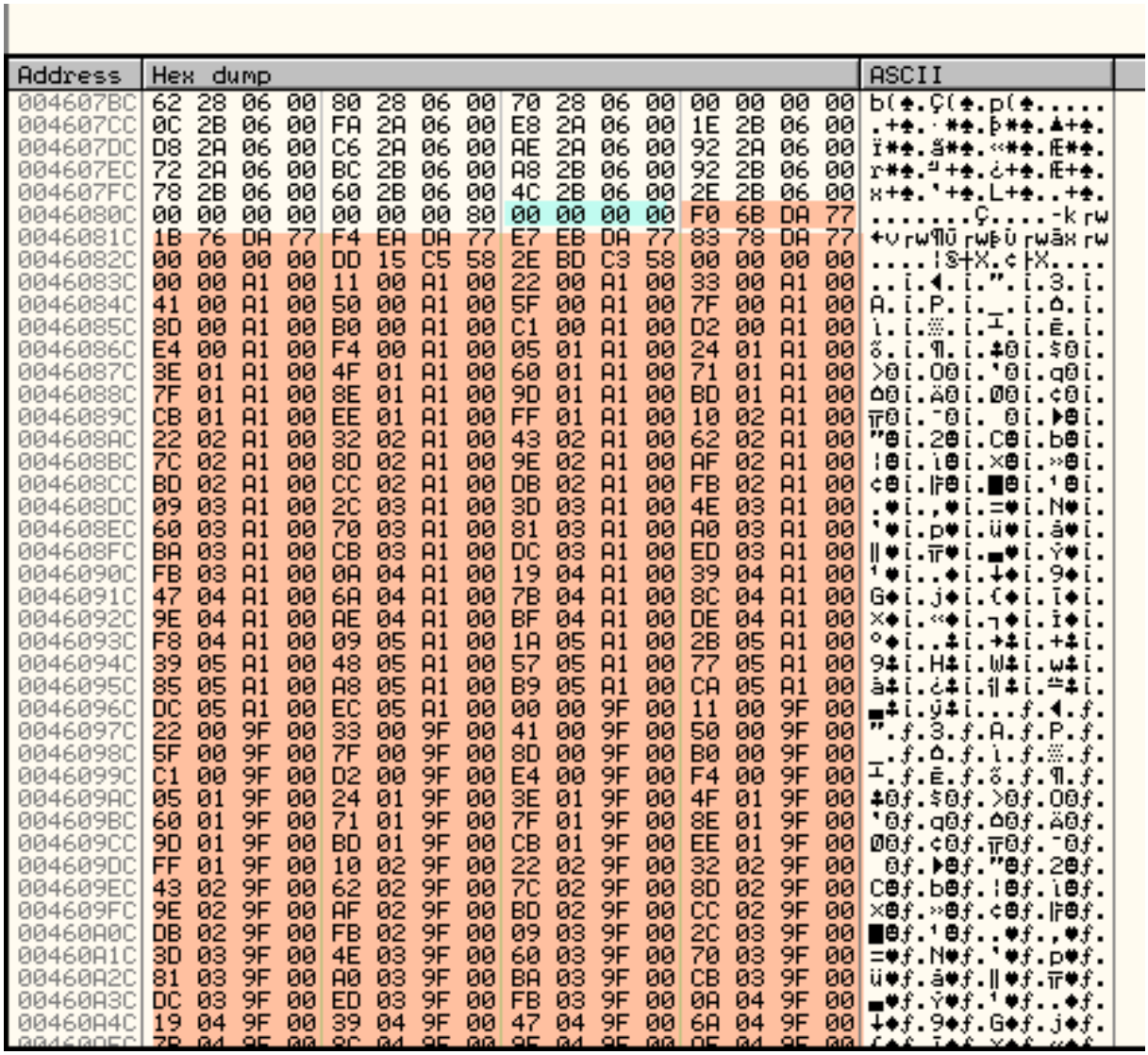
Address	Hex dump	ASCI
00460990	C1 00 9F 00 D2 00 9F 00 E4 00 9F 00 F4 00 9F 00	⊥.ƒ.
004609A0	05 01 9F 00 24 01 9F 00 3E 01 9F 00 4F 01 9F 00	♣☺ƒ.
004609B0	60 01 9F 00 71 01 9F 00 7F 01 9F 00 8E 01 9F 00	`☺ƒ.
004609C0	9D 01 9F 00 BD 01 9F 00 CB 01 9F 00 EE 01 9F 00	ØØƒ.
004609D0	FF 01 9F 00 10 02 9F 00 22 02 9F 00 32 02 9F 00	☺ƒ.
004609E0	43 02 9F 00 62 02 9F 00 7C 02 9F 00 8D 02 9F 00	C☻ƒ.
004609F0	9E 02 9F 00 AF 02 9F 00 BD 02 9F 00 CC 02 9F 00	×☻ƒ.
00460A00	DB 02 9F 00 FB 02 9F 00 09 03 9F 00 2C 03 9F 00	■☻ƒ.
00460A10	3D 03 9F 00 4E 03 9F 00 60 03 9F 00 70 03 9F 00	=♥ƒ.
00460A20	81 03 9F 00 A0 03 9F 00 BA 03 9F 00 CB 03 9F 00	ü♥ƒ.
00460A30	DC 03 9F 00 ED 03 9F 00 FB 03 9F 00 0A 04 9F 00	▄♥ƒ.
00460A40	19 04 9F 00 39 04 9F 00 47 04 9F 00 6A 04 9F 00	↓♦ƒ.
00460A50	7B 04 9F 00 8C 04 9F 00 9E 04 9F 00 AE 04 9F 00	{♦ƒ.
00460A60	BF 04 9F 00 DE 04 9F 00 F8 04 9F 00 09 05 9F 00	┐♦ƒ.
00460A70	1A 05 9F 00 2B 05 9F 00 39 05 9F 00 48 05 9F 00	→♣ƒ.
00460A80	57 05 9F 00 77 05 9F 00 85 05 9F 00 A8 05 9F 00	W♣ƒ.
00460A90	B9 05 9F 00 CA 05 9F 00 DC 05 9F 00 EC 05 9F 00	╣♣ƒ.
00460AA0	FD 05 9F 00 1C 06 9F 00 36 06 9F 00 47 06 9F 00	²♣ƒ.
00460AB0	58 06 9F 00 69 06 9F 00 77 06 9F 00 86 06 9F 00	X♠ƒ.
00460AC0	95 06 9F 00 B5 06 9F 00 C3 06 9F 00 E6 06 9F 00	ò♠ƒ.
00460AD0	F7 06 9F 00 08 07 9F 00 1A 07 9F 00 2A 07 9F 00	÷♠ƒ.
00460AE0	3B 07 9F 00 5A 07 9F 00 74 07 9F 00 85 07 9F 00	;•ƒ.
00460AF0	96 07 9F 00 A7 07 9F 00 B5 07 9F 00 C4 07 9F 00	û•ƒ.
00460B00	D3 07 9F 00 F3 07 9F 00 01 08 9F 00 24 08 9F 00	Ë•ƒ.
00460B10	35 08 9F 00 46 08 9F 00 58 08 9F 00 68 08 9F 00	5◘ƒ.
00460B20	79 08 9F 00 98 08 9F 00 B2 08 9F 00 C3 08 9F 00	y◘ƒ.
00460B30	D4 08 9F 00 E5 08 9F 00 F3 08 9F 00 02 09 9F 00	Ê◘ƒ.
00460B40	11 09 9F 00 31 09 9F 00 3F 09 9F 00 62 09 9F 00	◄.ƒ.
00460B50	73 09 9F 00 84 09 9F 00 96 09 9F 00 A6 09 9F 00	s.ƒ.
00460B60	B7 09 9F 00 D6 09 9F 00 F0 09 9F 00 01 0A 9F 00	ȧ.ƒ.
00460B70	12 0A 9F 00 23 0A 9F 00 31 0A 9F 00 40 0A 9F 00	↕.ƒ.
00460B80	4F 0A 9F 00 6F 0A 9F 00 7D 0A 9F 00 A0 0A 9F 00	O.ƒ.
00460B90	B1 0A 9F 00 C2 0A 9F 00 D4 0A 9F 00 C0 48 0F 77	▒.ƒ.
00460BA0	3B 4C 0F 77 94 A5 11 77 59 4B 0F 77 82 4E 0F 77	;L*w
00460BB0	98 D4 11 77 9B 50 0F 77 4F 50 0F 77 10 50 0F 77	ÿÊ◄w
00460BC0	3F 50 0F 77 D9 66 0F 77 50 48 0F 77 55 4C 0F 77	?P*w
00460BD0	C2 4B 0F 77 95 D2 11 77 80 5D 15 77 00 00 00 00	┬K*w
00460BE0	00 00 A7 00 11 00 A7 00 22 00 A7 00 33 00 A7 00	..§.
00460BF0	00 00 A0 00 11 00 A0 00 22 00 A0 00 33 00 A0 00	..á.

Видим, что далее есть несколько элементов, которые ведут в DLL, а затем выше них элементы, ведущие в 9Fxxxx – другую секцию, созданную распаковщиком.



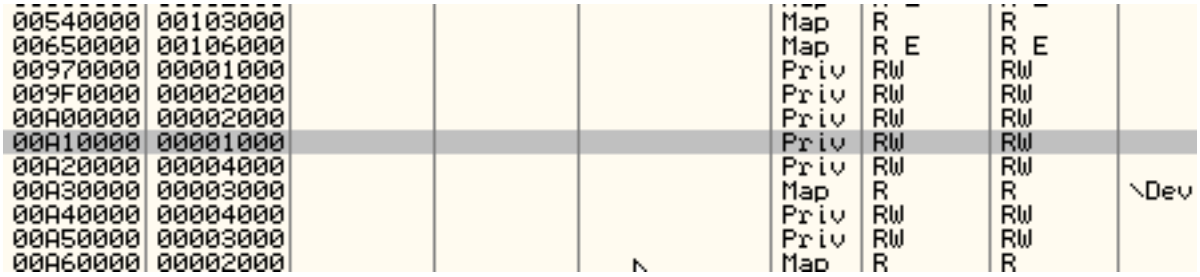
Address	Disassembly	Comment
0042DB9B	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
0042DBBF	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
0043070A	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
00430731	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
0043170F	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
00431752	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
00433301	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
004342EE	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
00434336	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
00434391	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
0043440D	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
00434B04	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
00434B33	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
004355E0	MOV EBP,DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
0043C0A4	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
0043DB25	CALL DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
0044668D	MOV EDI,DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669
004466D3	MOV EDI,DWORD PTR DS:[460AC0]	DS:[00460AC0]=009F0669

Видим, что указанные элементы содержат ссылки, поэтому это элементы IAT, спускаемся дальше.



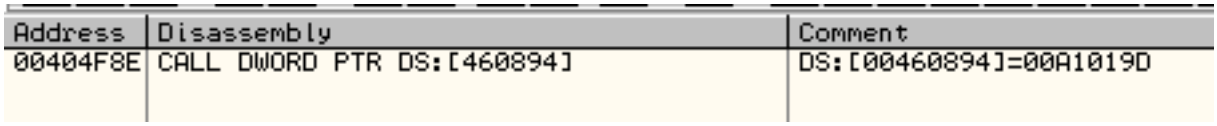
Address	Hex dump	ASCII
004607BC	62 28 06 00 80 28 06 00 70 28 06 00 00 00 00 00	b(♠.Ç(♠.p(♠.....
004607CC	0C 2B 06 00 FA 2A 06 00 E8 2A 06 00 1E 2B 06 00	.+♠.·*♠.Þ*♠.♣+♠.
004607DC	D8 2A 06 00 C6 2A 06 00 AE 2A 06 00 92 2A 06 00	Ï*♠.ã*♠.«*♠.Æ*♠.
004607EC	72 2A 06 00 BC 2B 06 00 A8 2B 06 00 92 2B 06 00	r*♠.╝+♠.¿+♠.Æ+♠.
004607FC	78 2B 06 00 60 2B 06 00 4C 2B 06 00 2E 2B 06 00	x+♠.'+♠.L+♠..+♠.
0046080C	00 00 00 00 00 00 00 80 00 00 00 00 F0 6B DA 77	.......Ç....-k rw
0046081C	1B 76 DA 77 F4 EA DA 77 E7 EB DA 77 83 78 DA 77	←v rw¶Ū rwÞŪ rwâx rw
0046082C	00 00 00 00 DD 15 C5 58 2E BD C3 58 00 00 00 00	....¦§┼X.¢├X....
0046083C	00 00 A1 00 11 00 A1 00 22 00 A1 00 33 00 A1 00	..í.◄.í.".í.3.í.
0046084C	41 00 A1 00 50 00 A1 00 5F 00 A1 00 7F 00 A1 00	A.í.P.í._.í.⌂.í.
0046085C	8D 00 A1 00 B0 00 A1 00 C1 00 A1 00 D2 00 A1 00	ì.í.▒.í.╨.í.Ē.í.
0046086C	E4 00 A1 00 F4 00 A1 00 05 01 A1 00 24 01 A1 00	♂.í.¶.í.♣0í.$0í.
0046087C	3E 01 A1 00 4F 01 A1 00 60 01 A1 00 71 01 A1 00	>0í.O0í.'0í.q0í.
0046088C	7F 01 A1 00 8E 01 A1 00 9D 01 A1 00 BD 01 A1 00	⌂0í.Ä0í.Ø0í.¢0í.
0046089C	CB 01 A1 00 EE 01 A1 00 FF 01 A1 00 10 02 A1 00	╦0í.¯0í. 0í.►0í.
004608AC	22 02 A1 00 32 02 A1 00 43 02 A1 00 62 02 A1 00	"0í.20í.C0í.b0í.
004608BC	7C 02 A1 00 8D 02 A1 00 9E 02 A1 00 AF 02 A1 00	!0í.ì0í.×0í.»0í.
004608CC	BD 02 A1 00 CC 02 A1 00 DB 02 A1 00 FB 02 A1 00	¢0í.╠0í.█0í.¹0í.
004608DC	09 03 A1 00 2C 03 A1 00 3D 03 A1 00 4E 03 A1 00	.♥í.,♥í.=♥í.N♥í.
004608EC	60 03 A1 00 70 03 A1 00 81 03 A1 00 A0 03 A1 00	'♥í.p♥í.ü♥í.á♥í.
004608FC	BA 03 A1 00 CB 03 A1 00 DC 03 A1 00 ED 03 A1 00	‖♥í.╦♥í.▄♥í.Ý♥í.
0046090C	FB 03 A1 00 0A 04 A1 00 19 04 A1 00 39 04 A1 00	¹♥í..♦í.↓♦í.9♦í.
0046091C	47 04 A1 00 6A 04 A1 00 7B 04 A1 00 8C 04 A1 00	G♦í.j♦í.{♦í.î♦í.
0046092C	9E 04 A1 00 AE 04 A1 00 BF 04 A1 00 DE 04 A1 00	×♦í.«♦í.┐♦í.ì♦í.
0046093C	F8 04 A1 00 09 05 A1 00 1A 05 A1 00 2B 05 A1 00	°♦í..♣í.→♣í.+♣í.
0046094C	39 05 A1 00 48 05 A1 00 57 05 A1 00 77 05 A1 00	9♣í.H♣í.W♣í.w♣í.
0046095C	85 05 A1 00 A8 05 A1 00 B9 05 A1 00 CA 05 A1 00	à♣í.¿♣í.╣♣í.╩♣í.
0046096C	DC 05 A1 00 EC 05 A1 00 00 00 9F 00 11 00 9F 00	▄♣í.ÿ♣í...ƒ.◄.ƒ.
0046097C	22 00 9F 00 33 00 9F 00 41 00 9F 00 50 00 9F 00	".ƒ.3.ƒ.A.ƒ.P.ƒ.
0046098C	5F 00 9F 00 7F 00 9F 00 8D 00 9F 00 B0 00 9F 00	_.ƒ.⌂.ƒ.ì.ƒ.▒.ƒ.
0046099C	C1 00 9F 00 D2 00 9F 00 E4 00 9F 00 F4 00 9F 00	┴.ƒ.Ē.ƒ.♂.ƒ.¶.ƒ.
004609AC	05 01 9F 00 24 01 9F 00 3E 01 9F 00 4F 01 9F 00	♣0ƒ.$0ƒ.>0ƒ.O0ƒ.
004609BC	60 01 9F 00 71 01 9F 00 7F 01 9F 00 8E 01 9F 00	'0ƒ.q0ƒ.⌂0ƒ.Ä0ƒ.
004609CC	9D 01 9F 00 BD 01 9F 00 CB 01 9F 00 EE 01 9F 00	Ø0ƒ.¢0ƒ.╦0ƒ.¯0ƒ.
004609DC	FF 01 9F 00 10 02 9F 00 22 02 9F 00 32 02 9F 00	0ƒ.►0ƒ."0ƒ.20ƒ.
004609EC	43 02 9F 00 62 02 9F 00 7C 02 9F 00 8D 02 9F 00	C0ƒ.b0ƒ.!0ƒ.ì0ƒ.
004609FC	9E 02 9F 00 AF 02 9F 00 BD 02 9F 00 CC 02 9F 00	×0ƒ.»0ƒ.¢0ƒ.╠0ƒ.
00460A0C	DB 02 9F 00 FB 02 9F 00 09 03 9F 00 2C 03 9F 00	█0ƒ.¹0ƒ..♥ƒ.,♥ƒ.
00460A1C	3D 03 9F 00 4E 03 9F 00 60 03 9F 00 70 03 9F 00	=♥ƒ.N♥ƒ.'♥ƒ.p♥ƒ.
00460A2C	81 03 9F 00 A0 03 9F 00 BA 03 9F 00 CB 03 9F 00	ü♥ƒ.á♥ƒ.‖♥ƒ.╦♥ƒ.
00460A3C	DC 03 9F 00 ED 03 9F 00 FB 03 9F 00 0A 04 9F 00	▄♥ƒ.Ý♥ƒ.¹♥ƒ..♦ƒ.
00460A4C	19 04 9F 00 39 04 9F 00 47 04 9F 00 6A 04 9F 00	↓♦ƒ.9♦ƒ.G♦ƒ.j♦ƒ.

Далее видим элементы, которые на моей машине ведут в A1xxxx – это ещё одна секция, созданная упаковщиком.



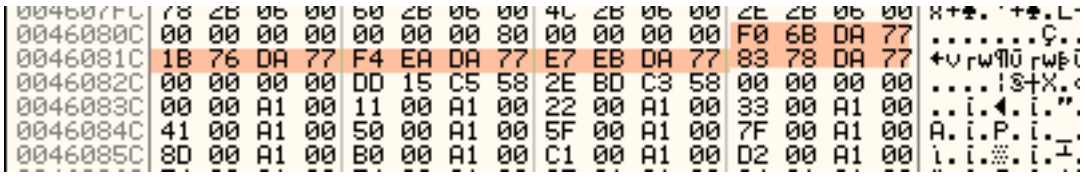
00540000	00103000			Map	R	R	
00650000	00106000			Map	R E	R E	
00970000	00001000			Priv	RW	RW	
009F0000	00002000			Priv	RW	RW	
00A00000	00002000			Priv	RW	RW	
00A10000	00001000			Priv	RW	RW	
00A20000	00004000			Priv	RW	RW	
00A30000	00003000			Map	R	R	\Dev
00A40000	00004000			Priv	RW	RW	
00A50000	00003000			Priv	RW	RW	
00A60000	00002000			Map	R	R	

Также, если поищем, то найдём в них ссылки.

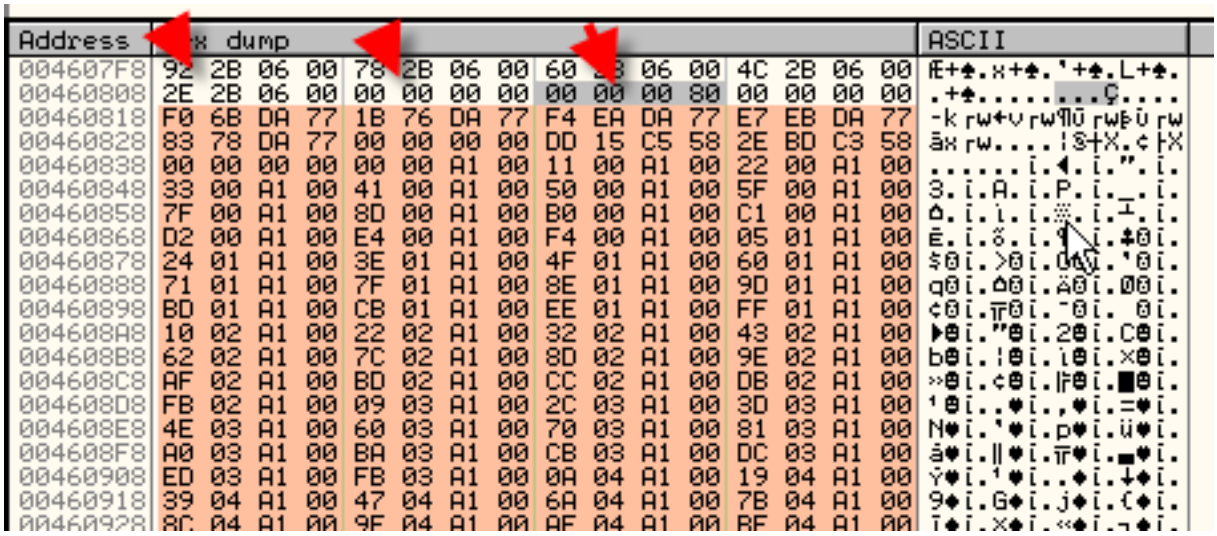


Address	Disassembly	Comment
00404F8E	CALL DWORD PTR DS:[460894]	DS:[00460894]=00A1019D

Значит, это элементы IAT, и продолжаем спускаться дальше, видим, что оказались здесь:



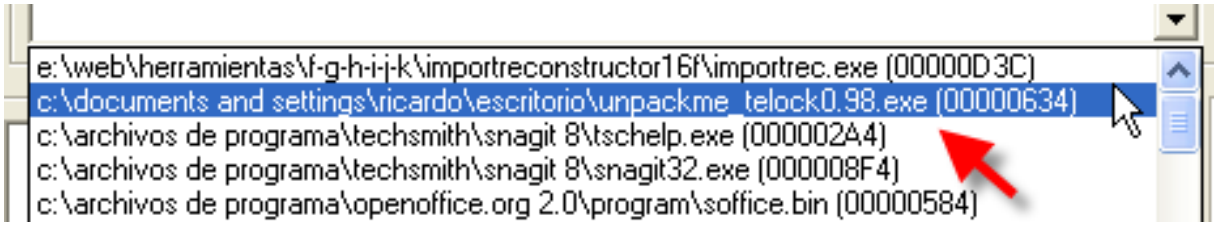
Где есть элементы, ведущие в какую-то DLL, затем разделения, а выше никакие элементы не содержат ссылки.



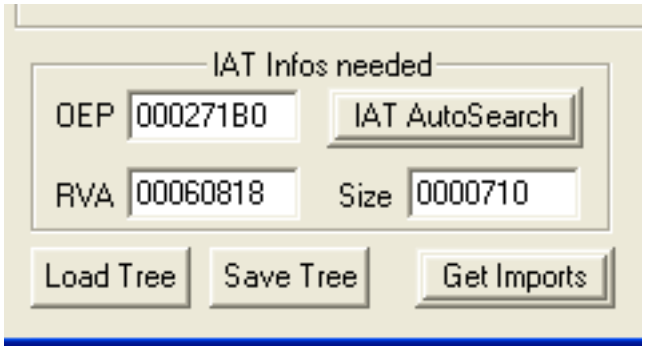
Как и в случае с CRUNCH'ем началом IAT является 460818, размер 710, и OEP равен 4271B0. Отнимаем от него базу образа.

```
OEP = 271B0
НАЧАЛО или RVA = 60818
РАЗМЕР = 710
```

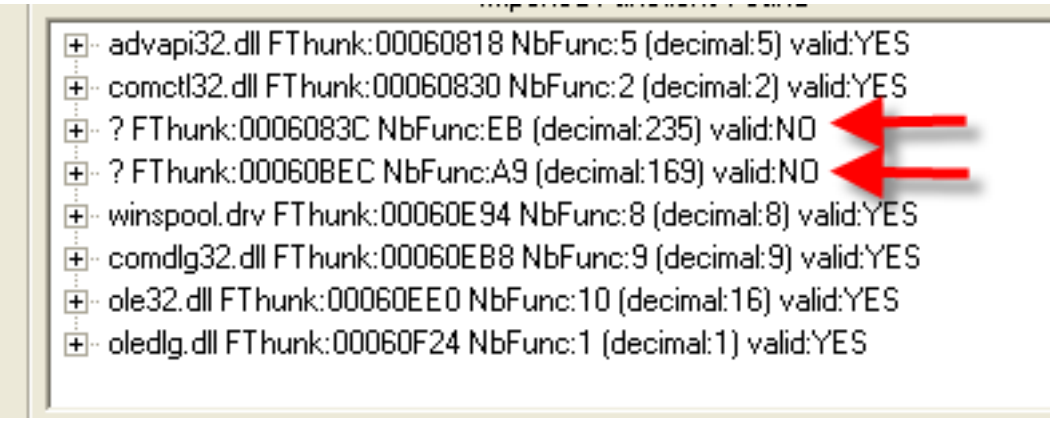
Поэтому открываем IMP REC, не закрывая OllyDbg.



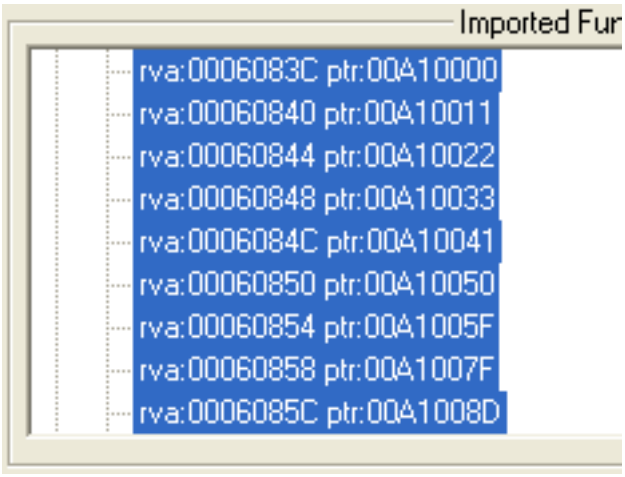
И устанавливаем найденные значения.



Теперь нажимаем GET IMPORTS.



Как видим, IMP REC обнаружил, что есть переадресовочные элементы и пометил их как NO, посмотрим их, нажав SHOW INVALIDS.



IMP REC показал нам элементы IAT, которые не указывают на какую-либо API-функцию и которые ведут в секции, созданные упаковщиком.

Конечно, мы не будем трассировать все эти неправильные элементы вручную, есть несколько методов, чтобы их починить. IMP REC предоставляет несколько возможностей, также есть другие «ручные» методы, но без трассировки, хе-хе.

Все эти методы мы рассмотрим в 37 главе, поэтому вы должны хорошо уяснить рассмотренную тему о переадресовочных элементах, так как в следующей главе будут изложены несколько методов, чтобы их починить. Пока что мы не можем починить дамп, так как IMP REC'у требуется, чтобы все элементы были отмечены как YES, то есть указывали на API-функции из DLL, и не было никаких переадресовочных элементов, так что эту проблему на надо будет решить.

Введение в крэкинг с нуля, используя OllyDbg - Глава 37

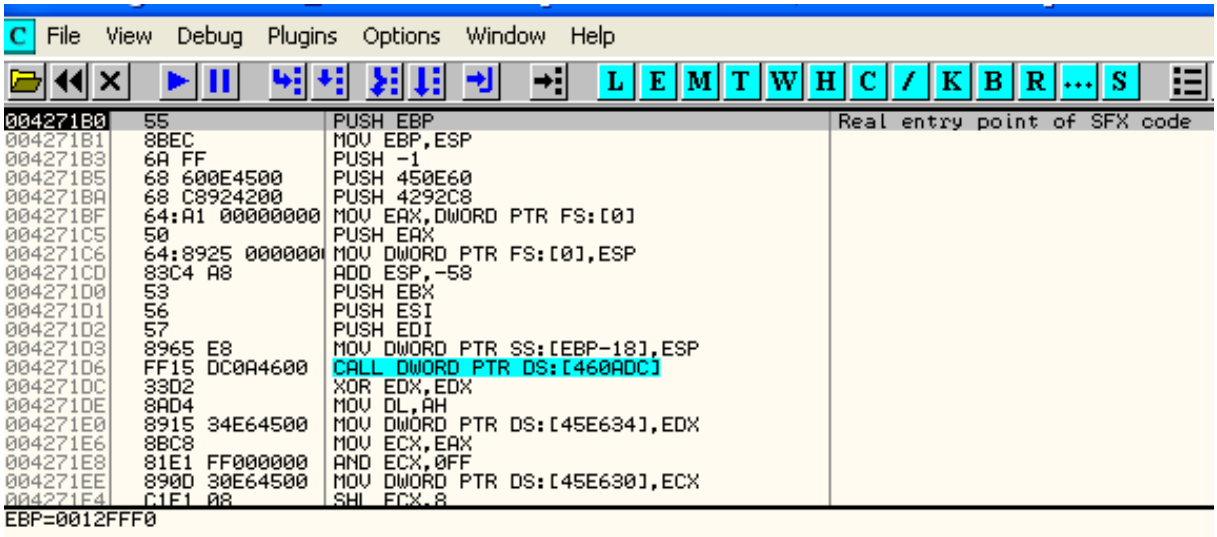
Автор: Рикардо Нарваха, пер. Aquila

Дата: 2008-12-13

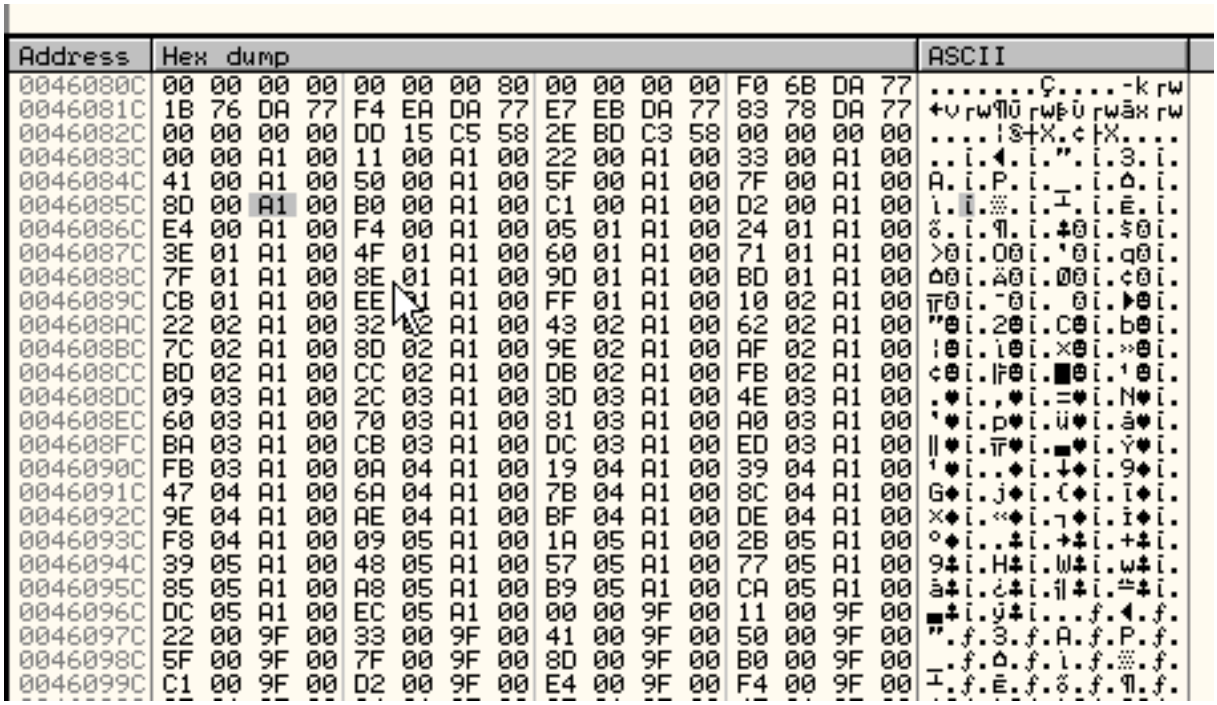
В этой части мы рассмотрим несколько методов, с помощью которых можно починить переадресовочные элементы, но наша задача не переутомиться – так как, как и в случае с нахождением OEP, методов столько же, сколько существует упаковщиков, и когда будем рассматривать какие-нибудь другие упаковщики, то там могут встречаться новые методы как для нахождения OEP, так и для починки IAT.

Эти методы являются общими концепциями, и в других упаковщиках они могут работать, а могут не работать, и тогда потребуется их скорректировать для данного случая, поэтому всегда хорошо пытаться что-то сделать и много практироваться, делая что-то разными способами.

Так же, как и когда мы пытались найти OEP, мы будем пробовать общие методы, а затем уже рассмотрим несколько упаковщиков, под которые надо будет адаптировать данные методы или использовать что-то новое специально для этого случая и свойственные только этому упаковщику.

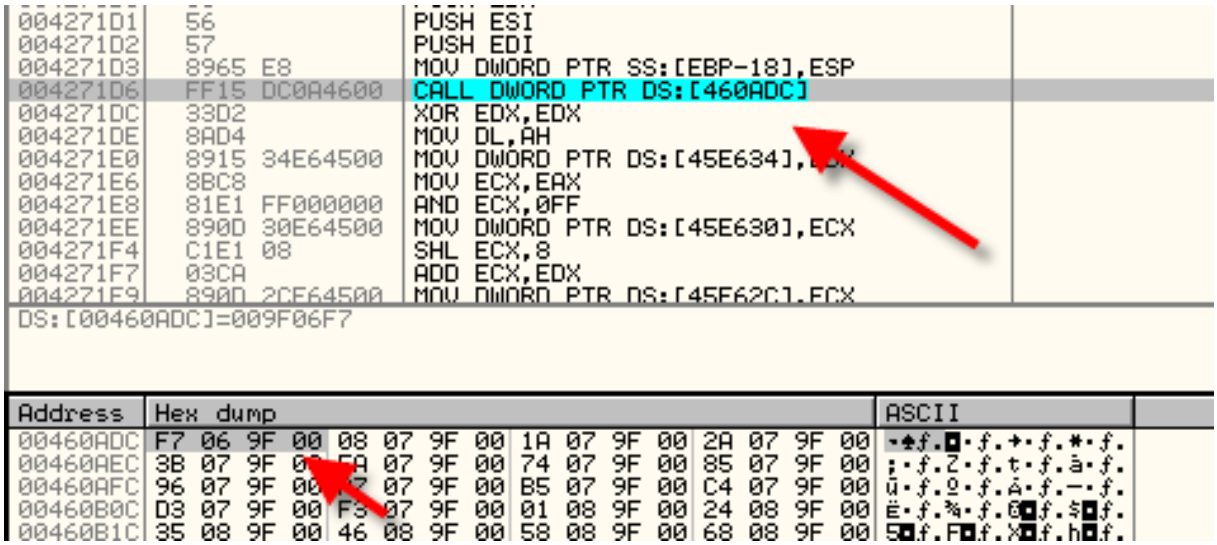


Ок, мы находимся на OEP telock'a, и как и в предыдущей главе, тут есть переадресовочные элементы на разные секции, созданные упаковщиком во время выполнения.

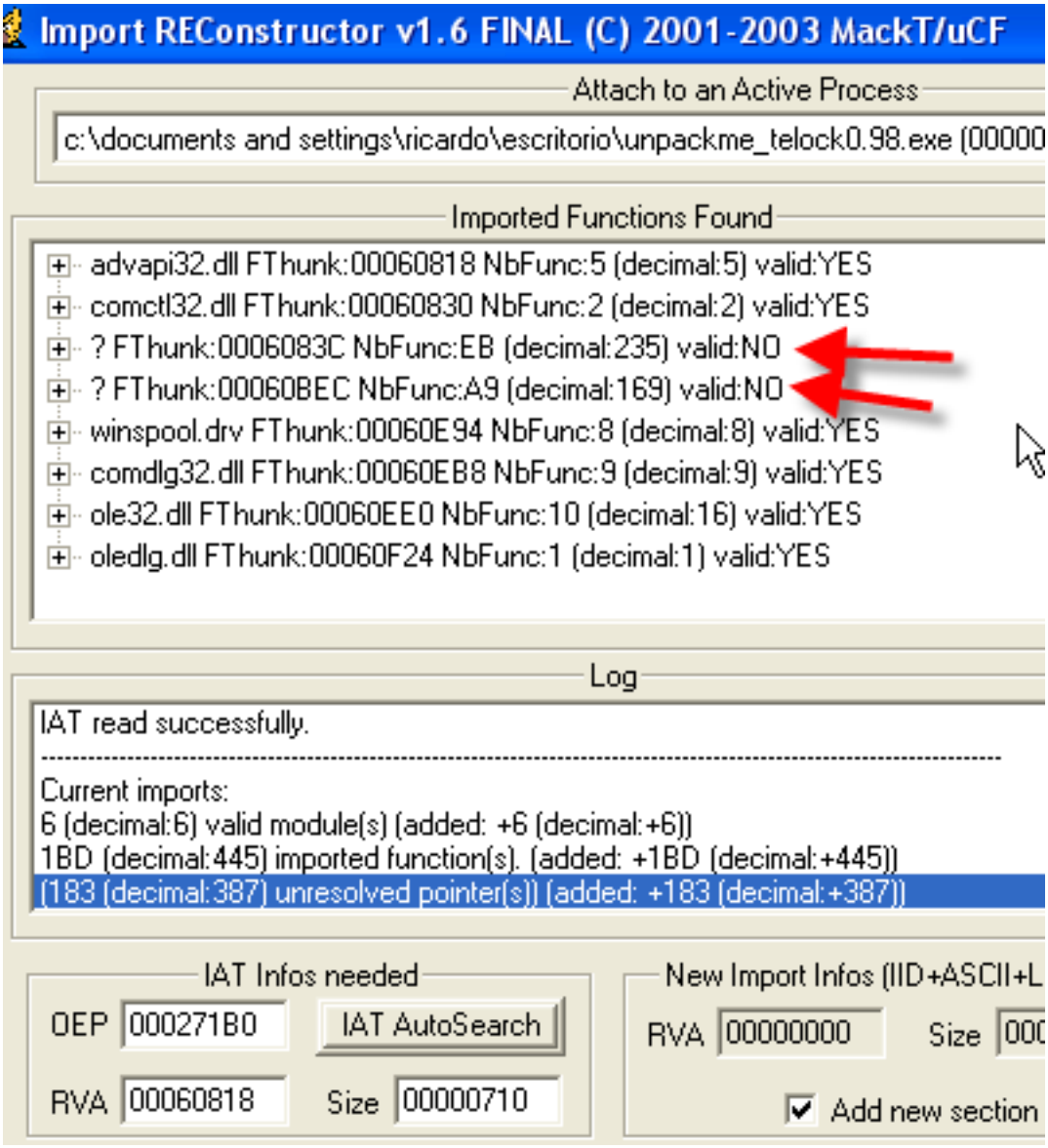


Окинем IAT и её элементы, которые нужно починить, общим взглядом. На самом деле, некоторые упаковщики переадресовывают всего лишь один-два элемента, поэтому желательно знать методы, позволяющие починить конкретный элемент, который можно обнаружить в самом начале. Конечно, если таких элементов много, то вы состаритесь прежде, чем их почините данным образом, но знать, как определить, какая API-функция относится к какому элементу, необходимо, так как иногда нужно проверить сомнительный элемент.

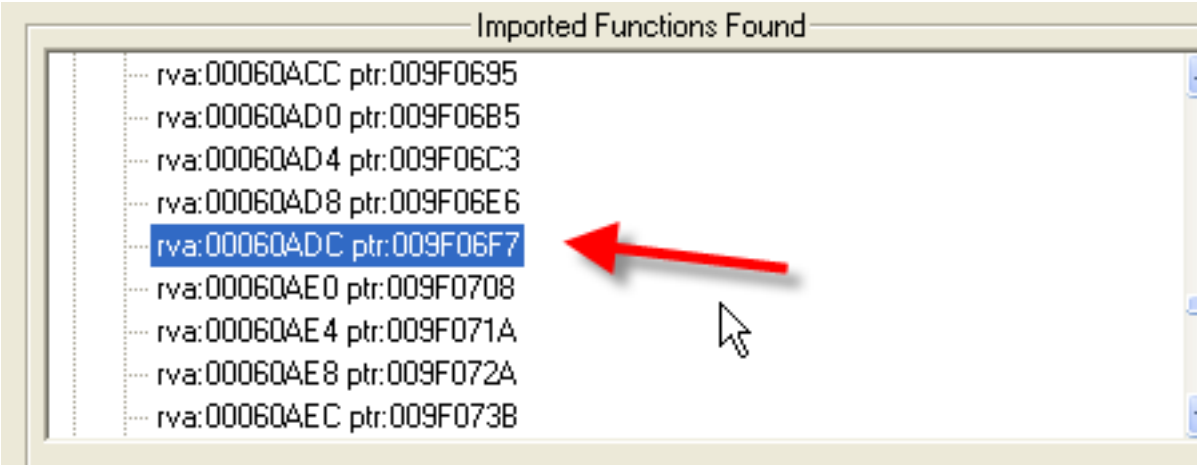
Возьмём элемент, который мы трассировали вручную и знаем, что он в итоге приводит к GetVersion.



Здесь есть CALL, а в DUMP'е – элемент IAT 460ADC, чьё содержимое на моей машине указывает на 9F06F7. Также у нас должен быть открыт IMP REC со всеми значениями, RVA, РАЗМЕРа и OEP, которые были найдены вручную, и если глянем на плохие элементы:



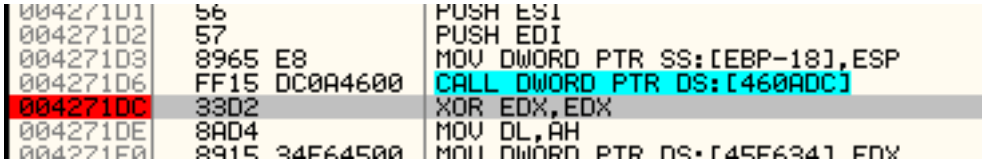
Раскрываем, чтобы увидеть эти элементы.



60ADC соответствует элементу IAT 460ADC.

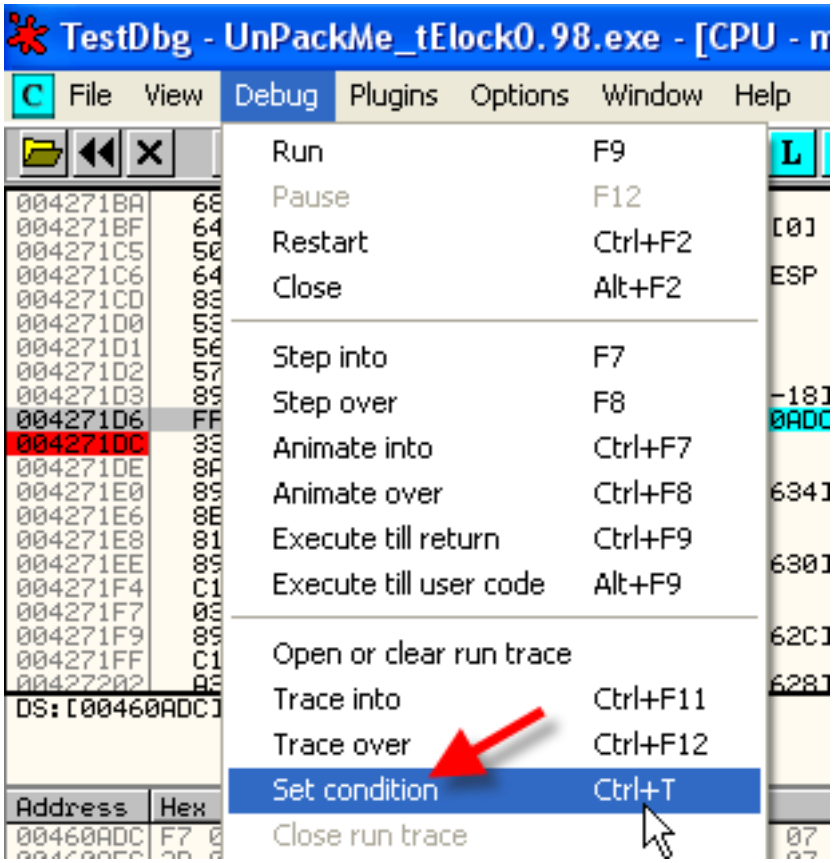
Вернёмся к OllyDbg и нашему элементу, мы уже рассматривали как дойти до API-функции, трассируя в ручную, и знаем, что у OllyDbg есть встроенный трассировщик. Используем его, чтобы сделать то же самое гораздо быстрее. Конечно, здесь у нас всего 5-6 строк, которые нужно

оттрасировать, но есть упаковщики, петляющие и виляющие на пути к API-функции, поэтому автоматическая трассировка бывает в таких случаях полезна.



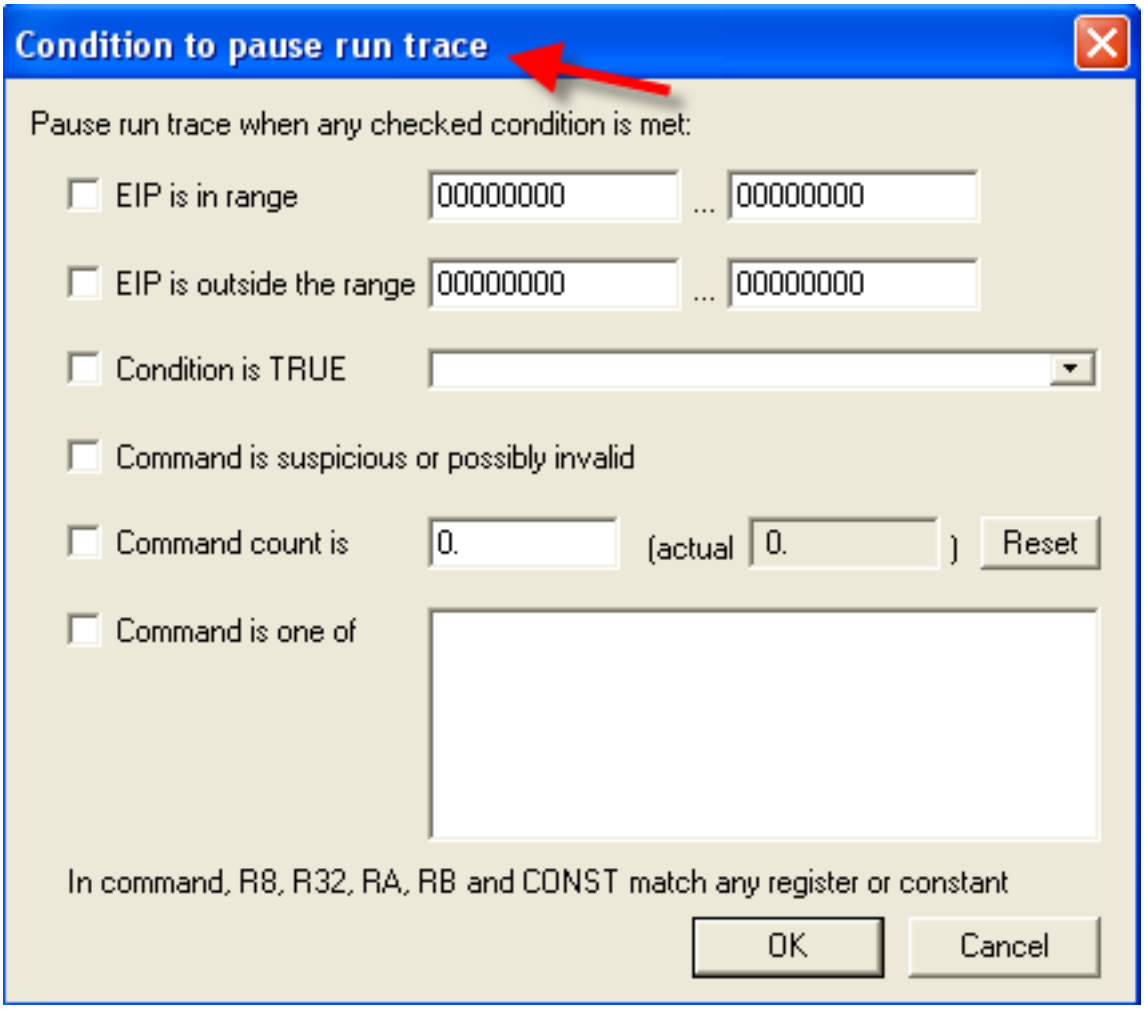
До начала трассировки устанавливаем BP на возврат из API-функции, чтобы не случилось так, что мы ошиблись и не оттрасировали до самого конца без остановки.

В DEBUG-SET CONDITION можно выбрать опции, при которых OllyDbg остановит трассировку.

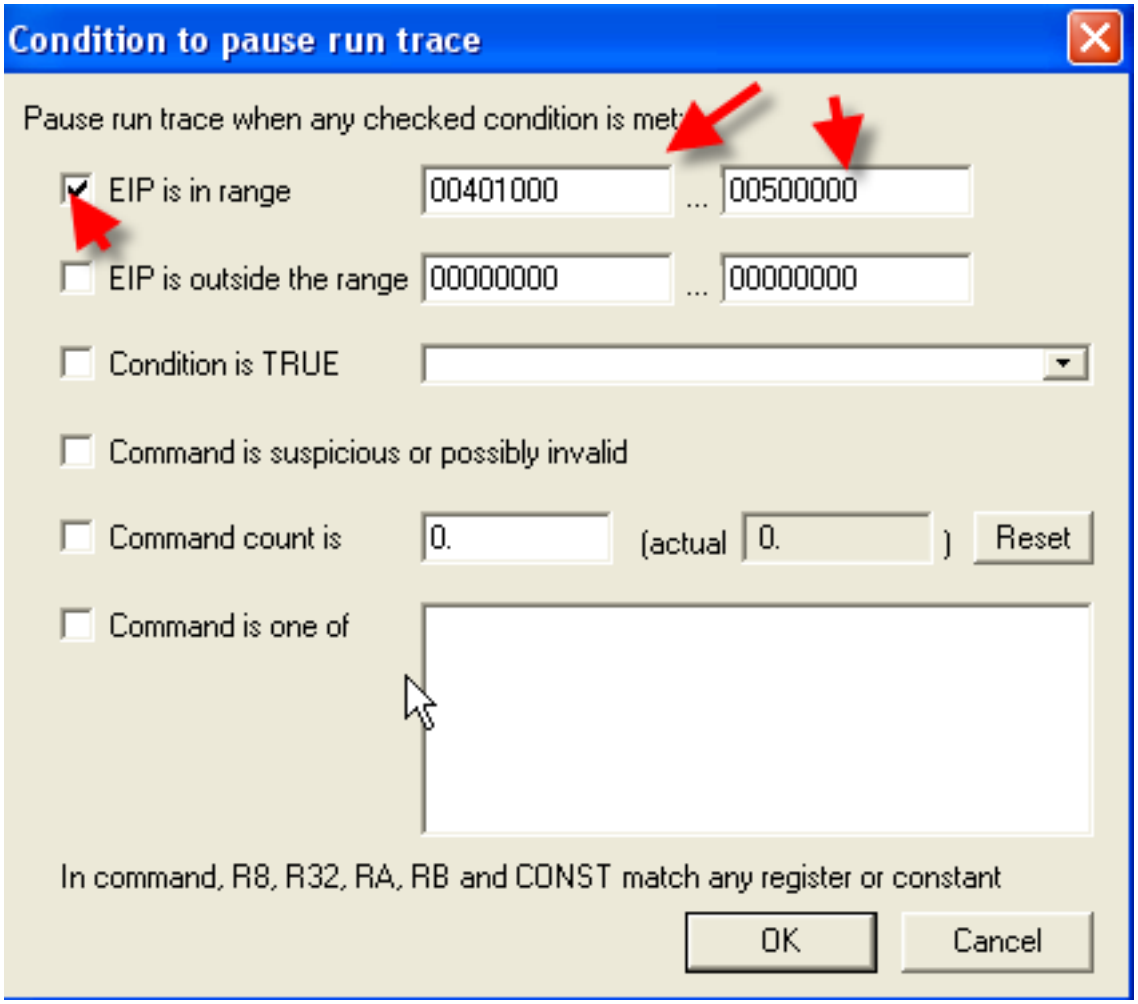


Видим окошко под названием «Условия остановки трассировки», посмотрим, что за условия и какие из них можно использовать в нашем случае.

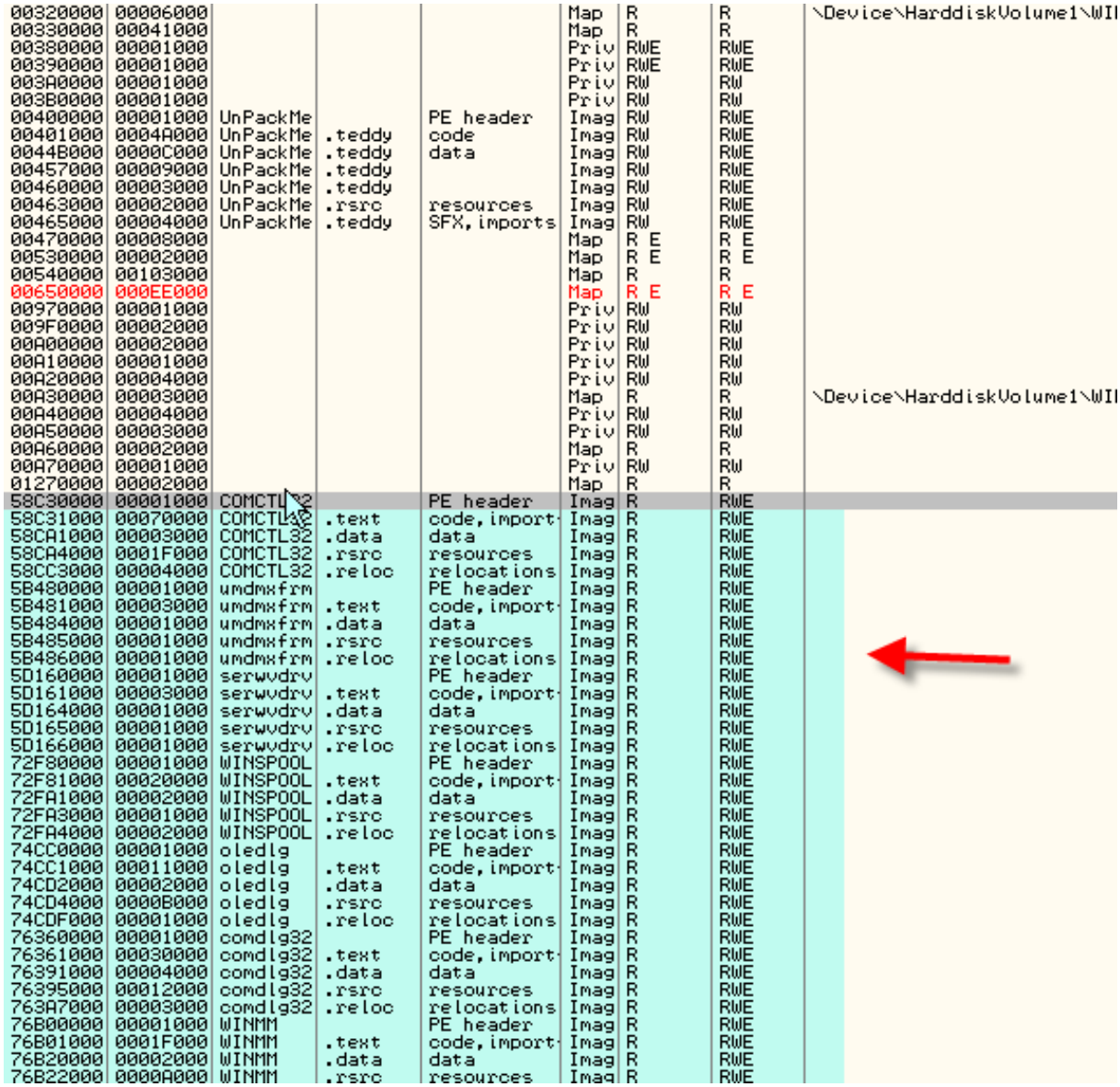
EIP IS IN RANGE означает, что если поставить здесь галочку, OllyDbg остановится, когда EIP окажется в границах интервала значений, заданного справа, например:



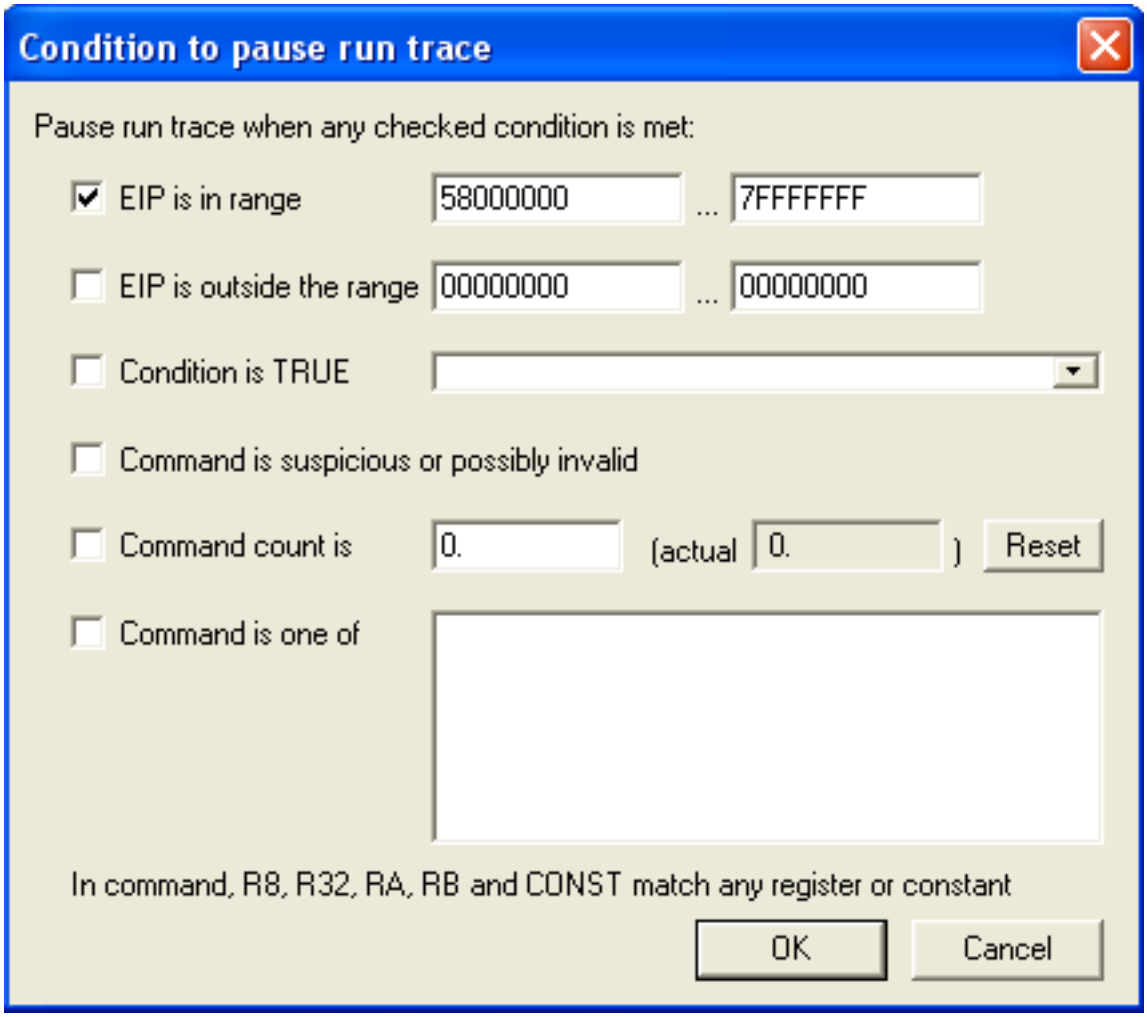
Этот пример означает, что OllyDbg будет трассировать, пока не увидит, что EIP содержит значение между 401000 и 500000. Конечно, это всего лишь пример, и в нашем случае он не помощью, так как мы хотим, чтобы остановка произошла на API-функции. Для нахождения нужных нам значений, посмотрим карту памяти.



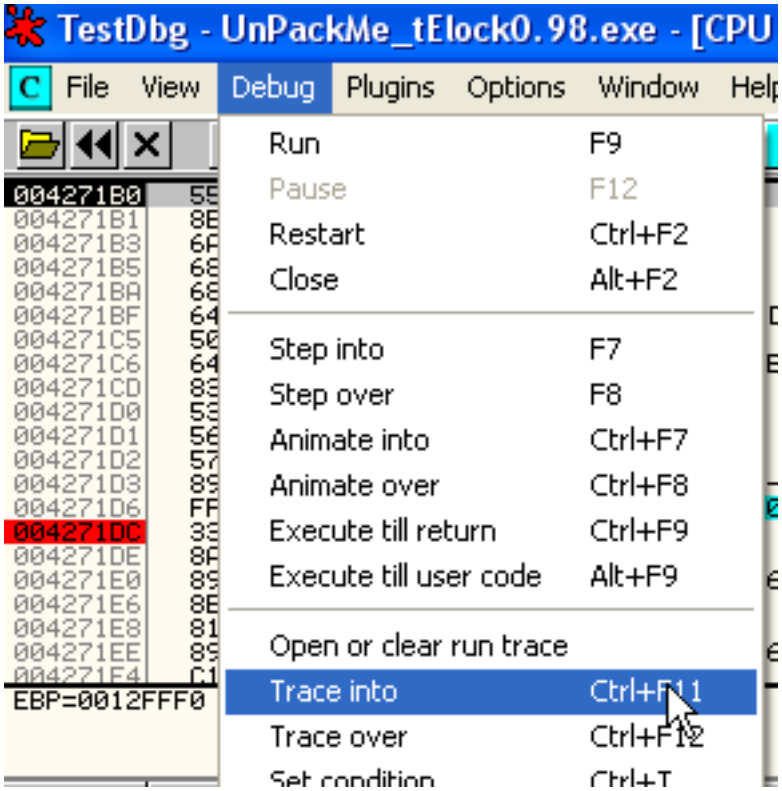
Здесь видим область, отмеченную лазурным цветом, которая может представлять для нас интерес, так как здесь остановился OllyDbg при трассировке, то есть отсюда пойдёт первая DLL. Мы не хотим останавливаться ни в секциях exe-файла, ни в секциях, созданных упаковщиком, так как это промежуточный код, и нам нужно останавливаться в DLL, поэтому отметим первый столбец и установим, что EIP должен быть между 58000000 и 7FFFFFF, то есть будет содержать адрес, максимально гарантирующий, что остановимся в какой-либо DLL (пояснение: не обязательно задавать точный адрес первой DLL, так как мы видели, что между 58000000 и 58C30000 (началом первой DLL) нет кода, так что мы там не остановимся, и во избежание необходимости задавать точные цифры можем без всяких проблем их слегка округлить).



Это всегда означает, что OllyDbg запустит построчную трассировку, и в каждой будет проверять, что выполняется указанное условие, то есть содержит ли EIP значение в пределах секции данной DLL, и если да, то остановит выполнение.

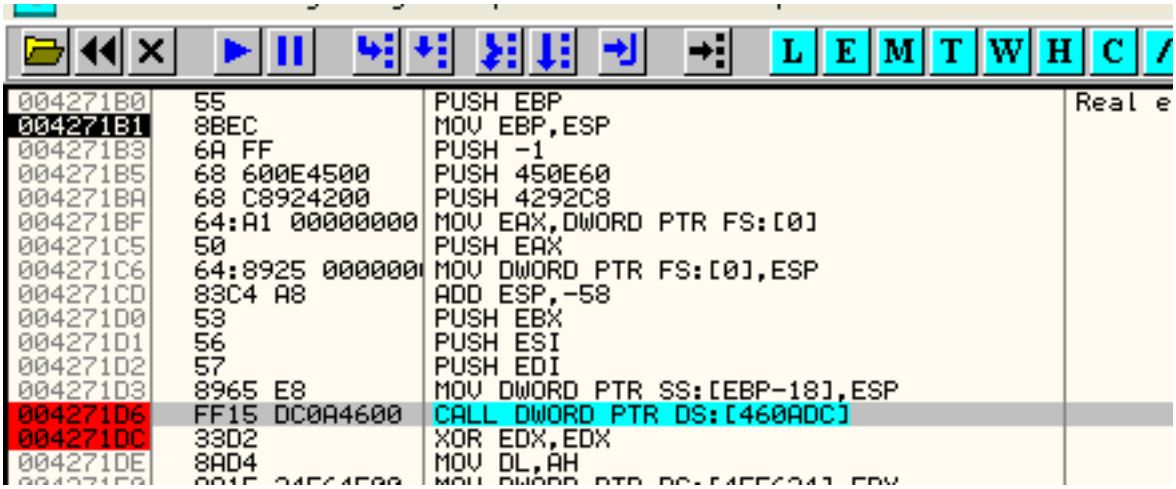


Доходим до CALL и устанавливаем на него BP.

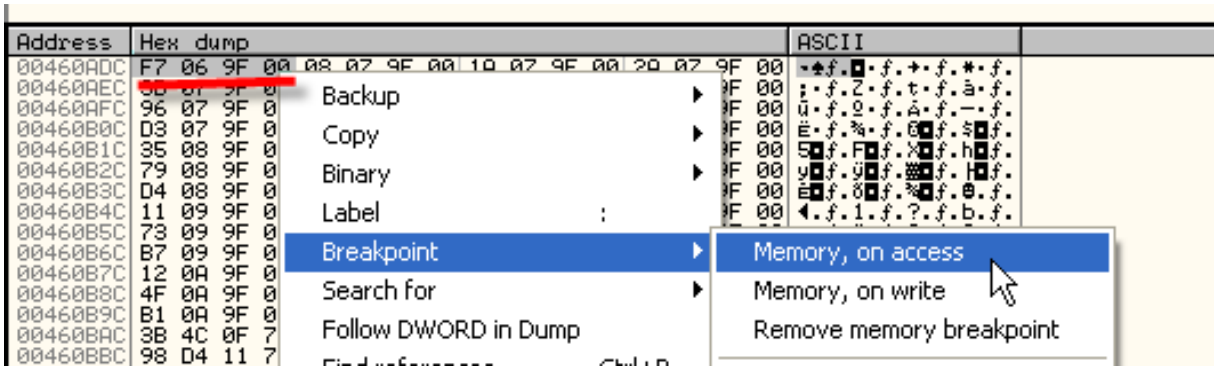


Если до этого был установлен MEMORY BREAKPOINT, использовавшийся для нахождения OEP, то убираем его и доходим до CALL.

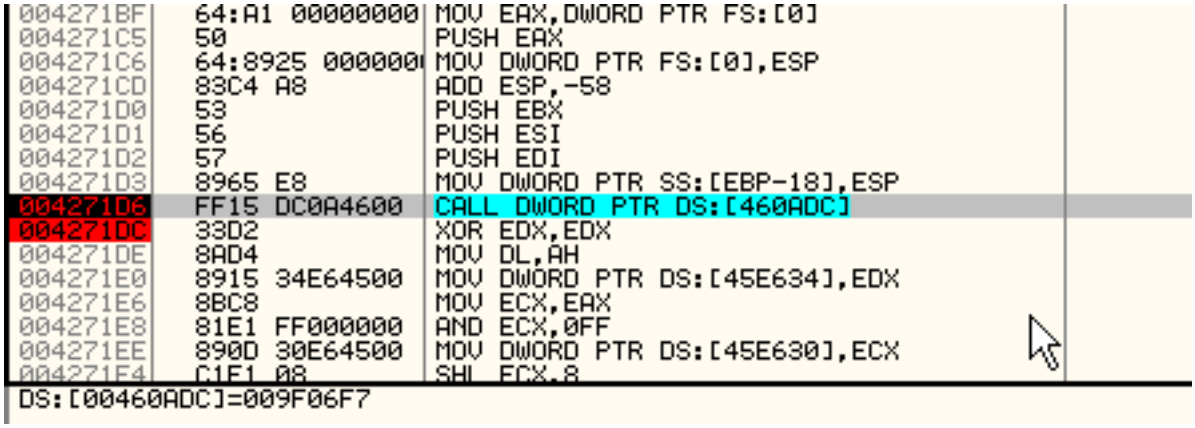
В случае, если нет CALL'а в начале, и не знаем, где может находиться он или JMP, можно отметить байты соответствующего элемента и установить на них MEMORY BREAKPOINT ON ACCESS, после чего нажать RUN. Остановка произойдёт, когда элемент будет прочитан, то есть сработает соответствующий JMP или CALL.



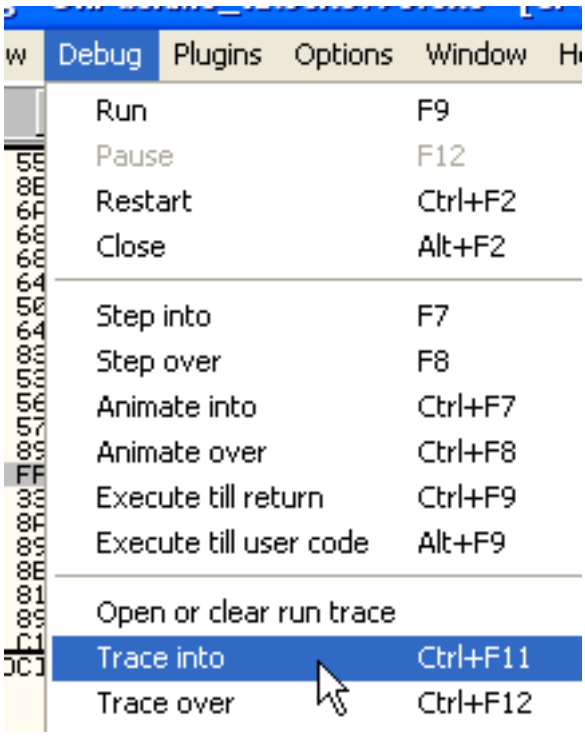
Хорошо, дошли до CALL'а с помощью любого из этих двух методов.



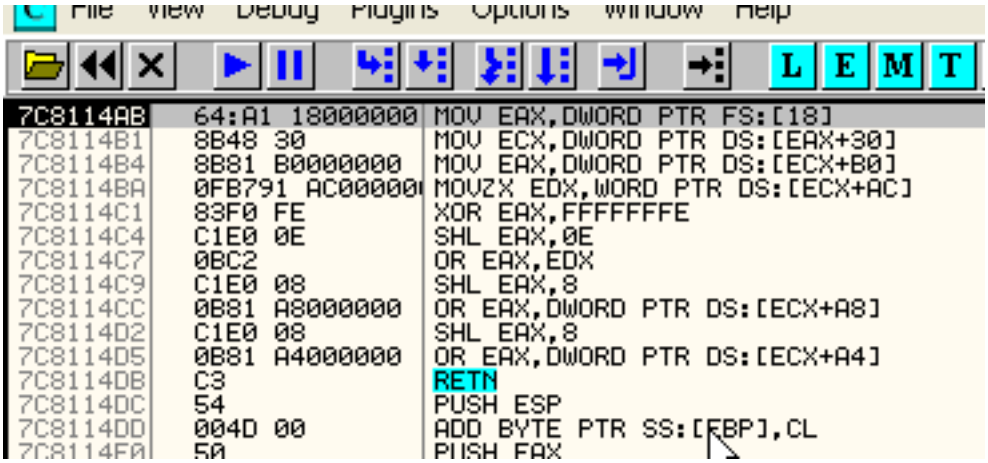
Входим в режим трассировки в OllyDbg.



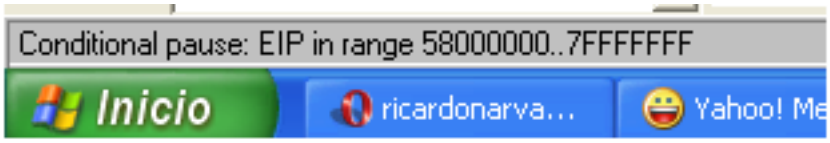
Важно выбрать TRACE INTO, чтобы трассировать строку за строкой. Если выберем TRACE OVER, не будем входить в CALL'ы и можем потерпеть неудачу в задуманном нами.



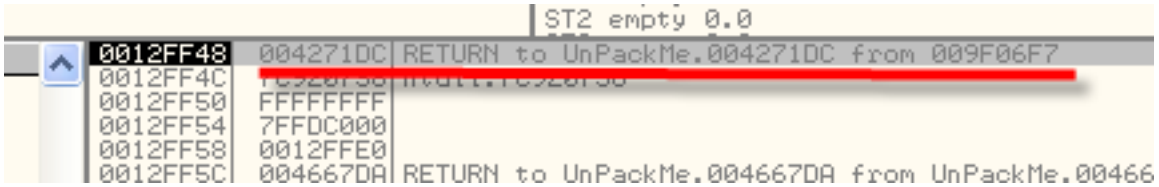
Теперь останавливаемся на условии, установленном нами. Если глянем в нижнюю часть OllyDbg:

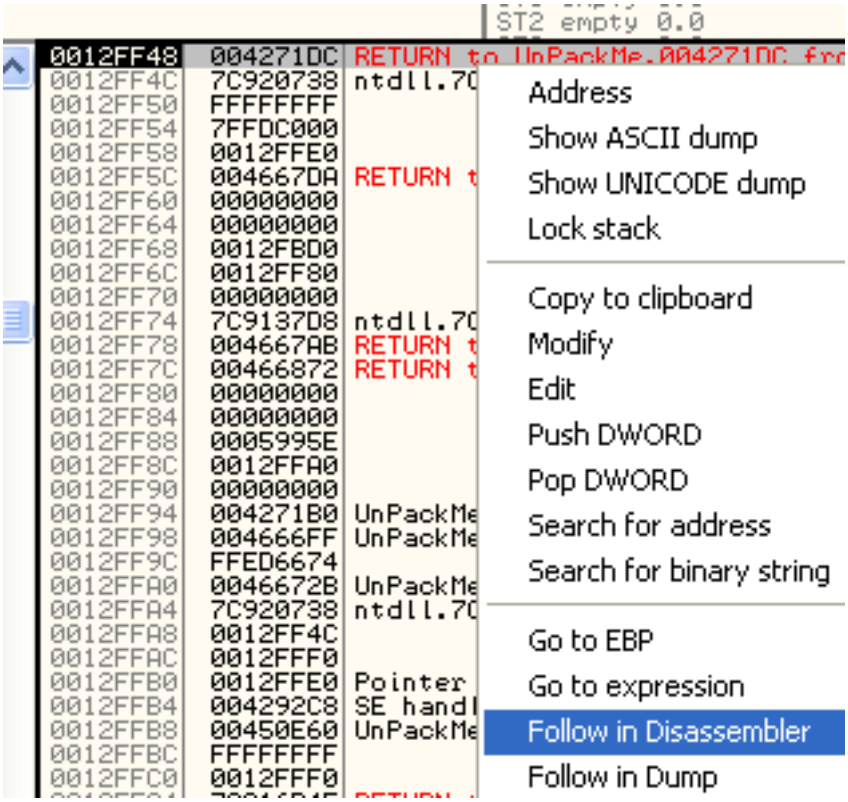


Остановились, так как EIP находится в диапазоне 58000000-7FFFFFFF. Давайте проверим и убедимся, что остановка не произошла из-за исключения или по какой другой причине.



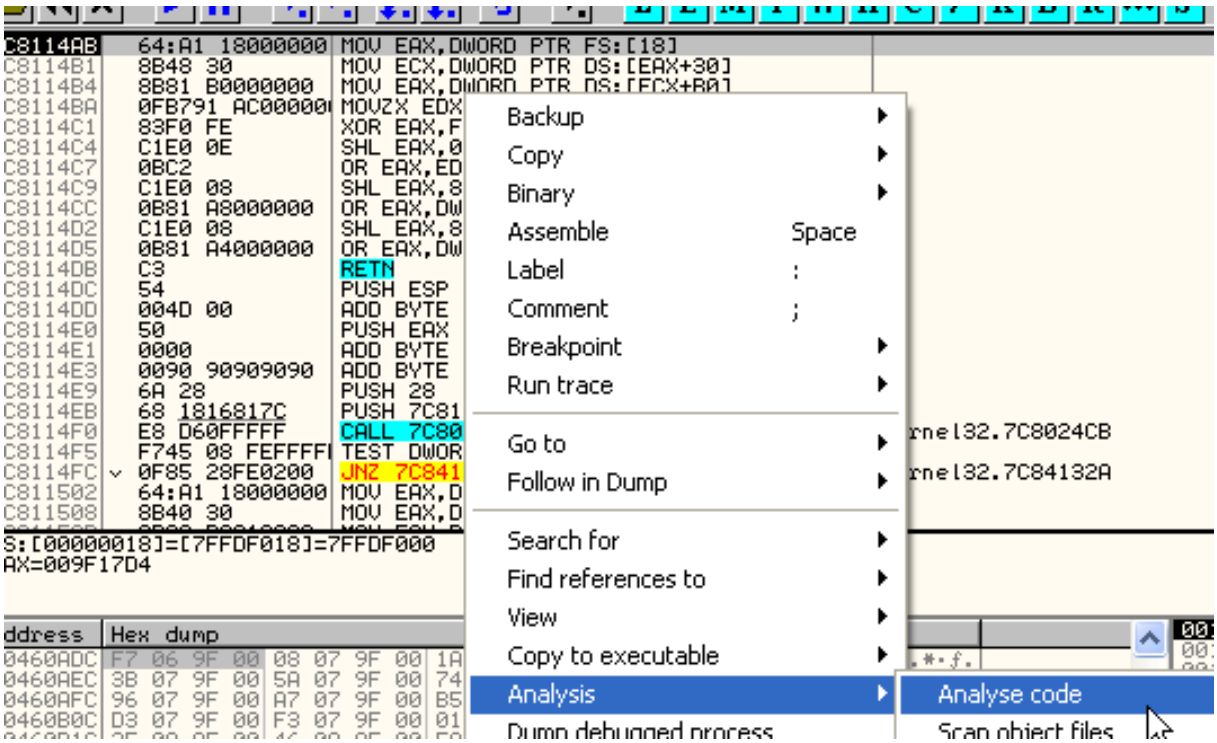
Следующее, что нужно сделать – это проверить, происходит ли после выполнения API-функции возврат в основную программу на строку после CALL, так как есть упаковщики, которые для запутывания идут сначала в одну API-функцию, а продолжают выполнение другой API-функцией, и истинной целью из них является последняя, и именно ею возвращённое значение будет использоваться программой и инструкцией, идущей непосредственно за CALL'ом.



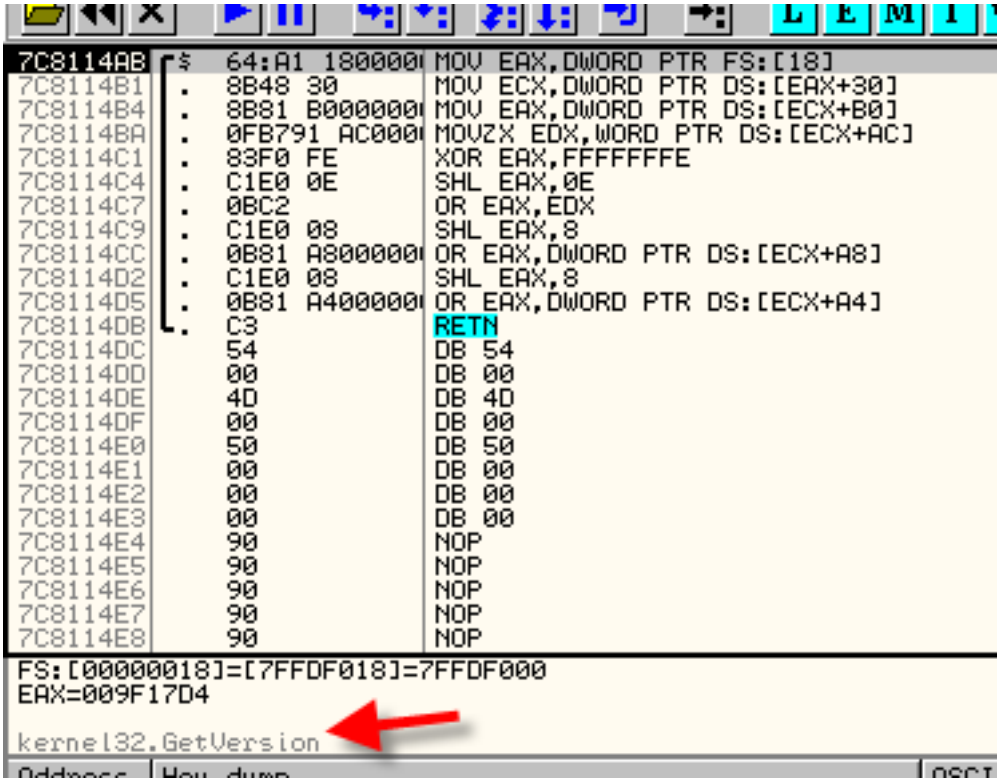


Возвращаемся в 4271DC, как и должно быть.

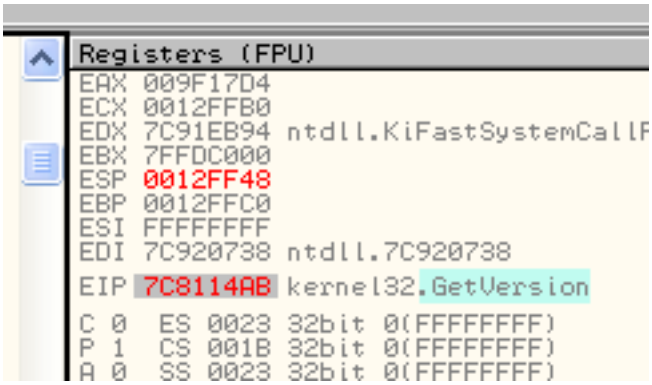
Ок, сейчас нам не удалось увидеть, что это за API-функция, так как OllyDbg ничего не сказал, и перво-наперво нужно проанализировать код из этой DLL.



Видим непосредственно внизу пояснение, в котором OllyDbg отображает имя API-функции, которой соответствует значение EIP.

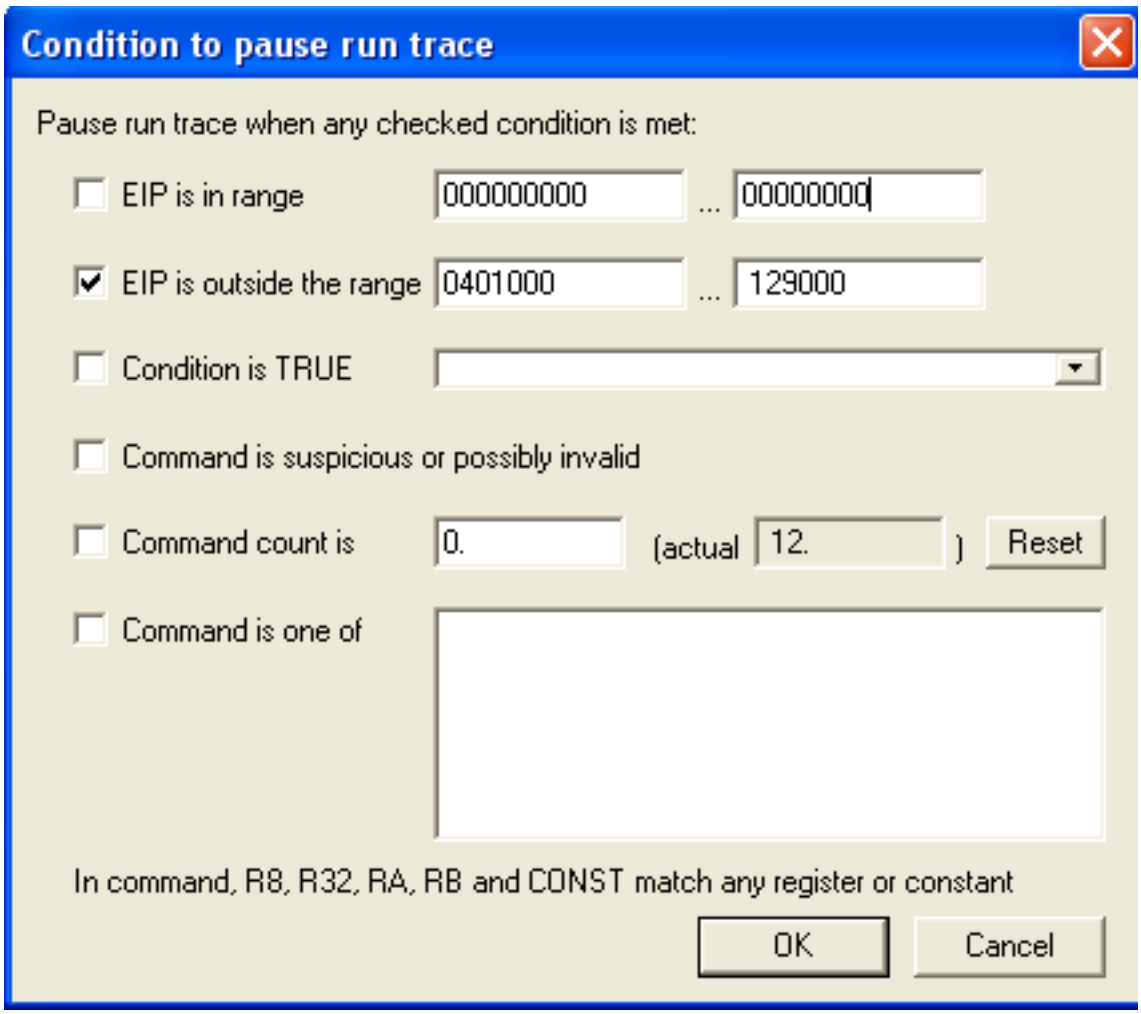


Другая возможность оттрасировать CALL, чтобы дойти до API-функции – это:



Данный случай – это противоположность предыдущего. OllyDbg проверяет, что EIP выходит за рамки секций EXE-файла и секций, идущих до первой DLL.

Другая возможность:



В редких случаях, когда упаковщик создаёт смешанные секции внутри DLL, можно использовать комбинированный метод: сначала остановиться при нахождении RET, а затем – когда первая строка стрека будет равна 4271DC, то есть возврату из API-функции. Этот метод имеет целью остановку на возврате из API-функции, и во многих случаях он эффективен для многих упаковщиков, которые эмулируют первые строки функции и прыгают на третью или четвёртую, обычно RET всегда соблюдается и остановка происходит на нужной строке. Если рестартуем OllyDbg и снова остановимся на CALL'е, то можем его быстренько опробовать.

То есть здесь нужно указать два условия за раз с помощью объединения их, используя “&&”, который указывает, что оба условия должны выполняться, то есть “&&” эквивалентно “И”.

В случае, если захотим остановиться на том месте, где выполняется одно из условий, используем “||”, что эквивалентно “ИЛИ”.

То есть

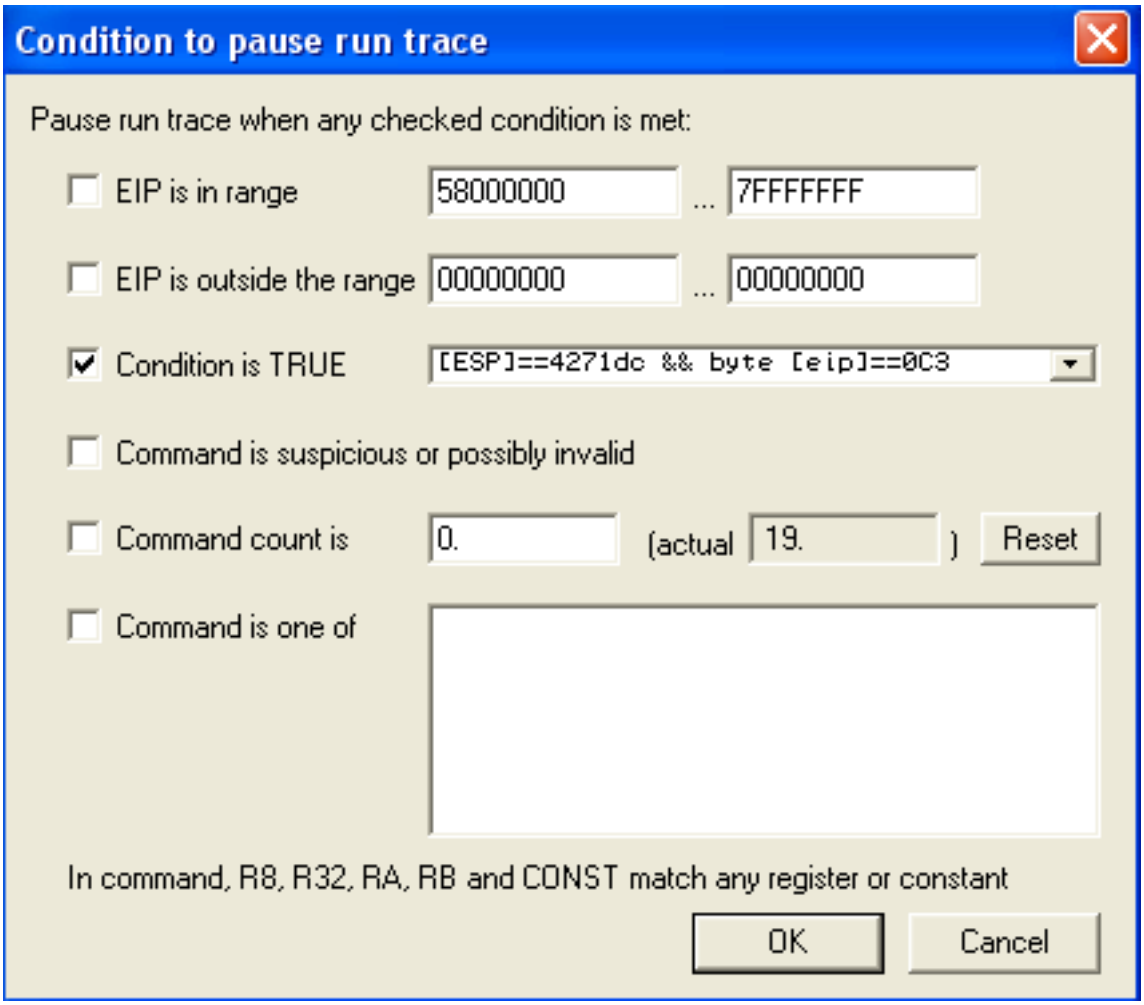
[ESP]==4271dc && byte [eip]==0C3

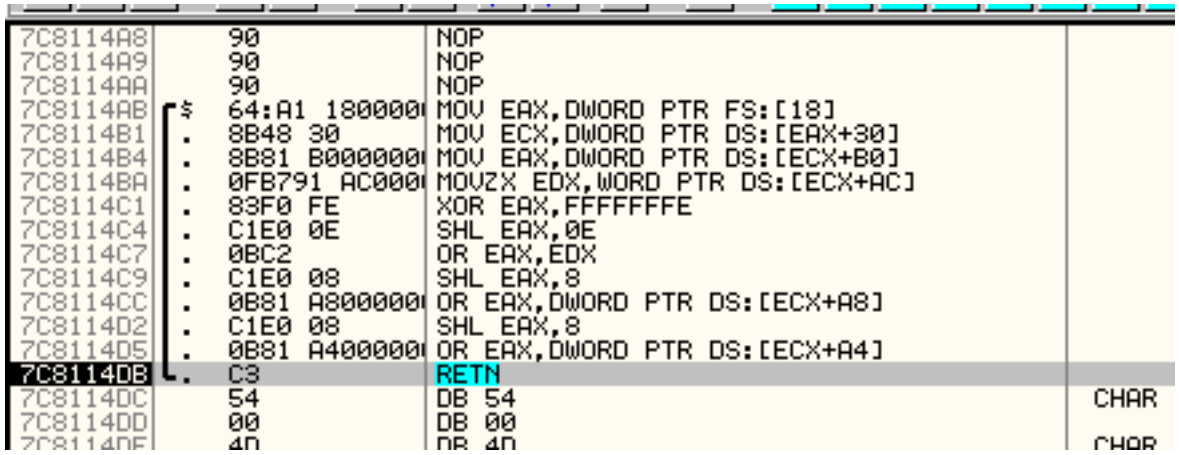
означает, что выполняются одновременно два условия, то есть

[ESP]==4271dc (первая строка стека, являющаяся точкой возврата из API-функции)

byte [eip]==0C3 (байт, на который указывает EIP и который запускается – RET, то есть C3)

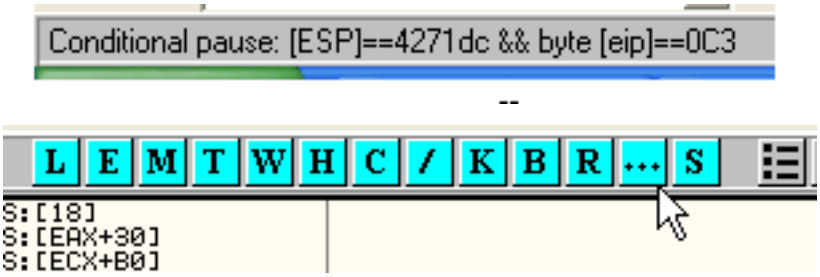
Если от CALL запустим трассировку с данным условием, то увидим, что произойдёт остановка на RET’е API-функции.



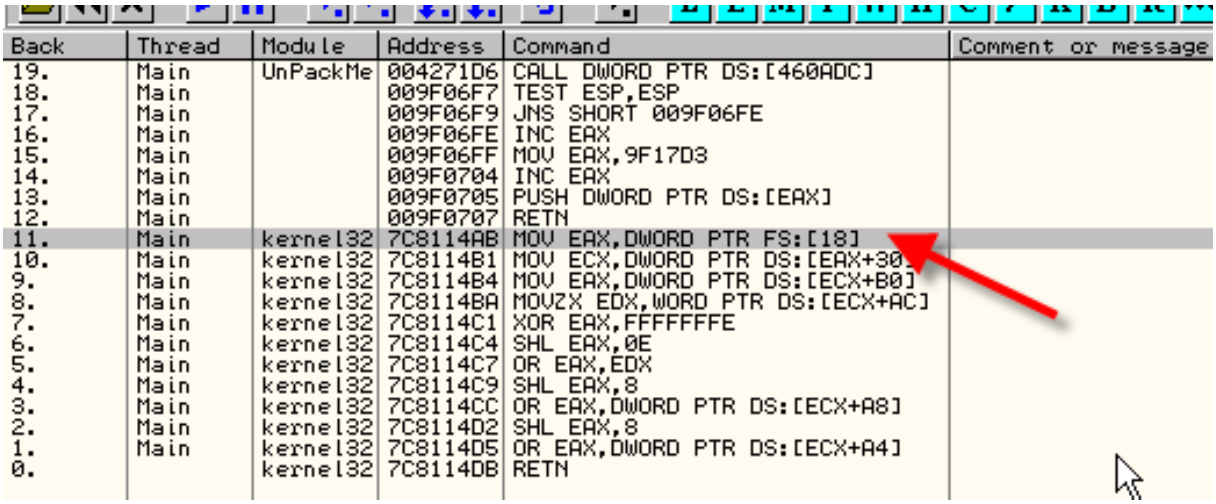


При срабатывании обоих условий за раз, трассировка останавливается. Сейчас, находясь на RET'е, нам сложнее узнать имя выполняющейся API-функции, так как не находимся в её начале. То же самое случается, когда упаковщик имитирует две-три первые строки API-функции и находимся на 4-ой или 5-ой – тогда OllyDbg не показывает имя, но есть различные способы, чтобы его узнать.

Смотрим список того, что мы трассируем, для чего нажимаем на кнопку с тремя точками.



Видим, что первая оттрасированная строка DLL отмечена стрелкой. Имя API-функции нам не показывается, так что сделали анализ кода, а затем – двойной щелчок, отмеченный стрелкой.



Видим, что OllyDbg нам показывает содержимое регистров на этот момент. Они отображаются серым, потому что это не текущие значения. Также нам показывается информация, связанная с выполнением данной строки, среди которой есть и имя API-функции. То же самое произойдёт, если вместо трассировки будем нажимать на клавишу «минус», OllyDbg также будет показывать информацию и значения, сохранённые при проходе через каждую инструкцию.

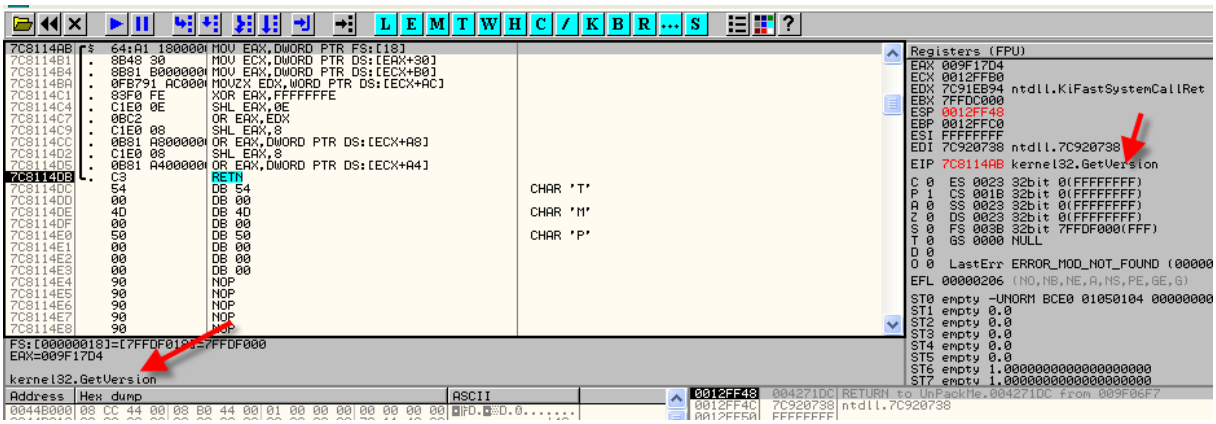
В случае с упаковщиками, которые эмулируют две или три первые строки API-функции, и при трассировке идут на четвёртую или пятую строку, трассировщик не проходит через первую строку, так как они эмулируются упаковщиком, поэтому на это нужно обращать внимание. Перейдём к

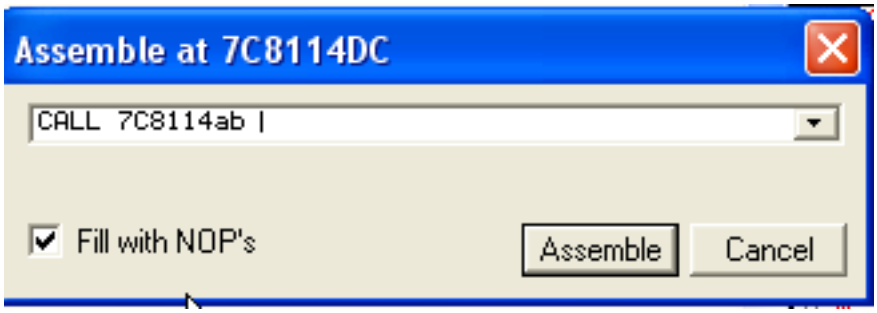
следующему методу.

В любой части после RET введём CALL (используя “Assemble” или нажав на клавишу пробела на строке), который должен вести в место, являющееся, как мы подозреваем, началом API-функции. В данном случае:

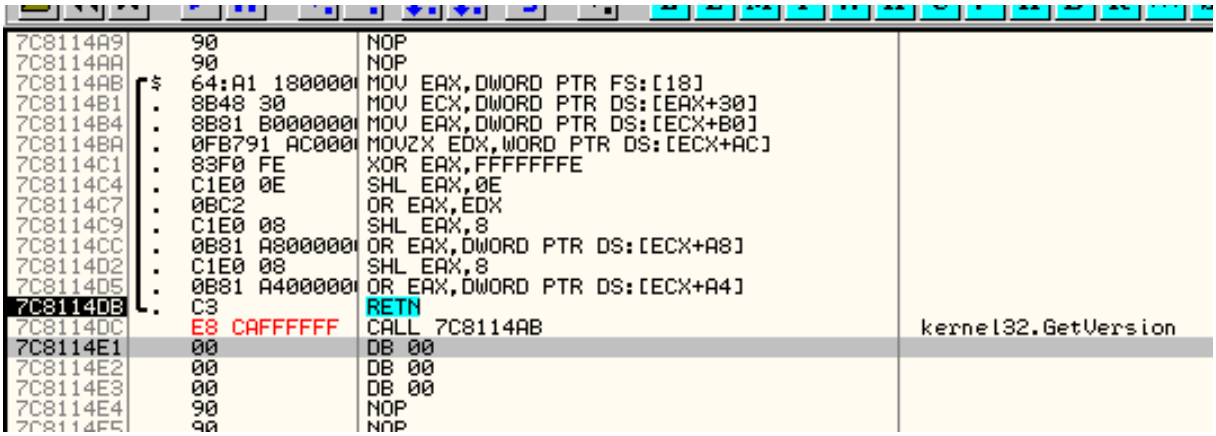
CALL 7C8114ab

Похоже на начало API-функции на моей машине, ввожу адрес:

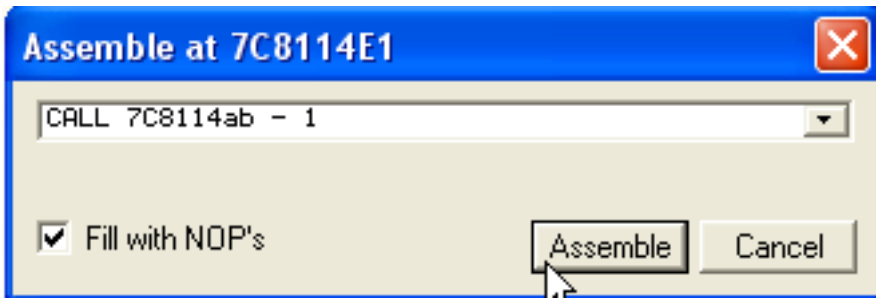




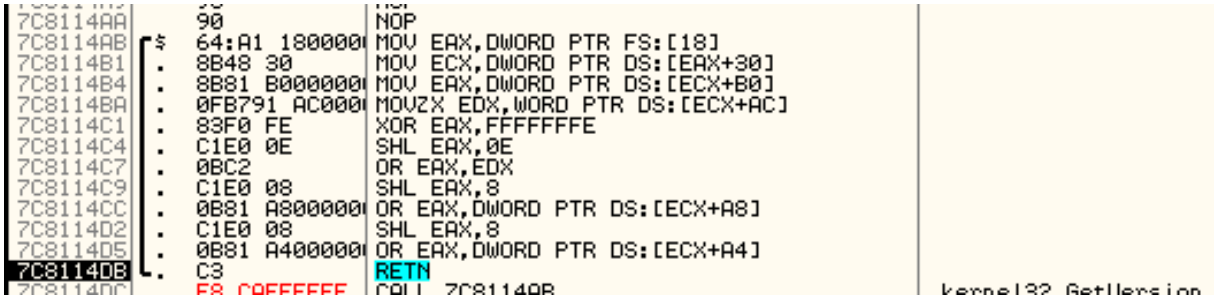
Если это действительно начало API-функции, OllyDbg покажет нам имя, если мы ошиблись, то можем повторить:



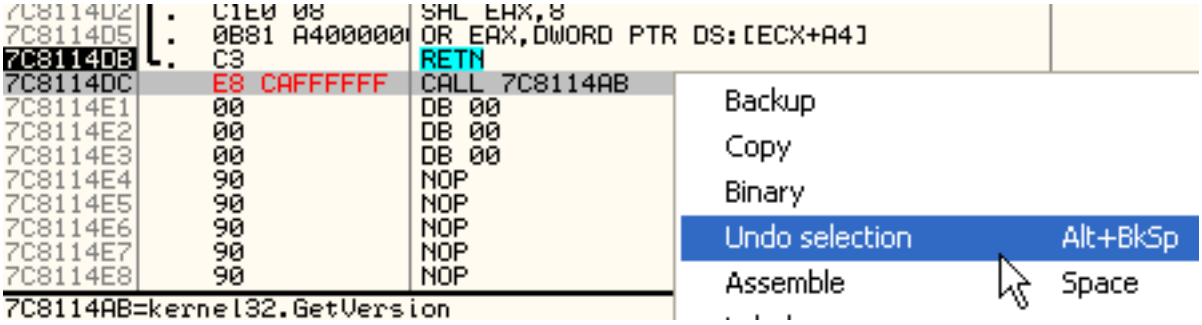
И так делаем в области, где должно располагаться начало API-функции, пока OllyDbg нам не покажет имя API-функции. Обычно довольно легко предположить, где находится начало,

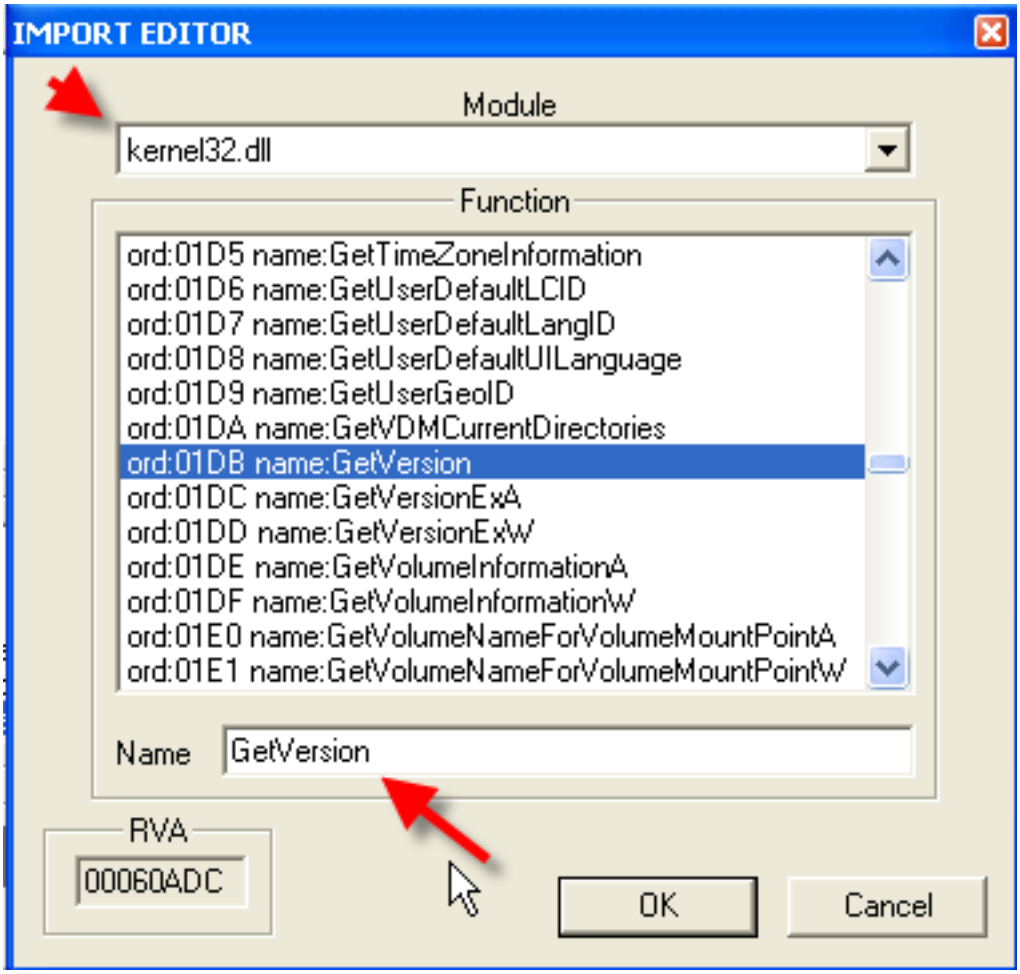


Убираем введённую нами строку с помощью UNDO SELECTION.



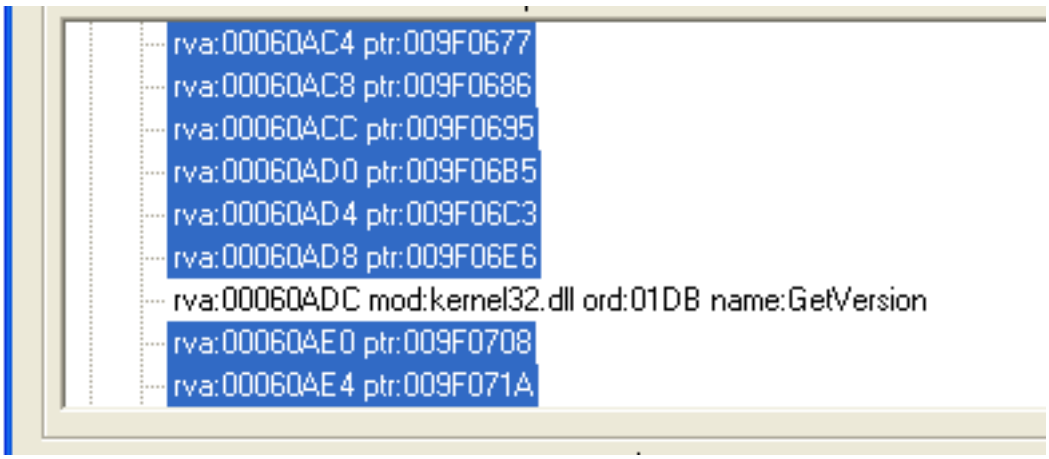
Ок, мы научились вручную определять имя API-функции, так что можем пойти в IMP REC, сделать двойной щелчок на неправильном элементе, который нужно починить, и ввести правильное имя.



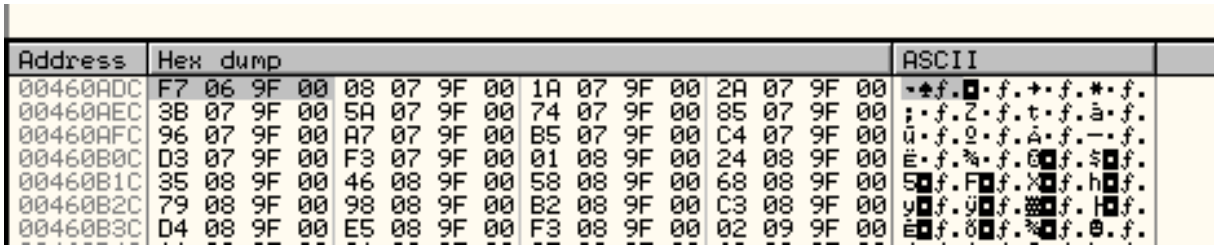


Теперь нажмём SHOW INVALID и увидим, что указанный элемент уже отмечен как правильный, что для нас очень хорошо.

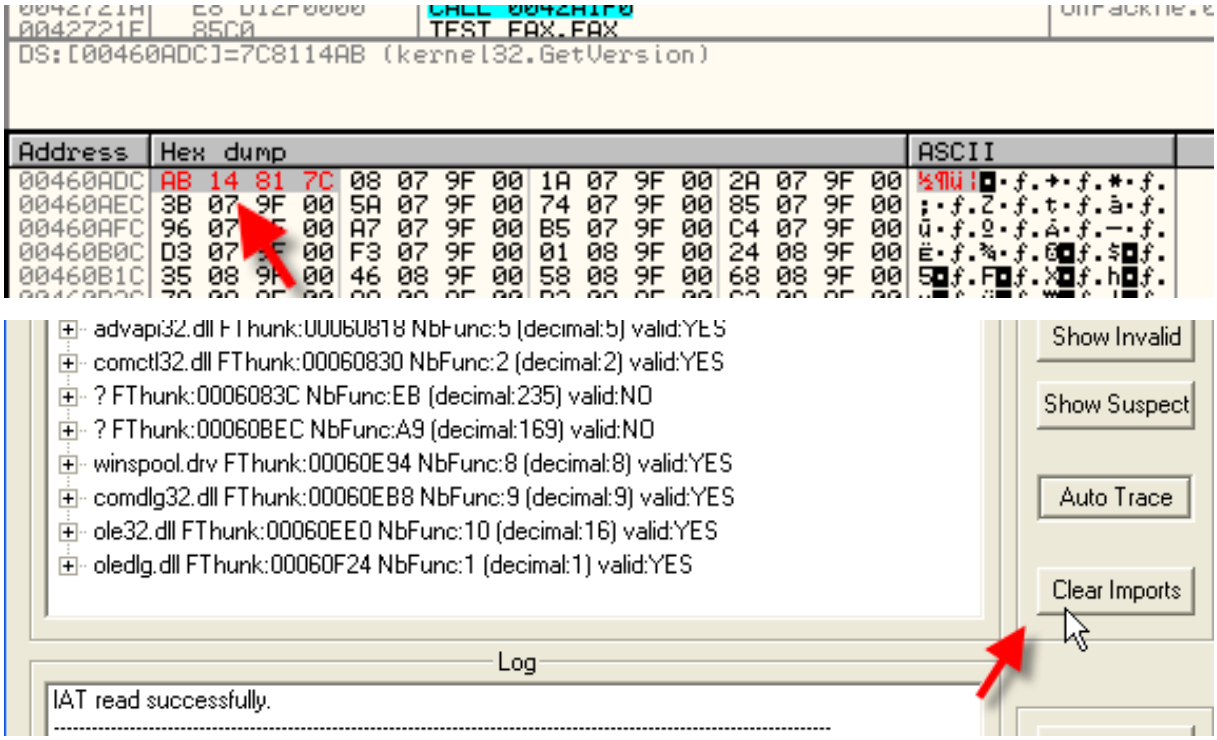
Другой способ, которым можно починить элемент – это написать в данном элементе с помощью OllyDbg правильный адрес API-функции на нашей машине.

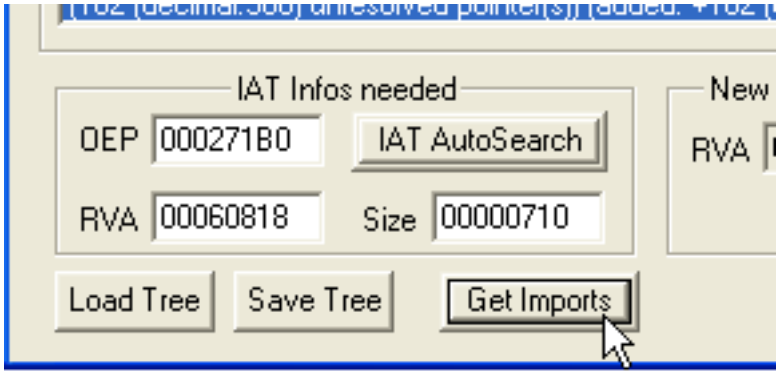


Элемент, который мы исследуем, находится по адресу 460ADC. Перезапишем его значение найденным адресом API-функции на нашей машине (мы ранее узнали, что API-функция начинается на моей машине по адресу 7C81114AB).



Теперь видим, что элемент починен. Если сейчас очистим в IMP REC'е все элементы, нажав на кнопку CLEAR IMPORTS, а потом снова нажмём GET IMPORTS, то увидим, что элемент отмечен как правильный.

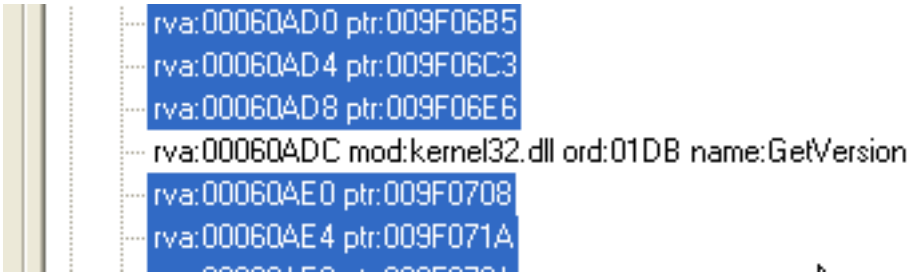




Как видим, указать правильный адрес API-функции в соответствующем элемент напрямую в IAT с помощью OllyDbg или указать имя API-функции в IMP REC'е для данного элемента – это одно и то же.

Хорошо, если проделаем это с каждым из переадресовочных элементов, а затем запустим дамп, то он будет работать, проблема в том что этот метод очень скучный, хотя и очень хорошо, для того, чтобы развеять любые сомнения, имеющиеся относительно каких-либо элементов.

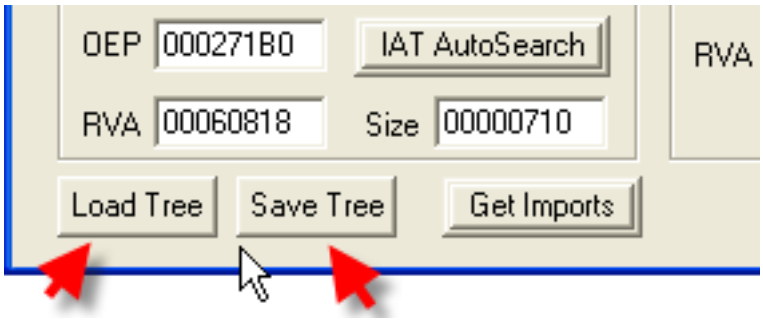
В IMP REC'е есть функция для помощи фанатикам ручного метода – он позволяет сохранить таблицу с исправлениями с помощью нажатия на кнопку "SAVE TREE", а потом, если прервали работу, можем вернуться на OEP программы, открыть IMP REC, установить данные OEP, RVA и SIZE, а затем загрузить сохранённый ранее IAT, используя кнопку "LOAD TREE".



Другим возможным способом починки переадресовочных элементов является использование плагинов для IMP REC. В данном случае идём в папку:

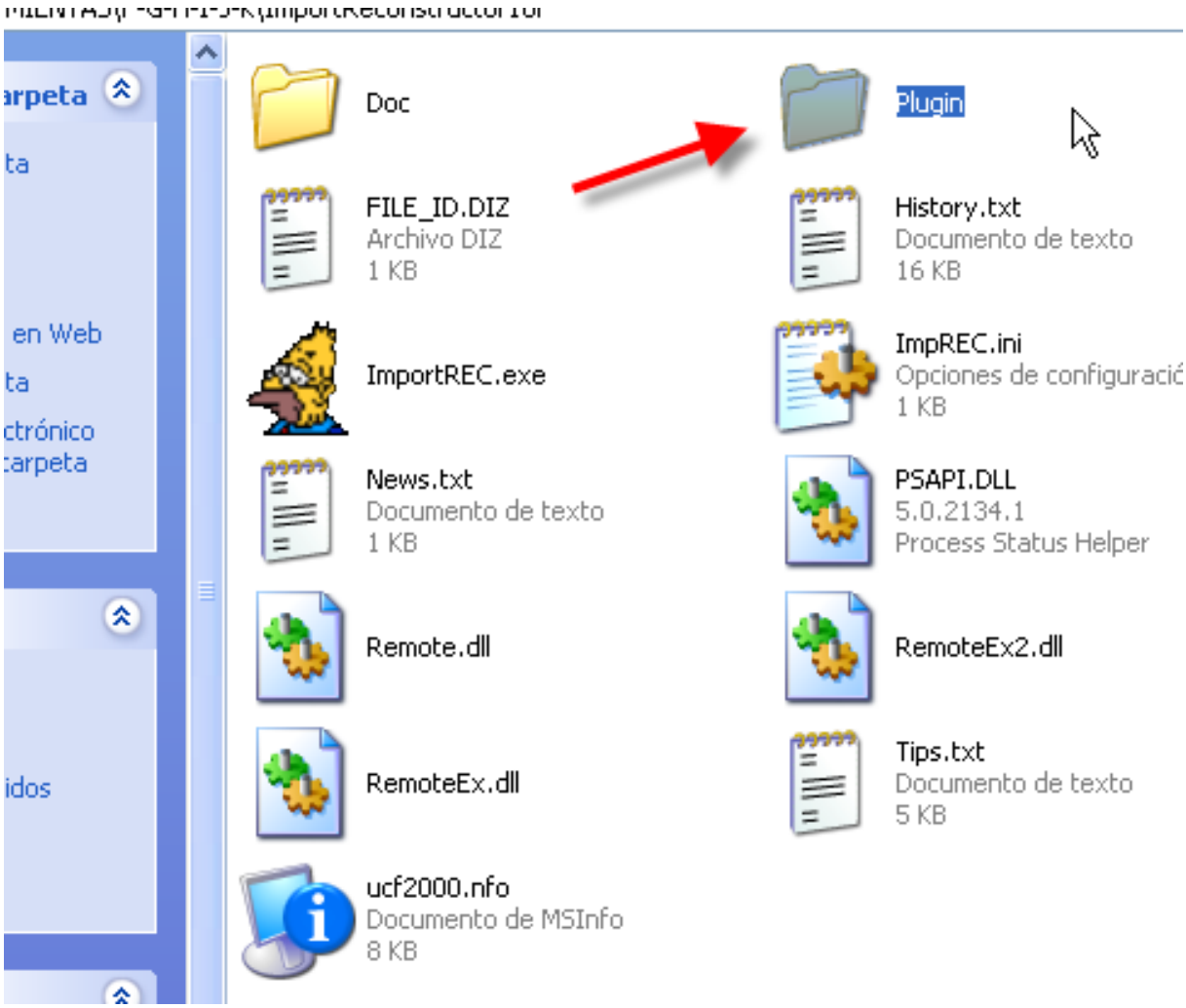
http://ricardonarvaja.info/WEB/OTROS/HERRAMIENTAS/L-M-N-O-P/Plugin_Import/

Видим, что есть плагин для telock, который нужно скопировать в папку плагинов для IMP REC'а. Посмотрим, как он работает.



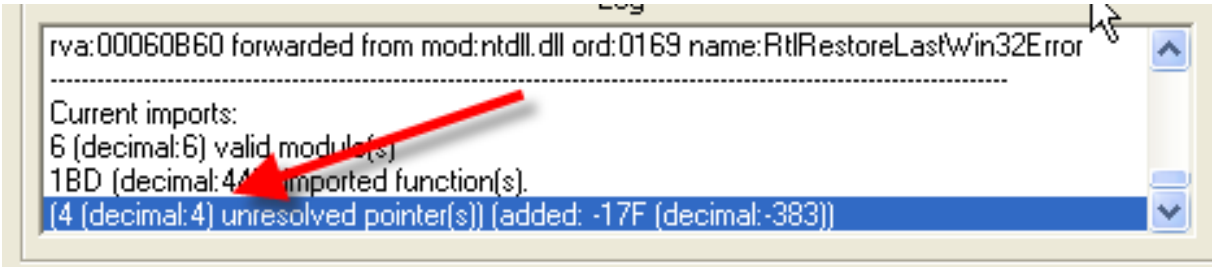
Копируем DLL в папку PLUGIN.

Теперь перезапускаем IMP REC, и возвратившись к вводу данных, нажимаем SHOW INVALID, делаем щелчок правой кнопкой мыши, выбираем PLUGIN TRACERS и ищем telock. После трассировки видим, что исправны все элементы, кроме четырёх, поэтому видим важность ручной трассировки, так как плагин пропустил несколько элементов, которые можно починить вручную.

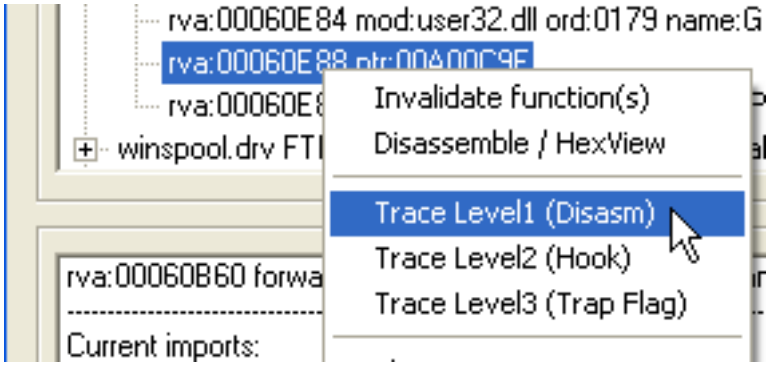


Здесь видим, что IMP REC сообщает нам об оставшихся 4 элементах, которые нужно починить. Их можно оттрассировать вручную, как мы это делали в начале. Таким образом, использование плагина имеет свои ограничения, так что всё зависит от того, существует ли плагин для упаковщика, который применяется в данном случае.

Кроме плагинов у IMP REC'а есть встроенные трассировщики. Вы можете попробовать, сработают ли она, но перед этим рекомендуется сохранить результаты работы, так как в большинстве случаев их использование приводит к зависанию IMP REC'а.



Отмечаем неправильный элементы и нажимаем правую кнопку мыши. Видим три уровня трассировки и сначала попробуем, что произойдёт, если использовать первый уровень.

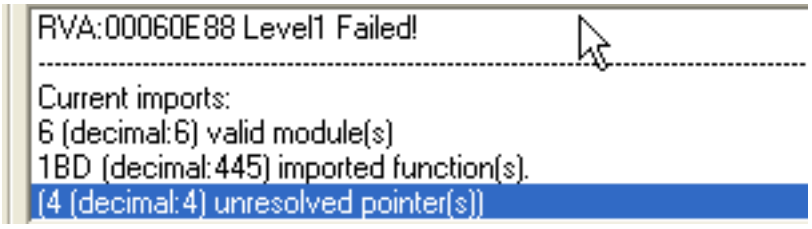


Неудача, пробуем с помощью остальных двух, после второго IMP REC виснет, поэтому несмотря на то, что эти методы существуют, в данном случае они не работают, а обычно приводят к зависанию. Срабатывают они только для самых простых упаковщиков.

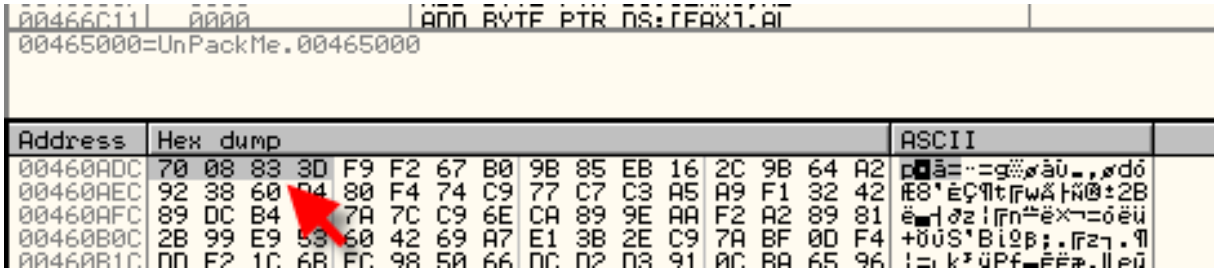
Следующий метод – это «JMP-CALL MAGICO», который стал популярен в туториалах CRACKSLATINOS о различных упаковщиках.

Этот метод основывается на попытке найти момент, когда упаковщик сохраняет значения в IAT, и сравнить, что происходит, когда сохраняется правильное значение, и что, когда неправильное.

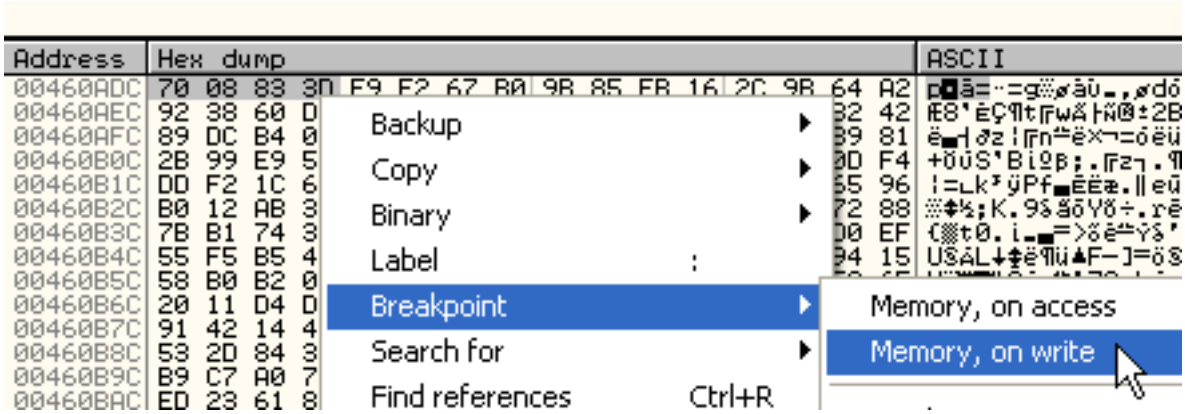
Используем в качестве примера элемент, которые мы исследовали ранее – GetVersion, являющийся переадресовочным или плохим. Перезапустим крэкми, упакованный telock'ом, и поищем указанный элемент в начале до запуска программы.



В DUMP'е смотрим адрес 460ADC, куда при запуске крэкми в некоторый момент до прибытия в OEP записывается плохое значение переадресовочного элемента, которое на моей машине равно 9F06F7, и это означает, что в этот момент упаковщик записывает данное значение в элемент.

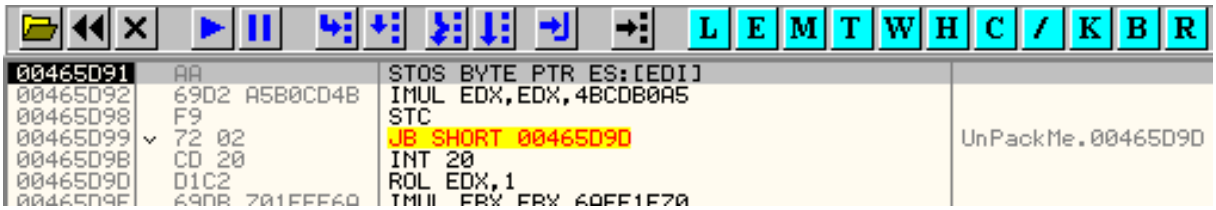


Для того, чтобы перехватить момент, в который происходит запись данного значения, обычно вначале на элемент устанавливается HARDWARE BPX ON WRITE, но так как этот упаковщик обнаруживает HARDWARE BPX, устанавливаем MEMORY BREAKPOINT ON WRITE



.

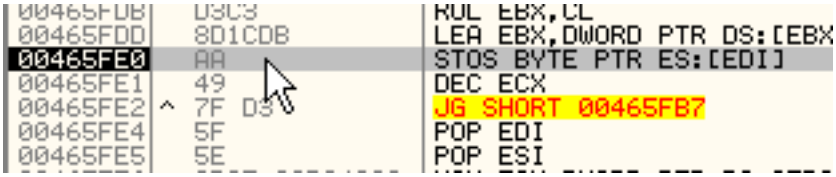
Нажав RUN, видим, что в первый раз MEMORY BREAKPOINT срабатывает здесь:



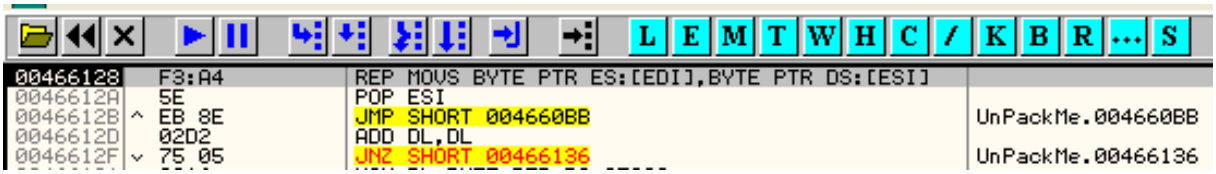
Видим, что это не то место, которое мы ищем, потому при выполнении строки с помощью F8 плохое значение не сохраняется в элементе, так что жмём RUN снова.

В следующий раз останавливаемся здесь:

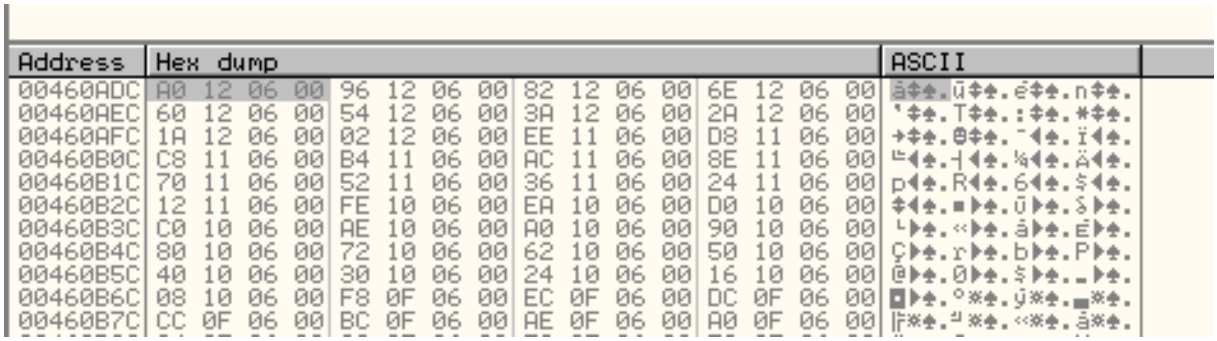
Видим, что здесь также не происходит сохранение плохого значения, опять жмём RUN.



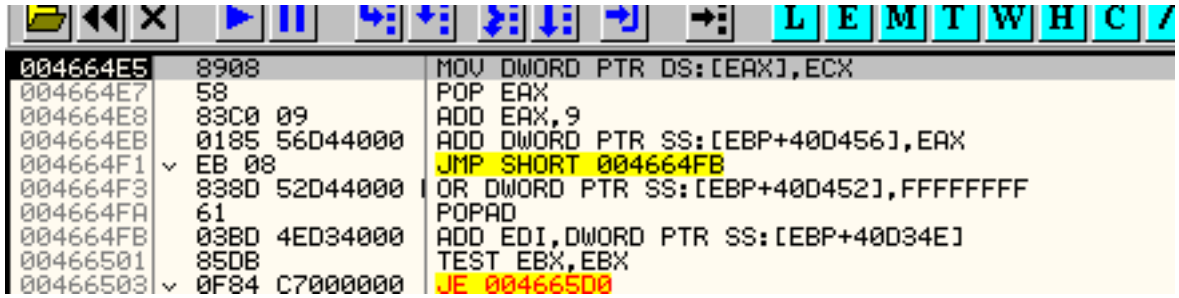
Следующее место также не содержит нужного нам.



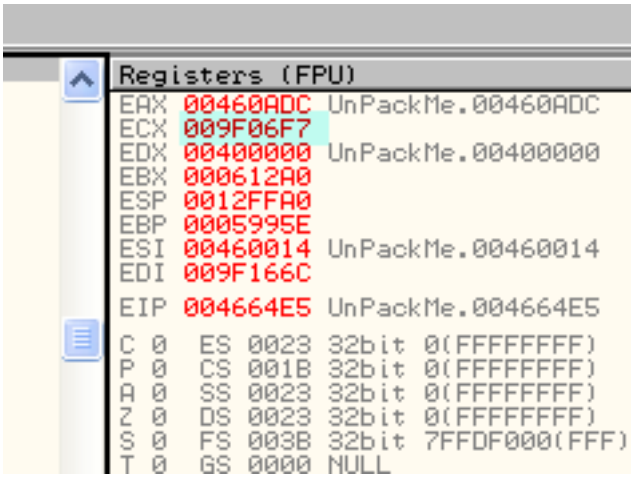
Продолжаем искать, где сохраняется плохое значение, и видим, что проходимые нами места его не содержат.



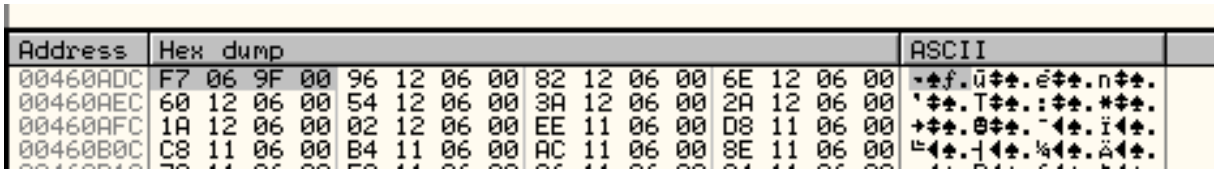
При проходе через «REP MOVS, IAT» IAT содержит эти значения, но плохого среди них нет, нажимаем RUN и продолжаем.



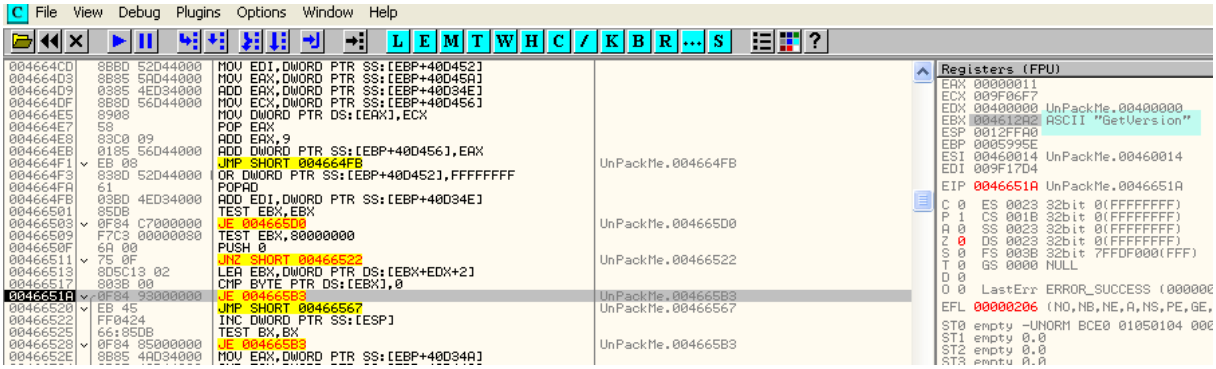
Когда прибываем сюда, видим, что ECX содержит плохое значение, которое сохраняется в элемент.



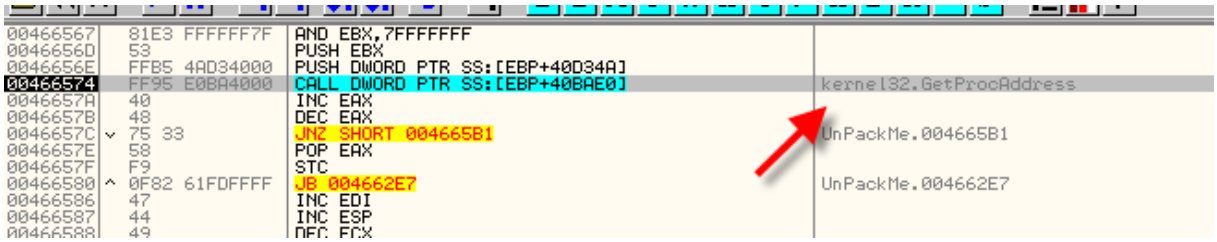
ECX равен 9F06F7, и это значение сохраняется по адресу, содержащемуся в EAX, то есть в 460ADC, в исследуемый нами элемент, и это и есть место, которое мы хотели найти.



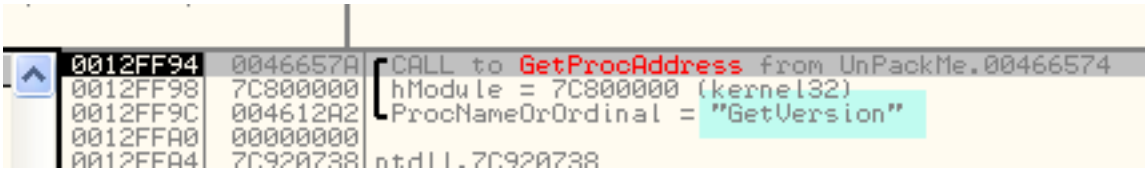
Видим, что при нажатии на F8 сохраняется плохое значение. Это первый шаг, который мы должны были сделать при использовании этого метода, а теперь нам предстоит куда более трудоёмка часть – найти волшебный переход.



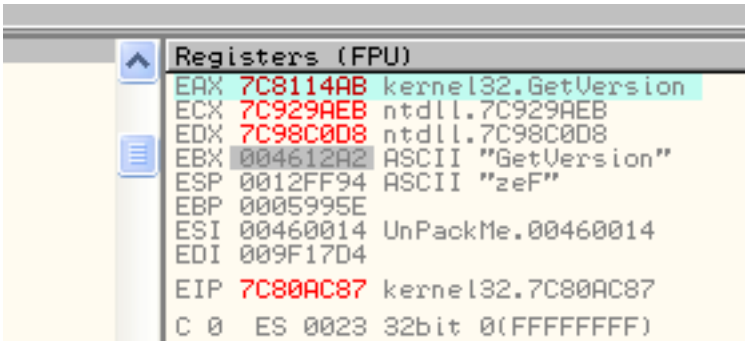
Как видим, дойдя до данного JE, чуть ниже в регистрах отображается имя API-функции (GetVersion), которая отвечает за данный элемент, загруженный плохим значением.



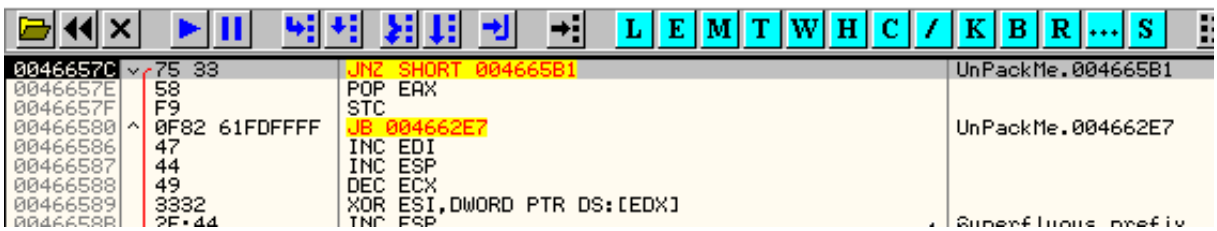
Далее видим, что дошли до вызова GetProcAddress для нахождения адреса GetVersion на моей машен, так как параметры следующие:



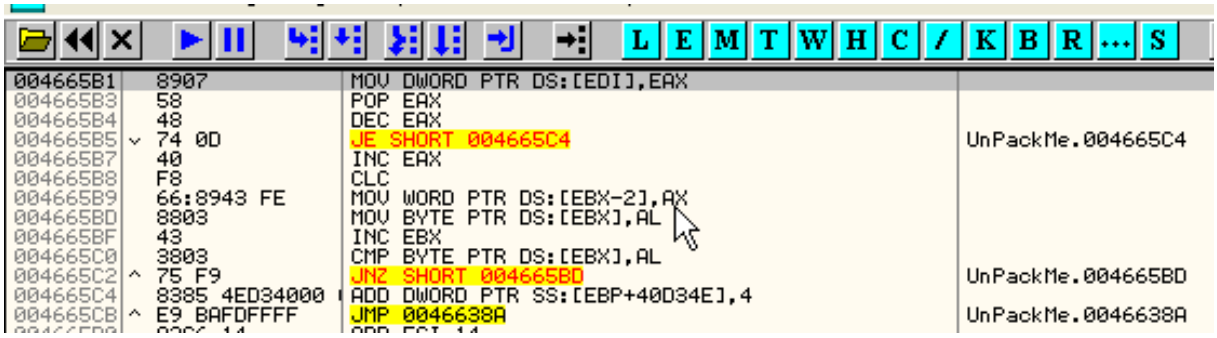
Заходим в API-функцию, чтобы посмотреть параметры, хотя это не обходимо. Проходим CALL с помощью F8.



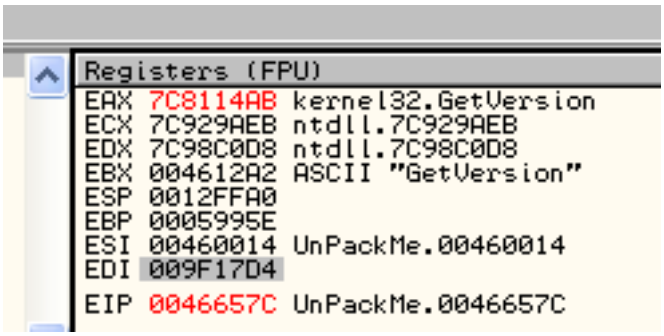
А в EAX нам возвращается адрес GetVestion на нашей машине. Продолжаем трассировку.



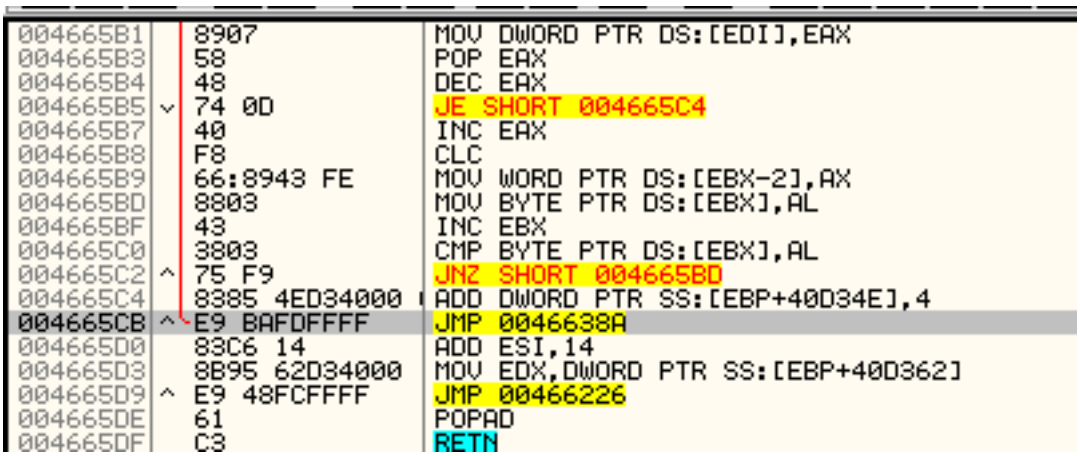
Мы у перехода. Посмотрим, куда он ведёт без трассировки. Нажимаем кнопку мыши и FOLLOW.



Видим, что здесь сохраняется адрес, но не в IAT, а в другое место, так как EDI равен:

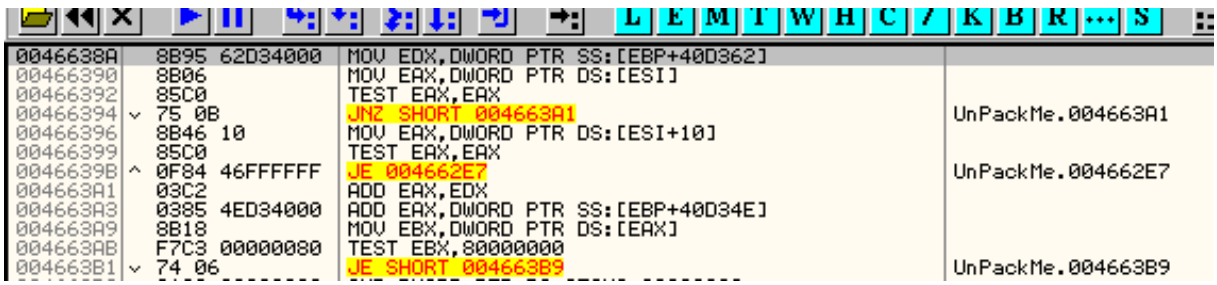


После сохранения данных значений идёт следующий JMP:



Который нас возвращает в начало процесса.

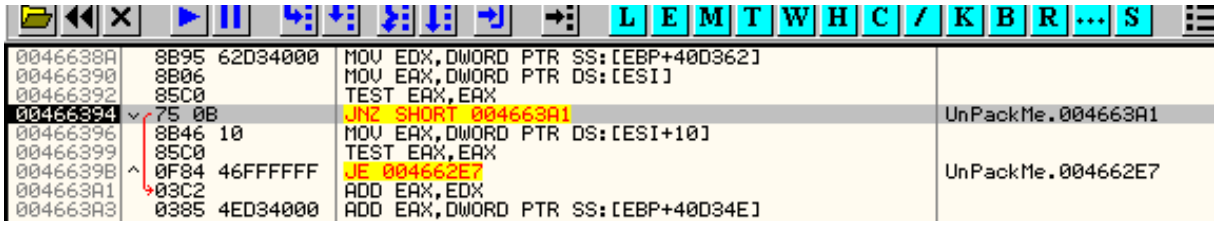
Смотрим с помощью FOLLOW, куда мы придём.



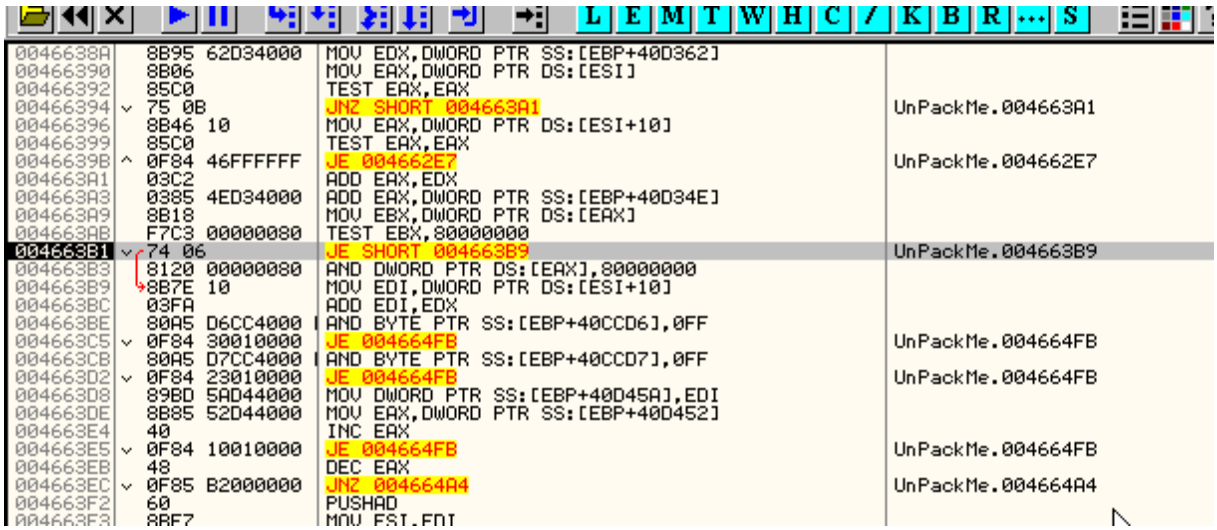
Здесь чётко видно, что EAX на второй строке потеряет правильный адрес API-функции на моей машине, и начнётся повторение цикла для следующего элемента, который должен быть неподалёку. Мы можем сконцентрироваться на тех плохих элементах, у которых есть условные переходы, чтобы сравнить их после трассировки с хорошими элементами.

Много раз я составлял себе табличку, которая сильно мне помогала, и где я отмечаль, что в данном чикле происходит с такими переходами у хороших элементов и что у плохих.

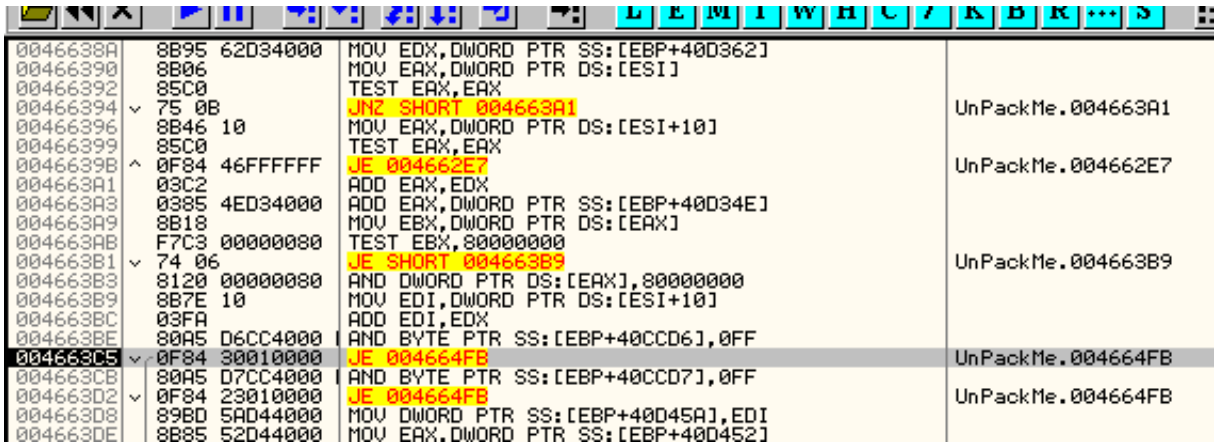
ДЛЯ ПЛОХИХ ЭЛЕМЕНТОВ (помещаю изображения всех переходов по мере их появления в трассировка вплоть до момента, когда происходит сохранение значения)



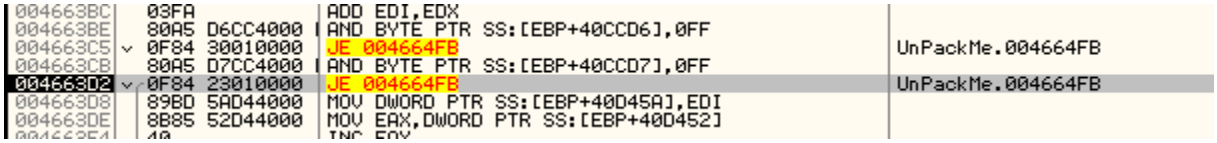
```
0046638A   8B95 62D34000   MOV EDX,DWORD PTR SS:[EBP+40D362]
00466390   8B06            MOV EAX,DWORD PTR DS:[ESI]
00466392   85C0            TEST EAX,EAX
00466394   75 0B           JNZ SHORT 004663A1                  UnPackMe.004663A1
00466396   8B46 10         MOV EAX,DWORD PTR DS:[ESI+10]
00466399   85C0            TEST EAX,EAX
0046639B   0F84 46FFFFFF   JE 004662E7                         UnPackMe.004662E7
004663A1   03C2            ADD EAX,EDX
004663A3   0385 4ED34000   ADD EAX,DWORD PTR SS:[EBP+40D34E]
```



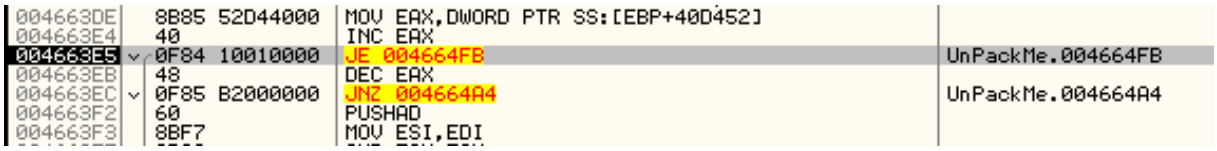
```
0046638A   8B95 62D34000   MOV EDX,DWORD PTR SS:[EBP+40D362]
00466390   8B06            MOV EAX,DWORD PTR DS:[ESI]
00466392   85C0            TEST EAX,EAX
00466394   75 0B           JNZ SHORT 004663A1                  UnPackMe.004663A1
00466396   8B46 10         MOV EAX,DWORD PTR DS:[ESI+10]
00466399   85C0            TEST EAX,EAX
0046639B   0F84 46FFFFFF   JE 004662E7                         UnPackMe.004662E7
004663A1   03C2            ADD EAX,EDX
004663A3   0385 4ED34000   ADD EAX,DWORD PTR SS:[EBP+40D34E]
004663A9   8B18            MOV EBX,DWORD PTR DS:[EAX]
004663AB   F7C3 00000080   TEST EBX,80000000
004663B1   74 06           JE SHORT 004663B9                   UnPackMe.004663B9
004663B3   8120 00000080   AND DWORD PTR DS:[EAX],80000000
004663B9   8B7E 10         MOV EDI,DWORD PTR DS:[ESI+10]
004663BC   03FA            ADD EDI,EDX
004663BE   80A5 D6CC4000   AND BYTE PTR SS:[EBP+40CCD6],0FF
004663C5   0F84 30010000   JE 004664FB                         UnPackMe.004664FB
004663CB   80A5 D7CC4000   AND BYTE PTR SS:[EBP+40CCD7],0FF
004663D2   0F84 23010000   JE 004664FB                         UnPackMe.004664FB
004663D8   89BD 5AD44000   MOV DWORD PTR SS:[EBP+40D45A],EDI
004663DE   8B85 52D44000   MOV EAX,DWORD PTR SS:[EBP+40D452]
004663E4   40              INC EAX
004663E5   0F84 10010000   JE 004664FB                         UnPackMe.004664FB
004663EB   48              DEC EAX
004663EC   0F85 B2000000   JNZ 004664A4                        UnPackMe.004664A4
004663F2   60              PUSHAD
004663F3   8BF7            MOV ESI,EDI
```



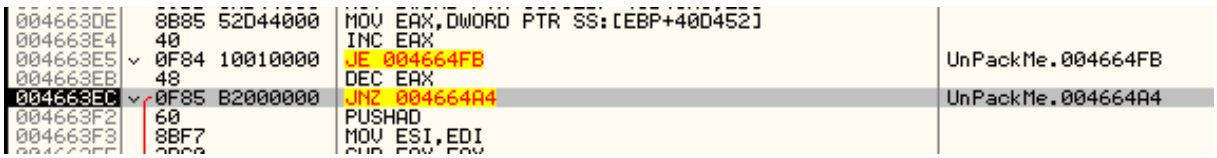
```
0046638A   8B95 62D34000   MOV EDX,DWORD PTR SS:[EBP+40D362]
00466390   8B06            MOV EAX,DWORD PTR DS:[ESI]
00466392   85C0            TEST EAX,EAX
00466394   75 0B           JNZ SHORT 004663A1                  UnPackMe.004663A1
00466396   8B46 10         MOV EAX,DWORD PTR DS:[ESI+10]
00466399   85C0            TEST EAX,EAX
0046639B   0F84 46FFFFFF   JE 004662E7                         UnPackMe.004662E7
004663A1   03C2            ADD EAX,EDX
004663A3   0385 4ED34000   ADD EAX,DWORD PTR SS:[EBP+40D34E]
004663A9   8B18            MOV EBX,DWORD PTR DS:[EAX]
004663AB   F7C3 00000080   TEST EBX,80000000
004663B1   74 06           JE SHORT 004663B9                   UnPackMe.004663B9
004663B3   8120 00000080   AND DWORD PTR DS:[EAX],80000000
004663B9   8B7E 10         MOV EDI,DWORD PTR DS:[ESI+10]
004663BC   03FA            ADD EDI,EDX
004663BE   80A5 D6CC4000   AND BYTE PTR SS:[EBP+40CCD6],0FF
004663C5   0F84 30010000   JE 004664FB                         UnPackMe.004664FB
004663CB   80A5 D7CC4000   AND BYTE PTR SS:[EBP+40CCD7],0FF
004663D2   0F84 23010000   JE 004664FB                         UnPackMe.004664FB
004663D8   89BD 5AD44000   MOV DWORD PTR SS:[EBP+40D45A],EDI
004663DE   8B85 52D44000   MOV EAX,DWORD PTR SS:[EBP+40D452]
```



```
004663BC   03FA            ADD EDI,EDX
004663BE   80A5 D6CC4000   AND BYTE PTR SS:[EBP+40CCD6],0FF
004663C5   0F84 30010000   JE 004664FB                         UnPackMe.004664FB
004663CB   80A5 D7CC4000   AND BYTE PTR SS:[EBP+40CCD7],0FF
004663D2   0F84 23010000   JE 004664FB                         UnPackMe.004664FB
004663D8   89BD 5AD44000   MOV DWORD PTR SS:[EBP+40D45A],EDI
004663DE   8B85 52D44000   MOV EAX,DWORD PTR SS:[EBP+40D452]
```



```
004663DE   8B85 52D44000   MOV EAX,DWORD PTR SS:[EBP+40D452]
004663E4   40              INC EAX
004663E5   0F84 10010000   JE 004664FB                         UnPackMe.004664FB
004663EB   48              DEC EAX
004663EC   0F85 B2000000   JNZ 004664A4                        UnPackMe.004664A4
004663F2   60              PUSHAD
004663F3   8BF7            MOV ESI,EDI
```



```
004663DE   8B85 52D44000   MOV EAX,DWORD PTR SS:[EBP+40D452]
004663E4   40              INC EAX
004663E5   0F84 10010000   JE 004664FB                         UnPackMe.004664FB
004663EB   48              DEC EAX
004663EC   0F85 B2000000   JNZ 004664A4                        UnPackMe.004664A4
004663F2   60              PUSHAD
004663F3   8BF7            MOV ESI,EDI
```

```
004664A4   8D85 14BB4000   LEA EAX,DWORD PTR SS:[EBP+40BB14]
004664AA   FFB5 FBCA4000   PUSH DWORD PTR SS:[EBP+40CAFB]
004664B0   0FB608          MOVZX ECX,BYTE PTR DS:[EAX]
004664B3   FF0C24          DEC DWORD PTR SS:[ESP]
004664B6   7E 05           JLE SHORT 004664BD                  UnPackMe.004664BD
004664B8   40              INC EAX
004664B9   03C1            ADD EAX,ECX
004664BB   EB F3           JMP SHORT 004664B0                  UnPackMe.004664B0
004664BD   890C24          MOV DWORD PTR SS:[ESP],ECX
004664C0   FF85 FBCA4000   INC DWORD PTR SS:[EBP+40CAFB]
```

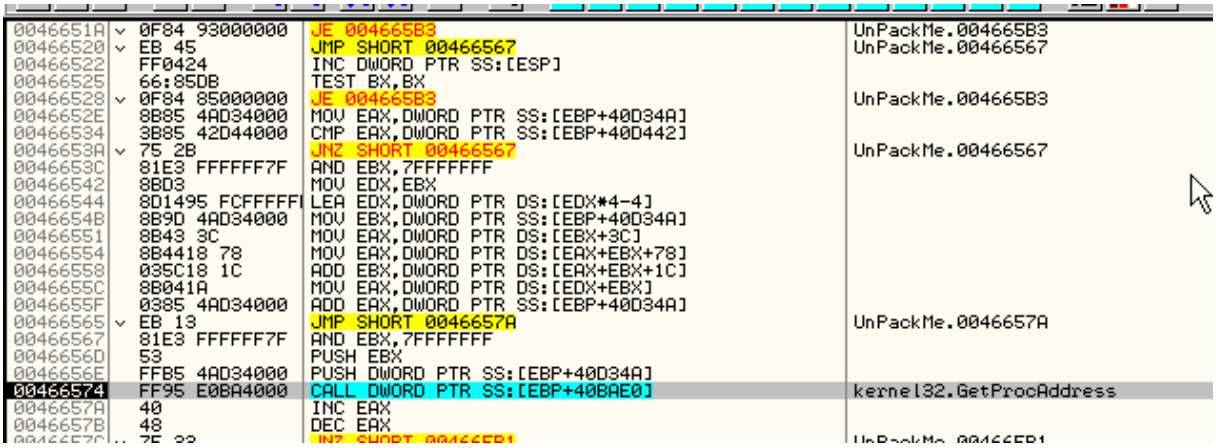
Это миницикл сразу после выхода из него:

```
004664FB   03BD 4ED34000   ADD EDI,DWORD PTR SS:[EBP+40D34E]
00466501   85DB            TEST EBX,EBX
00466503   0F84 C7000000   JE 004665D0                         UnPackMe.004665D0
00466509   F7C3 00000080   TEST EBX,80000000
0046650F   6A 00           PUSH 0
00466511   75 0F           JNZ SHORT 00466522                  UnPackMe.00466522
00466513   8D5C13 02       LEA EBX,DWORD PTR DS:[EBX+EDX+2]
00466517   803B 00         CMP BYTE PTR DS:[EBX],0
0046651A   0F84 93000000   JE 004665B3                         UnPackMe.004665B3
00466520   EB 45           JMP SHORT 00466567                  UnPackMe.00466567
00466522   FF0424          INC DWORD PTR SS:[ESP]
00466525   66:85DB         TEST BX,BX
00466528   0F84 85000000   JE 004665B3                         UnPackMe.004665B3
0046652E   8B85 4AD34000   MOV EAX,DWORD PTR SS:[EBP+40D34A]
00466534   3B85 42D44000   CMP EAX,DWORD PTR SS:[EBP+40D442]
0046653A   75 2B           JNZ SHORT 00466567                  UnPackMe.00466567
0046653C   81E3 FFFFFF7F   AND EBX,7FFFFFFF
00466542   8BD3            MOV EDX,EBX
00466544   8D1495 FCFFFFFF LEA EDX,DWORD PTR DS:[EDX*4-4]
0046654B   8B9D 4AD34000   MOV EBX,DWORD PTR SS:[EBP+40D34A]
00466551   8B43 3C         MOV EAX,DWORD PTR DS:[EBX+3C]
00466554   8B4418 78       MOV EAX,DWORD PTR DS:[EAX+EBX+78]
00466558   035C18 1C       ADD EBX,DWORD PTR DS:[EAX+EBX+1C]
0046655C   8B041A          MOV EAX,DWORD PTR DS:[EDX+EBX]
0046655F   0385 4AD34000   ADD EAX,DWORD PTR SS:[EBP+40D34A]
00466565   EB 13           JMP SHORT 0046657A                  UnPackMe.0046657A
00466567   81E3 FFFFFF7F   AND EBX,7FFFFFFF
0046656D   53              PUSH EBX
0046656E   FFB5 4AD34000   PUSH DWORD PTR SS:[EBP+40D34A]
00466574   FF95 E0BA4000   CALL DWORD PTR SS:[EBP+40BAE0]
00466579   40              INC EAX
```

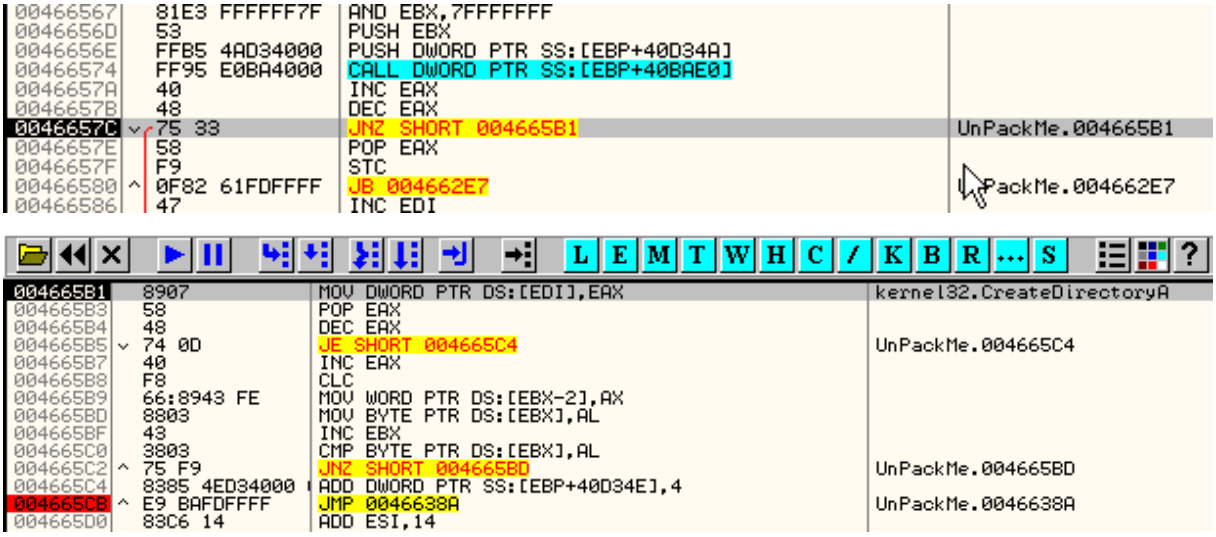
```
004664FB   03BD 4ED34000   ADD EDI,DWORD PTR SS:[EBP+40D34E]
00466501   85DB            TEST EBX,EBX
00466503   0F84 C7000000   JE 004665D0                         UnPackMe.004665D0
00466509   F7C3 00000080   TEST EBX,80000000
0046650F   6A 00           PUSH 0
00466511   75 0F           JNZ SHORT 00466522                  UnPackMe.00466522
00466513   8D5C13 02       LEA EBX,DWORD PTR DS:[EBX+EDX+2]
00466517   803B 00         CMP BYTE PTR DS:[EBX],0
0046651A   0F84 93000000   JE 004665B3                         UnPackMe.004665B3
00466520   EB 45           JMP SHORT 00466567                  UnPackMe.00466567
00466522   FF0424          INC DWORD PTR SS:[ESP]
00466525   66:85DB         TEST BX,BX
```

```
File  View  Debug  Plugins  Options  Window  Help
```

```
004664FB   03BD 4ED34000   ADD EDI,DWORD PTR SS:[EBP+40D34E]
00466501   85DB            TEST EBX,EBX
00466503   0F84 C7000000   JE 004665D0                         UnPackMe.004665D0
00466509   F7C3 00000080   TEST EBX,80000000
0046650F   6A 00           PUSH 0
00466511   75 0F           JNZ SHORT 00466522                  UnPackMe.00466522
00466513   8D5C13 02       LEA EBX,DWORD PTR DS:[EBX+EDX+2]
00466517   803B 00         CMP BYTE PTR DS:[EBX],0
0046651A   0F84 93000000   JE 004665B3                         UnPackMe.004665B3
00466520   EB 45           JMP SHORT 00466567                  UnPackMe.00466567
00466522   FF0424          INC DWORD PTR SS:[ESP]
00466525   66:85DB         TEST BX,BX
00466528   0F84 85000000   JE 004665B3                         UnPackMe.004665B3
0046652E   8B85 4AD34000   MOV EAX,DWORD PTR SS:[EBP+40D34A]
00466534   3B85 42D44000   CMP EAX,DWORD PTR SS:[EBP+40D442]
0046653A   75 2B           JNZ SHORT 00466567                  UnPackMe.00466567
0046653C   81E3 FFFFFF7F   AND EBX,7FFFFFFF
```

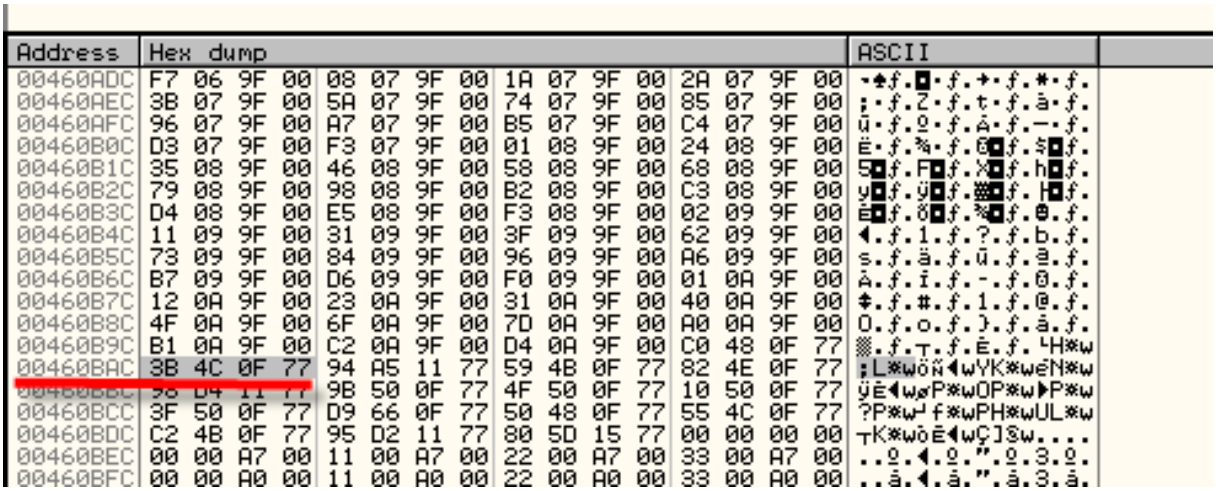


Здесь доходим до GetProcAddress, а затем делаем переход туда, где он сохраняется.



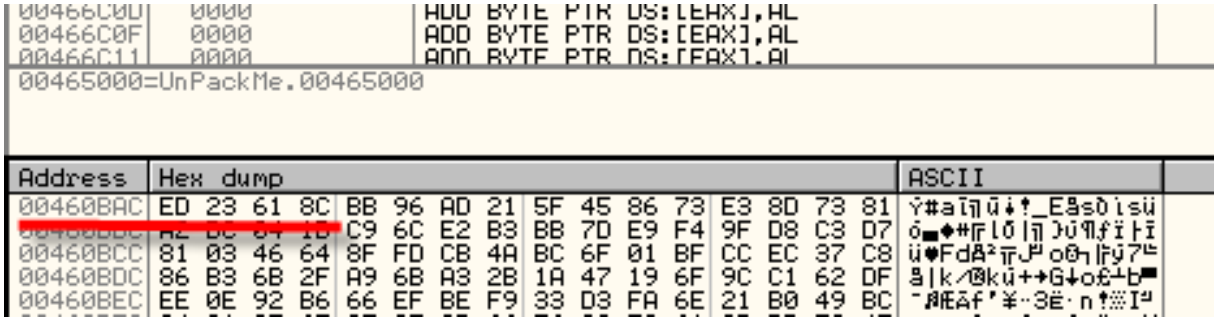
Здесь он сохраняется, и это полная трассировка для плохих элементов с рассмотрением того, как работают условные переходы в данном случае. Теперь идём в OEP и ищем хороший элемент для того, чтобы повторить процесс, сравнить и найти отличия.

Идём в OEP.

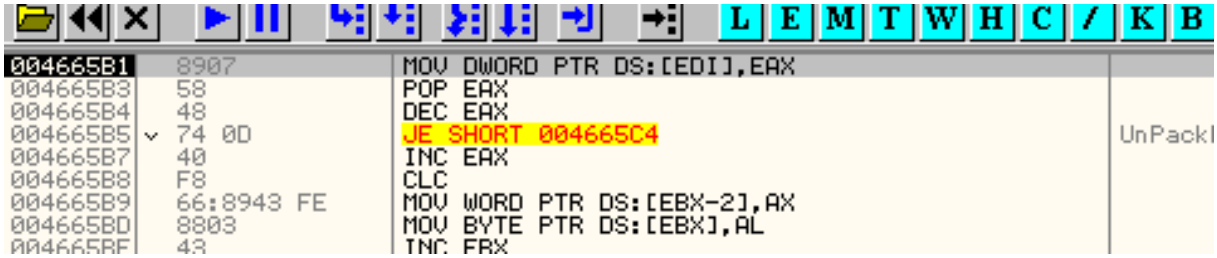


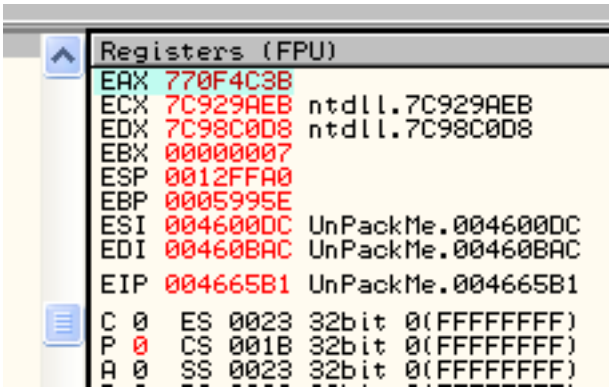
В качестве хорошего элемента выбираем 460BAC, теперь повторяем тот же процесс, что и с плохим.

Рестартуем и ищем элемент до запуска самой программы.

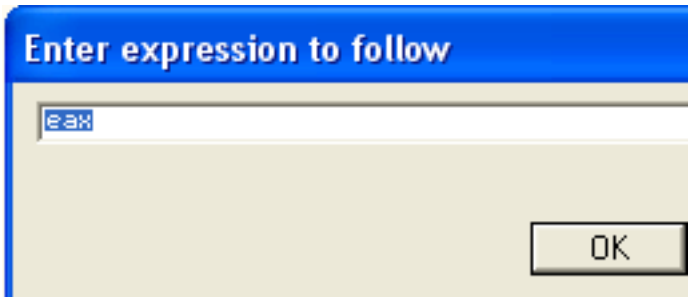


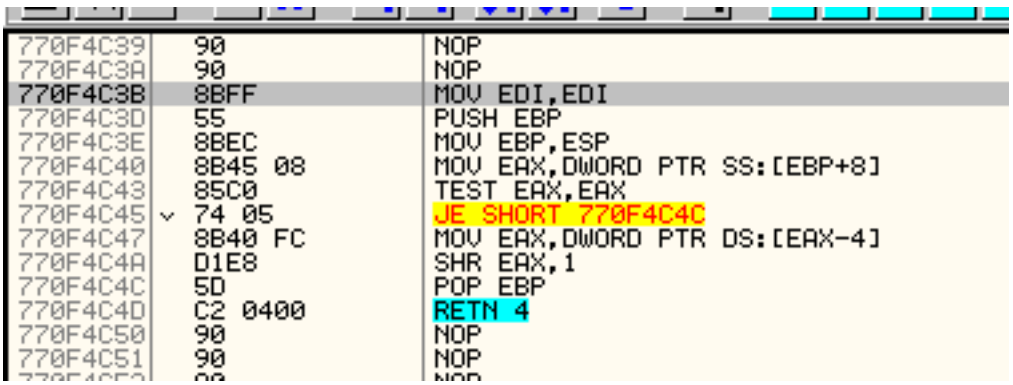
Теперь устанавливаем MEMORY BREAKPOINT ON WRITE, чтобы попытаться перехватить момент, в который упаковщик сохряет хороший элемент.



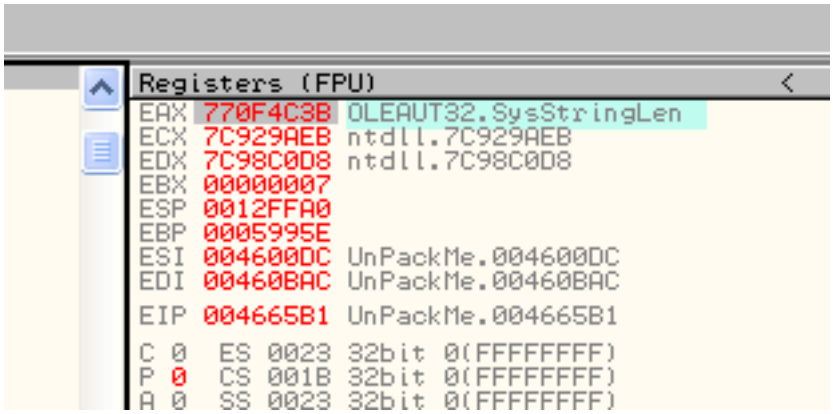


Видим, что EAX содержит значение возможной API-функции, хотя её имя нам сообщается, однако если посмотрим с помощью GOTO EXPRESSION-EAX.





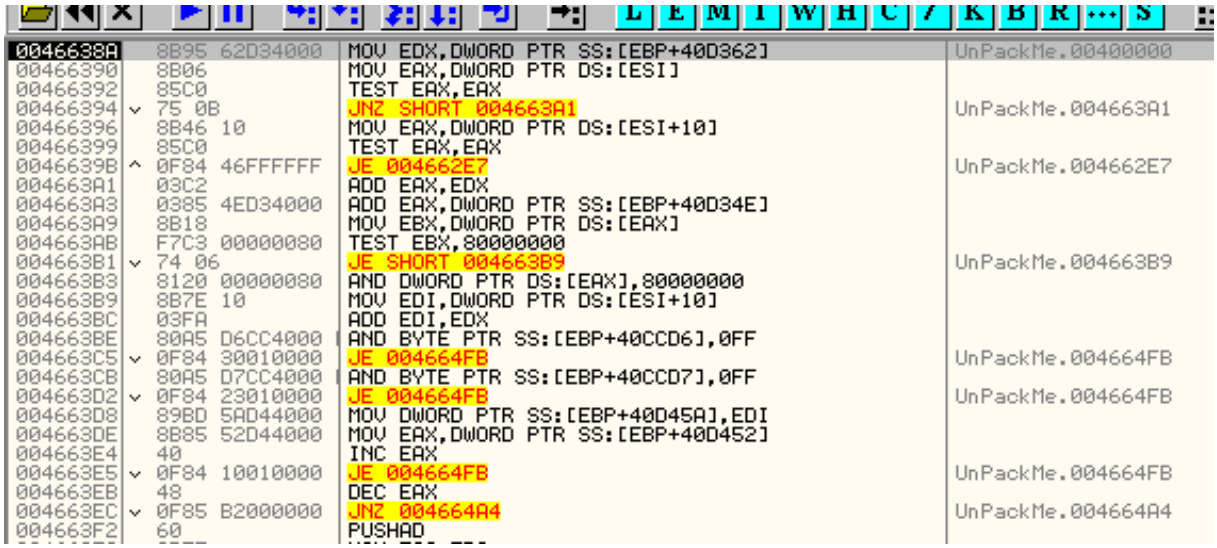
Видим, что мы прибыли сюда, и по возвращению к волшебному коду появляется имя функции, хе-хе.

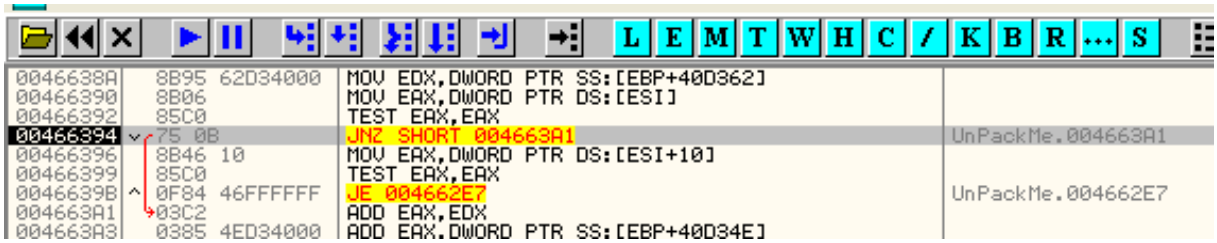


Теперь сделаем ту же процедуру, что проделали с плохим элементом, ожидаем, что всё будет идти в том же порядке.

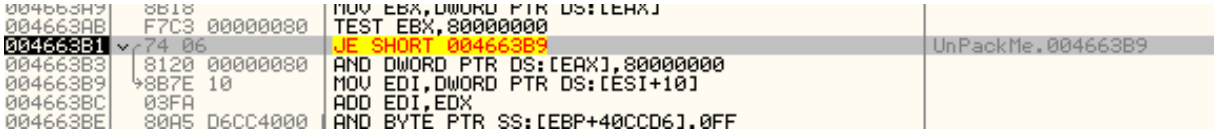
ДЛЯ ХОРОШИХ ЭЛЕМЕНТОВ

Доходим до начала цикла и начинаем трассировку.





Этот переход такой же как и в плохом элементе.



В плохих элементах перехода не происходило, но переход очень маленький, не думаю, что он решающий.

```
004663BC   03FA            ADD EDI,EDX
004663BE   80A5 D6CC4000   AND BYTE PTR SS:[EBP+40CCD6],0FF
004663C5   0F84 30010000   JE 004664FB
004663CB   80A5 D7CC4000   AND BYTE PTR SS:[EBP+40CCD7],0FF
004663D2   0F84 23010000   JE 004664FB
004663D8   89BD 5AD44000   MOV DWORD PTR SS:[EBP+40D45A],EDI
004663DE   8B85 52D44000   MOV EAX,DWORD PTR SS:[EBP+40D452]
```

Этот переход не равен тому, что в плохом элементе.

```
004663BC   03FA            ADD EDI,EDX
004663BE   80A5 D6CC4000   AND BYTE PTR SS:[EBP+40CCD6],0FF
004663C5   0F84 30010000   JE 004664FB                      UnPackMe.004664FB
004663CB   80A5 D7CC4000   AND BYTE PTR SS:[EBP+40CCD7],0FF
004663D2   0F84 23010000   JE 004664FB                      UnPackMe.004664FB
004663D8   89BD 5AD44000   MOV DWORD PTR SS:[EBP+40D45A],EDI
004663DE   8B85 52D44000   MOV EAX,DWORD PTR SS:[EBP+40D452]
004663E4   40              INC EAX
004663E5   0F84 10010000   JE 004664FB                      UnPackMe.004664FB
004663EB   48              DEC EAX
004663EC   0F85 B2000000   JNZ 004664A4                     UnPackMe.004664A4
004663F2   60              PUSHAD
```

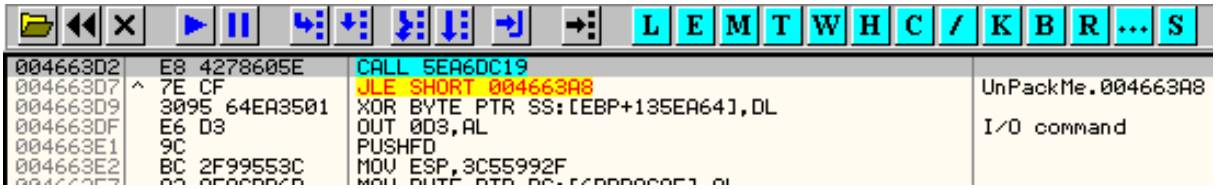
Здесь видим значительное отличие: этот переход в плохих элементах не срабатывает, а здесь – да, и видим, что перепрыгиваемый код велик по объёму.

```
004663BE   80A5 D6CC4000   AND BYTE PTR SS:[EBP+40CCD6],0FF
004663C5   0F84 30010000   JE 004664FB
004663CB   80A5 D7CC4000   AND BYTE PTR SS:[EBP+40CCD7],0FF
004663D2   0F84 23010000   JE 004664FB
004663D8   89BD 5AD44000   MOV DWORD PTR SS:[EBP+40D45A],EDI
004663DE   8B85 52D44000   MOV EAX,DWORD PTR SS:[EBP+40D452]
004663E4   40              INC EAX
004663E5   0F84 10010000   JE 004664FB
004663EB   48              DEC EAX
004663EC   0F85 B2000000   JNZ 004664A4
004663F2   60              PUSHAD
004663F3   8BF7            MOV ESI,EDI
004663F5   2BC0            SUB EAX,EAX
004663F7   40              INC EAX
004663F8   833F 00         CMP DWORD PTR DS:[EDI],0
004663FB   8D7F 04         LEA EDI,DWORD PTR DS:[EDI+4]
004663FE   75 F7           JNZ SHORT 004663F7
00466400   48              DEC EAX
00466401   0F84 EC000000   JE 004664F3
00466407   8BD8            MOV EBX,EAX
00466409   6BC0 31         IMUL EAX,EAX,31
0046640C   6A 04           PUSH 4
0046640E   68 00100000     PUSH 1000
00466413   50              PUSH EAX
00466414   6A 00           PUSH 0
00466416   FF95 ECBA4000   CALL DWORD PTR SS:[EBP+40BAEC]
0046641C   85C0            TEST EAX,EAX
0046641E   0F84 CF000000   JE 004664F3
00466424   8BFE            MOV EDI,ESI
00466426   8BCB            MOV ECX,EBX
00466428   8BF8            MOV EDI,EAX
0046642A   8985 56D44000   MOV DWORD PTR SS:[EBP+40D456],EAX
00466430   8BCB            MOV ECX,EBX
00466432   6BDB 29         IMUL EBX,EBX,29
00466435   03DF            ADD EBX,EDI
00466437   891C24          MOV DWORD PTR SS:[ESP],EBX
0046643A   B0 B8           MOV AL,0B8
0046643C   6A 00           PUSH 0
0046643E   50              PUSH EAX
0046643F   53              PUSH EBX
00466440   0FB74424 08     MOVZX EAX,WORD PTR SS:[ESP+8]
00466445   50              PUSH EAX
00466446   8D85 14BB4000   LEA EAX,DWORD PTR SS:[EBP+40BB14]
0046644C   0FB618          MOVZX EBX,BYTE PTR DS:[EAX]
0046644F   FF0C24          DEC DWORD PTR SS:[ESP]
00466452   7E 09           JLE SHORT 0046645D
00466454   40              INC EAX
Jump is taken
```

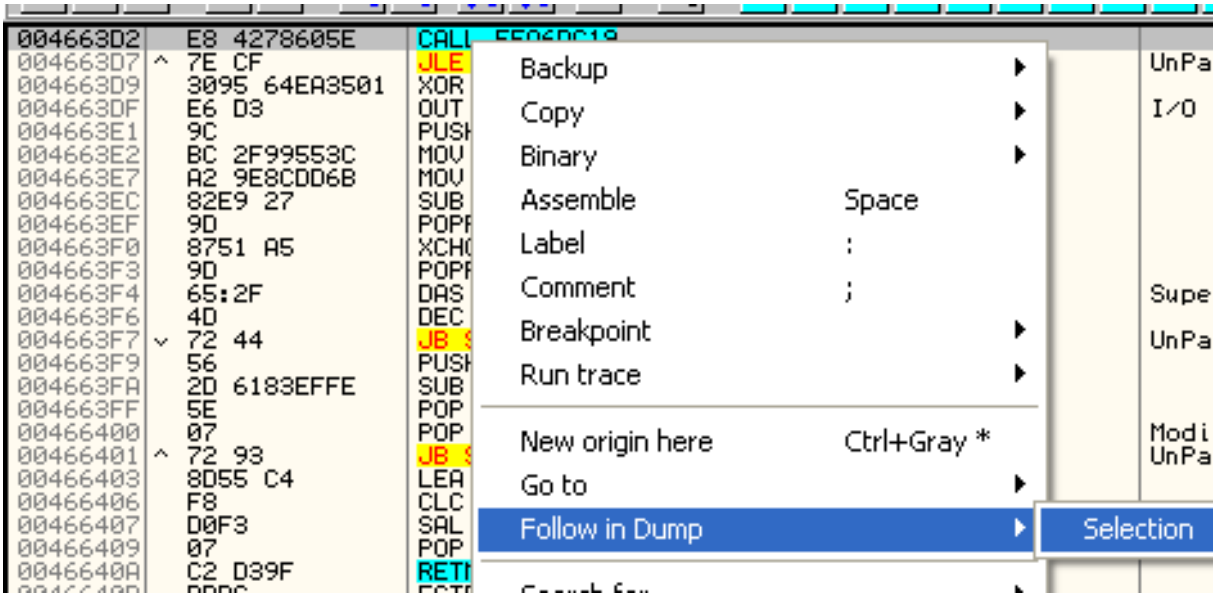
Здесь видим, что переход достаточно велик, что может приводить к значительной разнице в обработке элемента в зависимости от того, является ли он хорошим или плохим, то есть если всё

идёт, как мы считаем, то это может быть волшебный переход, который решает, будет ли элемент сохранён как хороший или плохой. Это значит, что для сохранения элемент как правильный, переход должен сработать, так что в данном случае есть значительная вероятность, что будет сделан прыжок.

Но этот переход не встречается в начале программы, поэтому мы должны быть очень внимательны, если рестартуем OllyDbg и будем искать этот переход.



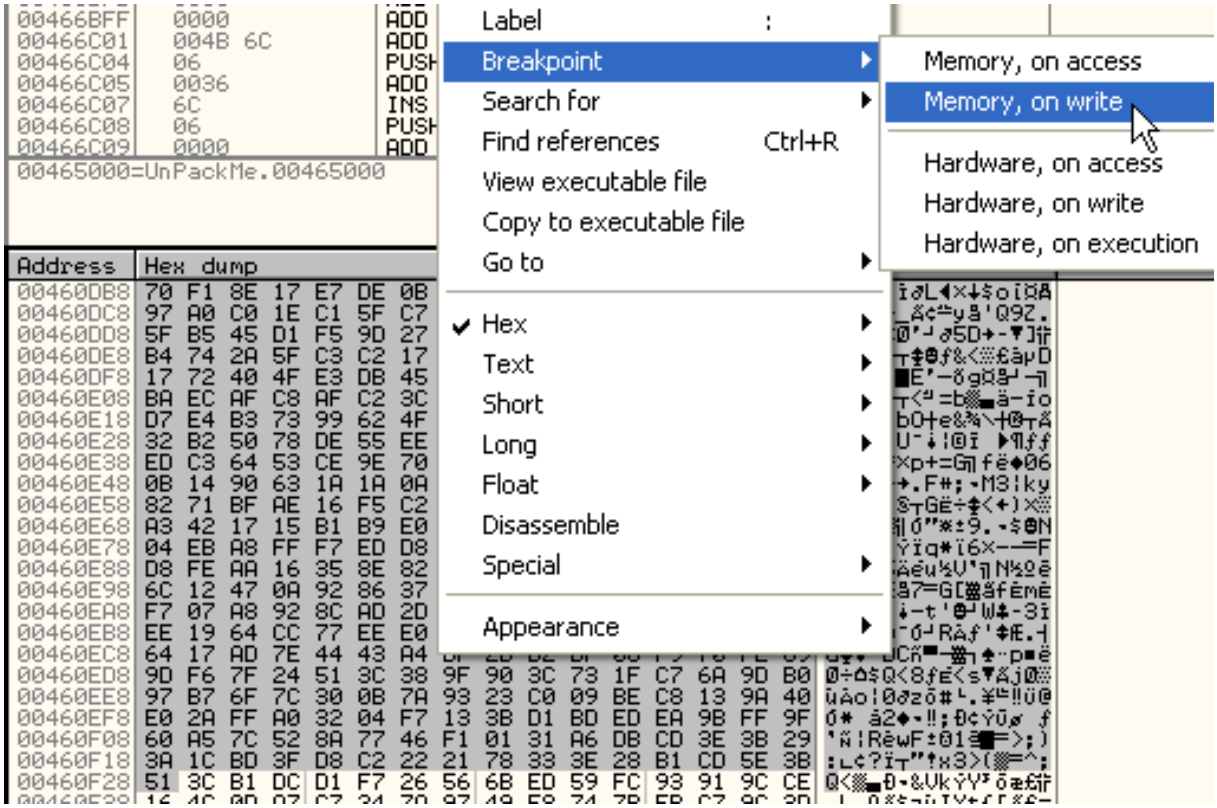
Видим, что в начале идёт чистый мусор, так что упаковщик создаёт этот переход гораздо позже.



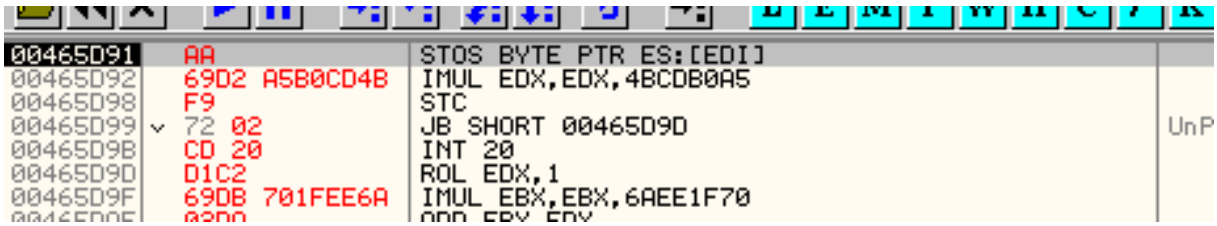
Дело в том, что так как останавливаемся здесь, HBP использовать нельзя, а BP будет обнаружен, так что нам нужно напрячь воображение.

Устанавливаем BPM ON WRITE, который охватывает весь IAT, чтобы остановиться, когда туда будет записано первое значение.

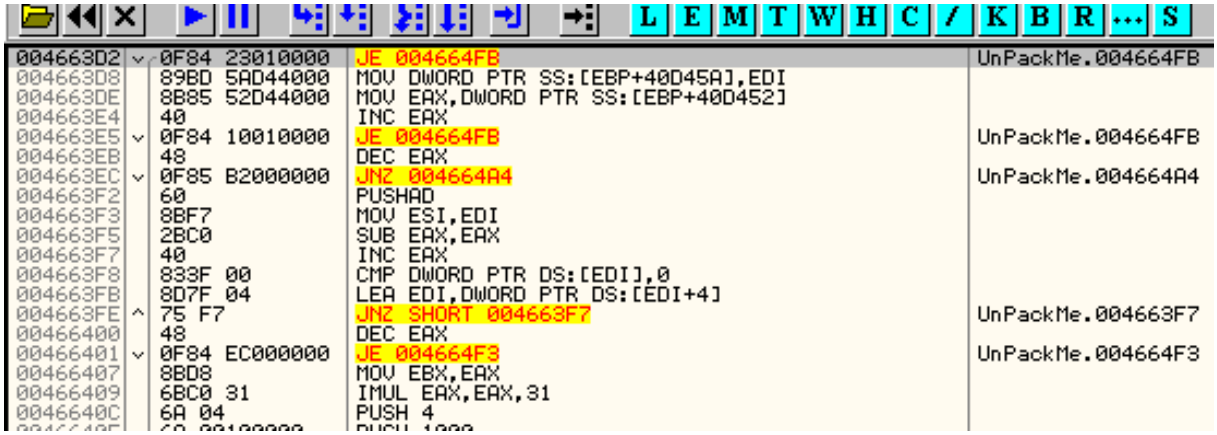
IAT начинается в 460818, а заканчивается в 460F28. Охватываем всю эту область с помощью BPM ON WRITE.



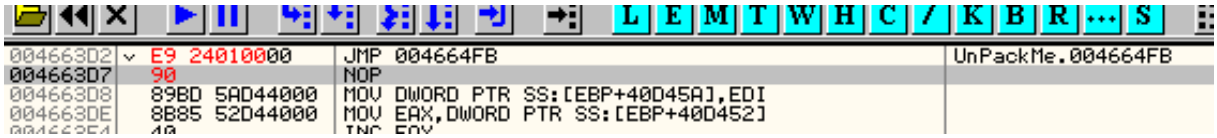
Весь IAT охвачен BPM ON WRITE, теперь продолжаем, пока не будет сохранено первое значение.



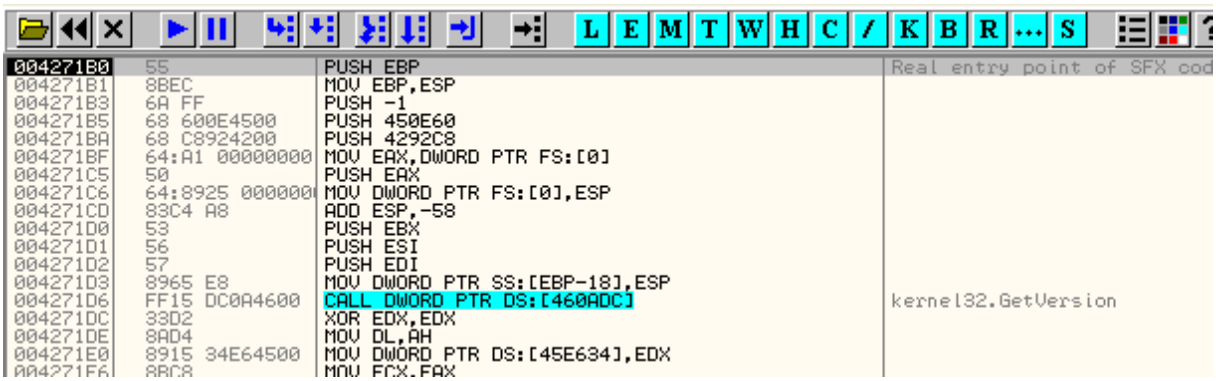
На STOS'ах останавливаемся множество раз, так как BPM охватывает весь IAT, так что сейчас мы узнаем, виден ли уже в этот момент возможный волшебный переход.



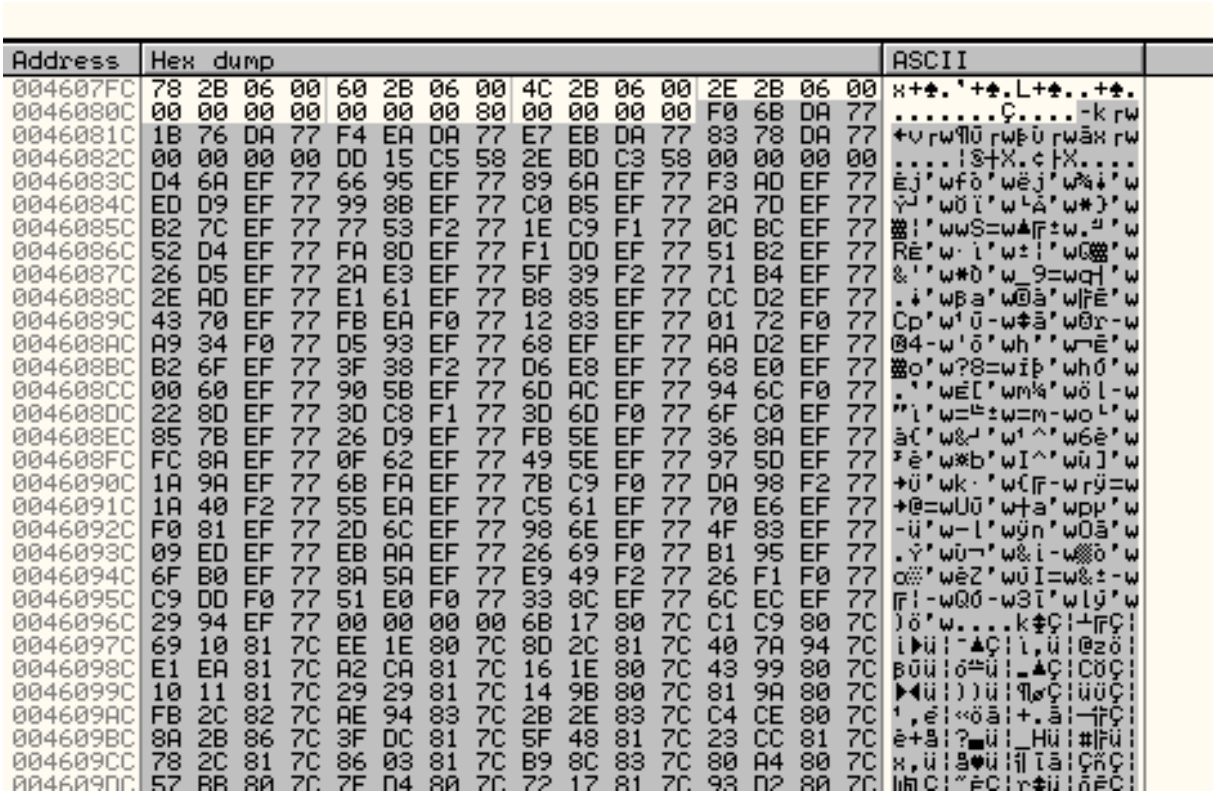
Так, он есть, мы проходим через него, и если заменить на JMP, снимем BPM и пойдём в OEP:



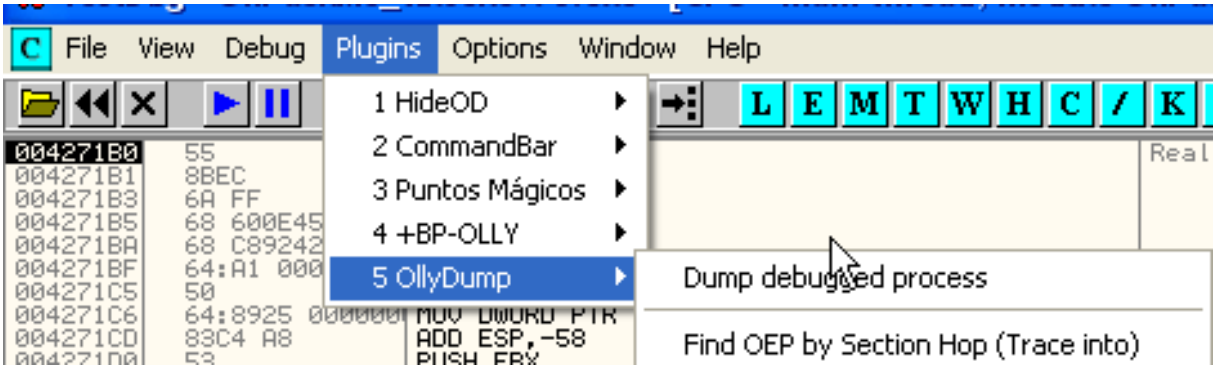
Есть две возможности: упаковщик обнаружит изменения и прекратит выполнение, или всё пройдёт нормально и мы окажемся в OEP, так что делаем RUN.

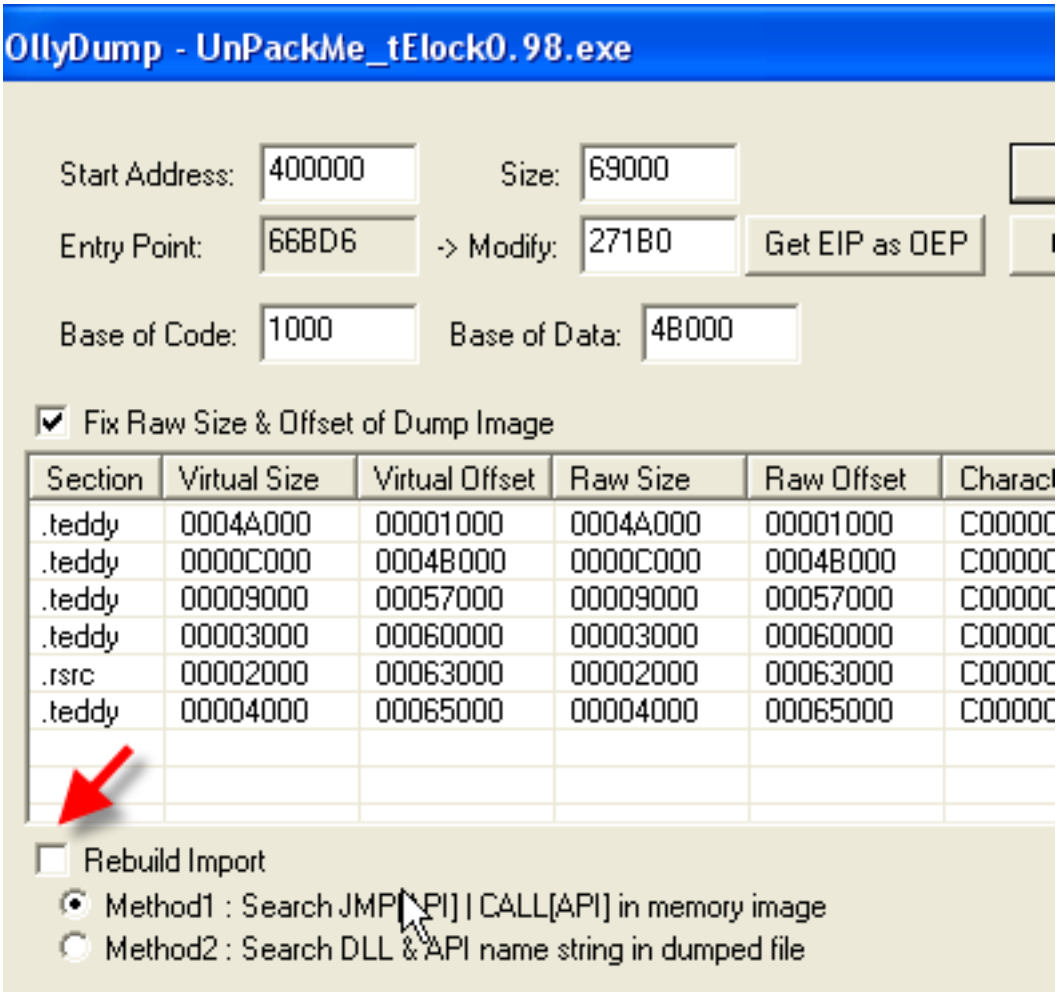


Прибываем в OEP, смотрим IAT:



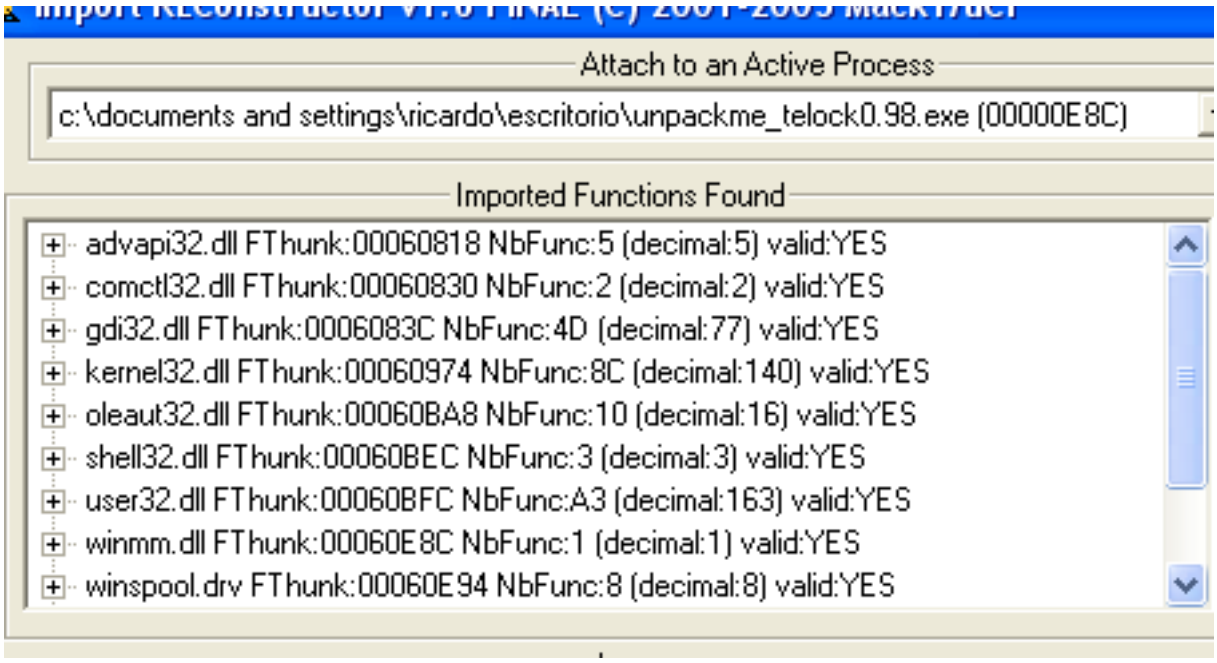
Все значения верны, хе-хе, наша IAT починена, теперь делаем дамп.





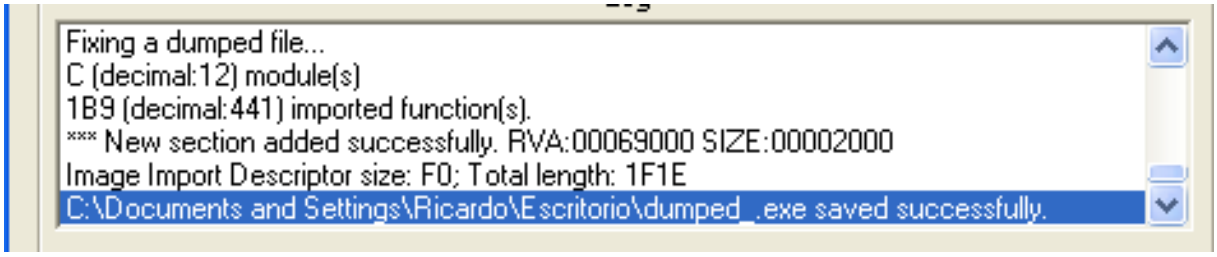
Снимаем галочку с REBUILD IMPORT.

И дампим. Потом открываем IMP REc и снова устанавливаем значения RVA, SIZE и OEP.



Видим, что сейчас всё в порядке, волшебный переход сделал своё волшебство, хе-хе.

Теперь чиним дамп, нажимая FIX DUMP.



Сохраняем его как dumped_.exe и запускаем, чтобы убедиться, что он работает.

Работает прекрасно, хе-хе.

Отлично, мы рассмотрели метод для нахождения волшебного перехода, который всегда различается у разных упаковщиков, но сравнив, что происходит при заполнении хороших и плохих элементов, всегда можем легко найти решение. В главе 38 мы продолжим рассмотрение упаковщиков.

Введение в крэкинг с нуля, используя OllyDbg - Глава 38

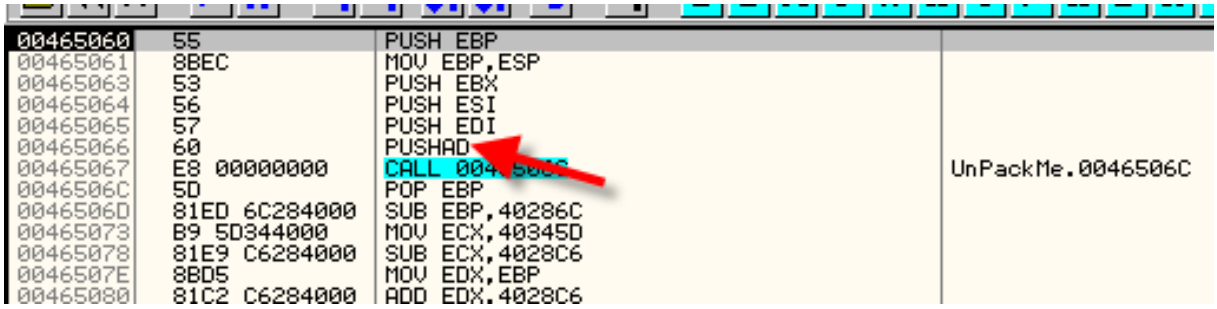
Автор: Рикардо Нарваха, пер. Aquila

Дата: 2009-01-11

В предыдущей главе мы рассмотрели одну из программ с переадресовочными элементами и как её починить, а теперь продолжаем, плавно повышая уровень сложность упаковщиков, сопровождая это практическими упражнениями.

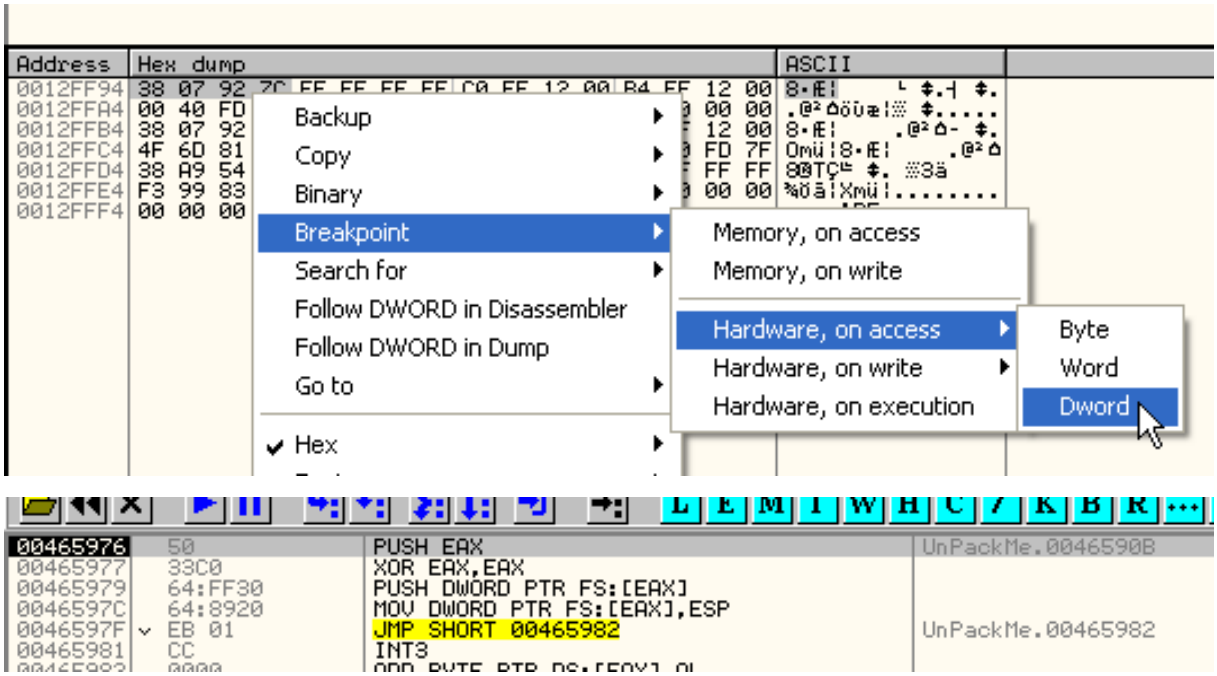
У нас есть распакуй-меня, упакованная Yoda Crypter 1.3, на котором мы будем практиковаться, разумеется, OllyDbg должен быть с установленными плагинами, необходимыми для борьбы с обнаружением и антиотладкой.

Находимся в точке входа.

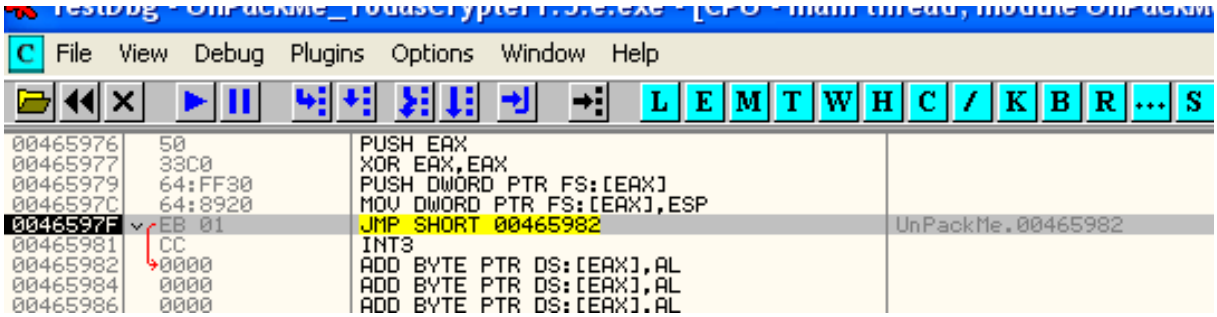


Сработает ли PUSHAD? Попробуем этот метод. С помощью трассировки идём досюда и проходим PUSHAD с помощью F7.

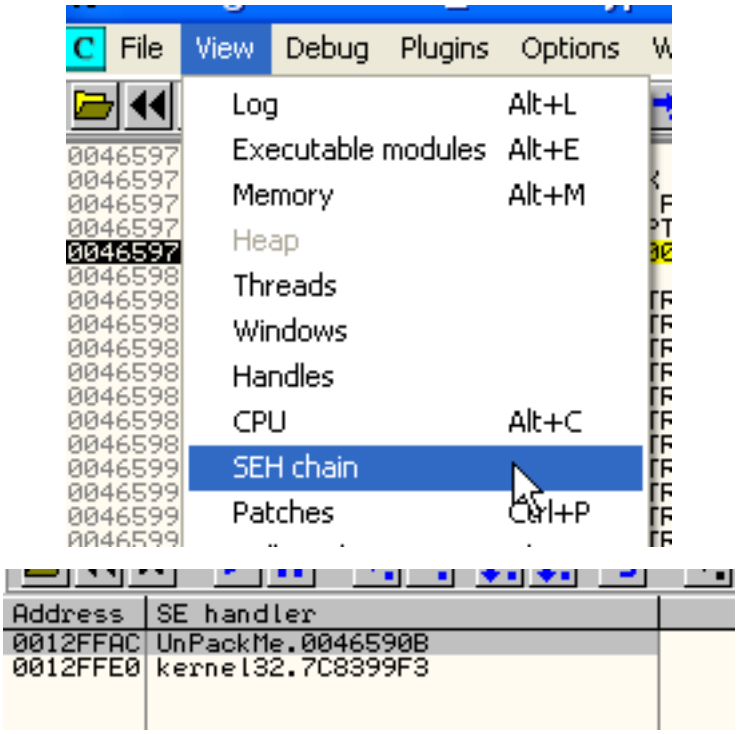
Делаем ESP-FOLLOW IN DUMP, а затем отмечаем первые 4 байта как в прошлые разы и устанавливаем HARDWARE BPX ON ACCESS.



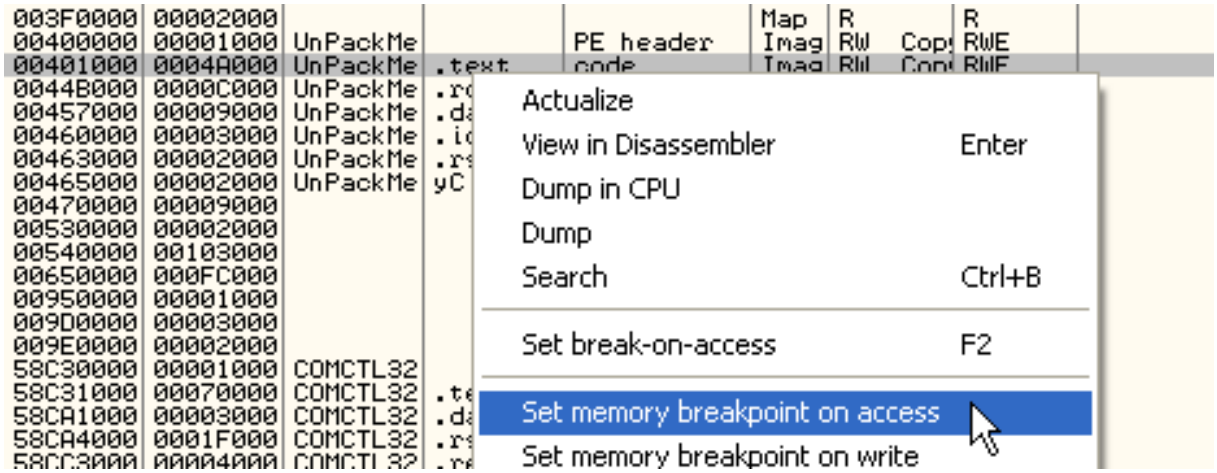
Видим процедуру, создающая обработчик исключений, а затем как совершается переход с помощью JMP в область, заполненную нулями, чтобы вызвать ошибку, так что если оттрассируем и дойдём до JMP:

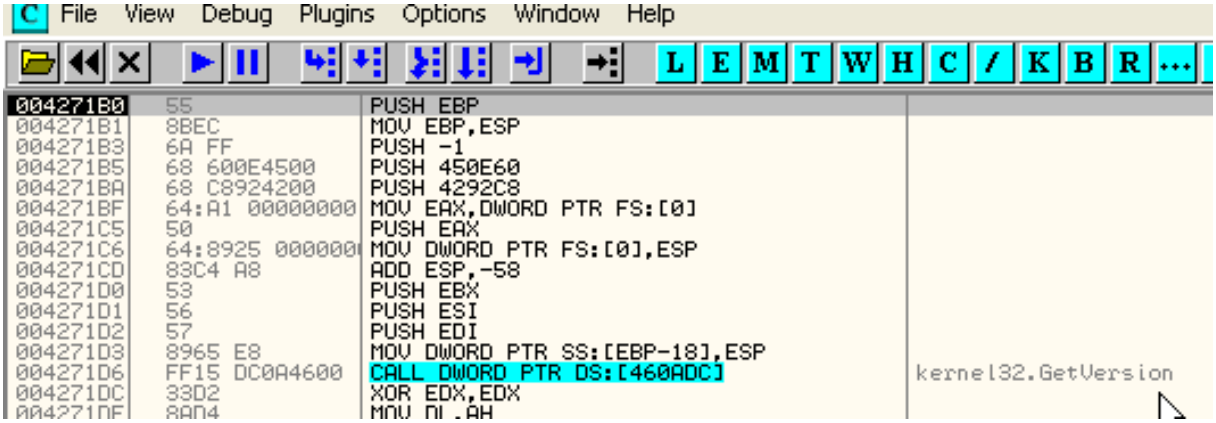


Здесь мы перейдём на ошибку, посмотрим, куда перейдём в обработчике исключений.

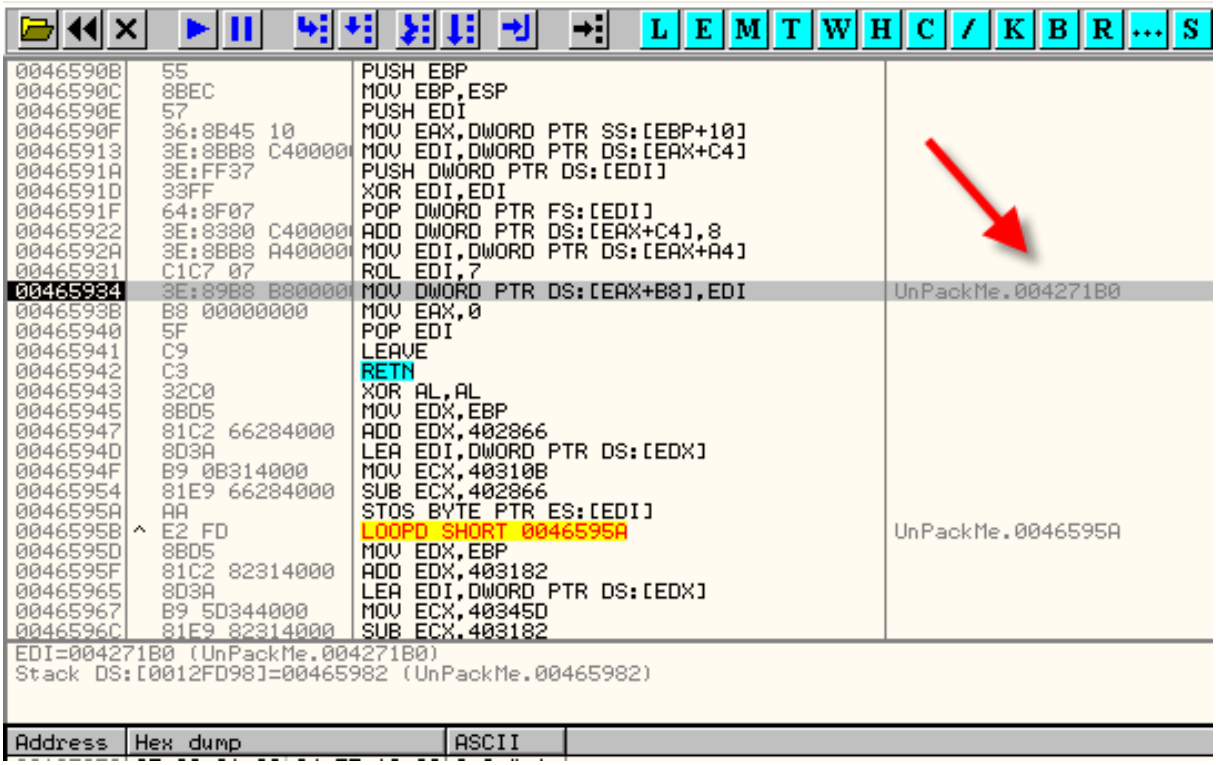


Тот, что в 46590B не сильно усложняет жизнь и приведёт в OEP, здесь мы можем установить BPM ON ACCESS на первую секцию и при проходе через исключение окажемся в OEP.



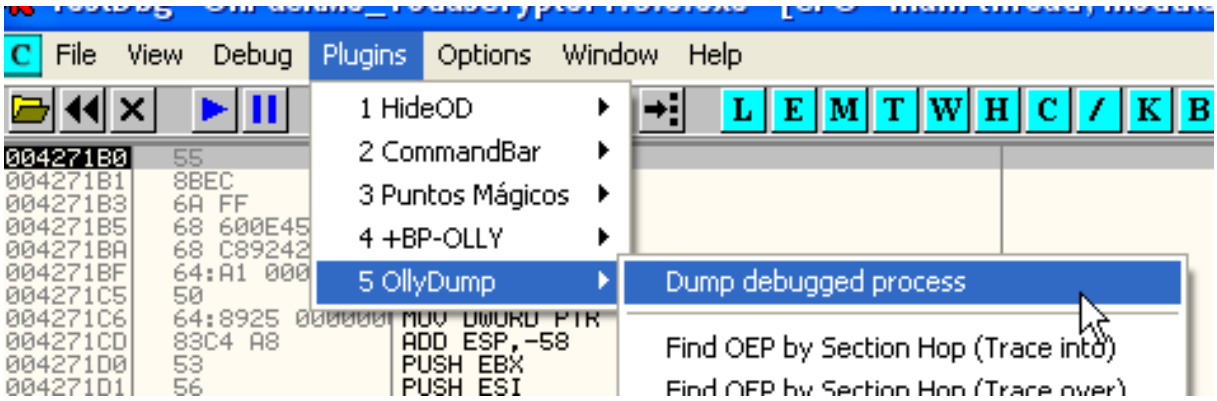


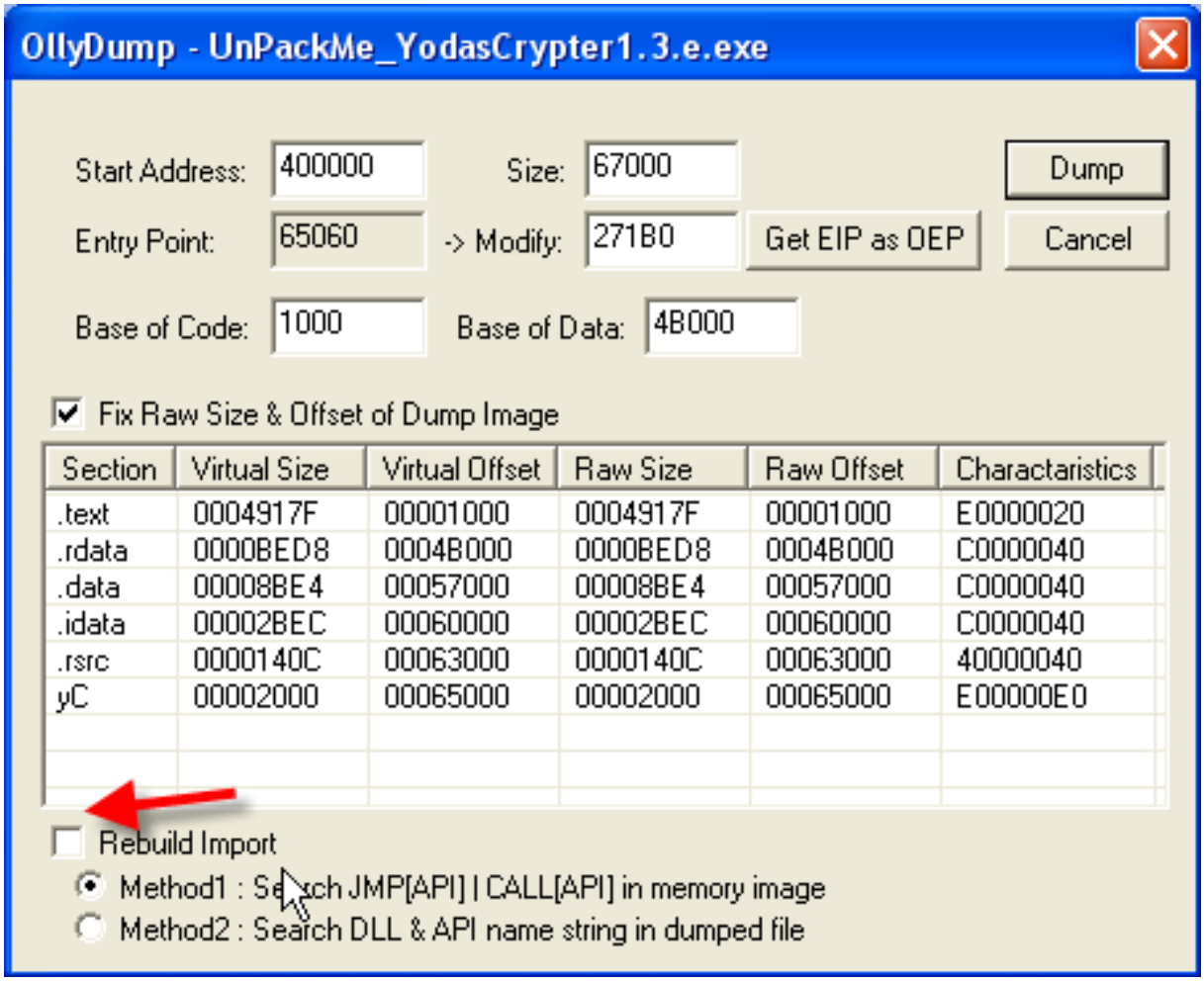
OEP равна 4271B0, так как запакован тот же самый крэкми, хе-хе.



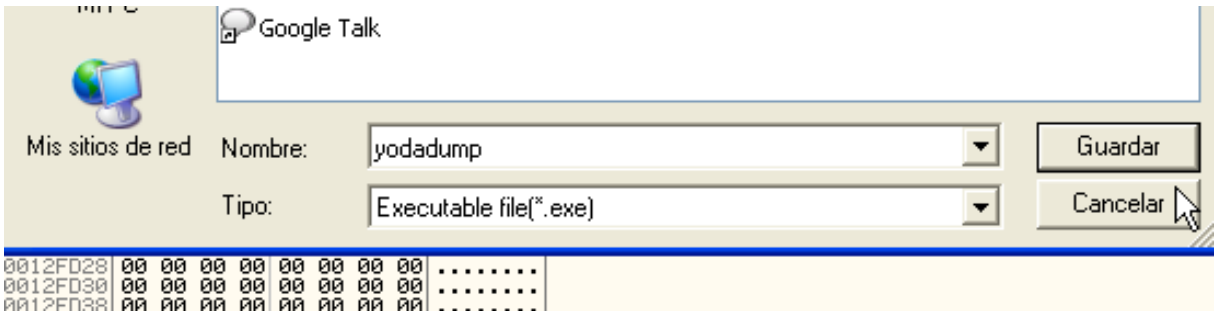
Те, кто хочет лучше исследовать этот вопрос, могут посмотреть, как обработчик исключений манипулирует контекстом для изменения EIP. Адрес исключения (т.е. где происходит исключение), обычно равный 465982, перезаписывается адрес OEP (т.е. 4271B0) для того, чтобы вернуться из исключения напрямую в OEP, хотя сейчас это не так важно. Выходим, отметив на будущее, что происходит со структурой CONTEXT.

Ок, вопрос заключается в том, находимся ли мы уже в OEP, делаем дамп.





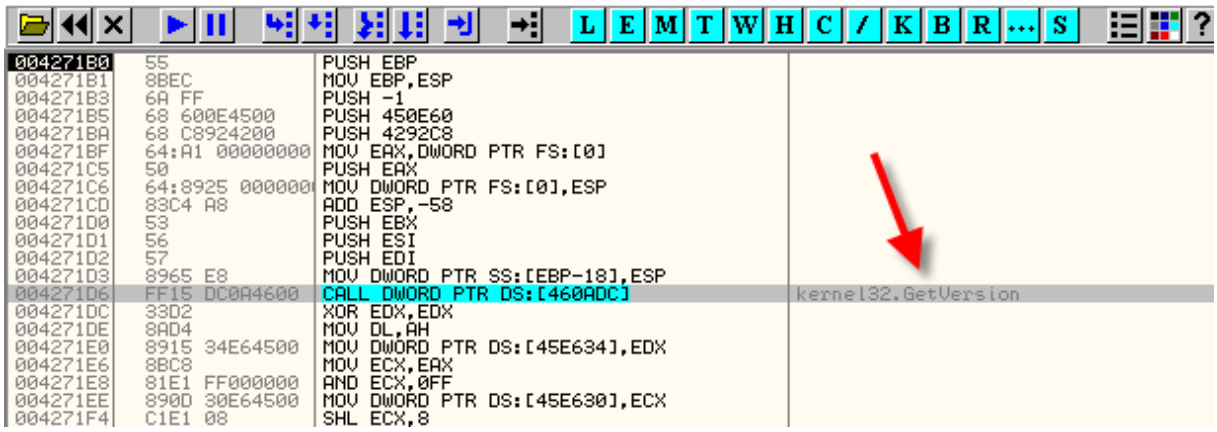
Снимаем галочку, так как не собираемся чинить IAT и делаем дамп.



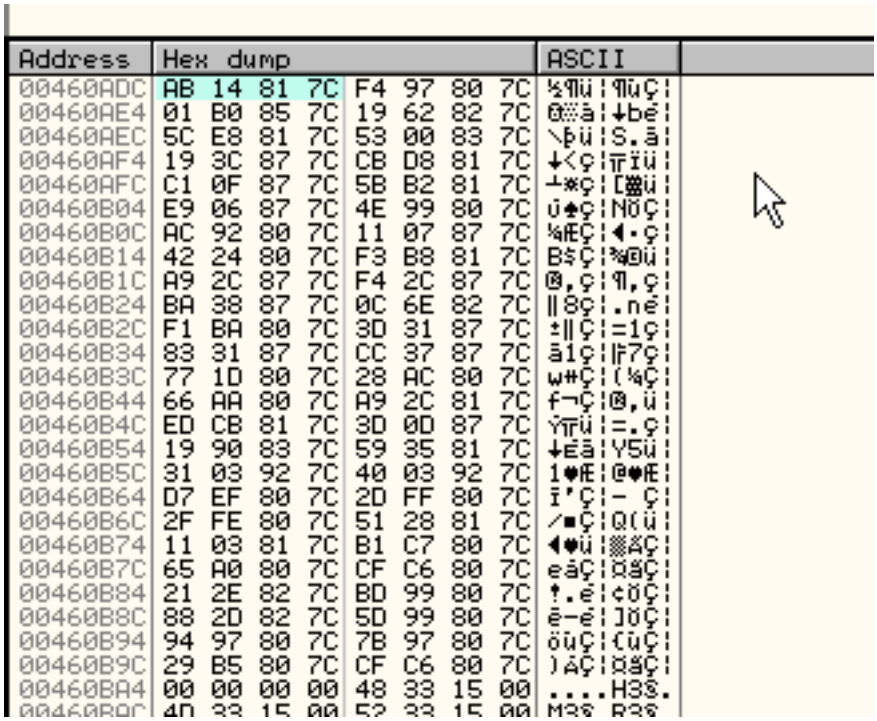
Конечно, если запустить yodadump.exe, то нам отобразится ошибка. Нам очень повезёт, если она запустится хотя бы на нашей машине, но мы уже видели, что в IAT'е есть переадресовочные элементы, которые точно не сработают при попытке получить доступ к этим несуществующим

адресам через дамп.

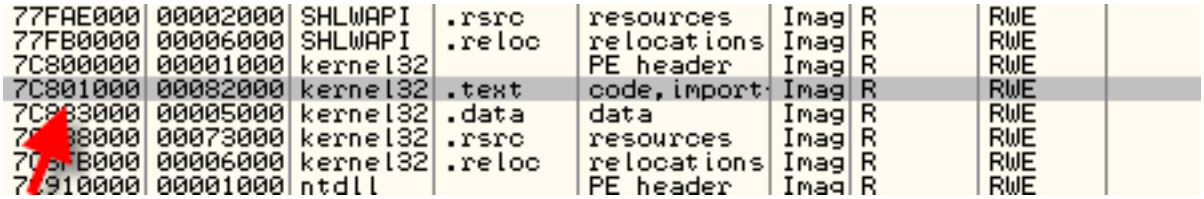
Ладно, начнём анализировать IAT, ищем вызов API-функции, это будет GetVersion, как обычно, хе-хе.



Итак, видим, что указанный вызов имеет значение 460ADC, и для того, что увидеть, куда он переходит, то есть какой элемент IAT ему соответствует, идём в DUMP.



Так, эта часть IAT выглядит как правильная, так как если посмотрим на карту памяти, куда ведут элементы, они все относятся к 7C8XXXXX или нулям, и эти адреса располагаются в секции кода kernel32.dll, то есть данные элементы являются правильными.



Однако если поищем ссылки на какой-нибудь из них наугад - щелчок правой кнопкой мыши - FIND REFERENCES - и вот ссылки:

Address	Disassembly	Comment
0040AEE0	CALL DWORD PTR DS:[460B34]	kernel32.ReadConsoleInputA

Если продолжим спускаться до разделения:

Address	Hex dump	ASCII
00460B8C	88 2D 82 7C 5D 99 80 7C	ê-é¦]öÇ¦
00460B94	94 97 80 7C 7B 97 80 7C	öùÇ¦{ùÇ¦
00460B9C	29 B5 80 7C CF C6 80 7C	)ÁÇ¦ÐãÇ¦
00460BA4	00 00 00 00 48 33 15 00	....H3§.
00460BAC	4D 33 15 00 52 33 15 00	M3§.R3§.
00460BB4	57 33 15 00 5C 33 15 00	W3§.\3§.
00460BBC	61 33 15 00 66 33 15 00	a3§.f3§.
00460BC4	6B 33 15 00 70 33 15 00	k3§.p3§.
00460BCC	75 33 15 00 7A 33 15 00	u3§.z3§.
00460BD4	7F 33 15 00 84 33 15 00	⌂3§.ä3§.
00460BDC	89 33 15 00 8E 33 15 00	ë3§.Ä3§.
00460BE4	93 33 15 00 00 00 00 00	ô3§.....
00460BEC	FE 69 A8 7C ED 69 A8 7C	■i¿¦Yi¿¦
00460BF4	80 0E A5 7C 00 00 00 00	Ç♫Ñ¦....
00460BFC	D5 2D 15 00 DA 2D 15 00	╒-§.┌-§.
00460C04	DF 2D 15 00 E4 2D 15 00	▀-§.ő-§.
00460C0C	E9 2D 15 00 EE 2D 15 00	ú-§.¯-§.
00460C14	F3 2D 15 00 F8 2D 15 00	¾-§.°-§.
00460C1C	FD 2D 15 00 02 2E 15 00	²-§.☻.§.
00460C24	07 2E 15 00 0C 2E 15 00	•.§...§.
00460C2C	11 2E 15 00 16 2E 15 00	◄.§.▬.§.
00460C34	1B 2E 15 00 20 2E 15 00	←.§. .§.
00460C3C	25 2E 15 00 2A 2E 15 00	%.§.*.§.
00460C44	2F 2E 15 00 34 2E 15 00	/.§.4.§.
00460C4C	39 2E 15 00 3E 2E 15 00	9.§.>.§.
00460C54	43 2E 15 00 48 2E 15 00	C.§.H.§.
00460C5C	4D 2E 15 00 52 2E 15 00	M.§.R.§.

Видим, что следующая группа ведёт в область памяти 15XXXX, посмотрим, что там:

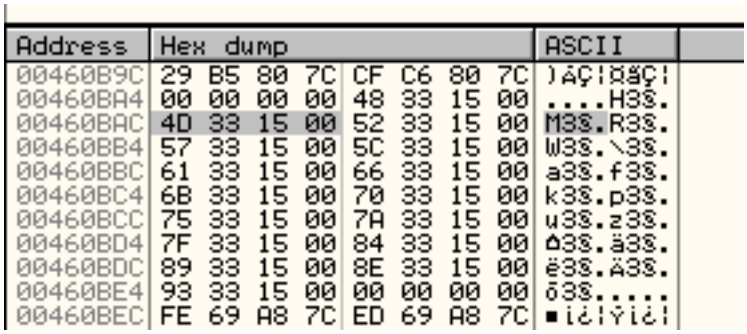
00130000	00002000				Priv	RWE	RWE	
00140000	00003000				Map	R	R	
00150000	00029000				Priv	RW	RW	
00250000	00006000				Priv	RW	RW	
00[illegible]000	00003000				Map	RW	RW	

Видим секцию, которая не в DLL, так что она точно создана упаковщиком. Перезапустим распакуй-меня, чтобы проверить это.

00130000	00002000				Priv	RWE	RWE	
00140000	00003000				Map	R	R	
00150000	00003000				Priv	RW	RW	
00250000	[illegible]6000				Priv	RW	RW	
00260000	[illegible]003000				Map	RW	RW	
00270000	00016000				Map	R	R	\Device\H
00290000	0003D000				Map	R	R	\Device\H
002D0000	00041000				Map	R	R	\Device\H
00320000	00006000				Map	R	R	\Device\H

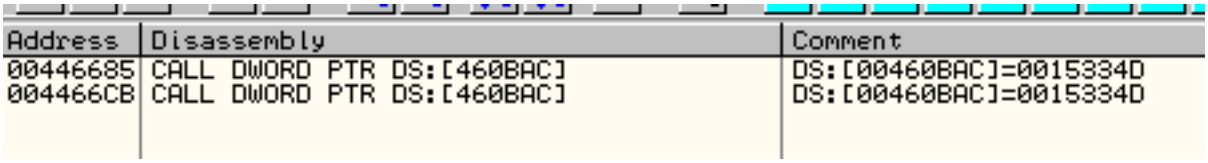
Видим, что есть секция, созданная системой, размером в 3000 байтов, но та, которую использует упаковщик гораздо больше и равна 29000 байта, так что упаковщик увеличил эту секцию для собственного использования.

Поэтому это элементы, переадресующие упаковщиком в данную секцию, посмотрим один из них, что он из себя представляет и как с ним работать.



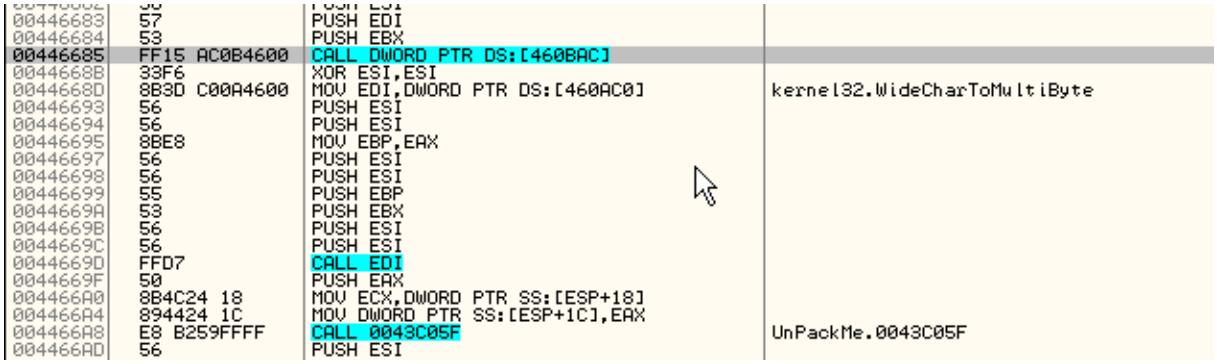
Address	Hex dump	ASCII
00460B9C	29 B5 80 7C CF C6 80 7C	)µÇ¦ÏÆÇ¦
00460BA4	00 00 00 00 48 33 15 00	....H3§.
00460BAC	4D 33 15 00 52 33 15 00	M3§.R3§.
00460BB4	57 33 15 00 5C 33 15 00	W3§.\3§.
00460BBC	61 33 15 00 66 33 15 00	a3§.f3§.
00460BC4	6B 33 15 00 70 33 15 00	k3§.p3§.
00460BCC	75 33 15 00 7A 33 15 00	u3§.z3§.
00460BD4	7F 33 15 00 84 33 15 00	⌂3§.ä3§.
00460BDC	89 33 15 00 8E 33 15 00	ë3§.Ä3§.
00460BE4	93 33 15 00 00 00 00 00	ô3§.....
00460BEC	FE 69 A8 7C ED 69 A8 7C	■i¿¦Ýi¿¦

Возьмём один из них, например этот, и поищем ссылки на него. Щелчок правой кнопкой мыши и FIND REFERENCES.



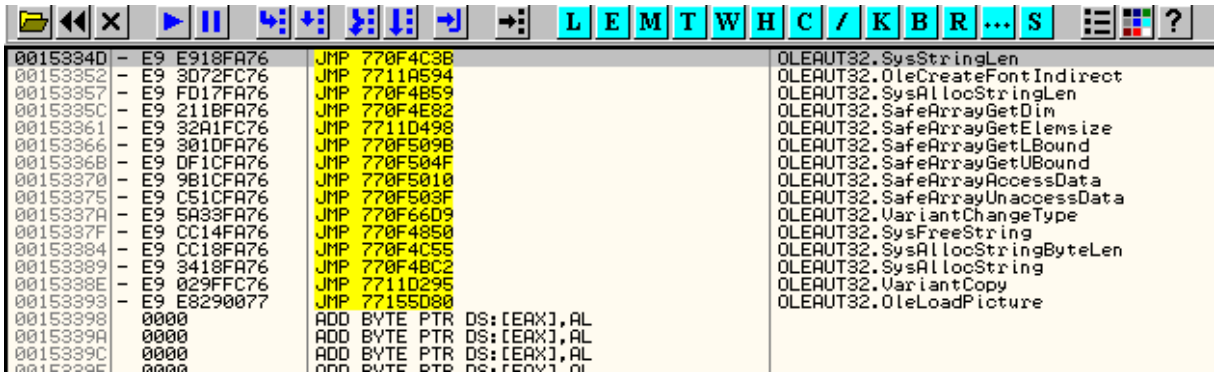
Address	Disassembly	Comment
00446685	CALL DWORD PTR DS:[460BAC]	DS:[00460BAC]=0015334D
004466CB	CALL DWORD PTR DS:[460BAC]	DS:[00460BAC]=0015334D

Есть два вызова, которые принимают значение данного элемента, посмотрим первый, делаем двойной щелчок на нём и попадаем в то место в листинге, где находится этот вызов.



00446683	57	PUSH EDI	
00446684	53	PUSH EBX	
00446685	FF15 AC0B4600	CALL DWORD PTR DS:[460BAC]	
0044668B	33F6	XOR ESI,ESI	
0044668D	8B3D C00A4600	MOV EDI,DWORD PTR DS:[460AC0]	kernel32.WideCharToMultiByte
00446693	56	PUSH ESI	
00446694	56	PUSH ESI	
00446695	8BE8	MOV EBP,EAX	
00446697	56	PUSH ESI	
00446698	56	PUSH ESI	
00446699	55	PUSH EBP	
0044669A	53	PUSH EBX	
0044669B	56	PUSH ESI	
0044669C	56	PUSH ESI	
0044669D	FFD7	CALL EDI	
0044669F	50	PUSH EAX	
004466A0	8B4C24 18	MOV ECX,DWORD PTR SS:[ESP+18]	
004466A4	894424 1C	MOV DWORD PTR SS:[ESP+1C],EAX	
004466A8	E8 B259FFFF	CALL 0043C05F	UnPackMe.0043C05F
004466AD	56	PUSH ESI	

Видим вызов, делаем щелчок правой кнопкой мыши, а затем выбираем FOLLOW, чтобы увидеть, что происходит в переадресовочной секции.

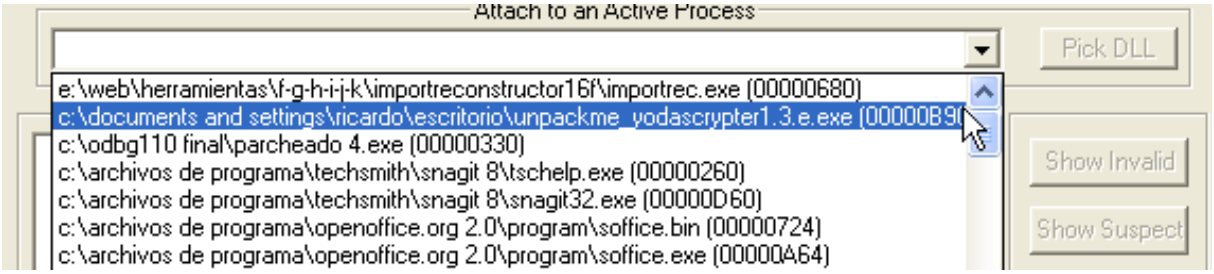


0015334D	E9 E918FA76	JMP 770F4C3B	OLEAUT32.SysStringLen
00153352	E9 3D72FC76	JMP 7711A594	OLEAUT32.OleCreateFontIndirect
00153357	E9 FD17FA76	JMP 770F4B59	OLEAUT32.SysAllocStringLen
0015335C	E9 211BFA76	JMP 770F4E82	OLEAUT32.SafeArrayGetDim
00153361	E9 32A1FC76	JMP 7711D498	OLEAUT32.SafeArrayGetElemsize
00153366	E9 301DFA76	JMP 770F509B	OLEAUT32.SafeArrayGetLBound
0015336B	E9 DF1CFA76	JMP 770F504F	OLEAUT32.SafeArrayGetUBound
00153370	E9 9B1CFA76	JMP 770F5010	OLEAUT32.SafeArrayAccessData
00153375	E9 C51CFA76	JMP 770F503F	OLEAUT32.SafeArrayUnaccessData
0015337A	E9 5A33FA76	JMP 770F66D9	OLEAUT32.VariantChangeType
0015337F	E9 CC14FA76	JMP 770F4850	OLEAUT32.SysFreeString
00153384	E9 CC18FA76	JMP 770F4C55	OLEAUT32.SysAllocStringByteLen
00153389	E9 3418FA76	JMP 770F4BC2	OLEAUT32.SysAllocString
0015338E	E9 029FFC76	JMP 7711D295	OLEAUT32.VariantCopy
00153393	E9 E8290077	JMP 77155D80	OLEAUT32.OleLoadPicture
00153398	0000	ADD BYTE PTR DS:[EAX],AL	
0015339A	0000	ADD BYTE PTR DS:[EAX],AL	
0015339C	0000	ADD BYTE PTR DS:[EAX],AL	

Видим, что без особых заморочек напрямую прыгаем на API-функцию SysStringLen, поэтому если будем трассировать вручную, то уже знаем, что это правильная функция для данного элемента.

Видим, что переадресация очень простая, сможет ли IMP REC починить её с помощью своих трассировщиков, чтобы не надо было искать волшебный переход?

Открываем IMP REC.

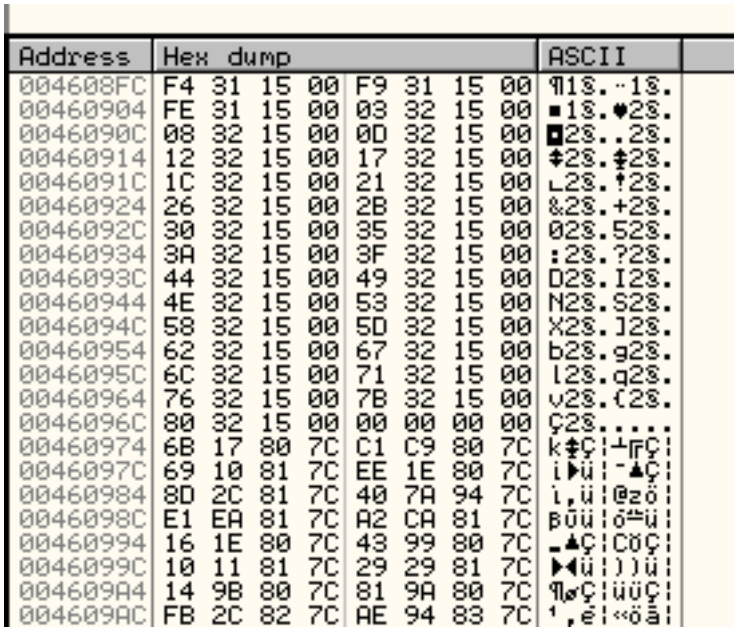


Вот процесс, остановленный на OEP; выбираем его в выпадающем меню.

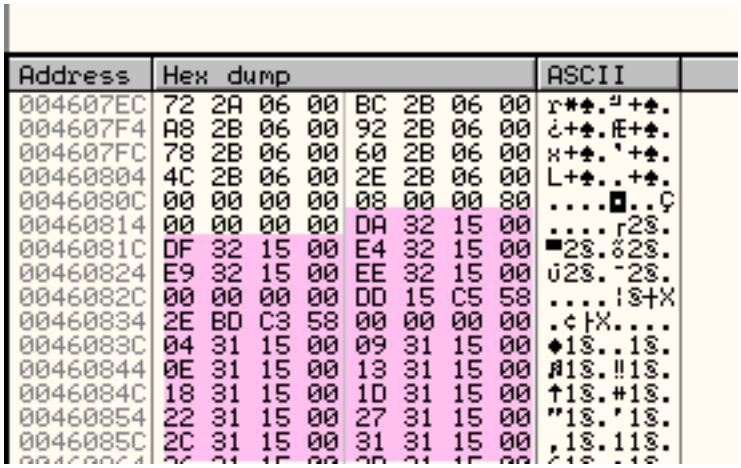
Не забываем, что нужно задать IMP REC'у три значения: OEP, НАЧАЛО IAT и РАЗМЕР.

OEP=4271B0, из чего вычитаем базу образа, остаётся 271B0.

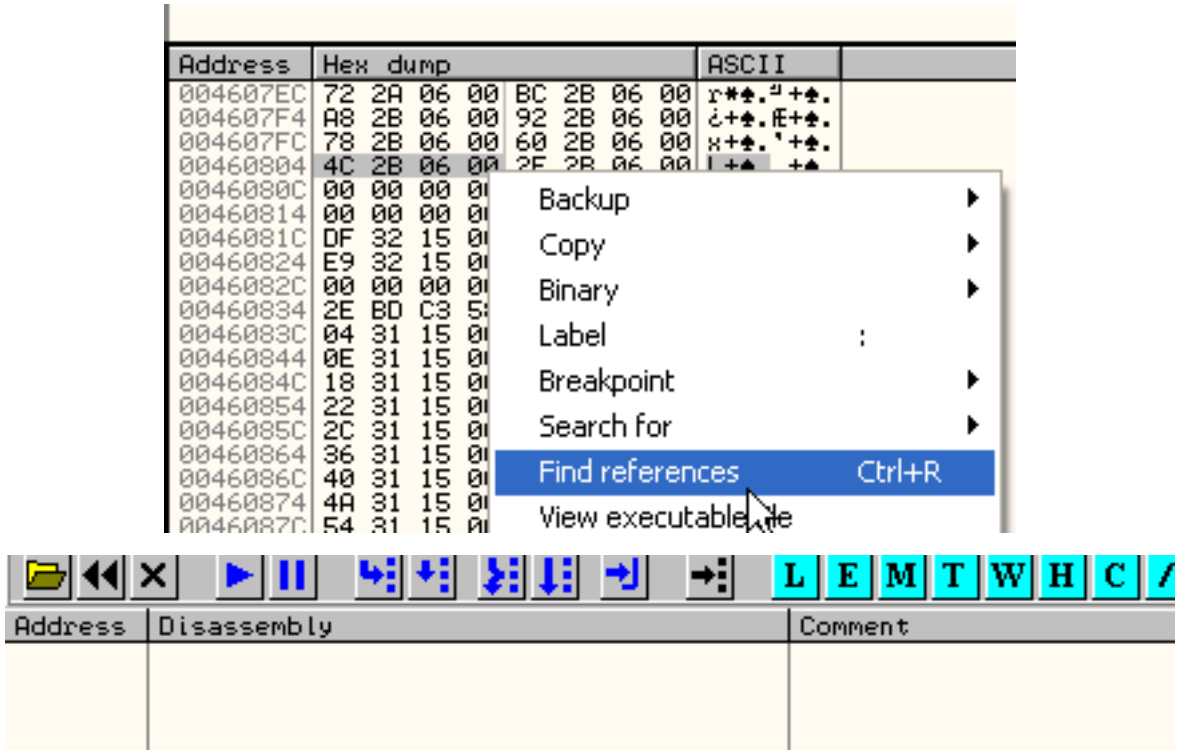
Начало IAT, если спустимся ниже:



Видим, что элементы IAT ведут или в секции кода DLL или в секцию 15XXXX.

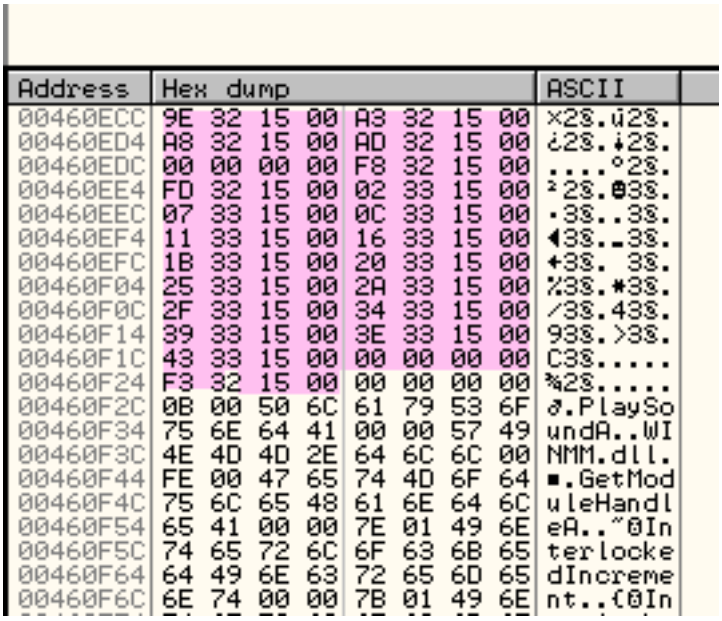


Смотрим до тех пор, пока схема повторяется. Если выше поискать ссылки на предмет, есть ли в IAT ещё элементы, ничего не находим.



Поэтому первый элемент IAT - это 460818, для IMP REC'a RVA=60818, если отнять базу образа.

Конец IAT находим таким же образом, спускаемся ниже, чтобы найти, где кончается закономерность, проявляющаяся в том, что элементы ведут или в секции кода DLL или в секцию 15XXXX.



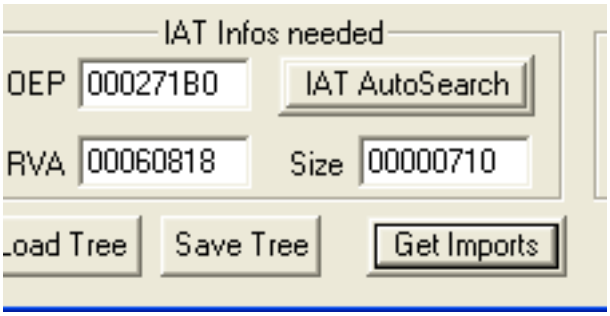
Здесь видим последний элементы IAT, находящийся по адресу 460f24, и если ниже поищем ссылки, то не найдём ни для какого элемента, так что в качестве конца IAT задаём 460F28. Теперь нужно найти её размер.

РАЗМЕР = КОНЕЦ - НАЧАЛО = 460f28 - 460818 = 710

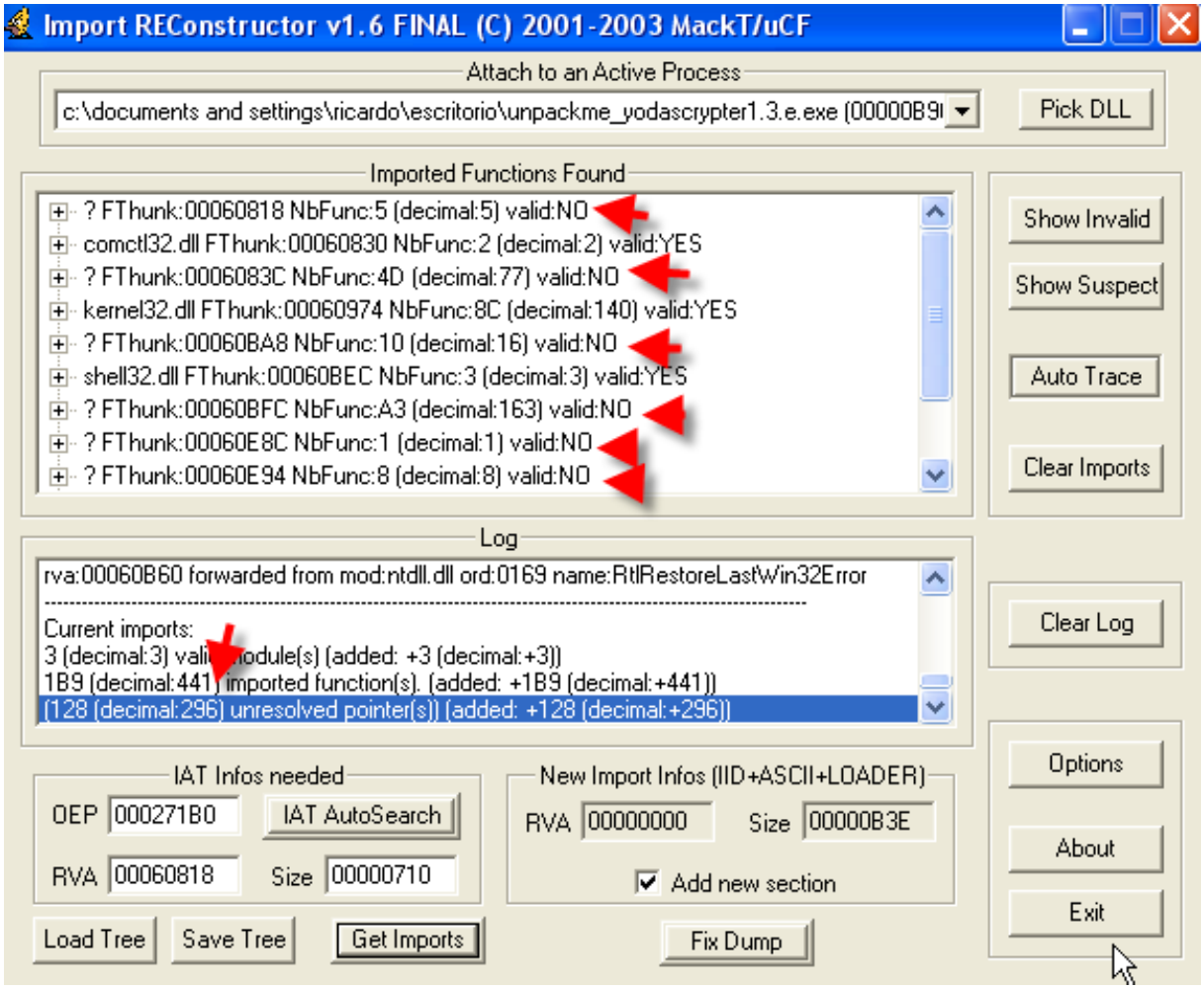
Так что задаём:

```
OEP=271B0
RVA или НАЧАЛО=60818
SIZE или РАЗМЕР=710
```

Забиваем эти значения в IMP REC.

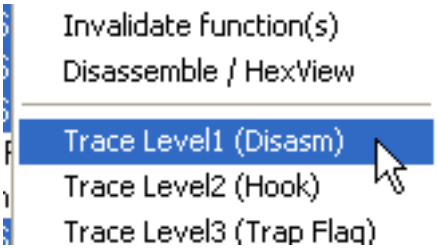


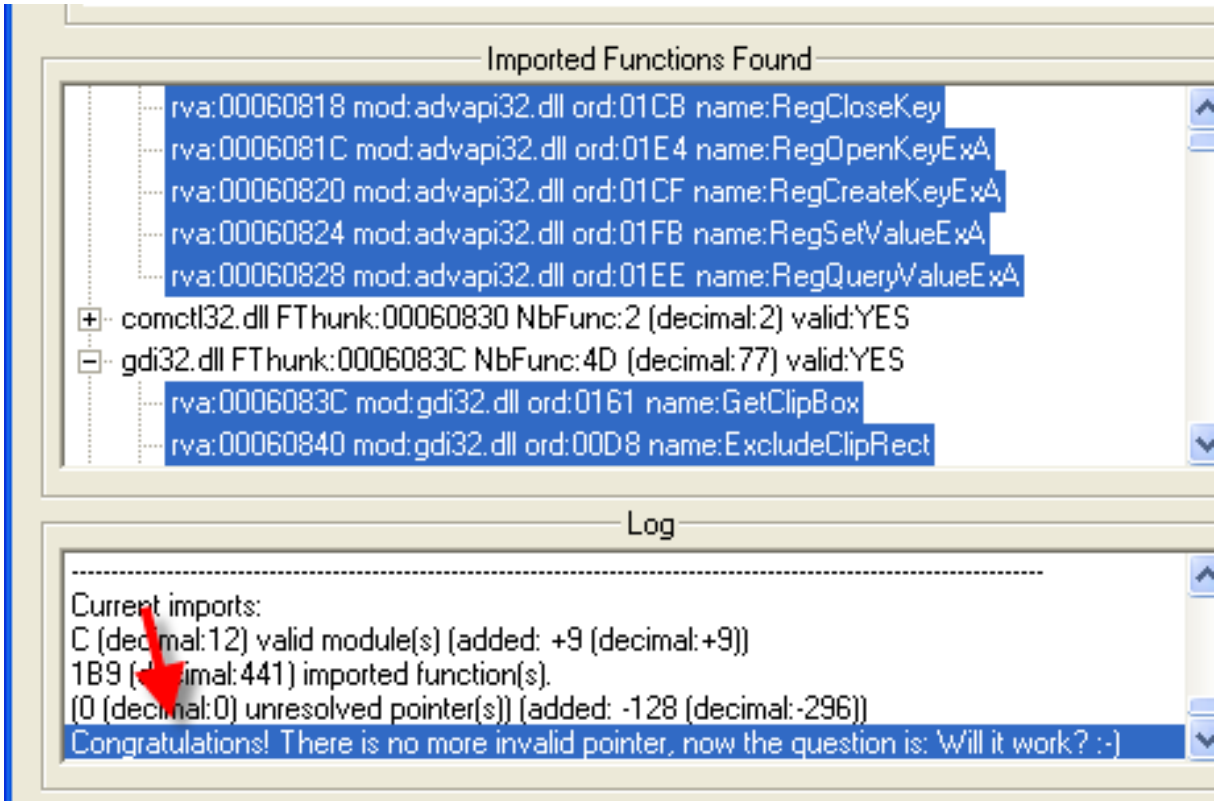
По нажатию на GET IMPORTS:



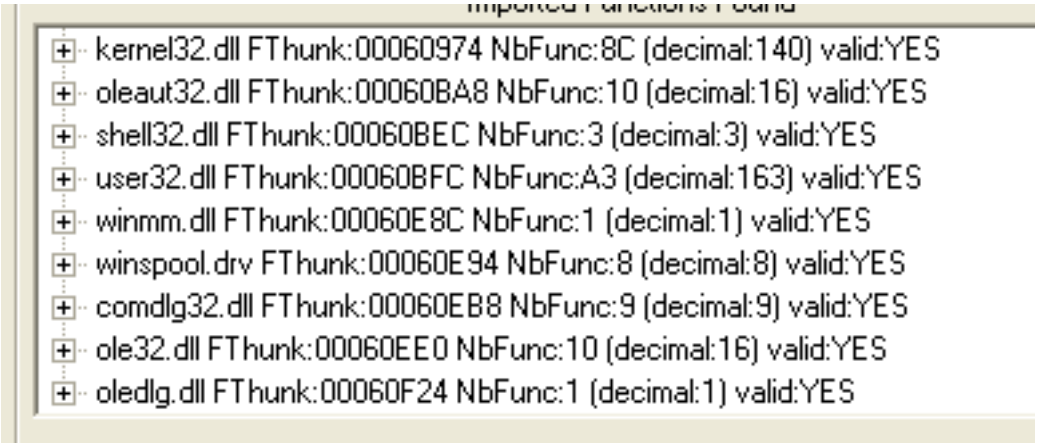
Видим, что есть 296 необработанных элементов, получится ли исправить их с помощью одного из встроенных трассировщиков?

Если нажмём на кнопку AUTO TRACE, программа повиснет. Пробуем с другими трассировщиками. Нажимаем SHOW INVALIDS и на отмеченных элементах делаем щелчок правой кнопкой мыши и выбираем “TRACE LEVEL 1”.

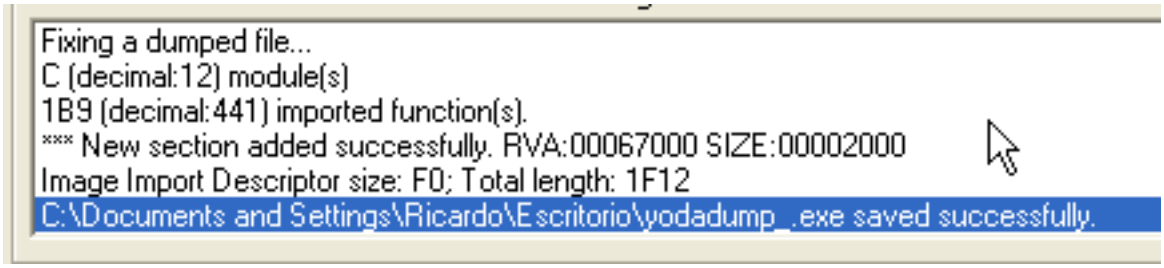




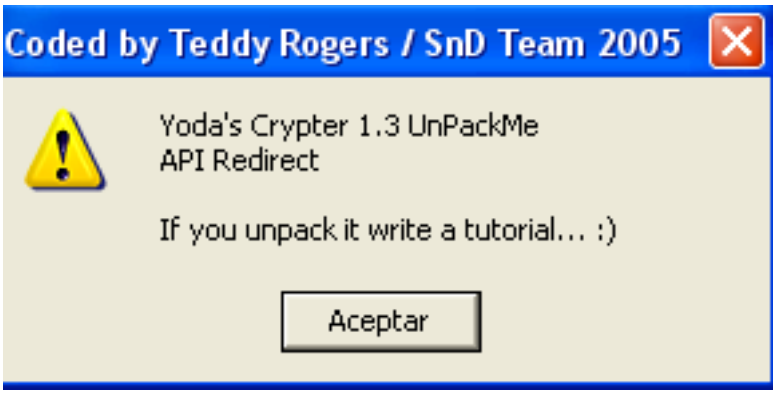
Говорит, что всё исправлено и неправильных элементов больше нет, поверим ему? Хе-хе. Если снова нажмём “SHOW INVALID”, то увидим, что всё помечено как YES.



Пробуем починить yodadump.exe, нажимаем “FIX DUMP”.



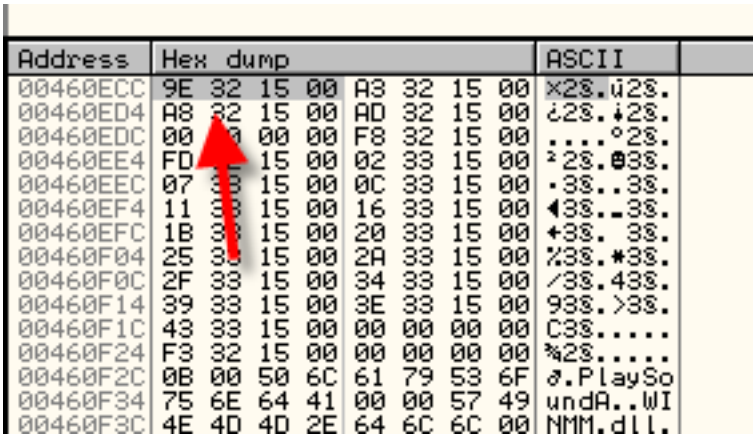
Думаю, что yodadump_.exe починен, попробуем запустить его.



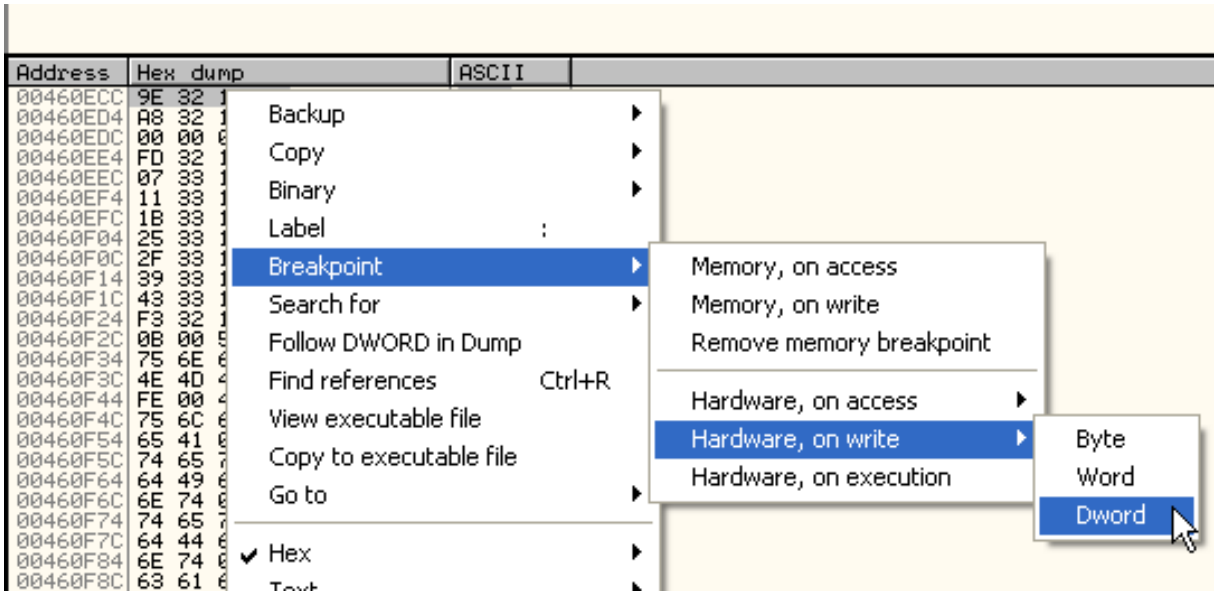
Хе-хе, в этот раз IMP REC показал себя и сэкономил мне много времени и сил.

А теперь слегка напряжёмся и посмотрим, есть ли волшебный переход у этого упаковщика?

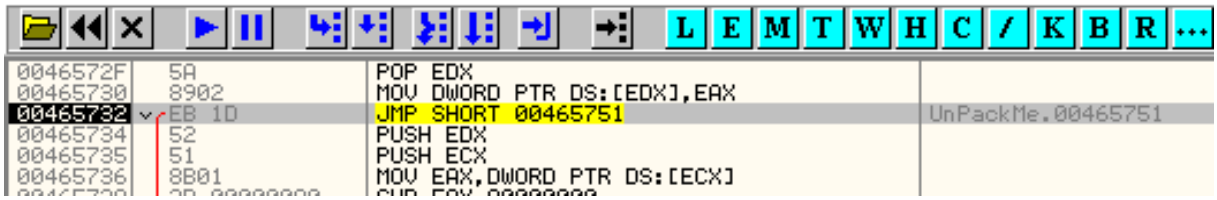
Ищем плохой элемент IAT остановленной на OEP программы.



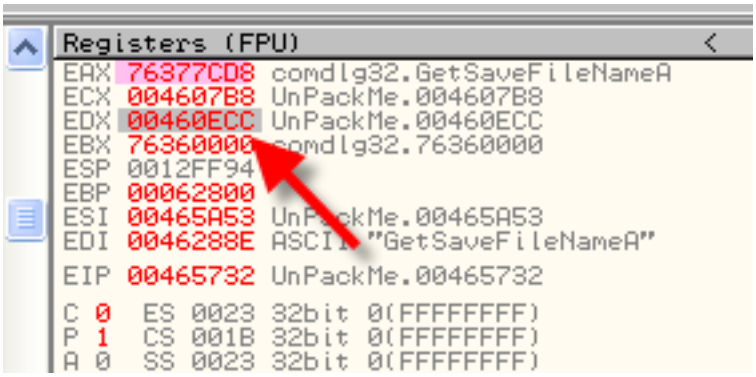
Устанавливаем на него HARDWARE BPX ON WRITE до рестарта программы, так как HBP остаются даже после этого, и как мы уже видели при использовании метода PUSHAD, упаковщик не реагирует на этот вид остановов.



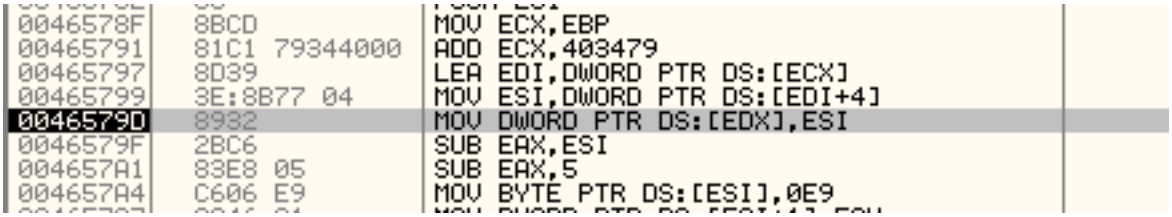
Очевидно, что метод заключается в том, чтобы остановиться, когда в элемент сохраняется плохое значение 15XXXX. Делаем рестарт и нажимаем RUN.



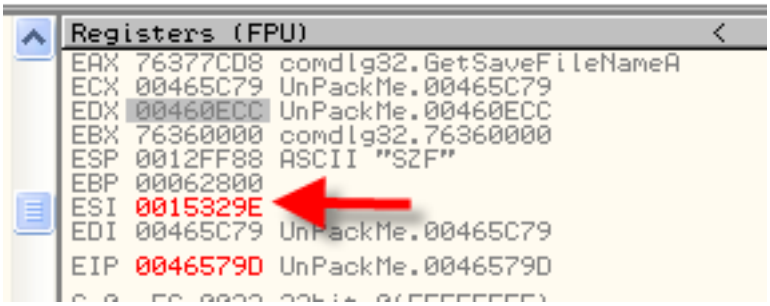
Останавливаемся здесь и сохраняем хорошее значение в элемент IAT. Видим, что EAX содержит значение API-функции.



И значение сохраняется в элемент, на который указывает EDX, и это значит, что в данном случае сначала сохраняется хорошее значение, а потом меняется на плохое. Продолжает трассировать, чтобы увидеть, где это происходит.

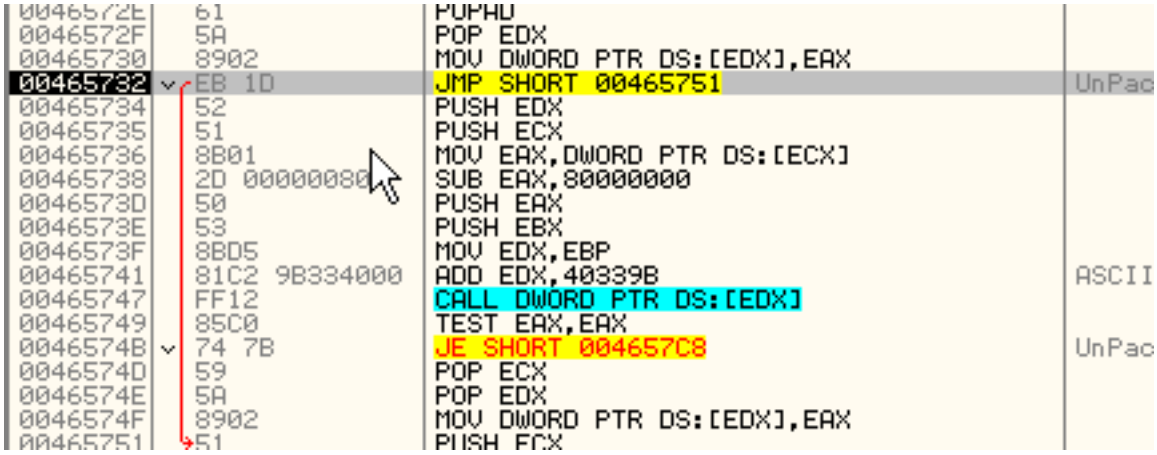


Видим, что всего лишь несколькими строками ниже ESI принимает плохое значение, и готовится к загрузке его в элемент.

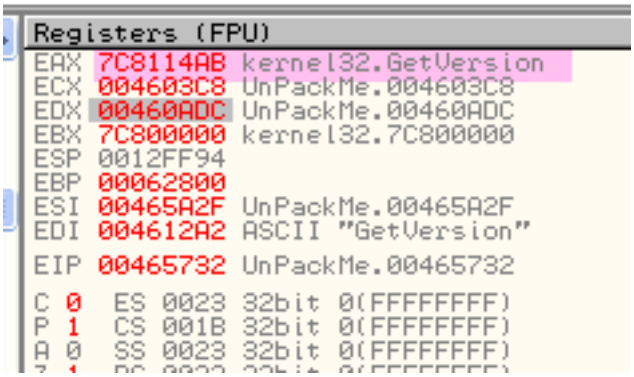


Это означает, что хорошие элементы получают значение только в первый раз, но не во второй, когда элемент перезаписывается плохим значением.

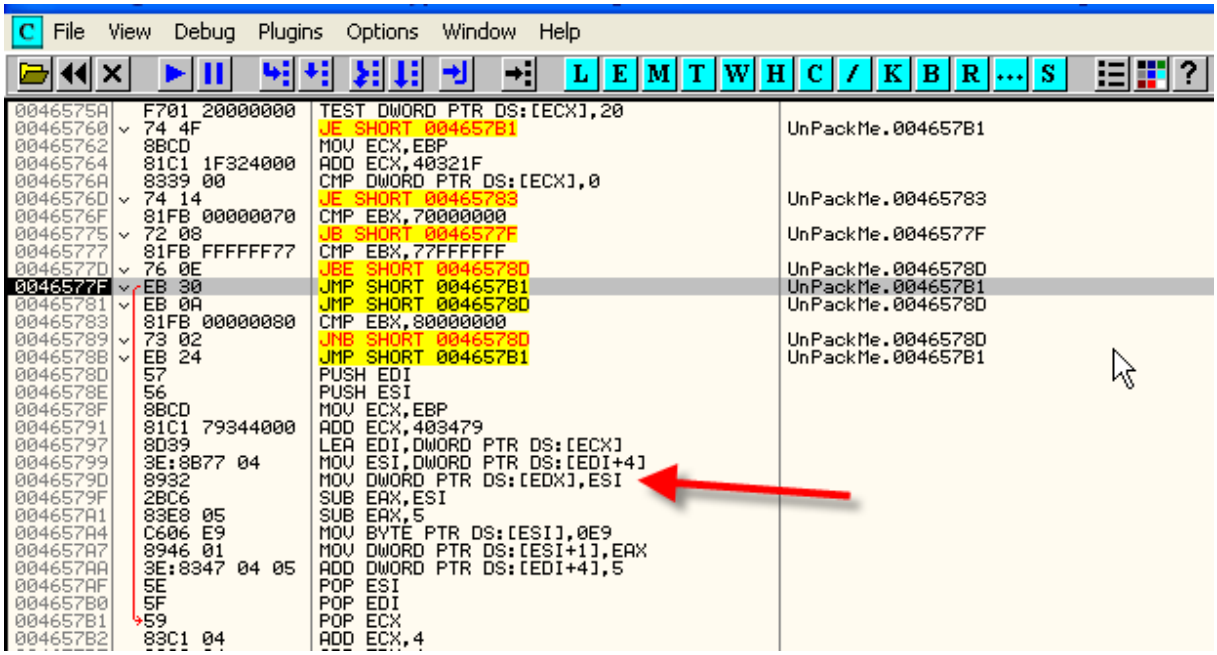
Повторяем процесс с хорошим элементом, связанным с GetVersion и находящимся несколько строки ниже OEP. Используем его и установим HBP ON WRITE.



Останавливаемся здесь при сохранении правильного адреса, находящегося в EAX.

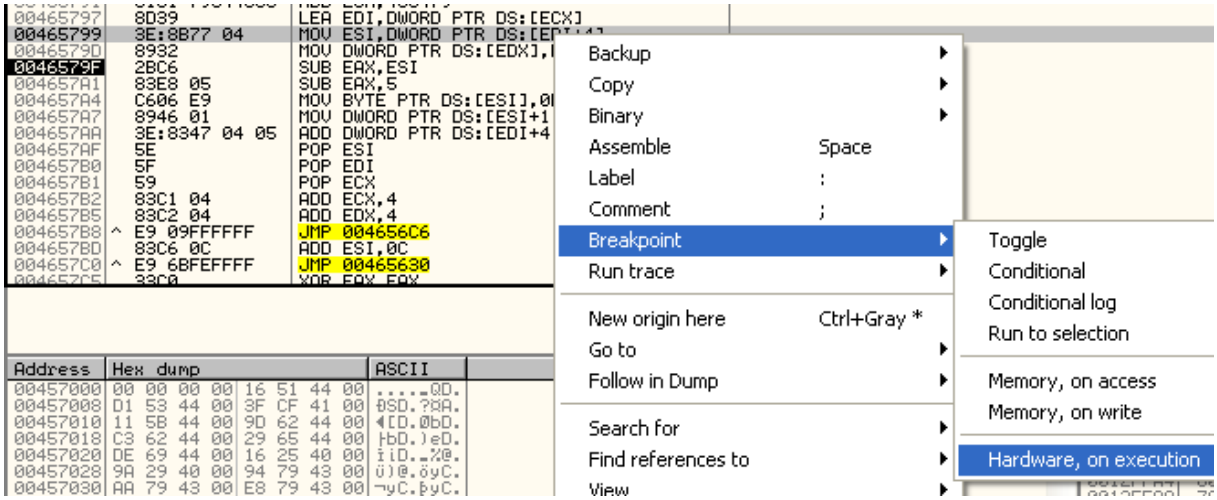


Немного потрассируем.

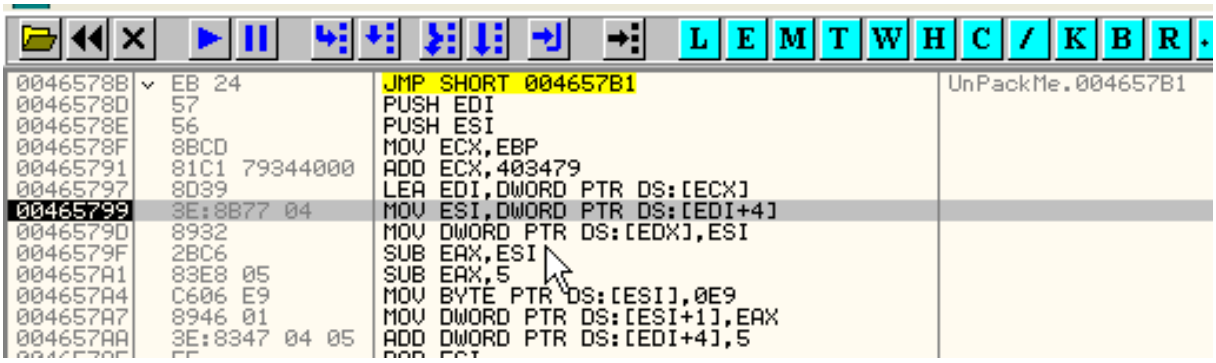


Видим, что оказались на JMP, который перепрыгивает через отмеченную красной стрелкой область, где содержимое элемента меняется на плохое значение, так что суть заключается в том, срабатывает или нет этот JMP или нет, предыдущие переходы, избегавшие этого JMP и есть волшебные переходы, посмотрим, были ли до JMP какие-то условные переходы.

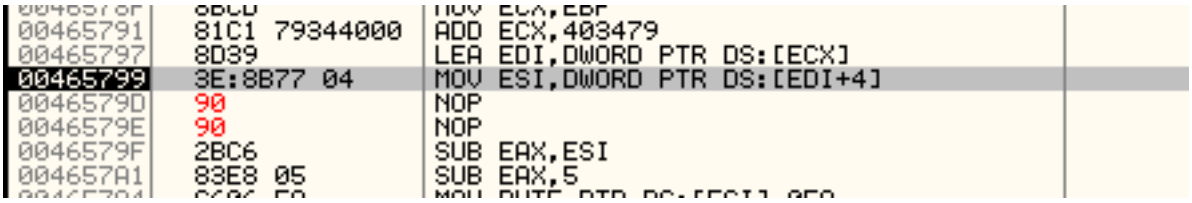
Мы также можем попытаться забить NOP'ами строку, где сохраняется плохое значение, так чтобы этого не происходило, и это будет нормальным решением, попробуем, сработает ли это.



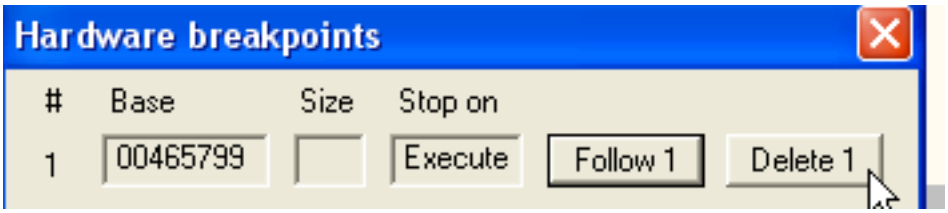
Отмечаем строку, где сохраняется плохое значение, и устанавливаем HBP ON EXECUTION, рестартуем и останавливаемся здесь в первый раз. Установили HBP на предыдущую строку, так как нам не нужно, чтобы плохое значение сохранилось.



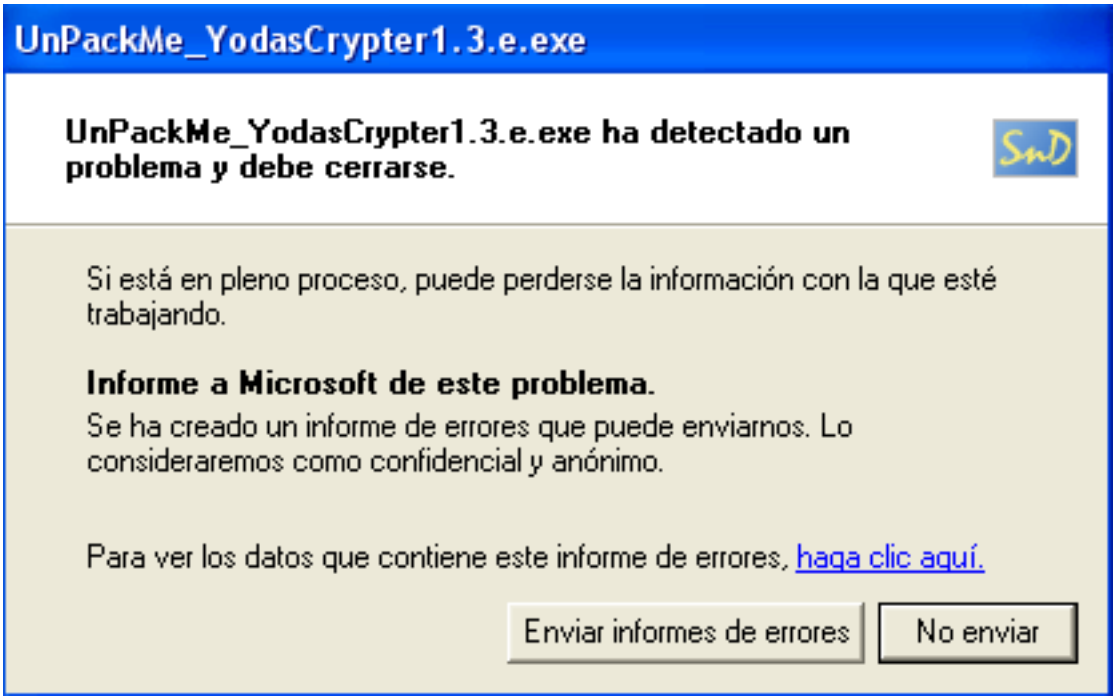
Остановились тут, попробуем забить NOP'ами строку, где сохраняется плохое значение.



Продолжаем, чтобы узнать, определиться ли сделанная нами забивка, и оказываемся в OEP.



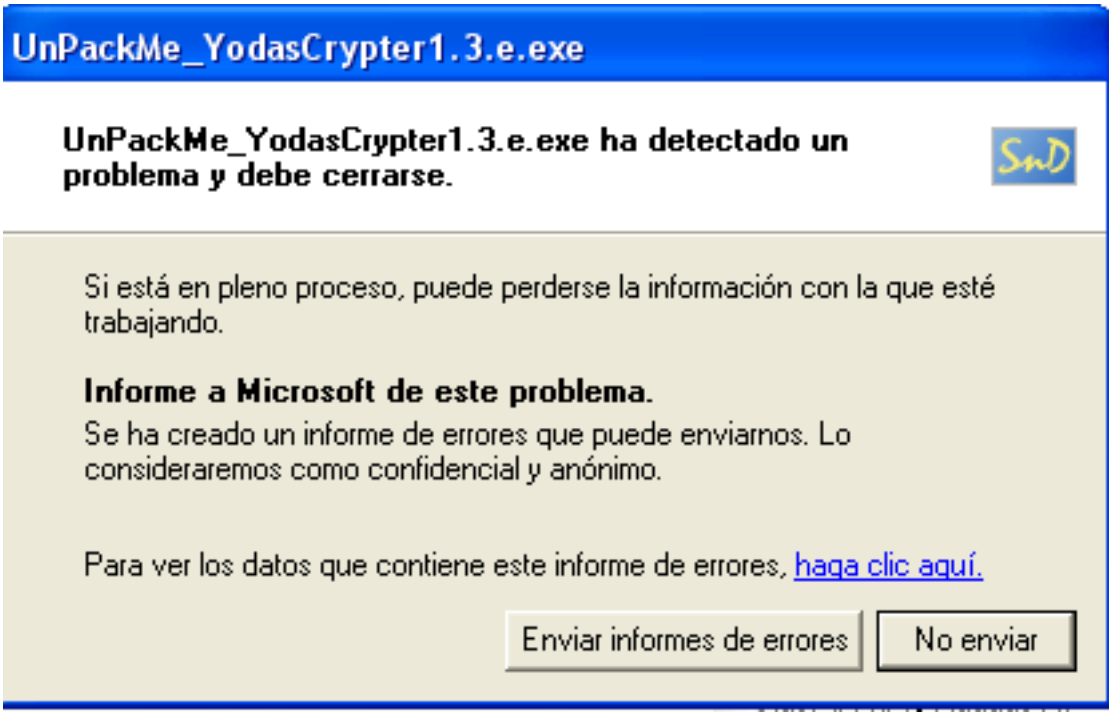
Снимаем HBP и делаем RUN.



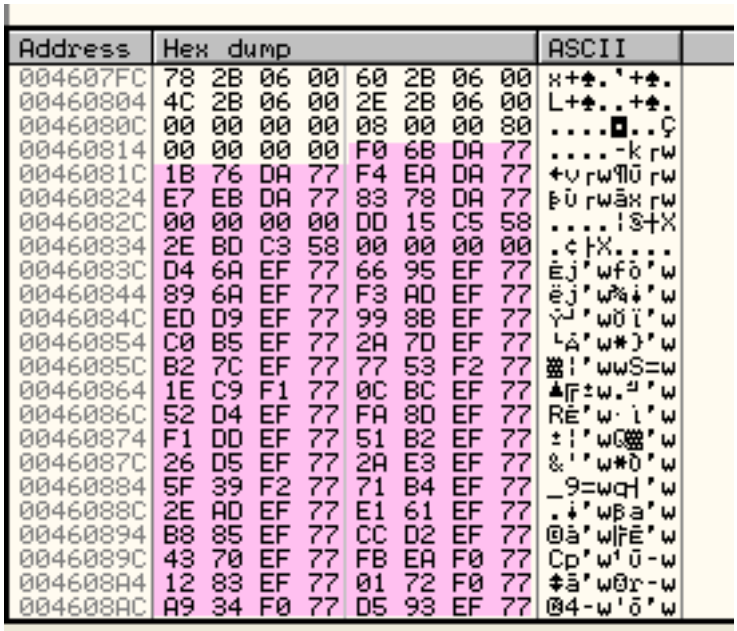
Да, чёртов упаковщик определил, что произошли изменения и выдал ошибку, так что забивка NOP'ами не годится, продолжаем работать с волшебным переходом.

```
0046576F   81FB 00000070   CMP EBX,70000000
00465775   72 08           JB SHORT 0046577F                  UnPackMe.0046577F
00465777   81FB FFFFFF77   CMP EBX,77FFFFFF
0046577D   76 0E           JBE SHORT 0046578D                 UnPackMe.0046578D
0046577F   EB 30           JMP SHORT 004657B1                 UnPackMe.004657B1
00465781   EB 0A           JMP SHORT 0046578D                 UnPackMe.0046578D
00465783   81FB 00000080   CMP EBX,80000000
00465789   73 02           JNB SHORT 0046578D                 UnPackMe.0046578D
0046578B   EB 24           JMP SHORT 004657B1                 UnPackMe.004657B1
0046578D   57              PUSH EDI
0046578E   56              PUSH ESI
```

Оттрасировав какой-нибудь плохой элемент, смотрим, является ли он волшебным, т.е. избегает ли вышеуказанного JMP, чтобы оказаться в области, где происходит сохранение плохого значения, и попробуем забить его NOP'ами, хотя сомневаюсь, что упаковщик не обнаружит забивки. Пробуем.



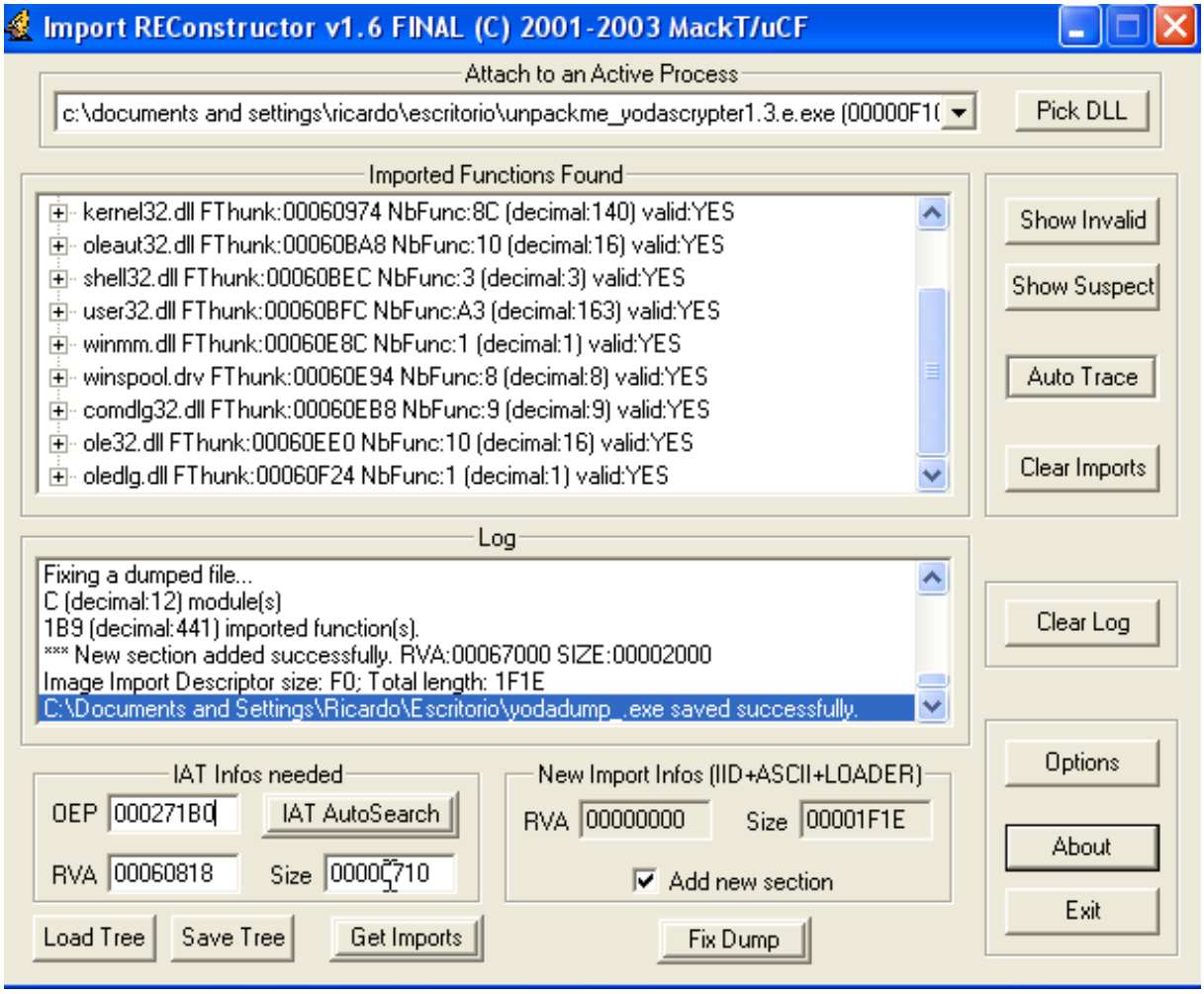
Программа и в этом случае выдаёт нам ошибку, но уже после починки таблицы.



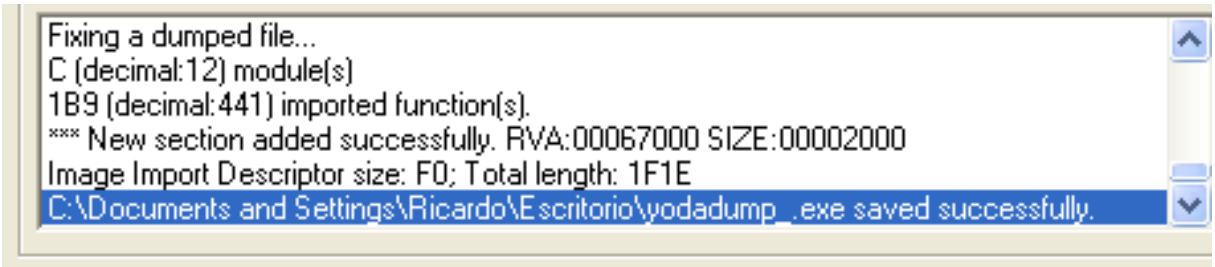
Address	Hex dump	ASCII
004607FC	78 2B 06 00 60 2B 06 00	x+♠.`+♠.
00460804	4C 2B 06 00 2E 2B 06 00	L+♠..+♠.
0046080C	00 00 00 00 08 00 00 80	....◘..Ç
00460814	00 00 00 00 F0 6B DA 77	....≡k┌w
0046081C	1B 76 DA 77 F4 EA DA 77	←v┌w¶Ω┌w
00460824	E7 EB DA 77 83 78 DA 77	τΩ┌wâx┌w
0046082C	00 00 00 00 DD 15 C5 58	....¦§┼X
00460834	2E BD C3 58 00 00 00 00	.¢├X....
0046083C	D4 6A EF 77 66 95 EF 77	Ôj∩wf∩w
00460844	89 6A EF 77 F3 AD EF 77	ëj∩w%i∩w
0046084C	ED D9 EF 77 99 8B EF 77	ÿ┘∩wÖï∩w
00460854	C0 B5 EF 77 2A 7D EF 77	└Á∩w*}∩w
0046085C	B2 7C EF 77 77 53 F2 77	▒¦∩wwS≥w
00460864	1E C9 F1 77 0C BC EF 77	▲╔±w.╝∩w
0046086C	52 D4 EF 77 FA 8D EF 77	RÈ∩w·ì∩w
00460874	F1 DD EF 77 51 B2 EF 77	±¦∩wQ▒∩w
0046087C	26 D5 EF 77 2A E3 EF 77	&'∩w*Ò∩w
00460884	5F 39 F2 77 71 B4 EF 77	_9≥wq┤∩w
0046088C	2E AD EF 77 E1 61 EF 77	.↓∩wßa∩w
00460894	B8 85 EF 77 CC D2 EF 77	@à∩w╠╥∩w
0046089C	43 70 EF 77 FB EA F0 77	Cp∩w'Ü-w
004608A4	12 83 EF 77 01 72 F0 77	↕â∩w☺r≡w
004608AC	A9 34 F0 77 D5 93 EF 77	@4-w'õ∩w

Видим, что IAT - правильная, так что с ней проблем нет. У нас есть две возможности. Одна - открыть другую копию крэкми в другом экземпляре OllyDbg и, ничего не меняя, остановится на OEP

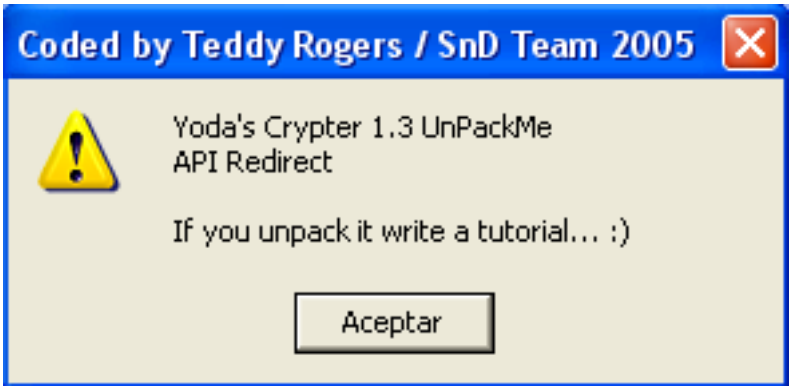
и скопировать правильную таблицу поверх плохой с помощью BYNARY COPY и BINARY PASTE, что проще всего. Также можно использовать напрямую этот процесс для IMP REC'a, так как хотя мы не прибыли в OEP, у нас есть правильная IAT, и это то, с чем работает IMP REC, остальное неважно, так что можем открыть процесс в IMP REC'e, установить правильные значения OEP, RVA и SIZE и нажать GET IMPORTS.



Видим, что IMP REC признал все значения правильными, и хотя остановленный процесс не служил в качестве источника для дампа, так как мы не дошли до OEP, это неважно, так как крэкми был сдамплен раньше, используя процесс, остановленный на OEP, и единственное, что в этом дампе неправильно - это IAT, которую мы можем починить, и всё будет будет работать. Нам нужно стереть предыдущий yodadump_.exe, который уже починен, и поискать дамп yodadump.exe и нажать "Fix Dump", и он починится.



Создан новый yodadump_.exe, смотрим, работает ли он.



Если работает нормально, то это значит, что мы теперь знаем: дамп необходимо делать из OEP, так как именно тогда вся программа уже распакована в памяти, но таблицу можно брать, когда она уже заполнена правильными значениями, хотя бы мы ещё и не были в OEP, потому что IMP REC ничего не меняет в дампе, кроме IAT, которая берётся правильная, так что дамп прекрасно работает.

В качестве упражнения прилагается файлик для распаковки, он очень прост, и думаю, что вы должны с ним справиться без особого труда.

Файлы к статье (находится в архиве `wasm_files.zip`: `files/0508/ollydbg38.zip`)

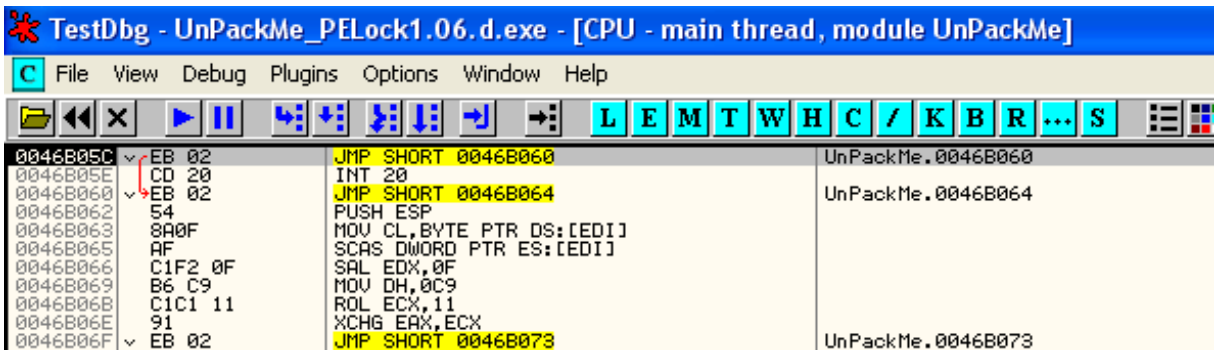
Введение в крэкинг с нуля, используя OllyDbg - Глава 39

Автор: Рикардо Нарваха, пер. Aquila

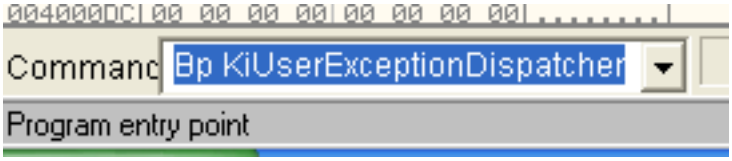
Дата: 2009-02-02

В этой и последующей главах мы будем разбирать украденные байты и скрипты для них, которые мы будем использовать для распакуй-меня, известного как UnPackMe_PELock1.06.d.exe, в котором есть своя изюминка, и который (или вариации которого) используется в некоторых отличных туториалах.

Находимся в начале программы, открытой в OllyDbg.

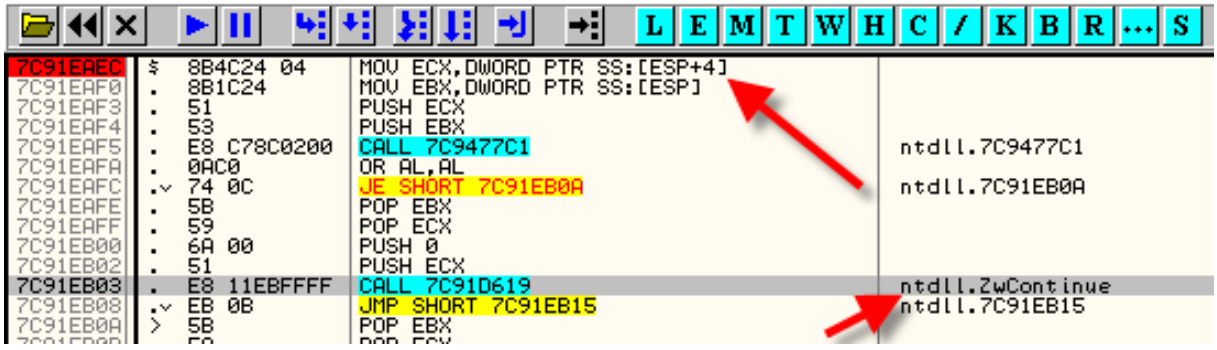


Используем разновидность метода исключений, позже увидим почему, в данном случае вместо того, чтобы полагаться на лог OllyDbg, сделаем собственный лог исключений, потому что в данных упаковщиках исключения камуфлируются, так что может получиться большая путаница, ведь OllyDbg несовершенен, и со мной это уже бывало в некоторых упаковщиках, так что будем изучать следующий BP.



Этот BP будет нашим самописцем исключений. OllyDbg их тоже записывает, но когда я встречаю упаковщик с большим количеством исключений, а тем более создающий нить (а я думаю, что этот её создаёт), то до того как оказаться в OEP, всегда полагаюсь на самописец, установленный мной, а не на встроенный OllyDbg. Дело в том, что все исключения должны проходить через место, в которое мы только что установили BP. Если посмотрим в OllyDbg:

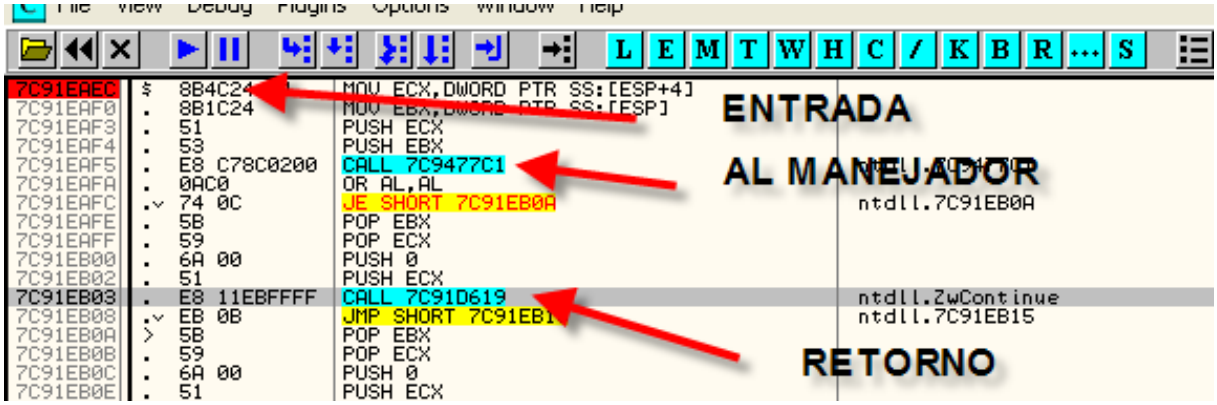
Мы легко можем проанализировать эту процедуру следующим образом:



То есть сначала идёт отмеченное место, отмеченное как ENTRADA (вход) – это начало процедуры, которя обрабатывает исключения в ntdll.dll, потом вызов, помеченный как AL MANEJADOR (ОБРАБОТЧИК), переход на обработчик исключений, который мы можем увидеть с помощью VIEW-SEH CHAIN, после запуска обработчика исключений идёт возврат на эту процедура, а затем до ВОЗВРАТА (RETORNO), где внутри этого вызова продолжается выполнение программы.

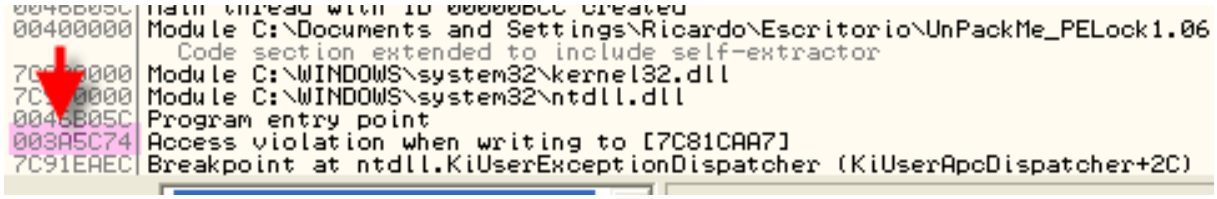
Для тех, кто начнёт трассировать, поясню, что когда выходим отсюда в программу, не можем использовать всё время трассировку, так как есть посреднические процедуры, работающие в нулевом кольце, и OLLYDBG не может их перехватить. В этом случае, если хотим остановиться в программе, нужно установить BP на обработчик, или на возврат, или BPM ON ACCESS на всю секцию, где происходит исключение, из-за чего OllyDbg, когда не сможет выполнить код, работающий в нулевом кольце, снова остановится в программе. Говорю это, потому что всегда найдётся человек, который захочет оттрассировать отсюда вглубь программы, и не сможет этого сделать, когда OllyDbg дойдёт до метса, где происходит переход в нулевом кольце.

Ок, теперь у нас есть всё, что нужно, и если нажмём RUN, то остановимся на первом исключении.

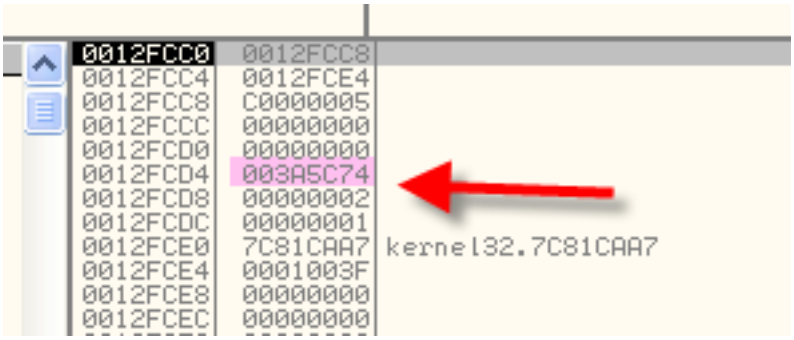


Видим в логе, что оно возникло на моей машине в 3a5c74, а сразу же после него останавливаемся на установленном нами BP, так как в настройках у нас отмечена опция DEBUGGING OPTIONS-EXCEPTIONS.

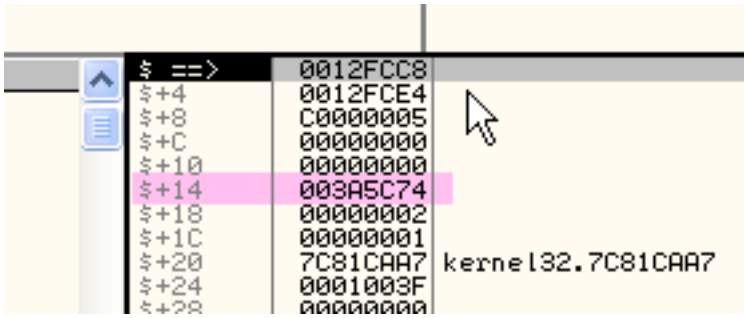
Теперь если взглянем на стек.



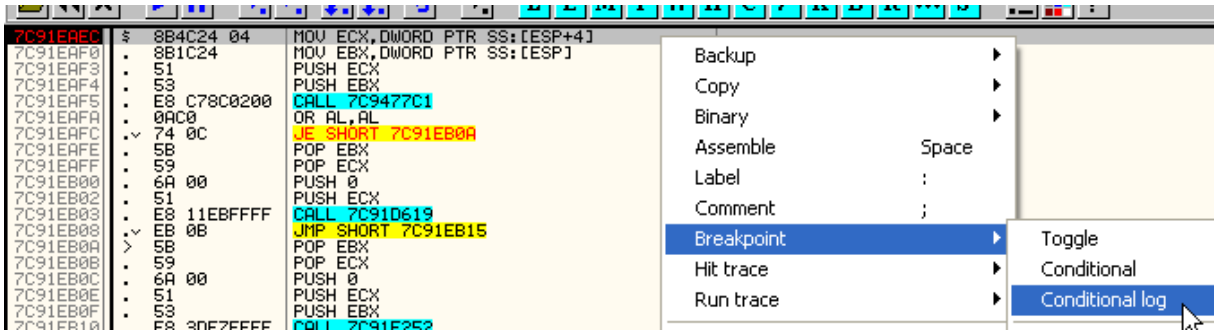
Видим, что несколькими строками ниже находится адрес, где произошло исключение, а точнее в [esp+14].



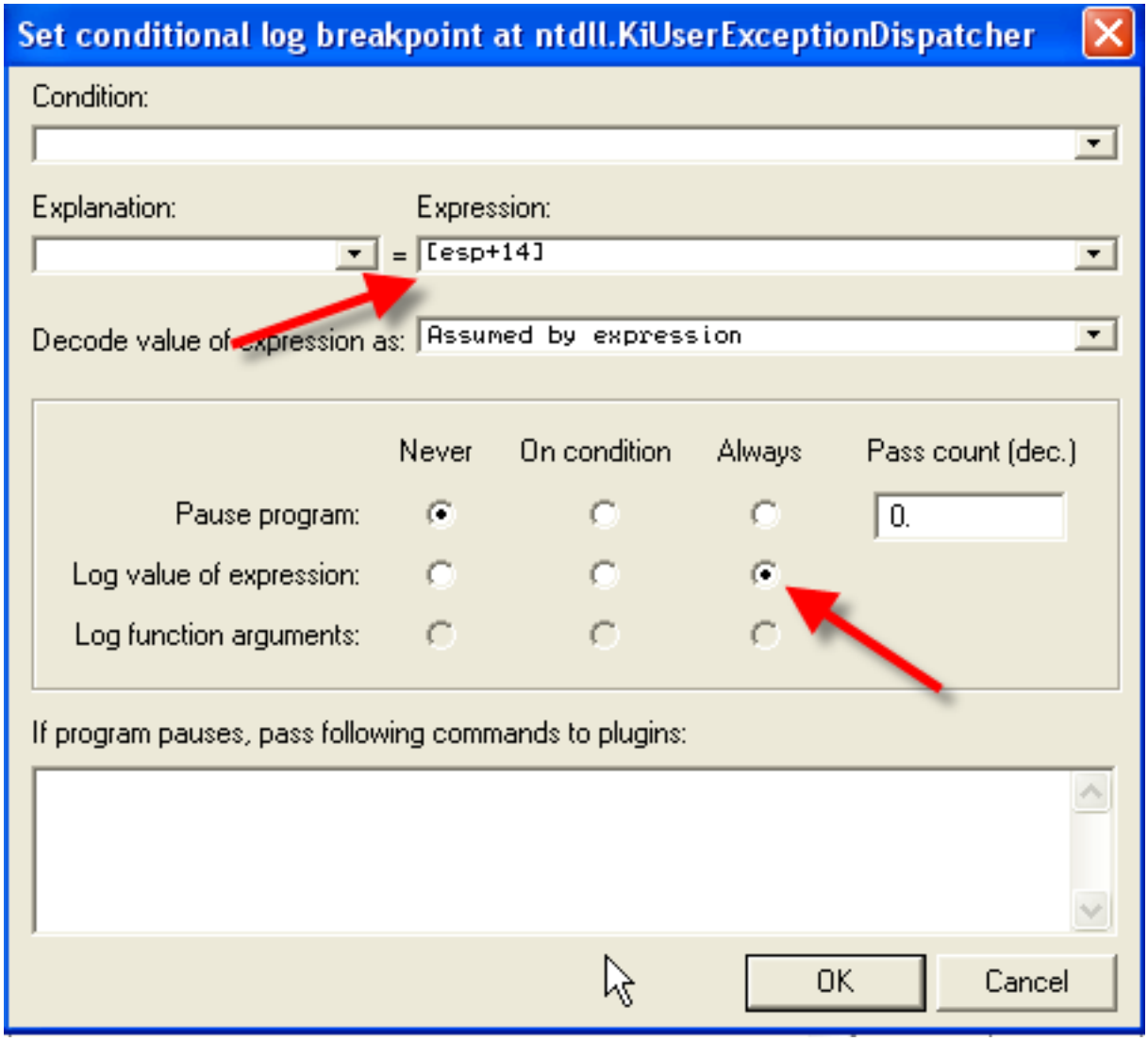
Поэтому если установим BREAKPOINT CONDITIONAL LOG вместо BP и установим, чтобы происходило отслеживание [ESP+14].



Поэтому нажимаем правую кнопку мыши и изменяем установленный нами BP на CONDITIONAL LOG.

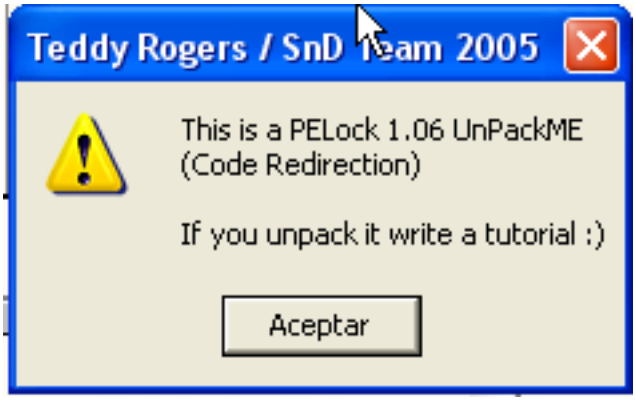


Отмечаем галочкой, чтобы всегда происходило отслеживание значения [esp+14] каждый раз, когда проходим эту точку, и чтобы не происходило остановки, а только запись в логе. Нажимаем RUN.



Видим, что программа запустилась. Странно, но почти ни один упаковщик не обнаруживает обычные или условные BP, установленные на KiUserExceptionDispatcher.

Смотрим, что происходит в логе.



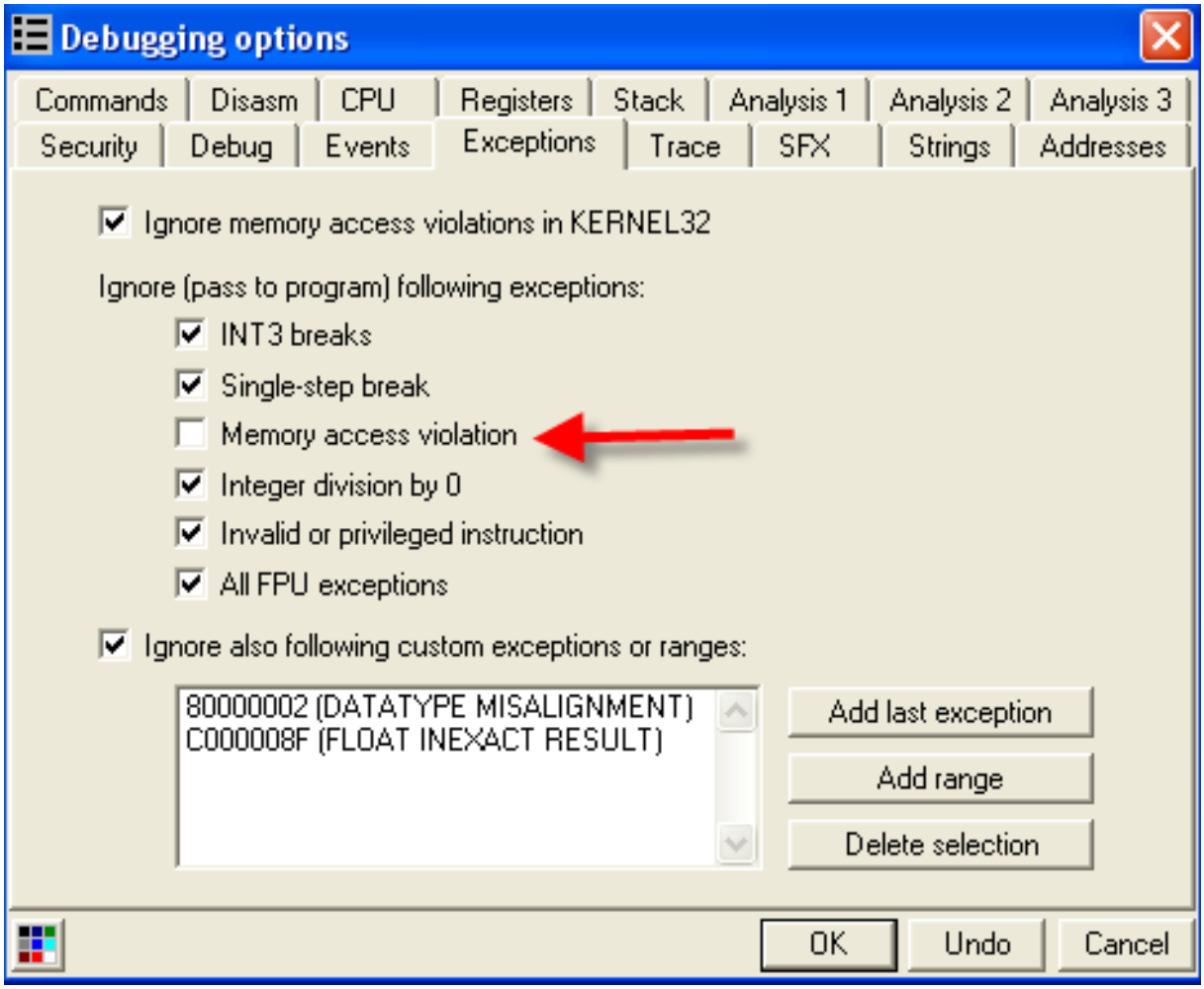
Доходим до сюда, и OllyDbg записывает, что здесь произошло; то, что нас интересует находится немного ниже.

```
Address  | Message
0046B05C | Program entry point
003A5C74 | Access violation when writing to [7C81CAA7]
7C91EAEC | Breakpoint at ntdll.KiUserExceptionDispatcher (KiUserApcDispatcher+2C)
003A1CED | Integer division by zero
7C91EAEC | COND: 003A1CED
003A24EB | Access violation when writing to [00000000]
7C91EAEC | COND: 003A24EB
003A24EE | Integer division by zero
7C91EAEC | COND: 003A24EE
003A2698 | Access violation when writing to [00000000]
7C91EAEC | COND: 003A2698
003A269B | Integer division by zero
7C91EAEC | COND: 003A269B
003A2851 | Access violation when writing to [00000000]
7C91EAEC | COND: 003A2851
003A2854 | Integer division by zero
7C91EAEC | COND: 003A2854
003A3374 | Access violation when reading [FFFFFFFF]
7C91EAEC | COND: 003A3374
003A33CB | Illegal instruction
7C91EAEC | COND: 003A33CB
003A3374 | Access violation when reading [FFFFFFFF]
7C91EAEC | COND: 003A3374
003A33CB | Illegal instruction
7C91EAEC | COND: 003A33CB
003A3374 | Access violation when reading [FFFFFFFF]
7C91EAEC | COND: 003A3374
003A33CB | Illegal instruction
7C91EAEC | COND: 003A33CB
003A3374 | Access violation when reading [FFFFFFFF]
7C91EAEC | COND: 003A3374
003A33CB | Illegal instruction
7C91EAEC | COND: 003A33CB
003A3652 | Access violation when writing to [00000000]
7C91EAEC | COND: 003A3652
003A3655 | Integer division by zero
7C91EAEC | COND: 003A3655
003A37DE | Access violation when writing to [00000000]
7C91EAEC | COND: 003A37DE
003A37E1 | Integer division by zero
7C91EAEC | COND: 003A37E1
003A3873 | Access violation when writing to [00000000]
7C91EAEC | COND: 003A3873
003A3876 | Integer division by zero
7C91EAEC | COND: 003A3876
76B00000 | Module C:\WINDOWS\system32\WINMM.dll
77D10000 | Module C:\WINDOWS\system32\USER32.dll
```

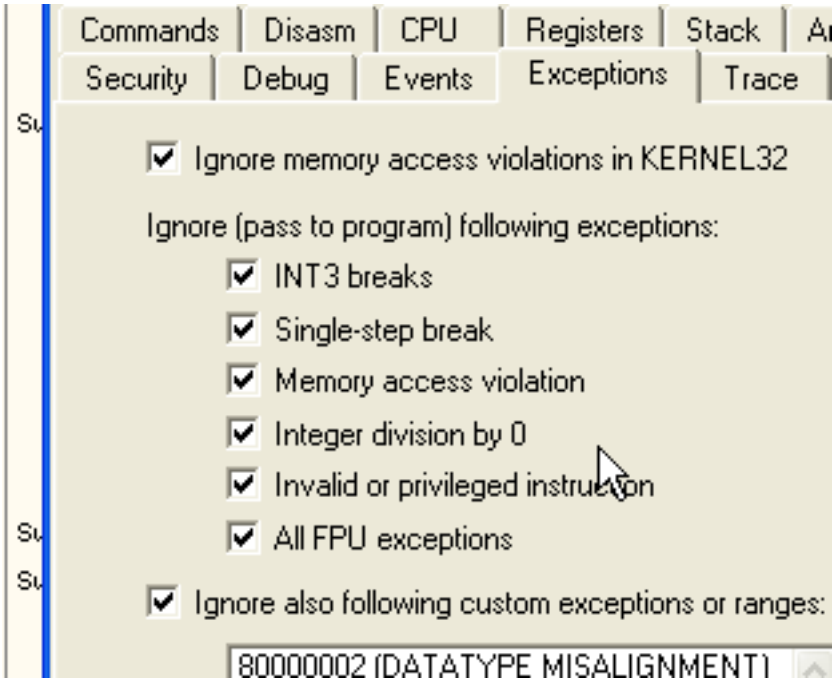
Видим, что последнее исключение – это ACCESS VIOLATION в 3A6744, но здесь мы не можем установить ни BP ни HE, так как они будут обнаружены упаковщиком, и программа не запустится. Смотрим:

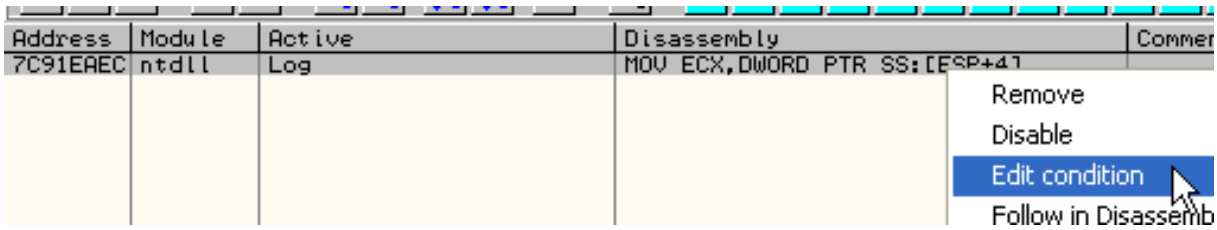
```
774B0000 | Module C:\WINDOWS\system32\ole32.dll
7C81084E | Breakpoint at kernel32.7C81084E
003A6744 | Access violation when writing to [00000000]
7C91EAEC | COND: 003A6744
7C810856 | New thread with ID 00000110 created
770F0000 | Module C:\WINDOWS\system32\OLEAUT32.dll
5B150000 | Module C:\WINDOWS\system32\uxtheme.dll
746B0000 | Module C:\WINDOWS\system32\MSCTF.dll
```

Возможность, которая у нас есть – это дойти до нужно места со снятой галочкой MEMORY ACCESS VIOLATION и останавливаться везде, пока не остановимся в последний раз. Хотя этот метод работает хорошо, лучше бы что-нибудь более типичное и быстрое, и что работало бы для всех типов исключений, так что поставим галочку обратно.

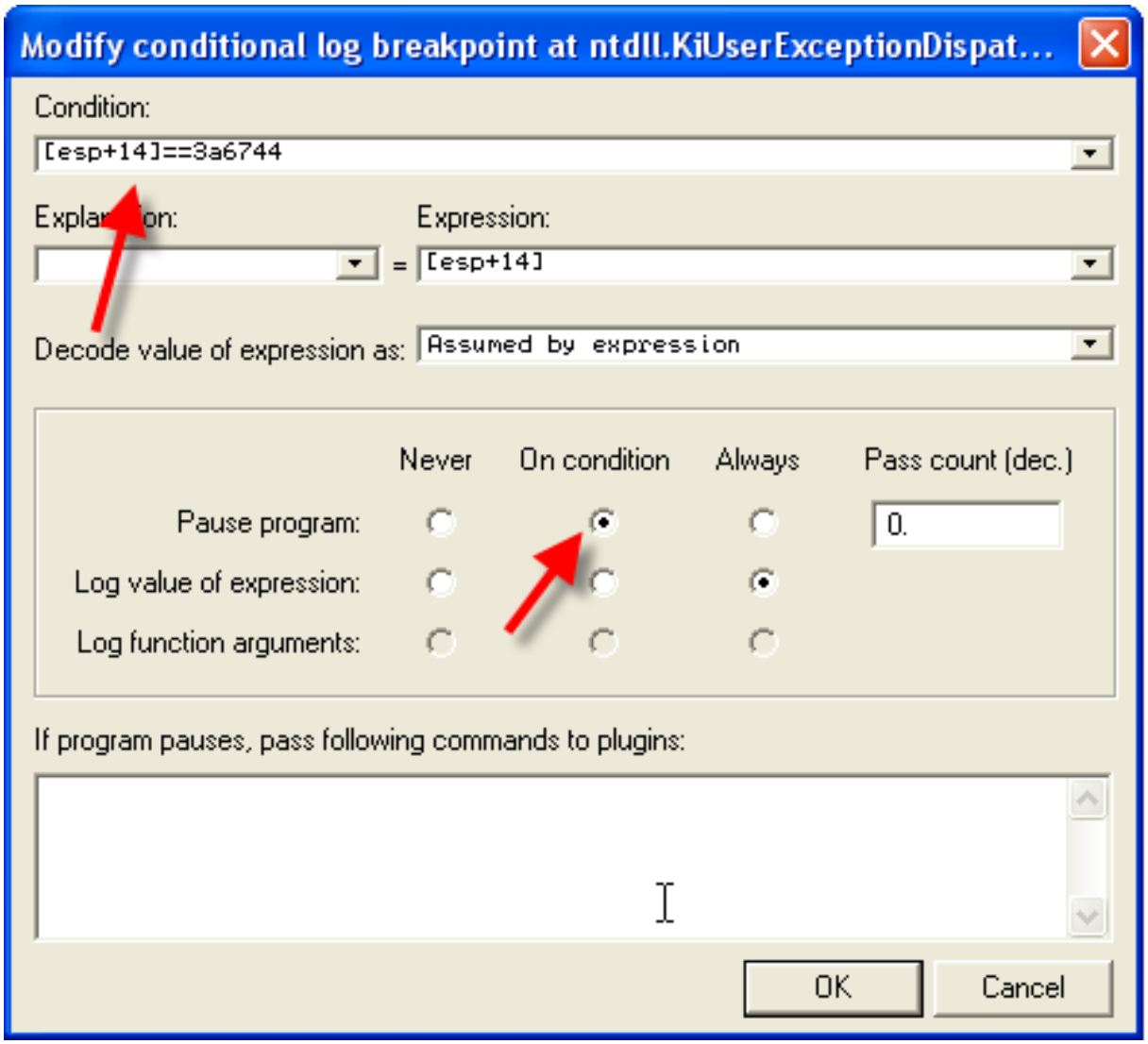


И без лишних страданий идём в B, где находится список точек останова, и даже мой условный BP тоже тут. Нажимаем правую кнопку мыши – EDIT CONDITION.

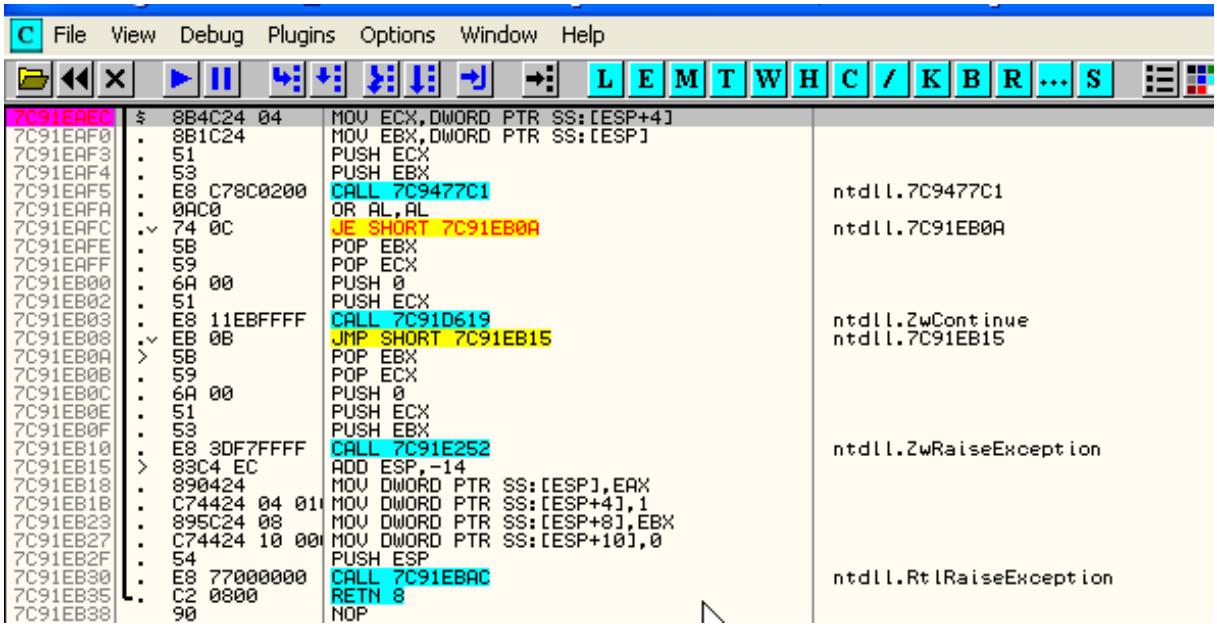




Всего лишь нужно добавить условие для остановки, если мы записываем [esp+14] и в момент, когда там содержится значение 3a7644, чтобы это была инструкция, где генерируется исключение. Тогда нам нужно записать [esp+14]==3a6744.

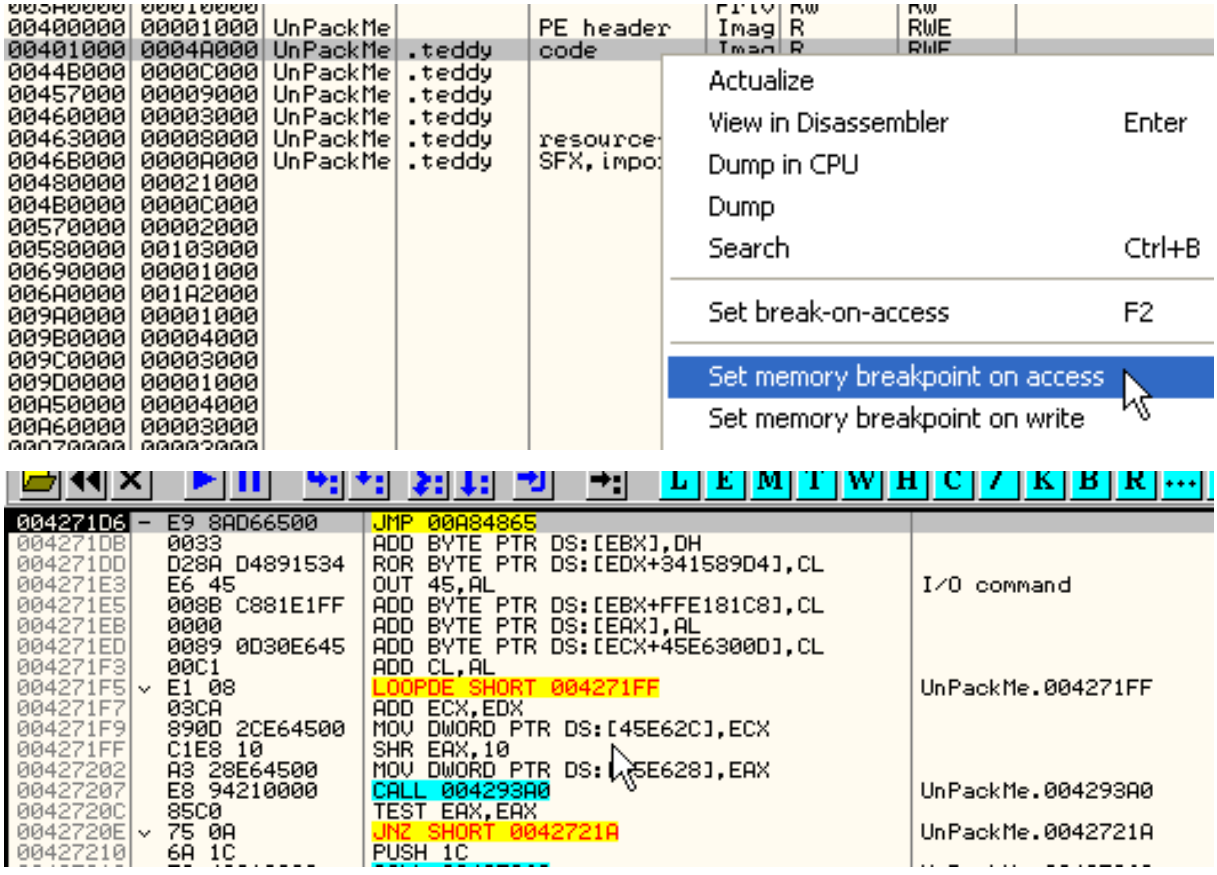


Отмечаем PAUSE PROGRAM-ON CONDITION, чтобы остановка происходила, когда выполняется заданное нами условие. Нажимаем RUN.



Здесь программа останавливается на последнем исключении, и нет нужды останавливаться на каждом из них. Кроме того, если мы захотим вернуться сюда, можем просто перезапустить распакуй-меня, снова нажать RUN и остановиться здесь. Это можно повторять столько раз, сколько нужно без необходимости подсчитывать исключения.

Конечно, если мы отсюда установим BPM ON ACCESS на первой секции, то остановимся на OEP.



Видим, что останавливаемся в первой секции на предполагаемой OEP, но что здесь точно происходит?

Здесь мы начинаем изучать тему о украденных байтах.

УКРАДЕННЫЕ БАЙТЫ – это байты программы, которые выполняются упаковщиком. Так же УКРАДЕННЫЙ КОД – это код программы, выполняемый упаковщиком.

Обычно украденные байты – это первые инструкции исходной программы, идущие с OEP, которые упаковщик стирает из первой секции и запускает в другой, собственной секции, а затем вместо того, чтобы переходить на OEP, переходит, например, на 5 или 6-ую строку, выполнив перед этим предыдущие строки в собственной секции, которая обычно идёт в исполняемом файле последней.

Для чего это делается? Если я сдамлю этот файл, починю IAT и установлю, что OEP – это 4271D6, то программа не запустится, так как отсутствуют первые строки, которые выполняются упаковщиком в собственной секции, поэтому в дампе их нет и они не выполняются

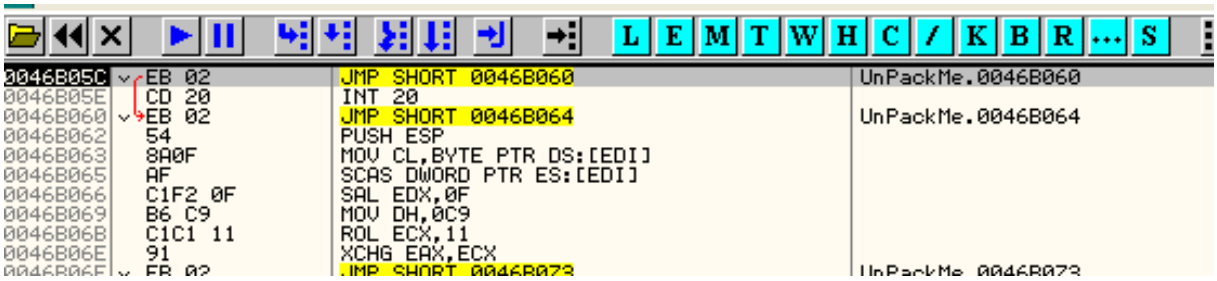
Что нужно делать?

Возможный метод – это попытаться оттрассировать процедуру упаковщика; после последнего исключения и прибытия в ФАЛЬШИВУЮ OEP трассируем и сохраняем в текстовый файл всё, что запускается, чтобы можно было проанализировать, что происходит после перехода на фальшивую OEP в 4271d6.

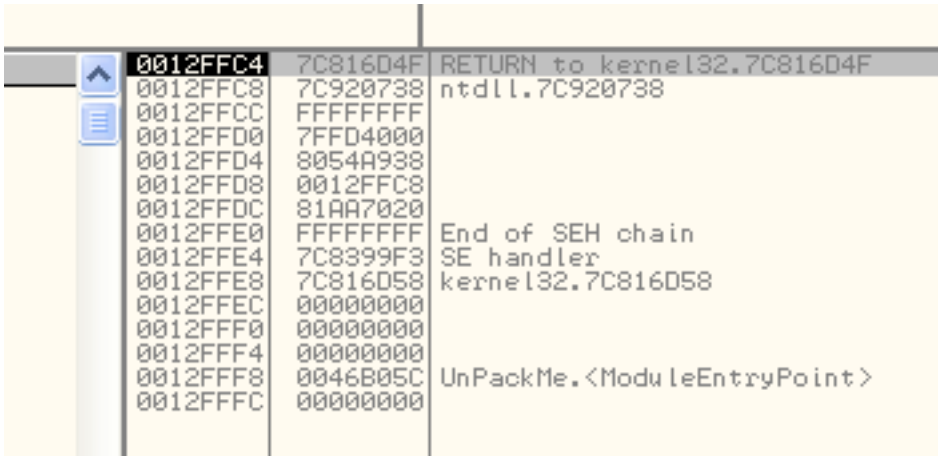
Поэтому мы и используем метод исключений образом, описанным выше, так как во многих случаях при трассировке с помощью OllyDbg есть столько разных возможности, галочек и опций, которые нужно попробовать, что если каждый раз для попадания в последнее исключение нужно пройти через все остальные, то можно сойти с ума, так что нам больше подходит нажатие RUN и остановка в одной, нужной нам точке на последнем исключении.

Перед рестартом программы взглянем на общую картину, что необходимо, если есть подозрение относительно украденных байтов.

Рестартуем программу.

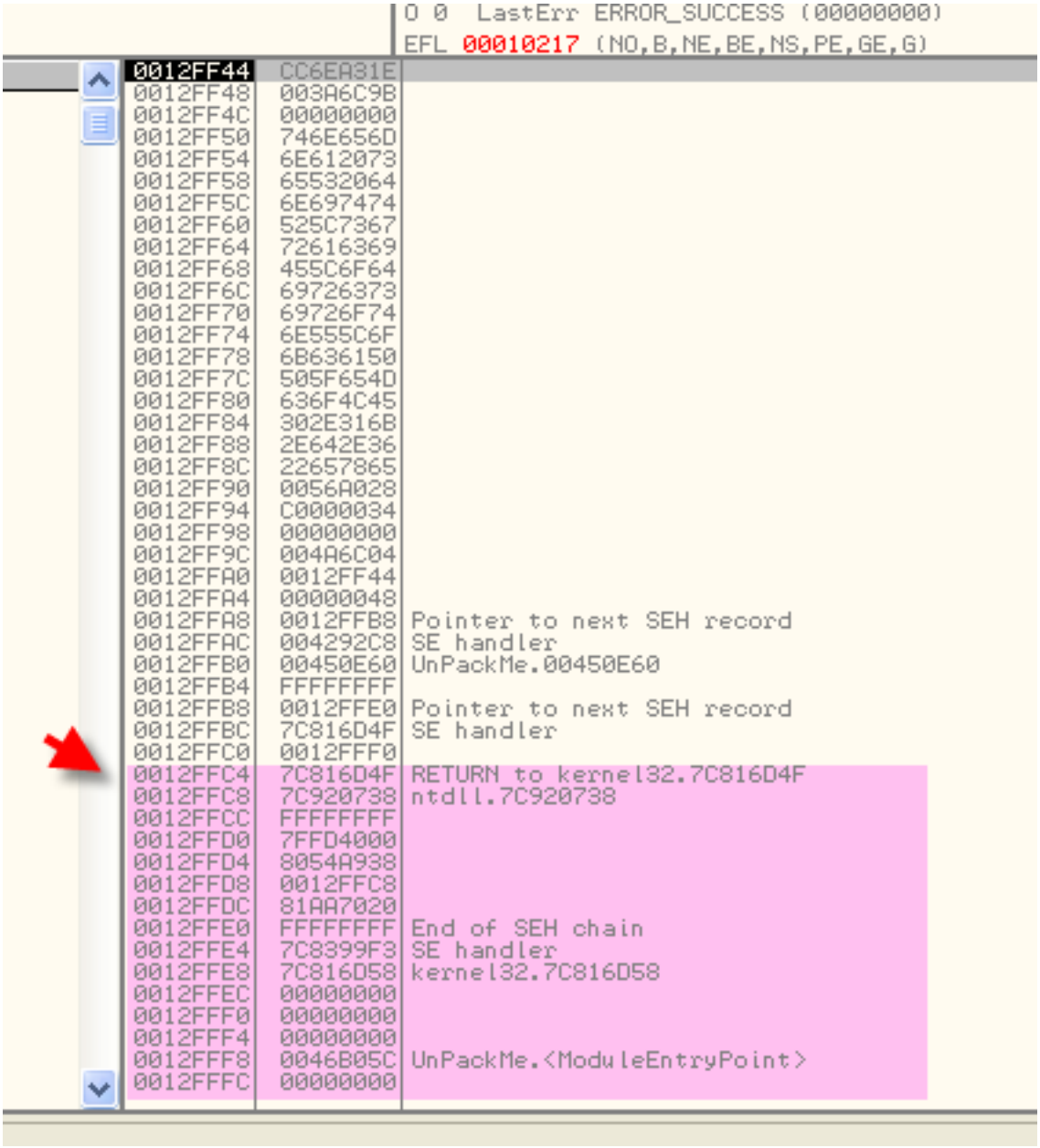


Если посмотрим на начало стека:

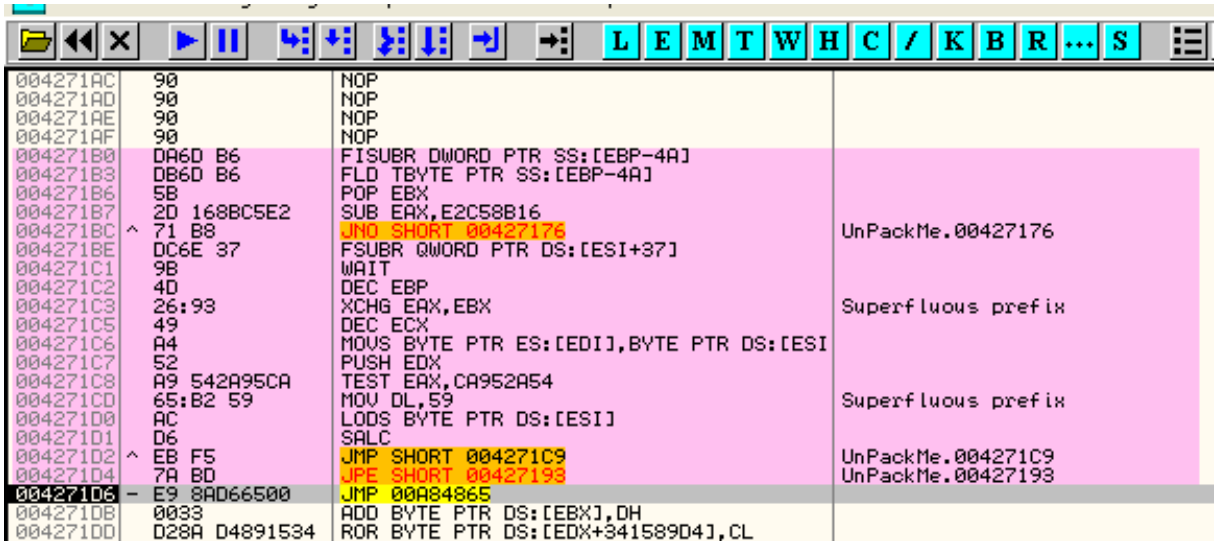


На моей машине это 12FFc4, и каково ни было бы его содержание, в OEP, за исключение очень странных случаев, программа должна начинаться со стеком в 12FFc4 или рядом с ним. Сверху этого адреса находится код, который выполняется из истинной OEP, находящейся до фальшивой.

Снова глянем на фальшивую OEP:

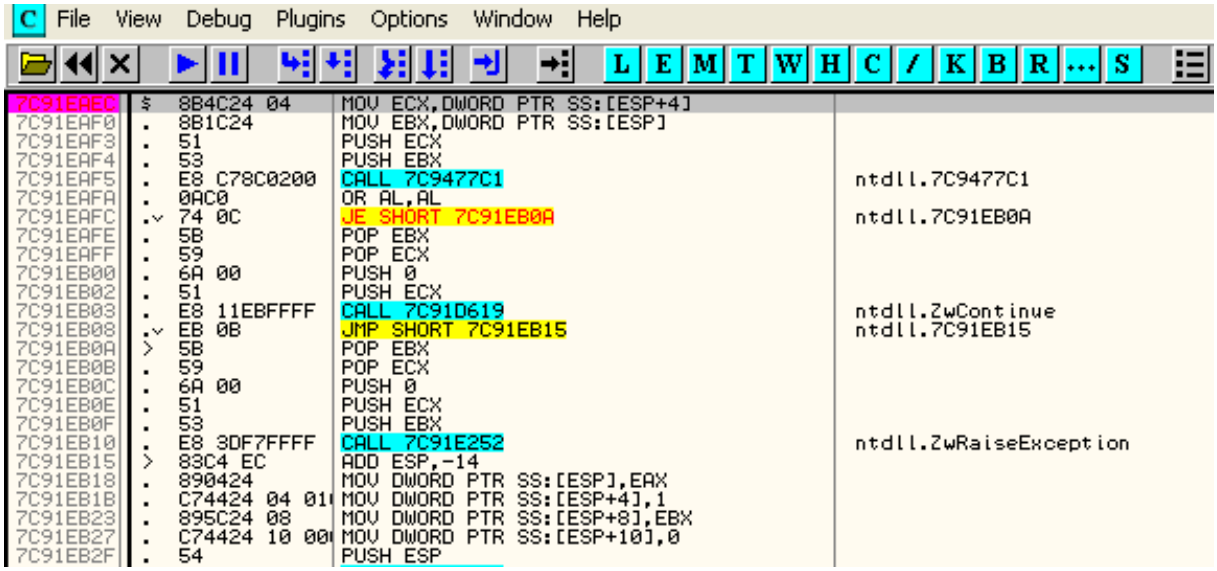


Видим, что если бы это была настоящая OEP, стек должен был бы начинаться в 12FFC4 или рядом. Всё, что есть сверху – это инструкции, выполняющиеся из программы, несколько строк, которые упаковщик убрал из программы и перенёс в другую секцию. Запускаются инструкции, которые должны находиться до фальшивой OEP и которые был стёрты. Если запишем данный крэкми и укажем в качестве OEP 4271b0, то если посмотрим на эту область:

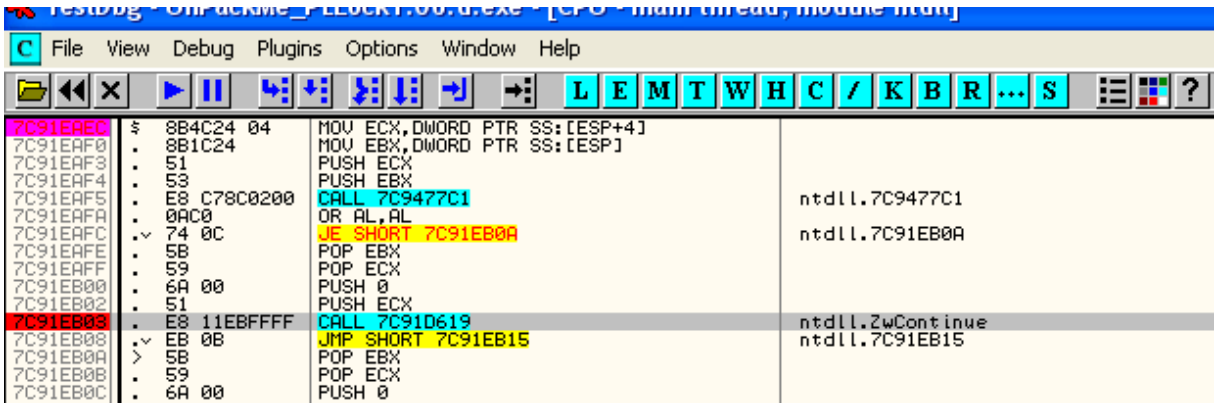


Видим, что упаковщик заменил исходные байты OEP и инструкций мусором, который ещё и не запустится. Если установим BPM ON ACCESS, то остановимся на первой строке исполняемого кода в этой секции, и не останавливается на этих байтах вплоть до 4271d6.

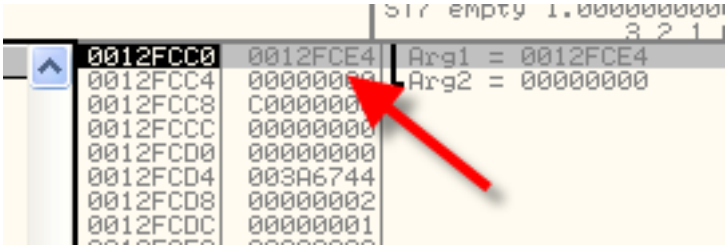
Есть много методов для поиска украденных байтов. Самым «классическим» является возврат из последнего исключения. Попробуем его, перезапустив распакуй-мена и вернувшись в последнее исключение.



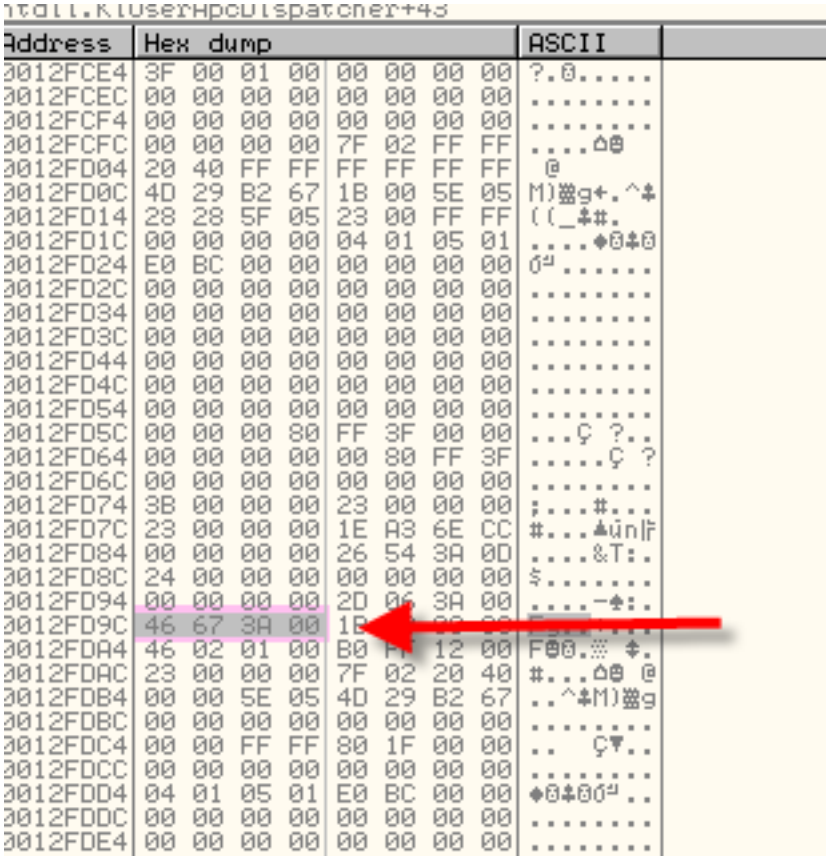
То же самое был сгенерировано в 3A6744, поэтому в данном случае мне не интересен обработчик исключений, так что устанавливаем BP на возврат.



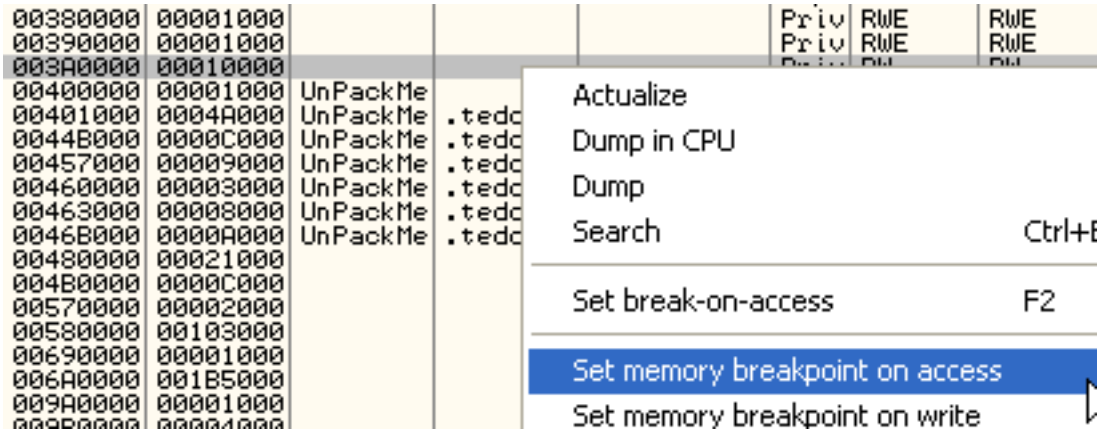
Сейчас нажимаем RUN, если кто-то хочет знать, куда вернётся программа, остановимся для установки туда BP. Посмотрим на стек.



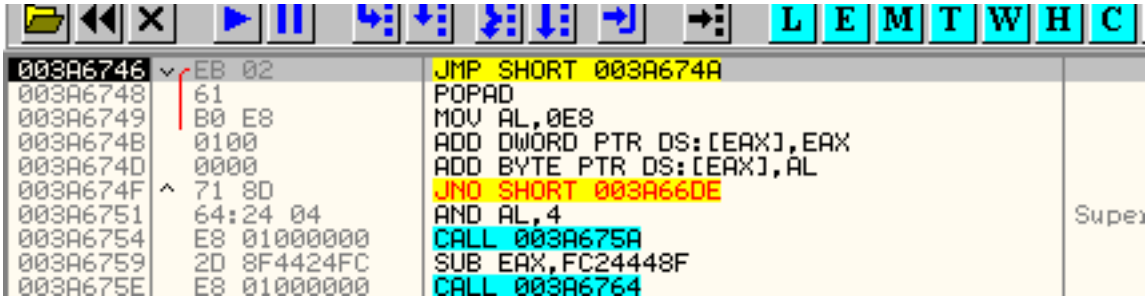
У нас есть параметр, который нам отмечает здесь начало структуры CONTEXT; посмотрим его в DUMP'е.

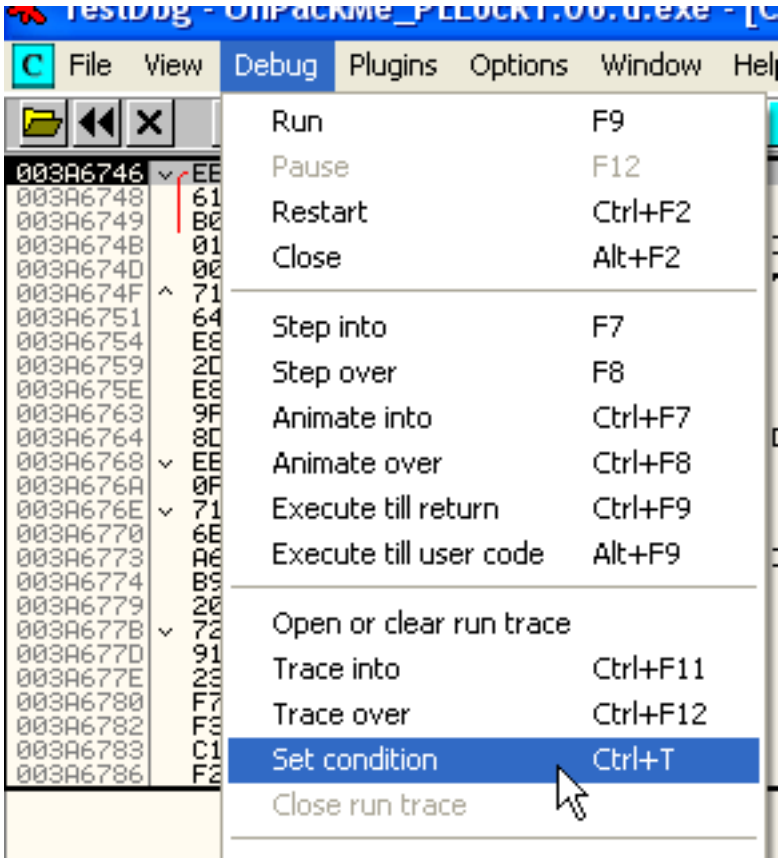


Те, кто уже знаком со структурой CONTEXT, знают, что это её EIP. Также там содержатся значения остальных регистров после возврата программы из исключения. В дальнейшем мы более детально изучим структуру CONTEXT, а пока нам нужно узнать только куда происходит возврат. Так как мы не хотим усложнять себе жизнь поиском в CONTEXT'е, установим BPM ON ACCESS на секцию, где происходит исключение.

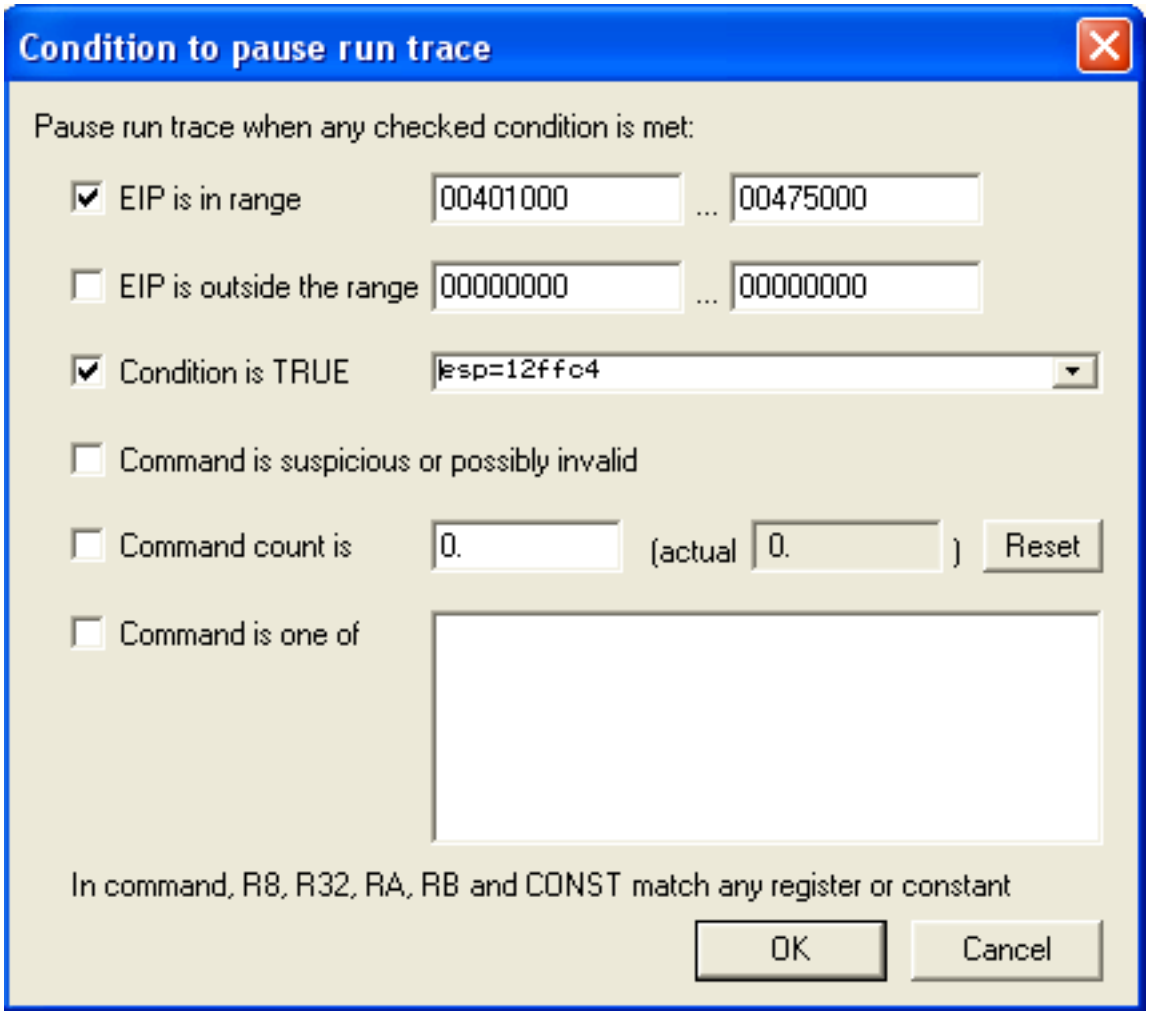


И делаем RUN. Посмотрим, остановится ли он в том же месте:

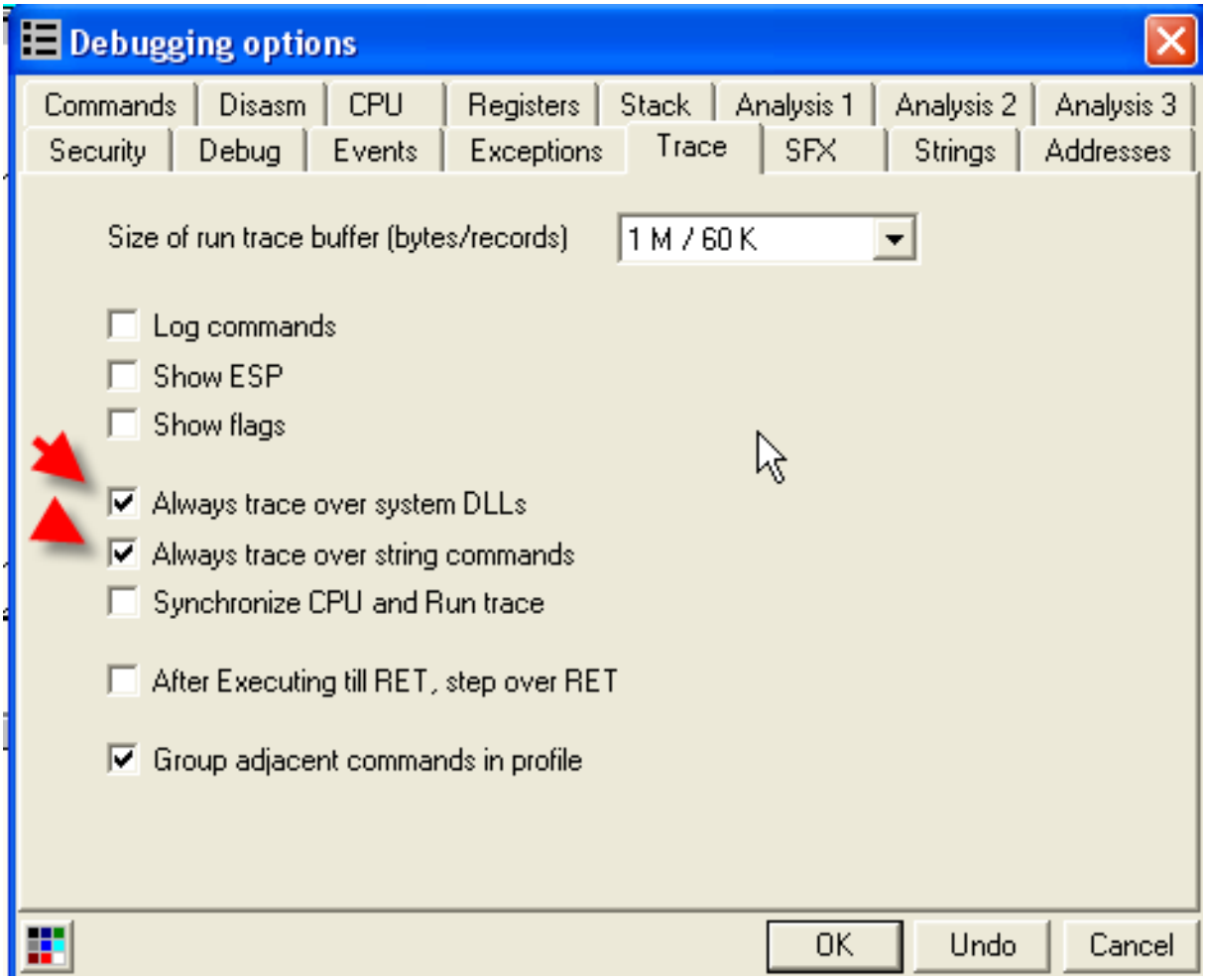




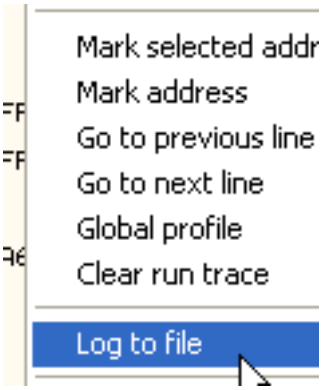
Здесь устанавливаем параметры трассировки, есть множество вариантов, так как можем задать, чтобы проверялись начальные значения регистров, и когда они все будут равны тем, чтобы были до запуска программы, то будем находится в OEP. Посмотрим как этот работает.



Первая останавливает на фальшивом OEP, но с трассировкой, что может позволить мне увидеть в логе инструкции, которые запустились после попадания сюда. Вторая галочка нужна для того, чтобы остановиться в момент попадания на истинную OEP, как мы уже видели, esp в этот момент равнее 12ffc4.

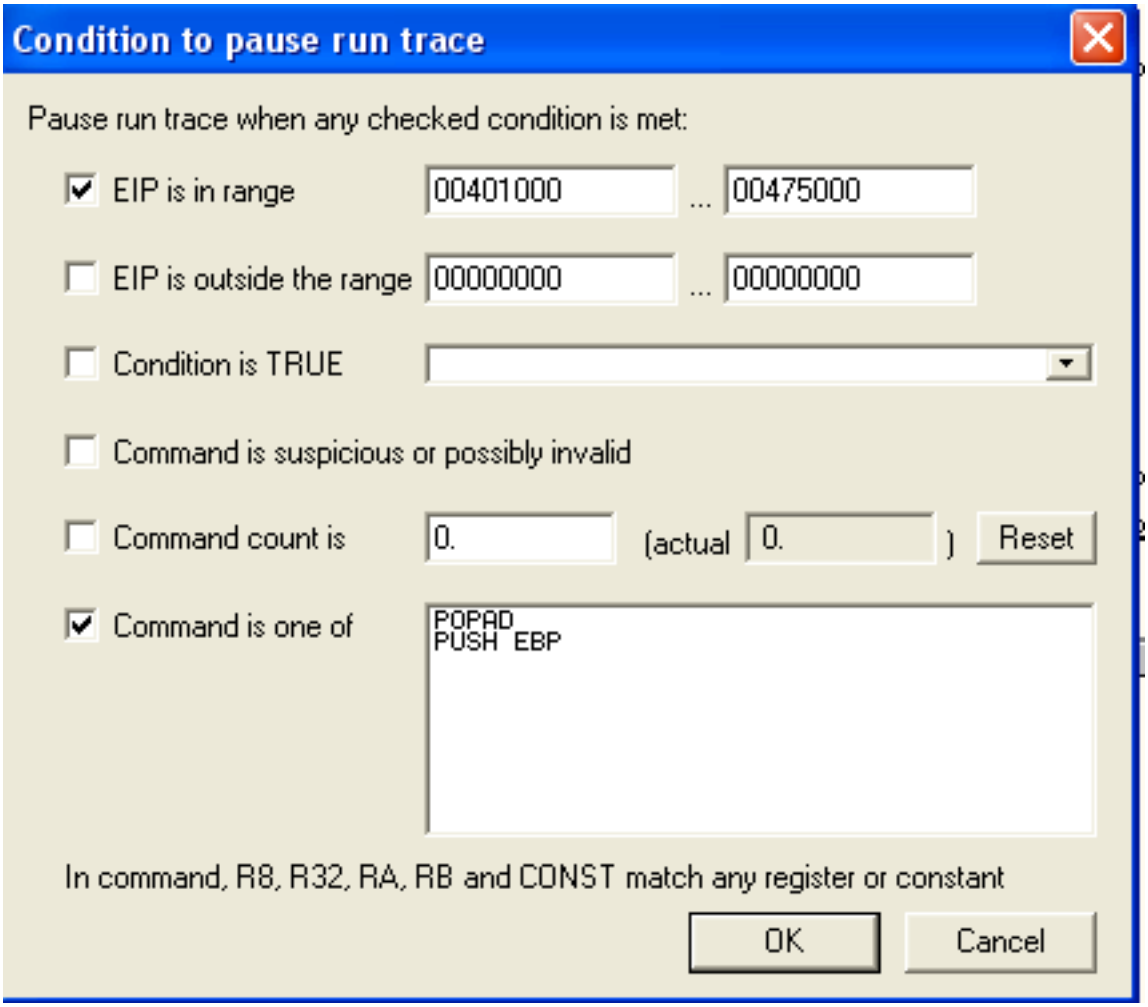


Отмечаем обе галочки в конфигурации, чтобы проверить, будет ли это работать так, как задумывалось, что наиболее вероятно, если нет, то можем выйти.

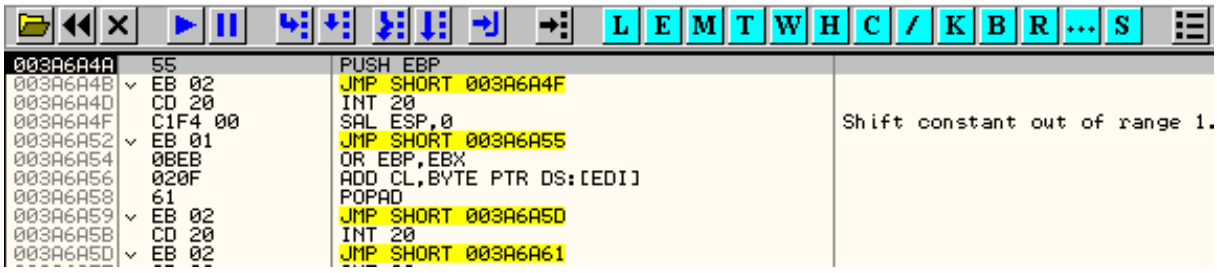


Также указываем, что сообщения в логе записывались в файл.

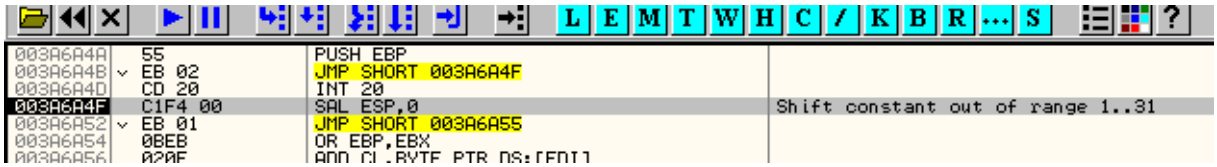
Нажимаем TRACE INTO и выскакивает окошко с условием для ESP, из-за которого приходится останавливаться много раз. Если выйти из него и продолжить делать TRACE INTO, то остановимся на фальшивой OEP, а в текстовом файле будут находиться все инструкции, которые запустились до этого. К сожалению, остановки на истинной OEP не произошло, не повезло.



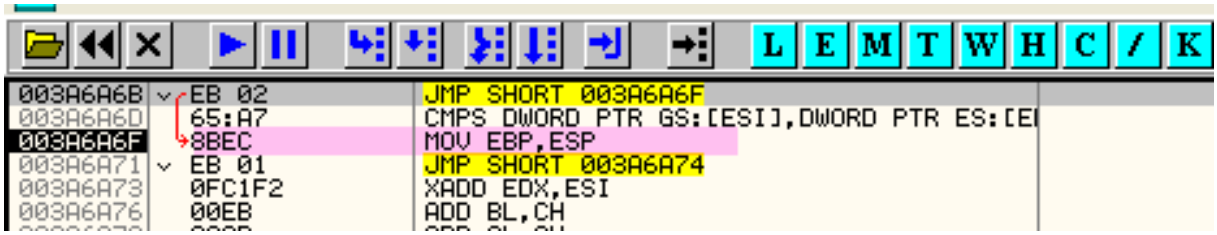
Одна из вещей, которую вы можете попытаться сделать, это установить, чтобы остановка происходила на каком-нибудь PUSH EBP или POPAD, которые зачастую или находятся непосредственно в OEP в одном случае, или восстанавливают регистры в другом. Это может сработать, а может и нет, так как зачастую PUSH EBP эмулируется, но ладно, посмотрим.



Это может быть истинный OEP, продолжаем трассировать.

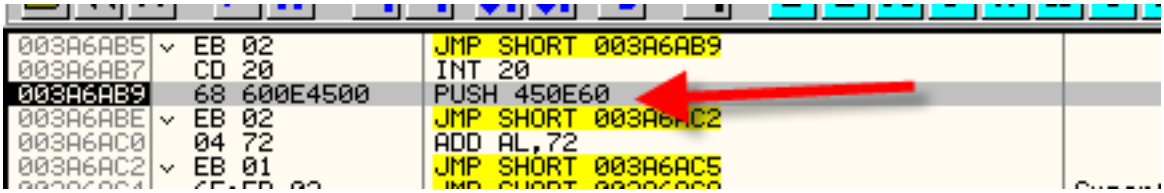


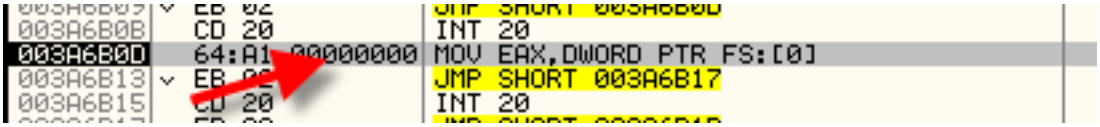
Конечно, настоящие инструкции не должны содержать переходы или SAL или другие мусорные инструкции.

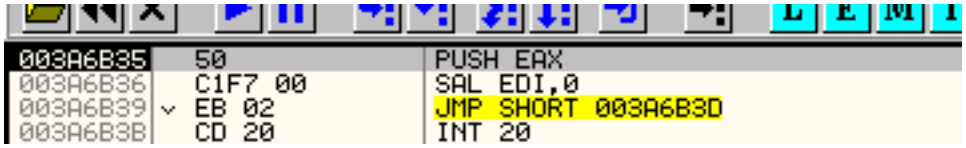


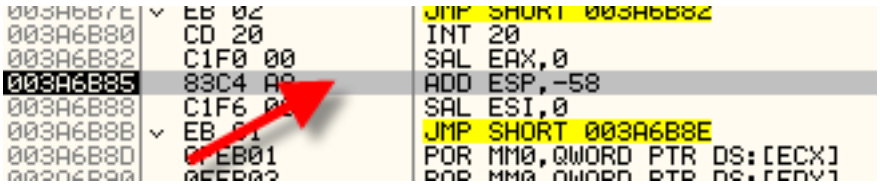
Похоже на вторую недостающую инструкцию. Они не эмулируются, а копируются напрямую, поэтому их легко обнаружить.

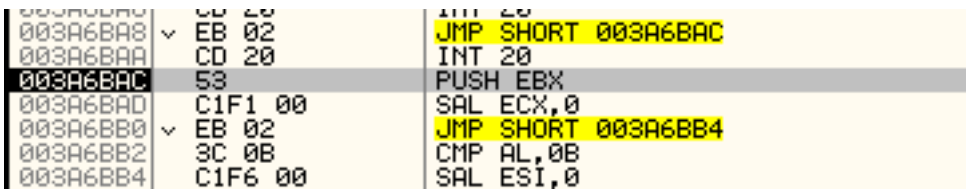
И так продолжаем трассировать и копировать хорошие инструкции, которые будут запущены до попадания в фальшивую OEP.

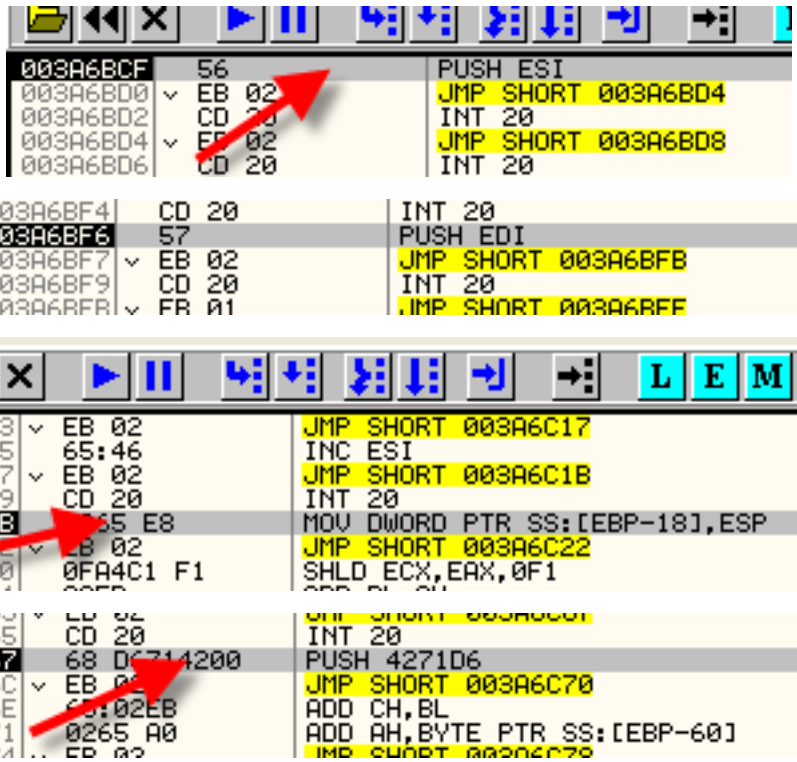




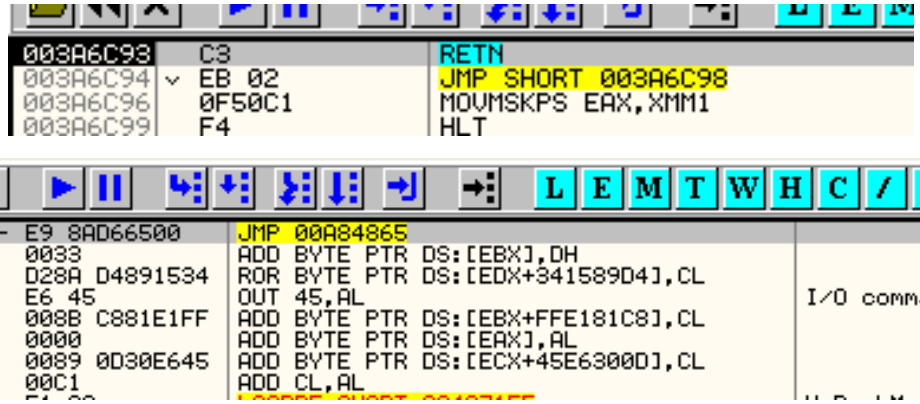




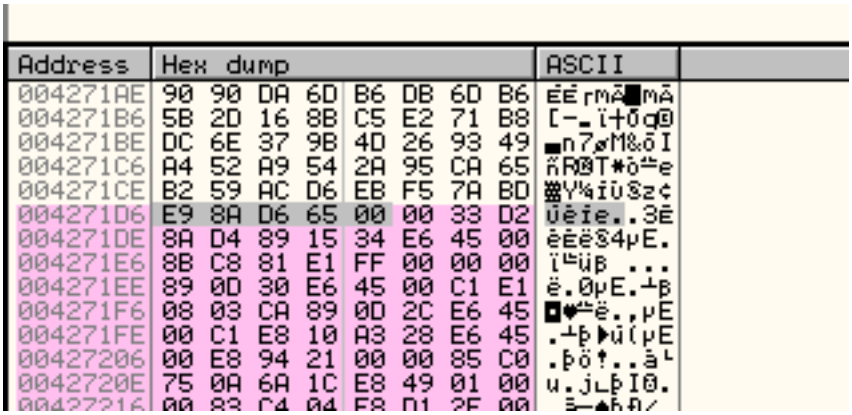




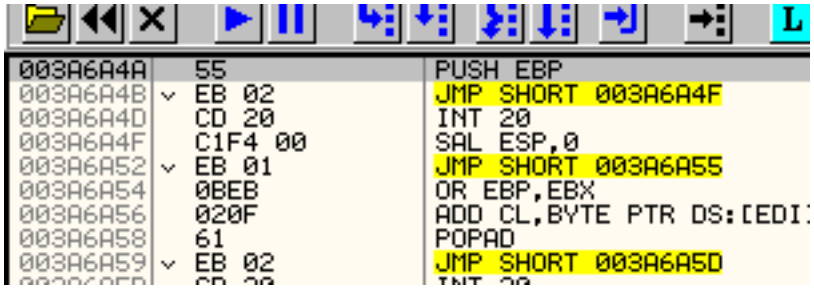
Здесь мы доходим до фальшивой OEP, так как делается PUSH 4271D6, а это адрес фальшивой OEP, а после идёт RET, так что мы перейдём туда.



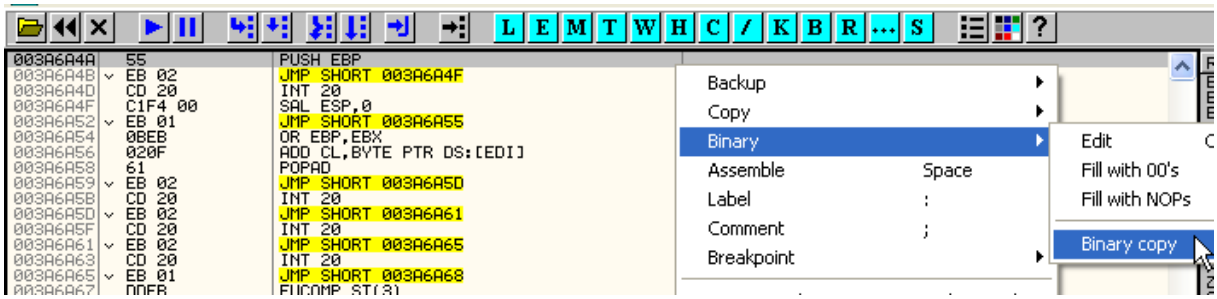
Конечно, сейчас должны быть скопированы все украденные инструкции и помещены до фальшивой OEP. Смотрим в DUMP'е.

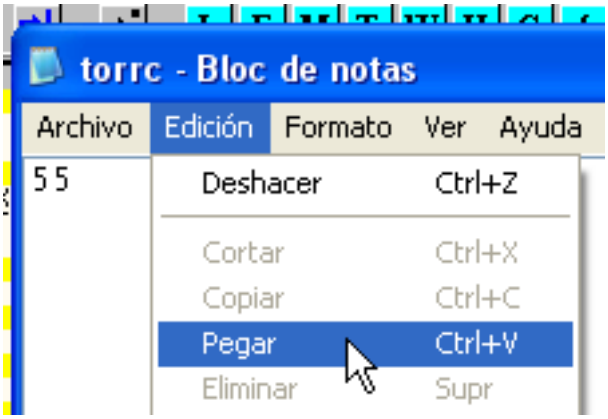


Здесь видим область фальшивой OEP, наверху находится место, куда должны попасть украденные байты. Мы уже проверили, что они все верны, и останавливаем немногим после истинной OEP.



И теперь-то делаем двоичную копию каждой строки, которая выполнилась, и копируем в нотепад байты кроме последнего PUSH и RET, так как это переход на фальшивую OEP, а не украденные байты.





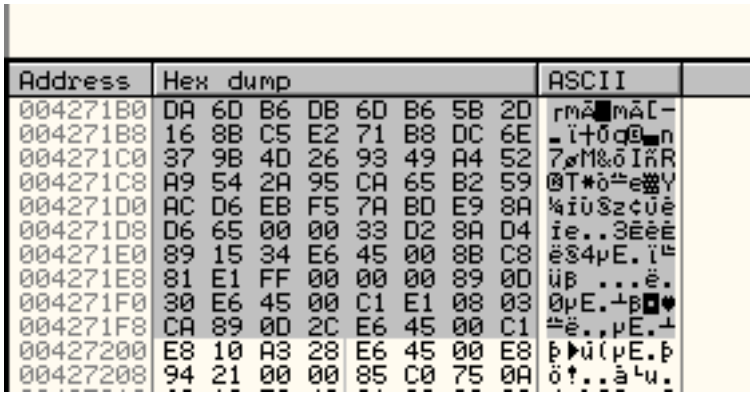
55 8B EC 6A FF 68 60 0E 45 00 68 C8 92 42 00 64 A1 00 00 00 00 50 64 89 25 00 00 00 00 83 C4 A8 53 56 57 89 65 E8

Это 38 украденных байтов, что в шестнадцатеричной системе равно 26-ти.

Так что отнимаем от адреса фальшивой OEP эти 26 байтов.

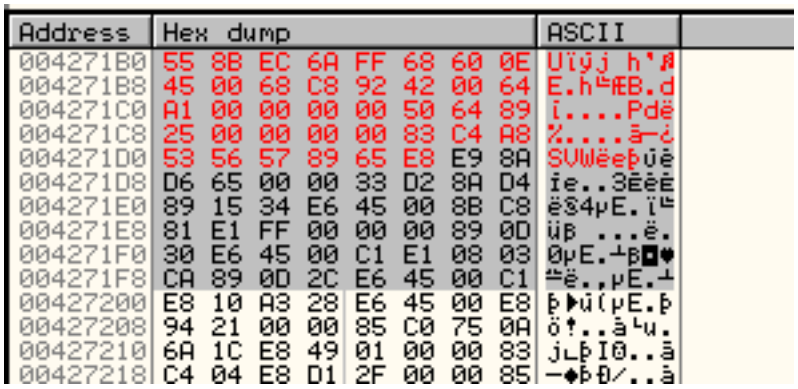
Теперь находим адрес, где должна была бы находиться истинная OEP.

То есть вместо 4271b0 нужно вставить найденные байты.



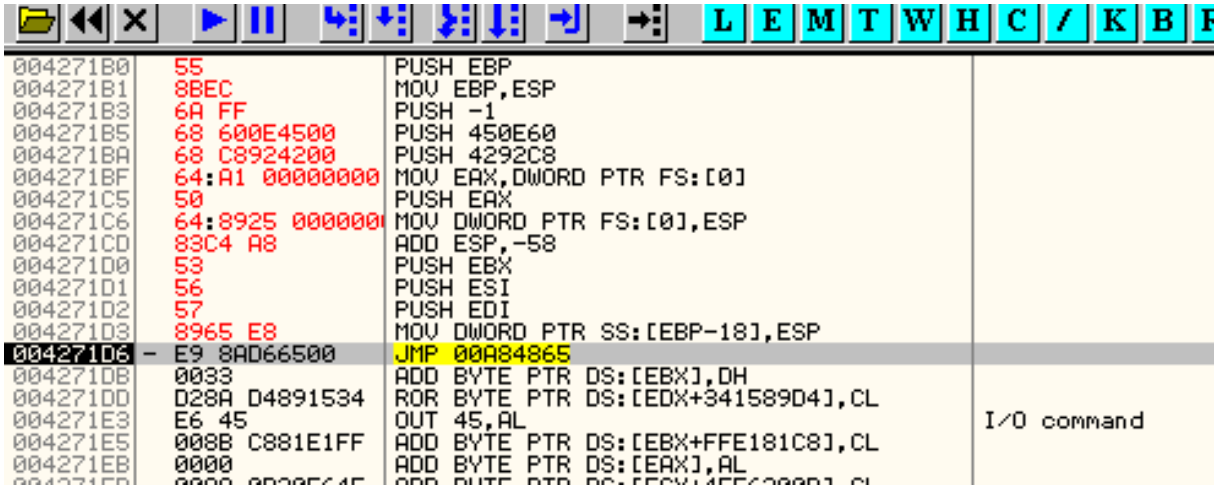
Address	Hex dump	ASCII
004271B0	DA 6D B6 DB 6D B6 5B 2D	rmA▀mA[-
004271B8	16 8B C5 E2 71 B8 DC 6E	_ï+ôq@▄n
004271C0	37 9B 4D 26 93 49 A4 52	7øM&ôIñR
004271C8	A9 54 2A 95 CA 65 B2 59	®T*ò╩e▒Y
004271D0	AC D6 EB F5 7A BD E9 8A	¼ïûßz¢ûè
004271D8	D6 65 00 00 33 D2 8A D4	ie..3Ëèë
004271E0	89 15 34 E6 45 00 8B C8	ë§4µE.ï╩
004271E8	81 E1 FF 00 00 00 89 0D	üß ...ë.
004271F0	30 E6 45 00 C1 E1 08 03	0µE.┴ß◘♥
004271F8	CA 89 0D 2C E6 45 00 C1	╩ë.,µE.┴
00427200	E8 10 A3 28 E6 45 00 E8	þ▶ú(µE.þ
00427208	94 21 00 00 85 C0 75 0A	ö!..à└u.

Отмечаем байты в нотепаде, копируем их в буфер обмена, а потом в OllyDbg делаем правой щелчок мыши и выбираем BINARY PASTE.



Address	Hex dump	ASCII
004271B0	55 8B EC 6A FF 68 60 0E	Uïÿj h`♯
004271B8	45 00 68 C8 92 42 00 64	E.h╚ÆB.d
004271C0	A1 00 00 00 00 50 64 89	í....Pdë
004271C8	25 00 00 00 00 83 C4 A8	%....ã─¿
004271D0	53 56 57 89 65 E8 E9 8A	SVWëeþûè
004271D8	D6 65 00 00 33 D2 8A D4	ie..3Ëèë
004271E0	89 15 34 E6 45 00 8B C8	ë§4µE.ï╩
004271E8	81 E1 FF 00 00 00 89 0D	üß ...ë.
004271F0	30 E6 45 00 C1 E1 08 03	0µE.┴ß◘♥
004271F8	CA 89 0D 2C E6 45 00 C1	╩ë.,µE.┴
00427200	E8 10 A3 28 E6 45 00 E8	þ▶ú(µE.þ
00427208	94 21 00 00 85 C0 75 0A	ö!..à└u.
00427210	6A 1C E8 49 01 00 00 83	j∟þI0..â
00427218	C4 04 E8 D1 2F 00 00 85	─♦þÐ/..à

После это смотрим, что стало с кодом.



```
004271B0    55              PUSH EBP
004271B1    8BEC            MOV EBP,ESP
004271B3    6A FF           PUSH -1
004271B5    68 600E4500     PUSH 450E60
004271BA    68 C8924200     PUSH 4292C8
004271BF    64:A1 00000000  MOV EAX,DWORD PTR FS:[0]
004271C5    50              PUSH EAX
004271C6    64:8925 000000  MOV DWORD PTR FS:[0],ESP
004271CD    83C4 A8         ADD ESP,-58
004271D0    53              PUSH EBX
004271D1    56              PUSH ESI
004271D2    57              PUSH EDI
004271D3    8965 E8         MOV DWORD PTR SS:[EBP-18],ESP
004271D6  - E9 8AD66500     JMP 00A84865
004271DB    0033            ADD BYTE PTR DS:[EBX],DH
004271DD    D28A D4891534   ROR BYTE PTR DS:[EDX+341589D4],CL
004271E3    E6 45           OUT 45,AL                        I/O command
004271E5    008B C881E1FF   ADD BYTE PTR DS:[EBX+FFE181C8],CL
004271EB    0000            ADD BYTE PTR DS:[EAX],AL
```

Стало лучше, можем сдампить и поменять OEP на 4271B0, так как мы решили проблему украденных байтов.

В следующей части мы рассмотрим скрипты, которые необходимы для починки IAT и остальное, что необходимо для победы над этим упаковщиком.

Файлы к статье (находится в архиве `wasm_files.zip`: `files/0509/ollydbg39.zip`)

Введение в крэкинг с нуля, используя OllyDbg - Глава 40

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2009-03-25

После того, как мы продолжили разбираться с таблицой IAT pelock'a, а также антидамповыми приёмами, в этой части будет рассмотрено две темы: одна, которая была пропущена ранее, и другая, которая очень поможет в работе с tpelock'ом или другой похожей программой, которая снимает или обнаруживает установленные HBP.

Первая тема была затронута в главе 39, но мы не уделили ей никакого внимания. Если вы скачивали файл, прилагаемый к предыдущей главе, то там был файл leer.txt, в котором было сказано:

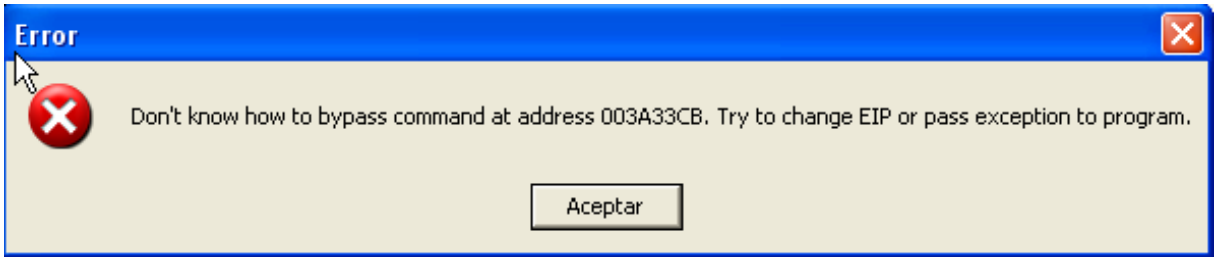
Для того, чтобы решить данный туториал, нужно использовать пропатченный ollydbg (в архиве Parcheado 4.rar). Необходимо разместить его в той же папке, что и Ollydbg.exe, так как у обычного Ollydbg есть баг обработки неправильных инструкций, которые выбрасывают окошко с выбором «да или да» для установки галочки на debugging options-exceptions для выхода. «Parcheado 4» чинит этот баг. На тот момент, когда я писал главу, я забыл об этом, но Solid мне напомнил, что нельзя решить туториал из-за этой проблемы, так что используйте фикс, и не будет никакой проблемы.

Рикардо Нарваха

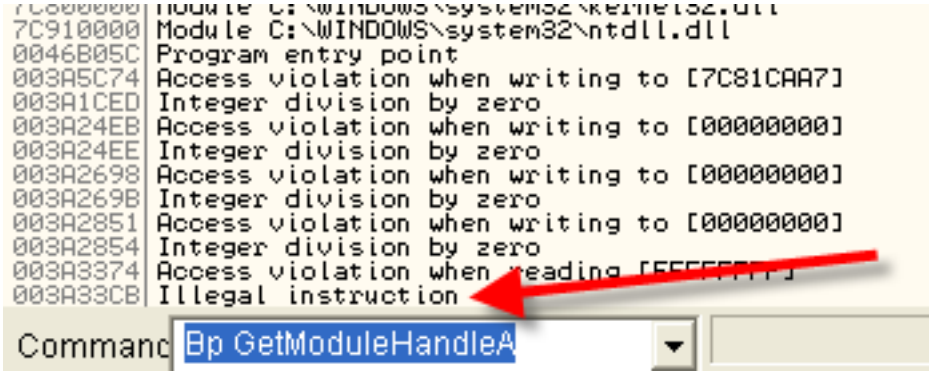
Пробуем пропатченный Ollydbg под названием «Parcheado 4», который находится в той же папке, что и Ollydbg.exe, и используем для решения туториал по PELOCK'у.

Что я хотел объяснить, так это главную вещь, которая делает этот «parcheado 4» особенным.

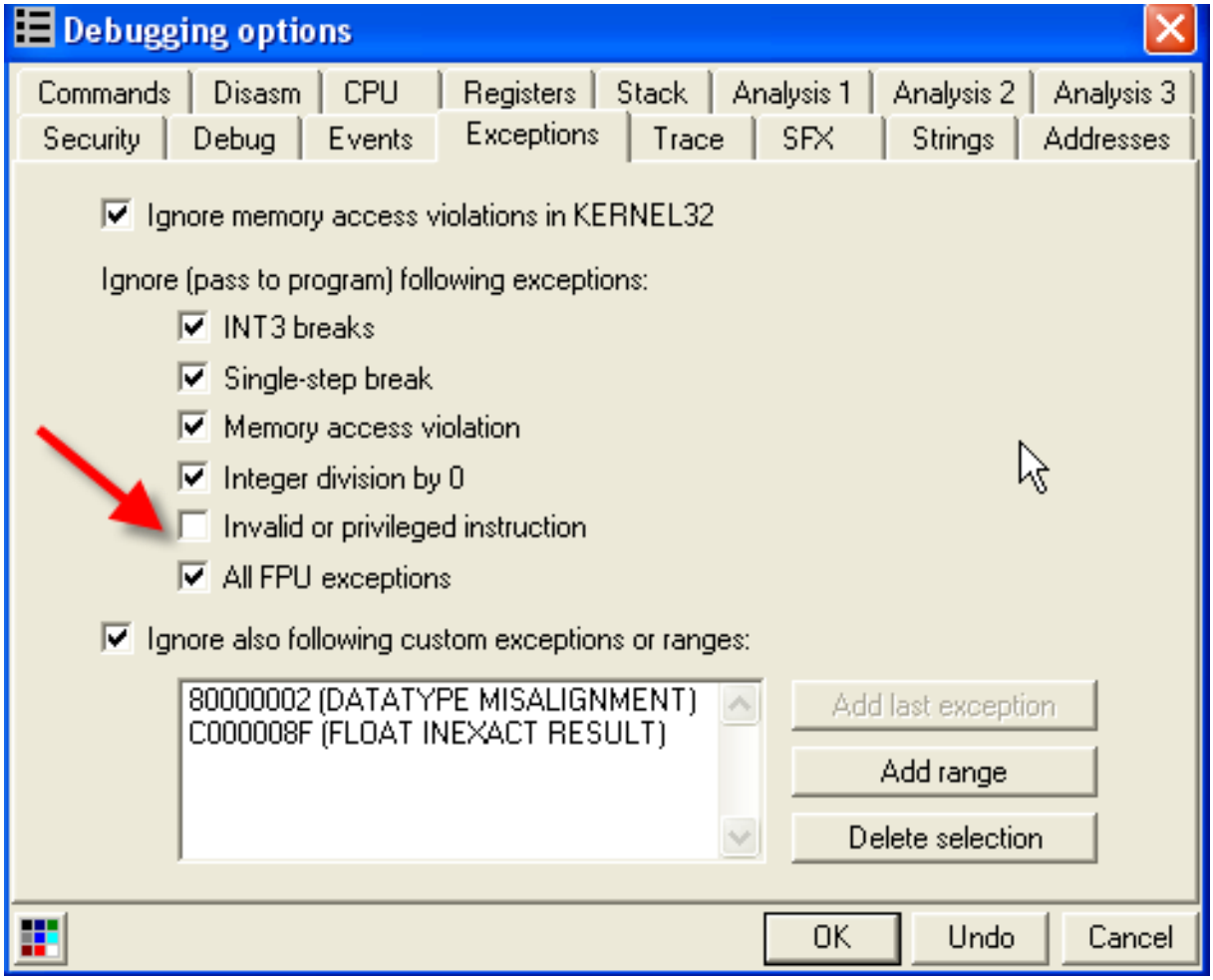
Хорошо, в Ollydbg есть баг обработки ислючений типа «ILLEGAL INSTRUCTION». Если в обычном Ollydbg с плагинами для его сокрытия отметим все галочки в DEBUGGING OPTIONS-EXCEPTIONS и нажмём RUN:



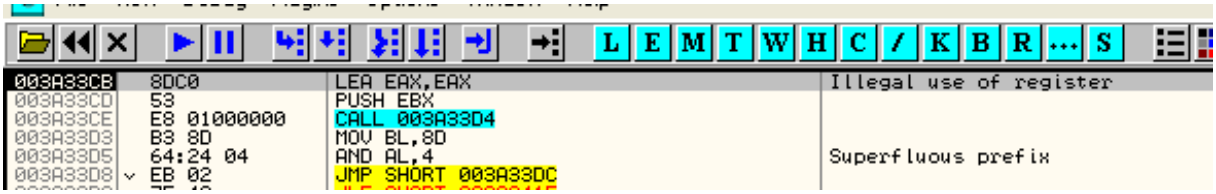
Получаем данную ошибку, в то время как в «parcheado 4» выполняется превосходно. Нажимаем на «Aceptar» и смотрим, что случилось в логе Ollydbg.exe.



Видим, что последнее исключение, до которого появляется окошко - это ILLEGAL INSTRUCTION. Сейчас посмотрим debugging options-exceptions, и уберём галочку с данного типа исключений.



Это и есть галочка, отвечающая за данное исключение, снимаем её, запускаем pelock и делаем RUN.

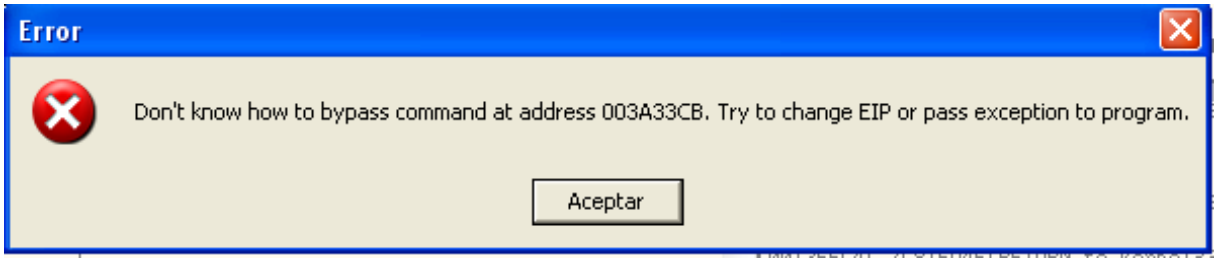


Останавливаемся на исключении, и если пройдём его с помощью SHIFT+F9, то не будет никакой проблемы и pelock запустится.

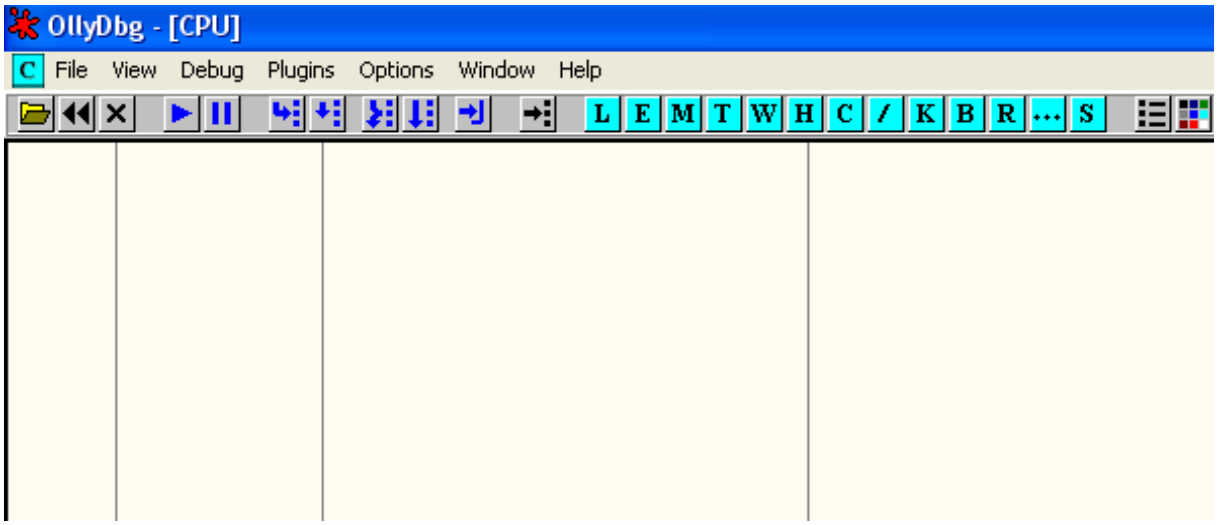
Это значит, что у Ollydbg проблема с прохождением данного исключения в автоматическом режиме.

Если просто нажать SHIFT+F9, то всё будет в порядке, но этот баг может стать проблемой, так как многие программисты генерируют 200 или 300 таких исключений, и их прохождение в ручном режиме по меньшей мере трудновыносимо.

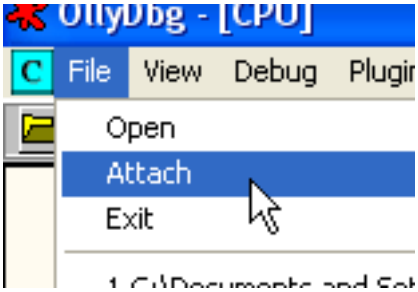
Теперь возвращаемся, чтобы установить галочку и снова получаем сообщением об ошибке.

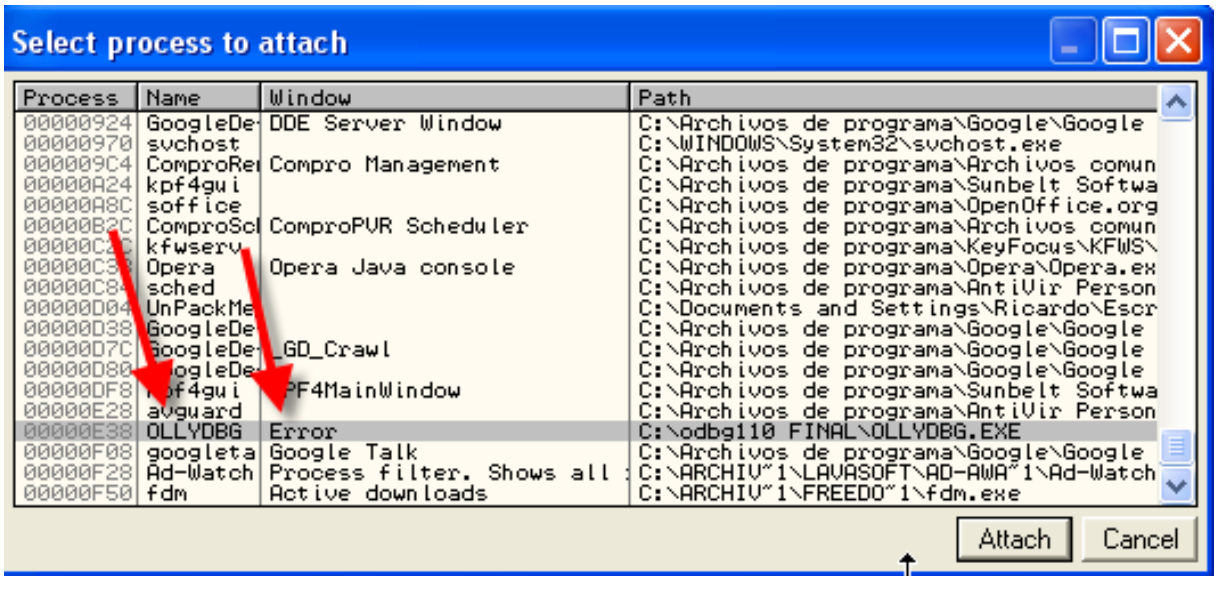


Как самые модные крэкеры, мы может попытаться крэкнуть сам Ollydbg, чтобы починить баг, для чего откроем второй пустой Ollydbg, с помощью которого и крэкнем остановку на сообщении с ошибкой.

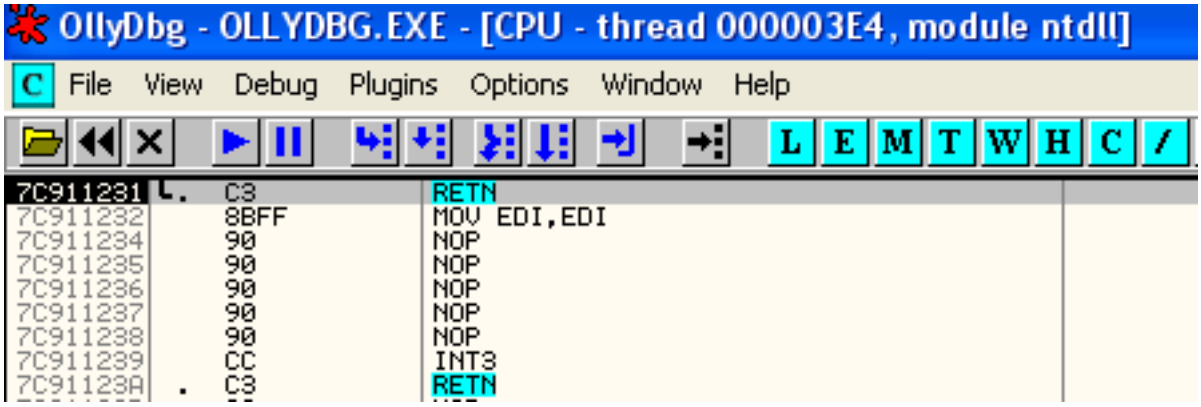


В этом пустом Ollydbg выполним FILE-ATTACH, чтобы прикрепить и отладить другой Ollydbg, который остановлен на сообщении с ошибкой.





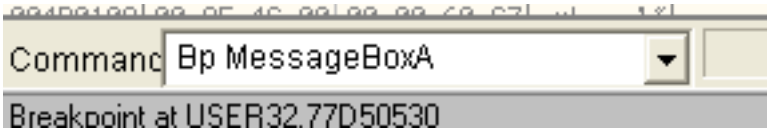
В открывшемся окне ищем процесс Ollydbg, чьё окно имеет название «Error», и прикрепляем его.

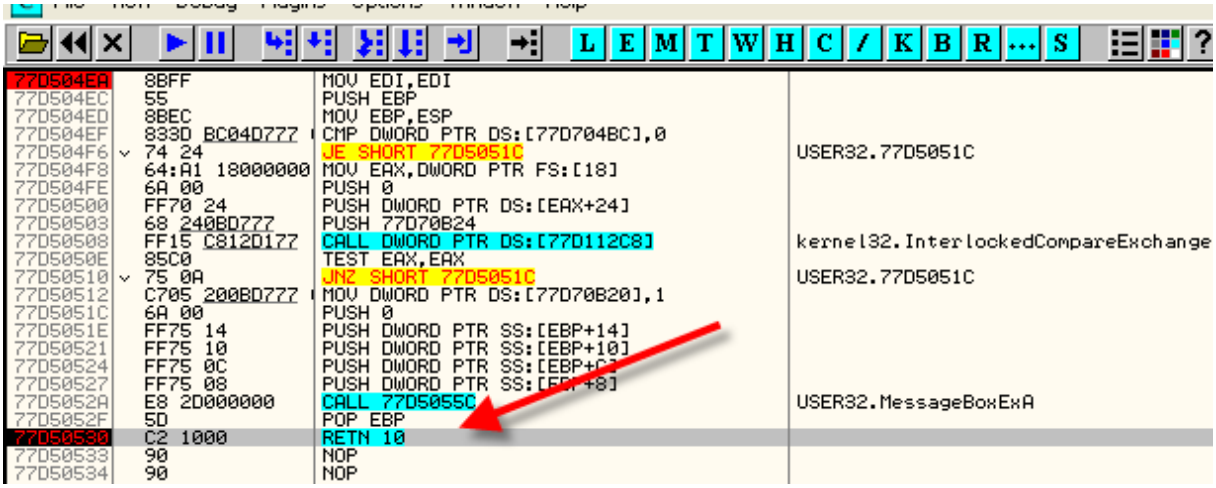


Останавливаемся здесь, делаем RUN.

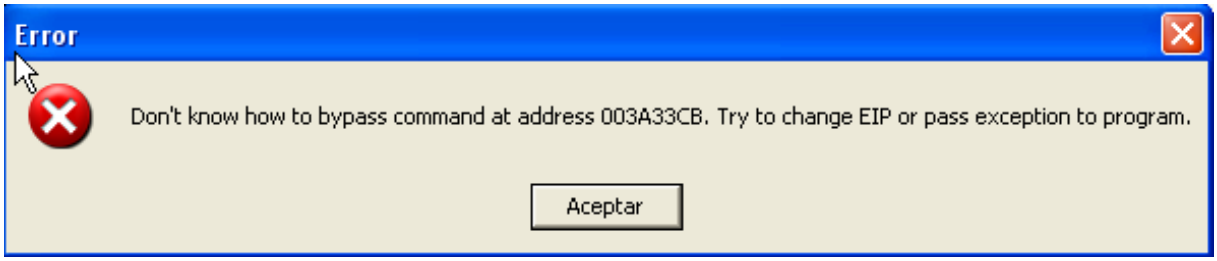
Теперь смотрим карту памяти.

Конечно, жертвой является другой Ollydbg, чьи секции мы здесь и наблюдаем. Установим Bp MessageBoxA.

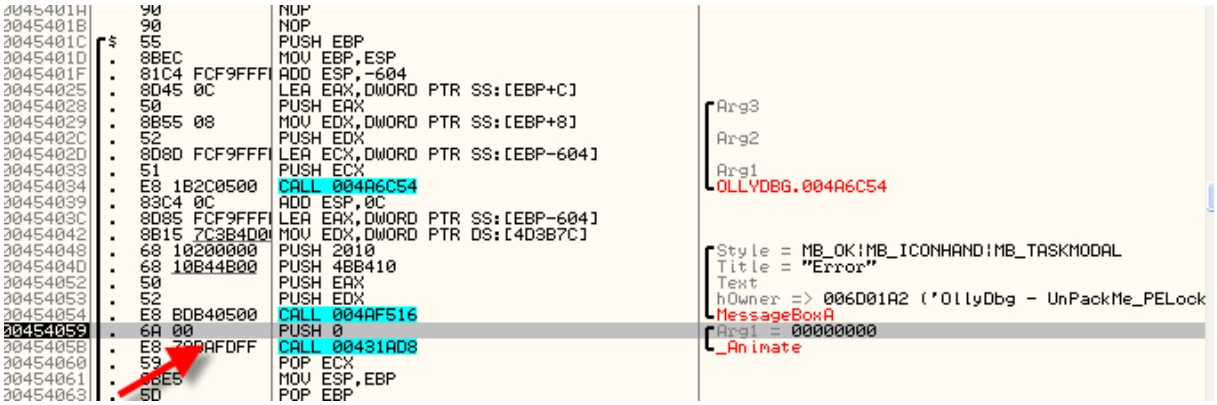




Установив BP на RET API-функции MessageBoxA и нажав на кнопку в окошке с сообщением, остановимся на RET при возврате из этого окошка.

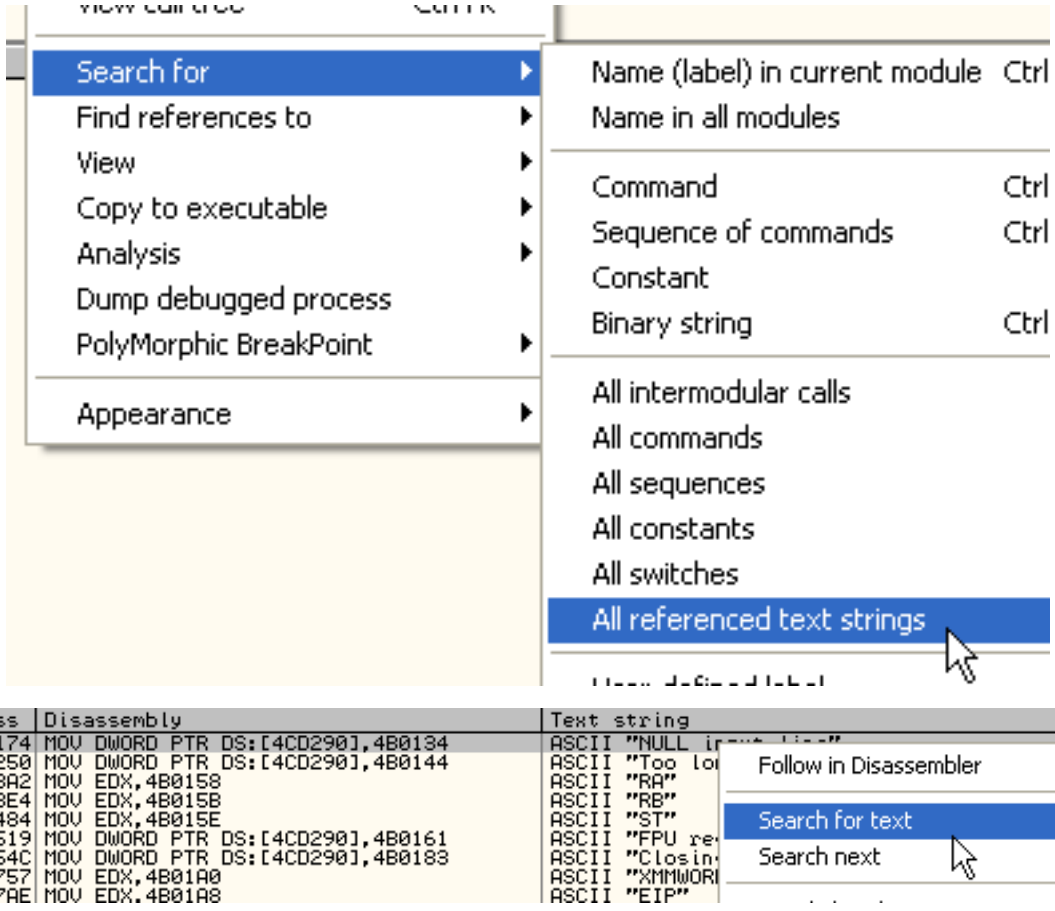


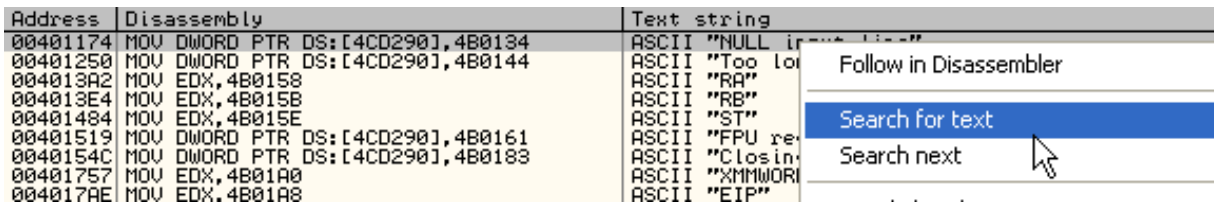
Здесь видим место в программе, куда возвратились из API-функции. Нажимаем F7.



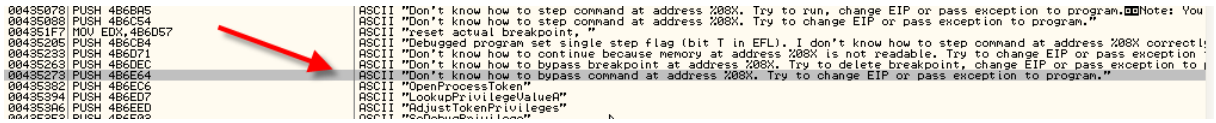
Если помните, таков был текст сообщения.

Поищем среди строк в OllyDbg.

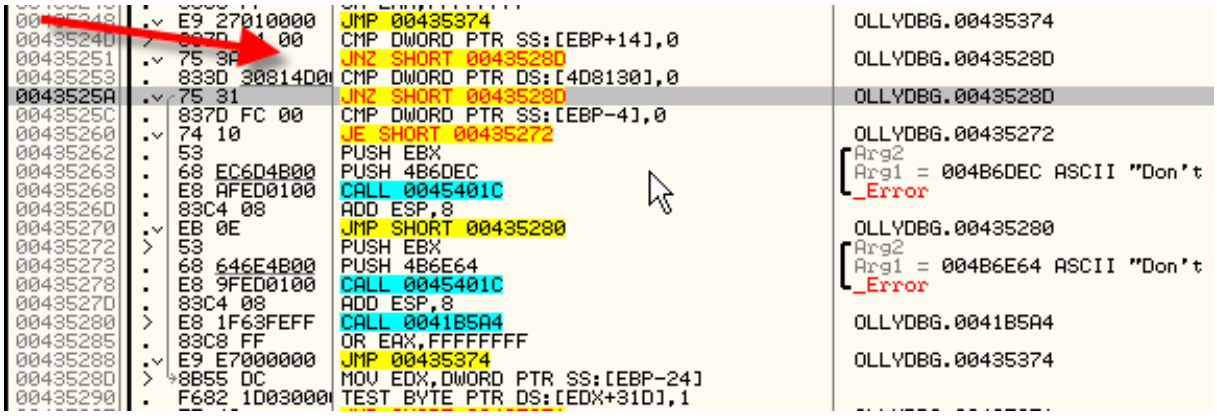




И здесь ищем слово “bypass”, которое есть в сообщении.

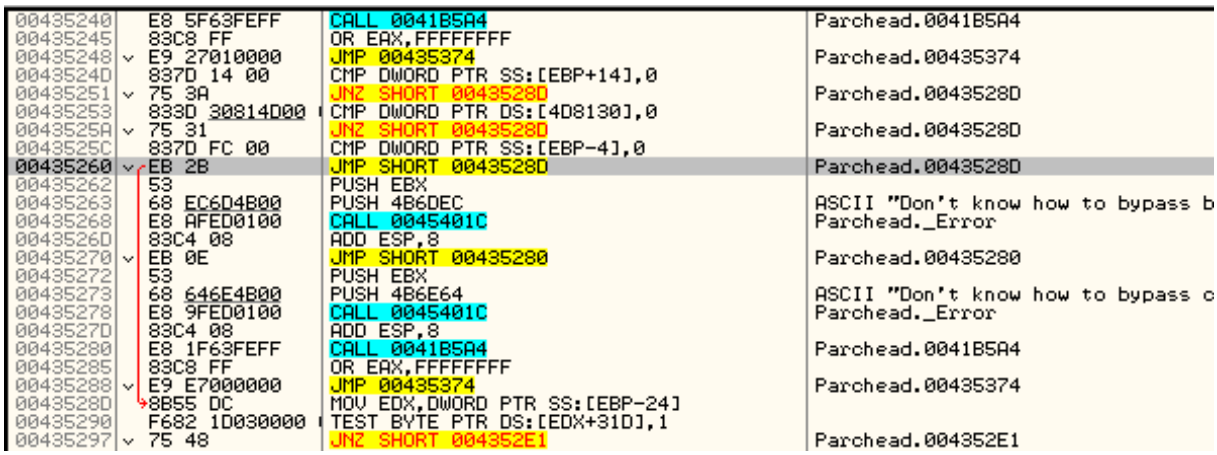


Видим, что это правильная строка, делаем двойной щелчок, чтобы перейти в место, где она используется в листинге.



Видим, что один из этих JNZ можно заменить на JMP и вот наша программа пропатчена, сохраняем наши изменения с помощью COPY TO EXECUTABLE и SAVE FILE.

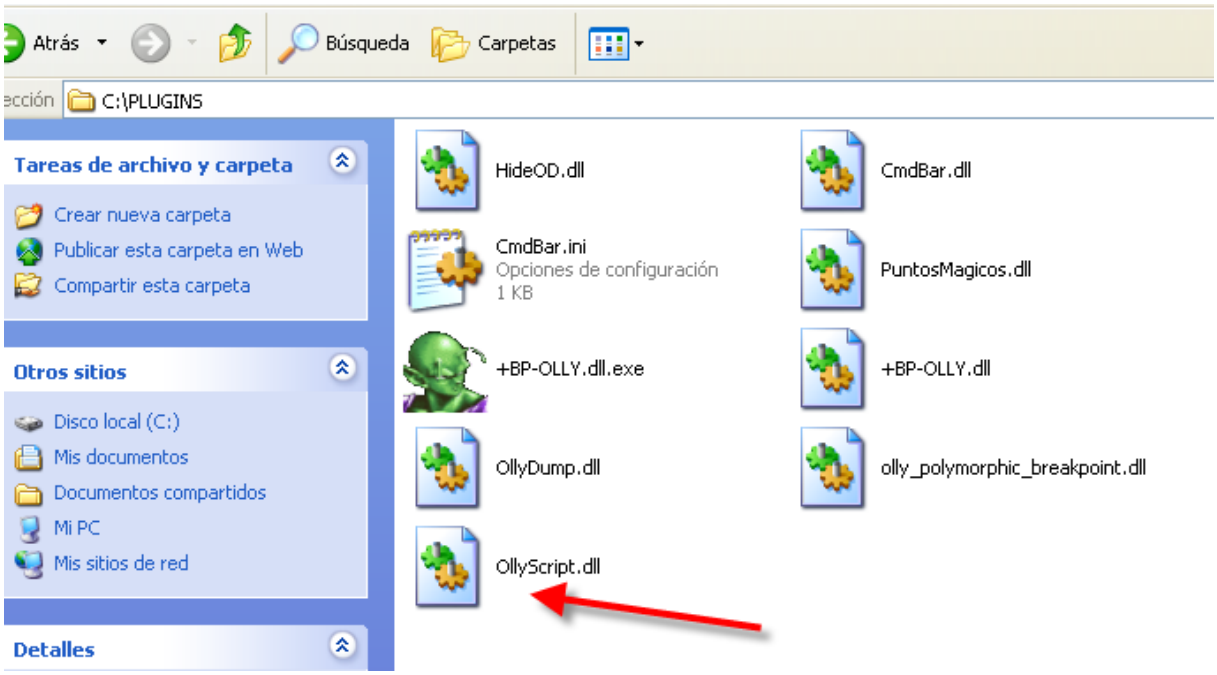
Откроем «parcheado 4» в OllyDbg и посмотрим эту область кода:



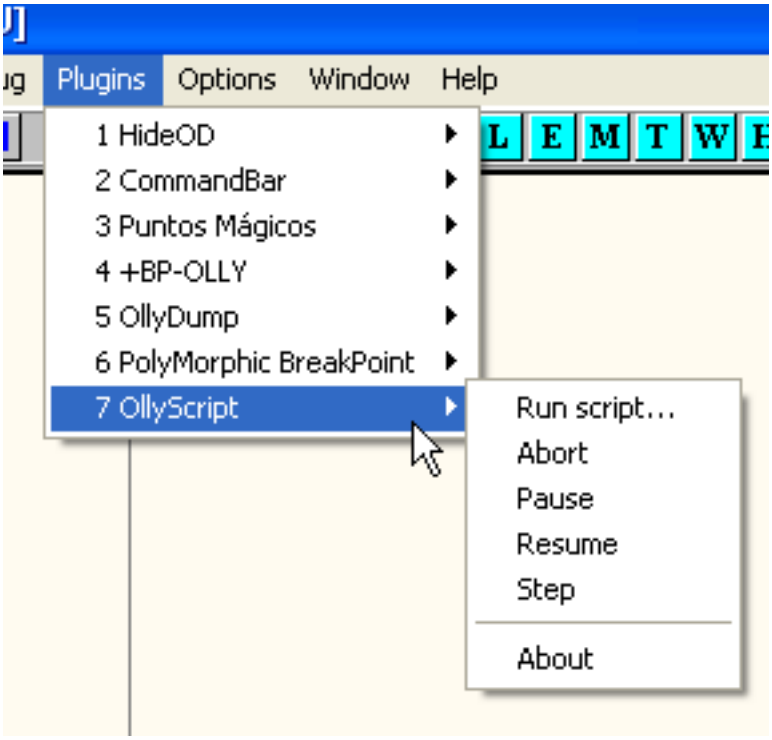
Видим, что переходы пропатчены таким образом, чтобы перепрыгнуть через окошко, всё, что нужно сделать - это изменить последний перед выводом окошка условный переход на JMP 43528D, а если переход случиться в другом месте, то окошко будет обойдено естественным путём.

Ок, вот и весь секрет «parcheado 4», поэтому в нём нет бага с ILLEGAL INSTRUCTION как в обычном Ollydbg.exe, так что можете его пропатчить сами или использовать тот, что шёл с 39 главой.

Вторая часть этого туториала заключается в создании простого скрипта, для чего мы будем использовать OLLYSCRIPT 0.92, который можно найти в файлах к этой главе. Положим DLL плагина в соответствующую папку.



Теперь открываем «parcheado 4».



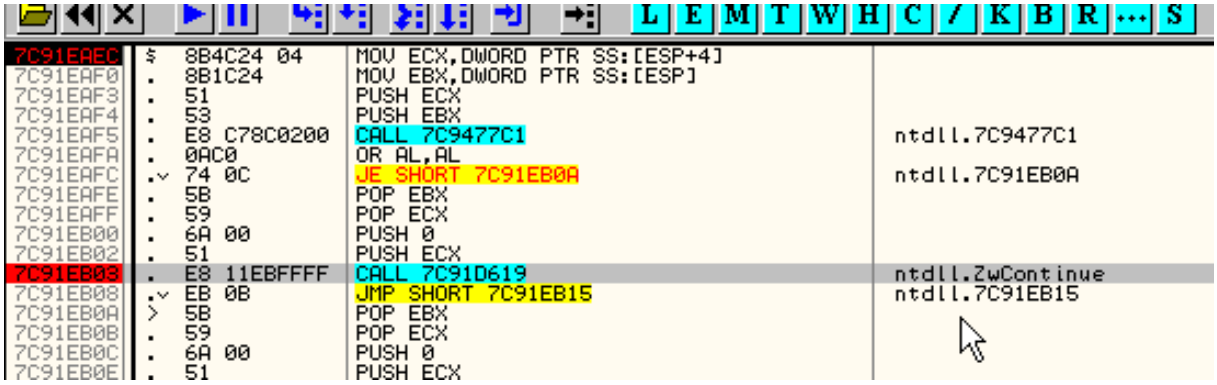
Видим, что наш плагин появился в соответствующем подменю.

Теперь открываем pelock, изучаемый нами. Идея в том, чтобы сделать минискрипт, который обеспечит работу HBP и не будет определяться упаковщиком.

Это будет простой скрипт. Команды можно посмотреть в README к плагину OLLYSCRIPT.

Идея следующая. Как видели в предыдущей части, каждый раз, когда останавливаемся на KiUserExceptionDispatcher, то находимся прямо перед тем местом, где операционная система доходит до обработчика исключений и где, как мы знаем, могут быть обнаружены и стёрты HBP, так что нам нужен скрипт, который каждый раз, когда программа останавливается на Bp KiUserExceptionDispatcher, стирал бы HBP, а при вызове ZwContinue восстанавливала бы его

обратно.



Так как это наш первый скрипт, то не будем его усложнять. BP по адресам KiUserExceptionDispatcher и вызова, ведущего к ZwContinue, устанавим вручную, так как мы уже знаем, где они находятся. Это можно сделать без особых проблем и скриптом, но незачем слишком условжнять, так что идея заключается в том, чтобы установить BP по адресам KiUserExceptionDispatcher и вызова, ведущего к ZwContinue, вручную согласно картинке.

Теперь, установив вручную BP, начинаем скрипт, который обычно начивается с

```
inicio:
```

Это метка, необходимая для того, если потребуется перейти в начало скрипта, что делается с помощью команды Jmp inicio. После inicio идёт HBP, который нужно установить. Например:

```
bphws 12ffc4, "r"
```

bphws - это команда, которая устанавливает аппаратную точку останова по указанному адресу, в данном случае это 12ffc4, а r в кавычках означает «на доступ» или «на чтение», что одно и то же. Также есть “w” - «на запись» и “x” - «на выполнение».

```
inicio:

bphws 12ffc4, "r"

trabajo:
```

После установки одной или нескольких аппаратных точек останова идёт ещё одна метка, чтобы можно было вернуться в место непосредственно после установки HBP.

```
inicio:

bphws 12ffc4, "r"

trabajo:

eob pirulo
run
```

Здесь видим команду eob, которая делает то же, что происходит, когда OllyDbg обнаруживает исключение или точку останова - передаёт управление по метке, в данном случае «pirulo». Это означает, что когда программа остановится на исключении или точке останова, то скрипт продолжит своё выполнение с метки pirulo. Дальше делаем RUN с помощью команды run.

Это главная часть, где OllyDbg начинает выполнять жертву. Нам нужно подготовиться к тому, что делать, когда будет обнаружено исключение или точка останова, для чего распишем метку pirulo:

```
pirulo:
cmp eip, 7c91eaec
je quitar
cmp eip, 7c91eb03
je restaurar
```

```
jmp final
```

На моей машине 7c91eaec - это адрес KiUserExceptionDispatcher, то есть когда происходит остановка на этом BP, eip моей машины будет содержать 7c91eaec, так что это инструкция гарантирует, что если мы действительно здесь, то происходит переход по метке quitar, чтобы стереть HBP.

Затем идёт другое сравнение на то, когда происходит остановка на другом BP, находящемся на CONTINUE. Когда обнаруживается, что программа остановилась на данном BP, то переходим на метку restaurar, чтобы заново установить HBP.

Дальше остаётся немного:

```
quitar:
bphwc 12ffc4
jmp trabajo
```

Когда переходим на quitar, чтобы стереть HBP, то для этого используем команду bphwc, а затем переходим на метку trabajo, чтобы продолжить выполнение программы, но уже без HBP.

Другая метка служит для восстановления HBP при вызове ZwContinue.

```
restaurar:
jmp inicio
```

Которая напрямую переходит на начало, где снова устанавливается HBP, а цикл начинается заново.

Конечно, нужно учитывать возможность остановки программы, когда OllyDbg останавливается на нашем HBP, в данном случае это последняя часть.

Как запускается eob, когда случается точка останова или исключение, так при остановке на нашем HBP будет совершён переход на pirulo, и так как нет никакого BP, который бы мы установили вручную, то управление передаётся по метке final:

```
pirulo:
cmp eip, 7c91eaec
je quitar
cmp eip, 7c91eb03
je restaurar
jmp final
```

Вот что там:

```
final:
MSGYN "Continuar?"
cmp $RESULT,1
je inicio
ret
```

MSGYN заставляет появиться окошко с опцией «continuar» (продолжить). Смотрим, подходит ли нам место, где мы остановились, или хотим продолжить выполнение до следующего раза. Если нажимаем «YES», то в переменную $RESULT будет помещено значение 1, а если нажмём «NO», то 0, поэтому когда хотим продолжить, то по нажатию на «YES» скрипт проверяет, равен ли $RESULT одному, и передаёт управление на начало скрипта, а если «NO», то следующей идёт инструкция ret, где выполнение скрипта завершается, и OllyDbg останавливается там, где мы хотим.

Вот полный исходный текст скрипта (не забудьте поставить вручную BP на KiUserExceptionDispatcher и CALL’e, ведущего к ZwContinue).

```
inicio:
bphws 12ffc4, "r"
trabajo:
eob pirulo
run
```

```
pirulo:
cmp eip, 7c91eaec
je quitar
cmp eip, 7c91eb03
je restaurar
jmp final

quitar:
bphwc 12ffc4
jmp trabajo

restaurar:
jmp inicio

final:
MSGYN "Continuar?"
cmp $RESULT,1
je inicio
ret
```

Внизу изображена большая часть скрипта с поясняющими стрелочками, в inicio устанавливаем HB, и запускаем программу. Когда встретится BP или исключение, будет совершён переход на метку pirulo.

Переход на pirulo совершается только тогда, когда есть BP или исключение, и в этом момент нужно проверить различные возможности: является ли этот BP одним из установленных нами вручную, или это аппаратное исключение, или же какое-то другое исключение. Проверяем EIP. Можем определить, откуда произошёл переход на метку. Если это случилось из BP в KiUserExceptionDispatcher, то снимаем HBP, чтобы он не был определён обработчиком исключений, и продолжаем выполнение программы, но уже без HE.

Когда срабатывает BP, установленный на ZwContinue, снова прыгаем на метку pirulo, сравниваем EIP, и нам нужно перейти на restaurar, так как в этот момент обработчик исключений уже пройден, и можем снова установить точку останова. В случае, если сработал другой BP или остановились на HE, то переходим на final, где можем решить, хотим ли остановиться или продолжить. Если останавливаемся, то выполняется инструкция ret, если продолжаем, то совершается переход на inicio.

```
inicio:
bphws 12ffc4, "r"
trabajo:
eob pirulo
run

pirulo:
cmp eip, 7c91eaec
je quitar
cmp eip, 7c91eb03
je restaurar
jmp final

quitar:
bphwc 12ffc4
jmp trabajo

restaurar:
jmp inicio

final:
MSGYN "Continuar?"
cmp $RESULT,1
je inicio
ret
```

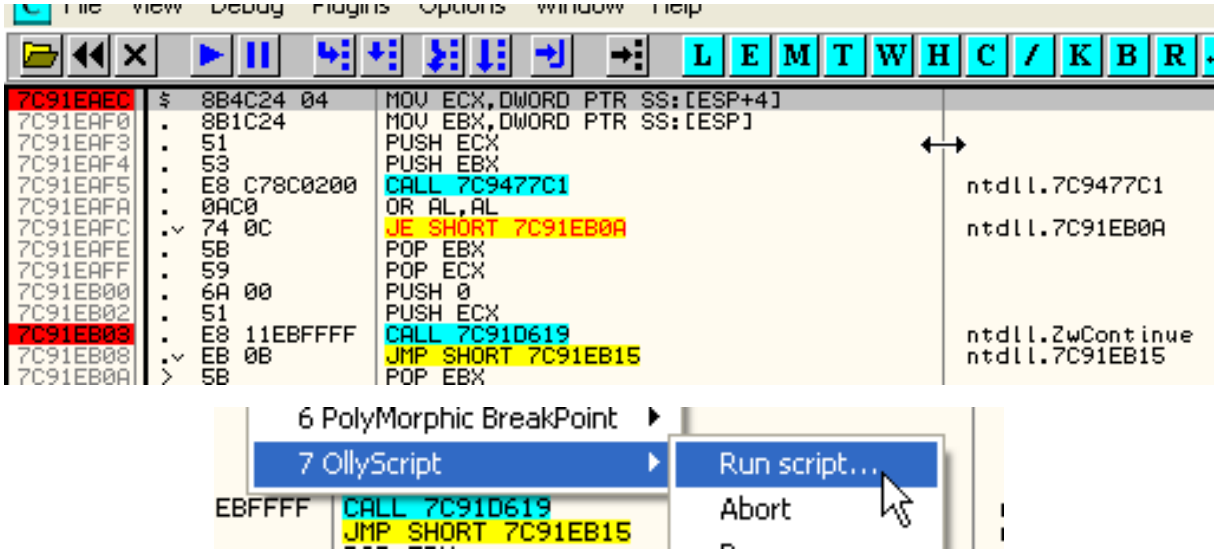
al encontrar excepcion o bp

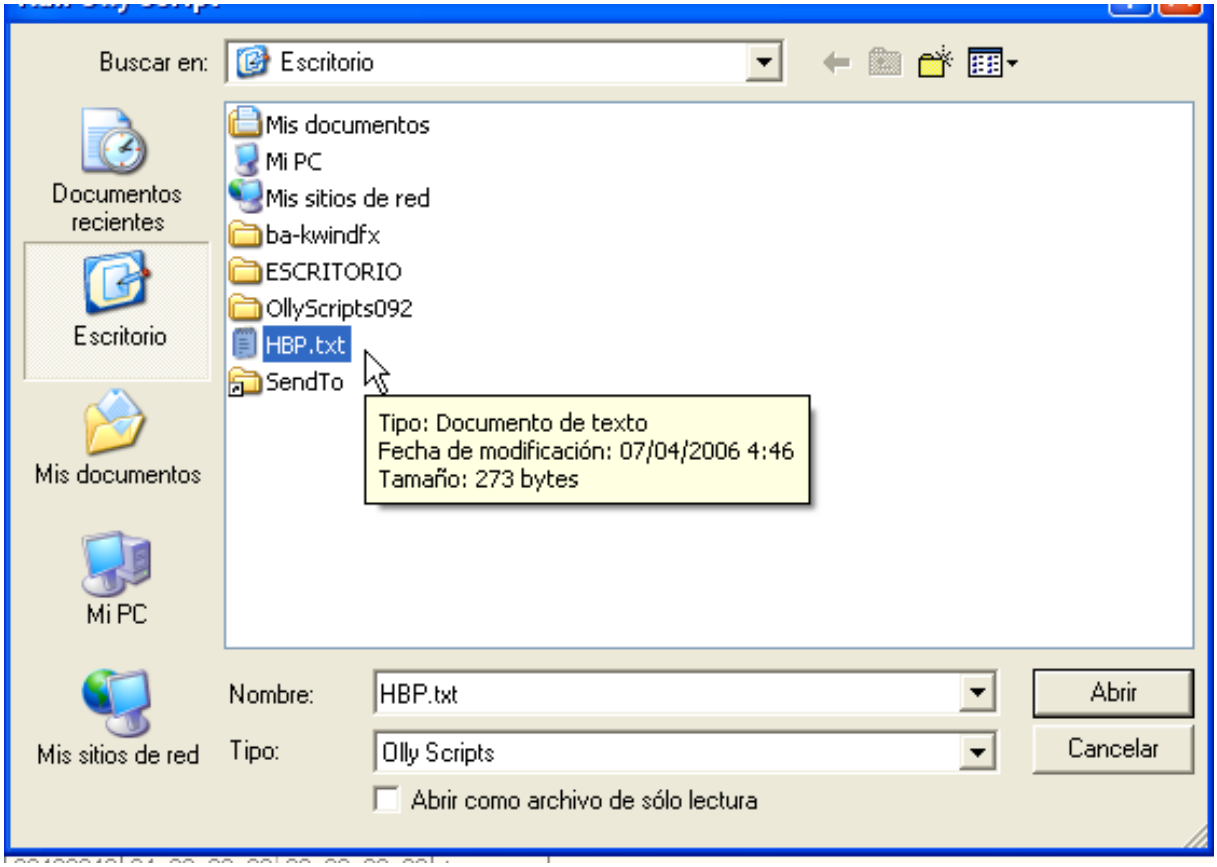
si estamos en KiUserExceptionDispatcher borramos los HE

si estamos en ZWcontinue restauramos

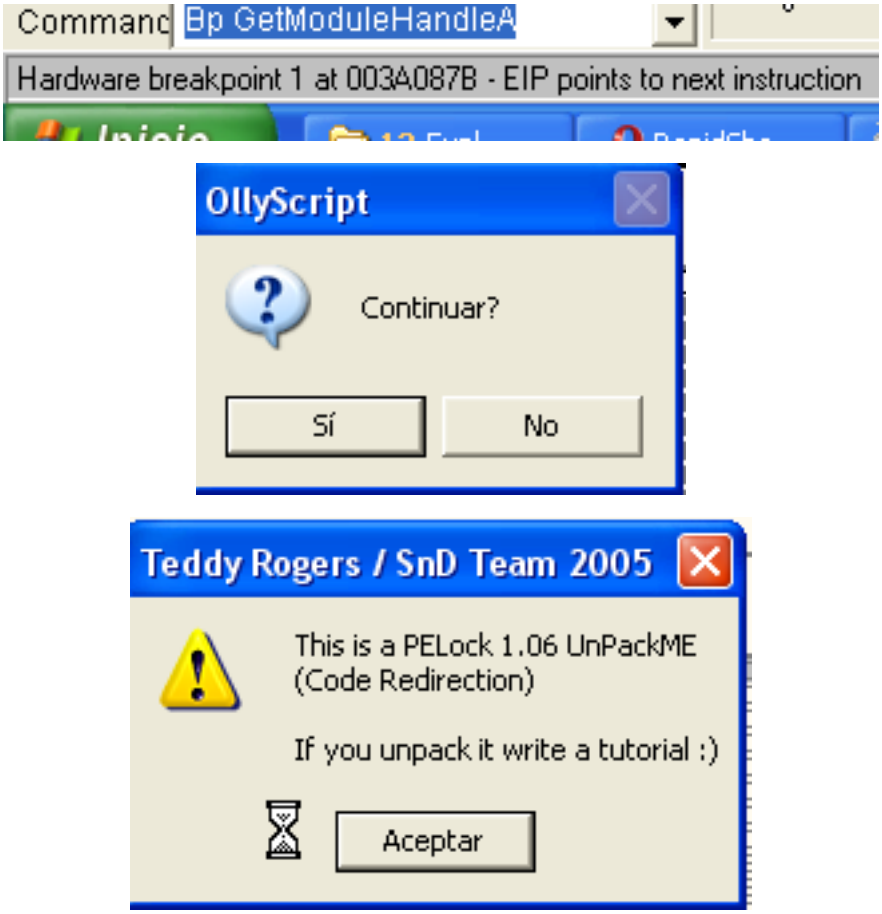
aqui decidimos si paramos o no

Попробуем скрипт в действии. Сохраняем его в текстовый файл и запускаем pelock, после чего устанавливаем вручную 2 BP и идём в меню плагина.

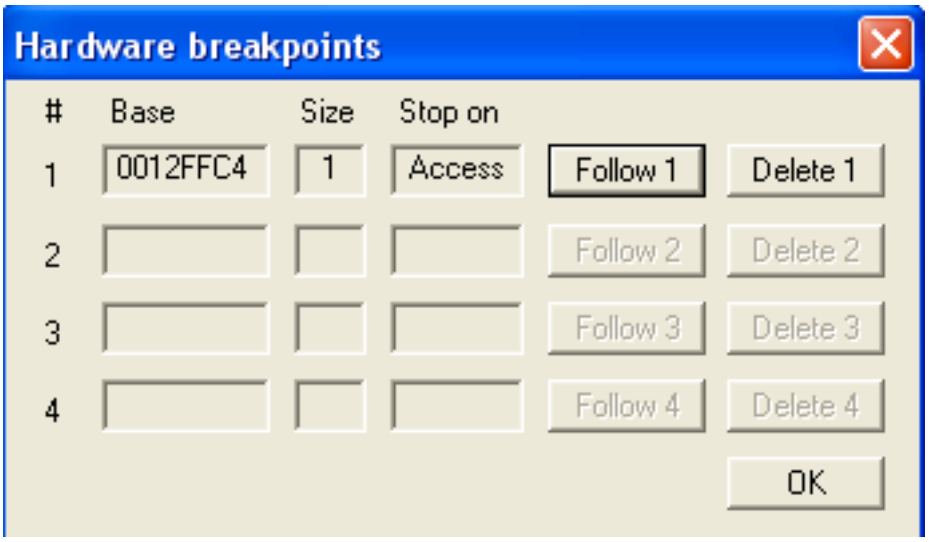




Видим, что остановились на HBP. Чтобы продолжить, нажимаем YES.



Видим, что программа прекрасно выполнилась и в окошке аппаратных точек останова видим:



То же самое при установке, поэтому программа выполняется с HBP и останавливается на них без каких-либо проблем, чего мы и хотели добиться.

В главе 41 мы продолжим делать скрипты и починим pelock полностью.

Файлы (находится в архиве `wasm_files.zip`: `files/0510/ollydbg40.zip`) к статье.

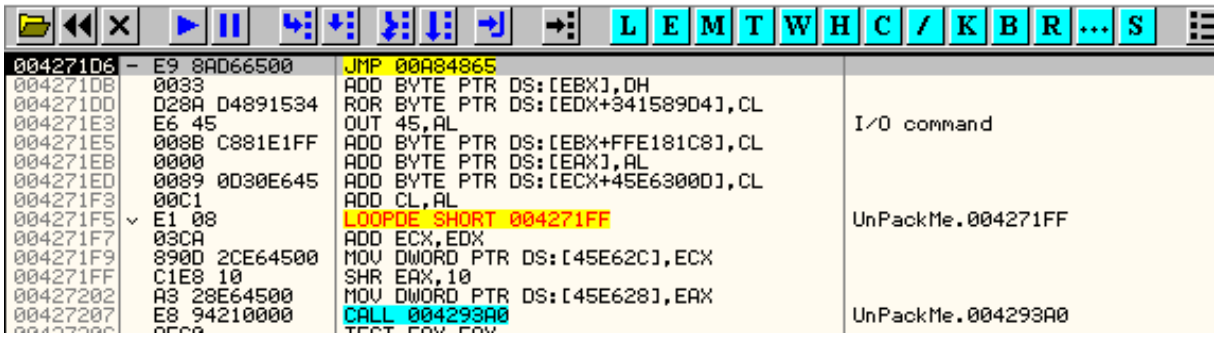
Введение в крэкинг с нуля, используя OllyDbg - Глава 41

Автор: Рикардо Нарваха, пер. Aquila

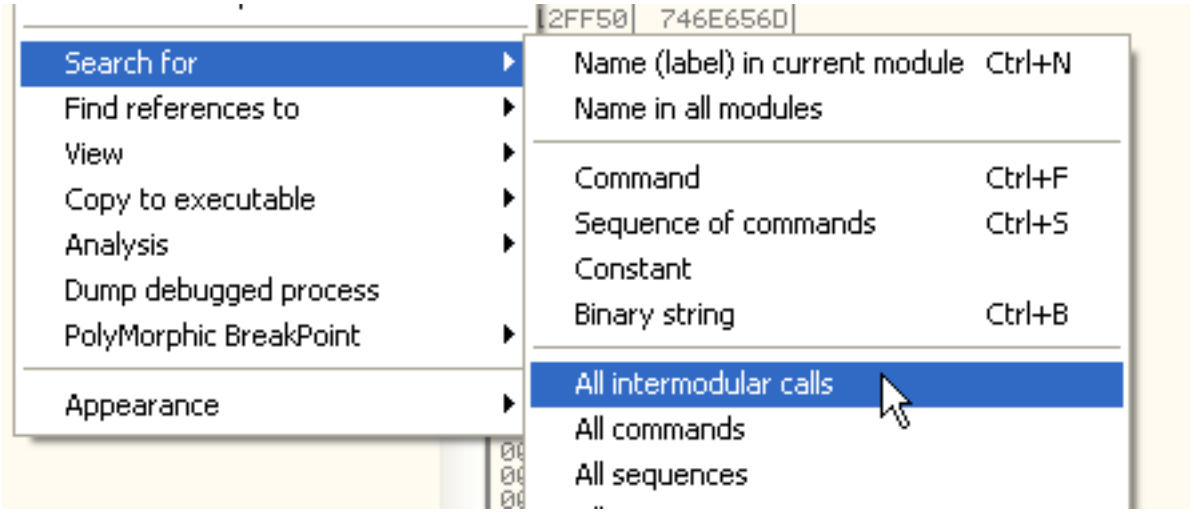
Дата: 2009-04-06

Продолжаем с распаковкой pelock, с которым мы пока не справились. Теперь начнём работать с IAT и посмотрим, существуют ли для него волшебные переходы или что-нибудь в этом роде.

Для этого снова идём OEP.

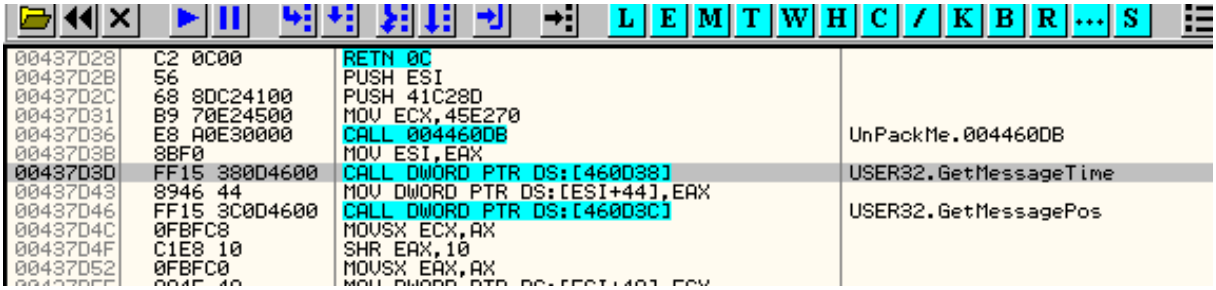


Ок, теперь идём в фальшивую OEP, ищем вызовы API-функций и с помощью клика на правую кнопку мыши смотрим SEARCH FOR-ALL INTERMODULAR CALLS.



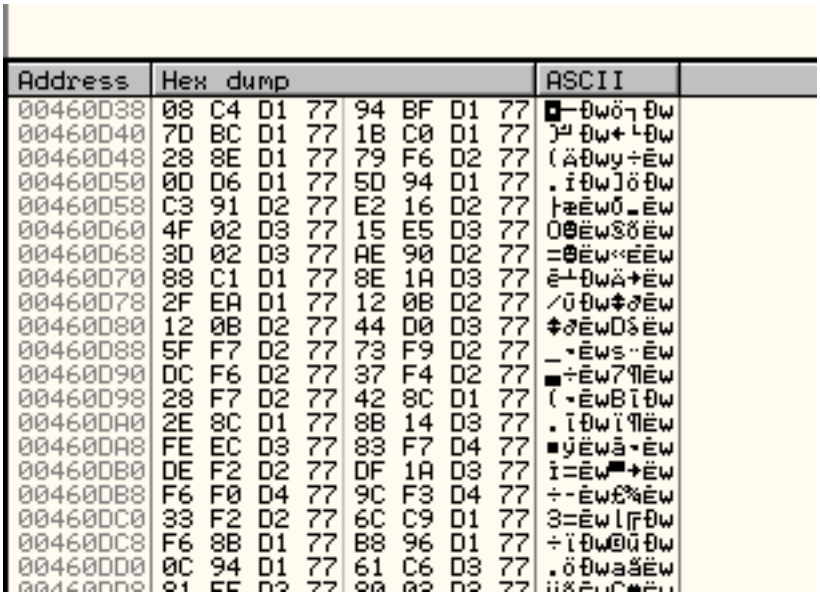
```
00437D3D CALL DWORD PTR DS:[460D38]   USER32.GetMessageTime
00437D46 CALL DWORD PTR DS:[460D3C]   USER32.GetMessagePos
00437EDD CALL 00437B9F                UnPackMe.00437B9F
00437F0A CALL DWORD PTR DS:[460D2C]   USER32.GetPropA
00437F66 CALL DWORD PTR DS:[460D30]   USER32.CallWindowProcA
00437F84 CALL DWORD PTR DS:[460D50]   USER32.SetWindowLongA
00437F8C CALL DWORD PTR DS:[460D34]   USER32.RemovePropA
00437FE0 CALL DWORD PTR DS:[460D30]   USER32.CallWindowProcA
0043808D CALL DWORD PTR DS:[460E78]   USER32.SendMessageA
004380D0 CALL DWORD PTR DS:[460D20]   USER32.GetLastActivePopup
004380E2 CALL DWORD PTR DS:[460D24]   USER32.GetForegroundWindow
00438100 CALL DWORD PTR DS:[460D28]   USER32.SetForegroundWindow
004381BA CALL DWORD PTR DS:[460D14]   USER32.CallNextHookEx
00438240 CALL 00437B9F                UnPackMe.00437B9F
00438257 CALL DWORD PTR DS:[460D54]   USER32.GetWindowLongA
00438278 CALL DWORD PTR DS:[460D50]   USER32.SetWindowLongA
0043828B CALL DWORD PTR DS:[460D50]   USER32.SetWindowLongA
004382A8 CALL DWORD PTR DS:[460D54]   USER32.GetWindowLongA
004382BD CALL DWORD PTR DS:[460D18]   USER32.SetPropA
004382C5 CALL DWORD PTR DS:[460D2C]   USER32.GetPropA
004382E4 CALL DWORD PTR DS:[460D50]   USER32.SetWindowLongA
004382F3 CALL DWORD PTR DS:[460D14]   USER32.CallNextHookEx
00438309 CALL DWORD PTR DS:[460D1C]   USER32.UnhookWindowsHookEx
0043833E CALL DWORD PTR DS:[460ACC]   kernel32.GetCurrentThreadId
0043834E CALL DWORD PTR DS:[460D10]   USER32.SetWindowsHookExA
0043838D CALL DWORD PTR DS:[460D1C]   USER32.UnhookWindowsHookEx
00438447 CALL DWORD PTR DS:[460D0C]   USER32.CreateWindowExA
00438644 CALL DWORD PTR DS:[460E78]   USER32.SendMessageA
00438681 CALL DWORD PTR DS:[460D50]   USER32.SetWindowLongA
004386DC CALL DWORD PTR DS:[460D08]   USER32.DestroyWindow
```

Вот они, выбираем один и делаем двойной щелчок на нём, чтобы посмотреть листинг.



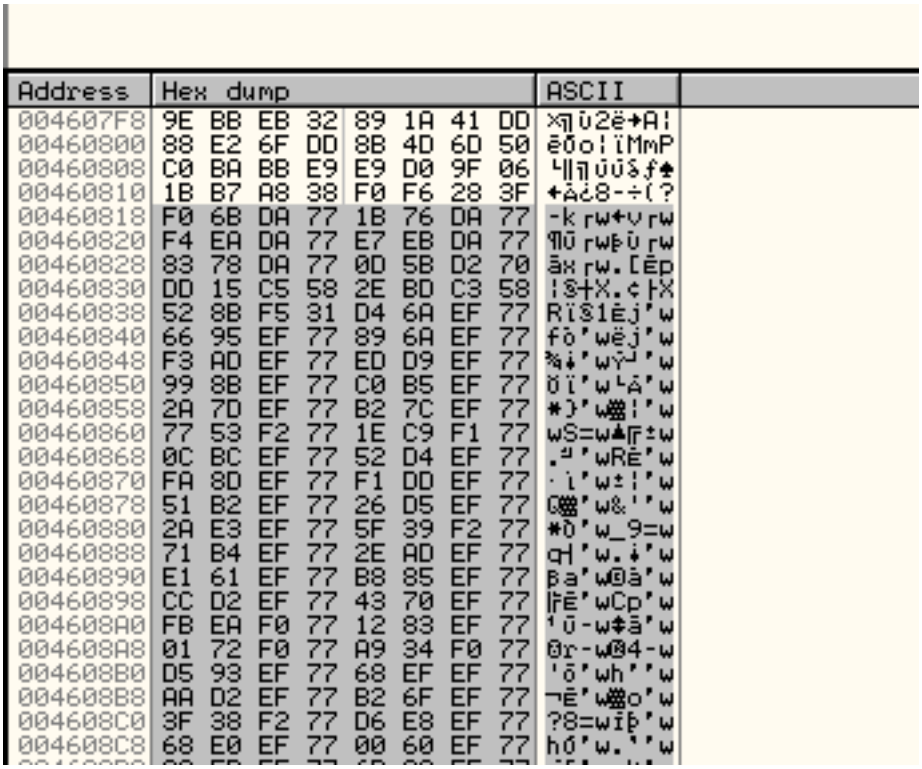
```
00437D28   C2 0C00          RETN 0C
00437D2B   56               PUSH ESI
00437D2C   68 8DC24100      PUSH 41C28D
00437D31   B9 70E24500      MOV ECX,45E270
00437D36   E8 A0E30000      CALL 004460DB                 UnPackMe.004460DB
00437D3B   8BF0             MOV ESI,EAX
00437D3D   FF15 380D4600    CALL DWORD PTR DS:[460D38]    USER32.GetMessageTime
00437D43   8946 44          MOV DWORD PTR DS:[ESI+44],EAX
00437D46   FF15 3C0D4600    CALL DWORD PTR DS:[460D3C]    USER32.GetMessagePos
00437D4C   0FBFC8           MOVSX ECX,AX
00437D4F   C1E8 10          SHR EAX,10
00437D52   0FBFC0           MOVSX EAX,AX
```

Это значит вызов API-функции принимают значение 460d38, что является адресом элемента IAT.



```
Address   Hex dump                  ASCII
00460D38  08 C4 D1 77 94 BF D1 77  ◘─ÐwöÂ¬Ðw
00460D40  7D BC D1 77 1B C0 D1 77  }╝Ðw←└Ðw
00460D48  28 8E D1 77 79 F6 D2 77  (ÄÐwy÷Ëw
00460D50  0D D6 D1 77 5D 94 D1 77  .ÍÐw]öÐw
00460D58  C3 91 D2 77 E2 16 D2 77  ├æËwŌ▬Ëw
00460D60  4F 02 D3 77 15 E5 D3 77  O☻ËwSõËw
00460D68  3D 02 D3 77 AE 90 D2 77  =☻Ëw«ÉËw
00460D70  88 C1 D1 77 8E 1A D3 77  ê┴ÐwÄ→Ëw
00460D78  2F EA D1 77 12 0B D2 77  /ūÐw↕∂Ëw
00460D80  12 0B D2 77 44 D0 D3 77  ↕∂ËwDŠËw
00460D88  5F F7 D2 77 73 F9 D2 77  _▪Ëws¨Ëw
00460D90  DC F6 D2 77 37 F4 D2 77  ▄÷Ëw7¶Ëw
00460D98  28 F7 D2 77 42 8C D1 77  (▪ËwBîÐw
00460DA0  2E 8C D1 77 8B 14 D3 77  .îÐwï¶Ëw
00460DA8  FE EC D3 77 83 F7 D4 77  ■ÿËwā▪Ëw
00460DB0  DE F2 D2 77 DF 1A D3 77  ì=Ëw▀→Ëw
00460DB8  F6 F0 D4 77 9C F3 D4 77  ÷-Ëw£%Ëw
00460DC0  33 F2 D2 77 6C C9 D1 77  3=Ëwl╔Ðw
00460DC8  F6 8B D1 77 B8 96 D1 77  ÷ïÐwØūÐw
00460DD0  0C 94 D1 77 61 C6 D3 77  .öÐwaãËw
```

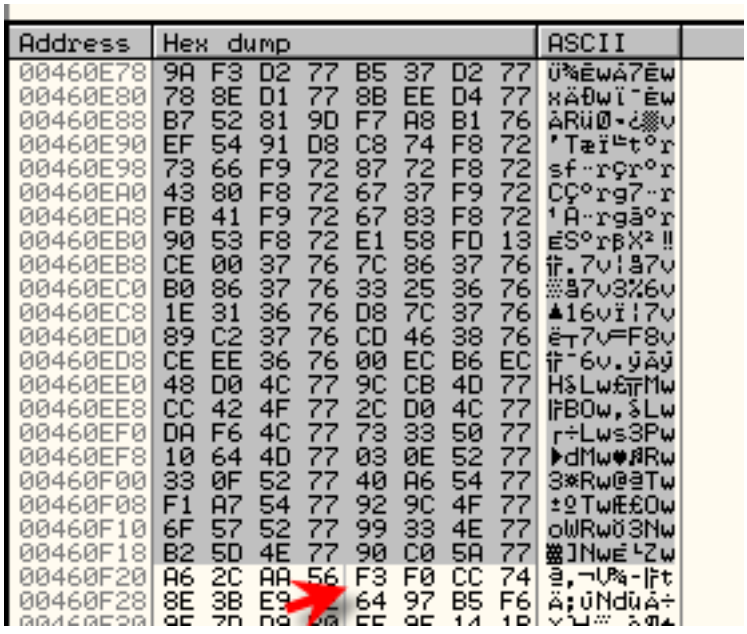
Вот IAT и видим, что все элементы – правильные, поднимаемся наверх, чтобы найти начало IAT.



Address	Hex dump	ASCII
004607F8	9E BB EB 32 89 1A 41 DD	
00460800	88 E2 6F DD 8B 4D 6D 50	
00460808	C0 BA BB E9 E9 D0 9F 06	
00460810	1B B7 A8 38 F0 F6 28 3F	
00460818	F0 6B DA 77 1B 76 DA 77	
00460820	F4 EA DA 77 E7 EB DA 77	
00460828	83 78 DA 77 0D 5B D2 70	
00460830	DD 15 C5 58 2E BD C3 58	
00460838	52 8B F5 31 D4 6A EF 77	
00460840	66 95 EF 77 89 6A EF 77	
00460848	F3 AD EF 77 ED D9 EF 77	
00460850	99 8B EF 77 C0 B5 EF 77	
00460858	2A 7D EF 77 B2 7C EF 77	
00460860	77 53 F2 77 1E C9 F1 77	
00460868	0C BC EF 77 52 D4 EF 77	
00460870	FA 8D EF 77 F1 DD EF 77	
00460878	51 B2 EF 77 26 D5 EF 77	
00460880	2A E3 EF 77 5F 39 F2 77	
00460888	71 B4 EF 77 2E AD EF 77	
00460890	E1 61 EF 77 B8 85 EF 77	
00460898	CC D2 EF 77 43 70 EF 77	
004608A0	FB EA F0 77 12 83 EF 77	
004608A8	01 72 F0 77 A9 34 F0 77	
004608B0	D5 93 EF 77 68 EF EF 77	
004608B8	AA D2 EF 77 B2 6F EF 77	
004608C0	3F 38 F2 77 D6 E8 EF 77	
004608C8	68 E0 EF 77 00 60 EF 77	

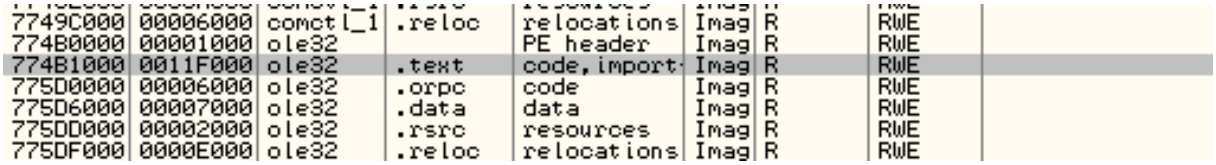
Видим, что начало IAT находится в 460818, так как если отметим какие-нибудь элементы выше и нажмём FIND REFERENCES, то не будет найдено никаких ссылок, ведущих в секции кода DLL. К сожалению, вся IAT правильная, поэтому нам не нужно искать волшебные переходы. Ищем конец IAT.

Вот и он.



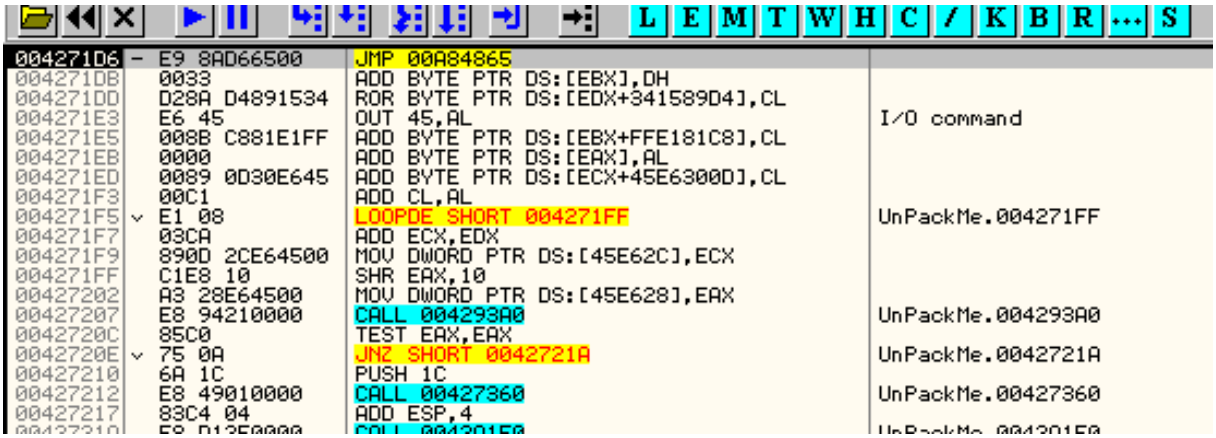
Address	Hex dump	ASCII
00460E78	9A F3 D2 77 B5 37 D2 77	
00460E80	78 8E D1 77 8B EE D4 77	
00460E88	B7 52 81 9D F7 A8 B1 76	
00460E90	EF 54 91 D8 C8 74 F8 72	
00460E98	73 66 F9 72 87 72 F8 72	
00460EA0	43 80 F8 72 67 37 F9 72	
00460EA8	FB 41 F9 72 67 83 F8 72	
00460EB0	90 53 F8 72 E1 58 FD 13	
00460EB8	CE 00 37 76 7C 86 37 76	
00460EC0	B0 86 37 76 33 25 36 76	
00460EC8	1E 31 36 76 D8 7C 37 76	
00460ED0	89 C2 37 76 CD 46 38 76	
00460ED8	CE EE 36 76 00 EC B6 EC	
00460EE0	48 D0 4C 77 9C CB 4D 77	
00460EE8	CC 42 4F 77 2C D0 4C 77	
00460EF0	DA F6 4C 77 73 33 50 77	
00460EF8	10 64 4D 77 03 0E 52 77	
00460F00	33 0F 52 77 40 A6 54 77	
00460F08	F1 A7 54 77 92 9C 4F 77	
00460F10	6F 57 52 77 99 33 4E 77	
00460F18	B2 5D 4E 77 90 C0 5A 77	
00460F20	A6 2C AA 56 F3 F0 CC 74	
00460F28	8E 3B E9 [redacted] 64 97 B5 F6	

До части выделенной серым нет проблем, есть ссылки а вот элемент 460f24 сомнителен, для него нет ссылок. То, что мы отметили серым это элементы, ведущие в секцию кода ole32.dll.

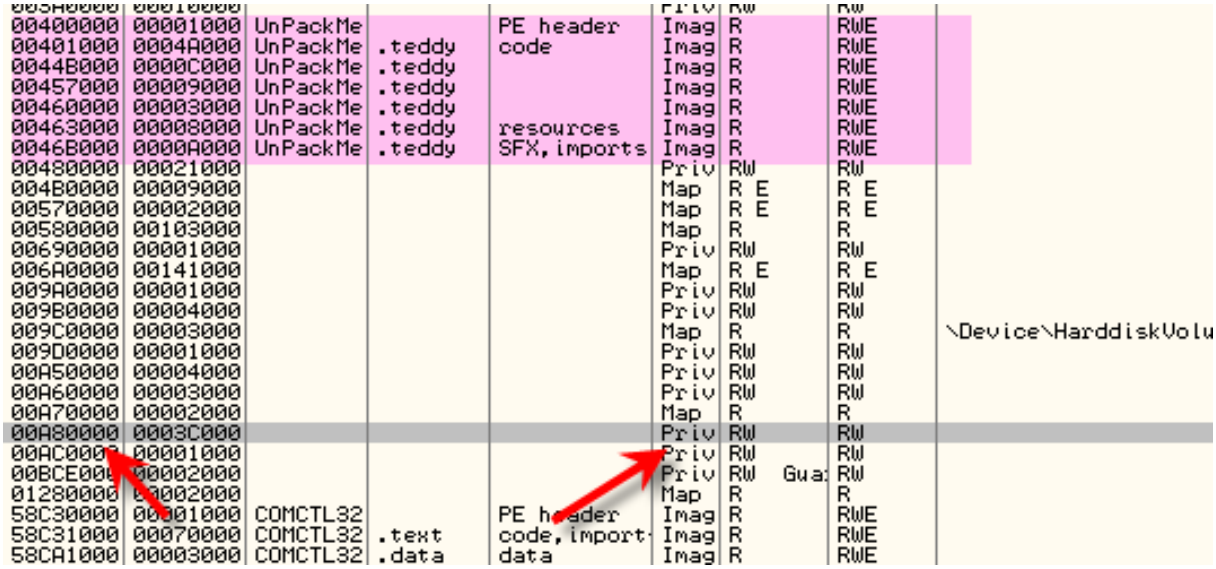


7749C000	00006000	comctl_1	.reloc	relocations	Imag	R	RWE
774B0000	00001000	ole32		PE header	Imag	R	RWE
774B1000	0011F000	ole32	.text	code,import	Imag	R	RWE
775D0000	00006000	ole32	.orpc	code	Imag	R	RWE
775D6000	00007000	ole32	.data	data	Imag	R	RWE
775DD000	00002000	ole32	.rsrc	resources	Imag	R	RWE
775DF000	0000E000	ole32	.reloc	relocations	Imag	R	RWE

Видим, что сделав щелчок правой кнопкой мыши на любом из элементов, ведущих в эту DLL, также нет ссылок, которые появляются здесь.

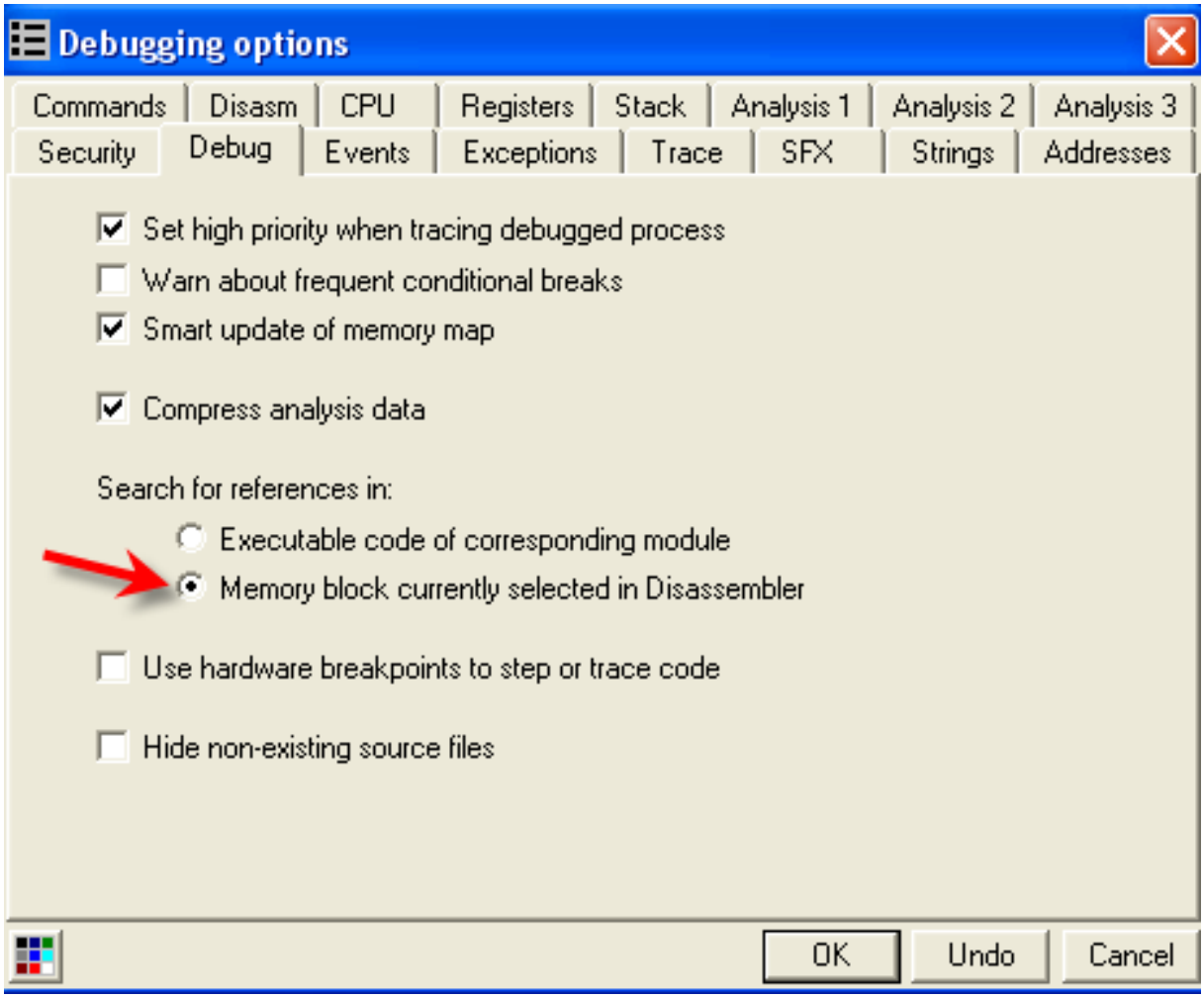


Если снова посмотрим на OEP, увидим, что редкий JMP, который на моей машине переходит в секцию по адресу Axxxxx, которая относится к секциям самого исполняемого файла (на вашей машине она может находиться по другому адресу). Посмотрим на карту памяти:



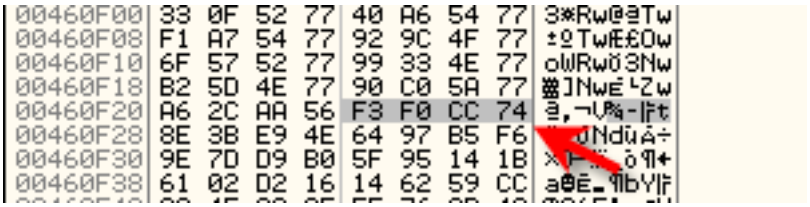
Видим, что программа переходит в эту секцию и что оригинальная программа, запускающаяся из OEP не может перейти в секцию, находящуюся вне exe-файла до того, как она будет создана с помощью VirtualAlloc или другой подобной функцией, поэтому делаем вывод, что эта секция была создана упаковщиком и не является часть оригинального кода программы, так же как и JMP, которые туда переходят, как, например, тот, который стоит в фальшивой OEP и многие другие.

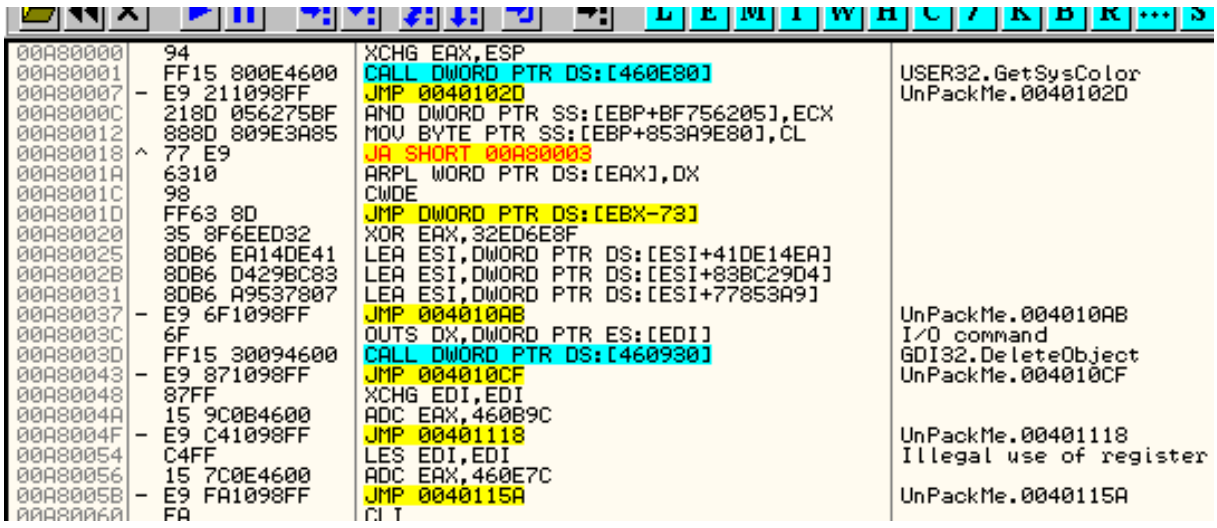
Поэтому возможно, что в эту секцию упаковщик поместил вызовы и переходы, которые берут значения из IAT. Откроем конфигурацию OllyDbg.



Ищем только в секции, которая отображается в листинге в данный момент, поэтому так как секция, созданная упаковщиком, в листинге не видна, поиск там производиться не будет.

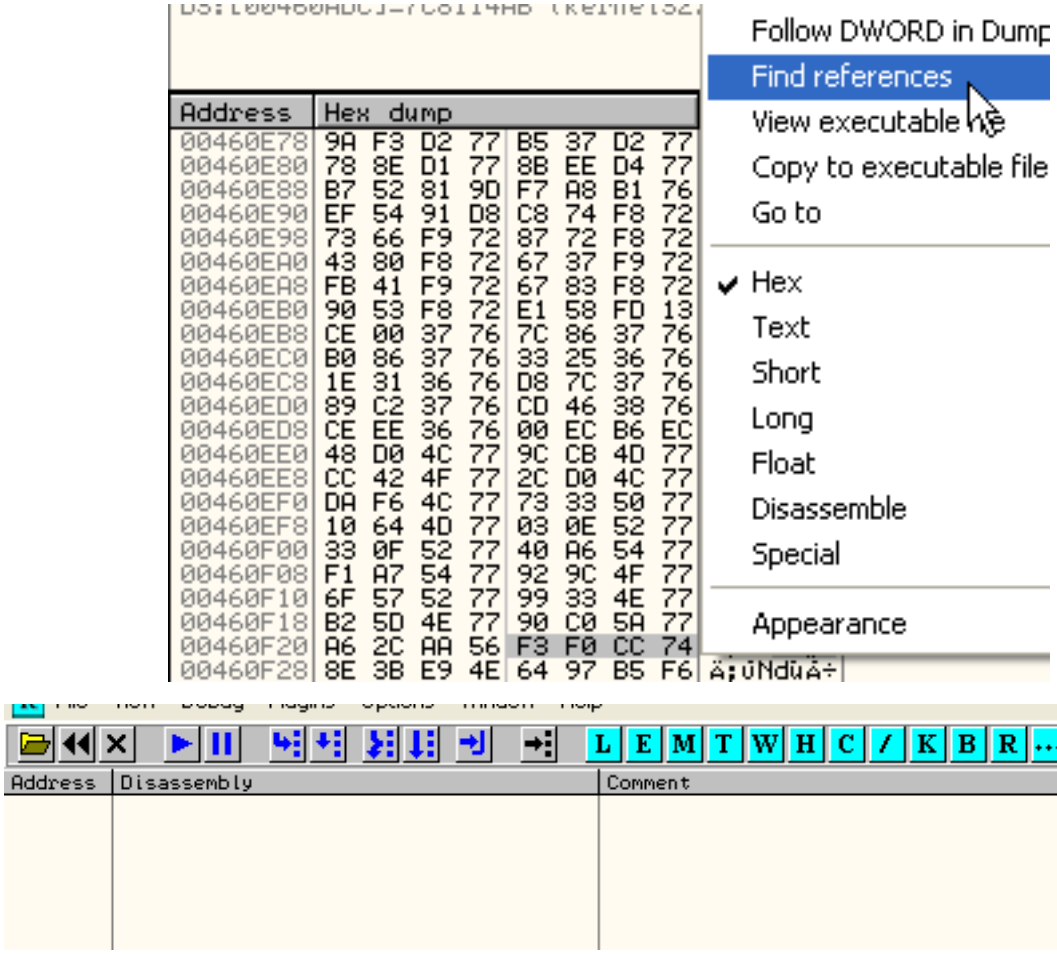
Пробуем искать ссылки в других секциях.

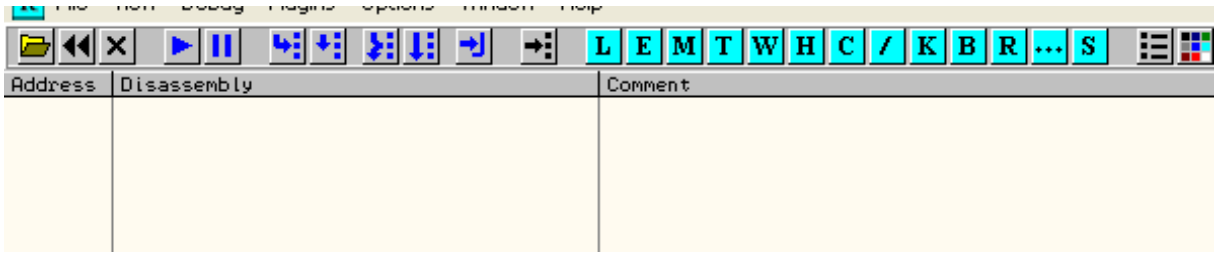




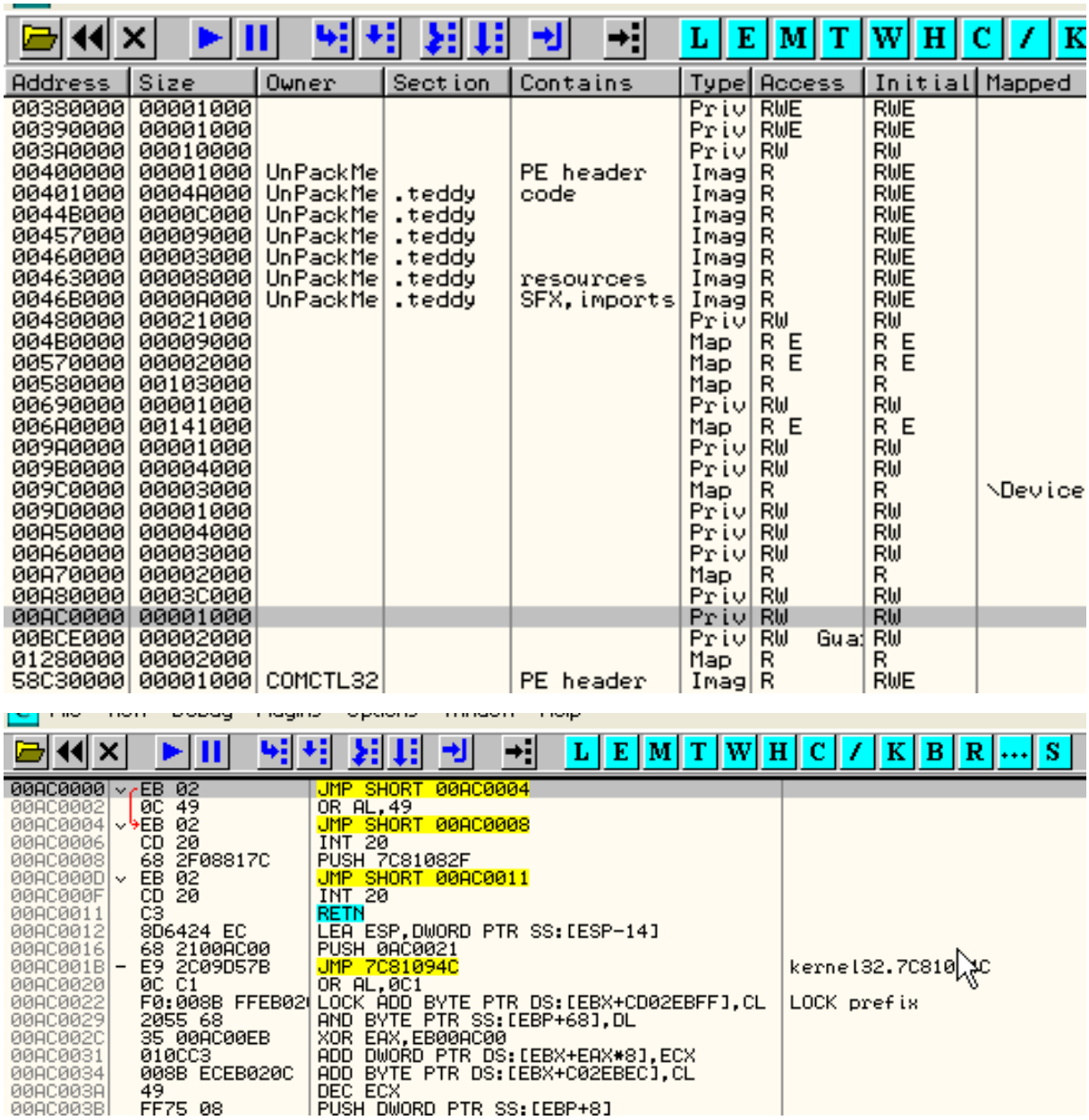
Здесь у нас одна из секций, созданных упаковщиком. Проверим, работает ли моя теория о том, что не найдём ссылки, так как вызовы протранслированы в другие секции (поэтому я упираю на то, что не нужно использовать только один метод; в данном случае, чтобы определить является ли элемент частью IAT или нет, можно или поискать в секциях, созданных упаковщиком, или посмотреть, ведут ли элементы в секцию кода какой-либо DLL через карту памяти и листинг).

Но ладно, сейчас у нас отображается одна из секций, созданных упаковщиком. Ищем в ней ссылки.





Ничего, но не падайте духом, так как у нас есть ещё секции, созданные упаковщиком, так что переходим к следующей.



Address	Size	Owner	Section	Contains	Type	Access	Initial	Mapped
00380000	00001000				Priv	RWE	RWE	
00390000	00001000				Priv	RWE	RWE	
003A0000	00010000				Priv	RW	RW	
00400000	00001000	UnPackMe		PE header	Imag	R	RWE	
00401000	0004A000	UnPackMe	.teddy	code	Imag	R	RWE	
0044B000	0000C000	UnPackMe	.teddy		Imag	R	RWE	
00457000	00009000	UnPackMe	.teddy		Imag	R	RWE	
00460000	00003000	UnPackMe	.teddy		Imag	R	RWE	
00463000	00008000	UnPackMe	.teddy	resources	Imag	R	RWE	
0046B000	0000A000	UnPackMe	.teddy	SFX,imports	Imag	R	RWE	
00480000	00021000				Priv	RW	RW	
004B0000	00009000				Map	R E	R E	
00570000	00002000				Map	R E	R E	
00580000	00103000				Map	R	R	
00690000	00001000				Priv	RW	RW	
006A0000	00141000				Map	R E	R E	
009A0000	00001000				Priv	RW	RW	
009B0000	00004000				Priv	RW	RW	
009C0000	00003000				Map	R	R	\Device
009D0000	00001000				Priv	RW	RW	
00A50000	00004000				Priv	RW	RW	
00A60000	00003000				Priv	RW	RW	
00A70000	00002000				Map	R	R	
00A80000	0003C000				Priv	RW	RW	
00AC0000	00001000				Priv	RW	RW	
00BCE000	00002000				Priv	RW	RW	
01280000	00002000				Map	R	R	
58C30000	00001000	COMCTL32		PE header	Imag	R	RWE	

```
00AC0000  EB 02          JMP SHORT 00AC0004
00AC0002  0C 49          OR AL,49
00AC0004  EB 02          JMP SHORT 00AC0008
00AC0006  CD 20          INT 20
00AC0008  68 2F08817C    PUSH 7C81082F
00AC000D  EB 02          JMP SHORT 00AC0011
00AC000F  CD 20          INT 20
00AC0011  C3             RETN
00AC0012  8D6424 EC      LEA ESP,DWORD PTR SS:[ESP-14]
00AC0016  68 2100AC00    PUSH 0AC0021
00AC001B  E9 2C09D57B    JMP 7C81094C                    kernel32.7C810
00AC0020  0C C1          OR AL,0C1
00AC0022  F0:008B FFEB02 LOCK ADD BYTE PTR DS:[EBX+CD02EBFF],CL  LOCK prefix
00AC0029  2055 68        AND BYTE PTR SS:[EBP+68],DL
00AC002C  35 00AC00EB    XOR EAX,EB00AC00
00AC0031  010CC3         ADD DWORD PTR DS:[EBX+EAX*8],ECX
00AC0034  008B ECEB020C  ADD BYTE PTR DS:[EBX+C02EBEC],CL
00AC003A  49             DEC ECX
00AC003B  FF75 08        PUSH DWORD PTR SS:[EBP+8]
```

Теперь ищем ссылки.

Ничего, но у нас несколько секций, и знаем, что вызов, читающий значения из элемента 460f24 будет выглядеть как:

```
CALL [460f24]
```

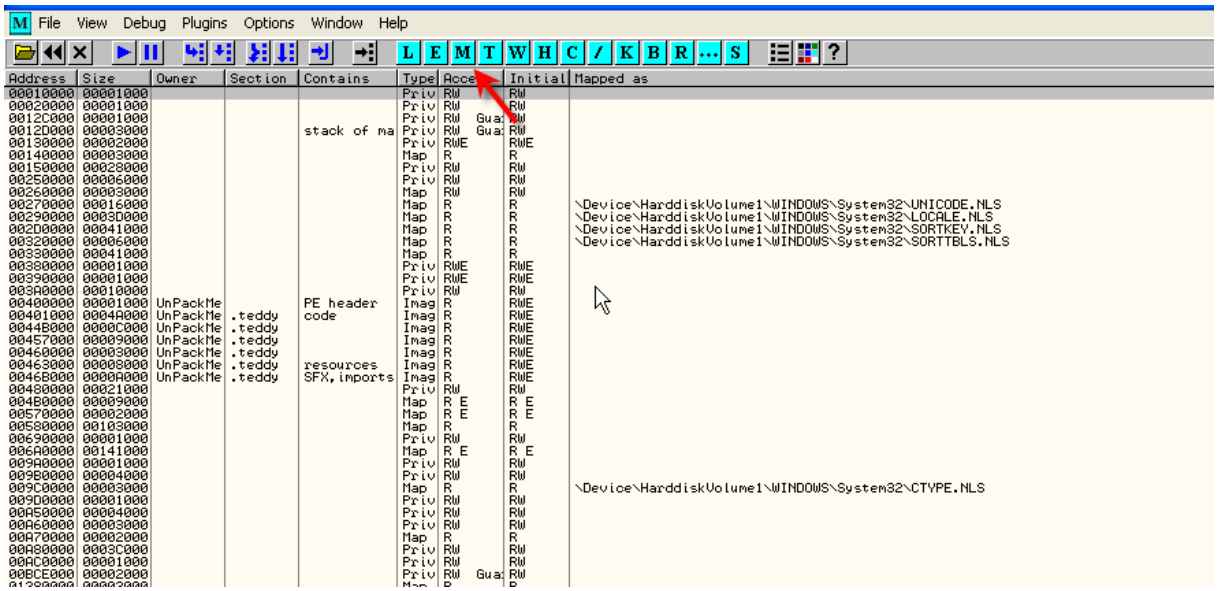
а байты, из которых состоит эта команда:

FF 15 24 0f 46 00

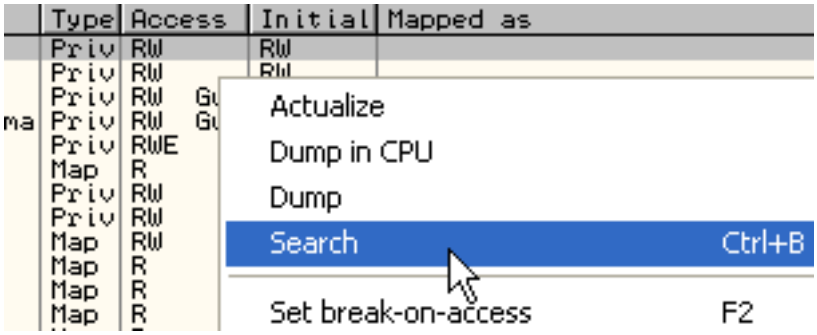
Например, если посмотрим другой вызов одной из API-функций:

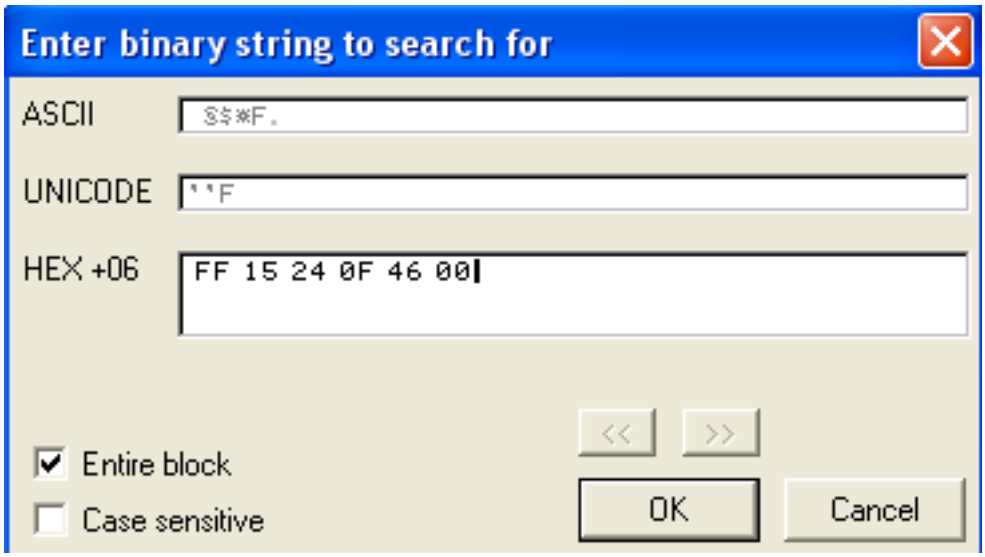
```
00A80037  - E9 6F1098FF    JMP 004010AB                    UnPackMe.004010AB
00A8003C    6F             OUTS DX,DWORD PTR ES:[EDI]      I/O command
00A8003D    FF15 30094600  CALL DWORD PTR DS:[4609330]     GDI32.DeleteObject
00A80043  - E9 871098FF    JMP 004010CF                    UnPackMe.004010CF
00A80048    87FF           XCHG EDI,EDI
00A8004A    15 9C0B4600    ADC EAX,460B9C
00A8004F  - E9 C41098FF    JMP 00401118                    UnPackMe.00401118
```

Видим, что последовательность байтов вызова равна «FF 15», а затем идёт адрес элемента в обратном порядке, так что можем поискать эти байты по всей памяти и таким образом выяснить, есть ли эта ссылка где-либо, что позволить нам избежать посекционного поиска.

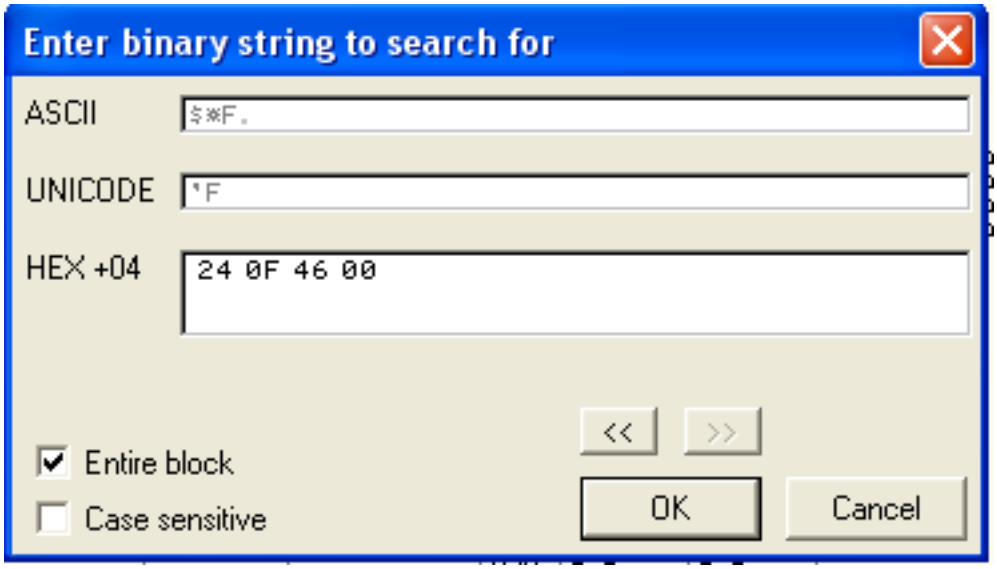


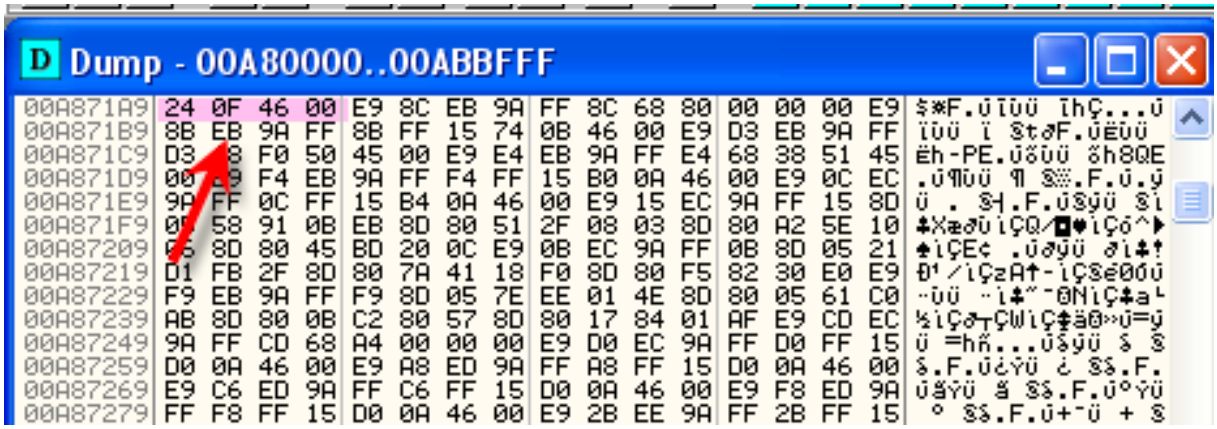
Идём в карту памяти и нажимаем на правую кнопку мыши.



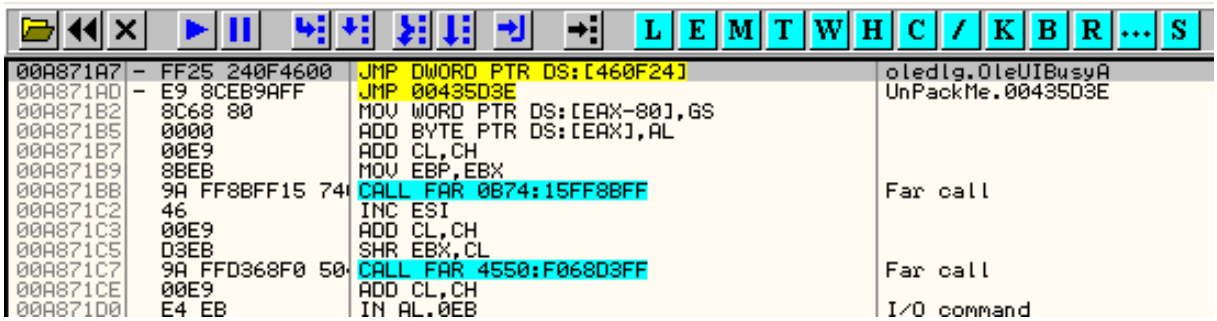


И пишем в поле «HEX» последовательность байтов, которую мы хотим найти. Нажимаем OK и не получаем результатов (grр), но так как мы настойчивые ребята, и есть множество возможностей вызвать API-функцию, не использующие косвенные вызовы, стираем «FF 15» и ищем в памяти только адрес.



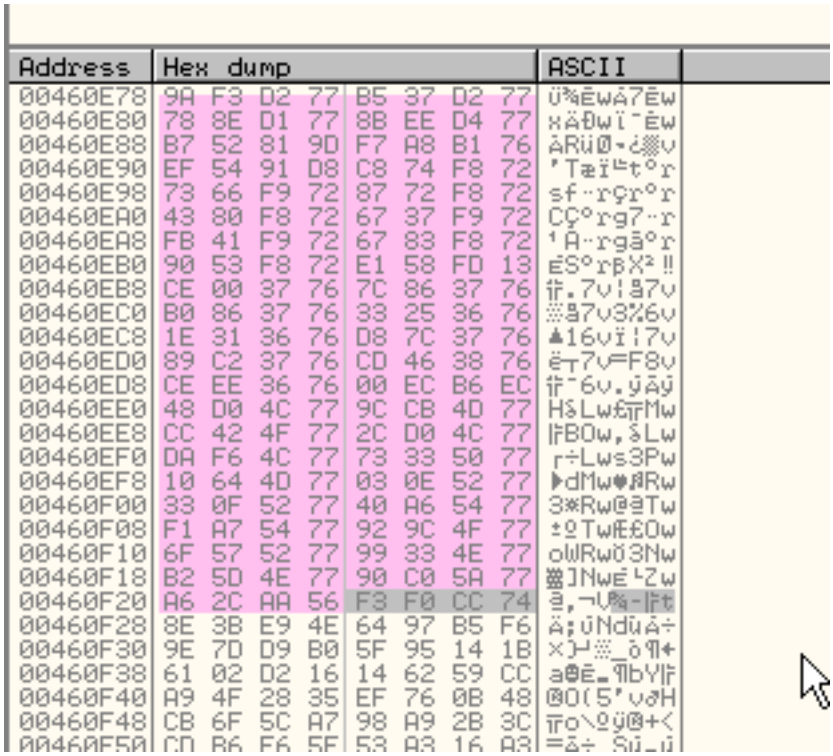


Находим что-то и как раз в одной из секций, созданных упаковщиком. Идём в листинг, чтобы посмотреть эту область.



Здесь находим эту чудесную ссылку, очевидно что мы пытались искать «FF 15» (косвенный вызов), но не «FF 25» (косвенный JMP), но ладно, проведя поиск по этому образцу мы получим ссылки на любую команду, которая вызывает API-функцию.

Таким образом, мы установили, что это элемент IAT.



И здесь IAT завершается, так как ниже, проведя поиск ссылок, я вижу, что нет значений, ведущих в секцию кода какой-либо DLL, таким образом, IAT заканчивается в 460f28.

Ок, данные из IAT равны:

LARGO=FINAL -INICIO= 460f28 - 460818= 710

Так что у нас получается следующее:

OEP=271B0
RVA o INICIO=60818
SIZE O LARGO=710

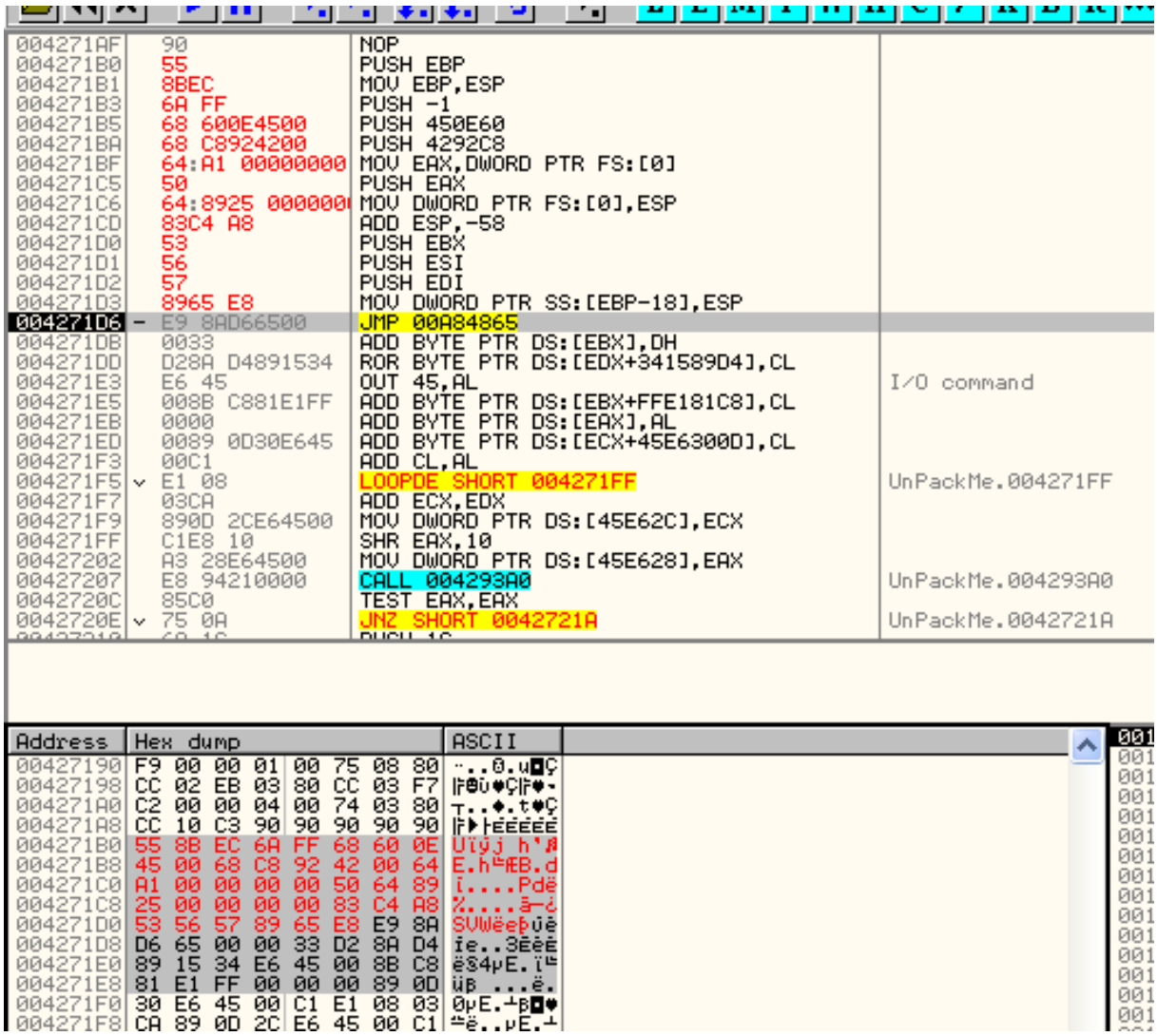
Теперь видим, что IAT правильная, так что нужно снова пойти в фальшивую OEP, добавить украденные байты и сделать дамп.

Идём в фальшивую OEP.

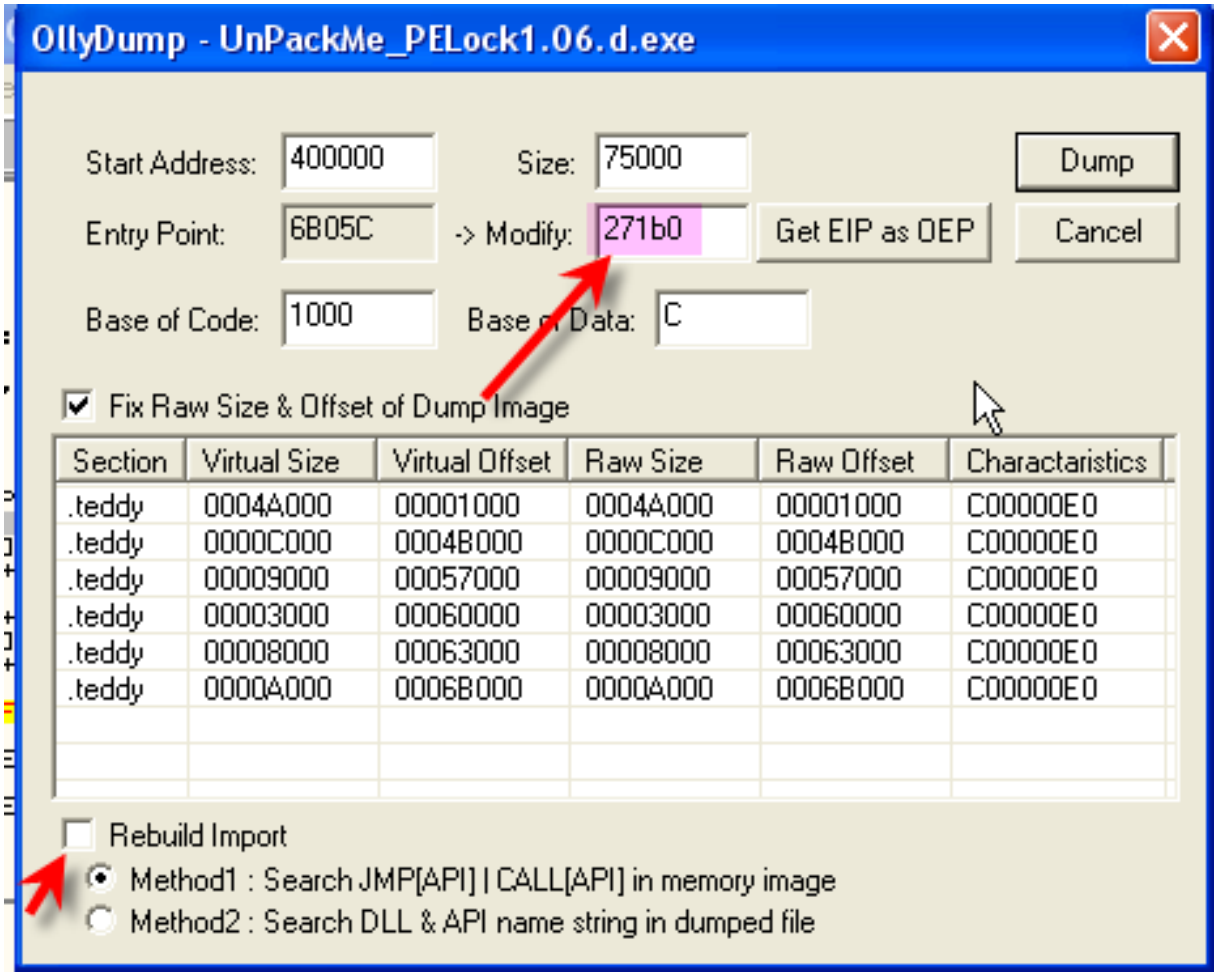
Украденные байты равны:

55 8B EC 6A FF 68 60 0E 45 00 68 C8 92 42 00 64 A1 00 00 00 00 50 64 89 25 00 00 00 00 83 C4 A8 53 56 57 89 65 E8

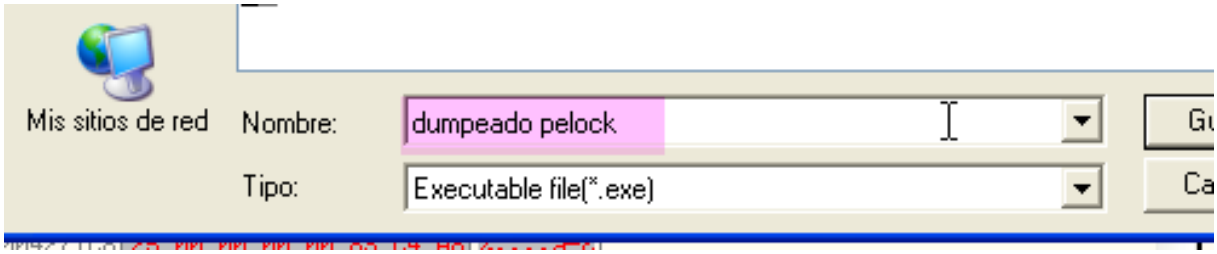
Прикрепляем их в дампе после 4271b0, как мы это делали в главе 39.



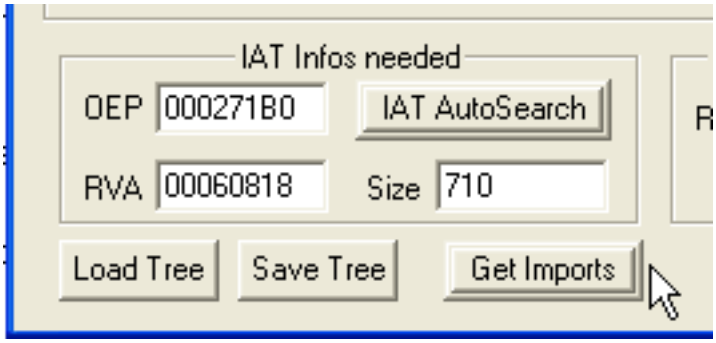
Теперь сдампим, изменив OEP на 4271b0.



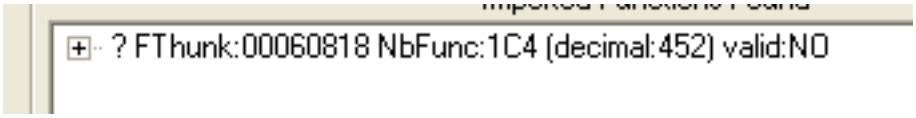
Здесь изменяем OEP, снимаем галочку с «REBUILD IMPORTS» и нажимаем «Dump», чтобы сделать дамп.



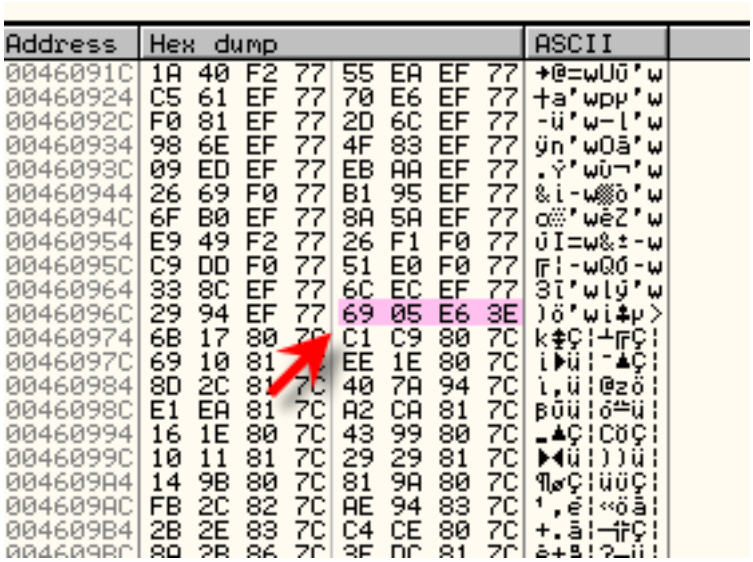
Сохраняем его как «dumpeado pelock», можем сейчас починить IAT, так как мы определили OEP. Открываем IMP REC.



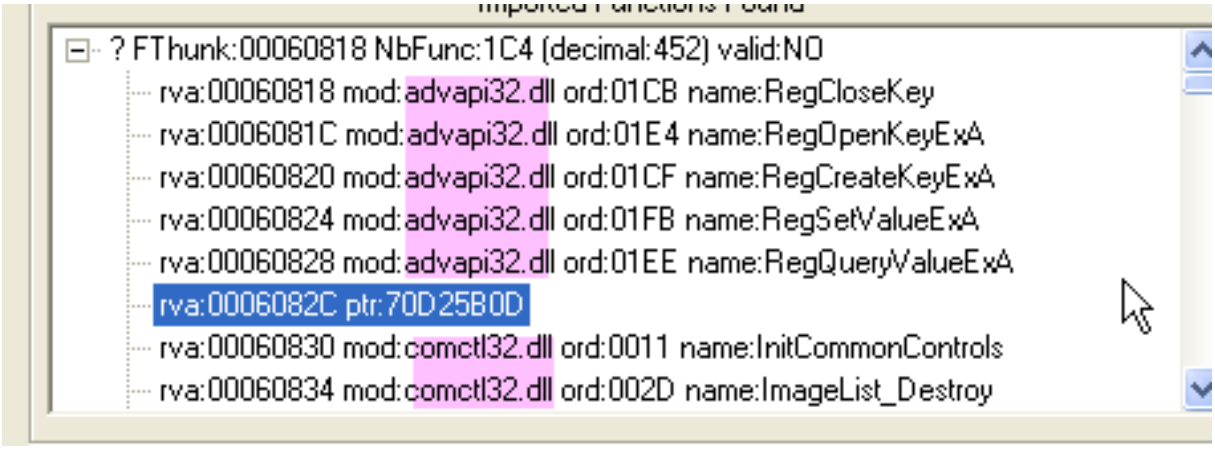
Устанавливаем данные IAT, найденные нами и нажимаем GET IMPORTS.



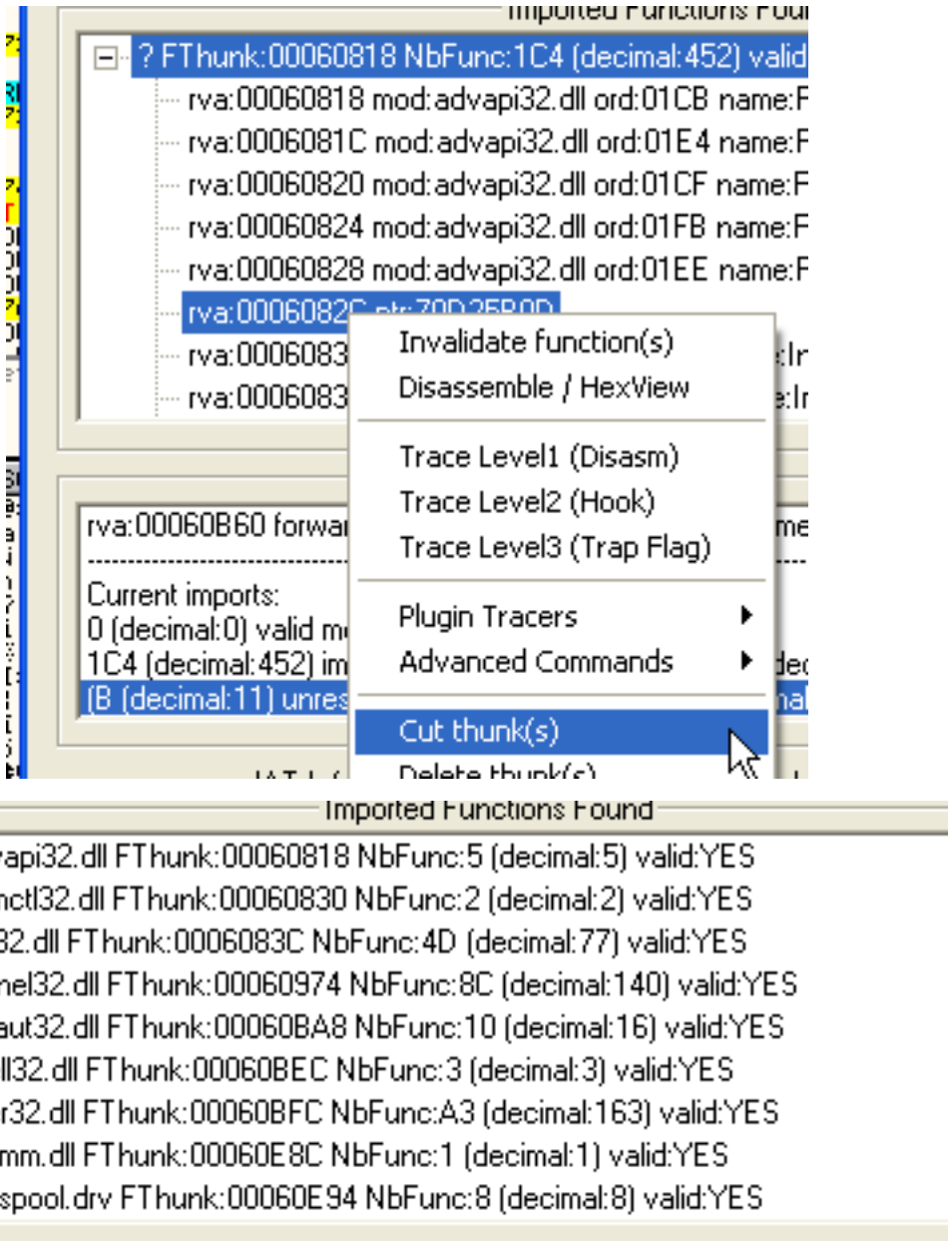
Говорит, что «NO», но давайте вспомним, что переадресовочных элементов нет. Проблема заключается в том, что упаковщик заменил нули, отделяющие элементы друг от друга на разный мусор. Давайте посмотрим.



Здесь ясно видно, что вместо нулей, отделяющих элементы для каждой из DLL, находится мусор. IMP REC легко это исправит, нажимаем SHOW INVALID.

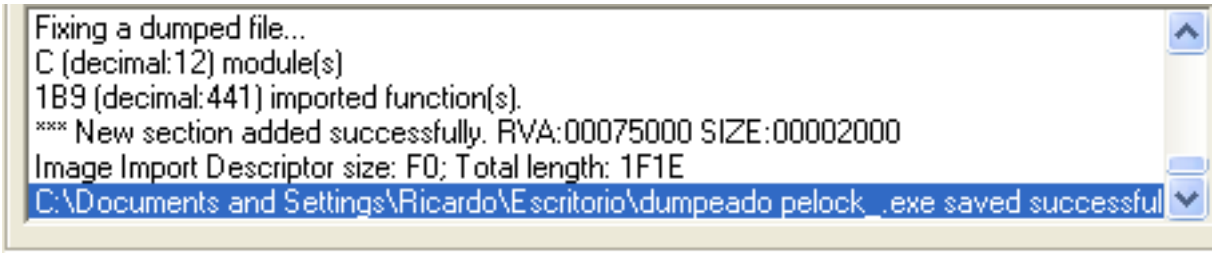


Вот подтверждение того, о чём мы говорили – элементы advapi32.dll и comctl32.dll отделены мусором, так что помечаем весь этот мусор, нажав SHOW INVALID, затем делаем правой кнопкой мыши выбираем CUT THUNKS, тем самым обнуляя все эти элементы.



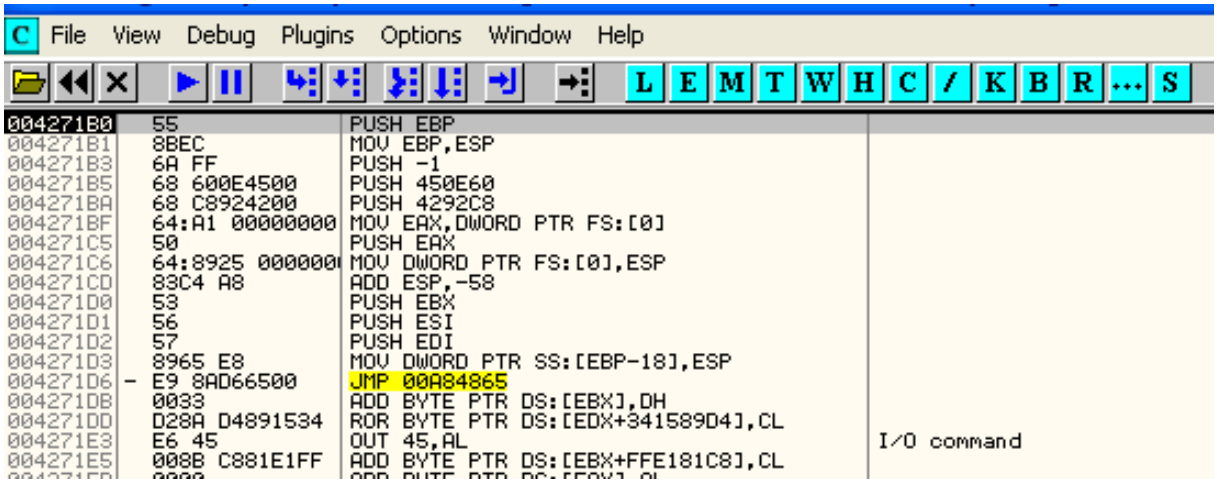
Как видим, после обнуления мусорных элементов, IMP REC показывает, что всё правильно и таблица починена.

Теперь нажимаем FIX DUMP для того, что починить дамп.

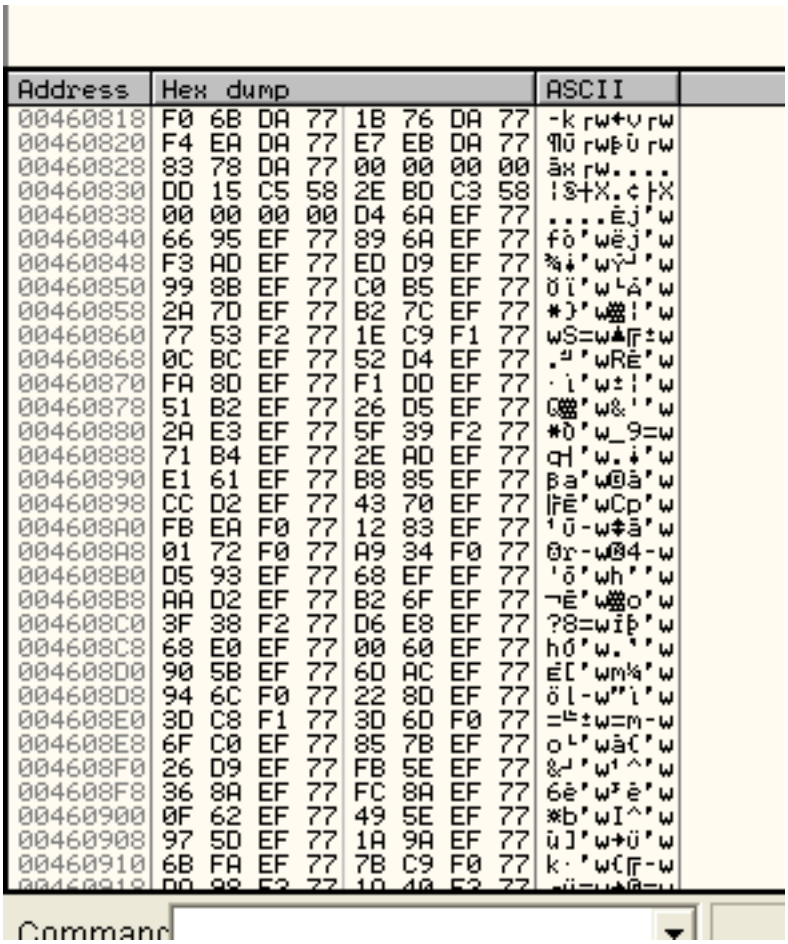


Создаётся «dumpeado pelock_.exe» с починенной IAT, и теперь, если запустим исправленный файл, то выдаст ошибку. Открываем его в OllyDbg без закрытия другого, который должен быть остановлен на OEP.

Так как выдал ошибку формата, передаём его LORDPE с опцией REBUILDPE, чтобы починить заголовок, и теперь можем видеть в Ollydbg результаты исправлений.



Видим, что остановлен на EP в правильном виде. Смотрим IAT.

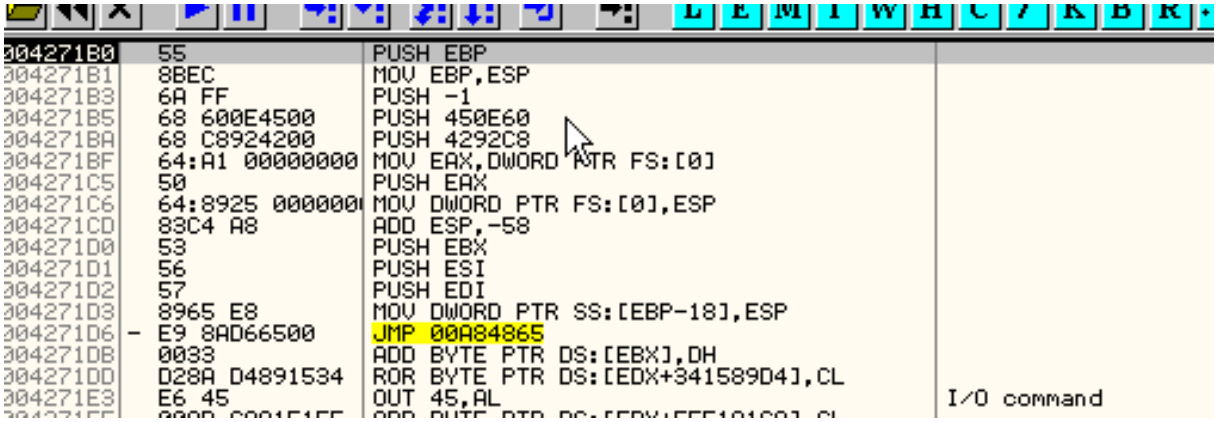


Здесь видно, что IAT была правильно починена IMP REC'ом. Видим, что он заменил мусорные значения нулями, для того, чтобы было нормальное разделение между различными DLL. Несомненно, что запустив программу тут же в OllyDbg, получим ошибку, что и происходит.

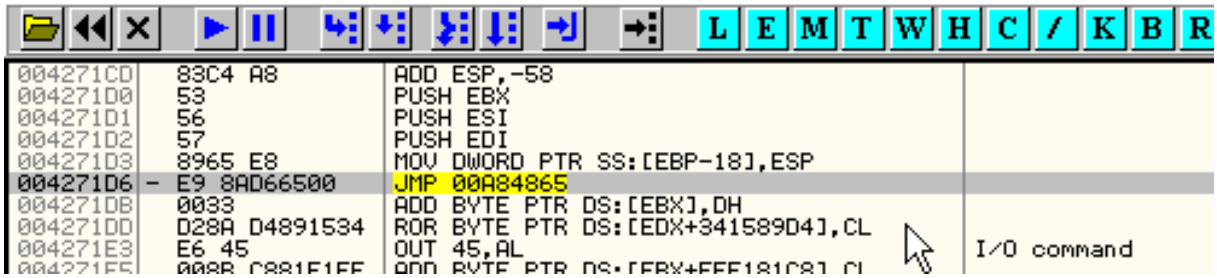
Мы дошли до следующего пункта, который необходимо изучить после починки дампа, IAT и украденных байтов. Этим трюком под названием “антидамп” пользуются многие упаковщики.

Есть много типов антидампа. Некоторые представляют собой просто проверку, совпадает ли размер программы или продолжительность секций, проверка, не добавил ли IMP REC секцию, которая всегда добавляется при починке IAT, но в данном случае антидампом являются различные секции, которые упаковщик создаёт во время выполнения и в которые перенаправляет некоторые части программы, и при распаковке этих секций нет, поэтому дамп не запускается, что ясно видно в

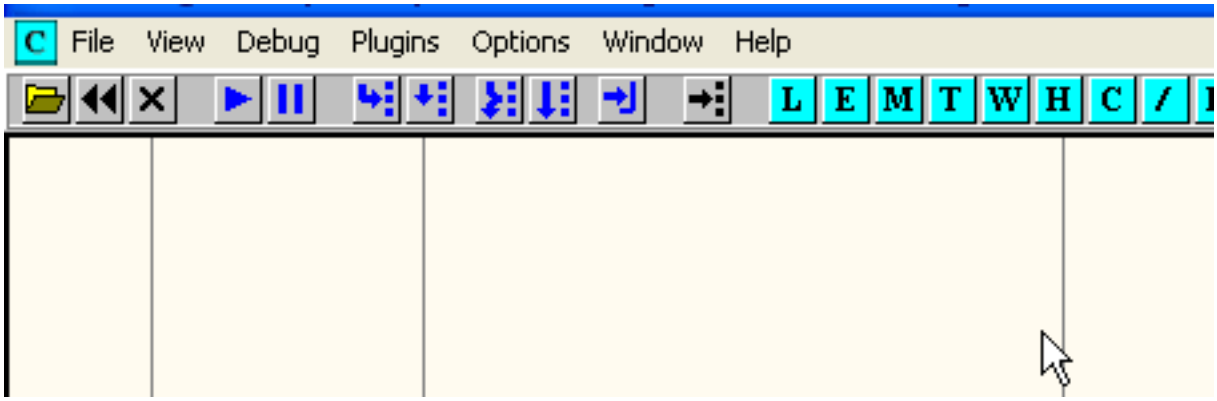
точке входа.



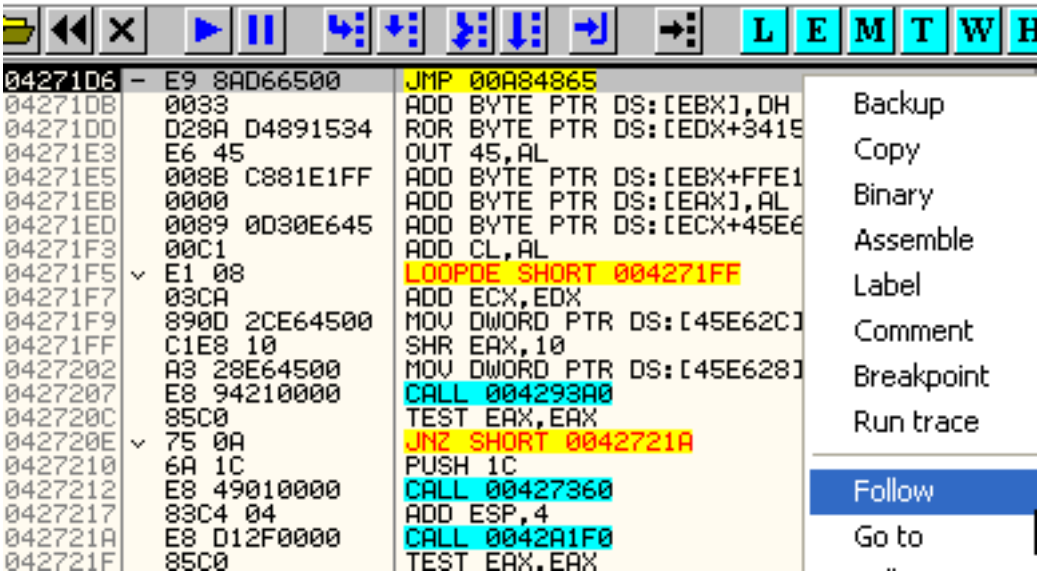
Программа запускается, но когда доходит до JMP, выдаёт ошибку. Трассируем досюда.



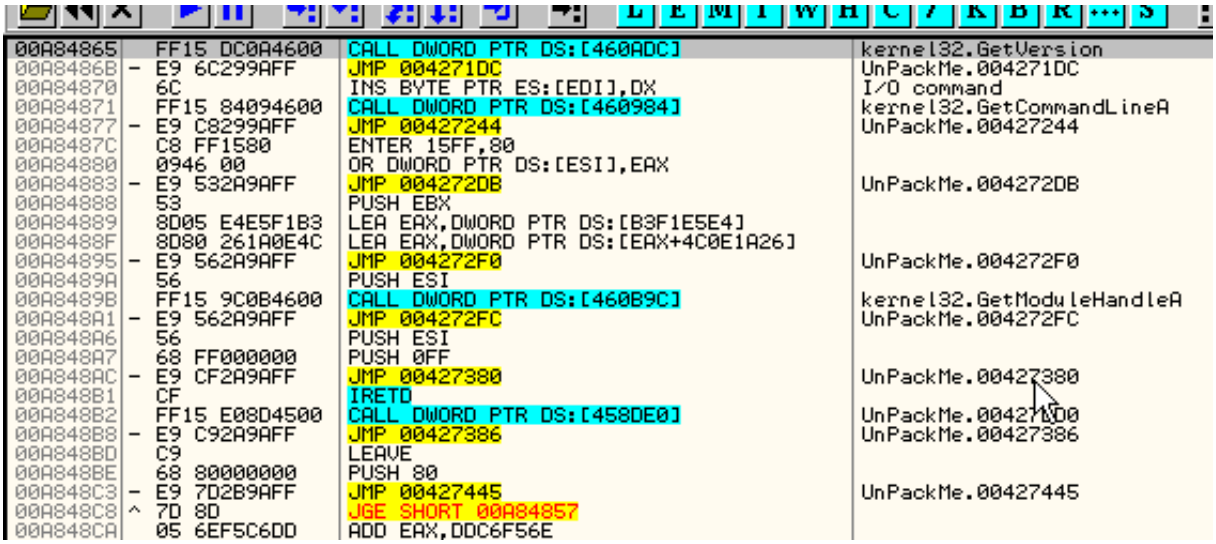
Видим, что если нажмём F7, то перейдём на несуществующую секцию.



И не можем продолжать. Если посмотрим на исходный файл, то увидим, что JMP ведёт в область, созданную упаковщиком во время выполнения.



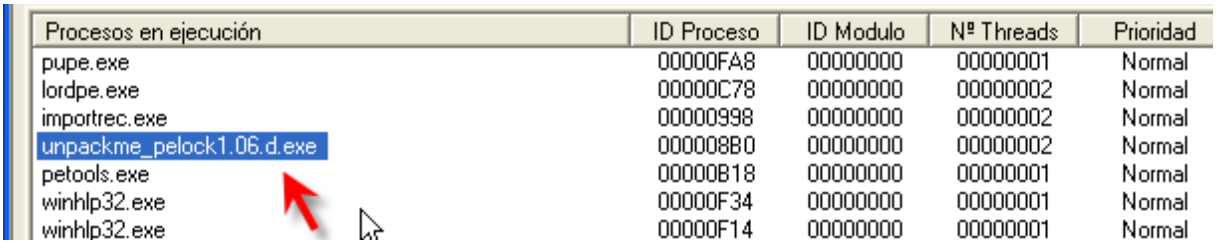
Делаем щелчок правой кнопкой мыши – FOLLOW, и отображается, куда ведёт JMP.



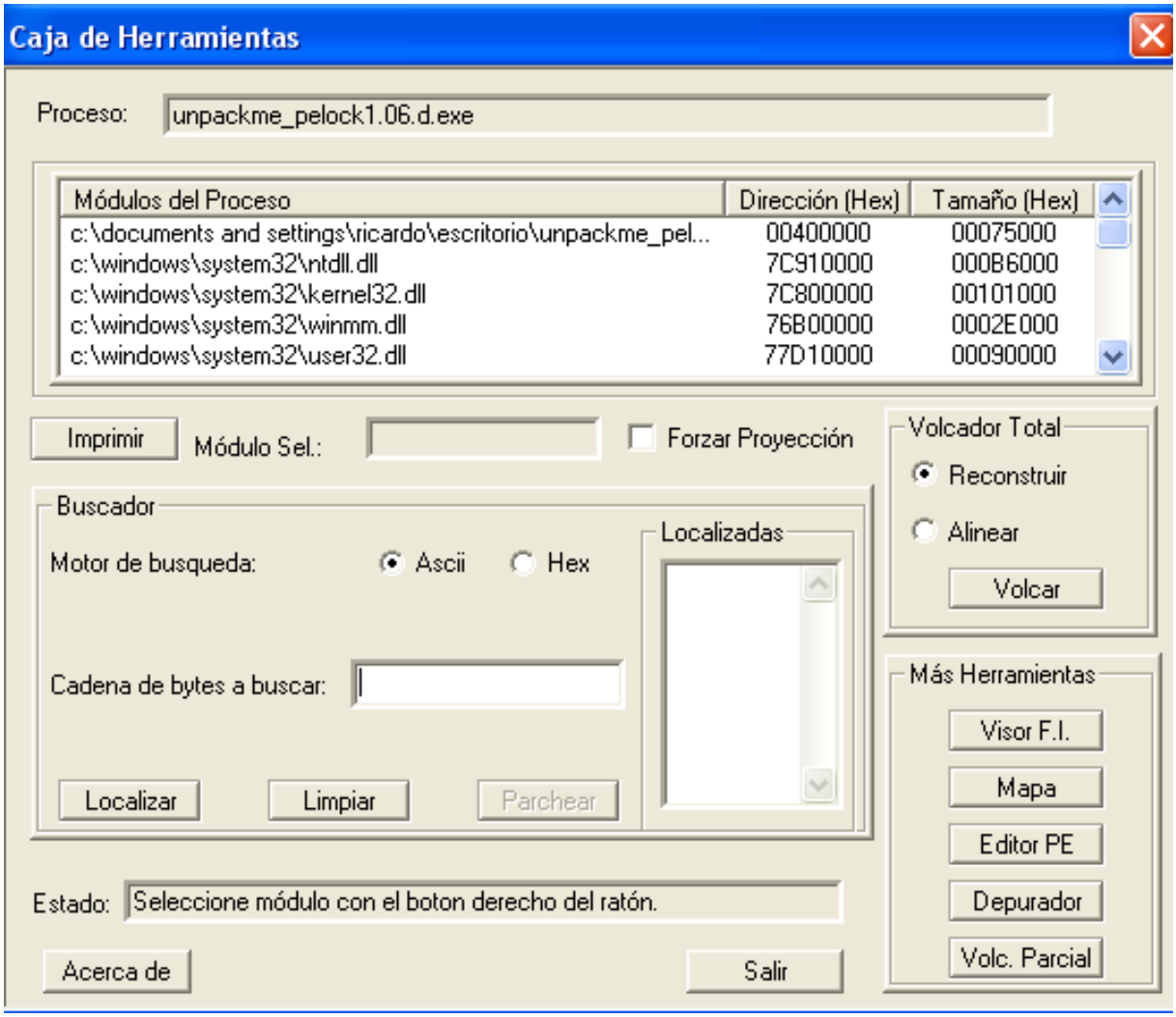
Здесь видим, что запускается CALL одной из API-функций, а потом происходит возврат в код программы. Есть много способов починить это, но наиболее простым будет добавить к дампу отсутствующую секцию и поместить её в то же место, как и в исходном файле. Идём-те глянем.

Смотрим отсутствующую секцию в исходной программе. Сначала добавим ту, из-за которой происходит ошибка в начале, потом, если будут снова ошибки, добавим другие.

Необходимо сдампить только ту секцию, из-за которой происходит ошибка, для чего используем “PUPE” -



Делаем щелчок правой кнопкой мыши - CAJA DE HERRAMIENTAS.

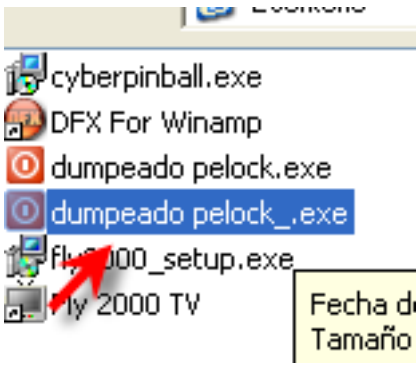


Открываем в ней нужный процесс, идём в карту памяти, чтобы сдампить отсутствующую секцию, для этого нажимаем кнопку MAPA (“карта” по-испански – прим. пер.)

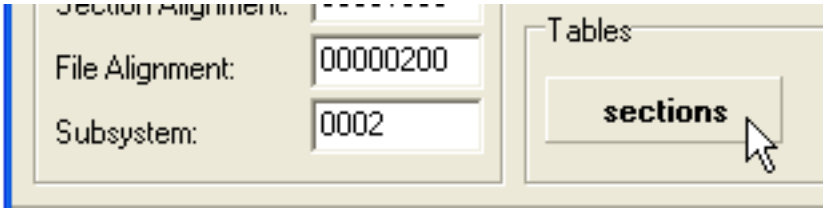
Вспоминаем, что отсутствующая секция начиналась в A80000, так что ищем этот адрес в карте памяти PUPE.

Теперь нажимаем VOLCAR и сохраняем под именем “секция”.

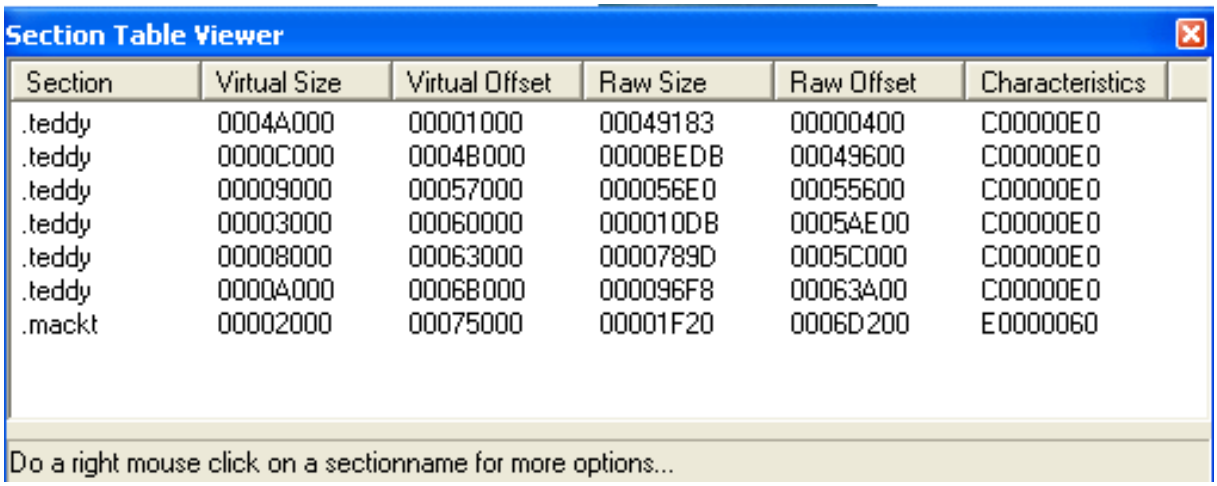
Теперь открываем PE EDITOR и добавляем её к дампу.



Ищем починенный дамп и открываем его в PE EDITOR'e.

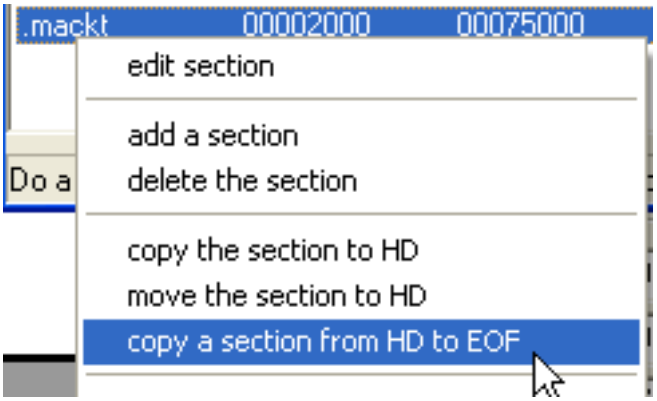


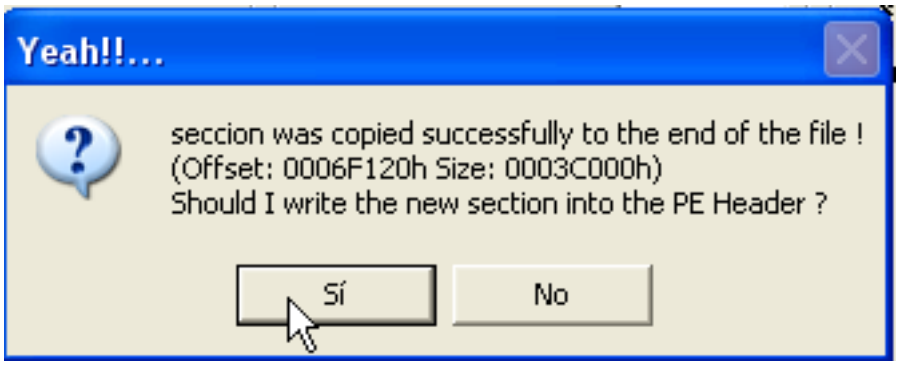
Нажимаем кнопку “sections”.

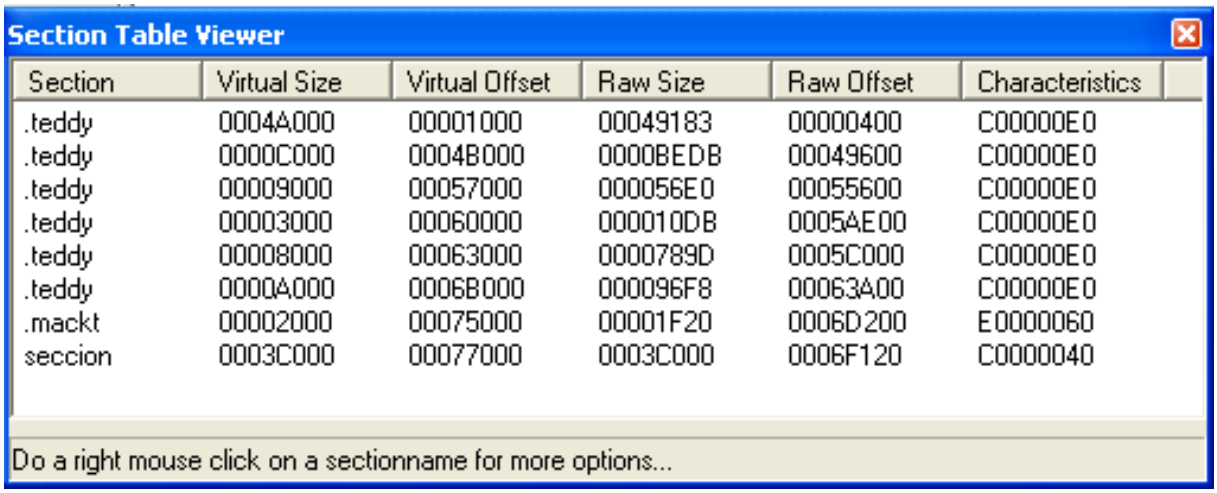


Здесь видим секции дампа, которые перешли туда из исходного файла, ещё секцию mackt, которую присоединяет IMP REC, чтобы починить IAT, теперь, наконец, добавляем отсутствующую секцию.

Делаем щелчок правой кнопкой мыши и выбираем COPY A SECTION FROM HD TO EOF (конец файла).

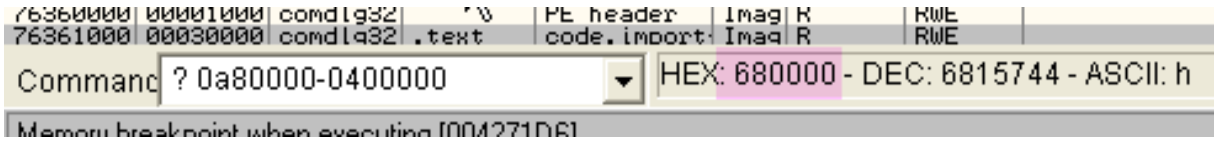




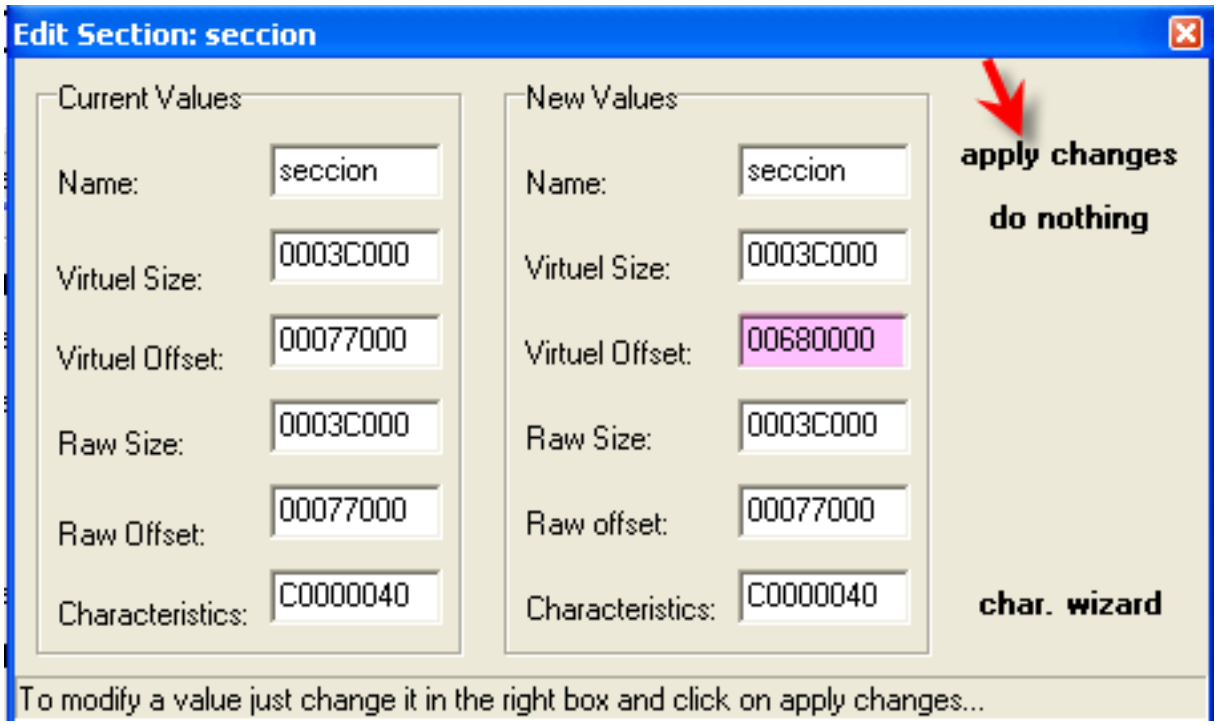


Теперь нам осталось только поменял VIRTUAL OFFSET на 0A80000, чтобы она запускалась отсюда, как и в исходном файле.

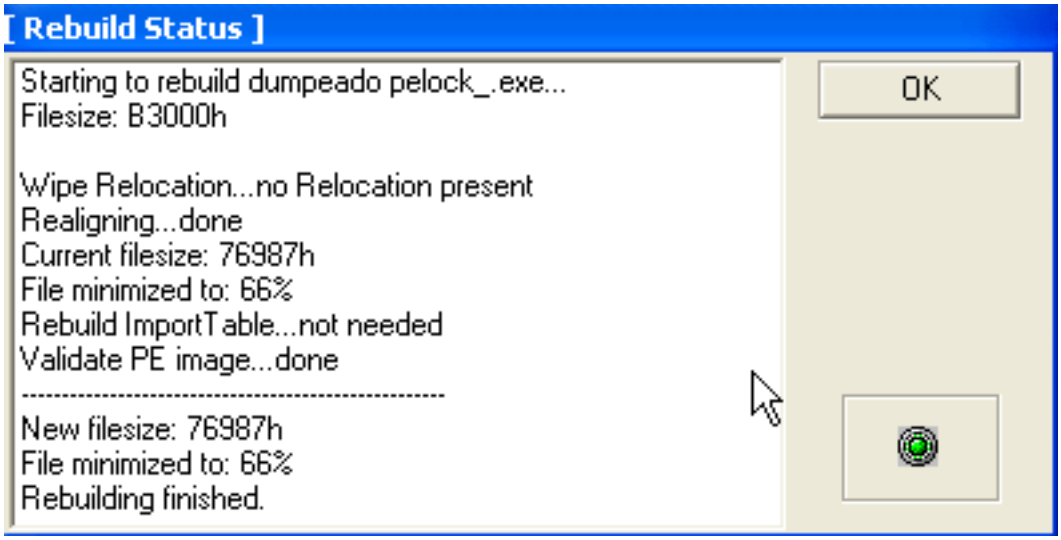
Меняем виртуальное смещение на 0A80000, отняв 400000, остаётся:

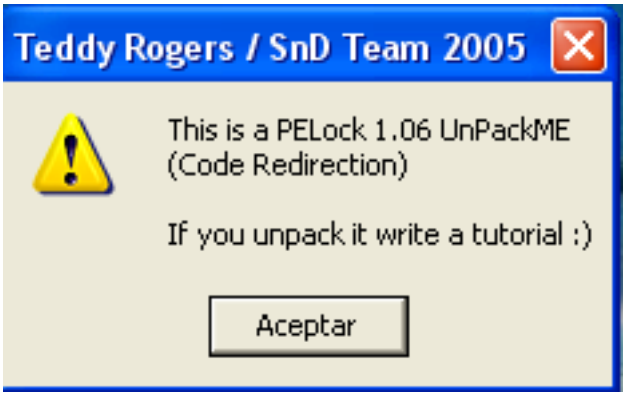


680000 является адресом виртуального смещения новой секции.

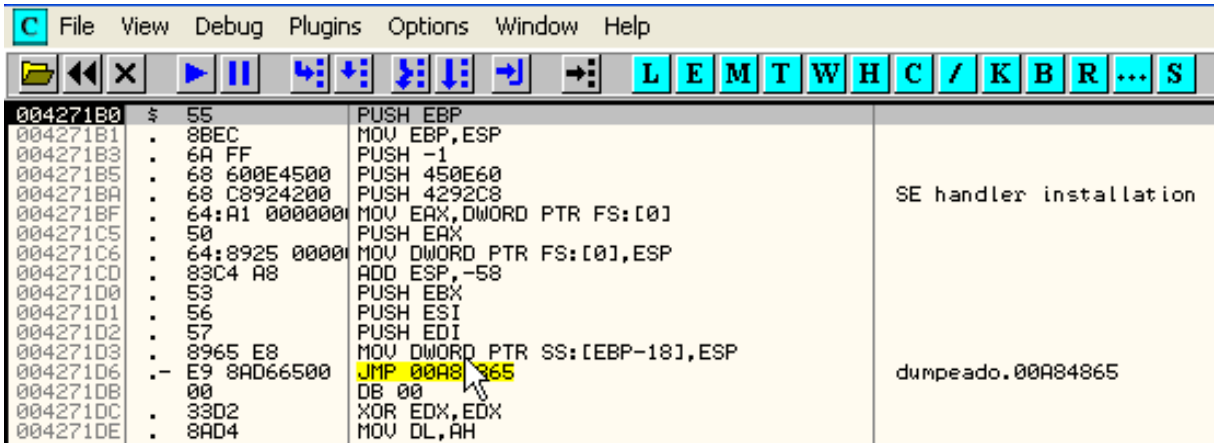


Теперь остаётся один шаг – снова починить заголовок с помощью LordPE, используя опцию REBUILD PE.

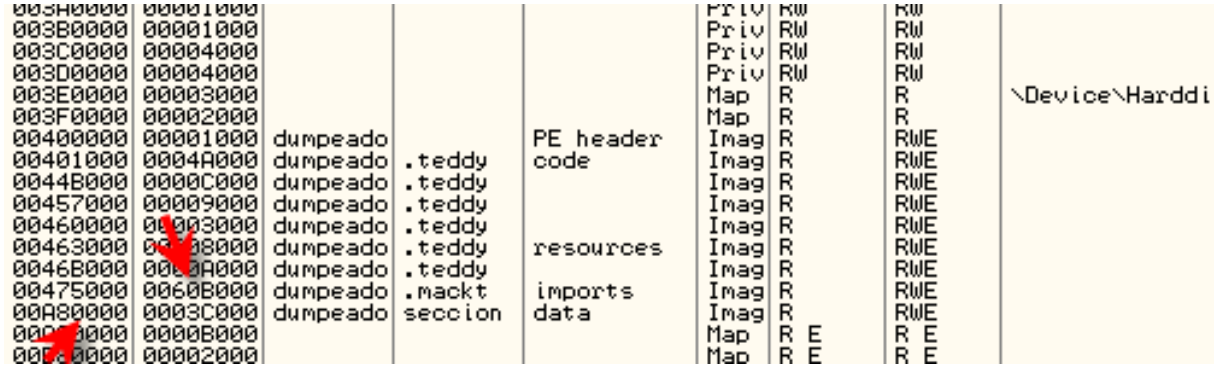




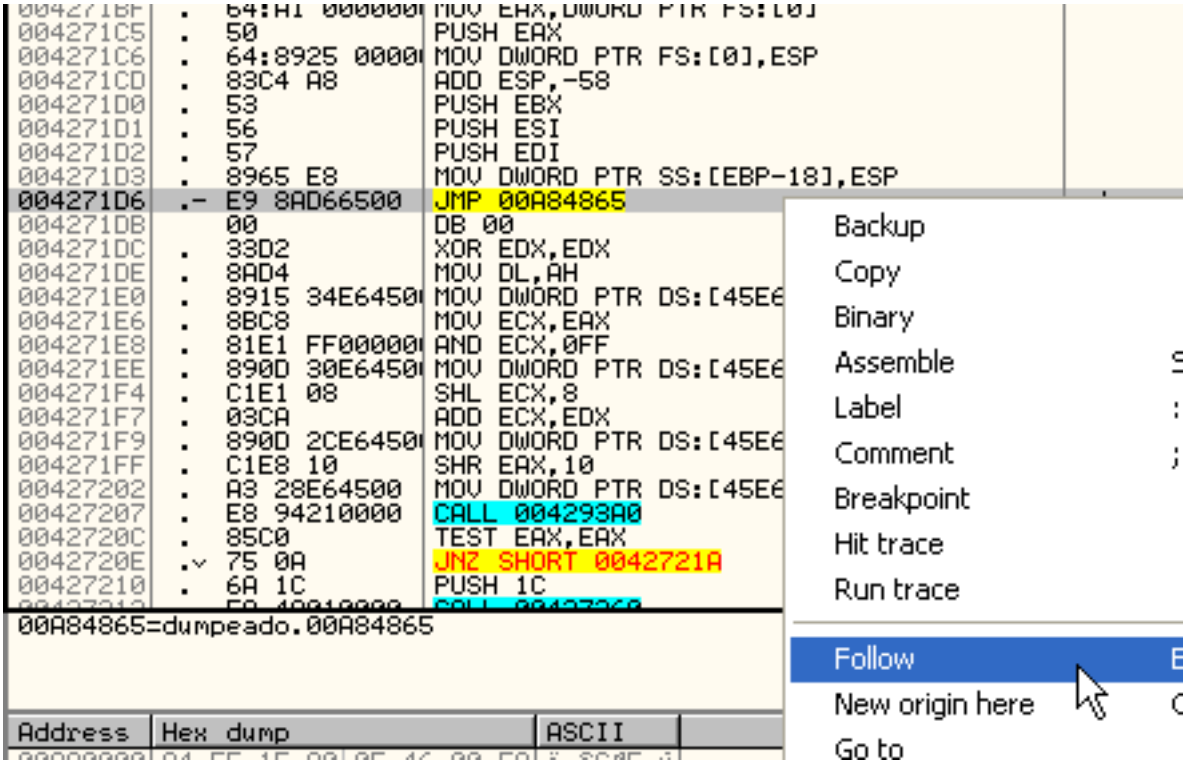
Запускаем и видим, что работаем, если открыть в OllyDbg.



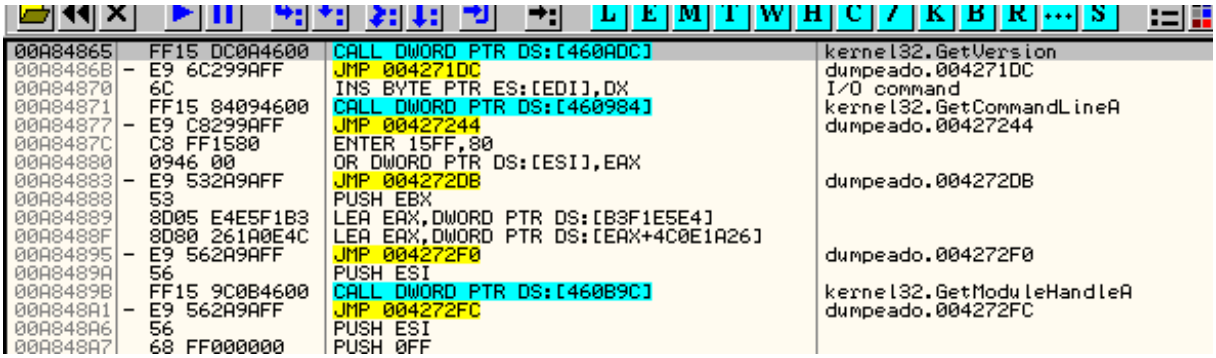
Смотрим карту памяти.



Видим, что LordPE, чтобы сделать дамп рабочим, увеличил предыдущую секцию, чтобы не было “провалов” между ней и той, которую добили с помощью PUPE.



Если посмотрим переход, то увидим, что сейчас у нас есть ранее отсутствовавший код.



Если образ оказывается слишком велик, то можем пропатчить всё, что нам нужно, а потом упаковать с помощью UPX, который уменьшит файл, но оставит при этом работоспособным, а также может распаковать его обратно. Если запустим GUIPEX, то там есть опции упаковки и распаковки. В данном случае это не является необходимым, так как дамп после починки и с добавленными секациями занимает меньше 500k.

Другой возможностью для тех, кто хочет изучть другой способ исправления антидампа, является добавить отсутствующую секцию или секции в продолжение последней, что не слишком сильно увеличит размер exe-файла, а затем поменять OEP в дампе, сделать вставку, чтобы расположить её в позиции созданной с помощью VirtualAlloc и скопировать байты, которые мы добавили туда. Этот метод очень хорошо изложен marcian'ом в одном из последних состязаний crackslatinos.

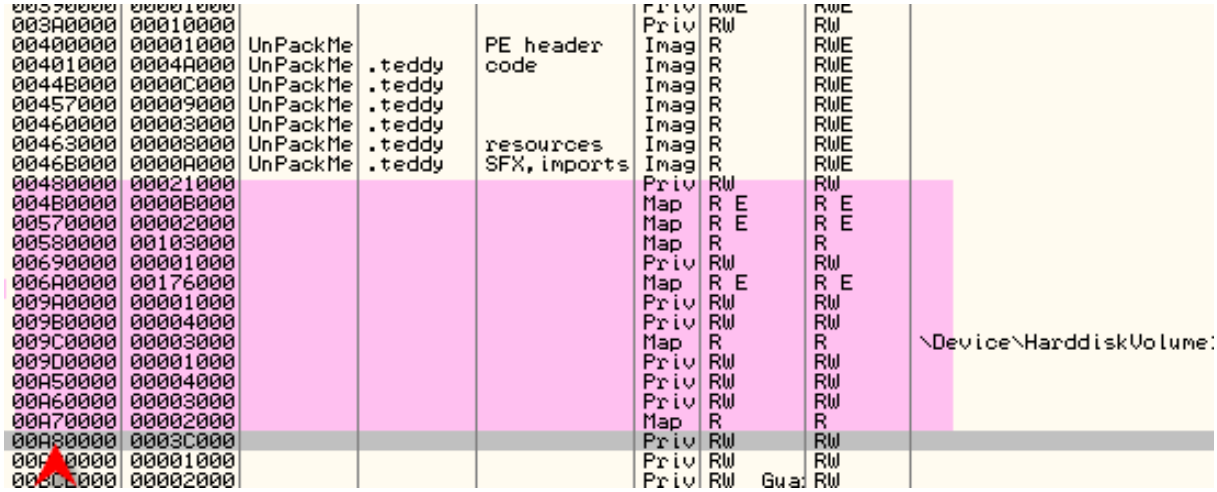
В этом pelock'овском распакуй-меня, который похож на тот, что мы изучали, IAT является переадресовочной. Я хотел починить её скриптом, но в нашем случаен не было переадресовочной IAT, так что применим скрипт в будущем, так как есть тысячи упаковщиков, где это может понадобиться.

marciano применяет другой метод, заключается в том, чтобы присоединить отсутствующую секцию вместо изменения Virtual Offset. Я много раз применял этот метод и он очень хорош, хотя если отсутствующих секций много, то добавлять их одну за одной несколько утомительно. С помощью метода добавления, используя LordPE, задействуется вся память между последней и добавленной секциями.

Таким образом, с помощью метода PeEditor'a первой удобно добавлять самую дальнюю секцию, так как после починки с помощью LordPE, вся память между последней секции и началом вставленной секции будет выделена без каких-либо проблем. Затем мы можем скопировать байты из исходной программы, остановленной на OEP в дамп, а затем вносить и сохранить изменения в нём, если будет отсутствовать какяя-то из “серединных” секций.

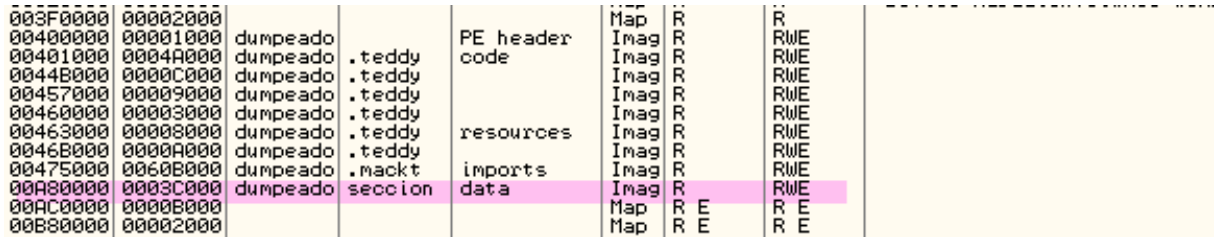
Думаю, что это хорошо понятно на нашем примере. Если самая дальняя добавляемая секция начинается в 0A80000, то LordPE увеличит последнюю секцию дампа так, чтобы она заканчивалась как раз у 0A80000, поэтому если надо будет добавить секцию, находящую до этого места, можно будет легко скопировать и вставить её, а потом сохранить изменения.

ИСХОДНАЯ КАРТА ПАМЯТИ



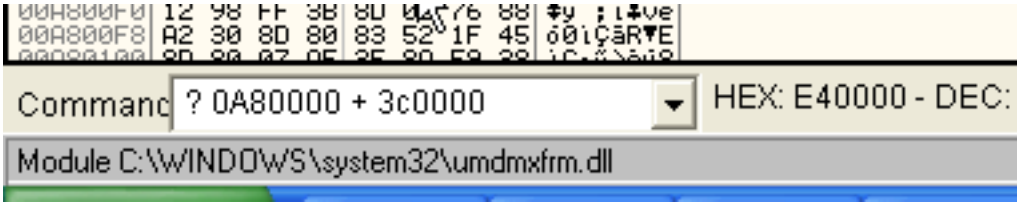
003A0000	00010000				Priv	RW	RW	
00400000	00001000	UnPackMe		PE header	Imag	R	RWE	
00401000	0004A000	UnPackMe	.teddy	code	Imag	R	RWE	
0044B000	0000C000	UnPackMe	.teddy		Imag	R	RWE	
00457000	00009000	UnPackMe	.teddy		Imag	R	RWE	
00460000	00003000	UnPackMe	.teddy		Imag	R	RWE	
00463000	00008000	UnPackMe	.teddy	resources	Imag	R	RWE	
0046B000	0000A000	UnPackMe	.teddy	SFX,imports	Imag	R	RWE	
00480000	00021000				Priv	RW	RW	
004B0000	0000B000				Map	R E	R E	
00570000	00002000				Map	R E	R E	
00580000	00103000				Map	R	R	
00690000	00001000				Priv	RW	RW	
006A0000	00176000				Map	R E	R E	
009A0000	00001000				Priv	RW	RW	
009B0000	00004000				Priv	RW	RW	
009C0000	00003000				Map	R	R	\Device\HarddiskVolume:
009D0000	00001000				Priv	RW	RW	
00A50000	00004000				Priv	RW	RW	
00A60000	00003000				Priv	RW	RW	
00A70000	00002000				Map	R	R	
00A80000	0003C000				Priv	RW	RW	
00AC0000	00001000				Priv	RW	RW	
00BC1000	00002000				Priv	RW Gua	RW	

КАРТА ПАМЯТИ ДАМПА



003F0000	00002000				Map	R	R
00400000	00001000	dumpeado		PE header	Imag	R	RWE
00401000	0004A000	dumpeado	.teddy	code	Imag	R	RWE
0044B000	0000C000	dumpeado	.teddy		Imag	R	RWE
00457000	00009000	dumpeado	.teddy		Imag	R	RWE
00460000	00003000	dumpeado	.teddy		Imag	R	RWE
00463000	00008000	dumpeado	.teddy	resources	Imag	R	RWE
0046B000	0000A000	dumpeado	.teddy		Imag	R	RWE
00475000	0060B000	dumpeado	.mackt	imports	Imag	R	RWE
00A80000	0003C000	dumpeado	seccion	data	Imag	R	RWE
00AC0000	0000B000				Map	R E	R E
00B80000	00002000				Map	R E	R E

Здесь видно, какие секции исходной программы, предшествующие A80000, существуют в дампе, и для них найдётся место, так как LordPE увеличил секцию mackt до места, куда присоединили недостающую секцию. Суммарный размер затребованной памяти составил с 401000 до A80000 + 3C0000 (размер последней серции) = 0E40000.



То есть любую секция, которая находится между 401000 и E40000 мы, если необходимо, сможем добавить в дамп.просто скопировав и вставив её из исходной программы, остановленной на OEP,

сохранив затем изменения.

Рассмотрим примеры, используя оба метода, так как оба полезны, так что вам нужно прочитать оба туториала – этот и marciano, и практиковать оба, поэтому бывают случаи, когда лучше использовать один из них, а в других – другой, в зависимости от обстоятельств.

Файлы (находится в архиве `wasm_files.zip`: `files/0511/ollydbg41.zip`) к статье.

Введение в крэкинг с нуля, используя OllyDbg - Глава 42

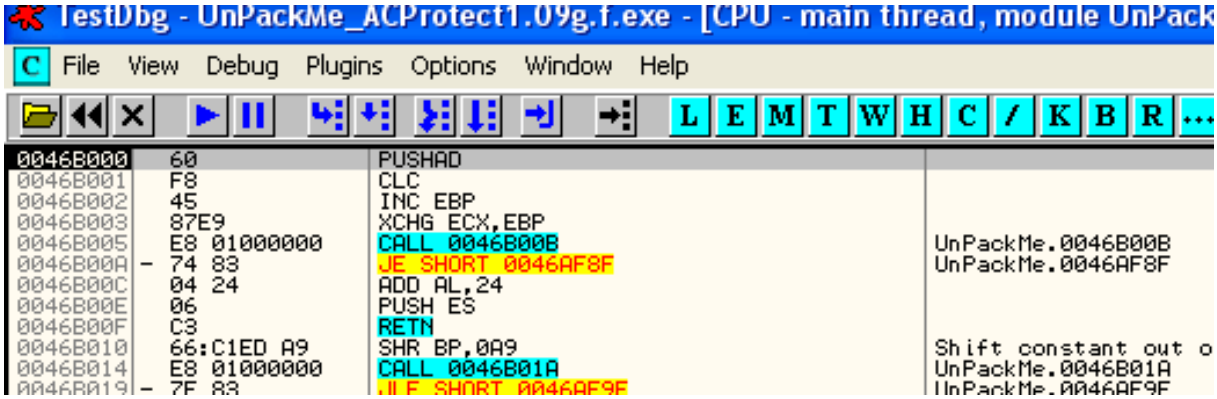
Автор: Рикардо Нарваха, пер. Aquila

Дата: 2009-05-14

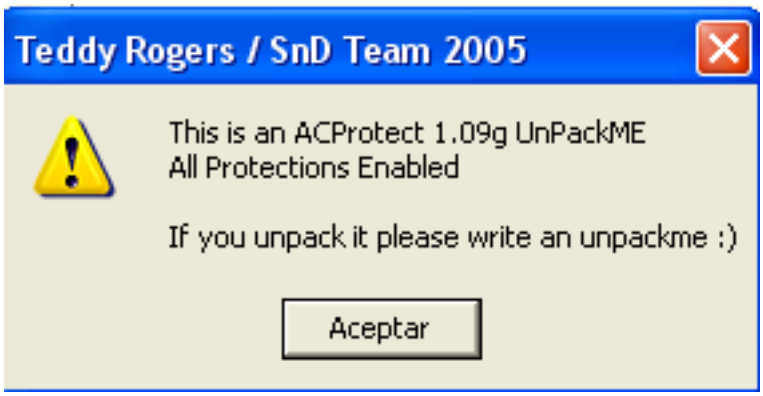
Ок, теперь мы перейдём к более сложным упаковщикам и увидим, что используемые методы могут быть очень разными в зависимости от случая, поэтому у нас должен быть большой запас различных трюков. В этот раз нас будет занимать ACPROTECT 1.09, распакуй-меня со всякого рода защитами.

В этом распакуй-меня есть всего понемножку, поэтому пойдём медленно, анализируя каждую часть защиты.

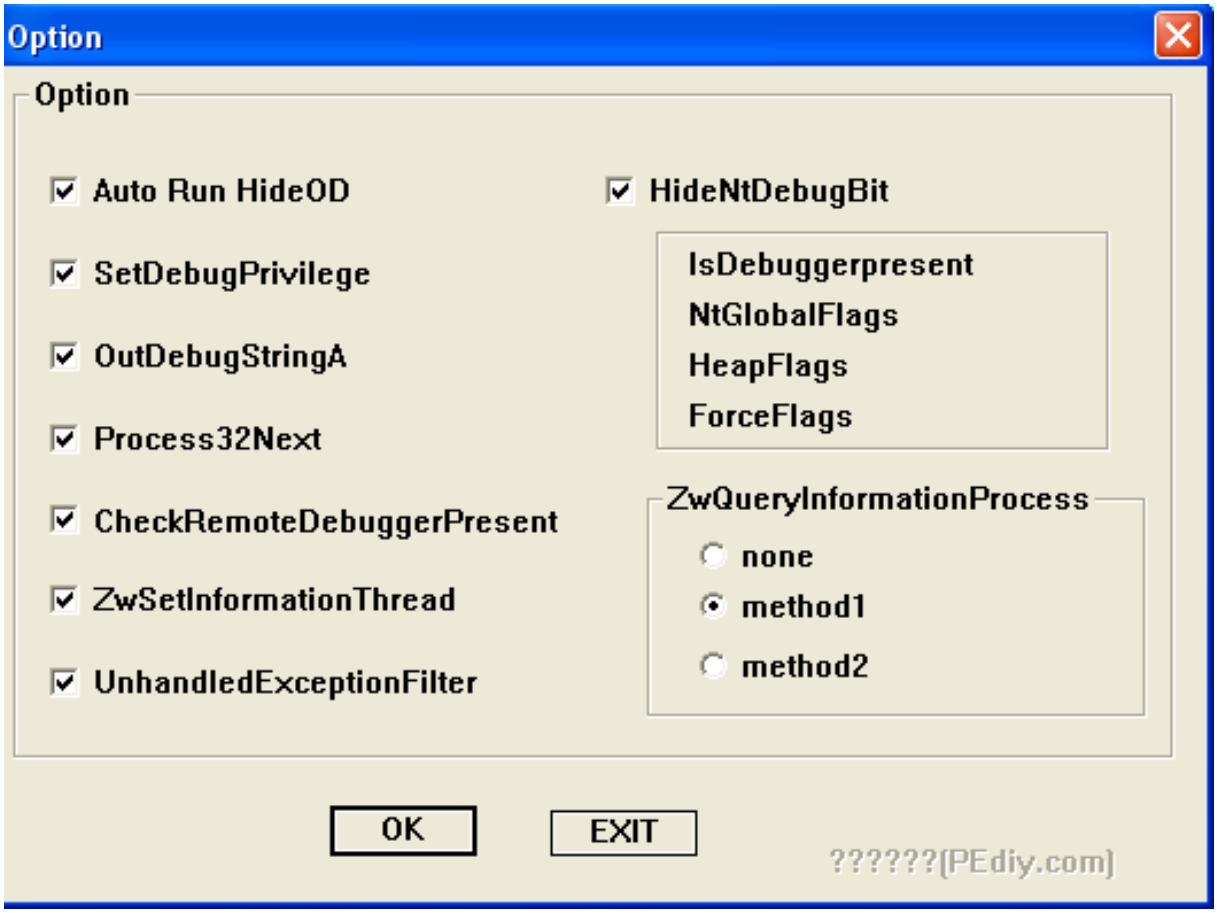
Распакуй-меня приложен к данной статье, так что здесь нет никакой проблемы: открываем его в пропатченной OllyDbg с установленными плагинами для её скрытия.



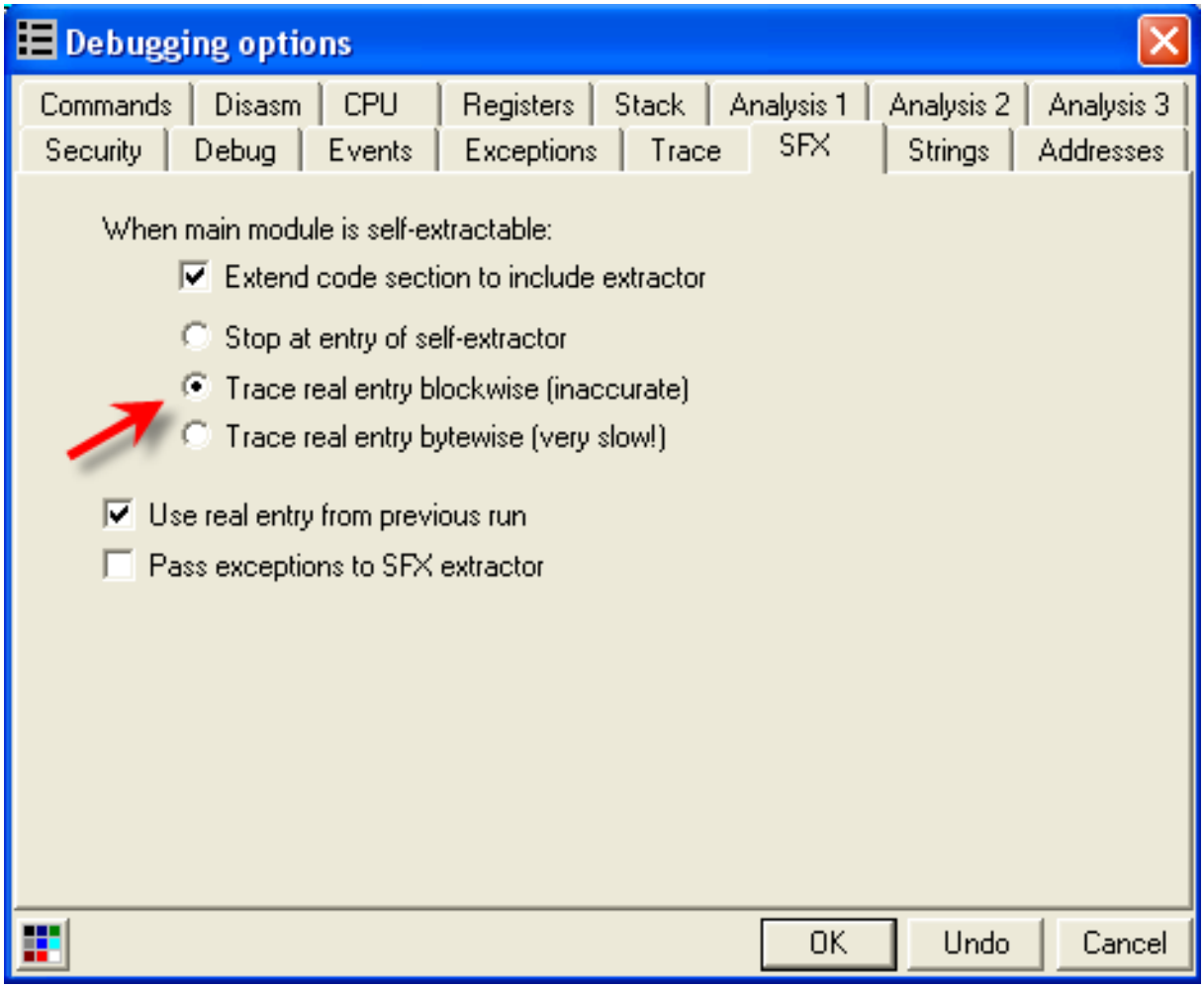
Вот он открыт в OllyDbg, и так как все защиты должны работать, проверить, запускается ли в OllyDbg до того, как мы что-нибудь сделаем с этим распакуй-меня.



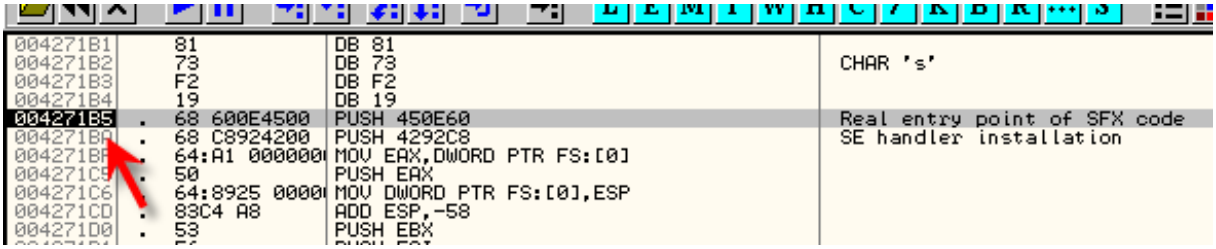
Ок, в моём parcheado4, который был приложен к предыдущей главе со всеми отмеченными исключениями и используя плагин HideOD 0.12 с конфигурацией, указанной на картинке ниже, нет никакой проблемы и всё прекрасно запускается. Вспомним, что этот упаковщик проверял с помощью API Process32Next, какие процессы открыты, и если обнаруживал, что он был открыт не с помощью двойного щелчка, то есть в отладчике или загрузчике, то не запускался, но плагин HideOD пытается защищаться от этого трюка и аннулирует антиотладку без каких-либо хлопот с нашей стороны.



Так что нам не приходится думать об этом, поэтому можем сконцетрироваться на нахождении OEP.

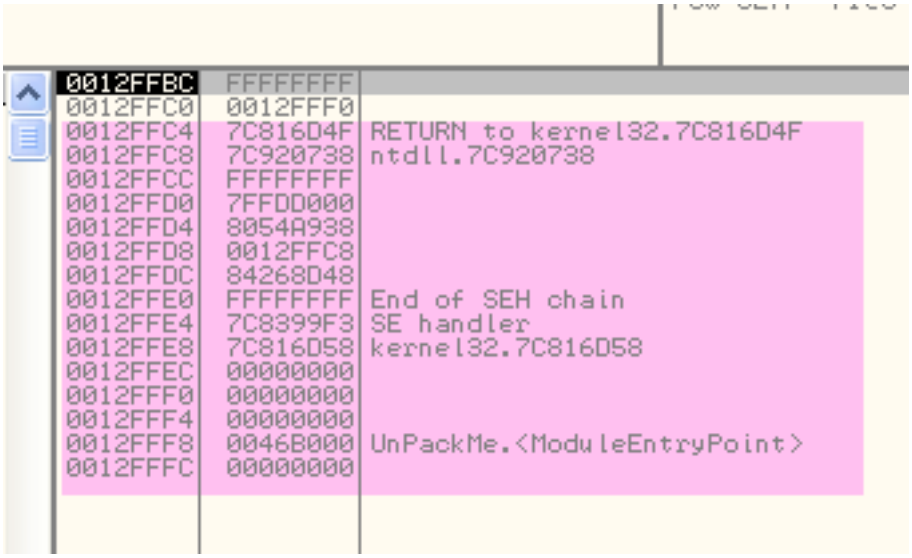


Из разных методов, которых так много используется для нахождения OEP в OllyDbg, используем встроенный поисковик OEP'ов, так что идём во вкладку SFX и устанавливаем галочку на «TRACE REAL ENTRY BLOCKWISE», нажимаем «OK» и перезапускаем OllyDbg, которая через некоторое время остановится здесь.

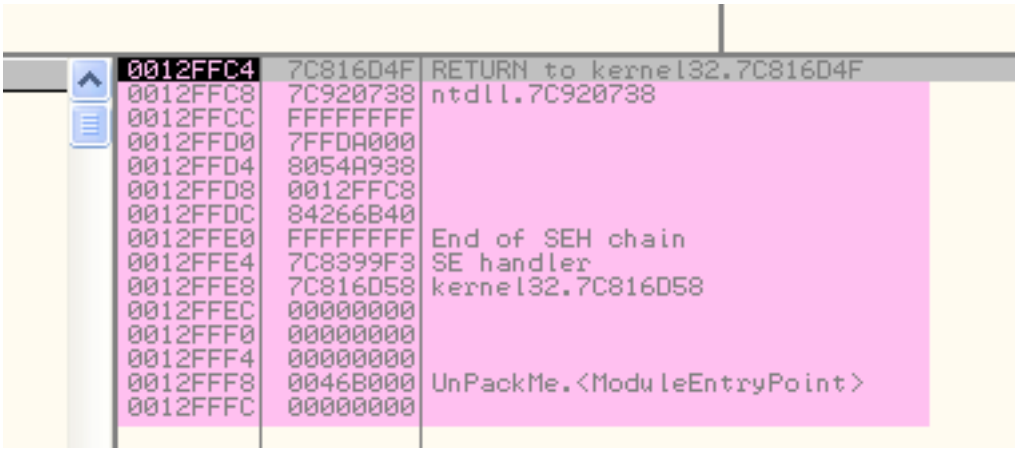


Теперь вопрос заключается в том, настоящий ли это OEP, или же это ситуация с украденными байтами.

Посмотрим в этот момент на стек.



Если уберём галочку поиска OEP'а и перезапустим:



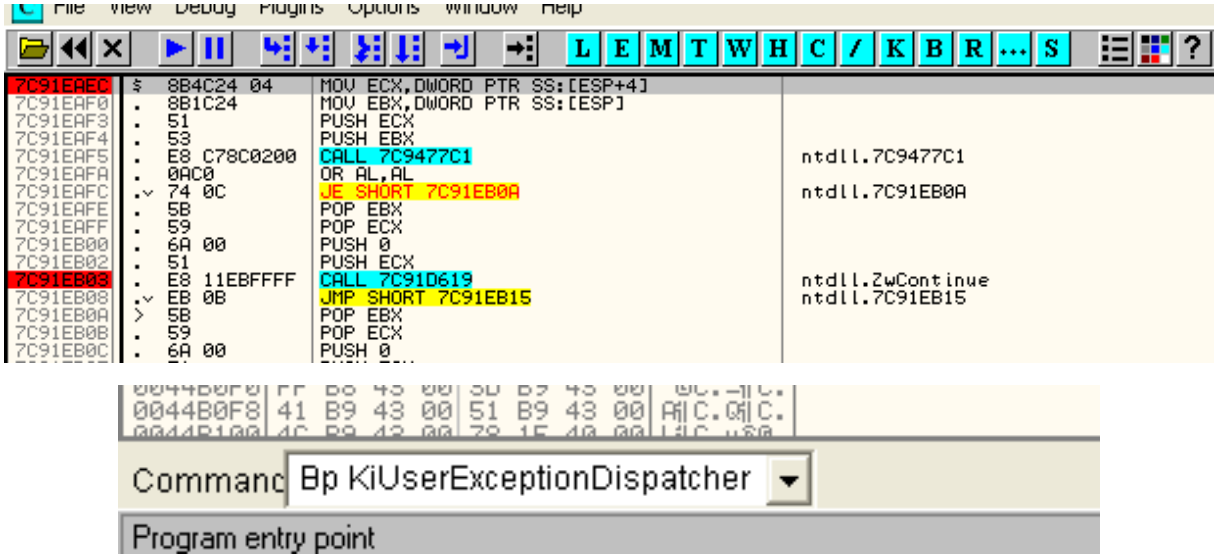
Видим, что когда произошла остановка на OEP, стек содержит на два значения больше, чем при запуске, и за исключением редких случаев - это признака украденных байтов, так что готовимся к их поиску.

Но сначала должны запустить скрипт для обработки аппаратных точек останова, который мы рассматривали в прошлых главах, так как из-за небольшого бага нам нужно применять его для этого и для многих других упаковщиков.

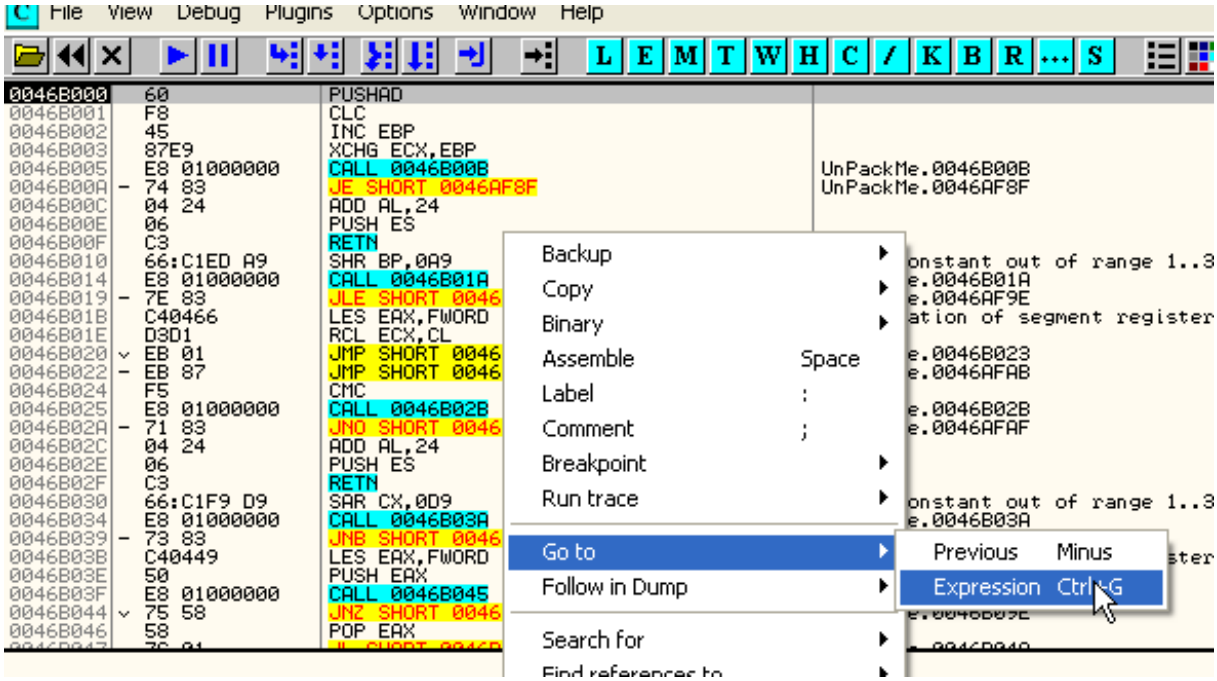
Теперь можем попробовать, так как если сейчас мы хотим остановиться на OEP, который нам уже известен, и установить на неё HBP ON EXECUTION, то видим, что не останавливаемся на нём, так как программа его дезактивировала. С использованием скрипта также не останавливается, так как здесь имеется баг.

До это ничего не указывает, где происходит ошибка, поэтому нужно разобраться как ведёт себя скрипт и как его нужно изменить.

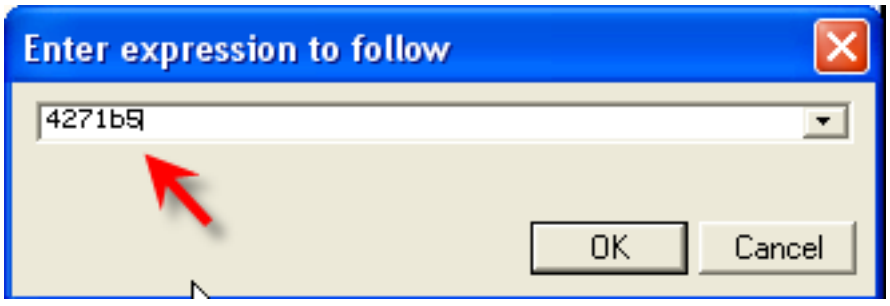
Запускаем снова распакуй-меня в OllyDbg и устанавливаем два BP. Столько же в KiUserExceptionDispatcher, сколько в CALL'е, которые ведёт нас к ZwContinue, как это показано на изображении.

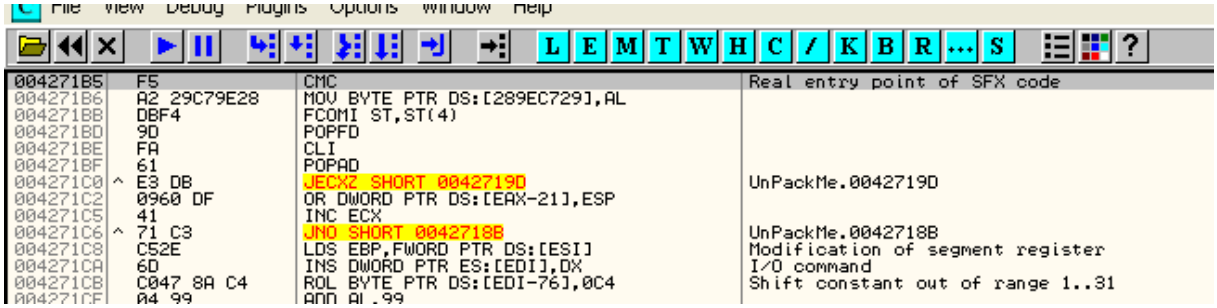


Теперь устанавливаем аппаратную точку останова на выполнение по адресу OEP, для того, чтобы попробовать, сотрётся оно или нет. Можем использовать другой адрес, но мы знаем точно, что здесь выполнение произойдёт точно, и случится это после всех манипуляций упаковщика, поэтому и используем именно этот адрес.

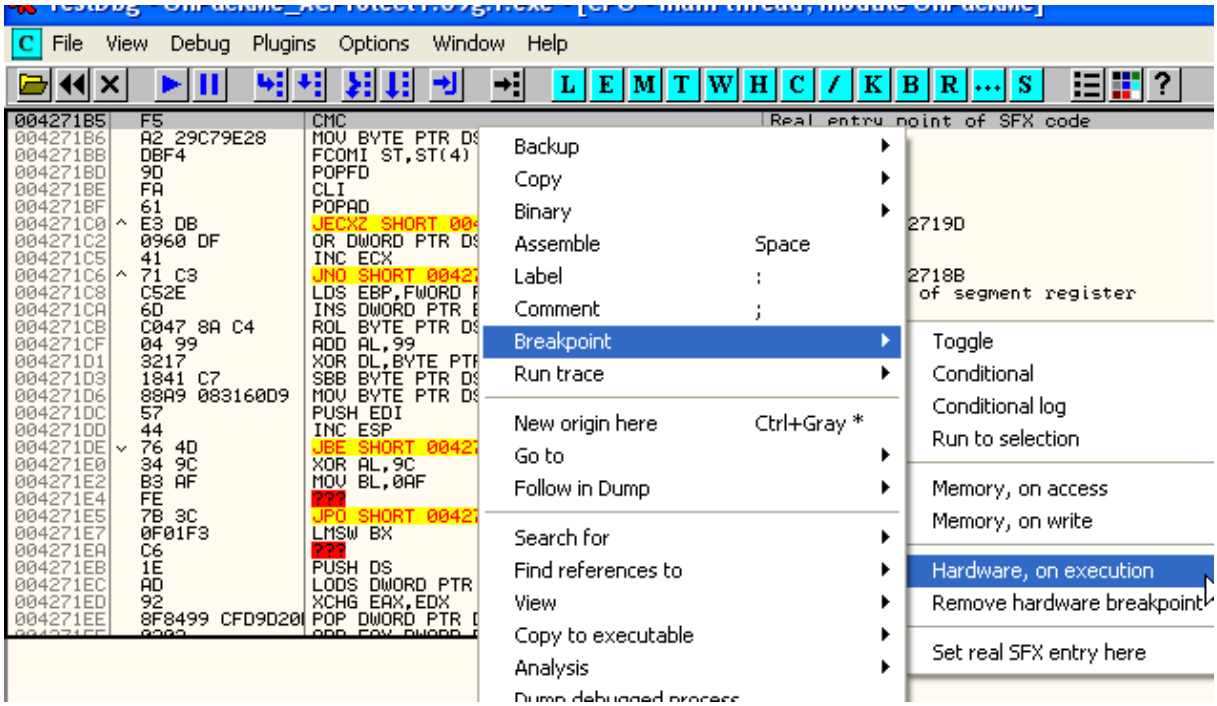


Здесь в начале выбираем «GOTO EXPRESSION» и устанавливаем адрес OEP, равный 4271B5.





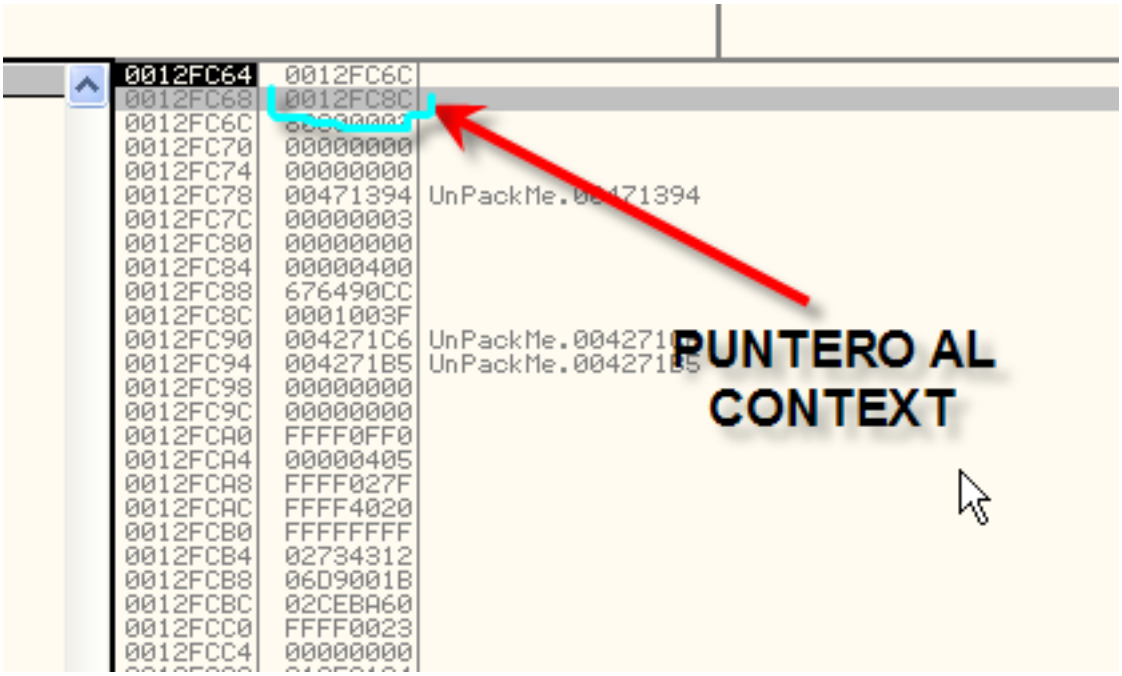
Здесь устанавливаем HBP ON EXECUTION.



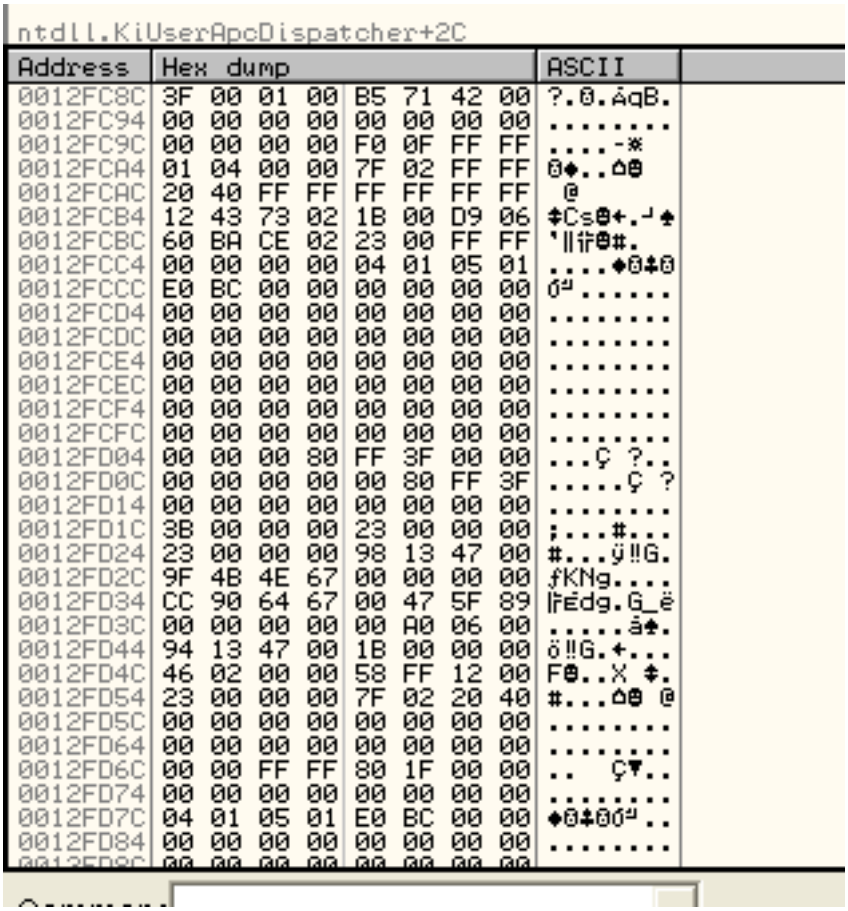
Отмечаем первую строку и устанавливаем HARDWARE BPX ON EXECUTION.

Теперь нажимаем RUN.

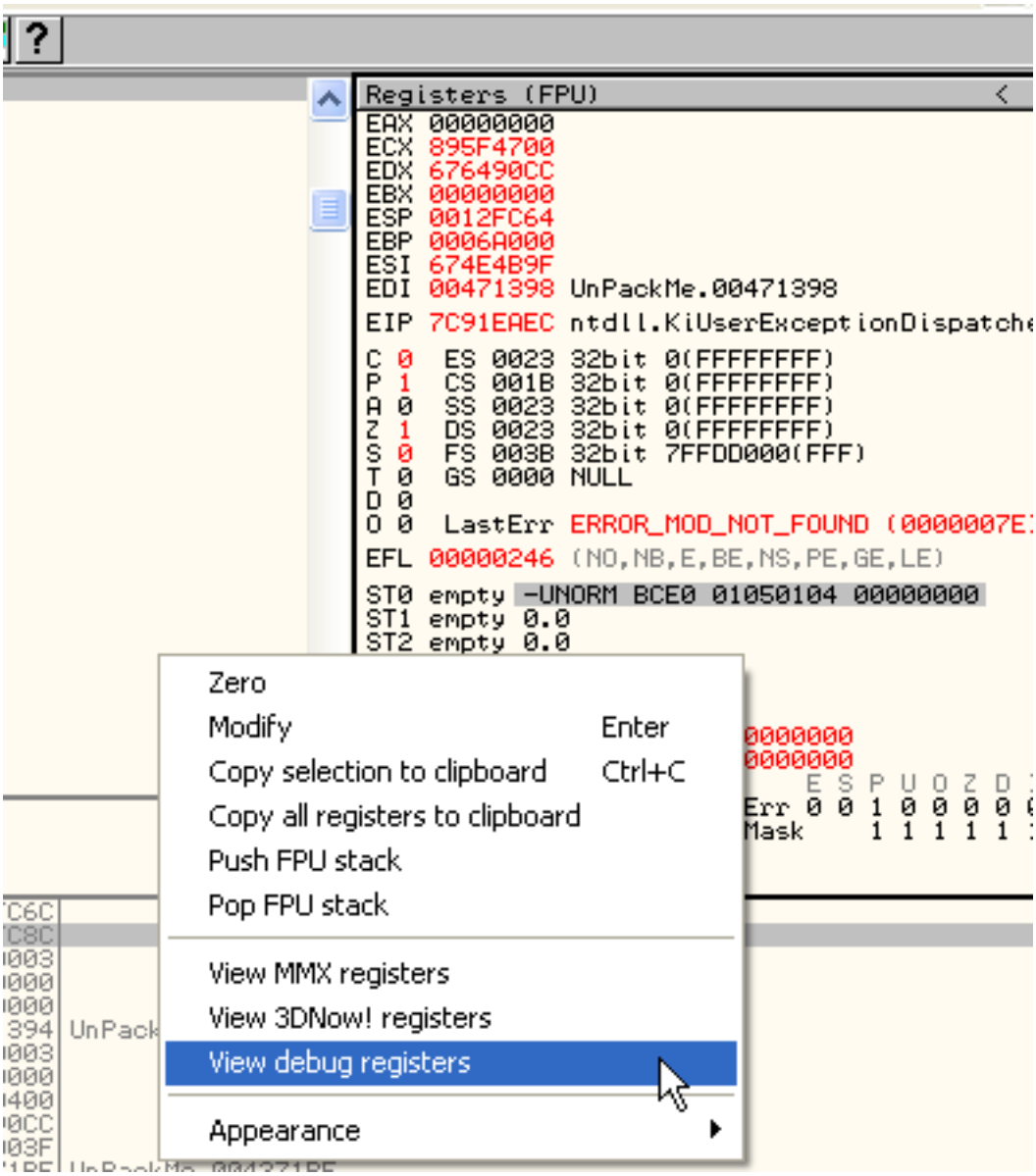
Останавливаемся на BP и смотрим стек.



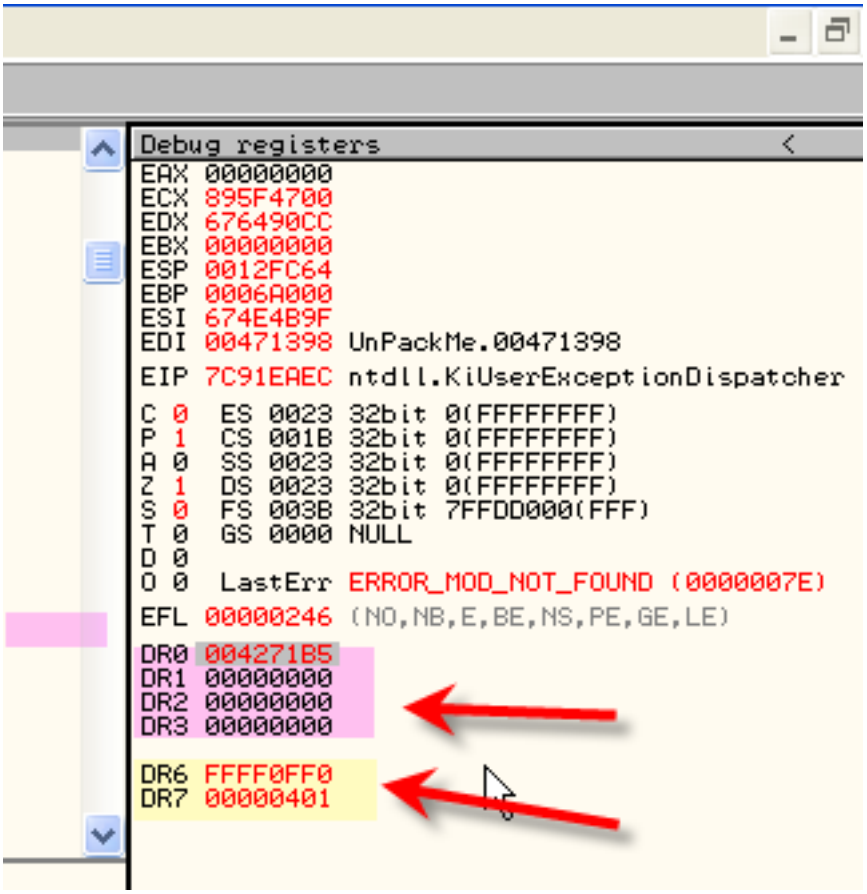
Когда останавливаемся на KiUserExceptionDispatcher, указатель на контекст находится во второй ячейке стека, смотрим контекст через дамп.



Конечно, не буду объясняет все значения структуры CONTEXT, но вот будем использовать на данный момент.

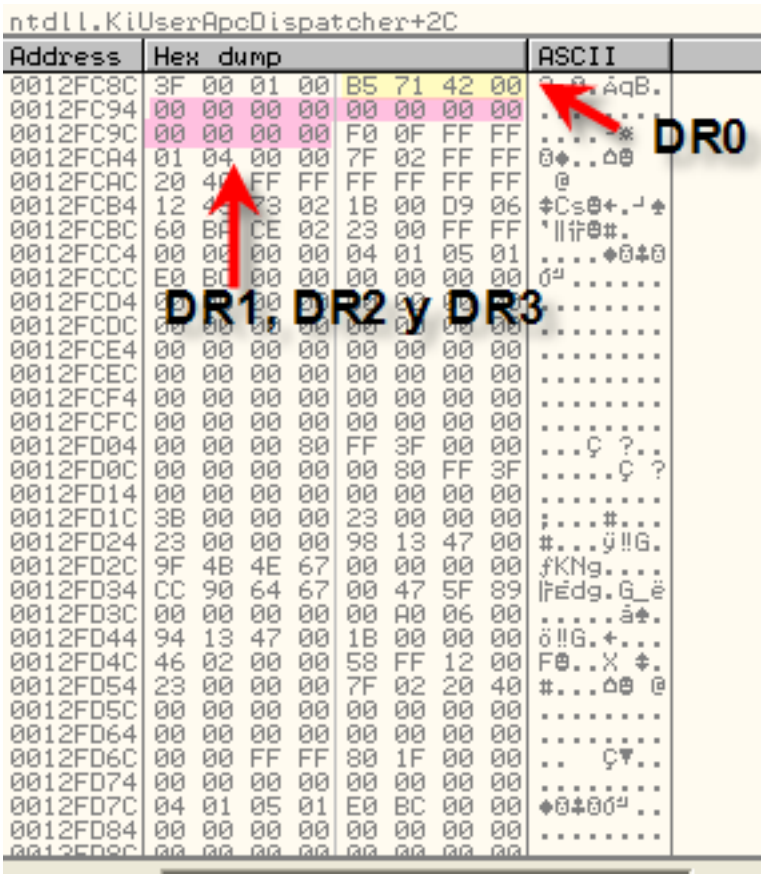


В регистрах меняем режим отображения, чтобы увидеть отладочные, если такой опции нет, значит, мы уже в правильном режиме отображения.



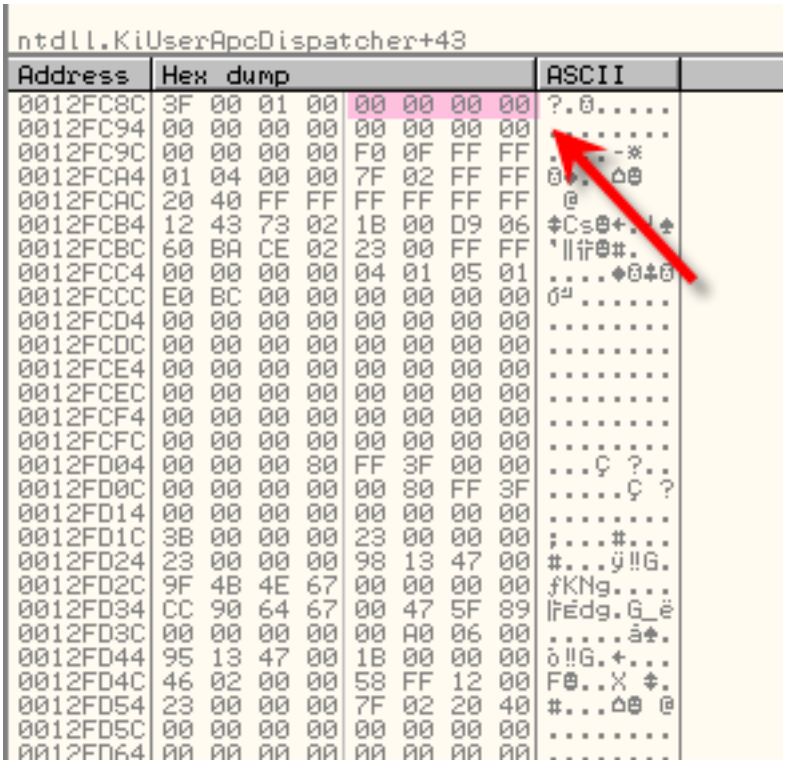
Это знаменитые отладочные регистры от DR0 до DR7. DR0 по DR3 соответствуют аппаратным точкам останова.

Когда программа останавливается на BP, то в этот момент отладочные регистры OllyDbg заполняется значениями HARDWARE BPX, которые они имеют на данный момент. Как видим, остановились на BP, и нам показываются текущие значения HBP, в DR0 находится 4271B5, что является значением установленного нами HBP, DR4 и DR5 не используются, DR6 нас пока не интересуется и DR7 указывает на тип аппаратной точки останова согласно таблице, которую мы рассмотрим позже. Важно то, что DR0 содержит значение нашего HARDWARE BPX. Теперь смотрим CONTEXT.



Здесь видим положение регистров DR0-DR3 в контексте. Жёлтым выделен адрес 4271B5, который соответствует установленному HBP, который сейчас активен. Другие три, выделенные розовым, равны нулю, так как мы установили только один HBP, поэтому они пусты.

То есть когда программа доходит до вышеупомянутого обработчика исключений, эти значения контекста обнуляются. Делаем RUN, чтобы оказаться в втором BP.

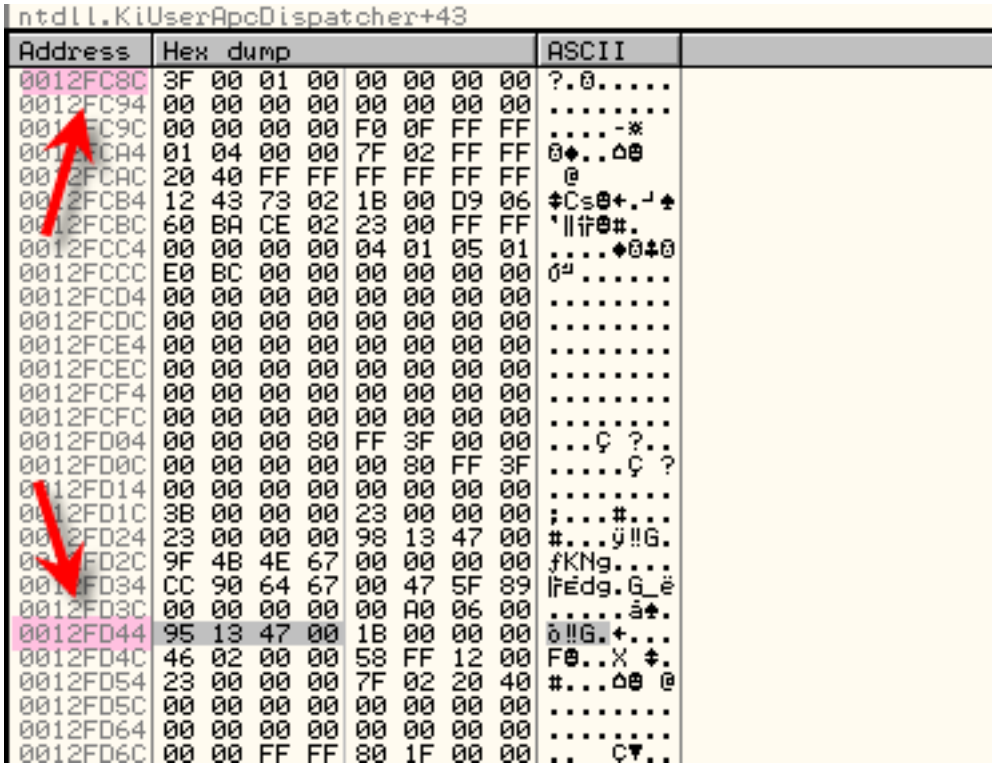


Видим, что при проходе через обработчик исключений в DR0 помещается ноль, и моя ошибка заключается в скрипте. Снова восстановим его, так как если помните, когда доходим до 2-го BP, скрипт восстанавливает их, но здесь ошибка, так как затирание DR0 - это только подготовка, установка контекста, истинное использование HBP происходит при выходе из ZwContinue и вход в RING0, до выхода из программы, то есть больше чем снова установить HBP в OLLYDBG не могу из за нуля в CONTEXT'e. При выходе из ZwContinue они будут стёрты.

(короче, Нарваха превзошёл сам себя в образности мыслей, и что он хотел сказать, я не понял J - прим. пер.)

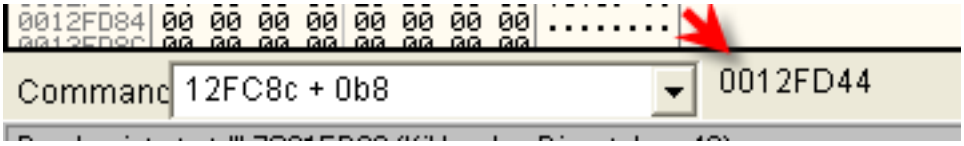
Тогда есть две возможности. Первая - это когда произойдёт остановка на первом BP сохранить значения, прочитанные из отладочных регистров в CONTEXT'e, а когда дойдём до второго BP, снова скопируем в контекст в Dr0…Dr3 имеющиеся у нас значения, которые будут верно восприняты при возвращении в программу.

Вторая возможность, которая пришла мне в голову, гораздо короче, но сначала надо узнать, где в CONTEXT'e находится адрес возврата в программу?



Если к адресу начала CONTEXT'a прибавим 0B8, то получим указатель на адрес возврата. На моей машине он равен:

12FC8c + 0b8 = 12FD44



То есть 12FD44 на моей машине указывает на адрес возврата в программ. Если в скрипте установим сюда BP, когда остановимся на нём, то значит, уже произошёл возврат в программу после стирания HBP, и можно снова их установить. Смотрим, что остаётся в скрипте.

```
--------------------------------------------------------------------------------


var aux
```

```
inicio:

bphws 4271b5, "x"

trabajo:

eob pirulo
run

pirulo:
log eip
cmp eip, 7c91eaec
je quitar
cmp eip, 7c91eb03
je restaurar
cmp eip,aux
je restaurar2
jmp final

quitar:
bphwc 4271b5
jmp trabajo

restaurar:
mov aux,esp
mov aux,[aux]
add aux,0b8
mov aux,[aux]
log aux
bp aux
jmp inicio

restaurar2:
bc aux
jmp inicio

final:
MSGYN "Continuar?"
cmp $RESULT,1
je inicio
ret

--------------------------------------------------------------------------------
```

В начале изменяемого нами скрипта находится

```
var aux
```

var - это команда для объявления переменной. Объявляем переменную aux, которую я использую для хранения и расчёта адрес возврата в программу после исключения.

```
restaurar:
mov aux,esp
mov aux,[aux]
add aux,0b8
mov aux,[aux]
log aux
bp aux
jmp inicio
```

Здесь видим часть, где восстанавливаются HBPы, потом в aux помещается значение ESP.

```
mov aux,[aux]
```

Вот, что здесь происходит: так как aux указывает на ESP, содержимое которого указывает на начало структуры CONTEXT, то после этой строки в aux будет находиться адрес начала структуры CONTEXT (в моём случае это 12fc8c), затем прибавляется 0b8, в результате чего получается 12fd44, а на ваших машинах это будет значение указатель на адрес возврата. И наконец:

```
mov aux,[aux]
```

Перемещаем в aux то, что там должно быть, теперь там, наконец-то, адрес последнего возврата, на следующей строке BP aux, устанавливаем BP на адресе возврата из исключения, а затем снова запускаем программу, и наконец:

```
pirulo:
log eip
cmp eip, 7c91eaec
je quitar
cmp eip, 7c91eb03
je restaurar
cmp eip,aux
je restaurar2
```

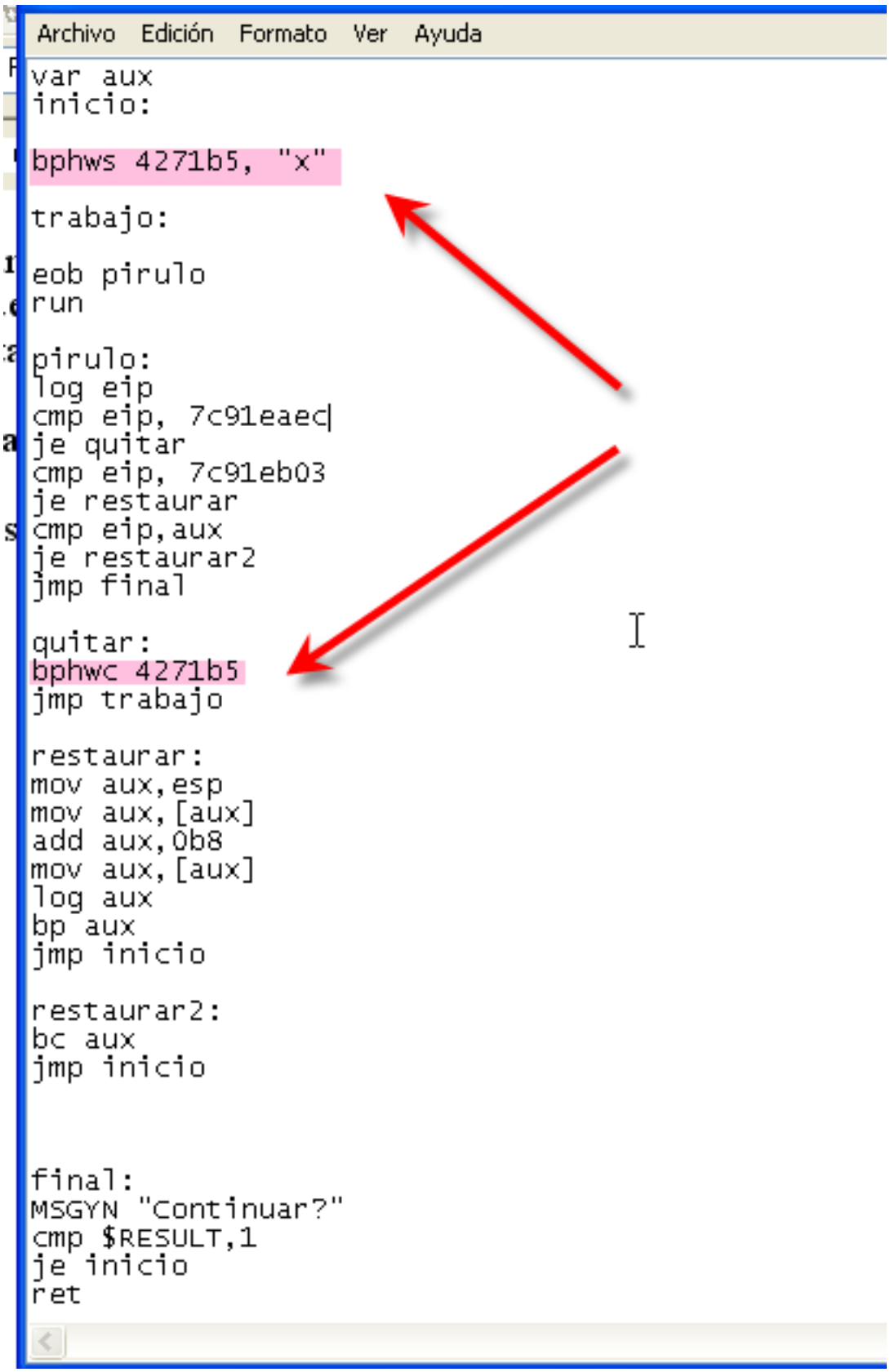
После расчёта исключений, смотрим вызывается ли по адресу возврату какое-нибудь из них, о котором мы знаем, что оно находится в точке возврата.

```
restaurar2:
bc aux
jmp inicio
```

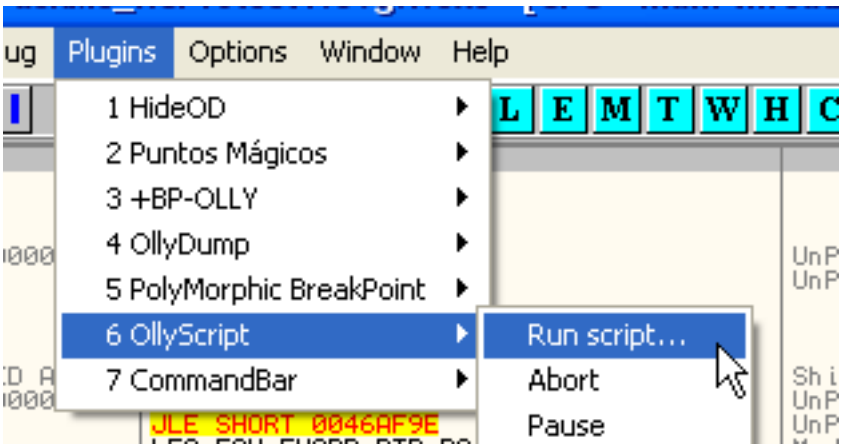
Поэтому, совершив возврат в программу, с помощью команды BC стираем точку останова, которую установили в адресе возврата, а затем возвращаемся в начало, где снова устанавливаем стёртые ранее HBP и делаем RUN с ними восстановленными.

Хорошо, как видим, сделанные изменения не слишком велики. Посмотрим, работают ли они, перезапустив программу.

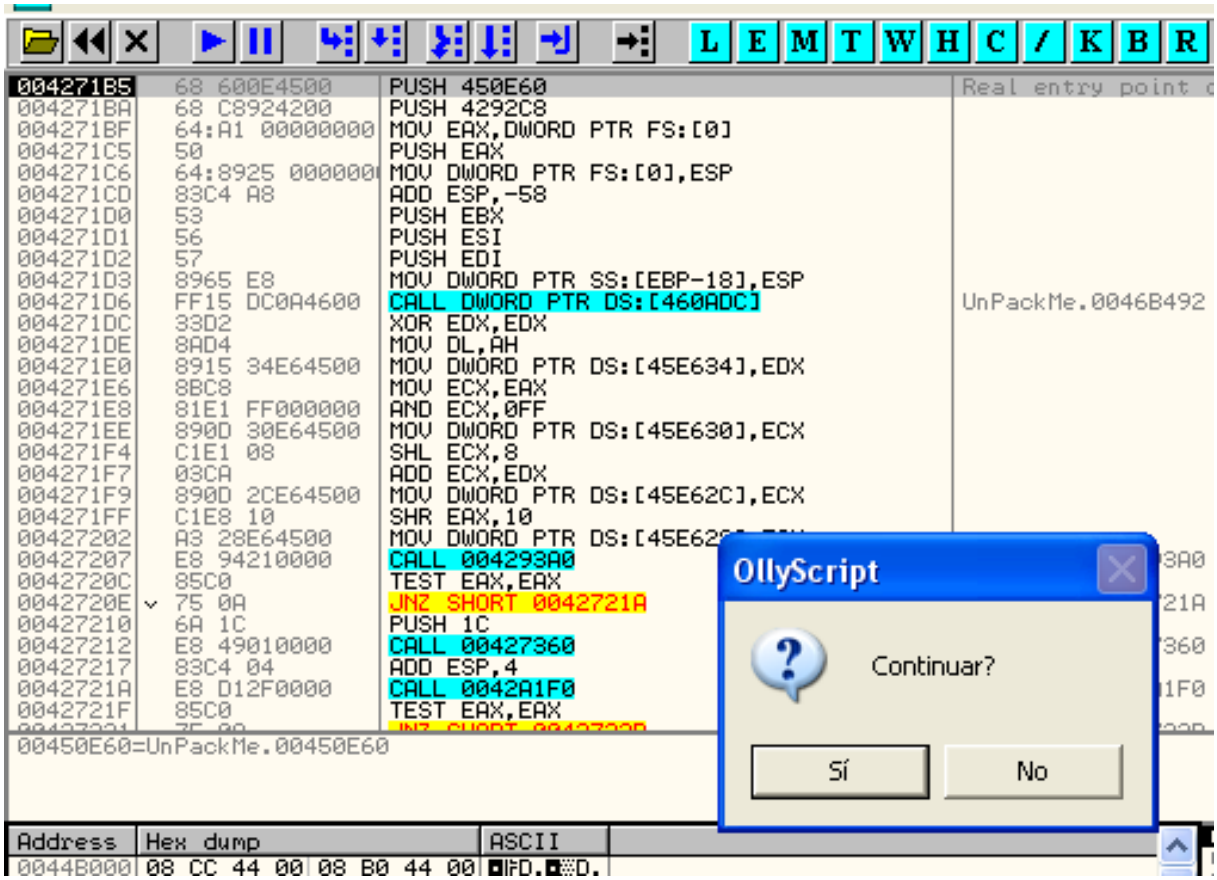
Редактируем скрипт, чтобы установить HBP по адресу OEP.



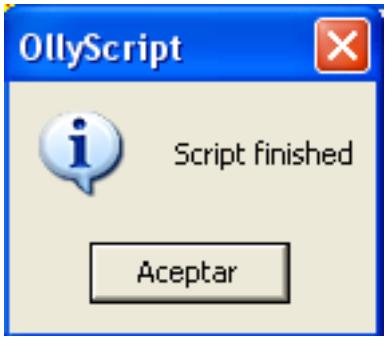
Здесь редактируем адрес HBP в запускаемом нами скрипте, устанавливаем два BP вручную, что необходимо для его работы, стираем все старые HBP, которые были.



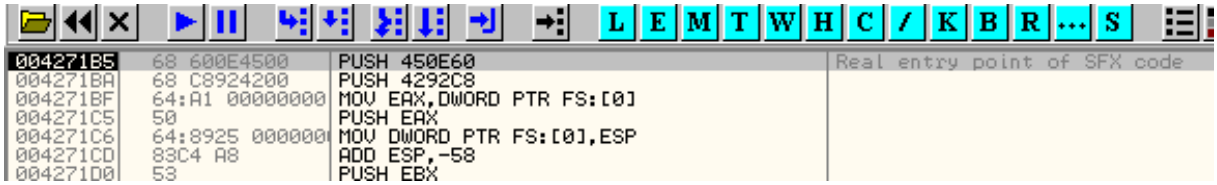
Запускаем скрипт (не забыв установить два BP и установив все галочки в разделе настроек исключений).



Здесь говорится, что сработал наш HBP, так как нам не нужно, чтобы скрипт продолжал работу, то нажимаем NO.



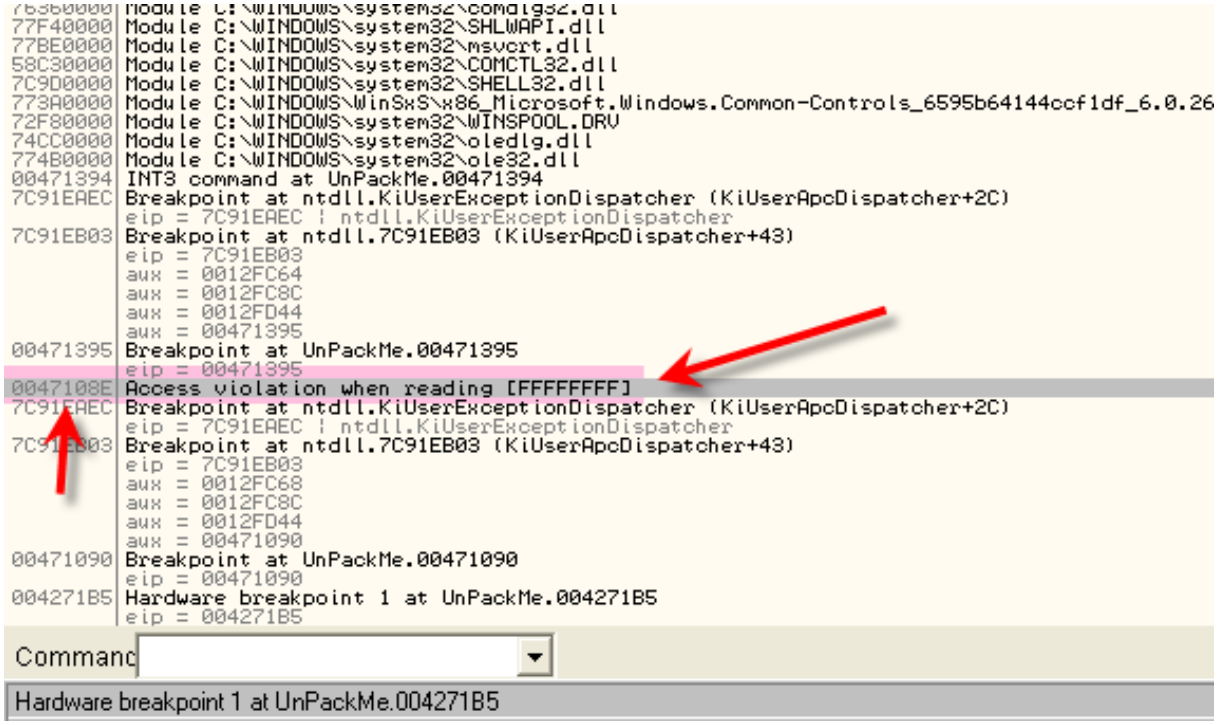
Избегаем стирания установленного нами HBP и останавливаемся на OEP, используя HBP.



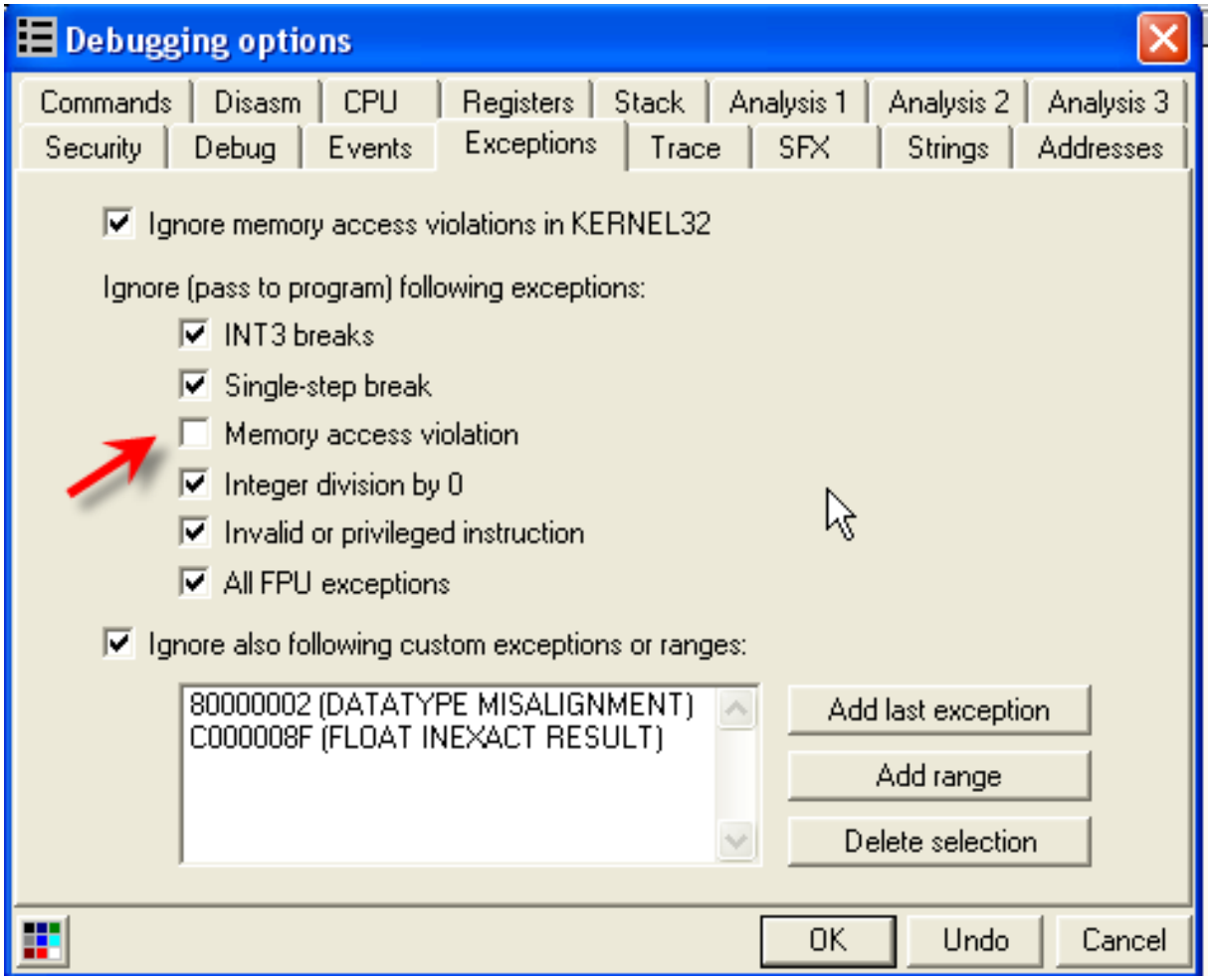
Для этих новых упаковщиков очень важно иметь какой-нибудь способ избегать стирания HBP, и этого нам удалось достичь.

Сейчас, так как можем легко управлять HBPами, переходим к украденным байтам.

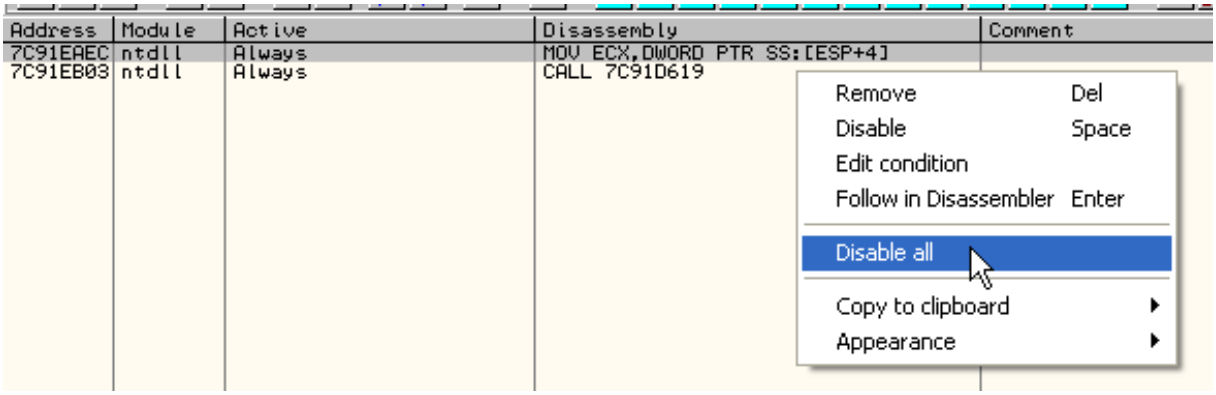
Смотрим, возможно ли дойти до последнего исключение и трассировать оттуда.



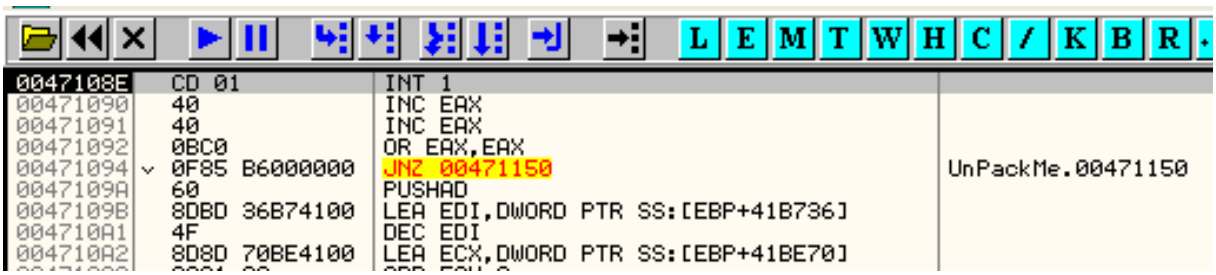
Лог показывает, что это последнее исключение до того, как прибудем в OEP, и как видим, это ACCESS VIOLATION WHEN READING. Снимаем галочку, чтобы остановиться на нём.



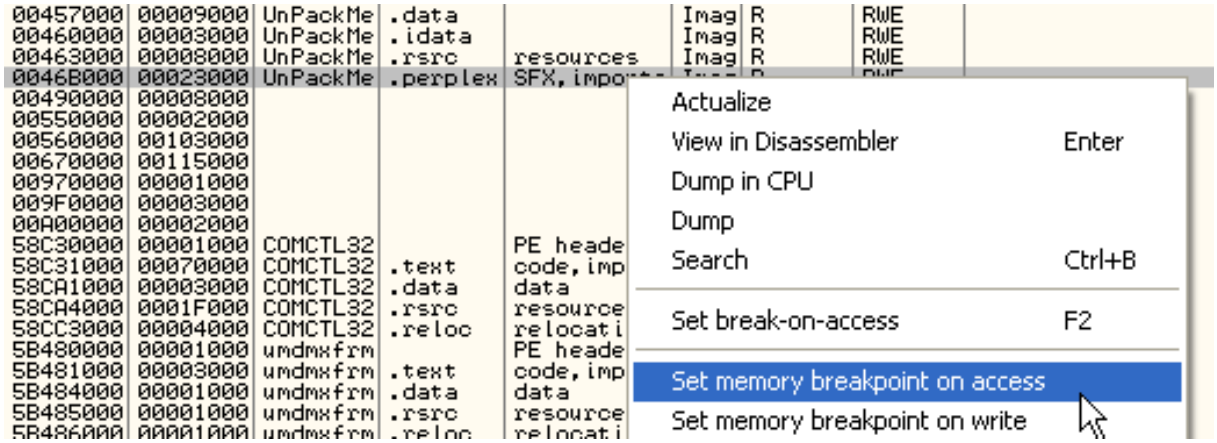
Запрещаем эти BP, чтобы использовать скрипт.



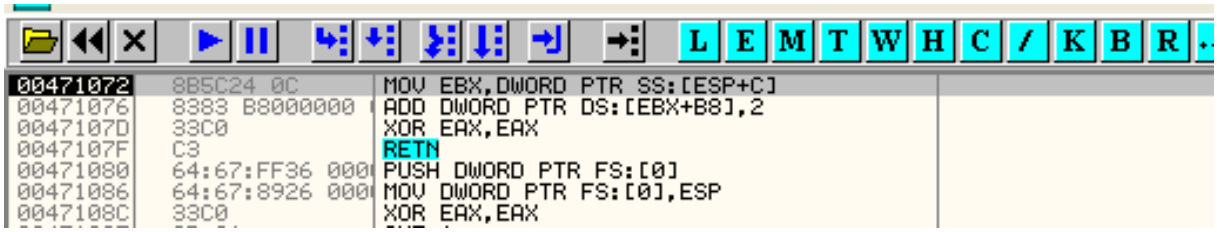
Теперь перезапускаем и делаем RUN.



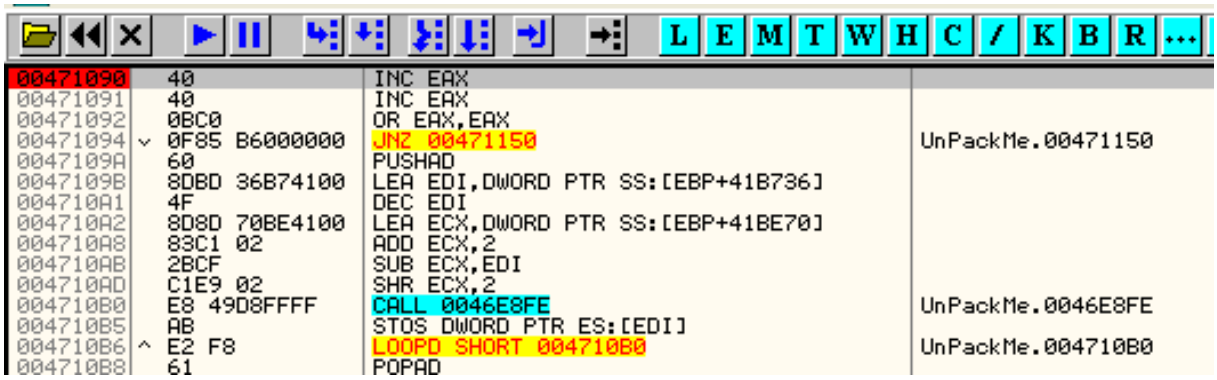
Здесь останавливаемся на последнем исключении, устанавливаем BPM ON ACCESS на секции исполняемого кода, и останавливаемся в обработчике исключений.



Проходим исключение с помощью SHIFT+F9.

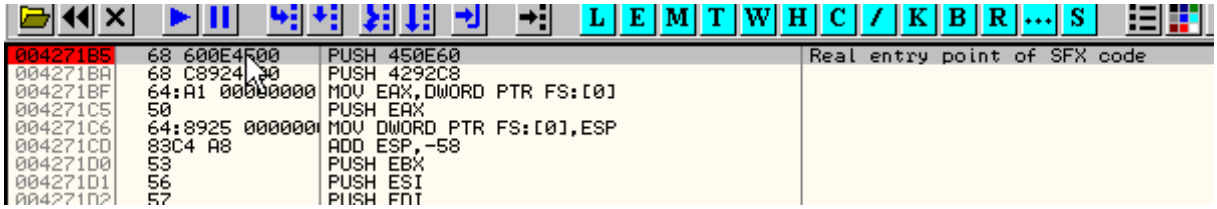


Теперь трассируем с помощью F7 строки обработчика исключений, пока не доходим до RET. Нажав на RUN, возвращаемся в программу в:



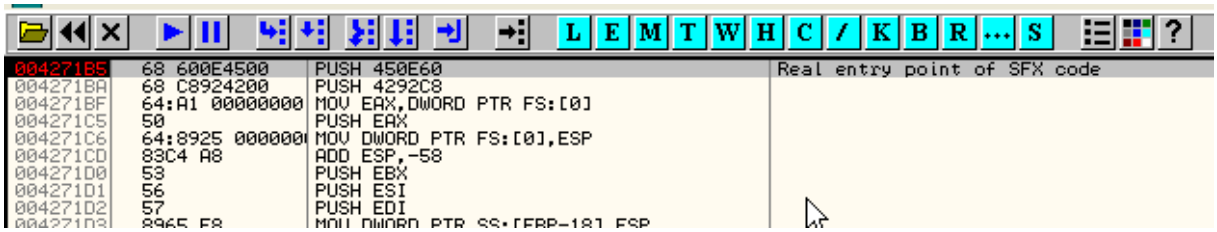

```
00471090    40              INC EAX
```

Теперь, раз уже прошли исключение, устанавливаем BP на OEP, так как распаковка уже произошла. BP послужит мне для того, чтобы остановится, когда завершим трассировку.



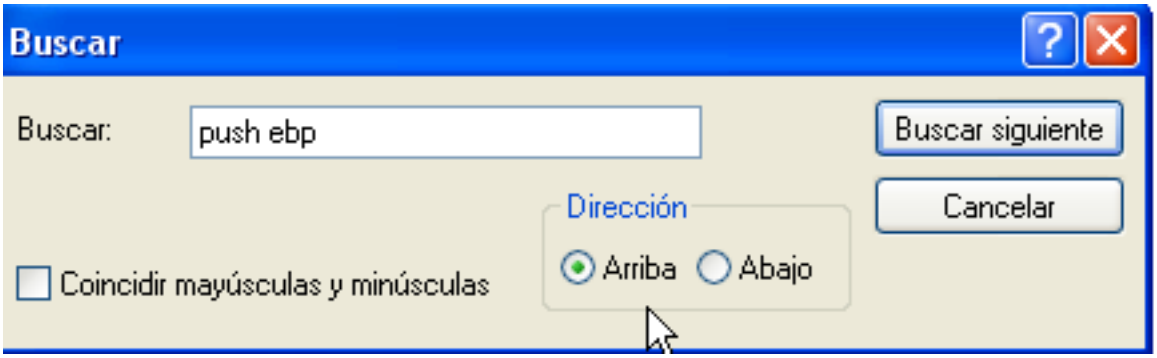
И с 471090 начинаем трассировку и видим, что она идёт довольно медленно, так как не совершается никаких переходов, и проходим инструкции одна за одной.

Здесь, наконец, останавливаемся на OEP, так что можем заглянуть в листинг txt.




```
00485EAE Main     JL SHORT 00485ECA
00485ECA Main     POPAD                                  ; EAX=00000000, ECX=0012FFB0,
    EDX=7C91EB94, EBX=7FFDF000, ESI=FFFFFFFF
00485ECB Main     JMP SHORT 00485ECE
00485ECE Main     JMP DWORD PTR DS:[485ED4]
    Breakpoint at UnPackMe.004271B5
```

Это последние строки txt, и поищем отсюда наверх PUSH EBP, который обычно является первой инструкцией программы.

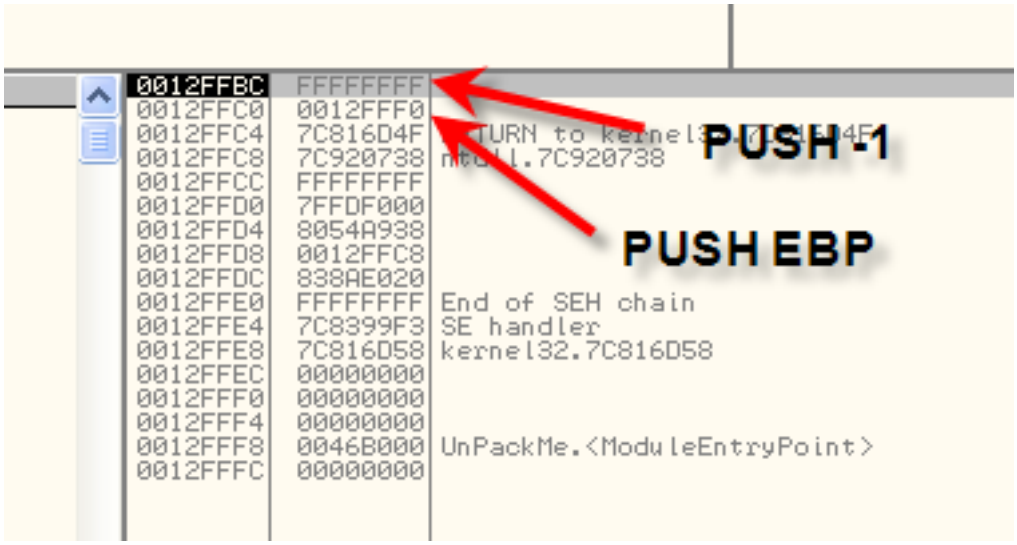



```
00485AF3 Main     PUSH EBP
00485AF4 Main     MOV EBP,ESP                            ; EBP=0012FFC0
00485AF6 Main     PUSH -1
00485AF8 Main     NOP
00485AF9 Main     PUSHAD
00485AFA Main     PUSHAD
00485AFB Main     CALL 00485B00
00485B00 Main     POP ESI                                ; ESI=00485B00
00485B01 Main     SUB ESI,6                              ; ESI=00485AFA
00485B04 Main     MOV ECX,35                             ; ECX=00000035
00485B09 Main     SUB ESI,ECX                            ; ESI=00485AC5
00485B0B Main     MOV EDX,E3D6D5FD                       ; EDX=E3D6D5FD
00485B10 Main     SHR ECX,2                              ; ECX=0000000D
00485B13 Main     SUB ECX,2                              ; ECX=0000000B
00485B16 Main     CMP ECX,0
```

Здесь видим украденные байты, а за ними начинается какая-то ерунда, но до этого правильное значение регистров сохраняется с помощью PUSHAD, которые восстанавливаются с помощью POPAD прямо перед прыжком на фальшивый OEP.

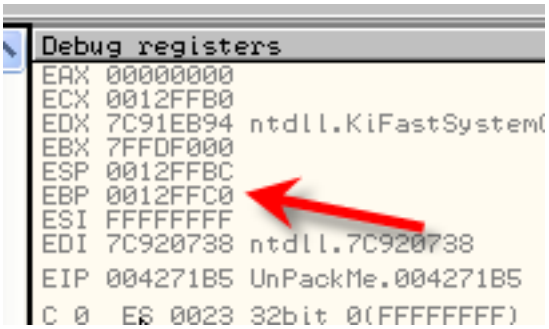
```
00485ECA Main     POPAD                                  ; EAX=00000000, ECX=0012FFB0,
    EDX=7C91EB94, EBX=7FFDF000, ESI=FFFFFFFF
00485ECB Main     JMP SHORT 00485ECE
00485ECE Main     JMP DWORD PTR DS:[485ED4]
    Breakpoint at UnPackMe.004271B5
```

Конечно, видим, что украденные байты совпадают со значениями, которые остались в стеке.



Кроме того другой регистр, который вначале может иметь случайное значение, это EBP, чьё содержимое изменяется с помощью MOV EBP, ESP.

```
00485AF3 Main     PUSH EBP
00485AF4 Main     MOV EBP,ESP                           ; EBP=0012FFC0
00485AF6 Main     PUSH -1
```



Листинг секции txt показывает нам, что EBP содержит значение 12FFC0, которое равно тому же значению, которое есть, когда выполнение доходит до фальшивого OEP. Всё, что дальше - просто трюке, чтобы сбить со следа. Видим, что после данного перехода EBP снова содержит значение 12FFC0, и это доказывает, что данные украденные байты являются правильными, а всё, что посередине, просто шелуха.

Ок, мы получили знаменитые украденные байты, изменили скрипт, и думаю, что для этой главы достаточно, поэтому продолжим в 43-ей.

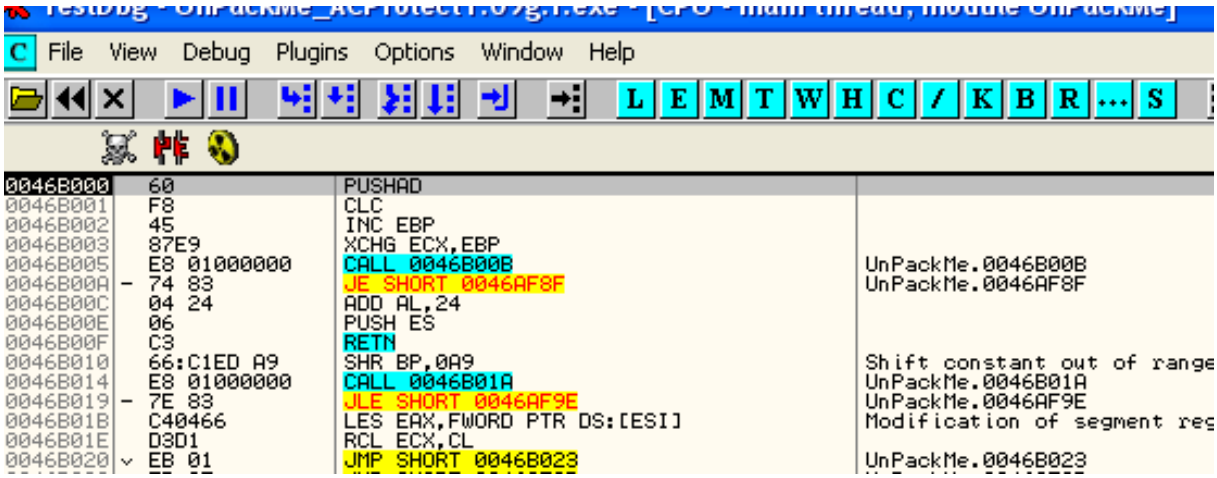
[missing file: /pub/23/files/ollydbg42.tar.gz]

Введение в крэкинг с нуля, используя OllyDbg - Глава 43

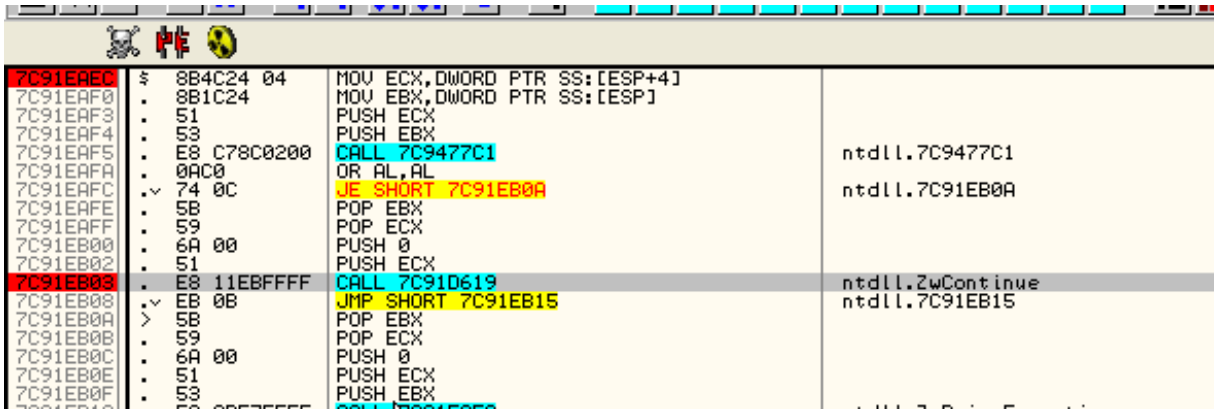
Автор: Рикардо Нарваха, пер. Aquila

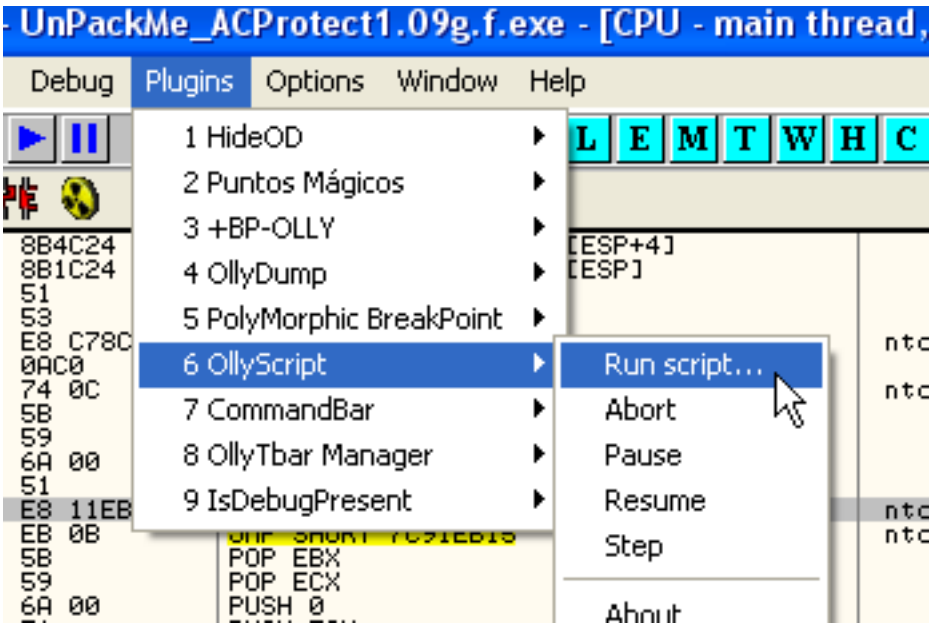
Дата: 2009-06-12

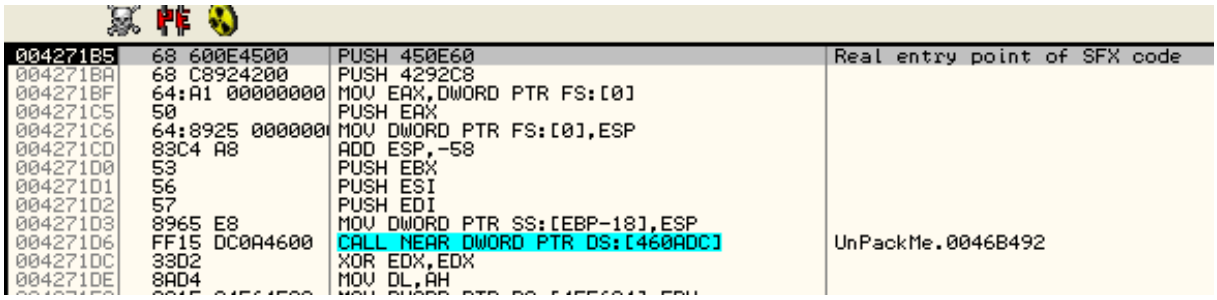
В предыдущей части мы сосредоточились на починке украденных байт, а сейчас нашей задачей является IAT. Снова открываем распакуй-меня в OllyDbg и доходим до фальшивой OEP с помощью скрипта.



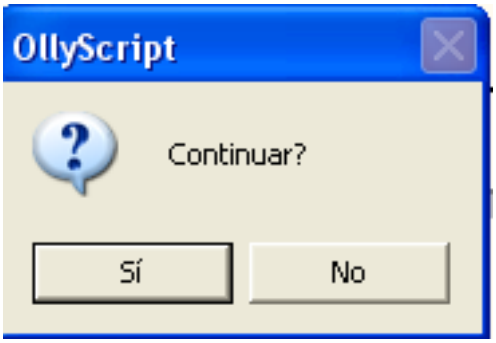
Сейчас находимся в начале программы. Теперь, поскольку у нас есть скрипт для того, чтобы попасть в OEP, который остался нам с прошлой главы, устанавливаем две необходимые точки останова и используем их.



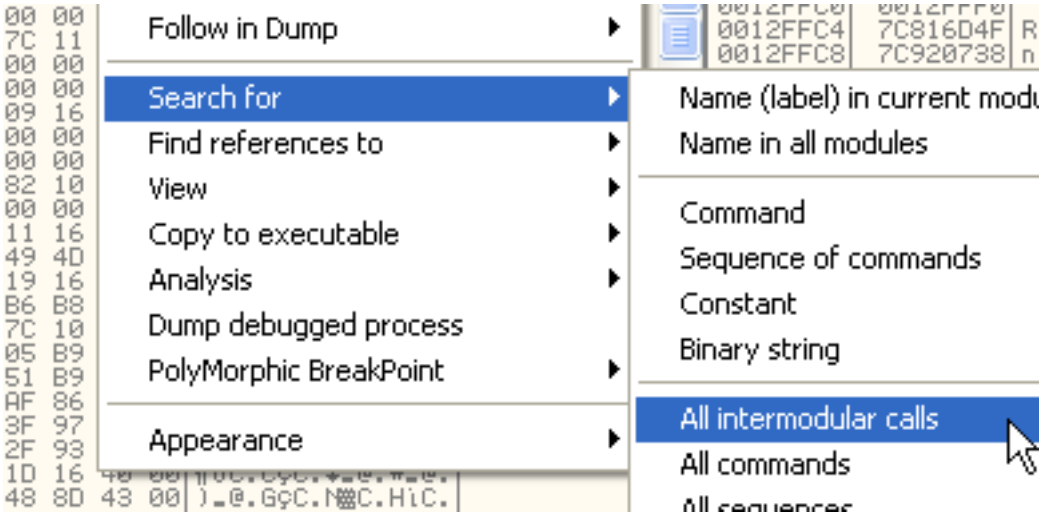




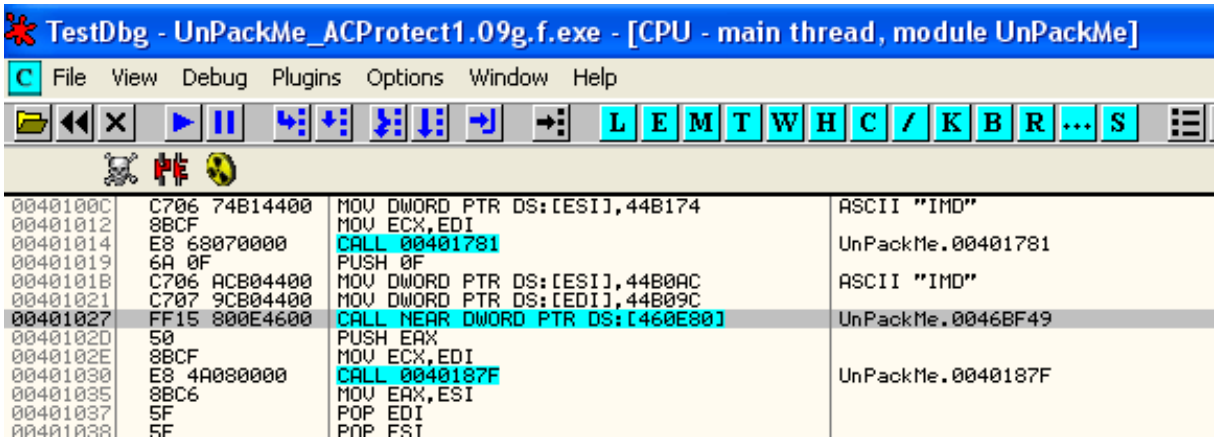
И сейчас находимся на OEP. Ниже рассмотрим, как улучшить скрипт, чтобы автоматически устанавливать две точки останова, и чтобы запрашивался адрес HBP. Пока редактировать не будем, послужит нам как есть.



Ищем вызов API-функции, для чего можем использовать SEARCH FOR INTERMODULARS CALLS.

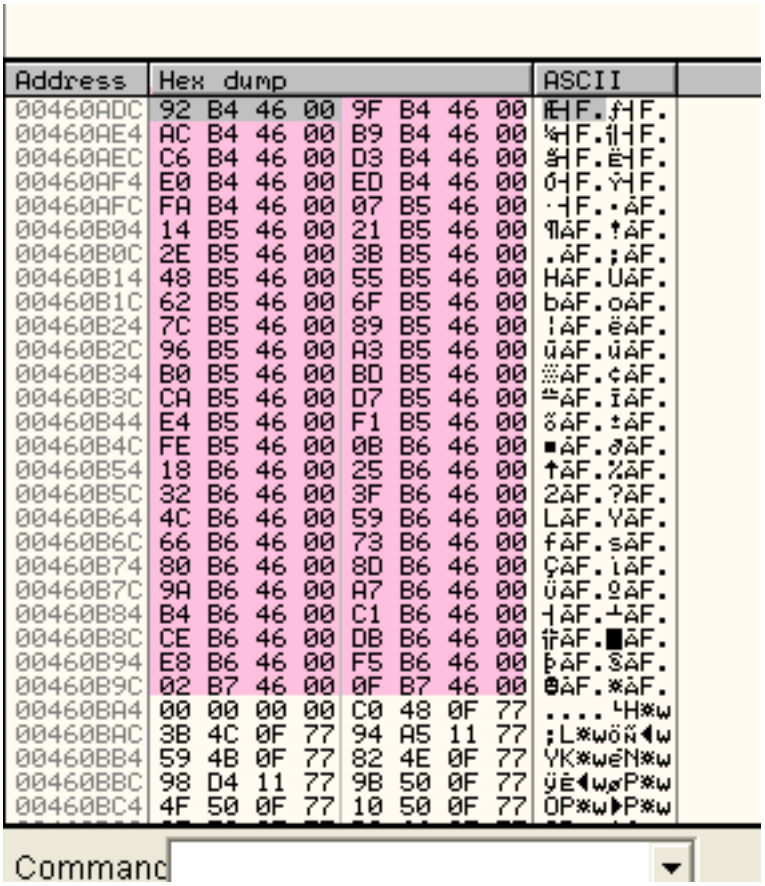


Видим несколько вызовов API-функции, которые берут значения из IAT, и видим несколько хороших элементов, так как здесь присутствует имя API функции. Большинство же, как несложно определить, являются переадресовочными элементами и никаких имён не показывают. Берём первый из плохих, т.е. переадресовочных, делаем двойной щелчок на нём и идём в листинг.



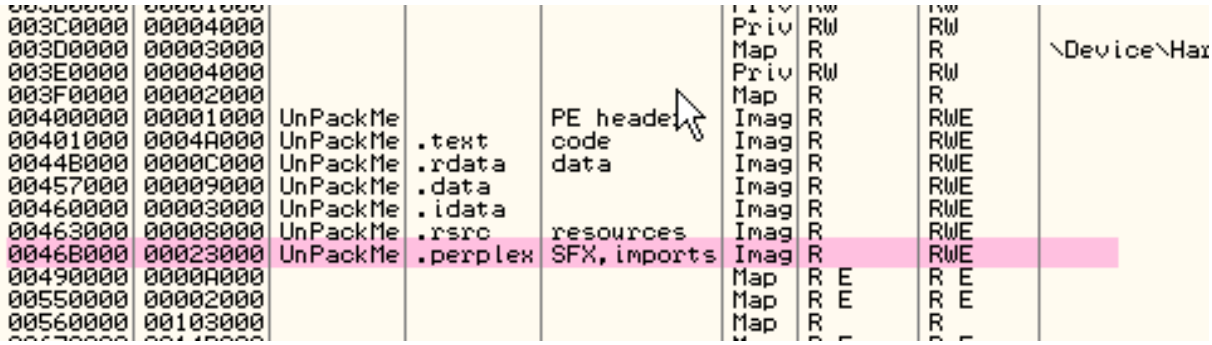
Здесь видим вызов, который берёт значение 460e80, что является элементом IAT. Смотрим IAT в дампе.

Видим ворох элементов, которые в данном случае являются переадресовочными не в секцию, созданную упаковщиком. Может в ту же секцию, где исполняется сам упаковщик? Смотрим карту памяти.



Address	Hex dump	ASCII
00460ADC	92 B4 46 00 9F B4 46 00	
00460AE4	AC B4 46 00 B9 B4 46 00	
00460AEC	C6 B4 46 00 D3 B4 46 00	
00460AF4	E0 B4 46 00 ED B4 46 00	
00460AFC	FA B4 46 00 07 B5 46 00	
00460B04	14 B5 46 00 21 B5 46 00	
00460B0C	2E B5 46 00 3B B5 46 00	
00460B14	48 B5 46 00 55 B5 46 00	
00460B1C	62 B5 46 00 6F B5 46 00	
00460B24	7C B5 46 00 89 B5 46 00	
00460B2C	96 B5 46 00 A3 B5 46 00	
00460B34	B0 B5 46 00 BD B5 46 00	
00460B3C	CA B5 46 00 D7 B5 46 00	
00460B44	E4 B5 46 00 F1 B5 46 00	
00460B4C	FE B5 46 00 0B B6 46 00	
00460B54	18 B6 46 00 25 B6 46 00	
00460B5C	32 B6 46 00 3F B6 46 00	
00460B64	4C B6 46 00 59 B6 46 00	
00460B6C	66 B6 46 00 73 B6 46 00	
00460B74	80 B6 46 00 8D B6 46 00	
00460B7C	9A B6 46 00 A7 B6 46 00	
00460B84	B4 B6 46 00 C1 B6 46 00	
00460B8C	CE B6 46 00 DB B6 46 00	
00460B94	E8 B6 46 00 F5 B6 46 00	
00460B9C	02 B7 46 00 0F B7 46 00	
00460BA4	00 00 00 00 C0 48 0F 77	....└H*w
00460BAC	3B 4C 0F 77 94 A5 11 77	;L*wöñ◄w
00460BB4	59 4B 0F 77 82 4E 0F 77	YK*wéN*w
00460BBC	98 D4 11 77 9B 50 0F 77	ÿÉ◄w¢P*w
00460BC4	4F 50 0F 77 10 50 0F 77	OP*w►P*w

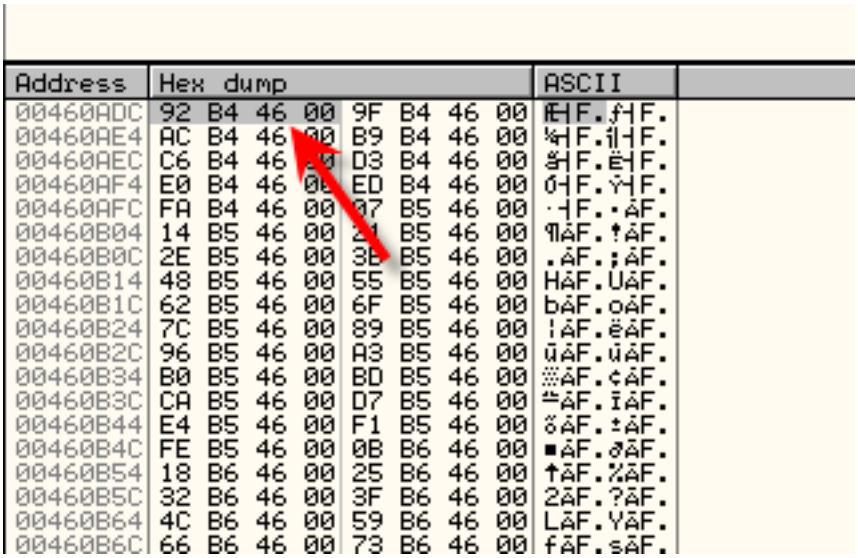
Command



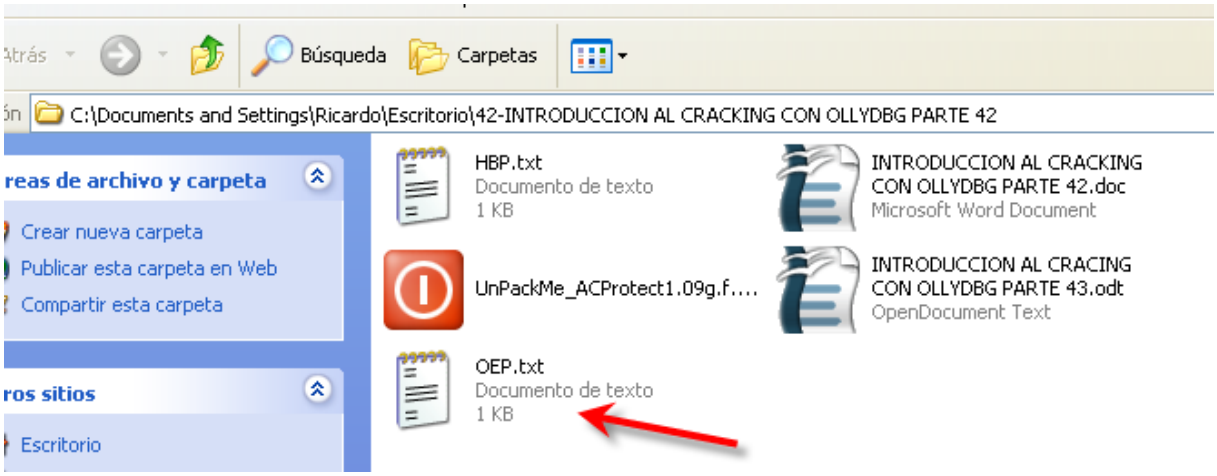
003C0000	00004000			Priv	RW	RW	
003D0000	00003000			Map	R	R	\Device\Har
003E0000	00004000			Priv	RW	RW	
003F0000	00002000			Map	R	R	
00400000	00001000	UnPackMe	PE header	Imag	R	RWE	
00401000	0004A000	UnPackMe	.text code	Imag	R	RWE	
0044B000	0000C000	UnPackMe	.rdata data	Imag	R	RWE	
00457000	00009000	UnPackMe	.data	Imag	R	RWE	
00460000	00003000	UnPackMe	.idata	Imag	R	RWE	
00463000	00008000	UnPackMe	.rsrc resources	Imag	R	RWE	
0046B000	00023000	UnPackMe	.perplex SFX,imports	Imag	R	RWE	
00490000	0000A000			Map	R E	R E	
00550000	00002000			Map	R E	R E	
00560000	00103000			Map	R	R	

Вот эта секция. Если спустимся ниже и посмотрим образ точки входа, где запускается упаковщик, то вспомним, что это был адрес 46b000, поэтому упаковщик не заморачивался сильно с созданием новой секции, чтобы скрыть в ней API-функции.

Вопрос заключается в том, что раз мы знаем, что это переадресовочные элементы, может нам повезёт, и мы найдём волшебный переход или другой способ починить IAT?



Используем указанный переход 460aDC, чтобы попытаться найти волшебный переход. Чтобы не пришлось менять скрипт из OEP, скопируем его в другую папку, переименуем его в OEP и скопируем обратно.



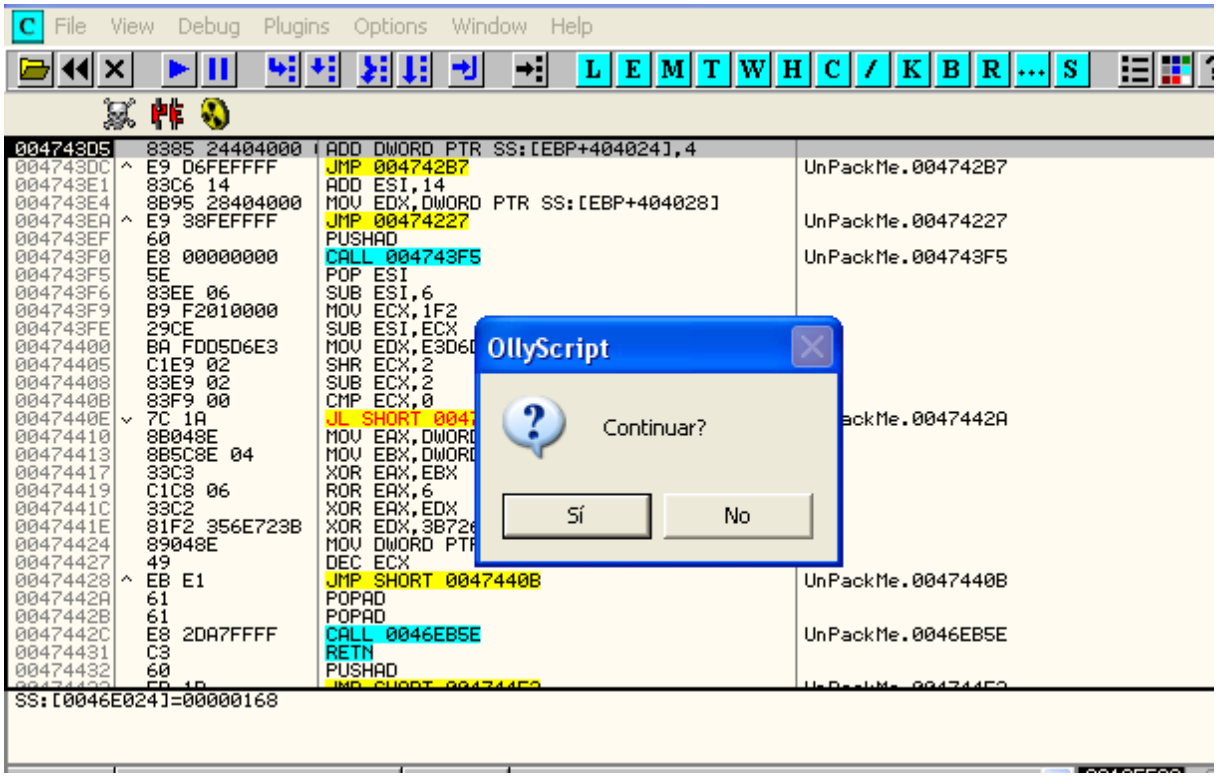
Таким образом, мы сохранили под именем OEP скрипт, который все используем для нахождения OEP, а теперь можем модифицировать HBP, чтобы найти волшебный переход.

Здесь меняем HBP на плохие элементы, а затем меняем тип на W, то есть на запись или WRITE.

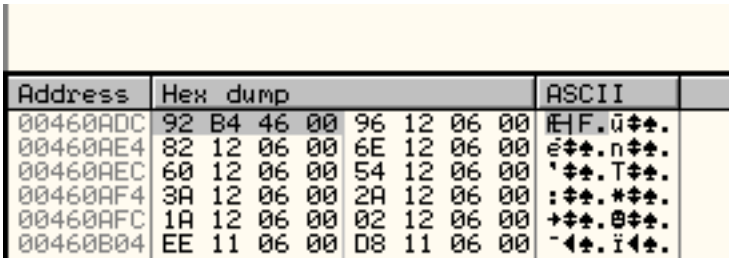
Теперь стираем HBP, установленные ранее, и делаем перезапуск.

```
Archivo  Edicion  Formato  Ver  Ayuda
var aux
inicio:

bphws 460aDC, "w"

trabajo:

eob pirulo
run

pirulo:
log eip
cmp eip, 7c91eaec
je quitar
cmp eip, 7c91eb03
je restaurar
cmp eip,aux
je restaurar2
jmp final

quitar:
bphwc 460aDC
jmp trabajo

restaurar:
mov aux,esp
mov aux,[aux]
```

Остановились тут, смотрим в DUMP плохой элемент, за которым мы будем наблюдать после того, как решим, тот ли это момент, чтобы прервать выполнение скрипта.

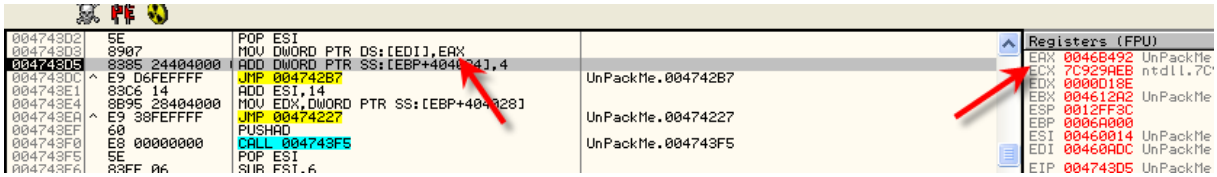


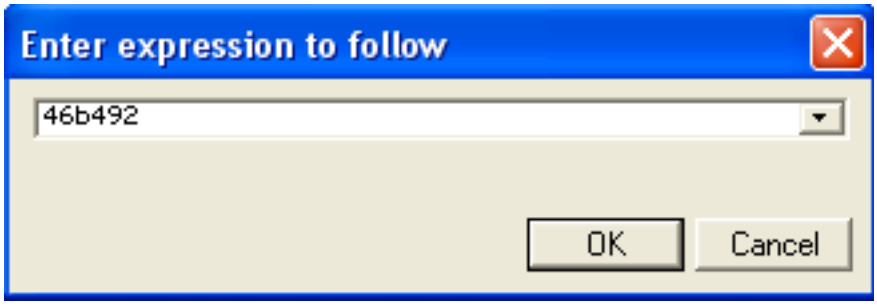
Вот здесь находится плохое значение, тут и происходит HBP ON WRITE. Останавливаемся на следующей инструкции после того места, где сработало исключение, так что смотрим предыдущую.



Вот эта инструкция, сохраняющая плохое значение, и видим, что EAX содержит значение, которое будет сохранено.

Проверим, происходит ли запись в переадресовочную зону, для чего нам не нужно идти до OEP, смотрим 46b492:



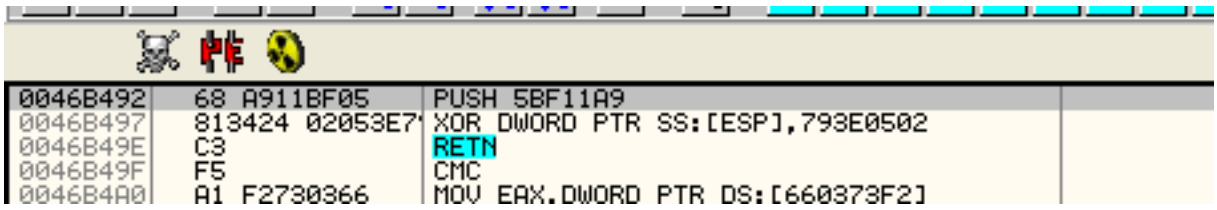


Видим, что переадресация довольно дурацкая – здесь помещается в стек 5bf11a9, а затем это значение XOR'ится с 793e0502. Результат этой операции становится первым значением стека, и когда доходит до выполнения RET'a, возврат произойдёт по этому адресу.

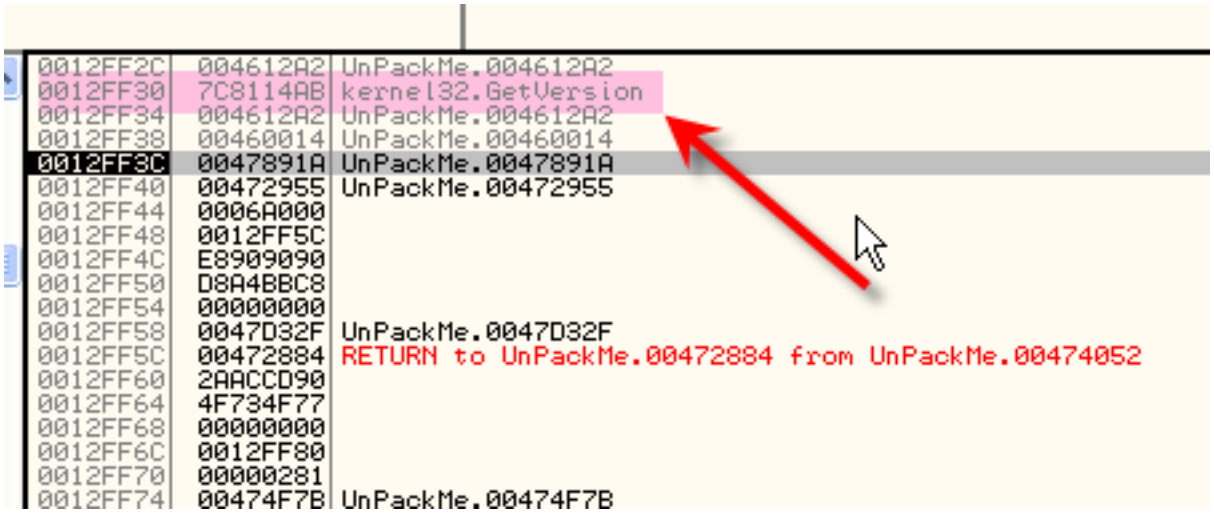
Смотрим, может ли посчитать адрес API-функции, куда, в итоге, произойдёт переход.

5bf11a9 XOR 793e0502=7C8114AB

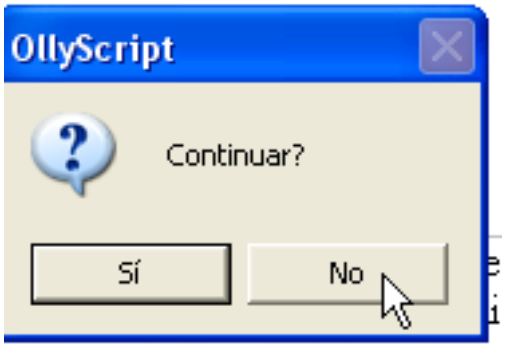
Это адрес API-функции, куда ведёт переадресация. Возвращаемся обратно в OllyDbg, и смотрим, найдём ли мы здесь что-нибудь.



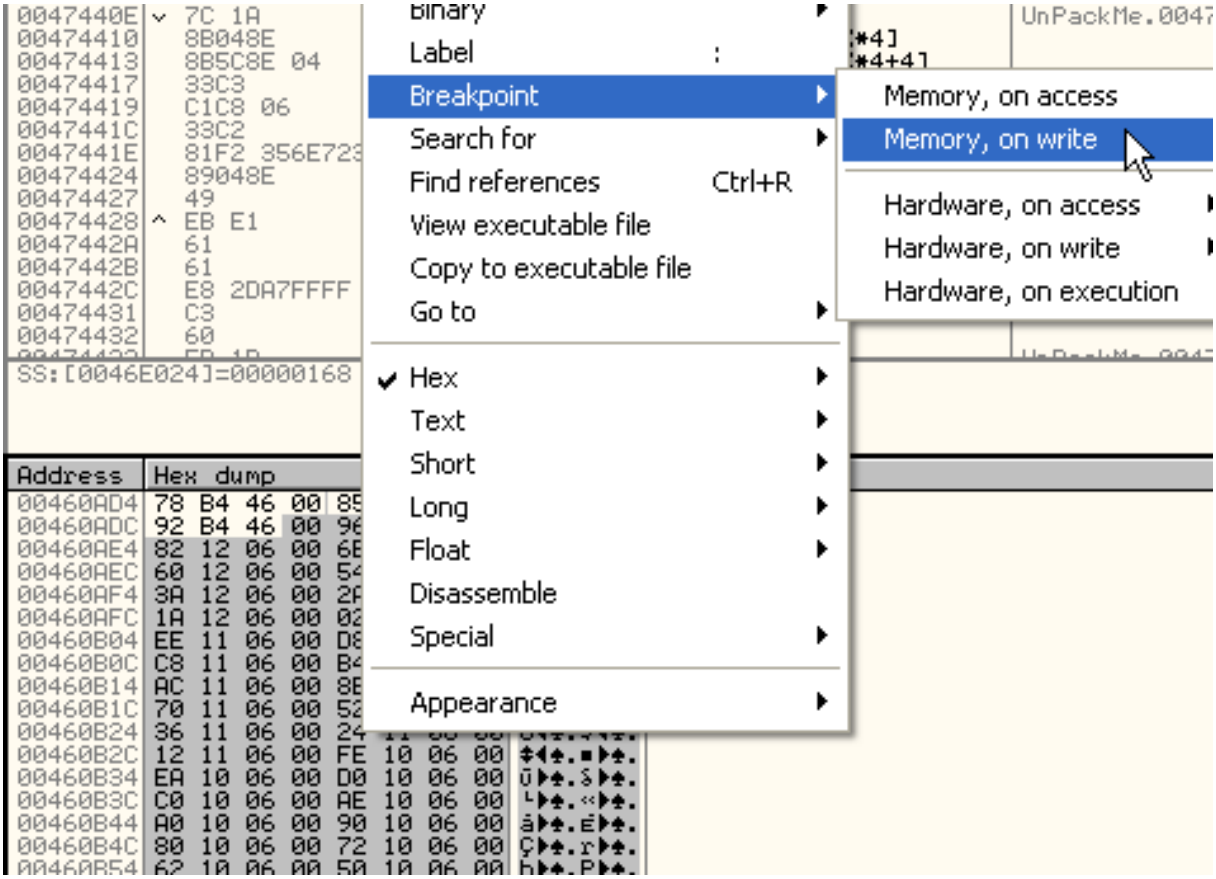
Если поднимемся чуть выше по стеку, то увидим там правильный адрес API-функции. Теперь нам нужно узнать, работает ли это для все переадресовочных элементов, так что останавливаем скрипт.



И смотрим, что произойдёт, если установим BPM ON WRITE на плохие элементы, которые ещё не заполнены из таблицы.

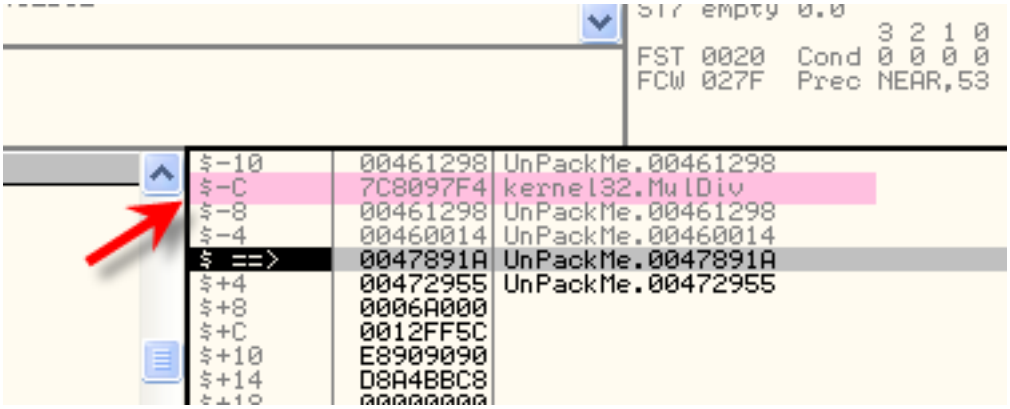


Делаем RUN и останавливаемся, чтобы остановиться на следующем элементе.

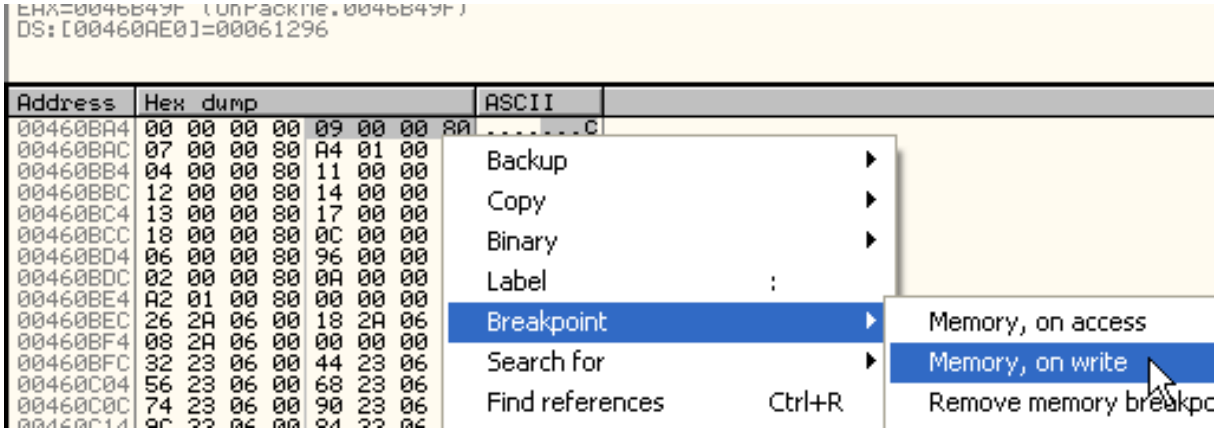


И видим, что это API-функция MulDiv. Если нас не затруднит пойти в переадресовочную область и сделать XOR, что ведёт сюда, так что мы видим возможность починки. Хорошая API-функция расположена в стеке, а точнее в [ebp-0c].

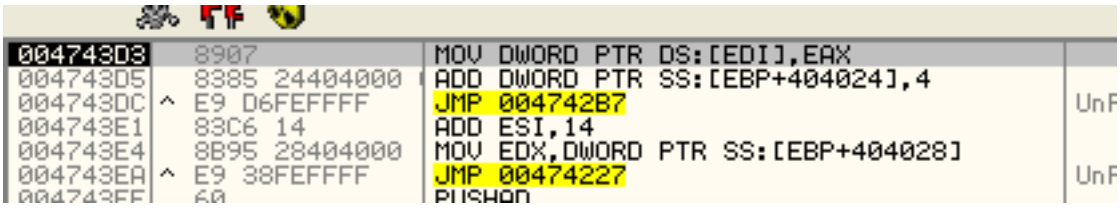
Видим, что когда сюда сохраняется что-нибудь хорошее, то происходит остановка в том же самом место. Посмотрим на образ IAT по состоянию на момент прибытия в OEP, и в 460ba8 находится хороший элемент.



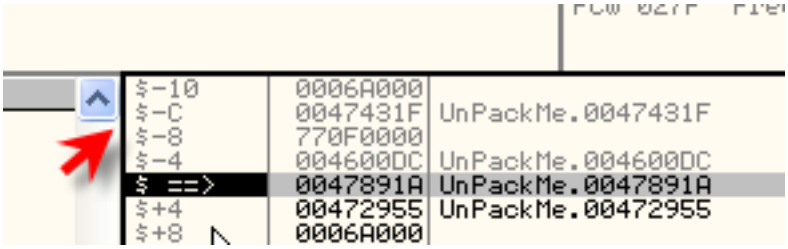
Так что устанавливаем BPM ON WRITE на него.



Делаем RUN и останавливаемся здесь.



Видим, что это тот же адрес, где сохраняются плохие элементы. Также в случае хороших элементов смотрим [ebp-0c], где находится правильный адрес.



Видим, что в случае с хорошими API-функциями правильного адреса здесь не сохраняется, что очень жалко, так как иначе было бы очень легко сделать скрипт, который был смотрел значение значение [esp-0c], но решить эту проблему также легко.

Ничего сложного, сделаем скрипт, используя тот же HBP.

Что мы сделаем? Установим HBP ON EXECUTION на то место, где сохраняются значения IAT, и когда скрипт определит, что остановились здесь, проверим, содержит ли EAX хорошее значение или плохое. В последнем случае сохраним значение [esp+0c] в [EDI], которое указывает на заполняемый элемент таблицы.

Смотрим скрипт.

```
var aux
var aux2

inicio:

bphws 4743d5, "x"

trabajo:

eob pirulo
run

pirulo:
log eip
cmp eip, 7c91eaec
je quitar
cmp eip, 7c91eb03
je restaurar
cmp eip,aux
je restaurar2
cmp eip,4743d5
je reparar
jmp final

quitar:
bphwc 4743d5
jmp trabajo

restaurar:
mov aux,esp
mov aux,[aux]
add aux,0b8
mov aux,[aux]
log aux
bp aux
jmp inicio

restaurar2:
bc aux
jmp inicio

reparar:
cmp eax, 500000
```

```
ja inicio
mov aux2, esp
sub aux2,0c
mov aux2, [aux2]
log aux2
mov [edi],aux2
jmp inicio

final:
MSGYN "Continuar?"
cmp $RESULT,1
je inicio
ret

--------------------------------------------------------------------------
```

Вот скрипт. Первое, что нужно пояснить – устанавливаем HBP ON EXECUTION на следующую инструкцию, то есть на 4743d5. Раз HBP ON EXECUTION, то остановка будет происходит точно в тот момент, когда происходит выполнение, чего не было бы с HBP ON WRITE или ON ACCESS.

Скрипт прост и основывается на том, что мы делали раньше, только устанавливаем HBP ON EXECUTION на 4743d5 – следующей строке после сохранения плохого значения.

cmp eip,4743d5

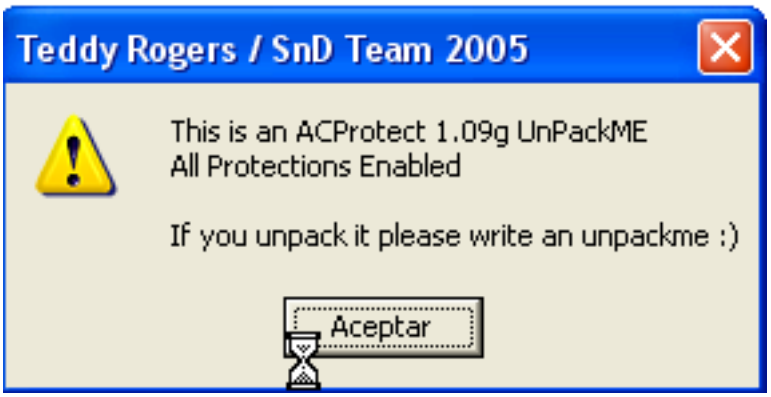
je reparar

В части pirulo, где скрипт получает контроль по нахождению исключения, добавляем к ней сравнение EIP с 4743d5, чтобы узнать, действительно ли находимся в желаемом месте, и оттуда прыгаем к reparar, где и творится магия.

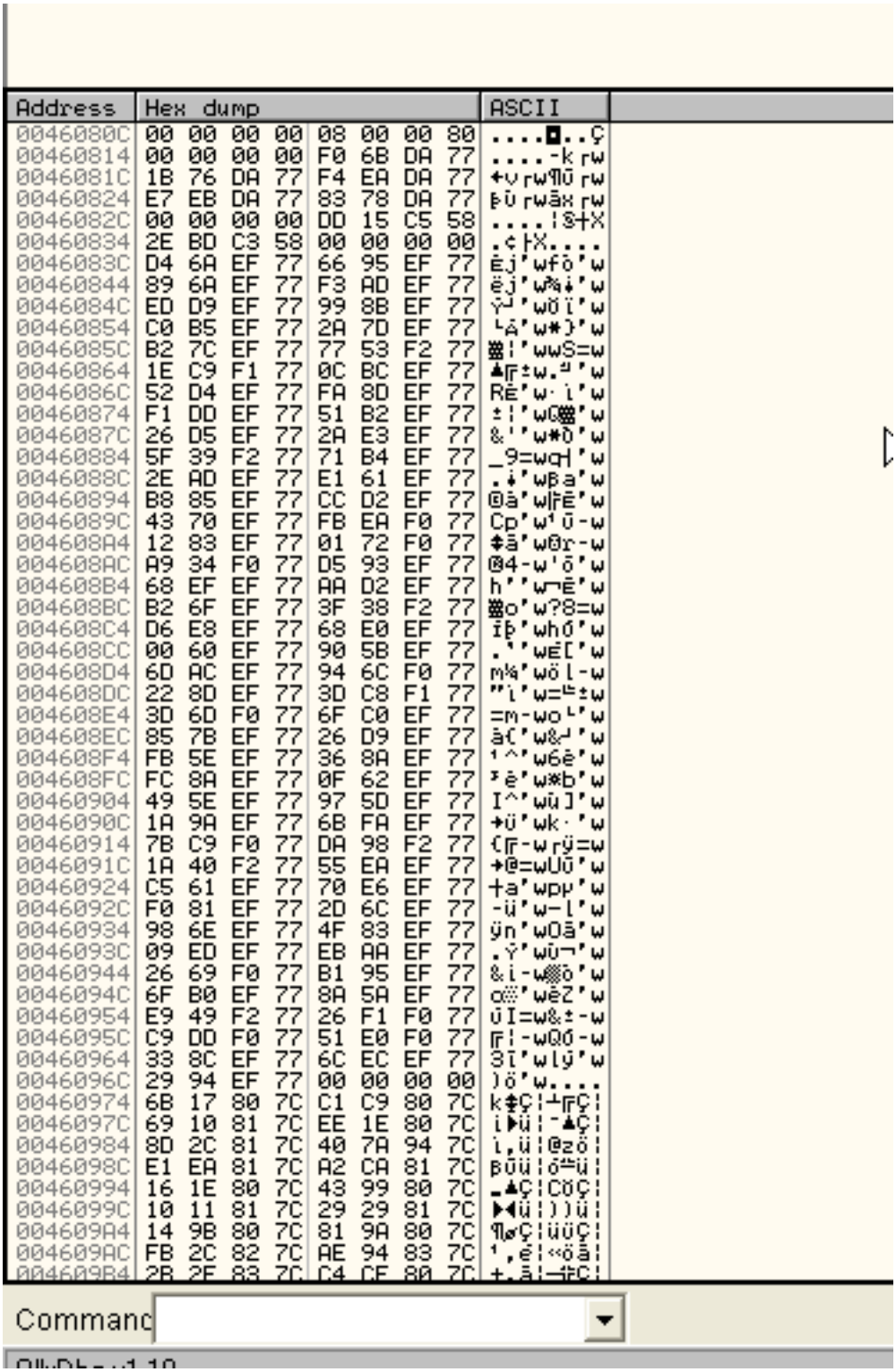
```
reparar:
cmp eax, 500000
ja inicio
mov aux2, esp
sub aux2,0c
mov aux2, [aux2]
log aux2
mov [edi],aux2
jmp inicio
```

В reparar мы делаем следующее: проверяем, содержит EAX хорошее значение или плохое, и как видим, все переадресовочные значения ведут в секцию упаковщика. Его адрес меньше 500000, поэтому если видим, что EAX больше 500000, то потому что это хорошая API-функция. В этом случае мы ничего не чиним и возвращаемся в начало. В противном случае у нас плохое значение, и используем переменную aux2, чтобы найти значение ESP, от которого отнимаем 0c, а затем находим его содержимое. Его записываем в лог, а затем сохраняем в [EDI], который указывает на элемент, откуда загружается плохое значение на предыдущей строке программы. Вот мы и разобрались с этим, затем возвращаемся и пробуем скрипт.

Перезапускаем программу. Обращаем внимание, чтобы не было старых HBP (есть ли есть, то стираем их) и убеждаемся, что уставлены два необходимых BP для правильной работы, и запускаем.

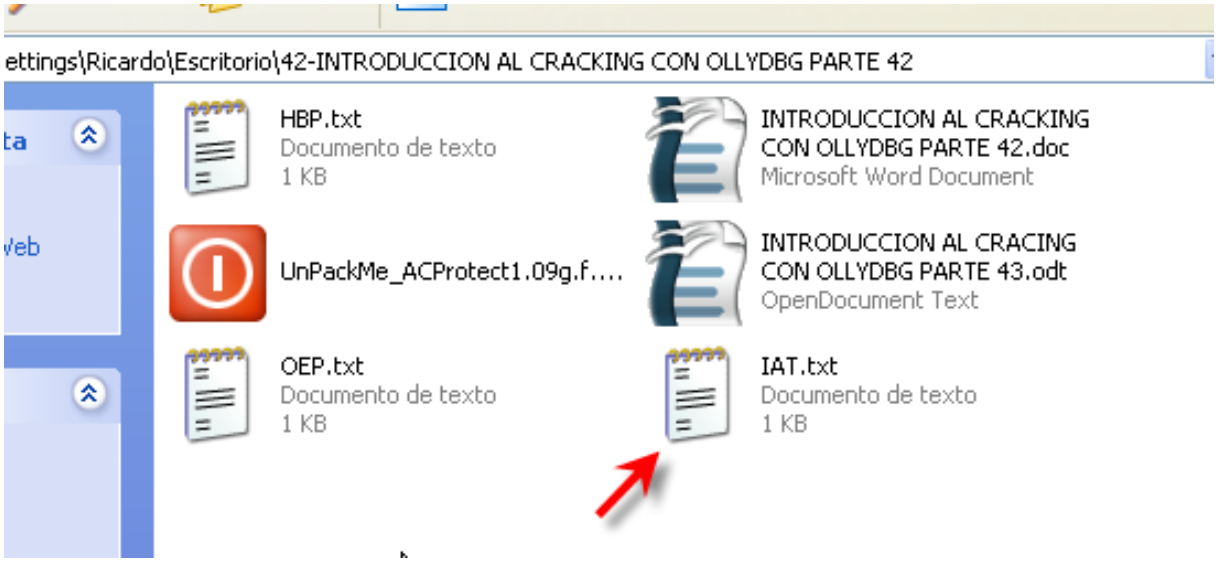


Запускается программа, смотрим, что осталось в IAT.

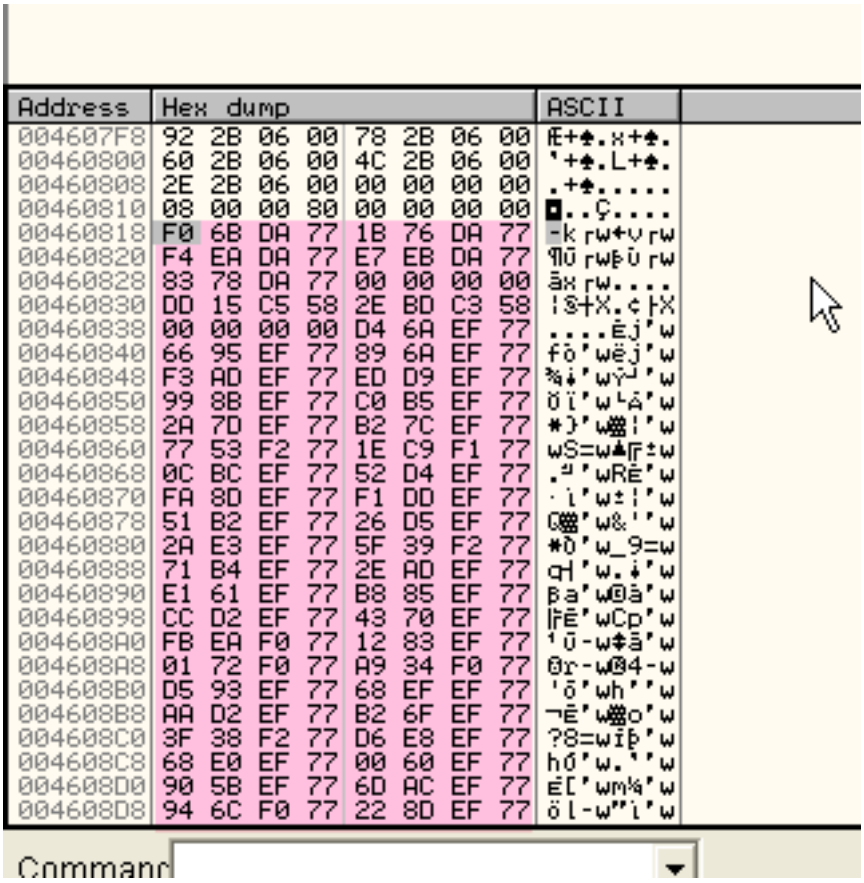


Красота, хе-хе. С помощью запущенной программы с починенным IAT можем починить дамп. Уже видели, что нет необходимости останавливаться на OEP, чтобы использовать процесс в IMP REC'e, когда есть правильная таблица, так что без проблем починим дамп.

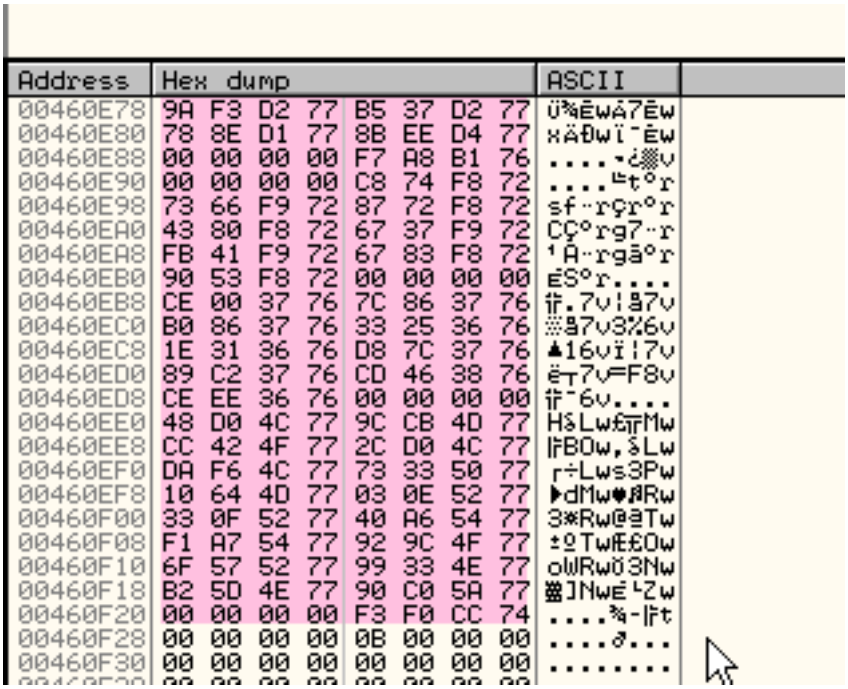
Переименовываем скрипт в IAT, и снова пробуем скрипты, теперь, когда они все разделены.



Единственное, что осталось, связанное с IAT – это найти начало и размер таблицы, которые необходимо указать в IMP REC'e.



Поднимаясь по IAT, ясно видим, что IAT начинается в 460818, конец в 460f28, и его также несложно найти:



Address	Hex dump	ASCII
00460E78	9A F3 D2 77 B5 37 D2 77	Ü¾ÉwÁ7Éw
00460E80	78 8E D1 77 8B EE D4 77	xÄÐwï¯Éw
00460E88	00 00 00 00 F7 A8 B1 76	....÷¿▒v
00460E90	00 00 00 00 C8 74 F8 72	....╚t°r
00460E98	73 66 F9 72 87 72 F8 72	sf¨rçr°r
00460EA0	43 80 F8 72 67 37 F9 72	CÇ°rg7¨r
00460EA8	FB 41 F9 72 67 83 F8 72	¹Á¨rgâ°r
00460EB0	90 53 F8 72 00 00 00 00	ÉS°r....
00460EB8	CE 00 37 76 7C 86 37 76	╬.7v¦å7v
00460EC0	B0 86 37 76 33 25 36 76	▒å7v3%6v
00460EC8	1E 31 36 76 D8 7C 37 76	▲16vï¦7v
00460ED0	89 C2 37 76 CD 46 38 76	ë┬7v═F8v
00460ED8	CE EE 36 76 00 00 00 00	╬¯6v....
00460EE0	48 D0 4C 77 9C CB 4D 77	HšLw£╦Mw
00460EE8	CC 42 4F 77 2C D0 4C 77	╠BOw,šLw
00460EF0	DA F6 4C 77 73 33 50 77	┌÷Lws3Pw
00460EF8	10 64 4D 77 03 0E 52 77	▶dMw♥♫Rw
00460F00	33 0F 52 77 40 A6 54 77	3*Rw@ªTw
00460F08	F1 A7 54 77 92 9C 4F 77	±§Tw'£Ow
00460F10	6F 57 52 77 99 33 4E 77	oWRwÖ3Nw
00460F18	B2 5D 4E 77 90 C0 5A 77	▓]NwÉ└Zw
00460F20	00 00 00 00 F3 F0 CC 74	....¾-╠t
00460F28	00 00 00 00 0B 00 00 00	....♂...
00460F30	00 00 00 00 00 00 00 00	....

• OEP=271b5
• INICIO=60818
• LARGO= 460f28-460818= 710

Можем использовать фальшивый OEP, нет проблем – потом это можно исправить вручную.

Друзья, в следующей части будем решать антидампы, так что оставайтесь на связи.

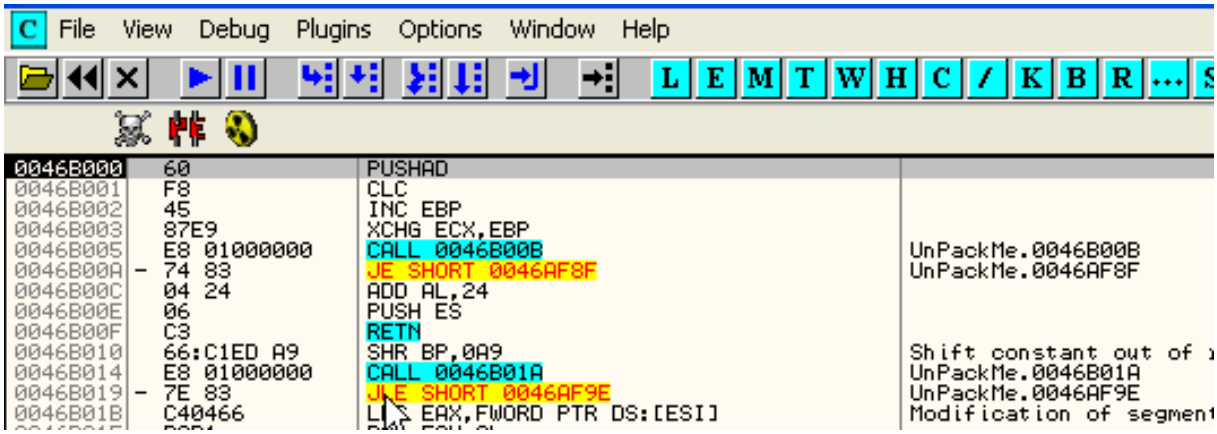
Файлы к статье (находится в архиве `wasm_files.zip`: `files/0516/ollydbg43.zip`)

Введение в крэкинг с нуля, используя OllyDbg - Глава 44

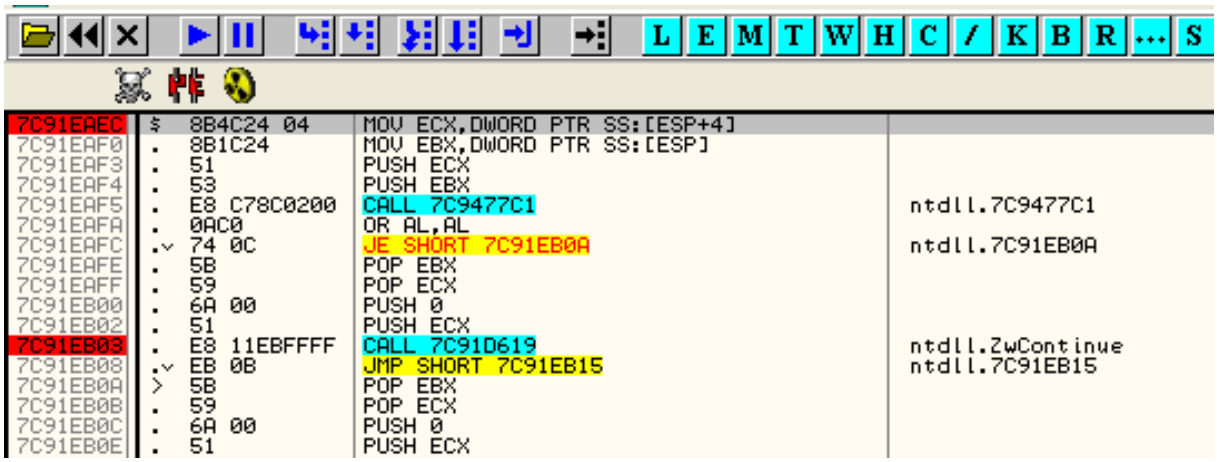
Автор: Рикардо Нарваха, пер. Aquila

Дата: 2009-06-25

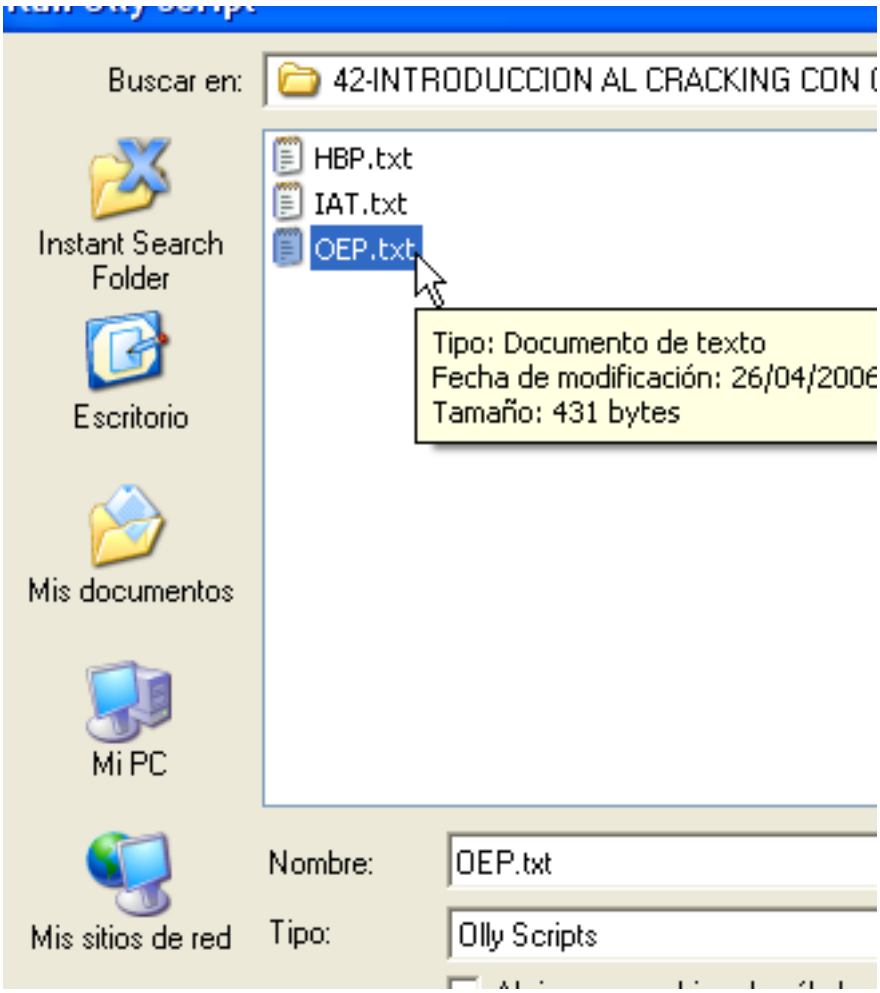
Ок, продолжаем как обычно. Мы уже нашли украденные байты, и у нас теперь есть скрипты для того, чтобы легко попасть в OEP и чтобы починить IAT, так что осталось сделать дамп.

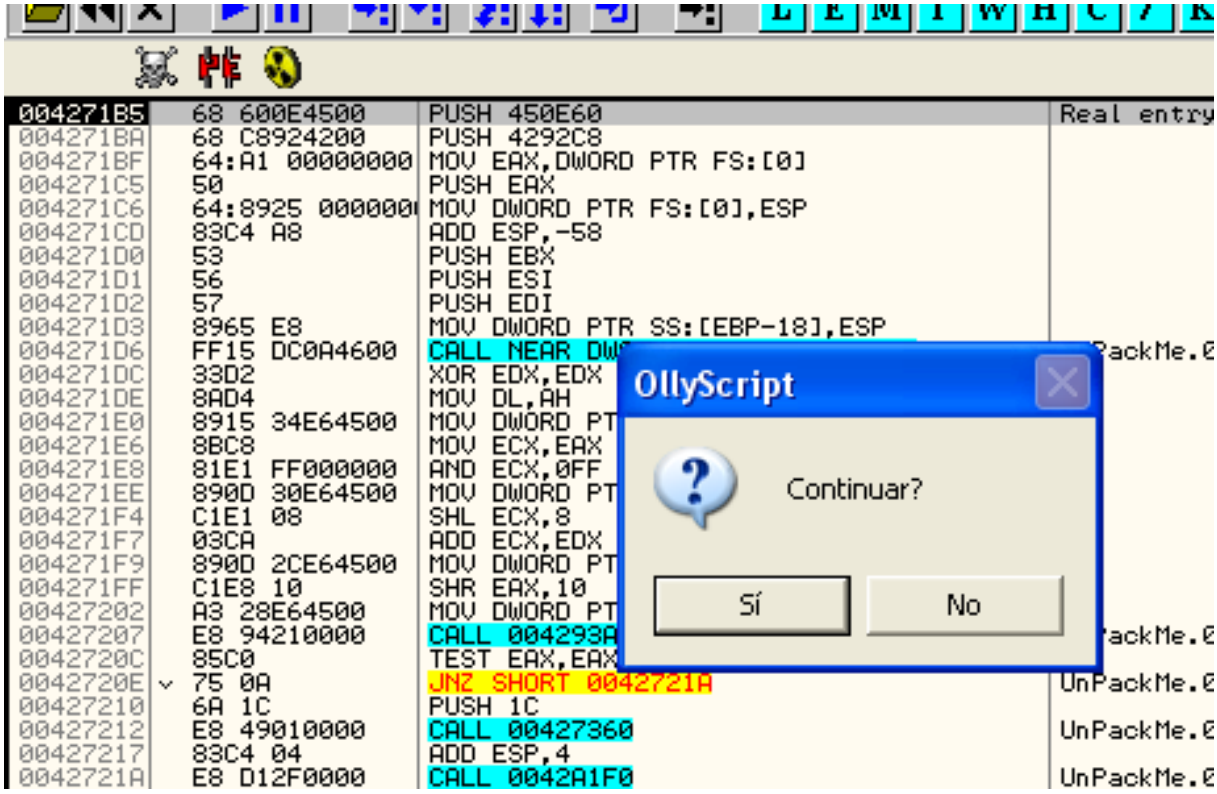


Открываем программу в OllyDbg и устанавливаем два BP, необходимых для работы скриптов, и стираем BPX, которые могли остаться от прошлого раза.

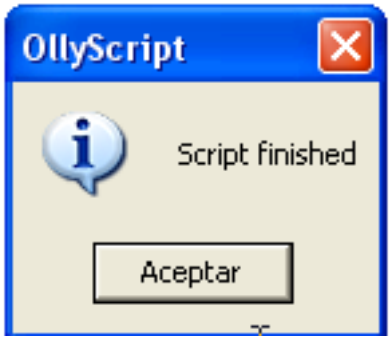


Теперь используем скрипт для OEP и тут же попадаем в него.

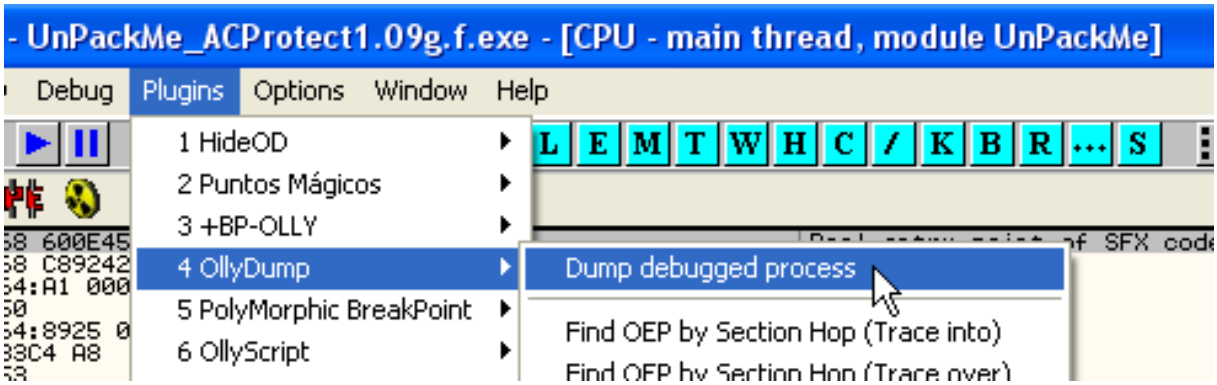


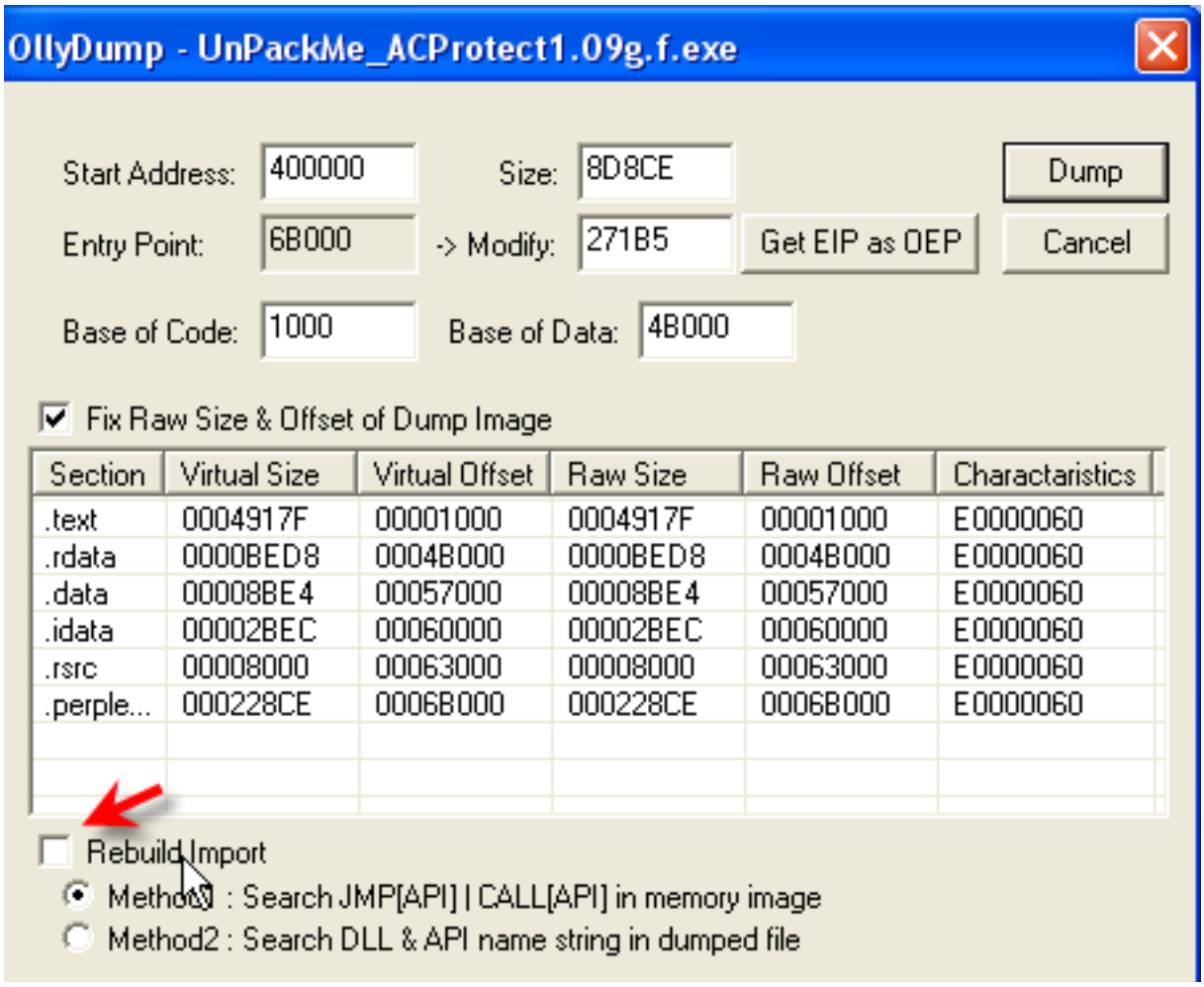


Отвечаем, что нет и находимся в OEP.

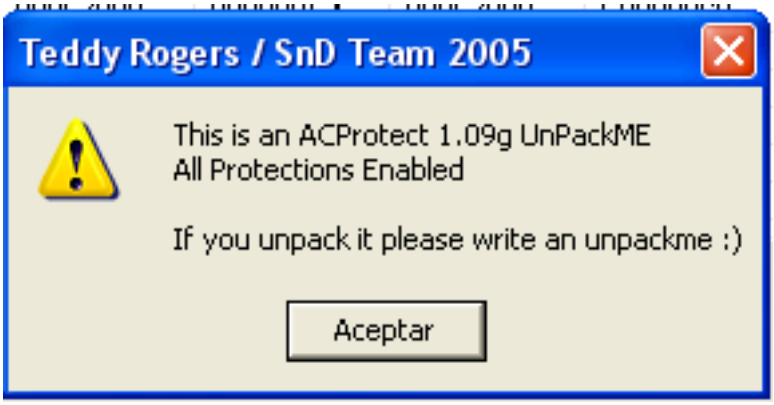


Теперь делаем дамп.





Снимаем галку и сохраняем как dumped.exe или другое имя, которое мы хотим. Теперь перезапускаем OllyDbg и используем скрипт для того, чтобы работать с починенной таблицей.



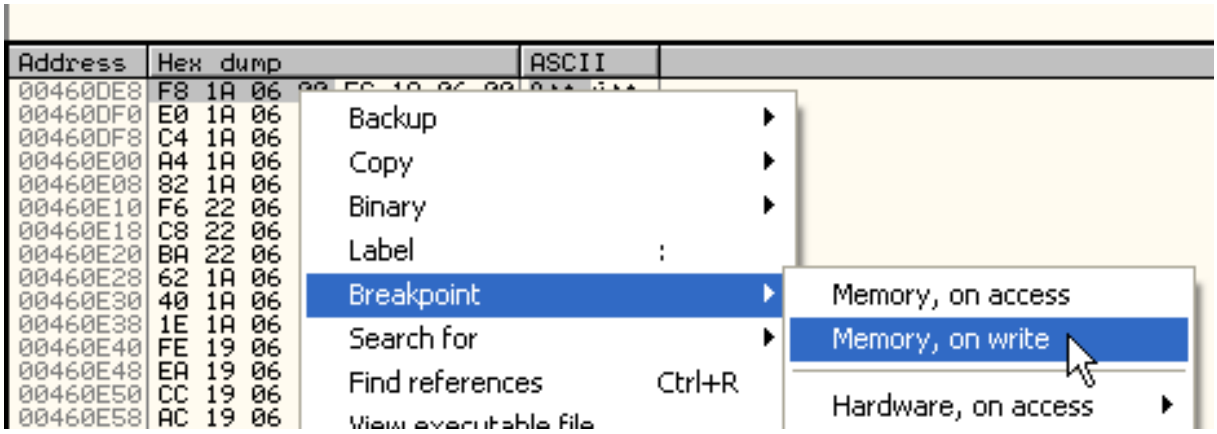
Здесь уже всё работает, так что открываем IMPORT RECONSTRUCTOR и ищем нужный нам процесс в списке процессов в выпадающем меню.

Теперь ищем данные OEP, RVA и РАЗМЕР IAT, как мы это делали в прошлой части.

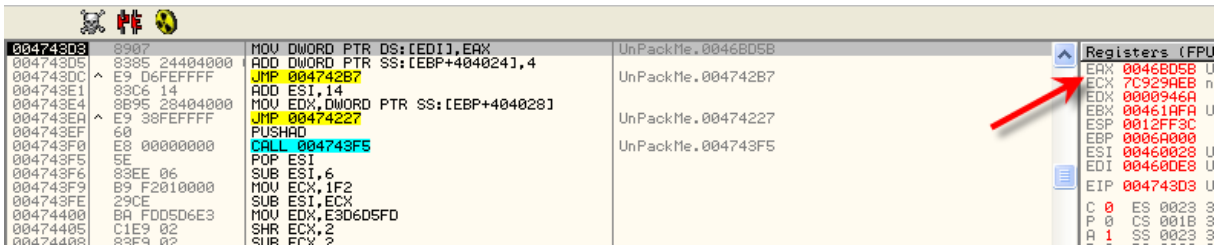
```
OEP=271b5
НАЧАЛО=60818
РАЗМЕР= 460f28-460818= 710
```

Видим, что все API-функции правильны кроме одной. Я её пропустил?

Можем посмотреть, что случится, если бы эта ошибка не была показана. Рестартуем OllyDbg и устанавливаем на этот элемент MEMORY BREAKPOINT ON WRITE.

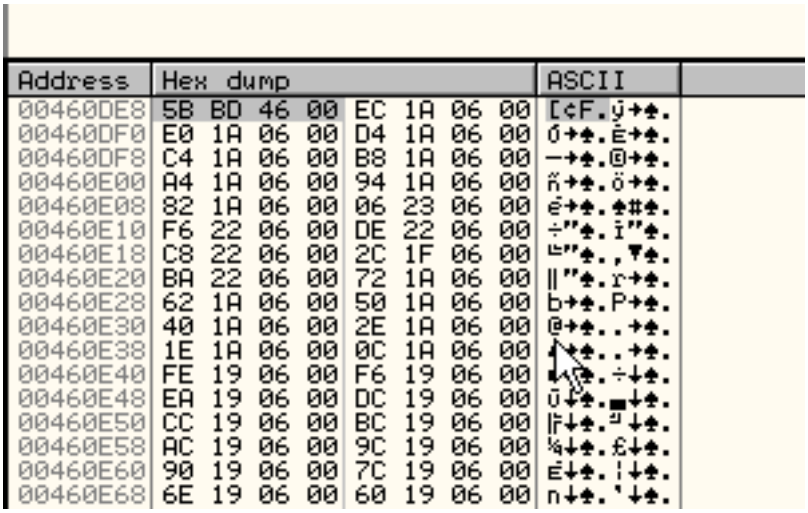


И делаем RUN, чтобы оказаться в месте, где сохраняется плохое значение.

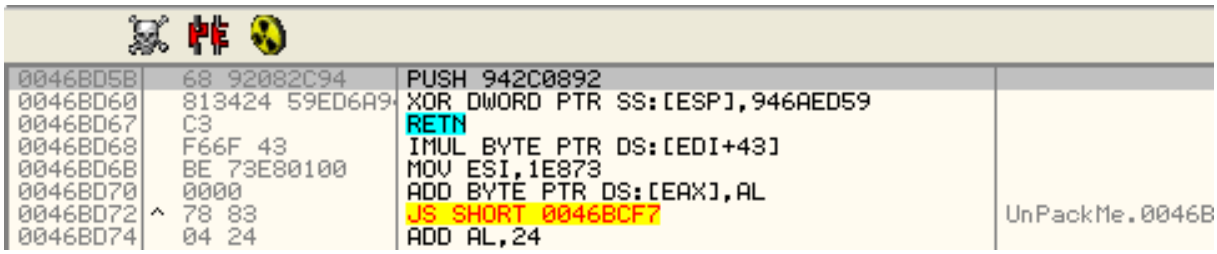


Здесь видим, что будет сохраняться значение и в ESP-0C нет никакой API-функции.

Видим, что скрипт изменил его на значение, которое в данном случае равно 46E5CB, так что видим значение, которое сохраняется в IAT без использования скрипта. Нажимаем F7.



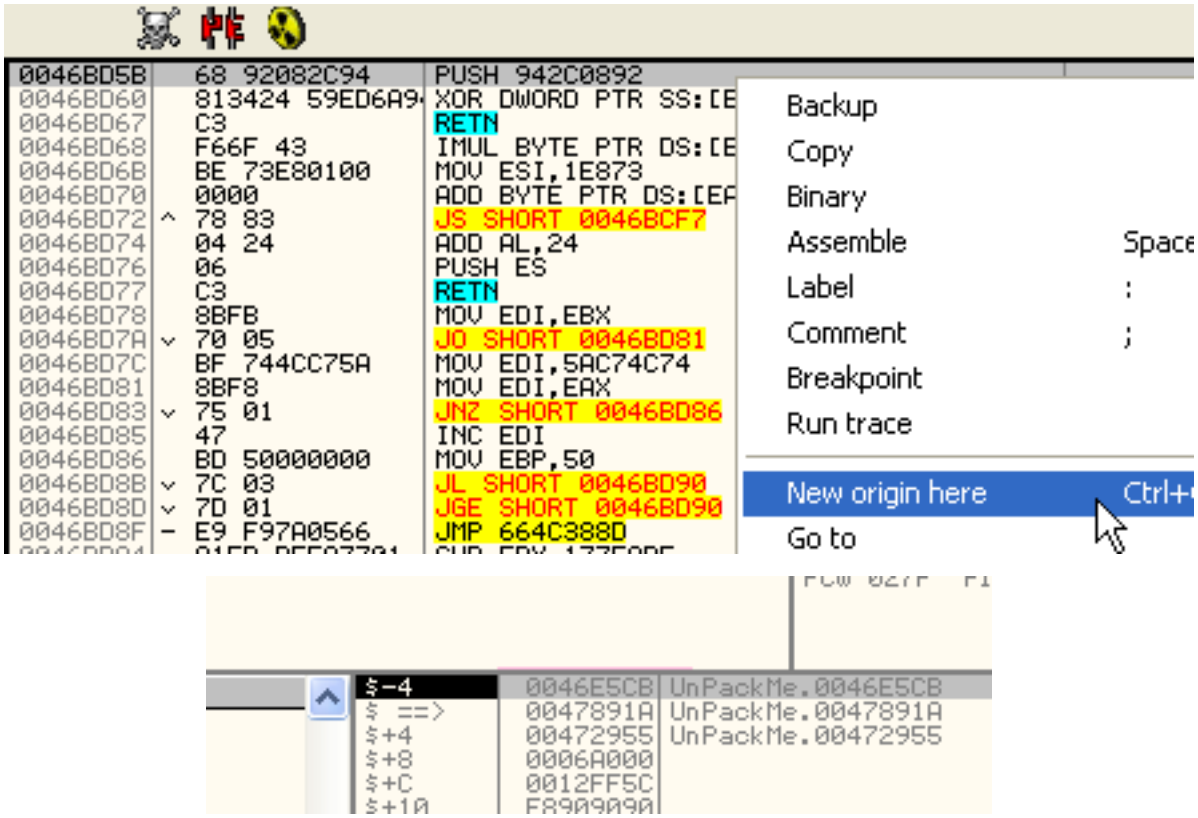
Значение, сохраняющееся без использования скрипта, будет плохим, так что смотрим, куда оно ведёт. Идём в 46BD5B.



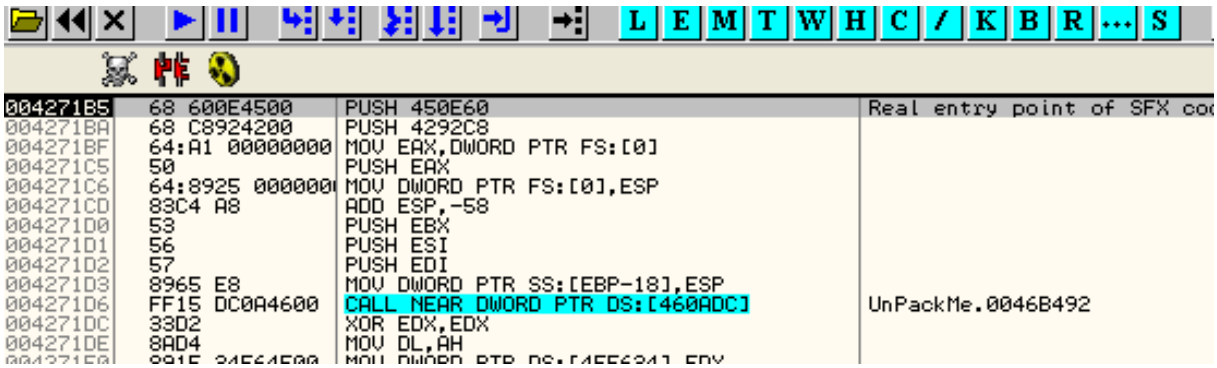
Хорошо, как видели раньше, выполняется PUSH константы, и над ней совершается XOR с другой константой, а результатом этой операции становится адрес API-функции. Считаем.

942c0892 xor 946aed59

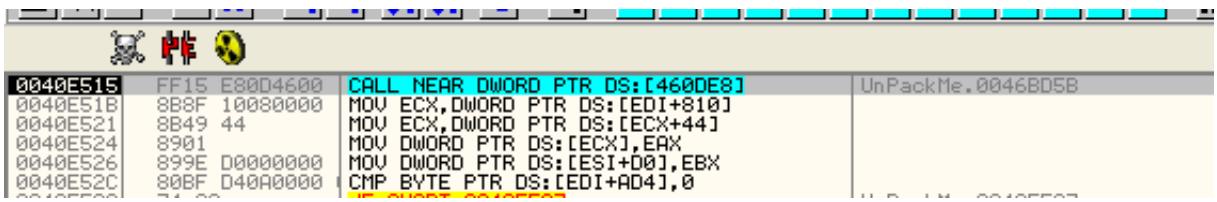
Если нет желания использовать калькулятор, может использовать то, что OllyDbg выполнит сама эти строки и получит результат, хе-хе.



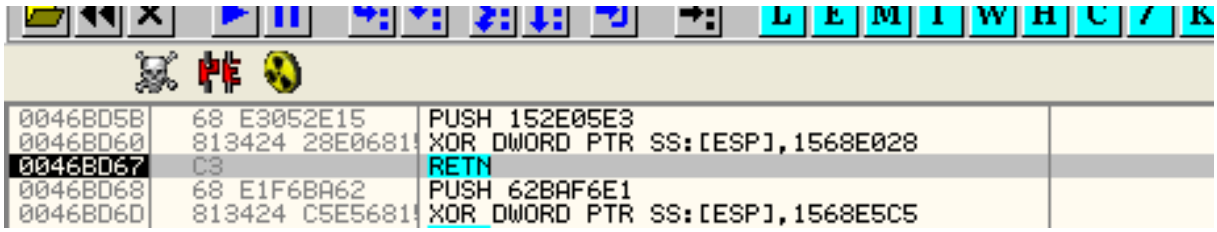
Видим, что результатом XOR является 46E5CB – это значение, которое скрипт поместит в элемент, поэтому скрипт не ошибается. Дело в том, что этот элемент особенный, так что рестартуем и прибываем в OEP, что посмотреть, найдём ли то, к чему динамически относится API-функция, то есть, откуда она запускается.



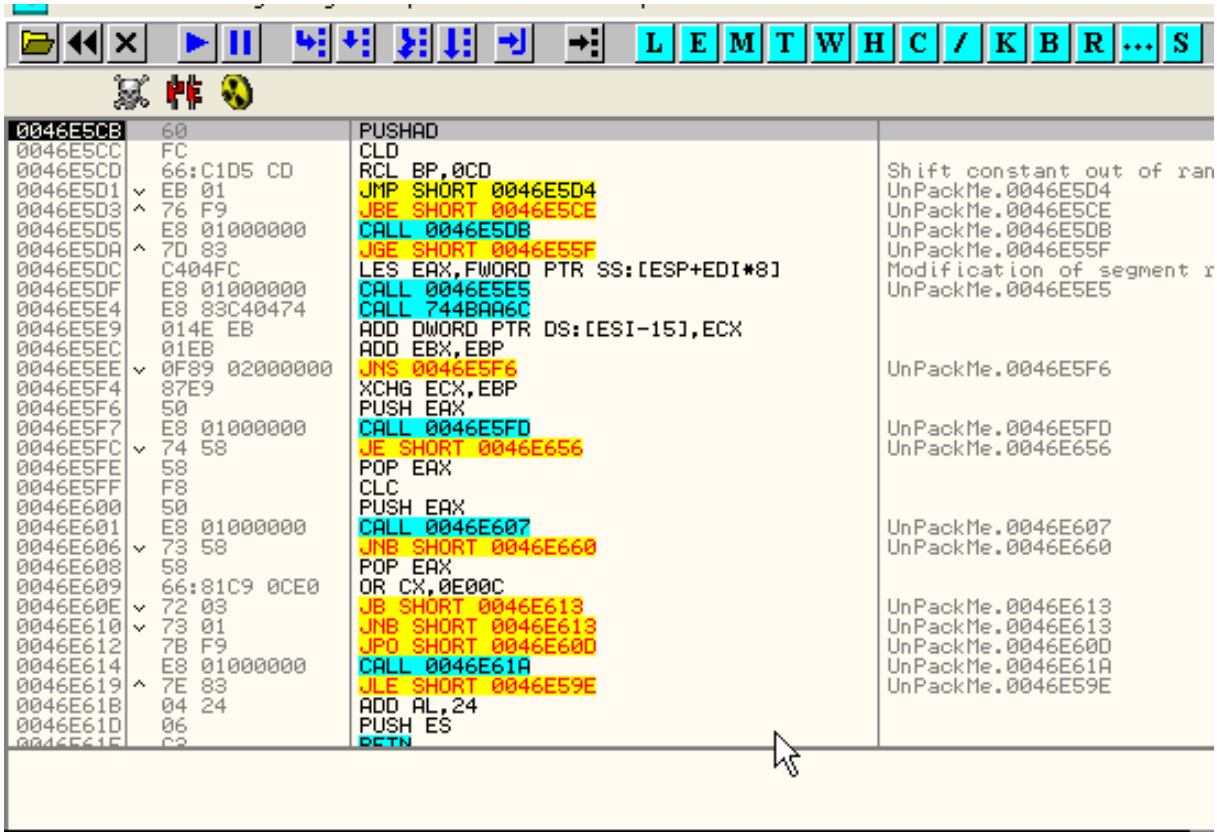
Тут оказываемся в OEP с помощью соответствующего скрипта и устанавливаем BPM ON ACCESS на элементе IAT, относительно которого хотим узнать, к какой API-функции он ведёт, для чего останавливаемся, чтобы получить к нему доступ.



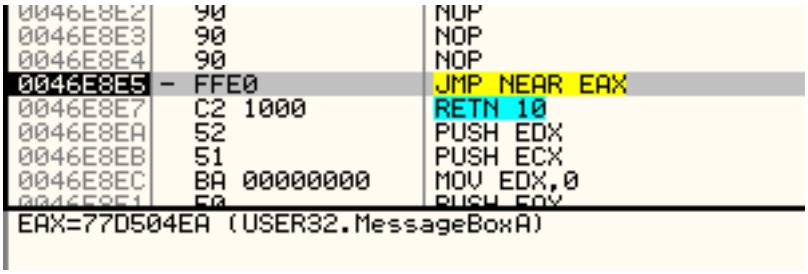
Останавливаемся, трассируем.



Конечно, доходим до RET и отсюда прыгаем:

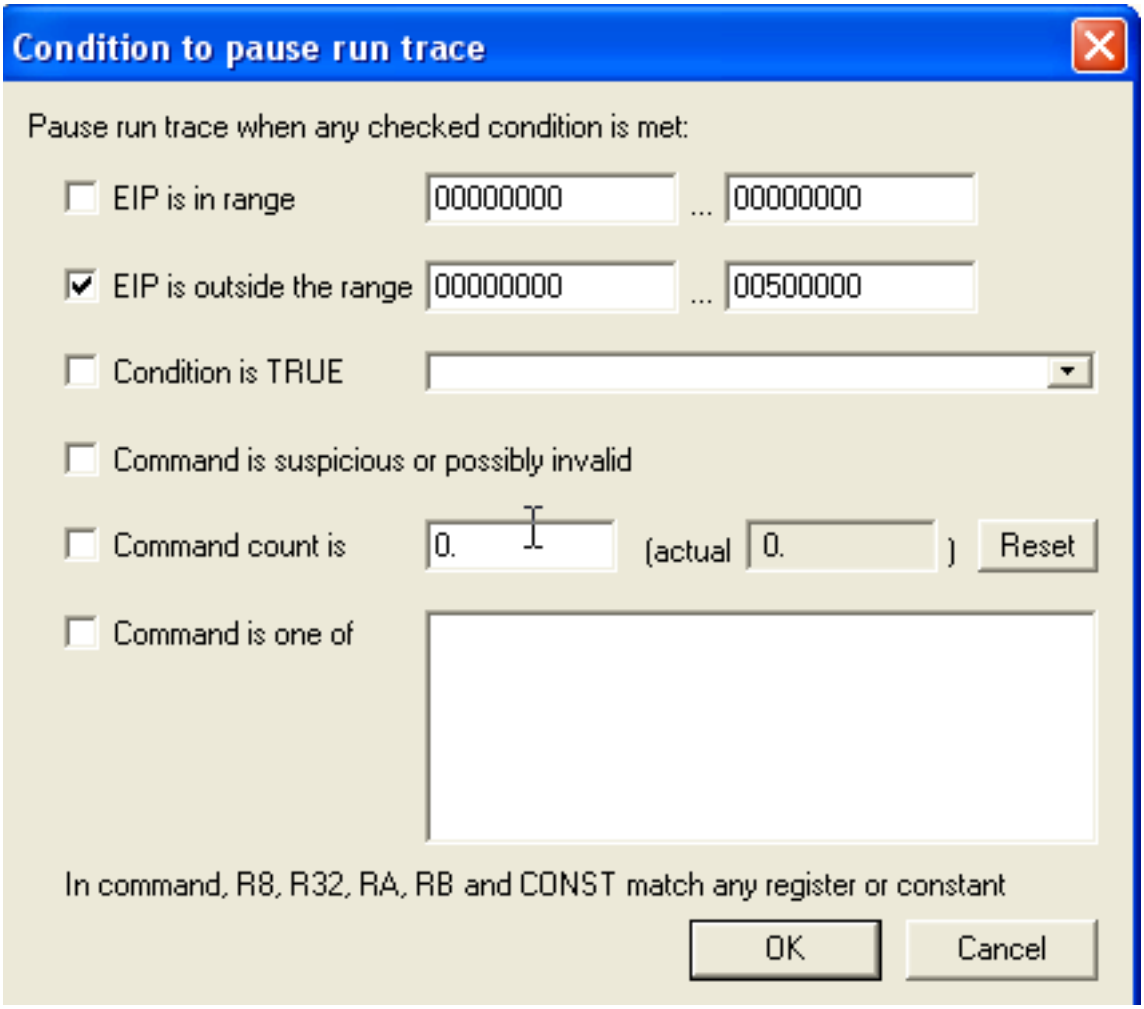


Посмотрим, ведёт ли это к какой-нибудь API-функции. Начинаем трассировать с помощью F7 и видим, что это ведёт в большой кусок кода, так что применяем метод pushad. Вот эта инструкция в начале, и останавливаемся на POPAD, до куда нужно оттрасировать некоторое количество строк.

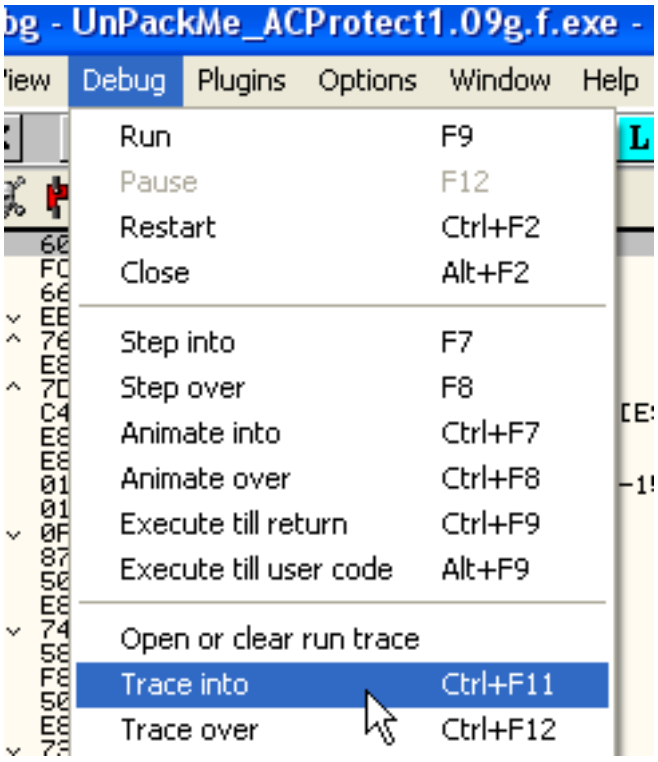


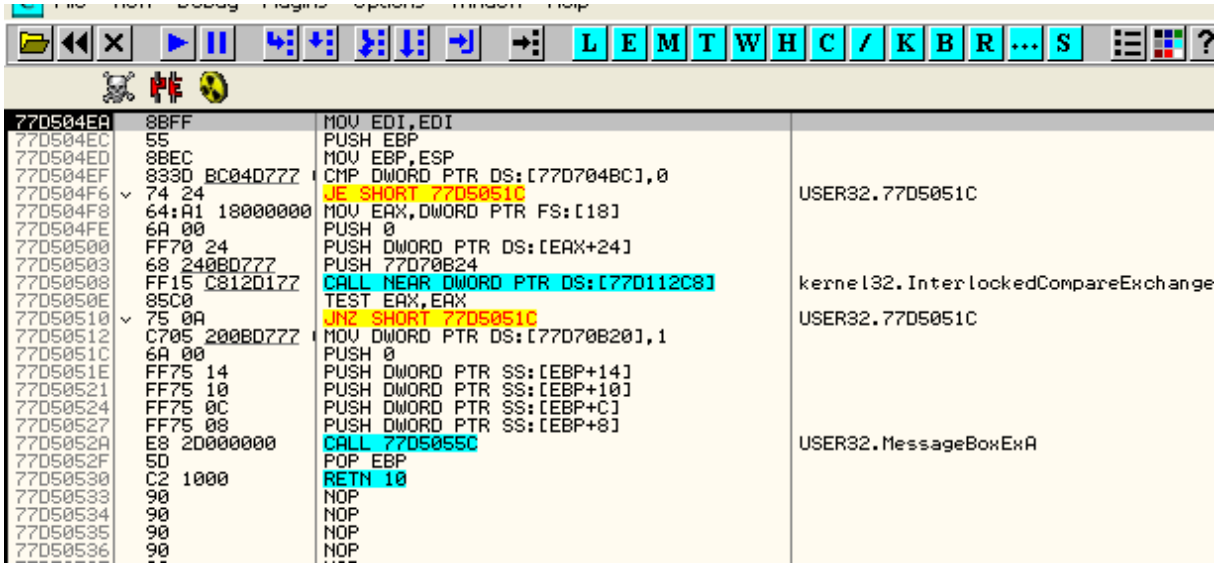
И видим, что API-функция, с которой у нас проблемы – это MessageBoxA.

Другой, более быстрый метод – это сделать RUN TRACE с условием, что необходимо остановиться, когда EIP станет, например, больше 500000, то есть когда произойдёт выход из секции в API-функцию. Смотрим:

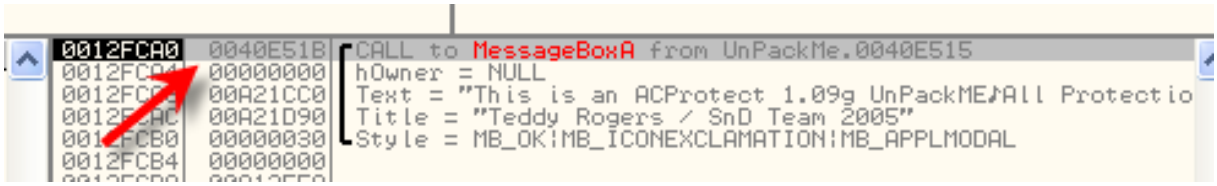


Здесь останавливаемся, когда EIP выходит за пределы 0-500000. Пробуем, сработает ли.

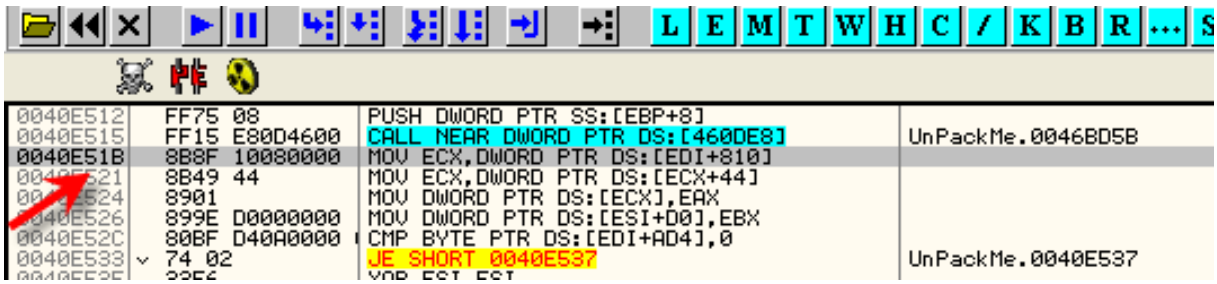




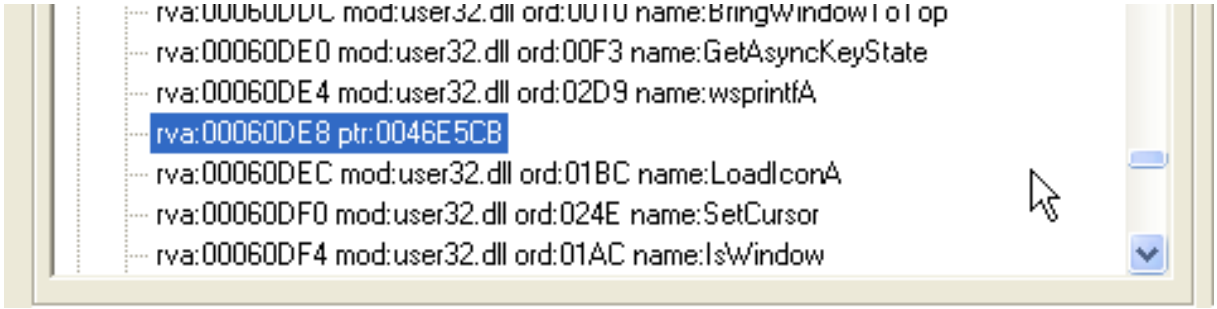
Останавливаемся прямо на API-функции MessageBoxA. Последняя проверка, чтобы убедиться, что это правильная API-функция и не фальшивка, заключается в том, чтобы возвратиться в исполняемый файл в то место, где находится продолжение вызова, из которого отправились. Это можно узнать в первой строке стека. Смотрим:



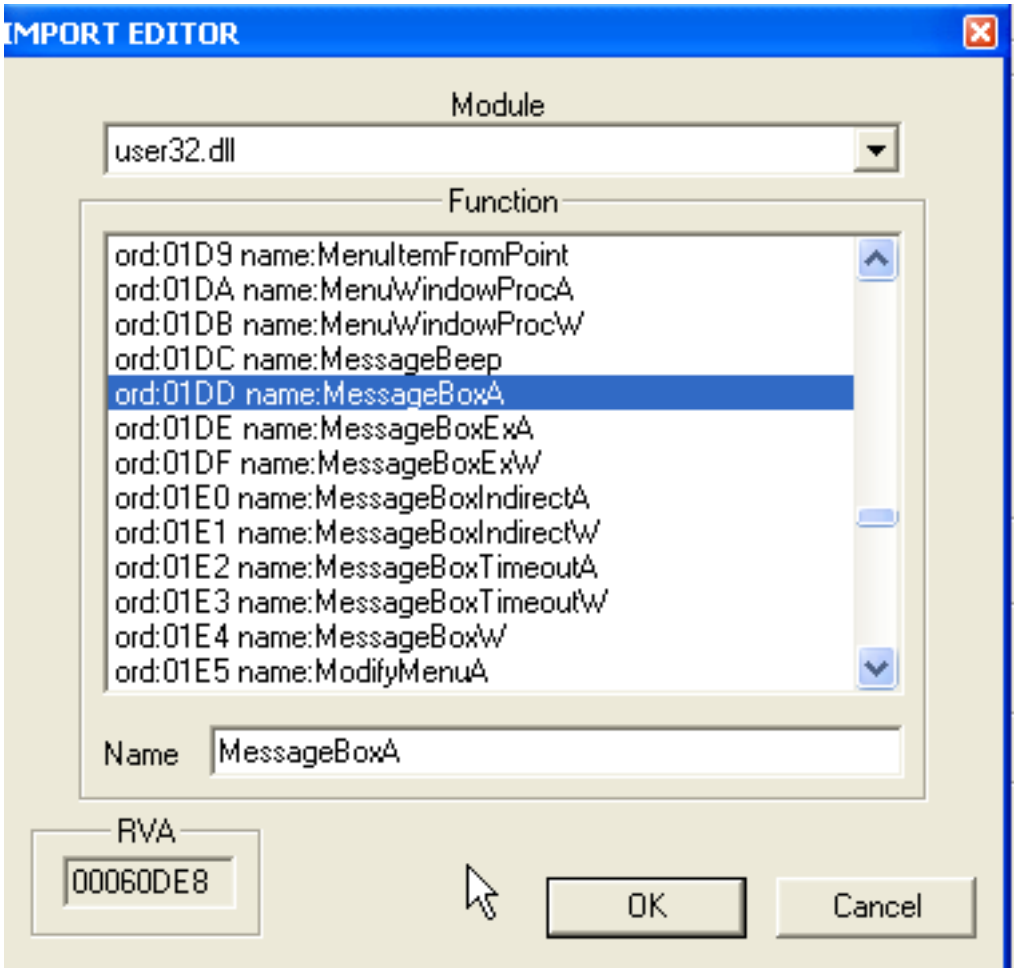
Адрес возврата – это 40E51B. Если пойдём туда:



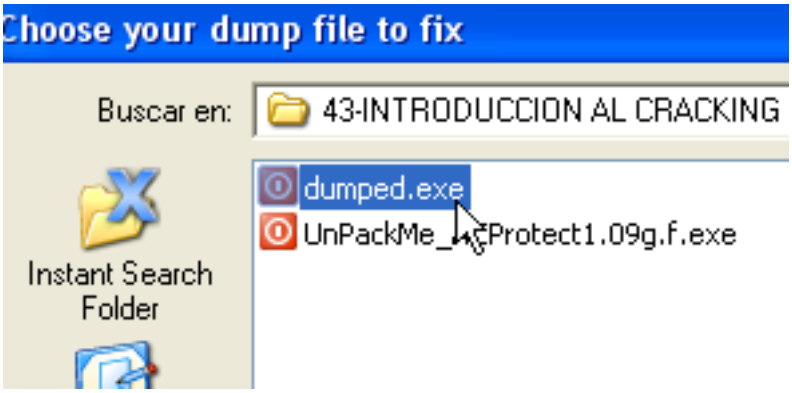
Возвращаемся прямо на следующую строку вызванного call'а, так что у нас уже есть все данные. Перезапускаем процесс с помощью скрипта для IAT и устанавливаем данные как в прошлый раз в IMP REC.



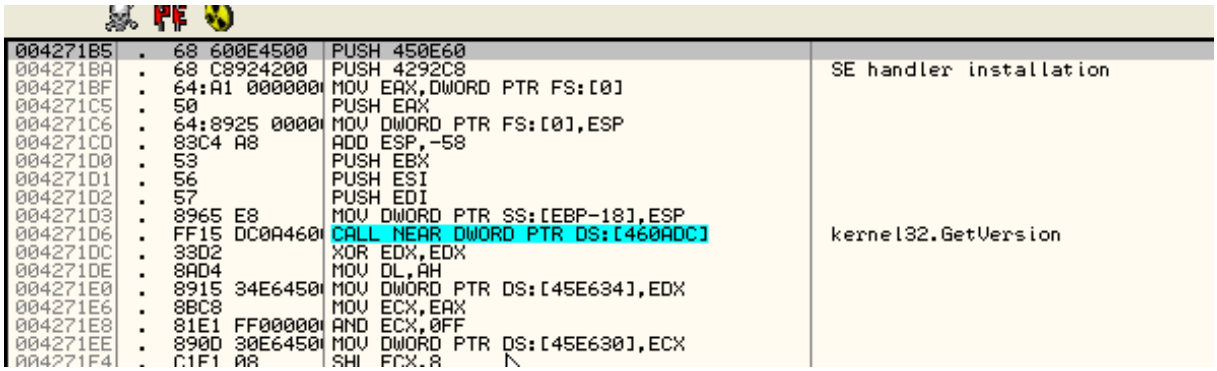
Делаем двойной щелчок по неправильному элементу и меняем его на MessageBoxA.



Теперь, если всё правильно, починим дамп с помощью FIX DUMP. Ищем файл дампа.



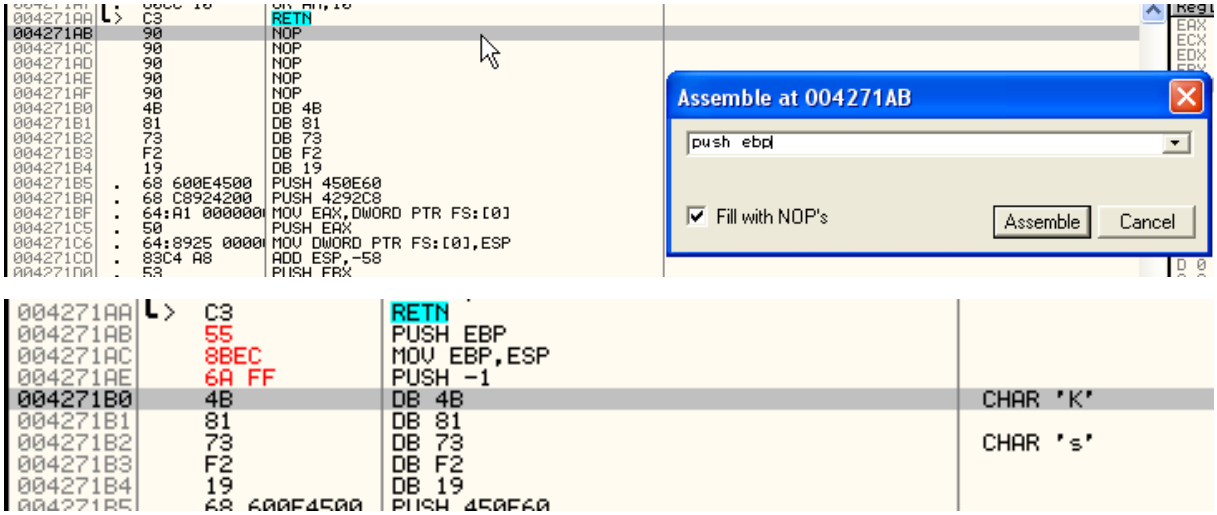
И сохраняем его как dumped_.exe. Нам осталось исправить OEP на предмет украденных байтов, так что открываем файл в OllyDbg.



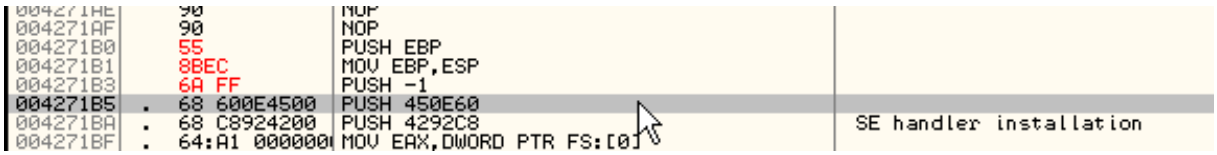
Нам нужно скопировать украденные байты и поменять OEP.

```
00485AF3 Main     PUSH EBP
00485AF4 Main     MOV EBP,ESP                              ; EBP=0012FFC0
00485AF6 Main     PUSH -1
```

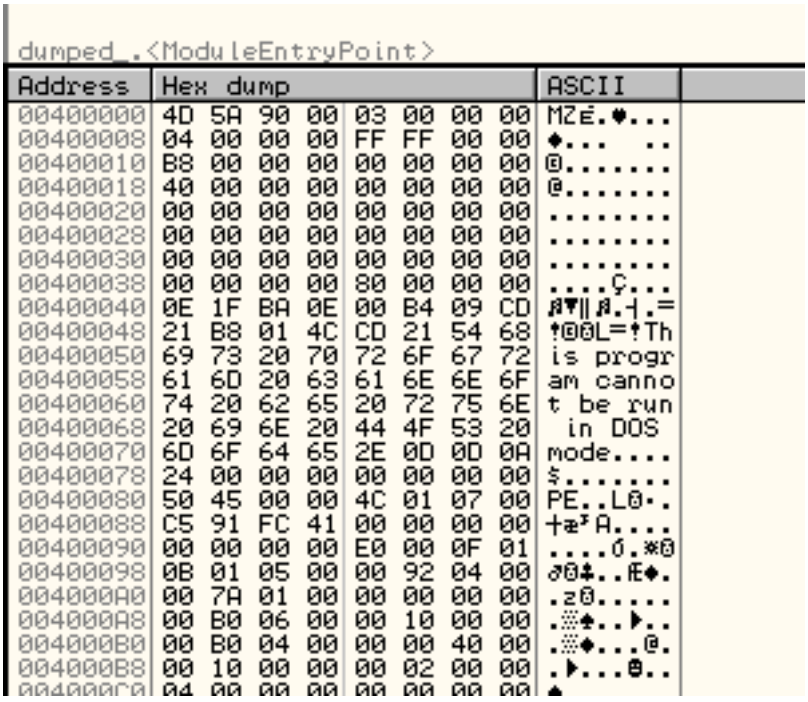
Вот украденные байты, которые мы можем ввести с помощью функции «Assemble».



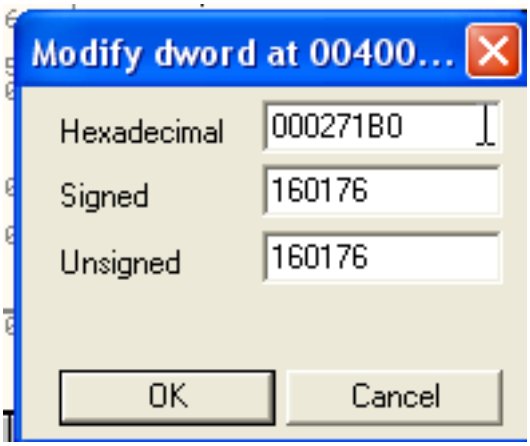
Видим введённые байты, которых ровно пять, так что теперь считаем: 4271b5 (фальшивая OEP) минус пять байт дают нам правильный OEP, то есть 4271b0.

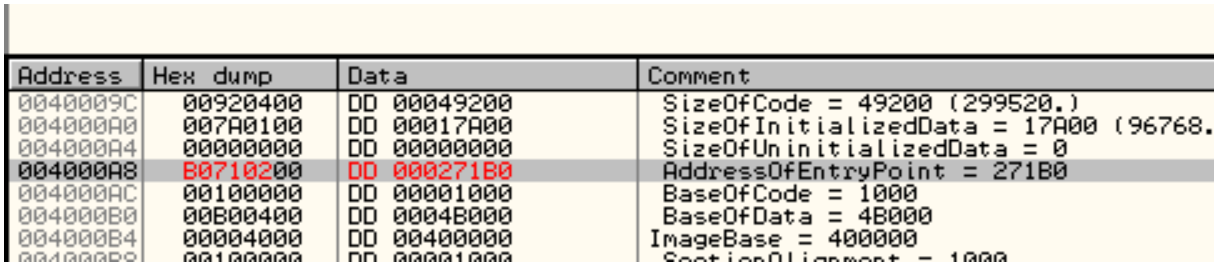


Теперь, если сохраним изменения с помощью COPY TO EXECUTABLE – SAVE FILE, мне останется поменять OEP. Перезапускаем и идём в заголовок дампа с помощью GOTO EXPRESSION 400000.

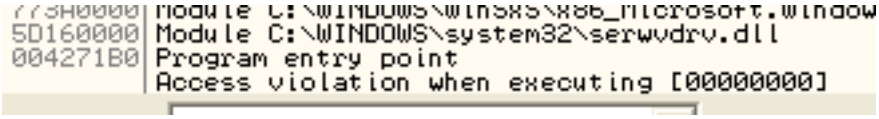


Переходим в режим SPECIAL-PE HEADER, ищем значение ADDRESS OF ENTRY POINT, меняем его и сохраняем изменения.

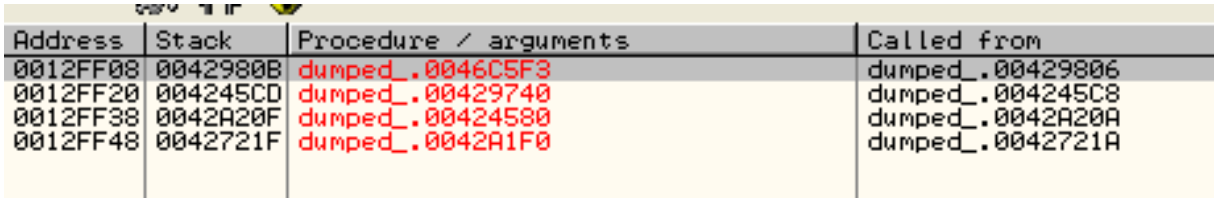




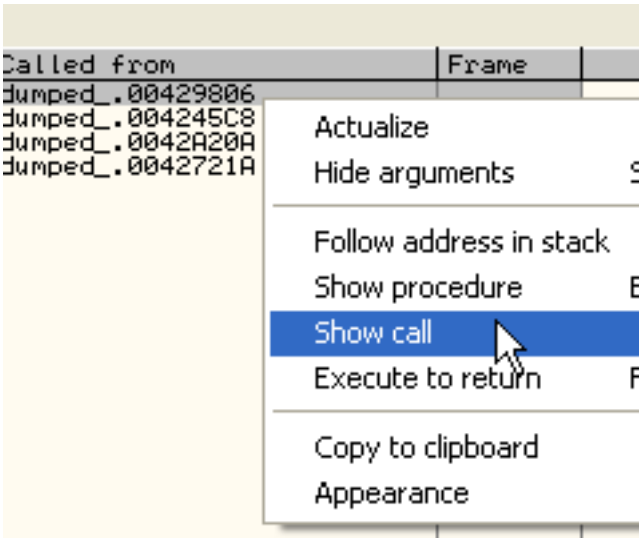
Ок, если запустим файл после починки, то будет показана ошибка. Снимаем все галочки с исключений и смотрим, какой ещё трюк был применён против нас.



Видим в LOG'e, что произошла ошибка, смотрим CALL STACK, то есть нажимаем кнопку с буквой «K», показывающую последние исполненные call'ы (можем также посмотреть в стеке RETURN TO…).

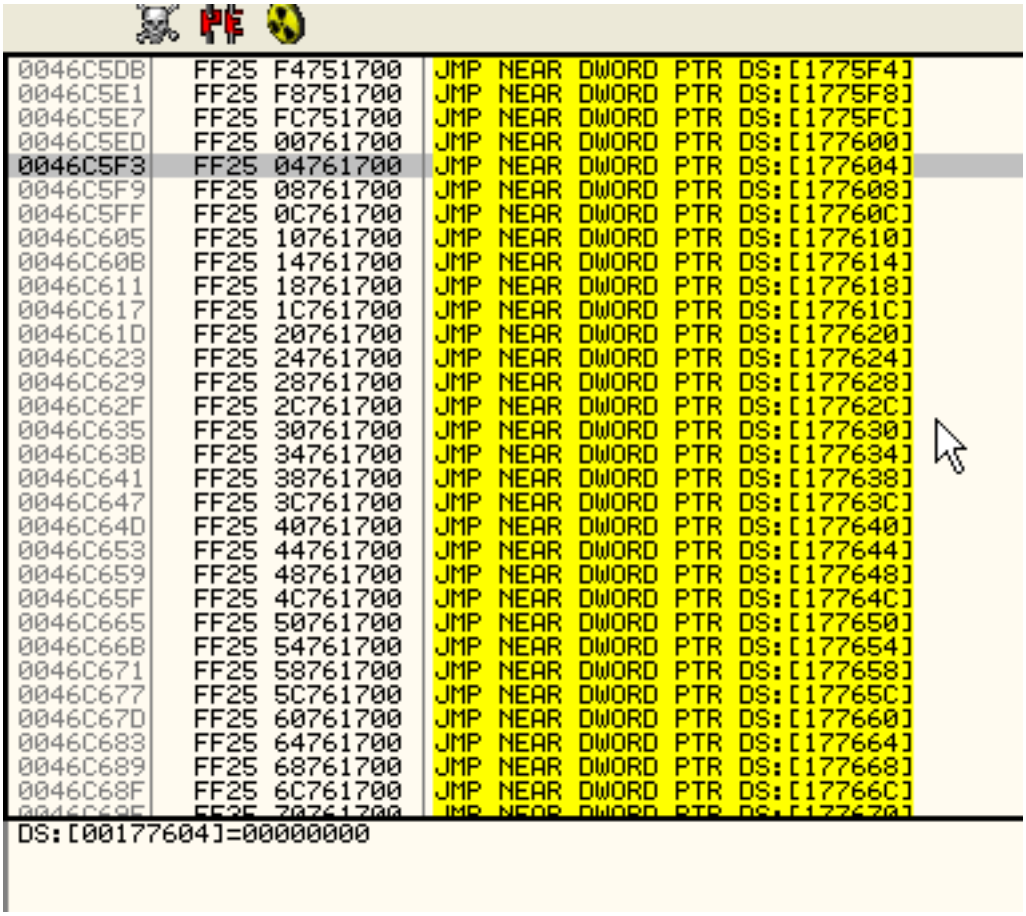


Последний CALL был выполнен из 429806, идём туда.



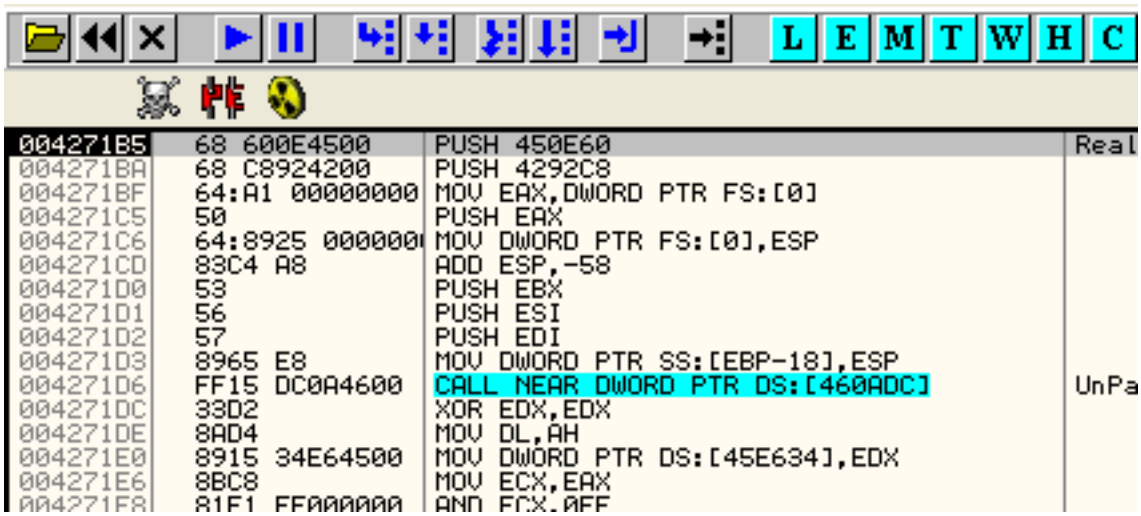
```
00429802  .  8B17          MOV EDX,DWORD PTR DS:[EDI]
00429804  .  2BD3          SUB EDX,EBX
00429806  .  E8 E82D0400   CALL 0046C5F3                    dumpe
0042980B  .  5F            POP EDI
0042980C  .  5E            POP ESI
0042980D  .  5D            POP EBP
0042980E  .  5B            POP EBX
0042980F  .  59            POP ECX
00429810  .  C3            RETN
00429811  >  8B4C24 10     MOV ECX,DWORD PTR SS:[ESP+10]
```

Продолжаем с помощью FOLLOW, чтобы посмотреть куда идёт CALL.



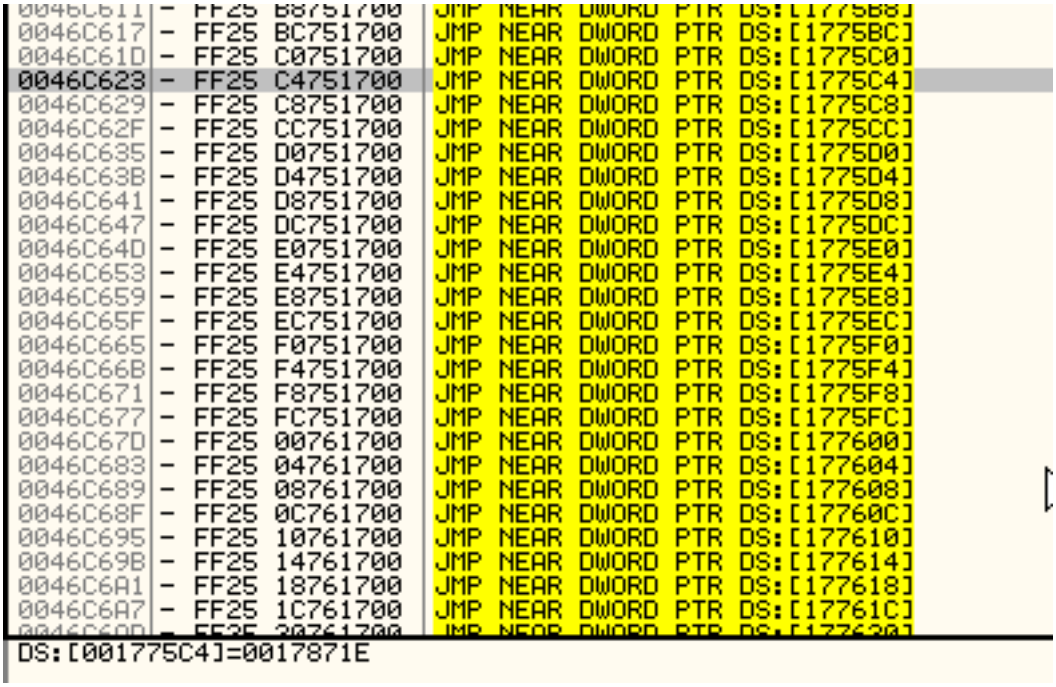
Хе-хе, таблица косвенных переходов, которая приводит к появлению ошибки, так как секции памяти, в которой находится 177658 и следующие адреса, нет в дампе и создаётся упаковщиком во время распаковки оригинального кода, так что нам осталось решить эту проблему, которая является одним из многих противоотладочных методов.

Запускаем оригинальный файл в другом OllyDbg и доходим до OEP с помощью скрипта, стирая старые HBP и устанавливая два необходимых BP.

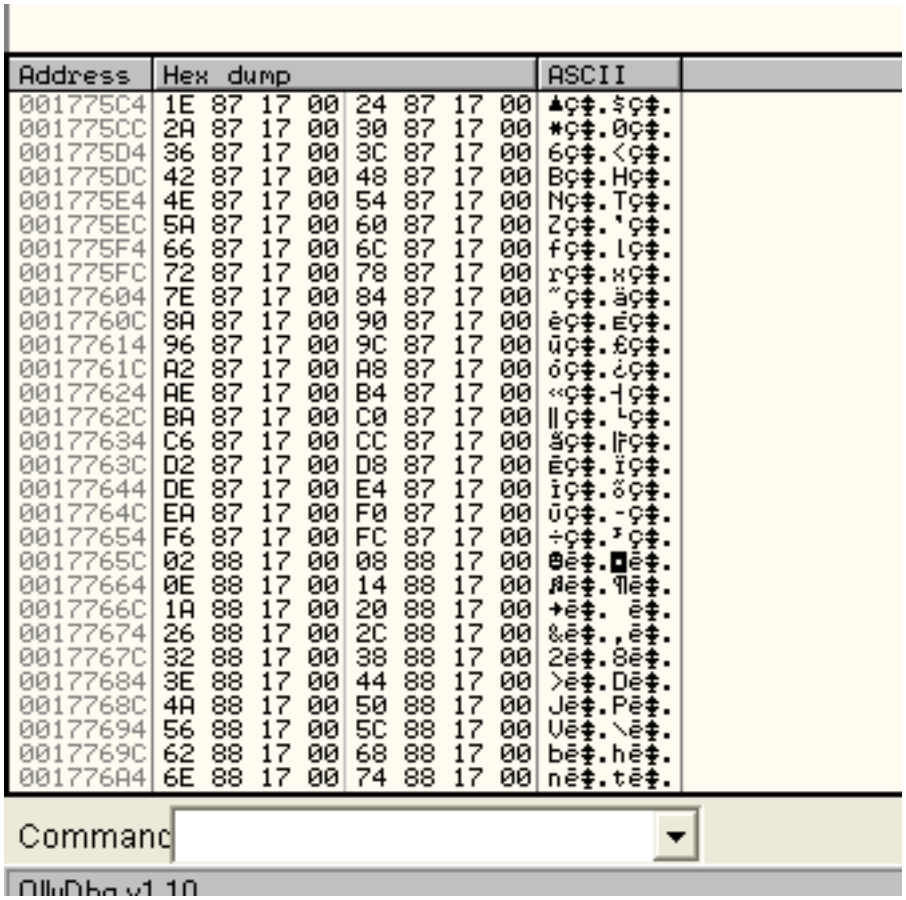


Alli llegamos vamos a mirar la zona de esos saltos indirectos aquí

Доходим до сюда, смотрим область этих косвенных переходов.



Если пойдём в DUMP, то там табличка, откуда берутся значения для перехода.



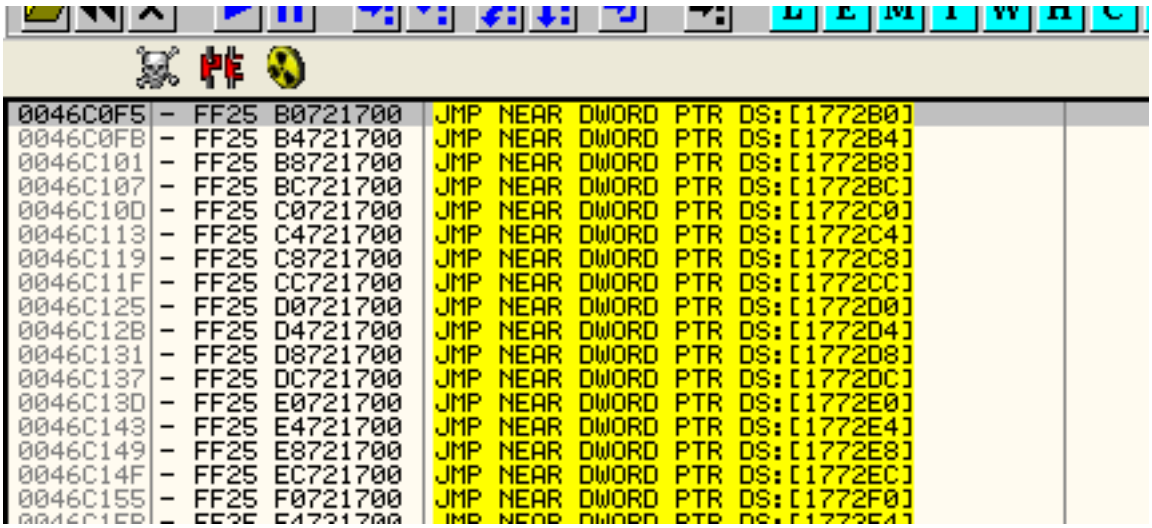
Address	Hex dump	ASCII
001775C4	1E 87 17 00 24 87 17 00	▲ç‡.$ç‡.
001775CC	2A 87 17 00 30 87 17 00	*ç‡.0ç‡.
001775D4	36 87 17 00 3C 87 17 00	6ç‡.<ç‡.
001775DC	42 87 17 00 48 87 17 00	Bç‡.Hç‡.
001775E4	4E 87 17 00 54 87 17 00	Nç‡.Tç‡.
001775EC	5A 87 17 00 60 87 17 00	Zç‡.`ç‡.
001775F4	66 87 17 00 6C 87 17 00	fç‡.lç‡.
001775FC	72 87 17 00 78 87 17 00	rç‡.xç‡.
00177604	7E 87 17 00 84 87 17 00	~ç‡.äç‡.
0017760C	8A 87 17 00 90 87 17 00	èç‡.Éç‡.
00177614	96 87 17 00 9C 87 17 00	ûç‡.£ç‡.
0017761C	A2 87 17 00 A8 87 17 00	óç‡.¿ç‡.
00177624	AE 87 17 00 B4 87 17 00	«ç‡.┤ç‡.
0017762C	BA 87 17 00 C0 87 17 00	‖ç‡.└ç‡.
00177634	C6 87 17 00 CC 87 17 00	ãç‡.╠ç‡.
0017763C	D2 87 17 00 D8 87 17 00	Ēç‡.╪ç‡.
00177644	DE 87 17 00 E4 87 17 00	▌ç‡.õç‡.
0017764C	EA 87 17 00 F0 87 17 00	ūç‡.-ç‡.
00177654	F6 87 17 00 FC 87 17 00	÷ç‡.ⁿç‡.
0017765C	02 88 17 00 08 88 17 00	☻ê‡.◘ê‡.
00177664	0E 88 17 00 14 88 17 00	♫ê‡.¶ê‡.
0017766C	1A 88 17 00 20 88 17 00	→ê‡. ê‡.
00177674	26 88 17 00 2C 88 17 00	&ê‡.,ê‡.
0017767C	32 88 17 00 38 88 17 00	2ê‡.8ê‡.
00177684	3E 88 17 00 44 88 17 00	>ê‡.Dê‡.
0017768C	4A 88 17 00 50 88 17 00	Jê‡.Pê‡.
00177694	56 88 17 00 5C 88 17 00	Vê‡.\ê‡.
0017769C	62 88 17 00 68 88 17 00	bê‡.hê‡.
001776A4	6E 88 17 00 74 88 17 00	nê‡.tê‡.

Command

OllyDbg v1.10

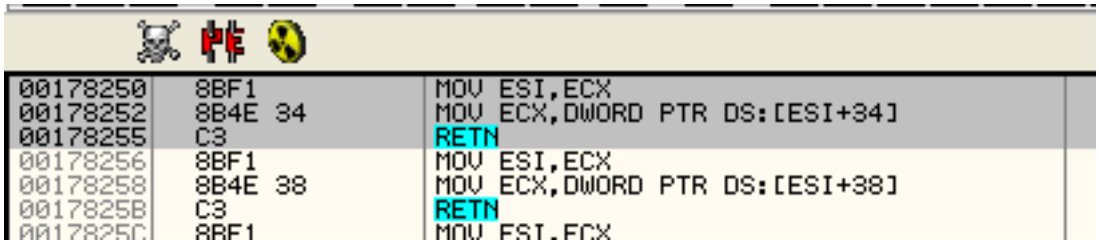
Ок, есть много способов решить эту проблему. Классический – это сделать инъекцию в то же место, где создаётся секция и скопировать все байты в вышеуказанную секцию, откуда всё сохраняется, а затем перейти на OEP, починив всё. Смотрим, сможем ли найти альтернативный метод, который были бы гораздо проще, так как описанный метод мы будем использовать часто в других упаковщиках.

Ищем первый переход в 46c0f5.

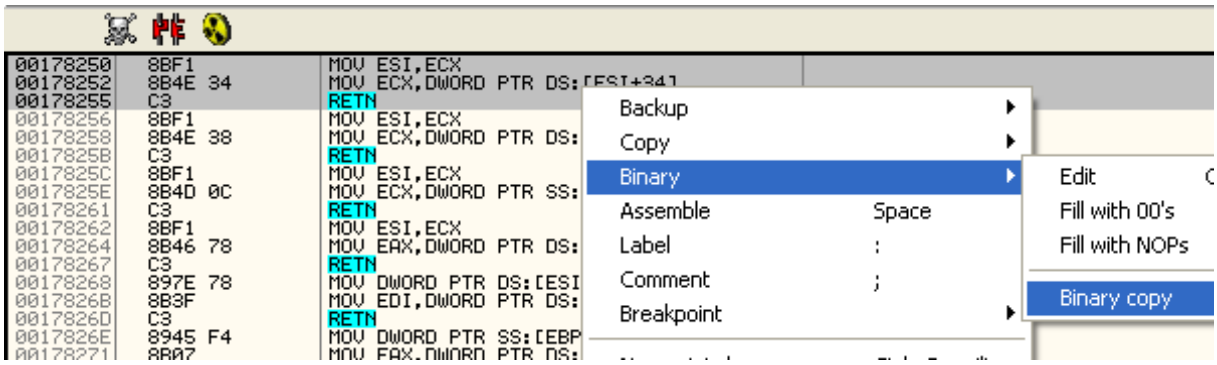


```
0046C0F5 - FF25 B0721700  JMP NEAR DWORD PTR DS:[1772B0]
0046C0FB - FF25 B4721700  JMP NEAR DWORD PTR DS:[1772B4]
0046C101 - FF25 B8721700  JMP NEAR DWORD PTR DS:[1772B8]
0046C107 - FF25 BC721700  JMP NEAR DWORD PTR DS:[1772BC]
0046C10D - FF25 C0721700  JMP NEAR DWORD PTR DS:[1772C0]
0046C113 - FF25 C4721700  JMP NEAR DWORD PTR DS:[1772C4]
0046C119 - FF25 C8721700  JMP NEAR DWORD PTR DS:[1772C8]
0046C11F - FF25 CC721700  JMP NEAR DWORD PTR DS:[1772CC]
0046C125 - FF25 D0721700  JMP NEAR DWORD PTR DS:[1772D0]
0046C12B - FF25 D4721700  JMP NEAR DWORD PTR DS:[1772D4]
0046C131 - FF25 D8721700  JMP NEAR DWORD PTR DS:[1772D8]
0046C137 - FF25 DC721700  JMP NEAR DWORD PTR DS:[1772DC]
0046C13D - FF25 E0721700  JMP NEAR DWORD PTR DS:[1772E0]
0046C143 - FF25 E4721700  JMP NEAR DWORD PTR DS:[1772E4]
0046C149 - FF25 E8721700  JMP NEAR DWORD PTR DS:[1772E8]
0046C14F - FF25 EC721700  JMP NEAR DWORD PTR DS:[1772EC]
0046C155 - FF25 F0721700  JMP NEAR DWORD PTR DS:[1772F0]
```

Он занимает 6 байт – инструкция JMP, ведущая в 178250, где также происходит выполнение 6 байт и возврат.

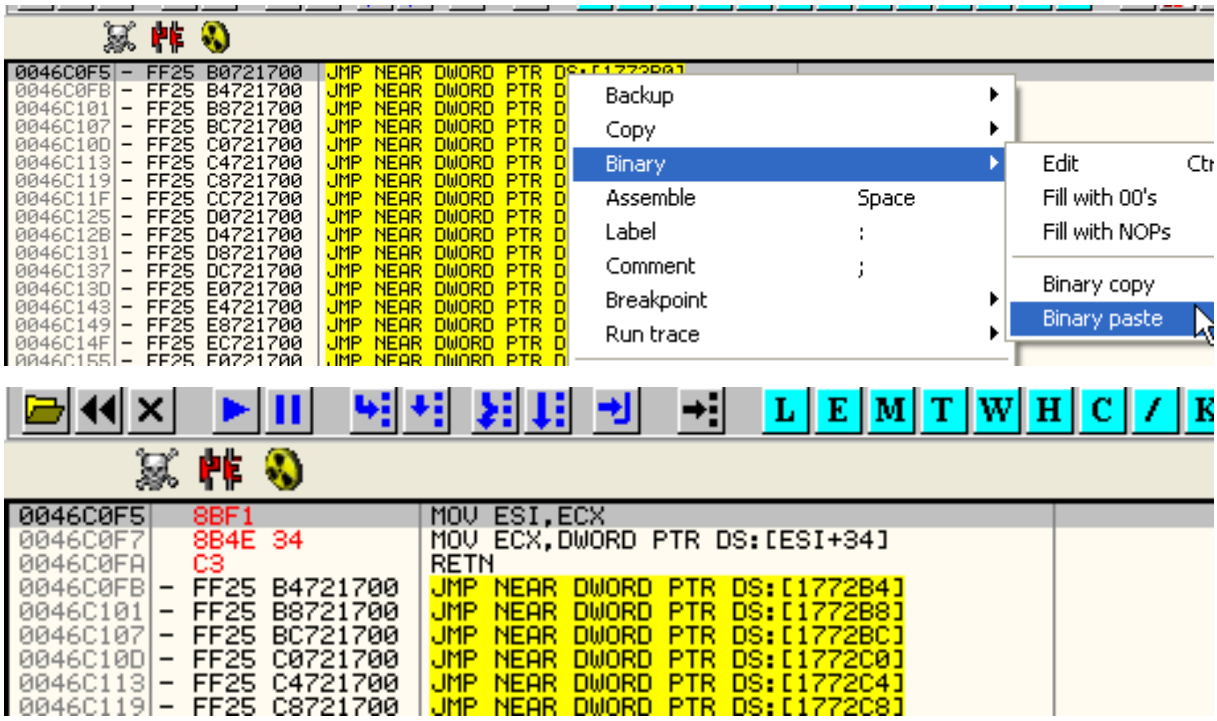


Идея заключается в том, чтобы заменить 6 байтов косвенного перехода на 6 байтов, которые действительно что-то делают.



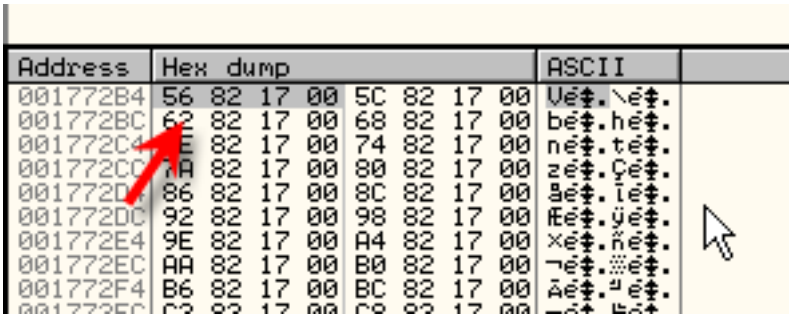
Отмечаем три строки и делаем BINARY COPY.

Возвращаемся к переходу и делаем BINARY PASTE.

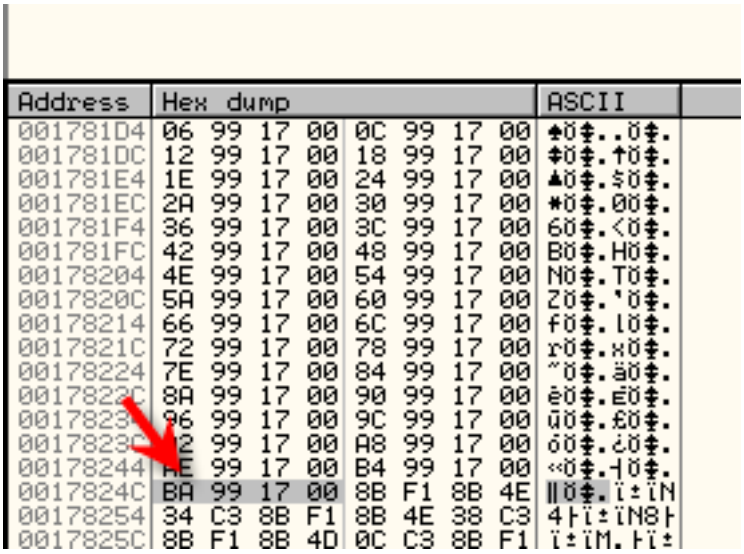


Должно работать. Эти косвенные переходы вызываются call'ами из программы, так что выполняется шесть байт, и при срабатывании RETN будет происходить возврат туда, откуда произошёл вызов.

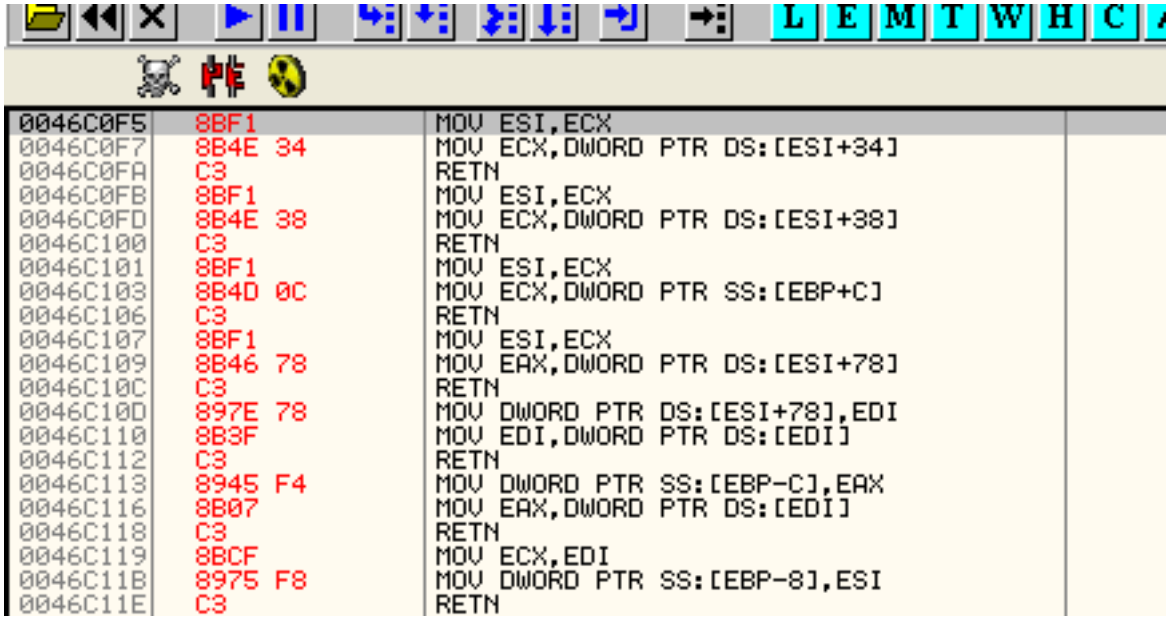
Поскольку переходы идут друг за другом и размер выполняемых команд всегда равен шести байтам, то мы можем скопировать сюда всю секцию, чтобы заменить переходы собственно тем, что должно выполняться.



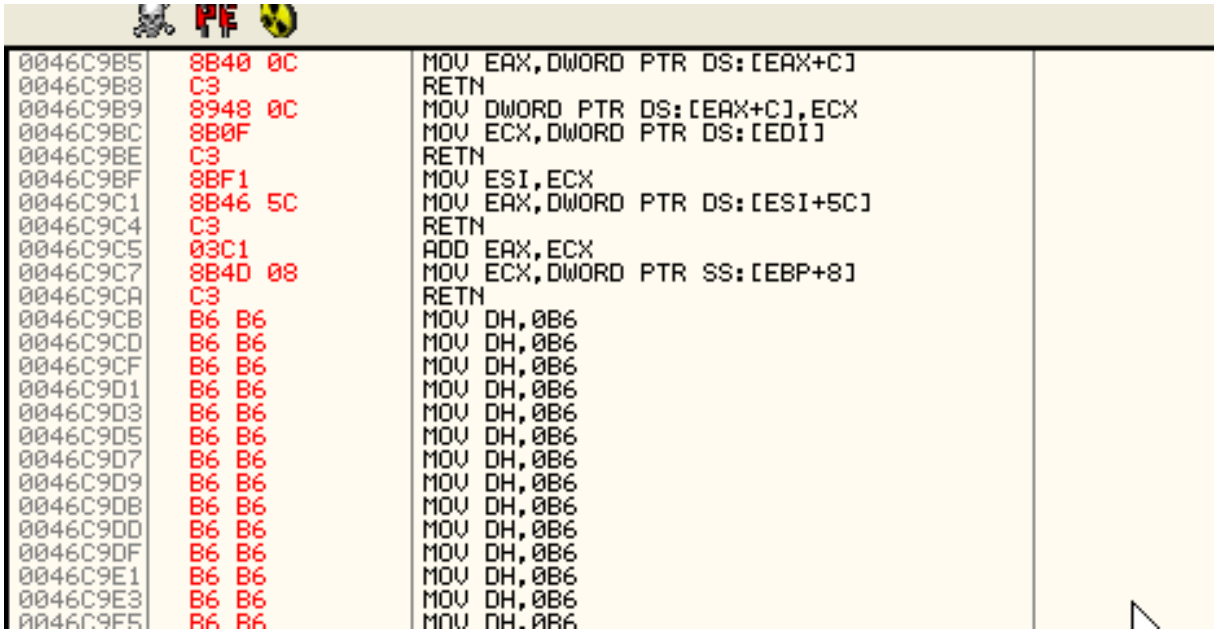
Байты начинают выполняться с 178256 – это первый элемент таблицы, а последний заканчивается в 1799ba.



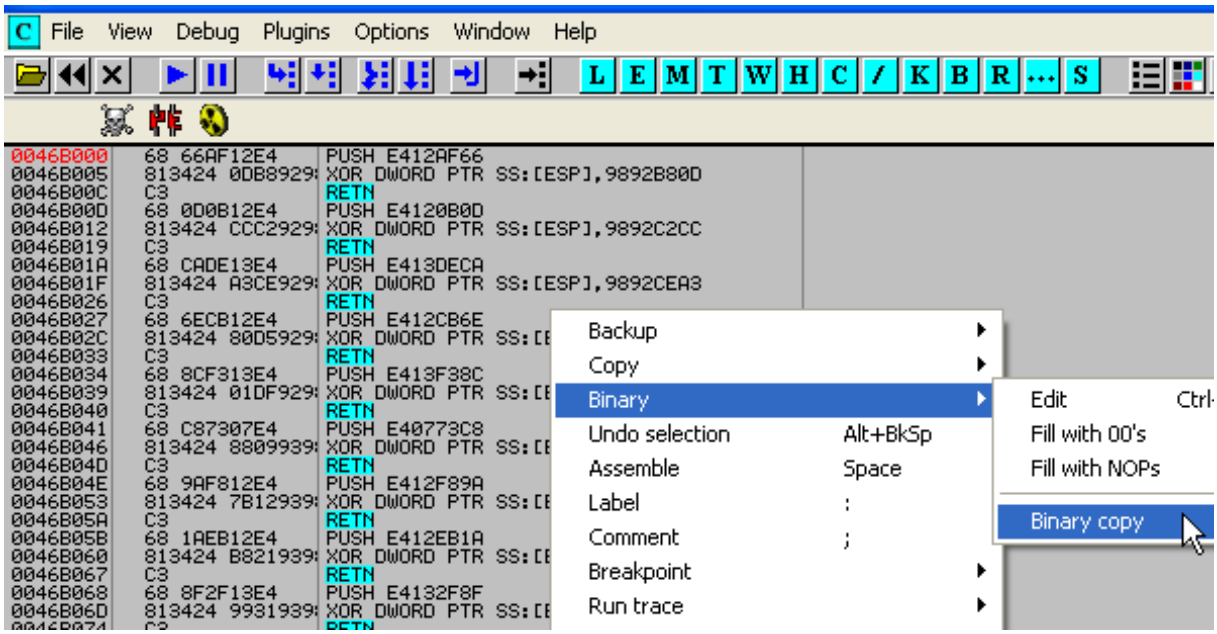
Так что копируем с помощью BINARY COPY всю область с нужными нам байтами и заменяем переходы с помощью BYNARY PASTE.



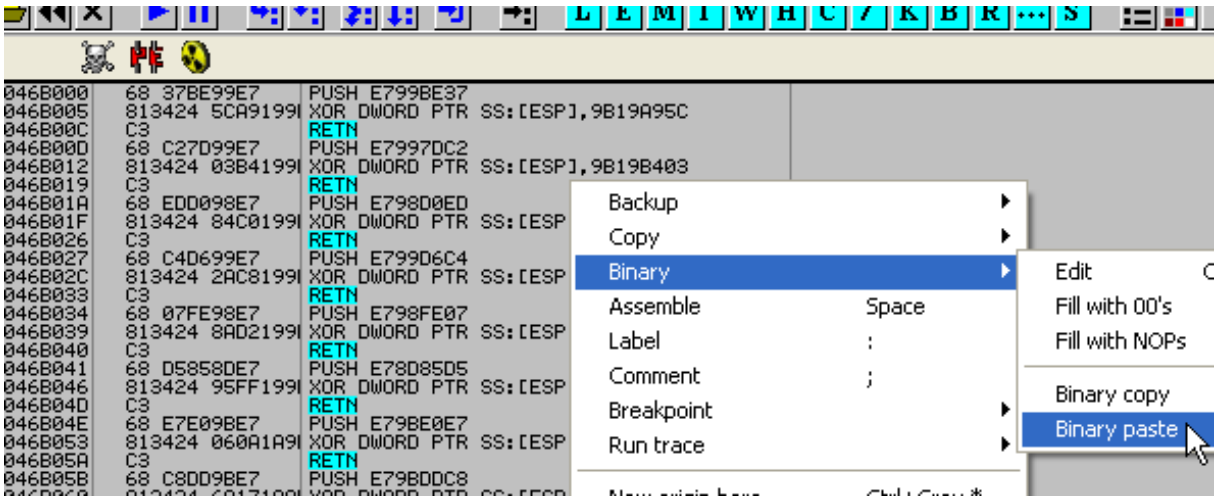
Видим, что получилось, также видим, что некоторые переходы в конце ведут к частям, у которых нет RET.



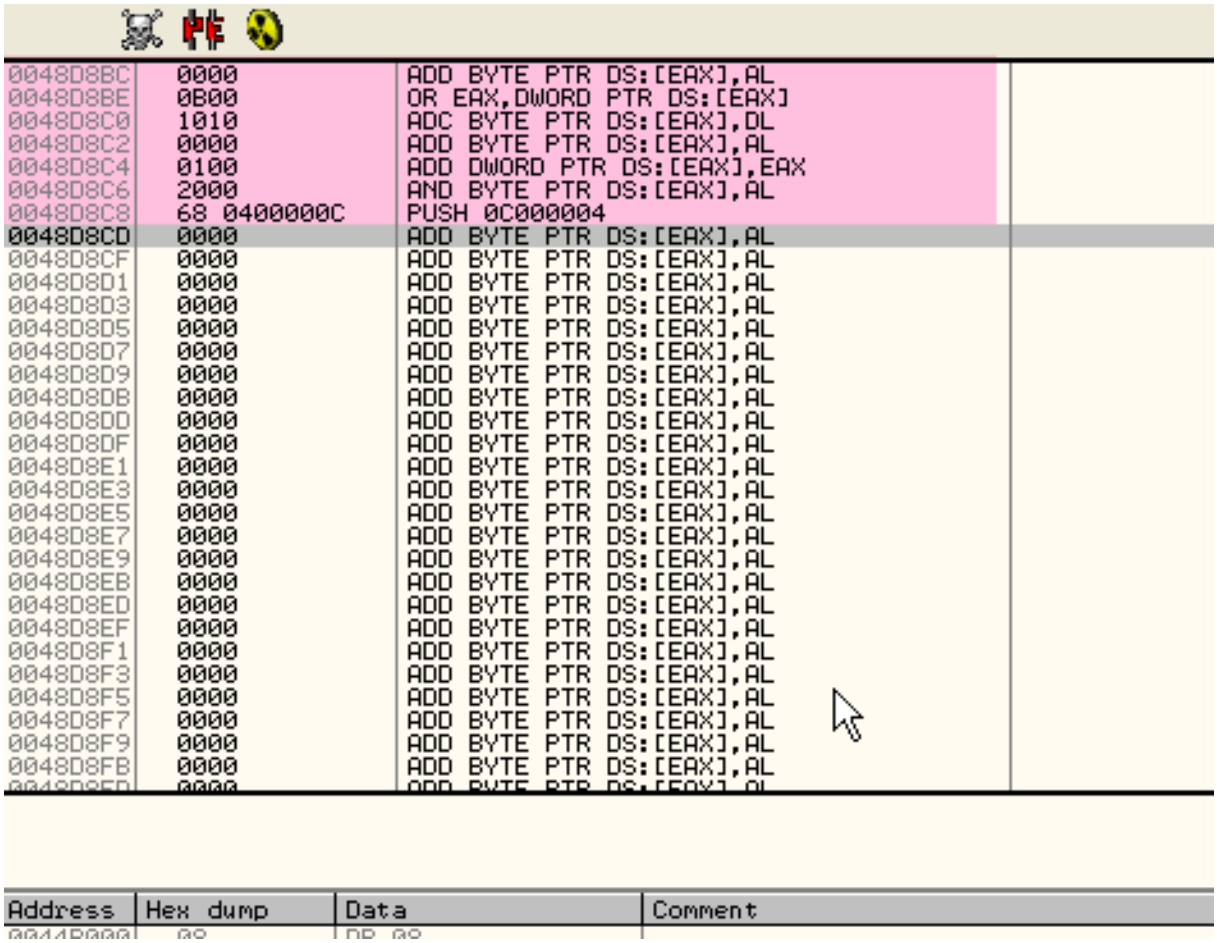
Я предположил, что эти переходы не используются, так что, по идее, проблем возникнуть не должно. Ок, починили секцию. Теперь может сделать BINARY COPY всей секции и скопировать её в файл дампа с помощью BYNARY PASTE.



Теперь сохраняем изменения в дамп с помощью COPY TO EXECUTABLE и так далее.



Чтобы сохранить их и нам не было сказано, что не можем этого сделать, делаем так: от конца секции поднимается до последней строки, где есть данные.



Отмечаем отсюда и до начала секции вверх и сохраняем изменения. Перезапускаем и пробуем.

Работает превосходно. Мы победили украденные байты, переадресовочную IAT, антидампы, и защиту против HBP.

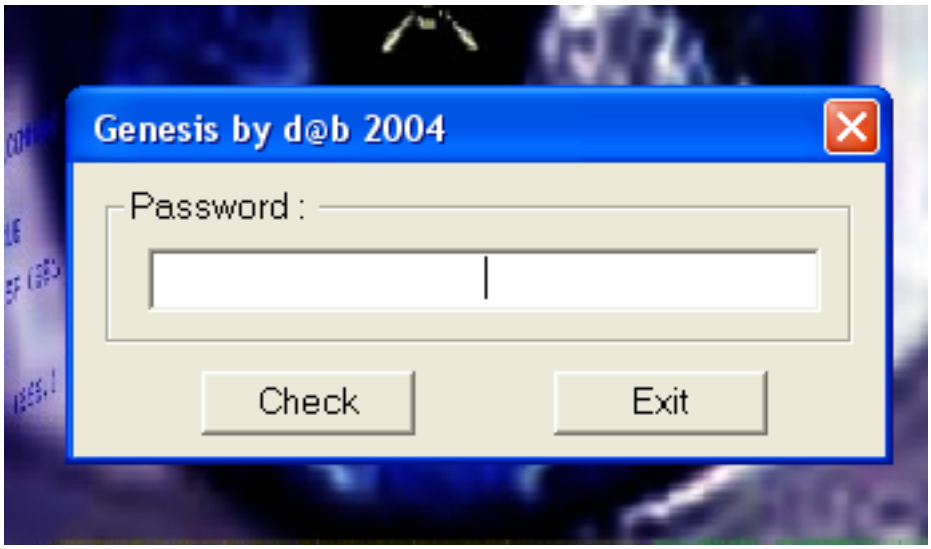
Введение в крэкинг с нуля, используя OllyDbg - Глава 45

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2009-08-07

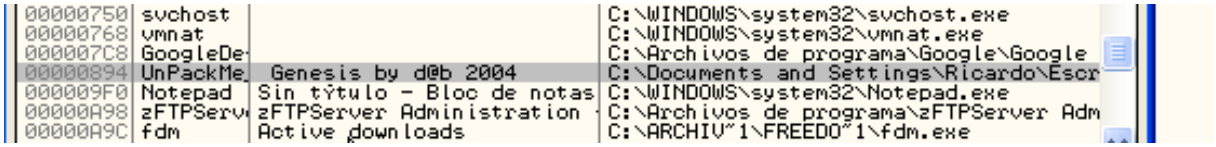
В этой главе мы рассмотрим распакуй-меня, сделанном с помощью упаковщика, который сложно победить в OllyDbg, так как в нём есть одна часть, которая не работает в Олли и выдаёт ошибку. Но мы всё равно будем использовать OllyDbg, несмотря ни на что. Мы могли бы использовать другой отладчик, но ведь это же «Введение в крэкинг с помощью OllyDbg», хе-хе.

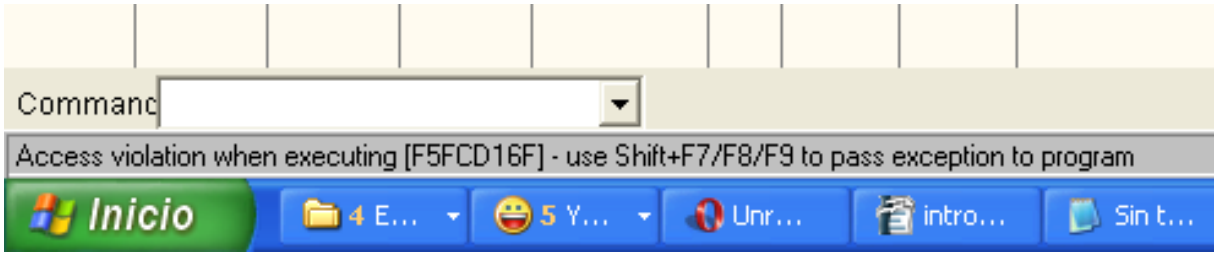
Так что даже с распухшими головами мы будем использовать OllyDbg вопреки всему. Распакуй-меня идёт вместе со статьёй, и он кажется реально противоОллиным. Запустим его без отладчика.



Вот в чём дело, когда он запущен, то пожирает 99% ресурсов моей машины. Давайте узнаем, делает ли он это для защиты или ещё почему.

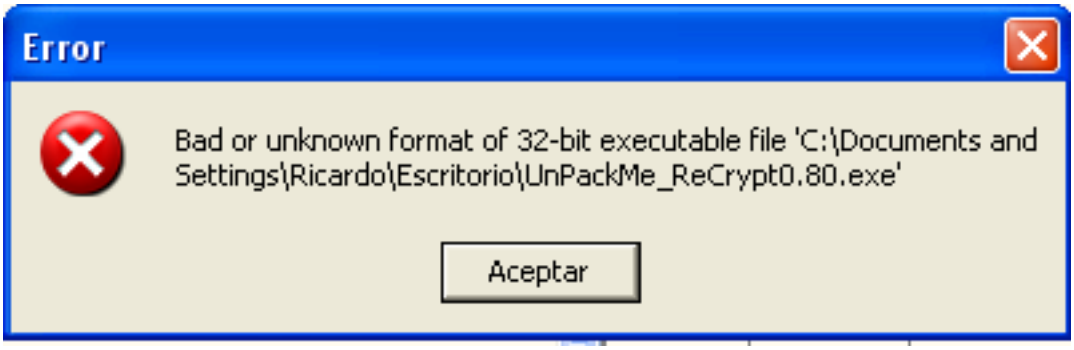
Если хотим приаттачиться к нему, то нужно пойти в FILE-ATTACH и поискать процесс.



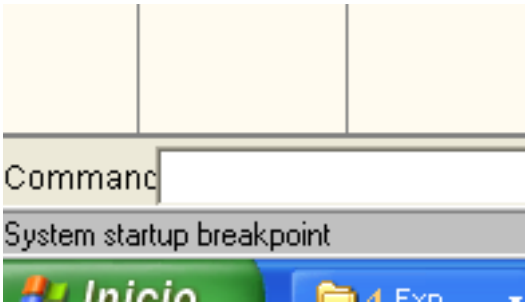


И не забудьте приаттачиться.

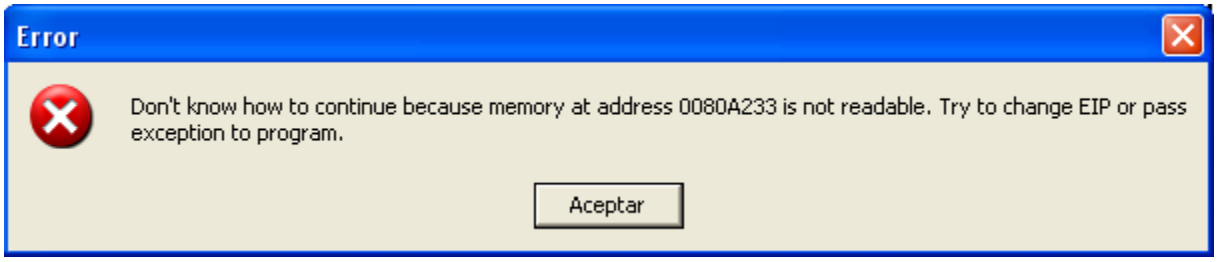
Если откроем в Ollydbg, пропатченной для поиска OEP, которую использовали в прошлых главах:



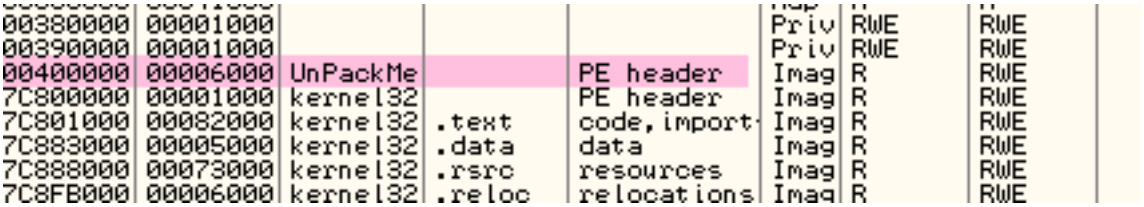
Если нажмём кнопку подтверждения:



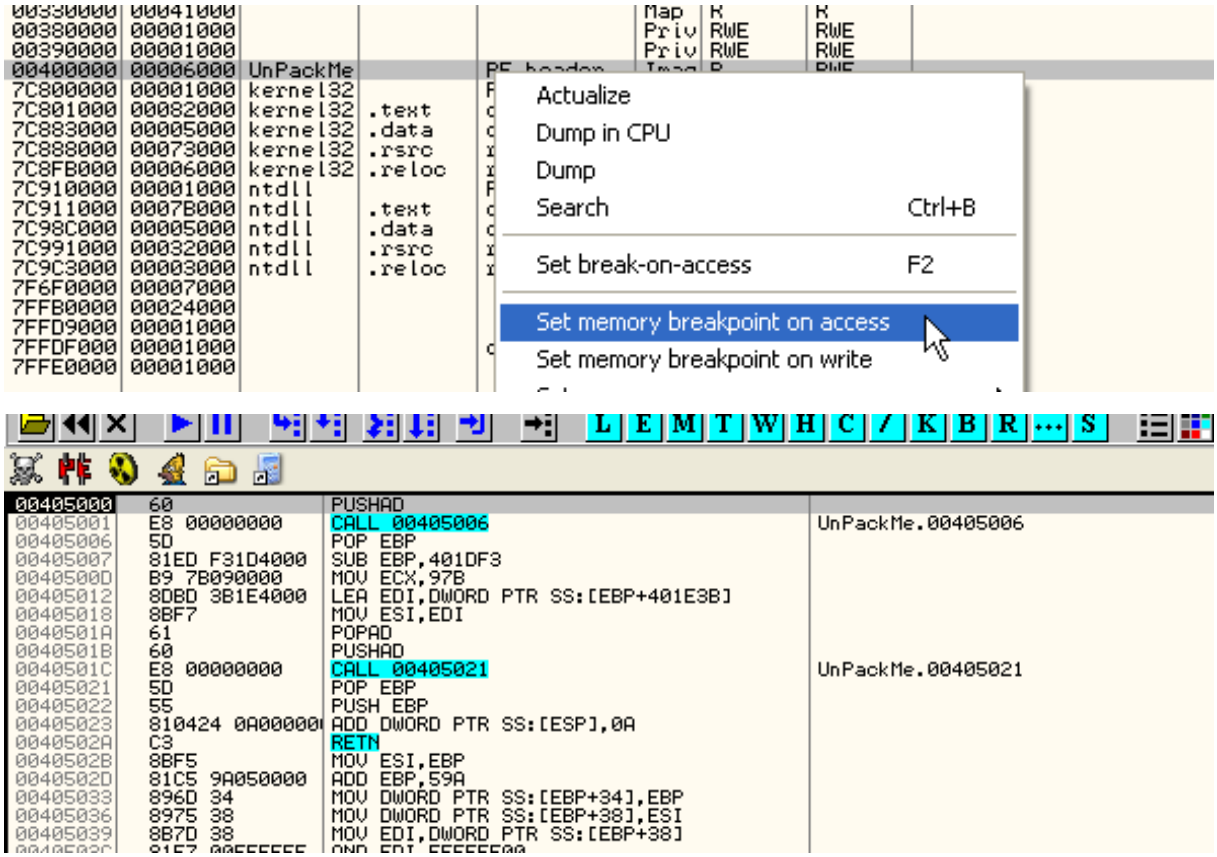
И если сделаем RUN:



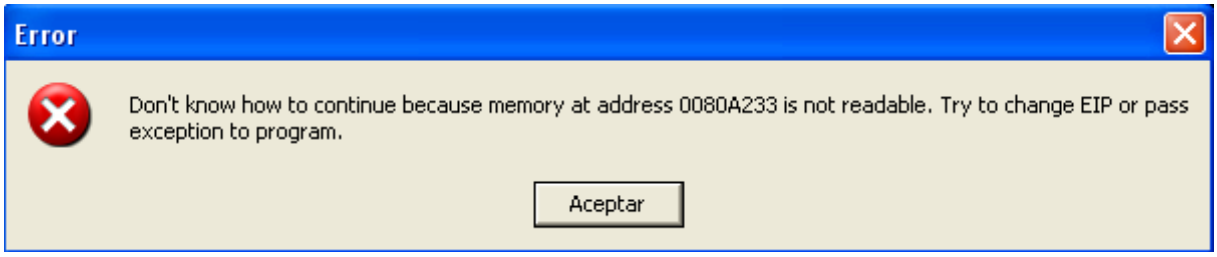
Перезапускаем и снова идём к SYSTEM STARTUP BREAKPOINT, а когда останавливаемся, то идём в окно «M».



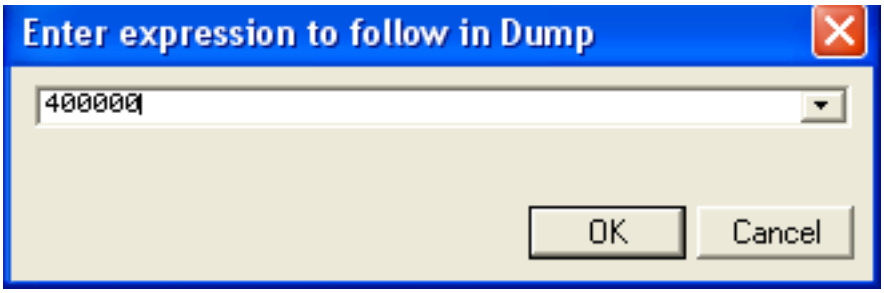
Видим, что тут есть одна секция, которая является результатом известной ошибки, когда меняется RVA и размеры в заголовке файла. Пока что установим BPM ON ACCESS на указанной секции. Помним, что эта OllyDbg - особенная, и останавливаемся только на выполнении кода, но не тогда, когда он читается или записывается.

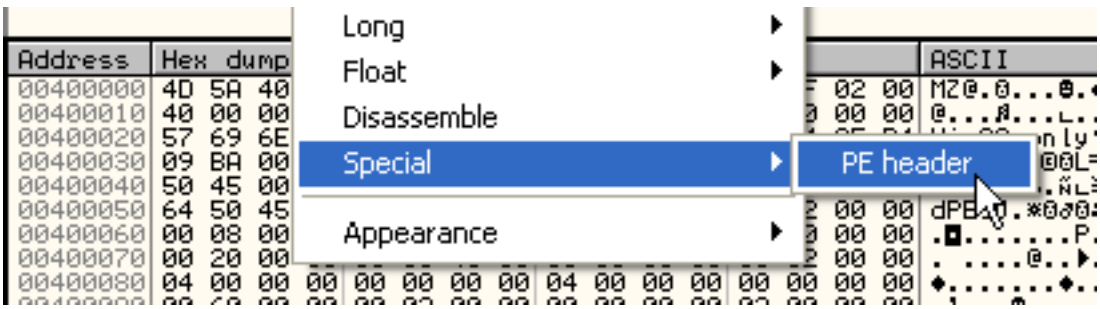


Теперь снимаем BPM и делаем RUN.

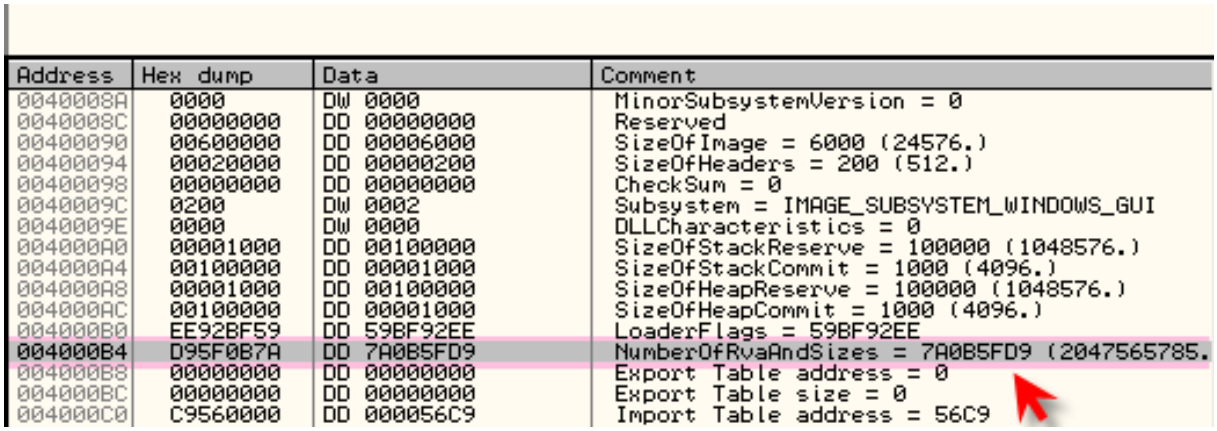


Ок, смотрим в заголовке значения RVA и размеров.



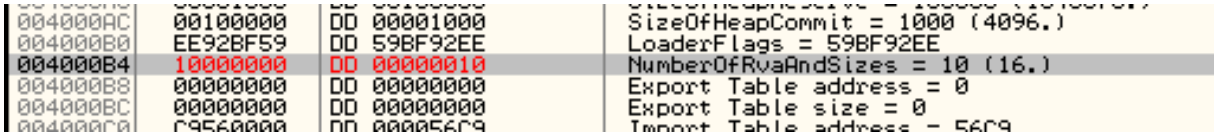


Теперь идём вниз, пока не найдём значение.



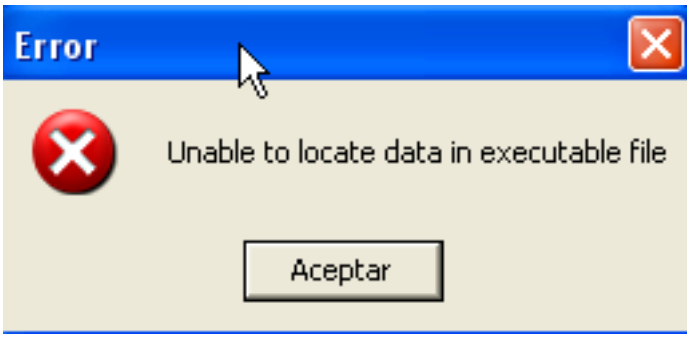
Address	Hex dump	Data	Comment
0040008A	0000	DW 0000	MinorSubsystemVersion = 0
0040008C	00000000	DD 00000000	Reserved
00400090	00600000	DD 00006000	SizeOfImage = 6000 (24576.)
00400094	00020000	DD 00000200	SizeOfHeaders = 200 (512.)
00400098	00000000	DD 00000000	CheckSum = 0
0040009C	0200	DW 0002	Subsystem = IMAGE_SUBSYSTEM_WINDOWS_GUI
0040009E	0000	DW 0000	DLLCharacteristics = 0
004000A0	00001000	DD 00100000	SizeOfStackReserve = 100000 (1048576.)
004000A4	00100000	DD 00001000	SizeOfStackCommit = 1000 (4096.)
004000A8	00001000	DD 00100000	SizeOfHeapReserve = 100000 (1048576.)
004000AC	00100000	DD 00001000	SizeOfHeapCommit = 1000 (4096.)
004000B0	EE92BF59	DD 59BF92EE	LoaderFlags = 59BF92EE
004000B4	D95F0B7A	DD 7A0B5FD9	NumberOfRvaAndSizes = 7A0B5FD9 (2047565785.
004000B8	00000000	DD 00000000	Export Table address = 0
004000BC	00000000	DD 00000000	Export Table size = 0
004000C0	C9560000	DD 000056C9	Import Table address = 56C9

Вот это значение, которое должно быть равно 10 в шестнадцатеричной системе. Посмотрим, что будет, если мы его изменим.

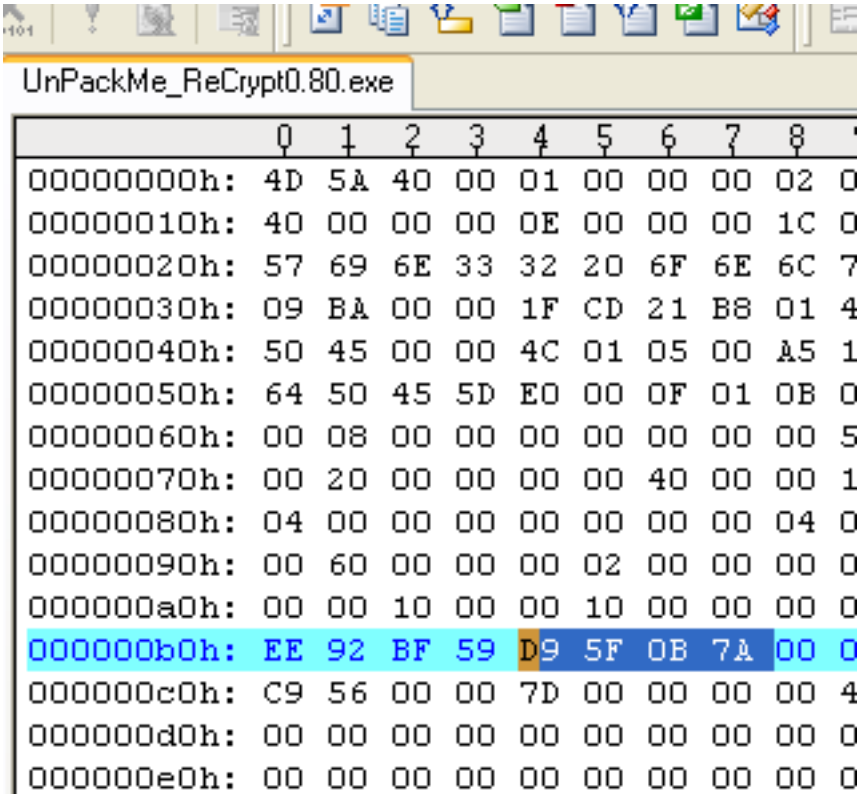


004000AC	00100000	DD 00001000	SizeOfHeapCommit = 1000 (4096.)
004000B0	EE92BF59	DD 59BF92EE	LoaderFlags = 59BF92EE
004000B4	10000000	DD 00000010	NumberOfRvaAndSizes = 10 (16.)
004000B8	00000000	DD 00000000	Export Table address = 0
004000BC	00000000	DD 00000000	Export Table size = 0

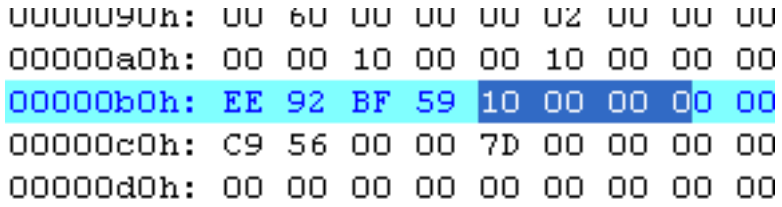
Пытаемся сохранить изменения.



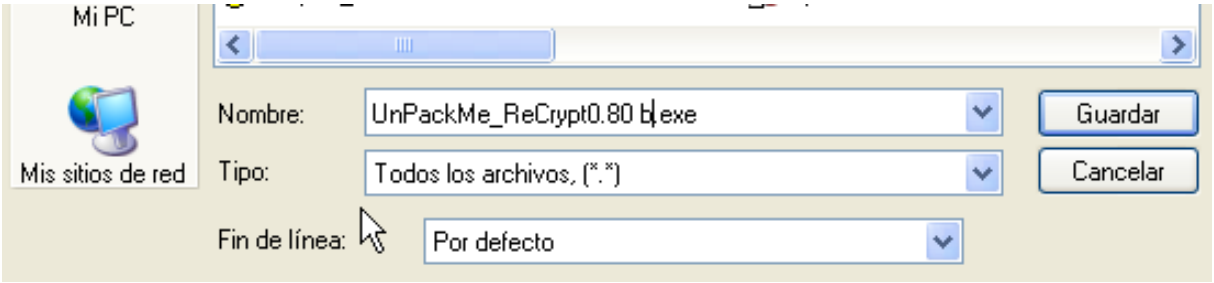
Грр... Попробуем изменить в шестнадцатеричном редакторе.



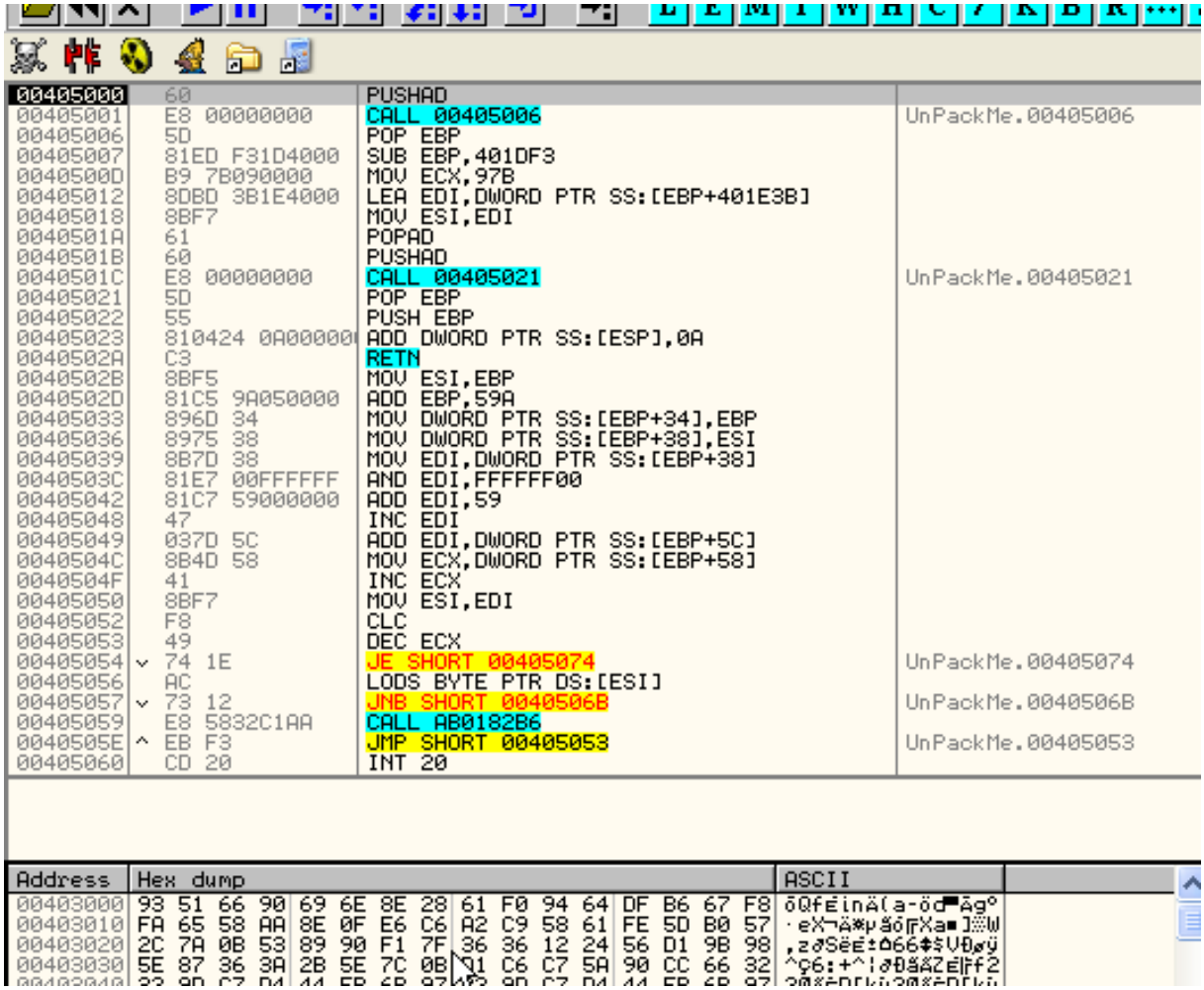
Меняем на 10.



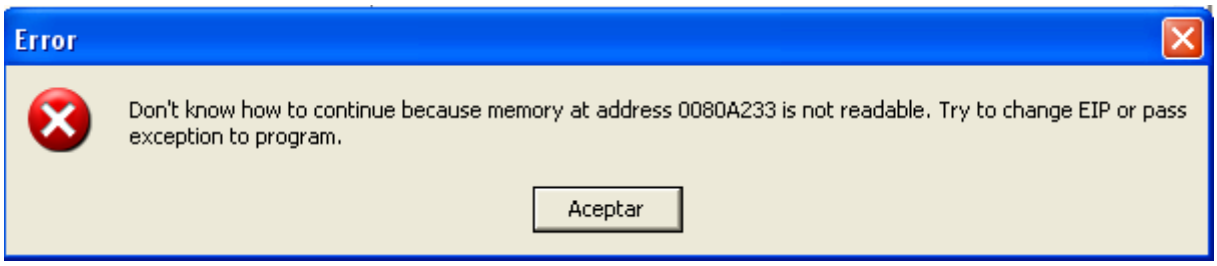
Теперь сохраняем изменения.



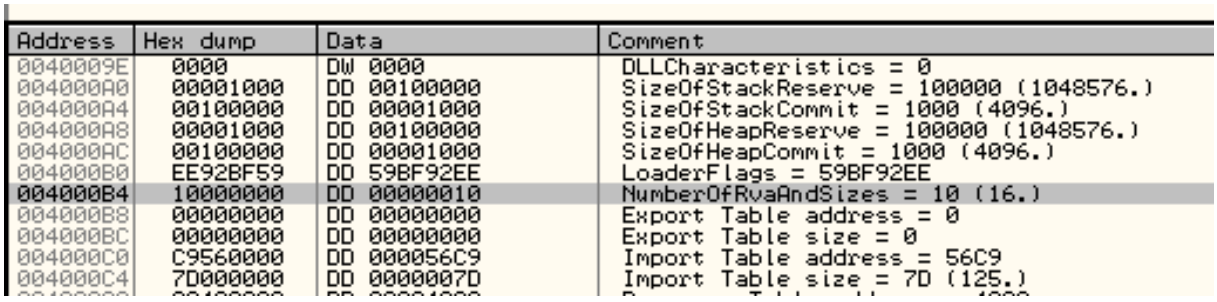
Сохраняем под другим именем.



Видим, что при открытии в Ollydbg происходит остановка на точке входа (как и должно быть). Починка файла с помощью данного приёма была правильной. Теперь делаем RUN.



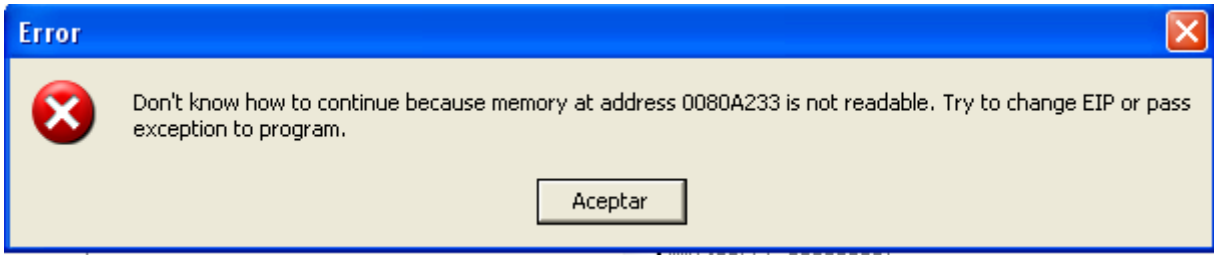
ГРРРР... Снова ошибка. Теперь, когда находимся на точке входа, изменим в памяти значения RVA и размеров на те, что были сначала. Это необходимо для правильной распаковки.



Так как мы уже находимся где нужно, установим нужное значение и проверим, сработает ли это.

Address	Hex dump	Data	Comment
0040009E	0000	DW 0000	DLLCharacteristics = 0
004000A0	00001000	DD 00100000	SizeOfStackReserve = 100000 (1048576.)
004000A4	00100000	DD 00001000	SizeOfStackCommit = 1000 (4096.)
004000A8	00001000	DD 00100000	SizeOfHeapReserve = 100000 (1048576.)
004000AC	00100000	DD 00001000	SizeOfHeapCommit = 1000 (4096.)
004000B0	EE92BF59	DD 59BF92EE	LoaderFlags = 59BF92EE
004000B4	D95F0B7A	DD 7A0B5FD9	NumberOfRvaAndSizes = 7A0B5FD9 (20475657
004000B8	00000000	DD 00000000	Export Table address = 0
004000BC	00000000	DD 00000000	Export Table size = 0

Значение установили, теперь делаем RUN.



Очевидно, что какая-то часть кода не пролезает через OllyDbg, пробуем оттрассировать, но не встречаем ничего странного, доходим до точки, где совершается переход на несуществующую секцию и до свидания. В OllyDbg не выполняется.

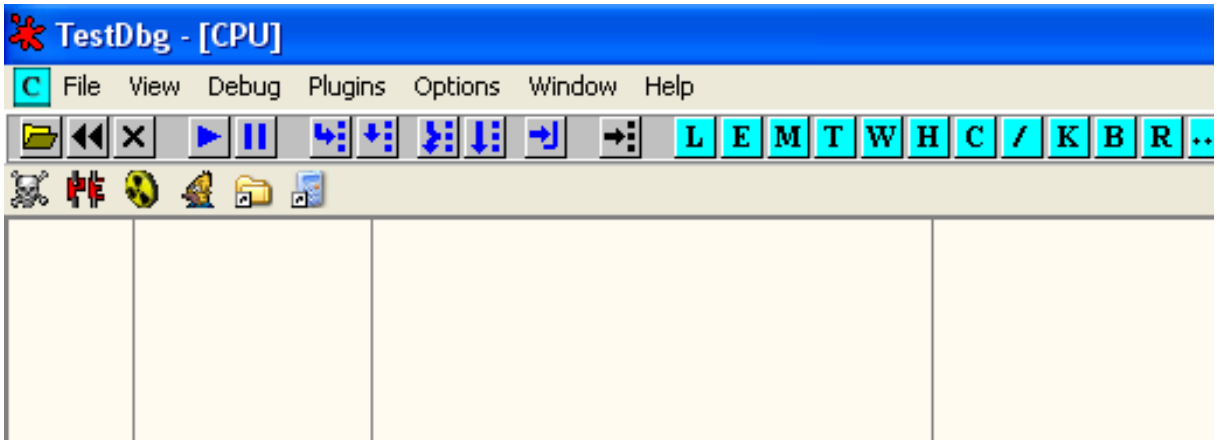
Мы хотим во что бы то ни стало показать методы, которые обычно используются в таких случаях. Здесь нам понадобится что необычное, для этого требуется вдохновение. Отойдём ненадолго, подумаем, что можно придумать такого неожиданного.

ДУМАЕМ

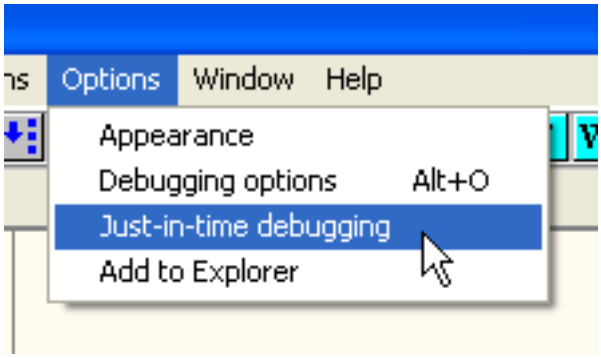
хе-хе

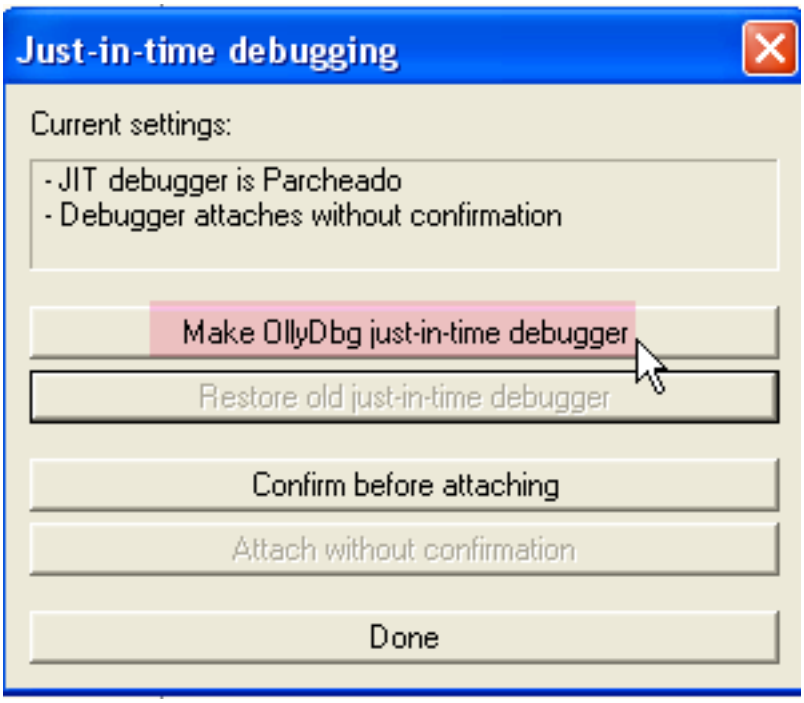
ДУМАЕМ ЕЩЁ

Хорошо, я покажу все возможности. Следующий способ мне не помог (но вам нужно быть знакомыми с этими методами, так как часто они работают).



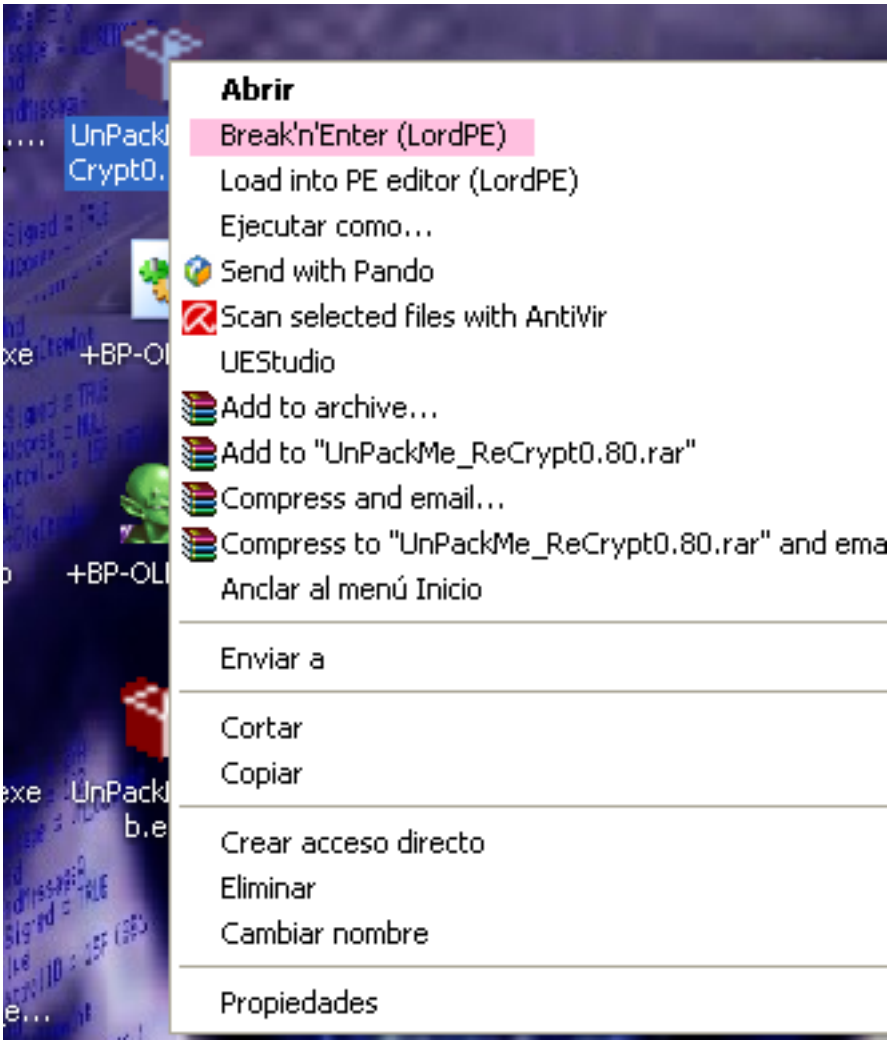
Открываем PARCHEADO 5 и устанавливаем его как JUST IN TIME DEBUGGER, то есть когда программа вызывает ошибку, отладчик автоматом подсоединяется к ней.



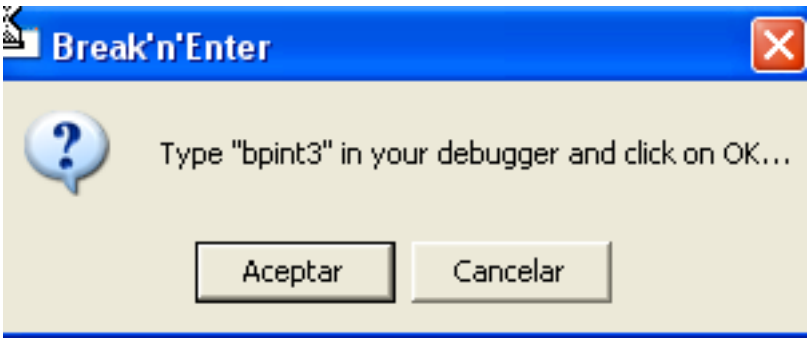


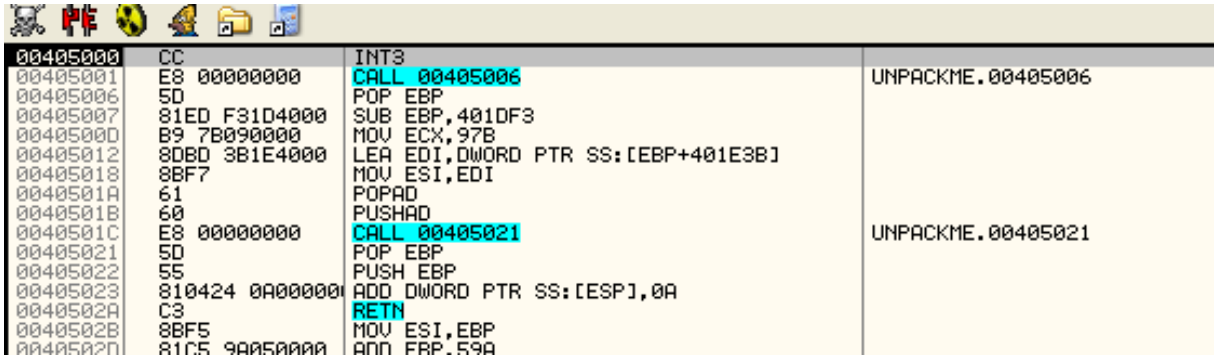
Нажимаем эту кнопку, чтобы перевести OllyDbg в режим JIT. Когда закончим работать с OllyDbg, следует восстановить старый JIT-отладчик с помощью кнопки «RESTORE OLD JUST IN TIME DEBUGGER», иначе OllyDbg будет запускаться при каждой программной ошибке, а это весьма утомительно.

Теперь закрываем PARCHEADO 5 и делаем щелчок правой кнопки на исходном файле распакуй-меня. Тот, где мы изменили RVA и другие размеры можно стереть, так как он нам не помог.

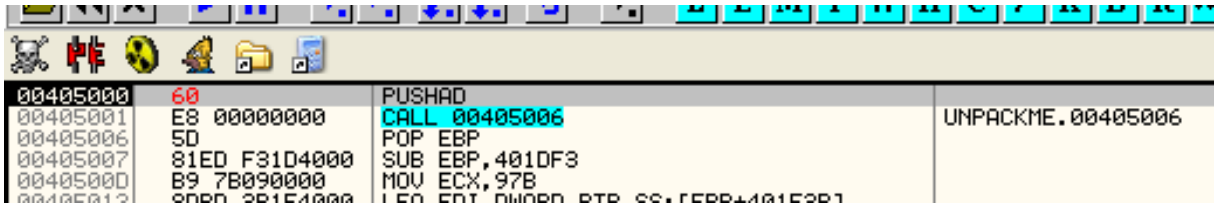


У тех из нас, кто иногда использует LORD PE DELUXE в системном меню правой кнопки мыши есть опция «BREAK AND ENTER», которая устанавливает INT3 на точку входа программы, что приводит к ошибке и запуска JIT-отладчика, которая присоединяется к вызвавшему ошибку процессу. Пробуем.

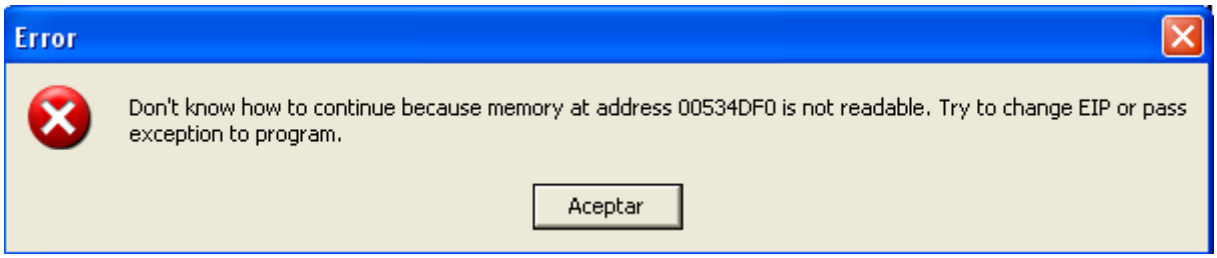




Запускается OllyDbg и присоединяется к процессу. Видим INT3, установленный LordPE. Заменяем INT3 на PUSHAD, который должен быть здесь.

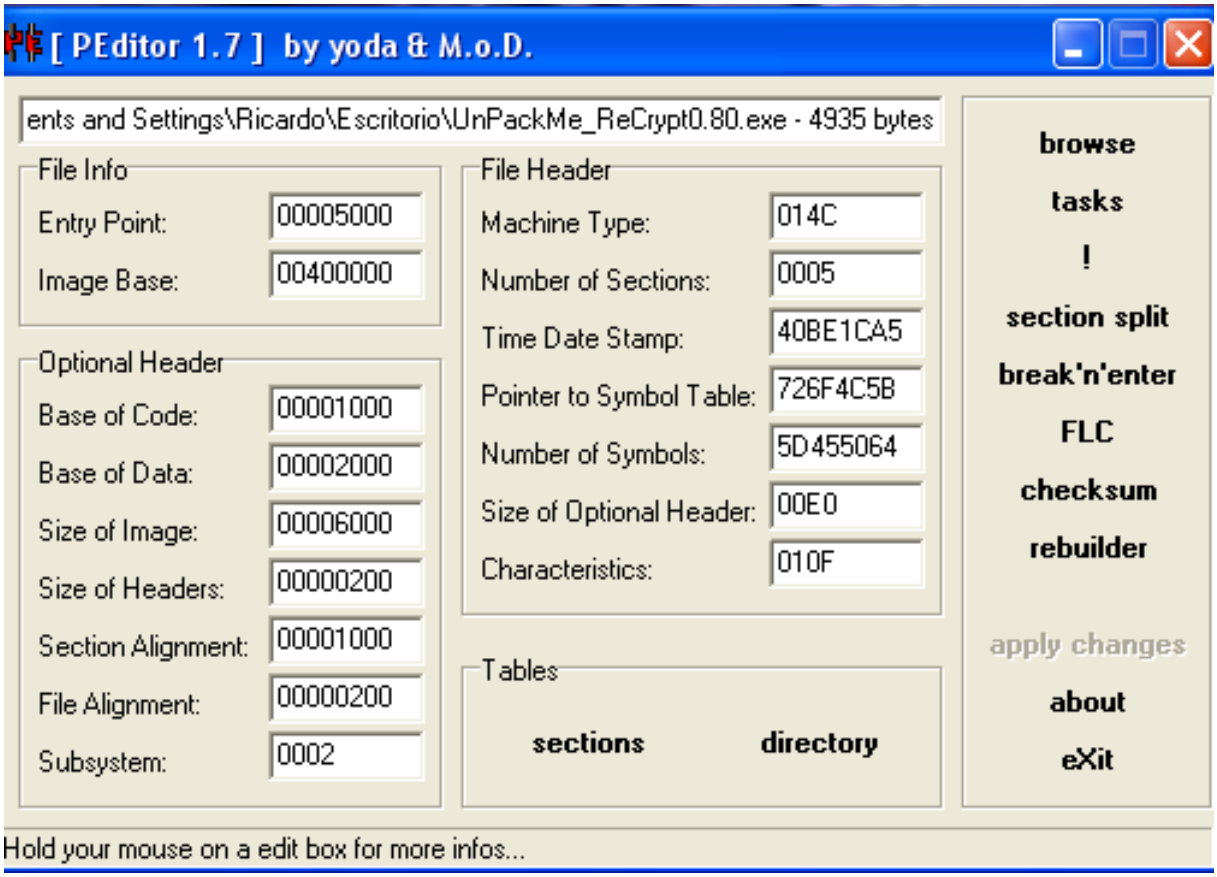


Делаем RUN.

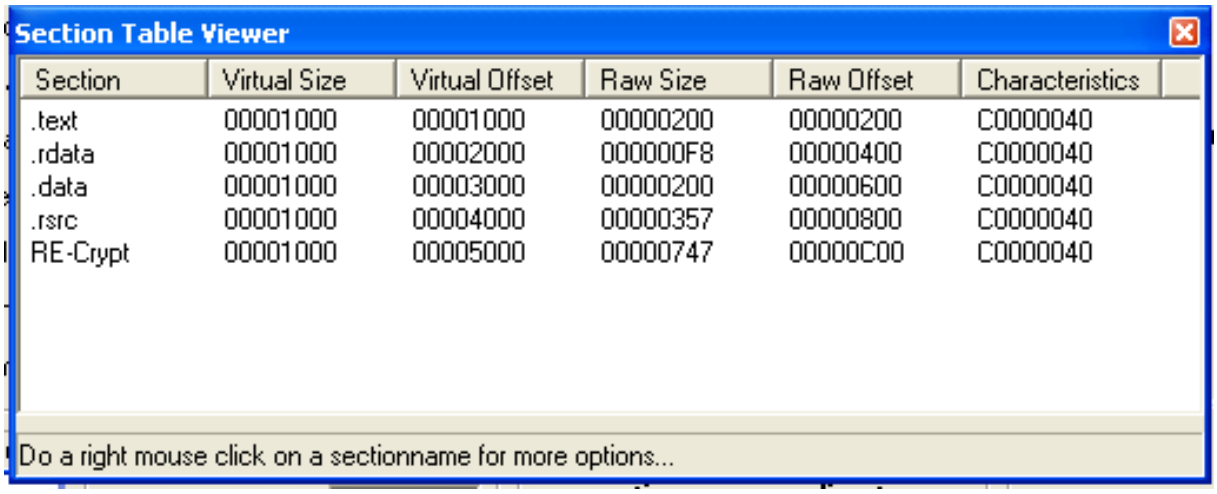


ГРРРРР, так как немного выполняется в OllyDbg, мы должны провести трассировку построчно и выяснить, что происходит, но по-правде, у меня нет никакого желания это делать, поэтому лучше ПОДУМАТЬ, ПОДУМАТЬ, ПОДУМАТЬ, хе-хе.

Всегда будем использовать исходный файл. Откроем его в PeEditor'е.

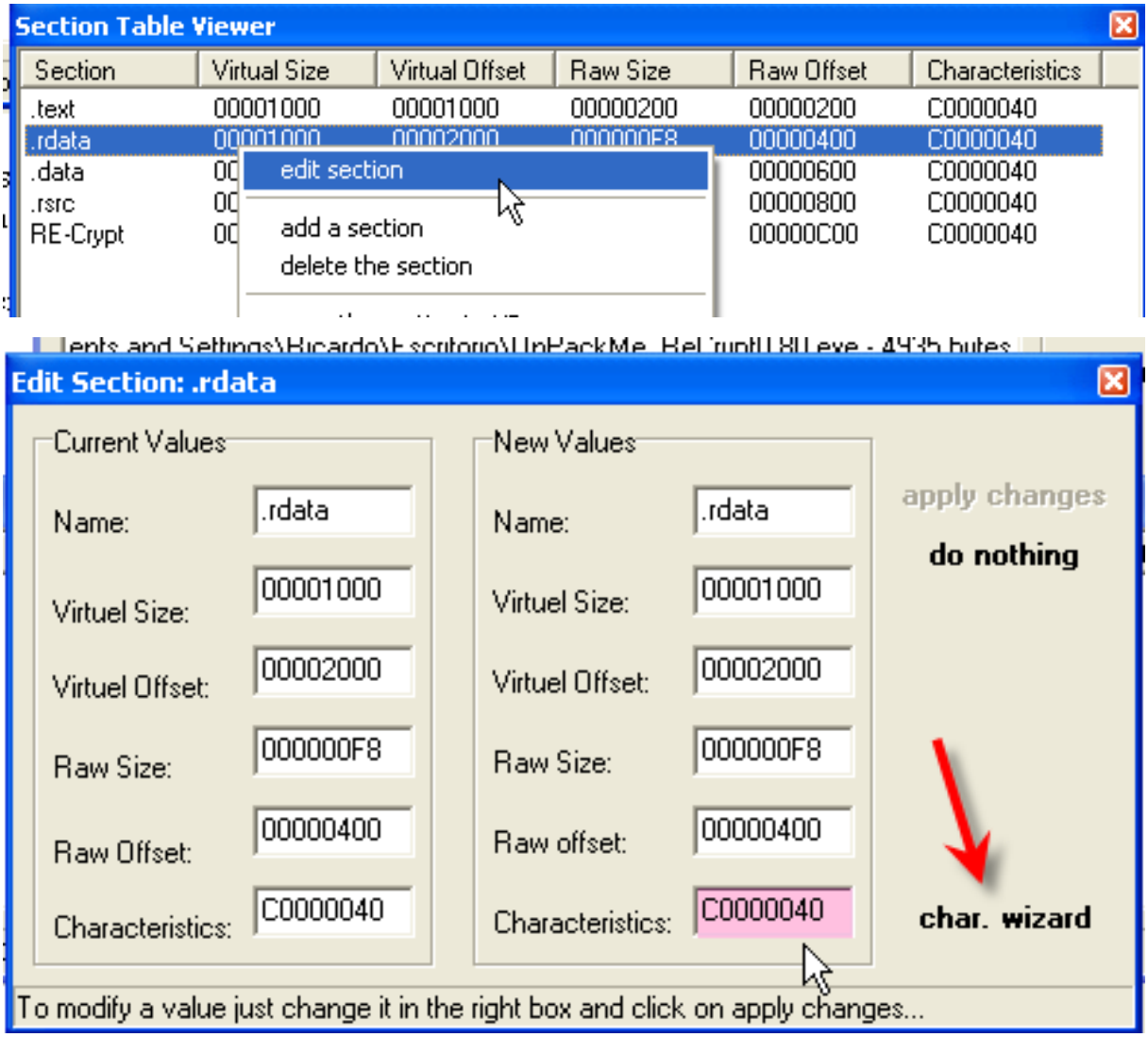


Теперь смотрим секции.

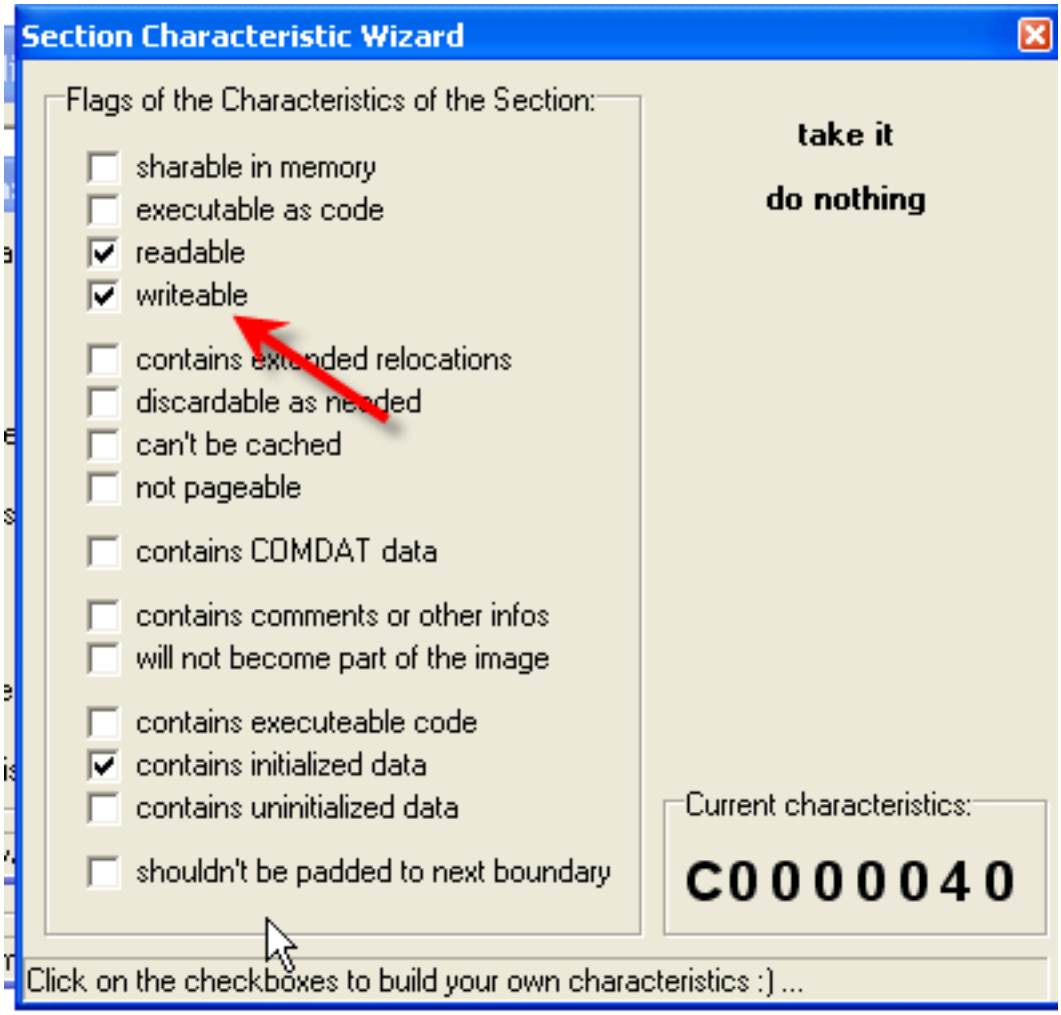


С помощью PeEditor'a можно прекрасно посмотреть все секции.

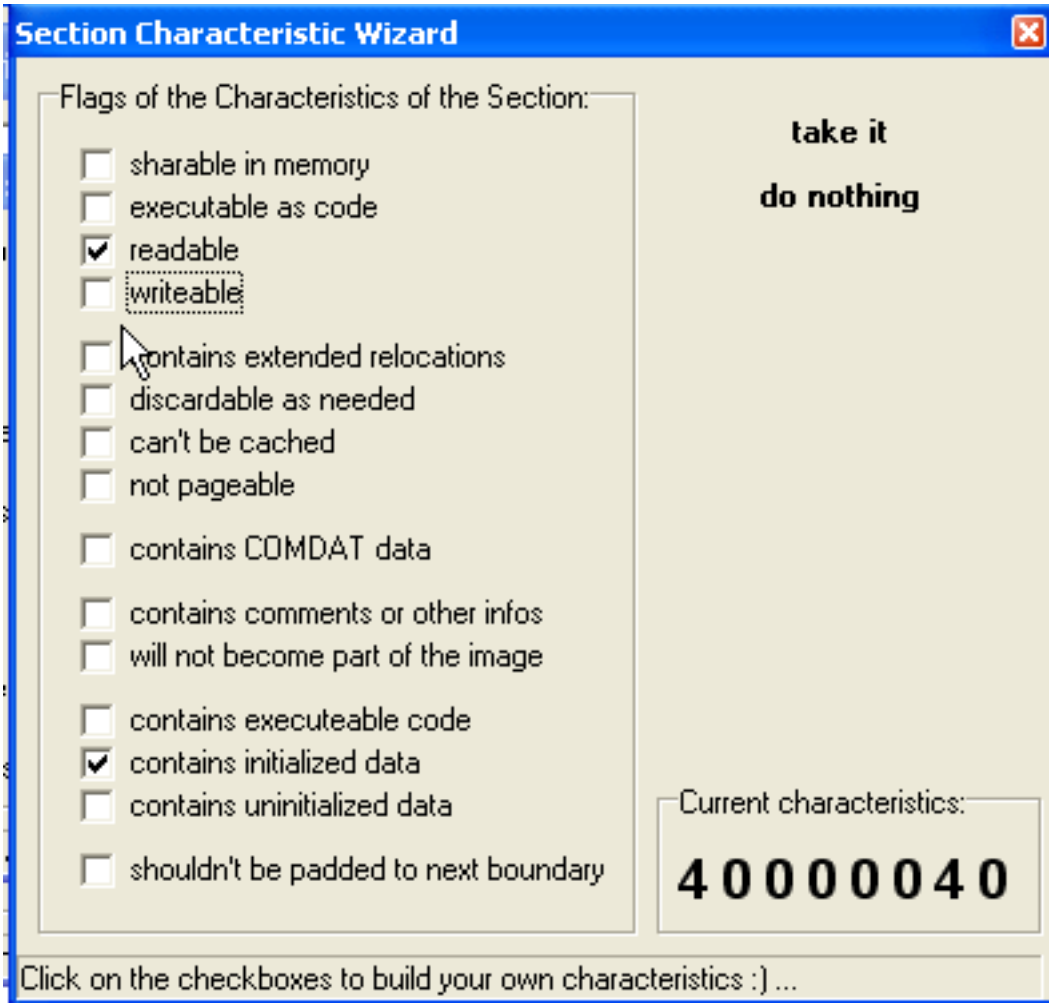
Мы ничего не знаем о программе, но очень возможно, что в ней есть функциональность большинства других упаковщиков, которые распаковываются в секцию, начинающуюся с 401000, то есть ту, которая у нас здесь идёт первой, так как здесь не показывается заголовок. Что произойдёт, если изменим параметры доступа к какой-нибудь секции, убрав право на запись, и произойдёт попытка записи сюда? Произойдёт ошибка и откроется PARCHEADO 5, который установлен как JIT-отладчик, хе-хе, так ли это? Можем попробовать это на первой секции, но думаю, что после распаковки программа сохранить API-функции из IAT, и это может быть в следующей секции, поэтому попробуем сначала с ней, а потом, если ничего не выйдет, можем попровать с первой.

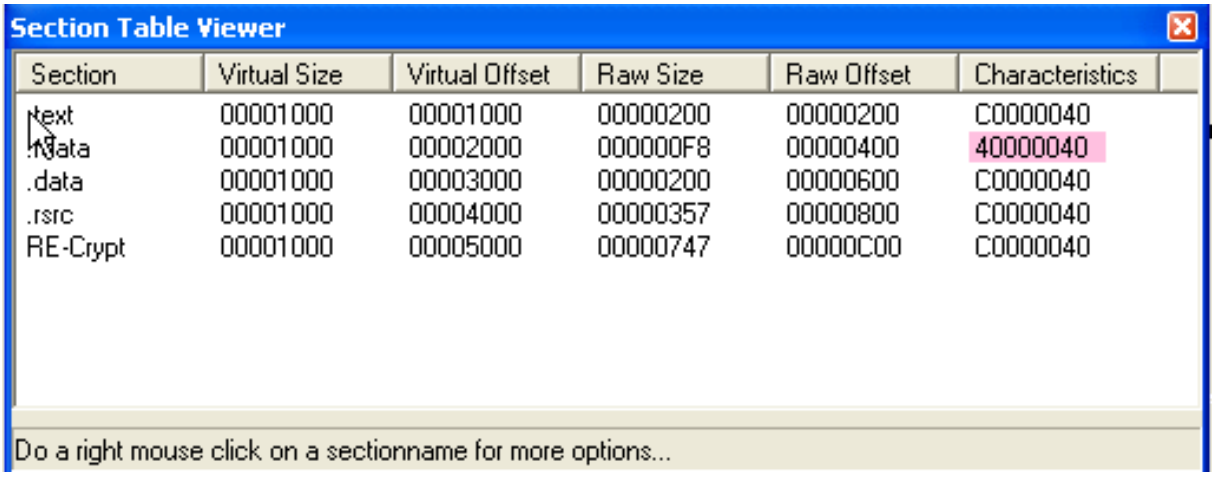


Открываем WIZARD, чтобы изменить параметры доступа к секции.



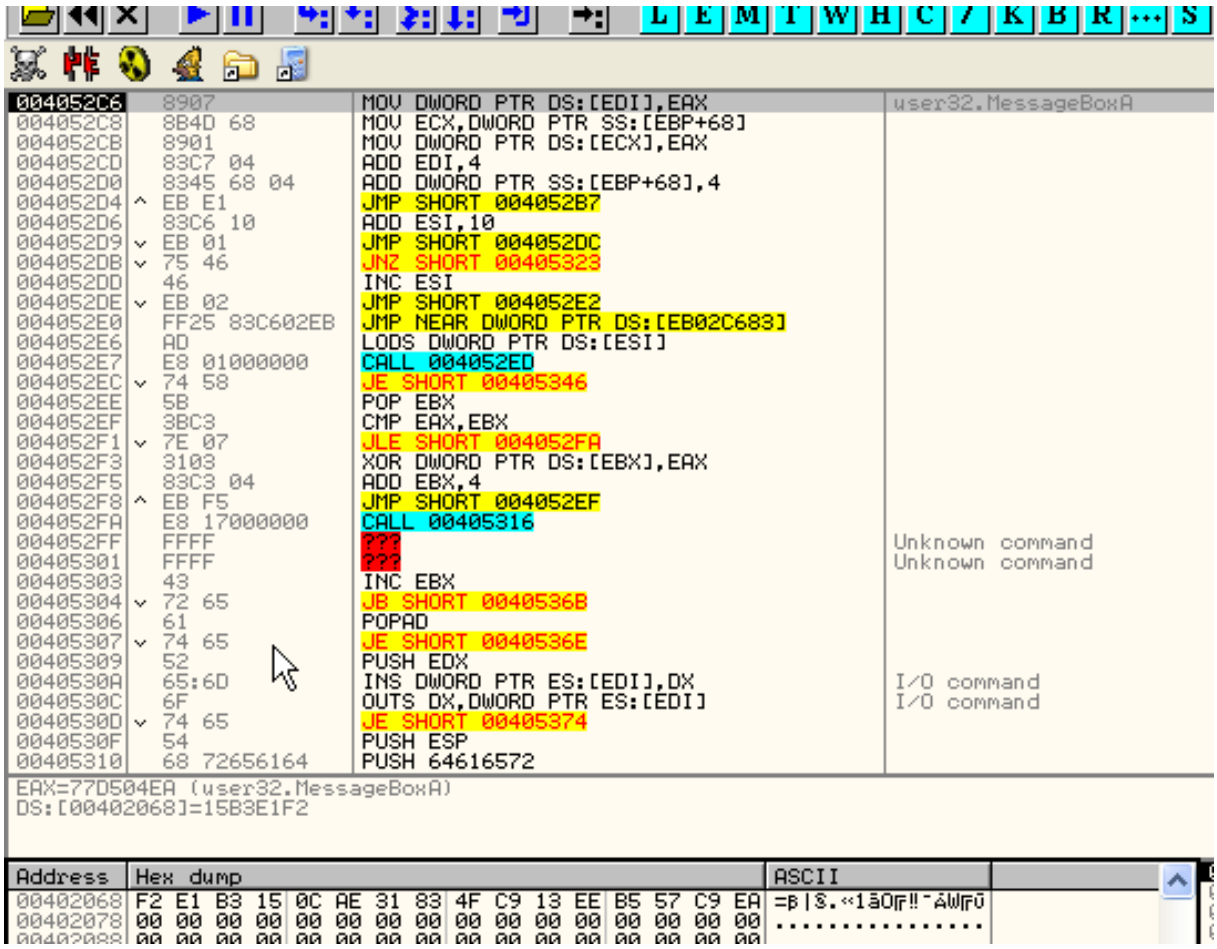
Снимаем галочку с «writeable».





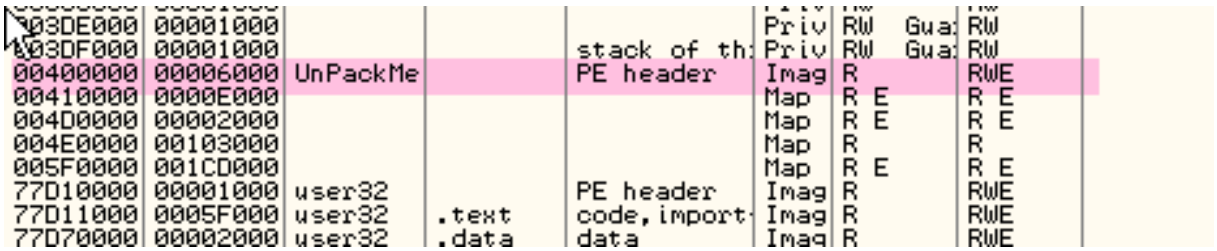
Здесь показаны изменённые параметры секции. Отметим, что автор может оказаться куда умнее меня и несмотря на начальное состояние секции менять права доступа к ней на желамые во время выполнения с помощью API-функции VirtualProtect, но ладно, смотрим.

Запускается этот распакуй-меня, у которого мы изменили права доступа, после чего появляется пропатченный OllyDbg, установленный как JIT-отладчик.

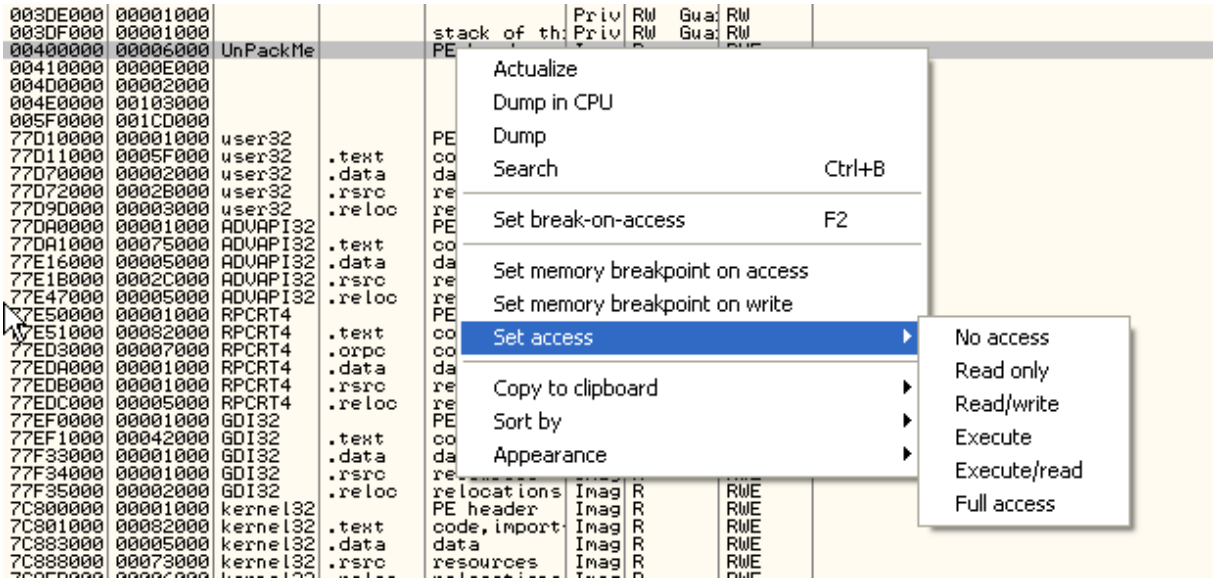


Видим, что программа остановилась, когда попыталась сохранить значение одной из API-функции в вышеуказанной секции (хе-хе, моё предположение было правильным), что привело к ошибке, открывшей OllyDbg, который автоматически подсоединился к программе.

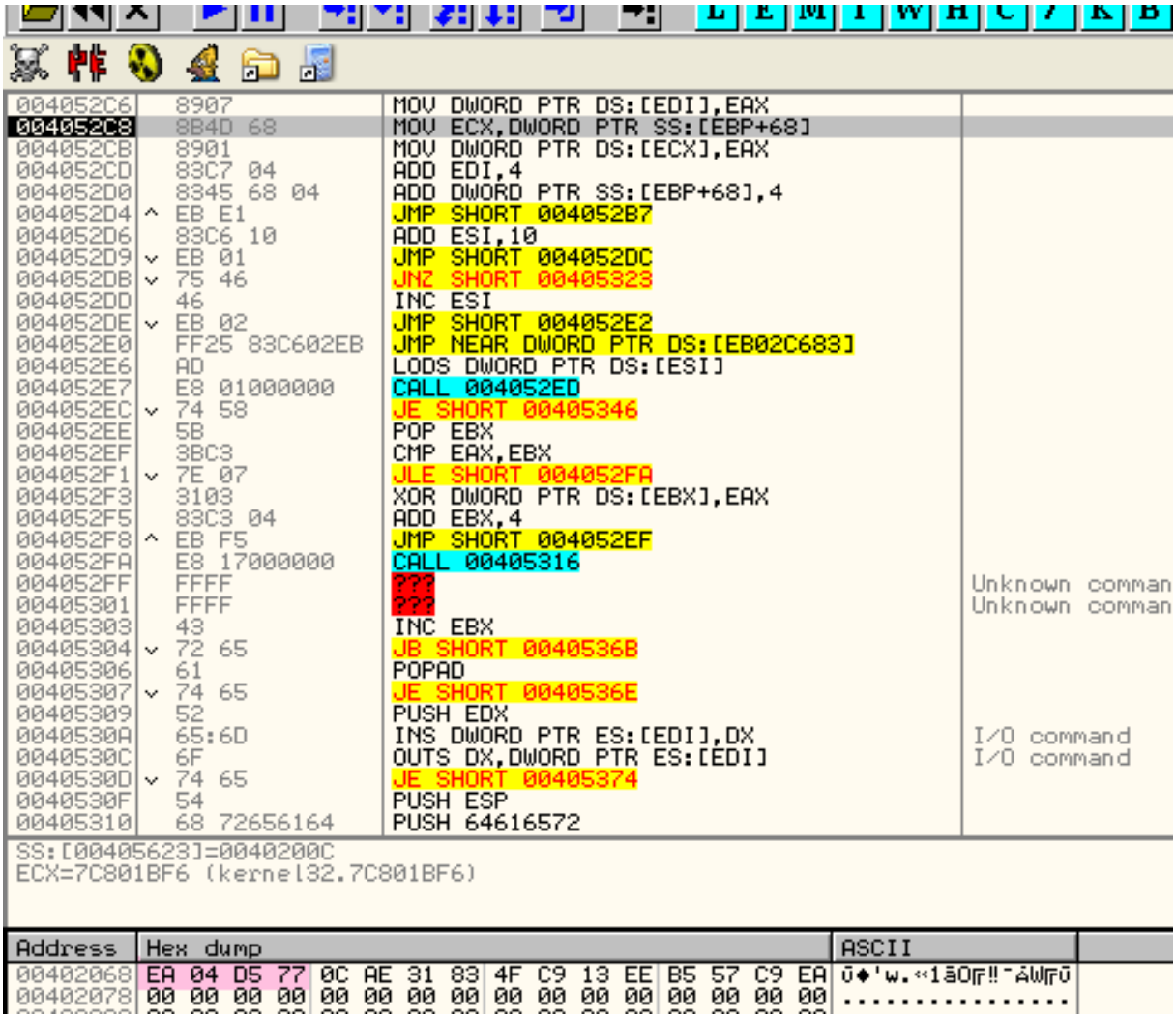
Теперь, чтобы можно было продолжить выполнение, нужно вручную изменить права доступа к этой секции, чтобы в неё можно было писать. Нажимаем «M».



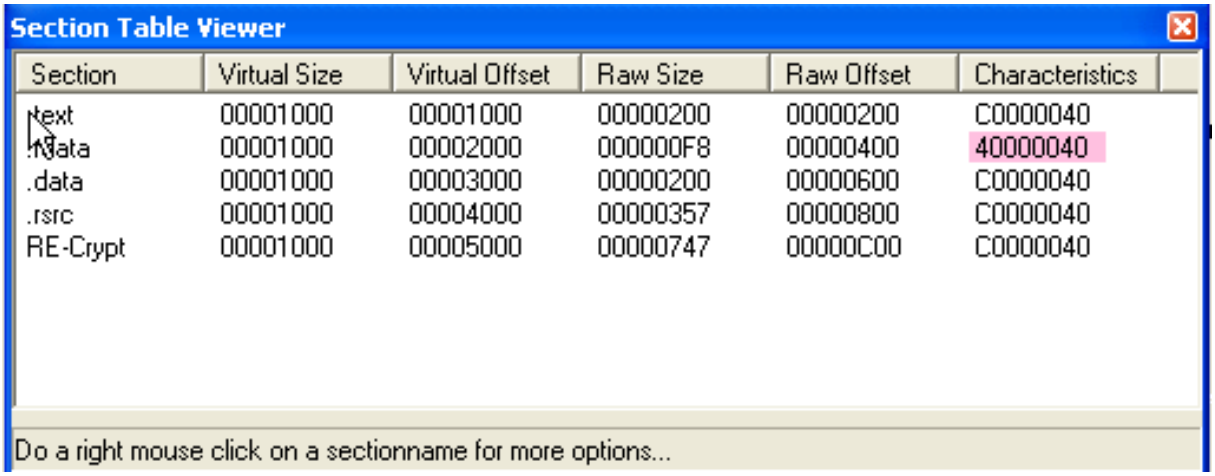
Неважно, что OllyDbg показывает нам только одну секцию. Делаем следующее: жмём на правую кнопку мыши и устанавливаем полный доступ, то есть «Full access».



Теперь пробуем выполнить инструкцию с помощью F7, посмотрим, нормально ли сохранится или снова выдаст ошибку.

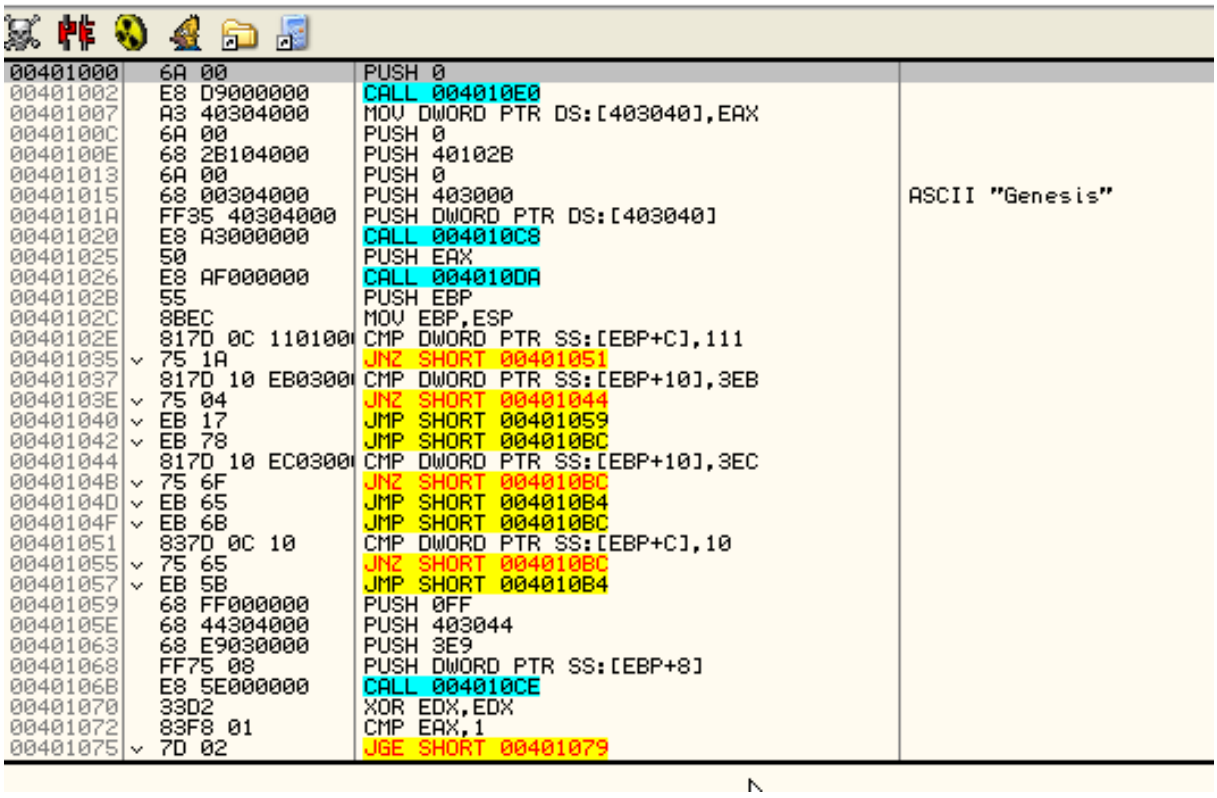


Видим, что сохранилось без каких-либо проблем, теперь запомним как располагаются секции в PeEditor'e, так как OllyDbg нам показывает эту информацию не очень хорошо.

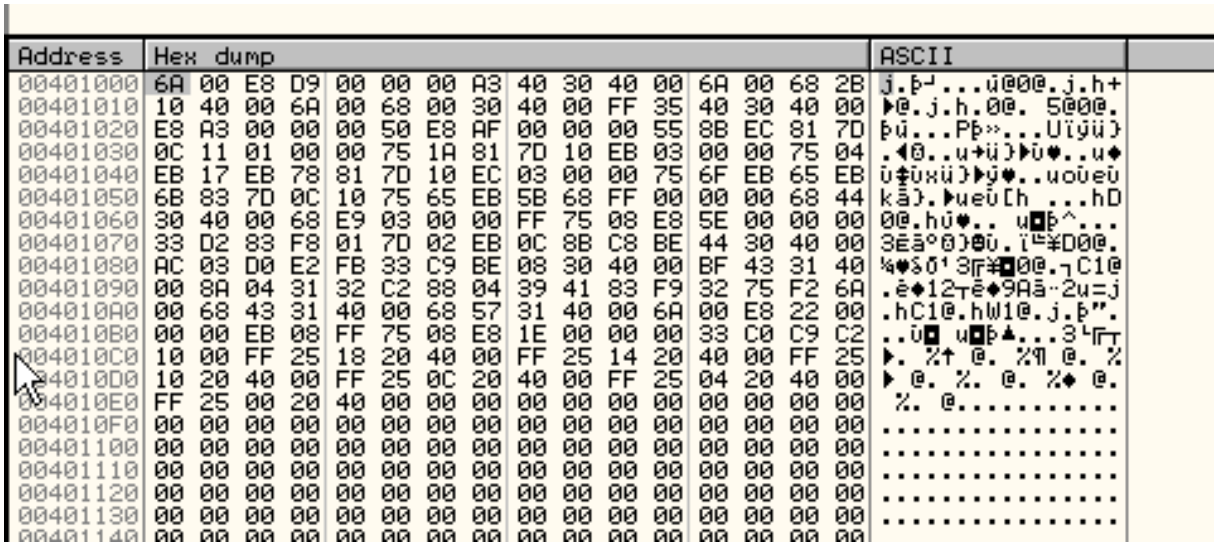


Видим, что первая секция занимает в памяти 1000 байт, так как VIRTUAL SIZE равен 1000, то есть располагается по адресам от 401000 до 402000. Следующая секция, права доступа к которой мы меняли, располагается с 402000 до 403000, хе-хе.

Если посмотрим, что находится в 401000, то увидим, что это распакованная программа.

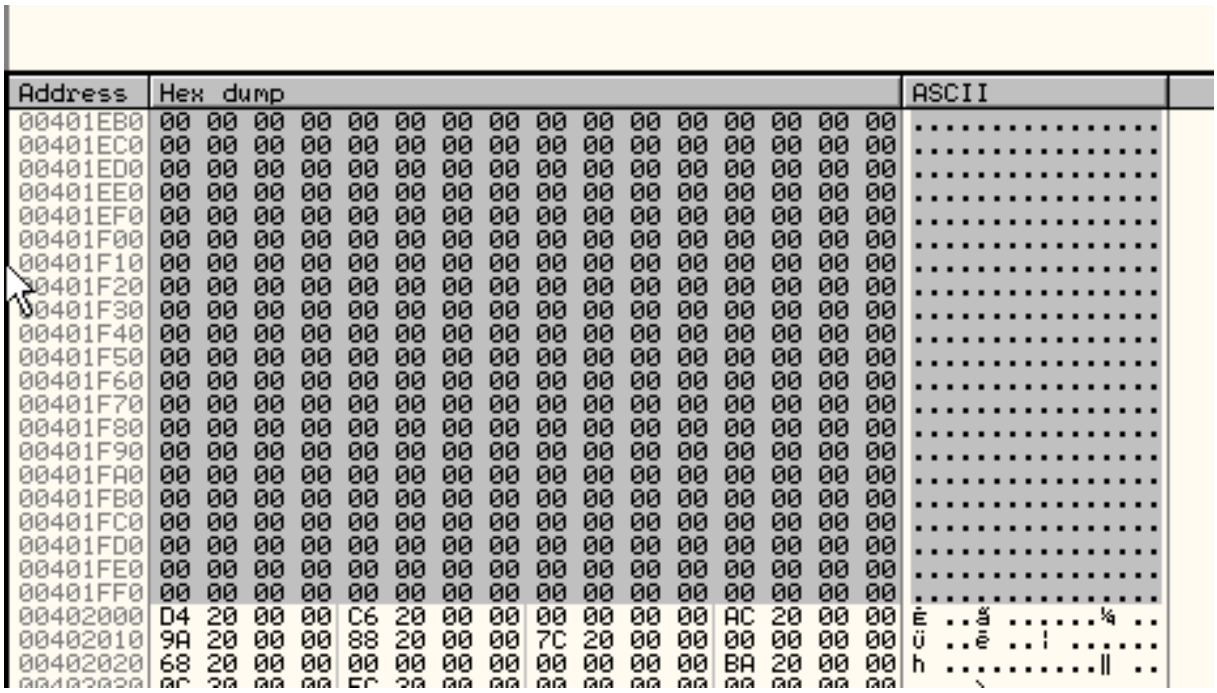


Так что, хотя не можем установить BPM на всю секцию с помощью карты памяти, так как OllyDbg показывает нам её неправильно, можем отметить в дампе байты с 401000 до 402000, то есть всю секцию, и установить на них BPM ON ACCESS, что по сути будет то же самое.

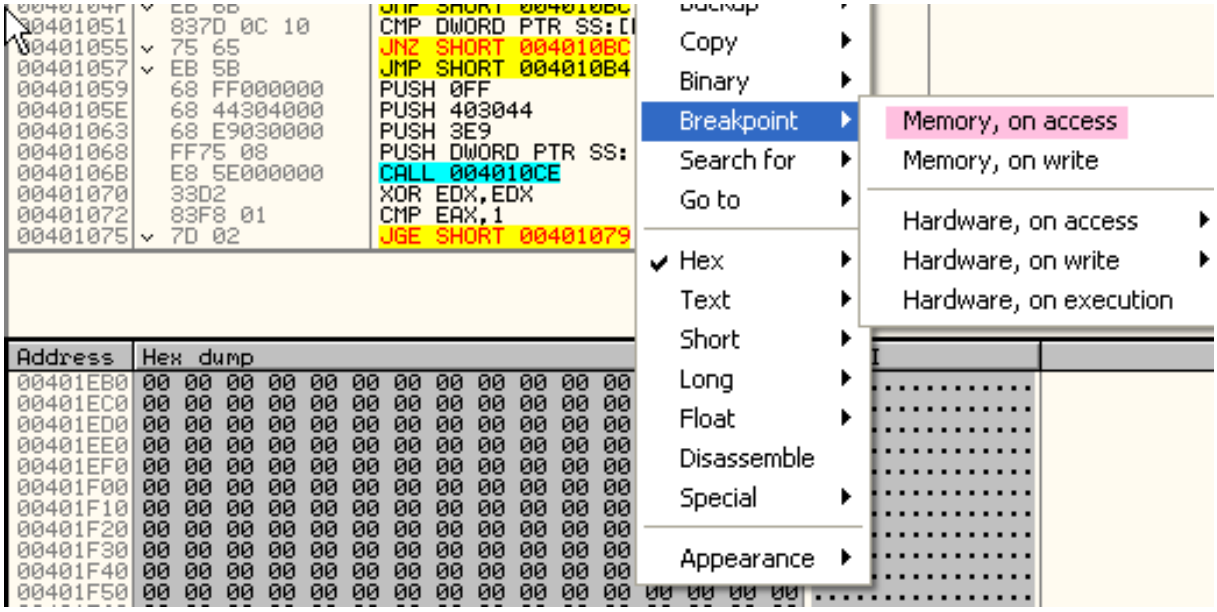


Address	Hex dump	ASCII
00401000	6A 00 E8 D9 00 00 00 A3 40 30 40 00 6A 00 68 2B	j.þ┘...ú@0@.j.h+
00401010	10 40 00 6A 00 68 00 30 40 00 FF 35 40 30 40 00	►@.j.h.0@. 5@0@.
00401020	E8 A3 00 00 00 50 E8 AF 00 00 00 55 8B EC 81 7D	þú...Pþ»...Uïÿü}
00401030	0C 11 01 00 00 75 1A 81 7D 10 EB 03 00 00 75 04	.◄@..u→ü}►ù♥..u♦
00401040	EB 17 EB 78 81 7D 10 EC 03 00 00 75 6F EB 65 EB	ù‡ùxü}►ÿ♥..uoùeù
00401050	6B 83 7D 0C 10 75 65 EB 5B 68 FF 00 00 00 68 44	kâ}.►ueù[h ...hD
00401060	30 40 00 68 E9 03 00 00 FF 75 08 E8 5E 00 00 00	0@.hú♥.. u◘þ^...
00401070	33 D2 83 F8 01 7D 02 EB 0C 8B C8 BE 44 30 40 00	3Ëâ°@}☻ù.ï╘¥D0@.
00401080	AC 03 D0 E2 FB 33 C9 BE 08 30 40 00 BF 43 31 40	¼♥§Ð'3╔¥◘0@.┐C1@
00401090	00 8A 04 31 32 C2 88 04 39 41 83 F9 32 75 F2 6A	.è♦12┬ê♦9Aâ¨2u=j
004010A0	00 68 43 31 40 00 68 57 31 40 00 6A 00 E8 22 00	.hC1@.hW1@.j.þ".
004010B0	00 00 EB 08 FF 75 08 E8 1E 00 00 00 33 C0 C9 C2	..ù◘ u◘þ▲...3└╔┬
004010C0	10 00 FF 25 18 20 40 00 FF 25 14 20 40 00 FF 25	►. %↑ @. %¶ @. %
004010D0	10 20 40 00 FF 25 0C 20 40 00 FF 25 04 20 40 00	► @. %. @. %♦ @.
004010E0	FF 25 00 20 40 00 00 00 00 00 00 00 00 00 00 00	%. @.....
004010F0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401100	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401110	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401120	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401130	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401140	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

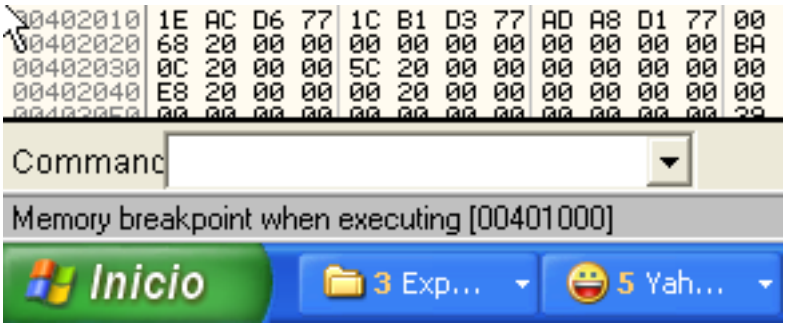
Затушёвываем всё с 401000 до 402000 и устанавливаем туда BPM ON ACCESS.



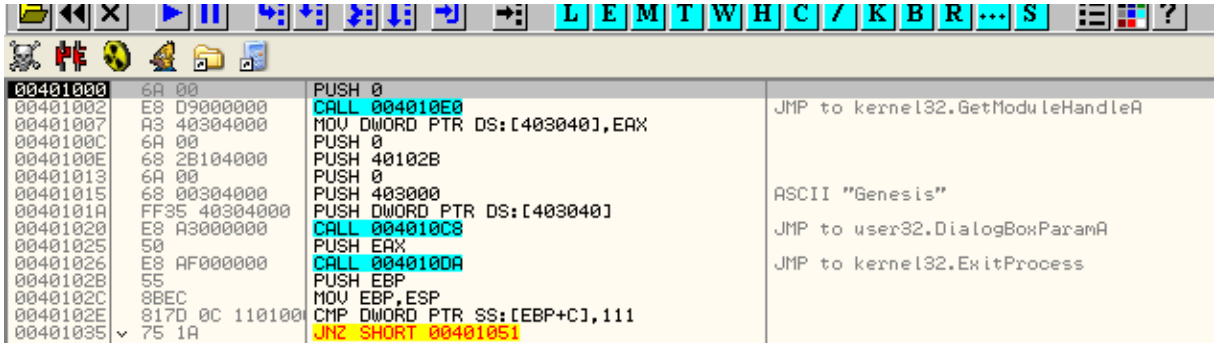
Address	Hex dump	ASCII
00401EB0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401EC0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401ED0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401EE0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401EF0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401F00	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401F10	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401F20	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401F30	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401F40	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401F50	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401F60	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401F70	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401F80	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401F90	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401FA0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401FB0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401FC0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401FD0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401FE0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00401FF0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
00402000	D4 20 00 00 C6 20 00 00 00 00 00 00 AC 20 00 00	╘ ..╟ .....¼ ..
00402010	9A 20 00 00 88 20 00 00 7C 20 00 00 00 00 00 00	ü ..ê ..| .....
00402020	68 20 00 00 00 00 00 00 00 00 00 00 BA 20 00 00	h ....‖ ..
00402030	0C 20 00 00 EC 20 00 00 00 00 00 00 00 00 00 00	



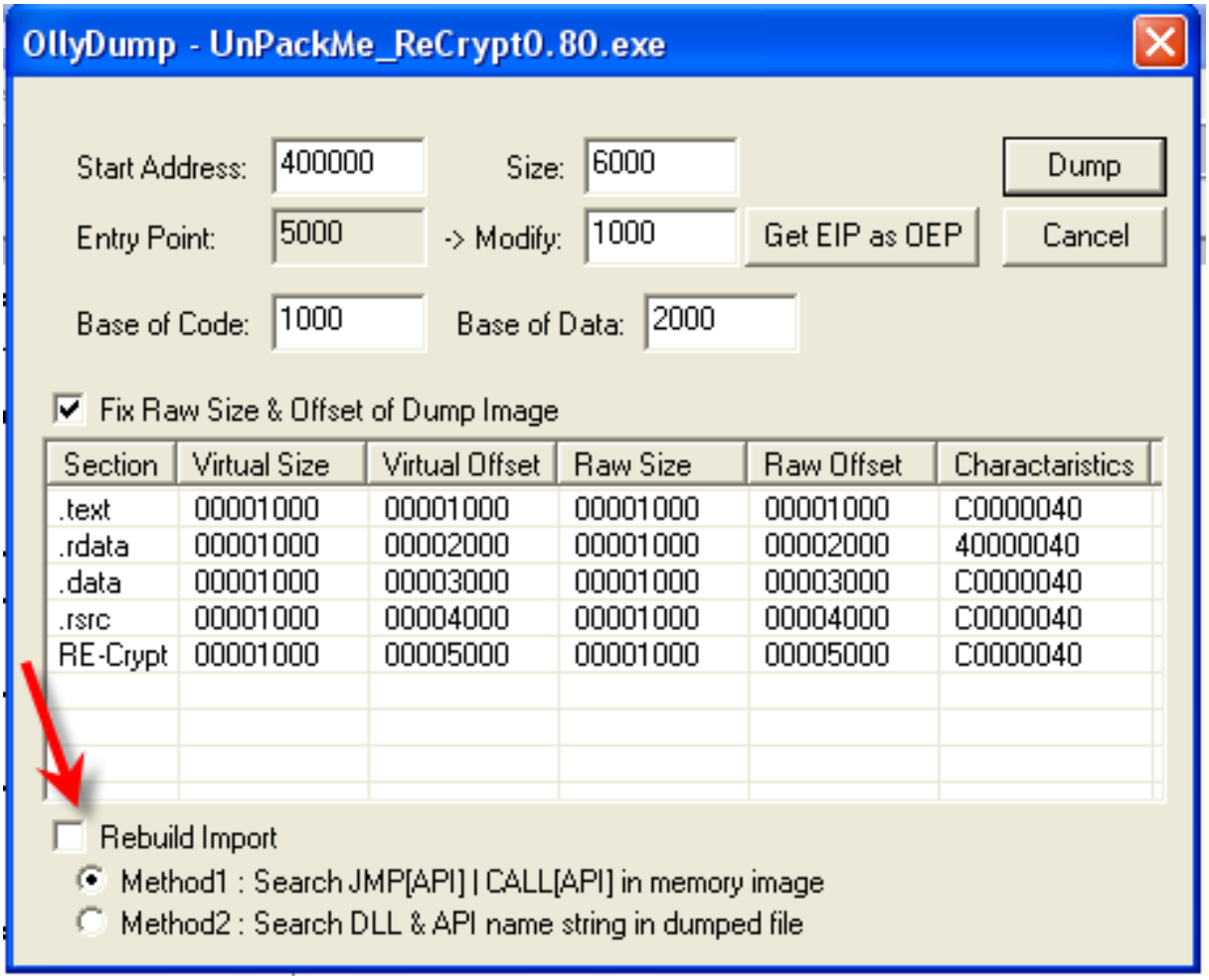
Теперь делаем RUN.



Теперь останавливаемся на OEP, который располагается точно в 401000.

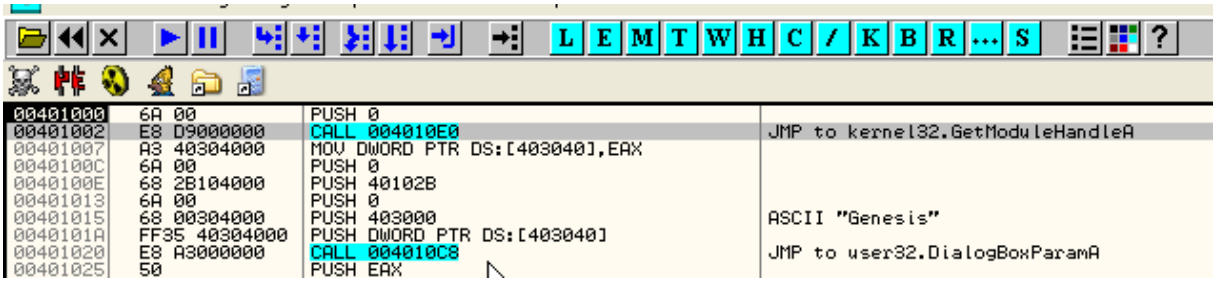


Ок, мы сделали самое сложное, теперь нужно сдампить программу.

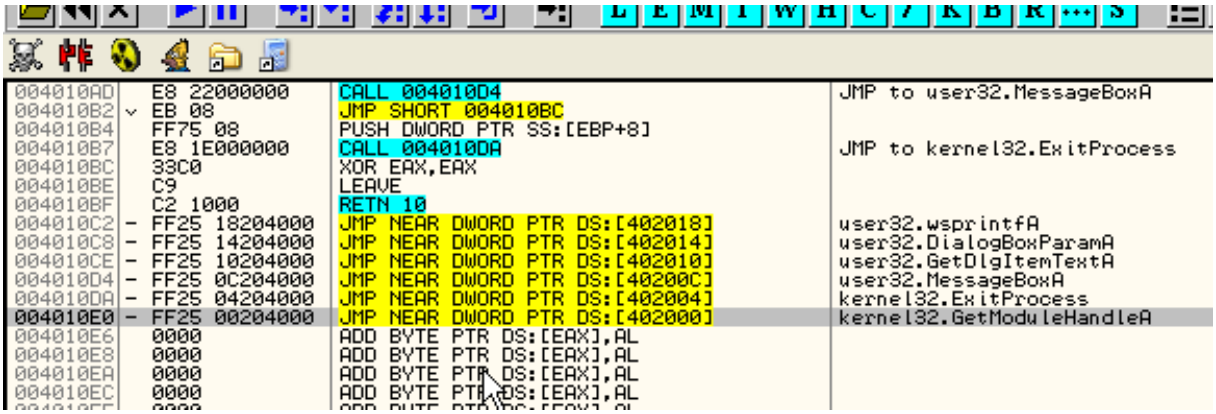


И сохраняем дам, теперь ищем IAT и данные для IMP REC'a.

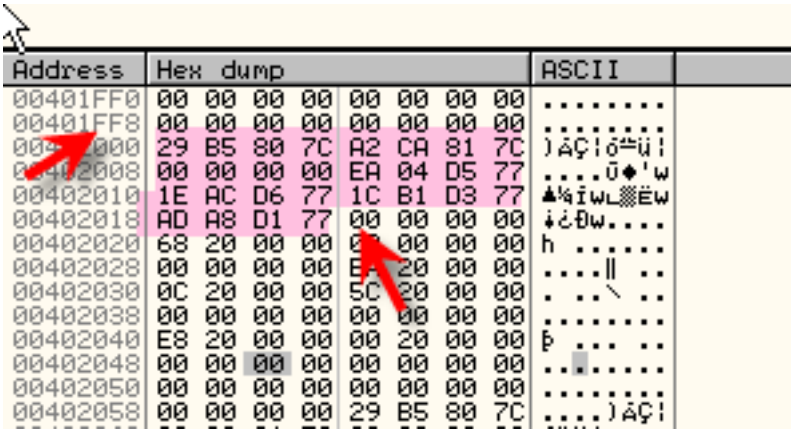
Под OEP есть вызов одной API-функции.



И если сделаем FOLLOW, то это приведёт нас к переходам на API-функции.

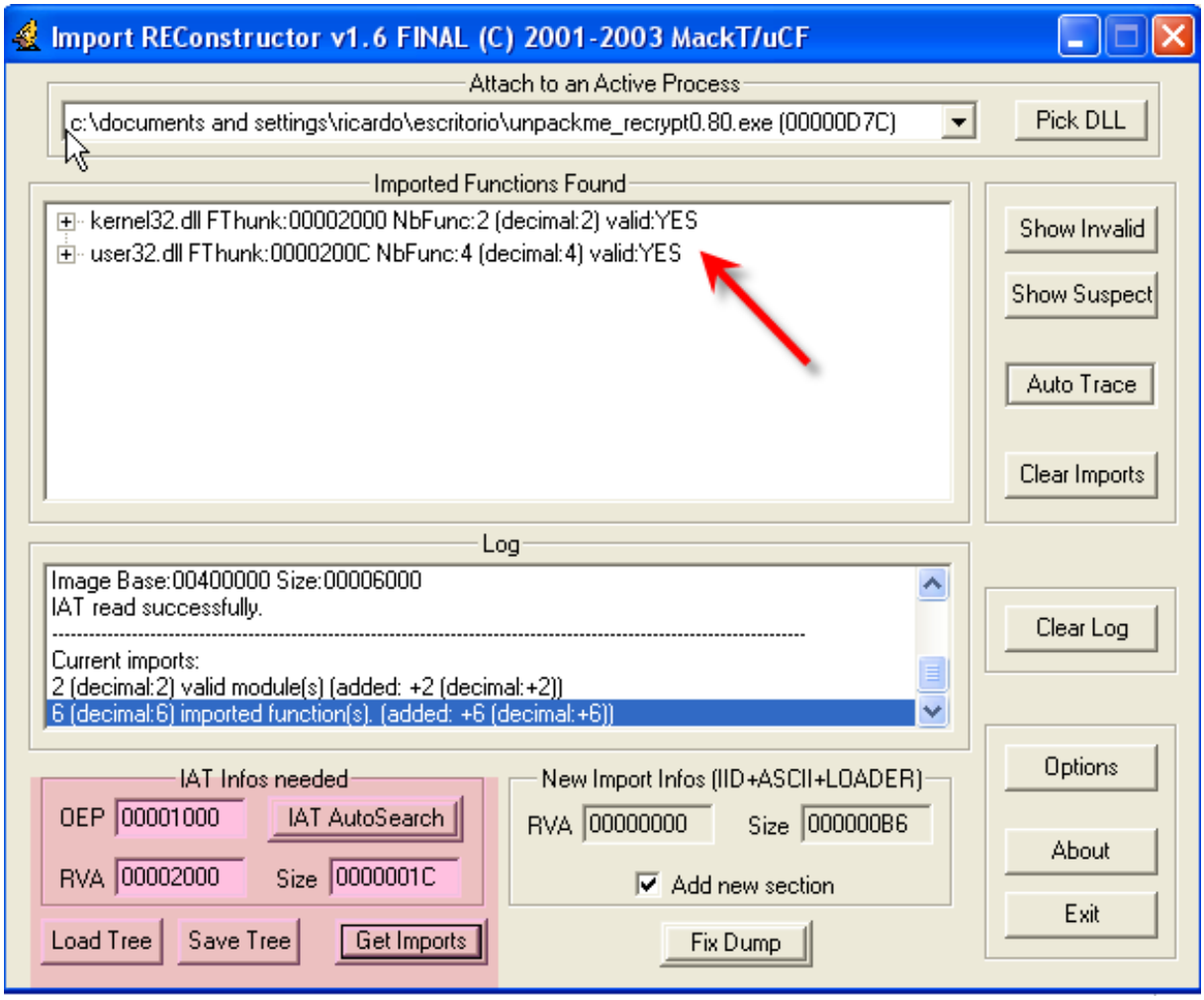


Так что найти IAT очень просто, посмотрев какой-нибудь из этих переходов, например тот, который ведёт на GetModuleHandleA содержит значение из области 402000, так что оно относится к IAT.



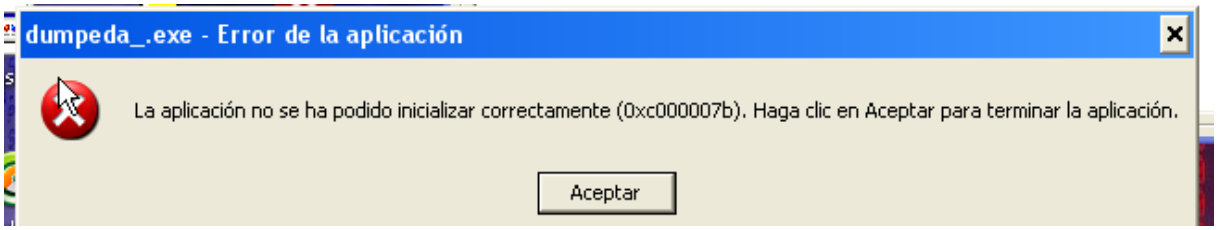
Эта IAT очень маленькая, так что начинается она в 402000, а заканчивается в 40201C, поэтому размер равен 1C. Открываем IMP REC и отнимаем от найденных адресов начала и конца базу образа, то есть 400000.

```
RVA или НАЧАЛО: 2000
SIZE: 1C
OEP: 1000
```

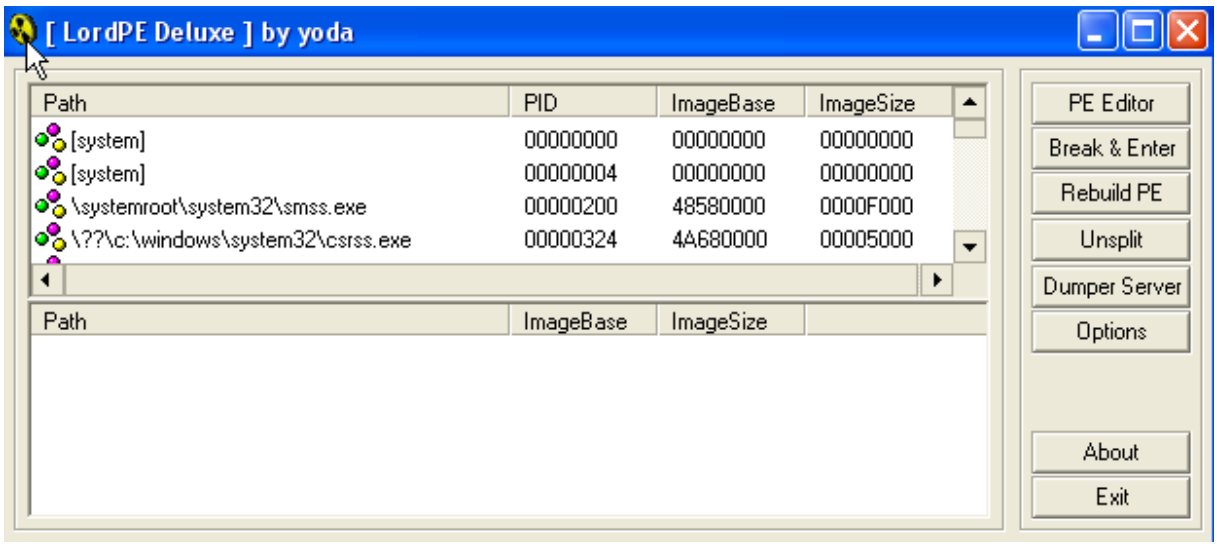


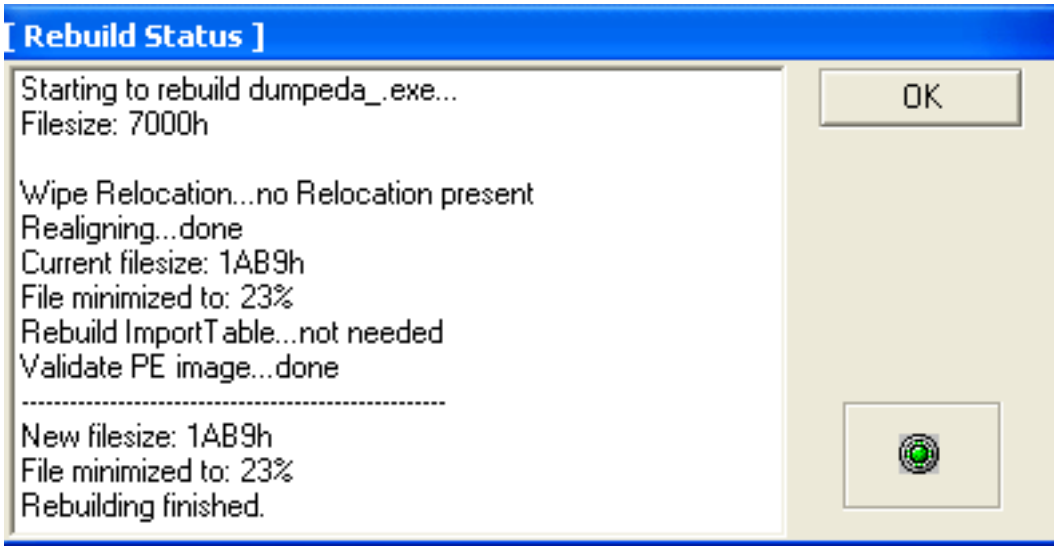
Заполняем поля для OEP, RVA и SIZE, а затем нажимаем GET IMPORTS. Оно говрит нам, что всё «YES», хе-хе.

Пропускаем дамп через «FIX DUMP», и при запуске нам выдаётся ошибка.

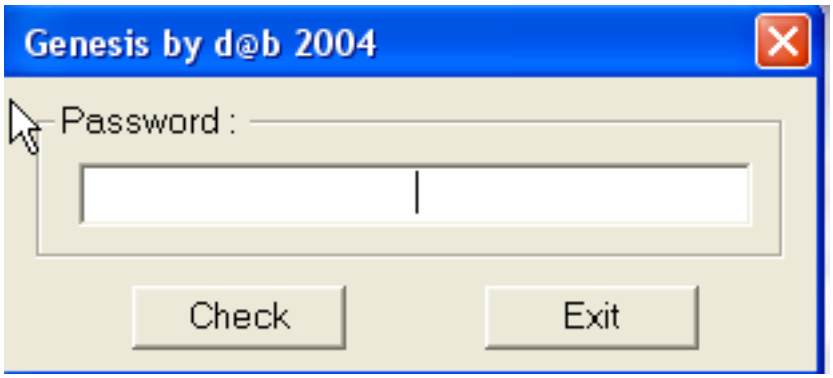


Нам потребуется обработать его в LordPE с помощью «Rebuild PE».





Запускаем, и прощай, упаковщик!



Наши усилия по применению необычных методов дали свои плоды. Некоторые вещи упаковщики делают одинаково, однако они также защищают себя против обычных методов.

Файлы к статье (находится в архиве `wasm_files.zip`: `files/0518/ollydbg45.zip`)

Дополнение к 45-ой главе «Введения в крэкинг, используя OllyDbg»

Автор: Lisa и Alquimista, пер. Aquila

Дата: 2009-08-10

От переводчика: приложении к 45-ой главе находился PDF на испанском языке, и эта статья является его переводом на русский язык.

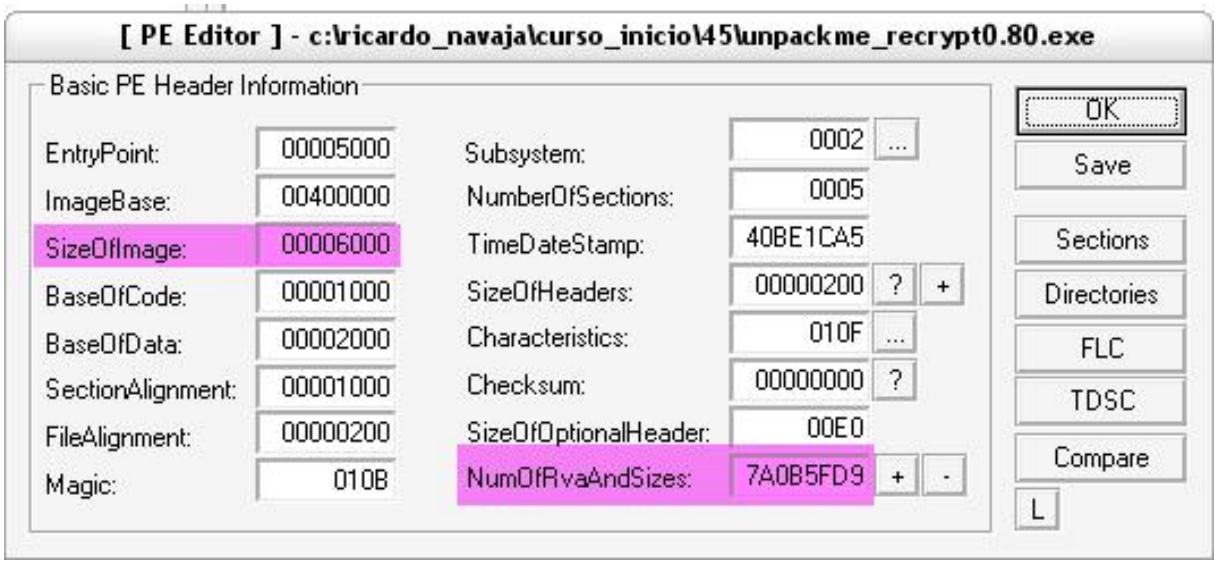
В этой статье показан другой способ противостоять упаковщику из вышеуказанной главы единственно с помощью щелчков по кнопкам мыши без необходимости делать что-то ещё.

Замечание: используемый Ollydbg - это обычная версия, единственные модификации - это Ollyghost и плагин Ollydump для дампа процесса.

• Также используется PE Editor, LordPE и две утилиты из списка - Estricina и Pokemon Anti_Attach.

Сводная информация

Первым делом до того, как атаковать программу в памяти, посмотрим на данные файла, для чего откроем программу в LordPE.



Эти данные нам понадобятся, так как упаковщик может изменить их во время распаковки, и мы ничего не сможет сделать. Как видим, данные NumberOfRvaAndsizes были изменены, так как знаем, что это поле содержало 10 в шестнадцатеричной системе. Изменим его, для чего необходимо его смещение, так что открывает редактор PE. Видим, что смещение равно B4.

Следующее, что нам необходимо знать - это OEP, и с этим нам поможет плагин PEID.

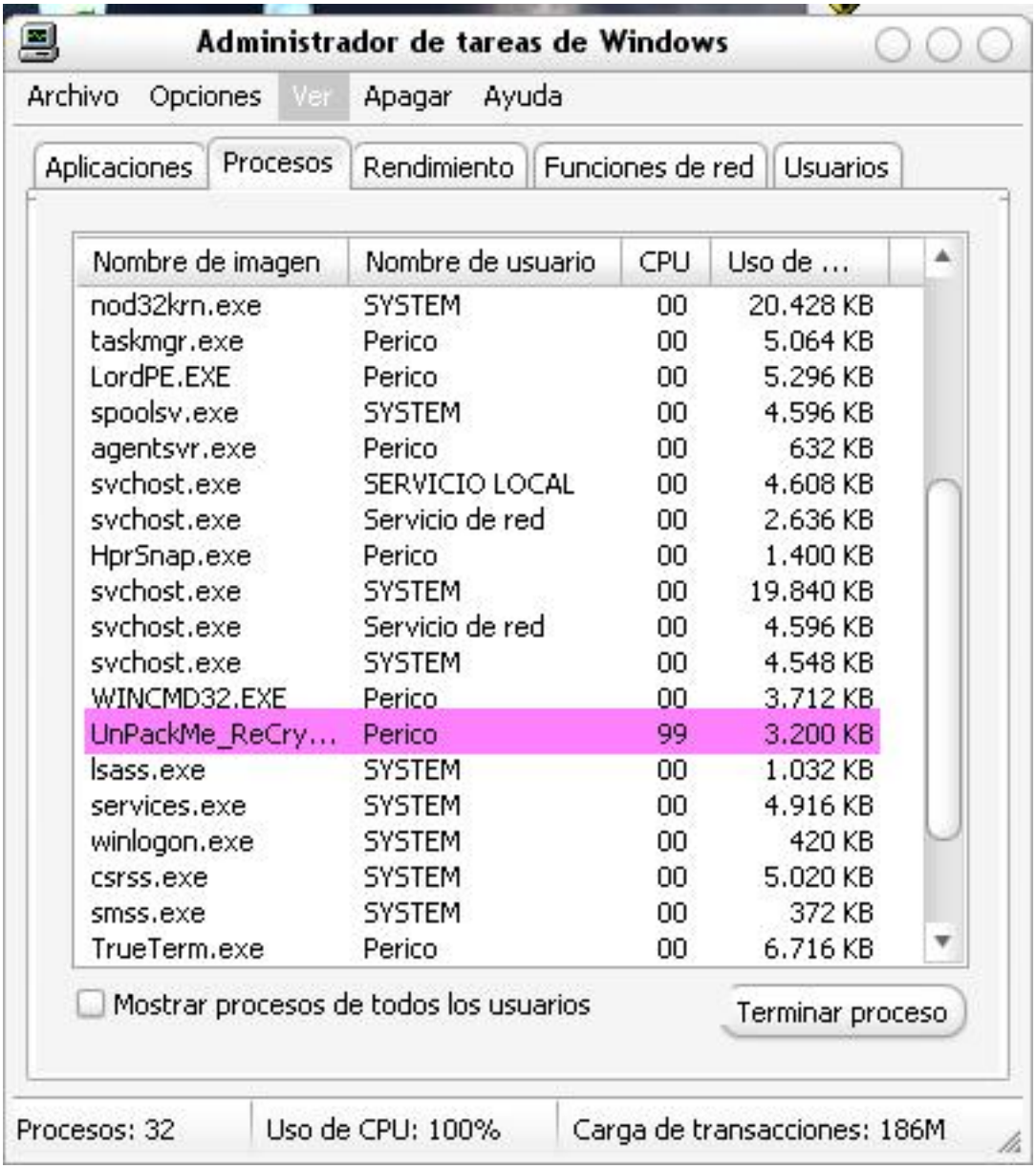
Ок, теперь у нас есть все сведения, которые нам нужны, чтобы начать... Стало быть, идём в атаку.

Полученые данные:

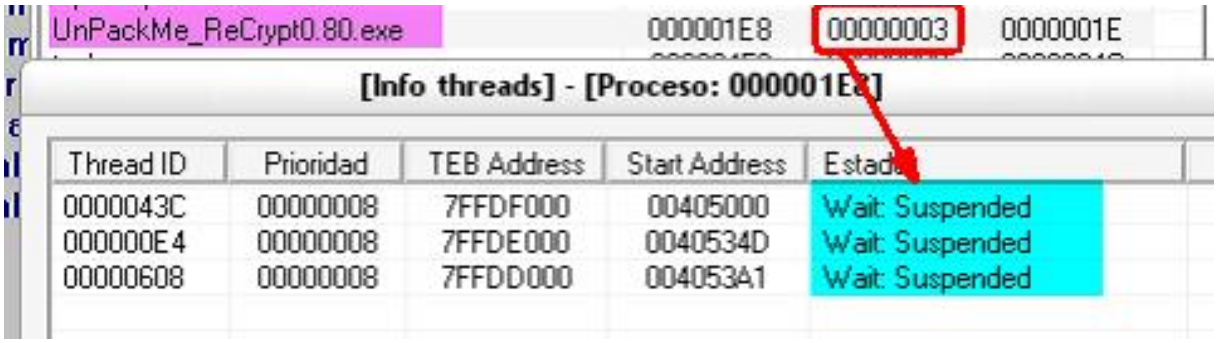
1. OEP 00401000
2. SizeOfImagen 00006000
3. NumberOfRvaAndSizes Находится по смещению b4 и было изменено на 10

Атака

Запускаем упаковщик вне OllyDbg, и он работает.



Чтобы работать с большим удовольствием, будем использовать утилиту Marciano, которая называется Estricina, с помощью которой остановим третью нить, которая содержит упаковщик (этот шаг не является необходимым, но делает работу более удобной).



Если мы делаем это, то загрузка процессора нитью падает с 99 процентов до нуля.

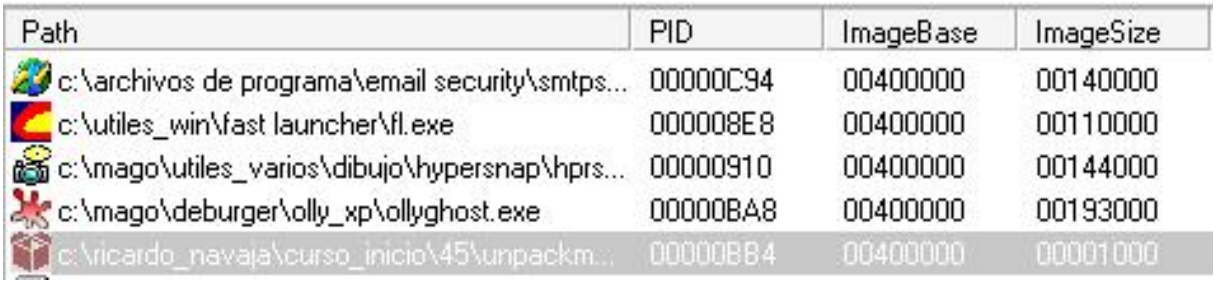
Nombre de imagen	Nombre de usuario	CPU	Uso de ...
SMTPServer.exe	Perico	00	5.600 KB
taskmgr.exe	Perico	01	2.504 KB
UnPackMe_ReCry...	Perico	00	3.332 KB
OllyGhost.exe	Perico	00	1.008 KB
HprSnap.exe	Perico	00	3.124 KB
fl.exe	Perico	00	6.428 KB
alg.exe	SERVICIO LOCAL	00	3.936 KB
nod32kui.exe	Perico	00	2.504 KB
ALCXMNTR.EXE	Perico	00	3.412 KB
jusched.exe	Perico	00	1.864 KB
THUNDE~1.EXE	Perico	00	1.832 KB
svchost.exe	SYSTEM	00	3.832 KB
nvsvc32.exe	SYSTEM	00	1.872 KB
nod32krn.exe	SYSTEM	00	20.152 KB
explorer.exe	Perico	02	26.900 KB
WINWORD.EXE	Perico	00	3.092 KB
WINCMD32.EXE	Perico	01	8.884 KB
spoolsv.exe	SYSTEM	00	4.600 KB
svchost.exe	SERVICIO LOCAL	00	4.692 KB

Теперь мы можем спокойно использовать нашу машину.

Атака в памяти, чтобы нас не обнаружили

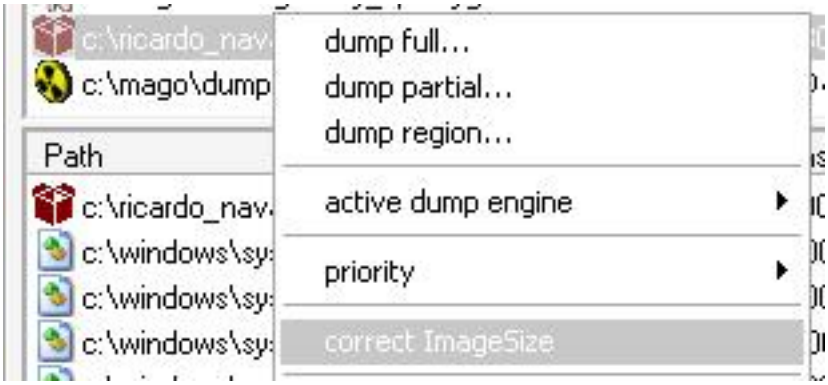
После использования Pokemos'a, чтобы избежать возможных противоприсоединительных и тому подобных приёмов, восстанавливаем значение NumberOfRvaSizes, которое, как мы помним, находится по смещению B4.

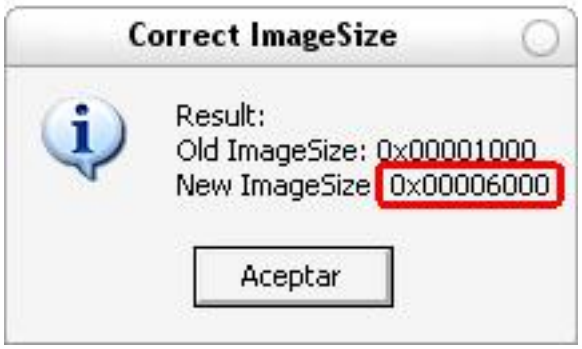
Хорошо, если была защита против присоединения, то она исчезла. Продолжаем атаковать упаковщик в памяти. Открываем LordPE, и ищем упаковщик в списке процессов.



Как видим, у него размер образа равна 0001000, и мы знаем, что он должен быть равен 0006000, и это запутывает OllyDbg и любой дампер, поэтому у нас на выходе и не получается хорошего исполняемого файла, кроме того это мешает использовать программу Import для восстановления таблицы импорта.

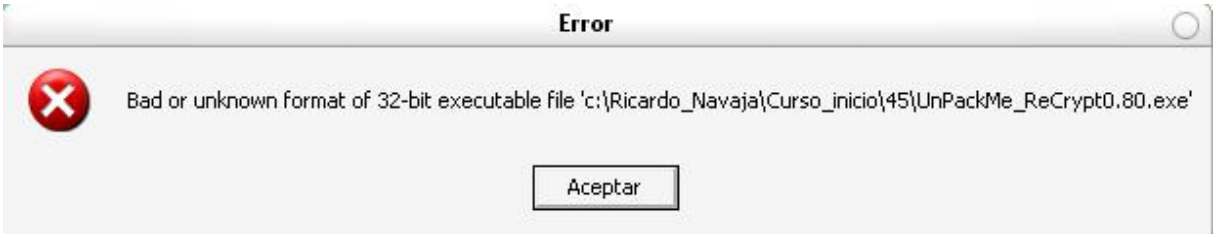
Вывод: нужно починить его, это понятно. Кликаем по нёму в LordPE правой кнопкой мыши и исправляем



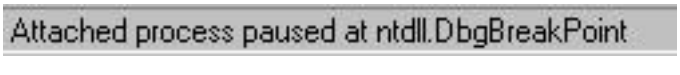


И как видим на скриншоте справа, размер стал 0006000, каким он и должен быть.

Уже починили часть программы в памяти, теперь подсоединяемся с помощью OllyDbg и видим, что она сообщает нам о том, что заголовок неверен.



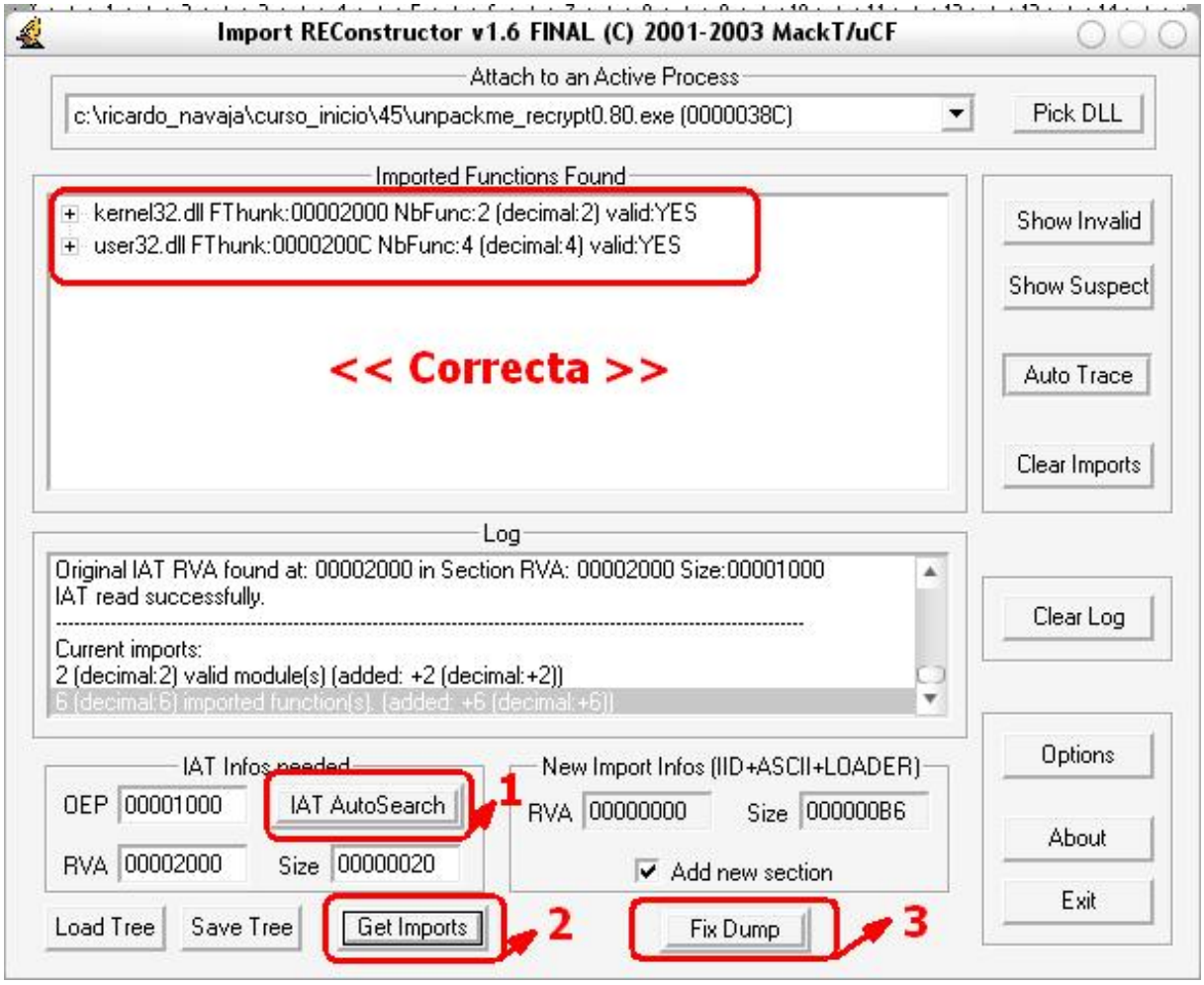
Жмём на "Ок" и программа аттакуется, останавливается на API-функции.



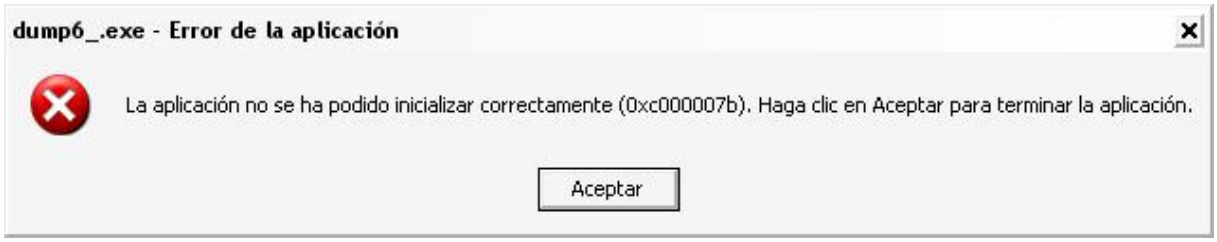
Теперь так как знаем его OEP, идём в окне кода в 00401000 и оказываемся на OEP программы.

И видим, что с ней всё в порядке... Нажимаем правую кнопку мышью.

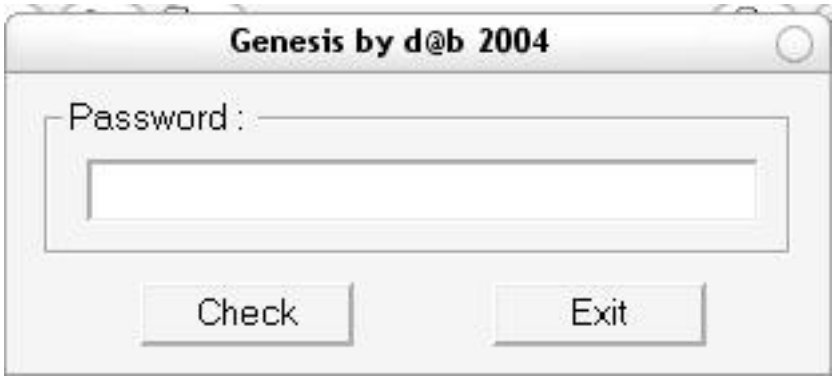
И видим в регистре EIP, что остановились на OEP, так что теперь переходим к сдампливанию с помощью плагина.



Осталось только добавить таблицу, которую подготавливаем с помощью Import. Вводим OEP и нажимаем три кнопки по порядку, выбираем сдампленный ранее файл и не выходим из окна, в котором заголовок находится в плохом состоянии. Об этом нам ещё ранее сказал OllyDbg.



Реконструируем заголовок с помощью какой-нибудь программы, у которой есть такая функция, в данном случае это LordPE, и программа становится нормальной.



И если посмотрим в администраторе задач Windows, увидим, что программа больше не пожирает ресурсов - упаковщик исчез.

Nombre de imagen	Nombre de usuario	CPU	Uso de ...
alg.exe	SERVICIO LOCAL	00	3.892 KB
jusched.exe	Perico	00	1.868 KB
WINWORD.EXE	Perico	00	2.012 KB
HprSnap.exe	Perico	00	1.292 KB
agentsvr.exe	Perico	00	1.128 KB
svchost.exe	SYSTEM	00	3.812 KB
nvsvc32.exe	SYSTEM	00	1.868 KB
nod32krn.exe	SYSTEM	00	18.440 KB
explorer.exe	Perico	00	20.412 KB
fl.exe	Perico	00	6.108 KB
WINCMD32.EXE	Perico	00	8.136 KB
spoolsv.exe	SYSTEM	00	4.592 KB
dump6_.exe	Perico	00	3.052 KB
wscntfy.exe	Perico	00	2.472 KB
svchost.exe	SERVICIO LOCAL	00	4.608 KB
svchost.exe	Servicio de red	00	2.632 KB
svchost.exe	SYSTEM	00	18.816 KB
svchost.exe	Servicio de red	00	4.564 KB
svchost.exe	SYSTEM	00	4.484 KB

Ок, достаточно на сегодня.

Введение в крэкинг с нуля, используя OllyDbg - Глава 46

Автор: Рикардо Нарваха, пер. Aquila

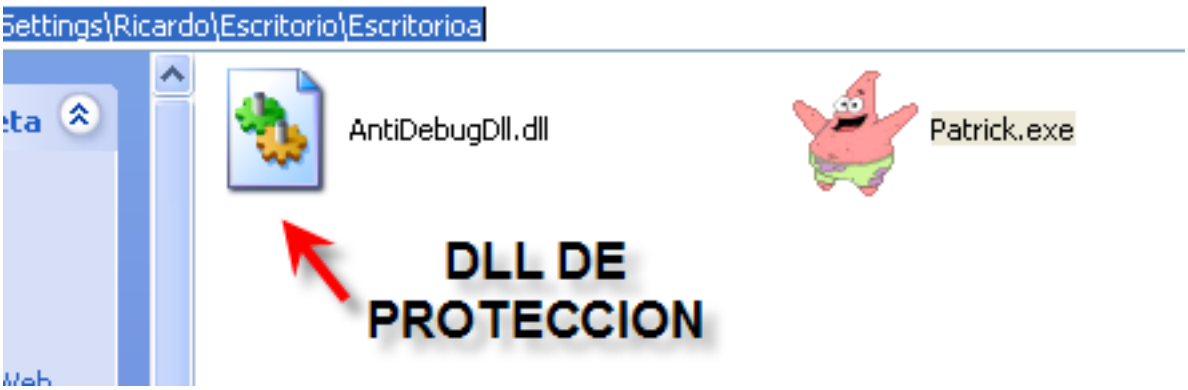
Дата: 2009-09-13

В этой главе продолжаем изучать методы, которые мы пытаемся применить, когда всё начинает идти не так, как мы ожидали. В этой главе для этих целей нам послужить прекрасный [missing file: ../files/0521/ollydbg46_patrickori.zip], который называется PATRICK.exe. Оно довольно мощное, и думаю, что прежде, чем углубляться в более сложные упаковщики, необходимо научиться этим альтернативным методом, включая те, о которых я рассказываю, но они не срабатывают, так как во многих случая нам придётся подстраиваться, адаптироваться и искать брешь среди лабиринта защиты, воздвигнутого программой, как, например, с этим крэкми, который мы собираемся распаковать.

Конечно, нужно упомянуть, что коммерческий упаковщик не всегда может иметь все стандартные приёмы упаковщика, сделанного специально (как в данном случае), так как если упаковщик коммерческий, то его должно быть возможно применить к любой программе, в то время как авторский может содержать специальную защиту, завязанную на конкретную программу.

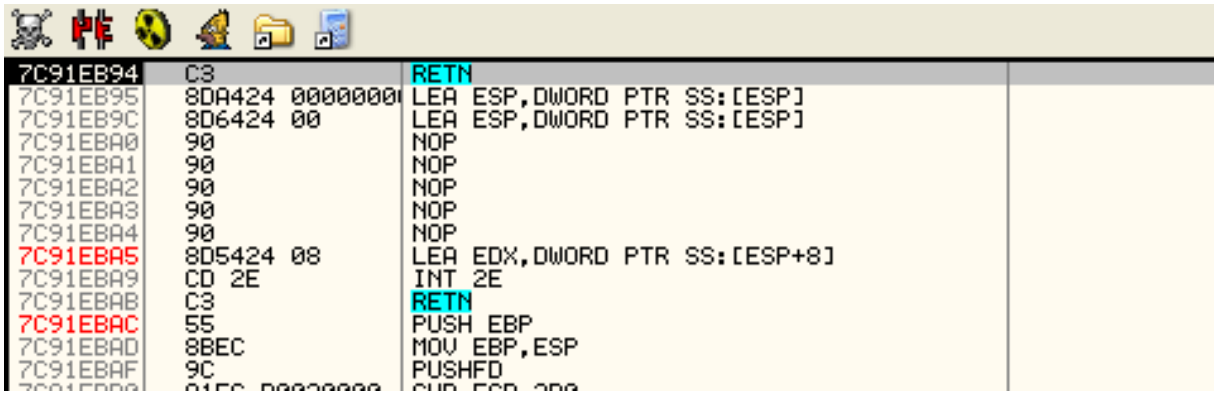
Хорошо, открываем наш OllyDbg, как обычно используем знакомый нам модифицированный OllyDbg, замаскированный плагинами (патч №4).

Согласно автору, целью является не только распаковть крэкми, но и заставить его запуститься без DLL защиты, которую мы видим рядом с exe-файлом.



Хорошо, это нам говорит, что указанная DLL, предположительно, ответственна за всё. Если удалим её из этой папки, то программа попросит у нас эту библиотеку и прекратит выполнение.

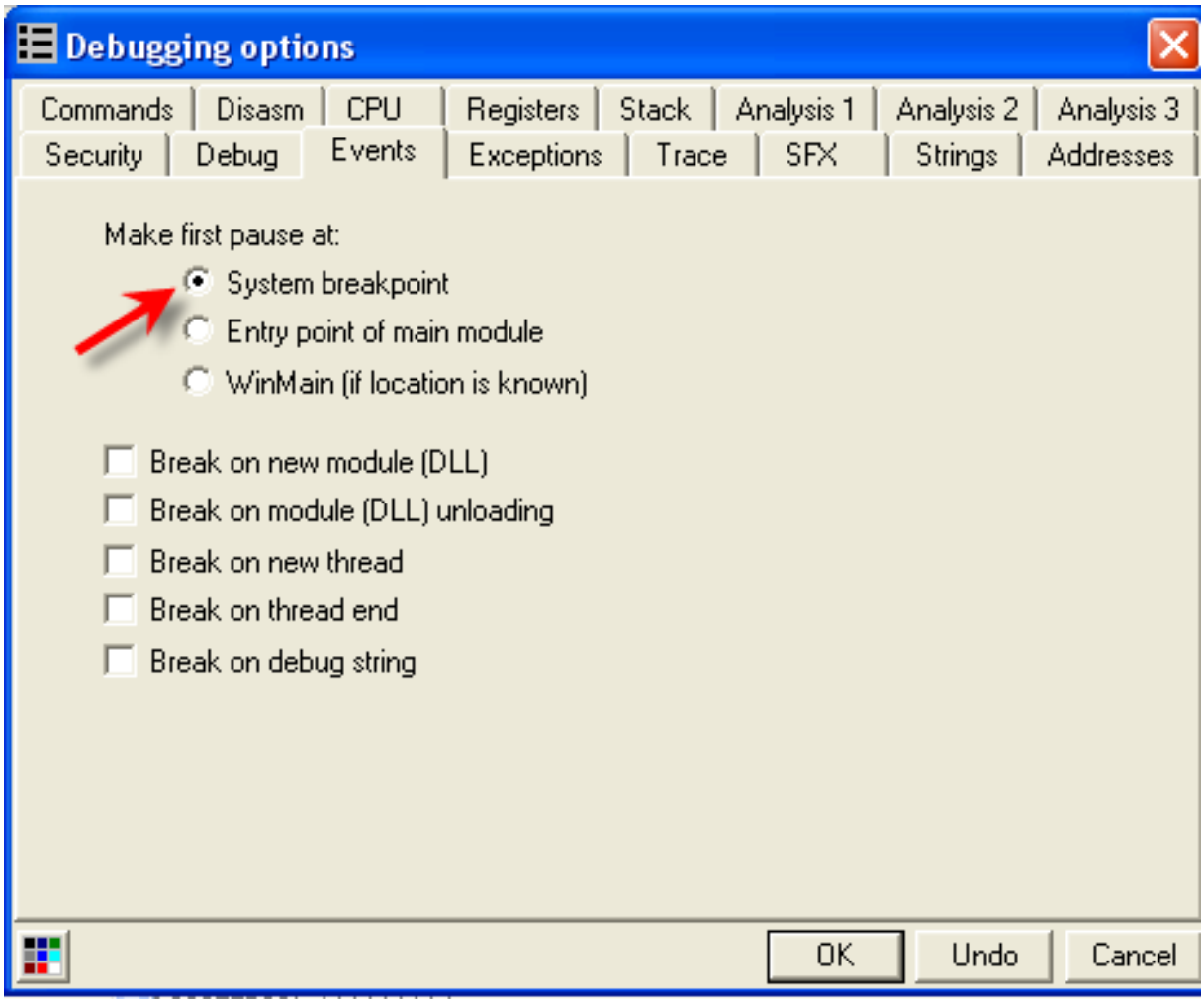
Ок, запускаем крэкми в патче №4.



Видим, что выполнение прекращается ещё до прибытия в точку входа.

Хорошо, посмотрим, как изменить, что OllyDbg останавливается до точки входа, то есть на системной точке останова, которая находится немного раньше, в которой система может остановиться немного раньше точки останова.

Для этого в настройках OllyDbg меняем в DEBUGGING OPTIONS-EVENTS.

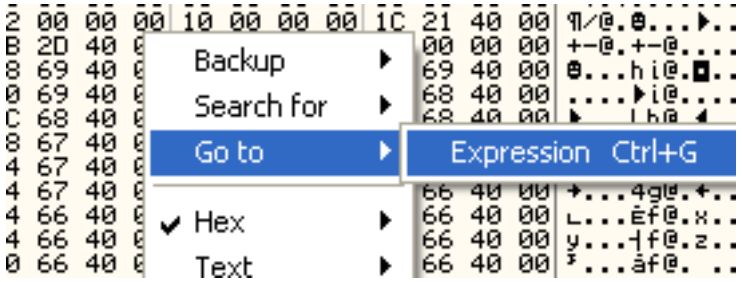


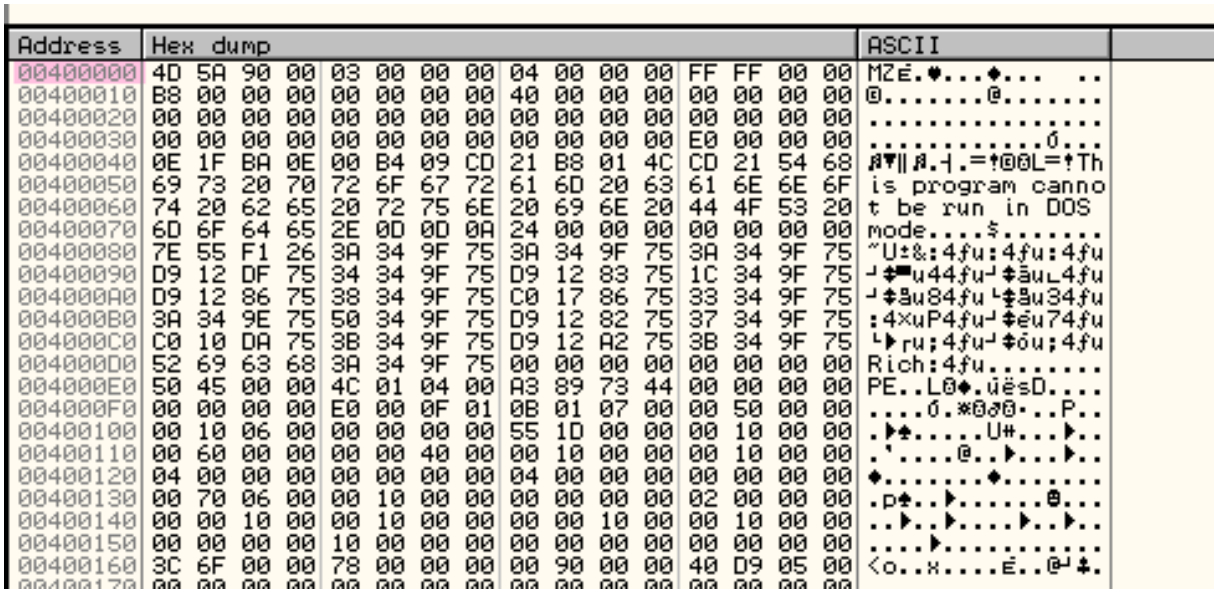
Первая настройка приводит к остановка на SYSTEM BREAKPOINT, и дело в том, что некая DLL запускается раньше того, как управление доходит до точки входа, обнаруживает, что программа выполняется под отладчиком, и завершает её.

Конечно, нам не нужно долго гадать, кто за это ответственен, так как системные библиотеки не защищают программы против отладки под OllyDbg.

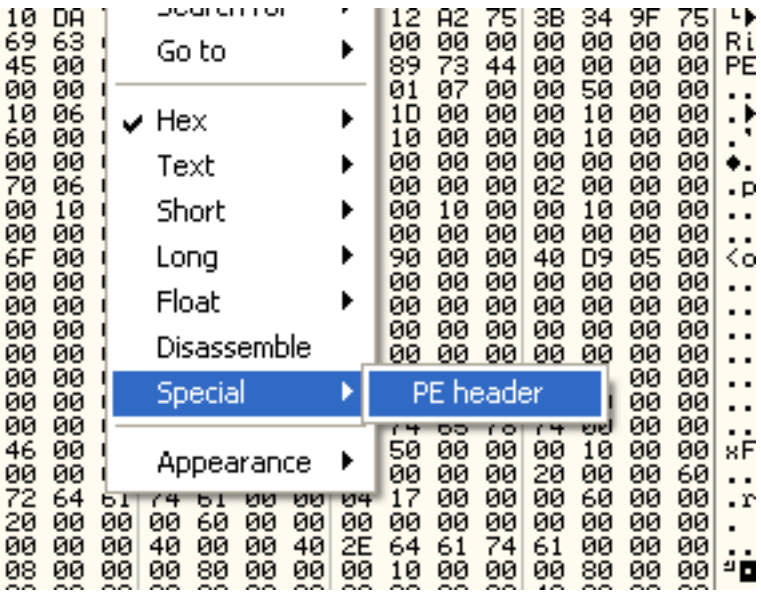
В любом случае, хорошо знать, что все DLL запускаются раньше прибытия в точку входа. Те, через которые проходит выполнение – это нормальные библиотеки и на нас не влияют, и поскольку OllyDbg в них не останавливается, то нам они особенно не интересны. Но посмотрим в программу.

Идём в заголовок с помощью GOTO EXPRESSION-400000.



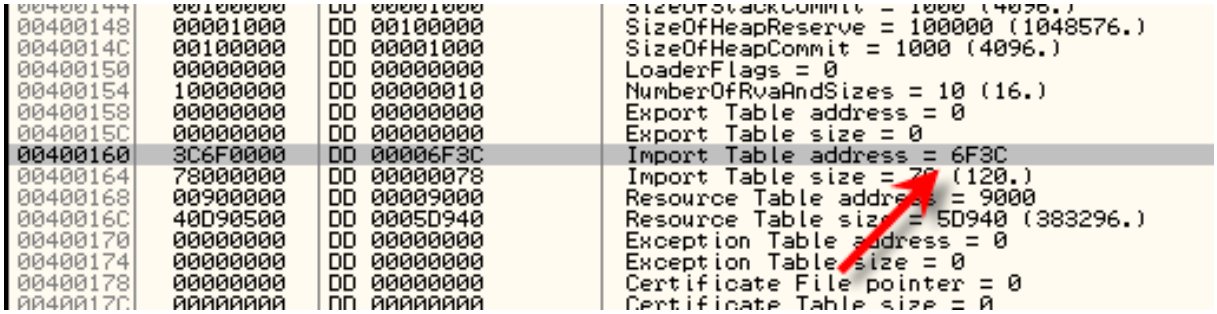


Меняем режим на SPECIAL- PE HEADER.



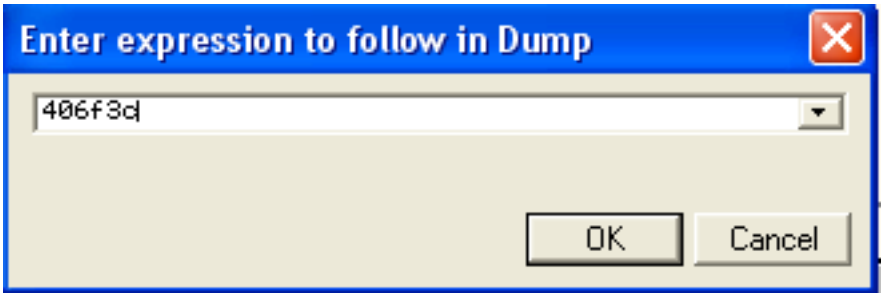
Address	Hex dump	Data	Comment
00400000	4D 5A	ASCII "MZ"	DOS EXE Signature
00400002	9000	DW 0090	DOS_PartPag = 90 (144.)
00400004	0300	DW 0003	DOS_PageCnt = 3
00400006	0000	DW 0000	DOS_ReloCnt = 0
00400008	0400	DW 0004	DOS_HdrSize = 4
0040000A	0000	DW 0000	DOS_MinMem = 0
0040000C	FFFF	DW FFFF	DOS_MaxMem = FFFF (65535.)
0040000E	0000	DW 0000	DOS_ReloSS = 0
00400010	B800	DW 00B8	DOS_ExeSP = B8
00400012	0000	DW 0000	DOS_ChkSum = 0
00400014	0000	DW 0000	DOS_ExeIP = 0
00400016	0000	DW 0000	DOS_ReloCS = 0
00400018	4000	DW 0040	DOS_TablOff = 40
0040001A	0000	DW 0000	DOS_Overlay = 0
0040001C	00	DB 00	
0040001D	00	DB 00	
0040001E	00	DB 00	
0040001F	00	DB 00	
00400020	00	DB 00	

Теперь спускаемся вниз и видим следующий указатель.

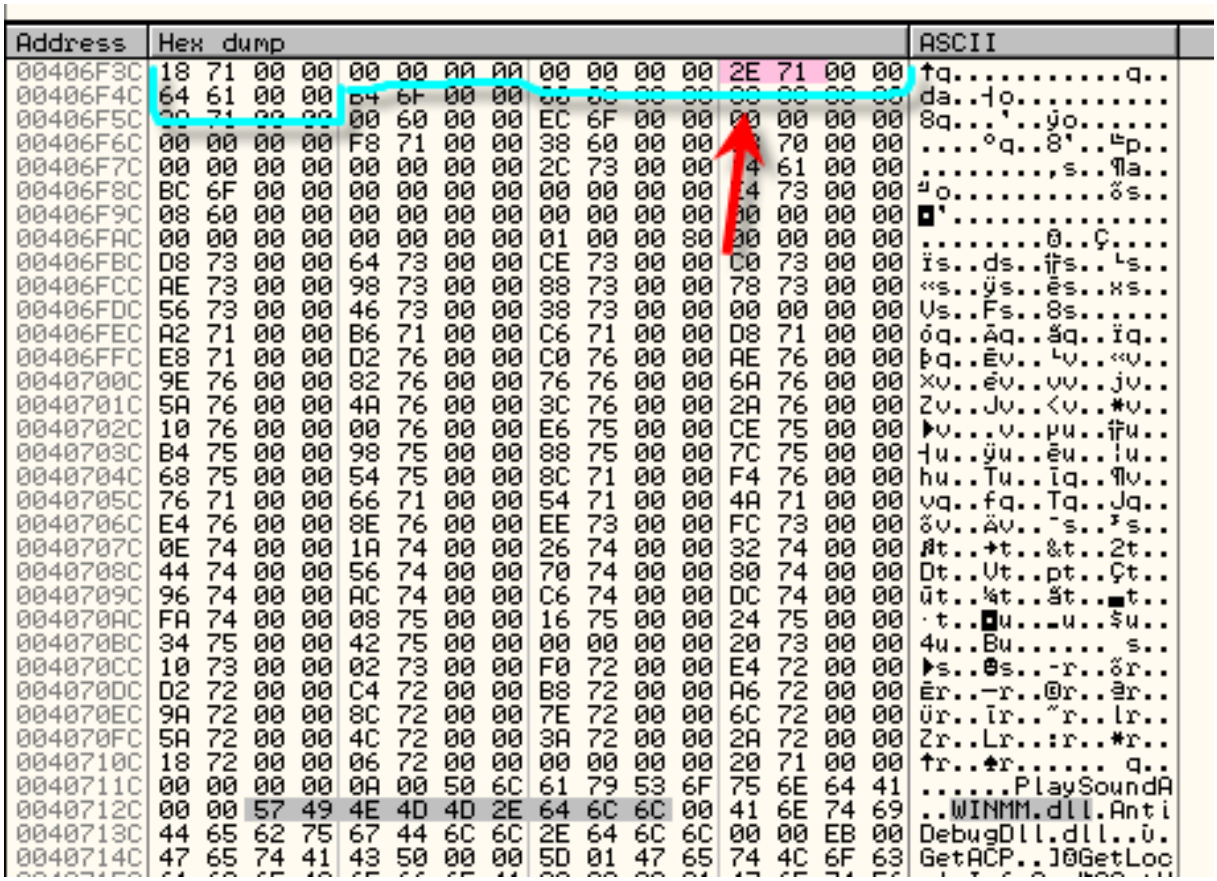


Как мы знаем, это указатель на IMPORT TABLE или IT (не путайте с IMPORT ADDRESS TABLE, которая IAT, к сожалению, это делают довольно многие, хе-хе).

Ок, как мы уже видении, если пойдём сюда в 6F3C + база образа (400000), то увидим IT.

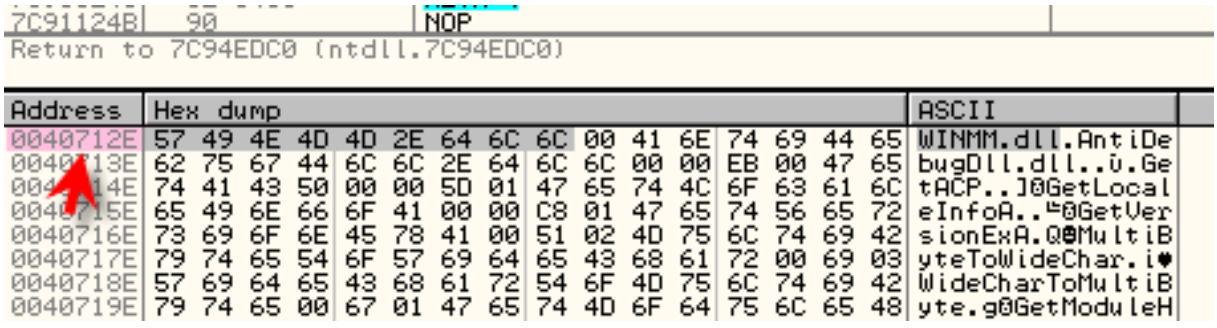


Выходим из специального режима.



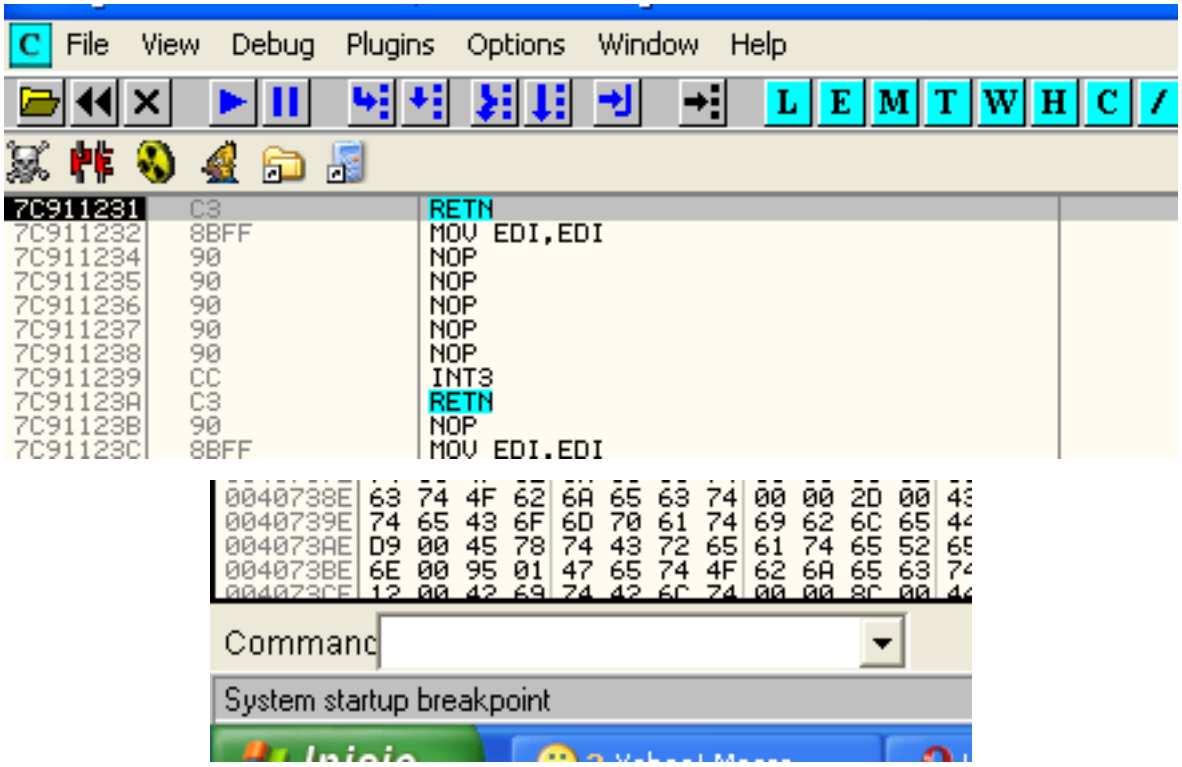
Если помним, первые пять DWORD'ов соответствуют первой DLL, следующие пять – второй, и так в порядке, в котором они загружаются в IAT.

Чтобы узнать имя первой DLL, нужен 4-ый DWORD, который является указателем на имя, то есть по адресу 407712e будет находится имя первой DLL, загружаемой в IAT. Посмотрим, что это за библиотека.



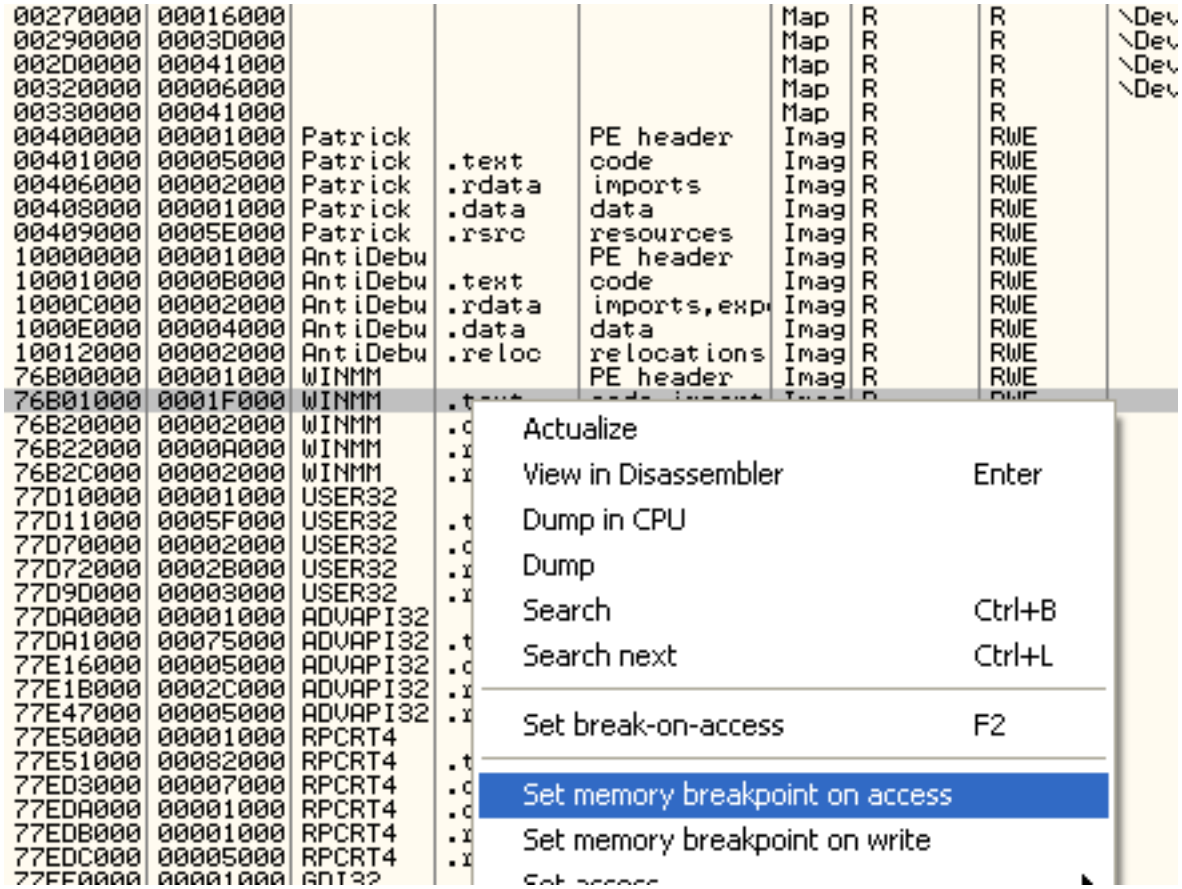
Там находится WINMM.dll – это первая DLL. Но запускается ли она раньше прибытия в точку входа, в которой останавливается OllyDbg?

Открываем её и то место, где сработал SYSTEM BREAKPOINT.

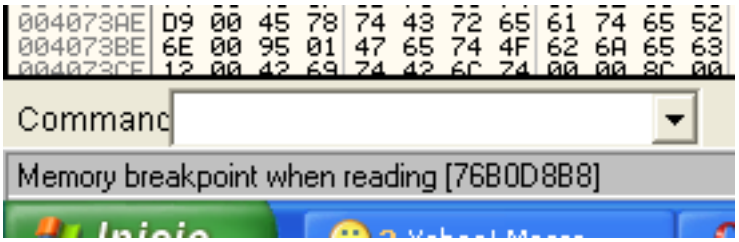


Останавливаемся здесь. Теперь знаем, что этот крэкми не доходит до EP, потому что завершается раньше в какой-то из DLL, который загружаются до неё. Это, не иначе как, происходит в библиотеке, прилагающейся к крэкми, но всё равно посмотрим, какие библиотеки запускаются раньше OEP, так что идём в карту памяти M и устанавливаем BPM ON ACCESS на секцию CODE вышеупомянутой DLL.

00290000	0003D000				Map	R	R	\Device\Hard
002D0000	00041000				Map	R	R	\Device\Hard
00320000	00006000				Map	R	R	\Device\Hard
00330000	00041000				Map	R	R	
00400000	00001000	Patrick		PE header	Imag	R	RWE	
00401000	00005000	Patrick	.text	code	Imag	R	RWE	
00406000	00002000	Patrick	.rdata	imports	Imag	R	RWE	
00408000	00001000	Patrick	.data	data	Imag	R	RWE	
00409000	0005E000	Patrick	.rsrc	resources	Imag	R	RWE	
10000000	00001000	AntiDebu		PE header	Imag	R	RWE	
10001000	0000B000	AntiDebu	.text	code	Imag	R	RWE	
1000C000	00002000	AntiDebu	.rdata	imports,exp	Imag	R	RWE	
1000E000	00004000	AntiDebu	.data	data	Imag	R	RWE	
10012000	00002000	AntiDebu	.reloc	relocations	Imag	R	RWE	
76B00000	00001000	WINMM		PE header	Imag	R	RWE	
76B01000	0001F000	WINMM	.text	code,import	Imag	R	RWE	
76B20000	00002000	WINMM	.data	data	Imag	R	RWE	
76B22000	0000A000	WINMM	.rsrc	resources	Imag	R	RWE	
76B2C000	00002000	WINMM	.reloc	relocations	Imag	R	RWE	
77D10000	00001000	USER32		PE header	Imag	R	RWE	
77D11000	0005F000	USER32	.text	code,import	Imag	R	RWE	
77D70000	00002000	USER32	.data	data	Imag	R	RWE	
77D72000	0002B000	USER32	.rsrc	resources	Imag	R	RWE	

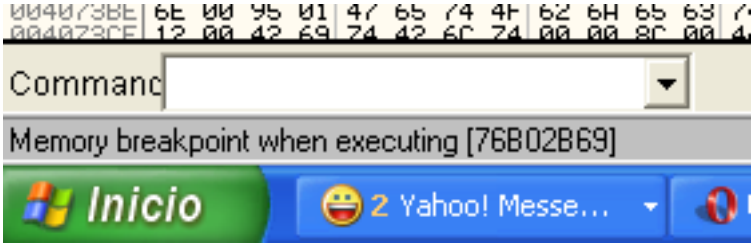


Ок, устанавливаем точку останова сюда, чтобы посмотреть, сработает ли она, делаем RUN несколько раз и смотрим в низ OllyDbg, фиксируя, происходит ли выполнение или чтение/запись. Продолжаем нажимать RUN.

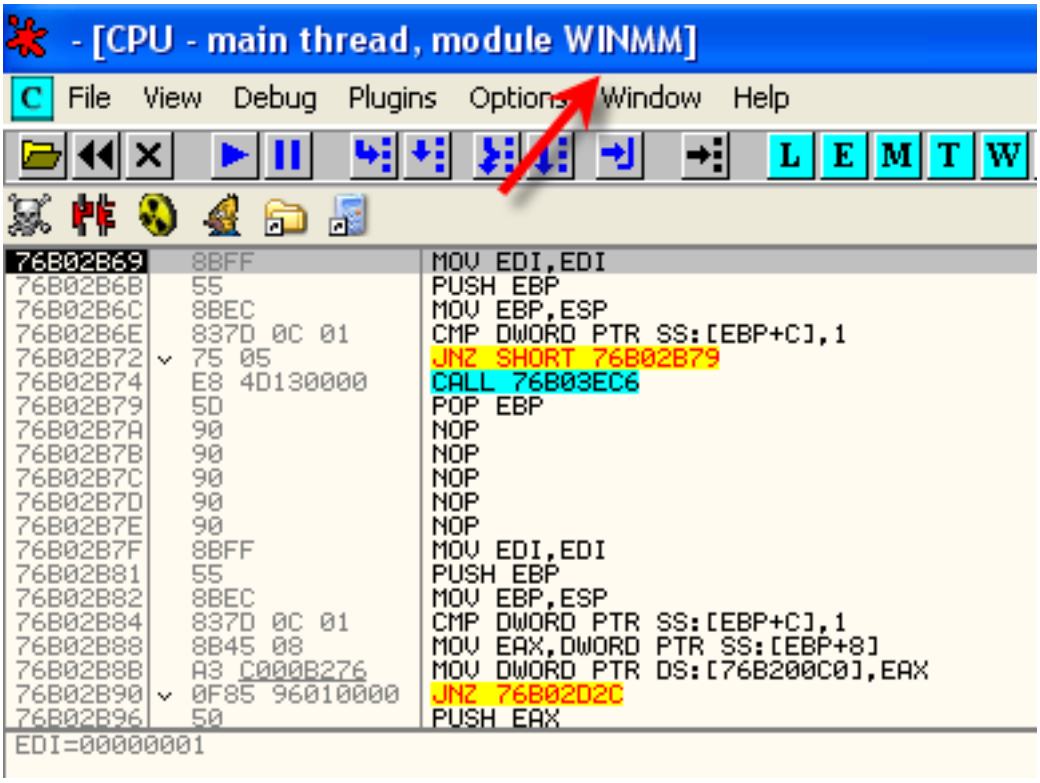


.

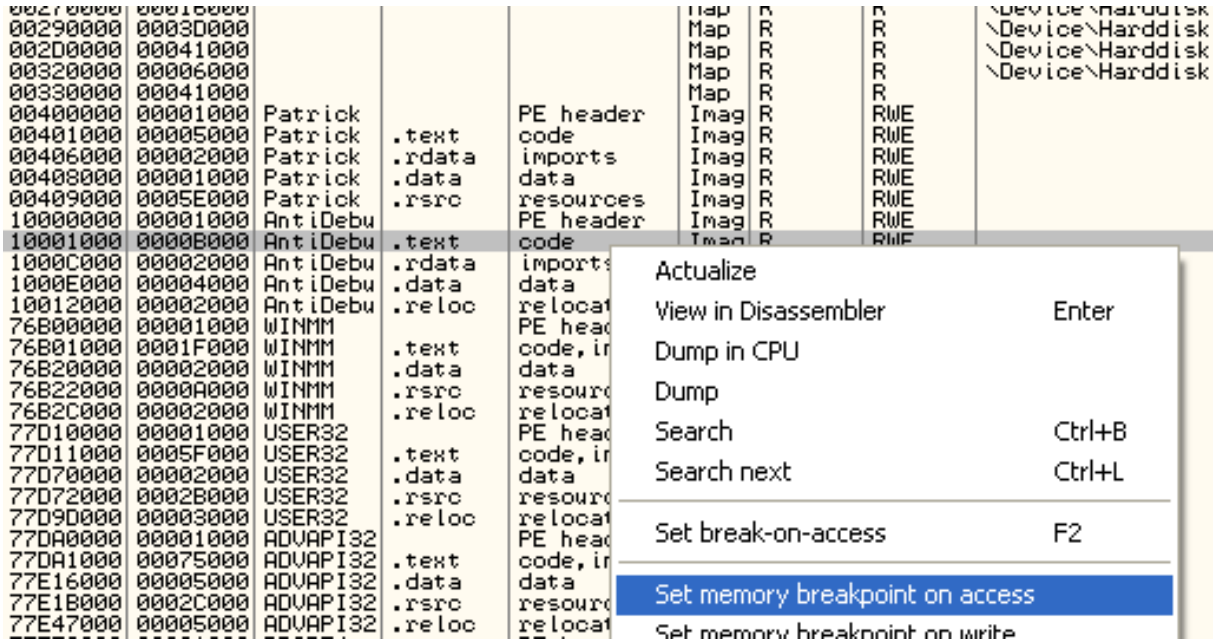
В первый раз, когда останавливаемся здесь, происходит чтение. Продолжаем, пока не произойдёт выполнение.



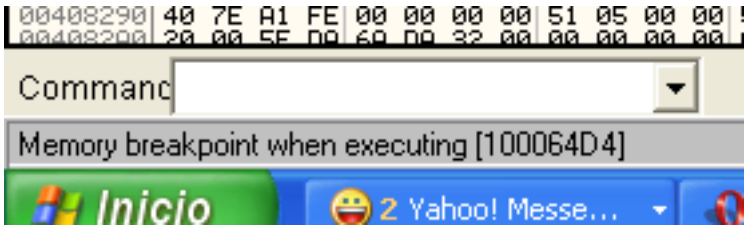
Останавливаемся здесь, когда начинается выполнение DLL.

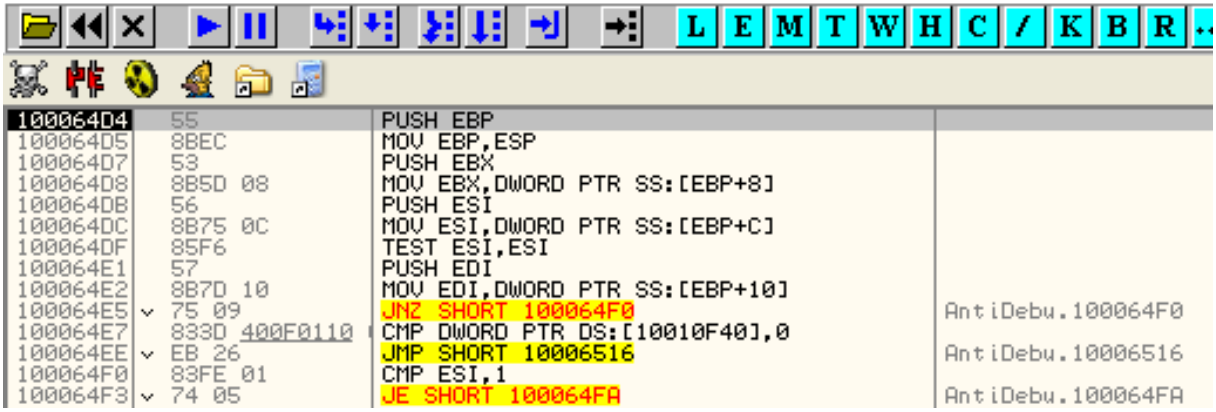


Это значит, что ничего странного в том, что эти DLL выполняются, нет. Теперь повторим это способ, но с DLL, которую хотим исследовать. Пеерзапустим OllyDbg и когда остановимся, то идём в M и устанавливаем BPM ON ACCESS на секцию кода в AntidebugDll.dll.



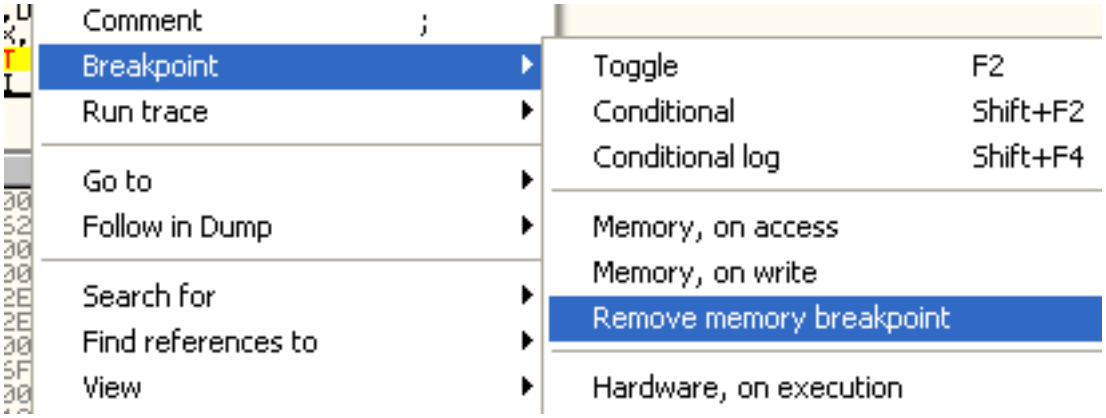
Делаем RUN и останавливаемся на выполнении DLL.



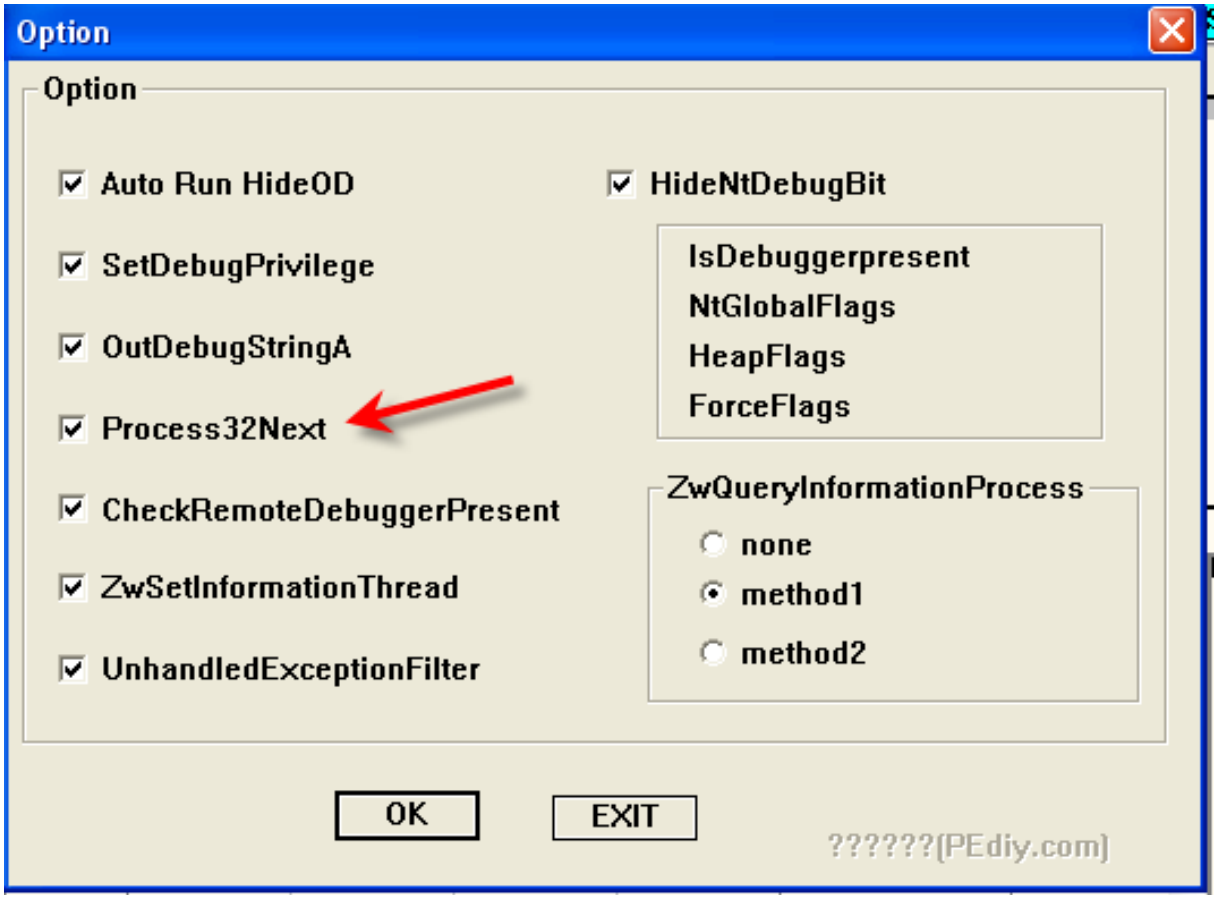


Хе-хе, тут начинаются проблемы.

Нам нужно, чтобы программа работала под OllyDbg, поэтому прежде всего посмотрим кое-что.

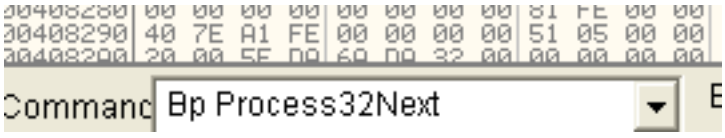


Хорошо, убираем BPM, установленный ранее, и так программа обнаруживает OllyDbg не по имени, не по окну, не с помощью всех иных приёмов, против которых служит HIDEOD, то заключаем, что где-то просматривается процесс, который открывает программу, а для этого, как мы уже знаем, служит приём с Process32Next, но HIDEOD, предположительно, также защищает против него. Что же здесь происходит?

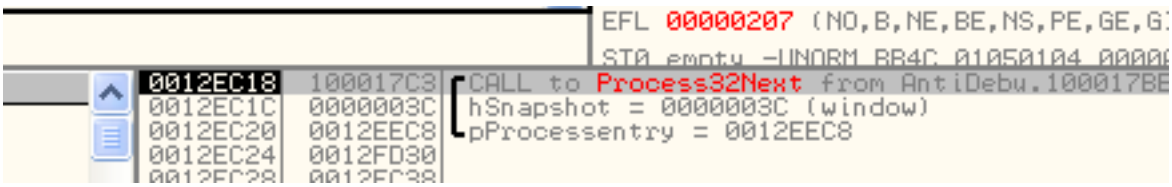


Таким образом, если сделать несколько дополнительных исследований, можно придти к заключению, что крэкми не работает, если запускается не из-под EXPLORER.EXE, то есть нужно, чтобы оно нормально запускалось из-под него двойным щелчком мыши, а если запустим из-под любой другой программы, то он работать не будет.

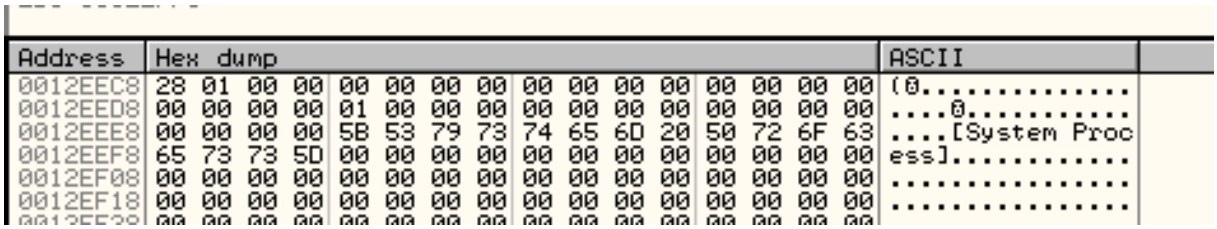
То, из-под какого процесса он запустился, крэкми узнаёт с помощью API-функции Process32Next, так что устанавливаем BP на него и смотрим.



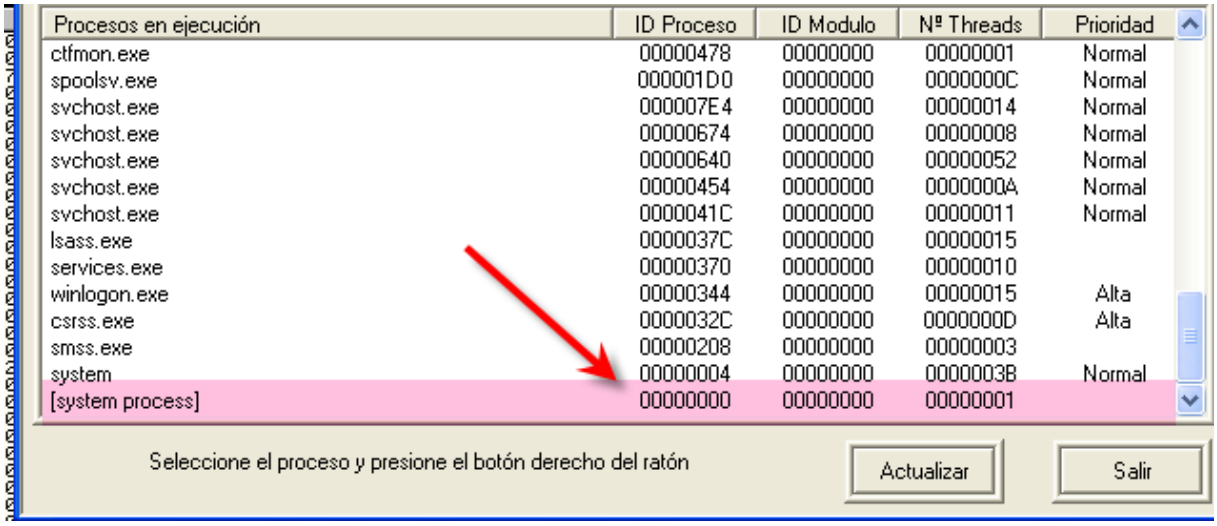
Делаем RUN.



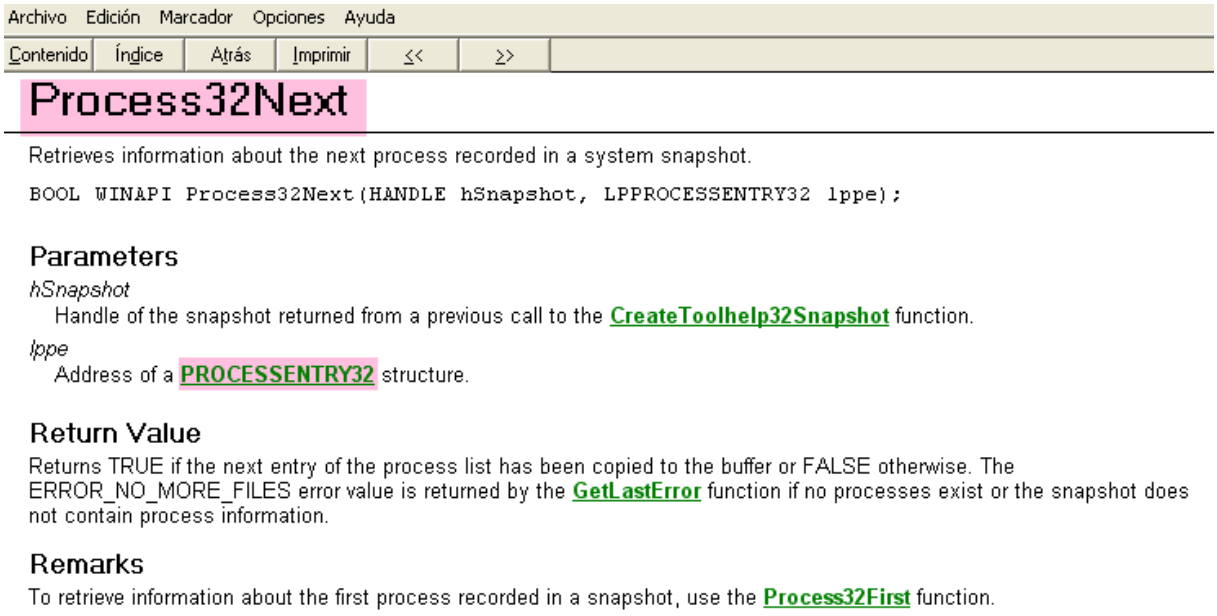
Здесь у нас результат работы функции, на моей машине он равен 12EEC8, и это адрес на отчёт.



Используем утилиту Pupe, которая нам покажет процессы по порядку и их PID'ы, чтобы продвинуться дальше.



Первый процесс здесь имеет PID равный нулю, и если глянем в WINAPI32:



Вот эта API-функция Process32Next и видим, что этот параметр, указывающий на отчёт, здесь называется структурой PROCESSENTRY32, если кликнем на ссылку, то узнаем о ней подробнее.

PROCESSENTRY32

```
typedef struct tagPROCESSENTRY32 {
    DWORD dwSize;
    DWORD cntUsage;
    DWORD th32ProcessID;
    DWORD th32DefaultHeapID;
    DWORD th32ModuleID;
    DWORD cntThreads;
    DWORD th32ParentProcessID;
    LONG  pcPriClassBase;
    DWORD dwFlags;
    char szExeFile[MAX_PATH];
} PROCESSENTRY32;
typedef PROCESSENTRY32 *  PPROCESSENTRY32;
typedef PROCESSENTRY32 *  LPPROCESSENTRY32;
```

Members

dwSize
Specifies the length, in bytes, of the structure. Before calling the **Process32First** function, set this member to sizeof(PROCESSENTRY32).

cntUsage
Number of references to the process. A process exists as long as its usage count is nonzero. As soon as its usage count becomes zero, a process terminates.

th32ProcessID
Identifier of the process. The contents of this member can be used by Win32 API elements.

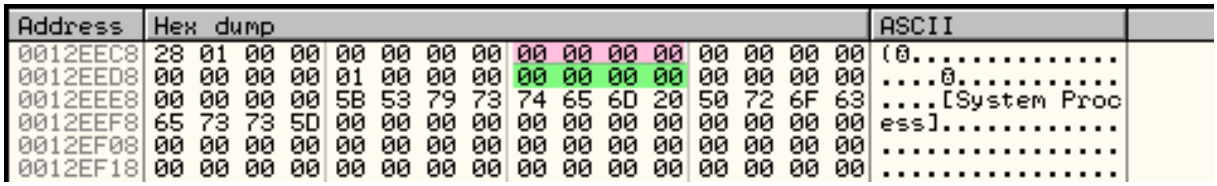
th32DefaultHeapID
Identifier of the default heap for the process. The contents of this member has meaning only to the tool help functions. It is not a handle, nor is it usable by Win32 API elements.

th32ModuleID
Module identifier of the process. The contents of this member has meaning only to the tool help functions. It is not a handle, nor is it usable by Win32 API elements.

cntThreads
Number of execution threads started by the process.

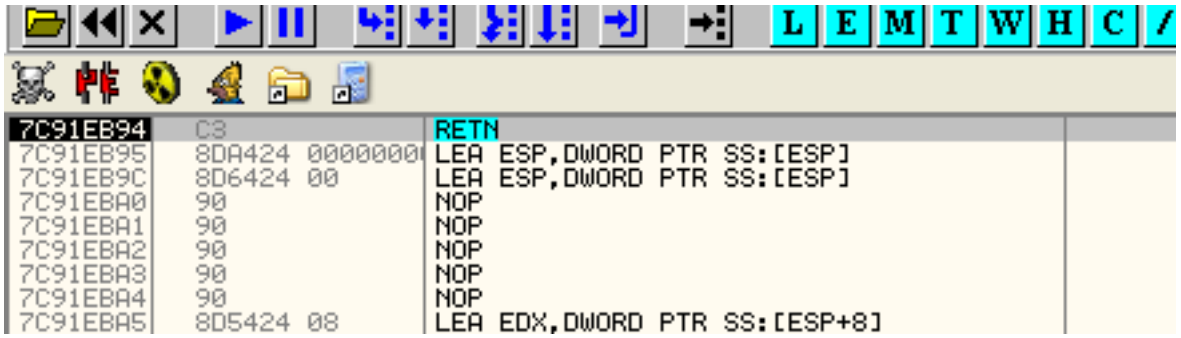
th32ParentProcessID
Identifier of the process that created the process being examined. The contents of this member can be used by Win32 API elements.

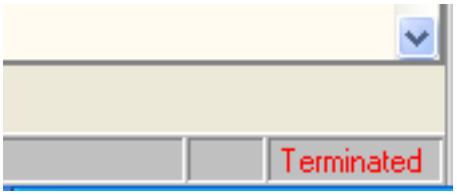
Здесь видим, что 3-ий DWORD – это PID изучаемого процесса, а 7-ой – это PID процесса, который его запустил, в данном случае оба равны нулю, то есть оба были запущенны системой.



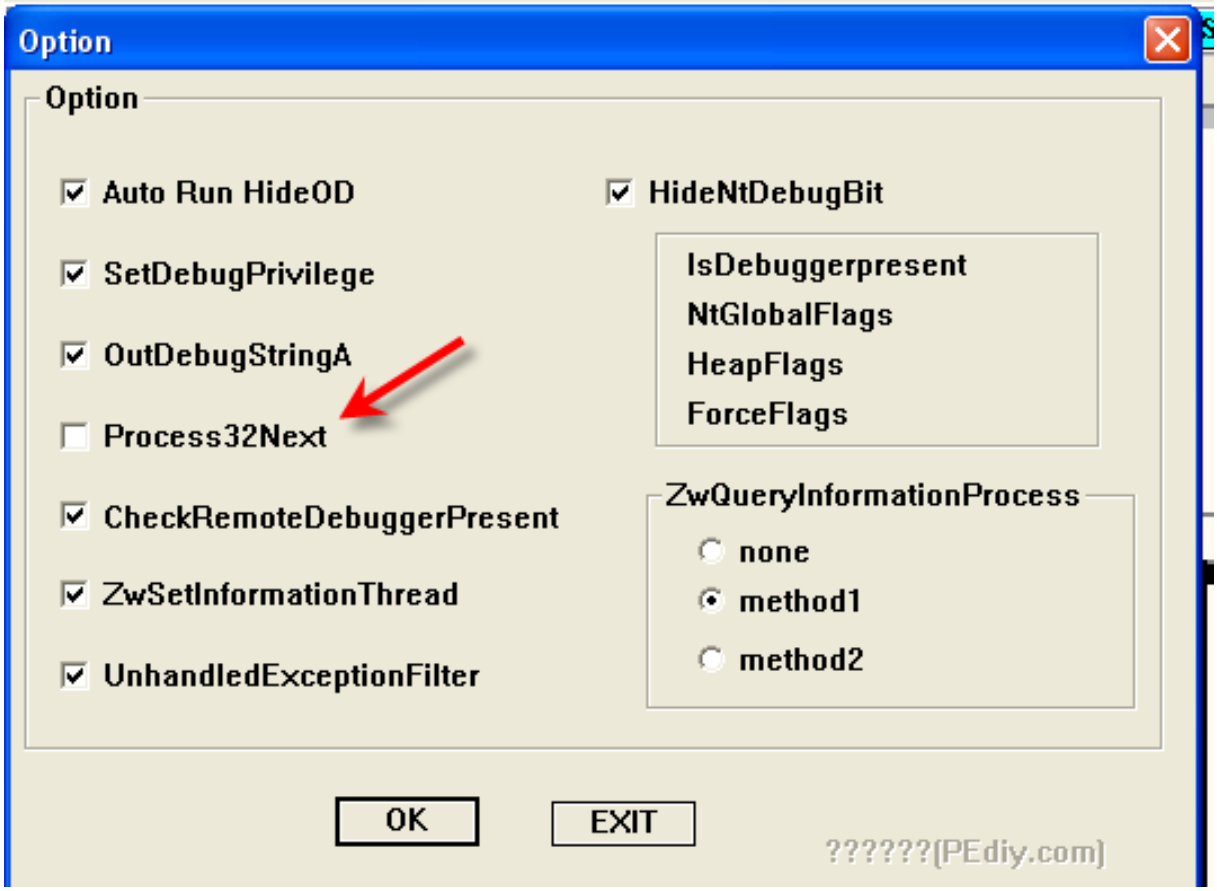
Address	Hex dump	ASCII
0012EEC8	28 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00	(0..............
0012EED8	00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 00	....0...........
0012EEE8	00 00 00 00 5B 53 79 73 74 65 6D 20 50 72 6F 63	....[System Proc
0012EEF8	65 73 73 5D 00 00 00 00 00 00 00 00 00 00 00 00	ess]............
0012EF08	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
0012EF18	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....

Розовым выделен PID процесса и зелёным – кто его запустил. Ничего особенного здесь нет, делаем RUN заново.

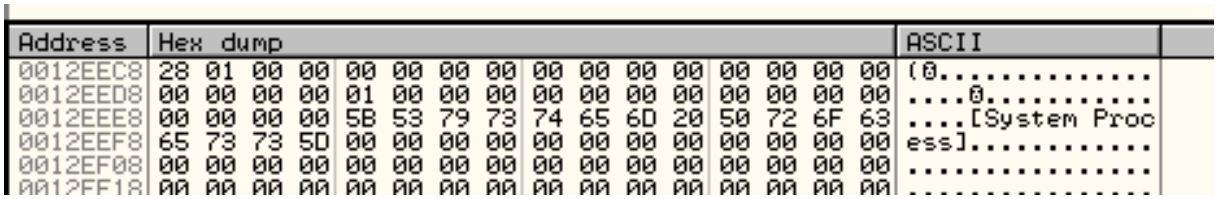




Эй, уже прекратила работу, но только изучая системный процесс, определить в чём причина закрытия нельзя. Попробуем отключить защиту HIDEOD, чтобы не возникали никакие конфликты. Снимаем в HIDEOD галочку, относящуюся к Process32Next и повторяем:

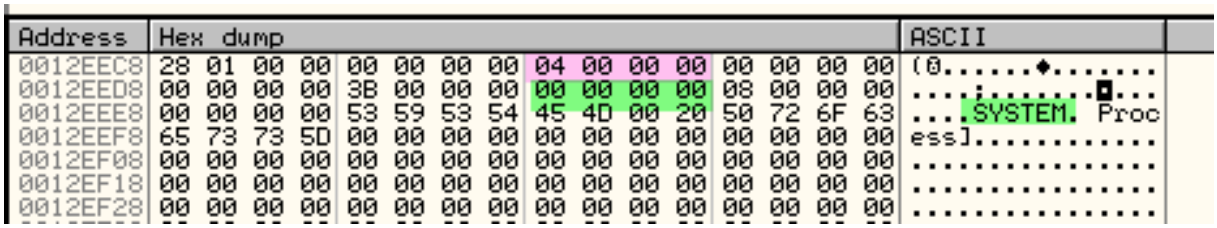


Теперь при перезапуске не нужно останавливать BPM заново на секцию кода, потому что можно использовать BP на API-функцию.

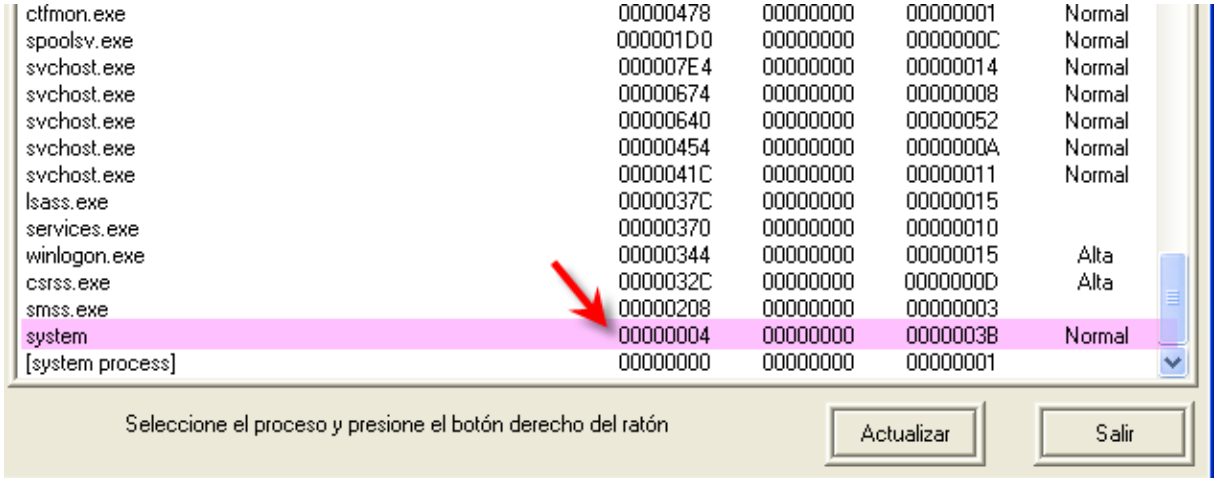


Теперь останавливаемся и смотри отчёт, в котором раньше было два PID’а равных нулю.

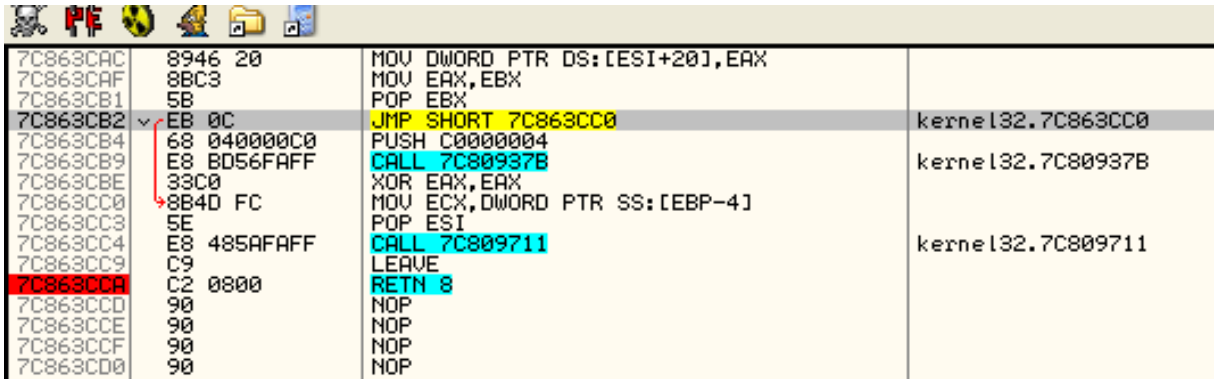
Аа, теперь понятна проблема. Когда в HIDEOD была установлена галочка на Process32Next, то по возвращению из API-функции EAX содержал ноль, что говорит о произошедшей ошибке при её выполнении, и программа сразу закрывалась. Без галочки EAX равен одному, и тогда начинается поиск процесса, так что мы теперь знаем, что этот случай надо решать без данной опции.



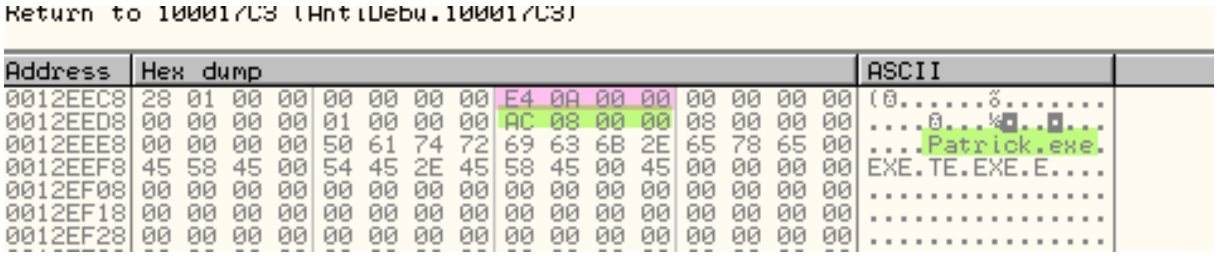
Ок, без галочки мы можем дойти до второго процесса в снимке процессов. Им является SYSTEM с PID'ом 4, который отмечен розовым цветом. Этот процессом был запущен процессом с нулевым PID'ом, то есть предыдущим.



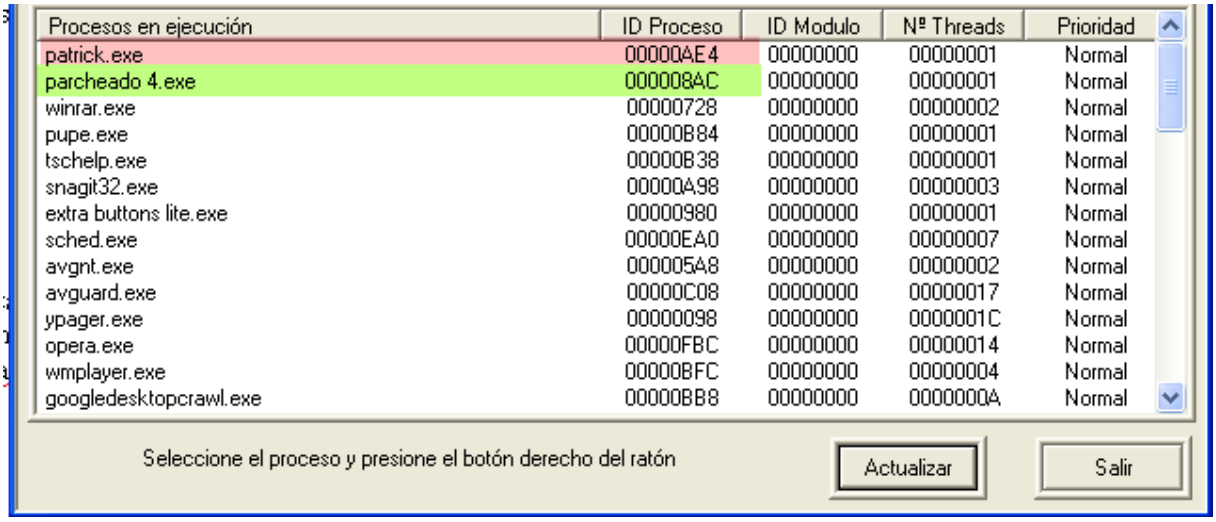
Хорошо, нам будет удобнее снять BP с начала API-функции и установить его на её RET, чтобы точно узнать, когда возвращается отчёт PATRICK'а при его завершении, и изменить его до того, как программа его прочтёт.



Теперь подолжим до того, как в PATRICK.exe появится отчёт.

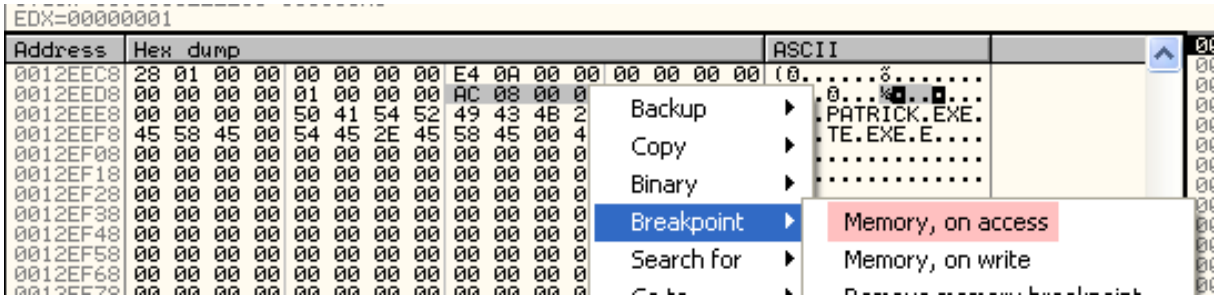


Здесь видим PID patrick'a, который равен AE4, и он был запущен процессом с PID'ом 8AC. Смотрим в PUPE, вспоминаем, что если перезапускали крэкми, то нужно обновить информацию в этой утилите, так что нажимаем кнопку "Actualizar".

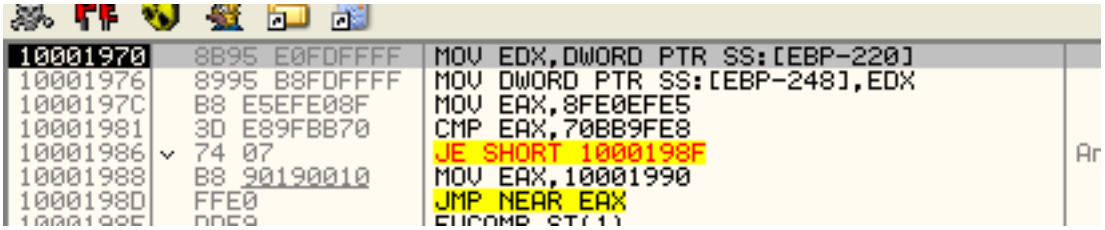


Patrick с PID'ом AE4 был запущен parcheado4 с PID'ом 8AC.

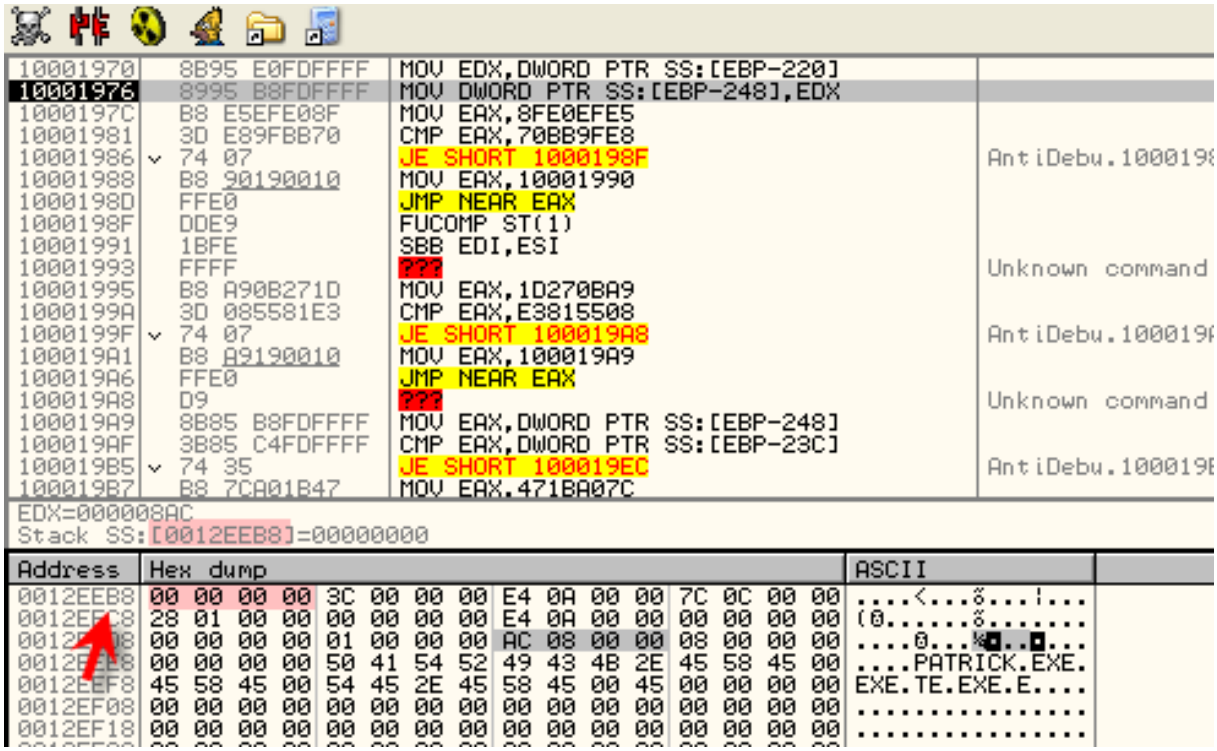
Вот истинная причина, почему крэкми, запущенный не из-под EXPLORER.exe завершает работу. Устанавливаем BPM ON ACCESS на PID parcheado 4, чтобы узнать, где происходит сравнение.



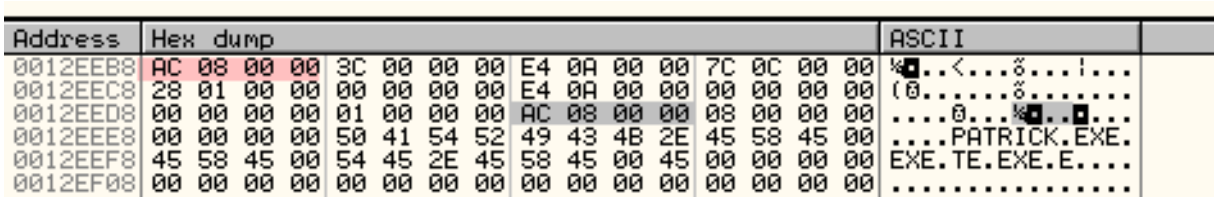
Делаем RUN и останавливаемся здесь.



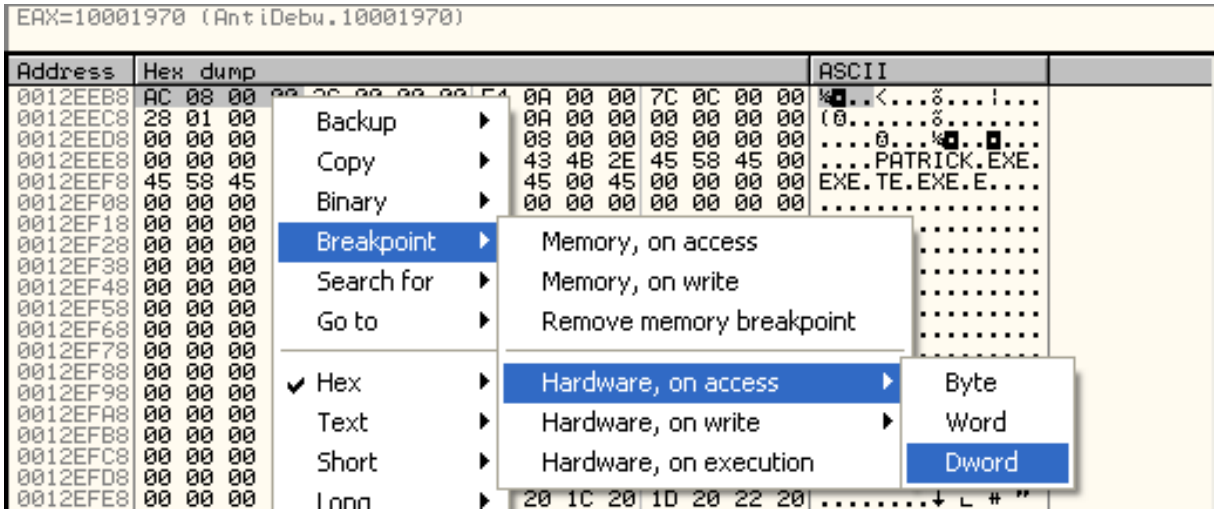
Где считывается PID и сохраняется в другое место. Смотрим, куда имеет, чтобы можно было установить туда HBP ON ACCESS.



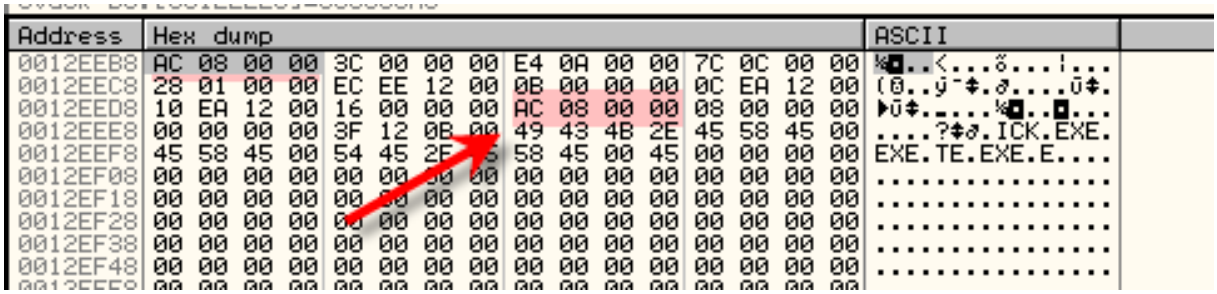
Нажимаем F7, чтобы сохранить по другому адресу.



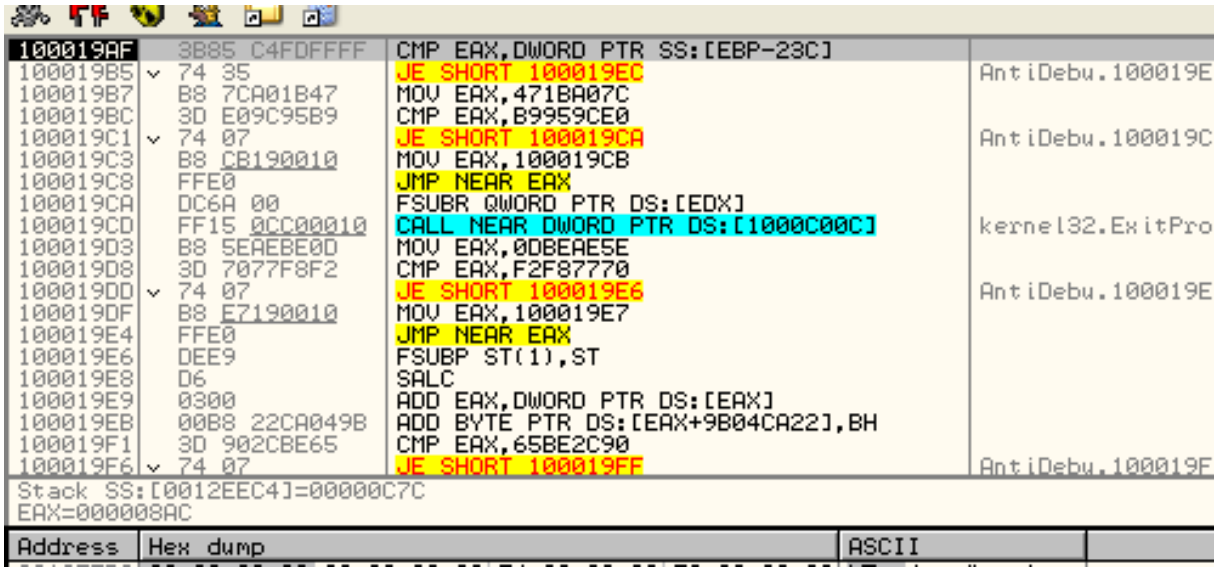
Так что установим HBP ON ACCESS туда и будем наблюдать за тем, когда там произойдёт чтение.



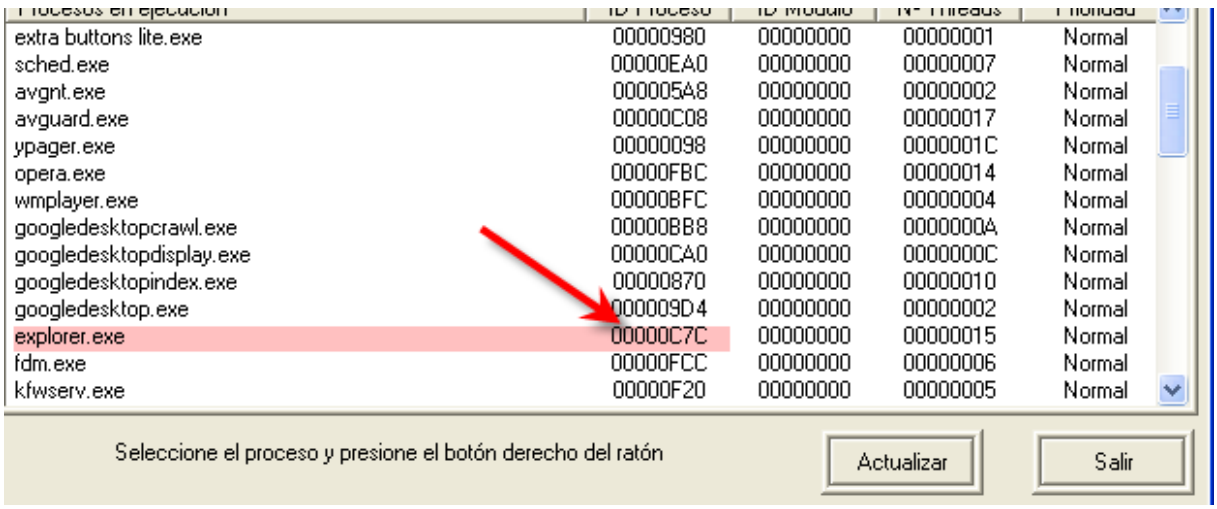
Если нажмём RUN, то увидим, что начальное значение отчёта стирается, но сохраняется куда-то, а куда именно мы должны выяснить с помощью HBP.



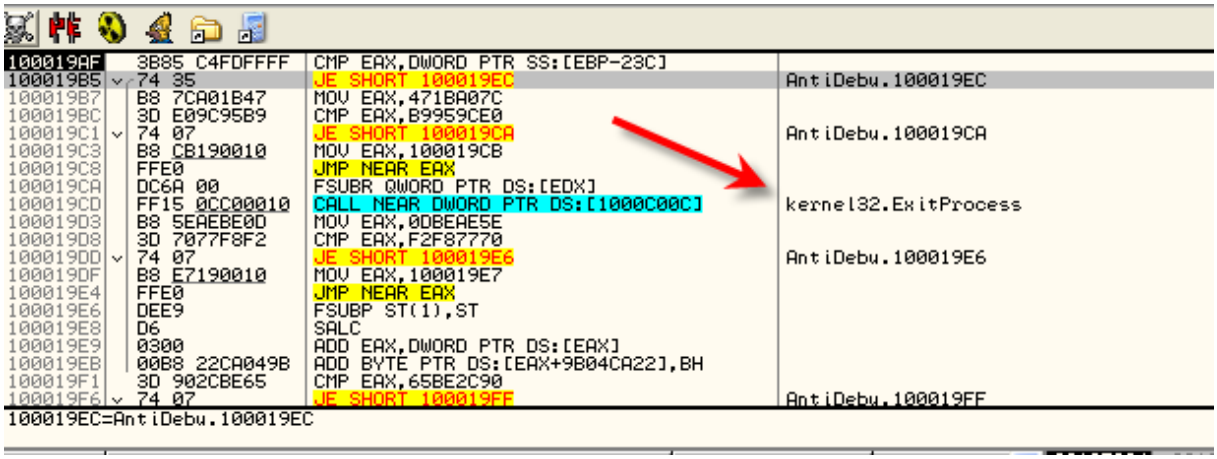
По нажатию на F7 останавливаемся на BPM ON WRITE, где оно стирается, что означает, что нам здесь BPM уже не нужен, так как уникальное значение защищается HBP в другом месте. Снимаем BPM.



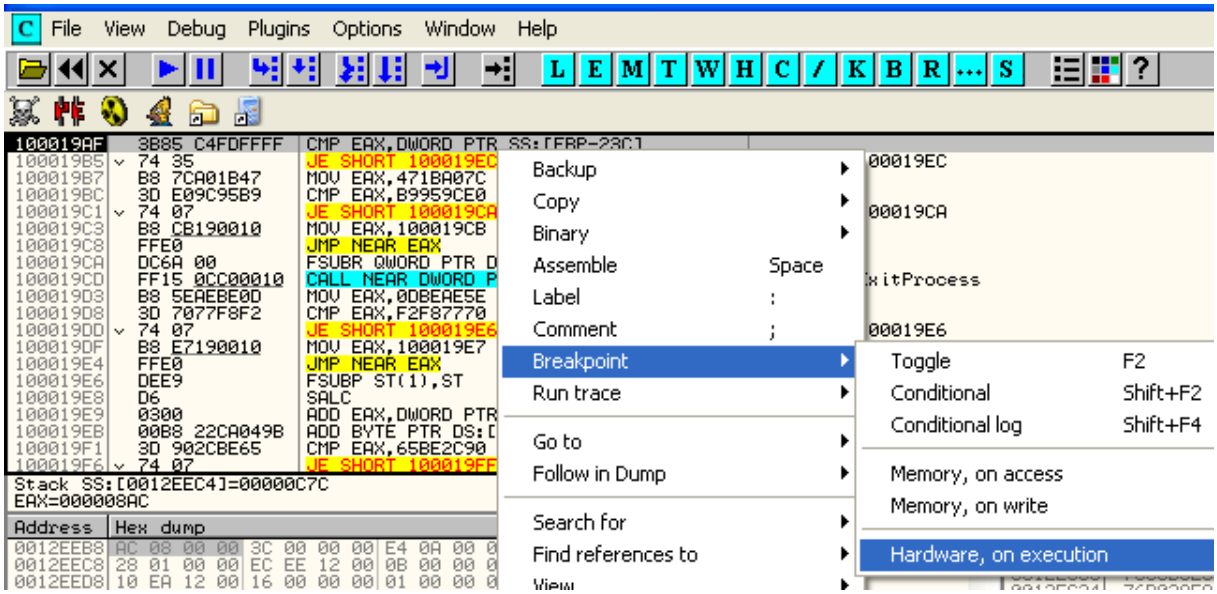
Делаем снова RUN и доходим до сравнения – здесь сравнивается 08AC, что является PID'ом parcheado4 или программы, которая запустила крэкми, и сравнивается он с C7C.



Как и можно было предположить, это PID Explorer'a.

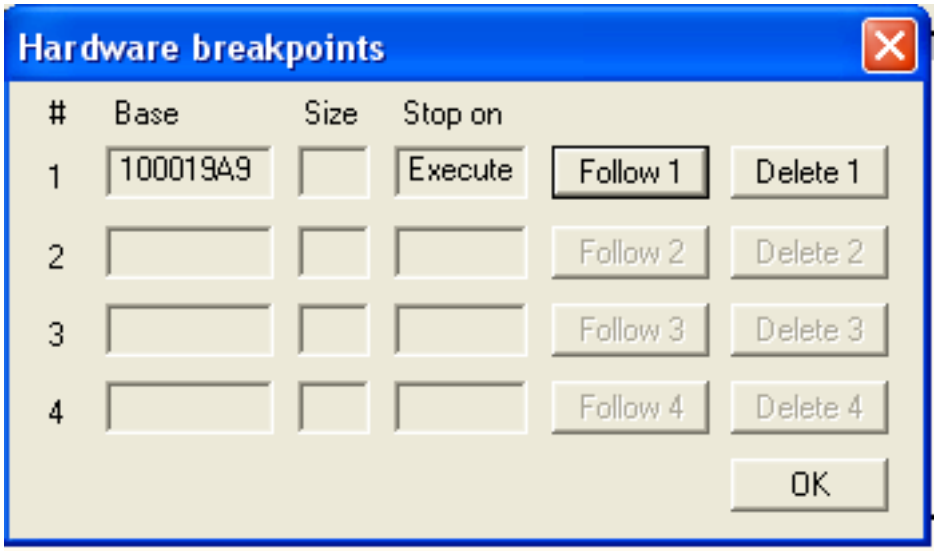


Видим, что если PID'ы запустившей крэкми программы и explorer'а равны, то происходит переход, чтобы обойти ExitProcess, а если различны, то перехода не происходит и крэкми завершает выполнение. Проблема в том, что если изменить DLL и сохранить эти изменения, то в ней есть другие проверки, из-за которых она не запустится. Так что лучшее, что мы можем сделать, чтобы не менять ничего – это снять предыдущий HBP и установить HBP ON EXECUTION на эту строку со сравнением.

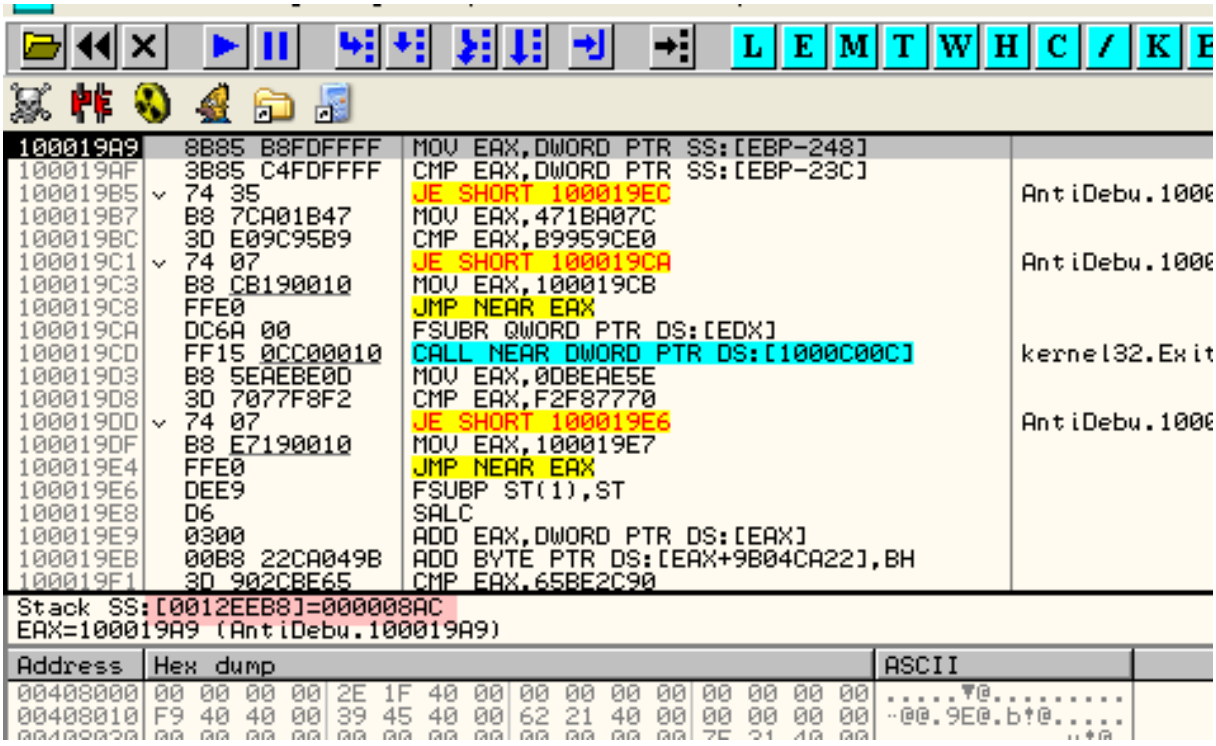


Таким образом, каждый раз, когда будем доходить до сравнения здесь, сможем вручную изменить PID parcheado 4 на PID explorer'а, чтобы они были равны и не было проблем.

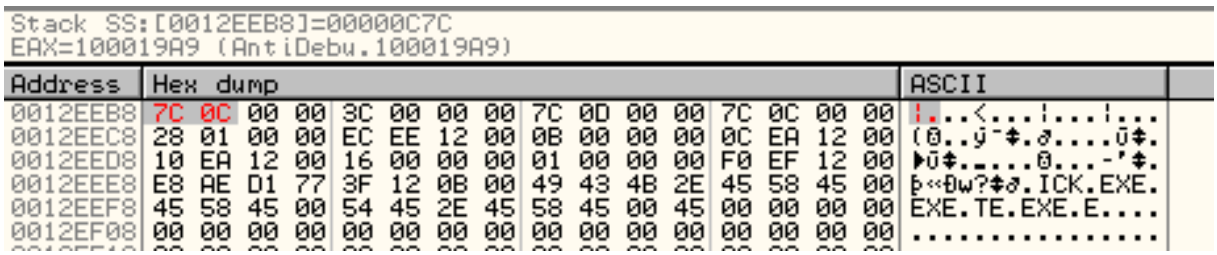
На самом деле будет более удобным установить его строку, предшествующую сравнению, так как именно там считывается PID parcheado 4 и помещается в EAX.



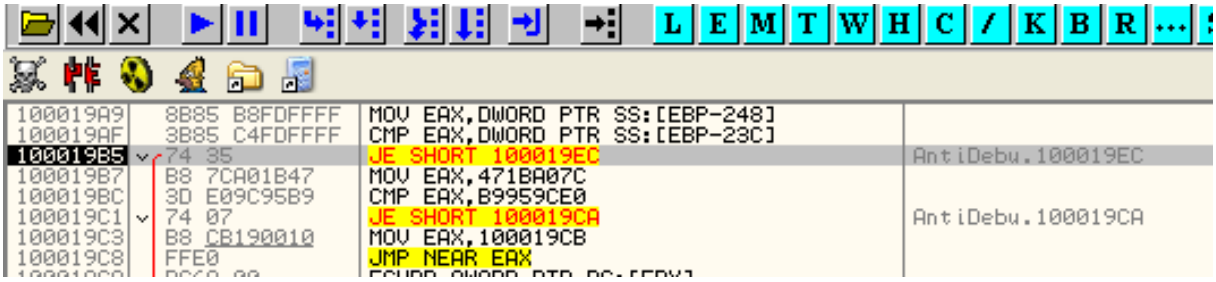
Теперь перезапускаем и смотрим, можем ли мы напрямую отредактировать PID без того, чтобы устанавливать Process32Nex и проходить через API-функции одну за другой. Стираем точки останова и перезапускаем.



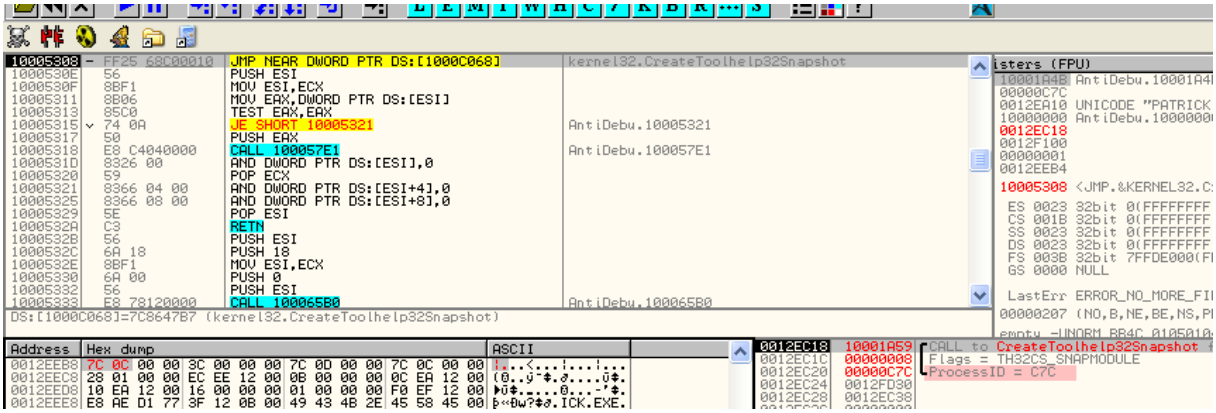
Останавливаемся здесь, и у нас есть информация, где надо редактировать. На моей машине это 12eeb8 – идём туда и меняем PID 8AC на C7C.



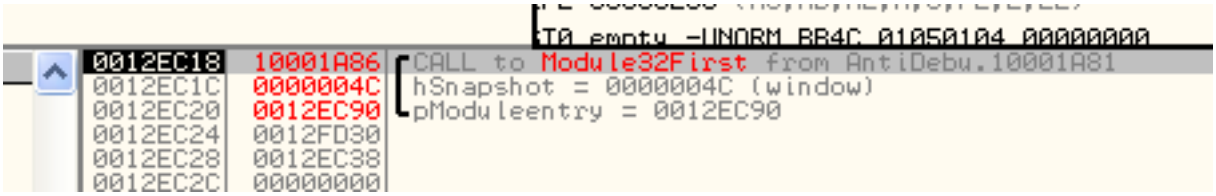
Если теперь нажмём F7, то раз оба PID'а равны – произойдёт переход, и программа продожит своё выполнение.



Это только первый из трюков, которые нам необходимо обойти. С помощью установленного HBP это легко сделав, заменив здесь PID и продолжив выполнение.



Немного оттрасировав отсюда, видим, что теперь крэкми хочет сделать снимок процесса с PID'ом C7C (на моей машине), чтобы убедиться, что это эксплорер, проверив другие его характеристики. Поскольку C7C на самом деле является эксплорером, то ничего странного он не замечается. Этот приём предназначается для противодействия попыткам переименовать другую программу в EXPLORER.exe, чтобы запустить из-под неё крэкми.



И если ещё немного оттрасируем, то увидим Module32First, которая служит для проверки модуля процесса.

```
typedef MODULEENTRY32 *  LPMODULEENTRY32;
```

Members

dwSize
Specifies the length, in bytes, of the structure. Before calling the **Module32First** function, set this member to sizeof(MODULEENTRY32).

th32ModuleID
Module identifier in the context of the owning process. The contents of this member has meaning only to the tool help functions. It is not a handle, nor is it usable by Win32 API elements.

th32ProcessID
Identifier of the process being examined. The contents of this member can be used by Win32 API elements.

GlblcntUsage
Global usage count on the module.

ProccntUsage
Module usage count in the context of the owning process.

modBaseAddr
Base address of the module in the context of the owning process.

modBaseSize
Size, in bytes, of the module.

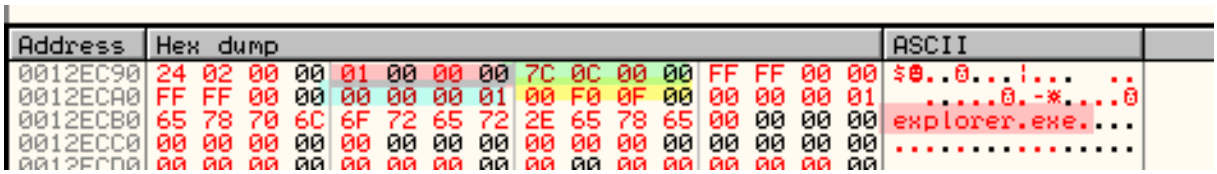
hModule
Handle of the module in the context of the owning process.

szModule
String containing the module name.

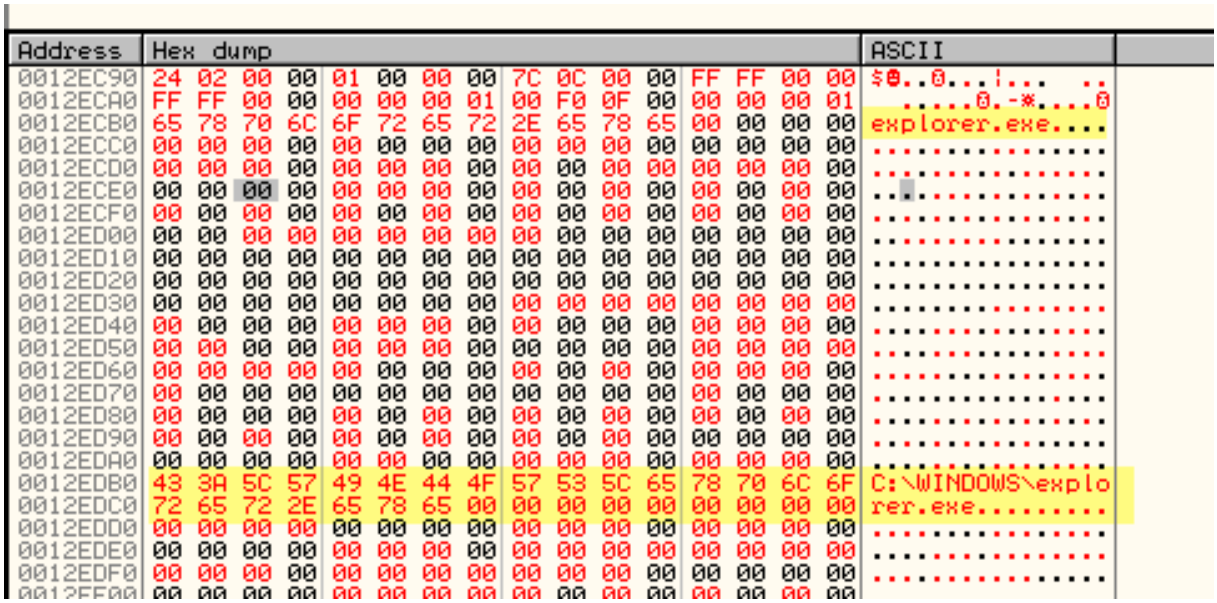
szExePath
String containing the location (path) of the module.

Note **modBaseAddr** and **hModule** are valid *only* in the context of the process specified by **th32ProcessID**.

Ок, здесь у нас отчёт, в данном случае – это информация о модулях. Точно мы не знаем, что проверяется, но так как ищется информация об explorer.exe, то знаем, что всё верно. Здесь мы видим данные о первом модуле, которым является сам исполняемый файл.

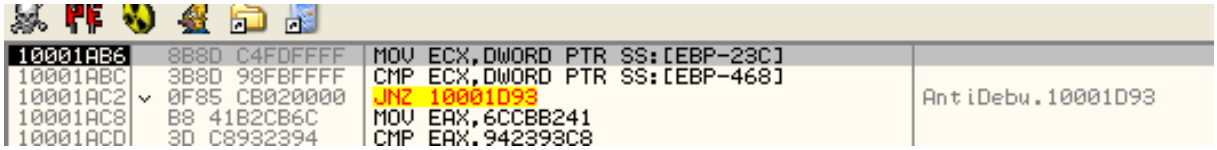


Самые важные данные следующие: роэовым выделен идентификатор 01, который имеет значение только внутри снимка и идентифицирует модули внутри процесса. Этот внутренний номер в снимке, который не слишком важен. Затем зелёным отмечен PID процесса, которому принадлежит модуль, а потом голубым цветом – адрес базы модуля, который в моём случае равен 1000000, то есть это база образа данного модуля. Жёлтым цветом – размер модуля. Кроме того, видим, что показывается имя модуля, а ниже идёт путь к нему.



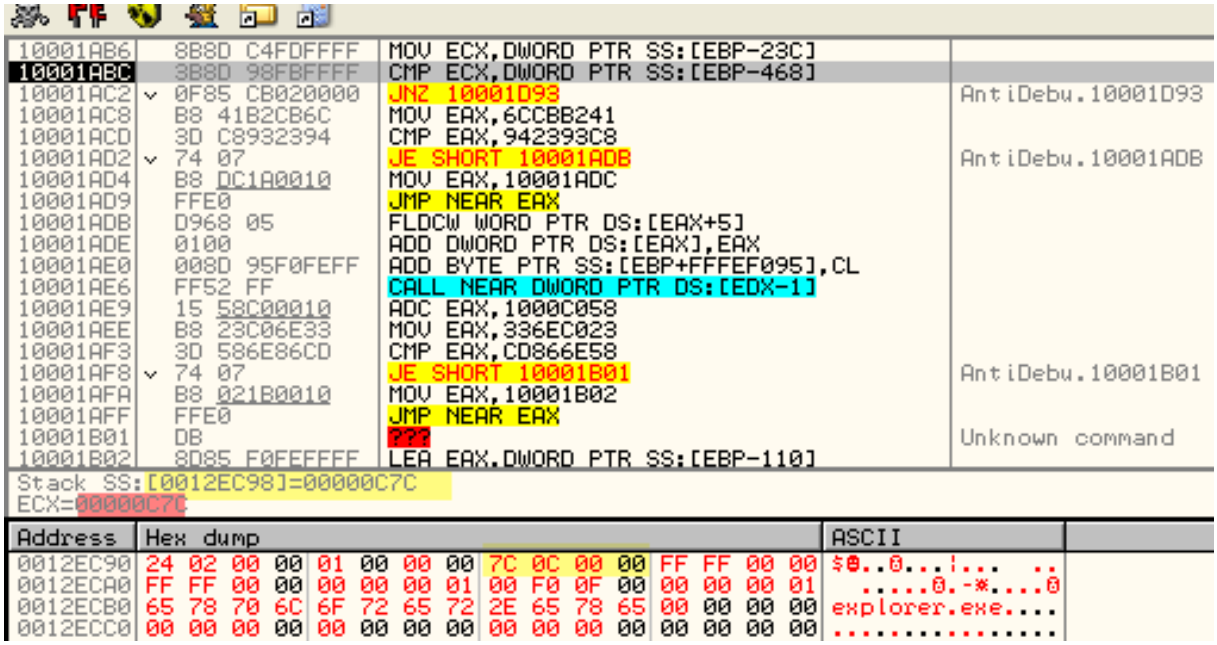
Address	Hex dump	ASCII
0012EC90	24 02 00 00 01 00 00 00 7C 0C 00 00 FF FF 00 00	$☻..0...|... ..
0012ECA0	FF FF 00 00 00 00 00 01 00 F0 0F 00 00 00 00 01	.....0.-*....0
0012ECB0	65 78 70 6C 6F 72 65 72 2E 65 78 65 00 00 00 00	explorer.exe....
0012ECC0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012ECD0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012ECE0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012ECF0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012ED00	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012ED10	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012ED20	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012ED30	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012ED40	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012ED50	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012ED60	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012ED70	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012ED80	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012ED90	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012EDA0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012EDB0	43 3A 5C 57 49 4E 44 4F 57 53 5C 65 78 70 6C 6F	C:\WINDOWS\explo
0012EDC0	72 65 72 2E 65 78 65 00 00 00 00 00 00 00 00 00	rer.exe.....
0012EDD0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012EDE0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012EDF0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0012EE00	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....

Хорошо, это даёт нам достаточно информации, так что посмотрим, что мы с этим сможем сделать.



```
10001AB6   8B8D C4FDFFFF   MOV ECX,DWORD PTR SS:[EBP-23C]
10001ABC   3B8D 98FBFFFF   CMP ECX,DWORD PTR SS:[EBP-468]
10001AC2   0F85 CB020000   JNZ 10001D93                      AntiDebu.10001D93
10001AC8   B8 41B2CB6C     MOV EAX,6CCBB241
10001ACD   3D C8932394     CMP EAX,942393C8
```

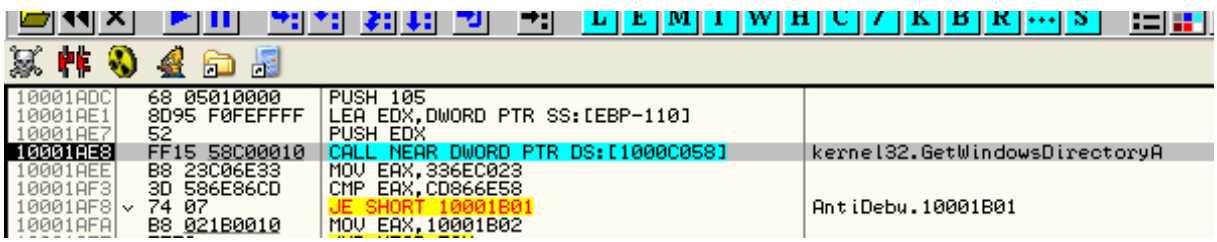
Оттрасировав немного, видим, что считывается C7C и сравнивается, что находится в этом отчёте, чтобы убедиться, что модули также относятся к explorer.exe.



```
10001AB6   8B8D C4FDFFFF   MOV ECX,DWORD PTR SS:[EBP-23C]
10001ABC   3B8D 98FBFFFF   CMP ECX,DWORD PTR SS:[EBP-468]
10001AC2   0F85 CB020000   JNZ 10001D93                      AntiDebu.10001D93
10001AC8   B8 41B2CB6C     MOV EAX,6CCBB241
10001ACD   3D C8932394     CMP EAX,942393C8
10001AD2   74 07           JE SHORT 10001ADB                 AntiDebu.10001ADB
10001AD4   B8 DC1A0010     MOV EAX,10001ADC
10001AD9   FFE0            JMP NEAR EAX
10001ADB   D968 05         FLDCW WORD PTR DS:[EAX+5]
10001ADE   0100            ADD DWORD PTR DS:[EAX],EAX
10001AE0   008D 95F0FEFF   ADD BYTE PTR SS:[EBP+FFFEF095],CL
10001AE6   FF52 FF         CALL NEAR DWORD PTR DS:[EDX-1]
10001AE9   15 58C00010     ADC EAX,1000C058
10001AEE   B8 23C06E33     MOV EAX,336EC023
10001AF3   3D 586E86CD     CMP EAX,CD866E58
10001AF8   74 07           JE SHORT 10001B01                 AntiDebu.10001B01
10001AFA   B8 021B0010     MOV EAX,10001B02
10001AFF   FFE0            JMP NEAR EAX
10001B01   DB              ???                               Unknown command
10001B02   8D85 F0FEFFFF   LEA EAX,DWORD PTR SS:[EBP-110]
Stack SS:[0012EC98]=00000C7C
ECX=00000C7C
```

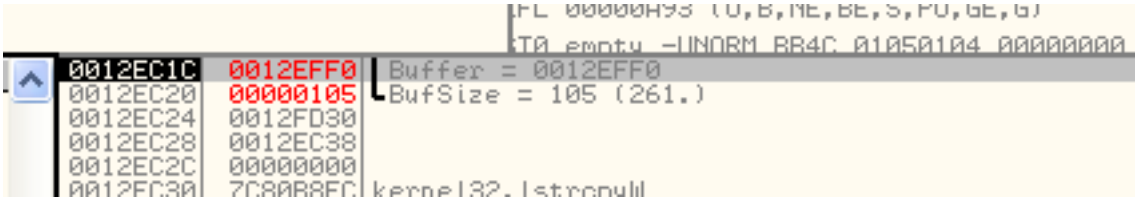
Address	Hex dump	ASCII
0012EC90	24 02 00 00 01 00 00 00 7C 0C 00 00 FF FF 00 00	$☻..0...|... ..
0012ECA0	FF FF 00 00 00 00 00 01 00 F0 0F 00 00 00 00 01	.....0.-*....0
0012ECB0	65 78 70 6C 6F 72 65 72 2E 65 78 65 00 00 00 00	explorer.exe....
0012ECC0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....

Продолжаем шаг за шагом.

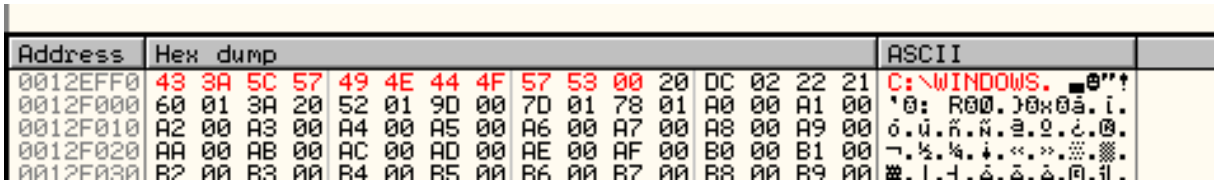


```
10001ADC   68 05010000     PUSH 105
10001AE1   8D95 F0FEFFFF   LEA EDX,DWORD PTR SS:[EBP-110]
10001AE7   52              PUSH EDX
10001AE8   FF15 58C00010   CALL NEAR DWORD PTR DS:[1000C058]  kernel32.GetWindowsDirectoryA
10001AEE   B8 23C06E33     MOV EAX,336EC023
10001AF3   3D 586E86CD     CMP EAX,CD866E58
10001AF8   74 07           JE SHORT 10001B01                 AntiDebu.10001B01
10001AFA   B8 021B0010     MOV EAX,10001B02
```

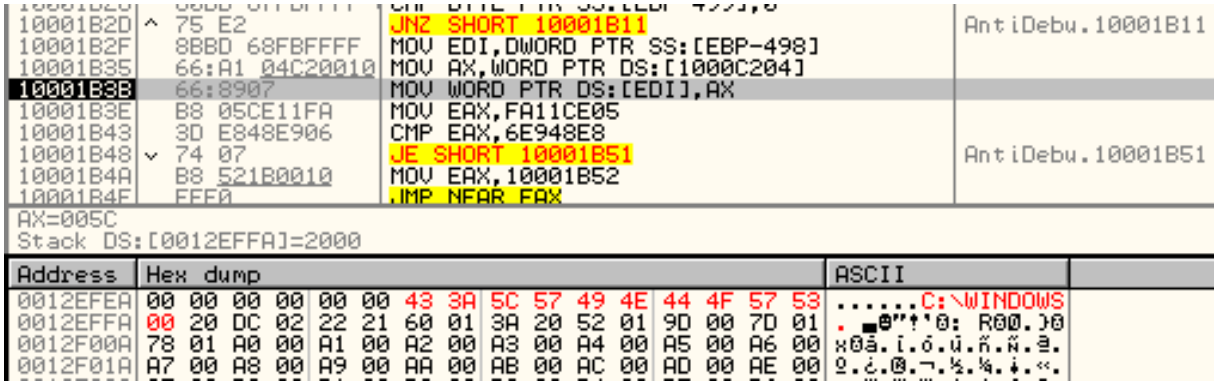
Здесь видим обращение к GetWindowsDirectory. Эта функция возвращает путь, по которому находится директория Windows, то есть куда установлена система, и explorer.exe всегда должен находится в ней, так что, как мы видим, путь из отчёта сравнивается с тем, что возвращает API-функция.



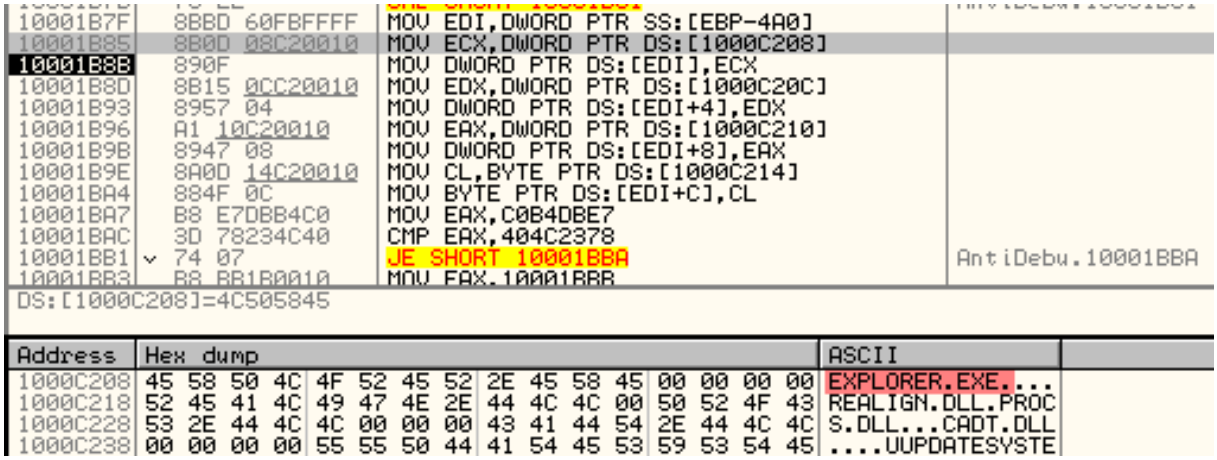
Смотрим в буфере, куда сохраняется путь.



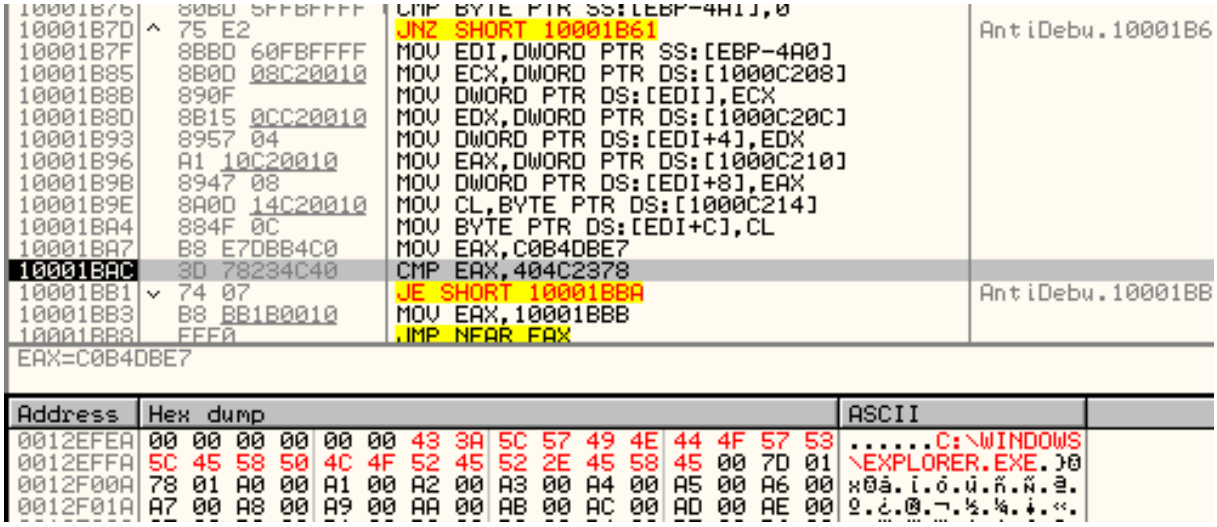
Сохраняется сюда, так что он готов для сравнения с путём в отчёте. В нашем случае проблем не будет, потому что в отчёте содержится путь до explorer.exe, так что спокойно продолжаем.



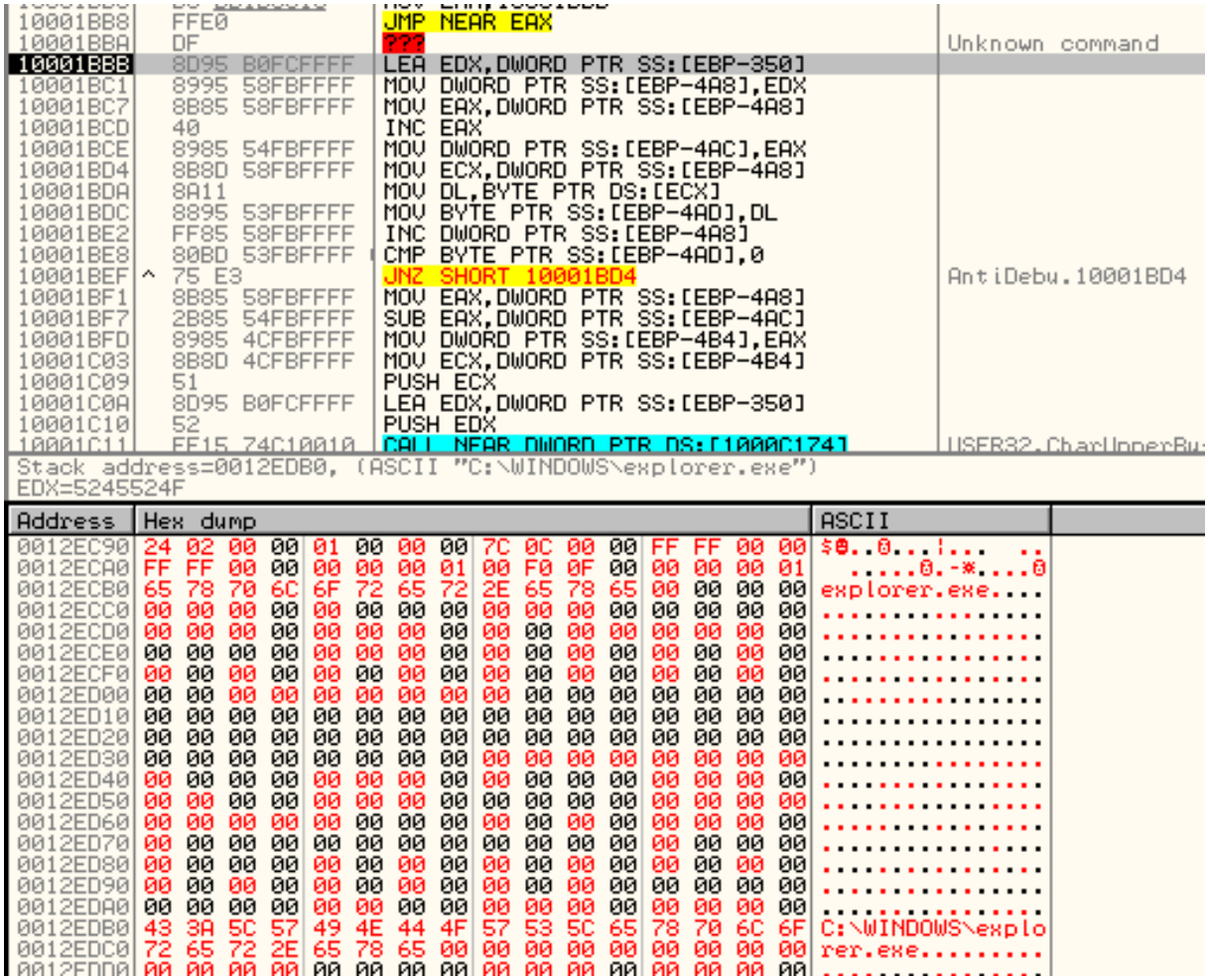
Видим, что чуть ниже к «C:/WINDOWS» добавляется «/», а затем «explorer.exe», чтобы можно было осуществить сравнение.



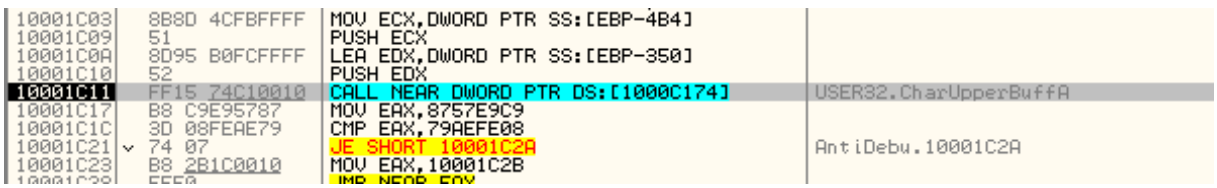
Здесь читаем и доформировываем путь.



Хех, прекрасно. Посмотрим, что тут делается.

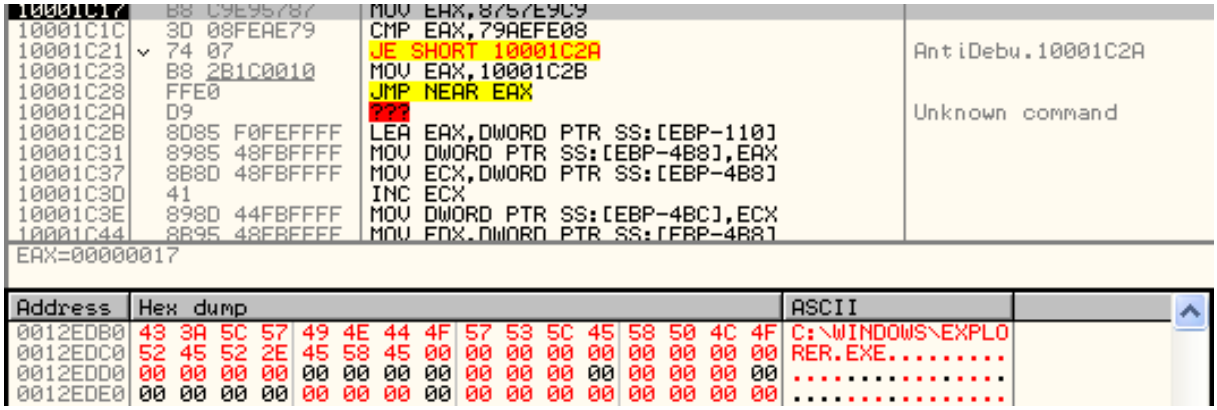


И подходим к сравнению. Здесь начинается считывание пути из отчёта.

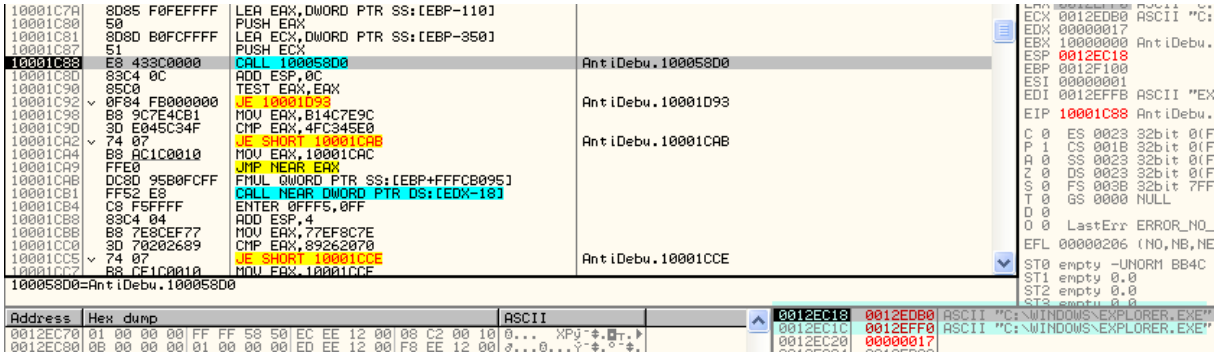


Доходим до CharUpperBuffA.

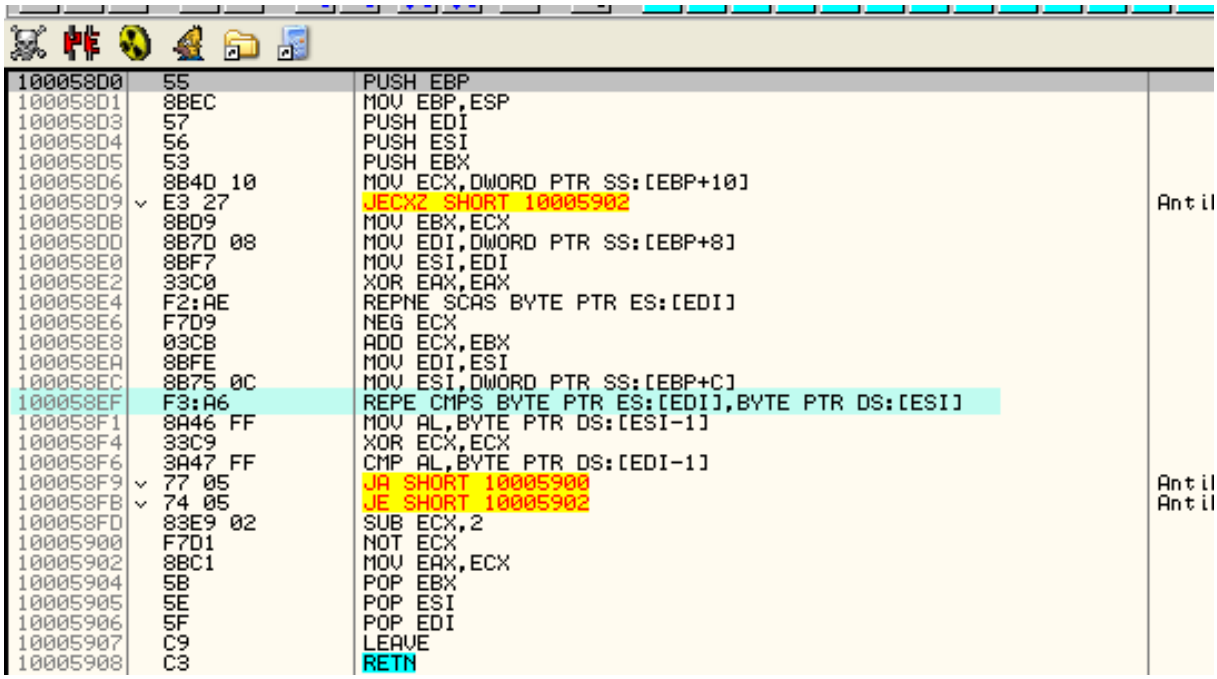
Эта API-функция преобразует строку, которая находится в BUFFER, в верхний регистр



Ничего особенного здесь нет, так что путь из отчёта преобразуется в верхний регистр. Это делается для того, чтобы было удобнее сравнивать, Путь создаётся с помощью функции GetSystemDirectory, к которой прибалвяется EXPLORER.EXE, также в верхнем регистре, так что переходим к самому сравнению.



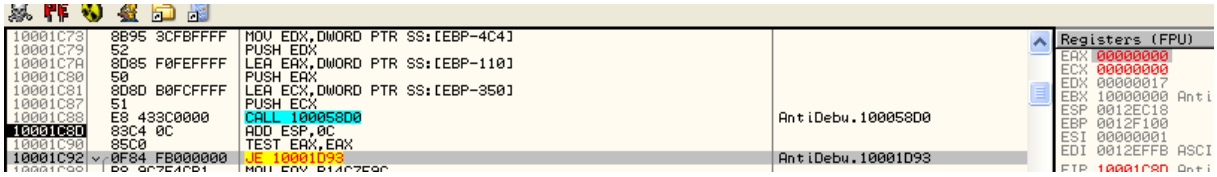
Доходим до CALL'a, который сравнивает обе строки, очевидно, что на выходе будет ноль в EAX. Ноль или единица означают, что строки равны/не равны соответственно.



Здесь видим строку, где стравниваются оба пути, так что если кто-то переименует OllyDbg в explorer.exe, то он должен запускать отладчик из директории WINDOWS, куда установлена система, и проблема заключается в том, что там находится настоящий запущенный EXPLORER.exe, так что

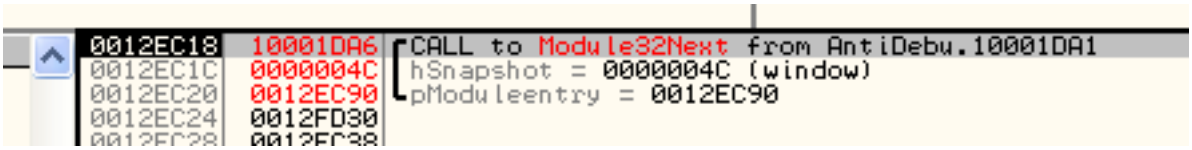
с этим методом могут возникнуть сложности.

Продолжаем и проходим через этот CALL с помощью F8.

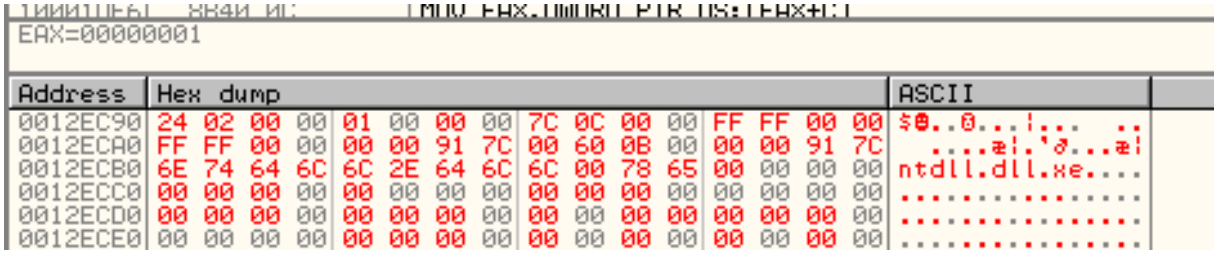


Видим, что EAX содержит ноль, поэтому на JE происходит переход.

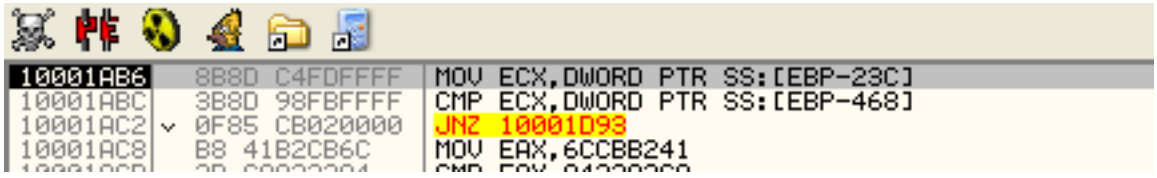
Если сразу посмотреть на финальное решение, то не надо было бы столь сильно хвататься за голову и многое из того, что мы делаем, не было бы необходимым, однако нет ничего плохого в том, чтобы учиться и узнавать, что мы можем делать, каким образом и где лежат пределы наших возможностей.

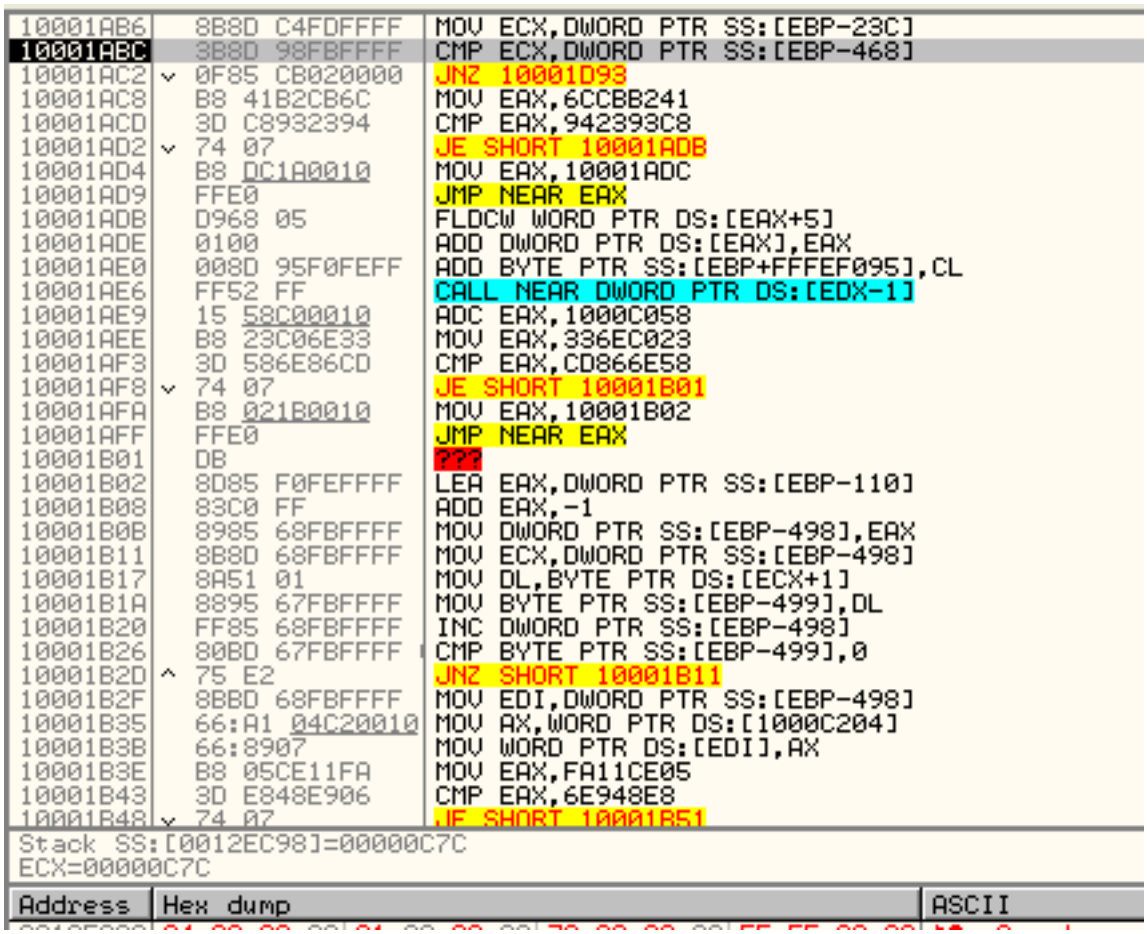


Видим, что крэкми всё ещё не убеждён в том, что всё в порядке, и продолжает искать данные о других модулях процесса, который его запустил. Здесь мы доходим до Module32Next, чтобы посмотреть отчёт о втором модуле.

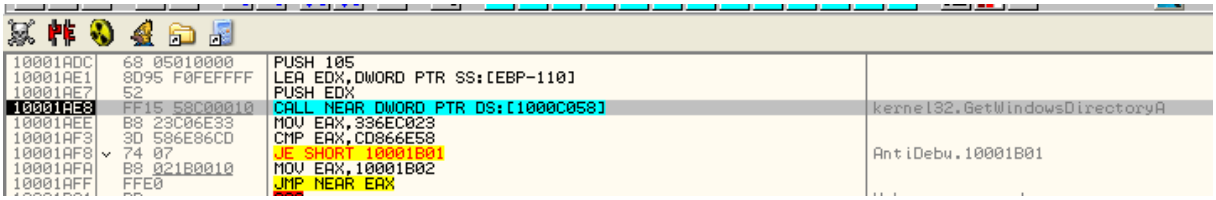


В данном случае это ntdll.dll. Видим, что те же данные, что и в прошлом случае. Идентификатор 01, который используется для определения всех модулей одного процесса, затем PID процесса (равный в моём случае 0C7C) и другие данные, такие как база и размер.

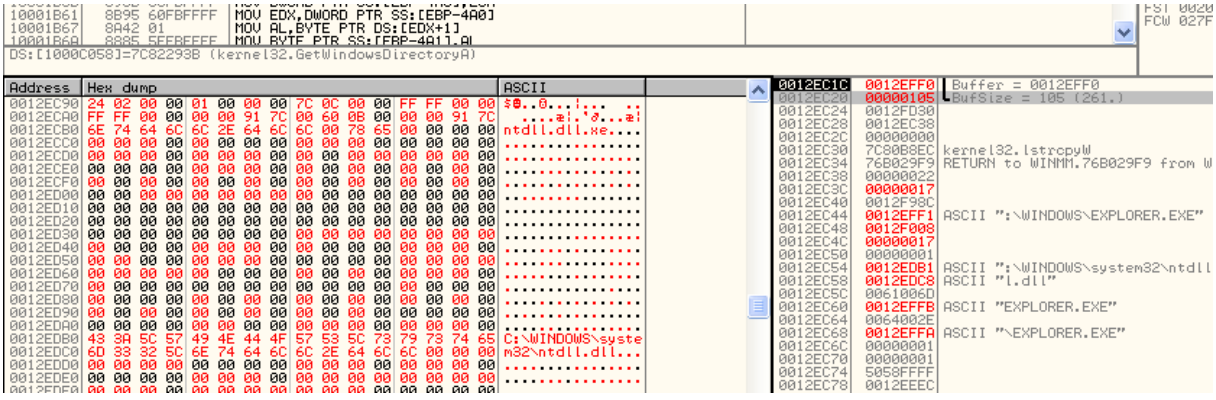




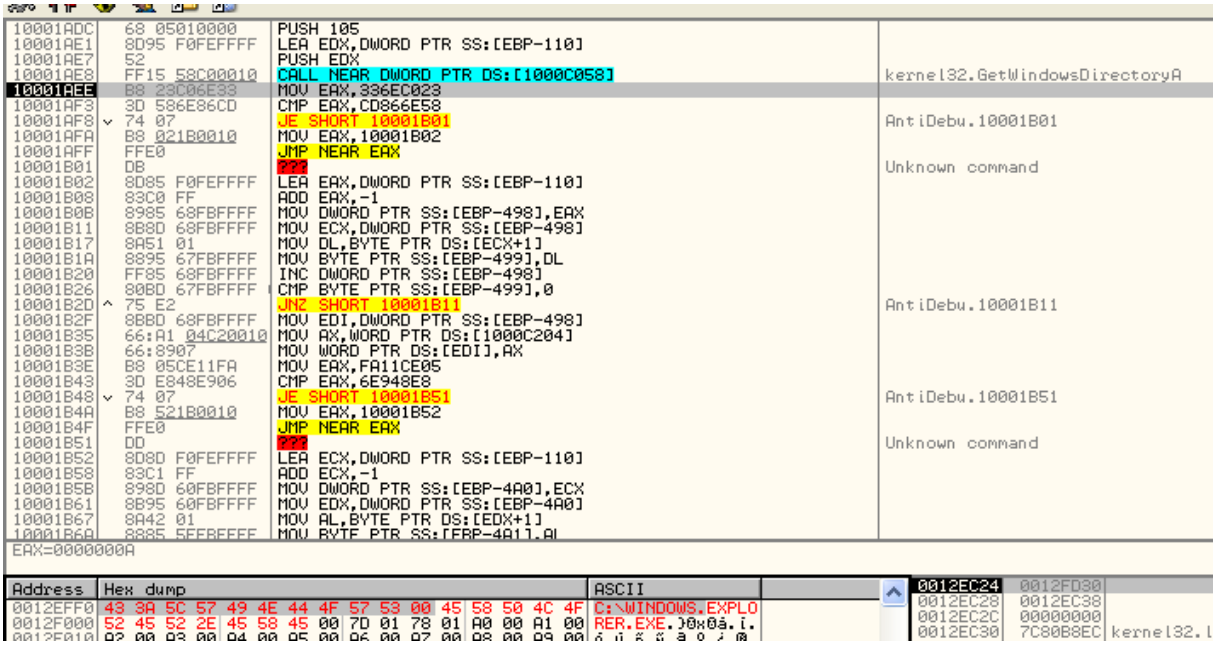
Продолжается сравнение, принадлежат ли все модули explorer.exe. Этого парня сложно убедить!



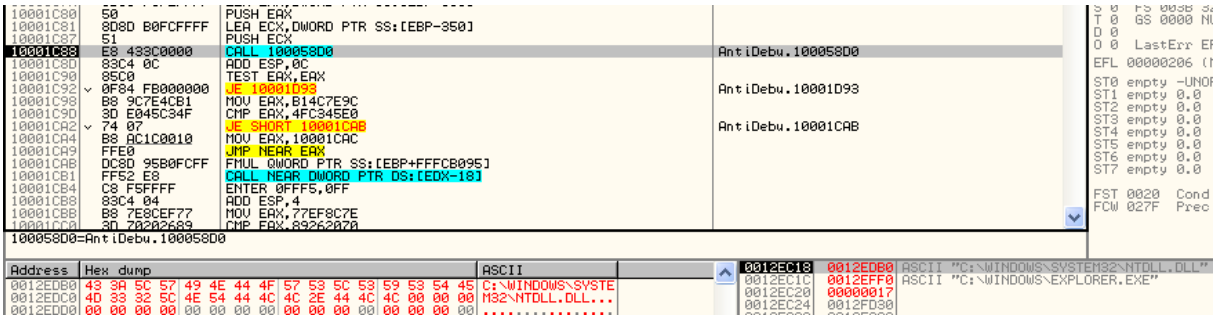
Уф, теперь GetWindowsDirectoryA.



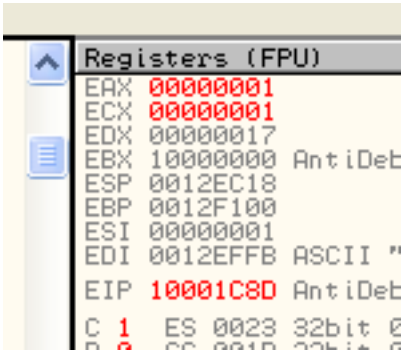
Держу пари, он сравнивает путь из отчёта с путём, возвращаемым данной API-функцией, не очень оригинально. Продолжаем смотреть.



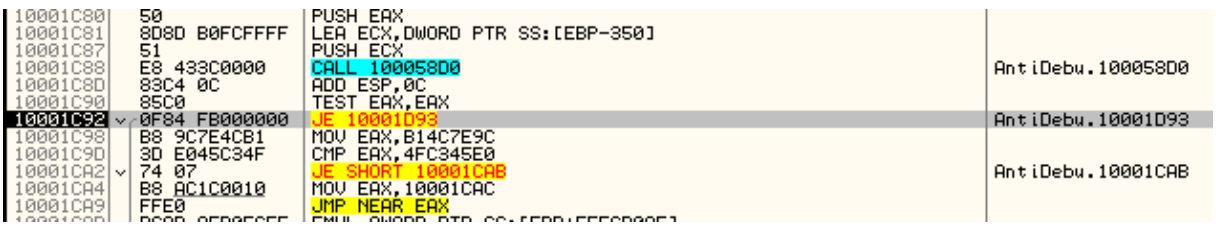
Здесь снова получаем путь, куда установлена Windows, и видим, что повторяется то же самое, что и с первым модулем, поэтому вдаваться в детали не будем и трассируем до вызова сравнения.



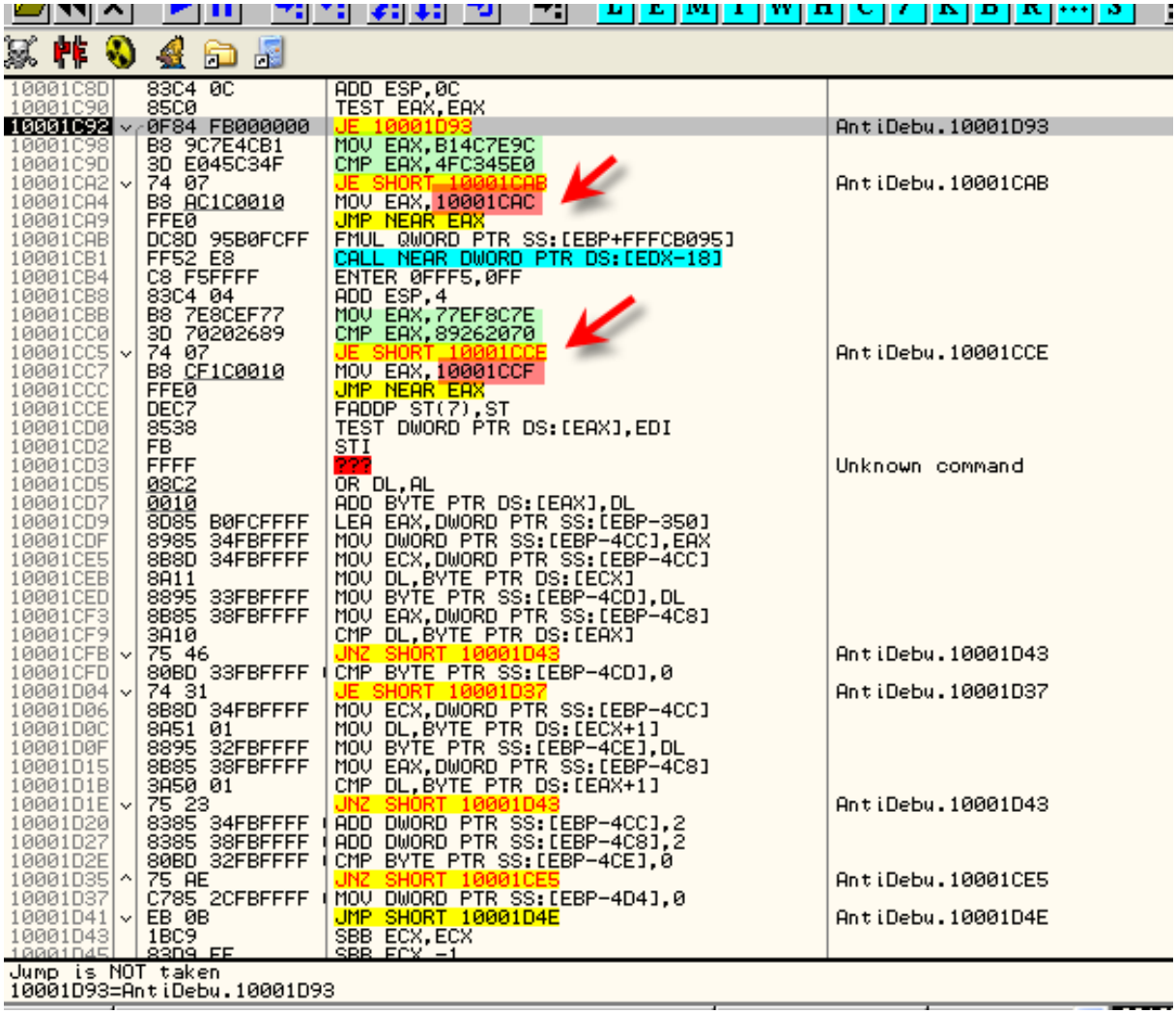
Добравшись туда, видим, что сравнивается путь к Эксплореру с путём к DLL. Они, естественно, не равны, так как системные библиотеки находятся в другой директории, так что они никоим образом не равны.



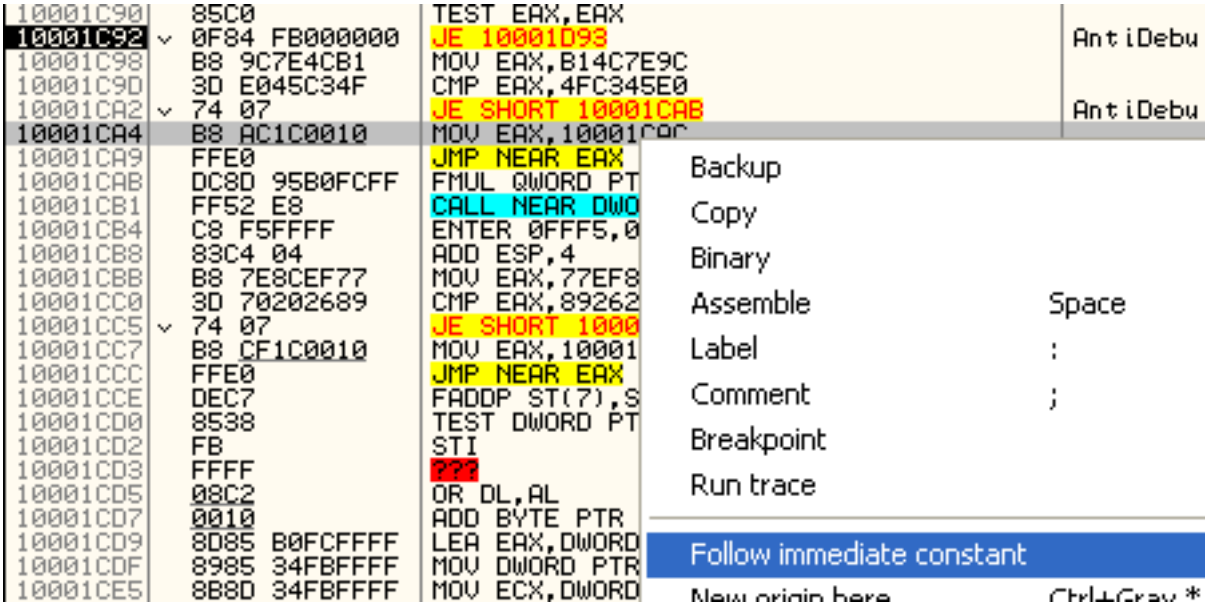
Поэтому после возвращения из вызова в EAX находится 1, так как они не равны и перехода не происходит.



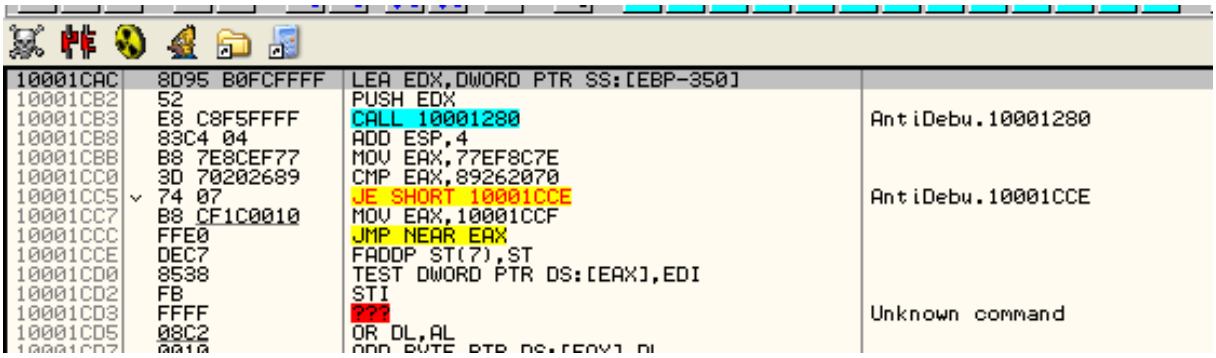
Продолжив, становится видна обфускация для запутывания кода.



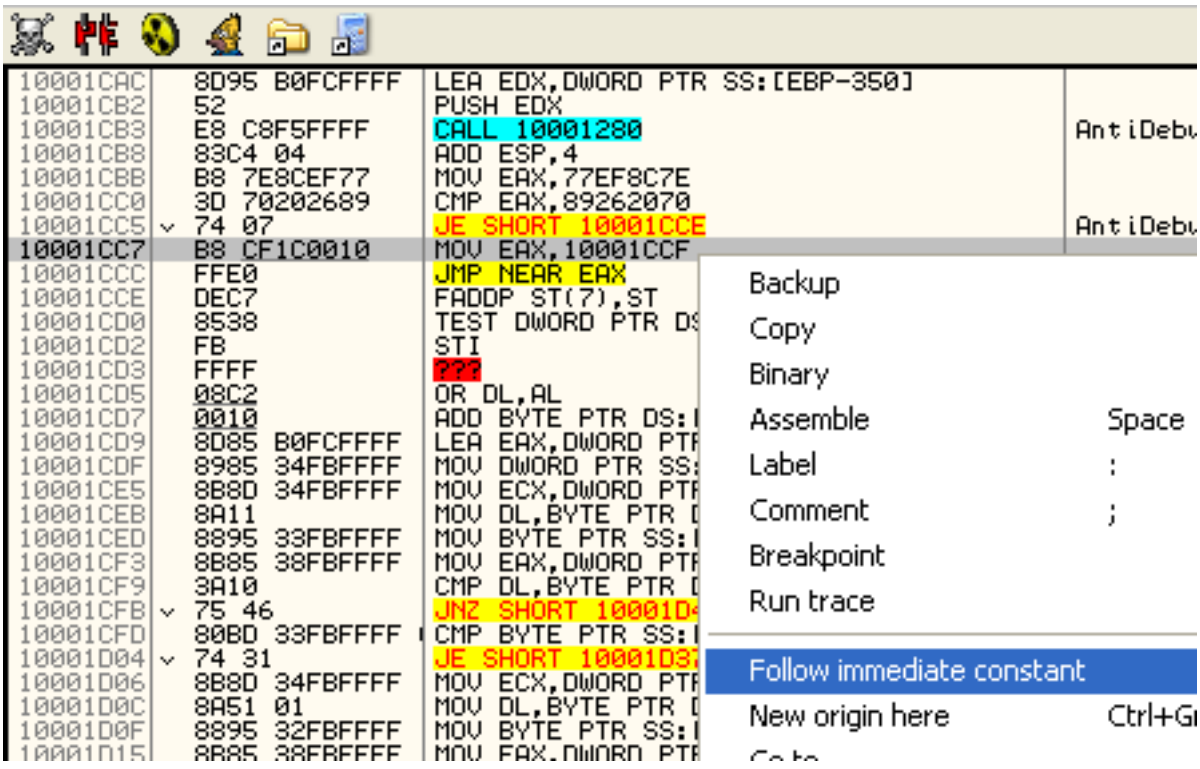
Как видим, здесь в EAX помещается константа, которая затем сравнивается с другой, и они никогда не будут равны, поэтому переход JE не срабатывается. Затем идёт MOV EAX, XXXXX, помещающий в регистр адреса (отмечены красным), а следом за этим – JMP NEAR EAX, который совершает переход по этому адресу, конечно, если будем трассировать, то увидим весь код. Обфускация не может предотвратить того, что мы будем трассировать и найдём настоящий код. Если не будем трассировать, а хотим быстро просмотреть запутанный код, направляем курсор на один из отмеченных MOV'ов, делаем щелчок правой кнопкой мыши и выбираем FOLLOW INMEDIATE CONSTANT.

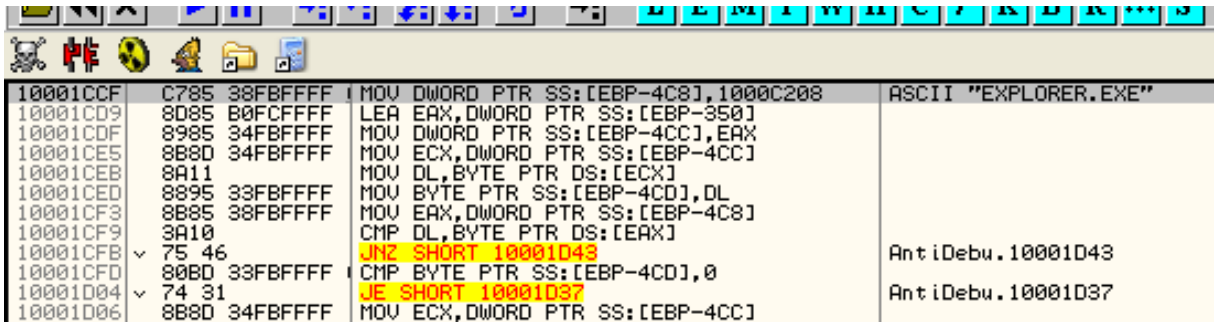


Ниже можем увидеть обфусцированный код без необходимости его выполнять.



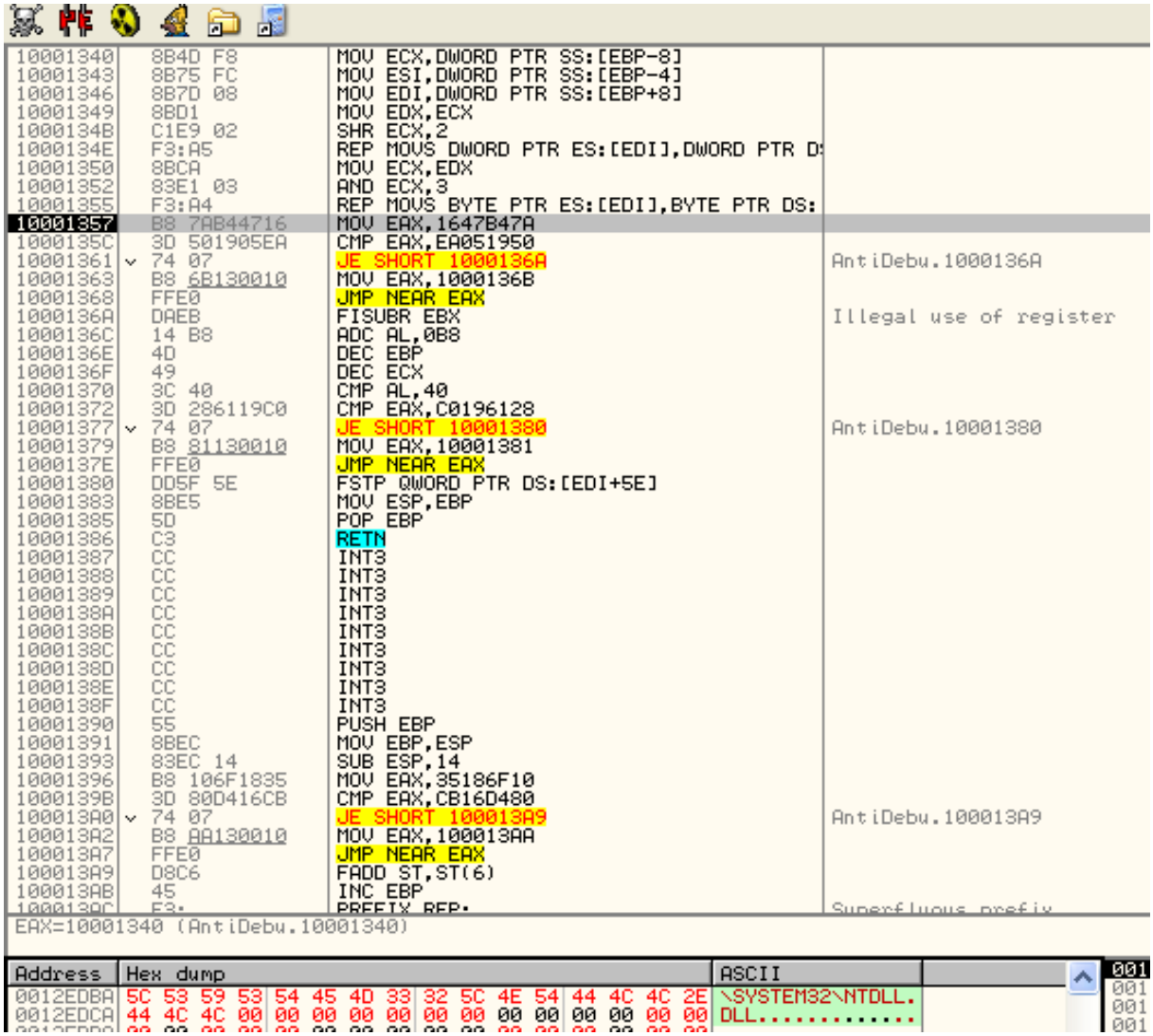
Здесь всё ясно видно без запуска, и если хотим посмотреть на следующий MOV, находящийся ниже, то повторяем.



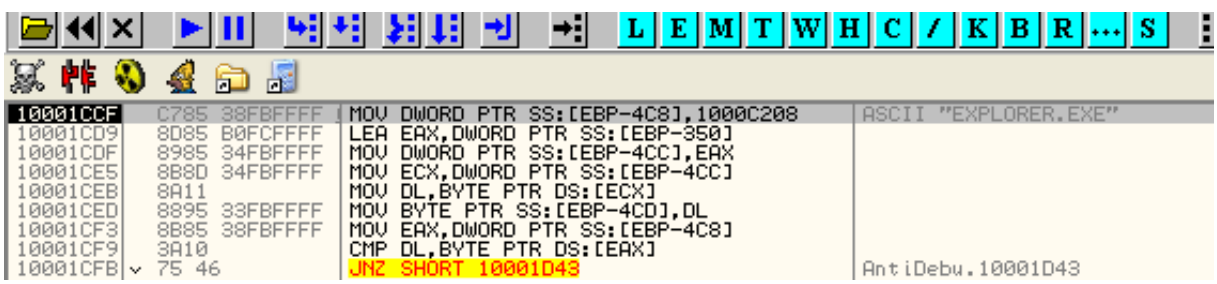


И видим обфусцированный код без необходимости его запускать.

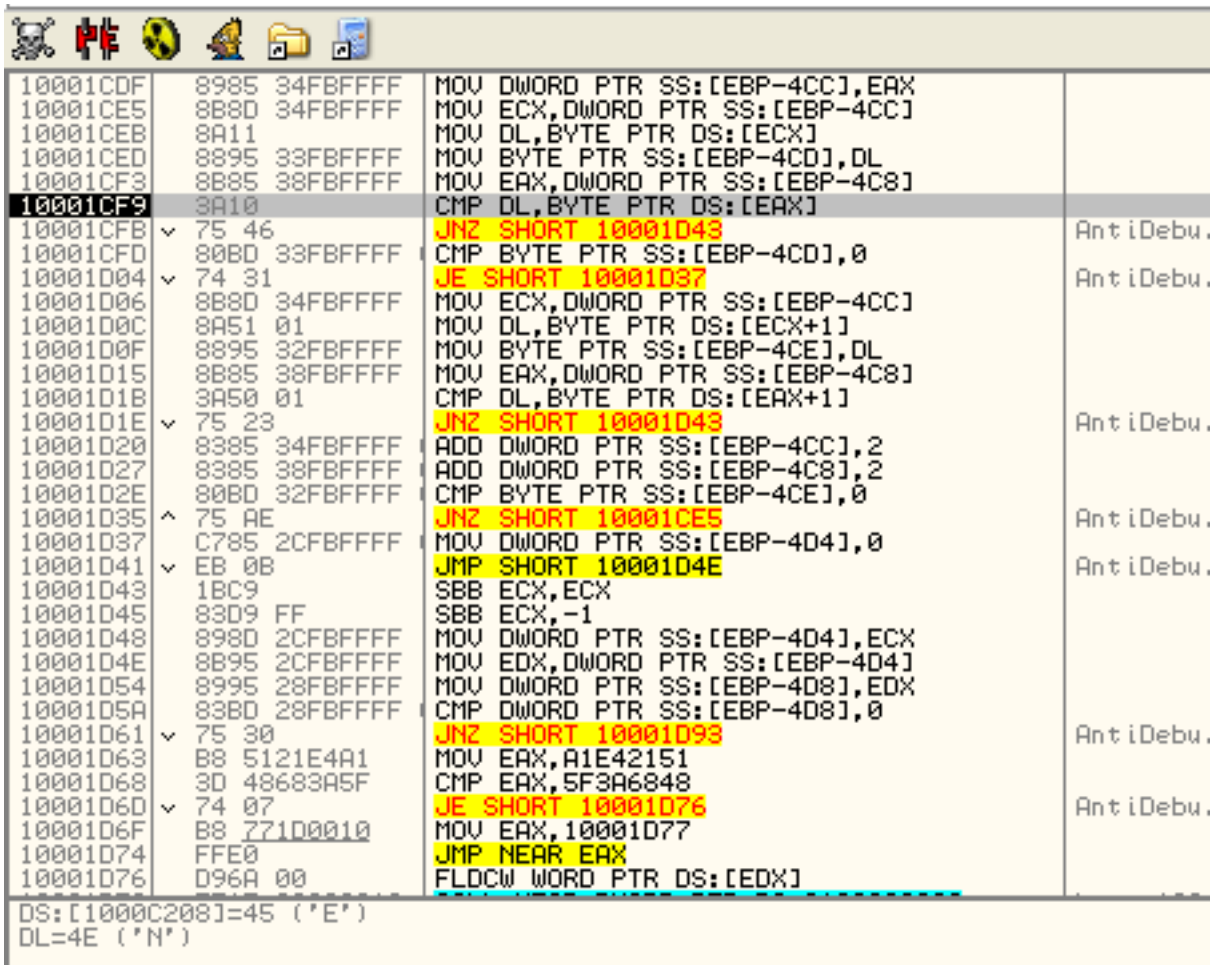
Это информационный уровне, пока продолжаем трассировать.



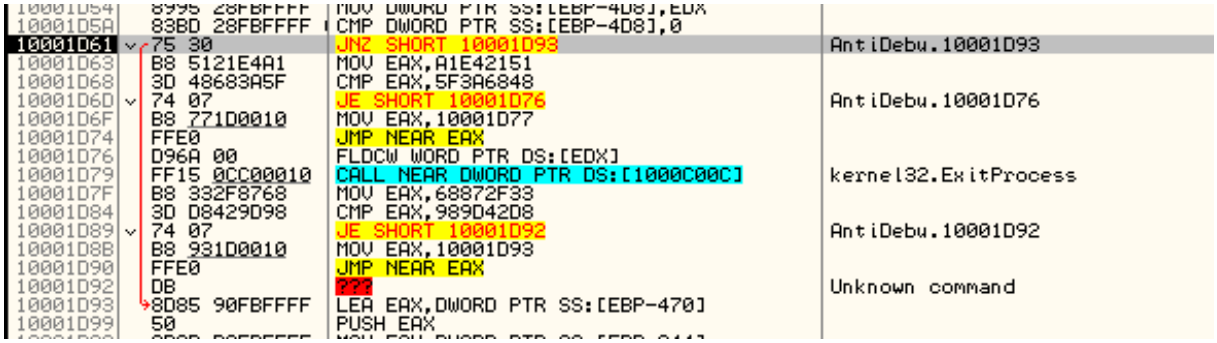
Продолжаем трассировать и видим, что игра с путями продолжается, в этот раз это SYSTEM32\NTDLL.DLL. Идём следом.



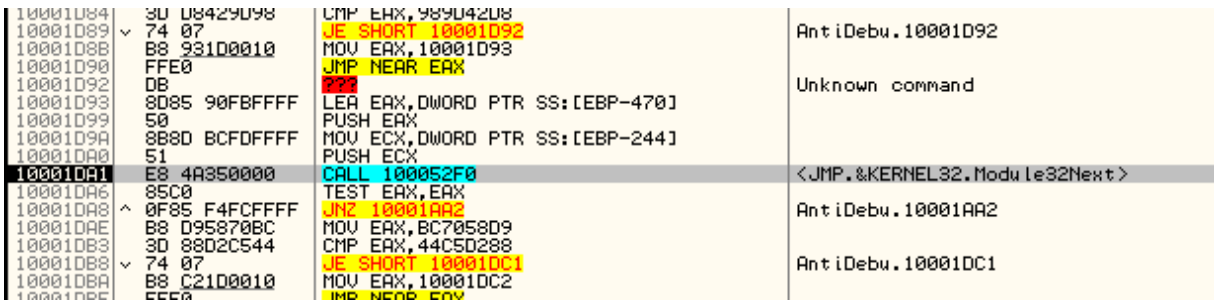
Снова считывается имя EXPLORER.exe.



Затем видим, что E из EXPLORER.exe сравнивается с N из NTDLL.DLL.



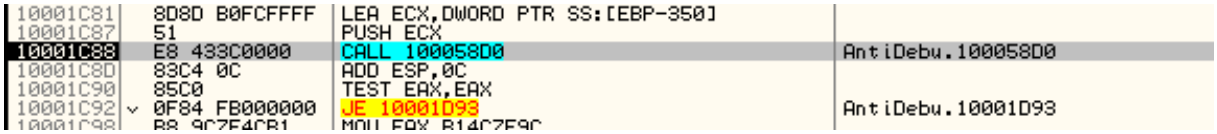
Затем идёт переход на ExitProcess, видим, что никакой проблемы нет и мы его минуем, так как очевидно, что эти два имени не равны, так что идём дальше.



Видим, что этот крэкми пробегает по всем модулям один за другим. Не вижу большого смысла в этой части, так как если процесс убедился, что его главный модуль находится по правильному пути, какой смысл в этих проверках?

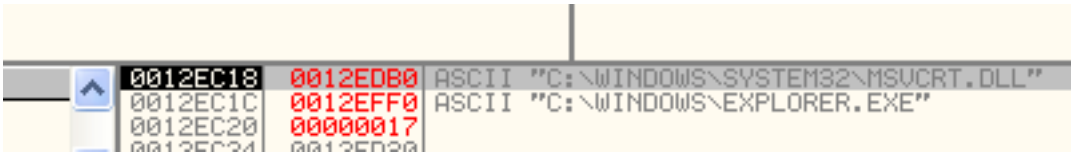
Но ладно, это прояснится по мере того, как продолжим трассировку, терпение.

Трассируем до сравнения.

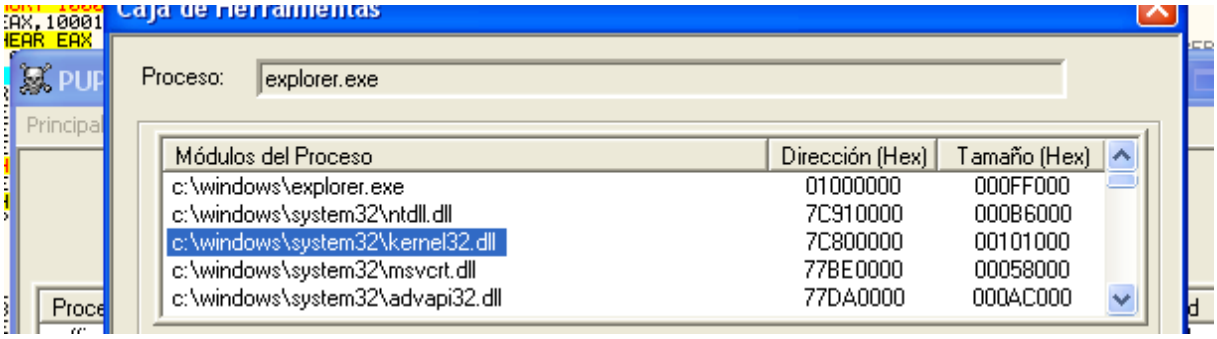


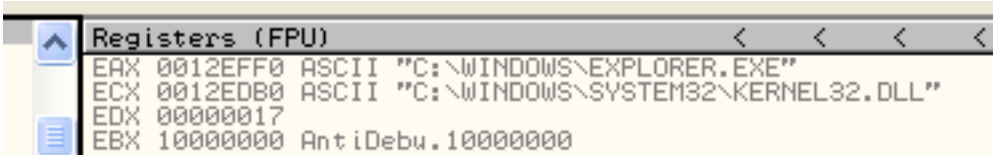
Видим, что покамест продолжается проверка модулей один за другим, и каждый из них сравнивается с EXPLORER.exe и только первый так назывался, поэтому для остальных сравнение каждый раз оказывается ложным.

Ладно, чтобы сэкономить на трассировке, устанавливаем HBP ON EXECUTION на CALL COMPARADOR, так нам удастся быстрее пройти через сравнения.

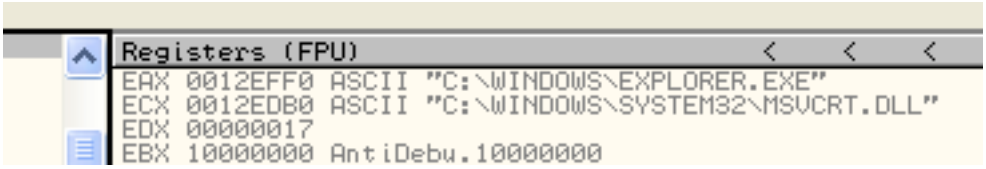


В PUPE у нас также есть список модулей Explorer'a, поэтому можем пройти один за другим, пока не закончится сравнение каждого из них. В PUPE входим в CAJA DE HERRAMIENTAS (Коробка с инструментами), смотрим модули.





Поскольку они располагаются по порядку, нам ничего не стоит следовать за ними.



Последним в моём случае идёт idle.dll.

```
c:\windows\system32\winspool.drv          72F80000    00026000
c:\windows\system32\duser.dll             6C6A0000    0004D000
c:\windows\system32\sxs.dll               76980000    000B1000
c:\archivos de programa\yahoo!\messenger\idle.dll    60300000    00007000
```

Так что нужно пройти все остальные, пока не встретим этот.

```
Registers (FPU)
EAX 0012EFF0 ASCII "C:\WINDOWS\EXPLORER.EXE"
ECX 0012EDB0 ASCII "C:\WINDOWS\SYSTEM32\DUSER.DLL"
EDX 00000017
EBX 10000000 AntiDebu.10000000
```

Мы уже почти у цели.

```
Registers (FPU)
EAX 0012EFF0 ASCII "C:\WINDOWS\EXPLORER.EXE"
ECX 0012EDB0 ASCII "C:\ARCHIVOS DE PROGRAMA\YAHOO!\MESSENGER\IDLE.DLL"
EDX 00000017
EBX 10000000 AntiDebu.10000000
ESP 0012EC18
```

Здесь прибываем в последний. Теперь можем продолжить трассировку, чтобы видеть, что происходит.

```
10001D54   8995 28FBFFFF   MOV DWORD PTR SS:[EBP-4D8],EDX
10001D5A   83BD 28FBFFFF   CMP DWORD PTR SS:[EBP-4D8],0
10001D61   75 30           JNZ SHORT 10001D93                  AntiDebu.10001D93
10001D63   B8 5121E4A1     MOV EAX,A1E42151
10001D68   3D 48683A5F     CMP EAX,5F3A6848
10001D6D   74 07           JE SHORT 10001D76                   AntiDebu.10001D76
10001D6F   B8 771D0010     MOV EAX,10001D77
10001D74   FFE0            JMP NEAR EAX
10001D76   D96A 00         FLDCW WORD PTR DS:[EDX]
10001D79   FF15 0CC00010   CALL NEAR DWORD PTR DS:[1000C00C]   kernel32.ExitProcess
10001D7F   B8 332F8768     MOV EAX,68872F33
10001D84   3D D8429D98     CMP EAX,989D42D8
10001D89   74 07           JE SHORT 10001D92                   AntiDebu.10001D92
10001D8B   B8 931D0010     MOV EAX,10001D93
10001D90   FFE0            JMP NEAR EAX
10001D92   DB              ???                                 Unknown command
10001D93   8D85 90FBFFFF   LEA EAX,DWORD PTR SS:[EBP-470]
10001D99   50              PUSH EAX
10001D9A   8B8D BCFDFFFF   MOV ECX,DWORD PTR SS:[EBP-244]
10001DA0   51              PUSH ECX
10001DA1   E8 4A350000     CALL 100052F0                       <JMP.&KERNEL32.Module32Next>
10001DA6   85C0            TEST EAX,EAX
10001DA8 ^ 0F85 F4FCFFFF   JNZ 10001AA2                        AntiDebu.10001AA2
10001DAE   B8 D95870BC     MOV EAX,BC7058D9
```

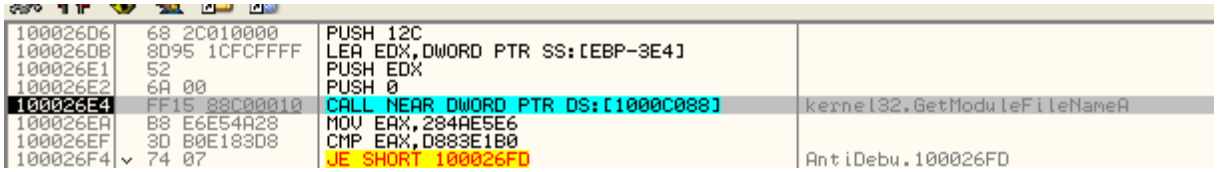
Видим, что продолжаем проскакивать ExitProcess и снова идём в Module32Next.

```
Address    Hex dump                                          ASCII
0012EC90   24 02 00 00 01 00 00 00 B8 0F 00 00 01 00 00 00   $☻..☺...☺☼..☺...
0012ECA0   01 00 00 00 00 00 30 60 00 70 00 00 00 00 30 60   ☺.....0`.p....0`
0012ECB0   69 64 6C 65 2E 64 6C 6C 00 00 52 56 00 35 2E 64   idle.dll..RV.5.d
0012ECC0   6C 6C 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ll..............
0012ECD0   00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0012ECE0   00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
```

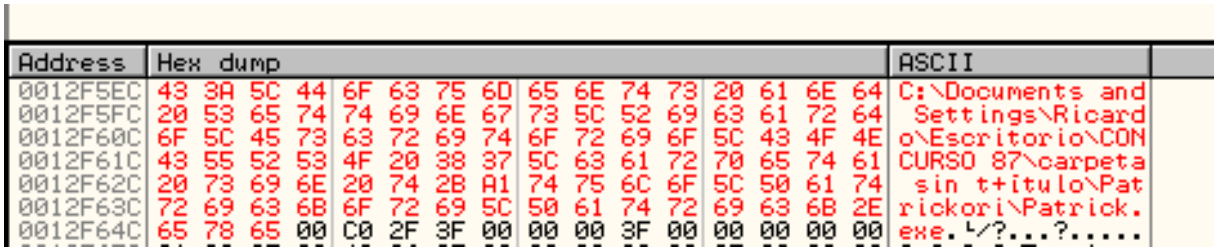
Здесь считывается последний. Если видите, то изменился PID explorer'a, это потому что в середине туториала перегружал машину, и PID изменился, но всё идёт точно таким же образом, только PID другой.

```
1000569A   CC              INT3
1000569B   3B0D BC050110   CMP ECX,DWORD PTR DS:[100105BC]
100056A1   75 01           JNZ SHORT 100056A4                  AntiDebu.100056A4
100056A3   C3              RETN
100056A4 ^ E9 C1FFFFFF     JMP 1000566A                        AntiDebu.1000566A
```

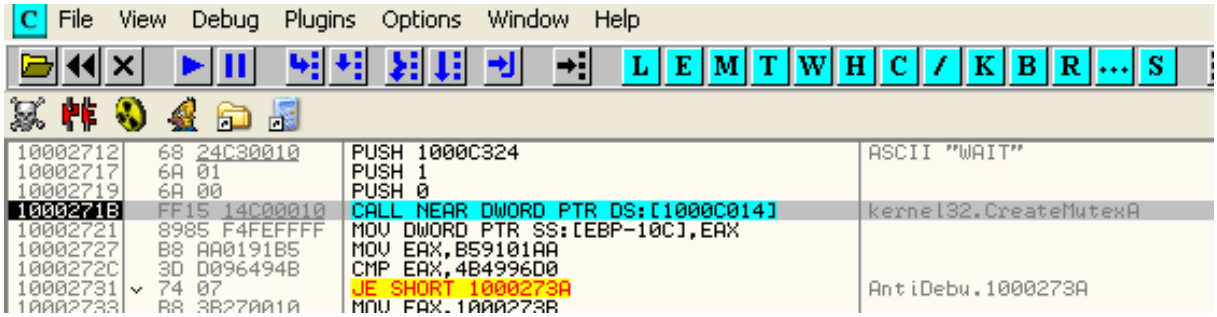
Видим, что это последнее сравнение, и снова минуем ExitProcess.



Ок, похоже, что крэкми устал повторяться, и начинается что-то иное. Доходим до API-функции GetModuleFileNameA.

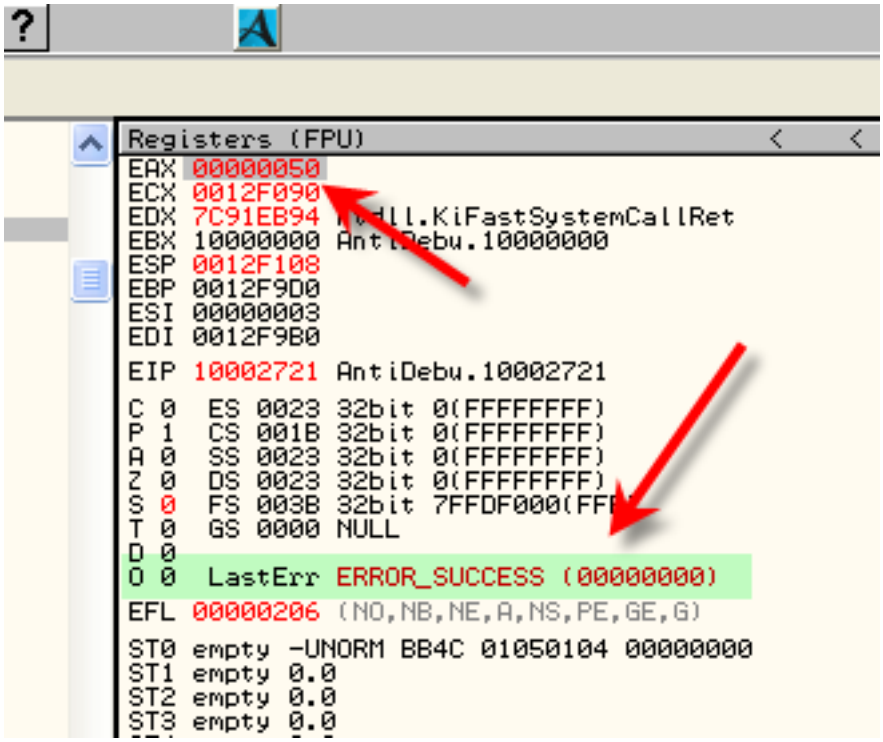


С помощью которого получаем путь до крэкми.



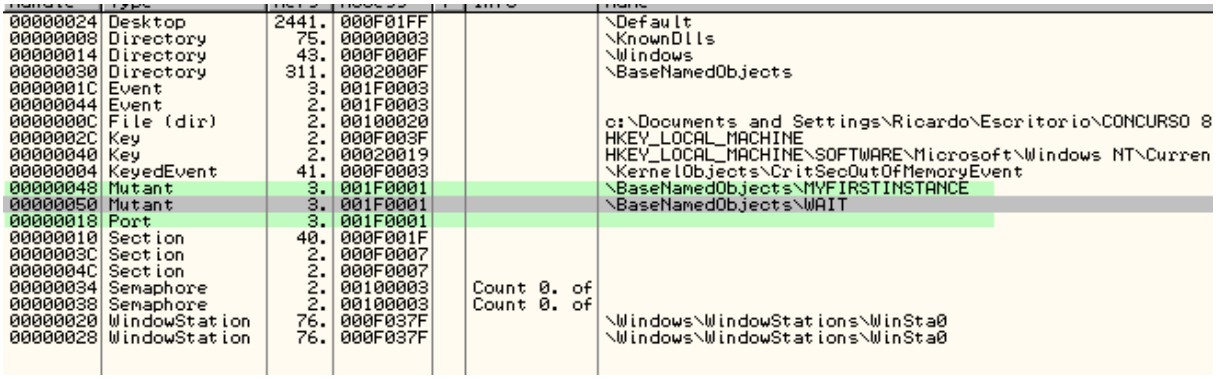
Хорошо, сейчас прибываем в CreateMutexA, который я объясню, раз мы не встречали его в предыдущих частях. Он используется тогда, когда программа запускает второй экземпляр (процесс) самой себя. С помощью этой API-функции можно проверить, в первый ли раз была запущена программа или это уже второй процесс. Здесь видим её работу.

Вот параметры функции.

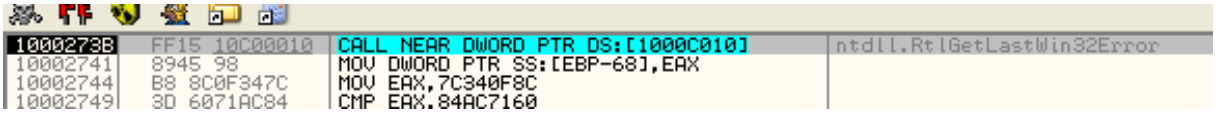


Пройдя API-функцию с помощью F8, видим, что система предоставляет хэндл, равный 50, а также OllyDbg нам сообщает об “ERROR SUCCESS”, что на русский можно перевести как удачное создание мутекса, то есть успех.

Видим, что есть два мутекса, один зовётся MYFIRSTINSTANCE, а другой WAIT, можем посмотреть более подробную информацию о них, если пойдём в окно H (хэндлы).

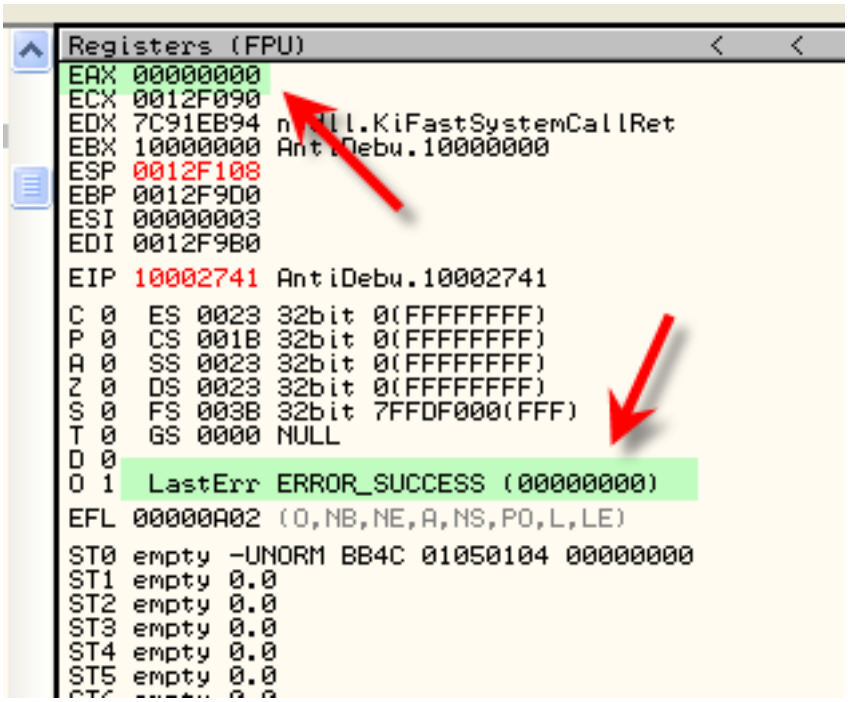


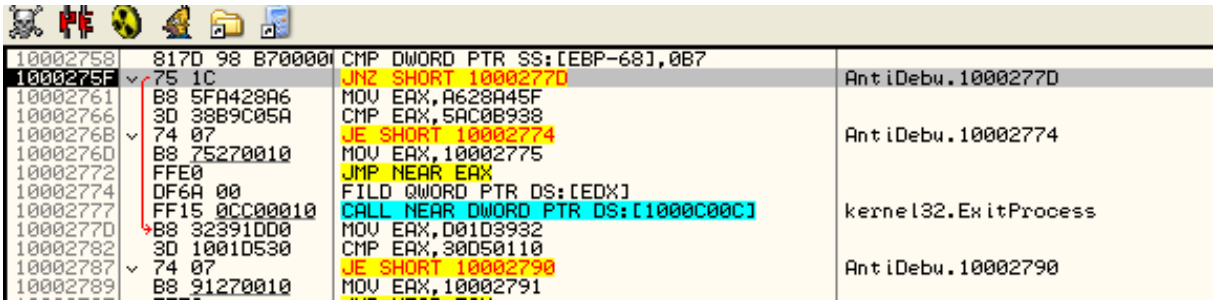
Один был очевидно создан в начале части, которую мы не трассировали, но вот она:



Затем вызывается API-функция, которая возвращает последнюю произошедшую ошибку. OllyDbg сообщила нам об успехе, а теперь программа с помощью данной API-функции узнаёт о том же.

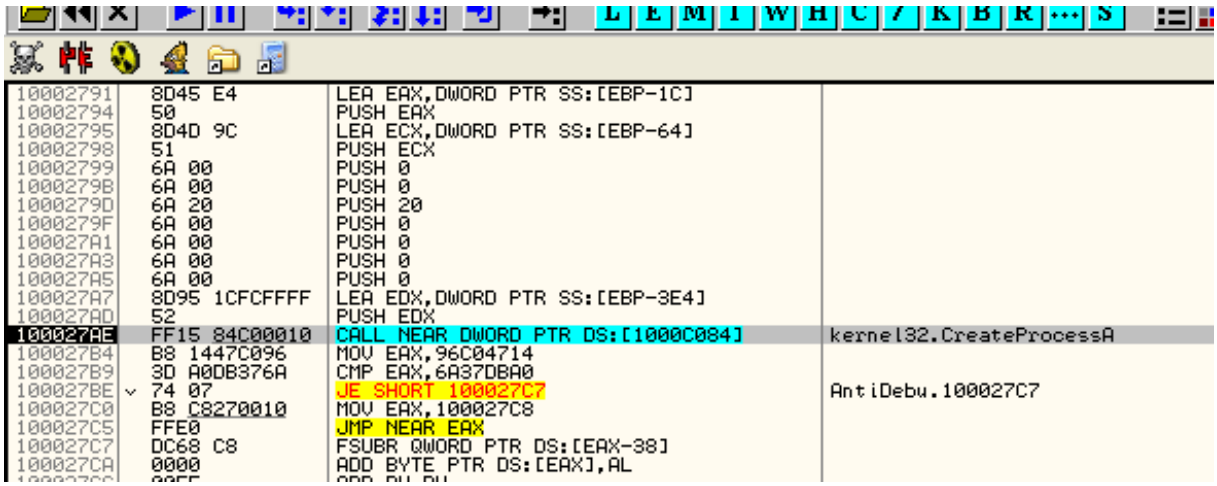
Как видим, после возврата из API-функции EAX содержит ноль, который в данном случае как раз и обозначает успешное выполнение.



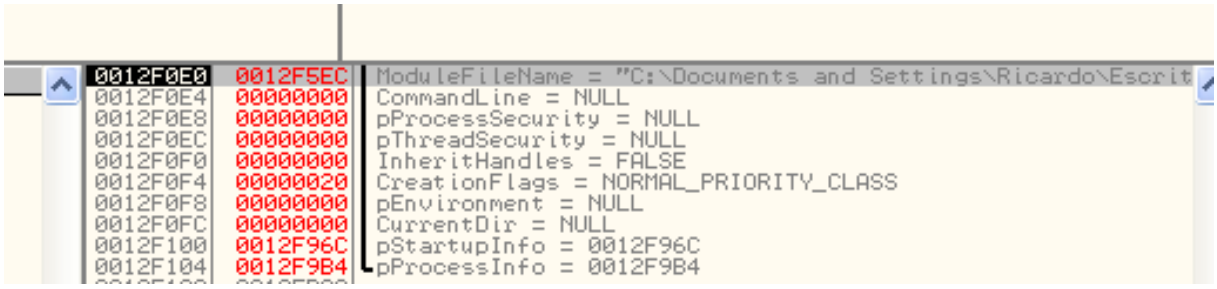


Здесь мы видим ловушку и предполагаем, что она называется WAIT. Первый из двух мутексов называется MY FIRST INSTANCE, и он решает, является ли этот процесс первым или вторым запущенным, и иногда вызывается WAIT, видим, что в этом случае в результате SUCCESS'a программа не идёт на ExitProcess, но в случае со вторым процессом, если здесь проверяется мутекс, а существует ранее запущенный процесс, то результат будет равен не нулю, а B7, что приведёт к завершению программы. Выполнение продолжится только, если это первый экземпляр крэкми. Пока что запомним, что если это первый экземпляр процесса, то мутекс будет равен SUCCESS, а если нет, то ERROR ALREADY EXISTS, и таким образом в большинстве случаев и выясняется, является ли этот экземпляр программы первым или нет.

Продолжаем дальше.



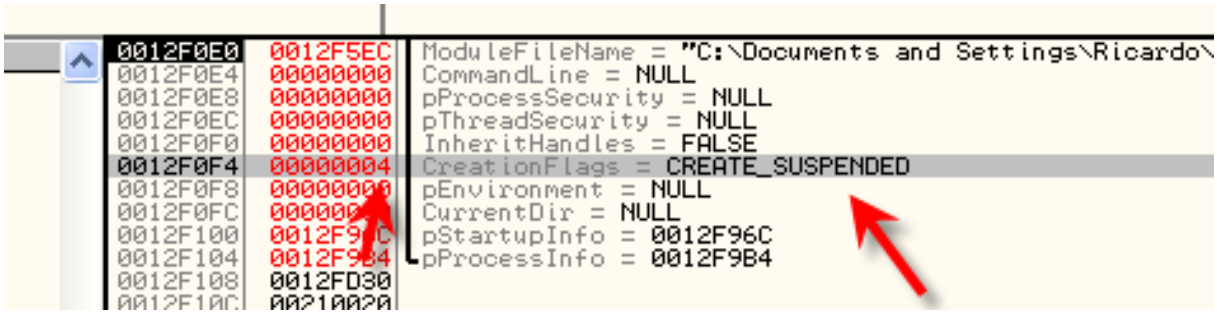
Так, здесь видим, что программа собирается создать второй процесс с помощью API-функции CreateProcessA.



Вот её параметры. Идея в том, что предположительно, если нажмём F8, то будет создан второй процесс, но прежде чем закроется этот, второй процесс пройдёт проверку мутексом WAIT, и так как первый процесс не был закрыт, то второй процесс завершится.

Очевидно, что если это делается, когда запущенна программа, этот первый процесс создаёт второй и немедленно его закрывает, гораздо раньше, чем второй начнёт проверять все модули и дойдёт до проверки MUTEX WAIT.

То есть нажатие на F8 не поможет нам, так как другой процесс азкроется. Мы может сделать немногое, попробуем его заморозить, для чего поменяем параметр CreationFlags на 4.



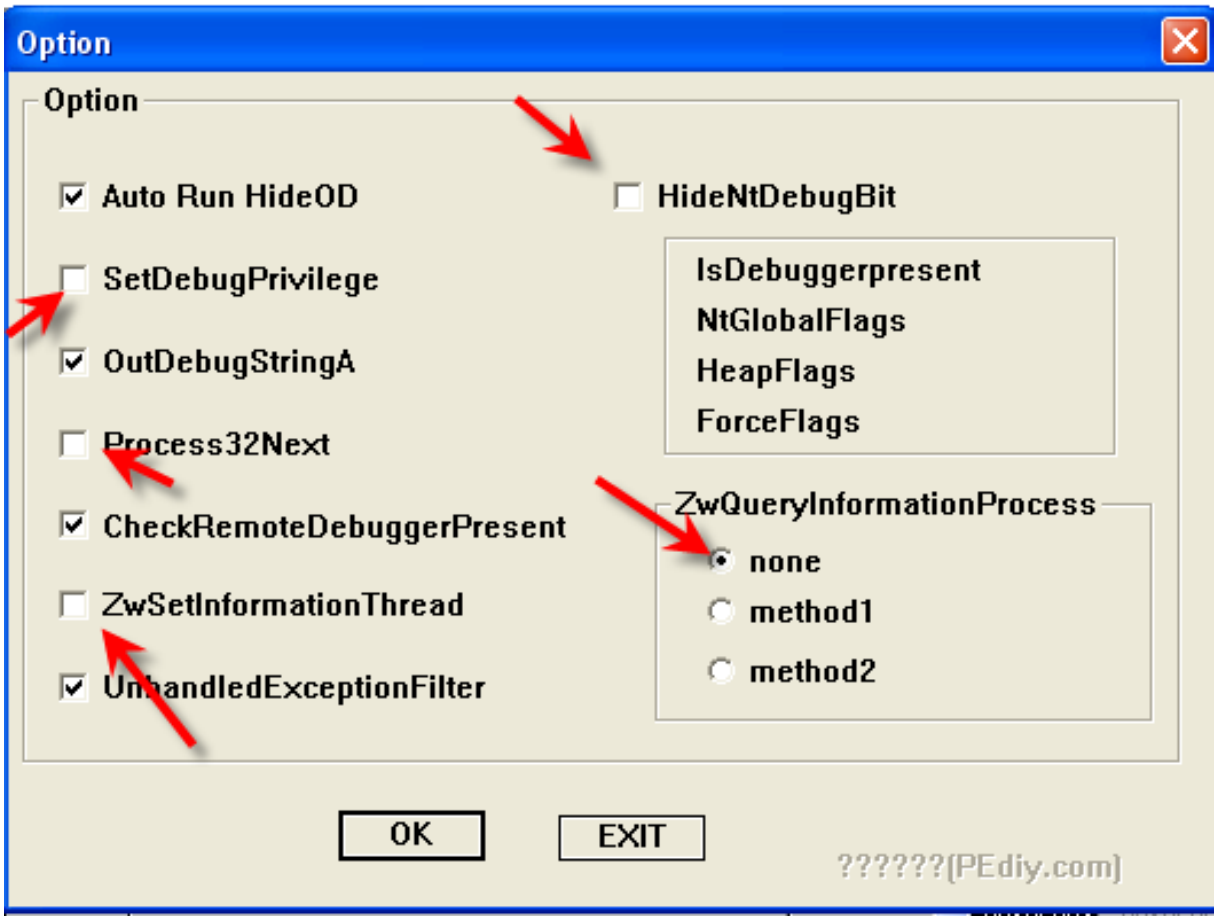
Смотрим, что произойдёт.

Как видим, при запуске второй процесса было возвращено EAX=0, и это означает, что создать его было невозможно. Это озадачивает.

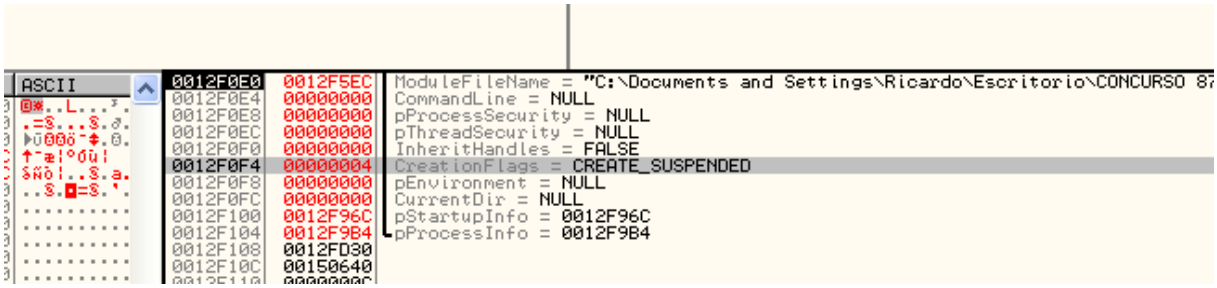
Устанавливаем HBP ON EXECUTION на вызов CreateProcessA, и идём пробовать, почему не создаётся второй процесс, то есть требуется два HBP, один раз, чтобы изменить PID parcheado 4 на PID эксплорера, а изменив его, а второй HBP здесь на CreateProcessA.

А, снова HideOD, возможно, что он каким-то образом меняет права доступа крэкми, не давая ему создать другой процесс.

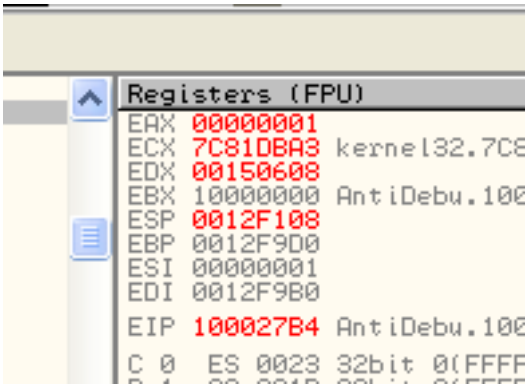
На картинке показано, какие галочки убираем в HideOD. Теперь смотрим, создастся ли второй процесс.



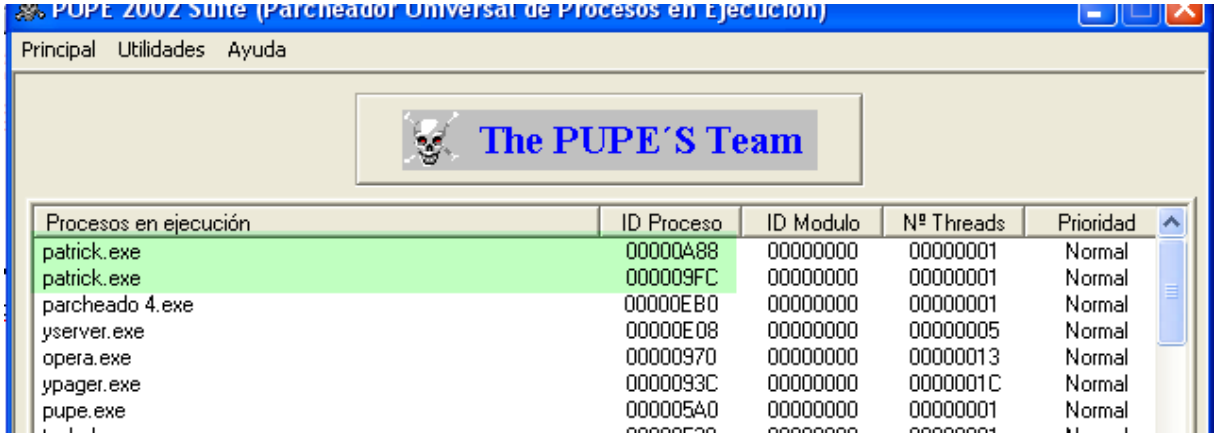
Установив галочки указанным образом, проходим API-функцию с помощью F8, и в EAX помещается EAX=1, так что мы на правильном пути.



Теперь снова доходим до нужного места при установленных как надо опциях HideOD и заново устанавливаем параметр функции, чтобы процесс создавался замороженным.



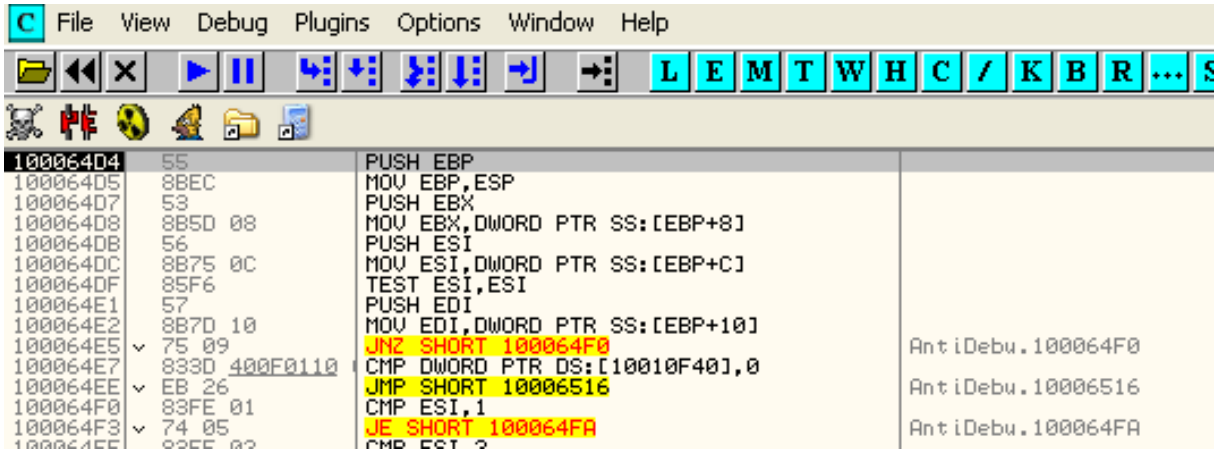
Посмотрим в Pupe.



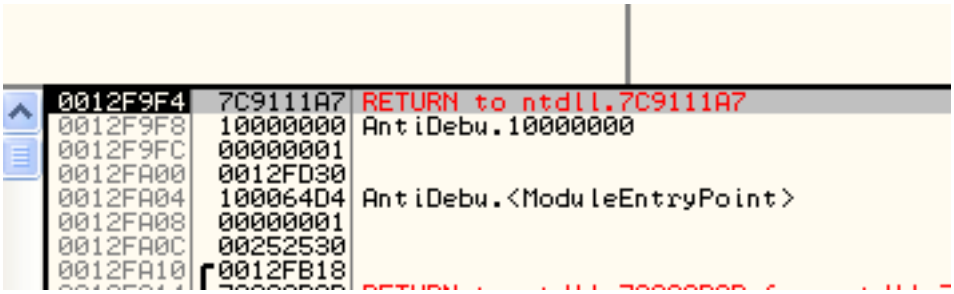
У нас есть два созданных patrick.exe, и второй заморожен, хе-хе.

Теперь пойдёт сложная тема – как остановить и приаттачиться к процессу, так как видим, что проверяется всё, а если что-то находится, то программа завершает выполнение. Это сделать не слишком просто.

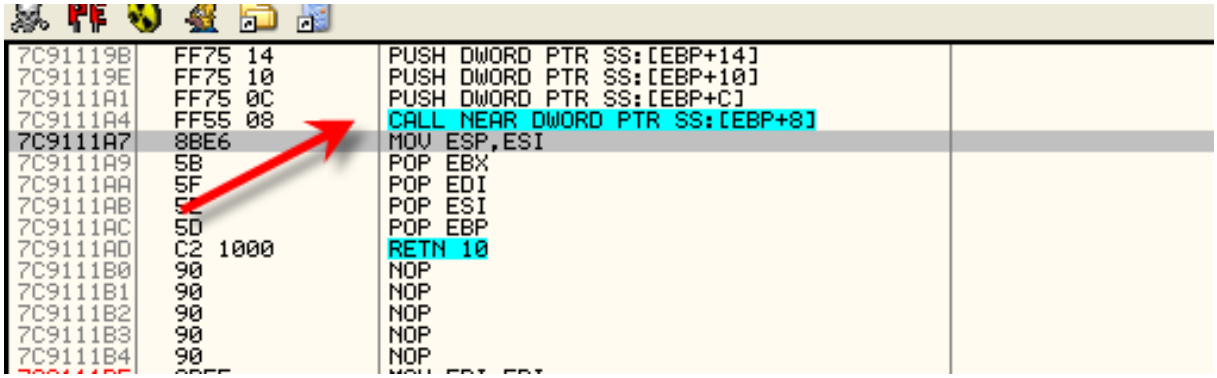
Попробуем следующий метод. Перезапускаем parcheado 4 и когда останавливаемся на системной точке останова, устанавливаем BPM ON ACCESS на секции кода antidebug.dl. Когда останавливаемся на её EP, смотрим это значение на нашей машине.



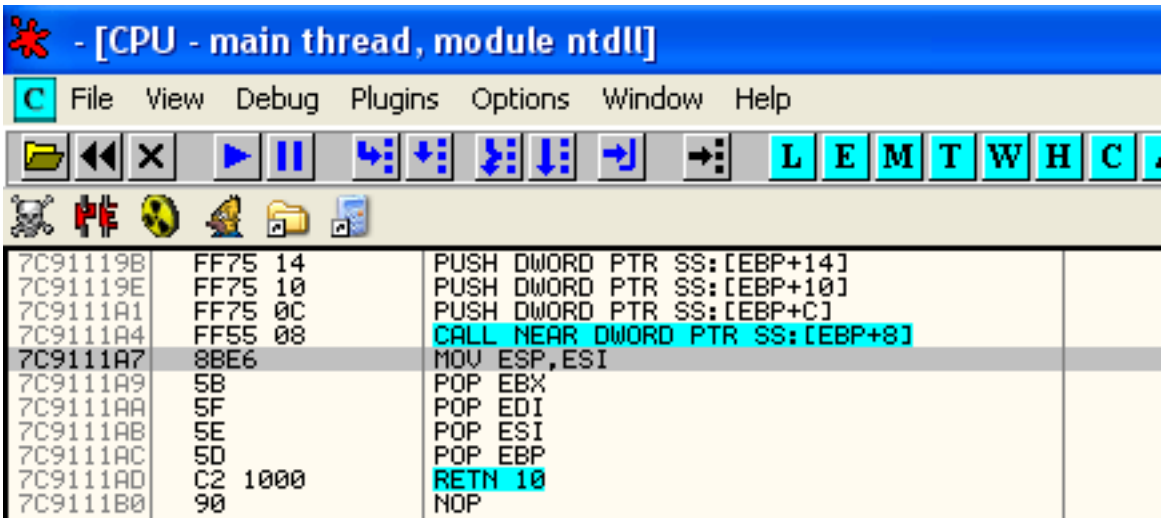
Здесь нам необходимо найти место, где останавливать выполнение второго процесса до того, как запустится DLL, но DLL ещё не загружена, можно сделать бесконечный цикл в этом же адресе, но во втором процессе, так что смотрим стек, чтобы выяснить, откуда попадаем в EP.



Идём, посмотрим эту область.

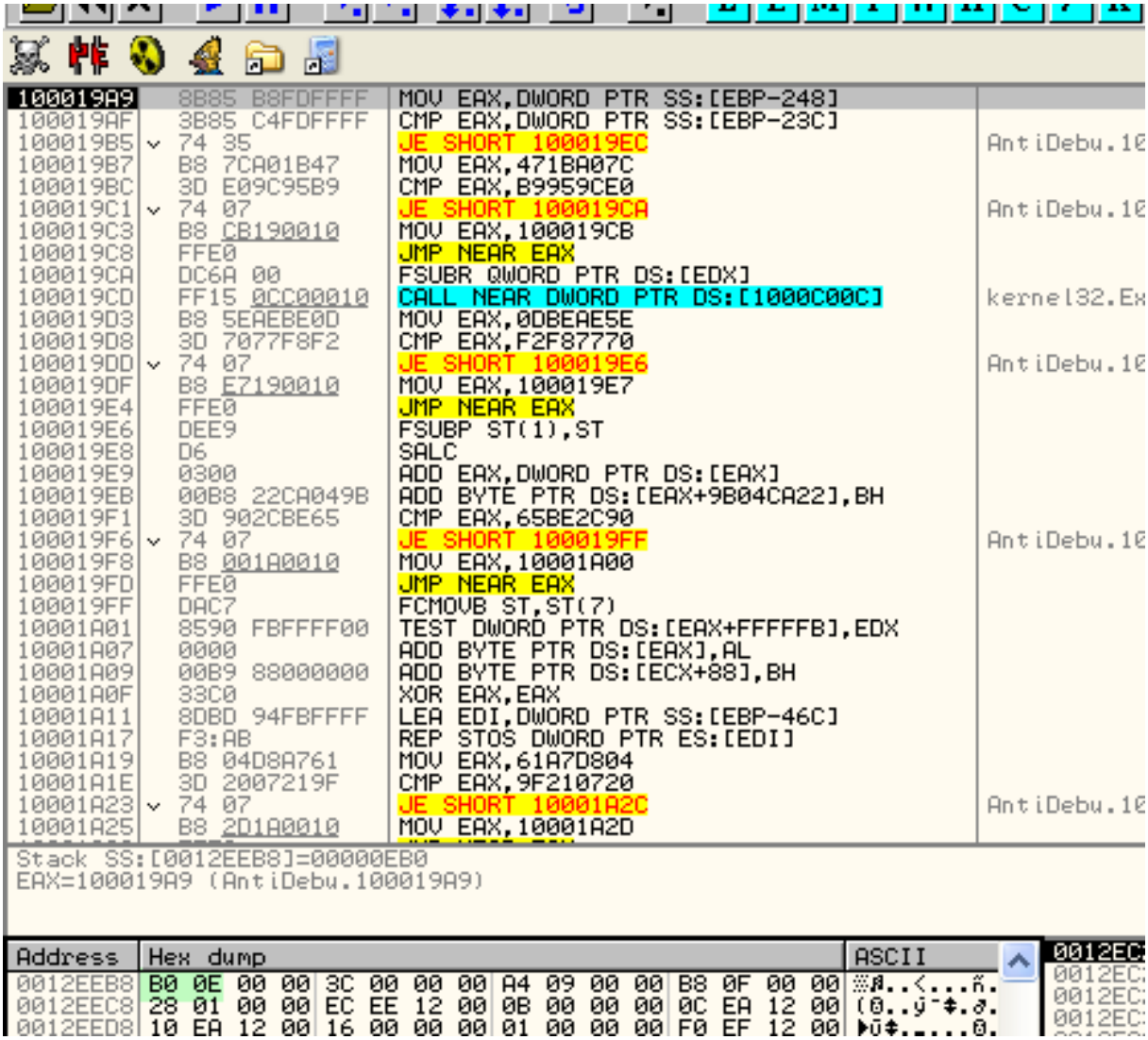


То есть этот вызов и есть тот, откуда совершается переход на точку входа DLL. Теперь здесь есть адреса, которые относятся к NTDLL.dll, той, которая запускается всегда, так что возвращаемся сюда, создаём замороженный процесс, и когда он заморожен, устанавливаем на этот вызов бесконечный цикл, чтобы когда он будет запущен снова, здесь было зациклено.

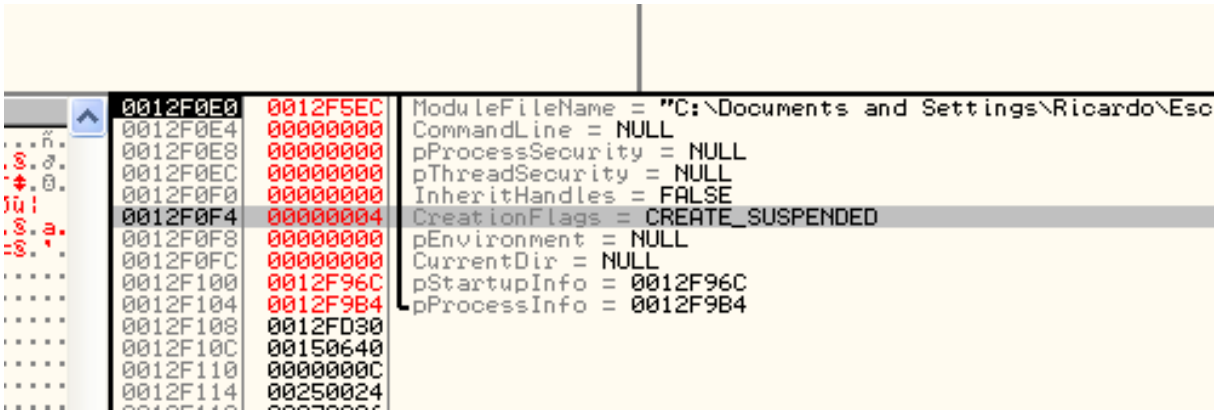


Так что отмечаем адрес. На моей машине для цикла он 7c9111a4, оригинальные байты равны FF 55.

Снова доходим до CreateProcessA, помним, что до остановки на HBP ON EXECUTION надо изменить PID эксплорера, чтобы нас не выкинуло.



Меняем PID parcheado 4 на PID эксплорера, а затем доходим до CreateProcessA.

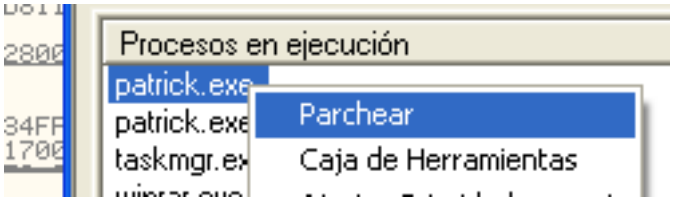


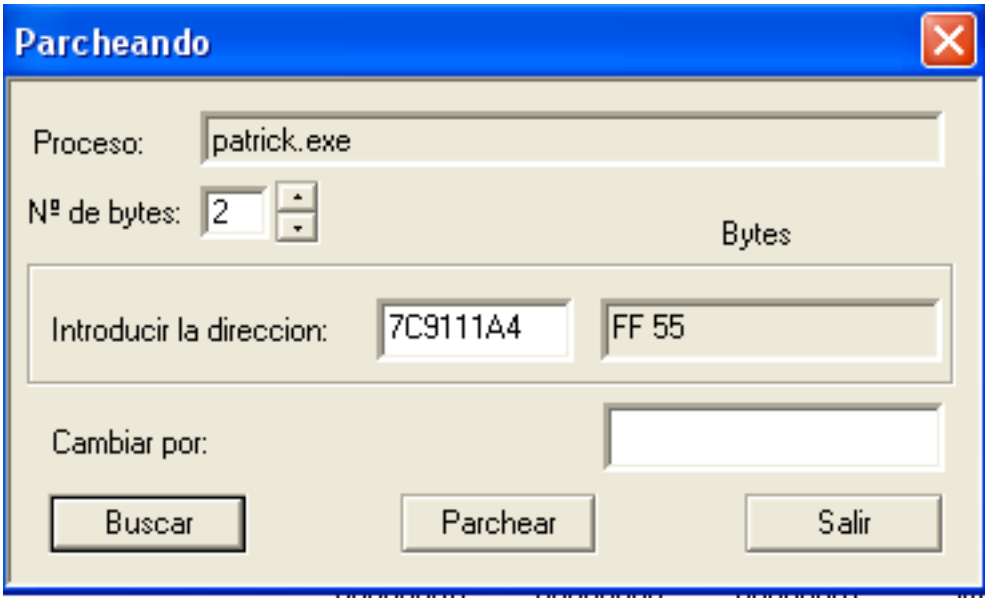
Заменяем 20 на 4 (CREATE_SUSPENDED).

Нажимаем F8, и успешно создаётся второй процесс.

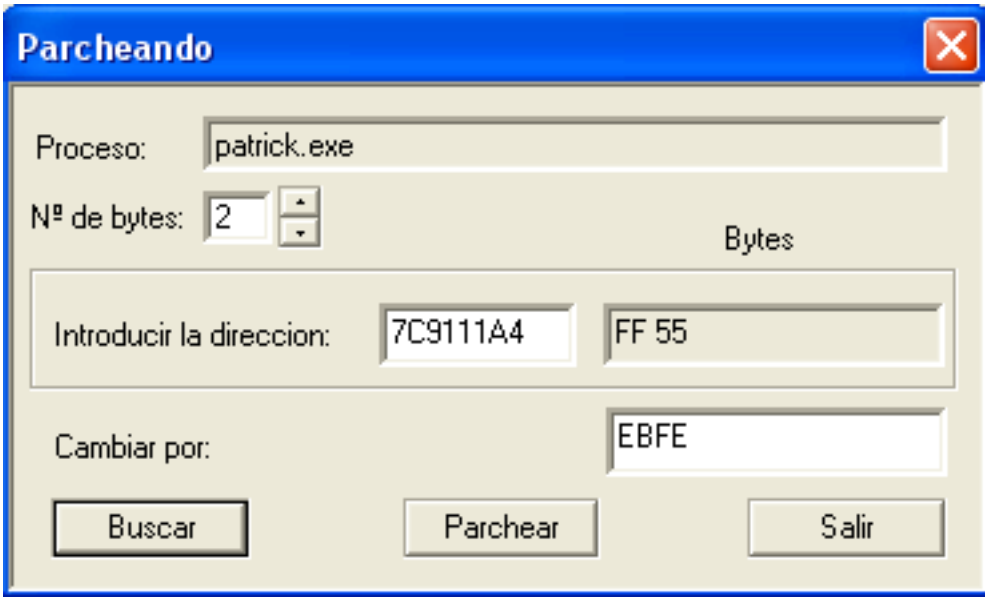
Вот оба процесса в PUPE.

Теперь выше открываем тот, что второй и выбираем “PARCHEAR” (патчить).



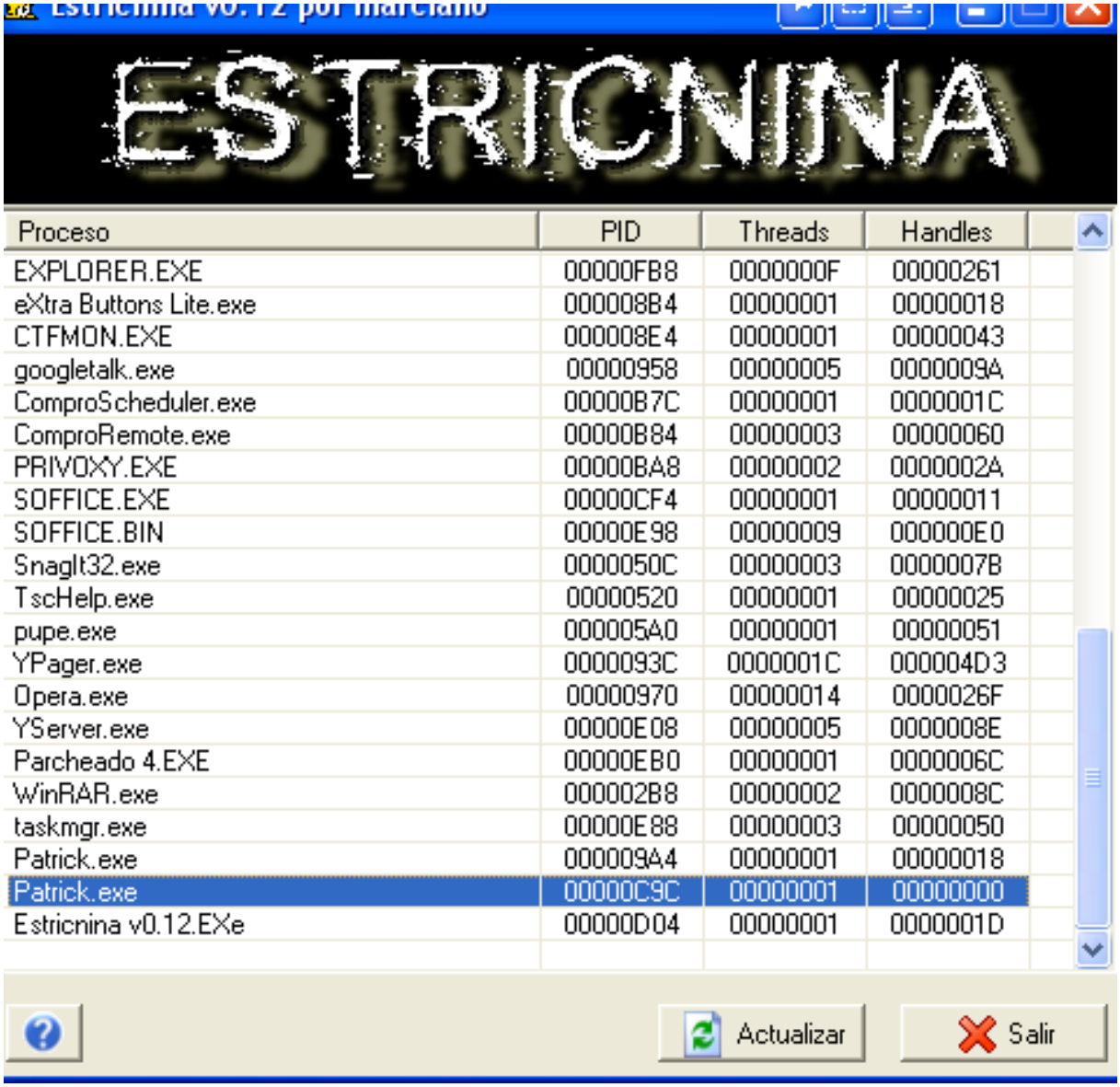


Пишем адрес, куда хотим добавить ЦИКЛ, и меняем оригинальные байты на EB FE, которые означают бесконечный цикл.

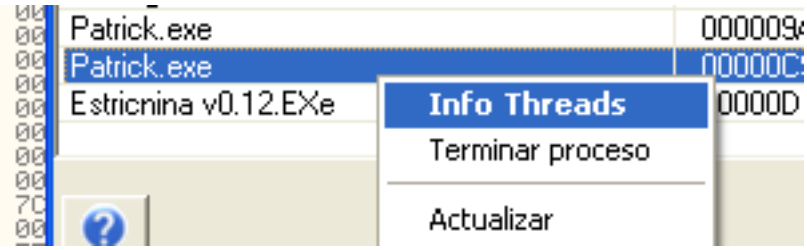


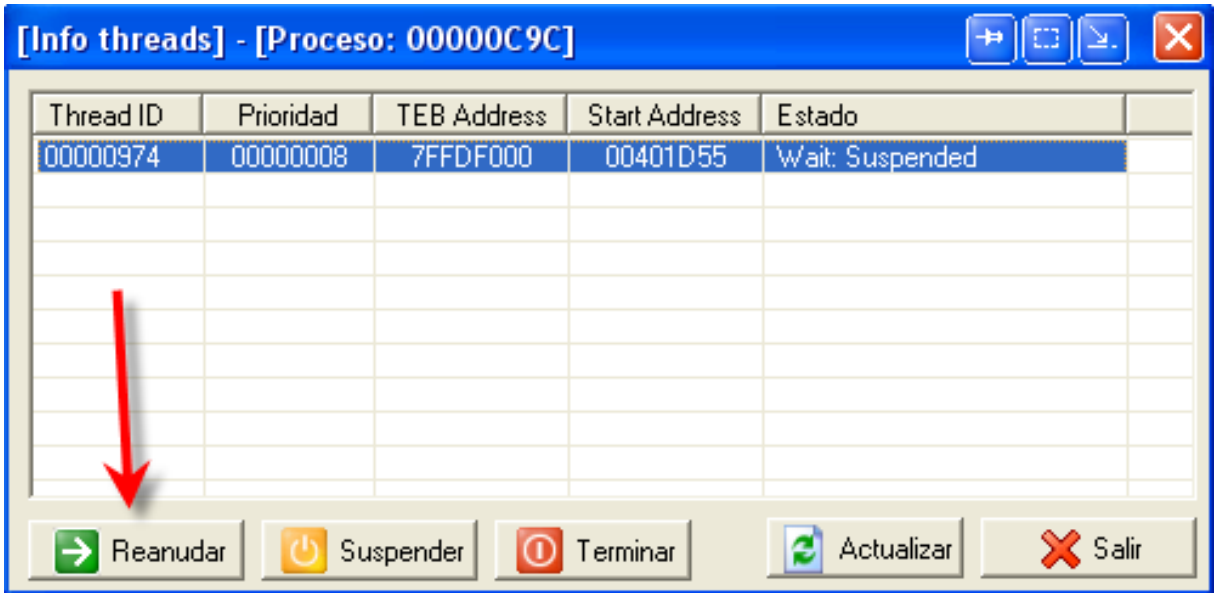
Затем нажимаем PARCHEAR, и в нужном месте установлен ЦИКЛ. Теперь до того, как приаттачиться к процессу, нужно снять задержку с процесса, чтобы он запустился и был зациклен.

Для этого используем прекрасную утилиту ARAPUMK’a, которая называется ESTRICNINA.



Хорошо, ищем по PID'у второй процесс, в данном случае вот он, и так как они здесь не по порядку, то нужно точно убедиться, какой из них является вторым. Делаем щелчок правой кнопкой мыши и выбираем "INFO THREADS".

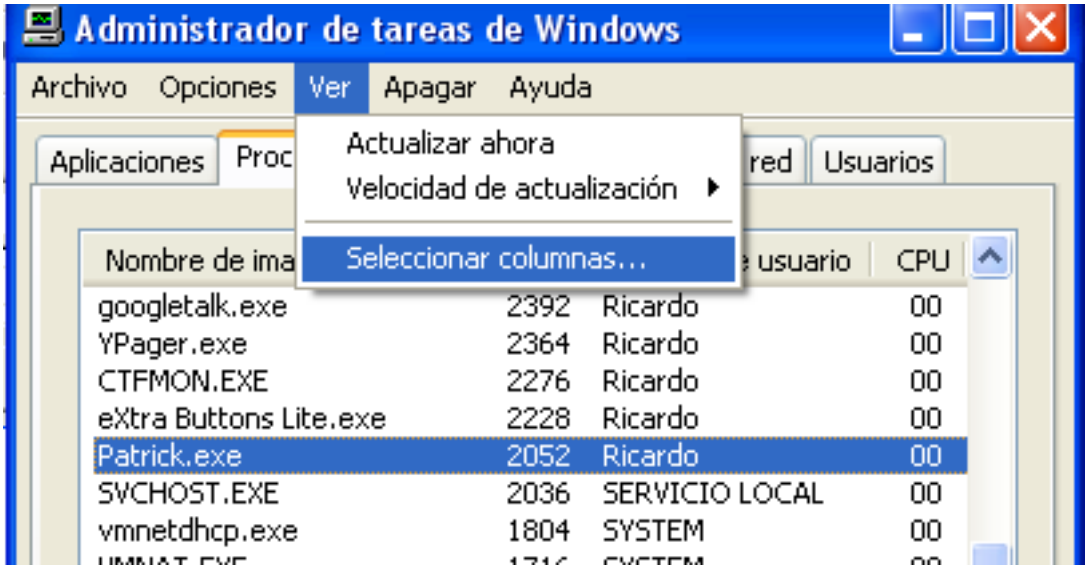


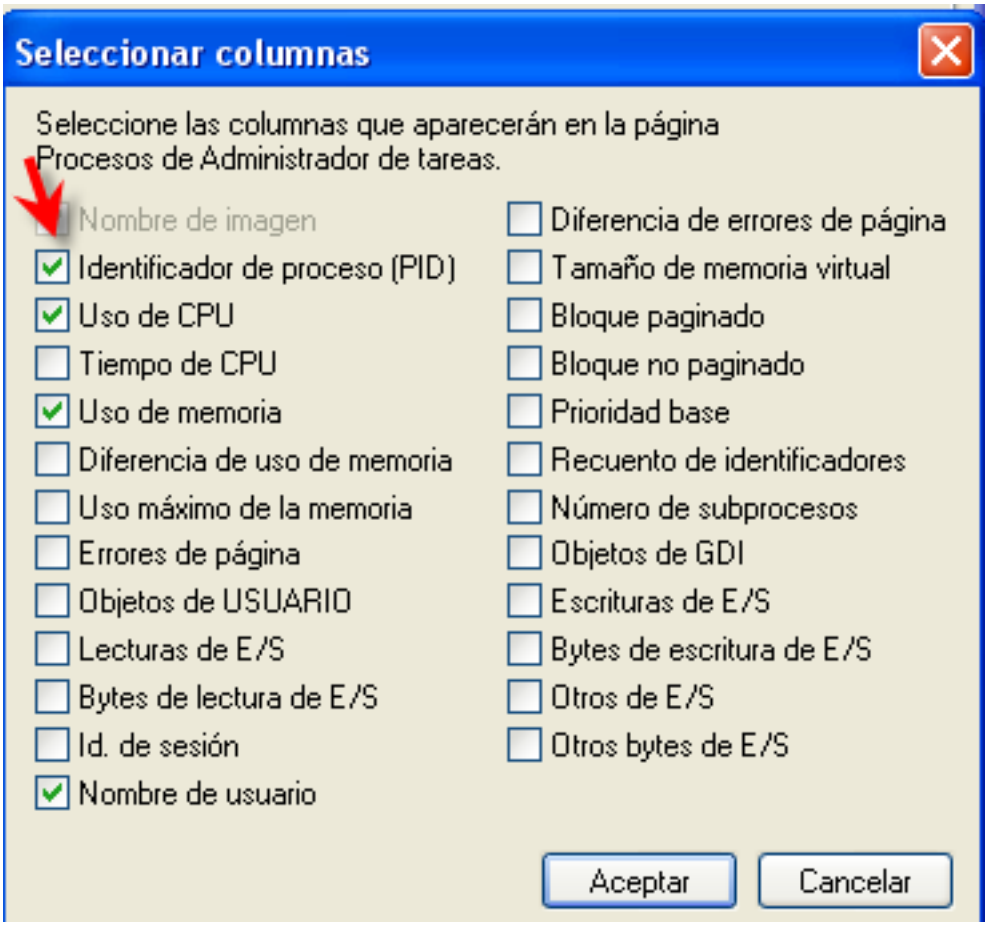


И нажимаем “REANUDAR”.

Теперь замороженный процесс запустился и зациклился, так что может присоединиться к нему.

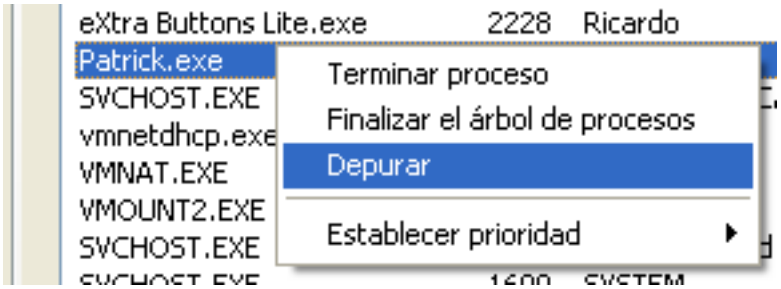
Для этого можно попробовать присоединиться напрямую с помощью функции attach отладчика, но когда я так делал, мне не отображались модули и не завершалось выполнение, может что-то с моим компьютером или с HideOD, поэтому используем следующий способ для присоединения. Установим PARCHEADO 4 как JIT, сделаем CTRL +ALT+DEL и выберем второй процесс patrick’a. Обратите внимание, что PID’ы не показываются, поэтому надо сделать так, чтобы отображалась колонка с ними.



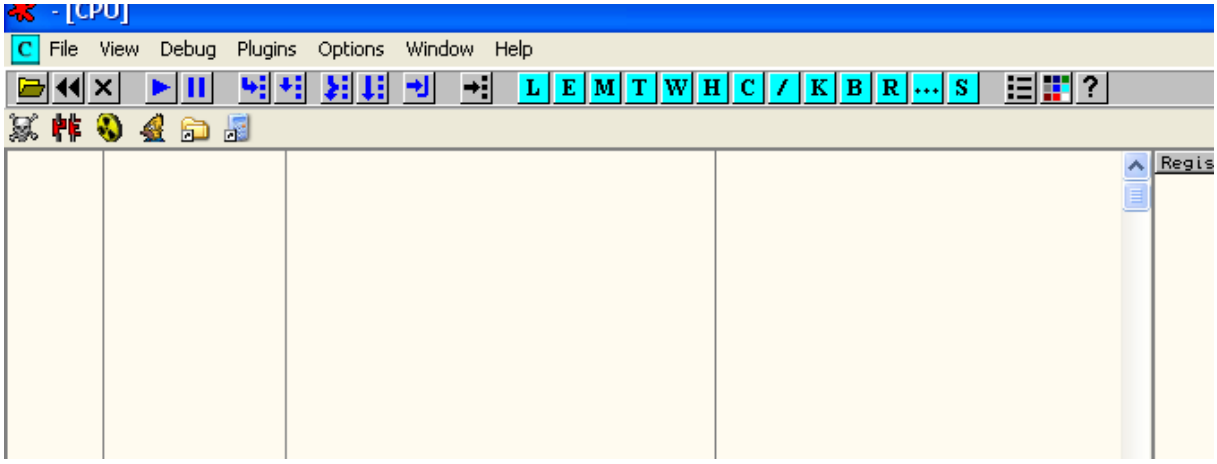


Теперь PID'ы видны, плохо то, что эти PID'ы в десятеричной системе, но ладно, в PUPE просто посмотрим, какой из двух больше, чтобы найти второй процесс.

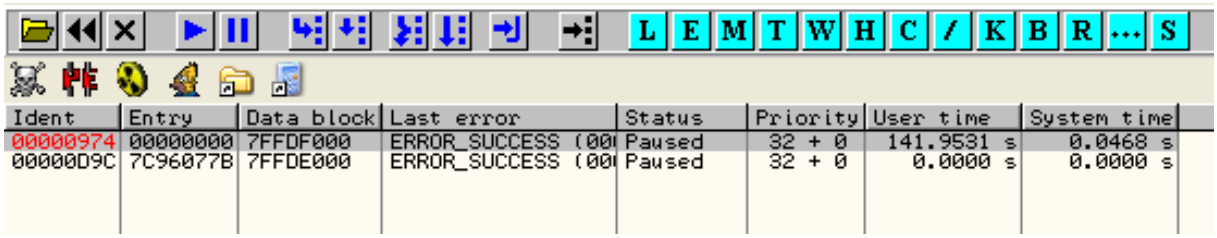
В моём случае у второго процесса PID больше, так что ищем самый высокий PID в списке процессов и выбираем DEPURAR («очистить»).



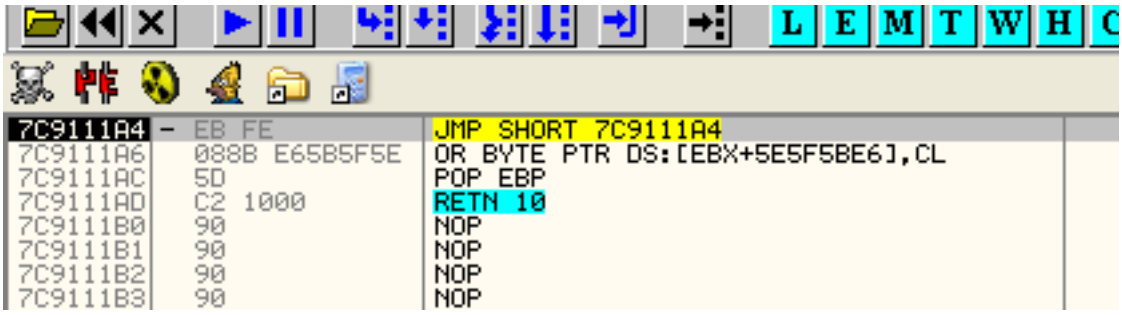
Здесь остаётся запущенным, затем делаем паузу.



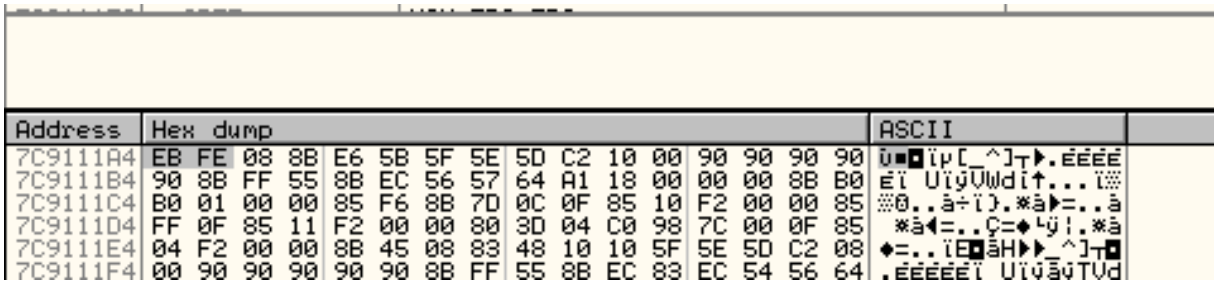
Похоже, что в OllyDbg ничего нет, но не пугайтесь, идём в T, то есть THREADS.



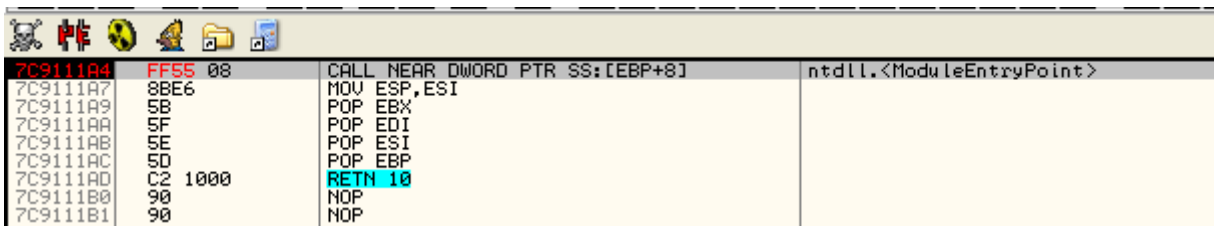
И делаем двойной щелчок на одной из двух ветвей. Находимся в бесконечном цикле, созданном нами ранее. Сделаем на нём щёлчок и не находимся в цикле. Попробуем по-другому.



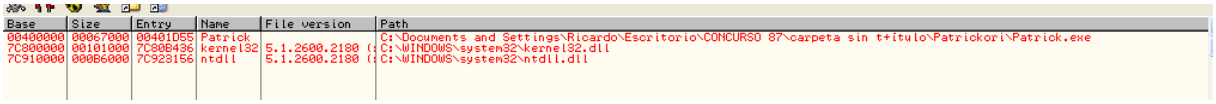
Аа, находимся здесь, теперь посмотрим эту часть в дампе, чтобы выйти из ЦИКЛА.



Оригинальные байты равны FF 55. Восстановим их.

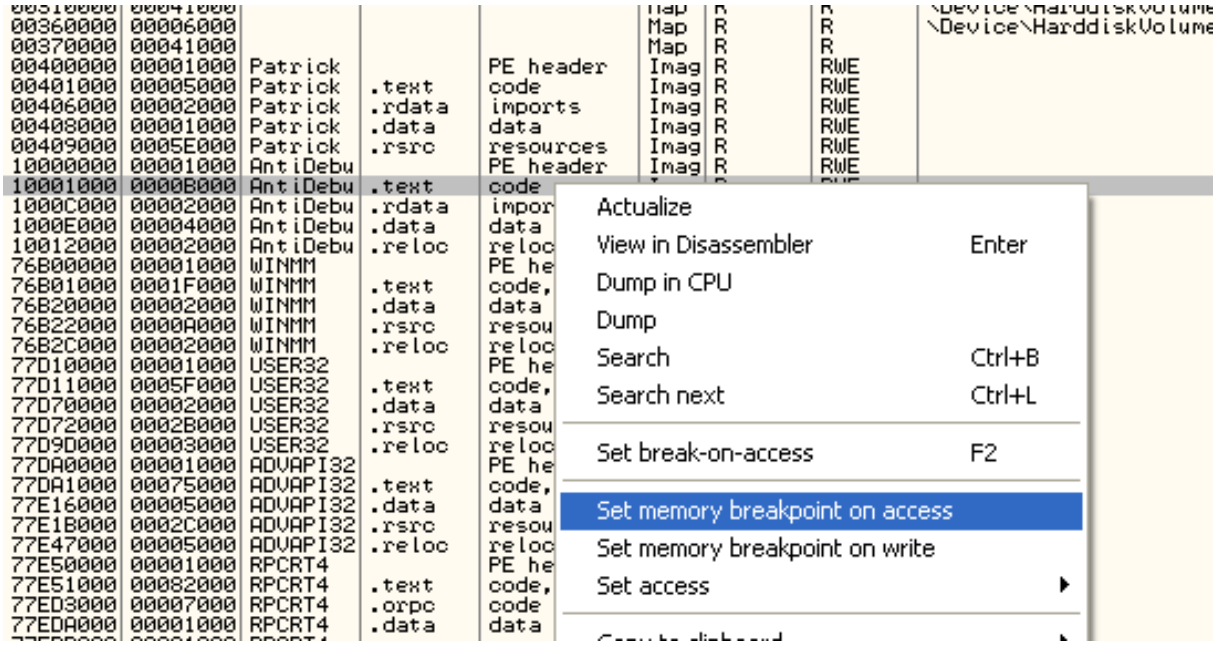


Если присоединились нормально, то смотрим в окне E (для исполняемых файлов) какие уже загруженны. Если там пусто, то возможно, что не удалось подсоединиться.

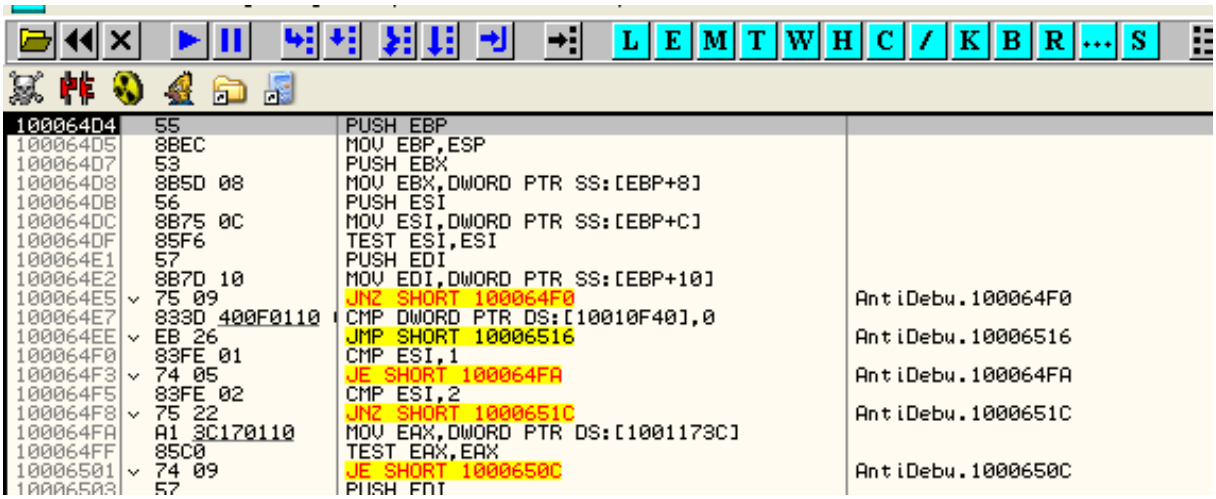


Видим, что хотя даже нет перехода на DLL, поэтому видно, что несколько раз происходит остановка на одном и том же адресе, так что устанавливаем BP и делаем RUN.

Каждый раз, когда происходит остановка, смотрим, не появилась ли antidebugdll.dll в списке модуле. Сейчас её видим, так что можем установить BPM ON ACCESS на её секцию кода.



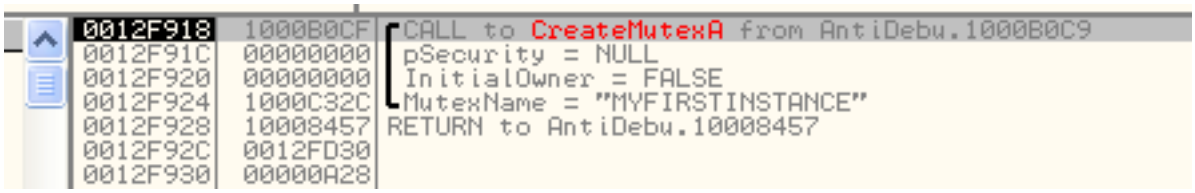
Устанавливаем BP и делаем RUN.



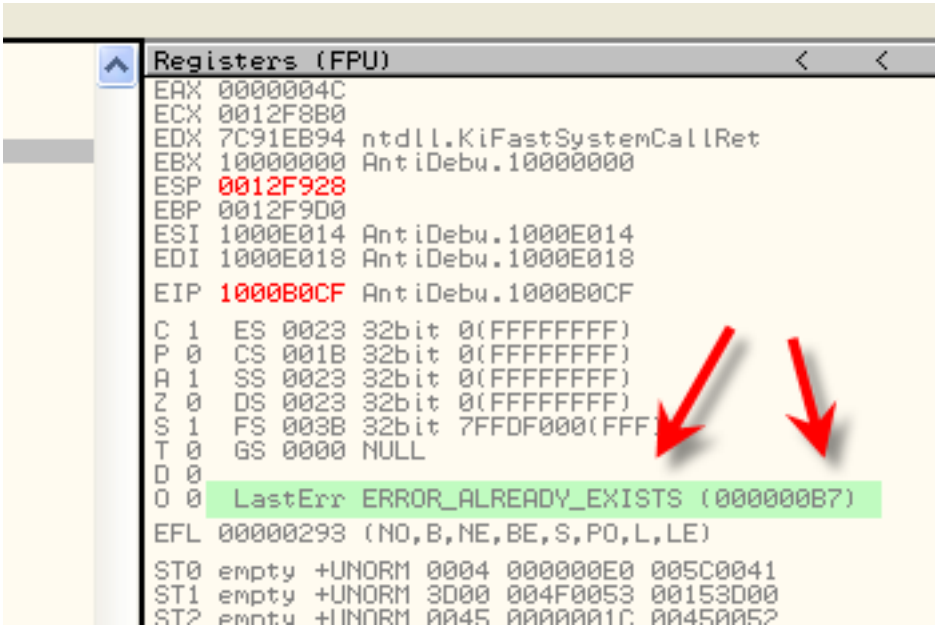
И останавливаемся на точке входа DLL. Уф, какой дикий напряг знать, что ничего из этого не нужно для решения этого крэкми, хе-хе, но всё это является практикой, которая нужна для овладения мастерством.

Попробуем поставить BP CreateMutexA, чтобы понять, что там происходит с мутексами и к какому процессу всё это относится.

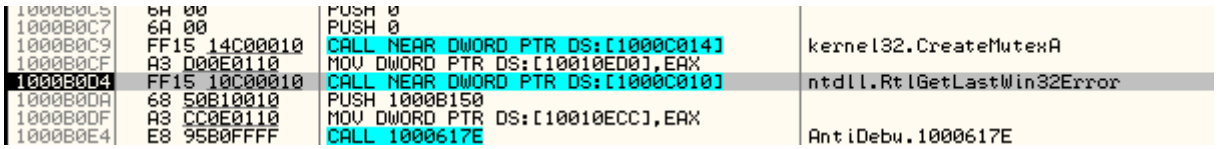
И делаем RUN.



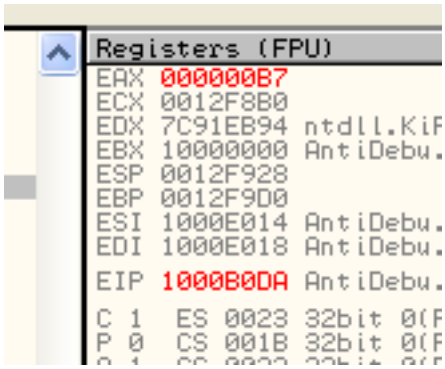
Видим, что остановились на создании мутекса “MY FIRST INSTANCE”, но это так как он уже был создан первым процессом, так что идём до RET.



Здесь видим, что когда первый процесс создавал MUTEX, который не существовал, результатом было ERROR_SUCCESS, а значение было равно нулю. Здесь же результат равен ERROR_ALREADY_EXISTS, то есть “уже существует” и возвращаемое значение равно B7.



Конечно, далее идёт API-функция, которая считывает последнюю ошибку, и после её срабатывания видим, что EAX = B7.



То есть теперь видим что разница между тем, когда процесс запускается в первый раз и когда во второй, заключается в разных ошибках при создании мутексов.

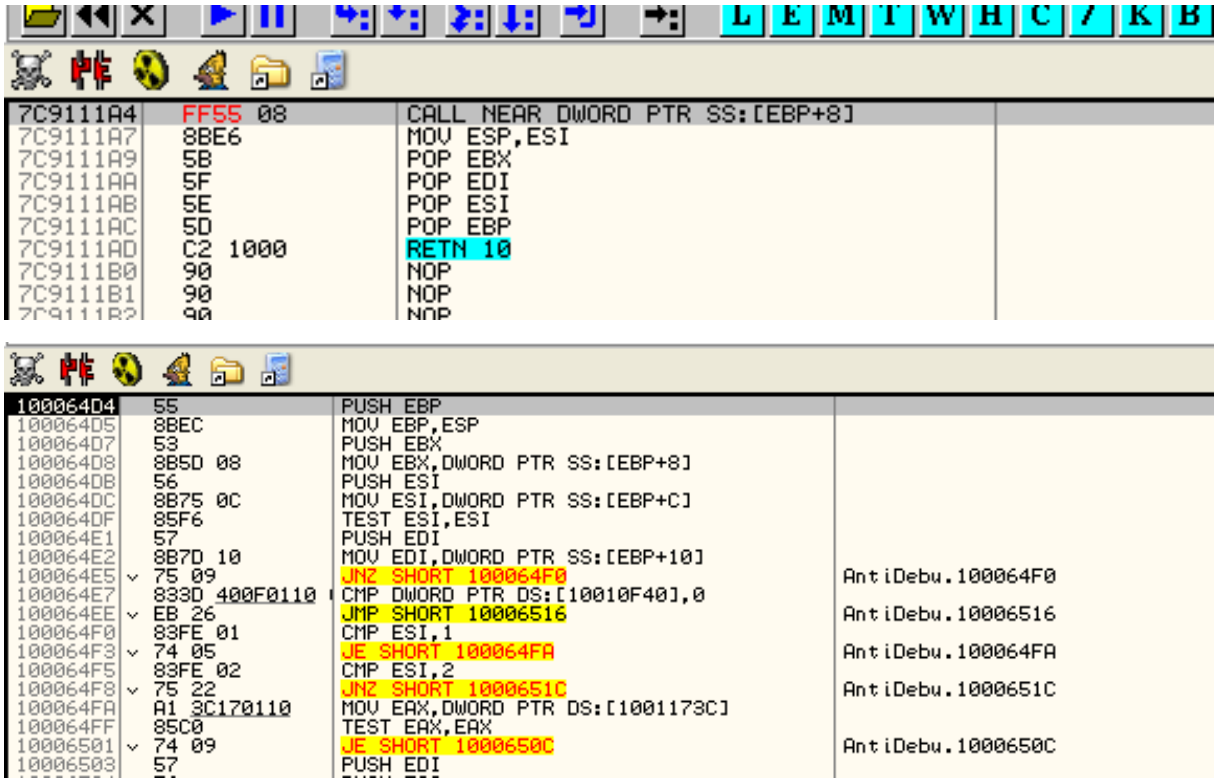
Ок, в общем, мы подошли к концу этой части. Нам ещё осталось проанализирвоать, что делать, когда запускается второй процесс, просто для изучения, так как для решения этого крэкми ничего из этого не нужно – всё это для обучения, а также, потому что я люблю всё потрошить до самого конца.

Введение в крэкинг с нуля, используя OllyDbg - Глава 47

Автор: Рикардо Нарваха, пер. Рома Стремилов

Дата: 2010-04-25

Продолжим с Patrick'ом. Когда я дошел до места, на котором мы остановились в предыдущей главе, то заметил, что после восстановления оригинальных байтов команды, где был бесконечный цикл для присоединения, было бы хорошо вместо обычного BP установить на эту инструкцию аппаратный брейк, поскольку обычный обнаруживается и DLL'ка не запускается. Установка HE обеспечивает остановки не хуже, а после появления в списке загруженных модулей AntiDebugDll.dll можно установить в ее секции кода BPM ON ACCESS и оказаться на ее Entry Point:



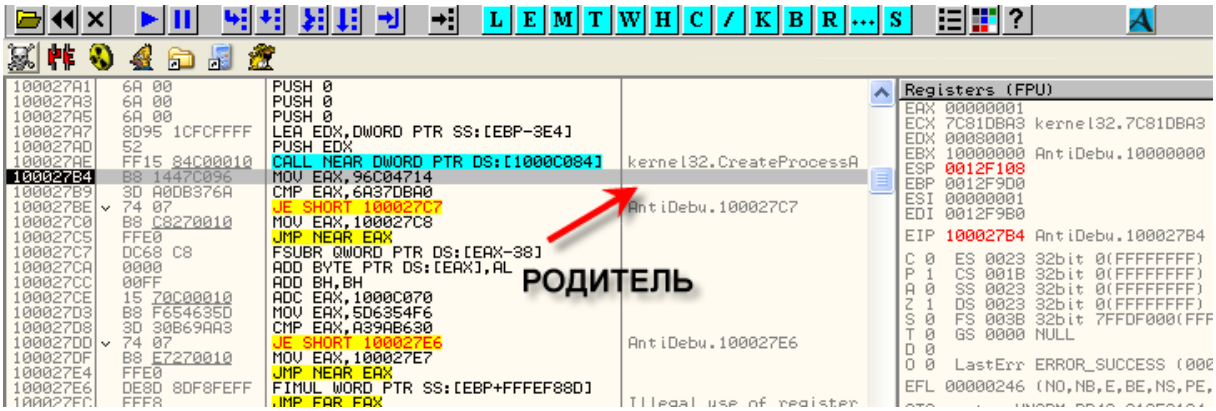
Это Entry Point DLL'ки. В данный момент все модули загружены. Теперь можно приступать, но продвигаться следует постепенно, шаг за шагом.

Наша задача — оказаться в OEP крэкми с помощью OllyDbg, а это сделать, как уже упоминалось, сложнее, чем с помощью способа, не использующего Олли. Но в данном случае мы пользуемся именно Олли, поэтому оставим тот способ на потом и продолжим наш сложный путь сквозь патрицианскую защиту, как сказал Jack.

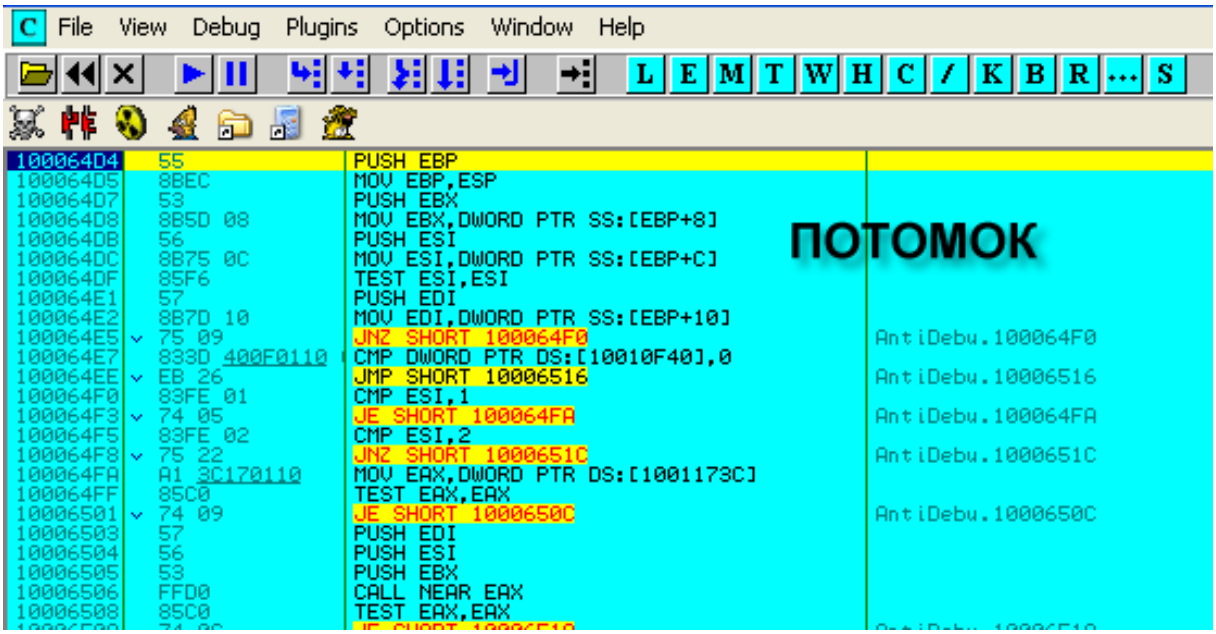
Давайте снова попадем в то место, где создается второй процесс: вначале изменим PID Олли на PID Проводника, затем, дойдя до CreateProcessA, изменим параметр dwCreationFlags для заморозки процесса и выполним эту функцию до ее команды RET. Далее установим бесконечный цикл в ntdll.dll, как это было описано в предыдущей главе. Присоединимся ко второму процессу. Затем, после снятия бесконечного цикла, установим на той же строке аппаратный брейк. Произведя нескольких остановок, дождемся загрузки AntiDebugDll.dll, после чего установим BPM ON ACCESS в секцию ее кода — и окажемся в ее EP. Всё это было проделано в предыдущей главе. Теперь открыты два OllyDbg: в одном отлаживается первый процесс, который мы назовем РОДИТЕЛЕМ (он находится в точке возврата из API-функции CreateProcessA), а в другом — второй процесс,

остановленный на EP AntiDebugDll.dll, пусть он будет ПОТОМКОМ.

Если вам что-либо из этого не понятно, то перед продолжением исследования крэкми стоит повторить пройденный материал и попрактиковаться, так как далее будет не проще (даже сложнее, хе-хе).



Таким образом, в родителе мы остановились на возврате из API-функции CreateProcessA.

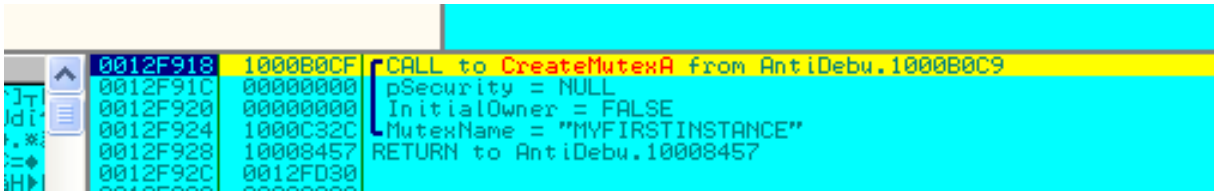


А в потомке — на EP AntiDebugDll.dll. Чтобы видеть, что это именно потомок, в этом отладчике желательно установить другую цветовую схему. Так я делаю не только для того, чтобы читателям было понятнее, какой из двух процессов я имею в виду — это и мне самому помогает не запутаться при взломе.

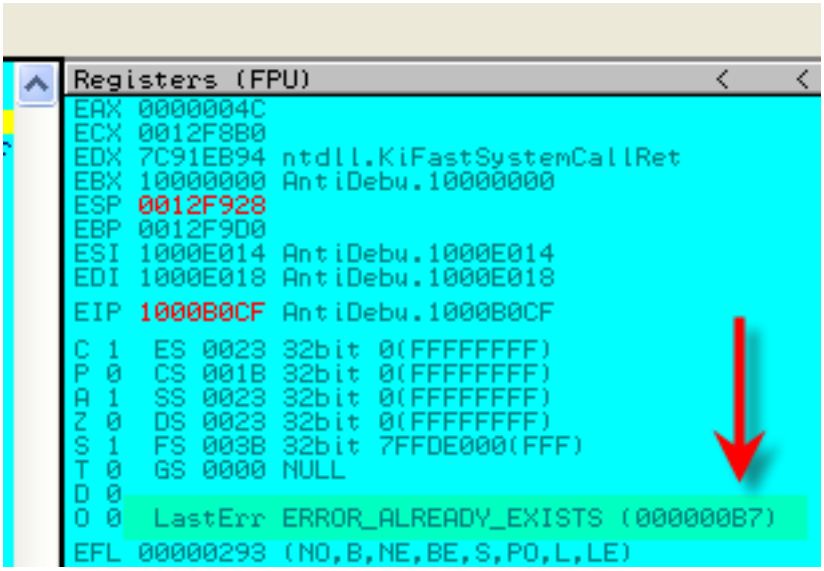
Теперь, по идее, нужно эмулировать одновременное выполнение двух процессов, но это невозможно. Однако известно, что процесс выполняется до места, где он ожидает действий другого, поэтому можно довести один процесс до состояния ожидания, а далее продолжать работать с другим.

Мы начнем с потомка. Как известно, если при его запуске он замечает что-либо подозрительное, то через несколько секунд закрывается. Следовательно, перед этим должна осуществляться проверка, какой это процесс: первый или второй, родительский или дочерний. В результате процесс станет выполнять соответствующую роль родителя или потомка. Важно понять, что, хотя эти процессы похожи (так как начинают выполняться с одной и той же строки), но, тем не менее, первый обнаруживает, что это первый процесс, а второй — что второй, поэтому они решают различные и даже противоположные задачи.

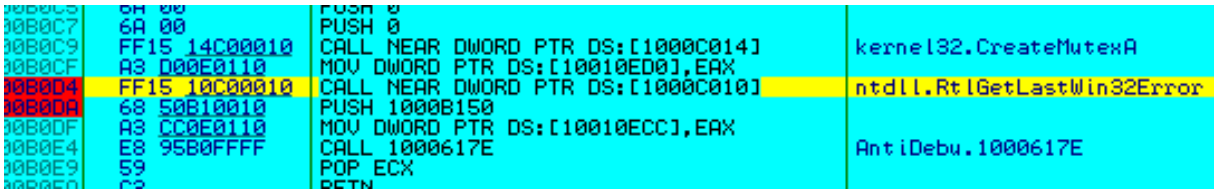
Как мы уже знаем, с помощью API-функции CreateMutexA проверяется, первый ли это запуск процесса, поэтому установим на нее BP в потомке.



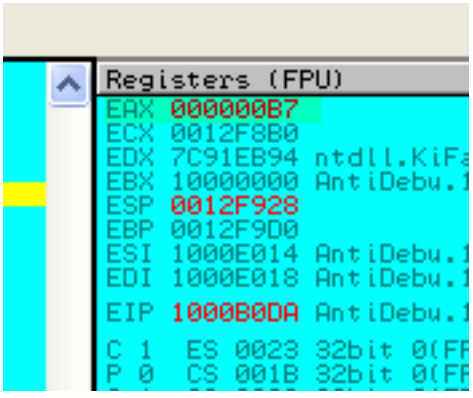
Остановка произошла здесь. Родитель уже выполнял эти действия и сейчас мьютекс уже существует, поэтому API-функция теперь должна возвратить 0B7 (ERROR_ALREADY_EXISTS), а не 00 (ERROR_SUCESS), как это было при создании мьютекса родителем. Дойдем до команды RET API-функции:

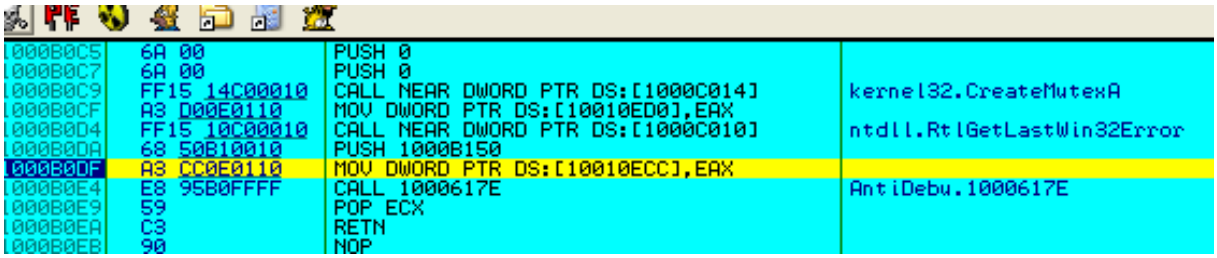


Код возвращенной ошибки говорит, что это была попытка создания уже созданного родителем мьютекса. Важно понимать, что когда программа обнаружит код спровоцированной ошибки B7, возвращенной API-функцией, она начнет выполнять совсем другую роль, нежели родитель, поэтому данный процесс надо будет рассматривать как отдельный процесс-потомок.

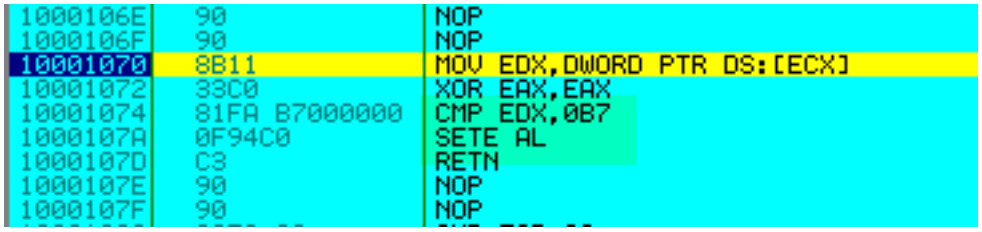


Теперь вызывается API-функция, возвращающая последнюю произошедшую ошибку, и программа получает значение B7, которое отладчик уже показал, т.е. ERROR_ALREADY_EXISTS.

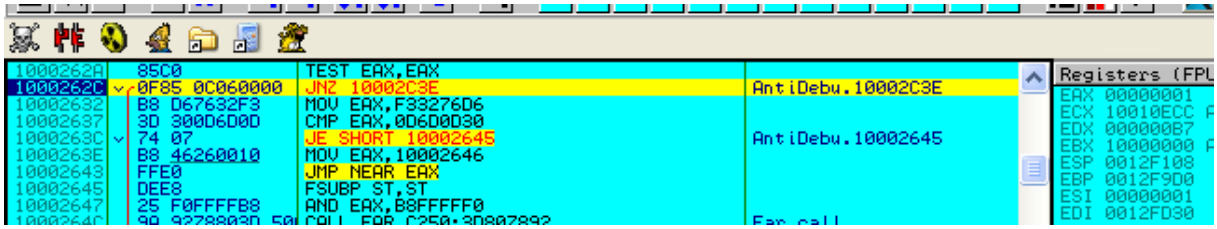




Затем это значение сохраняется. Если установить BPM ON ACCESS на ту ячейку памяти, где оно сохраняется и нажать RUN, то остановка произойдет в момент его считывания перед сравнением.



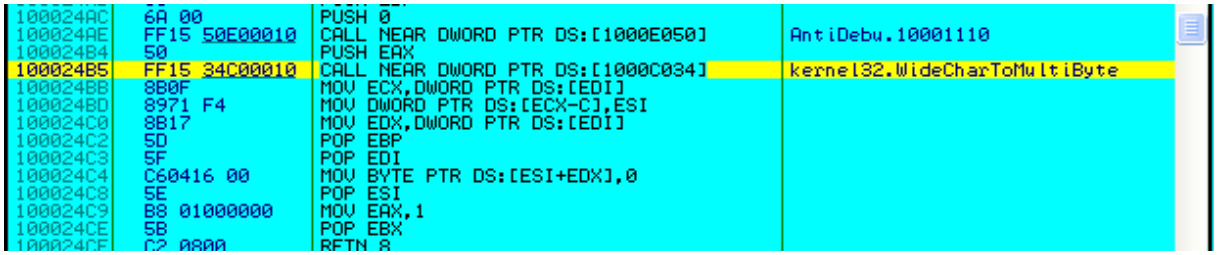
Здесь сохраненный код сравнивается с 0B7.



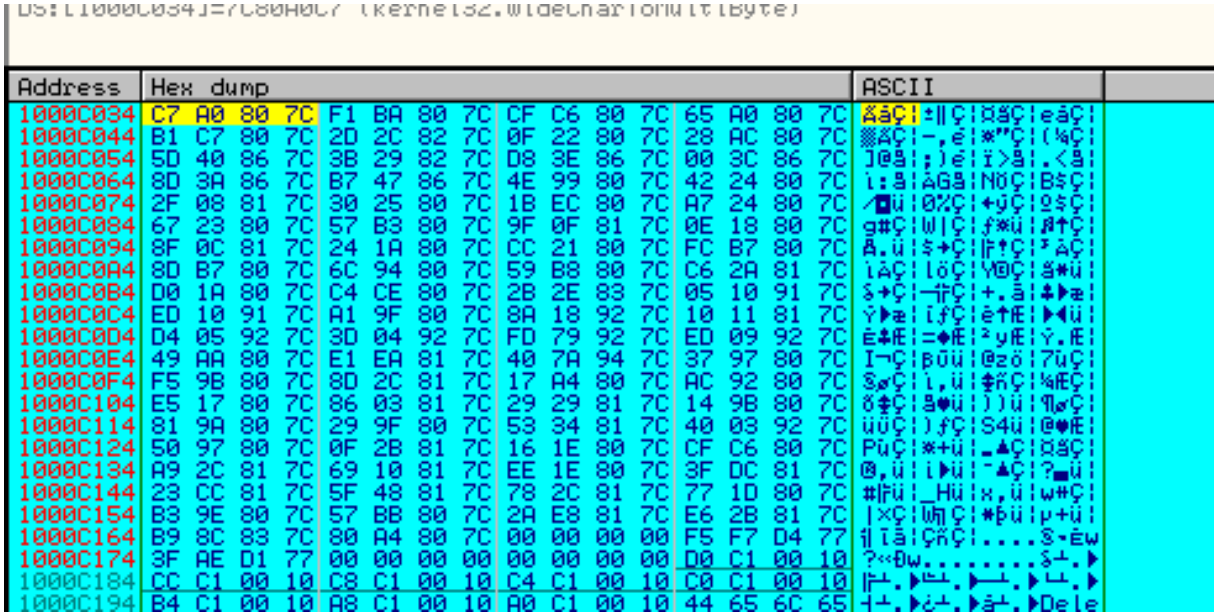
При истинном результате сравнения команда SETE поместит в EAX единицу. Далее идет решающий переход: если регистр EAX окажется ненулевым, то процесс станет потомком, иначе — родителем. В данном случае переход выполняется, поскольку это дочерний процесс. Если бы это был родитель, то переход бы не состоялся. Таким образом, здесь ход выполнения процессов разветвляется: до сих пор они делали одно и то же, а теперь родителю надо будет подтвердить, что он был создан Проводником и будет вызвана CreateProcessA для создания потомка. А то, что будет делать потомок, мы рассмотрим далее.

Для отслеживания API-функций существуют весьма полезные утилиты, подобные Kam и APISpy. Они логируют используемые в программе API-функции и помогают избежать лишней трассировки. Давайте попытаемся сделать собственный Kam с помощью Olly, что позволит нам переходить от API-функции к API-функции, сэкономит время и поможет в изучении таких насыщенных программ, как наш крэкми.

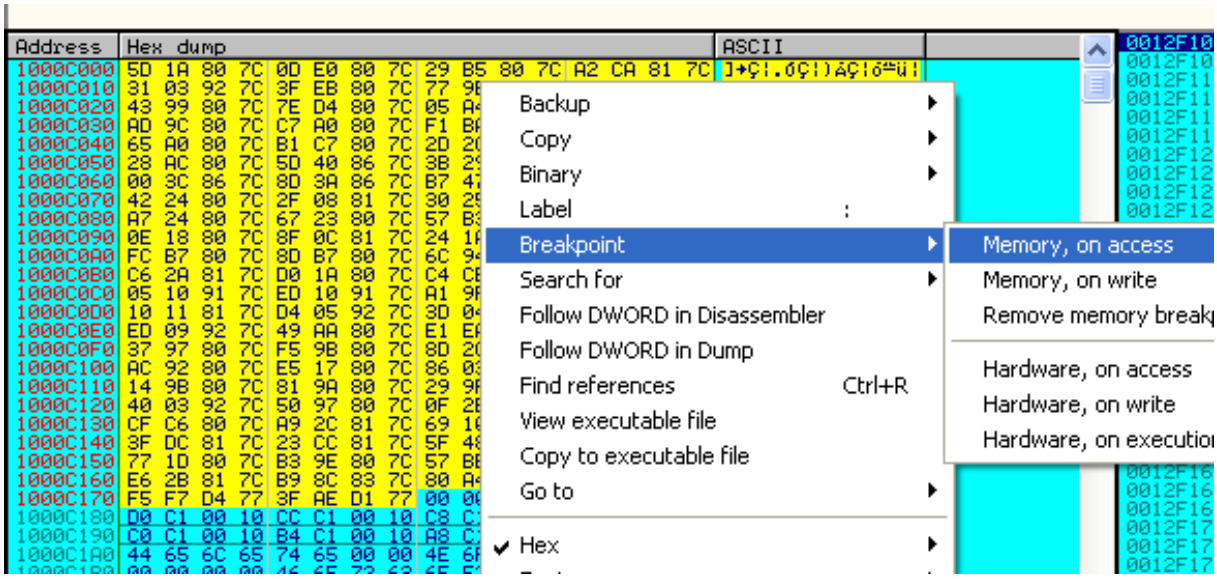
Прежде всего, надо в потомке найти IAT DLL'ки. Не трассируя, посмотрим в листинге выше команду CALL, вызывающую какую-либо API-функцию:



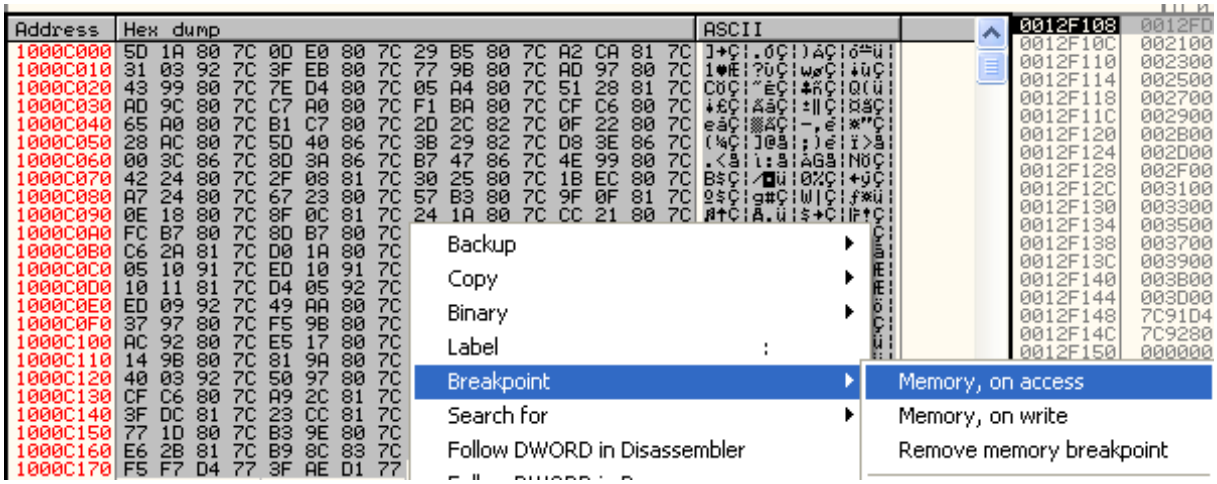
Вот подходящий вызов: он берет значение из адреса 1000C034, который должен соответствовать элементу IAT'а. Посмотрим его в DUMP'е:



Это IAT; после нахождения ее начала и конца установим BPM ON ACCESS на всю таблицу:

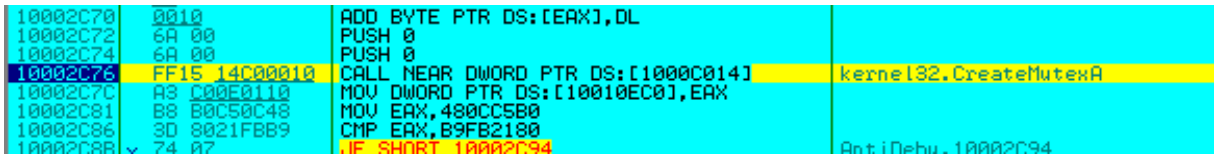


Сделав то же самое в родителе, получим второй собственный Kam. Конечно, IAT там находится по тем же адресам:



Таперь у нас есть два собственных Kam'a. Следует заметить, что они позволяют останавливаться на API-функциях из IAT'a DLL'ки. В случае с API-функциями не из IAT'a, полученными с помощью GetProcAddress, надо будет установить HARDWARE BPX ON ACCESS куда потребуется. Важно, что это работает с API-функциями из IAT'a, где наш Kam будут функционировать без нужды в чем-либо еще, хе-хе.

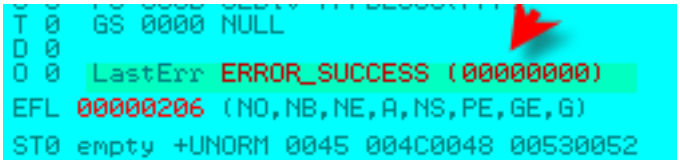
Вернемся к потомку, нажмем F9 и посмотрим, в какой API-функции он остановится. Кроме того, при каждой остановке будем смотреть параметры, передаваемые функции.



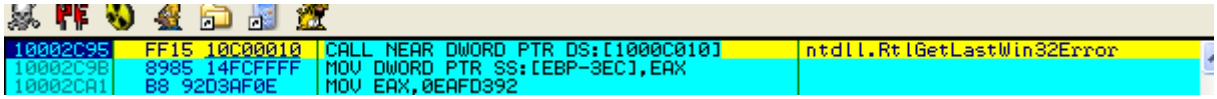
Однако, это избавило нас от часов трассировок: первая же остановка в потомке произошла снова в CreateMutexA при создании еще не существующего мьютекса:

Если вспомнить, то название рассмотренного нами ранее мьютекса, созданного родителем, было MYFIRSTINSTANCE, что означает: «мой первый экземпляр». А новый мьютекс называется MYMAININSTANCE, что значит: «мой главный экземпляр»; он создан потомком при выполнении программы, поэтому главный.

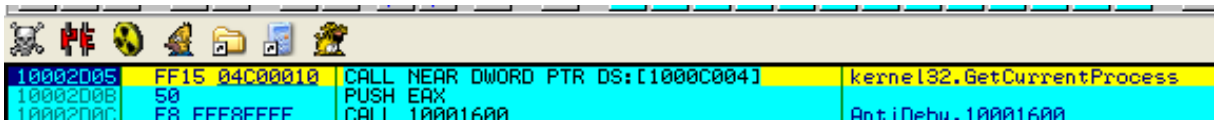
Нажмем F8, чтобы пройти API-функцию:



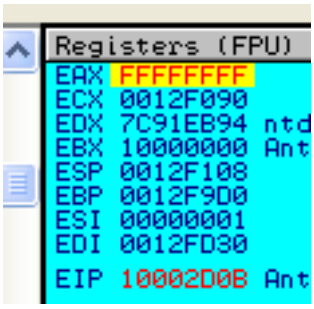
Не думаю, что результат этой второй проверки в процессе потомка на что-то влияет. Так как больше процессов не создается, мы не будем смотреть сравнение значений — в данном случае это не столь важно. Лучше нажмем F9 и перейдем с помощью нашего OllyKam'a к следующей API-функции, хе-хе.



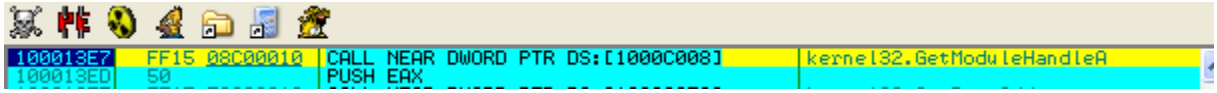
Эта функция возвращает последнюю ошибку, произошедшую в результате вызова CreateMutexA. Нажмем F9 и остановимся на очередной API-функции:



Чтобы можно было произвести какие-либо изменения в текущем процессе, надо получить его псевдохэндл. Данная API-функция возвращает FFFFFFFF, то есть хэндл текущего процесса (или псевдохэндл). Нажмем F8:

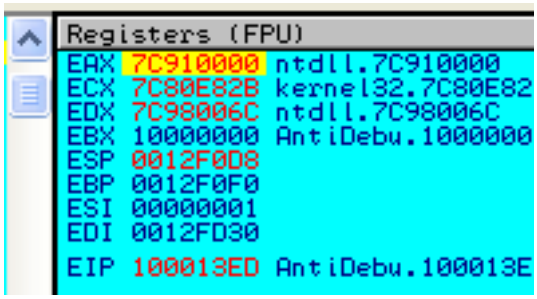


А теперь — F9:

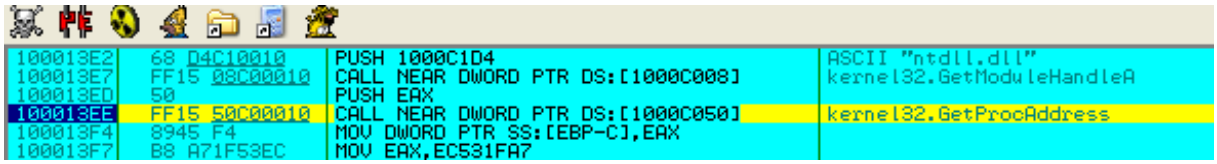


Для тех, кто не знаком с данными API-функциями, это будет хорошим упражнением. Если бы мы были на экзамене, а я был учителем, то я бы обязательно спросил: «Что возвращает эта API-функция?» Нам необходимо знать это до такой степени, чтобы и без помощи Win32 API, а только увидев это, мы и без нажатия на F8 могли понять, что здесь происходит, хе-хе.

Поскольку параметр функции — ntdll.dll, то после нажатия на F8 получим основание этой DLL'ки, которое затем станет параметром GetProcessAddress при поиске адреса новой API-функции.

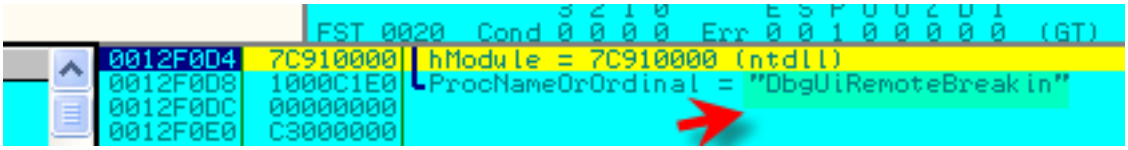


Надеюсь, я был прав. Нажмем F9:

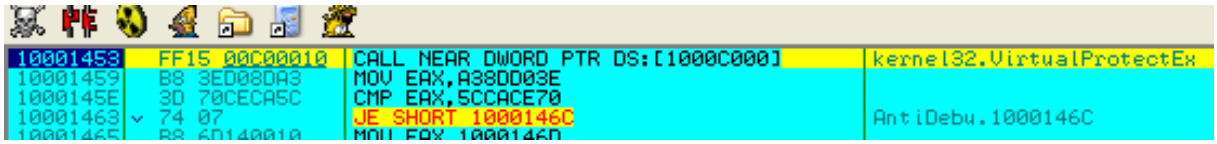


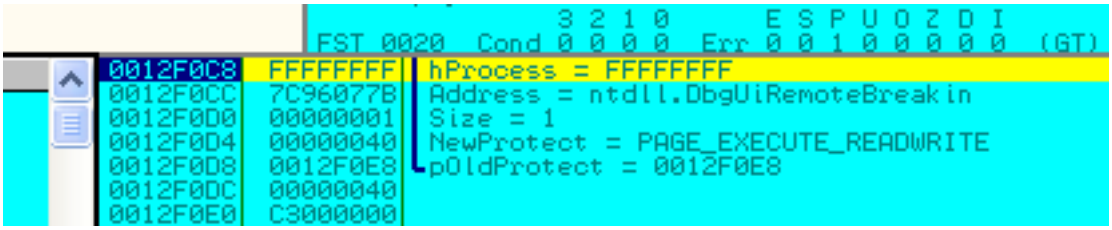
Ну, посмотреть чуть ниже было не так трудно, хе-хе.

Вот параметры функции:

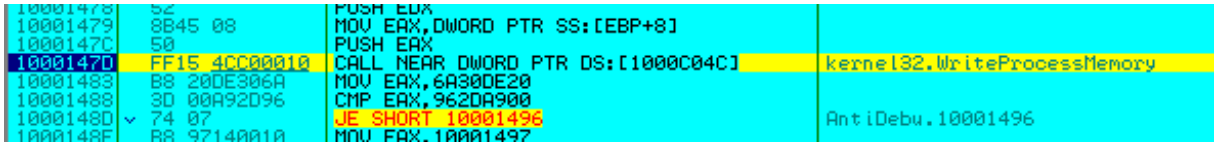


Хмм, этот вызов даст программе адрес указанной API-функции. Снова F9:

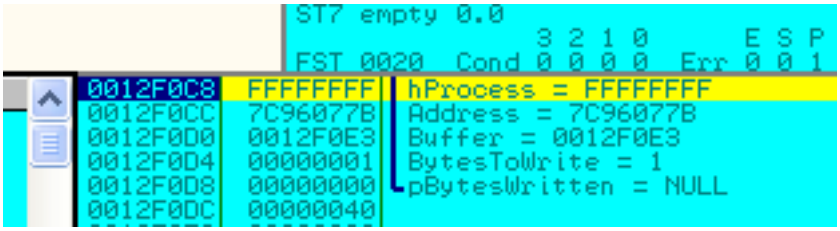




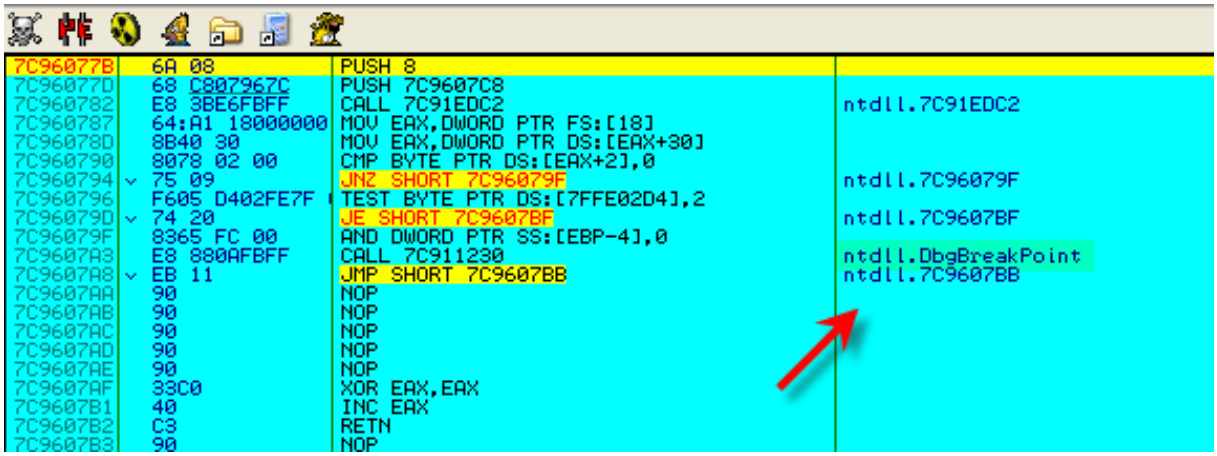
Итак, поскольку здесь пропатчивается API-функция того же самого процесса, то это реализация части системы антиотладки. Для тех, кто желает изучить данный вопрос глубже, можно сказать так: здесь изменяются настройки доступа упомянутой API-функции, чтобы в ней можно было писать.



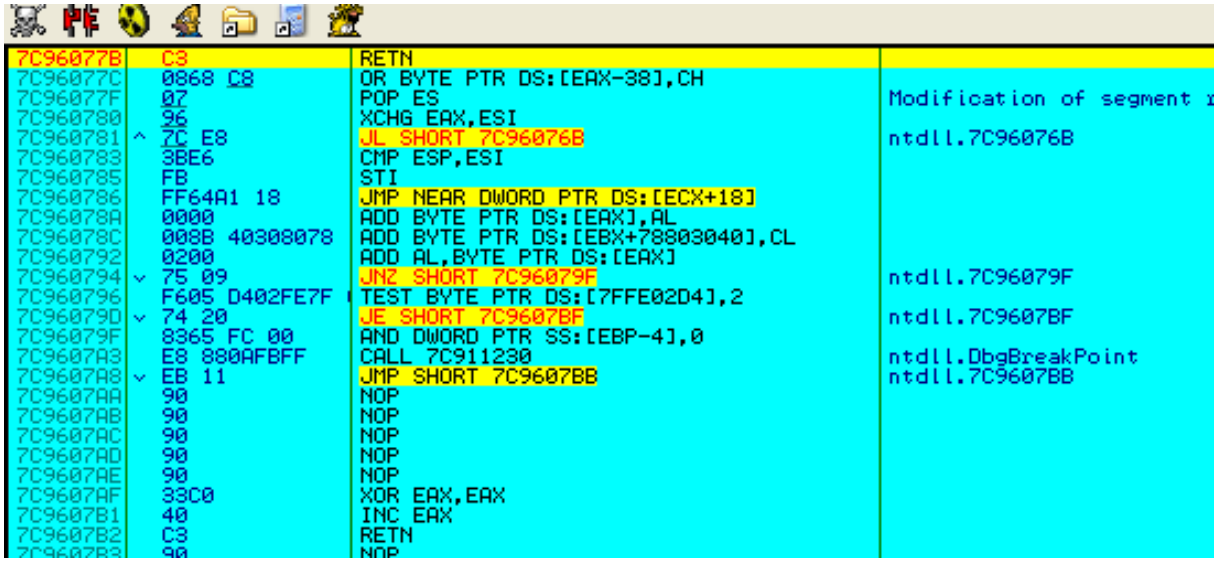
Изменения вносятся API-функцией WriteProcessMemory. Посмотрим ее параметры:



Итак, что за байт будет записан в этом процессе с помощью псевдохэндла FFFFFFFF по адресу исследуемой API-функции?

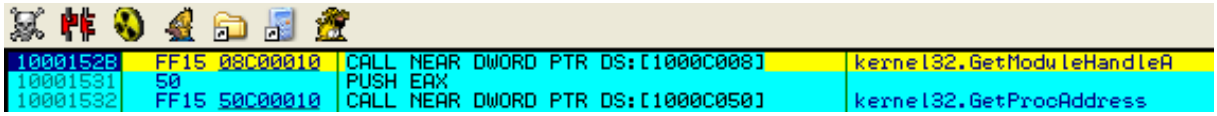


В этой процедуре есть вызов DbgBreakPoint. Если вспомнить, то после присоединения остановка произошла именно на DbgBreakPoint, так что данное место имеет к этому отношение. Выполнив API-функцию WriteProcessMemory, посмотрим, что изменится в DbgUiRemoteBreakin:

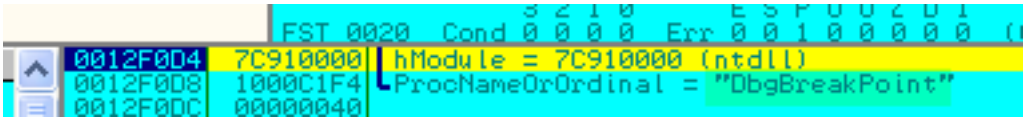


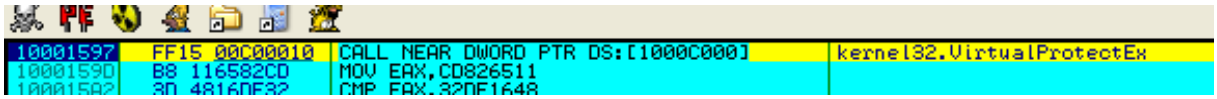
Эта функция аннулируется установкой команды RET в ее начале.

Теперь опять воспользуемся нашим OllyKam'ом, который достаточно много рассказывает нам о происходящем в программе и о вызываемых в ней API-функциях:

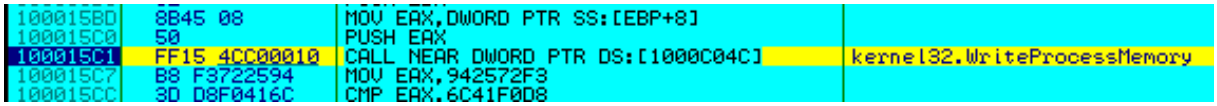


Снова извлекается основание ntdll.dll. Далее идет вызов GetProcAddress для поиска адреса процедуры DbgBreakPoint, чтобы пропатчить и ее.

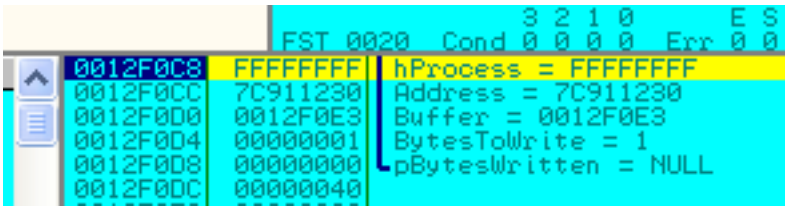




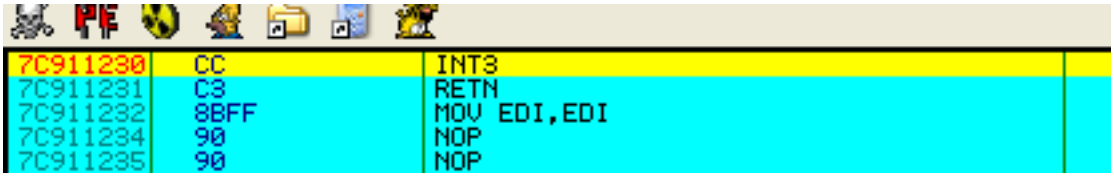
Устанавливаются настройки доступа для чтения/записи. Далее:



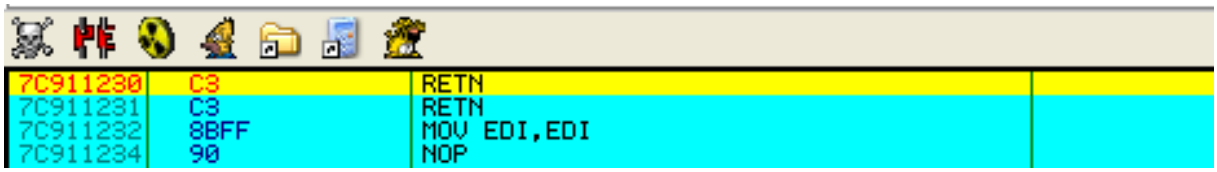
Теперь будут внесены изменения. Посмотрим параметры этой API-функции:



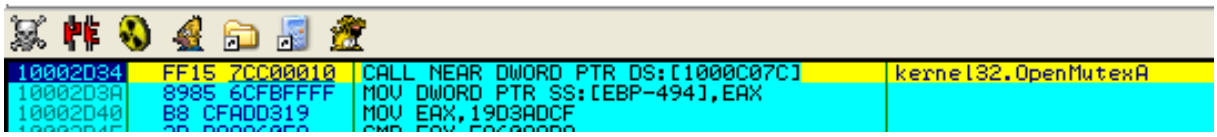
Она изменит один байт в процедуре DbgBreakPoint. Взглянем на нее до внесения изменений:



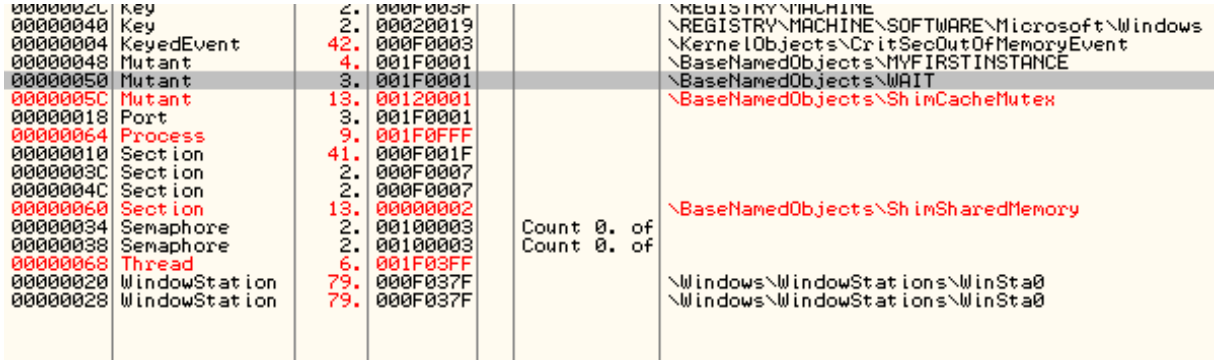
После нажатия на F8 начало функции станет таким:



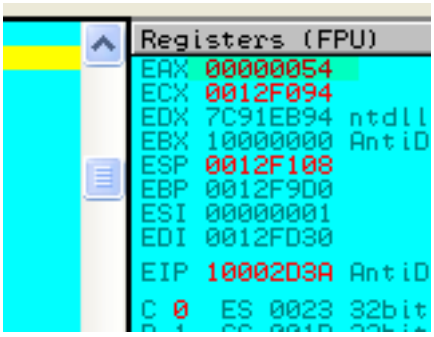
То есть удалена команда INT3, которая вызывает исключение, дающее нам возможность приаттачиться, а вместо нее установлена инструкция RET, не позволяющая нам это сделать. Хе-хе, OllyKam действительно помогает тем, кто желает изучить систему антиотладки! Нажмем F9:



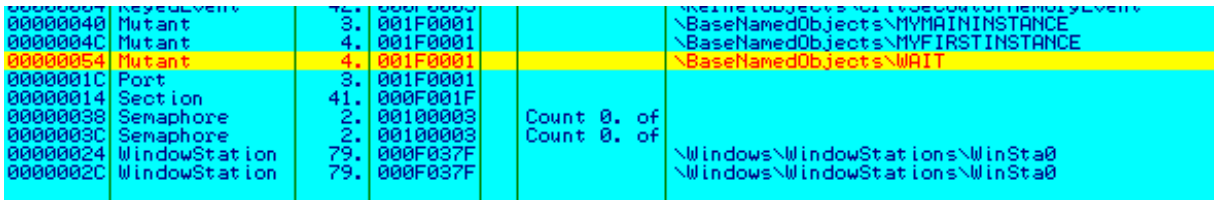
Остановились на API-функции OpenMutexA, которая используется при работе с мьютексами. Один из них уже создан родителем, и это можно проверить в окне хэндлов родительского процесса: там есть мьютекс под названием WAIT:



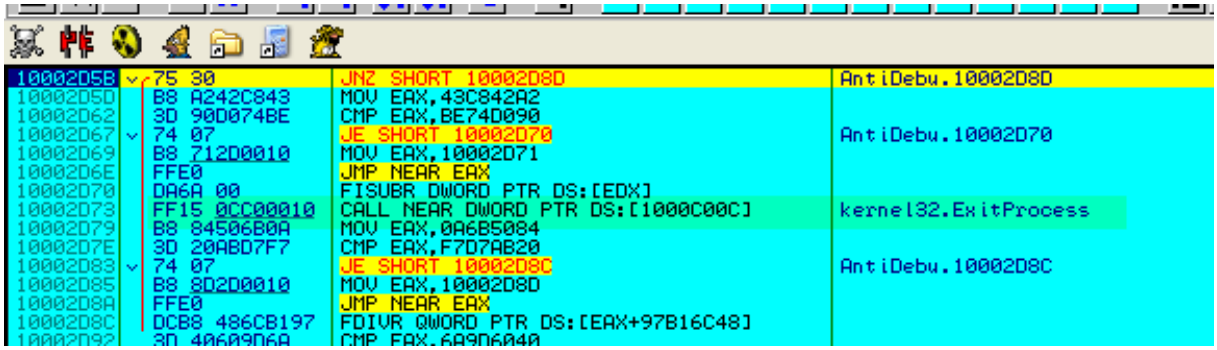
В потомке этот мьютекс ни разу не вызывался, и чтобы с ним можно было работать, нужно получить его хэндл с помощью функции OpenMutexA.



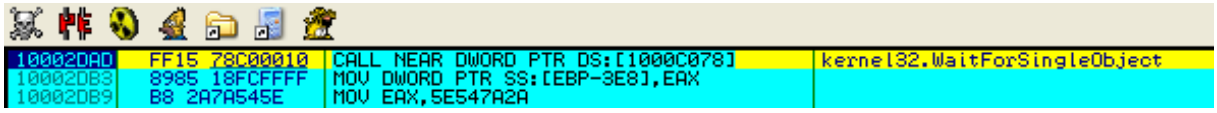
Как это обычно бывает при успешном открытии объектов, хэндл возвращается в регистре EAX.



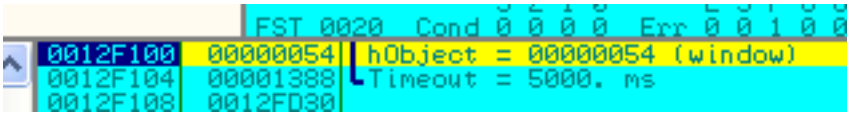
Теперь он загружен и появился в списке хэндлов потомка.



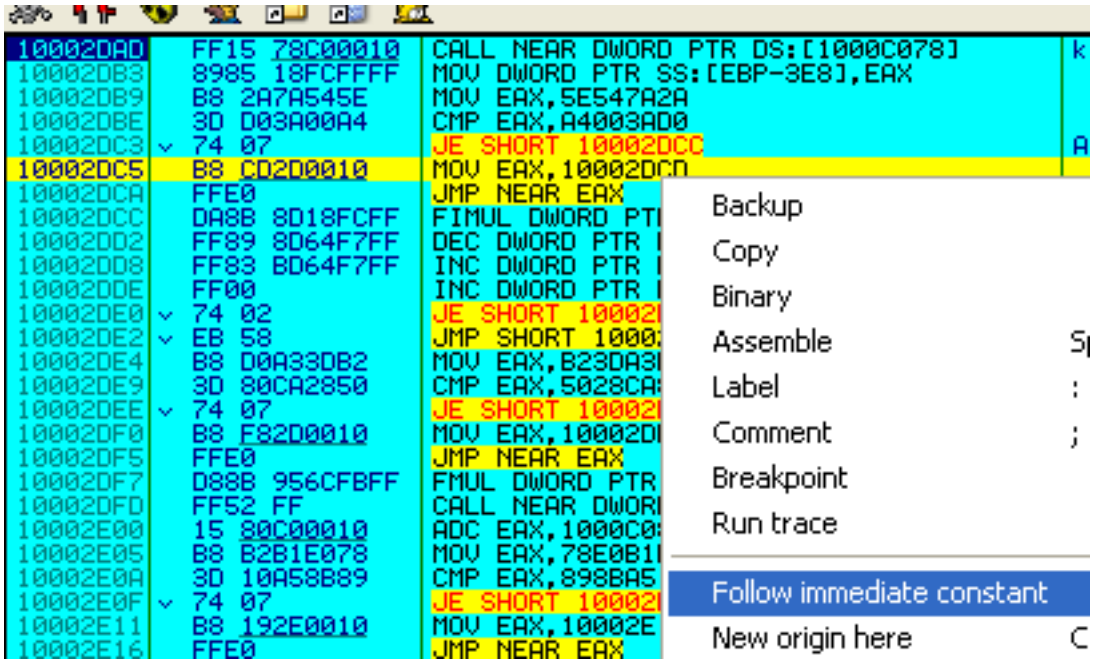
После сравнения полученного хэндла с нулем и выяснения, что такой объект уже был создан родителем (поскольку можно получить его хэндл), был предотвращен вызов ExitProcess. Это очередная проверка, в результате которой потомок мог закрыться. Нажмем F9:

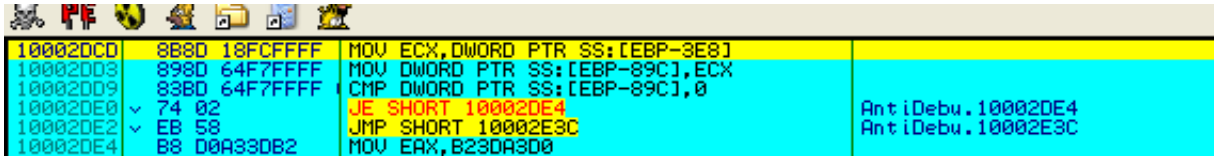


Вызов WaitForSingleObject — еще одно критическое место потомка, где придется бороться с Patrick'ом. Как известно, если родитель не выполнит определенные действия за несколько секунд или войдет в цикл и потратит время, потомок закроется. Данная функция как раз в этом и виновна. Посмотрим ее параметры:

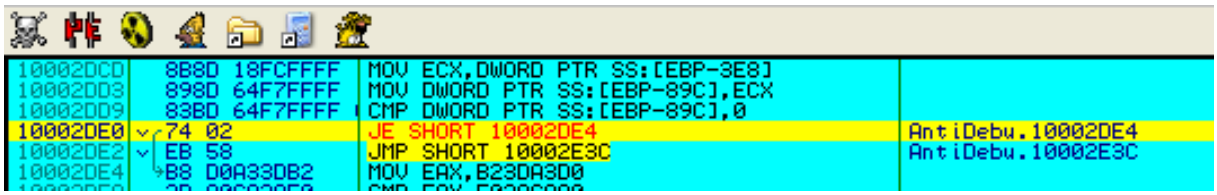


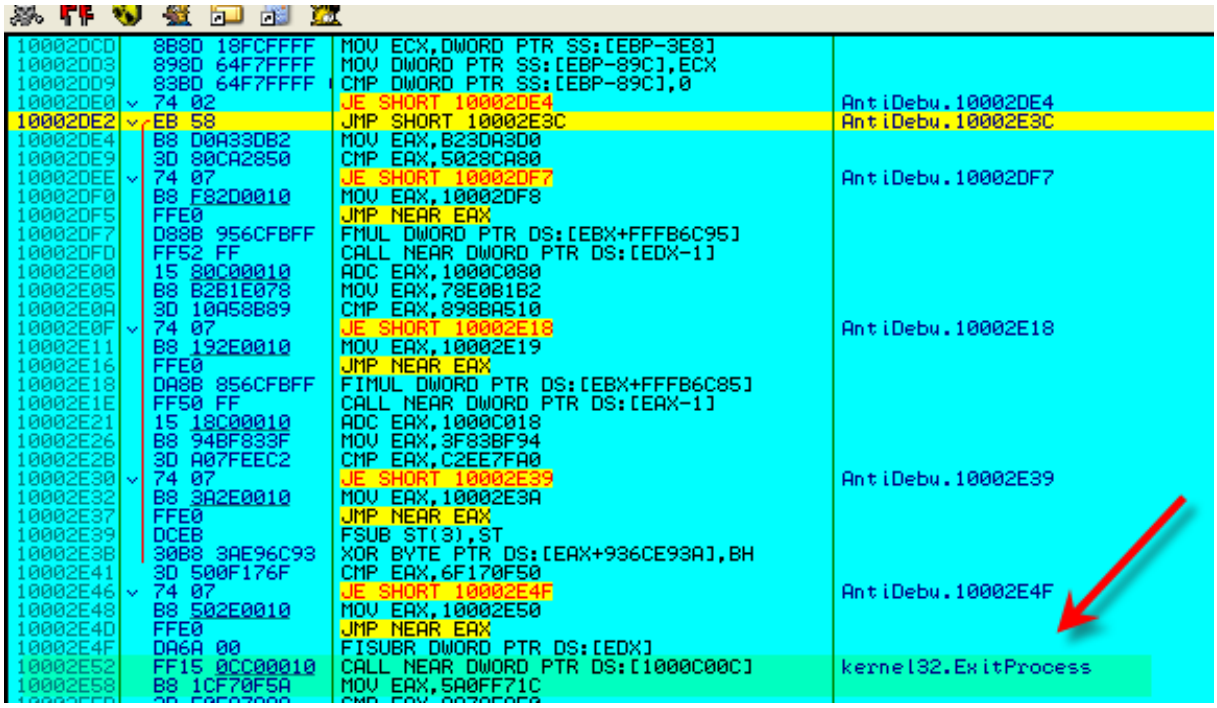
Первый — это хэндл мьютекса, названного WAIT. Значит, потомок будет ожидать от родителя определенных действий, а после освобождения этого мьютекса выполнение потомка продолжится. Проблема заключается во втором параметре, где указан период в 5000 мс: если за этот промежуток времени потомок не дождется от родителя освобождения мьютекса, он будет закрыт, и, как это ни прискорбно, закроется также и потомок. Посмотрим, не трассируя, что идет далее после вызова этой функции:





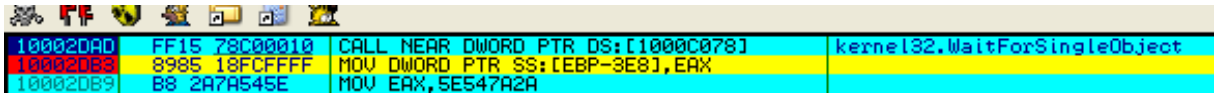
Здесь происходит сравнение с нулем, и, в зависимости от результата, выполняется или не выполняется обход JMP'а, ведущего к вызову ExitProcess.



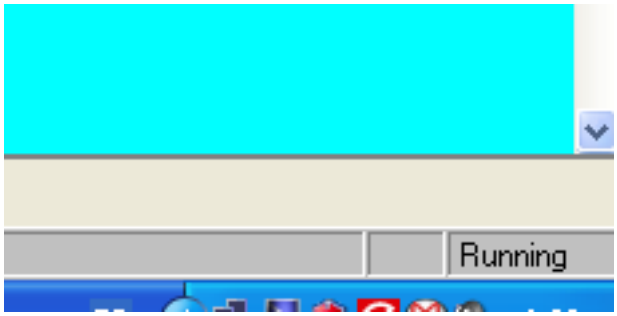


То есть, во избежание закрытия потомку следует не выходить из этой API-функции до тех пор, пока родитель не освободит мьютекс WAIT. Мы просто изменим параметр времени dwMilliseconds (Timeout в Olly) на INFINITE (бесконечность), и потомок не сможет выйти из функции WaitForSingleObject:

После замены значения параметра 00001388 на FFFFFFFF (–1), установим BP сразу за API-функцией, чтобы после освобождения мьютекса родителем произошла остановка:



Теперь нажмем F9:

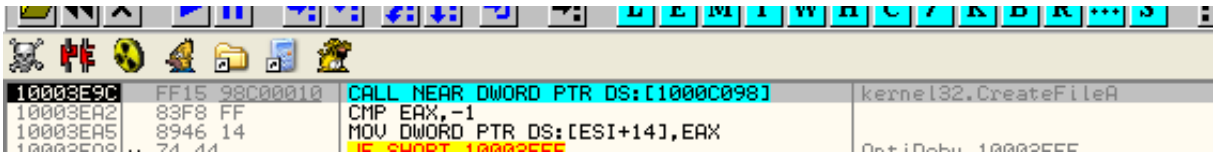


Как видим, потомок выполняется, и это будет продолжаться до тех пор, пока родитель не окажется в области, где освобождается мьютекс. Поэтому теперь мы ничего не сможем сделать в потомке. В момент освобождения мьютекса родителем произойдет остановка на установленном нами брейке, а сейчас, когда потомок вечно выполняется, есть время для продолжения работы с родителем.

Не стоит забывать, что если бы мы не изменили заданный в миллисекундах параметр времени API-функции WaitForSingleObject, это встревожило бы потомка и он перестал бы ждать родителя и отправился бы к ExitProcess, хе-хе. Бедный сын — воспитав его, мы заставили его послушно ожидать, хе-хе.

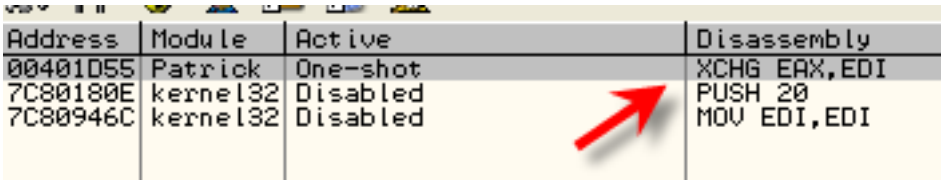
Теперь настала очередь трудиться родителю, поэтому начнем направлять его с помощью его OllyKam'a. Нажмем F9, чтобы узнать, что он будет делать.

Остановка произошла на API-функции Sleep, которая заставляет потерять его немного времени. Продолжая нажимать F9, пройдем не очень важную API-функцию и дойдем досюда:



Параметры этой функции стоит рассмотреть, поскольку здесь организован гнусный трюк и нужно быть настороже.

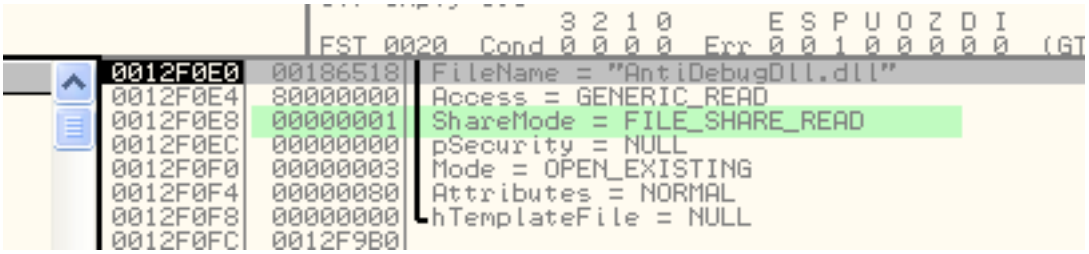
Но перед тем как продолжать объяснение трюка, следует заметить, что в данном случае API-функция CreateFileA используется для обнаружения изменения кода. Установленные брейки изменяют код, а это после сравнения дает различия, которые потом приводят к плохой расшифровке программы. Поэтому все BP следует удалить. Можно оставить BPM ON ACCESS, который используется в OllyKam'e, но придется удалить все BP **как в родителе, так и в потомке,** а BP, который мы поместили для остановки потомка после прекращения вечного выполнения, нужно заменить на HPB ON EXECUTION, иначе возникнут проблемы при расшифровке.



Эти BP в родителе нужно удалить все, даже One-shot, останавливающий OllyDbg на EP, так как он модифицирует код. А также следует удалить все BP в потомке.

После оставления насиженного добра можно продолжать изучение трюка, реализованного с помощью API-функции CreateFileA. Но теперь мы не будем устанавливать ни одного обычного брейкпоинта — все брейки должны быть либо BPM, либо HE.

Вот параметры API-функции CreateFileA:

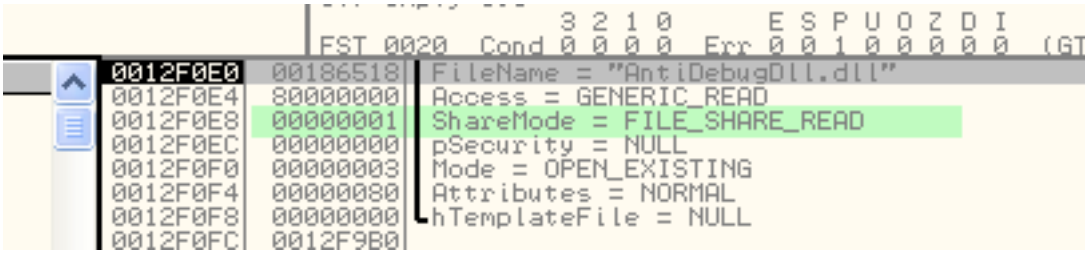


Здесь происходит что-то странное, что я считаю багом OllyDbg — другого объяснения я не нахожу. Если кто-то, прочитав дальнейшее, узнает, почему так происходит, скажите мне. Я никогда не видел такого противооллиного трюка и у меня нет никакой информации о том, что в API-функции CreateFileA может использоваться баг Olly. Поиски в форумах ничего не дали. Возможно, друг Fatmike обнаружил какой-то новоявленный баг. И только из-за того, что я довольно давно в крэкинге, после усиленных поисков удалось вычислить, что здесь происходит. Я постараюсь объяснить эти довольно необычные вещи как можно более ясно.

Если попытаться изменить в DLL'ке хотя бы один байт и сохранить изменения, то при следующем запуске программа узнает, что произошло изменение, дойдет до OEP, но окажется плохо распакованной, т.е. будет состоять из мусора, вызывающего ошибки.

Дело в том, что программа с помощью CreateFileA проверяет AntiDebugDll.dll, а затем с помощью ReadFile считывает и сравнивает байты файла с теми, которые выполняются, проверяя таким образом их равенство. При обнаружении несоответствия даже одного байта происходит плохая распаковка. Так должно быть в случае, если изменен один или более байтов, но мы в файле ничего не меняли, а делали всё в памяти, поэтому после считывания файла байты должны совпасть (если, конечно, не установлены BP). Однако, при использовании OllyDbg произойдет то же самое: программа плохо расшифруется и в OEP окажется мусор. Похоже, в Olly происходит нечто, из-за чего при прочтении файла программа ведет себя так, как если бы ее байты были изменены.

Вернемся к API-функции CreateFileA и ее параметрам:



Большинство из них довольно стандартны, но параметр dwShareMode требует особого внимания. Давайте выясним его возможные значения по Win32 API:

dwShareMode

Set of bit flags that specifies how the object can be shared. If *dwShareMode* is 0, the object cannot be shared. Subsequent open operations on the object will fail, until the handle is closed.

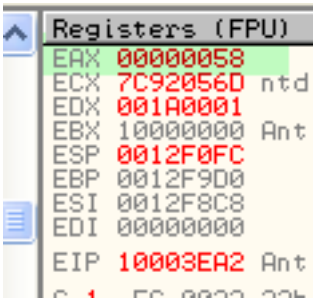
To share the object, use a combination of one or more of the following values:

Value	Meaning
FILE_SHARE_DELETE	**Windows NT only:** Subsequent open operations on the object will succeed only if delete access is requested.
FILE_SHARE_READ	Subsequent open operations on the object will succeed only if read access is requested.
FILE_SHARE_WRITE	Subsequent open operations on the object will succeed only if write access is requested.

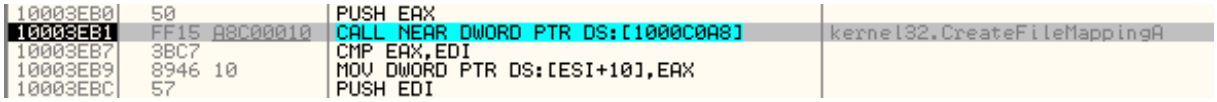
Здесь сказано, что если этот параметр равен FILE_SHARE_READ, то следующий вызов файла будет успешен только при установке параметра dwDesiredAccess в GENERIC_READ. Установка

нуля вместо FILE_SHARE_READ запрещает разделение доступа к файлу и следующая попытка его открытия будет тщетна, поскольку файл уже открыт.

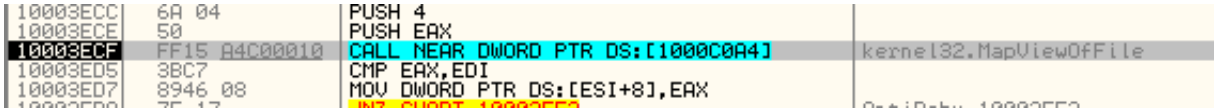
В нашем случае в следующий раз файл будет открыт только при установке параметра GENERIC_READ. Нажмем F8, чтобы посмотреть, будет ли возвращен хэндл:



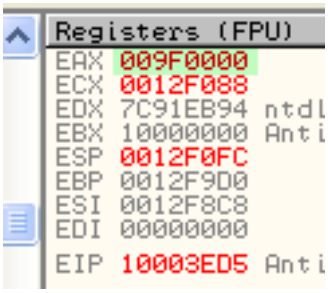
Хэндл получен. Теперь с помощью OllyKam'а перейдем к очередной API-функции:



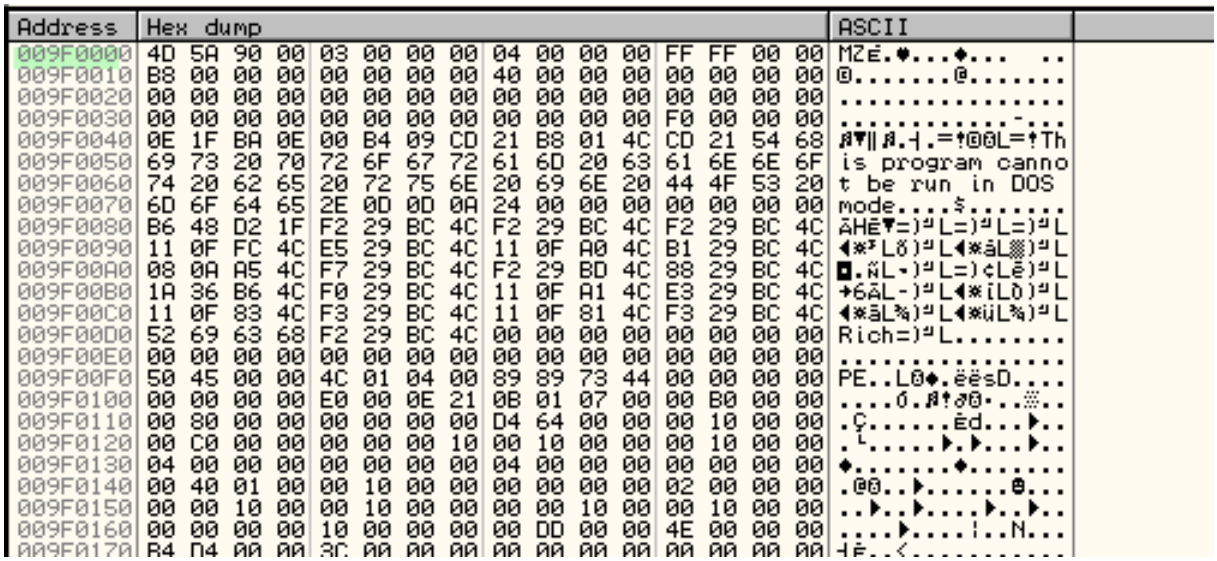
Здесь вызывается CreateFileMappingA, создающая проекцию файла в памяти, о чем можно прочитать в новых туториалах этого курса. Она не сильно влияет на функционирование программы. С ее помощью последняя получает хэндл проецированного файла.



Затем вызывается API-функция MapViewOfFile, которая мэппирует файл в память с помощью полученного хэндла. После нажатия на F8 станет известен адрес, по которому загружена проекция:

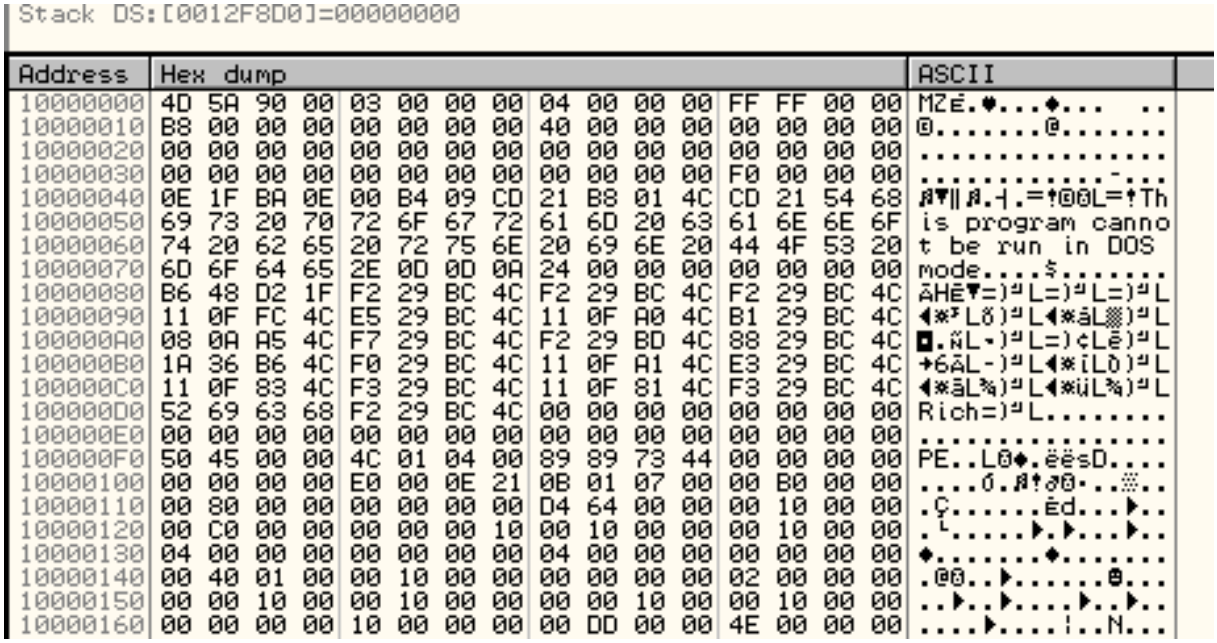


AntiDebugDll.dll загружена в память побайтно, начиная с адреса 9F0000. Казалось бы, теперь эти байты должны быть сравнены с теми, которые выполняются, но этого не происходит — сравнивается несколько байтов заголовка, а вовсе не код программы. Посмотрим в дампе загруженные байты DLL'ки:



Address	Hex dump	ASCII
009F0000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZÉ.♥...♦... ..
009F0010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	®.......@.......
009F0020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
009F0030	00 00 00 00 00 00 00 00 00 00 00 00 F0 00 00 00	............-...
009F0040	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	♫▼‖♫.┤.=↑@@L=↑Th
009F0050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program canno
009F0060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS
009F0070	6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00	mode....$.......
009F0080	B6 48 D2 1F F2 29 BC 4C F2 29 BC 4C F2 29 BC 4C	¶HÒ▼=)╝L=)╝L=)╝L
009F0090	11 0F FC 4C E5 29 BC 4C 11 0F A0 4C B1 29 BC 4C	◄☼ⁿLσ)╝L◄☼áL▒)╝L
009F00A0	08 0A A5 4C F7 29 BC 4C F2 29 BD 4C 88 29 BC 4C	◘.ñL-)╝L=)¢Lë)╝L
009F00B0	1A 36 B6 4C F0 29 BC 4C 11 0F A1 4C E3 29 BC 4C	→6¶L-)╝L◄☼íLΩ)╝L
009F00C0	11 0F 83 4C F3 29 BC 4C 11 0F 81 4C F3 29 BC 4C	◄☼âL≤)╝L◄☼üL≤)╝L
009F00D0	52 69 63 68 F2 29 BC 4C 00 00 00 00 00 00 00 00	Rich=)╝L........
009F00E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
009F00F0	50 45 00 00 4C 01 04 00 89 89 73 44 00 00 00 00	PE..L☺♦.ëësD....
009F0100	00 00 00 00 E0 00 0E 21 0B 01 07 00 00 B0 00 00	....ó.♫!♪☺·..▒..
009F0110	00 80 00 00 00 00 00 00 D4 64 00 00 00 10 00 00	.Ç.......Ôd...►..
009F0120	00 C0 00 00 00 00 00 10 00 10 00 00 00 10 00 00	.└.....►.►...►..
009F0130	04 00 00 00 00 00 00 00 04 00 00 00 00 00 00 00	♦.......♦.......
009F0140	00 40 01 00 00 10 00 00 00 00 00 00 02 00 00 00	.@☺.►.......☻...
009F0150	00 00 10 00 00 10 00 00 00 00 10 00 00 10 00 00	..►..►....►..►..
009F0160	00 00 00 00 10 00 00 00 00 DD 00 00 4E 00 00 00	....►....¦..N...
009F0170	B4 D4 00 00 3C 00 00 00 00 00 00 00 00 00 00 00	┤╘..<...........

Они должны совпадать с байтами выполняющейся в данный момент DLL'ки, загруженной по адресу 10000000. И действительно, это те же самые байты:

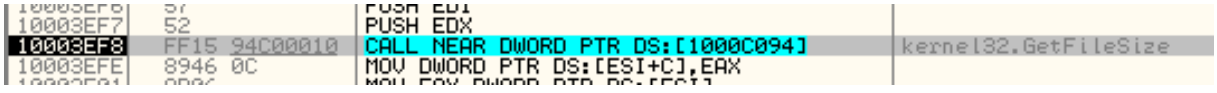


Stack DS:[0012F8D0]=00000000

Address	Hex dump	ASCII
10000000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZÉ.♥...♦... ..
10000010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	®.......@.......
10000020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
10000030	00 00 00 00 00 00 00 00 00 00 00 00 F0 00 00 00	............-...
10000040	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	♫▼‖♫.┤.=↑@@L=↑Th
10000050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program canno
10000060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS
10000070	6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00	mode....$.......
10000080	B6 48 D2 1F F2 29 BC 4C F2 29 BC 4C F2 29 BC 4C	¶HÒ▼=)╝L=)╝L=)╝L
10000090	11 0F FC 4C E5 29 BC 4C 11 0F A0 4C B1 29 BC 4C	◄☼ⁿLσ)╝L◄☼áL▒)╝L
100000A0	08 0A A5 4C F7 29 BC 4C F2 29 BD 4C 88 29 BC 4C	◘.ñL-)╝L=)¢Lë)╝L
100000B0	1A 36 B6 4C F0 29 BC 4C 11 0F A1 4C E3 29 BC 4C	→6¶L-)╝L◄☼íLΩ)╝L
100000C0	11 0F 83 4C F3 29 BC 4C 11 0F 81 4C F3 29 BC 4C	◄☼âL≤)╝L◄☼üL≤)╝L
100000D0	52 69 63 68 F2 29 BC 4C 00 00 00 00 00 00 00 00	Rich=)╝L........
100000E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
100000F0	50 45 00 00 4C 01 04 00 89 89 73 44 00 00 00 00	PE..L☺♦.ëësD....
10000100	00 00 00 00 E0 00 0E 21 0B 01 07 00 00 B0 00 00	....ó.♫!♪☺·..▒..
10000110	00 80 00 00 00 00 00 00 D4 64 00 00 00 10 00 00	.Ç.......Ôd...►..
10000120	00 C0 00 00 00 00 00 10 00 10 00 00 00 10 00 00	.└.....►.►...►..
10000130	04 00 00 00 00 00 00 00 04 00 00 00 00 00 00 00	♦.......♦.......
10000140	00 40 01 00 00 10 00 00 00 00 00 00 02 00 00 00	.@☺.►.......☻...
10000150	00 00 10 00 00 10 00 00 00 00 10 00 00 10 00 00	..►..►....►..►..
10000160	00 00 00 00 10 00 00 00 00 DD 00 00 4E 00 00 00	....►....¦..N...

Программисту было бы легко сделать сравнение прямо здесь, поскольку байты уже загружены, и в секции, созданной проецированием файла в память, нельзя устанавливать ни BPM ни HE. Можно было замести следы с помощью переходов и начать сравнение здесь, проверяя наличие измененных байтов, но, как бы это удивительно ни было, сравниваются только 4 или 5 значений в заголовке. Я трассировал дальше с помощью F7, так как в секции 9F0000 нельзя устанавливать ни BPM, ни HE. После долгой бессмысленной трассировки я заметил, что байты почему-то не сравниваются. Было бы проще установить брейки, но пришлось всё делать вручную. Это была одна из причин подозревать, что автор что-то замышляет. Затея, подобно первым шагам ребенка, завершилась, чтобы продолжиться позже. Какая-то причина однозначно существует!

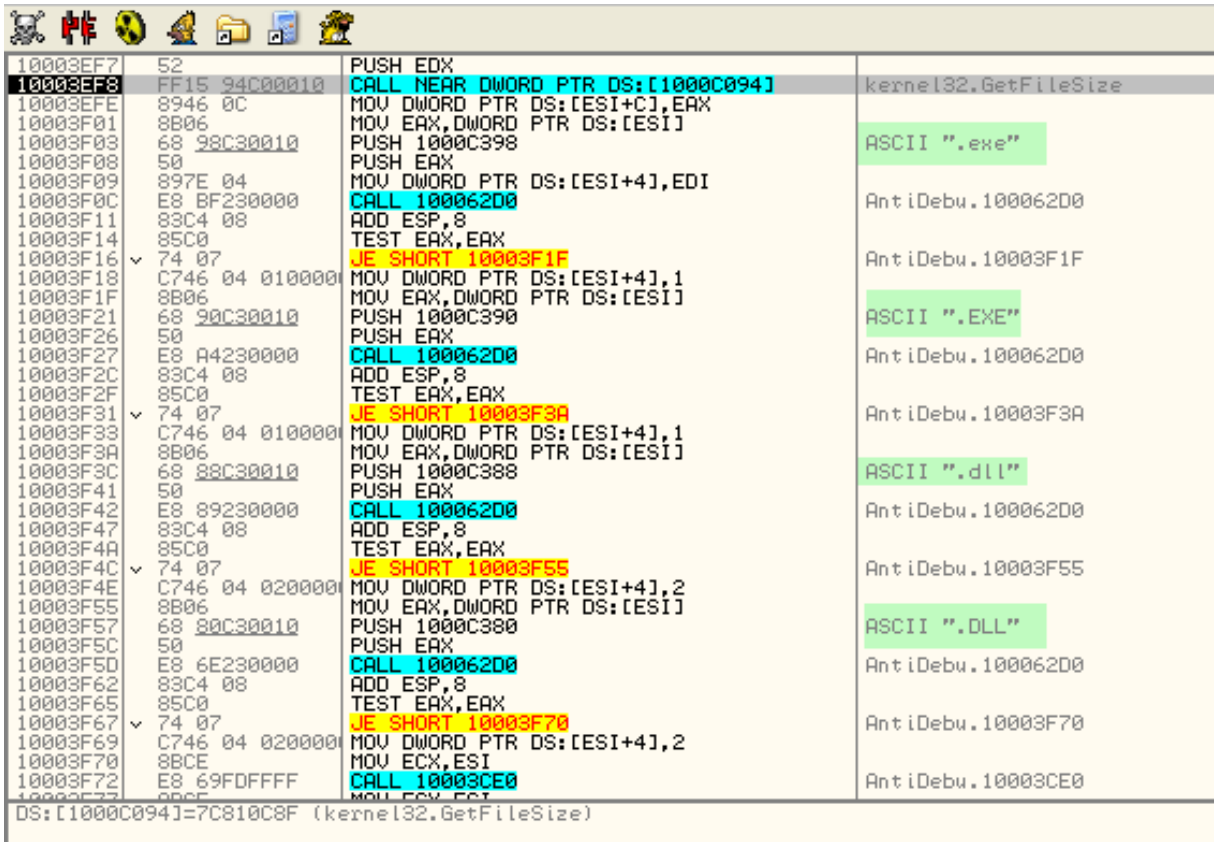
Что ж, далее воспользуемся OllyKam'ом:



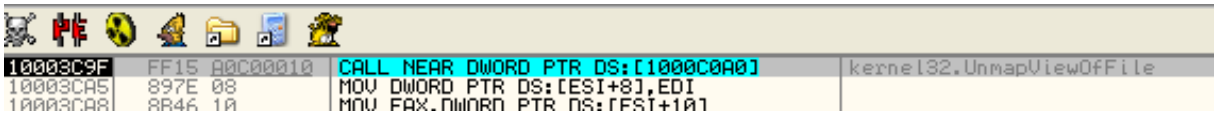
```
10003EF7   52              PUSH EDX
10003EF8   FF15 94C00010   CALL NEAR DWORD PTR DS:[1000C094]   kernel32.GetFileSize
10003EFE   8946 0C         MOV DWORD PTR DS:[ESI+C],EAX
```

Здесь считывается размер файла, чтобы узнать, был ли он изменен, а затем идут довольно нелепые сравнения расширения DLL'ки: dll или DLL, exe или EXE… Ну и пусть теряет свое время,

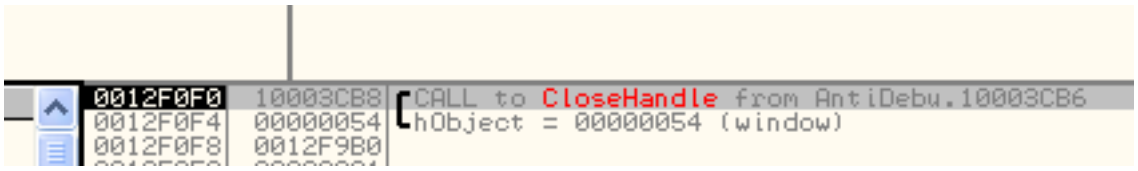
если ему так хочется, хе-хе.



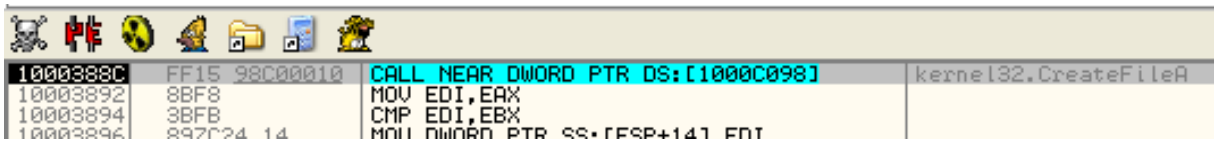
Продолжим:



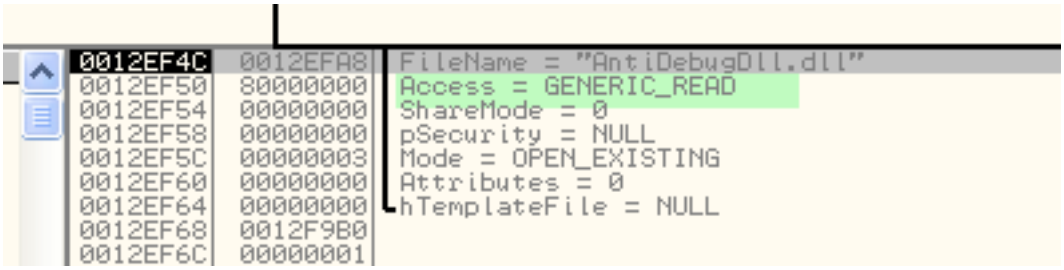
Здесь удаляется проекция файла, загруженного в 9F0000, то есть эти байты больше не потребуются.



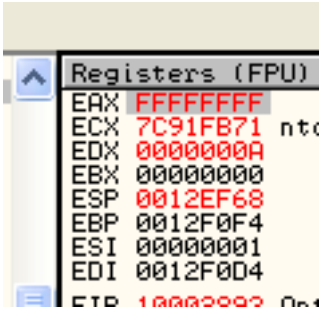
Затем закрывается хэндл только что удаленной проекции файла, и это должно поставить точку в данном вопросе.



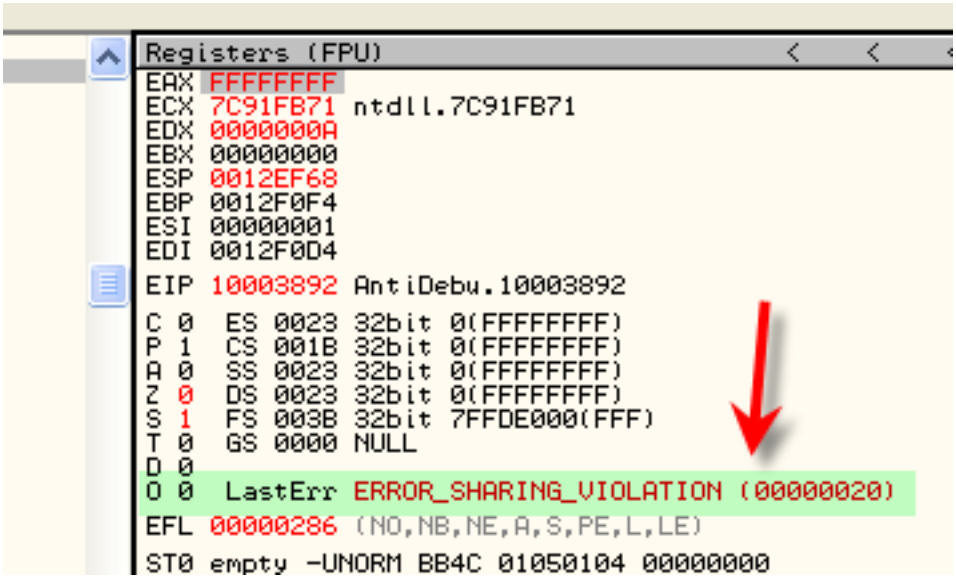
Тем не менее, снова вызывается CreateFileA, но уже с такими параметрами:



Параметр dwDesiredAccess по-прежнему установлен в GENERIC_READ, и, согласно Win32 API, при существующем файле его хэндл можно получить снова. Нулевой dwShareMode приводит к тому, что хэндл не возвращается только при последующих вызовах CreateFileA, а здесь такого ограничения нет, и он должен возвратиться, поскольку файл существует. Посмотрим, что произойдет.

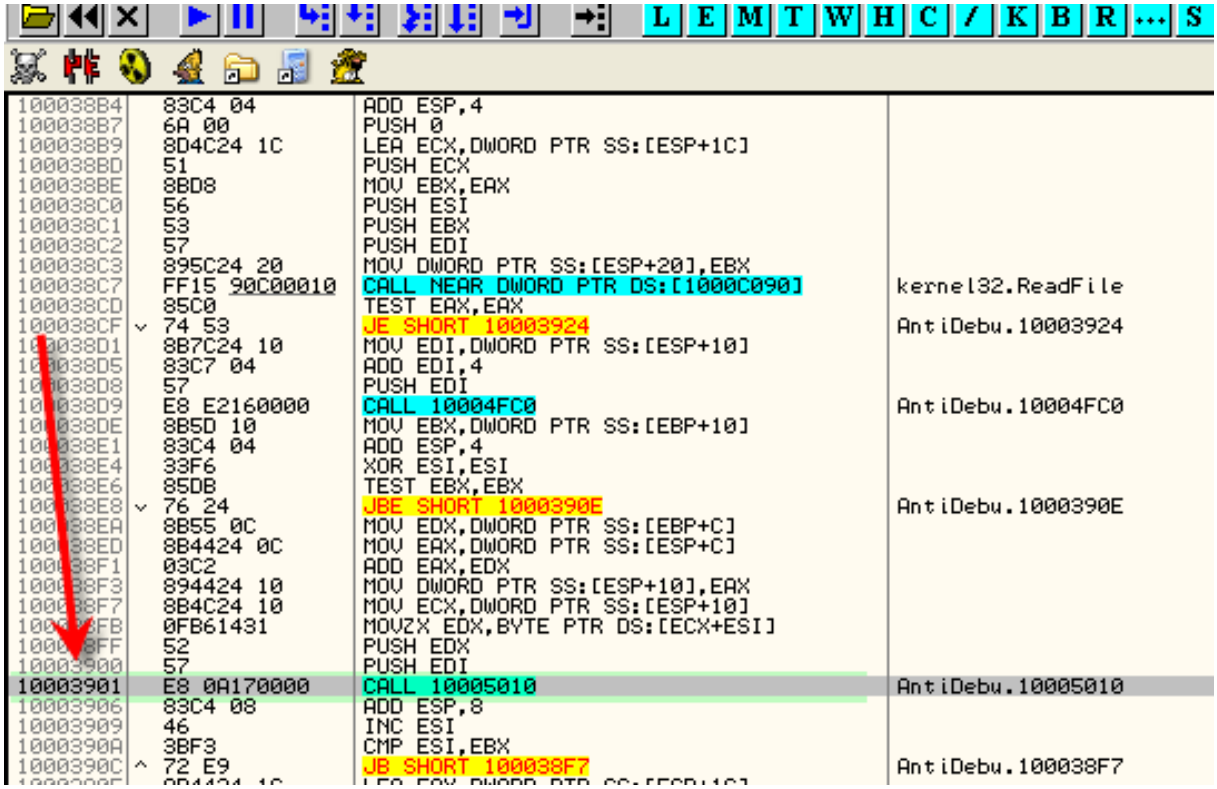


Вместо хэндла файла OllyDbg возвратил код FFFFFFFF, а также показал сообщение об ошибке разделения доступа ERROR_SHARING_VIOLATION:



Но мы видели, что всё было нормально и такой ошибки не должно было возникнуть. Похоже, предыдущее проецирование файла в память было выполнено неверно. После длительного исследования этой проблемы приходится сделать заключение, что в данном случае ошибочно функционирует сам отладчик.

Тот факт, что далее в программе хэндл, который должен был получен здесь, используется для чтения DLL'ки с помощью ReadFile и выполняется сравнение, когда это можно было сделать ранее, доказывает, что так было сделано намеренно. Кто-то нашел в OllyDbg баг и воспользовался им, чтобы в этом отладчике было невозможно считать байты файла и сравнить их: это приводит к тому же, что и изменение байтов — файл плохо расшифровывается. Все эти догадки были подтверждены экспериментально.



После неудачной попытки получения хэндла вызывается ReadFile и производится чтение файла с параметром FFFFFFFF. В результате этого в регистре EAX оказывается ноль и происходит переход. Если бы был указан хэндл, то чтение выполнилось бы и переход бы не состоялся. Это означает, что пока программа выполняется в OllyDbg, адрес в образе 10003901 никогда не будет достигнут, поскольку выше происходит переход из-за неудачного выполнения ReadFile.

А что, если попытаться обеспечить нормальную работу ReadFile вне OllyDbg, чтобы эта функция возвратила единицу и переход, обходящий указанную область, не состоялся?

Я воспользовался не очень информативным способом: в файерволе Kerio есть возможность установить в настройках галку, чтобы программы спрашивали разрешения на запуск. Я запустил Patrick'a вне OllyDbg, и окошко с вопросом выскочило. Потом в Pupe установил EB FE по адресу ntdll.dll, из которого происходит переход на EP DLL'ки (на моем компьютере это 7C9111A4) и ответил в диалоговом окне. Затем установил другой цикл несколькими строками выше в 7C911199 и снял первый, и так поочередно устанавливал один и снимал другой, пока DLL'ка не загрузилась. Потом поместил бесконечный цикл в 10003901 и удалил циклы из ntdll, чтобы программа запустилась и осталась там зацикленной. Это позволило приаттачиться к процессу и увидеть, что он был остановлен в месте, в которое из OllyDbg никогда не попасть и которое достижимо только в случае возвращения функцией CreateFileA валидного хэндла. Таким образом было установлено, что программа выполняется нормально и в OllyDbg действительно есть баг.

Сделаем так, чтобы OllyDbg оказался в месте возвращения хэндла: повторим всё и дойдем до CreateFileA:

```
ST7 empty 0.0
                 3 2 1 0      E S P U O Z D I
0012EF48  10003892  CALL to CreateFileA from AntiDebu.1000388C
0012EF4C  0012EFA8  FileName = "AntiDebugDll.dll"
0012EF50  80000000  Access = GENERIC_READ
0012EF54  00000000  ShareMode = 0
0012EF58  00000000  pSecurity = NULL
0012EF5C  00000003  Mode = OPEN_EXISTING
0012EF60  00000000  Attributes = 0
0012EF64  00000000  hTemplateFile = NULL
0012EF68  0012F9B0
0012EF6C  00000001
0012EF70  10000000  AntiDebu.10000000
0012EF74  00150000
```

Заставить OllyDbg возвратить валидный хэндл можно, заменив нулевое значение параметра dwShareMode единицей:

```
ST7 empty 0.0
                 3 2 1 0      E S P U O Z D I
0012EF48  10003892  CALL to CreateFileA from AntiDebu.1000388C
0012EF4C  0012EFA8  FileName = "AntiDebugDll.dll"
0012EF50  80000000  Access = GENERIC_READ
0012EF54  00000001  ShareMode = FILE_SHARE_READ
0012EF58  00000000  pSecurity = NULL
0012EF5C  00000003  Mode = OPEN_EXISTING
0012EF60  00000000  Attributes = 0
0012EF64  00000000  hTemplateFile = NULL
0012EF68  0012F9B0
```

Нажмем F8:

```
Registers (FPU)
EAX 00000058
ECX 7C92056D ntdll.
EDX 00150608
EBX 00000000
ESP 0012EF68
EBP 0012F0F4
ESI 00000001
EDI 0012F0D4

EIP 10003892 AntiDe
```

И вот наш злополучный хэндл! Этот трюк, кажущийся таким простым, когда он известен, заставил меня провести целые дни в изнурительных попытках выяснить, что же здесь происходит. Так что на самом деле он совсем не прост…

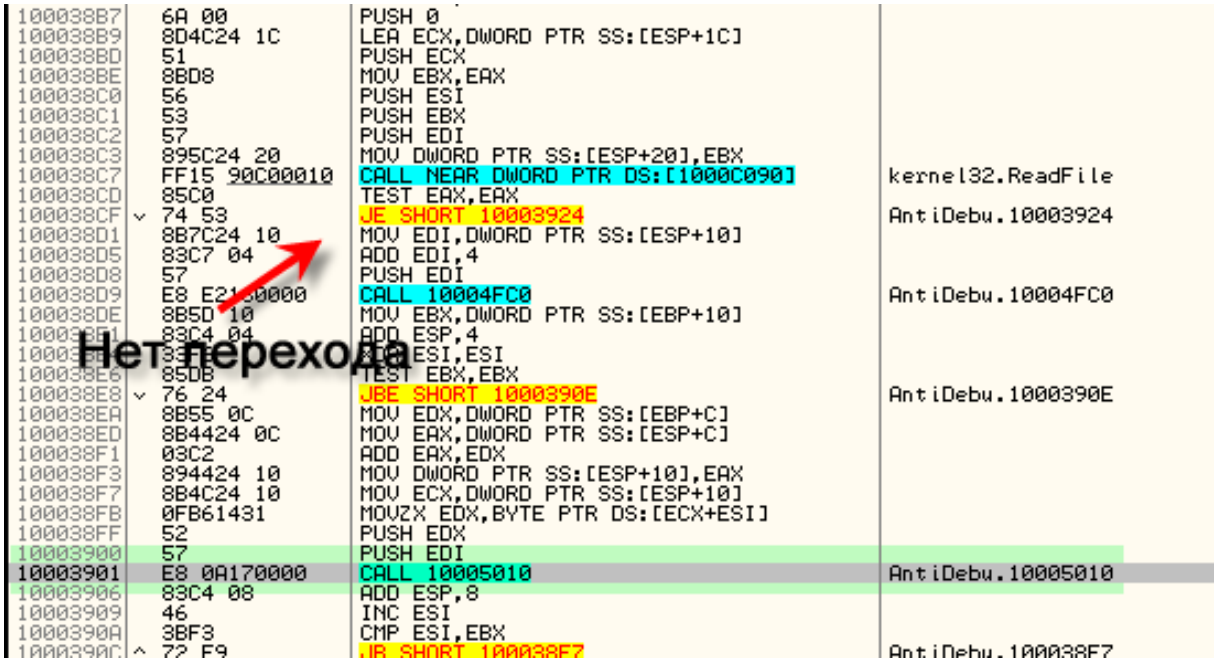
Ладно, теперь продолжим юзать наш OllyKam:

```
100038C7   FF15 90C00010  CALL NEAR DWORD PTR DS:[1000C090]   kernel32.ReadFile
100038CD   85C0           TEST EAX,EAX
100038CF   74 53          JE SHORT 10003924                   AntiDebu.10003924
100038D1   8B7C24 10      MOV EDI,DWORD PTR SS:[ESP+10]
```

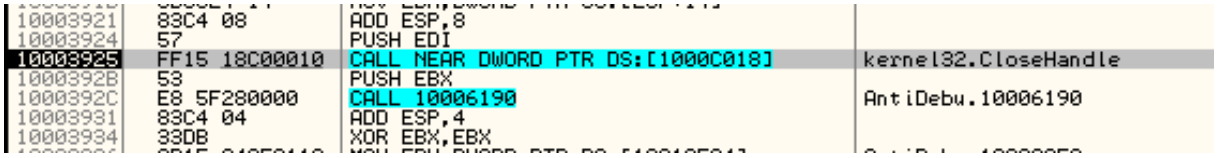
Дойдя до ReadFile, нажмем F8:

```
Registers (FPU)
EAX 00000001
ECX 7C801898 kerne
EDX 7C91EB94 ntdll
EBX 009F0048
ESP 0012EF68
EBP 0012F0F4
ESI 00013000
EDI 00000058

EIP 100038CD AntiD
```

Теперь EAX=1, как и должно быть, и обход адреса 10003901 не производится.

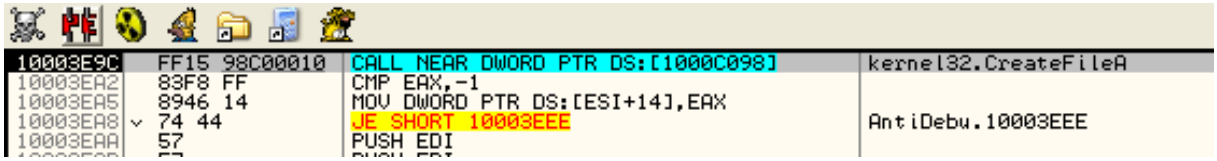


Сразу после сравнения идет закрытие хэдла:

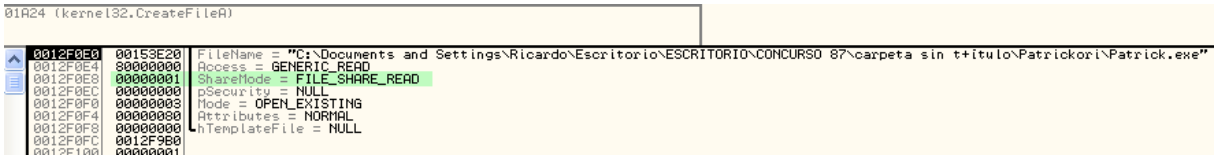


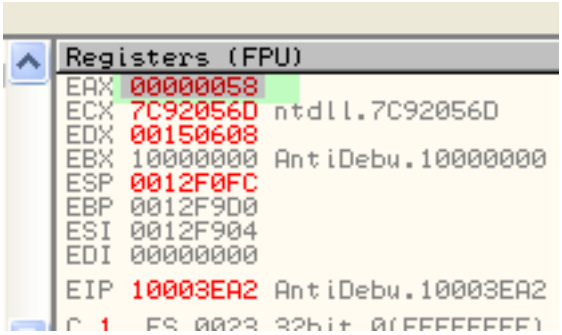
Нужно быть внимательным, поскольку этот трюк используется несколько раз.

OllyKam, вперед!



После нескольких остановок на не очень важных API-функциях снова вызывается CreateFileA, теперь уже для считывания файла Patrick.exe:





Так как это происходит впервые, то проблем не возникает и возвращается валидный хэндл, позволяющий работать с файлом.

```
10003EB0   50              PUSH EAX
10003EB1   FF15 A8C00010   CALL NEAR DWORD PTR DS:[1000C0A8]   kernel32.CreateFileMappingA
10003EB7   3BC7            CMP EAX,EDI
10003EB9   8946 10         MOV DWORD PTR DS:[ESI+10],EAX
10003EBC   57              PUSH EDI
10003EBD   75 0B           JNZ SHORT 10003ECA                  AntiDebu.10003ECA
10003EBF   FF15 18C00010   CALL NEAR DWORD PTR DS:[1000C018]   kernel32.CloseHandle
10003EC5   5F              POP EDI
10003EC6   32C0            XOR AL,AL
10003EC8   5E              POP ESI
10003EC9   C3              RETN
```

Теперь с файлом Patrick.exe происходит то же самое, что было с AntiDebugDll.dll: создается проекция файла в памяти, т.е. возвращается ее хэндл.

```
10003ECC   6A 04           PUSH 4
10003ECE   50              PUSH EAX
10003ECF   FF15 A4C00010   CALL NEAR DWORD PTR DS:[1000C0A4]   kernel32.MapViewOfFile
10003ED5   3BC7            CMP EAX,EDI
10003ED7   8946 08         MOV DWORD PTR DS:[ESI+8],EAX
10003EDA   75 17           JNZ SHORT 10003EF3                  AntiDebu.10003EF3
10003EDC   8B46 10         MOV EAX,DWORD PTR DS:[ESI+10]
10003EDF   8B3D 18C00010   MOV EDI,DWORD PTR DS:[1000C018]    kernel32.CloseHandle
10003EE5   50              PUSH EAX
```

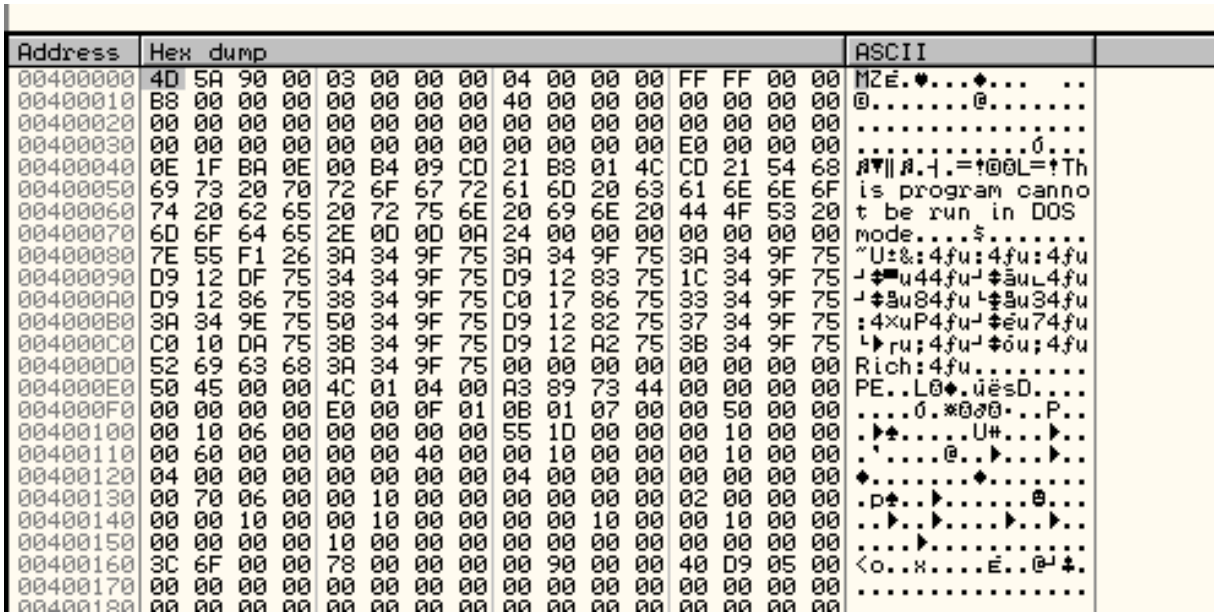
Здесь копируются байты файла и API-функция возвращает адрес проекции, в данном случае на моем компьютере это 0AF0000:

```
Registers (FPU)
EAX 00AF0000
ECX 0012F088
EDX 7C91EB94 ntd
EBX 10000000 Ant
ESP 0012F0FC
EBP 0012F9D0
ESI 0012F904
EDI 00000000

EIP 10003ED5 Ant
```

Если посмотрим этот адрес в дампе, то увидим отображение файла, содержащее считанные из файла Patrick.exe байты:

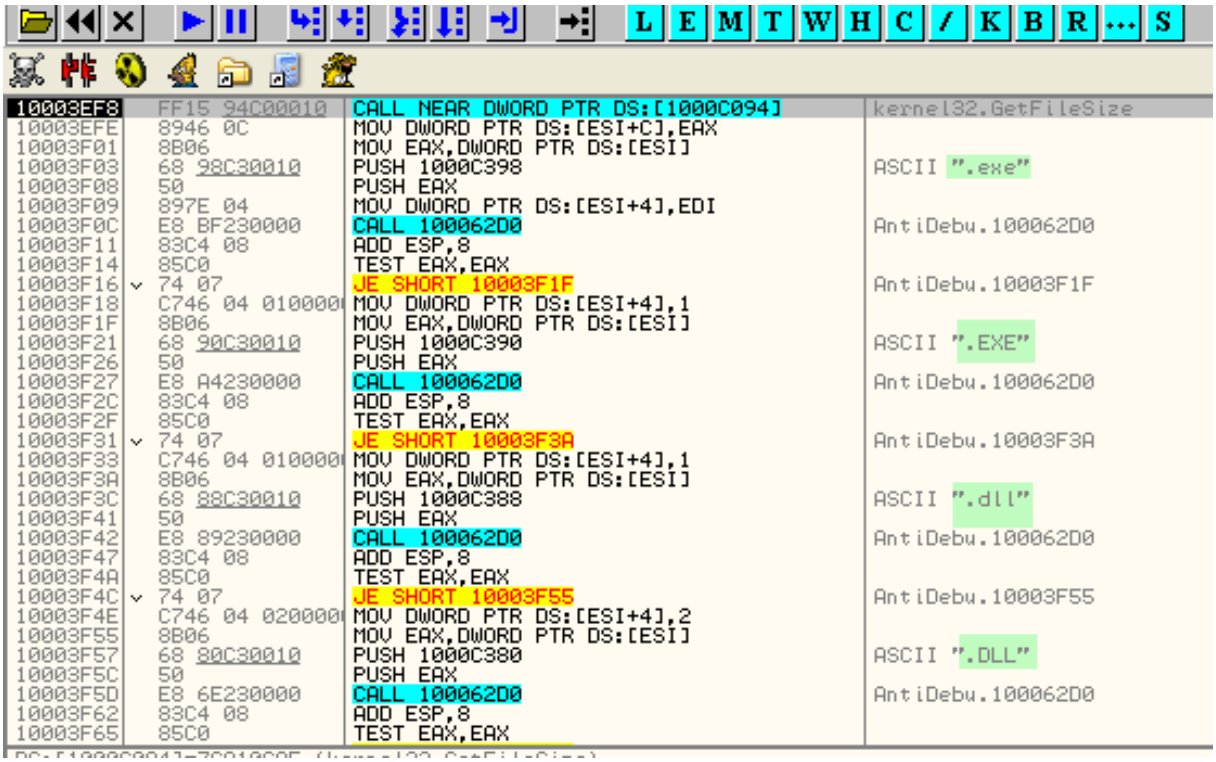
```
Address   Hex dump                                          ASCII
00AF0000  4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00  MZÉ.♥...♦...  ..
00AF0010  B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00  ®.......@.......
00AF0020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00AF0030  00 00 00 00 00 00 00 00 00 00 00 00 E0 00 00 00  ............ó...
00AF0040  0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68  ♫▼‖♂.┤.=↑@0L=↑Th
00AF0050  69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F  is program canno
00AF0060  74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20  t be run in DOS
00AF0070  6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00  mode....$.......
00AF0080  7E 55 F1 26 3A 34 9F 75 3A 34 9F 75 3A 34 9F 75  ~U±&:4ƒu:4ƒu:4ƒu
00AF0090  D9 12 DF 75 34 34 9F 75 D9 12 83 75 1C 34 9F 75  ┘↕▀u44ƒu┘↕âu∟4ƒu
00AF00A0  D9 12 86 75 38 34 9F 75 C0 17 86 75 33 34 9F 75  ┘↕åu84ƒu└↕åu34ƒu
00AF00B0  3A 34 9E 75 50 34 9F 75 D9 12 82 75 37 34 9F 75  :4×uP4ƒu┘↕éu74ƒu
00AF00C0  C0 10 DA 75 3B 34 9F 75 D9 12 A2 75 3B 34 9F 75  └►ru;4ƒu┘↕óu;4ƒu
00AF00D0  52 69 63 68 3A 34 9F 75 00 00 00 00 00 00 00 00  Rich:4ƒu........
00AF00E0  50 45 00 00 4C 01 04 00 A3 89 73 44 00 00 00 00  PE..L☺♦.üësD....
00AF00F0  00 00 00 00 E0 00 0F 01 0B 01 07 00 00 50 00 00  ....ó.☼☺♂☺...P..
00AF0100  00 10 06 00 00 00 00 00 55 1D 00 00 00 10 00 00  .►♠.....U↔...►..
00AF0110  00 60 00 00 00 00 40 00 00 10 00 00 00 10 00 00  .`....@..►...►..
00AF0120  04 00 00 00 00 00 00 00 04 00 00 00 00 00 00 00  ♦.......♦.......
00AF0130  00 70 06 00 00 10 00 00 00 00 00 00 02 00 00 00  .p♠..►.......☻...
00AF0140  00 00 10 00 00 10 00 00 00 00 10 00 00 10 00 00  ..►..►....►..►..
00AF0150  00 00 00 00 10 00 00 00 00 00 00 00 00 00 00 00  ....►...........
00AF0160  3C 6F 00 00 78 00 00 00 00 90 00 00 40 D9 05 00  <o..x.....@┘♣.
00AF0170  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00AF0180  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00AF0190  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00AF01A0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Конечно, они будут совпадать с байтами Patrick.exe, находящимися по адресу 400000:



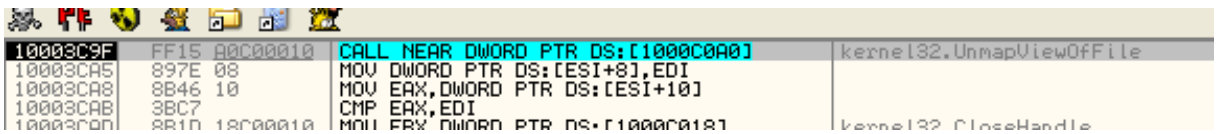
Address	Hex dump	ASCII
00400000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZé.♥...♦... ..
00400010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	®.......@.......
00400020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
00400030	00 00 00 00 00 00 00 00 00 00 00 00 E0 00 00 00	............ó...
00400040	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	♫▼‖♫.┤.=↑☺☺L=↑Th
00400050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program canno
00400060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS
00400070	6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00	mode....$.......
00400080	7E 55 F1 26 3A 34 9F 75 3A 34 9F 75 3A 34 9F 75	~U±&:4ƒu:4ƒu:4ƒu
00400090	D9 12 DF 75 34 34 9F 75 D9 12 83 75 1C 34 9F 75	┘↕▀u44ƒu┘↕âu∟4ƒu
004000A0	D9 12 86 75 38 34 9F 75 C0 17 86 75 33 34 9F 75	┘↕åu84ƒu└↨åu34ƒu
004000B0	3A 34 9E 75 50 34 9F 75 D9 12 82 75 37 34 9F 75	:4×uP4ƒu┘↕éu74ƒu
004000C0	C0 10 DA 75 3B 34 9F 75 D9 12 A2 75 3B 34 9F 75	└►ru;4ƒu┘↕óu;4ƒu
004000D0	52 69 63 68 3A 34 9F 75 00 00 00 00 00 00 00 00	Rich:4ƒu........
004000E0	50 45 00 00 4C 01 04 00 A3 89 73 44 00 00 00 00	PE..L☺♦.£ësD....
004000F0	00 00 00 00 E0 00 0F 01 0B 01 07 00 00 50 00 00	....ó.♫☺♂☺·...P..
00400100	00 10 06 00 00 00 00 00 55 1D 00 00 00 10 00 00	.►♠.....U↔...►..
00400110	00 60 00 00 00 00 40 00 00 10 00 00 00 10 00 00	.`....@..►...►..
00400120	04 00 00 00 00 00 00 00 04 00 00 00 00 00 00 00	♦.......♦.......
00400130	00 70 06 00 00 10 00 00 00 00 00 00 02 00 00 00	.p♠..►.......☻...
00400140	00 00 10 00 00 10 00 00 00 00 10 00 00 10 00 00	..►..►....►..►..
00400150	00 00 00 00 10 00 00 00 00 00 00 00 00 00 00 00	....►...........
00400160	3C 6F 00 00 78 00 00 00 00 90 00 00 40 D9 05 00	<o..x....É..@┘♣.
00400170	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	....
00400180	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Как видим, они абсолютно одинаковы. Далее, как это было с DLL'кой, сравнивается лишь несколько указателей заголовка, а не весь файл. Нажмем F9:



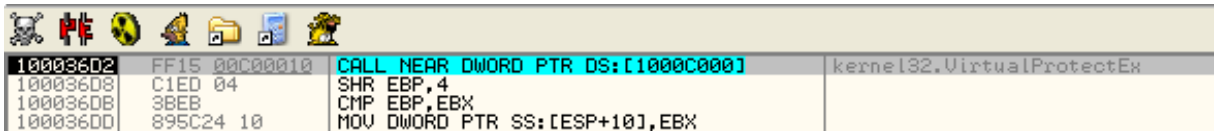
```
10003EF8   FF15 94C00010   CALL NEAR DWORD PTR DS:[1000C094]   kernel32.GetFileSize
10003EFE   8946 0C         MOV DWORD PTR DS:[ESI+C],EAX
10003F01   8B06            MOV EAX,DWORD PTR DS:[ESI]
10003F03   68 98C30010     PUSH 1000C398                       ASCII ".exe"
10003F08   50              PUSH EAX
10003F09   897E 04         MOV DWORD PTR DS:[ESI+4],EDI
10003F0C   E8 BF230000     CALL 100062D0                       AntiDebu.100062D0
10003F11   83C4 08         ADD ESP,8
10003F14   85C0            TEST EAX,EAX
10003F16   74 07           JE SHORT 10003F1F                   AntiDebu.10003F1F
10003F18   C746 04 010000  MOV DWORD PTR DS:[ESI+4],1
10003F1F   8B06            MOV EAX,DWORD PTR DS:[ESI]
10003F21   68 90C30010     PUSH 1000C390                       ASCII ".EXE"
10003F26   50              PUSH EAX
10003F27   E8 A4230000     CALL 100062D0                       AntiDebu.100062D0
10003F2C   83C4 08         ADD ESP,8
10003F2F   85C0            TEST EAX,EAX
10003F31   74 07           JE SHORT 10003F3A                   AntiDebu.10003F3A
10003F33   C746 04 010000  MOV DWORD PTR DS:[ESI+4],1
10003F3A   8B06            MOV EAX,DWORD PTR DS:[ESI]
10003F3C   68 88C30010     PUSH 1000C388                       ASCII ".dll"
10003F41   50              PUSH EAX
10003F42   E8 89230000     CALL 100062D0                       AntiDebu.100062D0
10003F47   83C4 08         ADD ESP,8
10003F4A   85C0            TEST EAX,EAX
10003F4C   74 07           JE SHORT 10003F55                   AntiDebu.10003F55
10003F4E   C746 04 020000  MOV DWORD PTR DS:[ESI+4],2
10003F55   8B06            MOV EAX,DWORD PTR DS:[ESI]
10003F57   68 80C30010     PUSH 1000C380                       ASCII ".DLL"
10003F5C   50              PUSH EAX
10003F5D   E8 6E230000     CALL 100062D0                       AntiDebu.100062D0
10003F62   83C4 08         ADD ESP,8
10003F65   85C0            TEST EAX,EAX
```

Опять происходит проверка расширения, но на этот раз у экзешника Patrick.exe.



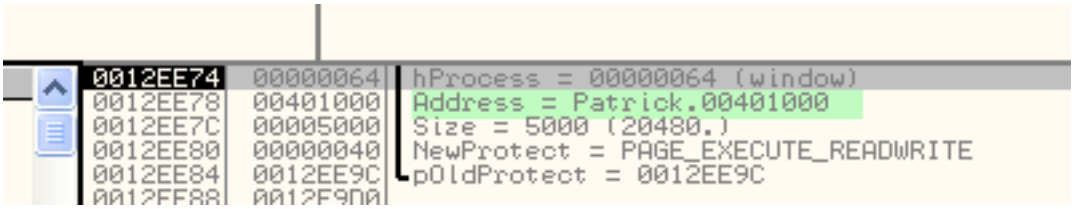
```
10003C9F   FF15 A0C00010   CALL NEAR DWORD PTR DS:[1000C0A0]   kernel32.UnmapViewOfFile
10003CA5   897E 08         MOV DWORD PTR DS:[ESI+8],EDI
10003CA8   8B46 10         MOV EAX,DWORD PTR DS:[ESI+10]
10003CAB   3BC7            CMP EAX,EDI
10003CAD   8B1D 18C00010   MOV EBX,DWORD PTR DS:[1000C018]   kernel32.CloseHandle
```

Затем изображение файла удаляется, хотя сравнение было неполным.

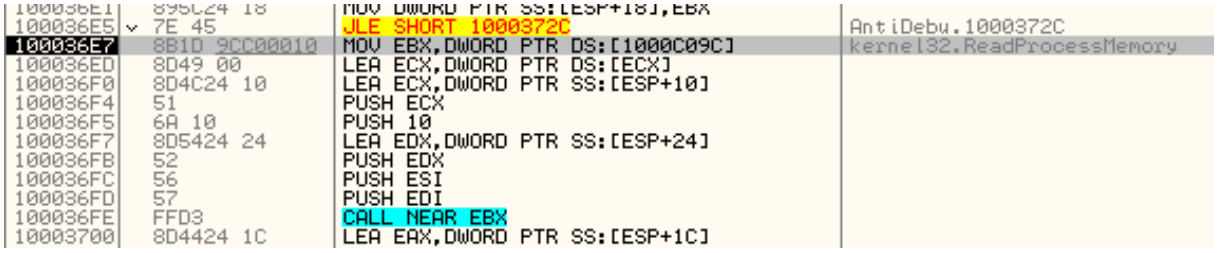


```
100036D2   FF15 00C00010   CALL NEAR DWORD PTR DS:[1000C000]   kernel32.VirtualProtectEx
100036D8   C1ED 04         SHR EBP,4
100036DB   3BEB            CMP EBP,EBX
100036DD   895C24 10       MOV DWORD PTR SS:[ESP+10],EBX
```

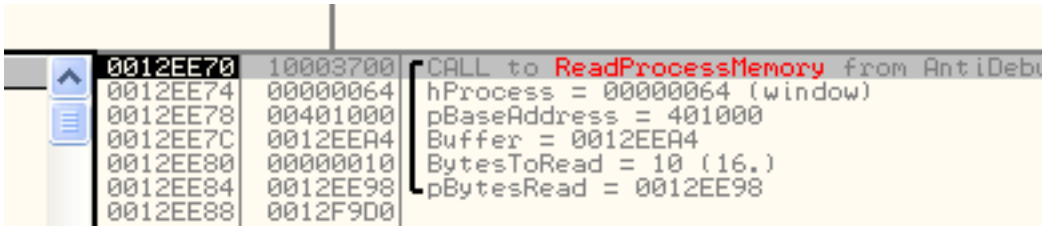
После пары незначительных API-функций изменяются настройки доступа потомка, находящегося в 401000; это нужно для расшифровки.



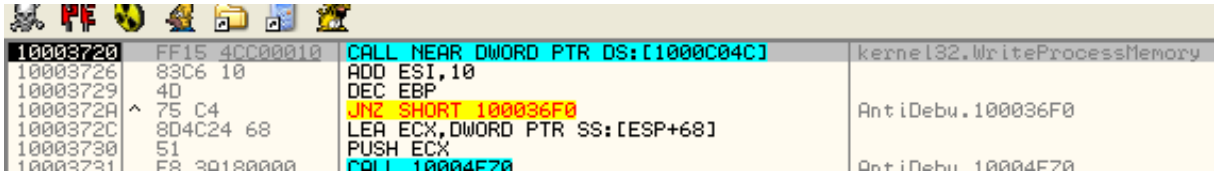
Через параметры функции передаются хэндл потомка и разрешение на чтение/запись в секции, начинающейся в 401000.

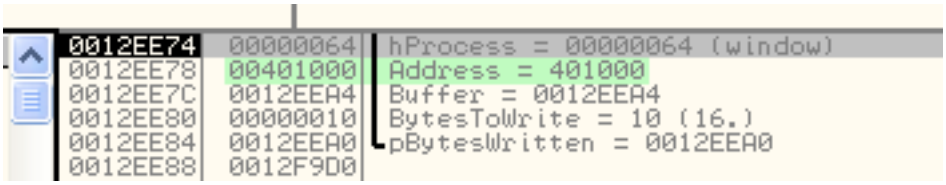


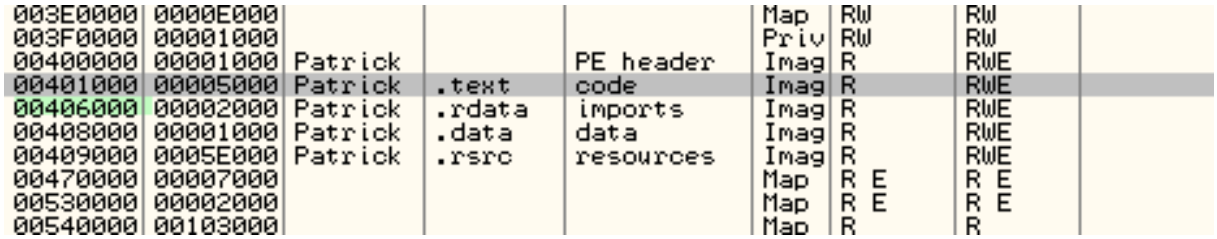
Так вызывается ReadProcessMemory — командой CALL, которая находится чуть ниже. Войдем в нее:



Из параметров функции видно, что будет прочитано 16 байтов, начиная с адреса 401000. После их расшифровки они будут сохранены там же функцией WriteProcessMemory. В 401000 родителя окончательные байты не видны, но это не проблема, так как далее потомок произведет для него повторную расшифровку.

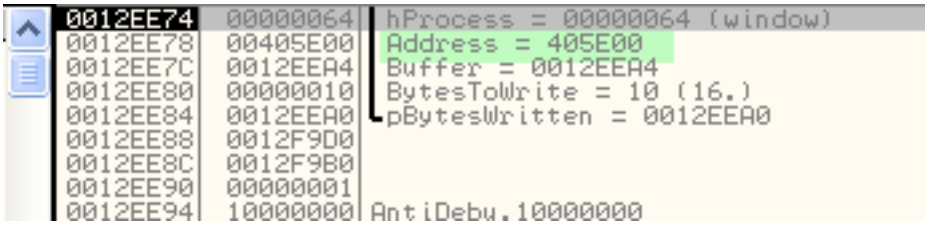




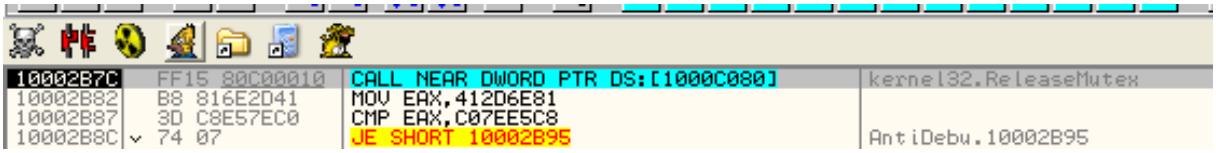


Этот цикл будет повторяться вплоть до начала следующей секции в 406000, так что нам придется терпеливо удерживать клавишу F9 нажатой, увеличивая адрес, пока не окажемся рядом с 406000,

где скорость надо будет замедлить, чтобы не проскочить.

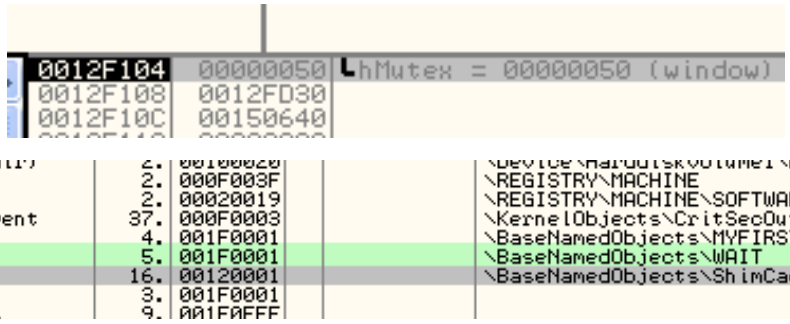


Скоро произойдет завершение цикла, поэтому следует отпустить клавишу и продолжать продвигаться уже отдельными нажатиями.



Затем вызывается RealeaseMutex для освобождения потомка, который продолжает выполняться. Вспомним, что потомок выйдет из функции WaitForSingleObject только при освобождении родителем мьютекса, названного WAIT.

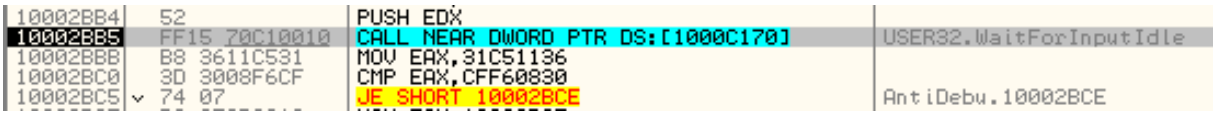
Если посмотреть параметры, то там указан хэндл 50 (так на моем компьютере):



Он, конечно же, соответствует мьютексу (или «Mutant», как указано в Olly) с именем WAIT. Сейчас потомок запущен и ожидает освобождения. После нажатия на F8 он будет освобожден и продолжит дальнейшее выполнение, поэтому на строке, следующей за вызовом WaitForSingleObject, был установлен брейк.

Нажмем F8, чтобы потомок освободился и остановился на HE.

Теперь свободны оба процесса, но мы пока продолжим работать с родителем: с помощью нашего OllyKam'a посмотрим, что будет далее:



Этой API-функцией устанавливается другое ожидание, на этот раз в родителе:

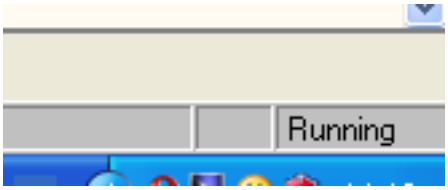
Remarks

The **WaitForInputIdle** function enables a thread to suspend its execution until a specified process has finished its initialization and is waiting for user input with no input pending. This can be useful for synchronizing a parent process and a newly created child process. When a parent process creates a child process, the **CreateProcess** function returns without waiting for the child process to finish its initialization. Before trying to communicate with the child process, the parent process can use **WaitForInputIdle** to determine when the child's initialization has been completed. For example, the parent process should use **WaitForInputIdle** before trying to find a window associated with the child process.

Единственный параметр задает период времени, после окончания которого происходит выход из API-функции. Но в данном случае это не случится, так как задана бесконечность; выход произойдет только в момент запуска потомка при вводе с клавиатуры или подобном событии.

Как и прежде, установим НЕ на следующей за вызовом команде и нажмем RUN:

```
10002BB5   FF15 70C10010   CALL NEAR DWORD PTR DS:[1000C170]   USER32.WaitForInputIdle
10002BBB   B8 3611C531     MOV EAX,31C51136
10002BC0   3D 3008F6CF     CMP EAX,CFF60830
10002BC5   74 07           JE SHORT 10002BCE                  AntiDebu.10002BCE
```



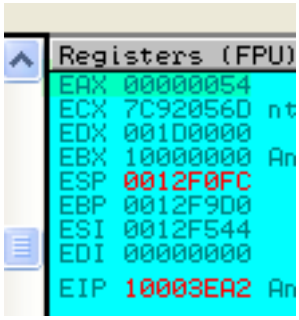
Теперь родитель будет находиться в состоянии Running до тех пор, пока отпрыск не вырастет, а мы тем временем поможем ему подрасти, хе-хе. Продолжим юзать его OllyKam:

```
10002DFF   FF15 80C00010   CALL NEAR DWORD PTR DS:[1000C080]   kernel32.ReleaseMutex
10002E05   B8 B2B1E078     MOV EAX,78E0B1B2
10002E0A   3D 10A58B89     CMP EAX,898BA510
10002E0F   74 07           JE SHORT 10002E18                  AntiDebu.10002E18
```

На этот раз мьютекс WAIT будет освобожден в потомке, но на родителя это никак не повлияет, поскольку он находится в функции WaitForInputIdle, а не WaitForSingleObject и ожидает запуска потомка. Продолжим:

```
10003E9C   FF15 98C00010   CALL NEAR DWORD PTR DS:[1000C098]   kernel32.CreateFileA
10003EA2   83F8 FF         CMP EAX,-1
10003EA5   8946 14         MOV DWORD PTR DS:[ESI+14],EAX
10003EA8   74 44           JE SHORT 10003EEE                  AntiDebu.10003EEE
```

Здесь потомок впервые считывает DLL'ку, чтобы проверить ее содержимое. Так как это происходит в первый раз, то проблем не возникает и извлекается валидный хэндл (скриншоты далее привожу без комментариев, ибо этот трюк уже объяснялся в родителе):




```
10003EB0   50              PUSH EAX
10003EB1   FF15 A8C00010   CALL NEAR DWORD PTR DS:[1000C0A8]   kernel32.CreateFileMappingA
10003EB7   3BC7            CMP EAX,EDI
10003EB9   8946 10         MOV DWORD PTR DS:[ESI+10],EAX
10003EBC   57              PUSH EDI
10003EBD   75 0B           JNZ SHORT 10003ECA                 AntiDebu.10003ECA
10003EBF   FF15 18C00010   CALL NEAR DWORD PTR DS:[1000C018]  kernel32.CloseHandle
10003EC5   5F              POP EDI
```

```
10003ECB   57              PUSH EDI
10003ECC   6A 04           PUSH 4
10003ECE   50              PUSH EAX
10003ECF   FF15 A4C00010   CALL NEAR DWORD PTR DS:[1000C0A4]   kernel32.MapViewOfFile
10003ED5   3BC7            CMP EAX,EDI
10003ED7   8946 08         MOV DWORD PTR DS:[ESI+8],EAX
10003EDA   75 17           JNZ SHORT 10003EF3                 AntiDebu.10003EF3
```

```
10003EF7   52              PUSH EDX
10003EF8   FF15 94C00010   CALL NEAR DWORD PTR DS:[1000C094]   kernel32.GetFileSize
10003EFE   8946 0C         MOV DWORD PTR DS:[ESI+C],EAX
10003F01   8B06            MOV EAX,DWORD PTR DS:[ESI]
10003F03   68 98C30010     PUSH 1000C398                       ASCII ".exe"
10003F08   50              PUSH EAX
10003F09   897E 04         MOV DWORD PTR DS:[ESI+4],EDI
10003F0C   E8 BF230000     CALL 100062D0                       AntiDebu.100062D0
10003F11   83C4 08         ADD ESP,8
10003F14   85C0            TEST EAX,EAX
10003F16   74 07           JE SHORT 10003F1F                   AntiDebu.10003F1F
10003F18   C746 04 010000  MOV DWORD PTR DS:[ESI+4],1
10003F1F   8B06            MOV EAX,DWORD PTR DS:[ESI]
10003F21   68 90C30010     PUSH 1000C390                       ASCII ".EXE"
10003F26   50              PUSH EAX
10003F27   E8 A4230000     CALL 100062D0                       AntiDebu.100062D0
10003F2C   83C4 08         ADD ESP,8
10003F2F   85C0            TEST EAX,EAX
10003F31   74 07           JE SHORT 10003F3A                   AntiDebu.10003F3A
10003F33   C746 04 010000  MOV DWORD PTR DS:[ESI+4],1
10003F3A   8B06            MOV EAX,DWORD PTR DS:[ESI]
10003F3C   68 88C30010     PUSH 1000C388                       ASCII ".dll"
10003F41   50              PUSH EAX
10003F42   E8 89230000     CALL 100062D0                       AntiDebu.100062D0
10003F47   83C4 08         ADD ESP,8
10003F4A   85C0            TEST EAX,EAX
10003F4C   74 07           JE SHORT 10003F55                   AntiDebu.10003F55
10003F4E   C746 04 020000  MOV DWORD PTR DS:[ESI+4],2
10003F55   8B06            MOV EAX,DWORD PTR DS:[ESI]
10003F57   68 80C30010     PUSH 1000C380                       ASCII ".DLL"
10003F5C   50              PUSH EAX
10003F5D   E8 6E230000     CALL 100062D0                       AntiDebu.100062D0
10003F62   83C4 08         ADD ESP,8
10003F65   85C0            TEST EAX,EAX
```

```
10003C9F   FF15 A0C00010   CALL NEAR DWORD PTR DS:[1000C0A0]   kernel32.UnmapViewOfFile
10003CA5   897E 08         MOV DWORD PTR DS:[ESI+8],EDI
10003CA8   8B46 10         MOV EAX,DWORD PTR DS:[ESI+10]
```

Очередь доходит до ключевого вызова:

```
1000388C   FF15 98C00010   CALL NEAR DWORD PTR DS:[1000C098]   kernel32.CreateFileA
10003892   8BF8            MOV EDI,EAX
10003894   3BFB            CMP EDI,EBX
```

```
0012EF4C   0012EFA8   FileName = "AntiDebugDll.dll"
0012EF50   80000000   Access = GENERIC_READ
0012EF54   00000000   ShareMode = 0
0012EF58   00000000   pSecurity = NULL
0012EF5C   00000003   Mode = OPEN_EXISTING
0012EF60   00000000   Attributes = 0
0012EF64   00000000   hTemplateFile = NULL
0012EF68   0012FD30
```

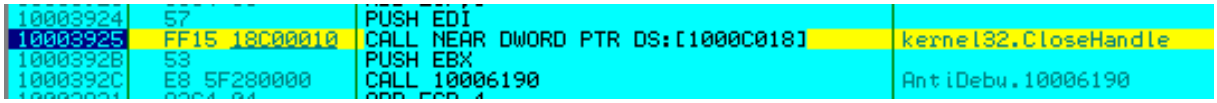
Как и прежде, обойдем баг Олли, изменив параметр dwShareMode на 1:

```
Registers (FPU)
EAX 00000054
ECX 7C92056D ntd
EDX 001E0000
EBX 00000000
ESP 0012EF68
EBP 0012F0F4
ESI 00000001
EDI 0012F0D4
EIP 10003892 Ant
```

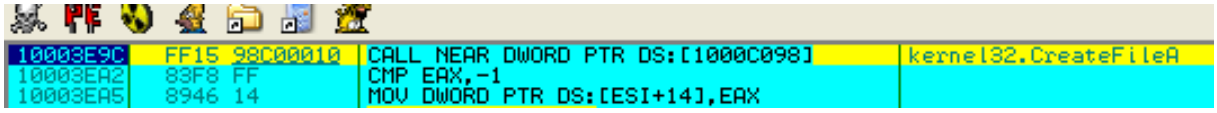
Возвращен валидный хэндл.

```
100038C7   FF15 90C00010   CALL NEAR DWORD PTR DS:[1000C090]   kernel32.ReadFile
100038CD   85C0            TEST EAX,EAX
100038CF   74 53           JE SHORT 10003924                   AntiDebu.10003924
100038D1   8B7C24 10       MOV EDI,DWORD PTR SS:[ESP+10]
```

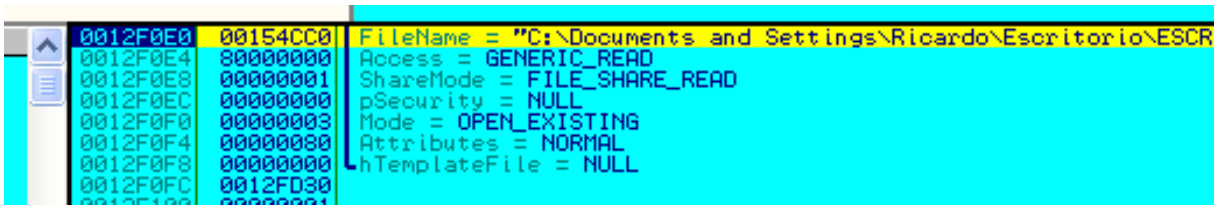
После пары незначительных API-функций вызывается ReadFile с хэндлом файла в параметрах для считывания его байтов и их сравнения. Так проверяется оригинальность файла.



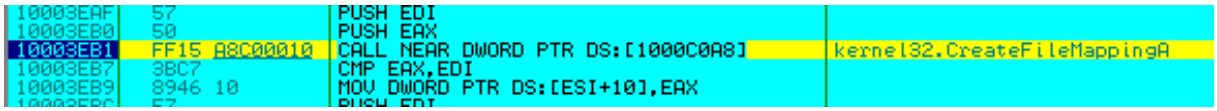
После проведения проверки хэндл закрывается.



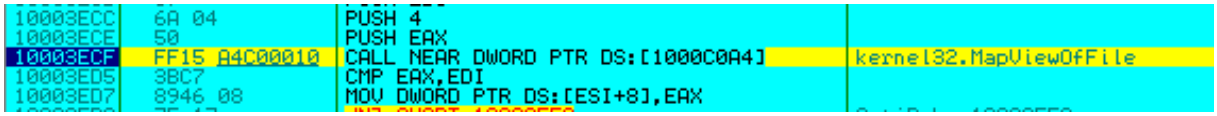
Теперь опять CreateFileA, но уже для экзешника Patrick.exe:

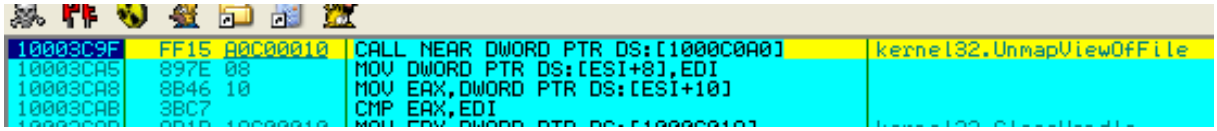


Здесь проблем не возникает и хэндл возвращается.

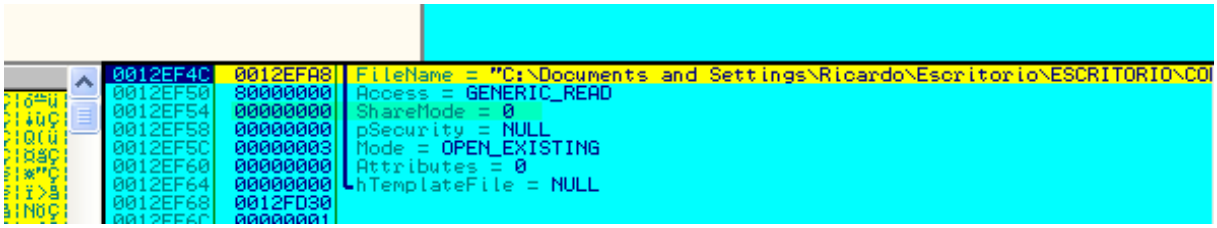


Происходит то же самое, что было в родителе, поэтому опустим подробности.

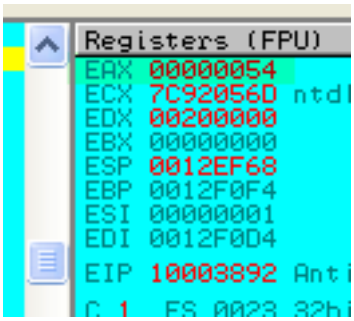




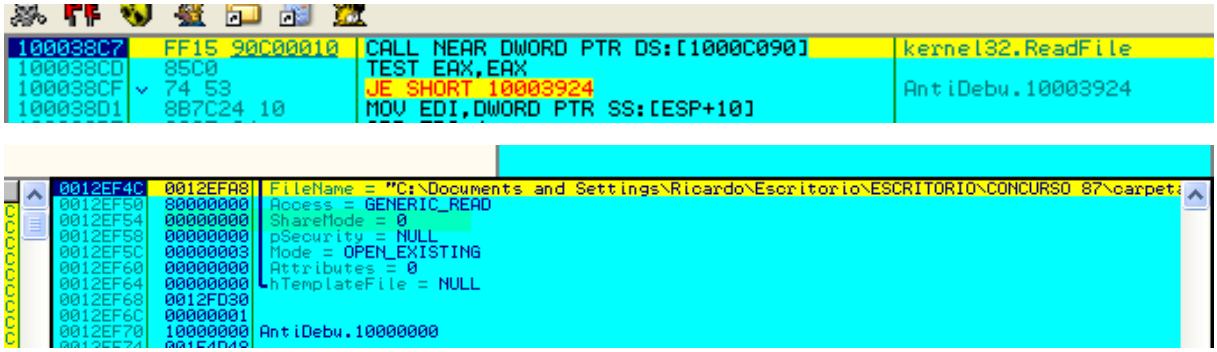
Теперь тот же трюк применяется к файлу Patrick.exe:



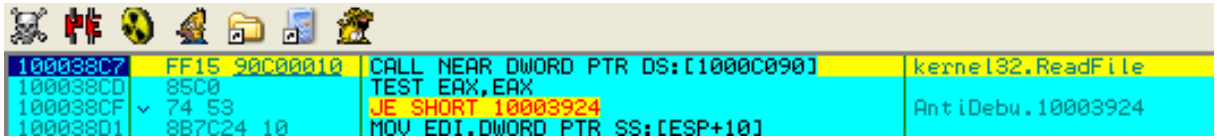
Но мы изменим dwShareMode на 1, и хэндл будет возвращен:



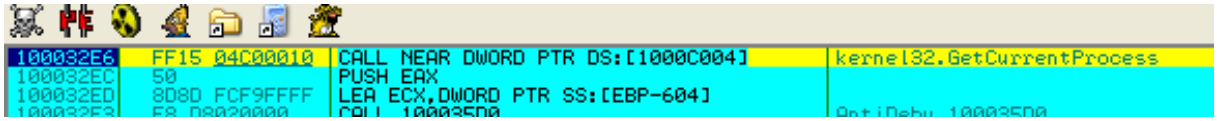
Продолжим нажимать на F9, и после нескольких API-функций дойдем до ReadFile, выполняющей проверку содержимого файла:



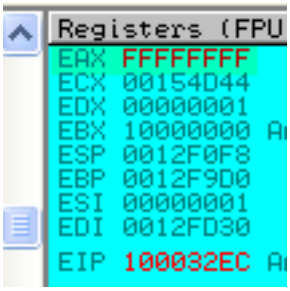
Опять то же самое! Что ж, изменим dwShareMode на 1. Предполагается, что если ты смог дойти досюда, то трюк уже раскрыт.

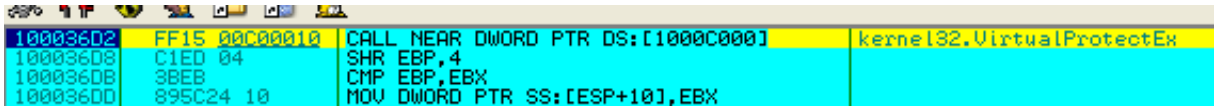


Снова вызывается ReadFile для проверки.

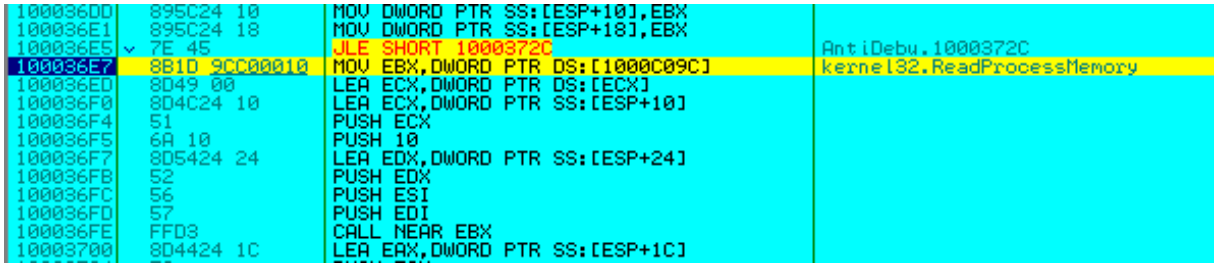


Дошли до GetCurrentProcess, которая возвращает псевдохэндл (хэндл выполняющегося процесса) FFFFFFFF. Наконец-то эти проверки пройдены и начинается что-то новое!

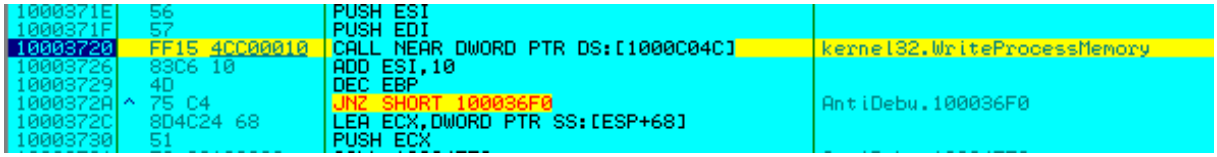




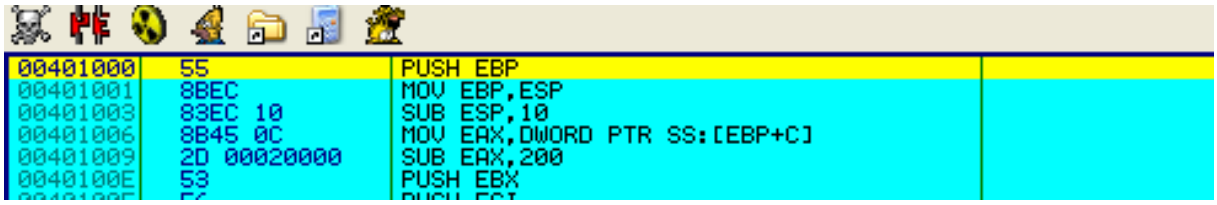
Здесь, как и ранее в родителе, разрешается чтение и запись в секцию кода программы для обеспечения функционирования ReadProcessMemory и WriteProcessMemory.



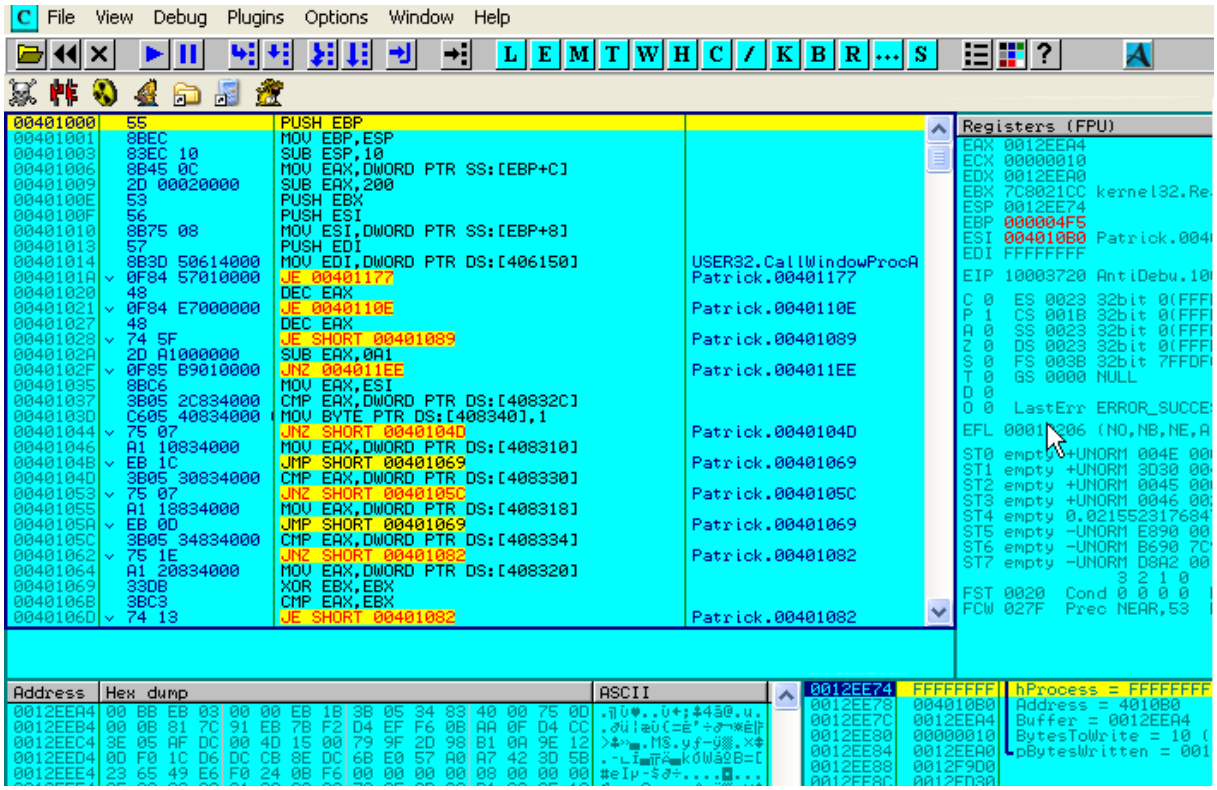
Теперь вызывается WriteProcessMemory:



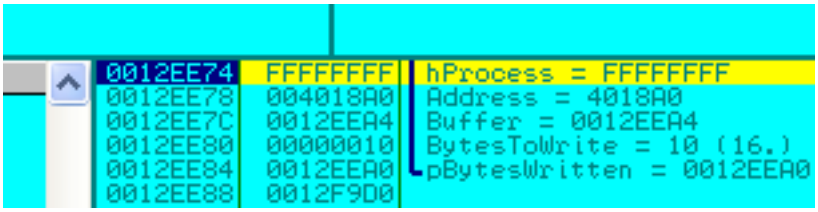
После нажатия на F8 расшифруются первые 16 байтов и их можно будет увидеть по адресу 401000:



Как видим, они расшифрованы верно.



Остается добавить только штрих, хе-хе.



Как и в прошлый раз, надо удерживать нажатой клавишу F9 до тех пор, пока не окажемся около 406000.

Область OEP расшифрована тоже правильно:

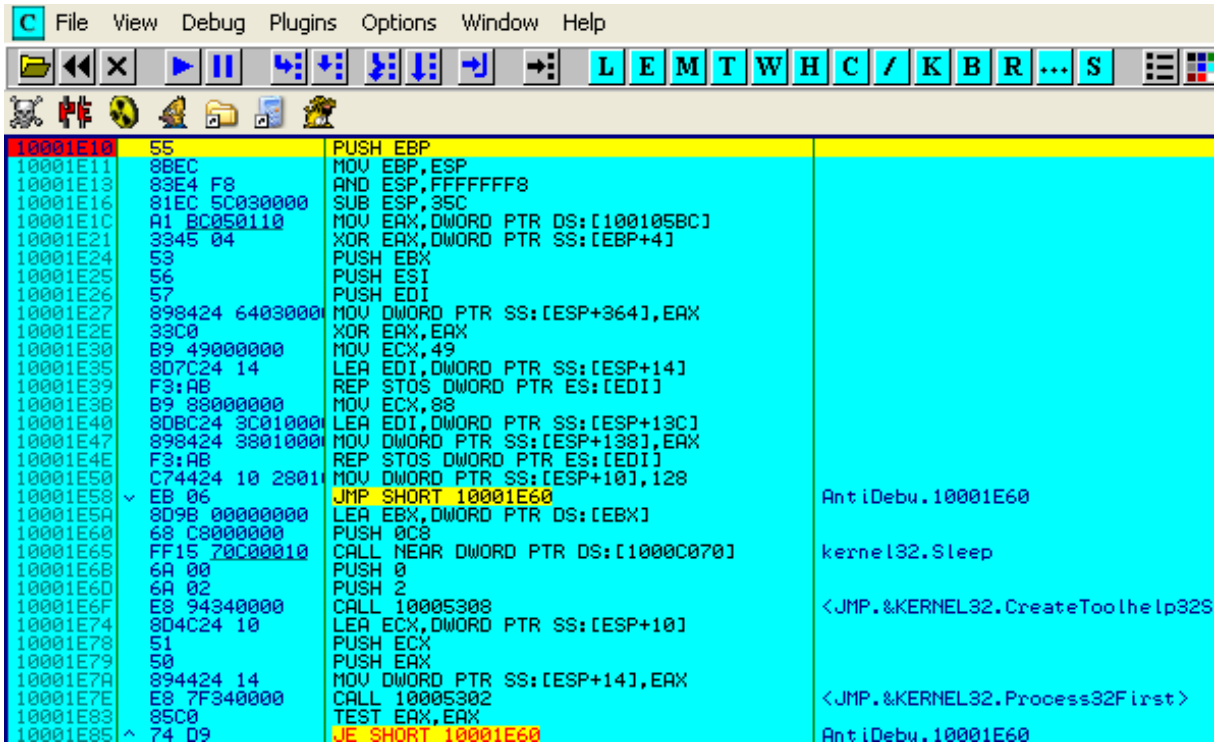
```
00401D55    6A 60           PUSH 60
00401D57    68 60644000     PUSH 406460
00401D5C    E8 6F030000     CALL 004020D0                    Patrick.004020D0
00401D61    BF 94000000     MOV EDI,94
00401D66    8BC7            MOV EAX,EDI
00401D68    E8 43FDFFFF     CALL 00401AB0                    Patrick.00401AB0
00401D6D    8965 E8         MOV DWORD PTR SS:[EBP-18],ESP
00401D70    8BF4            MOV ESI,ESP
00401D72    893E            MOV DWORD PTR DS:[ESI],EDI
00401D74    56              PUSH ESI
00401D75    FF15 AC604000   CALL NEAR DWORD PTR DS:[4060AC]  kernel32.GetVersionE
00401D7B    8B4E 10         MOV ECX,DWORD PTR DS:[ESI+10]
00401D7E    890D 84834000   MOV DWORD PTR DS:[408384],ECX
00401D84    8B46 04         MOV EAX,DWORD PTR DS:[ESI+4]
00401D87    A3 90834000     MOV DWORD PTR DS:[408390],EAX
00401D8C    8B56 08         MOV EDX,DWORD PTR DS:[ESI+8]
00401D8F    8915 94834000   MOV DWORD PTR DS:[408394],EDX
00401D95    8B76 0C         MOV ESI,DWORD PTR DS:[ESI+C]
00401D98    81E6 FF7F0000   AND ESI,7FFF
00401D9E    8935 88834000   MOV DWORD PTR DS:[408388],ESI
00401DA4    83F9 02         CMP ECX,2
00401DA7    74 0C           JE SHORT 00401DB5                Patrick.00401DB5
00401DA9    81CE 00800000   OR ESI,8000
00401DAF    8935 88834000   MOV DWORD PTR DS:[408388],ESI
```

```
100033E1    6A 00           PUSH 0
100033E3    6A 00           PUSH 0
100033E5    FF15 74C00010   CALL NEAR DWORD PTR DS:[1000C074]  kernel32.CreateThread
100033EB    B8 303ED0F0     MOV EAX,F0D03E30
100033F0    3D 80C5B112     CMP EAX,12B1C580
100033F5    74 07           JE SHORT 100033FE                AntiDebu.100033FE
100033F7    B8 FF330010     MOV EAX,100033FF
100033FC    FFE0            JMP NEAR EAX
100033FE    D88D 8DC4F9FF   FMUL DWORD PTR SS:[EBP+FFF9C48D]
10003404    FF51 6A         CALL NEAR DWORD PTR DS:[ECX+6A]
10003407    006A 00         ADD BYTE PTR DS:[EDX],CH
1000340A    68 101E0010     PUSH 10001E10
1000340F    6A 00           PUSH 0
10003411    6A 00           PUSH 0
10003413    FF15 74C00010   CALL NEAR DWORD PTR DS:[1000C074]  kernel32.CreateThread
10003419    B8 124C73B7     MOV EAX,B7734C12
1000341E    3D 10A0144C     CMP EAX,4C14A010
```

После окончания расшифровки создаются потоки защиты. Можно создать их замороженными, изменив параметр dwCreationFlags на 4.

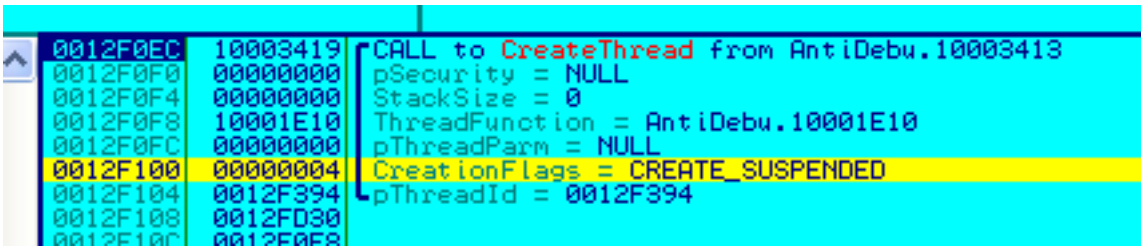
```
10002140    51              PUSH ECX
10002141    56              PUSH ESI
10002142    8B35 70C00010   MOV ESI,DWORD PTR DS:[1000C070]  kernel32.Sleep
10002148    68 C8000000     PUSH 0C8
1000214D    FFD6            CALL NEAR ESI
1000214F    C74424 04 0000  MOV DWORD PTR SS:[ESP+4],0
10002157    50              PUSH EAX
10002158    64:A1 18000000  MOV EAX,DWORD PTR FS:[18]
1000215E    3E:8B40 30      MOV EAX,DWORD PTR DS:[EAX+30]
10002162    3E:0FB640 02    MOVZX EAX,BYTE PTR DS:[EAX+2]
10002167    894424 08       MOV DWORD PTR SS:[ESP+8],EAX
1000216B    58              POP EAX
1000216C    8B4424 04       MOV EAX,DWORD PTR SS:[ESP+4]
10002170    85C0            TEST EAX,EAX
10002172    74 D4           JE SHORT 10002148                AntiDebu.10002148
10002174    6A 00           PUSH 0
10002176    FF15 0CC00010   CALL NEAR DWORD PTR DS:[1000C00C]  kernel32.ExitProcess
1000217C    CC              INT3
1000217D    90              NOP
1000217E    90              NOP
1000217F    90              NOP
10002180    68 0E000780     PUSH 8007000E
10002185    E8 F6EFFFFF     CALL 10001180                    AntiDebu.10001180
1000218A    CC              INT3
1000218B    90              NOP
1000218C    90              NOP
1000218D    90              NOP
```

Это начало первого треда.

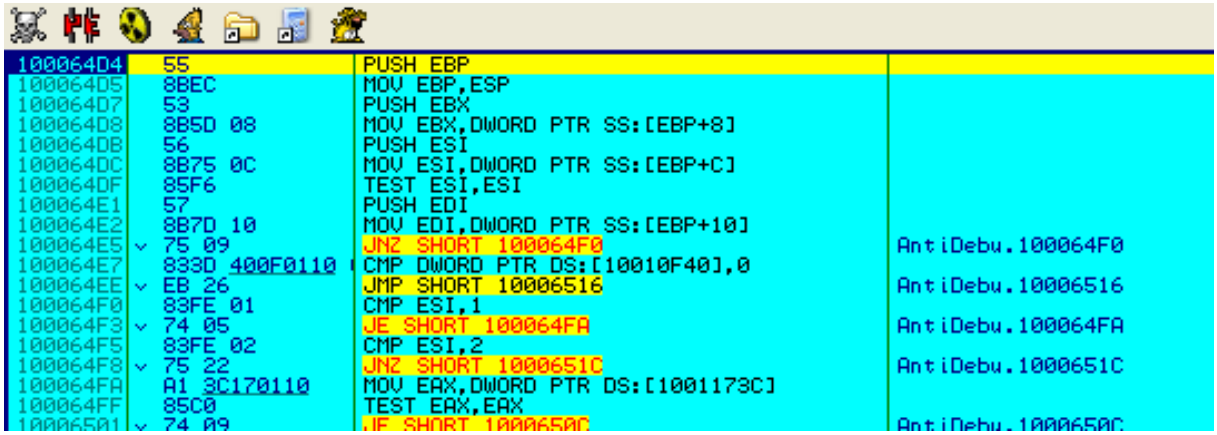


А это — второго. Здесь снова будет произведена проверка выполняющихся процессов, поскольку видны соответствующие API-функции.

Итак, изменим параметр dwCreationFlags на CREATE_SUSPENDED:

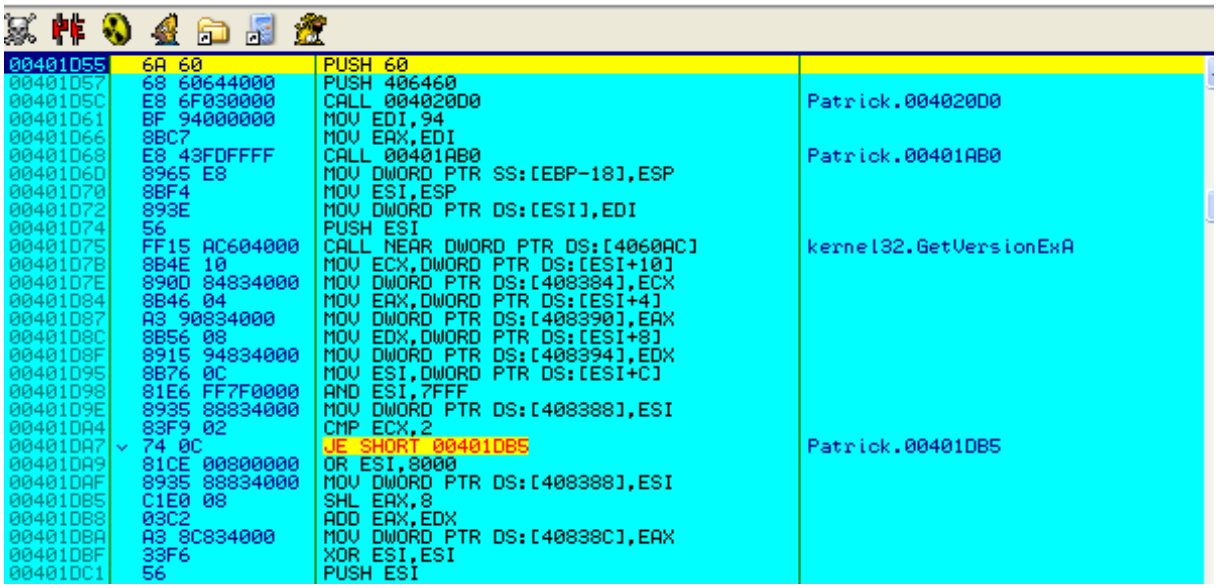


После создания тредов на всякий случай снимем все аппаратные брейки, установим BPM ON ACCESS в секцию кода Патрика (начинающуюся в 401000) и BREAK ON ACCESS [Break-on-access?] в секцию кода AntiDebugDll.dll. После нажатия на F9 произошла незапланированная остановка в этой области:

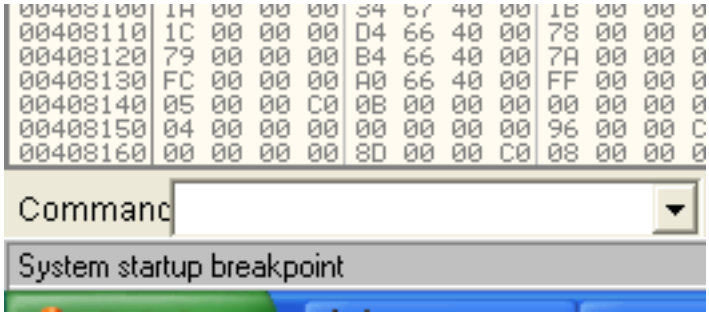


Можно оказаться на OEP Патрика быстрее, но так как галки в настройках исключений Олли оказались сняты и BPM установлен на всю секцию [в секцию кода AntiDebugDll.dll?], нажмем F9. Одно исключение генерируется этой командой:

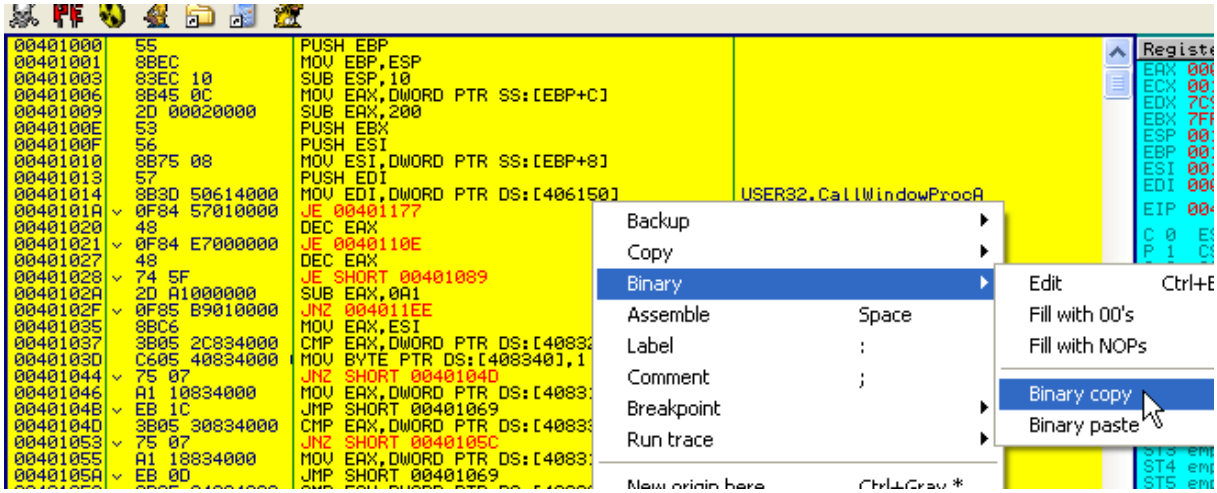
Это попытка перейти на адрес 00000000. После нажатия на Shift+F9 окажемся в OEP из-за остановки на BREAK ON ACCESS [Рикардо что-то мудрит…]



Теперь можно было бы сразу приступить к сдампливанию, но мы поступим иначе: откроем еще один Patrick.exe в третьем OllyDbg и оставим его на системном брейке:

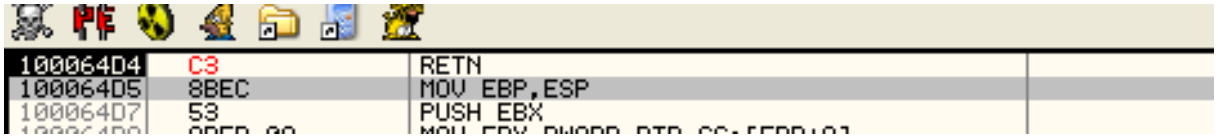


Теперь скопируем в его первую секцию все байты первой секции потомка:

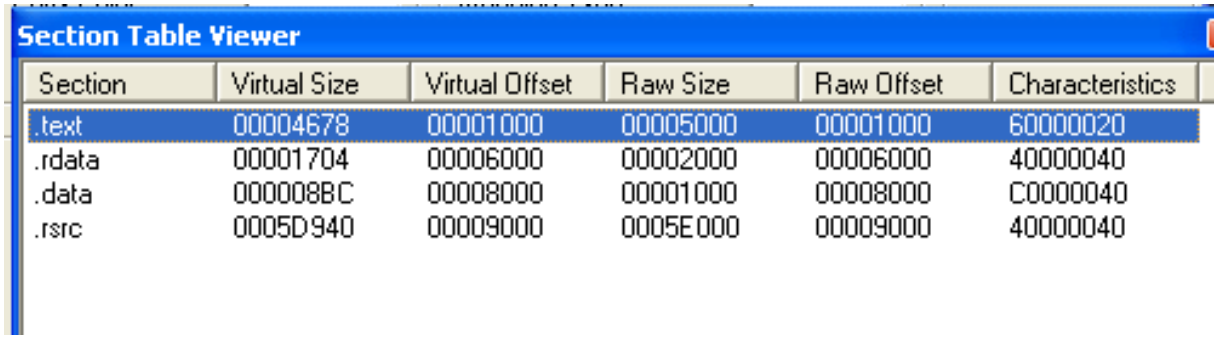


Сохраним изменения: Copy to executable. Поскольку здесь правильная IAT, то проблем возникнуть не должно.

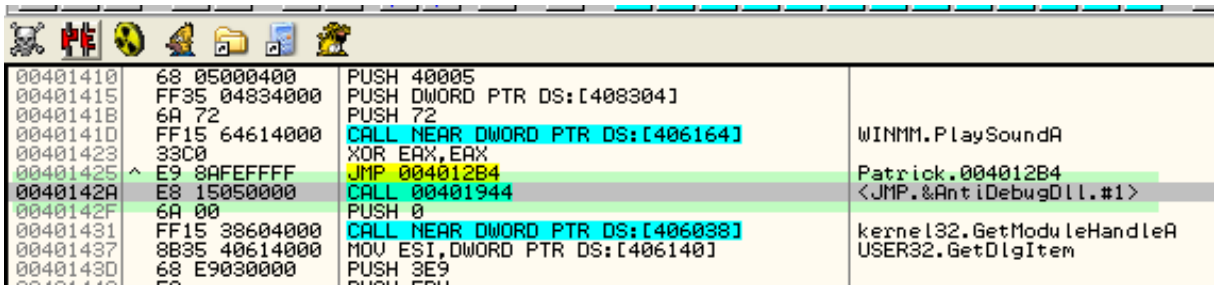
После открытия сохраненного Патрика в его первой секции содержится правильный код уже на System startup breakpoint. Но проблема в том, что далее запускается гнусная DLL'ка, поэтому перед ее удалением следует установить RET в ее EP и сохранить изменения:



Потом откроем Patrick.exe в PE Editor'е и изменим настройки доступа всех секций на E0000020:

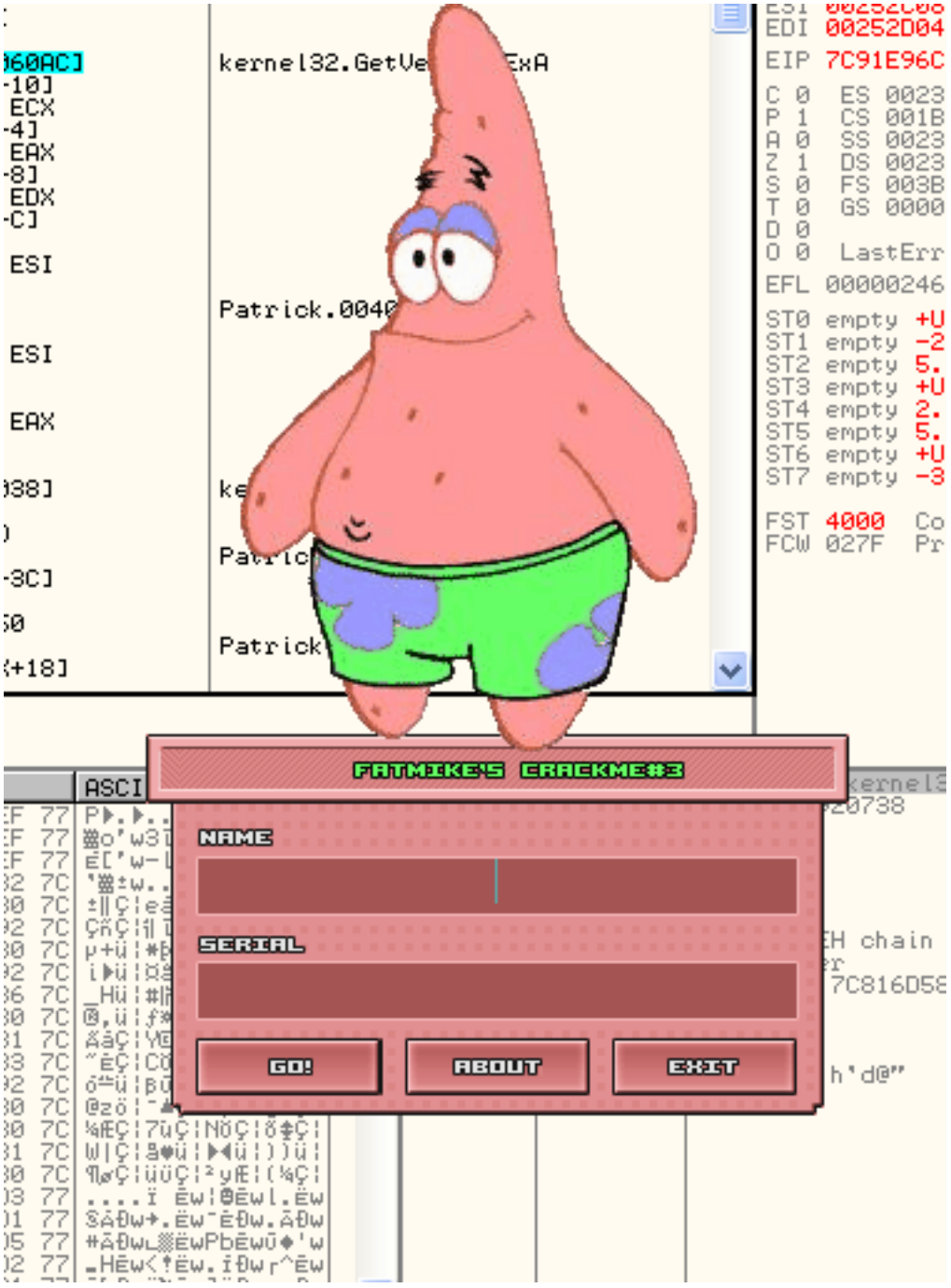


А также есть CALL, который вызывает AntiDebugDll.dll — просто занопаем его:

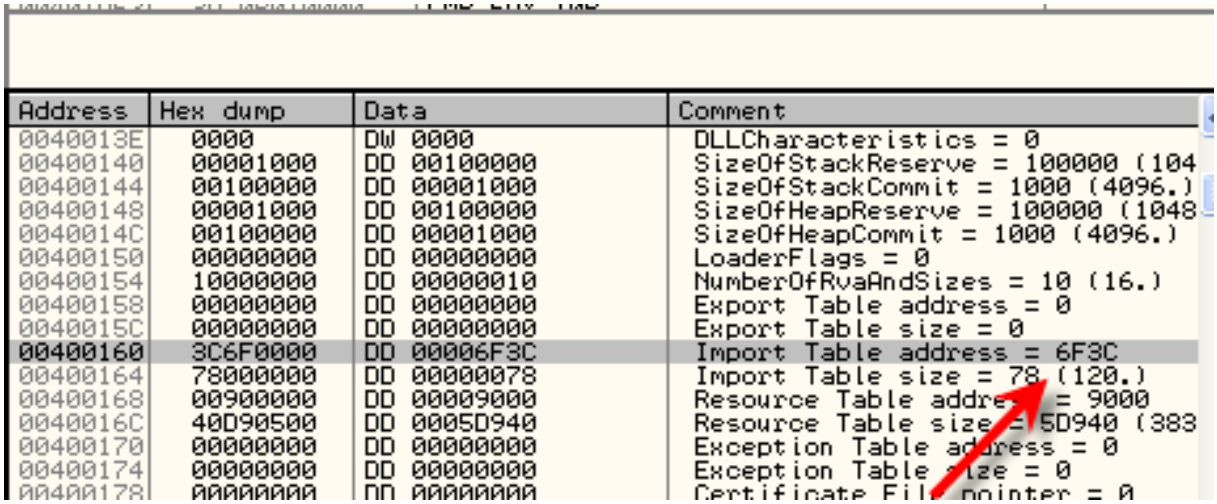


```
0040141D   FF15 64614000   CALL NEAR DWORD PTR DS:[406164]   WINMM.PlaySour
00401423   33C0            XOR EAX,EAX
00401425 ^ E9 8AFEFFFF     JMP 004012B4                      Patrick.004012
0040142A   90              NOP
0040142B   90              NOP
0040142C   90              NOP
0040142D   90              NOP
0040142E   90              NOP
0040142F   6A 00           PUSH 0
00401431   FF15 38604000   CALL NEAR DWORD PTR DS:[406038]   kernel32.GetM
00401437   8B35 40614000   MOV ESI,DWORD PTR DS:[406140]     USER32.GetDlg
0040143D   68 E9030000     PUSH 3E9
00401442   53              PUSH EBX
```

Сохраненим изменения и перезагрузим крэкми:

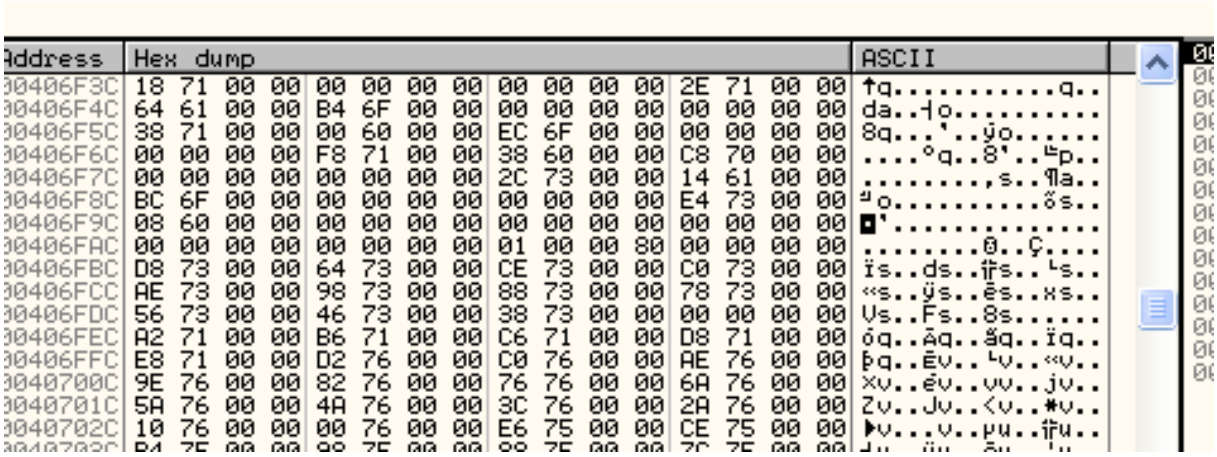


Теперь он запускается, но чтобы DLL'ка вообще не загружалась, следует сделать следующее:



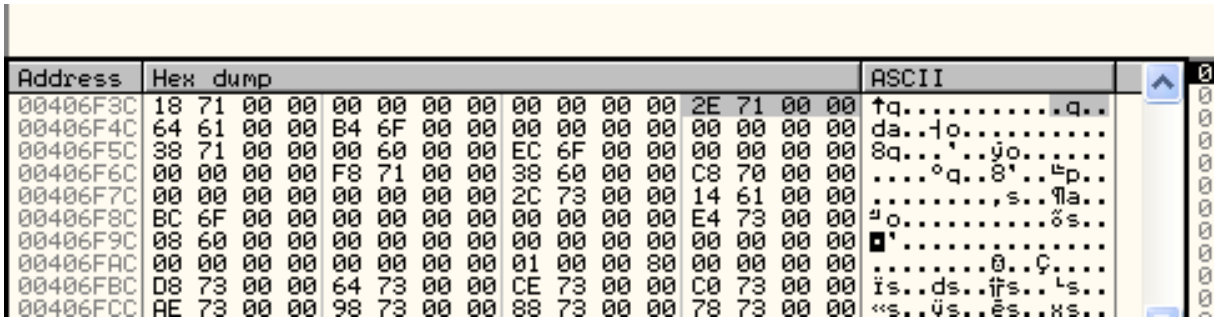
Address	Hex dump	Data	Comment
0040013E	0000	DW 0000	DLLCharacteristics = 0
00400140	00001000	DD 00100000	SizeOfStackReserve = 100000 (104
00400144	00100000	DD 00001000	SizeOfStackCommit = 1000 (4096.)
00400148	00001000	DD 00100000	SizeOfHeapReserve = 100000 (1048
0040014C	00100000	DD 00001000	SizeOfHeapCommit = 1000 (4096.)
00400150	00000000	DD 00000000	LoaderFlags = 0
00400154	10000000	DD 00000010	NumberOfRvaAndSizes = 10 (16.)
00400158	00000000	DD 00000000	Export Table address = 0
0040015C	00000000	DD 00000000	Export Table size = 0
00400160	3C6F0000	DD 00006F3C	Import Table address = 6F3C
00400164	78000000	DD 00000078	Import Table size = 78 (120.)
00400168	00900000	DD 00009000	Resource Table addre = 9000
0040016C	40D90500	DD 0005D940	Resource Table size = 5D940 (383
00400170	00000000	DD 00000000	Exception Table address = 0
00400174	00000000	DD 00000000	Exception Table size = 0
00400178	00000000	DD 00000000	Certificate File pointer = 0

Найдем в заголовке адрес начала IT — 406F3C.



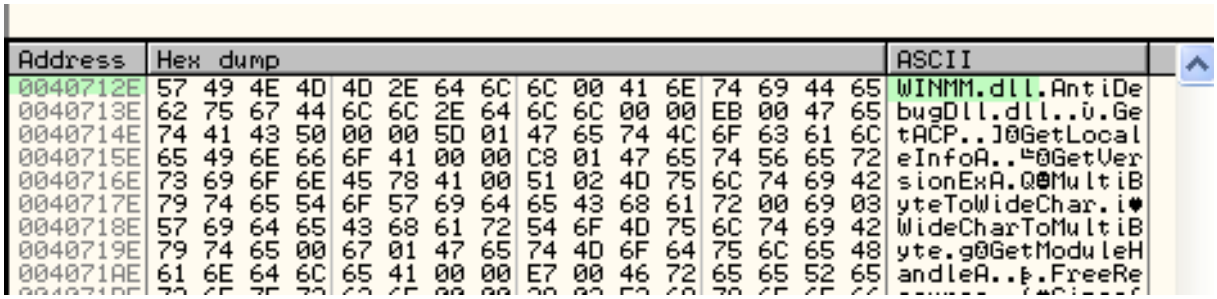
Address	Hex dump	ASCII
00406F3C	18 71 00 00 00 00 00 00 00 00 00 00 2E 71 00 00	↑q..........q..
00406F4C	64 61 00 00 B4 6F 00 00 00 00 00 00 00 00 00 00	da..┤o..........
00406F5C	38 71 00 00 00 60 00 00 EC 6F 00 00 00 00 00 00	8q...`..ìo......
00406F6C	00 00 00 00 F8 71 00 00 38 60 00 00 C8 70 00 00	....°q..8`..╚p..
00406F7C	00 00 00 00 00 00 00 00 2C 73 00 00 14 61 00 00	........,s..¶a..
00406F8C	BC 6F 00 00 00 00 00 00 00 00 00 00 E4 73 00 00	╝o..........õs..
00406F9C	08 60 00 00 00 00 00 00 00 00 00 00 00 00 00 00	◘`..............
00406FAC	00 00 00 00 00 00 00 00 01 00 00 80 00 00 00 00	........Ø..Ç....
00406FBC	D8 73 00 00 64 73 00 00 CE 73 00 00 C0 73 00 00	ïs..ds..╬s..└s..
00406FCC	AE 73 00 00 98 73 00 00 88 73 00 00 78 73 00 00	«s..ÿs..ês..xs..
00406FDC	56 73 00 00 46 73 00 00 38 73 00 00 00 00 00 00	Vs..Fs..8s......
00406FEC	A2 71 00 00 B6 71 00 00 C6 71 00 00 D8 71 00 00	óq..Âq..ãq..ïq..
00406FFC	E8 71 00 00 D2 76 00 00 C0 76 00 00 AE 76 00 00	Þq..Êv..└v..«v..
0040700C	9E 76 00 00 82 76 00 00 76 76 00 00 6A 76 00 00	xv..év..vv..jv..
0040701C	5A 76 00 00 4A 76 00 00 3C 76 00 00 2A 76 00 00	Zv..Jv..<v..*v..
0040702C	10 76 00 00 00 76 00 00 E6 75 00 00 CE 75 00 00	▶v...v..µu..╬u..

Это IT. Вспомним, что первые 5 DWORD'ов соответствуют 1-й DLL'ке. Узнаем ее имя по ее указателю — 4-му DWORD'у:



Address	Hex dump	ASCII
00406F3C	18 71 00 00 00 00 00 00 00 00 00 00 2E 71 00 00	↑q..........q..
00406F4C	64 61 00 00 B4 6F 00 00 00 00 00 00 00 00 00 00	da..┤o..........
00406F5C	38 71 00 00 00 60 00 00 EC 6F 00 00 00 00 00 00	8q...`..ìo......
00406F6C	00 00 00 00 F8 71 00 00 38 60 00 00 C8 70 00 00	....°q..8`..╚p..
00406F7C	00 00 00 00 00 00 00 00 2C 73 00 00 14 61 00 00	........,s..¶a..
00406F8C	BC 6F 00 00 00 00 00 00 00 00 00 00 E4 73 00 00	╝o..........õs..
00406F9C	08 60 00 00 00 00 00 00 00 00 00 00 00 00 00 00	◘`..............
00406FAC	00 00 00 00 00 00 00 00 01 00 00 80 00 00 00 00	........Ø..Ç....
00406FBC	D8 73 00 00 64 73 00 00 CE 73 00 00 C0 73 00 00	ïs..ds..╬s..└s..
00406FCC	AE 73 00 00 98 73 00 00 88 73 00 00 78 73 00 00	«s..ÿs..ês..xs..

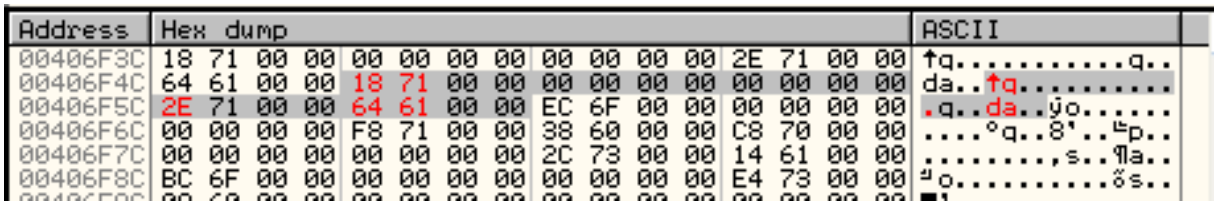
Имя первой DLL'ки находится в 40712E, посмотрим его:



Address	Hex dump	ASCII
0040712E	57 49 4E 4D 4D 2E 64 6C 6C 00 41 6E 74 69 44 65	WINMM.dll.AntiDe
0040713E	62 75 67 44 6C 6C 2E 64 6C 6C 00 00 EB 00 47 65	bugDll.dll..ù.Ge
0040714E	74 41 43 50 00 00 5D 01 47 65 74 4C 6F 63 61 6C	tACP..]0GetLocal
0040715E	65 49 6E 66 6F 41 00 00 C8 01 47 65 74 56 65 72	eInfoA..╚0GetVer
0040716E	73 69 6F 6E 45 78 41 00 51 02 4D 75 6C 74 69 42	sionExA.Q☻MultiB
0040717E	79 74 65 54 6F 57 69 64 65 43 68 61 72 00 69 03	yteToWideChar.i♥
0040718E	57 69 64 65 43 68 61 72 54 6F 4D 75 6C 74 69 42	WideCharToMultiB
0040719E	79 74 65 00 67 01 47 65 74 4D 6F 64 75 6C 65 48	yte.g0GetModuleH
004071AE	61 6E 64 6C 65 41 00 00 E7 00 46 72 65 65 52 65	andleA..ƥ.FreeRe

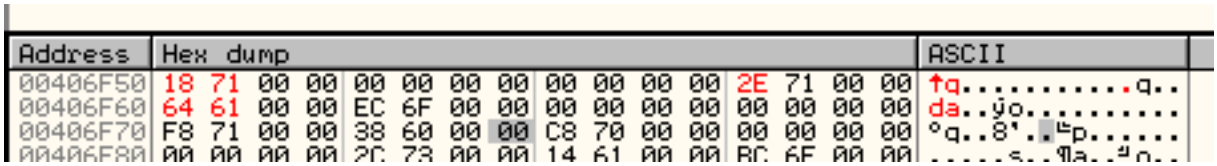
Это WINMM.DLL, а вторая — AntiDebugDll.dll. Таким образом, если скопировать первые 5 DWORD'ов таблицы импорта и вставить их в позицию следующих, то AntiDebugDll.dll аннулируется

и второй DLL’кой будет опять WINMM.DLL:



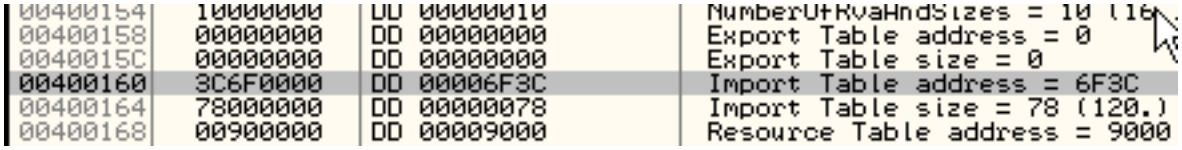
Address	Hex dump	ASCII
00406F3C	18 71 00 00 00 00 00 00 00 00 00 00 2E 71 00 00	↑q.....q..
00406F4C	64 61 00 00 18 71 00 00 00 00 00 00 00 00 00 00	da..↑q.....
00406F5C	2E 71 00 00 64 61 00 00 EC 6F 00 00 00 00 00 00	.q..da..ůo.....
00406F6C	00 00 00 00 F8 71 00 00 38 60 00 00 C8 70 00 00	....°q..8'..╚p..
00406F7C	00 00 00 00 00 00 00 00 2C 73 00 00 14 61 00 00	........,s..¶a..
00406F8C	BC 6F 00 00 00 00 00 00 00 00 00 00 E4 73 00 00	╝o..........ő s..

Осталось только сохранить изменения и установить новое начало IT — 406F50:



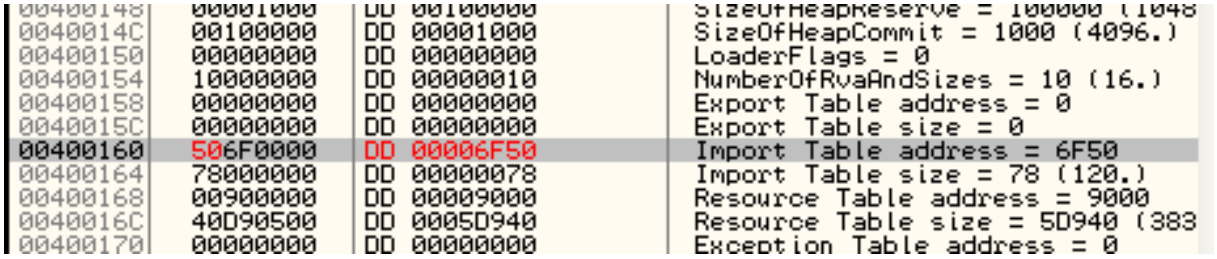
Address	Hex dump	ASCII
00406F50	18 71 00 00 00 00 00 00 00 00 00 00 2E 71 00 00	↑q.....q..
00406F60	64 61 00 00 EC 6F 00 00 00 00 00 00 00 00 00 00	da..ůo.....
00406F70	F8 71 00 00 38 60 00 00 C8 70 00 00 00 00 00 00	°q..8'..╚p.....

Сохранившись и перезагрузив крэкми, изменим адрес начала IT:



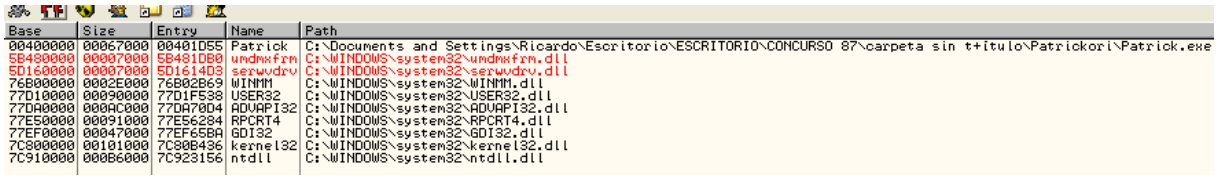
```
00400154   10000000   DD 00000010   NumberOfRvaAndSizes = 10 (16.
00400158   00000000   DD 00000000   Export Table address = 0
0040015C   00000000   DD 00000000   Export Table size = 0
00400160   3C6F0000   DD 00006F3C   Import Table address = 6F3C
00400164   78000000   DD 00000078   Import Table size = 78 (120.)
00400168   00900000   DD 00009000   Resource Table address = 9000
```

Теперь будет так:

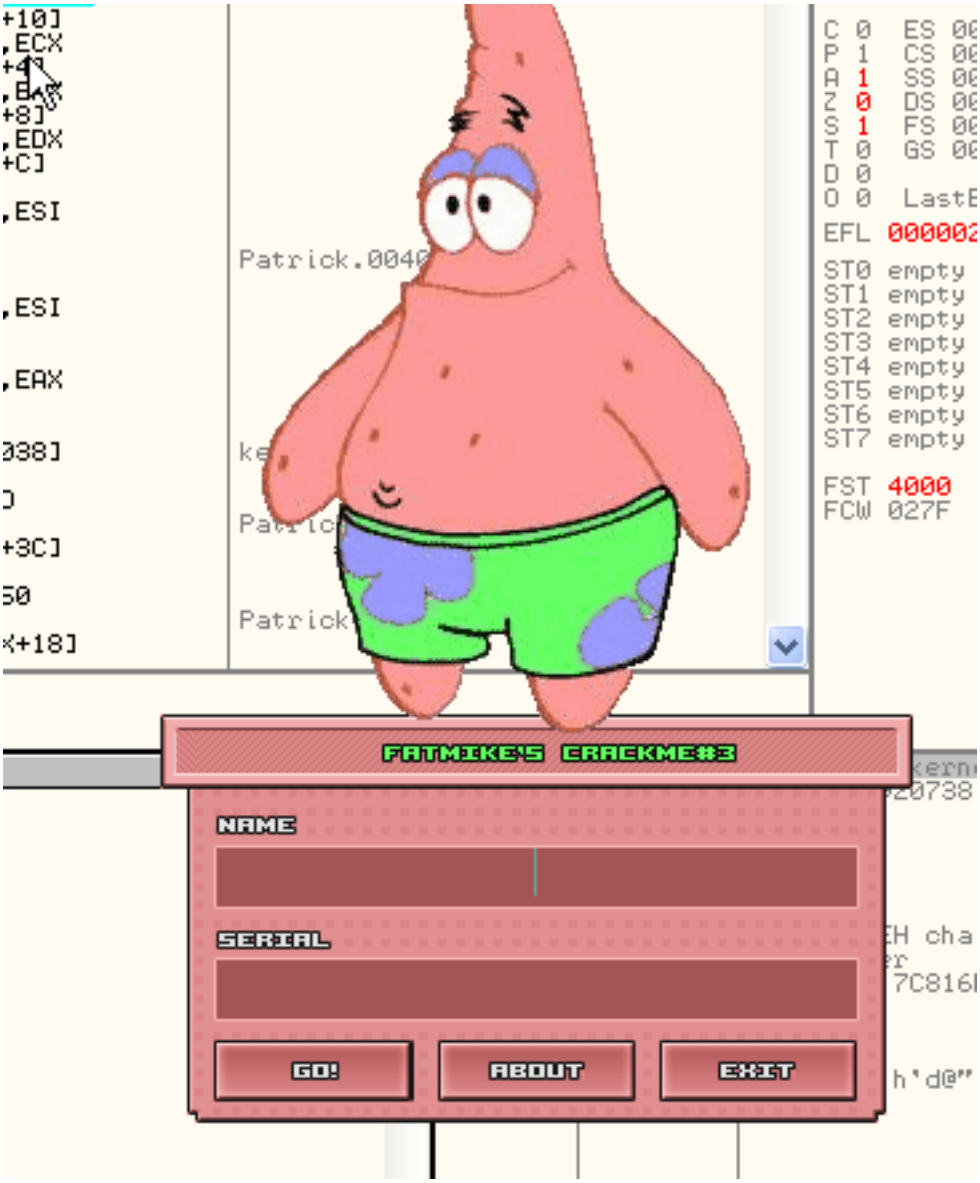


```
0040014C   00100000   DD 00001000   SizeOfHeapCommit = 1000 (4096.)
00400150   00000000   DD 00000000   LoaderFlags = 0
00400154   10000000   DD 00000010   NumberOfRvaAndSizes = 10 (16.)
00400158   00000000   DD 00000000   Export Table address = 0
0040015C   00000000   DD 00000000   Export Table size = 0
00400160   506F0000   DD 00006F50   Import Table address = 6F50
00400164   78000000   DD 00000078   Import Table size = 78 (120.)
00400168   00900000   DD 00009000   Resource Table address = 9000
0040016C   40D90500   DD 0005D940   Resource Table size = 5D940 (383
00400170   00000000   DD 00000000   Exception Table address = 0
```

После сохранения изменений и перезапуска крэкми противная DLL’ка не загружается и крэкми выполняется верно:



Base	Size	Entry	Name	Path
00400000	00067000	00401D55	Patrick	C:\Documents and Settings\Ricardo\Escritorio\ESCRITORIO\CONCURSO 87\carpeta sin t+itulo\Patrickori\Patrick.exe
5B480000	00007000	5B481DB0	wmdmxfrm	C:\WINDOWS\system32\wmdmxfrm.dll
5D160000	00007000	5D1614D3	serwvdrv	C:\WINDOWS\system32\serwvdrv.dll
76B00000	0002E000	76B02B69	WINMM	C:\WINDOWS\system32\WINMM.dll
77D10000	00090000	77D1F538	USER32	C:\WINDOWS\system32\USER32.dll
77DA0000	000AC000	77DA70D4	ADVAPI32	C:\WINDOWS\system32\ADVAPI32.dll
77E50000	00091000	77E56284	RPCRT4	C:\WINDOWS\system32\RPCRT4.dll
77EF0000	00047000	77EF65BA	GDI32	C:\WINDOWS\system32\GDI32.dll
7C800000	00101000	7C80B436	kernel32	C:\WINDOWS\system32\kernel32.dll
7C910000	000B6000	7C923156	ntdll	C:\WINDOWS\system32\ntdll.dll



Хе-хе, вот и финал внушающего страх Патрика!

Простое же решение состоит в изменении настройки доступа 40000040 секции данных файла Patrick.exe, что лишает его всех привилегий и нескольких строк ниже OEP, вызывающих ошибку при открытии в JIT-отладчике OllyDbg. В таком виде можно дойти чуть ниже OEP, дождаться распаковки и сделать то же самое: скопировать и вставить первую секцию в третий OllyDbg, изменить IT — и он заработает, хе-хе.

Прощай, ПАТРИК! хе-хе

Введение в крэкинг с нуля, используя OllyDbg - Глава 48

Автор: Рикардо Нарваха, пер. Рома Стремилов

Дата: 2010-04-26

Если вы загрузили главы 46 и 47 сразу после их появления, то, возможно, не читали добавленное к ним позднее примечание:

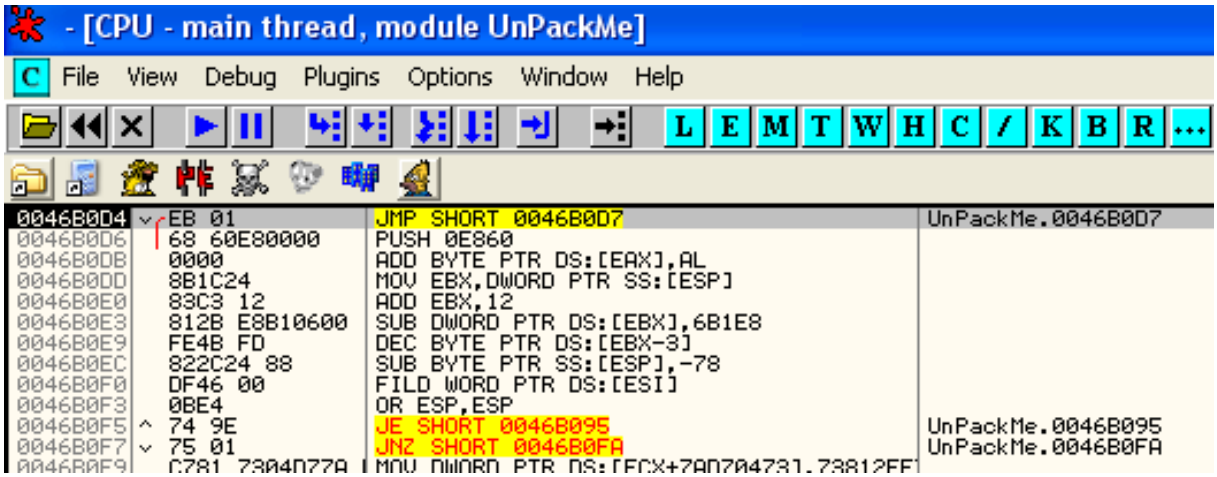
После того, как была написана 46 глава и еще не было завершено решение Patrick'а, я заметил, что эти туториалы слишком сложны для уровня, до которого мы дошли, так что если вам главы 46 и 47 покажутся очень сложными, то советую оставить их до тех времен, когда вы будете более подготовлены, а сейчас сразу перейти к 48 главе, соответствующей тому уровню, на котором мы остановились.

Рикардо Нарваха

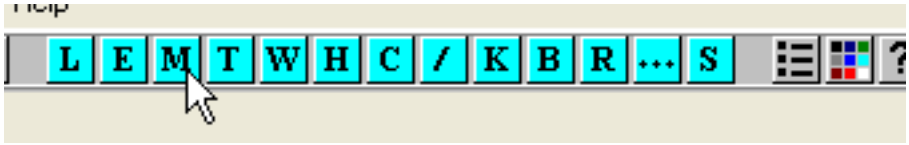
Так уж вышло, что когда я занялся Patrick'ом, то сначала решил его простым методом (упомянутым в конце 47-й главы) и посчитал, что он легок до неприличия. Как следствие, данные главы оказались труднее прежних и пришлось добавить процитированное выше примечание.

В этой главе мы рассмотрим упаковщик PeSpin 1.304 Full, о котором уже существуют достаточно хорошие туториалы. На самом деле, написать еще не существующий туториал практически невозможно, так как ресурс CracksLatinoS содержит очень хорошие статьи почти обо всех упаковщиках.

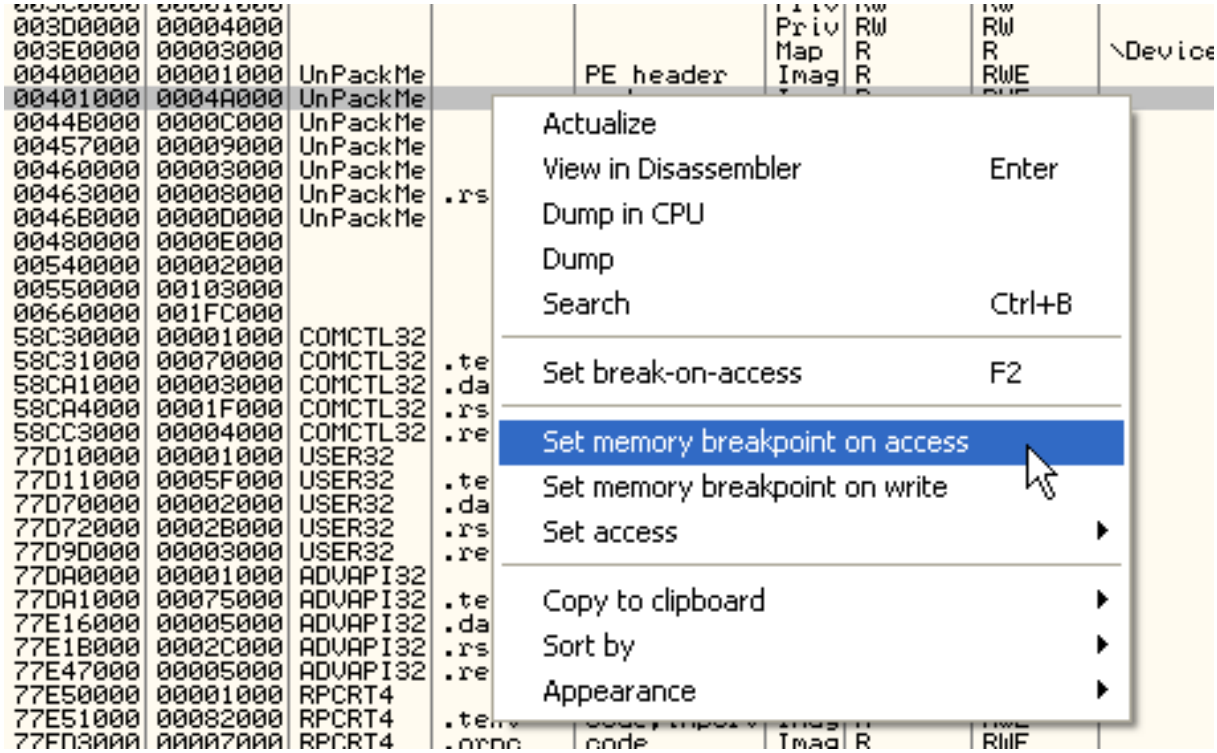
В PeSpin дойти до OEP очень просто.



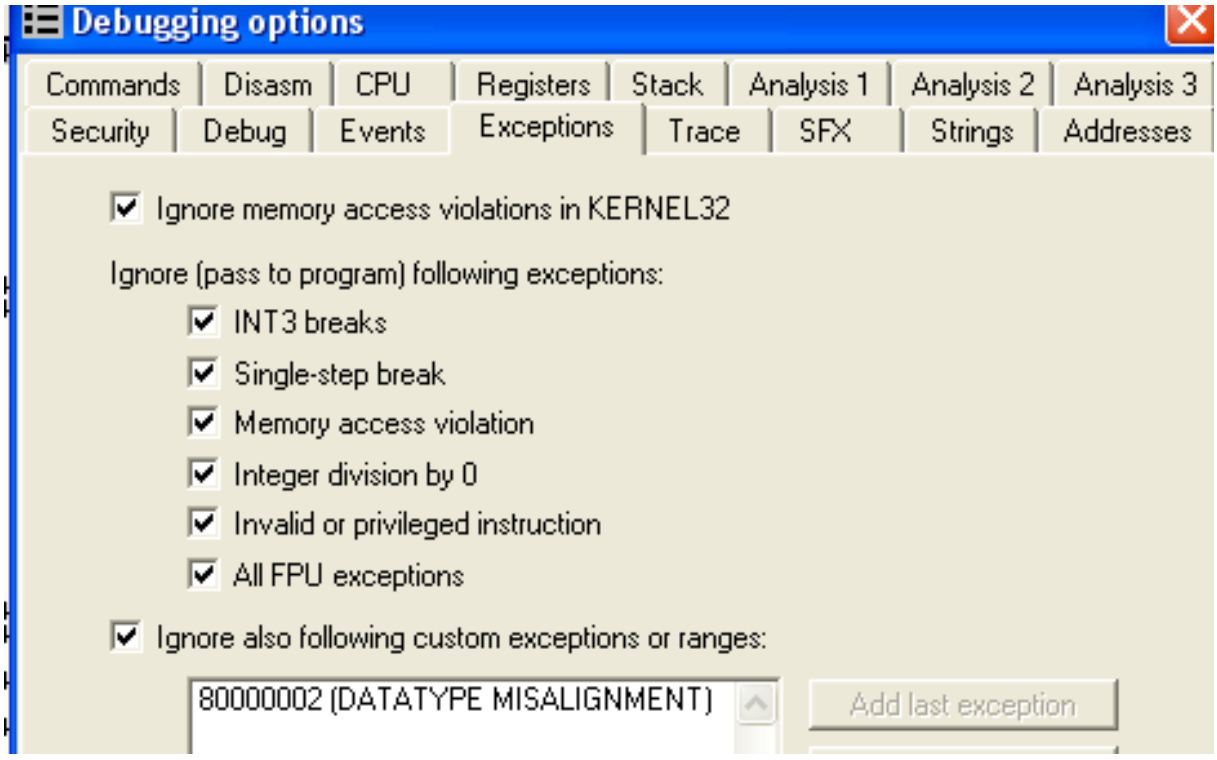
Сейчас анпэкми остановлен на его EP. Мы будем пользоваться Parcheado 5 — версией OllyDbg, которая предназначена для поиска OEP'ов.



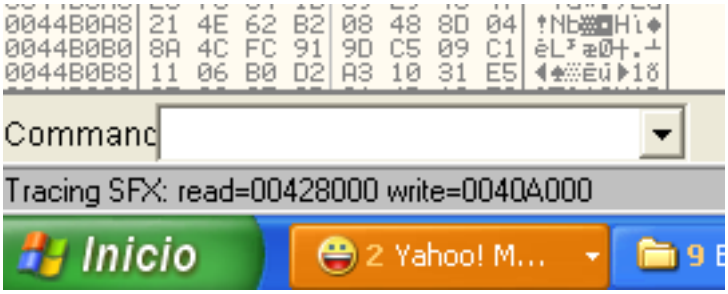
Открыв карту памяти, установим MEMORY BREAKPOINT ON ACCESS в первой секции после заголовка. Этот брейк равнозначен ON EXECUTION, поскольку, как помните, пропатченная для поиска OEP'ов версия отладчика в данном случае останавливается только при выполнении, а не при чтении или записи.



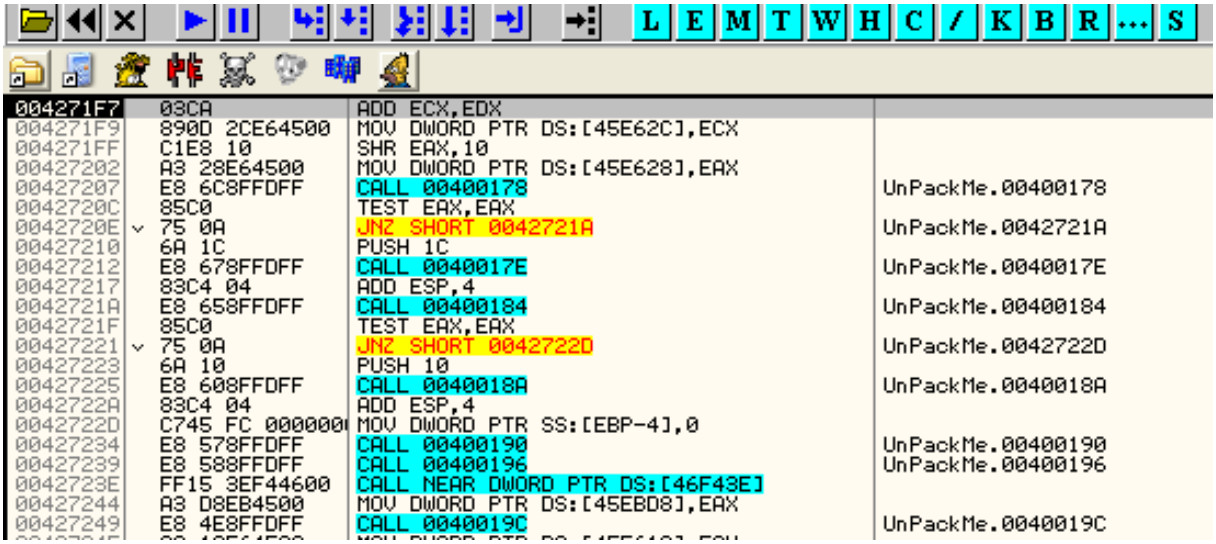
Следует убедиться, что все галки во вкладке Exceptions окна Debugging options установлены:



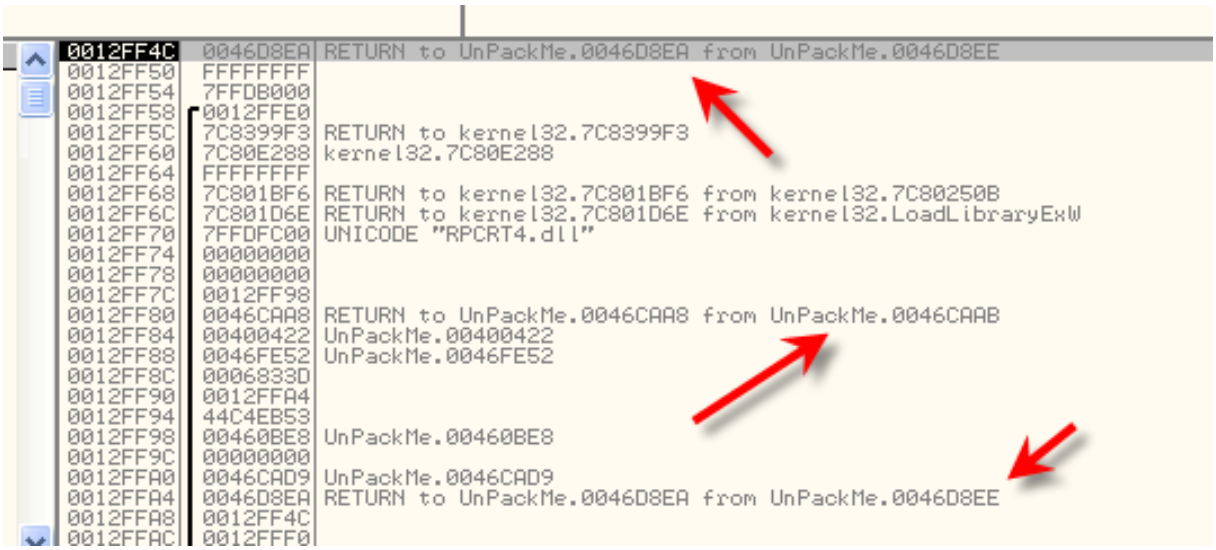
После нажатия на RUN можно пойти спокойно пить кофе:



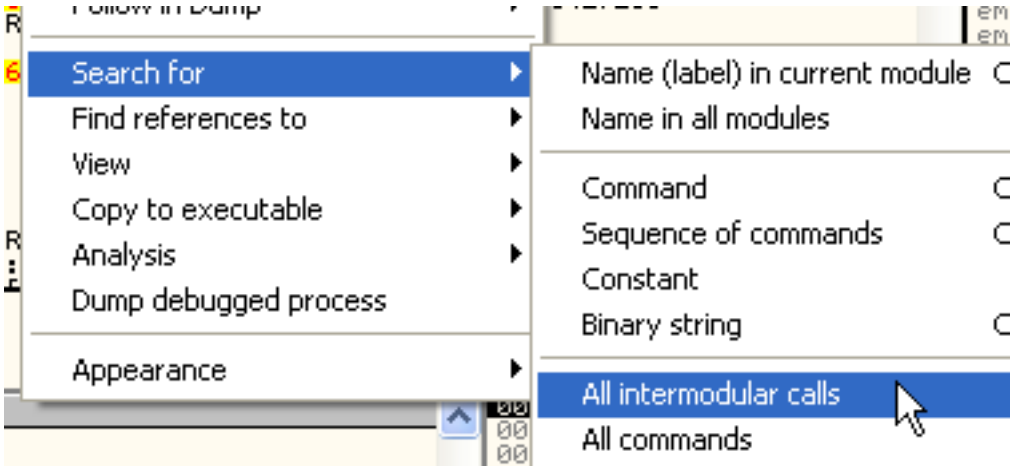
Как следует напившись кофе, хе-хе, обнаружим, что остановка произошла на непохожем на OEP месте, а значит, от команд исходной OEP ничего не осталось из-за украденных байтов:



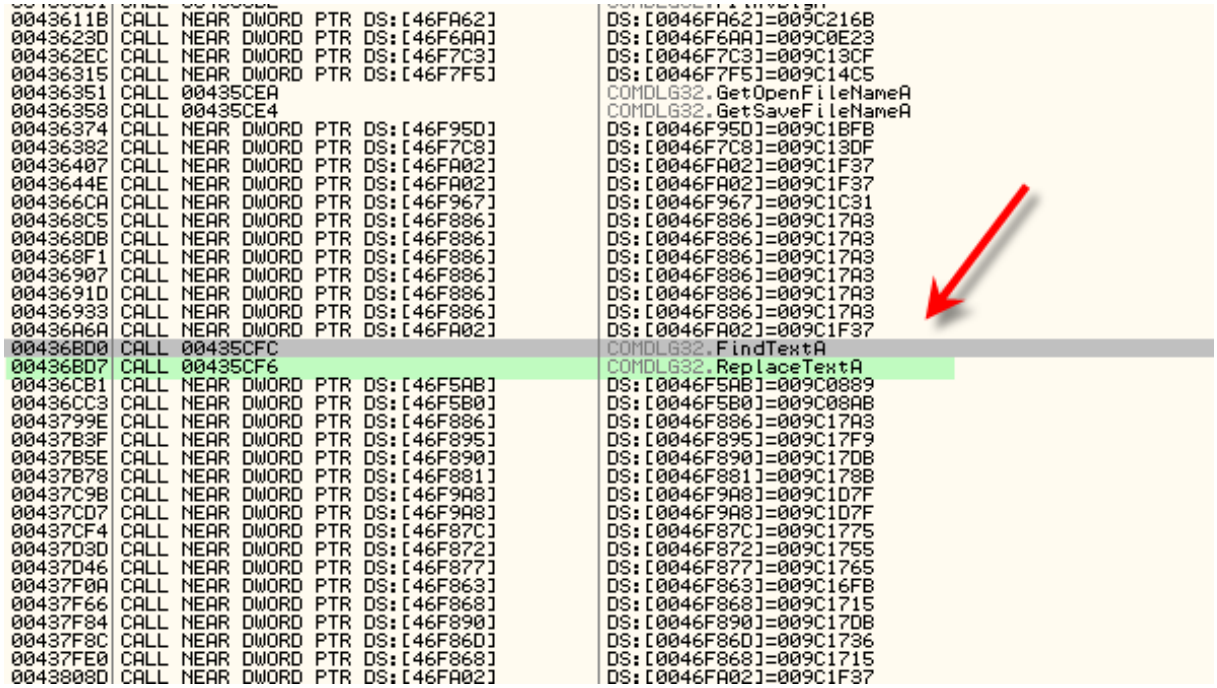
Посмотрим стек:



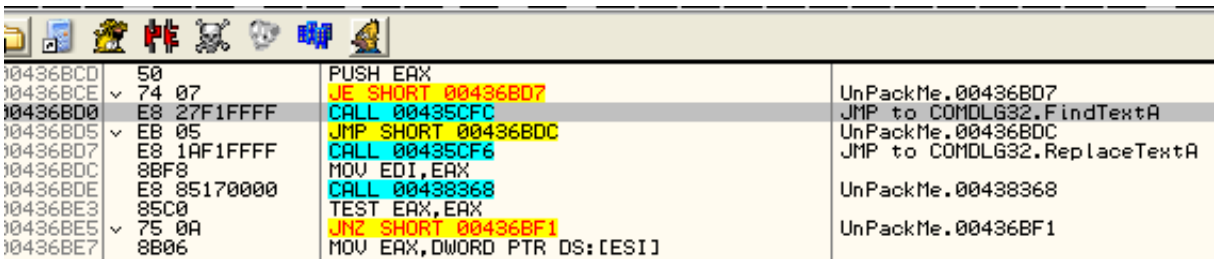
Можно заметить, что перед прибытием в ложную OEP было выполнено много кода, и это свидетельствует о том, что байты OEP были украдены.



Кроме того, если сделаем Search for –> All intermodular calls, то найдем очень мало вызовов API-функций:

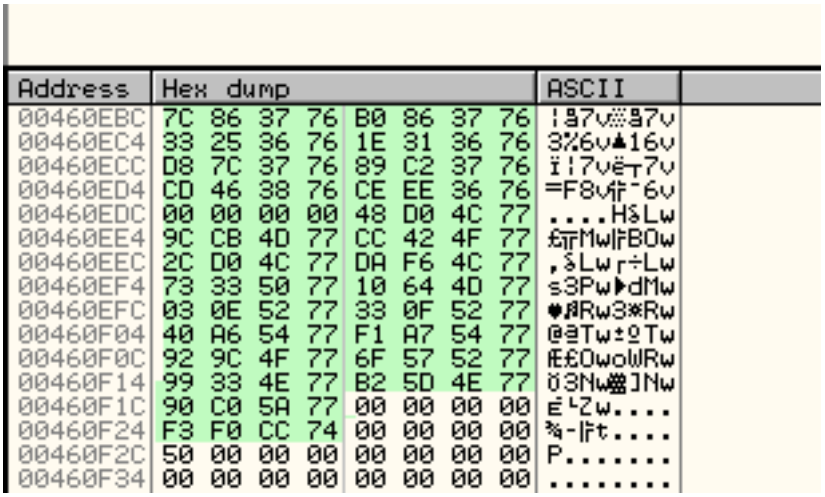


Посмотрим один из них:



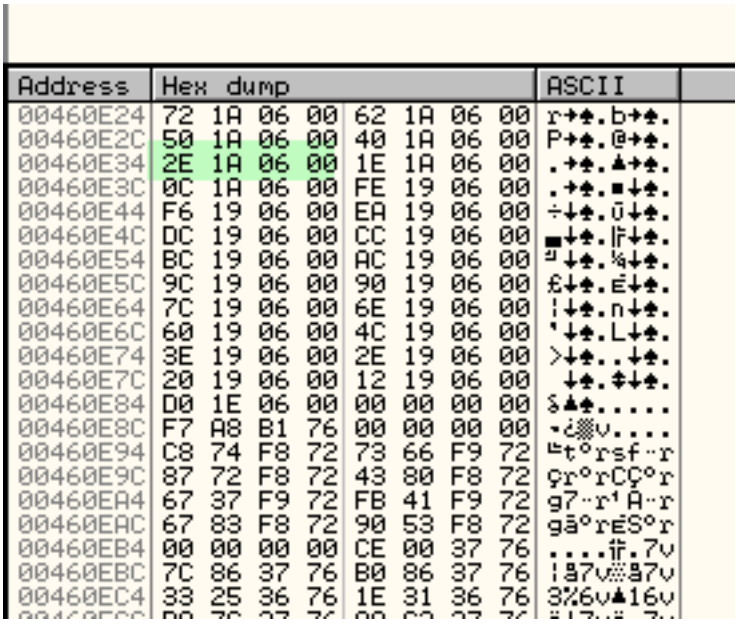
```
00435CBF   90              NOP
00435CC0 - FF25 43F44600   JMP NEAR DWORD PTR DS:[46F443]
00435CC6 - FF25 ECF64600   JMP NEAR DWORD PTR DS:[46F6EC]
00435CCC - FF25 F1F64600   JMP NEAR DWORD PTR DS:[46F6F1]
00435CD2 - FF25 D00E4600   JMP NEAR DWORD PTR DS:[460ED0]   COMDLG32.ChooseFontA
00435CD8 - FF25 D80E4600   JMP NEAR DWORD PTR DS:[460ED8]   COMDLG32.ChooseColorA
00435CDE - FF25 D40E4600   JMP NEAR DWORD PTR DS:[460ED4]   COMDLG32.PrintDlgA
00435CE4 - FF25 CC0E4600   JMP NEAR DWORD PTR DS:[460ECC]   COMDLG32.GetSaveFileNameA
00435CEA - FF25 C80E4600   JMP NEAR DWORD PTR DS:[460EC8]   COMDLG32.GetOpenFileNameA
00435CF0 - FF25 C40E4600   JMP NEAR DWORD PTR DS:[460EC4]   COMDLG32.GetFileTitleA
00435CF6 - FF25 C00E4600   JMP NEAR DWORD PTR DS:[460EC0]   COMDLG32.ReplaceTextA
00435CFC - FF25 BC0E4600   JMP NEAR DWORD PTR DS:[460EBC]   COMDLG32.FindTextA
00435D02 - FF25 B80E4600   JMP NEAR DWORD PTR DS:[460EB8]   COMDLG32.CommDlgExtendedError
00435D08 - FF25 B00E4600   JMP NEAR DWORD PTR DS:[460EB0]   WINSPOOL.ClosePrinter
00435D0E - FF25 940E4600   JMP NEAR DWORD PTR DS:[460E94]   WINSPOOL.EndDocPrinter
00435D14 - FF25 AC0E4600   JMP NEAR DWORD PTR DS:[460EAC]   WINSPOOL.StartPagePrinter
00435D1A - FF25 A80E4600   JMP NEAR DWORD PTR DS:[460EA8]   WINSPOOL.StartDocPrinterA
00435D20 - FF25 A40E4600   JMP NEAR DWORD PTR DS:[460EA4]   WINSPOOL.OpenPrinterA
00435D26 - FF25 A00E4600   JMP NEAR DWORD PTR DS:[460EA0]   WINSPOOL.EndPagePrinter
00435D2C - FF25 9C0E4600   JMP NEAR DWORD PTR DS:[460E9C]   WINSPOOL.WritePrinter
00435D32 - FF25 980E4600   JMP NEAR DWORD PTR DS:[460E98]   WINSPOOL.DocumentPropertiesA
00435D38 - FF25 240F4600   JMP NEAR DWORD PTR DS:[460F24]   OLEDLG.OleUIBusyA
00435D3E   CC              INT3
```

Дойдя до косвенных переходов на API-функции, заглянем в IAT:



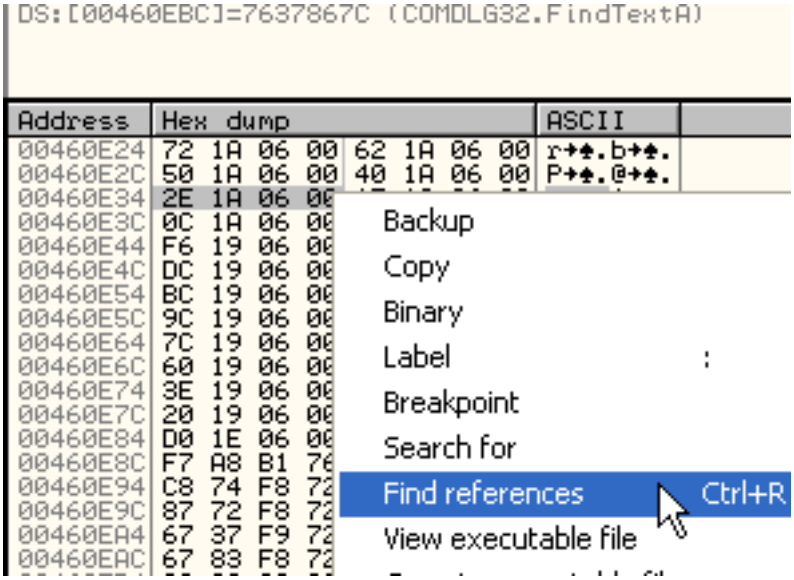
Address	Hex dump	ASCII
00460EBC	7C 86 37 76 B0 86 37 76	|å7vë7v
00460EC4	33 25 36 76 1E 31 36 76	3%6v▲16v
00460ECC	D8 7C 37 76 89 C2 37 76	ï|7vëT7v
00460ED4	CD 46 38 76 CE EE 36 76	=F8v╬ⁿ6v
00460EDC	00 00 00 00 48 D0 4C 77	....HŠLw
00460EE4	9C CB 4D 77 CC 42 4F 77	£╦Mw╠BOw
00460EEC	2C D0 4C 77 DA F6 4C 77	,ŠLwr÷Lw
00460EF4	73 33 50 77 10 64 4D 77	s3Pw►dMw
00460EFC	03 0E 52 77 33 0F 52 77	♥♫Rw3☼Rw
00460F04	40 A6 54 77 F1 A7 54 77	@ªTw±ºTw
00460F0C	92 9C 4F 77 6F 57 52 77	Æ£OwoWRw
00460F14	99 33 4E 77 B2 5D 4E 77	Ö3Nw]Nw
00460F1C	90 C0 5A 77 00 00 00 00	É└Zw....
00460F24	F3 F0 CC 74 00 00 00 00	¾-╠t....
00460F2C	50 00 00 00 00 00 00 00	P.......
00460F34	00 00 00 00 00 00 00 00	....

Здесь виден конец IAT'а — 460F28. Теперь поднимемся выше:

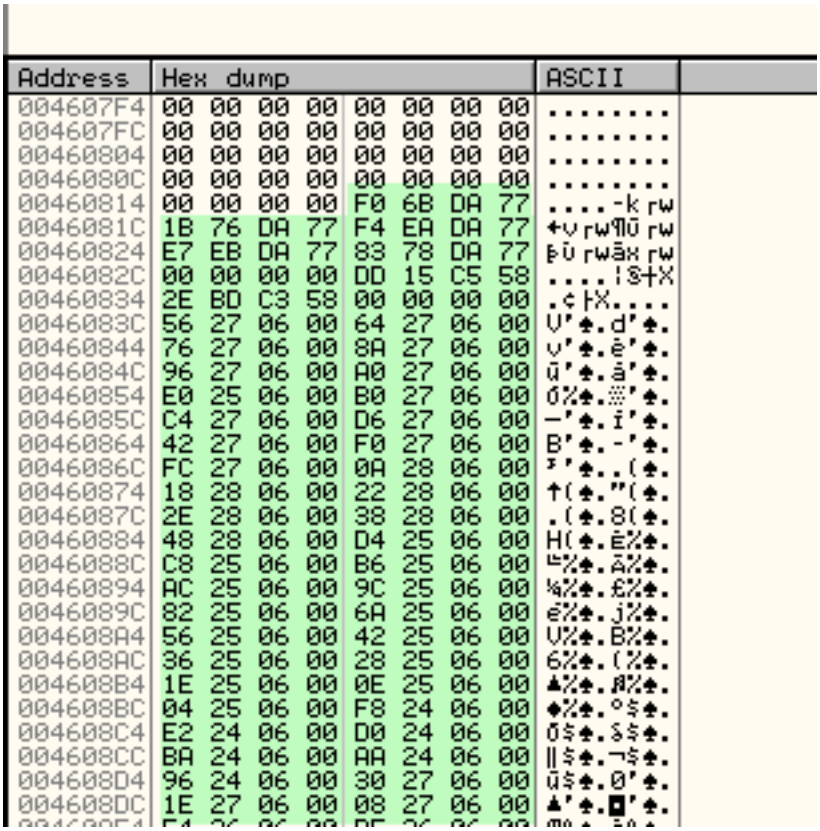


Address	Hex dump	ASCII
00460E24	72 1A 06 00 62 1A 06 00	r→♠.b→♠.
00460E2C	50 1A 06 00 40 1A 06 00	P→♠.@→♠.
00460E34	2E 1A 06 00 1E 1A 06 00	.→♠.▲→♠.
00460E3C	0C 1A 06 00 FE 19 06 00	.→♠.■↓♠.
00460E44	F6 19 06 00 EA 19 06 00	÷↓♠.Ω↓♠.
00460E4C	DC 19 06 00 CC 19 06 00	▄↓♠.╠↓♠.
00460E54	BC 19 06 00 AC 19 06 00	╝↓♠.¼↓♠.
00460E5C	9C 19 06 00 90 19 06 00	£↓♠.É↓♠.
00460E64	7C 19 06 00 6E 19 06 00	|↓♠.n↓♠.
00460E6C	60 19 06 00 4C 19 06 00	`↓♠.L↓♠.
00460E74	3E 19 06 00 2E 19 06 00	>↓♠..↓♠.
00460E7C	20 19 06 00 12 19 06 00	↓♠.‼↓♠.
00460E84	D0 1E 06 00 00 00 00 00	Š▲♠.....
00460E8C	F7 A8 B1 76 00 00 00 00	▪¿▒v....
00460E94	C8 74 F8 72 73 66 F9 72	╚t°rsf·r
00460E9C	87 72 F8 72 43 80 F8 72	çr°rCÇ°r
00460EA4	67 37 F9 72 FB 41 F9 72	g7·r¹Ä·r
00460EAC	67 83 F8 72 90 53 F8 72	gâ°rÉS°r
00460EB4	00 00 00 00 CE 00 37 76	....╬.7v
00460EBC	7C 86 37 76 B0 86 37 76	|å7v▒a7v
00460EC4	33 25 36 76 1E 31 36 76	3%6v▲16v

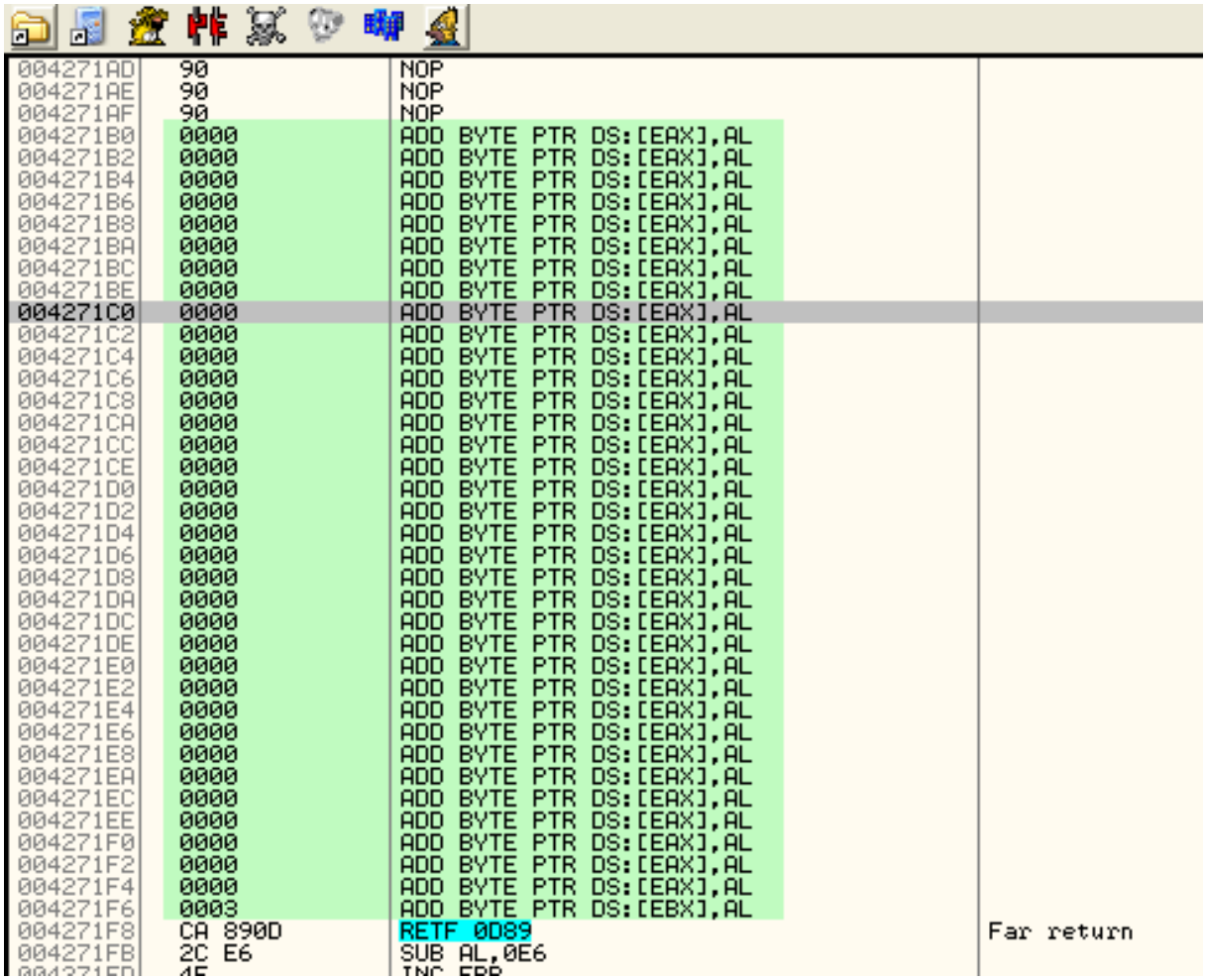
Похоже, это переадресовочные элементы. Чтобы проверить, принадлежат ли они IAT'у, посмотрим их референсы:



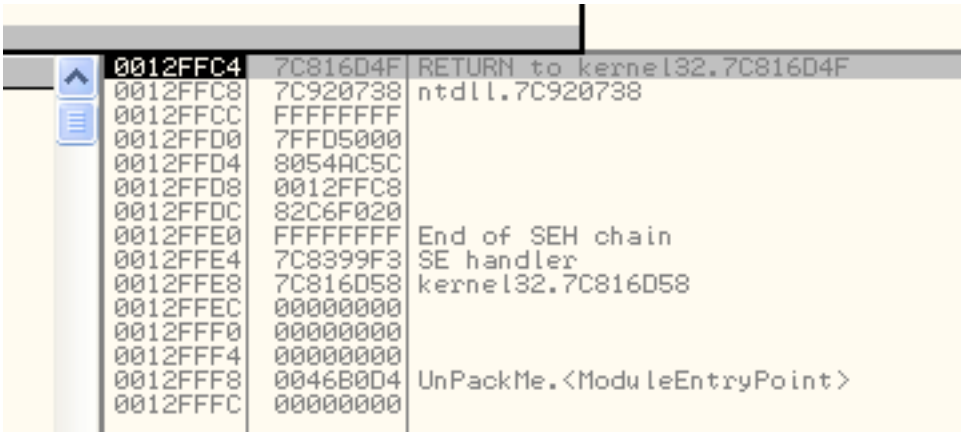
Поиск ничего не дает, но если подняться еще выше, то станет ясно, что это всё-таки часть IAT'а. Таким образом, в PeSpin используется и переадресация.



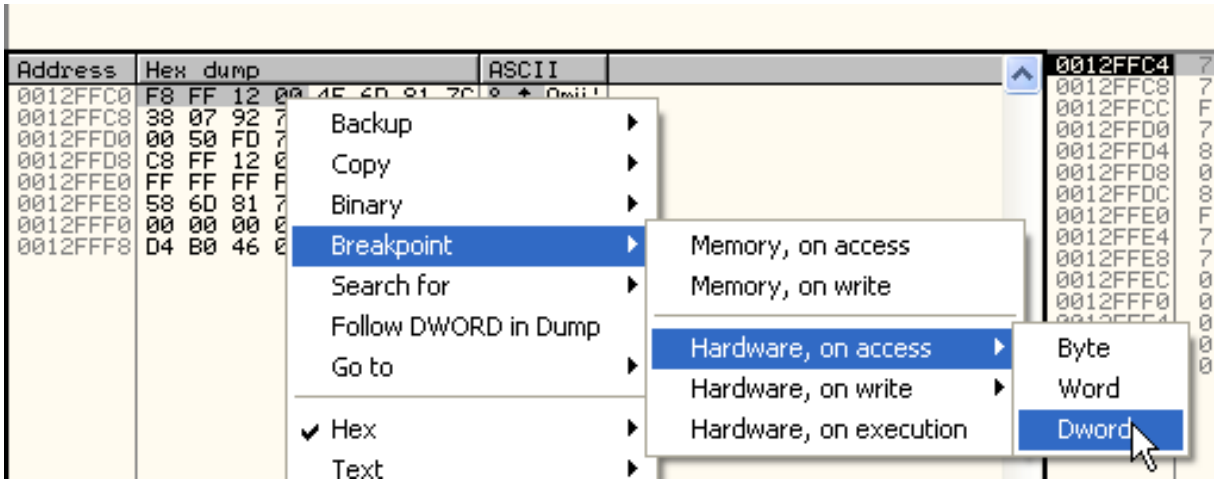
Начало IAT'а находится по адресу 460818. Мы еще вернемся к этому месту, когда будем исправлять IAT, а сейчас займемся возвращением украденных байтов; чуть выше ложной OEP для них как раз есть подходящая нулевая область:



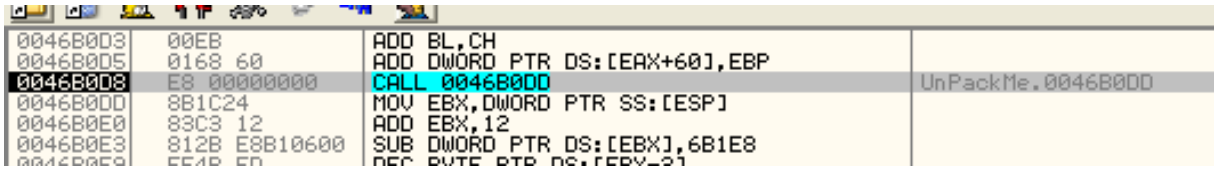
Перезагрузим программу и посмотрим состояние стека:



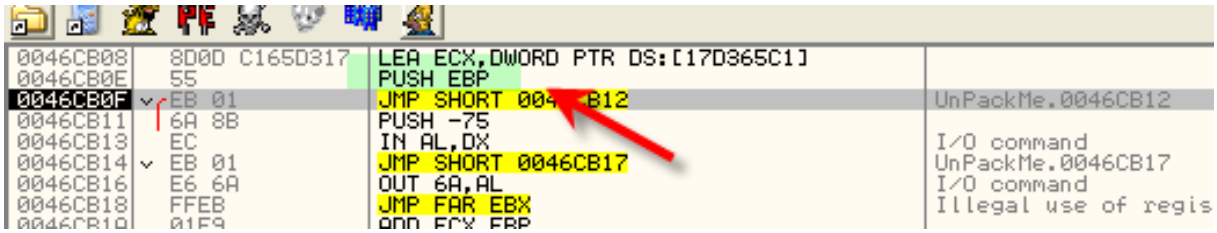
Здесь видно, что перед запуском анпэкми адрес вершины стека равен 12FFC4 (так на моем компьютере). Это означает, что при прибытии к истинной OEP вершина стека должна находиться по тому же адресу, то есть в 12FFC4. Обычно первой командой программы является PUSH EBP, которая записывает значение в следующую ячейку стека (12FFC0), поэтому найдем ее в дампе и установим на ней HARDWARE BPX ON WRITE. Такое рассуждение логично, но оно может и не дать результатов, если упаковщик обнаруживает аппаратные брейкпоинты или, для осложнения поиска OEP, меняет адреса стека.



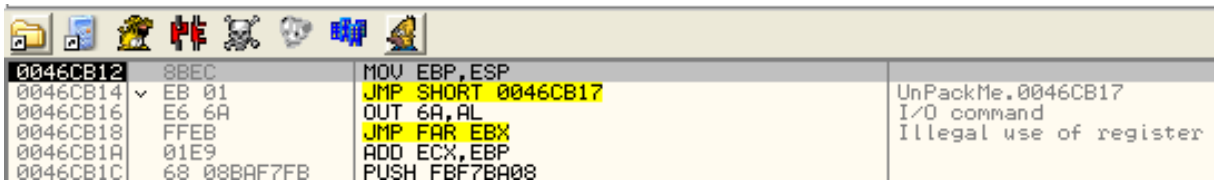
После установки брейка нажмем RUN:

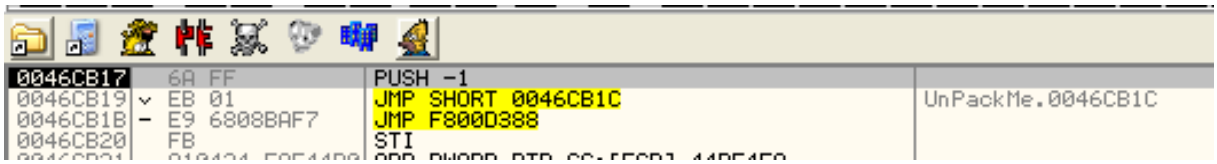


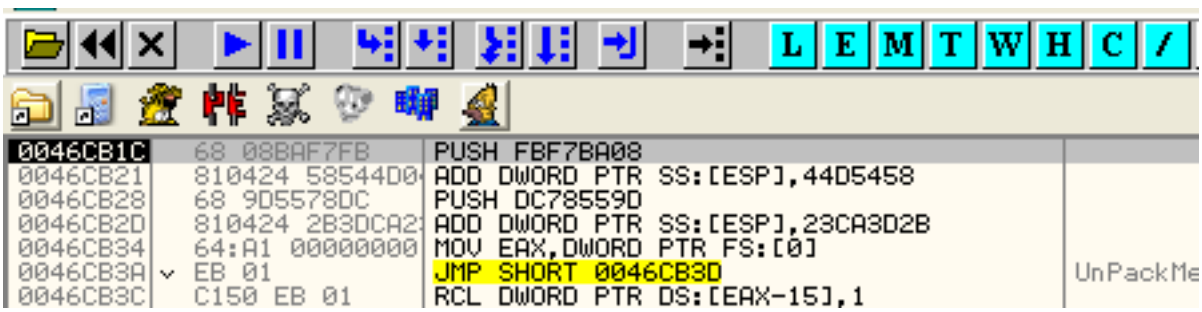
Первая остановка произошла здесь, но данная инструкция скорее всего принадлежит распаковщику. Нажмем F9 еще раз:



Теперь остановились на PUSH EBP, что весьма похоже на команду из OEP. Чтобы проверить это предположение, воспользуемся, хе-хе, трассировкой. Конечно же, JMP'ы нас не интересуют, так как они не влияют на состояние регистров или стека.

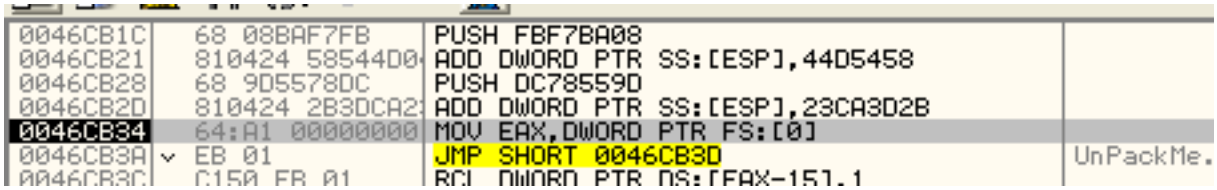




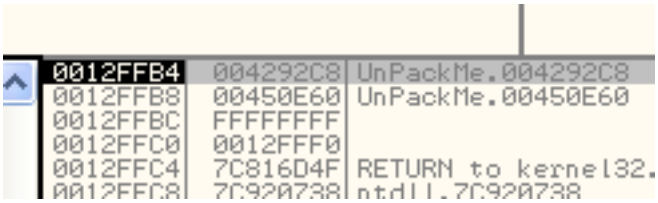


Здесь встречается необычная комбинация команд: сначала выполняется PUSH, а затем только что записанное в стек значение суммируется с константой:

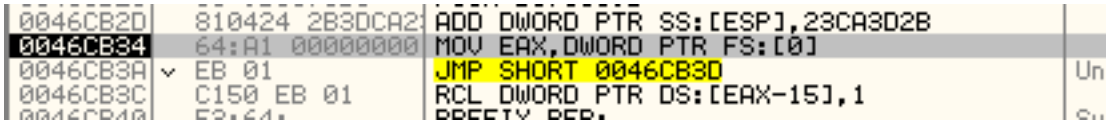
После выполнения инструкции ADD значение в стеке оказывается 450E60, поэтому данная комбинация равнозначна PUSH 450E60.



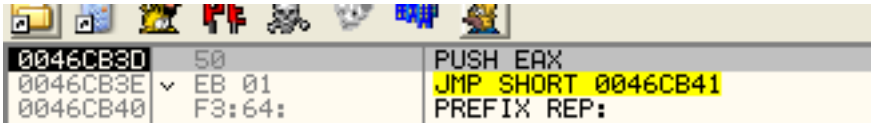
Затем этот трюк повторяется, но уже вместо PUSH 4292C8:



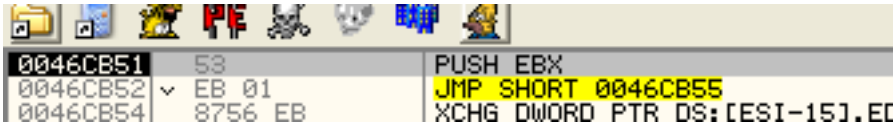
Далее идет еще одна подходящая инструкция:

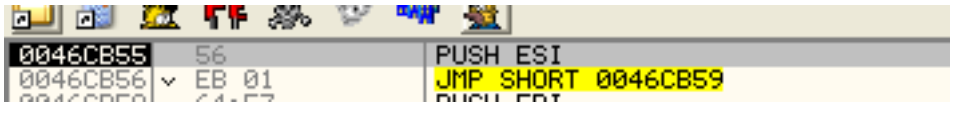


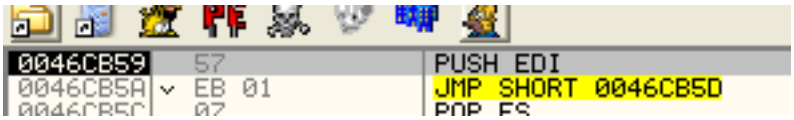
И еще парочка:

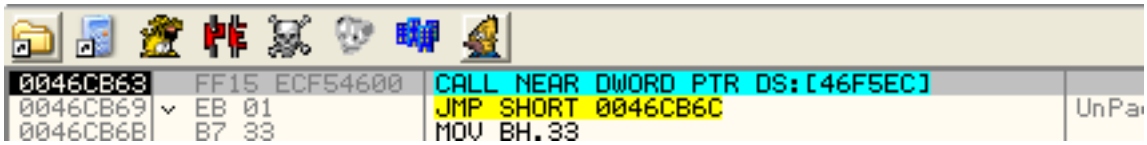


Продолжим:

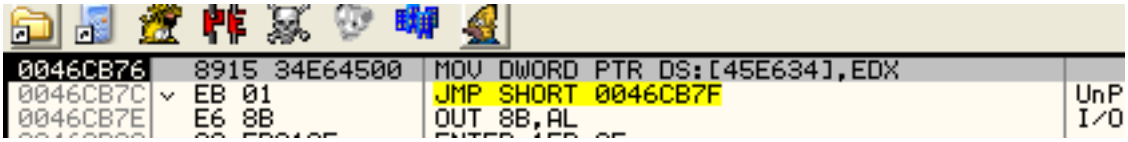


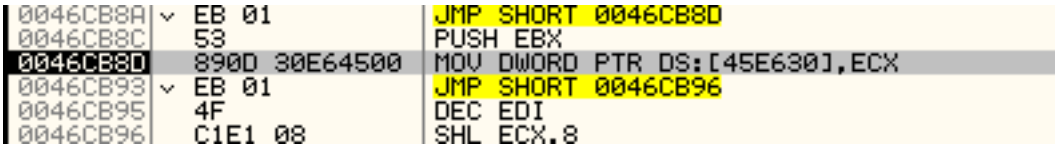


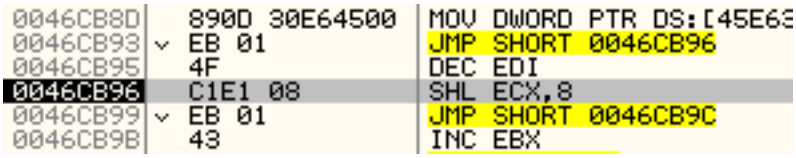


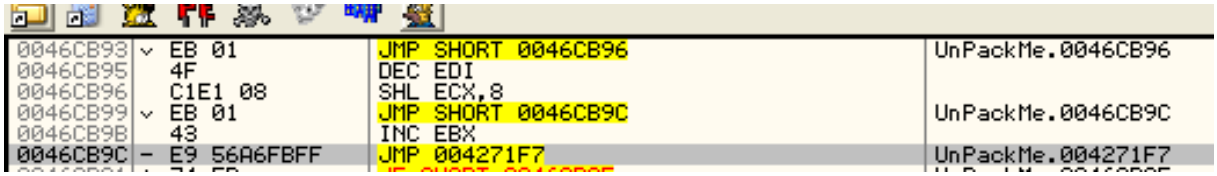


Сейчас мы просто скопируем этот CALL, а потом, при восстановлении IAT'а, посмотрим, относится ли он к какой-нибудь API-функции.

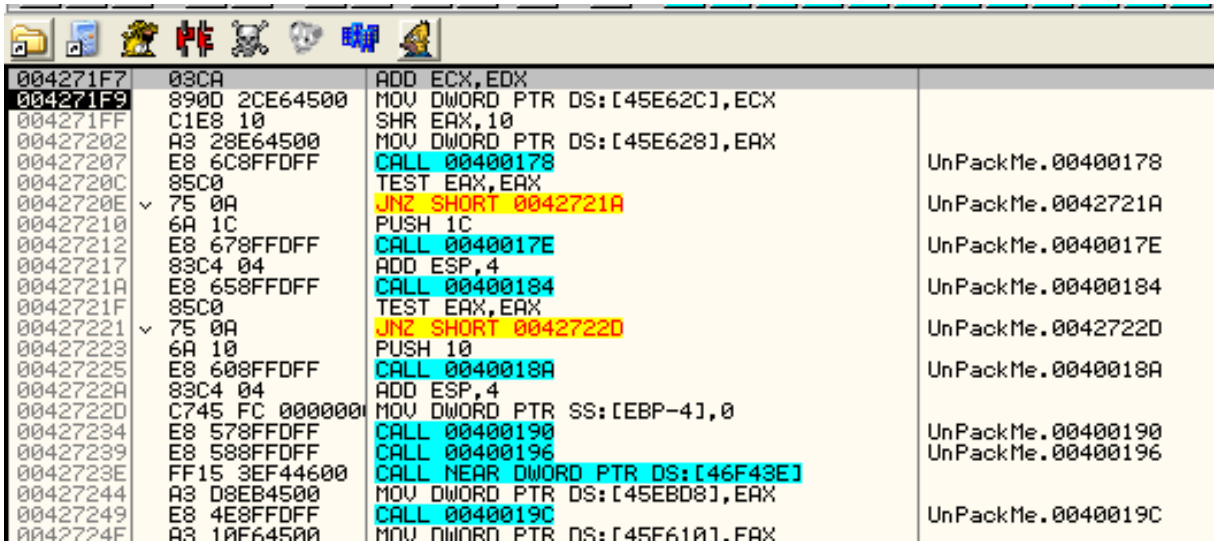




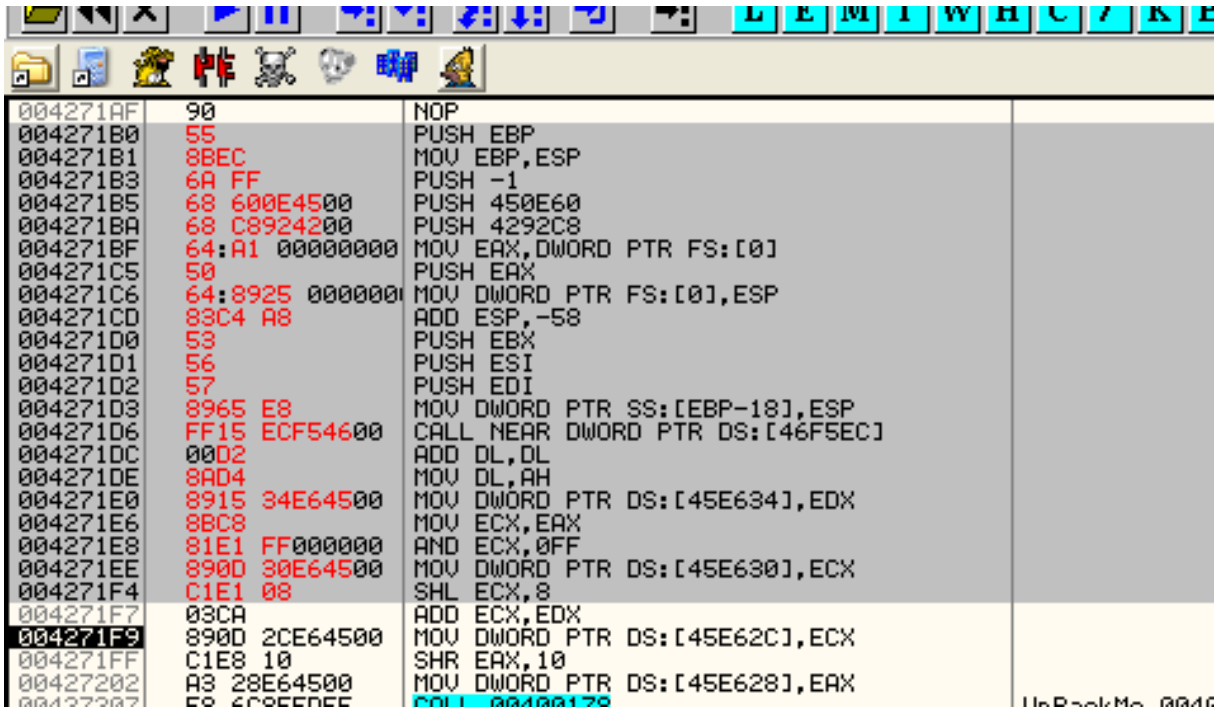




Переход на ложную OEP завершает трассировку, а мы тем временем узнали список украденных байтов, хе-хе.



Скопируем их в область OEP:



Они байт-в-байт заполнили нулевую область, так что теперь украденные байты возвращены, хотя мы даже не приступали к сдампливанию. В следующей главе мы будем разбираться с IAT'ом.

До встречи в 49-й главе!

Файлы к статье (находится в архиве `wasm_files.zip`: `files/0523/ollydbg48.zip`)

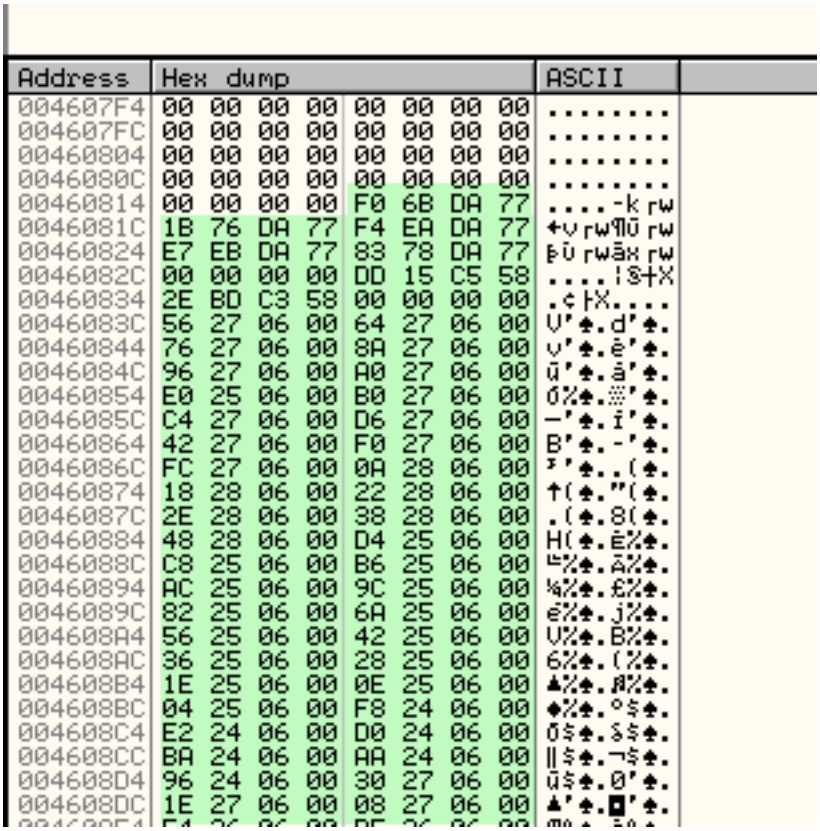
Введение в крэкинг с нуля, используя OllyDbg - Глава 49

Автор: Рикардо Нарваха, пер. Рома Стремилов

Дата: 2010-04-29

Теперь займемся восстановлением IAT PeSpin, что будет не так уж и просто.

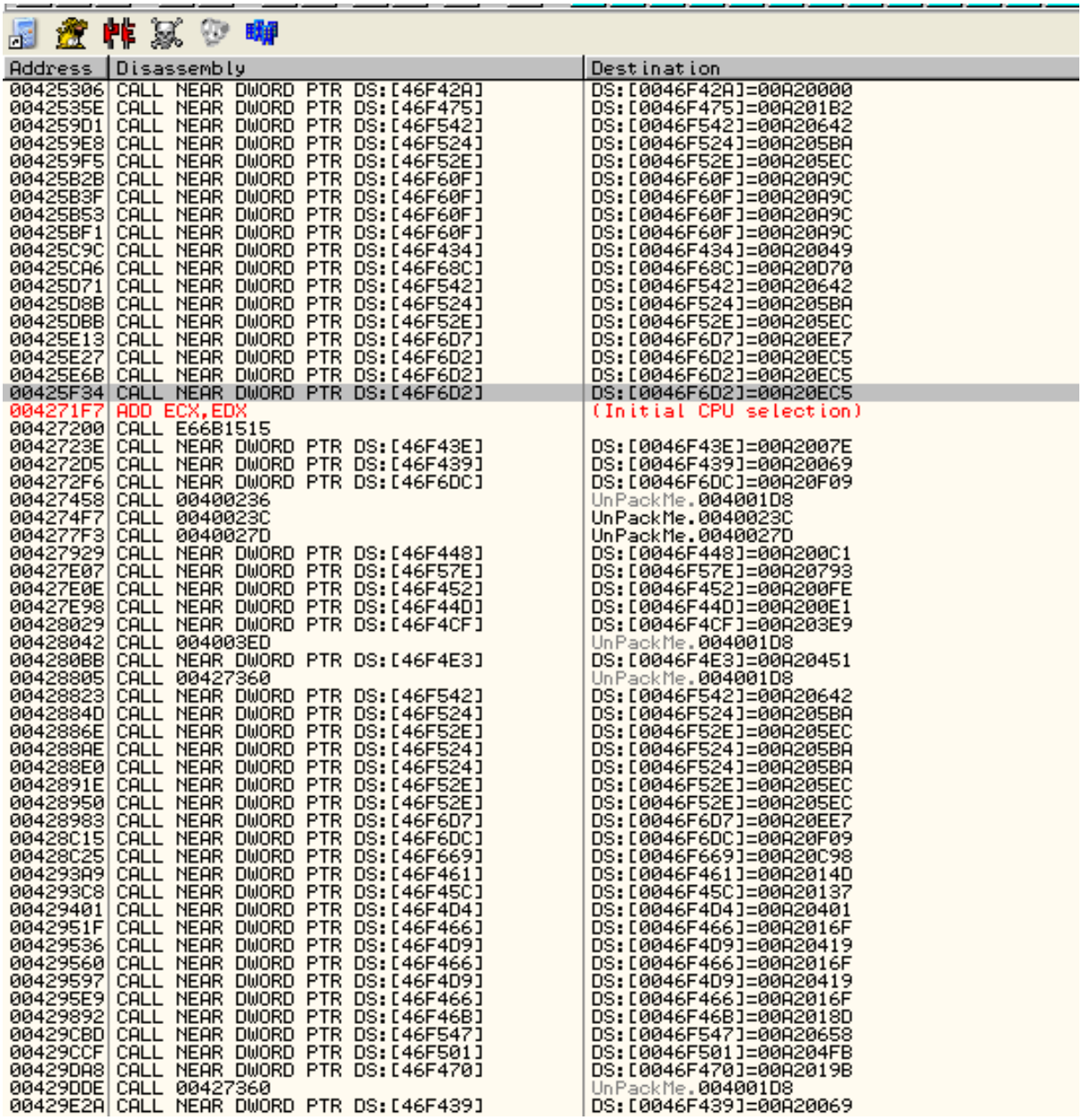
Возможно, вы обратили внимание, что когда выполнение доходит до предполагаемой OEP, API-функции вызываются не через IAT, начало которой в 460818 и окончание в 460F28:



Address	Hex dump	ASCII
004607F4	00 00 00 00 00 00 00 00	....
004607FC	00 00 00 00 00 00 00 00	....
00460804	00 00 00 00 00 00 00 00	....
0046080C	00 00 00 00 00 00 00 00	....
00460814	00 00 00 00 F0 6B DA 77	....-k rw
0046081C	1B 76 DA 77 F4 EA DA 77	←v rw¶Ω rw
00460824	E7 EB DA 77 83 78 DA 77	þΩ rwâx rw
0046082C	00 00 00 00 DD 15 C5 58	....¦§┼X
00460834	2E BD C3 58 00 00 00 00	.¢├X....
0046083C	56 27 06 00 64 27 06 00	V'♠.d'♠.
00460844	76 27 06 00 8A 27 06 00	v'♠.è'♠.
0046084C	96 27 06 00 A0 27 06 00	ü'♠.á'♠.
00460854	E0 25 06 00 B0 27 06 00	ó%♠.'♠.
0046085C	C4 27 06 00 D6 27 06 00	─'♠.í'♠.
00460864	42 27 06 00 F0 27 06 00	B'♠.-'♠.
0046086C	FC 27 06 00 0A 28 06 00	³'♠..(♠.
00460874	18 28 06 00 22 28 06 00	↑(♠."(♠.
0046087C	2E 28 06 00 38 28 06 00	.(♠.8(♠.
00460884	48 28 06 00 D4 25 06 00	H(♠.Ë%♠.
0046088C	C8 25 06 00 B6 25 06 00	╚%♠.Ã%♠.
00460894	AC 25 06 00 9C 25 06 00	¼%♠.£%♠.
0046089C	82 25 06 00 6A 25 06 00	é%♠.j%♠.
004608A4	56 25 06 00 42 25 06 00	V%♠.B%♠.
004608AC	36 25 06 00 28 25 06 00	6%♠.(%♠.
004608B4	1E 25 06 00 0E 25 06 00	▲%♠.♫%♠.
004608BC	04 25 06 00 F8 24 06 00	♦%♠.°$♠.
004608C4	E2 24 06 00 D0 24 06 00	ô$♠.§$♠.
004608CC	BA 24 06 00 AA 24 06 00	‖$♠.¬$♠.
004608D4	96 24 06 00 30 27 06 00	ü$♠.0'♠.
004608DC	1E 27 06 00 08 27 06 00	▲'♠.◘'♠.

Назовем приведенную выше таблицу ВЕРНАЯ IAT программы. Однако есть такие элементы IAT, которые не являются ссылками, не направляют ни к одной API-функции и содержат какие-то странные значения, не принадлежащие ни одной DLL’ке и никуда не переадресующие — они похожи на какой-то мусор.

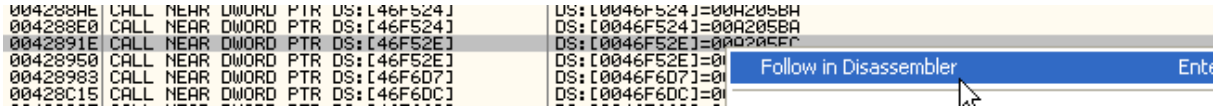
Сделаем Search for —> All intermodular calls:



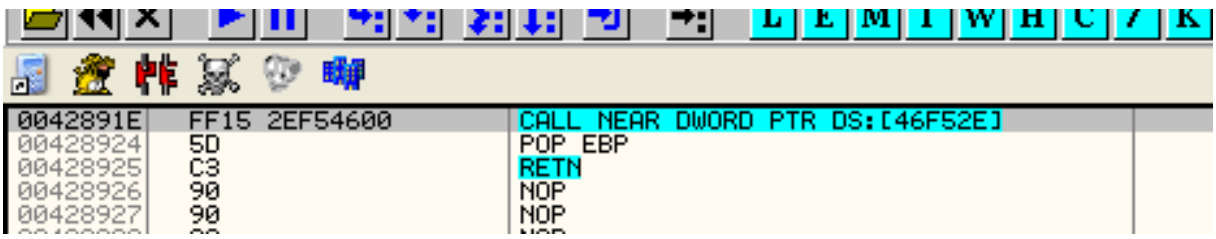
Address	Disassembly	Destination
00425306	CALL NEAR DWORD PTR DS:[46F42A]	DS:[0046F42A]=00A20000
0042535E	CALL NEAR DWORD PTR DS:[46F475]	DS:[0046F475]=00A201B2
004259D1	CALL NEAR DWORD PTR DS:[46F542]	DS:[0046F542]=00A20642
004259E8	CALL NEAR DWORD PTR DS:[46F524]	DS:[0046F524]=00A205BA
004259F5	CALL NEAR DWORD PTR DS:[46F52E]	DS:[0046F52E]=00A205EC
00425B2B	CALL NEAR DWORD PTR DS:[46F60F]	DS:[0046F60F]=00A20A9C
00425B3F	CALL NEAR DWORD PTR DS:[46F60F]	DS:[0046F60F]=00A20A9C
00425B53	CALL NEAR DWORD PTR DS:[46F60F]	DS:[0046F60F]=00A20A9C
00425BF1	CALL NEAR DWORD PTR DS:[46F60F]	DS:[0046F60F]=00A20A9C
00425C9C	CALL NEAR DWORD PTR DS:[46F434]	DS:[0046F434]=00A20049
00425CA6	CALL NEAR DWORD PTR DS:[46F68C]	DS:[0046F68C]=00A20D70
00425D71	CALL NEAR DWORD PTR DS:[46F542]	DS:[0046F542]=00A20642
00425D8B	CALL NEAR DWORD PTR DS:[46F524]	DS:[0046F524]=00A205BA
00425DBB	CALL NEAR DWORD PTR DS:[46F52E]	DS:[0046F52E]=00A205EC
00425E13	CALL NEAR DWORD PTR DS:[46F6D7]	DS:[0046F6D7]=00A20EE7
00425E27	CALL NEAR DWORD PTR DS:[46F6D2]	DS:[0046F6D2]=00A20EC5
00425E6B	CALL NEAR DWORD PTR DS:[46F6D2]	DS:[0046F6D2]=00A20EC5
00425F34	CALL NEAR DWORD PTR DS:[46F6D2]	DS:[0046F6D2]=00A20EC5
004271F7	ADD ECX,EDX	(Initial CPU selection)
00427200	CALL E66B1515	
0042723E	CALL NEAR DWORD PTR DS:[46F43E]	DS:[0046F43E]=00A2007E
004272D5	CALL NEAR DWORD PTR DS:[46F439]	DS:[0046F439]=00A20069
004272F6	CALL NEAR DWORD PTR DS:[46F6DC]	DS:[0046F6DC]=00A20F09
00427458	CALL 00400236	UnPackMe.004001D8
004274F7	CALL 0040023C	UnPackMe.0040023C
004277F3	CALL 0040027D	UnPackMe.0040027D
00427929	CALL NEAR DWORD PTR DS:[46F448]	DS:[0046F448]=00A200C1
00427E07	CALL NEAR DWORD PTR DS:[46F57E]	DS:[0046F57E]=00A20793
00427E0E	CALL NEAR DWORD PTR DS:[46F452]	DS:[0046F452]=00A200FE
00427E98	CALL NEAR DWORD PTR DS:[46F44D]	DS:[0046F44D]=00A200E1
00428029	CALL NEAR DWORD PTR DS:[46F4CF]	DS:[0046F4CF]=00A203E9
00428042	CALL 004003ED	UnPackMe.004001D8
004280BB	CALL NEAR DWORD PTR DS:[46F4E3]	DS:[0046F4E3]=00A20451
00428805	CALL 00427360	UnPackMe.004001D8
00428823	CALL NEAR DWORD PTR DS:[46F542]	DS:[0046F542]=00A20642
0042884D	CALL NEAR DWORD PTR DS:[46F524]	DS:[0046F524]=00A205BA
0042886E	CALL NEAR DWORD PTR DS:[46F52E]	DS:[0046F52E]=00A205EC
004288AE	CALL NEAR DWORD PTR DS:[46F524]	DS:[0046F524]=00A205BA
004288E0	CALL NEAR DWORD PTR DS:[46F524]	DS:[0046F524]=00A205BA
0042891E	CALL NEAR DWORD PTR DS:[46F52E]	DS:[0046F52E]=00A205EC
00428950	CALL NEAR DWORD PTR DS:[46F52E]	DS:[0046F52E]=00A205EC
00428983	CALL NEAR DWORD PTR DS:[46F6D7]	DS:[0046F6D7]=00A20EE7
00428C15	CALL NEAR DWORD PTR DS:[46F6DC]	DS:[0046F6DC]=00A20F09
00428C25	CALL NEAR DWORD PTR DS:[46F669]	DS:[0046F669]=00A20C98
004293A9	CALL NEAR DWORD PTR DS:[46F461]	DS:[0046F461]=00A2014D
004293C8	CALL NEAR DWORD PTR DS:[46F45C]	DS:[0046F45C]=00A20137
00429401	CALL NEAR DWORD PTR DS:[46F4D4]	DS:[0046F4D4]=00A20401
0042951F	CALL NEAR DWORD PTR DS:[46F466]	DS:[0046F466]=00A2016F
00429536	CALL NEAR DWORD PTR DS:[46F4D9]	DS:[0046F4D9]=00A20419
00429560	CALL NEAR DWORD PTR DS:[46F466]	DS:[0046F466]=00A2016F
00429597	CALL NEAR DWORD PTR DS:[46F4D9]	DS:[0046F4D9]=00A20419
004295E9	CALL NEAR DWORD PTR DS:[46F466]	DS:[0046F466]=00A2016F
00429892	CALL NEAR DWORD PTR DS:[46F46B]	DS:[0046F46B]=00A2018D
00429CBD	CALL NEAR DWORD PTR DS:[46F547]	DS:[0046F547]=00A20658
00429CCF	CALL NEAR DWORD PTR DS:[46F501]	DS:[0046F501]=00A204FB
00429DA8	CALL NEAR DWORD PTR DS:[46F470]	DS:[0046F470]=00A2019B
00429DDE	CALL 00427360	UnPackMe.004001D8
00429E2A	CALL NEAR DWORD PTR DS:[46F439]	DS:[0046F439]=00A20069

Здесь есть много CALL'ов, берущих значения из IAT второго вида. Назовем последнюю просто Таблицей, чтобы отличать ее от правильной IAT.

Посмотрим нашу Таблицу в DUMP'е. Все эти CALL'ы берут из нее значения, так что выберем любой:

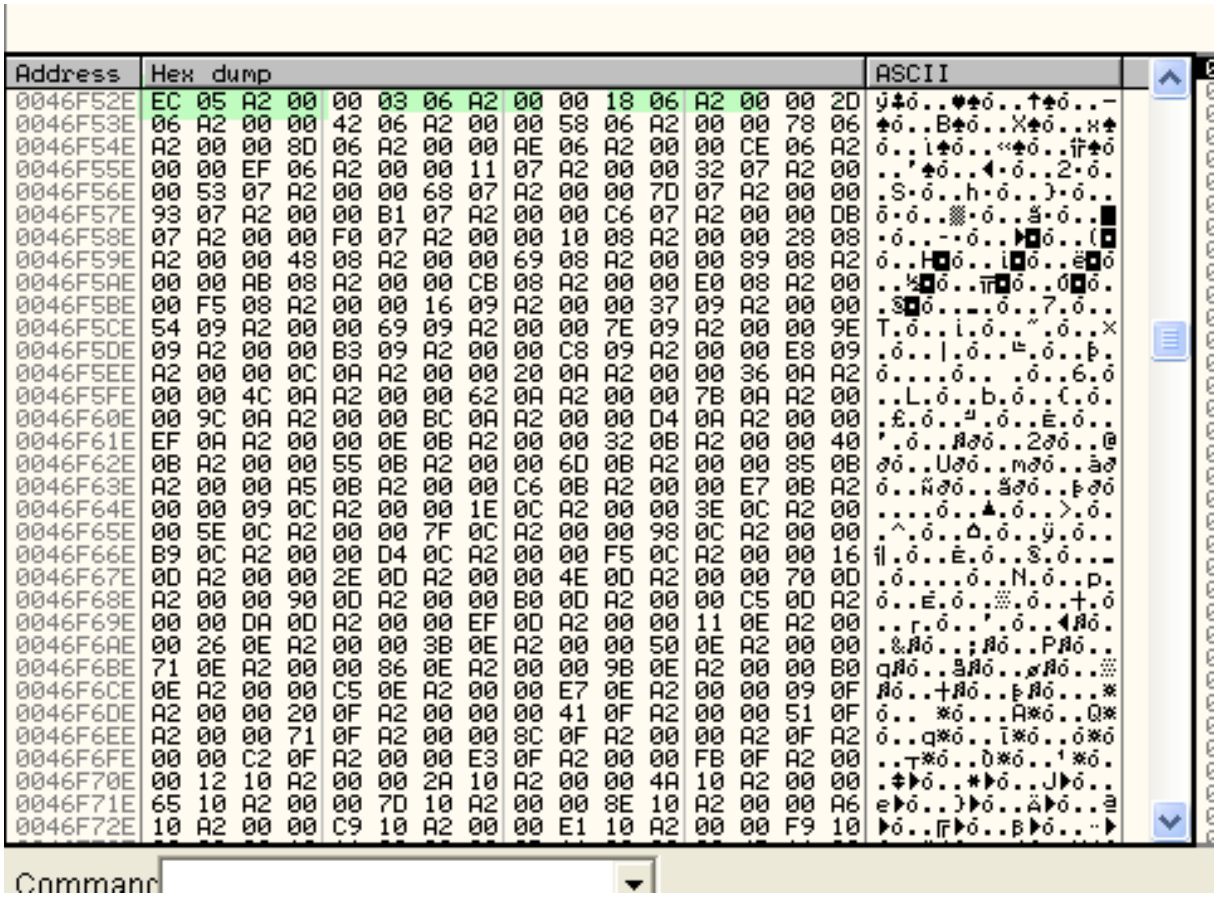


Вот этот CALL в листинге:

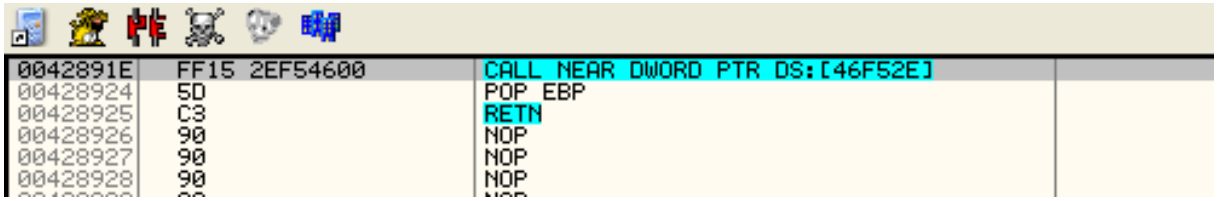


0042891E	FF15 2EF54600	CALL NEAR DWORD PTR DS:[46F52E]
00428924	5D	POP EBP
00428925	C3	RETN
00428926	90	NOP
00428927	90	NOP

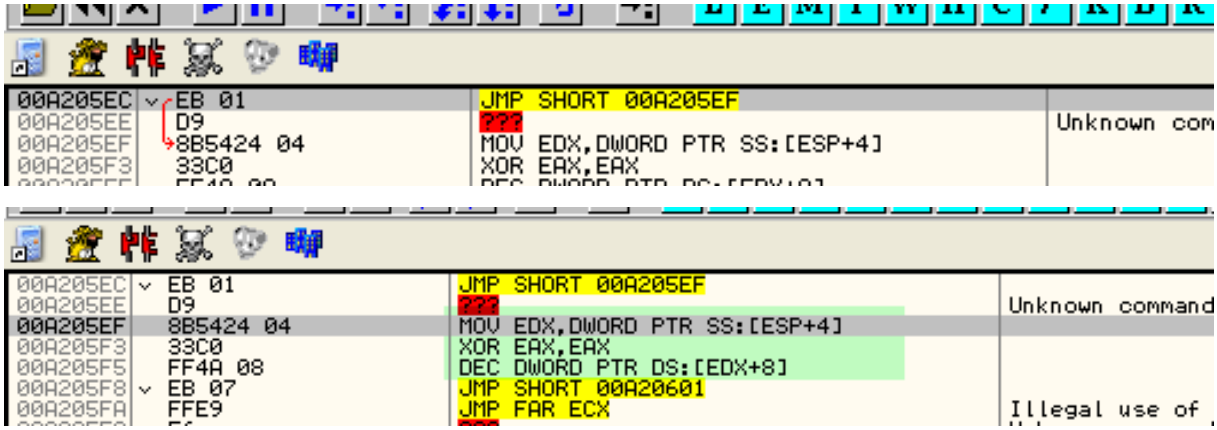
Теперь смотрим в Таблице:



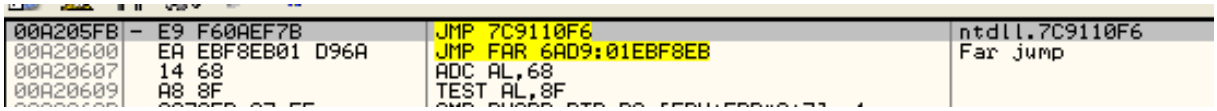
Здесь отсутствует нормальный порядок элементов, присущий IAT'у, но, тем не менее, это всё же Таблица, которая, судя по всему, служит для переадресации API-функций.



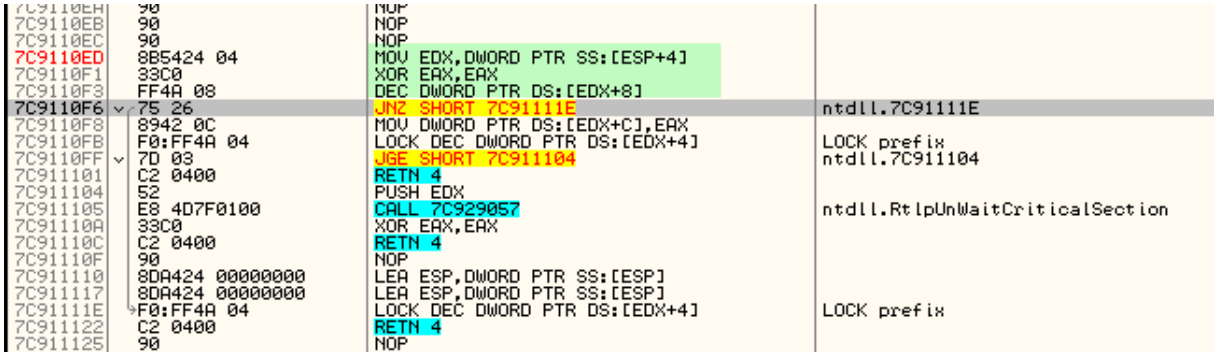
С помощью Follow проверим, ведет ли данный вызов к API-функции:



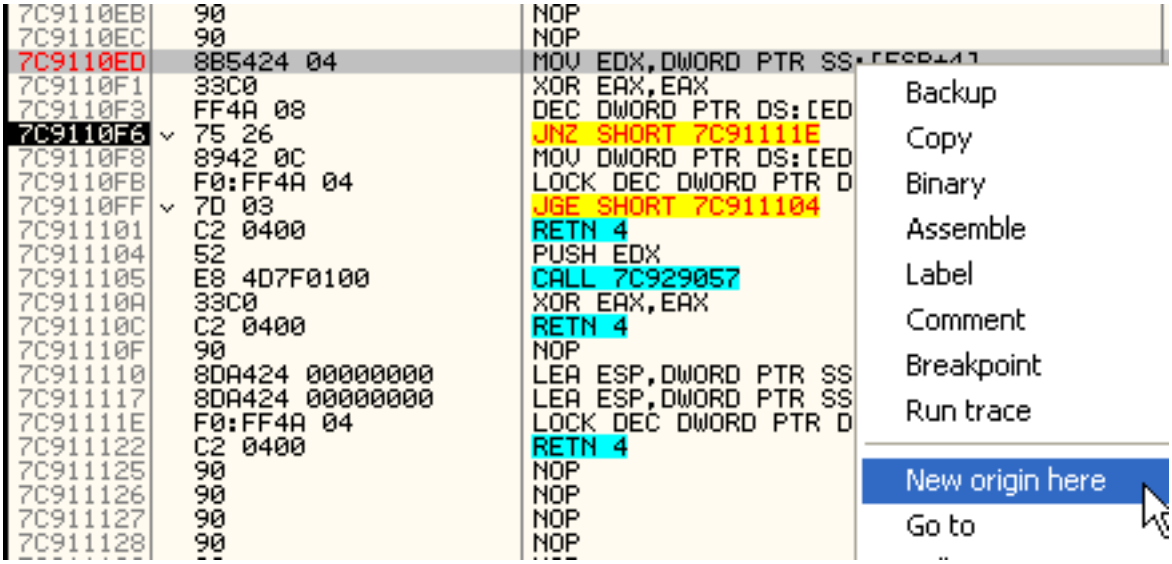
После пары переходов оказываемся здесь:



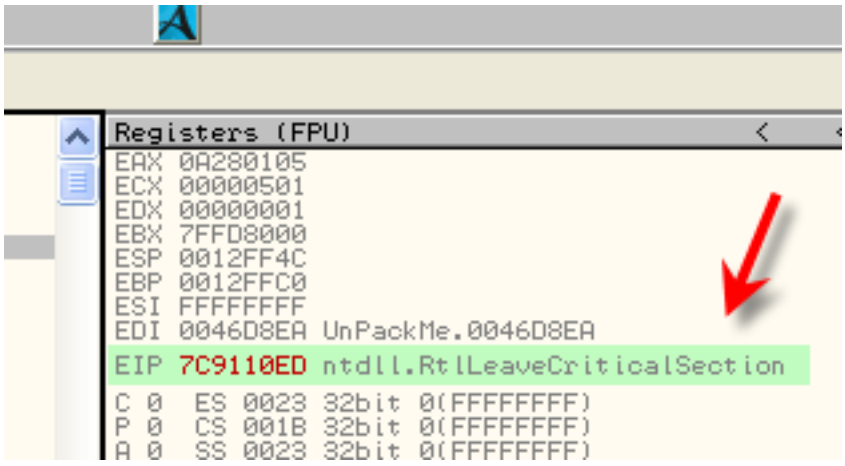
И далее:



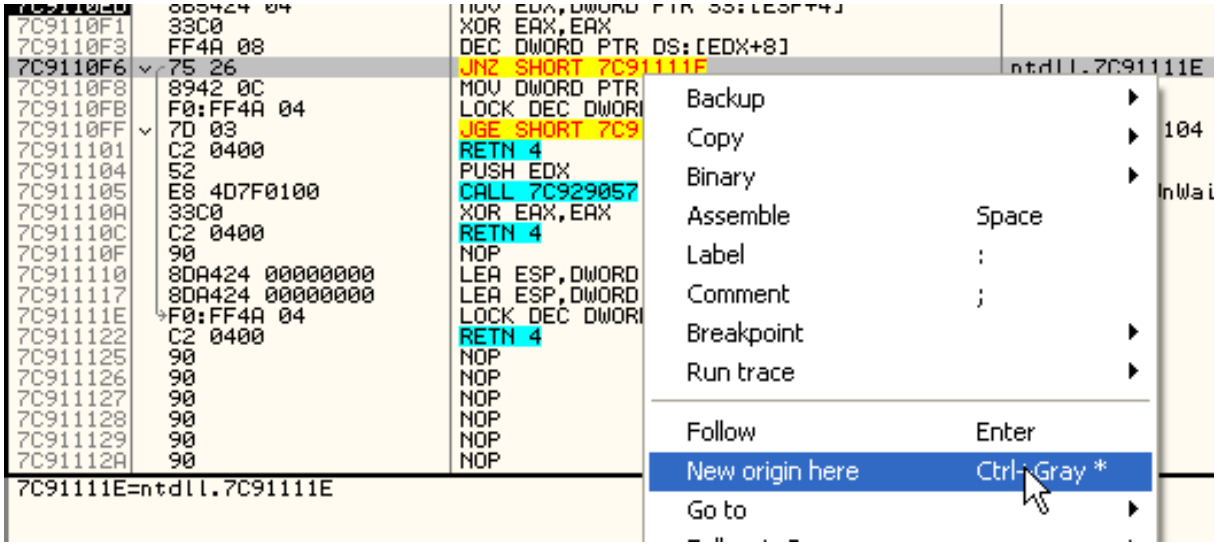
Переход произошел на 4-ю команду API-функции. Перед этим первые три выполняются в собственной секции, вероятно, для предотвращения остановки на BP и для усложнения определения имени API-функции. Тем не менее, это можно сделать, заменив значение EIP адресом самого начала API-функции с помощью правого клика мыши –> New origin here:



В окне регистров появится искомое имя:

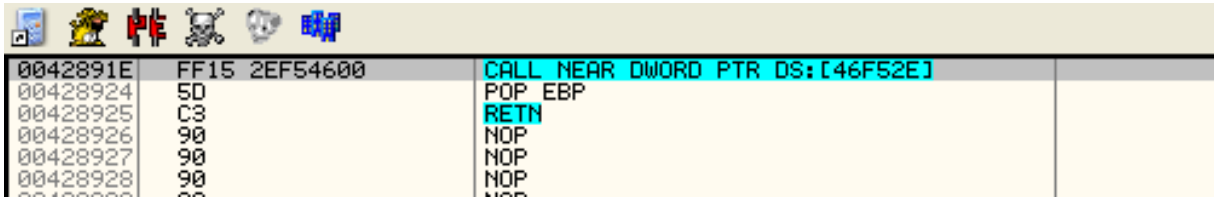


Узнав его, следует возвратиться к JNZ во избежание проблем:

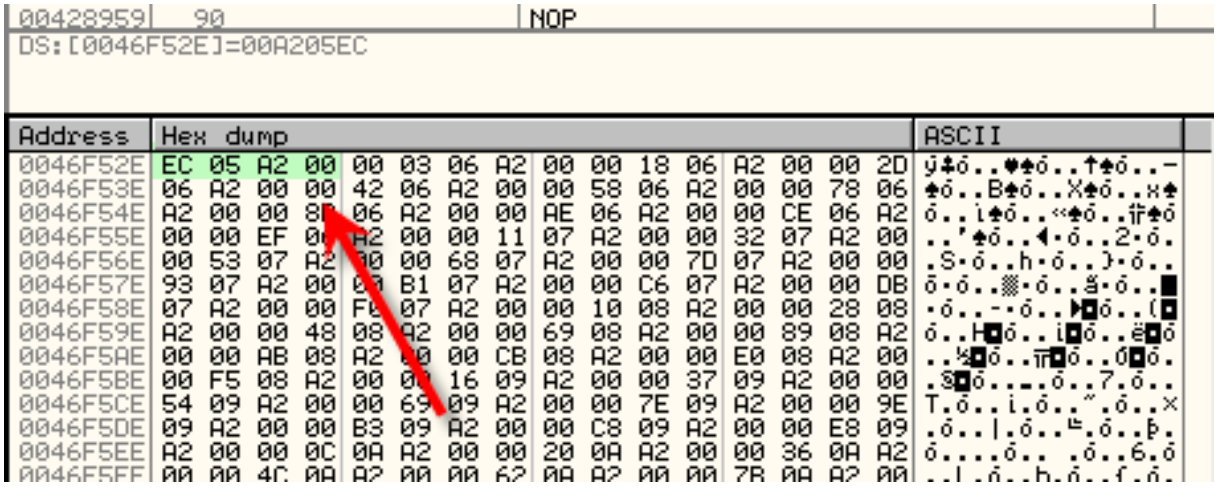


Что ж, это только начало: мы узнали имя API-функции, к которой приводит один CALL, а у нас есть еще исковерканная правильная IAT и CALL'ы, ведущие к API-функциям, которые вместо того, чтобы брать значения из правильной IAT, берут их из Таблицы, не стоящей того, чтобы ее восстанавливали.

Ладно, нам известно, что этот CALL

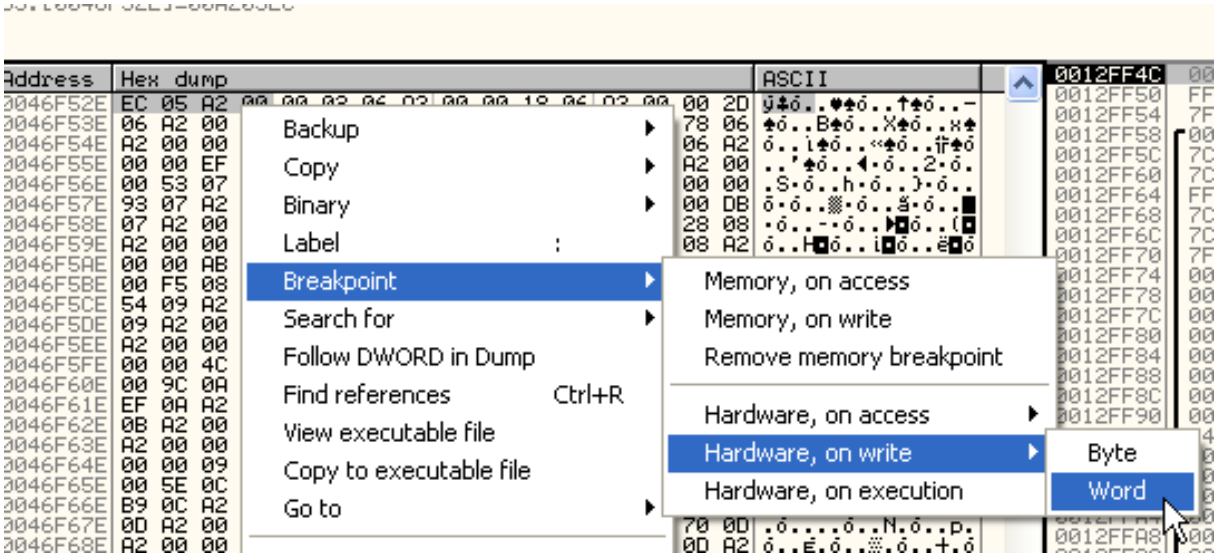


приводит к API-функции RtlLeaveCriticalSection, и это уже информация, которая пригодится в дальнейшем. Попытаемся решить анпэкми в несколько этапов. В первом из них возьмем наш CALL, использующий значение из Таблицы, и удалим переадресацию:

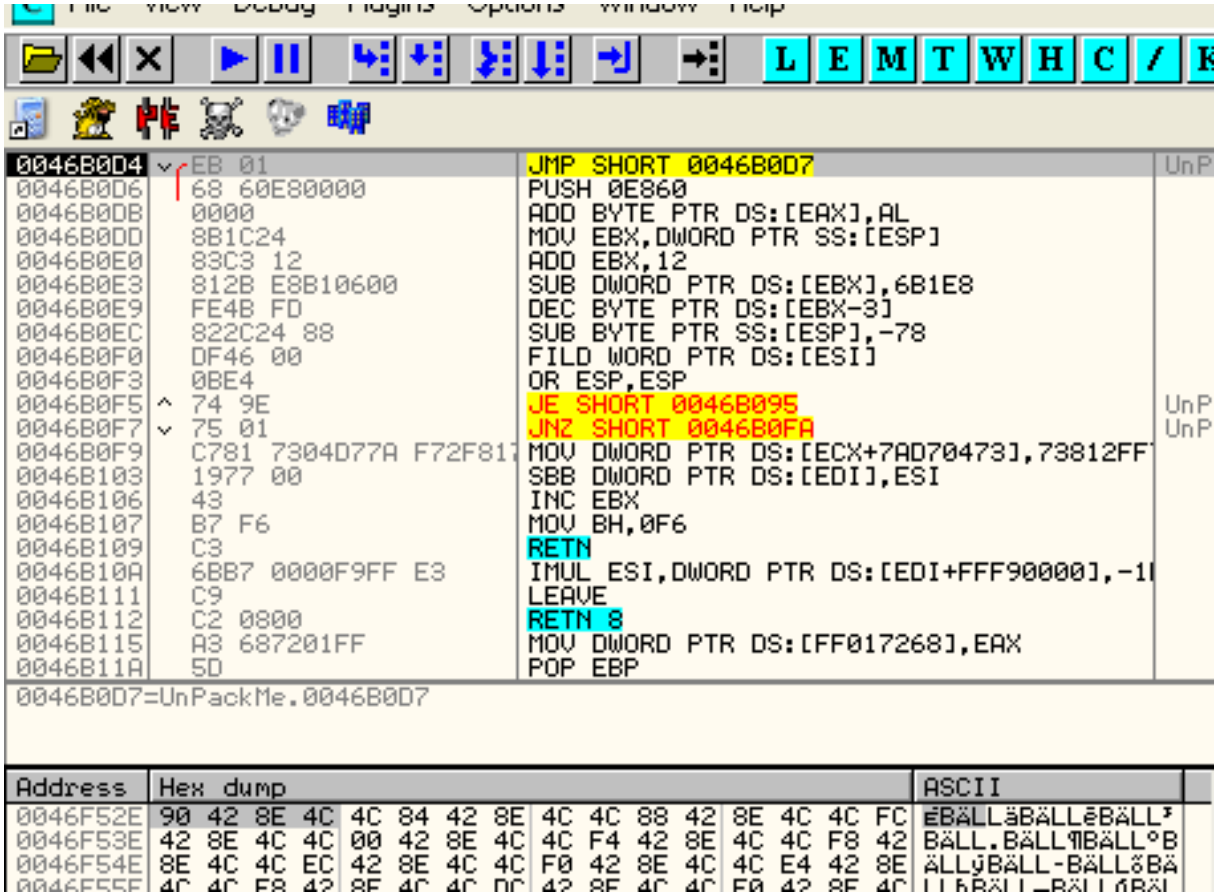


Нам нужно найти способ создания прямого обращения к правильным адресам API-функций, то есть в данном случае в Таблице вместо 00A205EC должно быть 7C9110ED (таков адрес упомянутой API-функции на моем компьютере).

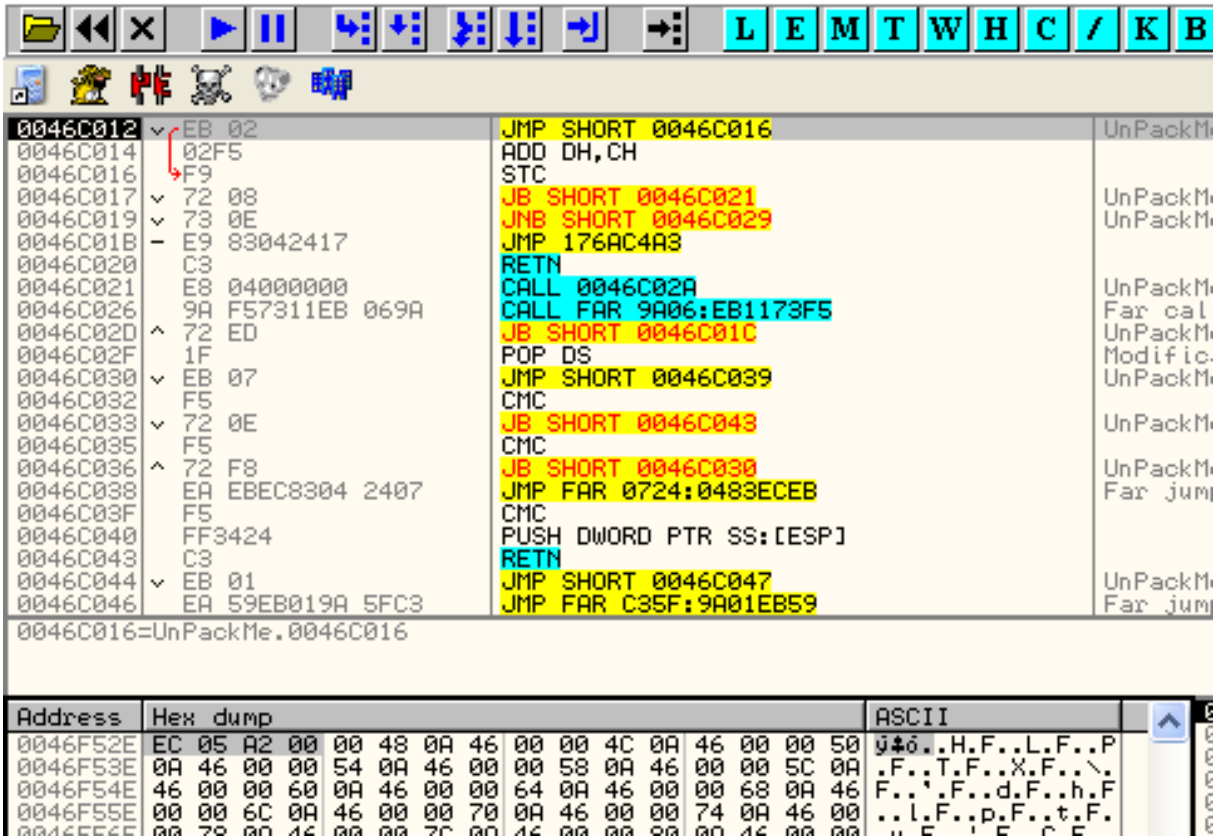
Чтобы узнать адрес инструкции, сохраняющей значение в элементе Таблицы, установим на нем HARDWARE BREAKPOINT ON WRITE:



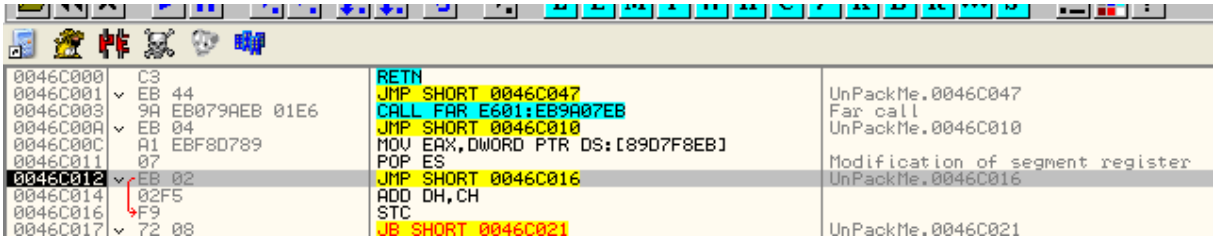
Перезагрузим программу и найдем то место, где сохраняется значение 00A205EC (так на моем компьютере, а у вас оно будет другим).



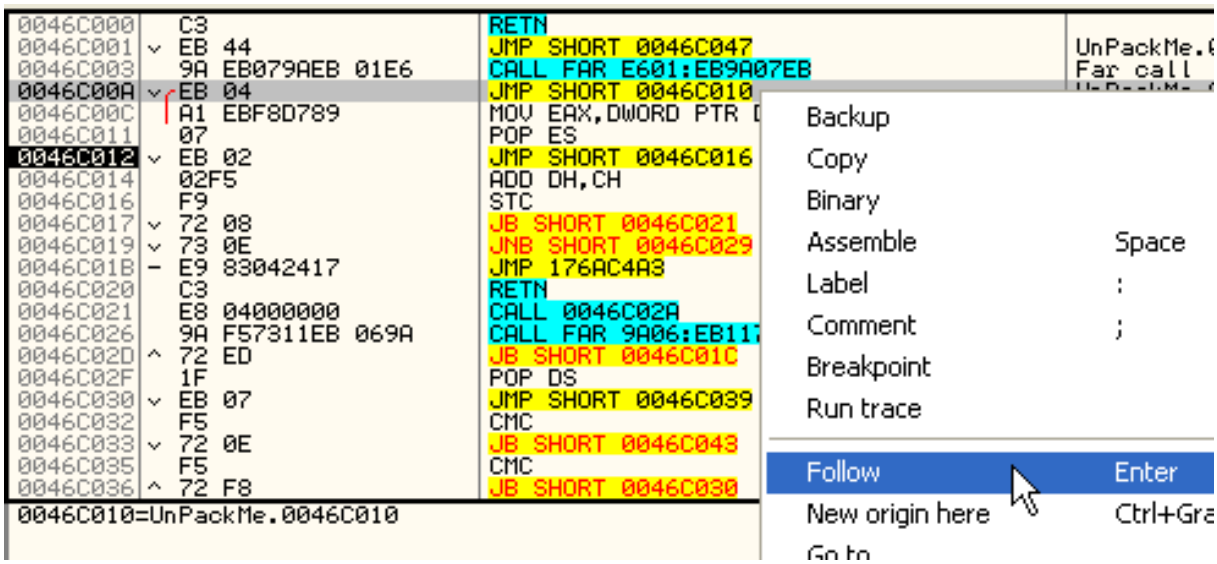
Нажмем RUN:

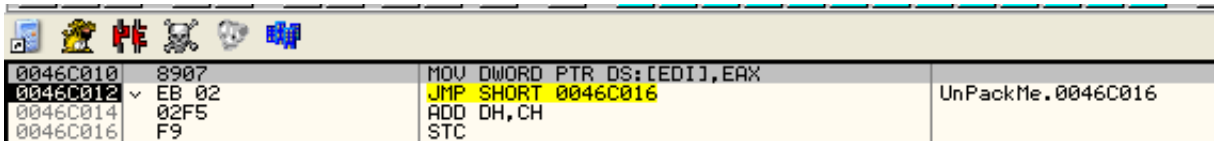


После пары остановок оказались здесь, при сохранении значения. Так как HARDWARE ON WRITE срабатывает всегда после выполнения команды, то посмотрим предыдущую строку:

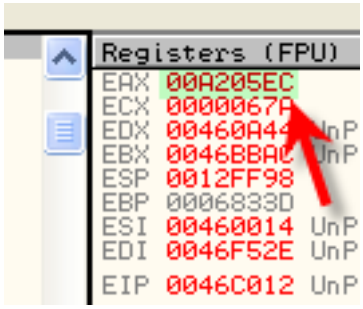


Эта область кода обфусцирована, поэтому реально выполненной инструкции не видно, но если выделить предшествующий JMP и сделать Follow, то она отобразится:

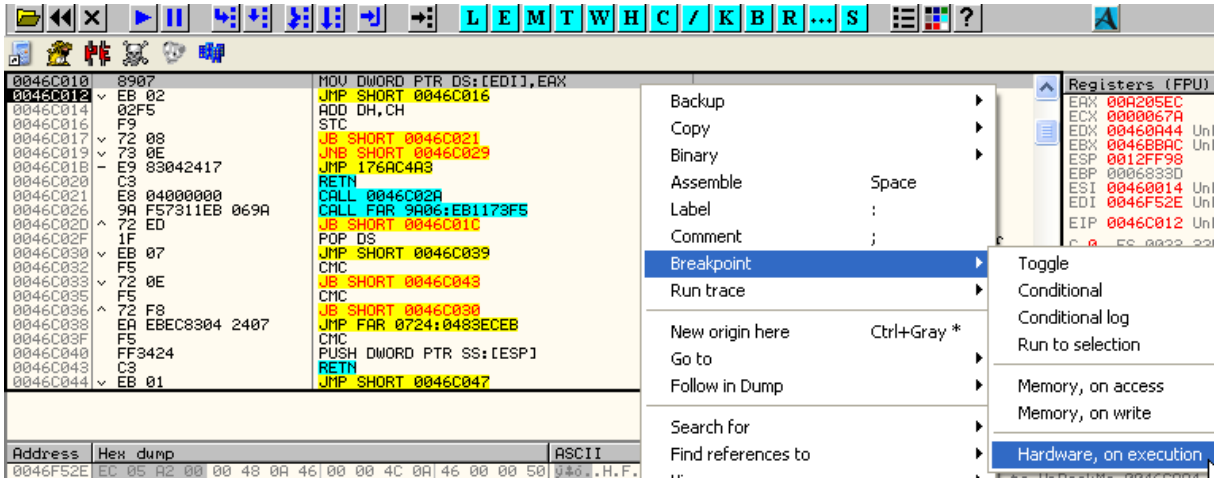


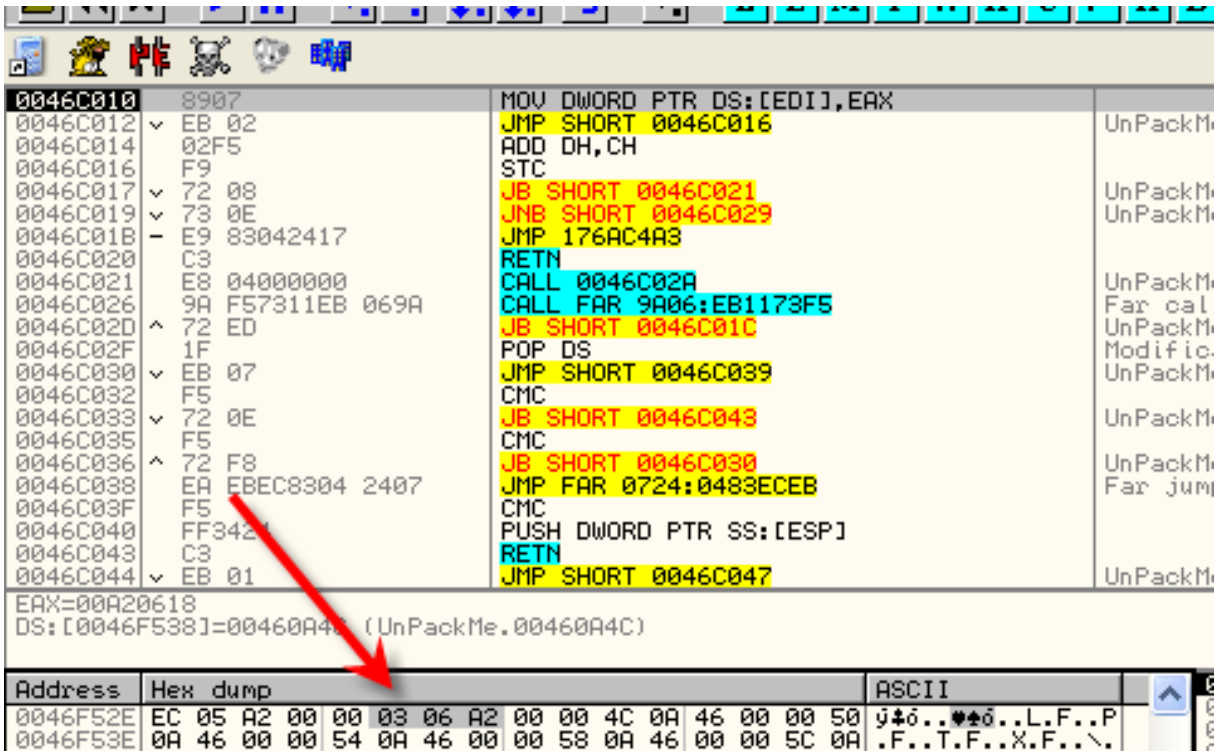


Данная команда сохраняет плохое значение; это становится очевидно, если посмотреть в окно регистров:



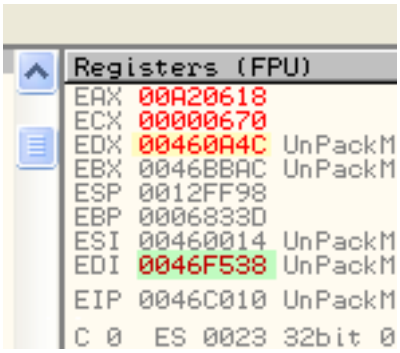
Установим на ней HARDWARE BPX ON EXECUTION:



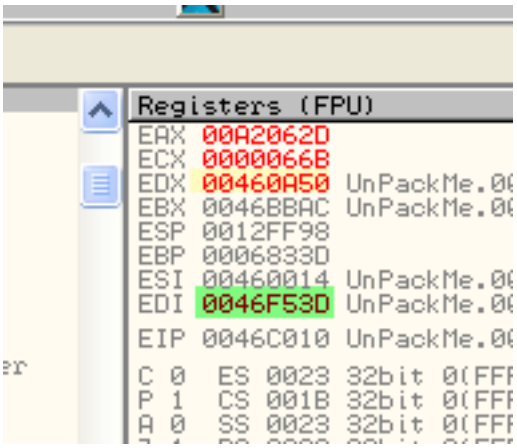


Нажатия на RUN приведут к перебору элементов Таблицы и ее заполнению.

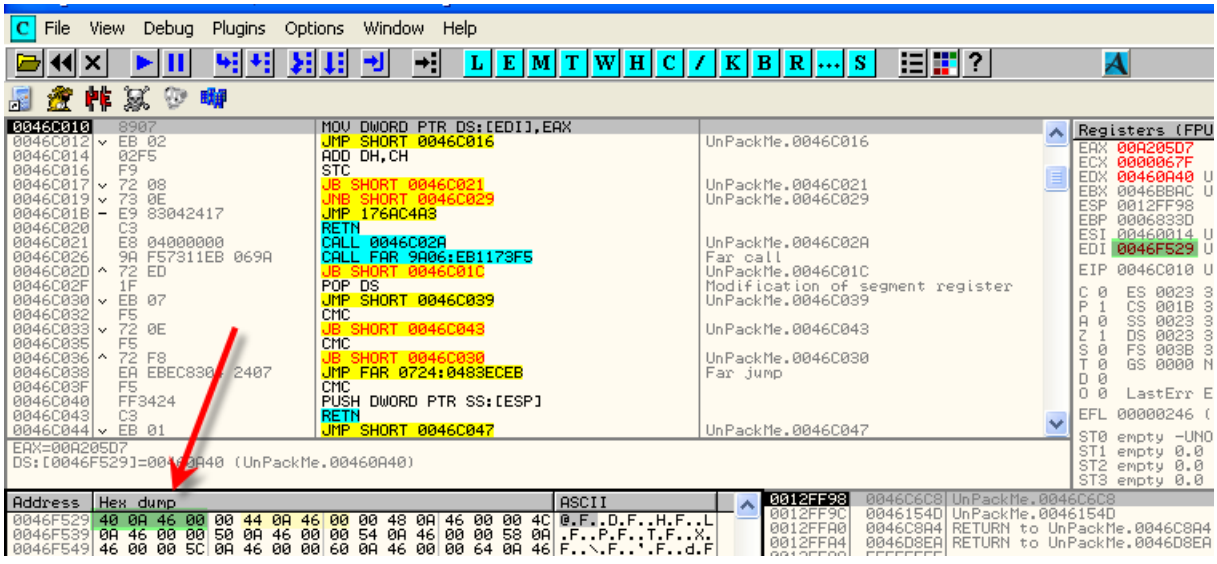
Одновременно с этим происходит и нечто подозрительное в регистрах. Обратим на это внимание при сохранении очередного элемента:



Нажмем RUN еще раз:



Можно заметить, что при увеличении значений в EDI, указывающих на элементы Таблицы, одновременно увеличиваются и значения EDX, указвающие на верную IAT, как если бы это были адреса, по которым должно происходить сохранение на самом деле вместо указанных в EDX. Но пока оставим это и сделаем вот что: перезагрузим анпэкми и дойдем до того места, где сохраняется элемент, идущий перед изучаемым нами:



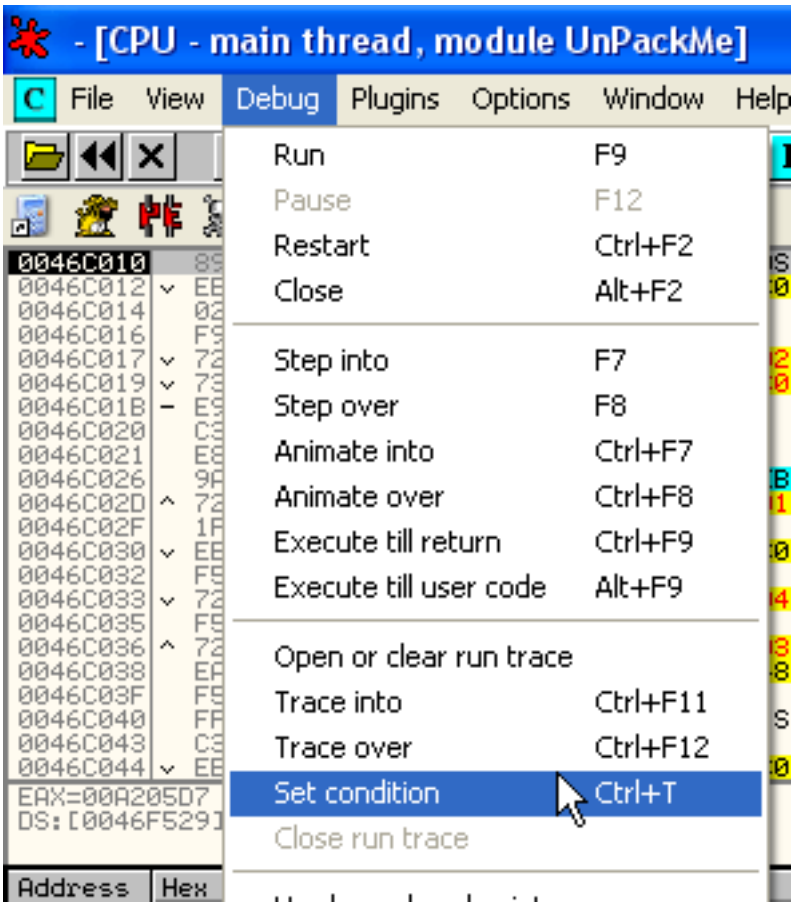
Здесь сохраняется значение в предыдущем элементе 46F529, и если еще раз нажмем F9 (не делайте этого), то окажемся на сохранении знакомого нам элемента.

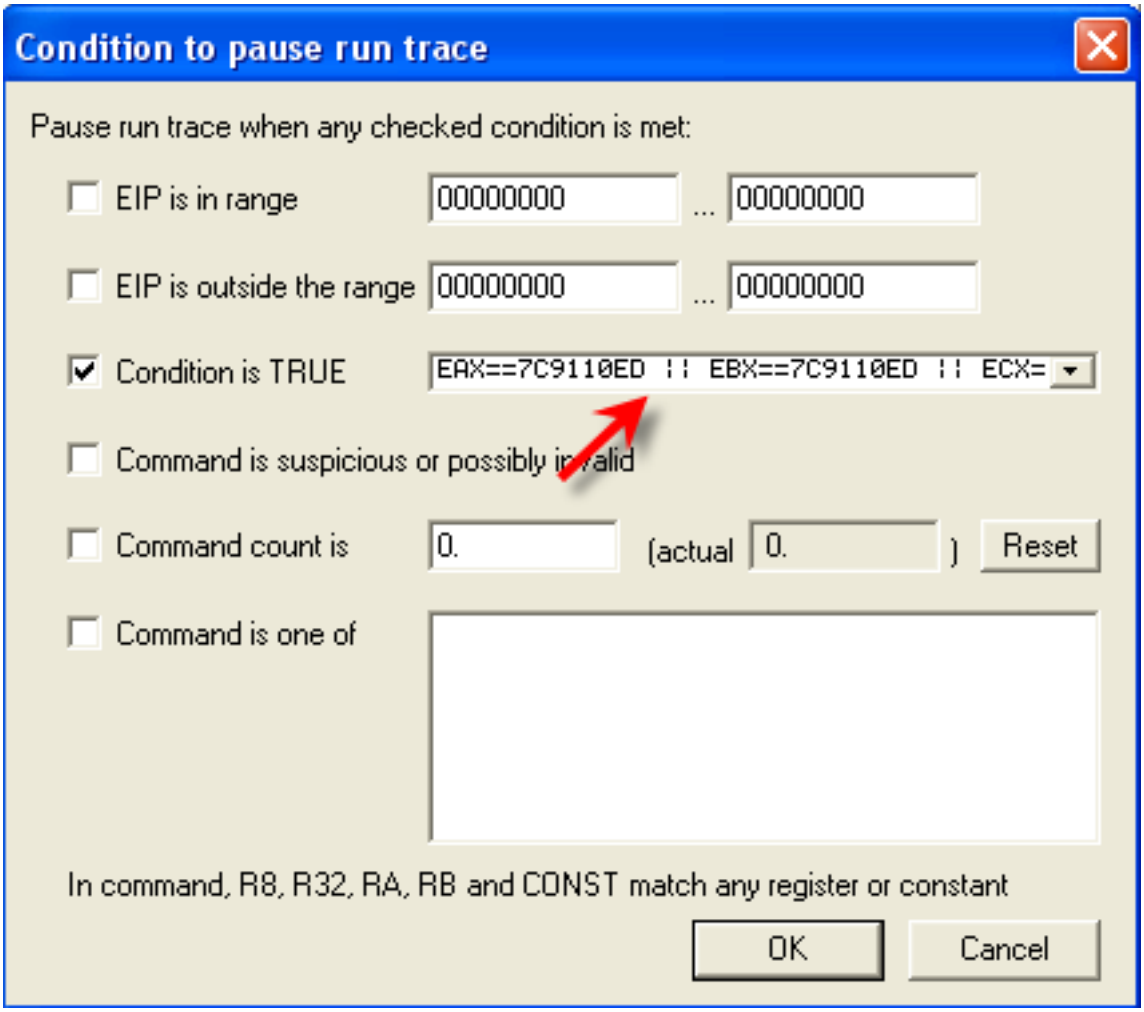
Отсюда придется трассировать, так как перед сохранением переадресовочного значения его необходимо вычислить, исходя из правильного адреса API-функции. Поскольку нам известно, что следующий элемент соответствует функции RtlLeaveCriticalSection, адрес которой 7C9110ED на моем компьютере, будем трассировать до тех пор, пока этот адрес не появится в одном из регистров.

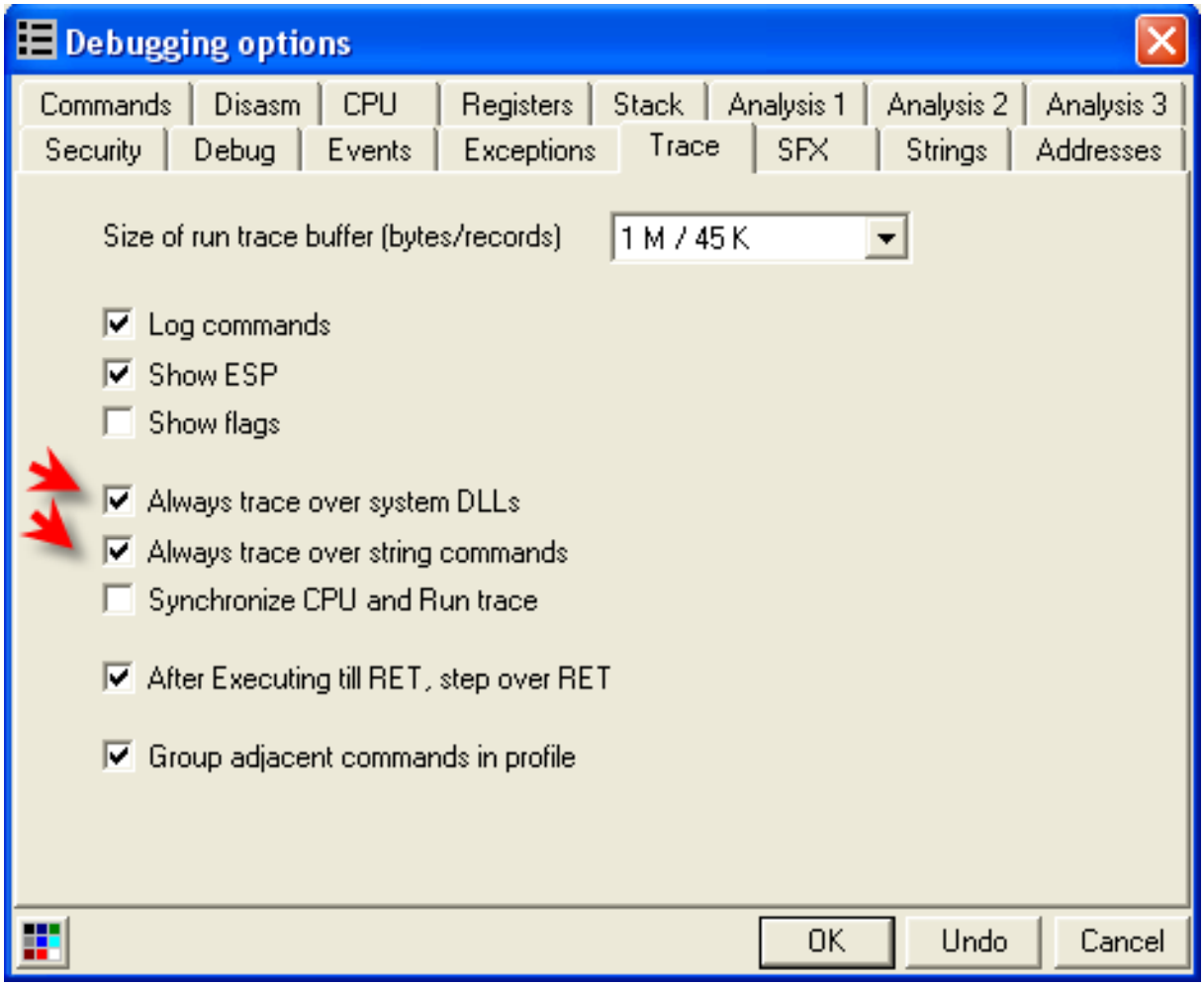
Можно попробовать запустить Trace into с таким условием: EAX==7C9110ED || EBX==7C9110ED || ECX==7C9110ED || EDX==7C9110ED || ESI==7C9110ED || EDI==7C9110ED

Следует всегда помнить, что в условиях трассировки используются двойной знак равенства и двойной знак | для логической операции ИЛИ.

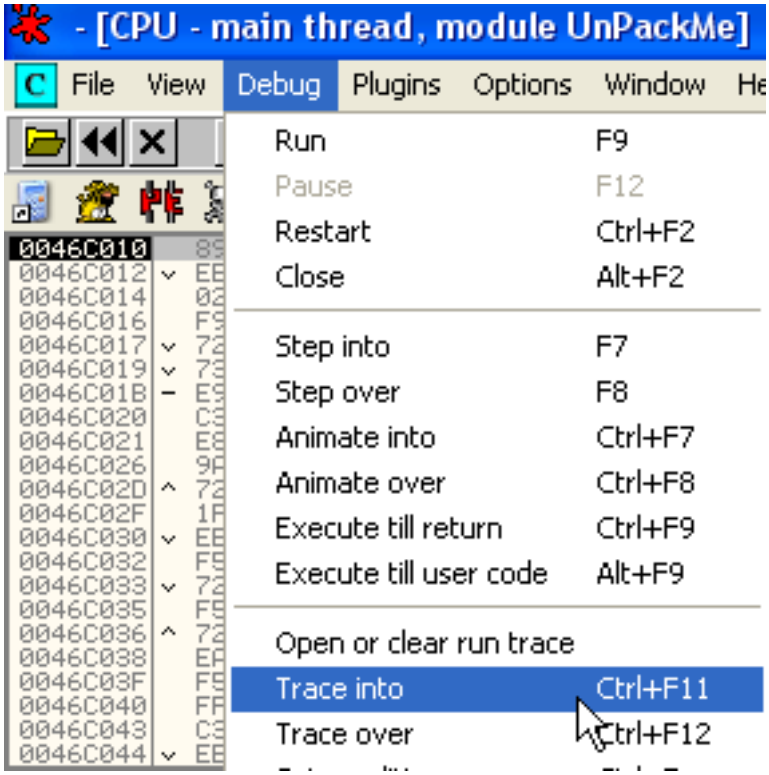
Приведенное условие остановит трассировку, когда в одном из регистров (EAX, EBX, ECX и т.д.) появится значение 7C9110ED.



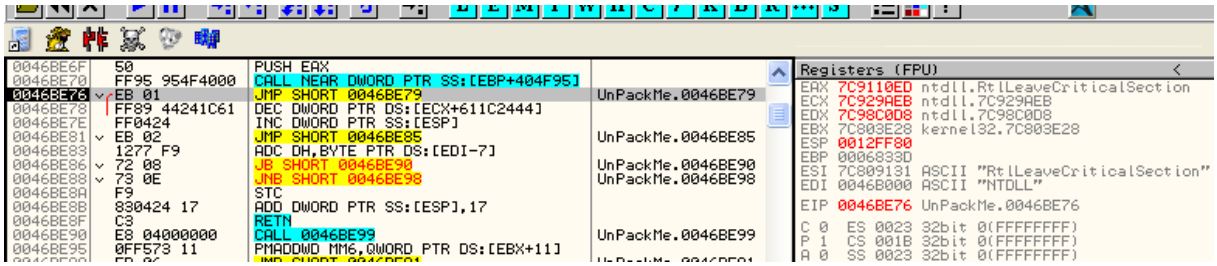

Condition to pause run trace
Pause run trace when any checked condition is met:
EIP is in range
00000000
...
00000000
EIP is outside the range
00000000
...
00000000
Condition is TRUE
EAX==7C9110ED || EBX==7C9110ED || ECX=
Command is suspicious or possibly invalid
Command count is
0.
(actual
0.
)
Reset
Command is one of
In command, R8, R32, RA, RB and CONST match any register or constant
OK
Cancel



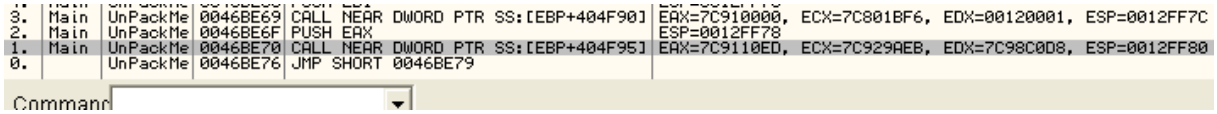
Здесь следует установить две галки, первую — чтобы не было остановок при прохождении по DLL'ке, вторую — чтобы не терять время на повторах REPS, считая такую команду единой инструкцией. Теперь запустим Trace into:



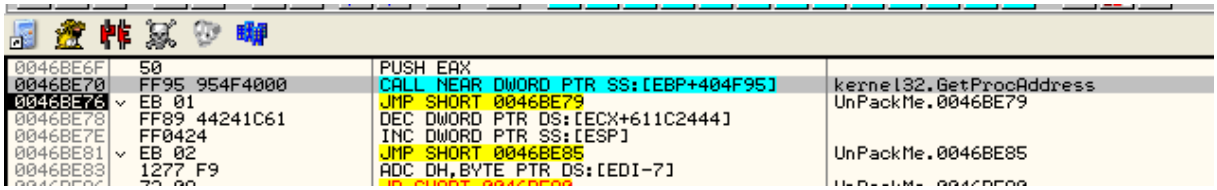
Наверно, стоило бы пойти и долго-долго пить кофе, ведь все эти команды, по идее, нужно было трассировать вручную, хе-хе.



И вот, остановка произошла здесь, после возврата из CALL'а. Посмотрим лог трассировки:

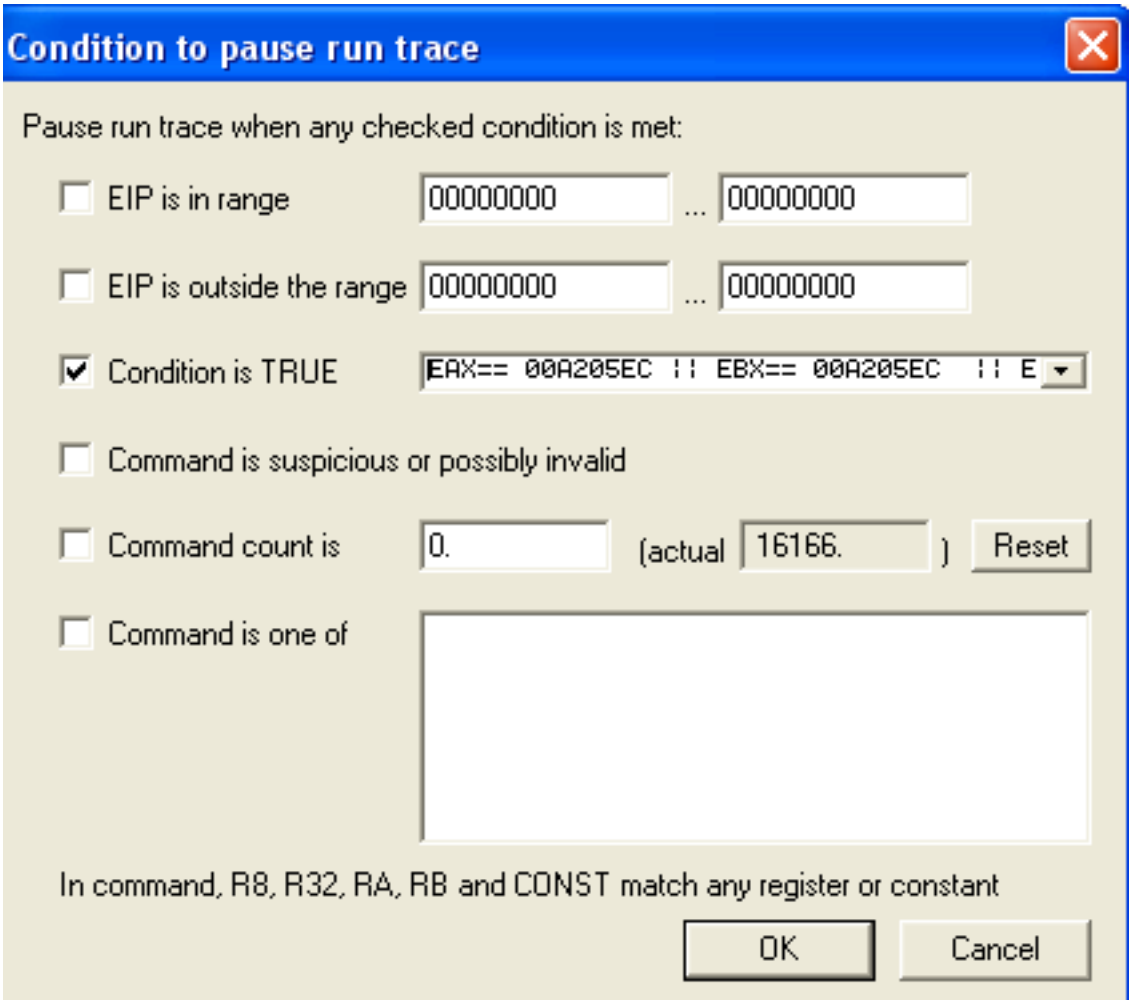


Дважды кликнув на указанной строке лога, окажемся в этом месте:

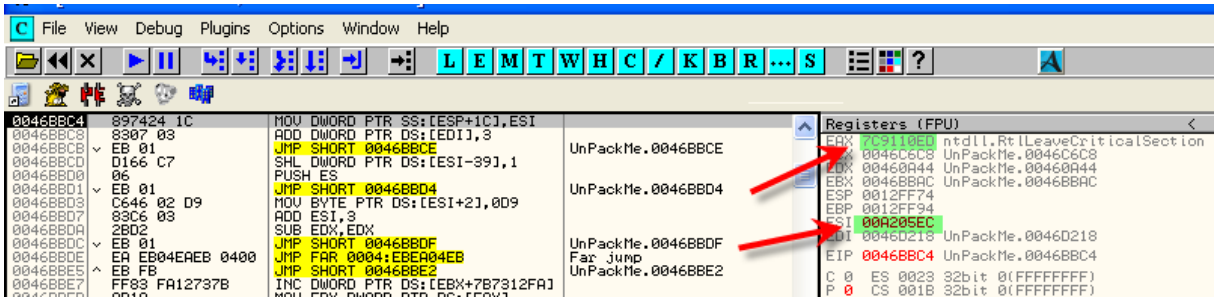


Отладчик подсветил команду, соответствующую строке лога, и это момент, когда программа что-то делает с адресом API-функции перед переадресацией в свою собственную секцию.

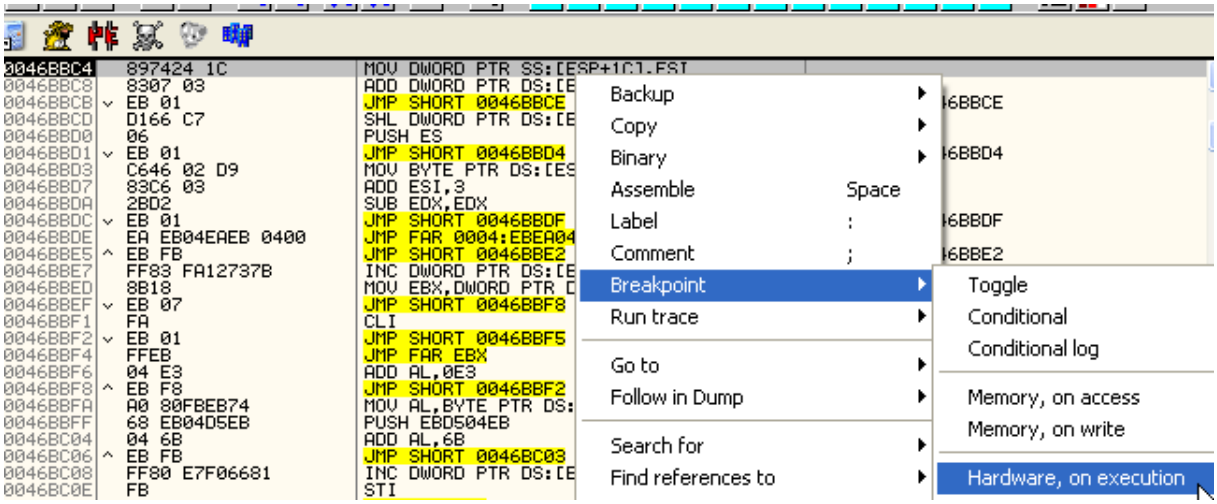
Хорошо, у нас есть первая точка, далее нужно найти место, где извлекается адрес 00A205EC, поэтому изменим условие трассировки следующим образом: EAX == 00A205EC || EBX == 00A205EC || ECX == 00A205EC || EDX == 00A205EC || ESI == 00A205EC || EDI == 00A205EC



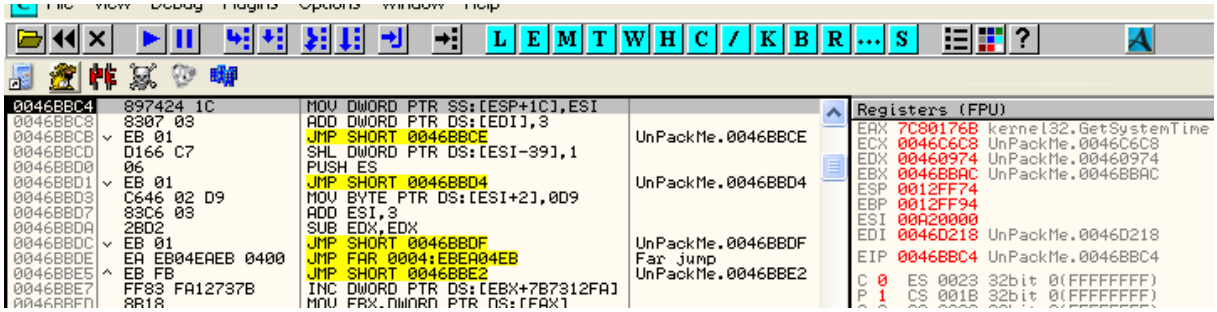
Снова запустим Trace into (Ctrl+F11):



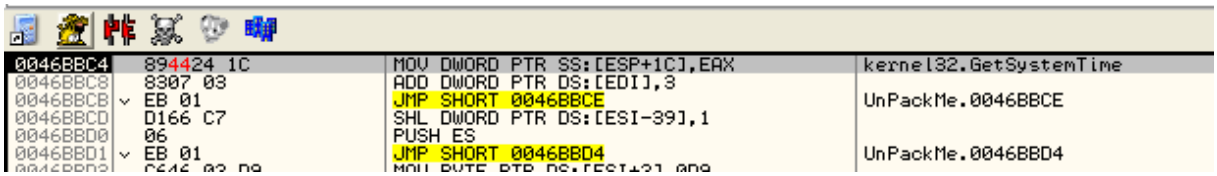
Теперь в регистре EAX находится правильный адрес API-функции, а в ESI — переадресовочный, так что это весьма правильный момент, хе-хе. Поэтому снимем все прежние HBP и установим новый здесь:



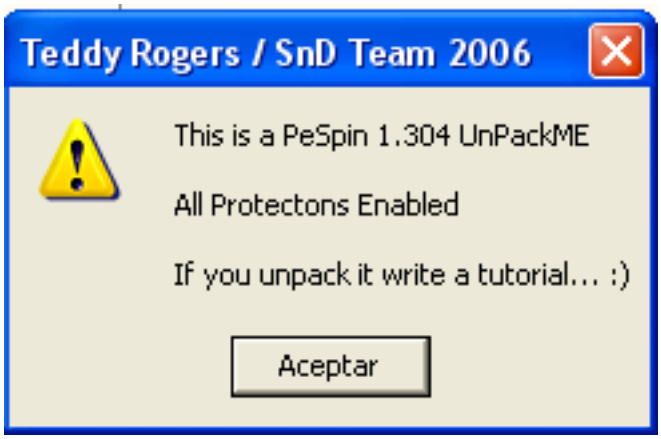
Перезагрузим программу и посмотрим, где будет первая остановка:



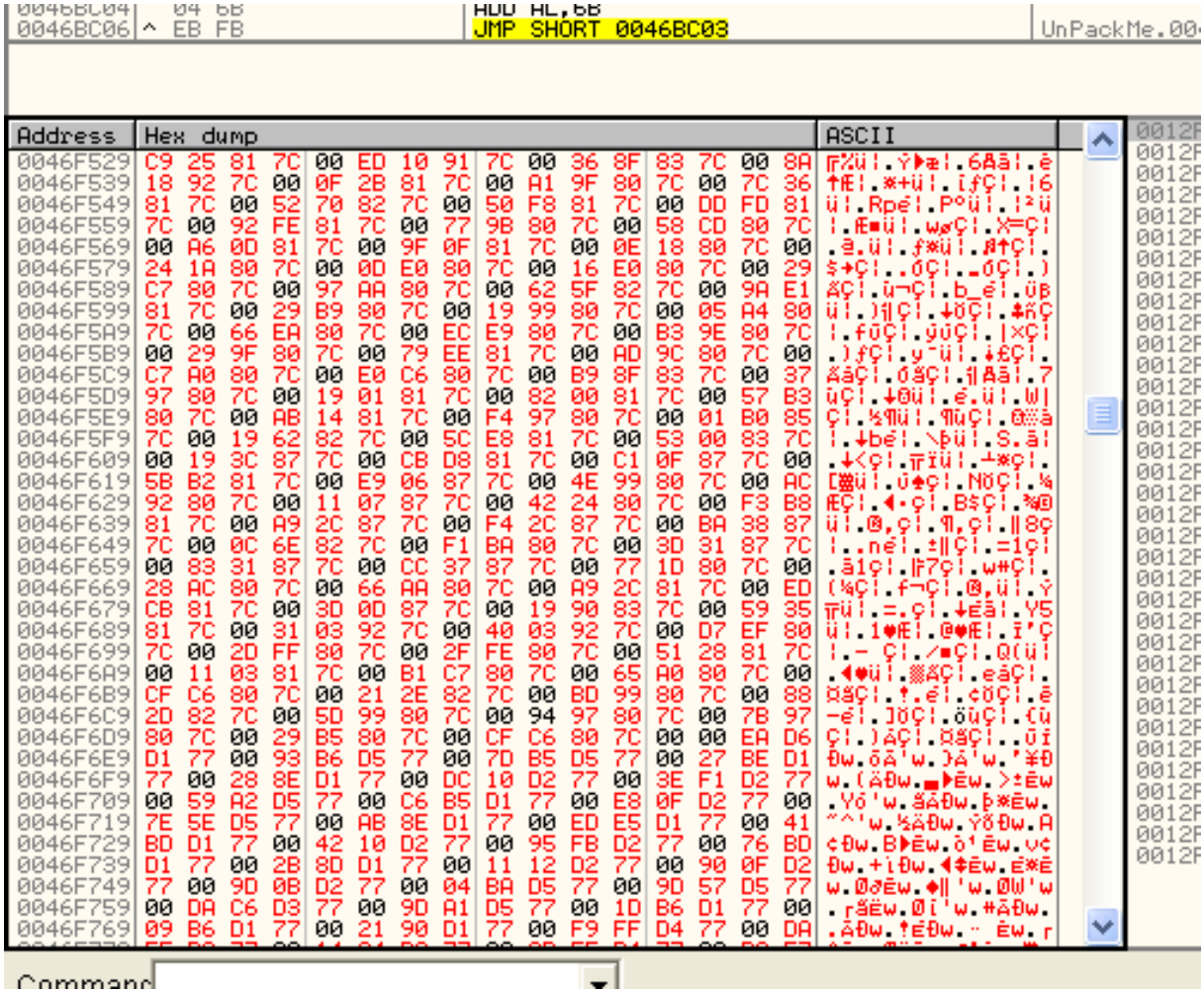
Произошло то же самое: в EAX находится правильный адрес API-функции, а в ESI — переадресовочный. Посмотрим, что будет, если изменить эту инструкцию следующим образом:



Данная команда сохраняет правильный адрес вместо переадресовочного. Теперь удалим HBP и нажмем на RUN:

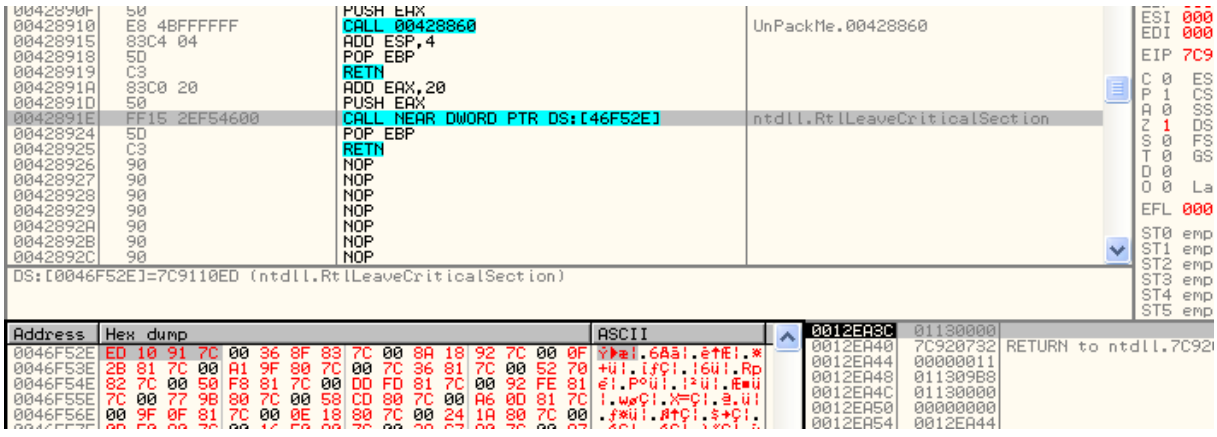


Программа запустилась и не обнаружила изменений. Посмотрим Таблицу:



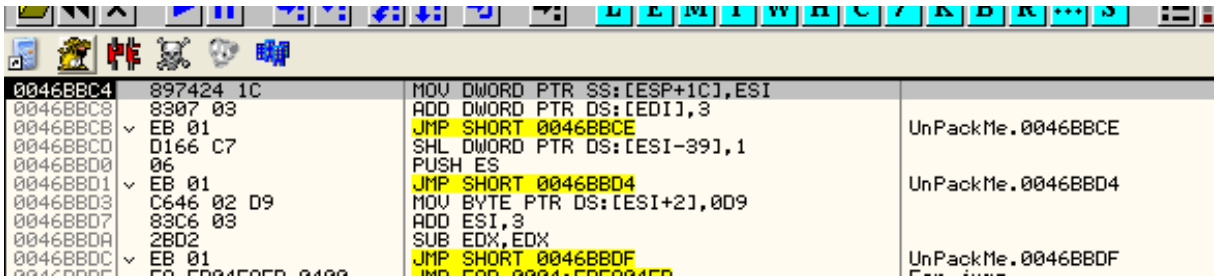
Наша первая цель, состоявшая в том, чтобы снять переадресацию элементов данной Таблицы, была только что выполнена: у нас есть прямые адреса API-функций, и Таблица заполняется верными значениями.

Взглянем на исследованный нами прежде элемент:

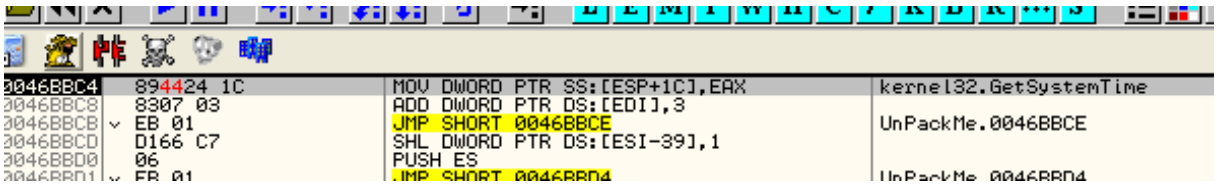


Теперь это прямой вызов API-функции, а не переадресованный. Но он всё еще берет значение из Таблицы, а не из IAT'а, поэтому на очереди второй этап — восстановление IAT таким образом, чтобы правильные значения извлекались только из нее.

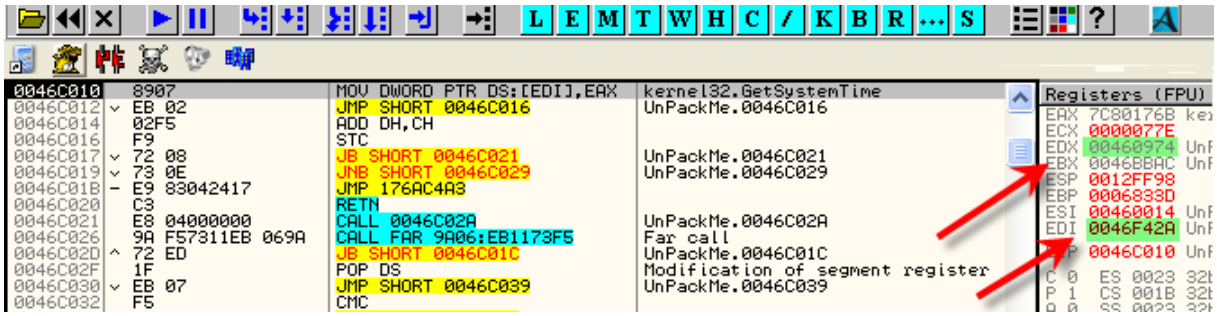
Мы уже выявили две интересные точки, одна — 46BBC4, где, заменив ESI на EAX, была ликвидирована переадресация Таблицы. Другая — 46C010, где происходит сохранение. Установим HBP ON EXECUTION на обе и перезагрузим анпэкми.



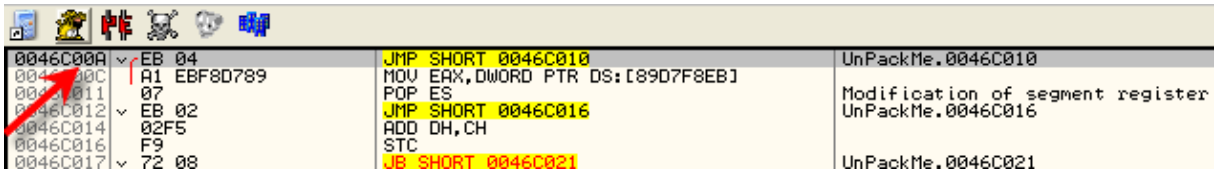
Первая остановка произошла здесь, где, как и прежде, следует заменить ESI на EAX:



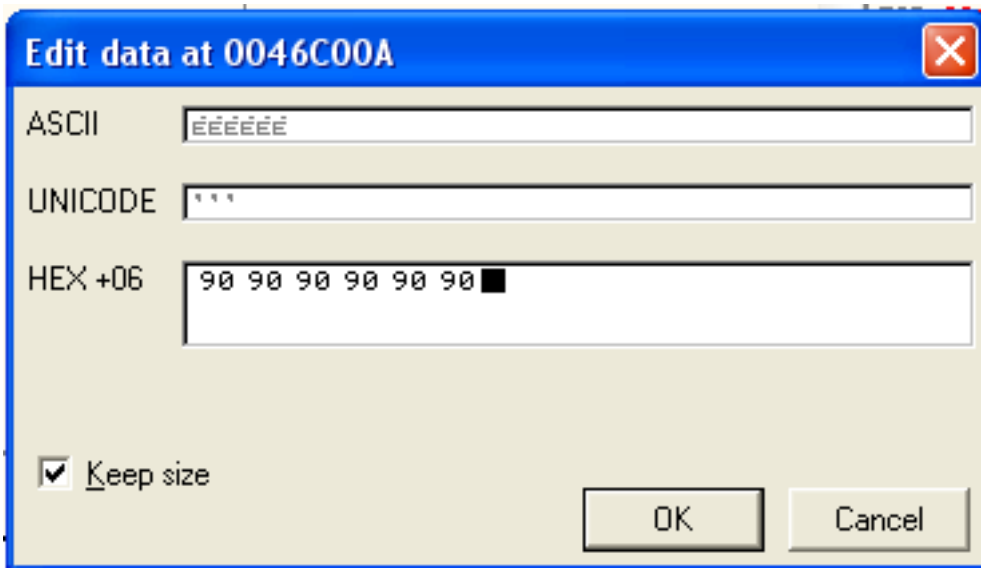
Снова запустим программу:



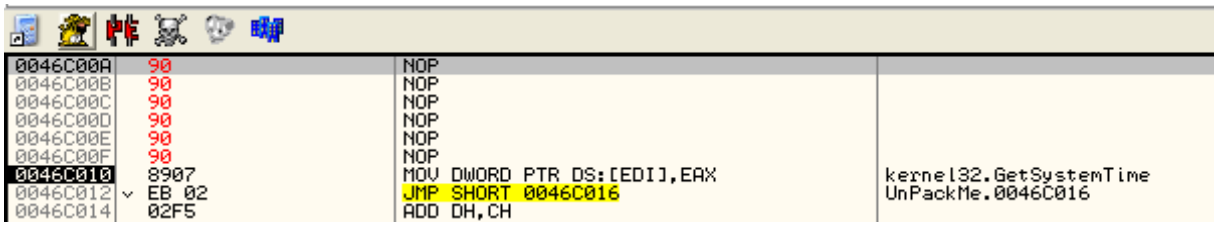
Вторая остановка произошла на сохранении правильного адреса API-функции в Таблицу. Кроме того, нам известно, что в EDX содержится адрес соответствующего элемента IAT'а, но просто заменить EDI на EDX мы не можем, так как CALL'ы берут значения из Таблицы, а не из IAT'а, и программа после замены работать не будет. Однако, вот что мы можем сделать:



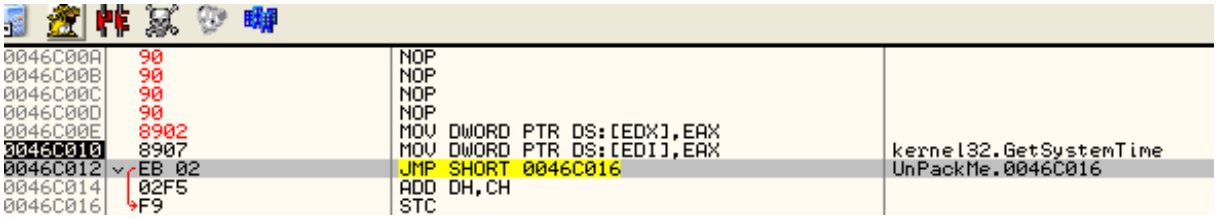
Нам известно, что этот JMP отправляет к инструкции, где происходит сохранение, и что находящиеся между ними байты — просто мусор, поэтому занопаем JMP вместе с мусором:



Должно стать так:

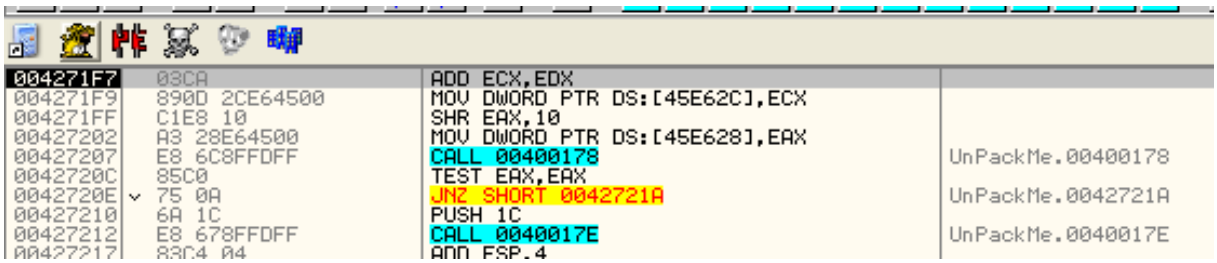


Осталась только инструкция, сохраняющая адрес API-функции в Таблице. Добавим команду, производящую сохранение того же самого адреса в соответствующем элементе IAT'а:

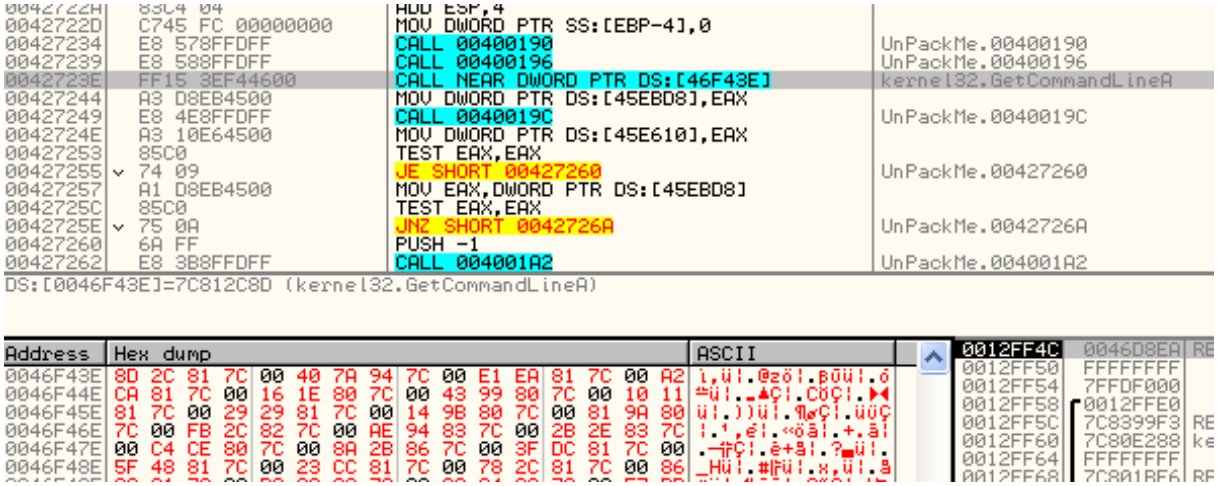


Теперь верный адрес API-функции будет сохраняться и в Таблице, и в IAT'е, что сохранит работоспособность. Снимем HBP и вернемся на шаг назад, чтобы был выполнен добавленный нами MOV.

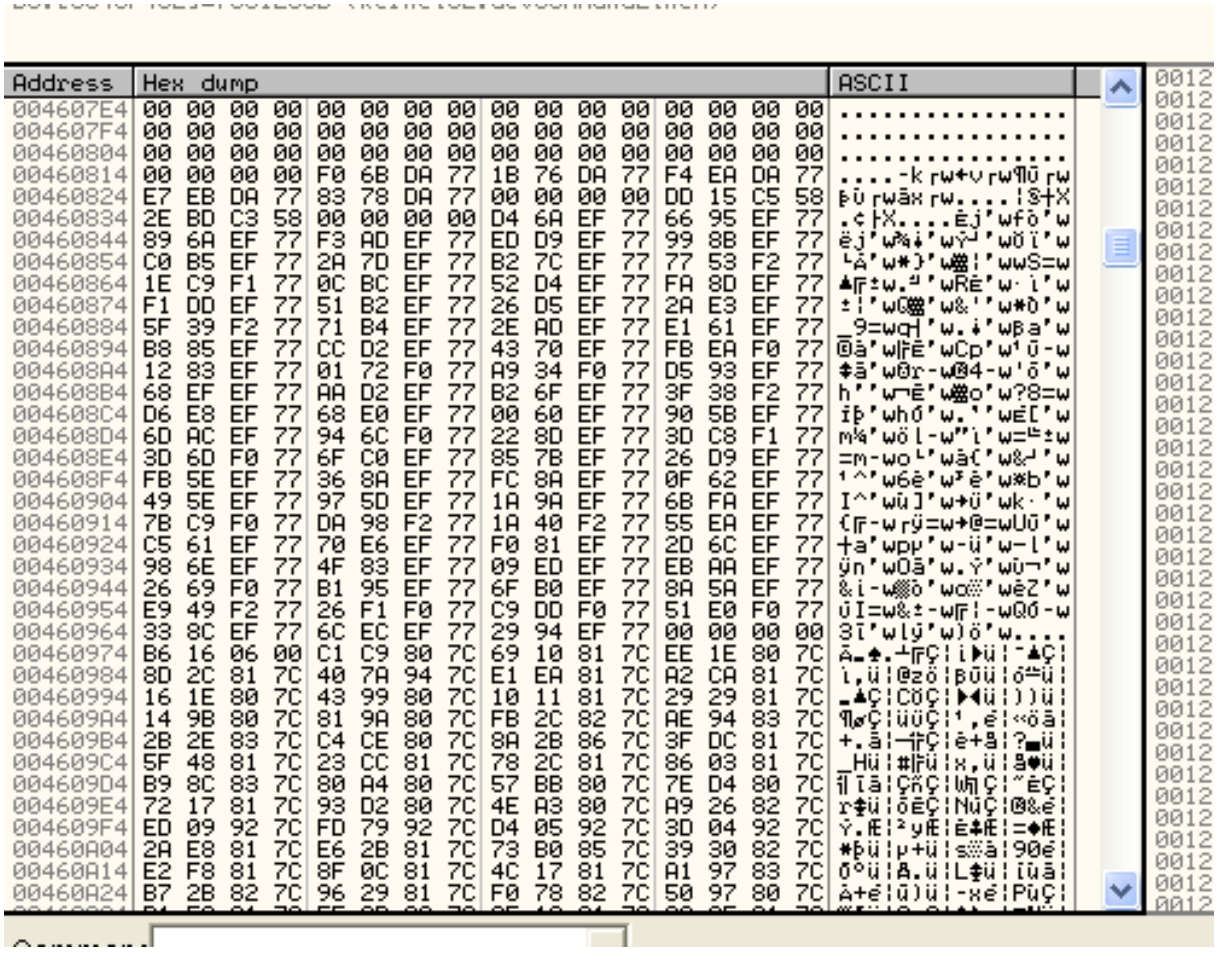
Если сейчас установить BPM ON ACCESS на секцию кода и нажать RUN, то вскоре окажемся на предполагаемой OEP.



Итак, прибыли к ложной OEP. Посмотрим, есть ли в обеих таблицах адрес API-функции, вызываемой CALL'ом, который можно найти в листинге чуть ниже:

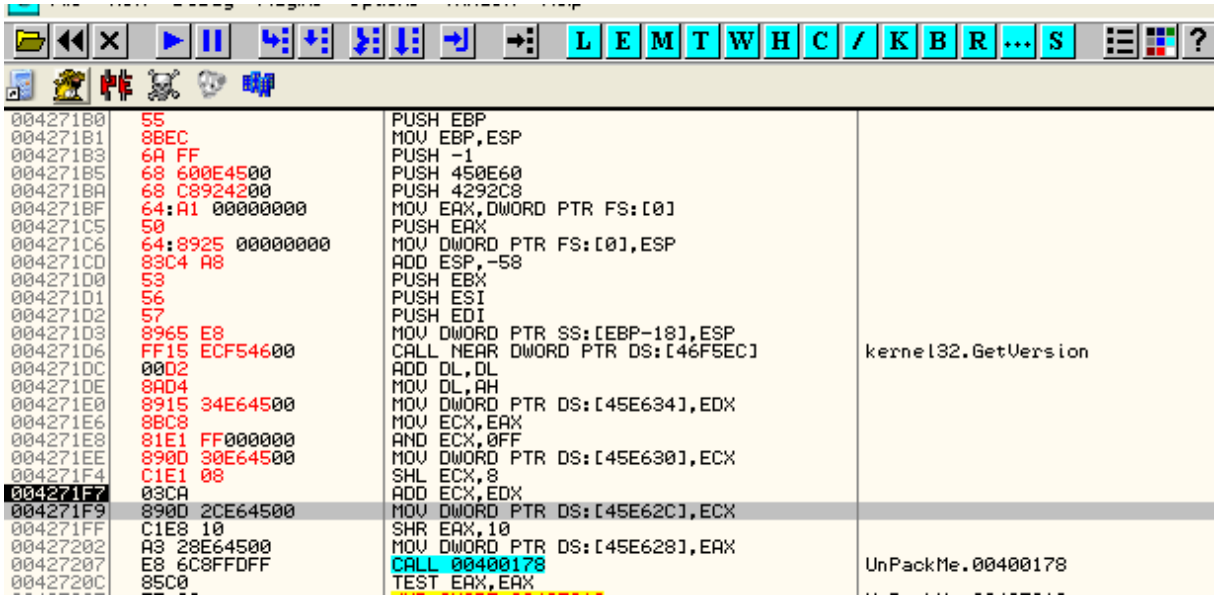


В Таблице верное значение есть, осталось взглянуть на IAT:



IAT тоже восстановлена. Остается всего лишь сделать так, чтобы программа брала значения из IAT'а, а не из Таблицы. Это легко реализуется простым скриптом, который, обнаружив CALL и узнав, какая API-функция им вызывается, ищет ее адрес в IAT'е и производит соответствующую замену, чтобы чтение происходило оттуда, а не из Таблицы.

Перед рассмотрением скрипта следует вернуть украденные байты, найденные нами ранее:



А вот и наш незамысловатый, как табуретка, скрипт:

```
var CALL
var tabla
var api
var iat
var programa
var end
mov programa,401000
mov iat,460818
start:
findop programa, #FF15??#
log $RESULT
mov CALL,$RESULT
cmp CALL,0
je final
cmp CALL,44904b
jae final
sigue:
add CALL,2
log CALL
mov tabla,[CALL]
log tabla
mov api, [tabla]
log api
cmp api,50000000
jb saltar
loop:
cmp api, [iat]
je solucionar
add iat,4
cmp iat,460f28
jae saltar
jmp loop
solucionar:
log iat
log CALL
mov [CALL],iat
cmp CALL,449068
jae final
saltar:
sti
mov eip,4271f7
mov programa,CALL
log programa
mov iat,460818
jmp start
final:
```

```
ret
Это начало:
var CALL
var tabla
var api
var iat
var programa
var end
```

В нем заявляются вспомогательные переменные, необходимые программе; далее мы разберем назначение каждой из них.

```
mov programa,401000
mov iat,460818
```

Это установка начальных значений: переменная programa задает адрес начала секции кода, откуда следует начинать поиск CALL'ов, а переменная iat — адрес начала IAT'а.

```
start:
findop programa, #FF15??#
log $RESULT
```

Это место, откуда начинается выполнение скрипта. Устанавливаем метку start и используем команду findop для поиска первого опкода, начинающегося с FF 15, то есть это будет поиск косвенных CALL'ов.

Здесь также задается ведение лога, хотя он и необязателен — он нужен лишь для учета происходящего в LOG'е OllyDbg, поэтому не будем больше заострять на нем внимание.

```
mov CALL,$RESULT
cmp CALL,0
je final
cmp CALL,44904b
jae final
```

Первый адрес косвенного CALL'а помещается в переменную CALL и происходит его сравнение с нулем. В случае, если ни один косвенный CALL найден не будет, результат окажется нулевым и выполнение скрипта подойдет к концу. Кроме того, проверяется, не восстановление ли это последнего косвенного CALL'а секции кода: если адрес выше заданного, скрипт также завершится.

```
sigue:
add CALL,2
log CALL
mov tabla,[CALL]
log tabla
mov api, [tabla]
log api
cmp api,50000000
jb saltar
```

Здесь адрес CALL'а увеличивается на 2 для указания косвенного адреса, находящегося на два байта ниже (сразу после FF 15).

Затем в переменную tabla помещается адрес элемента Таблицы, а его содержимое —адрес API-функции — в переменнаю api. Этот адрес API-функции и надо будет отыскать в IAT'е.

На всякий случай предусмотрим такую проверку: поскольку сейчас на моем компьютере API-функции расположены выше 50000000, то в случае, когда адрес станет меньше (и это будет точно не API-функция), ничего изменено не будет и программа перейдет к следующему значению.

```
loop:
cmp api, [iat]
je solucionar
add iat,4
cmp iat,460f28
jae saltar
jmp loop
```

Теперь у нас есть значение адреса API-функции, поэтому в данном цикле происходит его поиск по IAT'у. При его нахождении программа переходит к метке solucionar:

```
solucionar:
log iat
log CALL
mov [CALL],iat
```

Здесь элемент IAT'а записывается в положение, на которое указывает CALL, то есть сразу за байтами FF 15.

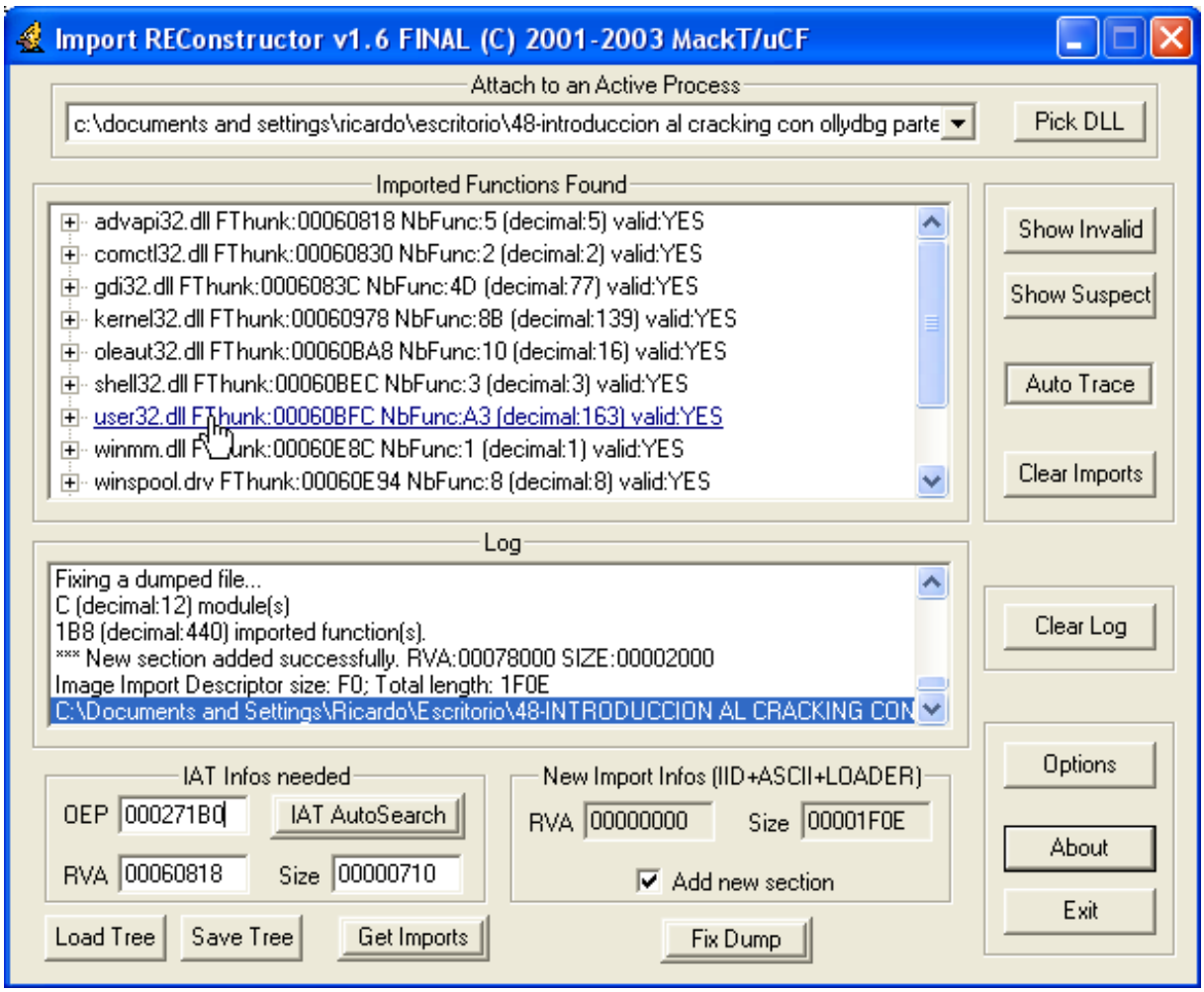
Далее выполнение скрипта повторяется: меняется значение переменной CALL, чтобы поиск происходил для следующего вызова, а также заново устанавливается переменная iat для поиска следующей API-функции.

```
sti
mov eip,4271f7
```

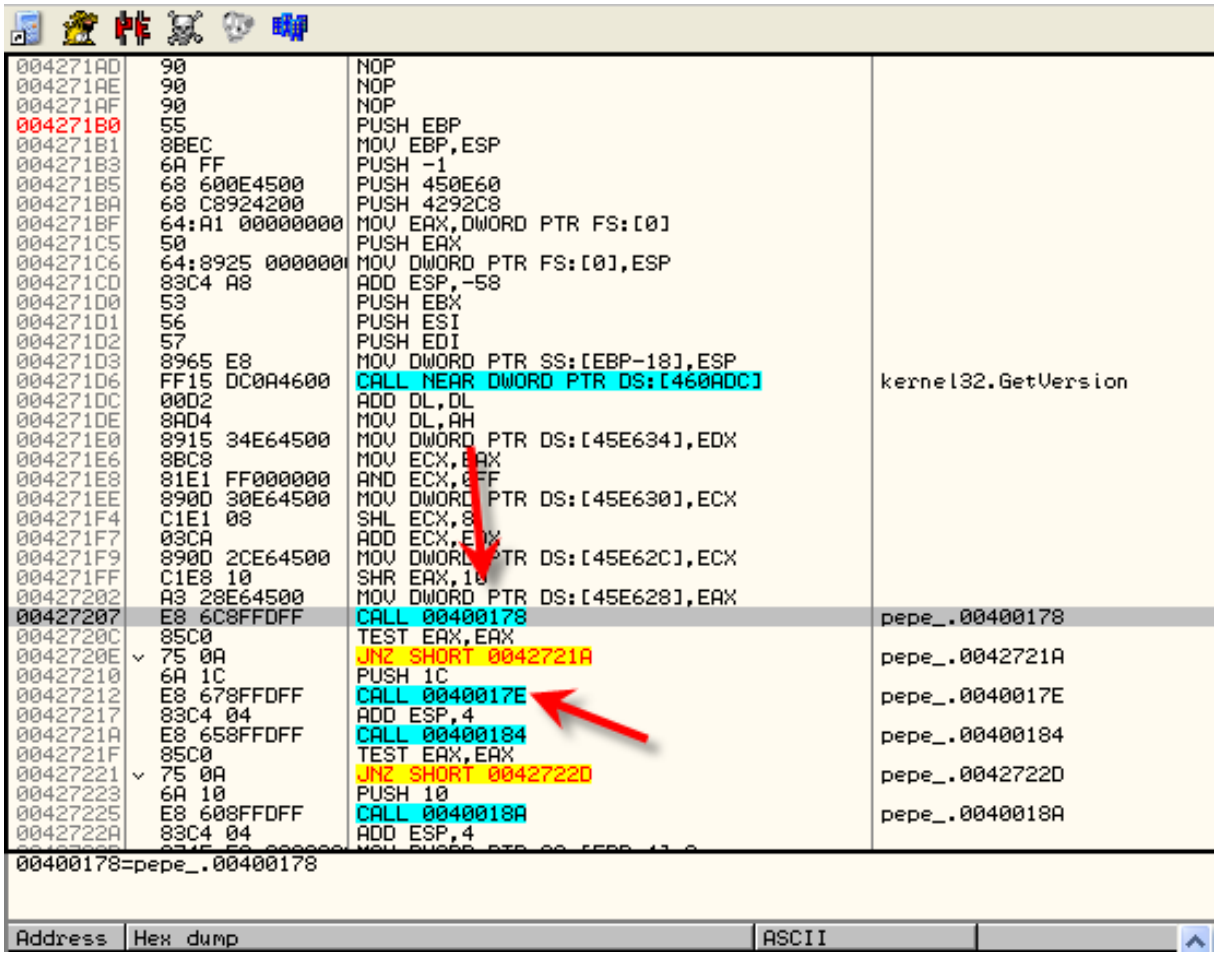
Эти две команды на самом деле не обязательны, но плагину OllyScript жизненно необходимо выполнять какую-либо программу, а не только производить поиск и менять значения, иначе со временем он зависает. Поэтому пришлось добавить команду STI, соответствующую одному нажатию на F7 или шагу вперед, а следом (также для предотвращения зависания) — возврат в ложную OEP.

К сожалению, OllyDbg при изменении скриптом большого количества байтов иногда достает своим MessageBox'ом, сообщающим, что произведено более 1000 патчей, но если несколько раз нажать на OK, скрипт завершится и всё будет восстановлено.

Теперь можно сдампливать и использовать ImpRec для восстановления IAT'а:

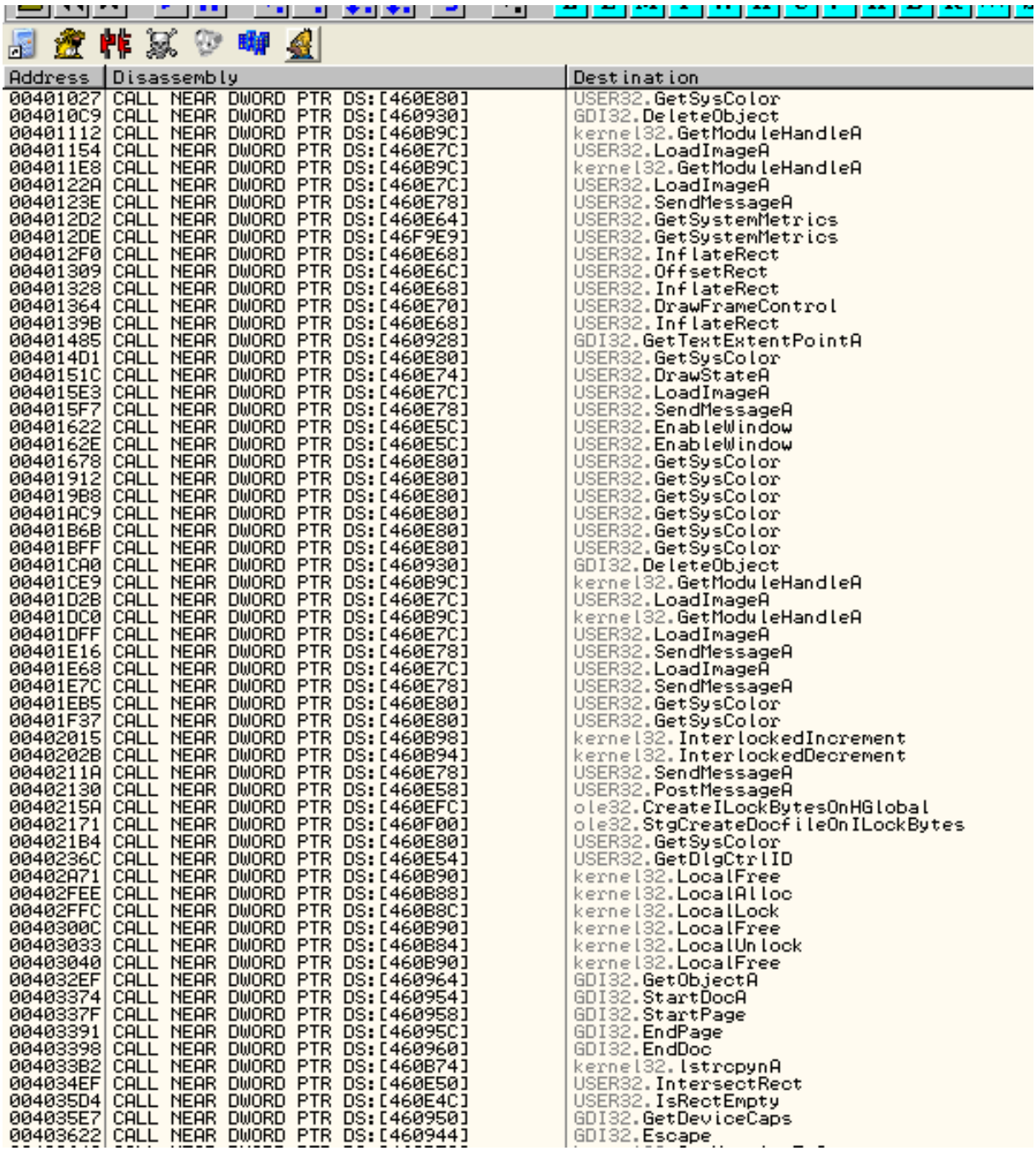


Я обнаружил один мусорный элемент, который удалил с помощью Cut Thunk(s), и запустил исправленный крэкми, но он и теперь не заработал.



Дело в том, что в листинге есть антидампы — переходы в область заголовка, а у дампа соответствующего кода в заголовке нет, так как там у него расположены новые данные заголовка, секций и т. д.

Выполним Search for –> All intermodular calls в исходной программе, остановленной на OEP:

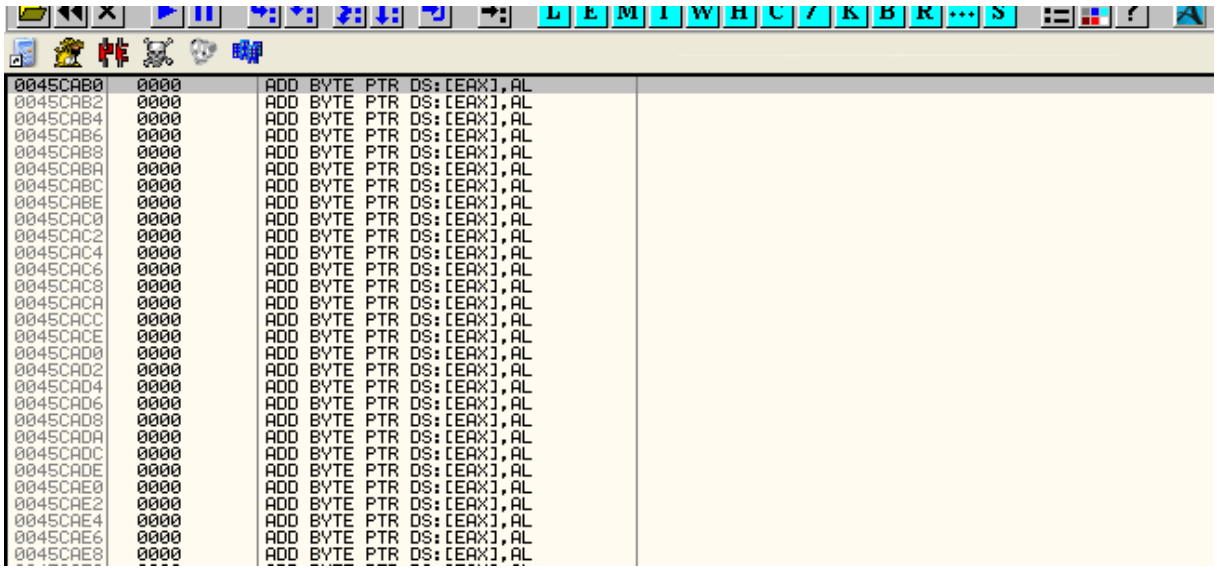


Address	Disassembly	Destination
00401027	CALL NEAR DWORD PTR DS:[460E80]	USER32.GetSysColor
004010C9	CALL NEAR DWORD PTR DS:[460930]	GDI32.DeleteObject
00401112	CALL NEAR DWORD PTR DS:[460B9C]	kernel32.GetModuleHandleA
00401154	CALL NEAR DWORD PTR DS:[460E7C]	USER32.LoadImageA
004011E8	CALL NEAR DWORD PTR DS:[460B9C]	kernel32.GetModuleHandleA
0040122A	CALL NEAR DWORD PTR DS:[460E7C]	USER32.LoadImageA
0040123E	CALL NEAR DWORD PTR DS:[460E78]	USER32.SendMessageA
004012D2	CALL NEAR DWORD PTR DS:[460E64]	USER32.GetSystemMetrics
004012DE	CALL NEAR DWORD PTR DS:[46F9E9]	USER32.GetSystemMetrics
004012F0	CALL NEAR DWORD PTR DS:[460E68]	USER32.InflateRect
00401309	CALL NEAR DWORD PTR DS:[460E6C]	USER32.OffsetRect
00401328	CALL NEAR DWORD PTR DS:[460E68]	USER32.InflateRect
00401364	CALL NEAR DWORD PTR DS:[460E70]	USER32.DrawFrameControl
0040139B	CALL NEAR DWORD PTR DS:[460E68]	USER32.InflateRect
00401485	CALL NEAR DWORD PTR DS:[460928]	GDI32.GetTextExtentPointA
004014D1	CALL NEAR DWORD PTR DS:[460E80]	USER32.GetSysColor
0040151C	CALL NEAR DWORD PTR DS:[460E74]	USER32.DrawStateA
004015E3	CALL NEAR DWORD PTR DS:[460E7C]	USER32.LoadImageA
004015F7	CALL NEAR DWORD PTR DS:[460E78]	USER32.SendMessageA
00401622	CALL NEAR DWORD PTR DS:[460E5C]	USER32.EnableWindow
0040162E	CALL NEAR DWORD PTR DS:[460E5C]	USER32.EnableWindow
00401678	CALL NEAR DWORD PTR DS:[460E80]	USER32.GetSysColor
00401912	CALL NEAR DWORD PTR DS:[460E80]	USER32.GetSysColor
004019B8	CALL NEAR DWORD PTR DS:[460E80]	USER32.GetSysColor
00401AC9	CALL NEAR DWORD PTR DS:[460E80]	USER32.GetSysColor
00401B6B	CALL NEAR DWORD PTR DS:[460E80]	USER32.GetSysColor
00401BFF	CALL NEAR DWORD PTR DS:[460E80]	USER32.GetSysColor
00401CA0	CALL NEAR DWORD PTR DS:[460930]	GDI32.DeleteObject
00401CE9	CALL NEAR DWORD PTR DS:[460B9C]	kernel32.GetModuleHandleA
00401D2B	CALL NEAR DWORD PTR DS:[460E7C]	USER32.LoadImageA
00401DC0	CALL NEAR DWORD PTR DS:[460B9C]	kernel32.GetModuleHandleA
00401DFF	CALL NEAR DWORD PTR DS:[460E7C]	USER32.LoadImageA
00401E16	CALL NEAR DWORD PTR DS:[460E78]	USER32.SendMessageA
00401E68	CALL NEAR DWORD PTR DS:[460E7C]	USER32.LoadImageA
00401E7C	CALL NEAR DWORD PTR DS:[460E78]	USER32.SendMessageA
00401EB5	CALL NEAR DWORD PTR DS:[460E80]	USER32.GetSysColor
00401F37	CALL NEAR DWORD PTR DS:[460E80]	USER32.GetSysColor
00402015	CALL NEAR DWORD PTR DS:[460B98]	kernel32.InterlockedIncrement
0040202B	CALL NEAR DWORD PTR DS:[460B94]	kernel32.InterlockedDecrement
0040211A	CALL NEAR DWORD PTR DS:[460E78]	USER32.SendMessageA
00402130	CALL NEAR DWORD PTR DS:[460E58]	USER32.PostMessageA
0040215A	CALL NEAR DWORD PTR DS:[460EFC]	ole32.CreateILockBytesOnHGlobal
00402171	CALL NEAR DWORD PTR DS:[460F00]	ole32.StgCreateDocfileOnILockBytes
004021B4	CALL NEAR DWORD PTR DS:[460E80]	USER32.GetSysColor
0040236C	CALL NEAR DWORD PTR DS:[460E54]	USER32.GetDlgCtrlID
00402A71	CALL NEAR DWORD PTR DS:[460B90]	kernel32.LocalFree
00402FEE	CALL NEAR DWORD PTR DS:[460B88]	kernel32.LocalAlloc
00402FFC	CALL NEAR DWORD PTR DS:[460B8C]	kernel32.LocalLock
0040300C	CALL NEAR DWORD PTR DS:[460B90]	kernel32.LocalFree
00403033	CALL NEAR DWORD PTR DS:[460B84]	kernel32.LocalUnlock
00403040	CALL NEAR DWORD PTR DS:[460B90]	kernel32.LocalFree
004032EF	CALL NEAR DWORD PTR DS:[460964]	GDI32.GetObjectA
00403374	CALL NEAR DWORD PTR DS:[460954]	GDI32.StartDocA
0040337F	CALL NEAR DWORD PTR DS:[460958]	GDI32.StartPage
00403391	CALL NEAR DWORD PTR DS:[46095C]	GDI32.EndPage
00403398	CALL NEAR DWORD PTR DS:[460960]	GDI32.EndDoc
004033B2	CALL NEAR DWORD PTR DS:[460B74]	kernel32.lstrcpynA
004034EF	CALL NEAR DWORD PTR DS:[460E50]	USER32.IntersectRect
004035D4	CALL NEAR DWORD PTR DS:[460E4C]	USER32.IsRectEmpty
004035E7	CALL NEAR DWORD PTR DS:[460950]	GDI32.GetDeviceCaps
00403622	CALL NEAR DWORD PTR DS:[460944]	GDI32.Escape

Здесь всё исправлено; посмотрим в районе OEP:

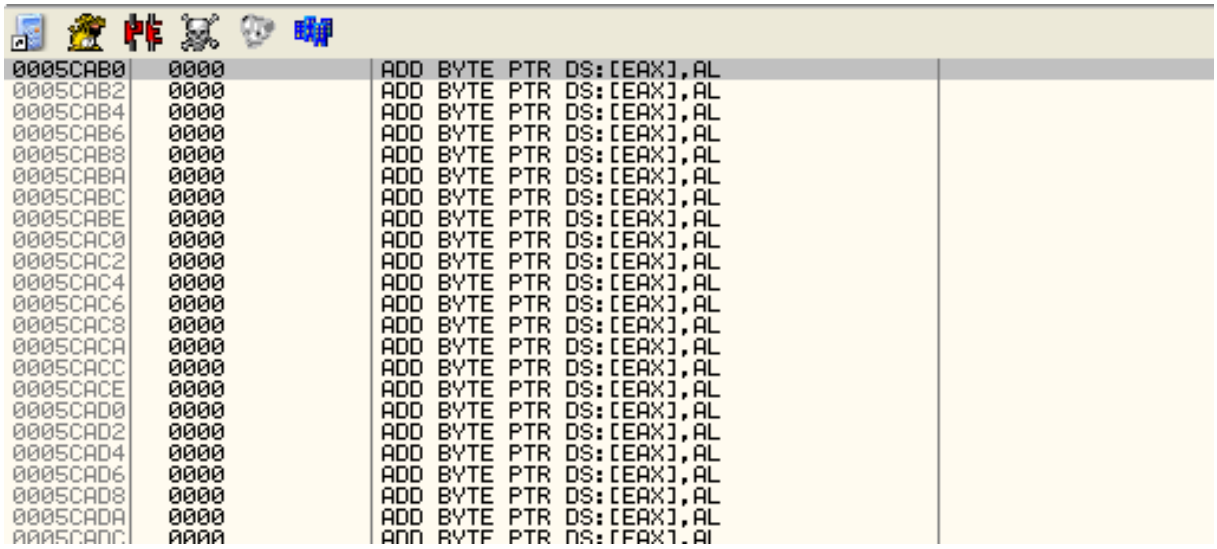
```
0042723E CALL NEAR DWORD PTR DS:[460984]    kernel32.GetCommandLineA
004272D5 CALL NEAR DWORD PTR DS:[460980]    kernel32.GetStartupInfoA
004272F6 CALL NEAR DWORD PTR DS:[460B9C]    kernel32.GetModuleHandleA
00427458 CALL 00400236                      UnPackMe.004001D8
004274F7 CALL 0040023C                      ntdll.RtlUnwind
004277F3 CALL 0040027D                      ntdll.RtlUnwind
00427929 CALL NEAR DWORD PTR DS:[46098C]    kernel32.RaiseException
00427E07 CALL NEAR DWORD PTR DS:[460A84]    kernel32.GetCurrentProcess
00427E0E CALL NEAR DWORD PTR DS:[46F452]    kernel32.TerminateProcess
00427E98 CALL NEAR DWORD PTR DS:[460990]    kernel32.ExitProcess
00428029 CALL NEAR DWORD PTR DS:[4609F8]    ntdll.RtlReAllocateHeap
00428042 CALL 004003ED                      UnPackMe.004001D8
004280BB CALL NEAR DWORD PTR DS:[460A08]    kernel32.GetCPInfo
00428805 CALL 00427360                      UnPackMe.004001D8
00428823 CALL NEAR DWORD PTR DS:[460A54]    kernel32.InitializeCriticalSection
0042884D CALL NEAR DWORD PTR DS:[460A3C]    ntdll.RtlEnterCriticalSection
0042886E CALL NEAR DWORD PTR DS:[460A44]    ntdll.RtlLeaveCriticalSection
004288AE CALL NEAR DWORD PTR DS:[460A3C]    ntdll.RtlEnterCriticalSection
004288E0 CALL NEAR DWORD PTR DS:[460A3C]    ntdll.RtlEnterCriticalSection
0042891E CALL NEAR DWORD PTR DS:[460A44]    ntdll.RtlLeaveCriticalSection
00428950 CALL NEAR DWORD PTR DS:[460A44]    ntdll.RtlLeaveCriticalSection
00428983 CALL NEAR DWORD PTR DS:[460B98]    kernel32.InterlockedIncrement
00428C15 CALL NEAR DWORD PTR DS:[460B9C]    kernel32.GetModuleHandleA
00428C25 CALL NEAR DWORD PTR DS:[460B40]    kernel32.GetProcAddress
004293A9 CALL NEAR DWORD PTR DS:[4609A0]    kernel32.HeapCreate
004293C8 CALL NEAR DWORD PTR DS:[46099C]    kernel32.HeapDestroy
00429401 CALL NEAR DWORD PTR DS:[4609FC]    ntdll.RtlAllocateHeap
0042951F CALL NEAR DWORD PTR DS:[4609A4]    kernel32.VirtualFree
00429536 CALL NEAR DWORD PTR DS:[460A00]    ntdll.RtlFreeHeap
00429560 CALL NEAR DWORD PTR DS:[4609A4]    kernel32.VirtualFree
00429597 CALL NEAR DWORD PTR DS:[460A00]    ntdll.RtlFreeHeap
004295E9 CALL NEAR DWORD PTR DS:[4609A4]    kernel32.VirtualFree
00429892 CALL NEAR DWORD PTR DS:[4609A8]    kernel32.VirtualAlloc
00429CBD CALL NEAR DWORD PTR DS:[460A58]    kernel32.GetFullPathNameA
00429CCF CALL NEAR DWORD PTR DS:[460A20]    kernel32.GetCurrentDirectoryA
00429DA8 CALL NEAR DWORD PTR DS:[4609AC]    kernel32.GetDriveTypeA
00429DDE CALL 00427360                      UnPackMe.004001D8
00429E2A CALL NEAR DWORD PTR DS:[460980]    kernel32.GetStartupInfoA
00429F00 CALL NEAR DWORD PTR DS:[46097C]    kernel32.GetFileType
00429F79 CALL NEAR DWORD PTR DS:[46097C]    kernel32.GetFileType
00429FBE CALL NEAR DWORD PTR DS:[460B80]    kernel32.SetHandleCount
0042A1F6 CALL NEAR DWORD PTR DS:[460A50]    kernel32.TlsAlloc
0042A21F CALL NEAR DWORD PTR DS:[460A38]    kernel32.TlsSetValue
0042A232 CALL NEAR DWORD PTR DS:[460ACC]    kernel32.GetCurrentThreadId
0042A272 CALL NEAR DWORD PTR DS:[460B5C]    ntdll.RtlGetLastWin32Error
0042A280 CALL NEAR DWORD PTR DS:[460A30]    kernel32.TlsGetValue
0042A2A6 CALL NEAR DWORD PTR DS:[460A38]    kernel32.TlsSetValue
0042A2B9 CALL NEAR DWORD PTR DS:[460ACC]    kernel32.GetCurrentThreadId
0042A2C9 CALL NEAR DWORD PTR DS:[460B60]    ntdll.RtlSetLastWin32Error
0042A2D6 CALL 00427360                      UnPackMe.004001D8
0042A2DF CALL NEAR DWORD PTR DS:[460B60]    ntdll.RtlSetLastWin32Error
0042AF92 CALL NEAR DWORD PTR DS:[460A6C]    kernel32.CloseHandle
```

Вот несколько вызовов, требующих восстановления, однако особого желания заморачиваться со вторым скриптом нет. Поскольку для успешного запуска дампа мы не можем напостоянно сохранить так много информации в заголовке, то сделаем следующее: поищем в дампе область памяти размером примерно 1000h байтов, в которую можно было бы писать, например, эту:

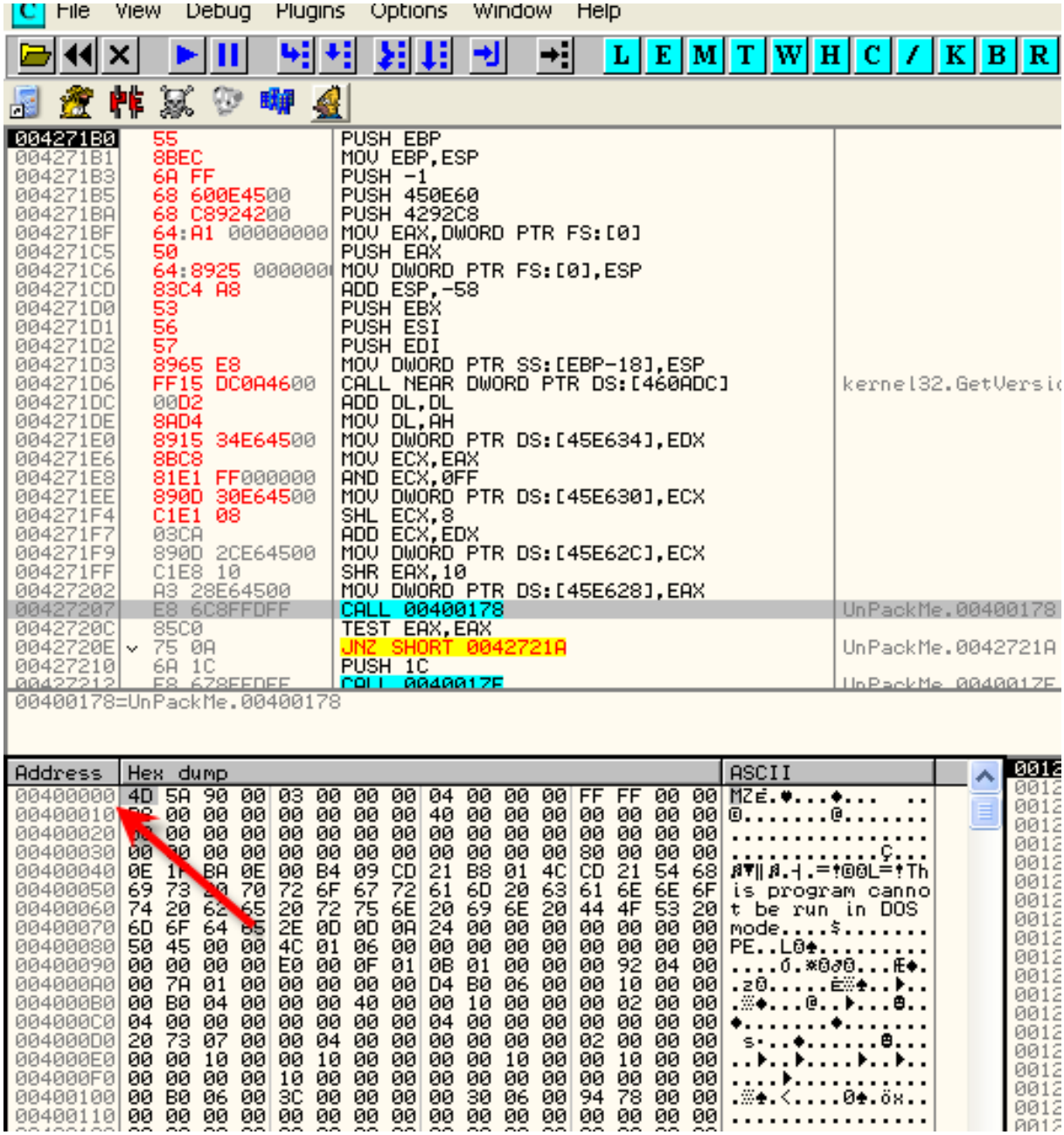


```
0045CAB0  0000  ADD BYTE PTR DS:[EAX],AL
0045CAB2  0000  ADD BYTE PTR DS:[EAX],AL
0045CAB4  0000  ADD BYTE PTR DS:[EAX],AL
0045CAB6  0000  ADD BYTE PTR DS:[EAX],AL
0045CAB8  0000  ADD BYTE PTR DS:[EAX],AL
0045CABA  0000  ADD BYTE PTR DS:[EAX],AL
0045CABC  0000  ADD BYTE PTR DS:[EAX],AL
0045CABE  0000  ADD BYTE PTR DS:[EAX],AL
0045CAC0  0000  ADD BYTE PTR DS:[EAX],AL
0045CAC2  0000  ADD BYTE PTR DS:[EAX],AL
0045CAC4  0000  ADD BYTE PTR DS:[EAX],AL
0045CAC6  0000  ADD BYTE PTR DS:[EAX],AL
0045CAC8  0000  ADD BYTE PTR DS:[EAX],AL
0045CACA  0000  ADD BYTE PTR DS:[EAX],AL
0045CACC  0000  ADD BYTE PTR DS:[EAX],AL
0045CACE  0000  ADD BYTE PTR DS:[EAX],AL
0045CAD0  0000  ADD BYTE PTR DS:[EAX],AL
0045CAD2  0000  ADD BYTE PTR DS:[EAX],AL
0045CAD4  0000  ADD BYTE PTR DS:[EAX],AL
0045CAD6  0000  ADD BYTE PTR DS:[EAX],AL
0045CAD8  0000  ADD BYTE PTR DS:[EAX],AL
0045CADA  0000  ADD BYTE PTR DS:[EAX],AL
0045CADC  0000  ADD BYTE PTR DS:[EAX],AL
0045CADE  0000  ADD BYTE PTR DS:[EAX],AL
0045CAE0  0000  ADD BYTE PTR DS:[EAX],AL
0045CAE2  0000  ADD BYTE PTR DS:[EAX],AL
0045CAE4  0000  ADD BYTE PTR DS:[EAX],AL
0045CAE6  0000  ADD BYTE PTR DS:[EAX],AL
0045CAE8  0000  ADD BYTE PTR DS:[EAX],AL
```

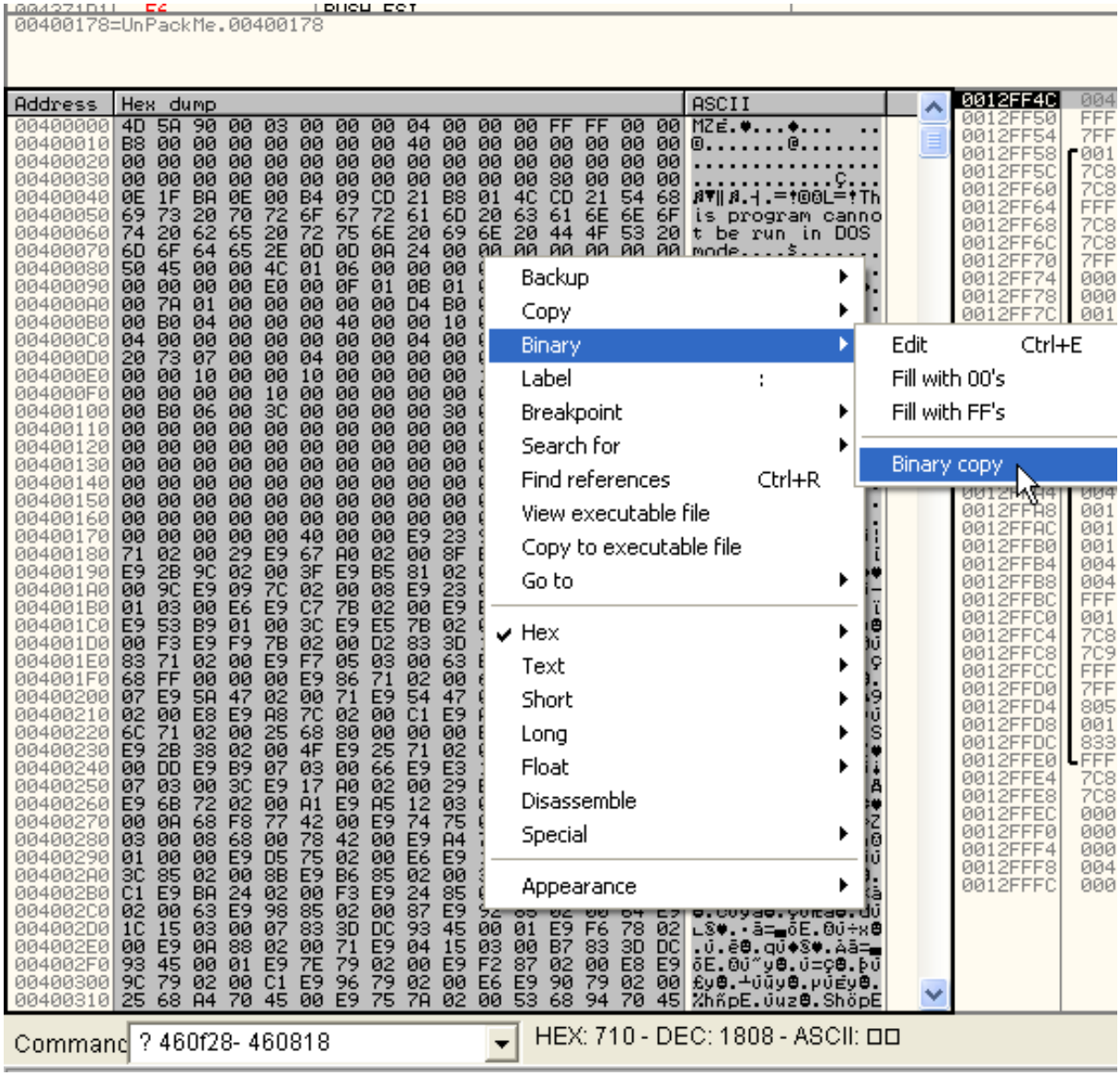
Убедимся, что она существует на самом деле и не принадлежит ни какой виртуальной секции, выбрав в контекстном меню View –> Executable file:



Теперь скопируем туда 1000h байтов заголовка исходной программы, остановленной на OEP'е:



Выделим всю эту секцию:

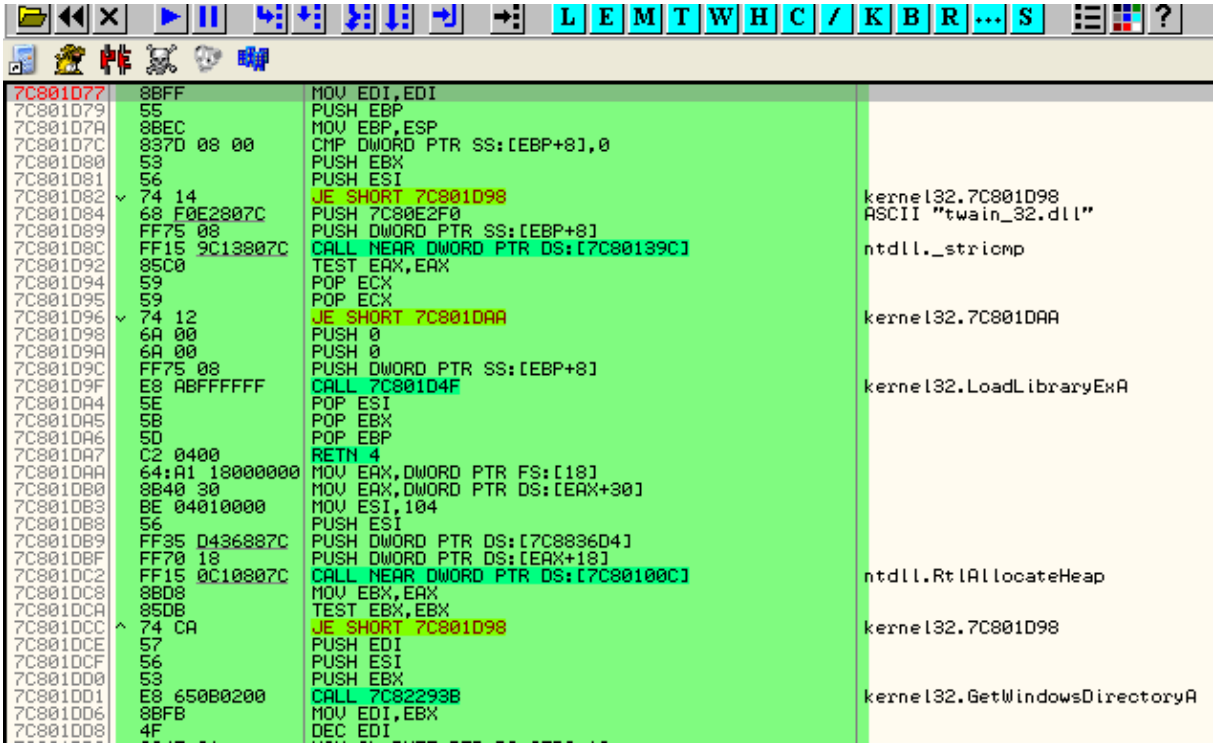


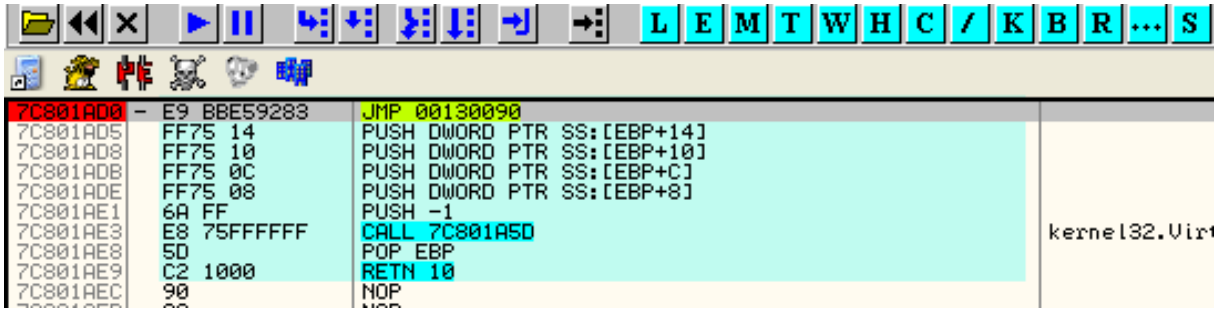
Скопировав ее, сделаем в выбранной области дампа Binary paste и сохраним изменения:

```
0045CAA2   0000          ADD BYTE PTR DS:[EAX],AL
0045CAA4   0000          ADD BYTE PTR DS:[EAX],AL
0045CAA6   0000          ADD BYTE PTR DS:[EAX],AL
0045CAA8   0000          ADD BYTE PTR DS:[EAX],AL
0045CAAA   0000          ADD BYTE PTR DS:[EAX],AL
0045CAAC   0000          ADD BYTE PTR DS:[EAX],AL
0045CAAE   0000          ADD BYTE PTR DS:[EAX],AL
0045CAB0   4D            DEC EBP
0045CAB1   5A            POP EDX
0045CAB2   90            NOP
0045CAB3   0003          ADD BYTE PTR DS:[EBX],AL
0045CAB5   0000          ADD BYTE PTR DS:[EAX],AL
0045CAB7   0004400       ADD BYTE PTR DS:[EAX+EAX],AL
0045CABA   0000          ADD BYTE PTR DS:[EAX],AL
0045CABC   FFFF          ???                                  Unknown command
0045CABE   0000          ADD BYTE PTR DS:[EAX],AL
0045CAC0   B8 00000000   MOV EAX,0
0045CAC5   0000          ADD BYTE PTR DS:[EAX],AL
0045CAC7   0040 00       ADD BYTE PTR DS:[EAX],AL
0045CACA   0000          ADD BYTE PTR DS:[EAX],AL
0045CACC   0000          ADD BYTE PTR DS:[EAX],AL
0045CACE   0000          ADD BYTE PTR DS:[EAX],AL
0045CAD0   0000          ADD BYTE PTR DS:[EAX],AL
0045CAD2   0000          ADD BYTE PTR DS:[EAX],AL
0045CAD4   0000          ADD BYTE PTR DS:[EAX],AL
0045CAD6   0000          ADD BYTE PTR DS:[EAX],AL
0045CAD8   0000          ADD BYTE PTR DS:[EAX],AL
0045CADA   0000          ADD BYTE PTR DS:[EAX],AL
0045CADC   0000          ADD BYTE PTR DS:[EAX],AL
0045CADE   0000          ADD BYTE PTR DS:[EAX],AL
0045CAE0   0000          ADD BYTE PTR DS:[EAX],AL
0045CAE2   0000          ADD BYTE PTR DS:[EAX],AL
0045CAE4   0000          ADD BYTE PTR DS:[EAX],AL
```

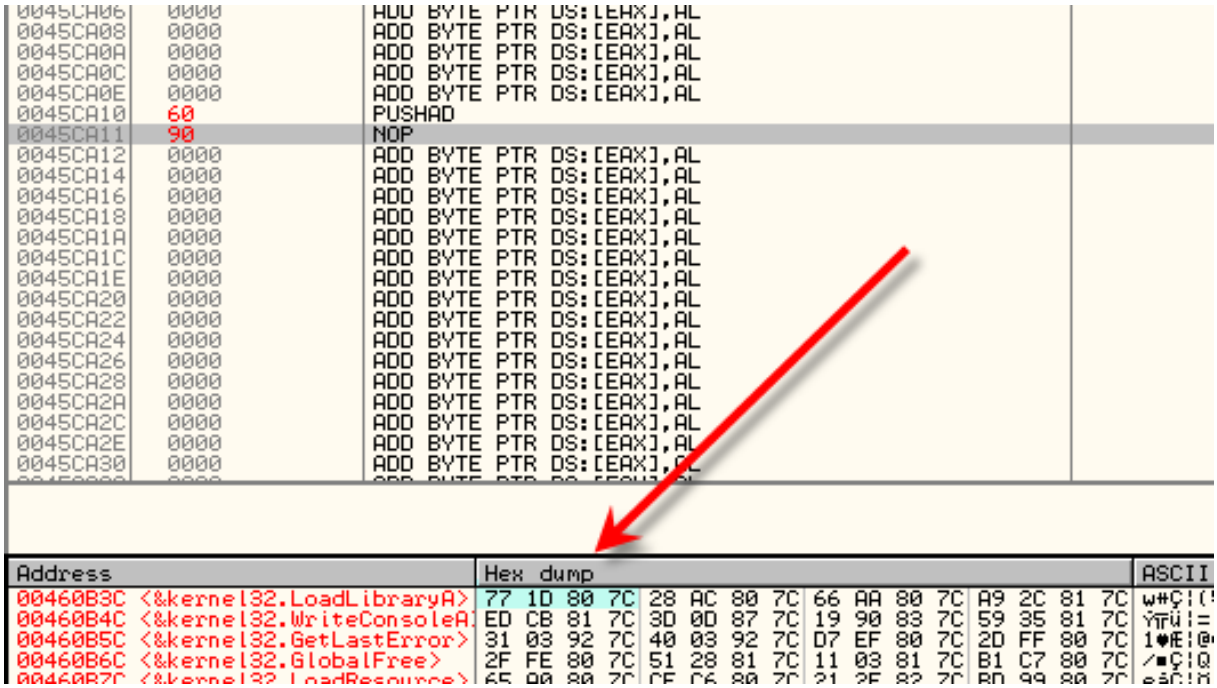
Теперь осталось чуть выше нашей области разместить вставку, которая перед запуском программы копировала бы эти байты в заголовок. Но она будет не так проста, как хотелось бы, поскольку IAT крэкми не содержит адреса API-функции VirtualProtect.

По идее, чтобы достичь совместимости со старыми системами, нам нужно использовать API-функции LoadLibrary и GetProcAddress, и, кроме того, найти адрес VirtualProtect, но я покажу вам более быстрое решение. Посмотрите на это:

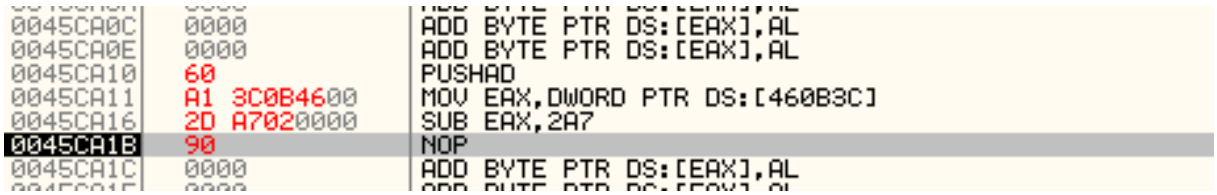




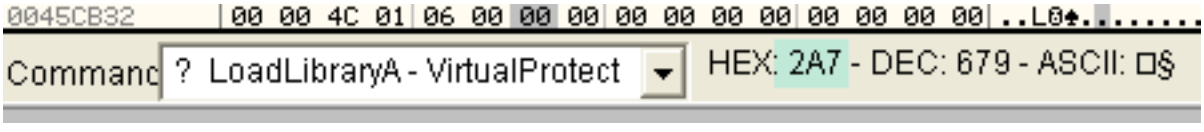
На скриншоте зеленым выделена API-функция LoadLibraryA, которая есть в IAT'е, а лазурным — нужная, но отсутствующая там VirtualProtect. Мы просто возьмем адрес LoadLibrary и вычтем из него нужное количество байт, чтобы дойти до VirtualProtect, поскольку во всей XP соблюдается дистанцирование.



Это элемент IAT'а, относящийся к LoadLibrary.



Загрузим его содержимое в EAX. Давайте посмотрим, каково расстояние между нашими API-функциями:



То есть, если вычесть 2A7 из адреса LoadLibraryA, то попадем на VirtualProtect. Проверим это:

```
0045CA0C   0000            ADD BYTE PTR DS:[EAX],AL
0045CA0E   0000            ADD BYTE PTR DS:[EAX],AL
0045CA10   60              PUSHAD
0045CA11   A1 3C0B4600     MOV EAX,DWORD PTR DS:[460B3C]
0045CA16   2D A7020000     SUB EAX,2A7
0045CA1B   90              NOP
0045CA1C   0000            ADD BYTE PTR DS:[EAX],AL
```

```
Registers (3DNow!)
EAX 7C801AD0 kernel32.VirtualProtect
ECX 0012FFB0
EDX 7C91EB94 ntdll.KiFastSystemCallRet
EBX 7FFDF000
ESP 0012FF84
EBP 0012FFF0
ESI FFFFFFFF
EDI 7C920738 ntdll.7C920738
EIP 0045CA1B pepe_.0045CA1B
```

Поскольку адрес одной API-функции находится через другую, расположенную в IAT'е, осталось изменить настройки доступа к заголовку, чтобы в него можно было писать.

VirtualProtect Quick Info

The **VirtualProtect** function changes the access protection on a region of committed pages in the virtual address space of the calling process. This function differs from **VirtualProtectEx**, which changes the access protection of any process.

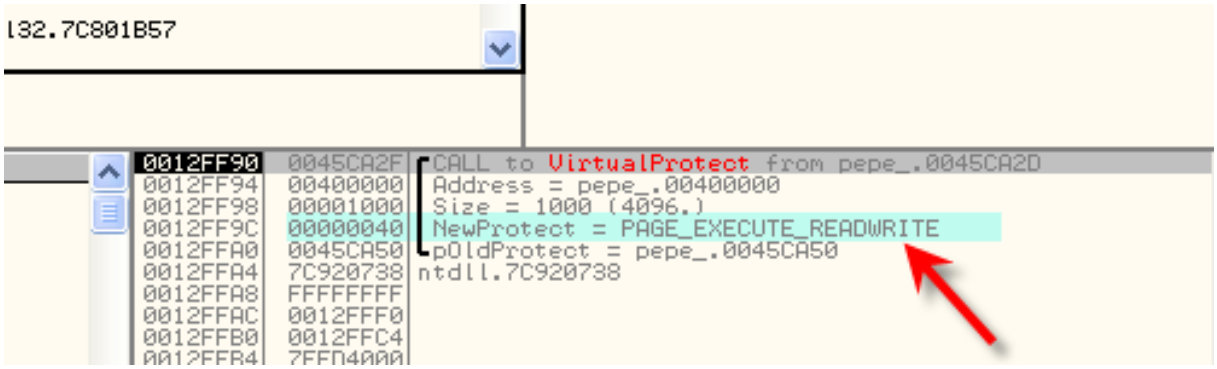
```
BOOL VirtualProtect(
  LPVOID lpAddress,        // address of region of committed pages
  DWORD dwSize,            // size of the region
  DWORD flNewProtect,      // desired access protection
  PDWORD lpflOldProtect    // address of variable to get old protection
);
```

Это описание и параметры API-функции, которая будет нами применена.

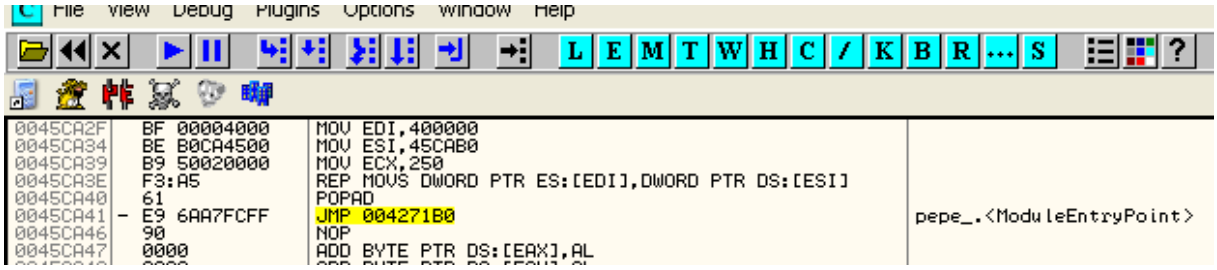
```
File  View  Debug  Plugins  Options  Window  Help
```

```
0045CA10   60              PUSHAD
0045CA11   A1 3C0B4600     MOV EAX,DWORD PTR DS:[460B3C]
0045CA16   2D A7020000     SUB EAX,2A7
0045CA1B   90              NOP
0045CA1C   68 50CA4500     PUSH 45CA50
0045CA21   6A 40           PUSH 40
0045CA23   68 00100000     PUSH 1000
0045CA28   68 00004000     PUSH 400000
0045CA2D   FFD0            CALL NEAR EAX
0045CA2F   BF 00004000     MOV EDI,400000
0045CA34   BE B0CA4500     MOV ESI,45CAB0
0045CA39   B9 50020000     MOV ECX,250
0045CA3E   F3:A5           REP MOVS DWORD PTR ES:[EDI],DWORD PTR DS:[ESI]
0045CA40   61              POPAD
0045CA41 - E9 6AA7FCFF     JMP 004271B0                    pepe_.<ModuleEntryPoint>
0045CA46   90              NOP
0045CA47   0000            ADD BYTE PTR DS:[EAX],AL
```

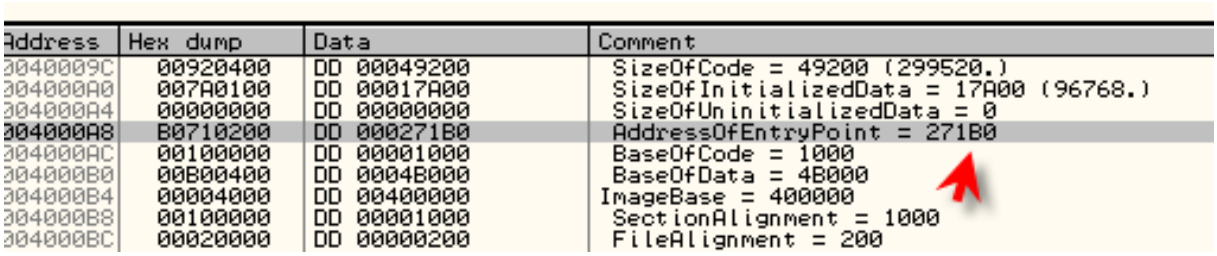
А это — код вставки полностью: сначала выполняется PUSHAD, потом в регистр EAX помещается адрес LoadLibrary, из которого вычитается 2A7 для получения адреса VirtualProtect. Затем, для изменения настроек доступа к заголовку (чтобы в него можно было писать), данная API-функция вызывается с такими параметрами:



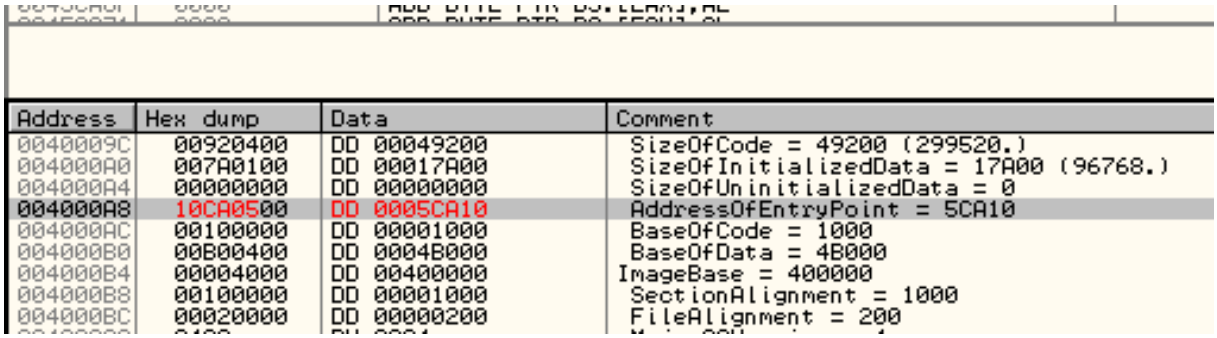
Здесь видно, что область 1000h байтов, начинающаяся в 400000, получает разрешение на запись. Потом байты заголовка исходной программы, сохраненные в 45CAB0, копируются в 400000. И, наконец, после команды POPAD происходит переход на EP.



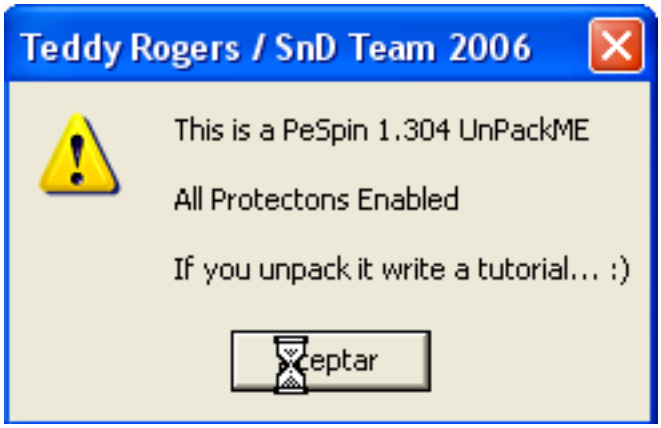
Всё, что нам осталось — это изменить EP в заголовке дампа, чтобы программа запускалась непосредственно с нашей вставки.



Заменим значение поля AddresOfEntryPoint на относительный адрес начала вставки 5CA10 (VA 45CA10).



Сохраним все изменения, и… вуаля:



Выполняется так, что аж ветер свистит, хе-хе.

До встречи в 50-й главе!

Исходник (находится в архиве `wasm_files.zip`: `files/0524/ollydbg49_spin.txt`)

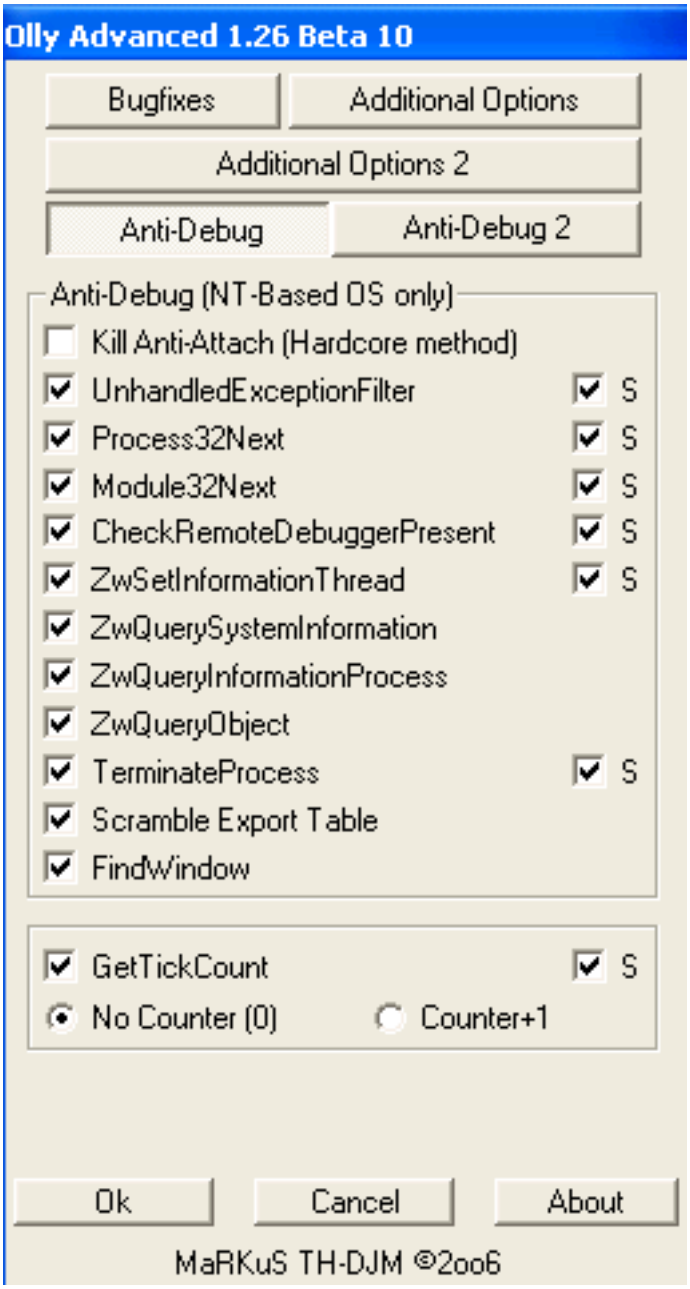
Введение в крэкинг с нуля, используя OllyDbg - Глава 50

Автор: Рикардо Нарваха, пер. Рома Стремилов

Дата: 2010-04-29

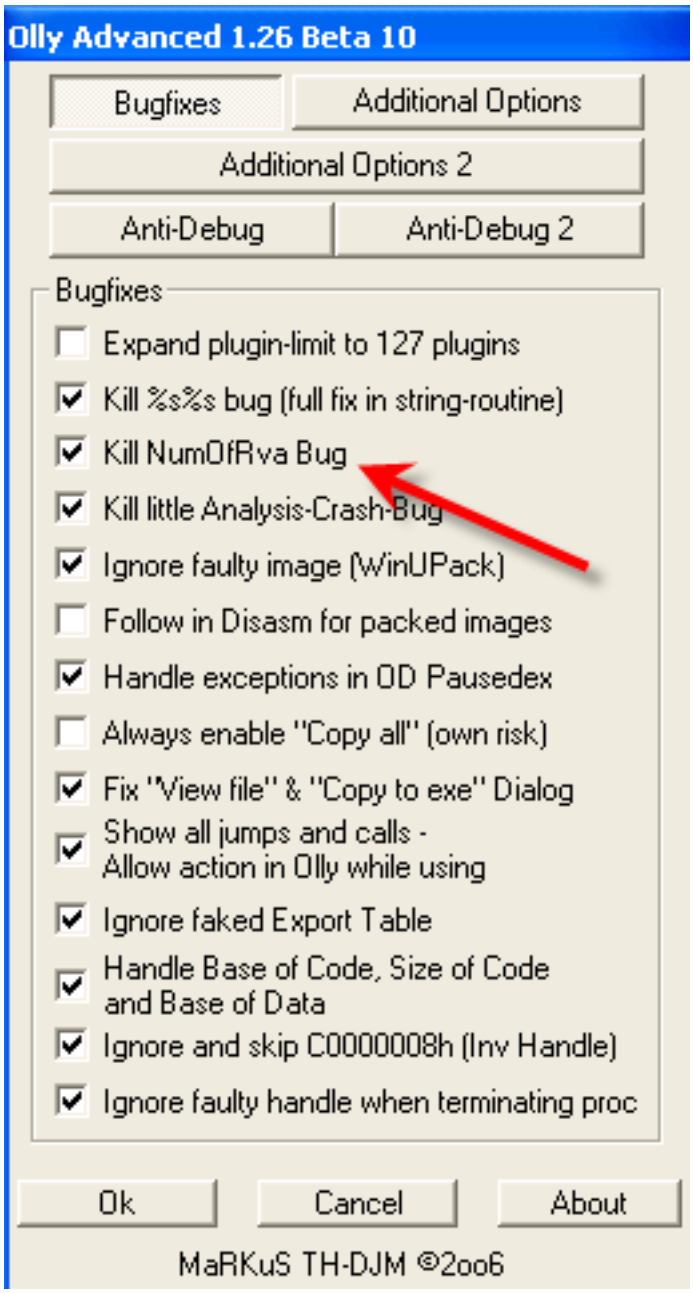
После рассмотрения двуличных Patrick'a и PeSpin эта глава может показаться простой, даже несмотря на то, что данный упаковщик не запускается ни в одной из версий OllyDbg, защищенных всеми плагинами, в том числе и наиболее эффективным из них — Olly Advanced. Но запустить его в OllyDbg, на самом деле, совсем не трудно, и мы вскоре этого добьемся, чтобы у нас осталось больше времени на отдых, хе-хе. Исследовать мы будем приложенный к данному туториалу файл UnPackMe_ReCrypt0.80.exe.

Что ж, попытаемся запустить подопытного в OllyDbg; пользоваться будем версией отладчика, предназначенной для поиска OEP'ов.

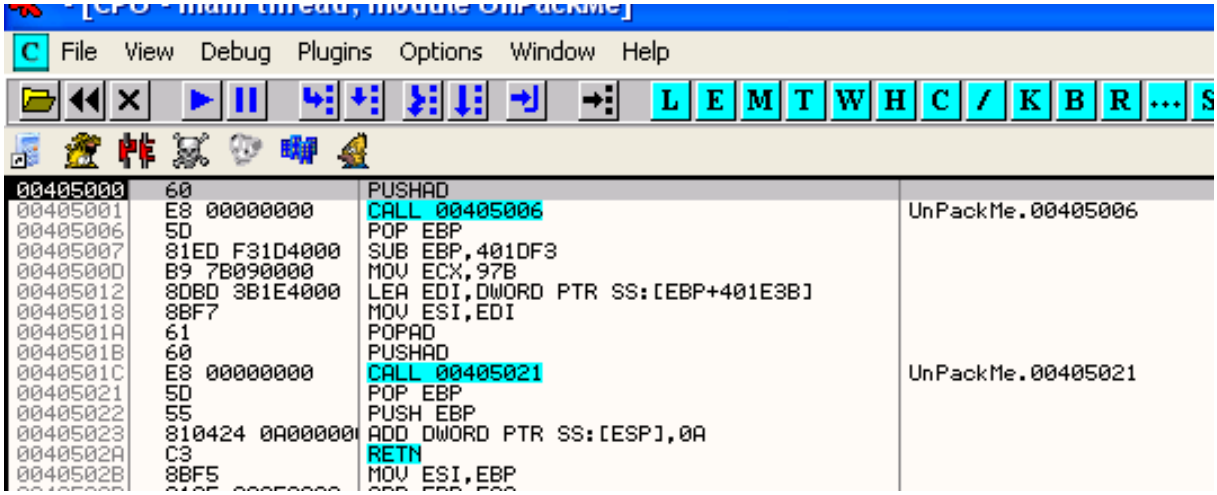


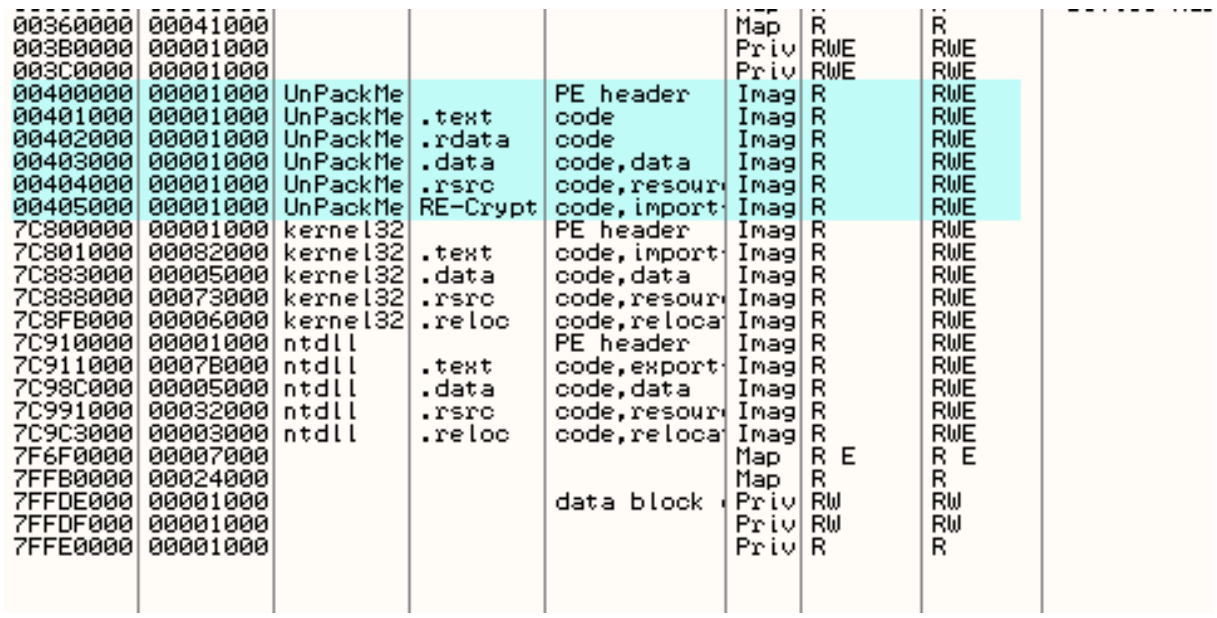
Можете поэкспериментировать с конфигурацией плагина. Конечно, он находится на стадии бета-тестирования и у него тоже есть ошибки, но он всё же выполняет свои задачи, хотя наш упаковщик и не запускается ни с одним из опробованных мной средств сокрытия.

Но у данного плагина есть еще и опция исправления бага секций, из-за которого при неправильнм значении заголовочного поля NumberOfRva показывается одна-единственная секция:

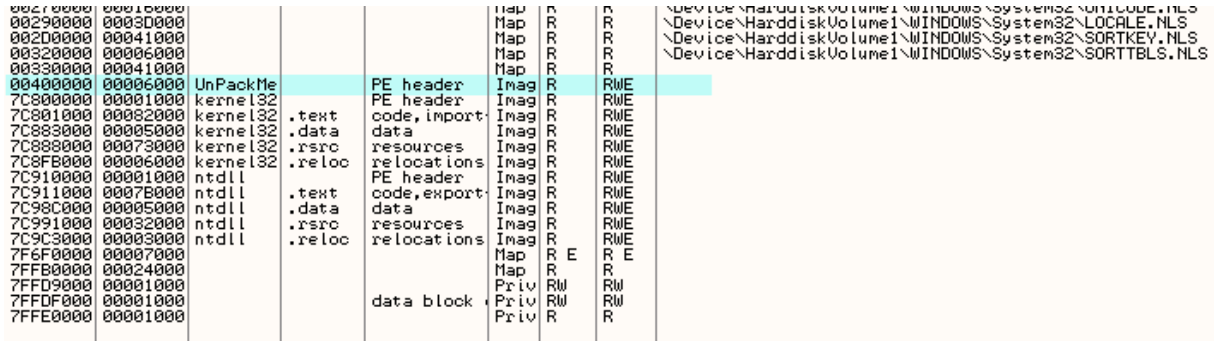


Хорошо, запустив программу, мы сэкономили время; посмотрим, где произошла остановка:



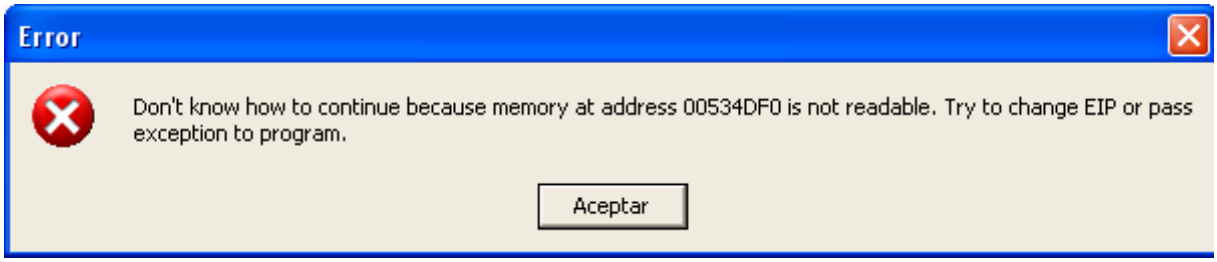


Теперь секции в карте памяти отображаются корректно, хотя без содействия плагина Olly Advanced это выглядело бы так:



И нам было бы сложно дойти до OEP, поскольку это первая секция. Конечно, мы могли бы открыть PE Editor, посмотреть размер первой секции и вручную установить BPM на все соответствующие байты в DUMP’е, но наш плагин всё упростил, хе-хе.

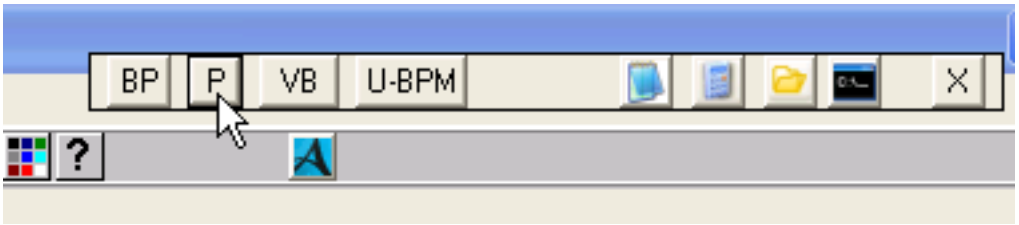
Нажмем RUN:



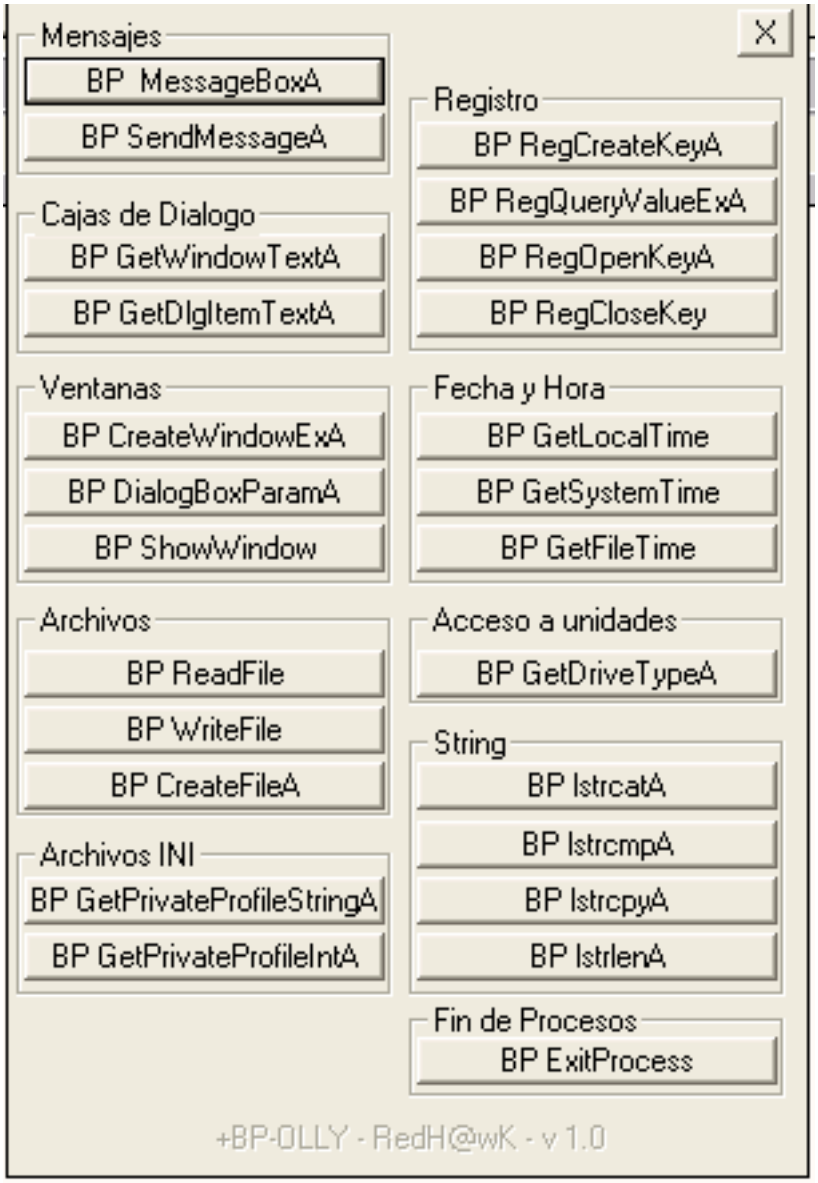
Привет и чау! И это в сверхзамаскированном OllyDbg! Разберемся, что же произошло; посмотрим лог, ответив на сообщение и подождав немного, если это потребуется:

```
00400000 Unload C:\Documents and Settings\Ricardo\Escritorio\ReCrypt 0.80\UnPackMe_ReCrypt0.80.exe
7C800000 Unload C:\WINDOWS\system32\kernel32.dll
7C910000 Unload C:\WINDOWS\system32\ntdll.dll
         Process terminated

         New process with ID 00000558 created
00405000 Main thread with ID 00000298 created
00400000 Module C:\Documents and Settings\Ricardo\Escritorio\ReCrypt 0.80\UnPackMe_ReCrypt0.80.exe
7C800000 Module C:\WINDOWS\system32\kernel32.dll
7C910000 Module C:\WINDOWS\system32\ntdll.dll
00405000 Program entry point
7C81EB33 Debug string:
00534DF0 Access violation when executing [00534DF0]
```

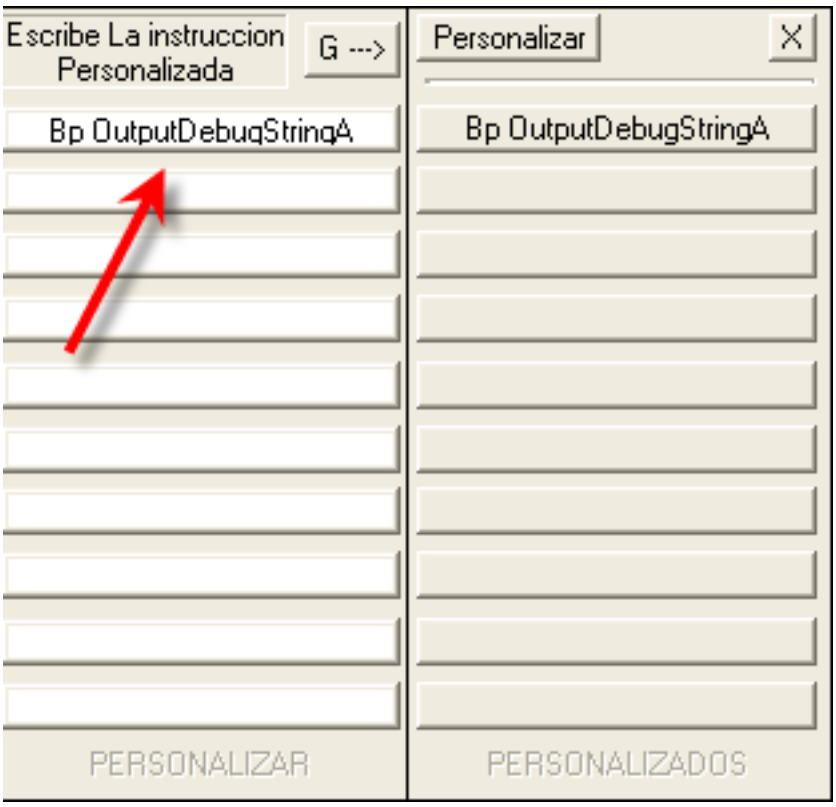
После запуска программы вызывается API-функция OutputDebugStringA, обычно использующаяся для подвешивания OllyDbg длинной строкой типа %$%$%$%$%$%$. Хотя Olly Advanced и другие плагины предоставляют защиту от подобных трюков, анпэкми всё же не запускается. Вызов данной API-функции генерирует исключение и программа не может выполняться дальше. Посмотрим, что произойдет, если на OutputDebugStringA установить BP:



Поскольку мне не хочется каждый раз вводить вручную имена API-функций, то я пользуюсь второй версией плагина +BP-OLLY моего друга Red'a, дающего возможность простым кликом устанавливать BP на наиболее часто используемые API-функции (в т. ч. и для VB), а также добавлять то, что нужно именно нам.



Поскольку в этом окне нужной нам API-функции нет, то добавим ее в список собственных, нажав на кнопку P (Personalizar):



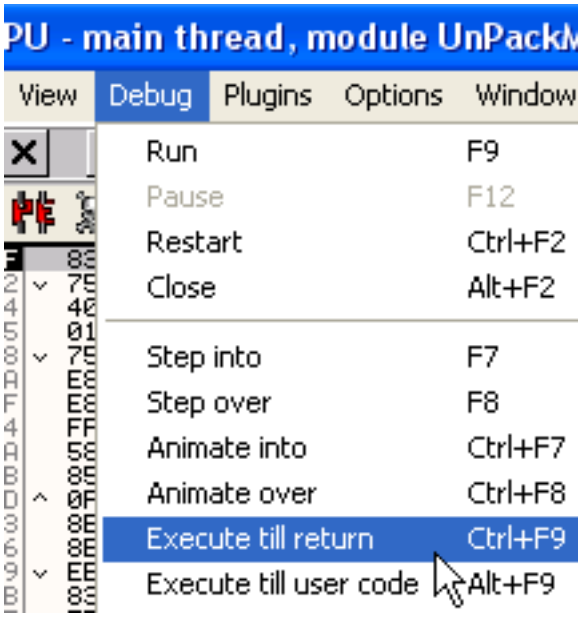
Обратите внимание, что при вводе имени функции следует соблюдать регистр букв. После нажатия на кнопку G при следующем открытии этого окна в нем появится кнопка с только что введенным именем:

Теперь соответствующий BP можно будет уставливать простым кликом, и нам больше никогда не придется вручную вводить имя этой API-функции для установки на нее простого брейка.

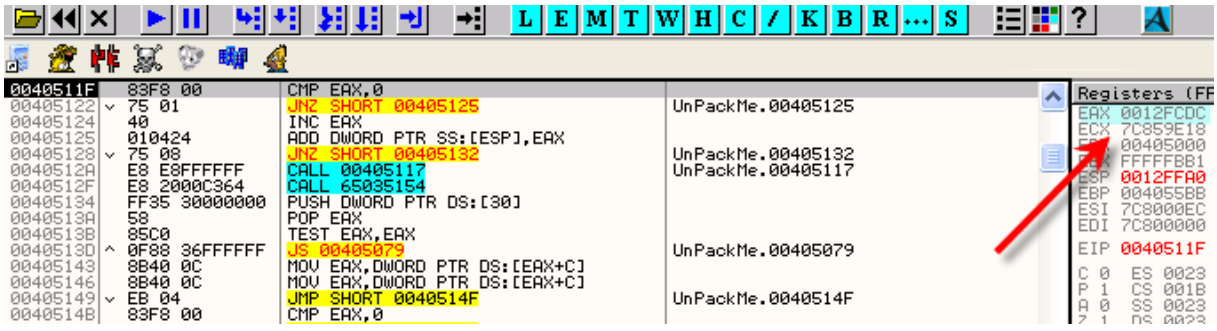
Однако запуск программы приводит всё к той же серьезной ошибке, поэтому простые BP на этой API-функции в данном упаковщике не подойдут. После перезагрузки анпэкми установим в Command Bar'е соответствующий HE и посмотрим, что это даст.

После удаления BP нажмем RUN:

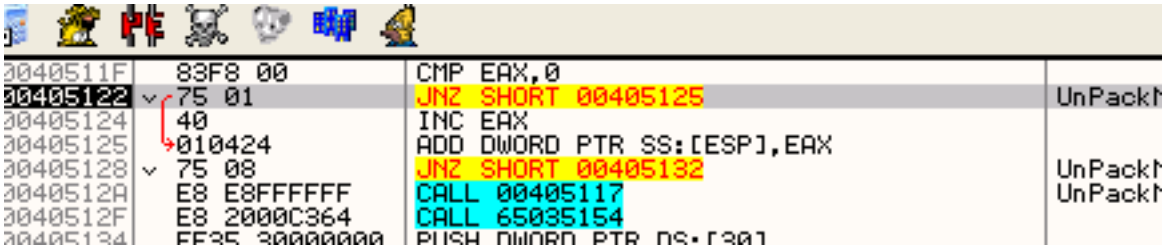
Брейк сработал. Дойдем до команды RET API-функции:



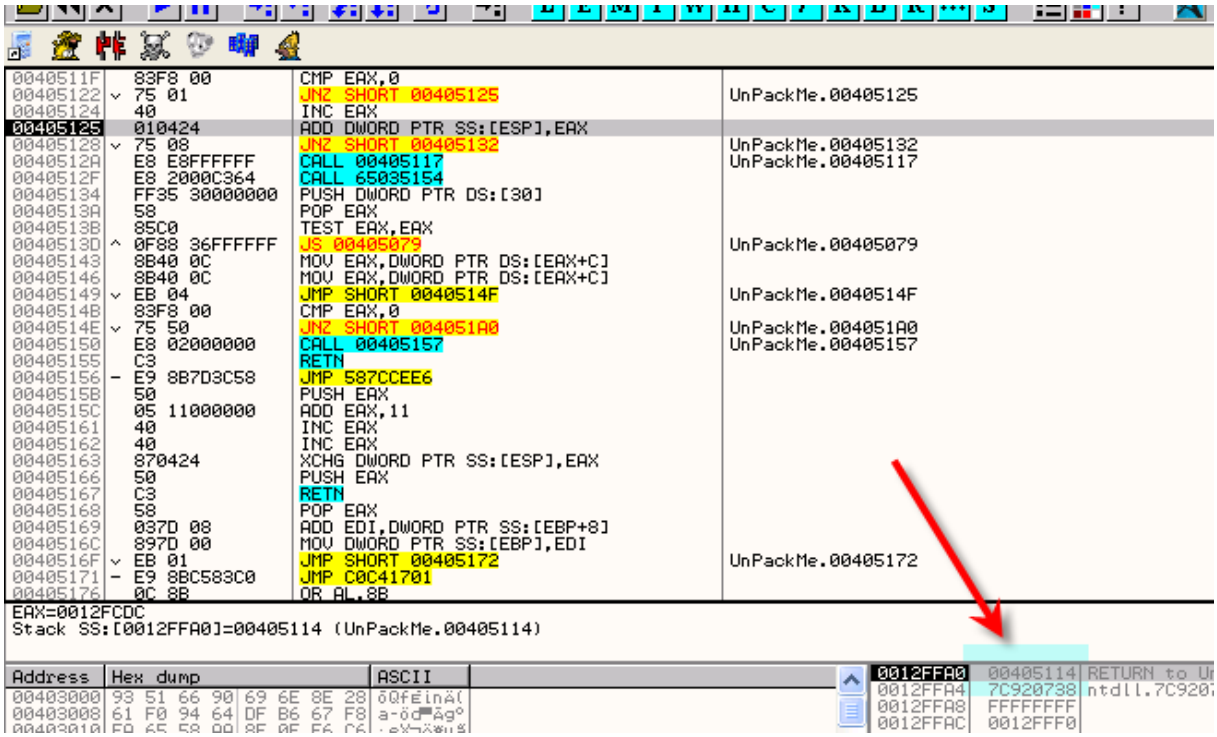
Возврат из API-функции происходит сюда:



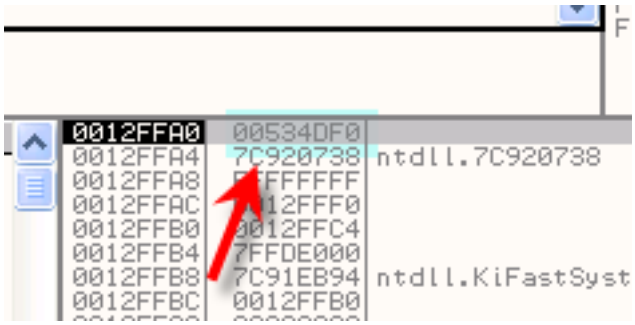
Здесь содержимое EAX сравнивается с нулем, но в данном случае оно не нулевое. Воспользуемся трассировкой для выяснения, что происходит дальше:



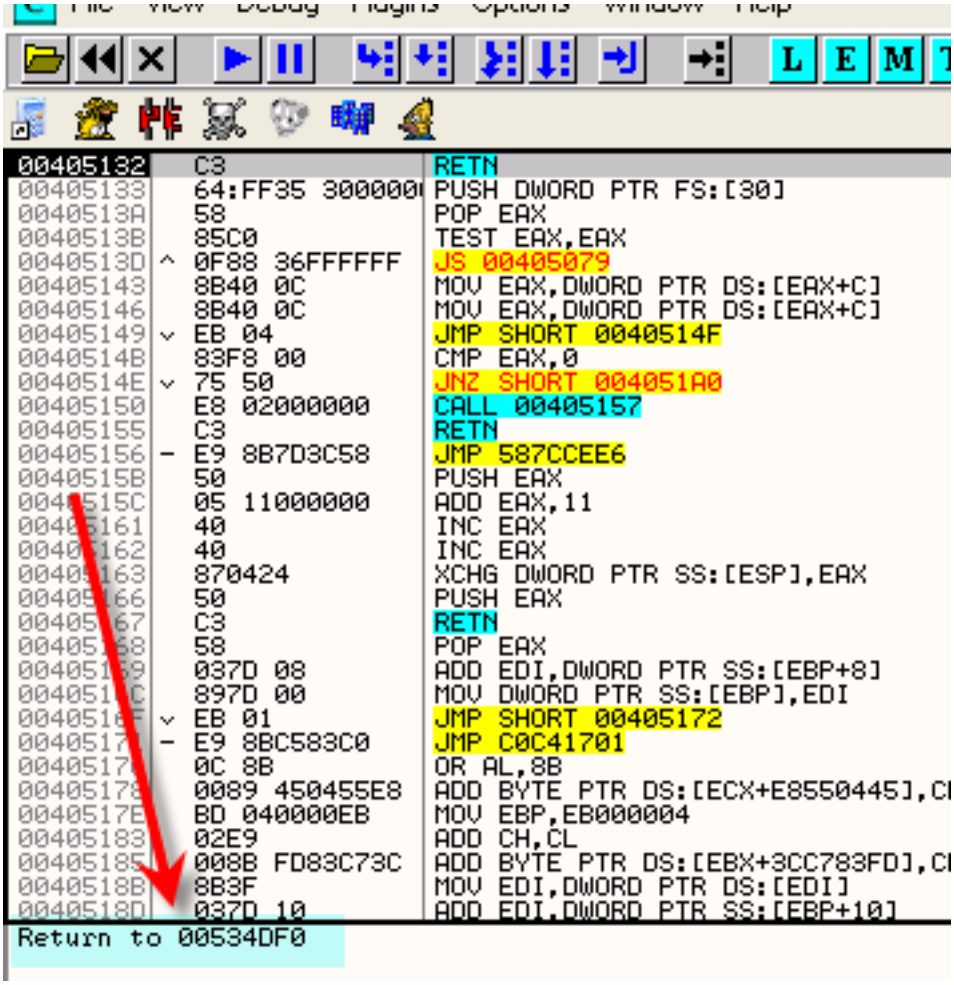
Это такой трюк: после выполнения условного перехода содержимое регистра EAX суммируется со значением, находящимся в вершине стека:



И стек становится таким:

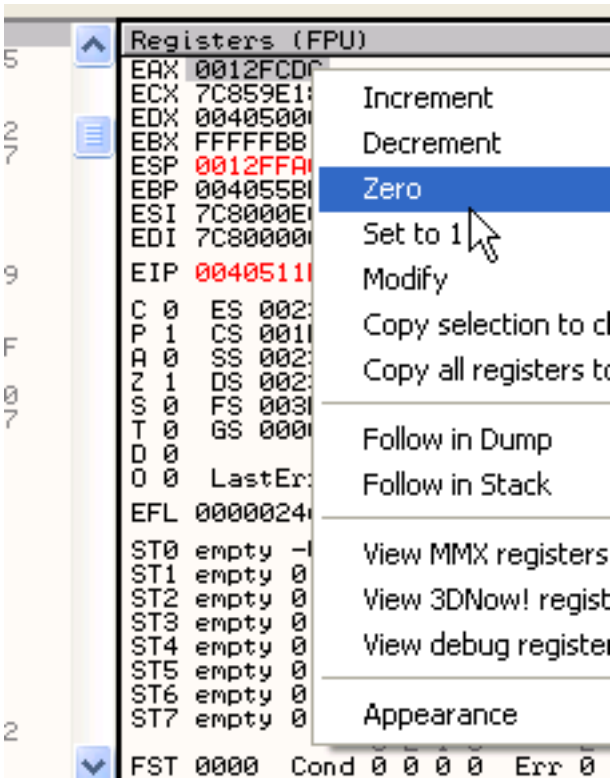


Дальнейшая трассировка приводит к команде RET, отправляющей выполнение на адрес, который был только что сохранен в вершине стека:

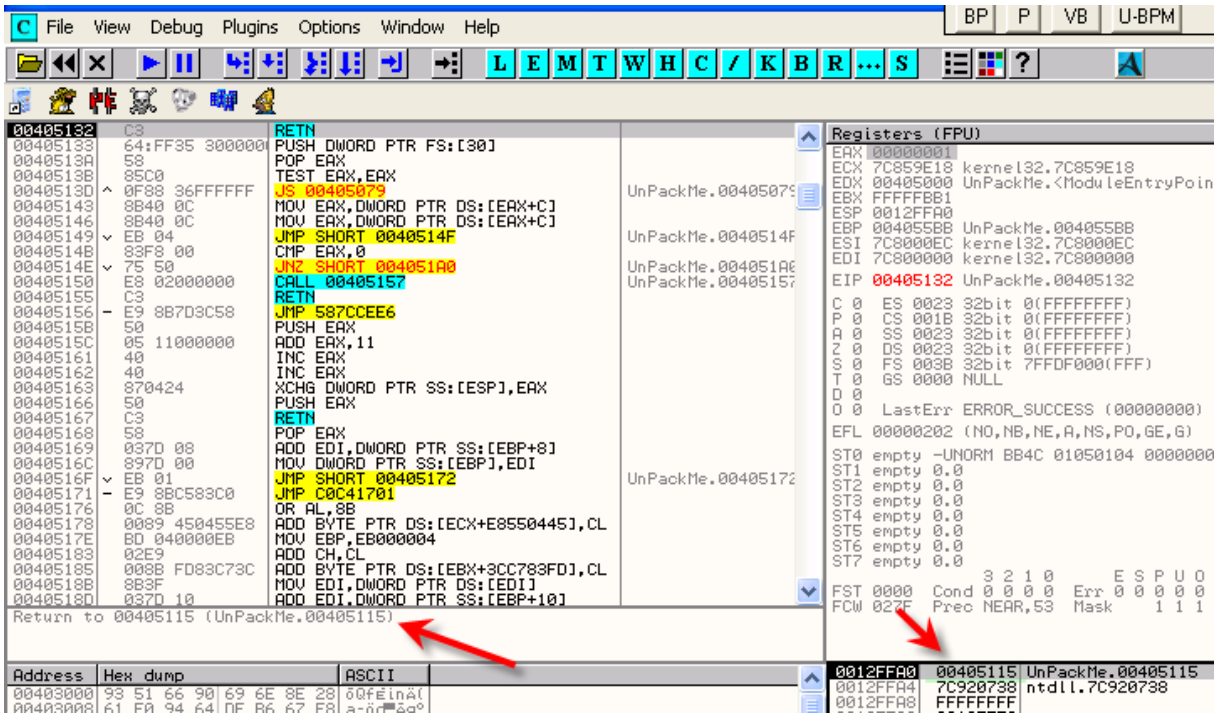


Поскольку такого адреса не существует, возникает исключение типа «No continuable» (Продолжение невозможно). Обработчик исключений обычно исправляет такое положение дел, но в данном случае он специально не установлен, чтобы программа далее просто завершилась.

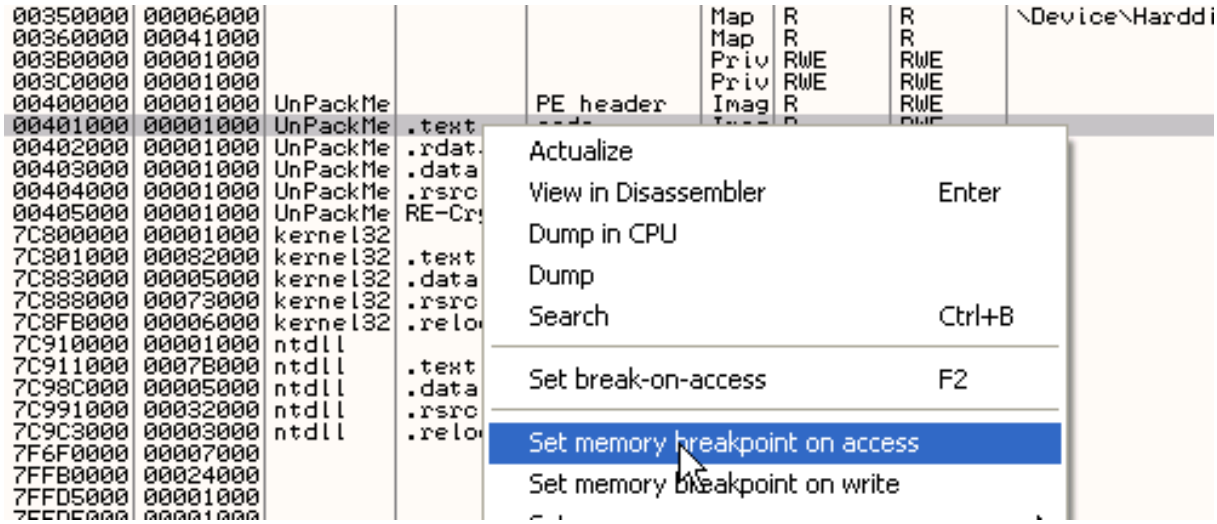
После перезагрузки анпэкми и возврата из OutputDebugStringA попробуем обнулить EAX:



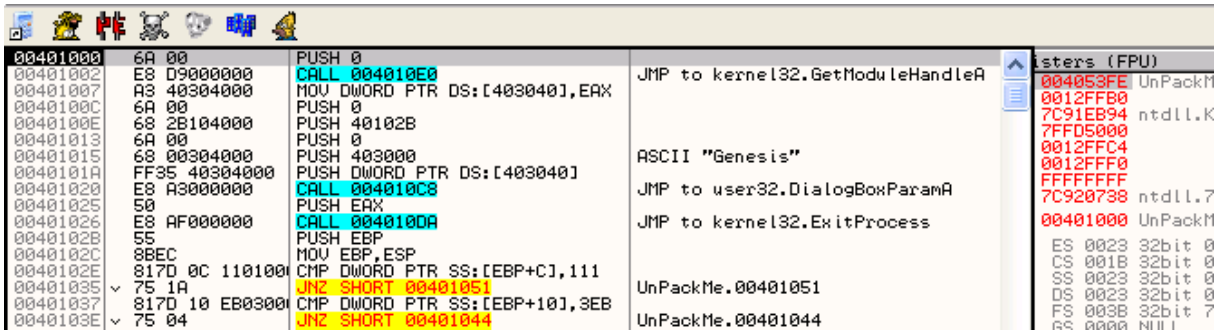
Теперь переход не срабатывает, команда INC EAX увеличивает нулевое содержимое EAX на единицу, результат операции суммируется со значением из вершины стека и выполняется RET:



Возврат произошел в существующий адрес и ошибок не возникло. Теперь установим BPM ON ACCESS в первой секции и посмотрим, дойдем ли до OEP.

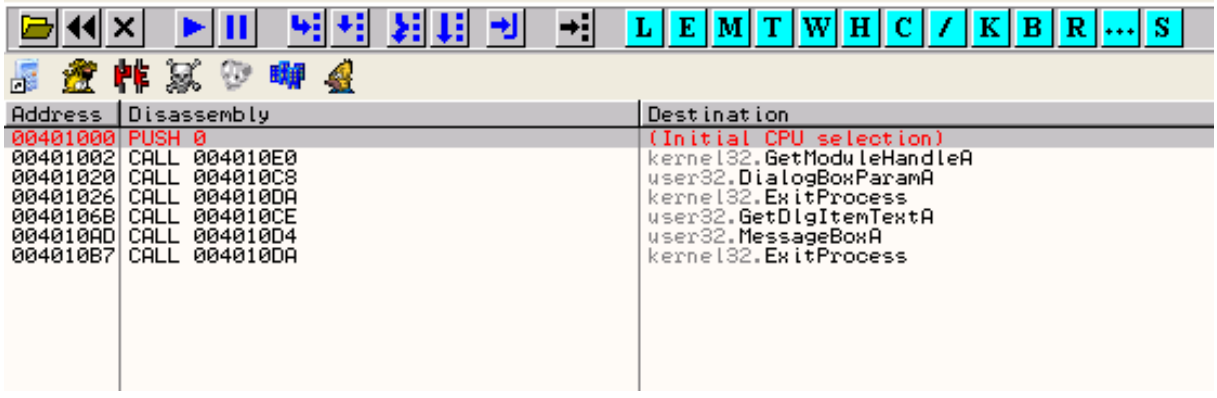


Так как наш OllyDbg пропатчен для остановок на OEP, то очередная остановка происходит при выполнении:

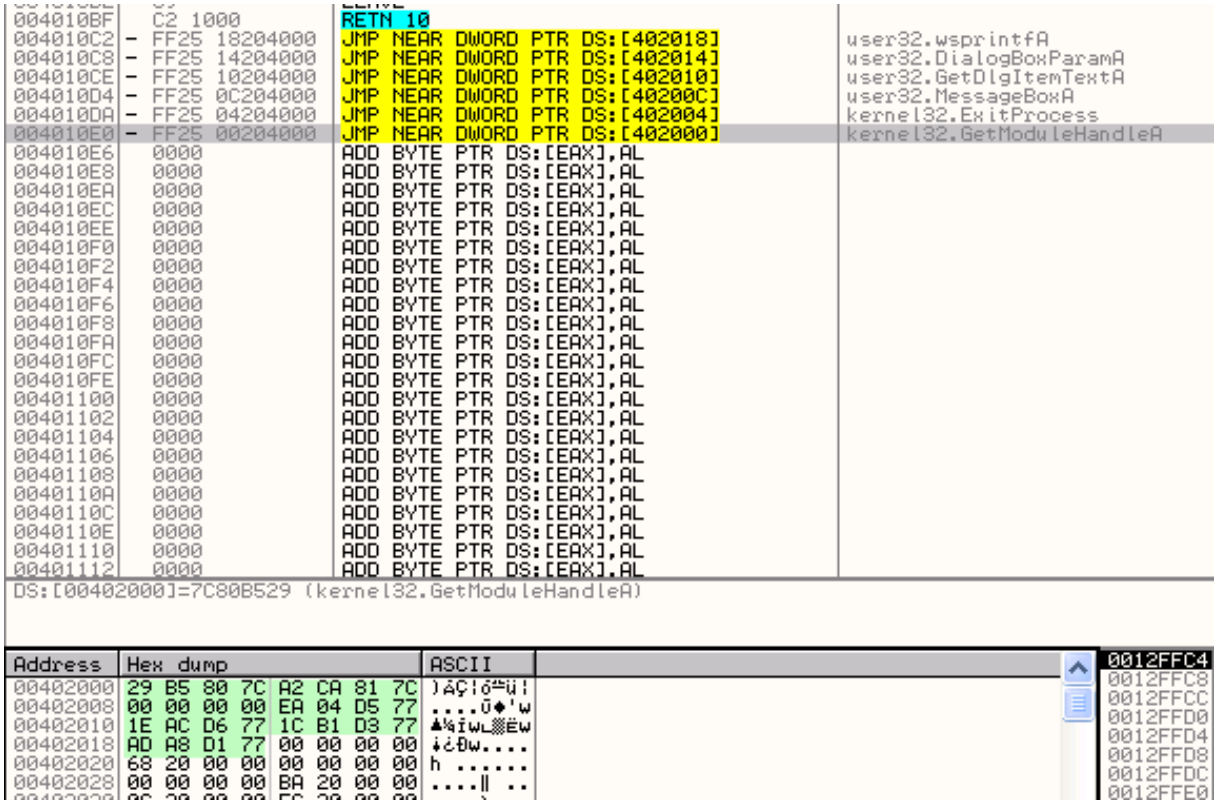


Это настоящая OEP и нам остается только петь и танцевать!

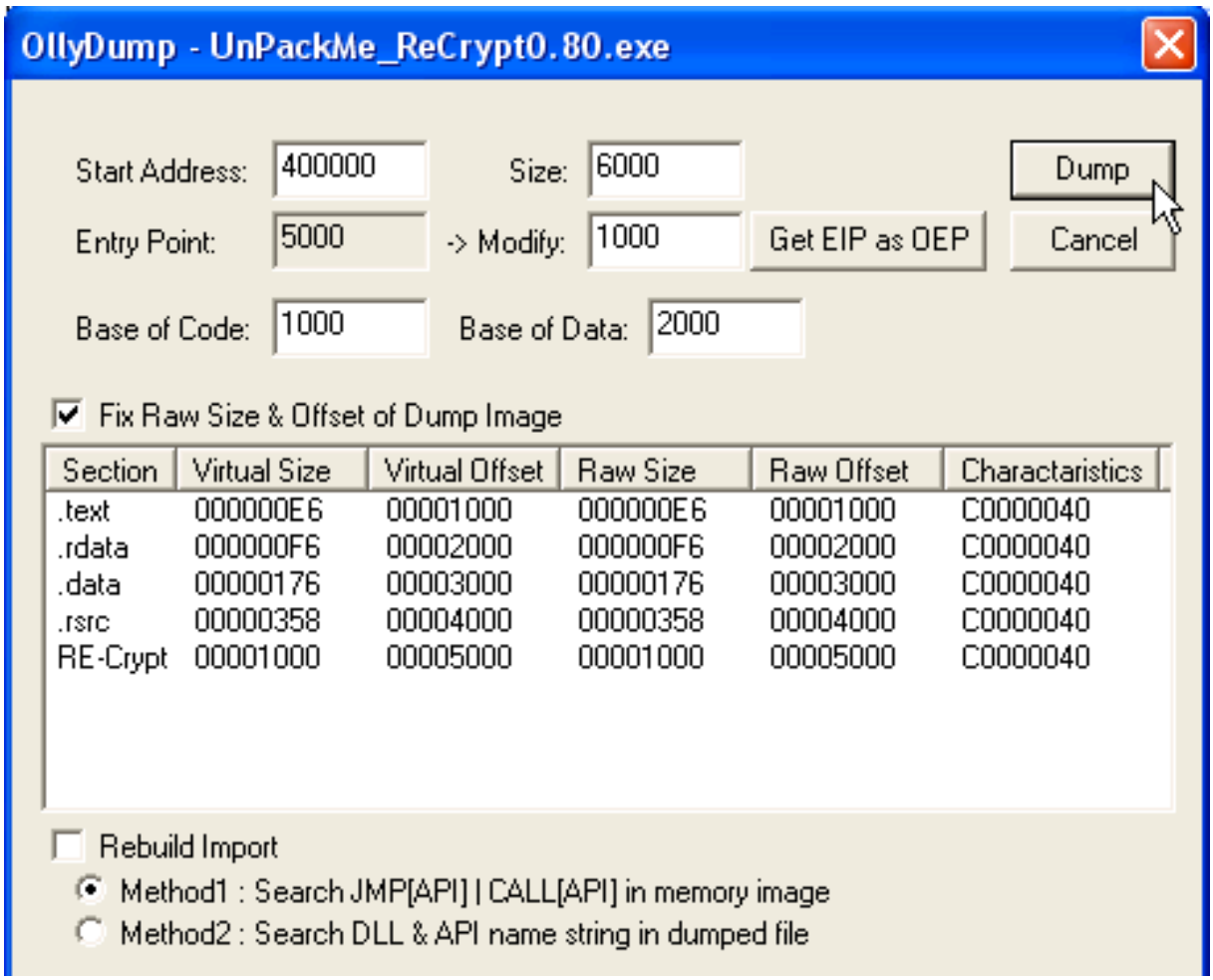
Сделаем Search for –> All intermodular calls:

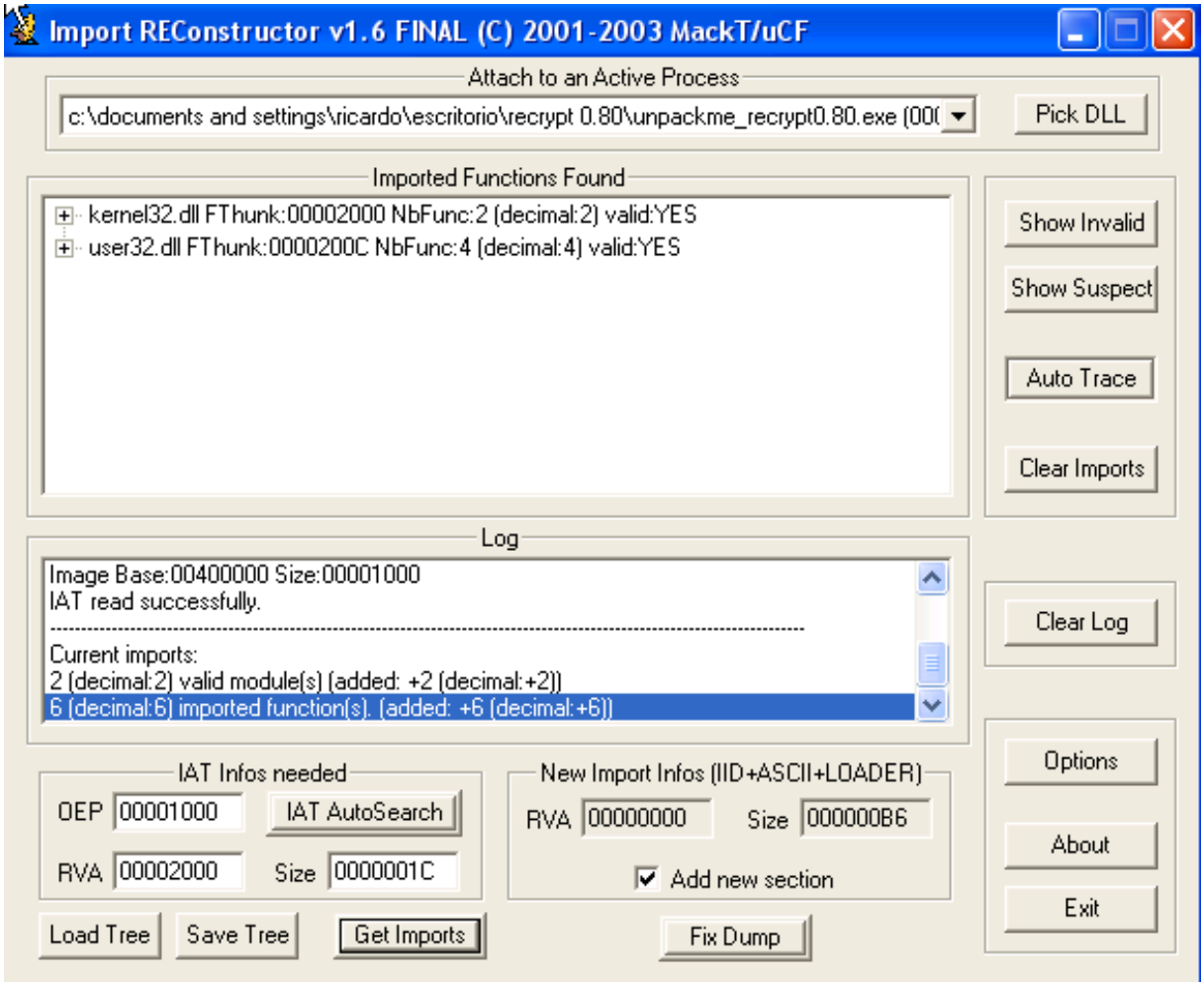


Как видим, API-функций мало, поэтому и IAT маленькая:

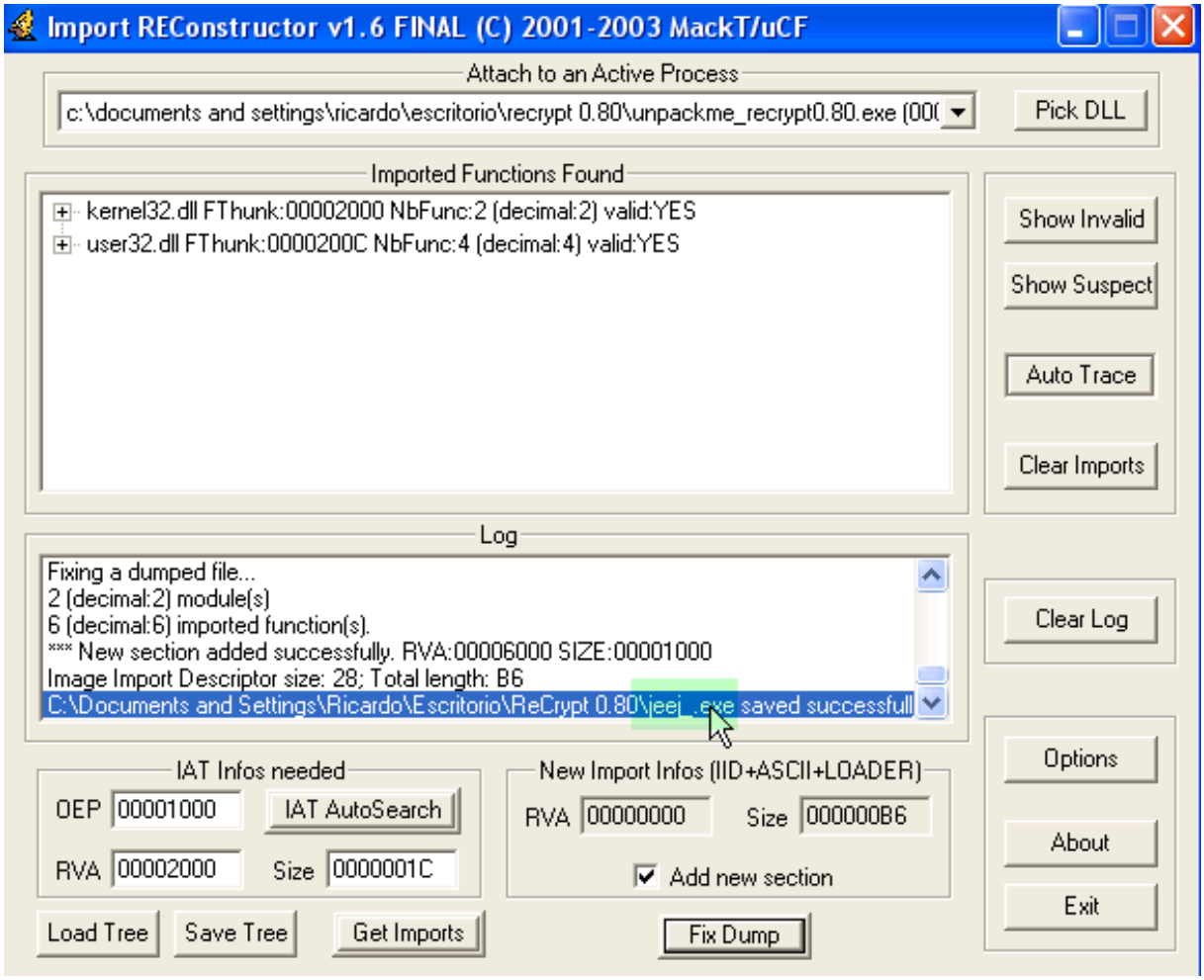


На скриншоте видно, что IAT заключена между адресами 402000 и 40201C, то есть RVA IAT = 2000, Size IAT = 1C и RVA OEP = 1000. Теперь сдампим программу, а затем восстановим файл в ImpRec’e:

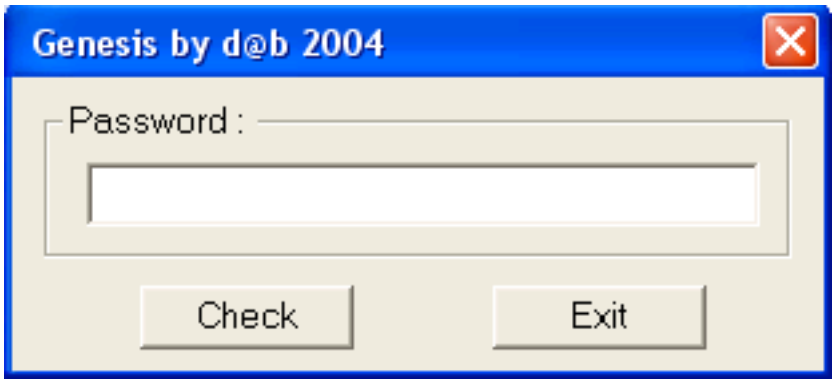

OllyDump - UnPackMe_ReCrypt0.80.exe
Start Address: 400000
Size: 6000
Dump
Entry Point: 5000
-> Modify: 1000
Get EIP as OEP
Cancel
Base of Code: 1000
Base of Data: 2000
Fix Raw Size & Offset of Dump Image
Section Virtual Size Virtual Offset Raw Size Raw Offset Charactaristics
.text 000000E6 00001000 000000E6 00001000 C0000040
.rdata 000000F6 00002000 000000F6 00002000 C0000040
.data 00000176 00003000 00000176 00003000 C0000040
.rsrc 00000358 00004000 00000358 00004000 C0000040
RE-Crypt 00001000 00005000 00001000 00005000 C0000040
Rebuild Import
Method1 : Search JMP[API] | CALL[API] in memory image
Method2 : Search DLL & API name string in dumped file



При сохранении исправленного дампа к его имени добавляется, как всегда, символ подчеркивания '_':



Попробуем запустить его…



Вот и решен наш простой упаковщик!

До встречи в 51-й главе!

ReCrypt 0.80 (находится в архиве `wasm_files.zip`: `files/0525/ReCrypt.0.80.rar`)

Введение в крэкинг с нуля, используя OllyDbg - Глава 51

Автор: Рикардо Нарваха, пер. Рома Стремилов

Дата: 2010-07-25

В этой главе мы кое-что изменим, и я думаю, что так будет лучше: теперь распаковка будет интерактивной. Однако это не означает, что Введение заканчивается — далее мы просто затронем другую тему, но прежде я сделаю так, чтобы вы немного поработали. В данном туториале мы интерактивно распакуем последнюю на сегодняшний день версию ASProtect'а. Я буду делать самое легкое, а вы — самое трудное, хе-хе.

Файл UnPackMe_ASProtect.2.3.04.26.a.exe не снабжен всей защитой, то есть это довольно простая версия протектора. Стоит лишь заметить, что если при попытке дойти до OEP появится сообщение об ошибке защиты, то следует скопировать файл в другое место, где он будет функционировать нормально. Или же, если в папке есть несколько файлов с различными именами и один из них начнет досаждать сообщениями об ошибке защиты и перестанет запускаться, то следует запустить другой, а затем прежний: если после этого ошибка исчезнет, то будет ясно, почему, хе-хе.

Кроме того, мы воспользуемся двумя новыми плагинами — OllyBonE и Weasle. В описании OllyBonE сказано:

Installation:

Copy ollybone.dll and i386/ollybone.sys to your OllyDbg directory.

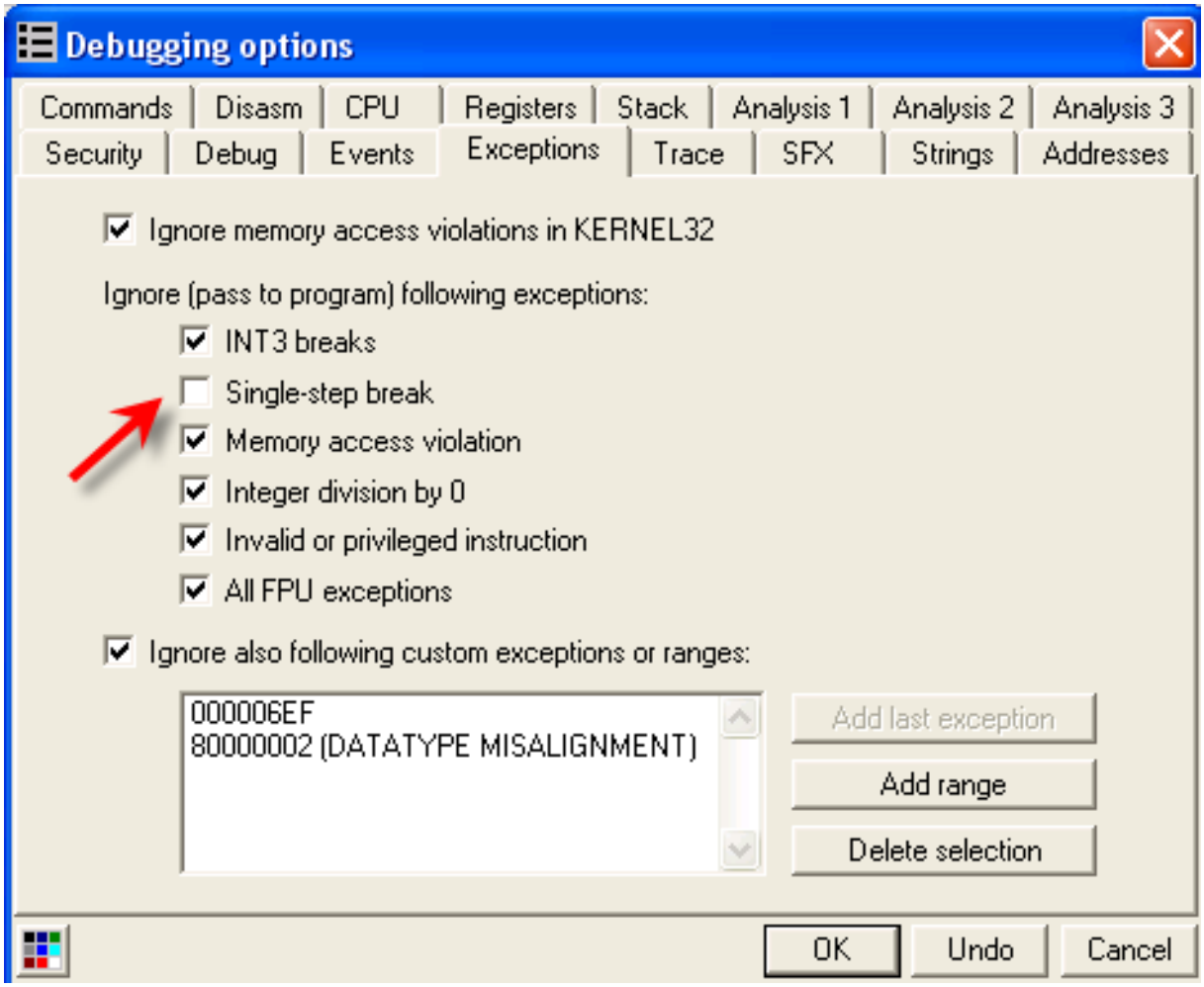
То есть нужно скопировать файл ollybone.dll в папку для плагинов и ollybone.sys в программную папку OllyDbg. Если это одна и та же папка, то оба файла следует скопировать в нее.

На скриншоте показана папка Olly и новый SYS-файл в ней, а DLL'ка скопирована в папку с плагинами (в моем случае это C:\Plugins).

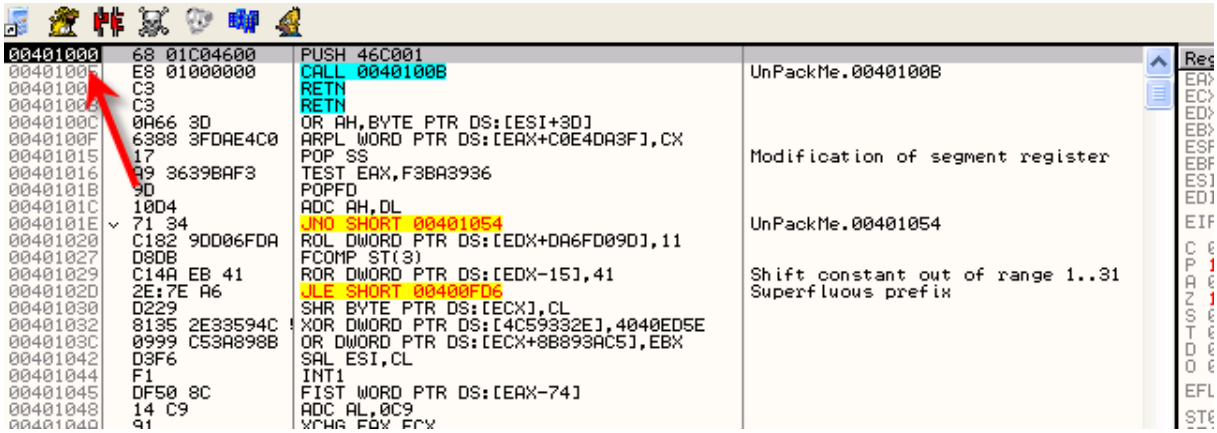
Что касается Weasle, то в руководстве к оригинальному пакету говорится, что при использовании модифицированного OllyDbg следует распаковать DLL'ку с помощью UPX 2.01 (мы уже видели, что эта программа также и распаковывает). Я сделал это заранее и приложил к статье готовую к использованию DLL'ку. Файл Importer.dll копируется в папку Olly, как видно на скриншоте, а RL!Weasle.dll — в папку с плагинами.

С ASProtect'ом нужно быть осторожным, поскольку он нередко обнаруживает установленные BP или HB, и тогда начинаются песни об ошибке защиты и приходится производить замену экзешника. Поэтому для достижения OEP мы воспользуемся OllyBonE — плагином, который посредством SYS-драйвера симулирует установку BREAKPOINT ON EXECUTION, что мы видели в версии

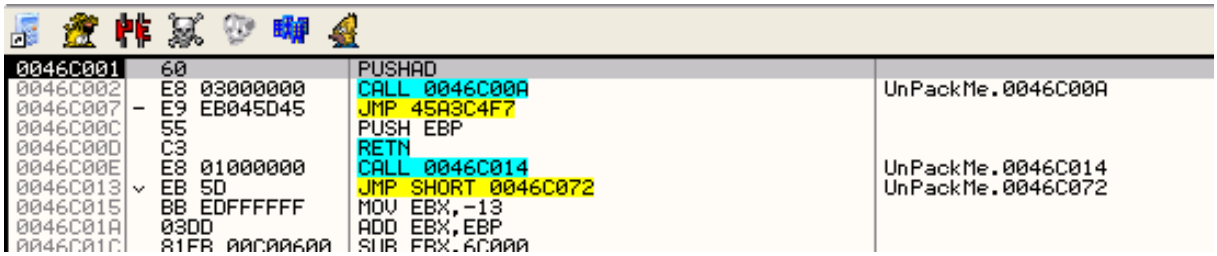
OllyDbg, пропатченной для поиска OEP'ов, однако данный плагин позволяет дойти до OEP гораздо быстрее. Его недостаток в том, что брейк можно устанавливать только в секциях EXE-файла, и для Visual Basic'а он не годится. Тем не менее он хорош для поиска OEP, и мы им воспользуемся, но сначала снимем одну галку в окне настроек исключений, иначе он работать не будет:



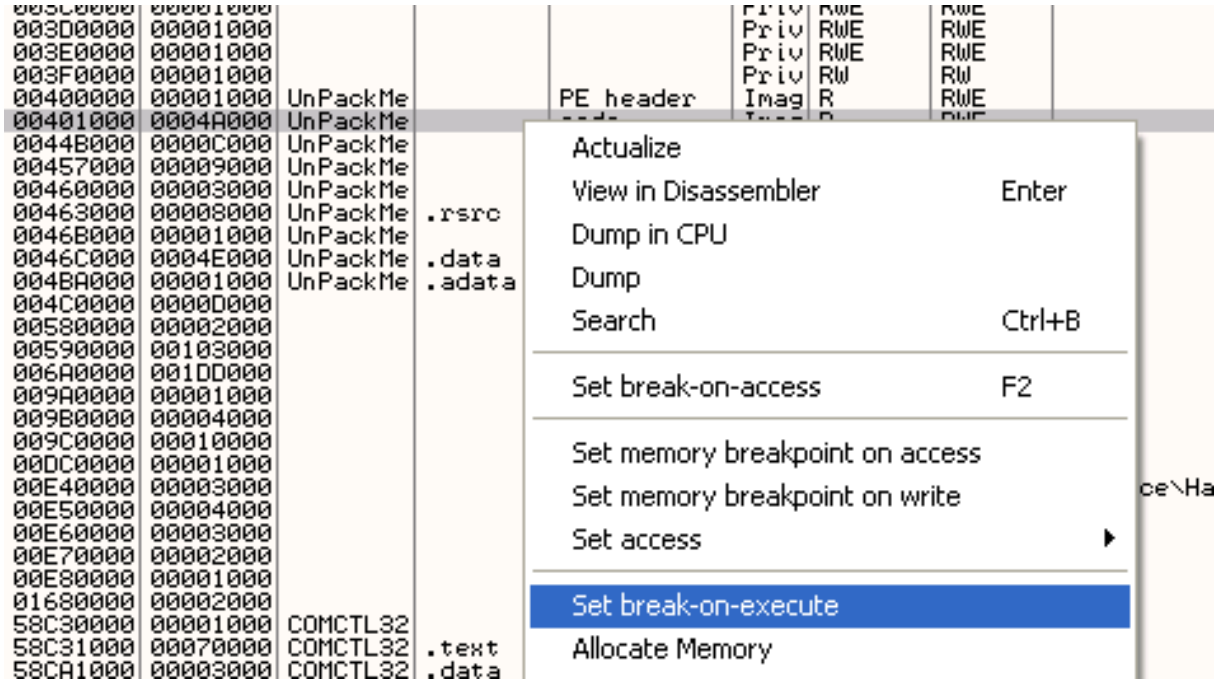
Остальные галки должны быть установлены, чтобы нам было проще добраться до цели. [У меня плагин заработал только после отключения списка добавленных исключений.]



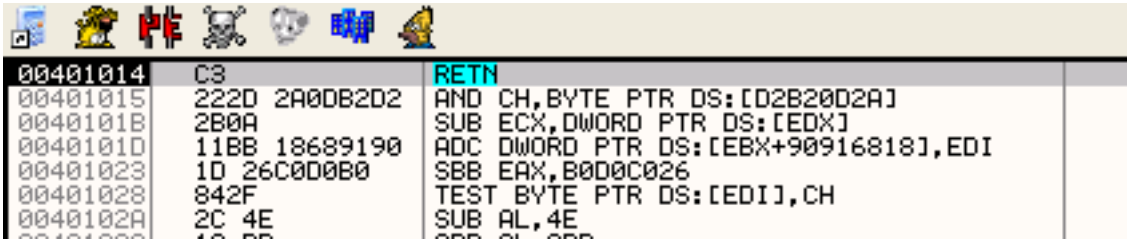
После запуска программа остановилась в первой секции, поэтому прежде установки break-on-execute придется трассировать несколько строк с помощью F7 вплоть до выхода из нее:



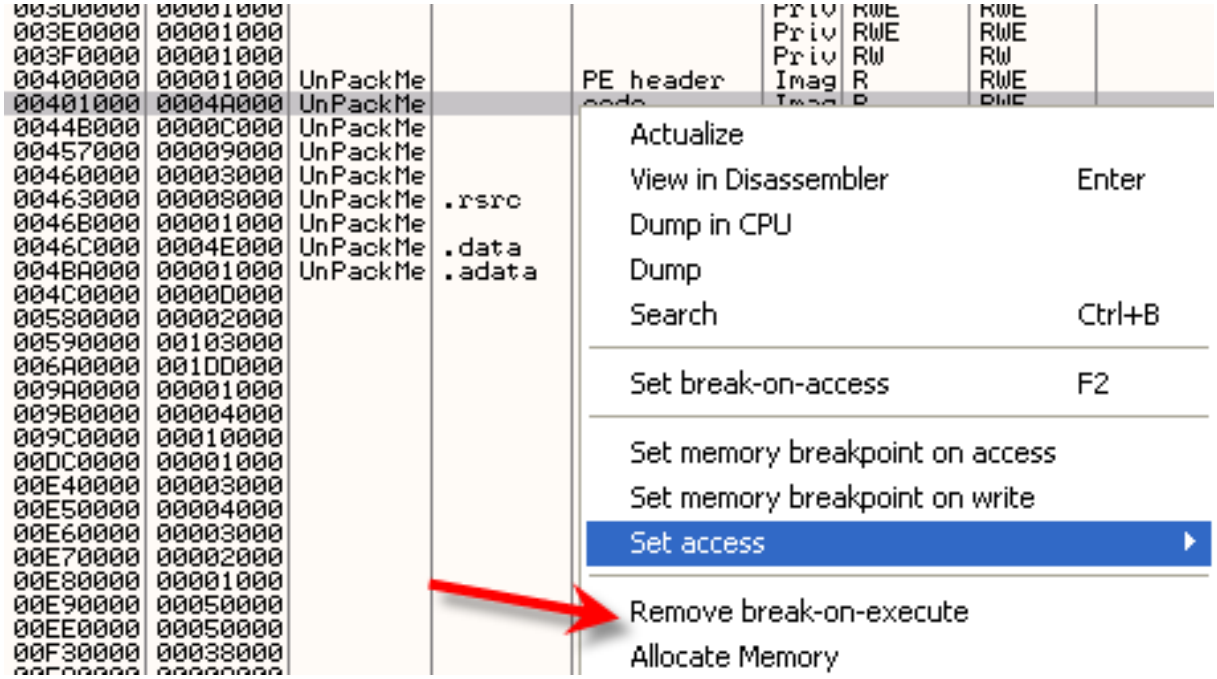
Теперь мы вне первой секции и можно устанавливать break-on-execute в карте памяти:



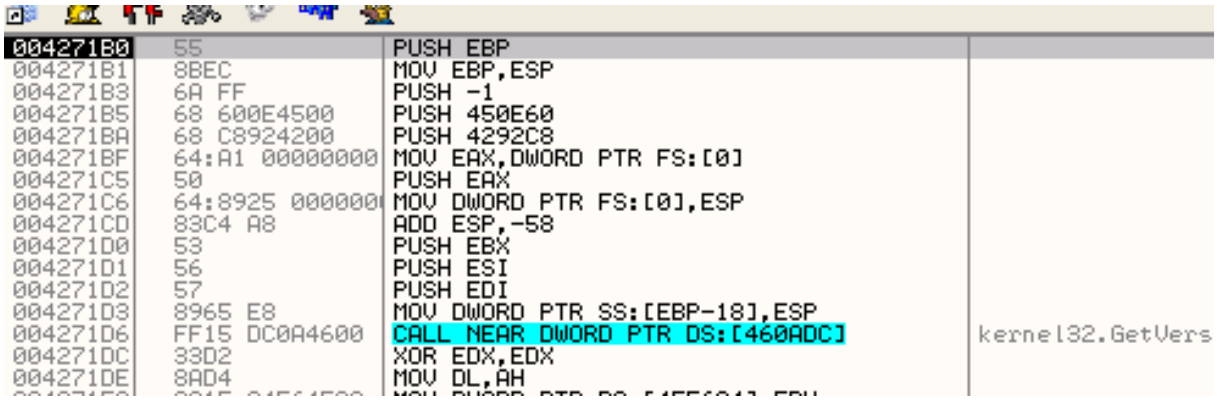
Остановка произошла здесь:



В любом другом упаковщике это была бы OEP, но ASProtect всегда выполняет RET и возвращается в область кода распаковщика. Снимем break-on-execute:

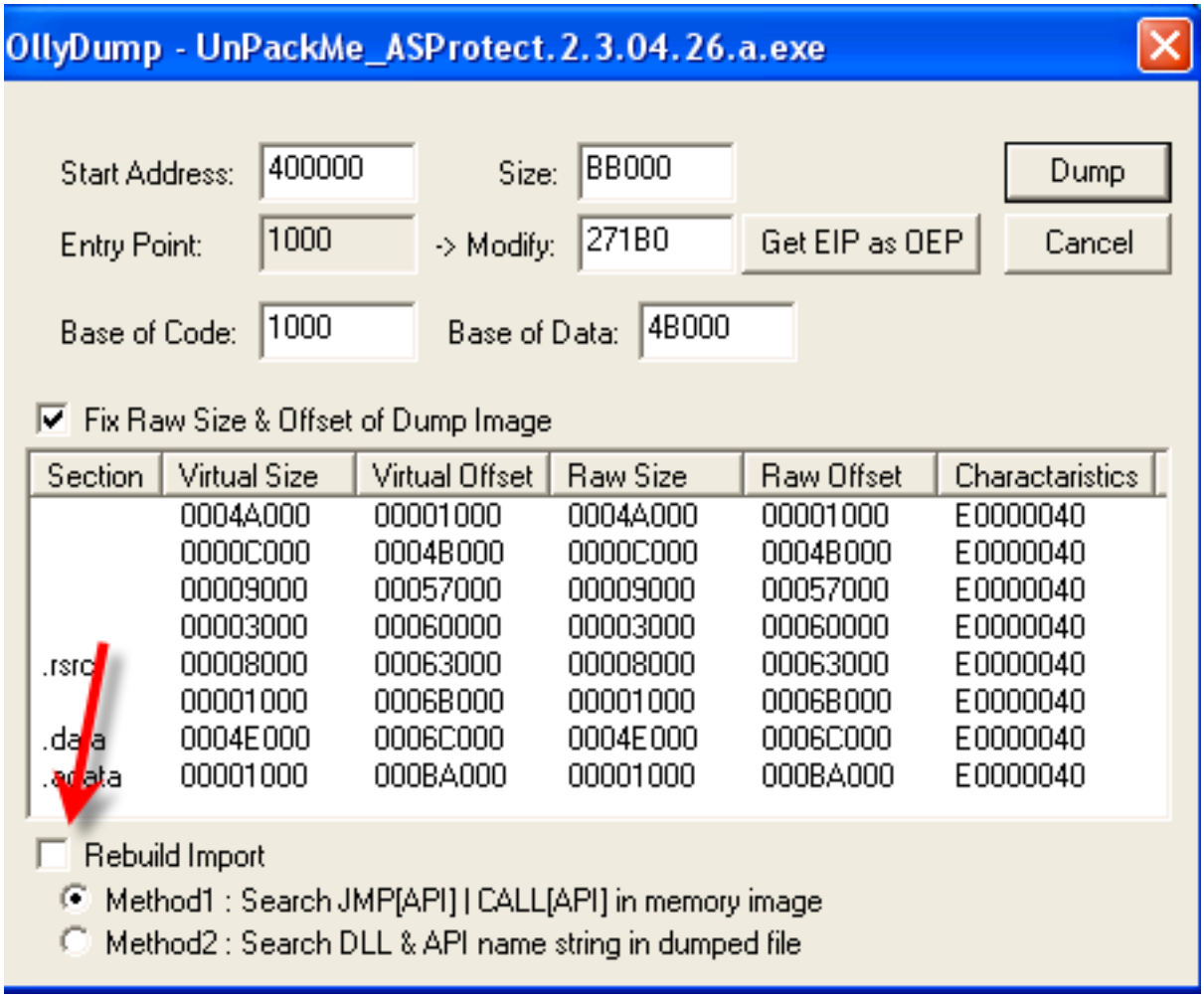


Для выхода из первой секции нажмем F7, и, повторно установив break-on-execute, — RUN.

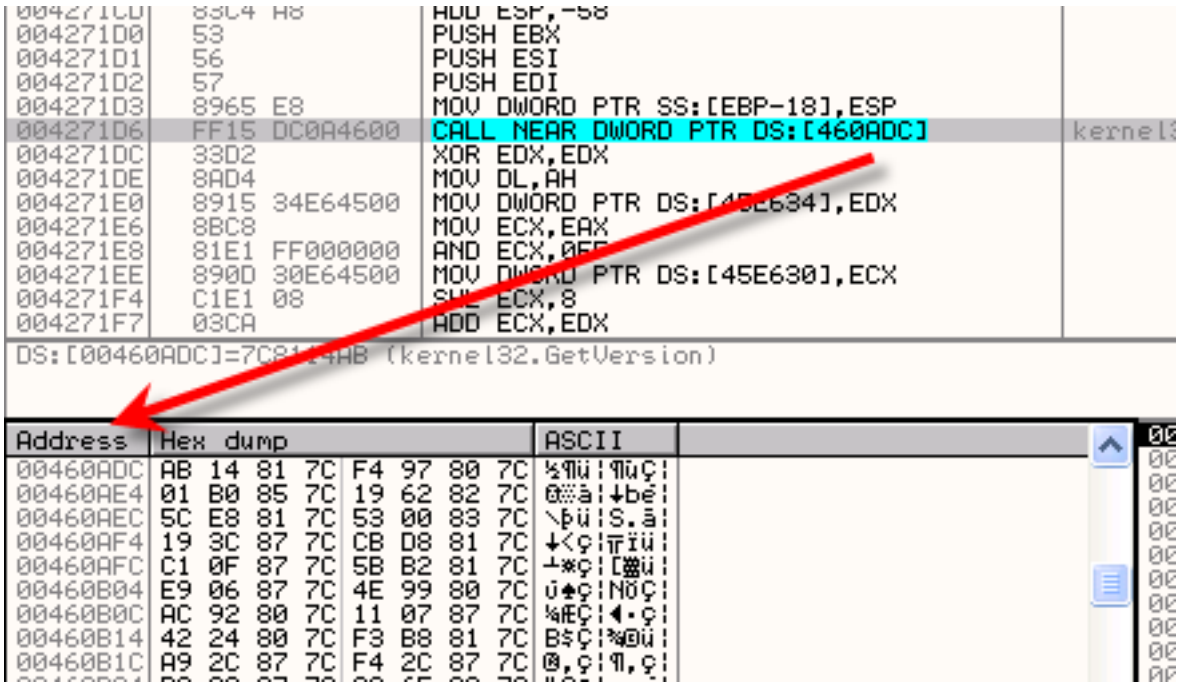


Теперь, оказавшись на OEP [см. Дополнение в конце главы], следует вспомнить, что это было обеспечено специальным драйвером и для дальнейшей трассировки или выполнения программы нужно снять break-on-execute, иначе нам придется повторить всё с начала. Запомните это!

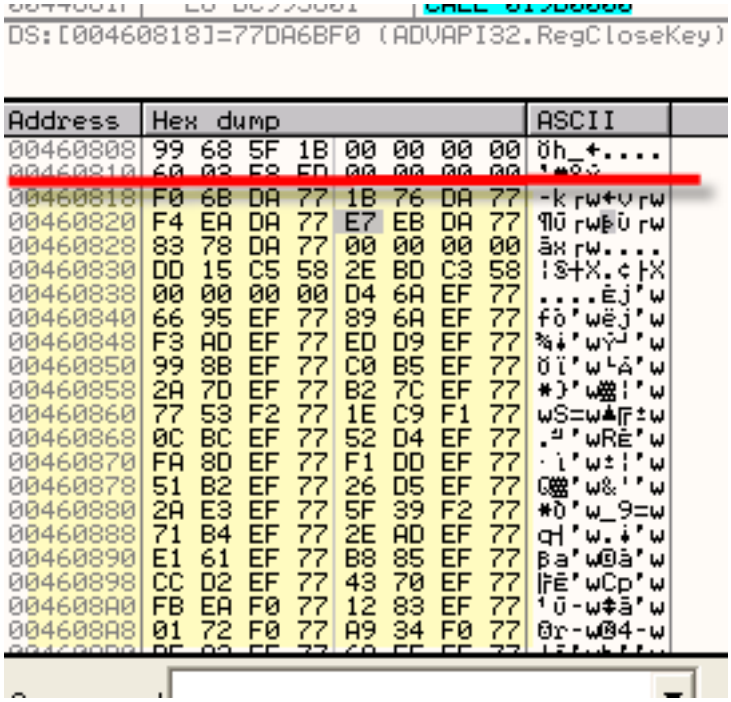
Хорошо, теперь сдампим процесс с помощью OllyDump, не забыв снять галку с Rebuild Import:



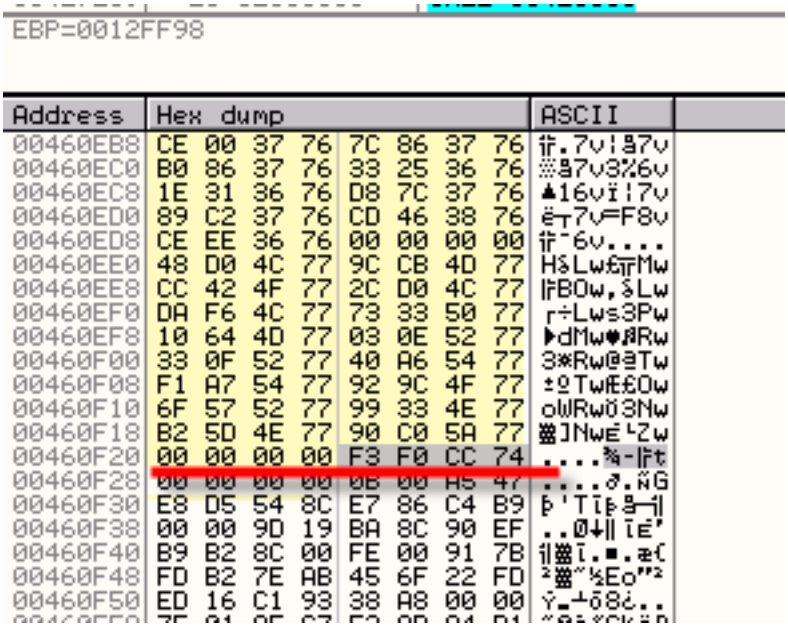
Затем отыщем начало и конец IAT'а. В листинге есть вызов API-функции, аргумент которого похож на адрес верного элемента IAT'а:



Если подняться, можно найти начало IAT — 460818:



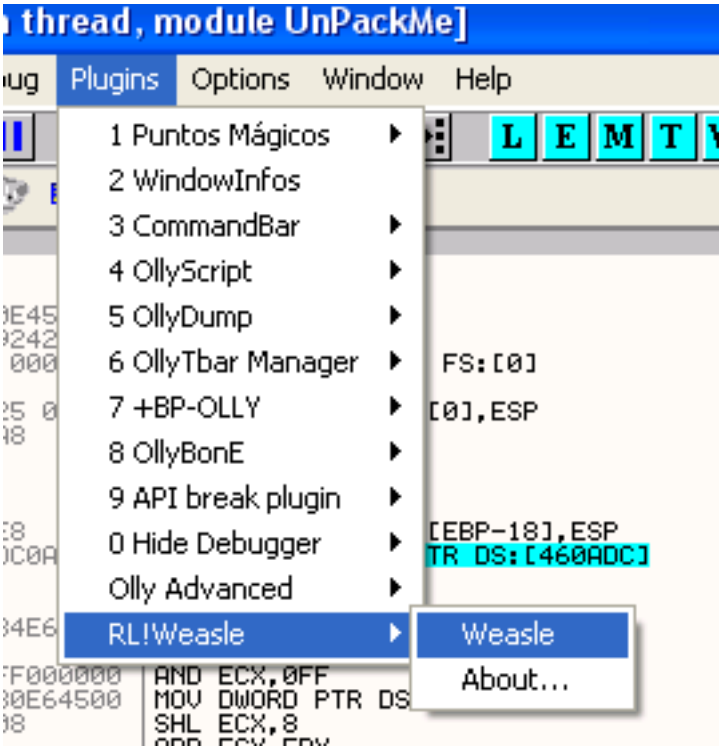
А если спуститься, то и ее конец — 460F28:

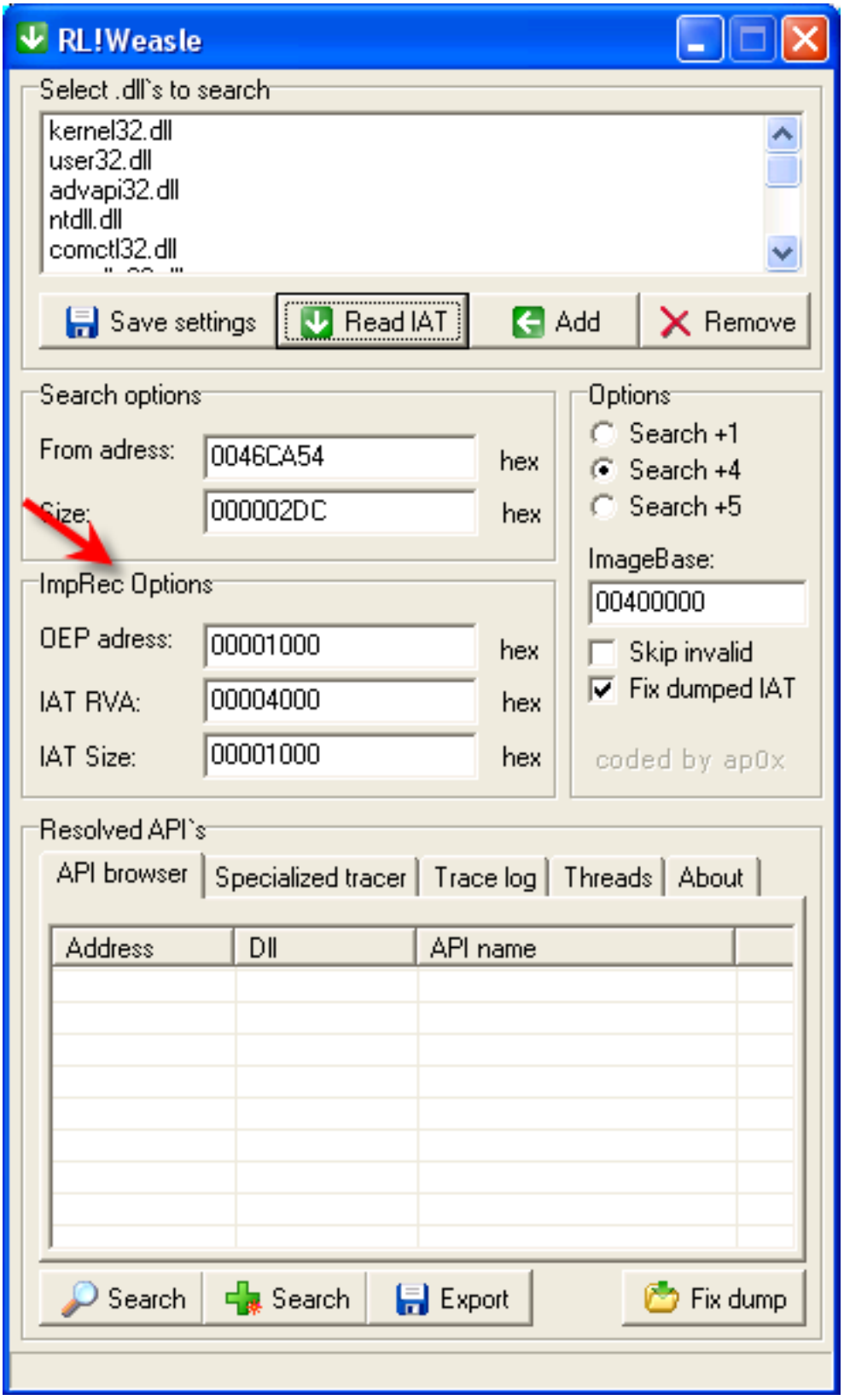


Таким образом, имеем:

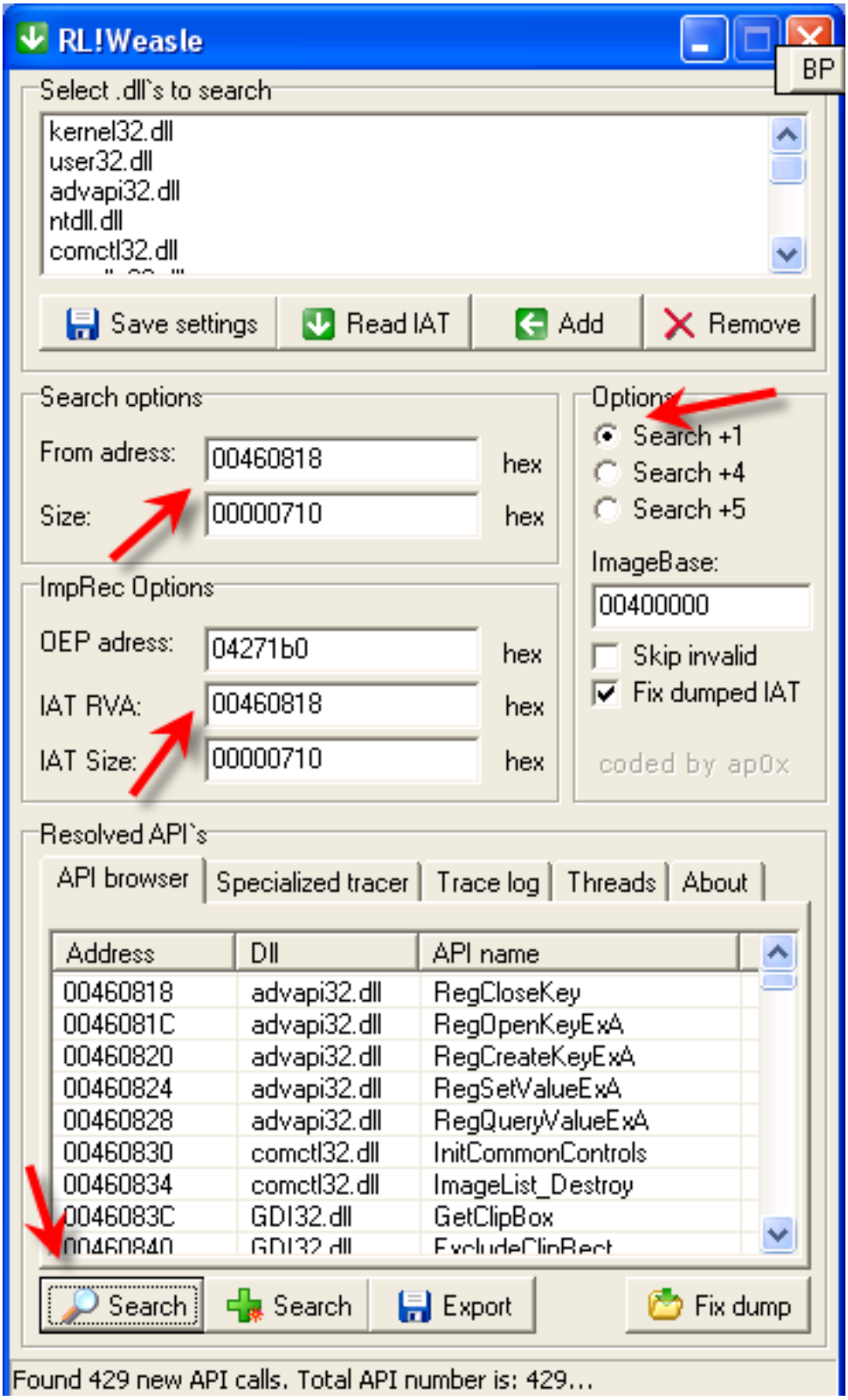
```
OEP = 4271B0
IAT = 460818
Size IAT = 710
```

Но на этот раз для восстановления IAT'а вместо ImpRec'а воспользуемся Weasle. Посмотрим, поможет ли он нам.

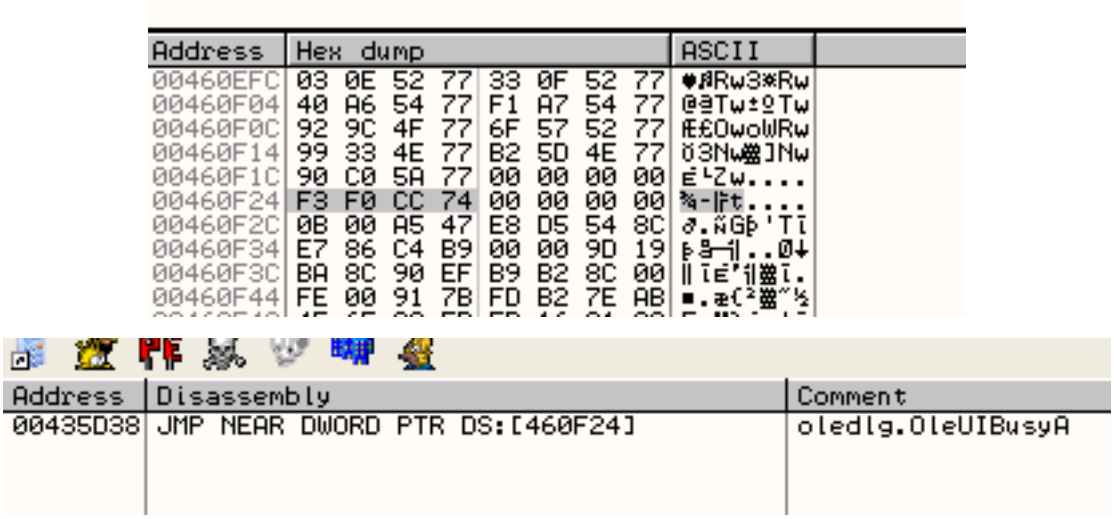

thread, module UnPackMe]
ug
Plugins
Options
Window
Help
1 Puntos Mágicos
2 WindowInfos
3 CommandBar
4 OllyScript
5 OllyDump
6 OllyTbar Manager
7 +BP-OLLY
8 OllyBonE
9 API break plugin
0 Hide Debugger
Olly Advanced
RL!Weasle
Weasle
About...
L
E
M
T
FS:[0]
[0],ESP
[EBP-18],ESP
TR DS:[460ADC]
AND ECX,0FF
MOV DWORD PTR DS
SHL ECX,8



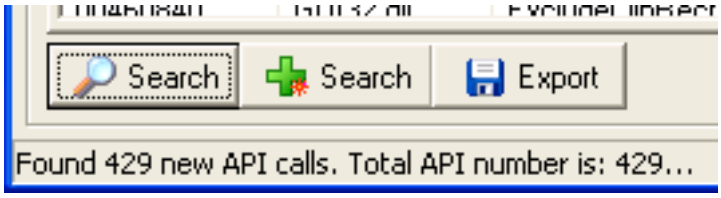
Введем найденные данные в поле «ImpRec Options»:



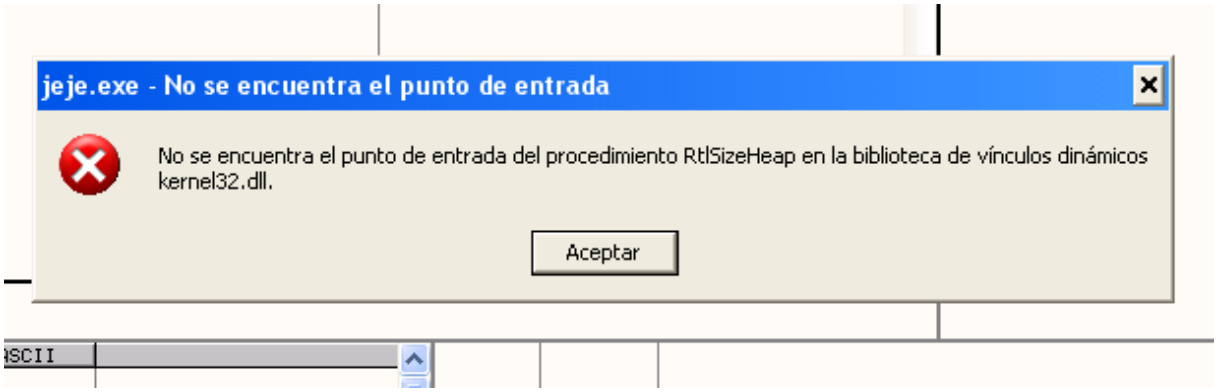
Кроме того, нужно скопировать начальный адрес IAT и ее размер в поля «From address» и «Size» области «Search options» и установить переключатель «Options» в положение «Search +1». По идее, этого должно быть достаточно для нормального считывания, однако когда я пользовался этим плагином, часто какая-либо DLL'ка оказывалась пропущенной, например, DLL'ка последнего элемента. Но это не проблема — ее можно добавить с помощью кнопки Add. Посмотрим соответствующий референс в Olly:



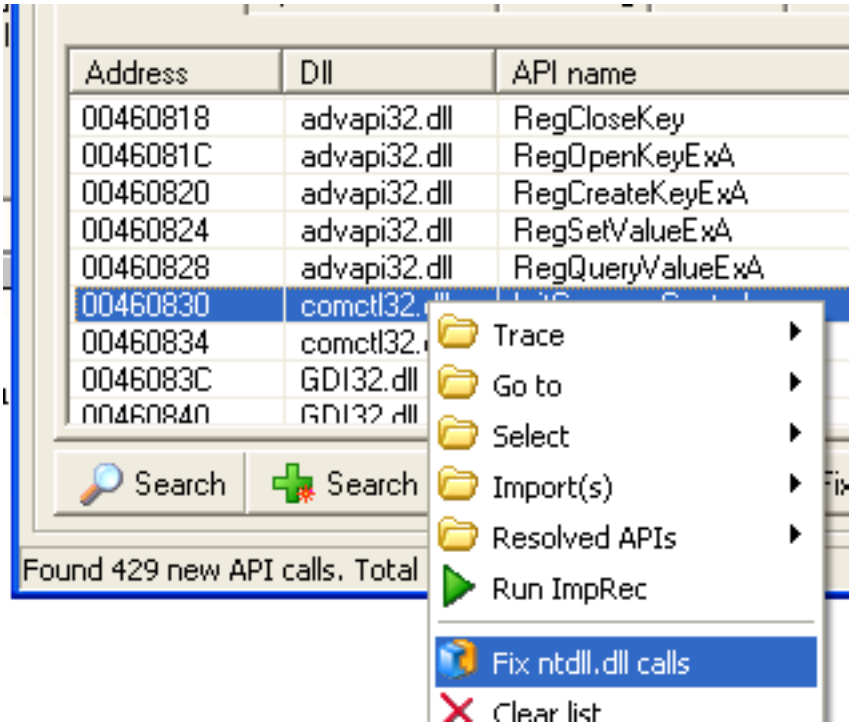
Как видно из скриншота, это oledlg.dll, которая отсутствует в верхнем списке, отображающем используемые в программе DLL'ки. Как правило, в таком случае я нажимаю кнопку Add и ввожу «oledlg.dll», а затем повторно нажимаю Search, и проблема исчезает. Следует помнить, что это только бета-версия плагина, т.е. у него могут быть баги. Посмотрим, насколько хорошо он произведет восстановление, ведь мы отнюдь не собираемся дожидаться релиза, хе-хе:



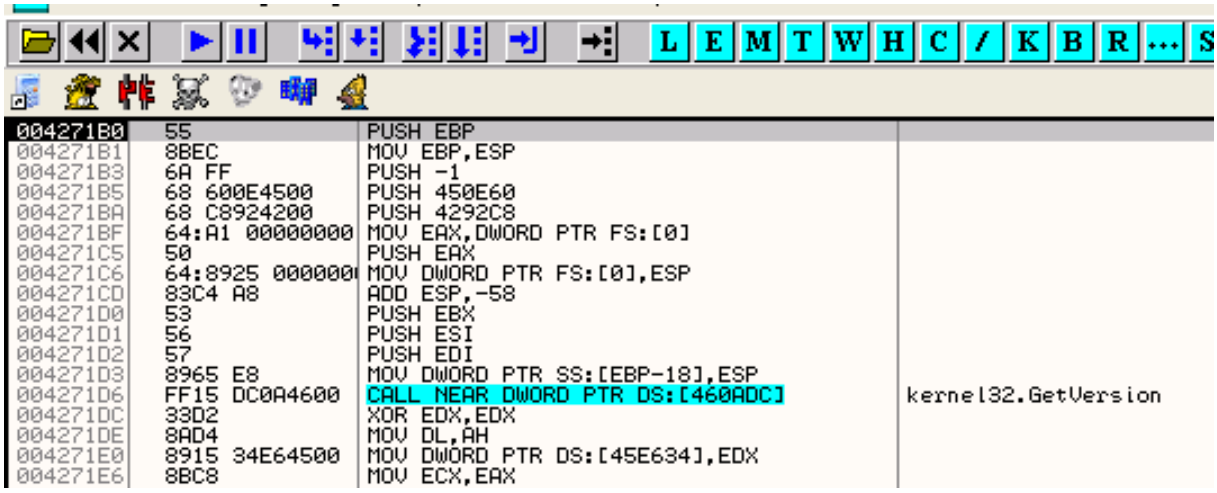
Теперь сообщается, что найдено 429 из 429. Запустим восстановленный дамп:



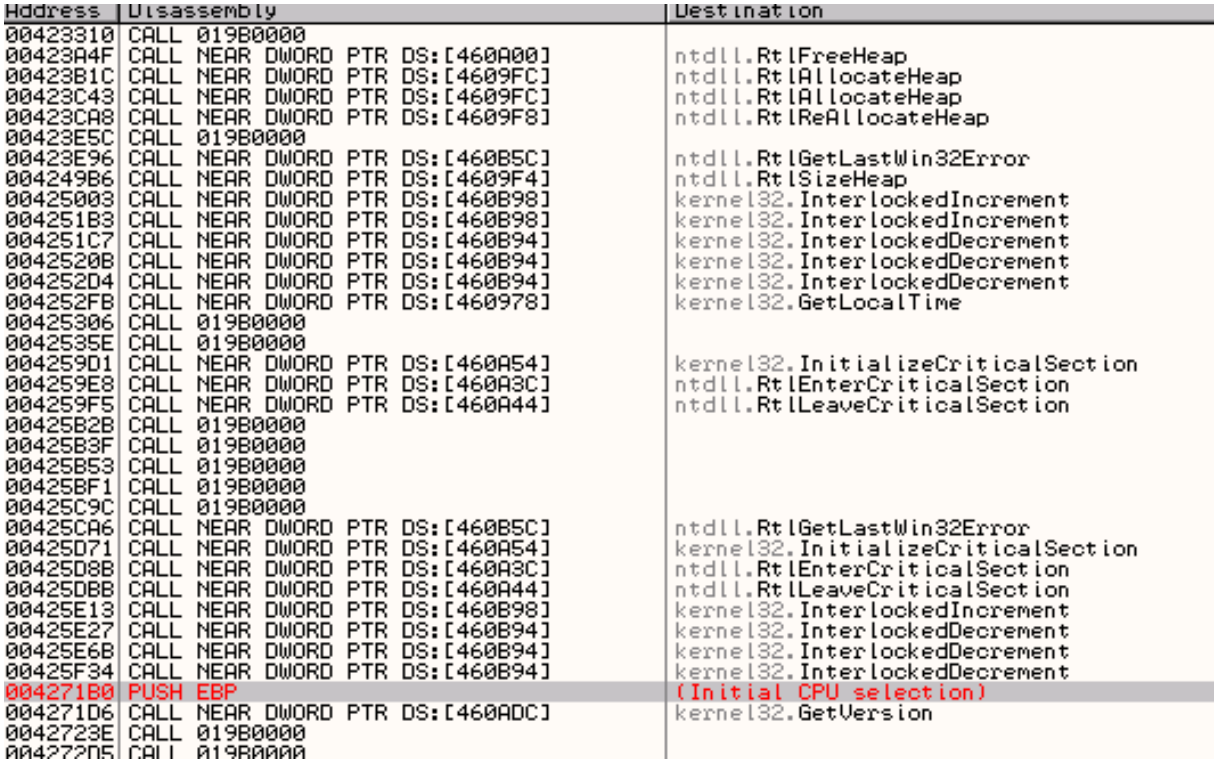
Ах, не хватило малой детали! Зато стало ясно, что вызовы ntdll плохо обрабатываются; в плагине есть опция, исправляющая это, и ее нужно всегда применять:



Теперь следует удалить только что полученный экзешник и восстановить прежний дамп, сменив расширение .bak на .exe. Если снова применить «Fix dump», то всё будет восстановлено, хотя программа и в этом случае не запустится, поскольку она исковеркана антидампами. Но дальнейшую работу придется выполнить уже вам по приведенному ниже плану.



У нас есть дамп, восстановленный с помощью Weasle и полностью исправленная IAT. Однако посмотрим Search for –> All intermodular calls:



CALL'ы на один и тот же адрес (019B0000 на моем компьютере) — это антидампы, поскольку такой адрес отсутствует в созданном нами файле.

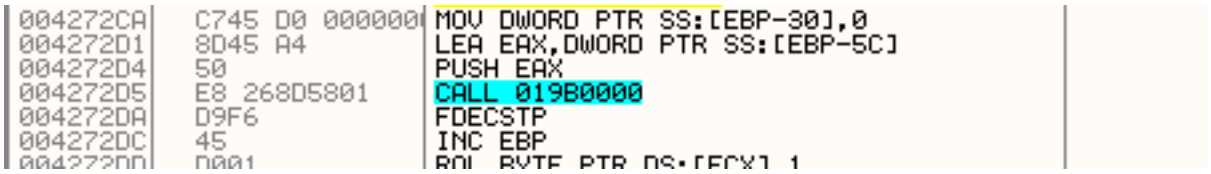
Ниже привожу домашнее задание, а чтобы вы не пугались, дам несколько подсказок.

Если вы захотите написать скрипт для исправления антидампов, то это нужно сделать в течение 15 дней. Все полученные мной скрипты я проверю, а в следующей главе упомяну автора полностью рабочего скрипта, который мне покажется наиболее простым и эффективным.

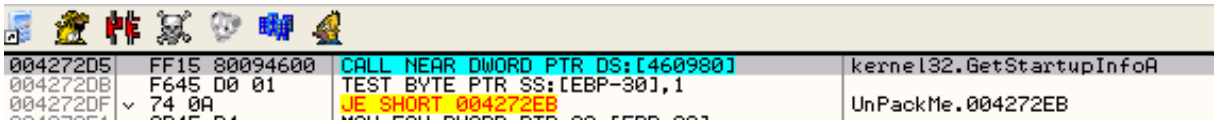
В качестве помощи вы можете воспользоваться распакованными нами ранее программами, чтобы посмотреть, к какой API-функции ведут CALL'ы в каждом случае. Скрипт должен функционировать в оригинальной программе, остановленной на OEP, и восстанавливать все эти CALL'ы полностью, чтобы затем можно было вручную скопировать с помощью Binary copy в дамп целую секцию, после чего он должен запуститься. Скрипт должен работать на любом компьютере, поскольку я буду испытывать его у себя. Если за 15 дней данную задачу никто не решит, мне придется сделать это самому, однако надеюсь, что до такого дело не дойдет, хе-хе. Вместе со скриптом следует выслать свое имя или ник. Победитель будет назван в 52-й главе, ибо думаю, что он того заслуживает за такую кропотливую работу.

Уточню еще раз: скрипт должен восстанавливать эти CALL'ы в оригинальной программе, но при этом не производить окончательных Binary copy/Binary paste, поскольку я намереваюсь сделать это вручную. Он должен исправлять вызовы таким образом, чтобы после Copy/Paste дамп работал нормально.

А теперь — подсказка. Посмотрим оригинальную программу, остановленную на OEP:



Если взглянуть на это же место в восстановленном нами дампе, то там инструкции окажутся другими. Вот тот же самый вызов в дампе:



Здесь CALL вызывает API-функцию (в данном случае — GetStartupInfoA), и возврат из нее происходит по адресу 4272DB (так как это 6-байтовая команда). В защищенной ASProtect'ом программе возврат должен происходить по тому же самому адресу. Дело в том, что он заменяется 5-байтовой командой (а 6-й байт — мусорным), и если установить BP на адрес возврата из CALL'а в упакованной ASProtect'ом программе, то он всегда будет на один байт больше адреса показанной нам в листинге следующей инструкции; в данном случае следующая команда отображена так:

```
004272D5 E8 268D5801 CALL 019B0000
004272DA D9F6 FDECSTP
```

Чтобы произошла остановка после возврата из API-функции, следует установить BP на 4272DB. Если трассировать вручную, то можно и поседеть, поэтому при решении данной задачи следует использовать брейки BPM. В какой-то момент своей работы программа должна считать и скопировать байты API-функции в другое место, поэтому следует установить BPM ON ACCESS на первые команды API-функции (GetStartupInfoA в данном случае). Так находится момент считывания правильной API-функции на данном компьютере. Потом нужно найти момент перезаписи инструкции CALL 019B0000, так как после считывания байтов API-функции они копируются в другое место, где и происходит их выполнение. С помощью BPM ON WRITE можно найти момент, когда CALL 19B0000 заменяется CALL'ом в это новое место. Затем останется лишь задействовать скрипт, восстанавливающий CALL: он просто заменит байты, которые должны были сохраниться, правильными, хе-хе.

Такова задача, посмотрим, кто выполнит ее лучше всех. Вам придется немало подумать и хорошо потрудиться, чтобы ваш скрипт работал на любом компьютере. Надеюсь, кто-нибудь решит эту задачу до 17 августа 2006.

Кроме имени победителя и рассмотрения его скрипта, в качестве утешительного приза я упомяну также и именах всех приславших мне рабочие скрипты.

Удачи и Good work!

Файлы к статье (находится в архиве `wasm_files.zip`: `files/0527/ollydbg51.zip`)

Дополнение

Поскольку в данном анпэкми с попаданием на OEP и с плагином OllyBonE сопряжен ряд трудностей, привожу переписку Рикардо с читателями этой главы и его посты на форуме RCE в отдельном файле (находится в архиве `wasm_files.zip`: `files/0527/(Rus)-To-Introduction-51-(OllyBonE).doc`).

Ralba Несмотря на то, что я делаю всё в точности как сказал маэстро в 51-й главе, не могу дойти даже до OEP. После установки break-on-execute мне выдаются два сообщения об ошибке, одно (не самое злостное) — Error 45, а от второго волосы дыбом встают. Такое впечатление, что процессор вот-вот из материнки выскочит, хе-хе. И это не считая того, что я не могу запустить данную версию ASProtect'а из Проводника простым двойным кликом! Так происходит только со мной или есть и другие? Я уж и копировал в различные места на жестком диске, чтобы изменить путь, но всё без толку…**Рикардо Нарваха** Этот упаковщик весьма чувствителен из-за RDTSC. Если он не запускается вне отладчика, то в Olly будет работать еще меньше. **OtupAtpaxa** Этот упаковщик — однозначно гадина в высшей степени, хотя, согласно Рикардо, это наименее защищенная версия. У меня на него почти весь вечер уходит — проще будет написать скрипт, чем заставить его дойти до

OEP! Из 20-ти попыток дойти до OEP удается 1, и ничего с этим не поделаешь. Он просто перемалывает меня в своих жерновах! Кроме того, данный анпэкми, как верно заметил Ralba, не только не работает в Olly, но и не запускается двойным кликом в Windows'е. Я крайне разочарован. Надеюсь, кто-нибудь поможет с ним разобраться :-)**Рикардо Нарваха** Если анпэкми не запускается вне отладчика, нужно скопировать его в другую папку, и там должен заработать. Пробовал новую версию Olly Advanced с включенным драйвером Anti-RDTSC fakerdtsc (метод 1)? Однажды я никак не мог запустить программу; перезагрузил, и проблема исчезла. **DAMIAN**

Otup, это потому что ты не пробовал приаттачиться. …

dapaf

OtupAtpaxa

Привет, Рикардо! Изменять папки я попробовал, однако результат всегда эвристический: 2 раза программа запускается, а 8 раз — нет… Большое спасибо за файлы, отправленные тобой этим вечером, но я их еще не смотрел. Надеюсь, завтрашний день будет более удачным.

dapaf, твоя идея, конечно, хороша, но у меня приаттачиться не получается… Полагаю, что если есть возможнось присоединиться к запущенному (и остановленному) процессу, то можно будет дойти до какого-нибудь вызова антидампа. Это, конечно, лучше, чем ничего.

Рикардо Нарваха Я бы сказал, что проблема скорее в процедуре антидампа, которая переписывает API-функцию из DLL'ки в созданную упаковщиком область памяти и модифицирует уже выполненный CALL, чтобы он указывал на эту область и чтобы после его выполнения было сложно идентифицировать API-функцию, поскольку он больше на нее не указывает. По-моему, трудность заключается именно в этом. **Ralba** Простите, маэстро, но что такое RDTSC? **Рикардо Нарваха** Инструкция RDTSC позволяет определять число циклов, прошедших с момента включения компьютера и может использоваться подобно GetTickCount для определения промежутка времени между двумя вызовами команды, что и применено в исследуемом ASProtect'е, но не два, а несколько раз. Эта команда осложняет процесс взлома и перехват API-функций, поскольку во время ручной трассировки каждый раз при проходе через RDTSC происходит приостановка выполнения, а это влияет на результат и далее начинаются проблемы. В те времена, когда еще не было драйверов Anti-RDTSC, приходилось останавливаться на командах RDTSC (с помощью пошаговой трассировки, либо ища во всём коде инструкции RDTSC и устанавливая на каждую BP, либо запуская условную трассировку Trace into с остановками на RDTSC), а затем после выполнения инструкции изменять значения регистров EAX и EDX, обнуляя или сокращая период времени между двумя вызовами. Теперь же появились специальные драйверы, входящие в состав Olly Advanced под названием Anti-RDTSC (fakerdtsc) — они изменяют в нулевом кольце свойства инструкции, чтобы блокировать ее работу в третьем. Данные драйверы изменяют один флаг, и это дает нам возможность работать с программой в Ring3. Таким образом, если в OllyDbg не задействовать драйвер Olly Advanced fakerdtsc, программа выполняться не будет. Могу предположить, что иногда она будет отказываться работать, если время вычислено очень точно. Так бывает, когда программисты предпринимают отчаянные антиотладочные меры. **OtupAtpaxa** Привет, Рикардо! Действительно, с последней версией Olly Advanced программа работает немного лучше. Архив fakerdtsc.rar теперь уже не столь актуален, так как он содержит файл fakerdtsc.sys, создаваемый во время работы Olly. … Спасибо. **Рикардо Нарваха**

Я добавил несколько файлов, которые можно скачать по прежней ссылке — это конфигурация Olly Advanced, необходимая для сокрытия отладки, поскольку заставить программу выполняться в Olly довольно трудно, а в туториале об этом сказано было недостаточно. Поэтому я перебрал комбинации настроек плагина и в конце концов заставил анпэкми работать. Я использовал Hide Debugger 1.24 со всеми установленными галками и Olly Advanced, настроенный как показано на скриншотах. Если у вас и теперь программа не работает, то вам придется найти свою собственную конфигурацию плагина таким же образом, как это сделал я — просто удаляя и устанавливая галки,

хе-хе.

Введение в крэкинг с нуля, используя OllyDbg - Глава 52

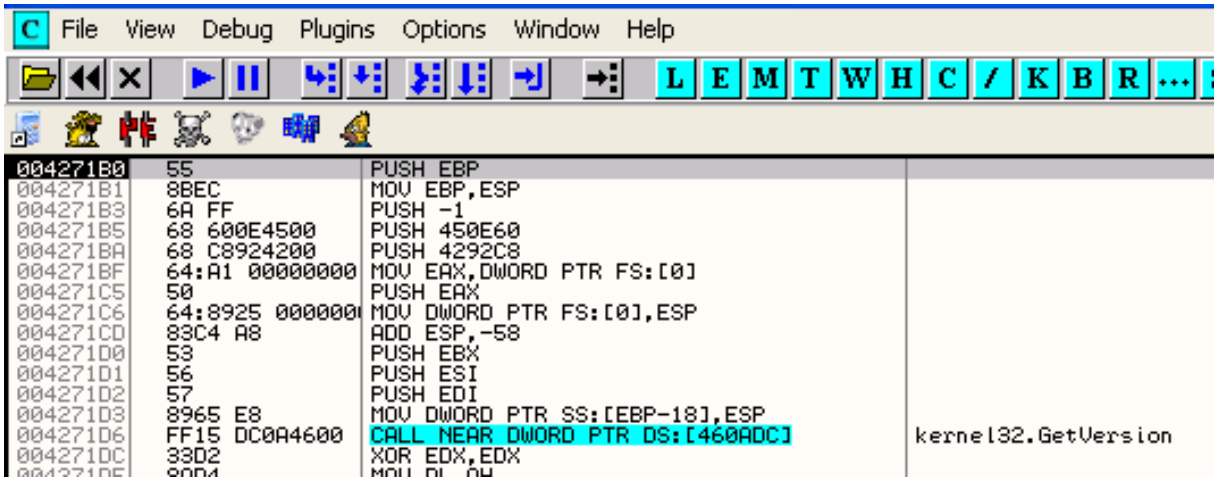
Автор: Рикардо Нарваха, пер. Рома Стремилов

Дата: 2010-07-25

Конкурс завершился, но из-за его трудности в нем приняли участие немногие. Однако победитель всё же объявился — им стал Hiei, написавший скрипт для восстановления CALL'ов в крэкми ASProtect из предыдущей части.

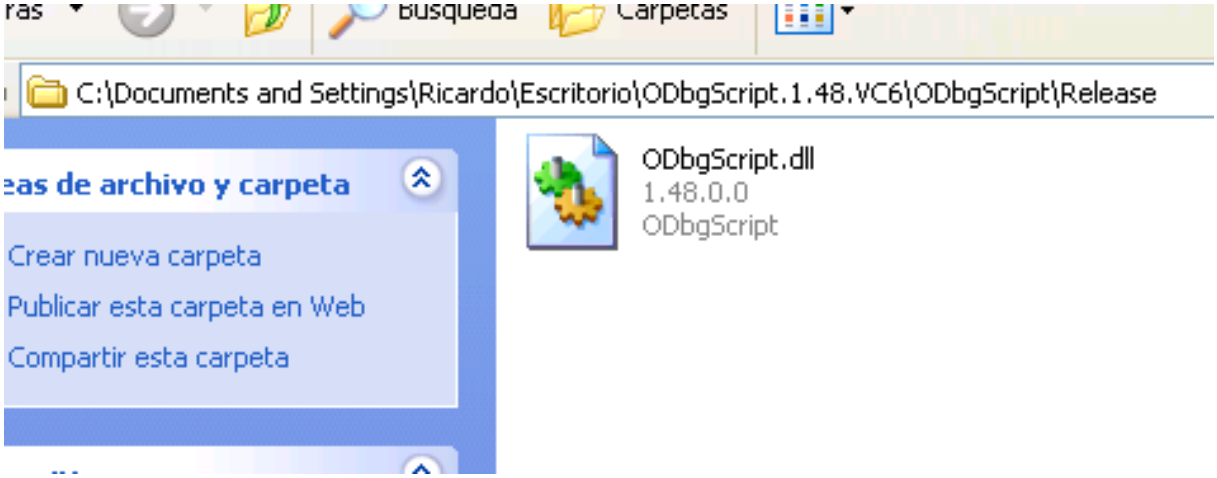
Хотелось бы отметить, что многие ASProtect'овые анпэкми не работают в отладчике даже с установленным плагином Olly Advanced, поскольку при включении драйвера Anti-RDTSC программа работает не на всех компьютерах, а без него анпэкми и вовсе отказывается выполняться. Я заметил, что в последней версии ASProtect'a, примененной в нескольких программах, данная защита не установлена и такие программы в OllyDbg работают нормально. По крайней мере, я еще не видел программ, использующих ее и нуждающихся в Anti-RDTSC для своей работы. Вероятно, это связано с проблемами совместимости, и данная защита снята полностью или ее использование сокращено до минимума из-за ее крайней проблематичности.

У кого же драйвер RDTSC Olly Advanced функционирует нормально, те могут легко дойти до OEP, а затем запустить скрипт, приложенный к данному туториалу.

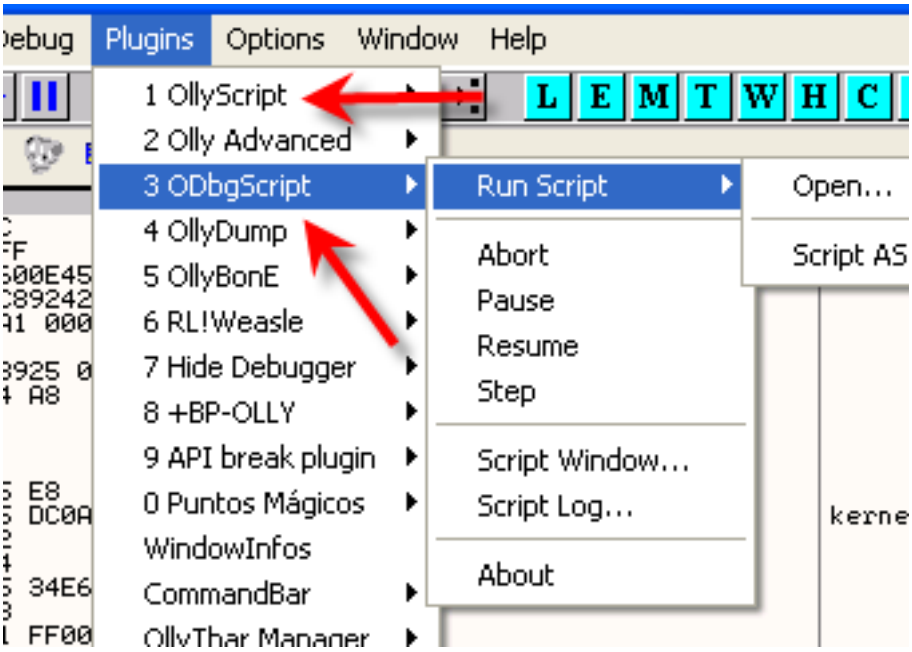


Это OEP, но при попытке запустить скрипт Hiei выдается сообщение, что установленная версия OllyScript устарела и ее надо обновить, поэтому пришлось искать новую, которая теперь называется ODbgScript. Этот проект продолжает развивать Epsylon, так как его первый автор SHaG больше им не занимается.

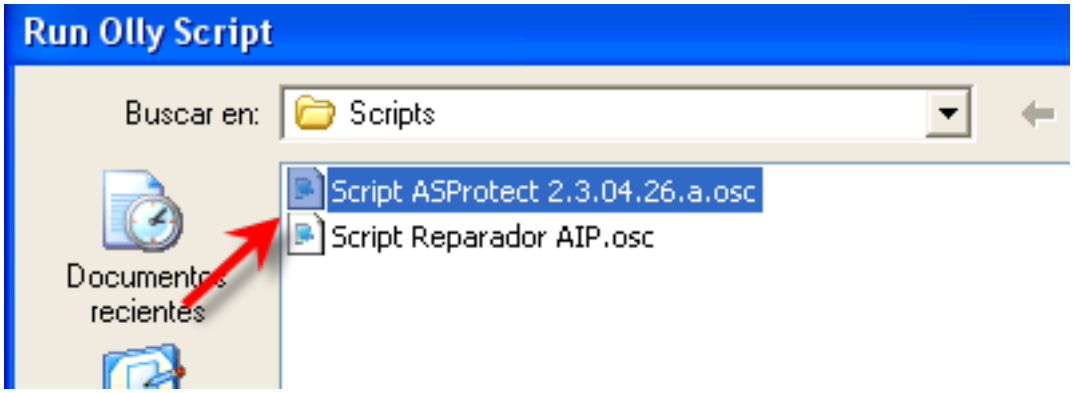
Скопируйте в папку с плагинами приложенную к статье DLL'ку:



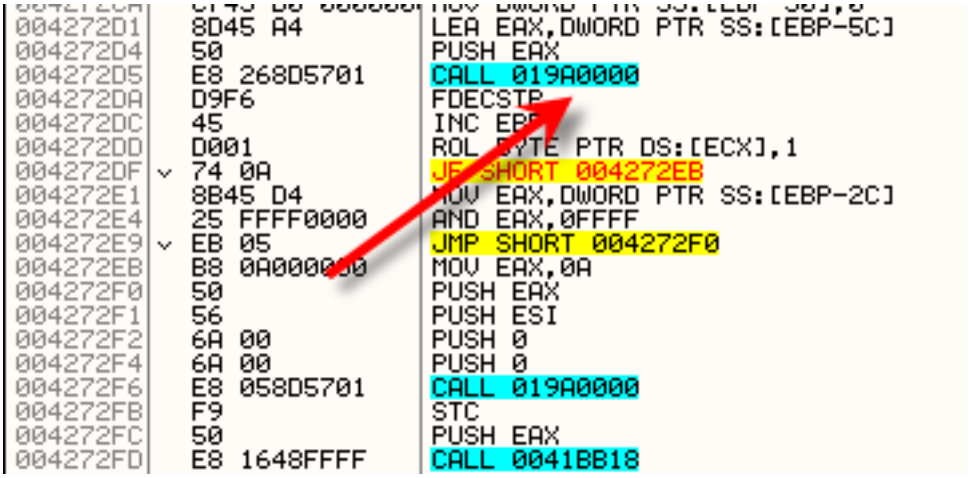
А затем перезапустите Olly и снова дойдите до OEP. Не забудьте деактивировать Break-on-execute, иначе из-за него возникнут проблемы.

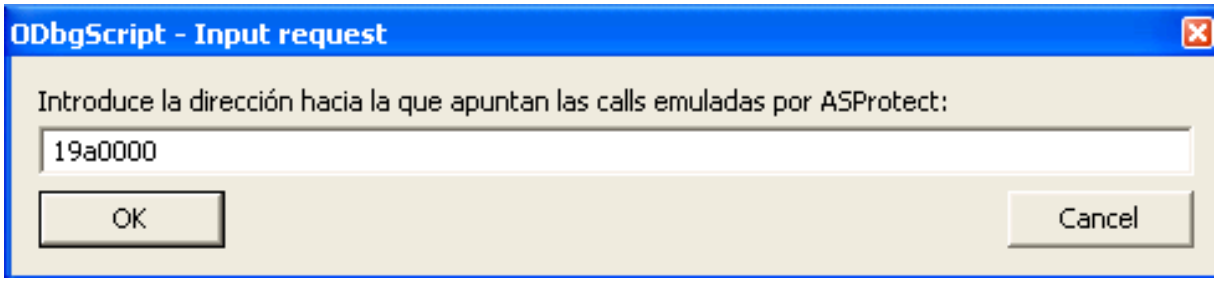


Хотя это всего лишь новая версия OllyScript'а, в меню отображаются оба плагина, поэтому следует выбрать новый и в нем открыть скрипт Hiei:

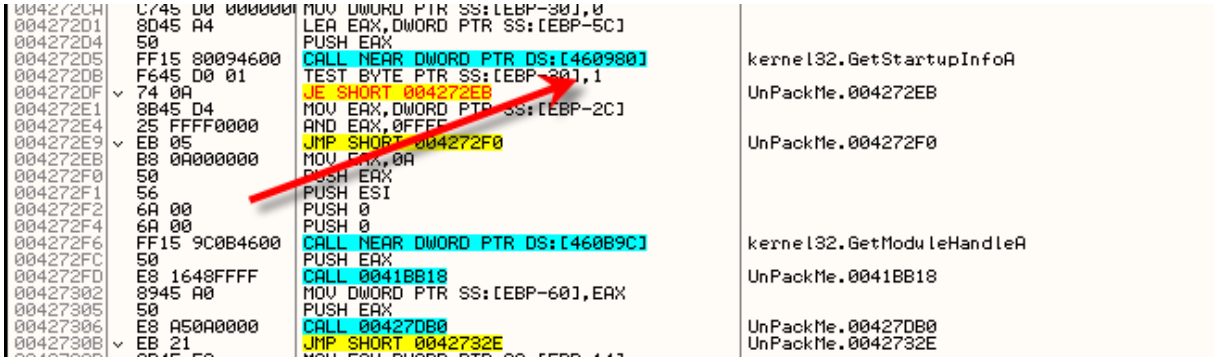


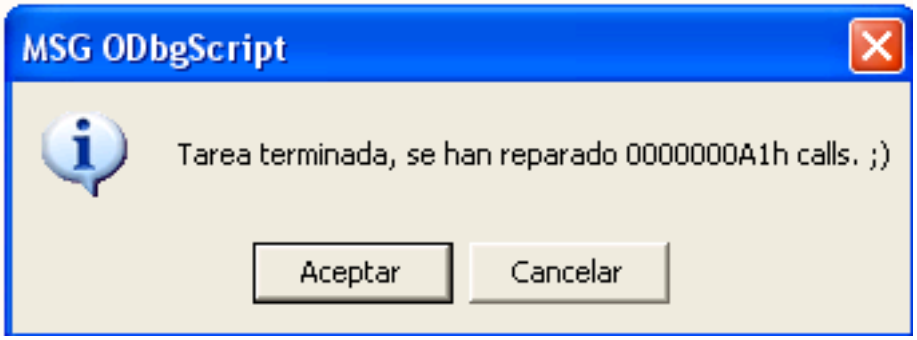
Сразу же после запуска запрашивается адрес, на который направляют проблемные CALL'ы, — на моем компьютере это 19A0000:





После нажатия на OK скрипт начинает свою работу, а немного позже наблюдаем результат:





На скриншоте показан CALL, который раньше указывал на 19A0000 (так на моем компьютере), а теперь он восстановлен вместе со всеми остальными. Hiei вполне справился с поставленной задачей, поздравляю!

Вот скрипт, содержащий комментарии самого автора, а далее привожу мои собственные:

```
------------------------------------------------------------------------------------------------------------
    -------------
/*
-============================================================-
.:[CracksLatinoS]:.
 Автор скрипта: Hiei
 Назначение: удаление защиты AIP в ASProtect SKE v2.3
 Исследуемая программа: UnPackMe_ASProtect.2.3.04.26.a.exe
 Конфигурация: ODbgScript v1.3x или выше; запуск из OEP; пропуск
 всех исключений
 Дата: 5 августа 2006
- = [Комментарий] = -
 Благодарности: Рикардо Нарвахе и marciano (ибо я слегка
 воспользовался его логикой для написания поискового движка)
-============================================================-
*/
var oep
var codebase
var codesize
var base_aspr
var base_aip
var ini_iat
var dir
var dir_iat
var sig
var dest
var api
var cont
cmp $VERSION,"1.30" // Проверка версии OllyScript'a.
jb err_version
ask "Введите основание, на которое ссылаются эмулирующие CALL'ы ASProtect'a:"
cmp $RESULT,0
je salir
mov base_aip,$RESULT // Сохранение введенного адреса.
mov oep,eip  // Сохранение значения OEP.
gmi eip,codebase  // Определение начального адреса секции кода.
mov codebase,$RESULT // Сохранение полученного значения.
gmi eip,codesize  // Определение размера секции кода.
mov codesize,$RESULT // Сохранение полученного значения.
add codesize,codebase // Сложение размера и нач. адреса.
mov ini_iat,460814 // Сохранение адреса начала IAT.
mov base_aspr,[46C048] // В 46C048 сохранено основание секции, где определяется, к каким API-
    функциям должны направлять CALL'ы.
add base_aspr,3B02E // К этому основанию добавляется константа для получения адреса, по которому
    видна API-функция.
bphws base_aspr,"x" // Установка HBP на адрес, где видна API-функция.
jmp buscar
buscar:
find codebase,#E8????????# // Поиск CALL'ов в секции кода.
```

```
cmp $RESULT,0 // Если ни одного CALL’а не найдено, процесс завершается.
je no_calls
mov dir,$RESULT // При обнаружении CALL’а его адрес сохраняется.
mov sig,dir  // Копирование найденного адреса в другую переменную.
mov dest,dir  // Копирование в еще одну переменную.
add sig,5  // sig содержит адрес инструкции, следующей за CALL’ом.
inc dest  // dest инкрементируется для получения опкодов после E8.
mov dest,[dest]  // Извлекается offset, закодированный в CALL’е после опкода E8
add dest,sig  // Теперь dest содержит адрес, к которому ведет CALL.
cmp dest,base_aip // Этот CALL есть в ASProtect’е?
je ejecutar
inc dir  // Если нет, обновляется указатель поиска.
mov codebase,dir // Указатель копируется и поиск повторяется.
jmp buscar
ejecutar:  // CALL ведет к Аспру.
mov eip,dir  // В EIP копируется адрес найденного CALL’а, ведущего в Аспр,
run   // и программа запускается.
eob comprobar  // Если срабатывает BP, управление передается на метку 'comprobar'.
comprobar:
cmp eip,base_aspr // BP находится в ожидаемой области?
jne inesperado
mov api,edx  // Если да, то из EDX сохраняется значение API-функции.
jmp buscar_api  // Далее – поиск API-функции в IAT’е.
buscar_api:
cmp ini_iat, 460F28 // Начало и конец IAT’а равны?
je error  // Если да, то API-функция в IAT’е не найдена.
cmp [ini_iat],api  // Поиск API-функции по значению указателя.
je reparar  // Если API-функция найдена, то ее надо восстановить.
add ini_iat,4  // Если нет – увеличить указатель на 4 во избежание ошибок поиска.
jmp buscar_api  // Продолжить поиск.
reparar:
mov dir_iat,ini_iat // Сохранение адреса IAT, где найдена API-функция.
ref dir  // Поиск ссылок на этот адрес.
cmp $RESULT,0 // Если ссылка есть, то нужно ассемблировать JMP.
jne reparar_jump
eval "Call dword[{dir_iat}]" // В противном случае – CALL.
asm dir,$RESULT
inc cont  // Счетчик для учета кол-ва восстановленных CALL’ов.
inc dir
mov codebase,dir
mov ini_iat,460814 // Обновление указателя, чтобы следующий поиск производился с начала IAT’а.
jmp buscar
reparar_jump:
eval "Jmp dword[{dir_iat}]" // Ассемблирование JMP’а.
asm dir,$RESULT
inc cont
inc dir
mov codebase,dir
mov ini_iat,460814 // Обновление указателя, чтобы следующий поиск производился с начала IAT’а.
jmp buscar
inesperado:
msg "Произошла внезапная остановка. Продолжить?"
cmp $RESULT,0
je salir
run
error:
eval "Ошибка. Следует вручную восстановить CALL по адресу: {dir}h"
msg $RESULT
run
no_calls:
bphwc base_aspr
eval "Работа завершена. Восстановлено {cont}h CALL’ов ;)"
msg $RESULT
jmp salir
err_version:
msg "Ошибка. Требуется более новая версия OllyScript’а"
ret
salir:
bphwc base_aspr
mov eip,oep
ret
```

Думаю, что комментарии, приведенные в самом скрипте, вполне достаточно описывают его работу. Обнаружение процедуры ASProtect'a, определяющей, какая API-функция должна использоваться, работает на любом компьютере, и при этом рекомендованные мной BPM'ы не задействованы.

Здесь происходит извлечение основания секции, созданной процедурой ASProtect'a:

```
mov base_aspr,[46C048] // В 46C048 сохранено основание секции, где определяется, к каким API-
    функциям должны направлять CALL'ы.
add base_aspr,3B02E // К этому основанию добавляется константа для получения адреса, по которому
    видна API-функция.
bphws base_aspr,"x" // Установка HBP на адрес, где видна API-функция.
jmp buscar
```

По адресу программы 46C048 сохранена база той секции, где определяется API-функция. После извлечения база суммируется с константой, и так находится ключевое место на любом компьютере, потому что секции равны, а варьируется лишь начальный адрес. В данном случае ключевое место — 3B02E от начала, и после его вычисления на нем устанавливается HBP для остановок при выполнении.

Затем под меткой buscar происходит поиск всех CALL'ов от начала секции 401000 и определяется, какие из них ведут в область ASProtect'a по введенному нами значению.

```
ejecutar:  // CALL ведет к Аспру.
mov eip,dir  // В eip копируется адрес найденного CALL'а, ведущего в Аспр,
run   // и программа запускается.
eob comprobar  // Если срабатывает BP, управление передается на метку 'comprobar'.
```

Здесь предусмотрена проверка BP на случай, если кто-нибудь забыл удалить отработанный BP или Break-on-execute, что могло привести к исключениям и нарушению в работе скрипта:

```
comprobar:
cmp eip,base_aspr // BP находится в ожидаемой области?
jne inesperado
mov api,edx  // Если да, то из EDX сохраняется значение API-функции.
jmp buscar_api  // Далее – поиск API-функции в IAT'е.
```

Если это ожидаемая область (то есть место, показывающее API-функцию), то в IAT'е должен быть элемент, соответствующий найденной API-функции.

```
buscar_api:
cmp ini_iat, 460F28 // Начало и конец IAT'а равны?
je error  // Если да, то API-функция в IAT'е не найдена.
cmp [ini_iat],api  // Поиск API-функции по значению указателя.
je reparar  // Если API-функция найдена, то ее надо восстановить.
add ini_iat,4  // Если нет – увеличить указатель на 4 во избежание ошибок поиска.
jmp buscar_api  // Продолжить поиск.
```

Здесь выполняется поиск соответствующего элемента во всей IAT, а после его нахождения происходит переход на метку reparar, где восстанавливаются CALL'ы или JMP'ы, направляющие в область ASProtect'a. Найденная API-функция верна и соответствует элементу IAT'a. Если требуется восстановить CALL, то он заменяется косвенным CALL'ом, берущим значение из упомянутого элемента IAT'a, а если JMP, то ассемблируется косвенный JMP и повторяется всё с начала. И так происходит до тех пор, пока ничего, требующего восстановления, обнаружено не будет, то есть всё уже исправлено.

Рассмотренный скрипт работает хорошо и написан довольно ясно. Помимо наличия комментариев, он также оформлен в виде блоков. Ведь так мало авторов утруждают себя подобными вещами, облегчающими анализ функционирования скриптов. Hiei, ты выиграл путевку в путешествие, хе-хе, но чтобы получить ее, тебе придется приехать сюда, ха-ха-ха. Большое спасибо за проделанный труд!

Следующий конкурс будет проще и состоять будет из двух частей (может быть на этот раз объявится большее число участников!) В первой части нужно написать скрипт к TPPpack для

прибытия в OEP и восстановления украденных байтов, что сделать совсем не трудно. Рассматриваемый анпэкми приложен к статье, и, кроме того, есть туториал marciano к Конкурсу-97, где объясняется, как это выполнить вручную. Так что всё предельно просто. Однако использовать можно только плагины Hide Debugger 1.24, HideOD и OllyScript (лучше его последнюю версию ODbgScript) и ничего более — это требуется в целях унификации.

Во второй части нужно написать скрипт для восстановления IAT TPPpack. Оба скрипта самостоятельны и могут быть представлены в двух частях, либо вместе — кто как хочет.

Иными словами, задача такая:

• 1-я часть: скрипт для поиска OEP и восстановления украденных байтов.
• 2-я часть: скрипт для полного восстановления IAT'а.

Допускается использование только 3-х упомянутых плагинов (но не стоит спрашивать, нужно ли отключать CommandBar, xe-xe). Решения следует отправить до 30 августа. Напоминаю, что их можно прислать как по-отдельностьи, так и вместе.

Туториал marciano скоро появится на моем сайте здесь.

Спасибо за участие и до встречи в 53-ей главе!

Файлы к статье (находится в архиве `wasm_files.zip`: `files/0528/ollydbg52.zip`)

Введение в крэкинг с нуля, используя OllyDbg - Глава 53

Автор: Рикардо Нарваха, пер. Aquila

Дата: 2010-10-21

Хорошо, победитель обоих частей конкурса из прошлой части - это Ularteck, который прислал оба скрипта и объяснение первой части в виде туториала, который мы используем здесь. Вот скрпит-победитель первой часчти, который предназначается для починки украденных байтов. Объяснение идёт ниже.

```
/*
##########################################################################################################
    ###########

                                          CracksLatinoS - 2006

    Скрипт создан: Ulaterck.

    Описание: Скрипт создан для курса «Введение в крэкинг с нуля, используя OllyDbg» Рикардо
    Нарвахи, главы 52-ой. Он предназначается для первой части: ищет OEP и чинит украденные байты.

    Цель: UnPackMe_TPPpack.exe
    Требования: ODBGScript 1.48 , HideDebugger 1.24 , HideOD,  остановленные на точке входа,  снять
    все случаи исключений меньше KERNEL32, так как используемый метод – это метод исключений
    Рикардо.

                - Перед запуском скрпита отметьте последнее найденное исключение, так как скрипт
    этого ожидает.

В любом случае смотрите объяснение, которое находится ниже.

    ##########################################################################################################
    ###############
*/

var dir_excep
var Newoep
var dir_JMP

var dir_CALL
var oep
var StartScan
var Opcodes
var temp
var temp2
var temp3

Datos:
mov Newoep, eip    // Сохраняем адрес точки входа
      // Отмечаем последний адрес скрипта
ask "Введите последнее исключение" //  Выводить окно для ввода данных.
cmp $RESULT,0 // Сравниваем, был ли введён какой-либо адрес.
je aviso                                  // Если нет, то прыгаем на метку aviso
mov dir_excep, $RESULT // Если ввели адрес сохраняем его в dir_excep.
jmp Inicio   // Прыгаем на метку Inicio, чтобы продолжить.

aviso:
msg "Выполняем скрипт заново – введите правильный адрес"
jmp final

Inicio:
run    // Запускаем программу
eoe verifica   // Если происходит исключение, переходим на метку verifica.

verifica:
```

```
cmp eip,dir_excep  // как только мы здесь из-за исключения, смотрим, последнее ли оно.
je ultima   // Если да, то переходим на метку ultima.
esto    // Если нет, то запускаем SHIFT + F9, чтобы пройти исключение.
jmp Inicio:   // И переходи на метку Inicio для поиска других исключений

ultima:
findop eip,#FFE0#  // Попав на последнее исключение, ищем переход  JMP EAX  на украденный код.
mov dir_JMP,$RESULT // Как нашли – сохраняем адрес в dir_JMP
bp dir_JMP   // Устанавливаем точку останов (F2) на переход JMP EAX.
esto    // Проходим исключениес помощью SHIFT + F9, чтобы оказаться в точке останова
bc dir_JMP   // Снимаем точку останова с перехода JMP EAX.
sti    // Запускаем F7, чтобы оказаться в украденном коде.

mov oep,eip   // Сохраняем адрес украденного OEP.
mov StartScan,eip  // Сохраняем адрес OEP также в StartScan, который используется для поиска
    Call'ов.

BuscarCall:   // Также начинаем искать прямые вызовы, чтобы починить их, когда будет создаваться
    двоичная копия, и прикрепить их к новому OEP.
    // Посмотрите MIniTuto.PARTE1.doc

 findop StartScan,#E8# // Ищем прямые вызовы, которые начинаются с опкода E8
 cmp $RESULT, 0  // Когда их больше нет, $RESULT равен 0 и переходим на
    // метку final.
 je final
 mov dir_CALL, $RESULT // Сохраняем адрес первого встреченного call (вызова).
 mov StartScan, $RESULT // Задаём, откуда ходим искать вызовы, в данном случае – оттуда, где был
    встречен первый вызов.
 add dir_CALL,1  // К адресу встреченного вызова прибавляем 1, чтобы миновать опкод E8.
 mov Opcodes, [dir_CALL] // Перемещаем опкоды из этого адреса в Opcodes.
 add Opcodes,StartScan // К этим опкодам прибавляем адрес вызова.
 add Opcodes,5  // К результату прибавляем 5 и получаем адрес, который указывает на вызов.
    // Для пояснений смотрите MiniTuto.PARTE1.doc.

// Теперь назначаем встреченному вызову новый опкод, чтобы при создании двоичного
// файла и вставке нового OEP этот вызов был починен.

mov temp, StartScan        // Помещаем адрес, содержащий StartScan, во временную переменную. (
    StartScan = адрес встреченного вызова.)
sub temp, oep       // От адреса вызова на починку отнимаем адрес OEP украденного кода.
mov temp2, Newoep       // Перемещаем адрес нового OEP (EntryPoint) в вторую временную переменную.
add temp,temp2       // К ней прибавляем значение первой.
sub Opcodes, temp  // Теперь отнимаем результат от адреса, на который указывает CALL.
sub Opcodes, 5  // И от того, что остаётся, отнимаем (в оригинале – прибавляем, но это не
    соответствует команде) 5, получая необходимые для починки этого вызова опкоды.

Editar:   // Начинаем редактировать вызов с помощью новых опкодов.
mov temp3, StartScan // Помещаем в третью временную переменную адрес вызова для починки,
    содержащийся в StartScan.
add temp3,1   // К адресу вызова прибавляем 1, чтобы не учитывать опкод E8.
mov [temp3], Opcodes // Чиним вызов новыми опкодами.
jmp BuscarCall

final:
ret
```

Теперь продолжаем объяснение, как всё это работает, написанное Ularteck'ом.

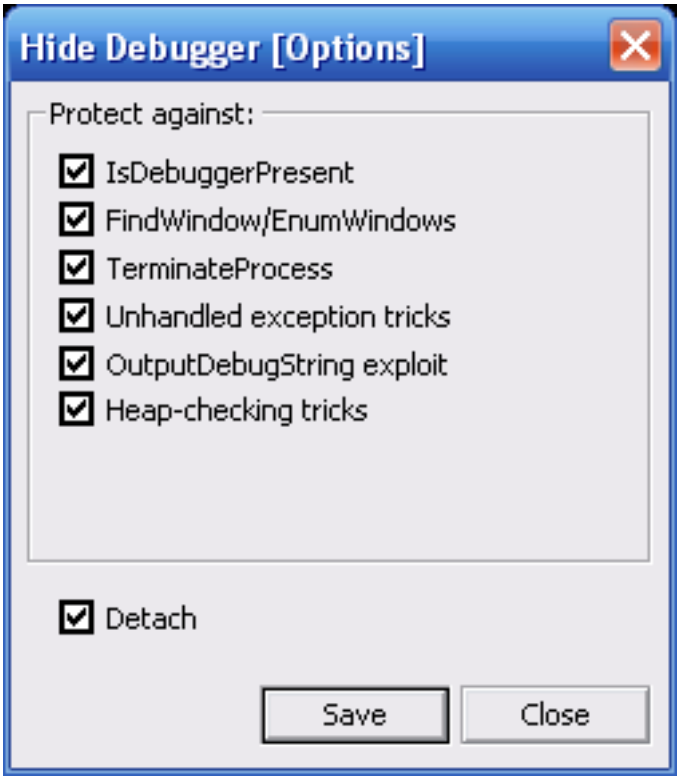
Часть 1: Ищем OEP и чиним украденные байты (написано Ularteck'ом)

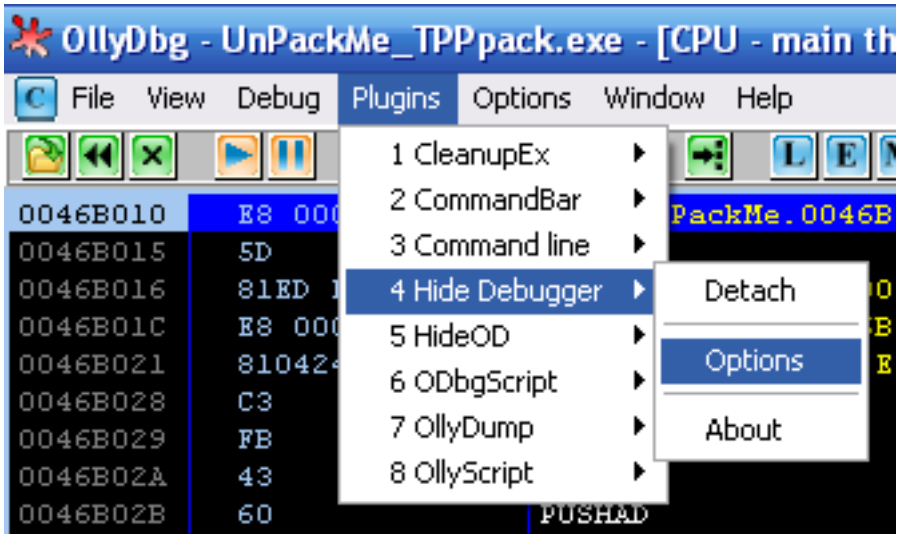
Введение:

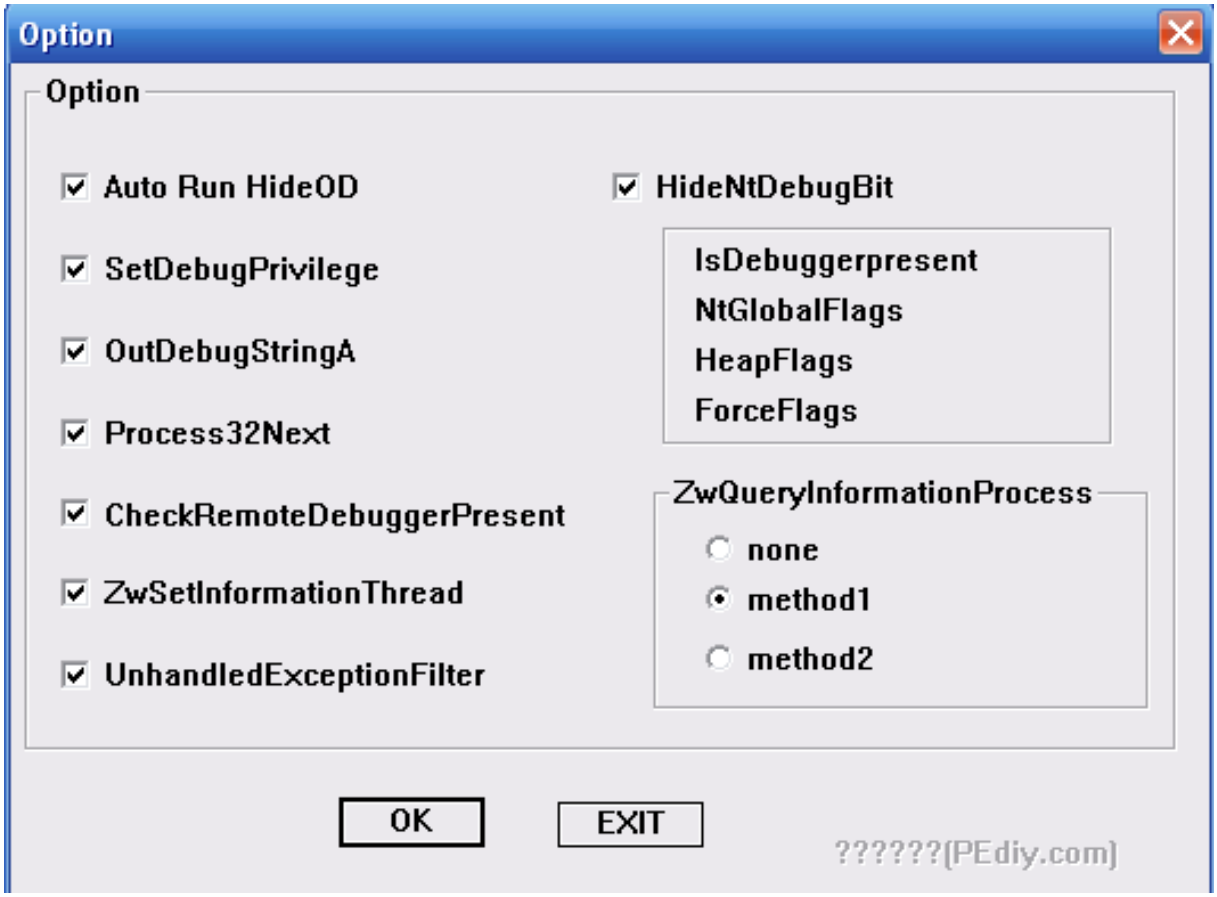
Хотя всё уже было объяснено в туториале №4 курса C97, написанного marciano, поясню некоторые вещи, чтобы пояснить как выполняется скрипт.

Marciano говорил об ошибке из-за антиотладочного OpenProcess'a, хотя у меня она не возникала.

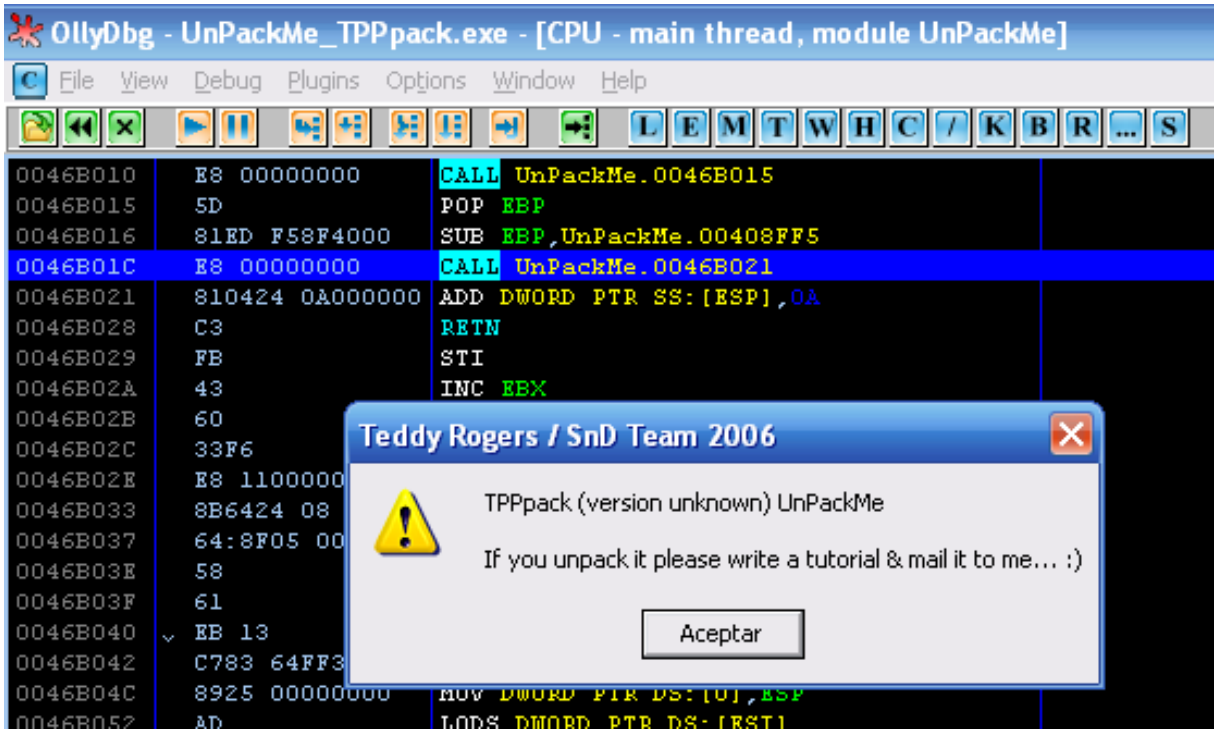
Вот конфигурация используемых плагинов.


Hide Debugger [Options]
Protect against:
IsDebuggerPresent
FindWindow/EnumWindows
TerminateProcess
Unhandled exception tricks
OutputDebugString exploit
Heap-checking tricks
Detach
Save
Close


OllyDbg - UnPackMe_TPPpack.exe - [CPU - main th
File View Debug Plugins Options Window Help
1 CleanupEx
2 CommandBar
3 Command line
4 Hide Debugger
5 HideOD
6 ODbgScript
7 OllyDump
8 OllyScript
Detach
Options
About
0046B010 E8 000
0046B015 5D
0046B016 81ED
0046B01C E8 000
0046B021 81042
0046B028 C3
0046B029 FB
0046B02A 43
0046B02B 60
PUSHAD

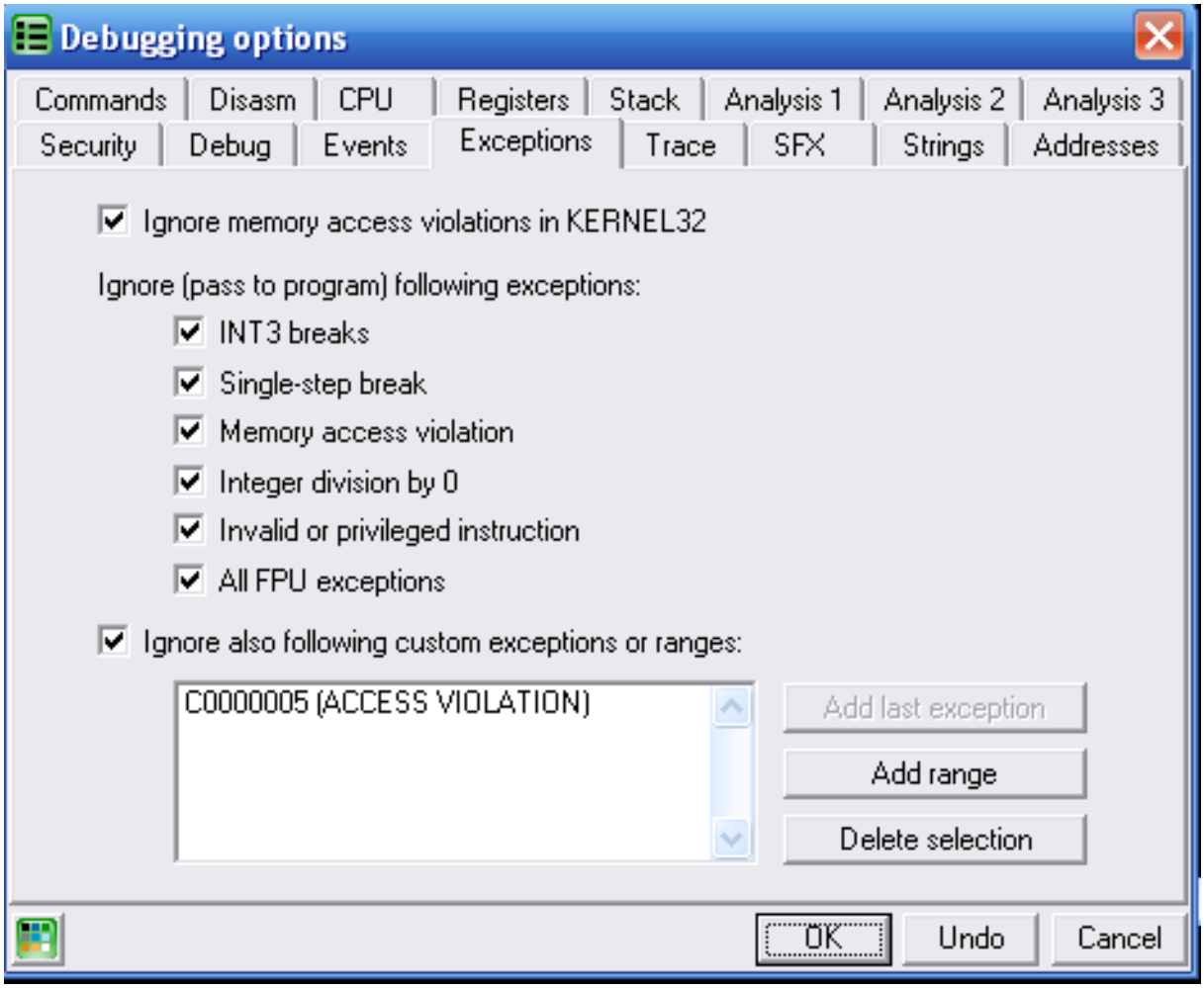


И с помощью этого крэкми у меня запустилось превосходно.



Метод, использованный для нахождения OEP для этой программы, мне понравился, так как часть встречается в новых версиях Asprotect, таких как 2.1 SKE , 2.2 SKE и 2.3 SKE, в которых есть украденный код

Конфигурируем исключения.

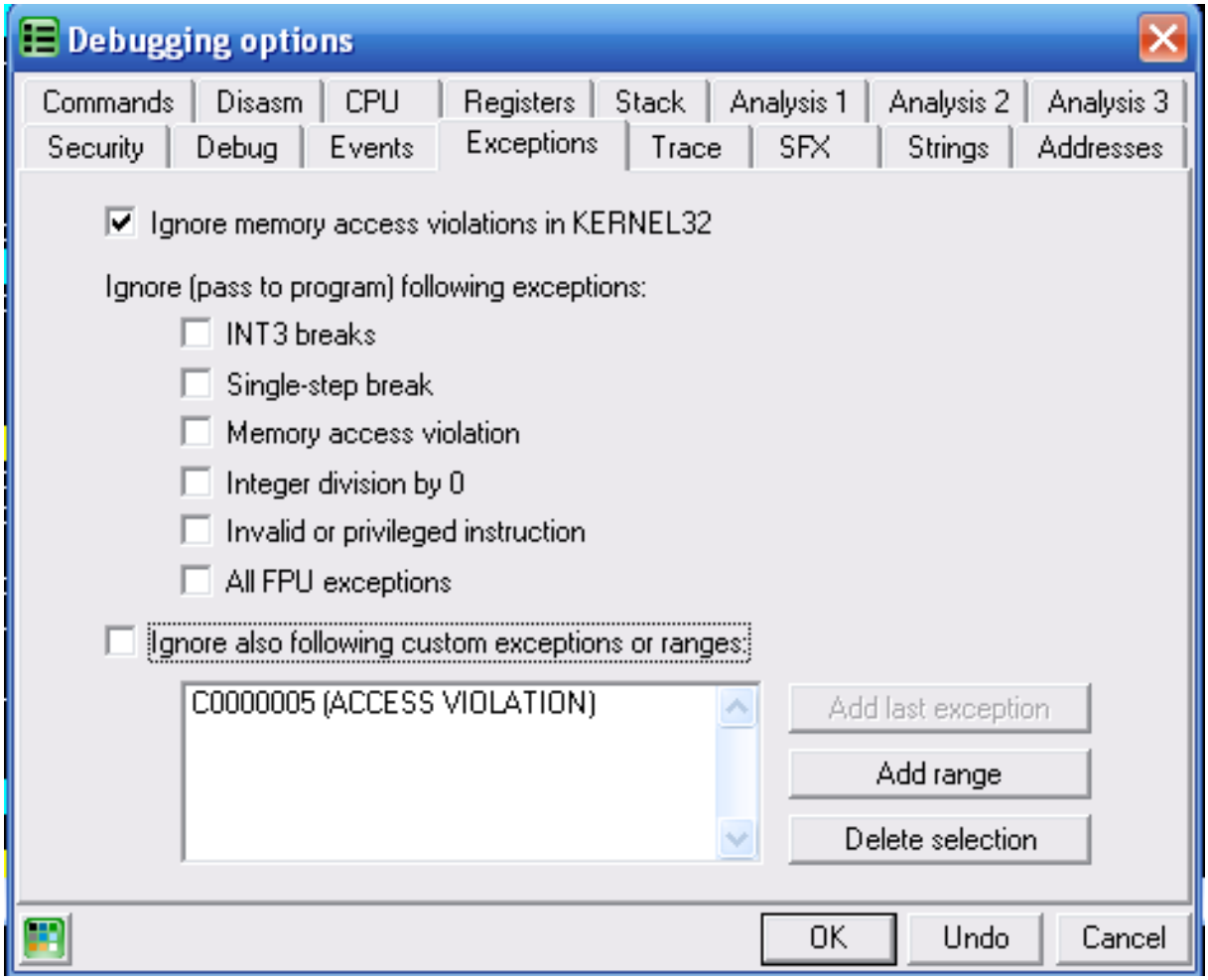


Запускаем программу, и как только она начинается выполняться, идём в Log, чтобы посмотреть, каким было последнее исключение.

В моём случае это 0046D36B, что и будем использовать для скрипта, однако сейчас выполним ручную процедуру по поиску OEP.

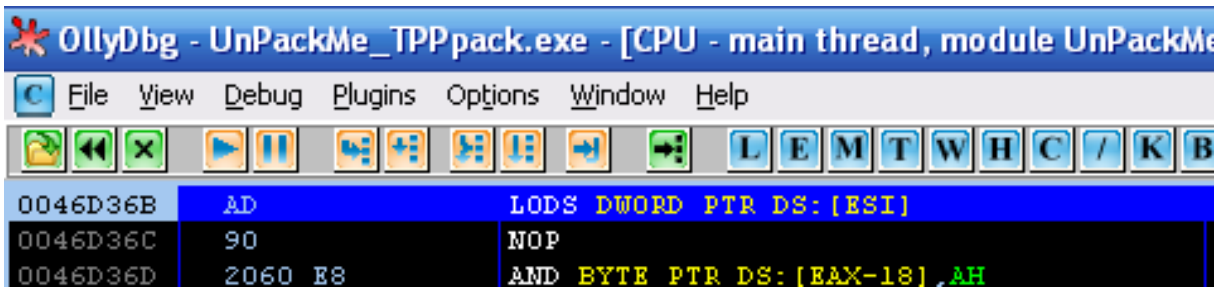
Перезапускаем.

Нажимаем ALT+O.



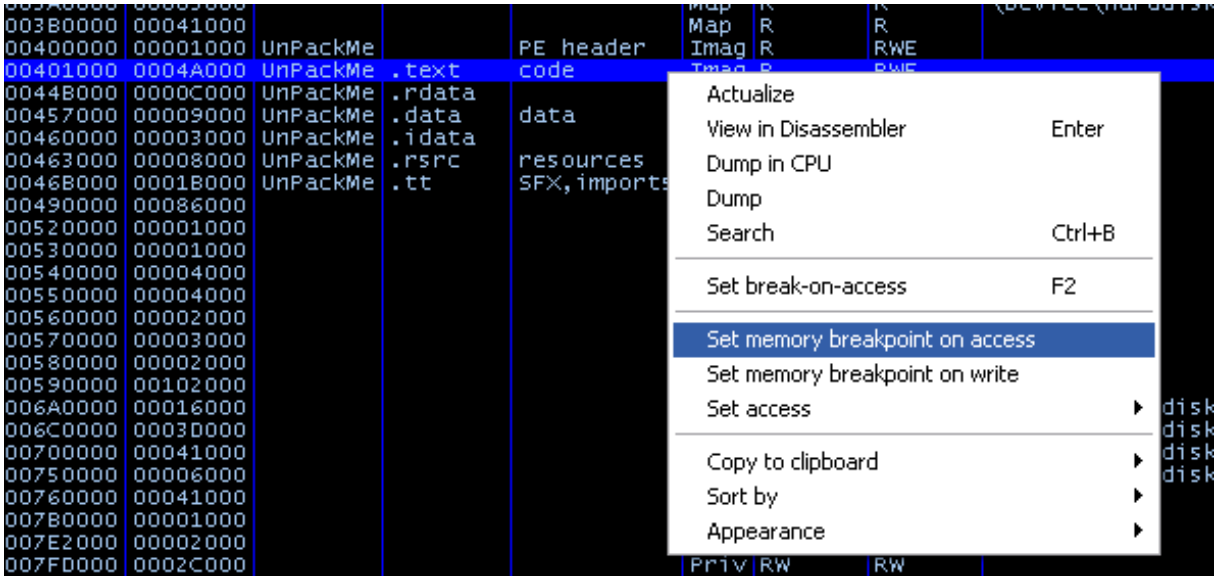
И конфигурируем согласно картинке выше.

Запускаем программу с помощью F9 и проходим исключения с помощью SHIFT + F9, пока не окажемся в исключении по адресу 0046D36B, то есть в последнем

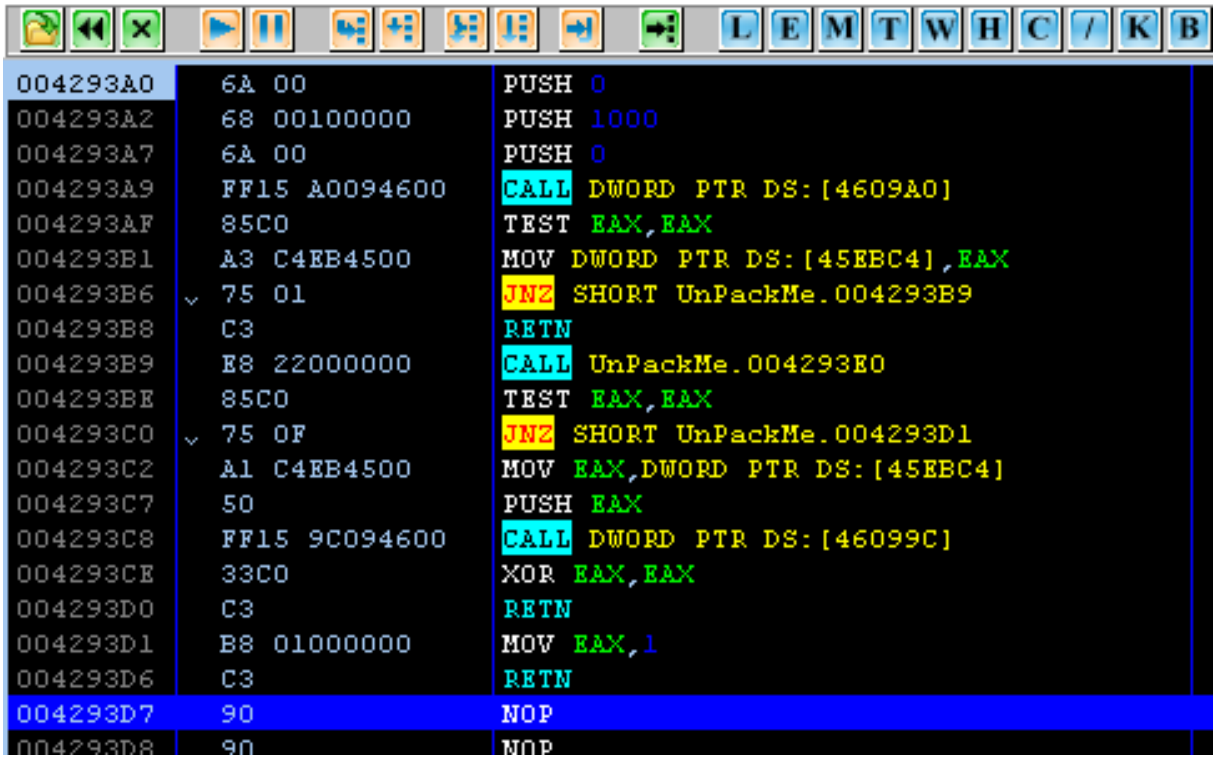


Теперь, находясь на последнем исключении, нажимаем ALT + M.

И устанавливаем точку останова на доступ к памяти в секции кода.

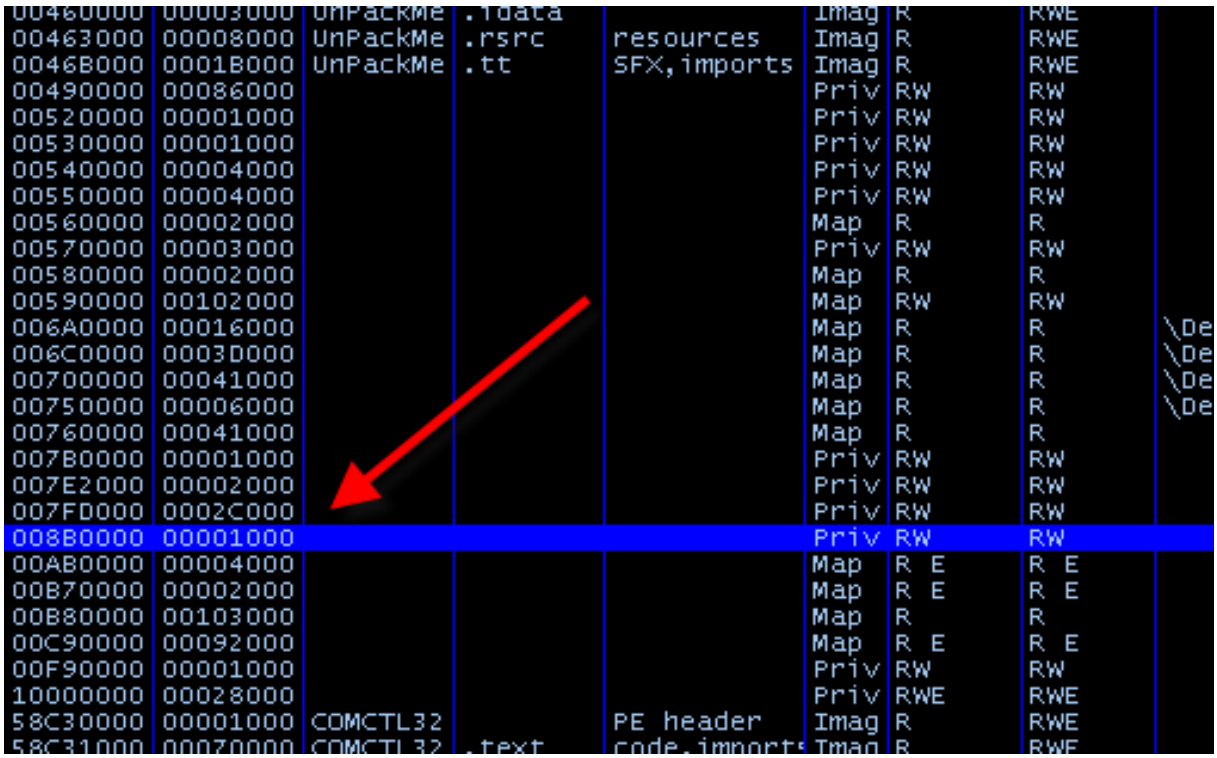


И нажимаем SHIFT + F9, чтобы пройти исключение и оказаться здесь.



Но это это не OEP. Если посмотрим на стек.

Видим, что это исполняемый код. Выполненный код находится по адресу 8B0EA4 в секции 8B0000.



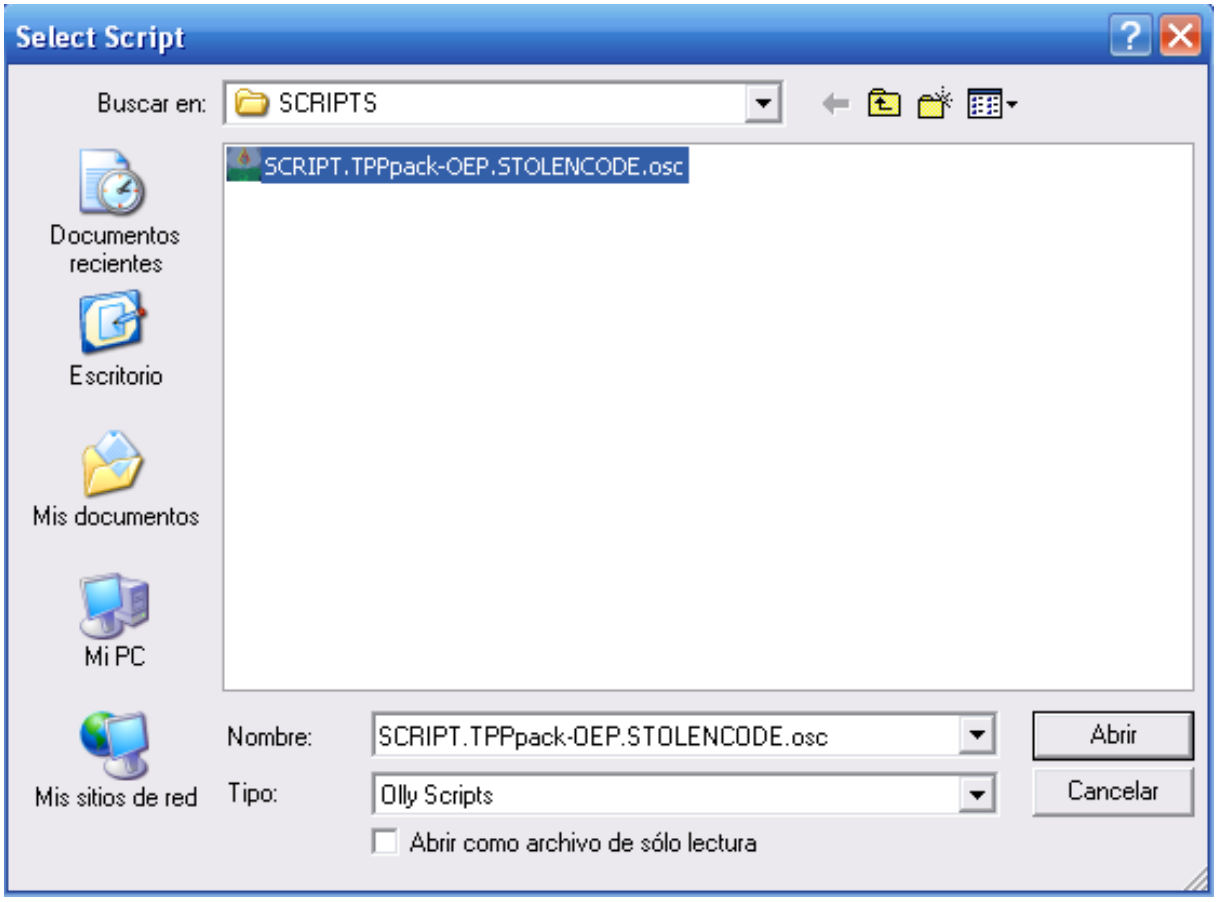
00460000	00003000	UnPackMe	.idata		Imag	R	RWE
00463000	00008000	UnPackMe	.rsrc	resources	Imag	R	RWE
0046B000	0001B000	UnPackMe	.tt	SFX,imports	Imag	R	RWE
00490000	00086000				Priv	RW	RW
00520000	00001000				Priv	RW	RW
00530000	00001000				Priv	RW	RW
00540000	00004000				Priv	RW	RW
00550000	00004000				Priv	RW	RW
00560000	00002000				Map	R	R
00570000	00003000				Priv	RW	RW
00580000	00002000				Map	R	R
00590000	00102000				Map	RW	RW
006A0000	00016000				Map	R	R
006C0000	0003D000				Map	R	R
00700000	00041000				Map	R	R
00750000	00006000				Map	R	R
00760000	00041000				Map	R	R
007B0000	00001000				Priv	RW	RW
007E2000	00002000				Priv	RW	RW
007FD000	0002C000				Priv	RW	RW
008B0000	00001000				Priv	RW	RW
00AB0000	00004000				Map	R E	R E
00B70000	00002000				Map	R E	R E
00B80000	00103000				Map	R	R
00C90000	00092000				Map	R E	R E
00F90000	00001000				Priv	RW	RW
10000000	00028000				Priv	RWE	RWE
58C30000	00001000	COMCTL32		PE header	Imag	R	RWE
58C31000	00070000	COMCTL32	.text	code,imports	Imag	R	RWE

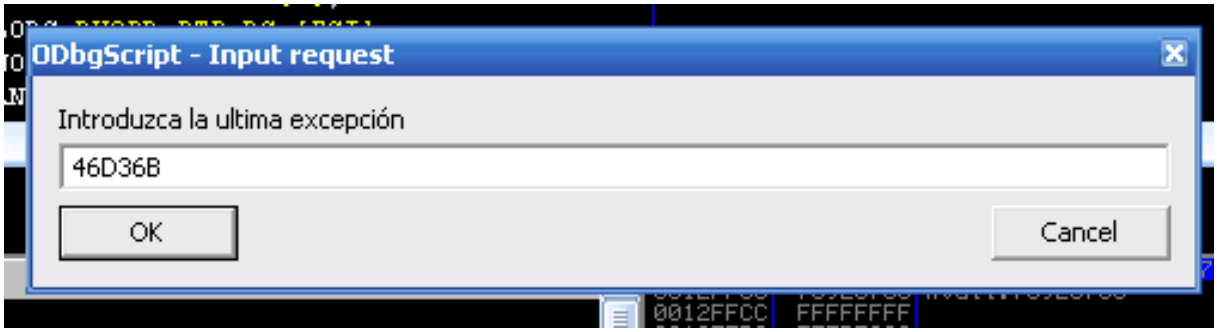
Так что перезапускаем. Делаем ту же процедуру, но точку останова устанавляем теперь на доступ к этой секции. Начинаем делать скрипт для автоматизации задач.

```
var dir_excep
var Newoep

Datos:
mov Newoep, eip                         // Guardamos la dirección del EntryPoint.
                                        // Anotando la ultima dirección del Script
ask "Introduzca la ultima excepción"    // Sacamos la cajita de dialogo para introducir el dato.
cmp $RESULT,0                           // Comparamos si se ha introducino alguna dirección.
je aviso                                // Si no nos dirijimos a la etiqueta aviso
mov dir_excep, $RESULT                  // Si introducimos una dirección la guardamos en dir_excep.
jmp Inicio                              // Saltamos a la etiqueta Inicio para comenzar.

aviso:
msg "Ejecute de nuevo el script e introduzca una dirección valida."
jmp final

Inicio:
run                         // Ejecutamos el programa
eoe verifica                // Si se produce una excepción nos dirijimos a la etiqueta verifica.
```

```
verifica:
cmp eip,dir_excep           // una vez que estemos aquí por una excepción comprobamos que sea la ultima.
je ultima                   // Si lo es nos vamos a la etiqueta ultima.
esto                        // Si no ejecuta SHIFT + F9 para pasar la excepción.
jmp Inicio:                 // Y saltamos  a la etiqueta Inicio para buscar otra excepción.

ultima:

final:
ret
```

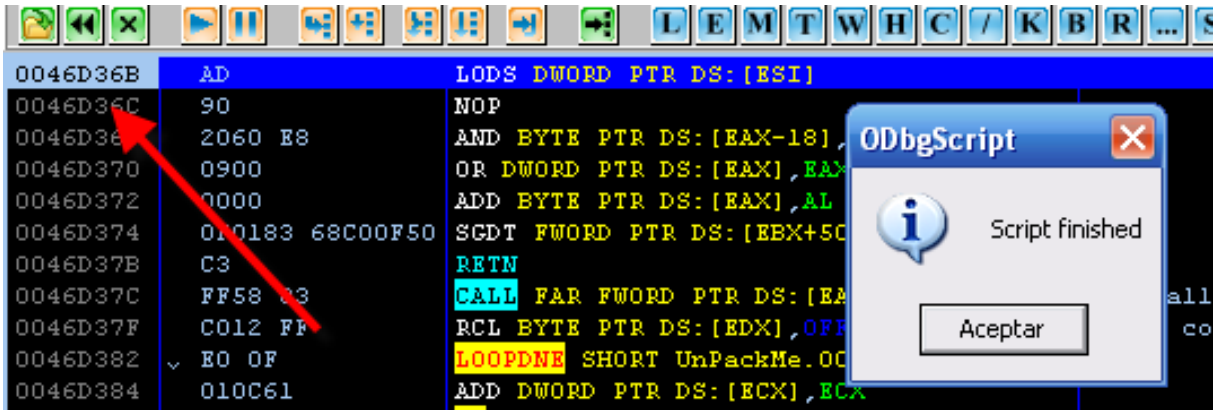
После этого перезапускаем Olly и запускаем скрипт.





Нам показывается диалоговое окно, в котором нужно указать последнее исключение. В моём случае это 46d36b.

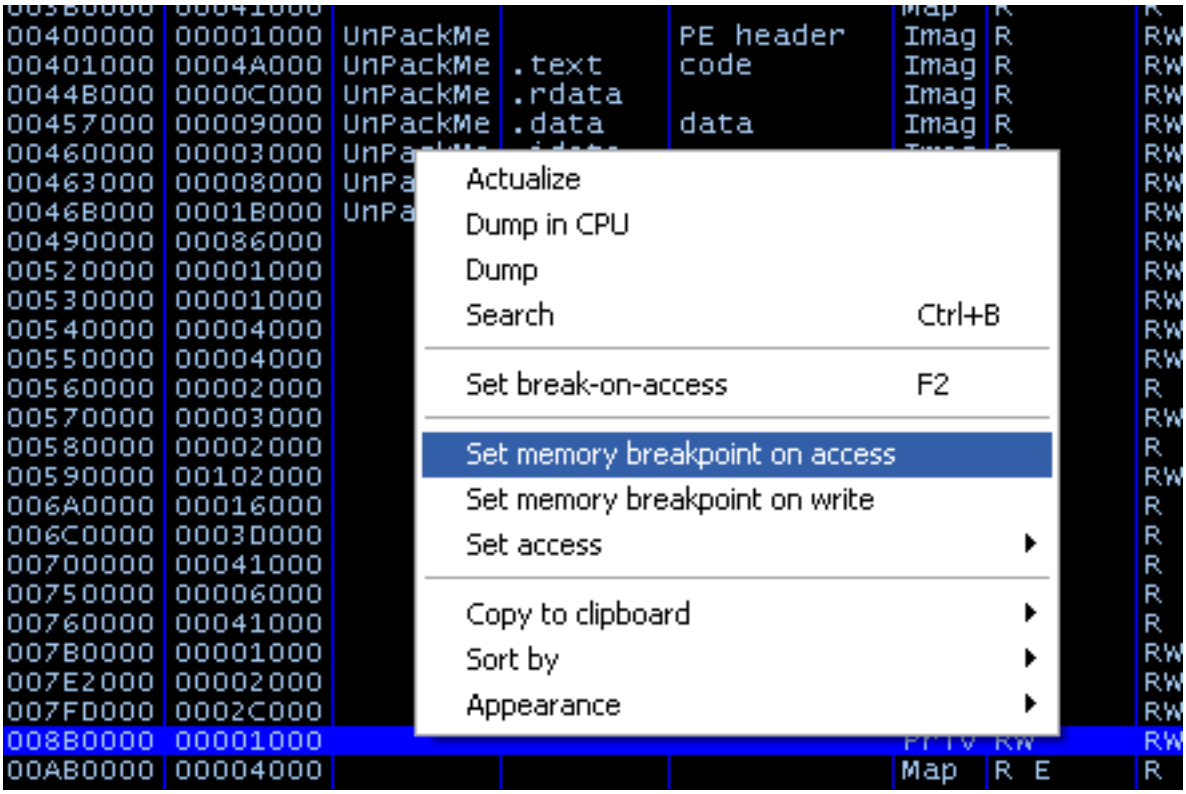
Нажимаем OK.



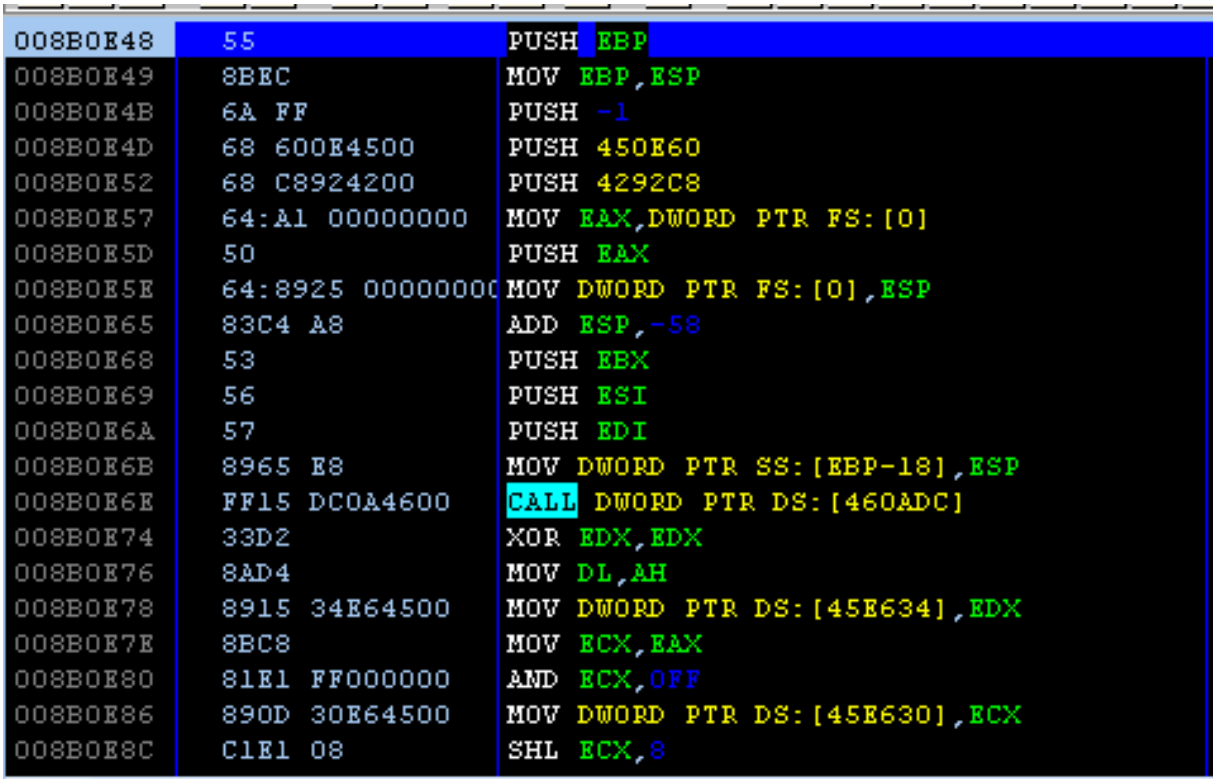
Y al momento termina cuando está en la ultima excepción. Presionamos ALT + M

И когда оказываемся в этом исключении – выполнение останавливается.

Нажимаем ALT + M.



И вместо установки BP на доступ к секции кода, делаем это на секцию где бы выполненный код. Нажимаем SHIFT + F9 и оказываемся в украденном коде.

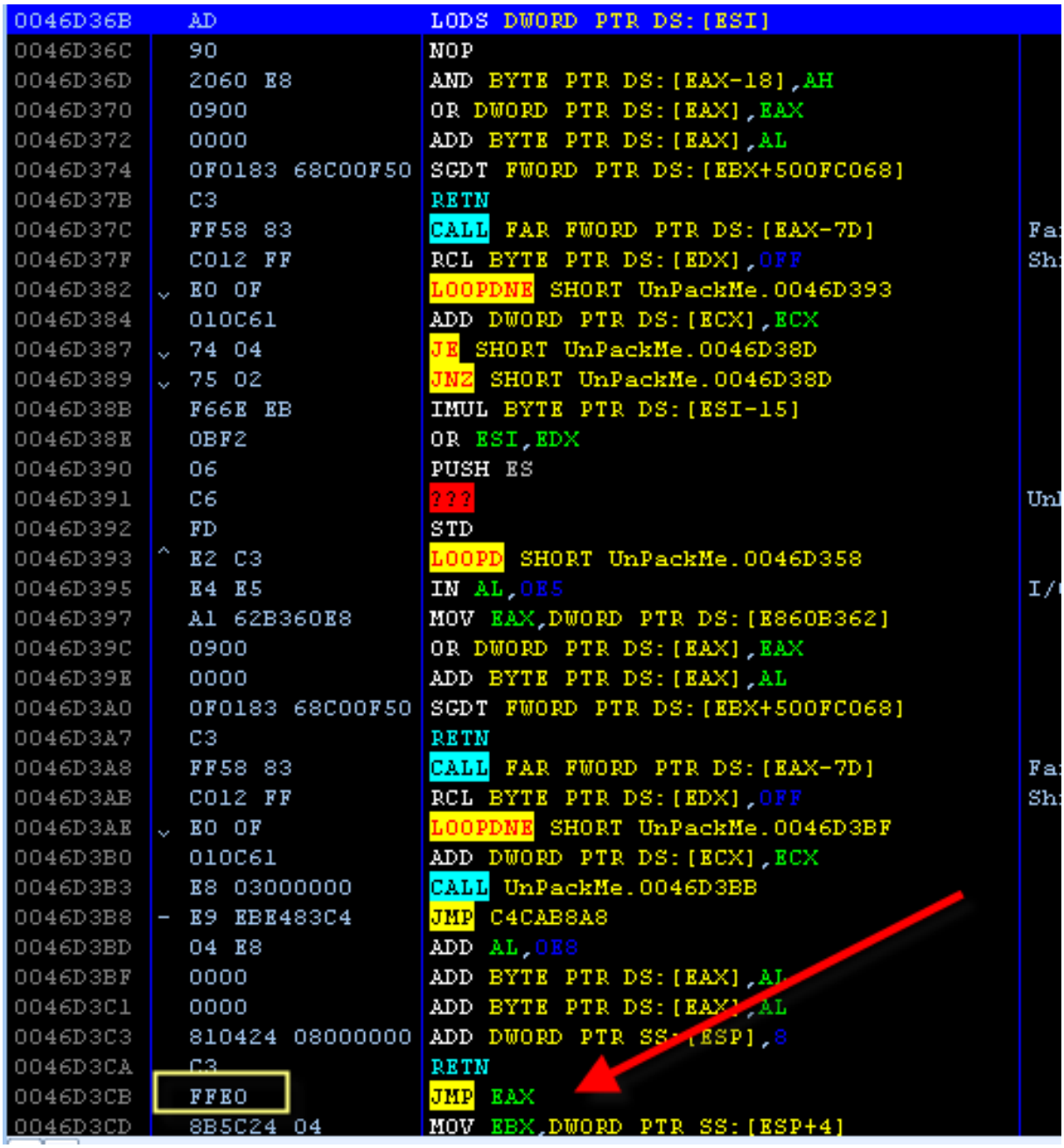


Si presionamos la tecla (-) vemos que nos sitúa en la ultima excepción

Если нажмём клавишу «-», то увидим, что находимся в последнем исключении.

И ниже находится переход, о котором нам говорил marciano, что он который ожидаемо переносит нас на OEP, и как только мы его нашли сразу модифицируем скрипт, чтобы когда в последнем исключении найдётся этот переход, на него среди опкодов FFEO был установлен точка останова, и когда находимся на переходе, мы нажимаем F7, чтобы перейти на украденный код.

В секции переменных скрипта добавляем ещё одну:



dir_JMP, в которой будет храниться адрес перехода JMP EAX.

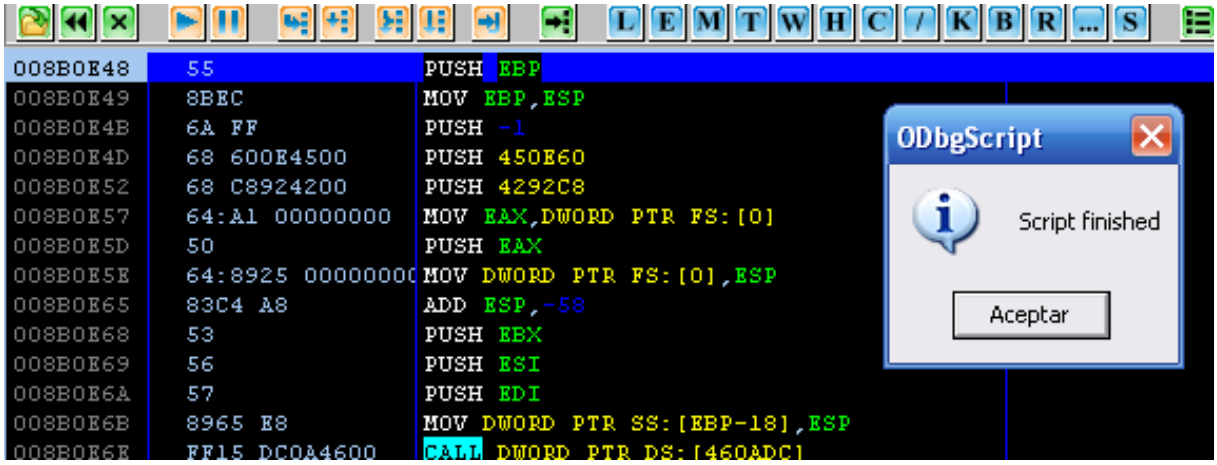
И в метке ultima добавляем следующее:

```
var dir_excep
var Newoep
var dir_JMP
Datos:
mov Newoep, eip
```

Пробуем.

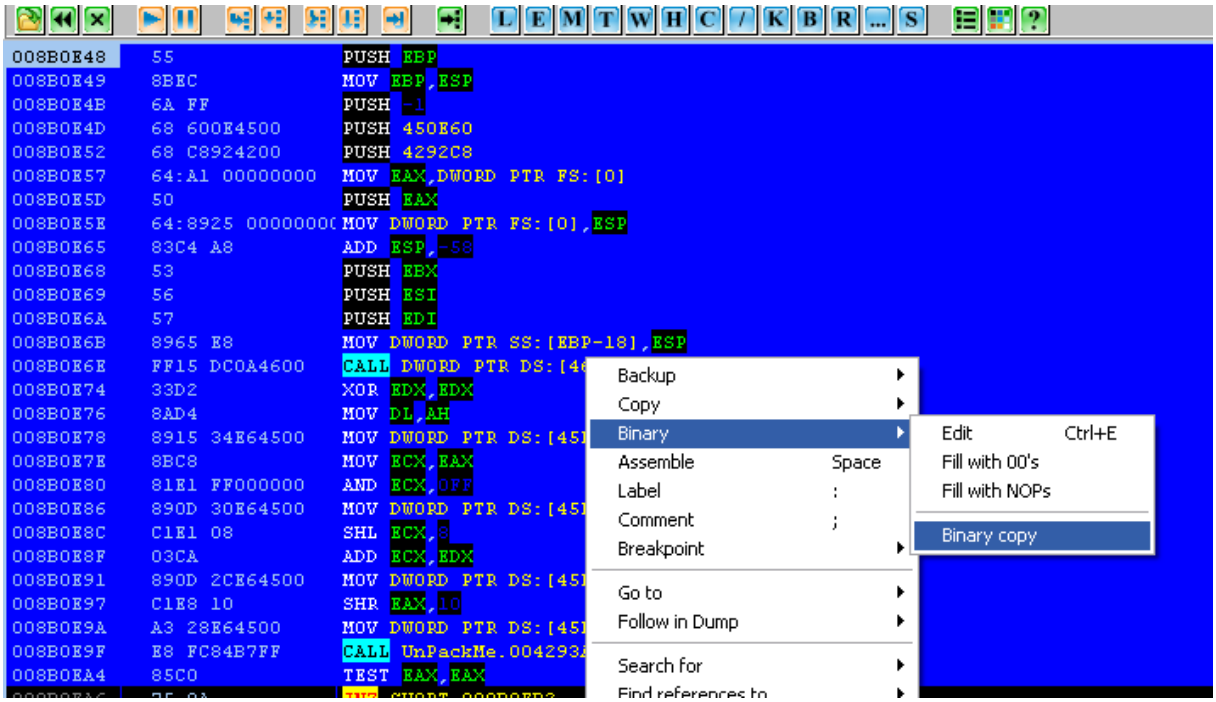
```
ultima:
findop eip,#FFE0#          // Al caer aquí por la ultima excepción buscamos el salto JMP EAX al Stolen Code.
mov dir_JMP,$RESULT        // Una vez encontrado aguardamos la dirección en dir_JMP
bp dir_JMP                 // Ponemos un breakpoint (F2) en el salto JMP EAX.
esto                       // Pasamos la excepción con SHIFT + F9 para caer en el bp.
bc dir_JMP                 // Quitamos el bp del Salto JMP EAX.
sti                        // Ejecuta F7 para caer en el Stolen Code.
```

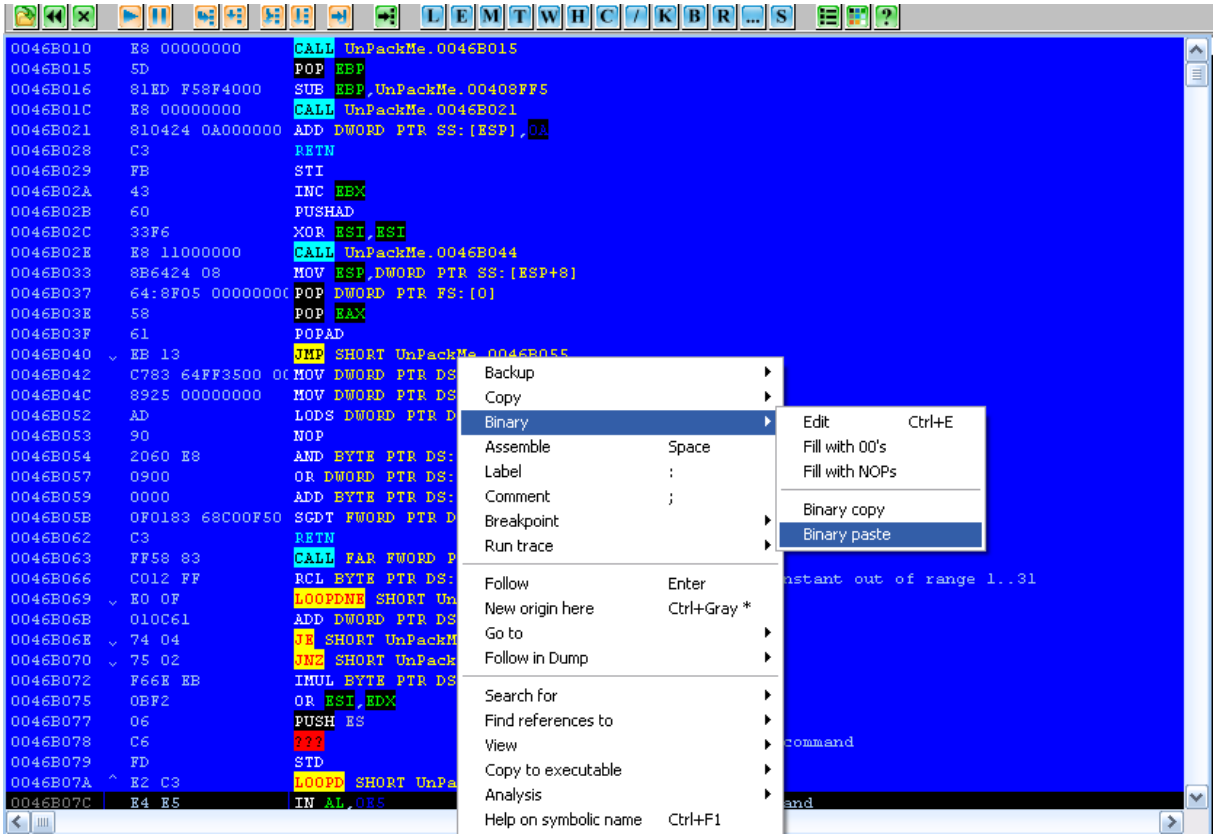
И готово — мы в OEP. Здесь дело усложняется, как сказал marciano, если скопируем этот код в точку входа:

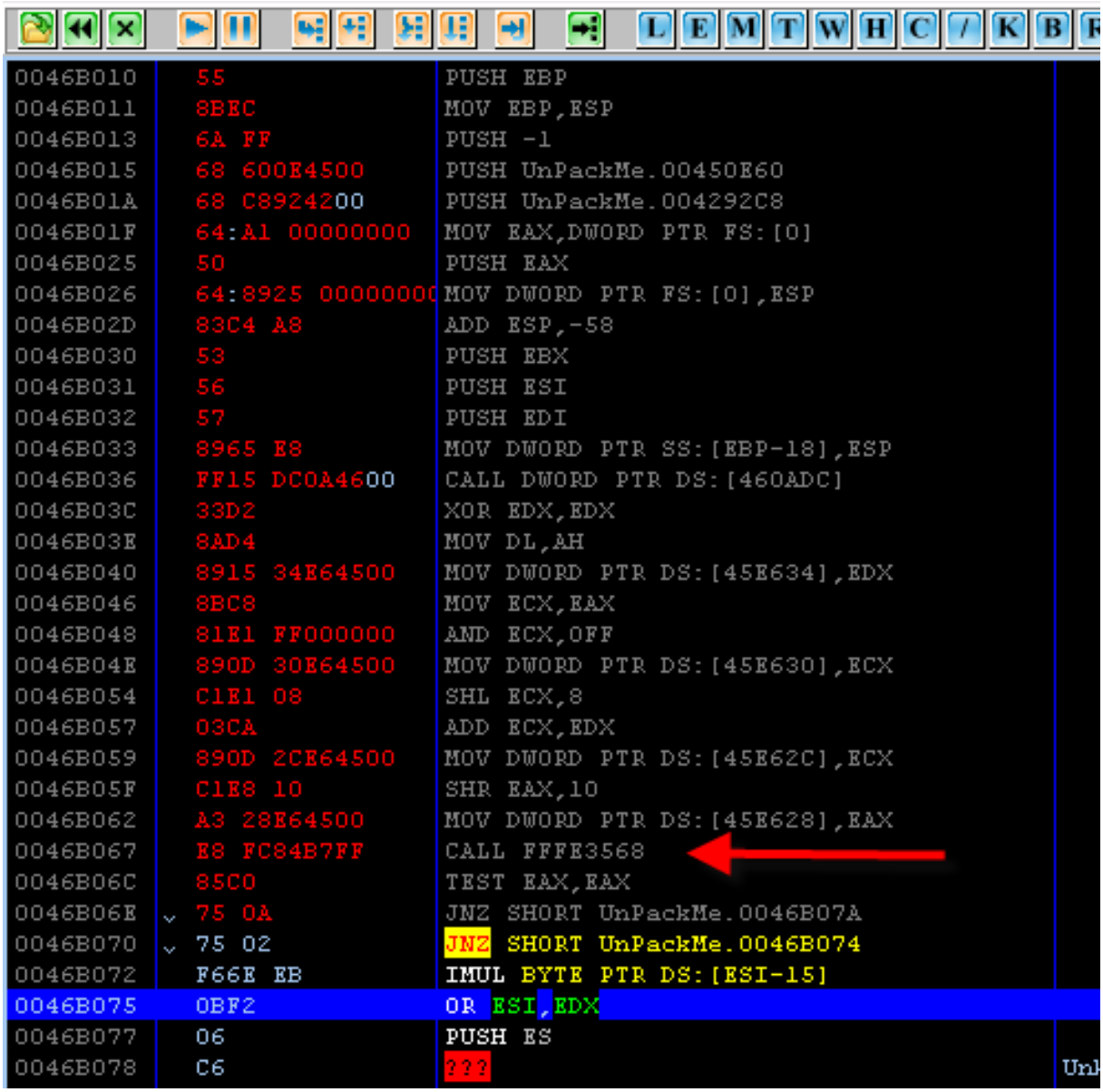


Просто скопируем кусок украденного кода, как миним прямой вызов, который находится в 008B0E9F.

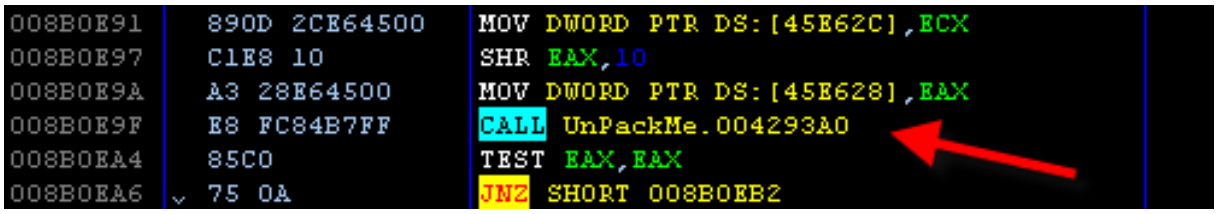
И вставляем в точку входа:




0046B010 E8 00000000 CALL UnPackMe.0046B015
0046B015 5D POP EBP
0046B016 81ED F58F4000 SUB EBP,UnPackMe.00408FF5
0046B01C E8 00000000 CALL UnPackMe.0046B021
0046B021 810424 0A000000 ADD DWORD PTR SS:[ESP],0A
0046B028 C3 RETN
0046B029 FB STI
0046B02A 43 INC EBX
0046B02B 60 PUSHAD
0046B02C 33F6 XOR ESI,ESI
0046B02E E8 11000000 CALL UnPackMe.0046B044
0046B033 8B6424 08 MOV ESP,DWORD PTR SS:[ESP+8]
0046B037 64:8F05 00000000 POP DWORD PTR FS:[0]
0046B03E 58 POP EAX
0046B03F 61 POPAD
0046B040 EB 13 JMP SHORT UnPackMe.0046B055
0046B042 C783 64FF3500 0C MOV DWORD PTR DS
0046B04C 8925 00000000 MOV DWORD PTR DS
0046B052 AD LODS DWORD PTR D
0046B053 90 NOP
0046B054 2060 E8 AND BYTE PTR DS:
0046B057 0900 OR DWORD PTR DS:
0046B059 0000 ADD BYTE PTR DS:
0046B05B 0F0183 68C00F50 SGDT FWORD PTR D
0046B062 C3 RETN
0046B063 FF58 83 CALL FAR FWORD P
0046B066 C012 FF RCL BYTE PTR DS:
0046B069 E0 0F LOOPDNE SHORT Un
0046B06B 010C61 ADD DWORD PTR DS
0046B06E 74 04 JE SHORT UnPackM
0046B070 75 02 JNZ SHORT UnPack
0046B072 F66E EB IMUL BYTE PTR DS
0046B075 0BF2 OR ESI,EDX
0046B077 06 PUSH ES
0046B078 C6 ???
0046B079 FD STD
0046B07A E2 C3 LOOPD SHORT UnPa
0046B07C E4 E5 IN AL,0E5
nstant out of range 1..31
command
end
Backup
Copy
Binary
Assemble Space
Label :
Comment ;
Breakpoint
Run trace
Follow Enter
New origin here Ctrl+Gray *
Go to
Follow in Dump
Search for
Find references to
View
Copy to executable
Analysis
Help on symbolic name Ctrl+F1
Edit Ctrl+E
Fill with 00's
Fill with NOPs
Binary copy
Binary paste



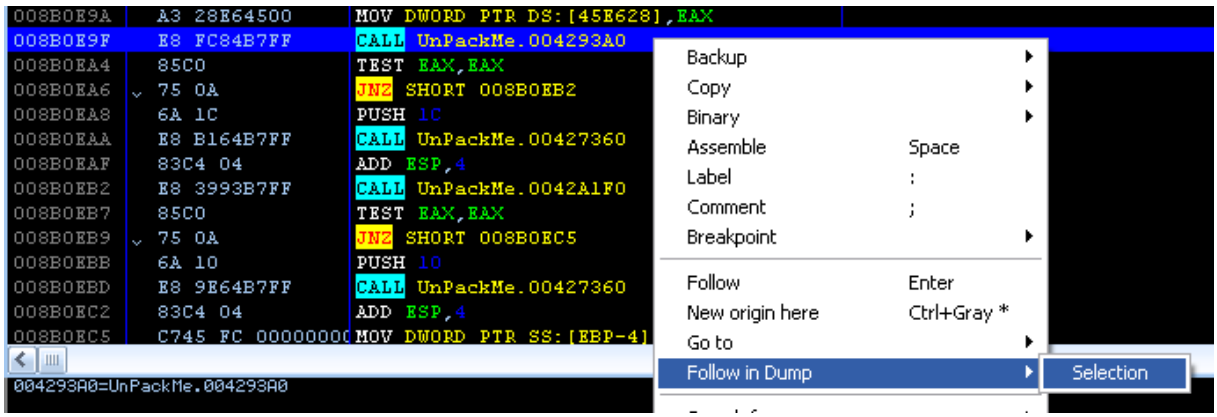
Здесь у нас немного вставленного когда, видимо, что это косвенный вызов, который находится по адресу 0046b067. Он испорчен, посмотрим, куда он должен указывать. Идём туда, где находится OEP.

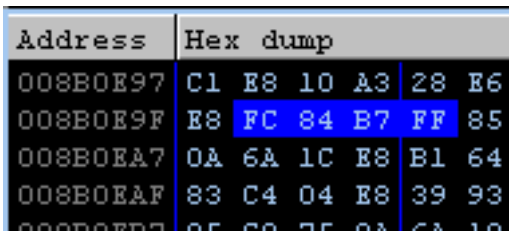


В оригинальном украденном коде находится CALL 004293A0.

Значит этот вызов нам нужно починить, чтобы, когда скопируем его в другую часть, он указывал в то же место. Как это сделать?

Идём по этому вызову в DUMP.





Не берём в расчёт E8 и используем четыре следующих байта.

FF B7 84 FC назовём их «OPCODES».

Берём адрес из Call 008B0E9F назовём DIR_CALL.

Теперь этот вызов становится CALL 004293A0, а адрес получили следующим образом:

УКАЗАТЕЛЬ = OPCODES + DIR_CALL + 5

УКАЗАТЕЛЬ = FFB784FC + 008B0E9F + 5 = 004293A0

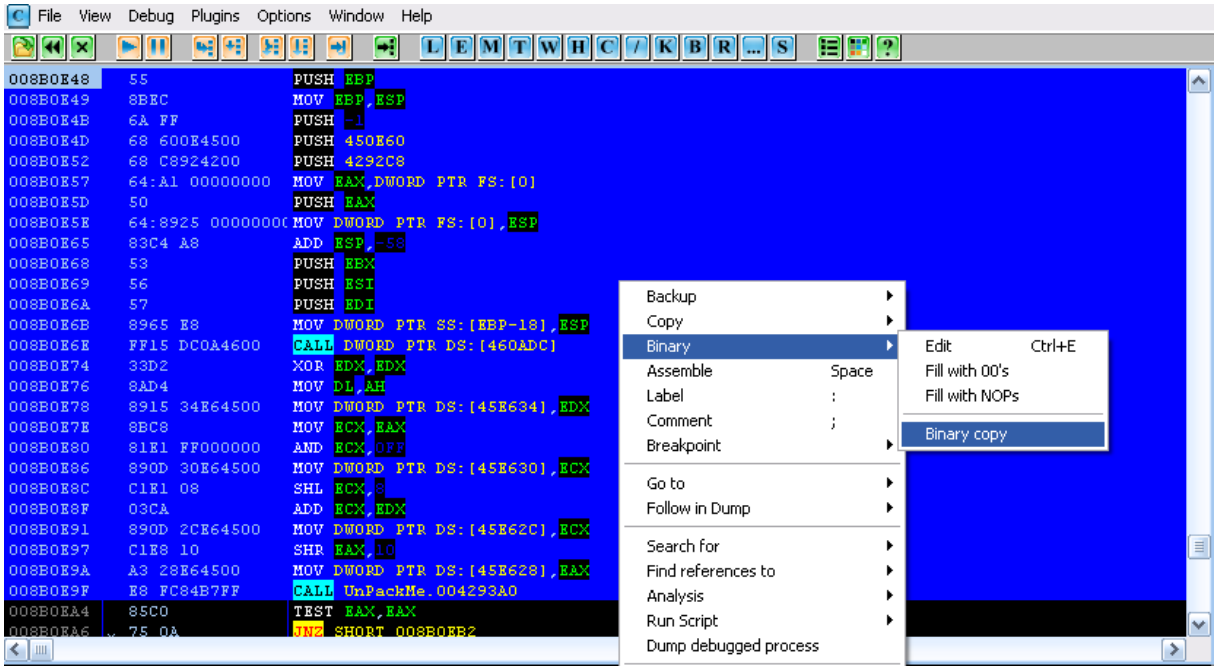
Хорошо, теперь у нас есть адрес, куда должен указывать вызов, чиним его с помощью новых опкодов и эти новые опкоды мы рассчитываем (не уверен, что правильно понял его здесь — прим. пер.) таким образом.

PUNTERO - DIRECCIÓN_NUEVA – 5 = NUEVOS OPCODES

УКАЗАТЕЛЬ – НОВЫЙ_УКАЗАТЕЛЬ – 5 = НОВЫЕ ОПКОДЫ

DIRECCIÓN _NUEVA: рассчитываем таким образом.

Например:



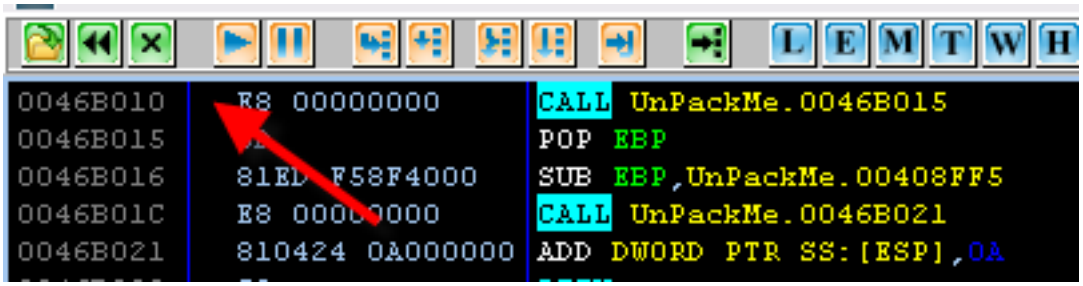
Если скопируем первые строки украденного кода и скопируем их в точку входа.

Видим, что первый прямой вызов будет находится по этому адресу в точке входа 0046B057. Поэтому этот адрес рассчитываем так:

ИСХОДНЫЙ АДРЕС – АДРЕС OEP (УКРАДЕННЫЙ КОД) + АДРЕС ТОЧКИ ВХОДА Где ИСХОДНЫЙ АДРЕС = 008B09EF

АДРЕС OEP (УКРАДЕННЫЙ КОД) = 008B0E48

И АДРЕС ТОЧКИ ВХОДА : 0046b010



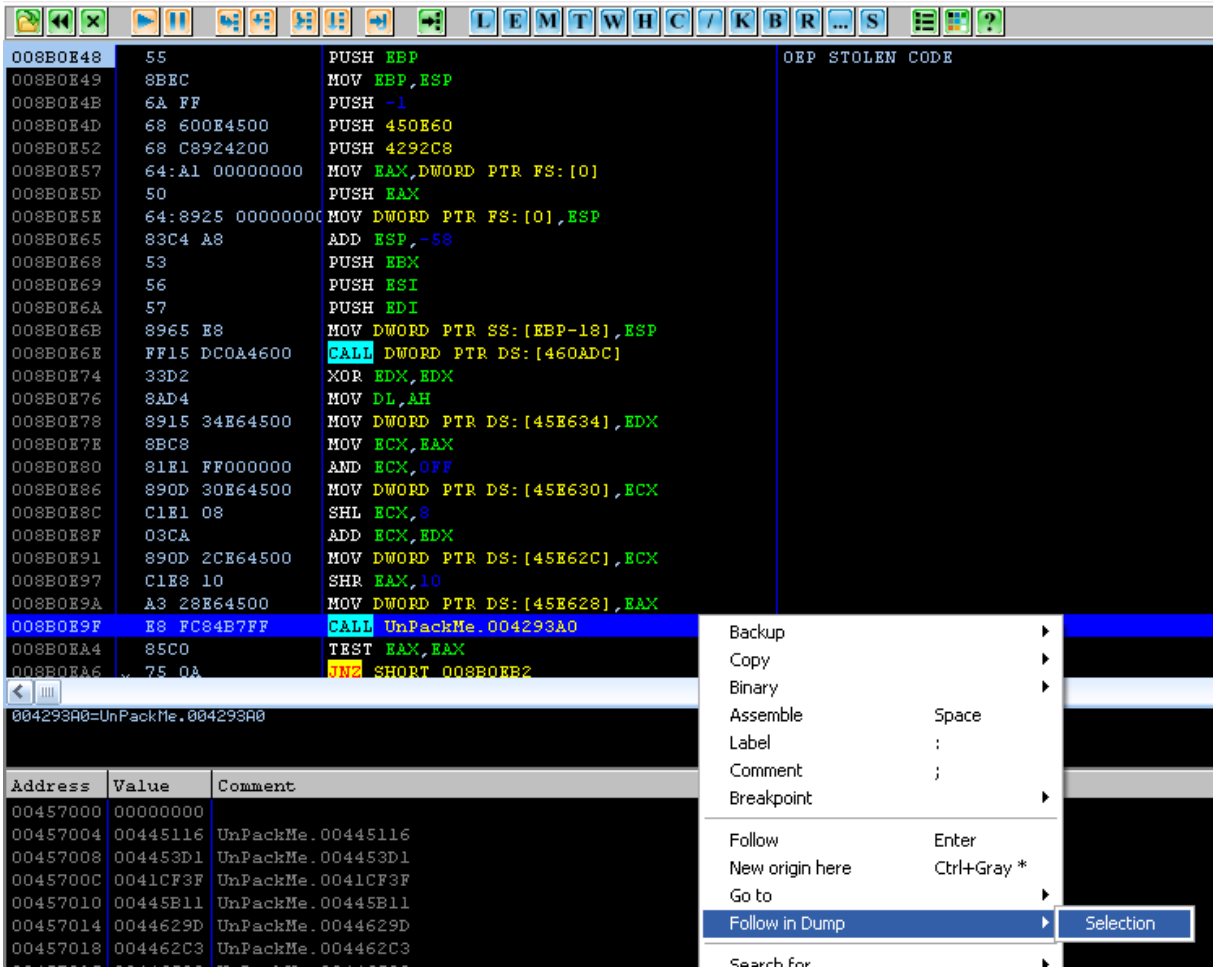
8B0E9F - 8B0E48 + 46B010= 0046B067 , по этому адресу будет находится наш починенный адрес.

Получается, что НОВЫЙ АДРЕС = 0046B067

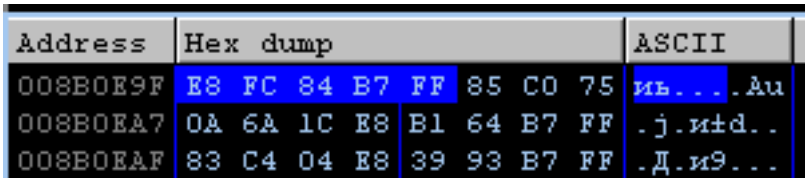
Теперь у нас есть данные, чтобы рассчитать новые опкоды.

004293A0 - 0046B067 – 5 = FFFBE334

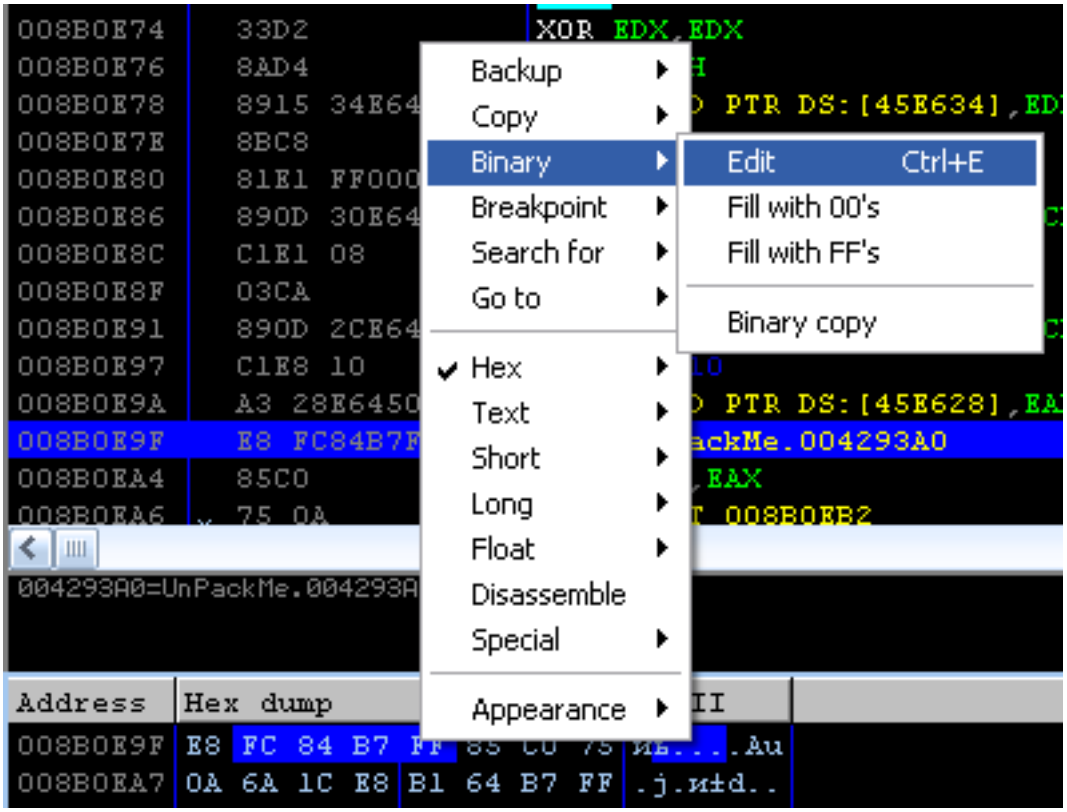
У нас есть новые опкоды. Теперь осталось их отредактировать вручную. Идём в первый прямой вызов в украденном коде.

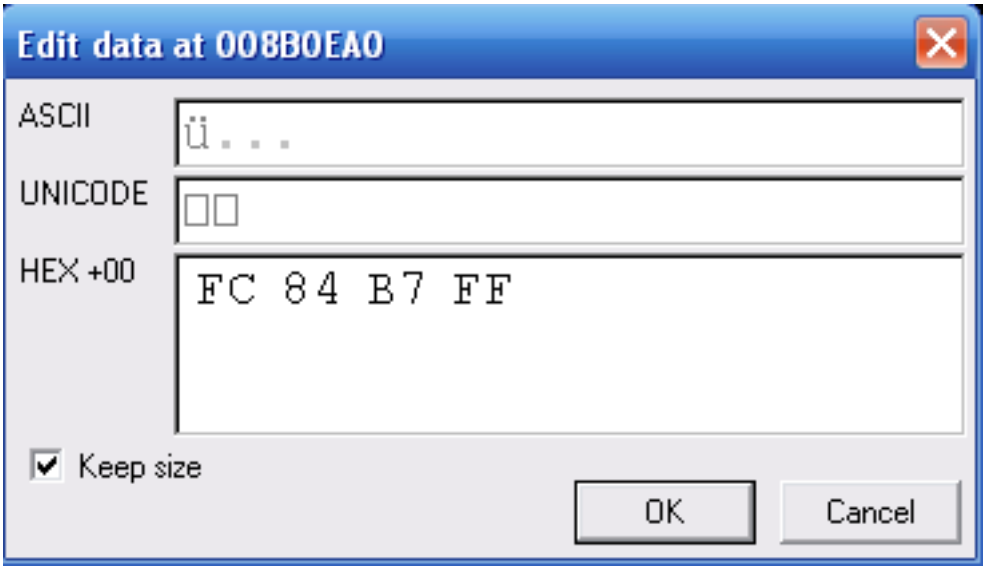


В дампе нам показывают вот что:

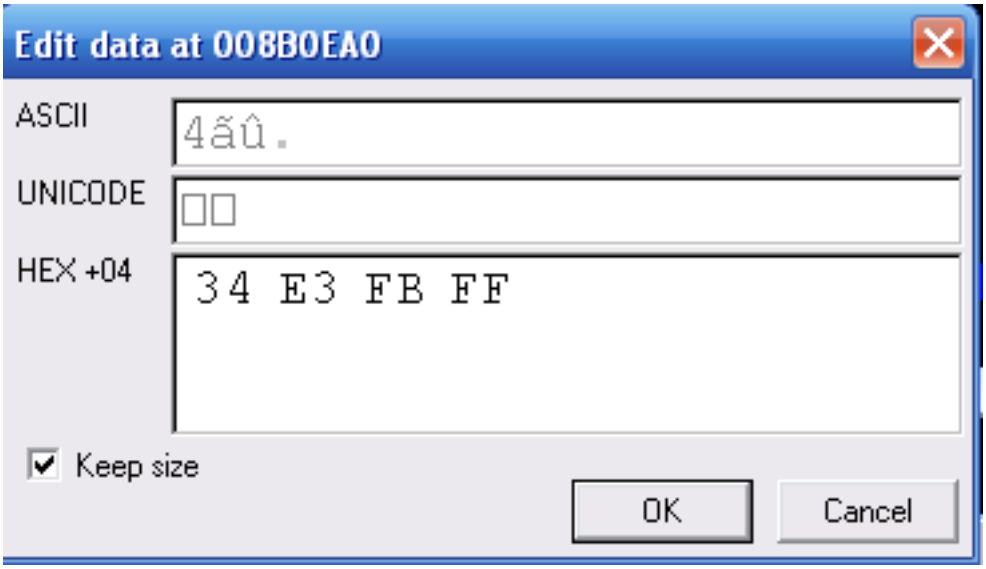


Не берём E8 и модифицируем следующие за ним четыре байта:



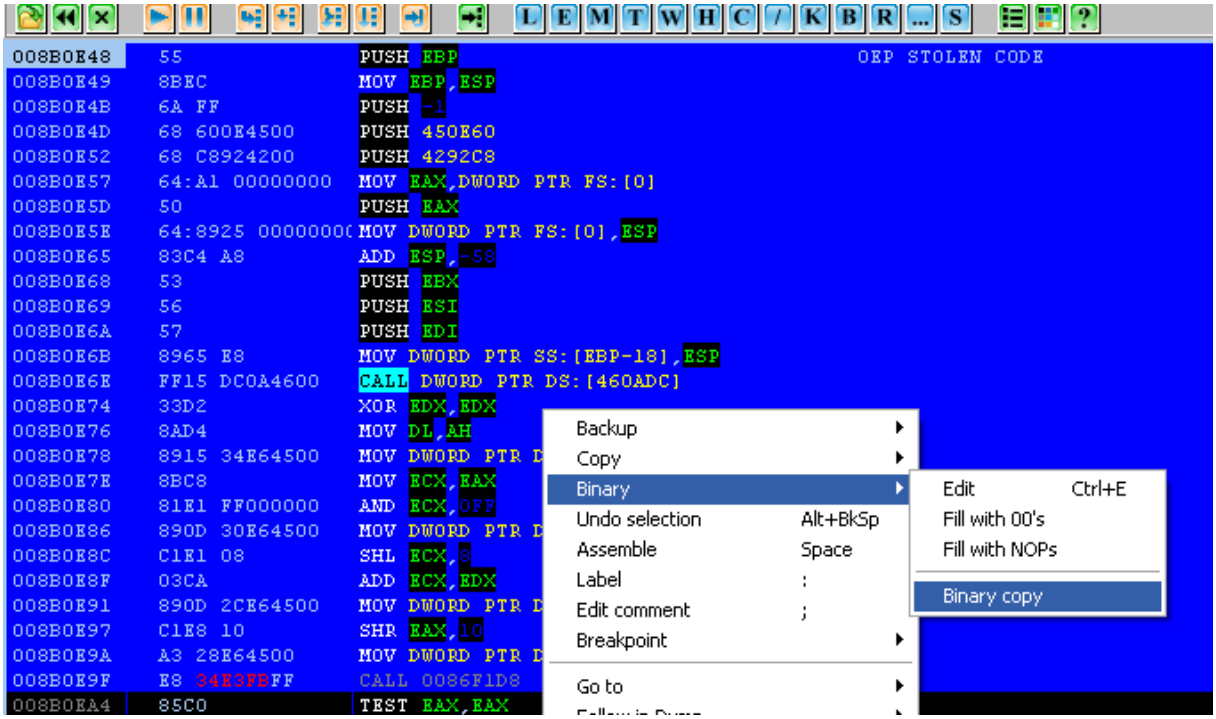


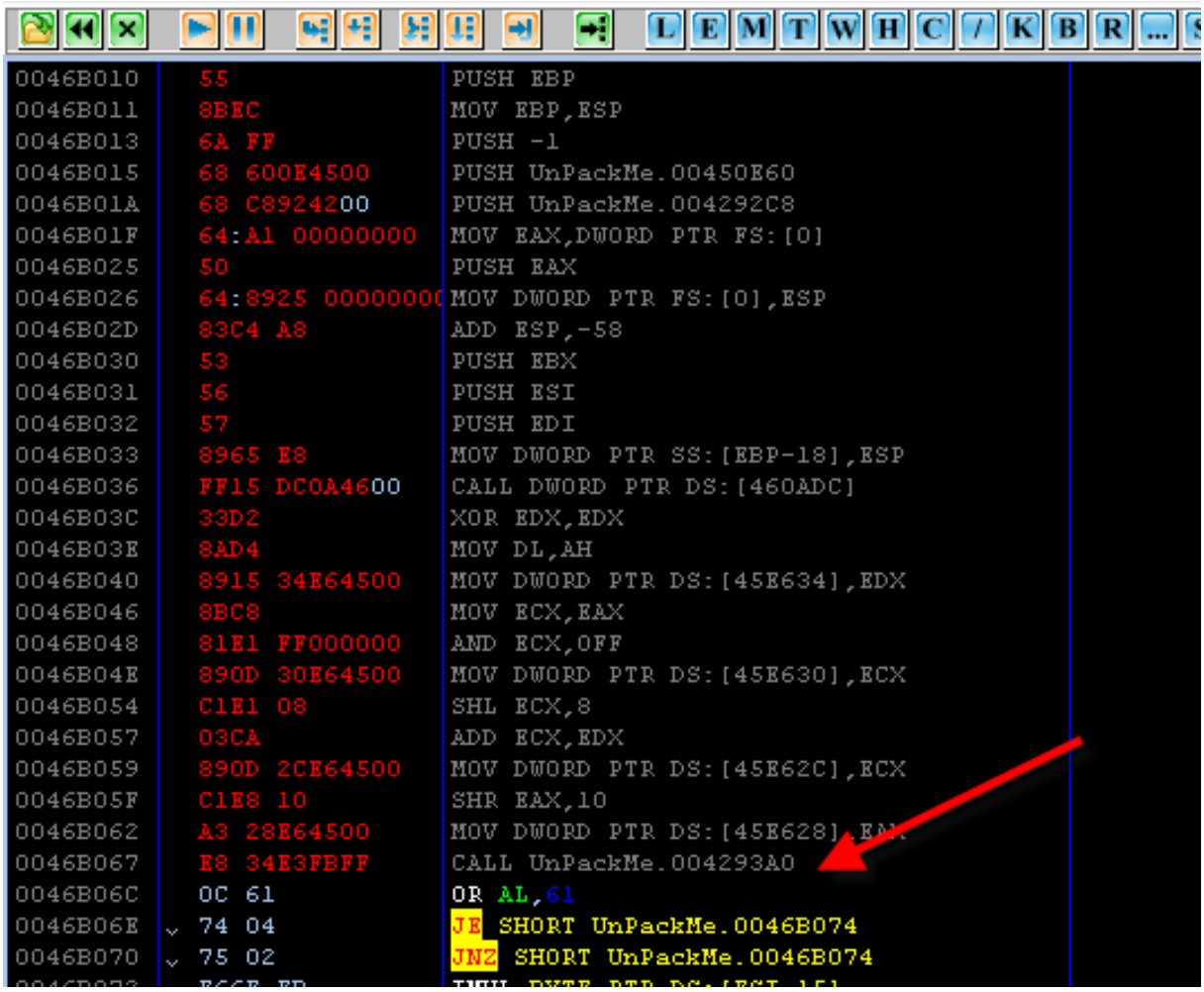
И заменяем их новыми опкодами.




```
008B0E97   C1E8 10          SHR EAX,10
008B0E9A   A3 28E64500      MOV DWORD PTR DS:[45E628],EAX
008B0E9F   E8 34E3FBFF      CALL 0086F1D8
008B0EA4   85C0             TEST EAX,EAX
```

Видим, что наш вызов стал ужасен, но если скопируем этот кусок кода и вставим в точку входа.





Остаётся починенный код, как раз такой, какой должен быть.

Теперь нужно сделать это для других прямых вызовов, но мы оставляем это скрипту, к которому добавляем следующее:

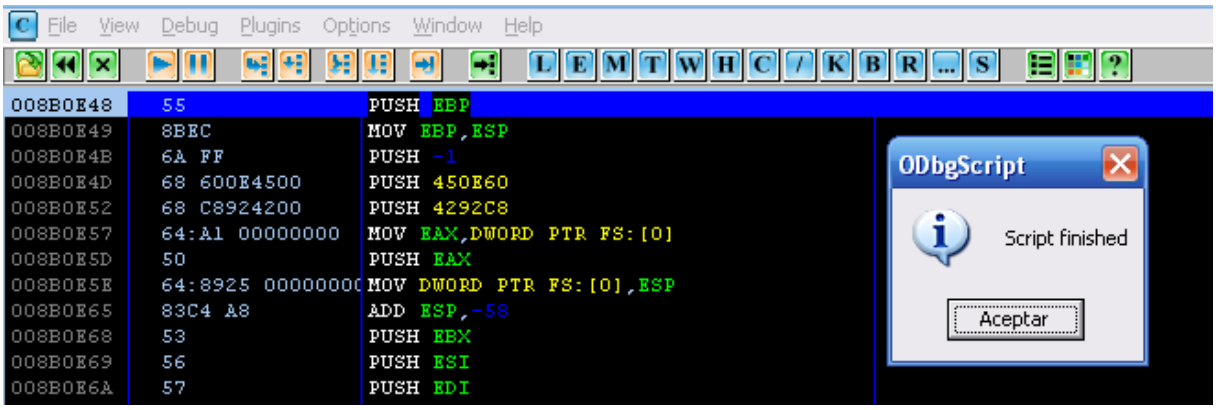
```
var dir_CALL
var oep
var StartScan
var Opcodes
var temp
var temp2
var temp3
```

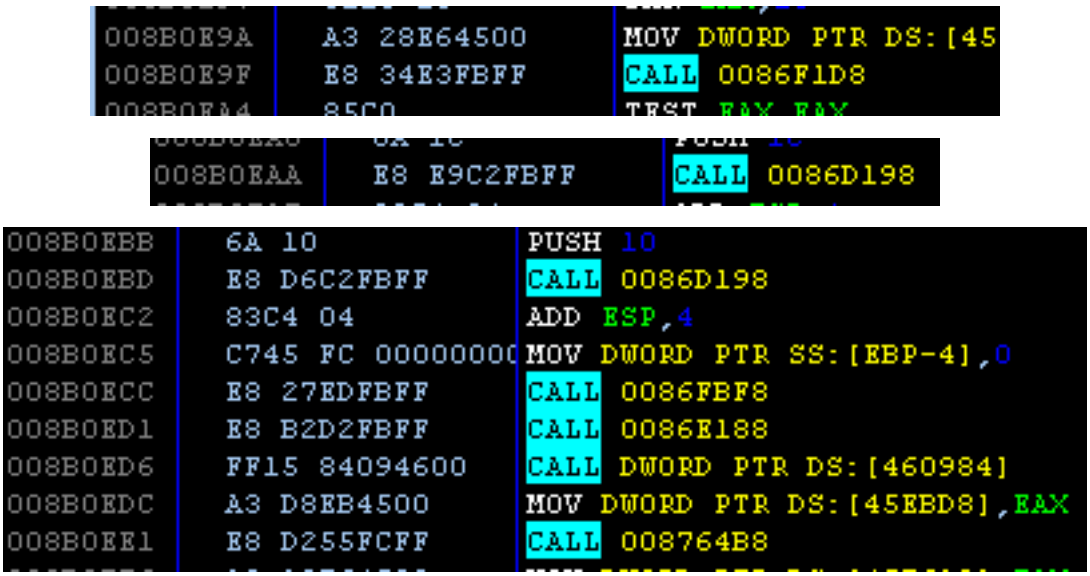
А потом ещё вот это:

```
ultima:
findop eip,#FFE0#          // Al caer aquí por la ultima excepción buscamos el salto JMP EAX al Stolen Code.
mov dir_JMP,$RESULT        // Una vez encontrado aguardamos la dirección en dir_JMP
bp dir_JMP                 // Ponemos un breakpoint (F2) en el salto JMP EAX.
esto                       // Pasamos la excepción con SHIFT + F9 para caer en el bp.
bc dir_JMP                 // Quitamos el bp del Salto JMP EAX.
sti                        // Ejecuta F7 para caer en el Stolen Code.

mov oep,eip                // Guardamos la dirección del OEP robado.
mov StartScan,eip          // Guardamos la dirección del oep tambien a StartScan que la usaremos para buscar Calls.

BuscarCall:                // Aqui empezaremos a buscar los calls directos para repararlos para que a la hora de
                           // de hacer binary copy y pegarlos al nuevo oep esos call directos queden bien.
                           // Nota ver MIniTuto.PARTE1.doc

 findop StartScan,#E8#     // Buscamos los calls directos que comienzan con su opcode E8
 cmp $RESULT, 0            // Cuando no encuentre mas $RESULT valdrá 0 y saltamos al final del script.
 je final
 mov dir_CALL, $RESULT     // Guardamos la dirección del primer call encontrado.
 mov StartScan, $RESULT    // Actualizamos desde donde queremos seguir buscando calls en este caso desde el primer c
 add dir_CALL,1            // A la dirección del call entrontrado le sumamos 1 esto para no tomar el OPCODE E8.
 mov Opcodes, [dir_CALL]   // Movemos los opcodes de esa dirección a OPcodes.
 add Opcodes,StartScan     // A esos Opcodes le Sumados la Dirección del Call.
 add Opcodes,5             // Al resultado le sumamos 5 y obtenemos la dirección que apunta el call.
                           // Nota ver MIniTuto.PARTE1.doc para aclarar.

// Ahora le asignaremos un nuevo opcode a este call encontrado para que a la hora de hacer bynary paste en el nuevo
// oep ese call quede arreglado.
```

```
// Ahora le asignaremos un nuevo opcode a este call encontrado para que a la hora de hacer bynary paste en el nuevo
// oep ese call quede arreglado.

mov temp, StartScan        // Movemos la dirección que contiene StartScan a un temporal. ( StartScan = Dirección del
sub temp, oep              // A la dirección de ese call a reparar le restamos la dirección del oep del stolen code.
mov temp2, Newoep          // Movemos la direccion del nuevo oep que es el EntryPoint a un temporal2.
add temp,temp2             // A ese temporal2 le sumamos el temporar1.
sub Opcodes, temp          //Ahora la dirección a la que apunta el CAll le restamos la operación anterior
sub Opcodes, 5             // y a lo que quedó le sumamos 5 y obtenemos los opcodes nuevos para ese call a reparar.


Editar:                    // Empezaremos a Editar el call con esos nuevos opcodes.
mov temp3, StartScan       // Movemos la dirección del call a reparar contenida en StartScan a un tercer temporal.
add temp3,1                // A la direccion del call le sumamos 1 para no tamar en cuanta el opcode E8.
mov [temp3], Opcodes       // Reparamos el call con los nuevos OPcodes.
jmp BuscarCall

final:
ret
```

Запускаем скрипт.





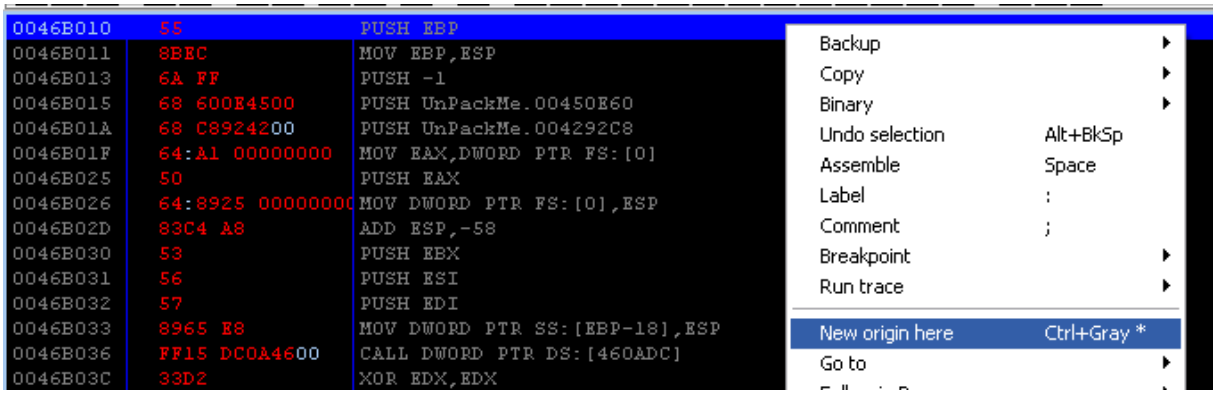
Видим, что починились все нужные вызовы.

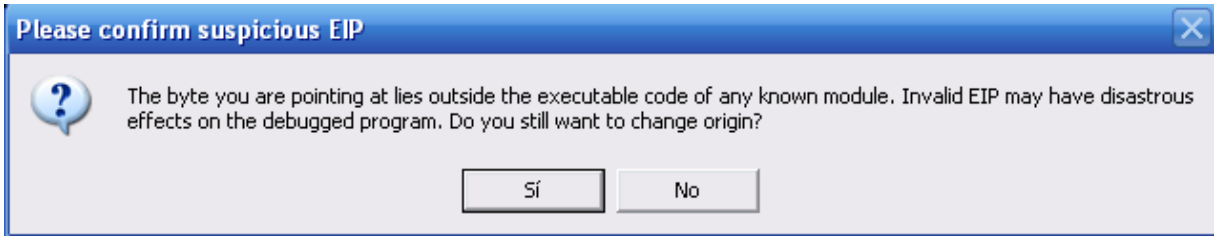
Выбираем весь украденный код: делаем Binary Copy.

И вставляем в точку входа.

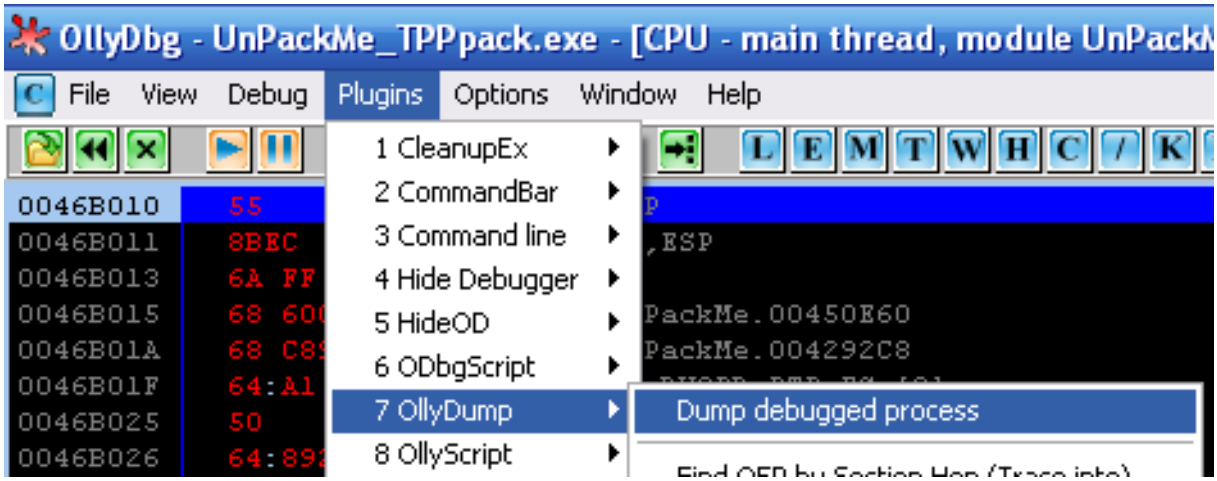
```
0046B010    55              PUSH EBP
0046B011    8BEC            MOV EBP,ESP
0046B013    6A FF           PUSH -1
0046B015    68 600E4500     PUSH UnPackMe.00450E60
0046B01A    68 C8924200     PUSH UnPackMe.004292C8
0046B01F    64:A1 00000000  MOV EAX,DWORD PTR FS:[0]
0046B025    50              PUSH EAX
0046B026    64:8925 00000000 MOV DWORD PTR FS:[0],ESP
0046B02D    83C4 A8         ADD ESP,-58
0046B030    53              PUSH EBX
0046B031    56              PUSH ESI
0046B032    57              PUSH EDI
0046B033    8965 E8         MOV DWORD PTR SS:[EBP-18],ESP
0046B036    FF15 DC0A4600   CALL DWORD PTR DS:[460ADC]
0046B03C    33D2            XOR EDX,EDX
0046B03E    8AD4            MOV DL,AH
0046B040    8915 34E64500   MOV DWORD PTR DS:[45E634],EDX
0046B046    8BC8            MOV ECX,EAX
0046B048    81E1 FF000000   AND ECX,0FF
0046B04E    890D 30E64500   MOV DWORD PTR DS:[45E630],ECX
0046B054    C1E1 08         SHL ECX,8
0046B057    03CA            ADD ECX,EDX
0046B059    890D 2CE64500   MOV DWORD PTR DS:[45E62C],ECX
0046B05F    C1E8 10         SHR EAX,10
0046B062    A3 28E64500     MOV DWORD PTR DS:[45E628],EAX
0046B067    E8 34E3FBFF     CALL UnPackMe.004293A0
0046B06C    85C0            TEST EAX,EAX
0046B06E    75 0A           JNZ SHORT UnPackMe.0046B07A
0046B070    6A 1C           PUSH 1C
0046B072    E8 E9C2FBFF     CALL UnPackMe.00427360
0046B077    83C4 04         ADD ESP,4
0046B07A    E8 71F1FBFF     CALL UnPackMe.0042A1F0
0046B07F    85C0            TEST EAX,EAX
0046B081    75 0A           JNZ SHORT UnPackMe.0046B08D
```

Находясь в точке входа, меняем EIP.

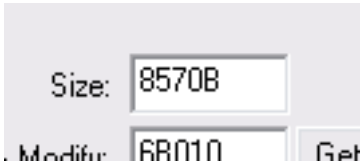




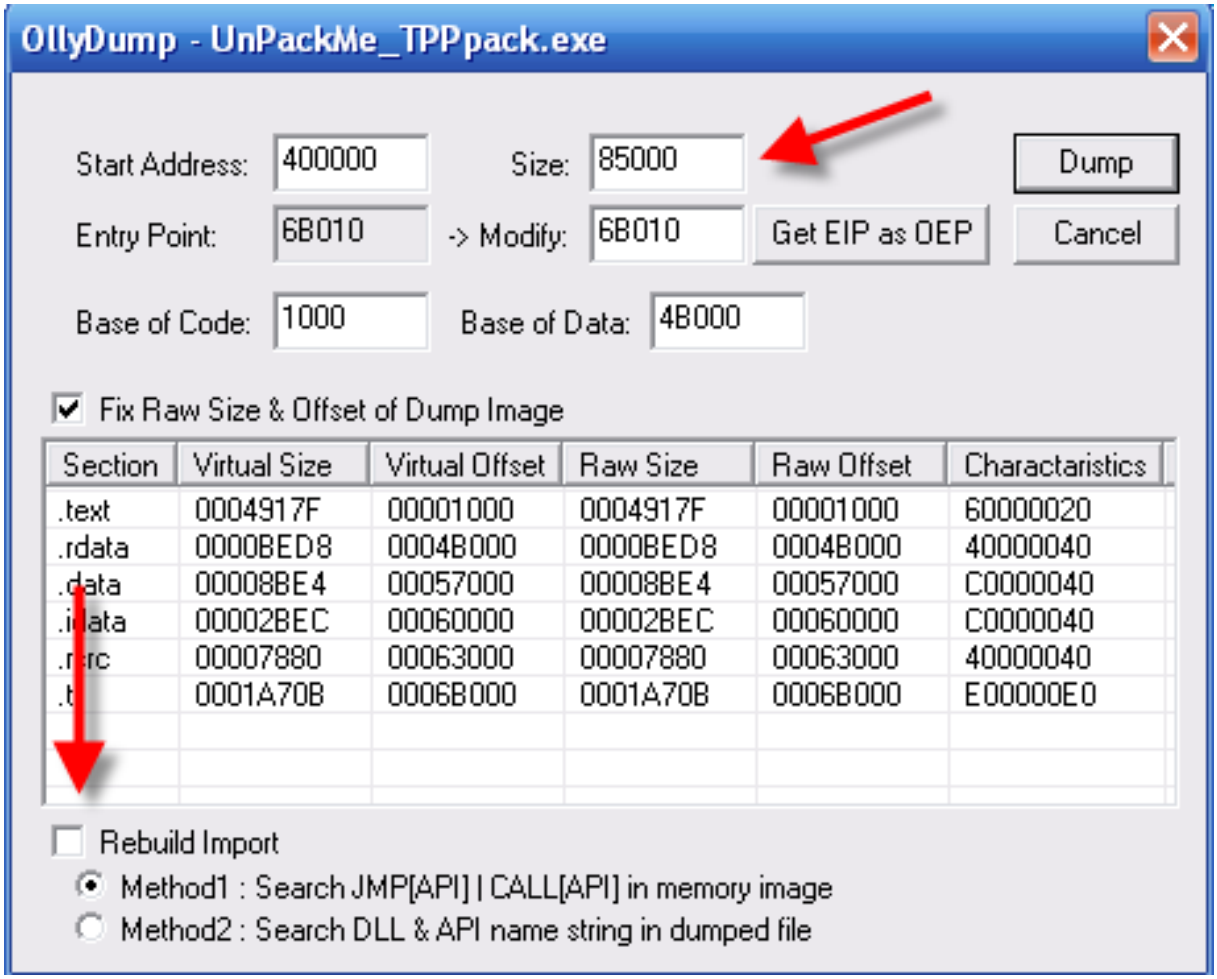
Говорим, что «Да» и делаем дамп.



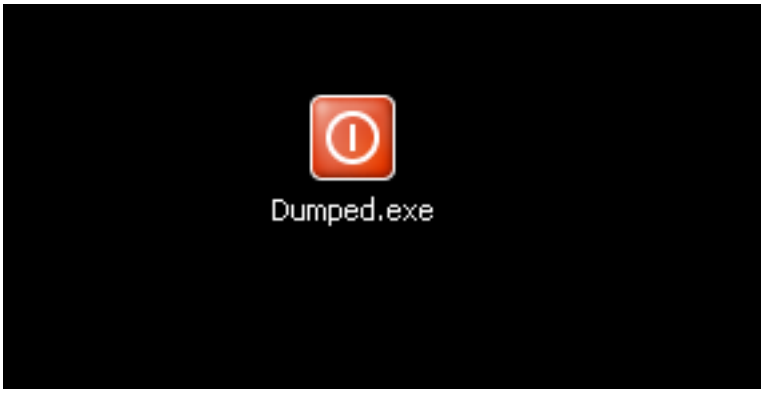
Marciano говорил, что нужно поменять это



На

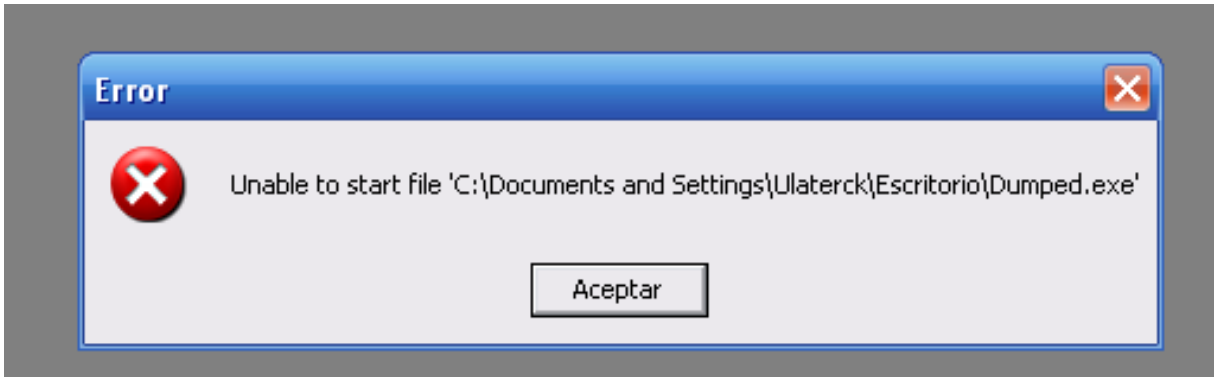


Ставим флажок «Rebuild Import» и нажимаем «Dump».

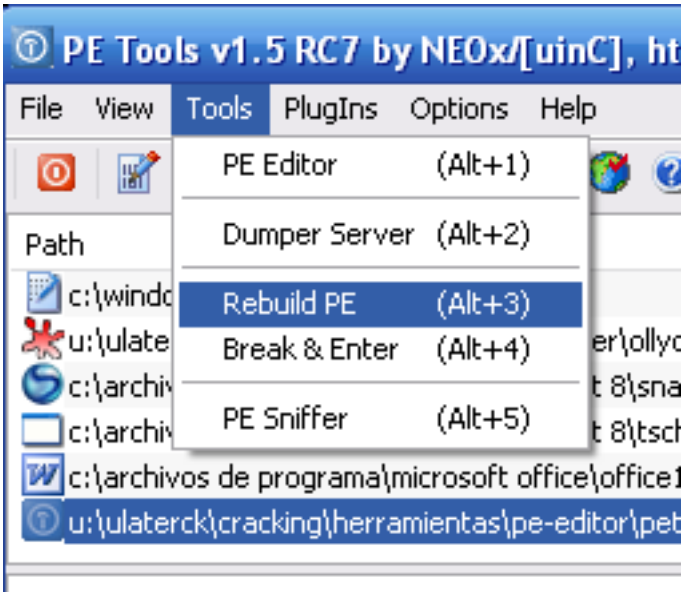


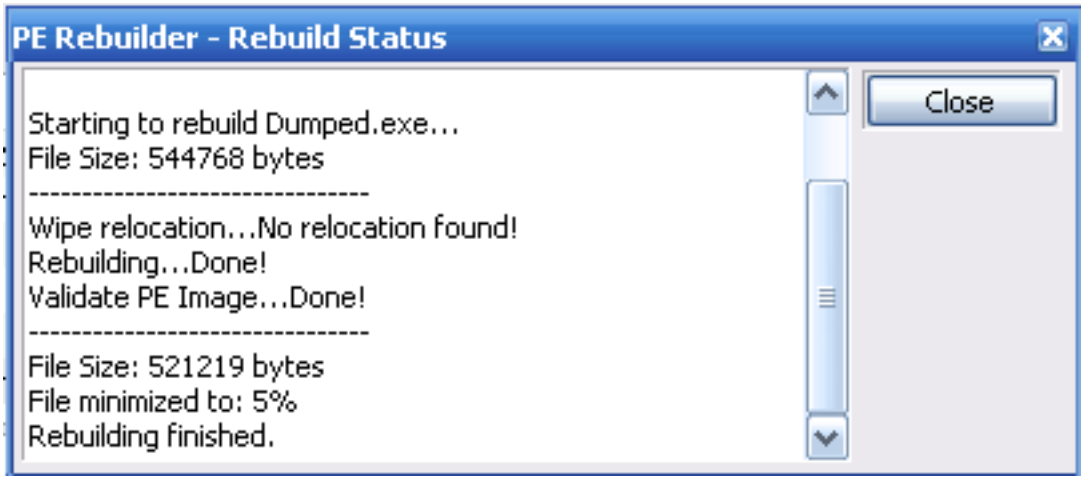
Вот наш сдампленный файл, в котором проблема с IAT.

Если попробуем открыть его с помощью Олли:

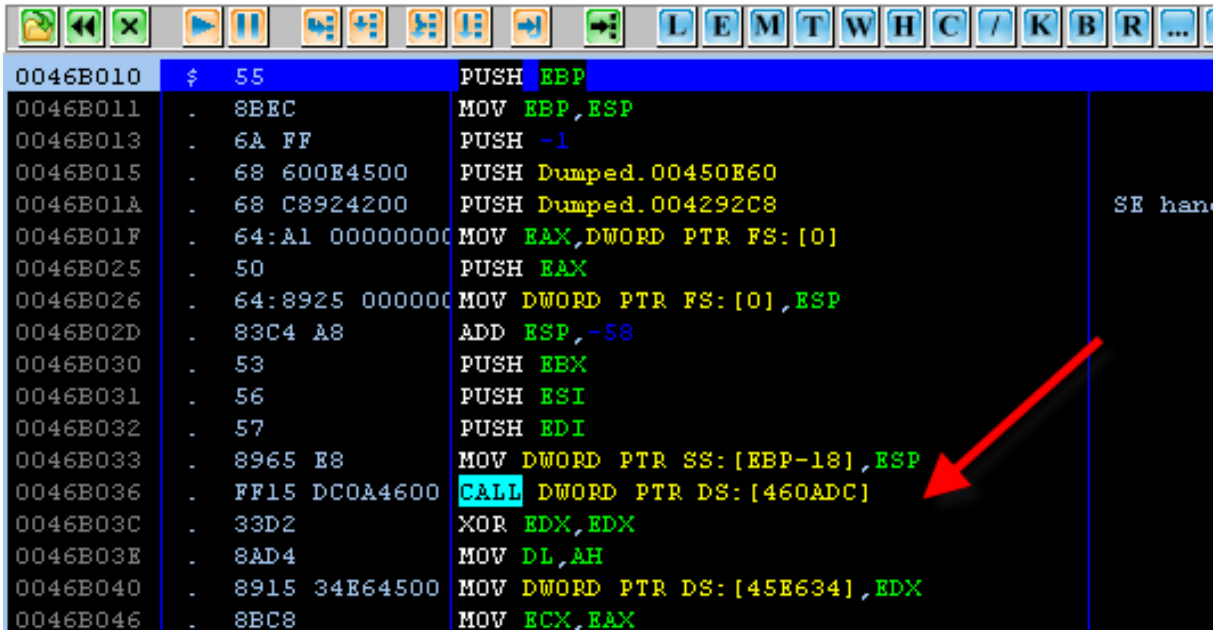


Попробуем починить его с помощью PE Editor'a.





И готово.



В плагине HideOD есть ошибки, которые приводят к тому, что некоторые программы не запускаются, поэтому лучше заменить его на HideDebugger и OllyAdvanced.

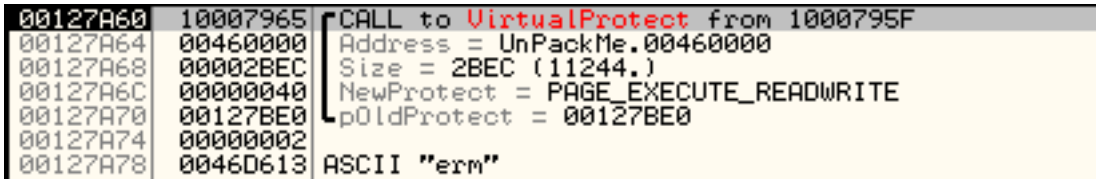
Ок, на этом можно завершить объяснение скрипта из первой части и перейти к скрипту из второй.

*/

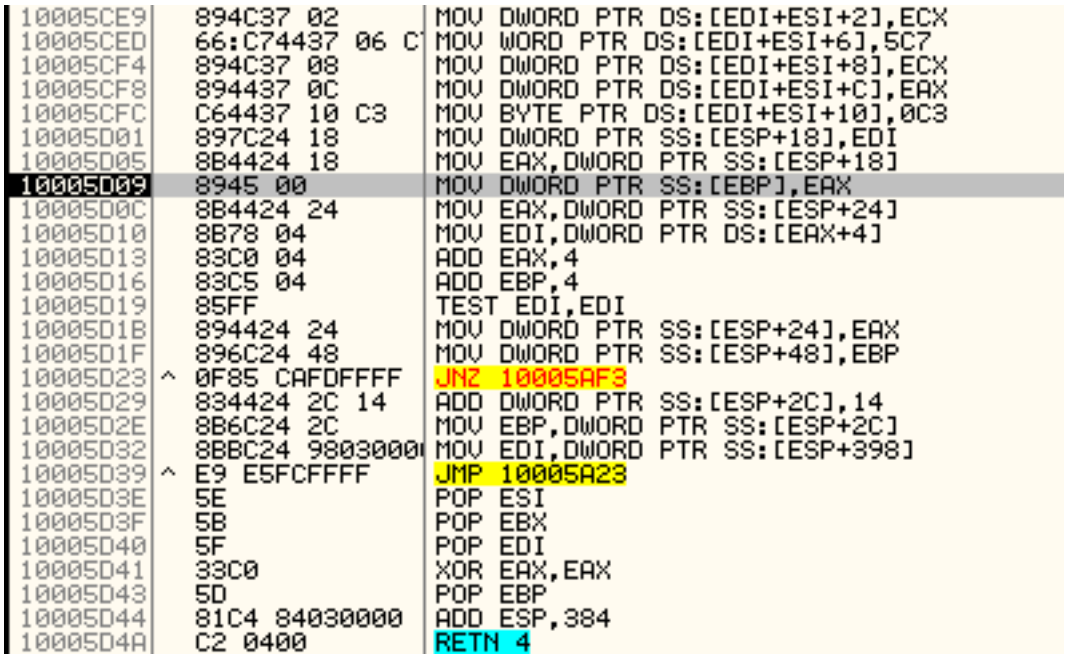
```
var base
var dir_VirtualAlloc
var dir_VirtualProtect
VAR dir_mov
Inicio:

gpa "VirtualAlloc", "kernel32.dll"    // Buscamos la dirección de la Funcion VirtualAlloc
mov dir_VirtualAlloc, $RESULT
log dir_VirtualAlloc

gpa "VirtualProtect", "kernel32.dll"
mov dir_VirtualProtect, $RESULT

bp dir_VirtualAlloc
run
eob info

info:

mov base, eax
log base
bc dir_VirtualAlloc
bp dir_VirtualProtect

Zona:
eob seccion
run

seccion:
cmp esi, 00460000
je retornar
jmp Zona

retornar:
bc dir_VirtualProtect
mov Reg_esp, [esp]
bp Reg_esp

eob zona_1
```

```
run

zona_1:
bc Reg_esp
find base, #897C24188B4424#
mov dir_mov, $RESULT
log dir_mov
jmp nopear

nopear:
bp dir_mov
eob nopear2
run
nopear2:
bc dir_mov
fill dir_mov, 4, 90

final:

msg "Mov nopeado, al terminar el script presiona run (F9)."

ret
```

Метод для починки IAT – классический, который был хорошо объяснён в туториале marciano. Устанавливаем BPM ON WRITE на одном из плохих элементов IAT, который нашли ранее, находясь в OEP, а затем пытаемся попасть в место, где сохраняется плохое значение. К сожалению во время выполнения функции VirtualProtect стирает BPM ON WRITE, так что нужно установить BP на этой функции, чтобы произвести восстановление вышеуказанного BPM, когда она отработает.

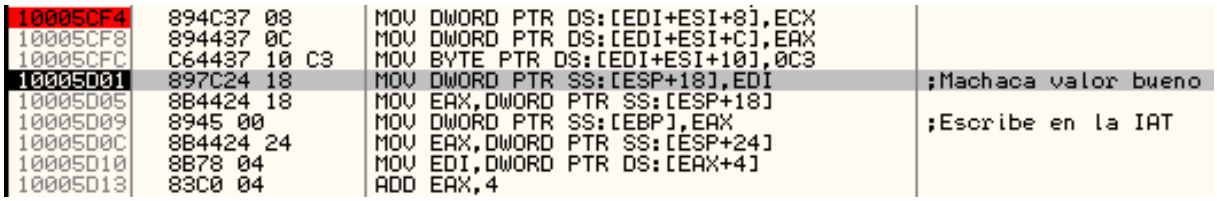


Здесь видим изображение, взятое из туториала marciano, когда происходит останов на VirtualProtect, чтобы изменить права доступа к области IAT, так что в этот момент должны дойти до RET и снова установить BPM ON WRITE, чтобы продолжить с помощью «RUN», пока не остановимся на месте, где плохое значение сохраняется в элементе.



Останавливаемся здесь, где происходит сохранение. Плохое значение находится в EAX, а EBP указывает на элемент IAT, за которым мы наблюдали.

Затем устанавливаем BP несколькими строками выше, и когда останавливаемся, то трассируя видим следующее:



Заменяем правильный адрес функции на плохое значение, которое сохраняется несколькими строками ниже, так что работа сводится к тому, чтобы заNOPить эту строку, так как правильный адрес будет сохраняться чуть позже.

Так что скрипт толже установить местонахождение этой инструкции и заNOPить её.

Таким образом, первое что делает скрипт — это ищет адрес API-функции VirtualAlloc, так как первый раз, когда останавливается на ней, получает адрес области, где находится инструкцию, которую нужно заNOPить. Этот адрес может быть разным на разных машинах, поэтому его необходимо искать.

```
var base
var dir_VirtualAlloc
var dir_VirtualProtect
VAR dir_mov
Inicio:

gpa "VirtualAlloc", "kernel32.dll"    // Buscamos la dirección de la Funcion VirtualAlloc
mov dir_VirtualAlloc, $RESULT
log dir_VirtualAlloc
```

Здесь сохраняем адрес вышеуказанной функции в переменную var dir_VirtualAlloc, потом делаем то же самое с функцией VirtualProtect.

```
gpa "VirtualProtect", "kernel32.dll"
mov dir_VirtualProtect, $RESULT
```

В переменной var dir_VirtualProtect сохраняем адрес соответствующей API-функции.

```
bp dir_VirtualAlloc
run
eob info

info:

mov base, eax
log base
bc dir_VirtualAlloc
bp dir_VirtualProtect
```

Затем нужно установить BP на VirtuallAlloc, делаем RUN и с помощью eob останавливаемся на указанном bp, сохраняем в переменной base адрес, где программа собирается создать область, в которой будет находится адрес для NOPинга.

Также устанавливаем BP на API-функции VirtualProtect.

```
Zona:
eob seccion
run

seccion:
cmp esi, 00460000
je retornar
jmp Zona
```

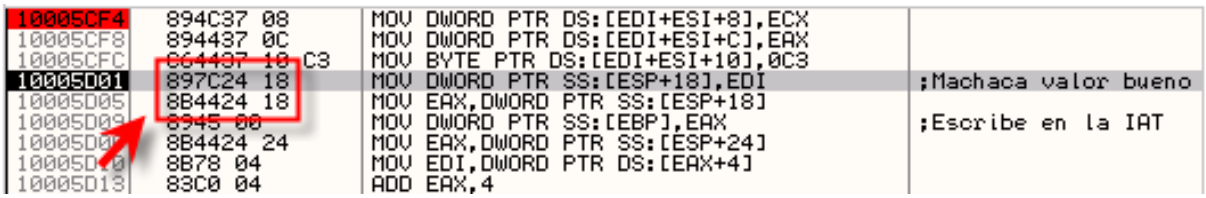
Когда останавливаемся на BP, сравниваем равен ли ESI 460000, что является началом IAT, так как VirtualProtect использует его как аргумент функции для защиты указанной зоны и стирает BPMы, которые там находится, и если равна, то идём в метку retornar.

```
retornar:
bc dir_VirtualProtect
mov Reg_esp, [esp]
bp Reg_esp
```

Где стирается BP на функции VirtualProtect, ищется значение, что будет находится в [esp] и устанавливается BP, чтобы остановиться, как только выйдем из API-функции.

```
zona_1:
bc Reg_esp
find base, #897C24188B4424#
mov dir_mov, $RESULT
log dir_mov
jmp nopear
```

Когда возвращаемся из API-функции, то уже можно искать инструкцию, которую будем NOPить. Искать надо по достаточно большой последовательности байтов, чтобы не встретить левые инструкции: 897C24188B4424.



После того, как нашли, делаем переход на процедуру заNOPления.

```
nopear:
bp dir_mov
eob nopear2
run
nopear2:
bc dir_mov
fill dir_mov, 4, 90

final:

msg "Mov nopeado, al terminar el script presiona run (F9)."

ret
```

Вот и всё объяснение и спасибо Ularteck'y за хорошую работу и marciano за его прекрасный туториал, из которого мы взяли изображения для объяснения 2-ой части скрипта.

Думаю, что это поможет вам освоить технику создания скриптов, думаю, что она очень мощная и в ней нет ничего трудного, есть приложить немного упорства.

В 54-ой части мы начнём новую тему.